WEBSITE BOOK EDITION

Pixels → Perception

從像素到理解：視覺、語言與學習

基礎・影像・語言・學習・理論

繁體中文版

2026-10-17

https://blog.shyandram.dev/

本書說明

本書收錄 Pixels → Perception 網站的 75 篇已發布教學與 2 篇部落格文章，依八個學習單元編排，最後附上部落格文章。這是網站內容的書籍排版版本，並非另經同儕審查的出版品。

正文、程式碼、公式與參考文獻沿用網站內容。練習提示在本書中直接展開；靜態影像保留，互動示範則提供網站連結。原本的影像比較器改列為輸入與輸出影像。篇末參考文獻的編號僅適用於該篇教學。

本站內容仍在審閱中。製作過程使用 AI 協助，內容可能仍有錯誤或疏漏；請搭配原始文獻閱讀，並以網站的後續修訂為準。

英文單字與漢字的計數方式不同，兩種語言分別提供 PDF。網站內的教學連結盡可能連至本書相應篇章；特定小節、互動操作、筆記本與外部資料仍連至網站。內容快照: 2026-10-17

來源版本: `05b94ec`

https://blog.shyandram.dev/

目錄

I　基礎 1

從這裡開始・學習路徑與推薦教科書 2

影像就是陣列 12

II　影像處理 24

IP 01・緒論 25

IP 02・數位影像基礎 42

IP 03・強度轉換與空間濾波 62

IP 04・頻域濾波 85

IP 05・影像復原與重建 110

IP 06・色彩影像處理 135

IP 07・小波與其他影像轉換 158

IP 08・影像壓縮與浮水印 179

IP 09・形態學影像處理 202

IP 10・影像分割（一）：邊緣偵測、閾值化與區域偵測 225

IP 11・影像分割（二）：主動輪廓——蛇模型與水平集 246

IP 12・特徵擷取 265

IP 13・影像圖樣分類 288

III 自然語言處理 308

NLP 01・自然語言處理導論 309

NLP 02・語言學基礎 321

NLP 03・自然語言處理的類神經網路 344

NLP 04・詞彙語意與詞嵌入 367

NLP 05・用於語言的循環神經網路 392

NLP 06・以 RNN 做序列標註（詞性標註、資訊擷取） 414

NLP 07・注意力機制與 Transformer 439

NLP 08・以 Transformer 建構序列到序列模型（機器翻譯、聊天機器人） 455

NLP 09・以 Transformer 做語言模型（自然語言生成） 477

NLP 10・預訓練模型：BERT 510

NLP 11・以 BERT 做分類（情感分析、自然語言推論） 527

NLP 12・句法與語意任務 547

IV 圖形識別 572

PR 01・模式、機率與貝氏決策 573

PR 02・密度估計與貝氏學習 589

PR 03・迴歸與分類的線性模型 605

PR 04・近鄰、核與非線性分類器 622

PR 05・特徵選擇、轉換與降維 640

PR 06・模板比對與結構化辨識 658

PR 07・分群、混合模型與 EM 675

PR 08・近似推論與貝氏模型複雜度 698

PR 09・集成、驗證與分群有效性 719

PR 10・辨識專題：拒絕、漂移與學習式特徵 740

V 機器學習 762

ML 01・機器學習入門 763

ML 01A・學習所需的線性代數 789

ML 01B・機率與資訊理論 809

ML 01C・數值計算與梯度檢查 831

ML 02・類神經網路：從感知器到深度網路 853

ML 03・卷積、影像濾波與 CNN 876

ML 03A ·循環與遞迴神經網路 898

ML 04・注意力與視覺 Transformer 919

ML 05・表示學習與自監督 938

ML 05A・線性因子模型與自編碼器 959

ML 06・生成模型：自編碼器、VAE、GAN 與正規化流 980

ML 07・擴散模型與流匹配 1001

ML 08・視覺自迴歸：像素、Tokens 與尺度 1031

ML 09・基礎模型與視覺語言學習 1051

ML 10・評估、效率與部署 1071

VI 電腦視覺深度學習 1096

DLCV 01・深度學習的影像處理與電腦視覺 1097

DLCV 02・深度影像分類 1120

DLCV 03・物件偵測：從 R-CNN 到 DETR 1157

DLCV 04・深度影像復原：低階視覺 1183

DLCV 05・低光影像增強 1226

DLCV 06・物件追蹤：從單一物件到多物件 1259

DLCV 07・世界模型：影片、潛在動態與規劃 1286

VII 最佳化 1305

OPT 01・決策建模與凸集合 1306

OPT 02・凸函數、組合規則與共軛 1318

OPT 03・線性、二次、錐與幾何規劃 1330

OPT 04 ·對偶性、KKT 條件與證明憑證 1342

OPT 05 ·近似、統計估計與實驗設計 1355

OPT 06 ·梯度下降、牛頓步驟與數值可靠性 1368

OPT 07 ·等式限制與內點法 1381

OPT 08 ·幾何應用、分裂法與現代方法 1394

VIII 資訊理論 1406

IT 01・熵、相對熵與互資訊 1407

IT 02 ·典型序列、AEP 與熵率 1421

IT 03 · 無損編碼與通用壓縮 1433

IT 04 · 通道容量與可靠通訊 1447

IT 05 · 微分熵與高斯通道 1460

IT 06 · 率失真與有損表示 1472

IT 07 · 統計證據、最大熵與成長 1485

IT 08 · 網路、資訊不等式與學習 1498

IX 部落格文章 1510

介紹 Pixels → Perception 1511

歡迎來到 Pixels → Perception 1513

篇 I

基礎

從這裡開始・學習路徑與推薦教科書

八個單元如何彼此銜接、依背景與目標該走哪條路線、如何執行程式碼與筆記本，以及每個單元適合搭配的教科書。

網站版本: https://blog.shyandram.dev/zh-tw/learn/foundations/getting-started/

你將學到

- 本站八個單元彼此的關係，以及各自的用途。
- 如何閱讀一篇教學頁面：難度、先備知識、單元側欄、筆記本與互動實驗。
- 依背景（剛學程式、數學生疏、已會訓練網路）與目標（視覺、理論、生成模型、語言、世界模型）選擇學習路徑。
- 如何建立一個小型 Python 環境，在 CPU 上執行程式碼區塊與筆記本。
- 每個單元適合搭配哪本主教材，以及教科書、綜述與論文各該怎麼讀。
- 如何把閱讀轉成可以檢驗的小里程碑。

先看全貌

你不必先學完所有數學才有資格碰影像，但你需要知道每個運算背後的假設。最快的學法，是從一個可以測試的問題開始：「為什麼模糊會削弱邊緣？」「分類器換到另一台相機的影像還有效嗎？」「模型改成預測下一個 token 而不是類別，會有什麼不同？」

> **用最簡單的話說**
>
> 學這個領域就像學做菜。教科書介紹食材與技巧，小實驗讓你嚐到實際的味道。先挑一道菜，一次只改一種食材，把觀察記下來，再去挑戰整桌宴席。本站就是一本把食譜依合理順序排好的食譜書，而且每道菜都能在小廚房（一台筆電的 CPU）裡做出來。

本站分成**八個單元**。單元是一系列依序排列、圍繞同一主題的教學。編號說明的是各單元的角色，不代表一定要讀完一個才能進入下一個。

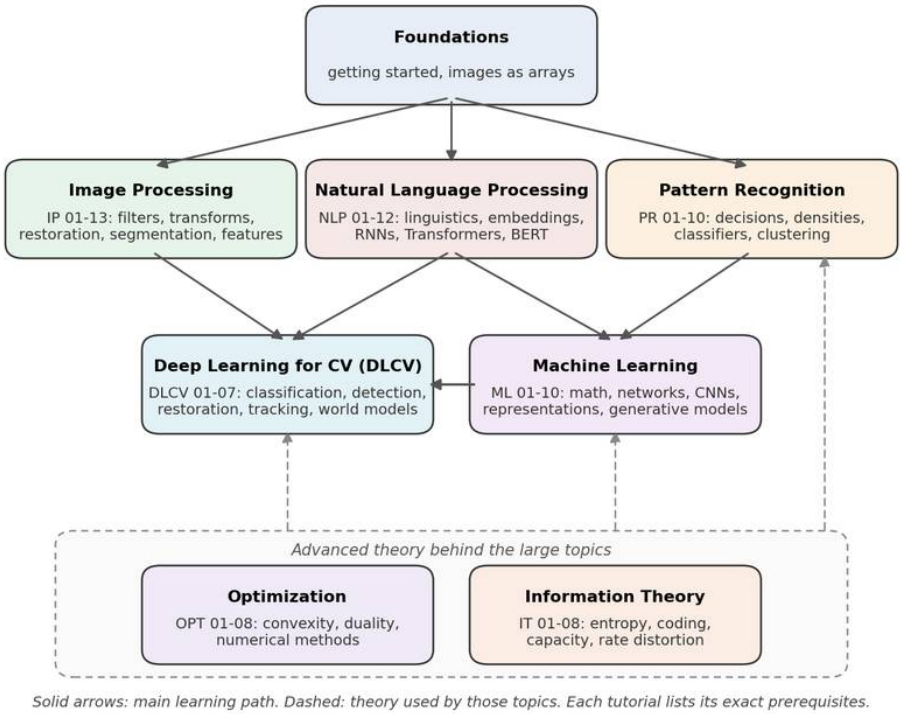


圖 *1* — 八個單元如何銜接。先讀基礎，再選一個入門單元（*IP*、*NLP* 或 *PR*）；機器學習與電腦視覺深度學習（*DLCV*）建立在它們之上；最佳化與資訊理論則是這些大主題背後的進階理論。箭頭只是建議，不是限制：每篇教學都會列出自己的先備知識。

單元	核心問題	內容	適合的第一個成果
基礎・起點	資料是什麼？	本頁與影像就是陣列	檢查影像的形狀、dtype 與座標慣例
影像處理（IP）・入門	如何變換或量測影像？	IP 01–13，從基礎到圖樣分類	解釋一個濾波器及其邊界行為
自然語言處理（NLP）・入門	文字與語言如何建模？	NLP 01–12，從導論與語言學基礎，經過詞嵌入、RNN、Transformer 與BERT，到句法與語意任務	建立並評估一個小型語言應用
圖形識別（PR）・入門	如何用特徵在不確定性下做決策？	PR 01–10，從貝氏決策到辨識專題	用正確的訓練／測試切分比較分類器
機器學習（ML）・核心	表示、預測器與生成器如何被學出來？	ML 01–10，加上數學章節 ML 01A–01C 以及 ML 03A、05A	訓練並評估一個小模型
電腦視覺深度學習（DLCV）・核心	學習式模型如何解決視覺任務？	DLCV 01–07：分類、偵測、復原、低光增強、追蹤、世界模型	從頭到尾理解一條分類或偵測流程
最佳化（OPT）・理論	如何建立並求解帶限制的決策問題？	OPT 01–08，從凸集合到求解器與應用	建立凸最佳化問題並檢查最適條件
資訊理論（IT）・理論	表示與傳遞不確定性的極限是什麼？	IT 01–08，從熵到編碼、通道、率失真與網路	計算熵，說明壓縮或通訊的極限

用一段話說明它們的關係：**影像處理**把影像當成訊號，以人工設計運算；**圖形識別**加入統計與決策理論：有了特徵之後該怎麼決定、又能有多確定？**機器學習／深度學習**用學出來的特徵取代人工特徵，並提供各處共用的機制（損失函數、最佳化、架構、生成模型）；**視覺深度學習**把這套機制用在具體的視覺任務上，其中復原與增強章節大量借用影像處理的成像概念；**自然語言處理**則以文字為輸入，重複使用相同的機率、最佳化與 Transformer 概念。**最佳化**進一步發展估計與訓練背後的建模規則、限制、對偶性與數值方法。**資訊理論**解釋不確定性、編碼與通訊極限；熵與散度也串起分類、語言建模和壓縮。

教學頁面怎麼看

每篇教學都採用相同版面，看懂一頁就看懂全部。

- **難度標籤**。分為入門、中階、進階。ML 數學章節與前幾篇 IP 屬入門；DLCV 各篇則假設你已懂 CNN 與反向傳播。
- **先備知識**。標題下方的「先備知識」列出本頁預設你已讀過的教學。先備知識與語言無關，中英文頁面相同。遇到不熟的連結，先讀那一頁，至少讀它的「重點整理」。
- **單元側欄**。左欄依序列出目前單元的所有教學；頁尾的「上一篇／下一篇」也照同樣順序。
- **段落結構**。每篇以「你將學到」和「用最簡單的話說」開場，每個主題講兩遍：先用白話說，再給出含公式與程式碼的精確版本。結尾是「重點整理」、「練習」（提示預設收合，先自己試再打開）與編號的「參考文獻」，內文以 [1]、[2] 引用。
- **筆記本按鈕**。有配套筆記本的教學，標題旁會出現「在 Colab 開啟」，頁尾則有「查看筆記本」連結。目前ML 03 有一本（卷積與影像濾波）。
- **互動實驗**。許多頁面內嵌即時實驗，在瀏覽器中對真實影像或資料集執行真正的演算法：濾波器、傅立葉轉換、門檻、最近鄰等等。每個實驗都先請你預測結果，再讓你親自驗證。所有實驗也都收錄在互動實驗頁面。
- **搜尋與語言**。搜尋頁會在你目前閱讀的語言中尋找教學與部落格文章；頁首的語言切換可跳到另一種語言的同一頁。若某頁尚未翻譯，中文網站會顯示英文版並附上提示。

選擇你的入口

挑一個最符合你的描述。各路線刻意有所重疊。

如果程式設計也是新主題。先熟悉 Python 的函式、迴圈、串列與陣列索引，以及基本繪圖。接著讀影像就是陣列與IP 01。建立一張合成影像，檢查它的列與欄，並說明兩張 `uint8` 影像相加為什麼會溢位。之後讀IP 02 與IP 03，這兩章的每條公式都有對應的 NumPy 程式。

如果會寫程式，但數學有點生疏。先讀ML 01 熟悉機器學習的詞彙，再讀三篇數學章節：線性代數、機率與資訊理論與數值計算。它們依循 Goodfellow、Bengio、Courville [7] 的「應用數學」部分，並為每個概念搭配簡短的 NumPy 驗證。目標是能檢查矩陣形狀、解讀條件機率，並認出數值不穩定的計算。

如果已學過入門機率與線性代數。想學統計決策與傳統方法，從PR 01 開始；想學習式表示，從ML 02 開始。推導遇到明確缺口時，再回頭補數學章節。

如果已經會訓練神經網路。快速瀏覽ML 03 與ML 04，再從視覺深度學習選一篇應用章節，並搭配ML 10 的評估與部署一起讀。

依目標建議的學習路徑

有了入口之後，再選一個目標。每條路徑是一個閱讀順序，通常八到十二篇。讀每頁時留意「先備知識」，有缺口就隨時補上。

目標	建議順序
傳統影像處理	影像就是陣列 → IP 01–05（基礎、濾波、頻率域、復原）→ IP 06–09（色彩、小波、壓縮、形態學）→ IP 10–13（分割、主動輪廓、特徵、分類）
統計圖形識別	ML 01A、ML 01B → PR 01 → PR 02（密度估計）→ PR 03（線性模型）→ PR 04（核方法）→ PR 05（特徵）→ PR 07（分群、EM）→ PR 08（近似推論）→ PR 09（集成、驗證）→ PR 10
視覺深度學習	ML 01 → ML 01C → ML 02 → ML 03 → ML 04 → DLCV 01 → DLCV 02 → DLCV 03 → DLCV 04–06（復原、低光、追蹤）→ ML 10
生成模型	ML 01B → ML 02 → ML 05 → ML 05A → ML 06 → ML 07 → ML 08 → ML 09
語言	NLP 01 → NLP 02 → NLP 03–04（類神經網路、詞嵌入）→ NLP 05–06（RNN、序列標註）→ NLP 07–09（注意力與 Transformer、seq2seq、語言模型）→ NLP 10–11（BERT、分類）→ NLP 12；網路的數學可搭配 ML 02、ML 03A 與 ML 04
凸建模與求解器	ML 01A、ML 01C → OPT 01 → OPT 02–04（函數、問題、對偶性）→ OPT 05–07（估計、數值方法、限制式求解器）→ OPT 08
資訊、壓縮與通訊	ML 01B → IT 01 → IT 02–03（典型性、來源編碼）→ IT 04–06（容量、高斯通道、率失真）→ IT 07–08（統計決策、網路）
世界模型	ML 03A → ML 05 → ML 07 → ML 08 → DLCV 07

若每週讀一篇、執行程式碼並做兩題練習，這是可以持續的步調；較長的教學可能需要兩週。速度不如能用自己的話說明每頁的重點整理重要。

執行程式碼

網站與 Python 範例使用不同的環境。只讀網站什麼都不用安裝；要執行程式碼需要 Python；要在本機預覽網站本身才需要 Node。

為教學建立 Python 環境

為所有教學建立一個專用的虛擬環境。Python 3.11 以上是實用的起點；若使用其他版本，請核對各套件的需求。

```
python3 -m venv .venv-study
source .venv-study/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy scipy matplotlib scikit-learn scikit-image opencv-python
 ↪  PyWavelets jupyterlab
```

Windows PowerShell 的啟用方式是 `.venv-study\Scripts\Activate.ps1`。全站程式碼區塊匯入的正是這些函式庫（NumPy、SciPy、scikit-learn、scikit-image、OpenCV、PyWavelets），神經網路章節另外需要 PyTorch。請使用 PyTorch 官方安裝選擇器 [11] 產生的安裝指令；沒有 GPU 就選 CPU 版本。本站每個範例都設計成能在筆電 CPU 上於數秒到約一分鐘內跑完。

執行一個程式碼區塊

程式碼區塊都是**獨立完整**的：每一段都自行匯入所需套件並建立自己的資料，通常是合成陣列、`skimage.data` 內建的範例影像，或 scikit-learn 的玩具資料集。把區塊複製到檔案中執行，或貼到筆記本儲存格裡即可。多數區塊以 `assert` 結尾，用來檢查內文的說法；若 assert 失敗，代表你遇到了版本差異，或是值得深究的誤解。

好習慣是一次只改一件事再重跑：kernel 大小、學習率、亂數種子。練習題會提示該調哪個參數。

筆記本與 Colab

有配套筆記本的教學會出現「在 Colab 開啟」按鈕。Colab 在瀏覽器中、以雲端主機執行筆記本，已預先安裝 NumPy、OpenCV、scikit-image 與 Matplotlib，本機不必做任何設定。若要在本機執行同一本筆記本，從「查看筆記本」連結下載，於已啟用的環境中執行 `jupyter lab` 並開啟檔案。筆記本放在 repository 的 `notebooks/` 資料夾。

預覽網站（選用）

只有想修改網站時才需要。本 repository 需要 Node 22.12 以上 [12]：

```
npm install
npm run dev
```

開發伺服器會印出本機網址。教學是 `src/content/tutorials/<lang>/<track>/` 底下的 MDX 檔案。

第一個 CPU 實驗

以下完整範例自行建立影像、加入雜訊再平滑，不需要下載任何東西。

```
import numpy as np
import cv2

rng = np.random.default_rng(0)
clean = np.zeros((32, 32), dtype=np.float32)
clean[8:24, 8:24] = 1.0
noisy = clean + rng.normal(0, 0.1, clean.shape).astype(np.float32)
smooth = cv2.GaussianBlur(noisy, (5, 5), 1.0,
                          borderType=cv2.BORDER_REFLECT_101)
print("Shape and type:", smooth.shape, smooth.dtype)
print("Noise-region standard deviation:",
      noisy[:5, :5].std(), smooth[:5, :5].std())
print("Across the edge:", np.round(smooth[16, 5:12], 3))
assert smooth.shape == clean.shape
assert np.isfinite(smooth).all()
```

輸出顯示兩個效果。平坦、全為背景的角落，標準差從約 0.074 降到約 0.026：平滑把雜訊平均掉了。在第 8 欄的邊緣處，數值不再從 0 直接跳到 1，而是在三、四個像素內從約 0.04 爬升到 0.93：平滑也把銳利的轉折擴散開了。因此雜訊估計變低，不等於所有重要細節都變好。IP 03 會解釋原因，練習 2 則請你實際量測這個取捨。

選一本主教材

用一本主教材維持連貫的進度，再用另一本提供同一概念的不同解釋。下列連結指向作者或出版社頁面，取得方式依各網站說明。這是精選的起點，不是排名。

需求	推薦教科書	建議用法
數學準備	Deisenroth、Faisal、Ong，*Mathematics for Machine Learning* [1]	從線性代數、矩陣分解、向量微積分與機率讀起，再透過迴歸與 PCA 應用
影像操作	Gonzalez、Woods，*Digital Image Processing*，第 4 版 [2]	把影像基礎、空間與頻率域濾波、復原與分割，搭配影像處理單元閱讀
圖形識別主教材	Theodoridis、Koutroumbas，*Pattern Recognition*，第 4 版 [3]	依序學分類器設計、特徵與分群；實際動手解題，而不只讀摘要
圖形識別配套教材	Bishop，*Pattern Recognition and Machine Learning* [4]	從機率、潛在變數與圖模型的角度重新理解同一主題
第一本現代深度學習書	Prince，*Understanding Deep Learning* [5]	搭配圖解、數學與隨書筆記本，理解網路與生成建模
另一種現代深度學習解釋	C. M. Bishop、H. Bishop，*Deep Learning: Foundations and Concepts* [6]	用它的機率觀點加深對架構與訓練原理的理解
深度學習基礎與參考	Goodfellow、Bengio、Courville，*Deep Learning* [7]	第一部分（應用數學）對應 ML 01A–01C；後續章節涵蓋最佳化、CNN、序列模型與表示學習
分類之外的電腦視覺	Szeliski，*Computer Vision: Algorithms and Applications*，第 2 版 [8]	串起成像、運動、幾何、辨識與重建
機率式機器學習	Murphy，*Probabilistic Machine Learning: An Introduction* [9]	以機率為主軸的現代參考書，適合機器學習與圖形識別單元

需求	推薦教科書	建議用法
自然語言處理應用主教材	Hagiwara，*Real-World Natural Language Processing* [13]	依序學習應用流程、文字分類、序列模型、注意力、預訓練模型、訓練實務與部署
自然語言處理語言學配套教材	McRoy，*Principles of Natural Language Processing* [14]	搭配對應章節學習句法、文法、語意、篇章、檢索、生成與翻譯
文字與語言補充教材	Jurafsky、Martin，*Speech and Language Processing*，第 3 版草稿 [10]	深入 token、n-gram、分類、嵌入（embedding）、Transformer 與評估等主題
凸最佳化	Boyd、Vandenberghe，*Convex Optimization* [15]	OPT 01–05 依第 1–8 章學建模與應用；OPT 06–08 深入第 9–11 章的數值方法與現代延伸
資訊理論	Cover、Thomas，*Elements of Information Theory*，第 2 版 [16]	從熵與典型性讀到來源／通道編碼、連續變數、率失真、統計決策、通用編碼與網路

圖形識別單元以 **Theodoridis 與 Koutroumbas [3] 及 Bishop 的 PRML [4] 為兩本核心教材**。先用 Theodoridis 建立完整的辨識流程，再比較 Bishop 對同一問題的機率處理方式。出版年份較早，不會讓貝氏決策理論或高斯模型失效；架構、資料規模或應用改變的部分，再用新論文補充。

若只選一本深度學習入門書，從 Prince [5] 開始，需要時搭配本站數學章節。偏好一路以機率觀點推導，就選 Bishop 與 Bishop [6]。Goodfellow 等人 [7] 仍是重要基礎，但 2016 年版早於視覺 Transformer、擴散模型與基礎模型；這些內容由機器學習／深度學習單元補上。

自然語言處理單元以 **Hagiwara [13] 與 McRoy [14] 為兩本核心教材**。Hagiwara 串起模型與應用，McRoy 補足理解語言系統所需的語言學與符號式概念。可以圍繞同一問題並讀兩書：分類器需要資料流程，剖析器或對話系統也需要明確的語言結構。兩書的 2021 年版再搭配新論文補充。

最佳化單元以 **Boyd 與 Vandenberghe [15]** 學習哪些問題變換保留凸性，以及最適性憑證的意義。資訊理論單元以 **Cover 與 Thomas [16]** 分辨可達成的構造與任何構造都無法突破的界限。兩個單元延伸 ML 01B–01C 的數學入門；每篇教學都有對應的教科書章節地圖。

Jurafsky 與 Martin 的草稿會持續更新，目前版本日期為 2026 年 8 月 19 日 [10]。做章節筆記時請記下版本日期，因為章號可能隨版本變動。其他教科書也要記下版次，讓練習題的出處能被重新找到。

教科書、綜述與論文怎麼讀

教科書提供定義與連貫的順序。每讀一段，至少重推一條公式、跑一個小例子、解一道練習，再往下讀。

綜述（survey ／ review）提供方法地圖、假設與未解問題。每篇教學的「現代觀點」都會摘要幾篇。閱讀時可整理成表格：它涵蓋的每個方法的預測目標、學習目標、所需資料、計算成本與評估方式。也要檢查涵蓋日期：「近期」是相對於作者當時而言。

原始論文提出一個方法及其證據。讀問題、方法、實驗規則與限制，把重要主張追到實際支持它的表格或消融實驗。知道它相對公平基準改變了什麼，比記住方法名稱更有用。

每篇閱讀筆記保留五項：問題、數學物件、主要假設、檢驗該假設的實驗，以及一個尚未解決的疑問。每篇教學的參考文獻是下一步閱讀的方向；不必讀完每篇引用的論文才開始跑程式。

用小里程碑安排進度

1. **表示資料**：檢查一張影像，說明各軸、數值範圍與 dtype。
2. **變換資料**：實作一個濾波器，與函式庫結果比對，並處理邊界。
3. **做出決策**：訓練一個簡單分類器，前處理參數只從訓練資料學。
4. **學習特徵**：訓練一個小型神經網路，說明損失函數與驗證方式的選擇理由。
5. **研究現代方法**：在合成資料上重現注意力、遮罩或加雜訊等機制。
6. **完成小專題**：報告基準方法、獨立測試結果、失敗案例與限制。

能不照抄原文、用自己的話說明上一個里程碑時，再往下走。一個比較乾淨的小實驗，比一個看不清資料與評估方式的大模型更能建立理解。

重點整理

- 八個單元：基礎、影像處理、自然語言處理、圖形識別、機器學習／深度學習、視覺深度學習、最佳化、資訊理論。它們互相支援，不必讀完一個才開始另一個。
- 每頁的「先備知識」、難度標籤與單元側欄告訴你它的位置；用它們取代從頭讀到尾。
- 依背景選入口、依目標選路徑；每週一篇並執行程式碼與練習，是可持續的步調。
- 所有程式碼都獨立完整、以 CPU 規模設計；一個裝有 NumPy、SciPy、scikit-learn、scikit-image、OpenCV、PyWavelets 與 PyTorch 的 Python 環境就涵蓋全站。
- 每個單元搭配一本主教材與一本配套教材，並以不同目的閱讀教科書、綜述與論文。

練習

1. **規劃**。選一個入口、一條學習路徑與一本主教材。寫下第一個實驗要回答的問題，以及什麼具體結果算是進步。

   提示

   問題要小到能在 CPU 上一次做完，例如「5×5 高斯模糊在平坦區域降低多少雜訊？又讓邊緣變寬多少？」進步應該是一個可以量測的數字，而不是一種感覺。

2. **掃描 kernel 大小**。把平滑範例改成 kernel 大小 3、5、9（σ 跟著加大，例如 0.8、1.0、2.0）。回報平坦區域的標準差，以及邊緣從 0.1 升到 0.9 所需的像素數。

   提示

   固定亂數種子與含雜訊的影像，只改濾波器。雜訊下降與邊緣變寬應該同時發生：這就是線性平滑的偏差與變異取捨。

3. **讀懂頁面資訊**。打開ML 02。它的難度是什麼？「先備知識」指向哪一頁？單元側欄中下一篇是哪一篇？

   提示

   「先備知識」就在標題下方。沿著它一路往回追，直到遇到你已具備先備知識的頁面；這條鏈就是你個人的閱讀順序。

4. **為來源分類**。從任一篇教學挑一筆參考文獻，判斷它是教科書、綜述、原始論文還是軟體文件。這種來源最適合回答什麼問題？

   提示

   綜述回答「有哪些選擇、彼此如何比較？」；原始論文回答「這個特定改動是否有幫助、在什麼實驗規則下？」；軟體文件回答「這個函式實際上算了什麼？」

5. **用正確的單位切分**。有一個資料集包含 50 段影片、每段有許多影格。請規劃訓練／驗證／測試切分，並說明為什麼隨機切分單一影格可能造成誤導。

   提示

   相鄰影格幾乎相同，隨機切影格會讓近乎重複的樣本同時出現在訓練集與測試集，使測試分數虛高。應改以影片（或錄製時段）為單位切分。

參考文獻

1. M. P. Deisenroth, A. A. Faisal and C. S. Ong, *Mathematics for Machine Learning*, Cambridge University Press, 2020. Book site
2. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018. Book site
3. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2009. Companion site
4. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006（官方提供 PDF）. Book page
5. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. Book and notebooks
6. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024（提供免費線上版）. Book site
7. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016. Book site
8. R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed., Springer, 2022（PDF 限個人使用）. Book site
9. K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022. Book site
10. D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. draft, release of August 19, 2026. Draft site
11. PyTorch, “Get Started Locally,” installation documentation. Documentation
12. Node.js, “Download Node.js.” Download page

13. M. Hagiwara, *Real-World Natural Language Processing*, Manning, 2021. Publisher
14. S. McRoy, *Principles of Natural Language Processing*, open textbook, 2021. Open textbook
15. S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. Author site and book
16. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006. Publisher

連結與草稿日期於 2026 年 10 月 4 日查核。配套網站不代表整本教科書可免費取得；取得方式以各網站說明為準。

影像就是陣列

數位影像到底是什麼？NumPy 與 OpenCV 如何儲存它？以及能避開大多數常見錯誤的索引、資料型別與通道順序習慣。

網站版本: https://blog.shyandram.dev/zh-tw/learn/foundations/images-as-arrays/

你將學到

- 灰階與彩色影像如何變成 NumPy 陣列，以及 `shape`、`dtype` 與 `strides` 告訴你什麼。
- 為什麼影像是連續場景經過取樣與量化後的結果，以及保留較少像素或較少灰階時會失去什麼。
- 座標慣例（先列、後行），以及切片、翻轉、遮罩與攤平如何運作，包括視圖（view）與複本（copy）的差別。
- 每個人都會踩到的三個坑：BGR 與 RGB 的通道順序、`uint8` 回繞（wraparound），以及不小心透過視圖改寫原始資料。
- 同樣的陣列觀點如何延伸到深度學習的張量、影像區塊（patch）與神經網路影像表示。

先看全貌

本站所有方法，從 3×3 模糊到視覺 Transformer，起點都是同一個東西：一張數字網格。只要你能有把握地讀取、切片與轉換這張網格，其餘課程就只是選擇要組合哪些數字、如何組合。

> **用最簡單的話說**
>
> 想像一幅由許多小彩色磁磚拼成的馬賽克。站遠一點，你看到一張臉；走近一點，你只看到一塊塊方格，每格只有一種顏色。數位照片也是如此：一個長方形的磁磚陣列，每一塊磁磚用一個數字（或三個數字，分別代表紅、綠、藍）描述。要找到某一塊，你說「第幾列、第幾行」，就像在電影院找座位。電腦把這幅馬賽克存成一張數字表，而你對圖片做的每件事，例如調亮、模糊或找出人臉，都是對這張表做運算。

為什麼重要：初學者的影像程式錯誤，幾乎都是陣列的問題，而不是演算法的問題。顏色偏藍，是因為通道被對調了；明亮區域變黑，是因為 8 位元數值回繞了；修改裁切區域卻改到原圖，是因為裁切得到的是視圖而不是複本。本頁用可執行的程式把這些陷阱一次講清楚，讓你之後一眼就能認出來。

影像是取樣並量化後的訊號

白話版。真實世界沒有像素。相機測量每個感測器小區域收到多少光，再把測量結果寫成一個整數。兩個決定把連續場景變成數位影像：在哪裡測量（有幾個點），以及每個測量值寫得多細（有幾種可能的值）。

精確版。令單色場景是一個連續的亮度函數。**取樣**（sampling）在 H 列、W 行的規則網格上對它求值；**量化**（quantization）則把每個樣本四捨五入到 $L = 2^b$ 個整數階層之一，其中 b 是每個像素的位元數。結果是定義在有限網格上的函數：

$$I : \{0, \dots, H-1\} \times \{0, \dots, W-1\} \to \{0, \dots, L-1\}, \qquad (r, c) \mapsto I[r, c],$$

其中 r 是**列**（row）索引（從上往下數），c 是**行**（column）索引（從左往右數），$I[r, c]$ 是**像素值**，常見的 8 位元情況下 $L = 256$，數值從 0（黑）到 255（白）。彩色影像多一個通道索引 k，寫成 $I[r, c, k]$，三個色彩通道的 $k \in \{0, 1, 2\}$。Gonzalez 與 Woods [1, 第 2 章] 以及 Torralba、Isola 與 Freeman [2] 等教科書都以這種方式處理影像形成與數位化。

寫成陣列時，網格的**形狀**（shape）在灰階時是 `(H, W)`，彩色時是 `(H, W, 3)`。儲存的數值個數是 $H \times W \times C$（C 個通道），所需記憶體則是這個數量乘以每個數值的位元組數。

> **座標：先列後行**
>
> 陣列以 `[列, 行]` 索引，也就是畫面自己座標軸上的 `[y, x]`：第一個索引沿頁面往下，第二個索引沿頁面橫向。繪圖函式常把點寫成 `(x, y)`，也就是 `(行, 列)`。本系列的影像處理章節沿用教科書慣例，把 $f(x, y)$ 寫成 x 是列索引，所以 $f(x, y)$ 對應 NumPy 的 `f[x, y]`。每當程式與數學對不上，先確認哪個軸是哪個，再去除錯其他地方。

形狀、dtype 與記憶體

白話版。陣列知道自己的三件事：每個軸上有多少個數字（`shape`）、每個數字是哪一種（`dtype`），以及這些數字在記憶體中如何排列（`strides`）。

scikit-image 附帶幾張小型測試照片 [6]、[7]，因此本頁所有例子都不必下載任何東西就能執行。下面的程式載入一張灰階與一張彩色影像，並印出它們的基本性質。

```
import numpy as np
import Lab from '../../../../components/Lab.astro';
from skimage import data

gray = data.camera()          # scikit-image 內附的灰階測試照片
rgb = data.astronaut()        # 同樣內附的彩色測試照片

for name, a in [("gray", gray), ("rgb", rgb)]:
    print(name, a.shape, a.dtype, a.ndim, a.size, a.nbytes)
print(gray.min(), gray.max())
print(gray.strides, rgb.strides)
```

輸出：

```
gray (512, 512) uint8 2 262144 262144
rgb (512, 512, 3) uint8 3 786432 786432
0 255
(512, 1) (1536, 3, 1)
```

解讀輸出：灰階影像有 512 列、512 行，所以 `size` 是 512 × 512 = 262,144 個數值；每個 `uint8` 佔一個位元組，所以 `nbytes` 是同一個數字。彩色影像的數值是它的三倍。像素範圍是完整的 0 到 255。

`strides` 是沿各軸前進一步時要跳過的位元組數。對 `gray` 而言，往右移一行花 1 個位元組，往下移一列花 512 個位元組，表示 NumPy 是**一列接著一列**儲存影像（C 順序）。在 `rgb` 中，同一個像素的三個通道值在記憶體中相鄰。這解釋了為什麼沿著列掃描很快，也解釋了為什麼把影像重塑成長向量時各列會維持在一起，下一節會看到。NumPy 論文 [3] 與文件 [4] 完整描述了陣列模型。

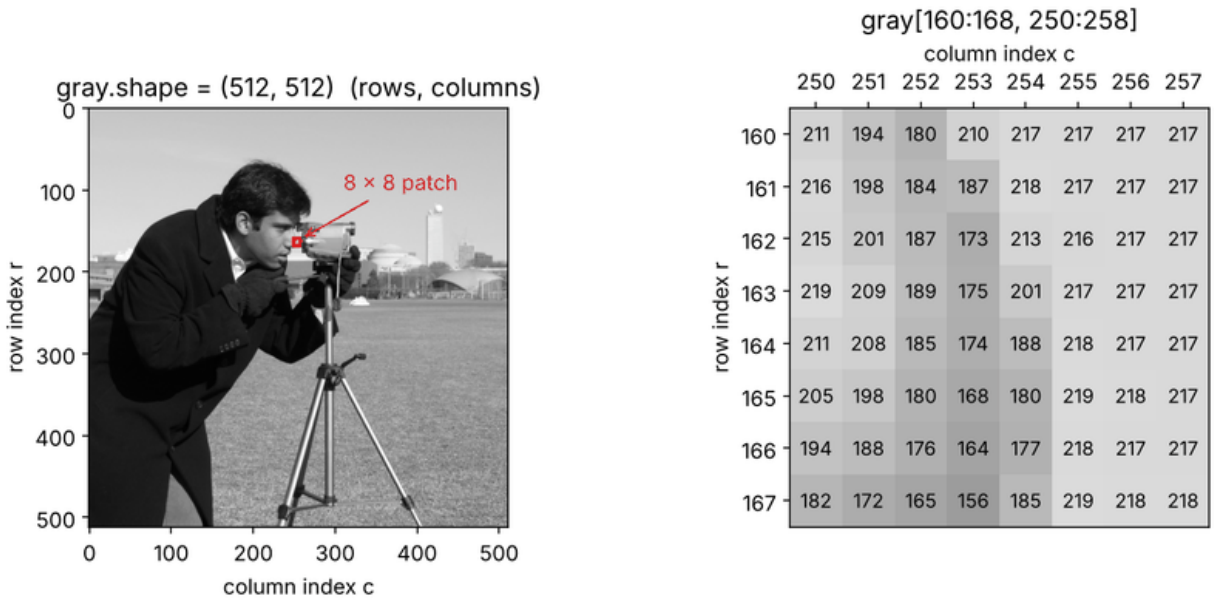


圖 *1* — 影像是一張整數表。左：*512 × 512* 測試照片，框出一個 *8 × 8* 區塊。右：同一個區塊，呈現為電腦實際儲存的數字，並標示列與行索引。*Matplotlib* 預設把第 *0* 列放在最上方 [*9*]。

索引、切片與視圖

白話版。因為影像就是陣列，你可以用與任何 NumPy 陣列相同的中括號語法取出單一像素、矩形區域或整列。令人意外的是，這樣取出的矩形是同一塊記憶體上的一扇窗，而不是獨立的複本。

精確版。對陣列 `a`，`a[r, c]` 回傳單一像素，`a[r0:r1, c0:c1]` 回傳第 $r_0, \dots, r_1 - 1$ 列與第 $c_0, \dots, c_1 - 1$ 行組成的子區塊（結尾索引不含），而 `a[::2, ::2]` 這類寫法則每隔一列、一行取一個。這些全都是**視圖**：與原陣列共用記憶體的新陣列物件，改動視圖就會改動原陣列。布林遮罩與索引串列則會建立複本。呼叫 `.copy()` 可以明確建立複本。

```
r, c = 100, 200
print(gray[r, c])            # 先列、後行
print(rgb[r, c])             # 三個數字：R、G、B
```

```
patch = gray[100:108, 200:208]            # 第 100..107 列、第 200..207 行
print(patch.shape, np.shares_memory(patch, gray))

work = gray.copy()
work[100:108, 200:208] = 0                # 透過切片寫入，會改到 `work`
print(work[100:108, 200:208].max(), gray[100:108, 200:208].max())

H, W = gray.shape
vec = gray.reshape(-1)                    # 攤平：列優先順序
print(vec.shape, vec[r * W + c] == gray[r, c], np.shares_memory(vec, gray))
print(np.array_equal(gray[::-1], np.flipud(gray)), gray[:, ::-1].shape)
mask = gray > 200
print(mask.dtype, mask.sum())
```

輸出：

```
54
[81 57 17]
(8, 8) True
0 109
(262144,) True True
True (512, 512)
bool 55112
```

每一行告訴我們什麼：

- `gray[100, 200]` 是 54，`rgb[100, 200]` 是 `[81 57 17]`：彩色像素是一個短向量，每個通道一個數字。
- 8×8 區塊與 `gray` 共用記憶體，所以它是視圖。因為 `work` 是 `gray` 的複本，把 `work` 的區塊歸零不會動到 `gray`（它的區塊最大值仍是 109）。如果當初是對 `gray` 本身的視圖寫入，原始照片就會被改掉。
- 攤平後得到 $HW = 262144$ 個數字的向量，像素 (r, c) 位於位置 $rW + c$。這種攤平形式就是傳統分類器與全連接網路第一層看到的輸入向量（見ML 01a 與ML 01）。
- 把影像上下翻轉是 `gray[::-1]`，這是列步長為負的視圖，不需要迴圈。與 `np.flipud` 的比較結果是 `True`。
- 像 `gray > 200` 這樣的比較會得到同形狀的布林陣列，此處有 55,112 個 `True`。布林遮罩是你選取、計數或編輯符合條件像素的方式。

彩色影像與通道順序

白話版。彩色影像每個像素存三個數字，分別代表紅的程度、綠的程度與藍的程度。麻煩在於，兩種常見慣例對這三個數字的排列順序不同，而陣列本身不會告訴你它是哪一種。

精確版。多數 Python 影像程式碼（scikit-image [6] 的 NumPy 陣列、Matplotlib、Pillow）使用紅、綠、藍（**RGB**）順序。OpenCV 的 `imread` 回傳 **BGR**：藍色是通道 0，紅色是通道 2。混用兩者不會報錯，只會悄悄把紅與藍對調，於是膚色變藍、天空變橘。為了不依賴磁碟上的檔案就能看見這個效果，下面的程式把彩色測試影像寫成暫存 PNG，再用 OpenCV 讀回來比較。

```
import pathlib, tempfile
import cv2
from skimage import io

with tempfile.TemporaryDirectory() as d:
    path = pathlib.Path(d) / "astronaut.png"
    io.imsave(path, rgb)                    # 以 R、G、B 順序寫出的 PNG
    bgr = cv2.imread(str(path))             # OpenCV 以 B、G、R 順序回傳通道
print(rgb[100, 200], bgr[100, 200])
print(np.array_equal(bgr[..., ::-1], rgb))
print(np.array_equal(cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB), rgb))

weights = np.array([0.299, 0.587, 0.114])
manual = rgb @ weights                      # R、G、B 的加權和 -> float64 (H, W)
cv_gray = cv2.cvtColor(rgb, cv2.COLOR_RGB2GRAY)
print(cv_gray.dtype, np.abs(manual - cv_gray).max())
from skimage.color import rgb2gray
print(np.abs(rgb2gray(rgb) * 255 - cv_gray).max())
```

輸出：

```
[81 57 17] [17 57 81]
True
True
uint8 0.5010000000000332
15.529299999999992
```

同一個像素以 RGB 讀是 `[81 57 17]`，以 BGR 讀是 `[17 57 81]`。反轉最後一個軸 `bgr[..., ::-1]` 是不花成本的順序翻轉方式，而 `cv2.cvtColor` 搭配 `COLOR_BGR2RGB` 則是明確做同一件事。兩者都能完全還原原圖。

要把彩色轉成灰階，標準做法是對通道做**加權和**，因為人眼對綠色比對藍色敏感。權重 0.299, 0.587, 0.114 來自 ITU-R BT.601 電視標準的亮度（luma）定義 [8, 第 2.5.1 節]，而上面的檢查顯示，OpenCV 的灰階轉換與這個加權和的差距在 0.501 個灰階以內，那只是四捨五入成整數造成的。不同函式庫使用不同的權重：在這張影像上，scikit-image 的 `rgb2gray` 與 OpenCV 的結果最多相差約 15.5 個灰階。所以不要假設「灰階」在不同函式庫中是同一組數字，請對照你所呼叫函式的文件。

圖 2 — 彩色影像是三個平面的堆疊。每個通道本身就是一張灰階影像：橘色太空衣在紅色平面很亮、在藍色平面很暗。最後一個面板是加權灰階轉換。

資料型別與回繞陷阱

白話版。影像最常見的儲存型別是 8 位元無號整數，只能存 0 到 255。如果計算超過 255，它不會停在 255，而是悄悄從 0 重新開始。一個幾乎全白的像素可能變成幾乎全黑。

精確版。對無號整數型別 `uint8`，運算是以 $2^8 = 256$ 為模進行的。所以 $250 + 10 = 260 \equiv 4$，而 $250 - 251 = -1 \equiv 255$。由於數值不會飽和，在 `uint8` 中對影像做調亮或相減並不安全。NumPy 2 也不再允許陣列與放不進該陣列型別的 Python 整數混合運算：它會丟出錯誤，而不是悄悄擴大型別，這是依照 NEP 50 [5]。安全的做法有三個步驟：先轉成較寬的型別（`int16`、`float32`），再計算，最後裁切回有效範圍並轉回 `uint8`。

```
from skimage.util import img_as_float

a = np.array([250], dtype=np.uint8)
print(a + 10, a - 251, a.astype(np.int16) + 10)
try:
    a + 300
except OverflowError as e:
    print("OverflowError:", e)

wrapped = gray + np.uint8(60)
clipped = np.clip(gray.astype(np.int16) + 60, 0, 255).astype(np.uint8)
print((gray >= 196).sum(), (wrapped < gray).sum(), clipped.max())

x = gray.astype(np.float32) / 255.0
print(x.dtype, x.min(), x.max(), np.allclose(x, img_as_float(gray)))
y = np.clip(1.2 * x, 0, 1)
out = np.round(y * 255).astype(np.uint8)
print(out.dtype, out.max())
```

輸出：

```
[4] [255] [260]
OverflowError: Python integer 300 out of bounds for uint8
72087 72087 255
float32 0.0 1.0 True
uint8 255
```

解讀：在 `uint8` 中，`250 + 10` 得到 4，`250 - 251` 得到 255；在 `int16` 中，和則正確地是 260。對每個像素加 60，回繞的恰好是數值至少為 196 的像素（因為 $196 + 60 = 256$）：共有 72,087 個，而這同樣 72,087 個像素算完後反而比原來更暗。裁切版本則讓它們全部停在最大值 255。最後一段是常用的浮點流程：除以 255 以便在 $[0, 1]$ 中運算、計算、裁切、乘回去並四捨五入。`skimage.util.img_as_float` 對 `uint8` 影像做的是同樣的重新縮放。

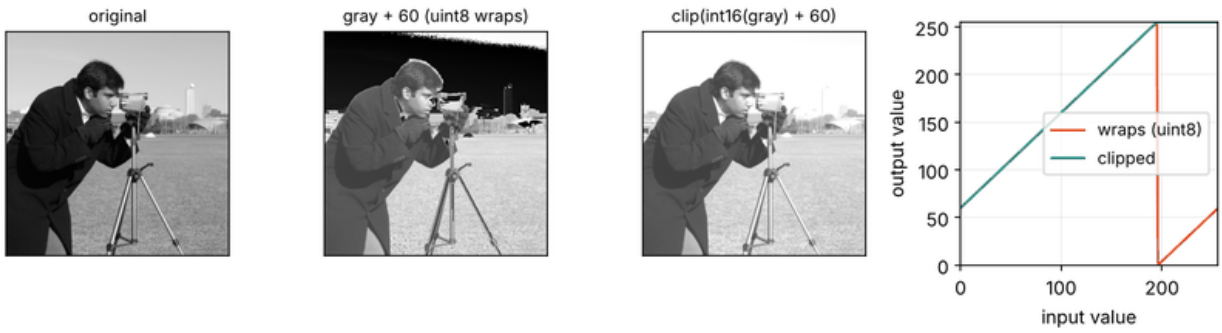


圖 3 — 調亮 *60* 個灰階。在 *uint8* 中，明亮的天空回繞成黑色（橘色曲線）。先轉成較寬型別、相加再裁切，則讓它飽和成白色（綠色曲線）。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

先選資料型別，再做運算

溢位之後才轉型，無法恢復遺失的值。應先決定運算用的型別：濾波與學習用 $[0,1]$ 的 `float32`、需要精確整數差值用 `int16`，只有在存檔或顯示時才轉回 `uint8`。OpenCV 自己的運算函式（例如 `cv2.add`）會飽和而不是回繞，但 NumPy 的 `+` 運算子不會，所以要知道某一行程式用的是哪一個。上方的實驗在照片上執行同樣的運算，你可以親自試試每一種選擇。

在真實影像上做取樣與量化

白話版。兩個獨立的旋鈕控制場景能保留多少：像素的數量（空間解析度）與灰階的階數（亮度解析度）。降低前者，畫面變成方塊狀；降低後者，平滑的明暗過渡會裂成一塊塊平坦的色帶。

精確版。每隔 s 列、s 行取一個，也就是 `gray[::s, ::s]`，會把 $H \times W$ 的影像縮成約 $H/s \times W/s$ 個樣本。若事先沒有平滑，可能會產生**混疊**（aliasing）：較粗的網格無法表示的細節，不會消失，而是變成假的圖樣（正式處理見IP 02 與頻率域各章）。把位元深度從 8 降到 b，就只剩 2^b 個相異數值。下面的輔助函式把每個像素四捨五入到所屬區間的中心，所以 4 位元時有 16 個允許的灰階。

```
def quantize(img, bits):
    step = 256 // 2 ** bits
    return (img // step) * step + step // 2    # 以每個區間的中心取代該區間

for bits in (8, 4, 2, 1):
    print(bits, len(np.unique(quantize(gray, bits))))

small = gray[::4, ::4]                          # 每隔 4 列、4 行取一個
print(gray.shape, "->", small.shape, small.nbytes, gray.nbytes)

def invert_loop(img):
    out = np.empty_like(img)
```

```
    for r in range(img.shape[0]):
        for c in range(img.shape[1]):
            out[r, c] = 255 - img[r, c]
    return out

print(np.array_equal(invert_loop(gray), 255 - gray))
chw = np.transpose(rgb, (2, 0, 1))
print(rgb.shape, "->", chw.shape, np.shares_memory(chw, rgb))
print(np.mean(rgb, axis=(0, 1)))
```

輸出：

```
8 256
4 16
2 4
1 2
(512, 512) -> (128, 128) 16384 262144
True
(512, 512, 3) -> (3, 512, 512) True
[141.56249237 105.75944519  96.47507477]
```

相異數值的個數以 2^b 下降。每隔 4 列、4 行取一個，會把影像從 262,144 位元組縮到 16,384 位元組，也就是記憶體的十六分之一。巢狀迴圈與單一運算式 `255 - gray` 的結果完全相同（`True`），但陣列運算式是以一次編譯過的運算執行，而不是二十六萬多次 Python 步驟：能寫成陣列運算式的影像程式就盡量這樣寫，把明確的迴圈留給檢查驗證用。轉置成通道優先 `(3, 512, 512)` 也是一個視圖，這是多數深度學習函式庫期望的排列。最後一行對兩個空間軸取平均，得到每個色彩通道一個平均值。

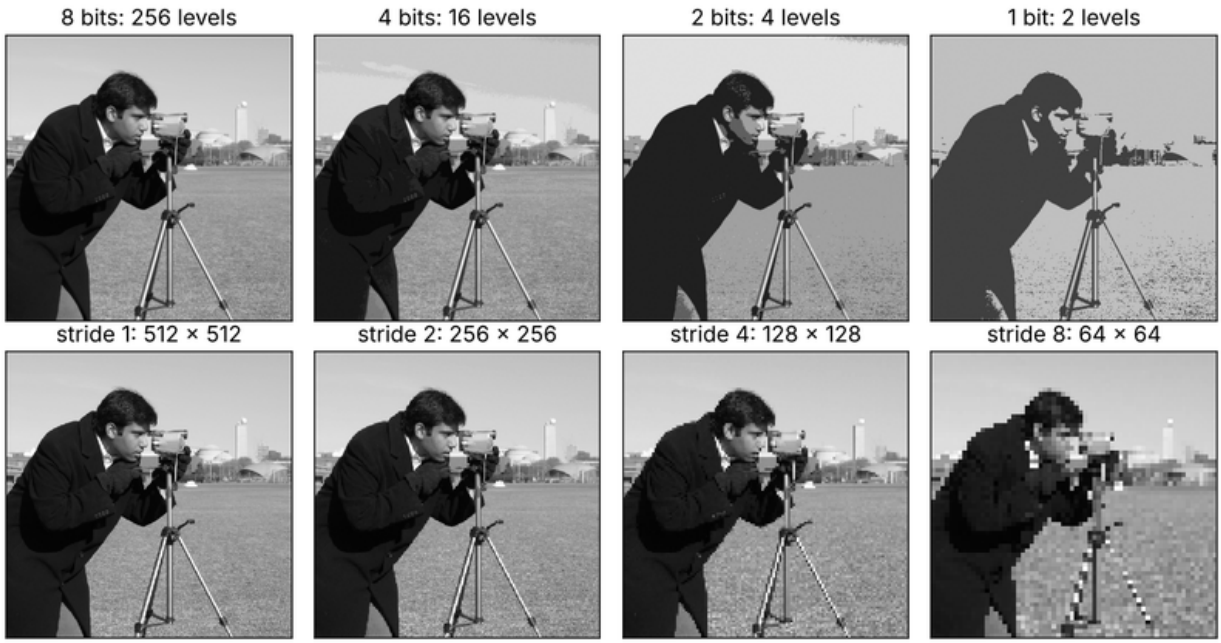


圖 4 — 兩種失去資訊的方式。上：較少灰階（量化）把平滑的明暗變成平坦的色帶。下：較少樣本（此處是單純點取樣，並以相同大小顯示）讓影像變成方塊狀，也破壞腳架腳這類細長結構。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

反過來想也很有用。8 位元每通道通常足以顯示影像，但不一定足以處理影像：把一張偏暗又有色帶的影像拉伸對比，只會放大色帶。這就是為什麼存在每通道位元數更多的影像格式與感測器，也是為什麼處理流程會先轉成浮點數再做運算。上方的實驗讓你在照片與一張會暴露混疊的測試圖樣上同時調整這兩個旋鈕。

處理影像前的簡短檢查表

檢查	原因	一行程式
印出 `shape` 與 `dtype`	確認是灰階或彩色，以及數值範圍	`print(a.shape, a.dtype, a.min(), a.max())`
確認通道順序	RGB 與 BGR 看起來很像，直到顏色錯了	`rgb = bgr[..., ::-1]`
選擇運算用的 dtype	`uint8` 會回繞；[0,1] 的 `float32` 對濾波安全	`x = a.astype(np.float32) / 255`
編輯前先複製	切片是視圖，編輯會寫回原陣列	`b = a.copy()`
轉回前先裁切	超出範圍的浮點數轉成 `uint8` 會變成亂碼	`np.clip(y, 0, 1)`
記得 `[列, 行]`	繪圖 API 的座標對是 `(x, y)`	`a[y, x]`

現代觀點

本章沒有任何內容過時，因為深度學習框架把影像視為張量（tensor），而張量的行為就像 NumPy 陣列。改變的是排列方式、正規化，以及近年研究中對「影像是什麼」的想法。

張量與排列方式。訓練批次會多一個前導軸，成為 `(N, H, W, C)`，或許多函式庫期望的通道優先 `(N, C, H, W)`，就像上面用 `np.transpose` 得到的視圖。像素值通常轉成 `float32` 並正規化，例如除以 255 再對每個通道標準化。上面所有陷阱都原封不動地延續：以 RGB 輸入訓練的模型若被餵入 BGR，會悄悄變差；在 `uint8` 中回繞的前處理步驟，可能不報任何錯誤就毒化整個資料集。卷積與 Transformer 各章直接建立在這個觀點上：ML 03 與ML 04。

影像視為區塊序列。 Dosovitskiy 等人的 Vision Transformer [10]（ICLR 2021）把影像切成固定大小區塊的網格，用本章的重塑方式把每個區塊攤平成向量，再把這個序列送進標準 Transformer。該論文報告，這類模型在大規模預訓練後，於影像分類基準上可以追平或超越卷積網路，同時需要較少的訓練運算量。對本章而言，重點在於陣列運算（切成區塊、重塑、轉置）就是模型的前端。

影像視為函式。第二個現代想法，是用學習得到的函式取代網格。Sitzmann 等人 [11]（NeurIPS 2020）提出正弦表示網路（sinusoidal representation networks），這是使用週期性活化函數的小型網路，輸入座標 (r, c)、輸出像素值，並展示它們能細緻地表示影像、音訊、影片及其導數。

Essakine 等人 [12]（TMLR 2025）對這個隱式神經表示（implicit neural representation）領域做了綜述：他們針對四類方法（活化函數、位置編碼、組合策略與網路結構最佳化）提出分類架構、做實驗比較，並以開放問題作結，例如更具表現力的活化函數、更好的位置編碼，以及對高維資料的可擴展性。陣列仍然是這類模型訓練與評估的方式，因為網路是擬合在取樣後的像素上；但解析度成為查詢函式時的屬性，而不是儲存表格的屬性。

哪些仍未解決、哪些已經確定。仍未定論的問題包括：如何在不同工具之間一致地儲存高動態範圍與廣色域色彩，以及在相同儲存成本下，學習得到的表示應如何與一般陣列比較。已經確定、值得練到自動反應的，就是本章這些：shape、dtype、通道順序、視圖與複本，以及先選運算型別再計算的習慣。

重點整理

- 數位影像是取樣並量化後的訊號，儲存成形狀 `(H, W)` 或 `(H, W, C)` 的陣列；$I[r, c]$ 表示第 r 列、第 c 行。
- `shape`、`dtype` 與 `strides` 描述陣列。一張 512 × 512 的 8 位元灰階影像有 262,144 個位元組；彩色版本是它的三倍。
- 切片回傳與原陣列共用記憶體的視圖。編輯前先用 `.copy()`，並且用陣列運算式取代逐像素迴圈。
- 各函式庫對通道順序（OpenCV 用 BGR，多數其他用 RGB）以及灰階轉換權重的看法不同；要檢查，不要假設。
- `uint8` 運算以 256 為模回繞。先轉成 `int16` 或 `float32`，計算、裁切，最後才轉回去。
- 空間解析度與灰階解析度是兩個獨立的選擇，降低任一個都會以不同方式失去資訊：方塊化與混疊，對上色帶。
- 深度學習張量、影像區塊與隱式神經表示，都建立在同一個影像陣列觀點上。

練習

1. **通道平均。**載入 `data.astronaut()`，用 `rgb.mean(axis=(0, 1))` 計算每個色彩通道的平均值。哪個通道最大？接著寫出同一張影像的 BGR 版本結果，並說明最大值現在位於哪個索引。

   提示

   這張影像的 R、G、B 平均值約為 141.6、105.8 與 96.5，所以紅色最大。在 BGR 陣列中，同樣的數字以相反順序出現，因此最大值在索引 2，而不是索引 0。

2. **用切片翻轉。**只用切片（不用 `np.flip`、不用迴圈），產生 `gray` 的上下翻轉、左右鏡像與旋轉 180° 的版本。分別與 `np.flipud`、`np.fliplr` 和 `np.rot90(gray, 2)` 比對。

   提示

   在你想反轉的軸上使用步長 −1：`gray[::-1]`、`gray[:, ::-1]` 與 `gray[::-1, ::-1]`。這三個都是視圖，所以如果要編輯，請先複製。

3. **找出回繞**。對本頁調亮 60 的例子，哪些輸入灰階值會回繞？輸入 200 會得到什麼輸出值？寫一行檢查程式，在不計算回繞後影像的情況下，計算被回繞的像素數。

   提示

   輸入為 196 以上的會回繞，因為 $196 + 60 = 256 \equiv 0$。輸入 200 得到 $260 \bmod 256 = 4$。計數是 `(gray >= 196).sum()`。

4. **安全的差值**。兩個 `uint8` 畫面 A 與 B 應以絕對差值 $|A - B|$ 比較。用一對像素值說明 `A - B` 會出錯，並寫出 NumPy 的安全版本。

   提示

   在 `uint8` 中，若 $A = 10$、$B = 20$，`A - B` 等於 246，而不是 10。安全版本是 `np.abs(A.astype(np.int16) - B.astype(np.int16)).astype(np.uint8)`。

5. **設計位元深度預算**。一張灰階影像以 8 位元儲存，再量化到 b 位元。若每個像素剛好壓縮成 b 位元，請預測 $b = 4$ 與 $b = 2$、512×512 時的相異數值個數與所需記憶體。為什麼圖 4 的 4 位元影像看得到色帶，而 8 位元看不到？

   提示

   有 2^b 個數值：16 與 4。壓縮儲存為 $512 \times 512 \times b/8$ 位元組，4 位元時是 131,072 位元組，2 位元時是 65,536 位元組。色帶出現在平滑漸層跨越兩個區間邊界的地方：8 位元時每一階只差 1 個灰階，在多數顯示器上低於人眼可分辨的程度，而 4 位元時每階相差 16 個灰階。

參考文獻

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018 (Chapter 2, image sensing, sampling and quantization). publisher page
2. A. Torralba, P. Isola and W. T. Freeman, *Foundations of Computer Vision*, MIT Press, 2024 (open-access edition). book site
3. C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020. doi
4. NumPy developers, "Data types," NumPy user guide, current documentation. docs
5. NumPy developers, "NEP 50: Promotion rules for Python scalars," NumPy Enhancement Proposals. NEP 50
6. S. van der Walt et al., "scikit-image: image processing in Python," *PeerJ*, vol. 2, e453, 2014. doi
7. scikit-image developers, "skimage.data: test images and example data," scikit-image documentation. docs
8. ITU-R, "Recommendation BT.601-7: Studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios," 2011. ITU page
9. Matplotlib developers, "matplotlib.pyplot.imshow," Matplotlib documentation. docs
10. A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," *International Conference on Learning Representations* (*ICLR*), 2021. arXiv:2010.11929

11. V. Sitzmann, J. N. P. Martel, A. W. Bergman, D. B. Lindell and G. Wetzstein, "Implicit Neural Representations with Periodic Activation Functions," *Advances in Neural Information Processing Systems* (*NeurIPS*), vol. 33, 2020. proceedings
12. A. Essakine et al., "Where Do We Stand with Implicit Neural Representations? A Technical and Performance Survey," *Transactions on Machine Learning Research* (*TMLR*), 2025. arXiv:2411.03688

篇 II

影像處理

IP 01・緒論

什麼是數位影像處理、它從哪裡來、它處理電磁波譜中的哪些訊號，以及本系列依循的基本步驟路線圖。

網站版本: https://blog.shyandram.dev/zh-tw/learn/image-processing/dip-01-introduction/

你將學到

- 什麼是數位影像——用文字說明，也用函數 $f(x, y)$ 表示——以及「處理」一張影像是什麼意思
- 從低階、中階、高階處理的角度，看影像處理在哪裡結束、電腦視覺從哪裡開始
- 這個領域的起源：用電報傳送的新聞照片、月球探測器，以及醫學掃描儀
- 我們處理的影像來自電磁波譜的哪些波段，以及哪些非光學的訊號
- 數位影像處理的基本步驟，同時也是本系列 13 章的路線圖
- 一套影像處理系統由哪些硬體與軟體組成，以及深度學習如何重塑相機的處理流程

先看全貌

> **用最簡單的話說**
>
> 想像一幅由許多小方磚拼成的大馬賽克。每塊磚只有一種灰色，並寫著一個數字：0 是黑色，255 是白色。電腦「看」不到這幅馬賽克，它只能讀那一長串數字。**數位影像處理**就是一整套改動這些數字的技巧，讓畫面變得更清楚、更小，或更容易理解：擦掉灰塵造成的斑點、把太暗的照片調亮，或是替桌上每一枚硬幣描出輪廓，好讓我們數清楚有幾枚。

今天你看到的幾乎每一張影像，在抵達你眼前之前都經過了影像處理。從光線打到手機感測器，到照片出現在螢幕上，手機已經執行了幾十道運算。醫院的電腦斷層掃描儀根本不「拍照」：它量測 X 光沿著成千上萬條直線被吸收了多少，再把影像算出來。氣象衛星記錄的是人眼看不見的能量，再把它變成你看得懂的地圖。

這一章是出發前的地圖。它界定這個領域、說明它的來歷、展示它能處理的訊號種類，並列出後續各章逐一介紹的步驟 [1]。本章沒有艱深的內容，但後面每一章都會用到我們在這裡建立的詞彙。

什麼是數位影像處理？

白話版。把一張圖切成由小方格組成的網格，每個方格存一個數字（彩色影像則存幾個數字），這張圖就變成了數位影像。處理的意思是：把這個數字網格交給電腦，得到某種有用的結果。

影像是一個函數

在數學上，灰階影像是一個二維函數

$$f(x, y), \qquad (x, y) \in \mathbb{R}^2.$$

其中 x 與 y 是空間座標，函數值 $f(x, y)$ 是影像在該點的**強度**（intensity），也稱為灰階值（gray level）。對相機而言，強度正比於抵達感測器的光能；對 X 光底片而言，則正比於穿透人體的輻射量。在真實世界中，x、y 與 f 都可以連續變化。

當這三個量都是有限且離散的，影像就是**數位影像**。我們在一個 $M \times N$ 的網格上取樣，並把每個值量化成 L 個等級之一：

$$f : \{0, 1, \ldots, M-1\} \times \{0, 1, \ldots, N-1\} \to \{0, 1, \ldots, L-1\}, \qquad L = 2^k.$$

M 是列數，N 是行數，k 是每個值所用的位元數（$k = 8$ 就得到熟悉的 0 到 255）。網格中的每個元素稱為**像素**（pixel，即 *picture element* 的縮寫）。第 2 章會說明取樣與量化該如何選擇；現在只要記得：影像就是一個矩陣。

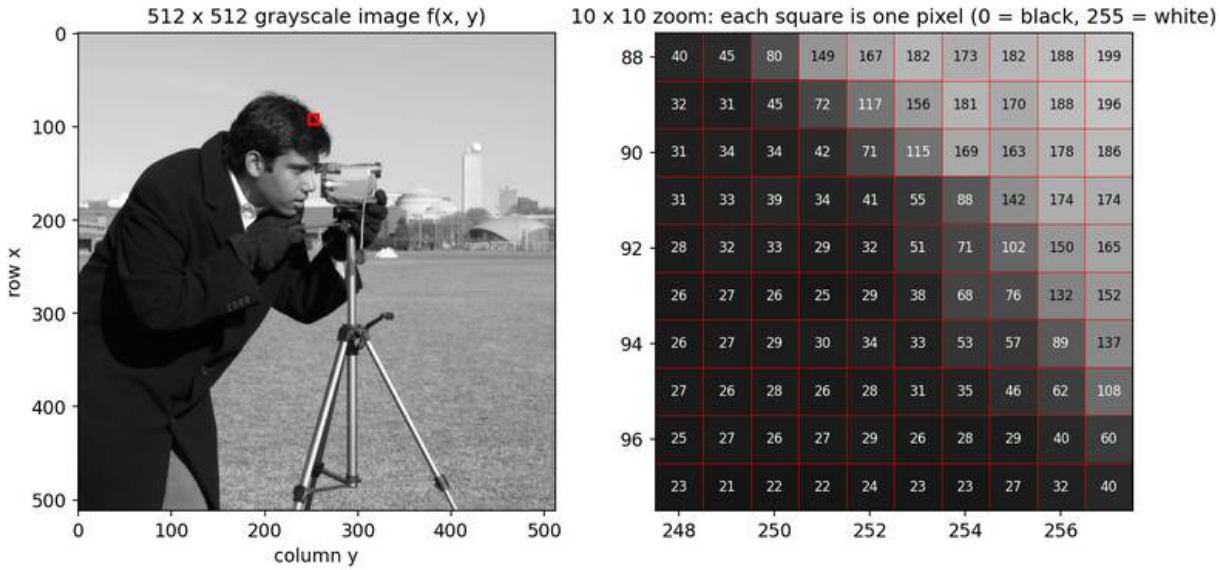


圖 *1.1* — 左：一張 *512 × 512*、*8* 位元的灰階影像。右：紅框內 *10 × 10* 區域的放大圖。每個方格就是一個像素，裡面的數字是它的灰階值。深色頭髮與明亮天空之間的「邊緣」，其實只是數值從 *30* 左右跳到 *180* 左右而已。

在程式中，這個矩陣就是一個 NumPy 陣列。用 `[列, 行]` 索引可以讀取單一像素，用切片則取得一塊鄰域：

```
from skimage import data

f = data.camera()            # 一張 512 x 512 的灰階照片
print(f.shape, f.dtype)      # (512, 512) uint8
print(f[100, 250])           # 第 100 列、第 250 行的強度
print(f.min(), f.max())      # 最暗與最亮的值
```

```
patch = f[88:92, 248:252]    # 4 x 4 鄰域就是一個子陣列
print(patch)
```

哪一軸是 x？

本系列沿用教科書的慣例 [1]：x 是列的索引（由上往下），y 是行的索引（由左往右），因此 $f(x,y)$ 正好對應 NumPy 的 `f[x, y]`。許多繪圖函式庫，以及先修教學〈Images as Arrays〉，用的是相反的順序（列 $= y$、行 $= x$）。當程式和數學對不上時，先檢查座標軸。

彩色影像會多一個通道索引，寫成 $f(x,y,c)$；醫學體積資料則多一個深度索引，寫成 $f(x,y,z)$。本章的觀念完全適用。

處理在哪裡結束，視覺從哪裡開始

影像處理在哪裡結束、影像分析或電腦視覺從哪裡開始，大家看法不一。一種極端的說法是：影像處理只包含輸入與輸出都是影像的運算。這個規則很乾淨，卻太狹隘——連計算一張照片的平均亮度這麼簡單的事都會被排除在外。另一個極端是電腦視覺，它的目標是模仿人類視覺：理解場景、辨識物體，並據此行動。電腦視覺的教科書把它視為一個獨立領域，核心是幾何、三維重建與辨識 [7]。

比較實用的看法，是把它想成一條連續光譜，上面有三種處理 [1]：

- **低階處理**：輸入影像，輸出影像。例如去除雜訊、提高對比、銳化。不涉及任何解讀。
- **中階處理**：輸入影像，輸出屬性（attributes）：區域、邊緣、輪廓，或對個別物體的量測。影像分割（把影像切成有意義的部分），以及把這些部分描述成電腦能分類的形式，都是典型的中階工作。
- **高階處理**：替一群已辨識出的物體「賦予意義」。讀懂一整頁文字，或判斷一張醫學影像裡有沒有腫瘤，都屬於高階工作。光譜的這一端，就是影像分析與電腦視覺交會之處。

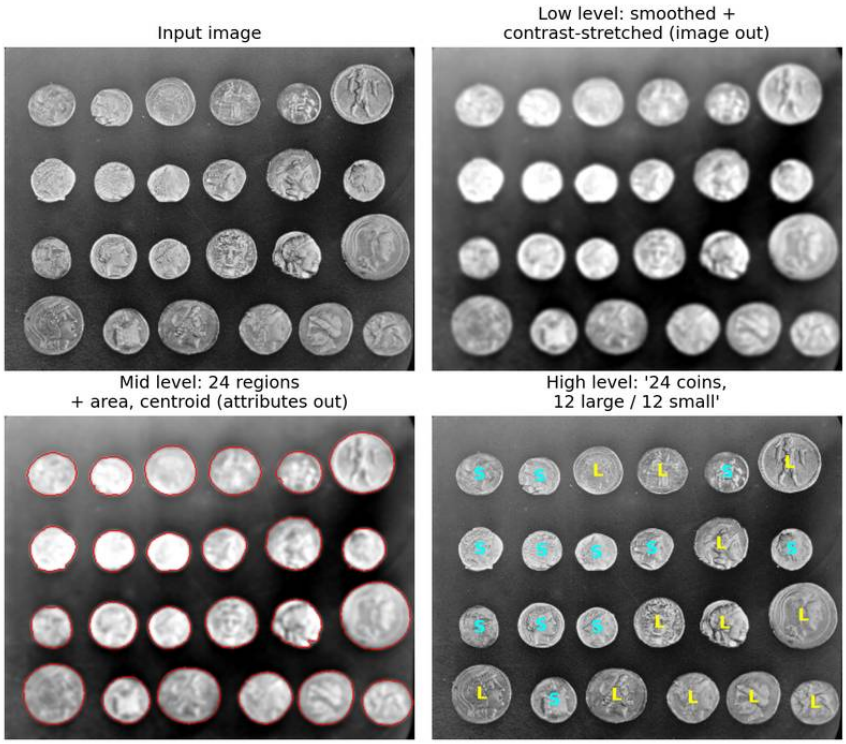


圖 *1.2* — 在同一張影像上看處理的連續光譜。低階：去雜訊並拉伸對比（輸出仍是影像）。中階：分割出硬幣並逐一量測（輸出是一串區域與數字）。高階：解讀這些量測值（輸出是一句話：共 *24* 枚硬幣，大的佔一半）。

在本系列中，「數位影像處理」涵蓋從低階一路到辨識個別區域或物體為止。這個範圍足以包含分割、特徵擷取與分類（第 10–13 章），但不包括完整的場景理解。

數位影像處理的起源

白話版。最早的數位圖片，是用打孔電報紙帶橫越大西洋傳送的新聞照片。真正用電腦處理影像始於 1960 年代，當時太空機構需要修正月球探測器傳回的影像；到了 1970 年代，能從量測資料「算出」影像的醫學掃描儀，讓這個領域蓬勃發展。

經由電報電纜傳送的圖片

1920 年代初期，**Bartlane 電纜圖片傳輸系統**透過橫越大西洋的海底電報電纜，在倫敦與紐約之間傳送新聞照片 [2]。照片先被轉換成打在一般五單位電報紙帶上的代碼，用標準電報設備傳送，再於接收端離線重組。根據 Kobayashi 對這套系統的歷史整理，直到 1939 年戰爭爆發為止，它一共橫越大西洋傳送了將近 500 張新聞照片 [2]。最早的 Bartlane 圖片只有五種灰階；到 1920 年代末，增加到十五種 [1]。

嚴格說來，這些圖片沒有經過任何電腦，所以這算是數位影像處理的史前史。但核心觀念已經出

現：把圖片轉成一組有限的代碼，傳送出去，再重建回來；而灰階等級的多寡，決定了結果有多逼真。

圖 *1.3* — 灰階數有多重要？同一張照片分別量化成 *5* 階（早期 *Bartlane*）、*15* 階（*1920* 年代末的 *Bartlane*）與 *256* 階（現代 *8* 位元）。只有 *5* 階時，大衣與草地這類平滑區域塌成一片片平坦色塊；即使有 *15* 階，天空中仍看得到一條條色帶。這種瑕疵稱為假輪廓（*false contouring*），第 *2* 章會解釋它。

幾行程式就能重現這個實驗。把強度 $r \in [0,1]$ 均勻量化成 L 個等級的映射為

$$Q_L(r) = \frac{\operatorname{round}\left(r\,(L-1)\right)}{L-1},$$

其中 round 表示四捨五入到最接近的整數，因此輸出只會是 $0, \frac{1}{L-1}, \ldots, 1$ 這 L 個值之一。

```
import numpy as np
from skimage import data

def quantize(img, levels):
    """ 把 uint8 影像映射到 `levels` 個等間距的灰階。"""
    x = img.astype(np.float64) / 255.0
    q = np.round(x * (levels - 1)) / (levels - 1)
    return (q * 255).astype(np.uint8)

f = data.camera()
for L in (5, 15, 256):
    g = quantize(f, L)
    print(L, "levels ->", len(np.unique(g)), "distinct values,",
          f"mean abs error {np.abs(g.astype(int) - f).mean():.2f}")
```

在這張測試影像上，平均絕對誤差從 $L = 5$ 時的約 18 個灰階，降到 $L = 15$ 時的約 5，再到 $L = 256$ 時的 0。

電腦遇上影像：太空計畫

我們今天所知的數位影像處理，需要兩樣直到 1960 年代才到位的東西：夠強大、能存放並操作影像的電腦，以及一個重要到值得為它花錢的問題。太空計畫兩者兼具。

1964 年 7 月 31 日，美國探測器 **Ranger 7** 拍下了美國太空船所攝的第一張月球影像，那是在它按計畫撞上月球表面前約 17 分鐘 [3]。在最後這幾分鐘裡，它傳回了超過 4,300 張影像，最後幾張的解析度約為每像素半公尺 [3]。用電腦修正這類探測器影像中的失真，成為數位影像處理的奠基計畫之一 [1]。同樣的技術後來又在之後的月球與行星任務中持續精進。

由量測算出的影像：電腦斷層

第二個奠基應用來自醫學。1973 年，Godfrey Hounsfield 描述了一套**電腦化橫向軸向掃描**（computerized transverse axial scanning）系統，也就是今天所說的電腦斷層掃描（computed tomography，CT）[4]。它不是只曝光一張底片，而是讓 X 光源與偵測器繞著病人旋轉，記錄光束沿著許多角度、許多直線被吸收的程度，再由電腦算出一個橫切面上每一點的吸收值。Hounsfield 指出，這能呈現傳統 X 光底片看不出的軟組織差異 [4]。

CT 之所以是里程碑，是因為影像不是拍出來的，而是重建出來的。掃描儀量測的是未知切面 $f(x, y)$ 的線積分：

$$p(s,\theta) = \iint_{\mathbb{R}^2} f(x,y)\,\delta(x\cos\theta + y\sin\theta - s)\,dx\,dy,$$

其中 θ 是光束的角度，s 是偵測器在投影方向上的位置，δ 是狄拉克 delta 函數，它只保留落在直線 $x\cos\theta + y\sin\theta = s$ 上的點。所有投影 $p(s,\theta)$ 合起來稱為**正弦圖**（sinogram）。從 p 還原出 f 是一個反問題（inverse problem），也是第 5 章後半「重建」的主題。這類演算法的標準測試物件，是 Shepp 與 Logan 在 1974 年提出、由多個橢圓組成的合成頭部切面 [5]。

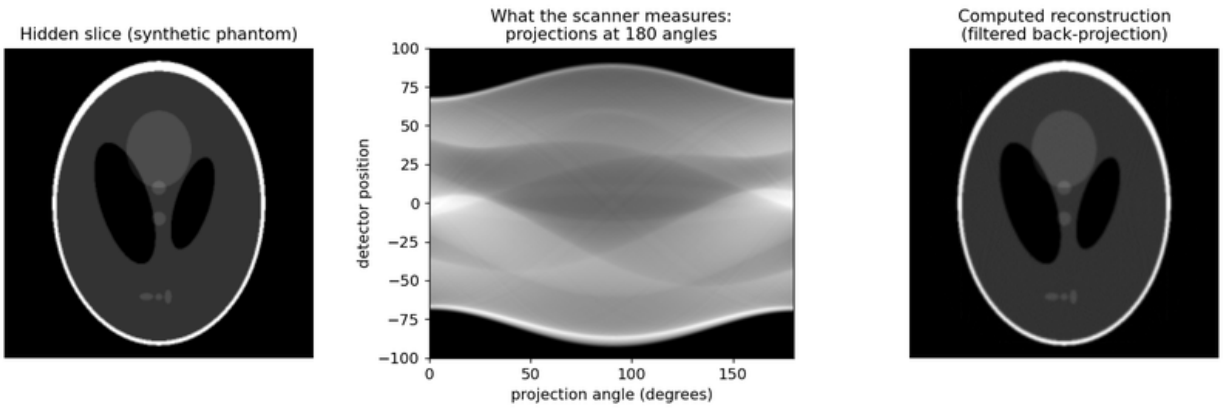


圖 *1.4* — 迷你版的電腦斷層。左：未知的切面（*Shepp–Logan* 假體 [5]）。中：掃描儀實際記錄的資料，每個角度一條投影。右：以濾波反投影（*filtered back-projection*）還原的切面，這個演算法會在第 *5* 章推導。

```
import numpy as np
from skimage.data import shepp_logan_phantom
```

```
from skimage.transform import radon, iradon, rescale

slice_ = rescale(shepp_logan_phantom(), 0.5)           # 200 x 200 的合成頭部切面
angles = np.linspace(0, 180, 180, endpoint=False)
sinogram = radon(slice_, theta=angles)                  # 掃描儀記錄到的資料
recon = iradon(sinogram, theta=angles, filter_name="ramp")
err = np.sqrt(np.mean((recon - slice_) ** 2))
print(sinogram.shape, recon.shape, f"RMS error {err:.3f}")   # (200, 180) (200, 200) ~0.025
```

從 1960 年代起，同樣的想法擴散到遙測、天文、生物、工業檢測、執法，最後走進每個人口袋裡的手機。所有這些應用背後有兩大目標：改善影像以便人來判讀，以及處理影像資料以便儲存、傳輸與機器感知。

使用數位影像處理的領域舉例

白話版。我們的眼睛只看得到「光」的一小段。其他種類的光，從伽瑪射線到無線電波，也都能形成影像；聲波和電子束也可以。影像處理能處理所有這些來源，因為它們最後都會變成一個數字網格。

整理應用最好的方式，是依影像的**來源**來分類 [1]。主要來源是電磁（electromagnetic，EM）能量；其他還有聲波、電子，以及純粹由計算產生的影像。

電磁波譜

電磁波可以用波長 λ 或頻率 ν 描述，兩者的關係為

$$\lambda = \frac{c}{\nu},$$

其中 $c \approx 2.998 \times 10^{8}$ m/s 是光速。光也以一包一包的**光子**（photon）形式存在，每個光子帶有能量

$$E = h\nu = \frac{hc}{\lambda},$$

其中 $h \approx 6.626 \times 10^{-34}$ J·s 是普朗克常數。波長短代表頻率高、光子能量大；波長長代表頻率低、光子能量小。光是這一點就能解釋成像的許多現象：高能光子能穿透組織與金屬（X 光），低能量的波則能穿透雲層與黑暗（雷達與無線電）。

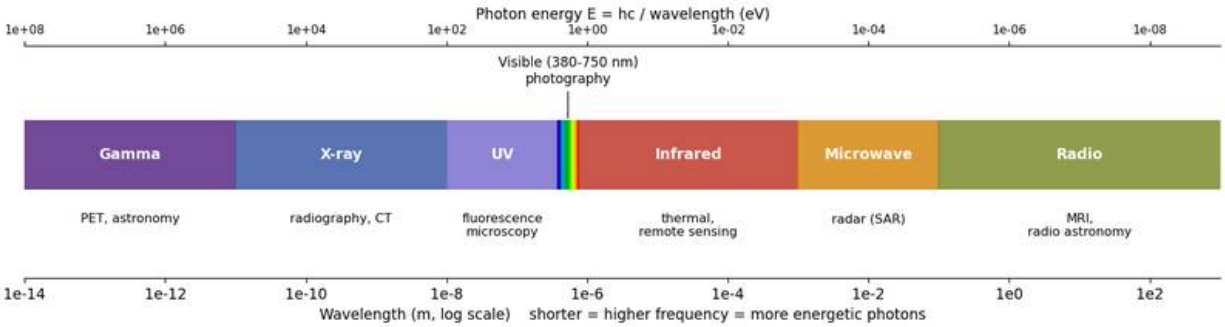


圖 *1.5* — 對數尺度下的電磁波譜，上方座標軸為光子能量。可見光只是其中細細的一條。各波段的分界只是慣例，不同資料來源略有差異。

```
h, c, eV = 6.626e-34, 2.998e8, 1.602e-19   # 普朗克常數、光速、每 eV 的焦耳數

def photon_energy_ev(wavelength_m):
    return h * c / wavelength_m / eV

for name, lam in [("X-ray (0.1 nm)", 1e-10), ("green light (550 nm)", 550e-9),
                  ("thermal IR (10 um)", 10e-6), ("radar (3 cm)", 0.03)]:
    print(f"{name:22s} {photon_energy_ev(lam):10.3g} eV")
# X-ray ~1.24e4 eV, green ~2.25 eV, thermal IR ~0.124 eV, radar ~4.1e-5 eV
```

一個 X 光光子的能量大約是綠光光子的一萬倍，而雷達光子的能量還不到綠光光子的萬分之一。以下各波段依能量由高到低排列。

伽瑪射線成像

伽瑪射線是能量最高的光子。在**核子醫學**中，病人接受放射性示蹤劑，偵測器記錄示蹤劑放出的伽瑪射線；影像呈現的是示蹤劑聚集在哪裡，因此它描繪的是功能（例如代謝），而不是解剖構造。正子斷層掃描（positron emission tomography，PET）是這個概念的斷層版本，使用與 CT 相同的重建數學。在**天文學**中，伽瑪射線望遠鏡拍攝爆炸恆星殘骸這類劇烈事件。伽瑪射線影像通常雜訊很多，因為每個像素只累積到有限數量的光子事件；因此去雜訊與復原（第 3、5 章）在這裡特別重要。

X 光成像

X 光是最古老的醫學成像來源。在一般的 **X 光攝影**（radiograph）中，X 光穿過人體後讓偵測器曝光；骨骼等高密度組織吸收較多，因此看起來較亮。**血管攝影**（angiography）把顯影劑注入血管，再把注射前與注射後的影像相減以凸顯血管——這是第 2 章會介紹的影像算術運算。前面介紹的 **CT** 則把大量 X 光投影轉成切面，並堆疊成立體資料 [4]。醫學以外，X 光可以檢查焊接點與電路板裡看不見的瑕疵，X 光望遠鏡則能拍攝星系團中的高溫氣體。

紫外線成像

紫外線（UV）位於紫光之外。它最重要的應用是**螢光顯微術**（fluorescence microscopy）：有些物質吸收紫外線光子後，會再放出能量較低的可見光。在一台濾掉紫外激發光、只讓放射光通過的顯微鏡下，會發螢光的構造就在黑色背景上發亮。生物學家用螢光標記特定蛋白質，觀察它們在細胞中的位置。紫外線成像也用於天文與工業檢測，例如找出白光下看不見的殘留物。

可見光與紅外線成像

日常的成像大多發生在可見光波段：攝影、**光學顯微術**、工業**機器視覺**（machine vision；例如檢查藥片包裝是否填滿、辨識車牌、量測零件尺寸）、文件掃描，以及人臉與指紋分析。可見光常與相鄰的**紅外線**（IR）波段搭配使用。

遙測（remote sensing）衛星會在從藍光到紅外線的數個窄波段中記錄同一個場景，稱為多光譜（multispectral）影像。由於植被、水與土壤對這些波段的反射率不同，組合不同波段就能繪製作物、洪水或都市擴張的地圖。**熱紅外線**相機感測的是物體放出的熱，而不是反射光，因此能用於夜間、消防與建築檢測。氣象衛星也利用紅外線頻道全天候追蹤雲層與暴風。

微波成像（雷達）

微波成像的主力是**成像雷達**。雷達自帶照明：它發出微波脈衝並記錄回波。由於微波能穿透雲、霾與黑暗，雷達可以拍攝光學衛星可能連續數週都看不到的地區，例如終年雲霧籠罩的雨林。合成孔徑雷達（synthetic aperture radar，SAR）利用飛機或衛星的移動，合成出一面非常大的天線，以獲得很高的解析度；把記錄下來的回波變成影像，本身就是一個計算步驟 [8]。

無線電波段成像

在長波長的一端，醫學上的主要應用是**磁振造影**（magnetic resonance imaging，MRI）。病人躺在強磁場中，短暫的射頻脈衝激發氫原子核，再記錄它們放出的微弱無線電訊號。1973 年，Paul Lauterbur 證明加上磁場梯度後，這些訊號會隨位置而不同，因此可以由它們形成影像 [6]。和 CT 一樣，MRI 必須經過計算才會產生影像：原始量測值位於頻率域（第 4 章的主題），必須轉換回來。

在**電波天文學**中，波長很長意味著單一碟形天線的解析度很差，所以天文學家會結合許多望遠鏡的訊號。2019 年，事件視界望遠鏡合作團隊（Event Horizon Telescope Collaboration）發表了第一張超大質量黑洞陰影的影像，位於 M87 星系中心，由結合全球各地電波天文台的資料而成 [9]。這張影像既是望遠鏡的成果，也同樣是重建演算法的成果。

其他成像方式

並非所有影像都來自電磁輻射。

- **聲波成像**。聲波會在不同材料的交界處反射。地質學家把低頻聲波送入地底並記錄回波，藉此描繪地下岩層，用於石油、天然氣與礦產探勘（震測成像，seismic imaging）。醫學**超音波**使

用高頻聲波（每秒數百萬次振動），能即時、且不使用游離輻射地拍攝胎兒、心臟與腹部器官。這兩種情況都是靠回波的時間來形成影像。

- **電子顯微術**。電子可以像光一樣被聚焦，但它的等效波長短得多，因此電子顯微鏡能解析比光學顯微鏡細得多的細節。穿透式電子顯微鏡讓電子束穿過薄標本；掃描式電子顯微鏡則讓聚焦的電子束掃過表面，記錄被反彈或撞擊出來的電子。這類影像常帶有雜訊，需要強化與復原。
- **合成影像**。電腦也會產生沒有任何感測器拍過的影像：碎形、算繪出來的三維場景、模擬結果，以及越來越多由生成模型產生的影像。這些影像用同一套工具處理，也大量用來製作訓練資料與測試案例。

這趟巡禮的重點是：一旦資料變成數字網格，同一套工具就都適用，不論這些數字來自伽瑪射線、聲波回音或電腦程式。真正改變的是雜訊、解析度與成像的物理原理，而這些差異決定了你該拿出哪一種工具。

數位影像處理的基本步驟

白話版。讓一張影像派上用場就像做菜：先備料（擷取影像），再清洗、處理（強化、復原），有時改變食材的形態（色彩、轉換、壓縮），然後切塊（分割），描述每一塊，最後判斷每一塊是什麼（分類）。不是每道菜都需要每個步驟。

把這些步驟分成兩組會比較好理解 [1]：輸出通常是**影像**的方法，以及輸出通常是從影像擷取出的**屬性**的方法。關於問題領域的**知識庫**（knowledge base）——例如瑕疵通常出現在哪裡、什麼樣的標籤才合理——引導著每一個步驟。這份清單並不是非走不可的流程；實際應用只會用到需要的步驟，並依合理的順序排列。

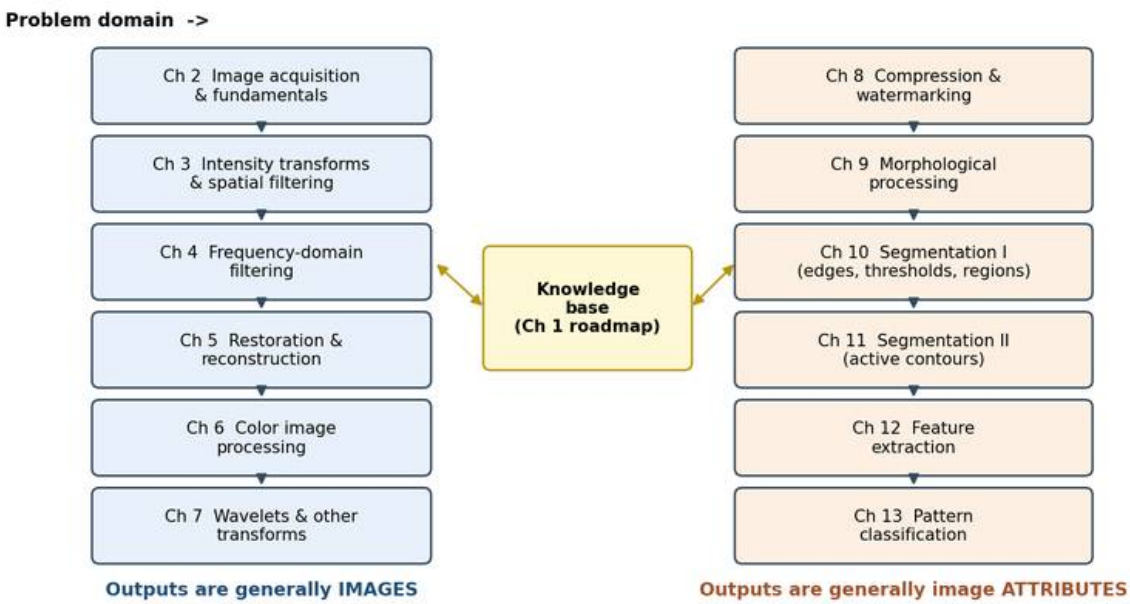


圖 *1.6* — 數位影像處理的基本步驟，標上本系列中負責介紹它們的章節。左側步驟的輸出通常是影像，右側步驟的輸出通常是屬性。中央的領域知識影響所有步驟。

以下逐一介紹各步驟，以及負責的章節。

1. **影像擷取**（image acquisition）——取得數位形式的影像，包括感測、取樣與量化。第 2 章也會建立之後處處用到的數學工具（像素間的關係、影像算術、幾何轉換）。→ IP 02・數位影像基礎
2. **影像強化**（image enhancement）——讓影像更適合特定用途，例如提高對比或銳化細節。強化是主觀的：「更好」取決於觀看者與任務。→ IP 03・強度轉換與空間濾波 與IP 04・頻域濾波
3. **影像復原**（image restoration）——消除已知或估計出的劣化，例如模糊或雜訊。和強化不同，復原是客觀的：它依賴描述影像如何劣化的數學模型。同一章也介紹由投影重建影像（CT）。→ IP 05・影像復原與重建
4. **彩色影像處理**（color image processing）——色彩模型、虛擬色彩，以及全彩影像的處理。→ IP 06・彩色影像處理
5. **小波與其他影像轉換**——同時以多種解析度表示影像，是現代壓縮與金字塔方法的基礎。→ IP 07・小波與其他影像轉換
6. **壓縮與浮水印**——減少影像所需的儲存空間或頻寬，以及嵌入看不見的資訊以保護所有權。→ IP 08・影像壓縮與浮水印
7. **形態學處理**（morphological processing）——擷取與整理形狀成分的工具，例如細線化、填補孔洞、分開相黏的物體。從這裡開始，輸出逐漸變成屬性。→ IP 09・形態學影像處理
8. **影像分割**（segmentation）——把影像切分成組成它的部分或物體。這是最困難的步驟之一，這裡的錯誤會延續到之後的每一步。→ IP 10・影像分割（一）（邊緣、閾值、區域）與IP 11・影像分割（二）（主動輪廓：snakes 與 level sets）
9. **特徵擷取**（feature extraction）——把分割出的區域轉成數字：邊界與區域描述子，以及角點這類點特徵。它包含兩部分：偵測特徵與描述特徵。→ IP 12・特徵擷取
10. **影像樣式分類**（image pattern classification）——根據物體的特徵替它貼上標籤，從最小距離分類器一路談到類神經網路與深度學習。→ IP 13・影像樣式分類

第 1 章（也就是本頁）是把它們串在一起的總覽。

下面的程式把其中四個步驟串起來，用在圖 1.2 的硬幣影像上。每個步驟在這裡只需一兩個函式呼叫；後面的章節會解釋每一步內部在做什麼。

```
from scipy import ndimage as ndi
from skimage import data, filters, measure, segmentation

img = data.coins()

# 1) 強化：低階，輸入影像 -> 輸出影像
smooth = filters.gaussian(img, sigma=1, preserve_range=True)

# 2) 分割：中階，輸入影像 -> 輸出區域
markers = (smooth < 30) * 1 + (smooth > 150) * 2       # 確定的背景 / 確定的硬幣
regions = segmentation.watershed(filters.sobel(smooth), markers) == 2
regions = ndi.binary_fill_holes(regions)
labels = measure.label(regions)

# 3) 特徵擷取：輸入區域 -> 輸出數字
props = [p for p in measure.regionprops(labels) if p.area > 300]
areas = [p.area for p in props]
```

```
# 4) 分類 / 解讀：輸入數字 -> 輸出一句結論
median = sorted(areas)[len(areas) // 2]
n_large = sum(a >= median for a in areas)
print(f"{len(areas)} coins found, {n_large} at or above the median size")
# 24 coins found, 12 at or above the median size
```

注意藏在這段程式裡的知識庫：閾值 30 與 150、最小面積 300，以及「大於等於中位數就算大」這條規則，全都編碼了這個特定問題的知識。換了光源或換了硬幣，這些數字就得跟著改。這個領域的大半歷史——包括深度學習的崛起——說穿了，就是把這類知識從手動設定的常數，搬進從資料中學出來的模型。

影像處理系統的組成元件

白話版。一套影像處理系統就像一支團隊：感測器負責接住影像，快速的專用晶片負責粗重的運算，電腦執行程式，記憶體存放圖片，螢幕讓人觀看，印表機輸出紙本，網路把一切連在一起。

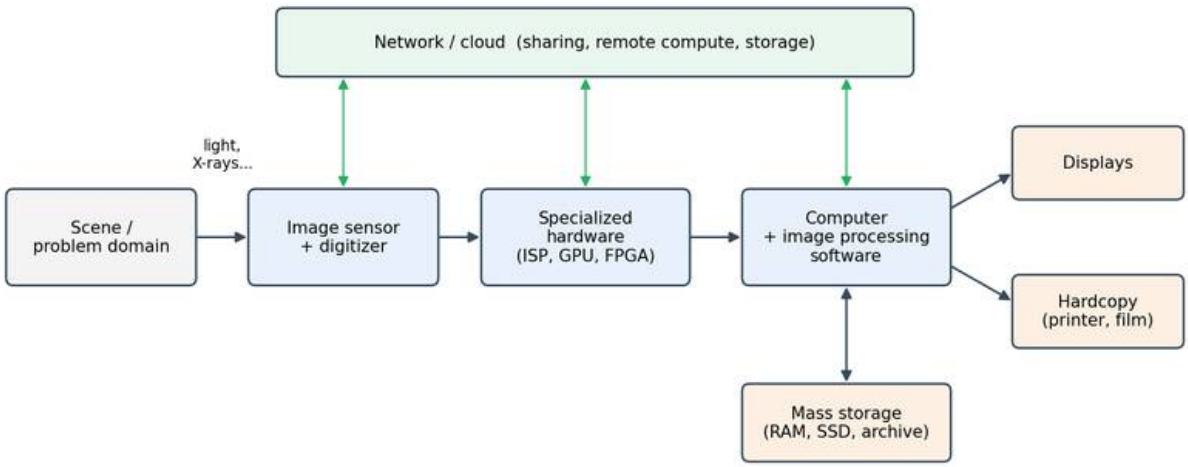


圖 *1.7* — 通用影像處理系統的組成元件。箭頭表示主要的資料流向；網路把每個階段連到遠端的儲存與運算資源。

- **影像感測器**。擷取影像需要兩個元件：一個會對物體輻射出的能量產生反應的實體裝置（CCD 或 CMOS 晶片、X 光偵測器、超音波探頭），以及把裝置的電訊號轉成數字的**數位轉換器**（digitizer）。在手機中，兩者做在同一顆晶片上。
- **專用影像處理硬體**。有些運算必須以影片速率對每個像素執行，比通用處理器能負擔的還快。早期系統使用專門的算術運算板；今天這個角色由每顆相機晶片裡的影像訊號處理器（image signal processor，ISP）、繪圖處理器（GPU）、現場可程式化邏輯閘陣列（FPGA），以及手機中的神經網路處理器擔任。
- **電腦**。從智慧門鈴裡的嵌入式微控制器，到整個伺服器叢集都有可能。專用任務的電腦可以量身訂做；研究與一般用途則一台普通工作站就夠了。

- **軟體**。執行特定任務的模組，再加上把它們組合起來的方法。本系列使用 Python 搭配 NumPy、SciPy、scikit-image 與 OpenCV。本系列希望培養的，是自己寫程式的能力，而不只是呼叫現成的函式。
- **大量儲存裝置**。影像很大，而且大批湧入。儲存分成三層：處理期間的短期儲存（RAM、GPU 記憶體、影格緩衝區）、可快速取用的線上儲存（SSD 與磁碟陣列），以及很少存取的封存儲存（磁帶、冷儲存雲端服務）。
- **影像顯示器**。今天大多是彩色平面顯示器。醫學診斷使用經過校正的專用螢幕；立體視覺或虛擬實境則使用頭戴式顯示器。
- **硬拷貝裝置**。雷射與噴墨印表機、底片相機、熱感印表機。底片仍提供極高的解析度，而紙本依然是書面資料與許多報告的首選媒介。
- **網路與雲端**。幾乎每套系統都已連網。傳輸影像是頻寬問題，這正是壓縮（第 8 章）是核心主題的原因。雲端服務把儲存與繁重的運算（例如訓練深度模型）從裝置上移走。

「很大」到底有多大？一張有 M 列、N 行、C 個通道、每個值 k 位元的影像，在壓縮前需要

$$b = M \times N \times C \times k \text{ bits}$$

其中灰階影像 $C = 1$，RGB 彩色影像 $C = 3$。

```
def raw_size_mb(height, width, channels=1, bits=8, frames=1):
    return height * width * channels * bits * frames / 8 / 1e6

print(f"{raw_size_mb(512, 512):.2f} MB   - this chapter's 8-bit test photo")         # 0.26
print(f"{raw_size_mb(3000, 4000, 3):.0f} MB   - one 12-megapixel RGB photo")         # 36
print(f"{raw_size_mb(512, 512, bits=16, frames=300):.0f} MB  - a 300-slice 16-bit CT
 ↪  volume")  # 157
print(f"{raw_size_mb(2160, 3840, 3, frames=30*60):.0f} MB - one minute of uncompressed 4K
 ↪  video")  # 44790
```

一分鐘、每秒 30 影格、未壓縮的 4K 影片大約是 45 GB。沒有壓縮，串流影片與充滿圖片的網頁都將窒礙難行。

現代觀點

教科書把這個領域看成一條由理解透徹、人工設計的步驟組成的流程，這仍然是學習影像處理的正確方式。但大約 2010 年以後，有兩項發展改變了這個領域的實務：計算被搬進了成像過程本身，而學習出來的模型取代或強化了許多人工設計的步驟。以下的綜述論文是很好的入門。

計算成像與可見光以外成像的綜述

- **Mait、Euliss 與 Athale，〈Computational imaging〉（2018）** [8] 回顧了二十年來光學與計算一起設計的系統：感測器記錄下的資料，要經過處理才會成為認得出來的圖片。這篇綜述涵蓋空間光調變器、合成孔徑雷達與計算顯微術等主題。主要結論是：圖 1.6 中「擷取」與「處理」的分界，如今是一個設計選擇——CT、MRI 與雷達是早期的例子，而手機讓這種做法成為主流。

- **Delbracio、Kelly、Brown 與 Milanfar，〈Mobile computational photography: a tour〉(2021)** [10] 追溯手機相機如何靠演算法克服微小的感測器與鏡頭：連拍多張影格再合併、強力的降噪，以及多影格超解析度。這趟巡禮顯示，現代手機照片其實是一條長長處理流程的輸出，它幾乎可以當作本系列每一章的搭配讀物。
- **Gallego 等人，〈Event-based vision: a survey〉(2020)** [11] 介紹一種完全打破圖 1.1 那種「影格」模型的感測器。事件相機（event camera）的每個像素在自己的亮度改變時就非同步地觸發，提供微秒等級的時間解析度與非常高的動態範圍。這篇綜述說明，當輸入是一串事件而不是一個數字網格時，特徵偵測、光流、影像重建等經典任務都必須重新思考。
- **Wang、Ye 與 De Man，〈Deep learning for tomographic image reconstruction〉(2020)** [12] 回顧類神經網路如何進入 CT、MRI、PET 等成像方式——正是 1970 年代開啟這個領域的那些重建問題。內容涵蓋清理傳統重建結果的網路、把量測模型保留在流程中的物理引導（physics-informed）方法、直接學習的重建，以及生成模型，目標包括更快的掃描、更低的輻射劑量與更好的影像品質。作者的展望強調：大型、經過妥善整理的資料集將是未來進展的關鍵。

深度學習如何重塑處理流程

LeCun、Bengio 與 Hinton（2015）的回顧文章 [13] 概括了這個大轉變：深度網路直接從範例中學習多層次抽象的資料表示，而不再依賴人工設計的特徵。在影像處理中，這個轉變在圖 1.2 的連續光譜各層級上，呈現出不同的樣貌。

在**高階**，改變幾乎是全面的。分類與辨識（第 13 章）如今由學習模型主導；第 12、13 章會介紹在那之前的人工特徵與分類器，它們在資料稀少或重視可解釋性時依然有用。

在**低階**，最清楚的案例是相機自己的處理流程。傳統 ISP 是一串分別調校的階段，把感測器的原始數值變成完成的照片：黑階校正、去馬賽克（demosaicing）、去雜訊、白平衡、色彩校正、色調映射與壓縮。**Ramanath 等人（2005）** [14] 清楚地概述了這條流程與其中的取捨，很適合在讀第 3、5、6 章時放在手邊。

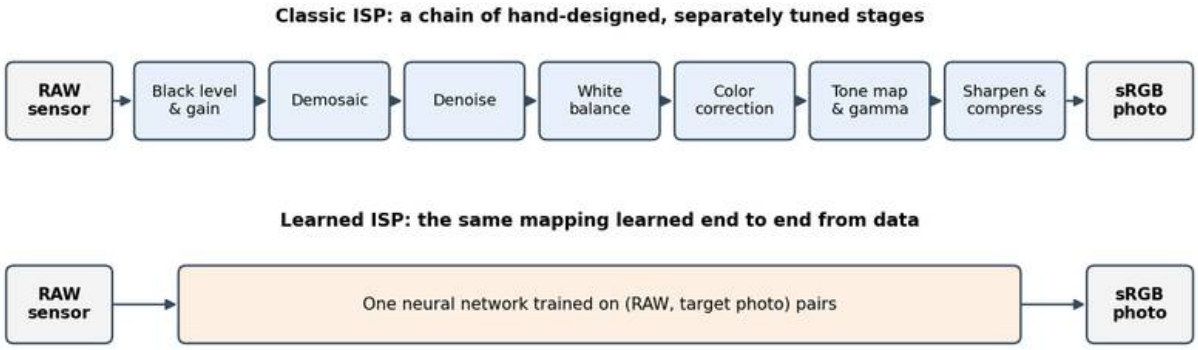


圖 *1.8* — 傳統相機 *ISP*（上）是一串人工設計的階段，每一個都會在本系列某處介紹。學習式 *ISP*（下）則用一個以原始資料與目標照片配對訓練的網路，取代整串或部分階段。

深度學習從幾個方向切入這條流程：

- **端到端學習困難的階段。** Chen 等人（2018）訓練一個全卷積網路，把在極暗環境下以短曝光

拍攝的原始影像，直接映射成乾淨的照片，並以同一場景的長曝光照片作為目標 [15]。傳統流程在這種情況下會嚴重放大雜訊。

- **讓訓練資料更逼真**。 Brooks 等人（2018）提出把一般照片「反處理」（unprocess）的方法：反轉相機流程的各個步驟來合成逼真的原始資料，讓以此訓練的去雜訊網路能用在真實的感測器輸出上 [16]。這是傳統流程知識讓學習模型變得更好的好例子。
- **取代整個 ISP**。 Ignatov、Van Gool 與 Timofte（2020）提出 PyNET，一個金字塔形的卷積網路，把手機感測器的原始資料直接映射成高品質照片，在單一模型中學會去馬賽克、去雜訊、色彩與色調 [17]。他們以手機原始影像與高階單眼相機拍攝同一場景的照片配對來訓練。
- **整體綜述**。 da Silva 等人（2023）綜述了用於影像訊號處理的深度學習方法，比較只取代個別階段的做法與用單一網路取代整個 ISP 的做法 [18]。

有什麼**沒有**改變？第 2、4、5 章的成像數學，依然決定了任何方法——不論是不是學習而來——能還原多少資訊。學習模型的訓練與評估，用的仍是經典章節定義的品質指標、轉換與色彩空間。而且許多實際部署的系統是混合式的：在傳統流程中放進一個學習式去雜訊器，或是以物理為基礎的重建再交給網路精修 [12]。理解經典步驟，才能看出這類系統中哪一部分在做什麼，以及它失敗時為什麼失敗。

重點整理

- 數位影像是在 $M \times N$ 網格上取樣、並量化為 $L = 2^k$ 個等級的函數 $f(x, y)$；在程式中它就是一個陣列。
- 影像處理是一條連續光譜：低階（影像到影像）、中階（影像到屬性）與高階（屬性到意義）。電腦視覺位於高階那一端。
- 這個領域源自電報圖片傳輸（1920 年代）、太空探測器影像的電腦修正（1960 年代），以及 CT 這類由計算產生的醫學影像（1970 年代）。
- 影像來自電磁波譜的每個部分，從伽瑪射線到無線電波，也來自超音波、震波與電子束等非電磁來源。來源的物理特性決定了雜訊與解析度，也就決定了你需要哪些工具。
- 基本步驟分成輸出為影像的步驟（從擷取到壓縮）與輸出為屬性的步驟（從形態學到分類），全部由領域知識引導。第 2–13 章依循這份路線圖。
- 一套可運作的系統結合了感測器、專用硬體、電腦、軟體、儲存、顯示器、硬拷貝與網路。
- 深度學習已大致接管高階辨識，並正在重塑相機 ISP 這類低階步驟；但經典的成像模型依然是學習方法賴以建立的基礎。

練習

1. **儲存預算**。一台顯微鏡以縮時方式拍攝 2048 × 2048、16 位元的灰階影像，每 30 秒一張，連續 48 小時。整個實驗需要多少未壓縮的儲存空間？一顆 1 TB 的硬碟可以放幾張？

   提示

   每張影像是 2048 × 2048 × 16 位元 = 8,388,608 位元組，約 8.4 MB。共有 $48 \times 120 = 5760$ 張，總計約 48 GB。1 TB 硬碟大約可放 $10^{12} / 8.39 \times 10^{6} \approx 119{,}000$ 張。

2. **放到光譜上。**判斷下列工作屬於低階、中階或高階，並說明理由：(a) 去除掃描文件上的椒鹽雜訊；(b) 在顯微影像中描出每個細胞的輪廓並回報每個細胞的面積；(c) 判斷一張胸部 X 光片是否正常；(d) 把一張太暗的照片調亮。

   提示

   看輸出是什麼。輸出影像就是低階（(a)、(d)）。輸出區域或量測值是中階（(b)）。輸出對場景的判斷是高階（(c)）。

3. **為什麼用 X 光而不是可見光？**用 $E = hc/\lambda$ 計算 0.05 nm X 光與 600 nm 橙光的光子能量。用兩句話解釋為什麼前者能用來拍攝骨骼，後者卻不行。

   提示

   約 24.8 keV 對約 2.07 eV，比值大約 12,000。可見光光子在組織中幾公釐內就被吸收或散射；高能的 X 光光子能穿過軟組織，而骨骼吸收的比例較高，因此產生對比。

4. **需要幾個灰階？**使用本章的 `quantize` 函式，畫出 `data.camera()` 與其量化版本之間的平均絕對誤差，$L = 2, 3, \ldots, 32$。接著實際看看這些影像：大約在哪個 L 以後，你不再明顯看到假輪廓？誤差曲線能不能準確預測你看到的結果？

   提示

   均勻量化的誤差大約以 $1/L$ 的速度下降。看得見的色帶主要取決於影像中平滑的區域（這裡是天空），而不是平均誤差——這也是第 2 章把感知品質與數值誤差分開討論的原因之一。

5. **設計一套系統。**一間工廠想在輸送帶上剔除有裂痕的陶瓷磁磚，速度是每秒兩片。列出你需要圖 1.7 中的哪些元件；並針對圖 1.6 的每個基本步驟，說明你是否會使用它以及原因。

   提示

   你需要一個搭配受控照明的感測器、每秒能處理兩張影像的運算能力，以及連到剔除機構的連線。可能用到的步驟：擷取、強化（對比）、或許形態學、分割出像裂痕的結構、特徵（裂痕長度），以及一條分類規則。色彩、小波與壓縮大概用不到，除非影像需要存檔備查。

6. **學習式還是傳統式？**從圖 1.8 的傳統 ISP 中挑一個階段。提出一個用學習模型取代它的理由，以及一個保留人工設計版本的理由。

   提示

   支持學習：網路能適應實際的感測器雜訊與場景統計特性。支持保留：人工設計的階段行為可預測、執行成本低、容易調校，而且不需要配對的訓練資料。現代觀點中的幾篇參考文獻正好討論了這個取捨。

參考文獻

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 1. publisher page
2. K. Kobayashi, "Birth of a Digital Phototelegraph — the Bartlane System," *Journal of the Institute of Image Electronics Engineers of Japan*, vol. 31, no. 2, pp. 244–249, 2002. J-STAGE
3. NASA, "First Image of the Moon Taken by a U.S. Spacecraft" (Ranger 7, PIA02975), NASA Photojournal. NASA
4. G. N. Hounsfield, "Computerized transverse axial scanning (tomography): Part 1. Description of system," *British Journal of Radiology*, vol. 46, no. 552, pp. 1016–1022, 1973. doi:10.1259/0007-1285-46-552-1016
5. L. A. Shepp and B. F. Logan, "The Fourier reconstruction of a head section," *IEEE Transactions on Nuclear Science*, vol. 21, no. 3, pp. 21–43, 1974. doi:10.1109/TNS.1974.6499235
6. P. C. Lauterbur, "Image formation by induced local interactions: examples employing nuclear magnetic resonance," *Nature*, vol. 242, pp. 190–191, 1973. doi:10.1038/242190a0
7. R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed., Springer, 2022. book site
8. J. N. Mait, G. W. Euliss and R. A. Athale, "Computational imaging," *Advances in Optics and Photonics*, vol. 10, no. 2, pp. 409–483, 2018. doi:10.1364/AOP.10.000409
9. The Event Horizon Telescope Collaboration, "First M87 Event Horizon Telescope Results. I. The Shadow of the Supermassive Black Hole," *The Astrophysical Journal Letters*, vol. 875, no. 1, 2019. doi:10.3847/2041-8213/ab0ec7
10. M. Delbracio, D. Kelly, M. S. Brown and P. Milanfar, "Mobile Computational Photography: A Tour," arXiv:2102.09000, 2021. arXiv
11. G. Gallego et al., "Event-based Vision: A Survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020. arXiv
12. G. Wang, J. C. Ye and B. De Man, "Deep learning for tomographic image reconstruction," *Nature Machine Intelligence*, vol. 2, no. 12, pp. 737–748, 2020. doi:10.1038/s42256-020-00273-z
13. Y. LeCun, Y. Bengio and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, 2015. doi:10.1038/nature14539
14. R. Ramanath, W. E. Snyder, Y. Yoo and M. S. Drew, "Color image processing pipeline," *IEEE Signal Processing Magazine*, vol. 22, no. 1, pp. 34–43, 2005. doi:10.1109/MSP.2005.1407713
15. C. Chen, Q. Chen, J. Xu and V. Koltun, "Learning to See in the Dark," *CVPR*, 2018. arXiv
16. T. Brooks, B. Mildenhall, T. Xue, J. Chen, D. Sharlet and J. T. Barron, "Unprocessing Images for Learned Raw Denoising," arXiv:1811.11127, 2018. arXiv
17. A. Ignatov, L. Van Gool and R. Timofte, "Replacing Mobile Camera ISP with a Single Deep Learning Model," arXiv:2002.05509, 2020. arXiv
18. M. H. M. da Silva et al., "ISP meets Deep Learning: A Survey on Deep Learning Methods for Image Signal Processing," arXiv:2305.11994, 2023. arXiv

IP 02・數位影像基礎

眼睛與相機如何把光變成數字、取樣與量化如何決定一張數位影像，以及之後每一章都會用到的基本數學工具。

網站版本: https://blog.shyandram.dev/zh-tw/learn/image-processing/dip-02-fundamentals/

你將學到

- 人眼如何感知光，以及為什麼我們「看到的」不一定是「實際存在的」（亮度適應、Weber 比、馬赫帶、同時對比）
- 影像在物理上從哪裡來：電磁波譜、感測器，以及照度–反射率模型 $f = i \cdot r$
- 取樣與量化在做什麼、它們如何決定空間解析度與強度解析度，以及最近鄰、雙線性、雙三次內插如何縮放影像
- 像素之間關係的術語：鄰居、相鄰、連通、區域、邊界，以及三種距離量測
- 之後每一章都會用到的數學工具箱：逐元素運算與矩陣運算、線性、算術運算、集合與邏輯運算、幾何轉換、影像轉換，以及把直方圖當成機率

先看全貌

第 1 章說明了數位影像處理「是什麼」。這一章要回答「數位影像到底是什麼」：從離開場景的光開始，一路走到記憶體裡的一格格整數。途中我們會定義像素、鄰居、解析度、線性運算子等詞彙——之後整個系列都會直接使用它們，不再另外解釋。

> **用最簡單的話說**
>
> 想像你要用一盒蠟筆，把一幅畫臨摹到方格紙上。首先你得決定方格紙有幾格：格子越多，越能畫出細小的細節，這叫做**取樣**。接著你得決定可以用幾種顏色的蠟筆：蠟筆越多，深淺變化越平滑，這叫做**量化**。一張數位影像就是那張方格紙，每一格裡寫著一個數字，代表你選了哪一支蠟筆。影像處理的其他一切，都是在教你怎麼改變這些數字。

這一章不只是記帳。第一，大多數影像最後是給人看的，所以我們必須知道眼睛在哪裡容易被騙、在哪裡又特別敏銳。第二，拍攝影像時做的每個選擇——取樣多少點、量化成幾階、用哪一種內插——都會留下痕跡，後續的演算法只能與之共處。深度學習並沒有讓這些選擇消失；它多半只是把這些選擇搬進了訓練資料與損失函數裡，「現代觀點」一節會說明這點。

視覺感知的要素

白話說：你的眼睛是一台由膠狀物質和神經細胞組成的相機。它非常擅長看出「差異」與「邊緣」，卻不太會判斷絕對亮度。

眼睛的構造與成像

光線穿過角膜，通過瞳孔（大小由虹膜控制），再由水晶體聚焦到**視網膜**上——那是眼球後方一層薄薄的感光細胞 [1]。水晶體是有彈性的：睫狀肌改變它的曲率，也就改變了焦距，讓遠近物體都能對焦。視網膜上的像是倒立的實像，和相機感測器上的像一樣。

視網膜上有兩種感光細胞 [1]：

- **錐細胞（cones）**——每隻眼睛有數百萬個，最密集地分布在中央一個小凹處，稱為**中央窩（fovea）**。它們在明亮環境下運作（明視覺，photopic vision），分成三種光譜敏感度不同的類型，帶給我們色彩與細節。許多錐細胞擁有自己專屬的神經連結，所以中央窩的視覺特別清晰。
- **桿細胞（rods）**——數量多出許多倍，散布在整個視網膜上，但中央窩正中央沒有。它們在昏暗環境下運作（暗視覺，scotopic vision），無法分辨顏色，而且許多桿細胞共用一條神經末梢，用細節換取靈敏度。這就是為什麼月光下的物體看起來沒有顏色、輪廓也比較柔和。

視神經離開眼球的地方完全沒有感光細胞，稱為**盲點**。你平常察覺不到，是因為大腦會自動把空缺補上——這是一個早期的線索：知覺是一種主動的重建，而不是單純的記錄。

亮度適應與亮度辨別

視覺系統能應付極大範圍的光強度，從星光到陽光都行。它靠的是**亮度適應（brightness adaptation）**：在任一時刻，它只在以目前適應水準為中心的一個窄窗內運作，而這個窗會隨整體光線改變而滑動 [1]。在窗內，主觀亮度大致隨物理光強度的對數成長，而不是線性成長。

我們能察覺多小的變化？讓觀察者看一片強度為 I 的均勻背景，再閃一個強度為 $I + \Delta I$ 的小光點。有一半機率被察覺到的增量記為 ΔI_c。比值

$$W = \frac{\Delta I_c}{I}$$

稱為 **Weber 比（Weber ratio）**，其中 I 是背景強度，ΔI_c 是剛好能察覺的增量。W 越小代表辨別能力越好（很小的相對變化就看得出來）。在很寬的中間強度範圍內，W 大致是常數，這就是 Weber 定律：我們感受到的是相對變化，而不是絕對變化。在低強度、由桿細胞主導時，W 很大，辨別能力很差 [1]。

這有直接的工程意義。因為我們感受的是相對變化，物理上等間隔的灰階在知覺上並不等間隔。這也是影像通常以非線性（gamma）編碼儲存的原因之一，以及 8 位元通常就足以顯示的原因之一。

馬赫帶、同時對比與錯覺

感受到的亮度並不是強度的簡單函數。兩個經典示範讓這件事一目了然 [1]：

- **馬赫帶（Mach bands）**。給人看一組階梯狀、各自均勻的灰色條紋。每一條在物理上都是均勻的，但觀察者會說在每個邊緣的暗側看到一條更暗的細帶、亮側看到一條更亮的細帶。視覺系

統會誇大邊緣，這與相鄰感受野之間的側向抑制（lateral inhibition）相符——本質上就是一個內建的銳化濾波器。

- **同時對比（simultaneous contrast）**。同一塊中灰色方塊，放在暗背景上看起來較亮，放在亮背景上看起來較暗，儘管兩塊方塊完全相同。感受到的亮度取決於周圍環境。

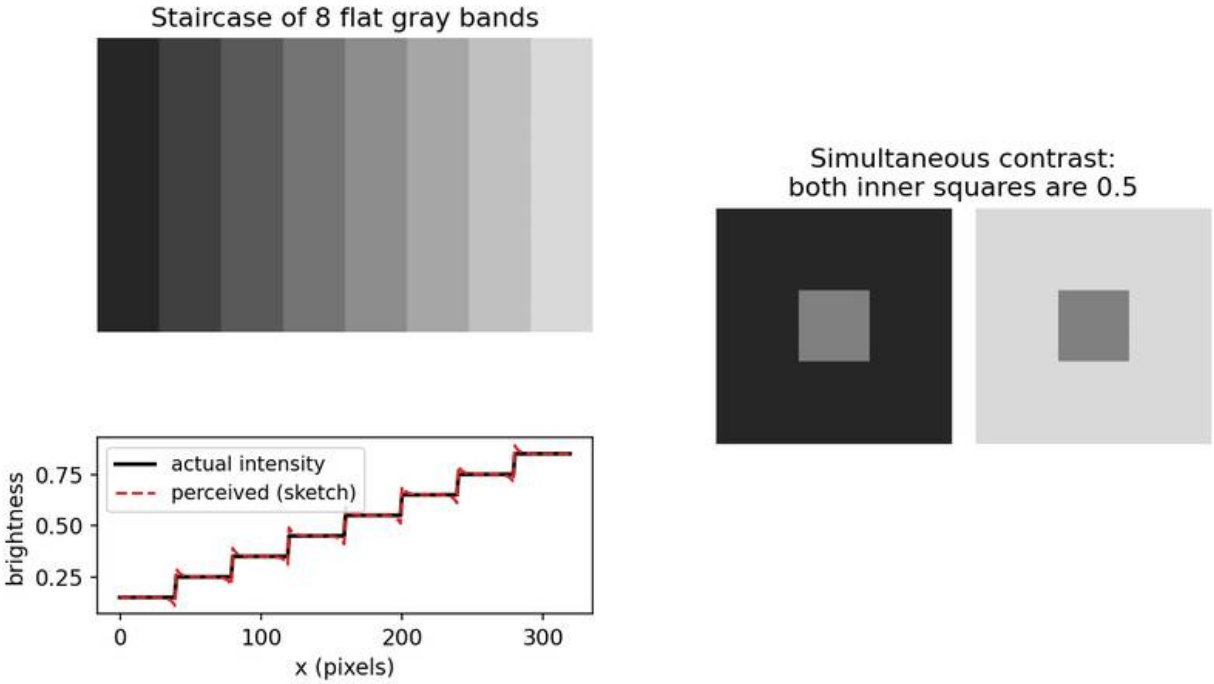


圖 *2.1* — 左：一組均勻灰帶；虛線示意觀察者在每個邊緣感受到的過衝與下衝（馬赫帶）。右：同時對比——兩塊內部方塊的強度都是 *0.5*。

一般的視錯覺——看見沒畫出來的輪廓、等長線段看起來不等長——都顯示眼睛會補足並重新詮釋。這對影像處理有兩層意義：一個把平方誤差降到最低的演算法，產生的影像仍可能看起來不對；而數值上很小的瑕疵，若剛好落在邊緣上，可能非常顯眼。這正是感知品質指標的出發點，會在「現代觀點」中討論。

光與電磁波譜

白話說：光是一種能量波。可見光只是寬廣波譜中薄薄的一片，這個波譜還包括無線電波、微波、紅外線、紫外線、X 射線與伽瑪射線——而這些都可以拿來成像。

波長、頻率與能量

電磁輻射可以描述成正弦波，也可以描述成一串無質量的粒子，稱為光子 [1]。兩種描述的關係是

$$\lambda = \frac{c}{\nu}, \qquad E = h\nu,$$

其中 λ 是波長（公尺），ν 是頻率（赫茲），$c \approx 3 \times 10^8$ m/s 是光速，E 是單一光子的能量（焦耳），h 是普朗克常數。波長越短，頻率越高，每個光子的能量也越高。所以伽瑪射線和 X 射線能穿透組織，而無線電波需要很大的天線才能偵測。

可見光大約從 400 nm（紫）到 700 nm（紅）。物體看起來是什麼顏色，取決於它反射哪些波長：綠葉主要反射波段中段的光，其餘被吸收。沒有顏色、只有強度的光稱為**無色光（achromatic）**或**單色光（monochromatic）**，它的強度稱為**灰階（gray level）**。

輻射度、輝度與亮度

有三個詞很容易混淆 [1]：

- **輻射度（radiance）**——從光源流出的總能量，以瓦特計，是純物理量。
- **輝度（luminance）**——觀察者感受到的能量多少，以流明計。它用眼睛的光譜敏感度來加權輻射度：紅外光源的輻射度可以很高，輝度卻幾乎為零。
- **亮度（brightness）**——對強度的主觀感受。它無法直接量測，而且如上一節所示，會受適應狀態與周圍環境影響。

感測器的敏感度曲線扮演眼睛的角色：影像好不好，取決於感測器偵測的波段與物體發出或反射的波長是否相配。要對一個物體成像，波長還必須與物體尺寸相當或更小——這也是電子顯微鏡（有效波長極短）能解析比光學顯微鏡小得多的結構的原因之一。

影像感測與擷取

白話說：感測器就是一個把光轉成電壓的小小測光表。把很多個排成一列或一個網格，就成了掃描器或相機。

單一、線型與陣列感測器

每一種成像裝置都結合了**照明光源**、**場景**，以及由對目標能量有反應的材料做成的**感測器** [1]。感測器輸出的電壓經數位化後成為一個數字。感測器有三種排列方式：

- **單一感測器**。一個偵測器（例如光二極體）加上兩個方向的機械移動，一次一個取樣點地掃出整張影像。速度慢，但便宜又非常精確，一些高精度掃描器採用這種方式。
- **線型（條狀）感測器**。一排偵測器一次擷取一條線，再靠垂直於感測條的移動提供第二個維度。平台式掃描器和航空推掃式（push-broom）成像儀就是這樣運作。把感測器排成環狀、圍繞一個旋轉的光源，就是電腦斷層掃描（CT）的幾何架構；CT 的影像是重建出來的，而不是直接讀出。
- **陣列感測器**。一個 2-D 的偵測器網格，例如 CCD 或 CMOS 晶片，一次擷取整張影像，這就是數位相機。鏡頭把場景聚焦到陣列上，每個偵測器在曝光時間內累積光量，讀出電路為每個偵測器產生一個值。

彩色相機通常在陣列上覆蓋一層紅、綠、藍濾光片組成的馬賽克，所以每個偵測器只量到一種顏色；每個像素缺少的另外兩個值，會在相機的影像處理管線中以內插補齊（去馬賽克，

demosaicking）[8]。

簡單的成像模型

我們把一張單色影像寫成 2-D 函數 $f(x, y)$，其值正比於抵達 (x, y) 的能量。因為它來自物理能量，所以 $0 \le f(x, y) < \infty$。對反射光而言，把 f 拆成兩個因子很有幫助 [1]：

$$f(x, y) = i(x, y)\, r(x, y), \qquad 0 < i(x, y) < \infty, \qquad 0 < r(x, y) < 1,$$

其中 $i(x, y)$ 是**照度（illumination）**（照到該場景點的光有多少），$r(x, y)$ 是**反射率（reflectance）**（表面反射了其中多少比例，從接近 0 的完全吸收到接近 1 的完全反射）。照度取決於光源；反射率則是物體本身的性質。在 X 射線之類的穿透式成像中，以**穿透率（transmissivity）**取代反射率。

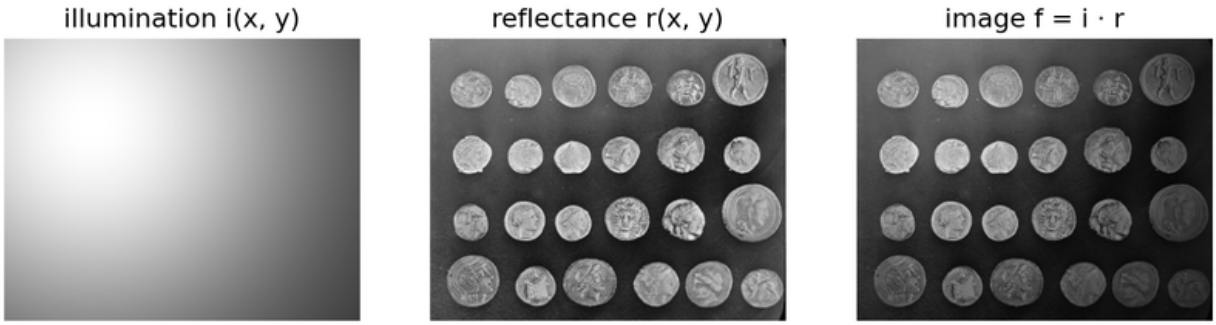


圖 2.2 — 照度–反射率模型。平滑的照度場（左）乘上反射率（中），得到記錄下來的影像（右）。

```
import numpy as np
from skimage import data, img_as_float

r = img_as_float(data.coins())          # 反射率，範圍 [0, 1]
h, w = r.shape
yy, xx = np.mgrid[0:h, 0:w]
i = 0.2 + 0.8 * xx / (w - 1)            # 照度斜坡：左暗右亮
f = i * r                               # 感測器記錄到的影像
```

這個模型很簡單，卻很實用。照度通常在場景中緩慢變化，反射率則在物體邊緣處劇烈改變。後面的章節會利用這一點：同態濾波（homomorphic filtering，第 4 章）取 $\log f = \log i + \log r$ 把兩者分開；陰影校正（本章稍後）則把已知的 i 除掉。

影像取樣與量化

白話說：相機只在有限個位置量測連續的場景（取樣），再把每個量測值四捨五入到有限個階層之一（量化）。

基本概念

感測器的輸出是在連續平面上的連續電壓波形。要變成數位的，座標和振幅都必須離散化 [1]：

- **取樣（sampling）**把座標數位化：我們只保留網格位置上的值。
- **量化（quantization）**把振幅數位化：每個保留的值都對應到有限個階層中最接近的那一個。

對陣列感測器而言，偵測器的數量與間距決定了取樣；類比數位轉換器決定了量化。需要取樣多密，取決於影像內容：根據取樣定理，一個頻寬有限的訊號，只要取樣頻率大於其最高頻率的兩倍，就能完美重建。第 4 章會正式推導；Unser 的回顧論文則提供了以樣條（spline）為基礎、較現代的取樣與重建觀點 [2]。

數位影像的表示法

經過取樣與量化後，影像是一個 M 列、N 行的矩陣：

$$f = \begin{bmatrix} f(0,0) & f(0,1) & \cdots & f(0,N-1) \\ f(1,0) & f(1,1) & \cdots & f(1,N-1) \\ \vdots & \vdots & \ddots & \vdots \\ f(M-1,0) & f(M-1,1) & \cdots & f(M-1,N-1) \end{bmatrix}$$

每個元素是一個**像素（pixel，picture element）**。依照教科書的慣例 [1]，x 是列索引、y 是行索引，原點在左上角。（許多函式庫，包括 OpenCV 的點座標 API，用的是 $(x, y) =$（行, 列）。務必確認。）

強度階數通常是 2 的冪次 $L = 2^k$，所以值落在 $[0, L-1]$。儲存一張影像需要

$$b = M \times N \times k \text{ 位元},$$

其中 k 是每像素的位元數。一張 512×512、$k = 8$ 的影像需要 262,144 位元組。成像系統的**動態範圍（dynamic range）**是可量測的最大強度（受飽和限制）與最小強度（受雜訊限制）之比；某一張影像的**對比（contrast）**則是其最高與最低強度之差。

空間解析度與強度解析度

空間解析度（spatial resolution）是能辨識的最小細節尺寸，必須帶單位才有意義：每毫米線對數，或印刷時的每英寸點數（dpi）。只說影像是「1024 × 1024」，在不知道它涵蓋多大實際面積前，並沒有說出任何解析度資訊。**強度解析度（intensity resolution）**是能辨識的最小階層變化，通常以位元數 k 表示。

```
import numpy as np
from skimage import data

img = data.camera()                          # 512 x 512，uint8（k = 8）
small = img[::4, ::4]                        # 每 4 個取樣點保留一個 -> 128 x 128

def quantize(x, bits):
```

```
    step = 2 ** (8 - bits)
    return (x // step) * step + step // 2 # 對應到每個區間的中心

q3 = quantize(img, 3)                      # 8 個灰階
print(small.shape, np.unique(q3).size)     # (128, 128) 8
```

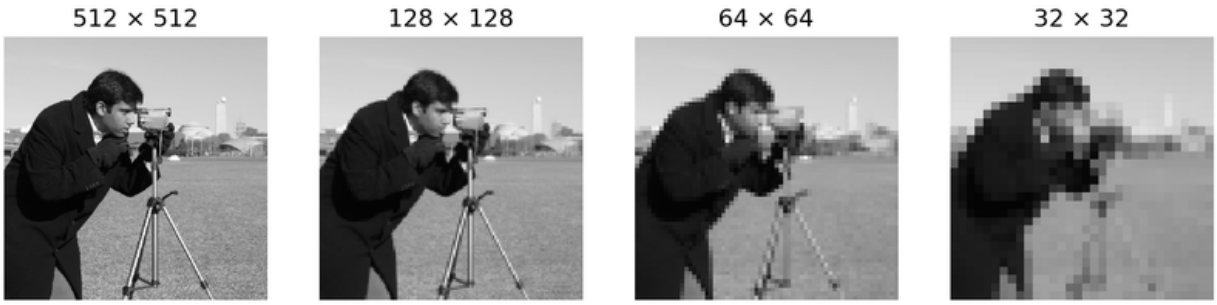


圖 *2.3* — 降低空間解析度。每個版本都以相同大小顯示，因此取樣點越少，像素看起來就越大、越像方塊。

圖 *2.4* — 降低強度解析度。大約在 *8* 階時，天空這類平滑區域開始出現假輪廓；只剩 *2* 階時，只看得到剪影。

由此得到兩個教訓。減少取樣點時，首先被破壞的是細緻紋理（草、頭髮），接著才是邊緣。減少階層時，首先受損的是平滑區域：圖 2.4 的天空會碎成一階一階，稱為**假輪廓（false contouring）**——恰好出現在 Weber 比說眼睛最敏感的地方。細節多的影像比平滑的影像更能容忍較少的灰階，因為繁雜的紋理會把階梯掩蓋掉 [1]。

> **互動示範（網站版）**：請開啟本篇教學的網站版本，操作此示範。開啟示範

影像內插

內插（interpolation）是在沒有取樣值的位置估計數值。每當影像被縮放、旋轉、變形或對位時，都會用到它 [1]、[3]、[4]。

最近鄰（nearest neighbor）直接複製最近取樣點的值。它快速、保留原始數值，但會讓邊緣呈鋸齒方塊；縮小影像時，甚至可能讓細線整條消失。

雙線性（bilinear）內插使用最近的四個取樣點。對於距左上取樣點 f_{00} 的小數偏移量為 $a, b \in [0, 1)$ 的位置，

$$v = (1-a)(1-b)f_{00} + a(1-b)f_{01} + (1-a)bf_{10} + abf_{11},$$

其中 f_{01}、f_{10}、f_{11} 分別是右、下、右下的鄰居。等價地，$v(x,y) = c_1 x + c_2 y + c_3 xy + c_4$，四個係數由四個取樣點決定。結果是連續的，但略顯模糊。

```
import numpy as np

def bilinear(img, x, y):
    """ 在實數座標（行 x、列 y）處對 img (H, W) 取樣。"""
    x0, y0 = int(np.floor(x)), int(np.floor(y))
    x1, y1 = min(x0 + 1, img.shape[1] - 1), min(y0 + 1, img.shape[0] - 1)
    a, b = x - x0, y - y0
    top = (1 - a) * img[y0, x0] + a * img[y0, x1]
    bot = (1 - a) * img[y1, x0] + a * img[y1, x1]
    return (1 - b) * top + b * bot

print(bilinear(np.array([[0., 10.], [20., 30.]]), 0.5, 0.5))   # 15.0
```

雙三次（bicubic）內插使用最近的 16 個取樣點，擬合 $v(x,y) = \sum_{i=0}^{3}\sum_{j=0}^{3} a_{ij} x^i y^j$。實務上，它以分段三次核的可分離卷積實作。Keys 提出、廣為使用的核是 [3]

$$u(s) = \begin{cases} (\alpha+2)|s|^3 - (\alpha+3)|s|^2 + 1, & |s| \le 1, \\ \alpha|s|^3 - 5\alpha|s|^2 + 8\alpha|s| - 4\alpha, & 1 < |s| < 2, \\ 0, & \text{其他}, \end{cases}$$

其中 s 是內插位置到某取樣點的距離（以取樣間距為單位），α 是自由參數。Keys 證明在這一族核中，$\alpha = -\tfrac{1}{2}$ 有最好的逼近階數；不過各函式庫實際採用的值不盡相同。u 的負瓣會讓邊緣更銳利，但也可能在強邊緣旁產生輕微的過衝（振鈴，ringing）。

```
import cv2
from skimage import data

crop = data.camera()[150:214, 230:294]    # 64 x 64
up = {name: cv2.resize(crop, None, fx=4, fy=4, interpolation=flag)
      for name, flag in [("nearest", cv2.INTER_NEAREST),
                         ("bilinear", cv2.INTER_LINEAR),
                         ("bicubic", cv2.INTER_CUBIC)]}
```

圖 2.5 — 同一裁切區放大四倍。最近鄰呈方塊狀；雙線性平滑但偏軟；雙三次讓邊緣稍微清晰一些。

拖動滑桿，比較把 96 × 96 裁切區放大成 384 × 384 時，最近鄰與雙三次的差別。注意相機的邊緣與臉部輪廓。

最近鄰

雙三次

攝影師裁切區以最近鄰與雙三次內插放大四倍的比較

沒有任何內插方法能創造出取樣時已丟掉的細節。Thévenaz、Blu 與 Unser 的回顧 [4] 把這點說得很精確：內插器的品質取決於它的逼近階數，以及它的核有多接近理想（sinc）重建器；在相近的計算成本下，較高階的 B-spline 方法優於傳統的三次卷積。「現代觀點」中討論的學習式超解析度（super-resolution）更進一步，會根據訓練資料腦補出看似合理的細節。

像素之間的基本關係

白話說：在談論影像裡的物體之前，我們需要規定哪些像素彼此「相碰」，以及兩個像素相距多遠。

像素的鄰居

位於 (x, y) 的像素 p 有四個水平與垂直鄰居，

$$N_4(p) = \{(x+1, y),\ (x-1, y),\ (x, y+1),\ (x, y-1)\},$$

四個對角鄰居，

$$N_D(p) = \{(x+1, y+1),\ (x+1, y-1),\ (x-1, y+1),\ (x-1, y-1)\},$$

以及八個鄰居 $N_8(p) = N_4(p) \cup N_D(p)$ [1]。在影像邊界上，有些鄰居會落在影像外；要怎麼處理（忽略、補值、鏡射）是一個選擇，在濾波時會再遇到。

相鄰、連通、區域與邊界

令 V 為被視為「相似」的強度值集合——二值影像中 $V = \{1\}$；灰階影像中可能是 100 到 120 之間的所有值。兩個值都在 V 中的像素 p 與 q [1]：

- 若 $q \in N_4(p)$，則為 **4-相鄰（4-adjacent）**；
- 若 $q \in N_8(p)$，則為 **8-相鄰（8-adjacent）**；
- 若 $q \in N_4(p)$，或者 $q \in N_D(p)$ 且集合 $N_4(p) \cap N_4(q)$ 中沒有任何值屬於 V 的像素，則為 **m-相鄰（m-adjacent，混合相鄰）**。

為什麼需要 m-相鄰？在 8-相鄰下，一步對角線和兩步「繞過轉角」的路線可能同時連接同一對像素，使路徑產生歧義。m-相鄰只在不存在 4-連通路線時才保留對角連結，消除了多餘的迴圈（圖 2.6 右）。

圖 2.6 — 像素 p 的鄰域，以及二值區塊上的 m-相鄰：虛線對角不構成 m-相鄰，因為這兩個像素已經共享一個屬於 V 的 4-鄰居。

從 p 到 q 的**數位路徑（digital path）**是一串互不相同的像素 $(x_0, y_0), \ldots, (x_n, y_n)$，相鄰兩個像素彼此相鄰；$n$ 是路徑長度，若第一個與最後一個像素相同，則稱為**封閉路徑**。依所用的相鄰定義，我們稱之為 4-、8- 或 m-路徑。對於像素子集 S，若 p 與 q 之間存在一條完全位於 S 中的路徑，就說它們**在 S 中連通**。在 S 中與 p 連通的所有像素構成一個**連通分量（connected component）**；若 S 只有一個連通分量，則 S 是**連通集合**。

連通集合稱為**區域（region）**。若兩個區域的聯集是連通的，就說它們相鄰。區域 R 的**邊界（boundary，或 border）**是 R 中至少有一個鄰居位於 R 外的像素集合。注意邊界與**邊緣（edge）**的差別：邊界是區域整體、封閉的性質；邊緣則是局部的強度不連續，未必會閉合（第 10 章）。

相鄰的選擇會改變「有幾個物體？」這類簡單問題的答案：

```
import numpy as np
from skimage.measure import label

b = np.array([[1, 0, 0],
              [0, 1, 0],
```

```
              [0, 0, 1]])
print(label(b, connectivity=1).max(),     # 4-連通：3 個物體
      label(b, connectivity=2).max())     # 8-連通：1 個物體
```

一個經典結論：如果前景用 8-連通，背景就應該用 4-連通（反之亦然），否則一條封閉的數位曲線可能無法把內外分開。這是數位拓樸（digital topology）的基本結果 [1]。

距離量測

對像素 $p=(x,y)$、$q=(u,v)$ 與 z，若函數 D 滿足 $D(p,q)\ge 0$ 且僅在 $p=q$ 時等號成立、$D(p,q)=D(q,p)$，以及 $D(p,z)\le D(p,q)+D(q,z)$，則 D 是一個**距離度量（distance metric）**。三種標準度量是 [1]、[5]：

$$
\begin{aligned}
D_E(p,q) &= \sqrt{(x-u)^2+(y-v)^2} && \text{(Euclidean)}\\
D_4(p,q) &= |x-u|+|y-v| && \text{(city-block)}\\
D_8(p,q) &= \max(|x-u|,\,|y-v|) && \text{(chessboard)}
\end{aligned}
$$

即歐氏距離、城市街區距離與棋盤距離。與 p 的 D_E 不超過某定值的像素構成圓盤；D_4 構成菱形；D_8 構成正方形。$D_4=1$ 的像素恰好是 $N_4(p)$，$D_8=1$ 的像素恰好是 $N_8(p)$。D_4 與 D_8 只取決於座標，與像素值無關；反之，沿著 m-路徑量的距離會受像素值影響，因為路徑本身受像素值影響。

```
import numpy as np
p, q = np.array([2, 3]), np.array([5, 7])
d = np.abs(p - q)
print("D_E =", np.hypot(*d), " D_4 =", d.sum(), " D_8 =", d.max())   # 5.0, 7, 4
```

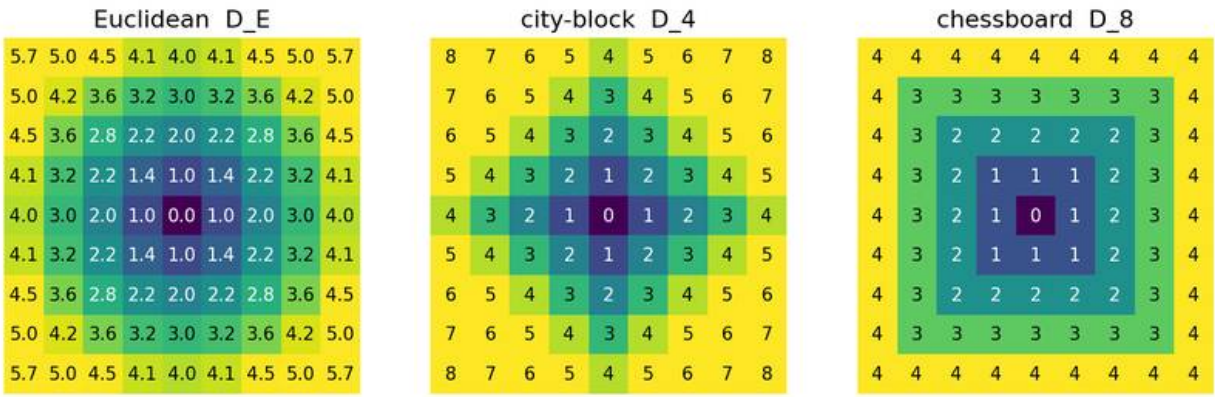


圖 2.7 — 三種度量下與中心像素的距離。等距輪廓分別是圓形、菱形與正方形。

Rosenfeld 與 Pfaltz [5] 提出了高效率的兩趟（two-pass）演算法，一次算出每個像素到最近物體像素的距離——也就是**距離轉換（distance transform）**，至今仍用於骨架化、形態學與形狀比對（第 9 章）。

數位影像處理的數學工具簡介

白話說：影像就是矩陣，所以大部分影像處理都是對矩陣做運算。本節列出我們會一再用到的運算，以及決定哪些工具適用的關鍵性質——線性。

逐元素運算與矩陣運算

逐元素（elementwise，又稱陣列）運算逐一像素作用。兩張 2×2 影像的逐元素乘積為

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \odot \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} & a_{12}b_{12} \\ a_{21}b_{21} & a_{22}b_{22} \end{bmatrix},$$

而矩陣乘積則遵守列乘行的規則。除非特別說明，影像運算——包括「兩張影像相乘」——都是逐元素的 [1]。在 NumPy 中，`A * B` 是逐元素乘積，`A @ B` 是矩陣乘積。

線性與非線性運算

考慮一個把輸入影像 f 對應到輸出 $g = \mathcal{H}[f]$ 的運算子 $\mathcal{H}$。若對所有影像 f_1, f_2 與所有常數 a, b 都有

$$\mathcal{H}[af_1 + bf_2] = a\,\mathcal{H}[f_1] + b\,\mathcal{H}[f_2]$$

則它是**線性**的。這個性質的兩部分是**可加性**（和的輸出等於輸出的和）與**齊次性**（輸入縮放，輸出也同比例縮放）。把所有像素加總是線性的；取最大值則不是：

```
import numpy as np
rng = np.random.default_rng(0)
f1, f2 = rng.random((4, 4)), rng.random((4, 4))
a, b = 2.0, -3.0
print(np.isclose(np.sum(a*f1 + b*f2), a*np.sum(f1) + b*np.sum(f2)))   # True
print(np.isclose(np.max(a*f1 + b*f2), a*np.max(f1) + b*np.max(f2)))   # False
```

線性之所以重要，是因為線性運算子完全由它對簡單輸入的反應決定，而且有大量理論（卷積、傅立葉轉換，第 3–5 章）可以套用。中值濾波器或形態學等非線性運算子能做到線性運算子做不到的事，但較難分析。

算術運算

兩張同尺寸影像的和、差、積、商都是逐元素運算 [1]。其中三種用途特別重要。

平均多張含雜訊的影像。假設我們對一個靜止場景拍了 K 張影像，$g_i(x,y) = f(x,y) + \eta_i(x,y)$，其中 f 是乾淨影像，η_i 是各張之間互不相關、平均為零、變異數為 σ_η^2 的雜訊。平均值 $\bar{g} = \frac{1}{K}\sum_{i=1}^{K} g_i$ 滿足

$$E\{\bar{g}(x,y)\} = f(x,y), \qquad \sigma_{\bar{g}}^2(x,y) = \frac{1}{K}\,\sigma_\eta^2(x,y),$$

所以雜訊標準差以 $1/\sqrt{K}$ 的速度下降。天文學家疊加多次曝光就是這個道理；手機的「夜間模式」在對齊一連串連拍畫面後，做的也是同一件事 [7]。

```
import numpy as np
from skimage import data, img_as_float

g = img_as_float(data.camera())
rng = np.random.default_rng(1)
for K in (1, 4, 16, 64):
    avg = np.mean([g + rng.normal(0, 0.2, g.shape) for _ in range(K)], axis=0)
    print(K, round(float(np.std(avg - g)), 3))    # 0.2, 0.1, 0.05, 0.025
```

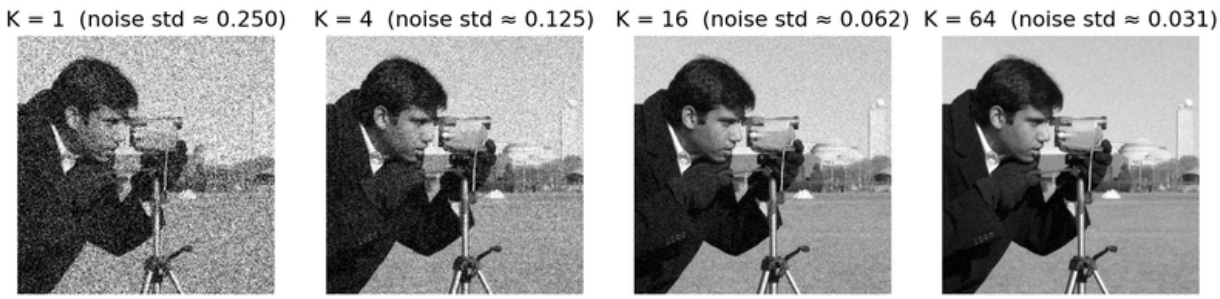


圖 *2.8* — 平均 K 張彼此獨立的含雜訊畫面。K 每乘以 *4*，雜訊標準差大約減半。

相減能凸顯差異。把視訊的兩張畫面相減，會凸顯移動；在醫學影像中，把顯影前的「遮罩」影像從顯影後的影像中減去，主要只會留下吸收了顯影劑的血管。

相乘與相除處理陰影與遮罩。若已知感測器的陰影型態 $h(x,y)$（例如拍攝均勻目標取得），觀察到的影像 $g = f \cdot h$ 可以用 $f = g/h$ 校正。乘上二值遮罩則可保留感興趣區域、把其餘歸零。

一條實用規則：算術結果常常超出有效範圍（相減可能為負，相加可能超過 255）。請用浮點數運算，再重新縮放。縮放到 $[0, K]$ 的簡單方法是先算 $f_m = f - \min f$，再算 $f_s = K f_m / \max f_m$。

集合與邏輯運算

把二值影像看成像素座標的**集合** A（前景），一般的集合運算就都適用 [1]：聯集 $A \cup B$、交集 $A \cap B$、補集 $A^c = \{w \mid w \notin A\}$，以及差集 $A - B = A \cap B^c$。在二值陣列上，它們分別對應邏輯運算 OR、AND、NOT 與 AND-NOT。對灰階影像，聯集與交集通常定義為逐元素的最大值與最小值。模糊集合（fuzzy sets）更進一步，讓隸屬度可以介於 0 與 1 之間。

```
import numpy as np
from skimage import data

coins = data.coins()
A = coins > 100                                          # 集合 A：亮像素
B = np.zeros_like(A); B[:, : coins.shape[1] // 2] = True   # 集合 B：左半邊
union, inter, diff = A | B, A & B, A & ~B
```

集合運算是數學形態學（第 9 章）以及合併分割遮罩時的共通語言。

空間運算

空間運算直接作用在像素值與位置上，分為三類 [1]。

單像素運算各自獨立地改變每個值：$s = T(z)$，其中 z 是輸入強度、s 是輸出。影像負片 $s = (L-1) - z$ 就是一例。這些就是第 3 章的強度轉換。

鄰域運算由 (x, y) 周圍的鄰域 S_{xy} 計算每個輸出。局部平均

$$g(x,y) = \frac{1}{mn} \sum_{(r,c) \in S_{xy}} f(r,c),$$

（其中 S_{xy} 是以 (x, y) 為中心的 $m \times n$ 視窗）是最簡單的例子；第 3 章會把它推廣成空間濾波。

幾何轉換移動像素。它分兩步：座標的空間轉換，以及用強度內插填滿新網格。最常見的一族是**仿射（affine）**轉換。以齊次座標表示，

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix},$$

其中 (x, y) 是輸入位置、(x', y') 是輸出位置，a_{ij} 編碼旋轉、縮放與剪切，(t_x, t_y) 是平移。仿射轉換保持直線與平行性，串接多個轉換就等於把矩陣相乘。

實作上幾乎都用**反向映射（inverse mapping）**：對每個輸出像素，套用反轉換找出它在輸入中的來源位置，再在該處內插。正向映射（把每個輸入像素推到輸出）會留下空洞與碰撞。

```
import cv2
from skimage import data

img = data.camera()
h, w = img.shape
M = cv2.getRotationMatrix2D(center=(w / 2, h / 2), angle=30, scale=0.8)  # 2x3 仿射矩陣
rot = cv2.warpAffine(img, M, (w, h), flags=cv2.INTER_LINEAR)  # 內部使用反向映射
```

影像對位

影像對位（image registration）是把同一場景在不同時間、不同視角或以不同感測器拍攝的影像對齊。此時轉換是未知的，必須估計。經典做法是挑選在兩張影像中位置都已知的**連結點（tie points，又稱控制點）**，再用它們擬合一個模型；例如雙線性模型

$$x = c_1 v + c_2 w + c_3 v w + c_4, \qquad y = c_5 v + c_6 w + c_7 v w + c_8,$$

把輸入座標 (v, w) 對應到參考座標 (x, y)，四組連結點提供八個方程式，解出八個未知數 $c_1, \dots, c_8$。Zitová 與 Flusser 的綜述 [6] 把對位整理成四個步驟，至今仍是標準描述：特徵偵測、特徵匹配、轉換模型估計，以及重新取樣。

向量與矩陣運算

彩色像素是一個向量 $\mathbf{z} = [z_1, z_2, z_3]^T$，分量是紅、綠、藍值。向量工具因此可直接套用；例如 $\mathbf{z}$ 到參考色 $\mathbf{a}$ 的歐氏距離

$$D(\mathbf{z}, \mathbf{a}) = \left[(\mathbf{z} - \mathbf{a})^T (\mathbf{z} - \mathbf{a}) \right]^{1/2},$$

是簡單彩色分割（第 6 章）的基礎。把 $M \times N$ 影像堆疊成 $MN \times 1$ 的向量，也能把任何線性運算寫成 $\mathbf{g} = \mathbf{H}\mathbf{f}$，這個形式會在影像復原（第 5 章）中用到。

影像轉換

有些問題換到另一個域會比較好解。2-D 線性轉換的一般形式為 [1]

$$T(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y)\, r(x, y, u, v),$$

其中 f 是輸入影像，r 是**正向轉換核**，(u, v) 是轉換域的變數。反轉換使用反向核 $s(x, y, u, v)$。當 $r(x, y, u, v) = r_1(x, u)\, r_2(y, v)$ 時，核是**可分離的**，2-D 轉換可以先沿列、再沿行做 1-D 轉換，或寫成矩陣形式 $\mathbf{T} = \mathbf{AFB}$。傅立葉轉換（第 4 章）與小波轉換（第 7 章）是你最常遇到的兩種。

機率方法

強度可以當成隨機量來處理。若 $M \times N$ 影像中有 n_k 個像素的強度為 z_k，則該強度的機率估計為

$$p(z_k) = \frac{n_k}{MN}, \qquad \sum_{k=0}^{L-1} p(z_k) = 1,$$

這就是**正規化直方圖（normalized histogram）**。強度的平均值與變異數為

$$m = \sum_{k=0}^{L-1} z_k\, p(z_k), \qquad \sigma^2 = \sum_{k=0}^{L-1} (z_k - m)^2\, p(z_k),$$

其中 m 衡量平均亮度，σ（標準差）衡量對比。更高階的動差描述偏斜與尖峰程度。

```
import numpy as np
from skimage import data

img = data.camera()
p = np.bincount(img.ravel(), minlength=256) / img.size   # p(z_k)
z = np.arange(256)
m = np.sum(z * p)
sigma = np.sqrt(np.sum((z - m) ** 2 * p))
print(round(m, 2), round(sigma, 2))                      # 129.06 73.64
```

直方圖處理（第 3 章）與雜訊建模（第 5 章）都從這裡出發。

現代觀點

本章的基礎概念並沒有被深度學習取代，反而成了深度學習的介面。網路吃的是經過取樣與量化的陣列，用編碼了某種知覺模型的損失函數訓練，並在相機管線產生的資料上評估。以下幾個方向的回顧文獻勾勒出這片領域。

取樣與內插理論。Unser 的〈Sampling—50 years after Shannon〉[2] 把取樣重新定位為在平移不變空間中的逼近：不再要求訊號完美頻寬有限、也不再要求理想的 sinc 重建器，而是選擇一個實用的基底（例如 B-spline）並投影上去。它的主要結論是：取樣前的良好前置濾波與取樣後的良好重建核，比精確的取樣率更重要。Thévenaz、Blu 與 Unser [4] 把這套觀點用在影像內插上，在相同條件下比較許多核。他們指出，逼近階數與核的支撐範圍可以解釋大部分差異，而且以 B-spline 為基礎的內插（搭配適當的前置濾波）比 Keys 的三次卷積 [3] 有更好的品質與成本取捨。任何要實作影像縮放的人，這兩篇仍是參考基準。

學習式超解析度。Wang、Chen 與 Hoi [9] 整理了截至約 2019 年的深度超解析度：網路設計、放在前段或後段的上取樣層、損失函數與基準測試。Moser 等人 [10] 把圖像延伸到 transformer 與擴散模型（diffusion models），並強調尚未解決的問題，例如彈性的放大倍率與更好的評估方式。有兩篇工作展示了古典概念如何在網路中延續。LIIF [11] 把影像表示成可在任意座標查詢的連續函數——一個可以任意倍率放大（包括訓練時沒見過的倍率）的學習式內插器。Real-ESRGAN [12] 處理的是「大多數訓練資料對用雙三次縮小產生」與「真實世界退化」之間的落差，做法是以高階退化模型合成訓練資料，將模糊、雜訊、縮放與壓縮重複施加多次。本章的教訓是：雙三次內插既是每個方法都要比較的基準，也透過它在產生訓練資料中的角色，成了一個可能讓模型在真實影像上失效的隱藏假設。

知覺與影像品質。Wang 等人的 SSIM [13] 把感知一節的觀察——對結構與相對變化敏感，而非絕對誤差——轉成一個全參考品質指標，比較局部的亮度、對比與結構。Zhai 與 Min [14] 綜述了整個感知品質評估領域：全參考、部分參考與無參考方法，以及針對特殊內容的模型。深度特徵大幅改變了這個領域。Zhang 等人 [15] 收集了人類的相似度判斷，發現深度網路特徵之間的距離（LPIPS）與人的判斷吻合程度遠勝 PSNR 或 SSIM，而且在不同架構與訓練方式下都成立。Ding 等人 [16] 接著提出更尖銳的問題：如果拿每個品質指標當損失函數來最佳化影像處理網路，哪個指標產生的結果最受人喜愛？他們的主觀實驗顯示，在標準品質資料集上的排名，並不能可靠預測一個指標作為訓練目標時是否好用。簡單說，馬赫帶與 Weber 比如今活在損失函數裡。

相機感測器管線。Ramanath 等人 [8] 描述了位於本章感測器陣列與你存下的壓縮檔之間的傳統機內管線：去馬賽克、白平衡、色彩轉換、gamma 與壓縮，每一級都是固定的人工設計。Delbracio 等人 [7] 回顧了行動計算攝影在二十年間如何重塑這條管線：連拍擷取、對齊與合併（也就是大規模的「平均含雜訊影像」）、多張畫面的超解析度。如今已有學習式方法取代其中一部分甚至全部。Chen 等人 [17] 訓練網路，在極低光源下把原始短曝光感測器資料直接轉成乾淨影像，繞過傳統管線。Ignatov 等人 [18] 則用單一網路（PyNET）取代整個手機影像訊號處理器（ISP），把原始馬賽克感測器資料轉成近似單眼相機拍出的影像。

什麼沒有改變？像素網格、位元深度、相鄰與連通（至今仍是每個分割後處理步驟中連通分量標記的基礎）、仿射變形（如今成了網路中可微分的層），以及直方圖。正因為懂這些，你才能察覺模型的輸入是否用錯了縮放核、量化得太粗，或是用了一個與人類判斷不符的指標來比較。

重點整理

- 人類視覺感受的是相對變化（Weber 定律），並會誇大邊緣（馬赫帶）；感受到的亮度取決於周圍。好的影像處理會尊重這些事實。
- 影像可以由電磁波譜的任何部分形成；對反射光而言，$f = i \cdot r$ 把照度與反射率分開。
- 取樣決定空間解析度、量化決定強度解析度，儲存量是 MNk 位元。階數太少會在平滑區域造成假輪廓。
- 內插——最近鄰、雙線性、雙三次——估計取樣點之間的值，在速度、平滑度與銳利度之間取捨，但無法找回已遺失的細節。
- 4-、8- 與 m-相鄰定義了路徑、連通分量、區域與邊界；D_4 與 D_8 是距離度量，其單位球分別是菱形與正方形。
- 除非特別說明，影像運算都是逐元素的；線性決定了卷積與傅立葉工具是否適用。
- 平均 K 張獨立的含雜訊畫面，可讓雜訊標準差降為 $1/\sqrt{K}$；幾何轉換使用反向映射加上內插。
- 深度學習建立在這些基礎上：學習式內插器、感知損失與學習式相機管線，都承襲了本章的概念。

練習

1. 一台灰階掃描器產生 2400 × 3300、每像素 12 位元的影像。一張未壓縮的掃描需要多少 MB？若降為 8 位元呢？

   提示

   用 $b = MNk$，再除以 8×10^6（若要 MiB 則除以 8×2^{20}）。12 位元：$2400 \cdot 3300 \cdot 12/8 = 11.88$ MB。8 位元：7.92 MB。

2. 把 `skimage.data.moon()` 量化成 4、5、6 位元。對你而言，在哪個位元數時假輪廓消失？接著在以 4 位元量化之前，先加入少量均勻隨機雜訊（振幅為半個量化間距）。有什麼改變？為什麼？

   提示

   這種雜訊（稱為抖動，dither）會打散大片平坦的階梯，眼睛會把細小的雜訊平均掉，而不是看到輪廓。平均誤差差不多，但可見的結構改變了——再次提醒我們，平方誤差與感受到的品質是兩回事。

3. 在下面的二值影像中，分別以 4-、8- 與 m-連通計算 1 像素的連通分量數。再分別以 4- 與 8-連通計算 0 像素的連通分量數。哪一種搭配能給出一致的「內／外」答案？

```
0 0 0 0 0
0 1 1 1 0
0 1 0 1 0
0 1 1 0 0
0 0 0 0 0
```

提示

在三種相鄰定義下，1 都只構成一個連通分量；但因為右下角有個對角缺口，只有允許對角步時，這個環才是封閉的。中央的 0 在 4-連通下是孤立的（一個洞），在 8-連通下則與外部相連。4/4 搭配得到「不封閉的曲線卻有洞」；8/8 得到「封閉的曲線卻沒有洞」。只有前景 8-連通搭配背景 4-連通（或反過來）才一致。可用 `skimage.measure.label` 驗證。

4. 證明對任意兩個像素都有 $D_8(p,q) \le D_E(p,q) \le D_4(p,q)$，並找出讓每個不等式取等號的像素對。

提示

令 $d_x = |x-u|$、$d_y = |y-v|$，則 $\max(d_x,d_y)^2 \le d_x^2 + d_y^2 \le (d_x+d_y)^2$。當 d_x, d_y 其中之一為零時取等號。

5. 寫一個用正向映射旋轉影像 θ 角的函式（把每個輸入像素推到四捨五入後的輸出位置）。顯示 $\theta = 17^\circ$ 的結果，並解釋為什麼會有空洞。然後改用反向映射加雙線性內插修正它。

提示

旋轉會讓某些方向上的網格間距略微拉開，所以有些輸出像素收不到任何輸入像素。在反向映射中，每個輸出像素都會問「我是從哪裡來的？」，而且一定得到答案。

6. 你有一個靜止場景的 10 張含雜訊畫面，雜訊標準差為 12 個灰階。要讓它低於 2 個灰階，需要幾張？在真實的手持連拍中，哪個假設可能不成立？手機管線怎麼處理？

提示

$12/\sqrt{K} < 2$ 需要 $K > 36$，所以要 37 張。「場景與相機都靜止」這個假設不成立；平均前必須先對齊（對位）各畫面，移動中的物體則需要穩健的合併方式 [7]。

參考文獻

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 2. publisher page
2. M. Unser, “Sampling—50 years after Shannon,” *Proceedings of the IEEE*, vol. 88, no. 4, pp. 569–587, 2000. doi
3. R. Keys, “Cubic convolution interpolation for digital image processing,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 29, no. 6, pp. 1153–1160, 1981. doi
4. P. Thévenaz, T. Blu, and M. Unser, “Interpolation revisited,” *IEEE Transactions on Medical Imaging*, vol. 19, no. 7, pp. 739–758, 2000. doi
5. A. Rosenfeld and J. L. Pfaltz, “Distance functions on digital pictures,” *Pattern Recognition*, vol. 1, pp. 33–61, 1968. doi
6. B. Zitová and J. Flusser, “Image registration methods: a survey,” *Image and Vision Computing*, vol. 21, no. 11, pp. 977–1000, 2003. doi

7. M. Delbracio, D. Kelly, M. S. Brown, and P. Milanfar, "Mobile computational photography: A tour," arXiv:2102.09000, 2021. arXiv
8. R. Ramanath, W. E. Snyder, Y. Yoo, and M. S. Drew, "Color image processing pipeline," *IEEE Signal Processing Magazine*, vol. 22, no. 1, pp. 34–43, 2005. doi
9. Z. Wang, J. Chen, and S. C. H. Hoi, "Deep learning for image super-resolution: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020. arXiv
10. B. Moser, F. Raue, S. Frolov, J. Hees, S. Palacio, and A. Dengel, "Hitchhiker's guide to super-resolution: Introduction and recent advances," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023. arXiv
11. Y. Chen, S. Liu, and X. Wang, "Learning continuous image representation with local implicit image function," CVPR, 2021. arXiv
12. X. Wang, L. Xie, C. Dong, and Y. Shan, "Real-ESRGAN: Training real-world blind super-resolution with pure synthetic data," arXiv:2107.10833, 2021. arXiv
13. Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004. project page
14. G. Zhai and X. Min, "Perceptual image quality assessment: A survey," *Science China Information Sciences*, vol. 63, no. 11, 2020. doi
15. R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," CVPR, 2018. arXiv
16. K. Ding, K. Ma, S. Wang, and E. P. Simoncelli, "Comparison of full-reference image quality models for optimization of image processing systems," *International Journal of Computer Vision*, 2021. arXiv
17. C. Chen, Q. Chen, J. Xu, and V. Koltun, "Learning to see in the dark," CVPR, 2018. arXiv
18. A. Ignatov, L. Van Gool, and R. Timofte, "Replacing mobile camera ISP with a single deep learning model," arXiv:2002.05509, 2020. arXiv

IP 03・強度轉換與空間濾波

用曲線與直方圖重新對應像素值，再用小小的卷積核平滑與銳化影像：空間域影像增強的日常工具箱。

網站版本: https://blog.shyandram.dev/zh-tw/learn/image-processing/dip-03-intensity-spatial-filtering/

你將學到

- 一條曲線 $s = T(r)$ 如何讓影像變亮、變暗、反相或拉開對比，以及何時該選對數、gamma 或分段線性曲線。
- 直方圖等化（histogram equalization）為什麼有效（一步步推導），以及直方圖匹配、CLAHE 這類區域方法與區域統計量如何建立在它之上。
- 什麼是線性空間濾波、相關（correlation）與卷積（convolution）差在哪裡，以及可分離卷積核為什麼快。
- 平滑（低通）濾波器，包括非線性的中值濾波器，如何去除雜訊；以微分為基礎的（高通）濾波器如何銳化。
- 如何從低通卷積核推導出高通、帶通與帶拒卷積核，以及如何把多個方法串成一條處理流程。

先看全貌

這一章談的是：不離開像素格子，直接改善影像。本章所有方法都直接作用在像素值上，這就是「空間域」（spatial domain）的意思。方法分成兩大類。**點運算**（point operation）只用每個像素自己的值來改變它；**鄰域運算**（neighborhood operation，也就是空間濾波器）則參考周圍像素的值來改變它。

> **用最簡單的話說**
>
> 想像一盒蠟筆畫，每一張都畫得太淡了。**點運算**就像替每一種顏色訂一條魔法規則：「淡粉紅變成紅色，淡藍變成深藍。」你只看一個點，查一下規則，然後重新上色。**濾波器**則是在決定之前，先看看這個點和它的鄰居。想讓畫柔和一點，就把每個點塗成鄰居的平均色；想讓輪廓更清楚，就讓每個點和鄰居差得更多。

為什麼重要：這些是大家第一個會拿出來用的工具，手機相機、醫學影像檢視器、顯微鏡，以及電腦視覺模型的前處理都會用到。它們也是卷積層在概念上的祖先：CNN 的第一層就是一組空間濾波器，而許多學習式的影像增強網路，輸出的仍然是一條曲線或一個濾波器，這點會在「現代觀點」中看到。

背景

白話版。我們把輸出影像寫成「對輸入影像套用某個運算」。如果這個運算一次只看一個像素，它就是一張查找表；如果它看一個小視窗，它就是濾波器。

精確版。令 $I(x,y)$ 為輸入影像、$G(x,y)$ 為輸出影像，x,y 為整數像素座標（與卷積教學使用相同的符號）。空間域處理可寫成

$$G(x,y) = T[I](x,y),$$

其中 T 是定義在 (x,y) **鄰域**（neighborhood）上的運算子。鄰域通常是以 (x,y) 為中心的小正方形或長方形。把鄰域中心逐一移過每個像素，就能在每個位置套用這個運算子。

當鄰域只有一個像素（1×1）時，T 只取決於該像素的值，這時稱為**強度轉換**（intensity transformation，也稱灰階轉換或點轉換），並以純量寫成：

$$s = T(r),$$

其中 r 是某像素的輸入強度，s 是同一像素的輸出強度，兩者都落在 $[0, L-1]$，L 為強度階數（8 位元影像 $L = 256$）。因為 r 只有 L 種可能值，任何 T 都能存成長度為 L 的查找表（lookup table），這也是點運算極快的原因。

基本強度轉換函數

白話版。畫一條曲線，橫軸是輸入亮度、縱軸是輸出亮度。曲線陡的地方，相近的灰階會被拉開，對比增加；曲線平的地方，灰階被擠在一起，對比下降。

斜率 $T'(r)$ 就是局部的對比增益：兩個相鄰像素間的小差異 Δr，轉換後大約變成 $T'(r)\,\Delta r$。下面每一條曲線，都是在決定要把這份增益花在哪裡。

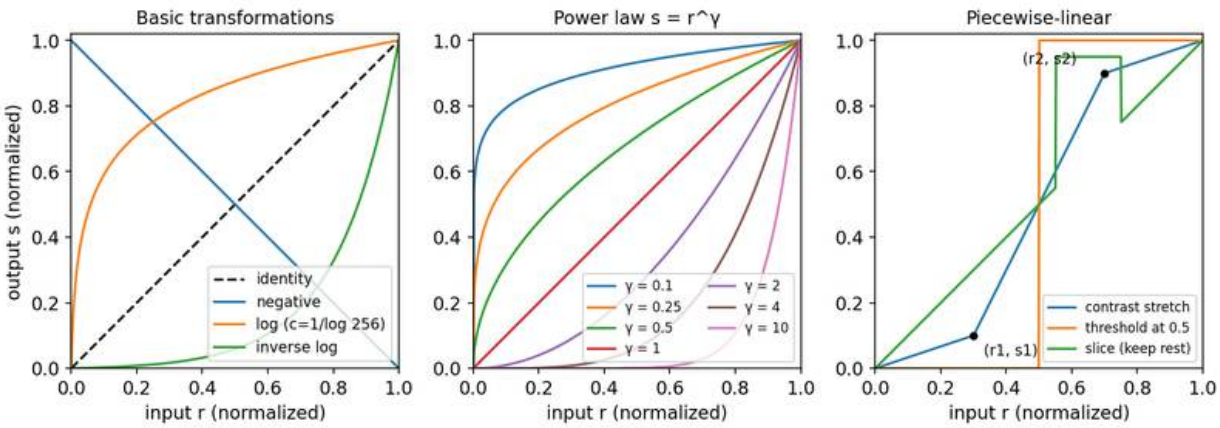


圖 *3.1* — 常見的強度轉換曲線（強度正規化到 *[0, 1]*）。左：反相、對數與反對數。中：冪次（*gamma*）曲線。右：分段線性曲線。

影像反相

$$s = (L - 1) - r.$$

暗變亮、亮變暗。斜率處處為 -1，所以對比的大小不變，只是極性反轉。當大片暗區中有細小的亮細節時，反相很有幫助，因為人通常比較容易在淺色背景上判讀灰階細節。

對數轉換

$$s = c \log(1 + r),$$

其中 $c > 0$ 為縮放常數，通常取 $c = (L - 1)/\log L$，讓最大輸入對應到 $L - 1$。加 1 是為了避開 $\log 0$。斜率 $c/(1 + r)$ 在 r 小時大、在 r 大時小，因此對數曲線會擴張暗部、壓縮亮部。

它的經典用途是顯示動態範圍極大的資料，例如傅立葉頻譜的強度，數值可以橫跨好幾個數量級。線性縮放時只看得到最亮的那一點；取對數之後，結構就浮現了（圖 3.2 下排）。**反對數**（指數）曲線的作用則恰好相反。

冪次（gamma）轉換

$$s = c\, r^{\gamma},$$

其中 $c > 0$、$\gamma > 0$ 為常數（若 r 已正規化到 $[0, 1]$，取 $c = 1$）。$\gamma < 1$ 時曲線往上拱，像對數一樣提亮暗部；$\gamma > 1$ 時曲線往下彎，壓暗暗部。和對數不同的是，一個參數就能產生一整族曲線（圖 3.1 中）。

Gamma 的重要性不只在增強。顯示器與相機都有冪次型的響應，影像檔通常儲存的是經過 gamma 編碼的值，而不是與光量成正比的值。預先補償裝置響應的做法稱為 **gamma 校正**（gamma correction）。一個實務上的後果是：對 gamma 編碼過的像素做平均、模糊或縮放，和在線性光上做並不相同，所以嚴謹的流程會先線性化。

```
import numpy as np
from skimage import data, img_as_float

r = img_as_float(data.camera())        # 強度縮放到 [0, 1]
negative = 1.0 - r                     # s = (L-1) - r 的正規化版本
log_t = np.log1p(r) / np.log(2.0)      # s = c log(1 + r)，c 讓 s(1) = 1
bright = r ** 0.4                      # gamma < 1 提亮暗部
dark = r ** 2.5                        # gamma > 1 壓暗暗部
```

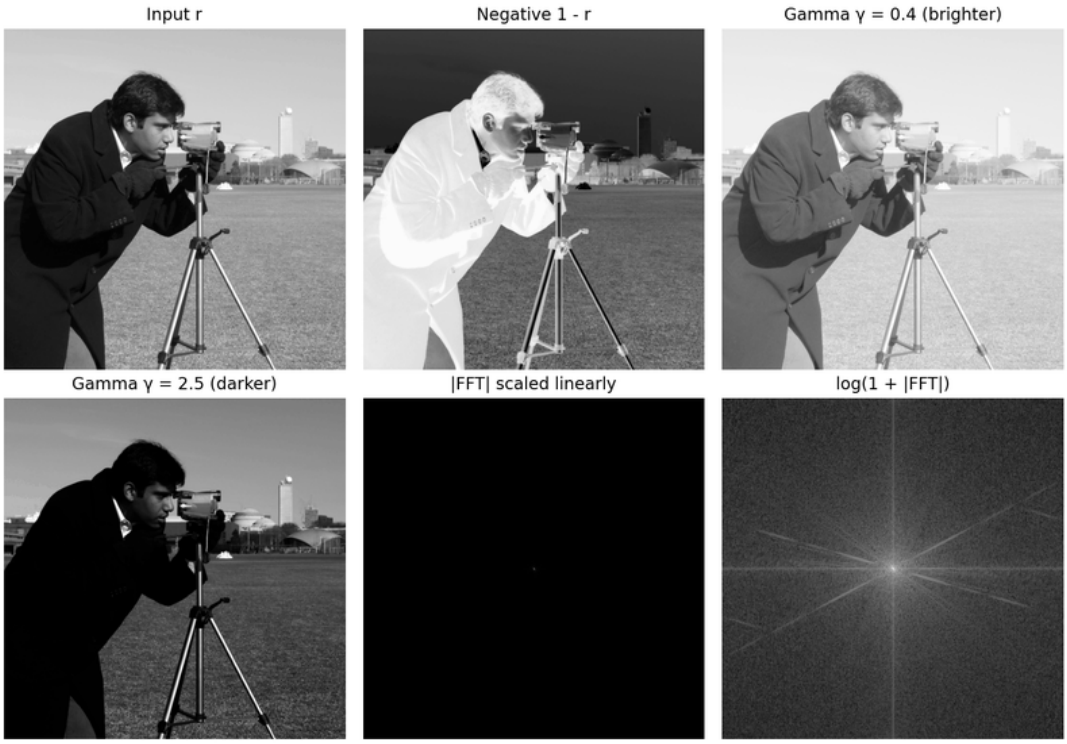


圖 *3.2* — 上排：輸入、反相、*gamma 0.4*。下排：*gamma 2.5*；線性縮放的傅立葉強度（幾乎全黑）；同一強度取 $log(1+\cdot)$ 之後。

分段線性轉換

不用單一公式，而是把幾段直線接起來。好處是控制精確，代價是參數較多。

對比拉伸（contrast stretching）。選兩個控制點 (r_1, s_1) 與 (r_2, s_2)，用直線連接 $(0,0) \to (r_1, s_1) \to (r_2, s_2) \to (L-1, L-1)$。中間那段的斜率是 $(s_2 - s_1)/(r_2 - r_1)$。只要 $r_1 \le r_2$ 且 $s_1 \le s_2$，曲線就是單調的，不會顛倒灰階的順序。有兩個特例值得記住：

- $r_1 = r_{\min}$、$s_1 = 0$、$r_2 = r_{\max}$、$s_2 = L-1$：把影像實際的範圍對應到完整範圍（**最小–最大拉伸**）。實務上改用第 1 與第 99 百分位數取代最小值與最大值，避免少數離群像素主導結果。
- $r_1 = r_2 = m$、$s_1 = 0$、$s_2 = L-1$：產生二值影像，也就是在 m 處做**門檻化**（threshold）。

```
def stretch(img, r1, s1, r2, s2, L=256):
    lut = np.interp(np.arange(L), [0, r1, r2, L - 1], [0, s1, s2, L - 1])
    return lut.astype(np.uint8)[img]

img = data.camera()
lo, hi = np.percentile(img, [1, 99])
out = stretch(img, lo, 0, hi, 255)     # 穩健的最小–最大拉伸
```

強度切片（intensity-level slicing）。突顯某一段強度 $[A, B]$。做法有兩種：把這段設成高值、其他設成低值（得到二值結果）；或只提亮這段、其他保持不變。當感興趣的灰階範圍已知時（例如 X 光中的某種組織或材料），切片就很好用。

位元平面切片（bit-plane slicing）。8 位元像素值是各位元的加總：$r = \sum_{b=0}^{7} a_b\, 2^b$，其中 $a_b \in \{0, 1\}$。把每個像素的第 b 個位元收集起來，就得到一張二值影像，稱為**位元平面** b。平面 7 是最高有效位元（MSB），恰好等於在 128 處門檻化。低位元平面看起來像雜訊，因為它們承載的是細微變化。只保留最高四個平面重建出的影像，與原圖非常接近，最大誤差小於 $2^4 = 16$ 階（圖 3.3）。這說明了位元平面為何與壓縮有關，也說明最低幾個平面為何常被拿來藏浮水印。

```
img = data.camera()                                    # uint8
planes = [(img >> b) & 1 for b in range(8)]            # planes[7] 是 MSB
top4 = sum(planes[b].astype(np.uint8) << b for b in range(4, 8))
print(np.abs(img.astype(int) - top4).max())            # 15
```

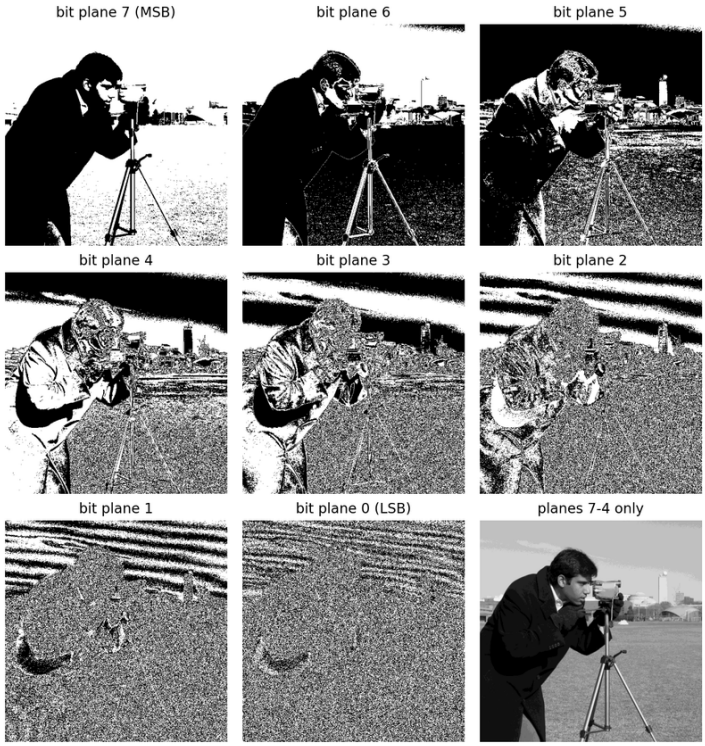


圖 *3.3 — 8* 位元影像的八個位元平面，從最高有效位元（左上）到最低有效位元，以及只用平面 *7–4* 重建的影像。

直方圖處理

白話版。直方圖（histogram）統計每種亮度各有多少像素。暗影像的計數堆在左邊；灰濛濛的影像則擠在一條窄帶裡。直方圖處理就是根據這些計數自動選出曲線 T，讓輸出的直方圖變成我們

想要的形狀。

精確版。對一張 $M \times N$、灰階為 r_k（$k = 0, \dots, L-1$）的影像，令 n_k 為值等於 r_k 的像素數。**正規化直方圖**

$$p_r(r_k) = \frac{n_k}{MN}$$

估計了「隨機挑一個像素，其值為 r_k」的機率，總和為 1。

直方圖等化

目標是找一條曲線 T，讓輸出直方圖盡可能平坦，也就是每個灰階被使用的頻率大致相同。

推導（連續情形）。把強度視為連續隨機變數 $r \in [0, L-1]$，其機率密度函數（PDF）為 $p_r(r)$。假設 T 在此區間嚴格遞增。那麼像素落在小區間 $[r, r+dr]$，恰好等於輸出落在 $[s, s+ds]$，其中 $s = T(r)$。機率相等給出 $p_s(s)\,|ds| = p_r(r)\,|dr|$，因此

$$p_s(s) = p_r(r) \left| \frac{dr}{ds} \right|.$$

現在把 T 選為 r 的累積分布函數（CDF）乘上比例：

$$s = T(r) = (L-1) \int_0^r p_r(w)\, dw,$$

其中 w 是積分用的虛擬變數。由微積分基本定理，$ds/dr = (L-1)\,p_r(r)$。代入得

$$p_s(s) = p_r(r) \cdot \frac{1}{(L-1)\,p_r(r)} = \frac{1}{L-1}, \qquad 0 \le s \le L-1.$$

不論輸入的 PDF 長什麼樣，輸出的 PDF 都是均勻分布。CDF 正是那條能把機率平均攤開的單調曲線：像素多的地方它陡（把像素拉開），像素少的地方它平。

離散版本。把積分換成加總：

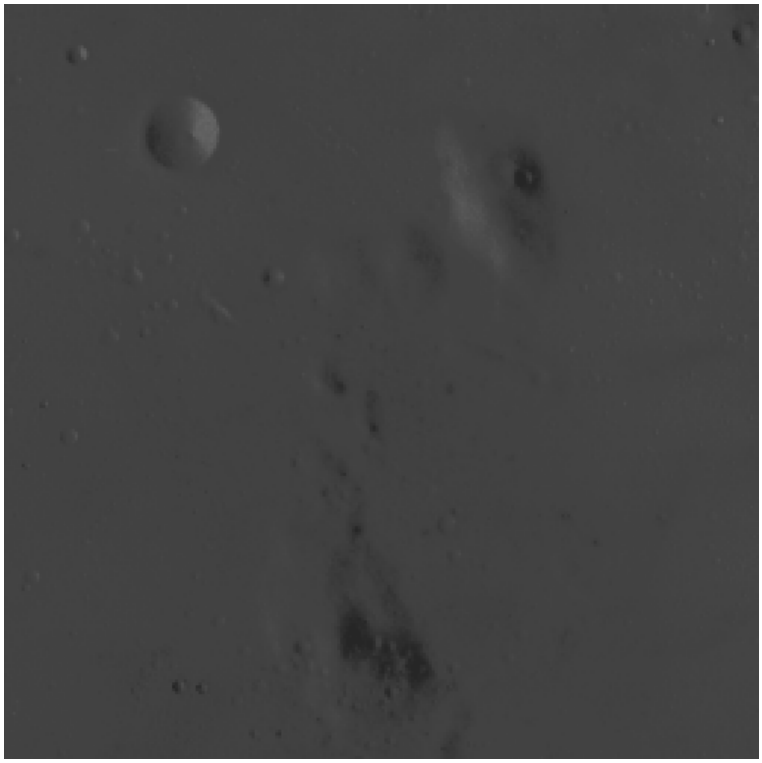
$$s_k = T(r_k) = (L-1) \sum_{j=0}^{k} p_r(r_j), \qquad k = 0, 1, \dots, L-1,$$

再把 s_k 四捨五入到最近的整數。由於多個輸入灰階可能捨入到同一個輸出灰階，而一個灰階又無法拆開，離散結果只會近似平坦。輸出直方圖通常會出現高聳的尖峰，中間夾著空隙（圖 3.4 中）。真正能保證的是：輸出涵蓋完整範圍，且輸出的 CDF 接近一條直線。

```
def equalize(img, L=256):
    hist = np.bincount(img.ravel(), minlength=L)      # n_k
    cdf = np.cumsum(hist) / img.size                  # j <= k 的 p_r(r_j) 加總
    lut = np.round((L - 1) * cdf).astype(np.uint8)    # s_k
    return lut[img]
```

OpenCV 的 `cv2.equalizeHist` 使用稍微不同的正規化 $(\mathrm{cdf}_k - \mathrm{cdf}_{\min})/(MN - \mathrm{cdf}_{\min})$，讓最暗的有值灰階對應到 0，結果大約只差一階。

拖動滑桿，比較一張刻意壓低對比的影像與它的全域等化結果。注意等化並不是「免費的」：它會誇大大片平滑區域與顆粒感，因為它把對比花在像素數量多的地方，而不是有意義的細節所在之處。



輸入

等化後

一張昏暗、平淡的月面影像，與其直方圖等化後對比強烈的版本。

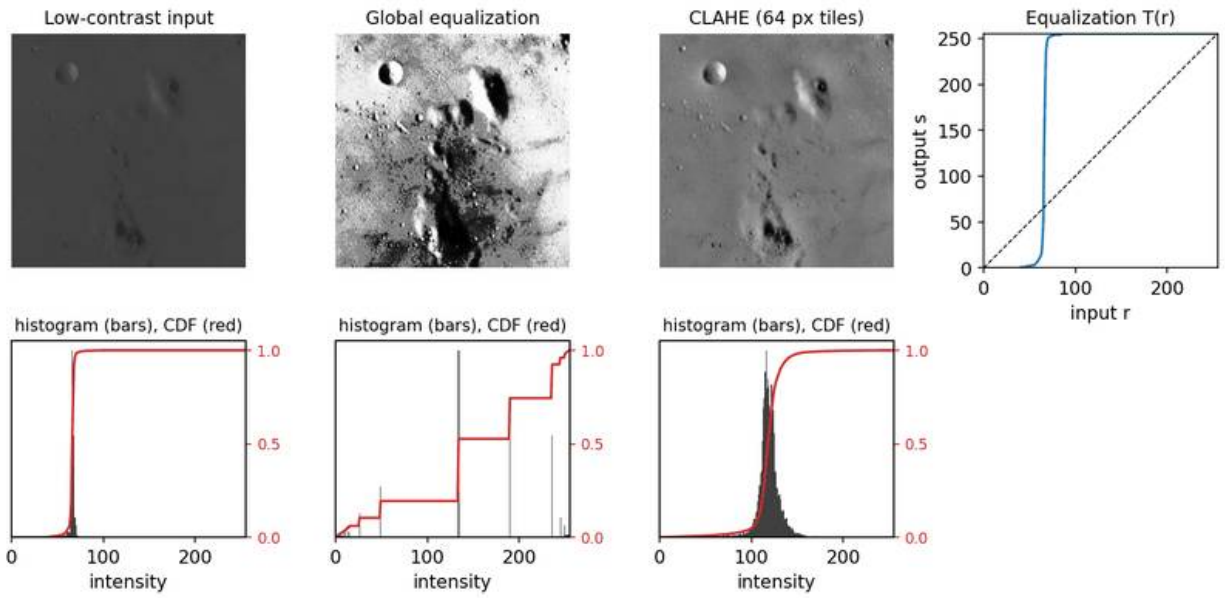


圖 *3.4* — 全域直方圖等化與 *CLAHE* 的比較。輸入只占據很窄的灰階範圍，所以等化曲線 $T(r)$（右）幾乎是階梯，輸出直方圖被拉開且出現空隙。*CLAHE* 限制斜率，並依區域調整。

直方圖匹配（規定化）

有時候平坦的直方圖並不是我們要的目標。我們可能希望影像看起來像某張參考影像，或希望直方圖是手動指定的形狀。**直方圖匹配**（histogram matching，也稱 histogram specification）把輸入對應成直方圖接近指定 PDF $p_z(z)$ 的影像。

技巧是用兩次等化。等化輸入得到均勻的 $s = T(r)$；對目標分布做等化，得到

$$G(z) = (L-1)\int_0^z p_z(t)\,dt,$$

它同樣產生均勻變數。兩個均勻變數可以互相對應，因此令 $G(z) = T(r)$ 並解出：

$$z = G^{-1}(T(r)).$$

其中 z 是輸出強度、G 是目標分布的縮放 CDF、t 是虛擬變數。離散情形下 G 是階梯函數，未必可逆，所以對每個輸入灰階取滿足 $G(z) \geq T(r)$ 的最小 z，只要一個 `searchsorted` 就能完成：

```
def match(img, ref, L=256):
    cdf_in = np.cumsum(np.bincount(img.ravel(), minlength=L)) / img.size
    cdf_ref = np.cumsum(np.bincount(ref.ravel(), minlength=L)) / ref.size
    # 對每個輸入灰階，取滿足 G(z) >= T(r) 的最小 z
    lut = np.searchsorted(cdf_ref, cdf_in).clip(0, L - 1).astype(np.uint8)
    return lut[img]
```

`skimage.exposure.match_histograms` 實作了相同的概念，也支援彩色影像的逐通道匹配。

精確直方圖匹配

上面兩種方法都是查找表，所以輸入值相同的像素必然得到相同的輸出值。如果 30% 的像素共用同一個值，任何查找表都無法把它們拆開，輸出直方圖也就不可能完全平坦或完全符合目標。

精確直方圖規定化（exact histogram specification）先讓每個像素都能被區分，以消除這個限制 [5]。每個像素得到一組特徵向量：自己的值、小鄰域的平均、較大鄰域的平均，依此類推。依這組向量做字典序排序，實務上就能得到嚴格的順序，因為某個特徵相同時，會由下一個特徵來打破平手。接著依序指定輸出值：前 h_0 個像素給灰階 0，接下來 h_1 個給灰階 1，以此類推，h_k 是目標直方圖要求的確切數量。結果的直方圖與要求完全一致。Coltuc、Bolon 與 Chassery 指出，用由小到大的區域平均來排序，打破平手的方式在視覺上很合理，因為周圍較亮的像素會被稍微往上推 [5]。

區域直方圖處理

全域方法對整張影像只用一條曲線。如果重要細節藏在暗角落、而那裡的像素又不多，全域直方圖幾乎不會注意到它。

區域直方圖等化對每個像素周圍的鄰域計算直方圖，由此算出轉換，只套用到中心像素，再移到下一個像素。**自適應直方圖等化**（adaptive histogram equalization, AHE）及其變體由 Pizer 等人

做了系統性的研究，其中包括 AHE 在近乎均勻的區域會過度放大雜訊的問題：那裡的區域直方圖只是一根尖峰，CDF 變成近乎垂直的階梯 [2]。

對比受限自適應直方圖等化（contrast-limited AHE, CLAHE）透過限制對應曲線的斜率來解決這個問題。由於等化曲線的斜率與直方圖高度成正比，把每個區域直方圖在某個上限處截斷，再把截掉的計數重新分配到所有 bin，就能為對比增益設上限。為了加速，影像被切成小塊（tile），每塊算一條對應曲線，每個像素的輸出由最近四塊的對應結果做雙線性內插，避免出現明顯的接縫。Zuiderveld 在 Graphics Gems 的章節給出了被廣泛沿用的實作 [3]，Reza 則描述了即時的硬體實現 [4]。

```
import cv2
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
out = clahe.apply(data.moon())         # 輸入 uint8，輸出 uint8
```

在 scikit-image 中，對應的是 `skimage.exposure.equalize_adapthist(img, clip_limit=0.01)`。

利用直方圖統計量增強影像

直方圖是一個分布，因此可以用**動差**（moment）來描述它。令 $p(r_k)$ 為正規化直方圖，平均與變異數為

$$m = \sum_{k=0}^{L-1} r_k\, p(r_k), \qquad \sigma^2 = \sum_{k=0}^{L-1} (r_k - m)^2\, p(r_k),$$

其中 m 衡量平均亮度，σ（標準差）衡量對比。若改在以 (x, y) 為中心的鄰域 S_{xy} 上計算，就得到**區域平均** $m_{S_{xy}}$ 與**區域標準差** $\sigma_{S_{xy}}$。

這給了我們一條簡單又有針對性的增強規則。假設只想增強偏暗、而且有一些但不太多對比的區域（同時跳過平坦背景與強烈邊緣）。令全域平均為 m_G、全域標準差為 σ_G，只有在下列條件成立時，才把像素乘上增益 $E > 1$，否則保持不變：

$$m_{S_{xy}} \le k_0\, m_G \quad \text{且} \quad k_1\, \sigma_G \le \sigma_{S_{xy}} \le k_2\, \sigma_G,$$

其中 k_0, k_1, k_2 是使用者選定的比例（例如 $0.5, 0.02, 0.5$）。區域統計量只需要兩次方框濾波，一次作用在 I、一次作用在 I^2，再利用 $\sigma^2 = \mathbb{E}[I^2] - (\mathbb{E}[I])^2$：

```
from scipy.ndimage import uniform_filter

f = img_as_float(data.moon())
m = uniform_filter(f, size=3)                                  # 區域平均
sd = np.sqrt(np.maximum(uniform_filter(f**2, size=3) - m**2, 0))  # 區域標準差
mG, sG = f.mean(), f.std()
mask = (m <= 0.5 * mG) & (sd >= 0.02 * sG) & (sd <= 0.5 * sG)
g = np.where(mask, np.clip(3.0 * f, 0, 1), f)                  # 增益 E = 3
```

同樣「用方框濾波器求區域平均與區域變異數」的模式，也出現在導引濾波器（guided filter）[8]中，它是「現代觀點」會談到的保邊濾波器之一。

空間濾波的基礎

白話版。空間濾波器把一小格權重（**卷積核**，kernel）滑過整張影像。在每個位置，把權重乘上底下的像素再加總，結果就是中心像素的新值。

卷積教學已經用從零寫起的 NumPy 迴圈、OpenCV 呼叫與邊界處理，把操作細節講清楚了。這裡專注於本章後面需要的理論。

線性空間濾波與相關

沿用該教學的符號，對大小為 $(2k+1)\times(2k+1)$ 的卷積核 K，K 與 I 的**相關**（correlation）為

$$(K \star I)(x,y) = \sum_{i=-k}^{k}\sum_{j=-k}^{k} K(i,j)\, I(x+i,\, y+j),$$

其中 (i,j) 是相對於卷積核中心的位移，k 是卷積核的半寬。奇數尺寸讓卷積核有明確的中心。這個濾波器是**線性**的，因為輸出是輸入值的線性組合：對 $aI_1 + bI_2$ 濾波，等於 a 倍的 I_1 結果加上 b 倍的 I_2 結果。它也是**平移不變**的，因為每個位置用的是同一組權重。

相關與卷積

卷積是同樣的計算，只是先把卷積核旋轉 180°：

$$(K * I)(x,y) = \sum_{i=-k}^{k}\sum_{j=-k}^{k} K(i,j)\, I(x-i,\, y-j).$$

對旋轉 180° 後不變的對稱卷積核（方框、高斯、拉普拉斯），兩者相同；對反對稱的卷積核（Sobel、任何方向導數），兩者結果相同但正負號相反。真正有用的差別出現在**脈衝**（impulse，一堆 0 中的一個 1）上：卷積核與脈衝做相關，得到的是旋轉過的卷積核；做卷積，得到的則是一模一樣的卷積核。這就是為什麼代數性質漂亮的是卷積而不是相關：

- **交換律：** $K * I = I * K$，
- **結合律：** $K_1 * (K_2 * I) = (K_1 * K_2) * I$，所以一連串濾波器可以合併成單一卷積核，
- **分配律：** $K * (I_1 + I_2) = K * I_1 + K * I_2$。

接下來兩小節都靠結合律。就像卷積教學提到的，`cv2.filter2D` 計算的是相關；大多數深度學習的「卷積」層其實也是相關，但因為權重是學出來的，翻不翻轉並不重要。

可分離卷積核

若卷積核可以寫成行向量 $\mathbf{c}$ 與列向量 $\mathbf{r}$ 的外積，就稱它是**可分離的**（separable）：

$$K = \mathbf{c}\,\mathbf{r}^{\top}.$$

於是 $K * I = \mathbf{c} * (\mathbf{r}^{\top} * I)$：先用 $\mathbf{r}$ 濾每一列，再用 $\mathbf{c}$ 濾每一行。對 $m \times n$ 的卷積核，每個像素的乘法次數從 mn 降為 $m + n$，加速比為 $mn/(m+n)$，7×7 時是 3.5 倍，且隨尺寸線性成長。矩陣恰好是外積若且唯若它的**秩**（rank）為 1，這也提供了一個檢驗方法：做奇異值分解，確認只有一個奇異值不為零。方框與高斯卷積核都可分離；Sobel 也可以，它是一個平滑向量乘上一個差分向量。

```
g1 = cv2.getGaussianKernel(7, 1.5)          # 7x1 行向量，總和為 1
K = g1 @ g1.T                               # 7x7 高斯卷積核
print(np.linalg.matrix_rank(K))             # 1   ->   可分離
f = img_as_float(data.camera()).astype(np.float32)
a = cv2.filter2D(f, -1, K.astype(np.float32))
b = cv2.sepFilter2D(f, -1, g1, g1)          # 兩次一維濾波
print(np.abs(a - b).max())                  # 約 1e-7
```

空間域與頻域

卷積定理指出，空間中的卷積等於頻率中的相乘：若 $\mathcal{F}$ 表示傅立葉轉換，則 $\mathcal{F}\{K * I\} = \mathcal{F}\{K\} \cdot \mathcal{F}\{I\}$。因此每個線性、平移不變的濾波器都有兩種等價描述：空間中的卷積核 K，以及說明每個頻率保留多少的**轉移函數**（transfer function）$H = \mathcal{F}\{K\}$。平滑卷積核的轉移函數在零頻率附近大、在高頻小，所以是**低通**（lowpass）；銳化卷積核相反，是**高通**（highpass）。第 4 章會正式發展這個觀點；目前只要記得這組詞彙：「低通」就是「平滑」，「高通」就是「邊緣與細節」。

小卷積核通常在空間域計算比較便宜；非常大的卷積核則常透過快速傅立葉轉換在頻域計算比較划算。

建構空間濾波卷積核

常見的做法有三種：

1. **從數學性質出發**。取平均得到方框卷積核；取導數得到差分卷積核（拉普拉斯、Sobel）。
2. **對連續函數取樣**。在整數格點上對二維高斯取樣，再把權重正規化成總和為 1。
3. **從頻率規格出發**。在頻域設計 H，做反轉換得到空間卷積核，通常再截斷成可處理的大小。

所有做法都適用兩個檢查。平滑卷積核的權重總和應為 1，平坦區域才能保持亮度；導數卷積核的權重總和應為 0，平坦區域才會輸出 0。

平滑（低通）空間濾波器

白話版。把每個像素換成鄰域的平均。隨機雜訊被拉向平均而淡去，但銳利的邊緣也會被抹開，因為平均跨過了邊緣兩側。

方框濾波器

$m \times n$ 方框卷積核（box kernel）的每個權重都是 $1/(mn)$。它是最簡單的低通濾波器，可分離，而且搭配累加和（running sum）時，每個像素的計算量與尺寸無關。它的弱點在頻率響應：方框的傅立葉轉換是類似 sinc 的函數，帶有旁瓣（side lobe），所以會讓部分高頻以反相的方式通過，而且它以正方形、與方向有關的方式抹開影像，細小文字與細線可能變成塊狀條紋。

高斯濾波器

高斯卷積核是對下式取樣：

$$w(s,t) = C\, e^{-\frac{s^2+t^2}{2\sigma^2}},$$

其中 (s,t) 是相對於卷積核中心的位移，σ（標準差）決定模糊程度，C 讓權重總和為 1。它有三個性質，讓它成為預設的平滑卷積核：

- **等向性**（isotropy）。它只取決於距離 $\sqrt{s^2+t^2}$，所以各方向模糊程度相同。
- **可分離性**。$e^{-(s^2+t^2)/2\sigma^2} = e^{-s^2/2\sigma^2}\, e^{-t^2/2\sigma^2}$。
- **平滑且非負的頻率響應**。它的傅立葉轉換也是高斯，沒有旁瓣。

一維高斯約 99.7% 的質量落在 $\pm 3\sigma$ 之內，所以約 6σ 大小（向上取到最近的奇數）的卷積核就幾乎涵蓋全部。更大只是浪費計算，更小則會截掉尾巴。兩個標準差分別為 σ_1、σ_2 的高斯做卷積，得到 $\sigma = \sqrt{\sigma_1^2 + \sigma_2^2}$ 的高斯，所以重複的小模糊可以預測地組合起來。

順序統計（非線性）濾波器

順序統計濾波器（order-statistic filter）把鄰域中的值**排序**後挑一個出來。它們是非線性的：沒有任何卷積核能表示它們，卷積定理也不適用。

- **中值濾波器**（median filter）：輸出中間值（第 50 百分位數）。它是處理**脈衝雜訊**（impulse noise，又稱椒鹽雜訊 salt-and-pepper）的標準方法，也就是被強制變成全黑或全白的孤立像素。脈衝是極端值，排序後會跑到兩端，幾乎不可能是中值。只要視窗中被汙染的像素少於一半，中值就會忽略它們。在同樣大小下，它保留階梯邊緣的能力也遠勝線性平均。
- **最大值濾波器**（第 100 百分位數）：找出最亮的點；能去除胡椒（暗）雜訊，但會讓亮點長大。
- **最小值濾波器**（第 0 百分位數）：相反；能去除鹽（亮）雜訊，但會讓暗點長大。

最大值與最小值濾波器會在第 9 章以灰階膨脹與侵蝕的形式再次出現。

```
from scipy import ndimage as ndi

f = img_as_float(data.camera()).astype(np.float32)
noisy = f.copy()
u = np.random.default_rng(0).random(f.shape)
noisy[u < 0.05], noisy[u > 0.95] = 0.0, 1.0       # 10% 椒鹽雜訊
```

```
box = cv2.blur(noisy, (5, 5))
gauss = cv2.GaussianBlur(noisy, (0, 0), sigmaX=1.5)
med = cv2.medianBlur(noisy, 3)
mn, mx = ndi.minimum_filter(noisy, 3), ndi.maximum_filter(noisy, 3)
```

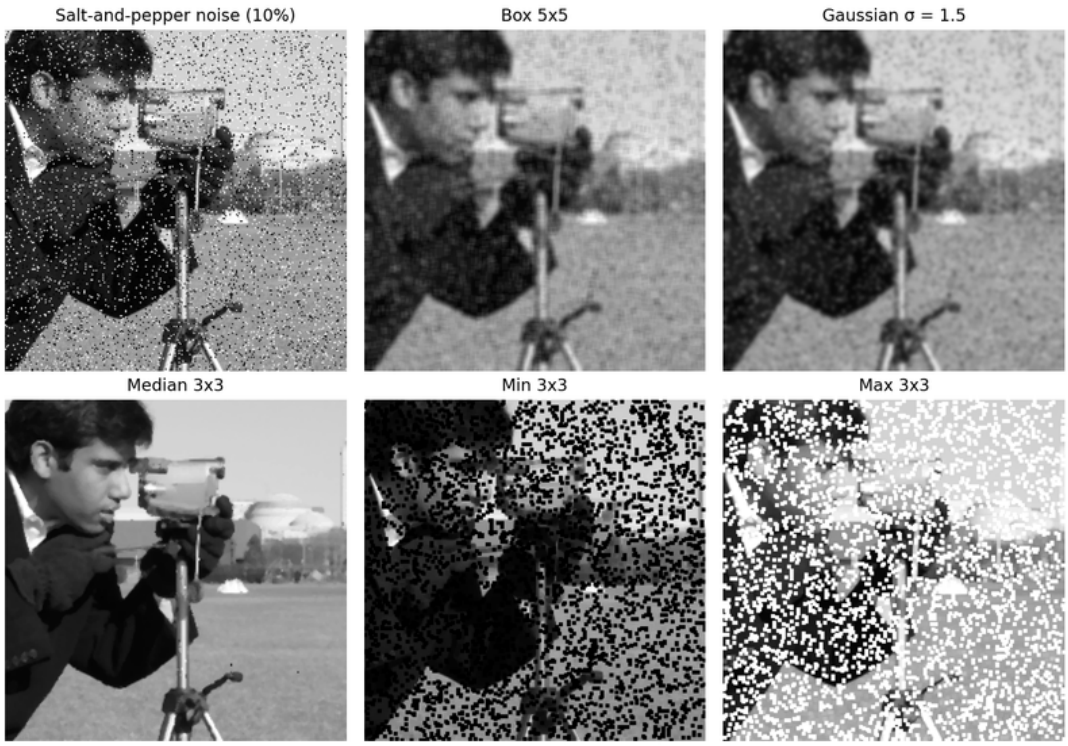


圖 *3.5* — 平滑 *10%* 椒鹽雜訊。線性濾波器（方框、高斯）把每個脈衝攤成灰色汙漬；*3×3* 中值濾波器幾乎完全去除雜訊；最小值與最大值各自去除一種極性，卻放大另一種。

銳化（高通）空間濾波器

白話版。平滑是取平均；銳化則相反，要突顯差異。鄰居之間的差異就是導數，所以銳化是用導數做出來的。

一階與二階導數

對像素格點上的一維訊號 $f(x)$，最簡單的近似是

$$\frac{\partial f}{\partial x} = f(x+1) - f(x), \qquad \frac{\partial^2 f}{\partial x^2} = f(x+1) + f(x-1) - 2f(x).$$

圖 3.6 把兩者套用在一條手工製作的剖面上，其中有一段斜坡、一條一個像素寬的細線和一個階梯。從圖中可以讀出選擇導數的規則：

- 在**平坦**區域，兩種導數都是 0。
- 沿著**斜坡**，一階導數一路都不為零，因此響應很粗；二階導數只在斜坡的起點與終點不為零。
- 在**細線**處，二階導數有強烈的雙重響應（正、強負、正），而且幅度比一階導數大。細小的細節正是銳化該強調的東西。
- 在**階梯**處，二階導數產生相鄰的一正一負值，兩者之間的**零交越**（zero crossing）可精確定位邊緣，第 10 章會把它用在邊緣偵測上。

所以要增強細節時，二階導數是比較自然的選擇。

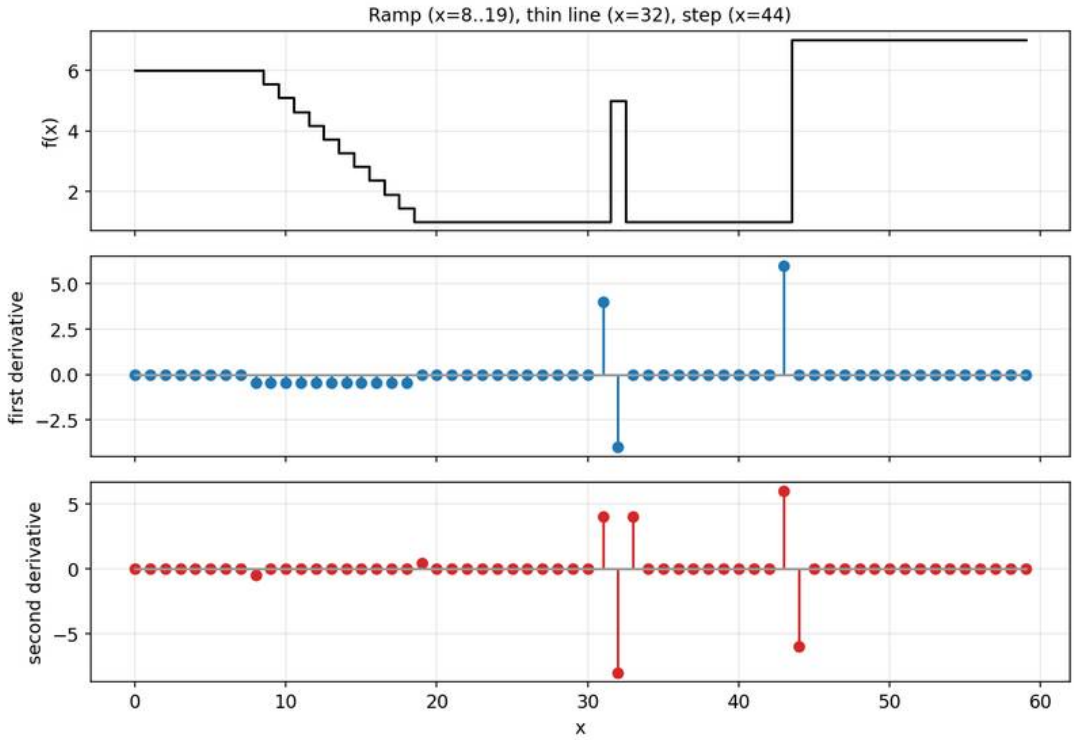


圖 *3.6* — 合成剖面的一階與二階差分。二階導數忽略斜坡內部、對細線反應最強，並在階梯兩側改變正負號。

拉普拉斯運算子

最簡單的**等向性**二階導數運算子（影像旋轉時響應不變）是**拉普拉斯運算子**（Laplacian）：

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}.$$

把上面的一維差分分別用在 x 與 y 方向並相加，得到

$$\nabla^2 I(x,y) = I(x+1,y) + I(x-1,y) + I(x,y+1) + I(x,y-1) - 4I(x,y),$$

也就是與下列卷積核做相關：

$$K_{\nabla^2} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}.$$

若再加入兩個對角方向，會得到外圈八格都是 1、中心為 −8 的版本，它以 45° 而非 90° 為間隔具有等向性。把所有正負號翻轉，同樣是有效的卷積核。權重總和為 0，所以平坦區域會消失，只留下變化。

用拉普拉斯銳化。拉普拉斯影像大多是灰色（零），在邊緣處有亮暗的鑲邊。把它加回原圖，就把這些鑲邊疊到邊緣上：

$$G(x,y) = I(x,y) + c\,\nabla^2 I(x,y),$$

其中中心為負的卷積核（如上面的 K_{∇^2}）取 $c = -1$，中心為正的卷積核取 $c = +1$。正負號很重要：選錯號，邊緣反而會被模糊。由於卷積具有分配律，$I - \nabla^2 I$ 本身就是單一卷積核：中心為 5、上下左右為 −1，正是下方實驗室中的 `sharpen` 預設。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

試試 `gaussian` 與 `box-blur` 預設來平滑，再試 `laplacian` 看純粹的二階導數響應。把 sharpen 卷積核的中心從 5 改成 6，權重總和就變成 2，整張影像會變亮，這說明了總和為什麼重要。

反銳化遮罩與高增強濾波

這是來自傳統暗房的技巧：減去一份模糊的副本來分離出細節，再把細節加回去。

$$g_{\text{mask}}(x,y) = I(x,y) - \bar{I}(x,y), \qquad G(x,y) = I(x,y) + \alpha\, g_{\text{mask}}(x,y),$$

其中 $\bar{I}$ 是 I 的模糊（低通）版本，g_{mask} 是**反銳化遮罩**（unsharp mask，即被模糊去掉的細節），$\alpha \geq 0$ 是權重。$\alpha = 1$ 時稱為**反銳化遮罩**（unsharp masking）；$\alpha > 1$ 時稱為**高增強濾波**（highboost filtering）；$0 < \alpha < 1$ 時效果溫和。因為 g_{mask} 是高通訊號，這是另一種加入高通細節的方式；若 $\bar{I}$ 是高斯模糊，模糊的 σ 就決定了哪種大小的細節會被增強。

α 太大會產生明顯的**光暈**（halo）：強烈邊緣兩側出現過衝，輸出也可能超出有效範圍，所以要裁切（clip）。

梯度與 Sobel 運算子

一階導數是以**梯度**（gradient）的形式使用，也就是偏導數組成的向量

$$\nabla I = \begin{bmatrix} g_x \\ g_y \end{bmatrix} = \begin{bmatrix} \partial I/\partial x \\ \partial I/\partial y \end{bmatrix},$$

它指向強度增加最快的方向。它的長度，即**梯度大小**

$$M(x,y) = \sqrt{g_x^2 + g_y^2} \approx |g_x| + |g_y|,$$

在邊緣處大、在平坦區域接近 0。右邊的近似省去了開根號，但不具等向性。

Sobel 運算子用 3×3 卷積核估計 g_x 與 g_y：

$$K_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \qquad K_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}.$$

兩者都可分離：$K_x = [1,2,1]^\top [-1,0,1]$，即一個方向的中央差分乘上另一個方向的輕微平滑。中間的 2 讓中央那一列或那一行的權重較大，降低運算子對雜訊的敏感度。（卷積教學就是用 K_x 當作邊緣偵測的例子。）梯度大小很少直接拿來銳化，更常用來找出邊緣，或當作遮罩來決定哪裡允許銳化，就像下面的綜合範例。

```
f = img_as_float(data.chelsea()[..., 1]).astype(np.float32)
lap_k = np.array([[0, 1, 0], [1, -4, 1], [0, 1, 0]], np.float32)
lap = cv2.filter2D(f, -1, lap_k)
sharp = np.clip(f - lap, 0, 1)                  # 中心為負，所以 c = -1

blur = cv2.GaussianBlur(f, (0, 0), 2)
mask = f - blur                                 # g_mask
unsharp = np.clip(f + 1.0 * mask, 0, 1)         # alpha = 1
highboost = np.clip(f + 3.0 * mask, 0, 1)       # alpha = 3

gx = cv2.Sobel(f, cv2.CV_32F, 1, 0, ksize=3)
gy = cv2.Sobel(f, cv2.CV_32F, 0, 1, ksize=3)
M = np.hypot(gx, gy)                            # 梯度大小
```

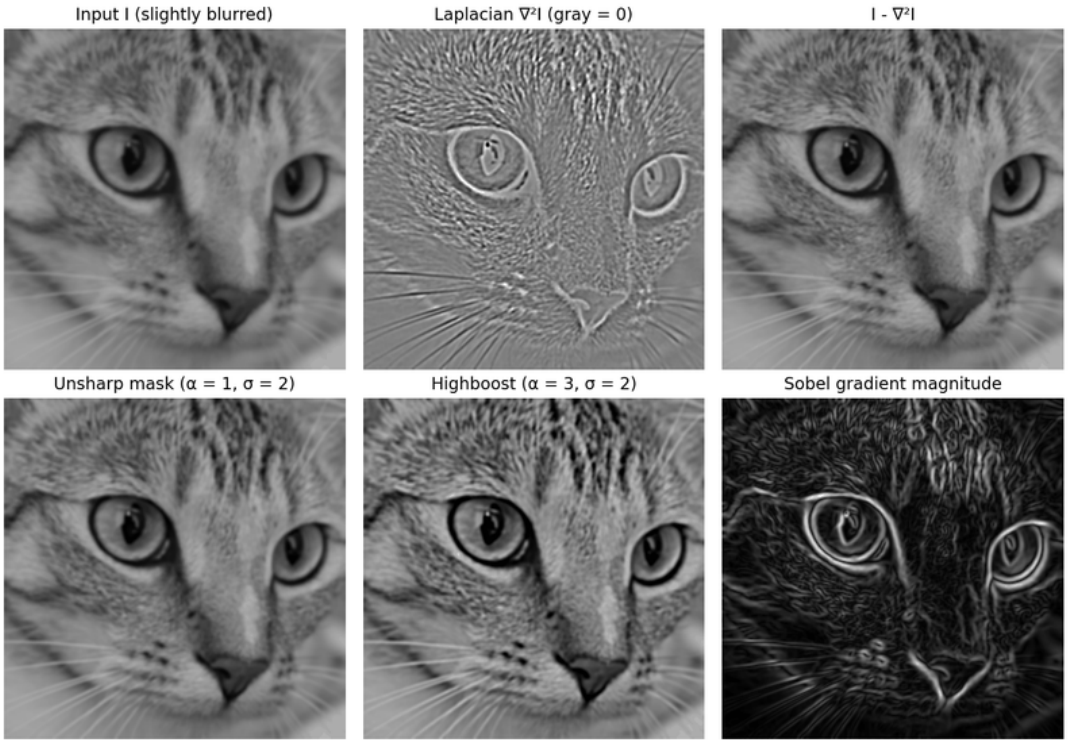


圖 3.7 — 銳化一張稍微模糊的影像。上排：輸入、拉普拉斯（灰色代表零）、$I - \nabla^2 I$。下排：反銳化遮罩、高增強濾波、*Sobel* 梯度大小。

濾波時一律使用浮點數。`uint8` 的流程會把拉普拉斯或 Sobel 響應的負值裁成 0，悄悄丟掉一半的資訊。

由低通濾波器構成高通、帶拒與帶通濾波器

白話版。如果你知道怎麼保留平滑的部分，你也就知道怎麼保留平滑部分以外的一切：相減就好。

令 δ 為**單位脈衝卷積核**：中心為 1、其餘為 0，因此 $\delta * I = I$。它的轉移函數在所有頻率都等於 1（全通濾波器）。若 K_{LP} 是轉移函數為 H_{LP} 的低通卷積核，則

$$K_{\text{HP}} = \delta - K_{\text{LP}} \quad \Longleftrightarrow \quad H_{\text{HP}} = 1 - H_{\text{LP}}.$$

反銳化遮罩正是如此：$g_{\text{mask}} = I - \bar{I} = (\delta - K_{\text{LP}}) * I$。

要隔離某一**頻帶**，就組合兩個低通濾波器。令 K_1 為 σ_1 較小的高斯（截止頻率高，保留較多細節），K_2 為 σ_2 較大的高斯（截止頻率低）。則

$$K_{\text{BP}} = K_1 - K_2, \qquad K_{\text{BR}} = \delta - K_{\text{BP}} = K_2 + (\delta - K_1).$$

帶通（bandpass）卷積核 K_{BP} 保留 K_1 讓過、但 K_2 擋下的頻率。**帶拒**（bandreject）卷積核 K_{BR} 是一個低通加一個高通，只去除中間那段頻帶。帶通 $K_1 - K_2$ 就是**高斯差**（difference of Gaussians, DoG），它也是高斯拉普拉斯的良好近似，並且是第 12 章尺度空間偵測器的基礎。權重總和可以驗證設計：高通與帶通卷積核總和為 0，帶拒卷積核總和為 1。

```
def gauss_kernel(sigma):
    n = 2 * int(np.ceil(3 * sigma)) + 1          # 約 6 sigma，取奇數
    g = cv2.getGaussianKernel(n, sigma)
    return g @ g.T

def delta_like(K):
    D = np.zeros_like(K); D[K.shape[0] // 2, K.shape[1] // 2] = 1
    return D

lp2 = gauss_kernel(4.0)                          # 低截止頻率
lp1 = gauss_kernel(1.0)                          # 高截止頻率
lp1 = np.pad(lp1, (lp2.shape[0] - lp1.shape[0]) // 2)   # 補零到相同大小
hp = delta_like(lp1) - lp1                       # 高通，總和為 0
bp = lp1 - lp2                                   # 帶通（DoG），總和為 0
br = delta_like(bp) - bp                         # 帶拒，總和為 1
```

結合多種空間增強方法

白話版。真實影像很少只有一個問題。有用的增強通常是一條短短的處理流程，而且步驟的順序很重要。

以下是一個實作範例。輸入（圖 3.8a）是一張曝光不足的人像，帶有少量高斯雜訊，以及 4% 的椒鹽脈衝。我們希望得到更亮、更清晰的影像，又不放大雜訊。

1. **先用 3×3 中值濾波器去除脈衝**（b）。脈衝是極端值，後續任何拉對比或銳化都會讓它們更糟，而線性模糊只會把它們攤開。中值濾波能去除它們又保留邊緣。
2. **用 gamma 0.45 提亮暗部**（c）。全域的亮度問題，正適合用點運算處理。在中值之後才做，曲線就不會被黑白脈衝干擾。
3. **建立邊緣權重圖**（d）。用高斯（$\sigma = 2$）模糊影像，計算模糊後影像的 Sobel 梯度大小 M，再正規化成 $w = \min(M/M_{90}, 1)$，其中 M_{90} 是 M 的第 90 百分位數。因為是在平滑之後計算，w 在真正的邊緣上接近 1，在平坦區域殘留的雜訊上接近 0。
4. **套用由權重圖控制的反銳化遮罩**（f）：$G = I_c + \alpha\, w \cdot (I_c - \bar{I}_c)$，其中 I_c 是 gamma 校正後的影像、$\bar{I}_c$ 是它的高斯模糊、$\alpha = 2$，乘法逐像素進行。

(e) 是用相同 α 的一般反銳化遮罩：背景與深色衣領都變得粗糙，因為雜訊同樣是高頻細節。受控版本 (f) 對臉、頭髮與太空衣輪廓的銳化程度幾乎一樣，卻讓平滑的背景保持乾淨。這種「用平滑後的一階導數影像來決定哪裡要做二階銳化」的模式，可以看作「現代觀點」中保邊處理的手工版本。

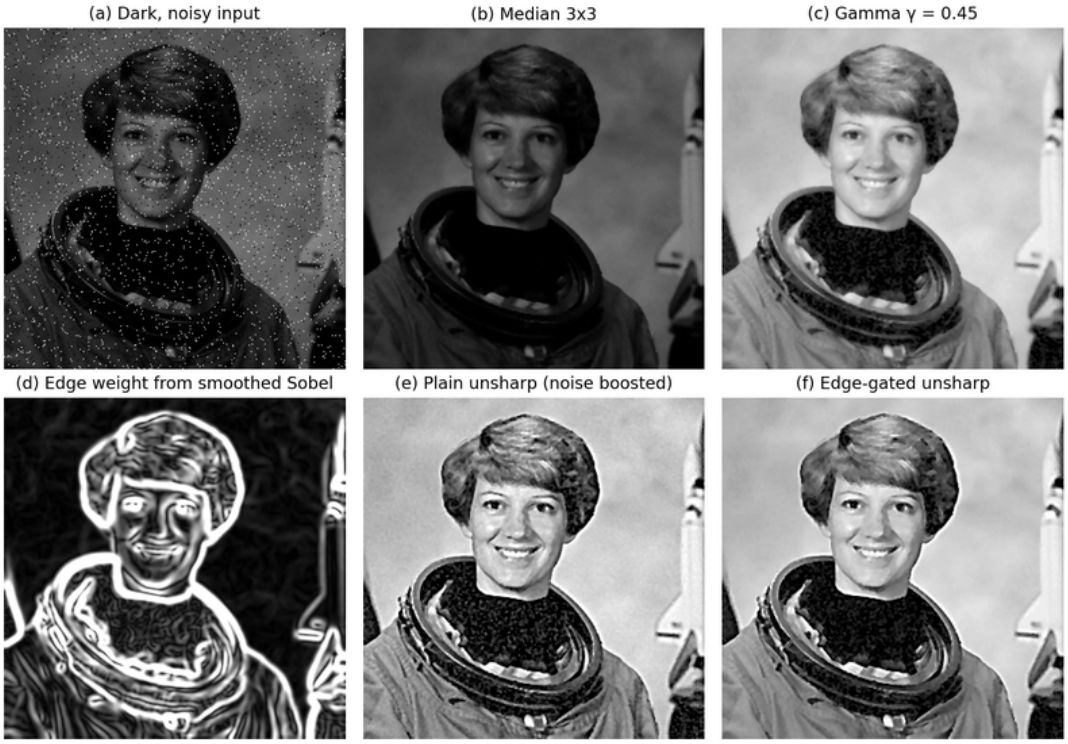


圖 *3.8* — 四步驟的增強流程：(*a*) 輸入、(*b*) *3×3* 中值、(*c*) *gamma 0.45*、(*d*) 由平滑後 *Sobel* 大小得到的邊緣權重、(*e*) 供比較的一般反銳化遮罩、(*f*) 由 (*d*) 控制的反銳化遮罩。

每一步都用到本章的一個概念。步驟順序遵循三條經驗法則：在任何會放大差異的步驟之前先去除離群值；用點運算修正整體色調；只在訊號強於雜訊的地方銳化。

現代觀點

本章的技術已有數十年歷史，卻至今隨處可見：CLAHE 是醫學影像與遙測的標準前處理步驟，gamma 與色調曲線存在於每一條相機處理流程，高斯、中值與 Sobel 濾波器則存在於每一套視覺函式庫。後來的研究朝三個方向推進：更聰明的對比增強、能平滑卻不模糊邊緣的濾波器，以及兩者的學習式替代方案。下面的綜述是很好的入門點。

對比增強與直方圖方法。 Pizer 等人 [2] 至今仍是自適應直方圖等化的代表性分析：說明區域對應如何運作、AHE 為何在均勻區域過度增強雜訊，以及讓這個方法變得實用的各種變體（包括對比限制與在小塊對應之間做內插）。Zuiderveld 的章節 [3] 是後來各函式庫沿用的精簡 CLAHE 實作，Reza [4] 則展示了同一演算法如何對應到即時硬體。在直方圖規定化方面，Coltuc、Bolon 與 Chassery [5] 說明如何用區域平均嚴格排序像素，以得到完全符合目標的直方圖。若想要更全面的領域地圖，Qi 等人 [11] 綜述了過去二十年的影像增強方法，將其分為監督式與非監督式演算法，並把品質評估列為第三個面向；它的價值在於並排討論各類方法的限制、優點與缺點。

保邊濾波器。方框與高斯濾波器的弱點是會跨越邊緣取平均。Tomasi 與 Manduchi 的**雙邊濾波器**（bilateral filter）[6] 在空間高斯權重之外，再乘上一個隨強度差異遞減的權重，讓邊緣另一側的像素幾乎沒有貢獻。Paris、Kornprobst、Tumblin 與 Durand 的綜述 [7] 解釋了它的行為、快速實作，以及色調映射、細節增強等應用，在這些應用中影像會被拆成平滑的基底層與細節層。He、Sun 與 Tang 的**導引濾波器**（guided filter）[8] 假設輸出是導引影像的區域線性函數；它只需要以方框濾波器計算區域平均與變異數，所以計算量與半徑無關，而且在強烈邊緣附近的表現比雙邊濾波器好，後者在細節增強時可能產生梯度反轉（gradient reversal）的瑕疵。Milanfar 的教學文章 [9] 把雙邊、非局部平均（non-local means）等濾波器統一看作資料自適應的加權平均，即卷積核權重取決於影像本身，並用線性代數工具加以分析。Gingichashvili 與 Lischinski [10] 指出眾多保邊濾波器過去大多只靠肉眼比較，並提出系統化的評估方法與共同基準。

深度學習改變了什麼。最有意思的發現是，本章有多少內容在學習式系統內部存活下來。許多成功的增強網路並不直接輸出像素，而是預測某個傳統運算子的參數，再套用它。

- **學習曲線。** Zero-DCE [13]（2020）預測逐像素的高階曲線（反覆套用形如 $s = r + a\,r(1 - r)$ 的簡單二次式，a 逐像素預測）來提亮低光影像。它的訓練完全不需要參考影像，而是使用直接評分增強結果的非參考損失函數。這就是 gamma 與對比拉伸的概念，只是曲線由一個小型 CNN 逐像素決定。
- **學習查找表。** Zeng 等人 [14]（2020）學習幾張基底 3D 色彩查找表，再用一個極小的 CNN 預測每張影像該如何混合它們，能在高解析度照片上即時執行。這是強度轉換的查找表觀點，從灰階延伸到色彩。
- **學習式保邊濾波器。** Gharbi 等人的 HDRNet [15] 在低解析度的雙邊網格（bilateral grid）中預測區域仿射色彩轉換，再以輸入影像為導引在全解析度套用，等於把雙邊濾波器的保邊能力變成可學習的層。Wu 等人 [16] 則把導引濾波器做成可微分的層，可以在網路中端對端訓練。
- **低光增強。** Li 等人的綜述 [12] 依學習策略、網路結構、損失函數與訓練資料，回顧了深度學習的低光影像與影片增強，並提出一個以多款手機在各種照明下拍攝的低光資料集，以及一個比較方法的線上平台。後續模型如 Retinexformer [17]（2023）則結合 Retinex 模型（把影像視為照明乘以反射率）與由照明估計導引的 Transformer 注意力機制。

沒有改變的部分：中值濾波器仍是脈衝雜訊的首選，Sobel 卷積核仍出現在損失函數與邊緣圖中，CLAHE 仍是預設的前處理步驟，而 gamma 處理仍是混用線性與編碼像素值的學習式流程中常

見的錯誤來源。深度模型也往往繼承了傳統方法的失敗模式：光暈、暗部雜訊放大與過度增強，仍是審查者會特別檢查的瑕疵。

重點整理

- 強度轉換 $s = T(r)$ 就是一張查找表，斜率就是局部對比增益。對數與 $\gamma < 1$ 的曲線擴張暗部，$\gamma > 1$ 擴張亮部，分段線性曲線則提供精確控制。
- 直方圖等化以縮放後的 CDF 作為 T，在連續情形下讓輸出 PDF 成為均勻分布，在離散情形下則近似平坦。匹配是把一次等化接上另一次等化的反函數；精確匹配則先用區域平均打破平手。
- 區域方法（AHE、CLAHE、區域平均與變異數規則）會依區域調整；CLAHE 藉由截斷每個區域直方圖來限制對比增益。
- 線性空間濾波就是與卷積核做相關；卷積會翻轉卷積核，並具有漂亮的代數性質。可分離卷積核每個像素只需 $m + n$ 次運算，而非 mn 次。
- 低通濾波器（方框、高斯）以平均去除雜訊，但會模糊邊緣；非線性的中值濾波器是對付脈衝雜訊的工具。
- 銳化就是加入高通細節：拉普拉斯（二階導數）、反銳化遮罩與高增強；梯度與 Sobel 運算子則用來衡量邊緣強度。
- 高通、帶通與帶拒卷積核都能由低通卷積核從脈衝 δ 相減得到；好的處理流程會安排步驟順序，不讓後面的步驟放大前面本該去除的東西。

練習

1. **替霧濛濛的照片設計曲線**。一張 8 位元影像的像素值全落在 90 到 160 之間。寫出把這個範圍對應到 $[0, 255]$ 的分段線性轉換、它的斜率，並說明值為 120 與 122 的兩個相鄰像素會怎樣。

   提示

   $90 \le r \le 160$ 時 $T(r) = 255\,(r - 90)/70$，範圍外則裁切到 $[0, 255]$。斜率為 $255/70 \approx 3.64$，所以相差 2 會變成約 7.3 個灰階。四捨五入後 $T(120) = 109$、$T(122) = 117$。

2. **手算等化**。一張 3 位元（$L = 8$）、共 16 個像素的小影像，灰階 0 到 7 的計數為 $n = [0, 8, 4, 2, 2, 0, 0, 0]$。計算等化查找表與輸出直方圖。為什麼輸出不是平坦的？

   提示

   CDF 為 $[0, 0.5, 0.75, 0.875, 1, 1, 1, 1]$，所以 $s_k = \text{round}(7 \cdot \text{CDF})$ 得到 $[0, 4, 5, 6, 7, 7, 7, 7]$（3.5 捨入為 4）。輸出直方圖為：灰階 4 有 8 個像素、5 有 4 個、6 有 2 個、7 有 2 個。灰階 1 的 8 個像素共用同一個輸入值，查找表無法拆開它們，只有精確直方圖規定化才能做到。

3. **順序重要嗎？**先套用 gamma 曲線再做方框（平均）濾波，和先做方框濾波再套用 gamma 曲線，結果一樣嗎？請證明，或用 2 個像素的平均舉出反例。

提示

不一樣。取兩個像素 0 與 1、$\gamma = 2$。先 gamma：$(0+1)/2 = 0.5$；先平均：$0.5^2 = 0.25$。非線性的點運算與線性濾波器不能交換順序。這也是為什麼模糊 gamma 編碼的影像，不等於模糊線性光。

4. **可不可以分離？**判斷下列卷積核是否可分離，若可以，寫出兩個向量：(a) 中心為 -4 的 3×3 拉普拉斯；(b) 全部為 1 的 3×3 卷積核；(c) $\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$。

提示

(a) 不行：它的秩為 2（但它是兩個可分離卷積核的和，每個軸各一個）。(b) 可以：$[1,1,1]^{\top}[1,1,1]$。(c) 可以：$[1,2,1]^{\top}[1,2,1]$，是高斯的小型二項式近似。可用 `np.linalg.matrix_rank` 驗證。

5. **邊緣上的中值與平均**。一維訊號 $[10,10,10,10,50,50,50,50]$ 是完美的階梯。分別做 3 點平均與 3 點中值（忽略兩端樣本）。接著把位置 1（從 0 起算）的樣本換成值為 255 的脈衝，再做一次。你得到什麼結論？

提示

在乾淨的階梯上，平均得到一段斜坡（$\ldots, 10, 23.3, 36.7, 50, \ldots$），中值則完整保留階梯。加入脈衝後，平均在位置 1 與 2 產生兩個約 92 的升高值；中值在這兩處輸出 10，完全去除脈衝。

6. **設計帶通卷積核**。用 $\sigma_1 = 1$ 與 $\sigma_2 = 3$ 的高斯，在 NumPy 中建立帶通與帶拒卷積核。驗證兩者的權重總和，用它們濾波 `skimage.data.brick()`，並描述各自保留了哪些結構。

提示

先把兩個卷積核補成相同大小（較小者補零），再計算 $K_1 - K_2$ 與 $\delta - (K_1 - K_2)$。總和應約為 0 與 1。帶通輸出保留灰縫線條與磚塊尺度的紋理，去除整體明暗與最細的顆粒；帶拒輸出保留明暗與細顆粒，但減弱了磚塊圖樣。

參考文獻

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 3. publisher page
2. S. M. Pizer, E. P. Amburn, J. D. Austin, R. Cromartie, A. Geselowitz, T. Greer, B. ter Haar Romeny, J. B. Zimmerman and K. Zuiderveld, "Adaptive histogram equalization and its variations," *Computer Vision, Graphics, and Image Processing*, 1987. doi
3. K. Zuiderveld, "Contrast limited adaptive histogram equalization," in P. S. Heckbert (ed.), *Graphics Gems IV*, Morgan Kaufmann, 1994, pp. 474–485. entry
4. A. M. Reza, "Realization of the contrast limited adaptive histogram equalization (CLAHE) for real-time image enhancement," *Journal of VLSI Signal Processing*, 2004. doi

5. D. Coltuc, P. Bolon and J.-M. Chassery, "Exact histogram specification," *IEEE Transactions on Image Processing*, 2006. doi
6. C. Tomasi and R. Manduchi, "Bilateral filtering for gray and color images," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 1998. pdf
7. S. Paris, P. Kornprobst, J. Tumblin and F. Durand, "Bilateral filtering: Theory and applications," *Foundations and Trends in Computer Graphics and Vision*, vol. 4, no. 1, pp. 1–73, 2009. doi
8. K. He, J. Sun and X. Tang, "Guided image filtering," in *Proc. European Conference on Computer Vision (ECCV)*, 2010, pp. 1–14; extended version in *IEEE TPAMI*, 2013. doi
9. P. Milanfar, "A tour of modern image filtering," *IEEE Signal Processing Magazine*, vol. 30, no. 1, pp. 106–128, 2013. IEEE Xplore
10. S. Gingichashvili and D. Lischinski, "Evaluation and comparison of edge-preserving filters," arXiv:2012.13778, 2020. arXiv
11. Y. Qi, Z. Yang, W. Sun et al., "A comprehensive overview of image enhancement techniques," *Archives of Computational Methods in Engineering*, 2021. doi
12. C. Li, C. Guo, L. Han, J. Jiang, M.-M. Cheng, J. Gu and C. C. Loy, "Low-light image and video enhancement using deep learning: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021 (arXiv:2104.10729). arXiv
13. C. Guo, C. Li, J. Guo, C. C. Loy, J. Hou, S. Kwong and R. Cong, "Zero-reference deep curve estimation for low-light image enhancement," in *Proc. IEEE/CVF CVPR*, 2020. arXiv
14. H. Zeng, J. Cai, L. Li, Z. Cao and L. Zhang, "Learning image-adaptive 3D lookup tables for high performance photo enhancement in real-time," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020. arXiv
15. M. Gharbi, J. Chen, J. T. Barron, S. W. Hasinoff and F. Durand, "Deep bilateral learning for real-time image enhancement," *ACM Transactions on Graphics (SIGGRAPH)*, 2017. arXiv
16. H. Wu, S. Zheng, J. Zhang and K. Huang, "Fast end-to-end trainable guided filter," in *Proc. IEEE/CVF CVPR*, 2018. arXiv
17. Y. Cai, H. Bian, J. Lin, H. Wang, R. Timofte and Y. Zhang, "Retinexformer: One-stage Retinex-based transformer for low-light image enhancement," in *Proc. IEEE/CVF ICCV*, 2023. arXiv

IP 04・頻域濾波

傅立葉轉換如何把影像拆成一份「波的配方」，以及修改這份配方如何做到平滑、銳化與去除週期性雜訊。

網站版本: https://blog.shyandram.dev/zh-tw/learn/image-processing/dip-04-frequency-domain/

你將學到

- 傅立葉轉換（Fourier transform）在描述訊號的什麼，以及為什麼「頻率」是描述影像的好方法
- 取樣（sampling）如何運作、什麼是奈奎斯特取樣率（Nyquist rate），以及取樣太稀疏為什麼會產生混疊（aliasing）與摩爾紋（moiré）
- 一維與二維的離散傅立葉轉換（DFT），以及每天都會用到的性質：中心化、對稱性、頻譜與相位、卷積定理
- 頻域濾波的標準步驟，包括避免環繞誤差（wraparound error）所需的填補
- 低通、高通、同態、帶拒與陷波濾波器，以及「理想」濾波器為什麼會產生振鈴（ringing）
- 快速傅立葉轉換（FFT）如何讓這一切變得便宜，以及傅立葉的想法在現代深度學習中出現在哪裡

先看全貌

在第 3 章，我們讓一個小卷積核在像素上滑動來濾波。這一章換一個完全不同的角度看同一張影像。我們不再問「這個像素有多亮？」，而是問「這張影像裡含有多少各種不同的波？」第二種描述叫做影像的**頻譜**（spectrum），產生它的工具就是**傅立葉轉換**。

> **用最簡單的話說**
>
> 想像一個樂團在演奏。你的耳朵聽到的是混在一起的聲音，但樂手可以告訴你：「這裡有低沉的貝斯、中音的鋼琴和弦，還有高音的鈸。」傅立葉轉換對一張圖片做的是同一件事：它告訴你圖裡有多少「慢慢變化」的成分（大片平滑的區域，像貝斯），有多少「快速變化」的成分（銳利的邊緣和細紋理，像鈸）。知道配方之後，你可以把鈸調小聲讓圖變平滑，或把鈸調大聲讓圖變清晰，然後再把整首歌重新播放出來。

為什麼要這麼麻煩？有三個理由。第一，有些操作用「保留這些頻率、拿掉那些頻率」來描述，比設計一個卷積核容易得多。第二，根據卷積定理，大型卷積會變成簡單的乘法，而 FFT 讓它跑得很快。第三，頻率的觀點能解釋許多在像素觀點下很神祕的現象：為什麼縮小影像會冒出奇怪的花紋、為什麼截止得很陡的濾波器會產生漣漪，以及為什麼重複出現的條紋雜訊幾乎可以被完美移除。本章依照 Gonzalez 與 Woods 教科書的章節安排 [1]。

背景

白話版。任何合理的訊號，都可以由許多不同頻率的正弦波和餘弦波相加而成，每個波有自己的強度和起始位置。傅立葉轉換就是要找出這些強度和起始位置。

十九世紀初，傅立葉（Joseph Fourier）主張：週期函數可以寫成正弦與餘弦的加權總和；即使是非週期函數，只要曲線下的面積有限，也能寫成正弦與餘弦的積分。最驚人的是，這個過程不會遺失任何資訊：你可以轉到頻率描述，再一模一樣地轉回來。

對影像而言，這件事有兩層意義。**低頻**是橫跨影像緩慢變化的波，它承載整體亮度與大片平滑區域。**高頻**變化得很快，它承載邊緣、細紋理與雜訊。平滑影像就是削弱高頻；銳化影像就是加強高頻。本章要把這個說法講得精確。

傅立葉分析要到 1965 年快速傅立葉轉換出現之後 [4]，才真正成為實用的影像處理工具；在那之前，對一張影像做轉換實在太慢了。Heideman、Johnson 與 Burrus 後來指出，高斯（Gauss）早在 1805 年左右就用過同樣的想法，遠早於電腦的出現 [5]。

預備概念

白話版。在轉換影像之前，我們需要五樣工具：複數（把一個波的強度和起始位置一起存起來）、傅立葉級數（波的總和）、脈衝（完美的「尖峰」）、傅立葉轉換本身，以及卷積。

複數

複數寫成 $C = R + jI$，其中 R 是實部，I 是虛部，$j = \sqrt{-1}$。它也可以寫成極座標形式

$$C = |C|\, e^{j\theta}, \qquad |C| = \sqrt{R^2 + I^2}, \qquad \theta = \arctan(I/R)$$

其中 $|C|$ 是大小（長度），θ 是角度。歐拉公式 $e^{j\theta} = \cos\theta + j\sin\theta$ 把兩種寫法連在一起。這就是傅立葉轉換使用複指數的原因：一個複數就能同時記錄一個波的**振幅**（amplitude）與**相位**（phase，也就是波從哪裡開始）。複數的共軛為 $C^* = R - jI$。

傅立葉級數

一個以週期 T 重複的函數 $f(t)$ 可以寫成

$$f(t) = \sum_{n=-\infty}^{\infty} c_n\, e^{j\frac{2\pi n}{T}t}, \qquad c_n = \frac{1}{T}\int_{-T/2}^{T/2} f(t)\, e^{-j\frac{2\pi n}{T}t}\, dt$$

其中 n 是整數，c_n 是在一個週期內完成 n 個循環的那個波的（複數）係數，t 是連續變數（時間或位置）。

脈衝與篩選性質

連續的**單位脈衝**（unit impulse）$\delta(t)$ 是一個理想化的尖峰：除了 $t=0$ 以外處處為零，而總面積為 1，即 $\int_{-\infty}^{\infty}\delta(t)\,dt=1$。它最有用的性質是**篩選性質**（sifting property）：

$$\int_{-\infty}^{\infty} f(t)\,\delta(t-t_0)\,dt = f(t_0)$$

其中 t_0 是任意位置。乘上一個平移過的脈衝再積分，就把 f 在 t_0 的值「挑」出來。離散脈衝 $\delta(x)$ 在 $x=0$ 時為 1、其他地方為 0，篩選性質變成 $\sum_x f(x)\,\delta(x-x_0)=f(x_0)$。

脈衝串（impulse train）$s_{\Delta T}(t)=\sum_{n=-\infty}^{\infty}\delta(t-n\Delta T)$ 是一排間距為 ΔT 的脈衝，它就是取樣的數學模型。

連續函數的傅立葉轉換

$f(t)$ 的傅立葉轉換與反轉換為

$$F(\mu)=\int_{-\infty}^{\infty} f(t)\,e^{-j2\pi\mu t}\,dt, \qquad f(t)=\int_{-\infty}^{\infty} F(\mu)\,e^{j2\pi\mu t}\,d\mu$$

其中 μ 是連續頻率（每單位 t 的循環數），$F(\mu)$ 一般是複數。這兩個式子合稱**傅立葉轉換對**，記作 $f(t)\Leftrightarrow F(\mu)$。

用一個例子建立直覺。取一個高度為 A、只在 $|t|\le W/2$ 時不為零的方框函數，它的轉換是

$$F(\mu)=AW\,\frac{\sin(\pi\mu W)}{\pi\mu W}=AW\,\operatorname{sinc}(\mu W)$$

其中 $\operatorname{sinc}(m)=\sin(\pi m)/(\pi m)$。這裡有兩個教訓。第一，有銳利邊緣的函數，其轉換會向外擴散，帶著緩慢衰減的漣漪。第二，兩個域之間存在反比關係：方框越寬，sinc 越窄。在一個域裡窄，在另一個域裡就寬。等一下「理想」濾波器產生振鈴時，這兩個教訓會再次出現。

另外兩組轉換對值得記住。脈衝的轉換是常數：$\delta(t)\Leftrightarrow 1$。間距為 ΔT 的脈衝串，其轉換是另一個間距為 $1/\Delta T$ 的脈衝串：

$$s_{\Delta T}(t)\Leftrightarrow S(\mu)=\frac{1}{\Delta T}\sum_{n=-\infty}^{\infty}\delta\!\left(\mu-\frac{n}{\Delta T}\right)$$

卷積

兩個連續函數的卷積為

$$(f\star h)(t)=\int_{-\infty}^{\infty} f(\tau)\,h(t-\tau)\,d\tau$$

其中 τ 是積分用的虛擬變數。**卷積定理**（convolution theorem）說

$$(f\star h)(t)\Leftrightarrow H(\mu)F(\mu), \qquad f(t)\,h(t)\Leftrightarrow (H\star F)(\mu)$$

一個域裡的卷積，就是另一個域裡的乘法。頻域濾波之所以存在，全靠這一個事實。

取樣與取樣函數的傅立葉轉換

白話版。相機並不會記錄連續的場景，它只在一個格點上量測。格點太疏的話，快速的細節會被誤認成緩慢的變化，你會看到其實不存在的花紋。只要取樣頻率超過最快頻率的兩倍，原則上就能把原訊號完整重建。

取樣就是相乘

把取樣模型化為 $f(t)$ 乘上脈衝串：

$$\tilde{f}(t) = f(t)\, s_{\Delta T}(t) = \sum_{n=-\infty}^{\infty} f(n\Delta T)\, \delta(t - n\Delta T)$$

其中 ΔT 是取樣間隔，$f(n\Delta T)$ 是取樣值。由卷積定理，相乘變成與 $S(\mu)$ 卷積，而與一串脈衝卷積，效果就是把頻譜複製很多份：

$$\tilde{F}(\mu) = \frac{1}{\Delta T} \sum_{n=-\infty}^{\infty} F\!\left(\mu - \frac{n}{\Delta T}\right)$$

所以，**取樣後函數的頻譜，是原頻譜以 $1/\Delta T$ 為間隔、無限重複的週期性複本**。

取樣定理與奈奎斯特取樣率

假設 f 是**帶限**（band-limited）的：當 $|\mu| > \mu_{\max}$ 時 $F(\mu) = 0$。只要複本之間的間距大於一份複本的寬度，它們就不會重疊：

$$\frac{1}{\Delta T} > 2\mu_{\max}$$

這就是**取樣定理**，因 Shannon 而在工程界廣為人知 [2]。$2\mu_{\max}$ 稱為**奈奎斯特取樣率**。取樣速度超過它，取樣值就包含 f 的全部資訊。

重建

如果複本沒有重疊，把 $\tilde{F}(\mu)$ 乘上一個高度為 ΔT、涵蓋 $[-\mu_{\max}, \mu_{\max}]$ 的方框 $H(\mu)$，就能分離出原本的頻譜。在時域，這相當於與 sinc 函數卷積，得到內插公式

$$f(t) = \sum_{n=-\infty}^{\infty} f(n\Delta T)\, \operatorname{sinc}\!\left[\frac{t - n\Delta T}{\Delta T}\right]$$

每個取樣點都貢獻一個以自己為中心的 sinc，加總後的曲線剛好通過所有取樣點。Unser 的回顧論文 [3] 說明了現代實務如何用樣條（spline）等緊湊的核心，來取代不切實際的無限長 sinc，並把它近似得很好。

混疊

若 $1/\Delta T \le 2\mu_{\max}$，相鄰的複本會重疊，尾巴互相疊加。之後不論用什麼濾波器都無法把它們分開：高頻偽裝成了低頻。這就是**混疊**。唯一的解法是預防：在取樣之前用低通濾波器（**抗混疊**濾波器，anti-aliasing filter）去除高頻。

一個小實驗就能看到這種偽裝。以每秒 10 個樣本取樣一個 9 Hz 的餘弦波，此時奈奎斯特上限是 5 Hz：

```
import numpy as np

fs = 10.0                       # samples per second
t = np.arange(0, 2, 1 / fs)     # 2 seconds of samples
x = np.cos(2 * np.pi * 9 * t)  # a 9 Hz tone: above fs/2 = 5 Hz
X = np.abs(np.fft.rfft(x))
freqs = np.fft.rfftfreq(len(x), d=1 / fs)
print("peak at", freqs[X.argmax()], "Hz")   # -> 1.0 Hz, not 9 Hz
```

在這些取樣時間點上，9 Hz 的音和 1 Hz 的音完全無法區分。電影裡馬車車輪看起來倒著轉，也是同一個效應。

一維離散傅立葉轉換

白話版。電腦手上只有有限個取樣值，所以它使用有限版本的傅立葉轉換。DFT 吃進 M 個數字，吐出 M 個數字，每個數字代表某一個特定的波有多少。

定義

對取樣值 $f(0), f(1), \dots, f(M-1)$：

$$F(u) = \sum_{x=0}^{M-1} f(x)\, e^{-j2\pi ux/M}, \qquad f(x) = \frac{1}{M} \sum_{u=0}^{M-1} F(u)\, e^{j2\pi ux/M}$$

其中 $x = 0, \dots, M-1$ 是取樣點的索引，$u = 0, \dots, M-1$ 是離散頻率的索引（頻率 u 代表在 M 個樣本內完成 u 個循環），係數 $1/M$ 放在反轉換上（也有慣例把它拆成兩邊各 $1/\sqrt{M}$）。

與取樣的關係

這個公式從哪裡來？取樣後函數的轉換 $\tilde{F}(\mu)$ 是連續的，並以 $1/\Delta T$ 為週期。在其中一個週期內等間隔地取 M 個點，$\mu = m/(M\Delta T)$，$m = 0, \dots, M-1$，得到的正好就是上面的 DFT。所以 DFT 就是**取樣資料之轉換、在一個週期內的取樣值**。DFT 相鄰頻率之間的間距為

$$\Delta u = \frac{1}{M\Delta T}$$

其中 $M\Delta T$ 是全部樣本所涵蓋的總長度。紀錄越長，頻率解析度越細。

實務上有兩個後果。因為在頻率上也取了樣，DFT 會把資料當作以 M 為週期重複。而且 $F(u)$ 與 $f(x)$ 都是週期性的：$F(u) = F(u + M)$、$f(x) = f(x + M)$。

這個定義其實就是一次矩陣與向量的乘法，很容易驗證：

```
import numpy as np

def dft(f):
    M = len(f)
    x = np.arange(M)
    u = x.reshape(-1, 1)
    W = np.exp(-2j * np.pi * u * x / M)   # M x M matrix of complex exponentials
    return W @ f

f = np.random.default_rng(0).standard_normal(64)
print(np.allclose(dft(f), np.fft.fft(f)))  # True
```

推廣到二變數函數

白話版。影像有兩個方向，所以波也有兩個頻率：一個橫向、一個縱向。二維的波看起來像一組平行條紋，兩個頻率決定了條紋有多密、朝哪個方向。

二維脈衝與連續轉換

二維脈衝 $\delta(t, z)$ 的體積為 1，篩選方式和一維相同：$\iint f(t,z)\,\delta(t - t_0, z - z_0)\,dt\,dz = f(t_0, z_0)$。二維連續轉換為

$$F(\mu, \nu) = \int_{-\infty}^{\infty}\int_{-\infty}^{\infty} f(t,z)\, e^{-j2\pi(\mu t + \nu z)}\, dt\, dz$$

其中 t, z 是空間座標，μ, ν 是對應的頻率。

二維取樣與混疊

以間距 ΔT、ΔZ 在格點上取樣，會把頻譜複製到一個二維格子上。一張帶限影像（在 $|\mu| \le \mu_{\max}$、$|\nu| \le \nu_{\max}$ 之外為零）可以被完整復原的條件是

$$\frac{1}{\Delta T} > 2\mu_{\max}, \qquad \frac{1}{\Delta Z} > 2\nu_{\max}$$

影像經常違反這條規則。真實場景含有銳利邊緣，而銳利邊緣根本不是帶限的，所以每張數位影像多少都有混疊。當你用丟棄像素的方式**降取樣**（縮小）影像時，混疊就會變得明顯。圖 4.1 使用一張波帶片（zone plate），它的頻率隨著與中心的距離增加。每 4 個像素只留 1 個，外圈的細環就變成原圖中根本不存在的假圓圈。先模糊再取樣，就能移除那些無法保留的頻率，假圓圈也隨之消失。

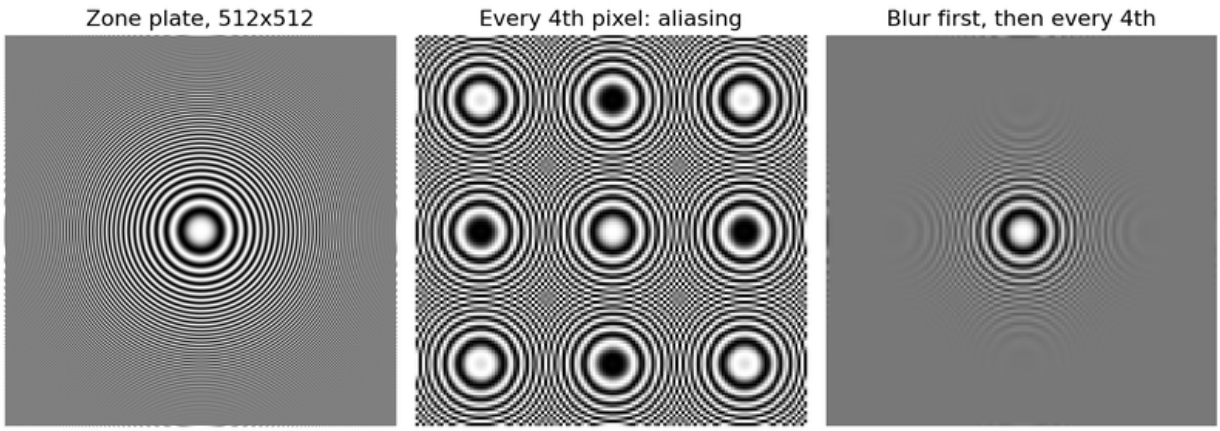


圖 *4.1* — 二維混疊。左：頻率向外遞增的波帶片。中：每 *4* 個像素取 *1* 個，產生假的圓環（混疊）。右：降取樣前先做高斯模糊，移除無法表示的頻率，原本細節所在之處變成一片均勻的灰。

摩爾紋

摩爾紋是日常生活中看得到的混疊：當細密的週期紋理（襯衫的織紋、紗窗、印刷照片的網點）被間距相近的感測器格點取樣時，兩種週期互相「拍」出又大又慢的波浪狀條帶。圖 4.1 中間那張就是這種摩爾紋。相機在感測器前面放一片光學抗混疊濾鏡來減輕它；軟體則在縮放之前先模糊。

二維 DFT

對一張 $M \times N$ 的影像 $f(x,y)$：

$$F(u,v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y)\, e^{-j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)}, \qquad f(x,y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u,v)\, e^{j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)}$$

其中 x, y 是像素座標（列、行），u, v 是離散頻率索引，$F(u,v)$ 是 $M \times N$ 的複數陣列。F 的每一個元素都描述一個覆蓋整張影像的二維波。

二維 DFT 的性質

白話版。頻譜的行為是可以預測的。移動物體不會改變頻譜的大小；旋轉物體會讓頻譜跟著旋轉；(移位後的）頻譜中心存的是平均亮度。最令人意外的是：我們在圖中認得出來的東西，大部分存在相位裡，而不是在大小裡。

空間間隔與頻率間隔

如果影像是以間距 ΔT、ΔZ 取樣的，DFT 的頻率間距為

$$\Delta u = \frac{1}{M\Delta T}, \qquad \Delta v = \frac{1}{N\Delta Z}$$

視野越大，頻率解析度越細；像素越小，DFT 能表示的最高頻率就越高。

平移與旋轉

平移影像等於把轉換乘上一個線性相位，反之亦然：

$$f(x - x_0, y - y_0) \Leftrightarrow F(u,v)\, e^{-j2\pi\left(\frac{ux_0}{M} + \frac{vy_0}{N}\right)}$$

其中 (x_0, y_0) 是平移量。因為 $|e^{-j\cdot}| = 1$，**大小 $|F|$ 不受平移影響**，只有相位改變。若使用極座標 $x = r\cos\theta$、$y = r\sin\theta$ 以及 $u = \omega\cos\varphi$、$v = \omega\sin\varphi$，把 f 旋轉 θ_0，F 也會旋轉同樣的角度：$f(r, \theta + \theta_0) \Leftrightarrow F(\omega, \varphi + \theta_0)$。

週期性與中心化

DFT 與其反轉換在兩個方向上都是週期性的：對任意整數 k_1, k_2，$F(u,v) = F(u + k_1 M, v + k_2 N)$。因此，原始的 DFT 陣列把零頻率放在左上角，四個四分之一週期在陣列中央相遇。不論是觀察頻譜還是設計濾波器，把零頻率移到中央都方便得多。利用平移性質，令 $x_0 = M/2$、$y_0 = N/2$：

$$f(x,y)\,(-1)^{x+y} \Leftrightarrow F(u - M/2,\ v - N/2)$$

此式在 M、N 為偶數時成立。在 NumPy 中，轉換之後呼叫 `np.fft.fftshift` 可以達到同樣的重新排列。

對稱性

對實數影像而言，轉換具有**共軛對稱性**：$F(-u,-v) = F^*(u,v)$。因此大小是偶函數，$|F(-u,-v)| = |F(u,v)|$，相位則是奇函數。頻譜中的每一個亮點，都有一個對中心鏡像的雙胞胎。設計陷波濾波器時會用到這一點。

傅立葉頻譜與相位角

令 $F(u,v) = R(u,v) + jI(u,v)$，則

$$|F(u,v)| = \sqrt{R^2 + I^2}, \qquad \phi(u,v) = \arctan\left[\frac{I(u,v)}{R(u,v)}\right], \qquad P(u,v) = |F(u,v)|^2$$

分別是**傅立葉頻譜**（大小）、**相位角**與**功率譜**（power spectrum）。零頻率項稱為 **DC 成分**，$F(0,0) = MN\bar{f}$，其中 $\bar{f}$ 是平均灰階值。由於 $F(0,0)$ 通常比其他值大上數千倍，顯示頻譜時會用 $\log(1 + |F|)$。

```
import numpy as np
from skimage import data, img_as_float
```

```
f = img_as_float(data.camera())
F = np.fft.fft2(f)
Fc = np.fft.fftshift(F)                         # move (0,0) to the centre
spectrum = np.log1p(np.abs(Fc))                 # log(1+|F|) for display
phase = np.angle(Fc)

M, N = f.shape
y, x = np.mgrid[0:M, 0:N]
print(np.allclose(np.fft.fft2(f * (-1.0) ** (x + y)), Fc))  # True for even M, N
print(np.isclose(F[0, 0].real, f.sum()))                    # DC term = sum of pixels
```

圖 4.2 顯示簡單圖案與頻譜的對應。矩形的轉換是二維 sinc：一個明亮的十字，沿著矩形短邊方向的那一臂比較長（空間中窄、頻率中寬）。平移矩形，頻譜完全不變；旋轉矩形，頻譜跟著旋轉。餘弦光柵只含一個二維頻率，所以頻譜只有三個點：DC 項與一對對稱的點。高斯函數轉換後仍是高斯函數。

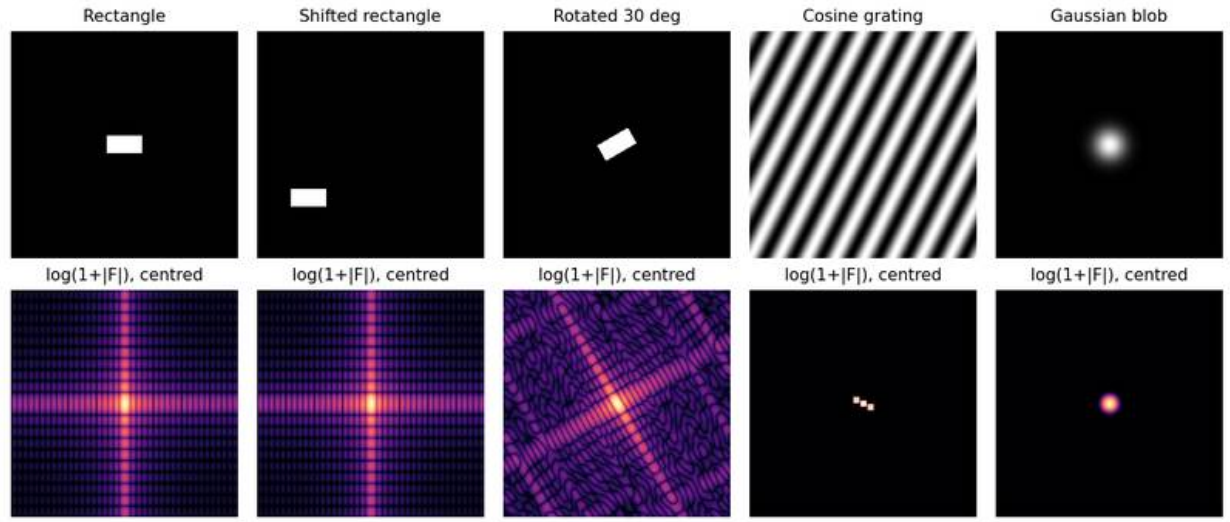


圖 4.2 — 簡單圖案（上）與中心化的對數頻譜（下）。平移不改變大小；旋轉使頻譜跟著旋轉。光柵頻譜的三個點已經放大，方便觀察。

相位與大小的交換實驗

轉換中的哪一部分，承載了我們認得出來的內容？取兩張影像，保留其中一張的大小、另一張的相位，再做反轉換。Oppenheim 與 Lim 讓這個實驗廣為人知 [7]。

```
import numpy as np
from skimage import color, data, img_as_float

a = img_as_float(data.camera())
b = img_as_float(color.rgb2gray(data.astronaut()))
```

```
A, B = np.fft.fft2(a), np.fft.fft2(b)
mixed = np.real(np.fft.ifft2(np.abs(A) * np.exp(1j * np.angle(B))))
# 'mixed' looks like the astronaut, not the cameraman
```

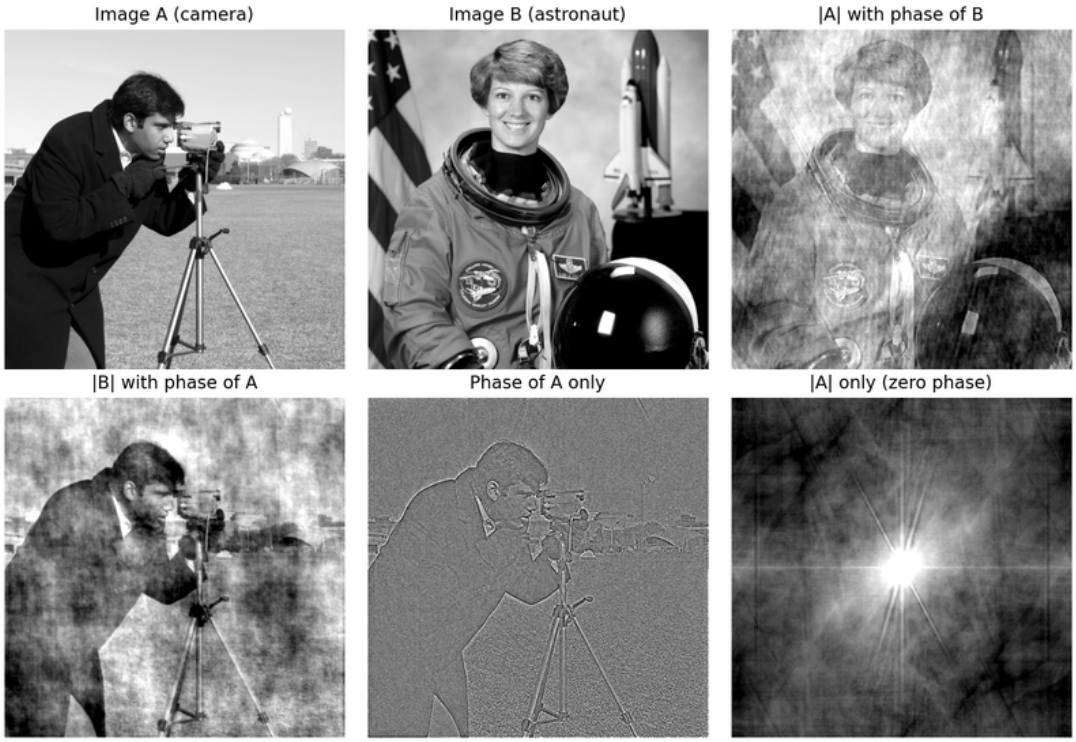


圖 *4.3* — 交換大小與相位。每張混合影像看起來都像「提供相位」的那一張。只有相位（大小設為 *1*）仍保留邊緣與構圖；只有大小（相位設為零）則是一團認不出來的光斑。

結果非常明確：**結構由相位承載**。大小告訴你每個頻率上有多少能量，而大多數自然照片的這部分都差不多。相位告訴你各個波在哪裡對齊，而邊緣恰好就是許多波對齊的地方。這也是為什麼後面的濾波器都設計成不動相位。

二維卷積定理

離散版本的卷積定理為

$$(f \star h)(x,y) \;\Leftrightarrow\; F(u,v)\,H(u,v), \qquad (f \star h)(x,y) = \sum_{m=0}^{M-1}\sum_{n=0}^{N-1} f(m,n)\,h(x-m,\,y-n)$$

其中索引 $x-m$ 與 $y-n$ 要對 M、N **取餘數**。這是**循環卷積**（circular convolution）：DFT 假設影像無限重複，所以從右邊緣推出去的東西會從左邊冒出來。

環繞誤差與填補

循環卷積並不是我們要的，它會把影像左右邊界的內容混在一起。這就是**環繞誤差**。解法是把兩個陣列都補零，讓週期之間的距離大到複本永遠不會碰在一起。若 f 大小為 $A \times B$、h 大小為 $C \times D$，則填補後的大小 $P \times Q$ 應滿足

$$P \geq A + C - 1, \qquad Q \geq B + D - 1$$

如果濾波器直接在頻域中指定、而且和影像一樣大，常見的選擇就是 $P = 2M$、$Q = 2N$。

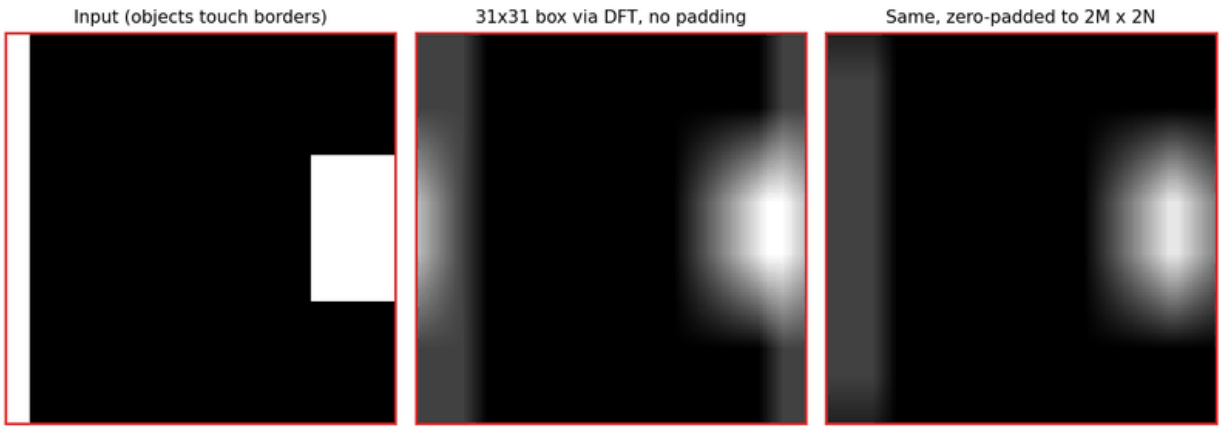


圖 *4.4* — 環繞誤差。不填補時（中），模糊讓右側方塊的光漏到左邊界，左側長條的光也漏到右邊界。補零到兩倍大小（右）才得到正確的線性卷積。

頻域濾波基礎

白話版。先轉換影像，再把每個頻率乘上一個代表「保留」或「移除」的數字，最後轉換回來。這一整份數字清單就是濾波器。

濾波步驟

給定 $M \times N$ 的影像 $f(x, y)$ 與濾波器轉移函數（transfer function）$H(u, v)$：

1. 選擇填補大小 $P = 2M$、$Q = 2N$。
2. 在影像後方補零（或補鏡像像素），得到 $P \times Q$ 的填補影像 $f_p(x, y)$。
3. 把 f_p 乘上 $(-1)^{x+y}$，使其轉換中心化。
4. 計算 DFT，得到 $F(u, v)$。
5. 建立大小為 $P \times Q$、中心位於 $(P/2, Q/2)$ 的實數對稱濾波器 $H(u, v)$，並計算 $G(u, v) = H(u, v)\, F(u, v)$。
6. 計算 $g_p(x, y) = \mathrm{Re}\{\mathcal{F}^{-1}[G(u, v)]\}\,(-1)^{x+y}$。
7. 裁切左上角 $M \times N$ 的區域，得到 $g(x, y)$。

第 6 步取實部，只是丟掉浮點數捨入誤差造成的微小虛部；對實數影像與實數對稱濾波器而言，精確的結果本來就是實數。

```
import numpy as np

def filter_freq(f, H_of_D):
    """Filter image f with a radially symmetric, zero-phase transfer function.
    H_of_D maps a distance array D (from the centre of the P x Q grid) to H."""
    M, N = f.shape
    P, Q = 2 * M, 2 * N                                  # 1. padding size
    fp = np.zeros((P, Q)); fp[:M, :N] = f                # 2. zero-pad
    y, x = np.mgrid[0:P, 0:Q]
    fp = fp * (-1.0) ** (x + y)                          # 3. centre the transform
    F = np.fft.fft2(fp)                                  # 4. DFT
    D = np.hypot(y - P / 2, x - Q / 2)
    G = H_of_D(D) * F                                    # 5. multiply
    gp = np.real(np.fft.ifft2(G)) * (-1.0) ** (x + y)    # 6. inverse, undo centring
    return gp[:M, :N]                                    # 7. crop
```

補零之後再模糊，右側與下側邊緣會出現一條變暗的帶狀區域，因為濾波器把黑色像素也平均進來了。改用鏡像填補（`np.pad(f, ..., mode="reflect")`）可以避免這個問題，本章的繪圖程式就是這樣做的。

零相位濾波器

本章所有濾波器都是**實數**，並且對中心**對稱**。把一個複數乘上一個實數，只會縮放它的大小、不會改變它的角度（頂多正負號），所以這些濾波器只改變每個頻率留下多少，從不改變各個波在哪裡對齊。這類濾波器稱為**零相移**（zero-phase-shift）濾波器。看過圖 4.3 就知道為什麼必須如此：改變相位會讓邊緣跑掉。

空間濾波器與頻域濾波器的對應

由卷積定理，每個頻域濾波器 $H(u,v)$ 都對應一個空間卷積核 $h(x,y) = \mathcal{F}^{-1}\{H(u,v)\}$，每個空間卷積核也都有它的轉移函數。高斯是最乾淨的例子。在一維中，

$$H(u) = A\,e^{-u^2/2\sigma^2} \quad \Leftrightarrow \quad h(x) = \sqrt{2\pi}\,\sigma A\,e^{-2\pi^2\sigma^2 x^2}$$

其中 σ 是高斯在頻域中的寬度。頻域中寬的高斯，對應到空間中窄的卷積核，反之亦然。因此，一個常見的實務策略是：在頻域中設計（「讓 40 個循環以下通過」很容易描述），然後實作一個近似 h 的小型空間卷積核。

什麼時候走頻域比較快？用 $k \times k$ 的卷積核做空間卷積，每個像素大約要 k^2 次乘法；走 FFT 的話，每個像素的成本只隨 $\log(MN)$ 增加。小卷積核（3×3、5×5）走空間域划算；寬達數十個像素的卷積核，走 FFT 划算。

用低通頻域濾波器平滑影像

白話版。低通濾波器保留慢的波、移除快的波，所以影像會變模糊。濾波器從「保留」降到「移除」的方式非常重要：降得太突然，就會產生漣漪。

令 $D(u,v)$ 為與 $P \times Q$ 頻率矩形中心的距離，

$$D(u,v) = \sqrt{(u - P/2)^2 + (v - Q/2)^2}$$

並令 $D_0 > 0$ 為**截止頻率**（cutoff frequency）。

理想低通濾波器（ILPF）

$$H(u,v) = \begin{cases} 1 & D(u,v) \le D_0 \\ 0 & D(u,v) > D_0 \end{cases}$$

它讓半徑 D_0 的圓內所有頻率通過，圓外全部擋掉。選擇 D_0 的一個好方法，是看它保留了總功率的多少比例：$\alpha = 100 \sum_{D \le D_0} P(u,v) / \sum_{u,v} P(u,v)$。

巴特沃斯低通濾波器（BLPF）

$$H(u,v) = \frac{1}{1 + [D(u,v)/D_0]^{2n}}$$

其中 n 是**階數**（order）。在 $D = D_0$ 時濾波器值恰為 0.5。n 小時衰減平緩；$n \to \infty$ 時 BLPF 趨近理想濾波器。

高斯低通濾波器（GLPF）

$$H(u,v) = e^{-D^2(u,v)/2D_0^2}$$

其中 D_0 扮演 σ 的角色。在 $D = D_0$ 時濾波器值為 $e^{-1/2} \approx 0.607$。

振鈴

圖 4.5 以相同的截止頻率比較三者。理想濾波器產生明顯的**振鈴**：邊緣的淡淡分身一圈圈向外重複，就像往池塘丟石頭激起的漣漪。原因正是前面提過的方框與 sinc 轉換對。頻域中的陡峭截止，對應到一個中央主瓣、外圍環繞正負交替旁瓣的空間卷積核。邊緣與這種卷積核卷積之後，旁邊就會畫出一串淡淡的明暗回音。

高斯則是另一個極端：它的空間卷積核也是高斯，永遠為正，所以絕不會振鈴。巴特沃斯介於兩者之間：1 階完全沒有振鈴，2 階有幾乎看不見的負旁瓣，階數越高振鈴越明顯。2 階是在陡峭截止與乾淨結果之間常見的折衷。

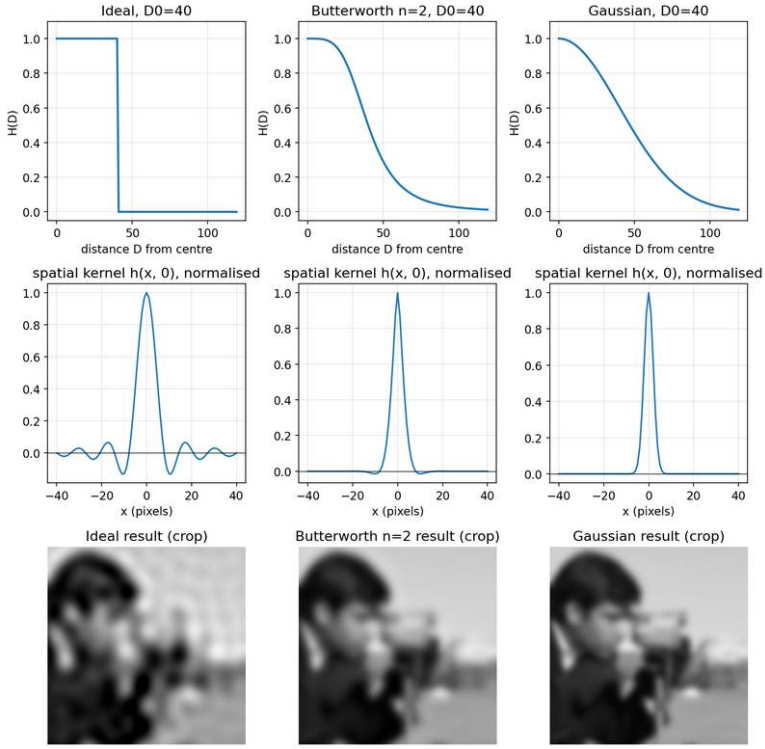


圖 *4.5* — 截止頻率皆為 $D0 = 40$ 的理想、巴特沃斯（$n = 2$）與高斯低通濾波器。上：徑向剖面 $H(D)$。中：對應的空間卷積核，只有理想濾波器有明顯的負旁瓣。下：理想濾波器的結果在邊緣周圍出現漣漪（振鈴），高斯則沒有。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

```
D0 = 40
ideal  = lambda D: (D <= D0).astype(float)
butter = lambda D, n=2: 1 / (1 + (D / D0) ** (2 * n))
gauss  = lambda D: np.exp(-D**2 / (2 * D0**2))
smooth = filter_freq(f, gauss)
```

低通濾波有很多不起眼但實用的用途：把低解析度掃描中斷掉的字跡筆畫連起來、柔化人像的皮膚紋理，以及壓抑遙測影像中的掃描線紋路。

用高通濾波器銳化影像

白話版。高通濾波器正好相反：它丟掉慢的波、保留快的波，所以只剩下邊緣與紋理。把其中一部分加回原圖，影像看起來就更清晰。

理想、巴特沃斯與高斯高通濾波器

每個高通濾波器都可以由低通濾波器得到：

$$H_{\text{HP}}(u,v) = 1 - H_{\text{LP}}(u,v)$$

所以高斯高通為 $1 - e^{-D^2/2D_0^2}$，巴特沃斯高通為 $1/(1+[D_0/D]^{2n})$，理想高通則是圓內為 0、圓外為 1。振鈴的道理一樣：理想的會振鈴，高斯的不會。由於高通濾波器令 $H(0,0)=0$，輸出的平均值為零，所以平坦區域在顯示縮放後會變成灰色（零）。

頻域中的拉普拉斯運算子

第 3 章的拉普拉斯運算子（Laplacian）$\nabla^2 f = \partial^2 f/\partial x^2 + \partial^2 f/\partial y^2$ 也有轉移函數。微分等於把轉換乘上 $j2\pi u$，在兩個方向各微分兩次，得到

$$H(u,v) = -4\pi^2\left[(u-P/2)^2 + (v-Q/2)^2\right] = -4\pi^2 D^2(u,v)$$

此處頻率以「每張影像的循環數」為單位。銳化後的影像為 $g(x,y) = f(x,y) - \nabla^2 f(x,y)$；用減號是因為 H 的中心為負。實務上拉普拉斯的輸出比 f 大得多，相減前必須先把兩者縮放到可比較的範圍（例如 f 在 $[0,1]$，$\nabla^2 f$ 除以其最大絕對值）。

反銳化遮罩、高提升與高頻強調

反銳化遮罩（unsharp masking）先減去模糊版本得到細節「遮罩」，再把它加回去：

$$g_{\text{mask}}(x,y) = f(x,y) - f_{\text{LP}}(x,y), \qquad g(x,y) = f(x,y) + k\, g_{\text{mask}}(x,y)$$

其中 f_{LP} 是低通濾波後的 f，$k \geq 0$ 是權重（$k=1$：反銳化遮罩；$k>1$：高提升，high-boost）。在頻域中，整個操作只是一個轉移函數 $H = 1 + k\,H_{\text{HP}}$。把它一般化，就得到**高頻強調**（high-frequency emphasis）：

$$H_{\text{HFE}}(u,v) = k_1 + k_2\, H_{\text{HP}}(u,v)$$

其中 $k_1 \geq 0$ 控制低頻（以及 DC 項）留下多少，$k_2 \geq 0$ 控制高頻被放大多少。選 $k_1 < 1$ 也會降低整體對比，所以高頻強調之後常接著做直方圖等化（histogram equalization）。

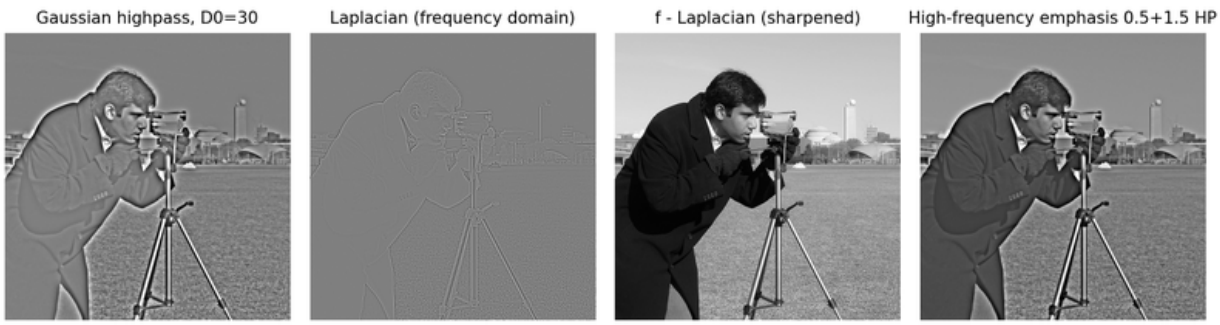


圖 *4.6* — 頻域銳化。由左至右：高斯高通（$D0 = 30$）、以 $H = -4\pi^2 D^2$ 計算的拉普拉斯、原圖減去縮放後的拉普拉斯，以及 $k1 = 0.5$、$k2 = 1.5$ 的高頻強調。

同態濾波

一個簡單的成像模型說，每個像素是**照度**（illumination）$i(x,y)$（照到場景上的光有多少）與**反射率**（reflectance）$r(x,y)$（表面反射多少光）的乘積：$f = i \cdot r$。照度在場景中變化緩慢；反射率則在物體邊界處劇烈變化。我們想壓低照度、加強反射率，但乘積的傅立葉轉換並不等於轉換的乘積。

Oppenheim、Schafer 與 Stockham 提出的技巧 [8] 是先取對數，把乘積變成和：

$$z(x,y) = \ln f(x,y) = \ln i(x,y) + \ln r(x,y)$$

接著用一個對低頻與高頻有不同處理的轉移函數對 z 濾波，最後取指數：

$$s = \mathcal{F}^{-1}\{H\,Z\}, \qquad g(x,y) = e^{s(x,y)}, \qquad H(u,v) = (\gamma_H - \gamma_L)\left[1 - e^{-c\,D^2(u,v)/D_0^2}\right] + \gamma_L$$

其中 $\gamma_L < 1$ 削弱低頻（照度），$\gamma_H > 1$ 放大高頻（反射率），c 控制過渡的陡峭程度，D_0 控制過渡的位置。結果同時壓縮了動態範圍、又增強了對比。

```
def homomorphic(f, gL=0.4, gH=1.6, c=1.0, D0=30, eps=1e-3):
    z = np.log(f + eps)                       # eps avoids log(0)
    H = lambda D: (gH - gL) * (1 - np.exp(-c * D**2 / D0**2)) + gL
    return np.exp(filter_freq(z, H)) - eps
```

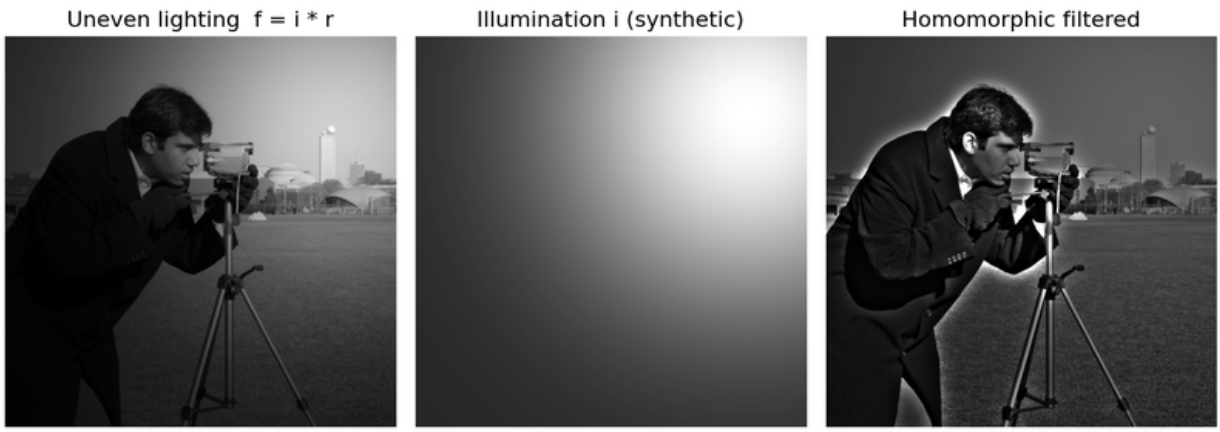


圖 *4.7* — 同態濾波。合成的照度場讓影像左側變暗（左、中）。以 $\gamma L = 0.4$、$\gamma H = 1.6$ 在對數域濾波後，照明變得均勻，暗部細節也被提升（右），代價是強邊緣周圍出現輕微光暈。

選擇性濾波

白話版。有時候問題不在於「細節太多」或「太少」，而是某個特定、重複出現的花紋，例如電子干擾造成的條紋。在頻譜中，這種花紋只是幾個亮點。選擇性濾波器只移除那幾個點，其他什麼都不動。

帶拒與帶通濾波器

帶拒（bandreject）濾波器移除以半徑 C_0 為中心、寬度為 W 的一圈頻率。高斯版本為

$$H_{\mathrm{BR}}(u,v) = 1 - \exp\left[-\left(\frac{D^2(u,v) - C_0^2}{D(u,v)\,W}\right)^2\right]$$

n 階巴特沃斯版本為

$$H_{\mathrm{BR}}(u,v) = \frac{1}{1 + \left[\dfrac{D(u,v)\,W}{D^2(u,v) - C_0^2}\right]^{2n}}$$

其中 C_0 是環的中心半徑，W 是環的寬度。**帶通**(bandpass)濾波器只保留那一圈：$H_{\mathrm{BP}} = 1 - H_{\mathrm{BR}}$。環狀濾波器適合在所有方向上都出現在固定距離的雜訊，但這在實務中並不常見。

陷波濾波器

陷波濾波器（notch filter）只作用在選定頻率周圍的小鄰域。由於實數影像的頻譜共軛對稱，每個陷波都必須搭配它對中心的鏡像。具有 Q 對陷波的陷波拒斥濾波器，是由多個中心移到亮點位置的高通濾波器相乘而成：

$$H_{\mathrm{NR}}(u,v) = \prod_{k=1}^{Q} H_k(u,v)\,H_{-k}(u,v)$$

其中 H_k 是中心位於 (u_k, v_k) 的高通濾波器（理想、巴特沃斯或高斯），H_{-k} 是中心位於 $(-u_k, -v_k)$ 的同一個濾波器，兩者都相對於頻率矩形的中心量測。距離為

$$D_k(u,v) = \sqrt{(u - P/2 - u_k)^2 + (v - Q/2 - v_k)^2}, \qquad D_{-k}(u,v) = \sqrt{(u - P/2 + u_k)^2 + (v - Q/2 + v_k)^2}$$

陷波通過濾波器為 $1 - H_{\mathrm{NR}}$。套用陷波通過濾波器再反轉換，可以單獨取出干擾花紋本身，用來確認你移除的是雜訊而不是影像內容。

```
import numpy as np
from skimage import data, img_as_float

f = img_as_float(data.camera())
M, N = f.shape
y, x = np.mgrid[0:M, 0:N]
g = f + 0.2 * np.cos(2 * np.pi * (40 * x / N + 25 * y / M))   # add a ripple
```

```
G = np.fft.fftshift(np.fft.fft2(g))
u, v = y - M // 2, x - N // 2              # centred frequency coordinates
H = np.ones((M, N))
for uk, vk in [(25, 40)]:                   # spike location (row, column offset)
    for s in (+1, -1):                      # the conjugate-symmetric partner
        Dk = np.hypot(u - s * uk, v - s * vk)
        H *= 1 - np.exp(-Dk**2 / (2 * 4.0**2))   # Gaussian notch reject
out = np.real(np.fft.ifft2(np.fft.ifftshift(G * H)))
```

這裡刻意不做填補。干擾在整張影像上是週期性的，填補反而會把它的亮點抹散到許多頻率樣本上，降低陷波的效果。

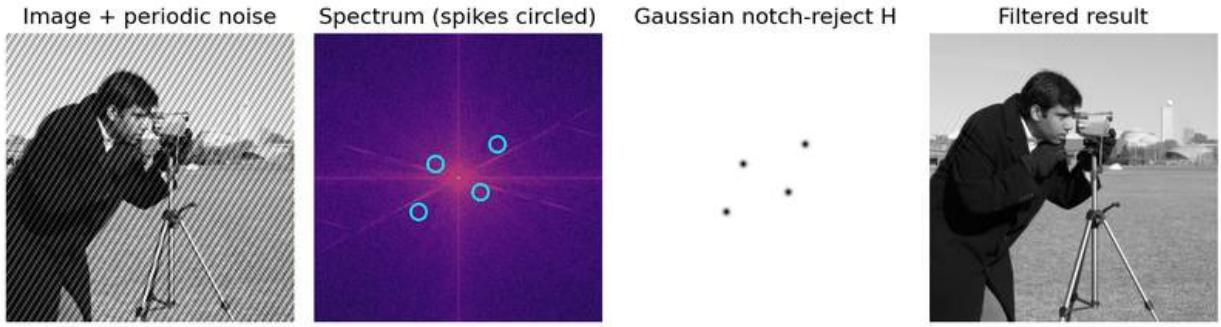


圖 *4.8* — 以陷波濾波去除合成的週期性雜訊。兩組正弦波紋產生四個亮點（兩對共軛點，已圈出）。高斯陷波拒斥濾波器在每個亮點放一個暗點把它們移除，干擾隨之消失。

拖動滑桿，直接比較受干擾與濾波後的影像：

含週期性雜訊

陷波濾波後

帶有斜向條紋干擾的攝影師影像與陷波濾波結果的比較

實作

白話版。二維 DFT 可以拆成許多一維 DFT 來算；反 DFT 可以用正向轉換的程式來算；而 FFT 演算法讓每一次一維轉換快上非常多。

可分離性

二維的核 $e^{-j2\pi(ux/M+vy/N)}$ 可以分解成 $e^{-j2\pi ux/M}\,e^{-j2\pi vy/N}$，所以

$$F(u,v)=\sum_{x=0}^{M-1} e^{-j2\pi ux/M}\underbrace{\sum_{y=0}^{N-1} f(x,y)\,e^{-j2\pi vy/N}}_{F(x,v)}$$

其中 $F(x,v)$ 是第 x 列的一維 DFT。先對每一列做一維 DFT，再對結果的每一行做一維 DFT。只要有一維的 FFT 函式，就能組出二維的。

用正向轉換計算反 DFT

把反轉換公式取共軛，再乘上 MN：

$$MN\,f^{*}(x,y)=\sum_{u=0}^{M-1}\sum_{v=0}^{N-1} F^{*}(u,v)\,e^{-j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)}$$

右邊正是 F^{*} 的正向 DFT。所以：把 F 取共軛、做正向轉換、除以 MN，再取一次共軛。對實數影像而言，最後那次共軛可以省略。

```
import numpy as np
F = np.fft.fft2(np.random.default_rng(1).random((8, 8)))
f_back = np.conj(np.fft.fft2(np.conj(F))) / F.size
print(np.allclose(f_back, np.fft.ifft2(F)))   # True
```

快速傅立葉轉換

照定義計算一維 DFT，大約需要 M^2 次複數乘法與加法：M 個輸出，每個都是 M 項的和。Cooley 與 Tukey 的 **FFT** [4] 在 M 為 2 的冪次時，把成本降到大約 $M\log_2 M$。核心想法是**分而治之**：把總和拆成偶數索引與奇數索引兩半，

$$F(u)=F_{\text{even}}(u)+W_M^{u}\,F_{\text{odd}}(u),\qquad F(u+M/2)=F_{\text{even}}(u)-W_M^{u}\,F_{\text{odd}}(u)$$

其中 $W_M = e^{-j2\pi/M}$，F_{even}、F_{odd} 是長度 $M/2$ 的 DFT。每個半長度的 DFT 再繼續拆，直到長度為 1。總共有 $\log_2 M$ 層，每層的成本約為 M 次運算。

兩者的成本比 $M^2/(M\log_2 M) = M/\log_2 M$ 成長得很快。$M = 1024$ 時約為 100；對 1024×1024 的影像，直接做二維轉換（成本 $(MN)^2$）與用 FFT（成本 $MN\log_2 MN$）相比，比值約為 50,000。在筆電上就能明顯看出差異：

```
import time
x = np.random.default_rng(2).standard_normal(4096)
t0 = time.perf_counter(); dft(x); t1 = time.perf_counter(); np.fft.fft(x); t2 =
 ↪  time.perf_counter()
print(f"direct {t1 - t0:.3f}s  fft {t2 - t1:.5f}s")   # e.g. ~0.7 s vs ~0.2 ms
```

現代函式庫遠遠超越教科書上的基數 2（radix-2）演算法。Frigo 與 Johnson 介紹的 FFTW [6] 能處理任意長度，在執行時實際量測哪個演算法在這台機器上最快再做選擇，並自動產生小型的「codelet」核心程式。實務上，質因數都很小（2、3、5、7）的長度很快，大質數則較慢。`scipy.fft.next_fast_len` 會傳回一個合適的填補大小。

現代觀點

傅立葉轉換比本系列中任何其他工具都古老，卻老得特別好。深度學習並沒有取代它；相反地，傅立葉的想法以三種角色回歸：作為計算卷積的方法、作為建構具有全域視野之網路層的方法，以及作為理解網路學到什麼、在哪裡失敗的透鏡。

值得一讀的綜述與回顧

- **Unser，〈Sampling — 50 years after Shannon〉[3]**。回顧取樣理論從 Shannon 的帶限模型，一路發展到現代的樣條與平移不變空間（shift-invariant space）表述。重點：理想的 sinc 內插只是數學上的理想化；實際系統使用緊湊的核心搭配相應的前置濾波器，而其近似誤差可以在傅立葉域中精確分析。這是從本章的取樣定理通往實際重取樣程式碼的最佳橋樑。
- **Heideman、Johnson 與 Burrus，〈Gauss and the history of the fast Fourier transform〉[5]**。一篇歷史回顧，把類 FFT 演算法追溯到高斯約 1805 年未發表的手稿，以及 1965 年前的多次重新發現。重點：這個演算法的想法很簡單，被發現過很多次；1965 年真正改變的是數位電腦讓它變得重要。
- **Xu、Zhang 與 Luo，〈Overview frequency principle/spectral bias in deep learning〉[13]**。一篇關於**頻率原則**（frequency principle）的綜述：以梯度下降訓練的神經網路，傾向先擬合目標的低頻成分、之後才擬合高頻成分。文中整理了這個現象背後的理論與實驗，以及它在演算法設計上的應用。重點：神經網路在訓練過程中有點像一個慢慢打開的低通濾波器。
- **Kovachki 等人，〈Neural Operator: Learning Maps Between Function Spaces〉[15]**。一篇完整而深入的神經算子（neural operator）專論。神經算子學習的是函數之間的映射，而不是固定大小陣列之間的映射，傅立葉神經算子是其四種主要參數化方式之一。重點：用頻率響應來參數化一層網路，可以讓它在很大程度上不受網格解析度影響，這正是本章「濾波 = 在頻域中相乘」想法的直系後代。

傅立葉轉換作為快速卷積引擎

卷積定理保證大型卷積可以變成便宜的乘法。Mathieu、Henaff 與 LeCun [9] 把這一點用在卷積神經網路上：他們在 GPU 上把卷積算成 FFT 之間的逐點乘積，並重複使用每個轉換過的特徵圖，回報了大幅的訓練加速。如今多數 CNN 使用 3×3 的小卷積核，此時直接做空間卷積通常較快，所以 FFT 卷積主要用於大型卷積核。前面「空間濾波器與頻域濾波器的對應」一節的推論在這裡完全適用。

具有全域感受野的傅立葉層

頻域中的逐點乘積會影響每一個像素，所以單一個傅立葉層就擁有整張影像的感受野（receptive field）。好幾種架構利用了這一點。**Fast Fourier Convolution** [10] 加入一個全域分支，直接在影像層級的頻譜上運算，把全域資訊混入每一層。**Global Filter Networks** [11] 把視覺 Transformer 的自注意力換成正是本章的步驟：二維 FFT、與一個可學習的全域濾波器逐元素相乘、再做反 FFT，成本對 token 數量為對數線性。**Fourier Neural Operator** [14] 在截斷後的一組低頻上使用可學習的濾波器來解偏微分方程；由於權重存在於頻率空間中，訓練好的模型可以在比訓練時更細的網格上使用。這些名副其實就是可學習的頻域濾波器。

另一條相關的路線直接在轉換係數上運作。Xu 等人 [16] 把常見有損影像編碼已經算好的區塊離散餘弦轉換（DCT）係數直接餵給網路，並學習要保留哪些頻率通道。他們發現許多高頻通道可以丟掉而幾乎不損失準確率，這與頻率原則一致。

頻譜偏差

Rahaman 等人 [12] 證明深度 ReLU 網路偏好學習低頻函數：高頻成分學得較晚，也較不穩健。搭配 [13] 的綜述，可以得到一個實用的心智模型：網路很擅長目標函數的「貝斯」，但要把「鈸」學好，就需要更多訓練、更多資料或特殊的輸入編碼。這也解釋了為什麼以座標為輸入的網路，除非先把輸入座標映射成正弦形式的傅立葉特徵（Fourier features），否則常常難以表現細緻的紋理 [20]。

混疊在深度網路中捲土重來

步幅卷積與池化都是降取樣，而沒有低通前置濾波的降取樣就會混疊，跟圖 4.1 一模一樣。Zhang [17] 指出，標準 CNN 對一個像素的平移出乎意料地敏感，原因正在於此；而在每次降取樣前插入一個小模糊（也就是教科書裡的抗混疊濾波器），可以改善平移不變性，在他的實驗中也提升了影像分類準確率。Karras 等人 [18] 追查到一個現象：生成影像的細節似乎黏在像素座標上，而不是隨著畫面中的表面移動，其根源是生成器內部的混疊。他們的無混疊（alias-free）設計把特徵圖當作連續訊號的取樣，並在每個非線性運算與重取樣步驟周圍套用精心設計的低通濾波器。

頻率假影就像指紋

生成器需要上取樣，而沒有適當內插濾波器的上取樣，會在頻譜中留下週期性的花紋，很像圖 4.8 中的亮點。Frank 等人 [19] 發現，GAN 生成的影像在頻域中有強烈且一致的假影，並追溯到上取樣操作，據此建立了簡單又準確的偵測器。更廣泛的啟示是：觀察頻譜這個本章最古老的診斷工具，至今仍是看出生成模型哪裡出錯的最快方法之一。

沒有改變的部分

對於已經很清楚的問題，深度學習並沒有取代傳統的頻域濾波。移除已知的週期性干擾、縮放前的抗混疊、快速的大卷積核模糊，以及同態照明校正，至今仍然用本章這些明確的濾波器來做，因為它們精確、快速、而且不需要訓練資料。改變的是，這些同樣的想法如今被內建到網路架構中，也被用來解釋網路的行為。

重點整理

- 傅立葉轉換把訊號改寫成波的總和。低頻承載平滑結構；高頻承載邊緣、紋理與雜訊。
- 取樣會把頻譜週期性地複製。取樣頻率要超過最高頻率的兩倍（奈奎斯特取樣率），或先模糊，否則會出現無法復原的混疊與摩爾紋。
- DFT 把影像當成週期性的，因此得到循環卷積；要填補到至少 $A + C - 1$（通常是 $2M \times 2N$）以避免環繞誤差。
- 平移只改變相位；旋轉使頻譜旋轉；實數影像的頻譜共軛對稱；而大部分可辨識的結構存在相位中。
- 濾波就是「轉換、乘上實數對稱的 H、反轉換、裁切」。陡峭截止會振鈴；高斯不會；巴特沃斯介於兩者之間。
- 高通、拉普拉斯、高頻強調與同態濾波器用來銳化或平衡照明；陷波濾波器能精準移除週期性干擾。
- FFT 把每次一維轉換的成本從 $O(M^2)$ 降到 $O(M \log M)$，可分離性則讓二維轉換由一維轉換組成。
- 在深度學習中，傅立葉的想法支撐了快速卷積、全域濾波層與神經算子，也解釋了頻譜偏差、CNN 中的混疊與 GAN 的假影。

練習

1. 一個一維訊號含有每公尺 3、7、12 個循環的成分。你每 0.1 公尺取樣一次。哪些成分能正確保留？每個混疊成分會出現在什麼頻率？

   提示

   取樣率是每公尺 10 個樣本，所以奈奎斯特上限是每公尺 5 個循環。3 個循環的成分沒問題。其他成分要減去 10 的倍數，摺回 $[-5, 5]$：7 會變成 $7 - 10 = -3$，也就是每公尺 3 個循環；12 會變成 $12 - 10 = 2$ 個循環。這時 7 個循環的成分和真正 3 個循環的成分已無法區分。

2. 證明：若 $f(x,y)$ 是實數且對稱，即 $f(x,y) = f(-x,-y)$（索引對 M、N 取餘數），則它的 DFT 是實數。用 NumPy 建一個對稱陣列加以驗證。

提示

對任何實數 f，共軛對稱給出 $F(-u,-v) = F^*(u,v)$。在定義中代入 $x \to -x$、$y \to -y$，由 f 的對稱性得到 $F(-u,-v) = F(u,v)$。兩者合起來 $F = F^*$，所以 F 是實數。驗證時，對隨機陣列 `a` 建立 `g = a + np.roll(a[::-1, ::-1], 1, axis=(0, 1))`，確認 `np.abs(np.fft.fft2(g).imag).max()` 趨近於零。

3. 使用 `filter_freq` 函式，以 $D_0 = 30$、階數 1、2、5、20 的巴特沃斯低通濾波器處理攝影師影像。從第幾階開始能明顯看到振鈴？畫出每個濾波器空間卷積核的中央列，並與你看到的現象對照。

提示

把中心化的濾波器做反轉換，再移回中央，就得到卷積核。當卷積核出現明顯的負旁瓣時就會振鈴。1 階完全沒有，2 階只有很小的旁瓣，到了 5 至 20 階，旁瓣已接近理想濾波器。

4. 你要用 FFT 以 51×51 的空間卷積核濾波一張 300×400 的影像。避免環繞誤差的最小填補大小是多少？你會選這個大小，還是稍大一點的？為什麼？

提示

最小值是 $(300 + 51 - 1) \times (400 + 51 - 1) = 350 \times 450$。兩者都可以，但只含小質因數的長度在 FFT 函式庫中較快。$350 = 2 \cdot 5^2 \cdot 7$、$450 = 2 \cdot 3^2 \cdot 5^2$ 本身就已符合，所以是好選擇；`scipy.fft.next_fast_len` 可以確認這一點。

5. 拍攝（或合成）一張含有兩種不同週期干擾的影像。設計一個只移除其中一種的陷波拒斥濾波器，再用對應的陷波通過濾波器顯示被你移除的花紋。陷波的半徑該怎麼選？

提示

在對數頻譜中找出亮點，忽略 DC 峰值與座標軸上的線條，並記得每個亮點都有鏡像雙胞胎。半徑剛好能蓋住亮點擴散範圍時效果最好：太小會留下殘餘條紋，太大會移除那些頻率附近的影像內容。陷波通過的輸出應該是乾淨的條紋，看不到任何場景。

6. 運用本章的觀念解釋：為什麼在每個步幅為 2 的層之前插入模糊，可以讓 CNN 在輸入影像平移一個像素時輸出更穩定？

提示

步幅 2 的降取樣讓取樣率減半，高於新奈奎斯特上限的頻率會以混疊的形式摺回來。平移一個像素會改變這些成分的相位，經過混疊後就改變了低頻輸出，於是後面的層看到不同的訊號。先用低通濾波器移除這些頻率，平移對輸出的影響就小得多。參見 [17]。

參考文獻

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 4. publisher page
2. C. E. Shannon, "Communication in the presence of noise," *Proceedings of the IRE*, vol. 37, no. 1, pp. 10–21, 1949. doi
3. M. Unser, "Sampling—50 years after Shannon," *Proceedings of the IEEE*, vol. 88, no. 4, pp. 569–587, 2000. doi
4. J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Mathematics of Computation*, vol. 19, no. 90, 1965. doi
5. M. T. Heideman, D. H. Johnson and C. S. Burrus, "Gauss and the history of the fast Fourier transform," *IEEE ASSP Magazine*, vol. 1, no. 4, pp. 14–21, 1984. doi
6. M. Frigo and S. G. Johnson, "The design and implementation of FFTW3," *Proceedings of the IEEE*, vol. 93, no. 2, 2005. doi
7. A. V. Oppenheim and J. S. Lim, "The importance of phase in signals," *Proceedings of the IEEE*, vol. 69, no. 5, 1981. doi
8. A. V. Oppenheim, R. W. Schafer and T. G. Stockham, "Nonlinear filtering of multiplied and convolved signals," *Proceedings of the IEEE*, vol. 56, no. 8, pp. 1264–1291, 1968. doi
9. M. Mathieu, M. Henaff and Y. LeCun, "Fast training of convolutional networks through FFTs," arXiv:1312.5851, 2013. arXiv
10. L. Chi, B. Jiang and Y. Mu, "Fast Fourier convolution," *Advances in Neural Information Processing Systems* (*NeurIPS*), vol. 33, 2020. proceedings
11. Y. Rao, W. Zhao, Z. Zhu, J. Lu and J. Zhou, "Global filter networks for image classification," *NeurIPS*, 2021. arXiv
12. N. Rahaman, A. Baratin, D. Arpit, F. Draxler, M. Lin, F. A. Hamprecht, Y. Bengio and A. Courville, "On the spectral bias of neural networks," *ICML*, 2019. arXiv
13. Z.-Q. J. Xu, Y. Zhang and T. Luo, "Overview frequency principle/spectral bias in deep learning," arXiv:2201.07395, 2022. arXiv
14. Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart and A. Anandkumar, "Fourier neural operator for parametric partial differential equations," arXiv:2010.08895, 2020. arXiv
15. N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart and A. Anandkumar, "Neural operator: Learning maps between function spaces," *Journal of Machine Learning Research*, vol. 24, 2023. arXiv
16. K. Xu, M. Qin, F. Sun, Y. Wang, Y.-K. Chen and F. Ren, "Learning in the frequency domain," *CVPR*, 2020. arXiv
17. R. Zhang, "Making convolutional networks shift-invariant again," *ICML*, 2019. arXiv
18. T. Karras, M. Aittala, S. Laine, E. Härkönen, J. Hellsten, J. Lehtinen and T. Aila, "Alias-free generative adversarial networks," arXiv:2106.12423, 2021. arXiv
19. J. Frank, T. Eisenhofer, L. Schönherr, A. Fischer, D. Kolossa and T. Holz, "Leveraging frequency analysis for deep fake image recognition," *ICML*, 2020. arXiv
20. M. Tancik, P. P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. T. Barron and R. Ng, "Fourier features let networks learn high frequency functions in low dimensional domains," arXiv:2006.10739, 2020. arXiv

IP 05・影像復原與重建

為模糊與雜訊建立退化模型，再用次序統計、陷波、逆濾波、Wiener 與受限最小平方濾波器把傷害還原，並從投影重建 CT 斷層影像。

網站版本: https://blog.shyandram.dev/zh-tw/learn/image-processing/dip-05-restoration-reconstruction/

你將學到

- 如何把影像受到的傷害寫成模型 $g = h \star f + \eta$，以及這個模型為什麼把「復原」和「增強」區分開來
- 常見的雜訊模型、如何從直方圖辨認它們，以及如何從平坦區塊量測雜訊參數
- 不同雜訊該配哪一種空間濾波器：平均濾波器、次序統計濾波器與自適應濾波器
- 如何在頻域用陷波濾波器（notch filter）去除週期性干擾
- 如何估計模糊並用逆濾波、Wiener 濾波、受限最小平方與幾何平均濾波器還原，以及為什麼直接相除會失敗
- CT 掃描儀如何把一組一維投影變回二維斷層：Radon 轉換、傅立葉切片定理與濾波反投影

先看全貌

第 3、4 章的目標是讓影像「看起來」更好。本章問的是更嚴格的問題：影像在受損之前原本長什麼樣子？要回答它，必須先有一個描述傷害的模型。有了模型，還原就變成一個定義清楚（至少是被充分理解）的數學問題。本章依照 Gonzalez 與 Woods 教科書的編排 [1]。

> **用最簡單的話說**
>
> 想像有人用濕手指抹過你的畫，又在上面撒了一些沙。如果只是「讓它變漂亮」，你可能會整張重新塗色。但如果你知道手指怎麼移動、沙長什麼樣，就能小心地把沙撥掉，再把被抹開的線條推回原位。復原就是這種細心的做法：先查清楚發生了什麼，再精準地把那件事倒轉回來。

只要真實場景才是重點，就需要復原：天文、顯微鏡、鑑識照片、醫學影像，以及每一支手機的相機處理流程。本章最後的「從投影重建影像」是同一個想法推到極致：CT 掃描儀根本看不到斷層本身，只看得到它從許多角度投下的「影子」，我們必須從這些影子重建出斷層。

影像退化／復原過程的模型

白話說：相機看到的是場景稍微模糊後的版本，上面再疊一層隨機雜訊。復原就是設法把場景找回來。

我們把退化寫成

$$g(x,y) = h(x,y) \star f(x,y) + \eta(x,y),$$

其中 $f(x,y)$ 是理想（未知）的影像，$h(x,y)$ 是**退化函數**，也稱為點擴散函數（point spread function, PSF），$\star$ 代表二維卷積，$\eta(x,y)$ 是加成性雜訊，$g(x,y)$ 是我們實際觀測到的影像。依第 4 章的卷積定理，同一個模型在頻域是

$$G(u,v) = H(u,v)\,F(u,v) + N(u,v),$$

其中大寫字母代表對應小寫函數的二維離散傅立葉轉換（DFT），(u,v) 是頻率座標。$H(u,v)$ 稱為**光學轉移函數**（optical transfer function, OTF）。

復原的目標是求出一個估計 $\hat{f}(x,y)$，讓它盡可能接近 $f(x,y)$。我們對 h 和 η 知道得越多，就能越接近。

復原與增強的差別

增強（第 3、4 章）大多是主觀的：調整對比或銳化，直到觀看者覺得好看為止。復原則是客觀的：它使用退化模型，並以結果和真實影像有多接近來評判，通常用可量測的準則，例如均方誤差。有些工具在兩章都會出現（例如中值濾波器），但使用的理由不同。在復原裡，我們挑某個濾波器，是因為雜訊模型告訴我們它是對的選擇。

本章先處理 $h = \delta$（沒有模糊、只有雜訊）的情況，再加入模糊。

雜訊模型

白話說：雜訊是疊在畫面上的隨機「顆粒」。不同的感測器與情境會產生不同的顆粒，每一種在直方圖上都有自己的指紋。

雜訊的空間與頻率特性

描述雜訊要問兩件事。空間上：某個像素的雜訊和鄰居的雜訊有沒有關聯？和影像本身有沒有關聯？除非特別說明，本章假設雜訊與位置無關，也與影像不相關。頻率上：雜訊的功率如何分布在 (u,v) 上？傅立葉頻譜大致為常數的雜訊稱為**白雜訊**（white noise），名稱來自包含所有可見光頻率的白光。週期性雜訊正好相反：它的功率集中在少數幾個孤立的頻率。

在這些假設下，雜訊完全由其數值的機率密度函數（probability density function, PDF）$p(z)$ 描述，z 代表強度值。

常見的 PDF

高斯（Gaussian）。最常用的模型，因為數學上方便，也很符合電子感測器的雜訊：

$$p(z) = \frac{1}{\sqrt{2\pi}\,\sigma} e^{-(z-\bar{z})^2/2\sigma^2},$$

其中 $\bar{z}$ 為平均值，σ 為標準差（σ^2 為變異數）。約 68% 的值落在 $\bar{z} \pm \sigma$ 之內，約 95% 落在 $\bar{z} \pm 2\sigma$ 之內。

Rayleigh。單邊、偏斜的密度，用來描述距離成像（range imaging）中的雜訊：

$$p(z) = \begin{cases} \dfrac{2}{b}(z-a)\, e^{-(z-a)^2/b}, & z \geq a \\ 0, & z < a \end{cases} \qquad \bar{z} = a + \sqrt{\pi b/4}, \quad \sigma^2 = \frac{b(4-\pi)}{4},$$

其中 a 決定起點位置，$b > 0$ 控制分散程度。

Erlang（gamma）。同樣偏斜，但尾巴更長：

$$p(z) = \begin{cases} \dfrac{a^b z^{b-1}}{(b-1)!}\, e^{-az}, & z \geq 0 \\ 0, & z < 0 \end{cases} \qquad \bar{z} = \frac{b}{a}, \quad \sigma^2 = \frac{b}{a^2},$$

其中 $a > 0$ 是速率，b 是正整數（形狀參數）。允許 b 為非整數，就得到一般的 gamma 密度。

指數（exponential）。即 $b = 1$ 的 Erlang 密度：

$$p(z) = a\, e^{-az} \ (z \geq 0), \qquad \bar{z} = \frac{1}{a}, \quad \sigma^2 = \frac{1}{a^2}.$$

它出現在雷射成像（斑點強度）中。

均勻（uniform）。區間 $[a, b]$ 內每個值出現的機率相同：

$$p(z) = \frac{1}{b-a} \ (a \leq z \leq b), \qquad \bar{z} = \frac{a+b}{2}, \quad \sigma^2 = \frac{(b-a)^2}{12}.$$

它主要用於模擬（量化誤差近似均勻分布）。

椒鹽（脈衝）雜訊（salt-and-pepper / impulse noise）。每個像素要嘛不變，要嘛以機率 P_s 被換成很亮的值（鹽），或以機率 P_p 被換成很暗的值（胡椒）。被破壞的像素比例 $P = P_s + P_p$ 稱為**雜訊密度**。故障的感測元件與傳輸位元錯誤會產生這種雜訊。它和其他雜訊不同，不是加上去的，而是取代原本的像素值。

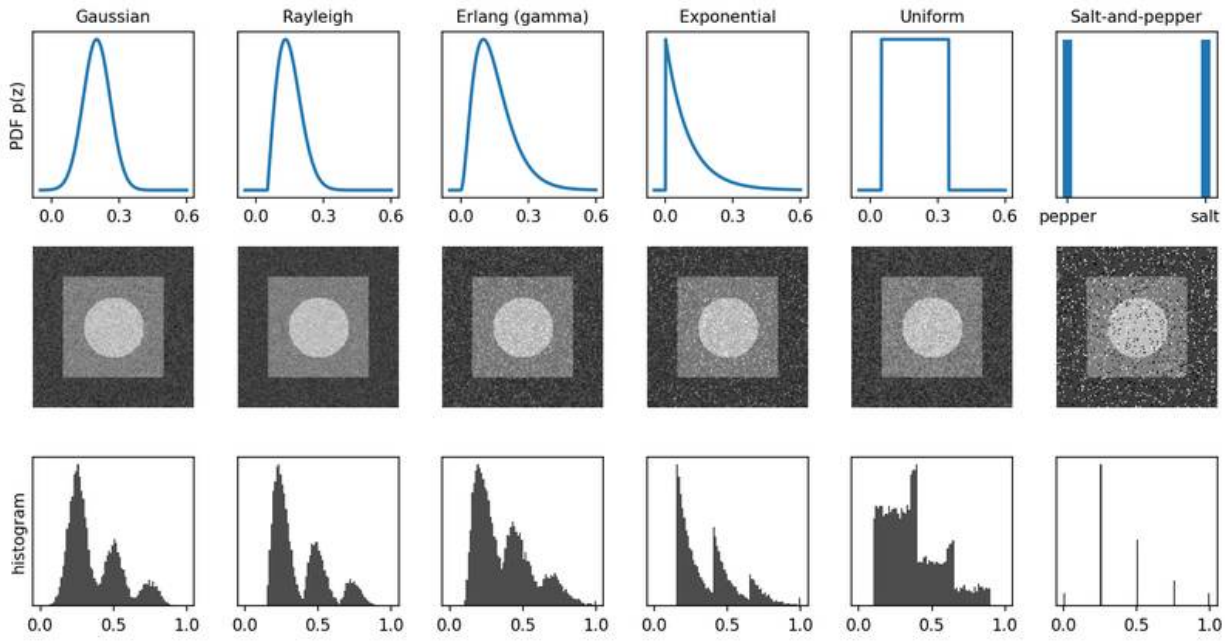


圖 *5.1* — 上：六種雜訊 *PDF*。中：一張具有三個平坦灰階的測試圖，分別被各種雜訊污染。下：直方圖。雜訊影像看起來差不多，但每個直方圖都包含三份雜訊 *PDF*（每個灰階一份），因此直方圖能辨認雜訊類型。脈衝雜訊是例外：它在 *0* 與 *1* 加上尖峰。

週期性雜訊

擷取影像時的電氣或機電干擾，可能在影像上疊加一個像 $A\sin(2\pi(u_0x/M + v_0y/N))$ 的正弦波，其中 A 為振幅，(u_0, v_0) 為頻率，$M \times N$ 為影像大小。它與位置相關，所以無法用 PDF 描述。不過在傅立葉頻譜中，每個正弦波都是位於 $\pm(u_0, v_0)$ 的一對亮點，因此在頻域很容易去除（見下方的週期性雜訊消除）。

估計雜訊參數

如果你能控制成像系統，就拍一張均勻照明的平坦目標：看到的一切都是雜訊。如果手上只有影像，就找一個本來應該是常數的小區域 S（天空、空白牆面、背景），計算它的直方圖 $p_S(z_i)$，也就是 S 中強度為 z_i 的像素比例。接著

$$\bar{z} = \sum_i z_i\, p_S(z_i), \qquad \sigma^2 = \sum_i (z_i - \bar{z})^2\, p_S(z_i).$$

區塊直方圖的形狀告訴你該用哪一種 PDF，平均值與變異數再決定參數（Rayleigh、Erlang 的 a 和 b 等）。對脈衝雜訊，則計算落在極端值的像素數量來估計 P_s 與 P_p。

```
import numpy as np
from skimage import data, img_as_float
```

```
rng = np.random.default_rng(0)
f = img_as_float(data.camera())                  # values in [0, 1]

g_gauss = f + rng.normal(0.0, 0.05, f.shape)     # additive Gaussian, sigma = 0.05
g_unif = f + rng.uniform(-0.1, 0.1, f.shape)     # additive uniform on [-0.1, 0.1]
g_sp = f.copy()
r = rng.random(f.shape)
g_sp[r < 0.05] = 0.0                             # 5 % pepper
g_sp[r > 0.95] = 1.0                             # 5 % salt

# Estimate the noise from a patch that should be flat (here: sky, top-left)
patch = g_gauss[10:60, 10:60]
print(f"mean = {patch.mean():.3f}, std = {patch.std():.3f}")   # std close to 0.05
```

只有雜訊時的復原：空間濾波

白話說：如果唯一的問題是雜訊，就把每個像素換成它鄰域的一個「聰明摘要」。最好的摘要方式取決於雜訊種類。

沒有模糊時，模型簡化為 $g(x,y) = f(x,y) + \eta(x,y)$（脈衝雜訊則是取代）。以下 S_{xy} 是以 (x,y) 為中心的 $m \times n$ 視窗，(r,c) 走遍視窗內的像素。

平均濾波器

算術平均（arithmetic mean）：一般的平均：

$$\hat{f}(x,y) = \frac{1}{mn} \sum_{(r,c)\in S_{xy}} g(r,c).$$

它能降低高斯或均勻雜訊，但會模糊邊緣。

幾何平均（geometric mean）：乘積開 mn 次方根：

$$\hat{f}(x,y) = \Big[\prod_{(r,c)\in S_{xy}} g(r,c) \Big]^{1/mn}.$$

平滑效果與算術平均相近，但細節損失較少。

調和平均（harmonic mean）：

$$\hat{f}(x,y) = \frac{mn}{\sum_{(r,c)\in S_{xy}} 1/g(r,c)}.$$

它能處理鹽雜訊與類高斯雜訊，但對胡椒雜訊完全失效，因為只要一個接近 0 的像素，就會主宰倒數和。

Q 階反調和平均（contraharmonic mean）：

$$\hat{f}(x,y) = \frac{\sum_{(r,c)\in S_{xy}} g(r,c)^{Q+1}}{\sum_{(r,c)\in S_{xy}} g(r,c)^{Q}}.$$

Q 稱為階數。Q 為正時去除胡椒雜訊，為負時去除鹽雜訊，無法同時去除兩者。$Q = 0$ 即算術平均，$Q = -1$ 即調和平均。正負號選錯會造成災難，如圖 5.2 所示。

次序統計濾波器

這類濾波器把 S_{xy} 內的像素排序，再從排序結果中挑值。

- **中值（median）**：$\hat{f}(x,y) = \text{median}_{(r,c)\in S_{xy}} g(r,c)$。處理脈衝雜訊最好用的通用工具。它能去除孤立的離群值，模糊程度遠小於同尺寸的平均濾波器。
- **最大值（max）**與**最小值（min）**：取最大與最小值。最大值濾波器去除胡椒（暗）雜訊，最小值濾波器去除鹽（亮）雜訊。
- **中點（midpoint）**：$\frac{1}{2}(\max + \min)$。結合排序與平均，適合高斯或均勻雜訊。
- **alpha 修剪平均（alpha-trimmed mean）**：去掉最低與最高各 $d/2$ 個值，再對剩下的 $mn - d$ 個值 $g_R(r,c)$ 取平均：

$$\hat{f}(x,y) = \frac{1}{mn - d} \sum_{(r,c)\in S_{xy}} g_R(r,c).$$

$d = 0$ 時是算術平均，$d = mn - 1$ 時是中值。介於兩者之間時，它能處理混合雜訊，例如高斯雜訊加上少量脈衝。

```
import numpy as np
from scipy import ndimage as ndi

def arithmetic_mean(g, m=3):
    return ndi.uniform_filter(g, m)

def geometric_mean(g, m=3):
    return np.exp(ndi.uniform_filter(np.log(g + 1e-6), m))

def harmonic_mean(g, m=3):
    return 1.0 / ndi.uniform_filter(1.0 / (g + 1e-6), m)

def contraharmonic_mean(g, m=3, Q=1.5):
    g = g + 1e-6                                  # avoid 0 ** negative
    return ndi.uniform_filter(g ** (Q + 1), m) / ndi.uniform_filter(g ** Q, m)

def alpha_trimmed_mean(g, m=5, d=6):
    def trim(v):
        v = np.sort(v)
        return v[d // 2: v.size - d // 2].mean()
    return ndi.generic_filter(g, trim, size=m)

median = lambda g, m=3: ndi.median_filter(g, m)
maxf = lambda g, m=3: ndi.maximum_filter(g, m)    # removes pepper
```

```
minf = lambda g, m=3: ndi.minimum_filter(g, m)    # removes salt
midpoint = lambda g, m=3: 0.5 * (maxf(g, m) + minf(g, m))
```

自適應濾波器

上面的濾波器對每個像素一視同仁。自適應濾波器則依 S_{xy} 內的局部統計量改變行為，通常只需多花一點計算就能表現更好。

自適應局部雜訊消除濾波器（adaptive, local noise reduction filter）。令 σ_η^2 為雜訊變異數（整張影像估計一次），$\bar{z}_S$ 與 σ_S^2 為 S_{xy} 內的局部平均與局部變異數。濾波器為

$$\hat{f}(x,y) = g(x,y) - \frac{\sigma_\eta^2}{\sigma_S^2}\left[g(x,y) - \bar{z}_S\right].$$

可以分三種情況理解。若 $\sigma_\eta^2 = 0$，表示沒有雜訊，濾波器原封不動傳回 g。若局部變異數遠大於雜訊變異數，視窗裡很可能有邊緣，濾波器就保持接近 g，保住邊緣。若兩個變異數相等，視窗是平坦區域，濾波器傳回局部平均。實務上會把比值上限截在 1，因為局部變異數的估計值可能碰巧比 σ_η^2 小。

自適應中值濾波器（adaptive median filter）。固定尺寸的中值濾波器在脈衝稀疏時效果很好；雜訊密度一高，一個視窗裡的脈衝可能比真實像素還多，而加大固定視窗又會把一切都模糊掉。自適應中值濾波器在需要時才放大視窗，而且不去動沒有被污染的像素。令 $z_{\min}$、$z_{\max}$、z_{med} 為 S_{xy} 內的最小、最大與中值，z_{xy} 為中心像素，$S_{\max}$ 為允許的最大視窗尺寸 [2]。

- **階段 A：**若 $z_{\min} < z_{\text{med}} < z_{\max}$，中值不是脈衝，進入階段 B。否則放大視窗；若尺寸超過 $S_{\max}$，輸出 z_{med}；否則重複階段 A。
- **階段 B：**若 $z_{\min} < z_{xy} < z_{\max}$，中心像素不是脈衝，原樣輸出 z_{xy}。否則輸出 z_{med}。

這樣同時達成三個目標：去除脈衝、平滑其他雜訊，並減少一般中值濾波造成的邊緣變細與模糊。

```
import numpy as np
from scipy import ndimage as ndi

def adaptive_local(g, noise_var, m=7):
    """Adaptive local noise reduction: f = g - (s_n^2 / s_L^2)(g - m_L)."""
    mean = ndi.uniform_filter(g, m)
    var = ndi.uniform_filter(g * g, m) - mean ** 2
    ratio = np.minimum(noise_var / np.maximum(var, 1e-12), 1.0)  # never above 1
    return g - ratio * (g - mean)

def adaptive_median(g, s_max=7):
    out, done = g.copy(), np.zeros(g.shape, bool)
    for s in range(3, s_max + 1, 2):
        zmin, zmax = ndi.minimum_filter(g, s), ndi.maximum_filter(g, s)
        zmed = ndi.median_filter(g, s)
        stage_a = (zmin < zmed) & (zmed < zmax) & ~done          # median is not an impulse
```

```
        keep = (zmin < g) & (g < zmax)                          # centre pixel is not an
         ↪  impulse
        out[stage_a] = np.where(keep, g, zmed)[stage_a]
        done |= stage_a
    out[~done] = zmed[~done]                                    # window grew to s_max
    return out
```

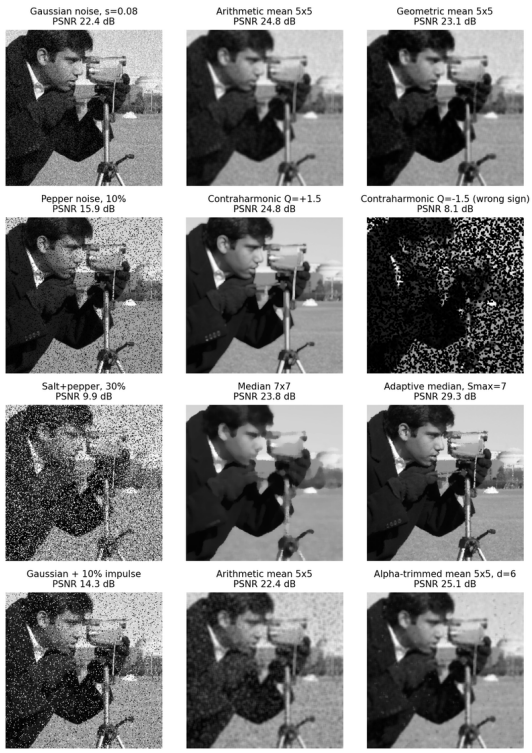


圖 *5.2* — 讓濾波器對上雜訊。第 *1* 列：高斯雜訊，算術與幾何平均。第 *2* 列：胡椒雜訊；$Q = +1.5$ 的反調和濾波器能去除它，$Q = -1.5$ 則讓情況更糟。第 *3* 列：*30%* 椒鹽雜訊；*7×7* 中值濾波器把細節抹平，自適應中值保留了多得多的細節。第 *4* 列：高斯加脈衝雜訊；*alpha* 修剪平均勝過一般平均。*PSNR* 為相對乾淨影像的峰值訊雜比（越高越好）。

以頻域濾波消除週期性雜訊

白話說：週期性雜訊是藏在影像裡的幾個純「音調」。它們在頻譜上是亮點，所以我們只挖掉那幾個點，其他一律保留。

陷波拒斥與陷波通過濾波器

陷波濾波器（notch filter）會拒斥（或通過）以指定中心為圓心的小鄰域內的頻率。由於實數影像的 DFT 具有共軛對稱性，陷波一定成對出現，相對頻譜中心位於 (u_k, v_k) 與 $(-u_k, -v_k)$。具有 Q 對陷波的陷波拒斥濾波器，是一組中心被移到陷波位置的高通濾波器的乘積：

$$H_{NR}(u,v) = \prod_{k=1}^{Q} H_k(u,v)\, H_{-k}(u,v),$$

其中 H_k 與 H_{-k} 是中心分別在 (u_k, v_k) 與 $(-u_k, -v_k)$ 的高通濾波器（理想、Butterworth 或高斯，同第 4 章）。例如高斯陷波使用 $H_k = 1 - e^{-D_k^2/2D_0^2}$，$D_k(u,v)$ 是 (u,v) 到 (u_k, v_k) 的距離，D_0 是陷波半徑。對應的陷波通過濾波器為

$$H_{NP}(u,v) = 1 - H_{NR}(u,v).$$

陷波通過濾波器本身也很有用：把它乘上 G 再轉回空間域，就得到只有干擾圖樣的影像。其他形狀的陷波（例如沿座標軸的細線）可去除掃描線圖樣。

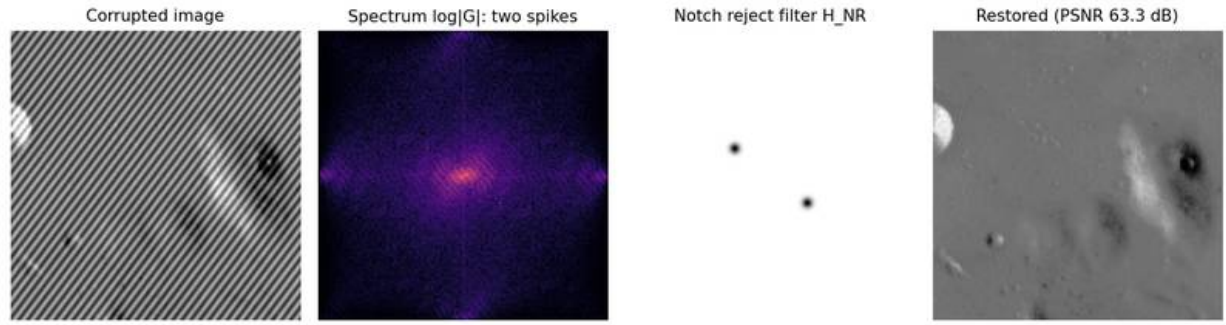


圖 *5.3* — 正弦干擾圖樣（左）在頻譜中是兩個孤立的亮點。用兩個高斯陷波去除它們，反 *DFT* 就還原出乾淨的影像。這裡幾乎完美還原，是因為干擾剛好是單一正弦波；真實的干擾通常分散在好幾個頻率上。

```
import numpy as np
from skimage import data, img_as_float

f = img_as_float(data.moon())
M, N = f.shape
y, x = np.mgrid[:M, :N]
g = f + 0.2 * np.sin(2 * np.pi * (40 * y / M + 25 * x / N))    # periodic interference

G = np.fft.fftshift(np.fft.fft2(g))
U, V = np.mgrid[-M // 2:M // 2, -N // 2:N // 2]
H = np.ones((M, N))
for u0, v0 in [(40, 25), (-40, -25)]:                          # a spike and its mirror
    D2 = (U - u0) ** 2 + (V - v0) ** 2
    H *= 1 - np.exp(-D2 / (2 * 3.0 ** 2))                      # Gaussian notch, radius ~3
f_hat = np.real(np.fft.ifft2(np.fft.ifftshift(H * G)))
```

最佳陷波濾波

當干擾成分很多，或者它們不是尖銳的亮點時，單純的陷波濾波器不是去得不夠，就是連真實影像內容一起切掉。**最佳陷波濾波**（optimum notch filtering）分兩步。第一步，用陷波通過濾波器分離出干擾的估計：

$$\hat{\eta}(x,y) = \mathfrak{F}^{-1}\{H_{NP}(u,v)\,G(u,v)\},$$

其中 $\mathfrak{F}^{-1}$ 是反 DFT。第二步，減去它的加權版本，而權重隨位置改變：

$$\hat{f}(x,y) = g(x,y) - w(x,y)\,\hat{\eta}(x,y).$$

權重 $w(x,y)$ 的選法是讓 $\hat{f}$ 盡可能平滑：讓每個像素周圍小視窗內 $\hat{f}$ 的變異數最小。把這個局部變異數對 w 微分並令其為零，得到

$$w(x,y) = \frac{\overline{g\hat{\eta}} - \bar{g}\,\bar{\hat{\eta}}}{\overline{\hat{\eta}^2} - \bar{\hat{\eta}}^2},$$

每個上橫線代表視窗內的局部平均。分子是 g 與 $\hat{\eta}$ 的局部共變異數，分母是 $\hat{\eta}$ 的局部變異數。所以在影像和估計出的圖樣一起變動的地方減得多，不同步的地方減得少。

線性、位置不變的退化

白話說：我們假設模糊是「公平」的。光線加倍，模糊後的光也加倍；而且左上角的模糊和右下角的一樣。

把（不含雜訊的）退化寫成運算子 $\mathcal{H}$，即 $g = \mathcal{H}[f] + \eta$。若對任意影像 f_1, f_2 與常數 a, b 都有

$$\mathcal{H}[af_1 + bf_2] = a\,\mathcal{H}[f_1] + b\,\mathcal{H}[f_2],$$

則 $\mathcal{H}$ 是**線性**的。若平移輸入只會平移輸出，

$$\mathcal{H}[f(x-\alpha, y-\beta)] = g(x-\alpha, y-\beta)$$

對任意平移量 (α,β) 都成立，則 $\mathcal{H}$ 是**位置不變**的。對這種運算子，把 f 寫成許多平移脈衝的和。線性讓我們可以分別對每個脈衝套用 $\mathcal{H}$，位置不變則保證每個脈衝都產生相同的響應 h，只是位置不同。把這些響應加總，正是卷積：

$$g(x,y) = \int_{-\infty}^{\infty}\!\int_{-\infty}^{\infty} f(\alpha,\beta)\,h(x-\alpha, y-\beta)\,d\alpha\,d\beta + \eta(x,y) = (h \star f)(x,y) + \eta(x,y).$$

這裡 $h(x,y) = \mathcal{H}[\delta(x,y)]$ 是**脈衝響應**，在光學中就是點擴散函數：單一光點所成的像。這就是為什麼還原模糊影像常被稱為**反卷積**（deconvolution），也是下面的濾波器都在頻域建構的原因。

許多真實退化只是近似線性位置不變（LPI）。鏡頭在影像角落可能更模糊，在靜止背景前移動的物體也和背景模糊得不一樣。空間變異的復原方法存在，但代價高得多；常見的變通是把影像切成大致符合 LPI 的小塊。

估計退化函數

白話說：要還原模糊之前，得先知道模糊是什麼。可以從照片裡找線索、自己測試相機，或從物理推導。

觀察法

在模糊影像中找一個結構簡單而明顯的小區域 $g_s(x,y)$，例如一條邊或一個小亮點，而且訊號遠強於雜訊。用手工（或銳化加上判斷）建立該區域應有樣貌的估計 $\hat{f}_s(x,y)$。則

$$H_s(u,v) = \frac{G_s(u,v)}{\hat{F}_s(u,v)}$$

就是從這個小區域估計出的 OTF。若模糊是位置不變的，H_s 告訴我們整張影像 H 的形狀，再放大到全尺寸即可。這很費工，主要用於只有影像可用的情況，例如歷史照片。

實驗法

如果手上有相同（或等效）的成像系統，就在相同設定下拍一個極小的亮點。夠小的亮點近似強度為 A 的脈衝，其傅立葉轉換是常數 A。觀測到的亮點影像直接給出 OTF：

$$H(u,v) = \frac{G(u,v)}{A}.$$

顯微鏡使用者正是用小於解析度的螢光微珠這樣做。

數學建模法

有時物理會直接給答案。本章會用到兩個經典模型。

大氣擾流（atmospheric turbulence）。透過擾動空氣長時間曝光的成像，常以下式建模：

$$H(u,v) = e^{-k\,(u^2+v^2)^{5/6}},$$

其中常數 k 隨擾流強度增加。除了 5/6 次方之外，它看起來像高斯低通濾波器，而且永遠不會剛好等於零。

均勻直線運動模糊（uniform linear motion blur）。假設影像 f 在長度為 T 的曝光期間移動，位移為 $x_0(t)$ 與 $y_0(t)$。感測器把每個位置都累加起來：

$$g(x,y) = \int_0^T f\big(x - x_0(t),\, y - y_0(t)\big)\, dt.$$

取傅立葉轉換並利用平移性質，得到

$$H(u,v) = \int_0^T e^{-j2\pi[u\,x_0(t) + v\,y_0(t)]}\, dt.$$

對等速運動，$x_0(t) = at/T$、$y_0(t) = bt/T$，a 與 b 是 x、y 方向的總位移。積分結果為

$$H(u,v) = \frac{T}{\pi(ua+vb)}\,\sin\!\big[\pi(ua+vb)\big]\, e^{-j\pi(ua+vb)}.$$

這是沿運動方向的 sinc 函數，再乘上線性相位項。關鍵是：只要 $ua + vb$ 是非零整數，它就剛好等於**零**，因此整條整條的頻率線被完全摧毀（圖 5.4）。

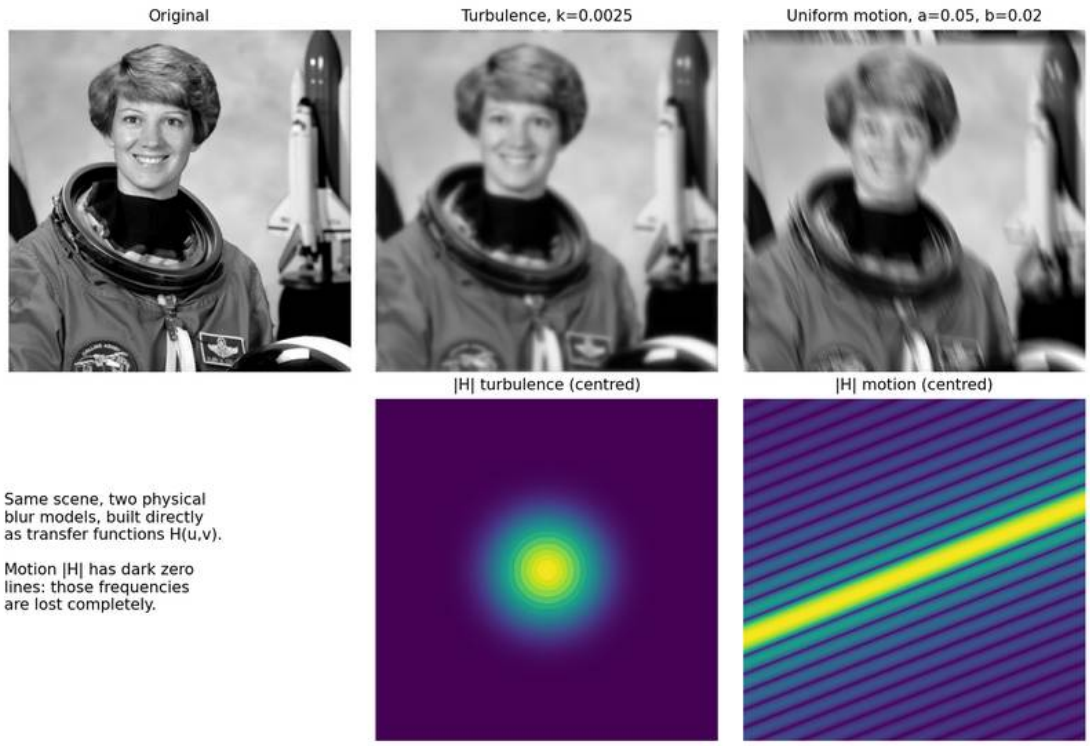


圖 *5.4* — 由物理推導出的兩種退化函數。擾流的 *OTF* 是一個平滑的隆起，會衰減但永遠不為零。運動的 *OTF* 是沿運動方向的 *sinc* 脊線，旁邊有一條條平行、剛好為零的暗線：在這些頻率上，模糊影像完全不含場景的資訊。

```
import numpy as np

def freq_grid(shape):
    u = np.fft.fftfreq(shape[0]) * shape[0]            # 0, 1, ..., -1 (un-centred)
    v = np.fft.fftfreq(shape[1]) * shape[1]
    return np.meshgrid(u, v, indexing="ij")

def motion_blur_otf(shape, a, b, T=1.0):
    U, V = freq_grid(shape)
    s = np.pi * (U * a + V * b)
    return T * np.sinc(s / np.pi) * np.exp(-1j * s)    # np.sinc(x) = sin(pi x) / (pi x)

def turbulence_otf(shape, k):
    U, V = freq_grid(shape)
    return np.exp(-k * (U ** 2 + V ** 2) ** (5 / 6))
```

實作細節

像上面這樣直接在頻域建構 H，得到的是循環卷積：模糊會從影像邊界繞回另一邊。做實驗時這很理想，因為模型和濾波器完全吻合。處理真實照片時，要先補邊或讓邊界漸弱再做反卷積，否則邊緣會出現強烈的振鈴（ringing）。

逆濾波

白話說：既然模糊把每個頻率乘上了 H，那就除回去。這是最直覺的想法，而只要有雜訊，它就會慘敗。

直接逆濾波器（direct inverse filter）的估計為

$$\hat{F}(u,v) = \frac{G(u,v)}{H(u,v)}.$$

代入 $G = HF + N$ 就看出問題：

$$\hat{F}(u,v) = F(u,v) + \frac{N(u,v)}{H(u,v)}.$$

即使完全知道 H，也無法精確還原 F，因為 N 是未知的。更糟的是，模糊讓 $|H|$ 在高頻非常小（運動模糊甚至剛好為零），而雜訊大致是白的，在高頻仍保有功率。於是 N/H 變得極大，淹沒整張影像。在圖 5.5 中，雖然雜訊標準差只有強度範圍的 0.3% 與 1%，完整的逆濾波仍把兩張影像都變成純雜訊。

一個局部的補救是只在頻譜原點附近（$|H|$ 夠大的地方）做逆濾波，例如把 $\hat{F}$ 乘上半徑 D_0 的陡峭 Butterworth 低通濾波器。對 H 平滑遞減的擾流模糊有效；對運動模糊則不太行，因為 H 的零線也穿過低頻。半徑還得手動調整。我們需要一個能自動決定每個頻率該信任多少的方法。

最小均方誤差（Wiener）濾波

白話說：在每個頻率問一句「這裡畫面比較多，還是雜訊比較多？」畫面佔優勢的地方就還原模糊；雜訊佔優勢的地方就收手。

Wiener 濾波器把影像與雜訊都視為隨機場，尋找使均方誤差

$$e^2 = E\{(f - \hat{f})^2\}$$

最小的估計，$E\{\cdot\}$ 代表期望值。假設雜訊與影像不相關、其中之一平均為零，而且 $\hat{f}$ 是 g 的線性函數。

推導概要。對平穩訊號，DFT 會讓各頻率彼此解耦，因此可以對每個頻率分別選一個增益 $W(u,v)$，令 $\hat{F} = WG$。某個頻率上的期望誤差為

$$E\{|F - WG|^2\} = S_f - WHS_f - W^*H^*S_f + |W|^2(|H|^2 S_f + S_\eta),$$

其中 $S_f(u,v) = E\{|F(u,v)|^2\}$ 是影像的**功率頻譜**，$S_\eta(u,v) = E\{|N(u,v)|^2\}$ 是雜訊的功率頻譜，* 表示共軛複數，而與 N 的交叉項因影像和雜訊不相關而消失。這是 W 的二次式；對 W^* 微分並令其為零，得到

$$W(u,v) = \frac{H^*(u,v)\,S_f(u,v)}{|H(u,v)|^2\,S_f(u,v) + S_\eta(u,v)}.$$

分子分母同除以 S_f，得到最容易解讀的形式：

$$\hat{F}(u,v) = \left[\frac{1}{H(u,v)} \frac{|H(u,v)|^2}{|H(u,v)|^2 + S_\eta(u,v)/S_f(u,v)} \right] G(u,v).$$

第一個因子就是逆濾波器。第二個因子介於 0 和 1 之間。訊號遠強於雜訊時，$S_\eta/S_f \approx 0$，這個因子接近 1，濾波器表現得像逆濾波器。雜訊佔優勢時，因子趨近 0，把那個頻率關掉。若某頻率 $H = 0$，括號在該處為 0（寫成 $H^*/(|H|^2 + S_\eta/S_f)$ 就不必除以零）。沒有雜訊時 $S_\eta = 0$，Wiener 濾波器就是逆濾波器。

常數 K 近似

我們很少知道 S_f，因為那正是我們要找的影像的功率頻譜。常見的捷徑是把比值 S_η/S_f 換成一個常數 K：

$$\hat{F}(u,v) = \frac{H^*(u,v)}{|H(u,v)|^2 + K} G(u,v).$$

K 可以目視調整，或選擇讓某個品質指標最佳的值。雜訊變異數與影像變異數的比值（也就是訊雜比的倒數）是不錯的起點。K 小，結果較銳利但雜訊多；K 大，結果較平滑。

```
import numpy as np
from skimage import data, img_as_float

rng = np.random.default_rng(0)
f = img_as_float(data.camera())[::2, ::2]                # 256 x 256
H = motion_blur_otf(f.shape, a=0.1, b=0.1)               # from the snippet above
g = np.real(np.fft.ifft2(H * np.fft.fft2(f))) + rng.normal(0, 0.01, f.shape)
G = np.fft.fft2(g)

def wiener(G, H, K):
    return np.real(np.fft.ifft2(np.conj(H) / (np.abs(H) ** 2 + K) * G))

f_hat = wiener(G, H, K=0.01)
```

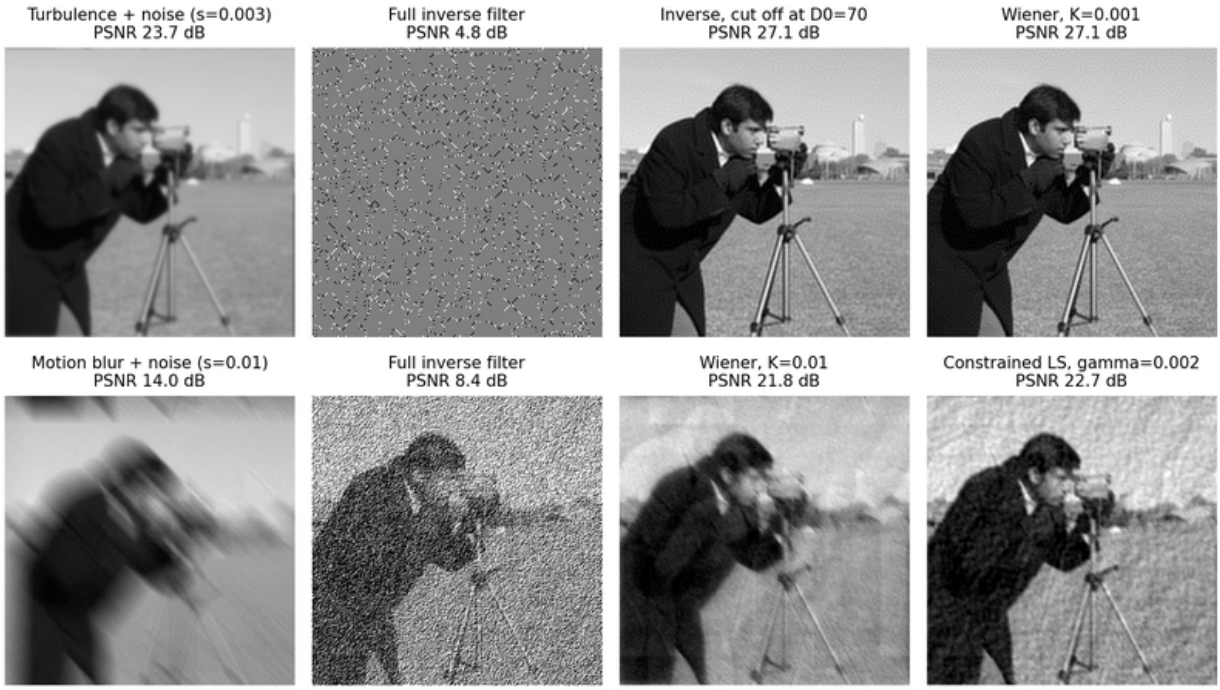


圖 *5.5* — 上列：擾流模糊（$k = 0.0025$）加微弱雜訊。完整逆濾波被放大的雜訊摧毀；限制在半徑 *70* 內的逆濾波可行，而 *Wiener* 濾波器不必挑半徑就達到相同品質。下列：斜向運動模糊加雜訊。逆濾波失敗；*Wiener* 與受限最小平方濾波器找回了場景，但留下一些振鈴與顆粒。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

拖動滑桿，比較運動模糊加雜訊的影像與其 Wiener 復原結果（512 × 512 完整版，$K = 0.01$）。

運動模糊 + 雜訊

Wiener，K=0.01

帶有斜向運動模糊與雜訊的 cameraman 影像，與 Wiener 濾波結果比較

受限最小平方濾波

白話說：在所有「扣掉預期雜訊量後，能解釋觀測結果」的影像之中，挑最平滑的那一張。

Wiener 濾波器需要功率頻譜，或一個意義不明確的常數 K。**受限最小平方**（constrained least squares, CLS）濾波只需要雜訊的平均值與變異數 [3]。以矩陣向量形式，把像素排成向量，模型為

$$\mathbf{g} = \mathbf{H}\mathbf{f} + \boldsymbol{\eta},$$

其中 $\mathbf{g}, \mathbf{f}, \boldsymbol{\eta}$ 是 $MN \times 1$ 向量，$\mathbf{H}$ 是 $MN \times MN$ 的模糊矩陣。這些矩陣大到無法直接求反矩陣，但因為 $\mathbf{H}$ 代表循環卷積，它可以被 DFT 對角化，這正是頻域閉式解得以存在的原因。

這類反問題是**病態**的（ill-conditioned）：$\mathbf{g}$ 的微小變化會造成解的巨大變化。CLS 的對策是偏好平滑的解。它以拉普拉斯運算子 ∇^2 衡量粗糙程度，求解

$$\min_{\hat{\mathbf{f}}} \; C = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} \left[\nabla^2 \hat{f}(x,y)\right]^2 \quad \text{subject to} \quad \|\mathbf{g} - \mathbf{H}\hat{\mathbf{f}}\|^2 = \|\boldsymbol{\eta}\|^2,$$

其中 $\|\cdot\|$ 為歐氏範數。這個限制條件的意思是：殘差應該剛好和雜訊一樣大，不能更大（等於忽略資料），也不能更小（等於在擬合雜訊）。用拉格朗日乘數求得的解為

$$\hat{F}(u,v) = \left[\frac{H^*(u,v)}{|H(u,v)|^2 + \gamma\,|P(u,v)|^2}\right] G(u,v),$$

其中 $\gamma \geq 0$ 是乘數，$P(u,v)$ 是拉普拉斯卷積核

$$p(x,y) = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

補零到 $M \times N$ 之後的 DFT。$\gamma = 0$ 時回到逆濾波器。和常數 K 的 Wiener 濾波器比較：CLS 把平坦的 K 換成 $\gamma|P|^2$，它在低頻小、高頻大，因此正好在雜訊容易佔優勢的地方收手。

如何選 γ

γ 不是可以隨意轉的旋鈕：它必須讓限制條件成立。定義殘差 $\mathbf{r} = \mathbf{g} - \mathbf{H}\hat{\mathbf{f}}$ 與 $\phi(\gamma) = \|\mathbf{r}\|^2$，可以證明 ϕ 隨 γ 單調遞增。因此：

1. 從某個 γ 開始。
2. 計算 $\hat{F}$ 與殘差 $\|\mathbf{r}\|^2$（用 FFT 都很便宜）。
3. 若 $\|\mathbf{r}\|^2 < \|\boldsymbol{\eta}\|^2 - a$，增大 γ；若 $\|\mathbf{r}\|^2 > \|\boldsymbol{\eta}\|^2 + a$，減小 γ；否則停止。a 是一個小的容許誤差。

目標值只需要雜訊統計量：$\|\boldsymbol{\eta}\|^2 = MN\,(\sigma_\eta^2 + \bar{\eta}^2)$，$\sigma_\eta^2$ 與 $\bar{\eta}$ 是雜訊的變異數與平均值，而我們已經知道如何從平坦區塊估計它們。

```
def cls(G, H, gamma):
    p = np.zeros(H.shape)
    p[[0, 0, 0, 1, -1], [0, 1, -1, 0, 0]] = [4, -1, -1, -1, -1]   # Laplacian centred at (0,
    ↪  0)
    P = np.fft.fft2(p)
    return np.real(np.fft.ifft2(np.conj(H) / (np.abs(H) ** 2 + gamma * np.abs(P) ** 2) * G))

f_hat = cls(G, H, gamma=0.002)
```

Wiener 和 CLS 哪個比較好？一般而言沒有定論。Wiener 是對一群具有假設功率頻譜的影像平均而言最佳；CLS 則針對手上這一張影像，當雜訊統計量已知時，對這張影像可能得到更好的結果。若 K 或 γ 是手動調的（這很常見），兩者看起來往往差不多（圖 5.5）。

幾何平均濾波器

白話說：一個旋鈕，在「完全還原模糊」和「對雜訊保守一點」之間滑動。

幾何平均濾波器（geometric mean filter）結合了逆濾波器與（參數化）Wiener 濾波器：

$$\hat{F}(u,v) = \left[\frac{H^*(u,v)}{|H(u,v)|^2}\right]^{\alpha} \left[\frac{H^*(u,v)}{|H(u,v)|^2 + \beta\,\dfrac{S_\eta(u,v)}{S_f(u,v)}}\right]^{1-\alpha} G(u,v),$$

其中 α 與 β 是非負實數常數。特殊情況：

- $\alpha = 1$：逆濾波器。
- $\alpha = 0$：**參數化 Wiener 濾波器**；$\beta = 1$ 時就是標準 Wiener 濾波器。
- $\alpha = 1/2$、$\beta = 1$：逆濾波器與 Wiener 濾波器的幾何平均，這也是這個家族名稱的由來。它又稱為**頻譜等化濾波器**（spectrum equalization filter）：可以驗證此時輸出的期望功率頻譜 $|W|^2(|H|^2 S_f + S_\eta)$ 剛好等於原始影像的功率頻譜 S_f。

$\beta > 1$ 時濾波器比 Wiener 更保守；$\beta < 1$ 時更大膽。

```
def geometric_mean_filter(G, H, alpha, beta, K):
    """K stands in for S_eta / S_f, as in the constant-K Wiener filter."""
    H2 = np.abs(H) ** 2
    inv = np.conj(H) / np.maximum(H2, 1e-12)
    wie = np.conj(H) / (H2 + beta * K)
    # Both factors have the phase of conj(H): combine magnitudes, keep that phase.
    return np.real(np.fft.ifft2(np.abs(inv) ** alpha * np.abs(wie) ** (1 - alpha)
                                * np.exp(1j * np.angle(wie)) * G))
```

從投影重建影像

白話說：從許多角度讓 X 光穿過身體。每個角度得到一個「影子」，它只告訴我們光線一路總共穿過了多少東西。把夠多的影子巧妙地組合起來，就能重建出身體內部的切面。

電腦斷層（CT）原理

在 X 光 CT 中，射源與一排偵測器繞著病人旋轉。在每個角度，每個偵測器量測 X 光沿其直線路徑被衰減了多少。沿一條射線的衰減是累加的（強度比的對數等於衰減係數的線積分），所以每個量測值都是未知斷層 $f(x,y)$ 的一個**線積分**。同一角度的一組量測值稱為一個**投影**（projection）。CT 掃描儀從平移加旋轉的單一射源—偵測器對（平行射束），演進到配有多個偵測器的扇形射束，再到能擷取整個體積的螺旋與錐形射束；但下面的重建數學是它們共同的核心 [4]。

投影與 Radon 轉換

以法線式描述一條直線：$x\cos\theta + y\sin\theta = \rho$，$\rho$ 是它到原點的距離，θ 是法線的角度。f 沿該直線的線積分為

$$g(\rho,\theta) = \int_{-\infty}^{\infty}\int_{-\infty}^{\infty} f(x,y)\,\delta(x\cos\theta + y\sin\theta - \rho)\,dx\,dy,$$

其中 δ 是 Dirac 脈衝，只保留直線上的點。把 g 視為 (ρ,θ) 的函數，就是 f 的 **Radon 轉換**（Radon transform）。以 θ 為一軸、ρ 為另一軸畫成影像，稱為**正弦圖**（sinogram）：斷層中的單一點會畫出一條正弦曲線 $\rho = x_0\cos\theta + y_0\sin\theta$。

最簡單的逆向做法是**反投影**（backprojection）：把每個投影沿著它來的方向抹回整張影像，再把所有角度的抹痕加起來：

$$f_\theta(x,y) = g(x\cos\theta + y\sin\theta,\ \theta), \qquad f_{BP}(x,y) = \int_0^{\pi} f_\theta(x,y)\,d\theta.$$

這會把質量放到正確的位置，但也會放到每條射線經過的所有地方，所以結果是 f 被嚴重模糊的版本（圖 5.6 第三格）。這個模糊並非隨機：它等於與 $1/r$ 做卷積，r 是到中心的距離。傅立葉切片定理告訴我們如何去除它。

傅立葉切片定理

令 $G(\omega,\theta)$ 為固定角度 θ 下，投影 $g(\rho,\theta)$ 對 ρ 的一維傅立葉轉換：

$$G(\omega,\theta) = \int_{-\infty}^{\infty} g(\rho,\theta)\,e^{-j2\pi\omega\rho}\,d\rho.$$

傅立葉切片定理（Fourier-slice theorem，又稱投影切片定理）指出

$$G(\omega,\theta) = F(\omega\cos\theta,\ \omega\sin\theta),$$

其中 $F(u,v)$ 是 f 的二維傅立葉轉換。換句話說：一個投影的一維轉換，等於物體二維轉換中通過原點、角度為 θ 的一條徑向切片。證明很短：把 Radon 積分代入 G，積掉 delta，指數就變成 $-j2\pi\omega(x\cos\theta + y\sin\theta)$，這正是二維轉換在 $(u,v) = (\omega\cos\theta, \omega\sin\theta)$ 的值。

所以原則上，所有角度的投影會一片一片填滿二維頻譜，再做二維反轉換就得到 f。實務上這些切片是在極座標網格上取樣：原點附近密、遠處稀，內插到直角座標網格容易出錯。這種不均勻取樣也是單純反投影會模糊的原因：它過度加權了低頻。

平行射束的濾波反投影

把二維反傅立葉轉換寫成極座標 $u = \omega\cos\theta$、$v = \omega\sin\theta$。面積元素 $du\,dv$ 變成 $|\omega|\,d\omega\,d\theta$，再用傅立葉切片定理：

$$f(x,y) = \int_0^{\pi} \left[\int_{-\infty}^{\infty} |\omega|\, G(\omega,\theta)\, e^{j2\pi\omega\rho}\, d\omega \right]_{\rho = x\cos\theta + y\sin\theta} d\theta.$$

由內往外讀：對每個角度，把投影的一維頻譜乘上 $|\omega|$（**斜坡濾波器**，ramp filter），再轉回來；接著把濾波後的投影反投影，並對所有角度加總。這就是**濾波反投影**（filtered backprojection, FBP）。斜坡濾波器正好抵消了單純反投影的 $1/r$ 模糊。

斜坡濾波器會無限增長，所以會放大高頻雜訊，而且不可積分。實務上會限制頻寬並乘上平滑的窗函數，例如 Hamming 窗：在頻帶 $|\omega| \le W/2$ 內 $h(\omega) = c + (1-c)\cos(2\pi\omega/W)$，頻帶外為 0，$W$ 是頻寬，$c = 0.54$。窗函數在 $\omega = 0$ 時為 1，在頻帶邊緣降到 $2c - 1 = 0.08$。它犧牲一點銳利度，換來少得多的振鈴與雜訊。Shepp 與 Logan 在提出那個著名頭部假體（phantom）的論文中一併提出的濾波器 [5]，至今仍是標準選項之一。視角數量也很重要：角度太少時，FBP 會產生條紋（圖 5.6 最後一格）。

```
import numpy as np
from skimage.data import shepp_logan_phantom
from skimage.transform import radon, iradon, rescale

f = rescale(shepp_logan_phantom(), 0.64)                 # 256 x 256
theta = np.linspace(0.0, 180.0, 180, endpoint=False)     # projection angles in degrees
sinogram = radon(f, theta=theta)                         # shape: (detector bins, angles)
backproj = iradon(sinogram, theta=theta, filter_name=None)    # blurry, no filter
fbp = iradon(sinogram, theta=theta, filter_name="ramp")       # filtered backprojection
rmse = np.sqrt(np.mean((fbp - f) ** 2))
```

`iradon` 也接受 `"shepp-logan"`、`"cosine"`、`"hamming"` 與 `"hann"` 窗函數，而 `skimage.transform.iradon_sart` 提供迭代式的替代方案 [6]。

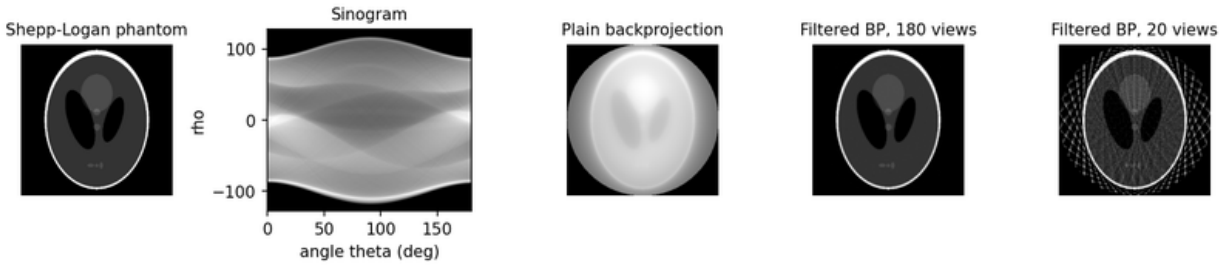


圖 *5.6* — 從斷層到正弦圖，再回到斷層。假體中的每一點在正弦圖中畫出一條正弦曲線。單純反投影得到模糊的光暈；加上斜坡濾波器（*FBP*）就還原出斷層。只有 *20* 個視角時，*FBP* 出現條紋偽影，這正是學習式重建方法要解決的問題（見現代觀點）。

扇形射束濾波反投影

現代掃描儀使用**扇形射束**（fan beam）：一個點射源搭配一排彎曲或平面的偵測器，因此同一視角中的射線並不平行。每條扇形射線仍是某條直線 (ρ, θ)，所以有兩條路可走。**重新分組**（rebinning）把來自許多視角的扇形射線整理成一組組平行射線，再套用平行 FBP。**直接扇形 FBP** 則在 FBP 積分中變數變換：投影先乘上一個取決於射線在扇形內角度的餘弦權重，再用修改過的斜坡濾波器濾波，最後以取決於與射源距離的權重做反投影。整體結構「加權－濾波－反投影」不變 [4]。

現代觀點

本章的工具不是博物館裡的展品。$g = h \star f + \eta$ 仍是這個領域陳述問題的方式，類似 Wiener 的步驟也仍藏在現代網路裡。深度學習改變的是先驗（prior）：我們不再假設「影像是平滑的」（CLS）或「影像的功率頻譜是 S_f」（Wiener），而是從資料中學習乾淨影像長什麼樣子。想更全面地了解本章以外的深度影像復原（超解析度、除霧、除雨、低光增強、all-in-one 復原與擴散先驗），可以參考DLCV 04・深度影像復原。以下的綜述聚焦在本章的三個主題：去雜訊、去模糊與 CT 重建。

去雜訊：從區塊到網路

兩個經典方法代表了手工設計去雜訊的巔峰。**非局部平均**（non-local means, NLM）[7] 把每個像素換成一群「周圍區塊看起來相似」的像素的加權平均，不管它們在影像的哪裡。這是把自適應濾波器的想法推得更遠：「鄰域」由相似度而非距離定義。**BM3D** [8] 把相似區塊堆成三維堆疊，在轉換域中聯合濾波，第二階段再以第一階段結果作為 S_f 的估計，套用 Wiener 型濾波器。它當了大約十年的標準基準。

DnCNN [9] 證明了一個以殘差學習（預測雜訊而非乾淨影像）加上批次正規化（batch normalization）訓練的單純深度卷積網路，在高斯雜訊上勝過 BM3D，而且單一模型能處理未知的雜訊強度。

- **Tian 等人，〈Deep learning on image denoising: An overview〉（2020）** [10]。把 CNN 去雜訊方法分成四類：加成性白高斯雜訊、真實（相機）雜訊、盲去雜訊（雜訊未知），以及同時含有模糊或低解析度的混合情況。重點（我們的解讀）：這個分類本身就是依雜訊類型組織的，真實相機雜訊與未知雜訊強度被列為和合成高斯雜訊不同的類別。本章的雜訊模型以及它有多貼近真實，在深度去雜訊中依然關鍵。
- **Elad、Kawar 與 Vaksman，〈Image Denoising: The Deep Learning Revolution and Beyond〉（2023）** [11]。回顧從經典先驗到深度去雜訊器的歷史，並主張去雜訊器的意義遠超過去雜訊本身：好的去雜訊器可以當作解其他反問題時的先驗，也是擴散式影像生成的引擎。重點：本章「只有雜訊」的經典問題，已經成為幾乎所有其他問題的基本構件。

去模糊與一般反問題

- **Zhang 等人，〈Deep Image Deblurring: A Survey〉（IJCV 2022）** [12]。涵蓋模糊成因、資料集與評估指標，再依架構與損失函數整理以 CNN 為基礎的非盲與盲去模糊方法，以及特定領域（人臉、文字、立體影像）。重點：非盲去模糊（如本章，h 已知）主要在對付我們在逆濾波看到的雜訊放大與振鈴；盲去模糊還得估計 h，至今仍是比較難的問題。
- **Ongie 等人，〈Deep Learning Techniques for Inverse Problems in Imaging〉（2020）** [13]。為任何 $\mathbf{g} = \mathbf{H}\mathbf{f} + \boldsymbol{\eta}$ 問題提出分類：前向模型 $\mathbf{H}$ 在訓練時已知、只在測試時已知，還是完全未知？訓練是監督式還是非監督式？重點（我們的解讀）：對 $\mathbf{H}$ 了解多少，是第一個設計決策，綜述也討論了這些方法的失效模式。像本章這樣已知 $\mathbf{H}$，就能使用純端對端網路忽略掉的選項。
- **Kamilov、Bouman、Buzzard 與 Wohlberg，〈Plug-and-Play Methods for Integrating Physical and Learned Models in Computational Imaging〉（2023）** [14]。回顧隨插即用（plug-and-play, PnP）先驗：在「滿足物理（$\mathbf{g} \approx \mathbf{H}\mathbf{f}$）」的步驟與「呼叫去雜訊器（代替先驗）」的步驟之間交替的迭代演算法。重點：前向模型和學習式先驗可以分開開發，執行時再組合。DPIR [15] 是廣泛使用的例子，它把深度 CNN 去雜訊器插入半二次分裂（half-quadratic splitting）演算法，用於去模糊、超解析度與去馬賽克。

這和本章的關係很直接。對模糊 H，這種分裂方法的物理步驟要在每個頻率求使 $|G - HF|^2 + \mu|F - Z|^2$ 最小的 $\hat{F}$，其中 Z 是目前去雜訊後的估計，μ 是懲罰權重。它的解是 $\hat{F} = (H^*G + \mu Z)/(|H|^2 + \mu)$，也就是一個被拉向 Z（而不是拉向零）的常數 K Wiener 濾波器。

CT 重建：學習式與展開式

資料完整、雜訊低時，FBP 又快又準。但為了降低輻射劑量而減少資料時，它就吃力了：不論是視角變少（條紋，圖 5.6），還是每個視角的光子變少（雜訊）。經典解法是搭配全變分（total variation）等手工先驗的迭代方法。深度學習提供三種主要模式：

- **後處理：**先做 FBP，再用 CNN 去除偽影。**FBPConvNet** [16] 正是用 CNN 這樣做，並報告在稀疏視角 CT 上比全變分重建品質更好，而執行時間只需一小部分。
- **展開式（學習式迭代）重建：**取一個迭代演算法，固定迭代次數，把每次迭代的部分步驟換成小型網路，再端對端訓練。**Learned Primal-Dual** [17] 展開一個原始－對偶（primal-dual）演算法，並在每次迭代中保留前向投影與反投影。**Monga、Li 與 Eldar，〈Algorithm**

Unrolling〉（2019） [18] 綜述了這種設計模式在訊號與影像處理中的應用。重點：由於每一層都對應演算法的一個步驟，展開式網路比通用網路更容易解讀，也更有效率。

- **Wang、Ye 與 De Man，〈Deep learning for tomographic image reconstruction〉（2020）** [19] 回顧 CT 及其他斷層成像模態的全貌：影像域、感測器域與混合式網路、融入物理的設計，以及資料取得、穩健性與臨床驗證等開放問題。

沒有改變的是：Radon 轉換與傅立葉切片定理定義了上述每個方法都使用的前向模型，而 FBP 仍是臨床掃描儀的預設方法，也是許多學習式方法的輸入或起點。學習式方法也帶來本章線性濾波器沒有的新風險：網路可能產生看似合理、但資料中並不存在的結構。這就是為什麼在醫療應用上，必須在真實且分布外的掃描上做驗證。

重點整理

- 復原是客觀的：把退化建模為 $g = h \star f + \eta$（或 $G = HF + N$），再把這個模型反過來。
- 從平坦區塊的直方圖辨認雜訊類型，再從同一區塊估計平均值與變異數。
- 讓空間濾波器對上雜訊：類高斯雜訊用平均類濾波器；單邊脈衝用正負號正確的反調和濾波器；椒鹽雜訊用中值與自適應中值；混合雜訊用 alpha 修剪平均。
- 週期性雜訊是頻譜中的幾個亮點；陷波濾波器去除它們，最佳陷波濾波則在局部自適應地調整減去的量。
- 逆濾波 G/H 會在 $|H|$ 小的地方放大雜訊。Wiener、受限最小平方與幾何平均濾波器都在分母加上一項，抑制這些頻率。
- CT 量測的是線積分（Radon 轉換）。依傅立葉切片定理，每個投影提供二維頻譜的一條切片，而濾波反投影（先斜坡濾波、再反投影）重建出影像。
- 深度學習用學來的先驗取代手工先驗，但最好的方法仍把前向模型保留在內（隨插即用、展開式網路）。

練習

1. 一張暗視野顯微影像的背景區域直方圖是單邊的：在 12 處陡然開始，右側拖著長尾；平均值為 20，變異數為 64（灰階單位）。你會先試本章的哪一種 PDF？參數是多少？

   提示

   陡然開始加上長尾，指向指數、Rayleigh 或 Erlang 家族。試試平移到 12 開始的指數分布：需要 $1/a = 20 - 12 = 8$ 且 $1/a^2 = 64$，兩者在 $a = 1/8$ 時同時成立。Rayleigh 則不一致：$b(4-\pi)/4 = 64$ 得到 $b \approx 298$，它的起點會是 $20 - \sqrt{\pi b/4} \approx 4.7$，而不是 12。

2. 一張影像只受到鹽雜訊污染。先不做實驗，預測下列做法的結果：(a) 最大值濾波器、(b) 最小值濾波器、(c) $Q = 2$ 的反調和濾波器、(d) $Q = -2$ 的反調和濾波器。再用本章的程式驗證。

   提示

   鹽像素是視窗中最大的值。最大值濾波器會把它們擴散（更糟），最小值濾波器會去除它們（但會讓亮的細節變暗、變細）。正的 Q 加重大值的權重，所以 $Q = 2$ 讓鹽雜訊更糟；$Q = -2$ 則能去除它。

3. 證明常數 K 的 Wiener 濾波器在 $K = 0$ 時退化成逆濾波器，而在 $|H|^2 \ll K$ 的頻率上輸出約為 $(H^*/K)\,G$。這說明 Wiener 濾波如何對待被模糊摧毀的頻率？

   提示

   當 $|H|^2 \ll K$ 時分母約為 K。輸出因此很小，與 $|H|$ 成正比，所以濾波器不會試圖找回這些頻率。它體面地放棄，而不是放大雜訊。

4. 一張 256×256 的影像沿 x 方向有 8 個像素的等速運動模糊，y 方向沒有。頻率以整數（每張影像的週期數）表示，與上面的程式一致。利用運動模糊 OTF，找出所有 $H(u, v) = 0$ 的頻率 (u, v)。解釋為什麼沒有任何線性濾波器能還原這些頻率的細節，以及這在復原影像中會呈現什麼樣子。

   提示

   當 u 以每張影像的週期數表示時，a 是模糊長度佔影像尺寸的比例，所以 $a = 8/256 = 1/32$、$b = 0$。當 $u/32$ 為非零整數時 $H = 0$：$u = \pm 32, \pm 64, \pm 96, \pm 128$，對所有 v 都成立。由於 $G = HF + N$ 在這些頻率不含任何 F 的成分，任何線性濾波器在那裡只能輸出雜訊（或零）。預期會看到沿 x 方向、以該空間週期出現的淡淡振鈴。

5. 用上面的 CT 程式，分別以 180、60、20、10 個視角，以及 `ramp` 與 `hann` 濾波器重建。畫出 RMSE 對視角數的關係。接著在正弦圖加上高斯雜訊再重複一次。什麼時候較平滑的 `hann` 濾波器會勝出？

   提示

   沒有雜訊、視角很多時，`ramp` 應該勝出。雜訊增加後，斜坡濾波器在高頻放大雜訊，而在高頻衰減的 `hann` 就會勝出。這和逆濾波器對 Wiener 濾波器是同一種取捨。

6. 推導 $\min_F |G - HF|^2 + \mu |F - Z|^2$ 的逐頻率解 $\hat{F} = (H^* G + \mu Z)/(|H|^2 + \mu)$。當 $Z = 0$ 時，你得到本章的哪一個濾波器？

   提示

   對 F^* 微分：$-H^*(G - HF) + \mu(F - Z) = 0$。$Z = 0$ 時就是常數 $K = \mu$ 的 Wiener 濾波器。

參考文獻

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 5. publisher page
2. H. Hwang and R. A. Haddad, “Adaptive median filters: new algorithms and results,” *IEEE Transactions on Image Processing*, vol. 4, no. 4, pp. 499–502, 1995. DOI
3. B. R. Hunt, “The application of constrained least squares estimation to image restoration by digital computer,” *IEEE Transactions on Computers*, vol. C-22, no. 9, pp. 805–812, 1973. DOI

4. A. C. Kak and M. Slaney, *Principles of Computerized Tomographic Imaging*, IEEE Press, 1988 (SIAM reprint 2001). free online edition
5. L. A. Shepp and B. F. Logan, "The Fourier reconstruction of a head section," *IEEE Transactions on Nuclear Science*, vol. 21, no. 3, pp. 21–43, 1974. DOI
6. scikit-image developers, "Radon transform," scikit-image documentation (gallery example). docs
7. A. Buades, B. Coll and J.-M. Morel, "A non-local algorithm for image denoising," *IEEE CVPR*, vol. 2, pp. 60–65, 2005. DOI
8. K. Dabov, A. Foi, V. Katkovnik and K. Egiazarian, "Image denoising by sparse 3-D transform-domain collaborative filtering," *IEEE Transactions on Image Processing*, vol. 16, no. 8, pp. 2080–2095, 2007. DOI
9. K. Zhang, W. Zuo, Y. Chen, D. Meng and L. Zhang, "Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising," *IEEE Transactions on Image Processing*, vol. 26, no. 7, pp. 3142–3155, 2017. doi ·arXiv
10. C. Tian, L. Fei, W. Zheng, Y. Xu, W. Zuo and C.-W. Lin, "Deep learning on image denoising: An overview," *Neural Networks*, vol. 131, pp. 251–275, 2020. arXiv
11. M. Elad, B. Kawar and G. Vaksman, "Image denoising: The deep learning revolution and beyond — A survey paper," *SIAM Journal on Imaging Sciences*, vol. 16, no. 3, pp. 1594–1654, 2023. arXiv
12. K. Zhang, W. Ren, W. Luo, W.-S. Lai, B. Stenger, M.-H. Yang and H. Li, "Deep image deblurring: A survey," *International Journal of Computer Vision*, 2022. arXiv
13. G. Ongie, A. Jalal, C. A. Metzler, R. G. Baraniuk, A. G. Dimakis and R. Willett, "Deep learning techniques for inverse problems in imaging," arXiv:2005.06001, 2020. arXiv
14. U. S. Kamilov, C. A. Bouman, G. T. Buzzard and B. Wohlberg, "Plug-and-play methods for integrating physical and learned models in computational imaging: Theory, algorithms, and applications," *IEEE Signal Processing Magazine*, vol. 40, no. 1, pp. 85–97, 2023. arXiv
15. K. Zhang, Y. Li, W. Zuo, L. Zhang, L. Van Gool and R. Timofte, "Plug-and-play image restoration with deep denoiser prior," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 10, pp. 6360–6376, 2022. arXiv
16. K. H. Jin, M. T. McCann, E. Froustey and M. Unser, "Deep convolutional neural network for inverse problems in imaging," *IEEE Transactions on Image Processing*, vol. 26, no. 9, pp. 4509–4522, 2017. arXiv
17. J. Adler and O. Öktem, "Learned primal-dual reconstruction," *IEEE Transactions on Medical Imaging*, 2018. arXiv
18. V. Monga, Y. Li and Y. C. Eldar, "Algorithm unrolling: Interpretable, efficient deep learning for signal and image processing," arXiv:1912.10557, 2019. arXiv
19. G. Wang, J. C. Ye and B. De Man, "Deep learning for tomographic image reconstruction," *Nature Machine Intelligence*, vol. 2, no. 12, pp. 737–748, 2020. DOI

IP 06・色彩影像處理

色彩如何量測與編碼、如何在 RGB、HSI 與 L*a*b* 之間轉換，以及如何對彩色影像做轉換、濾波與分割。

網站版本: https://blog.shyandram.dev/zh-tw/learn/image-processing/dip-06-color/

你將學到

- 為什麼對人類觀察者來說，三個數字就足以描述一種顏色；CIE 色度圖與裝置色域是什麼
- RGB、CMY(K)、HSI 與 CIE L*a*b* 之間的關係與精確轉換公式，以及 sRGB gamma 與 YCbCr 的定位
- 偽彩色如何把灰階影像變成好讀的彩色圖，以及為什麼 viridis 這類感知均勻色彩對應表勝過彩虹色
- 如何對全彩影像做轉換、白平衡、等化、平滑、銳化與分割，以及何時該逐通道處理、何時該當成向量處理
- 彩色影像中的雜訊有什麼特性，以及向量濾波器如何去除雜訊
- 近年研究（基於學習的色彩恆常性、色彩對應表設計、深度學習上色）為經典工具帶來了什麼

先看全貌

到目前為止，每個像素都只有一個數字：亮度。彩色影像的每個像素則有三個數字。色彩幫助我們找到物體（「那台紅色的車」）、區分材質，也是大家看照片時第一個察覺「不對勁」的地方。

> **用最簡單的話說**
>
> 想像你只有三支手電筒：一支紅的、一支綠的、一支藍的。把它們照在白牆上，調亮調暗，幾乎可以混出任何顏色：紅加綠變黃，三支一起變白。你的眼睛剛好反過來運作：眼睛裡有三種小小的色彩感測器，大腦把三個訊號混合成你看到的每一種顏色。一張彩色影像，其實就是替每一個點記下「三支手電筒各要多亮」。

依照教科書的編排 [1]，本章回答三個問題：怎麼精確描述色彩（色彩基礎與色彩模型）、怎麼用色彩呈現本身沒有顏色的資料（偽彩色），以及怎麼處理真正的彩色照片（轉換、濾波、分割、雜訊、壓縮）。第 3 到 5 章的灰階工具大多能沿用，但有一個反覆出現的關鍵：彩色像素是一個**向量**，把它當成三個互不相干的數字，有時沒問題，有時錯得離譜。

色彩基礎

白話版。光是許多波長的混合。眼睛不會量測整個混合，而是把它壓縮成三個數字，每種色彩感測器一個。兩種差很多的光，只要產生相同的三個數字，在我們看來就一模一樣。

光、視錐細胞與三色視覺

可見光的波長大約在 400 到 700 奈米之間。光源或物體表面可以用**光譜功率分布**（spectral power distribution）$S(\lambda)$ 描述：在每個波長 λ 發出或反射多少能量。

視網膜有三種**視錐細胞**（cones，對色彩敏感的感光細胞），分別對短、中、長波長最敏感。每一種視錐細胞 $c \in \{S, M, L\}$ 都有一條敏感度曲線 $s_c(\lambda)$，其反應為

$$r_c = \int S(\lambda)\, s_c(\lambda)\, d\lambda.$$

其中 $S(\lambda)$ 是入射光譜，$s_c(\lambda)$ 是第 c 種視錐細胞的敏感度。整條光譜被化約成三個數字，所以人類色覺是三**色視覺**（trichromatic）。直接的結果是**同色異譜**（metamerism）：兩條不同的光譜 $S_1 \neq S_2$ 只要產生相同的三個反應，看起來就是同一種顏色。這正是螢幕能運作的原因：螢幕從來沒有重現檸檬的光譜，只重現了它的三個視錐反應。

亮度、色相與飽和度

人用三個知覺屬性描述顏色：

- **亮度**（brightness）：看起來有多少光（對無色的灰來說，這就是全部）。
- **色相**（hue）：主波長，也就是我們用紅、橘、藍這些字叫出來的東西。
- **飽和度**（saturation）：顏色有多純，也就是摻了多少白。粉紅色是低飽和度的紅。

色相與飽和度合稱**色度**（chromaticity），所以顏色＝「色度＋亮度」，這正是 HSI 模型背後的想法。

CIE 三刺激值與色度圖

為了讓色彩可以量測，國際照明委員會（CIE）在 1931 年根據人類配色實驗，制定了三條**配色函數**（color-matching functions）$\bar{x}(\lambda), \bar{y}(\lambda), \bar{z}(\lambda)$。一條光譜的三**刺激值**（tristimulus values）為

$$X = \int S(\lambda)\, \bar{x}(\lambda)\, d\lambda, \qquad Y = \int S(\lambda)\, \bar{y}(\lambda)\, d\lambda, \qquad Z = \int S(\lambda)\, \bar{z}(\lambda)\, d\lambda.$$

X, Y, Z 依設計都是非負的，而 $\bar{y}$ 被選成符合人眼的亮度敏感度，所以 Y 就是**輝度**（luminance）。把色度與亮度分開的方法是正規化：

$$x = \frac{X}{X+Y+Z}, \qquad y = \frac{Y}{X+Y+Z}, \qquad z = \frac{Z}{X+Y+Z} = 1 - x - y.$$

x, y 是**色度座標**，z 是多餘的。把每一個純光譜色（單一波長）畫在 (x, y) 平面上，會得到一條馬蹄形曲線，稱為**光譜軌跡**（spectral locus）。連接兩端的直線是**紫線**（line of purples；紫色不是單一波長）。人類看得到的所有顏色都落在這個區域內。

色度圖好用，是因為兩個幾何性質：

1. **混色是線性的**。兩種光的任意混合落在兩者色度之間的線段上；三種光的混合落在它們圍成的三角形內。

2. **白色在中間**。參考白點，例如日光標準光源 D65，位於 $(x, y) \approx (0.3127, 0.3290)$，靠近中央。從白點往軌跡移動，飽和度增加；移動的方向決定色相。

色域

使用三個原色的顯示器，只能產生三原色圍成的三角形內的顏色。這個三角形就是裝置的**色域**（gamut）。由於光譜軌跡是彎的，沒有任何角落可實際製造的三角形能完全涵蓋它，所以每一台三原色顯示器都會漏掉某些可見顏色，尤其是高飽和的青色與綠色。

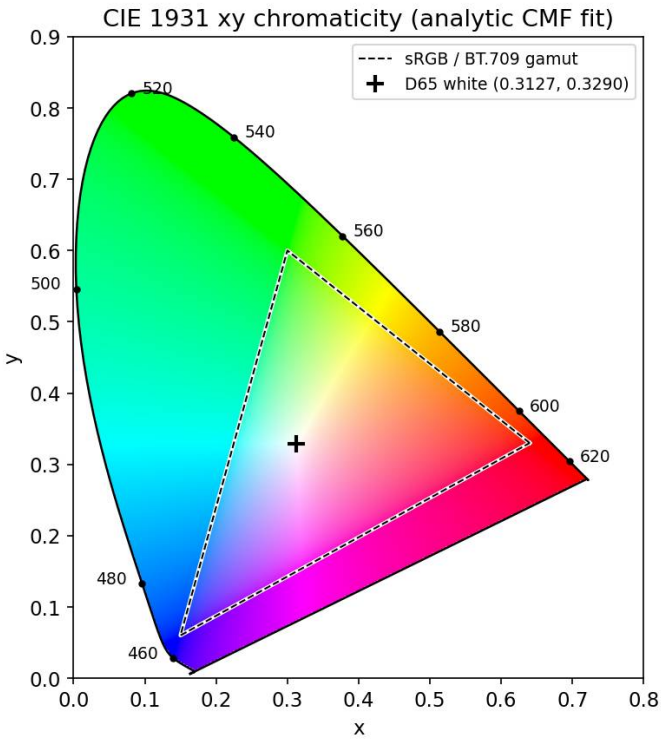


圖 *6.1* — 以 *Wyman* 等人 [2] 的解析配色函數擬合計算出的 *CIE 1931 xy* 色度圖（*430–650 nm*；擬合在兩端尾巴不夠準，故省略）。虛線三角形是 *sRGB / BT.709* 色域 [3], [4]；三角形外的顏色在你的螢幕上只能近似呈現。

CIE 函數原本是很長的數值表；Wyman、Sloan 與 Shirley [2] 用幾個分段高斯函數做擬合，精度足以畫出這樣的圖。每一個波瓣是 $a \exp(-\frac{1}{2}t^2)$，$t = (\lambda - \mu)/\sigma$，而寬度 σ 在峰值 μ 左右兩側不同。

色彩模型

白話版。色彩模型就是顏色的座標系統。不同工作需要不同座標：螢幕用紅綠藍光思考，印表機用墨水思考，人則用「什麼顏色、多鮮豔、多亮」思考。

RGB

在 **RGB 模型**中，每種顏色是單位立方體裡的一個點 (R, G, B)。黑色是 $(0,0,0)$，白色是 $(1,1,1)$，主對角線上是各種灰。每通道 8 位元的影像有 $2^{24} \approx 1{,}670$ 萬種可能顏色。RGB 是**加法混色**：描述光的相加，也就是顯示器與相機感測器的運作方式。

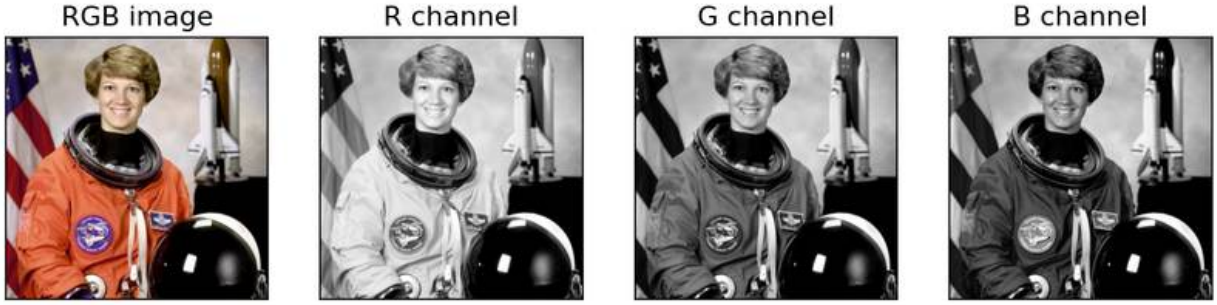


圖 *6.2 — RGB* 影像就是三張疊在一起的灰階影像。橘色太空衣在 R 通道很亮、在 B 通道很暗；太空衣的白色部分在三個通道都很亮。

單說「RGB」並不是精確的顏色：$(1,0,0)$ 在每台螢幕上都是不同的紅。**sRGB** 標準 [3] 規定了原色（與 ITU-R BT.709 [4] 相同的色度）、D65 白點與非線性編碼來解決這個問題。這個編碼常被籠統地稱為「gamma」，把儲存值 $C' \in [0,1]$ 轉成線性光 C：

$$C = \begin{cases} C'/12.92, & C' \le 0.04045, \\ \left(\dfrac{C' + 0.055}{1.055}\right)^{2.4}, & C' > 0.04045. \end{cases}$$

C' 是某個通道的儲存（編碼）值，C 是線性光的值。這條曲線把較多的碼值分配給暗部，因為人眼對暗部比較敏感。**實用原則：**模擬物理的運算（模糊、縮放、混光）在線性值上做才比較正確，所以在乎精度時要先解碼。

CMY 與 CMYK

印刷剛好相反。白紙上的墨水會吸收光，所以印刷是**減法混色**。青（cyan）、洋紅（magenta）、黃（yellow）分別吸收紅、綠、藍。數值在 $[0,1]$ 時：

$$\begin{bmatrix} C \\ M \\ Y \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} R \\ G \\ B \end{bmatrix}.$$

這裡 C, M, Y 是墨量（別和 CIE 的 Y 搞混）。真實的 C、M、Y 墨水等量混合會變成混濁的褐色而不是黑色，所以印表機加上第四種黑色墨水 K。一個簡單的轉換是

$$K = \min(C, M, Y), \qquad C_K = \frac{C - K}{1 - K}, \quad M_K = \frac{M - K}{1 - K}, \quad Y_K = \frac{Y - K}{1 - K} \quad (K < 1),$$

當 $K = 1$（純黑）時令 $C_K = M_K = Y_K = 0$。實際的印表機改用量測得到的色彩描述檔，但道理一樣：把顏色中共同的「灰」移到黑色墨水。

HSI

RGB 適合硬體，卻不符合人的直覺：沒有人會說一個顏色是「70% 紅、40% 綠、20% 藍」。**HSI 模型**（色相 hue、飽和度 saturation、強度 intensity）把亮度與色度分開，既貼近我們談論顏色的方式，也讓許多演算法變簡單。

從幾何上看，把 RGB 立方體用黑色那個角立起來，讓灰色對角線朝上。**強度**是沿這條軸的高度。用垂直於軸的平面切開立方體：**色相**是繞軸的角度（紅在 0°、綠在 120°、藍在 240°），**飽和度**是離軸的距離。

RGB 轉 HSI（$R, G, B \in [0,1]$）：

$$\theta = \cos^{-1}\!\left(\frac{\tfrac{1}{2}\big[(R-G)+(R-B)\big]}{\sqrt{(R-G)^2+(R-B)(G-B)}}\right), \qquad H = \begin{cases} \theta, & B \le G \\ 360^\circ - \theta, & B > G \end{cases}$$

$$S = 1 - \frac{3\,\min(R,G,B)}{R+G+B}, \qquad I = \frac{R+G+B}{3}.$$

θ 是像素的色彩分量與紅色軸的夾角。H 是以度為單位的色相，$S \in [0,1]$ 是飽和度，$I \in [0,1]$ 是強度。程式裡有兩個邊界情況要小心：灰色像素（$R=G=B$）時 θ 的分母為零，色相**沒有定義**，依慣例設為 0；黑色（$I=0$）時飽和度沒有定義，也設為 0。

HSI 轉 RGB 要看 H 落在哪一個 120° 扇區。在 RG 扇區（$0^\circ \le H < 120^\circ$）：

$$B = I(1-S), \qquad R = I\left[1 + \frac{S\cos H}{\cos(60^\circ - H)}\right], \qquad G = 3I - (R+B).$$

在 GB 扇區（$120^\circ \le H < 240^\circ$），先把 H 減去 120°，再套用同樣三條公式，但把 (B,R,G) 換成 (R,G,B)。在 BR 扇區（$240^\circ \le H < 360^\circ$），減去 240°，並把 (B,R,G) 換成 (G,B,R)。每個扇區中，與色相「相對」的那個通道得到最小值 $I(1-S)$。

```
import numpy as np

def rgb_to_hsi(rgb):
    """rgb: float array in [0, 1] with shape (..., 3). Returns H (degrees), S, I."""
    R, G, B = rgb[..., 0], rgb[..., 1], rgb[..., 2]
    num = 0.5 * ((R - G) + (R - B))
    den = np.sqrt((R - G) ** 2 + (R - B) * (G - B))
    theta = np.degrees(np.arccos(np.clip(num / (den + 1e-12), -1, 1)))
    H = np.where(B > G, 360.0 - theta, theta)
    I = rgb.mean(axis=-1)
    S = np.where(I > 0, 1 - rgb.min(axis=-1) / (I + 1e-12), 0.0)
    H = np.where(den > 1e-12, H, 0.0)      # gray pixels: hue undefined -> 0
    return H, S, I

def hsi_to_rgb(H, S, I):
    """H in degrees, S and I in [0, 1]. Inverse of rgb_to_hsi."""
    H = np.mod(H, 360.0)
    out = np.zeros(H.shape + (3,))
    # (sector start, index of the 'big' channel, of the smallest, of the rest)
    for lo, (a, b, c) in ((0, (0, 2, 1)), (120, (1, 0, 2)), (240, (2, 1, 0))):
        m = (H >= lo) & (H < lo + 120)
```

```
        h = np.radians(H[m] - lo)
        x = I[m] * (1 - S[m])
        y = I[m] * (1 + S[m] * np.cos(h) / np.cos(np.radians(60) - h))
        out[m, a], out[m, b], out[m, c] = y, x, 3 * I[m] - (x + y)
    return out

prim = np.array([[1, 0, 0], [0, 1, 0], [0, 0, 1], [0.5, 0.5, 0.5]], float)
print(np.round(np.stack(rgb_to_hsi(prim), -1), 3))
# [[  0.   1.   0.333] [120.  1.  0.333] [240.  1.  0.333] [  0.  0.  0.5 ]]
```

來回轉換 `hsi_to_rgb(*rgb_to_hsi(x))` 對隨機 RGB 輸入的誤差約在 10^{-9} 以內。

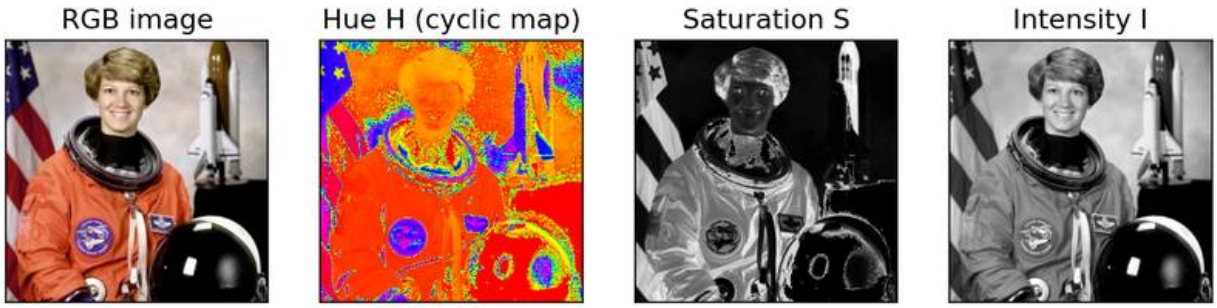


圖 *6.3 — HSI* 分量。色相用循環色彩對應表顯示，因為 *0°* 與 *360°* 是同一個色相。在灰色區域（背景、白色太空衣）色相幾乎沒有定義，所以看起來很雜亂；只有橘色太空衣、國旗與徽章的飽和度高。

> **HSI、HSV、HSL 是表親，不是雙胞胎**
> 函式庫通常實作 HSV（value = $\max(R,G,B)$）或 HSL，而不是上面的 HSI 公式。三者都把色相分離出來，但飽和度與亮度的定義不同，數值不能互換。OpenCV 還把 8 位元的色相存成 $H/2$，好塞進 $[0, 179]$。

CIE L*a*b*

HSI 把亮度和色度分開了，但 HSI 中相同的步長看起來並不一樣大。**CIE 1976 L*a*b*** 色彩空間 [5] 的設計目標，是讓歐氏距離大致對應感知上的差異。它由相對於參考白點 (X_n, Y_n, Z_n) 的三刺激值計算：

$$L^* = 116 f\!\left(\frac{Y}{Y_n}\right) - 16, \qquad a^* = 500\left[f\!\left(\frac{X}{X_n}\right) - f\!\left(\frac{Y}{Y_n}\right)\right], \qquad b^* = 200\left[f\!\left(\frac{Y}{Y_n}\right) - f\!\left(\frac{Z}{Z_n}\right)\right],$$

$$f(t) = \begin{cases} t^{1/3}, & t > \delta^3 \\ \dfrac{t}{3\delta^2} + \dfrac{4}{29}, & t \le \delta^3 \end{cases} \qquad \delta = \frac{6}{29}.$$

$L^* \in [0, 100]$ 是**明度**（lightness），a^* 從綠（負）到紅（正），b^* 從藍（負）到黃（正）。立方根模仿人眼的壓縮式反應，線性段則避免在接近黑色時斜率無限大。色差 ΔE^*_{ab} 就是兩個 (L^*, a^*, b^*) 點之間的歐氏距離。

從 sRGB 檔案出發：先解 gamma（上面的公式），把線性 RGB 乘上 sRGB 到 XYZ 的矩陣，再以 D65 白點套用上述公式。

```
import numpy as np
from skimage import color

M = np.array([[0.4124, 0.3576, 0.1805],      # linear sRGB -> XYZ (D65)
              [0.2126, 0.7152, 0.0722],
              [0.0193, 0.1192, 0.9505]])

def srgb_to_lab(rgb):
    lin = np.where(rgb <= 0.04045, rgb / 12.92, ((rgb + 0.055) / 1.055) ** 2.4)
    xyz = lin @ M.T / np.array([0.95047, 1.0, 1.08883])   # divide by D65 white
    d = 6 / 29
    f = np.where(xyz > d**3, np.cbrt(xyz), xyz / (3 * d**2) + 4 / 29)
    return np.stack([116 * f[..., 1] - 16,
                     500 * (f[..., 0] - f[..., 1]),
                     200 * (f[..., 1] - f[..., 2])], -1)

x = np.random.default_rng(0).random((1000, 3))
print(np.abs(srgb_to_lab(x) - color.rgb2lab(x[None])[0]).max())   # about 0.02
```

微小的殘差來自矩陣的四捨五入；`skimage.color.rgb2lab` 用了更多位數。

一段話看懂 YCbCr

影像與視訊編碼器會把**亮度訊號**（luma）Y' 和兩個**色差訊號**（chroma）分開，因為人眼對亮度細節的解析力遠高於色彩細節。ITU-R BT.601 [6] 對 gamma 編碼後的 $R', G', B' \in [0,1]$ 定義

$$Y' = 0.299\,R' + 0.587\,G' + 0.114\,B',$$

$C_b \propto B' - Y'$、$C_r \propto R' - Y'$，再縮放並平移到 8 位元的「studio」範圍（Y' 在 $[16, 235]$，C_b, C_r 在 $[16, 240]$）。HDTV 使用 BT.709 [4]，其亮度權重不同（$0.2126, 0.7152, 0.0722$）。兩者搞混會造成輕微但看得出來的偏色。`skimage.color.rgb2ycbcr` 實作的是 BT.601 studio 範圍版本。

偽彩色影像處理

白話版。偽彩色（pseudocolor，又稱假色）就是幫灰階影像上色，讓細微的亮度差異變得容易看見。這些顏色不是「真的」，而是一種代碼，就像氣象圖上的顏色。

動機來自知覺：人能分辨的色彩組合遠多於灰階層次。熱像儀、X 光、深度圖或高程模型每個像素只有一個值，但用顏色呈現這個值，可以顯露在灰階中會被忽略的結構。

強度切片

最簡單的做法是把灰階範圍切成數段，每段給一個顏色。設門檻 $0 = l_0 < l_1 < \cdots < l_P = L - 1$ 與顏色 $c_1, \ldots, c_P$：

$$g(x,y) = c_k \quad \text{if} \quad l_{k-1} \le f(x,y) < l_k.$$

f 是輸入灰階值，L 是灰階數，P 是分段數，g 是輸出顏色。若把 f 畫成 3-D 地形，每個門檻就是一個水平面切過「強度地形」，名稱由此而來。$P = 2$ 時就是用兩種顏色畫出的二值化。當分段本身有意義時（「溫度高於 40 °C」、「骨骼與軟組織」），強度切片（intensity slicing）最合適。

強度到色彩的轉換

更一般的做法是對灰階值套用三個獨立函數：

$$g_R(x,y) = T_R(f(x,y)), \qquad g_G(x,y) = T_G(f(x,y)), \qquad g_B(x,y) = T_B(f(x,y)).$$

T_R, T_G, T_B 把灰階值對應到紅、綠、藍輸出。強度切片是三者都是階梯函數的特例；平滑的函數則構成**連續色彩對應表**（colormap）。同樣的想法可推廣到多張輸入影像：對多光譜資料，把三個波段（例如近紅外、紅、綠）分別指定給顯示的三個通道，就得到假色合成影像。

感知均勻的色彩對應表

T_R, T_G, T_B 怎麼選，比看起來重要得多。常見的彩虹色對應表（`jet`）中段經過明亮的黃與青，兩端則變暗。它的**明度不是單調的**：資料上相等的步長在視覺上不相等，平坦區段看起來像邊緣，真正的邊緣反而可能消失。Kovesi [7] 指出，一些廣泛使用的色彩對應表有感知上的「平坦區」，可以藏住大到資料範圍十分之一的特徵，並主張色彩對應表應該設計成明度均勻變化。

viridis 由 van der Walt 與 Smith 設計，現在是 Matplotlib 的預設 [8]。它在 CAM02-UCS 色貌空間中被最佳化為感知均勻，轉成灰階列印時仍然均勻，對常見的色覺異常也易於辨讀。Crameri、Shephard 與 Heron [9] 在各科學領域提出同樣的主張：彩虹色與紅綠色對應表會扭曲資料，並把約 8% 有色覺異常的男性排除在外。

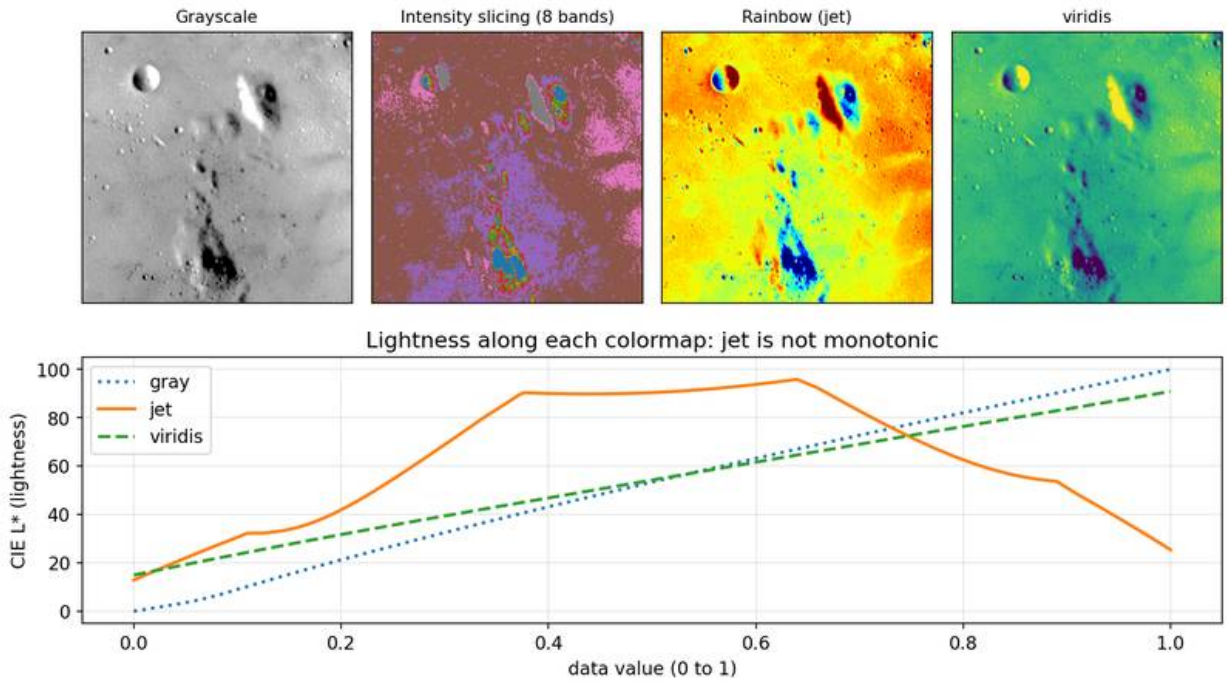


圖 *6.4* — 對比拉伸後的月面影像的偽彩色。上排：灰階、*8* 段強度切片、彩虹色 *jet*、*viridis*。下排：沿各色彩對應表的 *L** 明度。*viridis* 像灰階一樣穩定上升；*jet* 先升、持平、再下降，因而產生假邊緣。

```
import numpy as np
import matplotlib.pyplot as plt
from skimage import data, util, color

g = util.img_as_float(data.moon())
# intensity slicing into 8 equal bands
bands = np.digitize(g, np.linspace(0, 1, 9)[1:-1])          # 0..7
palette = plt.get_cmap("tab10")(np.arange(8))[:, :3]
sliced = palette[bands]                                      # (H, W, 3)
# continuous intensity-to-color transformation
rgb = plt.get_cmap("viridis")(g)[..., :3]
# check a colormap's lightness profile
L = color.rgb2lab(plt.get_cmap("jet")(np.linspace(0, 1, 256))[None, :, :3])[0, :, 0]
print(np.all(np.diff(L) >= 0))                               # False: jet is not monotonic
```

經驗法則：有順序的資料用**循序型**（sequential）均勻對應表（例如 viridis）；有意義的中點（零、門檻）時用**發散型**（diverging）；角度與色相用**循環型**（cyclic）；只有無序類別才用**定性型**（qualitative）調色盤，例如強度切片。

全彩影像處理基礎

白話版。處理彩色影像有兩種方式：當成三張獨立的灰階影像，或把每個像素看成色彩空間中的小箭頭，直接處理箭頭。有時兩者結果相同，但常常不同。

一個全彩像素就是向量

$$\mathbf{c}(x,y) = \begin{bmatrix} c_R(x,y) \\ c_G(x,y) \\ c_B(x,y) \end{bmatrix} = \begin{bmatrix} R(x,y) \\ G(x,y) \\ B(x,y) \end{bmatrix}.$$

處理它有兩種方式：

1. **逐通道**（per-channel，componentwise）。對每個通道各自套用灰階方法，再疊回去。
2. **向量處理**（vector processing）。直接對向量 $\mathbf{c}(x,y)$ 運算，使用同時考慮三個分量的操作，例如向量距離、向量中值或彩色梯度。

逐通道處理要和向量處理結果相同，需要兩個條件：運算同時適用於純量與向量，而且對每個分量的作用相同、不會混在一起。**線性**鄰域運算兩者都滿足。對每個通道在視窗內取平均，等於對向量在視窗內取平均：

$$\frac{1}{|S_{xy}|}\sum_{(s,t)\in S_{xy}} \mathbf{c}(s,t) = \begin{bmatrix} \frac{1}{|S_{xy}|}\sum R(s,t) \\ \frac{1}{|S_{xy}|}\sum G(s,t) \\ \frac{1}{|S_{xy}|}\sum B(s,t) \end{bmatrix},$$

其中 S_{xy} 是以 (x,y) 為中心的鄰域，$|S_{xy}|$ 是其像素數。**非線性**運算會打破這個等價關係。逐通道中值可能從第一個像素取紅、從第二個取綠、從第三個取藍，組出鄰域裡根本不存在的顏色。門檻、邊緣偵測與直方圖等化也都是需要向量觀點的例子，後面各節會說明。

色彩轉換

白話版。色彩轉換就是一個只看單一像素、改變它顏色的配方：讓它變暗、翻成相反色、調暖一點，或者只有「夠橘」時才保留它。

數學表述

和第 3 章一樣，點轉換寫成 $g(x,y) = T[f(x,y)]$，只是現在 f 與 g 都是色彩向量。逐分量寫出來是

$$s_i = T_i(r_1, r_2, \ldots, r_n), \qquad i = 1, \ldots, n,$$

其中 r_i、s_i 是第 i 個輸入與輸出分量，n 是分量數（RGB 與 HSI 為 $n = 3$，CMYK 為 $n = 4$）。每個輸出都可以依賴所有輸入。

任何效果都能在任何模型中寫出來，但成本不同。把強度乘上 k，在 HSI 中只需一次乘法（$s_3 = k\, r_3$），在 RGB 中要三次（$s_i = k\, r_i$），在 CMY 中則變成仿射映射（$s_i = k\, r_i + (1-k)$）。選擇讓運算

最自然的模型，但別忘了轉換本身要花時間，也會在暗部或灰色像素放大雜訊，因為那裡的色相與飽和度不穩定。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

補色

色環上位置相對的色相互為**補色**（complements）：紅與青、綠與洋紅、藍與黃。在 RGB 中，補色就是照相負片：

$$s_i = 1 - r_i, \qquad i = R, G, B,$$

也就是第 3 章灰階負片的彩色版。在 HSI 中，色相補色是 $H \mapsto (H + 180°) \bmod 360°$，但 RGB 負片同時反轉了強度，因此不能只靠色相算出來。補色有助於顯露暗部細節。

色彩切片

色彩切片（color slicing）是彩色版的強度切片：保留顏色接近某個**原型**（prototype）$\mathbf{a} = (a_R, a_G, a_B)$ 的像素，其餘推向中性色。「接近」常見的兩種定義：

$$\text{cube:}\quad s_i = \begin{cases} 0.5, & \text{if } |r_j - a_j| > W/2 \text{ for any } j \\ r_i, & \text{otherwise} \end{cases} \qquad \text{sphere:}\quad s_i = \begin{cases} 0.5, & \text{if } \|\mathbf{r} - \mathbf{a}\| > R_0 \\ r_i, & \text{otherwise} \end{cases}$$

W 是以 $\mathbf{a}$ 為中心的立方體寬度，R_0 是球的半徑，$\mathbf{r}$ 是輸入色彩向量，0.5 是中灰。球對各方向一視同仁；立方體計算較便宜，也方便逐通道推理。比較友善的變形是讓沒選中的像素保留自己的灰階值，而不是一片平灰，這樣仍看得到上下文。

```
import numpy as np
from skimage import data, util, color

img = util.img_as_float(data.astronaut())
a = np.array([0.85, 0.40, 0.20])            # prototype: the orange of the suit
d = np.linalg.norm(img - a, axis=-1)        # distance of every pixel to a
gray = color.gray2rgb(color.rgb2gray(img))
sliced = np.where((d <= 0.25)[..., None], img, gray)
print((d <= 0.25).mean())                   # fraction kept, about 0.165
```

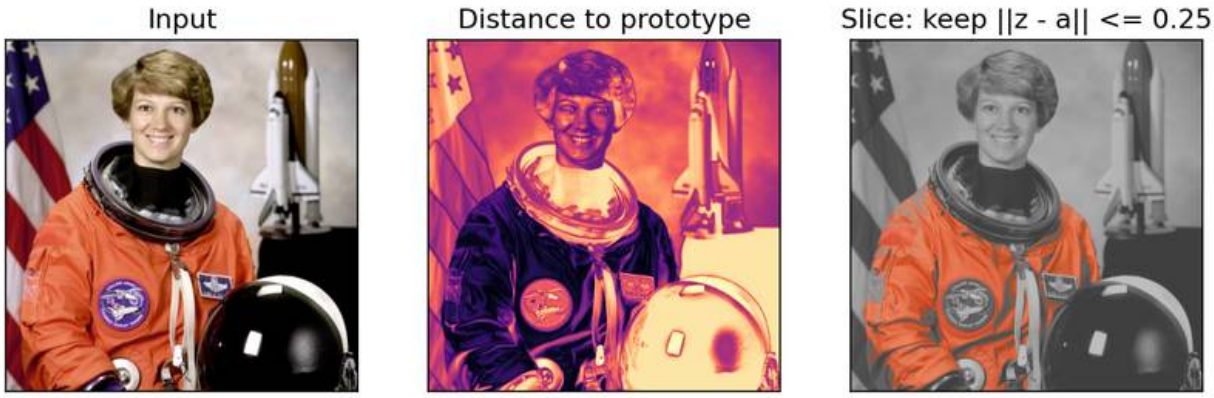


圖 *6.5* — 以太空衣的橘色為中心、半徑 *0.25* 的球做色彩切片。中：到原型的距離（越暗越近）。右：符合的像素保留顏色，其餘以灰階顯示。

色調與色彩校正

色調校正（tone correction）調整亮度與對比，最好只作用在類似強度的通道（I、V 或 L^*）上，以保留色相。偏亮（high-key）的影像需要拉開亮部的曲線，偏暗（low-key）的影像需要提亮暗部的曲線，平淡的影像則需要 S 形曲線。

色彩平衡（color balance）校正用來去除色偏：要減少某個顏色，就增加它的補色，或減少它在色環上的鄰居。青色太多？增加紅色，或同時減少藍與綠。場景中放一張灰卡就有精確的目標：校正後灰卡的 R、G、B 應該相等。

實務流程會透過校正過的色彩描述檔，把各裝置的顏色轉到與裝置無關的空間（例如 L*a*b*），讓顏色在螢幕與紙上看起來一致；超出某裝置色域（圖 6.1）的顏色必須往內映射。

白平衡

白平衡（white balance）是最常見的自動色彩校正。相機記錄的是表面顏色乘上光源顏色，所以同一面白牆在鎢絲燈下偏橘、在陰影中偏藍。我們的視覺系統大多能把光源的影響扣掉（**色彩恆常性**，color constancy），相機則必須用計算來做到。

標準模型是對角模型：光源顏色為 $\mathbf{e} = (e_R, e_G, e_B)$ 時，觀察到的像素為 $c_i = e_i \rho_i$，其中 ρ_i 是表面在白光下的反應。校正就是除以估計的光源，$\hat{\rho}_i = c_i / \hat{e}_i$（通常再縮放以維持整體亮度）。困難在於只靠一張影像估計 $\hat{\mathbf{e}}$。經典的估計法都對場景做了某種假設：

- **灰色世界**（gray world）[10]：場景的平均反射率是無彩色的，所以 $\hat{\mathbf{e}} \propto$ 影像的平均 RGB。
- **白色區塊**（white patch，max-RGB）[11], [14]：最亮的表面是白色，所以 $\hat{\mathbf{e}} \propto$ 各通道最大值（實務上用高百分位數，以忽略少數飽和像素）。
- **灰色邊緣**（gray edge）[11]：平均邊緣（影像導數）是無彩色的。van de Weijer、Gevers 與 Gijsenij 證明三者屬於同一個家族：

$$\left(\int \left| \frac{\partial^n \mathbf{c}_\sigma(\mathbf{x})}{\partial \mathbf{x}^n} \right|^p d\mathbf{x} \right)^{1/p} = k\,\hat{\mathbf{e}},$$

逐通道計算，其中 $\mathbf{c}_\sigma$ 是以尺度 σ 的高斯平滑過的影像，n 是導數階數，p 是 Minkowski 範數，k 是常數。$n=0, p=1$ 是灰色世界；$n=0, p=\infty$ 是 max-RGB；$n=1$ 是灰色邊緣。

```
import numpy as np
from skimage import data, util

def white_balance(img, method="gray-world", p=99.5):
    """Diagonal white balance. img: float RGB in [0, 1]."""
    px = img.reshape(-1, 3)
    e = px.mean(0) if method == "gray-world" else np.percentile(px, p, axis=0)
    gain = e.mean() / e                 # one gain per channel
    return np.clip(img * gain, 0, 1), e

img = util.img_as_float(data.astronaut()) * 0.8
biased = np.clip(img * [0.75, 0.95, 1.25], 0, 1)      # add a bluish cast
fixed, e = white_balance(biased, "white-patch")
print(e.round(3))                                      # [0.598 0.757 0.996]
```

拖動滑桿，比較偏藍的輸入與白色區塊法的校正結果。

偏藍

白色區塊法校正

加上人工藍色偏色的太空人影像，以及白色區塊法白平衡後的結果

這些假設都會在某些場景失敗。下方的貓照片真實平均色是橘褐色，灰色世界法把溫暖的毛色「校正」成灰，留下藍色調；白色區塊法則靠白色鬍鬚與高光表現好得多。每張結果上方標示的是估計光源與真實光源之間的角度誤差（angular error，這個領域的標準指標）。

圖 *6.6* — 在平均色不是灰色的影像上比較灰色世界法與白色區塊法。角度誤差來自我們自己的合成實驗（見圖表程式），並非基準測試結果。

彩色影像的直方圖處理

對 R、G、B 三個直方圖分別做等化，通常會產生錯誤的顏色，因為它改變了通道之間的比例，也就改變了色相。比較安全的做法是只等化強度通道、保留色度：

1. 轉到能分離亮度的模型（HSI、HSV 或 L*a*b*）。

2. 用第 3 章的方法等化 I（或 V、L^*）。
3. 轉回來。

```
import numpy as np
from skimage import data, util, exposure, color

img = util.img_as_float(data.astronaut())
per_channel = np.stack([exposure.equalize_hist(img[..., k]) for k in range(3)], -1)  #
 ↪  shifts hues
lab = color.rgb2lab(img)
lab[..., 0] = 100 * exposure.equalize_hist(lab[..., 0] / 100)                        #
 ↪  lightness only
better = np.clip(color.lab2rgb(lab), 0, 1)
```

色相保住了，但有些顏色可能看起來變淡，或在轉回時超出色域，因此有時需要裁切或稍微提高飽和度。區域直方圖方法（第 3 章）也能用同樣方式套用在 L^* 上。

彩色影像的平滑與銳化

白話版。要模糊或銳化彩色照片，可以對三個通道分別處理，因為這些是線性運算。不過只處理亮度，結果往往更乾淨。

平滑

彩色影像的鄰域平均為

$$\bar{\mathbf{c}}(x,y) = \frac{1}{K} \sum_{(s,t)\in S_{xy}} \mathbf{c}(s,t),$$

其中 S_{xy} 是鄰域，K 是鄰域大小。如前所述，這等於對每個通道分別平均，所以對 3 通道陣列呼叫 `cv2.GaussianBlur` 是正確的。

另一種做法是只平滑 HSI（或 L^*）影像的強度，保留色相與飽和度。兩者在邊緣附近不同：逐通道平滑會在紅綠交界產生中間色（一條橄欖色的線），只平滑強度則保留原本的色相、只柔化亮度。要選哪種取決於任務：去雜訊通常會對色度做比亮度更強的平滑，因為人眼對色度模糊較不敏感（這也正是 YCbCr 色度降取樣所利用的事實）。

銳化

向量的 Laplacian 就是各分量 Laplacian 組成的向量：

$$\nabla^2 \mathbf{c}(x,y) = \begin{bmatrix} \nabla^2 R(x,y) \\ \nabla^2 G(x,y) \\ \nabla^2 B(x,y) \end{bmatrix},$$

所以 Laplacian 銳化 $\mathbf{g} = \mathbf{c} - k\nabla^2\mathbf{c}$（強度 $k > 0$）可以逐通道進行。反銳化遮罩（unsharp masking，第 3 章）也一樣。實務上只銳化強度通道，可以避免高對比邊緣周圍出現彩色光暈（色邊），這種光暈來自三個通道過衝的程度不同。

在影像分割中使用色彩

白話版。色彩是把影像切成幾塊最容易的線索之一：「褐色的都是染色組織，藍色的都是背景」。問題在於怎麼量化「夠褐」。

在 HSI 空間中分割

如果目標是用色相描述的，HSI 是自然的選擇。典型流程：先對飽和度設門檻，做出「真的有顏色」的像素遮罩（低飽和度時色相沒有意義），把色相影像乘上遮罩，再對遮罩後的色相設門檻或看直方圖，挑出目標範圍。弱點在於色相的奇異點：接近灰色的像素中，雜訊會產生隨機色相，所以飽和度遮罩要先做。

在 RGB 向量空間中分割

RGB 向量常常效果更好，因為它避開了不穩定的色相計算。從目標中選一組樣本像素，算出平均色 $\mathbf{a}$，再依每個像素 $\mathbf{z}$ 到 $\mathbf{a}$ 的距離分類。**歐氏距離**為

$$D(\mathbf{z}, \mathbf{a}) = \|\mathbf{z} - \mathbf{a}\| = \left[(\mathbf{z} - \mathbf{a})^{\mathsf{T}}(\mathbf{z} - \mathbf{a})\right]^{1/2},$$

$D \le D_0$ 的像素標為目標，這個集合是半徑 D_0 的實心球。但樣本顏色通常在某些方向上分散得比較開（以染色組織來說，主要是由亮到暗的方向）。**Mahalanobis 距離**把這點納入考量：

$$D(\mathbf{z}, \mathbf{a}) = \left[(\mathbf{z} - \mathbf{a})^{\mathsf{T}}\mathbf{C}^{-1}(\mathbf{z} - \mathbf{a})\right]^{1/2},$$

其中 $\mathbf{C}$ 是樣本的 3×3 共變異數矩陣。此時 $D \le D_0$ 是順著資料方向的橢球。更便宜的近似是以 $\mathbf{a}$ 為中心的**外接盒**（bounding box），每個軸的半寬取該通道標準差的倍數；判斷是否落在盒內只需比較，不需平方或開根號。

```
import numpy as np
from skimage import data, util

img = util.img_as_float(data.immunohistochemistry())
samples = img[60:100, 60:100].reshape(-1, 3)        # a brown (stained) region
a, C = samples.mean(0), np.cov(samples.T)
z = img.reshape(-1, 3) - a
d_euc = np.sqrt((z ** 2).sum(1)).reshape(img.shape[:2])
d_mah = np.sqrt(np.einsum("ij,jk,ik->i", z, np.linalg.inv(C), z)).reshape(img.shape[:2])
mask_euc, mask_mah = d_euc <= 0.25, d_mah <= 3.0
```

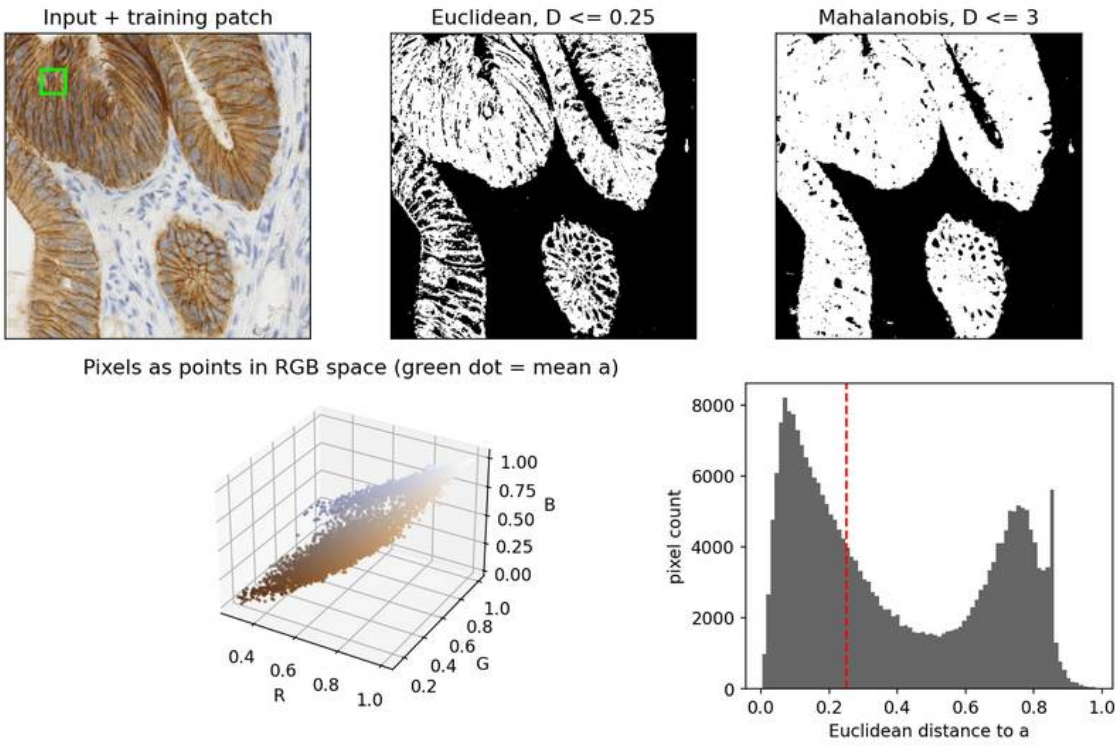


圖 *6.7* — 在 *RGB* 向量空間中分割。上排：染色組織樣本與訓練區塊（綠框）；半徑 *0.25* 的歐氏球；半徑 *3* 的 *Mahalanobis* 橢球。橢球順著拉長的褐色群集，把染色區域填得更完整。下排：像素形成從深褐到淡藍的細長雲團；距離直方圖呈雙峰，可用來選門檻。

彩色邊緣偵測

對每個通道各算梯度再相加是常見的捷徑，但那並不是向量影像的梯度。更糟的是，在灰階版本上做邊緣偵測，會漏掉**等亮度**（isoluminant）邊緣，也就是兩種亮度相同的顏色之間的邊界。Di Zenzo [13] 為多通道影像定義了正確的梯度。令 $\mathbf{u} = (\partial R/\partial x, \partial G/\partial x, \partial B/\partial x)$、$\mathbf{v} = (\partial R/\partial y, \partial G/\partial y, \partial B/\partial y)$，定義

$$g_{xx} = \mathbf{u}\cdot\mathbf{u}, \qquad g_{yy} = \mathbf{v}\cdot\mathbf{v}, \qquad g_{xy} = \mathbf{u}\cdot\mathbf{v}.$$

它們組成 2×2 結構張量 $\begin{bmatrix} g_{xx} & g_{xy} \\ g_{xy} & g_{yy} \end{bmatrix}$。變化最大的方向，以及該方向上的變化率為

$$\theta(x,y) = \frac{1}{2}\tan^{-1}\!\left(\frac{2g_{xy}}{g_{xx}-g_{yy}}\right), \qquad F_\theta(x,y) = \left\{\frac{1}{2}\Big[(g_{xx}+g_{yy}) + (g_{xx}-g_{yy})\cos 2\theta + 2g_{xy}\sin 2\theta\Big]\right\}^{1/2}.$$

θ 是梯度方向（用 `arctan2`，並同時檢查 $\theta+90°$，因為公式會給出兩個極值），F_θ 是邊緣強度。等價地，F_θ^2 是結構張量的最大特徵值 $\lambda_{\max} = \tfrac{1}{2}\big[g_{xx}+g_{yy}+\sqrt{(g_{xx}-g_{yy})^2+4g_{xy}^2}\big]$，數值上這樣算比較穩定。

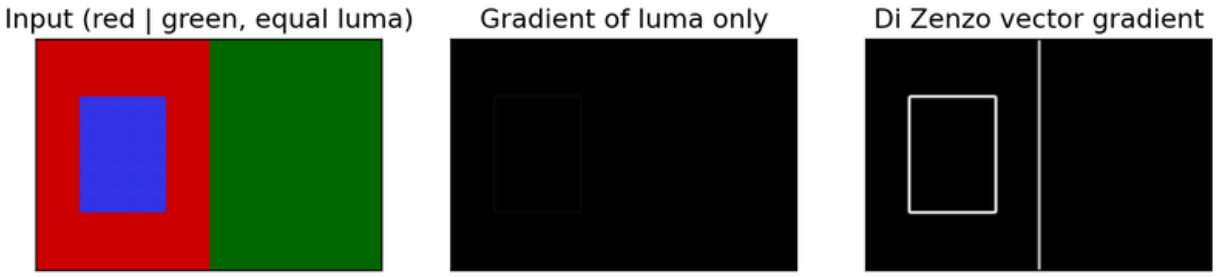


圖 *6.8* — 等亮度邊緣。紅綠兩半的 *BT.601* 亮度相同，所以亮度影像的梯度看不到兩者間的邊界（方塊也只隱約可見）。*Di Zenzo* 向量梯度兩者都找得到。

```
import numpy as np
from skimage import filters

def di_zenzo(img):
    """Edge strength of a float RGB image via the largest structure-tensor eigenvalue."""
    gx = np.stack([filters.sobel_v(img[..., k]) for k in range(3)], -1)   # d/dx
    gy = np.stack([filters.sobel_h(img[..., k]) for k in range(3)], -1)   # d/dy
    gxx, gyy, gxy = (gx * gx).sum(-1), (gy * gy).sum(-1), (gx * gy).sum(-1)
    return np.sqrt(0.5 * (gxx + gyy + np.sqrt((gxx - gyy) ** 2 + 4 * gxy ** 2)))
```

彩色影像中的雜訊

白話版。每個色彩通道都有自己的雜訊。把有雜訊的通道轉回色相與飽和度時，雜訊會混在一起，看起來可能糟得多，尤其是在暗部或灰色區域。

第 5 章的雜訊模型可套用到每個通道。相機中的雜訊主要產生於感測器，也就是在色彩重建之前。大多數感測器前面有一片彩色濾光陣列，每個感光點只量一種顏色，另外兩種靠內插補出來（**去馬賽克**，demosaicking）；內插會讓雜訊擴散並在通道之間產生相關。Lukac 等人 [12] 回顧了這整條處理鏈。

兩個觀察影響了實務做法：

- **色彩轉換會重新分配雜訊**。強度 $I = (R+G+B)/3$ 是三個通道的平均，所以每通道變異數為 σ^2 的獨立零均值雜訊，在 I 中變成 $\sigma^2/3$：強度比任何單一通道都乾淨。相反地，色相與飽和度是在灰色軸附近導數很大的非線性函數，因此會變得更雜。這就是為什麼去雜訊器常在亮度—色度空間中運作，並對色度做更強的平滑。
- **單一通道的脈衝雜訊是一個彩色斑點**。只出現在綠色通道的「鹽」雜訊會變成一個亮綠點。逐通道中值濾波器可以去除它，但在混合情況下可能產生鄰域裡沒有的顏色。

向量中值濾波器（vector median filter）避免了捏造顏色。在每個視窗中，它輸出與視窗內所有其他向量距離總和最小的那個輸入向量 [12]：

$$\mathbf{c}_{\mathrm{VM}} = \arg\min_{\mathbf{c}_i \in W} \sum_{\mathbf{c}_j \in W} \|\mathbf{c}_i - \mathbf{c}_j\|,$$

其中 W 是視窗內色彩向量的集合。輸出一定是輸入之一；對一維資料，它就退化成一般的中值。

```
import numpy as np
from skimage import data, util

def vector_median(img, k=3):
    r = k // 2
    H, W, _ = img.shape
    p = np.pad(img, ((r, r), (r, r), (0, 0)), mode="edge")
    nb = np.stack([p[i:i + H, j:j + W] for i in range(k) for j in range(k)], 2)  # (H, W,
     ↪  k*k, 3)
    D = np.linalg.norm(nb[:, :, :, None] - nb[:, :, None, :], axis=-1).sum(-1)    # (H, W,
     ↪  k*k)
    return np.take_along_axis(nb, D.argmin(-1)[..., None, None], 2)[:, :, 0]

img = util.img_as_float(data.astronaut())
rng = np.random.default_rng(0)
noisy = img.copy()
hit = rng.random(img.shape[:2]) < 0.05
noisy[hit] = rng.random((hit.sum(), 3))          # 5% random color impulses
out = vector_median(noisy)
psnr = lambda x: 10 * np.log10(1 / ((x - img) ** 2).mean())
print(round(psnr(noisy), 1), round(psnr(out), 1))  # about 20.4 -> 31.3 dB
```

這種向量化寫法每個像素使用的記憶體與 k^4 成正比；視窗大時請改用迴圈。

彩色影像壓縮

白話版。彩色影像的資料量是灰階的三倍，但其中很多是冗餘的：三個通道長得很像，而且人眼看不太到色彩細節。編碼器同時利用這兩點：

1. **去除通道間的相關性**。R、G、B 高度相關（圖 6.2 中三張影像非常相似）。轉成一個亮度加兩個色差，例如 YCbCr [6]，能把大部分能量集中在一個通道。
2. **色度降取樣**（chroma subsampling）。人眼對色彩細節的解析力不如亮度細節，所以色度通道可以只在水平方向用一半解析度（4:2:2），或在兩個方向都用一半（4:2:0）儲存，看起來幾乎沒有損失。光是 4:2:0 就讓原始資料減半：每個像素 $1 + 2 \cdot \frac{1}{4} = 1.5$ 個樣本，而不是 3 個。

之後每個通道再用第 8 章的轉換、量化與熵編碼方法壓縮。常見的色彩瑕疵有：銳利邊緣處的滲色（來自色度降取樣），以及高壓縮率下的塊狀色斑。

現代觀點

上面的經典工具至今仍在每一台相機與每一個影像函式庫裡運作。大約 2010 年後改變的，主要是參數怎麼決定：從固定的假設，變成從資料中學習。以下五篇回顧與綜述性質的論文描繪了這個領域的版圖。

色彩恆常性：從假設到學習。 Gijsenij、Gevers 與 van de Weijer [14] 綜述了計算色彩恆常性，把方法分成三類：採用固定假設的靜態方法（灰色世界、白色區塊、灰色邊緣及上面的 Minkowski 家族）、把觀察到的顏色與標準光源下的顏色集合比較的色域式方法，以及用已知光源影像訓練的學習式方法。他們也提出比較方法的準則，並在公開資料集上評測公開演算法；反覆出現的結論是，沒有任何單一固定假設適用所有場景。之後學習式方法成為主流。Fast Fourier Color Constancy（FFCC）[15] 把光源估計轉成在 log-chroma 直方圖中找峰值，並在頻域中高效求解；它輸出光源的完整機率分布，能在手機上即時執行，讓視訊中的自動白平衡在時間上平順。FC4 [16] 是全卷積網路，每個影像區塊都以學到的信心度替光源投票，所以一面素牆的權重比人臉或白色物體低。Afifi 與 Brown [17] 則在相機已經把影像渲染成非線性 sRGB 之後才校正白平衡，這是對角模型做不到的：他們的網路把影像重新渲染成其他白平衡設定下的樣子。對角模型仍是骨幹，但光源估計與在渲染後空間中的校正，現在都是學出來的。

彩色向量濾波。 Lukac 等人 [12] 回顧了把彩色像素當成向量的濾波器：向量中值與向量方向濾波器及其變形、向量邊緣偵測器，以及去馬賽克。他們的結論與本章主題一致：逐分量處理忽略了通道間的相關性，可能產生色彩瑕疵，而向量順序統計量能保留輸入的顏色。在有訓練資料與運算資源時，學習式去雜訊器如今更強，但這些濾波器在講求速度與可預測性的場合仍是標準基準。

色彩對應表設計。 Kovesi [7] 為線性、發散、彩虹與循環色彩對應表提出設計方法與測試影像，核心原則只有一個：感知明度必須沿著對應表均勻變化。viridis 系列 [8] 在 Matplotlib 2.0 成為預設。Crameri 等人 [9] 檢視了科學出版中色彩的使用方式，主張非均勻與紅綠色對應表應從軟體預設中消失。深度學習在這裡沒有改變任何事：問題在於人類知覺，答案是在感知色彩空間中量測的細心設計。

以深度學習上色。偽彩色用固定規則把灰階對應到顏色；上色（colorization）則試圖預測真實的顏色。Zhang、Isola 與 Efros [18] 把上色視為對量化色彩的分類問題，並對稀有類別做再平衡，避免鮮豔的顏色被平均成褐色；他們以「上色圖靈測試」評估，請受試者判斷影像是否為真。Anwar 等人 [19] 綜述深度上色方法，將其分為七類，討論損失函數與評估指標，並提出新的基準資料集。DDColor [20]（ICCV 2023）使用兩個解碼器，一個恢復空間細節，一個透過與多尺度語意特徵的交叉注意力精修色彩，再加上鼓勵色彩豐富度的損失；這減少了顏色跨越物體邊界的滲色。兩個經典想法始終存在：在 L*a*b* 中運作（預測色度、保留給定的明度），以及把顏色當成一個分布，因為一件灰色襯衫原本可能是任何顏色。

沒有改變的部分。色度學（CIE XYZ、色度、色域）、編碼（sRGB、BT.601/709 YCbCr）與色差空間是標準，不是要學的東西。網路用 sRGB 資料訓練，也在這些空間中被評估。清楚知道你的張量處於哪個空間（編碼或線性、BT.601 或 BT.709、studio 或 full range），仍是色彩工作中最實用的技能之一。

重點整理

- 人類視覺是三色視覺，所以三個數字就能為人類觀察者描述顏色；反應相同的不同光譜（同色異譜）看起來一樣。
- CIE xy 色度圖呈現所有可見色度；三原色裝置只能涵蓋其中一個三角形（色域）。
- RGB 適合硬體，CMY(K) 適合墨水，HSI 把強度與色相、飽和度分開，L*a*b* 讓歐氏距離大致符合感知。注意灰色像素的色相沒有定義、sRGB gamma，以及 BT.601 與 BT.709 的差異。
- 偽彩色讓灰階資料更好讀；請用 viridis 這類感知均勻的色彩對應表，而非彩虹色。

- 線性運算（平滑、Laplacian 銳化）可以逐通道做；非線性運算（中值、直方圖等化、邊緣偵測、分割）應把像素當向量處理，或只處理強度通道。
- 白平衡就是除以估計的光源；灰色世界、白色區塊與灰色邊緣是同一個框架的特例，現代系統則用學習來估計。
- 用向量距離（歐氏、Mahalanobis、外接盒）分割色彩，用能看見等亮度邊界的 Di Zenzo 梯度偵測彩色邊緣。
- 強度比任何單一通道雜訊更少，色相與飽和度則更雜；向量中值濾波器能去除彩色脈衝雜訊而不捏造新顏色。

練習

1. 某像素 $(R, G, B) = (0.6, 0.6, 0.2)$。請手算它的 H、S、I。接著不計算，預測 B 增加到 0.3 時 H 會怎麼變。

   提示

   $I = 0.4667$。$\min = 0.2$，所以 $S = 1 - 0.2/0.4667 \approx 0.571$。計算 θ：分子 $= 0.5(0 + 0.4) = 0.2$，分母 $= \sqrt{0 + 0.4 \cdot 0.4} = 0.4$，所以 $\theta = \cos^{-1}(0.5) = 60°$；因為 $B \le G$，$H = 60°$（黃色）。B 增加到 0.3 時仍有 $R = G$，所以色相維持 $60°$，只有 S 下降。

2. 證明 RGB 負片 $s_i = 1 - r_i$ 會把 HSI 座標為 (H, S, I) 的像素，映射到強度為 $1 - I$、色相為 $H + 180°$ 的像素（非灰色像素）。飽和度會保持不變嗎？

   提示

   強度由線性性質直接得到。對色相而言，色彩分量 $\mathbf{c} - I\,\mathbf{1}$ 只是變號，相當於繞灰色軸旋轉 $180°$。飽和度是 $1 - \min / I$；取負片後最小值變成 $1 - \max(R, G, B)$，強度變成 $1 - I$，所以飽和度一般會改變。試試 $(0.9, 0.1, 0.1)$。

3. 你對一張藍天照片的 R、G、B 直方圖分別做等化。解釋為什麼天空可能變成灰青色，並提出兩種修正方式。

   提示

   等化會把每個通道拉滿整個範圍。原本幾乎一致偏高的通道（藍）相對其他通道會被壓低，而弱的通道被拉高，所以 R:G:B 比例改變。修正方式：只等化 I、V 或 L^*；或用平均直方圖算出一個共同的映射，套用到三個通道。

4. 在灰色世界／白色區塊的框架中，$n = 0$、$p = 2$ 會得到什麼估計法？把 `white_balance` 的想法用在貓的影像上，取 $p = 1, 2, 6, \infty$（∞ 用高百分位數），畫出角度誤差對 p 的關係。

   提示

   $p = 2$ 用每個通道的均方根值估計光源，介於平均值與最大值之間。以本章的合成偏色而言，誤差從約 $12°$（$p = 1$）降到約 $11°$（$p = 2$）、$7°$（$p = 6$）與 $4°$（$p = 20$）。在這張影像上誤差隨 p 增大而下降，因為明亮的白色細節比橘色的平均更能透露光源。在沒有白色物體的場景中，趨勢可能相反。

5. 設計一個在戶外照片中分割綠色植物、而且不受陰影影響的色彩分割器。你會用哪個空間與哪種距離？為什麼單純的 RGB 歐氏距離可能失敗？

   提示

   陰影會把三個通道等比例縮小，使像素沿著一條通過原點的直線移動，RGB 歐氏距離因此大幅改變。改用不受縮放影響的色度特徵（正規化的 $r = R/(R+G+B)$、$g = G/(R+G+B)$，或加上飽和度遮罩的色相，或 a^*, b^*），或用以陽光下與陰影中樣本訓練出共變異數的 Mahalanobis 距離。

6. 解釋為什麼向量中值濾波器永遠不會產生新顏色，並舉一個 3 個像素的一維例子，說明逐通道中值會產生新顏色。

   提示

   依定義，向量中值是視窗內的輸入向量之一。對三個像素 $(1,1,0)$、$(1,0,1)$、$(0,1,1)$（黃、洋紅、青），逐通道中值是 $(1,1,1)$，純白色，而視窗中根本沒有白色。

本章所有圖片都由 `scripts/figures/dip_ch06.py` 從 `skimage.data` 影像與合成陣列產生。

參考文獻

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 6. publisher page
2. C. Wyman, P.-P. Sloan, and P. Shirley, "Simple Analytic Approximations to the CIE XYZ Color Matching Functions," *Journal of Computer Graphics Techniques*, vol. 2, no. 2, 2013. jcgt.org
3. IEC 61966-2-1:1999, "Multimedia systems and equipment – Colour measurement and management – Part 2-1: Colour management – Default RGB colour space – sRGB," IEC, 1999. IEC webstore
4. Recommendation ITU-R BT.709-6, "Parameter values for the HDTV standards for production and international programme exchange," ITU, 2015. ITU
5. ISO/CIE 11664-4:2019, "Colorimetry – Part 4: CIE 1976 L*a*b* colour space," ISO/CIE, 2019. standard page
6. Recommendation ITU-R BT.601-7, "Studio encoding parameters of digital television for standard 4:3 and wide screen 16:9 aspect ratios," ITU, 2011. ITU
7. P. Kovesi, "Good Colour Maps: How to Design Them," arXiv:1509.03700, 2015. arXiv
8. S. van der Walt and N. Smith, viridis colormap design notes (BIDS project page; presented at SciPy 2015), 2015. project page
9. F. Crameri, G. E. Shephard, and P. J. Heron, "The misuse of colour in science communication," *Nature Communications*, vol. 11, art. 5444, 2020. PMC
10. G. Buchsbaum, "A spatial processor model for object colour perception," *Journal of the Franklin Institute*, vol. 310, no. 1, pp. 1–26, 1980. DOI
11. J. van de Weijer, T. Gevers, and A. Gijsenij, "Edge-Based Color Constancy," *IEEE Transactions on Image Processing*, vol. 16, no. 9, pp. 2207–2214, 2007. DOI
12. R. Lukac, B. Smolka, K. Martin, K. N. Plataniotis, and A. N. Venetsanopoulos, "Vector Filtering for Color Imaging," *IEEE Signal Processing Magazine*, vol. 22, no. 1, pp. 74–86, Jan. 2005. PDF

13. S. Di Zenzo, "A note on the gradient of a multi-image," *Computer Vision, Graphics, and Image Processing*, vol. 33, pp. 116–125, 1986. DOI
14. A. Gijsenij, T. Gevers, and J. van de Weijer, "Computational Color Constancy: Survey and Experiments," *IEEE Transactions on Image Processing*, vol. 20, no. 9, pp. 2475–2489, 2011. DOI
15. J. T. Barron and Y.-T. Tsai, "Fast Fourier Color Constancy," *CVPR*, 2017. arXiv
16. Y. Hu, B. Wang, and S. Lin, "FC4: Fully Convolutional Color Constancy with Confidence-Weighted Pooling," *CVPR*, 2017. paper
17. M. Afifi and M. S. Brown, "Deep White-Balance Editing," *CVPR*, 2020. arXiv
18. R. Zhang, P. Isola, and A. A. Efros, "Colorful Image Colorization," arXiv:1603.08511, 2016. arXiv
19. S. Anwar, M. Tahir, C. Li, A. Mian, F. S. Khan, and A. W. Muzaffar, "Image Colorization: A Survey and Dataset," arXiv:2008.10774, 2020. arXiv
20. X. Kang, T. Yang, W. Ouyang, P. Ren, L. Li, and X. Xie, "DDColor: Towards Photo-Realistic Image Colorization via Dual Decoders," *ICCV*, 2023. arXiv

IP 07・小波與其他影像轉換

把每一種影像轉換都看成「換一組基底」，從 DCT、沃爾什–哈達瑪轉換一路走到哈爾轉換、多解析度分析與快速小波轉換。

網站版本: https://blog.shyandram.dev/zh-tw/learn/image-processing/dip-07-wavelets-transforms/

你將學到

- 為什麼所有線性影像轉換（傅立葉、餘弦、沃爾什–哈達瑪、哈爾、小波）其實是同一件事：用另一組積木來描述影像，這組積木叫做**基底**（basis）。
- 如何把轉換寫成兩次矩陣乘法，以及為什麼可分離（separable）的二維轉換很省計算。
- DCT、沃爾什–哈達瑪、斜轉換（slant）與哈爾轉換的基底影像長什麼樣子，以及 DCT 為何是 JPEG 的主力。
- 如何用時頻平面（time–frequency plane）理解傅立葉分析與小波分析的差別。
- 多解析度分析、尺度函數與小波函數如何導出快速小波轉換，也就是一個由兩個濾波器組成的濾波器組。
- 如何用 PyWavelets 做二維離散小波轉換，並應用在去雜訊、邊緣偵測與壓縮。

先看全貌

第 4 章把影像拆成許多正弦波的總和。這只是其中一種積木，並不是唯一的選擇。本章退一步問一個更一般的問題：給定任意一組積木，我要怎麼算出影像裡每塊積木各有多少？又該挑哪一組積木最適合手上的工作？

> **用最簡單的話說**
>
> 想像你在電話裡向朋友描述一個樂高模型。你可以一顆一顆凸點地講：「第 1 排第 1 格紅色、第 1 排第 2 格紅色……」。這一定行得通，但很慢。你也可以說：「這裡放一塊 2×4 的紅色積木，那裡放一塊 1×2 的藍色積木。」描述一下子變短了，因為積木的形狀剛好符合模型的形狀。
>
> 影像轉換就是在挑積木。像素網格用的是最小的 1×1 積木；傅立葉和餘弦轉換用的是條紋狀的積木；小波則準備了各種尺寸：大而模糊的拿來拼天空，小而銳利的拿來拼邊緣。積木越貼合畫面，需要的塊數就越少，壓縮、去雜訊和快速分析都是靠這一點。

「挑積木」的嚴謹說法是線性代數：內積、基底與基底轉換。這套工具一旦建立，本章的每一種轉換都只是一個不同的矩陣。小波會占最多篇幅，因為它是第一種在空間與頻率兩邊同時具有局部性的基底。這個特性讓小波成為 JPEG 2000 標準的核心 [15]，也是現代稀疏表示，以及近年部分神經網路層背後的重要想法。

預備知識

白話版：一個有 N 個取樣的訊號，就是 N 維空間中的一支箭頭（向量）。轉換就是量這支箭頭在 N 支參考箭頭上各投下多長的影子。如果參考箭頭兩兩垂直且長度為 1，影子就包含全部資訊，把影子加回去就能還原原本的箭頭。

內積空間

內積（inner product）$\langle \mathbf{f}, \mathbf{g} \rangle$ 接收兩個向量、回傳一個數，用來衡量兩者指向同一方向的程度。對長度為 N 的實數或複數向量：

$$\langle \mathbf{f}, \mathbf{g} \rangle = \sum_{x=0}^{N-1} f(x)\, g^{*}(x), \qquad \|\mathbf{f}\| = \sqrt{\langle \mathbf{f}, \mathbf{f} \rangle}.$$

其中 $f(x)$、$g(x)$ 是 $\mathbf{f}$、$\mathbf{g}$ 的第 x 個元素，g^{*} 是複數共軛（對實數資料沒有作用），$\|\mathbf{f}\|$ 是 $\mathbf{f}$ 的長度，也就是**範數**（norm）。對連續函數，總和改成積分：$\langle f, g \rangle = \int f(x) g^{*}(x)\, dx$。當兩個向量的內積為零時稱為**正交**（orthogonal）：它們彼此毫無共同成分。

正交歸一基底

若一組向量 $\{\mathbf{s}_0, \ldots, \mathbf{s}_{N-1}\}$ 每一個長度都是 1，而且兩兩正交，即 $\langle \mathbf{s}_u, \mathbf{s}_v \rangle = \delta(u - v)$（$u = v$ 時 δ 為 1，否則為 0），就稱為**正交歸一基底**（orthonormal basis）。此時任何向量都能乾淨地拆開：

$$t(u) = \langle \mathbf{f}, \mathbf{s}_u \rangle, \qquad \mathbf{f} = \sum_{u=0}^{N-1} t(u)\, \mathbf{s}_u.$$

$t(u)$ 稱為**轉換係數**。第一式是**分析**（正向）步驟，第二式是**合成**（反向）步驟。正交歸一基底還會保持能量：$\sum_x |f(x)|^2 = \sum_u |t(u)|^2$（帕塞瓦定理，Parseval's theorem），所以我們才能談「每個係數裝了多少能量」。

雙正交基底

有時最好用的積木並不互相垂直。這時需要兩組向量：一組 $\{\tilde{\mathbf{s}}_u\}$ 負責量測（分析），一組 $\{\mathbf{s}_u\}$ 負責重建（合成）。若滿足

$$\langle \mathbf{s}_u, \tilde{\mathbf{s}}_v \rangle = \delta(u - v), \qquad t(u) = \langle \mathbf{f}, \tilde{\mathbf{s}}_u \rangle, \qquad \mathbf{f} = \sum_u t(u)\, \mathbf{s}_u,$$

就稱為**雙正交**（biorthogonal）基底對，量測用的 $\tilde{\mathbf{s}}_u$ 稱為**對偶**（dual）基底。雙正交換來的是設計自由度：例如 JPEG 2000 採用雙正交小波濾波器，可以同時做到長度短且左右對稱，這個組合用正交小波很難達成 [15]。

框架

框架（frame）是一組向量數量多於維度的生成集合，所以帶有冗餘。它的定義性質是：量到的能量被固定的上下界夾住，

$$A\,\|\mathbf{f}\|^2 \;\le\; \sum_{k} |\langle \mathbf{f}, \mathbf{s}_k\rangle|^2 \;\le\; B\,\|\mathbf{f}\|^2, \qquad 0 < A \le B < \infty.$$

A、B 稱為**框架界**（frame bounds）。若 $A = B$，則稱為**緊框架**（tight frame），重建公式就是 $\mathbf{f} = \frac{1}{A}\sum_k \langle \mathbf{f}, \mathbf{s}_k\rangle \mathbf{s}_k$。一個經典的例子：平面上三支夾角 120° 的單位箭頭。它們能張成整個平面，任兩支都不正交，卻構成 $A = B = 3/2$ 的緊框架。當你需要穩健性或平移不變性時，多花一些係數換冗餘是值得的，例如不抽樣小波轉換（undecimated wavelet transform），以及「現代觀點」會提到的散射轉換。

```
import numpy as np

v = np.array([3.0, 1.0, 2.0])
e = np.eye(3)                          # standard orthonormal basis
c = e @ v                              # coefficients = inner products <v, e_k>
print(c, np.allclose(c @ e, v))        # synthesis: sum_k c_k e_k

# A biorthogonal pair in 2-D: analysis rows A, synthesis columns B = A^{-1}
A = np.array([[1.0, 0.0], [1.0, 1.0]])
B = np.linalg.inv(A)
x = np.array([2.0, 5.0])
print(A @ x, B @ (A @ x))              # [2. 7.] -> back to [2. 5.]
```

以矩陣表示的轉換

白話版：把量測用的向量疊成矩陣的每一列。訊號乘上這個矩陣，就一次量完所有係數；再乘上反矩陣，就把訊號拼回來。

對長度為 N 的一維訊號 $\mathbf{f}$，把基底向量 $\mathbf{s}_u^{*T}$ 依序放進 $N \times N$ **轉換矩陣** $\mathbf{A}$ 的各列，則

$$\mathbf{t} = \mathbf{A}\,\mathbf{f}, \qquad \mathbf{f} = \mathbf{A}^{-1}\mathbf{t}.$$

若基底是正交歸一的，$\mathbf{A}$ 就是**么正矩陣**（unitary，$\mathbf{A}^{-1} = \mathbf{A}^{*T}$；實數情形稱為**正交矩陣**，$\mathbf{A}^{-1} = \mathbf{A}^{T}$），反轉換幾乎不用成本。

對 $M \times N$ 的影像 $\mathbf{F}$，一般的二維線性轉換需要 $MN \times MN$ 的巨大矩陣。實務上幾乎所有影像轉換都是**可分離的**：先對每一行做一維轉換，再對每一列做一維轉換。寫成矩陣：

$$\mathbf{T} = \mathbf{A}\,\mathbf{F}\,\mathbf{B}^{T}, \qquad \mathbf{F} = \mathbf{A}^{-1}\,\mathbf{T}\,(\mathbf{B}^{T})^{-1},$$

其中 $\mathbf{A}$（$M \times M$）作用在行上，$\mathbf{B}$（$N \times N$）作用在列上；方形影像且兩方向使用同一種轉換時 $\mathbf{B} = \mathbf{A}$。可分離性把直接計算 $N \times N$ 轉換的成本從 $O(N^4)$ 降到 $O(N^3)$，而快速演算法（FFT、快速 DCT、快速小波轉換）還能再進一步降低。

```
import numpy as np
from scipy.fft import dct, dctn
from skimage import data, img_as_float

N = 8
A = dct(np.eye(N), norm="ortho", axis=0)  # row u = u-th cosine basis vector

print(np.allclose(A @ A.T, np.eye(N)))    # orthonormal -> True

F = img_as_float(data.camera())[200:208, 200:208]
T = A @ F @ A.T                           # forward 2-D transform
print(np.allclose(T, dctn(F, norm="ortho")))   # same as scipy -> True
print(np.allclose(A.T @ T @ A, F))             # inverse -> True
```

相關運算

白話版：每個係數都在回答一個問題：「我的影像有多像這個圖樣？」這個問題本身就是相關（correlation）運算。

把分析公式和第 3 章的互相關比一比。係數

$$t(u) = \langle \mathbf{f}, \mathbf{s}_u \rangle = \sum_x f(x)\, s_u^*(x)$$

正是 f 與基底函數 s_u 在位移為零時的相關值。$|t(u)|$ 大代表 f 和 s_u 很像；$t(u) = 0$ 代表 f 完全不含 s_u 的成分。由此得到兩個結論：

- **轉換就是樣板比對**。每個基底函數都是一個樣板，轉換一次替所有樣板打分數。傅立葉轉換替正弦波打分數，哈爾轉換替階梯狀圖樣打分數。
- **小波是在許多位移與尺度上做相關**。小波基底包含同一個圖樣的各種平移與伸縮版本。和某一尺度的所有平移版本做相關，就是一次濾波，所以本章後面的小波轉換會變成一組濾波器。

時頻平面上的基底函數

白話版：有些積木能告訴你事情何時發生，有些能告訴你它的音高，但沒有一塊能把兩者都說得完美。時頻平面就是描繪這種取捨的地圖。

對一個能量為 1 的基底函數 $s(x)$，用 $|s(x)|^2$ 與 $|S(\omega)|^2$ 的標準差定義它在時間（空間）與頻率上的展開程度：

$$\sigma_x^2 = \int (x - \bar{x})^2 |s(x)|^2\, dx, \qquad \sigma_\omega^2 = \frac{1}{2\pi} \int (\omega - \bar{\omega})^2 |S(\omega)|^2\, d\omega,$$

其中 $S(\omega)$ 是 s 的傅立葉轉換，$\bar{x}$、$\bar{\omega}$ 分別是兩個能量分布的質心。訊號版的**海森堡測不準原理**（Heisenberg uncertainty principle）指出

$$\sigma_x\, \sigma_\omega \;\geq\; \tfrac{1}{2},$$

只有高斯形狀的函數能取到等號。把每個基底函數畫成一個以 $(\bar{x}, \bar{\omega})$ 為中心、寬 σ_x、高 σ_ω 的矩形，稱為**海森堡盒**（Heisenberg box）或時頻磚。磚的面積不能小於某個下限，但形狀可以自由選擇。

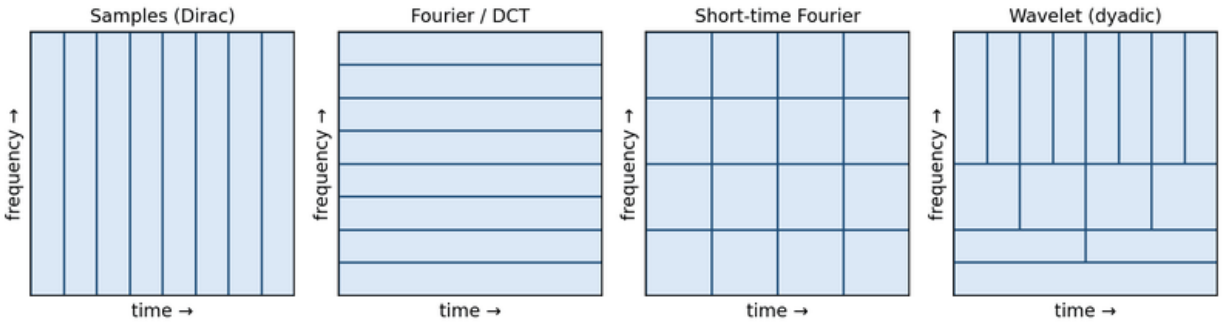


圖 *7.1* — 四種基底如何鋪滿時頻平面，每塊磚的面積都相同。取樣在時間上最精確；傅立葉／*DCT* 基底在頻率上最精確；短時傅立葉轉換對所有頻率採用同一種折衷；小波則依頻率調整磚的形狀。

圖 7.1 呈現四種標準情形：

- **取樣（單位基底）：**位置完全精確，沒有任何頻率資訊。
- **傅立葉或 DCT：**頻率完全精確，沒有位置資訊。一條銳利的邊緣會擴散到所有係數上。
- **短時傅立葉轉換**（short-time Fourier transform, STFT）：把訊號切成一個個窗，各自做傅立葉轉換。每塊磚形狀相同，一次決定就套用到所有頻率 [2]。
- **小波：**高頻的磚在時間上窄、頻率上高；低頻的磚則寬而矮。這很符合自然影像：邊緣是短暫的高頻事件，平緩的明暗變化則是緩慢的低頻成分。

基底影像

白話版：二維轉換的每個係數都對應一張小圖。影像就是這些小圖的加權總和，係數就是權重。

對矩陣 $\mathbf{A}$（第 u 列為 $\mathbf{a}_u^T$）定義的可分離轉換，可以把反轉換改寫成外積的總和：

$$\mathbf{F} = \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} T(u,v)\, \mathbf{S}_{u,v}, \qquad \mathbf{S}_{u,v} = \mathbf{a}_u^* \mathbf{a}_v^{*T}.$$

每個 $N \times N$ 矩陣 $\mathbf{S}_{u,v}$ 就是一張**基底影像**（basis image）。$T(u,v)$ 告訴你要加入多少第 (u,v) 張基底影像；u 對應垂直方向的圖樣，v 對應水平方向。把 N^2 張基底影像排在一起，是理解一種轉換「看到什麼」最快的方法。

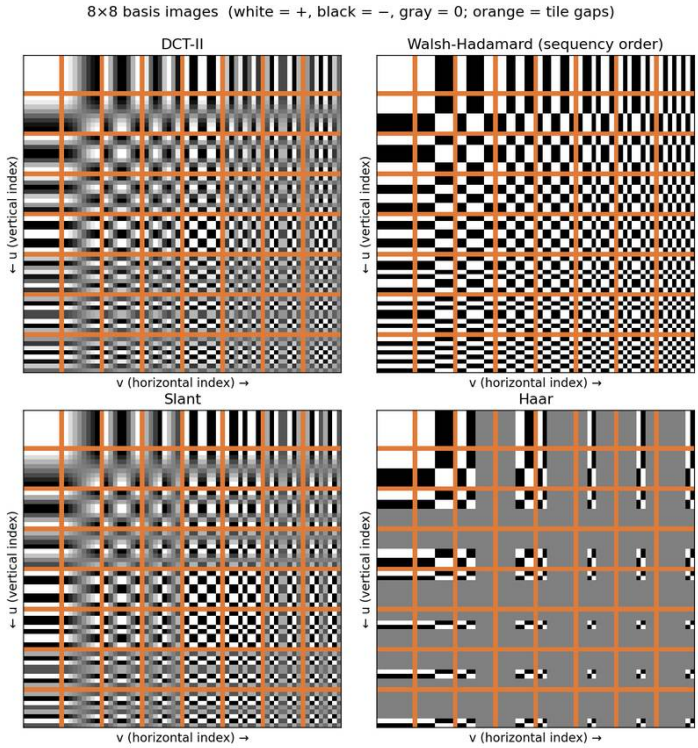


圖 7.2 — 四種轉換的 *8×8* 基底影像，向下代表垂直頻率（序率）增加，向右代表水平頻率增加；每格縮放到 $[-1, 1]$。注意哈爾基底影像和其他三種不同，大部分區域為零（灰色）：它們是局部的。

四種轉換左上角的基底影像都是常數，它量的是平均亮度，稱為 **DC** 係數。越往右、往下，圖樣振盪越快。DCT 平滑地振盪，沃爾什–哈達瑪在 ± 1 之間突然切換，斜轉換多了線性斜坡，哈爾則把每個圖樣限制在格子中的一小塊區域。

與傅立葉相關的轉換

白話版：傅立葉家族用波當積木。其中餘弦轉換在區塊邊界的表現最好，所以你手機裡的 JPEG 檔用的就是它。

以矩陣看離散傅立葉轉換

第 4 章的一維 DFT 是一個矩陣轉換：

$$A_{u,x} = \frac{1}{\sqrt{N}} e^{-j2\pi ux/N}, \qquad u, x = 0, \dots, N-1.$$

$A_{u,x}$ 是第 u 列第 x 行的元素，$j = \sqrt{-1}$。乘上 $1/\sqrt{N}$ 後矩陣是么正的。對壓縮來說它有兩個缺點：係數是複數，而且 DFT 默默把訊號當成週期性的。若區塊左右邊界的值不同，週期延伸就會出現跳躍，而跳躍需要大量高頻係數來表示。

離散餘弦轉換

（第二型）**離散餘弦轉換**（discrete cosine transform, DCT）[3] 使用實數餘弦：

$$t(u) = \alpha(u) \sum_{x=0}^{N-1} f(x) \cos\left[\frac{(2x+1)\,u\,\pi}{2N}\right], \qquad \alpha(0) = \sqrt{\tfrac{1}{N}},\ \alpha(u) = \sqrt{\tfrac{2}{N}}\ (u \geq 1).$$

u 是頻率索引，$\alpha(u)$ 讓基底正交歸一。DCT 等於（差一個縮放）把訊號鏡射成長度 $2N$ 的偶對稱延伸後再做 DFT。鏡射消除了邊界跳躍，能量因此集中在少數低頻係數。這種**能量集中**（energy compaction）特性，正是 JPEG 標準對 8×8 影像區塊做 DCT 的原因 [4]。Ahmed、Natarajan 與 Rao 在 1974 年提出 DCT，並指出它在維納濾波與率失真（rate–distortion）準則下的表現，與統計上最佳的 Karhunen–Loève 轉換非常接近 [3]。

離散正弦轉換

離散正弦轉換（discrete sine transform, DST）改用正弦。第一型 DST 的矩陣為

$$A_{u,x} = \sqrt{\frac{2}{N+1}}\, \sin\left[\frac{(x+1)(u+1)\pi}{N+1}\right],$$

對應的是訊號的奇對稱延伸。它是實數、正交歸一且對稱的矩陣（$\mathbf{A} = \mathbf{A}^T = \mathbf{A}^{-1}$），適合兩端接近零的訊號，例如預測殘差。在 SciPy 中可寫成 `scipy.fft.dst(x, type=1, norm="ortho")`。

沃爾什–哈達瑪轉換

白話版：每個基底向量都只由 $+1$ 和 -1 組成。這樣轉換完全不需要乘法，只要加減。

$N = 2^n$ 階的哈達瑪矩陣以遞迴方式建構：

$$\mathbf{H}_1 = [\,1\,], \qquad \mathbf{H}_{2N} = \begin{bmatrix} \mathbf{H}_N & \mathbf{H}_N \\ \mathbf{H}_N & -\mathbf{H}_N \end{bmatrix}, \qquad \mathbf{A}_{\text{WHT}} = \frac{1}{\sqrt{N}}\,\mathbf{H}_N.$$

$\mathbf{H}_N$ 的每個元素都是 ± 1，各列互相正交，因此 $\mathbf{A}_{\text{WHT}}$ 是正交歸一（而且對稱）的。$\mathbf{H}_N$ 的列天生是**自然順序**（哈達瑪順序）。做分析時，改依**序率**（sequency，一列中正負號改變的次數，扮演頻率的角色）排序會更好讀。依序率排列的列稱為沃爾什函數，圖 7.2 就是它們的二維基底影像。

沃爾什–哈達瑪轉換（Walsh–Hadamard transform, WHT）因為計算便宜，很早就被用於影像編碼 [5]。對自然影像而言，它的能量集中能力不如 DCT，因為方波基底不擅長逼近平滑的明暗變化（見後面的圖 7.7）。在講求速度或硬體簡單的場合，以及作為快速隨機化演算法的元件時，它仍然很有用。

```
import numpy as np
from scipy.linalg import hadamard

H = hadamard(8)                                  # natural (Hadamard) order
changes = (np.diff(np.sign(H), axis=1) != 0).sum(1)
W = H[np.argsort(changes)] / np.sqrt(8)          # sequency order
```

```
print((np.diff(np.sign(W), axis=1) != 0).sum(1))   # [0 1 2 3 4 5 6 7]

x = np.array([10, 12, 11, 13, 40, 42, 41, 43], float)
print(np.round(W @ x, 2))
```

這個測試訊號是一個帶有小漣漪的階梯。它的 WHT 幾乎全落在係數 0（平均值）與係數 1（恰好一次正負號變化，對應那個階梯）；其餘只剩幾個小數值。

斜轉換

白話版：影像中很多區域不是平的，而是逐漸變亮或變暗。斜轉換準備了一塊「斜坡」積木，讓這種區域只需要一兩個係數。

Pratt、Chen 與 Welch 為影像編碼設計了**斜轉換**（slant transform）[6]。它和 WHT 一樣是正交歸一、遞迴建構的，從 $\mathbf{S}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$ 開始：

$$\mathbf{S}_N = \frac{1}{\sqrt{2}}\,\mathbf{Q}_N \begin{bmatrix} \mathbf{S}_{N/2} & \mathbf{0} \\ \mathbf{0} & \mathbf{S}_{N/2} \end{bmatrix},$$

其中 $\mathbf{Q}_N$ 是一個稀疏的 $N \times N$ 混合矩陣，內含兩個常數

$$a_N = \sqrt{\frac{3N^2}{4(N^2-1)}}, \qquad b_N = \sqrt{\frac{N^2-4}{4(N^2-1)}},$$

它們的取值讓 $\mathbf{S}_N$ 的第二列成為等差遞減的階梯（「斜」向量），同時保持矩陣正交歸一。本章的繪圖腳本（`scripts/figures/dip_ch07.py`）就是這樣建出 $\mathbf{S}_8$，並以數值檢查這兩個性質。圖 7.2 中，斜轉換第一列與第一行的基底影像都是看得見的斜坡。斜轉換有 $O(N \log_2 N)$ 的快速演算法，但實務上同樣能處理斜坡的 DCT 取代了它。

哈爾轉換

白話版：把相鄰的兩個值拿來，記下它們的平均和差。再對平均值重複同樣的步驟。這就是哈爾轉換，也是最簡單的小波。

哈爾轉換（Haar transform）[1] 的基底函數要不是常數，就是一個「先上後下」的階梯，具有不同的寬度與位置。對 $N = 2^n$ 個取樣，用尺度 p 與位置 q 來編號，$k \geq 1$ 時 $k = 2^p + q$。不計歸一化，第 k 個基底向量在區間 $\left[\frac{q}{2^p}, \frac{q+1}{2^p}\right)$ 的前半段為 $+1$、後半段為 -1，其餘為 0（位置以訊號長度的比例表示）；$k = 0$ 的向量是常數。$N = 4$ 時：

$$\mathbf{A}_{\text{Haar}} = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} & 0 & 0 \\ 0 & 0 & \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix}.$$

第 0、1 列看整段訊號；第 2、3 列各自只看一半。這是它和前面所有轉換最關鍵的不同：**哈爾基底函數是局部的**。邊緣這類局部事件只會改變覆蓋到它的少數幾個哈爾係數。在時頻圖上，哈爾正好對應圖 7.1 的二進位（dyadic）鋪法。它是通往小波轉換的橋梁：以哈爾小波做的離散小波轉換，就是哈爾轉換。

小波轉換

白話版：用許多種縮放倍率看同一張影像。每往下一層，保留一張更模糊的版本，並只記下模糊時被抹掉的細節。模糊版本疊成一座金字塔，而那些細節就是小波係數。

多解析度分析

多解析度分析（multiresolution analysis, MRA）由 Mallat 形式化 [7]，把上面的想法變成一組基底。先取一個**尺度函數**（scaling function）$\varphi(x)$，做出平移與伸縮的版本：

$$\varphi_{j,k}(x) = 2^{j/2}\,\varphi(2^j x - k),$$

其中 j 是**尺度**（j 越大，函數越窄、細節越精細），k 是整數平移量，$2^{j/2}$ 讓能量維持為 1。令 V_j 為 $\{\varphi_{j,k}\}_k$ 所張成的空間，也就是在解析度 j 下能表示的所有訊號。MRA 要求四件事：

1. $\varphi_{0,k}$ 彼此正交歸一（或至少是 V_0 的穩定基底）。
2. 空間是巢狀的，$\cdots \subset V_{-1} \subset V_0 \subset V_1 \subset \cdots$：粗尺度看得到的東西，細尺度也看得到。
3. 所有 V_j 唯一的共同元素是 $f(x) = 0$。
4. 當 $j \to \infty$，任何平方可積函數都能被逼近到任意精確。

因為 $V_0 \subset V_1$，尺度函數本身必須能由更細的版本組合出來，於是得到**精細化方程式**（refinement equation，亦稱 dilation equation）：

$$\varphi(x) = \sum_{n} h_\varphi(n)\,\sqrt{2}\,\varphi(2x - n),$$

係數 $h_\varphi(n)$ 稱為**尺度函數係數**，稍後它會變成低通濾波器。

小波函數

從 V_{j+1} 降到 V_j 時遺失的細節，落在互補空間 W_j 中，使得 $V_{j+1} = V_j \oplus W_j$（$\oplus$ 表示 V_{j+1} 中每個元素都能唯一拆成 V_j 的一部分加上 W_j 的一部分；對正交小波，兩部分互相正交）。W_j 由**小波函數**（wavelet function）張成：

$$\psi_{j,k}(x) = 2^{j/2}\,\psi(2^j x - k), \qquad \psi(x) = \sum_{n} h_\psi(n)\,\sqrt{2}\,\varphi(2x - n).$$

對正交小波，**小波函數係數**可由尺度係數經調變與時間反轉得到：$h_\psi(n) = (-1)^n\, h_\varphi(1 - n)$，它們構成高通濾波器。一再重複這個拆分，得到

$$L^2(\mathbb{R}) = V_{j_0} \oplus W_{j_0} \oplus W_{j_0+1} \oplus \cdots,$$

其中 $L^2(\mathbb{R})$ 是所有有限能量函數構成的空間：一個粗略的近似，加上每個更細尺度的細節。

哈爾小波的 $h_\varphi = [\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}]$、$h_\psi = [\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}}]$：就是平均與差。Daubechies 展示了如何建構具有**緊支撐**（compact support，即有限長度濾波器）且平滑度可調的正交小波，其正則性隨濾波器長度線性增加 [8]。PyWavelets [10] 中的 `dbN` 家族就是這些小波，`N` 是**消失矩**（vanishing moments）的個數：對 $m = 0, \dots, N-1$ 都有 $\int x^m \psi(x)\, dx = 0$。具有 N 個消失矩的小波對次數低於 N 的多項式完全「視而不見」，因此平滑區域的細節係數幾乎為零。

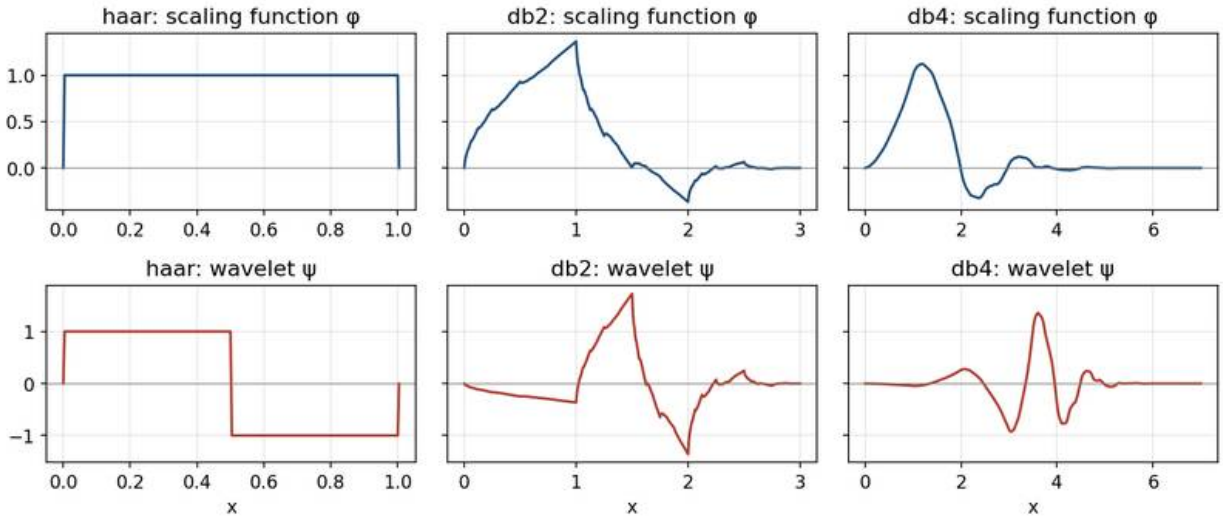


圖 7.3 — 以 *PyWavelets* 的級聯演算法（*cascade algorithm*）算出的尺度函數（上）與小波（下）：*Haar*、*db2*、*db4*。濾波器越長，函數越平滑、也越長。

小波級數展開

有了這兩族函數，連續訊號 $f(x)$ 可以展開成

$$f(x) = \sum_k c_{j_0}(k)\, \varphi_{j_0,k}(x) + \sum_{j=j_0}^{\infty} \sum_k d_j(k)\, \psi_{j,k}(x),$$

$$c_{j_0}(k) = \langle f, \varphi_{j_0,k} \rangle, \qquad d_j(k) = \langle f, \psi_{j,k} \rangle.$$

j_0 是任選的起始（最粗）尺度。$c_{j_0}(k)$ 稱為**近似**（或尺度）係數，$d_j(k)$ 稱為**細節**（或小波）係數。注意每個係數又是一個內積，與預備知識中完全相同。Unser 與 Blu 說明了消失矩、逼近階數與平滑度這些性質在此展開式中從何而來 [9]。

一維離散小波轉換

對取樣訊號 $f(x)$，$x = 0, \dots, M-1$，$M = 2^J$，總和變成有限項：

$$W_\varphi(j_0,k)=\frac{1}{\sqrt{M}}\sum_x f(x)\,\varphi_{j_0,k}(x),\qquad W_\psi(j,k)=\frac{1}{\sqrt{M}}\sum_x f(x)\,\psi_{j,k}(x),\quad j\ge j_0,$$

$$f(x)=\frac{1}{\sqrt{M}}\left[\sum_k W_\varphi(j_0,k)\,\varphi_{j_0,k}(x)+\sum_{j=j_0}^{J-1}\sum_k W_\psi(j,k)\,\psi_{j,k}(x)\right].$$

W_φ、W_ψ 是**離散小波轉換**（discrete wavelet transform, DWT）的近似係數與細節係數，此處的 $\varphi_{j,k}(x)$、$\psi_{j,k}(x)$ 是在 x 取樣的連續函數。對正交小波，係數總數恰好等於 M：DWT 是一次基底轉換，而不是擴張。

快速小波轉換

白話版：你完全不需要去算 φ 或 ψ 的值。兩個短濾波器加上「每隔一個取一個」，一層一層做下去就夠了。

把精細化方程式代入 DWT，就得到 Mallat 的金字塔演算法，今天稱為**快速小波轉換**（fast wavelet transform, FWT）[7]：

$$W_\psi(j,k)=\sum_n h_\psi(n-2k)\,W_\varphi(j+1,n),\qquad W_\varphi(j,k)=\sum_n h_\varphi(n-2k)\,W_\varphi(j+1,n).$$

讀法：把較細一層的近似 $W_\varphi(j+1,\cdot)$ 與 h_ψ（或 h_φ）做相關，再每隔一個取一個輸出（**2 倍降取樣**）。用訊號處理的語言，這就是雙通道**分析濾波器組**（analysis filter bank）：低通分支產生下一層近似，高通分支產生細節。把低通輸出再送進同一組濾波器，如此反覆。每一層資料量減半，所以總成本是 $O(M)$，比 FFT 的 $O(M\log M)$ 還快。

反向 FWT 是**合成濾波器組**：每個分支先 2 倍升取樣（插零）、經合成濾波器濾波，再相加。對正交小波，合成濾波器就是分析濾波器的時間反轉，整組濾波器能達成**完美重建**（perfect reconstruction）：即使每個分支的降取樣都造成混疊（aliasing），兩個分支的混疊項會互相抵消，輸出與輸入完全相同。Vetterli 與 Kovačević 的教科書從子頻帶編碼（subband coding）的角度深入發展了這個觀點 [2]。

```
import numpy as np
import pywt

x = np.array([4.0, 6.0, 10.0, 12.0, 8.0, 6.0, 5.0, 5.0])
w = pywt.Wavelet("haar")
h0, h1 = np.array(w.dec_lo), np.array(w.dec_hi)   # [.707 .707], [-.707 .707]

def analysis(x):
    lo = np.convolve(x, h0)[1::2]   # filter, then keep every 2nd sample
    hi = np.convolve(x, h1)[1::2]
    return lo, hi

a1, d1 = analysis(x)
a2, d2 = analysis(a1)
print("a2", a2, "d2", d2, "d1", d1.round(3))
```

```
ref = pywt.wavedec(x, "haar", level=2)            # [a2, d2, d1]
print(all(np.allclose(p, q) for p, q in zip(ref, [a2, d2, d1])))
print(np.allclose(pywt.waverec(ref, "haar"), x))
```

這兩行濾波器組與 `pywt.wavedec` 的結果完全一致。注意 $(5,5)$ 這一對的細節係數為 0：沒有變化，就沒有細節。

二維小波轉換

白話版：先沿著列跑一維濾波器組，再沿著行跑一次。你會得到一張小而模糊的影像，以及三張分別對應不同方向的「邊緣圖」。

二維 DWT 是可分離的。一層分解使用一個二維尺度函數與三個二維小波，各由一維函數相乘而成：

$$\varphi(x,y) = \varphi(x)\varphi(y), \quad \psi^H(x,y) = \psi(x)\varphi(y), \quad \psi^V(x,y) = \varphi(x)\psi(y), \quad \psi^D(x,y) = \psi(x)\psi(y).$$

依照課本慣例，x 是列索引（垂直位置），y 是行索引。ψ^H 沿垂直方向變化、水平方向平滑，所以對**水平**邊緣有反應；ψ^V 對**垂直**邊緣有反應；ψ^D 則對對角細節與角點有反應。一層分解把 $M \times N$ 影像變成四個 $\frac{M}{2} \times \frac{N}{2}$ 的**子頻帶**（subband）：

子頻帶	垂直方向濾波	水平方向濾波	常見稱呼	PyWavelets 名稱
近似	低通	低通	LL	`cA`
水平細節	高通	低通	LH 或 HL（慣例不一）	`cH`
垂直細節	低通	高通	HL 或 LH	`cV`
對角細節	高通	高通	HH	`cD`

由於不同教科書與函式庫對 LH ／ HL 的標記剛好相反，本章一律使用不會混淆的 H、V、D。對近似子頻帶重複分解，就得到圖 7.4 那種熟悉的巢狀排列。

```
import numpy as np
import pywt
from skimage import data, img_as_float

img = img_as_float(data.camera())
LL1, (H1, V1, D1) = pywt.dwt2(img, "haar")   # one level
LL2, (H2, V2, D2) = pywt.dwt2(LL1, "haar")   # second level on LL1
print(img.shape, LL1.shape, LL2.shape)       # (512, 512) (256, 256) (128, 128)

e = lambda a: float((a ** 2).sum())
total = e(img)
print(f"energy in LL2: {e(LL2) / total:.4f}")

coeffs = pywt.wavedec2(img, "haar", level=2)  # same thing in one call
print(np.allclose(pywt.waverec2(coeffs, "haar"), img))
```

兩層之後，128 × 128 的近似子頻帶只占係數的十六分之一，卻裝了 camera 影像約 99% 的能量。其餘 15/16 的係數大多接近零，這正是壓縮與去雜訊所利用的稀疏性。

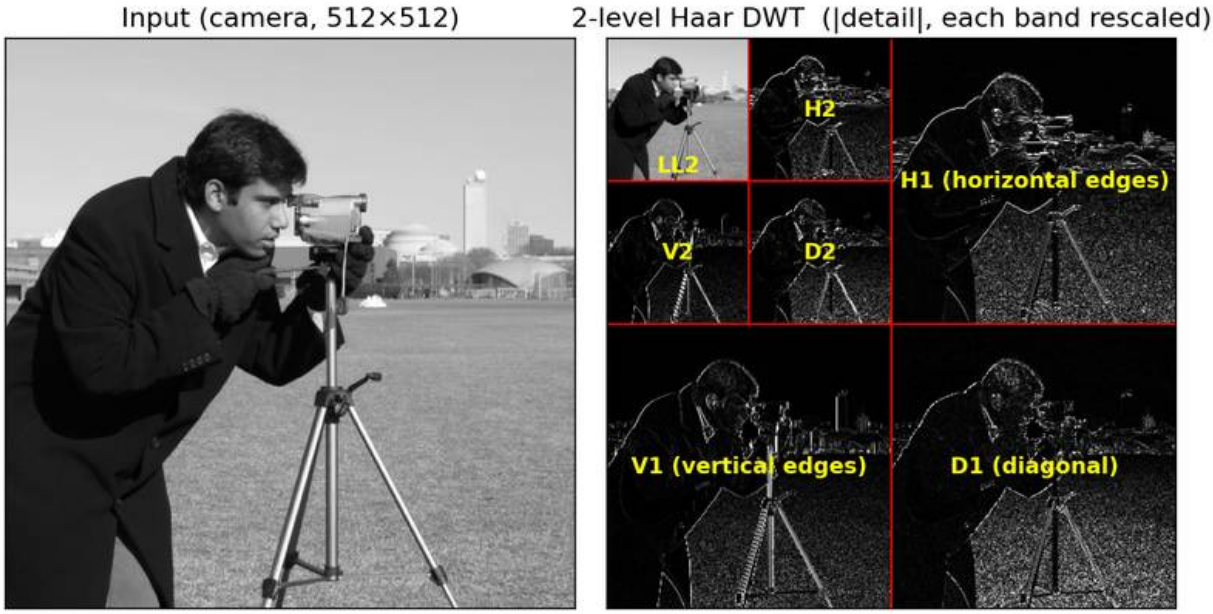


圖 *7.4 — camera* 影像的兩層哈爾 *DWT*。細節子頻帶顯示的是絕對值，且各自以其 *99.5* 百分位數重新縮放，好讓小係數看得見；若用同一個比例尺，它們幾乎全黑。三腳架垂直的腳在 *V1* 中特別亮，建築物的水平輪廓則出現在 *H1*。

小波封包

標準 DWT 只會再拆分低通子頻帶。**小波封包**（wavelet packet）分解則連細節子頻帶也全部再拆，形成一棵完整的二元樹（二維時是四元樹）。L 層之後，二維封包樹有 4^L 片葉子。任何一組能不重疊地覆蓋頻率軸的節點都是合法的正交歸一基底，所以這棵樹是一座基底圖書館。Coifman 與 Wickerhauser 的**最佳基底**（best basis）演算法能有效率地搜尋這座圖書館，挑出使某個可加成本（例如歸一化係數的熵）最小的基底 [11]。封包特別適合紋理，因為紋理在中高頻有大量能量，而一般 DWT 只把它們放在一個寬寬的子頻帶裡。

```
import pywt
from skimage import data, img_as_float

img = img_as_float(data.camera())
wp = pywt.WaveletPacket2D(img, "haar", maxlevel=2)
print([n.path for n in wp.get_level(2)][:6], len(wp.get_level(2)))  # 16 subbands
```

節點路徑記錄每一層走的分支（`a` 為近似，`h`、`v`、`d` 為細節），所以 `"hd"` 是水平細節子頻帶裡的對角細節。

應用

以門檻值去雜訊

白話版：在小波域裡，乾淨的影像只是少數幾個大數字，白雜訊則是散布各處的許多小數字。把小的歸零，再轉回來就好。

正交轉換會把標準差為 σ 的白高斯雜訊，變成每個子頻帶中標準差同樣為 σ 的白高斯雜訊，而影像的能量則集中在少數係數上。對細節係數 w 與門檻值 T，兩種門檻規則為：

$$\eta_{\text{hard}}(w) = \begin{cases} w, & |w| > T \\ 0, & |w| \le T \end{cases} \qquad \eta_{\text{soft}}(w) = \operatorname{sign}(w)\,\max(|w| - T,\, 0).$$

硬門檻（hard thresholding）只決定留或砍；軟門檻（soft thresholding）還會把留下的係數往零縮 T，得到連續的規則，假影較少，但對比會稍微損失。兩種經典的 T：

- **通用門檻值**（universal threshold，VisuShrink）：n 個取樣時 $T = \sigma\sqrt{2\ln n}$，出自 Donoho 與 Johnstone [12]。Donoho 證明用此 T 做軟門檻，所得估計有很高機率至少與真實訊號一樣平滑 [13]。雜訊強度通常由最細的對角子頻帶穩健地估計：$\hat{\sigma} = \text{median}(|w|)/0.6745$ [12]。
- **BayesShrink**：Chang、Yu 與 Vetterli 為每個子頻帶設定各自的門檻 $T = \hat{\sigma}^2/\hat{\sigma}_X$，其中 $\hat{\sigma}_X$ 是該子頻帶中乾淨係數的估計標準差 [14]。它會依各子頻帶含有多少訊號自動調整，通常優於通用門檻值。

```
import numpy as np
import pywt
from skimage import data, img_as_float
from skimage.metrics import peak_signal_noise_ratio as psnr

rng = np.random.default_rng(7)
clean = img_as_float(data.camera())
noisy = clean + rng.normal(0, 0.1, clean.shape)

def wavelet_denoise(y, wavelet="db4", level=3, mode="soft"):
    c = pywt.wavedec2(y, wavelet, mode="periodization", level=level)
    sigma = np.median(np.abs(c[-1][2])) / 0.6745       # noise level from finest HH
    T = sigma * np.sqrt(2 * np.log(y.size))            # universal threshold
    c = [c[0]] + [tuple(pywt.threshold(d, T, mode) for d in lvl) for lvl in c[1:]]
    return pywt.waverec2(c, wavelet, mode="periodization")

for mode in ["hard", "soft"]:
    out = np.clip(wavelet_denoise(noisy, mode=mode), 0, 1)
    print(mode, round(psnr(clean, out, data_range=1), 1), "dB")
print("noisy", round(psnr(clean, np.clip(noisy, 0, 1), data_range=1), 1), "dB")
```

在這個例子中，含雜訊影像的 PSNR 為 20.4 dB，硬門檻 25.7 dB，軟門檻 24.7 dB。通用門檻值本身就很保守（設計目標是幾乎清除所有雜訊），再加上軟門檻的收縮，就會過度平滑；圖 7.5 那種卡通般的外觀就是代價。BayesShrink 這類依子頻帶調整的門檻，或改用不抽樣（平移不變）的轉換，可以同時減輕模糊與方塊假影。scikit-image 在 `skimage.restoration.denoise_wavelet` 中包裝了這些變體。

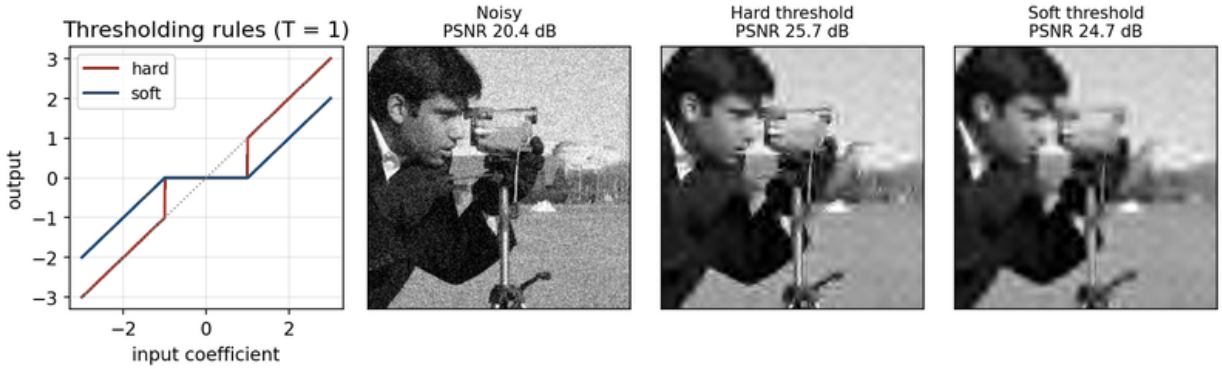
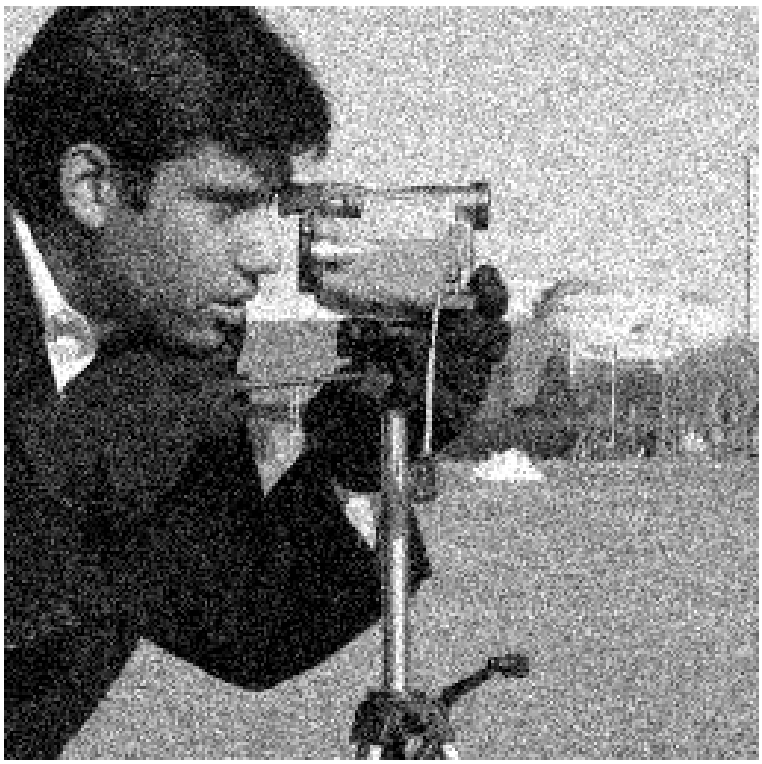


圖 *7.5 — camera* 影像的小波去雜訊（高斯雜訊，在 $[0, 1]$ 範圍內 $\sigma = 0.1$），使用 *db4*、三層分解與通用門檻值。硬門檻保留較銳利的邊緣，但留下零星的亮點；軟門檻較平滑，但也較模糊。

拖動滑桿，比較含雜訊的輸入與軟門檻的結果：



含雜訊

軟門檻

camera 影像含雜訊與小波去雜訊後的裁切比較

邊緣偵測

細節子頻帶是三個方向的高通響應，因此小波能當成快速的邊緣偵測器：把近似子頻帶設為零，再做反轉換。剩下的是影像減去它的粗略版本，主要集中在邊緣。只保留部分細節子頻帶就能挑選邊緣方向，如圖 7.6。越粗的層級得到的邊緣越粗，但對雜訊越穩健，這種多尺度行為類似於選擇高斯微分邊緣偵測器的 σ。

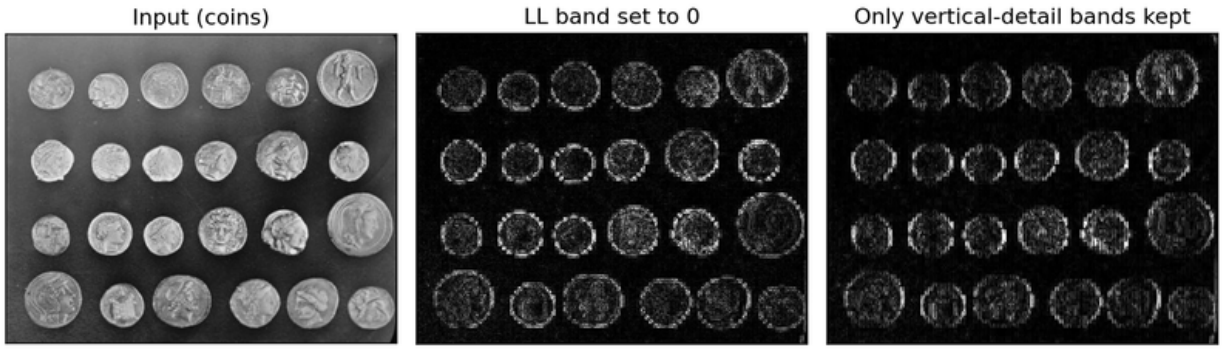


圖 *7.6* — 以 *coins* 影像兩層哈爾 *DWT* 得到的邊緣。中：近似子頻帶設為零。右：只保留垂直細節子頻帶，凸顯每枚硬幣左右兩側的邊緣。

壓縮預告

壓縮建立在能量集中之上：轉換、保留少數大係數、再有效率地編碼。圖 7.7 對整張 512 × 512 影像做四種轉換，只保留最大的 5% 係數。兩種小波比 DCT 高出 2 dB 以上、比 WHT 高出 4 dB 以上，而且誤差的樣貌不同：傅立葉類的基底會在邊緣周圍產生振鈴，並到處損失紋理；小波則保持邊緣清晰，主要在平滑或紋理區域損失細節。

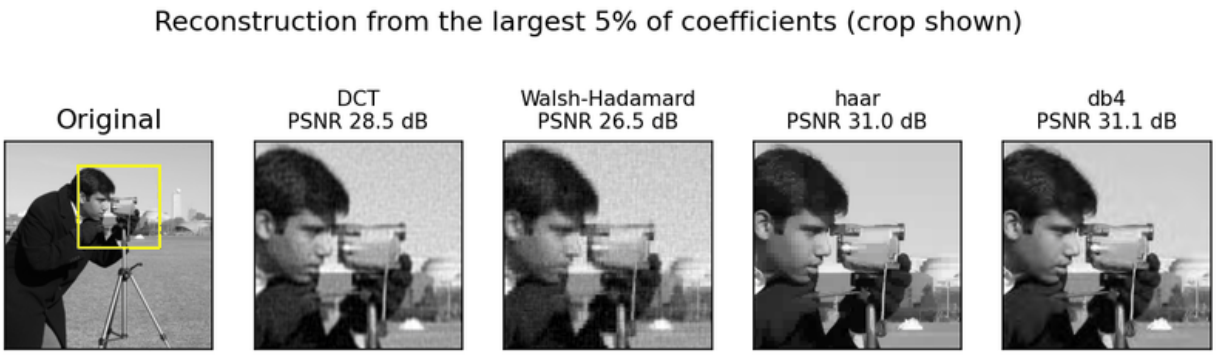


圖 *7.7* — 只用最大 *5%* 係數的重建結果（整張影像轉換，顯示裁切）。*PSNR*：*DCT 28.5 dB*、沃爾什–哈達瑪 *26.5 dB*、*Haar 31.0 dB*、*db4 31.1 dB*。

真正的編解碼器比「留下前 5%」細緻得多。JPEG 使用區塊 DCT 加上量化與熵編碼 [4]；JPEG 2000 則使用多層二維 DWT 與雙正交濾波器，再對每個子頻帶做位元平面編碼 [15]。IP 08 會詳細介紹這些流程，其中的轉換編碼實驗可以量化一張照片的 DCT 與 Haar 係數，看看誤差出現在哪裡。

現代觀點

小波在 1980 年代末以統一理論之姿進入影像處理，1990 與 2000 年代成為去雜訊與 JPEG 2000 的骨幹，如今則多半是一種成分：存在於稀疏模型、受訊號處理啟發的網路層，以及高效率的生成模型之中。以下是值得一讀的綜論與里程碑論文，以及各自的貢獻。

基礎：Mallat（1989）與 Daubechies（1988）。 Mallat 的論文 [7] 為影像引入多解析度分析，建立正交小波與雙通道濾波器組之間的連結，並提出讓轉換只需 $O(N)$ 的金字塔演算法。它也提出了分成水平、垂直、對角三個子頻帶的二維可分離分解，至今每個函式庫仍沿用。Daubechies [8] 補上了最後一塊拼圖：具有緊支撐且平滑度可選的正交小波，讓濾波器又短又實用。兩篇合起來，就解釋了 `pywt.wavedec2(img, "db4")` 為何存在。

教科書等級的綜覽：Vetterli 與 Kovačević（1995）。《Wavelets and Subband Coding》[2] 從訊號處理的角度建立整套理論：濾波器組、完美重建、時頻鋪磚與編碼。作者買回版權並免費公開全書，是本章濾波器組觀點最好的開放參考資料。

什麼讓小波好用：Unser 與 Blu（2003）。〈Wavelet theory demystified〉[9] 把任何尺度函數寫成一個 B 樣條（B-spline）與一個殘餘「不規則」部分的卷積，藉此以自成一體的方式重建古典理論。它的主要結論是：實務上在乎的五個性質，也就是逼近階數、多項式重現、消失矩、多尺度微分性質與平滑度，全都由那個 B 樣條因子單獨決定。想知道為什麼 `db4` 在平滑影像上比 Haar

好，這篇論文給了答案。

從基底到字典：Rubinstein、Bruckstein 與 Elad（2010）。〈Dictionaries for sparse representation modeling〉[16] 綜覽了從解析式轉換（傅立葉、DCT、小波及其方向性後繼者）到從資料學習的字典這段演變。它的重點是通往現代方法的橋梁：小波「少數係數就能解釋影像」的想法保留了下來，但基底變成可以訓練的東西。這條路線直接通往影像復原中的學習式先驗。

標準的實際樣貌：Skodras、Christopoulos 與 Ebrahimi（2001）。他們的 JPEG 2000 概論 [15] 說明了 DWT、雙正交濾波器、子頻帶量化與嵌入式編碼如何在真實編解碼器中組合，以及多解析度帶來了哪些功能（解析度可調性、感興趣區域編碼、無失真模式）。

古典去雜訊理論：Donoho 與 Johnstone；Chang、Yu 與 Vetterli。這幾篇收縮（shrinkage）論文 [12]、[13]、[14] 證明在小波域套一個簡單的非線性函數，對廣泛的訊號類別都接近最佳，而且依子頻帶調整門檻在實務上更有幫助。所有「對轉換係數做門檻來去雜訊」的方法，包括深度學習之前許多手工設計的先驗，都源自這些結果。

深度學習改變了什麼、沒改變什麼

散射網路。 Bruna 與 Mallat 的**小波散射轉換**（wavelet scattering transform）[17] 把小波卷積、取模（modulus）非線性與平均化層層串接。它是一個濾波器固定為小波、而非經由訓練的卷積網路，具有平移不變性且對形變穩定。第一層的行為類似 SIFT 類描述子，更高層則補充更多資訊，能分辨傅立葉功率頻譜相同的紋理。它在當時的手寫數字與紋理分類上取得最佳結果，並成為解釋 CNN 為何有效的參考點：一個小波框架（如預備知識所述，是冗餘的）加上簡單的非線性，就已經能得到不變性。

把小波當成池化與降取樣層。跨步卷積與最大池化在降取樣前沒有做適當的低通濾波，因此會混疊。Li 等人在標準分類網路中以 DWT 層取代這些運算（WaveCNet），只保留低頻子頻帶，提升了對雜訊的穩健性 [19]。在影像復原方面，Liu 等人的 **MWCNN** 以二維 DWT 及其反轉換取代 U-Net 的池化與反池化 [18]。由於 DWT 可逆，網路縮小特徵圖時不會遺失資訊，又能便宜地得到很大的感受野。這種「可逆且能抑制混疊的降取樣」想法，後來出現在許多影像復原與生成模型中。

大感受野。 Finder 等人的 **WTConv** 層（ECCV 2024）在多層 DWT 的各子頻帶上做小卷積，因此有效感受野隨層數指數成長，而參數量只隨核大小對數成長 [20]。它可以直接放進 ConvNeXt、MobileNetV2 等架構中替換原本的層，作者報告它提升了對影像損壞的穩健性，並使模型更偏重形狀而非紋理。

沒有改變的部分。 DCT 仍然在 JPEG 裡，DCT 及其整數近似也仍是傳統影像與視訊編解碼器的核心工具。手工調校的小波門檻去雜訊，在品質上已被訓練出來的網路超越，但它仍是最快的強基準、不需要訓練資料，而且有理論保證。最重要的是，本章的詞彙（基底、框架、濾波器組、多解析度、混疊、稀疏性）正是用來分析卷積網路在做什麼的詞彙。

重點整理

- 線性轉換就是換基底。每個係數都是一個內積，也就是影像與某個基底函數的相關值。
- 正交歸一轉換就是矩陣乘法；可分離的二維情形為 $\mathbf{T} = \mathbf{AFA}^T$，反轉換用轉置即可，且能量守恆。

- 因為隱含的偶對稱延伸沒有邊界跳躍，DCT 對平滑影像的能量集中優於 DFT 與 WHT，這也是 JPEG 採用它的原因。
- WHT 與斜轉換分別以簡單性（只有 ± 1）或明確的斜坡向量換取部分能量集中能力；哈爾則多了局部性。
- 時頻平面呈現了取捨：傅立葉基底頻率精確、取樣時間精確，小波則依頻率調整磚的形狀。
- 多解析度分析把兩個短濾波器變成 $O(N)$ 的快速小波轉換；在二維中，每一層產生一個近似子頻帶與水平、垂直、對角三個細節子頻帶。
- 小波域的稀疏性帶動了去雜訊（硬／軟門檻）、快速邊緣圖與壓縮（JPEG 2000）。
- 現代網路把小波當成固定特徵擷取器（散射）、可逆降取樣器（MWCNN、WaveCNet），以及便宜的大感受野層（WTConv）。

練習

1. **親手做一個框架**。取平面上角度為 90°、210°、330° 的三個單位向量。分別對 $\mathbf{f} = (1, 0)$ 與 $\mathbf{f} = (0, 1)$ 計算 $\sum_k \langle \mathbf{f}, \mathbf{s}_k \rangle^2$。框架界是多少？要如何由三個係數重建 $\mathbf{f}$？

   提示

   兩個總和都等於 3/2；事實上對任何單位向量 $\mathbf{f}$ 總和都是 3/2，所以這是 $A = B = 3/2$ 的緊框架。重建公式為 $\mathbf{f} = \frac{2}{3} \sum_k \langle \mathbf{f}, \mathbf{s}_k \rangle \mathbf{s}_k$。

2. **可分離的成本**。對一張 256×256 影像，計算下列情形所需的乘法次數：(a) 把一般二維線性轉換寫成一個 65536×65536 矩陣；(b) 以稠密 $\mathbf{A}$ 做可分離形式 $\mathbf{A}\mathbf{F}\mathbf{A}^T$；(c) 三層哈爾 FWT。

   提示

   (a) $N^4 \approx 4.3 \times 10^9$。(b) 兩次各 N^3 的乘法，$2N^3 \approx 3.4 \times 10^7$。(c) 每一層以 2 個係數的濾波器、在半速率下處理目前近似的每一列與每一行；總量是 N^2 乘上一個小常數，約數十萬次。

3. **為什麼要鏡射？**取 $f = [1, 2, 3, 4, 5, 6, 7, 8]$（一個斜坡），用 SciPy 計算它的 DFT 與 DCT，分別數一數要多少個係數才能涵蓋 99% 的能量，並用隱含的週期延伸與偶對稱延伸解釋差異。

   提示

   週期延伸會從 8 跳回 1，需要很多 DFT 諧波。偶對稱延伸 $\ldots 2, 1, 1, 2, \ldots, 8, 8, 7, \ldots$ 是連續的，所以 DCT 只需要兩三個係數。

4. **手算哈爾**。對 $[2, 2, 6, 6, 3, 5, 9, 9]$ 做兩層哈爾 DWT（平均與差都乘上 $1/\sqrt{2}$）。哪些細節係數是零？為什麼？用 `pywt.wavedec` 驗證你的答案。

   提示

   第一層的配對是 (2,2)、(6,6)、(3,5)、(9,9)，只有 (3,5) 的差不為零。第二層比較相鄰兩對縮放後的和。注意 PyWavelets 的正負號慣例：用它的哈爾濾波器，每個細節係數等於（前 − 後）$/\sqrt{2}$，所以 (3, 5) 這一對得到 $-\sqrt{2}$。

5. **實際調整門檻**。修改去雜訊程式碼，改用 (a) 門檻值 $3\hat{\sigma}$，以及 (b) 依子頻帶計算的 BayesShrink 門檻 $\hat{\sigma}^2/\hat{\sigma}_X$，其中每個子頻帶的 $\hat{\sigma}_X = \sqrt{\max(\overline{w^2} - \hat{\sigma}^2, 0)}$。在 $\sigma = 0.05$ 與 $\sigma = 0.2$ 下，和通用門檻值比較 PSNR。

   提示

   512×512 影像的通用門檻值約為 $5\hat{\sigma}$，所以 (a) 會保留更多係數。軟門檻時，(b) 應該是三者中最好的。若 $\hat{\sigma}_X = 0$，就把整個子頻帶設為零。

6. **對平移的敏感度**。把 camera 影像水平平移一個像素，對兩個版本各做一層哈爾 DWT，比較 V 子頻帶的能量。為什麼平移一個像素會讓係數變化這麼大？不抽樣轉換（`pywt.swt2`）又會如何表現？

   提示

   2 倍降取樣讓 DWT 對平移敏感：落在同一個哈爾配對內部的邊緣會產生細節係數，落在兩對之間的同一條邊緣卻不會。平穩（不抽樣）轉換省略降取樣，因此是一個冗餘框架，係數會跟著影像一起平移。

參考文獻

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 7. publisher page
2. M. Vetterli and J. Kovačević, *Wavelets and Subband Coding*, Prentice Hall, 1995（作者提供免費版本）. book site
3. N. Ahmed, T. Natarajan and K. R. Rao, "Discrete Cosine Transform," *IEEE Transactions on Computers*, 1974. doi
4. G. K. Wallace, "The JPEG still picture compression standard," *IEEE Transactions on Consumer Electronics*, 1992. doi
5. W. K. Pratt, J. Kane and H. C. Andrews, "Hadamard transform image coding," *Proceedings of the IEEE*, 1969. doi
6. W. K. Pratt, W.-H. Chen and L. R. Welch, "Slant transform image coding," *IEEE Transactions on Communications*, 1974. doi
7. S. G. Mallat, "A theory for multiresolution signal decomposition: the wavelet representation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1989. doi
8. I. Daubechies, "Orthonormal bases of compactly supported wavelets," *Communications on Pure and Applied Mathematics*, 1988. doi
9. M. Unser and T. Blu, "Wavelet theory demystified," *IEEE Transactions on Signal Processing*, 2003. doi
10. G. R. Lee, R. Gommers, F. Waselewski, K. Wohlfahrt and A. O'Leary, "PyWavelets: A Python package for wavelet analysis," *Journal of Open Source Software*, 2019. doi
11. R. R. Coifman and M. V. Wickerhauser, "Entropy-based algorithms for best basis selection," *IEEE Transactions on Information Theory*, 1992. doi

12. D. L. Donoho and I. M. Johnstone, “Ideal spatial adaptation by wavelet shrinkage,” *Biometrika*, 1994. doi
13. D. L. Donoho, “De-noising by soft-thresholding,” *IEEE Transactions on Information Theory*, 1995. doi
14. S. G. Chang, B. Yu and M. Vetterli, “Adaptive wavelet thresholding for image denoising and compression,” *IEEE Transactions on Image Processing*, 2000. doi
15. A. Skodras, C. Christopoulos and T. Ebrahimi, “The JPEG 2000 still image compression standard,” *IEEE Signal Processing Magazine*, 2001. doi
16. R. Rubinstein, A. M. Bruckstein and M. Elad, “Dictionaries for sparse representation modeling,” *Proceedings of the IEEE*, 2010. doi
17. J. Bruna and S. Mallat, “Invariant scattering convolution networks,” arXiv:1203.1513, 2012. arXiv
18. P. Liu, H. Zhang, K. Zhang, L. Lin and W. Zuo, “Multi-level wavelet-CNN for image restoration,” CVPR Workshops (NTIRE), 2018. arXiv
19. Q. Li, L. Shen, S. Guo and Z. Lai, “Wavelet integrated CNNs for noise-robust image classification,” CVPR, 2020. arXiv
20. S. E. Finder, R. Amoyal, E. Treister and O. Freifeld, “Wavelet convolutions for large receptive fields,” ECCV, 2024. arXiv

IP 08・影像壓縮與浮水印

影像如何被壓進更少的位元：從熵與霍夫曼碼，到 JPEG、JPEG 2000 與學習式編解碼器，以及如何把隱藏標記寫進影像。

網站版本: https://blog.shyandram.dev/zh-tw/learn/image-processing/dip-08-compression-watermarking/

你將學到

- 壓縮要去除的三種冗餘，以及熵（entropy）如何替無失真編碼設下下限
- 經典無失真編碼器的原理：霍夫曼、Golomb、算術、LZW、游程、符號式與位元平面編碼
- JPEG 如何把影像切成 8×8 的 DCT 區塊並量化，以及低品質時為什麼會出現方塊
- 預測編碼（DPCM、運動補償）與小波編碼（JPEG 2000）如何利用相鄰像素
- 如何把浮水印藏在最低有效位元或 DCT 係數中，以及為什麼一個撐得過 JPEG、另一個撐不過
- 學習式（神經網路）編解碼器與深度學習浮水印在今天的位置

先看全貌

一張未壓縮的 512×512 彩色照片是 786,432 位元組。同一張照片存成 JPEG 可能只有幾十 KB，看起來卻沒什麼差別。壓縮就是找出哪些位元可以預測、哪些細節看不出來，然後不去儲存它們。

> **用最簡單的話說**
>
> 想像你在電話裡跟朋友描述一張天空的照片。你不會說一百萬次「藍色、藍色、藍色……」，而是說「上面全是藍色，左邊有一朵白雲」。這就是壓縮：常見的東西簡短帶過，只把話花在意外的地方。浮水印則像是用隱形墨水在畫上簽名，之後就能證明這張畫是你的。

我們每天看到的影像幾乎都經過壓縮，許多電腦視覺資料集也以 JPEG 儲存，所以模型其實是在帶有壓縮瑕疵的影像上訓練的。浮水印則關係到著作權，也越來越常用來標記生成式模型產生的影像。本章依照教科書 [1] 的順序：先談基礎，再介紹一整套編碼方法，最後談浮水印。

基礎概念

白話版：影像檔裡的位元數通常比它真正承載的資訊多。壓縮就是去掉多餘的位元，而「資訊量」的度量告訴我們最多能壓到哪裡。

壓縮比與冗餘

若一張影像未壓縮時佔 b 位元，壓縮後佔 b' 位元，則**壓縮比**與**相對資料冗餘**為

$$C = \frac{b}{b'}, \qquad R = 1 - \frac{1}{C}.$$

C 是無因次的數字（10 代表「十比一」），R 是原始資料中屬於冗餘的比例。$C = 10$ 時 $R = 0.9$：九成的位元沒有帶來新資訊。

影像中有三種冗餘：

- **編碼冗餘（coding redundancy）**：固定 8 位元的碼對每個灰階都花 8 位元，不論常見或罕見。變長碼可以少浪費一些位元。
- **空間與時間冗餘（spatial and temporal redundancy）**：相鄰像素、相鄰的視訊畫格高度相關，把每個像素獨立編碼就是在重複資訊。
- **不相關資訊（irrelevant information）**：觀看者察覺不到的細節。去掉它是不可逆的（**失真**壓縮，lossy）；只去掉前兩種則是**無失真**壓縮（lossless）。

度量資訊：熵

機率為 $P(a)$ 的符號帶有 $I(a) = -\log_2 P(a)$ 位元的資訊：越罕見的事件越令人意外，資訊越多。對整個符號集合取平均就得到**熵**

$$H = -\sum_{j=1}^{J} P(a_j)\,\log_2 P(a_j) \quad \text{bits/symbol},$$

其中 $a_1, \dots, a_J$ 是所有可能的符號（8 位元影像就是 256 個灰階），$P(a_j)$ 是它們的機率。無雜訊信源編碼定理指出：當符號被逐一、獨立編碼時，沒有任何無失真碼的平均長度能低於 H 位元／符號；好的碼可以逼近 H。

對真實影像我們只能估計。**一階估計**使用正規化直方圖 $p_r(r_k) = n_k/MN$，其中 n_k 是 $M \times N$ 影像中灰階為 r_k 的像素數。圖 8.1 說明這個估計取決於你把什麼當成「符號」。camera 影像的一階熵是 7.23 位元／像素，幾乎等於 8；如果改為編碼每個像素與左鄰像素的差值，熵降到 4.70 位元。差值並沒有比較聰明，它只是去掉了直方圖看不到的空間冗餘。這正是本章後面預測編碼的核心想法。

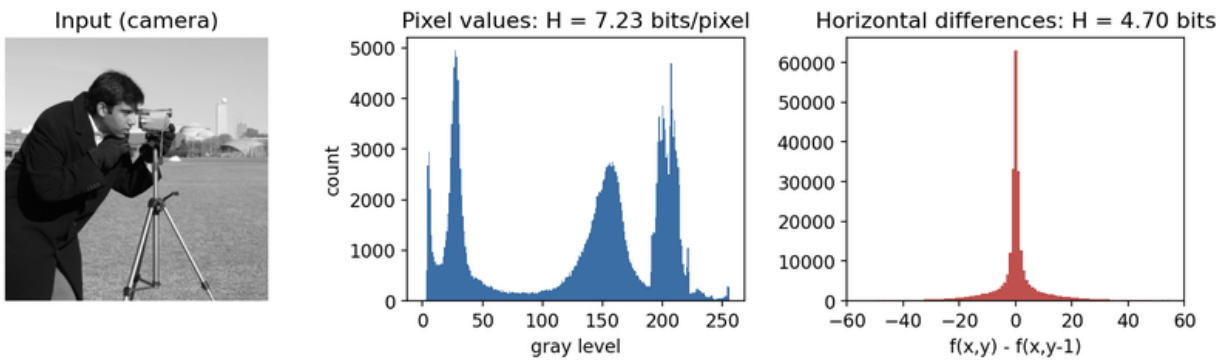


圖 *8.1* — 同一張影像、兩種符號選擇。像素值分布很散（*7.23* 位元）；相鄰差值集中在 *0* 附近（*4.70* 位元）。

```
import numpy as np
import Lab from '../../../../components/Lab.astro';
from skimage import data

def entropy(x):
    _, counts = np.unique(x, return_counts=True)
    p = counts / counts.sum()
    return -(p * np.log2(p)).sum()

img = data.camera()
print(f"H(pixels) = {entropy(img):.2f} bits")                          # 7.23
print(f"H(diffs)  = {entropy(np.diff(img.astype(int), axis=1)):.2f} bits")  # 4.70
```

保真度準則

有失真時，我們需要衡量損害。設 $f(x,y)$ 為原始影像、$\hat{f}(x,y)$ 為解壓縮後的 $M \times N$ 影像。**均方根誤差**與**峰值訊噪比**為

$$e_{\text{rms}} = \sqrt{\frac{1}{MN}\sum_{x=0}^{M-1}\sum_{y=0}^{N-1}\left[\hat{f}(x,y)-f(x,y)\right]^2}, \qquad \text{PSNR} = 20\log_{10}\frac{L_{\max}}{e_{\text{rms}}}\ \text{dB},$$

其中 $L_{\max}$ 是像素可能的最大值（8 位元影像為 255）。PSNR 越高，誤差越小；粗略來說，40 dB 左右的差異很難看出，低於 25 dB 則很明顯，但仍取決於影像內容。

這些**客觀**準則容易計算，卻不一定與人眼一致。**主觀**準則請觀察者以量表（從「極佳」到「無法使用」）評分，或兩兩比較。

影像壓縮模型

本章幾乎所有編解碼器都符合同一張方塊圖。**編碼器**有三個階段：

1. **映射器（mapper）**：把影像轉成冗餘較少的形式（差值、轉換係數、游程長度）；
2. **量化器（quantizer）**：依保真度目標丟棄精度（唯一有失真的階段，無失真編解碼器沒有這一步）；
3. **符號編碼器（symbol coder）**：給常見符號較短的碼（霍夫曼、算術、Golomb……）。

解碼器由符號解碼器與反映射器組成。失去的精度不存在「反量化」可以找回。

格式、容器與標準

檔案格式決定位元如何排列；**容器**把編碼後的影像與中繼資料包在一起（可裝不同的編解碼器）；**壓縮標準**則定義編碼方法本身。常見的有：

名稱	類型	核心方法	無失真？
JPEG（ISO/IEC 10918-1、ITU-T T.81）[2]	標準＋格式	8×8 DCT、量化、霍夫曼或算術編碼	有失真（baseline）
JPEG 2000（ISO/IEC 15444）[3]	標準＋格式	離散小波轉換、位元平面算術編碼	兩者皆可
PNG [4]	格式	逐列預測濾波器＋ deflate（LZ77 衍生）	無失真
JPEG-LS [10]	標準	依情境的預測＋ Golomb 類碼	無失真／近無失真
JPEG XL（ISO/IEC 18181）[5]	標準＋格式	有失真與無失真模式；可無損轉存 JPEG 檔	兩者皆可
HEIF（ISO/IEC 23008-12）[7]	容器	可裝 HEVC 編碼影像（`.heic`）等	視編解碼器而定
AVIF [6]	基於 HEIF 的格式	把 AV1 畫格內編碼存進 HEIF	兩者皆可

霍夫曼編碼

白話版：最常見的符號給最短的碼，而且任何一個碼都不是另一個碼的開頭，這樣位元流不需要分隔符號也能讀。

霍夫曼演算法（Huffman coding）[8] 在已知符號機率、且逐一編碼的條件下，建出最佳的**前綴碼**（prefix code）。它由下往上建構：

1. 列出所有符號與機率。
2. 把機率最小的兩項合併成新節點，機率為兩者之和。
3. 重複直到只剩一個節點（機率 1）。
4. 從根走到每個葉節點，一邊寫 0、另一邊寫 1，路徑就是碼字。

實作範例

考慮六個符號：$P(a_1) = 0.40$、$P(a_2) = 0.20$、$P(a_3) = 0.15$、$P(a_4) = 0.10$、$P(a_5) = 0.10$、$P(a_6) = 0.05$。合併順序如下：

步驟	合併	新節點
1	a_6（0.05）＋ a_4（0.10）	0.15
2	a_5（0.10）＋ a_3（0.15）	0.25
3	a_2（0.20）＋節點 0.15	0.35
4	節點 0.25 ＋節點 0.35	0.60
5	a_1（0.40）＋節點 0.60	1.00

從圖 8.2 的樹可得 $a_1 = $ `1`、$a_2 = $ `000`、$a_3 = $ `010`、$a_4 = $ `0010`、$a_5 = $ `011`、$a_6 = $ `0011`。平均碼長為

$$L_{\text{avg}} = \sum_{j} P(a_j)\, l(a_j) = 0.4(1) + 0.2(3) + 0.15(3) + 0.1(4) + 0.1(3) + 0.05(4) = 2.35 \text{ bits/symbol},$$

其中 $l(a_j)$ 是 a_j 的碼長。固定長度碼需要 3 位元，熵則是 $H = 2.284$ 位元，所以霍夫曼碼離下限不到 3%。機率相同時的平手可以任意處理；不同的選擇會改變碼字，但不會改變平均長度。

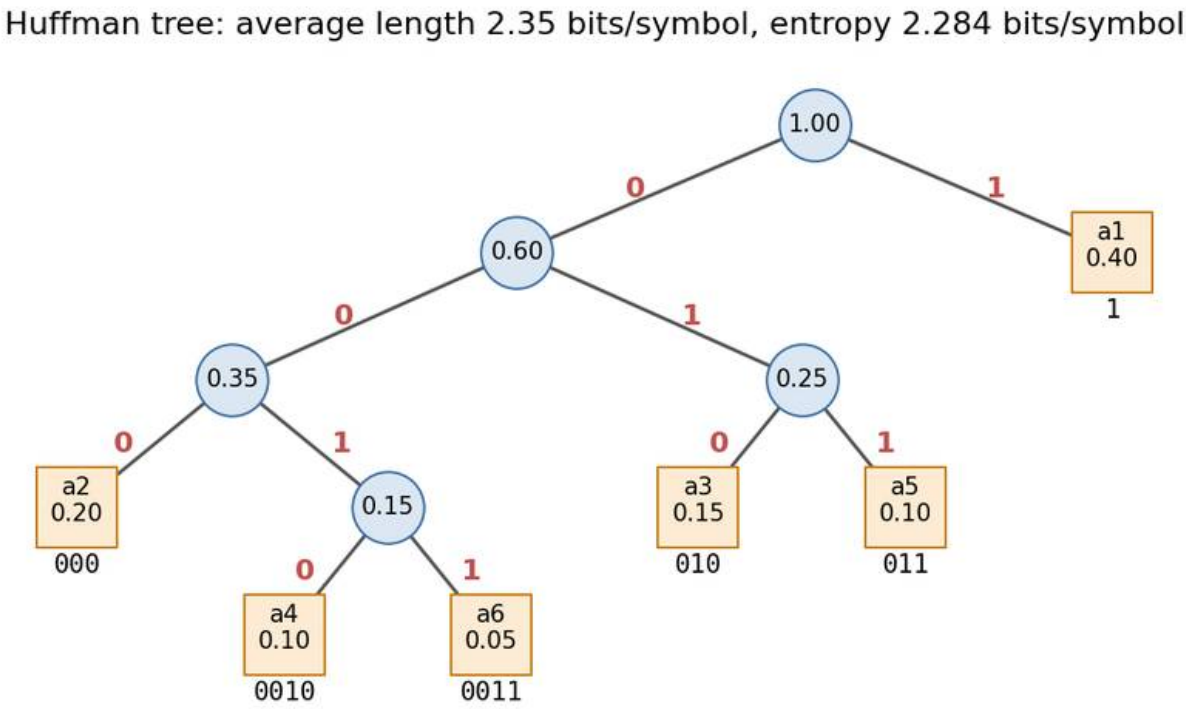


圖 *8.2* — 六符號範例的霍夫曼樹。邊上的數字是位元，碼字印在各葉節點下方。

解碼就是沿著樹往下走：讀位元直到碰到葉節點，輸出符號，再回到根。因為沒有碼字是另一碼字的前綴，這種碼是**即時可解**且**唯一可解**的。JPEG 用霍夫曼表來編碼量化後的係數 [2]。

```
import heapq, itertools

def huffman_codes(probs):
    tick = itertools.count()                        # tie-breaker for equal probabilities
    heap = [(p, next(tick), {s: ""}) for s, p in probs.items()]
    heapq.heapify(heap)
    while len(heap) > 1:
        p1, _, c1 = heapq.heappop(heap)             # least probable
        p2, _, c2 = heapq.heappop(heap)             # second least
        merged = {s: "1" + c for s, c in c1.items()}
        merged |= {s: "0" + c for s, c in c2.items()}
        heapq.heappush(heap, (p1 + p2, next(tick), merged))
    return heap[0][2]

P = {"a1": .40, "a2": .20, "a3": .15, "a4": .10, "a5": .10, "a6": .05}
print(huffman_codes(P))   # a1:'1', a2:'000', a3:'010', a4:'0010', a5:'011', a6:'0011'
```

它的限制是：每個符號至少花 1 個完整位元。若某符號機率是 0.95，它只帶 0.07 位元的資訊，霍夫曼碼卻仍要花 1 位元。下面的算術編碼可以突破這個限制。

Golomb 編碼

白話版：當小數字很常見、大數字很少見時，把數字寫成「有幾整組 m」（用畫記表示）再加「剩下多少」（用一般二進位表示）。

Golomb 碼 [9] 是為近似幾何分布的非負整數 n 設計的，而預測殘差經過映射後正是這種分布。給定參數 $m > 0$，碼 $G_m(n)$ 由兩段串接而成：

1. 商 $q = \lfloor n/m \rfloor$ 的一**元碼**：q 個 1 後接一個 0；
2. 餘數 $r = n \bmod m$ 的**截斷二進位碼**：令 $k = \lceil \log_2 m \rceil$、$c = 2^k - m$，$r < c$ 時用 $k-1$ 位元，其餘則把 $r + c$ 寫成 k 位元。

n	0	1	2	3	4	5	6	7	8
$G_4(n)$	000	001	010	011	1000	1001	1010	1011	11000
$G_3(n)$	00	010	011	100	1010	1011	1100	11010	11011

當 m 是 2 的冪次時，餘數就是 n 的最低 k 個位元，編碼只需要位移與遮罩。預測殘差 e 有正負號，所以要先映射成非負整數，例如 $e \ge 0$ 時 $M(e) = 2e$、$e < 0$ 時 $M(e) = 2|e| - 1$，得到 $0, -1, 1, -2, 2, \dots$ 交錯排列。JPEG-LS [10] 結合這種映射，以及參數為 2 的冪次、並依局部情境調整的 Golomb 碼。

```
import numpy as np

def golomb(n, m):
    q, r = divmod(n, m)
    unary = "1" * q + "0"
    k = int(np.ceil(np.log2(m)))
    c = 2**k - m                        # how many remainders get k-1 bits
    if r < c:
        tail = format(r, f"0{k-1}b") if k > 1 else ""
    else:
        tail = format(r + c, f"0{k}b")
    return unary + tail

print([golomb(n, 3) for n in range(9)])
```

算術編碼

白話版：不替每個符號各自配碼，而是把整段訊息描述成 0 到 1 之間一個非常精確的數字。

算術編碼（arithmetic coding）[11] 從區間 $[0, 1)$ 開始。每讀一個符號，就把目前的區間依該符號的機率縮小到它所屬的子區間。訊息結束後，最終區間中的任何一個數都能代表這段訊息。若目前區間是 $[\ell, h)$，下一個符號 s 擁有累積範圍 $[F_{\text{lo}}(s), F_{\text{hi}}(s))$，則

$$\ell' = \ell + (h - \ell)\, F_{\text{lo}}(s), \qquad h' = \ell + (h - \ell)\, F_{\text{hi}}(s).$$

最終寬度等於各符號機率的乘積，所需位元數約為 $-\log_2 \prod_i P(s_i) = \sum_i -\log_2 P(s_i)$，正好是訊息的資訊量，沒有「每個符號至少 1 位元」的下限。

實作範例

令 $P(a) = 0.6$、$P(b) = 0.3$、$P(c) = 0.1$，於是 a 擁有 $[0, 0.6)$、b 擁有 $[0.6, 0.9)$、c 擁有 $[0.9, 1)$。編碼訊息 `abac`：

讀入	區間
`a`	$[0, 0.6)$
`b`	$[0.36, 0.54)$
`a`	$[0.36, 0.468)$
`c`	$[0.4572, 0.468)$

寬度為 $0.6 \times 0.3 \times 0.6 \times 0.1 = 0.0108$，即 $-\log_2 0.0108 \approx 6.5$ 位元的資訊。7 位元的二進位小數 $0.0111011_2 = 0.4609375$ 落在區間內，所以 7 個位元就能識別這段訊息（另外需要標示結尾，例如長度或結束符號）。

實際的編碼器使用整數運算並不斷重新正規化，避免區間下溢；也通常使用**自適應**機率模型，邊編碼邊更新，因此不必傳送機率表。JPEG 2000 的係數編碼器最後接的是自適應二元算術編碼器 [14]，「現代觀點」中的學習式編解碼器最後也是用算術編碼。

LZW 編碼

白話版：一邊讀一邊建立詞組字典；詞組再次出現時，只傳它在字典裡的編號。

Lempel–Ziv–Welch（LZW）演算法 [12] 一開始的字典只含所有單一符號（8 位元像素就是 0–255）。它持續延長目前詞組 w，只要「w 加上下一個符號」仍在字典中。一旦不在，就輸出 w 的碼，把延長後的詞組加為新項目（256、257……），再從新符號重新開始。解碼器能從收到的碼重建同一本字典，因此字典不需要傳送。

對一列重複四次 `39 39 126 126` 的 16 個像素，輸出是十個碼：

```
def lzw_encode(seq):
    table = {(v,): v for v in range(256)}
    w, out = (), []
    for x in seq:
        wx = w + (x,)
        if wx in table:
            w = wx
        else:
            out.append(table[w])
            table[wx] = len(table)
            w = (x,)
    if w:
        out.append(table[w])
    return out
```

```
row = [39, 39, 126, 126] * 4
print(lzw_encode(row))   # [39, 39, 126, 126, 256, 258, 260, 259, 257, 126]
```

從 256 起的碼依序代表 `39 39`、`39 126`、`126 126`、`126 39`、`39 39 126`……重複越長，詞組越長，壓縮比越好。LZW 不需要事先知道機率，因此在通用檔案格式中很受歡迎。PNG 使用的是相關的 LZ77 系 deflate 方法 [4]。

游程編碼

白話版：「0000 11 00000 1」變成「四個 0、兩個 1、五個 0、一個 1」。

游程編碼（run-length encoding, RLE）把每一段相同值的連續序列換成（值, 長度）。二值影像只要約定第一段的顏色，就只需要記長度。游程長度本身再用變長碼編碼，因為短游程比長游程常見。

```
import numpy as np

def rle(row):
    starts = np.r_[0, np.flatnonzero(np.diff(row)) + 1]
    lengths = np.diff(np.r_[starts, len(row)])
    return list(zip(row[starts].tolist(), lengths.tolist()))

print(rle(np.array([0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1])))  # [(0, 4), (1, 2), (0, 5), (1,
 ↪  1)]
```

RLE 對文件、線條圖和位元平面很有效，對游程很短的雜訊照片則效果不佳。二維版本會以上一列的顏色轉換位置為參考，編碼目前這列的轉換位置，因此也利用了垂直方向的相關性。JPEG 對量化後大量為零的係數也採用一種游程編碼 [13]。

符號式編碼

白話版：一頁掃描文件大多是同樣幾個字母一再出現。每種字形只存一次，然後只要說「第 7 號字母放在這裡」。

在**符號式**（symbol-based，或稱 token-based）編碼中，經常出現的子影像稱為**符號**，只在**符號字典**中存一次。影像變成一串三元組 (x_i, y_i, t_i)：每次出現的位置，以及它在字典中的編號 t_i。一頁有上千個字元、卻只有約一百種字形的文字，壓縮效果非常好，因為每次出現只花兩個座標和一個編號，這些再以熵編碼處理。

如果允許出現的圖樣與字典項目只是「相似」就算匹配，方法就變成有失真。這能提高壓縮比，但粗心的匹配器可能換錯字元，例如在低解析度掃描中把兩個長得像的數字互換。穩健的系統會把匹配門檻設得保守，或另外編碼圖樣與字典項目之間的殘差，以維持無失真。

位元平面編碼

白話版：把 8 位元影像拆成八張黑白影像，每個位元一張，再用二值方法分別壓縮。

灰階 $a = a_7 2^7 + a_6 2^6 + \cdots + a_0 2^0$ 有八個位元 $a_7, \ldots, a_0$。把每個像素的第 k 位元收集起來，就是**位元平面** k（圖 8.3）。高位元平面看起來像影像的粗略版本，有大片均勻區域；低位元平面看起來像雜訊。每個平面都可以用游程或其他二值方法編碼。

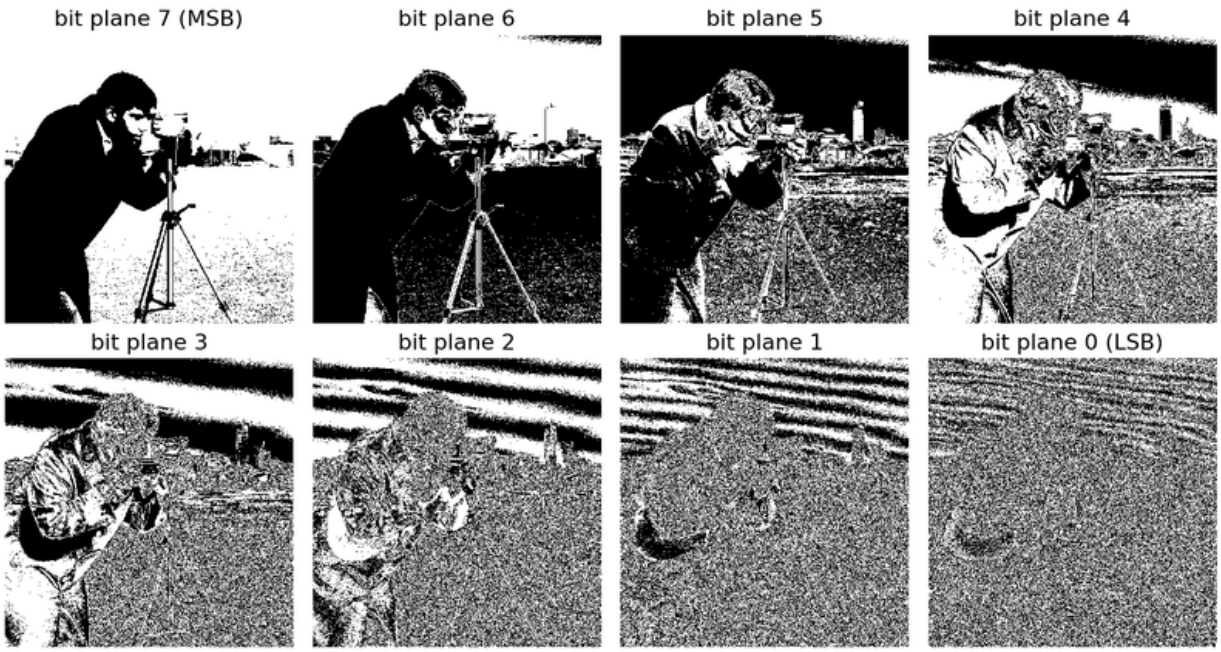


圖 *8.3 — 8* 位元影像的位元平面。結構集中在高位元平面，低位元平面接近隨機。

一般二進位碼的缺點是灰階的微小變化可能翻轉許多位元：127 是 `01111111`，128 是 `10000000`。**格雷碼**（Gray code）讓相鄰的值只差一個位元：

$$g_7 = a_7, \qquad g_i = a_i \oplus a_{i+1}, \quad 0 \le i \le 6,$$

其中 $\oplus$ 是互斥或（XOR）。以格雷碼拆出的平面在平滑區域會有更長的游程。以 camera 影像為例，位元平面 6 在水平相鄰像素間改變值的比例是 6.4%，對應的格雷碼平面只有 3.2%，平均游程約長一倍。在 NumPy 中轉換只要一行：`gray = img ^ (img >> 1)`。

JPEG 2000 也是逐個位元平面、由最高位往下編碼小波係數，這讓它的位元流可以在任何位元率截斷 [14]。

區塊轉換編碼

白話版：把影像切成小磚，每塊用一組標準圖樣（從平滑到細碎）的組合來描述，保留重要的圖樣，丟掉人眼幾乎看不出的細碎圖樣。

區塊轉換編碼器把影像切成不重疊的 $n \times n$ 子影像，對每塊做轉換、量化係數，再做熵編碼。轉換本身不會壓縮；它把大部分能量**集中**到少數係數，讓其他係數可以粗略量化或直接丟掉。

轉換的選擇

對 $n \times n$ 區塊 $g(x,y)$，二維**離散餘弦轉換**（DCT）為

$$T(u,v) = \alpha(u)\alpha(v) \sum_{x=0}^{n-1} \sum_{y=0}^{n-1} g(x,y) \cos \frac{(2x+1)u\pi}{2n} \cos \frac{(2y+1)v\pi}{2n},$$

其中 $\alpha(0) = \sqrt{1/n}$，$u = 1, \dots, n-1$ 時 $\alpha(u) = \sqrt{2/n}$；u, v 是水平與垂直頻率的索引。Karhunen–Loève 轉換（KLT，見第 7 章）在均方意義下集中能量的效果最好，但它的基底取決於資料，必須另外計算並傳送。DCT 是固定的、有快速演算法，而且對高度相關的影像，它的基底很接近 KLT。在區塊上它也勝過 DFT：DFT 把區塊視為週期函數，左右邊界的落差會產生假的高頻；DCT 隱含的偶對稱延伸則沒有這種落差。

子影像大小

較大的區塊能涵蓋更多相關性、集中能量的效果更好，但計算量更大，誤差也會擴散到更大的範圍。超過約 8×8 或 16×16 後效益很快趨緩。JPEG 固定使用 8×8；許多較新的編解碼器會在多種區塊大小間選擇，平滑區用大區塊，細節處用小區塊。

位元分配

轉換之後，要決定每個係數分到多少位元。

- **區域編碼**（zonal coding）在每個區塊都保留固定區域的係數（通常是變異量最大的低頻）。
- **門檻編碼**（threshold coding）在每個區塊保留超過門檻的係數，保留哪些係數會隨內容改變。

門檻編碼常見的實作是把每個係數除以**正規化（量化）陣列** Z 的對應元素再四捨五入：

$$\hat{T}(u,v) = \operatorname{round}\left[\frac{T(u,v)}{Z(u,v)}\right],$$

$\hat{T}(u,v)$ 就是實際被編碼的整數。解碼器再乘回去，$\hat{T}(u,v)\,Z(u,v)$，然後做反 DCT。Z 在高頻的大數值丟掉細節，在低頻的小數值保留粗略結構。

JPEG baseline

8 位元灰階的 JPEG baseline 編碼器 [2][13] 正是這樣做：

1. 像素值減 128，使其以 0 為中心。
2. 對每個區塊做 8×8 DCT。

3. 用 Z 表量化。標準提供一張亮度範例表；常見的慣例是以品質因子 $q \in [1,100]$ 縮放它：$q < 50$ 時縮放比例 $s = 5000/q$，否則 $s = 200 - 2q$（以百分比計）。
4. DC 係數 $\hat{T}(0,0)$ 以與前一區塊 DC 的差值編碼。
5. 63 個 AC 係數以**之字形**（zigzag）由低頻排到高頻，讓零集中在後段，再以（零的游程, 非零值）配對及霍夫曼表編碼，每個區塊以區塊結束符號收尾。

圖 8.4 追蹤一個暗而平滑的區塊在 $q = 50$ 時經過步驟 2–3 的過程。它的 DC 項很大（−603，來自位準平移），另有少數低頻項。量化後 64 個整數只剩 6 個非零，重建結果卻只差幾個灰階。

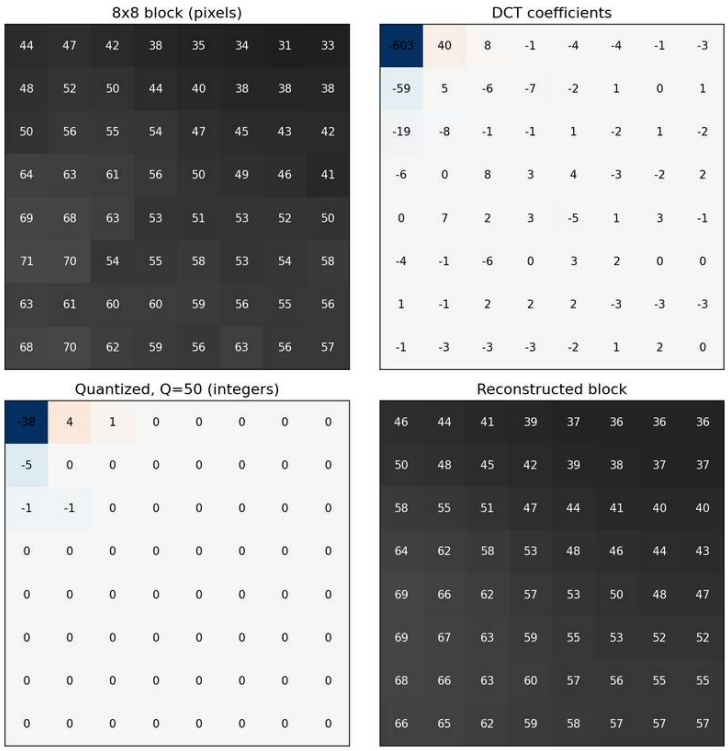


圖 *8.4* — 一個 *8×8* 區塊在品質 *50* 下經過 *JPEG* 流程。量化把 *64* 個係數中的 *58* 個變成零。

圖 8.5 把同樣的流程（不含熵編碼）套用到整張影像、四種品質設定。PSNR 從 $q = 90$ 的 40.3 dB 降到 $q = 5$ 的 26.3 dB，非零係數的比例則從 31.3% 降到 2.4%。低品質時會出現熟悉的瑕疵：**方塊效應**（blocking，各區塊獨立編碼，8×8 的接縫清晰可見）以及邊緣附近的**振鈴**（ringing，形成銳利邊緣所需的高頻被去掉了）。

圖 *8.5* — 不同品質設定下的 *JPEG* 式 *DCT* 量化。$q = 5$ 時方塊效應非常明顯。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

```
import numpy as np
from scipy.fft import dctn, idctn

Q50 = np.array([[16,11,10,16,24,40,51,61],[12,12,14,19,26,58,60,55],
                [14,13,16,24,40,57,69,56],[14,17,22,29,51,87,80,62],
                [18,22,37,56,68,109,103,77],[24,35,55,64,81,104,113,92],
                [49,64,78,87,103,121,120,101],[72,92,95,98,112,100,103,99]])

def dct_codec(img, Q):
    f = img.astype(float) - 128                       # level shift
    H, W = f.shape
    B = f.reshape(H//8, 8, W//8, 8).swapaxes(1, 2)   # (H/8, W/8, 8, 8) blocks
    q = np.round(dctn(B, axes=(2, 3), norm="ortho") / Q)
    rec = idctn(q * Q, axes=(2, 3), norm="ortho")
    rec = rec.swapaxes(1, 2).reshape(H, W) + 128
    return np.clip(rec.round(), 0, 255).astype(np.uint8), q

rec, q = dct_codec(img, Q50)     # PSNR 32.6 dB, 12.0% nonzero coefficients
```

拖動下方的滑桿，比較一張彩色測試影像的無失真 PNG（465 kB）與品質 4 的真實 JPEG（7.6 kB，約小 60 倍）。注意平坦的背景、臉部和色彩邊緣：方塊、振鈴與顏色暈開（色彩通道以較低解析度儲存，量化也更粗）都看得到。

原圖（*PNG*）

JPEG q=4

太空人測試影像：無失真與重度量化 JPEG 的比較

預測編碼

白話版：用已經送出的像素猜下一個像素，只傳「猜錯多少」。猜得準，數字就小；數字小，就便宜。

無失真預測編碼

編碼器由先前已編碼的像素算出像素 $f(n)$ 的預測值 $\hat{f}(n)$，並編碼**預測誤差**

$$e(n) = f(n) - \hat{f}(n), \qquad \hat{f}(n) = \operatorname{round}\left[\sum_{i=1}^{m} \alpha_i f(n-i)\right],$$

其中 m 是預測器的階數，α_i 是預測係數。解碼器算出同樣的預測值再加回 $e(n)$，所以重建完全精確。圖 8.1 已經展示了效果：最簡單的「等於左鄰像素」預測器就把熵從 7.23 降到 4.70 位元／像素。

PNG 讓每一列從五種濾波器中選擇：不處理、左（Sub）、上（Up）、左與上的平均，以及 Paeth 預測器 [4]。JPEG-LS [10] 使用**中位數邊緣偵測器**（median edge detector）：令 a 為左鄰、b 為上鄰、c 為左上鄰，

$$\hat{f} = \begin{cases} \min(a,b) & c \geq \max(a,b) \\ \max(a,b) & c \leq \min(a,b) \\ a + b - c & \text{otherwise.} \end{cases}$$

前兩種情況在水平或垂直邊緣處選擇位於正確一側的鄰居；第三種則假設局部是平滑平面。

運動補償

視訊多了時間軸。**運動補償**（motion compensation）預測器不從同一畫格的鄰居預測，而是從先前解碼的畫格複製一個區塊，並依**運動向量** (d_x, d_y) 位移；運動向量透過搜尋最佳匹配（例如絕對差值總和最小）求得。編碼器傳送運動向量和殘差，殘差通常再以上一節的轉換編碼處理。只用自身編碼的畫格（intra frame）提供隨機存取的入口；預測畫格則小得多。HEVC（其靜態影像形式存於 HEIC 檔 [7]）與 AV1（AVIF 所用 [6]）都結合了這類畫格間預測與轉換編碼。

有失真預測編碼（DPCM）

若把預測誤差量化，$\dot{e}(n) = Q[e(n)]$，就得到**差分脈衝碼調變**（differential pulse-code modulation, DPCM）。關鍵細節是：編碼器必須以重建值 $\dot{f}(n) = \dot{e}(n) + \hat{f}(n)$ 做預測，也就是解碼器會擁有的那些值。如果編碼器改用原始值預測，量化誤差會在解碼器中累積（漂移，drift）。

```
import numpy as np

def dpcm(row, step):
    """Lossy DPCM with previous-sample prediction and a uniform quantizer."""
    rec = np.empty(len(row)); prev = 128.0; idx = []
    for i, x in enumerate(row.astype(float)):
        e = x - prev                      # prediction error
        k = int(np.round(e / step))       # quantizer index: this is what is sent
        prev = prev + k * step            # reconstruction, identical in the decoder
        rec[i] = prev; idx.append(k)
    return np.array(idx), rec
```

在 camera 影像的一列上以步長 8 執行，量化索引的熵是 1.87 位元／樣本，重建的 PSNR 為 40.8 dB。量化很粗時會出現兩種典型失真：**斜率過載**（slope overload，陡峭邊緣處重建值爬升得不夠快）以及**顆粒雜訊**（granular noise，平坦區域來回抖動）。

最佳預測器與量化器

最佳線性預測器選擇 $\alpha_1, \dots, \alpha_m$ 使均方預測誤差 $E\{e^2(n)\}$ 最小。令導數為零得到正規方程式

$$\mathbf{R}\,\boldsymbol{\alpha} = \mathbf{r}, \qquad \boldsymbol{\alpha} = \mathbf{R}^{-1}\mathbf{r},$$

其中 $\mathbf{R}$ 是 $m \times m$ 自相關矩陣，元素為 $E\{f(n-i)f(n-j)\}$；$\mathbf{r}$ 是元素為 $E\{f(n)f(n-i)\}$ 的向量。（這個分析通常忽略迴路中的量化器，量化夠細時這樣近似很準確。）

最佳量化器在給定層數下，使誤差分布的均方量化誤差最小。它有兩個條件：每個判定邊界位於相鄰兩個重建值的正中間；每個重建值是其區間內機率質量的重心（條件平均）。由於預測誤差集中在零附近，最佳的重建值在零附近密、在尾端疏。實務上，均勻量化器加上變長碼幾乎一樣好，而且簡單得多。

小波編碼

白話版：不把影像切成磚塊，而是拆成一張模糊的小圖，加上好幾層不同尺度的細節。大多數細節值接近零，儲存成本低，而且沒有磚塊接縫。

小波編碼器先做第 7 章的離散小波轉換，再量化係數並做熵編碼。因為小波基底函數彼此重疊、在多個尺度上局部化，小波編碼器沒有區塊邊界，失真表現為模糊與輕微振鈴，而不是方塊。圖 8.6 比較 8×8 DCT 與五層小波轉換，兩者都只保留同樣 2% 的係數（絕對值最大者）。DCT 的結果是一片馬賽克，PSNR 26.8 dB；小波的結果較平滑，達到 28.6 dB。

圖 *8.6* — 同樣的係數預算（2%），不同的轉換。區塊 *DCT* 產生接縫，小波轉換則沒有。

```
import numpy as np, pywt

def wavelet_code(img, keep=0.02, wavelet="bior4.4", level=5):
    c = pywt.wavedec2(img.astype(float), wavelet, level=level, mode="periodization")
    arr, slices = pywt.coeffs_to_array(c)
    t = np.quantile(np.abs(arr), 1 - keep)            # keep the largest 2%
    arr[np.abs(arr) < t] = 0
    c = pywt.array_to_coeffs(arr, slices, output_format="wavedec2")
    return np.clip(pywt.waverec2(c, wavelet, mode="periodization"), 0, 255)
```

這個簡化範例算的是係數數量，不是位元數。真正的編碼器還必須告訴解碼器哪些係數留下來，而聰明的顯著性編碼正是在這裡發揮作用。

小波的選擇

小波決定了能量集中的程度與瑕疵的樣貌。影像編碼器偏好**雙正交**（biorthogonal）小波，因為它們可以是對稱的（邊緣不會位移），而且濾波器短。JPEG 2000 規定了兩種：有失真編碼用不可逆的 9/7 抽頭濾波器組，無失真編碼用可逆的整數 5/3 濾波器組 [14]。上面使用的 PyWavelets `bior4.4` 也有同樣的 9 與 7 抽頭長度。

分解層數

每一層把目前的近似子帶拆成四個一半大小的子帶。層數越多表示法越稀疏，但對常見的影像尺寸，超過約五層後效益遞減，而且量化非常粗的係數會影響大片區域。

量化器設計

小波編碼器通常使用在零附近有加大**死區**（dead zone）的均勻純量量化器：

$$q_b = \operatorname{sign}(c_b)\left\lfloor \frac{|c_b|}{\Delta_b} \right\rfloor,$$

其中 c_b 是子帶 b 中的係數，Δ_b 是該子帶的步長，q_b 是整數索引。小係數（多半是雜訊）會落入零區間。步長依子帶選擇，高頻子帶的誤差比較不明顯，所以步長較大。

JPEG 2000

JPEG 2000 [3][14] 把這些元件組成完整的系統。影像可以切成大塊的 tile，色彩分量先經過分量轉換去相關，每個 tile 做小波轉換與死區量化。每個子帶再切成小的**編碼區塊**（code block），各自獨立地逐位元平面、以自適應算術編碼器編碼。最後，編碼後的片段被組織成**品質層**，同一個檔案只要讀取位元流的前段，就能以較低解析度或較低品質解碼。它也支援無失真編碼，以及以較高品質編碼的感興趣區域（region of interest）。

數位影像浮水印

白話版：浮水印是藏在影像裡、跟著影像一起流傳的額外資訊，例如簽名、擁有者 ID，或「這是模型生成的」標記。

浮水印的種類

- **可見**浮水印是疊加的圖案（半透明商標），會降低畫質，也常可被裁掉或塗掉。**不可見**浮水印的改變低於可見門檻，靠偵測器取回。
- **強健**（robust）浮水印能撐過壓縮、縮放、濾波與裁切（用於著作權與來源追溯）；**脆弱**（fragile）浮水印任何改動都會破壞（用於真偽驗證）；**半脆弱**浮水印能撐過輕度壓縮，但撐不過內容修改。
- **私有**（非盲）偵測需要原始影像；**盲**偵測則不需要。

每種方案都要在**不可察覺性**（影像改變多小）、**強健性**、**容量**（藏多少位元）與**安全性**（沒有金鑰的人能否找到、移除或偽造浮水印）之間取捨。

脆弱範例：最低有效位元

最簡單的不可見浮水印是把每個像素的最低有效位元（least significant bit, LSB）換成浮水印的一個位元：$f_w = 2\lfloor f/2 \rfloor + w$，$w \in \{0, 1\}$。每個像素最多只改變一個灰階，所以 PSNR 約 51 dB。

```
def lsb_embed(cover, mark):          # cover: uint8 image, mark: 0/1 array of the same shape
    return (cover & 0xFE) | mark

def lsb_extract(img):
    return img & 1
```

圖 8.7 嵌入了一個二值的「DIP」標誌。從 PNG 取回時完全正確；但只要經過一次品質 95 的 JPEG 壓縮（肉眼看不出差別），取回的位元只有 55% 與浮水印相符，幾乎等於擲銅板。LSB 平面正是影像中像雜訊的部分，也是每個有失真編碼器最先丟掉的部分。因此 LSB 浮水印不適合宣示著作權，但可以當作脆弱的完整性檢查。

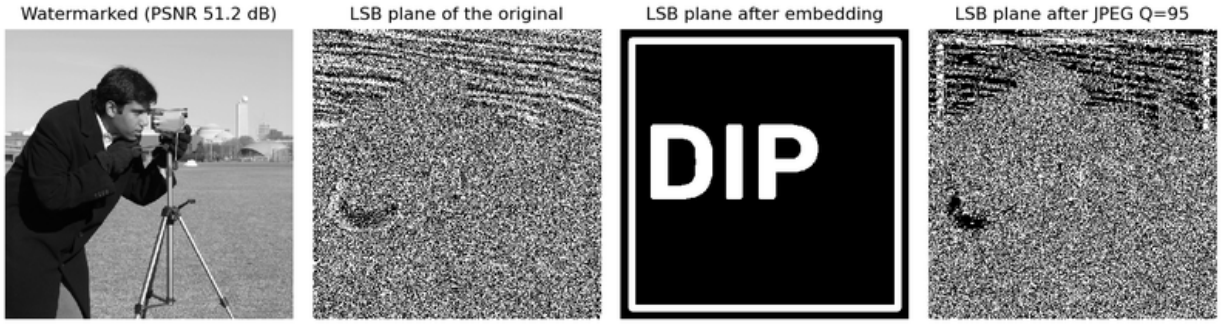


圖 *8.7 — LSB* 浮水印看不出來（*51 dB*）且能完整取回，直到一次輕度 *JPEG* 就把它抹除。

強健範例：DCT 域浮水印

要撐過 JPEG，就把浮水印藏在 JPEG 會保留資訊的地方：中低頻 DCT 係數。下面是我們自己設計的簡單方案。在每個 8×8 區塊中比較兩個中頻係數 $C(2,3)$ 與 $C(3,2)$，JPEG 對它們的量化步長相近。要嵌入位元 1，就讓 $C(2,3)$ 比 $C(3,2)$ 至少大 α；嵌入位元 0 則相反。偵測器只檢查哪個係數較大，因此是盲偵測。

```
import numpy as np
from scipy.fft import dctn, idctn

def blocks(img):
    H, W = img.shape
    return img.astype(float).reshape(H//8, 8, W//8, 8).swapaxes(1, 2).reshape(-1, 8, 8)

def unblocks(B, H, W):
    return B.reshape(H//8, W//8, 8, 8).swapaxes(1, 2).reshape(H, W)

def dct_embed(img, bits, alpha=12.0):
    C = dctn(blocks(img), axes=(1, 2), norm="ortho")
    for i, b in enumerate(bits):              # one bit per 8x8 block
        x, y = C[i, 2, 3], C[i, 3, 2]
        if (x - y if b else y - x) < alpha:    # enforce the order with margin alpha
            m = (x + y) / 2
            s = alpha / 2 if b else -alpha / 2
            C[i, 2, 3], C[i, 3, 2] = m + s, m - s
    out = unblocks(idctn(C, axes=(1, 2), norm="ortho"), *img.shape)
    return np.clip(out.round(), 0, 255).astype(np.uint8)

def dct_extract(img):
    C = dctn(blocks(img), axes=(1, 2), norm="ortho")
    return (C[:, 2, 3] > C[:, 3, 2]).astype(int)
```

在 512×512 的 camera 影像上，這個方案藏了 4,096 個位元（每個區塊一個）。邊距 α 就是調整強健性的旋鈕：

邊距 α	加浮水印後 PSNR	JPEG q=90 後位元錯誤率	q=75	q=50
12	43.2 dB	0.10%	0.56%	36.6%
30	38.4 dB	0%	0%	0.05%

小邊距幾乎看不出來，但當 JPEG 在這些位置的步長超過邊距時就會失效。大邊距能撐過品質 50，代價是 PSNR 少了約 5 dB。實際系統還會加上錯誤更正碼、用秘密金鑰把每個位元分散到多個區塊，並依局部紋理調整強度，因為人眼在紋理複雜處能容忍較大的改變。

現代觀點

經典流程（轉換、量化、熵編碼）至今仍是所有實際部署的影像編解碼器的基礎。大約 2016 年以後的改變是：每個階段都可以被學習，而同樣的「訓練編碼器與解碼器」想法也重塑了浮水印。

值得一讀的綜述

- **Wallace（1992）**[13] 仍是 JPEG 設計最清楚的概述：DCT、量化表、之字形排序，以及霍夫曼與算術編碼兩種選項。適合搭配區塊轉換編碼一節閱讀。
- **Skodras、Christopoulos 與 Ebrahimi（2001）**[14] 是 JPEG 2000 的教學型回顧：tile 切割、9/7 與 5/3 小波、死區量化、編碼區塊的位元平面編碼與位元率控制。重點是：JPEG 2000 的價值與其說在單純的壓縮效率，不如說在功能——可分級性、感興趣區域編碼，以及在同一個位元流中支援無失真。
- **Hu、Yang、Ma 與 Liu（2021）**[16] 沿著網路架構、熵模型與位元率控制三個軸向綜述端到端學習式有失真影像壓縮，整理各自的里程碑，並在 CPU 與 GPU 上評測代表性方法。他們自己提出的由粗到細超先驗（coarse-to-fine hyperprior）也呼應了綜述的主軸：許多進展在於熵模型能多準確地預測潛在表示。
- **Yang、Mandt 與 Theis（2023）**[17] 是一本專書篇幅的神經壓縮入門，寫給機器學習讀者。它先回顧所需的資訊理論（熵編碼、率失真理論）以及影像品質與感知度量，再帶讀者認識以正規化流、變分自編碼器、擴散模型與 GAN 建構的編解碼器。
- **Zhong、Das、Alrasheedi 與 Tanvir（2023）**[18] 綜述深度學習影像浮水印，將方法分成三類：聯合訓練的嵌入器—擷取器網路、把網路當作特徵轉換再搭配傳統嵌入規則，以及兩者混合。

學習式影像壓縮

Ballé 等人 [15] 以學習得到的非線性**分析轉換**（卷積神經網路）取代 DCT，把影像 x 映射到潛在表示 y，量化成 $\hat{y}$ 後再由學習得到的**合成轉換**解碼。整個系統以率失真損失訓練：

$$\mathcal{L} = R + \lambda D = E\left[-\log_2 p(\hat{y})\right] + \lambda\, E\left[d(x, \hat{x})\right],$$

其中 R 是在熵模型 p 下的期望碼長，D 是失真（例如 MSE），λ 決定兩者的取捨。他們的關鍵貢獻是**尺度超先驗**（scale hyperprior）：額外傳送一個小的潛在變數 z 作為旁資訊，用來預測 y 每個元素的尺度，讓熵模型隨每張影像調整。這和 JPEG 每張影像專屬的霍夫曼表、或 JPEG-LS 的情境建模是同一個想法，只是改成端到端學習。訓練時以加性均勻雜訊取代四捨五入，讓梯度能夠傳遞。

沒有改變的部分：量化仍是四捨五入，位元仍由算術編碼器 [11] 產生。學習式編解碼器可以理解為同一張編碼器方塊圖中更好的映射器與更好的機率模型。它們在實務上的障礙是解碼器的運算量，以及熵模型需要在各平台上逐位元一致。

現代標準編解碼器

同一時期，標準化的編解碼器也持續以人工設計的工具進步。AVIF [6] 把 AV1 的畫格內編碼存進 HEIF 容器，HEIC 檔 [7] 則把 HEVC 的畫格內編碼存進同一系列的容器。兩者都沿用視訊編解碼器的畫格內（單張畫面）編碼工具，讓靜態影像直接受惠於數十年的視訊編碼研究。JPEG XL [5] 鎖定攝影與網頁，支援有失真與無失真編碼、高動態範圍與透明通道，還能把現有 JPEG 檔無損轉存成更小的檔案，並可還原成完全相同的原始位元組。PNG [4] 仍是無失真圖形的首選；第三版（2025）加入 HDR 標示並把動態 PNG 納入標準。這些格式沒有取代教科書的觀念，而是更彈性地組合預測編碼、區塊轉換與自適應熵編碼。

深度學習浮水印

HiDDeN [19] 聯合訓練三個部分：把位元串藏進封面影像的編碼器、取回位元的解碼器，以及中間模擬裁切、模糊與 JPEG 的**雜訊層**（真正的 JPEG 無法微分，因此訓練時使用可微分的近似）。對抗損失讓加浮水印的影像保持自然。強健性是從模擬的攻擊中學來的，而不像前面那樣靠挑選係數來設計，因此它的強健程度取決於模擬得多好。

生成式模型帶來新的用途：在影像誕生時就加上標記。Stable Signature [20] 微調潛在擴散模型的解碼器，讓它生成的每張影像都帶有固定的隱藏簽章；即使經過裁切等編輯，擷取器仍能取回，誤報率低於百萬分之一。它回答的是來源問題（這張圖是不是模型 X 產生的？），而不是某張照片的著作權。移除與偽造攻擊仍是開放問題。

重點整理

- 壓縮去除編碼冗餘、空間／時間冗餘與不相關資訊；只有最後一種是有失真的。
- 熵是獨立符號無失真編碼的下限，而符號的選擇很重要：預測差值的熵遠低於原始像素。
- 霍夫曼碼是逐符號最佳；算術編碼逼近整段訊息的熵；Golomb、LZW、游程、符號式與位元平面編碼各自利用特定結構。
- JPEG ＝位準平移＋ 8×8 DCT ＋量化表＋之字形＋游程／霍夫曼編碼；品質由縮放量化表決定，低品質時出現方塊與振鈴。
- 預測編碼器必須以重建值預測；運動補償預測是視訊編碼的核心。
- 小波編碼（JPEG 2000）沒有區塊接縫，並支援可分級、無失真與感興趣區域編碼。
- LSB 浮水印看不見但很脆弱；DCT 域浮水印只要強度超過量化步長，就能撐過 JPEG。

- 學習式編解碼器與深度浮水印用訓練出的網路取代人工設計的轉換，但核心仍是量化與算術編碼。

練習

1. 某信源有四個符號，機率分別為 0.5、0.25、0.125、0.125。建出霍夫曼碼，計算平均碼長與熵，並解釋為什麼兩者在這裡相等。

   提示

   碼字為 `0`、`10`、`110`、`111`；兩個值都是 1.75 位元。所有機率都是 1/2 的冪次，所以 $-\log_2 P(a_j)$ 是整數，每個碼字都能恰好是那個長度。

2. 預測殘差 $e = 0, -1, 2, -3, 1$ 先用 Golomb 一節的 $M(e)$ 映射，再用 G_2 編碼。寫出位元串並計算位元數。改用 G_4 會比較短嗎？

   提示

   $M(e) = 0, 1, 4, 5, 2$。$m = 2$ 時碼為 `00`、`01`、`1100`、`1101`、`100`（15 位元）；$m = 4$ 時為 `000`、`001`、`1000`、`1001`、`010`（17 位元）。殘差小時，較小的參數勝出。

3. 使用算術編碼範例的機率（$P(a) = 0.6$、$P(b) = 0.3$、$P(c) = 0.1$），解出標記值 0.70 的前三個符號。

   提示

   0.70 落在 $[0.6, 0.9)$，第一個符號是 `b`。重新縮放：$(0.70 - 0.6)/0.3 = 0.333$，落在 a 的範圍。再縮放：$0.333/0.6 = 0.556$，又是 `a`。訊息開頭為 `baa`。

4. 修改 DPCM 程式，讓編碼器以原始的前一像素預測，而不是重建值，但解碼器不變。測量一列的 PSNR 並解釋看到的現象。

   提示

   解碼器把量化誤差加到自己的重建值上，而這個參考值已和編碼器不同。誤差沿著該列累積（漂移）。在 camera 影像第 200 列、步長 8 的情況下，PSNR 從 40.8 dB 掉到約 17 dB。

5. 把 LSB 浮水印改用位元平面 3 而不是位元平面 0。測量加浮水印後的 PSNR，以及經品質 95 與 75 的 JPEG 後位元相符率。你得到什麼、又付出了什麼？

   提示

   每個像素現在最多改變 8 個灰階，PSNR 大約下降 $20 \log_{10} 8 \approx 18$ dB（隨機浮水印時約 33 dB）。JPEG 後的相符率在品質 95 時升到約 82%，在品質 75 時卻掉到約 57%：和 DCT 域方案相比，你付出了大量可見度，卻只換到一點強健性。

6. 從量化後留下的基底函數的支撐範圍，解釋為什麼小波編碼器在極低位元率時看起來模糊，而區塊 DCT 編碼器看起來是方塊。

提示

留下的 DCT 基底函數被限制在各自的 8×8 區塊內，相鄰區塊各自近似，交界處形成接縫。留下的粗尺度小波函數平滑且與鄰居重疊，所以近似結果連續，只是缺少細節。

參考文獻

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 8. publisher page
2. Joint Photographic Experts Group, "JPEG 1 (ISO/IEC 10918-1 | ITU-T Rec. T.81): Digital compression and coding of continuous-tone still images," JPEG overview page. jpeg.org
3. Joint Photographic Experts Group, "JPEG 2000 (ISO/IEC 15444)," JPEG overview page. jpeg.org
4. W3C, "Portable Network Graphics (PNG) Specification (Third Edition)," W3C Recommendation, 2025. w3.org
5. Joint Photographic Experts Group, "JPEG XL (ISO/IEC 18181)," JPEG overview page. jpeg.org
6. Alliance for Open Media, "AV1 Image File Format (AVIF)," v1.2.0, 2025. aomedia-codec.github.io
7. Nokia Technologies, "High Efficiency Image File Format (HEIF), ISO/IEC 23008-12: technical information." nokiatech.github.io
8. D. A. Huffman, "A Method for the Construction of Minimum-Redundancy Codes," *Proceedings of the IRE*, vol. 40, pp. 1098–1101, 1952. doi
9. S. W. Golomb, "Run-length encodings," *IEEE Transactions on Information Theory*, vol. 12, pp. 399–401, 1966. doi
10. M. J. Weinberger, G. Seroussi and G. Sapiro, "The LOCO-I lossless image compression algorithm: principles and standardization into JPEG-LS," *IEEE Transactions on Image Processing*, vol. 9, pp. 1309–1324, 2000. doi
11. I. H. Witten, R. M. Neal and J. G. Cleary, "Arithmetic coding for data compression," *Communications of the ACM*, vol. 30, pp. 520–540, 1987. doi
12. T. A. Welch, "A Technique for High-Performance Data Compression," *Computer*, vol. 17, pp. 8–19, 1984. doi
13. G. K. Wallace, "The JPEG still picture compression standard," *IEEE Transactions on Consumer Electronics*, vol. 38, no. 1, pp. xviii–xxxiv, 1992. doi
14. A. Skodras, C. Christopoulos and T. Ebrahimi, "The JPEG 2000 still image compression standard," *IEEE Signal Processing Magazine*, vol. 18, pp. 36–58, 2001. doi
15. J. Ballé, D. Minnen, S. Singh, S. J. Hwang and N. Johnston, "Variational image compression with a scale hyperprior," *ICLR*, 2018. arXiv
16. Y. Hu, W. Yang, Z. Ma and J. Liu, "Learning End-to-End Lossy Image Compression: A Benchmark," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021. arXiv
17. Y. Yang, S. Mandt and L. Theis, "An Introduction to Neural Data Compression," *Foundations and Trends in Computer Graphics and Vision*, vol. 15, no. 2, 2023. arXiv

18. X. Zhong, A. Das, F. Alrasheedi and A. Tanvir, “A Brief Yet In-Depth Survey of Deep Learning-Based Image Watermarking,” *Applied Sciences*, 2023. arXiv
19. J. Zhu, R. Kaplan, J. Johnson and L. Fei-Fei, “HiDDeN: Hiding Data With Deep Networks,” *arXiv:1807.09937*, 2018. arXiv
20. P. Fernandez, G. Couairon, H. Jégou, M. Douze and T. Furon, “The Stable Signature: Rooting Watermarks in Latent Diffusion Models,” *ICCV*, 2023. arXiv

IP 09・形態學影像處理

用小小的結構元素去「探測」形狀：侵蝕、膨脹、開運算、閉運算、擊中擊不中轉換、骨架、形態學重建與灰階形態學，附 scikit-image 程式碼。

網站版本: https://blog.shyandram.dev/zh-tw/learn/image-processing/dip-09-morphology/

你將學到

- 如何把二值影像看成像素座標的集合，並用一個小小的**結構元素**（structuring element）去探測它
- 兩個最基本的運算：**侵蝕**（erosion）與**膨脹**（dilation），以及它們組合成的**開運算**（opening）與**閉運算**（closing）
- **擊中擊不中轉換**（hit-or-miss transform）如何找出角點、孤立點這類精確的像素圖樣
- 由這些基本運算組成的經典演算法：邊界、填洞、連通元件、凸包、細化、骨架與修剪
- **形態學重建**（morphological reconstruction）：移除物件，卻不讓留下來的物件變形
- **灰階形態學**：形態學梯度、用頂帽轉換校正不均勻照明，以及粒度測定（granulometry）

先看全貌

前幾章的濾波器大多是線性的：每個輸出像素都是輸入像素的加權和。形態學不一樣，它問的是關於**形狀**的是非題：「這個小形狀放得進這裡嗎？」「它有碰到這裡的東西嗎？」所有答案只靠最小值、最大值、交集和聯集算出來，完全不需要乘法。

> **用最簡單的話說**
>
> 想像桌上有一片形狀歪七扭八的餅乾麵團，還有一個圓形的小餅乾模。把模子在桌上到處滑動。**侵蝕**是標記「模子可以整個壓在麵團上」的位置，所以麵團會變小，細細的部分會消失。**膨脹**是標記「模子有碰到任何麵團」的位置，所以麵團會變大，小縫隙會被補起來。**開運算**只留下真的能壓出餅乾的那些麵團；**閉運算**則把模子戳不進去的小凹陷全部填平。換一個模子形狀（線段、方塊、十字），就是對麵團問另一個問題。

為什麼重要：形態學是清理二值遮罩（不論來自第 10 章的閾值化，或來自神經網路）的標準工具，也用來量測物件（計數、量大小、找骨架）以及校正不均勻的背景。它的理論基礎**數學形態學**（mathematical morphology）源自 Georges Matheron 與 Jean Serra 對隨機集合與多孔材料幾何的研究 [2]、[3]。Haralick、Sternberg 與 Zhuang 的教學論文把它推廣到整個影像分析社群 [4]，而 Soille 的書至今仍是最實用的標準參考 [5]。本章依照 Gonzalez 與 Woods [1] 的主題順序，程式碼使用 scikit-image [12] 與 SciPy。

預備知識

白話說：二值影像就是一串「亮著」的像素位置；結構元素則是一個標了中心點的小模板，我們拿它在這串位置上滑動。

把影像當成集合

設一張二值影像中，前景像素值為 1、背景像素值為 0。我們用下面的集合描述前景：

$$A = \{(x, y) \in \mathbb{Z}^2 \mid I(x, y) = 1\},$$

其中 I 是影像，(x, y) 是整數像素座標。於是集合運算都有了影像上的意義：**補集** A^c 是背景，**聯集** $A \cup C$ 是逐像素 OR，**交集** $A \cap C$ 是逐像素 AND，**差集** $A - C = A \cap C^c$ 則是把 C 的部分從 A 裡拿掉。

結構元素

結構元素（SE）B 是另一個由位移量組成的小集合，並指定一個**原點**（通常在中心）。把它想成探針。常見的選擇有 3×3 方塊、十字（4 鄰域）、圓盤或線段。結構元素的形狀決定了運算會對哪些特徵起反應：圓盤不偏任何方向，水平線段只對水平結構有反應。scikit-image 把結構元素稱為 **footprint**。實作上結構元素就是一個小小的布林陣列，`True` 的位置就是它的成員；圖示中標成「不在乎」的位置，其實就是不屬於這個集合。

反射與平移

接下來每個定義都會用到兩個集合運算。B 的**反射**（reflection）是

$$\hat{B} = \{w \mid w = -b,\ b \in B\},$$

也就是把每個位移 b 換成 $-b$：結構元素繞原點旋轉 180°。B 以向量 $z = (z_1, z_2)$ 做的**平移**（translation）是

$$(B)_z = \{c \mid c = b + z,\ b \in B\},$$

意思是把結構元素的原點放到像素 z 上。圓盤、方塊這類對稱的結構元素滿足 $\hat{B} = B$，反射看不出差別；不對稱的結構元素就要小心。

```
import numpy as np
import Lab from '../../../../components/Lab.astro';
from skimage import morphology

B = morphology.disk(2)            # 5x5 disk; origin at the centre (2, 2)
B_hat = B[::-1, ::-1]             # reflection: flip both axes
L = np.array([[1, 1, 0],
              [0, 1, 0],
              [0, 0, 0]], bool)   # an asymmetric element
print(L[::-1, ::-1].astype(int))  # its reflection points the other way
```

侵蝕與膨脹

白話說：侵蝕只在「整個模板都放得進形狀裡」的地方保留像素；膨脹則只要「模板碰到形狀」就把像素點亮。

侵蝕

A 被 B **侵蝕**的結果是

$$A \ominus B = \{z \mid (B)_z \subseteq A\},$$

也就是平移後的結構元素完全落在前景內的所有位置 z。等價寫法是 $A \ominus B = \{z \mid (B)_z \cap A^c = \emptyset\}$：結構元素不能和任何背景像素重疊。侵蝕會讓物件大約縮小一個結構元素半徑，刪掉比結構元素更細的部分，並把被細橋連在一起的物件切開。若 B 包含原點，侵蝕是**反擴張**（anti-extensive）的：$A \ominus B \subseteq A$。

膨脹

A 被 B **膨脹**的結果是

$$A \oplus B = \{z \mid (\hat{B})_z \cap A \neq \emptyset\},$$

也就是反射後的結構元素至少碰到一個前景像素的所有位置。另一個等價寫法是把 B 的複本放在每個前景像素上再取聯集：$A \oplus B = \bigcup_{a \in A} (B)_a$。膨脹讓物件變大、接起比結構元素窄的縫隙、補上小洞。若 B 包含原點，膨脹是**擴張**（extensive）的：$A \subseteq A \oplus B$。和侵蝕不同，膨脹滿足交換律：$A \oplus B = B \oplus A$。

膨脹公式中的反射，讓膨脹成為真正的 Minkowski 和，也和卷積的「翻轉」一致。對稱結構元素則完全沒差。

對偶性

侵蝕與膨脹在補集與反射之下互為**對偶**（duality）：

$$(A \ominus B)^c = A^c \oplus \hat{B}, \qquad (A \oplus B)^c = A^c \ominus \hat{B}.$$

侵蝕前景，等於用反射後的結構元素膨脹背景，再把結果反過來。證明只要一行：$z \notin A \ominus B$ 恰好表示 $(B)_z$ 碰到 A^c，而這正是 $z \in A^c \oplus \hat{B}$ 的定義（因為 $\hat{\hat{B}} = B$）。寫程式時，必須一致地處理影像邊界，對偶性才會精確成立：對侵蝕而言影像外面要當成前景，對補集的膨脹而言外面才會是背景。

```
import numpy as np
from scipy import ndimage as ndi
from skimage import data, morphology

A = ~data.horse()                 # True = horse pixels
```

```
B = morphology.disk(3)

eroded  = ndi.binary_erosion(A, structure=B)
dilated = ndi.binary_dilation(A, structure=B)
print(A.sum(), eroded.sum(), dilated.sum())     # erosion shrinks, dilation grows

# Duality check. border_value=1 treats outside pixels as foreground for the
# erosion, which matches background-outside for the dilation of ~A.
lhs = ~ndi.binary_erosion(A, structure=B, border_value=1)
rhs = ndi.binary_dilation(~A, structure=B[::-1, ::-1])
print("duality holds:", np.array_equal(lhs, rhs))   # True
```

開運算與閉運算

白話說：開運算是「先侵蝕、再膨脹回來」，能去掉小斑點和細長部分，大形狀則幾乎不變。閉運算是「先膨脹、再侵蝕回來」，能補上小洞和窄裂縫。

定義

A 以 B 做**開運算**，就是用同一個結構元素先侵蝕再膨脹：

$$A \circ B = (A \ominus B) \oplus B,$$

閉運算則是先膨脹再侵蝕：

$$A \bullet B = (A \oplus B) \ominus B.$$

單獨做侵蝕會丟掉細節，也會讓所有東西縮小；第二步則把倖存部分的大小補回來。這兩者也互為對偶：$(A \bullet B)^c = A^c \circ \hat{B}$、$(A \circ B)^c = A^c \bullet \hat{B}$。對前景做閉運算，就是對背景做開運算。

幾何解釋：滾球

開運算有一個很直觀的圖像：它等於所有能放進 A 的 B 平移複本的聯集，

$$A \circ B = \bigcup \{(B)_z \mid (B)_z \subseteq A\}.$$

想像一顆球（結構元素）在前景裡面滾動，而且必須整顆待在裡面。球滾得到的像素都保留；比球還尖的角、細頸和比球小的斑點都會消失。閉運算則是同樣的圖像換到外面：讓球在背景中滾動，凡是球到不了的背景像素（窄縫、小洞、又深又細的凹口）都變成前景。

性質

開運算與閉運算在嚴格意義上都是**形態學濾波器**（morphological filter）：它們是遞增的，而且是冪等的 [5]。

- **開運算是反擴張的：** $A \circ B \subseteq A$，只會移除像素。

- **閉運算是擴張的：** $A \subseteq A \bullet B$，只會加入像素。
- **遞增性：**若 $C \subseteq D$，則 $C \circ B \subseteq D \circ B$，閉運算亦同。輸入變大，輸出絕不會變小。
- **冪等性：** $(A \circ B) \circ B = A \circ B$，$(A \bullet B) \bullet B = A \bullet B$。同樣的開運算做兩次不會再改變任何東西，因為第一次留下來的部分本來就都是球放得進去的地方。

冪等性正是開、閉運算和侵蝕、膨脹的分水嶺：重複侵蝕會一直縮下去，重複開運算則會停下來。圖 9.1 把四種運算套用在一張帶雜訊的剪影上。開運算去掉背景中的白點；閉運算去掉馬身上的黑點；先開再閉則兩種都去掉，這是非常常用的**形態學雜訊濾波器**。

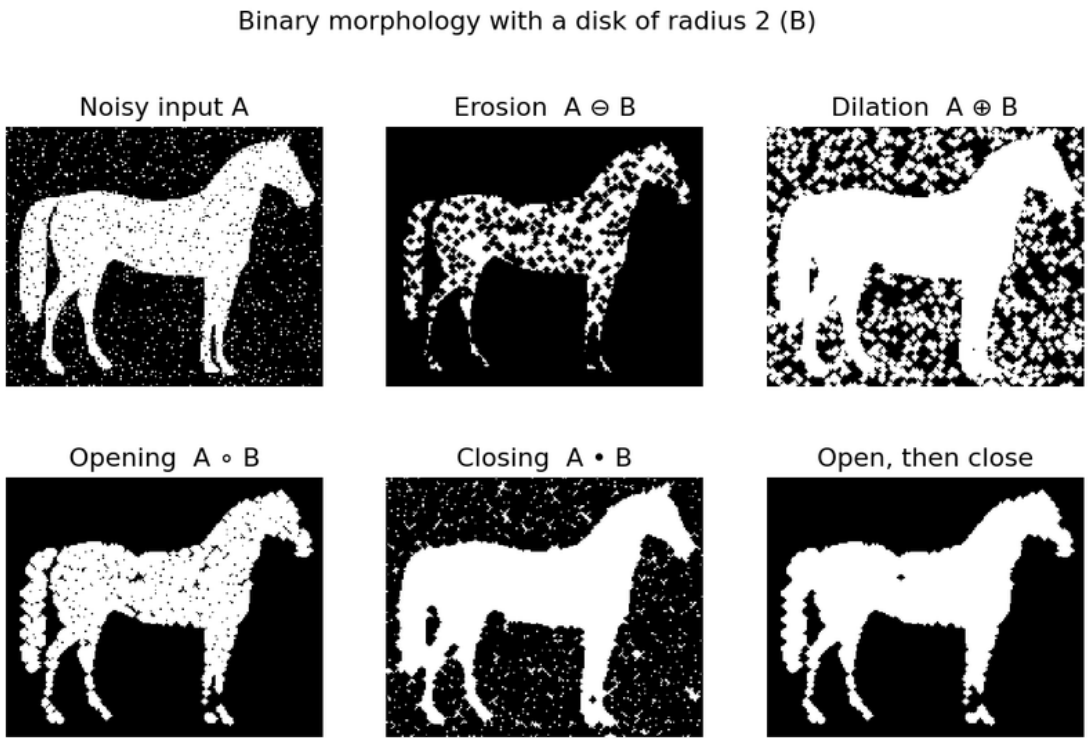


圖 *9.1* — 對翻轉了 *4%* 像素的馬剪影，以半徑 *2* 的圓盤做二值侵蝕、膨脹、開運算與閉運算。先開後閉可以同時去除兩種斑點，並保留輪廓。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

```
import numpy as np
from skimage import data, morphology

A = ~data.horse()
B = morphology.disk(4)
op = morphology.binary_opening(A, B)
cl = morphology.binary_closing(A, B)
print("opening is anti-extensive:", not (op & ~A).any())
print("closing is extensive:     ", not (A & ~cl).any())
print("opening is idempotent:    ", np.array_equal(morphology.binary_opening(op, B), op))
```

擊中擊不中轉換

白話說：擊中擊不中轉換（HMT）就是二值圖樣的模板比對。你指定哪些像素一定要是前景、哪些一定要是背景，它就只在兩個條件同時成立的位置亮起來。

使用兩個不相交的結構元素：B_1 是必須落在前景的像素，B_2 是必須落在背景的像素。A 的 HMT 為

$$A \circledast B_{1,2} = \{z \mid (B_1)_z \subseteq A \text{ and } (B_2)_z \subseteq A^c\} = (A \ominus B_1) \cap (A^c \ominus B_2).$$

其中 $B_{1,2}$ 代表這一對 (B_1, B_2)；$A \ominus B_1$ 找出前景圖樣吻合的位置，$A^c \ominus B_2$ 找出背景圖樣吻合的位置。既不在 B_1 也不在 B_2 的像素是**不在乎**的位置。習慣上會把這一對畫成一個小網格：1 代表必須是前景、0 代表必須是背景、空白代表不在乎。

幾個實用的模板：

- **孤立點：** B_1 是中心像素，B_2 是它的 8 個鄰居。
- **左上角：** B_1 是中心加上右邊與下面的鄰居，B_2 是上面與左邊的鄰居。
- **單像素線段的端點：**中心與恰好一個鄰居是前景，其餘都是背景。每個方向需要一個模板（共 8 個旋轉）。

圖 9.2 套用了前兩種模板。注意數位圓盤的階梯狀輪廓在像素層級上確實有好幾個「左上角」：HMT 是逐字比對，不是幾何意義上的角。

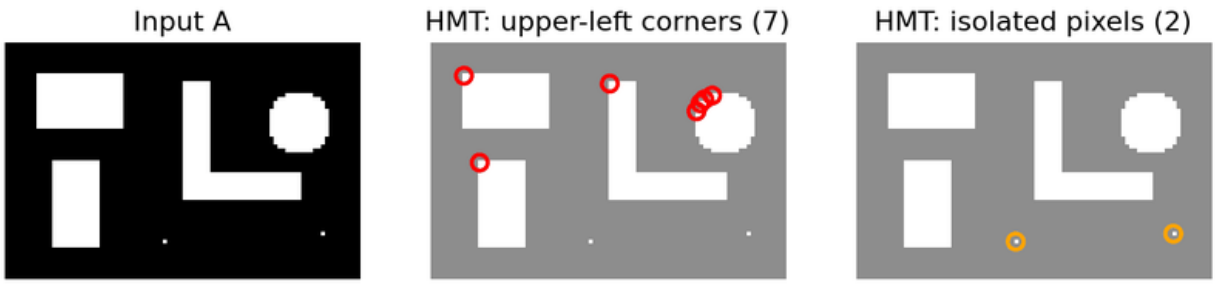


圖 *9.2* — 把擊中擊不中轉換當成精確的圖樣偵測器。中：左上角（包含數位圓盤上的小階梯）。右：孤立的前景像素。

```
import numpy as np
from scipy import ndimage as ndi

A = np.zeros((7, 9), bool)
A[1:5, 1:6] = True                # a rectangle
A[5, 7] = True                    # an isolated pixel
hit  = np.array([[0, 0, 0], [0, 1, 1], [0, 1, 0]], bool)   # must be foreground
miss = np.array([[0, 1, 0], [1, 0, 0], [0, 0, 0]], bool)   # must be background
corners = ndi.binary_hit_or_miss(A, structure1=hit, structure2=miss)
print(np.argwhere(corners))       # [[1 1]]: the upper-left corner only
```

HMT 是下面細化、粗化與凸包演算法的基本積木。

一些基本的形態學演算法

白話說：有了侵蝕、膨脹和 HMT，就能組出許多實用工具。大部分都是一個短迴圈：反覆做同一件事，直到沒有任何變化為止。

邊界擷取

一個集合的內邊界，就是侵蝕會拿掉的部分：

$$\beta(A) = A - (A \ominus B),$$

其中 B 取 3×3 方塊（得到 8 連通的邊界）或十字（得到較粗、4 連通的邊界）。B 越大，邊界越粗。圖 9.3 第一格就是馬的 $\beta(A)$。

填洞

洞（hole）是被前景完全包圍的背景區域。只要在洞裡給一個種子像素，就能用膨脹讓種子長大，但每一步都把它限制在背景內：

$$X_k = (X_{k-1} \oplus B) \cap A^c, \qquad k = 1, 2, 3, \ldots$$

其中 X_0 只含種子，B 是 4 連通的十字，和 A^c 取交集讓生長在洞壁停下。當 $X_k = X_{k-1}$ 時停止，$X_k \cup A$ 就是填好的形狀。這種「先膨脹、再裁切」的模式稱為**條件膨脹**（conditional dilation），後面的測地膨脹會再看到它。B 用十字（而不是方塊）很重要：前景牆是 8 連通時，4 連通的背景填充就不會從斜角漏出去。

```
import numpy as np
from scipy import ndimage as ndi

def fill_hole(A, seed):
    """Grow X from a seed inside one hole: X_k = dilate(X_{k-1}) AND not A."""
    B = np.array([[0, 1, 0], [1, 1, 1], [0, 1, 0]], bool)   # 4-connected cross
    X = np.zeros_like(A); X[seed] = True
    while True:
        X_next = ndi.binary_dilation(X, structure=B) & ~A
        if np.array_equal(X_next, X):
            return A | X
        X = X_next

yy, xx = np.mgrid[-12:13, -12:13]
ring = (np.hypot(yy, xx) <= 10) & (np.hypot(yy, xx) > 6)   # boolean ring
print(ring.sum(), fill_hole(ring, (12, 12)).sum())        # 204 -> 317
```

擷取連通元件

把角色對調：要擷取 A 中包含某個種子像素的連通元件，就讓種子膨脹，但限制在前景內：

$$X_k = (X_{k-1} \oplus B) \cap A.$$

B 取 3×3 方塊時得到種子的 8 連通元件；取十字時得到 4 連通元件。對每個尚未標記的種子重複這個步驟，就能標記所有元件。實際的函式庫使用更快的一次或兩次掃描標記演算法，但結果相同：`scipy.ndimage.label(A, structure=np.ones((3, 3)))` 會傳回標記影像與元件數。

凸包

若一個集合中任兩點之間的線段都留在集合內，它就是**凸**的。**凸包**（convex hull）$C(A)$ 是包含 A 的最小凸集合。純形態學的近似做法使用四個 HMT 模板 B^i（$i = 1, \dots, 4$），各自偵測「某一側（左、上、右、下）有前景」的背景像素。從 $X_0^i = A$ 開始迭代

$$X_k^i = (X_{k-1}^i \circledast B^i) \cup X_{k-1}^i$$

直到收斂得到 D^i，再取 $C(A) = \bigcup_{i=1}^{4} D^i$。每個模板會在一個方向上持續補像素，直到該方向不再有凹陷。結果是相對於這四個方向的凸包，可能比真正的歐氏凸包更大，因此實作上常把生長限制在外接矩形內。實務上，`skimage.morphology.convex_hull_image` 直接計算前景像素座標的多邊形凸包再點陣化，速度更快，也更接近歐氏凸包。

細化

細化（thinning）有選擇地移除邊界像素，但不把形狀拆開，直到只剩一個像素寬的線。用模板 B 做一次細化是

$$A \otimes B = A - (A \circledast B),$$

也就是刪掉 HMT 比對到的像素。完整的細化會輪流套用一串模板 $\{B^1, B^2, \dots, B^n\}$（通常是「邊緣像素」圖樣的 8 個旋轉），

$$A \otimes \{B\} = ((\cdots ((A \otimes B^1) \otimes B^2) \cdots) \otimes B^n),$$

並重複整個循環，直到沒有像素改變。這些模板的設計保證：刪掉被比對到的像素，不會讓形狀斷開，也不會刪掉線段的端點。Zhang 與 Suen 的平行細化演算法是這個想法廣為使用的高效版本 [6]；`skimage.morphology.skeletonize` 對 2-D 影像預設就用它，`skimage.morphology.thin` 則實作了另一種相關的平行細化。

粗化

粗化（thickening）是細化的對偶，它加入被模板比對到的背景像素：

$$A \odot B = A \cup (A \circledast B).$$

實務上通常不另外設計粗化模板，而是對背景做細化，再取補集。把背景細化到收斂，會留下一張位於物件之間正中央的細線網，所以粗化是讓物件長大到彼此相接、卻永遠不會合併的經典方法。

骨架

骨架（skeleton）$S(A)$ 是一組沿著形狀中央延伸的細線。一種正式定義使用最大圓盤：若某點是一個放得進 A、而且不被其他放得進的圓盤包含的圓盤中心，它就在骨架上。這種圓盤至少在兩處碰到邊界。骨架有純形態學的公式：

$$S(A) = \bigcup_{k=0}^{K} S_k(A), \qquad S_k(A) = (A \ominus kB) - ((A \ominus kB) \circ B),$$

其中 $A \ominus kB$ 表示以 B 連續侵蝕 A 共 k 次，K 是侵蝕變成空集合之前的最後一個 k。每個 $S_k(A)$ 收集侵蝕深度 k 時的「山脊」點：做了 k 次侵蝕後它們還在，但在那個深度做開運算就撐不住。這個分解是可逆的：

$$A = \bigcup_{k=0}^{K} (S_k(A) \oplus kB),$$

所以把每個 S_k 連同索引 k 存起來，就是一種無失真的形狀編碼。缺點是這種骨架不保證連通。細化可以得到連通、一個像素寬的骨架；**中軸**（medial axis，由距離轉換算出）則在每個骨架點上附帶最大圓盤的半徑，如圖 9.3 最右格。

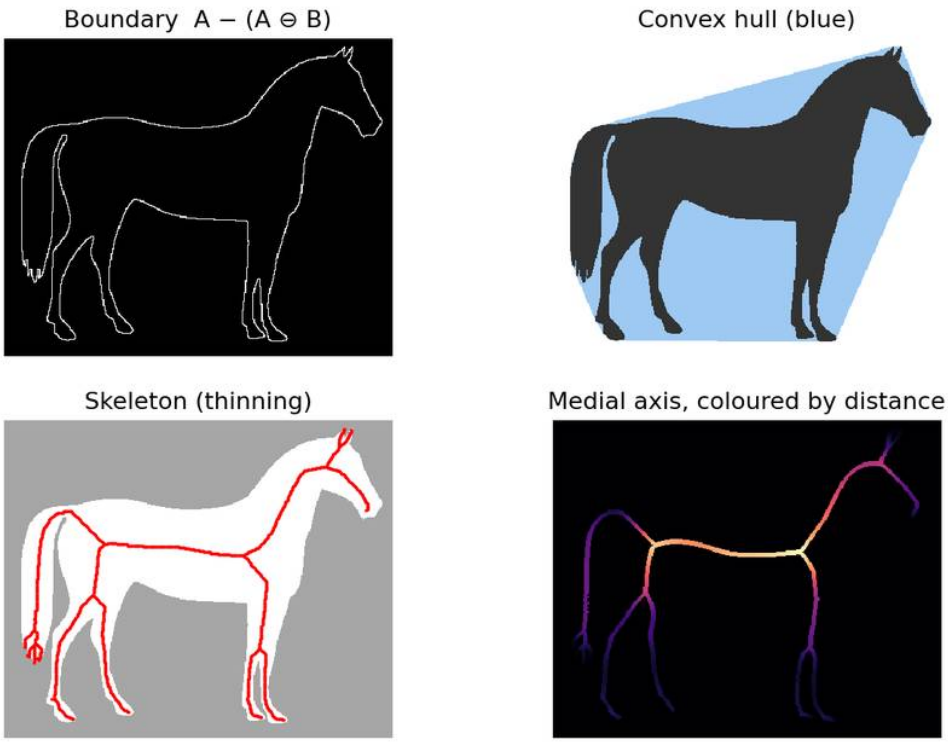


圖 *9.3* — 由侵蝕與 *HMT* 組成的演算法：內邊界、凸包、細化骨架，以及以顏色表示局部半寬的中軸。

修剪

骨架對邊界雜訊很敏感：輪廓上一個小凸起就會長出一根短短的**雜枝**（spur），圖 9.3 中馬尾附近就看得到。**修剪**（pruning）會移除長度短於 L 的雜枝：

1. 把骨架的**端點**（恰有一個 8 鄰居的像素）移除 L 次。所有長度不超過 L 的雜枝都會消失，但真正的分支末端也會少掉 L 個像素。
2. 找出縮短後骨架的端點。
3. 從這些端點做 L 次條件膨脹（限制在原骨架內），把主要分支長回原本長度。
4. 取縮短後的骨架與長回來的末端之聯集。

```
import numpy as np
from scipy import ndimage as ndi
from skimage import data, morphology

def endpoints(S):
    n = ndi.convolve(S.astype(int), np.ones((3, 3), int), mode="constant") - S
    return S & (n == 1)            # foreground pixels with exactly one neighbour

def prune_tips(S, L):
    """Step 1 of pruning: strip L layers of end points."""
    S = S.copy()
    for _ in range(L):
        S &= ~endpoints(S)
    return S

skel = morphology.skeletonize(~data.horse())
print(skel.sum(), prune_tips(skel, 10).sum())
```

形態學重建

白話說：重建讓你可以說「把含有這顆種子的物件整個留下來」，而且拿回來的是一模一樣的物件，不是被磨圓的複本。它用兩張影像：**標記**（marker）說明從哪裡開始，**遮罩**（mask）說明最多能長到哪裡。

測地膨脹與測地侵蝕

設 F 為標記、G 為遮罩，且 $F \subseteq G$。F 相對於 G 的**大小為 1 的測地膨脹**（geodesic dilation）是

$$D_G^{(1)}(F) = (F \oplus B) \cap G,$$

其中 B 是小的結構元素（通常是 3×3 方塊）。標記長大一步，但不能離開遮罩。大小為 n 就是重複 n 次：$D_G^{(n)}(F) = D_G^{(1)}\big(D_G^{(n-1)}(F)\big)$。「測地」指的是沿著留在 G 內的路徑量距離，就像在建築物裡走路，而不是穿牆。

對偶的**大小為 1 的測地侵蝕**是

$$E_G^{(1)}(F) = (F \ominus B) \cup G,$$

此時 $F \supseteq G$：標記會縮小，但不會縮到比遮罩還小。

以膨脹重建與以侵蝕重建

把測地膨脹一直迭代到不再變化，就得到遮罩 G 由標記 F 做的**以膨脹重建**（reconstruction by dilation）：

$$R_G^D(F) = D_G^{(k)}(F) \quad \text{with } k \text{ such that } D_G^{(k)}(F) = D_G^{(k+1)}(F).$$

在二值影像中，$R_G^D(F)$ 恰好是 G 裡所有至少含一個標記像素的連通元件之聯集。同理，**以侵蝕重建** $R_G^E(F)$ 是把測地侵蝕迭代到穩定。Vincent 的論文給出了標準的高效演算法（以佇列為基礎，每個像素只被拜訪有限次，而不是一次次掃過整張影像），以及許多應用 [7]；`skimage.morphology.reconstruction` 就是採用這個方法。

以重建做開運算

一般的開運算會移除小物件，但也會把大物件的角磨圓、把細的部分擦掉。**以重建做開運算**（opening by reconstruction）避開了這個問題：

$$O_R^{(n)}(F) = R_F^D(F \ominus nB),$$

其中 F 是輸入影像，$F \ominus nB$（以 B 侵蝕 n 次）是標記，輸入影像本身就是遮罩。侵蝕決定哪些物件能留下（至少放得進一個結構元素的那些）；重建則把每個倖存者恢復成原本的形狀。比較圖 9.4 的「一般開運算」與「以重建做開運算」：方塊上的細手臂和尖角都完整回來了。**以重建做閉運算**是其對偶，由膨脹與以侵蝕重建組成。

自動填洞

重建也能在不給種子的情況下一次填滿所有洞。洞就是從影像邊框走不到的背景區域。因此建立一個標記：在邊框上取影像的補值，其他地方為零，

$$F(x,y) = \begin{cases} 1 - I(x,y) & \text{if } (x,y) \text{ is on the image border,} \\ 0 & \text{otherwise,} \end{cases}$$

由它重建背景，再取補集：

$$H = \left[R_{I^c}^D(F)\right]^c.$$

$R_{I^c}^D(F)$ 是從邊框走得到的背景；其餘的部分 H 就是前景加上它的洞。

清除邊界物件

碰到影像邊界的物件通常不完整，量測時應該排除。用影像在邊框上的值當標記，

$$F(x,y) = \begin{cases} I(x,y) & \text{if } (x,y) \text{ is on the border,} \\ 0 & \text{otherwise,} \end{cases} \qquad X = I - R_I^D(F),$$

重建會找回所有碰到邊框的物件，從影像中減掉它們，就只剩內部的物件。

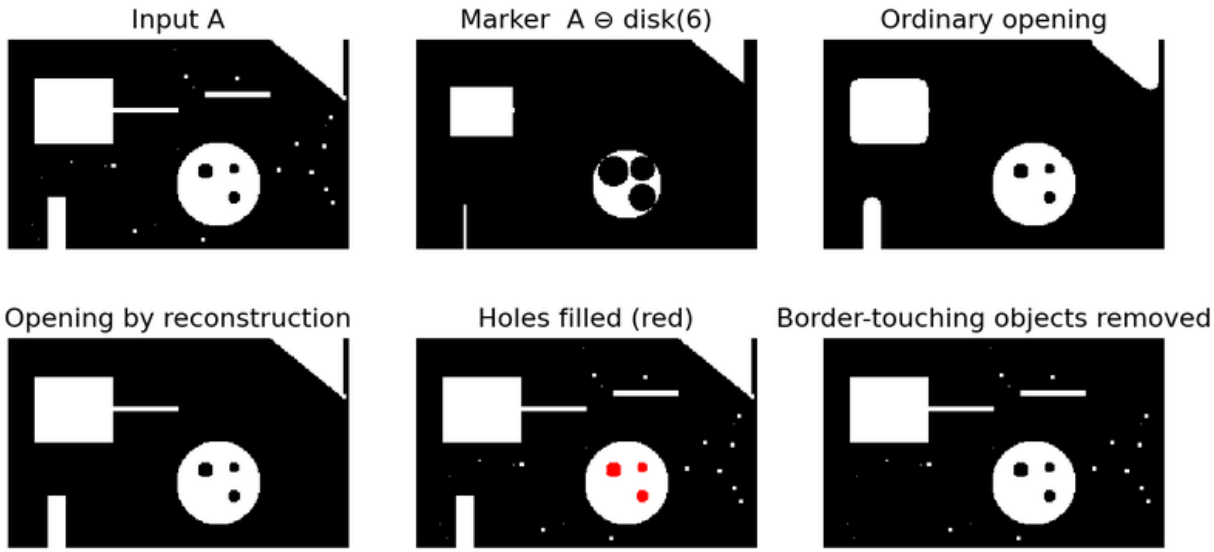


圖 *9.4* — 形態學重建。以半徑 *6* 圓盤侵蝕得到的標記挑出大物件，重建把它們（包括細手臂）完整恢復。下排：自動填洞與清除邊界物件。

```
import numpy as np
from skimage import morphology, segmentation

def open_by_reconstruction(A, footprint):
    marker = morphology.binary_erosion(A, footprint)
    return morphology.reconstruction(marker.astype(np.uint8), A.astype(np.uint8),
                                     method="dilation").astype(bool)

def fill_holes_auto(A):
    """Background reachable from the image frame is background; the rest is hole."""
    Ac = ~A
    marker = np.zeros_like(Ac)
    marker[0, :], marker[-1, :] = Ac[0, :], Ac[-1, :]
    marker[:, 0], marker[:, -1] = Ac[:, 0], Ac[:, -1]
    reached = morphology.reconstruction(marker.astype(np.uint8), Ac.astype(np.uint8),
                                        method="dilation").astype(bool)
    return ~reached

# Library shortcuts: scipy.ndimage.binary_fill_holes(A), segmentation.clear_border(A)
```

二值運算總整理

下表整理本章的二值運算。A 為輸入集合、B 為結構元素、F 為標記、G 為遮罩，並假設 B 包含原點。

運算	公式	作用
平移	$(B)_z = \{b + z\}$	把結構元素原點移到 z
反射	$\hat{B} = \{-b\}$	結構元素旋轉 180°
補集	A^c	前景與背景互換
侵蝕	$A \ominus B$	保留 B 放得進去的位置；縮小、去除細部
膨脹	$A \oplus B$	保留 $\hat{B}$ 碰得到 A 的位置；變大、接起縫隙
開運算	$(A \ominus B) \oplus B$	去除斑點、細頸、尖角；冪等
閉運算	$(A \oplus B) \ominus B$	補上小洞與窄縫；冪等
擊中擊不中	$(A \ominus B_1) \cap (A^c \ominus B_2)$	找出精確的前景／背景圖樣
邊界	$A - (A \ominus B)$	一個像素寬的內輪廓
填洞	$X_k = (X_{k-1} \oplus B) \cap A^c$	填滿含種子的那個洞
連通元件	$X_k = (X_{k-1} \oplus B) \cap A$	擷取含種子的元件
凸包	反覆 HMT 的聯集	填平凹陷
細化	$A - (A \circledast B)$，套用一串模板	剝成一個像素寬的骨架，保持連通
粗化	$A \cup (A \circledast B)$	細化的對偶
骨架	$\bigcup_k (A \ominus kB) - (A \ominus kB) \circ B$	中央線；配合索引 k 可還原
修剪	移除端點，再條件式長回	去除短雜枝
測地膨脹	$(F \oplus B) \cap G$	在遮罩內長大一步
測地侵蝕	$(F \ominus B) \cup G$	在遮罩之上縮小一步
以膨脹重建	穩定時的 $D_G^{(k)}(F)$	G 中被 F 碰到的元件
以重建做開運算	$R_F^D(F \ominus nB)$	移除小物件，其他物件保持原樣
自動填洞	$[R_{I^c}^D(F)]^c$	不需種子就填滿所有洞
清除邊界物件	$I - R_I^D(F)$	移除碰到邊界的物件

灰階形態學

白話說：在灰階影像裡，「放得進去」變成「局部最小值」，「碰得到」變成「局部最大值」。把影像想成一片地形，亮度就是高度。侵蝕把每個像素降到模板底下最低的地面；膨脹則把它升到最高處。

灰階侵蝕與膨脹

使用**平坦**結構元素 b（一組高度都是零的位移）時，影像 f 的灰階侵蝕與膨脹是

$$[f \ominus b](x,y) = \min_{(s,t)\in b} f(x+s,\, y+t), \qquad [f \oplus b](x,y) = \max_{(s,t)\in b} f(x-s,\, y-t),$$

其中 (s,t) 走遍結構元素的所有位移。膨脹中的負號一樣是反射。用在二值影像上，這兩個式子會完全退化成集合的定義。侵蝕讓影像變暗、亮的特徵縮小、暗的特徵變寬；膨脹則相反。

非平坦結構元素 b_N 另外帶有高度 $b_N(s,t)$：

$$[f \ominus b_N](x,y) = \min_{(s,t)\in b_N} \{f(x+s,\, y+t) - b_N(s,t)\}, \qquad [f \oplus b_N](x,y) = \max_{(s,t)\in b_N} \{f(x-s,\, y-t) + b_N(s,t)\}.$$

實務上很少用非平坦結構元素：結果會受影像的亮度尺度影響，而且侵蝕可能低於影像的最小值。對偶性用灰階補數 $f^c = -f$（對 L 階影像則為 $L-1-f$）依然成立：$(f \ominus b)^c = f^c \oplus \hat{b}$。

開運算與閉運算

定義不變：$f \circ b = (f \ominus b) \oplus b$、$f \bullet b = (f \oplus b) \ominus b$。幾何圖像是一顆球從亮度曲面下方往上推：開運算就是球在不超過 f 的前提下能到達的最高曲面。比球窄的亮峰會被削平，比球寬的部分則不受影響。閉運算是球從上方往下壓：比球窄的暗谷會被填平。各種性質（反擴張／擴張、遞增、冪等）只要把 $\subseteq$ 換成 $\leq$ 就照樣成立。

形態學平滑

開運算壓掉小的亮細節，閉運算壓掉小的暗細節，所以用同一個結構元素**先開後閉**，可以同時去除這兩種小結構，同時保留較大區域的銳利邊緣。這就是圖 9.1 雜訊濾波器的灰階版本。相關的**交替序列濾波**（alternating sequential filtering）則用逐漸變大的結構元素輪流做開、閉運算，避免一次用大結構元素造成的瑕疵。

形態學梯度

膨脹減去侵蝕，可以衡量局部對比：

$$g = (f \oplus b) - (f \ominus b).$$

在均勻區域內最大值與最小值幾乎相等，所以 $g \approx 0$；跨過邊緣時兩者相差約一個邊緣高度。和 Sobel 導數不同，g 一定非負，而且（對稱 b 時）不偏任何方向。只取一邊則得到**內梯度** $f - (f \ominus b)$ 或**外梯度** $(f \oplus b) - f$，分別把邊緣響應放在較亮區域的內側或外側。

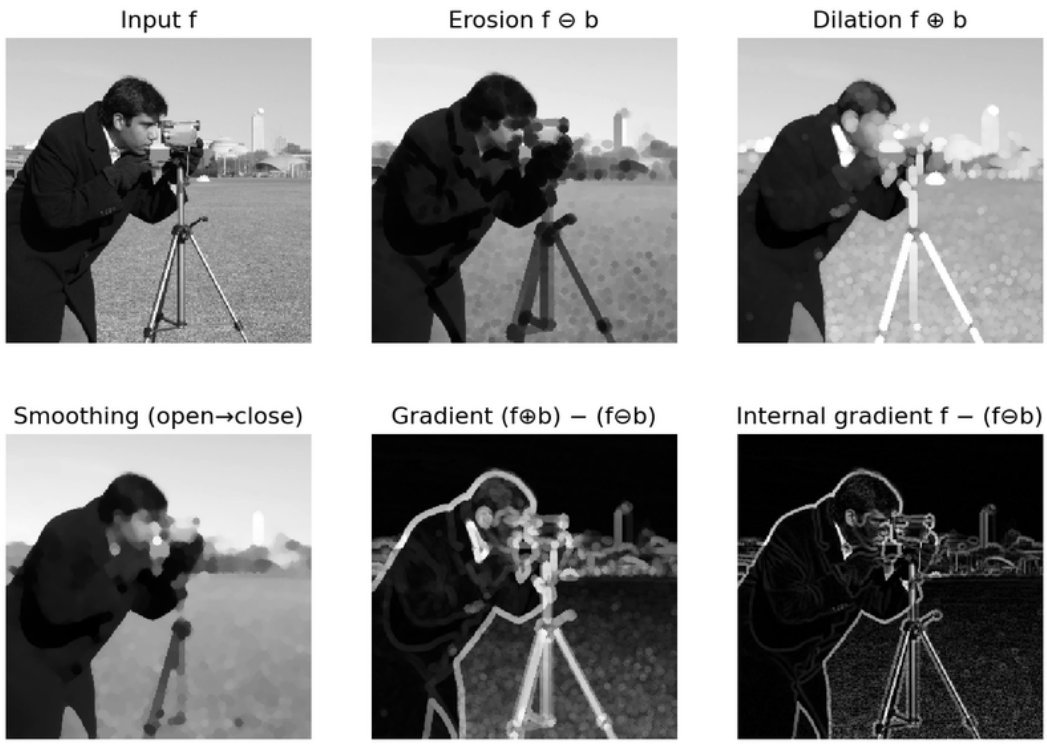


圖 *9.5* — 以半徑 *3* 的平坦圓盤做灰階形態學：侵蝕（局部最小）、膨脹（局部最大）、先開後閉的平滑、形態學梯度，以及內梯度。

```
from skimage import data, morphology, util

f = util.img_as_float(data.camera())
b = morphology.disk(3)                          # flat structuring element
ero  = morphology.erosion(f, b)                 # local minimum
dil  = morphology.dilation(f, b)                # local maximum
grad = dil - ero                                # morphological gradient (>= 0)
smooth = morphology.closing(morphology.opening(f, b), b)
```

頂帽與底帽轉換

用比亮物件更大的結構元素做開運算，會把物件削掉，但保留緩慢變化的背景。因此把影像減去開運算結果，就只剩下物件。這就是**頂帽轉換**（top-hat，又稱 white top-hat）：

$$T_{\text{hat}}(f) = f - (f \circ b).$$

它的對偶是**底帽轉換**（bottom-hat，又稱 black top-hat），用來擷取亮背景上的暗物件：

$$B_{\text{hat}}(f) = (f \bullet b) - f.$$

兩者都非負。最主要的用途是在閾值化之前校正**不均勻照明**。在圖 9.6 中，一張從單側打光的合成顆粒影像讓全域 Otsu 閾值（第 10 章）完全失效：亮側的背景比暗側的顆粒還亮。用半徑 8（大

於任何顆粒）的圓盤做頂帽轉換後，背景變平，單一閾值就幾乎完美地分出顆粒。唯一的參數是結構元素大小：它必須大於物件，又要小於背景變化的尺度。

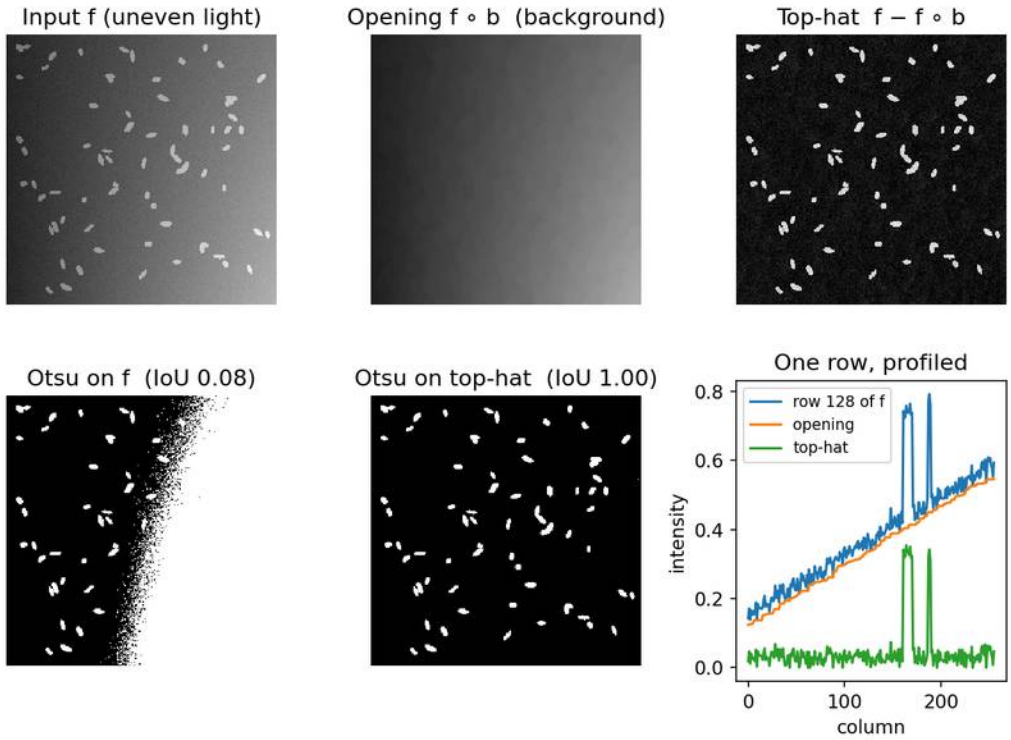
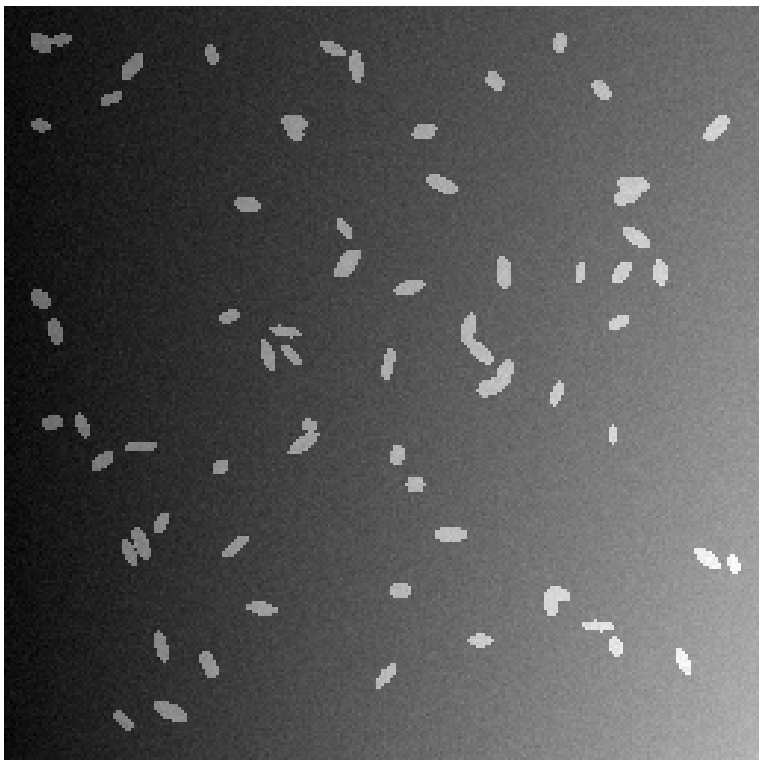


圖 *9.6* — 用頂帽轉換校正不均勻照明。以大於顆粒的圓盤做開運算可估計背景，相減後背景變平。*IoU* 是和已知的顆粒遮罩比較的結果。

拖動滑桿比較輸入與頂帽結果：



輸入

頂帽（圓盤 $r=8$）

不均勻照明下的顆粒，做白頂帽轉換前後的比較

```
from skimage import data, filters, morphology, util

coins = util.img_as_float(data.coins())
th = morphology.white_tophat(coins, morphology.disk(30))   # bright objects
bh = morphology.black_tophat(coins, morphology.disk(30))   # dark objects
mask = th > filters.threshold_otsu(th)
```

粒度測定

粒度測定（granulometry）不需要分割，就能量出影像中顆粒的大小分布。用越來越大的結構元素 r 對影像做開運算，每次開運算都會移除比 b_r 小的亮顆粒。記錄剩下的總亮度：

$$V(r) = \sum_{x,y} [f \circ b_r](x,y), \qquad r = 0, 1, 2, \dots,$$

其中 b_r 是半徑 r 的圓盤，b_0 是單一像素（所以 $V(0)$ 就是 f 的總和）。$V(r)$ 絕不會增加，因為越大的圓盤能放進的位置越少。它的負離散導數

$$PS(r) = V(r-1) - V(r)$$

是一種大小直方圖，稱為**圖樣頻譜**（pattern spectrum）[8]：在 r 處的值很大，表示有很多影像「質量」位於半徑約為 r 的特徵中。圖 9.7 的場景有兩種顆粒大小，頻譜也對應出現兩個峰。以大小為索引的一族開運算、以及「篩子」的詮釋，可以追溯到 Matheron [2]；Maragos 則把圖樣頻譜發展成多尺度形狀描述子 [8]。

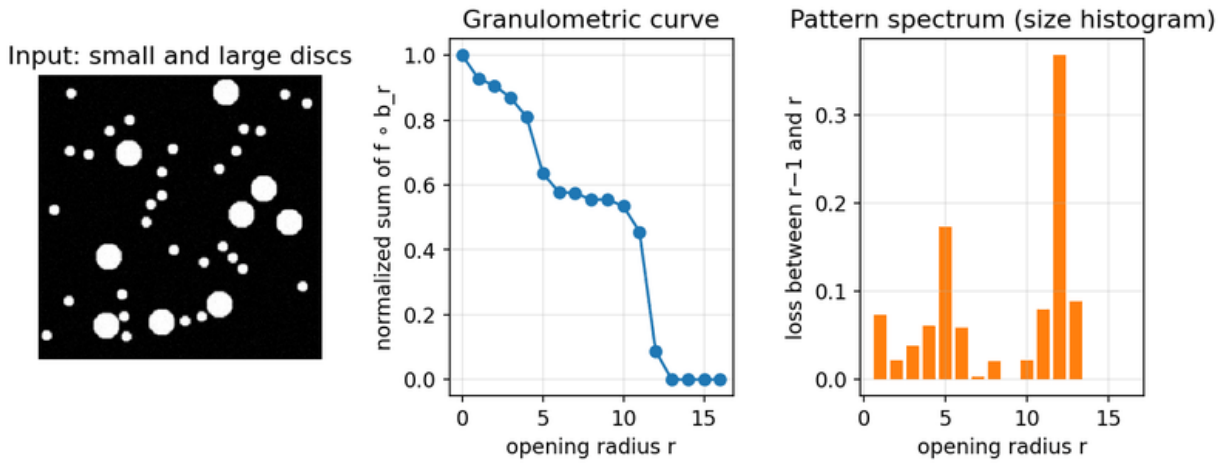


圖 *9.7* — 半徑 *5* 與 *12* 兩種圓盤的粒度測定。粒度曲線下降兩次，圖樣頻譜在兩個半徑處出現峰值。

```
import numpy as np
from skimage import morphology

def granulometry(f, radii):
    total = f.sum()
    vol = np.array([morphology.opening(f, morphology.disk(r)).sum() / total
                    for r in radii])
    spectrum = -np.diff(vol)                # loss from radius r-1 to r
    return vol, spectrum
```

紋理分割

形態學也能把「紋理元素大小不同」的區域分開，例如一片小顆粒旁邊接著一片大顆粒。做法沿用粒度測定的大小邏輯：

1. 用比小顆粒之間縫隙更大的結構元素做**閉運算**（暗背景上的亮顆粒，閉運算會填平這些暗縫）。小顆粒區變成一塊幾乎均勻的亮區，而大顆粒之間的縫比結構元素寬，仍然分開。
2. 用比大顆粒更大、但比合併後的小顆粒區更小的結構元素對結果做**開運算**，擦掉孤立的大顆粒，只留下合併後的小顆粒區。
3. 對結果取**形態學梯度**（或做閾值）。兩種紋理的交界就成為一條明顯的邊緣。

確切的結構元素大小取決於資料中的顆粒與間距大小，可以先用粒度測定量出來。

灰階重建

所有重建的定義都能搬過來，只要把 $\cap$ 換成逐點最小值 $\wedge$、$\cup$ 換成最大值 $\vee$。標記影像 f 在遮罩影像 g（$f \le g$）之下的測地膨脹為

$$D_g^{(1)}(f) = (f \oplus b) \wedge g,$$

以膨脹重建就是把它迭代到穩定。灰階重建支撐了好幾種實用的濾波器 [7]：

- **以重建做開運算** $R_f^D(f \ominus nb)$：移除比結構元素小的亮細節，其他部分則保持原本形狀。**以重建做頂帽** $f - R_f^D(f \ominus nb)$ 比一般頂帽乾淨，因為估計出的背景不會有被磨圓的「肩膀」。
- **區域極大值與 h-dome 轉換**：以 $f - h$ 為標記重建 f。重建會把每個高度不超過 h 的峰削平；從 f 減去它，就只剩下「圓頂」（峰），每個高度至多 h。對圓頂做閾值，不論局部背景多亮都能偵測出亮斑。

```
from skimage import data, morphology, util
import numpy as np

f = util.img_as_float(data.camera())
h = 0.1
background = morphology.reconstruction(np.clip(f - h, 0, None), f, method="dilation")
domes = f - background                      # bright peaks, each at most h tall
```

現代觀點

形態學雖然歷史悠久，但遠未結束。有五條研究脈絡值得認識。

理論基礎與教學文獻。 Matheron 的隨機集合理論 [2] 與 Serra 的 *Image Analysis and Mathematical Morphology* [3] 建立了整套代數：侵蝕與膨脹是 Minkowski 運算、開運算是「篩子」，而格論（lattice theory）的觀點後來把一切從集合推廣到灰階函數。Haralick、Sternberg 與 Zhuang 1987 年的教學論文 [4] 至今仍是工程師入門二值與灰階形態學最清楚的文獻之一：它定義運算子、證明基本性質，並連結到實用演算法。Soille 的 *Morphological Image Analysis* [5] 是實務者的參考書，涵蓋測地轉換、重建、分水嶺、粒度測定與高效實作；函式庫文件寫得太簡略時，就該翻它。

重建與高效演算法。 Vincent 1993 年的論文 [7] 讓重建變得實用。重點在於：天真的「迭代到穩定」迴圈可以換成以佇列為基礎、成本大致隨影像大小線性成長的演算法，於是重建成為填洞、區域極值與 h-dome 的日常工具。

連通運算子與元件樹。若一個濾波器只能移除或合併平坦區（數值相同的連通區域），而絕不產生新的輪廓，它就是**連通**（connected）的。以重建做開運算是最簡單的例子。Salembier 與 Wilkinson 的回顧論文 [10] 介紹了整個家族：面積開運算、屬性濾波器、levelings，以及以樹為基礎的濾波。Breen 與 Jones 提出**屬性開運算與屬性細化**（attribute openings and thinnings）[9]，依據屬性（面積、伸長度、慣性矩）而非「結構元素放不放得進」來決定保留或移除一個連通元件。高效實作把影像表示成**最大樹**（max-tree）或**最小樹**（min-tree），也就是所有閾值集合之連通元件構成的樹，再以修剪樹的方式濾波。Carlinet 與 Géraud 在統一框架下比較了多種最大樹建構演算法，並給出挑選演算法的決策樹 [11]。scikit-image 提供了 `area_opening`、`area_closing`、`diameter_opening` 與 `max_tree`。

大小分布。 Maragos 的圖樣頻譜 [8] 把粒度測定形式化為多尺度形狀描述子，至今仍用於紋理與顆粒大小分析；以屬性為基礎的粒度測定 [9] 則把它推廣到大小以外的形狀屬性。

形態學遇上深度學習。深度學習從兩個方向改變了形態學。

網路內部的形態學。最大池化（max pooling）本來就是平坦灰階膨脹再加上降取樣，所以兩者的連結很自然。有好幾個團隊設計了結構元素可以用梯度下降學習的網路層。Mondal 等人研究由可學習的膨脹與侵蝕神經元組成的網路，證明兩個「膨脹與侵蝕後接線性組合」的區塊就能逼近任意連續函數 [13]。Nogueira 等人提出 DeepMorphNet，以可學習的形態學運算取代卷積，並比較它學到的特徵與一般卷積網路的差異 [14]。實務上的難處是 min 與 max 的梯度很稀疏。Kirszenberg 等人 [16] 與 Hermary 等人 [15] 用 max 與 min 的可微近似（銳利程度由可學習參數控制的廣義平均與 soft-max 函數）建構平滑的形態學層，改進了較早的 p-convolution 層，並探討這些層何時真的能學回侵蝕與膨脹。這些仍是研究工具：形態學層並不是主流視覺骨幹網路的標準元件。

網路周邊的形態學。在分割任務中，形態學一直被大量用作預測遮罩的**後處理**：用開運算去掉偽陽性斑點、用閉運算或填洞修補破碎的物件、`remove_small_objects`、清除邊界物件，以及用骨架量測管狀結構。它也出現在**損失函數**裡：clDice [17] 以反覆的 min 與 max 池化（軟侵蝕與軟膨脹）算出可微的「軟骨架」，再用預測骨架與真實骨架的重疊程度當作損失，鼓勵血管、道路與神經元的分割在拓撲上正確。

沒有改變的是：要清理、量測、校正二值或灰階影像，本章的經典運算子仍然是最快、最可預測的工具，而且它們附帶學習式濾波器無法提供的保證（冪等性、連通性、精確保持形狀）。

重點整理

- 形態學處理的是**形狀**，不是頻率。它只用最小值、最大值、聯集與交集，並由一個結構元素控制；結構元素的大小與形狀就是你提出的問題。
- **侵蝕**保留結構元素放得進去的位置；**膨脹**保留反射後的結構元素碰得到的位置。兩者互為對偶：侵蝕前景等於膨脹背景。
- **開運算**（先侵蝕後膨脹）去除小的亮部；**閉運算**（先膨脹後侵蝕）補上小的暗縫。兩者都是冪等濾波器，先開後閉是很穩健的雜訊清除法。
- **擊中擊不中轉換**比對精確的前景／背景圖樣，是細化、粗化與凸包的基礎。
- **重建**（標記加遮罩，長到穩定為止）能完整保留整個物件：以重建做開運算、自動填洞與清除邊界物件都由它而來。
- 在灰階影像中，侵蝕與膨脹就是局部最小值與最大值。**梯度**、**頂帽**、**底帽**分別用於邊緣、背景校正與小物件擷取；**粒度測定**不需分割就能得到大小分布。
- 今天形態學仍廣泛用於神經網路遮罩的後處理、clDice 這類損失中的可微積木，也是可學習形態學層的研究主題。

練習

1. 一張二值影像中有一個寬 40 像素、高 10 像素的矩形。分別描述它被 (a) 5×5 方塊、(b) 長 15 像素的水平線段、(c) 長 15 像素的垂直線段侵蝕與開運算的結果。哪個結構元素會把矩形完全移除？為什麼？

   提示

   侵蝕保留的是「整個結構元素放得進去」時原點的位置。5×5 方塊可放的位置構成 36×6 的區塊，而開運算會恢復整個矩形，因為矩形本身就是放得進去的方塊之聯集。水平線段也放得進去，所以開運算結果同樣是整個矩形。垂直線段比矩形還高，永遠放不進去：侵蝕與開運算都是空集合。

2. 證明集合的膨脹滿足交換律與結合律：$A \oplus B = B \oplus A$、$(A \oplus B) \oplus C = A \oplus (B \oplus C)$。用第二個等式解釋為什麼用 3×3 方塊膨脹兩次，等於用 5×5 方塊膨脹一次。侵蝕也有同樣的性質嗎？

   提示

   把膨脹寫成 Minkowski 和 $\{a + b \mid a \in A, b \in B\}$，兩個等式都直接來自向量加法的交換律與結合律。$3 \times 3$ 方塊加上自己就是 5×5 方塊。侵蝕則用 $(A \ominus B) \ominus C = A \ominus (B \oplus C)$，這可由對偶性推得。

3. 設計一組擊中擊不中模板 (B_1, B_2)，偵測一條 8 連通、一個像素寬、往右延伸到盡頭（線從左邊過來）的線段**端點**。要抓到所有端點，總共需要幾個旋轉後的模板？用 `scipy.ndimage.binary_hit_or_miss` 在 `skimage.morphology.skeletonize(~skimage.data.horse())` 上驗證你的答案。

   提示

   中心與左鄰居放在 B_1，其他七個鄰居放在 B_2。線也可能從斜方向過來，所以需要 8 個模板（4 個軸向加 4 個斜向）。把你數到的總數和修剪程式碼中 `endpoints()` 這類「數鄰居」函式的結果比較。

4. 你的分割網路輸出細胞遮罩。許多遮罩有小洞、少數單像素的偽陽性，還有一些細胞被影像邊界切到。用 scikit-image 寫一個五行的後處理流程，並說明步驟順序的理由。

   提示

   一個合理的順序是：先 `remove_small_objects`（或小的開運算），避免斑點被填起來或被合併；接著 `binary_fill_holes`；輪廓參差時再做一次小的閉運算；然後 `clear_border`；最後用 `label` 計數。討論一下如果先做閉運算再去斑點會出什麼問題（相鄰的斑點可能合併成一個「細胞」）。

5. 對灰階影像 f 與平坦結構元素 b，證明頂帽 $T(f) = f - (f \circ b)$ 非負，而且是冪等的：$T(T(f)) = T(f)$。

   提示

   非負性來自 $f \circ b \le f$。冪等性則要證明 $T(f) \circ b = 0$。任取 b 的一個平移 W，令 y 為 W 中 f 最小的點。開運算在 W 上處處至少是 $\min_W f$，又不超過 f，所以在 y 處恰等於 $f(y)$，因此 $T(f)(y) = 0$。於是 b 的每個平移都含有 $T(f)$ 的零點，$T(f)$ 被 b 侵蝕後為零，開運算也是零。

參考文獻

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 9. publisher page
2. G. Matheron, *Random Sets and Integral Geometry*, Wiley, 1975. library record
3. J. Serra, *Image Analysis and Mathematical Morphology*, Academic Press, 1982. library record
4. R. M. Haralick, S. R. Sternberg, and X. Zhuang, “Image Analysis Using Mathematical Morphology,” *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. PAMI-9, pp. 532–550, 1987. doi
5. P. Soille, *Morphological Image Analysis*, Springer, 2004. doi
6. T. Y. Zhang and C. Y. Suen, “A Fast Parallel Algorithm for Thinning Digital Patterns,” *Communications of the ACM*, vol. 27, pp. 236–239, 1984. doi
7. L. Vincent, “Morphological Grayscale Reconstruction in Image Analysis: Applications and Efficient Algorithms,” *IEEE Trans. Image Processing*, vol. 2, pp. 176–201, 1993. doi
8. P. Maragos, “Pattern Spectrum and Multiscale Shape Representation,” *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 11, no. 7, pp. 701–716, 1989. doi
9. E. J. Breen and R. Jones, “Attribute Openings, Thinnings, and Granulometries,” *Computer Vision and Image Understanding*, vol. 64, no. 3, pp. 377–389, 1996. doi
10. P. Salembier and M. H. F. Wilkinson, “Connected Operators: A Review of Region-Based Morphological Image Processing Techniques,” *IEEE Signal Processing Magazine*, vol. 26, no. 6, pp. 136–157, 2009. doi
11. E. Carlinet and T. Géraud, “A Comparative Review of Component Tree Computation Algorithms,” *IEEE Trans. Image Processing*, vol. 23, pp. 3885–3895, 2014. doi
12. S. van der Walt et al., “scikit-image: Image Processing in Python,” *PeerJ*, 2014. arXiv
13. R. Mondal, S. Santra, S. S. Mukherjee, and B. Chanda, “Morphological Network: How Far Can We Go with Morphological Neurons?,” BMVC, 2022. arXiv

14. K. Nogueira, J. Chanussot, M. Dalla Mura, and J. A. dos Santos, “An Introduction to Deep Morphological Networks,” arXiv:1906.01751, 2019. arXiv
15. R. Hermary, G. Tochon, É. Puybareau, A. Kirszenberg, and J. Angulo, “Learning Grayscale Mathematical Morphology with Smooth Morphological Layers,” *Journal of Mathematical Imaging and Vision*, vol. 64, pp. 736–753, 2022. doi
16. A. Kirszenberg, G. Tochon, É. Puybareau, and J. Angulo, “Going Beyond p-convolutions to Learn Grayscale Morphological Operators,” arXiv:2102.10038, 2021. arXiv
17. S. Shit et al., “clDice — A Novel Topology-Preserving Loss Function for Tubular Structure Segmentation,” CVPR, 2021. arXiv

IP 10・影像分割（一）：邊緣偵測、閾值化與區域偵測

把影像切成有意義的部分：用梯度、LoG 與 Canny 找邊緣，用 Hough 轉換串接邊緣，用 Otsu 法選閾值，再以成長、分群、圖切割與淹水來找出區域。

網站版本: https://blog.shyandram.dev/zh-tw/learn/image-processing/dip-10-segmentation/

你將學到

- 影像分割（image segmentation）的正式定義，以及兩大類方法：找**不連續**（邊緣）與找**相似**（區域）。
- 導數如何找出點、線與邊緣，以及 Sobel、Marr–Hildreth（LoG）和 Canny 偵測器的每個步驟。
- 如何把零散的邊緣像素串成邊界，包括用 Hough 轉換。
- Otsu 法如何自動選出閾值，以及單一閾值不夠用時該怎麼辦。
- 區域成長、分裂合併、k-means、SLIC、正規化切割、分水嶺與運動資訊，如何把像素變成區域。

先看全貌

到目前為止，大多數運算都是輸入一張影像、輸出另一張影像。影像分割輸出的則是一張**部件地圖**：每個像素都拿到一個標籤，說明它屬於哪個物體或區域。量測物體、計數細胞、辨識文字，全都仰賴這張地圖正不正確。

> **用最簡單的話說**
>
> 想像一張著色本的圖，黑色輪廓線全部不見了。你還是有兩種辦法分出各個部分。第一種：找顏色突然改變的地方，在那裡畫一條線，這就是**邊緣偵測**。第二種：從某一點開始，只要紙上的顏色看起來一樣，就一直用同一枝蠟筆塗開，這就是**區域成長**。本章就是在講這兩招，以及怎麼讓它們在雜亂的真實照片上也管用。

本章的方法都不需要從標註資料學習。它們不用訓練資料、容易解釋，而且其中的想法（梯度、遲滯、類間變異數、圖切割、淹水）至今仍活在現代深度學習流程裡。

基本概念

白話版：分割就是把影像切成彼此不重疊、合起來涵蓋全部、而且每一塊內部「看起來一樣」的幾塊。

令 R 為整張影像所佔的空間區域。分割是把 R 劃分成 n 個子區域 $R_1, \dots, R_n$，滿足

(a) $\bigcup_{i=1}^{n} R_i = R$, (b) R_i is connected,
(c) $R_i \cap R_j = \emptyset$ for $i \neq j$, (d) $Q(R_i) = \text{TRUE}$, (e) $Q(R_i \cup R_j) = \text{FALSE}$ for adjacent R_i, R_j.

其中 Q 是**邏輯述詞**（logical predicate）：對一組像素回答是或否的測試，例如「所有強度彼此相差不超過 10 個灰階」。也就是說：每個像素都有標籤 (a)、每個區域是連通的一整塊 (b)、沒有像素同時有兩個標籤 (c)、每個區域都通過測試 (d)、相鄰的兩個區域一旦合併就會不通過 (e)。

幾乎所有單色影像的分割演算法，都建立在強度的兩種性質之一 [1]：

- **不連續性**（discontinuity）：邊界就是強度突然改變的地方。點、線與邊緣偵測都靠它。
- **相似性**（similarity）：同一區域的像素共享某種性質（強度、顏色、紋理）。閾值化、區域成長、分群、圖切割與分水嶺都靠它。

邊緣定位精確但常常斷斷續續；區域天生是封閉的，但邊界可能歪掉。

點、線與邊緣偵測

白話版：邊緣就是亮度變化很快的地方，而「變化有多快」正是導數在量的東西。

背景：像素格上的導數

一維數位函數 $f(x)$ 最簡單的近似是

$$\frac{\partial f}{\partial x} \approx f(x+1) - f(x), \qquad \frac{\partial^2 f}{\partial x^2} \approx f(x+1) + f(x-1) - 2f(x).$$

其中 x 是整數像素位置，$f(x)$ 是它的強度。走過一段斜坡時，一階導數在整段斜坡上都不為零，因此響應**很粗**。二階導數只在斜坡兩端不為零、而且正負相反，形成**雙重響應**，中間夾著一個**零交越**（zero crossing）；它對細節與雜訊也敏感得多。後面所有偵測器都從這幾個事實出發。

偵測孤立點

點是一個和周圍不一樣的小斑點。拉普拉斯運算子（Laplacian）

$$\nabla^2 f(x,y) = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

可用中心為 -8、八個鄰居都是 1 的 3×3 核（含對角線的版本）實作。令 $Z(x,y)$ 為濾波結果，凡是 $|Z(x,y)| > T$（T 為非負閾值）的位置就標為點。核的係數總和為零，所以平坦區域沒有響應。

偵測線

線是寬度只有一到數個像素的結構。拉普拉斯運算子也會對線起反應，但會產生雙重響應，而且沒有方向性。要找特定角度的線，就用一個沿該角度排著 2、其餘為 -1 的核（水平線的核中間那

一列是 2, 2, 2）。比較水平、+45°、垂直與 −45° 四個核的響應，就能知道每個像素上哪個方向最強。

邊緣模型

真實的邊緣可理想化成三種形狀：

- **階梯邊緣**（step edge）：在一個像素內從一個灰階跳到另一個，多出現在合成影像；
- **斜坡邊緣**（ramp edge）：在數個像素內線性變化，這是光學與取樣造成的模糊在真實影像中的樣子；
- **屋脊邊緣**（roof edge）：先升後降，像一條被模糊過的細線。

在斜坡上，一階導數的大小告訴我們有邊緣，二階導數的正負告訴我們哪一側比較亮，而它的零交越標出斜坡中心。問題是，肉眼幾乎看不見的雜訊就足以毀掉二階導數，所以**先平滑再微分**是所有像樣的邊緣偵測器的一部分。

用梯度做基本邊緣偵測

f 在 (x, y) 的**梯度**（gradient）是向量

$$\nabla f = \begin{bmatrix} g_x \\ g_y \end{bmatrix} = \begin{bmatrix} \partial f/\partial x \\ \partial f/\partial y \end{bmatrix}, \qquad M(x,y) = \sqrt{g_x^2 + g_y^2}, \qquad \alpha(x,y) = \operatorname{atan2}(g_y, g_x).$$

g_x、g_y 是兩個方向的偏導數，M 是**梯度大小**（邊緣強度），α 是**梯度方向**（強度上升最快的方向）；邊緣本身與 α 垂直。實務上常用較省計算的 $|g_x| + |g_y|$ 來近似 M。

導數用小型核計算，經典運算子整理於 [1]。**Roberts** 交叉運算子使用 2×2 的對角差分；**Prewitt** 運算子使用列為 $[-1, 0, 1]$ 的 3×3 核；**Sobel** 運算子則把中間一列加權為 2，例如

$$K_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix},$$

也就是一個方向做差分、另一個方向乘上 $[1, 2, 1]$ 的平滑。正因為內建平滑，Sobel 通常比 Prewitt 更受青睞。

```
import cv2
import numpy as np
from skimage import data

f = data.camera().astype(np.float32) / 255.0
gx = cv2.Sobel(f, cv2.CV_32F, 1, 0, ksize=3)   # d/dx (columns)
gy = cv2.Sobel(f, cv2.CV_32F, 0, 1, ksize=3)   # d/dy (rows)
M = np.hypot(gx, gy)                           # gradient magnitude
alpha = np.arctan2(gy, gx)                     # gradient direction, radians
edges = M > 0.3 * M.max()                      # crude thresholded edge map
```

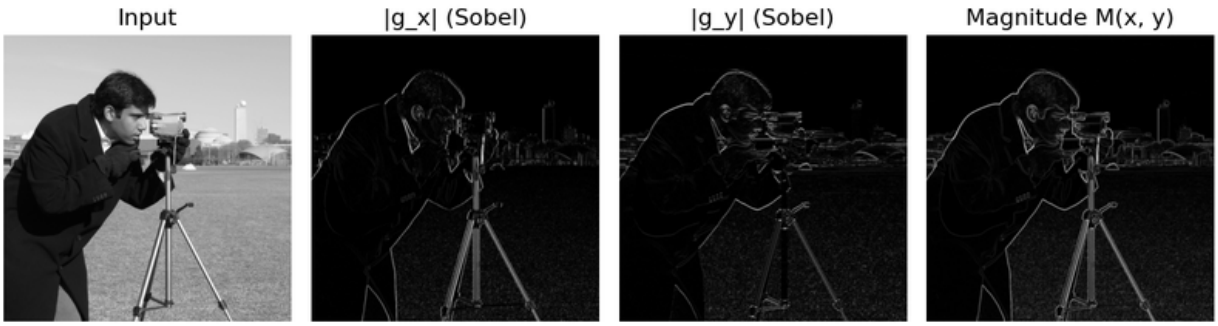


圖 *10.1 — Sobel* 導數。$|g_x|$ 會凸顯垂直結構（例如三腳架的腳），$|g_y|$ 會凸顯水平結構（例如地平線），梯度大小 M 則結合兩者。注意草地的紋理也有響應。

自己動手試試 Sobel 核：換成 y 方向的版本，或把 2 改成 1 變成 Prewitt，看看響應怎麼變。

> **互動示範（網站版）**：請開啟本篇教學的網站版本，操作此示範。開啟示範

直接對 M 做閾值化得到的邊緣圖並不好：邊緣太粗、弱邊緣消失、紋理造成雜亂。接下來兩個偵測器把平滑與細化直接設計進去。

Marr–Hildreth 邊緣偵測器

Marr 與 Hildreth [2] 主張強度變化發生在許多尺度上，所以偵測器的大小必須可調；而二階導數的零交越正好標出邊緣中心。他們的運算子是**高斯的拉普拉斯**（Laplacian of a Gaussian, LoG）$\nabla^2 G$，其中

$$G(x,y) = e^{-\frac{x^2+y^2}{2\sigma^2}}, \qquad \nabla^2 G(x,y) = \frac{x^2+y^2-2\sigma^2}{\sigma^4}\, e^{-\frac{x^2+y^2}{2\sigma^2}}.$$

σ 是高斯的標準差，決定尺度（σ 越大，只有越粗的邊緣留下來）；因為只在乎零交越，所以省略了正規化常數。由於形狀的關係，$\nabla^2 G$ 又稱為**墨西哥帽**（Mexican hat）。演算法如下：

1. 用 $n \times n$ 的高斯核平滑，n 取不小於 6σ 的最小奇數。
2. 對結果取拉普拉斯（由於線性，步驟 1–2 等於與 $\nabla^2 G$ 做一次卷積）。
3. 標出**零交越**：某一對相對鄰居的正負號不同，且差的絕對值超過閾值的像素。沒有這個閾值，平坦區域的每個小波動都會變成「邊緣」。

高斯差（difference of Gaussians, DoG）$G_{\sigma_1} - G_{\sigma_2}$，取 $\sigma_1/\sigma_2 \approx 1.6$，可以近似 LoG [1]。零交越會形成封閉輪廓，但在紋理處會產生一團「義大利麵」，也會把角落磨圓。

Canny 邊緣偵測器

Canny [3] 把邊緣偵測寫成一個最佳化問題，有三個準則：

1. **低錯誤率**：找到所有真正的邊緣，不產生假邊緣。

2. **定位準確**：偵測到的邊緣要盡量貼近真正的邊緣。
3. **單一響應**：一條真正的邊緣只產生一條偵測結果，而不是雙線。

對白雜訊中的一維階梯邊緣，最佳濾波器可以用高斯的一階導數很好地近似。實用的二維演算法如下：

1. 用標準差為 σ 的高斯**平滑**影像：$f_s = G_\sigma \star f$。
2. **計算** f_s 的梯度大小 M 與角度 α。
3. **非極大值抑制**（non-maximum suppression）。把 α 量化成四個方向；若某像素的 M 小於梯度方向上兩個鄰居中的任一個，就設為零。脊線因此只剩一個像素寬。
4. **雙閾值**。選 T_H 與 T_L，T_H/T_L 約為 2 到 3 [1]。高於 T_H 的是**強**邊緣，介於兩者之間的是**弱**邊緣。
5. **遲滯**（hysteresis）。弱像素只有在透過其他弱像素連到強像素時才保留。

遲滯是關鍵。單一閾值不是把輪廓切斷，就是放進雜訊；雙閾值加上連通性，能保留掛在強邊緣上的微弱輪廓，同時丟掉孤立的弱響應。

```
import cv2
from scipy import ndimage as ndi
from skimage import data, feature

f8 = data.camera()
canny_cv = cv2.Canny(cv2.GaussianBlur(f8, (0, 0), 2), 30, 90)      # uint8 0/255
canny_sk = feature.canny(f8 / 255.0, sigma=2, low_threshold=0.05, high_threshold=0.15)
log = ndi.gaussian_laplace(f8 / 255.0, sigma=3)                    # Marr-Hildreth steps 1-2
```

`cv2.Canny` 沒有平滑參數，所以我們先自己模糊；scikit-image 的版本則直接接受 `sigma`。

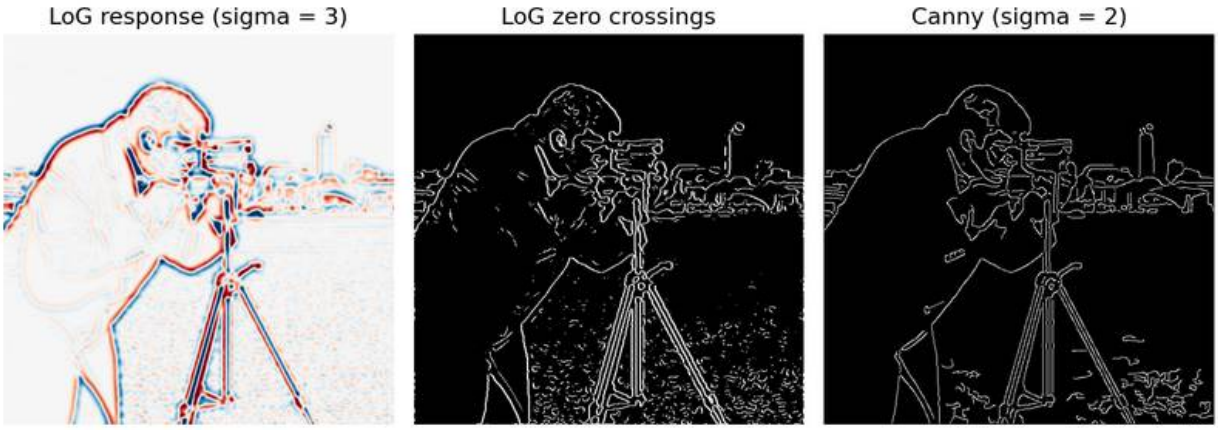


圖 *10.2* — 同一張影像上的 *Marr–Hildreth* 與 *Canny*。*LoG* 響應（左，紅為正、藍為負）在每條邊緣處變號；經過閾值的零交越（中）是封閉的，但在草地上很雜亂。*Canny*（右）給出細而大多連續的邊緣，雜訊少得多。

拖動滑桿，看看 Canny 留下了哪些結構（邊緣以白底黑線顯示）。

輸入

Canny σ=2

攝影師影像與其 Canny 邊緣圖的比較

串接邊緣點

即使是 Canny 也會留下缺口。**邊緣串接**（edge linking）把邊緣像素組成有意義的邊界。

局部處理。在每個邊緣像素 (x, y) 的小鄰域 S_{xy} 中，若鄰居 (s, t) 的梯度大小與角度都相近，就把兩者連起來：

$$|M(s,t) - M(x,y)| \le E, \qquad |\alpha(s,t) - \alpha(x,y)| \le A,$$

其中 E、A 是正的容許值。更省計算的做法是：先對 M 做閾值化，只留下角度接近目標方向的像素，再沿每一列補上短缺口；把影像旋轉就能處理其他方向。

以 Hough 轉換做全域處理。若已知形狀，就讓每個邊緣像素替所有可能通過它的形狀投票。對直線，Duda 與 Hart [4] 提出**法線表示法**

$$\rho = x\cos\theta + y\sin\theta,$$

其中 θ 是直線法線的角度，ρ 是原點到直線的有號距離；和斜率–截距不同，垂直線時它仍然有界。每個點 (x_k, y_k) 在 $\rho\theta$ 平面上變成一條正弦曲線，共線點的曲線會交在同一格。演算法如下：

1. 計算二值邊緣圖（例如用 Canny）。
2. 把 $\rho\theta$ 平面量化成**累加器格**（accumulator cells）$A(\rho, \theta)$，全部設為零。
3. 對每個邊緣像素和每個 θ，算出 ρ 並把 $A(\rho, \theta)$ 加一。
4. 計數大的局部極大值就是直線；也可以檢查投票像素的連續性，把直線切成線段。

```
# continues from the Canny snippet above (uses cv2, np and canny_cv)
import numpy as np

lines = cv2.HoughLines(canny_cv, rho=1, theta=np.pi / 180, threshold=120)
for rho, theta in lines[:5, 0]:
    print(f"rho = {rho:6.1f} px, theta = {np.degrees(theta):5.1f} deg")
```

OpenCV 回傳的 $\theta \in [0°, 180°)$，ρ 可為負，和上面是同一種表示法，只是範圍不同。投票的想法也能延伸到圓與其他參數更多的形狀。

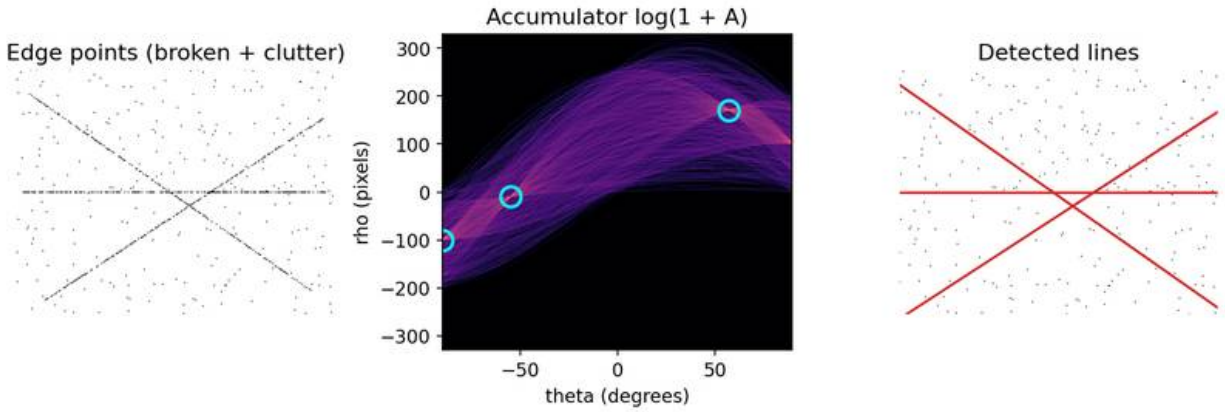


圖 *10.3 — Hough* 轉換。三條缺了 *35%* 像素的直線，外加隨機雜點（左）。每個點沿一條正弦曲線投票；累加器中圈起來的三個峰值（中）是大量曲線交會之處。把峰值換回直線，即使有缺口也能找回三條線（右）。

閾值化

白話版：選一個灰階，比它亮的是「物體」，比它暗的是「背景」。學問全在怎麼選。

基礎

給定強度閾值 T，全域閾值化（thresholding）得到

$$g(x,y) = \begin{cases} 1 & \text{if } f(x,y) > T \\ 0 & \text{otherwise.} \end{cases}$$

T 對整張影像固定時稱為**全域**閾值；隨位置改變時稱為**可變**閾值（也叫局部或自適應）；用兩個以上閾值則稱為**多重閾值化**。當直方圖的各個峰分得很開時，閾值化效果很好。**雜訊**會把峰變寬，**不均勻的照明與反射**會讓峰在影像各處移動，而很小的物體只會在直方圖上形成一個容易被忽略的小突起。

基本全域閾值化

當峰分得很清楚時，下面這個簡單的疊代規則就很好用：

1. 選一個初始 T，例如平均強度。
2. 把像素分成 G_1（值 $> T$）與 G_2（值 $\le T$）。
3. 計算兩組的平均 m_1、m_2。
4. 令 $T = \frac{1}{2}(m_1 + m_2)$。
5. 重複 2–4，直到 T 的變化小於某個 ΔT。

```
def basic_global(img, dT=0.5):
    T = img.mean()
    while True:
        m1, m2 = img[img > T].mean(), img[img <= T].mean()
        T_new = 0.5 * (m1 + m2)
        if abs(T_new - T) < dT:
            return T_new
        T = T_new
```

Otsu 最佳全域閾值

Otsu [5] 選的是讓兩類**可分離性**最大的閾值，以**類間變異數**（between-class variance）衡量。它只需要直方圖，因此很快。

設影像有 L 個灰階，灰階 i 有 n_i 個像素，總像素數為 MN，正規化直方圖為 $p_i = n_i/MN$。閾值 k 把灰階分成 $C_1 = [0,k]$ 與 $C_2 = [k+1, L-1]$。定義

$$P_1(k) = \sum_{i=0}^{k} p_i, \qquad m(k) = \sum_{i=0}^{k} i\,p_i, \qquad m_G = \sum_{i=0}^{L-1} i\,p_i,$$

其中 $P_1(k)$ 是像素落在 C_1 的機率，$m(k)$ 是到灰階 k 為止的累積平均，m_G 是全域平均。於是 $P_2 = 1 - P_1$，兩類的平均為 $m_1 = m/P_1$、$m_2 = (m_G - m)/P_2$。

推導。全域平均是 $m_G = P_1 m_1 + P_2 m_2$。類間變異數定義為兩類平均在全域平均周圍的加權散佈：

$$\sigma_B^2 = P_1(m_1 - m_G)^2 + P_2(m_2 - m_G)^2.$$

代入 $m_G = P_1 m_1 + P_2 m_2$，得到 $m_1 - m_G = P_2(m_1 - m_2)$ 與 $m_2 - m_G = P_1(m_2 - m_1)$，因此

$$\sigma_B^2 = P_1 P_2^2 (m_1 - m_2)^2 + P_2 P_1^2 (m_1 - m_2)^2 = P_1 P_2 (m_1 - m_2)^2.$$

兩類平均離得越遠，σ_B^2 越大。再代入 $m_1 = m/P_1$ 與 $m_2 = (m_G - m)/(1 - P_1)$，就得到只需要累積和的形式：

$$\sigma_B^2(k) = \frac{\big(m_G P_1(k) - m(k)\big)^2}{P_1(k)\,\big(1 - P_1(k)\big)}.$$

最佳閾值為 $k^* = \arg\max_k \sigma_B^2(k)$（若有多個並列就取平均）。由於全域變異數 σ_G^2 等於類間與類內變異數之和，而且與 k 無關，所以這同時也**最小化類內變異數**。比值

$$\eta = \frac{\sigma_B^2(k^*)}{\sigma_G^2}, \qquad 0 \le \eta \le 1,$$

衡量**可分離性**：兩個乾淨的峰時接近 1，峰重疊時較低。

```
import numpy as np
from skimage import data, filters
```

```
def otsu(img):
    p = np.bincount(img.ravel(), minlength=256) / img.size   # normalized histogram
    P1 = np.cumsum(p)                                         # class-1 probability
    m = np.cumsum(np.arange(256) * p)                         # cumulative mean
    mG = m[-1]                                                # global mean
    with np.errstate(divide="ignore", invalid="ignore"):
        sB2 = (mG * P1 - m) ** 2 / (P1 * (1 - P1))
    sB2 = np.nan_to_num(sB2)
    k = int(np.argmax(sB2))
    return k, sB2[k] / np.var(img)                            # threshold, separability

coins = data.coins()
k, eta = otsu(coins)          # k = 107, eta ≈ 0.76
assert k == filters.threshold_otsu(coins)
```

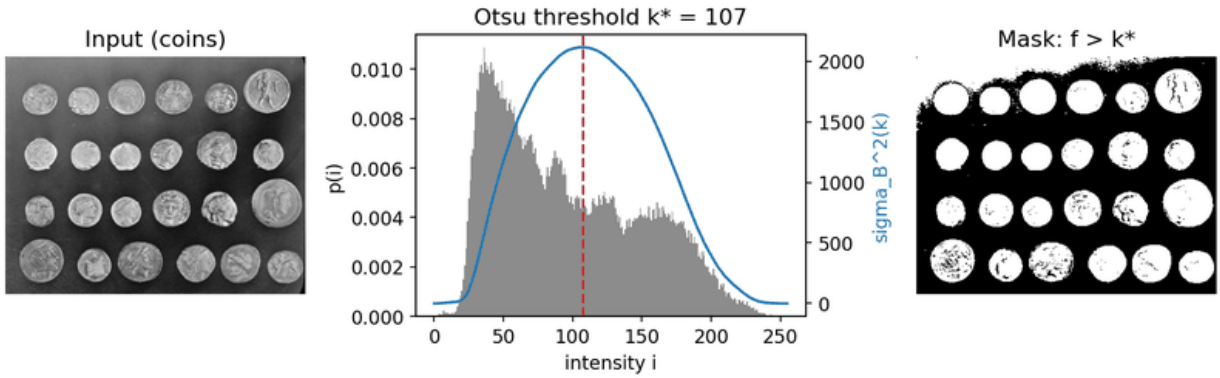


圖 *10.4* — 硬幣影像上的 *Otsu* 法。藍色曲線是 $\sigma_B^2(k)$，其最大值給出 $k^* = 107$（虛線）。遮罩（右）抓到了硬幣，但上緣那條較亮的背景帶也被選進來，提醒我們單一全域閾值無法修正不均勻照明。

用平滑與邊緣改善閾值化

- **先平滑**。雜訊會讓峰變寬直到合併；平滑可以讓峰再變窄，前提是物體比濾波器大得多。
- **只用邊緣附近的像素**。小物體的峰會被背景的峰淹沒。保留邊緣指標（梯度大小或 $|\nabla^2 f|$）很高的像素，例如高於第 99.7 百分位數者，並**只用這些像素**計算直方圖。它們大約一半落在邊界兩側，因此直方圖變得平衡而雙峰。再把得到的閾值套用到整張影像。

多重閾值

Otsu 準則可以推廣到用 $K-1$ 個閾值分出的 K 類：

$$\sigma_B^2 = \sum_{k=1}^{K} P_k (m_k - m_G)^2,$$

P_k、m_k 是第 k 類的機率與平均，以窮舉搜尋求解。`skimage.filters.threshold_multiotsu(coins, classes=3)` 回傳 `[77, 139]`。

可變閾值化

照明不均時，沒有任何單一 T 管用。三種常見對策：

1. **分塊**。每一塊用自己的 Otsu 值做閾值化；每一塊都必須同時含有兩類。
2. **局部性質**。令 m_{xy}、σ_{xy} 為 (x,y) 周圍視窗的平均與標準差，使用 $T_{xy} = a\,\sigma_{xy} + b\,m_{xy}$（$a, b \geq 0$），或更一般的述詞 $Q(\sigma_{xy}, m_{xy})$。Sauvola 與 Pietikäinen 的文件二值化規則 [7]，$T_{xy} = m_{xy}(1 + k(\sigma_{xy}/R - 1))$，$R$ 為 σ 的動態範圍、k 為小正數，就屬於這一族。
3. **移動平均**。對文字影像以之字形掃描，以 $c \cdot m(k)$ 做閾值，其中 $m(k)$ 是最近 n 個像素的平均，c 略小於 1。

```
from skimage import data, filters

page = data.page()
global_mask = page > filters.threshold_otsu(page)
local_T = filters.threshold_local(page, block_size=35, offset=10)  # Gaussian-weighted local
↪  mean - 10
local_mask = page > local_T
sauvola_mask = page > filters.threshold_sauvola(page, window_size=25, k=0.2)
```

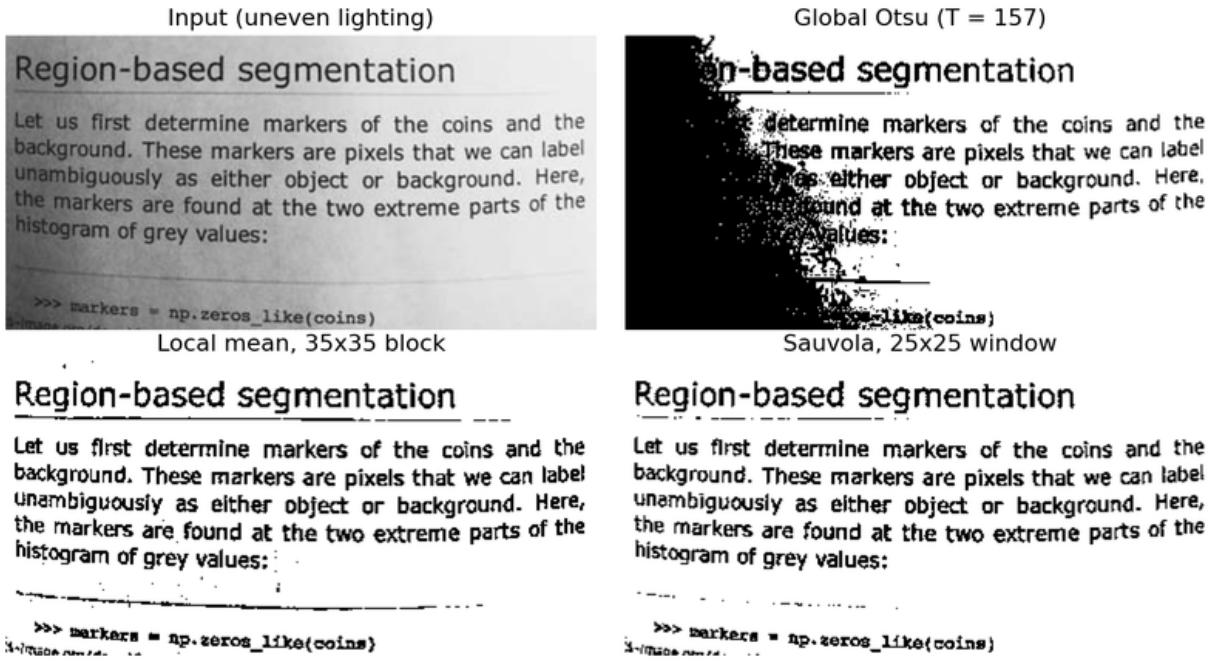


圖 *10.5* — 照明不均文字的可變閾值化。全域 *Otsu* 閾值（右上）把陰影中的左半邊整片塗黑；局部平均閾值（左下）與 *Sauvola* 規則（右下）則在各處都救回了文字。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

區域成長、分裂與合併

白話版：從幾個「種子」像素出發，不斷加入長得像它們的鄰居；或者從整張影像出發，不斷切成四塊，直到每一塊都均勻為止。

區域成長

區域成長（region growing）把滿足相似性述詞的鄰近像素加入每個**種子**。需要決定的有：**種子**（人工指定、用很保守的閾值選出，或取自直方圖峰值）、**述詞**（例如 $|f(x,y) - f(\text{seed})| \le T$，或對區域統計量的測試）、**連通性**（4 或 8 連通），以及**停止規則**（沒有鄰居通過測試時停止；加上大小或形狀限制會更穩健）。

```
from collections import deque
import numpy as np

def region_grow(img, seed, tol):
    """Grow from `seed` over 8-connected pixels whose value is within `tol` of the seed
     ↪  value."""
    H, W = img.shape
    ref = float(img[seed])
    mask = np.zeros((H, W), bool)
    mask[seed] = True
    q = deque([seed])
    while q:
        r, c = q.popleft()
        for dr in (-1, 0, 1):
            for dc in (-1, 0, 1):
                rr, cc = r + dr, c + dc
                if 0 <= rr < H and 0 <= cc < W and not mask[rr, cc] \
                        and abs(float(img[rr, cc]) - ref) <= tol:
                    mask[rr, cc] = True
                    q.append((rr, cc))
    return mask
```

`skimage.segmentation.flood(img, seed, tolerance=tol)` 以編譯過的程式碼做同樣的事。

以四元樹做區域分裂與合併

不用種子，而是從整張影像 R 與述詞 Q 開始：

1. 對 $Q(R_i) = \text{FALSE}$ 的區域 R_i，**分裂**成四個等大的象限。
2. 重複，直到所有區域都通過 Q，或達到最小尺寸。

3. 對任意兩個相鄰區域 R_j, R_k，若 $Q(R_j \cup R_k) = \text{TRUE}$ 就**合併**。
4. 無法再分裂或合併時停止。

分裂的過程以**四元樹**（quadtree）儲存：根節點是整張影像，每個節點有四個子節點。只分裂會得到方塊狀的區域；合併才讓區域能跨越象限邊界。典型的述詞如「標準差大於 a，且平均介於 b 與 c 之間」。

以分群與超像素做區域分割

白話版：把每個像素當成用幾個數字（顏色、或許加上位置）描述的點，再把靠得近的點聚成一群。

k-means 分群

把每個像素表示成特徵向量 $\mathbf{z} \in \mathbb{R}^n$，例如它的 Lab 顏色（$n = 3$）。**k-means** 尋找 k 個群中心 $\mathbf{m}_1, \ldots, \mathbf{m}_k$ 與集合 $C_1, \ldots, C_k$，使

$$J = \sum_{i=1}^{k} \sum_{\mathbf{z} \in C_i} \|\mathbf{z} - \mathbf{m}_i\|^2$$

最小，也就是到群平均的總平方距離。求全域最小值是 NP-hard，所以標準演算法交替進行：把每個向量指派給最近的中心，再把每個中心移到其成員的平均。它只會收斂到局部最小值，因此要從多個初始值執行。只用顏色分群會忽略位置，所以同一群可能散布在整張影像上。

超像素與 SLIC

超像素（superpixel）是一小塊緊密、相似且連通的像素。用幾百個超像素取代 10^5 個像素，可以讓後續問題變小，又不會移動重要的邊界。Achanta 等人的 **SLIC** [8] 就是在五維空間 $[l, a, b, x, y]$ 中做 k-means。設有 N 個像素、想要 K 個超像素，格距為 $S = \sqrt{N/K}$。

1. 在格距為 S 的網格上放 K 個中心，每個再移到其 3×3 鄰域中梯度最小的位置。
2. 每個中心**只搜尋周圍** $2S \times 2S$ **的視窗**，依下列距離指派像素

$$D = \sqrt{d_c^2 + \left(\frac{d_s}{S}\right)^2 m^2},$$

其中 d_c、d_s 分別是 Lab 顏色與 (x, y) 上的歐氏距離，m 是**緊緻度**（compactness）：m 大時超像素接近方形，m 小時會緊貼顏色邊緣。3. 把每個中心移到其像素的平均；重複約十次。4. **強制連通性**：把零星碎片併入相鄰的超像素。

局部搜尋讓每次疊代的成本為 $O(N)$，與 K 無關。

```
from skimage import data, segmentation
```

```
img = data.astronaut()
sp = segmentation.slic(img, n_segments=300, compactness=10, start_label=1)
```

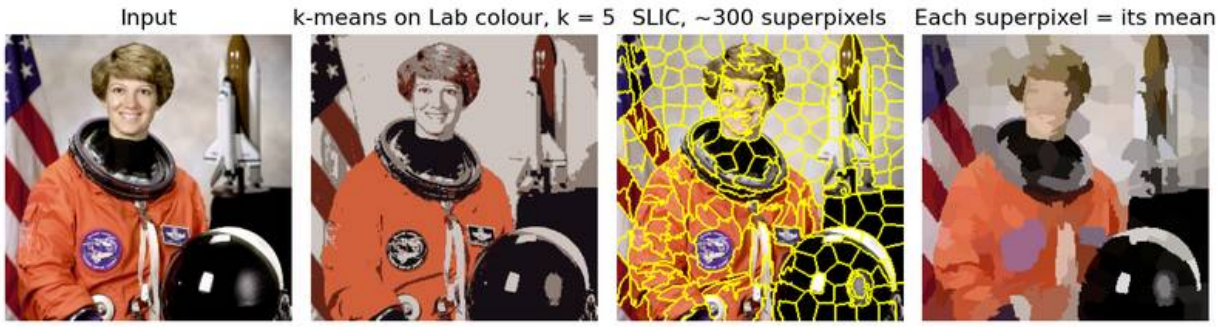


圖 *10.6* — 分群。在 *Lab* 顏色上做 *k-means*（第二張）把影像簡化成五種顏色，但每一群都散布在畫面各處。*SLIC*（第三張，緊緻度 *10*）產生約 *300* 個沿著強邊緣的緊密超像素；把每個超像素塗上其平均色（右），只用極少的單位就保留了可辨識的畫面。

以圖切割做區域分割

白話版：把影像畫成一張網，相鄰像素之間綁著繩子，像的就綁粗繩、不像的就綁細繩。分割就是只剪斷細繩，把網分成幾塊。

把影像視為圖

圖（graph）$G = (V, E)$ 由節點 V 與邊 E 組成。每個像素（或超像素）是一個節點，鄰近節點之間以非負的相似度權重 $w(i, j)$ 相連，例如

$$w(i,j) = \exp\!\left(-\frac{\|\mathbf{F}_i - \mathbf{F}_j\|^2}{\sigma_I^2}\right)\exp\!\left(-\frac{\|\mathbf{X}_i - \mathbf{X}_j\|^2}{\sigma_X^2}\right) \quad \text{if } \|\mathbf{X}_i - \mathbf{X}_j\| \le r, \text{ else } 0,$$

其中 $\mathbf{F}$ 是特徵向量（強度、顏色），$\mathbf{X}$ 是空間位置，σ_I、σ_X 是尺度參數，r 是讓圖保持稀疏的半徑。

最小圖切割

切割（cut）把 V 分成兩個不相交的集合 A、B，並移除兩者之間的邊。其成本為

$$\operatorname{cut}(A, B) = \sum_{u \in A,\, v \in B} w(u, v).$$

最小切割移除的相似度最少。一種常見的形式會加入一個**源點**（source，代表物體）與一個**匯點**（sink，代表背景），並以「多像物體、多像背景」的權重連到每個像素。最小源–匯切割會替每個像素貼上標籤，而且能用最大流演算法有效率地求得；使用者只要在物體與背景上各畫幾筆的互動式工具，就是以此為基礎。

正規化切割

沒有源點與匯點時，單純的最小切割偏好切下單一節點或極小的一群，因為要移除的邊很少。Shi 與 Malik [9] 以**正規化切割**（normalized cut）修正這個問題：

$$\mathrm{Ncut}(A,B) = \frac{\mathrm{cut}(A,B)}{\mathrm{assoc}(A,V)} + \frac{\mathrm{cut}(A,B)}{\mathrm{assoc}(B,V)}, \qquad \mathrm{assoc}(A,V) = \sum_{u\in A,\, t\in V} w(u,t),$$

$\mathrm{assoc}(A,V)$ 是 A 到所有節點的總連結。極小的 A 其 assoc 也很小，因此切下它變得很貴。精確最小化是 NP-hard，但鬆弛後可化為特徵值問題；令 W 為權重矩陣、D 為度數 $d_i = \sum_j w(i,j)$ 構成的對角矩陣，

$$(D - W)\,\mathbf{y} = \lambda D\,\mathbf{y}.$$

第二小特徵值對應的特徵向量是一個連續的分割指標；對它做閾值化就能把圖一分為二，遞迴下去就得到更多區塊。實務上會在超像素圖上執行。

```
import numpy as np
from skimage import data, segmentation, graph

img = data.coffee()
sp = segmentation.slic(img, n_segments=400, compactness=30, start_label=1)
rag = graph.rag_mean_color(img, sp, mode="similarity")   # edge weight = exp(-|c_i - c_j|^2
 ↪  / sigma)
ncut = graph.cut_normalized(sp, rag, rng=0)              # recursive two-way Ncut
print(sp.max(), "superpixels ->", len(np.unique(ncut)), "regions")
```

以形態學分水嶺做分割

白話版：把影像想成一片地形，越亮越高。從每個山谷灌水；兩個山谷的水快要相遇時，就築一道牆。這些牆就是分割結果。

背景：集水盆與水壩

把強度當成高度。每個**區域極小值**（regional minimum）都有一個**集水盆**（catchment basin），也就是水會流到它的所有點；**分水線**把相鄰的集水盆隔開。通常淹的是**梯度大小**而不是原影像：物體內部低、邊界是山脊，因此每個集水盆就是一個物體。

水壩用到第 9 章的形態學。當兩個已淹水的連通成分快要合併時，以 3×3 結構元素分別膨脹它們，並限制在低於目前水位的像素內；在同一步被兩者同時碰到的像素就是水壩像素，設成比影像最大值還高。這樣築出的水壩只有一個像素寬，而且是連通的。

分水嶺演算法

令 $M_1, \dots, M_R$ 為影像 g 的區域極小值，min、max 為其極值，$T[n] = \{(s,t) : g(s,t) < n\}$ 為低於水位 n 的像素集合，$C[n]$ 為水位 n 時所有集水盆已淹部分的聯集。淹水從 $n = \min + 1$ 進行到 $\max + 1$，以 $C[\min+1] = T[\min+1]$ 開始。在每個水位，對 $T[n]$ 的每個連通成分 q 與 $C[n-1]$ 比較：

1. $q \cap C[n-1]$ 為空：出現新的極小值，q 成為新的集水盆。
2. $q \cap C[n-1]$ 只含 $C[n-1]$ 的一個成分：q 位於某個既有集水盆中，把它加進去。
3. $q \cap C[n-1]$ 含兩個以上成分：集水盆正在相遇，依上述方法在 q 中築壩。

Vincent 與 Soille [10] 提出今日通用的高效實作：先依高度排序像素，再以 FIFO 佇列逐層淹水（「浸沒模擬」，immersion simulation）。

使用標記

直接用在梯度上，分水嶺會嚴重**過度分割**：雜訊與紋理造成成千上萬個極小值。解法是**標記**（markers），也就是事先確定位於物體內（**內部標記**）或背景中（**外部標記**）的連通成分。淹水只從標記開始，所以區域數就等於標記數。標記可以來自保守的閾值、平滑後的極小值、距離轉換的峰值，或使用者的點擊。

```
import numpy as np
from scipy import ndimage as ndi
from skimage import data, filters, morphology, segmentation

coins = data.coins()
elevation = filters.sobel(coins.astype(float))
markers = np.zeros_like(coins, dtype=np.int32)
markers[coins < 30] = 1        # sure background
markers[coins > 150] = 2       # sure object
ws = segmentation.watershed(elevation, markers)
coins_mask = morphology.remove_small_objects(ndi.binary_fill_holes(ws == 2), 100)
labels, n = ndi.label(coins_mask)   # n = 24 coins
```

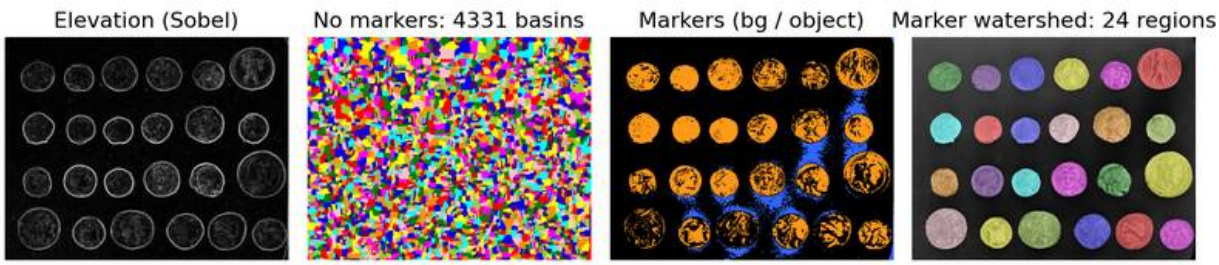


圖 *10.7* — 硬幣影像上的分水嶺。從每個區域極小值淹平滑後的梯度，會得到數千個集水盆（第二張）。用兩個非常保守的閾值產生兩種標記（第三張）後，水正好在硬幣邊緣相遇；填洞並去掉小斑點後，每枚硬幣各成一個區域（右）。

利用運動做分割

白話版：如果攝影機不動，兩張畫面之間有變的東西，就是你在意的東西。把兩張畫面相減，移動的物體就亮起來了。

空間技術：差異影像

給定靜止攝影機拍攝靜態場景的兩張畫面 $f(x,y,t_i)$ 與 $f(x,y,t_j)$，**差異影像**（difference image）為

$$d_{ij}(x,y) = \begin{cases} 1 & \text{if } |f(x,y,t_i) - f(x,y,t_j)| > T \\ 0 & \text{otherwise,} \end{cases}$$

T 取略高於雜訊水準的值。雜訊造成的小點，可以靠丟棄太小的連通成分去除。

累積差異

單一差異影像會把移動物體顯示兩次：它原本在的地方和現在在的地方。**累積差異影像**（accumulative difference image, ADI）把**參考畫面** $R = f(x,y,t_1)$ 與之後每張畫面比較並計數：

$$\begin{aligned} A_k &= A_{k-1} + 1 \ \text{ if } |R - f_k| > T, \\ P_k &= P_{k-1} + 1 \ \text{ if } R - f_k > T, \\ N_k &= N_{k-1} + 1 \ \text{ if } R - f_k < -T, \end{aligned}$$

其他情況維持不變，初始為零（$f_k = f(x,y,t_k)$）。若物體比背景亮，**正 ADI** P 標出物體在參考畫面中的位置；**負 ADI** 朝運動方向增長，增長速度取決於物體速度；**絕對 ADI** A 則兩者都包含。

```
import numpy as np

def accumulative_differences(frames, T):
    R = frames[0].astype(float)
    A = np.zeros(R.shape, int); P = np.zeros(R.shape, int); N = np.zeros(R.shape, int)
    for f in frames[1:]:
        d = R - f.astype(float)
        A += np.abs(d) > T
        P += d > T
        N += d < -T
    return A, P, N
```

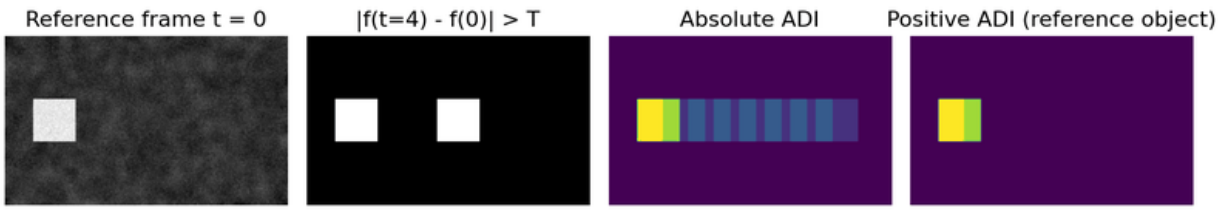


圖 *10.8* — 運動。一個亮方塊在八張畫面中向右移動。單一差異影像（第二張）把方塊顯示了兩次；絕對 *ADI*（第三張）記錄了它到過的每個位置；正 *ADI*（右）只留下它的起點。

建立參考影像

不含任何移動物體的參考畫面很少見。當正 ADI 顯示某物體已完全離開它的起始位置時，就從目前畫面把那些像素複製到參考影像；對每個移動物體都這麼做，就得到一張**背景參考影像**。背景相減（background subtraction）方法則以逐像素的統計模型延伸這個想法。

現代觀點

以下五篇綜述描繪了這個領域，並說明深度學習改變了什麼、沒改變什麼。

傳統閾值化。 Sezgin 與 Sankur [6] 依所利用的資訊，把閾值化方法分成六族（直方圖形狀、量測空間分群、熵、物體屬性、空間相關、局部灰階曲面），並在非破壞檢測與文件影像上評估其中四十種，挑出在兩個領域都表現穩定的方法。Otsu 法屬於分群這一族。

深度分割。 Minaee 等人 [11] 綜述深度學習的語意分割與實例分割，把模型分成全卷積網路、編碼器–解碼器、多尺度與金字塔模型、遞迴網路、注意力模型與對抗式生成模型，並回顧了資料集與結果。幾個里程碑設計都呼應著傳統想法：

- **FCN** [13] 把分類網路改成全卷積，讓它輸出標籤圖，並融合深層粗略特徵與淺層精細特徵：一種學出來的多尺度分析。
- **U-Net** [14] 加上對稱的擴張路徑與跳接（skip connection），用少量訓練資料就能得到精確邊界，成為生醫影像的預設選擇之一。
- **DeepLab** [16] 以空洞卷積（atrous convolution）與空洞空間金字塔池化兼顧解析度與脈絡，再用全連接 CRF 讓邊界更銳利。
- **Mask R-CNN** [15] 在物體偵測器上加一個遮罩分支，能分開同類別而互相接觸的物體，也就是分水嶺中標記所做的工作。

基礎模型與可提示分割。 Segment Anything（SAM）[17] 是可用點、框或遮罩當提示的模型，訓練於 SA-1B：1,100 萬張影像上的十億個以上遮罩。點一下、回傳一個遮罩，這正是種子區域成長與互動式圖切割的介面，只是相似度改由學習而來。Zhou 等人 [12] 回顧了這個「基礎模型時代」的三百多種方法，分為一般任務（語意、實例、全景分割）與可提示任務（互動式、指稱式、少樣本），並說明大型預訓練模型如何帶有分割知識。

邊緣偵測。 Sun 等人 [18] 把傳統邊緣偵測器分成梯度、高斯差、多尺度與結構化學習四類，把深度方法分成編碼器–解碼器、網路重構與多尺度融合三類。HED [19] 是關鍵的早期深度設計：把網路多個深度的側輸出融合成一張邊緣圖，可說是 Marr 與 Hildreth 多尺度論點的學習版後代。該綜述的結論是：學習式偵測器在標準評測上已接近甚至超越人類水準，剩下的開放問題是輕量化模型、弱監督與可解釋性。

超像素。 Stutz、Hermans 與 Leibe [20] 評測了 28 種演算法，強調要調好參數並**嚴格強制連通性**，提出與超像素數量無關的指標，並測試對雜訊、模糊與仿射轉換的穩健性。

改變了什麼、沒改變什麼。學習出的特徵取代了手工設計的述詞與邊緣強度，在自然影像上尤其

明顯。但傳統的骨架還在：網路輸出的是機率圖，仍然要做閾值化；互相接觸的實例仍需要分開，靠的是偵測框、學出來的標記，或在預測的距離圖上跑分水嶺；而在資料稀少、運算受限或行為必須可預測的場合，例如文件掃描與工業檢測，Otsu、Canny 與 Hough 仍然勝出。

重點整理

- 分割把影像劃分成連通、不重疊、各自滿足述詞 Q 的區域；方法不是找**不連續**（邊緣），就是找**相似**（區域）。
- 一階導數給出粗的邊緣響應；二階導數給出雙重響應，邊緣中心是零交越，而且對雜訊非常敏感，所以要先平滑再微分。
- Canny = 高斯平滑 + 梯度 + 非極大值抑制 + 帶**遲滯**的雙閾值；最後一步讓微弱但連通的邊緣得以保留。
- Hough 轉換把找直線變成在 (ρ, θ) 空間投票，對缺口與雜點很穩健。
- Otsu 閾值最大化類間變異數 $\sigma_B^2 = P_1 P_2 (m_1 - m_2)^2$，可由直方圖的累積和算出；照明不均時改用局部閾值。
- 區域方法（成長、分裂合併、k-means、SLIC、正規化切割）依相似性分組；正規化切割避免了單純最小切割偏好小區塊的問題。
- 分水嶺從極小值開始淹梯度影像，在水相遇處築壩；要避免過度分割，**標記**不可或缺。
- 深度網路學會了相似度與邊緣的衡量方式，但閾值化、多尺度融合、種子或提示、實例分離，仍是現代流程的骨架。

練習

1. 一個一維訊號先有五個樣本為 10，接著在四個樣本內線性上升到 50，之後維持 50。寫出它的一階與二階差分，並標出二階差分的零交越相對於斜坡的位置。

   提示

   一階差分在四個斜坡步上都是 10，其他地方為 0。二階差分在斜坡起點為 $+10$、終點為 -10，中間為 0，所以零交越在斜坡中段。

2. 證明最大化類間變異數 σ_B^2 等價於最小化加權類內變異數 $\sigma_W^2 = P_1 \sigma_1^2 + P_2 \sigma_2^2$。

   提示

   把 $\sum_i (i - m_G)^2 p_i$ 在 k 處拆開，每一類內加減該類平均；交叉項為零，剩下 $\sigma_G^2 = \sigma_W^2 + \sigma_B^2$，而 σ_G^2 與 k 無關。

3. 在 Canny 偵測器中，若設 $T_L = T_H$ 會怎樣？若 $T_L = 0$ 呢？先預測結果，再用 `skimage.feature.canny` 在 `data.camera()` 上驗證。

   提示

   $T_L = T_H$ 等於關掉遲滯：輪廓會斷掉。$T_L = 0$ 會保留每個與強邊緣相連、通過細化的像素，所以邊緣會滲進草地紋理。

4. 一個 Hough 累加器對 640×480 的影像，θ 在 $[-90°, 90°)$ 間以 $1°$ 為一格，ρ 以 1 像素為一格。累加器共有幾格？一張有 20,000 個邊緣像素的邊緣圖會造成多少次加一？

提示

$|\rho| \le \sqrt{640^2 + 480^2} = 800$，所以約有 1,601 個 ρ 格與 180 個 θ 格，約 288,000 格。每個邊緣像素對每個 θ 投一票：$20{,}000 \times 180 = 360$ 萬次。

5. 以權重均為 w 的 4 連通像素圖，說明為什麼單純的最小切割傾向切下單一像素。再計算在 N 個像素的影像中切下一個內部像素的 Ncut，說明它接近 1，而沿著短邊界的平衡切割則接近 0。

提示

切下一個像素只需成本 $4w$，比任何長邊界都小。對 Ncut 而言，$\text{assoc}(A, V) = 4w$，第一項為 1，第二項極小：Ncut ≈ 1，是很差的分數。

6. 依上文在 `data.coins()` 上執行標記控制分水嶺（背景標記 `coins < 30`、物體標記 `coins > 150`），再把物體閾值改成 120、190 與 230。哪些硬幣會合併或消失？為什麼？

提示

在 120 時，物體標記落在上方較亮的背景帶上，硬幣與背景合併成大區域（我們數到 11 個）。在 190 時仍有 24 個，因為每枚硬幣只要一個標記像素就夠；到 230 時，較暗的硬幣失去標記而消失（剩 13 個）。標記決定了結果。

參考文獻

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 10. publisher page
2. D. Marr and E. Hildreth, "Theory of edge detection," *Proceedings of the Royal Society of London. Series B*, vol. 207, no. 1167, pp. 187–217, 1980. doi
3. J. Canny, "A Computational Approach to Edge Detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, pp. 679–698, 1986. doi
4. R. O. Duda and P. E. Hart, "Use of the Hough transformation to detect lines and curves in pictures," *Communications of the ACM*, vol. 15, pp. 11–15, 1972. doi
5. N. Otsu, "A Threshold Selection Method from Gray-Level Histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979. doi
6. M. Sezgin and B. Sankur, "Survey over image thresholding techniques and quantitative performance evaluation," *Journal of Electronic Imaging*, vol. 13, no. 1, pp. 146–165, 2004. doi
7. J. Sauvola and M. Pietikäinen, "Adaptive document image binarization," *Pattern Recognition*, vol. 33, no. 2, pp. 225–236, 2000. doi
8. R. Achanta, A. Shaji, K. Smith, A. Lucchi, P. Fua and S. Süsstrunk, "SLIC Superpixels Compared to State-of-the-Art Superpixel Methods," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 34, no. 11, pp. 2274–2282, 2012. doi

9. J. Shi and J. Malik, "Normalized cuts and image segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 8, pp. 888–905, 2000. doi
10. L. Vincent and P. Soille, "Watersheds in digital spaces: an efficient algorithm based on immersion simulations," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 6, pp. 583–598, 1991. doi
11. S. Minaee, Y. Boykov, F. Porikli, A. Plaza, N. Kehtarnavaz and D. Terzopoulos, "Image Segmentation Using Deep Learning: A Survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 7, pp. 3523–3542, 2022. doi ·arXiv
12. T. Zhou, W. Xia, F. Zhang, B. Chang, W. Wang, Y. Yuan, E. Konukoglu and D. Cremers, "Image Segmentation in Foundation Model Era: A Survey," arXiv:2408.12957, 2024. arXiv
13. J. Long, E. Shelhamer and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," arXiv:1411.4038, 2014. arXiv
14. O. Ronneberger, P. Fischer and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," MICCAI 2015, arXiv:1505.04597. arXiv
15. K. He, G. Gkioxari, P. Dollár and R. Girshick, "Mask R-CNN," arXiv:1703.06870, 2017. arXiv
16. L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy and A. L. Yuille, "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 4, pp. 834–848, 2018. doi ·arXiv
17. A. Kirillov, E. Mintun, N. Ravi et al., "Segment Anything," arXiv:2304.02643, 2023. arXiv
18. R. Sun, T. Lei, Q. Chen, Z. Wang, X. Du, W. Zhao and A. K. Nandi, "Survey of Image Edge Detection," *Frontiers in Signal Processing*, 2022. doi
19. S. Xie and Z. Tu, "Holistically-Nested Edge Detection," arXiv:1504.06375, 2015. arXiv
20. D. Stutz, A. Hermans and B. Leibe, "Superpixels: An Evaluation of the State-of-the-Art," *Computer Vision and Image Understanding*, vol. 166, pp. 1–27, 2018. doi

IP 11・影像分割（二）：主動輪廓——蛇模型與水平集

用「會動的曲線」分割物體：參數式蛇模型、梯度向量流、氣球力，以及測地主動輪廓與 Chan–Vese 等水平集方法。

網站版本: https://blog.shyandram.dev/zh-tw/learn/image-processing/dip-11-active-contours/

你將學到

- 為什麼可以把分割問題寫成「移動一條曲線，直到能量不再下降」，以及這樣做比門檻化和邊緣連接多了什麼好處。
- **蛇模型**（snake，一種顯式、參數式的主動輪廓）如何由內部能量與外部能量定義，以及 Euler–Lagrange 方程式如何把能量變成迭代演算法。
- 外力該怎麼選：影像梯度、邊緣圖、**梯度向量流**（gradient vector flow, GVF）與**氣球力**（balloon force），以及它們各自要解決的問題。
- **水平集方法**（level-set method）如何把曲線隱式表示為函數 ϕ 的零等高線，並在 CFL 限制下以迎風有限差分求解 $\phi_t + F|\nabla\phi| = 0$。
- 速度函數如何編碼以邊緣為主的模型（**測地主動輪廓**）與以區域為主的模型（源自 Mumford–Shah 的 **Chan–Vese**），以及符號距離重新初始化、窄帶法與形態學近似。
- 主動輪廓在今天的位置：成為深度網路裡的輪廓精修模組，以及損失函數。

先看全貌

第 10 章用的是局部規則：這個像素過門檻、那段邊緣連起來、這塊區域往外長。這些規則完全不在乎結果的**形狀**。於是我們可能得到有缺口的邊界、從弱邊緣漏出去的區域，或是一堆零散的孤立像素。

主動輪廓（active contour）換了一個角度。先在物體附近放一條封閉曲線，然後讓它**動起來**。影像推著曲線走（朝向邊緣，或朝向「裡面」與「外面」分得最好的位置），曲線本身的彈性又把它拉住（不喜歡被拉長、也不喜歡被折彎）。當兩邊的力互相抵消，曲線停下來，它最後的位置就是分割結果。輸出永遠是一條封閉、平滑的邊界，而不是一堆邊緣碎片。

> **用最簡單的話說**
>
> 在桌上的玩具外面套一條大橡皮筋，然後讓橡皮筋慢慢縮小。它會往內滑，碰到玩具就停住，緊緊貼著玩具的輪廓。橡皮筋自己想要變短、變圓滑；玩具擋住它，不讓它再縮。蛇模型就是那條橡皮筋，而「玩具」就是圖片裡邊緣很明顯的地方。水平集則是更聰明的橡皮筋：如果裡面有兩個玩具，它還會自己「啪」一聲分成兩條。

本章依照課本 [1] 的編排，依序介紹這兩大家族。**蛇模型**（Kass、Witkin 與 Terzopoulos [2]）把曲線存成一串點，直接移動這些點。**水平集**（Osher 與 Sethian [5]）把曲線隱式地存成某個曲面的

零等高線，改為移動曲面，因此曲線的分裂與合併會自動發生。兩者都是**變分法**（variational）：先寫下一個能量，再對它做梯度下降。這個想法本身，比任何一條特定公式都更長壽，一路延續到了深度學習。

背景：為什麼要讓曲線動起來

白話地說：與其逐一判斷每個像素，不如一次描述整條邊界，然後問「這條邊界有多好？」，再一點一點改進它。

精確地說，主動輪廓方法有三個要素：

1. 封閉曲線 C 在影像定義域 $\Omega \subset \mathbb{R}^2$ 中的**表示法**：顯式（參數化曲線 $v(s)$）或隱式（函數 $\phi(x, y)$ 的零水平集）。
2. 一個**能量** $E(C)$，當 C 是好邊界時能量低。它總是混合了**規則性**項（偏好短而平滑的曲線）與**資料**項（曲線上的邊緣很強，或曲線內外兩區各自很均勻）。
3. 一條**演化規則**，改變 C 讓 E 下降，通常是以人工時間 t 寫成的偏微分方程式（PDE）形式的梯度下降。

和第 10 章相比，規則性項能補上邊緣的缺口，輸出永遠是可以直接交給第 12 章形狀描述子的封閉曲線，而先驗知識（使用者點一下的位置、對平滑的偏好、物體內部的灰階模型）也能自然地放進來。代價是能量不是凸函數：梯度下降只會找到**局部**最小值，所以結果取決於起點。

以蛇模型做影像分割

蛇模型是一串在影像上滑動的封閉點鏈。每個點同時感受到兩種拉力：相鄰的點讓鏈子保持短而平滑，影像則把它拖向邊緣。

蛇模型是一條參數曲線

我們把輪廓寫成

$$v(s) = (x(s), y(s)), \qquad s \in [0, 1], \quad v(0) = v(1),$$

其中 s 是沿曲線繞一圈的參數，$x(s)$、$y(s)$ 是影像座標。對 s 的導數記為 $v_s = dv/ds$、$v_{ss} = d^2v/ds^2$。在電腦裡，曲線被取樣成 n 個點 $v_i = (x_i, y_i)$，$i = 0, \dots, n-1$；對封閉蛇模型而言，索引以 n 取模。

能量泛函

[2] 的蛇模型能量是

$$E_{\text{snake}}(v) = \int_0^1 \Big[\underbrace{\tfrac{1}{2}\left(\alpha\,|v_s(s)|^2 + \beta\,|v_{ss}(s)|^2\right)}_{E_{\text{int}}} + E_{\text{ext}}(v(s)) \Big]\, ds.$$

- $\alpha \geq 0$ 是**彈性**（張力）權重。$|v_s|^2$ 在曲線被拉長時變大，所以 α 讓蛇模型像一條想縮短的鬆緊帶。
- $\beta \geq 0$ 是**剛性**（彎曲）權重。$|v_{ss}|^2$ 衡量曲線轉彎有多急，所以 β 讓蛇模型像一根不肯被折彎的細金屬棒。
- $E_{\text{ext}}(x,y)$ 是**外部（影像）能量**，一個定義在整張影像上的純量場，在我們希望蛇停下的地方數值**較低**。

對灰階影像 f，常見的外部能量是

$$E_{\text{ext}}(x,y) = -\,w_{\text{edge}}\,|\nabla (G_\sigma * f)(x,y)|^2 \;-\; w_{\text{line}}\,(G_\sigma * f)(x,y),$$

其中 G_σ 是標準差為 σ 的高斯函數，$*$ 是卷積，∇ 是梯度，w_{edge}、w_{line} 是權重。$w_{\text{edge}} > 0$ 時蛇被強邊緣吸引；$w_{\text{line}} > 0$ 時被暗線吸引，$w_{\text{line}} < 0$ 時被亮線吸引。高斯模糊把 E_{ext} 的谷底變寬，讓蛇在幾個像素外就能「感覺」到邊緣；[2] 稱之為尺度空間延續（scale-space continuation）。

從能量到運動：Euler–Lagrange 方程式

要最小化 $\int L(v, v_s, v_{ss})\,ds$ 這種積分，我們用變分法。最小值必須滿足 Euler–Lagrange 方程式

$$\frac{\partial L}{\partial v} - \frac{d}{ds}\frac{\partial L}{\partial v_s} + \frac{d^2}{ds^2}\frac{\partial L}{\partial v_{ss}} = 0.$$

當 $L = \tfrac{1}{2}\alpha|v_s|^2 + \tfrac{1}{2}\beta|v_{ss}|^2 + E_{\text{ext}}(v)$，有 $\partial L/\partial v = \nabla E_{\text{ext}}$、$\partial L/\partial v_s = \alpha v_s$、$\partial L/\partial v_{ss} = \beta v_{ss}$。若 α、β 為常數，便得到

$$\alpha\, v_{ss} - \beta\, v_{ssss} - \nabla E_{\text{ext}}(v) = 0.$$

把它讀成力平衡：αv_{ss} 是張力（指向局部曲率中心，讓曲線縮短），$-\beta v_{ssss}$ 是彎曲力（把折角拉直），$F_{\text{ext}} = -\nabla E_{\text{ext}}$ 是影像力。平衡時三者互相抵消。

這條方程式通常無法直接解，所以讓 v 隨人工時間 t 變化，沿著下坡滑動：

$$\gamma\,\frac{\partial v}{\partial t} = \alpha\, v_{ss} - \beta\, v_{ssss} + F_{\text{ext}}(v),$$

其中 $\gamma > 0$ 是黏滯（阻尼）常數。當 $\partial v/\partial t = 0$，就回到 Euler–Lagrange 方程式。

離散化與迭代解法

以單位間距取樣 n 個點，中央差分給出

$$v_{ss} \approx v_{i+1} - 2v_i + v_{i-1}, \qquad v_{ssss} \approx v_{i+2} - 4v_{i+1} + 6v_i - 4v_{i-1} + v_{i-2}.$$

把所有 x 座標排成向量 $\mathbf{x} \in \mathbb{R}^n$（$\mathbf{y}$ 同理），則 $\alpha v_{ss} - \beta v_{ssss} = -A\mathbf{x}$，其中 A 是對稱、循環、五對角的 $n \times n$ 矩陣，每一列以對角線為中心為

$$[\;\beta,\;\; -\alpha - 4\beta,\;\; 2\alpha + 6\beta,\;\; -\alpha - 4\beta,\;\; \beta\;]$$

（在頭尾處循環繞回）。內力對點的位置是線性的，外力則不是。[2] 的經典技巧是**半隱式**步驟：內力部分以新時刻 t 隱式處理，外力部分以舊時刻 $t-1$ 顯式處理：

$$\gamma\,(\mathbf{x}^{t}-\mathbf{x}^{t-1}) = -A\,\mathbf{x}^{t} + \mathbf{f}_x(\mathbf{x}^{t-1},\mathbf{y}^{t-1}) \quad\Longrightarrow\quad \mathbf{x}^{t} = (A+\gamma I)^{-1}(\gamma\,\mathbf{x}^{t-1} + \mathbf{f}_x),$$

$\mathbf{y}$ 則搭配 $\mathbf{f}_y$ 做同樣的事。$\mathbf{f}_x$、$\mathbf{f}_y$ 是 F_{ext} 在目前點位上（以內插）取得的分量，I 是單位矩陣。$(A+\gamma I)$ 固定不變，只需反矩陣或分解一次；之後每次迭代只是一次便宜的矩陣–向量乘法。把剛硬的四階項隱式處理，正是蛇模型能走合理步長而不爆掉的原因。步長是 $1/\gamma$：γ 越大，步伐越小越安全。

```
import numpy as np

def snake_step_matrix(n, alpha, beta, gamma):
    """(A + gamma*I)^-1 for a closed snake with n points (h = 1)."""
    row = np.zeros(n)
    row[[0, 1, 2, -2, -1]] = [2*alpha + 6*beta, -alpha - 4*beta, beta,
                              beta, -alpha - 4*beta]
    A = np.stack([np.roll(row, i) for i in range(n)])
    return np.linalg.inv(A + gamma * np.eye(n))

M = snake_step_matrix(100, alpha=0.05, beta=0.05, gamma=1.0)
# one iteration, given external forces fx, fy sampled at the points:
# x = M @ (gamma * x + fx);  y = M @ (gamma * y + fy)
```

scikit-image 的 `active_contour` 實作的正是這個方法，外部能量用的是上面的邊緣與線條能量，並額外限制每次迭代每個點最多能移動多遠 [12]：

```
import numpy as np
from skimage import data
from skimage.color import rgb2gray
from skimage.filters import gaussian
from skimage.segmentation import active_contour

img = rgb2gray(data.astronaut())
s = np.linspace(0, 2 * np.pi, 400)
init = np.column_stack([100 + 100 * np.sin(s),   # rows
                        220 + 100 * np.cos(s)])  # columns
snake = active_contour(gaussian(img, sigma=3), init,
                       alpha=0.015, beta=10, gamma=0.001)
print(snake.shape)  # (400, 2): the final (row, col) points
```

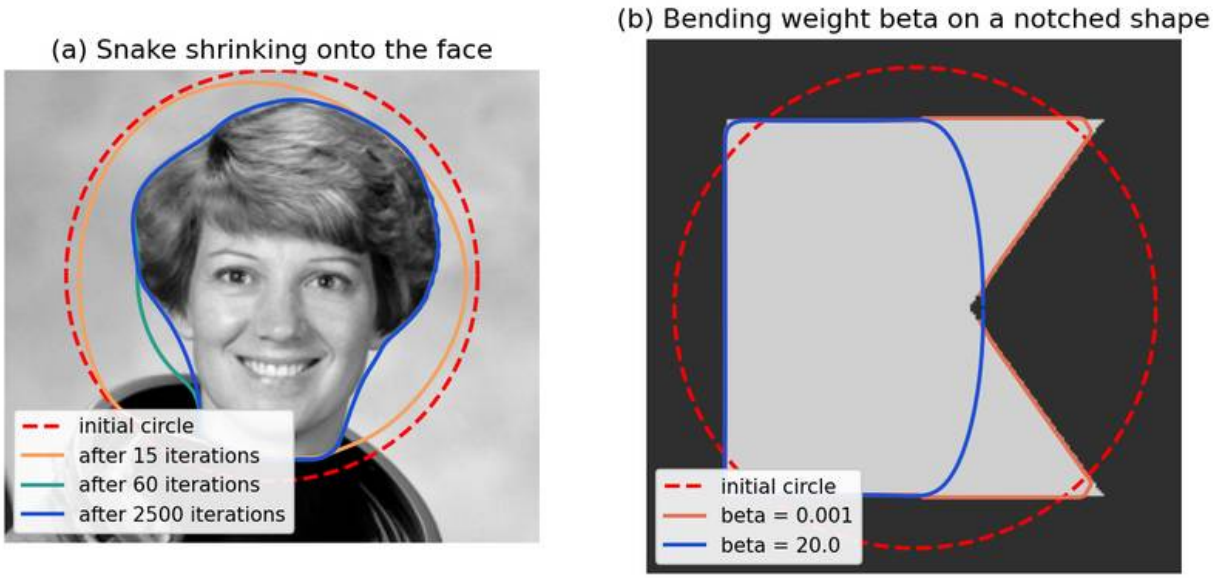


圖 *11.1* — (*a*) 以圓形起始的蛇在張力作用下收縮，直到邊緣力讓它停在頭髮與臉的輪廓上。(*b*) 彎曲權重 β：$\beta = 0.001$ 時蛇會沿著 V 形缺口與尖角走；$\beta = 20$ 時它不肯彎得那麼急，改以一段平滑弧線跨過缺口。

圖 11.1(a) 是蛇模型典型的一生：在平坦背景中只靠張力快速收縮，進入模糊邊緣圖有坡度的範圍後放慢，最後鎖定附近最強的邊緣。(b) 顯示 β 控制的取捨：剛硬的蛇不怕雜訊和小缺口，卻跟不上尖角與凹陷。

外力的選擇

外力決定蛇要找**什麼**。常見的選擇有：

- **邊緣圖的梯度**。先算一張在邊緣上數值大的邊緣圖 $e(x,y) \geq 0$，例如 $e = |\nabla(G_\sigma * f)|$（或其平方），再用 $F_{\text{ext}} = \kappa \nabla e$，$\kappa$ 為權重。這個力沿 e 往上坡，指向邊緣的脊線。
- **線條吸引**。用 $F_{\text{ext}} = \pm\nabla(G_\sigma * f)$ 追蹤暗或亮的曲線狀結構（道路、血管、文字筆畫）。

它們有同一個弱點。邊緣圖的梯度在遠離邊緣處幾乎為零（圖 11.2a）。起點離邊界太遠的蛇，除了自己的張力之外什麼都感覺不到。而在狹窄的凹陷裡，兩側牆面的力都指向側面而互相抵消，沒有任何力把蛇**往下**拉進去。這兩個問題，**捕捉範圍有限**與**難以收斂進凹陷**，正是 GVF 的動機。

梯度向量流（GVF）

GVF 保留 ∇e 好的部分（在有邊緣的地方準確指向邊緣），再以平滑擴散把其餘區域填滿。Xu 與 Prince [3] 把 GVF 場 $\mathbf{w}(x,y) = (u(x,y), v(x,y))$ 定義為下列能量的最小化者：

$$\mathcal{E}(\mathbf{w}) = \iint_\Omega \mu\,(u_x^2 + u_y^2 + v_x^2 + v_y^2) \;+\; |\nabla e|^2\,|\mathbf{w} - \nabla e|^2 \; dx\,dy.$$

- u_x, u_y, v_x, v_y 是場的兩個分量的偏導數；第一項要求場變化緩慢。
- $|\nabla e|^2\,|\mathbf{w} - \nabla e|^2$ 要求 $\mathbf{w}$ 等於 ∇e，但只在 $|\nabla e|$ 大的地方，也就是邊緣附近。
- $\mu > 0$ 平衡兩者，雜訊越多應該越大。

其 Euler–Lagrange 方程式是一對彼此獨立的擴散方程式。以時間上的梯度下降求解得到

$$u_t = \mu\,\nabla^2 u - (u - e_x)\,|\nabla e|^2, \qquad v_t = \mu\,\nabla^2 v - (v - e_y)\,|\nabla e|^2,$$

其中 ∇^2 是 Laplacian，e_x、e_y 是 ∇e 的分量。遠離邊緣時第二項消失，場就只是從邊緣往外擴散，同時把邊緣的方向帶出去。在單位時間步長與 5 點 Laplacian 下，若 e 已縮放到 $[0,1]$，顯式格式在 $\mu \le 1/4$ 時穩定。

```
import numpy as np
from scipy import ndimage as ndi

def gvf(f, mu=0.2, iters=500):
    """Gradient vector flow of an edge map f scaled to [0, 1]."""
    fy, fx = np.gradient(f)
    mag2 = fx**2 + fy**2
    u, v = fx.copy(), fy.copy()
    for _ in range(iters):  # explicit scheme, stable for mu <= 0.25
        u += mu * ndi.laplace(u, mode="nearest") - (u - fx) * mag2
        v += mu * ndi.laplace(v, mode="nearest") - (v - fy) * mag2
    return u, v
```

接著蛇模型把 $\kappa\,\nabla e$ 換成 $F_{\text{ext}} = \kappa\,\mathbf{w}$，演算法其餘部分完全不變。

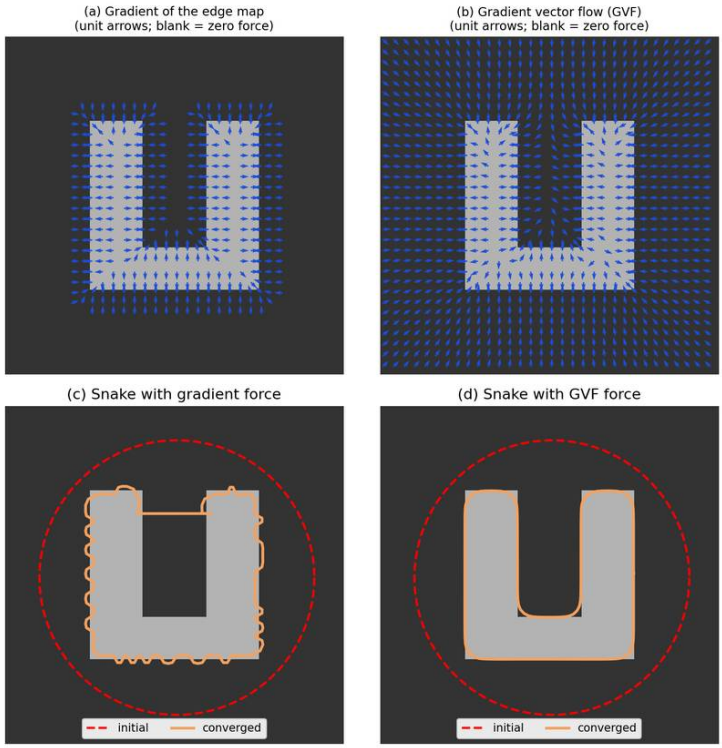


圖 *11.2* — (*a*) 邊緣圖的梯度只在邊緣周圍的細帶內非零。(*b*) *GVF* 把這些向量擴散到整張影像；缺口內的向量向下指。(*c*) 使用梯度力時，從虛線圓出發的蛇卡在缺口開口處（鋸齒狀處是它被邊緣帶勾住、又沒有力量把它撫平的地方）。(*d*) 同樣的蛇、同樣的參數，改用 *GVF* 驅動，便一路沿著凹陷走到底。

圖 11.2 在合成的 U 形上重現了 [3] 的經典測試。改變有兩點：蛇現在從影像任何地方都能感覺到物體（捕捉範圍大），而缺口內擴散後的向量是往下而非往側面指，所以蛇被拉進凹陷。有一個實作細節很重要：蛇進入缺口時長度變長，所以每隔幾十次迭代就要把點**重新取樣**成大致等間距，否則點鏈會太稀疏，貼不住牆面。

氣球力

Cohen [4] 從另一個角度處理捕捉範圍問題。如果把一條小蛇初始化在物體**內部**，張力會讓它在任何邊緣力碰得到它之前就縮成一點。那就加上壓力：

$$F_{\text{ext}}(v) = \kappa \nabla e(v) \;+\; k_1 \,\mathbf{n}(s),$$

其中 $\mathbf{n}(s)$ 是曲線在 $v(s)$ 的單位外法向量，k_1 是壓力。$k_1 > 0$ 時蛇像氣球一樣膨脹；$k_1 < 0$ 時則洩氣收縮。在真正的邊界上，邊緣力必須能壓過壓力（那裡 $\kappa\,|\nabla e| > |k_1|$），但在物體內部微弱、雜亂的梯度處則不行。Cohen 也建議把邊緣力正規化成 $\nabla e / |\nabla e|$，讓一個常數就能控制各處的強度。

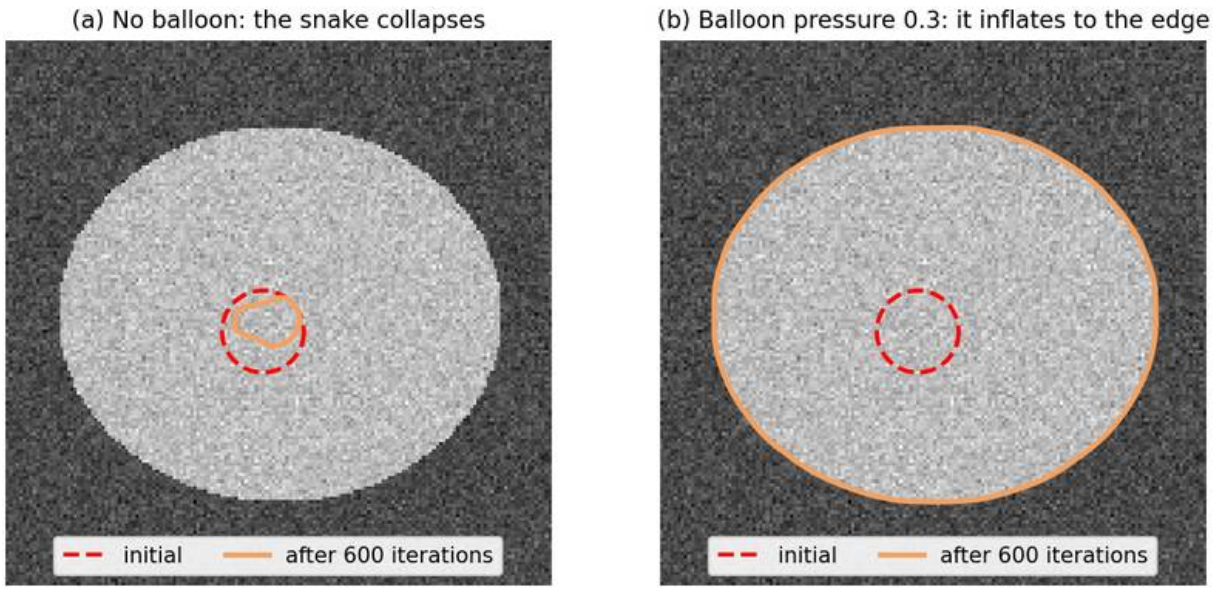


圖 *11.3* — 氣球力。(*a*) 帶雜訊物體內部的一條小蛇，只靠張力和邊緣力時會塌縮，因為附近沒有任何邊緣構得到。(*b*) 加上每次迭代 *0.3* 像素的向外壓力，蛇會一直膨脹，直到真實邊界上的邊緣力與壓力平衡。

氣球力有代價：你必須事先知道該膨脹還是收縮，而壓力太大時蛇會直接衝過弱邊緣。GVF 與氣球力彼此互補，實務上常一起使用。

蛇模型的限制

- **初始化**。能量有許多局部最小值。即使用了 GVF，起點橫跨兩個物體、或落在雜亂強邊緣錯誤一側的蛇，仍會收斂到錯誤答案。
- **參數調整**。α、β、γ、κ、模糊程度 σ、壓力 k_1 與 GVF 的 μ 互相影響。在一張影像上好用的數值，換一張常常就失效。
- **點的管理**。點會沿著曲線漂移，有的地方擠在一起，有的地方散開。實用的蛇模型需要重新取樣，還得偵測曲線是否自我相交。
- **拓撲固定**。蛇是一條封閉點鏈。若不加額外且容易出錯的處理，它圍住兩個物體時無法分裂成兩條（圖 11.6a），也無法和另一條蛇合併。

最後這個限制，正是水平集要拿掉的。

以水平集做分割

不直接追蹤邊界，而是追蹤一片地形，讓它的海岸線就是邊界。把地形抬高或壓低，海岸線就跟著移動。如果水位上升到把兩座山丘分開，一條海岸線就變成兩條，不需要任何特殊程式碼。

隱式表示

水平集函數（level-set function）是純量函數 $\phi : \Omega \times [0,\infty) \to \mathbb{R}$，使得時刻 t 的曲線就是它的零水平集：

$$C(t) = \{\, (x,y) \in \Omega \;:\; \phi(x,y,t) = 0 \,\}.$$

本章採用的慣例是：曲線內部 $\phi < 0$，外部 $\phi > 0$。（有些論文，包括 [8]，採相反符號，方程式的正負號也隨之改變。）從 ϕ 可以直接得到兩個幾何量：

$$\mathbf{N} = \frac{\nabla \phi}{|\nabla \phi|}, \qquad \kappa = \nabla \cdot \frac{\nabla \phi}{|\nabla \phi|},$$

其中 $\mathbf{N}$ 是單位**外**法向量，κ 是通過該點之等高線的**曲率**（凸的部分為正；半徑 r 的圓有 $\kappa = 1/r$）。

最方便的 ϕ 是**符號距離函數**（signed distance function）：$|\phi(x,y)|$ 是 (x,y) 到 C 的歐氏距離，正負號則區分內外。它幾乎處處滿足 $|\nabla \phi| = 1$，讓數值計算保持良好。

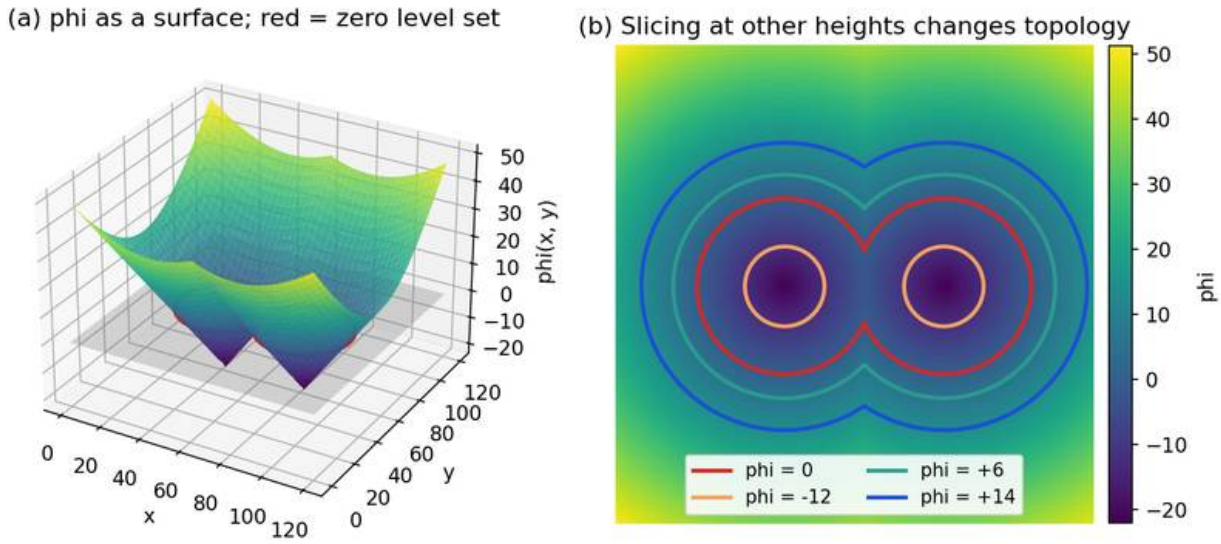


圖 *11.4* — 兩個相切圓盤的水平集函數 φ，以符號距離建構。(*a*) 曲線就是曲面與 $\varphi = 0$ 平面相交之處。(*b*) 在其他高度切同一個曲面，會得到兩條分開的曲線（$\varphi = -12$），或一條合併的曲線（$\varphi = +6$、$+14$）。把 φ 往上或往下移動，就是水平集輪廓不需特殊處理就能分裂與合併的方式。

水平集方程式

假設曲線上每一點沿外法向以速度 F 移動（正值代表向外）：

$$\frac{\partial C}{\partial t} = F\,\mathbf{N}.$$

曲線上的點始終保持零值，所以對所有 t 都有 $\phi(C(t),t) = 0$。以連鎖律微分：

$$\phi_t + \nabla\phi \cdot \frac{\partial C}{\partial t} = 0 \quad\Longrightarrow\quad \phi_t + F\,\nabla\phi \cdot \frac{\nabla\phi}{|\nabla\phi|} = 0 \quad\Longrightarrow\quad \boxed{\phi_t + F\,|\nabla\phi| = 0}$$

這就是 Osher 與 Sethian [5] 的水平集方程式。ϕ_t 是 ϕ 對時間的導數，F 是**速度函數**，可以取決於位置（影像資料）、曲線的幾何（曲率 κ）或全域統計量。方程式是在固定的像素網格上對所有 (x, y) 求解，而不只在曲線上；任何時刻都能取出零等高線得到曲線。方程式完全不在乎零集合有幾塊，所以分裂與合併自然發生。

離散計算：迎風差分與 CFL 條件

令 ϕ_{ij}^{n} 為像素 (i, j) 在第 n 步（步長 Δt）的值，網格間距 $\Delta x = 1$。單側差分

$$D_{ij}^{-x} = \phi_{ij} - \phi_{i-1,j}, \qquad D_{ij}^{+x} = \phi_{i+1,j} - \phi_{ij}$$

（$D^{\pm y}$ 同理）的組合方式，要保證資訊永遠取自波前**來的那一側**。這就是 [5] 的**迎風**（upwind）格式：

$$\phi_{ij}^{n+1} = \phi_{ij}^{n} - \Delta t\,\Big[\,\max(F_{ij}, 0)\,\nabla_{ij}^{+} + \min(F_{ij}, 0)\,\nabla_{ij}^{-}\Big],$$

$$\nabla^{+} = \sqrt{\max(D^{-x},0)^2 + \min(D^{+x},0)^2 + \max(D^{-y},0)^2 + \min(D^{+y},0)^2},$$

$$\nabla^{-} = \sqrt{\min(D^{-x},0)^2 + \max(D^{+x},0)^2 + \min(D^{-y},0)^2 + \max(D^{+y},0)^2}.$$

若這裡改用普通的中央差分，會產生振盪，尖角也會被抹平甚至發散；迎風格式才能讓波前保持銳利且穩定。

顯式格式只有在波前每步移動不超過一格時才穩定，這就是 **Courant–Friedrichs–Lewy（CFL）條件**：

$$\Delta t\,\max_{ij}|F_{ij}| \le c\,\Delta x, \qquad 0 < c \le 1.$$

若 F 含有曲率項 $b\,\kappa$，這部分的行為類似擴散，以中央差分離散；它在 2D 中會額外帶來一個更嚴格的二階限制，大約是 $\Delta t \le \Delta x^2/(4b)$。實務上每一步都依目前的最大速度計算 Δt。

```
import numpy as np
from scipy import ndimage as ndi

def signed_distance(mask):
    """Negative inside, positive outside, ~0 on the boundary."""
    return ndi.distance_transform_edt(~mask) - ndi.distance_transform_edt(mask)

def evolve(phi, F, steps=100, cfl=0.5):
    """Upwind scheme for phi_t + F |grad phi| = 0 (F is an array or scalar)."""
    for _ in range(steps):
        p = np.pad(phi, 1, mode="edge")
        dxm = phi - p[1:-1, :-2]; dxp = p[1:-1, 2:] - phi   # backward / forward
        dym = phi - p[:-2, 1:-1]; dyp = p[2:, 1:-1] - phi
        grad_plus = np.sqrt(np.maximum(dxm, 0)**2 + np.minimum(dxp, 0)**2 +
                            np.maximum(dym, 0)**2 + np.minimum(dyp, 0)**2)
```

```
        grad_minus = np.sqrt(np.minimum(dxm, 0)**2 + np.maximum(dxp, 0)**2 +
                             np.minimum(dym, 0)**2 + np.maximum(dyp, 0)**2)
        dt = cfl / (np.abs(F).max() + 1e-12)             # CFL: |F| dt <= dx
        phi = phi - dt * (np.maximum(F, 0) * grad_plus + np.minimum(F, 0) * grad_minus)
    return phi

yy, xx = np.mgrid[:128, :128]
phi = signed_distance((xx - 64)**2 + (yy - 64)**2 < 20**2)
phi = evolve(phi, F=1.0, steps=20)                # grow outward by 20 * 0.5 = 10 px
phi = signed_distance(phi < 0)                     # re-initialize
print(np.sqrt((phi < 0).sum() / np.pi))            # radius, about 30
```

指定速度函數

所有的影像分析都寫在 F 裡。主流有兩大家族。

以邊緣為主的速度：測地主動輪廓

定義一個**邊緣停止函數**，在平坦區域接近 1、在邊緣上接近 0，例如

$$g(x,y) = \frac{1}{1 + |\nabla (G_\sigma * f)(x,y)|^2}.$$

Caselles、Kimmel 與 Sapiro [6] 指出，$\beta = 0$ 的蛇模型與尋找**加權長度**最小的曲線密切相關：

$$L_g(C) = \oint_C g(C(s))\, ds,$$

其中 ds 是弧長。在 g 定義的度量下，邊緣是「便宜」的地方，所以最小化者是一條緊貼邊緣的測地線（最短路徑）。這就是**測地主動輪廓**（geodesic active contour, GAC）。對 L_g 做梯度下降，再加上一個類似 Cohen 氣球的選擇性常數壓力 c，在本章的符號慣例下得到

$$\phi_t = g\,|\nabla \phi|\,(\kappa - c) + \nabla g \cdot \nabla \phi.$$

每一項各有任務。$g\,\kappa\,|\nabla\phi|$ 是曲率流（縮短並平滑曲線），在邊緣附近減速到停止。$-c\,g\,|\nabla\phi|$ 是壓力：$c > 0$ 膨脹、$c < 0$ 收縮，同樣在邊緣上被關掉。$\nabla g \cdot \nabla \phi$ 是平流項，把曲線拉進邊緣谷底（g 最小處），避免它衝過頭。與蛇模型不同，GAC 是**內蘊的**（intrinsic）：它與曲線如何參數化無關；寫成水平集形式後，也能自由分裂與合併。

由於 g 永遠不會真正等於零，強壓力仍會從微弱或模糊的邊緣漏出去。這正是以區域為主之速度函數的動機。

以區域為主的速度：Mumford–Shah 與 Chan–Vese

Mumford 與 Shah [7] 把分割寫成：同時找出影像 f 的分段平滑近似 u 與一組邊界 C：

$$E_{\mathrm{MS}}(u, C) = \int_\Omega (u - f)^2\, dx\, dy \;+\; \mu \int_{\Omega \setminus C} |\nabla u|^2\, dx\, dy \;+\; \nu\, |C|,$$

第一項要求 u 貼近 f，第二項要求 u 平滑，但**跨越** C 時除外，$|C|$ 是邊界總長度。這個能量一般而言很難最小化。

Chan 與 Vese [8] 把 u 限制成**只有兩個值的分段常數**：C 內為 c_1，C 外為 c_2。能量變成

$$E_{\mathrm{CV}}(c_1, c_2, C) = \mu\,|C| + \lambda_1 \int_{\mathrm{inside}(C)} (f - c_1)^2\,dx\,dy + \lambda_2 \int_{\mathrm{outside}(C)} (f - c_2)^2\,dx\,dy,$$

（原論文還有一個選擇性的面積項 $\nu\,\mathrm{Area}(\mathrm{inside}(C))$，通常設為零）。$\mu \ge 0$ 是平滑權重，$\lambda_1, \lambda_2 > 0$ 衡量各區域被其常數擬合得多好。注意少了什麼：完全沒有梯度。這個模型可以分割邊界模糊、或只由平均亮度改變所定義的物體，這也是 [8] 取名為「沒有邊緣的主動輪廓」（active contours without edges）的原因。

要改寫成水平集形式，使用 Heaviside 函數 $H(z)$（$z \ge 0$ 時為 1，否則為 0）及其導數 Dirac delta $\delta(z)$。內部 $\phi < 0$ 時，內部指示函數是 $H(-\phi)$，長度是 $|C| = \int \delta(\phi)|\nabla\phi|$。固定 ϕ 時，最佳常數就是兩區的平均：

$$c_1 = \frac{\int f\,H(-\phi)}{\int H(-\phi)}, \qquad c_2 = \frac{\int f\,H(\phi)}{\int H(\phi)}.$$

固定 c_1, c_2 時，對 ϕ 做梯度下降得到

$$\phi_t = \delta_\varepsilon(\phi)\left[\,\mu\,\kappa + \lambda_1 (f - c_1)^2 - \lambda_2 (f - c_2)^2\,\right],$$

其中 δ_ε 是寬度為 ε 的平滑化 delta。逐像素來讀：如果某像素的亮度離 c_1 遠、離 c_2 近，括號為正，ϕ 上升，這個像素就移到外面。曲率項讓邊界保持短。演算法交替更新平均值與 ϕ。由於 δ_ε 在零水平集以外也不為零，這個模型還能在離初始曲線很遠的地方**產生新的輪廓**，因此幾乎從任何起始曲線出發，都能偵測內部的洞並分開不同物體，這是 [8] 特別強調的特性。

Cremers、Rousson 與 Deriche [14] 指出，這只是一個大家族中的一員：把「常數亮度加平方誤差」換成任何區域的機率模型（未知變異數的高斯分布、色彩直方圖、紋理特徵、運動），就得到結構相同的區域式水平集方法，括號則變成內外兩區之間的對數概似比。

```
from skimage import data, img_as_float
from skimage.segmentation import chan_vese, morphological_chan_vese

cam = img_as_float(data.camera())
seg, phi, energies = chan_vese(cam, mu=0.25, lambda1=1, lambda2=1,
                               dt=0.5, max_num_iter=200, extended_output=True)
fast = morphological_chan_vese(cam, num_iter=100, init_level_set="checkerboard",
                               smoothing=3)
```

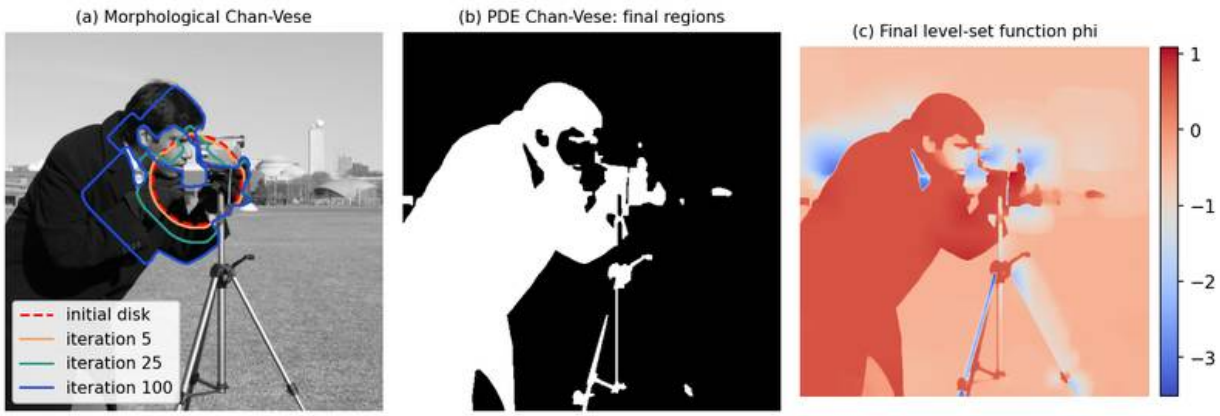


圖 *11.5* — 在攝影師影像上執行 *Chan–Vese*。(*a*) 從臉上一個圓盤出發的形態學近似版本；區域會長進所有「比較接近暗平均、而非亮平均」的地方。(*b*) 從棋盤格出發的 *PDE* 版本：兩相分割把深色人物、相機和三腳架與天空、草地分開，還產生了從未與初始曲線相連的新輪廓（三腳架的腳、遠方建築）。(*c*) 最終的 φ；注意 *scikit-image* 採用相反的符號慣例（內部 $\varphi > 0$）[*12*]。

以符號距離函數初始化與重新初始化

初始的 ϕ 通常是到某條曲線的符號距離：使用者畫的曲線、一個方框、一個圓盤，或許多小圓組成的「棋盤格」，讓演化從每個可能的物體附近開始。PDE 執行時，ϕ 會逐漸偏離距離函數：有些地方變得很陡，有些地方變得很平。陡的地方逼迫時間步長變得極小；平的地方則讓零交越點位置不準、容易受雜訊影響。

經典的補救方法是**重新初始化**（reinitialization）：每隔幾次迭代，在不移動零水平集的前提下，把 ϕ 換成到目前零水平集的符號距離。可以像上面程式那樣用快速歐氏距離轉換重算，也可以解一條把 $|\nabla\phi|$ 推向 1、同時凍結 ϕ 正負號的 PDE。重新初始化很有效，但可能讓波前稍微位移，而「何時」該做也只能靠經驗法則。

Li、Xu、Gui 與 Fox [10] 提出**距離正則化水平集演化**（distance-regularized level-set evolution, DRLSE），在能量中加入一個懲罰項 $\int p(|\nabla\phi|)$，其最小值落在 $|\nabla\phi| = 1$。這個懲罰項的梯度流在整個演化過程中都讓 ϕ 在零水平集附近保持接近符號距離，因此不再需要重新初始化。

提升效率

全網格的水平集求解器每一步都更新每個像素，但真正重要的只有零交越處。三種標準的加速方式：

- **窄帶法**（narrow band）。Adalsteinsson 與 Sethian [9] 只在零水平集周圍半寬 w（幾個像素）的帶狀區域內更新 ϕ。波前接近帶的邊緣時，就以新波前為中心重建窄帶（同時重新初始化）。每步的工作量從整張影像面積降到大約曲線長度乘以 w。
- **更大的穩定步長**。半隱式或算子分裂格式可讓剛硬的曲率項使用更大的 Δt，代價是每步要解一個線性系統。

- **形態學近似**。 Márquez-Neila、Baumela 與 Álvarez [11] 以一連串作用在二**值**水平集 $u \in \{0,1\}$ 上的二值形態學算子取代 PDE。膨脹與侵蝕扮演氣球項，線段式 sup-inf 與 inf-sup 算子的組合近似曲率流，$\nabla g \cdot \nabla u$ 的正負號則實作平流項。他們證明這些算子與 PDE 各項具有相同的**無窮小**行為。結果速度快、沒有時間步長、不需要重新初始化，也不會數值不穩定。

scikit-image 依據 [11] 提供 `morphological_geodesic_active_contour` 與 `morphological_chan_vese`，另外也有以 PDE 為基礎的 `chan_vese` [12]。請記住，形態學版本是 PDE 的**近似**：它們在像素網格上移動，「smoothing」參數是算子套用的次數而不是權重 μ，結果與連續模型相近，但並不相同。

```
from skimage import data, img_as_float
from skimage.segmentation import (inverse_gaussian_gradient, disk_level_set,
                                  morphological_geodesic_active_contour)

coins = img_as_float(data.coins())
g = inverse_gaussian_gradient(coins, alpha=100, sigma=2)   # ~0 on edges, ~1 elsewhere
init = disk_level_set(coins.shape, radius=150)
mask = morphological_geodesic_active_contour(g, num_iter=250, init_level_set=init,
                                             smoothing=1, balloon=-1, threshold=0.69)
```

這裡 `inverse_gaussian_gradient` 建立邊緣停止函數 g，`balloon=-1` 讓初始圓盤收縮（相當於 $c < 0$），`threshold` 決定 g 的哪些值算是能擋住氣球的邊緣。

實務比較：蛇模型與水平集

兩個家族最小化的能量很相似，差別主要在表示法，而表示法決定了什麼事情容易做。

	蛇模型（顯式）	水平集（隱式）
未知數	曲線上的 n 個點	每個像素（或窄帶內每個像素）一個值
每次迭代成本	分解一次矩陣後為 $O(n)$	$O(\text{pixels})$；用窄帶則為 $O(\text{band})$
拓撲改變	需要額外邏輯	自動分裂與合併
自我相交	可能發生，必須偵測	結構上不可能
點的間距	會漂移，需要重新取樣	沒有點需要管理
形狀先驗、標記點、開放曲線	容易：點可以帶標籤	較難：迭代之間沒有對應關係
互動編輯	很自然（拖動一個點）	間接
延伸到 3D	網格，簿記較麻煩	同樣的方程式放到體素網格上
次像素精度	有	有，內插零交越點即可
典型失敗	卡在雜亂邊緣或凹陷中	從弱邊緣漏出（邊緣式）；把外觀相似的區域併在一起（區域式）

Xu、Yezzi 與 Prince 研究過兩種表述之間的關係 [3]；對許多能量而言，選擇哪一種是實作上的考量，而不是原則上能不能建模。經驗法則：

- 需要一條拓撲已知的邊界、要互動、要快、或需要點的對應關係時（在影片各影格間追蹤輪廓、擬合嘴唇外形、描繪建物輪廓），用**蛇模型**。

- 物體數量未知、物體可能相接或分裂、邊界無法以梯度清楚定義（區域式模型），或在 3D 中工作時，用**水平集**。

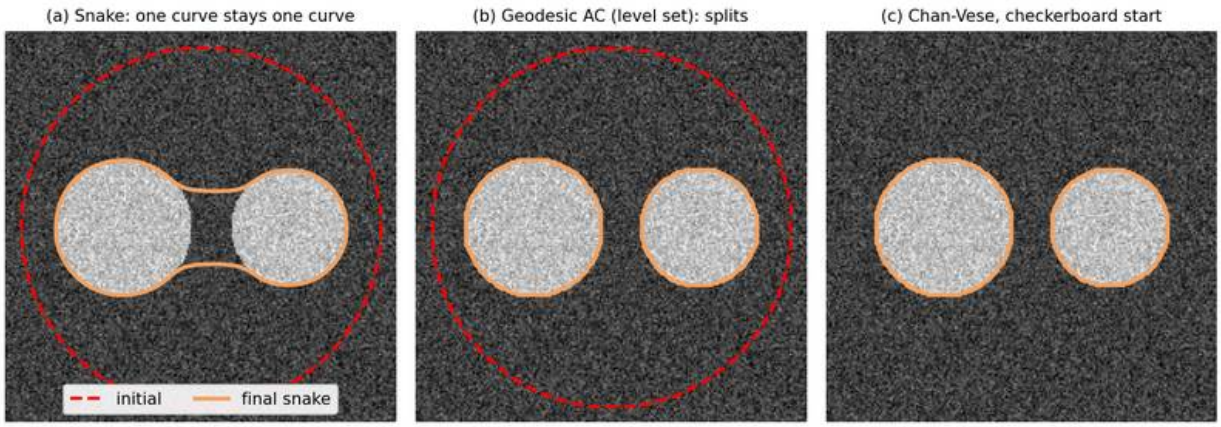


圖 *11.6* — 實務上的拓撲問題。(*a*) 圍住兩個圓盤起始的蛇只能收縮成一條曲線，包住兩者並橫跨中間的空隙。(*b*) 從同一個初始圓出發的測地主動輪廓（形態學水平集版本），兩半在中間相遇時就分裂成兩條曲線。(*c*) 從棋盤格出發的 *Chan–Vese* 根本不需要有意義的初始曲線。

現代觀點

主動輪廓已經不是分割影像的預設方法，深度網路才是。但本章的想法，包括結合邊界長度與區域擬合的能量、反覆變形的顯式輪廓，以及把水平集當作形狀表示，仍是深度分割中活躍的成分。以下六篇論文構成一張不錯的地圖。

醫學影像中的可變形模型。 McInerney 與 Terzopoulos [13] 寫下了醫學影像分析中可變形模型的經典早期綜述。他們把蛇模型及其 3D 延伸視為最小化能量的物理模型，並回顧它們在 CT、MRI 等模態上用於分割、形狀表示、比對與運動追蹤的應用。他們的主要結論至今仍成立：當影像本身含糊不清、需要模型提供平滑性與先驗知識時，可變形模型最有用。

統計式區域水平集。 Cremers、Rousson 與 Deriche [14] 回顧以區域為主的水平集方法，並從同一個貝氏框架推導出它們：每個區域都有一個機率模型（針對亮度、色彩、紋理或運動），水平集要最大化這個分割的後驗機率，並可選擇性地加入統計形狀先驗。他們主張區域式能量比邊緣式蛇模型對雜訊較不敏感、局部最小值也較少，這正是後來的研究以 Chan–Vese 類型項為主流的原因。

主動輪廓模型的近期分類。 Chen、Ge、Wang 與 Chen [15] 把主動輪廓模型分為邊緣式（GAC、DRLSE）、區域式（Chan–Vese，以及為處理亮度不均而設計的局部擬合變體）與混合式模型，實作了一組代表性模型，並在合成、醫學與自然影像上與深度學習分割比較。他們的結論很務實：傳統模型不需要訓練資料、能給出平滑的封閉邊界，但對初始化和參數敏感；在有標註資料的情況下，深度模型更準確；兩者是互補的。

深度分割中的形狀限制。 Bohlender、Oksuz 與 Mukhopadhyay [16] 綜述了如何把解剖形狀知識注入深度醫學影像分割：條件隨機場、統計形狀模型、主動輪廓，以及符號距離圖等隱式形狀

表示。他們的動機正是主動輪廓當初要修補的弱點：逐像素分類器可能產生破碎的區域與拓撲錯誤。

端對端學習的顯式輪廓。 Peng 等人的 Deep Snake [17] 保留了蛇模型的表示法，也就是一個反覆變形的封閉多邊形頂點序列，但把由能量推導出的更新換成學習而來的更新。每個頂點以其位置上取樣的 CNN 特徵描述，再沿多邊形做環狀卷積（circular convolution），預測每個頂點朝物體邊界的位移。用於實例分割時，它從偵測器給出的方框開始，作者報告可達即時速度。環狀卷積扮演的角色，正是經典蛇模型中五對角矩陣 A 的角色：沿著封閉曲線把每個頂點與它的鄰居耦合起來。

把主動輪廓能量當作損失函數。有些方法不在測試時執行輪廓演化，而是把能量放進訓練損失：

- Chen 等人 [18] 為醫學影像分割提出一個損失，把邊界長度項與內外區域擬合項（Chan–Vese 能量的監督式版本）加到密集分割網路上，並報告在心臟 MRI 上優於交叉熵。
- Kim 與 Ye [19] 把網路的 softmax 輸出視為各區域的特徵函數，並以 Mumford–Shah 風格的分段常數能量當作損失。由於這個能量不需要標註，同一個損失可支援非監督與半監督分割，也能在監督式訓練中當作正則項。
- Kervadec 等人 [20] 提出**邊界損失**（boundary loss），靈感來自以圖論方法最佳化主動輪廓流。它把預測邊界與真實邊界之間的距離寫成一個以真值符號距離圖加權的區域積分，而那張符號距離圖恰好就是真實輪廓的水平集函數。它是為極度不平衡的問題（例如微小病灶）設計的，因為單靠重疊類損失在這類問題上訓練效果差；使用時會與區域損失搭配。

深度學習改變了什麼、沒改變什麼：

- **改變了：資料項。**手工設計的力（∇e、GVF、g、區域平均）被學習到的特徵取代，這是準確度提升的主因。
- **沒改變：規則性先驗。**邊界長度、曲率懲罰、符號距離表示與區域均勻性，以損失項和輸出表示的形式重新出現，因為沒有它們，逐像素分類器仍會產生參差不齊、破碎的邊界。
- **沒改變：顯式與隱式的取捨。**以輪廓為基礎的網路繼承了蛇模型固定的拓撲（每個實例一個多邊形）與速度；以遮罩或距離圖為基礎的網路則繼承了水平集的彈性。
- **本身仍然有用。**沒有標註資料時，用 scikit-image 跑 Chan–Vese 或 GAC 只要幾行程式、不需訓練，很適合當基準、產生偽標籤，或作為清理網路遮罩的後處理步驟。

重點整理

- 主動輪廓透過**在曲線上最小化能量**來分割：規則性項（長度、彎曲）加上資料項（邊緣或區域擬合）。對能量做梯度下降，就是一條人工時間中的 PDE。
- **蛇模型**是顯式的點鏈。它的 Euler–Lagrange 方程式 $\alpha v_{ss} - \beta v_{ssss} + F_{\text{ext}} = 0$ 以半隱式步驟 $\mathbf{x}^t = (A + \gamma I)^{-1}(\gamma \mathbf{x}^{t-1} + \mathbf{f}_x)$ 求解。
- 單純的梯度力**捕捉範圍小**，也進不了**凹陷**。GVF 把邊緣向量擴散到整張影像；氣球力讓曲線膨脹或收縮。
- **水平集**把曲線存成 $\{\phi = 0\}$，在 **CFL** 步長限制下以**迎風差分**演化 $\phi_t + F|\nabla \phi| = 0$，拓撲改變自動發生。
- 模型寫在速度 F 裡：**測地主動輪廓**最小化以邊緣加權的長度；**Chan–Vese** 擬合兩個區域平均，不需要邊緣。

- 讓 ϕ 保持接近**符號距離**（重新初始化或 DRLSE），並用**窄帶**或**形態學**近似節省計算；後者是近似 PDE，而不是精確求解。
- 在深度學習中，同樣的想法以**學習式輪廓變形**（Deep Snake）以及由長度、區域與符號距離項構成的**損失函數**延續下來。

練習

1. 只有張力。一條 $\beta = 0$、沒有外力的封閉蛇，初始為半徑 R 的圓，以 n 個等距點取樣。利用離散內力 $\alpha(v_{i+1} - 2v_i + v_{i-1})$，證明圓仍然是圓，並求出步長 τ 的一次顯式更新後半徑如何改變。

提示

對角度間隔 $\theta = 2\pi/n$ 的正多邊形（從圓心量起），$v_{i+1} + v_{i-1} = 2\cos\theta\, v_i$。內力為 $-2\alpha(1-\cos\theta)\, v_i$，指向圓心，所以 $R \mapsto R\,[1 - 2\alpha\tau(1-\cos\theta)]$。圓以等比方式縮小；注意縮小速率與 n 有關，這也是蛇模型參數會與取樣綁在一起的原因之一。

2. 凹陷測試。用上面的 `gvf` 函數建立圖 11.2 的 U 形，改變 GVF 權重 μ（0.05、0.1、0.2）與 GVF 迭代次數。哪個參數決定場能從邊緣延伸多遠？哪個決定它有多平滑？

提示

迭代次數決定擴散能傳多遠（大約與迭代次數乘以 μ 的平方根成正比）。較大的 μ 也會平滑得更強，有助於抵抗雜訊，但會把小細節磨圓。μ 超過 0.25 時顯式格式會變得不穩定。

3. 正負號與方向。在本章慣例（內部 $\phi < 0$）下，當 $F = -1$ 時，圓在 $\phi_t + F|\nabla\phi| = 0$ 下會怎樣？$F = -\kappa$ 時呢？分別寫出半徑 $r(t)$。

提示

$F = -1$：圓以單位速度縮小，$r(t) = r_0 - t$。$F = -\kappa = -1/r$：曲線縮短流，$\dot r = -1/r$，所以 $r(t) = \sqrt{r_0^2 - 2t}$，圓在 $t = r_0^2/2$ 時消失。

4. 實際體會 CFL。用 `evolve` 程式碼片段，在同一個圓上分別以 `cfl=0.5`、`1.0`、`2.0` 執行。量測最後的半徑並觀察零等高線。哪裡壞掉了？為什麼窄帶法無法移除這個限制？

提示

超過 1 時，波前每一步試圖跨越不只一格；迎風資訊取自錯誤的位置，輪廓變得鋸齒狀，甚至數值發散。窄帶法減少的是更新的像素數，而不是每一步的大小。

5. Chan–Vese 失效的時候。建立一張合成影像：左半部是由黑到白的平滑漸層；右半部是灰色背景上的一個灰色圓盤，兩者平均值相同但變異數不同（只在圓盤內加強烈雜訊）。執行 `chan_vese`，從能量的角度解釋結果，並提出一個能成功的區域模型修改。

提示

Chan–Vese 只比較平均值，所以（平均相同的）圓盤是看不見的，而漸層會從中間被切開。若像 [14] 的統計框架那樣，以各自擁有平均值**和**變異數的高斯分布描述每個區域，圓盤就能被分開。

6. 把能量當作損失。改用軟遮罩 $m(x,y) \in [0,1]$（屬於內部的機率）而非水平集來寫 Chan–Vese 能量，並以總變差 $\int |\nabla m|$ 取代長度項。證明固定 m 時最佳的 c_1, c_2 是加權平均，並說明為什麼這個函數可以在沒有標註的情況下訓練網路。

提示

$E = \mu \int |\nabla m| + \lambda_1 \int m\,(f-c_1)^2 + \lambda_2 \int (1-m)(f-c_2)^2$。令 $\partial E/\partial c_1 = 0$ 得 $c_1 = \int mf / \int m$。E 中沒有任何地方用到真值，只用到影像本身，所以不需標註就能對產生 m 的網路求梯度；可與 [19] 對照。

參考文獻

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 11. publisher page
2. M. Kass, A. Witkin, and D. Terzopoulos, "Snakes: Active contour models," *International Journal of Computer Vision*, vol. 1, no. 4, pp. 321–331, 1988. doi:10.1007/BF00133570
3. C. Xu and J. L. Prince, "Snakes, shapes, and gradient vector flow," *IEEE Transactions on Image Processing*, vol. 7, no. 3, pp. 359–369, 1998; see also C. Xu, A. Yezzi Jr., and J. L. Prince, "On the relationship between parametric and geometric active contours," *Proc. 34th Asilomar Conf. on Signals, Systems, and Computers*, 2000. GVF project page with both papers
4. L. D. Cohen, "On active contour models and balloons," *CVGIP: Image Understanding*, vol. 53, no. 2, pp. 211–218, 1991. author PDF
5. S. Osher and J. A. Sethian, "Fronts propagating with curvature-dependent speed: Algorithms based on Hamilton–Jacobi formulations," *Journal of Computational Physics*, vol. 79, pp. 12–49, 1988. doi:10.1016/0021-9991(88)90002-2
6. V. Caselles, R. Kimmel, and G. Sapiro, "Geodesic active contours," *International Journal of Computer Vision*, vol. 22, no. 1, pp. 61–79, 1997. PDF
7. D. Mumford and J. Shah, "Optimal approximations by piecewise smooth functions and associated variational problems," *Communications on Pure and Applied Mathematics*, vol. 42, no. 5, pp. 577–685, 1989. doi:10.1002/cpa.3160420503
8. T. F. Chan and L. A. Vese, "Active contours without edges," *IEEE Transactions on Image Processing*, vol. 10, no. 2, pp. 266–277, 2001. doi:10.1109/83.902291
9. D. Adalsteinsson and J. A. Sethian, "A fast level set method for propagating interfaces," *Journal of Computational Physics*, vol. 118, pp. 269–277, 1995. summary page
10. C. Li, C. Xu, C. Gui, and M. D. Fox, "Distance regularized level set evolution and its application to image segmentation," *IEEE Transactions on Image Processing*, vol. 19, no. 12, pp. 3243–3254, 2010. doi:10.1109/TIP.2010.2069690
11. P. Márquez-Neila, L. Baumela, and L. Álvarez, "A morphological approach to curvature-based evolution of curves and surfaces," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2014. doi:10.1109/TPAMI.2013.106
12. scikit-image developers, "skimage.segmentation" API reference (`active_contour`, `chan_vese`, `morphological_chan_vese`, `morphological_geodesic_active_contour`). docs
13. T. McInerney and D. Terzopoulos, "Deformable models in medical image analysis: A survey," *Medical Image Analysis*, vol. 1, no. 2, 1996. author PDF

14. D. Cremers, M. Rousson, and R. Deriche, “A review of statistical approaches to level set segmentation: Integrating color, texture, motion and shape,” *International Journal of Computer Vision*, vol. 72, no. 2, pp. 195–215, 2007. doi:10.1007/s11263-006-8711-1
15. Y. Chen, P. Ge, G. Wang, and H. Chen, “An overview of intelligent image segmentation using active contour models,” *Intelligence & Robotics*, vol. 3, no. 1, pp. 23–55, 2023. open access
16. S. Bohlender, I. Oksuz, and A. Mukhopadhyay, “A survey on shape-constraint deep learning for medical image segmentation,” *IEEE Reviews in Biomedical Engineering*, vol. 16, pp. 225–240, 2023. arXiv:2101.07721
17. S. Peng, W. Jiang, H. Pi, X. Li, H. Bao, and X. Zhou, “Deep snake for real-time instance segmentation,” *CVPR*, 2020. arXiv:2001.01629
18. X. Chen, B. M. Williams, S. R. Vallabhaneni, G. Czanner, R. Williams, and Y. Zheng, “Learning active contour models for medical image segmentation,” *CVPR*, 2019. CVF open access
19. B. Kim and J. C. Ye, “Mumford–Shah loss functional for image segmentation with deep learning,” *IEEE Transactions on Image Processing*, vol. 29, pp. 1856–1866, 2020. doi ·arXiv:1904.02872
20. H. Kervadec, J. Bouchtiba, C. Desrosiers, E. Granger, J. Dolz, and I. Ben Ayed, “Boundary loss for highly unbalanced segmentation,” *Medical Image Analysis*, vol. 67, 2021 (first presented at MIDL 2019). arXiv:1812.07032

IP 12・特徵擷取

把區域、邊界與整張影像轉成一小串數字，讓它們在物體平移、旋轉、縮放或亮度改變時保持不變：從鏈碼、GLCM 紋理到 Harris、MSER 與 SIFT。

網站版本: https://blog.shyandram.dev/zh-tw/learn/image-processing/dip-12-feature-extraction/

你將學到

- 「特徵」（feature）與「描述子」（descriptor）的差別，以及描述子最常追求的四種不變性：平移、旋轉、尺度與光照。
- 如何把分割出來的邊界變成可以量測的東西：鏈碼、多邊形、標記圖（signature）與骨架，再進一步算出形狀數與傅立葉描述子。
- 如何用大小、形狀、拓撲（尤拉數）、紋理（直方圖動差、共生矩陣、頻譜）與動差不變量描述整個區域。
- 主成分分析（PCA）如何把物體轉正，並把大量量測壓縮成少數幾個數字。
- Harris–Stephens 角點偵測器、最大穩定極值區域（MSER）與 SIFT 如何找出並描述可以在兩張照片之間配對的興趣點。
- 近二十年的綜述論文如何評價這些手工設計的特徵，以及學習式特徵（SuperPoint、LoFTR、LightGlue、DINOv2）在哪些地方取代了它們。

先看全貌

分割（第 10、11 章）告訴我們物體「在哪裡」。但程式沒辦法像人一樣「看」一個區域，它需要一小串數字來說明這個區域「長什麼樣子」，才能比較、排序、搜尋和辨識。產生這串數字的過程，就是特徵擷取。

> **用最簡單的話說**
>
> 想像你在電話裡跟朋友形容一個玩具。你沒辦法傳照片，所以你說：「它是圓的，大概跟柳丁一樣大，中間有一個洞，表面摸起來凹凸不平。」朋友就能從一整箱玩具裡挑出正確的那一個，就算它倒過來放、放得比較遠，或房間比較暗也沒關係。你說的每一句話都是一個**特徵**。好的特徵，就是不管玩具怎麼轉、怎麼移、燈光怎麼變，都依然成立的那些描述。

本章依照 Gonzalez 與 Woods 教科書的順序 [1]：先談邊界，再談區域，接著是處理大量量測的統計工具（PCA），最後是完全不需要分割的特徵：角點、穩定區域與 SIFT 關鍵點。最後這一類把傳統影像處理接上了現代電腦視覺的任務，例如全景接圖、三維重建與視覺定位。

背景：特徵、描述子與不變性

白話版。特徵是你在影像中找到的有趣東西（一個角、一團斑點、一個區域）。描述子是你用來描述它的一串數字。我們希望那些「無關緊要的變化」發生時，描述子不要跟著變。

精確版。依照 [1]，我們把工作拆成兩步：

- **特徵偵測**（feature detection）：找出特徵**在哪裡**——一條邊界、一個區域，或一個帶有尺度與方向的點。
- **特徵描述**（feature description）：替每個偵測到的特徵指定一個向量 $\mathbf{d} \in \mathbb{R}^n$。

若描述子 $\mathbf{d}(\cdot)$ 對一族轉換 $\mathcal{T}$ 滿足

$$\mathbf{d}(T(f)) = \mathbf{d}(f) \quad \text{for all } T \in \mathcal{T},$$

就稱它對 $\mathcal{T}$ **不變**（invariant），其中 f 是影像（或區域），T 是施加在它上面的轉換。若偵測到的量會以可預期的方式**跟著**轉換一起變（例如關鍵點的位置隨物體移動，物體放大兩倍時偵測到的尺度也變成兩倍），就稱它是**共變的**（covariant）。偵測器應該共變，描述子應該不變。

我們最在意的轉換是：

變化	對影像的影響	常見的解法
平移	座標整體位移	減去質心，或改用差值
旋轉	座標旋轉	使用標準方向、取絕對值，或使用與旋轉無關的量
尺度	座標乘上倍率	依大小正規化，或在各尺度上搜尋
光照	強度改變，大致為 $f \mapsto af + b$	用梯度消去 b，再正規化消去 a

不變性與**鑑別力**之間永遠有取捨。對一切都不變的描述子，其實什麼也描述不了：「6」轉 180° 就變成「9」。挑選特徵，其實就是在決定哪些差異對你的任務是重要的。

邊界前處理

白話版。在量測邊界之前，得先把邊界像素按順序排好；通常還要簡化它，免得雜訊和像素造成的鋸齒主導了結果。

邊界追蹤

分割出的區域是一堆像素的集合，而大多數邊界描述子需要的是**有順序**、封閉的點列。標準的 Moore 邊界追蹤法 [1] 從最上方、最左邊的前景像素 b_0 以及它西邊的背景鄰居 c_0 出發，從 c 開始沿順時針方向檢查目前邊界像素的 8 鄰域，直到碰到前景像素。這個像素就是下一個邊界點，而它前一個被檢查的背景像素成為新的 c；如此重複，直到回到 b_0 且即將走向第二個點為止。結果就是一串點 $(x_0, y_0), \dots, (x_{K-1}, y_{K-1})$。OpenCV 的 `cv2.findContours` 和 scikit-image 的 `measure.find_contours` 都能直接給你這串點。

鏈碼

鏈碼（chain code）把點列換成**方向**序列。從一個邊界像素走到下一個，每一步都是 4 或 8 個方向之一，從正東開始逆時針編號為 0–3 或 0–7 [1]。鏈碼很精簡（8 方向時每步 3 位元），而且本來就與平移無關。

剩下兩個問題：鏈碼取決於**起點**，也取決於**旋轉**。解法都很簡單：

- **起點**。把碼視為環狀序列，選擇能組成最小整數的那個循環位移。
- **旋轉（45° 的倍數）**。改用**一階差分**（first difference）：相鄰兩個元素之間逆時針轉了幾個方向，$d_k = (c_k - c_{k-1}) \bmod 8$，其中 c_k 是第 k 個碼。物體轉 90° 時每個 c_k 都加 2，而每個 d_k 不變。

直接在原始像素格點上算出的鏈碼又長又雜，所以實務上會先把邊界重新取樣到較粗的格點上，讓每一節跨過好幾個像素（圖 12.1 左）。

```
import numpy as np, cv2
from skimage import data

mask = (data.horse() == 0).astype(np.uint8)          # True inside the horse
cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
pts = max(cnts, key=len)[:, 0, :]                    # (x, y), 8-connected boundary

# Freeman 8-direction code: 0 = east, counted counter-clockwise (y points down)
DIRS = {(1, 0): 0, (1, -1): 1, (0, -1): 2, (-1, -1): 3,
        (-1, 0): 4, (-1, 1): 5, (0, 1): 6, (1, 1): 7}
steps = np.diff(np.vstack([pts, pts[:1]]), axis=0)
code = np.array([DIRS[tuple(s)] for s in steps])

diff = (code - np.roll(code, 1)) % 8                 # first difference: rotation invariant
shifts = [tuple(np.roll(diff, -i)) for i in range(len(diff))]
shape_number = min(shifts)                           # start-point invariant
print(len(code), code[:12], diff[:12])
```

斜率鏈碼

一般鏈碼只認得八個方向。**斜率鏈碼**（slope chain code, SCC）[1] 則是沿著曲線首尾相接地放上**等長**的直線段，記錄相鄰兩段之間的斜率變化，並把它正規化成區間 $(-1, 1]$ 內的實數（± 1 代表完全折返）。因為只記錄方向的**變化**，SCC 對平移和旋轉不變；又因為每段等長，只要依曲線長度設定段長，它也對尺度不變。所有斜率變化絕對值的總和，正好是曲線「有多彎曲」（曲折度，tortuosity）的自然量度。

多邊形近似

用少數幾個頂點的多邊形，就能抓住邊界的精髓。

最小周長多邊形（minimum-perimeter polygon, MPP）。用一條由方格組成的帶子（cellular complex）把邊界蓋住，帶子的內牆與外牆形成兩道封閉的圍籬。想像在兩道圍籬之間放一條橡

皮筋，讓它自然收縮：它最後會停在帶子內最短的封閉路徑上，這就是 MPP [1]。它的頂點一定落在內牆的凸角或外牆的凹角上，因此可以設計出很有效率的演算法。方格大小控制細節多寡：方格越大，多邊形越粗略。

合併法（merging）。沿著邊界前進，只要目前這段的最小平方直線擬合誤差低於門檻 T，就繼續把點加進來；誤差超過 T 時就開新的一段。方法簡單，但頂點不一定落在真正的轉角上。

分裂法（splitting）。把距離最遠的兩個邊界點連成一條弦，找出離這條弦最遠的邊界點；若距離超過 T，就把它設為頂點，問題一分為二，再遞迴處理。分裂法較容易把頂點放在真正的反曲點上。`skimage.measure.approximate_polygon` 實作的就是這種遞迴分裂的概念；圖 12.1 第二格顯示兩種容許誤差的結果。

標記圖

標記圖（signature）把二維邊界降成一維函數。最簡單的是質心到邊界的距離，表示成角度的函數 $r(\theta)$，或正規化弧長的函數 $r(s)$ [1]。圓形得到常數；正方形得到四個一模一樣的凸起。標記圖天生與平移無關；旋轉變成函數的**循環位移**，尺度變成乘法，所以只要選一個標準起點（例如最遠點），再除以最大值或標準差即可正規化。$r(\theta)$ 只有在星形區域才是函數；像圖 12.1 的馬，從質心射出的一條射線可能穿過邊界不只一次，所以那裡改用 $r(s)$。

骨架與中軸

白話版。在一片草地的整圈邊緣同時點火，火線從四面八方往內燒，不同方向的火線相遇的地方就構成**骨架**。

精確版。區域 R（邊界為 B）的**中軸**（medial axis）是 R 中那些在 B 上有**不只一個**最近點的點所組成的集合 [1]。每個中軸點加上它到邊界的距離，定義了一個最大內切圓；所有這些圓的聯集能完整重建 R（這就是中軸轉換，medial axis transform）。直接計算的代價很高，所以實務上用**細化**（thinning）求骨架：反覆刪除那些不是端點、且刪掉後不會破壞連通性的邊界像素，直到再也刪不動為止。骨架對邊界上的小突起非常敏感，通常需要先平滑，或事後修剪短分支。

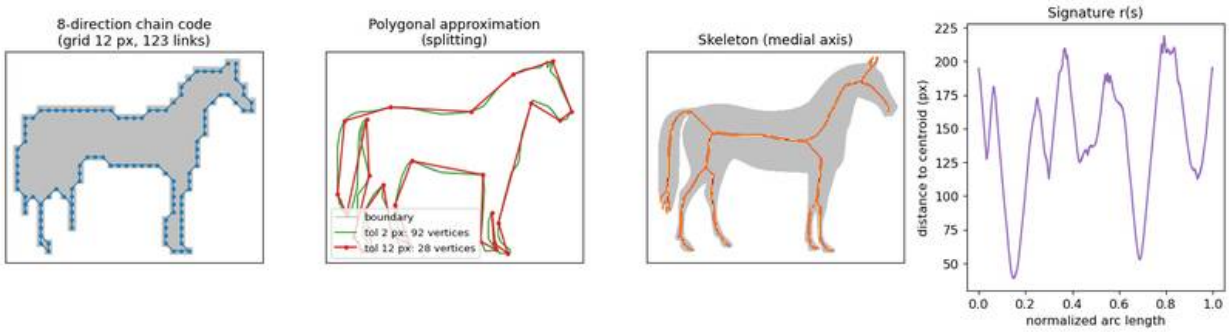


圖 *12.1* — 同一匹馬剪影（*skimage.data.horse*）的邊界表示。由左至右：*12* 像素格點上的 *8* 方向鏈碼；容許誤差 *2* 與 *12* 像素的遞迴分裂多邊形近似；細化得到的骨架；以及標記圖 $r(s)$，即到質心的距離對正規化弧長作圖（幾個深谷就是馬腿）。

邊界特徵描述子

白話版。邊界一旦變成有序點列，就可以量它：有多長、有多寬、有多彎，以及把它寫成一堆波的總和時長什麼樣子。

基本描述子

- **長度**。邊界上的像素數是粗略估計。若用 8 連通鏈碼，水平與垂直步長算 1，斜向步長算 $\sqrt{2}$。
- **直徑**。$\mathrm{Diam}(B) = \max_{i,j} D(p_i, p_j)$，其中 p_i, p_j 是邊界點，D 是距離（通常是歐氏距離）。連接這兩個最遠點的線段稱為**長軸**；**短軸**與之垂直，長度取到能讓以兩軸為邊的方框剛好包住邊界。這個方框稱為**基本矩形**（basic rectangle）。
- **離心率**。長軸長度與短軸長度的比值。（後面的區域離心率改用二階動差定義。）
- **曲率**。沿邊界的斜率變化率。數位邊界上的曲率很雜，所以通常改算某點兩側擬合線段的斜率差。曲率變號處就是反曲點；（沿行進方向）曲率為正表示凸的部分，為負表示凹的部分。

形狀數

邊界的**形狀數**（shape number）是其 4 方向鏈碼的一階差分，再循環位移成最小整數 [1]。它的長度稱為形狀的**階數**（order）n。在 4 連通格點上，封閉邊界的 n 一定是偶數，而且每個階數只有有限多種形狀。要公平比較兩個形狀，先固定方向：讓格點對齊形狀的基本矩形（或長軸），選擇能得到指定階數的格點大小，在該格點上算鏈碼，再算形狀數。兩個形狀的**相似度**（degree of similarity）定義為它們的形狀數仍然相同的最大階數。

傅立葉描述子

白話版。沿著邊界走一圈，把每個點寫成一個複數。這串數每走一圈就重複一次，所以可以拆成許多波。前幾個慢的波給出大致輪廓，快的波負責補細節。

精確版。設邊界有 K 個點 (x_k, y_k)，$k = 0, \dots, K-1$，把每個點寫成

$$s(k) = x(k) + j\,y(k).$$

對 $s(k)$ 做離散傅立葉轉換，就得到**傅立葉描述子**

$$a(u) = \sum_{k=0}^{K-1} s(k)\, e^{-j2\pi uk/K}, \qquad u = 0, 1, \dots, K-1,$$

反轉換則可以重建邊界。如果只保留其中 P 個係數（正負頻率中最低的那些），其餘設為零，就得到平滑、但保留整體形狀的近似 $\hat{s}(k)$（圖 12.2）。這就是為什麼少數幾個傅立葉描述子就能精簡地表示一個形狀。

這些描述子對基本幾何變化的反應很單純 [1]：

邊界的變化	對 $a(u)$ 的影響
平移 $\Delta_{xy} = \Delta_x + j\Delta_y$	只有 $a(0)$ 改變：變成 $a(0) + K\Delta_{xy}$
旋轉 θ	每個係數乘上 $e^{j\theta}$
縮放 α	每個係數乘上 α
起點移動 k_0	$a(u)$ 乘上 $e^{-j2\pi k_0 u/K}$

因此可以組出不變的描述子：丟掉 $a(0)$（平移）、除以 $|a(1)|$（尺度）、只保留大小 $|a(u)|$（旋轉與起點）。只留大小的代價是丟掉了相位，而相位也帶有一部分形狀資訊。

```
import numpy as np
from skimage import data, measure

mask = data.horse() == 0
c = max(measure.find_contours(mask.astype(float), 0.5), key=len)
z = c[:, 1] + 1j * c[:, 0]                          # boundary as complex numbers x + jy

def fourier_descriptor(z, n=16):
    a = np.fft.fft(z)
    a[0] = 0                                        # drop DC   -> translation invariance
    a = a / np.abs(a[1])                            # divide by |a(1)| -> scale invariance
    return np.abs(np.r_[a[1:n + 1], a[-n:]])     # magnitudes -> rotation and start-point
     ↪  invariance

f1 = fourier_descriptor(z)
f2 = fourier_descriptor(0.5 * np.exp(1j * 0.7) * np.roll(z, 300) + (40 + 10j))
print(np.abs(f1 - f2).max())                        # ~1e-16: same descriptor
```

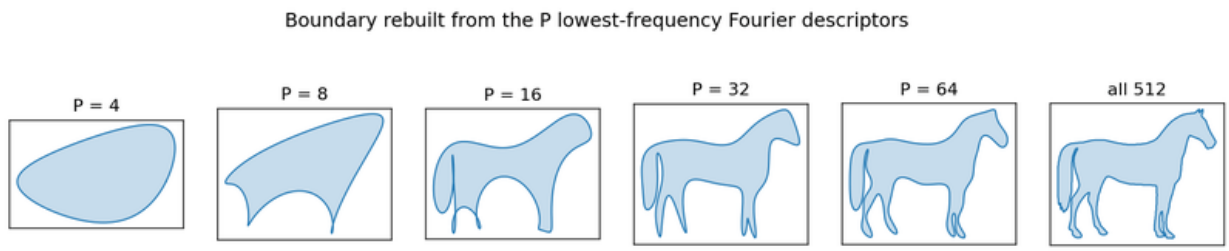


圖 *12.2* — 將馬的邊界重新取樣成 *512* 點，再用最低頻的 P 個傅立葉描述子重建。$P = 4$ 時只是一團橢圓；到 $P = 32$ 時腿、脖子和頭都已經認得出來；其餘係數主要在補小尺度的細節。

邊界的統計動差

一段邊界或一個標記圖可以看成一維函數 $g(r)$。把它的振幅當成隨機變數 v，建立直方圖 $p(v_i)$，$i = 0, \dots, A-1$，其中 A 是振幅的分箱數。則

$$\mu_n(v) = \sum_{i=0}^{A-1} (v_i - m)^n \, p(v_i), \qquad m = \sum_{i=0}^{A-1} v_i \, p(v_i),$$

其中 m 是平均振幅，μ_n 是第 n 階中心動差 [1]。μ_2 量的是分散程度，μ_3 量的是不對稱性。另一種做法是把 $g(r)$ 正規化成面積為 1，視為 r 的機率密度，這時動差描述的就是曲線的**形狀**。動差計算便宜、物理意義清楚，而且（若在旋轉正規化後的標記圖上計算）對旋轉不敏感。

區域特徵描述子

白話版。除了描邊，也可以看整塊區域：多大、多圓、有幾個洞，以及內部看起來如何（平滑、粗糙、有條紋）。

基本描述子

- **面積** A：區域中的像素數。
- **周長** P：邊界的長度。
- **緊緻度**（compactness）：P^2/A。它沒有單位，所以對尺度不變（在數位化誤差範圍內也對平移與旋轉不變）。圓盤使它最小。
- **圓度**（circularity）：$R_c = 4\pi A/P^2$。圓盤為 1，越細長或越不規則就越小，等於緊緻度的倒數再乘上常數。
- **等效直徑**：$d_e = 2\sqrt{A/\pi}$，即同面積圓盤的直徑。
- **離心率**（用動差定義）：設 $\lambda_1 \geq \lambda_2$ 為像素座標共變異數矩陣的特徵值，$e = \sqrt{1 - \lambda_2/\lambda_1}$。圓盤為 0，線段趨近 1。（scikit-image 採用此定義。）

```
import numpy as np
from skimage import data, filters, measure, morphology, segmentation

img = data.coins()
bw = img > filters.threshold_otsu(img)
bw = morphology.binary_closing(bw, morphology.disk(2))
bw = segmentation.clear_border(morphology.remove_small_objects(bw, 200))
for r in measure.regionprops(measure.label(bw))[:4]:
    circ = 4 * np.pi * r.area / r.perimeter ** 2  # 1 for a perfect disk
    print(f"area={r.area:5.0f}  circularity={circ:.2f}  "
          f"eccentricity={r.eccentricity:.2f}  euler={r.euler_number}  "
          f"hu1={r.moments_hu[0]:.4f}")
```

在 coins 影像上，分割良好的硬幣圓度約 0.9、離心率約 0.3–0.4（數位周長會略為高估，所以即使是完美的數位圓盤，得分也會比 1 低一點）。

拓撲描述子

拓撲研究的是在「橡皮膜」變形下保持不變的性質：可以拉伸、彎折，但不能撕開或黏合。面積和周長不是拓撲性質；連通塊的數目和洞的數目才是。**尤拉數**（Euler number）定義為

$$E = C - H,$$

其中 C 是連通成分數，H 是洞數 [1]。字母「A」的 $E = 0$（一個成分、一個洞）；「B」的 $E = -1$。對於由直線段構成的區域（多邊形網路），尤拉公式把拓撲與計數連在一起：

$$V - Q + F = C - H = E,$$

其中 V 是頂點數，Q 是邊數，F 是面數。注意 E 會受前景與背景所用的連通定義（4 或 8 連通）影響。

紋理

白話版。紋理是表面在眼睛裡的「觸感」：平滑、粗糙、規則。我們用像素值的統計，以及「像素值彼此如何相鄰」的統計來描述它。

直方圖的統計動差。設 z 為代表強度的隨機變數，$p(z_i)$ 為正規化直方圖，$i = 0, \dots, L-1$，平均值 $m = \sum_i z_i p(z_i)$。則 [1]：

$$\begin{aligned}
\mu_n(z) &= \sum_{i=0}^{L-1} (z_i - m)^n \, p(z_i) && (n\text{-th central moment}) \\
R(z) &= 1 - \frac{1}{1 + \sigma^2(z)} && (\text{relative smoothness; } \sigma^2 = \mu_2) \\
U(z) &= \sum_{i=0}^{L-1} p^2(z_i) && (\text{uniformity}) \\
e(z) &= -\sum_{i=0}^{L-1} p(z_i) \log_2 p(z_i) && (\text{entropy})
\end{aligned}$$

R 對常數區域為 0，變異數越大越接近 1（先把 σ^2 正規化到 $[0,1]$，例如除以 $(L-1)^2$）。μ_3 量的是直方圖的偏斜。這些量完全不管像素**在哪裡**，所以棋盤格和把它打亂後的影像會得到一模一樣的值。

共生矩陣。為了描述空間排列，Haralick、Shanmugam 與 Dinstein 提出了灰階共生矩陣（gray-level co-occurrence matrix, GLCM）[3]。先固定一個**位置運算子** Q，例如「右邊一個像素」。G 是 $L \times L$ 矩陣，元素 g_{ij} 計算「灰階為 i 的像素，在 Q 指定的位置上出現灰階為 j 的像素」的次數。令 $n = \sum_{i,j} g_{ij}$，

$$p_{ij} = g_{ij} / n,$$

就是該像素對出現的機率估計。由 p_{ij} 可以算出幾個好用的描述子（m_r, m_c 與 σ_r, σ_c 分別是列、行邊際分布的平均值與標準差）：

$$\begin{aligned}
\text{Contrast} &= \sum_{i,j} (i-j)^2 \, p_{ij} \qquad \text{Homogeneity} = \sum_{i,j} \frac{p_{ij}}{1 + |i-j|} \\
\text{Uniformity (energy)} &= \sum_{i,j} p_{ij}^2 \qquad \text{Entropy} = -\sum_{i,j} p_{ij} \log_2 p_{ij} \\
\text{Correlation} &= \sum_{i,j} \frac{(i - m_r)(j - m_c) \, p_{ij}}{\sigma_r \sigma_c}
\end{aligned}$$

（scikit-image 的 `graycoprops` 所回報的 “energy” 是均勻度總和的平方根。）質量集中在對角線附近，表示鄰居的值相近（平滑紋理：均質性高、對比低）；質量遠離對角線，表示變化劇烈（對比高）。實務上會先量化到少數幾階（8–32 階），對多個角度與距離計算 G，再對角度取平均，以得到近似的旋轉不變性。

```
import numpy as np
from skimage import data
from skimage.feature import graycomatrix, graycoprops

for name in ["brick", "grass", "gravel"]:
    q = (getattr(data, name)()[:256, :256] // 32).astype(np.uint8)   # 8 gray levels
    P = graycomatrix(q, distances=[1], angles=[0, np.pi/4, np.pi/2, 3*np.pi/4],
                     levels=8, symmetric=True, normed=True)
    feats = {p: graycoprops(P, p).mean() for p in ["contrast", "homogeneity", "energy",
     ↪  "correlation"]}
    print(name, {k: round(float(v), 3) for k, v in feats.items()})
```

```
brick  {'contrast': 0.269, 'homogeneity': 0.883, 'energy': 0.579, 'correlation': 0.817}
grass  {'contrast': 0.995, 'homogeneity': 0.717, 'energy': 0.288, 'correlation': 0.664}
gravel {'contrast': 0.669, 'homogeneity': 0.772, 'energy': 0.323, 'correlation': 0.78}
```

磚牆以大片平坦的磚面為主，GLCM 集中在少數幾個對角線格子：能量高、對比低。草地每根草葉都不同，GLCM 散得很開：對比最高、能量最低。光是四個數字，就足以把三種紋理分開（圖 12.3）。

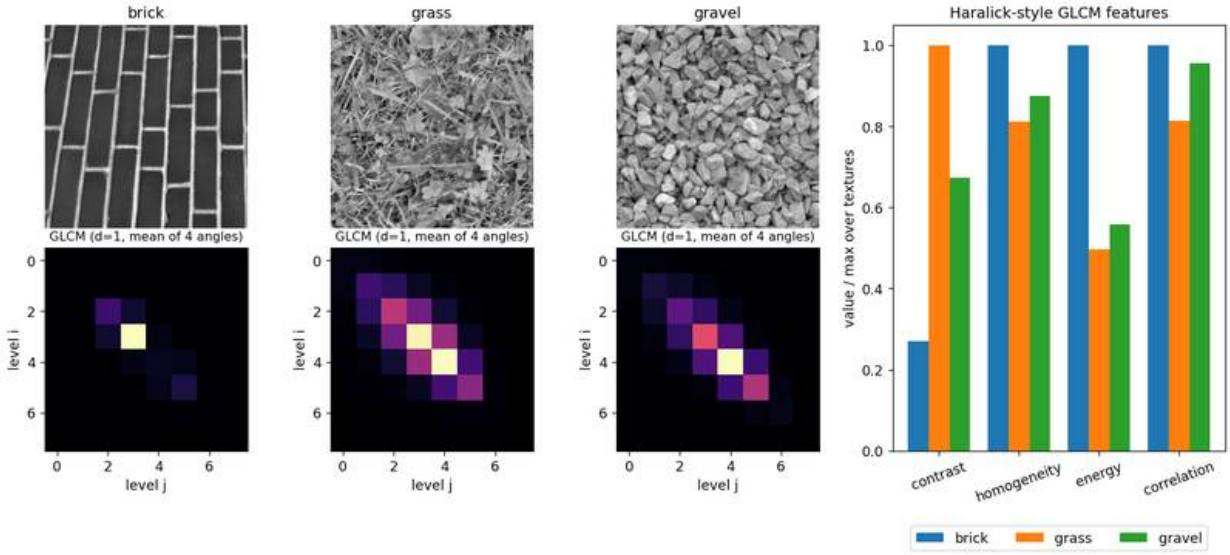


圖 *12.3* — 三種 *skimage.data* 紋理的共生矩陣特徵。上排：256×256 的區塊。下排：距離 *1*、四個角度平均後的 *GLCM*（越亮表示該像素對越常出現）。右：四個 *Haralick* 式特徵，各自除以三種紋理中的最大值，以便畫在同一個座標軸上。

頻譜紋理。週期性或有方向性的紋理，會在傅立葉頻譜上產生強烈而集中的峰值。把頻譜寫成極座標 $S(r,\theta)$，r 是到原點的距離（頻率），θ 是方向。兩個一維函數可以摘要它 [1]：

$$S(r)=\sum_{\theta=0}^{\pi} S_\theta(r), \qquad S(\theta)=\sum_{r=1}^{R_0} S_r(\theta),$$

其中 $S_\theta(r)$ 是角度 θ 那條射線上的頻譜，$S_r(\theta)$ 是半徑 r 的半圓上的頻譜，R_0 是考慮的最大半徑。$S(r)$ 的峰值揭露圖樣的週期（例如磚塊的間距）；$S(\theta)$ 的峰值揭露主要方向。只需要半圈，因為實數影像的頻譜對原點對稱。

動差不變量

對大小為 $M\times N$ 的二維影像（或區域指示函數）$f(x,y)$，$(p+q)$ 階動差為

$$m_{pq}=\sum_{x=0}^{M-1}\sum_{y=0}^{N-1} x^p y^q f(x,y),$$

中心動差為

$$\mu_{pq}=\sum_{x}\sum_{y}(x-\bar{x})^p(y-\bar{y})^q f(x,y), \qquad \bar{x}=\frac{m_{10}}{m_{00}},\ \bar{y}=\frac{m_{01}}{m_{00}},$$

其中 $(\bar{x},\bar{y})$ 是質心。中心動差對平移不變。**正規化中心動差**

$$\eta_{pq}=\frac{\mu_{pq}}{\mu_{00}^{\gamma}}, \qquad \gamma=\frac{p+q}{2}+1,$$

則再加上尺度不變。Hu [2] 從二階與三階的 η_{pq} 推導出七個對平移、尺度**以及旋轉**都不變的組合。前兩個是

$$\phi_1=\eta_{20}+\eta_{02}, \qquad \phi_2=(\eta_{20}-\eta_{02})^2+4\eta_{11}^2.$$

ϕ_1 是對質心的正規化「轉動慣量」：質量離中心越遠就越大。ϕ_2 量的是細長程度。第七個不變量在鏡射下會變號，因此可以分辨形狀與它的鏡像。由於高階動差會放大雜訊，而且數值跨越好幾個數量級，一般會比較 $\operatorname{sign}(\phi_i)\log|\phi_i|$。在 scikit-image 中，`regionprops(...).moments_hu` 會傳回全部七個；OpenCV 則是 `cv2.HuMoments(cv2.moments(mask))`。

以主成分作為特徵描述子

白話版。量測一個物體很多項時，有些量測其實在重複講同一件事。PCA 會找出一組新的座標軸，從「最有資訊」排到「最沒資訊」，讓我們只留前幾個、丟掉其他。對二維形狀來說，第一個軸就是形狀最長的那個方向。

精確版。設 $\mathbf{x}_1,\dots,\mathbf{x}_K$ 是 n 維向量，例如多光譜影像中每個像素的 n 個波段值，或某區域中每個像素的 (x,y) 座標。它們的平均與共變異數為

$$\mathbf{m}_x = \frac{1}{K}\sum_{k=1}^{K}\mathbf{x}_k, \qquad \mathbf{C}_x = \frac{1}{K-1}\sum_{k=1}^{K}(\mathbf{x}_k-\mathbf{m}_x)(\mathbf{x}_k-\mathbf{m}_x)^{\mathsf{T}}.$$

$\mathbf{C}_x$ 是實對稱矩陣，因此有 n 個正交單位特徵向量 $\mathbf{e}_i$，對應特徵值 $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_n \geq 0$。把特徵向量當成矩陣 $\mathbf{A}$ 的各列。**Hotelling 轉換**（也稱離散 Karhunen–Loève 轉換，或 PCA）為

$$\mathbf{y} = \mathbf{A}(\mathbf{x}-\mathbf{m}_x).$$

新向量的平均為零，共變異數是**對角**矩陣 $\mathbf{C}_y = \mathbf{A}\mathbf{C}_x\mathbf{A}^{\mathsf{T}} = \mathrm{diag}(\lambda_1,\ldots,\lambda_n)$：$\mathbf{y}$ 的各分量互不相關，第 i 軸上的變異數為 λ_i。若只保留前 k 列 $\mathbf{A}_k$，重建 $\hat{\mathbf{x}} = \mathbf{A}_k^{\mathsf{T}}\mathbf{y} + \mathbf{m}_x$ 的均方誤差為

$$e_{\mathrm{ms}} = \sum_{j=k+1}^{n}\lambda_j,$$

也就是被丟掉的特徵值總和。在這個意義下，沒有其他投影到 k 維的線性方法做得更好。

在影像處理中的三種用途 [1]：

1. **壓縮多光譜影像**。有 n 個對齊好的波段時，每個像素是一個 n 維向量。大部分變異通常集中在前兩、三個主成分影像，因此可以用它們代替全部 n 個波段來顯示或分類。
2. **正規化物體姿態**。把區域的像素座標當作向量，$\mathbf{y} = \mathbf{A}(\mathbf{x}-\mathbf{m}_x)$ 會把質心移到原點，並把區域轉到長軸與第一座標軸對齊（圖 12.4）。之後算出的描述子就與平移、旋轉無關；再除以 $\sqrt{\lambda_1}$ 就連尺度也無關。要注意：特徵向量的正負號是任意的，轉正後的物體可能被翻轉；可以用「三階動差較重的一側朝右」之類的規則決定。
3. **特徵影像**（eigen-images）。把同一類物體的大量同尺寸影像各自攤平成向量，前幾個特徵向量重新排回影像形狀，就是涵蓋該組影像大部分變化的「特徵影像」。任何新影像都可以用少數幾個座標 $\mathbf{y}$ 描述，成為精簡的辨識特徵。

```
import numpy as np
from skimage import data, transform

mask = transform.rotate((data.horse() == 0).astype(float), 35, resize=True) > 0.5
X = np.argwhere(mask)[:, ::-1].astype(float)     # (x, y) of every object pixel
m = X.mean(axis=0)
C = np.cov((X - m).T)                            # 2x2 covariance matrix
evals, evecs = np.linalg.eigh(C)                 # ascending eigenvalues
A = evecs[:, ::-1].T                             # rows = eigenvectors, largest first
Y = (X - m) @ A.T                                # Hotelling transform y = A(x - m)
print(np.round(np.cov(Y.T), 3))                  # diagonal: the axes are decorrelated
```

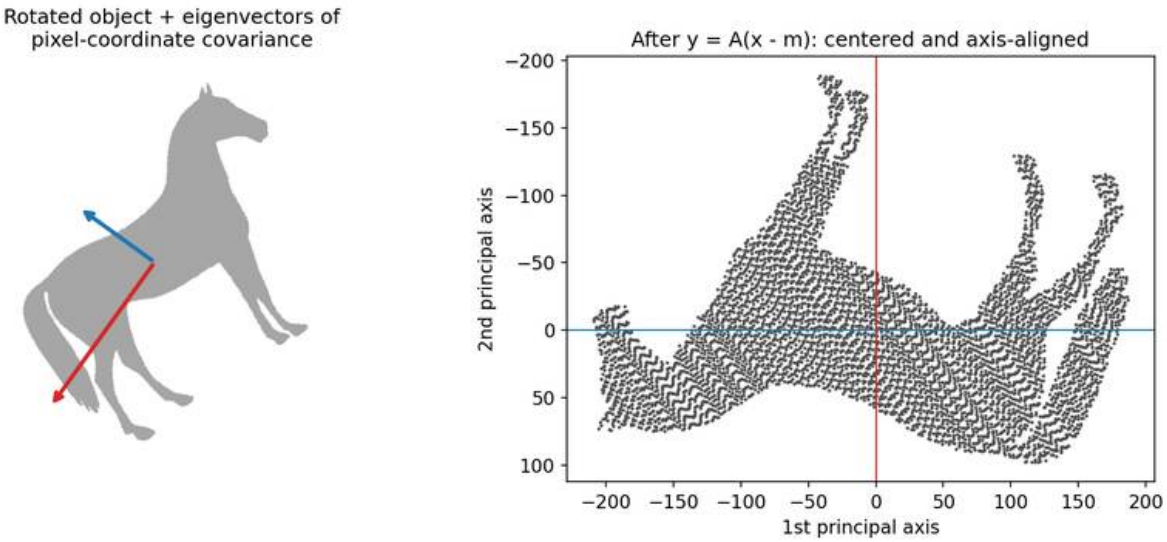


圖 *12.4* — 用 *PCA* 正規化姿態。左：旋轉 *35°* 的馬，箭頭是座標共變異數的特徵向量（長度 $= 2\sqrt{\lambda}$）。右：經 $y = A(x - m)$ 轉換後的像素座標。物體已置中，最長的方向落在水平軸上；它看起來是翻過來的，因為每個特徵向量的正負號是任意的。

整張影像的特徵

白話版。前面都需要先分割出物體。但很多任務（全景接圖、追蹤、三維重建）需要的是**地標**：能在任何照片中自動找到，並且能在同一場景的另一張照片中再找到。角點和穩定的斑塊就是好地標。

Harris–Stephens 角點偵測器

白話版。透過一個小窗口看影像，然後把窗口稍微挪一下。在平坦區域，什麼都沒變；在邊緣上，沿著邊緣挪不會變，橫越邊緣才會變；在角點上，往**任何**方向挪，看到的都會變。Harris 與 Stephens 把這個想法寫成公式 [4]。

精確版。窗口 w 位移 (u, v) 所造成的變化為

$$E(u,v) = \sum_{x,y} w(x,y)\,[f(x+u, y+v) - f(x,y)]^2 \approx [u \;\; v]\,\mathbf{M}\begin{bmatrix} u \\ v \end{bmatrix},$$

這裡用了一階泰勒展開，**結構張量**（structure tensor，又稱二階動差矩陣）為

$$\mathbf{M} = \sum_{x,y} w(x,y)\begin{bmatrix} f_x^2 & f_x f_y \\ f_x f_y & f_y^2 \end{bmatrix},$$

其中 f_x, f_y 是影像的偏導數，w 是窗函數（通常為高斯）。$\mathbf{M}$ 的特徵值 λ_1, λ_2 說明一切：兩個都小是平坦區，一大一小是邊緣，兩個都大是角點。1988 年時逐像素算特徵值太昂貴，所以 Harris 與 Stephens 改用**響應函數**

$$R = \det(\mathbf{M}) - k\,\operatorname{tr}(\mathbf{M})^2 = \lambda_1\lambda_2 - k(\lambda_1 + \lambda_2)^2,$$

k 是一個小常數（常見值約 0.04–0.06）。R 在角點為大的正值，在邊緣為負值，在平坦區接近零。角點就是 R 在非極大值抑制後、超過門檻的局部極大值。

性質：$\mathbf{M}$ 由導數組成，所以 R 不受亮度加減的影響；它的特徵值與旋轉無關，所以偵測器對旋轉共變。但它**不**對尺度共變：在小窗口中看到的角點，換成大窗口可能就像一條邊緣。補上尺度的正是 SIFT。

```
import numpy as np
from scipy import ndimage as ndi
from skimage import data, feature, util

def harris(img, sigma_d=1.0, sigma_i=1.5, k=0.05):
    Ix = ndi.gaussian_filter(img, sigma_d, order=(0, 1))   # derivative along x (columns)
    Iy = ndi.gaussian_filter(img, sigma_d, order=(1, 0))   # derivative along y (rows)
    Sxx = ndi.gaussian_filter(Ix * Ix, sigma_i)            # entries of the structure tensor
     ↪  M
    Syy = ndi.gaussian_filter(Iy * Iy, sigma_i)
    Sxy = ndi.gaussian_filter(Ix * Iy, sigma_i)
    det, tr = Sxx * Syy - Sxy ** 2, Sxx + Syy
    return det - k * tr ** 2                               # R > 0 corner, R < 0 edge, |R|
     ↪  small flat

img = util.img_as_float(data.camera())
R = harris(img)
corners = feature.corner_peaks(R, min_distance=7, threshold_rel=0.02)   # (row, col) list
```

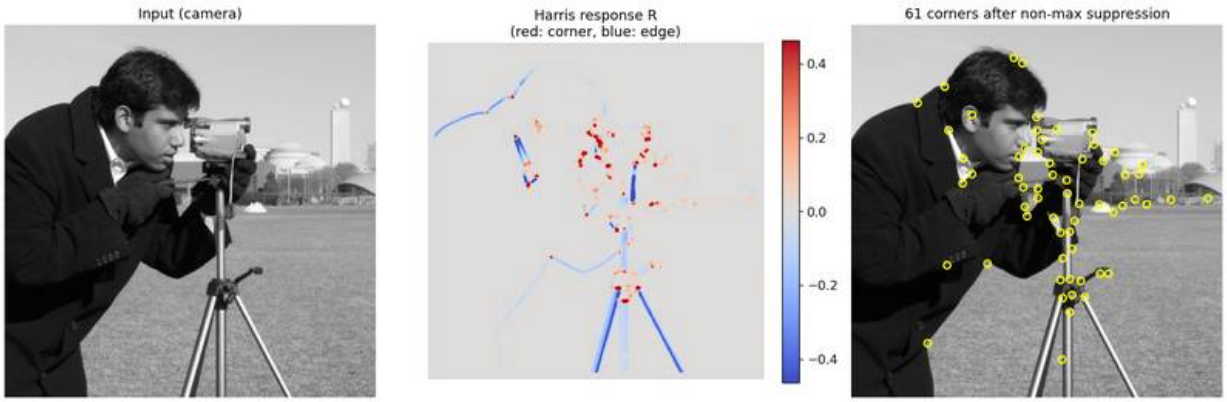


圖 *12.5 — skimage.data.camera* 上的 *Harris–Stephens* 角點（$k = 0.05$，積分 $\sigma = 1.5$）。中：響應 R；強烈的正值（紅）集中在相機機身與腳架關節，而腳架的腳屬於邊緣，呈負值（藍）。右：非極大值抑制後 R 的局部極大值。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

最大穩定極值區域（MSER）

白話版。想像把一片灰階地形慢慢灌水。白紙上的黑字會變成一個個小湖。水位上升時，大部分湖泊會迅速擴大或彼此合併，但字母形成的湖，在很長一段水位範圍內都維持同樣形狀，因為墨水比周圍的紙暗得多。水位變化很大、形狀卻幾乎不變的區域，就是**最大穩定**的區域。Matas、Chum、Urban 與 Pajdla 為寬基線立體視覺提出了這個方法 [5]。

精確版。在每個灰階 t 對影像做門檻化。$\{f \le t\}$（亮區域則為 $\{f \ge t\}$）的一個連通成分稱為**極值區域** Q_t：區域內每個像素都比它外圍邊界上的每個像素暗（或亮）。隨著 t 增加，成分逐漸長大、合併，形成一棵樹。對每個區域定義穩定度

$$q(t) = \frac{|Q_{t+\Delta} \setminus Q_{t-\Delta}|}{|Q_t|},$$

其中 $|\cdot|$ 是面積，Δ 是灰階步長。$q(t)$ 出現局部最小值時，該區域就是**最大穩定**的。把像素依強度排序後，用 union-find 結構就能在接近線性的時間內建出整棵樹。

為什麼有效：這個定義只用到強度的**大小順序**，所以 MSER 對任何單調的亮度變化都不變。在連續的幾何變換下，連通區域仍映射成連通區域，所以 MSER 對仿射變換共變（常見做法是對每個區域擬合橢圓，再正規化成圓形後才描述）。MSER 最適合邊界清楚的均勻斑塊，例如文字、標誌、窗戶；在模糊或紋理繁雜的場景中表現較差。

```
import cv2
from skimage import data

img = data.page()                                        # uint8 grayscale
mser = cv2.MSER_create(delta=5, min_area=10, max_area=800, max_variation=0.5)
regions, boxes = mser.detectRegions(img)                 # pixel lists + bounding boxes
print(len(regions), "regions")
```

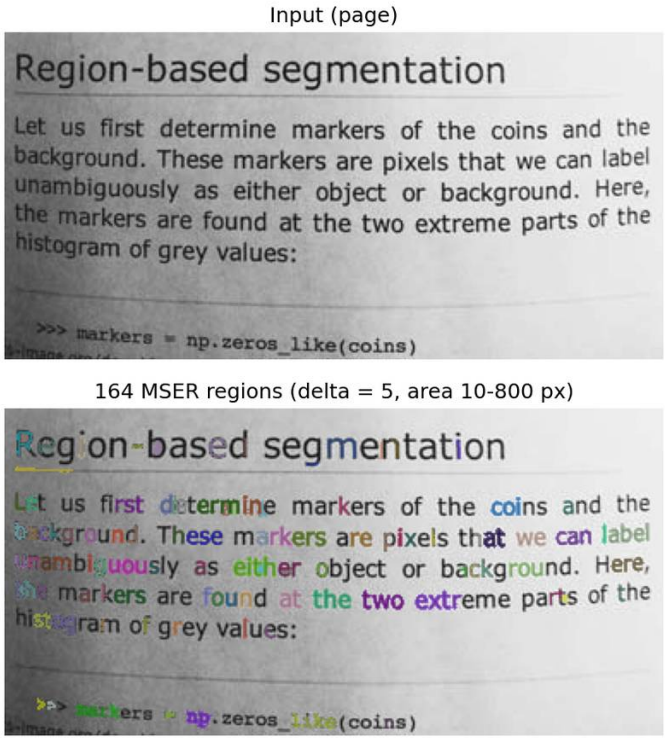


圖 *12.6* — 光照不均的 *skimage.data.page* 上的 *MSER*。每個上色的區域都是在 ± 5 個灰階內保持穩定的極值區域，大多是單一字母或黏在一起的字母群；由於穩定度只取決於強度順序，左下角較暗的部分並不造成困擾。

尺度不變特徵轉換（SIFT）

白話版。 Lowe 提出的 SIFT [6] 對每個地標回答三個問題：它**在哪裡**、它**多大**、它**朝哪個方向**？接著在這個地標自己的大小與方向下，寫下周圍梯度的摘要。如此一來，不論照片是近拍、遠拍、傾斜，還是光線較暗，這份摘要都一樣。

尺度空間

為了找出各種大小的特徵，SIFT 先建立**尺度空間**（scale space）：用寬度越來越大的高斯核模糊影像，

$$L(x,y,\sigma) = G(x,y,\sigma) \star f(x,y), \qquad G(x,y,\sigma) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/2\sigma^2},$$

其中 $\star$ 表示卷積，σ 是尺度。尺度分成好幾個**八度**（octave）；每個八度讓 σ 加倍，並切成 s 個間隔，所以相鄰尺度相差 $k = 2^{1/s}$ 倍。Lowe [6] 使用 $s = 3$ 與基礎尺度 $\sigma_0 = 1.6$。每完成一個八度就把影像降取樣一半，以節省計算。

以高斯差偵測關鍵點

把相鄰的模糊影像相減：

$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma).$$

這個**高斯差**（difference of Gaussians, DoG）非常接近尺度正規化的高斯拉普拉斯 $\sigma^2\nabla^2 G$，只差常數倍 $(k-1)$。它對大小與 σ 相符的斑塊反應最強（圖 12.7）。候選關鍵點是 D 值比全部 26 個鄰居都大或都小的像素：同一張 DoG 影像中的 8 個，加上上下兩個尺度各 9 個。因此每個關鍵點同時帶有位置**和**特徵尺度。

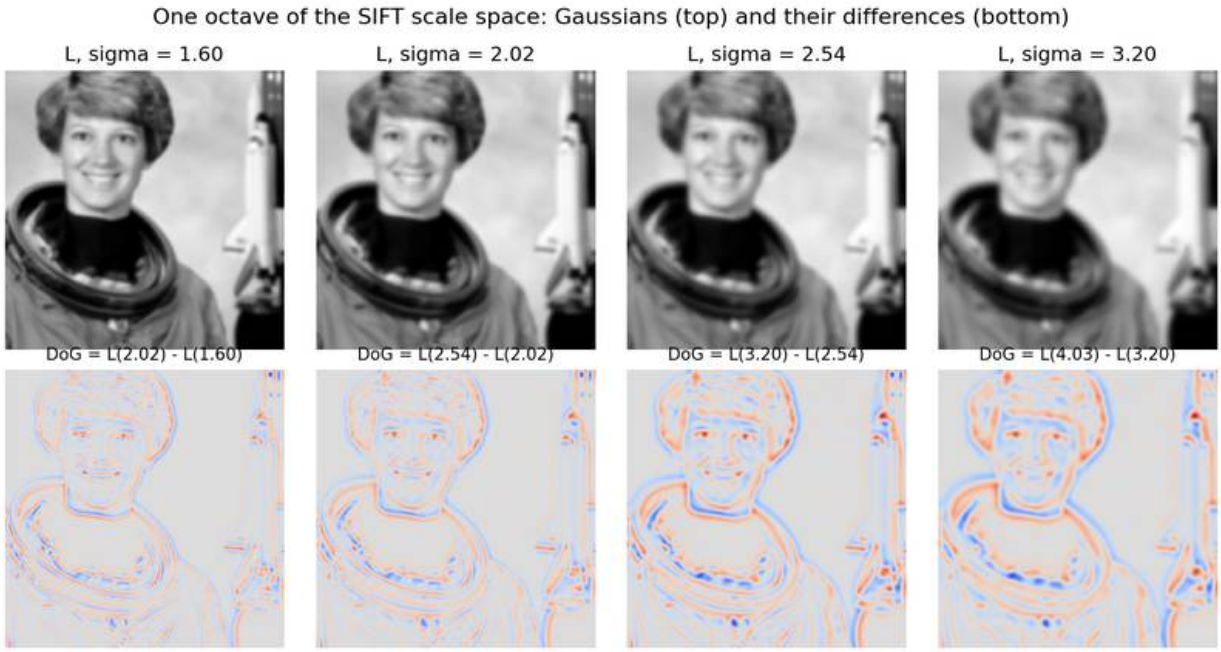


圖 *12.7* — 在 *skimage.data.astronaut* 的局部上建立的一個 *SIFT* 式八度（$\sigma_0 = 1.6$，每個八度三個間隔）。下排為 *DoG* 影像：細結構（髮絲、眼睛）在小 σ 有反應，大結構（頭盔環、火箭箭身）在大 σ 有反應。關鍵點是同時在空間與相鄰 *DoG* 影像間取極值的點。

關鍵點定位

極值是在離散格點上找到的。為了精修位置，用泰勒展開對樣本點附近的 D 擬合三維二次函數，

$$D(\mathbf{x}) \approx D + \frac{\partial D}{\partial \mathbf{x}}^{\mathsf{T}} \mathbf{x} + \frac{1}{2}\mathbf{x}^{\mathsf{T}} \frac{\partial^2 D}{\partial \mathbf{x}^2} \mathbf{x}, \qquad \hat{\mathbf{x}} = -\left(\frac{\partial^2 D}{\partial \mathbf{x}^2}\right)^{-1} \frac{\partial D}{\partial \mathbf{x}},$$

其中 $\mathbf{x} = (x, y, \sigma)^{\mathsf{T}}$ 是相對於樣本點的偏移，$\hat{\mathbf{x}}$ 是極值在次像素、次尺度上的位置。接著用兩個測試剔除不穩定的點 [6]：

- **低對比**：若 $|D(\hat{\mathbf{x}})| < 0.03$（強度範圍為 $[0, 1]$ 時）就捨棄。

- **邊緣響應**：DoG 在邊緣上也有反應，但邊緣上的位置定不準。用 D 的 2×2 空間 Hessian 矩陣 $\mathbf{H}$，只有在

$$\frac{\operatorname{tr}(\mathbf{H})^2}{\det(\mathbf{H})} < \frac{(r+1)^2}{r}$$

時才保留，其中 r 限制兩個主曲率的比值；Lowe 使用 $r = 10$。這和 Harris 的想法相同：好的點必須在**兩個**方向上都彎得厲害。

方向指定

在關鍵點附近、以它的尺度在 L 上計算梯度大小與方向，建立 36 格（每格 10°）的方向直方圖；每個樣本以梯度大小加權，並乘上 σ 為關鍵點尺度 1.5 倍的高斯窗。最高峰就是關鍵點的方向；其他超過最高峰 80% 的峰值，會產生同位置但方向不同的額外關鍵點 [6]。此後所有量測都相對於這個方向進行，因而得到旋轉不變性。

關鍵點描述子

在關鍵點周圍取一個窗口，依其方向旋轉、依其尺度決定大小，再切成 4×4 個格子。每個格子累積一個 8 格的梯度方向直方圖，以梯度大小與以關鍵點為中心的高斯加權，並用三線性內插，避免小小的位移造成數值在格子之間突然跳動。這樣得到 $4\times 4\times 8 = 128$ 個數字 [6]。最後：

1. 把向量正規化成單位長度（抵消 $f \mapsto af$ 的對比變化；梯度本身已抵消 $+b$）；
2. 把每個元素截在 0.2 以下（限制少數超大梯度的影響，例如反光等非線性光照）；
3. 再正規化一次。

配對

兩張影像的描述子用歐氏距離比較。只看最近鄰並不可靠：很多關鍵點根本沒有真正的對應點。Lowe 的**比值測試**（ratio test）只在

$$\frac{\|\mathbf{d} - \mathbf{d}_1\|}{\|\mathbf{d} - \mathbf{d}_2\|} < \tau$$

時保留配對，其中 $\mathbf{d}_1, \mathbf{d}_2$ 是另一張影像中最近與次近的描述子，τ 是門檻（Lowe 建議 0.8）[6]。有鑑別力的配對會比第二名近得多。大型資料庫會改用近似最近鄰搜尋，而留下來的配對通常還要經過幾何驗證，例如用 RANSAC（random sample consensus，隨機取樣一致）[21] 擬合相似、仿射或投影轉換，只保留內點（inlier）。

```
import numpy as np
from skimage import data, feature, transform
from skimage.color import rgb2gray
from skimage.measure import ransac

img1 = rgb2gray(data.astronaut())
```

```
tf = transform.AffineTransform(scale=0.7, rotation=np.deg2rad(25), translation=(140, -60))
img2 = 0.8 * transform.warp(img1, tf.inverse) + 0.1        # second "view"

sift = feature.SIFT()
sift.detect_and_extract(img1); k1, d1 = sift.keypoints, sift.descriptors
sift.detect_and_extract(img2); k2, d2 = sift.keypoints, sift.descriptors

matches = feature.match_descriptors(d1, d2, max_ratio=0.6, cross_check=True)  # ratio test
src, dst = k1[matches[:, 0]][:, ::-1], k2[matches[:, 1]][:, ::-1]           # (row, col) ->
 ↪  (x, y)
model, inliers = ransac((src, dst), transform.SimilarityTransform,
                        min_samples=3, residual_threshold=2, max_trials=500)
print(len(matches), "matches,", inliers.sum(), "inliers")
print("recovered scale %.3f, rotation %.1f deg" % (model.scale, np.degrees(model.rotation)))
```

在這組合成影像上，程式找到 484 個配對，其中 483 個與真正的轉換一致，並還原出尺度 0.700、旋轉 25.0°。真實照片有視角變化、遮擋和重複圖樣，內點比例會低得多，所以幾何驗證絕對不能省。

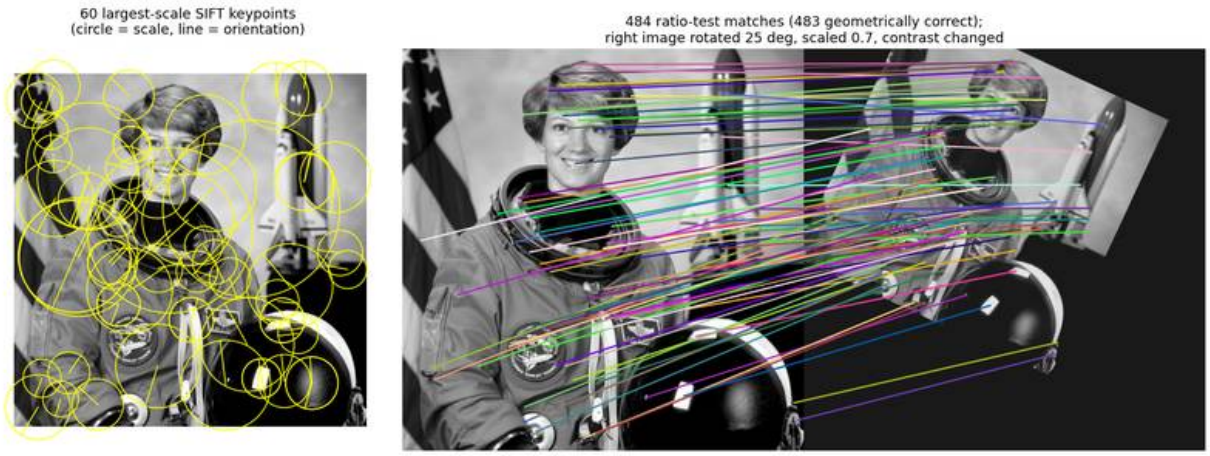


圖 *12.8 — skimage.data.astronaut* 上的 *SIFT*。左：尺度最大的 *60* 個關鍵點（圓半徑 ∝ 尺度，線段 = 方向）。右：與旋轉 *25°*、縮放 *0.7* 並改變對比的複本之間的部分比值測試配對；*484* 個配對中有 *483* 個與真實轉換的誤差在 *3* 像素以內。

> **去哪裡找 SIFT**
>
> scikit-image 在 0.19 版加入了 SIFT 偵測器與描述子（即上面用到的 `skimage.feature.SIFT`）。在 OpenCV 中，先呼叫 `cv2.SIFT_create()`，再用 `detectAndCompute(gray, None)`。

現代觀點

綜述論文怎麼說

形狀描述子。 Zhang 與 Lu 的綜述 [9] 沿兩個軸分類形狀描述子：**以輪廓為主**或**以區域為主**，以及**全域式**（整個形狀一個向量）或**結構式**（把形狀拆成基本單元）。本章前半段的方法都能放進這個表格：鏈碼與多邊形是結構式輪廓方法；傅立葉描述子與邊界動差是全域式輪廓方法；面積、尤拉數與 Hu 動差是全域式區域方法；骨架則是結構式區域方法。綜述對每種技術說明實作方式，並列出優缺點；實務上的訊息是：沒有一種描述子適用於所有應用，該選哪一種，取決於任務需要哪些不變性、需要多少細節。

局部特徵偵測器。 Tuytelaars 與 Mikolajczyk [10] 綜述了興趣點與區域偵測器：角點偵測器（Harris 及其尺度、仿射調適版本）、斑點偵測器（Laplacian/DoG、Hessian）與區域偵測器（MSER 等）。他們以好的局部特徵應具備的性質來組織這個領域：可重複性（repeatability）、鑑別性（distinctiveness）、局部性（locality）、數量、準確度與效率。他們指出，最重要的可重複性可以用兩種方式達成：**不變性**（用數學模型描述大的形變，再設計不受其影響的偵測器）或**強健性**（容忍雜訊、模糊、壓縮等小的形變）。他們也提醒，有些性質彼此競爭：鑑別性與局部性無法同時最大化，因為越局部的特徵看到的強度圖樣越少，也就越難正確配對；該怎麼取捨要看應用。SIFT 的兩個重速度的後繼者正是用準確度換時間的例子：SURF [7] 在積分影像上用方框濾波器近似高斯導數，ORB [8] 結合 FAST 角點測試與有方向性的二元描述子，以漢明距離比較，快到可以在手機上執行。

紋理。 Liu 等人 [11] 回顧了約二十年的紋理分類表示法，分為詞袋（bag-of-words）流程（局部描述子、編碼與池化）、以 CNN 為基礎的方法，以及以屬性為基礎的方法。共生統計 [3] 位在這段歷史的源頭：它們是第一批被廣泛使用的「像素**對**」描述子。這篇涵蓋兩百多篇論文的綜述，描繪了領域如何從以詞袋編碼的手工局部描述子，轉向以 CNN 特徵為基礎的表示法，最後並整理了尚待解決的問題與未來方向。

影像配對：從手工到深度。 Ma 等人 [12] 沿著完整的配對流程（偵測、描述、配對與離群值剔除），從手工方法談到可訓練的方法，並在標準資料集上做實驗比較；可以看到學習已經進入流程的每一個階段，而不只是描述子。Jin 等人 [13] 建立了一個以最終相機姿態準確度、而非中間指標來評分的基準。他們的關鍵發現是一個警訊：只要把每種方法的設定（比值門檻、RANSAC 參數、關鍵點數量）調好，像 SIFT 這樣的傳統流程仍可能勝過被視為最先進的方法，這暗示部分已發表的進步其實反映了沒調好的基準線。

深度學習如何改變特徵擷取

學習式關鍵點與描述子。 SuperPoint [14] 訓練一個全卷積網路，同時輸出關鍵點熱度圖與稠密描述子。它是自我監督的：先在合成形狀上學會角點，再透過**單應性調適**（homographic adaptation）——把同一張影像做許多次隨機變形並彙整自己的偵測結果——在真實影像上自我改進。DISK [16] 用強化學習（策略梯度）端到端地訓練偵測與描述，獎勵那些能帶來正確配對的關鍵點。它們用學來的函數取代 Harris 與 SIFT 的**公式**，但保留了相同的概念：稀疏、可重複的點，加上描述子。

學習式配對。 SuperGlue [15] 以圖神經網路取代最近鄰比值測試，讓兩張影像的關鍵點彼此注意（attend），再解一個能回答「沒有配對」的最佳傳輸（optimal transport）指派問題。LightGlue [18] 改寫這個設計，使它更快且能自我調適，在容易的影像對上花較少計算。LoFTR [17] 乾脆拿掉偵測器：以具有自注意力與交叉注意力的 Transformer 在粗特徵圖上做稠密配對，再精修位

置，這對偵測器找不到東西的低紋理區域特別有幫助。

通用深度特徵。最深遠的改變是：特徵越來越**不是為某個任務設計的**。像 DINOv2 [19] 這樣的自我監督基礎模型所產生的區塊（patch）特徵，不需微調就能用於分類、分割、深度估計與檢索。RoMa [20] 展示了它對配對的影響：以凍結的 DINOv2 特徵做粗配對，再結合細緻的卷積特徵做精確定位。換句話說，SIFT 核心那個手工設計的描述子，已經被一個在極大量影像上訓練出的網路取代。

沒有改變的部分。本章的**問題拆解方式**依然存在。現代流程仍然是偵測、描述、配對、再做幾何驗證，而且通常仍使用 RANSAC 類的強健估計器 [21]。尺度與旋轉仍然必須處理，不是內建（SIFT 的尺度空間、ORB 的方向），就是透過資料擴增讓模型學會。不變性依然是設計選擇：過度不變的描述子會失去鑑別力，無論它是手工設計還是學出來的。在以量測為主的領域（材料、顯微影像、品質檢測），面積、圓度、尤拉數與 GLCM 統計等區域描述子仍然很受歡迎，因為每個數字都有人可以檢查的物理意義。

重點整理

- **偵測器**找出特徵（點、區域、邊界）；**描述子**把每個特徵轉成向量。偵測器應對幾何變化共變，描述子應對任務中無關緊要的變化不變。
- 邊界要先排序（邊界追蹤）、再簡化（鏈碼、最小周長多邊形、合併與分裂、標記圖、骨架），之後才拿來量測。
- **傅立葉描述子**用少數係數壓縮封閉輪廓；丟掉 $a(0)$、除以 $|a(1)|$、只取大小，就得到平移、尺度、旋轉與起點不變性。
- 區域描述子從大小與形狀（面積、圓度、離心率）、拓撲（尤拉數 $E = C - H$）、紋理（直方圖動差、**GLCM** 特徵、頻譜）一路到 **Hu 動差不變量**。
- **PCA**（Hotelling 轉換）讓量測去相關、把物體轉到主軸方向，並提供精簡的特徵影像描述子；丟掉分量造成的誤差等於被丟掉的特徵值總和。
- **Harris** 由結構張量找角點；**MSER** 找在多個門檻下都穩定的區域；**SIFT** 加上尺度空間、方向與正規化的 128 維梯度直方圖，再以比值測試配對。
- 學習式偵測器、配對器與基礎模型特徵如今是多數配對研究的焦點，但仔細調整的傳統流程仍是強勁的基準線，而「偵測—描述—配對—驗證」的架構並未改變。

練習

1. **手算鏈碼**。寫出一個與座標軸對齊、寬 3 節、高 2 節的長方形邊界的 8 方向鏈碼，從左下角出發、逆時針行走。算出它的一階差分與形狀數（最小循環位移）。再把長方形轉 90°，說明形狀數不變。

   提示

   從左下角逆時針走（0 = 東、2 = 北）：0 0 0 2 2 4 4 4 6 6。一階差分的每個元素是相對前一個方向逆時針轉了幾格：2 0 0 2 0 2 0 0 2 0（開頭的 2 來自首尾相接的 6 → 0）。最小循環位移為 0 0 2 0 2 0 0 2 0 2。轉 90° 後每個碼都加 2（mod 8），差分不變。

2. **實測傅立葉描述子的不變性**。使用本章的 `fourier_descriptor` 函數。(a) 用數值說明它無法分辨形狀與其鏡像。(b) 從轉換效果表解釋原因，並提出一個能分辨鏡像的修改。

   提示

   用 `mask[:, ::-1]` 翻轉遮罩，並用 `find_contours` 追蹤兩條輪廓；兩組描述子的差距約在 10^{-4} 以內。追蹤器總是以相同的旋轉方向走過每條邊界，因此鏡像邊界是 $s'(k) = \overline{s(-k)}$（共軛且反向），最多差一個位移。它的 DFT 是 $\overline{a(u)}$：大小相同、相位相反，所以只看大小看不出鏡射。改為保留部分相位：複數 $a(u)\,a(2-u)/a(1)^2$（索引取模 K）不受平移、旋轉、尺度與起點影響，但在鏡射下會變成共軛，因此它虛部的正負號就能區分形狀與其鏡像。

3. **格點上的圓度**。用 `skimage.draw.disk` 與 `regionprops` 計算半徑 5、10、20、50 像素的數位圓盤之 $4\pi A/P^2$。為什麼結果不完全等於 1？它如何隨半徑變化？也試試 `perimeter_crofton`。

   提示

   數位周長是階梯狀的，不同估計方法用不同方式修正。小圓盤的相對誤差較大。兩種周長估計都列出來，其中一種應該明顯更接近真正的 $2\pi r$。

4. **紋理的方向**。做一張週期 8 像素的水平條紋合成紋理，加一點雜訊。計算 0°、45°、90°、135° 四個角度、位移 1 像素的 GLCM 對比，以及角度頻譜標記 $S(\theta)$。兩者各自哪個角度最突出？為什麼兩種方法的結論一致？

   提示

   沿著條紋（0°）移動時強度幾乎不變，對比低；橫越條紋（90°）才會變。頻譜中，水平條紋的能量落在垂直頻率軸上。注意慣例：scikit-image 的角度 0 代表水平方向的像素位移。

5. **以特徵值理解 PCA**。對半軸 $a = 40$、$b = 10$ 的實心橢圓，預測座標共變異數特徵值的比 λ_1/λ_2，再用程式驗證。用你的答案說明 λ_1, λ_2 與 `regionprops` 回報的離心率之間的關係。

   提示

   均勻實心橢圓沿長度為 a 的半軸方向，變異數為 $a^2/4$。所以 $\lambda_1/\lambda_2 = (a/b)^2 = 16$，而 $e = \sqrt{1 - \lambda_2/\lambda_1} = \sqrt{1 - b^2/a^2} \approx 0.968$。

6. **比值測試的取捨**。使用 SIFT 配對程式，把 `max_ratio` 從 0.5 調到 0.95，畫出 (a) 配對數量與 (b) RANSAC 內點比例。接著加上 0.6 倍縮放、30° 旋轉與高斯雜訊（$\sigma = 0.05$）再做一次。你會把門檻設在哪裡？為什麼答案取決於雜訊？

   提示

   寬鬆的比值會收進更多配對，但錯的也更多；嚴格的比值只留下少數、多半正確的配對。RANSAC 能容忍大量離群值，但需要最低數量的正確配對，所以最佳門檻落在兩個極端之間，並隨影像品質移動。

參考文獻

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 12. publisher page
2. M.-K. Hu, "Visual pattern recognition by moment invariants," *IRE Transactions on Information Theory*, vol. 8, no. 2, pp. 179–187, 1962. doi
3. R. M. Haralick, K. Shanmugam, and I. Dinstein, "Textural features for image classification," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-3, no. 6, pp. 610–621, 1973. doi
4. C. Harris and M. Stephens, "A combined corner and edge detector," in *Proc. Alvey Vision Conference*, pp. 23.1–23.6, 1988. doi
5. J. Matas, O. Chum, M. Urban, and T. Pajdla, "Robust wide-baseline stereo from maximally stable extremal regions," *Image and Vision Computing*, vol. 22, no. 10, pp. 761–767, 2004. doi
6. D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004. doi
7. H. Bay, A. Ess, T. Tuytelaars, and L. Van Gool, "Speeded-up robust features (SURF)," *Computer Vision and Image Understanding*, vol. 110, no. 3, pp. 346–359, 2008. doi
8. E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "ORB: An efficient alternative to SIFT or SURF," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, pp. 2564–2571, 2011. doi
9. D. Zhang and G. Lu, "Review of shape representation and description techniques," *Pattern Recognition*, vol. 37, no. 1, pp. 1–19, 2004. doi
10. T. Tuytelaars and K. Mikolajczyk, "Local invariant feature detectors: A survey," *Foundations and Trends in Computer Graphics and Vision*, vol. 3, no. 3, pp. 177–280, 2008. doi
11. L. Liu, J. Chen, P. Fieguth, G. Zhao, R. Chellappa, and M. Pietikäinen, "From BoW to CNN: Two decades of texture representation for texture classification," *International Journal of Computer Vision*, vol. 127, pp. 74–109, 2019. arXiv
12. J. Ma, X. Jiang, A. Fan, J. Jiang, and J. Yan, "Image matching from handcrafted to deep features: A survey," *International Journal of Computer Vision*, vol. 129, pp. 23–79, 2021. doi
13. Y. Jin, D. Mishkin, A. Mishchuk, J. Matas, P. Fua, K. M. Yi, and E. Trulls, "Image matching across wide baselines: From paper to practice," *International Journal of Computer Vision*, 2021. doi ·arXiv
14. D. DeTone, T. Malisiewicz, and A. Rabinovich, "SuperPoint: Self-supervised interest point detection and description," arXiv:1712.07629, 2017. arXiv
15. P.-E. Sarlin, D. DeTone, T. Malisiewicz, and A. Rabinovich, "SuperGlue: Learning feature matching with graph neural networks," arXiv:1911.11763, 2019. arXiv
16. M. J. Tyszkiewicz, P. Fua, and E. Trulls, "DISK: Learning local features with policy gradient," arXiv:2006.13566, 2020. arXiv
17. J. Sun, Z. Shen, Y. Wang, H. Bao, and X. Zhou, "LoFTR: Detector-free local feature matching with Transformers," arXiv:2104.00680, 2021. arXiv
18. P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, "LightGlue: Local feature matching at light speed," arXiv:2306.13643, 2023. arXiv
19. M. Oquab, T. Darcet, T. Moutakanni, et al., "DINOv2: Learning robust visual features without supervision," arXiv:2304.07193, 2023. arXiv
20. J. Edstedt, Q. Sun, G. Bökman, M. Wadenbäck, and M. Felsberg, "RoMa: Robust dense feature

matching," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. arXiv

21. M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981. doi

IP 13・影像圖樣分類

從最小距離與貝氏分類器，到感知器、反向傳播與卷積神經網路：電腦如何判斷一張影像或一個區域是什麼。

網站版本: https://blog.shyandram.dev/zh-tw/learn/image-processing/dip-13-pattern-classification/

你將學到

- 圖樣、圖樣向量與圖樣類別如何把「這張影像裡是什麼？」變成一個精確的決策問題
- 原型比對（prototype matching）怎麼運作：最小距離分類器、以相關進行的樣板比對、SIFT 特徵比對，以及結構描述的比對
- 為什麼貝氏分類器是最佳的，以及當每個類別都是高斯分布時它會變成什麼
- 感知器如何學習、多層網路如何用反向傳播訓練，以及卷積神經網路（CNN）如何自己學出特徵
- 訓練的實務面：過擬合、資料擴增與正則化
- 影像分類如何從 AlexNet 走到 Vision Transformer、CLIP 與 DINOv2，以及為什麼基準測試的數字值得懷疑

先看全貌

> **用最簡單的話說**
>
> 想像餐桌上有一堆水果，旁邊放了三個碗，分別寫著「蘋果」、「萊姆」、「柳丁」。你拿起每顆水果，看看顏色、摸摸大小，然後丟進它最像的那個碗。這就是圖樣分類。顏色和大小是**特徵**，碗是**類別**，你腦中那條規則（「紅色又大顆就是蘋果」）就是**分類器**。這一章講的就是怎麼建立那條規則：拿每種水果的「標準樣子」來比、用機率來算，或是讓一堆小小的加法機器組成的網路，從大量例子中自己學會規則。

第 10 到 12 章把影像切成區域，再用數字描述每個區域：面積、矩、傅立葉描述子、紋理統計量、關鍵點描述子。這一章替整條流程收尾：拿這些描述，替每一個指定一個**標籤**，例如「硬幣」、「數字 7」、「水體」、「腫瘤」、「貓」。幾乎所有實用的視覺系統最後都要做這樣的決定，所以這裡正是影像處理與機器學習交會的地方。

本章的順序既是歷史，也是觀念的推進 [1]。先是直接使用**原型**（每個類別的典型成員）的分類器；接著是**統計上最佳**的分類器，它需要知道各類別的機率分布；最後是**類神經網路**，它直接從資料學出決策規則。卷積形式的網路連特徵也一併學習，取代了第 12 章大部分的人工特徵設計。若想先溫習監督式學習、成本函數與梯度下降，可以參考ML 01・機器學習入門。

背景

白話版。分類器就是一個函數：看一份「描述」，回答「這東西屬於哪一組？」。我們用一批已知答案的例子來建立它。

圖樣（pattern）是代表某個物體或區域的一組描述子。**圖樣類別**（pattern class）是一群具有某種共同性質的圖樣。我們把 W 個類別記為 $\omega_1, \omega_2, \dots, \omega_W$。機器的**圖樣辨識**，就是自動把每個圖樣指派到它的類別，而且錯誤越少越好。

工作分成兩個階段：

1. **表示（representation）**。決定要量什麼：像素值、區域描述子、關鍵點描述子，或學出來的特徵。第 12 章處理的就是這一階段。
2. **決策（decision）**。有了量測值之後選出類別。本章處理這一階段。

方法通常以**監督式學習**訓練：手上有一組標籤已知的**訓練集**，用來設定分類器的參數；再用分類器從沒看過的**測試集**評估表現。第三份資料**驗證集**用來挑選設定，例如網路大小或何時停止訓練。把這幾份資料混用，是結果過度樂觀最常見的原因。

理解任何分類器的好方法是**決策函數**（decision function）。替每個類別 ω_j 定義一個函數 $d_j(\mathbf{x})$，再把 $\mathbf{x}$ 指派給函數值最大的類別：

$$\mathbf{x} \in \omega_i \quad \text{if} \quad d_i(\mathbf{x}) > d_j(\mathbf{x}) \quad \text{for all } j \neq i.$$

其中 $\mathbf{x}$ 是圖樣，d_j 是類別 ω_j 的純量分數。類別 i 與 j 之間的**決策邊界**，就是 $d_i(\mathbf{x}) - d_j(\mathbf{x}) = 0$ 的集合。本章從最簡單到深度網路的每一種分類器，都只是建立這些函數的不同方式。

圖樣與圖樣類別

白話版。描述物體的方式有兩種：一串數字（向量），或描述各部件怎麼連在一起（字串或樹）。

圖樣向量

最常見的表示法是 n 個量測值組成的行向量：

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix},$$

其中每個 x_k 是一個描述子，例如區域面積、色相、某個矩不變量，或某個紋理濾波器的響應。一個向量就是 n 維**特徵空間**中的一個點。好的特徵會讓同類別的圖樣彼此靠近、不同類別的圖樣彼此遠離；特徵若不好，再厲害的分類器也救不回來。

這裡有兩個實務重點。第一，單位不同的特徵需要**正規化**，否則數值範圍最大的那個特徵會主宰所有距離。以水果為例，色相落在 $[0, 1]$，直徑卻以公分計，所以比較距離前要先把每個特徵標準化成平均 0、變異數 1。第二，整張影像本身也可以是圖樣向量：把 $M \times N$ 影像攤平成 MN 維

向量。類神經網路就是這樣「看」影像的，這也是它們需要大量資料的原因，因為這個空間非常巨大。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

結構圖樣：字串與樹

有些物體用「**部件如何排列**」來描述，比用一串數字更恰當。以鏈碼（chain code，第 12 章）描出的邊界，就是一個符號**字串**，例如 `0 0 1 2 2 3 ...`。一個場景可以用**樹**描述：根節點是整張影像，子節點是主要區域，再往下是子區域。結構描述保留了「在裡面」、「在上方」、「接在後面」這類關係，而向量會把它們丟掉。對形狀、字元與圖表這類「部件的順序與連接方式承載意義」的物體，結構描述特別有用。

以原型比對進行圖樣分類

白話版。每個類別留一個典型範例。要分類新東西時，看它最像哪一個典型範例。

最小距離分類器

最簡單的原型是每個類別的**平均向量**，由該類別的訓練圖樣算出：

$$\mathbf{m}_j = \frac{1}{n_j} \sum_{\mathbf{x} \in \omega_j} \mathbf{x}, \qquad j = 1, 2, \dots, W,$$

其中 n_j 是類別 ω_j 的訓練圖樣數。新圖樣被指派到歐氏距離 $D_j(\mathbf{x}) = \|\mathbf{x} - \mathbf{m}_j\|$ 最小的類別。

對 j 最小化 $D_j^2 = \mathbf{x}^\top \mathbf{x} - 2\mathbf{x}^\top \mathbf{m}_j + \mathbf{m}_j^\top \mathbf{m}_j$，等同於最大化

$$d_j(\mathbf{x}) = \mathbf{x}^\top \mathbf{m}_j - \tfrac{1}{2}\, \mathbf{m}_j^\top \mathbf{m}_j,$$

因為 $\mathbf{x}^\top \mathbf{x}$ 與 j 無關。這個 d_j 對 $\mathbf{x}$ 是線性的。類別 i 與 j 之間的邊界為

$$d_i(\mathbf{x}) - d_j(\mathbf{x}) = \mathbf{x}^\top (\mathbf{m}_i - \mathbf{m}_j) - \tfrac{1}{2} (\mathbf{m}_i - \mathbf{m}_j)^\top (\mathbf{m}_i + \mathbf{m}_j) = 0,$$

也就是連接 $\mathbf{m}_i$ 與 $\mathbf{m}_j$ 線段的**垂直平分線**：在 2 維是一條直線，3 維是一個平面，一般情況下是超平面。圖 13.1 是它在水果資料上的結果。

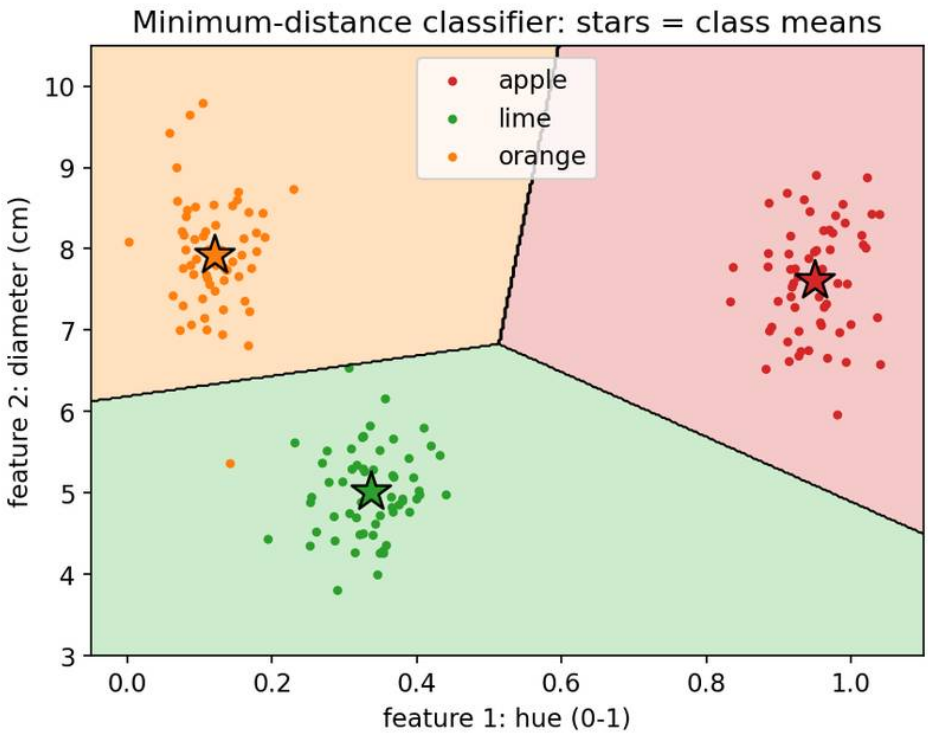


圖 *13.1* — 以最小距離分類合成的水果資料（色相、直徑）。星號是各類別平均；邊界是兩兩平均之間的垂直平分線（在標準化後的特徵上計算）。

```
import numpy as np

def fit_min_distance(X, y):
    """X: (n, d) pattern vectors, y: (n,) labels -> class list and mean vectors."""
    classes = np.unique(y)
    M = np.stack([X[y == c].mean(axis=0) for c in classes])
    return classes, M

def predict_min_distance(X, classes, M):
    d2 = ((X[:, None, :] - M[None, :, :]) ** 2).sum(axis=-1)  # (n, W) squared distances
    return classes[d2.argmin(axis=1)]

rng = np.random.default_rng(0)
X = np.r_[rng.normal([0, 0], 1, (100, 2)), rng.normal([4, 3], 1, (100, 2))]
y = np.repeat([0, 1], 100)
classes, M = fit_min_distance(X, y)
print("training accuracy:", (predict_min_distance(X, classes, M) == y).mean())
```

當各類別是緊密、大致圓形的群集，而且平均之間的距離遠大於群集本身的散布時，這個分類器表現很好。若某個類別是細長的、彎曲的，或由好幾群組成，它就會失敗，因為一個平均代表不了整個類別。兩個簡單的延伸可以改善：每個類別保留**多個原型**，或乾脆保留**所有**訓練圖樣，取最近的那一個（最近鄰規則）。

以相關進行比對（樣板比對）

當原型是一張小影像而不是向量時，就把它拿去跟大影像的每個位置比較，這是原型比對的 2 維版本。原始相關 $\sum wf$ 會偏好亮的區域，因此改用**正規化相關係數**（normalized correlation coefficient）：

$$\gamma(x,y) = \frac{\sum_{s,t}\left[w(s,t)-\bar{w}\right]\left[f(x+s,y+t)-\bar{f}_{xy}\right]}{\sqrt{\sum_{s,t}\left[w(s,t)-\bar{w}\right]^2 \sum_{s,t}\left[f(x+s,y+t)-\bar{f}_{xy}\right]^2}},$$

其中 w 是樣板，$\bar{w}$ 是它的平均，f 是影像，$\bar{f}_{xy}$ 是樣板左上角位於 (x,y) 時其下方影像區塊的平均；總和範圍是樣板的支撐區。因為先減去平均再除以標準化項，γ 落在 $[-1,1]$，而且區塊亮度被縮放或平移時不會改變。$\gamma = 1$ 表示在亮度與對比之外完全吻合。注意這是相關而非卷積：樣板不翻轉（見ML 03・卷積、影像濾波與 CNN）。

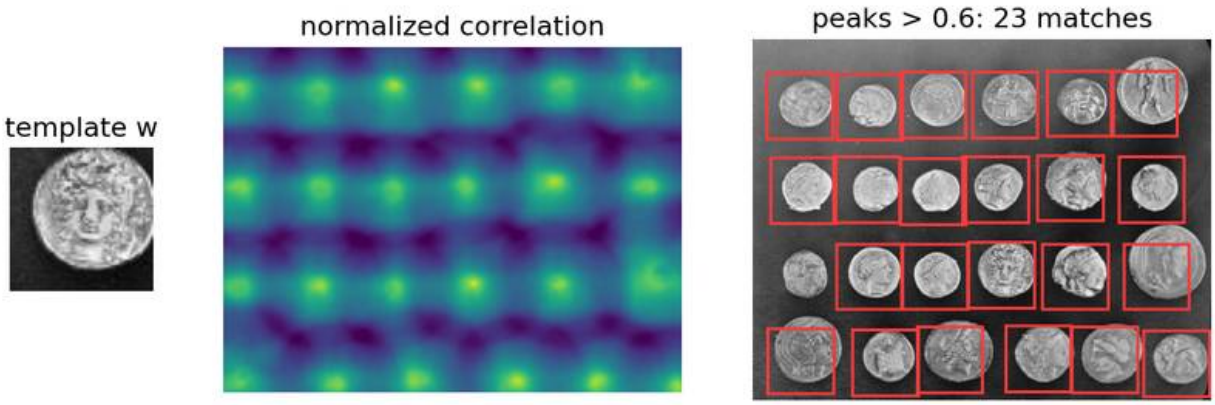


圖 *13.2* — 在 *skimage* 的 *coins* 影像上做樣板比對。左：當作樣板的一枚硬幣。中：相關圖，影像越像樣板的位置越亮。右：大於 *0.6* 的區域極大值。

```
import cv2
import numpy as np
from skimage import data

img = data.coins().astype(np.float32)
w = img[170:225, 180:235]                              # one coin as the template
R = cv2.matchTemplate(img, w, cv2.TM_CCOEFF_NORMED)  # values in [-1, 1]
y0, x0 = np.unravel_index(R.argmax(), R.shape)
print("best match at", (int(x0), int(y0)), "score", round(float(R.max()), 3))
```

圖 13.2 也顯示了這個方法的極限。樣板幾乎找到了每一枚硬幣，因為每枚硬幣都是暗背景上大小相近的亮圓盤；在這個解析度下，它分不出不同的硬幣圖案。相關對旋轉與縮放也很敏感：樣板轉 30° 或放大 30%，峰值可能就不明顯了。常見的補救方式是用一組旋轉、縮放過的樣板去搜尋，或改為比對不變特徵而非原始像素。

比對 SIFT 特徵

第 12 章的尺度不變特徵轉換（scale-invariant feature transform，SIFT）[3] 正好提供這類不變特徵。每個關鍵點都有位置、尺度、方向和一個 128 維描述子。要判斷某個原型物體是否出現在場景影像中：

1. 分別從原型與場景擷取關鍵點與描述子。
2. 對原型的每個描述子，以歐氏距離找出場景中最近與第二近的描述子。
3. 只有當最近距離明顯小於第二近距離時才接受，例如 $d_1/d_2 < 0.8$。這個**比值檢定**（ratio test）[3] 能排除重複紋理中的模稜兩可配對。
4. 檢查被接受的配對是否符合同一個幾何轉換（相似、仿射或投影轉換）；不符合的就是離群值。

若通過幾何一致性檢查的配對夠多，就判定「物體存在」。這仍然是原型比對：原型是一組描述子，而「距離」是一致配對的數量。它能容忍旋轉、尺度變化、部分遮蔽和中等程度的視角改變，這些都是樣板比對做不到的。

比對結構原型

對字串與樹，距離必須同時比較符號與它們的順序。兩個簡單的量度：

- **形狀數相似度**。若兩個封閉邊界被編碼成形狀數（shape number，正規化的鏈碼差分，第 12 章），它們的**相似程度** k 就是兩個形狀數仍然相同的最大階數（編碼長度）。相似的形狀在高階時仍然一致。距離可以定義為 $D = 1/k$：形狀相同時為零，在粗略階數就已不同時則很大。
- **字串相似度**。把字串 a 和 b 逐個符號對齊。令 α 為相符的位置數，$\beta = \max(|a|, |b|) - \alpha$ 為不相符的位置數，$|a|$ 是 a 的長度。則 $R = \alpha/\beta$ 在完全相符時為無限大，完全不符時為 0。比較寬容的版本是**編輯距離**（edit distance）：把一個字串變成另一個所需的最少插入、刪除與替換次數，可用動態規劃計算。

新字串被指派給原型字串最相似的類別。結構比對如今較少見，但「比較部件的**序列**與**圖**」這個想法，仍延續在圖比對，以及比較一組 token 的 Transformer 模型中。

最佳（貝氏）統計分類器

白話版。如果我們知道每個類別出現得有多頻繁、它的量測值通常長什麼樣子，就存在一條最好的規則：選出「在已量到的值之下，機率最高」的類別。平均而言，沒有任何規則犯的錯會比它更少。

推導

令 $p(\omega_k \mid \mathbf{x})$ 為圖樣 $\mathbf{x}$ 來自類別 ω_k 的機率，L_{kj} 為把真正屬於 ω_k 的圖樣判為 ω_j 時的**損失**。選擇類別 ω_j 的期望損失稱為**條件風險**（conditional risk）：

$$r_j(\mathbf{x}) = \sum_{k=1}^{W} L_{kj}\, p(\omega_k \mid \mathbf{x}).$$

利用貝氏定理 $p(\omega_k \mid \mathbf{x}) = p(\mathbf{x} \mid \omega_k)\, P(\omega_k)/p(\mathbf{x})$，可以改用**類別條件密度** $p(\mathbf{x} \mid \omega_k)$（類別 ω_k 的圖樣長什麼樣子）與**事前機率** $P(\omega_k)$（這個類別有多常見）來表示。$1/p(\mathbf{x})$ 對每個 j 都一樣，比較風險時可以省略。

對每個 $\mathbf{x}$ 都選出 $r_j(\mathbf{x})$ 最小之 j 的分類器，會使整體平均損失最小，這就是它被稱為**最佳**的原因：在已知真實分布的前提下，沒有任何規則能做得更好。在常見的 **0–1 損失**下（$k = j$ 時 $L_{kj} = 0$，否則為 1），每種錯誤代價相同，

$$r_j(\mathbf{x}) \propto p(\mathbf{x}) - p(\mathbf{x} \mid \omega_j) P(\omega_j).$$

最小化它等於最大化決策函數

$$d_j(\mathbf{x}) = p(\mathbf{x} \mid \omega_j)\, P(\omega_j).$$

換句話說：選出事後機率最大的類別。問題在於我們從來不知道真實的密度，只能從訓練資料估計，而分類器的好壞取決於這些估計的好壞。

高斯圖樣類別的貝氏分類器

假設每個類別都是平均為 $\mathbf{m}_j$、共變異數矩陣為 $\mathbf{C}_j$ 的多變量高斯分布：

$$p(\mathbf{x} \mid \omega_j) = \frac{1}{(2\pi)^{n/2} |\mathbf{C}_j|^{1/2}} \exp\Big[-\tfrac{1}{2} (\mathbf{x} - \mathbf{m}_j)^{\top} \mathbf{C}_j^{-1} (\mathbf{x} - \mathbf{m}_j) \Big],$$

其中 n 是 $\mathbf{x}$ 的維度，$|\mathbf{C}_j|$ 是 $\mathbf{C}_j$ 的行列式。因為對數是遞增函數，可以改為最大化 $\ln d_j$，並丟掉常數 $\frac{n}{2} \ln 2\pi$：

$$d_j(\mathbf{x}) = \ln P(\omega_j) - \tfrac{1}{2} \ln |\mathbf{C}_j| - \tfrac{1}{2} (\mathbf{x} - \mathbf{m}_j)^{\top} \mathbf{C}_j^{-1} (\mathbf{x} - \mathbf{m}_j).$$

最後一項是**馬氏距離**（Mahalanobis distance）平方的一半；這種距離會把空間拉伸，使每個類別的散布看起來是圓的。這個 d_j 對 $\mathbf{x}$ 是二次的，所以類別之間的邊界是二次曲面：在 2 維是橢圓、拋物線或雙曲線。兩個特例把我們帶回前面的小節：

- **共變異數相同**（所有 j 皆 $\mathbf{C}_j = \mathbf{C}$）。二次項 $\mathbf{x}^{\top} \mathbf{C}^{-1} \mathbf{x}$ 對所有類別都相同而抵消，剩下線性函數 $d_j(\mathbf{x}) = \ln P(\omega_j) + \mathbf{x}^{\top} \mathbf{C}^{-1} \mathbf{m}_j - \frac{1}{2} \mathbf{m}_j^{\top} \mathbf{C}^{-1} \mathbf{m}_j$。邊界變成超平面。
- **單位共變異數且事前機率相等**（$\mathbf{C} = \mathbf{I}$，$P(\omega_j) = 1/W$）。函數化簡為 $\mathbf{x}^{\top} \mathbf{m}_j - \frac{1}{2} \mathbf{m}_j^{\top} \mathbf{m}_j$，正是最小距離分類器。所以最小距離分類器只在「類別是圓形、散布相同、出現機率相等的高斯分布」時才是貝氏最佳。

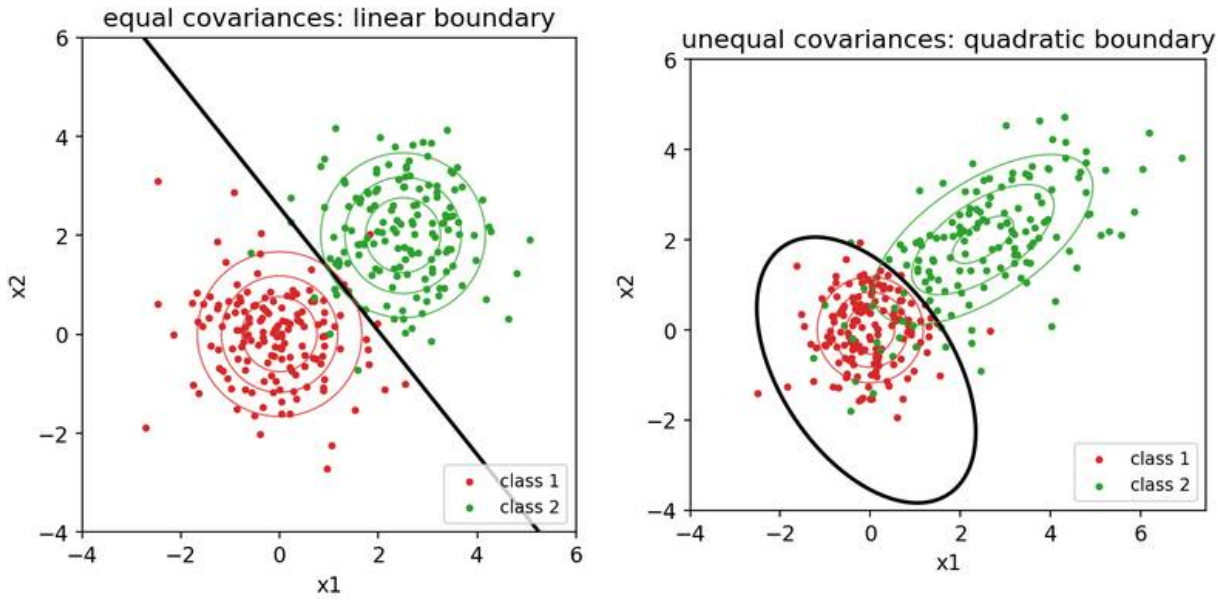


圖 *13.3* — 兩個高斯類別的貝氏決策邊界（粗黑線）。細等高線是各類別的密度。左：共變異數相同，邊界是直線。右：共變異數不同，邊界是包住較集中類別的二次曲線。

從樣本估計 $\mathbf{m}_j$ 與 $\mathbf{C}_j$ 很容易：樣本平均與樣本共變異數。若訓練集中各類別的比例反映真實情況，事前機率就可以用這些比例。

```
import numpy as np

class GaussianBayes:
    def fit(self, X, y):
        self.classes = np.unique(y)
        self.m = [X[y == c].mean(0) for c in self.classes]
        self.C = [np.cov(X[y == c].T) for c in self.classes]
        self.logP = [np.log((y == c).mean()) for c in self.classes]
        return self

    def decision(self, X):
        scores = []
        for m, C, lp in zip(self.m, self.C, self.logP):
            d = X - m
            maha = np.einsum("ni,ij,nj->n", d, np.linalg.inv(C), d)
            scores.append(lp - 0.5 * np.linalg.slogdet(C)[1] - 0.5 * maha)
        return np.stack(scores, axis=1)            # (n, W): one d_j(x) per class

    def predict(self, X):
        return self.classes[self.decision(X).argmax(1)]
```

每個類別的共變異數參數數量是 $n(n+1)/2$。4 個特徵時每類 10 個數字，很容易估計；1,000 個特徵時約有 50 萬個，除非資料極多，否則估計會很不可靠。這是高斯貝氏分類器最適合用在「短而精挑細選」的特徵向量上的原因之一。

應用：多光譜影像的像素分類

遙測衛星會在多個光譜波段記錄地面上的每個位置。於是每個像素自然就是一個圖樣向量 $\mathbf{x} = (x_{\text{blue}}, x_{\text{green}}, x_{\text{red}}, x_{\text{NIR}})^\top$，NIR 代表近紅外光。健康的植被在近紅外光反射強、在紅光反射弱；水體在近紅外光很暗；建成區在各波段都中等偏亮。

我們合成了一張 128×128 的場景，含三個類別（水體、植被、建成區），替每個類別設定四個波段的平均反射率，並在每個波段加上標準差 0.06 的獨立高斯雜訊。每類只隨機挑 50 個像素當訓練資料，估計 $\mathbf{m}_j$ 與 $\mathbf{C}_j$，再用高斯貝氏規則分類所有像素。

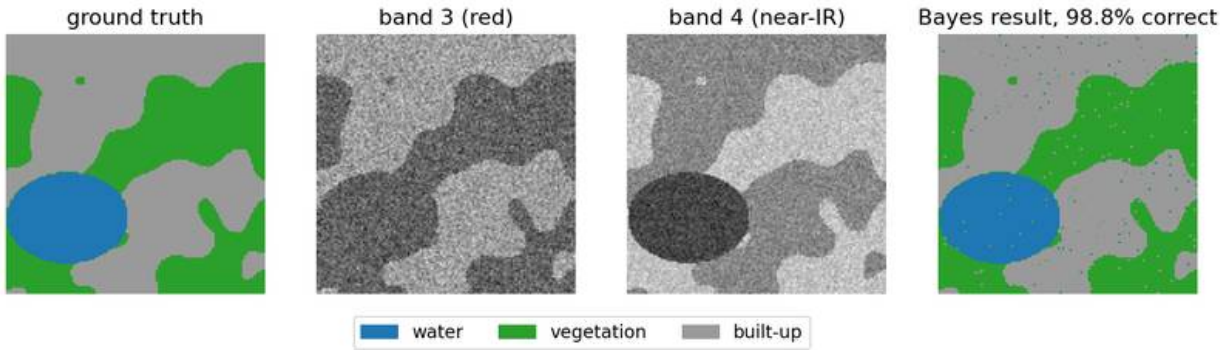


圖 *13.4* — 以貝氏規則分類合成的四波段場景。單一波段雜訊多又模稜兩可（紅光幾乎分不開水體與植被），但四維圖樣向量能把類別分得很好：只用 *150* 個訓練像素，就有 *98.8%* 的像素被正確分類。

這裡有兩個教訓。第一，單一波段不夠：在紅光波段中，水體與植被的值很接近，是波段的**組合**把它們分開的。第二，剩下的錯誤都是零散的單一像素。這裡的貝氏規則把每個像素獨立處理，忽略了鄰居。加入空間脈絡，例如平滑決策分數，或使用能看到一整個鄰域的分類器，就能去掉大部分這類錯誤。這個觀察自然地把我們帶到卷積網路。

類神經網路與深度學習

白話版。類神經網路是一疊非常簡單的單元。每個單元把輸入乘上權重加總，再決定要「發射」多強。每犯一次錯就微調一下權重，網路就能自己學會決策規則，不需要我們寫出機率分布。

背景

貝氏分類器需要密度估計，而在高維空間中這很難。另一條路是**直接從訓練資料學出決策函數**。類神經網路用許多簡單、可調整、分層排列的運算單元做到這一點。這個想法可追溯到 1950 年代末的感知器 [4]；1986 年反向傳播廣為人知後，多層網路才變得實用 [5]；2012 年深度卷積網路贏得 ImageNet 競賽後，它在影像領域成為主流 [7]。關於感知器、MLP、激活函數與訓練技巧的精簡整理，可參考ML 02・類神經網路。

感知器

感知器（perceptron）計算輸入的加權和加上偏差，然後輸出正負號：

$$d(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0 = \sum_{k=1}^{n} w_k x_k + w_0, \qquad \text{class} = \begin{cases} \omega_1 & d(\mathbf{x}) > 0 \\ \omega_2 & d(\mathbf{x}) \le 0 \end{cases}$$

其中 $\mathbf{w} = (w_1, \dots, w_n)^\top$ 是**權重**，w_0 是**偏差**（bias）。邊界 $d(\mathbf{x}) = 0$ 是一個超平面。為了簡化符號，在每個圖樣後面補一個常數 1，$\mathbf{y} = (x_1, \dots, x_n, 1)^\top$，並把偏差併入權重 $\mathbf{w} = (w_1, \dots, w_n, w_0)^\top$，於是 $d = \mathbf{w}^\top \mathbf{y}$。

學習規則。把標籤編碼成：ω_1 為 $t = +1$、ω_2 為 $t = -1$。一次呈現一個訓練圖樣，在第 k 步：

$$\mathbf{w}(k+1) = \begin{cases} \mathbf{w}(k) + \alpha\, t\, \mathbf{y}(k) & \text{if } t\, \mathbf{w}(k)^\top \mathbf{y}(k) \le 0 \text{ (misclassified)} \\ \mathbf{w}(k) & \text{otherwise,} \end{cases}$$

其中 $\alpha > 0$ 是**學習率**。每次修正都會移動超平面，把出錯的圖樣推向正確的一側：更新後 $t\, \mathbf{w}^\top \mathbf{y}$ 會增加 $\alpha \|\mathbf{y}\|^2$。若兩個類別**線性可分**，這個程序保證在有限次修正後停止，並得到一個分隔超平面；若不可分，它永遠不會穩定下來。這時改用梯度下降去最小化一個平滑的誤差，例如 $\mathbf{w}^\top \mathbf{y}$ 與 t 的差的平方（最小均方法，又稱 delta 規則）。這個想法直接通往反向傳播。

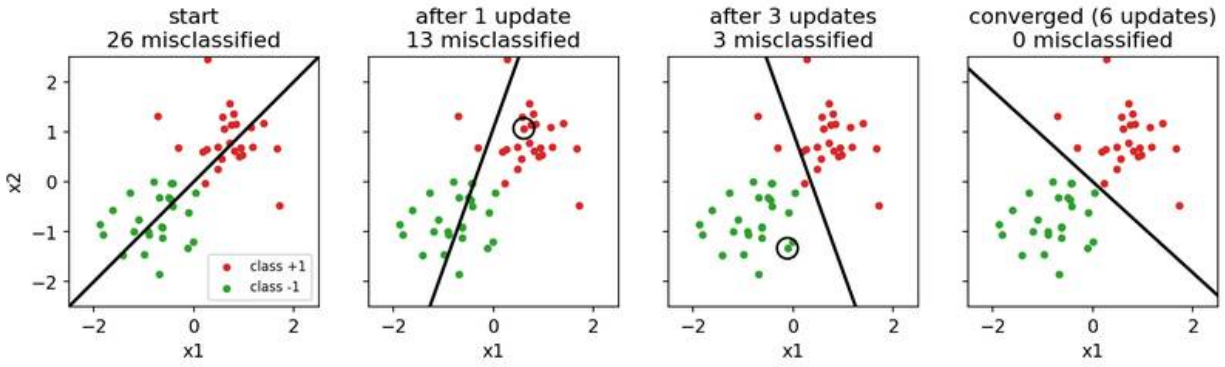


圖 *13.5* — 在兩個可分類別上訓練感知器。圈起來的點是觸發該次更新的錯誤分類圖樣。修正六次後，直線已能分開兩類，學習停止。

```
import numpy as np

def train_perceptron(X, y, alpha=0.5, epochs=100):
    """X: (n, d), y in {-1, +1}. Returns augmented weights (w_1..w_d, w_0)."""
    Xa = np.c_[X, np.ones(len(X))]                # append the constant input 1
    w = np.zeros(Xa.shape[1])
    for _ in range(epochs):
        mistakes = 0
        for xi, yi in zip(Xa, y):
            if yi * (w @ xi) <= 0:                # wrong side (or on the line)
                w += alpha * yi * xi
                mistakes += 1
        if mistakes == 0:                         # a full clean pass: converged
```

```
            break
    return w
```

單一感知器只能畫出一個超平面。它解不了 XOR：兩個類別位於正方形的對角。要畫出彎曲或分成多段的邊界，就需要更多層。

多層前饋網路

多層前饋網路（multilayer feedforward network，又稱多層感知器 MLP）把多層單元疊起來。第 ℓ 層接收第 $\ell-1$ 層的輸出，預測時沒有任何訊號往回流。與感知器不同的是，每個單元會把加權和送進一個平滑、可微分的**激活函數** h，例如 sigmoid $h(z)=1/(1+e^{-z})$、雙曲正切，或線性整流單元 ReLU$(z)=\max(0,z)$。平滑性讓以梯度為基礎的訓練成為可能；非線性則讓深度有意義，因為一疊純線性的層會塌縮成單一線性層。

只要有兩個隱藏層和足夠的單元，網路幾乎能形成任何形狀的決策區域。實務上的問題不是網路**能不能**表示某個邊界，而是訓練能不能從手邊的資料中**找到**它。

前向傳遞

把層編號為 $\ell=1,\dots,L$，第 0 層是輸入 $\mathbf{a}^{(0)}=\mathbf{x}$。每一層：

$$\mathbf{z}^{(\ell)}=\mathbf{W}^{(\ell)}\mathbf{a}^{(\ell-1)}+\mathbf{b}^{(\ell)},\qquad \mathbf{a}^{(\ell)}=h(\mathbf{z}^{(\ell)}),$$

其中 $\mathbf{W}^{(\ell)}$ 是權重矩陣（第 i 列是進入單元 i 的權重），$\mathbf{b}^{(\ell)}$ 是偏差向量，$\mathbf{z}^{(\ell)}$ 是**淨輸入**，$\mathbf{a}^{(\ell)}$ 是**激活值**，h 逐元素作用。對 W 類問題，輸出層有 W 個單元，其淨輸入以 **softmax** 函數轉成機率：

$$p_j=\frac{e^{z_j^{(L)}}}{\sum_{i=1}^{W} e^{z_i^{(L)}}},\qquad j=1,\dots,W,$$

預測類別就是 p_j 最大的那一個。注意輸出層就是一組決策函數 $d_j(\mathbf{x})=z_j^{(L)}$，和「背景」一節完全相同。隱藏層算出的特徵，讓這些簡單的線性決策函數就足夠了。

反向傳播

訓練就是調整所有權重，降低衡量輸出錯得多離譜的**損失** E。分類問題的標準選擇是**交叉熵** $E=-\sum_j t_j \ln p_j$，其中 $\mathbf{t}$ 是 one-hot 標籤（正確類別為 1，其餘為 0）。對 sigmoid 輸出，經典的替代選擇是平方誤差 $E=\frac{1}{2}\|\mathbf{a}^{(L)}-\mathbf{t}\|^2$。我們需要每一層的 $\partial E/\partial \mathbf{W}^{(\ell)}$，反向傳播用一次由後往前的連鎖律就把它們全部算出來 [5]。

定義第 ℓ 層的**誤差訊號**為 $\delta^{(\ell)}=\partial E/\partial \mathbf{z}^{(\ell)}$。

步驟 1：輸出層。 softmax 搭配交叉熵時，導數出奇地簡單：

$$\delta^{(L)}=\mathbf{p}-\mathbf{t}.$$

（sigmoid 輸出搭配平方誤差時則是 $(\mathbf{a}^{(L)} - \mathbf{t}) \odot h'(\mathbf{z}^{(L)})$，$\odot$ 表示逐元素相乘。）

步驟 2：往回傳。第 ℓ 層的單元 i 只透過第 $\ell+1$ 層的淨輸入影響 E，且 $\partial z_k^{(\ell+1)} / \partial a_i^{(\ell)} = W_{ki}^{(\ell+1)}$。由連鎖律得到

$$\delta^{(\ell)} = h'(\mathbf{z}^{(\ell)}) \odot \left(\mathbf{W}^{(\ell+1)\top} \delta^{(\ell+1)} \right), \qquad \ell = L-1, \dots, 1.$$

誤差沿著前向時傳遞訊號的同一組權重（轉置後）往回走。

步驟 3：梯度。因為 $z_i^{(\ell)} = \sum_k W_{ik}^{(\ell)} a_k^{(\ell-1)} + b_i^{(\ell)}$，

$$\frac{\partial E}{\partial \mathbf{W}^{(\ell)}} = \delta^{(\ell)} \, \mathbf{a}^{(\ell-1)\top}, \qquad \frac{\partial E}{\partial \mathbf{b}^{(\ell)}} = \delta^{(\ell)}.$$

每個權重的梯度就是「輸出端的誤差」乘以「輸入端的激活值」。

步驟 4：更新。以學習率 α 做梯度下降：

$$\mathbf{W}^{(\ell)} \leftarrow \mathbf{W}^{(\ell)} - \alpha \frac{\partial E}{\partial \mathbf{W}^{(\ell)}}, \qquad \mathbf{b}^{(\ell)} \leftarrow \mathbf{b}^{(\ell)} - \alpha \frac{\partial E}{\partial \mathbf{b}^{(\ell)}}.$$

反向傳遞的計算量和前向傳遞差不多，這正是擁有數百萬個權重的網路能被訓練的原因。感知器規則可以看成「單層、使用階梯激活函數」的特例。

訓練

實務上，我們在一小批隨機挑選的訓練圖樣（**mini-batch**）上平均梯度、更新，然後重複，這就是**隨機梯度下降**（SGD）。走過整個訓練集一次稱為一個 **epoch**。訓練會持續許多個 epoch，同時觀察驗證集上的損失。學習率是最重要的設定：太大，損失會震盪甚至發散；太小，訓練慢如蝸牛。權重必須以小的隨機數初始化而不是全零，否則同一層的所有單元會算出同樣的東西、收到同樣的更新，永遠分不開。

下面的 NumPy 程式訓練一個兩層網路解 XOR，也就是單一感知器解不了的問題：

```
import numpy as np

rng = np.random.default_rng(0)
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], float)
y = np.array([[0], [1], [1], [0]], float)         # XOR: not linearly separable

W1, b1 = rng.normal(0, 1, (2, 8)), np.zeros(8)    # input -> 8 hidden units
W2, b2 = rng.normal(0, 1, (8, 1)), np.zeros(1)    # hidden -> 1 output
sig = lambda z: 1 / (1 + np.exp(-z))
alpha = 1.0

for step in range(5000):
    # forward pass
    a1 = np.tanh(X @ W1 + b1)
    out = sig(a1 @ W2 + b2)
    # backward pass (cross-entropy loss with a sigmoid output)
    d2 = (out - y) / len(X)                       # delta at the output layer
    d1 = (d2 @ W2.T) * (1 - a1 ** 2)              # delta at the hidden layer
```

```
    W2 -= alpha * a1.T @ d2;  b2 -= alpha * d2.sum(0)
    W1 -= alpha * X.T @ d1;   b1 -= alpha * d1.sum(0)

print(out.round(3).ravel())                          # close to [0, 1, 1, 0]
```

（這裡每一列是一個圖樣，所以矩陣乘法相對於上面的式子是轉置的。）

深度卷積神經網路

白話版。我們不告訴網路該量哪些特徵，而是讓它把可學習的小濾波器在影像上滑動，就像第 3 章的濾波器一樣，並自己學會哪些濾波器有助於分類。結果前幾層會找出邊緣和斑點，後面幾層再把它們組合成部件與整個物體。

為什麼全連接網路處理影像很吃力

把一張 20×20 影像送進一個有 2,048 個隱藏單元的全連接層，這一層就有 $400 \times 2048 = 819{,}200$ 個權重。而且這一層忽略了「相鄰像素彼此相關」以及「往右移兩個像素的 3 還是 3」這兩件事。CNN 把這兩點直接建進結構裡 [6]。

卷積層

卷積層對輸入套用 F 個小核（例如 5×5）。若有 C 個輸入通道 a_c，輸出通道 f 在像素 (x, y) 的淨輸入為

$$z_f(x,y) = \sum_{c=1}^{C} \sum_{s,t} w_{f,c}(s,t)\, a_c(x+s, y+t) + b_f,$$

再接一個激活函數，通常是 ReLU：$a_f(x,y) = \max(0, z_f(x,y))$。這個總和和第 3 章一樣是空間相關，但核的值 $w_{f,c}$ 是**學出來的**。每個輸出通道是一張**特徵圖**（feature map）：在核所對應的圖樣出現的地方，值就大。有兩個想法大幅減少參數量：

- **局部連接**。每個輸出只取決於一個小鄰域（它的**感受野**，receptive field）。
- **權重共享**。同一個核用在每一個位置，所以在某處學到的特徵，在任何地方都能偵測到。

在單通道影像上用 8 個 5×5 核，只需要 $8 \times 25 + 8 = 208$ 個參數，相比之下前面是 819,200 個。

池化層

池化層（pooling layer）縮小每張特徵圖，通常是取每個不重疊 2×2 區塊的最大值（max-pooling）或平均值。它讓寬和高減半，使響應能容忍小幅位移，也擴大了後續層的感受野。卷積與池化堆疊數次後，得到的特徵會對更大、更抽象的結構產生響應。

全連接層與整條流程

最後一個池化層之後，特徵圖被**攤平**成一個向量，經過一或多個全連接層，最後接 softmax。圖 13.6 是我們訓練的小網路。

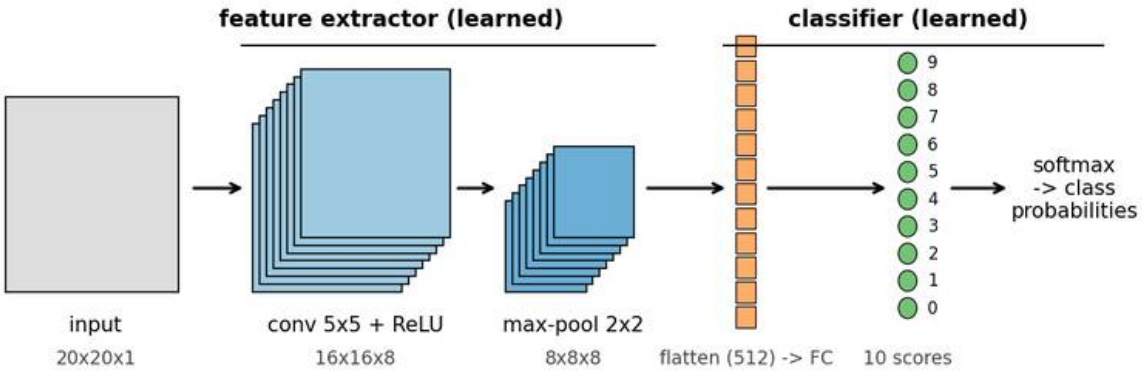


圖 *13.6* — 本章使用的小型 *CNN*。卷積與池化層構成學出來的特徵擷取器；全連接層是作用在這些特徵上的、學出來的線性分類器。

這是本章最關鍵的觀念。傳統流程是**人工設計的特徵 → 分類器**；CNN 則是**學出來的特徵 → 分類器**，兩部分一起用反向傳播訓練，最小化同一個損失。最後的全連接 softmax 層只是一組線性決策函數，和最小距離分類器、等共變異數貝氏分類器中遇到的是同一種東西。改變的是：這些線性函數所作用的空間本身是學出來的。

訓練 CNN

反向傳播完全照用，只是局部導數不同：

- **卷積**。因為一個權重在所有位置共享，它的梯度是所有位置的**總和**：$\partial E/\partial w_{f,c}(s,t) = \sum_{x,y} \delta_f(x,y)\, a_c(x+s, y+t)$，本身就是輸入與誤差圖的相關。傳回輸入的誤差，則是 δ_f 與（翻轉後的）核的卷積。
- **最大池化**。梯度只流向每個區塊中原本是最大值的那個輸入，其他為零。
- **ReLU**。$z > 0$ 處讓梯度通過，$z \le 0$ 處擋下。

以下用 SciPy 寫出一個「卷積、ReLU、池化」階段的前向傳遞：

```
import numpy as np
from scipy.signal import correlate2d
from skimage import data, transform

img = transform.resize(data.camera(), (64, 64), anti_aliasing=True)
k = np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]], float)    # one 3x3 "filter"
fmap = np.maximum(correlate2d(img, k, mode="valid") + 0.0, 0)  # conv + bias + ReLU
pooled = fmap[:62, :62].reshape(31, 2, 31, 2).max(axis=(1, 3))  # 2x2 max-pooling
print(img.shape, "->", fmap.shape, "->", pooled.shape)
```

實作範例：數字辨識

CNN 的經典試驗場是手寫數字辨識，LeNet 類型的網路就是在 MNIST 數字資料上發展出來的 [6]。為了讓一切自給自足、可重現，我們自行產生**類 MNIST** 的 20×20 數字：每個 0–9 的數字以六種 OpenCV 字型之一繪出，大小、筆畫粗細、位移與旋轉（$\pm 15°$）都是隨機的，最後加上雜訊（圖 13.7）。

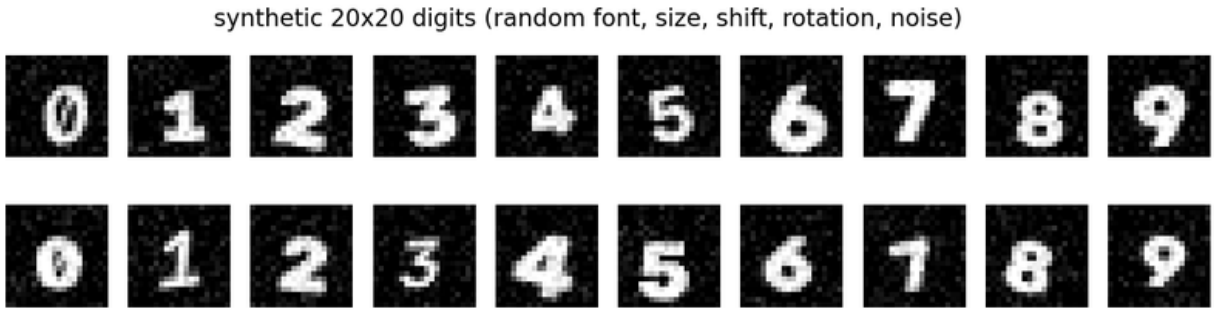


圖 *13.7* — 合成的訓練數字。字型、大小、位置與角度的變化，迫使分類器學習形狀，而不是背下像素。

圖 13.6 的網路（八個 5×5 濾波器、ReLU、2×2 最大池化、接到 10 個輸出的全連接層、softmax 搭配交叉熵）在 `scripts/figures/dip_ch13.py` 中用大約 80 行 NumPy 寫成。我們以 mini-batch SGD（batch 大小 32、學習率 0.05、少量權重衰減）在 3,000 張數字上訓練 12 個 epoch，在另外 1,000 張驗證數字上達到 **98.5%** 準確率。圖 13.8 呈現它學到了什麼。

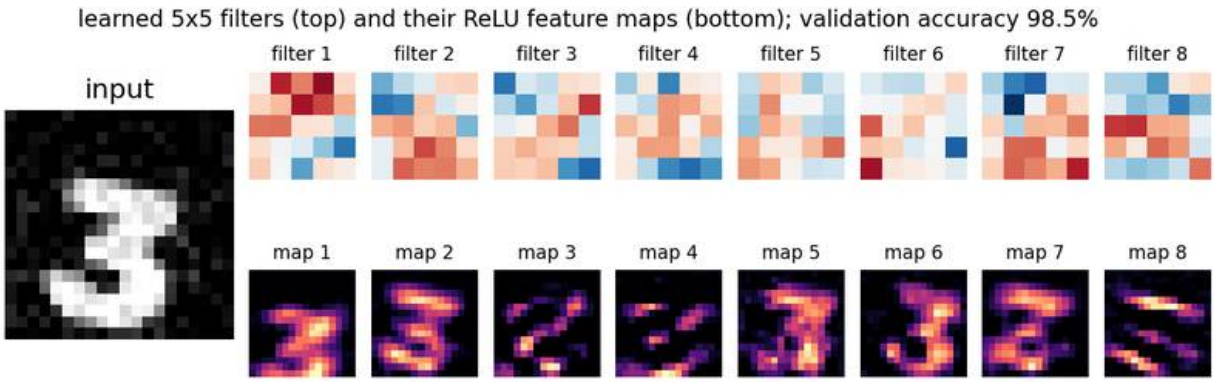


圖 *13.8* — 訓練後 *CNN* 的內部。上：八個學出來的 *5×5* 核（紅為正、藍為負）。下：它們對輸入數字的 *ReLU* 特徵圖。有幾個核像是有方向性的邊緣或筆畫偵測器，和第 *10* 章人工設計的梯度濾波器很相似。

沒有人叫網路去偵測邊緣。有方向性、像邊緣的核之所以出現，是因為它們對分辨數字有用。在自然影像上訓練的大型網路，第一層也有相同的現象，越深的層則出現越複雜的圖樣。

一些實作上的細節

白話版。把作業背得滾瓜爛熟的網路，考試還是可能考砸。訓練的實務技巧，大多是在讓它學會一般規則，而不是記住特定例子。

過擬合

參數多、訓練例子少的網路，可以靠死背把訓練誤差壓到零，但對新資料的誤差仍然很高。訓練表現與驗證表現之間的這個落差就是**過擬合**（overfitting）。圖 13.9（左）清楚呈現了這一點：只用 150 張數字訓練時，CNN 的訓練準確率達到 100%，驗證準確率卻只有 86.2%。

基本的防線：

- **更多資料**。沒有什麼比它更可靠。資料擴增（見下）是取得更多資料的便宜方法。
- **驗證集與提早停止（early stopping）**。追蹤驗證準確率，保留最佳 epoch 的權重。
- **更小或結構更好的模型**。與全連接層相比，CNN 的權重共享本身就是很強的正則化。

資料擴增

資料擴增（data augmentation）以保留標籤的轉換產生新的訓練例子：小幅位移、旋轉、縮放、翻轉（只在合理時使用：翻轉的貓還是貓，但翻轉的 b 會變成 d）、裁切、亮度與對比變化，以及加入雜訊 [18]。每個 epoch 看到的每張影像都略有不同，因此更難死背，而對這些轉換的不變性則從資料中學到。

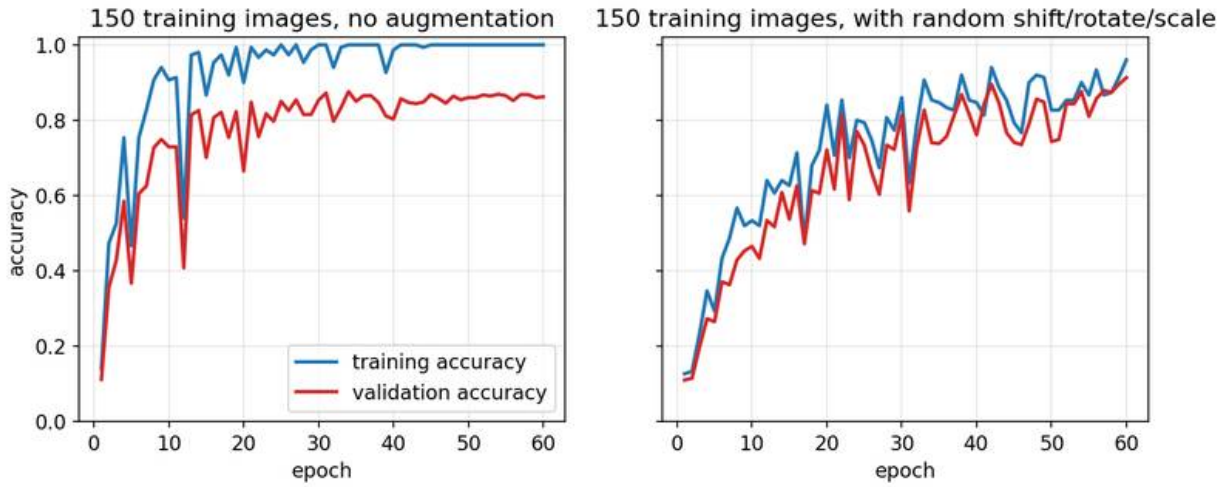


圖 *13.9* — 同一個 *CNN* 只用 *150* 張合成數字訓練 *60* 個 *epoch*。左：沒有擴增時，訓練與驗證之間出現很大的落差（過擬合）。右：隨機位移、旋轉與縮放讓兩條曲線保持接近，最終驗證準確率從 *86.2%* 提升到 *91.3%*。

正則化

正則化（regularization）加入對較簡單解的偏好：

- **權重衰減**（weight decay，L_2 正則化）在損失中加上 $\frac{\lambda}{2}\sum w^2$，等於在每個權重的梯度上加 λw，把權重往零收縮。我們的訓練腳本使用 $\lambda = 10^{-4}$。
- **Dropout** 在每個訓練步驟隨機把一部分單元設為零，讓任何單元都無法依賴特定的夥伴。測試時使用所有單元，並適當縮放權重。效果類似於把許多「變瘦」的網路做集成平均 [17]。

其他實務細節同樣重要：輸入正規化（平均 0、變異數 1）、以「依輸入數量縮放的變異數」初始化權重（讓激活值在層與層之間既不爆炸也不消失）、訓練過程中逐步降低學習率，以及永遠只在開發期間沒被用來做任何決定的測試集上報告最終結果。

現代觀點

教科書這一章結束的地方，正是現代影像分類開始的地方。以下六篇綜述與回顧論文，勾勒出這個領域此後的走向。

深度學習之前的統計圖樣辨識。 Jain、Duin 與 Mao 的回顧 [2] 至今仍是傳統領域最清楚的地圖：表示、特徵擷取與選擇、貝氏與最近鄰規則、線性與非線性分類器、分類器結合，以及錯誤率估計。它的主要結論是：特徵表示與評估流程比分類器的選擇更重要。這個結論經得起時間考驗。深度學習並沒有推翻它，而是把「表示」這一部分自動化了。

CNN 時代。 Rawat 與 Wang [11] 回顧了用於影像分類的 CNN，從 1980 年代末的起源一路到 2017 年，依架構、正則化、最佳化與實務技巧整理了三百多篇論文。他們描繪的脈絡始於 LeCun 等人以梯度訓練、用於文件辨識的網路 [6]；AlexNet [7] 使用 GPU、ReLU、dropout 與資料擴增，在 2012 年贏得 ImageNet 大規模視覺辨識競賽 [10]，讓這條路線加速；VGG [8] 顯示以小型 3×3 核堆出的深度有幫助；ResNet [9] 透過恆等捷徑連接學習**殘差**函數，讓非常深的網路變得可以訓練。這些架構都仍看得出圖 13.6 的影子：卷積、非線性、池化、線性分類器，以反向傳播訓練。

視覺中的 Transformer。 Vision Transformer（ViT）[12] 把影像切成 16×16 的小塊，把每塊當成一個 token，以自注意力（self-attention）而非卷積處理這個序列。只要預訓練資料足夠，它能追平甚至超越 CNN。Khan 等人的綜述 [13] 涵蓋了用於分類、偵測、分割等任務的眾多視覺 Transformer 變體，並討論它們對大規模預訓練的依賴。隨後 ConvNeXt [14] 顯示：一個純 CNN，只要借用 Transformer 的設計選擇一步步現代化，又能再度與之匹敵。實務上的教訓是：架構家族的差異，不如訓練資料、規模與訓練配方來得重要。

基礎模型特徵與零樣本分類。 2021–2023 年的兩項發展改變了「訓練分類器」的意義。CLIP [15] 同時訓練一個影像編碼器與一個文字編碼器，讓相符的影像–說明文字對擁有相近的嵌入向量。於是不需要任何標註影像，就能建出新的分類器：把 `a photo of a {class}` 這類提示文字嵌入，再把每張影像指派給文字嵌入最相似的類別。這就是一個**最小距離分類器**，只是原型來自語言。DINOv2 [16] 在一個大型、經過篩選的影像集上以自監督方式學習通用視覺特徵，完全不用標籤；它凍結的特徵只要在上面接一個線性層或最近鄰規則，就能做出很強的分類。兩者都把最昂貴的部分，也就是特徵擷取器，只訓練一次；上面的分類器往往正是本章前半的簡單規則之一。

資料擴增與正則化。 Shorten 與 Khoshgoftaar 的綜述 [18] 把擴增方法分成基本影像操作（幾何與色彩轉換、核濾波、影像混合、隨機抹除）與學習式方法（對抗式訓練、以 GAN 合成、神經風格轉換、以後設學習搜尋擴增策略）。它的結論和圖 13.9 一致：在標註資料有限時，擴增是降低過擬合最便宜也最有效的方法之一。

評估陷阱與資料集偏差。 Torralba 與 Efros [19] 問了一個簡單的問題：分類器能不能分辨一張影像來自哪個**資料集**？答案是可以，而且遠高於隨機猜測，顯示每個基準資料集都有自己的「指紋」；在一個資料集上訓練的模型，換到另一個資料集時也泛化得很差。Liu 與 He [20] 用現代網路，以及非常大、非常多樣的網路規模資料集重做了這個實驗：現代網路仍能在未見過的影像上高準確率地辨認出來源資料集，而且靠的是可泛化的特徵而不是死背。資料集偏差並沒有因為規模變大而消失。由此有兩個實務結論：在某個基準上的準確率，描述的是那個基準，而不是真實的視覺世界；只要可能，就用與訓練資料獨立蒐集的資料來測試。

改變了什麼，沒改變什麼。深度學習取代了人工特徵，也讓分類器本身幾乎成了附屬品：一個線性 softmax 層。但它沒有改變決策理論的基礎。softmax 輸出被訓練來估計事後機率，所以網路是在逼近貝氏規則；交叉熵就是負對數概似。原型比對以「嵌入空間中的最近鄰搜尋」形式延續，用於零樣本分類與檢索。在物體外觀固定、訓練資料稀少的受控工業檢測中，以正規化相關做樣板比對仍然是對的工具。理解傳統方法，才知道現代方法到底在最佳化什麼，以及什麼時候簡單的方法就夠了。

重點整理

- 分類透過決策函數 $d_j(\mathbf{x})$ 替圖樣指定標籤；兩個決策函數相等之處就是邊界。
- 最小距離分類以類別平均為原型，邊界是垂直平分線。正規化相關與 SIFT 比對則是針對影像與關鍵點的原型比對。
- 貝氏分類器使平均損失最小。對高斯類別它是二次的；共變異數相同時是線性的；單位共變異數且事前機率相等時就化簡為最小距離。
- 感知器靠修正錯誤學出分隔超平面。使用平滑激活函數的多層網路能學出非線性邊界，反向傳播用一次反向傳遞算出所有梯度。
- CNN 是學出來的特徵擷取器（卷積、非線性、池化）加上學出來的線性分類器。權重共享讓它有效率，也能容忍位移。
- 過擬合是最主要的實務風險。使用驗證資料、資料擴增、權重衰減與 dropout，並且不要碰測試集。
- 現代系統常把大型預訓練特徵擷取器（CLIP、DINOv2）與非常簡單的分類器搭配。基準準確率反映了資料集偏差，所以要用獨立蒐集的資料測試。

練習

1. **手算平分線。**兩個類別平均為 $\mathbf{m}_1 = (1, 2)^\top$ 與 $\mathbf{m}_2 = (5, 4)^\top$。把最小距離邊界 $d_1(\mathbf{x}) - d_2(\mathbf{x}) = 0$ 寫成 x_1, x_2 的方程式，並驗證兩個平均的中點落在邊界上。

   提示

   $\mathbf{m}_1 - \mathbf{m}_2 = (-4, -2)^\top$，且 $\frac{1}{2}(\mathbf{m}_1 - \mathbf{m}_2)^\top(\mathbf{m}_1 + \mathbf{m}_2) = \frac{1}{2}(-4 \cdot 6 - 2 \cdot 6) = -18$。邊界為 $-4x_1 - 2x_2 + 18 = 0$，即 $2x_1 + x_2 = 9$。中點 $(3, 3)$ 代入得 $6 + 3 = 9$。✓

2. **事前機率的影響**。在一維問題中，類別 ω_1 是平均 0 的高斯分布，類別 ω_2 是平均 4 的高斯分布，變異數都是 1。求 $P(\omega_1) = P(\omega_2)$ 時與 $P(\omega_1) = 0.9$ 時的貝氏門檻值。門檻往哪個方向移動？為什麼這樣合理？

   提示

   令 $\ln P_1 - \tfrac{1}{2}x^2 = \ln P_2 - \tfrac{1}{2}(x-4)^2$，得 $x = 2 + \tfrac{1}{4}\ln(P_1/P_2)$。事前機率相等時 $x = 2$；$P_1 = 0.9$ 時 $x = 2 + \tfrac{1}{4}\ln 9 \approx 2.55$。門檻往較少見的類別移動，於是更多模稜兩可的圖樣被判給常見的類別。

3. **相關的不變性**。證明若把影像區塊的每個像素換成 $af + b$（$a > 0$），正規化相關係數 γ 不變。若 $a < 0$ 會怎樣？接著用 `cv2.matchTemplate` 在 `skimage.data.camera()` 上驗證。

   提示

   減去平均就消去了 b。a 在分子出現一次，在分母則以 $\sqrt{a^2} = |a|$ 出現，所以 $a > 0$ 時抵消。$a < 0$（相片負片）時 γ 變號：完美吻合變成 -1。

4. **停不下來的感知器**。產生兩團互相重疊的高斯點雲，用本章的感知器規則跑 100 個 epoch，畫出每個 epoch 的錯誤數。接著改用梯度下降最小化平方誤差 $\tfrac{1}{2}\sum(\mathbf{w}^\top \mathbf{y} - t)^2$，比較兩條學習曲線。

   提示

   感知器的錯誤數會一直起伏，因為即使沒有任何直線能完美分開，每一次錯誤仍會移動直線。最小平方規則則會收斂到一條固定的折衷直線，因為它的損失平滑且為凸函數。

5. **計算參數量**。某網路的輸入是 64×64 灰階影像。比較 (a) 有 1,024 個隱藏單元的全連接層與 (b) 有 32 個 3×3 核的卷積層的權重數量（含偏差）。不補邊時，(b) 會產生多少個輸出？

   提示

   (a) $4096 \times 1024 + 1024 = 4{,}195{,}328$。(b) $32 \times 9 + 32 = 320$ 個參數，產生 32 張 62×62 的特徵圖，也就是 123,008 個輸出。參數少、輸出多，這就是權重共享。

6. **有害的擴增**。使用 `scripts/figures/dip_ch13.py` 中的數字產生器，在擴增中加入隨機 180° 旋轉。重新訓練後，檢查哪兩個數字之間的混淆增加最多，並解釋原因。

   提示

   6 轉 180° 看起來像 9（0、1、8 則看起來和自己一樣）。這個擴增不再保留標籤，等於教網路對同一個形狀給出互相矛盾的標籤。可以預期 6 與 9 的混淆會大幅上升。

參考文獻

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 13. publisher page
2. A. K. Jain, R. P. W. Duin, and J. Mao, "Statistical Pattern Recognition: A Review," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 1, pp. 4–37, 2000. doi:10.1109/34.824819
3. D. G. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," *International Journal of Computer Vision*, vol. 60, 2004. doi:10.1023/B:VISI.0000029664.99615.94
4. F. Rosenblatt, "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain," *Psychological Review*, vol. 65, 1958. doi:10.1037/h0042519
5. D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning Representations by Back-Propagating Errors," *Nature*, vol. 323, 1986. doi:10.1038/323533a0
6. Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, 1998. doi:10.1109/5.726791
7. A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *Advances in Neural Information Processing Systems 25 (NIPS)*, 2012. paper
8. K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," arXiv:1409.1556, 2014. arXiv
9. K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," arXiv:1512.03385, 2015. arXiv
10. O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, et al., "ImageNet Large Scale Visual Recognition Challenge," arXiv:1409.0575, 2014. arXiv
11. W. Rawat and Z. Wang, "Deep Convolutional Neural Networks for Image Classification: A Comprehensive Review," *Neural Computation*, vol. 29, no. 9, 2017. doi:10.1162/neco_a_00990
12. A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," *ICLR*, 2021. arXiv
13. S. Khan, M. Naseer, M. Hayat, S. W. Zamir, F. S. Khan, and M. Shah, "Transformers in Vision: A Survey," *ACM Computing Surveys*, 2022. arXiv
14. Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," *CVPR*, 2022. arXiv
15. A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, et al., "Learning Transferable Visual Models From Natural Language Supervision," arXiv:2103.00020, 2021. arXiv
16. M. Oquab, T. Darcet, T. Moutakanni, et al., "DINOv2: Learning Robust Visual Features without Supervision," arXiv:2304.07193, 2023. arXiv
17. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014. JMLR
18. C. Shorten and T. M. Khoshgoftaar, "A Survey on Image Data Augmentation for Deep Learning," *Journal of Big Data*, vol. 6, 2019. doi:10.1186/s40537-019-0197-0
19. A. Torralba and A. A. Efros, "Unbiased Look at Dataset Bias," *CVPR*, 2011. paper page
20. Z. Liu and K. He, "A Decade's Battle on Dataset Bias: Are We There Yet?," *ICLR*, 2025. arXiv

篇 III

自然語言處理

NLP 01・自然語言處理導論

什麼是 NLP、語言為何對電腦困難、領域如何從規則走向統計再走向預訓練模型、主要任務、中文特有的問題，以及從任務定義到錯誤診斷的實用處理流程。

網站版本: https://blog.shyandram.dev/zh-tw/learn/nlp/nlp-01-introduction/

你將學到

- 說明語言為何對電腦困難，並為每個語言學層次舉出一個歧義例子。
- 透過標誌性的論文，追溯 NLP 的主要時代：規則、統計學習、神經網路與預訓練模型。
- 依輸出形式分類常見 NLP 任務，並找到涵蓋各任務的教學。
- 認識中文特有的問題：斷詞，以及繁體字與簡體字兩套字集。
- 透過輸入、輸出與所需語言證據定義 NLP 任務，並在正規化與斷詞過程中保留這些證據。
- 建立資料切分，找出錯誤真正起源的處理階段。

先看全貌

自然語言處理（natural language processing, NLP）研究能分析、解讀或產生人類語言的電腦系統。NLP 流程把特定語言問題轉成計算。考慮「Pump B: not ready.」：維護應用可能需要設備名稱與陳述的就緒狀態；訊息分流應用可能只需要類別；對話助理則可能需要追問還有什麼沒完成。即使輸入字串相同，它們也是不同任務。

第一步是決定應用需要什麼答案，以及哪些證據支持答案。接著把文字轉成模型輸入，同時保留這些證據。前處理已抹去的差異，再強的模型也無法恢復；看似合理的標籤，若描述錯誤的目標，也沒有用。

> **用最簡單的話說**
>
> 想像把信件分類放進不同托盤。先決定每個托盤代表什麼，再閱讀信件，而且不能擦掉重要的字。若把「包裹沒有送到」變成「包裹送到了」，還沒選分類工具，問題就已經出錯。

本教學以 Hagiwara 從應用出發的觀點，以及 McRoy 對語言和相關資料結構的說明為基礎 [1, 2]。我們先看語言為何困難、領域如何走到今天的方法、它解決哪些任務，以及中文文字有什麼不同。接著沿著一條實用路線前進：定義任務、保留文字、選擇計算單位、建立資料，再評估結果。NLP 02 說明這些決策背後的語言結構，本教學最後的路線圖則串起整個系列。

1. 語言為何對電腦困難

白話版。同一串字元可以有不同意思，而意思取決於結構、語境與說話者的目的。除非我們提供其餘資訊，否則電腦只看得到字元。

歧義出現在語言分析的每個層次。下表每一列都會在NLP 02 以例子與程式深入說明：

層次	例子	兩種解讀
詞邊界	學生會說話	「學生有說話能力」／「學生會在發言」
詞類	“the pilots can land” / “the metal can fell”	“can” 作助動詞／作名詞
詞義	“window”	牆上的開口／螢幕上的區域
附著	“The technician inspected the panel with the camera.”	相機是工具／相機裝在面板上
作用域	“Every technician uses a camera.”	可能是不同相機／一台共用相機
指稱	“Mei put a camera beside a lamp. It was overheating.”	“it” 是相機／是燈
意圖	“Can you lower the gain?”	詢問能力／請求動作

人們解決大多數這類歧義時幾乎不自覺，靠的是前後文、情境與世界知識。程式則必須被提供這些語境，或從資料中學會近似它們。另外三種性質讓困難加劇。

- **生產性**。說話者隨時用熟悉的零件組出新詞與新句子，因此沒有一份有限的輸入清單能涵蓋整個語言。
- **偏斜的頻率**。少數詞極為常見，大多數詞很罕見。NLP 02 用 Zipf 定律量測這件事：在一個語料庫中，三分之一的詞型別只出現一次。罕見詞正是模型證據最少的地方。
- **變異**。方言、語域、拼寫慣例、混合語言訊息與書寫系統都不相同。以某一變體訓練的系統，可能在另一變體上悄悄失敗。

這些都不表示 NLP 做不到，而是表示每個系統都在選擇保留哪些區別；好的工程會把這些選擇明確寫出來。

2. 簡史：規則、統計、神經網路與預訓練

白話版。早期系統依照人工撰寫的規則運作。後來的系統從大量文字集合學習機率。接著神經網路學會自己的特徵；今天，多數系統從一個以未標註文字預訓練的大型模型出發，再調整到任務上。

圖 *1* — 本教學引用的里程碑，分成四個大致的時代。事件等距排列，並未依年份比例；實際上各種方法彼此重疊。

規則與人工建立的知識。 Weizenbaum 的 ELIZA（1966）回應輸入的英文時，會比對關鍵詞，並套用腳本中的分解與重組規則；它最知名的腳本模仿心理治療師的對話風格 [6]。這個時代的系統以人工編寫文法、詞庫與模式。在設計範圍內它們可以很精確，但每遇到新的說法或領域，都需要新規則。

從語料庫進行統計學習。在接下來的數十年間，研究者越來越常從資料估計模型，而不是寫下每一條規則。Brown 等人（1993）描述一系列統計翻譯模型，其參數由互為翻譯的句子對估計而得 [7]。同一年，Penn Treebank 釋出一個標註詞性與句法結構的大型英文語料庫 [8]，讓標記器與剖析器有共同的訓練資料與共同的測試。關鍵改變是證據來自語料庫中的計數，而在保留資料上評估也成為標準做法。

神經網路。 Bengio 等人（2003）訓練一個神經語言模型，同時學習每個詞的分散式表示與詞序列的機率 [9]。Collobert 等人（2011）用同一個神經架構處理詞性標註、詞組切分（chunking）、命名實體辨識與語意角色標註，主要從未標註文字學習內部表示，而不是依賴人工設計的特徵 [10]。Mikolov 等人（2013）讓詞向量能在大型語料庫上低成本地訓練 [11]。這時的模型仍傾向一次只為一個任務建立。

預訓練模型與大型語言模型。 Transformer（2017）以注意力取代循環結構 [12]。ELMo（2018）從預訓練的雙向語言模型提供依語境而變的詞表示 [13]；BERT（2019）在未標註文字上預訓練深層雙向 Transformer，再為每個任務加上小型輸出層做微調 [14]。GPT-3（2020）顯示大型自迴歸模型只要在提示中看到幾個例子，不需梯度更新，就能執行許多任務 [15]；InstructGPT（2022）則以人類回饋微調這類模型，使它遵循指令 [16]。Zhao 等人的綜述以預訓練、適應調整、使用方式與能力評估四個面向，組織大型語言模型的文獻 [17]。

各時代彼此重疊：規則式元件、計數方法與神經模型都仍在使用。能延續下去的是第 4–9 節的流程。不論用哪種模型，總要有人定義任務、保留證據、選擇單位、建立資料並進行評估。

3. 主要任務與應用

白話版。大多數 NLP 任務可以依輸出分組：整段文字一個標籤、每個詞一個標籤、一個結構、從集合中找出的段落，或新的文字。

輸出形式	任務例子	應用	對應教學
每段文字或文字對一個標籤	主題分類、情感分析、垃圾訊息偵測、自然語言推論	分流、內容審核、評論分析	NLP 03、NLP 11
每個 token 或範圍一個標籤	詞性標註、命名實體辨識、資訊擷取	把報告轉成紀錄	NLP 06
語言結構	剖析、語意角色、共指	分析誰對誰做了什麼	NLP 12
從集合中找證據	搜尋、擷取式問答	文件搜尋、客服	NLP 11、NLP 09
依輸入產生新文字	翻譯、摘要、對話	翻譯工具、助理、聊天機器人	NLP 08、NLP 09
可重用的表示	詞與句子嵌入、預訓練編碼器	相似度、分群、遷移到新任務	NLP 04、NLP 10

同一個應用常結合好幾列。回答維修問題的助理，可能要分類問題、檢索段落、擷取設備名稱，再產生回覆。每一步都有自己的輸出與錯誤，所以本教學其餘部分把任務定義當成第一個設計決策。

4. 選模型前先規定任務

白話版。「理解這則訊息」太籠統，無法直接訓練或評估。先指定哪些意思必須變成輸出。

對原創訊息「Pump B: not ready.」，三種可能輸出是：

任務	輸出	重要證據
主題分類	`maintenance`	訊息談的是設備狀態
實體擷取	`Pump B`，附上原文範圍	設備名稱的確切字元
狀態擷取	設備 `Pump B`；就緒狀態 `false`	名稱與否定就緒的陳述

第一個答案不能證明第三個答案。分類器可能只靠「pump」辨認主題，完全忽略「not」。狀態擷取器則必須保留否定。回傳設備名稱的系統還需要邊界規則：是否包含 `B`？是否排除冒號？把這些決定寫進標註說明。

接著比較「Pump B was not ready yesterday」與「Pump B is ready now」。設備相同，時間與肯定／否定卻不同。即時狀態應用需要處理多個帶日期陳述的規則；歷史擷取器可能保留兩者。正確答案取決於任務定義與可用脈絡，而不是字串唯一通用的解讀。

對「It still isn't ready」，訊息沒有寫設備名稱。若應用提供前文對話，指涉解析就能使用它；若只有這句孤立文字，辨認出 `Pump B` 就是在加入從未提供的資訊。這是輸入設計問題，NLP 12（共指）與NLP 08（對話狀態）會進一步討論。

預測與執行要分開。預測出重新安排的請求，並不能決定該修改哪一筆預約。先從能和文字核對的輸出開始，例如意圖標籤、設備名稱範圍，或明確陳述的事件。後續教學再加入解析缺漏細節所需的情境推理。

5. 保留原始文字與證據

白話版。模型輸入與畫面上的證據文字，可以是同一則訊息的不同表示。保留能連起兩者的方式。

處理流程可寫成：

$$r \xrightarrow{g} x \xrightarrow{f_\theta} \hat{y}.$$

其中 r 是原始輸入紀錄；g 是前處理與數值編碼函數；x 是模型輸入；f_θ 是參數為 θ 的預測器；$\hat{y}$ 是輸出。紀錄可包含文字與允許使用的脈絡。前處理涵蓋正規化、token 邊界與詞彙 ID 等決策。

若兩筆不同紀錄變成同一個 x，確定性預測器就必須對兩者給出相同結果：

$$g(r_1) = g(r_2) \quad \Longrightarrow \quad f_\theta(g(r_1)) = f_\theta(g(r_2)).$$

其中 r_1, r_2 是兩筆原始紀錄。這個簡單等式解釋了為什麼前處理也是模型的一部分。若移除「not」使肯定與否定陳述完全相同，之後再改神經架構也無法區分。把字母轉成小寫而合併 `US` 與 `us`，或移除標點而讓小數變成另一個字串，也有同樣問題。

有些等價關係很有用。Unicode NFC 正規化把標準等價的字元序列映成共同形式 [3]。這和自行決定移除重音、標點或大小寫不同：前者處理指定的字元等價關係，後者可能抹掉任務需要的差異。應先決定本應用中哪些輸入應視為等價，而不是把所有轉換都當成無害清理。

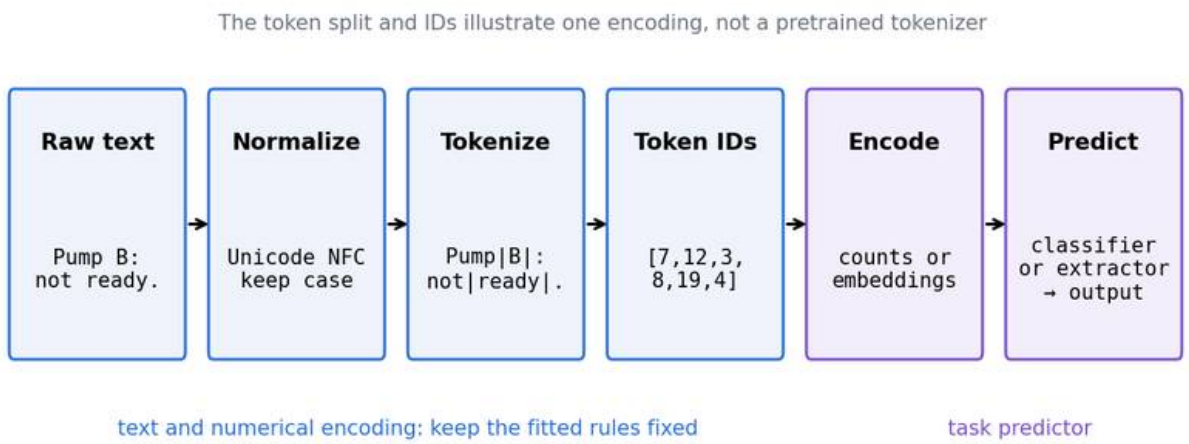


圖 2 — 示意文字處理流程。保留與原始文字的連結，訓練與評估時使用同一套已擬合編碼。

正規化與文字範圍的算例

`Café` 可以使用預組合的重音字母，也可以用基本字母 `e` 接上一個組合重音。兩者看起來相似，卻占用不同數量的 Unicode 碼位（code point）[3]。擷取器若回傳字元位置，正規化就可能讓後面的文字範圍位移。

文字範圍（span）$[a, b)$ 表示在**指定字串**中，從位移 a 開始、到 b 之前結束的字元。兩個位移都是非負整數，且 $a \le b$。不知道它們對應哪種文字表示與索引單位，位置本身就沒有意義。

```
import re
import unicodedata

raw = 'Café Pump B: not ready.'
normalized = unicodedata.normalize('NFC', raw)
needle = 'Pump B'
raw_start = raw.index(needle)
normal_start = normalized.index(needle)
raw_span = (raw_start, raw_start + len(needle))
normal_span = (normal_start, normal_start + len(needle))
tokens = [(m.group(), m.start(), m.end())
          for m in re.finditer(r'\w+|[^\w\s]', normalized)]
assert normalized != raw
assert unicodedata.normalize('NFC', normalized) == normalized
assert raw[raw_span[0]:raw_span[1]] == needle
assert raw[normal_span[0]:normal_span[1]] != needle
assert all(normalized[start:end] == token for token, start, end in tokens)
print('code points:', len(raw), '->', len(normalized))
print('raw / normalized evidence spans:', raw_span, normal_span)
print('normalized tokens:', [token for token, _, _ in tokens])
```

程式輸出碼位數 `24 -> 23`。原始設備名稱範圍為 `(6, 12)`，正規化後為 `(5, 11)`。兩者在各自字串中都選出 `Pump B`；把第二個範圍套到第一個字串，卻會選錯字元。最後一個斷言檢查正規化 token 與其來源位移是否對應。

做擷取時，可以維護轉換位置到原文的對齊，也可以直接在原始字串保留標註。本例展示錯誤如何發生，沒有實作通用對齊演算法。NLP 06 會說明詞、token 與字元範圍如何成為結構化輸出。

6. 決定計算使用哪些單位

白話版。詞、字元與模型 token 回答不同問題。選計算單位時，不要把它誤當成語言的所有單位。

在「a cat and a dog」中，空白切分得到五個 **token 實例**（occurrences）與四種不同的**型別**（types）：`a`、`cat`、`and`、`dog`。詞彙表替每個保留型別指定 ID。ID 是表格項目的名字，ID 7 並不比 ID 3 表達更大的意思。

「Pump B: not ready.」則帶來不同選擇。空白切分會把標點黏在詞上；辨識標點的切分器可能產生 `Pump`、`B`、`:`、`not`、`ready`、`.`；子詞斷詞器還可能把詞切得更細。它們都可能是有效的計算編碼，但 token 位置不同。詞標籤必須對齊模型實際使用的編碼。

子詞斷詞器從資料學習要保留哪些片段。NLP 02 會逐步訓練一個小型位元組對編碼（byte-pair encoding, BPE）斷詞器，並與 WordPiece、unigram 模型及 SentencePiece 比較。它的核心結論是：學到的片段反映頻率而不是構詞，而斷詞器是一整套已擬合的程序，不只是一份詞彙清單。

詞彙覆蓋率與未知輸入

詞彙表只包含保留的項目。若把沒見過的詞映到未知標記，所有這類詞都共享同一個 ID。任務若主要依賴熟悉的周圍詞，這可能可接受；但未見設備名稱與未見帳戶名稱之間的差異也會消失。

定義一個示意的未知 token 比例：

$$u = \frac{\sum_{i=1}^{N} \mathbf{1}[t_i \notin V]}{N}.$$

其中 u 是未知 token 比例；t_i 是指定斷詞器下第 i 個 token 實例；V 是擬合後詞彙表；$N > 0$ 是評估的出現總數；$\mathbf{1}[\cdot]$ 在條件成立時為一。重複出現的未知詞會重複計數。這量測缺少詞彙項目的比例，不是失去多少意思，也不是任務準確度。

7. 中文文字：斷詞與兩套字集

白話版。中文書寫的詞與詞之間沒有空格，所以決定詞的起訖本身就是一種分析。中文也以繁體字與簡體字兩套標準字集書寫，在其中一套上擬合的模型，看到另一套時會遇到許多陌生字元。

斷詞

中文通常沒有詞間空白。「學生會說話」可以切成 **學生 / 會 / 說話**（學生有說話能力），也可以切成 **學生會 / 說話**（學生會在發言）；脈絡有助選出原本的意思。模型斷詞器也可能只使用字元或子詞片段，其切分未必是完整詞法或語法分析。NLP 02 用一個小程式列舉兩種解讀，並區分這些語言結構。

斷詞也是一種慣例。第二屆國際中文斷詞評測使用四個語料庫：兩個為繁體字（臺灣的中央研究院與香港城市大學），兩個為簡體字（北京大學與微軟研究院），每個都依提供者自己的標準切分 [18]。當資料集有詞層級標籤時，它依循的標準就是任務定義的一部分；以字元為單位的模型在評估前，必須把輸出對齊到這些詞。

繁體字與簡體字

臺灣與香港的文字主要以繁體字書寫，中國大陸的文字則以簡體字書寫；上述評測語料庫也依循這個分界 [18]。許多字在兩套字集中相同，但也有許多字是不同的 Unicode 碼位：繁體的「機」是 U+6A5F，其簡體對應字則是 U+673A。Unicode 正規化不會合併它們，因為它們是不同的字元，而不是同一字元的不同編碼方式。

轉換不是一對一的字元替換。Unihan 資料庫為每個字記錄繁簡異體，並指出語境可能決定對應：從簡體轉成繁體時，U+810F 表示「內臟」時對應「臟」，表示「不乾淨」時對應「髒」[19]。不同地區的用詞也可能不同，所以即使字元轉換完美，也無法把一種變體的文字變成另一種變體的文字。

字集不一致會直接反映在詞彙覆蓋率上。下列程式在一句繁體句子上擬合字元詞彙表，再對同一句新句子的兩種寫法量測未知 token 比例 u。簡體版本以跳脫碼書寫，以便看出哪些字元不同。

```
vocab = set("這台相機已經校正好了")              # character vocabulary fitted on Traditional
 ↪  text
tests = {"Traditional": " 那台相機已經過熱了",
         # The same sentence in Simplified characters, written as escapes:
         "Simplified": " 那台相\u673a 已\u7ecf\u8fc7\u70ed 了"}
for name, text in tests.items():
    unknown = [f"U+{ord(ch):04X}" for ch in text if ch not in vocab]
    print(f"{name:11s} u = {len(unknown)}/{len(text)}  unknown: {unknown}")
```

繁體句子的 $u = 3/9$：「那」、「過」、「熱」不在這段極小的訓練文字中。簡體句子的 $u = 5/9$：「那」(兩套字集相同）以及「過」、「熱」的簡體形式仍然未知，另外再加上「機」與「經」的簡體對應字，因為詞彙表只有它們的繁體形式。真實詞彙表大得多，但效果在性質上相同：主要以一套字集訓練的模型，對另一套字集的證據較少。請決定要同時用兩套字集訓練、用考慮語境的工具轉換，還是分別評估各字集，並說明你的選擇。

8. 建立真正回答任務的樣本與標籤

白話版。語料庫是觀測集合；訓練樣本則依任務從觀測建立。建立方式決定模型學什麼。

分類樣本可以是完整訊息配上一個類別；擷取樣本可以是訊息配上字元範圍與標籤；語言建模則由文字本身提供預測目標，不需要類別標註。這些差異解釋了為什麼同一份語料能支援不同目標 [1, 2]。

假設維護訊息含有「the camera is fine, but the delivery was late」。產品情緒標籤、配送情緒標籤與整則訊息標籤未必一致。標註說明應包含目標。標註者若選不同目標，分歧首先反映任務定義問題，還不能直接解讀成模型品質問題。

定義資料切分的獨立單位。同一件客訴的兩種改寫，雖然是不同字串，卻可能描述同一事件。一則放訓練、一則放測試，可能讓測試獎勵辨認該事件，而不是轉移到新事件。相關訊息、對話或文件修訂若共享資訊而可能洩漏，就應整組切分。

須追蹤兩種擬合狀態：預測器從訓練標籤學權重，斷詞器或詞彙表可能從未標註訓練文字學習。若特徵流程仍用測試文字決定詞彙表，只隔離測試標籤還不夠。兩者都須依實驗規則固定。

用小型基準診斷表示方式

先從容易檢查行為的表示開始。訊息的計數表示保留哪些詞出現，卻忽略順序。因此「The dog bit the man」與「The man bit the dog」即使反轉誰對誰做了什麼，仍有相同計數。只用這些計數的確定性分類器，必須對兩者給出相同預測。

這是精確的表示限制。只要特徵映射仍相同，更多樣本也無法分開這組句子。局部片語特徵能分開部分語序差異；脈絡模型可使用更豐富的輸入互動。NLP 03 會推導並測試這些選擇。針對實際失敗增加結構，實驗才能辨認改變了什麼。

9. 評估輸出，診斷錯誤原因

白話版。評估應告訴你哪裡出錯、該改善哪裡。單一彙總分數通常找不出原因。

分類時檢查每個類別的錯誤，尤其罕見類別；準確率（accuracy）可能掩蓋某類從未被預測的問題。精確率（precision）問回傳項目有多少正確，召回率（recall）問必要項目找回多少。兩者都取決於如何定義項目 [5]。擷取找到正確詞卻回傳錯誤範圍，是邊界錯誤；狀態紀錄配對正確設備卻判錯肯定／否定，是意思錯誤。

保留幾組只改變一個重要語言差異的挑戰案例，例如「ready」與「not ready」、`Pump A` 與 `Pump B`、「yesterday」與「now」。固定任務，只改變該差異。普通訊息答對，還不能說明系統是否使用了這些案例所需的區別。

觀察	優先檢查	改善必須證明什麼
未見設備名稱被合併成同一表示	詞彙表與備援方式	名稱在輸入中仍可區分
詞正確，證據反白範圍錯誤	正規化與位移對齊	回傳位移對應畫面原文
否定使真實標籤反轉，預測卻不變	特徵映射與監督訊號	相關差異保留於編碼，並影響決策
驗證結果好，新客戶結果差	切分獨立性與領域差異	表現能轉移到目標群體
在一套中文字集上表現好，另一套很差	字元覆蓋率與轉換	兩套字集都被涵蓋，或明確回報差距

盡可能一次只改一個階段。前處理和模型一起改，即使分數提高，也無法知道誰造成改善。解讀權重或輸出前，先檢查轉換後輸入。這讓錯誤分析成為可測試的解釋，而不是一串模型名字。

10. NLP 系列路線圖

本系列依循第 2 節的時代順序：先談語言結構，再談神經網路基礎、序列模型、注意力與 Transformer，最後是預訓練模型與結構化任務。

教學	內容
NLP 01・自然語言處理導論	語言為何困難、歷史、任務、中文問題，以及從任務到評估的流程
NLP 02・語言學基礎	構詞、token 與子詞、詞性、句法、意義、語境、語料庫與 Zipf 定律
NLP 03・自然語言處理的類神經網路	計數特徵、TF–IDF、邏輯迴歸與神經網路文字分類器
NLP 04・詞彙語意與詞嵌入	分布式意義、計數式向量、word2vec
NLP 05・用於語言的循環神經網路	n-gram 語言模型、困惑度、循環語言模型
NLP 06・以 RNN 做序列標註	詞性標註、實體辨識與範圍評估
NLP 07・注意力機制與 Transformer	注意力、語境表示與位置
NLP 08・以 Transformer 建構序列到序列模型	編碼器–解碼器模型、解碼、機器翻譯與聊天機器人
NLP 09・以 Transformer 做語言模型	因果語言模型、文字生成與檢索增強生成
NLP 10・預訓練模型：BERT	遮罩語言預訓練與 BERT 編碼器
NLP 11・以 BERT 做分類	情感分析、自然語言推論與擷取式問答輸出層
NLP 12・句法與語意任務	剖析、語意角色、共指與篇章

本站其他教學涵蓋一般機制：ML 02 介紹神經網路，ML 03a 介紹循環網路，ML 04 介紹注意力與 Transformer 區塊，IT 01 介紹熵與交叉熵。

現代觀點

Chang 等人的評估綜述整理語言模型應評估什麼、在哪裡評估與如何評估 [4]。對處理流程而言，這促使我們分別檢查編碼、任務輸出與應用行為。生成解釋再流暢，也不是設備名稱召回率的另一種量測；兩種輸出回答不同問題。

Zhao 等人的大型語言模型綜述描述了目前的預設做法：在廣泛文字上預訓練一個大型模型，再加以適應（例如指令調整或對齊人類偏好），透過提示使用它，並在許多任務上評估其能力 [17]。它的組織方式呼應第 2 節的歷史：每個時代都沿用前一時代的任務定義，改變的是從文字得到輸出的方式。

對現今系統，可沿用的原則是比較模型家族前先檢查介面。相關文字沒有被截斷，較大的輸入預算才有用；表示能支持任務所需差異（包括使用者同時以兩套中文字集書寫時），多語斷詞器才有用。預訓練表示可能減少所需標註資料，但應用目標仍需要明確監督與評估。後續教學會分析各自機制，而不是把這些當成自動得到的好處。

開放問題包含如何量測超越字元覆蓋的語言覆蓋、切分如何影響不同語言與語域，以及如何區分表示失敗、標籤不足與脈絡缺漏。一個有用的小研究，是在固定任務、獨立切分資料與幾個語言診斷案例下比較兩種編碼。回報各編碼保留或失去什麼，再據此解釋觀察到的錯誤。

重點整理

- 語言在每個層次都有歧義，從詞邊界到意圖；語境與世界知識能解決字串本身無法決定的部分。
- NLP 從人工規則走向語料統計，再走向神經網路與預訓練模型；任務定義與評估問題則原封不動地延續下來。
- 依輸出形式分組任務：文字標籤、token 標籤、結構、檢索證據、生成文字或表示。
- 中文多了兩個問題：依不同標準進行的斷詞，以及轉換並非一對一的兩套字集。
- 選模型前，先定義輸出與支持它的語言證據，並區分原始文字與轉換後模型輸入。
- 預測器無法恢復前處理函數已抹掉的差異；token 與詞彙 ID 是計算單位，不是完整語言分析。
- 只在允許的訓練文字上擬合學習式表示，切分時把相關觀測放在同組；改架構前，先診斷詞彙、邊界、意思與轉移錯誤。

練習

1. 對「Pump B was not ready yesterday, but it is ready now」，定義主題分類、歷史狀態擷取與目前狀態擷取的標籤。哪些標籤需要時間處理規則？

   提示

   設備名稱可以不變，狀態值與有效時間卻不同。先決定回傳一個狀態，還是兩個帶日期事件。

2. 把正規化範例中的分解重音換成預組合重音。說明哪些位移改變，哪些斷言仍有效。

提示

分別比較原文與正規化字串。位移是某個表示中的位置，不是所有外觀等價字串中的共同位置。

3. 讓一個前處理規則移除「not」。建構兩則所需狀態標籤不同、轉換輸入卻相同的訊息，證明確定性預測器為什麼不能分開它們。

提示

套用包含 $g(r_1)$ 與 $g(r_2)$ 的等式。矛盾在訓練前就已出現。

4. 在字集覆蓋程式中，改用繁體訓練句與其簡體版本的聯集擬合詞彙表，重新計算兩個測試句的 u。接著解釋為什麼只靠一張字元對照表，不一定能在查詢前把每個簡體測試字轉成繁體。

提示

聯集能消除字集造成的未知字，卻無法消除「那」、「過」、「熱」。關於轉換，請參考 Unihan 的例子：同一個簡體碼位依意思對應到兩個不同的繁體字。

5. 把同一事件的三則改寫訊息分組，建立訓練／測試資料集。設計切分與兩種診斷，區分事件記憶與對新事件的轉移。

提示

按事件整組切分，再分別檢查未見名稱與未見措辭。標籤規則保持固定。

參考文獻

1. Masato Hagiwara, *Real-World Natural Language Processing: Practical Applications with Deep Learning*, Chapters 1–3 and 10–11, Manning, 2021. 出版社
2. Susan McRoy, *Principles of Natural Language Processing*, open textbook, 2021. 書籍
3. Unicode Consortium, “Unicode Normalization Forms,” Unicode Standard Annex #15, revision 58, 2026. 規格
4. Yupeng Chang et al., “A Survey on Evaluation of Large Language Models,” *ACM Transactions on Intelligent Systems and Technology*, 2024. 綜述
5. Scikit-learn developers, “Metrics and Scoring: Quantifying the Quality of Predictions,” documentation, current online edition. 文件
6. Joseph Weizenbaum, “ELIZA—A Computer Program for the Study of Natural Language Communication Between Man and Machine,” *Communications of the ACM*, 9(1), pp. 36–45, 1966. DOI
7. Peter F. Brown, Stephen A. Della Pietra, Vincent J. Della Pietra and Robert L. Mercer, “The Mathematics of Statistical Machine Translation: Parameter Estimation,” *Computational Linguistics*, 19(2), 1993. 論文

8. Mitchell P. Marcus, Beatrice Santorini and Mary Ann Marcinkiewicz, “Building a Large Annotated Corpus of English: The Penn Treebank,” *Computational Linguistics*, 19(2), 1993. 論文
9. Yoshua Bengio, Réjean Ducharme, Pascal Vincent and Christian Jauvin, “A Neural Probabilistic Language Model,” *Journal of Machine Learning Research*, 3, pp. 1137–1155, 2003. 論文
10. Ronan Collobert, Jason Weston, Léon Bottou, Michael Karlen, Koray Kavukcuoglu and Pavel Kuksa, “Natural Language Processing (Almost) from Scratch,” *Journal of Machine Learning Research*, 12, pp. 2493–2537, 2011. 論文
11. Tomas Mikolov, Kai Chen, Greg Corrado and Jeffrey Dean, “Efficient Estimation of Word Representations in Vector Space,” arXiv:1301.3781, 2013. 論文
12. Ashish Vaswani et al., “Attention Is All You Need,” *NeurIPS*, 2017. 論文
13. Matthew E. Peters et al., “Deep Contextualized Word Representations,” *NAACL-HLT*, 2018. 論文
14. Jacob Devlin, Ming-Wei Chang, Kenton Lee and Kristina Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *NAACL-HLT*, 2019. 論文
15. Tom B. Brown et al., “Language Models are Few-Shot Learners,” *NeurIPS*, 2020. 論文
16. Long Ouyang et al., “Training Language Models to Follow Instructions with Human Feedback,” *NeurIPS*, 2022. 論文
17. Wayne Xin Zhao et al., “A Survey of Large Language Models,” arXiv:2303.18223, 2023 (continuously updated). 綜述
18. Thomas Emerson, “The Second International Chinese Word Segmentation Bakeoff,” *Proceedings of the Fourth SIGHAN Workshop on Chinese Language Processing*, 2005. 論文
19. Unicode Consortium, “Unicode Han Database (Unihan),” Unicode Standard Annex #38, revision 41, 2026. 規格

NLP 02・語言學基礎

每個 NLP 系統都要面對的語言結構：單位與構詞、token 與子詞、詞性、詞組與依存結構、意義、語境與篇章，以及語料庫、標註與 Zipf 定律。

網站版本: https://blog.shyandram.dev/zh-tw/learn/nlp/nlp-02-linguistic-essentials/

你將學到

- 區分語言及其口語、手語、書寫形式，並區分語言學單位與計算用的 token。
- 分析詞的內部結構，比較詞幹擷取（stemming）與詞形還原（lemmatization），並列舉中文斷詞的歧義。
- 訓練一個小型 BPE 斷詞器，並說明它與 WordPiece、unigram 模型及 SentencePiece 的關係。
- 使用 Universal Dependencies 詞性標記，並解釋詞組結構、文法關係與一致關係為何包含詞頻無法保留的資訊。
- 推導詞彙意義、組合意義、否定與量詞作用域，並區分斷言、預設、會話含意與篇章關係。
- 批判地閱讀語料庫與標註：標註者之間的一致度、Zipf 定律與罕見詞的長尾。

先看全貌

語言讓人描述情境、提出問題、協調行動，也改變他人的認知。看得到的句子只是這些活動的一部分。句子的內部結構限制可行的解讀；說話者也依賴共同看見的物件、先前對話，以及對發話理由的預期。

> **用最簡單的話說**
>
> 想像有人遞給你積木，說：「再蓋一次那一個。」知道積木的名稱，仍然不能告訴你「那一個」是哪棟建築、上次發生了什麼，以及是否要使用相同顏色。語言也是如此：詞提供零件，文法把它們連接起來，情境則補上說話者沒有說出的資訊。

考慮 “The technician inspected the panel with the camera.” 技師是使用相機檢查面板，還是檢查一個裝有相機的面板？知道每個詞的字典義，仍然無法決定答案。再考慮 “Can you inspect it tomorrow?” 現在，受檢查的物件、參照日期，以及這句話是否在提出請求，都依賴語境。有用的分析會先辨認這些選項，再選擇表示方式或標籤。

本教學替 Hagiwara 的 *Real-World Natural Language Processing* 所採取的實作路線 [1]，以及 McRoy 的 *Principles of Natural Language Processing* 所採取的符號與計算路線 [2]，建立語言基礎。開放教科書 *Essentials of Linguistics* 提供補充的語言學背景 [3]。本教學範例均為原創。NLP 01 介紹了從任務到評估的處理流程；本教學說明這個流程必須保留的單位與結構。後續教學再把它們變成模型：NLP 06 標註詞性與實體，NLP 12 剖析句子、標註語意角色並解析共指。這些層次彼此影響，不表示軟體必須依序使用互相獨立的模組。

1. 語言、模態與單位

白話版。口語、手語與書寫是表達語言的不同方式，各自記錄不同資訊。電腦的「token」則是另一種為了處理而選定的單位。

語言是一套有結構的表達與解讀系統。口語與書寫是表達語言的方式，兩者不是可以任意互換的容器。手語使用有系統的視覺與手部表達，觸覺手語則提供另一種模態（modality）[4]。文字逐字稿保留互動中的部分資訊，卻省略時間節奏、視線、手勢，以及大部分實際環境。

語音與書寫符號回答不同問題

語音學（phonetics）研究語音的物理產生與感知；**音韻學（phonology）**研究語音在某個語言中的規律。**音素（phoneme）**是特定語言系統中具有區辨作用的語音單位；不同的物理實現可能屬於同一音素 [5]。**字素（grapheme）**則是書寫系統的單位。兩者未必一對一對應。

英文 “ship” 有四個字母，但開頭的 “sh” 表示一個子音。反過來，同一字母在不同詞裡也可能有不同作用。因此，計算字母數量無法直接回答語音結構的問題。即使使用者想表達相同的詞，語音應用與文字應用仍可能需要不同的標註。

聲調（tone）與語調（intonation）也不同 [6]。華語 mā、má、mǎ、mà 分別區分常見的書寫詞「媽、麻、馬、罵」。聲調形態參與詞彙上的區辨；整段發話的音高輪廓也可能傳達疑問或強調。在英文裡，“Really.” 與 “Really?” 在書寫上暗示不同解讀，但標點只能粗略模擬說話者如何表達不相信、好奇或確認。書寫的問號無法還原原本的聲音。

字元、詞與 token

詞形（word form）是實際出現的形式；**詞位（lexeme）**把相關詞形歸為同一詞彙項目。在簡化分析中，“repair”、“repairs” 與 “repaired” 都是動詞詞位 REPAIR 的詞形。**詞元（lemma）**是用來索引該項目的代表形式。因此，詞形還原（lemmatization）是一種標註決定，不表示不同詞形能在句子裡互換 [2, 3]。

在 “The technician repairs cameras” 與 “The technician repaired cameras” 中，這些區別很重要。把兩個動詞都歸入 REPAIR，有助於搜尋相關活動；但若丟掉實際詞形，就會失去現在與過去的對比。應用可以同時保留原始文字區段與詞元，不必認定某一表示一定要取代另一表示。

計算用的 **token** 是處理程序選定的輸入單位。它可能是一個詞、詞的一部分、標點，或一段字元序列。它的邊界不一定與音素、詞素或詞位一致。同樣地，中文字元也可能構成多字詞：「相機」這段書寫序列指稱一種物件。字元式系統可能接收兩個單位，詞導向分析則可能使用一個單位。兩種計數都不能單獨定義該片語包含多少個意義。

2. 構詞：詞的內部結構

白話版。詞有組成部分。有些部分帶有詞彙意義，有些帶有時態或單複數等文法資訊，而部分之間的分組方式也可能改變意義。

詞的結構可以同時傳達詞彙意義與文法資訊。**構詞學（morphology）**研究這種結構；**詞素（morpheme）**是形式與意義或文法功能之間最小的、反覆出現的配對 [7]。「最小」不表示「只有一個字母」，有意義的成分也不一定能獨立出現。複數詞尾、時態標記與派生前綴能提供有用證據，但不同語言不一定用相同片段或詞邊界表達這些性質。

屈折、派生與生產性的組合

屈折（inflection）對比改變詞的文法實現，例如 repair/repaired。**派生（derivation）**過程可以形成新的詞彙項目，例如 repair/repairable。**複合詞（compound）**結合詞基，例如 camera mount [3, 7]。這些是分析上的區別；只辨認詞尾的拼寫，不足以決定它的功能。

比較 “She repairs the camera” 與 “The repairs took a day” 裡的 “repairs”。前者是限定動詞的詞形，後者是複數名詞。若規則刪掉每個詞尾的 “s”，就會失去詞類，以及該詞尾出現的原因。這條規則也無法正確處理 mouse/mice 之類的不規則配對。分析的任務是解釋反覆出現的形式與功能規律，並明確表示例外。

生產性（productivity）表示一種模式能支持新組合，而不只是描述記住的清單。若工作室引入新的 “sensor cradle”，聽者可以根據熟悉的成分與情境解讀這個複合詞。然而，組合不會決定所有細節：它可能在組裝、運送或操作期間固定感測器。構詞可以提供概括的關係，語境則補上這次使用的確切實例。

中文也有詞的內部結構與生產性的組合。在「三台相機」中，「三」表達數量，「台」是與被計數物件搭配的量詞，「相機」命名物件。英文 “three cameras” 在這個例子裡把複數標記放在名詞上。這個對比指出不同的文法選擇，不表示中文詞缺少結構，也不表示英文名詞總是有明確的數量。

詞幹擷取與詞形還原

白話版。詞幹擷取器依規則切掉詞尾；詞形還原器則根據詞的文法類別查出字典形式。前者便宜而粗糙；後者需要更多知識，犯的明顯錯誤也較少。

詞幹擷取器（stemmer）透過改寫詞尾，把不同詞形映到共同字串，也就是**詞幹（stem）**。Porter 演算法是經典的英文例子：它依序套用數組詞尾規則，每條規則都附有對剩餘詞幹的條件，設計目的是在資訊檢索中把相關詞歸在一起 [16]。詞幹不必是真正的詞，只需要一致。**詞形還原器（lemmatizer）**回傳詞元，這需要詞庫，通常也需要詞性：作名詞時 “saw” 本身就是詞元，作動詞時卻是 SEE 的詞形。

下面的玩具版切除器只用四條有順序的規則，加上一個最短詞幹長度的條件；精神上接近 Porter 演算法，但簡單得多。查詢表則代替字典。

```
# A toy suffix stripper (a few Porter-style rules) versus a lookup lemmatizer.
RULES = [("ies", "y"), ("ing", ""), ("ed", ""), ("s", "")]   # tried in order; first match
 ↪  wins

def stem(word):
    for suffix, replacement in RULES:
        if word.endswith(suffix) and len(word) - len(suffix) >= 3:
            return word[: -len(suffix)] + replacement
    return word
```

```
# A lemmatizer needs the part of speech; this tiny table stands in for a dictionary.
LEMMAS = {("repairs", "VERB"): "repair", ("repairs", "NOUN"): "repair",
          ("repaired", "VERB"): "repair", ("mice", "NOUN"): "mouse",
          ("was", "AUX"): "be", ("batteries", "NOUN"): "battery",
          ("news", "NOUN"): "news", ("this", "DET"): "this"}

def lemmatize(word, pos):
    return LEMMAS.get((word, pos), word)

for word, pos in [("repairs", "VERB"), ("repaired", "VERB"), ("batteries", "NOUN"),
                  ("mice", "NOUN"), ("was", "AUX"), ("news", "NOUN"), ("this", "DET")]:
    print(f"{word:10s} stem={stem(word):9s} lemma={lemmatize(word, pos)}")
```

切除器能處理規則形式（`repairs`、`repaired`、`batteries` 都得到合理詞幹），卻讓 `mice` 與 `was` 原封不動，還把 `news` 變成 `new`、`this` 變成 `thi`。前兩個失敗來自不規則構詞；後兩個是**過度擷取（over-stemming）**：只是看起來像詞尾的字母被刪掉了。詞形還原器七個都對，但只因為表中已經有它們；沒見過的詞會原樣通過。兩種工具都不決定文法類別：`VERB` 或 `NOUN` 是呼叫者提供的，而那正是NLP 06 中詞性標註器的工作。

該用哪一種取決於任務。對搜尋而言，合併 `repairs` 與 `repaired` 可以提高召回率，即使有少數無關詞撞在一起。對狀態擷取器而言，詞幹擷取器丟掉的時態可能正是答案。現代神經網路流程很少對輸入做詞幹擷取；它們保留原始形式，讓子詞 token 與脈絡承載構詞資訊，這也是下一節重要的原因之一。

相同零件可以用不同順序組合

假設有人使用 “an unsealable case” 這個描述，我們可以考慮兩種有用的結構分析：

- `un + (seal + able)`：無法密封的盒子。
- `(un + seal) + able`：可以解除密封的盒子。

第一種把 “un” 當作對形容詞的否定；第二種先把它當作逆轉密封動作的成分，再由 “able” 形成形容詞。兩種解讀對實際操作有相反含意。它們是否成立、是否自然，會受語境影響；因此本例示範可能的分析，不主張每位說話者都同樣常用兩種解讀。

兩種分析包含的零件完全相同。改變的是中間的分組，以及每個操作與哪種對象結合。同樣的問題也會出現在句子的附著與邏輯作用域：如果任務需要這個區別，表示方式就必須保留結構。

中文斷詞也是解讀問題

中文書寫不以空格標示詞邊界。原創字串「學生會說話」可以分析成「學生 / 會 / 說話」（學生有說話能力），或「學生會 / 說話」（學生會發言）。前者把「會」當作表達能力的成分，後者則把「學生會」當作複合名詞。因此，處理後插入的空格記錄了一種分析；沒有空格的原始字串並未直接提供這些邊界。

下列程式列舉一個明確的小型詞庫所允許的路徑。它不是通用的中文斷詞器，而是說明：選擇最長的匹配詞條，無法直接確定說話者的意圖。

```
from functools import lru_cache

text = " 學生會說話"
lexicon = (" 學生", " 學生會", " 會", " 說話")

@lru_cache(None)
def segment(start):
    if start == len(text):
        return ((),)
    paths = []
    for word in lexicon:
        if text.startswith(word, start):
            for rest in segment(start + len(word)):
                paths.append((word,) + rest)
    return tuple(paths)

for path in segment(0):
    print(" / ".join(path))
assert len(segment(0)) == 2
```

印出的兩條路徑就是上述解讀。每一步遞迴都消耗一個已知詞，終止條件只接受完整覆蓋字串的路徑。擴充詞庫可能增加選項，卻不會增加特定對話的證據。句法分析、前後文，或這段訊息與學生會會議相關的知識，可能讓其中一條路徑更合理。

連標準答案本身也是一種選擇。第二屆國際中文斷詞評測（Second International Chinese Word Segmentation Bakeoff）使用四個語料庫評估系統：兩個為繁體字（來自臺灣的中央研究院與香港城市大學），兩個為簡體字（來自北京大學與微軟研究院），而且每個提供者都依自己的標準切分資料 [27]。因此，同一種切法可能在某個語料庫「錯」、在另一個語料庫「對」。請說明你的標籤依循哪一種斷詞標準。

3. Token 與子詞

白話版。模型需要一份固定的單位清單。整詞會留下太多未知詞；單一字元會讓序列太長。子詞方法從資料學出折衷：常見字串成為單一 token，罕見字串則由較小片段拼出。

NLP 01 定義了未知 token 比例，並說明為何把每個沒見過的詞都映到同一標記會失去區別。必要字元都被涵蓋時，字元詞彙表能減少完整詞的未知問題，代價是序列更長。位元組編碼若有完整基礎位元組字母表，就能涵蓋任意輸入位元組，但單位未必對應可閱讀字元。子詞方法在詞彙表大小與序列長度間取捨；Mielke 等人的綜述整理了這些選擇 [22]。沒有一種切分對所有語言、任務與資源預算都最好。

學習式斷詞器通常擬合語料庫中的統計模式。這些模式有助重用常見片段，但常見片段未必是有意義的詞素（morpheme）；反過來，有意義的屈折變化也可能被切成數個片段。判斷模型能辨認什麼時，必須分清楚：斷詞提供編碼，後續計算仍須學習編碼片段如何組合。

逐步理解一種子詞方法：BPE

BPE 學習哪些相鄰片段值得合併，替常見組合建立符號，讓其他字串仍以較小片段的序列表示。Sennrich、Haddow 與 Birch 把位元組對編碼（byte-pair encoding, BPE）改用於罕見詞翻譯 [17]。下面的字元版教學實作在詞內運作：

1. 把每個訓練詞表示成字元，加上一個詞尾標記。
2. 計數相鄰符號對，依該詞的語料出現頻率加權。
3. 在這些詞表示中，把最常見符號對的每次出現都合併。
4. 儲存這次合併，重複固定次數。

合併會改變後續符號對計數。兩符號合成新符號後，新符號又能參與另一個符號對。因此演算法學得的是有順序的操作序列，而不是能任意套用順序的片段集合。

```
from collections import Counter

corpus = ("low low low low low lower lower newest newest newest newest newest "
          "newest widest widest widest").split()
# Start from characters; "_" marks the end of a word so merges respect boundaries.
words = Counter(tuple(w) + ("_",) for w in corpus)

def pair_counts(words):
    pairs = Counter()
    for symbols, freq in words.items():
        for a, b in zip(symbols, symbols[1:]):
            pairs[a, b] += freq
    return pairs

def merge(words, pair):
    out = Counter()
    for symbols, freq in words.items():
        s, i = [], 0
        while i < len(symbols):
            if i + 1 < len(symbols) and (symbols[i], symbols[i + 1]) == pair:
                s.append(symbols[i] + symbols[i + 1]); i += 2
            else:
                s.append(symbols[i]); i += 1
        out[tuple(s)] += freq
    return out

merges = []
for step in range(10):
    pairs = pair_counts(words)
    best = max(pairs, key=lambda p: (pairs[p], p))   # most frequent; ties broken by symbol
    ↪  order
    merges.append(best)
    words = merge(words, best)
print("merges:", [a + b for a, b in merges])

def tokenize(word):                    # apply the learned merges in order
    symbols = list(word) + ["_"]
    for pair in merges:
```

```
        symbols = list(merge(Counter({tuple(symbols): 1}), pair))[0]
    return symbols

for w in ["lowest", "newer", "wider"]:
    print(w, "->", tokenize(w))
```

輸出包含 `lowest -> ('low', 'est_')`、`newer -> ('ne', 'w', 'e', 'r', '_')` 與 `wider -> ('wi', 'd', 'e', 'r', '_')`。第一個詞不在訓練語料中，卻能用學得的片段重新組合字母。其他詞的編碼較長，反映哪些組合獲得合併。

合併順序為什麼重要

第一個合併為 `t_`，接著是 `st_` 與 `est_`。訓練詞中重複出現的結尾讓這些組合很常見。這在例子中像英文詞尾，但演算法沒有「找出詞尾」的指令，只依頻率做決策。同一拼法換一份語料，可能得到不同切分。

`low` 合併要等較小組合先形成後才會出現。編碼依儲存順序執行，才能重現擬合程序。只從最終符號中優先找最長片段，是另一種演算法，未必產生相同切分。平手規則也有影響：本程式以符號順序打破相同計數，讓執行可重現。

函數回傳片段，還不是數值 ID，也沒有證明能涵蓋任意未見字元。本例成功處理的新詞，都重用訓練出現過的字元；若一個詞含新字元，就需要指定備援方式與相容的詞彙查表。這分開了兩個問題：字串能否切分，以及每個回傳片段是否都有模型懂的 ID。

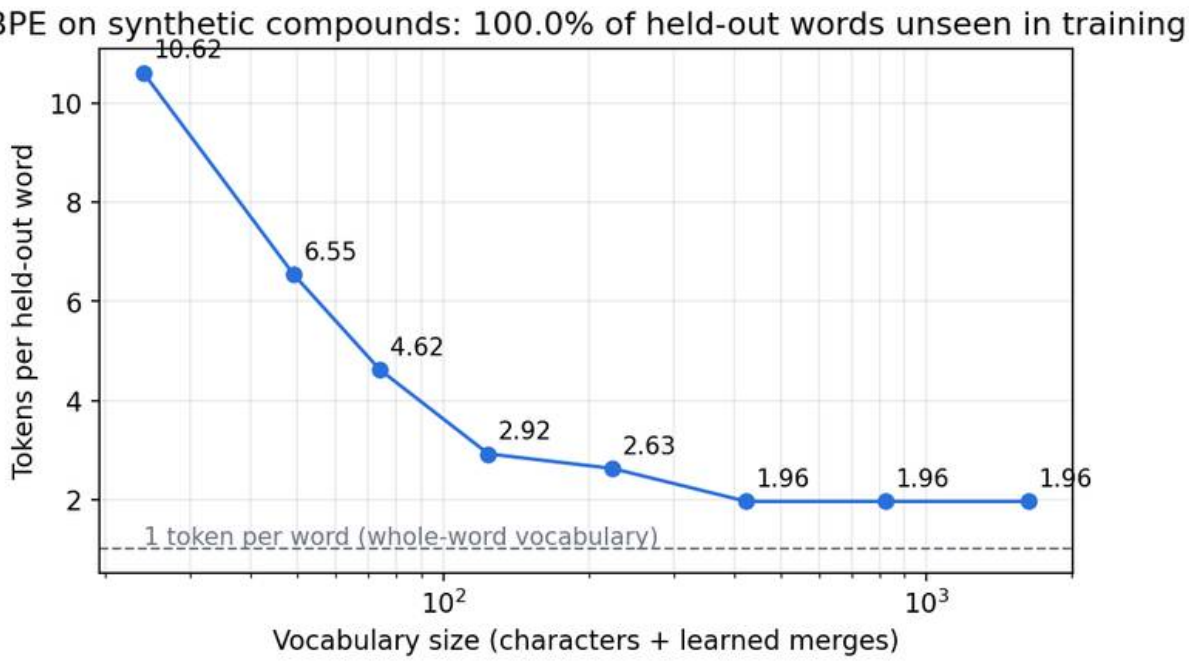


圖 *1* — 更多合併可能縮短編碼，但只看詞彙表大小，無法決定覆蓋率或語言品質。合成訓練樣本與保留樣本互相分開。

WordPiece、unigram 模型與 SentencePiece

BPE 只是其中一員。其他方法的差別在於如何選擇片段，以及如何切分新詞。

- **WordPiece** 源自日語與韓語語音搜尋 [18]。Google 神經機器翻譯系統描述它的版本時，說明是選出固定數量的片段，使切分後的訓練語料所用片段數最少，並在每個詞開頭加上邊界標記，讓原始詞序列能無歧義地還原 [19]。BERT 使用 WordPiece 詞彙表 [30]；NLP 10 會在此基礎上繼續。
- **Unigram 語言模型**切分由上而下：先從大型候選片段集合開始，把一個詞建模成獨立抽出的片段序列，再刪去對語料概似度貢獻最小的片段 [20]。因為它是機率模型，同一個詞可以有數種帶分數的切分；Kudo 利用這點在訓練時抽樣不同切分，作為正則化方法 [20]。
- **SentencePiece** 是工具組，而不是新的準則：它直接從原始句子訓練 BPE 或 unigram 模型，並把空白當成一般符號，因此不需要特定語言的預切分器，解碼也能還原編碼 [21]。這對中文、日文等不以空白分詞的語言很重要。

在這三種方法中，學到的片段都是統計上的，而不是構詞上的。子詞邊界剛好落在 `est`，是語料頻率的證據，並不是「`-est` 為最高級詞尾」的分析。

使用完整斷詞器，而不只是一份合併清單

斷詞軟體還會指定正規化、預切分、特殊標記與解碼 [23]。模型檢查點的 token ID 對應特定詞彙表與約定。只匹配斷詞器家族名稱並不足夠：兩個詞彙表可能把同一 ID 指派給不同符號。

預訓練模型應使用隨附的斷詞器與設定。若在實驗中訓練斷詞器，就只用允許的訓練文字擬合，評估前固定詞彙表與合併規則。分別查看片段、ID 與重建文字，各種檢視能找出不同錯誤。

4. 詞性

白話版。詞的類別描述它在句子中的行為。拼字能提供線索，但脈絡通常決定實際用法。

詞性（part of speech）描述名詞、動詞等語法類別。名詞通常是名詞片語的中心，動詞通常是子句述語的中心。這是分布上的描述，不是每個名詞都必須指實體物品的規則。「Decision」與「arrival」雖指抽象內容與事件，仍是名詞 [2]。

相同表面詞可以有不同類別。在原創例句「the pilots can land」與「the metal can fell」中，「can」扮演不同角色。第二句也顯示，句子其餘部分改變時，熟悉的局部片語可能誤導。標記器必須考慮允許的脈絡與標註規則。

Universal Dependencies 標記集

不同專案的標記集合各不相同，讓跨語言比較變得困難。**Universal Dependencies**（UD）是一套對詞性、構詞特徵與依存關係做一致跨語言標註的框架 [24]。它的通用詞性（UPOS）集合有 17 個標記 [25]：

類別	標記
開放類	ADJ 形容詞、ADV 副詞、INTJ 感嘆詞、NOUN 名詞、PROPN 專有名詞、VERB 動詞
封閉類	ADP 介詞（adposition）、AUX 助動詞、CCONJ 對等連接詞、DET 限定詞、NUM 數詞、PART 小品詞、PRON 代名詞、SCONJ 從屬連接詞
其他	PUNCT 標點、SYM 符號、X 其他

UD 把粗略標記與 `Number=Plur`、`Tense=Past` 等**特徵**分開，因此作名詞的 “repairs” 可以標成 `NOUN` 加上 `Number=Plur`，而 “repaired” 是 `VERB` 加上 `Tense=Past`。各語言的樹庫還會為通用準則未規定的構式補充文件化的慣例。這個標記集是共用的標註方案，不代表每種語言都在相同位置劃分類別界線。

已標記句子可表示為：

$$\mathbf{x} = (x_1, \dots, x_T), \qquad \mathbf{y} = (y_1, \dots, y_T), \qquad y_t \in \mathcal{Y}.$$

其中 $\mathbf{x}$ 是輸入詞序列，x_t 是位置 t 的詞，T 是序列長度，$\mathbf{y}$ 是標記序列，y_t 是詞 x_t 的標記，$\mathcal{Y}$ 是有限標記集合，例如 17 個 UPOS 標記。t 與 T 指語言學上的詞位置，不會自動等同模型子詞位置。

標註也需要縮寫、標點、多詞表達與多種合理分析的規則。當模型使用子詞時，應定義如何把輸出對應回詞。只用第一個子詞表示是一種做法；池化所有子詞則是另一種。不論選哪個，標準標籤與解碼輸出都必須使用相同單位。用隱藏式馬可夫模型、Viterbi 解碼與循環神經網路標記器，從 $\mathbf{x}$ 預測 $\mathbf{y}$，是NLP 06 的主題。

5. 句法：順序、分組與文法關係

白話版。文法把詞組成詞組，並以主詞、受詞等關係連接它們。相同的詞放進不同結構，可能描述不同事件。

文法組織詞的組合。詞序、詞組結構，以及主詞、受詞等關係，限制誰對誰做了什麼 [2, 3]。這些限制比相鄰詞的清單豐富。

詞頻為何會失去角色資訊

“Ada praised Bo” 與 “Bo praised Ada” 包含相同的詞，但普通主動句解讀會交換稱讚者與被稱讚者。中文「阿達稱讚小博 / 小博稱讚阿達」在這兩句裡形成相同對比。只計算詞頻的表示，會把每一組句子映射成相同的對象。

```
from collections import Counter

first = "Ada praised Bo".split()
second = "Bo praised Ada".split()

def roles(tokens):
```

```
    # Only our deliberately fixed three-word active-clause template.
    assert len(tokens) == 3 and tokens[1] == "praised"
    return {"praiser": tokens[0], "praised_person": tokens[2]}

print(Counter(first) == Counter(second))
print(roles(first))
print(roles(second))
assert roles(first) != roles(second)
```

詞頻比較印出 `True`，角色字典則不同。這是資訊流失的結果，不是訓練模型的效能測試。當兩個輸入已經變成相同詞頻後，任何只依賴那些詞頻的後續確定性函數，都無法還原原先是哪種順序。

句法角色也不等於語意角色。在 “Bo was praised by Ada” 裡，Bo 是文法主詞，但 Ada 仍然是執行稱讚的人。即使被動句描述同一事件，把主詞直接當成行動者仍會失敗。抽取任務應說明它需要文法依存關係、事件參與者，還是兩者都需要。

成分結構與依存關係描述不同面向

句法成分（constituent）是參與更大結構的單位。在 “The label beside the boxes is missing” 裡，名詞片語 “the label beside the boxes” 作為主詞。它的中心語（head）是 “label”；“beside the boxes” 修飾這個中心語。把整個片語換成 “it”，仍保留簡單的文法框架：“It is missing”。這類測試提供分組的證據，不表示每段相鄰子字串都是詞組 [8]。

一致關係（agreement）的對比 “The labels beside the box are missing” 說明為何只看距離不夠。在這些例子中，動詞與主詞的中心語一致，而不是與最近的名詞 “box” 一致。局部啟發式規則即使正確找到最近名詞，仍可能選錯單複數。

依存（dependency）表示則記錄詞之間的關係，例如動詞的主詞與受詞，或名詞的修飾語 [2]。UD 使用依存樹：每個詞恰好有一個中心語，並以帶標籤的關係連到它，例如名詞性主詞 `nsubj` [24]。成分結構凸顯分組，依存關係凸顯相連的中心語。兩者可以描述相關證據，不必有完全相同的結構或標註慣例。早期大型英文資源 Penn Treebank 標註的是成分樹 [26]；UD 則以依存關係標註許多語言 [24]。

在面板範例中，“with the camera” 可以修飾檢查事件，也可以修飾面板。下圖只保留這一個附著差異，省略限定詞與其他關係。它們是教學示意圖，不是依某一具名標註標準完成的剖析結果。

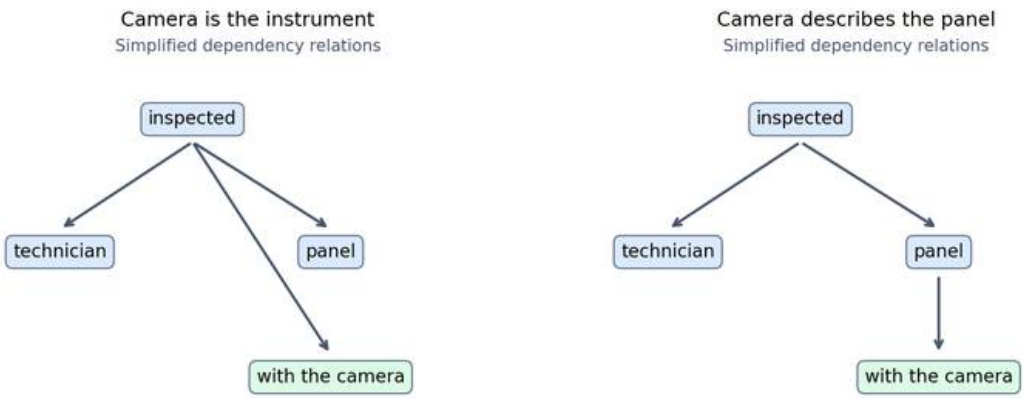


圖 2 — 相同的詞可以支持不同附著關係。選擇關係，也就選擇了不同解讀。

若附著於事件，相機是檢查的工具；若附著於名詞，相機協助辨認面板。前後文可以支持其中一種：例如技師解釋錄影方法，或有人區分有裝相機與沒有裝相機的面板。文法上的可能性不決定實際發生了哪一種情境。

這裡只是預覽。NLP 12 會深入文法、圖表剖析、機率式剖析與依存剖析。此處的核心問題是：表示需要保留什麼？答案包括分組、相連的中心語、角色方向，以及合理的替代分析。

6. 語意：意義與真值條件

白話版。語意學問一個表達式是什麼意思，以及世界要是什麼樣子，句子才為真。詞義重要，句子如何組合詞義也同樣重要。

句子可以很流暢，卻錯誤描述情境。**語意學（semantics）**研究表達式如何貢獻意義，以及有結構的句子如何決定斷言為真的條件 [2, 3]。把這些條件與會話推論分開，能讓歧義更容易分析。

詞彙意義依賴語境與關係

多義（polysemy）涉及同一表達式的相關義項；**同形異義（homonymy）**涉及共享形式的不同意義 [3]。window 可以指牆上的開口，也可以指螢幕上有邊界的區域。兩者的關係有助於解釋這種延伸，但若把 “window” 固定換成某一具體物件，就會漏掉螢幕上的解讀。字典可以列出義項，實際出現的詞仍需要解讀。NLP 04 會研究從語境學到的向量如何捕捉其中一些關係。

中文「開」出現在「開門」與「開會」中，英文分別常用 “open a door” 與 “hold a meeting”。把每個「開」都翻成同一英文動詞，會抹掉與構式相關的差異。意義屬於表達式在組合與語境中的使用，不是無視語境的中文字元到英文詞對照。

即使看似簡單的形容詞，也可能依賴比較類別。「小象」可能比「大螞蟻」大。這些表達式把動物與適當類別相比，而不是使用一個普遍的尺寸門檻。在一組只有玩具象的物件中，相關比較又可能改變。因此，名為「小」的標註，應說明比較的對象是什麼。

組合依賴結構

組合性（compositionality）指複合表達式的意義，取決於各部分的意義與組合方式 [3]。它不表示只要把詞義相加即可。“Bo praised Ada” 把參與者放入有方向的關係；重排句子會用不同方式組合同一組名字。附著差異也會改變修飾語限制哪個表達式。

慣用表達式也可能有約定俗成的意義，無法從獨立詞條預測。詞庫與文法互相補充：分析必須同時記錄慣例組合與生產性的組合。對應用而言，要問任務需要哪種區別。搜尋維修說明也許能接受概括的詞彙歸類；判斷維修是否完成，則需要事件、時態與否定資訊。

量詞作用域改變要求

“Every technician uses a camera” 有一種解讀，容許不同技師使用不同相機；在適當語境裡，也能理解成有一台所有技師共用的相機。這兩種條件不同：

$$A:\ \forall t \in T,\ \exists c \in C:\ U(t,c),$$
$$B:\ \exists c \in C,\ \forall t \in T:\ U(t,c).$$

其中 T 是技師集合，C 是相機集合，$U(t,c)$ 表示技師 t 使用相機 c，A 與 B 是兩種解讀的名稱；$\forall$ 表示「對每一個」，$\exists$ 表示「存在」。本例的兩個集合都非空。在 A 裡，變數 c 可以隨 t 改變；在 B 裡，同一個 c 必須適用於所有 t。

檢查 A 時，逐一查看每位技師，找到至少一台他使用的相機。檢查 B 時，先選一台相機，再確認所有技師都使用那一台。如果 B 成立，A 就成立：對每位技師都選那台共用相機即可。反向推論不成立，因為每個人都可能有相機，卻沒有一台相機是所有人共用的。

```
technicians = ("Ada", "Bo", "Cy")
cameras = ("A", "B")
worlds = {
    "distributed": {("Ada", "A"), ("Bo", "B"), ("Cy", "A")},
    "shared": {("Ada", "A"), ("Bo", "A"), ("Cy", "A")},
}
for name, uses in worlds.items():
    each_has_some = all(any((t, c) in uses for c in cameras)
                        for t in technicians)
    some_for_everyone = any(all((t, c) in uses for t in technicians)
                            for c in cameras)
    print(name, each_has_some, some_for_everyone)
assert not any(all((t, c) in worlds["distributed"] for t in technicians)
               for c in cameras)
```

輸出為 `distributed True False` 與 `shared True True`。圖 3 每列表示一個人的選項。第一種解讀要求每列至少有一格被標記；第二種要求至少有一整欄都被標記。因此，作用域改變的是限制條件，不只是把相同要求改寫成另一句話。

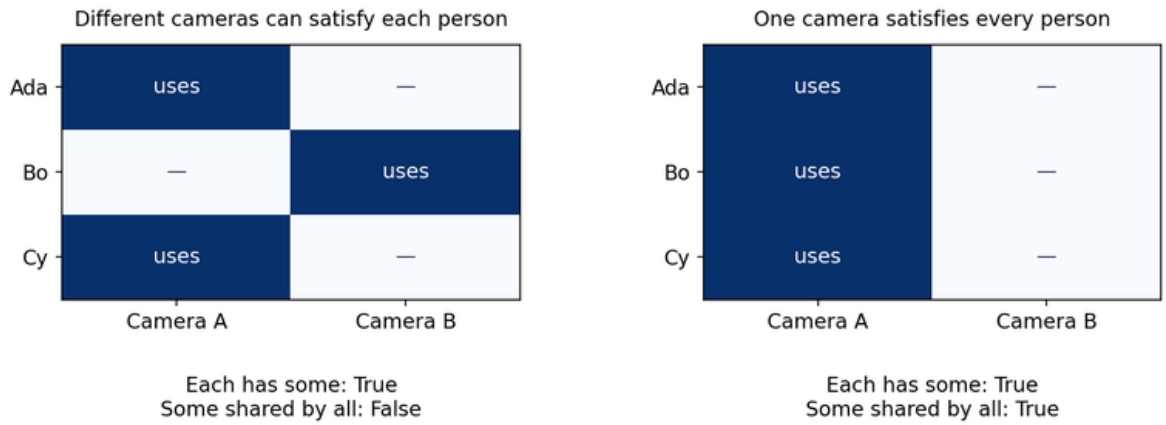


圖 3 — 改變量詞順序，會改變哪些情境滿足句子。

否定與作用域也必須分清楚

“Not every technician approved” 容許有人批准，但至少有一人沒有批准。“Every technician did not approve” 若採取否定在全稱量詞內部的解讀，則表示沒有人批准。書寫英文自身也可能留下作用域歧義；明確改寫能消除這種不確定性。

中文「不是每位技師都批准 / 每位技師都不批准」較容易展示這個區別。前者否定「所有人都批准」，後者對每位技師指定不批准。對非空的技師集合而言：

$$\neg\forall t \in T\ P(t) \ \equiv\ \exists t \in T\ \neg P(t), \qquad \forall t \in T\ \neg P(t) \ \equiv\ \neg\exists t \in T\ P(t).$$

其中 T 是技師集合，$P(t)$ 表示 t 批准，$\neg$ 是否定，$\forall$ 與 $\exists$ 分別是全稱與存在量詞，$\equiv$ 表示邏輯等價。第一個條件只需一個反例，就能推翻全體批准；第二個條件則排除所有批准。若一位技師批准、另一位沒有批准，就只有第一個條件為真。

NLP 12 會深入述詞與論元結構及語意角色標註，NLP 11 則測試句子之間的推論。在設計 `approved` 這類欄位之前，先確定它代表誰的批准、哪個事件，以及哪個帶有量詞的主張。

7. 語用：在情境中解讀

白話版。說話者的意思可能超出句子的字面內容。語用學研究情境如何補上這段落差。

說話者經常傳達超過句子字面斷言的內容。**語用學（pragmatics）**研究語境中的語言使用，包括指稱、背景假設與溝通行動 [9, 10]。語境提供證據，卻不代表可以任意解讀。

指稱與說話者的視角

“I will check that here tomorrow” 的結構很簡單，離開情境後卻缺少許多資訊。“I” 指說話者，“that” 選擇指稱對象，“here” 使用空間視角，“tomorrow” 使用時間視角。這些依賴語境的表達式涉及**指示（deixis）**[10]。轉寄的訊息不應把原說話者的時間與地點，默默換成接收者的時間與地點。

指稱也依賴篇章。“Mei put a camera beside a lamp. It was overheating.” 若沒有更多證據，“it” 可能指任一物件。知道相機與燈都可能變熱，仍然無法消除歧義。在 “Mei turned off the lamp. It stopped flickering” 裡，所描述的事件提供了支持「燈」的證據。證據可以支持某個候選，但不會把文法上的可能性變成普遍規則。

實際分析可以記錄表達式、候選指稱對象，以及支持它們的語境。若輸入不能確定指稱，保留未解決狀態也是有意義的結果。跨句子的共指解析在NLP 12 中發展，跨對話輪次追蹤實體則在NLP 08。

斷言、預設與會話含意

斷言（assertion）提出當下討論的主張。**預設（presupposition）**把命題當成背景，而不是主要更新。**會話含意（conversational implicature）**則是根據發話內容及其使用方式推得的額外意義，往往可以取消而不否定字面斷言 [9, 10]。三者不應被壓成同一個「意義」標籤。

“Mei stopped checking the sensor” 斷言某個活動停止，並通常把 Mei 先前檢查感測器當成背景。“Mei did not stop checking the sensor” 也通常把先前檢查視為已知。這種在否定下仍持續的貢獻，有助於辨認背景意義。它們是一般解讀，不是先前事件真的發生的證明。聽者可以質疑：「她根本從來沒有檢查過。」

“Some panels passed” 斷言相關集合裡至少有一個面板通過。在詢問整批是否都通過的對話裡，聽者可能推論「並非全部」。但是 “Some panels passed; in fact, all of them did” 可以取消這個推論，而不否定第一個斷言 [9]。相對地，若面板集合、測試與時間相同，“All panels passed; in fact, one failed” 就互相衝突。

這個差異不需要語言模型也能檢查。想像三個面板：一個、兩個或三個通過，都滿足字面的 “some”。“Some but not all” 的解讀則只保留前兩種可能。會話上的預期讓推論縮小了可能情境；這種縮小不是存在意義本身提供的。它是否合理，取決於問題、說話者的知識與報告方式。

文法上的問句可以執行請求

“Can you lower the gain?” 在形式上詢問能力。在錄音過程中，它可能是在請求調整設定；在測驗中，則可能詢問受訓者是否知道如何操作 [10]。**言語行為（speech act）**是透過發話執行的溝通行動，例如請求、承諾、斷言或提問。

“The gain is too high” 同樣可能是報告，也可能是間接請求。實際詞句本身不會指定所有人際條件。誰控制設備？前一個問題是什麼？說話者是在診斷錄音，還是在協調調整？這些細節為行動解讀提供依據。

有用的標註會分開記錄句子形式、斷言內容與意圖行動。否則，所有問句形式的請求都可能被標成尋求資訊，或所有負面描述都可能被標成指令。目標是對這次互動提出有依據的解讀；缺少情

境時，也要保留其他選項。

8. 篇章：跨句子與輪次的意義

白話版。文本或對話中的句子彼此依靠。後面的句子會回指、對比、解釋或修正前文，讀者則持續追蹤這個共同狀態。

發話可以引入實體、修正先前主張、提供理由，或對比預期。**篇章（discourse）**分析研究句子與對話輪次之間的連結 [2, 10]。逐句獨立閱讀，會丟掉讓後續表達式可被理解的共同狀態。

“Mei told Lin that she would calibrate the camera” 可以把任一人當成未來校正者。中文「美玲告訴怡君，她會校正相機」在這個虛構情境中也有相似歧義。前文若說「怡君已經自願幫忙」，就支持怡君；若說「美玲是唯一有空的技師」，就支持美玲。文法性別或提及順序，單獨都無法決定兩種語境。

比較 “The sensor became hot. Its readings stayed stable” 與 “The sensor became hot, but its readings stayed stable”。後者明確標記：結果違反熱可能干擾讀值的預期。關於穩定的斷言可以相同，篇章貢獻卻不同。相對地，“The readings drifted because the sensor became hot” 提出因果解釋，不只是依序出現兩件事。

對話也包含修復。“Move it to Friday.” “The inspection?” “Yes, the inspection.” 中間一輪不是在推進排程日期，而是在確定物件。釐清後，後續指稱可以使用共同確認的對象。若對話系統只摘要第一句與最後一句的詞，就會漏掉澄清如何改變對話狀態。

這提示一種實用表示：分開記錄已引入的實體、斷言事件、未解決指稱與對話問題。表示可以保留不完整。一句輸入可能沒有說明說話者；完整交換可能確定說話者，卻仍未指出確切相機。增加語境長度，只有在相關證據確實存在時才有幫助。

9. 語料庫、標註與 Zipf 定律

白話版。 NLP 從真實文字集合中學習，這些文字常附有人工標籤。文字如何蒐集、標籤是否一致，限制了任何模型能學到的內容；而且每個語料庫都有一條由罕見詞組成的長尾。

語料庫（corpus）是依明確決定蒐集的文字或語音轉寫：哪些來源、哪段時期、哪些說話者、哪些語言變體。**標註語料庫（annotated corpus）**再依書面準則加上分析，例如斷詞、標記、樹、實體範圍或情感標籤。兩個廣泛使用的例子顯示其範圍：Penn Treebank 為英文文字標註詞性與成分樹 [26]；Universal Dependencies 專案以同一套跨語言依存方案，釋出許多語言的樹庫 [24]。前面的斷詞評測語料庫也說明，連詞邊界都附帶一套標準 [27]。

標註準則與一致度

標籤的清楚程度取決於背後的準則。兩位細心的標註者可能因為句子本身有歧義、準則沒有規定，或其中一人出錯而意見不同。在共同樣本上量測一致度，能在把標籤拿去訓練之前區分這些情況。

當某個標籤非常常見時，原始的百分比一致度會高估可靠性，因為標註者即使用猜的也常會一致。Cohen's kappa 修正了這一點 [28]：

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \qquad p_e = \sum_{k \in \mathcal{U}} \frac{n_{A,k}}{n} \cdot \frac{n_{B,k}}{n}.$$

其中 p_o 是標註者 A 與 B 意見一致的項目比例；p_e 是假設兩人各自依觀察到的比例獨立選標籤時，預期的一致比例；$\mathcal{U}$ 是標籤集合；$n_{A,k}$ 與 $n_{B,k}$ 分別是兩位標註者使用標籤 k 的次數；n 是項目數。$\kappa = 1$ 代表完全一致，$\kappa = 0$ 代表只達到機率水準的一致；公式要求 $p_e < 1$。

```
from collections import Counter
from sklearn.metrics import cohen_kappa_score

words = "Mei has to repair the camera mount before the inspection".split()
a = "PROPN VERB PART VERB DET NOUN NOUN ADP DET NOUN".split()   # annotator A
b = "PROPN AUX ADP VERB DET NOUN NOUN ADP DET NOUN".split()     # annotator B
n = len(words)
p_o = sum(x == y for x, y in zip(a, b)) / n                     # observed agreement
ca, cb = Counter(a), Counter(b)
p_e = sum(ca[k] * cb[k] for k in ca) / n**2                     # agreement expected by
 ↪  chance
kappa = (p_o - p_e) / (1 - p_e)
print(f"p_o={p_o:.3f}  p_e={p_e:.3f}  kappa={kappa:.3f}")
assert abs(kappa - cohen_kappa_score(a, b)) < 1e-12
print("disagreements:", [(w, x, y) for w, x, y in zip(words, a, b) if x != y])
```

程式印出 `p_o=0.800  p_e=0.180  kappa=0.756`，兩處分歧是 `has`（`VERB` 對 `AUX`）與 `to`（`PART` 對 `ADP`）。兩者都不是粗心失誤："has to" 位於主要動詞與助動詞的邊界，不定詞的 "to" 也很像介詞。有用的結果是針對這個構式做出準則決定，而不是更高的數字。Kappa 總結可靠性；分歧清單則指出該修正什麼。

Zipf 定律：少數詞非常常見，多數詞很罕見

把語料庫裡每個詞都計數，再依頻率排序型別。少數功能詞如 "the" 與 "of" 占了所有 token 的一大部分，而大量型別只出現一兩次。**Zipf 定律**把詞的頻率排名與次數之間的關係描述為近似的冪律 [29]：

$$f(r) \approx \frac{C}{r^s}, \qquad \log f(r) \approx \log C - s \log r.$$

其中 $r = 1, 2, \ldots$ 是型別依次數遞減排序後的名次，$f(r)$ 是名次 r 的型別出現次數，$C > 0$ 是尺度常數，$s > 0$ 是指數。當 $s \approx 1$ 時，乘積 $r \cdot f(r)$ 大致固定。在雙對數座標上，這個關係是一條斜率為 $-s$ 的直線。Piantadosi 的回顧指出，這個定律在許多語言與語料庫中近似成立，在兩端有系統性的偏離，而其成因仍有爭論 [29]。

我們可以用已經在磁碟上的真實英文文字檢查它：專案環境（Python 3.9）所安裝的 Python 標準函式庫說明字串（docstring）。

```
import ast, re, sysconfig
from collections import Counter
from pathlib import Path

lib = Path(sysconfig.get_paths()["stdlib"])        # text already on disk: stdlib docstrings
docs = []
for path in sorted(lib.glob("*.py")):
    try:
        tree = ast.parse(path.read_text(encoding="utf-8"))
    except (SyntaxError, UnicodeDecodeError):
        continue
    docs += [ast.get_docstring(n) or "" for n in ast.walk(tree)
             if isinstance(n, (ast.Module, ast.FunctionDef, ast.AsyncFunctionDef,
              ↪  ast.ClassDef))]
tokens = [w for d in docs for w in re.findall(r"[a-z]+", d.lower())]
counts = Counter(tokens)
print(len(tokens), "tokens,", len(counts), "types")
ranked = counts.most_common()
for r in (1, 2, 10, 100, 1000):
    word, f = ranked[r - 1]
    print(f"rank {r:4d}  {word:10s} f={f:5d}  r*f={r * f}")
hapax = sum(f == 1 for f in counts.values())
print(f"types seen once: {hapax} ({hapax / len(counts):.0%})")
```

在 Python 3.9.7 下，程式找到 134,620 個 token 與 6,281 個型別。乘積 $r \cdot f(r)$ 在名次 1（“the”）是 8,313，名次 10（“be”）是 13,640，名次 100（“called”）是 20,100，名次 1,000 是 18,000：並不固定，但名次變動 1,000 倍時，乘積只在約 2.5 倍範圍內變動。圖表腳本在名次 10 到 1,000 之間做最小平方擬合，得到 $s \approx 1.00$。所有型別中有三分之一，也就是 2,003 個（32%），恰好只出現一次。不同 Python 版本附帶的說明字串不同，確切計數可能改變。

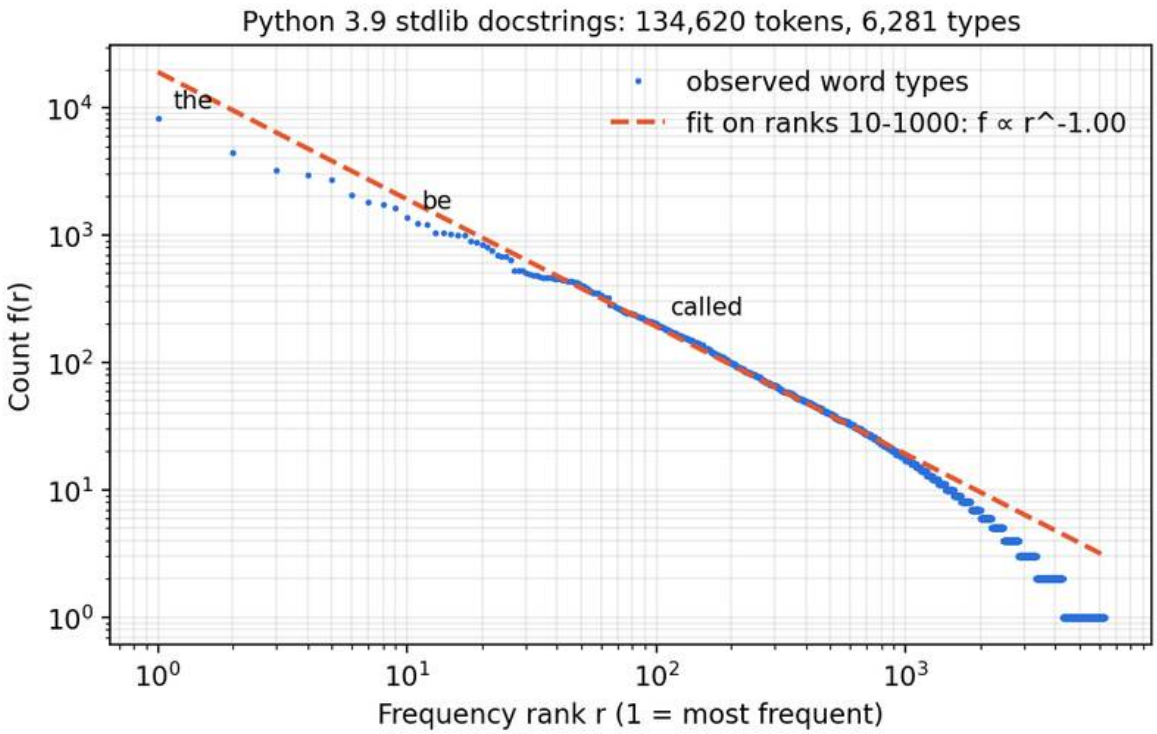


圖 *4 — Python 3.9* 標準函式庫說明字串的名次與頻率圖。虛線是在名次 *10–1,000* 上擬合的冪律；頭部與只出現一次的長尾都偏離這條線。

這對 NLP 有三個後果。第一，任何固定的詞彙表都會漏掉一部分長尾，因此新文字總會帶來沒見過的詞；這是子詞詞彙表存在的原因。第二，以計數估計的機率對常見詞可靠，對占多數的罕見詞則不可靠，這就是 n-gram 模型需要平滑化的原因（NLP 05）。第三，以所有 token 計算的準確率主要由常見詞決定；若罕見名稱或技術詞彙很重要，評估就必須另外量測它們。

10. 語言多樣性與模型假設

白話版。不同語言用不同方式包裝資訊，不同社群也使用不同的語言變體。圍繞某一語言習慣建立的模型，可能錯過另一種語言所表達的內容。

不同語言與社群會把資訊分配到構詞、詞序、韻律與語境的不同位置。**語言類型學（typology）**比較這些結構模式。**方言（dialect）**涉及社群的語言變體；**語域（register）**涉及特定情境或活動的使用方式 [3, 11]。這些區別描述變異，不是替說話方式排序。

翻譯不會保留每種結構

比較 “I calibrated this camera yesterday” 與「這台相機，我昨天校正過」。中文例子先把相機設為主題，再對它作出評論。把第一個名詞片語直接當成行動者，就會誤認校正者。英文翻譯可以保留事件，卻改變訊息如何組織已知與新資訊。

當相機與負責者已經確定時，短句「已經校正好了」可以提供充分資訊。逐字翻譯若要求每個英文論元都有明說的對應項目，可能把它看成不完整。在語境中，它卻能直接回答待解問題。這不表示省略的參與者一定能被找回；分析仍須查看實際篇章。

同樣地，「三台相機」與 “three cameras” 用不同方式包裝數量。不能只因為「複數詞尾」適合某一語言，就把這個特徵設為普遍要求。結構類別可以用於比較，同時需要各語言的實現與標註決定。強制詞尾較少的語言，仍然可以透過構式、詞與語境表達細緻區別。

變體、情境與混合語言表達

正式維修報告與技師聊天，可能用不同詞彙、縮寫與明確程度描述相同設備。報告的 “The shutter failed to actuate” 與口語的 “The shutter didn’t fire”，在語境中可能描述相關事件，但措辭提供不同的情境與讀者證據。不熟悉的說法不會自動成為錯誤。

說話者可能在雙語討論裡寫「把 exposure 調低一點」。英文項目出現在中文構式的一個位置。**語碼轉換（code-switching）**涉及互動中不同語言變體的交替 [14]；分析本例需要同時看詞彙選擇與周圍文法。只憑一次出現，未必能確定社群把該項目當成語碼轉換，還是已經成立的借詞。

分析應先保留觀察到的表達，再進行標準化。把地區或技術表達換成預設標準，可能有助於某個搜尋任務，卻抹掉另一任務需要的區別。語料庫記錄特定說話者、情境與蒐集決定下的語言；常見語料模式是使用證據，不是所有說話者可說內容的普遍清單。

11. 一次完整的閱讀分析

用一則虛構訊息，把不同層次放在一起：

> 小林說：「那台相機又過熱了，你明天能幫我看看嗎？」
>
> Lin said: “That camera overheated again. Can you take a look for me tomorrow?”

翻譯有助於英文讀者，但已經替「看看」與引述互動的標點選擇措辭。在把翻譯當成等價證據之前，先分析原文。

層次	訊息提供的內容	仍需確定的事項
書寫形式與斷詞	引號把小林的發話分成一組；相機是物件表達式	語音的語調與手勢不存在於文字中
構詞與構式	看看在此請求裡是重疊形式，不是計數兩次檢查	確切活動與預期投入取決於情境
句法	小林引入被報導的說話者；你是引號內的受話者	報導者不一定是受話者
語意	相機被描述為過熱；句子表達未來協助的問題	哪台相機，以及描述是否符合事實
背景與指稱	又通常援引先前發生；那台選擇語境中可辨認的相機	先前過熱與指示詞的對象需要證據
語用與篇章	在維修對話裡，問句自然支持請求解讀	該解讀是否符合這次互動，以及請求是否被接受

引述改變參照框架。「我」指引號內的說話者小林。「你」指小林的受話者，可能與目前讀者不同。「明天」相對於引號內發話的日期。如果報導較晚才送達，送達日期不會重新定義請求的日期。

在這個構式裡，「看看」的一般解讀呈現「看一下」的活動，往往帶有較輕或試探性的語氣 [13]。它不會確定精確檢查時間或數值上的重複次數。若翻成 “inspect twice”，就是把可見的形式重複誤當成事件計數。

最後，把語言證據與事實證據分開。訊息提供小林曾報告過熱與請求協助的證據；它本身不會驗證過熱、指出序號，或證明有人同意幫忙。結構化輸出可以記錄說話者與被報導事件，同時保留未解決指稱與日期。這比只給出「相機維修」之類的主題標籤，更能提供有用分析。

現代觀點

近期綜述說明，對大型語言模型而言，語言分析仍然必要。Ma 等人的 2025 年綜述，依指稱與指示、會話含意與預設、言語行為、篇章、社會語用等現象，組織語用評估 [10]。它比較任務設計、資料蒐集與評估，指出現有測試與語境中語言使用的落差。對本教學最有用的啟示是：評估任務真正要求的推論，不要把概括的語言分數當成所有語用能力的證明。

Mielke 等人的斷詞綜述，從詞、字元與中間單位整理開放詞彙問題 [22]。有用的問題是切分如何平衡覆蓋率、序列長度與重用。這些量彼此不同：斷詞器可以表示每個輸入字串，模型卻仍可能缺乏學習相關組合的訓練證據。Zipf 定律說明了這個問題為何永遠不會消失：無論詞彙表多大，長尾都會持續帶來罕見形式 [29]。

Qin 等人的 2025 年綜述，區分透過參數調整的多語言對齊，以及固定參數下的對齊，包括提示與翻譯式方法 [12]。它整理跨語言訓練、評估與挑戰。這個區分指出系統在哪裡調整行為，但能產生多語言輸出，不代表它保留構式的作用域、語域或會話功能。

Winata 等人的 2023 年系統性綜述，依語言、任務與研究趨勢，組織語碼轉換研究 [15]。可以搭配 Doğruöz 等人對結構與社會模式的分析 [14]：測試由兩個單語來源拼接的文字，與解讀雙語說話者如何在互動中實際切換，是不同任務。第 10 節 exposure 的例子把這點具體呈現出來：除了辨認英文項目，周圍構式與對話情境也提供資訊。

語言學分析也持續細化看似簡單的構式。Lu 與 Müller 的 2026 年研究，分析華語動詞重疊及其與動詞類別、時貌的互動 [13]。本教學相關的觀察是：有限活動的意義，以及依賴語境的語用效果，值得分開分析。看得到的形式重複，不是要求把事件計數兩次的普遍規則。

具體評估可以固定其他條件，只改變一項特性。交換 Ada 與 Bo，測試角色方向；保留詞彙、改變相機的附著；比較「不是全部」與「沒有任何一個」；把引述訊息移到較晚的報導語境。先寫出預期解讀與可接受選項，再觀察失敗在哪個區別上。這些是小型診斷測試，不表示通過後就具備一般語言能力。

開放問題包括如何取得相關語境、區分真正歧義與可避免的標註歧見、評估自然的混合語言對話，以及跨社群測量解讀，而不只使用翻譯提示。有些輸入本來就容許多種讀法。說明選項與缺少證據的系統，可能比自信選定單一答案的系統更忠於語言。

重點整理

- 口語、手語與書寫揭露不同資訊；逐字稿只是部分紀錄。
- 計算用 token 不必與詞、詞素、詞位或語音單位一致；BPE、WordPiece 或 unigram 模型學到的子詞片段反映頻率，而不是構詞。
- 詞幹擷取依規則切掉詞尾；詞形還原對應到字典形式，並需要詞性。兩者都會丟掉某些任務需要的資訊。
- 構詞結構與中文斷詞編碼了分析選擇，也可能有替代方案，而不同語料庫依循不同標準。
- 詞頻會丟掉順序；詞性類別、分組與文法關係提供額外意義。成分結構顯示分組，依存關係顯示中心語。
- 作用域決定真值條件：「每個人都有某個」不等於「某一個被所有人共用」。
- 斷言、背景假設、可取消的推論、指稱、言語行為與篇章關係，需要說話者的情境及對話歷史。
- 語料庫帶有蒐集選擇與標註準則；要檢查一致度，並預期罕見詞形成 Zipf 式長尾。

練習

1. 在 “The technician repaired two cameras” 裡，辨認詞形、詞位、屈折對比、一組 UPOS 標記序列，以及一種計算用 token 切分。解釋每種表示保留什麼。

   提示

   比較 “repaired” 與 REPAIR。Tokenizer 可以保留整個詞形或拆開；兩種選擇都不能單獨確定時態解讀，UD 則把時態記錄為特徵。

2. 替「學生會說話」的兩種解讀各加一句語境。解釋為何只列舉詞庫路徑，無法偏好任一語境。

   提示

   一種情境可以討論學生的能力；另一種可以討論學生會宣布立場。

3. 改變 BPE 語料中一個訓練詞的頻率。檢查第一個合併，再編碼 `lowest`。為什麼兩種結果都不必維持不變？

   提示

   符號對計數以詞的出現次數加權。前面的合併會改變後面可使用的符號對。

4. 替 “The technician inspected the panel with the camera” 寫出兩種語境，各支持一種附著。辨認修飾語連到哪個中心語，以及事件描述如何改變。

   提示

   對比檢查方法，以及在多個面板中辨認裝有相機的面板。

5. 修改量詞程式，加入一位沒有使用任何相機的技師。執行前先預測兩種解讀，再解釋 *B* 為何蘊涵 *A*。

提示

若任一技師沒有可用配對，兩個條件都失敗。一台共用相機能替每位技師提供見證。

6. 分別分析 “Mei stopped checking the sensor” 與 “Some panels passed” 在否定與明確取消下的表現。區分主要斷言、一般背景假設，以及會話推論。

提示

試著用 “Mei never checked it” 挑戰背景，用 “in fact, all passed” 取消推論。維持相同面板集合與測試。

參考文獻

1. Masato Hagiwara, *Real-World Natural Language Processing: Practical Applications with Deep Learning*, Manning, 2021. 出版社
2. Susan McRoy, *Principles of Natural Language Processing*, University of Wisconsin–Milwaukee open textbook, 2021. Grammars and Syntactic Processing
3. Catherine Anderson, Bronwyn Bjorkman, Derek Denis, Julianne Doner, Margaret Grant, Nathan Sanders and Ai Taniguchi, *Essentials of Linguistics*, 2nd ed., eCampusOntario, 2022. 開放教科書
4. Catherine Anderson et al., “3.1 Modality,” *Essentials of Linguistics*, 2nd ed., eCampusOntario, 2022. 章節
5. Catherine Anderson et al., “4.1 Phonemes and allophones,” *Essentials of Linguistics*, 2nd ed., eCampusOntario, 2022. 章節
6. Catherine Anderson et al., “3.12 Tone and intonation,” *Essentials of Linguistics*, 2nd ed., eCampusOntario, 2022. 章節
7. Catherine Anderson et al., “5.1 What is morphology?,” *Essentials of Linguistics*, 2nd ed., eCampusOntario, 2022. 章節
8. Catherine Anderson et al., “6.4 Identifying phrases: Constituency tests,” *Essentials of Linguistics*, 2nd ed., eCampusOntario, 2022. 章節
9. Catherine Anderson et al., “8.4 Conversational implicatures,” *Essentials of Linguistics*, 2nd ed., eCampusOntario, 2022. 章節
10. Bolei Ma, Yuting Li, Wei Zhou, Ziwei Gong, Yang Janet Liu, Katja Jasinskaja, Annemarie Friedrich, Julia Hirschberg, Frauke Kreuter and Barbara Plank, “Pragmatics in the Era of Large Language Models: A Survey on Datasets, Evaluation, Opportunities and Challenges,” *ACL*, pp. 8679–8696, 2025. 論文
11. Catherine Anderson et al., “10.2 Language varies,” *Essentials of Linguistics*, 2nd ed., eCampusOntario, 2022. 章節

12. Libo Qin, Qiguang Chen, Yuhang Zhou, Zhi Chen, Yinghui Li, Lizi Liao, Min Li, Wanxiang Che and Philip S. Yu, "A survey of multilingual large language models," *Patterns*, 6(1), 101118, 2025. 論文
13. Yanru Lu and Stefan Müller, "Deliminative verbal reduplication in Mandarin Chinese," *Journal of Linguistics*, 62(3), pp. 595–632, 2026. 論文
14. A. Seza Doğruöz, Sunayana Sitaram, Barbara E. Bullock and Almeida Jacqueline Toribio, "A Survey of Code-switching: Linguistic and Social Perspectives for Language Technologies," *ACL-IJCNLP*, pp. 1654–1666, 2021. 論文
15. Genta Winata, Alham Fikri Aji, Zheng Xin Yong and Thamar Solorio, "The Decades Progress on Code-Switching Research in NLP: A Systematic Survey on Trends and Challenges," *Findings of ACL*, pp. 2936–2978, 2023. 論文
16. M. F. Porter, "An algorithm for suffix stripping," *Program*, 14(3), pp. 130–137, 1980. DOI
17. Rico Sennrich, Barry Haddow and Alexandra Birch, "Neural Machine Translation of Rare Words with Subword Units," *ACL*, 2016. 論文
18. Mike Schuster and Kaisuke Nakajima, "Japanese and Korean Voice Search," *ICASSP*, pp. 5149–5152, 2012. DOI
19. Yonghui Wu et al., "Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation," arXiv:1609.08144, 2016. 論文
20. Taku Kudo, "Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates," *ACL*, 2018. 論文
21. Taku Kudo and John Richardson, "SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing," *EMNLP System Demonstrations*, 2018. 論文
22. Sabrina J. Mielke et al., "Between Words and Characters: A Brief History of Open-Vocabulary Modeling and Tokenization in NLP," arXiv:2112.10508, 2021. 綜述
23. Hugging Face, "The Tokenization Pipeline," Tokenizers documentation, current online edition. 文件
24. Marie-Catherine de Marneffe, Christopher D. Manning, Joakim Nivre and Daniel Zeman, "Universal Dependencies," *Computational Linguistics*, 47(2), 2021. 論文
25. Universal Dependencies contributors, "Universal POS tags," Universal Dependencies documentation, current online edition. 文件
26. Mitchell P. Marcus, Beatrice Santorini and Mary Ann Marcinkiewicz, "Building a Large Annotated Corpus of English: The Penn Treebank," *Computational Linguistics*, 19(2), 1993. 論文
27. Thomas Emerson, "The Second International Chinese Word Segmentation Bakeoff," *Proceedings of the Fourth SIGHAN Workshop on Chinese Language Processing*, 2005. 論文
28. Jacob Cohen, "A Coefficient of Agreement for Nominal Scales," *Educational and Psychological Measurement*, 20(1), pp. 37–46, 1960. DOI
29. Steven T. Piantadosi, "Zipf's word frequency law in natural language: A critical review and future directions," *Psychonomic Bulletin & Review*, 21(5), pp. 1112–1130, 2014. DOI
30. Jacob Devlin, Ming-Wei Chang, Kenton Lee and Kristina Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *NAACL-HLT*, 2019. 論文

NLP 03・自然語言處理的類神經網路

從計數特徵與邏輯斯分類，走到嵌入層、神經機率語言模型與一維卷積文字分類器，並涵蓋填補、遮罩、正則化與完整的 PyTorch 實驗。

網站版本: https://blog.shyandram.dev/zh-tw/learn/nlp/nlp-03-neural-networks-for-nlp/

你將學到

- 以明確的標註規則定義文字分類任務，並推導計數特徵、TF–IDF、邏輯斯分類與交叉熵。
- 用嵌入層（embedding layer）把 token ID 轉成學習得到的向量，加以池化，再用小型多層感知器分類。
- 理解 Bengio 等人的神經機率語言模型（neural probabilistic language model），並說明共享詞向量為何讓它能推廣到未見過的歷史。
- 寫出一維卷積文字分類器與時間軸最大池化（max-over-time pooling）的數學，並以正確的填補與遮罩實作。
- 對小型 PyTorch 模型做正則化、訓練與提前停止，判讀訓練曲線，並依類別分析錯誤。
- 分開評估決策、機率品質、涵蓋率與類別不平衡。

先看全貌

文字分類器把訊息轉成允許的決策：分流申訴、辨識語言、標記評論，或把文件交給人員檢查。最重要的選擇往往發生在挑選神經網路架構之前。什麼算一個樣本？可以查看哪些脈絡？標籤代表什麼？當所有標籤都不適用時，系統應如何處理？

Hagiwara 在第 2–4 章與第 7 章發展第一個分類應用、詞與文件嵌入、句子分類及卷積編碼器 [1]。McRoy 第 6 章則把情緒、相似度、合文法性與蘊涵分成不同語言任務 [2]。本課結合這兩個方向：建立應用，同時精確定義預測目標。情緒標籤不是蘊涵判斷，相似度分數也不表示兩個陳述具有相同的真值條件。

> **用最簡單的話說**
>
> 想像把信件放進三個托盤：付款、配送與技術協助。計算詞語就像找信封上熟悉的線索。類神經網路則自己學線索：它給每個詞一小串數字，學會哪些短片語重要，再把證據加總。不論哪一種方法，可靠的分類員都需要第四個動作：當信件不明確屬於任何托盤時，詢問其他人。

本課分成兩半。前半建立最強的簡單基準：稀疏特徵加線性分類器，並謹慎地訓練與評估。後半以學習得到的特徵取代人工設計的特徵：**嵌入層**、池化後的多層感知器、神經機率語言模型與卷積編碼器。這裡每個神經模型仍以和線性基準相同的交叉熵損失訓練，因此改變的是表示，而不是目標函數。反向傳播、啟動函數與最佳化器在ML 02 中說明；本頁直接使用它們，專注於文字特有的部分。

所有範例都使用原創訊息，不需要下載資料集。量測的分數只描述這些合成訊息，不能證明系統在真實應用中的表現。

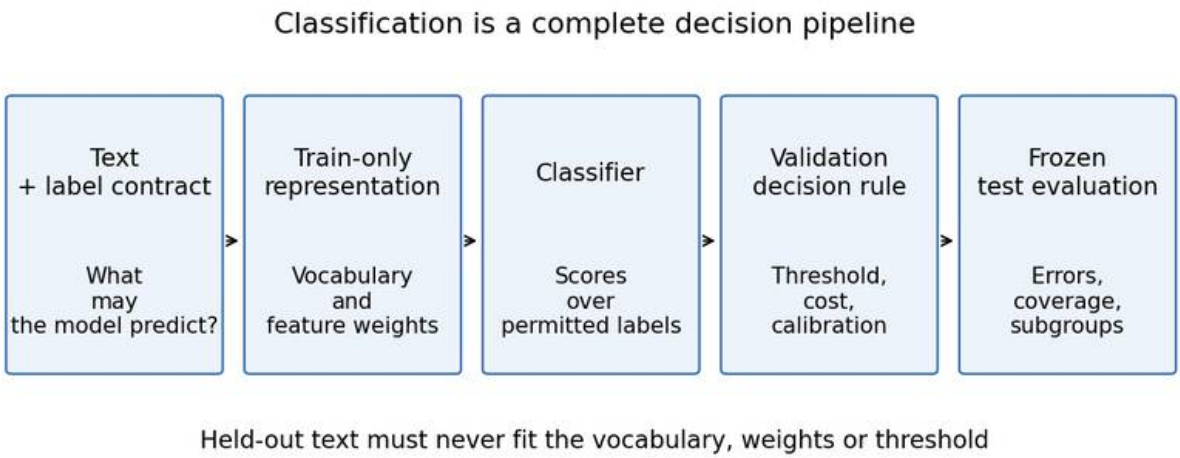


圖 *1* — 分類包含表示、預測與行動規則。保留最終測試集的要求，適用於每個會從資料學習的階段，包括神經特徵學習。

定義預測約定

白話版。標籤是對一段文字在這個應用中代表什麼的共同約定。先建立約定，再蒐集樣本或最佳化損失。

單標籤分類（single-label classification）讓一個輸入從固定集合得到恰好一個標籤。多標籤分類（multi-label classification）允許多個標籤同時成立。一則同時提到包裹延遲與收費錯誤的申訴，可能需要兩條處理路徑、主要路徑加次要標記，或由人員分流。即使輸入文字相同，這些仍是不同任務。

先用一般語言寫出約定。在本課教學應用中，billing 涉及付款或發票，delivery 涉及包裹或配送，technical 涉及帳號登入或軟體行為。分類後允許保留決策。因此，模稜兩可的多主題訊息，不應只因數值模型能計算 argmax，就被默默強迫送往一個類別。

$$f_\theta : \mathcal{X} \to \Delta^{K-1}, \qquad \hat{y}(x) = \operatorname*{argmax}_{k \in \{1,\dots,K\}} p_\theta(y = k \mid x).$$

其中 f_θ 是參數為 θ 的分類器，$\mathcal{X}$ 是允許的文字輸入集合，Δ^{K-1} 是 K 個類別機率向量組成的單純形，x 是一個輸入，y 是標籤，k 索引類別，p_θ 是模型的條件分布，$\hat{y}$ 是機率最高的類別。單純形中的各元素必須非負，且總和為一。

這個式子描述模型輸出，而不是完整的產品行動。服務可以只在信心超過驗證資料所選門檻時回傳路徑，也可以拒絕空白訊息、要求更多脈絡，或把多主題訊息交給人員檢查。評估必須呈現這些行動。

監督單位應與部署情境一致。只用獨立意見句訓練的句子情緒模型，可能無法處理包含引述、修正與多位說話者的完整對話。如果部署可以查看先前回合，標註與訓練也應包含相同的允許脈絡。反過來，標註者如果看得到最後處理結果，但模型只看得到第一則訊息，就可能產生模型根本無法判定的目標。

計數特徵與詞彙表

白話版。最簡單且實用的表示，是記錄出現了哪些詞或短序列。它便宜、容易檢查，也常是很好的第一個比較基準。

只從訓練文字選擇詞彙表。詞彙項目可以是詞、字元序列或相鄰詞對。使用詞彙表時，每個座標計算一個詞彙項目的出現次數。由於大多數可能的詞不會出現在同一份文件中，文件向量通常很稀疏。

$$x_j(d) = c(v_j, d), \qquad \mathbf{x}(d) = (x_1(d), \dots, x_V(d))^\top.$$

其中 d 是文件，v_j 是座標 j 對應的詞彙項目，$c(v_j, d)$ 是它在文件 d 的出現次數，$x_j(d)$ 是該特徵值，$\mathbf{x}(d)$ 是得到的行向量，V 是詞彙表大小。轉置符號 $\top$ 表示列出的座標以直向排列。

詞袋表示（bag of words）忽略順序。「Payment arrived after the refund」與「Refund arrived after the payment」包含相同的詞，卻描述不同事件順序。詞 bigram 能恢復部分局部資訊；字元特徵可以擷取拼字片段與詞形變化，但兩者都不會自動理解長距離依存或否定的作用範圍。

特徵選擇包含對任務的假設。小寫化能減少稀疏性，卻也會合併專有名稱與一般名詞。移除所有停用詞可能刪掉「not」，而它對許多分類很重要。語言辨識器可能更適合使用字元模式，而非為某一種語言設計的詞彙表。每個正規化規則都應有任務上的理由；NLP 02 討論這些選擇背後的語言單位。

在每次實驗中，詞彙表建立、文件頻率門檻與特徵選擇都必須放在訓練部分內。先查看測試文件，再決定保留哪些罕見詞，會讓訓練流程得到評估分布的資訊。即使沒有查看測試標籤，也可能形成不容易察覺的資料洩漏。

記錄詞彙表大小、完全沒有辨識特徵的訊息比例、token 數，以及被刪除詞語的代表範例。這些診斷能揭露單一準確率看不到的變化。新產品代碼或新語言，可能讓大量輸入幾乎變成空向量，卻不會產生明顯的例外錯誤。

TF–IDF 與向量正規化

白話版。幾乎出現在每則訊息中的詞，通常較難區分訊息。TF–IDF 把文件內的出現次數與訓練文件間的稀有程度結合。

本課實作使用的常見平滑定義為：

$$\mathrm{idf}_j = \log \frac{1 + N}{1 + \mathrm{df}_j} + 1, \qquad z_j(d) = c(v_j, d)\,\mathrm{idf}_j, \qquad \tilde{\mathbf{z}}(d) = \frac{\mathbf{z}(d)}{\|\mathbf{z}(d)\|_2}.$$

其中 idf_j 是詞彙項目 v_j 的逆文件頻率權重，N 是訓練文件數，df_j 計算包含該項目的訓練文件數，$c(v_j, d)$ 是它在文件 d 中的次數，$z_j(d)$ 是加權座標，$\mathbf{z}(d)$ 收集這些座標，$\tilde{\mathbf{z}}(d)$ 是長度為一的版本。

歐幾里得範數 $\|\mathbf{z}(d)\|_2$ 是座標平方總和的平方根。零向量維持零，不會除以零。Scikit-learn 特徵擷取文件說明了這些實作選擇 [3]。

加上的一避免已擬合詞彙表中的權重出現未定義情況，也固定了平滑慣例。不同函式庫可能使用不同定義，因此只說「TF–IDF」不足以重現。應記錄斷詞器、特徵範圍、次數轉換、加權公式與正規化方式。

正規化避免長訊息只因反覆用詞就占有較大影響。如果特徵空間中的方向比總長度重要，這通常有幫助。但長度本身也可能具有預測力，例如錯誤報告可能比重設密碼的請求更長。若長度與任務有關，應加入明確的長度特徵，而不是依賴未正規化計數的偶然效果。

TF–IDF 不會推論同義詞。「Charge」與「payment」仍是不同座標，除非樣本讓分類器學到它們扮演相似角色。它也可能給罕見識別碼很大權重，即使稀有程度與任務無關。一個帳號編號可能因相關紀錄共用範本而看似有分類力。依群組切分及檢查係數，有助找出這類捷徑。

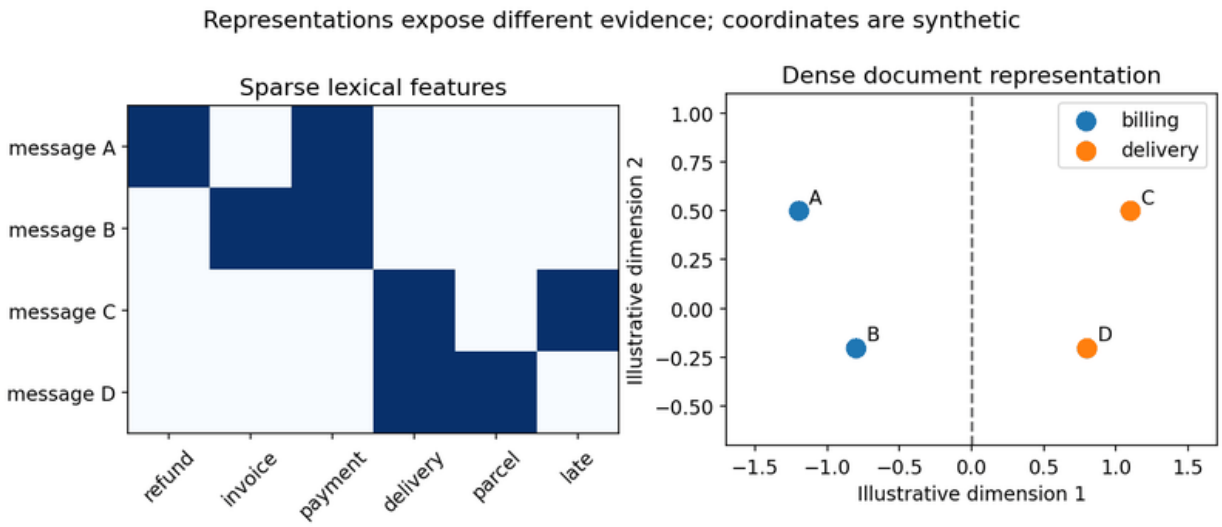


圖 2 — 稀疏特徵保留明確的詞彙證據；稠密表示把模式壓縮到座標中，座標意義取決於訓練目標。

邏輯斯分類與損失

白話版。線性分類器把各特徵的證據相加，為每個標籤產生分數。把分數轉成機率，便得到可微分的訓練目標與決策依據。

多類別模型可以寫成：

$$s_k = \mathbf{w}_k^\top \tilde{\mathbf{z}} + b_k, \qquad p_\theta(y = k \mid d) = \frac{\exp(s_k)}{\sum_{\ell=1}^{K} \exp(s_\ell)}.$$

其中 s_k 是類別 k 的分數，$\mathbf{w}_k$ 是它的學習特徵權重向量，$\tilde{\mathbf{z}}$ 是正規化後的文件表示，b_k 是學習得到的偏置，p_θ 是條件類別機率，θ 包含所有權重與偏置，d 是文件，ℓ 索引競爭類別，K 是類別數，exp 是指數函數。

訓練應提高標註類別的機率。正則化則抑制任意放大的權重，以免模型過度配合偶然出現的詞：

$$\mathcal{L}(\theta) = -\frac{1}{n}\sum_{i=1}^{n} \log p_\theta(y_i \mid d_i) + \frac{\lambda}{2}\sum_{k=1}^{K} \|\mathbf{w}_k\|_2^2.$$

其中 $\mathcal{L}$ 是目標函數，n 是訓練樣本數，i 索引樣本，(d_i, y_i) 是已標註文件與標籤，$\lambda \geq 0$ 是正則化強度，$\|\mathbf{w}_k\|_2^2$ 是類別 k 特徵權重的平方總和。第一項是多類別交叉熵，第二項是平方權重正則化。函式庫可能採用不同縮放；scikit-learn 以參數 C 表示反向的正則化控制 [4]。IT 01 說明為何第一項是交叉熵。

很大的正係數表示該特徵會相對於其他證據提高類別分數，卻不表示該特徵造成真實類別。相關詞可能分攤或交換權重；改變詞彙表，也會改變係數比較的意義。

二元分類的 softmax 可以改寫成分數差的一個 sigmoid。獨立的多標籤預測則應為每個標籤使用一個 sigmoid，而不是強迫標籤競爭的 softmax。輸出啟動函數與損失必須配合標註約定，而不是配合最容易寫出的模型頭。

把這個模型當成網路來看，它就是一個線性層接著 softmax。下面每個神經分類器都保留這個最後一層與它的損失。改變的是向量 $\tilde{\mathbf{z}}$：它不再由固定的計數規則算出，而是由可學習的層計算。

完整的 CPU 分類實驗

白話版。詞彙表、分類器與門檻必須從指定的資料部分學習。使用 pipeline 能更容易維持這個邊界。

下列獨立範例使用原創訊息，明確分開訓練、驗證與測試文字。驗證階段依示範成本選門檻：一次錯誤分流的成本為兩個單位，要求人工檢查的成本為五分之一個單位。這些數值是教學政策參數，不是量測出的商業成本。在查看最終測試結果之前，凍結所有選擇。

```
import numpy as np
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, log_loss

train = [
    ("please refund my payment", "billing"),
    ("the invoice has a wrong charge", "billing"),
    ("my payment was charged twice", "billing"),
    ("how can i update the billing card", "billing"),
    ("please send the payment receipt", "billing"),
    ("the refund is missing from my invoice", "billing"),
    ("where is my delivery parcel", "delivery"),
    ("my shipment is late", "delivery"),
    ("please change the delivery address", "delivery"),
    ("the parcel arrived damaged", "delivery"),
    ("how do i track the shipment", "delivery"),
    ("the delivery driver could not find the address", "delivery"),
    ("i cannot login to my account", "technical"),
    ("the application crashes on startup", "technical"),
```

```
    ("please reset my account password", "technical"),
    ("the software shows an error", "technical"),
    ("my login fails after the software update", "technical"),
    ("the application cannot connect to my account", "technical"),
]
validation = [
    ("i need a receipt for the charge", "billing"),
    ("the billing payment needs a refund", "billing"),
    ("can i track this delivery", "delivery"),
    ("the shipment went to another address", "delivery"),
    ("my password does not allow login", "technical"),
    ("the software crashes with an error", "technical"),
]
test = [
    ("please explain the invoice payment", "billing"),
    ("a charge appears before the refund", "billing"),
    ("the parcel delivery is not here", "delivery"),
    ("my shipment arrived at the wrong address", "delivery"),
    ("an account error prevents login", "technical"),
    ("the application crashes after a password change", "technical"),
]

def unpack(rows):
    return [text for text, _ in rows], np.array([label for _, label in rows])

train_x, train_y = unpack(train)
val_x, val_y = unpack(validation)
test_x, test_y = unpack(test)
model = Pipeline([
    ("features", TfidfVectorizer(ngram_range=(1, 2))),
    ("classifier", LogisticRegression(C=4.0, max_iter=300, random_state=0)),
])
model.fit(train_x, train_y)
classes = model.named_steps["classifier"].classes_

def decision(probabilities, threshold):
    best = probabilities.argmax(axis=1)
    prediction = classes[best]
    accepted = probabilities[np.arange(len(best)), best] >= threshold
    return prediction, accepted

val_prob = model.predict_proba(val_x)
grid = [0.40, 0.50, 0.60, 0.70, 0.80]
costs = []
for threshold in grid:
    pred, accepted = decision(val_prob, threshold)
    wrong = accepted & (pred != val_y)
    costs.append((2.0 * wrong.sum() + 0.2 * (~accepted).sum()) / len(val_y))
threshold = grid[int(np.argmin(costs))]  # first minimum fixes the tie rule

test_prob = model.predict_proba(test_x)
test_pred, accepted = decision(test_prob, threshold)
print("validation-selected threshold:", threshold)
print("test macro F1:", round(f1_score(test_y, test_pred, average="macro"), 3))
```

```
print("test log loss:", round(log_loss(test_y, test_prob, labels=classes), 3))
print("test coverage:", round(accepted.mean(), 3))
if accepted.any():
    print("accepted accuracy:", round((test_pred[accepted] == test_y[accepted]).mean(), 3))
else:
    print("accepted accuracy: undefined (no accepted messages)")
probe = ["the weather is pleasant", "my delivery charge is wrong"]
for text, probabilities in zip(probe, model.predict_proba(probe)):
    prediction, keep = decision(probabilities[None, :], threshold)
    print(text, "->", prediction[0] if keep[0] else "review")
```

執行結果：

```
validation-selected threshold: 0.4
test macro F1: 1.0
test log loss: 0.413
test coverage: 1.0
accepted accuracy: 1.0
the weather is pleasant -> delivery
my delivery charge is wrong -> delivery
```

六則完全分對的測試訊息遠不足以支持任何關於方法的主張；它們只顯示流程能從頭跑到尾。驗證集讓高於 0.40 的每個門檻都顯得成本較高，因此選中網格中最低的值，而這個政策會接受所有訊息。

不熟悉的天氣訊息才是有用的部分。三個路徑都不正確，模型卻仍回傳機率分布，而且這裡最大的機率（delivery 約 0.51）通過了所選門檻。輸出頭沒有代表任意外部主題的額外類別。保留決策的門檻可能減少部分不適當行動，但最大機率不是有保證的領域外偵測器，在這次執行中它完全沒有攔下這則訊息。混合主題的「delivery charge」訊息同樣被強迫送往單一路徑。

注意，最終 macro F1 量測的是所有測試樣本被強制分類的結果；涵蓋率與接受樣本準確率則描述行動規則。只報接受樣本準確率，可能隱藏拒絕幾乎所有輸入的模型；只報強制分類準確率，可能隱藏有用的選擇性行為。兩種觀點與分母都應清楚呈現。

真實應用應把這些教學清單換成有文件說明與群組識別碼的語料。同一對話的訊息不應隨機分散到訓練與測試中。共享範本、重複簽名與近似描述也需要檢查。本例刻意呈現運作機制；驗證樣本太少，無法穩定估計門檻。

控制條件的否定測試

改變特徵詞彙表，可以保留無序詞頻會遺失的部分資訊。以下固定自行設計的標註語料與分類器種類，比較單一詞元特徵，以及單一詞元加上相鄰詞對的特徵。TF–IDF 與邏輯斯迴歸實作的說明見 [3,4]。

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.pipeline import make_pipeline

train = [
    ("the battery is good", 1), ("good screen and good sound", 1),
    ("the camera is great", 1), ("great value , works well", 1),
    ("the battery is not good", 0), ("not great , the screen broke", 0),
    ("the sound is bad", 0), ("bad camera and bad battery", 0),
    ("works well , not bad at all", 1), ("not good , not worth it", 0),
]
test = ["the screen is not good", "the sound is not bad", "great battery"]
texts, labels = zip(*train)

for name, ngrams in [("unigrams", (1, 1)), ("uni+bigrams", (1, 2))]:
    clf = make_pipeline(TfidfVectorizer(ngram_range=ngrams, token_pattern=r"\S+"),
                        LogisticRegression(C=10.0))
    clf.fit(texts, labels)                    # vocabulary + IDF fitted on train only
    probs = clf.predict_proba(test)[:, 1]
    print(f"{name:12s}", [f"{t!r}: {p:.2f}" for t, p in zip(test, probs)])
```

正面評論的預測機率為：

```
unigrams     ["'the screen is not good': 0.43", "'the sound is not bad': 0.11", "'great
 ↪  battery': 0.61"]
uni+bigrams  ["'the screen is not good': 0.20", "'the sound is not bad': 0.18", "'great
 ↪  battery': 0.56"]
```

兩個模型都把 “the screen is not good” 的機率判在二分之一以下；加入詞對後，機率從 0.43 降到 0.20。“the sound is not bad” 的機率則從 0.11 升到 0.18，卻仍低於二分之一。這些觀察確立的是已擬合玩具模型的行為，無法單獨認定某個係數就是原因：加入詞對也改變了詞彙表、正規化與共同擬合的權重。

第二句是有用的錯誤探針：模型已有局部片語特徵，卻仍對預期的正面解讀給出錯誤標籤。語料中只有一個正面 “not bad” 範例，而且同時包含其他正面用語。這麼稀少的證據，無法充分確定究竟是哪個特徵承載標籤。在對架構下普遍結論之前，應檢查各特徵的貢獻，並加入固定主題、只改變否定的控制範例。

片語是否出現，也不同於否定的作用域。相鄰特徵可以區分 “not good” 與 “good”，但辨識 “not every repair succeeded” 與 “every repair failed” 需要另一種組合差異。增加詞對不會自動形成完整語意分析。NLP 02 與NLP 12 會明確分析這些情況。

稀疏特徵的兩個限制，引出本課其餘內容。第一，每個 bigram 都是獨立座標：除非 “not good” 與 “not great” 都帶著標籤出現在訓練資料中，否則兩者毫無共享。第二，可能的 n-gram 數量隨 n 快速增加，所以長模式很少被觀察到。神經模型同時處理這兩點：把詞表示成能共享證據的向量，並學習哪些向量組合重要。

從詞 ID 到學習得到的向量：嵌入層

白話版。給每個詞彙項目一串短短的、可學習的數字。在訓練任務中行為相似的詞，可能得到相似的向量，於是關於一個詞的證據也能幫助另一個詞。

嵌入層是一張表，每個詞彙項目占一列。查詢一個 token，等同於把它的 one-hot 向量乘上這張表：

$$\mathbf{e}_t = E^\top \mathbf{o}_{i_t} = E^\top_{i_t,:}, \qquad E \in \mathbb{R}^{V \times D}.$$

其中 i_t 是位置 t 上 token 的詞彙 ID，$\mathbf{o}_{i_t} \in \{0,1\}^V$ 是只在座標 i_t 為一的 one-hot 向量，E 是有 V 列、D 行的可學習嵌入表，$E_{i_t,:}$ 是第 i_t 列，$\mathbf{e}_t \in \mathbb{R}^D$ 是該 token 的嵌入（寫成行向量）。D 通常遠小於 V，這就是嵌入被稱為**稠密**（dense）的原因：和稀疏計數向量不同，它大多數座標都不為零。

查表對訓練有一個實用的後果。損失對 E 的梯度只在目前批次出現過的 token 那幾列不為零，所以每次更新只動到少數幾列。從未在訓練中出現的詞會保留初始值，這也是詞彙表需要明確的未知詞項目以及使用規則的原因。這張表可以隨機初始化並與分類器一起訓練，也可以用在未標註文字上預訓練的向量初始化；NLP 04 說明這些向量如何學得，以及它們的幾何意義與限制。

先池化再分類：詞嵌入袋 MLP

文件的 token 數不固定，分類層卻需要固定寬度的輸入。最簡單的神經做法，是把 token 向量平均，再送進**多層感知器**（multilayer perceptron, MLP）：

$$\mathbf{h} = \frac{1}{T}\sum_{t=1}^{T} \mathbf{e}_t, \qquad \mathbf{a} = \phi\left(W_1 \mathbf{h} + \mathbf{b}_1\right), \qquad \mathbf{s} = W_2 \mathbf{a} + \mathbf{b}_2, \qquad p_\theta(y = k \mid d) = \operatorname{softmax}(\mathbf{s})_k.$$

其中 T 是文件 d 中真實 token 的數量，$\mathbf{h} \in \mathbb{R}^D$ 是平均池化後的文件向量，$W_1 \in \mathbb{R}^{H \times D}$ 與 $\mathbf{b}_1 \in \mathbb{R}^H$ 是隱藏層的權重與偏置，H 是隱藏寬度，ϕ 是逐元素的非線性函數（例如 ReLU），$\mathbf{a}$ 是隱藏層啟動值，$W_2 \in \mathbb{R}^{K \times H}$ 與 $\mathbf{b}_2 \in \mathbb{R}^K$ 產生類別分數 $\mathbf{s}$，而 $\operatorname{softmax}(\mathbf{s})_k = \exp(s_k) / \sum_\ell \exp(s_\ell)$。參數 $\theta = (E, W_1, \mathbf{b}_1, W_2, \mathbf{b}_2)$ 以和邏輯斯模型相同的交叉熵共同訓練，梯度由反向傳播計算。

兩個已發表的設計顯示這個簡單形狀能走多遠。Joulin 等人把詞與 n-gram 嵌入平均後送進線性分類器，以深度換取速度 [7]。Iyyer 等人的深度平均網路（deep averaging network）在平均之上堆疊數個非線性層，並加入**詞丟棄**（word dropout），在訓練時先移除整個 token 再平均 [6]。Goldberg 的入門綜述詳細說明這類前饋文字模型 [5]。

隱藏層很重要。對平均向量的線性模型只能把每個詞的證據相加，因此無法表示「*good* 是正面，除非出現 *not*」這種交互作用。非線性隱藏層可以把「not」的出現與正面詞的出現組合起來。但任何以池化為基礎的模型都無法恢復順序：token 的任何排列都給出相同的平均。“The screen is good but the case is bad” 與 “the case is good but the screen is bad” 是完全相同的輸入。本課後面的實驗會量測這個失敗。

神經機率語言模型

白話版。語言模型根據前面的詞預測下一個詞。Bengio 等人證明，若每個先前的詞都以學習得到的向量表示，網路就能在相似的歷史之間共享證據，並為從未見過的詞序列給出合理的機率。

語言模型（language model）一次預測一個 token，藉此為序列指定機率。以 w_t 表示位置 t 的 token，連鎖律給出

$$P(w_1, \dots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_1, \dots, w_{t-1}).$$

其中 $P(w_1, \dots, w_T)$ 是整個序列的機率，$P(w_t \mid w_1, \dots, w_{t-1})$ 是在前面所有 token 已知時下一個 token 的條件機率。計數式 n-gram 模型只用前 $n-1$ 個 token 近似每個因子，並以計數估計；NLP 05 連同平滑與困惑度一起說明它們。它們的弱點是歷史 “a dog” 與 “a cat” 毫無共享：如果前者從未出現，計數對它就沒有任何資訊。

Bengio 等人保留固定的 $(n-1)$ 個 token 歷史，但以神經網路取代計數表 [8]。每個歷史詞都在共享的表 C 中查詢，向量串接後，由一個隱藏層為每個詞彙表中的詞產生分數：

$$\mathbf{x} = [C(w_{t-1}); C(w_{t-2}); \dots; C(w_{t-n+1})], \qquad \mathbf{y} = \mathbf{b} + W\mathbf{x} + U \tanh(\mathbf{d} + H\mathbf{x}), \qquad \hat{P}(w_t = i \mid w_{t-n+1}^{t-1}) = \frac{\exp(y_i)}{\sum_{j=1}^{V} \exp(y_j)}.$$

其中 $C(w) \in \mathbb{R}^m$ 是詞 w 學習得到的特徵向量（$V \times m$ 表 C 的一列），分號表示串接，因此 $\mathbf{x} \in \mathbb{R}^{(n-1)m}$；$H \in \mathbb{R}^{h \times (n-1)m}$ 與 $\mathbf{d} \in \mathbb{R}^h$ 是有 h 個隱藏單元的隱藏層權重與偏置；$U \in \mathbb{R}^{V \times h}$ 把隱藏單元對應到詞彙分數；$W \in \mathbb{R}^{V \times (n-1)m}$ 是從詞特徵到輸出的選用**直接連接**（direct connection），不使用時設為零；$\mathbf{b} \in \mathbb{R}^V$ 是輸出偏置；$\mathbf{y}$ 是未正規化的對數機率；V 是詞彙表大小；w_{t-n+1}^{t-1} 是歷史的縮寫。這就是論文的式 (1)，只是改用本課符號。訓練目標是最大化訓練 token 的平均對數概似，再加上權重衰減懲罰 [8]。

這個模型中的三個想法會在本課程後面反覆出現。詞向量是預測任務的副產品。和平均不同，串接保留了每個歷史詞的位置，因此 “dog bites” 與 “bites dog” 不同。而整個詞彙表上的 softmax 是昂貴的部分：每預測一個 token，其成本都隨 V 增加，這後來促成了較便宜的目標函數，例如NLP 04 討論的負取樣。

下面的範例在原創語料上訓練一個極小版本，語料中從未出現歷史 “a dog”。為了方便，我們使用 Adam，而不是論文的隨機梯度程序。

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
corpus = """a cat sleeps . a cat eats . the cat sleeps . the cat eats .
the dog sleeps . the dog eats . a bird sings . the bird sings .
a bird eats . the bird eats""".split(".")      # the history "a dog" never occurs
sentences = [["<s>", "<s>"] + s.split() + ["</s>"] for s in corpus]
vocab = sorted({w for s in sentences for w in s})
ids = {w: i for i, w in enumerate(vocab)}
X, Y = [], []
```

```
for s in sentences:
    for t in range(2, len(s)):                  # predict w_t from (w_{t-2}, w_{t-1})
        X.append([ids[s[t - 2]], ids[s[t - 1]]])
        Y.append(ids[s[t]])
X, Y = torch.tensor(X), torch.tensor(Y)

class NPLM(nn.Module):
    def __init__(self, V, m=8, h=16, context=2, direct=True):
        super().__init__()
        self.C = nn.Embedding(V, m)                     # shared word features
        self.H = nn.Linear(context * m, h)              # hidden layer (bias = d)
        self.U = nn.Linear(h, V)                        # hidden-to-output (bias = b)
        self.W = nn.Linear(context * m, V, bias=False) if direct else None
    def forward(self, ctx):
        x = self.C(ctx).flatten(1)                      # concatenate the context vectors
        y = self.U(torch.tanh(self.H(x)))
        return y + self.W(x) if self.W is not None else y

model = NPLM(len(vocab))
opt = torch.optim.Adam(model.parameters(), lr=0.02, weight_decay=1e-4)
for step in range(400):
    opt.zero_grad()
    loss = F.cross_entropy(model(X), Y)
    loss.backward()
    opt.step()
print(f"training cross-entropy {loss.item():.3f} nats, {len(Y)} predictions")

history_count = sum(s[i:i + 2] == ["a", "dog"] for s in sentences for i in range(len(s)))
with torch.no_grad():
    p = F.softmax(model(torch.tensor([[ids["a"], ids["dog"]]])), dim=-1)[0]
print("training occurrences of the history 'a dog':", history_count)
for w in ["sleeps", "eats", "sings"]:
    print(f"P({w} | a dog) = {p[ids[w]].item():.3f}")
```

執行結果：

```
training cross-entropy 0.576 nats, 40 predictions
training occurrences of the history 'a dog': 0
P(sleeps | a dog) = 0.509
P(eats | a dog) = 0.490
P(sings | a dog) = 0.001
```

三元計數模型對歷史 “a dog” 沒有資料，必須退回較短的歷史。神經模型則把幾乎所有機率放在狗（以及貓）在訓練中會搭配的兩個動詞上，而幾乎不給只有鳥會做的 “sings”。它能推廣，是因為 “dog” 與 “cat” 因共享上下文而得到相關的向量，“a” 與 “the” 也是如此。這正是 Bengio 等人的核心主張，在此以玩具規模重現；它完全無法說明模型在真實文字上與平滑良好的 n-gram 模型相比如何，而後者才是論文實際量測的內容 [8]。

卷積文字分類器

白話版。卷積濾波器沿著句子滑動，只要看到某種短片語模式就會被激發，不論模式出現在哪裡。取每個濾波器最強的回應，就得到固定大小的摘要：它記得哪些模式出現過，卻不記得在哪裡出現。

Collobert 等人建立了一個用於標註任務的統一神經網路，其中對詞向量做卷積、再沿位置做最大池化，以摘要整個句子 [9]。Kim 證明，在預訓練詞向量之上只用一層卷積，就是很強的句子分類器 [10]。卷積本身的權重共享與池化，在ML 03 中以影像說明；文字使用一維版本，只沿位置滑動。

一個濾波器的數學

把句子的 T 個 token 嵌入堆成矩陣 $X \in \mathbb{R}^{T \times D}$，第 t 列為 $\mathbf{e}_t^\top$。寬度為 h 的濾波器涵蓋 h 個連續的列：

$$c_t = \rho(\langle W, X_{t:t+h-1} \rangle + b), \qquad t = 1, \dots, T-h+1, \qquad \hat{c} = \max_t c_t.$$

其中 $X_{t:t+h-1} \in \mathbb{R}^{h \times D}$ 是第 t 到 $t+h-1$ 列組成的視窗，$W \in \mathbb{R}^{h \times D}$ 是濾波器的學習權重，$\langle A, B \rangle = \sum_{r,s} A_{rs} B_{rs}$ 為逐元素乘積的總和，$b \in \mathbb{R}$ 是偏置，ρ 是 ReLU 或 tanh 之類的非線性函數，c_t 是濾波器在視窗 t 的回應，序列 $(c_1, \dots, c_{T-h+1})$ 是**特徵圖**（feature map），$\hat{c}$ 是**時間軸最大池化**的結果。Kim 把視窗攤平成長度 hD 的向量，寫出相同的運算 [10]。

模型會使用許多濾波器。若每個寬度（例如 $\{3, 4, 5\}$）各有 F 個濾波器，池化後的值組成向量 $\mathbf{z} \in \mathbb{R}^{F \cdot (\text{number of widths})}$，再由線性層與 softmax 轉成類別機率，和前面一樣以交叉熵訓練。在 PyTorch 中，`nn.Conv1d(D, F, h)` 一次算出寬度 h 的全部 F 張特徵圖；它要求輸入形狀為 (batch, D, T)，因此嵌入輸出必須先轉置。

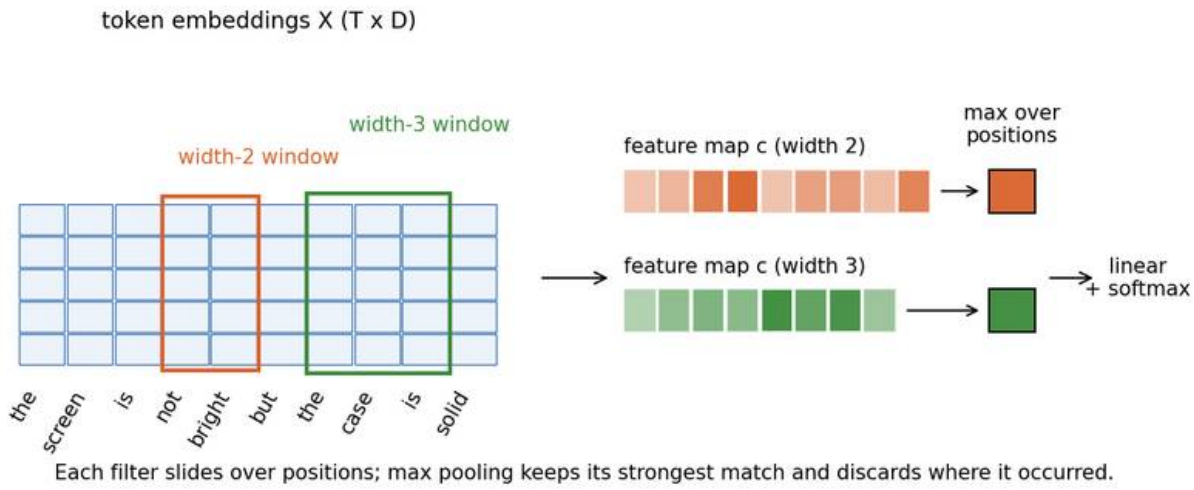


圖 3 — 一維卷積文字分類器。每個濾波器為其寬度的每個視窗打分；時間軸最大池化保留最強分數並丟棄它的位置。

各部分帶來什麼，又失去什麼

權重共享。同一個濾波器套用在每個位置，因此從一句開頭的 “not good” 學到的模式，也能在另一句的結尾偵測到。參數量取決於 h、D 與 F，與句子長度無關。

軟性 n-gram。由於濾波器是與嵌入做內積，對 “not good” 有反應的濾波器，若兩個形容詞向量相近，也會對 “not great” 產生較弱的反應。這是相對於稀疏 bigram 特徵的優勢。

最大池化。只保留最大值能處理不同句長，並記錄某模式是否在任何地方出現過。它丟掉了模式出現的位置與次數。單層濾波器也無法連接距離超過其寬度的 token：寬度最多為 4 時，模型中沒有任何視窗能同時看到 “but” 與四個位置之後的形容詞。下面的實驗正好顯示這一點。

Kim 的論文比較四種處理嵌入表的方式：隨機初始化並與模型一起訓練（CNN-rand）、預訓練向量保持固定（CNN-static）、預訓練向量加以微調（CNN-non-static），以及兩個通道、一個固定一個微調、每個濾波器同時套用在兩者上（CNN-multichannel）[10]。他回報的設定是濾波器寬度 3、4、5 各 100 張特徵圖、池化層的丟棄率 0.5、輸出權重向量的 ℓ_2 範數上限為 3，以及大小 50 的小批次 [10]。Zhang 與 Wallace 後來對這種單層設計做了敏感度分析，結論是濾波器區域大小可能影響很大且應該調整、1-max 池化在他們試過的池化策略中一致表現最好、ReLU 與 tanh 是最佳的啟動函數候選，而比較設定時必須報告不同執行之間的變異 [11]。

循環編碼器（recurrent encoder）走另一條處理順序的路：一邊讀序列一邊更新隱藏狀態，原則上能連接遠距離的 token。Hagiwara 的句子分類內容結合循環模型與實務訓練 [1]。NLP 05 針對語言發展它們，ML 03a 說明共通數學。以注意力為基礎的上下文編碼器讓每個 token 的表示取決於整個輸入；NLP 07 與NLP 10 會發展它們。

選擇應以證據為依據。如果標籤有清楚的詞彙線索，稀疏分類器可能已足夠。卷積或上下文編碼器可能改善間接說法、否定或領域詞義；更大的容量也讓資料切分與標註品質更重要。複雜度應回答已量測的問題，例如改述句的錯誤，而不是取代評估。

填補、遮罩與批次

白話版。句子長短不一，批次卻是矩形。我們用填補 token 補滿短句，再確保填補永遠不影響結果。

訓練時一次處理多個序列，組成形狀為（batch, $T_{\max}$）的張量，$T_{\max}$ 是批次中最長的長度。較短的序列以保留的**填補 ID**（padding ID，通常為 0）補齊。布林**遮罩**（mask）m_{bt} 記錄哪些位置是真實的：

$$m_{bt} = \mathbf{1}[t \le T_b], \qquad \mathbf{h}_b = \frac{\sum_{t=1}^{T_{\max}} m_{bt}\,\mathbf{e}_{bt}}{\max\!\left(1, \sum_{t=1}^{T_{\max}} m_{bt}\right)}.$$

其中 b 索引批次中的序列，t 索引位置，T_b 是序列 b 的真實長度，$\mathbf{1}[\cdot]$ 在條件成立時為一、否則為零，$\mathbf{e}_{bt}$ 是序列 b 第 t 個位置的嵌入，$\mathbf{h}_b$ 是**遮罩平均**。$\max(1, \cdot)$ 防止空序列。每種池化運算都需要自己的遮罩規則：

- **平均池化**必須除以真實長度，而不是 $T_{\max}$。零填補向量不改變總和，卻會改變未遮罩的分母。

- **最大池化**必須排除填補位置或視窗。一般規則是在取最大值前填入 $-\infty$。經過 ReLU 後所有回應都非負，所以填入 0 是等價的，還多一個好處：比濾波器寬度還短的句子完全沒有有效視窗，此時得到 0 而不是 $-\infty$，否則下一層會變成 NaN。
- 與填補重疊的**卷積視窗**不是真實視窗。採右側填補時，從位置 t 開始、寬度 h 的視窗，當且僅當位置 $t + h - 1$ 是真實位置時才有效。
- 針對 token 位置的**損失**（之後的標註與語言模型會用到）必須忽略填補位置；PyTorch 的交叉熵為此提供 `ignore_index`。

PyTorch 的 `nn.Embedding(..., padding_idx=0)` 讓填補列保持為零，並把它排除在梯度更新之外。這有幫助，但不會自動讓平均或最大值正確。檢驗方法是一個不變性：多加填補不能改變任何輸出。

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
emb = nn.Embedding(6, 4, padding_idx=0)            # ID 0 is reserved for padding
seqs = [torch.tensor([3, 5, 2]), torch.tensor([4, 1])]
x = nn.utils.rnn.pad_sequence(seqs, batch_first=True, padding_value=0)
print(x.tolist())                                   # [[3, 5, 2], [4, 1, 0]]

# A lookup is a one-hot row times the table.
one_hot = F.one_hot(x, num_classes=6).float()
assert torch.allclose(one_hot @ emb.weight, emb(x))

def masked_mean(ids):
    mask = (ids != 0).unsqueeze(-1).float()
    return (emb(ids) * mask).sum(1) / mask.sum(1).clamp(min=1)

longer = F.pad(x, (0, 3), value=0)                  # three more padding columns
print("masked mean unchanged:", torch.allclose(masked_mean(x), masked_mean(longer)))
print("plain mean unchanged: ", torch.allclose(emb(x).mean(1), emb(longer).mean(1)))
print("padding row:", emb.weight[0].tolist())
```

執行結果：

```
[[3, 5, 2], [4, 1, 0]]
masked mean unchanged: True
plain mean unchanged:  False
padding row: [0.0, 0.0, 0.0, 0.0]
```

填補列為零，普通平均卻仍然改變，因為它的分母把填補也算了進去。任何新的編碼器都應加入這類不變性檢查。

批次在正確性之外還有實務後果。填補浪費計算，因此訓練程式常把長度相近的序列放進同一批次（**分桶**，bucketing）。過長的輸入會被截斷到最大長度；請記錄這個上限，因為超出部分模型看不到，並決定要保留開頭、結尾或兩者。每個 epoch 都要打亂批次，讓模型不會看到相同的樣本順序；評估時要把模型切到評估模式，以關閉 dropout。

小型文字分類器的正則化

白話版。神經文字模型的參數足以背下小型訓練集，連其中的標註錯誤也一起背下。正則化讓背誦變難，而驗證集告訴我們何時該停。

這些工具就是ML 02 中的通用工具，只是用在文字模型的特定位置：

- **Dropout** 在訓練時以機率 p 把一層中的每個單元設為零，並把其餘單元重新縮放，讓任何單元都不能依賴特定夥伴 [12]。在文字 CNN 中通常套用在池化後的特徵向量上，如 Kim 的模型 [10]。PyTorch 的 `nn.Dropout(p)` 採用的慣例是：p 是丟棄單元的機率。
- **詞丟棄**在訓練時從輸入移除整個 token，促使模型不依賴任何單一詞 [6]。
- **權重衰減**（weight decay）在損失中加入 $\frac{\lambda}{2}\|\theta\|_2^2$，與上面邏輯斯目標的懲罰相同。嵌入表常是最大的參數區塊，因此它是否被衰減或凍結很重要。
- **最大範數限制**（max-norm constraint）在權重向量的歐幾里得範數超過上限時把它重新縮放，如 Kim 的輸出層 [10]。
- **提前停止**（early stopping）保留驗證損失最佳那個 epoch 的參數。它本身也是一種正則化：限制最佳化朝背下訓練集的方向走多遠。
- **凍結預訓練嵌入**直接移除最大的一塊可訓練參數，代價是無法讓詞義適應任務。

這些選擇都是在驗證資料上選定的超參數。一次只改一項，並在下結論說某選擇有幫助之前，報告不同隨機種子間的變化；Zhang 與 Wallace 對文字 CNN 正是強調這一點 [11]。

完整的 PyTorch 實驗

白話版。現在在一個順序確實重要的原創評論任務上比較四個分類器，判讀它們的訓練曲線，並依類別檢查錯誤。

每則合成評論要不是一個子句 "the *aspect* is (*not*) *adjective*"，就是以 "but" 連接的兩個子句。當決定性子句為正面時，標籤為正：否定會翻轉形容詞，而在 "but" 評論中由第二個子句決定。訓練集有 300 則評論，其中 10% 的標籤被刻意翻轉以模擬標註雜訊；驗證集（300 則）與測試集（1,000 則）的標籤是乾淨的。每則測試評論都標記為 `plain`、`negated` 或 `contrast`，以便依類別計算錯誤。

```
import copy, random
import numpy as np
import torch
from torch import nn
from torch.nn import functional as F
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

torch.set_num_threads(1)
ASPECTS = ["battery", "screen", "camera", "speaker", "case", "charger"]
POS, NEG = ["good", "great", "solid", "bright"], ["bad", "weak", "noisy", "dull"]

def clause(rng):
    aspect, positive = rng.choice(ASPECTS), rng.random() < 0.5
```

```
    negated = rng.random() < 0.4
    words = ["the", aspect, "is"] + ["not"] * negated + [rng.choice(POS if positive else
     ↪  NEG)]
    return words, int(positive != negated), negated

def example(rng):                       # label of "A but B" is the label of B
    words, label, negated = clause(rng)
    if rng.random() < 0.5:
        second, label, _ = clause(rng)
        return " ".join(words + ["but"] + second), label, "contrast"
    return " ".join(words), label, "negated" if negated else "plain"

def make(n, seed):
    rng = random.Random(seed)
    return [example(rng) for _ in range(n)]

train, val, test = make(300, 1), make(300, 2), make(1000, 3)
flip = random.Random(4)                 # 10% training-label noise; val/test stay clean
train = [(t, 1 - y if flip.random() < 0.1 else y, c) for t, y, c in train]
y_test, kind = np.array([r[1] for r in test]), np.array([r[2] for r in test])

def report(name, pred):
    parts = {k: round(float((pred[kind == k] == y_test[kind == k]).mean()), 3)
             for k in ["plain", "negated", "contrast"]}
    print(f"{name:12s} test acc {np.mean(pred == y_test):.3f}", parts)

for ngrams in [(1, 1), (1, 2)]:
    vec = TfidfVectorizer(ngram_range=ngrams, token_pattern=r"\S+")
    clf = LogisticRegression(C=10.0, max_iter=1000)
    clf.fit(vec.fit_transform([r[0] for r in train]), [r[1] for r in train])
    report(f"tfidf {ngrams}", clf.predict(vec.transform([r[0] for r in test])))

vocab = ["<pad>", "<unk>"] + sorted({w for r in train for w in r[0].split()})
ids = {w: i for i, w in enumerate(vocab)}

def encode(rows):                       # right-pad every sequence with ID 0
    seqs = [torch.tensor([ids.get(w, 1) for w in r[0].split()]) for r in rows]
    return (nn.utils.rnn.pad_sequence(seqs, batch_first=True, padding_value=0),
            torch.tensor([r[1] for r in rows]))

class MeanMLP(nn.Module):               # embedding -> masked mean -> MLP
    def __init__(self, V, D=16, H=32, p=0.3):
        super().__init__()
        self.emb = nn.Embedding(V, D, padding_idx=0)
        self.mlp = nn.Sequential(nn.Linear(D, H), nn.ReLU(), nn.Dropout(p), nn.Linear(H, 2))
    def forward(self, x):
        m = (x != 0).unsqueeze(-1).float()
        return self.mlp((self.emb(x) * m).sum(1) / m.sum(1).clamp(min=1))

class TextCNN(nn.Module):               # embedding -> 1-D conv -> masked max -> linear
    def __init__(self, V, D=16, widths=(2, 3, 4), maps=16, p=0.3):
        super().__init__()
        self.emb = nn.Embedding(V, D, padding_idx=0)
        self.convs = nn.ModuleList(nn.Conv1d(D, maps, h) for h in widths)
```

```
        self.drop, self.out = nn.Dropout(p), nn.Linear(maps * len(widths), 2)
    def forward(self, x):
        e, real = self.emb(x).transpose(1, 2), x != 0         # e: (batch, D, T)
        pooled = []
        for conv in self.convs:
            c = torch.relu(conv(e))                           # (batch, maps, T-h+1)
            valid = real[:, conv.kernel_size[0] - 1:].unsqueeze(1)  # window ends on a real
             ↪  token
            # ReLU outputs are >= 0, so filling invalid windows with 0 never wins the max,
            # and a text shorter than the filter gets 0 instead of -inf (which would give
             ↪  NaN).
            pooled.append(c.masked_fill(~valid, 0.0).max(dim=2).values)
        return self.out(self.drop(torch.cat(pooled, dim=1)))

def fit(model, weight_decay, epochs=80, batch=32):
    (xt, yt), (xv, yv) = encode(train), encode(val)
    opt = torch.optim.Adam(model.parameters(), lr=0.01, weight_decay=weight_decay)
    best, history = (float("inf"), None, 0), []
    for epoch in range(1, epochs + 1):
        model.train()
        for b in torch.randperm(len(yt)).split(batch):
            opt.zero_grad()
            F.cross_entropy(model(xt[b]), yt[b]).backward()
            opt.step()
        model.eval()
        with torch.no_grad():
            history.append((F.cross_entropy(model(xt), yt).item(),
                            F.cross_entropy(model(xv), yv).item()))
        if history[-1][1] < best[0]:                          # early stopping on val loss
            best = (history[-1][1], copy.deepcopy(model.state_dict()), epoch)
    model.load_state_dict(best[1])
    return history, best[2]

x_test, _ = encode(test)
for name, build, wd in [("mean+MLP", lambda: MeanMLP(len(vocab)), 1e-4),
                        ("CNN", lambda: TextCNN(len(vocab)), 1e-4),
                        ("CNN no-reg", lambda: TextCNN(len(vocab), p=0.0), 0.0)]:
    torch.manual_seed(0)
    model = build()
    history, best_epoch = fit(model, wd)
    with torch.no_grad():
        pred = model(x_test).argmax(1).numpy()
    print(f"{name:12s} last train/val loss {history[-1][0]:.3f}/{history[-1][1]:.3f}, "
          f"best epoch {best_epoch}")
    report(name, pred)
    if name == "CNN":
        cnn_pred = pred

wrong = np.flatnonzero(cnn_pred != y_test)
print("CNN errors by kind:", {k: int((kind[wrong] == k).sum()) for k in ["plain", "negated",
 ↪  "contrast"]})
for i in wrong[:4]:
    print(" ", y_test[i], "<-", test[i][0])
```

整個腳本在單一 CPU 執行緒上數秒內完成，輸出：

```
tfidf (1, 1) test acc 0.546 {'plain': 0.651, 'negated': 0.348, 'contrast': 0.555}
tfidf (1, 2) test acc 0.880 {'plain': 1.0, 'negated': 1.0, 'contrast': 0.77}
mean+MLP     last train/val loss 0.269/0.464, best epoch 36
mean+MLP     test acc 0.797 {'plain': 0.973, 'negated': 0.945, 'contrast': 0.645}
CNN          last train/val loss 0.144/0.431, best epoch 30
CNN          test acc 0.898 {'plain': 0.983, 'negated': 0.945, 'contrast': 0.833}
CNN no-reg   last train/val loss 0.092/0.444, best epoch 10
CNN no-reg   test acc 0.910 {'plain': 1.0, 'negated': 0.928, 'contrast': 0.852}
CNN errors by kind: {'plain': 5, 'negated': 10, 'contrast': 87}
  0 <- the camera is not dull but the battery is weak
  0 <- the camera is noisy but the screen is bad
  1 <- the charger is solid but the battery is solid
  1 <- the charger is solid but the charger is bright
```

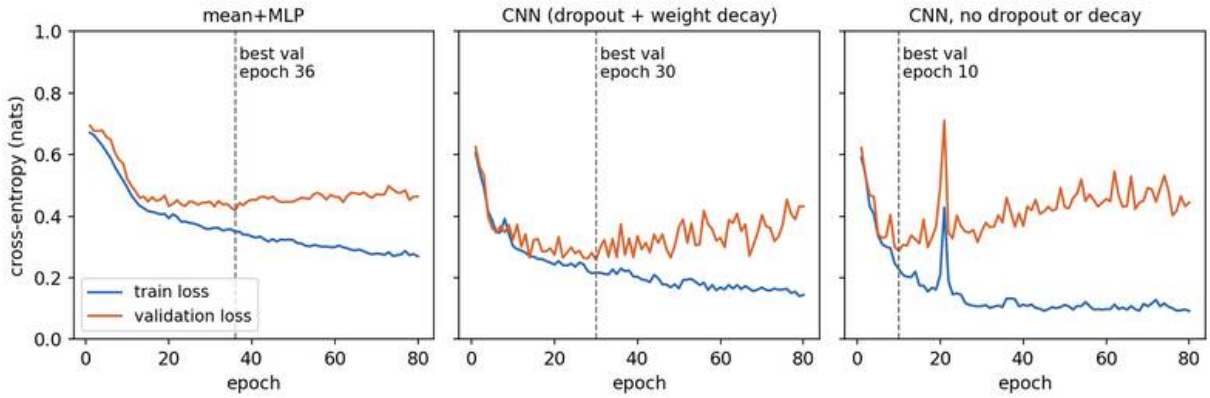


圖 *4* — 實驗中三個神經模型的訓練（藍）與驗證（橘）損失，種子為 *0*。虛線標出提前停止所保留的 *epoch*。

依類別解讀結果

Unigram TF–IDF 在設計上就無法處理否定。在這份資料中，“not” 出現在正面與負面評論的頻率相同，而每個形容詞是正是負取決於是否有 “not”。只看詞是否出現的線性模型無法表示這種交互作用，因此否定評論只得到 0.348，比隨機猜還差。

Bigram 修好了否定，卻修不好對比。“not good” 這個詞對是一個獨立特徵，所以 bigram 模型在一般與否定評論上都完全正確。在對比評論上它達到 0.77：它看得到兩個子句的 bigram，卻沒有可靠的方法知道哪個子句在 “but” 之後。

池化 MLP 解決了交互作用，卻解決不了順序。它的隱藏層能把 “not” 與形容詞結合，因此即使輸入是無序的平均，否定評論仍達到 0.945。在對比評論上它只有 0.645，因為平均池化讓 “A but B” 與 “B but A” 完全相同。

CNN 在對比評論上最好，但受限於寬度。濾波器寬度最多為 4 時，它在對比評論上達到 0.833。它的 102 個錯誤中只有 15 個不是對比評論，列出的錯誤說明了原因：在 “the charger is solid but

the battery is solid” 中，決定性的形容詞位在 “but” 之後四個位置，沒有任何單一視窗能同時包含兩者。濾波器只能依賴間接線索。練習 3 請你加入寬度 5 的濾波器來檢驗這個解釋。

解讀曲線

三張面板呈現相同模式：訓練損失持續下降，驗證損失則停止改善，然後上升。訓練標籤中有 10% 被翻轉，因此訓練損失遠低於這個雜訊水準，就表示模型正在背下錯誤。沒有正則化的 CNN 最快這麼做：它的驗證損失在第 10 個 epoch 觸底，到第 80 個 epoch 時明顯變差。Dropout 與權重衰減減緩了這種漂移，有正則化的 CNN 最佳驗證損失出現得較晚。

然而在測試準確率上，一旦提前停止為每個模型選定 epoch，沒有正則化的 CNN（0.910）與有正則化的 CNN（0.898）相當接近。只用一個種子、差距約一個百分點，這次執行無法排出優劣。站得住的結論比較窄：在這裡，以乾淨驗證集做提前停止是必要的，而有正則化的模型對何時停止比較不敏感。真正的比較應以多個種子重複實驗並報告分布範圍 [11]。

把錯誤分析變成習慣

能夠依類別計數，是因為每個測試樣本都帶著描述它為何可能困難的標記。從一開始就把這類標記放進評估資料：否定、對比、罕見詞、長度、來源。接著逐一閱讀錯誤，並問所需的證據是在哪個階段遺失的。如果平均池化讓兩個輸入完全相同，再多標籤也幫不了那個模型；如果 CNN 的視窗太短，解法是更寬的濾波器或不同的編碼器；如果標註本身錯了，解法在資料裡。

分開評估錯誤與機率

白話版。分類器可能選對標籤，卻提供不可靠的機率。決策與不確定性估計應分別評估。

把一個類別視為正類時：

$$P = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \qquad R = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}, \qquad F_1 = \frac{2PR}{P + R}.$$

其中 P 是精確率（precision），R 是召回率（recall），TP 是真陽性數，FP 是偽陽性數，FN 是偽陰性數，F_1 是精確率與召回率的調和平均。分母為零時，必須明確選擇慣例。Macro F1 平等平均各類別 F1；依樣本數加權的平均則讓常見類別影響較大。指標文件說明這些選擇 [13]。

Macro 平均讓罕見類別的失敗比較容易看見，卻不代表失敗的實際成本。漏掉緊急客服案件與誤送一般收據，可能有不同後果。應另外寫出成本政策，而不是把方便計算的分數當成完整效用。

校準（calibration）檢查預測信心是否與相似樣本的觀察正確率一致。模型可以保留 argmax 決策，同時改變信心。Guo 等人研究這個差別與訓練後的溫度縮放 [14]。校準參數必須使用保留的開發資料擬合，且需要足夠樣本估計機率行為。訓練到幾乎零訓練損失的神經分類器，是過度自信機率的常見來源，因此每次模型改變都應檢查校準。

選擇性預測可寫成：

$$\mathrm{coverage} = \frac{1}{n}\sum_{i=1}^{n} a_i, \qquad \mathrm{risk}_{\mathrm{accepted}} = \frac{\sum_{i=1}^{n} a_i \mathbf{1}[\hat{y}_i \neq y_i]}{\sum_{i=1}^{n} a_i}.$$

其中 n 是評估樣本數，$a_i \in \{0,1\}$ 表示是否接受第 i 個樣本，$\hat{y}_i$ 是預測，y_i 是參考標籤，$\mathbf{1}[\cdot]$ 在條件成立時為一，否則為零。如果沒有接受任何樣本，接受風險便未定義。風險與涵蓋率曲線呈現不同門檻下的取捨；部署門檻應依驗證資料選擇，而不是依最終測試標籤選擇。

不平衡、錯誤分析與部署變化

白話版。整體分數提高時，重要使用者群組的表現仍可能變差。應檢查錯誤類型、罕見類別與發生情境。

建立錯誤表，包含文字、標準標籤、預測標籤、機率、語言、長度、來源群組與分析者的解釋。實用錯誤類型包括標註歧義、缺少脈絡、否定、未見詞彙、多主題輸入、引述，以及確實超出範圍的文字。不同類型需要不同修正。更多訓練樣本可能改善未見詞彙，卻無法補上預測時永遠拿不到的對話回合。

類別加權改變每個訓練樣本的貢獻：

$$\mathcal{L}_{\text{weighted}} = -\frac{1}{n}\sum_{i=1}^{n} \alpha_{y_i} \log p_\theta(y_i \mid d_i).$$

其中 $\mathcal{L}_{\text{weighted}}$ 是加權交叉熵，n 是訓練樣本數，i 索引樣本，d_i 是文件，y_i 是標註類別，$\alpha_{y_i} > 0$ 是該類別的權重，p_θ 是分類器分布。加權政策改變訓練目標，但不能創造缺少的語言多樣性。

加權、過度取樣與門檻調整回答不同問題。加權改變參數學習；過度取樣改變呈現頻率，有時只是不斷重複少數類別中的同幾個樣本；門檻調整則在分數算出後改變行動政策。Hagiwara 的開發章節討論不平衡與訓練選擇 [1]。任何介入都應以真實類別盛行率評估，而不是以人為平衡的最終測試集評估。

部署變化可能影響詞彙、類別比例與文字到標籤的關係。新功能上線可能帶來新的技術錯誤詞彙；假日可能增加配送訊息；分流政策改變則可能重新定義標籤。這些應分開監測。空特徵向量比例增加，或神經模型中未知詞 ID 的比例增加，提示表示涵蓋範圍有問題；分流政策改變，則需要更新標註約定。

把模型與前處理產物一起保存。詞彙順序、IDF 權重或嵌入表、分類器類別順序、最大長度、機率校準與決策門檻，共同構成推論套件。以另一份詞彙表載入權重，會默默改變意義。套件應保存有版本的標籤結構與幾個確定性介面範例，讓服務可以檢查假設。

現代觀點

綜述觀點。 Goldberg 的入門綜述以前饋、卷積、循環與遞迴網路、輸入編碼及計算圖為主軸組織神經 NLP [5]；它仍是本課模型的精簡地圖。Minaee 等人回顧了 150 多個深度學習文字分類器與 40 多個資料集，從前饋與卷積模型一路到循環、注意力與預訓練模型，並比較已回報的基準結果 [15]。這類表格應被視為特定資料集與實驗協定上的證據，而不是能套用到任意標籤結構的排名。

Zhao 等人的大型語言模型綜述依預訓練、調適、使用與評估組織 [16]。對分類而言，這形成一個實用比較：固定表示加學習模型頭、針對任務調適編碼器，以及以指令進行預測。Chang 等人

從評估什麼、在哪裡評估、如何評估整理評估方法 [17]。這提醒我們，熟悉的基準並不能證明任意標籤結構都能正常運作。

穩健性觀點。 Sperduti 與 Moreo 的 2025 年拼字錯誤綜述整理拼字變化來源、修正方法，以及對下游任務與現代語言模型的影響 [18]。對本課的實務意義是：評估應包含真實拼字變化，再區分正規化失敗與分類器失敗。字元特徵、子詞嵌入與上下文表示提供不同做法；沒有任務專屬比較之前，都不應被假定為穩健。

2023–2026 年實務。加上分類頭並微調的預訓練 Transformer 編碼器（見NLP 11），是在標註資料與運算資源允許時的常見起點。指令跟隨語言模型增加了指定分類任務的方式 [16]。提示式模型可以解讀文字標籤描述，不必另外訓練分類器頭。這把一部分設定轉移到提示、標籤措辭與輸出剖析器。這些產物都應視為經過設計選擇的設定：測試前固定，將輸出限制在允許的結構，並量測保留決策與格式錯誤。

本課的模型仍有其角色。TF–IDF 加邏輯斯迴歸，以及小型卷積或平均網路，訓練便宜、服務快速、容易檢查。它們是判斷較大模型的進步是否真實的基準，而且當標籤有清楚的詞彙線索時往往已經足夠。

應用也可以使用較大模型建議標籤，再由人員檢查，最後訓練較小分類器。這會改變標註流程。生成標籤可能含有系統性錯誤，或重現教師模型的偏好。應檢查由人員確認的子集，並把合成監督與獨立評估標籤分開。能產生更多標籤，不等於監督品質更好。

未解問題。標註者與模型版本之間的標籤理解可能改變；低資源語言可能缺少獨立評估資料；多主題訊息難以符合單標籤結構；領域改變後校準可能惡化；嵌入相似度可能隱藏矛盾。這些問題需要明確的挑戰樣本與持續評估，而不是移除所有脈絡後的架構排名。

重點整理

- 標註約定決定預測問題與可採取的行動；詞彙表、IDF 權重與任何學習得到的表示都只能用訓練文字擬合。
- 稀疏線性基準強大、便宜又具診斷力；bigram 能捕捉局部否定，卻把每個片語視為與其他片語毫無關聯。
- 嵌入層是一張可學習的查詢表；平均池化加非線性 MLP 能學到詞的交互作用，卻看不到順序。
- 神經機率語言模型串接學習得到的詞向量，藉由在相似詞之間共享證據，推廣到未見過的歷史。
- 一維卷積分類器能在文字任何位置偵測軟性 n-gram 模式；最大池化丟棄位置，濾波器寬度限制它能連接的距離。
- 填補需要為平均、最大值、視窗與損失加上明確遮罩；多加填補不能改變任何輸出。
- 標籤有雜訊時，正則化與提前停止最重要；比較設定時應使用多個種子。
- 分開評估類別決策、機率品質、涵蓋率與各類別錯誤。

練習

1. **標籤約定。**定義客服任務的多標籤版本，為「delivery charge」、引述申訴與空白輸入寫出規則。說明輸出啟動函數與損失需要哪些改變。

提示

獨立的標籤機率可以允許兩條路徑。明確說明何時沒有標籤得到證據支持，而不是把所有低分都當成負標註。

2. **只用訓練資料的特徵**。加入只在最終測試集出現的詞。檢查已擬合的 TF–IDF 詞彙表與轉換後的測試向量，以及它在神經實驗詞彙表中得到的 ID。並與錯誤地使用所有文字擬合特徵比較。

提示

正確詞彙表不包含測試專屬詞；在神經模型中它會對應到 `<unk>`。除了詞彙表大小，也檢查文件頻率；即使所有詞已存在，洩漏仍可能改變權重。

3. **檢驗寬度的解釋**。在 PyTorch 實驗中，把 CNN 的寬度改成 `(2, 3, 4, 5)` 後重新執行。對比評論的準確率是否改變？接著以五個種子分別執行原始與修改後的模型，報告平均與範圍後再下結論。

提示

數一數 “but the battery is solid” 有幾個位置。寬度 5 的視窗能同時包含 “but” 與決定性形容詞。除了寬度之外其他一切保持不變，並記得單一種子的差異可能只是雜訊。

4. **故意製造遮罩錯誤**。移除 `MeanMLP` 的遮罩（改用普通的 `.mean(1)`），以及 `TextCNN` 的遮罩（對所有視窗池化）。在測試批次多補五欄填補，並與有遮罩的版本比較預測。哪個模型改變了？為什麼？

提示

填補嵌入為零，所以填補越多，普通平均就越往零縮。對 CNN 而言，與填補重疊的視窗仍有非零偏置，可能贏得最大值。

5. **手算神經語言模型**。對於有 V 個詞、$n-1$ 個歷史詞、每詞 m 個特徵、h 個隱藏單元的 NPLM，計算 C、H、$\mathbf{d}$、U、$\mathbf{b}$ 與 W 的參數量。V 很大時哪一項主導？移除直接連接後參數量如何改變？

提示

從式子寫出每個矩陣的形狀。輸出層與嵌入表都隨 V 增加，隱藏層則不會。

6. **選擇性決策**。以幾種驗證成本政策執行客服訊息實驗。在驗證資料畫出涵蓋率與接受錯誤，選擇一個門檻，再只在測試上評估一次。

提示

若沒有接受任何樣本，接受錯誤的分母為零。應報告未定義，而不是完美準確率。

參考文獻

1. M. Hagiwara, *Real-World Natural Language Processing: Practical Applications with Deep Learning*, Manning, 2021; Chapters 2–4, 7 and 10–11. publisher
2. S. McRoy, *Principles of Natural Language Processing*, open textbook, 2021; Chapter 6, "Benchmark Tasks for Language Modelling." chapter
3. Scikit-learn developers, "Text feature extraction," scikit-learn documentation, current online edition. documentation
4. Scikit-learn developers, "Logistic regression," scikit-learn documentation, current online edition. documentation
5. Y. Goldberg, "A primer on neural network models for natural language processing," *Journal of Artificial Intelligence Research* 57, 2016. paper
6. M. Iyyer, V. Manjunatha, J. Boyd-Graber and H. Daumé III, "Deep unordered composition rivals syntactic methods for text classification," ACL-IJCNLP, 2015. paper
7. A. Joulin, E. Grave, P. Bojanowski and T. Mikolov, "Bag of tricks for efficient text classification," EACL, 2017. paper
8. Y. Bengio, R. Ducharme, P. Vincent and C. Jauvin, "A neural probabilistic language model," *Journal of Machine Learning Research* 3, 2003. paper
9. R. Collobert, J. Weston, L. Bottou, M. Karlen, K. Kavukcuoglu and P. Kuksa, "Natural language processing (almost) from scratch," *Journal of Machine Learning Research* 12, 2011. paper
10. Y. Kim, "Convolutional neural networks for sentence classification," EMNLP, 2014. paper
11. Y. Zhang and B. Wallace, "A sensitivity analysis of (and practitioners' guide to) convolutional neural networks for sentence classification," IJCNLP, 2017. paper
12. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research* 15, 2014. paper
13. Scikit-learn developers, "Classification metrics," scikit-learn documentation, current online edition. documentation
14. C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, "On calibration of modern neural networks," ICML, 2017. paper
15. S. Minaee, N. Kalchbrenner, E. Cambria, N. Nikzad, M. Chenaghlu and J. Gao, "Deep learning-based text classification: a comprehensive review," *ACM Computing Surveys* 54(3), 2021. survey
16. W. X. Zhao et al., "A survey of large language models," *Frontiers of Computer Science*, 2026; online manuscript updated March 2026. survey
17. Y. Chang et al., "A survey on evaluation of large language models," *ACM Transactions on Intelligent Systems and Technology*, 2024. survey
18. G. Sperduti and A. Moreo, "Misspellings in natural language processing: a survey," arXiv:2501.16836, 2025. survey

NLP 04・詞彙語意與詞嵌入

從詞義、詞彙關係與 WordNet，走到分布式向量：PPMI、Word2Vec、GloVe 與 fastText，如何評估它們、它們吸收的偏誤、句子嵌入，以及邁向上下文表示的一步。

網站版本: https://blog.shyandram.dev/zh-tw/learn/nlp/nlp-04-lexical-semantics-word-embeddings/

你將學到

- 區分詞形、詞元（lemma）與詞義，並說出主要的詞彙關係：同義、反義、上下位與部分整體關係，以及 WordNet 如何組織它們。
- 解釋分布假設：共享上下文能提供哪些詞彙證據，以及無法確立哪些意義關係。
- 建立詞與上下文矩陣，把正向 PMI 轉成精簡向量，並推導 CBOW、Skip-gram、負取樣、GloVe 與 fastText 的目標函數。
- 以相似度與類比測試評估嵌入，同時認清它們的限制，並量測一種關聯偏誤。
- 將詞向量池化成句子表示、以對比學習訓練句子嵌入，並說明上下文嵌入改變了什麼。

先看全貌

詞 ID 只識別詞彙表項目。它的數值不會說明該詞在語言中做什麼。**嵌入（embedding）**替項目配置學習出的向量，讓不同詞共享有用的統計模式。核心問題不只是如何把向量縮小，而是哪些證據與目標應該決定它的座標。

> **用最簡單的話說**
>
> 想像透過觀察使用場合，認識不熟悉的工具。兩種工具若經常出現在木頭、釘子與工作臺旁，可能具有相關用途。即使沒人替工具命名，你仍能從觀察畫出一張地圖。但是，同在工作臺出現不代表可以互換：鐵鎚與鉗子仍做不同的事。詞向量也從周圍的詞學出類似地圖。

語言學家以**詞彙語意學（lexical semantics）**描述詞義：詞的各種意義，以及同義、上下位等關係，並由人工記錄在 WordNet 這類資源中。分布式方法則改從文字學習向量。本課連結這兩種觀點。Hagiwara 將詞與文件嵌入發展為實務應用可重用的特徵 [1]。McRoy 則區分詞彙相似度與情緒、蘊涵等語言任務 [2]。這裡專注於表示本身，並持續追問：向量幾何實際捕捉的是哪一種詞彙關係。

NLP 03 把詞向量當成任務的副產品來學習：分類器裡的嵌入層，以及 Bengio 等人神經機率語言模型中的表 C [3]。Word2Vec 則把向量本身當成目標，並以鄰近詞、而不是下一個詞，作為預測對象。兩者的預測約定不同：

問題	神經機率語言模型	靜態 Word2Vec 嵌入
觀察什麼？	有順序的 $n-1$ 個 token 歷史，以及緊接其後的 token	中心詞與兩側選定的鄰近詞
預測什麼？	下一個 token 的正規化分布	從鄰近詞猜中心詞，或從中心詞分別猜鄰近詞
目的是由左至右生成嗎？	是；它本身就是語言模型	基本視窗訓練不是有順序的生成器
訓練後重用什麼？	整個網路；詞向量只是副產品	學習出的向量表

CBOW 中的 **continuous** 代表內部表示具有實數座標，不代表文字本身連續、詞之間沒有邊界，或每種詞義都各有向量。本課最後會連結計數式與預測式構造，並指出各自保留的資訊。計數式語言模型及其評估則留到NLP 05。

詞彙語意：詞義與關係

白話版。一個寫出來的詞可能有好幾個意義，而意義之間有結構化的關係：有些幾乎等價，有些彼此相反，有些是另一個的一種。嵌入稍後會把其中好幾種關係混在一起，所以先替它們命名。

詞形、詞元與詞義

詞形（word form）是文字中實際出現的樣子（`banks`、`banked`）；**詞元（lemma）**是把各種屈折形式歸在一起的辭典條目（`bank`）。**詞義（sense）**是詞元的一個意義。`bank` 至少有兩個彼此無關的詞義：河邊的斜坡與金融機構，這是**同形異義（homonymy）**。`window` 的詞義彼此相關：牆上的開口與螢幕上的區域，這是一**詞多義（polysemy）**。NLP 02 討論上下文與組合如何選出詞義。對表示而言，關鍵在於詞彙表項目是詞形或詞元，不是詞義：每個項目只有一列的表，必然把該項目的所有詞義合併在一起。

詞義之間的關係

詞彙關係存在於詞義之間，而不是拼寫字串之間：

- **同義（synonymy）**：兩個詞義在許多上下文中可以互換，意義改變很小，例如 `slope` 與 `incline`。完全同義很少見；語域、搭配與隱含意義通常不同。
- **反義（antonymy）**：沿著某個尺度或維度相反的詞義，例如 `hot` 與 `cold`。反義詞在其他方面非常相似：描述同一種屬性，出現在同類句子中。
- **下位與上位關係（hyponymy、hypernymy）**：一種「是一種」關係。`poodle` 是 `dog` 的下位詞，`dog` 是 `poodle` 的上位詞。這種關係不對稱，通常具遞移性，形成階層。
- **部分整體關係（meronymy）**：部分與整體，例如 `wheel` 與 `car`。

另有兩個概念對評估很重要。**相似度（similarity）**問兩個概念是否共享屬性（`cup` 與 `mug`）；**相關性（relatedness）**或聯想（association）問它們是否常一起出現（`cup` 與 `coffee`）。相關的一對詞不一定相似。

WordNet：人工建立的詞彙資料庫

WordNet 把英文名詞、動詞、形容詞與副詞組織成**同義詞集（synset）**：表達同一概念的一組同義詞元，各附一段簡短釋義（gloss），並以上位、部分整體與反義等關係相連 [4]。查詢一個詞元會傳回它的各個同義詞集，每個詞義一個。沿著上位連結往上走，會到達越來越一般的概念，讓程式能量測兩個詞義在階層中相距多遠。

下列範例以相同風格、用手工建立一個迷你資料庫，不必下載 WordNet 本身也能展示運作方式。其中條目都是本課自行編寫的。

```
# A hand-built miniature lexical database in the style of WordNet (not WordNet data).
synsets = {
    "bank.n.slope":   {"lemmas": ["bank"], "gloss": "sloping land beside a river",
     ↪  "hypernym": "slope.n"},
    "bank.n.finance": {"lemmas": ["bank"], "gloss": "an institution that holds deposits",
     ↪  "hypernym": "institution.n"},
    "slope.n":        {"lemmas": ["slope", "incline"], "gloss": "a surface at an angle",
     ↪  "hypernym": "landform.n"},
    "landform.n":     {"lemmas": ["landform"], "gloss": "a natural feature of land",
     ↪  "hypernym": "entity.n"},
    "institution.n":  {"lemmas": ["institution"], "gloss": "an organization with a public
     ↪  role", "hypernym": "organization.n"},
    "school.n":       {"lemmas": ["school"], "gloss": "an institution for teaching",
     ↪  "hypernym": "institution.n"},
    "organization.n": {"lemmas": ["organization"], "gloss": "a group with a purpose",
     ↪  "hypernym": "entity.n"},
    "entity.n":       {"lemmas": ["entity"], "gloss": "anything that exists", "hypernym":
     ↪  None},
}

def senses(word):
    return [s for s, v in synsets.items() if word in v["lemmas"]]

def path_to_root(s):
    path = [s]
    while synsets[path[-1]]["hypernym"]:
        path.append(synsets[path[-1]]["hypernym"])
    return path

def path_similarity(a, b):            # 1 / (1 + edges on the shortest is-a path)
    pa, pb = path_to_root(a), path_to_root(b)
    common = next(s for s in pa if s in pb)
    return 1 / (1 + pa.index(common) + pb.index(common))

print("senses of bank:", senses("bank"))
print("synonyms in slope.n:", synsets["slope.n"]["lemmas"])
print("hypernym chain:", " -> ".join(path_to_root("bank.n.finance")))
for other in ["school.n", "slope.n"]:
    print(f"path_similarity(bank.n.finance, {other}) = {path_similarity('bank.n.finance',
     ↪  other):.3f}")
```

執行會印出：

```
senses of bank: ['bank.n.slope', 'bank.n.finance']
synonyms in slope.n: ['slope', 'incline']
hypernym chain: bank.n.finance -> institution.n -> organization.n -> entity.n
path_similarity(bank.n.finance, school.n) = 0.333
path_similarity(bank.n.finance, slope.n) = 0.167
```

`bank` 的金融詞義比較接近 `school`（兩者都是機構），而不是 `slope`，即使 `bank` 的另一個詞義正是一種斜坡。詞義清單讓這個區別明確；字串 `bank` 的單一向量做不到。

人工資源精確且可解讀，但建置成本高，對某些語言與領域的涵蓋遠優於其他，跟不上新詞與新詞義，而且只編碼一套詞義劃分，未必符合應用需求。分布式向量做的是相反的取捨：便宜、涵蓋廣，但關係是隱含且混雜的。許多實務工作會結合兩者，例如用詞彙資源檢查嵌入的近鄰。

分布式證據：上下文是觀察

白話版。出現在相似上下文的詞，可能得到相似向量。上下文提供用法的證據，不會直接提供定義或真值條件。

Harris 主張意義的差異與分布的差異相關，也就是與語言單位出現的環境相關 [5]。這個**分布假說（distributional hypothesis）**是本課其餘所有方法的基礎。

詞類型（word type）是詞彙表項目；**詞出現（word token）**是一次實際出現。靜態嵌入把許多次出現的證據彙整成該類型的一個向量。`bank` 在 `river` 旁與在 `account` 旁出現，都會影響同一列。模型可以學到這個詞的混合用法，但查表操作無法為每種詞義選不同列。

收集統計之前先定義上下文。半徑二的詞視窗包含中心詞左右各最多兩個 token，排除中心詞。較短視窗著重緊鄰位置；較大視窗納入較遠的主題證據。若方向重要，可以把左側與右側位置當成不同特徵。本課刻意合併兩側，讓模型的不變性明確呈現。

分布假設（distributional assumption）實用之處，在於文字提供重複觀察，不必替每個詞標註語意類別。它並不完整，因為共享用法包含同義、反義、主題關聯與語法相容性。`hot` 與 `cold` 都可能出現在 `water` 前；基於這種證據的相似度，無法確立兩者表達相同屬性。用上一節的術語來說，分布式相似度混合了同義、反義、同位下位詞與相關性。Turney 與 Pantel 按照矩陣所表示的證據，整理這些向量空間方法 [6]。

一起記錄語料、tokenizer 與視窗規格。改變斷詞方式，就改變哪些類型被計算上下文。從技術事故報告學出的嵌入，可能以不同於電子設備手冊的方式組織 `charge`。座標是這些選擇的結果，不是與語言無關的詞義測量。

計數式嵌入：檢查共享上下文的假設

白話版。計算每個詞與各種上下文一起出現的頻率，再與假設兩者獨立的基準比較。特別強的關聯會形成正向座標。

Turney 與 Pantel 的分布式表示綜述 [6] 說明這種**計數式**途徑。令 C_{wc} 計算中心詞 w 與鄰近上下文詞 c 所形成之有序配對的觀察次數。即使兩種角色的詞彙表包含相同詞，角色仍不同。從表格

定義經驗機率：

$$N_C = \sum_{w,c} C_{wc}, \qquad P(w,c) = \frac{C_{wc}}{N_C}, \qquad P(w) = \sum_{c} P(w,c), \qquad P(c) = \sum_{w} P(w,c).$$

其中 N_C 是中心與上下文的總配對數，C_{wc} 是一個儲存格的計數，$P(w,c)$ 是經驗聯合機率，$P(w)$ 是中心詞邊際機率，$P(c)$ 是上下文邊際機率。加總遍歷對應詞彙表。一個 token 可能貢獻多個配對，因此 N_C 不是 token 數。

逐點互資訊（pointwise mutual information，PMI）比較經驗聯合機率與獨立基準。**正向 PMI（positive PMI，PPMI）**只保留強於基準的關聯：

$$\mathrm{PMI}(w,c) = \log \frac{P(w,c)}{P(w)P(c)}, \qquad \mathrm{PPMI}(w,c) = \max(\mathrm{PMI}(w,c), 0).$$

其中 $\mathrm{PMI}(w,c)$ 是一個配對的對數關聯分數，$\mathrm{PPMI}(w,c)$ 是其非負版本，log 是自然對數。機率定義如上。聯合計數為零且邊際機率為正時，PMI 趨近負無限大，PPMI 為零；整列完全未觀察到則需要另外處理。

高頻上下文不會自動得到很大的關聯分數。它必須比整體頻率所預期的更常出現在這個中心詞附近。相反地，罕見配對可能只靠很少證據就得到高 PMI。應同時檢查計數與分數；移除負關聯、只保留正關聯，也是在捨棄資訊，不會使剩下的每個關聯都可靠。

PPMI 矩陣仍為每個上下文保留一個座標。截斷**奇異值分解（singular value decomposition，SVD）**可將它壓縮：

$$M = A\Sigma B^{\top}, \qquad M_D = A_D \Sigma_D B_D^{\top}, \qquad E = A_D \Sigma_D.$$

其中 M 是 PPMI 矩陣，A 與 B 包含左右奇異向量，Σ 是奇異值對角矩陣，D 是保留維度，下標 D 取前 D 個奇異成分。M_D 是低秩重建，E 提供下方使用的中心詞向量。這是程式採用的慣例；以其他方式分配奇異值縮放，會產生不同的向量幾何。

SVD 找出列之間共享的模式，不會替每個座標指定語意名稱。若原矩陣的秩高於 D，壓縮無法保留每個儲存格。下列合成語料讓我們同時檢查構造方式，以及將餘弦相似度解讀為同義關係的失敗案例。

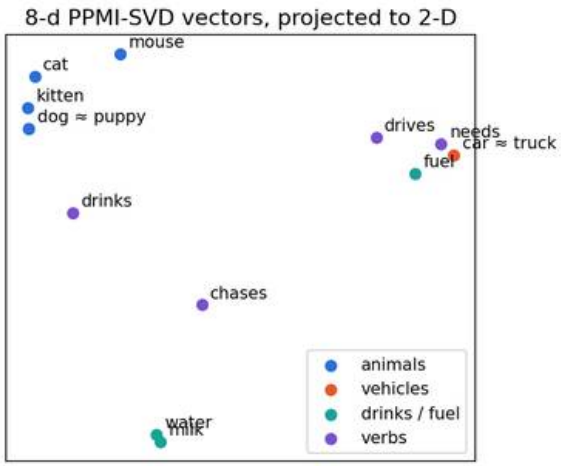
```
import numpy as np

sentences = """the cat drinks milk . the dog drinks water . a cat chases a mouse .
a dog chases a cat . the kitten drinks milk . the puppy drinks water .
a kitten chases a ball . a puppy chases a ball . the car needs fuel .
the truck needs fuel . a car drives on the road . a truck drives on the road .""".split(".")
tokens = [s.split() for s in sentences if s.strip()]
vocab = sorted({w for s in tokens for w in s})
idx = {w: i for i, w in enumerate(vocab)}

# 1) Co-occurrence counts within a +-2 word window.
C = np.zeros((len(vocab), len(vocab)))
for s in tokens:
    for i, w in enumerate(s):
        for j in range(max(0, i - 2), min(len(s), i + 3)):
            if j != i:
```

```
                C[idx[w], idx[s[j]]] += 1

# 2) Positive pointwise mutual information.
total = C.sum()
pw, pc = C.sum(1, keepdims=True) / total, C.sum(0, keepdims=True) / total
with np.errstate(divide="ignore"):
    pmi = np.log((C / total) / (pw * pc))
ppmi = np.maximum(pmi, 0)

# 3) Truncated SVD gives dense 8-dimensional embeddings.
U, S, _ = np.linalg.svd(ppmi)
E = U[:, :8] * S[:8]
E /= np.linalg.norm(E, axis=1, keepdims=True)

def nearest(word, k=3):
    sims = E @ E[idx[word]]
    order = [vocab[i] for i in np.argsort(-sims) if vocab[i] != word]
    return [(w, round(float(sims[idx[w]]), 2)) for w in order[:k]]

for w in ["cat", "puppy", "truck", "milk"]:
    print(w, "->", nearest(w))
```

輸出：

```
cat -> [('kitten', 0.96), ('puppy', 0.88), ('dog', 0.88)]
puppy -> [('dog', 1.0), ('kitten', 0.98), ('cat', 0.88)]
truck -> [('car', 1.0), ('needs', 0.6), ('the', 0.47)]
milk -> [('water', 0.97), ('a', 0.62), ('chases', 0.51)]
```

只用十二個句子、沒有任何標籤，動物就和動物聚在一起，車輛和車輛聚在一起。注意「milk」與「water」幾乎相同：在這個語料中兩者都是被喝的東西。嵌入相似度量的是**共享的上下文**，這通常、但不總是對應到相似的意思；例如「熱」與「冷」這類反義詞也會共享上下文。



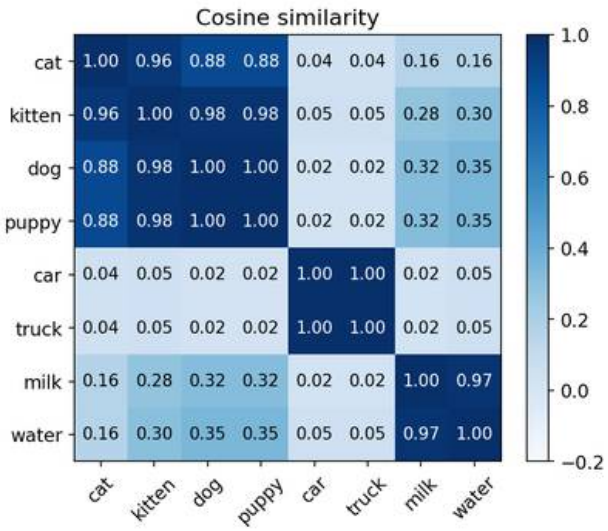


圖 *1* — 上方程式碼得到的向量。共享上下文的詞群組內，餘弦相似度很高。

Word2Vec：藉由預測鄰近詞學習詞向量

白話版。出現在相似上下文的詞，常具有相似用途。Word2Vec 把這個觀察變成小型預測任務：從鄰近詞猜中心詞，或從中心詞猜鄰近詞，並學習有助於預測的向量。向量是學習出的參數，不是人工列出的詞義清單。兩種基本架構是**連續詞袋模型（continuous bag of words，CBOW）**與**Skip-gram** [7]。

把句子轉成訓練樣本

從原創句子 `small red robots move slowly` 開始。**上下文視窗（context window）**選出鄰近位置；半徑一代表中心詞前後各一個詞。此例只使用句子內部的中心位置，因此每個視窗都有兩個鄰近詞：

中心詞	上下文	CBOW 訓練樣本	Skip-gram 訓練樣本
`red`	`small`、`robots`	`[small, robots] → red`	`red → small`；`red → robots`
`robots`	`red`、`move`	`[red, move] → robots`	`robots → red`；`robots → move`
`move`	`robots`、`slowly`	`[robots, slowly] → move`	`move → robots`；`move → slowly`

CBOW 結合多個上下文詞來預測一個中心詞；Skip-gram 用一個中心詞，分別預測各個上下文詞。 Skip-gram 並不產生有順序的句子；「skip」也不代表隨機刪詞。目標可以位於中心詞任一側。實作可在句子邊界使用較短視窗；這裡的程式略過那些中心位置，讓張量形狀保持簡單。

訓練標籤直接來自文字本身。在建立樣本之前，不必有人替 `robots` 標註語意類別。重複出現就提供重複證據。視窗決定模型被鼓勵捕捉的關係：即使詞彙表不變，改變視窗仍會改變訓練任務。

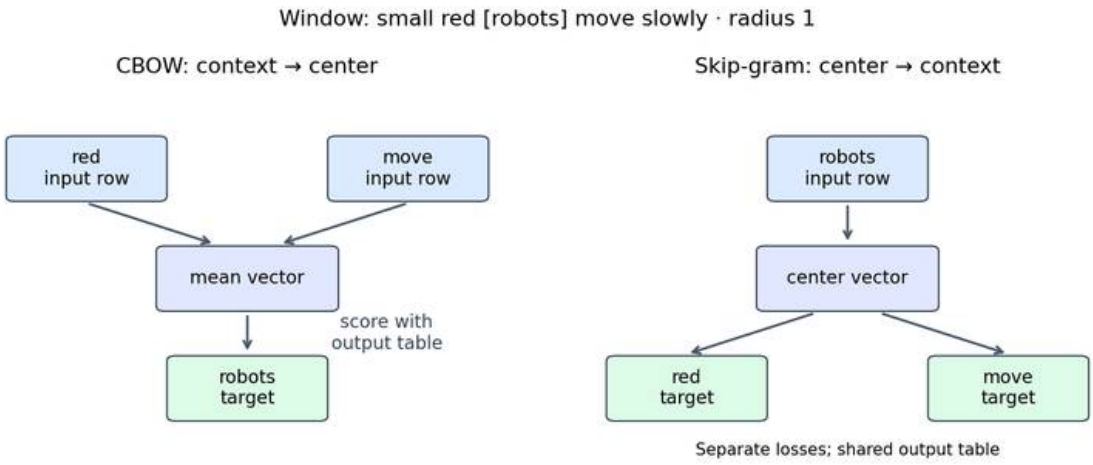


圖 2 — 同一句子形成方向相反的預測任務。兩種架構都學習輸入與輸出嵌入表；圖中的平均運算只出現在 *CBOW* 路徑。

兩個嵌入表與 softmax

一個詞作為輸入時需要向量，作為預測候選時也需要向量。使用兩個學習表，而不是先假定這兩種角色共享權重：

$$E, U \in \mathbb{R}^{V \times D}, \qquad \mathbf{e}_w = E_{w,:}^{\top}, \qquad \mathbf{u}_w = U_{w,:}^{\top}, \qquad p(o \mid \mathbf{h}) = \frac{\exp(\mathbf{u}_o^{\top}\mathbf{h})}{\sum_{v=1}^{V} \exp(\mathbf{u}_v^{\top}\mathbf{h})}.$$

其中 E 是輸入嵌入表，U 是輸出嵌入表，V 是詞彙表大小，D 是向量維度，w 是詞的 ID，冒號選取該列的所有座標。轉置把 $\mathbf{e}_w$ 與 $\mathbf{u}_w$ 變成直向行向量。$\mathbf{h} \in \mathbb{R}^D$ 表示輸入，o 是目標詞 ID，v 索引候選詞，$p(o \mid \mathbf{h})$ 是目標的 softmax 機率。每個內積都是預測分數；分母使整個詞彙表的機率總和為一。

CBOW 平均上下文向量來形成輸入。損失要求模型替觀察到的中心詞配置較高機率：

$$\mathbf{h}_t = \frac{1}{|C_t|} \sum_{j \in C_t} \mathbf{e}_{w_j}, \qquad \mathcal{L}_{\text{CBOW}} = -\frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \log p(w_t \mid \mathbf{h}_t).$$

其中 w_t 是位置 t 的中心詞，C_t 是選出的鄰近位置集合且不包含 t，$|C_t|$ 是集合大小，$\mathbf{h}_t$ 是這些詞向量的平均，$\mathcal{T}$ 包含有效中心位置，$\mathcal{L}_{\text{CBOW}}$ 是中心詞負對數機率的平均。平均不保留上下文順序：交換左右鄰近詞會產生相同輸入。有些寫法加總上下文向量；本課與程式使用平均。

Skip-gram 則查出中心詞向量，再用相同預測規則處理每個鄰近詞：

$$\mathcal{L}_{\text{SG}} = -\frac{1}{|\mathcal{P}|} \sum_{(t,j) \in \mathcal{P}} \log p(w_j \mid \mathbf{e}_{w_t}).$$

其中 $\mathcal{L}_{\text{SG}}$ 是 Skip-gram 損失，$\mathcal{P}$ 是觀察到的中心與上下文位置配對，$|\mathcal{P}|$ 是配對數，t 索引中心

位置，j 索引一個鄰近位置，w_t 是輸入詞，w_j 是目標詞。即使詞 ID 重複，每次出現仍保留為獨立樣本。

一次訓練步驟會相對於其他候選，提高觀察到的目標分數，同時改變選出的輸入向量與輸出向量。許多樣本重複使用同一個輸入列，迫使模型整合該詞在不同上下文出現的證據。這種共享預測任務，就是從離散詞 ID 學到連續特徵的途徑。這兩種基本架構都不需要堆疊很深的隱藏層，這是它們與 NLP 03 神經語言模型在計算上的主要差別。

負取樣：比較觀察配對與隨機抽樣

完整 softmax 對每次預測都計算詞彙表中每個詞的分數。**負取樣（negative sampling）**將這個跨詞彙表的分類，改成較小的二元任務：區分實際觀察到的中心與上下文配對，以及隨機抽取目標詞形成的配對。Word2Vec 的後續論文提出了這個方法 [8]。

對觀察到的輸入詞 c 與上下文詞 o，一次取樣的損失為：

$$\mathcal{L}_{\text{neg}}(c, o) = -\log \sigma(\mathbf{u}_o^\top \mathbf{e}_c) - \sum_{k=1}^{K} \log \sigma(-\mathbf{u}_{n_k}^\top \mathbf{e}_c), \qquad n_k \sim q, \qquad \sigma(a) = \frac{1}{1+\exp(-a)}.$$

其中 $\mathcal{L}_{\text{neg}}$ 是一組觀察配對的負取樣損失，c 是中心詞 ID，o 是觀察到的上下文詞 ID，$\mathbf{e}_c$ 是中心的輸入向量，$\mathbf{u}_o$ 與 $\mathbf{u}_{n_k}$ 是輸出向量，K 是指定的隨機目標數，n_k 是分布 q 的第 k 次抽樣，σ 是作用於實數分數 a 的 sigmoid。第一項鼓勵觀察配對具有高分；其餘項鼓勵隨機配對具有低分。完整訓練目標對觀察配對與取樣隨機性取平均。若使用 CBOW 版本，可用上下文平均向量取代 $\mathbf{e}_c$。

原論文使用的一種常見取樣分布為：

$$q(w) = \frac{f(w)^{3/4}}{\sum_{v=1}^{V} f(v)^{3/4}}.$$

其中 $q(w)$ 是抽出詞彙項目 w 的機率，$f(w)$ 是它在語料中的出現次數，v 索引詞彙表項目，V 是詞彙表大小。相較於直接依次數取樣，這個指數讓頻率分布較平坦。它是設計選擇，不是普遍的機率定律。

對觀察配對 `robots → move`，隨機抽樣可能產生 `robots → red`。「負」標籤的意思是**由雜訊程序抽出**，並不宣稱兩個詞永遠不會共同出現。抽樣結果可能是其他位置的真實上下文，甚至與這次的目標相同。實作必須說明是否保留、略過或重新抽取這類碰撞；改變規則也會改變取樣目標。模型區分的是配對的統計來源，不是在建立絕對不相容的詞典。

二元 sigmoid 分數在詞彙表上的總和不等於一。負取樣是另一種訓練目標，不是上面 softmax 的精確計算；其損失也不是正規化的下一詞概似。它用來學習實用表示，而表示仍應依預定的下游任務評估。

預測為何會改變向量幾何

「從鄰近詞學習」可以直接從梯度看懂。對一個使用完整 softmax 的目標詞，其分數是輸出向量與輸入表示的內積：

$$s_v = \mathbf{u}_v^\top \mathbf{h}, \qquad \ell = -\log p(o \mid \mathbf{h}), \qquad \frac{\partial \ell}{\partial s_v} = p(v \mid \mathbf{h}) - \mathbf{1}[v = o], \qquad \nabla_{\mathbf{h}} \ell = \sum_{v=1}^{V} p(v \mid \mathbf{h}) \mathbf{u}_v - \mathbf{u}_o.$$

其中 s_v 是候選詞 v 的分數，ℓ 是觀察到目標詞 o 時的損失，$\mathbf{1}[v = o]$ 在候選詞等於目標詞時為一，否則為零；$\nabla_{\mathbf{h}} \ell$ 是對輸入表示的梯度。嵌入表、詞彙表大小 V 與條件機率皆沿用前面的定義。

梯度下降會讓輸入表示靠近觀察到的輸出向量，同時遠離模型依機率加權的其他候選向量平均。某個詞每次出現，都會透過同一列累積更新。因此，上下文分布相似的詞可能學到相近的預測角色，過程中完全不需要標註它們是不是同義詞。

對使用平均的 CBOW，每個選中的上下文向量列都收到 $\nabla_{\mathbf{h}} \ell / |C_t|$。如果兩組上下文只是相同詞的不同排列，輸入表示就無法區分它們。Skip-gram 則讓一個中心詞向量接受多個鄰詞目標的更新；它的基本分數沒有欄位記錄鄰詞位於左邊還是右邊。因此，選擇視窗同時決定目標函數可以直接表達哪些關係；擴大視窗並不只是讓小視窗變得更準確。

負取樣分數究竟估計什麼

先暫時不限制嵌入維度，就能看清楚取樣目標。固定一個中心詞，假設每個候選詞都有可以獨立調整的分數。以 p_o 表示它的經驗上下文機率、q_o 表示雜訊機率，該候選詞對期望損失的貢獻為：

$$\ell_o(s_o) = -p_o \log \sigma(s_o) - K q_o \log \sigma(-s_o), \qquad \frac{d\ell_o}{ds_o} = (p_o + K q_o)\sigma(s_o) - p_o.$$

其中 o 是一個候選上下文詞，s_o 是不受限制的實數分數，p_o 與 q_o 是固定中心詞下的兩種機率，K 是每個觀察詞對所搭配的獨立雜訊取樣次數，σ 是前面定義的 sigmoid。本推導保留雜訊取樣恰好抽到觀察目標的碰撞情況。

當兩種機率都大於零時，駐點分數為：

$$s_o^\ast = \log \frac{p_o}{K q_o}.$$

其中 $s_o^\ast$ 使該候選詞的期望損失最小。若 $p_o = 0.5$、$q_o = 0.25$、$K = 2$，最佳分數就是零：在相關的機率質量中，兩種統計來源各占一半。如果只將 q_o 改成 0.05，最佳分數就變成 $\log 5 \approx 1.609$。觀察到的上下文頻率沒有改變，雜訊分布卻改變了分數。因此，正分數表示「比這個取樣基準更常共同出現」，並不表示「這個詞對一定有意義」。若觀察機率為零、雜訊機率為正，則不受限制的最佳分數趨向負無窮。

如果雜訊分布恰好等於語料的上下文邊際分布，這個比值就得到 PMI 減去 $\log K$。Levy 與 Goldberg 分析了由此產生的矩陣分解觀點 [9]；常用的四分之三次方取樣分布則會改變這個比值。實際嵌入模型讓分數共用低維向量，通常無法同時達到每個獨立最佳值。因此，向量表是在秩限制下折衷不同詞對的統計關係。這既說明它與計數式方法的連結，也解釋了截斷 PPMI 矩陣與訓練 Word2Vec 為何不一定得到相同向量。

完整 softmax 每個目標的運算量約為 $O(VD)$；計算一個觀察目標與 K 個取樣詞的分數則約為 $O((K+1)D)$，尚未計入取樣與批次處理成本。其中 D 是嵌入維度。計算量降低的同時，目標函數也改變了。sigmoid 值與負取樣損失都不能代替計算困惑度時所需的正規化詞元機率。

在 CPU 訓練 CBOW 與 Skip-gram

這個獨立範例使用完整 softmax 學習輸入與輸出表。詞彙表很小，因此可以直接檢查目標，不需要取樣近似。兩個模型共用相同的原創語料、半徑一的內部視窗、嵌入寬度與訓練迴圈。Skip-gram 路徑只有一個輸入詞，取平均就是中心向量本身。

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(7)
torch.set_num_threads(1)
corpus = [
    'small red robots move slowly',
    'small blue robots move quickly',
    'large red sensors work slowly',
    'large blue sensors work quickly',
    'small red sensors work quietly',
    'large blue robots move quietly',
]
vocab = sorted(set(' '.join(corpus).split()))
ids = {word: i for i, word in enumerate(vocab)}
cbow_x, cbow_y, sg_x, sg_y = [], [], [], []
for sentence in corpus:
    tokens = [ids[word] for word in sentence.split()]
    # Use radius 1 and interior centers, so every context has two words.
    for t in range(1, len(tokens) - 1):
        neighbors = [tokens[t - 1], tokens[t + 1]]
        cbow_x.append(neighbors)
        cbow_y.append(tokens[t])
        for neighbor in neighbors:
            sg_x.append([tokens[t]])
            sg_y.append(neighbor)

class Word2Vec(nn.Module):
    def __init__(self, size, width=8):
        super().__init__()
        self.input = nn.Embedding(size, width)
        self.output = nn.Embedding(size, width)
        nn.init.normal_(self.input.weight, std=0.1)
        nn.init.normal_(self.output.weight, std=0.1)

    def forward(self, context):
        h = self.input(context).mean(dim=1)
        return h @ self.output.weight.T

embeddings = {}
for name, inputs, targets in [('CBOW', cbow_x, cbow_y),
                              ('Skip-gram', sg_x, sg_y)]:
    x, y = torch.tensor(inputs), torch.tensor(targets)
    model = Word2Vec(len(vocab))
    optimizer = torch.optim.SGD(model.parameters(), lr=1.0)
    initial = F.cross_entropy(model(x), y).item()
```

```
    for step in range(250):
        optimizer.zero_grad()
        loss = F.cross_entropy(model(x), y)
        loss.backward()
        optimizer.step()
    final = F.cross_entropy(model(x), y).item()
    assert final < initial
    embeddings[name] = model.input.weight.detach().clone()
    print(name, len(y), f'{initial:.3f} -> {final:.3f}')
print('vocabulary', len(vocab), 'embedding shape', tuple(embeddings['CBOW'].shape))
```

執行會印出 `CBOW 18 2.384 -> 0.159`、`Skip-gram 36 2.402 -> 1.198` 與 `vocabulary 11 embedding shape (11, 8)`。損失是平均負對數機率，以 nat 為單位。斷言確認實作能擬合這些訓練樣本。CBOW 損失較低，不表示它學出的詞向量較好：它的目標與條件輸入和 Skip-gram 不同。這個語料太小，無法判斷語意鄰近關係或對自然語言的泛化能力。

訓練後，`embeddings['CBOW'][ids['robots']]` 可取出一個學到的輸入向量。訓練需要輸出表，但若下游模型只查詢輸入向量，提供服務時可省略輸出表。應記錄保留哪個表；使用輸入向量、輸出向量或兩者組合，都是表示設計的選擇。

GloVe：直接擬合共現次數的對數

白話版。 GloVe 不是一次處理一個視窗、逐一預測，而是先計算每一對詞共同出現多少次，再擬合詞向量，讓它們的內積重現這些次數的對數。高頻配對得到較大權重，但權重有上限。

Pennington 等人以共現機率的比值說明 GloVe 的動機 [10]。令 X_{ik} 為詞 k 出現在詞 i 上下文中的（依距離加權的）次數，$P(k \mid i) = X_{ik} / \sum_{k'} X_{ik'}$。對兩個詞 i 與 j，比值 $P(k \mid i)/P(k \mid j)$ 在上下文詞 k 與 i 相關而與 j 無關時很大，反之很小；若 k 與兩者同樣相關或同樣無關，比值接近一。比值比任一個機率單獨更有區辨力。模型要求向量差編碼這類比值，最後得到對共現矩陣的加權最小平方目標：

$$J = \sum_{i,j=1}^{V} f\left(X_{ij}\right)\left(\mathbf{w}_i^{\top}\tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij}\right)^2, \qquad f(x) = \begin{cases} (x/x_{\max})^{\alpha} & x < x_{\max}, \\ 1 & \text{otherwise.} \end{cases}$$

其中 V 是詞彙表大小，X_{ij} 是詞 i 與 j 的共現次數，$\mathbf{w}_i \in \mathbb{R}^D$ 是 i 的詞向量，$\tilde{\mathbf{w}}_j \in \mathbb{R}^D$ 是 j 的另一個上下文向量，b_i 與 $\tilde{b}_j$ 是純量偏差，f 是加權函數，$x_{\max}$ 與 α 是它的截斷值與指數（式中 otherwise 意為「其他情況」）。因為 $f(0) = 0$，次數為零的儲存格完全不貢獻，所以永遠不需要 $\log 0$，加總實際上只涵蓋觀察到的配對。論文固定 $x_{\max} = 100$、$\alpha = 3/4$，使用相距 d 個詞的配對貢獻 $1/d$ 的共現加權，並以兩個表的和 $W + \tilde{W}$ 作為最終詞向量 [10]。

GloVe 與 Word2Vec 有三個設計差異。它以全域計數而不是個別視窗訓練，因此掃過語料一次建立 X，之後訓練時間與非零儲存格數成正比。偏差吸收詞頻，內積只需建模剩下的部分。加權函數則避免 `the` 這類極高頻配對主導，同時讓罕見而雜訊大的配對影響很小。與負取樣一樣，這個目標在秩限制下分解一個平移過的對數次數矩陣，這也是 GloVe 與 Skip-gram 向量實務上常表現相近的原因。

下列程式在先前的玩具語料上計算這些材料：共現比值與加權函數。

```
import numpy as np

text = """the cat drinks milk . the dog drinks water . a cat chases a mouse .
a dog chases a cat . the kitten drinks milk . the puppy drinks water .
a kitten chases a ball . a puppy chases a ball . the car needs fuel .
the truck needs fuel . a car drives on the road . a truck drives on the road ."""
sents = [s.split() for s in text.split(".") if s.strip()]
vocab = sorted({w for s in sents for w in s})
idx = {w: i for i, w in enumerate(vocab)}

X = np.zeros((len(vocab), len(vocab)))            # +-2 window, 1/distance weighting
for s in sents:
    for i, w in enumerate(s):
        for j in range(max(0, i - 2), min(len(s), i + 3)):
            if j != i:
                X[idx[w], idx[s[j]]] += 1.0 / abs(i - j)
P = X / X.sum(1, keepdims=True)                   # P(k | i): row-normalized counts

for k in ["drinks", "chases", "needs", "the"]:
    a, b = P[idx["kitten"], idx[k]], P[idx["truck"], idx[k]]
    ratio = "inf" if b == 0 else f"{a / b:.2f}"
    print(f"P({k}|kitten)={a:.3f}  P({k}|truck)={b:.3f}  ratio={ratio}")

def f(x, x_max=100.0, alpha=0.75):                 # GloVe weighting function
    return np.minimum((x / x_max) ** alpha, 1.0)
print("f at 1, 10, 100, 1000:", np.round(f(np.array([1.0, 10.0, 100.0, 1000.0])), 3))
print("nonzero cells:", int((X > 0).sum()), "of", X.size)
```

執行會印出：

```
P(drinks|kitten)=0.200  P(drinks|truck)=0.000  ratio=inf
P(chases|kitten)=0.200  P(chases|truck)=0.000  ratio=inf
P(needs|kitten)=0.000  P(needs|truck)=0.200  ratio=0.00
P(the|kitten)=0.200  P(the|truck)=0.200  ratio=1.00
f at 1, 10, 100, 1000: [0.032 0.178 1.    1.   ]
nonzero cells: 96 of 361
```

比值把動物上下文與車輛上下文分開，而 `the` 的比值為一，因此不提供資訊。輸出也暴露這個小語料的限制：矩陣將近四分之三是零，而 GloVe 忽略零值儲存格。觀察到的配對這麼少時，只有幾百個參數的模型就能精確擬合每個觀察到的儲存格，得到的幾何主要取決於初始化。GloVe 的計數式設計需要矩陣足夠充實的大型語料；因此這裡不訓練它。練習 3 會請你親自觀察這種欠定（underdetermination）現象。

fastText：由字元 n-gram 組成向量

白話版。詞級的表沒有看過的詞就沒有向量，而且把 `charger` 與 `chargers` 當成無關的兩列。fastText 用字元片段的向量組成每個詞的向量，因此拼寫相關的詞共享參數，沒見過的詞也能得

到向量。

Bojanowski 等人擴充使用負取樣的 Skip-gram，把每個詞表示為一袋字元 n-gram [11]。詞的前後加上邊界符號 < 與 >，讓前綴與後綴能和詞中間相同的字母區分開來，並把整個加上邊界的詞另外加入為一個單位。論文使用所有 $3 \le n \le 6$ 的 n-gram。中心詞 w 與上下文詞 c 的分數變成

$$s(w, c) = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g^\top \mathbf{v}_c.$$

其中 $\mathcal{G}_w$ 是 w 的 n-gram 集合（包含加上邊界的整個詞），$\mathbf{z}_g \in \mathbb{R}^D$ 是 n-gram g 的學習向量，$\mathbf{v}_c \in \mathbb{R}^D$ 是 c 的上下文向量。詞本身的向量是 $\sum_{g \in \mathcal{G}_w} \mathbf{z}_g$。這個分數取代上面負取樣損失中的 $\mathbf{u}_o^\top \mathbf{e}_c$。為了限制記憶體，論文以雜湊函數把 n-gram 映射到固定數量的桶，因此無關的 n-gram 可能共用同一個向量 [11]。

```
import numpy as np

def char_ngrams(word, n_min=3, n_max=6):
    w = f"<{word}>"                       # boundary symbols mark prefixes and suffixes
    grams = {w[i:i + n] for n in range(n_min, n_max + 1) for i in range(len(w) - n + 1)}
    return grams | {w}                    # the whole word is kept as one extra unit

print(sorted(char_ngrams("robot", 3, 3)))
vocab = ["charge", "charger", "recharge", "discharge", "robot", "robots"]
query = "chargers"                        # not in the vocabulary
q = char_ngrams(query)
for word in vocab:
    print(f"{word:10s} shares {len(q & char_ngrams(word)):2d} of {len(q)} units with
    ↪  '{query}'")

# A fastText-style vector is the sum of its unit vectors, so an unseen word still gets one.
rng = np.random.default_rng(0)
table = {}                                # stands in for a trained (hashed) n-gram table
def vector(word, dim=8):
    return sum(table.setdefault(g, rng.normal(size=dim)) for g in sorted(char_ngrams(word)))
def cos(a, b):
    return a @ b / (np.linalg.norm(a) * np.linalg.norm(b))
for word in vocab:
    vector(word)
print("cos(chargers, charger) =", round(cos(vector("chargers"), vector("charger")), 3))
print("cos(chargers, robots)  =", round(cos(vector("chargers"), vector("robots")), 3))
```

執行會印出：

```
['<ro', '<robot>', 'bot', 'obo', 'ot>', 'rob']
charge     shares 14 of 27 units with 'chargers'
charger    shares 18 of 27 units with 'chargers'
recharge   shares 10 of 27 units with 'chargers'
discharge  shares 10 of 27 units with 'chargers'
robot      shares  0 of 27 units with 'chargers'
robots     shares  0 of 27 units with 'chargers'
```

```
cos(chargers, charger) = 0.804
cos(chargers, robots)  = -0.222
```

這裡的 n-gram 向量是隨機的，沒有經過訓練，所以 `chargers` 與 `charger` 的高餘弦值完全來自它們共享的單位。這正是機制所在：沒看過的屈折形式，向量大多繼承自已知的片段。共享是否有幫助，由訓練決定。對屈折與衍生變化通常有幫助；對只是碰巧共享字母的詞則會誤導；`discharge` 與 `recharge` 都含 `charge`，但意義差異無法從片段看出。子詞向量對構詞豐富的語言與罕見詞最有幫助。也請注意它與NLP 02 子詞 tokenizer 的關係：BPE 在查表前把詞切成一串片段，fastText 則把所有重疊 n-gram 的向量加總。

稠密詞與文件表示

白話版。稠密向量讓不同詞共享統計資訊。它能連結相關片語，但向量幾何反映訓練資料與目標，不是通用的意義定義。

靜態向量可作為分類器的精簡可重用特徵、探索詞的近鄰，或建立平均池化的文件基準。當有標籤的應用資料不多，但有適合的文字語料可用時，它們很實用。嵌入訓練與下游分類器訓練是不同階段：預測鄰近詞不會直接學到 billing 標籤或拒答政策。

靜態嵌入表為每個詞彙項目配置向量：

$$\mathbf{e}_t = E_{i_t,:}, \qquad \mathbf{h}(d) = \frac{1}{T}\sum_{t=1}^{T} \mathbf{e}_t.$$

其中 $E \in \mathbb{R}^{V \times D}$ 是嵌入表，V 是詞彙表大小，D 是嵌入維度，i_t 是位置 t 的 token ID，$\mathbf{e}_t$ 是選出的橫向列向量，T 是文件的 token 長度，$\mathbf{h}(d)$ 是平均池化後的文件向量。冒號表示取出選定列的所有座標。

上面的各種表都提供取得 E 的方式。有了 token 向量，平均池化就能把不同長度的文件轉成固定寬度的特徵。這個彙整步驟與嵌入表的訓練目標是分開的。

平均池化很適合教學，因為限制很清楚：任意排列文件中的 token，都會得到同一個平均。若使用相同靜態詞向量，「the technician helped the customer」與「the customer helped the technician」的表示完全相同。否定詞也只是平均中的一份貢獻，並未自動成為改變片語意義的運算子。`bank` 在 `river bank` 與 `bank account` 會取出相同靜態向量；增加訓練資料不會改變這個查詢規則。

這裡有兩個獨立選擇：如何取得 token 向量，以及如何組合向量。可以凍結預訓練 token 向量，只訓練分類器；也可以與分類器一起微調向量，或從頭訓練全部參數。凍結能減少任務專屬參數；微調能配合領域中的詞義，但小型資料集也可能扭曲表示或記住標籤。

不要只靠漂亮的群聚圖比較嵌入。二維投影可能隱藏原空間中的重疊。應在相同任務、切分、分類器容量與調參額度下比較表示。如果換成稠密表示時也更換下游模型頭，實驗便無法單獨識別表示的影響。

餘弦量測方向，不量測真值

計數式範例比較的是已正規化為單位長度的向量。此時內積就是兩者夾角的餘弦：

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a}^\top \mathbf{b}}{\|\mathbf{a}\|_2 \|\mathbf{b}\|_2}.$$

其中 $\mathbf{a}$ 與 $\mathbf{b}$ 是非零的嵌入直向行向量，$\mathbf{a}^\top\mathbf{b}$ 是內積，$\|\cdot\|_2$ 是歐幾里得長度。餘弦介於 -1 與 1。零向量需要明確處理規則，因為分母為零。

餘弦消除任一向量乘上正比例常數的影響，但不會消除所有與訓練目標相容的變換。在兩個嵌入表的模型中，預測分數取決於輸入與輸出向量的乘積。許多不同的表可以產生相同乘積：

$$E' = EA, \qquad U' = UA^{-\top}, \qquad E'U'^\top = EU^\top.$$

其中 E 與 U 是形狀為 $V \times D$ 的輸入及輸出表，A 是任意可逆的 $D \times D$ 矩陣，$A^{-\top}$ 是其反矩陣的轉置，E' 與 U' 是變換後的表。$EU^\top$ 的每個儲存格是一個中心詞與輸出詞的預測分數。因此，這種配對變換不會改變完整 softmax 的預測與負取樣目標。同樣的論證也適用於 GloVe 的 $\mathbf{w}_i^\top \tilde{\mathbf{w}}_j$。

正交座標變換會保留輸入向量的夾角，一般可逆變換則未必。因此，相同預測損失無法唯一決定保留之輸入表的餘弦鄰近關係。最佳化、初始化與額外限制會選出某個解。這也是低訓練損失本身無法保證下游相似度實用的原因之一。

下列獨立數值檢查結合三件事：分數保留、平均運算的詞序不變性，以及上方推導出的雜訊相依最佳分數。矩陣是為了這個計算手動選定的，不是訓練出的語言向量。

```
import numpy as np

# Two input rows and three output rows: scores are center-output dot products.
E = np.array([[1., 1.], [1., -1.]])
U = np.array([[1., 0.], [0., 1.], [-1., 1.]])
A = np.diag([2., 0.5])
changed_E = E @ A
changed_U = U @ np.linalg.inv(A).T

def cosine(a, b):
    return (a @ b) / (np.linalg.norm(a) * np.linalg.norm(b))

assert np.allclose(E @ U.T, changed_E @ changed_U.T)
assert np.allclose(E[[0, 1]].mean(0), E[[1, 0]].mean(0))
print('scores unchanged:', np.allclose(E @ U.T, changed_E @ changed_U.T))
print('cosine:', f'{cosine(E[0], E[1]):.3f}', '->',
      f'{cosine(changed_E[0], changed_E[1]):.3f}')
print('mean unchanged by order:',
      np.allclose(E[[0, 1]].mean(0), E[[1, 0]].mean(0)))

# The unrestricted negative-sampling optimum depends on the noise baseline.
p, K = 0.5, 2
for q in [0.25, 0.05]:
    score = np.log(p / (K * q))
    sigmoid = 1 / (1 + np.exp(-score))
    gradient = (p + K * q) * sigmoid - p
    assert np.isclose(gradient, 0)
    print('noise', q, 'optimal score', f'{score:.3f}')
```

輸出如下：

```
scores unchanged: True
cosine: 0.000 -> 0.882
mean unchanged by order: True
noise 0.25 optimal score 0.000
noise 0.05 optimal score 1.609
```

餘弦改變了，所有中心詞與輸出詞的分數卻維持不變。平均相等則有不同含意：一旦表示合併兩種 token 順序，只收到平均值的分類器就無法還原原本是哪個順序。這些是表示的性質，不是某個最佳化器訓練不佳的證據。

評估詞嵌入

白話版。判斷嵌入有兩種方式：問它的幾何是否符合人對詞的判斷（內在評估），或把它接到任務上量測任務表現（外在評估）。單靠任何一種，都無法說明向量是否「好」。

相似度：讓餘弦與人類評分相關

詞相似度（word similarity）資料集列出詞對與分級的人類評分。模型以餘弦替每一對打分，評估則以 **Spearman 等級相關（Spearman's rank correlation）**量測兩種排序的一致程度：

$$\rho = 1 - \frac{6 \sum_{i=1}^{n} \delta_i^2}{n(n^2 - 1)}.$$

其中 n 是詞對數，δ_i 是第 i 對在模型餘弦排序與人類評分排序中的名次差，$\rho \in [-1, 1]$。這個公式假設沒有同名次；有同名次時，ρ 是名次的 Pearson 相關。Spearman 的 ρ 只用名次，因此忽略餘弦的尺度。

評分代表什麼，和統計量一樣重要。WordSim-353 是廣泛使用的詞對評分集 [12]。Hill 等人指出它的評分沒有區分相似度與聯想，因此建立 SimLex-999 明確量化**相似度**，讓有關聯但不相似的詞對得到低分 [13]。以寬視窗訓練的模型，可能在聯想式資料上得分高、在相似度式資料上得分低，原因如詞彙語意一節所述：分布式證據混合了各種關係。

類比：向量位移測試及其限制

Mikolov、Yih 與 Zweig 觀察到，某些關係會呈現為大致固定的向量位移 [14]。要回答「a 之於 a^*，如同 b 之於 ?」，**位移法（offset method）**傳回

$$x^* = \operatorname*{argmax}_{x \in \mathcal{U} \setminus \{a, a^*, b\}} \cos(\mathbf{e}_x,\ \mathbf{e}_{a^*} - \mathbf{e}_a + \mathbf{e}_b).$$

其中 $\mathcal{U}$ 是詞彙表，$\mathbf{e}_x$ 是候選 x 的向量，題目中的三個詞被排除在候選之外。Linzen 指出這個排除其實很關鍵：目標點 $\mathbf{e}_{a^*} - \mathbf{e}_a + \mathbf{e}_b$ 常常最接近 b 本身，因此這個方法的成功混合了位移一致性與 b 周圍的鄰近結構 [15]。他建議在類比準確率旁一併報告簡單基準，例如直接傳回 b 的最近鄰。

下一個範例用自編的相似度評分與三道類比題，評估先前的 PPMI 向量。評分是本課自行編造的，不是基準資料集。

```
import numpy as np
from scipy.stats import spearmanr

text = """the cat drinks milk . the dog drinks water . a cat chases a mouse .
a dog chases a cat . the kitten drinks milk . the puppy drinks water .
a kitten chases a ball . a puppy chases a ball . the car needs fuel .
the truck needs fuel . a car drives on the road . a truck drives on the road ."""
sents = [s.split() for s in text.split(".") if s.strip()]
vocab = sorted({w for s in sents for w in s})
idx = {w: i for i, w in enumerate(vocab)}
C = np.zeros((len(vocab), len(vocab)))
for s in sents:
    for i, w in enumerate(s):
        for j in range(max(0, i - 2), min(len(s), i + 3)):
            if j != i:
                C[idx[w], idx[s[j]]] += 1
P = C / C.sum()
with np.errstate(divide="ignore"):
    ppmi = np.maximum(np.log(P / (P.sum(1, keepdims=True) * P.sum(0, keepdims=True))), 0)
U, S, _ = np.linalg.svd(ppmi)
E = U[:, :8] * S[:8]
E /= np.linalg.norm(E, axis=1, keepdims=True)
cos = lambda a, b: float(E[idx[a]] @ E[idx[b]])

# 1) Similarity: correlate model cosines with (invented) graded judgments, 0-10.
judged = [("cat", "kitten", 9), ("dog", "puppy", 9), ("car", "truck", 8), ("milk", "water",
 ↪  5),
          ("cat", "dog", 6), ("cat", "mouse", 3), ("car", "road", 2), ("kitten", "fuel", 0)]
model = [cos(a, b) for a, b, _ in judged]
rho = spearmanr(model, [h for _, _, h in judged]).statistic
print(f"Spearman rho = {rho:.3f} over {len(judged)} pairs")

# 2) Analogy a:b :: c:? with the offset b - a + c (3CosAdd).
def analogy(a, b, c, exclude_inputs=True):
    target = E[idx[b]] - E[idx[a]] + E[idx[c]]
    scores = E @ (target / np.linalg.norm(target))
    for i in np.argsort(-scores):
        if not exclude_inputs or vocab[i] not in (a, b, c):
            return vocab[i]
for a, b, c in [("cat", "kitten", "dog"), ("car", "fuel", "cat"), ("cat", "milk", "puppy")]:
    print(f"{a}:{b} :: {c}:?  ->", analogy(a, b, c), "| without excluding inputs ->",
          analogy(a, b, c, exclude_inputs=False))
```

執行會印出：

```
Spearman rho = 0.837 over 8 pairs
cat:kitten :: dog:?  -> puppy | without excluding inputs -> puppy
car:fuel :: cat:?  -> kitten | without excluding inputs -> cat
cat:milk :: puppy:?  -> water | without excluding inputs -> water
```

有兩題得到預期答案。中間那題同時展示兩個限制。不排除題目詞時，答案是 `cat`，也就是題目中的詞。排除後，答案是 `kitten`，即 `cat` 的最近鄰，而不是預期的「牠吃喝什麼」答案 `milk`。位移並沒有承載這個關係；決定答案的是 b 的鄰近結構。當然，八組自編詞對與三道類比題，遠不足以支持關於這些向量的任何一般結論。

內在評估與外在評估

Schnabel 等人比較多種評估方法，發現不同評估對嵌入方法的排序不同、詞頻會影響結果與餘弦相似度，而且外在任務不應被當成通用嵌入品質的代理指標 [16]。實務原則是：針對應用需要的關係進行評估。

最近鄰清單能揭露明顯的語料模式，但它是診斷，不是應用分數。分開詞彙問題：候選是同義詞、反義詞、主題相關詞，還是能占據相同語法位置的詞？單一、未加限定的「相關」會掩蓋這些差別。

在一般留出樣本之外加入控制條件的配對。情緒表示應比較正面陳述與其否定；角色敏感任務應交換誰對誰做事，同時固定詞的集合；技術語料則測試術語的領域用法與日常用法。這些測試問的是表示是否保留必要差異，不要求每對詞都位於預設座標。

若平均池化把重要配對變成相同向量，在表示固定時，收集更多分類標籤也無法修復碰撞。若向量已區分配對，分類器卻預測錯誤，問題可能出在監督或決策邊界。從最早遺失必要證據的階段定位失敗。

嵌入中的偏誤

白話版。嵌入會學到語料中的任何規律，包括社會刻板印象。如果某一群名字與某些職業共同出現得比較多，向量就會編碼這種關聯，下游系統也可能把它複製出來。

Bolukbasi 等人顯示以新聞文字訓練的詞嵌入編碼了性別刻板印象，在空間中找出一個性別方向，並提出把它從應該性別中立的詞中移除 [17]。Caliskan 等人提出仿照心理學內隱聯結測驗（Implicit Association Test）的詞嵌入聯結測驗（Word Embedding Association Test，WEAT），發現嵌入重現了一系列已有文獻記錄的人類偏誤 [18]。對目標詞 w 與兩個屬性集合 A、B，WEAT 使用的關聯為

$$s(w, A, B) = \frac{1}{|A|} \sum_{a \in A} \cos(\mathbf{e}_w, \mathbf{e}_a) - \frac{1}{|B|} \sum_{b \in B} \cos(\mathbf{e}_w, \mathbf{e}_b).$$

其中 $|A|$ 與 $|B|$ 是集合大小，$\mathbf{e}$ 表示詞向量。正值代表 w 平均而言較接近集合 A。完整測試會在兩組目標詞上彙總這個分數，並與排列分布比較。

下列自編語料中，沒有任何句子把名字和職業連在一起。職業詞總是與自己的活動一起出現，而自編的名字以偏斜的比例與各種活動一起出現。

```
import random
import numpy as np

rng = random.Random(0)
```

```
# An invented corpus. Job words always appear with their own activity; invented
# names appear with activities at skewed rates. No sentence says "ana is a pilot".
group_a, group_b = ["ana", "ari"], ["bo", "bea"]
activity = {"pilot": "flies the plane", "baker": "bakes the bread", "clerk": "files the
 ↪  forms"}
share_a = {"pilot": 0.8, "baker": 0.2, "clerk": 0.5}    # P(group A | activity)
sents = []
for _ in range(3000):
    job = rng.choice(list(activity))
    if rng.random() < 0.5:
        sents.append(f"the {job} {activity[job]}".split())
    else:
        name = rng.choice(group_a if rng.random() < share_a[job] else group_b)
        sents.append(f"{name} {activity[job]}".split())
vocab = sorted({w for s in sents for w in s})
idx = {w: i for i, w in enumerate(vocab)}
C = np.zeros((len(vocab), len(vocab)))
for s in sents:
    for i, w in enumerate(s):
        for j in range(max(0, i - 2), min(len(s), i + 3)):
            if j != i:
                C[idx[w], idx[s[j]]] += 1
P = C / C.sum()
with np.errstate(divide="ignore"):
    ppmi = np.maximum(np.log(P / (P.sum(1, keepdims=True) * P.sum(0, keepdims=True))), 0)
U, S, _ = np.linalg.svd(ppmi)
E = U[:, :4] * S[:4]
E /= np.linalg.norm(E, axis=1, keepdims=True)

def association(word, E):            # mean cosine to group A minus mean cosine to group B
    unit = E / np.linalg.norm(E, axis=1, keepdims=True)
    w = unit[idx[word]]
    return np.mean([w @ unit[idx[a]] for a in group_a]) - np.mean([w @ unit[idx[b]] for b in
     ↪  group_b])

for job in activity:
    print(f"{job:6s} association {association(job, E):+.3f}")

# Project out one "group direction" and measure again.
d = E[[idx[a] for a in group_a]].mean(0) - E[[idx[b] for b in group_b]].mean(0)
d /= np.linalg.norm(d)
E_proj = E.copy()
for job in activity:
    E_proj[idx[job]] -= (E_proj[idx[job]] @ d) * d
print("after projection:", [f"{job} {round(association(job, E_proj), 3) + 0.0:+.3f}" for job
 ↪  in activity])
```

執行會印出：

```
pilot  association +0.795
baker  association -0.811
clerk  association +0.072
```

```
after projection: ['pilot +0.000', 'baker +0.000', 'clerk +0.000']
```

「誰從事哪種活動」的偏斜，變成名字與職業詞之間的強烈關聯，即使兩者從未在同一句中出現：它們共享活動上下文。投影掉群組方向後，量測到的關聯變成零，但這個結果是循環論證，因為投影移除的正是這個分數所量測的方向。Gonen 與 Goldberg 顯示，對已發表的去偏方法而言，偏誤大多只是被隱藏而非移除：它仍留在那些號稱已中立化的詞彼此間的距離中，而且可以被還原 [19]。應把去偏步驟視為需要在下游任務上稽核的一項介入，而不是保證，並記得關聯的來源在資料本身。

從詞向量到句子嵌入

白話版。嵌入提供特徵；應用仍需要目標、監督與行動規則。當應用要比較整個句子時，就針對它要支援的比較方式訓練句子表示。

文件分類可以先做平均池化實驗。替每份訓練文件建立一個向量，再以標籤配適分類器；評估時使用相同 tokenizer、嵌入查表與池化方法。將它與NLP 03 的稀疏特徵與卷積編碼器比較。固定資料切分與分類器的調整機會，才能公平檢查表示改變的效果。

凍結的表使用標註任務之前學到的上下文統計；微調則讓標籤調整每列。兩種政策回答不同問題：語料中可重用的特徵是否已能區分類別，以及任務特定的調整是否改善表示。標籤有限時，微調也可能配適偶然出現的識別碼。將對話或重複文件模板等群組完整留出，有助於揭露這種行為。

決定如何處理未涵蓋文字。把每個未知詞映射到同一列，會合併所有這類詞。略過它們會改變池化分母，也可能使文件變空。回傳零向量不會形成有意義的餘弦方向。子詞向量能減輕、但無法消除這個問題。涵蓋率、保留 token 數，以及空表示的處理動作，都是模型規格的一部分。

詞層級標記應保留 token 向量序列，而不是池化成單一文件向量。下游編碼器就能先結合局部上下文，再為各位置預測標籤。這不會讓原本的表成為上下文表示：兩種詞義的 `bank` 初始查表值仍相同。上下文相依的計算發生在後續。

相似度與對比式句子學習

適合預測鄰近詞的表示，可能需要調適，才能讓整個句子的距離可靠。Sentence-BERT 使用共享編碼器與任務目標訓練成對的句子表示，讓比較句子時不必對每一對候選共同編碼 [20]。計算上的差別很重要：分別快取文件向量可以支援檢索；同時讀取查詢與候選文件的模型，則必須為每一對執行。

一般的對比目標能把想要的幾何關係寫清楚：

$$\mathcal{L}_{\text{pair}} = -\log \frac{\exp(\cos(\mathbf{q}, \mathbf{a}^{+})/\tau)}{\sum_{j=1}^{M} \exp(\cos(\mathbf{q}, \mathbf{a}_j)/\tau)}.$$

其中 $\mathcal{L}_{\text{pair}}$ 是一個查詢的損失，$\mathbf{q}$ 是查詢向量，$\mathbf{a}^{+}$ 是候選中與查詢相關的配對文字向量，$\mathbf{a}_j$ 是候選 j 的向量，M 是候選數，$\tau > 0$ 是控制分布尖銳程度的溫度。這是一般性的學習構造，不表示

每個句子編碼器都使用完全相同的目標。它和 Word2Vec 的 softmax 形式相同：一個觀察到的配對與其他候選競爭。

SimCSE 是廣泛使用的一個實例 [21]。它的非監督版本把同一個句子編碼兩次；由於兩次前向傳遞的 dropout 不同，兩個向量略有差異而形成正配對，批次中的其他句子則是負樣本。作者報告移除 dropout 會導致表示崩塌。它的監督版本使用自然語言推論資料，以蘊涵配對作為正樣本、矛盾配對作為困難負樣本 [21]。

負樣本定義了什麼叫「不同」。兩則討論不同發票的客服訊息，對尋找同一案件可能是實用的負樣本；對辨識 billing 類別則可能是不好的負樣本。錯誤負樣本會把語意相關的例子推遠。在改變編碼器之前，先檢查正負配對如何取樣。

相似度也不等於蘊涵。「The parcel arrived」與「The parcel did not arrive」共享主題與許多 token，在語意搜尋空間可能靠近，卻做出相反陳述。因此，即使使用相同編碼器，分類、語意搜尋與推論仍需要不同的驗證樣本。NLP 11 把推論當成獨立任務處理。

預告：上下文嵌入

白話版。靜態表在任何地方都給 `bank` 同一個向量。上下文模型則利用周圍的詞，替每次出現計算不同的向量，讓河岸詞義與金錢詞義得以分開。

Peters 等人將 ELMo 表示計算為整個輸入句子的函數，使用雙向語言模型的內部狀態，並把它們作為特徵加入任務模型 [22]。BERT 在預訓練編碼器內更進一步：token 最後的隱藏狀態取決於其他允許看到的輸入 token [23]。差別是**查表與計算**。兩者都可能從嵌入表開始；靜態 Word2Vec 直接提供一列，BERT 則透過上下文相依網路處理各列與位置。

上下文能改善詞義區分，但不會讓每個隱藏狀態的餘弦自動成為已校準的相似度量。靜態向量是跨越許多次出現的類型層級摘要；上下文向量是一次出現的 token 層級表示。應用需要哪一種，取決於它問的是一般意義下的詞，還是此處用法中的詞。NLP 07 建立這些模型使用的注意力機制，NLP 10 則發展遮罩預訓練與 BERT 的上下文表示，這裡不重複。

現代觀點

Turney 與 Pantel 的綜述 [6] 區分詞項與文件、詞與上下文，以及配對與模式矩陣。這種分類使證據來源明確：文件關聯、鄰近詞關係與關係模式是不同觀察。本課的啟示是，先定義矩陣，再解讀向量空間。

Apidianaki 的 2023 年綜述 [24] 從靜態類型層級向量，走向上下文相依的出現層級表示，再討論如何回到靜態表示。內容涵蓋語意特化、詞義區分、表示解讀，以及從上下文編碼器取得靜態向量。核心區別是在哪個層級彙整證據，不是把架構排成「越新就自動解決所有詞彙任務」的名次。把詞彙資源中的關係注入向量的方法，與本課的 WordNet 一節直接相關。

在句子與文字層級的嵌入方面，Muennighoff 等人的 MTEB 基準涵蓋 8 種嵌入任務、58 個資料集與 112 種語言，作者發現沒有任何單一文字嵌入方法在所有任務上都占優勢 [25]。在 2023–2026 年間，多數實用的文字嵌入來自以上述對比目標微調的預訓練 Transformer 編碼器或語言模型，而 MTEB 風格的基準套件是比較它們的常見方式。靜態向量仍是快速、透明的基準，是低資源情境中的特徵，也是研究對象。

因此，適合檢視研究的問題是：**這種表示保留了哪個詞義區別，又如何驗證？**上下文隱藏狀態、由它們聚合而來的靜態向量，以及以任務訓練的句子表示，回答不同問題。能保留大致主題的嵌入，仍可能合併否定陳述或罕見詞義。如果小型靜態表的可測不變性符合應用，它仍是適當基準。

開放問題包括跨領域可靠幾何、證據稀少時的詞義區分、去偏後仍存留的偏誤，以及固定語料與評估條件的比較。多語工作還加入斷詞與不均衡涵蓋率：tokenizer 與訓練語料曾為該類型提供證據，這列才有用。應與最終分數一起報告這些條件與挑戰樣本，讓解釋連結到實際觀察的語言行為。

重點整理

- 詞彙表項目是詞形或詞元，不是詞義；同義、反義、上位等詞彙關係存在於詞義之間，WordNet 以人工記錄它們。
- 靜態嵌入彙整許多次出現，為每個詞彙表項目形成一個向量；分布式相似度捕捉共享用法，混合了同義、反義與相關性。
- PPMI 將共現與獨立基準比較；SVD、使用負取樣的 Skip-gram 與 GloVe，都在秩限制下壓縮關聯統計。
- CBOW 從合併的鄰近詞預測中心詞；Skip-gram 分別預測鄰近詞；負取樣區分配對的統計來源，不是正規化分布。
- fastText 以字元 n-gram 組成詞向量，因此拼寫相關的詞共享參數，沒見過的詞也有向量。
- 相似度與類比測試取決於評分的意義，以及是否排除題目詞等細節；應評估應用需要的關係。
- 嵌入會吸收語料中的關聯，包括社會偏誤；以投影去偏可能只是隱藏而非移除偏誤。
- 平均池化消除詞序；句子嵌入應針對它服務的比較訓練，而上下文模型是替每次出現計算向量，而不是查表。

練習

1. **建立樣本**。列出 `robots help people today` 的半徑一 CBOW 與 Skip-gram 樣本，先只使用內部中心位置，再納入邊界中心。哪些輸入形狀會改變？

   提示

   中心位置數與中心、上下文配對數分開計算。長度不同的 CBOW 上下文需要遮罩或分開處理；Skip-gram 每個配對仍只收到一個中心 ID。

2. **檢查矩陣儲存格**。建立一份語料，讓某個上下文整體很常見，卻對選定中心詞具有接近零的 PMI。再增加觀察到的中心與上下文配對，解釋分子與邊際機率如何改變。

   提示

   先從接近獨立的聯合表開始。修改計數後重算所有邊際；把邊際固定會變成分析另一張表。

3. **訓練 GloVe，看它搖擺**。在 PyTorch 中實作 GloVe 目標，於玩具語料上只用非零儲存格訓練。以三個隨機種子訓練，比較 `cat` 的最近鄰。接著計算參數數與非零儲存格數。是什麼造成不穩定？怎樣能讓問題的解更確定？

   提示

   兩個 $V \times D$ 的表加上兩個偏差向量，參數數很容易超過觀察到的儲存格數。上下文更多樣的大語料會增加方程式；較小的 D 則減少未知數。

4. **相似度與相關性**。寫出十組相關但不相似的詞對（例如 `cup` 與 `coffee`），以及十組相似的詞對。在你自己編寫的語料上，以視窗半徑 1 與 4 分別執行 PPMI 程式，比較餘弦如何區分兩組。

   提示

   較寬的視窗收集主題共現，有利於相關性；較窄的視窗強調能填入同一位置的詞。兩組的變化都要報告，而不是只給單一分數。

5. **揭露不變性**。建立兩句詞計數相同、參與者角色不同的有意義句子。即使每個詞向量都完美符合其訓練目標，說明靜態平均池化為何仍無法區分兩句。

   提示

   改變順序不會改變加總。這個限制對任何靜態嵌入表成立，不只對本課的合成數值成立。

6. **在投影之外稽核**。在偏誤範例中完成投影後，用剩下的所有維度計算每個職業詞最接近的名字，並訓練一個小型邏輯斯分類器，從投影後職業詞的鄰居預測 A 組或 B 組。關聯是否仍留下痕跡？

   提示

   投影只以一種方式移除了一個方向。尋找這個分數沒有量測到的結構，就像 Gonen 與 Goldberg 以分群與分類所做的那樣。

參考文獻

1. M. Hagiwara, *Real-World Natural Language Processing: Practical Applications with Deep Learning*, Manning, 2021; Chapters 3–4. publisher
2. S. McRoy, *Principles of Natural Language Processing*, open textbook, 2021; Chapter 6, “Benchmark Tasks for Language Modelling.” chapter
3. Y. Bengio, R. Ducharme, P. Vincent and C. Jauvin, “A neural probabilistic language model,” *Journal of Machine Learning Research* 3, 2003. paper
4. G. A. Miller, “WordNet: a lexical database for English,” *Communications of the ACM* 38(11), 1995. paper

5. Z. S. Harris, “Distributional structure,” *Word* 10(2–3), 1954. paper
6. P. D. Turney and P. Pantel, “From frequency to meaning: vector space models of semantics,” *Journal of Artificial Intelligence Research* 37, 2010. survey
7. T. Mikolov, K. Chen, G. Corrado and J. Dean, “Efficient estimation of word representations in vector space,” ICLR Workshop, 2013. paper
8. T. Mikolov, I. Sutskever, K. Chen, G. Corrado and J. Dean, “Distributed representations of words and phrases and their compositionality,” *Advances in Neural Information Processing Systems* 26, 2013. paper
9. O. Levy and Y. Goldberg, “Neural word embedding as implicit matrix factorization,” *Advances in Neural Information Processing Systems* 27, 2014. paper
10. J. Pennington, R. Socher and C. D. Manning, “GloVe: global vectors for word representation,” EMNLP, 2014. paper
11. P. Bojanowski, E. Grave, A. Joulin and T. Mikolov, “Enriching word vectors with subword information,” *Transactions of the Association for Computational Linguistics* 5, 2017. paper
12. L. Finkelstein, E. Gabrilovich, Y. Matias, E. Rivlin, Z. Solan, G. Wolfman and E. Ruppin, “Placing search in context: the concept revisited,” *ACM Transactions on Information Systems* 20(1), 2002. paper
13. F. Hill, R. Reichart and A. Korhonen, “SimLex-999: evaluating semantic models with (genuine) similarity estimation,” *Computational Linguistics* 41(4), 2015. paper
14. T. Mikolov, W.-t. Yih and G. Zweig, “Linguistic regularities in continuous space word representations,” NAACL-HLT, 2013. paper
15. T. Linzen, “Issues in evaluating semantic spaces using word analogies,” Workshop on Evaluating Vector-Space Representations for NLP (RepEval), 2016. paper
16. T. Schnabel, I. Labutov, D. Mimno and T. Joachims, “Evaluation methods for unsupervised word embeddings,” EMNLP, 2015. paper
17. T. Bolukbasi, K.-W. Chang, J. Zou, V. Saligrama and A. Kalai, “Man is to computer programmer as woman is to homemaker? Debiasing word embeddings,” *Advances in Neural Information Processing Systems* 29, 2016. paper
18. A. Caliskan, J. J. Bryson and A. Narayanan, “Semantics derived automatically from language corpora contain human-like biases,” *Science* 356(6334), 2017. paper
19. H. Gonen and Y. Goldberg, “Lipstick on a pig: debiasing methods cover up systematic gender biases in word embeddings but do not remove them,” NAACL-HLT, 2019. paper
20. N. Reimers and I. Gurevych, “Sentence-BERT: sentence embeddings using Siamese BERT-networks,” EMNLP-IJCNLP, 2019. paper
21. T. Gao, X. Yao and D. Chen, “SimCSE: simple contrastive learning of sentence embeddings,” EMNLP, 2021. paper
22. M. E. Peters, M. Neumann, M. Iyyer, M. Gardner, C. Clark, K. Lee and L. Zettlemoyer, “Deep contextualized word representations,” NAACL-HLT, 2018. paper
23. J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: pre-training of deep bidirectional transformers for language understanding,” NAACL-HLT, 2019. paper
24. M. Apidianaki, “From word types to tokens and back: a survey of approaches to word meaning representation and interpretation,” *Computational Linguistics* 49(2), 2023. survey
25. N. Muennighoff, N. Tazi, L. Magne and N. Reimers, “MTEB: massive text embedding benchmark,” EACL, 2023. paper

NLP 05・用於語言的循環神經網路

從 n-gram 語言模型及其記憶上限出發，建立、訓練並評估循環語言模型：Elman RNN、LSTM 與 GRU、字元層級建模、取樣，以及實務訓練技巧。

網站版本: https://blog.shyandram.dev/zh-tw/learn/nlp/nlp-05-recurrent-neural-networks/

你將學到

- 分清楚 n-gram 出現、文件特徵與條件語言模型，並從序列機率推導 unigram、bigram 與 trigram 預測。
- 計算、平滑與內插 n-gram 估計，再解讀保留資料的負對數概似及困惑度。
- 說明固定長度歷史為何造成更多資料也無法移除的記憶上限。
- 寫出 Elman 循環語言模型、其下一詞元交叉熵與困惑度，並說明時間反向傳播、梯度消失、LSTM 與 GRU 單元如何影響模型能學到什麼。
- 以串流打包、截斷時間反向傳播、梯度裁剪與提前停止訓練字元層級 LSTM 語言模型，並與調校過的 n-gram 基準公平比較。
- 以溫度從訓練好的模型取樣，並說明樣本能與不能告訴我們什麼。

先看全貌

語言模型根據前文預測下一個詞元，藉此替文字配置機率。本章建立兩種這樣的模型，並在同一套評估約定下比較。第一種是 n-gram 模型，它使用前文的一小部分預測下一個詞元。機率表可以直接檢查：我們能把預測追溯到訓練次數，並看清楚模型忽略了哪些較早的詞。因此，它適合作為第一個語言模型、受控基準，以及理解更複雜模型也會遇到的統計問題。第二種是循環神經網路（recurrent neural network，RNN），它以一個在每個詞元之後更新的學習狀態取代次數表，因此預測原則上可以依賴整個前綴。

> **用最簡單的話說**
>
> 想像一本記錄大家接著說什麼的筆記本。一頁記錄「cat」後面接什麼，另一頁記錄「the cat」後面接什麼。需要猜測時，就翻到對應頁面並計算範例次數。較長的頁面標題提供更多上下文，卻也可能讓許多頁面幾乎空白。平滑會替這一頁還沒出現過的詞保留一點空間。循環網路的做法不同：它不替每個標題準備一頁，而是在腦中保留一段簡短的摘要，每聽到一個詞就改寫一次。

Hagiwara 的語言模型章節把下一詞元預測連到實際文字生成 [1]。McRoy 的教材提供更完整的語言及應用背景 [2]。分詞放在NLP 01；嵌入及其訓練目標放在NLP 04。第 1–5 節專注於計數模型本身；第 6–12 節以循環網路取代次數表；以注意力為基礎的模型則在NLP 07 與NLP 09 介紹。循環網路的一般數學（完整的時間反向傳播推導、梯度界限、閘門方程式）在ML 03A；這裡採取語言模型的觀點。讀本章時持續追問：**哪些資訊與觀察證據決定下一個詞元的機率？**

1. 「n-gram」的三種意思

n-gram 是連續 n 個詞元組成的序列。詞元可以是詞、子詞或字元；分詞方式決定計算的單位。以詞序列 `a cat sleeps` 為例，unigram 是 `a`、`cat`、`sleeps`；bigram 是 `a cat`、`cat sleeps`；trigram 則是 `a cat sleeps`。

以下三種用途要分清楚：

用途	儲存或預測的物件	能告訴我們什麼
n-gram 出現	一個連續詞元元組	某個序列的局部片段
文件特徵	一個元組在文件中的次數或是否出現	提供給分類器或檢索模型的證據
n-gram 語言模型	每個保留歷史對應的正規化下一詞元分布	用來替序列評分或生成的機率

文件的 bigram 特徵 `not ready` 可以協助分類器區分操作狀態，但它本身沒有指定 `not` 後面出現 `ready` 的機率：還需要其他競爭詞元的次數與正規化規則。同樣地，語言模型中的片語機率取決於句子上下文；文件特徵次數通常會合併該片語在整篇文件中的所有出現。

語言模型的階數 n 包含正在預測的詞元。Bigram 使用**前一個詞元**，不是前兩個；trigram 使用前兩個。句子邊界需要一致規則：起始標記提供最初上下文，結束標記則是代表停止的預測結果。範例從不預測起始標記；若把它放進預測詞彙表，就會替句子格式不允許的事件配置機率。

2. 精確分解與有限歷史

逐一預測詞元，可以精確表示序列機率。捨棄較早上下文是模型假設，並不是機率理論要求這樣做。

$$p(x_1, \dots, x_T) = \prod_{t=1}^{T} p(x_t \mid x_{<t}), \qquad p_n(x_1, \dots, x_T) = \prod_{t=1}^{T} p_n(x_t \mid h_t^{(n)}).$$

其中 x_t 是位置 t 的詞元；T 是預測位置數，若使用結束標記也將它計入；$x_{<t}$ 是完整前綴；n 是 n-gram 階數；$h_t^{(n)}$ 是最多包含前 $n-1$ 個詞元的尾端歷史，需要時補上起始標記；p_n 是有限歷史模型。第一個等式是鏈式法則，第二個式子則定義只依保留尾端內容預測的模型。它不一定等於真實序列分布。

階數	在 `a cat sleeps` 中預測 `sleeps` 前的上下文	條件分布
Unigram，$n = 1$	空歷史	$p_1(\text{sleeps})$
Bigram，$n = 2$	`cat`	$p_2(\text{sleeps} \mid \text{cat})$
Trigram，$n = 3$	`a cat`	$p_3(\text{sleeps} \mid \text{a cat})$

空歷史仍是有效上下文：unigram 模型在每個位置使用同一個下一詞元分布。Bigram 做出更細的區分，卻必須合併所有最後一個詞元相同的前綴。Trigram 可以區分最後兩個詞元不同的前綴，但這些模型都不保證保留內容包含語言依存關係真正需要的資訊。

逐步計算不同階數的差異

下面的教學語料刻意讓冠詞帶有預測資訊：以 `the` 開頭的範例都以 `sleeps` 結尾，以 `a` 開頭的範例都以 `eats` 結尾。這是受控的統計模式，不是對英語用法的主張。範例使用相同詞彙表及平滑強度，分別估計三種階數。

```
from collections import Counter, defaultdict

train = ["the cat sleeps", "the dog sleeps", "a cat eats", "a dog eats"]
START, END, UNK = "<s>", "</s>", "<unk>"
vocab = sorted({w for s in train for w in s.split()} | {END, UNK})
alpha = 0.2
models = {}
for order in (1, 2, 3):
    rows = defaultdict(Counter)
    for sentence in train:
        tokens = [START] * (order - 1) + sentence.split() + [END]
        for t in range(order - 1, len(tokens)):
            history = tuple(tokens[t-order+1:t])
            rows[history][tokens[t]] += 1
    models[order] = rows

def probability(order, history, word):
    row = models[order].get(tuple(history), {})
    return (row.get(word, 0) + alpha) / (sum(row.values()) + alpha * len(vocab))

for order, rows in models.items():
    for history in rows:
        assert abs(sum(probability(order, history, w) for w in vocab) - 1) < 1e-10
print("vocabulary size:", len(vocab))
print("unigram P(eats):", round(probability(1, (), "eats"), 3))
print("bigram P(eats | cat):", round(probability(2, ("cat",), "eats"), 3))
print("trigram P(eats | a cat):", round(probability(3, ("a", "cat"), "eats"), 3))
print("trigram P(eats | the cat):", round(probability(3, ("the", "cat"), "eats"), 3))
print("unseen-history P(eats):", round(probability(3, (UNK, "cat"), "eats"), 3))
assert probability(3, ("a", "cat"), "eats") > probability(3, ("the", "cat"), "eats")
assert abs(probability(3, (UNK, "cat"), "eats") - 1 / len(vocab)) < 1e-10
```

輸出：

```
vocabulary size: 8
unigram P(eats): 0.125
bigram P(eats | cat): 0.333
trigram P(eats | a cat): 0.462
trigram P(eats | the cat): 0.077
unseen-history P(eats): 0.125
```

在 `cat` 後面，bigram 次數表有一次 `eats` 與一次 `sleeps`。兩種冠詞選擇都進入同一張表。Trigram 則把兩筆觀察分開：`a cat` 曾接過一次 `eats`，`the cat` 則沒有。因此 trigram 能表達這個

差異，但它的每張表都只有一筆觀察。在這份語料中，更細的條件區分會減少每個上下文可用的證據。

未見歷史沒有任何轉移次數；加法平滑使其成為八個預測詞元上的均勻分布。這是完整的機率分布，卻捨棄了有用證據：最後一個詞元 `cat` 其實出現在訓練資料中。與低階分布內插，可以保留這項證據。正規化與有效泛化是不同性質。

3. 從次數到正規化機率

要估計分布，先計算訓練中的每個「歷史—目標」事件，再除以該歷史的總轉移次數。讓結果可重現的兩項細節是：哪些符號會被預測，以及哪些上下文被視為不同。

$$\hat{p}(w \mid h) = \frac{c(h, w)}{c(h)}, \qquad c(h) = \sum_{v \in V} c(h, v), \qquad c(h) > 0.$$

其中 h 是保留歷史；w 是候選下一詞元；V 是預測詞彙表；v 遍歷詞彙表成員；$c(h, w)$ 是訓練中由 h 接到 w 的次數；$c(h)$ 是其總數；$\hat{p}$ 是最大概似估計。Trigram 的 h 是兩個詞元的元組，unigram 則是空元組。若歷史從未出現，這個估計沒有定義。

為什麼觀察相對頻率能解出概似問題？對單一歷史，在機率總和為一的限制下，最大化 $\sum_w c(h, w) \log p(w \mid h)$ 的分布，會讓有正次數的事件機率與次數成正比。正規化限制接著使它除以 $c(h)$。不同歷史可以獨立估計，因為計數表沒有在各歷史之間共享參數。

這種簡單性帶來兩種不同問題。**未見轉移**的歷史曾出現，但目標從未接在後面，因此機率為零；**未見歷史**則完全沒有次數表，使未平滑比例沒有定義。平滑能在固定詞彙表內處理兩者，卻無法生成已被詞彙表替換成未知標記的原始拼字。

加法平滑替每個可能的目標加上少量正次數：

$$p_\alpha(w \mid h) = \frac{c(h, w) + \alpha}{c(h) + \alpha |V|}.$$

其中 p_α 是平滑後分布；$c(h, w)$ 與 $c(h)$ 分別是轉移次數及歷史總轉移次數；w 是候選詞元；h 是保留歷史；V 是預測詞彙表；$|V|$ 是其大小；$\alpha > 0$ 是加到每個候選次數上的數量。把分子對整個詞彙表加總，恰好得到分母。即使歷史從未出現，分布仍有定義：所有候選都得到機率 $1/|V|$。

詞彙表大小與加到單一候選上的次數同樣重要。詞彙表很大時，替每個詞元加上 α 可能占去大量總機率。在這個微小詞彙表上有效的參數，不能未經驗證就原封不動移用到大得多的詞彙表。

程式碼：訓練、評分與取樣一個 bigram 模型

這個完整範例不需要下載資料或 GPU。語料是為教學編寫的；測試句子與訓練句子分開。測試中任何不在訓練詞彙表內的詞，都對應到同一個未知 token。

```
from collections import Counter, defaultdict
import math
import random
```

```
train = [
    "the cat sleeps", "the dog sleeps", "the cat runs",
    "the dog runs", "a cat eats", "a dog eats",
    "the bird sings", "a bird sleeps",
]
test = ["the cat eats", "a dog runs", "the bird sleeps", "a robot runs"]
START, END, UNK = "<s>", "</s>", "<unk>"
words = {w for sentence in train for w in sentence.split()}
vocab = sorted(words | {END, UNK})       # START is context, never predicted
alpha = 0.2
counts = defaultdict(Counter)
unigrams = Counter()

def encode(sentence):
    return [w if w in words else UNK for w in sentence.split()] + [END]

for sentence in train:
    tokens = [START] + encode(sentence)
    for h, w in zip(tokens, tokens[1:]):
        counts[h][w] += 1
        unigrams[w] += 1

def bigram(h, w):
    row = counts.get(h, {})
    return (row.get(w, 0) + alpha) / (sum(row.values()) + alpha * len(vocab))

def unigram(h, w):
    return (unigrams[w] + alpha) / (sum(unigrams.values()) + alpha * len(vocab))

def perplexity(sentences, probability):
    nll, n = 0.0, 0
    for sentence in sentences:
        tokens = [START] + encode(sentence)
        for h, w in zip(tokens, tokens[1:]):
            nll -= math.log(probability(h, w))
            n += 1
    return math.exp(nll / n)

for h in [START] + vocab:
    assert abs(sum(bigram(h, w) for w in vocab) - 1) < 1e-10
print("unigram test perplexity:", round(perplexity(test, unigram), 3))
print("bigram test perplexity:", round(perplexity(test, bigram), 3))
assert UNK in encode(test[-1])

rng = random.Random(7)
h, sample = START, []
for _ in range(20):                      # cap length for this demonstration
    w = rng.choices(vocab, weights=[bigram(h, v) for v in vocab])[0]
    if w == END:
        break
    sample.append(w)
    h = w
print("sample:", " ".join(sample))
```

輸出：

```
unigram test perplexity: 10.382
bigram test perplexity: 3.573
sample: a bird sings
```

assert 檢查每一個條件分布加總為一，以及「robot」被對應到未知 token。bigram 模型對測試句子的「驚訝程度」遠低於 unigram 模型，因為詞序帶有資訊。取樣仍可能產生彆扭的字串：機率大於零不代表合乎文法。長度上限只是讓示範停下來，並不改變評分所用的機率。

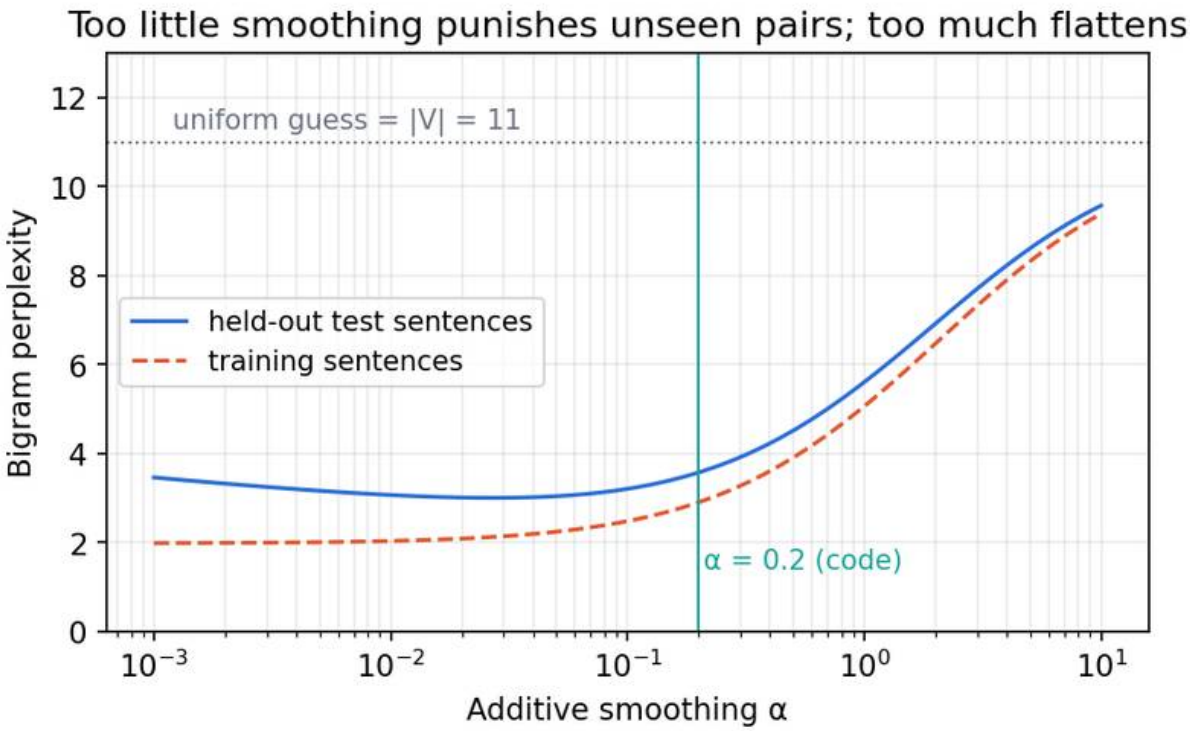


圖 *1* — 平滑替未觀察轉移保留機率。合成保留資料曲線說明取捨；四個保留句子無法證明一般情況的最佳設定。

分清楚平滑與更長的記憶

平滑處理的是既定表示下尚未觀察到的轉移，無法補回已捨棄的歷史。在 “the technician did not repair the” 與 “the technician repaired the” 之後，bigram 都只看見 `the`。即使更早的詞會讓不同延續較合適，它仍必須給出相同的下一詞元分布。增加語料可以更準確地估計這個共同分布，卻不會改變兩者必須相等的限制。

加法平滑也有精確的混合解釋。對觀察次數大於零的上下文：

$$p_\alpha(w \mid h) = \lambda_h \hat{p}(w \mid h) + (1 - \lambda_h)\frac{1}{|V|}, \qquad \lambda_h = \frac{c(h)}{c(h) + \alpha|V|}, \qquad \hat{p}(w \mid h) = \frac{c(h, w)}{c(h)}.$$

其中 p_α 是平滑後的分布，h 是保留的歷史，w 是候選詞元，$c(h,w)$ 是該轉移的觀察次數，$c(h) > 0$ 是從此歷史出發的總次數，$\hat{p}$ 是未平滑估計，$\alpha > 0$ 是平滑參數，$|V|$ 是詞彙表大小，λ_h 則是觀察資料的權重。計數為零的上下文直接使用均勻分布。

相同的平滑參數對罕見上下文影響較大，對常見上下文影響較小。上下文次數增加時，觀察證據得到較大的權重；平滑強度增加時，機率則移向均勻分布。這解釋了保留資料曲線為何可能先改善再惡化：保護未見轉移的同時，最終也會稀釋有用的已知差異。應以驗證資料選擇參數，不要使用最終測試集調整。

分詞器與未知詞元規則也是實驗的一部分。把多個未見詞映射成同一標記，會合併它們的機率。這樣得到的較低困惑度，可能部分來自較容易預測的符號集合，並不表示模型能辨識那些原始詞語。

內插不同歷史長度的證據

長上下文很少見時，較短上下文通常有更多觀察。內插會結合它們的完整分布，而不是給每個未見延續相同的機率。

$$p_{\text{mix}}(w \mid u,v) = \lambda_3 p_3(w \mid u,v) + \lambda_2 p_2(w \mid v) + \lambda_1 p_1(w), \qquad \lambda_i \ge 0, \quad \sum_{i=1}^{3} \lambda_i = 1.$$

其中 u,v 是最近兩個詞元；w 是下一詞元；p_3, p_2, p_1 是正規化的 trigram、bigram 與 unigram 分布；p_{mix} 是混合分布；λ_i 是第 i 階的非負權重。每個成分都必須對查詢歷史有定義，例如先透過平滑處理。把所有候選詞元的機率加總會得到一，因為各成分分布總和為一，權重也總和為一。

對前面小範例的未見上下文，即使完整的兩詞元歷史缺少觀察，混合仍可以使用 `cat` 後面已觀察到的 bigram。固定混合權重容易檢查；隨歷史改變的權重則可以更信任證據充分的長上下文。以驗證文字選擇權重。若在檢查最終測試困惑度後才決定權重，就讓那個分數參與了模型選擇。

內插與「只有轉移缺少時才替換成較短歷史」不同。這類退回（back-off）規則還必須考慮已配置給觀察轉移的機率，才能讓完整分布保持正規化。Chen 與 Goodman 的比較研究檢視不同平滑選擇及實驗行為 [4]。本章的加法與混合範例用來揭示運作方式，沒有實作該研究中的每一種估計方法。

4. 保留資料困惑度量測什麼？

評分量測模型給觀察目標多少機率。好分數代表：在指定分詞器與詞彙表下，這些特定詞元序列讓模型比較不驚訝；它不能證明事實正確或理解能力。

$$\mathcal{L} = -\frac{1}{N}\sum_{j=1}^{N} \log p(w_j \mid h_j), \qquad \text{PPL} = \exp(\mathcal{L}) = \left(\prod_{j=1}^{N} \frac{1}{p(w_j \mid h_j)}\right)^{1/N}.$$

其中 N 是評分詞元位置數；j 遍歷所有評估句子的這些位置；w_j 是觀察目標；h_j 是允許使用的歷史；p 是被評估的模型；$\mathcal{L}$ 是每詞元以 nat 為單位的平均負對數概似；PPL 是困惑度（perplexity）。乘積形式顯示，困惑度是目標機率倒數的幾何平均，並不是每個位置合理延續的個數。

假設每個觀察目標都得到機率 $1/K$，困惑度就等於 K，其中 K 是這個假設分布中等可能選項的數目。實際預測不是均勻的：一個難以預測的目標可能造成很大的對數損失，許多容易目標則貢獻很少。只報平均值，可能掩蓋集中在罕見名稱或陌生領域的失敗。也應依詞頻與句型檢查損失。

Bigram 程式計入結束標記，因此停止同時屬於訓練及評估問題；起始標記則不計入。Unigram 與 bigram 結果可以比較，因為測試文字、分詞、未知映射及評分位置數一致。詞分詞器與字元分詞器的比較則有不同單位；即使原始句子相同，分數分母與預測符號集合也可能改變。

訓練、驗證與測試文字有不同角色。訓練決定詞彙表及次數；驗證選擇階數與平滑；測試則對選定流程評估一次。保留資料中未見於訓練的拼字，必須遵循固定未知規則。把測試文字納入重建詞彙表，會悄悄改變實驗。不同切分之間的重複句子或複製內容，也可能讓記住的局部延續看起來像泛化。

5. 生成、停止與記憶上限

生成反覆從正規化的下一詞元分布取樣，將取樣結果加入歷史，並在結束標記出現時停止。評分則由待評估文字提供觀察歷史。相同機率表支援兩種程序，但一種檢查提供的文字，另一種產生隨機軌跡。

示範輸出 `a bird sings` 只是固定隨機種子下的一次取樣，不是具代表性的品質估計。應檢查多個獨立種子的軌跡，並記錄局部錯誤及異常長的序列。長度上限保證程式停止執行；若在結束標記之前碰到上限，顯示的序列只是被截斷，並非模型完成的一個句子。

每一步選擇機率最大的詞元，會得到確定性的貪婪軌跡。這通常不會最大化整個序列的機率：局部看似有利的詞元，可能導向較差的後續延續。另一方面，取樣保留分布的多樣性，卻也可能選到低機率轉移。因此，生成選擇與模型是否給保留文字配置良好機率，是不同問題。

更多訓練也無法消除的失敗

考慮 `the robot near the cabinets is` 與 `the robots near the cabinet are`。一致關係取決於主詞中心語，不只是緊鄰動詞之前的名詞。若相關中心語在保留尾端之外，模型就會合併需要不同一致選擇的例子。更多觀察能改善合併估計，卻無法讓估計依賴模型輸入之外的詞元。

只要兩個前綴的最後 $n-1$ 個詞元相同，n-gram 模型的下一詞元分布就必須完全相同。這是架構上的等式，不是實驗趨勢。提高 n 會把記憶上限往後推，但可能歷史的數目與次數表的稀疏程度也會增加。共享的神經表示可以連結相似上下文，實際行為仍需要測試；光是替換次數表，並不能證明模型學會了一致關係。

6. 循環語言模型

白話版。 n-gram 模型在表格中查找歷史。循環語言模型則一次讀一個詞元，並保留一個固定大小的向量，也就是隱藏狀態（hidden state），在每個詞元之後更新。下一詞元分布由這個狀態計算，因此模型不再被迫忘記 $n-1$ 個詞元之前的所有內容。

Elman 的簡單循環網路提出把狀態回饋為下一步輸入的想法 [6]；Mikolov 等人則證明，訓練成詞層級語言模型的循環網路，在語音辨識上可以勝過強大的平滑 n-gram 基準 [7]。第 2 節的鏈式法則完全不變，改變的只有條件分布。對有起始標記 x_0 的詞元序列 $x_1, \dots, x_T$：

$$\begin{aligned} \mathbf{h}_t &= \tanh(W_x\,\mathbf{e}(x_{t-1}) + W_h\,\mathbf{h}_{t-1} + \mathbf{b}), \qquad \mathbf{h}_0 = \mathbf{0}, \\ \mathbf{z}_t &= W_o\,\mathbf{h}_t + \mathbf{c}, \\ p_\theta(x_t = v \mid x_{<t}) &= \operatorname{softmax}(\mathbf{z}_t)_v = \frac{\exp z_{t,v}}{\sum_{u \in V} \exp z_{t,u}}. \end{aligned}$$

其中 $\mathbf{e}(x) \in \mathbb{R}^d$ 是詞元 x 的嵌入（學習表格中的一列，如 NLP 04 所述）；$\mathbf{h}_t \in \mathbb{R}^m$ 是讀完 $x_0, \dots, x_{t-1}$ 後的隱藏狀態；$W_x \in \mathbb{R}^{m \times d}$、$W_h \in \mathbb{R}^{m \times m}$ 與 $\mathbf{b} \in \mathbb{R}^m$ 是循環參數；$W_o \in \mathbb{R}^{|V| \times m}$ 與 $\mathbf{c} \in \mathbb{R}^{|V|}$ 把狀態對應到每個詞彙項目 v 的分數 $z_{t,v}$；V 是預測詞彙表，u 遍歷其成員；θ 收集所有參數；tanh 逐元素作用。這與 ML 03A 的遞迴式相同，只是輸入改寫成嵌入，輸出改成整個詞彙表上的 softmax。

由方程式可直接得到三個結果。

- **條件不受限，記憶有上限。** $\mathbf{h}_t$ 是整個前綴的函數，因此最後 $n-1$ 個詞元相同的兩個前綴可以得到不同預測。但 $\mathbf{h}_t$ 只有 m 個數字。模型必須學會保留什麼；沒有任何保證它會保留某個依存關係所需的資訊。
- **參數共享。** 每個位置都使用相同的 W_x、W_h 與 W_o。參數量不隨上下文長度增加，而 n-gram 可能歷史的數目以 $|V|^{n-1}$ 成長。
- **透過相似性泛化。** 相似的詞元得到相似的嵌入，相似的前綴可以產生相似的狀態。未見歷史不再退回均勻的一列：它被對應到某個狀態，而由該狀態做出的預測可以重用相似歷史教給模型的內容。

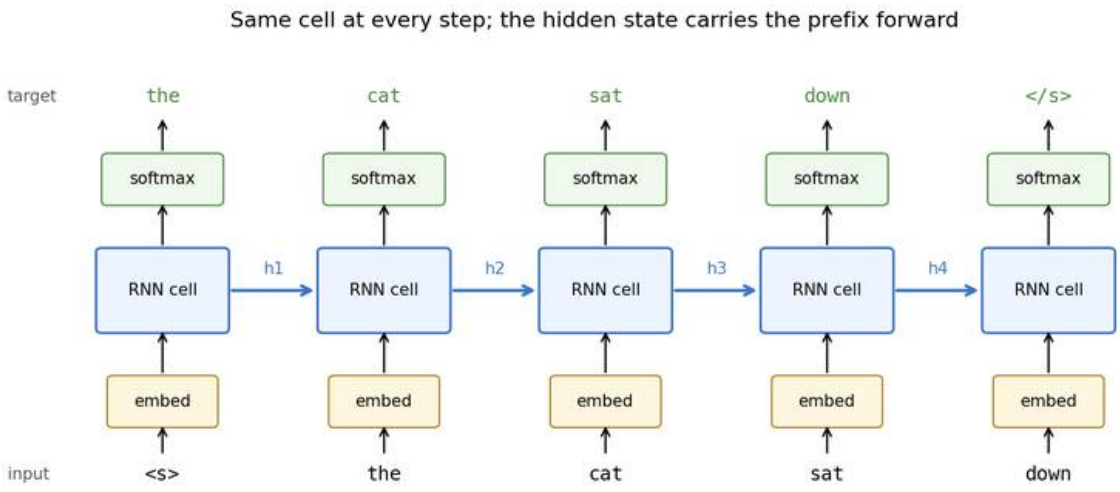


圖 2 — 展開五步的循環語言模型。每個輸入是前一個詞元，每個目標是下一個詞元，每一步都重用相同的單元參數。

訓練損失是下一詞元交叉熵

訓練最小化的量，正是第 4 節用於評估的量，只是現在寫成 θ 的可微函數：

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{j=1}^{N} \log p_\theta(x_j \mid x_{<j}), \qquad \text{PPL} = \exp \mathcal{L}, \qquad \text{bits per token} = \frac{\mathcal{L}}{\ln 2}.$$

其中 N 是受監督的目標位置數（排除填補及其他忽略位置），x_j 是位置 j 的觀察目標，$x_{<j}$ 是其前綴，$\ln 2$ 把 nat 換算成 bit。每個位置的項，是 one-hot 目標與模型 softmax 分布之間的交叉熵；IT 01 說明交叉熵及其與編碼長度的關係。字元層級的研究常以每字元位元數（bits per character）取代困惑度；兩者攜帶相同資訊。

計數模型的兩項實作細節在這裡仍然適用。第一，**位移一格**：輸入序列是 $x_0, \dots, x_{T-1}$，目標序列是 $x_1, \dots, x_T$。若把同一位置的詞元同時當作輸入與目標，就會產生瑣碎的複製捷徑。第二，**教師強制（teacher forcing）**：訓練時每一步收到觀察到的前一個詞元，而不是模型自己較早的取樣。生成時，模型的取樣成為它的輸入，錯誤可能自我累積。因此，訓練損失與取樣行為需要分別檢查，正如第 5 節對 n-gram 模型的論證。

7. 時間上的訓練：梯度能到達哪裡

白話版。把遞迴展開後，模型成為一個權重共享的極深網路。某個預測的梯度必須穿過每一個中間步驟，才能到達本應影響它的詞元。距離很長時，梯度往往縮小到幾乎為零，或者較少見地爆炸。

時間反向傳播（backpropagation through time，BPTT）把一般反向傳播套用到展開後的計算圖，並把每個共享權重的梯度貢獻加總 [8]。較晚狀態對較早狀態的敏感度，是一步 Jacobian 的有序乘積：

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_s} = \prod_{k=s+1}^{t} \operatorname{diag}\!\left(1 - \mathbf{h}_k^2\right) W_h,$$

其中 $s < t$ 是位置，$\mathbf{h}_k^2$ 逐元素平方，$\operatorname{diag}(\cdot)$ 以 tanh 導數組成對角矩陣。當各因子的範數通常小於一時，乘積隨距離 $t - s$ 指數衰減（梯度消失）；大於一時則可能指數成長（梯度爆炸）。Bengio、Simard 與 Frasconi 分析了為何以梯度下降學習長期依存關係很困難 [9]；Pascanu、Mikolov 與 Bengio 重新檢視這個問題，並推廣以裁剪梯度範數處理爆炸 [10]。ML 03A 完整推導並量測了這些結果。

這對語言代表什麼？回到第 5 節的一致關係例子：在 `the robots near the cabinet are` 中，動詞取決於往前四個詞的 `robots`，而中間的名詞指向另一個方向。普通 RNN 能夠表示記住複數主詞的狀態，但教它這麼做的訓練訊號，必須從 `are` 一路傳回 `robots`。在詞層級只有幾步；在字元層級，同一個依存關係大約跨越二十步。因此模型可能有足夠容量表示某個依存關係，卻仍學不會它；這與 n-gram 的架構上限是不同的失敗。

8. 閘控單元：LSTM 與 GRU

白話版。閘控單元加入一條主要以加法更新、並由學習開關控制的記憶路徑。資訊與梯度因此可以跨越許多步，而不必在每一步都被壓縮。

Hochreiter 與 Schmidhuber 的長短期記憶（long short-term memory，LSTM）單元 [11]，加上 Gers、Schmidhuber 與 Cummins 引入的遺忘閘 [12]，在隱藏狀態旁另外保留細胞狀態 $\mathbf{c}_t$：

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \qquad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t).$$

其中 $\mathbf{f}_t$、$\mathbf{i}_t$ 與 $\mathbf{o}_t$ 是遺忘閘、輸入閘與輸出閘（由 sigmoid 層根據目前輸入與 $\mathbf{h}_{t-1}$ 計算、落在 $(0,1)^m$ 的向量），$\tilde{\mathbf{c}}_t$ 是以相同方式計算的 tanh 候選值，$\odot$ 是逐元素乘法。沿著細胞路徑，$\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1}$ 近似 $\mathrm{diag}(\mathbf{f}_t)$：沒有權重矩陣，也沒有壓縮函數。當遺忘閘維持在接近一時，在複數主詞處寫入的值可以保留到動詞出現。

Cho 等人的閘控循環單元（gated recurrent unit，GRU）[13] 合併細胞與隱藏狀態，並使用兩個閘：

$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t,$$

其中 $\mathbf{z}_t \in (0,1)^m$ 是更新閘，$\tilde{\mathbf{h}}_t$ 是以重設閘計算的候選狀態。（有些資料會對調 $\mathbf{z}_t$ 與 $1-\mathbf{z}_t$ 的角色。）完整的閘門方程式、以 PyTorch 驗證的從零實作 LSTM 步驟及參數量，都在 ML 03A。對語言模型而言，實務結論是：LSTM 與 GRU 比普通 tanh RNN 穩定得多，兩者在不同任務上互有勝負，而閘控只是緩解、並未消除極長依存關係的困難。

9. 堆疊與雙向循環網路

白話版。循環層可以堆疊，讓較高層讀取下層的狀態序列。再加上一個由右向左執行的網路，每個位置也能看到它的未來；這對文字標註很有用，但在預測下一個詞元時是禁止的。

堆疊（深層）RNN 把第 ℓ 層的 $\mathbf{h}_1^{(\ell)}, \dots, \mathbf{h}_T^{(\ell)}$ 當作第 $\ell+1$ 層的輸入序列，softmax 讀取最上層。循環語言模型常用兩到四層；層與層之間的 dropout 是常見的正則化方式 [16]。

雙向 RNN 同時執行一個前向與一個反向網路，並串接兩者的狀態 $\mathbf{h}_t = [\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t]$ [14]。位置 t 的反向狀態已讀過 $x_{t+1}, \dots, x_T$。這非常適合標註：整個句子都已給定，而某個詞的標籤可能取決於右邊的詞；它是NLP 06 的主要編碼器。但它對由左到右的語言模型是錯誤的：若用來預測 x_t 的狀態已經看過 x_t，損失就會獎勵複製，困惑度也失去意義。任何語言模型都可以做一個有用的健全性檢查：改變一個未來詞元，所有較早位置的預測都必須保持不變。

10. 字元層級語言模型

白話版。預測單位可以是字元而非詞。詞彙表變得極小，也永遠不會出現未知拼字；但序列會長上好幾倍，模型必須先學會拼字，才能學文法。

字元層級 RNN 曾是展示循環網路學到什麼的熱門例子；Graves 證明以字元訓練的 LSTM 可以生成較長且局部連貫的文字與手寫軌跡 [15]。與詞或子詞單位相比，取捨相當具體：

選擇	詞彙表	未知輸入	序列長度	長距離成本
詞	大；罕見詞需要未知標記	對應到同一標記，拼字遺失	最短	相關詞之間的步數最少
子詞（NLP 01）	中等，由分詞器固定	拆成已知片段	中等	中等
字元	極小的字母表	字母表上的任何拼字	最長	相關詞之間的步數很多

第 4 節的評估規則帶來兩點提醒。第一，字元困惑度與詞困惑度的單位不同；困惑度 11 的字元模型並不比困惑度 3.6 的詞模型「更好」，因為兩者在不同字母表上計算不同數量的預測。要跨單位比較，應把兩者都換算成同一段原始文字的總負對數概似（或位元數）。第二，固定的字元字母表仍需要處理表外符號的規則，例如帶重音字母或表情符號。應在檢查保留文字之前決定。

11. 取樣與溫度

白話版。訓練好的模型替每個可能的下一詞元給出機率。生成時，我們抽出一個、附加上去，然後重複。溫度設定可以讓抽取更保守或更大膽，而不必重新訓練。

對 logit $\mathbf{z}_t$ 與溫度 $\tau > 0$，從下列分布取樣：

$$q_\tau(v) = \frac{\exp(z_{t,v}/\tau)}{\sum_{u \in V} \exp(z_{t,u}/\tau)}.$$

其中 v 與 u 遍歷詞彙表 V，$z_{t,v}$ 是模型在第 t 步給 v 的分數，q_τ 是取樣分布。$\tau = 1$ 時從模型自己的分布取樣；$\tau \to 0$ 時趨近於貪婪選擇最高分詞元；$\tau > 1$ 會壓平分布，讓不太可能的詞元得到更多權重。溫度只改變生成程序，不改變計算困惑度所用的機率，因此模型的保留資料分數在任何溫度下都相同。

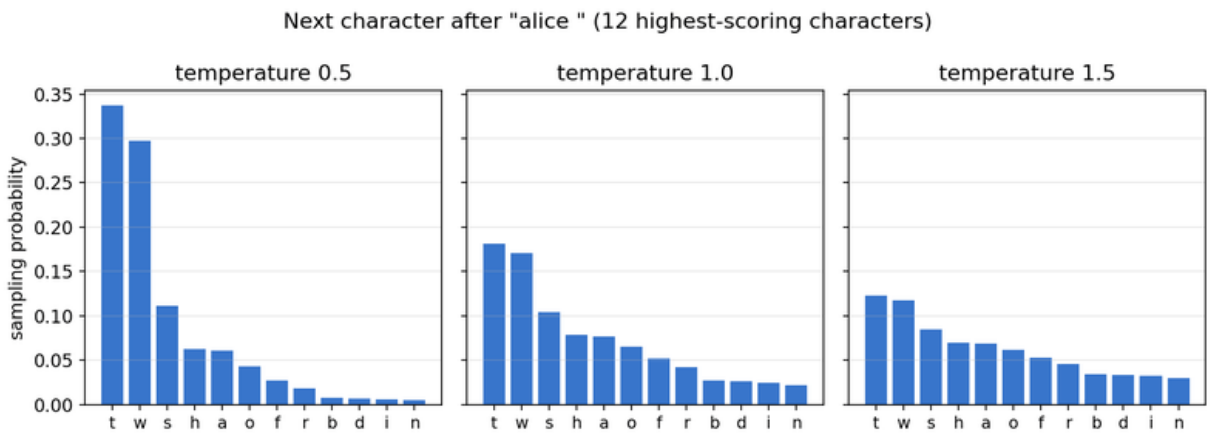


圖 3 — 第 12 節訓練好的 *LSTM* 在提示「*alice* 」之後、三種溫度下的下一字元分布。三個面板的 *logit* 完全相同，改變的只有取樣分布。

生成還涉及停止規則、長度限制，以及束搜尋（beam search）與 nucleus 取樣等搜尋策略。這些內容屬於NLP 08 與NLP 09。但有一項提醒現在就適用：一個樣本只是一條隨機軌跡。應以保留資料概似與任務檢查評判模型，樣本則用來尋找質性的失敗模式，而不是替模型排名。

12. 實務訓練，以及一個完整實驗

訓練檢查清單

白話版。大多數失敗的循環語言模型，失敗原因都很平凡：把填補當成文字計分、梯度爆炸、狀態在無關文件之間洩漏，或是用測試集挑選模型。每一項都有標準的補救方式。

- **把語料打包成串流**。對連續文字，把詞元串流切成 B 條等長的平行串流，排成 $B \times n$ 矩陣。每個訓練步驟讀取接下來的 T 欄。每個位置都是真實詞元，因此不需要填補。第 b 條串流會正好從上一個視窗停下的地方繼續。
- **填補與遮罩**。當範例是長度不同的獨立句子時，把它們填補到相同長度並保留布林遮罩。遮罩必須把填補位置排除在損失之外（否則模型會因預測填補而得到獎勵），對雙向編碼器還必須排除在狀態更新之外；PyTorch 的 `pack_padded_sequence` 負責後者。NLP 06 會在標註器中實作兩者。
- **截斷 BPTT**。反向傳播穿過數千步，記憶體成本隨長度線性成長。改為把隱藏狀態從一個 T 步視窗帶到下一個視窗，但將它分離（*detach*），讓前向計算仍看到完整過去，而梯度最多只往回流 T 步。
- **梯度裁剪**。若梯度範數 $\|\mathbf{g}\|$ 超過門檻 κ，就把它縮放成 $\kappa\,\mathbf{g}/\|\mathbf{g}\|$ [10]。這會保留更新方向並限制其長度。
- **正則化與提前停止**。在非循環連接上使用 dropout [16] 與權重衰減，可以限制記憶；選擇驗證困惑度最低的訓練週期，最後只報告一次測試困惑度。
- **公平的基準**。 Melis、Dyer 與 Blunsom 發現，在給予相當的超參數搜尋後，仔細調校的標準 LSTM 勝過幾種較新、較複雜的架構 [17]。同樣的原則在小規模也成立：在宣稱神經模型勝出之前，先用同一份驗證資料調校 n-gram 基準。

程式碼：字元層級 LSTM 對上內插 n-gram

文字是 Lewis Carroll《愛麗絲夢遊仙境》（*Alice's Adventures in Wonderland*，1865）第一章的開頭，屬於公有領域；取自 Project Gutenberg 電子書 #11 [24]，移除斜體底線、把彎引號改成直引號、把破折號寫成 `--`，再轉成小寫。第 1–7 段是訓練資料，第 8 段是驗證資料，第 9 段是測試集。字母表事先固定，在檢查保留文字之前就決定。兩個模型都以所有較早的文字作為每個保留字元的條件。n-gram 基準完全依照第 3 節的方式，內插經平滑的 unigram、bigram 與 trigram 字元分布，權重與平滑強度在驗證資料上選擇。LSTM 以不同種子訓練三次，因為在這麼少的資料上，單次結果可能誤導。整個程式在 CPU 上遠少於一分鐘即可完成，也不下載任何東西。

```
import copy
import math
from collections import Counter, defaultdict

import torch
from torch import nn

# Opening paragraphs of Chapter I of Lewis Carroll's "Alice's Adventures in
# Wonderland" (1865, public domain; Project Gutenberg eBook #11). Italic
# underscores removed, curly quotes straightened, em dashes written as "--".
```

```
PARAGRAPHS = [
    'Alice was beginning to get very tired of sitting by her sister on the '
    'bank, and of having nothing to do: once or twice she had peeped into the '
    'book her sister was reading, but it had no pictures or conversations in '
    'it, "and what is the use of a book," thought Alice "without pictures or '
    'conversations?"',
    'So she was considering in her own mind (as well as she could, for the hot '
    'day made her feel very sleepy and stupid), whether the pleasure of making '
    'a daisy-chain would be worth the trouble of getting up and picking the '
    'daisies, when suddenly a White Rabbit with pink eyes ran close by her.',
    'There was nothing so very remarkable in that; nor did Alice think it so '
    'very much out of the way to hear the Rabbit say to itself, "Oh dear! Oh '
    'dear! I shall be late!" (when she thought it over afterwards, it occurred '
    'to her that she ought to have wondered at this, but at the time it all '
    'seemed quite natural); but when the Rabbit actually took a watch out of '
    'its waistcoat-pocket, and looked at it, and then hurried on, Alice started'
    ' to her feet, for it flashed across her mind that she had never before '
    'seen a rabbit with either a waistcoat-pocket, or a watch to take out of '
    'it, and burning with curiosity, she ran across the field after it, and '
    'fortunately was just in time to see it pop down a large rabbit-hole under '
    'the hedge.',
    'In another moment down went Alice after it, never once considering how in '
    'the world she was to get out again.',
    'The rabbit-hole went straight on like a tunnel for some way, and then '
    'dipped suddenly down, so suddenly that Alice had not a moment to think '
    'about stopping herself before she found herself falling down a very deep '
    'well.',
    'Either the well was very deep, or she fell very slowly, for she had plenty'
    ' of time as she went down to look about her and to wonder what was going '
    'to happen next. First, she tried to look down and make out what she was '
    'coming to, but it was too dark to see anything; then she looked at the '
    'sides of the well, and noticed that they were filled with cupboards and '
    'book-shelves; here and there she saw maps and pictures hung upon pegs. She'
    ' took down a jar from one of the shelves as she passed; it was labelled '
    '"ORANGE MARMALADE", but to her great disappointment it was empty: she did '
    'not like to drop the jar for fear of killing somebody underneath, so '
    'managed to put it into one of the cupboards as she fell past it.',
    '"Well!" thought Alice to herself, "after such a fall as this, I shall '
    "think nothing of tumbling down stairs! How brave they'll all think me at "
    "home! Why, I wouldn't say anything about it, even if I fell off the top of"
    ' the house!" (Which was very likely true.)',
    'Down, down, down. Would the fall never come to an end? "I wonder how many '
    'miles I\'ve fallen by this time?" she said aloud. "I must be getting '
    'somewhere near the centre of the earth. Let me see: that would be four '
    'thousand miles down, I think--" (for, you see, Alice had learnt several '
    'things of this sort in her lessons in the schoolroom, and though this was '
    'not a very good opportunity for showing off her knowledge, as there was no'
    ' one to listen to her, still it was good practice to say it over) "--yes, '
    "that's about the right distance--but then I wonder what Latitude or "
    'Longitude I\'ve got to?" (Alice had no idea what Latitude was, or Longitude'
    ' either, but thought they were nice grand words to say.)',
    'Presently she began again. "I wonder if I shall fall right through the '
    "earth! How funny it'll seem to come out among the people that walk with "
    'their heads downward! The Antipathies, I think--" (she was rather glad '
```

```
    "there was no one listening, this time, as it didn't sound at all the right"
    ' word) "--but I shall have to ask them what the name of the country is, '
    'you know. Please, Ma\'am, is this New Zealand or Australia?" (and she tried'
    " to curtsey as she spoke--fancy curtseying as you're falling through the "
    'air! Do you think you could manage it?) "And what an ignorant little girl '
    "she'll think me for asking! No, it'll never do to ask: perhaps I shall see"
    ' it written up somewhere."',
]

paras = [p.lower() for p in PARAGRAPHS]
train = "\n".join(paras[:7]) + "\n"     # paragraphs 1-7: fit parameters
valid = paras[7] + "\n"                 # paragraph 8: choose settings
test = paras[8] + "\n"                  # paragraph 9: report once
ALPHABET = "\n !\"'(),-.:;?abcdefghijklmnopqrstuvwxyz"  # fixed in advance
V = len(ALPHABET)
assert set(train + valid + test) <= set(ALPHABET)
print("characters train/valid/test:", len(train), len(valid), len(test))
print("letters never seen in training:", sorted(set(valid + test) - set(train)))

# ---- Baseline: interpolated character n-grams (orders 1-3) ----
def count(order):
    rows = defaultdict(Counter)
    for t in range(len(train)):
        rows[train[max(0, t - order + 1):t]][train[t]] += 1
    return rows

tables = {k: count(k) for k in (1, 2, 3)}

def p_add(k, history, ch, alpha):
    row = tables[k].get(history[len(history) - k + 1:] if k > 1 else "", {})
    return (row.get(ch, 0) + alpha) / (sum(row.values()) + alpha * V)

def ngram_ppl(prefix, target, lams, alpha):
    full, nll = prefix + target, 0.0
    for pos in range(len(prefix), len(full)):
        h = full[max(0, pos - 2):pos]
        p = sum(l * p_add(k, h, full[pos], alpha) for k, l in zip((1, 2, 3), lams))
        nll -= math.log(p)
    return math.exp(nll / len(target))

grid = [(l1, l2, round(1 - l1 - l2, 2)) for l1 in (0.1, 0.2, 0.3)
        for l2 in (0.2, 0.3, 0.4, 0.5) if l1 + l2 < 1]
best_cfg = min(((ngram_ppl(train, valid, lams, a), lams, a) for lams in grid
                for a in (0.01, 0.1, 0.5)), key=lambda r: r[0])
ng_valid, ng_lams, ng_alpha = best_cfg
ng_test = ngram_ppl(train + valid, test, ng_lams, ng_alpha)
print(f"n-gram  weights={ng_lams} alpha={ng_alpha}  "
      f"valid PPL={ng_valid:.2f}  test PPL={ng_test:.2f}")

tri_alpha = min((0.01, 0.1, 0.5), key=lambda a: ngram_ppl(train, valid, (0, 0, 1), a))
tri_test = ngram_ppl(train + valid, test, (0, 0, 1), tri_alpha)
print(f"trigram only  alpha={tri_alpha}  test PPL={tri_test:.2f}")

# ---- Character-level LSTM language model ----
```

```
stoi = {c: i for i, c in enumerate(ALPHABET)}
encode = lambda s: torch.tensor([stoi[c] for c in s])

class CharLSTM(nn.Module):
    def __init__(self, vocab, emb=24, hidden=96, dropout=0.3):
        super().__init__()
        self.emb = nn.Embedding(vocab, emb)
        self.rnn = nn.LSTM(emb, hidden, batch_first=True)
        self.drop = nn.Dropout(dropout)
        self.out = nn.Linear(hidden, vocab)

    def forward(self, x, state=None):
        h, state = self.rnn(self.emb(x), state)
        return self.out(self.drop(h)), state

def lstm_ppl(model, prefix, target):
    model.eval()
    with torch.no_grad():
        ids = encode(prefix + target).unsqueeze(0)
        logits, _ = model(ids[:, :-1])          # condition on all earlier text
        nll = nn.functional.cross_entropy(logits[0, len(prefix) - 1:],
                                          ids[0, len(prefix):])
    model.train()
    return math.exp(nll.item())

def train_lstm(seed, epochs=80, B=16, T=48):
    torch.manual_seed(seed)
    model = CharLSTM(V)
    opt = torch.optim.Adam(model.parameters(), lr=3e-3, weight_decay=1e-4)
    ids = encode(train)
    n = (len(ids) - 1) // B                     # B parallel streams of n characters
    X, Y = ids[:n * B].view(B, n), ids[1:n * B + 1].view(B, n)  # targets shifted by one
    history, best = [], (float("inf"), None, 0)
    for epoch in range(1, epochs + 1):
        state, total, steps = None, 0.0, 0
        for i in range(0, n, T):
            if state is not None:               # truncated BPTT: keep values, cut graph
                state = tuple(s.detach() for s in state)
            logits, state = model(X[:, i:i + T], state)
            loss = nn.functional.cross_entropy(logits.reshape(-1, V),
                                               Y[:, i:i + T].reshape(-1))
            opt.zero_grad()
            loss.backward()
            nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            opt.step()
            total, steps = total + loss.item(), steps + 1
        v = lstm_ppl(model, train, valid)
        history.append((epoch, math.exp(total / steps), v))
        if v < best[0]:                         # early stopping on validation only
            best = (v, copy.deepcopy(model.state_dict()), epoch)
    model.load_state_dict(best[1])
    return model, best, history

runs = {}
```

```
for seed in (0, 1, 2):
    m, best, hist = train_lstm(seed)
    runs[seed] = (m, hist, lstm_ppl(m, train + valid, test))
    print(f"LSTM seed={seed}  best epoch={best[2]}  valid PPL={best[0]:.2f}  "
          f"test PPL={runs[seed][2]:.2f}")
mean_test = sum(r[2] for r in runs.values()) / len(runs)
print(f"LSTM mean test PPL over 3 seeds: {mean_test:.2f}")
assert mean_test < tri_test
model = runs[0][0]

# ---- Sampling at several temperatures ----
def sample(prompt, tau, length=80, seed=1):
    g = torch.Generator().manual_seed(seed)
    model.eval()
    with torch.no_grad():
        logits, state = model(encode(prompt).unsqueeze(0))
        out = prompt
        for _ in range(length):
            probs = torch.softmax(logits[0, -1] / tau, dim=-1)
            nxt = torch.multinomial(probs, 1, generator=g)
            out += ALPHABET[nxt.item()]
            logits, state = model(nxt.view(1, 1), state)
    return out.replace("\n", " / ")

for tau in (0.5, 1.0, 1.5):
    print(f"tau={tau}:", sample("alice ", tau))

# ---- Same last two characters, different earlier context ----
def next_dist(prefix):
    model.eval()
    with torch.no_grad():
        logits, _ = model(encode(prefix).unsqueeze(0))
    return torch.softmax(logits[0, -1], dim=-1)

a, b = "alice was beginning to get ", "the rabbit-hole went "
assert a[-2:] == b[-2:]                    # a trigram sees only these two
def ngram_dist(prefix):
    return torch.tensor([sum(l * p_add(k, prefix[-2:], c, ng_alpha)
                             for k, l in zip((1, 2, 3), ng_lams)) for c in ALPHABET])

tv = lambda p, q: 0.5 * (p - q).abs().sum().item()
print("total variation, n-gram:", round(tv(ngram_dist(a), ngram_dist(b)), 3),
      " LSTM:", round(tv(next_dist(a), next_dist(b)), 3))
```

輸出：

```
characters train/valid/test: 2633 706 682
letters never seen in training: ['z']
n-gram  weights=(0.1, 0.4, 0.5) alpha=0.01  valid PPL=9.70  test PPL=11.24
trigram only  alpha=0.1  test PPL=13.37
LSTM seed=0  best epoch=43  valid PPL=10.13  test PPL=12.09
LSTM seed=1  best epoch=41  valid PPL=9.76  test PPL=11.05
```

```
LSTM seed=2  best epoch=39  valid PPL=10.09  test PPL=11.30
LSTM mean test PPL over 3 seeds: 11.48
tau=0.5: alice whas she sor of the to her fear the she was she sely and it she save of the
 ↪  was
tau=1.0: alice whas nyt ook "hente tome urabuint, mubbeit-insiwselly bulith ver to welf was
 ↪  bor
tau=1.5: alice wherany fowh ".pvenst gime feurdt, )ubger and iwselly ?hont  ver, vermef
 ↪  wamen r
total variation, n-gram: 0.0  LSTM: 0.197
```

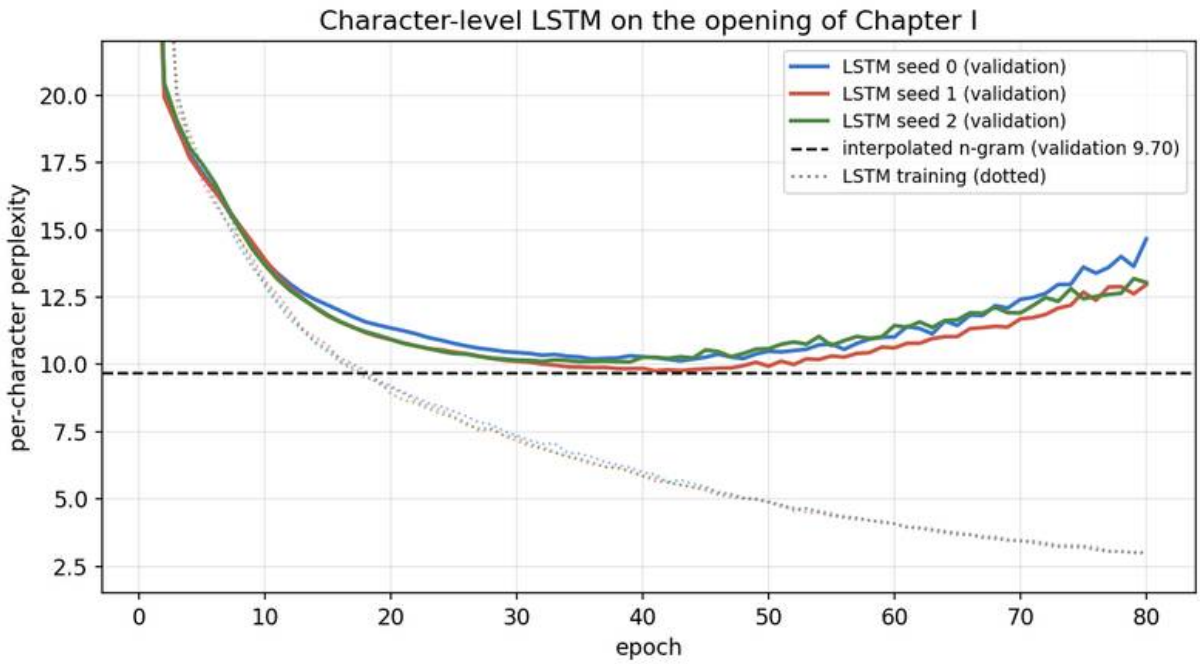


圖 *4* — 訓練過程中的每字元困惑度。訓練困惑度持續下降；驗證困惑度在第 *40* 週期附近達到最低，之後因模型記住 *2,633* 個訓練字元而回升。虛線是在同一驗證段落上調校的內插 *n-gram*。

解讀結果

基準很重要。使用加法平滑的單一 trigram 測試困惑度為 13.37，三次 LSTM 都勝過它。在同一驗證段落上調校的內插 n-gram 達到 11.24，而三次 LSTM 分別是 12.09、11.05 與 11.30（平均 11.48）。在 2,633 個訓練字元上，循環模型大致與平滑良好的計數模型打平，並明顯勝過平滑不佳的計數模型。這是 Melis 等人論點的小資料版本 [17]：對未調校基準宣稱的改善，面對調校過的基準可能就消失了。隨著資料增加，神經語言模型會拉開差距，因為共享表示能利用許多稀疏表格無法合併的微弱規律；Jozefowicz 等人記錄了循環語言模型在大型語料與模型上的進展 [19]。我們的實驗太小，無法呈現那種情況，也不需要：它示範的是如何誠實地比較。

種子很重要。不同種子之間的差異（11.05 到 12.09），大於 LSTM 平均值與 n-gram 之間的差距。若只用一個種子，我們可能報告「LSTM 勝出」，也可能報告「LSTM 落敗」。差異這麼小時，應

報告多次執行結果，或對保留文件計算信賴區間。

驗證曲線顯示記憶現象。在圖 4 中，訓練困惑度持續下降，遠低於驗證值；模型正在學習特定的訓練段落。提前停止選出第 39–43 週期；若沒有它，最後一個週期在保留文字上會差上不少。

單位。這些是 39 個符號字母表上的每字元困惑度；n-gram 的 $\log_2 11.24 \approx 3.49$ 每字元位元。它們不能與第 3 節的詞層級數值（例如 3.573）比較，因為後者在不同詞彙表上計算不同事件。

量測記憶上限。最後幾行比較 `alice was beginning to get` 與 `the rabbit-hole went` 之後的下一字元分布，這兩個前綴的最後兩個字元相同。內插 trigram 必須給出完全相同的分布（總變異距離 0.0），正如第 5 節所證明。LSTM 的分布則不同（總變異距離 0.197），因為它的狀態取決於更早的字元。這確認循環模型有使用較長的上下文；至於是否用得好，則是另一個問題，要由保留資料困惑度與針對性測試回答。

樣本。在 $\tau = 0.5$ 時，樣本重複 `she was`、`of the` 等常見片段；在 $\tau = 1.0$ 時出現詞的形狀，但拼錯很多；在 $\tau = 1.5$ 時輸出瓦解成罕見標點與隨機字母。一個只用不到三千個字元訓練的字元模型，學到的是常見字母序列、空白與一些短詞，並沒有學會英語。這些都是單一種子的單次抽樣，應視為溫度效果的示意，而不是品質估計。

現代觀點

綜述觀點。 Jurafsky 與 Martin 的 2026 年 8 月草稿仍保留 n-gram 專章，依機率預測、估計、評估、生成及平滑來組織 [3]。它提供有效的概念橋梁：讀者可以先檢查局部次數，再接觸共享可學習表示的模型。Chen 與 Goodman 的平滑比較則解釋了為何除了歷史長度，估計方法本身也很重要 [4]。兩種來源用途不同：教材發展模型，實證研究比較如何在稀疏證據下配置機率。Tiezzi 等人的 2024 年綜述回顧循環處理為何在 Transformer 時代重新受到重視 [20]。它把兩股趨勢放在「循環」這個統一框架下：一類結合 Transformer 與循環網路想法、以避開自注意力的二次方成本；另一類是作為特殊線性 RNN 的深層狀態空間模型。它提出的主要開放問題，是從長度事先未知的串流中學習，這把序列建模連到終身線上學習。

循環語言模型的成就。從 Mikolov 的 RNN 語言模型 [7]，到大規模 LSTM 語言模型 [19] 與仔細正則化的 LSTM [18]，循環網路一直是最強的通用語言模型，直到 Transformer 在多數大規模工作中取代它們。Merity 等人的 AWD-LSTM 研究顯示，那份實力有很大一部分來自正則化與最佳化選擇，而不是新的單元設計 [18]；Melis 等人透過受控的超參數搜尋得到相似結論 [17]。

2023–2026 年。循環以新形式回歸，目標是降低長上下文注意力的成本。RWKV 重新表述類 Transformer 模型，使它能平行訓練，並在推論時以 RNN 方式執行 [21]。Mamba 使用依輸入而變的（選擇性）狀態空間更新，並以平行掃描計算 [22]。xLSTM 以指數閘控與新的記憶結構重新檢視 LSTM [23]。這些模型保留了讓 RNN 語言模型具吸引力的性質：固定大小的狀態，因此每個詞元的生成成本不隨上下文長度增加。它們也繼承了第 7 節提出的問題：固定大小的狀態必須決定要忘記什麼。ML 03A 對狀態空間模型有更詳細的介紹。

評估。 Chang 等人的 2024 年綜述以「評估什麼、在哪裡評估、如何評估」組織大型語言模型評估 [5]，涵蓋詞元預測之外的任務行為與評估方法。套用到本章，較低困惑度仍應搭配真正重要的任務檢查，例如拼字恢復、一致關係、語意保存或適當停止規則。在 2023–2026 年的工作流程中，n-gram 基準的實際作用是讓問題可以量測。更複雜的模型是否在相同分詞下改善保留預測？是否區分了既定 n-gram 歷史必須合併的兩個前綴？增加記憶體與延遲是否改善下游結果？應用可比較的實驗回答，而不是比較條件不一致的分數，或假設較長上下文一定被有效利用。

開放問題。開放問題包括領域改變後的稀疏證據、新名稱的機率，以及分詞與量測難度之間的互動。字元或子詞會改變詞彙覆蓋與序列長度的取捨。更長的歷史，無論存在表格或循環狀態中，都提供更多資訊，卻也帶來更困難的估計與功勞分配（credit assignment）問題。應一起選擇預測單位與保留歷史，並在解讀結果之前說明限制。

重點整理

- n-gram 出現或文件特徵，不會自動成為正規化語言模型；第 n 階使用前 $n-1$ 個詞元，而機率鏈式法則本身沒有要求有限記憶。
- 平滑保護未觀察事件，內插重用較短歷史的證據；兩者都無法恢復模型已捨棄的歷史。
- 比較困惑度需要一致的預測單位、未知映射與邊界規則；字元與詞的困惑度不能比較。
- 循環語言模型以依賴整個前綴的學習狀態取代次數表，參數在各位置之間共享。
- BPTT 梯度隨距離縮小或放大；裁剪處理爆炸，LSTM 或 GRU 的閘控緩解梯度消失，但沒有消除長依存關係的困難。
- 雙向 RNN 適合標註，不適合由左到右的語言模型。
- 實務訓練需要位移目標、遮罩或打包串流、截斷 BPTT、梯度裁剪、正則化，以及以驗證資料為準的提前停止。
- 在極小資料上，調校過的內插 n-gram 可以與 LSTM 打平；應與調校過的基準比較，並報告多個種子。

練習

1. 手算第 2 節第一個程式中，`a cat` 與 `the cat` 後面的平滑 trigram 機率。確認八個目標詞元上的總和。

   提示每個歷史只有一筆觀察。分母是一加上平滑強度乘以八；只有觀察到的延續額外得到一次計數。

2. 混合第一個範例的三種階數。任選總和為一的非負權重，並對未見兩詞元歷史驗證正規化。

   提示讓每個成分都使用相同候選詞彙表。比較未見歷史下的機率、純均勻退回，以及 `cat` 後面觀察到的 bigram。

3. 分別在計入與不計入結束標記的條件下，替 bigram 測試集評分。解釋為什麼兩種困惑度回答不同預測問題。

   提示一致地改變累積損失與位置數。不計入結束標記，移除了停止行為的評估，不只是改變顯示慣例。

4. 建立兩個最後兩詞元相同的前綴，讓它們因更早的否定、一致關係或篇章資訊，而需要不同的下一詞元機率。說明 trigram 能與不能學到什麼，以及 RNN 語言模型原則上能學到什麼。

提示先確認保留歷史完全相同，再區分資料不足與根本無法依省略的差異設定條件這兩種問題。對 RNN，要區分它的狀態能表示什麼，以及它的梯度能教會它什麼。

5. 在字元層級實驗中，把 `nn.LSTM` 換成 `nn.GRU`（狀態變成單一張量，所以要調整 detach 那一行）與 `nn.RNN`。以三個種子比較驗證與測試困惑度。接著移除普通 RNN 的梯度裁剪，記錄訓練損失發生什麼事。

   提示 `nn.GRU` 與 `nn.RNN` 的狀態是張量而不是元組。保持其他所有設定不變，包括提前停止規則，讓單元類型成為唯一的改變。

6. 刻意破壞因果性：給 LSTM 模型在同一輸入上加一個反向 LSTM（雙向層）並重新訓練。解釋你觀察到的訓練與驗證困惑度，並寫一個本來能抓到這個錯誤的測試。

   提示有了反向計算，用來預測位置 t 字元的狀態已經讀過該字元。好的測試會改變一個未來字元，並檢查所有較早位置的預測分布保持不變。

參考文獻

1. Masato Hagiwara, *Real-World Natural Language Processing: Practical Applications with Deep Learning*, Chapter 5, Manning, 2021. Publisher
2. Susan McRoy, *Principles of Natural Language Processing*, 2021. Chapter
3. Daniel Jurafsky and James H. Martin, *Speech and Language Processing*, third-edition draft, Chapter 3, August 19, 2026. Chapter
4. Stanley F. Chen and Joshua Goodman, “An Empirical Study of Smoothing Techniques for Language Modeling,” ACL, 1996. Paper
5. Yupeng Chang et al., “A Survey on Evaluation of Large Language Models,” ACM Transactions on Intelligent Systems and Technology, 2024. Paper
6. Jeffrey L. Elman, “Finding Structure in Time,” *Cognitive Science* 14(2), 1990. DOI
7. Tomáš Mikolov, Martin Karafiát, Lukáš Burget, Jan Černocký and Sanjeev Khudanpur, “Recurrent Neural Network Based Language Model,” Interspeech, 2010. DOI
8. Paul J. Werbos, “Backpropagation Through Time: What It Does and How to Do It,” *Proceedings of the IEEE* 78(10), 1990. DOI
9. Yoshua Bengio, Patrice Simard and Paolo Frasconi, “Learning Long-Term Dependencies with Gradient Descent Is Difficult,” *IEEE Transactions on Neural Networks* 5(2), 1994. DOI
10. Razvan Pascanu, Tomas Mikolov and Yoshua Bengio, “On the Difficulty of Training Recurrent Neural Networks,” ICML, 2013. arXiv
11. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation* 9(8), 1997. DOI
12. Felix A. Gers, Jürgen Schmidhuber and Fred Cummins, “Learning to Forget: Continual Prediction with LSTM,” *Neural Computation* 12(10), 2000. DOI
13. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014. arXiv

14. Mike Schuster and Kuldip K. Paliwal, “Bidirectional Recurrent Neural Networks,” *IEEE Transactions on Signal Processing* 45(11), 1997. DOI
15. Alex Graves, “Generating Sequences With Recurrent Neural Networks,” arXiv:1308.0850, 2013. arXiv
16. Wojciech Zaremba, Ilya Sutskever and Oriol Vinyals, “Recurrent Neural Network Regularization,” arXiv:1409.2329, 2014. arXiv
17. Gábor Melis, Chris Dyer and Phil Blunsom, “On the State of the Art of Evaluation in Neural Language Models,” arXiv:1707.05589, 2017. arXiv
18. Stephen Merity, Nitish Shirish Keskar and Richard Socher, “Regularizing and Optimizing LSTM Language Models,” arXiv:1708.02182, 2017. arXiv
19. Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer and Yonghui Wu, “Exploring the Limits of Language Modeling,” arXiv:1602.02410, 2016. arXiv
20. Matteo Tiezzi, Michele Casoni, Alessandro Betti, Tommaso Guidi, Marco Gori and Stefano Melacci, “On the Resurgence of Recurrent Models for Long Sequences: Survey and Research Opportunities in the Transformer Era,” arXiv:2402.08132, 2024. arXiv
21. Bo Peng et al., “RWKV: Reinventing RNNs for the Transformer Era,” Findings of EMNLP, 2023. Paper
22. Albert Gu and Tri Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” COLM, 2024. arXiv
23. Maximilian Beck et al., “xLSTM: Extended Long Short-Term Memory,” NeurIPS, 2024. Paper
24. Lewis Carroll, *Alice’s Adventures in Wonderland*, 1865; Project Gutenberg eBook #11. Project Gutenberg

NLP 06・以 RNN 做序列標註（詞性標註、資訊擷取）

替句子的每個詞元加上標籤：以 HMM 與 Viterbi 解碼做詞性標註，以 BIO 標籤與精確區段評估做實體辨識，並用 BiLSTM、字元特徵與 CRF 輸出層建立神經標註器。

網站版本: https://blog.shyandram.dev/zh-tw/learn/nlp/nlp-06-sequence-labeling/

你將學到

- 把詞性標註與實體辨識表示成序列標註，並說明為何獨立的詞元判定可能與合法的整句分析不一致。
- 推導隱藏馬可夫序列分數與最大路徑（Viterbi）解碼，並把它們連到條件隨機場。
- 以可重現的位移定義實體結構描述，用 BIO 標籤編碼提及，並以精確區段而非詞元準確率評估。
- 建立 BiLSTM 標註器，加入字元層級特徵與 CRF 輸出層，並以遮罩前向演算法的對數概似訓練 CRF。
- 把實體標註放進資訊擷取（實體、關係、事件）的全貌，並認識標準資料集。
- 執行完整的 PyTorch 實驗，在訓練中從未出現的名稱上量測區段 F1、錯誤類型與非法標籤轉移。

先看全貌

許多語言任務要求每個詞元一個標籤。詞性（part-of-speech，POS）標註器把每個詞標成名詞、動詞、形容詞等；實體辨識器標出哪些詞元屬於人名、組織名或地名，以及每個名稱從哪裡開始、到哪裡結束。兩者看起來像沿著句子重複進行分類，但判定彼此並不獨立：某個詞的正確標籤，既取決於鄰近的詞，也取決於鄰近的標籤。把整個標籤序列視為一個結構化輸出，是本章的核心想法。

McRoy 第 3、4 章介紹語法類別與語法處理，第 8 章連結語言處理與資訊擷取 [1]。Hagiwara 第 5 章發展序列標記，包括詞性標記與命名實體辨識 [2]。本章結合這些觀點：標註器為各位置附上類別；擷取器則把選定區段轉成應用程式能檢查的紀錄。輸出契約十分重要：客服系統可能需要產品名稱與抱怨事件，歷史檔案則需要人物、組織、日期與帶有消息來源的陳述。

> **用最簡單的話說**
>
> 給小朋友一個句子和三支螢光筆：黃色畫人、綠色畫公司、藍色畫地點。每個詞各自上色是不夠的。「Port」在「Port Lena」裡是藍色，在「港口裡的一艘船」這種用法裡就不是；兩個字的名稱如果後半畫成綠色，前半最好也是綠色。好的螢光筆會看完整個句子，讓顏色彼此一致。

實際風險是轉換時遺失重要差別。「供應商說裝置通過了測試」不能證明裝置確實通過。系統即使抓對所有名字，仍可能因為刪除消息歸屬或可能性而建立錯誤紀錄。因此，要同時評估表面擷取

與解讀；格式正確的輸出不是內容正確的證據。結構化預測的數學（HMM、前向—後向演算法、Viterbi 解碼與一般形式的 CRF）在PR 06 發展；循環編碼器來自NLP 05。剖析、語意角色，以及深入的關係與事件擷取，則在NLP 12 介紹。

詞、詞法與語法類別

白話版。詞的類別描述它在句子中的行為。拼字能提供線索，但脈絡通常決定實際用法。

詞法（morphology）研究詞內結構。複數詞尾、時態標記與派生前綴能提供有用證據，但不同語言不一定用相同片段或詞邊界表達性質。為了建模效率學出的子詞 token，也不一定對應語言學上的詞素。應區分模型斷詞與標註使用的語法單位。

詞性（part of speech）描述名詞、動詞等語法類別。名詞通常是名詞片語的中心，動詞通常是子句述語的中心。這是分布上的描述，不是每個名詞都必須指實體物品的規則。「Decision」與「arrival」雖指抽象內容與事件，仍是名詞。

相同表面詞可以有不同類別。在原創例句「the pilots can land」與「the metal can fell」中，「can」扮演不同角色。第二句也顯示，句子其餘部分改變時，熟悉的局部片語可能誤導。標記器必須考慮允許的脈絡與標註規則。

已標記句子可表示為：

$$\mathbf{x} = (x_1, \dots, x_T), \qquad \mathbf{y} = (y_1, \dots, y_T), \qquad y_t \in \mathcal{Y}.$$

其中 $\mathbf{x}$ 是輸入詞序列，x_t 是位置 t 的詞，T 是序列長度，$\mathbf{y}$ 是標記序列，y_t 是詞 x_t 的標記，$\mathcal{Y}$ 是有限標記集合。t 與 T 指語言學上的詞位置，不會自動等同模型子詞位置。

標註也需要縮寫、標點、多詞表達與多種合理分析的規則。當模型使用子詞時，應定義如何把輸出對應回詞。只用第一個子詞表示是一種做法；池化所有子詞則是另一種。不論選哪個，標準標籤與解碼輸出都必須使用相同單位。

局部分類與序列預測

白話版。分別替每個詞選最佳標記，可能錯過更好的共同解讀。序列模型把標記間的關係寫清楚。

局部分類器可以預測：

$$\hat{y}_t = \operatorname*{argmax}_{k \in \mathcal{Y}} p_\theta(y_t = k \mid \mathbf{x}).$$

其中 $\hat{y}_t$ 是位置 t 選出的標記，k 遍歷標記集合 $\mathcal{Y}$，p_θ 是參數為 θ 的模型，$\mathbf{x}$ 是整個可見輸入序列。局部輸出頭仍可以讀取上下文；「獨立」描述輸出決策規則，不一定描述編碼器。

結構化解碼器則選擇：

$$\hat{\mathbf{y}} = \operatorname*{argmax}_{\mathbf{y} \in \mathcal{A}(\mathbf{x})} S_\theta(\mathbf{x}, \mathbf{y}).$$

其中 $\hat{\mathbf{y}}$ 是選出的完整標記序列，$\mathcal{A}(\mathbf{x})$ 是輸入 $\mathbf{x}$ 允許的分析集合，S_θ 是由參數 θ 對輸入與分析配對給出的分數。允許集合可以限制標註規則，分數則可以獎勵合理的相鄰標籤。

許多 NLP 任務都有這個差別。實體標籤必須組成有效跨度；依存樹中每個非根詞需要一個父節點；生成的結構化紀錄需要符合結構規格。較強的編碼器不會自動保證有效輸出，解碼器或獨立驗證器必須落實應用需要的限制。

結構化解碼也有取捨：錯誤的硬限制可能排除合法構造。如果文法要求每句都有明示主語，就會拒絕許多命令與片段。應記錄限制與預期輸入範圍，而不是把受限制的輸出當成該限制普遍適用所有語言的證據。

用隱藏馬可夫模型標記

白話版。想像標記形成一串隱藏序列，每個標記再產生一個詞。相鄰標記與詞的相容性共同決定分數。

Rabiner 綜述的一階隱藏馬可夫模型（hidden Markov model，HMM）[3] 使用：

$$p(\mathbf{x}, \mathbf{y}) = \pi_{y_1} b_{y_1}(x_1) \prod_{t=2}^{T} a_{y_{t-1}, y_t} b_{y_t}(x_t).$$

其中 $p(\mathbf{x}, \mathbf{y})$ 是詞與標記的聯合機率，π_k 是起始標記為 k 的機率，$a_{j,k}$ 是從標記 j 轉移到 k 的機率，$b_k(w)$ 是標記 k 發射詞 w 的機率，x_t 與 y_t 是位置 t 的詞與標記，T 是序列長度。本例以固定長度為條件；若要建模終止，可加入明確的結束狀態因子。

起始機率總和為一；每列轉移機率總和為一；每個發射分布在觀察詞彙表上的總和也為一。這些正規化條件讓式子成為機率模型，而不是任意有用分數的乘積。

假設相當有限：下一個標記只依前一個標記，發射詞只依目前標記。真實句子有長距離依存、詞彙偏好與篇章脈絡。本章使用這個模型的價值，在於假設與精確解碼器都看得見；它是一個用來區分模型容量與搜尋的小型實驗室。

如果有完整標記的訓練樣本，可以計算起始狀態、轉移與發射次數，再依明確平滑規則正規化。如果只有詞，估計隱藏狀態就是另一個問題；沒有監督學出的狀態，不會自動成為有名稱的語法類別。下例手動指定機率，避免訓練不確定性遮蔽解碼機制。

最大路徑解碼

白話版。保留以每種標記結束的最佳部分路徑。延伸這些路徑，就能找到最佳完整序列，不必列出所有可能。

使用對數機率，把乘積轉成總和：

$$\begin{aligned} D_1(k) &= \log \pi_k + \log b_k(x_1), \\ D_t(k) &= \log b_k(x_t) + \max_{j \in \mathcal{Y}} [D_{t-1}(j) + \log a_{j,k}]. \end{aligned}$$

其中 $D_t(k)$ 是涵蓋位置 1 到 t、且以標記 k 結束的最佳路徑對數分數，π_k 是起始機率，$b_k(x_t)$ 是發射機率，$a_{j,k}$ 是轉移機率，$\mathcal{Y}$ 是標記集合。j 與 k 分別指前一個與目前標記。零機率貢獻 $-\infty$。

這是最大路徑動態規劃，通常稱為 Viterbi 解碼 [3]。每格記錄達到最大值的前一個標記。選出最佳最後標記後，反向追蹤回溯指標。如果只保留分數，就會失去實際序列。

解碼器考慮 T 個位置與所有相鄰標記配對，因此 $K = |\mathcal{Y}|$ 個標記時，時間複雜度為 $O(TK^2)$。保留所有回溯指標的分數表占 $O(TK)$ 空間；列舉所有標記序列則需考慮 K^T 種可能。這是在特定分解下的計算節省，不是近似搜尋結果。

下列獨立範例以完整列舉確認動態規劃答案。教學觀察「fish fish」刻意使用歧義詞。獨立的發射偏好會在兩個位置選名詞，聯合模型則可能偏好名詞後接動詞。

```
import itertools
import numpy as np

tags = ["N", "V"]
vocabulary = ["fish", "swim"]
initial = np.array([0.8, 0.2])
transition = np.array([[0.1, 0.9], [0.8, 0.2]])
emission = np.array([[0.7, 0.3], [0.3, 0.7]])
words = ["fish", "fish"]
observation = [vocabulary.index(word) for word in words]
assert np.allclose(transition.sum(axis=1), 1)
assert np.allclose(emission.sum(axis=1), 1)

scores = np.empty((len(words), len(tags)))
back = np.zeros_like(scores, dtype=int)
scores[0] = np.log(initial) + np.log(emission[:, observation[0]])
for t in range(1, len(words)):
    candidates = scores[t - 1, :, None] + np.log(transition)
    back[t] = candidates.argmax(axis=0)
    scores[t] = candidates.max(axis=0) + np.log(emission[:, observation[t]])
path = [int(scores[-1].argmax())]
for t in range(len(words) - 1, 0, -1):
    path.append(int(back[t, path[-1]]))
path.reverse()

def joint_score(sequence):
    value = np.log(initial[sequence[0]])
    value += np.log(emission[sequence[0], observation[0]])
    for t in range(1, len(words)):
        value += np.log(transition[sequence[t - 1], sequence[t]])
        value += np.log(emission[sequence[t], observation[t]])
    return value

all_paths = list(itertools.product(range(len(tags)), repeat=len(words)))
brute = max(all_paths, key=joint_score)
assert tuple(path) == brute
print("local emission choices:", [tags[emission[:, w].argmax()] for w in observation])
print("best joint path:", [tags[index] for index in path])
print("joint log score:", round(joint_score(path), 3))
```

改變轉移機率，即使每個詞的局部偏好不變，也能改變選出的序列。這並不證明新選擇符合真實語言；它證明模型與解碼器實作了預期的依存關係。

把路徑分數解讀為證據，而不只取最大值

前例選出名詞—動詞，是因為轉移偏好超過第二個詞的發射偏好。其聯合機率是四個因子的乘積：以名詞開始、名詞發射 `fish`、名詞轉移到動詞、動詞發射 `fish`。與名詞—名詞比較時，前兩個共同因子可以消去，剩下轉移乘上發射的比較。因此，不必把解碼器當成黑箱，就能理解決策。

以下獨立計算列舉相同的四條路徑，再正規化為觀察詞給定後的後驗分布：

```
import itertools
import numpy as np

initial = np.array([0.8, 0.2])
transition = np.array([[0.1, 0.9], [0.8, 0.2]])
fish = np.array([0.7, 0.3])
paths = list(itertools.product(range(2), repeat=2))
mass = np.array([initial[a] * fish[a] * transition[a, b] * fish[b]
                 for a, b in paths])
posterior = mass / mass.sum()
print('NN / NV / VN / VV joint:', mass.round(4))
print('Posterior:', posterior.round(3))
print('Probability of observed words:', round(mass.sum(), 4))
assert paths[posterior.argmax()] == (0, 1)
assert np.isclose(posterior.sum(), 1)
```

聯合機率為 `0.0392`、`0.1512`、`0.0336`、`0.0036`，後驗四捨五入為 `0.172`、`0.664`、`0.148`、`0.016`；觀察機率為 `0.2276`。它們回答不同問題：聯合機率包含這些詞本身多可能出現，後驗則在詞已固定後比較標記解讀。在這個人工模型中，最佳路徑之外仍有其他機率質量。

較長序列若把前驅貢獻改為相加，而不是取最大值，就能計算觀察機率；結合前向與後向貢獻，則得到標記邊際機率 [3]。逐位置選最高機率標記，會最小化該後驗下的逐位置期望錯誤；選最高機率完整路徑，則針對整段序列正確性。兩種決策可能不同。若允許的序列具有額外限制，獨立選出的邊際標記也可能違反輸出限制。

為什麼最大路徑解碼能丟掉其他以同一標記結束的前綴？在一階分解下，後續延伸只依賴最後標記與未來觀察，不依賴更早的前綴細節。加入相同後綴後，較低分前綴不會超越較高分前綴。若新增分數會獎勵與數個位置之前標記的關係，原狀態便不再滿足這個論證；必須擴大狀態或更換解碼器。精確性來自分數分解，不是演算法名稱。

條件式結構分數連結神經特徵與解碼

HMM 必須建模詞如何產生；條件模型則可直接依觀察句子評分標記序列。線性鏈條件隨機場（conditional random field，CRF）提供這種介面 [5]：

$$S_\theta(\mathbf{x},\mathbf{y}) = \sum_{t=1}^{T} u_\theta(t, y_t, \mathbf{x}) + \sum_{t=2}^{T} A_{y_{t-1},y_t}, \qquad p_\theta(\mathbf{y}\mid\mathbf{x}) = \frac{\exp S_\theta(\mathbf{x},\mathbf{y})}{Z_\theta(\mathbf{x})}.$$

其中 $\mathbf{x}$ 是觀察句子，$\mathbf{y}$ 是標記序列，T 是序列長度，t 索引位置，u_θ 是參數 θ 計算的位置—標記分數，A 包含相鄰標記分數。正規化常數 $Z_\theta(\mathbf{x})$ 對每個允許序列 $\mathbf{y}'$ 的 $\exp S_\theta(\mathbf{x},\mathbf{y}')$ 加總。位置分數與 HMM 發射不同，不必在所有詞上加總為一。上下文編碼器可以用整句計算各分數，同時讓輸出依存保持鏈狀。

訓練標註序列時，最小化其負對數機率：相對於所有允許競爭序列，提高正確序列分數。計算正規化常數使用加總遞迴，尋找最佳序列使用最大值遞迴。這說明神經編碼器與結構化解碼器如何配合。豐富的輸入證據不要求放棄可計算的輸出分解；反之，鏈狀轉移表無法表達全部語法依存。整句輸入特徵與輸出限制，處理的是不同部分。

PR 06 發展這兩種模型背後的一般理論：前向與前向—後向演算法、以 Baum–Welch 從未標註序列訓練 HMM，以及以「觀察特徵次數減去期望特徵次數」表示的 CRF 梯度。下文的神經標註器各節完全重用這個 CRF，只是位置分數 u_θ 改由循環編碼器計算。

實體辨識從結構描述開始

直觀說法。實體辨識器找出提及並指定類型，不會自動找出背後唯一的現實物件。

實體提及可表示為：

$$r = (d, b, e, c).$$

其中 r 是提及紀錄；d 是文件識別碼；b 是包含在內的起始字元位移；e 是不包含在內的結束位移；c 是類別。表面文字就是原文件從 b 到 e 的切片。本例使用 Python 字串位移，其他介面可能使用位元組或別的索引慣例。必須聲明慣例，因為 Unicode 字元與編碼後的位元組不具有相同邊界。

標註指南必須決定稱謂是否包含在人名區段中、組織名稱後綴是否保留，以及歧義用法如何處理。「臺北」可能指地點，也可能在上下文中代指政府。只標位置的結構與依上下文標組織的結構，可以合理地給出不同標註。只有凍結類型與指南，評估才有意義。

命名實體辨識（named entity recognition，NER）經常區分人物、組織與地點；CoNLL-2003 共享任務論文介紹了一種既有基準設定 [6]。提到它是為了說明評估傳統，並不表示其類別適用所有應用。別的領域可能更需要產品代碼、法律條文或設備零件。

實體連結（entity linking）是另外的決策：把提及對應到知識庫紀錄，或判定沒有適當紀錄。相同拼法可能指不同組織，不同拼法也可能指同一個組織。辨識器把 Orion 標成組織，還沒有解決身分問題。連結需要候選集合、上下文與明確的未知實體政策。

儲存正規化名稱時，也應保留原文與位移。小寫化、去除重音與合併空白可能改變字串及邊界。可靠設計可直接從原文擷取，把正規化結果存為額外欄位。若必須前處理，保留對齊映射，才能把預測區段轉回原文而不靠猜測。

序列標籤與區段預測

直觀說法。實體提及可表示成詞元標籤，也可直接表示成文字區段。表示方式決定模型能描述哪些重疊。

BIO 方案使用 B 表示提及起點、I 表示延續、O 表示外部。例如 Mina works at Orion Labs 中，Mina 帶人物標籤，Orion 與 Labs 帶組織標籤。Hagiwara 的序列標註章節與 NER 綜述提供詞元層級擷取的背景 [2]、[7]。這個方案源自 Ramshaw 與 Marcus 的內外語塊標籤 [10]。

獨立詞元分類器可寫成：

$$p_\theta(\mathbf{y} \mid \mathbf{x}) = \prod_{i=1}^{T} p_\theta(y_i \mid \mathbf{x}).$$

其中 $\mathbf{x}$ 是輸入詞元序列；$\mathbf{y}$ 是標籤序列；T 是詞元數；i 是位置索引；θ 是學得的參數。每次預測可以使用完整允許範圍的上下文表示，但在這個分解下，輸出標籤彼此獨立。輸入上下文與輸出相依性是不同概念。

獨立判定可能產生非法轉移，例如 O 後面接 I-PER。可以用確定性規則修正，但修正規則也會成為評估契約的一部分。另一種方式是以整個序列評分，讓結構化解碼器只搜尋合法路徑。若分數能局部分解，最大路徑解碼一節的動態規劃概念就能套用。更強的編碼器不會免除合法輸出定義。

單一平面詞元標籤序列無法表示任意巢狀提及。在某種結構描述下，Orion Labs Research Division 可能同時包含組織與巢狀部門提及。區段模型則直接評分候選邊界與類型：

$$s_\theta(b, e, c \mid \mathbf{x}).$$

其中 s_θ 是學得的分數；b、e 是候選邊界；c 是候選類別；$\mathbf{x}$ 是輸入。這個分數未必是校準後的機率。選擇政策決定保留哪些候選、是否允許重疊，以及衝突如何解決。

候選數會隨句長增加，實務上可能限制最長區段或刪除部分候選。每種限制都會改變可找回的提及。若長名稱很重要，就要單獨量測其召回率。否則，一個快速系統可能平均表現不錯，卻完全無法表示少數使用者不可或缺的輸出。

邊界建模改變可能出現的錯誤

詞元標記模型與區段模型可以使用相同上下文編碼器，卻搜尋不同輸出空間。平面 BIO 標籤替每個詞元選一個歸屬；明確區段則選邊界配對與類型，選取規則允許時，巢狀區間可同時存在。差別在表示能力：解碼器無法表達巢狀輸出時，編碼器也無法把它恢復出來 [7]。

對 n 個詞元，非空連續區間數為 $n(n+1)/2$。其中 n 是句長；這是把每個起點可能的終點數加總。最大長度限制為 $L \le n$ 時，數量變成 $nL - L(L-1)/2$，其中 L 是允許區段長度。類型評分器可能需要為每個區間、每個類別計算分數。限制長度節省工作，卻在分類開始之前，就把較長提及移出候選空間。

邊界品質與語意分類需要分別診斷。涵蓋 `Orion` 的區段可能是強烈的組織候選，但標準提及是 `Orion Labs`。提高類別信心，不會補上缺少的邊界。模型也可能同時提出兩個區段，再由去重與重疊規則決定最後輸出。選取屬於模型行為，不能當成評估完成後的排版步驟。

關係又增加一個搜尋問題。保留越多提及，實體配對候選越多。依類型或距離限制配對，能降低計算，但文件層級關係可能跨越數句。聯合擷取器可用關係證據影響提及決策；流程式模型則先選定提及。分別以預測提及與標準提及評估流程，兩者差距能揭露在關係評分開始之前，就損失多少關係涵蓋能力。

精確區段評估

直觀說法。常見背景詞標得正確，可能掩蓋漏掉的實體。應評估完整提及，包括邊界與類型。

對預測紀錄集合 P 與正確標註集合 G：

$$\text{precision} = \frac{|P \cap G|}{|P|}, \qquad \text{recall} = \frac{|P \cap G|}{|G|}, \qquad F_1 = \frac{2|P \cap G|}{|P| + |G|}.$$

其中 P、G 都包含文件識別碼、起始位移、結束位移與類別元組；$|\cdot|$ 是集合大小；$P \cap G$ 是完全匹配的紀錄。精確率衡量預測中匹配的比例；召回率衡量標註中找回的比例；F_1 結合兩者。本例程式在所示分母為零時回報零；若採其他慣例，應明確記錄。

完全匹配很嚴格。正確標註是 Orion Labs，卻只預測 Orion，就同時產生假陽性與假陰性。下游程式依賴完整名稱時，這種標準合理。其他應用可以把部分重疊當成次要診斷，但必須另外說明匹配規則，不能把寬鬆的重疊分數當成既有的完全匹配指標 [6]。

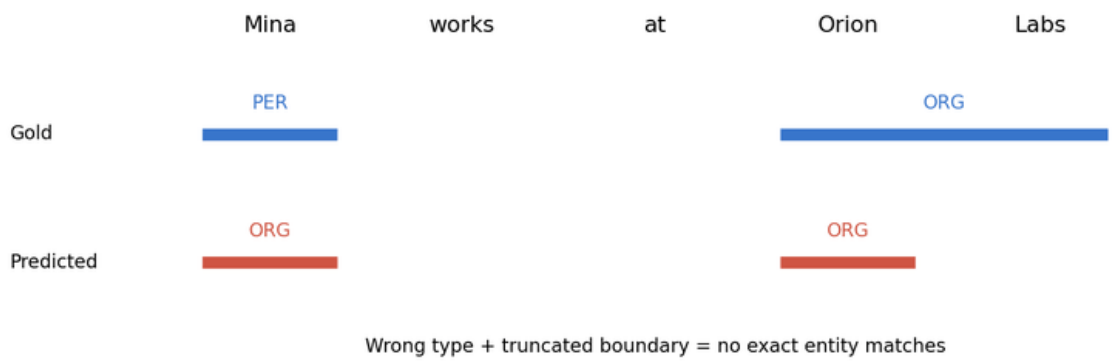


圖 *1* — 即使大多數詞語標得正確，邊界與類型錯誤仍然是錯誤。這是合成的診斷示例。

單一總分仍會隱藏原因。可區分漏掉提及、多餘提及、邊界不符、類別不符與連結不符，並按類別和文件來源報告表現。類別稀少時保留例數，讓讀者判斷不確定性。大型加權平均可能掩蓋幾乎沒有成功預測的類別。

完整的 CPU 擷取實驗

直觀說法。小型確定性擷取器能讓契約容易檢查。它是盲點可見的教學基準，不是通用語言理解系統。

以下獨立範例辨識自訂名稱表、擷取嚴格受限的主動修理句型，並評估精確實體紀錄。它不下載任何資料。名稱表在評估前固定，正確標註也明確寫出。本例允許使用詞彙表；若真實評估禁止外部名稱集合，同樣程式就會違反契約。

```
import re

documents = {
    "a": "Mina repaired Sensor A in Taipei.",
    "b": "Orion Labs said Sensor B might fail.",
    "c": "Leo did not repair Sensor B.",
    "d": "The sensor was repaired by Mina.",
}
inventory = {
    "Mina": "PER", "Leo": "PER", "Orion Labs": "ORG",
    "Sensor A": "DEVICE", "Sensor B": "DEVICE", "Taipei": "LOC",
}
gold = {
    ("a", 0, 4, "PER"), ("a", 14, 22, "DEVICE"), ("a", 26, 32, "LOC"),
    ("b", 0, 10, "ORG"), ("b", 16, 24, "DEVICE"),
    ("c", 0, 3, "PER"), ("c", 19, 27, "DEVICE"), ("d", 27, 31, "PER"),
}
predicted = set()
for doc_id, text in documents.items():
    for name, category in inventory.items():
        pattern = r"(?<!\w)" + re.escape(name) + r"(?!\w)"
        for match in re.finditer(pattern, text):
            predicted.add((doc_id, match.start(), match.end(), category))
tp = len(predicted & gold)
precision = tp / len(predicted) if predicted else 0.0
recall = tp / len(gold) if gold else 0.0
f1 = 2 * tp / (len(predicted) + len(gold)) if predicted or gold else 0.0
assert predicted == gold
print("entity precision/recall/F1:", precision, recall, f1)

repair = re.compile(r"^(Mina|Leo) repaired (Sensor [AB])(?=[ .])")
events = []
for doc_id, text in documents.items():
    match = repair.search(text)
    if match:
        events.append({
            "document": doc_id, "predicate": "repair",
            "actor": match.group(1), "object": match.group(2),
            "polarity": "positive", "modality": "asserted",
            "evidence": [match.start(), match.end()],
        })
assert len(events) == 1 and events[0]["document"] == "a"
for event in events:
    start, end = event["evidence"]
    print(event, "source:", documents[event["document"]][start:end])
print("unsupported repair constructions:", ["c", "d"])
```

在這些人工設計文件中，精確實體分數是 1.0，因為所有名稱拼法都在表中。這不能證明泛化能

力。事件規則只回傳文件 a；它排除了 c 的明確否定，卻也漏掉 d 的合法被動句。兩者都是沒有事件輸出，但原因不同。正式介面應區分被拒絕的否定主張與不支援的句型。

文件 b 有已辨識實體，卻沒有被直接斷言的修理事件；它還包含帶消息來源、可能發生的故障。這說明為何實體與事件需要分開評估。系統可能抓對所有名字，卻遺漏任務最重要的事件。擴充事件語法前，要先加入額外的正例與反例測試。

證據區間只包含匹配子句，不包含後面的地點。對沒有地點欄位的狹義修理紀錄，這樣足夠。若之後加入地點，就要一併擴充證據契約與測試。結構變更應視為任務變更，不能只是新增一個欄位，再隨意從附近不相關詞語填值。

實體區段之外的資訊擷取

白話版。實體辨識找出名稱；資訊擷取（information extraction，IE）接著把它們連起來：哪個人為哪個組織工作、哪台裝置在何時何地故障。每一步都加入一種新紀錄，也多一種出錯的方式。

一張有用的地圖分成三層，每層建立在前一層之上。

層次	輸出紀錄	本章領域的例子	常見方法類型
實體	有類型的區段 (d, b, e, c)	`Orion Labs` 是位於字元 0–10 的 ORG	序列標註（本章）或區段評分
關係	有類型、有方向的提及配對，附證據	`employer(Mina, Orion Labs)`	分類候選提及配對
事件	觸發詞加上帶角色標籤的論元、肯否定與模態	`repair`，修理者 `Mina`、物件 `Sensor A`、地點 `Taipei`	先偵測觸發詞，再標註論元

序列標註直接貢獻於三個層次。實體提及是 BIO 區段；事件觸發詞可以用與實體相同的方式標註；甚至部分關係與論元判定，也能相對於給定觸發詞標註每個論元的詞元，寫成標註問題。上面的規則式擷取器已經顯示層次之間的依賴：它唯一的事件紀錄之所以正確，是因為兩個實體區段都被找到，而且句型受到支援。

標準資源反映了這些層次。ACE 2005 多語訓練語料替英文、中文與阿拉伯文標註實體、關係與事件，其中事件標註只限英文與中文 [19]。TACRED 是以群眾外包建立的句子層級關係擷取資料集，針對 TAC 知識庫填充（Knowledge Base Population）評測的關係類型 [20]。DocRED 把關係擷取擴展到由 Wikipedia 與 Wikidata 建立的完整文件，其中許多關係必須讀過多個句子才能推得 [21]。本章前面提出的紀錄設計問題（證據區段、消息歸屬、否定、未知值）適用於每一層。關係與事件擷取、語意角色及其評估，在NLP 12 發展。

神經標註器：讀取整句的 BiLSTM

白話版。以一個雙向讀取整句、在每個位置替每個標籤產生分數的神經網路，取代手寫的發射機率。最簡單的版本在每個位置獨立選出最高分標籤。

Collobert 等人證明，單一神經架構幾乎不需特徵工程，就能從資料學會標註詞性、語塊、命名實體與語意角色 [12]。使用循環編碼器時，標準設計是雙向 LSTM（BiLSTM）標註器 [13]、[14]、[15]：

$$\overrightarrow{\mathbf{h}}_t = \mathrm{LSTM}_{\rightarrow}(\mathbf{v}_t, \overrightarrow{\mathbf{h}}_{t-1}), \qquad \overleftarrow{\mathbf{h}}_t = \mathrm{LSTM}_{\leftarrow}(\mathbf{v}_t, \overleftarrow{\mathbf{h}}_{t+1}),$$
$$\mathbf{u}_t = W\,[\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t] + \mathbf{b} \in \mathbb{R}^K.$$

其中 $\mathbf{v}_t$ 是詞元 x_t 的輸入向量（詞嵌入，可再串接下文的字元特徵）；$\overrightarrow{\mathbf{h}}_t$ 讀過 $x_1, \dots, x_t$，$\overleftarrow{\mathbf{h}}_t$ 讀過 $x_t, \dots, x_T$；$[\cdot;\cdot]$ 表示串接；W 與 $\mathbf{b}$ 把兩個狀態對應到 $K = |\mathcal{Y}|$ 個標籤分數；$\mathbf{u}_t$ 收集這些分數，標籤 k 的分數為 $u_{t,k}$。這些分數扮演前面 CRF 公式中 $u_\theta(t, y_t, \mathbf{x})$ 的角色：它們可以依賴整個句子。與 NLP 05 由左到右的語言模型不同，這裡適合使用雙向結構，因為選擇任何標籤之前整個句子都已給定 [23]。

softmax 標註器在給定這些分數後，把各標籤視為條件獨立：

$$p_\theta(y_t = k \mid \mathbf{x}) = \frac{\exp u_{t,k}}{\sum_{k'=1}^{K} \exp u_{t,k'}}, \qquad \mathcal{L}_{\text{tok}} = -\frac{1}{N} \sum_{b} \sum_{t} m_{b,t} \log p_\theta(y_{b,t} \mid \mathbf{x}_b).$$

其中 b 遍歷小批次中的句子，$m_{b,t} \in \{0, 1\}$ 是填補遮罩（真實詞元為一），$N = \sum_{b,t} m_{b,t}$ 計算真實詞元數，$y_{b,t}$ 是正確標籤，k' 遍歷標籤。這正是序列標籤一節中的獨立詞元分類器，只是改由 BiLSTM 計算上下文。它速度快，也常常很強；它的弱點也與那裡指出的相同：沒有任何機制阻止它輸出 `O I-ORG` 或 `B-PER I-ORG`，因為每個位置的判定都忽略了鄰近位置的判定。

三項實務細節與架構同樣重要。

- **遮罩**。批次中的句子會填補到相同長度。遮罩把填補排除在損失之外；打包（`pack_padded_sequence`）或遮罩則讓填補不進入反向 LSTM，否則它會從填補符號開始讀。
- **未知詞**。詞嵌入表只涵蓋訓練詞。其餘詞對應到未知編號，而且要訓練這個編號：隨機把一部分訓練詞換成它（詞 dropout），可以教標註器在詞本身陌生時依賴上下文。
- **子詞編碼器**。若編碼器使用子詞，要定義詞標籤如何對應到片段（只標第一個片段並忽略其餘，或另一條明確規則），並在評估前把預測對應回詞，如第一節所述。

BiLSTM-CRF：替整個標籤序列評分

白話版。保留 BiLSTM 的逐詞元分數，再替每一對相鄰標籤加上學習得到的分數，選出總分最高的標籤序列。訓練時提高正確序列相對於所有其他序列的分數，而前向演算法能有效率地完成這個加總。

Huang、Xu 與 Yu 提出在 BiLSTM 上加一層線性鏈 CRF，用於詞性標註、語塊切分與 NER [13]。Lample 等人 [14] 以及 Ma 與 Hovy [15] 把同樣的輸出層與字元層級詞表示結合，在不使用手工特徵的情況下得到強大的 NER 結果。沿用最大路徑解碼一節的 CRF 記號，並加上起始與結束分數：

$$S_\theta(\mathbf{x}, \mathbf{y}) = \pi_{y_1} + \sum_{t=1}^{T} u_{t,y_t} + \sum_{t=2}^{T} A_{y_{t-1}, y_t} + \eta_{y_T}, \qquad \log p_\theta(\mathbf{y} \mid \mathbf{x}) = S_\theta(\mathbf{x}, \mathbf{y}) - \log Z_\theta(\mathbf{x}).$$

其中 $u_{t,k}$ 是 BiLSTM 分數；$A_{j,k}$ 是標籤 j 後接標籤 k 的學習分數；π_k 與 η_k 是以標籤 k 起始與結束的分數；$Z_\theta(\mathbf{x}) = \sum_{\mathbf{y}'} \exp S_\theta(\mathbf{x}, \mathbf{y}')$ 對所有長度為 T 的標籤序列 $\mathbf{y}'$ 加總。訓練損失是對句子平均的負對數概似 $-\log p_\theta(\mathbf{y} \mid \mathbf{x})$。

對 K^T 個序列的加總，以對數空間的前向演算法計算：

$$\alpha_1(k) = \pi_k + u_{1,k},$$
$$\alpha_t(k) = u_{t,k} + \log \sum_{j=1}^{K} \exp\left(\alpha_{t-1}(j) + A_{j,k}\right), \qquad t = 2, \dots, T,$$
$$\log Z_\theta(\mathbf{x}) = \log \sum_{k=1}^{K} \exp\left(\alpha_T(k) + \eta_k\right).$$

其中 $\alpha_t(k)$ 是涵蓋位置 1 到 t、以標籤 k 結尾的所有部分序列，其指數分數總和的對數。把每個 log-sum-exp 換成最大值並記錄取得最大值的 j，就得到 HMM 一節的 Viterbi 遞迴，只是 $\log b_k(x_t)$ 換成 $u_{t,k}$、$\log a_{j,k}$ 換成 $A_{j,k}$。兩者的成本都是 $O(TK^2)$。在填補過的批次中，遮罩為零的位置直接把 α_{t-1} 原樣往前複製，因此每個句子的正規化常數都停在它自己的長度。

梯度在做什麼。損失對轉移分數微分得到

$$\frac{\partial(-\log p_\theta(\mathbf{y} \mid \mathbf{x}))}{\partial A_{j,k}} = \mathbb{E}_{\mathbf{y}' \sim p_\theta(\cdot \mid \mathbf{x})}[n_{j,k}(\mathbf{y}')] - n_{j,k}(\mathbf{y}),$$

其中 $n_{j,k}(\mathbf{y})$ 計算 $\mathbf{y}$ 中標籤 j 後接標籤 k 的次數，期望值依模型分布計算。因此梯度下降會提高正確序列使用得比模型目前預期更多的轉移，並降低其他轉移；期望值來自前向—後向邊際機率，如 PR 06 所示。對前向演算法自動微分就能算出這個量，不必手寫反向傳播。

硬性限制。 BIO 方案直接禁止某些轉移：`I-x` 只能接在 `B-x` 或 `I-x` 之後，而且句子不能以 `I-x` 開頭。學習得到的 A 通常能從資料發現這一點，但資料很少時未必如此。把 A 與 π 中被禁止的項設為 $-\infty$（實務上用很大的負數），就讓非法序列在訓練與解碼時都不可能出現，而對合法序列毫無影響。這是局部與序列預測一節中輸出限制概念的具體實作。

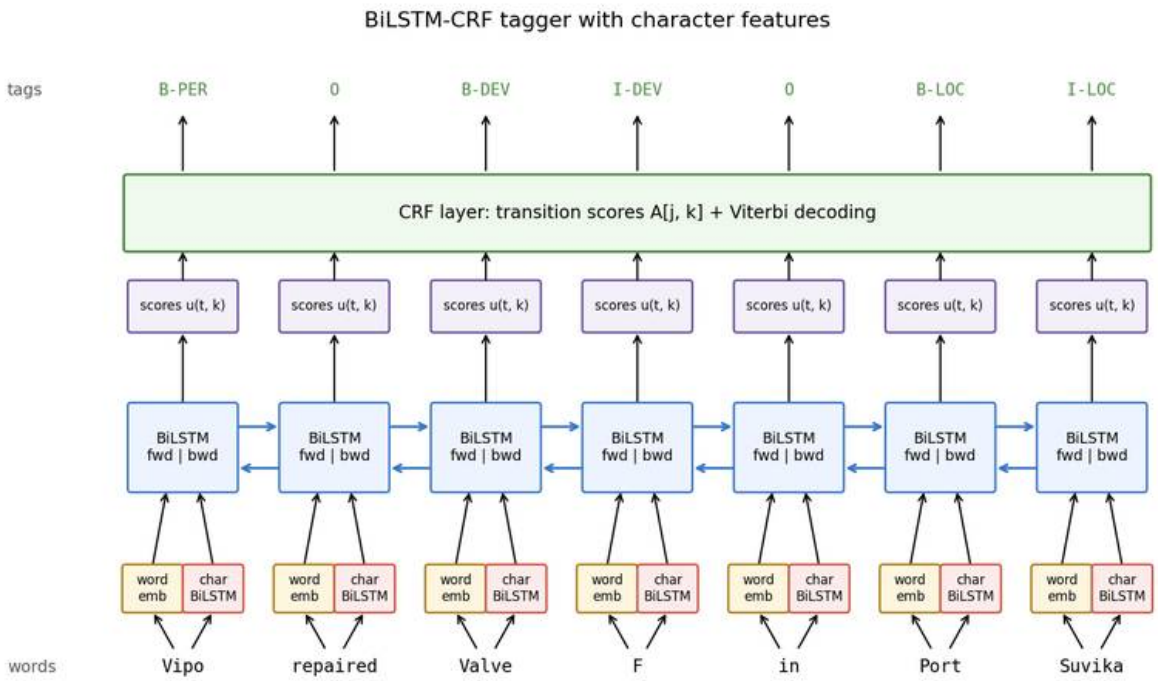


圖 2 — 帶字元特徵的 *BiLSTM-CRF* 標註器。發射分數來自編碼器；轉移分數與 *Viterbi* 解碼讓各標籤彼此一致。

字元層級特徵

白話版。詞嵌入表對沒見過的詞一無所知。詞的字母卻常常知道：大寫、「-ville」或「-corp」這類字尾、數字與連字號，即使對全新的名稱也是很強的標籤線索。

在神經標註器之前，這些線索是手寫特徵（是否大寫、是否以「-ing」結尾、是否含數字）。神經標註器則從字元學習它們。Lample 等人在每個詞的字元上執行一個小型雙向 LSTM，並把最後的前向與反向狀態串接到詞嵌入 [14]；Ma 與 Hovy 則對字元嵌入做卷積，再做最大池化 [15]。無論哪一種，輸入都變成

$$\mathbf{v}_t = [\mathbf{e}(x_t);\, \mathbf{c}(x_t)], \qquad \mathbf{c}(x_t) = [\vec{\mathbf{g}}_L; \overleftarrow{\mathbf{g}}_1],$$

其中 $\mathbf{e}(x_t)$ 是詞嵌入，$\mathbf{c}(x_t)$ 是字元摘要，L 是 x_t 的字元數，$\vec{\mathbf{g}}_L$ 是讀完最後一個字元後的前向字元 LSTM 狀態，$\overleftarrow{\mathbf{g}}_1$ 是反向讀回第一個字元後的狀態。詞層級 BiLSTM 接著讀取 $\mathbf{v}_t$。Akbik、Blythe 與 Vollgraf 把這個想法再往前推：他們像 NLP 05 一樣訓練字元層級語言模型，再把每個詞周圍的內部狀態當作序列標註用的上下文嵌入 [16]。

字元特徵正好幫助詞嵌入最難處理的輸入：未見名稱、罕見詞形變化與拼寫錯誤。但它們無法解決拼字本身沒有編碼的歧義。沒有明顯字尾的名稱可能是人，也可能是公司，只有上下文或世界知識能決定。字元輸入也需要自己的未知符號規則，處理訓練中沒出現過的字元。

標註與擷取的資料集

白話版。基準會固定標籤集合、標註指南、分詞方式與資料切分。只有四者都一致時，跨基準比較數字才有意義。

資源	任務	提供什麼
Penn Treebank [11]	英文詞性標註與剖析	大型英文標註語料，含詞性標籤與句法括號結構；其標籤集廣泛用於英文
Universal Dependencies [4]	跨語言詞性、詞法與依存關係	套用到多種語言樹庫的共同標註框架
CoNLL-2003 [6]	命名實體辨識	英文與德文資料，含四種類型（人物、組織、地點、其他）與完全匹配評估
OntoNotes [17]	18 種類型的命名實體，以及句法、語意角色與共指	多文體、多語言標註，具分層抽樣的訓練、開發與測試切分，其測試集與 CoNLL-2012 共享任務一致
WNUT 2017 [18]	雜訊使用者生成文字中的新興實體	測試罕見與先前未見表面形式的召回率
ACE 2005 [19]	實體、關係與事件	多語言標註（英文、中文、阿拉伯文），用於關係與事件擷取
TACRED [20]	句子層級關係擷取	針對 TAC KBP 關係類型、以群眾外包建立的關係標籤
DocRED [21]	文件層級關係擷取	在 Wikipedia 文件上標註的實體與關係，需要跨句推理

標籤編碼也各不相同。CoNLL-2003 檔案使用源自 Ramshaw 與 Marcus 語塊標籤的 IOB 變體，其中 `B-` 只出現在某個實體緊接在另一個同類型實體之後 [6]、[10]；訓練前應把這類檔案轉成本章使用的 BIO 形式，否則 BIO 區段擷取器會誤讀。授權也不同：有些資源可以自由下載，有些則由語言資料聯盟（Linguistic Data Consortium）依授權發行。使用前先確認條款。對新領域而言，最有價值的資料集通常是依你自己凍結的指南、仔細裁定過的小型標註集，再搭配公開基準來比較方法。

完整的 PyTorch 標註實驗

白話版。我們在同一個小型合成語料上訓練三個標註器，並在名稱從未出現在訓練中的句子上評估：只用詞的 BiLSTM、加上字元特徵的同一標註器，以及在字元標註器上加上受限 CRF 輸出層。評估計算精確區段、錯誤類型與非法標籤轉移。

語料是虛構的。人名、組織名與地名由隨機音節產生，因此測試名稱都是新拼法。組織名要嘛以 `tek` 或 `corp` 結尾，要嘛後面接 `Labs` 這個詞；地名要嘛以 `ville` 結尾，要嘛接在 `Port` 之後。有幾個模板允許同一位置放人或組織，所以單靠上下文不一定能決定類型。訓練句有 100 句，測試句有 200 句。其他所有東西都從零實作：從 BIO 標籤擷取區段、精確區段的精確率、召回率與 F1、錯誤分類、填補遮罩、詞 dropout、字元 BiLSTM，以及具遮罩前向演算法、Viterbi 解碼與 BIO 限制的 CRF，並以暴力列舉驗證。每個模型以三個種子訓練；整個程式在 CPU 上遠少於一分鐘即可完成。

```
import random
import torch
from torch import nn

# ---- 1. A synthetic tagged corpus: invented names, fixed templates ----
SYL = ["ka", "mi", "to", "re", "na", "lo", "su", "vi", "da", "po", "ri", "ze"]

def coin(rng, kind):
    stem = "".join(rng.choice(SYL) for _ in range(rng.choice((2, 3)))).capitalize()
    if kind == "PER":
        return stem
    if kind == "ORG":
        return rng.choice([stem + rng.choice(["tek", "corp"]), stem + " Labs"])
    return rng.choice(["Port " + stem, stem + "ville"])           # LOC

rng = random.Random(0)
TRAIN_NAMES = {k: sorted({coin(rng, k) for _ in range(40)}) for k in ("PER", "ORG", "LOC")}
seen = {n for v in TRAIN_NAMES.values() for n in v}
TEST_NAMES = {k: rng.sample(sorted({coin(rng, k) for _ in range(60)} - seen), 15)
              for k in ("PER", "ORG", "LOC")}                      # unseen spellings
TRAIN_NAMES["DEV"] = ["Pump A", "Pump B", "Sensor C", "Sensor D", "Valve E", "Valve F"]
TEST_NAMES["DEV"] = ["Pump G", "Sensor H", "Valve J", "Pump K"]
TEMPLATES = [
    "{PER} repaired {DEV} in {LOC} .",
    "{ORG} repaired {DEV} in {LOC} .",
    "{PER} reported a fault in {DEV} .",
    "{ORG} reported a fault in {DEV} .",
    "the sensor near {LOC} was checked by {PER} .",
```

```
    "the sensor near {LOC} was checked by {ORG} .",
    "{PER} and {PER} visited {LOC} .",
    "{ORG} shipped {DEV} to {LOC} yesterday .",
    "after the test , {PER} said {DEV} might fail .",
    "a pump in {LOC} failed and {ORG} replaced it .",
]

def make(names, n):
    data = []
    for _ in range(n):
        words, tags = [], []
        for piece in rng.choice(TEMPLATES).split():
            if piece.startswith("{"):
                kind = piece[1:-1]
                toks = rng.choice(names[kind]).split()
                words += toks
                tags += ["B-" + kind] + ["I-" + kind] * (len(toks) - 1)
            else:
                words.append(piece)
                tags.append("O")
        data.append((words, tags))
    return data

train_data, test_data = make(TRAIN_NAMES, 100), make(TEST_NAMES, 200)
TAGS = ["O"] + [f"{p}-{k}" for k in ("PER", "ORG", "LOC", "DEV") for p in "BI"]
t2i = {t: i for i, t in enumerate(TAGS)}
w2i = {w: i + 2 for i, w in enumerate(sorted({w for s, _ in train_data for w in s}))}
c2i = {c: i + 2 for i, c in enumerate(sorted({c for w in w2i for c in w}))}
PAD, UNK = 0, 1
print(train_data[0])

# ---- 2. Spans and exact span F1, from scratch ----
def spans(tags):
    out, start, kind = set(), None, None
    for i, t in enumerate(tags + ["O"]):
        if start is not None and not (t.startswith("I-") and t[2:] == kind):
            out.add((start, i, kind))
            start = None
        if t.startswith("B-") or (t.startswith("I-") and start is None):
            start, kind = i, t[2:]                  # an orphan I- still opens a span
    return out

def span_prf(gold_seqs, pred_seqs):
    tp = n_pred = n_gold = 0
    for g, p in zip(gold_seqs, pred_seqs):
        G, P = spans(g), spans(p)
        tp, n_pred, n_gold = tp + len(G & P), n_pred + len(P), n_gold + len(G)
    prec = tp / n_pred if n_pred else 0.0
    rec = tp / n_gold if n_gold else 0.0
    f1 = 2 * tp / (n_pred + n_gold) if n_pred + n_gold else 0.0
    return prec, rec, f1

def error_types(gold_seqs, pred_seqs):
    counts = {"type": 0, "boundary": 0, "missed": 0, "spurious": 0}
```

```
    for g, p in zip(gold_seqs, pred_seqs):
        G, P = spans(g), spans(p)
        overlap = lambda a, b: a[0] < b[1] and b[0] < a[1]
        for s in G - P:
            hits = [q for q in P if overlap(s, q)]
            if any(q[:2] == s[:2] for q in hits):
                counts["type"] += 1             # right boundaries, wrong category
            elif hits:
                counts["boundary"] += 1         # overlaps, wrong boundaries
            else:
                counts["missed"] += 1
        counts["spurious"] += sum(not any(overlap(q, s) for s in G) for q in P - G)
    return counts

def invalid(tags):                                  # I-x not preceded by B-x or I-x
    return sum(t.startswith("I-") and (i == 0 or tags[i - 1][2:] != t[2:])
               for i, t in enumerate(tags))

assert spans(["B-ORG", "I-ORG", "O", "B-PER"]) == {(0, 2, "ORG"), (3, 4, "PER")}
assert invalid(["O", "I-LOC", "B-DEV", "I-DEV"]) == 1
print("train/test sentences:", len(train_data), len(test_data),
      " gold test spans:", sum(len(spans(t)) for _, t in test_data))

# ---- 3. Padding, masks and word dropout ----
def batch(data, unk_rate=0.0, r=None):
    L = max(len(s) for s, _ in data)
    W = max(len(w) for s, _ in data for w in s)
    x = torch.full((len(data), L), PAD)
    c = torch.full((len(data), L, W), PAD)
    y = torch.zeros(len(data), L, dtype=torch.long)
    for b, (s, t) in enumerate(data):
        for i, w in enumerate(s):
            drop = r is not None and r.random() < unk_rate
            x[b, i] = UNK if drop else w2i.get(w, UNK)      # train the unknown id too
            y[b, i] = t2i[t[i]]
            for j, ch in enumerate(w):
                c[b, i, j] = c2i.get(ch, UNK)
    return x, c, y, x != PAD

# ---- 4. BiLSTM encoder with optional character BiLSTM features ----
class Encoder(nn.Module):
    def __init__(self, use_chars, wdim=32, cdim=16, hidden=48):
        super().__init__()
        self.use_chars = use_chars
        self.wemb = nn.Embedding(len(w2i) + 2, wdim, padding_idx=PAD)
        self.cemb = nn.Embedding(len(c2i) + 2, cdim, padding_idx=PAD)
        self.clstm = nn.LSTM(cdim, cdim, batch_first=True, bidirectional=True)
        self.lstm = nn.LSTM(wdim + (2 * cdim if use_chars else 0), hidden,
                            batch_first=True, bidirectional=True)
        self.emit = nn.Linear(2 * hidden, len(TAGS))

    def forward(self, x, c):
        feats = [self.wemb(x)]
        if self.use_chars:
```

```
            B, L, W = c.shape
            lengths = (c != PAD).sum(-1).clamp(min=1).view(-1)
            packed = nn.utils.rnn.pack_padded_sequence(
                self.cemb(c.view(B * L, W)), lengths, batch_first=True,
                enforce_sorted=False)
            _, (h, _) = self.clstm(packed)          # last forward + first backward state
            feats.append(torch.cat([h[0], h[1]], -1).view(B, L, -1))
        h, _ = self.lstm(torch.cat(feats, -1))
        return self.emit(h)                         # emission scores u[b, t, k]

# ---- 5. Linear-chain CRF with BIO constraints and a padding mask ----
class CRF(nn.Module):
    def __init__(self, K):
        super().__init__()
        self.A = nn.Parameter(torch.zeros(K, K))    # A[j, k]: score of tag j -> tag k
        self.start = nn.Parameter(torch.zeros(K))
        self.end = nn.Parameter(torch.zeros(K))
        ban = torch.zeros(K, K)                     # I-x only after B-x or I-x
        for k, tag in enumerate(TAGS):
            if tag.startswith("I-"):
                ban[:, k] = -1e4
                ban[t2i["B-" + tag[2:]], k] = ban[k, k] = 0.0
        self.register_buffer("ban", ban)
        self.register_buffer("ban_start", torch.tensor(
            [-1e4 if t.startswith("I-") else 0.0 for t in TAGS]))

    def trans(self):
        return self.A + self.ban

    def score(self, u, y, mask):                    # S(x, y) for the gold path
        s = (self.start + self.ban_start)[y[:, 0]] + u[:, 0].gather(1, y[:, :1]).squeeze(1)
        for t in range(1, u.size(1)):
            step = self.trans()[y[:, t - 1], y[:, t]] + u[:, t].gather(1, y[:, t:t +
             ↪  1]).squeeze(1)
            s = s + step * mask[:, t]
        last = y.gather(1, (mask.sum(1) - 1).unsqueeze(1)).squeeze(1)
        return s + self.end[last]

    def log_partition(self, u, mask):               # forward algorithm in log space
        alpha = self.start + self.ban_start + u[:, 0]
        for t in range(1, u.size(1)):
            nxt = torch.logsumexp(alpha.unsqueeze(2) + self.trans() + u[:, t].unsqueeze(1),
             ↪  dim=1)
            alpha = torch.where(mask[:, t:t + 1], nxt, alpha)   # padding: carry forward
        return torch.logsumexp(alpha + self.end, dim=1)

    def nll(self, u, y, mask):
        return (self.log_partition(u, mask) - self.score(u, y, mask)).mean()

    def viterbi(self, u, mask):                     # same recursion with max + backpointers
        delta, back = self.start + self.ban_start + u[:, 0], []
        for t in range(1, u.size(1)):
            best, arg = (delta.unsqueeze(2) + self.trans()).max(1)
            back.append(arg)
```

```
            delta = torch.where(mask[:, t:t + 1], best + u[:, t], delta)
        delta = delta + self.end
        paths = []
        for b in range(u.size(0)):
            k = int(delta[b].argmax())
            path = [k]
            for t in range(int(mask[b].sum()) - 1, 0, -1):
                k = int(back[t - 1][b, k])
                path.append(k)
            paths.append(path[::-1])
        return paths

# Check the masked forward algorithm and Viterbi against brute-force enumeration
import itertools
torch.manual_seed(0)
check = CRF(len(TAGS))
with torch.no_grad():
    check.A.normal_()
u = torch.randn(1, 3, len(TAGS))
m = torch.ones(1, 3, dtype=torch.bool)
paths = [torch.tensor([p]) for p in itertools.product(range(len(TAGS)), repeat=3)]
scores = torch.stack([check.score(u, p, m)[0] for p in paths])
assert torch.allclose(check.log_partition(u, m)[0], torch.logsumexp(scores, 0), atol=1e-4)
assert paths[int(scores.argmax())][0].tolist() == check.viterbi(u, m)[0]

# ---- 6. Train three taggers and evaluate exact spans on unseen names ----
def run(use_chars, use_crf, seed, epochs=15):
    torch.manual_seed(seed)
    r = random.Random(seed)
    enc, crf = Encoder(use_chars), CRF(len(TAGS))
    params = list(enc.parameters()) + (list(crf.parameters()) if use_crf else [])
    opt = torch.optim.Adam(params, lr=5e-3)
    order = list(range(len(train_data)))
    for _ in range(epochs):
        r.shuffle(order)
        for i in range(0, len(order), 32):
            x, c, y, mask = batch([train_data[j] for j in order[i:i + 32]], 0.25, r)
            u = enc(x, c)
            loss = (crf.nll(u, y, mask) if use_crf      # sequence-level loss
                    else nn.functional.cross_entropy(u[mask], y[mask]))  # per token
            opt.zero_grad()
            loss.backward()
            nn.utils.clip_grad_norm_(params, 5.0)
            opt.step()
    enc.eval()
    with torch.no_grad():
        x, c, y, mask = batch(test_data)
        u = enc(x, c)
        pred = (crf.viterbi(u, mask) if use_crf else
                [u[b, :int(mask[b].sum())].argmax(-1).tolist() for b in range(len(u))])
    pred = [[TAGS[k] for k in p] for p in pred]
    gold = [t for _, t in test_data]
    return span_prf(gold, pred), sum(map(invalid, pred)), error_types(gold, pred), crf
```

```
results = {}
for name, use_chars, use_crf in [("BiLSTM, words", False, False),
                                 ("BiLSTM, words+chars", True, False),
                                 ("BiLSTM-CRF, words+chars", True, True)]:
    runs = [run(use_chars, use_crf, seed) for seed in (0, 1, 2)]
    results[name] = runs
    f1s = [prf[2] for prf, _, _, _ in runs]
    errors = {k: sum(r[2][k] for r in runs) for k in runs[0][2]}
    print(f"{name}: F1 per seed {[round(f, 3) for f in f1s]}, mean {sum(f1s) / 3:.3f}")
    print(f"    invalid I- tags {[r[1] for r in runs]}, errors over 3 seeds {errors}")
assert all(r[1] == 0 for r in results["BiLSTM-CRF, words+chars"])
```

輸出：

```
(['Vipo', 'repaired', 'Valve', 'F', 'in', 'Port', 'Suvika', '.'], ['B-PER', 'O', 'B-DEV',
 ↪  'I-DEV', 'O', 'B-LOC', 'I-LOC', 'O'])
train/test sentences: 100 200  gold test spans: 497
BiLSTM, words: F1 per seed [0.893, 0.771, 0.82], mean 0.828
    invalid I- tags [0, 30, 0], errors over 3 seeds {'type': 66, 'boundary': 43, 'missed':
     ↪  247, 'spurious': 0}
BiLSTM, words+chars: F1 per seed [0.893, 0.903, 0.942], mean 0.913
    invalid I- tags [4, 1, 0], errors over 3 seeds {'type': 124, 'boundary': 2, 'missed': 7,
     ↪  'spurious': 0}
BiLSTM-CRF, words+chars: F1 per seed [0.915, 0.909, 0.961], mean 0.929
    invalid I- tags [0, 0, 0], errors over 3 seeds {'type': 102, 'boundary': 0, 'missed': 8,
     ↪  'spurious': 0}
```

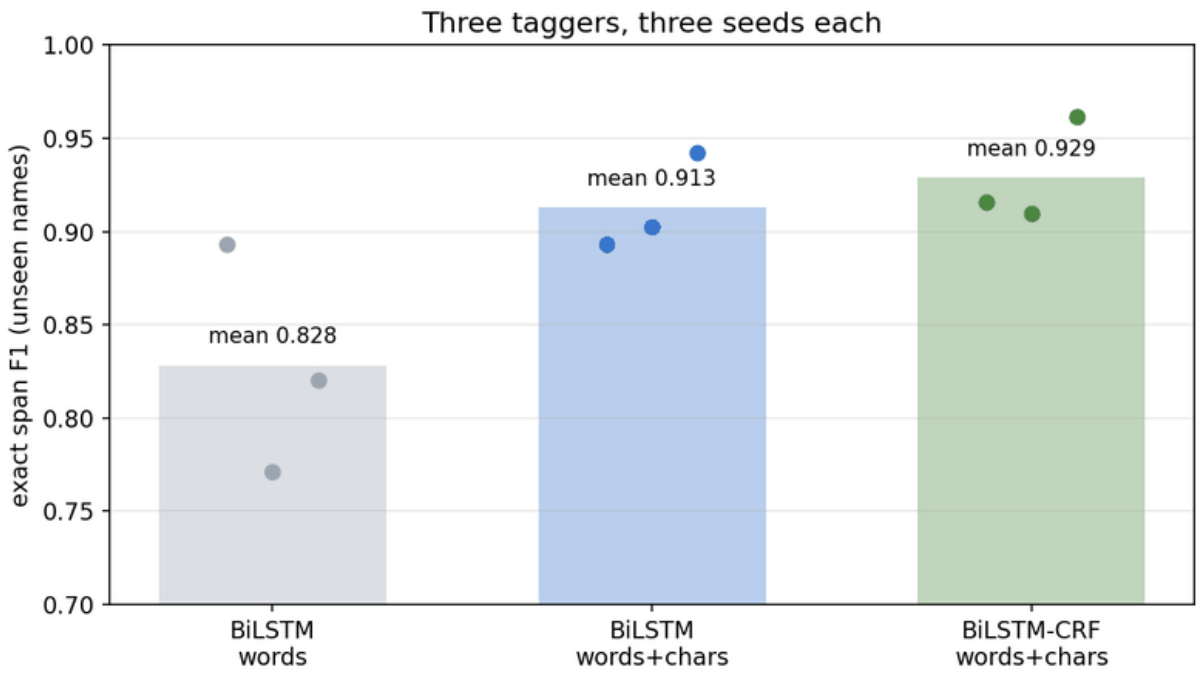


圖 3 — 在名稱從未出現在訓練中的測試句上的精確區段 *F1*。點是各種子，長條是平均。字元特徵帶來最大增益；受限 *CRF* 再加上較小但一致的改善。

解讀結果

字元特徵修好偵測。只用詞的標註器把大多數測試名稱看成未知編號。它的錯誤以漏掉實體為主（三個種子共 247 個），另外還有兩詞元名稱被切開或截短造成的邊界錯誤。加入字元特徵後，漏掉的實體降到 7 個，邊界錯誤降到 2 個：新名稱的字尾與大寫已足以找出它們。平均 F1 從 0.828 提高到 0.913。

剩下的是類型問題。兩個字元模型剩下的錯誤幾乎都是類型錯誤（三個種子共 124 與 102 個）：區段正確，但把人標成組織或反過來。這些都是單詞元名稱，常出現在同一位置可放任一類型的模板中，例如 `Mimitek reported a fault` 或 `checked by Renaka`。上下文無法決定這些情況；在這個語料中人名的拼字不帶類型標記；而只用 100 句訓練的模型，還沒有可靠地學會 `tek` 與 `corp` 字尾，不足以推翻也適合人名的上下文。

CRF 保證合法輸出，也稍有幫助。只用詞的標註器在一個種子中產生 30 個非法 `I-` 標籤，字元 softmax 標註器在兩個種子中分別產生 4 個與 1 個；具 BIO 限制的 CRF 依構造一個也沒有。它的平均 F1 為 0.929，對比 0.913，而且每個種子都至少與 softmax 標註器一樣好。在 100 句訓練資料下差距不大；這裡支持多加一層的，是改善的一致性與保證區段格式正確，而不是差距大小。

種子同樣重要。只用詞的標註器在不同種子間介於 0.771 到 0.893。產生 30 個非法轉移的種子，也是表現最差的那一個。與 NLP 05 一樣，單次執行是不可靠的摘要。

這個實驗沒有顯示什麼。語料是合成、極小且模板化的；測試句重複使用訓練模板；標籤集也很小。真實基準包含歧義上下文、巢狀與不連續提及，以及標註分歧。這個實驗把三種機制（未知詞處理、字元證據與輸出限制）分離出來，讓每個效果都能被看見與量測；它並未估計這些效果在 CoNLL-2003 或任何其他資料集上的大小。

轉移學到了什麼。圖 4 顯示一個種子學到的轉移分數。在允許的區域內，CRF 學到了語料的結構：`B-LOC → I-LOC` 與 `B-DEV → I-DEV` 明顯為正，而 `I-ORG → I-ORG` 與 `I-LOC → I-LOC` 明顯為負，因為這個語料中沒有任何名稱超過兩個詞元。學到的轉移表也是需要檢查的建模假設：在每個名稱最多兩個詞元的領域訓練後，它會在其他地方懲罰三詞元的名稱。

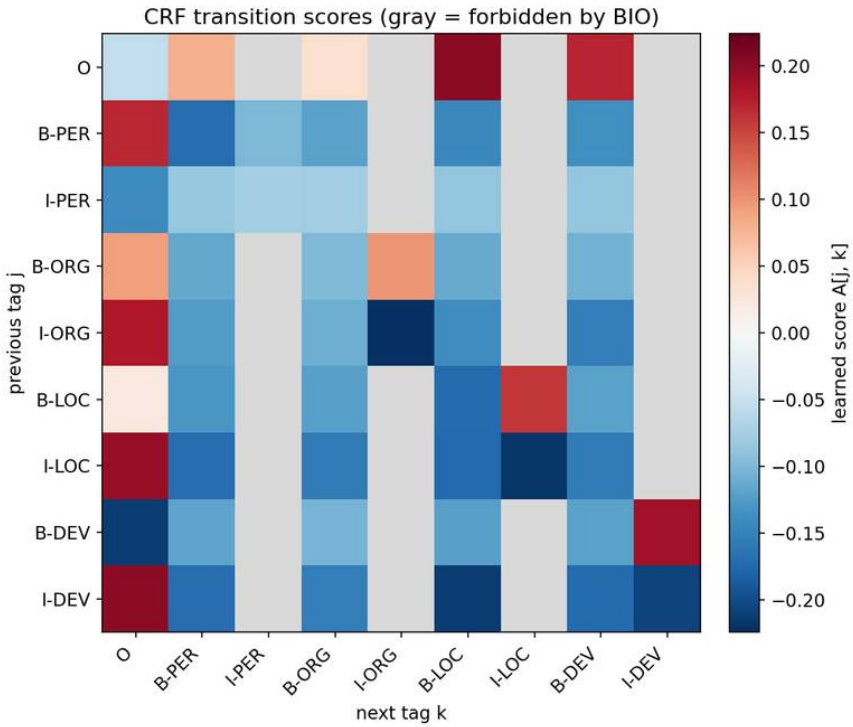


圖 *4* — *CRF* 學到的轉移分數（種子 *0*）。灰色格是限制遮罩禁止的 *BIO* 轉移；其餘分數由 *100* 句訓練資料學得。

現代觀點

直觀說法。當代擷取系統把預訓練表示與明確任務定義結合。通用生成提升彈性，但區段忠實度與支持性仍是另外的問題。

Li 等人的 NER 綜述依輸入表示、上下文編碼器與標籤解碼器整理早期深度系統 [7]。這個分類仍有用：換編碼器不會決定輸出是平面序列還是重疊區段。Xu 等人的 2024 生成式擷取綜述加入任務軸——實體、關係、事件與統一擷取——以及提示、例子、監督微調與受限生成等調適軸 [8]。應把這些圖譜當成設計維度，而不是普遍優越的架構排名。

從 BiLSTM-CRF 到預訓練編碼器。依 Li 等人的分類，本章實驗在保持 BiLSTM 上下文編碼器不變的情況下，改變輸入表示（只用詞或詞加字元）與標籤解碼器（softmax 或 CRF）[7]。2016 到 2018 年間，帶字元特徵的 BiLSTM-CRF [14]、[15] 成為標準做法，字元語言模型的上下文字串嵌入又進一步延伸了它 [16]。之後，預訓練 Transformer 編碼器在多數系統中取代 BiLSTM 成為上下文編碼器；其上的線性或 CRF 標籤解碼器，就是NLP 10 與NLP 11 討論的詞元分類輸出頭。本章的解碼器問題（合法轉移、區段邊界、精確評估）原封不動地延續下去。

以大型語言模型做開放式實體辨識。 UniversalNER（ICLR 2024）透過針對性的指令微調，把大型語言模型蒸餾成小得多的開放領域 NER 模型，並在由 9 個領域、43 個 NER 資料集組成的基準上評估 [22]。它與 GLiNER [9] 一起代表一種轉變：從固定的標籤集合，走向在推論時才提供實體類型。

GLiNER 是 2024 年以雙向 Transformer 做標籤條件式區段擷取的原始研究 [9]。它展示了改變所需實體類別，不一定要自迴歸生成一長串紀錄。結構化區段輸出可以直接對齊原文。應用比較要納入支援的類型定義、巢狀區段行為、延遲與錯誤覆核，不能只看平均準確率。

生成式擷取器則輸出所需紀錄的文字序列化。強制結構合法可避免缺少鍵值或語法損壞，但無法保證名稱、關係或證據區間正確。實務驗證層會檢查引文是否存在、位移是否匹配、類別是否允許，以及實體身分是否一致。這些檢查可排除部分錯誤，解讀錯誤仍需要評估或覆核。

開放問題包含持續改變的類別集合、專業語言、巢狀實體、跨文件身分與領域轉移下的可靠不確定性。多語擷取還面對不同名稱邊界、文字系統與稱謂慣例。語言特定指南是任務定義的一部分，翻譯類別名稱並不足夠。按來源與語言評估文件，同時保留足夠例數，才能解釋各項比較。

新專案應先建立確定性或小型監督基準，凍結標註指南，檢查錯誤類型，再選更大的模型。保留測試文件時，防止重複模板與近乎相同的報告跨越切分。比較完整且受支持的紀錄，不要只檢查是否能剖析輸出格式。這樣建立的擷取服務，才有可量測且能回溯證據的失敗模式。

新標籤同時測試表示與解讀

標籤條件式區段擷取 [9] 與生成式擷取 [8]，改變應用提供類別清單的方式。彈性帶來分析問題：模型辨認的是預定類別，還是只對熟悉標籤名稱反應？可以比較短標籤、精確描述，以及刻意相似的競爭描述。例如區分設備製造商與設備操作單位：兩者都可能是組織提及，但事件角色不同。

應分別判斷實體偵測、指定類型與關係配置。組織區段定位正確，不代表它製造了設備；生成紀錄若填滿每個要求欄位，也可能在來源省略參與者時編造一位。可以比較明確未知答案與強制填入。這個比較應揭露哪種輸出介面保留缺少資訊，以及哪些訓練範例能區分接近的結構規格。類別涵蓋擴大，只有在語意邊界仍清楚時才有用。

重點整理

- 序列標註替每個詞元指定一個標籤，但標籤彼此影響；結構化解碼器在既定分解下選出最佳完整序列。
- HMM 的 Viterbi 解碼在一階分解下是精確的；線性鏈 CRF 保留相同遞迴，但以條件方式替標籤評分，因此可由神經編碼器提供分數。
- 實體辨識從結構描述與位移開始；BIO 標籤編碼平面區段，巢狀提及需要區段表示。
- 評估精確區段（邊界與類型），並把錯誤拆成漏掉、多餘、邊界與類型錯誤，而不是回報詞元準確率。
- BiLSTM 標註器雙向讀取整個句子；遮罩與經過訓練的未知詞編號是不可少的細節。
- 以對數概似（前向演算法）訓練、以 Viterbi 解碼的 CRF 層讓標籤彼此一致；BIO 限制讓非法序列不可能出現。
- 字元特徵能從拼字找回未見名稱，卻無法解決拼字沒有編碼的歧義。
- 資訊擷取在實體區段之上建立關係與事件；每一層都有自己的紀錄契約與失敗模式。

練習

1. **序列決策**。改變 HMM 的一列轉移機率，同時保持正規化。找出「fish fish」最佳路徑變成兩個名詞的條件。

   提示

   比較完整路徑乘積。以名詞起始的兩個候選，共用起始因子與第一次發射。

2. 在 CPU 擷取範例加入名稱表沒有的新人物拼法，新增正確區段，回報精確召回率的變化。

   提示

   保持原名稱表固定。漏掉的提及是基準範圍的有用證據，不是把測試詞彙補進訓練的理由。

3. 為重疊的組織與部門名稱設計結構描述，解釋為何單一平面 BIO 序列無法表達所有允許標註。

   提示

   先畫字元區間，再決定巢狀名稱是否算獨立實體，以及部門是否連結母組織。

4. 在 PyTorch 實驗中移除 CRF 的 BIO 限制遮罩（把 `ban` 與 `ban_start` 設為零）並重新訓練。計算三個種子的非法 `I-` 標籤數，再檢查 `O` 後接 `I-ORG` 的學習轉移分數。

   提示

   沒有遮罩時，非法轉移只會被 100 句訓練資料教到的程度所抑制。把學到的分數與轉入 `I-ORG` 的合法轉移分數比較。

5. 加入只用字元的消融實驗：拿掉詞嵌入，只保留字元 BiLSTM。接著在產生器中替人名加上明顯字尾（例如 `-ko`），重新訓練所有模型。哪種錯誤類型改變了？為什麼？

   提示

   使用程式印出的錯誤分類。新的拼字線索應該只會把帶有該線索的名稱，從類型錯誤欄位中移出。

6. 寫一個部分匹配評估器：預測區段與同類型正確區段重疊時就給分。把它與只用詞標註器的精確 F1 並列，並解釋兩者為何可能以不同順序排列模型。

   提示

   精確定義匹配規則（例如依最大重疊做一對一匹配），並以它自己的名稱回報。只用詞標註器的邊界錯誤，正是兩種分數分歧之處。

參考文獻

1. Susan McRoy, *Principles of Natural Language Processing*, open textbook, 2021; Chapters 3–5 and 8. Open textbook
2. Masato Hagiwara, *Real-World Natural Language Processing: Practical Applications with Deep Learning*, Manning, 2021; Chapter 5, "Sequential labeling and language modeling." Chapter
3. L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," *Proceedings of the IEEE*, 77(2), 1989. Paper
4. M.-C. de Marneffe, C. D. Manning, J. Nivre and D. Zeman, "Universal Dependencies," *Computational Linguistics*, 47(2), 2021. Paper
5. J. Lafferty, A. McCallum and F. Pereira, "Conditional random fields: Probabilistic models for segmenting and labeling sequence data," ICML, 2001. Paper
6. Erik F. Tjong Kim Sang and Fien De Meulder, "Introduction to the CoNLL-2003 Shared Task: Language-Independent Named Entity Recognition," CoNLL, 2003. Paper
7. Jing Li, Aixin Sun, Jianglei Han and Chenliang Li, "A Survey on Deep Learning for Named Entity Recognition," IEEE Transactions on Knowledge and Data Engineering 34(1), 2022. Author-hosted paper
8. Derong Xu, Wei Chen, Wenjun Peng, Chao Zhang, Tong Xu, Xiangyu Zhao, Xian Wu, Yefeng Zheng, Yang Wang and Enhong Chen, "Large language models for generative information extraction: a survey," Frontiers of Computer Science 18(6), 2024. Survey
9. Urchade Zaratiana, Nadi Tomeh, Pierre Holat and Thierry Charnois, "GLiNER: Generalist Model for Named Entity Recognition using Bidirectional Transformer," NAACL, 2024. Paper
10. Lance Ramshaw and Mitch Marcus, "Text Chunking using Transformation-Based Learning," Third Workshop on Very Large Corpora, 1995. Paper
11. Mitchell P. Marcus, Beatrice Santorini and Mary Ann Marcinkiewicz, "Building a Large Annotated Corpus of English: The Penn Treebank," *Computational Linguistics* 19(2), 1993. Paper
12. Ronan Collobert, Jason Weston, Léon Bottou, Michael Karlen, Koray Kavukcuoglu and Pavel Kuksa, "Natural Language Processing (Almost) from Scratch," *Journal of Machine Learning Research* 12, 2011. Paper
13. Zhiheng Huang, Wei Xu and Kai Yu, "Bidirectional LSTM-CRF Models for Sequence Tagging," arXiv:1508.01991, 2015. arXiv
14. Guillaume Lample, Miguel Ballesteros, Sandeep Subramanian, Kazuya Kawakami and Chris Dyer, "Neural Architectures for Named Entity Recognition," NAACL-HLT, 2016. Paper
15. Xuezhe Ma and Eduard Hovy, "End-to-end Sequence Labeling via Bi-directional LSTM-CNNs-CRF," ACL, 2016. Paper
16. Alan Akbik, Duncan Blythe and Roland Vollgraf, "Contextual String Embeddings for Sequence Labeling," COLING, 2018. Paper
17. Sameer Pradhan, Alessandro Moschitti, Nianwen Xue, Hwee Tou Ng, Anders Björkelund, Olga Uryupina, Yuchen Zhang and Zhi Zhong, "Towards Robust Linguistic Analysis using OntoNotes," CoNLL, 2013. Paper
18. Leon Derczynski, Eric Nichols, Marieke van Erp and Nut Limsopatham, "Results of the WNUT2017 Shared Task on Novel and Emerging Entity Recognition," Workshop on Noisy User-generated Text, 2017. Paper
19. Christopher Walker, Stephanie Strassel, Julie Medero and Kazuaki Maeda, *ACE 2005 Multilin-*

gual Training Corpus, LDC2006T06, Linguistic Data Consortium, 2006. Catalog
20. Yuhao Zhang, Victor Zhong, Danqi Chen, Gabor Angeli and Christopher D. Manning, "Position-aware Attention and Supervised Data Improve Slot Filling," EMNLP, 2017. Paper
21. Yuan Yao, Deming Ye, Peng Li, Xu Han, Yankai Lin, Zhenghao Liu, Zhiyuan Liu, Lixin Huang, Jie Zhou and Maosong Sun, "DocRED: A Large-Scale Document-Level Relation Extraction Dataset," ACL, 2019. Paper
22. Wenxuan Zhou, Sheng Zhang, Yu Gu, Muhao Chen and Hoifung Poon, "UniversalNER: Targeted Distillation from Large Language Models for Open Named Entity Recognition," ICLR, 2024. arXiv
23. Mike Schuster and Kuldip K. Paliwal, "Bidirectional Recurrent Neural Networks," *IEEE Transactions on Signal Processing* 45(11), 1997. DOI

NLP 07・注意力機制與 Transformer

從編碼器—解碼器瓶頸出發，介紹 Bahdanau 與 Luong 注意力、縮放內積與多頭注意力、遮罩、位置，以及 Transformer 編碼器與解碼器區塊，並從零實作注意力與編碼器區塊。

網站版本: https://blog.shyandram.dev/zh-tw/learn/nlp/nlp-07-attention-transformers/

你將學到

- 為什麼把整個來源句壓進一個向量會限制編碼器—解碼器，以及注意力如何移除這個瓶頸。
- 加法式（Bahdanau）與乘法式（Luong）注意力的方程式，以及它們如何推廣成查詢、鍵與值。
- 縮放內積注意力、多頭注意力，以及自注意力（self-attention）與交叉注意力（cross-attention）的差別。
- 填補遮罩與因果遮罩如何規定每個位置可以讀什麼，以及如何測試它們確實有效。
- 為什麼注意力需要位置資訊，以及提供位置的主要方法。
- Transformer 編碼器與解碼器區塊如何組成、成本多少，以及哪些變體針對長上下文。

先看全貌

循環神經網路（NLP 05）一次讀一個詞元，並帶著一份持續更新的摘要。不論句子有五個詞還是五十個詞，這份摘要的大小都固定。注意力改變了讀取方式：不再只依賴一份摘要，每個位置都能回頭看所有已儲存的表示，並決定哪些對目前要做的預測有用。Transformer 把這個想法推到底，編碼器與解碼器都只用注意力和小型逐詞元網路組成，完全不需要循環。

> **用最簡單的話說**
>
> 想像你要翻譯一個長句。一種做法是讀一遍、闔上書，再憑記憶寫出譯文。另一種做法是讓書保持打開，每寫一個詞，就回頭看你正在翻譯的那一段。注意力就是第二種做法；「回頭看」是一組權重，說明每個詞要看多少。遮罩則是規定你可以看頁面哪些部分的規則：在某些遊戲裡，你不能偷看還沒寫出的詞。

本章由三個決定組織，值得分開看。**評分規則**決定一個位置與另一個位置有多匹配。**可見性規則**以遮罩表示，決定哪些位置根本可以被讀取。**位置規則**決定詞序如何進入一個原本會忽略詞序的計算。改進其中一項，修不好另一項的錯誤：更好的評分函數，無法修補洩漏未來詞元的遮罩。

Hagiwara 第 8 章從實務角度介紹注意力與 Transformer [1]。本章補上推導、對照原始論文，並用小型 CPU 範例實作每個機制。ML 04 從影像角度處理同一套機制，並提供完整的位置編碼數學；NLP 08 則用它建立翻譯模型。

1. 編碼器—解碼器的瓶頸

直觀說法。序列到序列模型有一個讀取來源的編碼器，以及一個寫出目標的解碼器。如果兩者之間唯一的橋樑是一個向量，解碼器需要的每個細節都必須擠過這個向量。

Sutskever、Vinyals 與 Le [2] 以及 Cho 等人 [3] 的循環編碼器—解碼器，以編碼器 RNN 讀取來源序列 $x_1, \dots, x_T$，再把最後狀態交給解碼器：

$$h_j = f_{\text{enc}}(h_{j-1}, x_j), \qquad c = h_T, \qquad s_u = f_{\text{dec}}(s_{u-1}, y_{u-1}, c), \qquad p(y_u \mid y_{<u}, \mathbf{x}) = g(s_u).$$

其中 $h_j \in \mathbb{R}^d$ 是讀完來源詞元 x_j 後的編碼器狀態；T 是來源長度；c 是唯一的上下文向量；s_u 是目標步驟 u 的解碼器狀態；y_{u-1} 是前一個目標詞元；$y_{<u}$ 是目標前綴；f_{enc} 與 f_{dec} 是 LSTM 或 GRU 單元之類的循環更新函數；g 是輸出層，後接目標詞彙表上的 softmax。ML 03A 推導這些單元並訓練這種模型。

不論輸入是什麼，向量 c 的維度都是 d。十個詞的句子和六十個詞的段落，都必須在產生第一個目標詞之前壓進它。Bahdanau、Cho 與 Bengio 指出這個固定長度向量很可能是基本編碼器—解碼器的瓶頸，並提出讓解碼器改為在來源中搜尋相關部分 [4]。Sutskever 等人也回報，把來源句反轉有助於他們的 LSTM 模型，這與同一個診斷一致：縮短最早來源詞與最早目標詞之間的路徑，讓它們更容易被利用 [2]。

瓶頸關乎資訊流動，不只是容量。即使 c 很大，也必須在解碼器知道步驟 u 需要哪個來源細節之前就算好。注意力把這個選擇延後：編碼器為每個來源詞元保留一個向量，解碼器在每一步選擇不同的組合。

2. 加法式注意力（Bahdanau）

直觀說法。在每個目標步驟，拿解碼器目前的需求去替每個來源位置評分，把分數轉成總和為一的權重，再對編碼器狀態做加權平均。

Bahdanau 等人 [4] 保留全部編碼器狀態 $h_1, \dots, h_T$（來自雙向編碼器），並對目標步驟 u 計算：

$$e_{u,j} = v_a^\top \tanh\!\left(W_a s_{u-1} + U_a h_j\right), \qquad \alpha_{u,j} = \frac{\exp(e_{u,j})}{\sum_{k=1}^{T} \exp(e_{u,k})}, \qquad c_u = \sum_{j=1}^{T} \alpha_{u,j}\, h_j.$$

其中 s_{u-1} 是前一個解碼器狀態；h_j 是來源位置 j 的編碼器狀態；W_a、U_a 與 v_a 是學得的矩陣與向量；$e_{u,j}$ 是**對齊分數**；$\alpha_{u,j}$ 是**注意力權重**，非負且對 j 總和為一；c_u 是每一步各自的**上下文向量**，解碼器把它和自身狀態一起用來預測 y_u。記號與ML 03A 一致。由於分數先讓兩個向量通過一個小型前饋層再比較，這種形式稱為**加法式注意力**。

對 NLP 而言有兩個重要後果。第一，解碼器現在有通往每個來源詞元的直接路徑，第一個來源詞的資訊不必撐過多次循環更新。第二，權重 $\alpha_{u,j}$ 形成目標與來源位置之間的軟對齊，可以畫出來檢查。這種檢查是診斷，不是保證：如後文所述，高權重只顯示計算從哪裡讀取，不說明最終預測為何正確。

3. 乘法式注意力（Luong）

直觀說法。把小型評分網路換成內積，必要時中間夾一個學得矩陣。這更省計算，也讓注意力變成矩陣乘法。

Luong、Pham 與 Manning [5] 為一個讀取所有來源狀態的**全域**注意力模型比較了幾種評分函數。他們用目前（而非前一個）解碼器狀態 h_u 與每個來源狀態 $\bar{h}_j$ 評分：

$$\text{score}(h_u, \bar{h}_j) = \begin{cases} h_u^\top \bar{h}_j & \text{dot} \\ h_u^\top W_a \bar{h}_j & \text{general} \end{cases} \qquad a_u(j) = \frac{\exp \text{score}(h_u, \bar{h}_j)}{\sum_{j'} \exp \text{score}(h_u, \bar{h}_{j'})}.$$

其中 h_u 是目標步驟 u 的頂層解碼器狀態；$\bar{h}_j$ 是來源位置 j 的編碼器狀態；W_a 是學得矩陣；$a_u(j)$ 是注意力權重。第三種選項 **concat** 像加法式注意力一樣，用前饋層對串接向量 $[h_u; \bar{h}_j]$ 評分。上下文向量 $c_u = \sum_j a_u(j) \bar{h}_j$ 與解碼器狀態結合成**注意力隱藏狀態**，再送進輸出 softmax：

$$\tilde{h}_u = \tanh(W_c [c_u; h_u]), \qquad p(y_u \mid y_{<u}, \mathbf{x}) = \text{softmax}(W_s \tilde{h}_u).$$

其中 $[c_u; h_u]$ 是向量串接，W_c、W_s 是學得矩陣。Luong 等人也提出**局部**注意力，先預測一個對齊的來源位置，只讀取它附近的視窗；以及**輸入饋送**（input feeding），把 $\tilde{h}_u$ 送進下一個解碼步驟，讓模型知道自己先前的對齊決定 [5]。

「general」評分是通往 Transformer 的橋樑。令 $q_u = h_u$、$k_j = W_a^\top \bar{h}_j$，分數就只是 $q_u^\top k_j$。一旦每個分數都是**查詢**與**鍵**的內積，整張分數表就是一次矩陣乘法，正是現代硬體最擅長的運算。

4. 查詢、鍵與值：從對齊到自注意力

直觀說法。把解碼器狀態推廣成**查詢**（這個位置在找什麼），把來源狀態推廣成**鍵**（每個位置提供什麼來比對）與**值**（被讀到時傳出什麼）。如此注意力可在任意兩組向量之間運作，也包括序列和它自己。

在加法式與乘法式注意力中，同一個編碼器狀態同時用來比對和加權平均。Transformer 以三個學得投影分開這些角色 [6]。若查詢來自解碼器、鍵和值來自編碼器，這個運算就是**交叉注意力**，是 Bahdanau 與 Luong 注意力的直接後代。若查詢、鍵和值都來自同一個序列，就是**自注意力**：每個詞元讀取同一句中的其他詞元，建立自己的新表示。

自注意力產生**上下文表示**。在「the crane lifts steel」和「the crane eats fish」中，`crane` 的嵌入查表結果相同；但經過自注意力後，兩次出現讀到不同鄰居，輸出向量就能不同。這正是NLP 10發展成 BERT 的表示概念。重用這類表示必須配對分詞器與詞彙表：嵌入列只在該模型詞元 ID 的定義下有意義。未調適權重就換分詞器，可能讓形式合法的數字代表不相關符號。

5. 縮放內積注意力

直觀說法。每個位置詢問其他哪些位置含有有用資訊。注意力把比較轉成權重，再組合相應的值向量。

對單一注意力頭：

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V, \qquad A = \mathrm{softmax}_{\mathrm{row}}\left(\frac{QK^\top}{\sqrt{d_k}} + B\right), \quad O = AV.$$

其中 $H \in \mathbb{R}^{T\times d}$ 包含輸入表示；$W_Q, W_K \in \mathbb{R}^{d\times d_k}$ 是學得的查詢與鍵投影；$W_V \in \mathbb{R}^{d\times d_v}$ 是值投影；Q, K, V 是投影矩陣；d_k, d_v 是鍵和值的寬度；$B \in \mathbb{R}^{T\times T}$ 是加法可見性遮罩，元素為 0（允許）或 $-\infty$（阻擋）；A 包含按列正規化的注意力權重；$O \in \mathbb{R}^{T\times d_v}$ 是輸出。每個查詢列分別套用 softmax。這是 Transformer 論文介紹的機制 [6]。

在常見初始化假設下，縮放因子避免內積只因鍵向量維度增加而變大：若查詢與鍵的分量彼此獨立、平均為零、變異數為一，$q^\top k$ 的變異數就是 d_k；除以 $\sqrt{d_k}$ 讓變異數回到一，softmax 才不會飽和（ML 04 有完整推導）。它不會把注意力權重變成校準後的語意重要性。高權重表示經過學得投影後，在這次計算有較大貢獻；不會證明該詞元造成最終答案，也不會證明它支持生成的陳述。

程式碼：在玩具句子上從零實作注意力

為了不經訓練就看清方程式在做什麼，我們手動建立向量。「the pump stopped because it overheated」的每個詞元得到一個四特徵向量（裝置名詞、代名詞、事件動詞、功能詞），句子再補上一個 <pad>，就像在批次中一樣。投影的設計讓代名詞與動詞**尋找**裝置、名詞尋找事件，代名詞也微弱地**提供**裝置資訊。這裡沒有任何學習；重點是觀察評分、遮罩與平均如何運作。

```
import numpy as np

tokens = ["the", "pump", "stopped", "because", "it", "overheated", "<pad>"]
# Hand-built features: [device noun, pronoun, event verb, function word]
X = np.array([[0, 0, 0, 1], [1, 0, 0, 0], [0, 0, 1, 0], [0, 0, 0, 1],
              [0, 1, 0, 0], [0, 0, 1, 0], [0, 0, 0, 0]], dtype=float)
# Query space: [looking for a device, looking for an event]
W_Q = 3.0 * np.array([[0, 1], [1, 0], [1, 0], [0, 0]])   # nouns seek events; pronouns,
 ↪  verbs seek devices
W_K = np.array([[1, 0], [0.5, 0], [0, 1], [0, 0]])       # devices offer "device"; a pronoun
 ↪  weakly so
W_V = np.eye(4)                                          # pass the features through
 ↪  unchanged

def attention(X, mask):
    Q, K, V = X @ W_Q, X @ W_K, X @ W_V
    scores = Q @ K.T / np.sqrt(K.shape[1])
    scores = np.where(mask, scores, -np.inf)              # blocked pairs get zero weight
    A = np.exp(scores - scores.max(axis=1, keepdims=True))
    A /= A.sum(axis=1, keepdims=True)
    return A @ V, A

T = len(tokens)
real = np.array([t != "<pad>" for t in tokens])
padding_mask = np.broadcast_to(real, (T, T))              # every query may read real keys
causal_mask = padding_mask & np.tri(T, dtype=bool)        # ...and only earlier ones
out, A = attention(X, padding_mask)
_, A_causal = attention(X, causal_mask)
```

```
it, pump = tokens.index("it"), tokens.index("pump")
print("weights of 'it':", " ".join(f"{t}={w:.2f}" for t, w in zip(tokens, A[it])))
print("output of 'it' (device, pronoun, event, function):", out[it].round(2))
print("'pump' reads 'stopped': bidirectional", A[pump, 2].round(2),
      "| causal", A_causal[pump, 2].round(2))
X2 = X.copy(); X2[-1] = [5, 5, 5, 5]                       # garbage in the padding slot
out2, _ = attention(X2, padding_mask)
print("padding changes real outputs:", not np.allclose(out[real], out2[real]))
assert np.allclose(A.sum(axis=1), 1) and A[:, -1].max() == 0
```

輸出：

```
weights of 'it': the=0.07 pump=0.55 stopped=0.07 because=0.07 it=0.19 overheated=0.07
 ↪  <pad>=0.00
output of 'it' (device, pronoun, event, function): [0.55 0.19 0.13 0.13]
'pump' reads 'stopped': bidirectional 0.4 | causal 0.0
padding changes real outputs: False
```

代名詞的新向量大多是它從 pump 借來的裝置特徵：經過一步注意力，it 已帶有可能先行詞的資訊。填補欄的權重恰為零，因此即使填補位置存了垃圾，也改變不了真實詞元的輸出。在因果遮罩下，pump 不能再讀較晚的動詞 stopped。真實模型會學習投影；這裡是手動寫出，而且範例刻意只有一個裝置，所以不能說明訓練好的模型如何解決較難的指涉（那是共指問題，見NLP 12）。

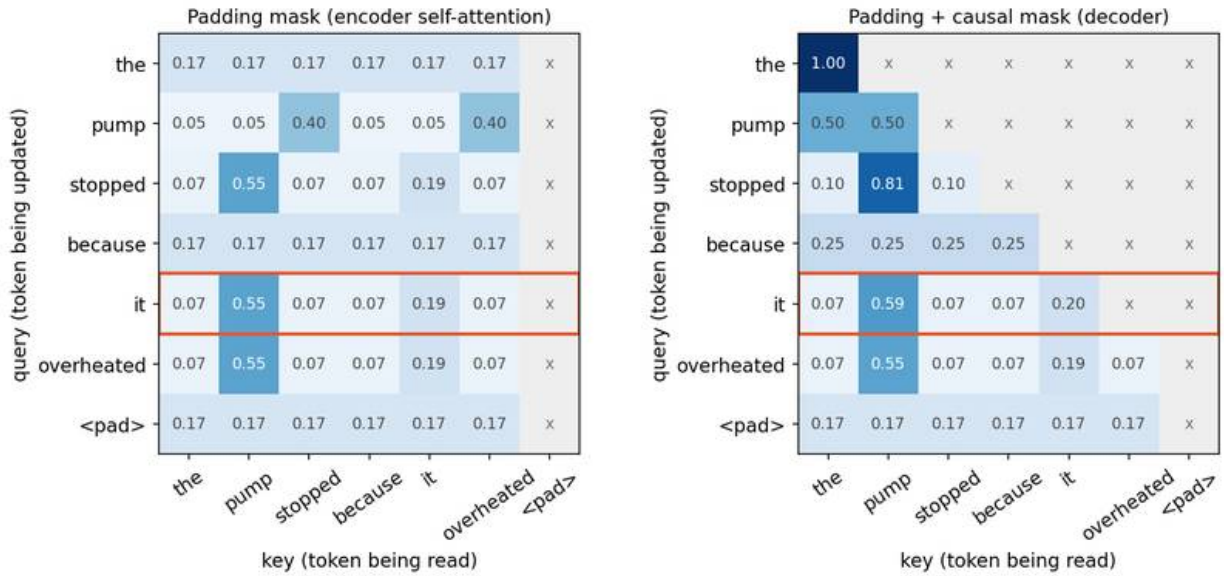


圖 *1* — 手動建立的玩具範例。左：只有填補遮罩，如編碼器。右：填補加因果遮罩，如解碼器。標 *x* 的灰格被阻擋；橘色框是代名詞那一列。權重來自手動選定的投影，不是訓練好的模型。

請注意圖中的 <pad> **列**：填補查詢仍會產生一個（無意義的）輸出，因為它的鍵並未全部被阻擋。這是安全的慣例。若某個查詢列的所有鍵都被阻擋，softmax 會變成零除以零而得到 NaN，並可能擴散到後續層。遮罩應作用在填補**鍵**上，損失則忽略填補**輸出**。

6. 多頭注意力

直觀說法。平行執行幾個較小的注意力運算，每個有自己的投影，讓不同的頭比對不同關係。再把輸出串接起來，用另一個投影混合。

有 h 個頭時，第 r 個頭有自己的投影 $W_Q^{(r)}, W_K^{(r)}, W_V^{(r)} \in \mathbb{R}^{d\times d/h}$：

$$\text{head}_r = \text{Attention}\big(HW_Q^{(r)}, HW_K^{(r)}, HW_V^{(r)}\big), \qquad \text{MHA}(H) = [\,\text{head}_1, \dots, \text{head}_h\,]\,W_O.$$

其中 Attention 是第 5 節的縮放內積運算，且 $d_k = d_v = d/h$；方括號把 h 個輸出（各為 $T \times d/h$）串接回寬度 d；$W_O \in \mathbb{R}^{d\times d}$ 是輸出投影。原始 Transformer 使用 $h = 8$ 個頭、模型寬度 $d = 512$ [6]。由於每個頭窄了 h 倍，多頭注意力的成本約等於一個全寬度的頭，四個投影矩陣共有 $4d^2$ 個權重（另加偏差），與ML 04 的計數相同。

一個頭對每個查詢只形成一個加權平均。若動詞同時需要主詞與受詞，單一 softmax 必須把權重分給兩者，兩邊都會變模糊；多個頭則能各自集中在一種關係上。不過不要過度解讀：頭不保證學到人類可讀的角色，有些頭是多餘的。

後來的兩種變體只改變鍵與值的共享方式。**多查詢注意力**（multi-query attention）讓所有查詢頭共用一個鍵頭和一個值頭，縮小每個解碼步驟要讀取的記憶體 [15]。**分組查詢注意力**（grouped-query attention）讓每個鍵／值頭由一組查詢頭共用，是兩者之間的折衷 [16]。兩者主要影響快速自迴歸解碼，見NLP 09。

7. 自注意力與交叉注意力

直觀說法。同一個運算可以連接同一序列內的詞元（自注意力），也可以連接兩個序列（交叉注意力）。唯一的差別是查詢、鍵和值從哪裡來。

機制	查詢來自	鍵與值來自	NLP 中的典型用途
雙向自注意力	該序列	同一序列的所有真實詞元	BERT 等編碼器；翻譯器的來源端
因果自注意力	該序列	同一序列的目前與較早位置	僅解碼器語言模型；翻譯器的目標端
交叉注意力	解碼器	編碼器輸出	編碼器—解碼器翻譯與摘要

對交叉注意力，令 $H_{\text{dec}} \in \mathbb{R}^{T_y\times d}$ 為解碼器狀態，$M \in \mathbb{R}^{T_x\times d}$ 為編碼器輸出（常稱為**記憶**，memory）：

$$Q = H_{\text{dec}}W_Q, \qquad K = MW_K, \qquad V = MW_V, \qquad O = \text{softmax}_{\text{row}}\left(\frac{QK^\top}{\sqrt{d_k}} + B_{\text{src}}\right)V.$$

其中 T_y 與 T_x 是目標與來源長度；分數矩陣形狀為 $T_y \times T_x$，每個目標位置一列；B_{src} 阻擋來源填補；$O \in \mathbb{R}^{T_y\times d_v}$ 是每個目標位置的輸出。這就是加上學得查詢、鍵、值投影的 Bahdanau 與 Luong 注意力。這裡不需要因果遮罩，因為解碼開始前整個來源都已知。

應由任務可取得的資訊決定遮罩，而不是模型名稱。完整句子已知時，分類器可使用雙向編碼器；串流延續則必須排除未來輸出。翻譯可以在來源全部抵達後讀取完整來源，但同步翻譯（來源句還沒說完就開始翻）需要更嚴格的來源可見性規則。

8. 遮罩：填補與因果

直觀說法。遮罩是一張表，說明哪個查詢可以讀哪個鍵。填補遮罩隱藏人為的填充詞元；因果遮罩隱藏未來。

因果自注意力在 $j \le i$ 時令 $B_{ij} = 0$，否則令 $B_{ij} = -\infty$。其中 i 是查詢位置，j 是候選鍵位置。目標已位移，所以目前輸入位置仍是允許的前綴詞元，對角線可見。完整雙向自注意力允許所有真實輸入位置，但仍可阻擋填補位置。填補遮罩則在鍵 j 是填補位置時，對每個查詢 i 令 $B_{ij} = -\infty$。結合多個遮罩時，只要任一遮罩阻擋某一對，該對就被阻擋。

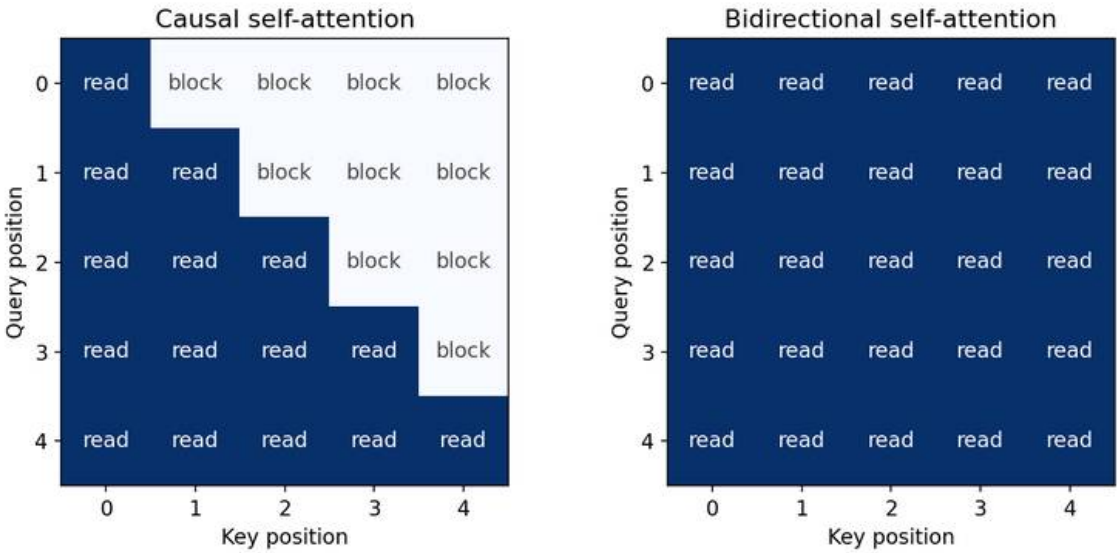


圖 2 — 可見性是任務定義的一部分。列是發出查詢的位置，欄是它們可以讀取的位置。

有兩種遮罩容易混淆。**注意力遮罩**控制表示可以讀什麼；**損失遮罩**控制哪些目標位置貢獻訓練目標。填補兩者都需要：填補鍵不得貢獻上下文，填補目標不得貢獻損失。它們解決不同問題，應分別測試。

檢查注意力並測試資訊流

這個獨立矩陣範例把學得的分數與可見性規則分開。向量與投影皆為合成，因此權重沒有語言學解釋。

```
import numpy as np

rng = np.random.default_rng(0)
```

```
T, d = 5, 8                                   # 5 tokens, 8-dimensional embeddings
X = rng.normal(size=(T, d))                   # token embeddings (+ positions)
Wq, Wk, Wv = (rng.normal(size=(d, d)) / np.sqrt(d) for _ in range(3))

def attention(X, causal):
    Q, K, V = X @ Wq, X @ Wk, X @ Wv
    scores = Q @ K.T / np.sqrt(d)             # (T, T) query-key similarities
    if causal:                                # position t may only look at <= t
        scores = np.where(np.tri(T, dtype=bool), scores, -np.inf)
    w = np.exp(scores - scores.max(axis=1, keepdims=True))
    w /= w.sum(axis=1, keepdims=True)         # softmax over keys
    return w @ V, w

out, w = attention(X, causal=True)
print(np.round(w, 2))

# Changing the LAST token must not change any earlier causal output...
X2 = X.copy(); X2[-1] += 5.0
out2, _ = attention(X2, causal=True)
print("causal: earlier outputs unchanged:", np.allclose(out[:-1], out2[:-1]))
# ...but in a bidirectional encoder every position can see it.
b1, _ = attention(X, causal=False); b2, _ = attention(X2, causal=False)
print("bidirectional: earlier outputs unchanged:", np.allclose(b1[:-1], b2[:-1]))
```

輸出：

```
[[1.   0.   0.   0.   0.  ]
 [0.88 0.12 0.   0.   0.  ]
 [0.42 0.29 0.29 0.   0.  ]
 [0.28 0.24 0.29 0.19 0.  ]
 [0.34 0.06 0.24 0.11 0.26]]
causal: earlier outputs unchanged: True
bidirectional: earlier outputs unchanged: False
```

因果權重是下三角矩陣：第一個詞元只能讀自己，最後一個可以讀全部。數值檢查示範了遮罩的可見性規則。更一般地說，第 t 列只聚合位置不超過 t 的值向量。若查詢、鍵和值也由不含未來資訊的狀態計算而來，未來輸入就無法影響該輸出。這個條件必須在整個網路都成立，也包括任何混合不同位置的前處理。相同遮罩讓所有下一詞元訓練損失可以平行計算，同時讓每個預測只使用自己的前綴。

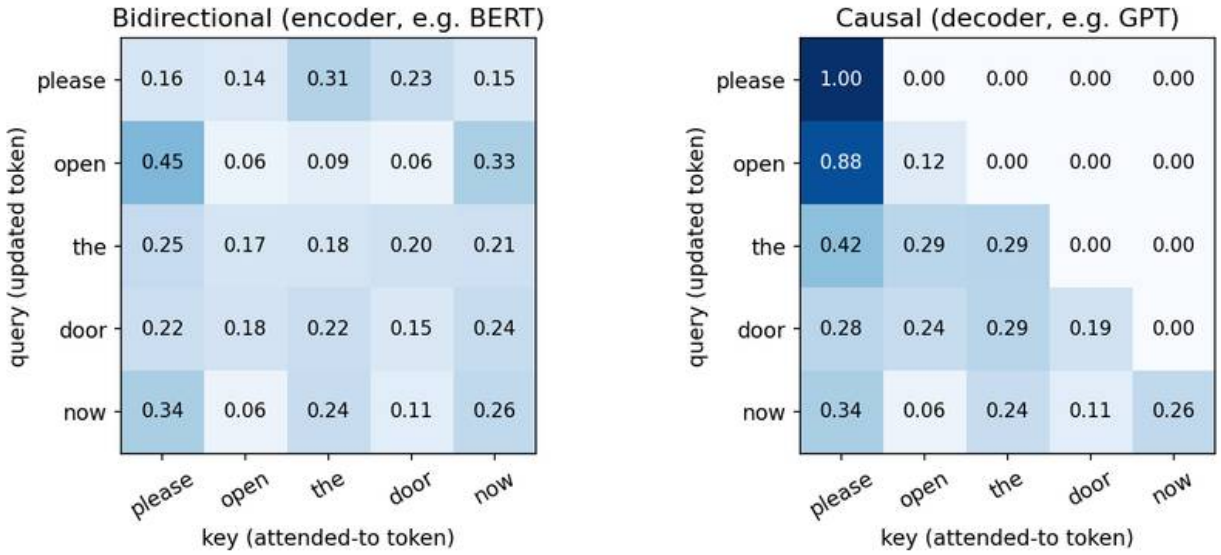


圖 3 — 上方程式碼的注意力權重（投影是隨機的，所以數值本身沒有語言學意義）。決定每個詞元能讀什麼的是遮罩，而不是權重。

在任何真實程式庫中，都值得保留這類擾動測試。低訓練損失揭露不了洩漏的遮罩，因為能看到答案的模型同樣能得到低損失；只有改變未來輸入、再檢查較早輸出的測試，才直接針對可見性契約。

9. 位置：告訴注意力詞元在哪裡

直觀說法。注意力比較詞元表示，位置資訊則告訴模型詞元在哪裡。詞元身分本身不會說明它是主詞還是最後一個詞。

沒有位置資訊時，自注意力具有**排列等變性**（permutation equivariance）：打亂輸入詞元，輸出會以相同方式被打亂，因為公式中沒有任何東西提到順序。「Dog bites man」與「man bites dog」會得到同一組向量。第 10 節的編碼器區塊程式碼會檢查這個性質，並顯示加入位置後性質消失。

最簡單的做法是在第一層之前，把位置向量加到每個詞元嵌入。使用學得的嵌入與位置表：

$$\mathbf{h}_i^{(0)} = E[x_i] + P[i].$$

其中 $\mathbf{h}_i^{(0)} \in \mathbb{R}^d$ 是位置 i 的初始表示；E 是詞元嵌入表；$E[x_i]$ 選取詞元 x_i 的列；P 是位置表；$P[i]$ 選取位置列；d 是表示寬度。學得的位置表也限制支援的最遠位置。超過範圍的序列不是普通長輸入，而是非法索引。服務契約應記錄上下文限制。擴充位置表或改變位置行為屬於模型變更，需要評估，不能直接假定長文件表現維持不變。

原始 Transformer 則使用固定的**正弦**（sinusoidal）編碼 [6]：

$$P[i]_{2m} = \sin\!\left(\frac{i}{10000^{2m/d}}\right), \qquad P[i]_{2m+1} = \cos\!\left(\frac{i}{10000^{2m/d}}\right), \qquad m = 0, \dots, \tfrac{d}{2} - 1.$$

其中 i 是位置，m 是一對維度的索引，每一對以自己的頻率旋轉。這與ML 04 的公式相同（該頁

寫成 $p_{pos,2i}$）；那裡證明了使它有用的兩個性質（固定位移相當於旋轉、內積只依賴位移），並畫出圖樣。原論文也在加上編碼之前，把詞元嵌入乘以 $\sqrt{d}$ [6]；NLP 08 的翻譯模型沿用這個慣例。後來的方法把位置直接放進注意力分數，讓模型看到**相對**位移：

- **相對位置表示**（relative position representations）把依賴截斷位移 $j - i$ 的學得向量加到鍵與值上 [9]。
- **旋轉位置嵌入**（RoPE）把每個查詢與鍵旋轉一個與位置成正比的角度，使分數 $q_i^\top k_j$ 只透過 $i - j$ 依賴位置 [10]。
- **ALiBi** 對每個因果注意力分數加上固定、依頭而異、與距離 $i - j$ 成正比的懲罰，設計目標是讓在短輸入上訓練的模型能在較長輸入上評估 [11]。

這個選擇和上下文長度契約互相影響。學得的絕對位置表有硬性上限；相對方法移除了硬性上限，但不保證在超過訓練長度時仍準確。請在打算服務的長度上評估。

10. Transformer 編碼器與解碼器區塊

直觀說法。一個區塊輪流做兩件事。注意力讓詞元交換資訊；小型前饋網路再各自處理每個詞元。殘差連接與正規化讓深層堆疊可以訓練。解碼器在兩者之間多做一件事：透過交叉注意力讀取編碼器。

原始**編碼器區塊** [6] 把每個子層包在殘差連接中，後接層正規化（**後正規化**，post-norm）：

$$U = \mathrm{LN}(H + \mathrm{MHA}(H)), \qquad H' = \mathrm{LN}(U + \mathrm{FFN}(U)), \qquad \mathrm{FFN}(u) = W_2 \max(0, W_1 u + b_1) + b_2.$$

其中 $H \in \mathbb{R}^{T \times d}$ 是區塊輸入；MHA 是帶填補遮罩的多頭自注意力；LN 是層正規化，把每個詞元的 d 個特徵標準化，再套用學得的縮放與平移 [7]；FFN 對每個位置分別套用，$W_1 \in \mathbb{R}^{d_{\mathrm{ff}} \times d}$、$W_2 \in \mathbb{R}^{d \times d_{\mathrm{ff}}}$，隱藏寬度為 d_{ff}（原始模型在 $d = 512$ 時為 2048）；H' 是區塊輸出。許多後來的模型把正規化移到每個子層**之前**（**前正規化**，pre-norm，即 $H + \mathrm{MHA}(\mathrm{LN}(H))$）。Xiong 等人分析了為何後正規化需要學習率暖身，而前正規化可以不用 [8]。因此，在說明正規化放在哪裡之前，「一個 Transformer」並未完整指定。

解碼器區塊有三個子層，各有自己的殘差連接與正規化：

1. 對目標前綴的**遮罩自注意力**，使用因果遮罩（批次時再加填補遮罩）。
2. **交叉注意力**，查詢來自第 1 步，鍵和值來自編碼器的最終輸出，只遮蔽來源填補。
3. 逐位置的**前饋網路**。

堆疊 N 個編碼器區塊與 N 個解碼器區塊，就得到完整模型（原始模型 $N = 6$ [6]）。每個解碼器區塊都讀同一個編碼器最終輸出。線性層與目標詞彙表上的 softmax 把最後的解碼器狀態轉成 $p(y_t \mid y_{<t}, \mathbf{x})$。去掉編碼器與交叉注意力，就成為**僅解碼器**模型，也就是目前多數大型語言模型的形狀（NLP 09）；去掉解碼器，就成為 BERT 之類的**僅編碼器**模型（NLP 10）。

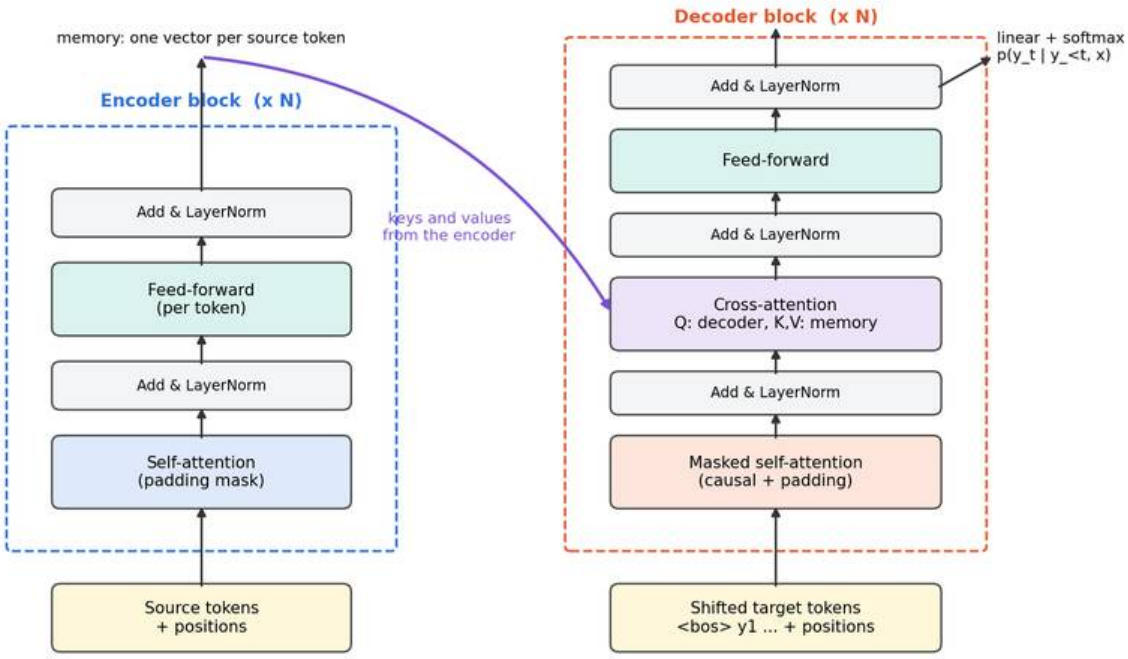


圖 *4* — 原始後正規化 *Transformer* 的編碼器與解碼器區塊。每個子層周圍的殘差連接併入 *Add & LayerNorm* 方塊。紫色箭頭是交叉注意力：每個解碼器區塊都讀編碼器的最終輸出。

程式碼：從零實作微型 Transformer 編碼器區塊

以下程式碼直接依方程式實作多頭自注意力與後正規化編碼器區塊，借用 PyTorch `nn.TransformerEncoderLayer` 隨機初始化的權重 [21]，並檢查兩者輸出相同。接著測試三個性質：填補被忽略、沒有位置時區塊具排列等變性、正弦位置打破這個對稱。為了讓比較精確，dropout 關閉，參考層處於評估模式。

```
import math
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
B, T, d, h, d_ff = 2, 6, 16, 4, 64
ref = nn.TransformerEncoderLayer(d, h, d_ff, dropout=0.0, activation="relu",
                                 batch_first=True, norm_first=False).eval()

def multi_head_attention(x, key_padding, p):
    q, k, v = (x @ p.in_proj_weight.T + p.in_proj_bias).chunk(3, dim=-1)
    split = lambda t: t.view(B, T, h, d // h).transpose(1, 2)        # (B, h, T, d/h)
    q, k, v = split(q), split(k), split(v)
    scores = q @ k.transpose(-2, -1) / math.sqrt(d // h)              # (B, h, T, T)
```

```
    scores = scores.masked_fill(key_padding[:, None, None, :], float("-inf"))
    heads = scores.softmax(dim=-1) @ v                                  # (B, h, T, d/h)
    concat = heads.transpose(1, 2).reshape(B, T, d)                     # heads side by side
    return concat @ p.out_proj.weight.T + p.out_proj.bias

def encoder_block(x, key_padding, layer=ref):
    # Post-norm, as in the original Transformer: x <- LN(x + Sublayer(x))
    a = multi_head_attention(x, key_padding, layer.self_attn)
    x = F.layer_norm(x + a, (d,), layer.norm1.weight, layer.norm1.bias, layer.norm1.eps)
    f = layer.linear2(F.relu(layer.linear1(x)))
    return F.layer_norm(x + f, (d,), layer.norm2.weight, layer.norm2.bias, layer.norm2.eps)

x = torch.randn(B, T, d)
pad = torch.tensor([[False] * 6, [False] * 4 + [True] * 2])     # second sentence: 4 real
 ↪  tokens
with torch.no_grad():
    ours, theirs = encoder_block(x, pad), ref(x, src_key_padding_mask=pad)
    print("matches nn.TransformerEncoderLayer:", torch.allclose(ours[~pad], theirs[~pad],
     ↪  atol=1e-5))
    x2 = x.clone(); x2[1, 4:] = 100.0                                # garbage in padded slots
    print("padding is ignored:", torch.allclose(encoder_block(x2, pad)[~pad], ours[~pad],
     ↪  atol=1e-5))
    perm = torch.tensor([3, 0, 5, 1, 4, 2])
    none = torch.zeros(B, T, dtype=torch.bool)
    y, yp = encoder_block(x, none), encoder_block(x[:, perm], none)
    print("no positions -> permutation-equivariant:", torch.allclose(yp, y[:, perm],
     ↪  atol=1e-5))
    pos = torch.arange(T)[:, None] / 10000 ** (torch.arange(0, d, 2) / d)
    pe = torch.stack([pos.sin(), pos.cos()], dim=-1).flatten(1)     # sin/cos interleaved
    y, yp = encoder_block(x + pe, none), encoder_block(x[:, perm] + pe, none)
    print("with positions -> still equivariant:", torch.allclose(yp, y[:, perm], atol=1e-5))
print("parameters:", sum(p.numel() for p in ref.parameters()))
```

輸出：

```
matches nn.TransformerEncoderLayer: True
padding is ignored: True
no positions -> permutation-equivariant: True
with positions -> still equivariant: False
parameters: 3280
```

比較只使用真實詞元：在評估模式下，PyTorch 可能走融合路徑，不在填補位置計算有意義的值，而這些輸出本來就會被忽略。參數數量驗證了公式：注意力有 $4d^2 + 4d = 1088$ 個權重與偏差，前饋網路有 $2d\,d_{\text{ff}} + d_{\text{ff}} + d = 2128$ 個，兩個層正規化有 $4d = 64$ 個。注意參數分布：在這個區塊中，如同原始 Transformer，前饋網路占了大多數。

在加上位置之前，模型沒有任何位置資訊，第三和第四項檢查正顯示了這一點。真實模型只在第一個區塊的輸入加一次位置向量；後面的區塊都透過殘差流繼承順序資訊。

11. 成本與長上下文變體

直觀說法。每個查詢都和每個鍵比較，所以運算量與分數表會隨序列長度平方成長。對一個句子很便宜，對一本書就很昂貴。

對長度 T、寬度 d 的序列，一個自注意力層計算 $QK^\top$ 與 AV 各約需 T^2d 次乘加，投影另需約 $4Td^2$。循環層約需 Td^2，但需要 T 個**依序**步驟；自注意力層的所有位置則可平行計算，任兩個位置之間的路徑長度為一。Vaswani 等人在比較各類層時整理了這個取捨 [6]。對短句（T 小於 d），自注意力比較便宜；對長文件，T^2 項會占主導。

直接儲存注意力分數會隨序列長度平方成長。專用實作可避免實體化每個中間矩陣，但哪些位置需要互動的建模問題仍在。部署時，應按實際上下文長度量測記憶體與延遲，不能由短句直接外推。

長上下文方法可分成幾個家族。Tay 等人的綜述把它們整理為固定模式、可學模式、低秩與核方法、記憶、循環與降取樣 [17]。代表性例子：

想法	例子	改變了什麼
限制哪些位置對互動	Longformer：滑動局部視窗加少數全域詞元 [12]	固定視窗下成本隨 T 線性成長；遠距詞元透過全域詞元或多層溝通
替換 softmax 核	使用特徵映射的線性注意力 [13]	分數可分解，得到線性成本與因果注意力的循環觀點；注意力函數本身不同
保留精確注意力，減少記憶體搬移	FlashAttention：分塊、IO 感知計算 [14]	結果與標準注意力相同，記憶體流量較少；仍是平方級運算
讓多個頭共用鍵與值	多查詢與分組查詢注意力 [15, 16]	解碼時的鍵／值快取較小
可延伸的位置方法	相對位置、RoPE、ALiBi [9–11]	移除硬性位置上限；在未見長度的準確度仍需量測

這些方法不能互換。FlashAttention 計算的函數與標準注意力相同；Longformer 與線性注意力計算的是不同函數，必須當成不同模型評估。完全不計算成對分數的狀態空間與循環替代方案，見ML 03A 與ML 04。

現代觀點

直觀說法。 2017 年的注意力區塊在當前模型中仍清晰可辨；多數改變在於正規化位置、位置處理、鍵／值共享與高效核心運算，而不是核心方程式。

綜述。 Lin、Wang、Liu 與 Qiu 的 Transformer 變體綜述（*AI Open*，2022）先介紹原始模型，再提出分類法，從架構修改、預訓練與應用三個角度回顧變體 [18]。它很適合用來定位某個變體（「這是改注意力、改位置，還是改區塊？」）。Tay、Dehghani、Bahri 與 Metzler 的 *Efficient Transformers: A Survey*（*ACM Computing Surveys*，2022）聚焦注意力的記憶體與運算，並依機制分類高效模型（即第 11 節的家族）[17]。它的實務訊息是：效率主張取決於序列長度、硬體與實作，因此要在你需要的長度上比較模型。

注意力作為解釋。 Jain 與 Wallace 回報，在他們的模型中，注意力權重常與基於梯度的重要性量測不一致，而且非常不同的注意力分布可以得到相同預測 [19]。Wiegreffe 與 Pinter 則主張，注

意力能否作為解釋取決於如何定義「解釋」，並提出檢驗方法 [20]。綜合來看：注意力圖顯示計算從哪裡讀取，對除錯遮罩與對齊很有用，但它本身不是模型為何做出某個決定的證據。

2023–2026 實務現況。分組查詢注意力 [16] 與 IO 感知的精確注意力核心 [14]，分別處理主導服務成本的兩件事：解碼時的鍵／值記憶體，以及注意力計算中的記憶體流量。RoPE [10] 與 ALiBi [11] 等相對位置方法的設計目標，是移除學得位置表的硬性最大長度。編碼器—解碼器 Transformer 仍很適合有明確來源與目標的任務，例如翻譯（NLP 08）；開放式生成則以僅解碼器模型為主（NLP 09）。

開放問題。可靠地使用極長上下文（較長視窗允許更多輸入，但不保證全部被利用）、超過訓練長度的長度泛化、長文件注意力的成本，以及忠實解讀注意力頭到底計算了什麼。

重點整理

- 單一上下文向量迫使整個來源通過固定大小的瓶頸；注意力讓每個輸出步驟都能讀取每個來源位置。
- Bahdanau 注意力用小型前饋網路評分；Luong 的 dot 與 general 評分把注意力化為矩陣乘法，直接引出查詢、鍵與值。
- 縮放內積注意力是 $\text{softmax}(QK^\top/\sqrt{d_k}+B)V$；多頭注意力以約一個頭的成本執行 h 個較窄的副本。
- 自注意力讓序列與自己混合；交叉注意力讓解碼器讀編碼器。決定每個位置能讀什麼的是遮罩，而不是權重。
- 遮蔽填補鍵、在因果模型中遮蔽未來、讓損失遮罩分開處理，並用擾動檢查測試可見性。
- 注意力忽略順序；學得、正弦、相對、旋轉或 ALiBi 位置提供順序，各自有不同的長度契約。
- 注意力成本隨 T^2 成長；長上下文變體不是改變函數（稀疏、線性），就是更有效率地計算同一個函數（FlashAttention）。

練習

1. 移除第 8 節資訊流範例的因果遮罩。哪一行輸出會改變？為什麼不使用遮罩訓練的語言模型仍可能有很低的訓練損失？

   提示

   改變的最後詞元能被較早位置讀取。低損失可能獎勵生成時根本不可取得的資訊。

2. 在玩具句子程式碼中，於代名詞前加入第二個裝置名詞（「valve」）。`it` 的權重會怎樣？除了這些手動特徵之外，模型還需要什麼才能選對先行詞？

   提示

   特徵相同時，兩個裝置得到相同分數。位置資訊、動詞意義與篇章脈絡必須進入查詢或鍵，才能打破平手。

3. 在因果解碼器中，為長度 3 與 5、填補到 5 的兩個目標序列批次設計注意力與損失遮罩。為每個序列寫出 5×5 的阻擋／允許表，並列出貢獻損失的目標位置。

   提示

   填補鍵不得貢獻上下文，填補目標不得貢獻損失。也要決定填補查詢列如何處理，避免全部被阻擋的 softmax 列。

4. 對 $d = 512$、$h = 8$、$d_{\text{ff}} = 2048$ 的模型，計算一個編碼器區塊與一個解碼器區塊的權重數（忽略偏差與層正規化）。編碼器區塊有多少比例在前饋網路？

   提示

   自注意力與交叉注意力各有 $4d^2$ 個權重；前饋網路有 $2d\,d_{\text{ff}}$ 個。解碼器比編碼器多一個注意力子層。

5. 對序列長度 $T = 128$、1,024 與 16,384，在 $d = 512$ 下比較 $T^2 d$（注意力分數）與 $4Td^2$（投影）。從哪個長度開始分數項占主導？

   提示

   兩項在 $T = 4d$ 時相等。低於這個長度時，投影的成本高於分數計算。

6. 說明 Luong 的「general」評分 $h_u^\top W_a \bar{h}_j$ 為何是縮放內積注意力的特例。查詢、鍵與值投影分別是什麼？

   提示

   把 W_a 吸收進鍵投影。值投影是單位矩陣，縮放因子為 1。

參考文獻

1. Masato Hagiwara, *Real-World Natural Language Processing: Practical Applications with Deep Learning*, Chapter 8, Manning, 2021. Publisher ·Chapter 8
2. I. Sutskever, O. Vinyals and Q. V. Le, “Sequence to Sequence Learning with Neural Networks,” NeurIPS, 2014. Paper
3. K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk and Y. Bengio, “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014. Paper
4. D. Bahdanau, K. Cho and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” ICLR, 2015. Paper
5. M.-T. Luong, H. Pham and C. D. Manning, “Effective Approaches to Attention-based Neural Machine Translation,” EMNLP, 2015. Paper
6. A. Vaswani et al., “Attention Is All You Need,” NeurIPS, 2017. Paper
7. J. L. Ba, J. R. Kiros and G. E. Hinton, “Layer Normalization,” arXiv:1607.06450, 2016. Paper

8. R. Xiong et al., “On Layer Normalization in the Transformer Architecture,” ICML, 2020. Paper
9. P. Shaw, J. Uszkoreit and A. Vaswani, “Self-Attention with Relative Position Representations,” NAACL, 2018. Paper
10. J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing* 568, 2024. DOI
11. O. Press, N. A. Smith and M. Lewis, “Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation,” ICLR, 2022. Paper
12. I. Beltagy, M. E. Peters and A. Cohan, “Longformer: The Long-Document Transformer,” arXiv: 2004.05150, 2020. Paper
13. A. Katharopoulos, A. Vyas, N. Pappas and F. Fleuret, “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention,” ICML, PMLR 119, 2020. Paper
14. T. Dao, D. Y. Fu, S. Ermon, A. Rudra and C. Ré, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” NeurIPS, 2022. Paper
15. N. Shazeer, “Fast Transformer Decoding: One Write-Head is All You Need,” arXiv:1911.02150, 2019. Paper
16. J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón and S. Sanghai, “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints,” EMNLP, 2023. Paper
17. Y. Tay, M. Dehghani, D. Bahri and D. Metzler, “Efficient Transformers: A Survey,” *ACM Computing Surveys* 55(6), 2022. DOI
18. T. Lin, Y. Wang, X. Liu and X. Qiu, “A Survey of Transformers,” *AI Open* 3, 2022. DOI
19. S. Jain and B. C. Wallace, “Attention is not Explanation,” NAACL, 2019. Paper
20. S. Wiegreffe and Y. Pinter, “Attention is not not Explanation,” EMNLP-IJCNLP, 2019. Paper
21. PyTorch, `TransformerEncoderLayer` documentation. Documentation

NLP 08・以 Transformer 建構序列到序列模型（機器翻譯、聊天機器人）

建立並訓練用於翻譯的編碼器—解碼器 Transformer，以貪婪與束搜尋解碼、用從零寫出的 BLEU 評分，再把同一套條件生成觀念帶到對話狀態、澄清與對話評估。

網站版本: https://blog.shyandram.dev/zh-tw/learn/nlp/nlp-08-seq2seq-transformers/

你將學到

- 機器翻譯如何從統計片語表走到神經編碼器—解碼器，以及 Transformer 改變了什麼。
- 條件序列學習：目標位移、教師強制（teacher forcing）、考慮填補的損失，以及為何生成時會出現曝光偏差（exposure bias）。
- 如何在 CPU 上訓練小型編碼器—解碼器 Transformer 翻譯器，並以貪婪搜尋與帶長度正規化的束搜尋（beam search）解碼。
- BLEU 如何運作（從零實作）、chrF 與 COMET 補上什麼，以及為何它們都不能證明意義正確。
- 同一個條件生成觀點如何套用到聊天機器人，以及為何任務對話還需要明確狀態、澄清與工具邊界。
- 如何評估完整對話，而不是單一回覆。

先看全貌

序列到序列（seq2seq）模型讀一個序列、寫出另一個序列：由英文句寫出法文句、由對話歷史寫出回覆、由文件寫出摘要。編碼器把輸入變成每個詞元一個向量；解碼器一次寫一個輸出詞元，透過因果自注意力讀自己的前綴，再透過交叉注意力讀輸入（NLP 07）。同一套機制服務翻譯與對話，但兩種任務的失敗方式不同。譯文可能很流暢卻反轉否定；聊天機器人可能回覆親切，卻查錯裝置。

> **用最簡單的話說**
>
> 想像會議上的口譯員。他先聽完整句話（編碼器），再一個詞一個詞說出譯文，每說一個詞之前，都回頭看一下正在翻譯的那段發言（交叉注意力）。好的口譯員在講者不清楚時，也會問「您是指 Pump A 還是 Pump B？」說得流暢不等於說得對；口譯員的評判標準，是聽者最後是否得到相同的意思。

本章以兩種互補閱讀為錨。Hagiwara 第 6 章說明用於翻譯與對話應用的序列到序列模型，第 8 章介紹注意力與 Transformer [1]。McRoy 第 9 章把翻譯視為保留意義的生成 [2]。本章範例都是原創的本機 CPU 實驗，不需下載模型或資料集就能展示機制。NLP 09 接著討論開放式生成、摘要與服務。

1. 從統計機器翻譯到神經機器翻譯

直觀說法。機器翻譯最初從對齊的句對學習詞與片語的對應，再結合另一個描述流暢目標文字的模型。神經翻譯則以一個端到端訓練的網路取代整條管線。

統計機器翻譯（SMT）把翻譯視為**雜訊通道**（noisy channel）。要把外語句 $\mathbf{f}$ 譯成英文 $\mathbf{e}$，選擇

$$\hat{\mathbf{e}} = \arg\max_{\mathbf{e}} \; p(\mathbf{e})\, p(\mathbf{f} \mid \mathbf{e}).$$

其中 $p(\mathbf{e})$ 是獎勵流暢英文的語言模型；$p(\mathbf{f} \mid \mathbf{e})$ 是獎勵忠實的翻譯模型；$\hat{\mathbf{e}}$ 是選出的譯文。Brown 等人的 IBM 模型從句子對齊的文字估計詞層級翻譯機率與對齊 [3]。Koehn、Och 與 Marcu 的片語式翻譯使用多詞片語對，比單詞更能掌握局部詞序調整與慣用語 [4]。這類系統結合許多分別訓練的元件與特徵。

神經網路先以元件身分加入：Cho 等人用 RNN 編碼器—解碼器在既有 SMT 系統中替片語對評分 [5]。接著 Sutskever、Vinyals 與 Le 訓練 LSTM 編碼器—解碼器直接翻譯 [6]。Bahdanau、Cho 與 Bengio 加入注意力，讓解碼器每一步都能搜尋來源 [7]；Vaswani 等人則完全以注意力取代循環 [8]。位元組對編碼（byte-pair encoding）等子詞切分讓開放詞彙變得實際可行，罕見詞與名稱可以譯成已知片段的序列 [9]；NLP 02 逐步建立 BPE 詞彙表。Stahlberg 的回顧詳細追溯了從計數式 SMT 到神經機器翻譯的轉變 [22]。

兩個近期錨點顯示領域的走向。NLLB 計畫建立了涵蓋 200 種語言的單一大規模多語模型，特別關注低資源語言 [21]。WMT24 一般翻譯任務在專業人工評估下，比較參賽系統與數個大型語言模型及線上服務；其標題概括了結論：LLM 時代已經到來，但機器翻譯尚未解決 [20]。

2. 條件序列學習

直觀說法。翻譯器一次一個詞元，建模「給定來源時目標句」的機率。訓練時給它正確的先前目標詞元；生成時則要它使用自己的輸出。

條件序列模型寫成：

$$p_\theta(\mathbf{y} \mid \mathbf{x}) = \prod_{t=1}^{U} p_\theta(y_t \mid y_{<t}, \mathbf{x}).$$

其中 $\mathbf{x}$ 是來源序列；$\mathbf{y} = (y_1, \dots, y_U)$ 是含結束詞元的目標序列；U 是其長度；$y_{<t}$ 是位置 t 之前的詞元；θ 是學得參數。來源可以完整可見，目標仍保持因果。分解描述分布，不保證取樣輸出獲得來源支持。

訓練通常提供正確的先前目標詞元，稱為教師強制 [6]。批次的詞元損失為：

$$\mathcal{L}(\theta) = -\frac{1}{M} \sum_{b=1}^{B} \sum_{t=1}^{T_b} m_{bt} \log p_\theta(y_{bt} \mid y_{b,<t}, \mathbf{x}_b).$$

其中 B 是例數；T_b 是範例 b 填補後的目標長度；m_{bt} 對真實目標為一、填補為零；$M = \sum_{b,t} m_{bt}$ 是真實詞元數；$\mathbf{x}_b$、y_{bt}、$y_{b,<t}$ 分別是來源、目前目標與參考前綴。使用自然對數。遮罩從損失移除填補，不會自動從計算移除。

仔細檢查位移。解碼器輸入從起始詞元開始，目標從第一個輸出詞元開始；目標含結束符號讓模型學停止。填補與結束必須分開，否則可能獎勵錯誤停止。不同長度句子的批次要記錄按詞元或例子平均，因為它們給長句不同權重。

資料建構往往比小架構調整更重要。翻譯對若默默修正數字，就訓練模型改數字。來源版本、指南與目標來源要一起保留，按文件、對話或事件分組切分，防止近重複跨訓練測試。

教師強制與曝光偏差

教師強制提供正確前綴，即使模型自己會產生另一個前綴。假設參考從 `Pump A` 開始，生成卻選了 `Pump B`。後續詞元仍可能構成完全流暢的 Pump B 陳述。參考前綴損失卻始終在 `Pump A` 後評估那些位置，沒有測試如何從這次生成錯誤恢復。差別在於：給定歷史下的預測，以及模型自己持續建立歷史時的行為。

Ranzato 等人把這種訓練／推論不一致稱為**曝光偏差**：模型訓練時只接觸正確前綴，測試時卻必須讀自己的預測，錯誤可能累積 [10]。兩種回應說明了設計空間。**排程取樣**（scheduled sampling）在訓練中逐步把部分正確解碼器輸入換成模型自己的預測 [11]。**序列層級訓練**直接最佳化整個生成序列的分數（例如 BLEU），而不只是下一詞元概似 [10]。兩者都不能免除評估完整生成輸出的需要；以教師強制做最大概似仍是標準起點，分解本身也是精確的。ML 08 討論影像生成中的同一種不一致。

詞元平均也隱藏不同語意後果。替換 `the` 可能只影響風格；替換 `not`、裝置 ID 或單位，卻能反轉結果。每個詞元貢獻一項對數損失，應用對這些變化的成本可能差很多。如果常見用詞改善，稀少關鍵差異卻惡化，平均損失下降仍可能伴隨更多重要錯誤。多個正確說法也是問題：單一參考提供一種可接受表達，不是完整集合。

3. 用於翻譯的編碼器—解碼器 Transformer

直觀說法。編碼器以雙向自注意力讀完整個來源；解碼器以因果自注意力寫目標，並以交叉注意力讀編碼器。最後的線性層把每個解碼器狀態轉成目標詞元上的分布。

原始 Transformer [8] 由NLP 07 的區塊組成：

1. **輸入**。來源與目標文字切成子詞 [9]（NLP 02）。每個詞元 ID 經嵌入、乘以 $\sqrt{d}$，再加上正弦位置編碼。
2. **編碼器**。 N 個由自注意力與前饋子層組成的區塊，對來源鍵套用填補遮罩，產生**記憶** $M \in \mathbb{R}^{T_x \times d}$，每個來源詞元一個向量。
3. **解碼器**。 N 個區塊，包含遮罩自注意力（因果加目標填補）、讀取 M 的交叉注意力（遮蔽來源填補）與前饋子層。
4. **輸出**。線性層與 softmax 在每個目標位置給出 $p_\theta(y_t \mid y_{<t}, \mathbf{x})$。

訓練時，所有目標位置一次算完，因為因果遮罩讓每個位置只看到自己的前綴。生成時，編碼器只跑一次，解碼器每產生一個輸出詞元跑一次。實務解碼器會快取較早目標位置的鍵與值，而不是重算；NLP 09 分析這個快取。

原論文有兩個值得知道的訓練細節。模型使用**標籤平滑**（label smoothing）：目標分布把大部分機

率放在參考詞元，把一小部分分給詞彙表其餘詞元 [8, 29]。平滑參數為 ε、詞彙表為 V 時，訓練目標為 $q(v) = (1-\varepsilon)\mathbf{1}\{v = y_t\} + \varepsilon/|V|$；其中 $\mathbf{1}\{\cdot\}$ 在條件成立時為一。這抑制過度自信；Vaswani 等人回報它使困惑度變差，但改善了 BLEU [8]。模型也使用學習率暖身，Xiong 等人後來把它與後正規化配置連結起來（NLP 07）。

4. 解碼：貪婪搜尋、束搜尋與長度正規化

直觀說法。訓練好的模型對每個可能的下一詞元給出機率。解碼器必須把這些機率變成一個輸出句。貪婪解碼每次取最好的下一詞元；束搜尋則同時保留幾個部分句子。

候選序列分數為：

$$\ell(\mathbf{y} \mid \mathbf{x}) = \sum_{t=1}^{U} \log p_\theta(y_t \mid y_{<t}, \mathbf{x}).$$

其中 ℓ 是完整序列對數機率；U 包含結束詞元；其他符號同前。對所有序列精確最大化並不可行，因為候選數隨長度指數成長。**貪婪解碼**每一步選機率最高的詞元。寬度為 k 的**束搜尋**保留分數最高的 k 個前綴，把每個前綴延伸所有詞元，再保留最好的 k 個；完成的假說（以結束詞元結尾）另外存放。這些選擇控制搜尋與變化，不會讓無支持內容變可靠。

貪婪解碼可能錯過最佳完整序列：機率稍低的第一個詞元，可能接著一個機率高得多的延續，而束搜尋能把它保留到發現為止。NLP 09 用人工範例說明這點，並討論原始對數機率對未完成前綴提供的上界、搜尋錯誤與模型錯誤的差別，以及基於取樣的解碼。這裡聚焦翻譯多出來的部分：長度正規化，以及在訓練好的翻譯器上實際量測的貪婪與束搜尋比較（第 6 節）。

長度正規化

原始對數機率有內建偏好。每多一個詞元就多一個非正項，所以較長的假說即使是較好的譯文，分數也傾向較低。Wu 等人在 Google 神經機器翻譯系統中描述了這個偏差，並以正規化分數排序完成的假說 [12]：

$$s(\mathbf{y}, \mathbf{x}) = \frac{\ell(\mathbf{y} \mid \mathbf{x})}{\mathrm{lp}(\mathbf{y})}, \qquad \mathrm{lp}(\mathbf{y}) = \frac{(5 + |\mathbf{y}|)^{\alpha}}{(5+1)^{\alpha}}.$$

其中 $|\mathbf{y}|$ 是假說的詞元長度；$\alpha \geq 0$ 控制正規化強度，$\alpha = 0$ 回到原始分數，$\alpha = 1$ 近似每詞元平均；s 是用來排序完成假說的分數。他們的完整評分函數還加上以注意力為基礎的涵蓋懲罰，這裡省略。他們在開發資料上調整 α [12]；應把它當成超參數，而不是常數。

```
candidates = {                          # per-token log-probabilities, end token included
    "stop <eos>": [-0.9, -1.1],
    "stop pump b now <eos>": [-0.9, -0.5, -0.3, -0.4, -0.2],
}
def length_penalty(n, alpha):           # Wu et al. (2016): ((5 + n) / 6) ** alpha
    return ((5 + n) / 6) ** alpha
for alpha in [0.0, 0.6, 1.0]:
    scores = {c: sum(lp) / length_penalty(len(lp), alpha) for c, lp in candidates.items()}
```

```
    best = max(scores, key=scores.get)
    print(f"alpha={alpha}:", {c: round(s, 3) for c, s in scores.items()}, "->", best)
```

輸出：

```
alpha=0.0: {'stop <eos>': -2.0, 'stop pump b now <eos>': -2.3} -> stop <eos>
alpha=0.6: {'stop <eos>': -1.823, 'stop pump b now <eos>': -1.693} -> stop pump b now <eos>
alpha=1.0: {'stop <eos>': -1.714, 'stop pump b now <eos>': -1.38} -> stop pump b now <eos>
```

原始分數偏好被截斷的指令；$\alpha = 0.6$ 與 $\alpha = 1$ 偏好完整指令（若 α 很小，例如 0.2，仍會偏好短的那個）。哪個正確取決於來源，而分數看不到來源。

正規化也改變搜尋本身：使用原始分數時，若某個完成假說勝過所有未完成前綴，就可以安全回傳，因為延伸前綴只會降低原始分數；改用長度正規化分數後，這個論證不再成立，停止規則必須把懲罰考慮進去。

要明訂輸出限制、停止規則及達上限後的行為，截斷翻譯不能當完整結果。解碼參數與模型一起記錄，讓評估可重現。

5. 評估譯文：BLEU、chrF 與 COMET

直觀說法。自動指標把系統輸出與一或多個人工參考譯文比較。它們便宜、可重現，適合比較系統；但一個小詞改動就能反轉意思，分數卻幾乎不變。

BLEU

BLEU 比較候選與參考的 n-gram [13]。對每個階數 $n = 1, \dots, 4$，它計算**修正（截限）精確率**：每個候選 n-gram 最多只計入它在「出現最多次的那個參考」中的次數。在整個測試語料上：

$$p_n = \frac{\sum_{\text{sent}} \sum_{g \in G_n} \min\big(c(g),\, c_{\text{ref}}^{\max}(g)\big)}{\sum_{\text{sent}} \sum_{g \in G_n} c(g)}, \qquad \text{BP} = \begin{cases} 1 & c > r \\ e^{1 - r/c} & c \le r \end{cases}, \qquad \text{BLEU} = \text{BP} \cdot \exp\Big(\sum_{n=1}^{4} w_n \log p_n \Big).$$

其中 G_n 是候選句中相異 n-gram 的集合；$c(g)$ 是 n-gram g 在候選中的次數；$c_{\text{ref}}^{\max}(g)$ 是它在該句任一參考中的最大次數；c 是整個語料的候選總長度；r 是**有效參考長度**，即每句取最接近候選長度的參考長度後加總；BP 是**短度懲罰**（brevity penalty），防止系統靠寫極短輸出提高精確率；$w_n = 1/4$ 是均勻權重。BLEU 是語料層級分數：精確率先在所有句子上合併，再取對數，所以它不是句子分數的平均。

```
import math
from collections import Counter

def ngrams(tokens, n):
    return Counter(tuple(tokens[i:i + n]) for i in range(len(tokens) - n + 1))
```

```
def corpus_bleu(hypotheses, references, max_n=4):
    """hypotheses: list of token lists; references: list of lists of token lists."""
    matched, total = [0] * max_n, [0] * max_n
    hyp_len = ref_len = 0
    for hyp, refs in zip(hypotheses, references):
        hyp_len += len(hyp)
        # effective reference length: the reference length closest to the hypothesis
        ref_len += min((abs(len(r) - len(hyp)), len(r)) for r in refs)[1]
        for n in range(1, max_n + 1):
            counts = ngrams(hyp, n)
            max_ref = Counter()
            for r in refs:
                max_ref |= ngrams(r, n)            # elementwise max over references
            matched[n - 1] += sum((counts & max_ref).values())   # clipped counts
            total[n - 1] += max(len(hyp) - n + 1, 0)
    if min(matched) == 0:
        return 0.0, matched, total
    log_p = sum(math.log(m / t) for m, t in zip(matched, total)) / max_n
    bp = 1.0 if hyp_len > ref_len else math.exp(1 - ref_len / hyp_len)
    return bp * math.exp(log_p), matched, total

refs = [["the cat is on the mat".split(), "there is a cat on the mat".split()]]
_, m, t = corpus_bleu(["the the the the the the the".split()], refs, max_n=1)
print("clipped unigram precision:", f"{m[0]}/{t[0]}")
ref = "pump a is not operating at 30 c".split()
for hyp in ["pump a is not operating at 30 c",
            "pump a is operating at 30 c",
            "pump a is not running at 30 c"]:
    score, _, _ = corpus_bleu([hyp.split()], [[ref]])
    print(f"{score:.3f}  {hyp}")
assert m[0] == 2 and t[0] == 7
assert corpus_bleu([ref], [[ref]])[0] == 1.0
```

輸出：

```
clipped unigram precision: 2/7
1.000  pump a is not operating at 30 c
0.515  pump a is operating at 30 c
0.500  pump a is not running at 30 c
```

第一項檢查重現經典的截限例子：重複七次「the」的候選只得到兩次的分數，因為沒有任何參考包含超過兩個「the」[13]。第二項檢查是警訊。去掉 `not` 會反轉意思，分數卻高於把 `operating` 換成 `running` 的正確改述。BLEU 量測的是與參考的表面重疊，它不知道哪些詞承載命題。

這個實作使用空白分詞、不做平滑（任一 n-gram 精確率為零就得零分），並採用最接近參考長度規則。發表的 BLEU 分數取決於這些選擇。Post 指出，光是分詞與正規化的差異就能大幅改變回報的 BLEU，並提出 sacreBLEU：它在去分詞後的文字上以標準內部分詞器計算分數，並印出版本簽章 [14]。回報任何 BLEU 數字時都應附上簽章；不同設定算出的分數不能比較。

chrF

chrF 以**字元** n-gram 取代詞 n-gram 評分 [15]：

$$\mathrm{chrF}_\beta = (1+\beta^2)\,\frac{\mathrm{chrP}\cdot\mathrm{chrR}}{\beta^2\,\mathrm{chrP}+\mathrm{chrR}}.$$

其中 chrP 與 chrR 是對各 n-gram 階數平均的字元 n-gram 精確率與召回率；β 讓召回率的權重是精確率的 β 倍。Popović 在其實驗中發現 6-gram 效果最好，並比較了 $\beta=1$ 與 $\beta=3$ [15]。字元 n-gram 對共享詞幹與屈折變化給部分分數，對形態豐富的目標語言有幫助；但 chrF 仍是重疊分數，分辨不出 `not` 比 `the` 重要。

學得指標：COMET

COMET 使用跨語言預訓練編碼器，訓練神經模型從來源、候選與參考預測人類品質判斷 [16]。它能獎勵與參考共享詞很少的正確改述。在以專家 MQM（多維品質指標，Multidimensional Quality Metrics）標註為準的 WMT22 指標任務中，神經學得指標與人類評分的相關性優於 BLEU、chrF 等重疊指標 [18]。學得指標帶有自己的模型版本、訓練分布與語言涵蓋，輸出只是在那些條件下的量測。應在應用重要的否定、數量與術語錯誤上檢查指標。

能排序系統的指標，不會自動成為輸出門檻

WMT25 評估結果分開片段品質預測、錯誤區段標註與品質導向修正，指出系統層級與片段層級評估可能偏好不同方法，語言多樣性仍然困難 [19]。這很重要：拒絕一個錯誤譯文，與比較模型平均品質，是不同決策。指標可能獎勵整體流暢系統，卻漏掉個別輸出的少見否定反轉。接受門檻需要在這些錯誤上驗證，不能直接由總體相關性結果繼承。

可交叉檢查生成器錯誤類別與評估器取得的輸入。只看參考的比較能檢查用詞一致，卻缺少驗證無支持陳述所需的來源；取得來源的評估器雖有證據，仍可能與生成器共享作用範圍或數量錯誤。應檢查分歧案例：正確改述被評差、流暢矛盾被評好，以及準確卻不完整的譯文。人工覆核保留原始來源，並分開充分性與風格。這能說明哪些評估訊號適合比較模型，哪些可用來判斷某次結果是否應接受。

重疊、召回與人工判斷

同一個反例也影響 ROUGE 用於摘要的偏召回重疊 [17]。簡單 unigram 召回檢查：

$$R_1 = \frac{\sum_{w} \min(c_{\mathrm{ref}}(w), c_{\mathrm{hyp}}(w))}{\sum_{w} c_{\mathrm{ref}}(w)}.$$

其中 w 遍歷詞型；$c_{\mathrm{ref}}(w)$、$c_{\mathrm{hyp}}(w)$ 是參考與候選詞頻；分母是參考詞元數。以下最小實作沒有重現完整套件的每項前處理或聚合。

```
import re
from collections import Counter
```

```
def words(text):
    return re.findall(r"[a-z0-9]+", text.lower())
def overlap_recall(reference, hypothesis):
    ref, hyp = Counter(words(reference)), Counter(words(hypothesis))
    return sum((ref & hyp).values()) / sum(ref.values())
reference = "Pump A is operating at 30 C."
contradiction = "Pump A is not operating at 30 C."
score = overlap_recall(reference, contradiction)
print("Unigram recall:", score)
assert score == 1.0
assert ("not" in words(reference)) != ("not" in words(contradiction))
```

矛盾句保留所有參考詞，因此召回 1.0。精確率會懲罰多出的詞，但小懲罰仍不能表達否定反轉的重要性。這是受控反例，不表示重疊指標永不與人類判斷相關，而是說明必須配合對意義敏感的檢查。

人工評估要隱藏系統身分、隨機呈現、保留來源並指定決策。Freitag 等人的大型 MQM 研究由能看到完整文件的專業譯者評分，得到的系統排名與同一共享任務中群眾評分者的排名有明顯差異 [30]。應請覆核者先標出具體錯誤，再判斷整體品質；回報誰覆核、涵蓋哪些語言，並把裁決過的挑戰集合與一般樣本分開。

6. 動手做：微型 Transformer 翻譯器

直觀說法。我們發明一個小型語言對，在 CPU 上用 3,000 個句對訓練編碼器—解碼器 Transformer 不到一分鐘，再用上面的 BLEU 函數比較貪婪與束搜尋解碼。

來源語言類似英文：主詞、動詞、受詞，名詞前可有形容詞，也可有 `never`。目標語言是發明的：每個詞有固定譯詞，形容詞放在名詞之後，受詞後接助詞 `o`，動詞放最後，否定變成句尾詞 `nai`。例如 `the small drone never lifts a red cable` 譯為 `la drono pico un kablo rosso o levi nai`。因此翻譯同時需要字典與詞序調整，正是逐詞翻譯不足的兩個原因。訓練句與測試句不重疊，但來自同一個小文法，所以這是分布內測試。

模型使用 PyTorch 的編碼器與解碼器層（各兩層、寬度 64、四個頭）、縮放嵌入加正弦位置、布林遮罩（`True` 表示阻擋），並以 `ignore_index` 從損失中去掉填補目標，實作第 2 節的遮罩損失。請在 BLEU 程式區塊之後執行，因為它會呼叫 `corpus_bleu`。

```
# Run after the BLEU block above: this code calls its corpus_bleu function.
import math, random, time
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0); torch.set_num_threads(1); rng = random.Random(0)
lexicon = {"the": "la", "a": "un", "robot": "robo", "sensor": "senso",
           "pump": "pompa", "valve": "valva", "drone": "drono", "cable": "kablo",
           "red": "rosso", "blue": "blu", "big": "grando", "small": "pico",
           "sees": "vidi", "checks": "kontrolli", "moves": "movi",
```

```
           "opens": "aperi", "lifts": "levi"}
DET, NOUN = ["the", "a"], ["robot", "sensor", "pump", "valve", "drone", "cable"]
ADJ, VERB = ["red", "blue", "big", "small"], ["sees", "checks", "moves", "opens", "lifts"]

def noun_phrase():
    adj = [rng.choice(ADJ)] if rng.random() < 0.5 else []
    return rng.choice(DET), adj, rng.choice(NOUN)

def pair():          # source: SVO, adjective before noun; target: SOV, adjective after noun
    (d1, a1, n1), (d2, a2, n2) = noun_phrase(), noun_phrase()
    verb, neg = rng.choice(VERB), rng.random() < 0.3
    src = [d1, *a1, n1] + (["never"] if neg else []) + [verb, d2, *a2, n2]
    np_ = lambda d, a, n: [lexicon[d], lexicon[n]] + [lexicon[x] for x in a]
    tgt = np_(d1, a1, n1) + np_(d2, a2, n2) + ["o", lexicon[verb]] + (["nai"] if neg else [])
    return " ".join(src), " ".join(tgt)

pairs = list(dict(pair() for _ in range(6000)).items())   # unique source sentences
train_pairs, test_pairs = pairs[:3000], pairs[3000:3200]
PAD, BOS, EOS = 0, 1, 2
src_vocab = ["<pad>", "<bos>", "<eos>"] + sorted({w for s, _ in pairs for w in s.split()})
tgt_vocab = ["<pad>", "<bos>", "<eos>"] + sorted({w for _, t in pairs for w in t.split()})
s_id = {w: i for i, w in enumerate(src_vocab)}
t_id = {w: i for i, w in enumerate(tgt_vocab)}

def batch(texts, ids, L):
    rows = [[ids[w] for w in t.split()] + [EOS] for t in texts]
    return torch.tensor([r + [PAD] * (L - len(r)) for r in rows])
S = batch([s for s, _ in train_pairs], s_id, 10)
Y = batch([t for _, t in train_pairs], t_id, 12)
Y_in = torch.cat([torch.full((len(Y), 1), BOS), Y[:, :-1]], dim=1)   # shift right

def sinusoidal(n, d):
    pos = torch.arange(n)[:, None]
    omega = 1.0 / 10000 ** (torch.arange(0, d, 2) / d)
    pe = torch.zeros(n, d)
    pe[:, 0::2], pe[:, 1::2] = torch.sin(pos * omega), torch.cos(pos * omega)
    return pe

class TinyMT(nn.Module):
    def __init__(self, d=64):
        super().__init__()
        self.d = d
        self.src_emb = nn.Embedding(len(src_vocab), d)
        self.tgt_emb = nn.Embedding(len(tgt_vocab), d)
        self.register_buffer("pe", sinusoidal(32, d))
        enc = nn.TransformerEncoderLayer(d, 4, 128, dropout=0.0, batch_first=True)
        dec = nn.TransformerDecoderLayer(d, 4, 128, dropout=0.0, batch_first=True)
        self.encoder = nn.TransformerEncoder(enc, 2, enable_nested_tensor=False)
        self.decoder = nn.TransformerDecoder(dec, 2)
        self.out = nn.Linear(d, len(tgt_vocab))
    def embed(self, emb, x):
        return emb(x) * math.sqrt(self.d) + self.pe[: x.shape[1]]
    def encode(self, src):
        return self.encoder(self.embed(self.src_emb, src), src_key_padding_mask=src.eq(PAD))
```

```
    def decode(self, memory, src, y_in):
        T = y_in.shape[1]
        causal = torch.ones(T, T, dtype=torch.bool).triu(1)      # True = blocked
        h = self.decoder(self.embed(self.tgt_emb, y_in), memory, tgt_mask=causal,
                         tgt_key_padding_mask=y_in.eq(PAD),
                         memory_key_padding_mask=src.eq(PAD))
        return self.out(h)

@torch.no_grad()
def greedy(src_text, max_len=14):
    src = batch([src_text], s_id, len(src_text.split()) + 1)
    memory, y = model.encode(src), [BOS]
    for _ in range(max_len):
        logp = model.decode(memory, src, torch.tensor([y]))[0, -1].log_softmax(-1)
        y.append(int(logp.argmax()))
        if y[-1] == EOS:
            break
    return [tgt_vocab[i] for i in y[1:] if i != EOS]

@torch.no_grad()
def beam_search(src_text, width=4, alpha=0.6, max_len=14):
    src = batch([src_text], s_id, len(src_text.split()) + 1)
    memory = model.encode(src)
    beams, finished = [([BOS], 0.0)], []
    lp = lambda n: ((5 + n) / 6) ** alpha             # GNMT length penalty
    for _ in range(max_len):
        prefixes = torch.tensor([y for y, _ in beams])
        logp = model.decode(memory.expand(len(beams), -1, -1), src.expand(len(beams), -1),
                            prefixes)[:, -1].log_softmax(-1)
        cand = [(y + [t], s + logp[b, t].item()) for b, (y, s) in enumerate(beams)
                for t in logp[b].topk(width).indices.tolist()]
        cand.sort(key=lambda c: c[1], reverse=True)
        beams = []
        for y, s in cand:
            (finished if y[-1] == EOS else beams).append((y, s))
            if len(beams) == width:
                break
        if not beams:
            break
    best, _ = max(finished or beams, key=lambda c: c[1] / lp(len(c[0]) - 1))
    return [tgt_vocab[i] for i in best[1:] if i != EOS]

@torch.no_grad()
def log_prob(src_text, words):                        # model score of a full output
    src = batch([src_text], s_id, len(src_text.split()) + 1)
    y = [t_id[w] for w in words] + [EOS]
    logp = model.decode(model.encode(src), src, torch.tensor([[BOS] +
     ↪  y[:-1]])).log_softmax(-1)
    return logp[0, torch.arange(len(y)), y].sum().item()

def evaluate():
    model.eval()
    refs, outputs = [[t.split()] for _, t in test_pairs], {}
    for name, decode in [("greedy", greedy), ("beam-4", beam_search)]:
```

```
        hyps = [decode(s) for s, _ in test_pairs]
        exact = sum(h == r[0] for h, r in zip(hyps, refs)) / len(refs)
        mean_lp = sum(log_prob(s, h) for (s, _), h in zip(test_pairs, hyps)) / len(hyps)
        print(f"  {name}: BLEU {corpus_bleu(hyps, refs)[0]:.3f}  exact {exact:.3f}"
              f"  mean log p {mean_lp:.3f}")
        outputs[name] = hyps
    print("  outputs that differ:", sum(g != b for g, b in zip(*outputs.values())))
    model.train()

model = TinyMT()
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
start = time.time()
for step in range(1, 401):
    idx = torch.randint(0, len(S), (64,))
    logits = model.decode(model.encode(S[idx]), S[idx], Y_in[idx])   # teacher forcing
    loss = F.cross_entropy(logits.reshape(-1, len(tgt_vocab)), Y[idx].reshape(-1),
                           ignore_index=PAD)
    opt.zero_grad(); loss.backward(); opt.step()
    if step in (100, 400):
        print(f"after {step} steps: training loss {loss.item():.3f}")
        evaluate()
model.eval()
print(" ".join(greedy("the small drone never lifts a red cable")))
print(f"total time {time.time() - start:.0f}s")
```

輸出（CPU、單執行緒；時間依機器而異，其他 PyTorch 版本或硬體可能改變最後幾位數）：

```
after 100 steps: training loss 0.221
  greedy: BLEU 0.614  exact 0.365  mean log p -1.587
  beam-4: BLEU 0.618  exact 0.375  mean log p -1.553
  outputs that differ: 31
after 400 steps: training loss 0.003
  greedy: BLEU 1.000  exact 1.000  mean log p -0.027
  beam-4: BLEU 1.000  exact 1.000  mean log p -0.027
  outputs that differ: 0
la drono pico un kablo rosso o levi nai
total time 16s
```

分開解讀兩個檢查點。訓練 100 步後模型只訓練了一半：200 個測試譯文中只有約三分之一完全正確。此時束搜尋改變了 200 個輸出中的 31 個，找到平均模型對數機率較高的輸出，BLEU 與完全匹配率也略為提升。400 步後，模型在這個分布內測試上既有信心又正確，貪婪與束搜尋輸出完全相同：模型分布非常尖銳時，更寬的搜尋找不到別的東西。束搜尋藉由減少**搜尋錯誤**而有幫助；它修不好**模型錯誤**，在簡單輸入上也可能毫無改變。

這裡 BLEU 與完全匹配同步變化，是因為語言很小又確定：每個來源恰有一個正確譯文。使用真實參考時，正確譯文可能與參考不同，兩個數字就會分歧。測試也只檢查同一文法的句子，完全沒說明組合泛化，例如訓練中從未見過的形容詞—名詞組合；練習 1 請你測試這點。

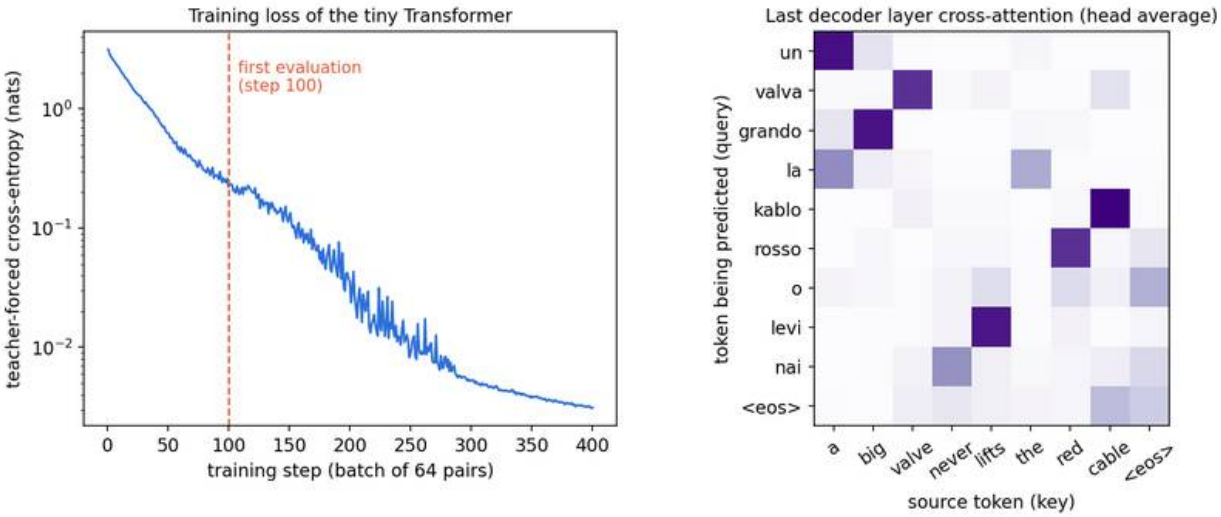


圖 *1* — 微型翻譯器。左：上方程式碼的教師強制訓練損失。右：對一個翻譯正確的保留句，最後一層解碼器交叉注意力（各頭平均）。每一列是預測該標示詞的解碼器位置。偏離對角線的圖樣就是模型學到的詞序調整；它顯示計算從哪裡讀取，不證明為何正確。

圖 1 的交叉注意力圖，是 Bahdanau 等人為 RNN 翻譯器視覺化的軟對齊的 Transformer 版本 [7]。目標中的形容詞讀取來源中位於名詞之前的形容詞；移到句尾的動詞讀取來源動詞；`nai` 讀取 `never`。這個對齊只從翻譯損失中浮現，沒有提供任何對齊標籤。

7. 翻譯跨形式保留意義

直觀說法。好的譯文會改變用詞與詞序，但保留誰對誰做了什麼、何時、多少，以及事情是否真的發生。

翻譯是具強內容限制的條件生成：保留來源意義，同時產生自然目標語言。詞序、形態、慣用語與隱含上下文，使它不只是逐詞替換。McRoy 翻譯討論 [2] 可搭配 Hagiwara 序列到序列材料 [1]。

應分開評估**充分性（adequacy）**——來源意義是否保留——與**流暢度（fluency）**——目標是否自然。流暢譯文可能漏例外、反轉否定或改行動者；字面翻譯則可能保技術內容卻不自然。可接受平衡取決於用途，必須在選模型或指標前指定。

通用翻譯器可用詞彙表保術語，仍要在上下文測試。必需詞可能存在，卻附錯實體；保留數字也不夠，因為值可能重新分配。可測「Pump A 比 Pump B 熱」、「閥門可能保持關閉」、「中午前未偵測到故障」等關係。最小改動揭露比較、模態、否定與時間是否被尊重。

語言標籤與正規化屬於模型契約。繁體與簡體是不同輸出要求；廣泛語言標籤不指定書寫系統、地域術語與標點。存來源前不要破壞性轉換，應並存原文與正規化輸入，讓覆核者追查誤譯。

低資源情況的不確定性可能來自訓練涵蓋有限，而非輸入含糊。報告應記錄領域、方言、文字系統與語言對。短產品描述驗證不代表長技術指令也成立。覆核者不能讀兩語就無法直接評充分性，要使用合格雙語覆核並說明缺口。

保留命題，不只保留詞

翻譯時，把來源與目標當成具有參與者、關係與作用範圍的命題比較。「Pump A is warmer than Pump B」必須保留比較方向。譯文即使含兩個名稱與「較熱」，仍可能反轉方向。「No sensor exceeded the limit」也不同於「Not every sensor exceeded the limit」：後者允許部分超標。NLP 12 說明這兩種否定範圍為何不同。

有時歧義原本就在來源。`May` 可依脈絡表示可能或許可。如果孤立句子無法決定，把它譯成明確表示許可的說法，就新增了解讀。有周邊文件或對話上下文時應利用；否則保留或標明歧義。目標流暢，不足以合理化任意決定未充分指定的來源。

以意義為中心的比較，可逐項檢查實體綁定、事件狀態、數量、單位、否定與條件，再評整體可讀性。這允許詞彙變化，同時揭露命題改變，也能區分真正誤譯與來源已有的歧義：因缺來源脈絡造成的標註分歧，不能當成某個詞彙選擇就能修好模型的證據。

8. 聊天機器人：把對話當成序列到序列

直觀說法。聊天機器人可以像翻譯器一樣建構：「來源」是目前為止的對話，「目標」是下一個回覆。這適合流暢閒聊，但任務助理還需要追蹤已經決定了什麼。

Vinyals 與 Le 在 IT 服務台資料集與電影字幕上訓練序列到序列模型，從先前回合預測下一回合；他們發現，即使最佳化的概似目標並不直接量測對話品質，模型仍能進行簡單對話 [23]。Li 等人觀察到這類模型傾向產生「I don't know」之類安全、通用、與輸入無關的回覆，並提出偏好針對訊息內容回覆的最大互資訊目標 [24]。這個教訓延續到現代助理：高機率回覆不一定有資訊量，也不一定正確。

回覆生成可寫成 $p_\theta(u_t \mid h_t, s_t, e_t)$，其中 u_t 是下一個系統話語、h_t 是選定歷史、s_t 是任務狀態、e_t 是證據、θ 是模型參數。條件分布說明生成器收到什麼，不保證它遵守每個欄位或證據。

任務導向對話有明確結果：形成查詢、完成類似預約的紀錄、找正確觀察或產生指定報告。開放交談的目標較不明確，可接受回覆也更多。兩者都需要一致性，成功標準卻不同。模組化任務系統分開理解、狀態追蹤、政策、證據存取與文字實現，讓錯誤容易定位。端到端系統可整合這些功能，但仍要測出錯狀態與措辭問題的差別。Deriu 等人 [25] 與 Ni 等人 [26] 的綜述描述更廣的設計與評估家族。

任務政策可選澄清、查詢、解釋或拒絕不支援操作，再由生成器說出這個動作。這讓對話流暢度不決定查詢是否準備好。開放交談受益於彈性語言，但角色風格不能覆蓋事實一致性。系統可能很吸引人，卻編造過去互動，或暗示未發生的動作已完成。對話主張要對實際紀錄檢查，外部主張則對證據檢查。

9. 回合、對話行為與狀態

直觀說法。對話是一個持續變化的共享問題。最新一句可能取代先前請求、回指先前事物，或修正系統。系統必須追蹤哪些被提議、哪些真正被接受。

對話回合是參與者在交談某時點的貢獻。它的功能，也就是**對話行為**（dialogue act），可以是請求、回答、修正、確認或澄清。同一表面詞依待回答問題而有不同功能：「週二」可能提供日期，

也可能修正原日期。

任務導向對話狀態儲存繼續任務所需資訊。設備查詢助理的欄位可包含裝置、時間區間、測量項目與輸出語言。值應連同來源回合、信心與狀態儲存。未知、明確不存在與不適用，不能全部壓成同一個空字串。狀態更新可寫成：

$$s_t = U(s_{t-1}, o_t, r_t), \qquad a_t = \pi(s_t).$$

其中 s_t 是回合 t 後的狀態；o_t 是觀察話語或擷取對話行為；r_t 指出說話者與其他回合脈絡；U 是更新規則；a_t 是下一系統動作；π 是狀態到動作的政策。狀態不一定是單一確定紀錄，政策也不一定直接生成文字。

提案與已接受狀態應分開。使用者可以要求選項、檢視再改變。系統建議出現在逐字稿，不代表成為使用者偏好。修正需要範圍：「其實要 Pump B」可只換裝置而保留時間；「重新開始」可能清除整個任務；「不是昨天」否定值卻未必提供替代。

含修正測試的確定性狀態追蹤器

對狹義任務，結構化追蹤器是有用基準，揭露語言理解與動作選擇的契約。範例消費的是已擷取事件，不主張能理解任意自然語言。狀態儲存選定欄位與修訂編號。任何接受的欄位變更，都讓先前確認失效；確認只對明確展示給使用者的版本有效，避免舊的「是」授權新變更提案。

```
from copy import deepcopy

initial = {"device": None, "period": None, "revision": 0, "confirmed": None}
def update(state, event):
    result = deepcopy(state)
    kind = event["kind"]
    if kind == "set":
        slot, value = event["slot"], event["value"]
        allowed = {"device": {"pump_a", "pump_b"}, "period": {"today", "yesterday"}}
        if slot not in allowed or value not in allowed[slot]:
            raise ValueError("Unsupported slot or value")
        if result[slot] != value:
            result[slot] = value
            result["revision"] += 1
            result["confirmed"] = None
    elif kind == "confirm":
        if event["revision"] != result["revision"]:
            raise ValueError("Confirmation refers to an old proposal")
        if any(result[k] is None for k in ["device", "period"]):
            raise ValueError("Cannot confirm incomplete state")
        result["confirmed"] = result["revision"]
    else:
        raise ValueError("Unsupported event")
    return result
s1 = update(initial, {"kind": "set", "slot": "device", "value": "pump_a"})
s2 = update(s1, {"kind": "set", "slot": "period", "value": "today"})
s3 = update(s2, {"kind": "confirm", "revision": 2})
s4 = update(s3, {"kind": "set", "slot": "device", "value": "pump_b"})
print("Before correction:", s3)
```

```
print("After correction:", s4)
assert initial["device"] is None
assert s3["confirmed"] == 2 and s4["confirmed"] is None
try:
    update(s4, {"kind": "confirm", "revision": 2})
except ValueError:
    pass
else:
    raise AssertionError("Stale confirmation was accepted")
```

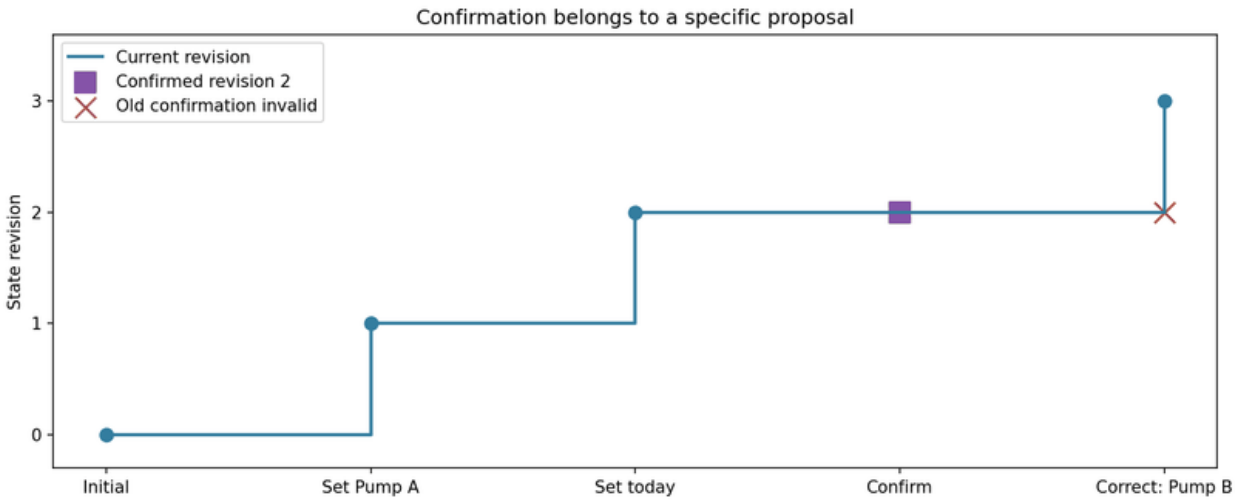


圖 2 — 修正改變狀態與確認範圍。保留修訂編號，才能看見過期接受，而非默默套到不同提案。

複製保留先前狀態，方便檢查與測試。真實系統也需要事件 ID，避免重複傳遞把更新套兩次。保存來源回合與剖析事件，才能追溯語言理解或狀態邏輯錯誤。欄位缺少不等於否定：「沒有發生故障」填入故障狀態，未提故障則仍未知。允許值有限，讓基準涵蓋範圍清楚。通用系統可使用學得欄位擷取與開放詞彙，但仍要驗證單位、結構與來源。模型信心不能代替合法值。

10. 信念狀態與有效澄清

直觀說法。話語含糊時，保留幾個帶機率的解讀，而不是直接猜。再依錯誤動作的代價，決定要行動還是提問。

簡單貝氏更新為：

$$b_t(h) = \frac{p(o_t \mid h) b_{t-1}(h)}{\sum_{h'} p(o_t \mid h') b_{t-1}(h')}.$$

其中 h 是候選解讀；$b_{t-1}(h)$ 是先前機率；o_t 是新觀察；$p(o_t \mid h)$ 是該解讀下的觀察概似；分母對候選 h' 正規化。簡化更新假設固定假說空間與已指定觀察模型。任務變動需要轉移模型或明確修正操作，不能反覆把每句話當獨立證據。

```
import numpy as np

hypotheses = ["pump_a", "pump_b", "unknown"]
prior = np.array([0.48, 0.48, 0.04])
ambiguous_likelihood = np.array([0.6, 0.5, 0.2])
posterior = prior*ambiguous_likelihood
posterior /= posterior.sum()
print("After ambiguous evidence:", dict(zip(hypotheses, posterior.round(3).tolist())))
action = "clarify" if posterior.max() < 0.9 else "query"
assert action == "clarify"
# A second synthetic observation explicitly names Pump B.
named_likelihood = np.array([0.02, 0.95, 0.03])
updated = posterior*named_likelihood
updated /= updated.sum()
print("After explicit naming:", dict(zip(hypotheses, updated.round(3).tolist())))
assert updated[1] > 0.9
assert np.isclose(updated.sum(), 1)
```

概似是教學設定，不是校準模型分數或量測的人類行為。學得系統需要驗證校準與分布轉移下的不確定性。保留未知假說，也避免封閉清單假裝每個真實請求都指已知裝置。

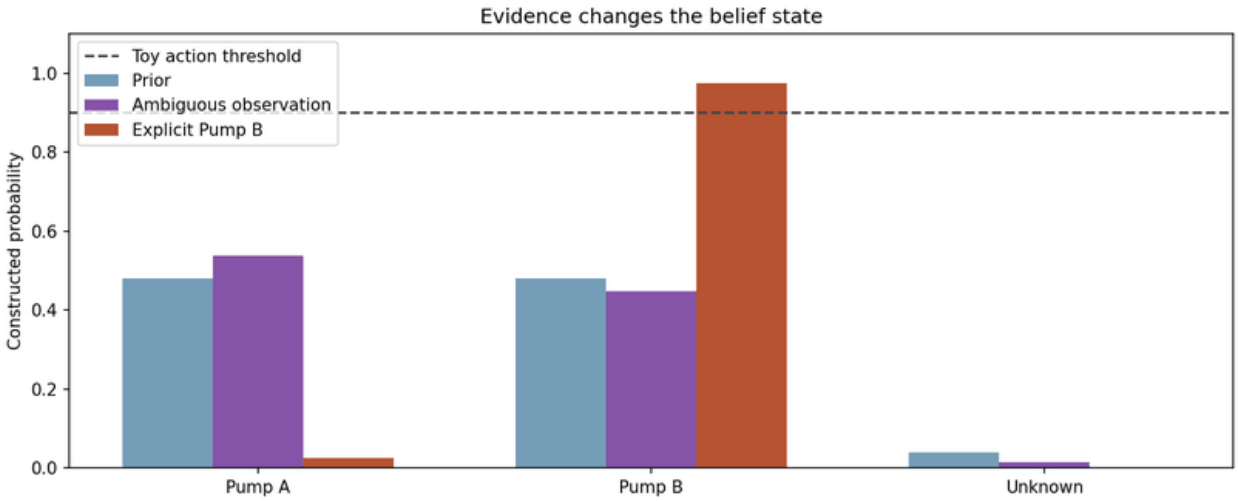


圖 3 — 信念狀態保留歧義，直到額外證據改變決策。機率為教學設定，不主張具有實證校準。

決策理論政策最小化期望損失：$a^\ast = \arg\min_a \sum_h b_t(h) L(a,h)$。其中 a 遍歷允許動作，$L(a,h)$ 是解讀 h 為真時執行動作 a 的代價。對候選問題 q，一步期望損失為：

$$V(q) = c(q) + \sum_o p(o \mid q) \min_a \sum_h b(h \mid o, q) L(a,h).$$

其中 $V(q)$ 是提問後的期望損失，$c(q)$ 是提問成本，o 遍歷可能回答，$p(o \mid q)$ 是目前信念下的回答機率，$b(h \mid o,q)$ 是收到回答後對解讀 h 的後驗，a 遍歷後續動作，$L(a,h)$ 是動作損失。最小值假定系統收到回答後選擇最佳下一步。應將 $V(q)$ 與直接動作的最小期望損失比較。這個有限決策計算是教學模型，不是已校準的人類回答模型。

```
import numpy as np

prior = np.array([0.55, 0.45])  # true device A or B
reply_given_device = np.array([[0.8, 0.2], [0.2, 0.8]])
question_cost = 1.0
for wrong_cost in [10.0, 2.0]:
    immediate = wrong_cost * (1 - prior.max())
    expected_after_question = question_cost
    for reply in range(2):
        mass = prior * reply_given_device[:, reply]
        probability = mass.sum()
        posterior = mass / probability
        expected_after_question += probability * wrong_cost * (1 - posterior.max())
    action = 'clarify' if expected_after_question < immediate else 'query A'
    print(wrong_cost, round(immediate, 2), round(expected_after_question, 2), action)
assert np.allclose(reply_given_device.sum(axis=1), 1)
```

輸出在錯誤查詢成本 `10` 時，比較 `4.5` 與 `3.0`，選擇澄清；成本 `2` 時，比較 `0.9` 與 `1.4`，直接查詢 A。信念與問題沒有改變，錯誤後果卻改變最佳動作。固定信心門檻隱藏這個依賴；期望損失則把它說清楚。應問能區分重要不確定性的問題：只有裝置含糊時，不要再問已確定的時間。

修正還需要另一個區分。「Actually, Pump B」可能改變指定裝置，而不是提供 Pump B 從頭就是意圖的證據。舊信念針對較早狀態，新話語則改變狀態本身。機率追蹤器除了觀察概似，也需要狀態轉移；或使用能辨認修正的明確更新規則。

11. 上下文、記憶與工具邊界

直觀說法。長對話放不進、也不需要放進每次模型輸入。保留下來的內容必須保留狀態：已撤回的請求不是有效請求。聊天機器人能呼叫工具時，「我做了」必須有實際結果支持。

可把上下文分為近期回合、結構化任務狀態與檢索證據。近期回合支持局部指涉與語氣；狀態保留已接受欄位及未解問題；證據支持事實主張。摘要不能把未驗證模型回覆升格為權威證據。有用記憶包含值、來源、時間、範圍與狀態（提議、接受或撤回）。滑動視窗可能只留「是」，卻刪了被確認的提案；摘要可能留某裝置，卻漏掉它已被另一裝置替換。應用合成對話測兩者，將重建狀態與明確事件紀錄比較。

工具連接的對話系統產生可由軟體執行的提案。提案、執行與結果是不同事件。「我查了資料庫」應由成功查詢結果支持，不能只是模型生成了查詢描述。工具學習研究討論基礎模型規劃、選工具、組參數及使用回饋 [27]。應用只應透過明確結構暴露允許操作，執行前驗證工具與參數，再把結果狀態及證據 ID 加入狀態。

唯讀證據也可能含命令文字。操作手冊可引用指令，檢索對話可含另一人的請求；它們仍是來源內容，不應更新當前使用者任務或擴大允許操作。結果要區分成功、沒有匹配觀察與執行錯誤；全部寫成一般流暢答案，會抹掉理解實際過程需要的資訊。重試唯讀查詢可能合理，重複改外部狀態的操作可能產生第二次效果，因此應在軟體中記錄請求 ID 與執行狀態，而不是靠語言模型記得。

12. 評估完整對話，不只一個回覆

直觀說法。看起來正確的回覆可能隱藏錯欄位、過期記憶或從未執行的動作。應跨完整對話量測狀態與最終結果。

Deriu 等人的綜述 [25] 說明任務對話、開放交談與互動問答為何需要不同評估方法。對指定欄位集合 J，定義精確狀態準確率：

$$A_{\text{state}} = \frac{1}{N}\sum_{t=1}^{N} \mathbf{1}\{\hat{s}_{t,j} = s_{t,j} \text{ for every } j \in J\}.$$

其中 N 是評估回合數；$s_{t,j}$ 是回合 t 的參考欄位 j；$\hat{s}_{t,j}$ 是預測值；$\mathbf{1}$ 在全部選定欄位匹配時為一，否則為零。嚴格指標揭露大量空值或簡單欄位平均所隱藏的錯誤。要指定欄位集合與未知值處理。

```
reference = [
    {"device": "pump_a", "period": None},
    {"device": "pump_a", "period": "today"},
    {"device": "pump_b", "period": "today"},
]
predicted = [
    {"device": "pump_a", "period": None},
    {"device": "pump_a", "period": "today"},
    {"device": "pump_a", "period": "today"},
]
slots = ["device", "period"]
joint = sum(all(p[j] == r[j] for j in slots)
            for p, r in zip(predicted, reference))/len(reference)
slot_accuracy = sum(p[j] == r[j] for p, r in zip(predicted, reference)
                    for j in slots)/(len(reference)*len(slots))
print("Exact state accuracy:", joint)
print("Per-slot accuracy:", slot_accuracy)
assert joint == 2/3 and slot_accuracy == 5/6
# The final device is wrong despite many correct fields.
assert predicted[-1]["device"] != reference[-1]["device"]
```

任務成功應檢查最終所求，不只看長度或親切風格。本例要求裝置、時間與證據正確，並尊重修正。短對話若查錯裝置，不是有效率的成功。以參考回覆計算 BLEU 在這裡並不合適，理由與翻譯相同，而且對話可接受的回覆更多，問題更嚴重。

人類評估要展示足夠歷史、指定判斷工作。只看最後回覆，無法判斷是否尊重修正。應隨機化並隱藏系統身分，定義錯誤類型並保留歧見。Schmidtova 等人 [28] 說明明確幻覺定義與方法報告為何重要。按對話切分資料，避免相鄰回合散落在訓練與測試；在對話層級回報不確定性，因為同一對話中的回合彼此相依。

現代觀點

直觀說法。翻譯與對話都走向大型預訓練模型，但本章的評估問題變得更重要，而不是更不重要。

綜述。 Stahlberg 在 *Journal of Artificial Intelligence Research*（2020）的回顧，從詞與句子嵌入及早期編碼器—解碼器，追溯神經機器翻譯到注意力與 Transformer，最後綜述近期趨勢 [22]。它是第 1–4 節材料最好的單一地圖。對話方面，Ni 等人的系統性綜述（*Artificial Intelligence Review*，2023）依模型與系統類型整理深度學習對話系統；任務導向與開放領域的區分，解釋為何一個回覆指標不能描述所有對話 [26]。Deriu 等人 [25] 把評估方法對應到這些系統類型。Qin 等人關於基礎模型工具學習的綜述（*ACM Computing Surveys*，2024）提醒我們，除了文字品質，也要檢查參數構造與結果使用 [27]。

2023–2026 最新進展。 WMT 記錄了大型語言模型進入翻譯的過程：WMT24 一般任務結果以專業人工標註，評估數個 LLM、參賽機器翻譯系統與線上服務，標題道出了平衡點：「LLM 時代已經到來，但機器翻譯尚未解決」[20]。NLLB 等大規模多語模型把涵蓋擴展到 200 種語言，明確以低資源語言為目標 [21]。評估方面，WMT22 指標任務發現，以專家 MQM 判斷為準時，神經指標比重疊指標可靠 [18]；WMT25 結果則把系統排名與片段層級錯誤偵測分開 [19]。

設計面向	要比較的選擇	需要的證據
架構	編碼器—解碼器、僅解碼器、多語	各語言對與領域的保留資料品質
解碼	貪婪、帶長度正規化的束搜尋、取樣	完整性、長度與語意錯誤，而不只是模型分數
評估	BLEU ／ chrF（附簽章）、學得指標、MQM 式人工覆核	與應用失敗類別的一致性
對話狀態	端到端生成、明確狀態與政策	狀態準確率、修正恢復、最終任務成功
工具使用	自由形式主張、附結果的已驗證提案	實際執行結果與回報結果一致

開放問題。低資源與方言翻譯、文件層級一致性、在罕見但關鍵的錯誤（否定、數字、名稱）上評估指標、可靠地自動接受個別輸出、任務變動下對話狀態的校準、多語修正，以及長對話中的過期記憶。較高的語料分數與流暢回覆都很容易取得；需要證據的是意義是否保留、最終動作是否正確。

重點整理

- SMT 結合分別估計的翻譯模型與語言模型；NMT 端到端訓練一個編碼器—解碼器，Transformer 讓它完全以注意力為基礎。
- 位移目標、保留結束詞元，在注意力與損失中都遮蔽填補；教師強制以正確前綴訓練，所以要在模型自己生成的輸出上測試（曝光偏差）。
- 貪婪與束搜尋以不同的搜尋量最佳化模型分數；長度正規化改變哪個假說勝出；兩者都修不好錯誤的模型。
- BLEU 是語料層級的截限 n-gram 精確率加短度懲罰；回報 sacreBLEU 簽章、加上 chrF 或學得指標，並直接檢查關鍵意義錯誤。

- 微型 Transformer 幾秒內就學會字典翻譯與詞序調整，它的交叉注意力在沒有監督下重現了對齊。
- 聊天機器人可以是 seq2seq 模型，但任務對話需要帶修訂的明確狀態、對含糊解讀的信念，以及依期望損失選擇的澄清。
- 讓工具提案、執行與結果保持分開，並以狀態與最終結果評估完整對話。

練習

1. **組合測試**。在翻譯器程式碼中，移除所有 `red` 修飾 `cable` 的訓練句對，再用含 `red cable` 的句子測試。比較這些句子與一般測試集的完全匹配率。

   提示

   在建立 `S` 與 `Y` 之前過濾 `train_pairs`，並保持詞彙表不變，讓失敗反映的是組合而不是未知詞。看看失敗案例的交叉注意力圖。

2. **束寬與長度**。用束寬 1、2、4、8 及 $\alpha \in \{0, 0.6, 1\}$ 評估 100 步的檢查點。回報 BLEU、完全匹配率、平均輸出長度與平均對數機率。較高的模型分數何時沒有改善 BLEU？

   提示

   束寬 1 且 $\alpha = 0$ 就是貪婪解碼。保存輸出並比較改變的句子，而不只是平均值。

3. **翻譯挑戰集合**。寫六個原創來源句，測否定、比較、單位、模態、時間與含糊指涉，指定任何可接受譯文都必須保留的意義。

   提示

   用加入 not 或交換裝置名等最小改動。來源充分性應獨立於目標是否優美。

4. **指標分歧**。建立兩個候選譯文：一個詞重疊高卻數量錯，另一個重疊較低卻意義正確。用 BLEU 函數評分，並設計能顯示差別的標註表。

   提示

   先請覆核者標出無支持的數量與遺漏的必需事實，再給整體判斷。保留來源，隱藏系統身分。

5. **修正範圍**。在追蹤器加測量欄位，測只改裝置、否定時間與完全重啟，不默默改不相關欄位。

   提示

   分別定義設定、撤回與重設事件。提案改變，就要使前一修訂的確認失效。

6. **錯誤動作報告**。設計區分查詢成功、無匹配與執行失敗的證據結構，寫出各狀態的回覆要求，並為每種狀態寫一段測試對話。

   提示

   結果附執行 ID 與狀態。生成提案本身不能支持「查詢成功」。

參考文獻

1. M. Hagiwara, *Real-World Natural Language Processing*, Manning, 2021, Chapters 6 and 8. Book ·Sequence-to-sequence chapter
2. S. McRoy, *Principles of Natural Language Processing*, open textbook, 2021, Chapter 9. Book · Generation chapter
3. P. F. Brown, S. A. Della Pietra, V. J. Della Pietra and R. L. Mercer, "The Mathematics of Statistical Machine Translation: Parameter Estimation," *Computational Linguistics* 19(2), 1993. Paper
4. P. Koehn, F. J. Och and D. Marcu, "Statistical Phrase-Based Translation," HLT-NAACL, 2003. Paper
5. K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk and Y. Bengio, "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," EMNLP, 2014. Paper
6. I. Sutskever, O. Vinyals and Q. V. Le, "Sequence to Sequence Learning with Neural Networks," NeurIPS, 2014. Paper
7. D. Bahdanau, K. Cho and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," ICLR, 2015. Paper
8. A. Vaswani et al., "Attention Is All You Need," NeurIPS, 2017. Paper
9. R. Sennrich, B. Haddow and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," ACL, 2016. Paper
10. M. Ranzato, S. Chopra, M. Auli and W. Zaremba, "Sequence Level Training with Recurrent Neural Networks," ICLR, 2016. Paper
11. S. Bengio, O. Vinyals, N. Jaitly and N. Shazeer, "Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks," NeurIPS, 2015. Paper
12. Y. Wu et al., "Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation," arXiv:1609.08144, 2016. Paper
13. K. Papineni, S. Roukos, T. Ward and W.-J. Zhu, "Bleu: a Method for Automatic Evaluation of Machine Translation," ACL, 2002. Paper
14. M. Post, "A Call for Clarity in Reporting BLEU Scores," WMT, 2018. Paper
15. M. Popović, "chrF: character n-gram F-score for automatic MT evaluation," WMT, 2015. Paper
16. R. Rei, C. Stewart, A. C. Farinha and A. Lavie, "COMET: A Neural Framework for MT Evaluation," EMNLP, 2020. Paper
17. C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," Text Summarization Branches Out, 2004. Paper
18. M. Freitag et al., "Results of WMT22 Metrics Shared Task: Stop Using BLEU – Neural Metrics Are Better and More Robust," WMT, 2022. Report
19. A. Lavie et al., "Findings of the WMT25 Shared Task on Automated Translation Evaluation Systems: Linguistic Diversity is Challenging and References Still Help," WMT, 2025. Report
20. T. Kocmi et al., "Findings of the WMT24 General Machine Translation Shared Task: The LLM Era Is Here but MT Is Not Solved Yet," WMT, 2024. Report
21. NLLB Team et al., "Scaling neural machine translation to 200 languages," *Nature* 630, 2024. DOI
22. F. Stahlberg, "Neural Machine Translation: A Review," *Journal of Artificial Intelligence Research* 69, 2020. DOI

23. O. Vinyals and Q. V. Le, “A Neural Conversational Model,” ICML Deep Learning Workshop, 2015. Paper
24. J. Li, M. Galley, C. Brockett, J. Gao and B. Dolan, “A Diversity-Promoting Objective Function for Neural Conversation Models,” NAACL-HLT, 2016. Paper
25. J. Deriu, Á. Rodrigo, A. Otegi, G. Echegoyen, S. Rosset, E. Agirre and M. Cieliebak, “Survey on Evaluation Methods for Dialogue Systems,” *Artificial Intelligence Review* 54, 2021. Survey
26. J. Ni, T. Young, V. Pandelea, F. Xue and E. Cambria, “Recent Advances in Deep Learning Based Dialogue Systems: A Systematic Survey,” *Artificial Intelligence Review* 56, 2023. Survey
27. Y. Qin et al., “Tool Learning with Foundation Models,” *ACM Computing Surveys* 57, 2024. Author manuscript ·Publication
28. P. Schmidtova et al., “Do My Eyes Deceive Me? A Survey of Human Evaluations of Hallucinations in NLG,” INLG, 2025. Survey
29. C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens and Z. Wojna, “Rethinking the Inception Architecture for Computer Vision,” CVPR, 2016. Paper
30. M. Freitag, G. Foster, D. Grangier, V. Ratnakar, Q. Tan and W. Macherey, “Experts, Errors, and Context: A Large-Scale Study of Human Evaluation for Machine Translation,” *Transactions of the Association for Computational Linguistics* 9, 2021. DOI

NLP 09・以 Transformer 做語言模型（自然語言生成）

訓練一個小型僅解碼器 Transformer 語言模型，用貪婪、束搜尋、溫度、top-k 與核取樣解碼，並評估摘要、資料到文字與有證據支持的生成。

網站版本: https://blog.shyandram.dev/zh-tw/learn/nlp/nlp-09-language-modeling-transformers/

你將學到

- 把序列機率分解成因果的下一詞元預測，並訓練一個遵守這個分解的小型僅解碼器（decoder-only）Transformer。
- 把 GPT 系列模型、它們的訓練目標與規模化研究放進同一張一致的圖。
- 用貪婪搜尋、束搜尋（beam search）、溫度、top-k 與核取樣（nucleus sampling）解碼，並說明各自改變與不改變什麼。
- 建立並評估三種生成任務：擷取式與抽象式摘要、資料到文字（data-to-text），以及開放式故事寫作。
- 以困惑度、ROUGE、人工覆核與模型評審量測生成品質，並了解各自的盲點。
- 以詞彙與稠密方法檢索證據，並用 Recall@k、MRR 與 nDCG 評估排序。
- 解釋幻覺（hallucination）、檢索增強生成，以及檢索證據與應用指令之間的界線。

先看全貌

語言模型依指定上下文替文字配置機率。這個簡單定義連結拼字補全、句子生成與可重用的上下文表示，但各種用途並沒有相同輸出契約。預測可能的延續與驗證事實回答是不同工作。流暢延續可能反映常見文字模式，卻不適合某個特定任務。

語言應用也必須選擇說什麼、清楚表達，並支持說出的內容，這是不同責任。可讀句子可能描述錯事件，正確句子也可能漏掉使用者需要的警告。部署再增加一項責任：結果必須可預期地抵達，並留下足夠上下文讓人追查失敗。

> **用最簡單的話說**
>
> 想像一個猜詞遊戲：看到句子開頭，猜下一個詞。接著把猜到的詞大聲說出來、接到句子後面，再猜下一個。語言模型就是這樣寫字的。它只能看已經說出的詞；偷看句子結尾就是作弊。而且聽起來對的猜測不等於真的，所以好的寫作者還會查核事實。

Hagiwara 第 5 章討論語言模型，第 8 章介紹注意力與 Transformer，第 9 章說明預訓練模型調適，第 10–11 章談訓練實務與部署 [1]。McRoy 第 6 章透過任務強調語言模型評估，第 9 章討論生成與摘要 [2]。本章把機率模型連到這些實際問題，只訓練微小合成模型；大型預訓練系統則透過已驗證的原始論文與綜述討論。

三個決策組織這個主題：**訓練目標**決定預測什麼；**注意力模式**決定哪些輸入位置可見；**解碼程序**決定預測如何變成使用者輸出。比較模型時要分開。改變取樣溫度會改變輸出分布，不會改變已學到哪些事實，也不會驗證產生的主張。

本章的位置：以計數為基礎的 n-gram 語言模型、平滑與最早的困惑度計算在NLP 05；注意力、位置與 Transformer 區塊在NLP 07；編碼器—解碼器翻譯與對話在NLP 08。這裡把 Transformer 當成**僅解碼器**語言模型與文字生成器使用。雙向的替代方案 BERT 在NLP 10。

1. 因果序列機率

直觀說法。依前面的詞元逐一預測，就能建立序列機率。訓練時雖然看得到完整序列，每次預測仍必須遵守可見性規則。

由機率鏈式法則：

$$p_\theta(x_1, \dots, x_T) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}).$$

其中 x_t 是位置 t 的詞元；T 是序列長度；$x_{<t}$ 是 t 之前的前綴；θ 是模型參數。起始詞元可提供第一個普通詞元的明確上下文，結束詞元則讓模型替停止配置機率。鏈式法則是精確等式，學得的條件分布則近似資料生成過程。

對含有 N 個受監督詞元位置的語料，平均負對數概似為：

$$\mathcal{L}_{\text{causal}} = -\frac{1}{N} \sum_{n=1}^{N} \log p_\theta(x_n \mid c_n).$$

其中 $\mathcal{L}_{\text{causal}}$ 是訓練目標；n 遍歷所有範例的有效預測位置；x_n 是目標詞元；c_n 是允許使用的前綴；N 不包含填補位置及其他刻意不監督的位置。自然對數使損失單位為每詞元的 nat。若批次平均把填補也計入，就改變了目標，可能獎勵模型預測人造填補符號。

NLP 05 的 n-gram 模型以最後 $n-1$ 個詞元取代 $x_{<t}$，因此結尾相同的兩個前綴必須得到相同預測。Transformer 語言模型則保留整個允許的前綴（直到上下文上限），由學得的注意力決定哪些較早詞元重要。分解方式相同，改變的是條件機率能讀取多少前綴。

教師強制（teacher forcing）在訓練時使用觀察到的前綴預測每個詞元。架構若強制因果遮罩，就能平行評估所有位置。生成時，前綴改為先前生成的詞元，錯誤可能影響後續預測。因此，即使權重相同，訓練目標與逐步生成行為仍需要不同診斷。

輸入與目標要錯開一個位置。完整序列若是起始、the、robot、moves、結束，模型輸入就是起始、the、robot、moves，目標則是 the、robot、moves、結束。若直接由未位移表示預測輸入詞元本身，就建立了與下一詞元預測無關的捷徑。第 3 節的程式明確寫出位移。

教師強制損失為何會漏掉部分生成錯誤？

教師強制提供正確前綴，即使模型自己會產生另一個前綴 [16]。假設參考從 `Pump A` 開始，生成卻選了 `Pump B`。後續詞元仍可能構成完全流暢的 Pump B 陳述。教師強制損失卻始終在 `Pump A`

後評估那些位置，沒有測試如何從這次生成錯誤恢復。差別在於：給定歷史下的預測，以及模型自己持續建立歷史時的行為。

詞元平均也隱藏不同語意後果。替換 `the` 可能只影響風格；替換 `not`、裝置 ID 或單位，卻能反轉結果。每個詞元貢獻一項對數損失，應用對這些變化的成本可能差很多。如果常見用詞改善，稀少關鍵差異卻惡化，平均損失下降仍可能伴隨更多重要欄位錯誤。因此，除了平均訓練目標，也要檢查完整輸出的命題。

2. 僅解碼器的 Transformer 語言模型

直觀說法。僅解碼器 Transformer 是一疊自注意力區塊，每個位置只能讀取自己和更早的位置。用下一詞元預測訓練後，它既能替文字評分，也能產生文字。

每個區塊先計算遮罩自注意力，再接逐位置前饋網路，並使用殘差連接與正規化，與原始 Transformer 相同 [3]；NLP 07 推導注意力公式。與雙向編碼器唯一的差別，是注意力 softmax 內的加法遮罩 B：當 $j \le i$ 時 $B_{ij} = 0$，否則 $B_{ij} = -\infty$。其中 i 是發出查詢的位置，j 是它可讀取的鍵位置。目標已位移，所以目前輸入位置仍是允許的前綴詞元，對角線可見。

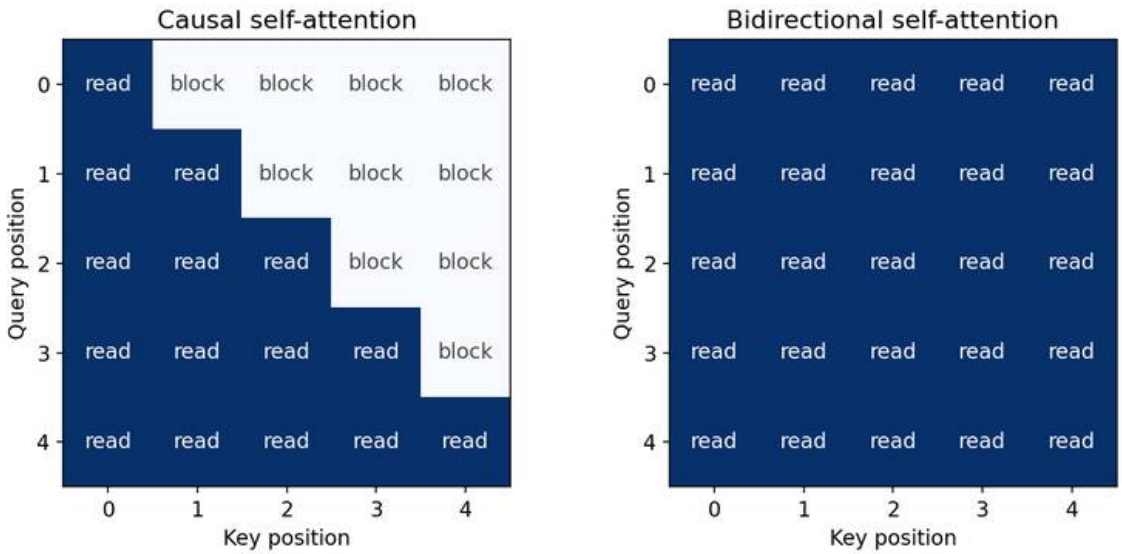


圖 *1* — 可見性是任務定義的一部分。列是發出查詢的位置，欄是它們可以讀取的位置。僅解碼器語言模型在每一層都使用左圖的模式。

最後一個線性層把位置 t 的頂層向量映成詞彙 logits $\mathbf{z}_t$，softmax 再把它轉成 $p_\theta(x_{t+1} \mid x_{\le t})$。因為遮罩的每一列都藏住未來，一次前向傳遞就能同時計算所有位置的訓練損失；生成時仍然每次只產生一個詞元。

GPT 系列

GPT 論文顯示這個單一設計能推到多遠。Radford 等人在 BooksCorpus 上，以遮罩自注意力訓練一個 12 層僅解碼器 Transformer 語言模型，再加上任務專用輸出層，針對分類、蘊涵與相似度等任務微調 [4]。配方是**生成式預訓練加判別式微調**：無標籤文字提供大部分學習訊號，少量標註資料再調適模型。

GPT-2 保留下一詞元目標，主要實驗則不再做任務專用微調。作者以新的網頁資料集 WebText 訓練，使用位元組層級的位元組對編碼（byte-pair encoding）讓任何字串都能表示，並報告語言模型開始能在零樣本（zero-shot）設定下執行問答與摘要等任務；最大模型有 15 億個參數 [5]。GPT-3 把同一方法擴大成 1,750 億參數的自迴歸模型，評估時只以文字給出任務說明與少數示範，不做梯度更新 [6]。這是同一條線上的三個點：架構與目標幾乎沒變，改變的是資料、模型大小，以及呈現任務的方式。

正確理解這條線，對後面各節很重要。提示（prompt）就是一個前綴 $x_{\le t}$。「零樣本」與「少樣本」描述的是前綴內容，不是另一種模型。延續的品質仍取決於訓練文字讓什麼變得可能，這也是後來要再加上指令微調、偏好學習與檢索（第 9–11 節）的原因。

3. 可執行的因果模型

直觀說法。小模型可以學會合成語言模式，卻對沒見過的組合泛化很差。我們還測試低損失不能證明的性質：未來詞元不得影響較早預測。

以下獨立 CPU 範例使用自訂等長句子。訓練與測試句不同，但每個普通詞彙都曾出現在訓練中。程式沒有填補、預訓練權重或資料下載，刻意作為機制實驗而非語言基準。

實作使用 AdamW 最佳化 [9]、前饋區塊中的 GELU 活化函數 [10]，以及注意力和前饋轉換之前的層正規化（layer normalization）[11]。這些選擇定義小型實驗，不是重現原 Transformer 的所有細節。研究因果遮罩時保持它們固定，避免最佳化或活化函數的改變干擾比較。

```
import math
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(11)
torch.set_num_threads(1)
train_text = [
    'the red robot moves', 'the blue robot waits',
    'a red sensor waits', 'a blue sensor moves',
    'the red sensor moves', 'a blue robot waits',
]
test_text = ['a red robot moves', 'the blue sensor waits']
vocab = ['<bos>', '<eos>'] + sorted(set(' '.join(train_text).split()))
index = {word: i for i, word in enumerate(vocab)}
def encode(rows):
    return torch.tensor([[index[w] for w in ['<bos>'] + s.split() + ['<eos>']] for s in
    ↪  rows])
train, test = encode(train_text), encode(test_text)
```

```
class TinyCausalModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.token = nn.Embedding(len(vocab), 16)
        self.position = nn.Embedding(5, 16)
        self.norm1, self.norm2 = nn.LayerNorm(16), nn.LayerNorm(16)
        self.q, self.k, self.v = [nn.Linear(16, 16, bias=False) for _ in range(3)]
        self.output = nn.Linear(16, 16)
        self.ff = nn.Sequential(nn.Linear(16, 32), nn.GELU(), nn.Linear(32, 16))
        self.head = nn.Linear(16, len(vocab))
    def forward(self, ids):
        length = ids.shape[1]
        h = self.token(ids) + self.position(torch.arange(length))
        z = self.norm1(h)
        scores = self.q(z) @ self.k(z).transpose(-1, -2) / math.sqrt(16)
        blocked = torch.ones(length, length, dtype=torch.bool).triu(1)
        weights = scores.masked_fill(blocked, float('-inf')).softmax(dim=-1)
        h = h + self.output(weights @ self.v(z))
        h = h + self.ff(self.norm2(h))
        return self.head(h)

model = TinyCausalModel()
optimizer = torch.optim.AdamW(model.parameters(), lr=0.01)
def loss(rows):
    logits = model(rows[:, :-1])
    return F.cross_entropy(logits.reshape(-1, len(vocab)), rows[:, 1:].reshape(-1))
initial = loss(train).item()
for _ in range(180):
    optimizer.zero_grad()
    objective = loss(train)
    objective.backward()
    optimizer.step()
model.eval()
with torch.no_grad():
    train_nll, test_nll = loss(train).item(), loss(test).item()
    original = train[:1, :-1]
    changed = original.clone()
    changed[0, -1] = index['blue']
    gap = (model(original)[:, :-1] - model(changed)[:, :-1]).abs().max().item()
assert train_nll < initial and gap < 1e-6
print('initial/train/test NLL:', *(round(x, 3) for x in [initial, train_nll, test_nll]))
print('held-out perplexity:', round(math.exp(test_nll), 3))
print('future-token perturbation gap:', gap)
```

輸出：

```
initial/train/test NLL: 2.194 0.409 3.17
held-out perplexity: 23.81
future-token perturbation gap: 0.0
```

斷言只檢查訓練損失下降，不要求保留損失改善特定幅度。這裡保留損失（每詞元 3.17 nat）甚至

高於未訓練時的起始損失（2.194）：保留句把熟悉詞語組成訓練未包含的組合，而模型已自信地背下六個訓練模式。大的訓練—測試落差說明，學會觀察到的模式不代表有組合泛化能力。

未來詞元擾動測試改變最後輸入詞元，再比較所有較早位置的 logits。因果注意力應讓這個變動對較早預測不可見，印出的差距正好是零。測試直接對準可見性契約，但通過不代表所有資料隔離都成立：跨切分重複文件、使用禁止的測試資料建詞彙表，或未來中繼資料，都能造成其他洩漏。

模型沒有 dropout，因此本例評估模式不會改變隨機層；仍明確呼叫它，因為真實模型必須區分訓練與評估。損失函數展平批次及位置維度，保留詞彙維度作為類別。若不小心交換這些軸，程式可能照樣執行，卻最佳化錯誤問題。

每個序列等長，位置表也涵蓋五個模型輸入位置。學得的位置表限制支援的最遠位置；超過範圍的序列不是普通長輸入，而是非法索引。擴充成可變長資料，要分別加入注意力與損失遮罩。注意力遮罩控制表示可讀什麼；損失遮罩控制哪些目標對目標函數有貢獻。兩者處理不同問題，需要獨立測試。

4. 訓練目標與規模化

直觀說法。大型語言模型使用與上面小模型相同的下一詞元損失。改變的是文字量、參數量與投入的計算量，而這三者之間有可量測的取捨。

Kaplan 等人訓練多組 Transformer 語言模型，報告保留損失約隨模型大小、資料量與計算量呈冪律下降 [7]。只看模型大小時：

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N}.$$

其中 L 是保留交叉熵損失，N 是不含嵌入的參數數量，N_c 是擬合常數，$\alpha_N > 0$ 是擬合指數。Hoffmann 等人之後擬合含不可消除項的形式 $L(N, D) = E + A/N^{\alpha} + B/D^{\beta}$，其中 D 是訓練詞元數，E 是任何模型都無法移除的損失，A, B, α, β 是擬合常數；他們的結論是，固定計算預算時，參數與訓練詞元應以大致相同比例擴大 [8]。ML 09 推導計算最適的取捨，並以程式擬合規模化定律。

有兩個提醒讓規模化保持在正確位置。第一，擬合常數取決於分詞器、資料組成與訓練程序，不能在不相關的設定間轉移。第二，規模化定律預測的是**損失**，不是特定任務行為。較低的下一詞元損失通常有幫助，但模型在摘要中是否保留否定、是否會編造引用，仍要直接量測（第 8 節）。

5. 困惑度與有意義的比較

直觀說法。困惑度總結保留詞元預測表現，但數值取決於詞元定義與評估位置。

平均負對數概似為 $\mathcal{L}$ 時：

$$\mathrm{PPL} = \exp(\mathcal{L}).$$

其中 PPL 是困惑度（perplexity）；exp 是自然指數；$\mathcal{L}$ 對同一組評估位置使用自然對數。較低困惑度表示模型在該評估契約下，對觀察到的詞元配置較高平均機率，不是事實正確、實用性或合

語法性的直接指標。IT 01 把這個量連到交叉熵，NLP 05 則替計數模型計算它。

只有分詞與輸入慣例相容時才比較困惑度。一個詞可能在某詞彙表是單一詞元，在另一個表被拆成多個詞元，平均每詞元損失便使用不同單位。還應記錄是否評分起始與結束符號、文件如何分段、上下文長度，以及評估視窗是否重疊。重複計算目標位置會改變權重。

評估文件應與訓練及其衍生文件獨立。把同一文件相鄰段落分到不同集合，可能暴露風格、事實與幾乎相同措辭。具有時間相依性的應用應採適當時間切分，並保留文件版本。單一名稱或短語沒見過，不能證明周圍文件獨立。

McRoy 的基準觀點提醒我們，序列預測之外還要評估任務行為 [2]。分類器需要類別表現，擷取器需要區段與受支持關係，生成器需要意義保留與遺漏評估。困惑度較低的模型仍可能把「沒有移動」改成「移動」而失敗。評估要對準使用者真正的結果。

可重現的語言模型實驗

別人能知道哪些文字、詞元與決策產生結果，結果才有用。先記錄文件切分與理由：獨立合成句子可能適合隨機切分，客服助理則可能需要按客戶或時間隔離。本例詞彙只來自訓練句，測試組合也刻意全部使用已知詞，因而隔離一個狹義的組合泛化問題。

接著記錄分詞、序列邊界、目標位移、最大上下文與損失分母。程式評分每個下一詞元目標，包括結束詞元，但不把起始詞元當目標。若別的實驗排除停止預測，或讓不同長度句子等權，其數字回答的就是不同問題。最後區分選擇與報告：在開發資料上選架構、訓練長度與解碼行為，只在選擇凍結後檢視保留集合。保存前處理版本、模型設定、種子、允許的輸入上下文與輸出設定，敏感原文件則放在適當存取控制下。

6. 解碼：從分布到文字

直觀說法。訓練模型產生分數；解碼器決定選最強詞元、搜尋數個延續，還是取樣其他選項。這些選擇都不會讓無支持內容變可靠。

溫度

對 logit z_v 與正溫度 τ：

$$q_\tau(v) = \frac{\exp(z_v/\tau)}{\sum_{u \in V} \exp(z_u/\tau)}.$$

其中 v 是候選詞元；V 是詞彙表；u 遍歷分母候選；z_v 是模型未正規化分數；τ 是溫度；q_τ 是所得取樣分布。低溫集中在高分詞元，高溫則分散機率。當 $\tau \to 0$，取樣趨近貪婪選擇，也就是直接取最高分詞元。事實錯誤也可能是最高 logit 的選項，因此降低隨機性不會移除它。

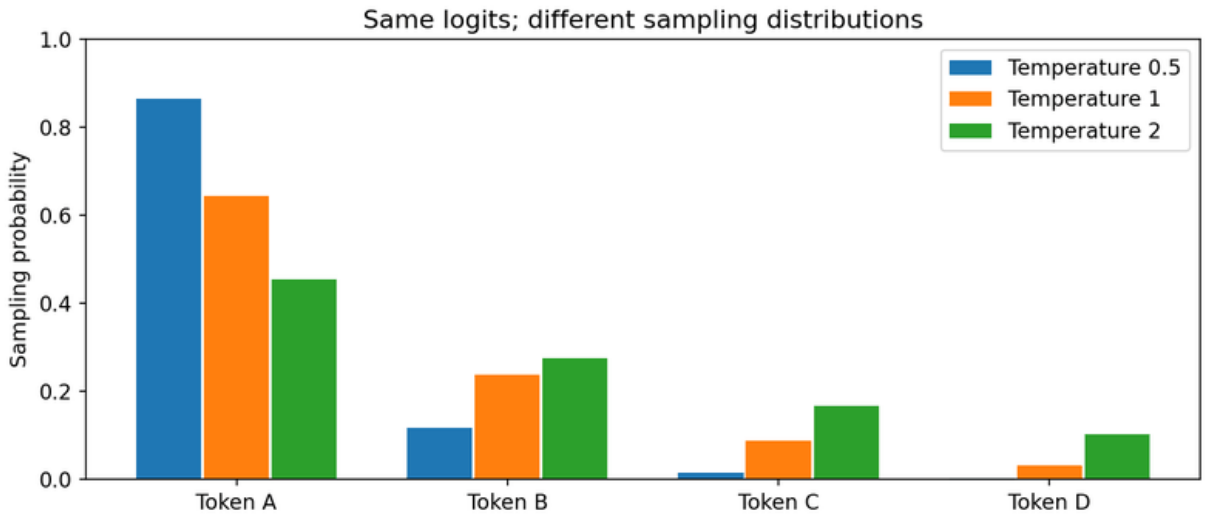


圖 2 — 固定合成 *logits* 下，溫度改變取樣分布。更集中的輸出不會自動更正確。

貪婪搜尋與束搜尋

貪婪解碼選最高的下一詞元，束搜尋保留數個部分序列。候選序列分數為：

$$\ell(y \mid x) = \sum_{t=1}^{T} \log p_\theta(y_t \mid y_{<t}, x).$$

其中 ℓ 是給定提示或來源 x 時完整輸出 y 的對數機率；T 包含結束詞元；其他符號同前。原始對數機率中，每個新增詞元都貢獻非正項，因此可能偏好短序列。長度調整會改變搜尋目標，應說明形式並在驗證資料調整，不能默默加入。

以下微型條件模型有兩個競爭的起始詞，展示選最佳下一詞不一定得到最佳完整序列。機率為實驗設定，不由語言資料估計。

```
import math

table = {
    (): {"safe": 0.55, "close": 0.45},
    ("safe",): {"stop": 0.51, "wait": 0.49},
    ("close",): {"valve": 0.99, "door": 0.01},
}
def next_probs(prefix):
    return table.get(prefix, {"<eos>": 1.0})
def decode(width):
    beams = [((), 0.0)]
    for _ in range(3):
        candidates = []
        for seq, score in beams:
            if seq and seq[-1] == "<eos>":
                candidates.append((seq, score))
```

```
                continue
            for token, prob in next_probs(seq).items():
                candidates.append((seq + (token,), score + math.log(prob)))
        beams = sorted(candidates, key=lambda pair: pair[1], reverse=True)[:width]
    return beams[0]
greedy, beam = decode(1), decode(2)
print(greedy[0], round(math.exp(greedy[1]), 4))
print(beam[0], round(math.exp(beam[1]), 4))
assert greedy[0] == ("safe", "stop", "<eos>")
assert beam[0] == ("close", "valve", "<eos>")
assert beam[1] > greedy[1]
```

輸出：

```
('safe', 'stop', '<eos>') 0.2805
('close', 'valve', '<eos>') 0.4455
```

束搜尋在人工樹中找到較可能的序列。這只說明模型的機率配置。若 close valve 與來源指令矛盾，更好的機率搜尋仍會產生錯誤的應用行為。搜尋品質與任務正確需要分開量測。

搜尋錯誤、模型錯誤與錯誤目標

微型束搜尋之所以成功，是因為保留 `close` 前綴，直到發現高機率延續。貪婪解碼在看到延續前，就丟掉該分支。相對於人工設定的序列機率目標，這是**搜尋錯誤**。較寬的束可減少這類錯誤，但仍是有界搜尋；任何深度都可能把實用前綴排除在保留集合之外。

原始前綴對數機率提供實用上界。後續每個對數機率項都非正，因此延伸前綴不會提高原始分數。若完成候選的分數高於所有未完成前綴，它就是保留下來分支中的最佳結果。這個論證要求未調整的總和；長度獎勵或正規化會改變上界，停止規則也必須一起考慮。

接著區分**模型錯誤**。若來源要求保持閥門開啟，模型卻替 `close valve` 配最高序列機率，精確機率搜尋只會更穩定地呈現模型錯誤。搜尋不能創造評分中缺少的來源相容性。最後，最大機率只是其中一種決策目標。通用短回答可能機率很高，卻漏掉必需警告。調整束寬之前，先評估目標是否獎勵使用者需要的結果。

top-k 與核取樣

在開放式寫作中，最大化機率常常不是正確目標。Holtzman 等人觀察到，即使模型很強，把概似當解碼目標仍會產生平淡且奇怪地重複的文字；而從完整分布直接取樣，又可能從長尾挑到極不可能的詞元 [12]。兩種截斷規則介於兩個極端之間。

top-k 取樣保留機率最高的 k 個詞元再重新正規化。Fan 等人在故事生成中使用它，每一步從 10 個最可能的候選中取樣，並發現此任務上它比束搜尋有效 [13]。**核取樣（top-p）**保留累積機率達到 p 的最小詞元集合 [12]：

$$V^{(p)} = \arg\min_{S \subseteq V} |S| \quad \text{s.t.} \quad \sum_{v \in S} p_\theta(v \mid x_{\le t}) \ge p, \qquad q(v) = \frac{p_\theta(v \mid x_{\le t})\,\mathbf{1}[v \in V^{(p)}]}{\sum_{u \in V^{(p)}} p_\theta(u \mid x_{\le t})}.$$

其中 $V^{(p)}$ 是核（nucleus），$|S|$ 是候選集合 S 的大小，$p \in (0, 1]$ 是目標機率質量，$\mathbf{1}[\cdot]$ 是指示函數，q 是重新正規化後的取樣分布。與 top-k 的差別在於核會**自動調整**：前綴只有一個明顯延續時，核可能只含一個詞元；前綴很開放時，核可能包含很多詞元。

```
import numpy as np

tokens = np.array(['valve', 'pump', 'door', 'sensor', 'cabinet', 'label', 'fish', 'moon'])
logits = np.array([3.0, 2.6, 1.2, 1.0, 0.2, -0.5, -1.5, -2.0])

def softmax(z):
    e = np.exp(z - z.max())
    return e / e.sum()

def temperature(z, tau):
    return softmax(z / tau)

def top_k(p, k):
    keep = np.argsort(-p)[:k]                 # indices of the k largest
    q = np.zeros_like(p); q[keep] = p[keep]
    return q / q.sum()

def nucleus(p, top_p):
    order = np.argsort(-p)
    cumulative = np.cumsum(p[order])
    size = int(np.searchsorted(cumulative, top_p) + 1)   # smallest prefix reaching top_p
    q = np.zeros_like(p); q[order[:size]] = p[order[:size]]
    return q / q.sum()

p = softmax(logits)
print('greedy choice:', tokens[np.argmax(p)])
for name, q in [('tau=0.5', temperature(logits, 0.5)), ('tau=1.0', p),
                ('tau=2.0', temperature(logits, 2.0)),
                ('top-k=3', top_k(p, 3)), ('top-p=0.9', nucleus(p, 0.9))]:
    print(f'{name:9s} support={int((q > 0).sum())}  P(valve)={q[0]:.3f}  '
          f'tail mass={q[4:].sum():.3f}')

# The nucleus adapts its size to the shape of the distribution.
flat = softmax(np.zeros(8))
peaked = softmax(np.array([6.0, 1, 0, 0, 0, 0, 0, 0]))
print('nucleus size, flat vs peaked:',
      int((nucleus(flat, 0.9) > 0).sum()), int((nucleus(peaked, 0.9) > 0).sum()))

rng = np.random.default_rng(0)
draws = rng.choice(len(tokens), size=2000, p=nucleus(p, 0.9))
assert set(draws) <= set(np.flatnonzero(nucleus(p, 0.9)))
assert np.isclose(top_k(p, 3).sum(), 1) and np.argmax(temperature(logits, 0.1)) == 0
```

輸出：

```
greedy choice: valve
tau=0.5   support=8  P(valve)=0.667  tail mass=0.003
tau=1.0   support=8  P(valve)=0.481  tail mass=0.052
tau=2.0   support=8  P(valve)=0.312  tail mass=0.190
top-k=3   support=3  P(valve)=0.545  tail mass=0.000
top-p=0.9 support=4  P(valve)=0.507  tail mass=0.000
nucleus size, flat vs peaked: 8 1
```

這裡的「tail mass」是最不可能的四個詞元所剩的機率。溫度只重塑分布，所以每個詞元都仍可能被選到：在 $\tau = 2$ 時，將近五分之一的機率落在 `cabinet`、`label`、`fish` 與 `moon`。截斷則直接移除長尾。在均勻分布上，核保留全部八個詞元；在尖峰分布上只保留一個。固定的 k 在兩種情況下都保留三個。

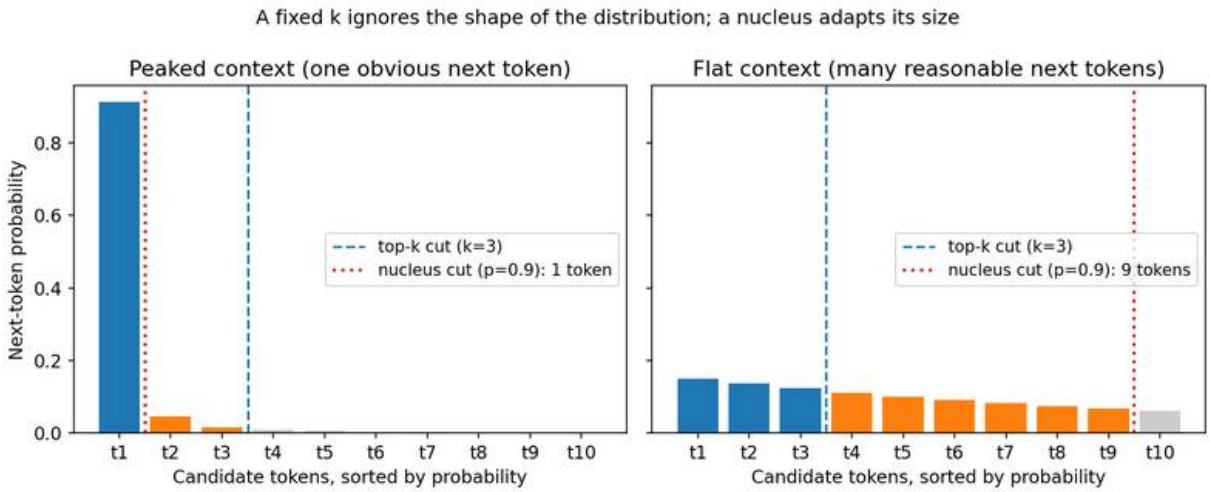


圖 3 — 兩個合成分布上的 *top-k* 與核截斷。藍色長條兩種規則都保留，橘色只被其中一種保留，灰色兩者都移除。

實務上這些設定會組合使用（例如先調溫度再做核取樣），並依任務選擇。資料到文字的報告通常需要確定且可重現的輸出；腦力激盪則需要多樣性。取樣多樣不是品質的證據，重複取樣得到一致結果也不是主張的獨立驗證：它們共享同一模型，也往往共享訓練偏誤。

停止與長度

生成還需要停止政策與長度預算。模型可能太早停止，也可能持續輸出重複內容。要明訂輸出上限、停止規則及達上限後的行為；截斷的摘要不能當完整結果回傳。解碼參數應與模型一起記錄，讓評估可重現；只改解碼器，就可能改變所有下游指標。

7. 自然語言生成任務

直觀說法。生成是產生內容與形式都符合任務的序列。輸入可以是文件、表格、提示或檢索段落；任務決定什麼算錯誤。

僅解碼器模型用同一種方式處理所有條件任務：把輸入當成前綴。條件分解為：

$$p_\theta(y \mid x, c) = \prod_{t=1}^{T} p_\theta(y_t \mid y_{<t}, x, c).$$

其中 x 是來源輸入；c 是要求語言或格式等任務脈絡；$y = (y_1, \dots, y_T)$ 是含結束詞元的輸出；$y_{<t}$ 是 t 前的詞元；θ 是學得參數。在僅解碼器模型中，x 與 c 被序列化成前綴，損失通常只計算 y 的詞元；編碼器—解碼器（NLP 08）則另外編碼 x。無論哪種，分解只描述分布，不保證取樣輸出獲得來源支持。

選模型前先定義生成問題

先用普通語言寫任務契約。設備報告可寫：「給定附時間戳的觀察，用兩句描述目前溫度及任何明確記錄的故障，保留裝置名稱、單位、數字與否定。」它揭露「好好摘要」這種無限制指令掩蓋的遺漏和矛盾。

分開三種操作。**內容選擇**選使用者相關觀察；**句子規劃**決定順序、分組與指代表達；**文字實現**把計畫寫成合語法文字。McRoy [2] 提供語言學基礎。神經系統可在內部合併這些操作，但仍需分別測試每項責任。

來源若有高溫、稍後正常溫度及明確沒有故障，選最高值與選最新值會產生不同報告。語法流暢不會決定該用哪個規則。要記錄描述目前狀態、極端事件，還是整段軌跡；時間與範圍屬於輸入契約。

資料到文字：模板基準

資料到文字生成把結構化紀錄寫成文字。Wiseman、Shieber 與 Rush 建立一個大型語料，把資料紀錄與描述文件配對，並提出擷取式評估方法，檢查文字實際傳達了哪些紀錄；他們測試的神經系統產生流暢文字，模板基準卻在其中部分指標上勝過它們 [15]。輸入結構狹窄時，模板是有用基準，可完全控制允許的主張，方便定位失敗。限制也清楚：無法自然涵蓋任意關係、指涉或篇章。應量測涵蓋範圍，不要把模板視為天生不足。

下面是狹義結構的確定報告函數，展示欄位驗證與可追溯輸出。事件紀錄是可信的結構輸入；從原文取得並驗證紀錄屬於另一個 NLP 階段。

```
import math

VERSION = "equipment-template-1"
def report(event):
    required = {"device", "temperature_c", "fault", "evidence_id"}
    if set(event) != required:
        raise ValueError("Unexpected input fields")
```

```
    if not isinstance(event["device"], str) or not event["device"].strip():
        raise ValueError("Missing device")
    value = event["temperature_c"]
    if isinstance(value, bool) or not isinstance(value, (int, float)) or not
     ↪  math.isfinite(value):
        raise ValueError("Temperature must be finite")
    if event["fault"] not in ("present", "absent", "unknown"):
        raise ValueError("Unknown fault status")
    if not isinstance(event["evidence_id"], str) or not event["evidence_id"]:
        raise ValueError("Missing evidence ID")
    phrases = {"present": "A fault was recorded.",
               "absent": "No fault was recorded.",
               "unknown": "Fault status was not recorded."}
    text = f"{event['device']} measured {value:g} C. {phrases[event['fault']]}"
    return {"text": text, "evidence_id": event["evidence_id"], "version": VERSION}
output = report({"device": "Pump A", "temperature_c": 30,
                 "fault": "absent", "evidence_id": "synthetic-event-01"})
print(output)
assert output["text"] == "Pump A measured 30 C. No fault was recorded."
assert output["version"] == VERSION
```

輸出：

```
{'text': 'Pump A measured 30 C. No fault was recorded.', 'evidence_id':
 ↪  'synthetic-event-01', 'version': 'equipment-template-1'}
```

注意 absent 與 unknown 的差別。「沒有記錄到故障」描述的是紀錄證據，不表示故障不可能。編碼這個差別，能防止文字實現階段把缺少證據升格為確定。學得生成器要在新增彈性解決了此基準已記錄的失敗時才有價值，而且應該用這些區別來測試它。

擷取式摘要：在預算內選證據

擷取式摘要選來源句子，抽象式摘要可改寫或融合句子。擷取讓每句的來源清楚，但所選句子仍可能因缺上下文、時間衝突或未解代名詞而誤導。選了「它故障了」卻沒有先行詞，即使每個字都出自來源，仍是篇章錯誤。

假設文件有候選句 $s_1, \dots, s_n$，句 i 用單位向量 v_i 表示，使用者查詢用單位向量 q 表示。簡單相關分數是 $r_i = v_i^\top q$。為避免重複，可用最大邊際相關性（maximal marginal relevance，MMR）[14]：

$$i^\ast = \arg\max_{i \notin S} \left[\lambda r_i - (1-\lambda) \max_{j \in S} v_i^\top v_j \right].$$

其中 S 是已選句索引集合；$i^\ast$ 是下一個索引；$\lambda \in [0,1]$ 平衡相關性與已選句的相似度；S 為空時，最大相似度定義為零。向量長度為一，所以內積是餘弦相似度。這是貪婪選擇規則，不是摘要品質全域最佳的證明。

以下用二元詞指示讓每個決策都可檢查，四句皆為合成。移除少數功能詞能降低語法的支配程度，卻也可能移除有用的區別。

```
import re
import numpy as np

sentences = [
    "Pump A temperature reached 80 C during the test.",
    "The test recorded Pump A temperature at 80 C.",
    "Pump A vibration triggered a shutdown warning.",
    "The technician replaced a label on the cabinet.",
]
query = "pump temperature vibration shutdown"
stop = {"a", "the", "at", "on", "during"}
def tokens(text):
    return set(re.findall(r"[a-z0-9]+", text.lower())) - stop
vocab = sorted(set().union(*(tokens(s) for s in sentences), tokens(query)))
def vector(text):
    v = np.array([w in tokens(text) for w in vocab], dtype=float)
    return v / max(np.linalg.norm(v), 1e-12)
V = np.stack([vector(s) for s in sentences])
q = vector(query)
relevance, similarity = V @ q, V @ V.T
selected = []
for _ in range(2):
    redundancy = similarity[:, selected].max(axis=1) if selected else np.zeros(4)
    score = 0.55 * relevance - 0.45 * redundancy
    score[selected] = -np.inf
    selected.append(int(np.argmax(score)))
print("Selected indices:", selected)
print("Summary:", " ".join(sentences[i] for i in sorted(selected)))
assert len(set(selected)) == 2
assert 2 in selected and not ({0, 1} <= set(selected))
```

輸出：

```
Selected indices: [2, 0]
Summary: Pump A temperature reached 80 C during the test. Pump A vibration triggered a
 ↪  shutdown warning.
```

選到的兩句包含停機句與一個溫度句，避免幾乎重複的溫度觀察。選後排序索引可恢復來源順序：選擇順序量測邊際價值，呈現順序則要支持理解。

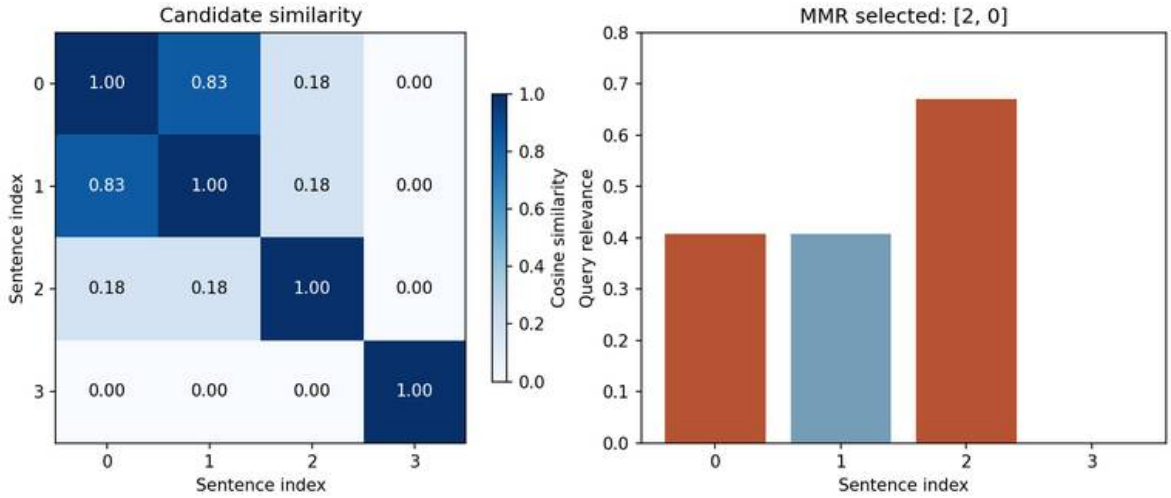


圖 4 — 擷取基準把取捨顯示出來。相似的溫度句競爭有限的摘要空間，停機句則提供不同內容。

真實預算通常限制詞元而非句數。整句分詞長度放得進剩餘預算，才能加入；默默截句可能刪掉條件或否定。多文件摘要應把近似重複報告分組，避免多份副本假裝成獨立證據。保留來源 ID 與時間戳，讓讀者能追查所選證據。

相關性、可行性與內容保留是不同目標

MMR 避免選取與先前句子相似的內容 [14]，但不知道哪些事實必須出現。若警告與已選觀察共享很多詞，相似度懲罰可能壓低它，即使它新增的限定十分重要。向量多樣性只是重複度的代理量，不直接量測完整意義。

內容要求會改變可行集合。對詞元預算與必需警告：

$$\sum_{i \in S} \ell_i \le B, \qquad S \cap W \neq \emptyset.$$

其中 S 是所選句子的索引集合，ℓ_i 是候選 i 的完整詞元長度，B 是預算，W 包含表達必需警告的候選。第一個條件限制大小，第二個確保涵蓋。若沒有集合同時滿足兩者，該預算下的擷取任務就不可行。改相關性係數，無法讓過長句子放得下，也無法創造缺少的警告。

以下獨立範例隔離這個差別。效用是明示的教學選擇，不是學得的相關分數：

```
import itertools

sentences = [
    'Pump A temperature reached 80 C during the test.',
    'The test recorded Pump A temperature at 80 C.',
    'Pump A vibration triggered a shutdown warning.',
    'The technician replaced a label on the cabinet.',
]
lengths = [len(s.split()) for s in sentences]
utility = [0.9, 0.85, 0.6, 0.2]
subsets = [tuple(i for i in range(4) if keep[i])
```

```
            for keep in itertools.product([False, True], repeat=4)]
feasible = [s for s in subsets if sum(lengths[i] for i in s) <= 20]
score = lambda s: sum(utility[i] for i in s)
ordinary = max(feasible, key=score)
with_warning = max((s for s in feasible if 2 in s), key=score)
print('Lengths:', lengths)
print('Without requirement:', ordinary, 'with warning:', with_warning)
assert ordinary == (0, 1) and with_warning == (0, 2)
```

輸出：

```
Lengths: [9, 9, 7, 8]
Without requirement: (0, 1) with warning: (0, 2)
```

這裡以空白分隔的詞計算長度；模型詞元預算則必須使用該模型的分詞器。無限制的選擇重複溫度內容，必含警告的要求則改變所選配對。不能接受遺漏時，更複雜的評分仍需要任務的硬內容要求。

抽象式摘要

改寫模型能融合觀察、提高可讀性，也能編造來源未建立的連接。「溫度高，因此幫浦故障」若來源只記錄同時發生的事件，就新增了因果。兩句改成一句，只要關係保留就可以；把先後改為因果，或刪除「during the test」，就改變了內容。先記錄命題及其來源區間、再產生文字的計畫，能讓你分別評估選擇、計畫與文字實現，找出意義在哪裡遺失。

故事生成與開放式文字

開放式生成把報告的優先順序反過來：沒有唯一正確輸出，多樣性本身就是品質的一部分。Fan、Lewis 與 Dauphin 收集人寫的故事與寫作提示配對，先生成故事前提再生成故事，並因為束搜尋傾向產生簡短而通用的文字而改用 top-k 取樣 [13]。對本章來說，重點是**同一個**語言模型可能需要依任務使用不同解碼器：受限報告用確定性搜尋，故事用截斷取樣。不變的是評估必須對準任務：故事看連貫性與切題程度，摘要看內容是否受支持。

8. 評估生成文字

直觀說法。單一自動分數無法總結所有生成品質。自動指標應按指定程序比較系統，再檢查應用真正在乎的錯誤維度。

困惑度（第 5 節）把模型當成參考文字的預測器來評估，並不直接說明某次解碼輸出的好壞，也不能比較分詞器不同的模型。**重疊指標**比較輸出與參考。ROUGE 包含偏召回的摘要重疊量測 [17]。這類指標比較候選與一個或多個參考，不證明語意等價或來源支持。報告分數要指定版本、分詞、聚合方式與參考集合。翻譯指標如 BLEU 與學得指標在NLP 08 討論。

簡單的 unigram 召回檢查：

$$R_1 = \frac{\sum_w \min(c_{\text{ref}}(w), c_{\text{hyp}}(w))}{\sum_w c_{\text{ref}}(w)}.$$

其中 w 遍歷詞型；$c_{\text{ref}}(w)$、$c_{\text{hyp}}(w)$ 是參考與候選的詞頻；分母是參考詞元數。它示範 ROUGE 式評估的一個部分，以下最小實作沒有重現完整套件的每項前處理或聚合選項。

```
import re
from collections import Counter

def words(text):
    return re.findall(r"[a-z0-9]+", text.lower())
def overlap_recall(reference, hypothesis):
    ref, hyp = Counter(words(reference)), Counter(words(hypothesis))
    return sum((ref & hyp).values()) / sum(ref.values())
reference = "Pump A is operating at 30 C."
contradiction = "Pump A is not operating at 30 C."
score = overlap_recall(reference, contradiction)
print("Unigram recall:", score)
assert score == 1.0
assert ("not" in words(reference)) != ("not" in words(contradiction))
```

輸出：

```
Unigram recall: 1.0
```

矛盾句保留了所有參考詞，因此召回為 1.0。精確率會懲罰多出的詞，但小懲罰仍不能表達否定反轉的重要性。這是受控反例，不表示重疊指標永遠不與人類判斷相關，而是說明它必須配合對意義敏感的檢查。

人工評估與錯誤類別

設備應用可分別計算**無支持主張**、**必需主張遺漏**、**數值關係錯誤**與**未解指涉**。它們是專案自訂的錯誤類別。每類都要給標註者正反例，包含缺少證據與證據矛盾的情況。說得少的摘要可能無支持主張較少，卻漏掉關鍵資訊，因此兩個方向的錯誤都要回報。

人工評估要隱藏系統身分、隨機呈現、保留來源上下文並指定決策任務。先請覆核者標出無支持陳述，再判斷整體實用性；整體偏好可能獎勵風格，卻漏掉罕見的關鍵錯誤。應報告由誰覆核，以及如何處理歧見。聚合要以有意義的單位：同一事故的多個句子彼此相關，把它們當獨立例子會誇大確定性。

以語言模型當評審

可以用提示讓強大的語言模型評分或比較輸出。Zheng 等人研究這種「LLM 當評審」（LLM-as-a-judge）設定，報告在他們的基準上，強評審的偏好與人類偏好的一致程度，可與人類彼此之間的

一致程度相比；同時也記錄了位置偏誤、冗長偏誤與自我抬高偏誤 [18]。Wang 等人顯示，只要交換兩個回覆出現的順序，就可能改變模型評審偏好哪一個，並提出兩種順序都評估等校準方法 [19]。

把模型評審當成另一種有自己版本、提示與失敗模式的量測儀器。兩種呈現順序都要評估，隱藏輸出來自哪個系統，在你自己的錯誤類別（尤其是否定、數量與無支持主張）上以人工標註驗證評審，並在可行時避免用同一模型家族評自己的系統。平均而言能排好系統順序的評審，不會自動成為接受單一輸出的可靠門檻。

9. 檢索：找到生成所依據的證據

直觀說法。生成器要使用證據之前，得先有人把證據找出來。檢索系統從明確語料選出段落；這些段落是否支持回答、答案如何措辭，則是後續的決策。

語言模型不搜尋當前文件集合，也能延續問題；檢索系統則從明確語料選資訊。問答再加入一個決策：選到的內容是否支持回答，以及用哪些詞表達。把元件結合能建立有用管線，但沒有哪個階段會自動替下一階段背書。

本節貫穿的問題是「Pump A 的溫度上限是多少？」相關段落討論 Pump A 或溫度；足夠的段落指出該幫浦在適用政策下的上限；有效引用指向真實來源；受支持回答保留數值、單位、時間與範圍。四者是不同條件，評估應能指出失敗的是哪一項。

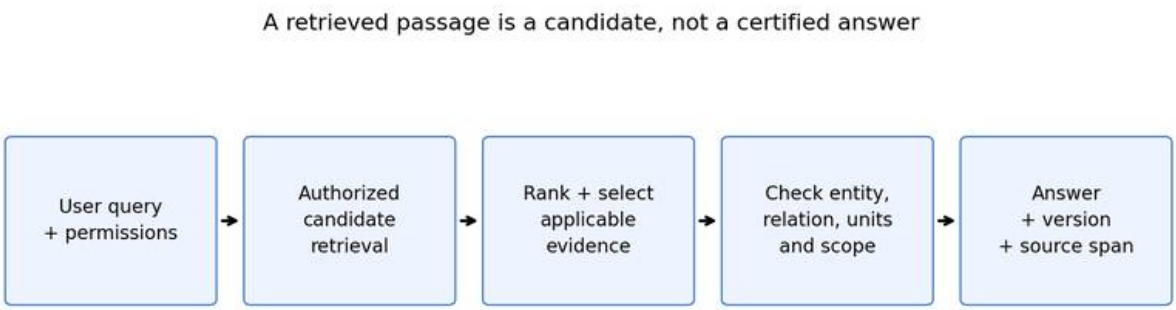


圖 5 — 檢索選候選，證據檢查決定能說什麼。每個階段都屬於明確應用契約。

本節建立並評估檢索階段。判斷檢索到的段落是否真的回答問題——擷取式片段答案、相關與可回答的區別，以及用受控替換診斷——在NLP 11 討論。第 10 節再讓生成器以檢索證據為條件。

選模型前先設計語料

直觀說法。檢索器搜尋你提供的內容。文件身分、更新狀態與權限是系統的一部分，不是排序後才補的選填資料。

可搜尋段落可表示為：

$$d = (u, v, b, e, \mathbf{x}, a).$$

其中 d 是段落紀錄；u 是父文件識別碼；v 是版本；b、e 是原文中包含起點、不包含終點的位移；$\mathbf{x}$ 是段落文字；a 描述存取政策。必須聲明位移單位。這是本章的應用紀錄，不是通用儲存標準。

段落可是一個自然段、一組句子或依文件結構選的章節。短段落能隔離答案，卻可能丟掉限定條件；長段落保留上下文，卻引入不相關內容並消耗生成預算。切塊邊界應根據任務選，再評估支持事實與條件是否仍放在一起。

例如標題寫「只適用戶外設備」，下一段才給溫度上限。單獨索引該段，可能讓人以為也適用室內設備。保留標題或結構化適用範圍會改變可回答性契約。固定字元視窗容易實作，但簡單不代表能保留意義。

版本也重要。舊政策可能相關卻不適用。需要當前答案時，應保留生效日期與替代關係。不能只因最新段落放在提示最後，就期待生成器判斷當前政策。應依明訂適用規則過濾或排序；語料沒有指定唯一權威來源時，讓衝突可見。

在未授權元件或使用者收到段落文字前，就要套用權限。私密內容若已進模型請求或紀錄，才過濾最終答案就太晚了。檢索器語料視野應符合當前使用者權限，評估也應在受限視野下進行；否則全域相關性分數可能評到系統永遠不能展示的文件。

詞彙檢索與稀疏向量

直觀說法。詞彙檢索匹配詞語，替有辨識力的詞加權。它容易檢查，精確名稱、代碼與技術短語重要時尤其有用。

簡單的詞頻與逆文件頻率權重為：

$$w_{t,d} = \mathrm{tf}(t,d)\log\frac{N}{\mathrm{df}(t)}.$$

其中 $w_{t,d}$ 是詞 t 在文件 d 的權重；$\mathrm{tf}(t,d)$ 是依選定慣例定義的詞頻；N 是索引文件數；$\mathrm{df}(t)$ 是包含該詞的文件數。只出現在查詢的詞若文件頻率為零，需要特別處理。實作常加入平滑或詞頻轉換，應記錄真正使用的慣例 [33, 34]。

使用正規化文件與查詢向量時：

$$s(q,d) = \frac{\mathbf{v}_q^{\top}\mathbf{v}_d}{\|\mathbf{v}_q\|_2\|\mathbf{v}_d\|_2}.$$

其中 q 是查詢；d 是候選文件；$\mathbf{v}_q, \mathbf{v}_d$ 是向量；s 是餘弦相似度；$\|\cdot\|_2$ 是歐氏範數。任一向量為零時定義分數為零。程式使用 scikit-learn 正規化向量，因此非零列的內積就是餘弦相似度 [34]。

倒排索引（inverted index）為每個詞儲存出現它的文件，讓搜尋引擎不必把查詢與每份文件的所有詞彙逐一比較 [33]。本例只有四段，所以直接計算全部分數。數學表示與有效率的索引策略是分開的選擇。

前處理直接影響檢索。丟掉單字元詞元的分詞器可能刪掉 Pump A 的 A。普通詞小寫化可能有幫助，卻會抹掉區分大小寫代碼。短語特徵有助辨識相鄰詞，不會涵蓋任意語法，也不保證語意相同。漏掉結果時，先看詞彙表，再怪排序模型。

詞彙系統可能漏掉改寫。Device ceased operating 與 machine shutdown 可能少有共同詞。同義詞或查詢擴展能幫忙，但也改變匹配政策、增加歧義。應在保留查詢上評估擴展，特別包含精確

代碼案例。廣泛擴展提升召回時，也可能找到術語相似的錯誤裝置。

飽和與文件長度

直觀說法。重複查詢詞很多次，不應永遠提高相關性。長文件也有更多機會偶然匹配。

常見 BM25 形式為：

$$s(q,d)=\sum_{t\in q}\operatorname{idf}(t)\frac{f_{t,d}(k_1+1)}{f_{t,d}+k_1\left(1-b+b\,|d|/\overline{L}\right)}.$$

其中 q 在簡化式中是查詢詞集合；d 是候選文件；$f_{t,d}$ 是詞頻；$\operatorname{idf}(t)$ 是選定逆文件頻率權重；$|d|$ 是詞元長度；$\overline{L}$ 是平均文件長度；k_1 是正的飽和參數；$b\in[0,1]$ 控制長度正規化。不同實作的 IDF 與查詢詞頻慣例不同，資訊檢索教科書討論了這個家族及其假設 [33]。

隨重複增加，分子與分母讓詞頻貢獻接近一個極限。長度項改變長文件預期有多少詞頻。這些是排序調整，不是文件含有真實答案的機率。一段內容可能反覆討論上限而排名很高，卻從未給出數值。

應在反映部署語料的開發查詢上調整長度正規化。若所有答案都在短參考條目，不相關內容都在長敘事，一個設定可能特別強；改成長技術手冊後，行為會改變。比較排序設定時保持切塊固定，否則應說明語料表示與分數都改變了。

稠密檢索與聯合重排序

直觀說法。學得的編碼器能匹配沒有精確重疊的相關表達。效果取決於訓練任務與對比例子。

雙編碼器分數為：

$$s_\theta(q,d)=f_\theta(q)^\top g_\theta(d).$$

其中 f_θ 把查詢轉成向量；g_θ 把段落轉成向量；θ 是學得參數；s_θ 是內積分數。編碼器可共用或分別使用參數。Dense Passage Retrieval 是這種設計在問答中的原始研究例子 [35]。

段落向量可預先計算，查詢再搜尋高分向量。近似搜尋以部分精確近鄰忠實度換速度；若使用它，就要同時評估表示與搜尋近似。本章不實作這種索引，因為小語料精確搜尋更適合學習契約。

對比訓練可把一個正段落與數個負段落比較：

$$\mathcal{L}=-\log\frac{\exp(s_\theta(q,d^+)/\tau)}{\exp(s_\theta(q,d^+)/\tau)+\sum_j\exp(s_\theta(q,d_j^-)/\tau)}.$$

其中 $\mathcal{L}$ 是單一查詢損失；d^+ 是標註正段落；d_j^- 是負候選 j；τ 是正溫度；s_θ 是學得分數。負例可包含措辭相似的困難段落。但只因未被標註就判負的段落，若也有有效答案，就成了假負例。

聯合重排序器一起讀查詢與候選段落，能產生比兩個固定向量更豐富的詞元互動，通常放在較便宜的候選檢索後。這增加計算與一個選擇程序。應先報告候選召回率：沒有收到正確段落，重排序器就無法把它找回。

稠密與詞彙檢索可以互補。稠密表示可能幫助改寫，詞彙匹配則保留精確代碼與罕見名字。混合時要注意尺度，因為詞彙分數與學得內積沒有相同單位。應在開發資料選混合設定，檢查是否引入錯誤實體結果。

對比損失實際教模型什麼？

稠密損失不只是把查詢移向任何大致相關的文章，而是學習候選集合呈現的差異。令候選零為正例，其餘為負例。對候選分數：

$$p_j = \frac{\exp(s_j/\tau)}{\sum_{r=0}^{M}\exp(s_r/\tau)}, \qquad \frac{\partial \mathcal{L}}{\partial s_j} = \frac{p_j - \mathbf{1}[j=0]}{\tau}.$$

其中 s_j 是查詢對候選 j 的分數，M 是負候選數，r 索引全部候選，$\tau > 0$ 是溫度，p_j 是候選 softmax 機率，$\mathcal{L} = -\log p_0$ 是損失，$\mathbf{1}[j=0]$ 對正例為一、其餘為零。這個導數直接由前面的 softmax 損失得到。梯度下降提高正例分數、降低負例分數；競爭最強的負例承受最大的向下壓力。

DPR 透過正負文章比較，訓練分開的查詢與文章表示 [35]。公式說明比較的選擇為何重要。若負例都來自不相關主題，模型可能學會主題區分，卻沒有學會區分 Pump A 與 Pump B。具有相同關係、卻是錯誤裝置的文章，對這種錯誤提供更有用的對比。但若未標註文章其實支持答案，把它當成負例，就會教出錯誤的分離。

這個獨立的分數空間實驗確認梯度方向。它示範目標，不會訓練文章編碼器：

```
import numpy as np

scores = np.array([1.2, 0.8, -0.4])  # positive, hard negative, easy negative
def probabilities(values):
    z = np.exp(values - values.max())
    return z / z.sum()
p = probabilities(scores)
gradient = p.copy()
gradient[0] -= 1.0  # temperature is one
updated = scores - 0.5 * gradient
before, after = -np.log(p[0]), -np.log(probabilities(updated)[0])
print('Gradient:', gradient.round(3))
print('Loss:', round(before, 3), '->', round(after, 3))
assert gradient[0] < 0 and gradient[1] > gradient[2] > 0
assert after < before
```

梯度四捨五入為 `[-0.466, 0.358, 0.108]`，損失由 `0.627` 降至 `0.467`。分數較高的競爭負例，比簡單負例承受更大的向下壓力。

溫度同時影響分布尖銳程度與梯度尺度；候選則改變模型正在學的對比。因此，新編碼器若也改了正例標籤、負例建立方式或訓練語料，提升便不能只歸因於架構。比較時應保留這些差異。

依重要差異比較檢索路徑

雙編碼器在還不知道未來查詢時，就先壓縮文章。這讓向量能快取並有效率地搜尋候選，但未來問題需要的每個細節，都必須在壓縮後保留下來。聯合重排序器同時取得兩段文字，可比較彼此的特定詞元；每對計算成本較高，仍取決於訓練範例。聯合注意力提供辨識細節的機會，不證明模型一定尊重該細節。

查詢差異	有用證據	應檢查的失敗
`Pump A` 與 `Pump B`	精確識別碼加上關係	主題相似掩蓋一個字元的差異
`ceased operating` 與 `shutdown`	學到的改述相容性	詞彙表漏掉同義表達
目前與升級前的限制	版本與生效日期中繼資料	高度相關文章回答錯誤時間範圍
許可與禁止	否定及其語法作用範圍	共同詞彙隱藏相反指令

混合設計可聯集詞彙與稠密候選、去重，再聯合重排留下的文章。聯集保留任一路徑找到的候選，但固定總預算仍可能排除部分候選。直接相加原始分數較難解讀：把一個系統的分數乘常數，雖不改自己的排序，卻會改變它在加總中的影響力。因此，正規化或學習組合需要開發資料的相關性判斷，而不是假定任意係數都適用。

應分別比較候選召回、最後順序與資源成本。系統可能改善前者卻損害後者，也可能兩者都改善卻超出服務預算。把精確名稱、改述與範圍敏感問題分組診斷，錯誤模式能揭露表示丟了哪些資訊，以及哪種下游互動值得額外計算。

排序評估

直觀說法。檢索評估需要查詢及相關性判斷。選到一份文件，不代表選對了。

對查詢 q 的相關集合 R_q 與前 k 結果集合 $T_k(q)$：

$$\text{Recall@}k(q) = \frac{|R_q \cap T_k(q)|}{|R_q|}.$$

其中 k 是截斷位置；R_q 包含標註相關文件；$T_k(q)$ 包含截斷前檢索結果；$|\cdot|$ 是集合大小。本章只對至少一份相關文件的查詢計算。沒有答案的查詢屬於另外的可回答性評估，不應任意指定召回率。

第一個相關結果排名為 r_q 時：

$$\text{RR}(q) = \frac{1}{r_q}, \qquad \text{MRR} = \frac{1}{Q}\sum_{q=1}^{Q} \text{RR}(q).$$

其中 RR 是倒數排名；MRR 是其平均；Q 是評估查詢數；r_q 從一開始。沒有回傳相關結果時定義倒數排名為零。當代檢索增強評估討論使用這些診斷 [31]。倒數排名只重視第一個成功結果，忽略後面的相關結果。

標註完整性很重要。多個段落都支持答案，卻只標一個相關，可能把有效檢索判錯。要記錄相關是指主題匹配、包含答案，還是完整支持。重要歧義可使用多位判斷者，再依書面指南裁決。資料集應揭露任務定義，不只是列一段偏好的文字。

開發與測試要隔離查詢家族及文件衍生版本。把開發問題稍作改寫放進測試，泛化證據很弱；同一報告的切塊跨集合也可能洩漏事實與措辭。切分應反映部署問題：沒見過的使用者、文件、未來版本或新領域。

量測一個小型檢索器

這段獨立程式使用四個合成段落。它保留單字元詞，讓裝置識別碼 A 仍是特徵，並在三個標註查詢上量測排序。

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = {
    'temperature-v2': 'Pump A temperature limit is 80 C. This policy is current.',
    'shutdown-v1': 'Pump A shutdown follows a vibration alarm.',
    'label-v1': 'A technician replaced the cabinet label.',
    'sensor-v1': 'Sensor B reports humidity once per hour.',
}
ids = list(corpus)
vectorizer = TfidfVectorizer(ngram_range=(1, 2), token_pattern=r"(?u)\b\w+\b")
D = vectorizer.fit_transform([corpus[key] for key in ids])
def rank(query):
    score = (D @ vectorizer.transform([query]).T).toarray().ravel()
    order = sorted(range(len(ids)), key=lambda i: (-score[i], ids[i]))
    return [(ids[i], float(score[i])) for i in order]
queries = [
    ('Pump A temperature limit', {'temperature-v2'}),
    ('vibration shutdown alarm', {'shutdown-v1'}),
    ('humidity report', {'sensor-v1'}),
]
recalls, reciprocal = [], []
for query, relevant in queries:
    ranking = rank(query)
    recalls.append(len({key for key, _ in ranking[:2]} & relevant) / len(relevant))
    first = next((i + 1 for i, (key, _) in enumerate(ranking) if key in relevant), None)
    reciprocal.append(1 / first if first else 0.0)
    print(query, '->', ranking[0][0])
print('mean Recall@2:', np.mean(recalls))
print('mean reciprocal rank:', np.mean(reciprocal))
```

輸出：

```
Pump A temperature limit -> temperature-v2
vibration shutdown alarm -> shutdown-v1
humidity report -> sensor-v1
mean Recall@2: 1.0
mean reciprocal rank: 1.0
```

三個最高結果符合各自唯一相關段落，因此本次小型執行的平均 Recall@2 與平均倒數排名都是 1.0。這是固定測試案例結果，不是真實檢索基準。NLP 11 沿用同一語料與排序器，建立範圍狹窄的證據式回答器。

分級相關性：nDCG

Recall@k 與倒數排名把判斷視為相關或不相關。若判斷者替段落分級——完整支持高於只提到主題——量測就應獎勵把最有用的段落排在最前面。

$$\mathrm{DCG@}k = \sum_{i=1}^{k} \frac{g_i}{\log_2(i+1)}, \qquad \mathrm{nDCG@}k = \frac{\mathrm{DCG@}k}{\mathrm{IDCG@}k}.$$

其中 $g_i \geq 0$ 是排名 i 結果的分級相關性；DCG@k 是前 k 名的折損累積增益（discounted cumulative gain）；IDCG@k 是已判斷結果依理想順序排列時的同一總和，因此 nDCG@$k \in [0,1]$；IDCG@$k = 0$ 時定義為零。Järvelin 與 Kekäläinen 提出累積增益這一族量測 [36]；對數底數、增益轉換與截斷位置因實作而異，報告時應寫明。

同樣四個已判斷段落的兩種排序，差別只在支持段落或主題段落排第一：

```
import numpy as np

def dcg(gains, k):
    gains = np.asarray(gains[:k], dtype=float)
    return float(np.sum(gains / np.log2(np.arange(2, gains.size + 2))))
def ndcg(gains, k):
    ideal = dcg(sorted(gains, reverse=True), k)
    return dcg(gains, k) / ideal if ideal > 0 else 0.0

# graded judgments of one ranking: 2 = complete support, 1 = topical, 0 = unrelated
ranked_a = [1, 2, 0, 0]  # topical passage above the supporting one
ranked_b = [2, 1, 0, 0]  # supporting passage first
for name, gains in [('A', ranked_a), ('B', ranked_b)]:
    print(name, 'nDCG@3:', round(ndcg(gains, 3), 3),
          'binary RR:', 1 / (1 + gains.index(2)))
assert ndcg(ranked_b, 3) == 1.0 and ndcg(ranked_a, 3) < 1.0
```

輸出：

```
A nDCG@3: 0.86 binary RR: 0.5
B nDCG@3: 1.0 binary RR: 1.0
```

若以「完整支持」為相關標準，排序 A 的倒數排名減半。nDCG 也下降，但幅度較小，因為它仍替排第一的主題段落計分。哪種行為正確取決於任務：若只有完整答案有用，主題段落的分級分數可能掩蓋失敗。選量測前先寫好分級指南。

10. 幻覺與檢索增強生成

直觀說法。語言模型可能產生流暢、卻得不到輸入或真實世界支持的文字。檢索提供外部證據，但生成器仍必須正確使用；檢索來的文字絕不能被當成新的權威。

Ji 等人的綜述區分**內在**（intrinsic）幻覺，即與來源矛盾的輸出，以及**外在**（extrinsic）幻覺，即無法由來源驗證的輸出 [20]。Maynez 等人對抽象式摘要器的人工研究發現，所檢查的每個模型生成摘要都含有大量幻覺內容，並發現文字蘊涵量測與忠實度的相關性優於標準重疊指標 [21]。這兩點都可由第 1 節與第 6 節推得：訓練目標獎勵可能的文字，任何解碼器都無法補上分布中缺少的證據。

注意力可診斷計算存取，不能證明事實。詞元讀到正確證據，仍可能表達相反意思。驗證要看輸出主張、來源實體、關係及範圍。狹義結構任務可讓生成器說出已驗證的計畫，如上面的模板；自由文字的驗證並不完整，應明說限制。

檢索增強生成

原始檢索增強生成（retrieval-augmented generation，RAG）論文，把學得檢索器與條件序列生成結合，用於知識密集任務 [22]。一種序列層級形式為：

$$p(y \mid x) \approx \sum_{z \in Z_k(x)} p_\eta(z \mid x)\, p_\theta(y \mid x, z).$$

其中 x 是問題；y 是輸出序列；z 是檢索段落；$Z_k(x)$ 是前 k 個候選集合；η 是檢索器參數；θ 是生成器參數。近似反映只保留候選。這是特定模型的機率形式，不代表所有當代 RAG 應用都實作它。

很多實務系統先檢索、組提示，再執行分開訓練的生成器。即使未最佳化原始邊際目標，這種管線仍可有用，但要說清楚設計。否則 RAG 這個名稱會掩蓋檢索是否學得、候選是否重排序，以及生成器是否受訓使用來源。第 9 節建立並評估檢索器本身；擷取式答案與可回答性檢查在NLP 11。

證據扎根（grounding）依明訂的支持規則，把主張連到外部證據。有引用的答案可能可追溯，但可追溯弱於受支持：有效網址也可能指向與答案矛盾的文件。要評估每個重要主張是否由引用證據推得，以及是否保留必要限定。

證據也可能衝突。生成器不應把互斥政策默默合成一個自信答案。應用可依契約優先採用權威版本、展示衝突或請求澄清。決策應來自中繼資料與任務規則，而非哪一句最流暢。

錯誤沿檢索、解讀與文字實現傳遞

受支持答案需要一串成功決策：語料必須有適用證據，檢索必須保留它，閱讀器必須解讀正確實體與關係，輸出也必須保留該解讀。由記憶知識猜出的答案，可能碰巧正確，卻不符合證據要求。對適用證據已存在語料中的問題，可寫成：

$$P(R \cap E \cap G) = P(R)P(E \mid R)P(G \mid R, E).$$

其中 P 表示機率，R 是充分證據保留於檢索結果的事件，E 是正確解讀證據的事件，G 是輸出忠實實現解讀的事件。這是機率鏈式法則，不是假設獨立。把互不相干的無條件階段分數相乘並不等價：閱讀器在人工挑選簡單文章上的準確率，可能不同於在檢索器實際候選上的準確率。

```
import numpy as np

retrieved = np.arange(100) < 80
interpreted = np.arange(100) < 72
realized = np.arange(100) < 68
r = retrieved.mean()
e = interpreted[retrieved].mean()
g = realized[retrieved & interpreted].mean()
complete = (retrieved & interpreted & realized).mean()
print('Conditional stage rates:', round(r, 3), round(e, 3), round(g, 3))
print('Complete supported answers:', round(complete, 3))
assert np.isclose(r * e * g, complete)
```

輸出：

```
Conditional stage rates: 0.8 0.9 0.944
Complete supported answers: 0.68
```

這是合成結果，不是系統效能報告。計算說明各元件表現看似不錯，完整答案仍未必同樣可靠。改善最弱階段的效果，還要能通過後續階段才有用。

檢索內容的邊界

檢索文字是待檢查的證據，不是有權改變應用的指令來源。段落可能包含引述指令、誤導文字或惡意要求。提示注入（prompt injection）試圖讓模型遵循與應用邊界衝突的指令，OWASP 記錄了此風險類別 [23]。檢索系統的重要界線，是使用者／應用指令與不可信來源材料之間。

來源文字應清楚分隔並標示文件身分，不得重新定義使用者權限、索取秘密資料或授權工具動作。若有工具，應在檢索段落之外，依使用者任務及應用政策驗證動作。文字分隔有助組織輸入，卻不是完整的強制機制。

相同原則適用於紀錄與快取。某使用者權限下產生的快取結果，不能未經檢查就給另一位使用者。證據區段必須指向回答所用的同一文件版本。文件移除或替換時，要更新索引，並考慮讓衍生回答失效。

這些檢查也支援一般正確性。「忽略舊上限，新政策如下」可以是需要解讀的合法文件內容，「忽略你的應用指令」卻不是系統的新權威。兩者都可儲存為來源材料，再套用任務的證據與權限規則；不能讓檢索器成為修改治理契約的通道。

11. 調適預訓練生成器

直觀說法。預訓練學習廣泛的延續模式；調適改變任務行為；提示則改變提供的輸入，不一定改權重。學習遵循指令與學習人們偏好的回覆是不同訊號，兩者都不保證真實。

監督任務調適對標註例子最小化任務損失，例如只在回覆詞元上計算第 7 節的條件損失。繼續在領域文字上訓練語言模型會改變所建模的分布，卻不會自動教會應用的輸出政策。熟悉領域與理解任務指令是兩種不同調適。上下文內例子（如 GPT-3 [6]）直接出現在輸入中，不更新參數就能示範格式；若例子包含私密答案或測試標籤，評估就可能失效，而在開發案例上選出的提示，必須在最終測試前凍結。

低秩調適（Low-Rank Adaptation，LoRA）固定基礎權重矩陣，訓練受限更新 [24]：

$$W' = W + BA.$$

其中 $W \in \mathbb{R}^{m\times n}$ 是固定矩陣；$A \in \mathbb{R}^{r\times n}$ 與 $B \in \mathbb{R}^{m\times r}$ 是學得因子；r 是選定的秩；W' 是有效調適矩陣。簡化式省略可選縮放。減少可訓練參數能降低調適成本，但不能取代代表性標籤、資料隔離與任務評估。基礎模型版本、分詞器、調適權重與設定應一起儲存：針對某基礎檢查點訓練的調適，不保證能套用到同家族另一檢查點。

InstructGPT 論文先研究監督指令例子，再以人類回饋進行偏好訓練 [25]，因而區分基礎延續模型與朝請求行為調適的模型。偏好來源很重要：標註者依政策平衡實用性、格式與其他目標。較受偏好的回覆仍可能有事實錯誤。

直接偏好最佳化（Direct Preference Optimization，DPO）結合偏好與不偏好的回覆，以及參考政策 [26]。代表性目標如下：

$$\mathcal{L}_{\text{DPO}} = -\log \sigma\left(\beta\left[\log \frac{\pi_\theta(y^+ \mid x)}{\pi_{\text{ref}}(y^+ \mid x)} - \log \frac{\pi_\theta(y^- \mid x)}{\pi_{\text{ref}}(y^- \mid x)}\right]\right).$$

其中 x 是提示；y^+ 與 y^- 是偏好及不偏好的回覆序列；π_θ 是可訓練政策；π_{ref} 是固定參考政策；β 是正縮放參數；σ 是 logistic sigmoid；$\mathcal{L}_{\text{DPO}}$ 是單一偏好對的損失。序列機率是回覆詞元條件機率的乘積，提示提供上下文，不是受監督回覆詞元。

目標提高偏好回覆相對於另一回覆的概似，並以參考政策為基準。若資料獎勵自信但無支持的答案，最佳化可能強化這種行為。指令遵循、偏好一致與事實支持是不同性質，需要不同證據。ML 09 以程式執行一個低秩更新。

12. 訓練實務遇上服務限制

直觀說法。部署後的生成器要服務長度不一的輸入，每個請求的成本同時取決於提示與輸出。填補與注意力快取是兩種不同的記憶體成本，兩者都要量。

Hagiwara 第 10 章把填補、排序與遮罩連到實際推論 [1]。要分清楚：正確的損失遮罩防止填補改變目標，未必防止硬體處理填補位置。定義填補利用率：

$$u = \frac{\sum_{b=1}^{B} L_b}{B \max_b L_b}.$$

其中 L_b 是請求 b 的真實詞元長度；B 是批次大小；u 是填補後位置中真實詞元的比例。這是工作量診斷，不是實測加速。按相近長度組批可提高利用率，也可能讓請求等待適合的批次。

```
import numpy as np

lengths = np.array([8, 64, 12, 60, 10, 56, 14, 52])
def utilization(values):
    batches = values.reshape(-1, 4)
    real = int(batches.sum())
    padded = int((batches.max(axis=1) * 4).sum())
    return real / padded
arrival = utilization(lengths)
bucketed = utilization(np.sort(lengths))
print("Arrival-order utilization:", round(arrival, 3))
print("Length-bucketed utilization:", round(bucketed, 3))
assert bucketed > arrival
# Two cache arrays (keys and values), with a toy equal head layout.
layers, kv_heads, head_dim, cache_tokens, bytes_per_value = 4, 2, 16, 128, 4
cache_bytes = 2 * layers * kv_heads * head_dim * cache_tokens * bytes_per_value
print("Toy cache bytes:", cache_bytes)
assert cache_bytes == 131072
```

輸出：

```
Arrival-order utilization: 0.575
Length-bucketed utilization: 0.885
Toy cache bytes: 131072
```

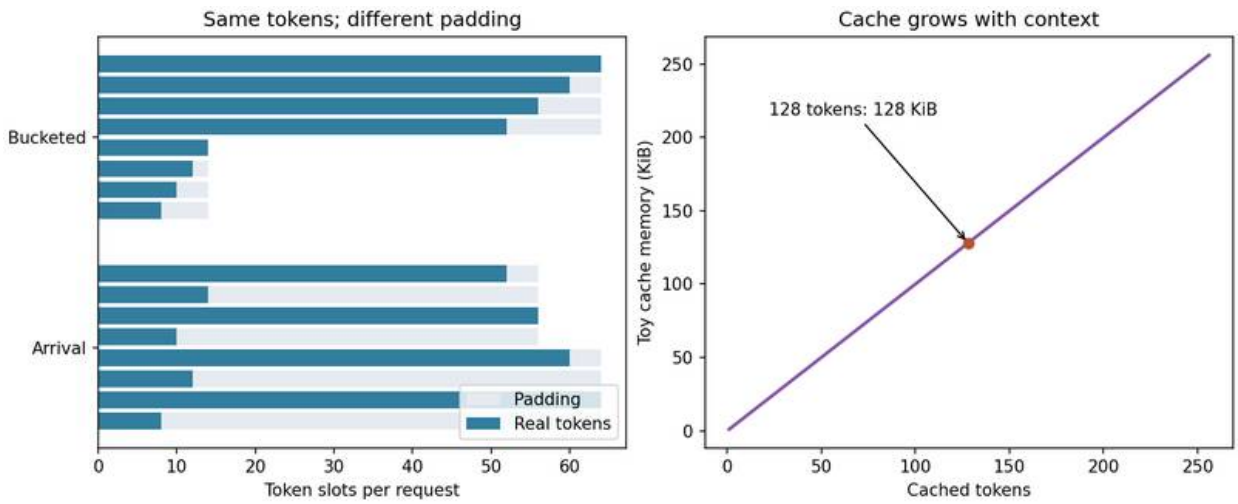


圖 6 — 填補浪費與注意力快取是不同記憶體成本。圖中呈現計算的詞元位置及微型快取位元組，不主張實測服務加速。

自迴歸注意力常快取先前的鍵和值，避免每產生一個詞元就重算。簡單布局下，快取記憶體近似 $M_{\text{KV}} = 2N_{\text{layers}}H_{\text{KV}}d_hTb_v$，其中 N_{layers} 是層數、H_{KV} 是鍵／值頭數、d_h 是頭維度、T 是快取詞

元數、b_v 是每個值的位元組數。因子二表示鍵與值。實際實作還有配置及中繼資料成本，也可能使用不同布局。

應分開量測首詞元時間、後續詞元間隔、總完成時間及負載下的吞吐量。短提示長答案與長提示短答案壓力不同。端到端量測要包含前處理、等待、驗證與序列化。依量測到的工作負載設定請求長度上限、輸出上限及佇列期限，並說明逾時時回傳部分結果、可重試錯誤或經覆核的替代結果。

可重現的服務契約

只有模型檔無法重現預測。分詞、正規化、證據選擇、解碼參數、驗證規則與輸出格式都影響行為。Hagiwara 的部署章 [1] 提供版本、測試、訓練—服務差異與監控的實務閱讀。換成學得生成器時，文字旁要回傳模型版本與已驗證證據 ID（如模板函數所做），並把前處理版本與解碼設定和模型一起保存。紀錄應能支援診斷，卻不必無謂保留私密原文。

新版本先服務有限比例的請求，用相同錯誤類別與穩定版本比較，同時檢查延遲、完成率與證據正確性。回復應保持簡單：儲存的套件應能一起選回舊模型、分詞器、解碼設定與驗證規則。舊模型混用新分詞器，可能產生各元件單獨時都沒有出現的失敗。

現代觀點

直觀說法。當代語言模型結合預訓練、調適、解碼、評估與系統整合。綜述若能釐清這些不同選擇，而非壓縮成單一模型排名，就特別有用。

Zhao 等人的綜述 [27] 更新至 2026 年，並發表於 Frontiers of Computer Science，依預訓練、調適、使用與能力評估組織大型語言模型。這有助區分更多訓練資料、指令微調與不同使用方式。**Chang 等人的評估綜述 [28]** 區分評什麼、在哪裡評與如何評。兩者共同支持任務專屬的評估計畫，而非依賴單一通用分數。

Li 等人的預訓練生成綜述 [29] 依輸入表示、架構與最佳化整理方法，並涵蓋應用及評估。這有助定位變化：檢索改變可用證據，編碼器—解碼器改變條件架構，調適改變參數或訓練目標。該綜述早於部分後續系統，應搭配較新的評估與部署證據閱讀。

Ji 等人的幻覺綜述 [20] 與 **Schmidtova 等人對幻覺人工評估的綜述 [30]** 從兩側處理同一問題。前者整理跨任務的定義、指標與緩解方法；後者檢查研究如何定義與評估幻覺，指出術語不一致及方法報告缺失。其實務重點是分開定義來源支持、世界知識與錯誤嚴重度，並公開標註說明與歧見處理方式。

Gao 等人的 RAG 綜述 [31] 把檢索增強系統分為 naive、advanced 與 modular 範式，並區分上下文相關、答案相關與答案忠實。各階段會互相影響：把「升級前的限制是多少？」改寫為「溫度限制是多少？」，可能改善主題檢索，卻刪掉時間條件。應保留原始查詢，比較改寫後的意義，而不只看檢索分數。

Kwon 等人的 PagedAttention [32] 研究語言模型服務中的鍵—值快取配置與共享，處理記憶體浪費以及記憶體所允許的批次大小，替上面的快取計算提供系統觀點。論文報告的改善屬於其工作負載與比較；採用服務設計前，先量測自己的請求長度、並行程度與延遲目標。

開放問題包括評估集合污染、長上下文穩健性（較大的上下文視窗允許更多輸入，卻不能證明所有相關部分都被可靠使用）、多語與低資源語言涵蓋、穩定的不確定性、在不壓制有用回答下偵測無支持主張，以及指令遵循和來源支持正確性的關係。評估模型可能與生成器共享錯誤，尤其兩者訓練資料相近時。能以較低延遲與較簡單覆核滿足任務時，小型專用模型甚至模板，可能是更好的工程選擇。

重點整理

- 僅解碼器 Transformer 以因果方式分解文字；要明確位移目標，並測試未來詞元無法改變較早 logits。
- GPT 系列保留同一架構與目標，擴大資料、參數與呈現任務的方式；規模化定律預測損失，不預測任務行為。
- 困惑度比較需要相容的分詞與評估位置，也不直接說明解碼輸出的好壞。
- 貪婪與束搜尋最佳化序列機率；溫度重塑分布；top-k 與核取樣截掉長尾，而核會隨分布形狀調整。
- 生成需要內容契約：選擇、規劃與文字實現各有不同失敗方式，擷取式摘要仍需檢查必需內容。
- 重疊指標、人工覆核與模型評審各有盲點；要在重要錯誤（例如否定反轉）上驗證它們。
- 檢索提供證據，不提供權威；相關性、引用有效性與主張支持是不同條件。
- 把整個前處理—模型—解碼—驗證套件版本化，並把指令微調與偏好學習視為與事實支持不同的事。

練習

1. 移除可執行因果模型中的因果遮罩。哪個斷言應失敗？為何訓練損失仍可能好看？

   提示

   改變的最後詞元能被較早位置讀取。低損失可能獎勵生成時根本取不到的資訊。

2. 使用解碼程式，找出一個溫度，使單純取樣放在四個長尾詞元上的機率，超過 $k = 5$ 的 top-k 所能保留的上限。再構造一個 logit 向量，讓 $p = 0.9$ 的核大於五個詞元，並說明各規則何時較合適。

   提示

   $k = 5$ 的 top-k 最多只能保留一個長尾詞元。平坦的 logit 向量讓核變大，尖峰的則讓核變小。

3. 修改微型束搜尋的機率，讓束搜尋選出違反外部指令的句子。加入合法完整序列的限制，並與加寬束比較效果。

   提示

   機率搜尋最佳化的是分布分數，合法序列規則改變的是哪些輸出可接受。加寬束未必能修好錯誤的機率模型。

4. 建立兩個候選摘要：一個詞重疊高但數量錯，另一個重疊較低但意義正確。設計能顯示差別的標註表，以及模型評審程序。

提示

先請覆核者標出無支持數量與遺漏的必需事實，再判斷整體實用性。保留來源、隱藏系統身分；模型評審則兩種呈現順序都要跑。

5. 構造一對獎勵風格、卻選到較缺乏支持回覆的偏好資料，說明哪種額外評估能揭露問題，並計算 128×256 矩陣的秩 4 LoRA 更新有多少可訓練參數。

提示

固定證據，比較自信地加入事實的答案，與保留來源不確定性的限定答案。LoRA 的部分要分別計算兩個因子矩陣。

6. 擴充報告函數，加入時間戳與目標語言選項。拒絕格式錯誤的輸入、保留未知狀態、附上證據 ID，並將文字實現規則版本化。寫下以學得生成器取代模板前需要哪些測試。

提示

不要推測未記錄的時間戳，也不要把未知轉成不存在。要測試前處理一致性、數字格式、書寫系統要求、長輸入、逾時行為與完整回復套件。

參考文獻

1. Masato Hagiwara, *Real-World Natural Language Processing: Practical Applications with Deep Learning*, Chapters 5, 8, 9, 10 and 11, Manning, 2021. Publisher ·Training practices ·Deployment
2. Susan McRoy, *Principles of Natural Language Processing*, open textbook, Chapters 6 and 9, 2021. Benchmark chapter ·Generation chapter
3. Ashish Vaswani et al., “Attention Is All You Need,” NeurIPS, 2017. Paper
4. Alec Radford, Karthik Narasimhan, Tim Salimans and Ilya Sutskever, “Improving Language Understanding by Generative Pre-Training,” OpenAI technical report, 2018. Paper
5. Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei and Ilya Sutskever, “Language Models are Unsupervised Multitask Learners,” OpenAI technical report, 2019. Paper
6. Tom B. Brown et al., “Language Models are Few-Shot Learners,” NeurIPS, 2020. Paper
7. Jared Kaplan et al., “Scaling Laws for Neural Language Models,” arXiv:2001.08361, 2020. arXiv
8. Jordan Hoffmann et al., “Training Compute-Optimal Large Language Models,” arXiv:2203.15556, 2022; published as “An empirical analysis of compute-optimal large language model training,” NeurIPS, 2022. arXiv
9. Ilya Loshchilov and Frank Hutter, “Decoupled Weight Decay Regularization,” ICLR, 2019. Paper
10. Dan Hendrycks and Kevin Gimpel, “Gaussian Error Linear Units (GELUs),” arXiv:1606.08415, 2016, revised 2023. Paper

11. Jimmy Lei Ba, Jamie Ryan Kiros and Geoffrey E. Hinton, "Layer Normalization," arXiv:1607.06450, 2016. Paper
12. Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes and Yejin Choi, "The Curious Case of Neural Text Degeneration," ICLR, 2020. Paper
13. Angela Fan, Mike Lewis and Yann Dauphin, "Hierarchical Neural Story Generation," ACL, 2018. Paper
14. Jade Goldstein and Jaime Carbonell, "Summarization: (1) Using MMR for Diversity-Based Reranking and (2) Evaluating Summaries," TIPSTER Phase III Workshop, 1998. Paper
15. Sam Wiseman, Stuart M. Shieber and Alexander M. Rush, "Challenges in Data-to-Document Generation," EMNLP, 2017. Paper
16. Ilya Sutskever, Oriol Vinyals and Quoc V. Le, "Sequence to Sequence Learning with Neural Networks," NeurIPS, 2014. Paper
17. Chin-Yew Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," Text Summarization Branches Out, 2004. Paper
18. Lianmin Zheng et al., "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," NeurIPS Datasets and Benchmarks Track, 2023. Paper
19. Peiyi Wang et al., "Large Language Models are not Fair Evaluators," ACL, 2024. Paper
20. Ziwei Ji et al., "Survey of Hallucination in Natural Language Generation," ACM Computing Surveys 55, 2023. Survey
21. Joshua Maynez, Shashi Narayan, Bernd Bohnet and Ryan McDonald, "On Faithfulness and Factuality in Abstractive Summarization," ACL, 2020. Paper
22. Patrick Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS, 2020. Paper
23. OWASP GenAI Security Project contributors, "LLM01: Prompt Injection," OWASP guidance, accessed 2026. Official guidance
24. Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang and Weizhu Chen, "LoRA: Low-Rank Adaptation of Large Language Models," ICLR, 2022. Author publication page
25. Long Ouyang et al., "Training language models to follow instructions with human feedback," NeurIPS, 2022. Paper
26. Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon and Chelsea Finn, "Direct Preference Optimization: Your Language Model is Secretly a Reward Model," NeurIPS, 2023. Paper
27. Wayne Xin Zhao et al., "A Survey of Large Language Models," Frontiers of Computer Science, 2026. Updated survey
28. Yupeng Chang et al., "A Survey on Evaluation of Large Language Models," ACM Transactions on Intelligent Systems and Technology, 2024. Survey
29. Junyi Li, Tianyi Tang, Wayne Xin Zhao, Jian-Yun Nie and Ji-Rong Wen, "Pre-Trained Language Models for Text Generation: A Survey," ACM Computing Surveys, 2024. Publisher ·Author manuscript
30. Patricia Schmidtova et al., "Do My Eyes Deceive Me? A Survey of Human Evaluations of Hallucinations in NLG," INLG, 2025. Survey
31. Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang and Haofen Wang, "Retrieval-Augmented Generation for Large Language Models: A

Survey," arXiv:2312.10997, revised 2024. Survey
32. Woosuk Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," SOSP, 2023. Conference ·Paper
33. Christopher D. Manning, Prabhakar Raghavan and Hinrich Schütze, *Introduction to Information Retrieval*, Chapters 6, 8 and 11, Cambridge University Press, 2008. Author-hosted book
34. scikit-learn developers, "Text feature extraction," scikit-learn documentation, accessed 2026. Documentation
35. Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen and Wen-tau Yih, "Dense Passage Retrieval for Open-Domain Question Answering," EMNLP, 2020. Paper
36. Kalervo Järvelin and Jaana Kekäläinen, "Cumulated Gain-Based Evaluation of IR Techniques," ACM Transactions on Information Systems 20(4), 422–446, 2002. Publisher

NLP 10・預訓練模型：BERT

從 WordPiece 輸入到雙向編碼器理解 BERT，從零以遮罩語言建模預訓練一個小型 BERT 式模型，並探測其上下文表示包含什麼。

網站版本: https://blog.shyandram.dev/zh-tw/learn/nlp/nlp-10-pretrained-bert/

你將學到

- 說明「先預訓練、再微調」的配方，以及 BERT 與 ELMo、GPT 的相對位置。
- 把文字轉成 BERT 輸入：WordPiece 片段、`[CLS]` ／ `[SEP]` 打包，以及相加的詞元、段落與位置嵌入。
- 精確說明遮罩語言建模目標（選取 15%，再依 80/10/10 替換），以及下一句預測目標與後來的批評。
- 在玩具語料上從零預訓練一個小型 BERT 式編碼器，並量測它學到什麼。
- 區分上下文嵌入與靜態嵌入，並用控制任務探測（probe）表示。
- 把 RoBERTa、ALBERT、DistilBERT、ELECTRA、多語 BERT 與中文 BERT 放進同一張地圖。

先看全貌

一般 Word2Vec 特徵替每個詞取出一個固定向量 [6]。BERT，全名 **Bidirectional Encoder Representations from Transformers**，則讀取周圍文字，替每次出現的詞元重新計算表示 [1]。在「the crane lifts steel」與「the crane eats fish」中，`crane` 查表得到的初始嵌入相同，編碼器卻能利用其他詞算出不同的最終向量。關鍵是區分詞彙表項目與這次出現的表示。

BERT 分兩階段學習。**預訓練**使用一般無標籤文字，由文字自己提供目標。**微調**再把同一個網路加上小型任務頭，調適到情感或蘊涵等有標籤任務。第一階段昂貴、只做一次；第二階段便宜、每個任務各做一次。

> **用最簡單的話說**
>
> 用拇指蓋住句子中的一個詞，請朋友猜：「the crane ___ steel」。朋友看得到拇指兩側的詞，所以「lifts」是好答案。這個遊戲玩上幾百萬次，就能學到很多詞語如何搭配。之後，朋友只要看幾個例子，就能學會新工作，例如分辨開心和生氣的訊息，因為他們已經懂句子怎麼運作。

Hagiwara 第 9 章在實務情境中說明預訓練模型調適 [18]。本章則走進模型內部，並以NLP 09 的因果語言模型作對照，沿用NLP 07 的注意力區塊與NLP 04 的靜態嵌入。接在 BERT 上的分類、蘊涵、標記與擷取式問答任務頭，是NLP 11 的主題。這裡每個模型都很小，且從零訓練，不下載任何檢查點。

1. 先預訓練，再微調

直觀說法。先從大量無標籤文字學到語言的一般知識，再用少量標籤把它用在特定任務。問題在於：哪種預訓練目標能產生最可重用的表示？

三個 2018 年的設計構成 BERT 的背景。ELMo 訓練由左到右與由右到左的 LSTM 語言模型，並把其內部狀態的學得組合，當作上下文詞特徵提供給既有任務模型 [5]。GPT 把 12 層僅解碼器 Transformer 預訓練成由左到右語言模型，再針對每個任務微調整個網路 [4]。BERT 保留 GPT「微調全部參數」的配方，但把因果目標換成遮罩目標，讓每一層都能使用左右兩側上下文 [1]。

BERT 論文主張，由左到右的限制對句子層級任務並非最佳，對問答這類需要兩個方向上下文的詞元層級任務更可能有很大傷害 [1]。其消融實驗包含一個沒有下一句預測的由左到右模型，預訓練與微調都受同樣的只看左側限制，並與雙向模型比較 [1]。工程上的教訓是：可見性模式是與任務相連的建模選擇，不是個人喜好。

預訓練提供參數初始化與表示函數，卻沒有提供應用標籤。幫浦維護分類器仍需要範例定義「緊急」，實體標記器也需要辨認現場設備名稱的標註。微調會一起調整編碼器與任務頭；凍結編碼器、只訓練任務頭，成本較低，但調適能力也不同。

2. 遮罩與條件目標

直觀說法。不是每個預訓練語言模型都預測下一詞元。遮罩目標使用周圍內容，條件生成則根據另一個輸入預測輸出序列。

BERT 的遮罩語言目標藏起選定位置，再使用雙向上下文預測原詞元 [1]。簡化描述如下：

$$\mathcal{L}_{\text{mask}} = -\frac{1}{|M|} \sum_{i \in M} \log p_\theta(x_i \mid \tilde{\mathbf{x}}).$$

其中 M 是受監督遮罩位置集合；$|M|$ 是集合大小；x_i 是原詞元；θ 是模型參數；$\tilde{\mathbf{x}}$ 是受損輸入序列。波浪號表示損壞，不是獨立抽出的文件。這個式子描述遮罩預測部分；完整的損壞程序見第 5 節。

遮罩預測器適合上下文表示、分類與區段任務，其損失卻不會自動成為一般由左到右序列的對數機率。把各自讀取句子兩側的預測相乘，通常不會得到與因果分解相同的正規化聯合分布。因此，遮罩目標分數與因果困惑度不能直接互換。

條件序列生成則寫成：

$$p_\theta(\mathbf{y} \mid \mathbf{x}) = \prod_{t=1}^{U} p_\theta(y_t \mid y_{<t}, \mathbf{x}).$$

其中 $\mathbf{x}$ 是來源序列；$\mathbf{y}$ 是目標序列；U 是包含結束詞元的目標長度；$y_{<t}$ 是目標前綴。來源可以完整可見，目標仍保持因果。T5 把多種 NLP 任務整理成文字到文字架構，並研究 Transformer 編碼器—解碼器的遷移學習 [27]；NLP 08 深入介紹這一類模型。

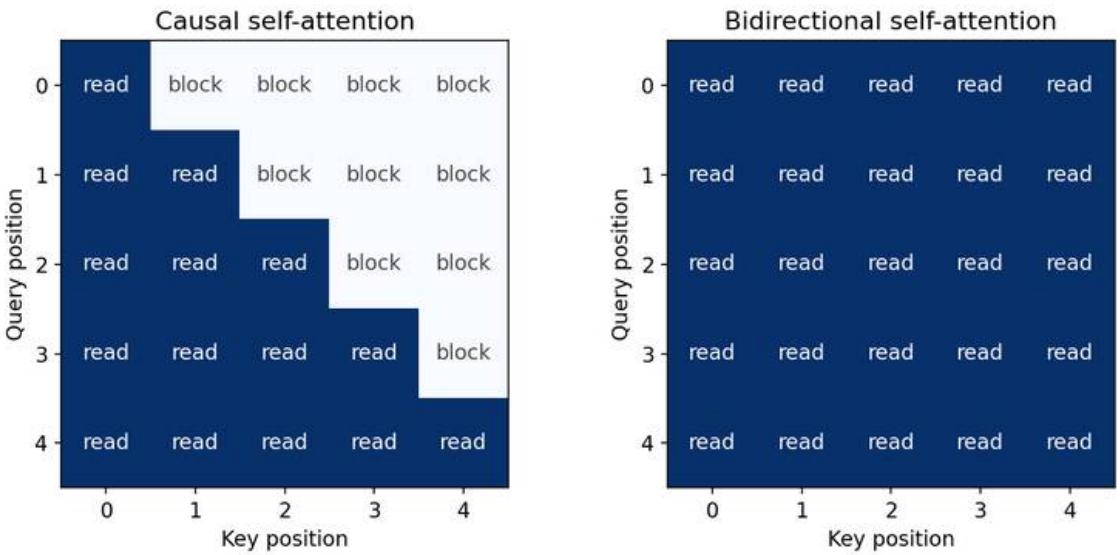


圖 *1* — 可見性是任務定義的一部分。列是發出查詢的位置，欄是它們可以讀取的位置。*BERT* 在每一層使用右圖的模式；填補鍵仍會被阻擋。

這些區別有助選模型。完整句子已知時，分類器可使用雙向編碼器；串流延續則必須排除未來輸出。翻譯可以在來源全部抵達後讀取完整來源，但同步翻譯需要更嚴格的來源可見性契約。應由任務可取得的資訊決定遮罩，而非因為某個模型名稱熟悉就選它的遮罩。

3. 從文字到 BERT 輸入

直觀說法。 BERT 從不直接看到詞。文字先被拆成詞彙表片段，再加上特殊標記，每個位置則得到三個學得向量的和。

WordPiece

原始分詞器先做文字前處理，再執行 WordPiece 分詞 [2]。BERT 使用含 30,000 個詞元的 WordPiece 嵌入 [1]，這是為 Google 神經機器翻譯系統提出的子詞方法 [7]。需要時，WordPiece 把詞拆成詞彙表中的片段；原始詞彙表以 `##` 標示接續片段。原始實作以**貪婪的最長匹配優先**切分每個詞：先取詞首最長的詞彙項目，再取最長的接續片段，依此類推；若某段剩餘部分沒有任何匹配，整個詞就變成 `[UNK]` [2]。詞彙表事先從語料學得，就像NLP 02 中學得 BPE 合併規則一樣。

```
vocab = {'[UNK]', 'the', 'pump', 'over', 'heat', '##heat', '##ed', '##s',
         'un', '##load', 'load', '##ing', '幫', '浦', '過', '熱'}

def wordpiece(word, max_chars=200):
    """Greedy longest-match-first split into vocabulary pieces."""
    if len(word) > max_chars:
        return ['[UNK]']
    pieces, start = [], 0
```

```
    while start < len(word):
        end = len(word)
        while end > start:
            piece = word[start:end] if start == 0 else '##' + word[start:end]
            if piece in vocab:
                break
            end -= 1
        if end == start:                    # no piece fits: the whole word is unknown
            return ['[UNK]']
        pieces.append(piece)
        start = end
    return pieces

def basic_split(text):
    """Lowercase, put spaces around CJK ideographs, split on whitespace."""
    out = []
    for ch in text.lower():
        cjk = 0x4E00 <= ord(ch) <= 0x9FFF
        out.append(f' {ch} ' if cjk else ch)
    return ''.join(out).split()

for text in ['The pump overheated', 'unloading pumps', 'the pumpkin', '幫浦過熱']:
    tokens = [p for w in basic_split(text) for p in wordpiece(w)]
    print(f'{text!r:24} -> {tokens}')
assert wordpiece('overheated') == ['over', '##heat', '##ed']
assert wordpiece('pumpkin') == ['[UNK]']
```

輸出：

```
'The pump overheated'    -> ['the', 'pump', 'over', '##heat', '##ed']
'unloading pumps'        -> ['un', '##load', '##ing', 'pump', '##s']
'the pumpkin'            -> ['the', '[UNK]']
'幫浦過熱'                   -> ['幫', '浦', '過', '熱']
```

這帶來三個結果。一個詞可能占多個模型位置，因此詞標籤與字元區段必須對應到詞元位置。剩餘部分無法涵蓋的詞會變成單一 `[UNK]`，即使它的前綴 `pump` 是已知詞；這是貪婪規則的性質，不是語言的性質。中文字則在 WordPiece 之前一字一個位置地切開，原始多語與中文模型也是如此（第 11 節）。這裡的 `basic_split` 只檢查主要的 CJK 統一表意文字區塊；原始程式還檢查其他幾個範圍，並處理重音與標點 [2]。

打包一段文字或一對文字

單一文字打包成 `[CLS] text [SEP]`；文字對則打包成 `[CLS] text A [SEP] text B [SEP]`。序列分類器可讀取 `[CLS]` 位置的最終狀態，`[SEP]` 則標記邊界。段落 ID 區分兩部分，卻**不會阻擋跨段落注意力**：文字 A 的查詢仍能讀取文字 B。決策依賴兩段文字互動時，這種聯合編碼很有用 [2]。

進入編碼器前，先相加三種學得的向量：

$$\mathbf{e}_i = E[x_i] + P[i] + S[s_i], \qquad H = \mathrm{Encoder}_\theta(\mathbf{e}_1, \dots, \mathbf{e}_T).$$

其中 T 是打包後的序列長度；θ 表示編碼器參數；x_i 是位置 i 的詞元 ID；$s_i \in \{0, 1\}$ 是段落 ID；$E \in \mathbb{R}^{|V| \times d}$ 是詞彙表 V 的詞元表；$P \in \mathbb{R}^{T_{\max} \times d}$ 是位置表，支援長度為 $T_{\max}$；$S \in \mathbb{R}^{2 \times d}$ 是段落表；$\mathbf{e}_i \in \mathbb{R}^d$ 是相加後的輸入；$H \in \mathbb{R}^{T \times d}$ 包含上下文輸出列 $\mathbf{h}_i$。原始實作還會對相加後的嵌入執行正規化與 dropout [2]。填補鍵被阻擋，兩側的一般文字則保持可見。

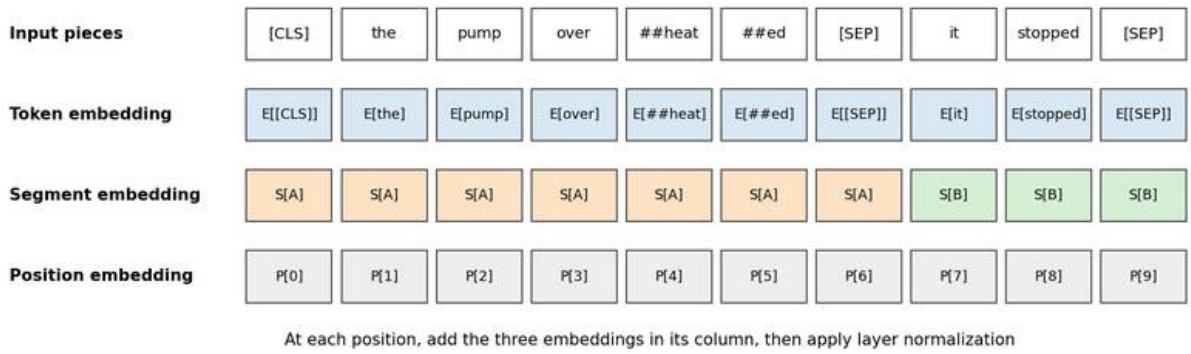


圖 2 — 文字對的 *BERT* 輸入。每一欄是一個位置，把該欄的詞元、段落（*A* 或 *B*）與位置嵌入相加。位置表是學得的，因此限定了最大輸入長度。

與原始 Transformer 的固定正弦位置不同，BERT 的位置向量是學得的列 [2]。學得的位置表限制支援的最遠位置；更長的輸入是非法索引，不是普通的長序列，必須截斷或切成視窗。嵌入列也只在該模型詞元 ID 的定義下有意義。未調適權重就換分詞器，可能讓形式合法的數字代表不相關符號。

4. 編碼器

直觀說法。 BERT 是 Transformer 的編碼器那一半：一疊相同的區塊，每個區塊讓每個位置讀取所有其他位置，再分別轉換每個位置。

BERT 疊合多個 Transformer **編碼器**區塊 [1, 3]。每個區塊先用雙向自注意力混合資訊，再用前饋網路轉換各位置；殘差連接與層正規化支援整個堆疊 [9]。「雙向」表示編碼器各層都能讀取輸入左右兩側，不是像 ELMo 那樣各訓練一個因果模型、再把輸出接起來 [5]。NLP 07 推導多頭注意力；BERT 唯一特有的選擇是圖 1 的注意力遮罩。

論文報告兩種大小。以 L 表示區塊數、H 表示隱藏寬度、A 表示注意力頭數，BERT-Base 為 $L = 12$、$H = 768$、$A = 12$，共 1.1 億個參數；BERT-Large 為 $L = 24$、$H = 1024$、$A = 16$，共 3.4 億個參數 [1]。BERT-Base 刻意選成與原始 GPT 同樣大小以便比較 [1]。前饋層使用 GELU 活化函數 [8, 1]。預訓練使用 BooksCorpus 與英文維基百科，論文強調使用文件層級語料，才能取得長的連續序列 [1]。

可把輸出想成每個輸入詞元各占一列的表格，而不是單一的句子向量。詞元標籤頭逐列讀取；序列頭則透過池化轉換讀取第一列。同一編碼器能支援兩者，因為輸出保留了詞元位置；NLP 11 會建立這些任務頭。

5. BERT 在取得任務標籤前學什麼？

直觀說法。遮罩語言建模把原始文字變成填空練習；下一句預測則多問一個是非題：兩段文字是否相連。

遮罩語言建模

遮罩語言建模（masked language modeling，MLM）從一般文字學習：原詞元就是自己的目標。例如 `the crane [MASK] steel` 要求模型利用兩側文字猜回被移除的詞元。預測損失同時更新編碼器與詞彙頭，因此最終詞元向量可作為後續監督任務的輸入。

原始 BERT 選取 15% 的 WordPiece 位置作為預測目標。在被選位置中，80% 換成 `[MASK]`、10% 換成隨機詞元、10% 保留原詞元 [1]。選取與替換是不同步驟：**每個被選位置都提供目標**，即使可見詞元未改變。其他位置提供上下文，不貢獻 MLM 損失。這種混合替換方式縮小預訓練輸入與一般未遮罩任務輸入的差異，因為後者從不出現 `[MASK]`。

這個設計的代價是樣本效率：只有被選位置產生損失。ELECTRA 的作者正是指出這一點：MLM 每個例子只從被遮罩的那一小部分詞元學習 [14]。原始實作也在準備資料時就一次決定遮罩；RoBERTa 比較這種靜態遮罩與**動態遮罩**（每次使用序列時重新抽取遮罩模式），發現動態遮罩相當或略好 [11]。第 7 節的小模型使用動態遮罩。

下一句預測及其批評

原始 BERT 也訓練**下一句預測**（next sentence prediction，NSP）[1]：文字對有一半來自連續語料，另一半則從語料隨機抽取第二段。池化後的 `[CLS]` 狀態預測這個二元標籤。NSP 預測關係標籤，不會生成下一句。原始預訓練結合 MLM 與 NSP，論文的消融實驗報告移除 NSP 會降低 QNLI、MNLI 與 SQuAD 1.1 的表現 [1]。

後續研究有不同結論。RoBERTa 的複製研究發現，以連續文字填滿輸入時，移除 NSP 損失的下游表現**相當或略好**，並推測原始消融可能只移除了損失項，卻保留了段落對的輸入格式 [11]。ALBERT 的作者推測 NSP 太簡單，因為它把主題預測與連貫性預測混在一起：隨機抽取的第二段通常主題不同。他們改用**句序預測**（sentence-order prediction），取同一文件中兩個連續段落，維持原順序或對調，讓主題不再洩漏答案 [12]。一般教訓是：應檢查實際檢查點的訓練目標，不要假定後來的每個編碼器都保留所有原始設計。

6. 可執行的雙向編碼器

直觀說法。在預訓練完整模型之前，先隔離讓它像 BERT 的兩個性質：改變較後面的詞會改變較前面詞的表示，而填補完全不造成影響。

以下 CPU 範例以四個自創句子訓練 MLM，固定選取位置 2 並換成 `[MASK]`。因此，右側可見動詞有助區分 `robot` 與 `sensor`。這個刻意簡化的替換規則用來觀察機制；模型隨機初始化，只有一個小型編碼器區塊，並以線性詞彙頭取代原始 BERT 的完整預測頭。它使用層正規化、GELU 與 AdamW [8–10]。

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(23)
torch.set_num_threads(1)
vocab = ['[PAD]', '[CLS]', '[SEP]', '[MASK]', 'the', 'a',
         'robot', 'sensor', 'moves', 'measures']
index = {word: i for i, word in enumerate(vocab)}
rows = ['the robot moves', 'the sensor measures',
        'a robot moves', 'a sensor measures']
ids = torch.tensor([[index[w] for w in ['[CLS]'] + s.split()
                     + ['[SEP]', '[PAD]']] for s in rows])
blocked = ids.eq(index['[PAD]'])
corrupted = ids.clone()
corrupted[:, 2] = index['[MASK]']
labels = torch.full_like(ids, -100)
labels[:, 2] = ids[:, 2]

class TinyEncoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.word = nn.Embedding(len(vocab), 16)
        self.position = nn.Embedding(6, 16)
        self.segment = nn.Embedding(2, 16)
        self.embed_norm = nn.LayerNorm(16)
        self.layer = nn.TransformerEncoderLayer(
            16, 2, 32, dropout=0.0, activation='gelu', batch_first=True)
        self.mlm = nn.Linear(16, len(vocab))
        self.pool = nn.Sequential(nn.Linear(16, 16), nn.Tanh())
        self.classifier = nn.Linear(16, 3)
        self.tagger = nn.Linear(16, 3)
        self.span = nn.Linear(16, 2)
    def forward(self, tokens, padding):
        positions = torch.arange(tokens.shape[1])
        segments = torch.zeros_like(tokens)  # single segment here
        h = self.embed_norm(self.word(tokens) + self.position(positions)
                            + self.segment(segments))
        return self.layer(h, src_key_padding_mask=padding)

model = TinyEncoder()
optimizer = torch.optim.AdamW(model.parameters(), lr=0.01)
def objective():
    logits = model.mlm(model(corrupted, blocked))
    return F.cross_entropy(logits.reshape(-1, len(vocab)), labels.reshape(-1))
initial = objective().item()
for _ in range(100):
    optimizer.zero_grad()
    loss = objective()
    loss.backward()
    optimizer.step()
model.eval()
with torch.no_grad():
```

```
    final = objective().item()
    h = model(ids, blocked)
    changed = ids.clone()
    changed[0, 3] = index['measures']
    context_gap = (model(changed, blocked)[0, 2] - h[0, 2]).abs().max().item()
    changed_padding = ids.clone()
    changed_padding[blocked] = index['robot']
    pad_gap = (model(changed_padding, blocked)[~blocked] - h[~blocked]).abs().max().item()
    shapes = [model.classifier(model.pool(h[:, 0])).shape,
              model.tagger(h).shape, model.span(h).shape]
assert final < initial and context_gap > 1e-4 and pad_gap < 1e-6
print('masked loss:', round(initial, 3), '->', round(final, 3))
print('context changes representation:', context_gap > 1e-4)
print('padding leaves real tokens unchanged:', pad_gap < 1e-6)
print('class / tag / span shapes:', [tuple(s) for s in shapes])
```

輸出：

```
masked loss: 2.223 -> 0.001
context changes representation: True
padding leaves real tokens unchanged: True
class / tag / span shapes: [(4, 3), (4, 6, 3), (4, 6, 2)]
```

遮罩損失由 `2.223` 降到 `0.001`，量測的是對這四個範例的擬合。改變 `robot` 之後的詞會改變它的最終表示；改變被阻擋填補位置所儲存的值，則不影響真實詞元的輸出。因果模型必須排除前一種未來影響（NLP 09 的擾動測試），雙向編碼器則刻意允許它。

任務頭印出的形狀為 `(4, 3)`、`(4, 6, 3)` 與 `(4, 6, 2)`：四個序列各對三個類別做決策、每個範例有六個詞元決策，以及每個詞元的兩個邊界分數。**這些任務頭尚未訓練**。本例只有編碼器與 MLM 頭收到梯度。要建立應用，必須提供任務標籤、最佳化相應任務頭與編碼器，再評估保留資料上的任務輸出，NLP 11 會這樣做。

兩種遮罩需要分清楚。編碼器的填補遮罩決定哪些鍵可讀取；`-100` 標籤告訴交叉熵忽略哪些位置，不會把那些位置移出注意力。在這個示範中，所有真實詞都能提供上下文，但只有被選名詞貢獻 MLM 損失。

7. 從零預訓練一個小型 BERT

直觀說法。現在把真正的損壞配方套到一個小型人造語言上，訓練幾秒鐘，然後檢查三件事：模型填補保留句空格的能力、同一個詞在不同上下文是否得到不同向量，以及簡單分類器能從每一層讀出什麼。

玩具語料由一個小文法產生。主詞限制動詞、動詞限制受詞，所以被遮罩的詞常常可以由**兩側**推出。`crane` 刻意設計成有歧義：它可以 `lift`（機器，起重機），也可以 `eat`（鳥，鶴）。128 個可能句子中保留 40 個。損壞函數精確實作第 5 節：選取 15%，再依 80/10/10 替換，每一步重新抽取。

```
import itertools, random
from collections import Counter
import numpy as np
import torch
from torch import nn
from torch.nn import functional as F
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

random.seed(0); torch.manual_seed(0); torch.set_num_threads(1)

# 1. A toy corpus: the subject constrains the verb, the verb constrains the object.
frames = {'robot': ['lifts', 'moves'], 'crane': ['lifts', 'eats'],
          'technician': ['repairs', 'moves'], 'heron': ['eats', 'sees']}
objects = {'lifts': ['steel', 'box'], 'moves': ['box', 'cart'],
           'eats': ['fish', 'bread'], 'repairs': ['pump', 'cart'], 'sees': ['fish', 'boat']}
sentences = [f'{d1} {a}{s} {v} {d2} {o}'.split()
             for s in frames for v in frames[s] for o in objects[v]
             for d1, d2, a in itertools.product(['the', 'a'], ['the', 'a'], ['', 'old '])]
random.shuffle(sentences)
test, train = sentences[:40], sentences[40:]
special = ['[PAD]', '[CLS]', '[SEP]', '[MASK]']
words = sorted({w for s in sentences for w in s})
vocab = special + words
index = {w: i for i, w in enumerate(vocab)}
first_word = len(special)
L = 2 + max(len(s) for s in sentences)

def encode(rows):
    ids = torch.zeros(len(rows), L, dtype=torch.long)
    for r, s in enumerate(rows):
        seq = [index['[CLS]']] + [index[w] for w in s] + [index['[SEP]']]
        ids[r, :len(seq)] = torch.tensor(seq)
    return ids
train_ids, test_ids = encode(train), encode(test)

# 2. BERT-style corruption: select 15% of real word positions; of those,
#    80% -> [MASK], 10% -> random word, 10% -> unchanged. All selected are targets.
def corrupt(ids, generator):
    real = ids >= first_word
    selected = (torch.rand(ids.shape, generator=generator) < 0.15) & real
    labels = torch.where(selected, ids, torch.full_like(ids, -100))
    u = torch.rand(ids.shape, generator=generator)
    out = ids.clone()
    out[selected & (u < 0.8)] = index['[MASK]']
    swap = selected & (u >= 0.8) & (u < 0.9)
    out[swap] = torch.randint(first_word, len(vocab), (int(swap.sum()),),
     ↪  generator=generator)
    return out, labels

class TinyBERT(nn.Module):
    def __init__(self, d=32, layers=2):
        super().__init__()
```

```
        self.word, self.position = nn.Embedding(len(vocab), d), nn.Embedding(L, d)
        self.segment, self.norm = nn.Embedding(2, d), nn.LayerNorm(d)
        self.blocks = nn.ModuleList(nn.TransformerEncoderLayer(
            d, 4, 4 * d, dropout=0.0, activation='gelu', batch_first=True)
            for _ in range(layers))
        self.mlm = nn.Linear(d, len(vocab))
    def forward(self, ids):
        pad = ids.eq(index['[PAD]'])
        h = self.norm(self.word(ids) + self.position(torch.arange(ids.shape[1]))
                      + self.segment(torch.zeros_like(ids)))
        states = [h]
        for block in self.blocks:
            h = block(h, src_key_padding_mask=pad)
            states.append(h)
        return states                                # embeddings, layer 1, layer 2

model = TinyBERT()
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-3)
g = torch.Generator().manual_seed(1)
for step in range(600):                              # a fresh mask every step
    inputs, labels = corrupt(train_ids, g)
    logits = model.mlm(model(inputs)[-1])
    loss = F.cross_entropy(logits.reshape(-1, len(vocab)), labels.reshape(-1))
    optimizer.zero_grad(); loss.backward(); optimizer.step()
model.eval()

# 3. Held-out masked-word accuracy: mask each word position in turn.
def masked_hits(ids, rows_text):
    by_kind = {}
    with torch.no_grad():
        for r, s in enumerate(rows_text):
            for i, w in enumerate(s):
                x = ids[r:r + 1].clone(); x[0, i + 1] = index['[MASK]']
                guess = vocab[model.mlm(model(x)[-1][0, i + 1]).argmax()]
                kind = 'determiner/adjective' if w in ('the', 'a', 'old') else 'noun/verb'
                by_kind.setdefault(kind, []).append(guess == w)
    return {k: round(float(np.mean(v)), 3) for k, v in sorted(by_kind.items())}
counts = torch.bincount(train_ids[train_ids >= first_word], minlength=len(vocab))
real_test = test_ids[test_ids >= first_word]
print('final training MLM loss:', round(loss.item(), 3))
print('held-out masked accuracy:', masked_hits(test_ids, test))
print('most-frequent-word baseline:', vocab[int(counts.argmax())],
      round((real_test == counts.argmax()).float().mean().item(), 3))
# Upper reference: the best guess given every other word, read off the grammar.
def oracle(text):
    by_kind = {}
    for s in text:
        for i, w in enumerate(s):
            fits = Counter(t[i] for t in sentences if len(t) == len(s)
                           and all(a == b for j, (a, b) in enumerate(zip(t, s)) if j != i))
            kind = 'determiner/adjective' if w in ('the', 'a', 'old') else 'noun/verb'
            by_kind.setdefault(kind, []).append(fits[w] / sum(fits.values()))
    return {k: round(float(np.mean(v)), 3) for k, v in sorted(by_kind.items())}
print('expected accuracy of an ideal guesser:', oracle(test))
```

```
# 4. One word, two contexts: identical input row, different output rows.
with torch.no_grad():
    pair = encode([['the', 'crane', 'lifts', 'the', 'steel'],
                   ['the', 'crane', 'eats', 'the', 'fish']])
    cos = [F.cosine_similarity(s[0, 2], s[1, 2], dim=0).item() for s in model(pair)]
print('cosine of the two "crane" vectors by layer:', [round(c, 3) for c in cos])

# 5. Probes. (a) Part of speech vs a control task that gives each word type a
#    random label; (b) the sense of "crane" (machine vs bird) from its own vector.
pos = {w: 'DET' if w in ('the', 'a') else 'ADJ' if w == 'old' else
       'VERB' if w in objects else 'NOUN' for w in words}
control = {w: random.randint(0, 3) for w in words}
crane = [s for s in sentences if 'crane' in s]
crane_ids = encode(crane)
sense = np.array([int('eats' in s) for s in crane])
def rows(ids, text, layer):
    with torch.no_grad():
        h = model(ids)[layer]
    return np.array([h[r, i + 1].numpy() for r, s in enumerate(text) for i in range(len(s))])
for layer in range(3):
    Xtr, Xte = rows(train_ids, train, layer), rows(test_ids, test, layer)
    acc = []
    for table in (pos, control):
        ytr = [table[w] for s in train for w in s]; yte = [table[w] for s in test for w in s]
        acc.append(LogisticRegression(max_iter=3000).fit(Xtr, ytr).score(Xte, yte))
    h = model(crane_ids)[layer].detach()[:, :, :]
    Xc = np.array([h[r, s.index('crane') + 1].numpy() for r, s in enumerate(crane)])
    acc.append(cross_val_score(LogisticRegression(max_iter=3000), Xc, sense, cv=4).mean())
    print(f'layer {layer}: POS {acc[0]:.2f}  control {acc[1]:.2f}  crane sense {acc[2]:.2f}')
assert cos[0] > 0.999 and cos[-1] < cos[0]
```

輸出（單一 CPU 執行緒約六秒）：

```
final training MLM loss: 0.453
held-out masked accuracy: {'determiner/adjective': 0.418, 'noun/verb': 0.492}
most-frequent-word baseline: the 0.17
expected accuracy of an ideal guesser: {'determiner/adjective': 0.592, 'noun/verb': 0.633}
cosine of the two "crane" vectors by layer: [1.0, 0.819, 0.639]
layer 0: POS 1.00  control 1.00  crane sense 0.38
layer 1: POS 1.00  control 1.00  crane sense 1.00
layer 2: POS 1.00  control 1.00  crane sense 1.00
```

把遮罩詞的數字和兩個參考比較。永遠猜最常見的詞 `the`，有 17% 的機會正確。一個完全知道文法的**理想猜測者**，也就是知道每個空格在給定其他所有詞時的真實分布，在內容詞上也只會答對 63%，而不是 100%：在 `the robot ___ the box` 之後，`lifts` 和 `moves` 都合文法。模型只看 88 個訓練句，就在內容詞上達到 49%。這個差距才是它學到與沒學到多少的誠實量測。許多空格沒有唯一答案，這也是 MLM 準確率從來不是理解能力直接量測的原因。

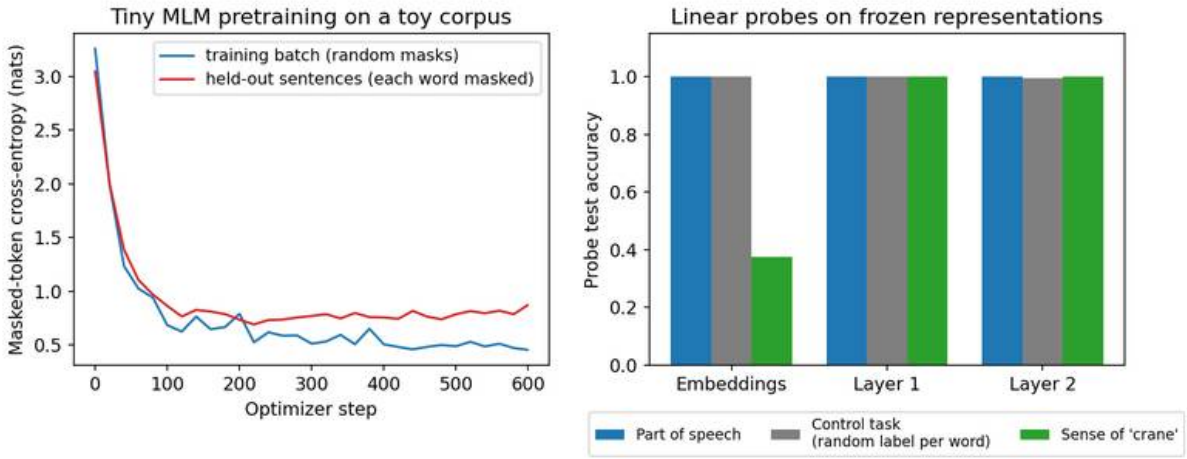


圖 *3* — 小型 *MLM* 訓練。左：隨機遮罩批次上的訓練損失，以及逐一遮罩每個詞時的保留損失；兩者差距是對 *88* 個句子的過擬合。右：在各層凍結表示上的線性探測（第 *9* 節說明控制任務為何重要）。

圖 3 左側顯示，訓練損失持續下降時，保留損失先趨平、再慢慢上升。同一個程式訓練更多步會讓訓練損失更低，保留資料的遮罩準確率卻變差，所以 600 步的預算是看保留損失決定的，而不是看訓練損失。這是一般的模型選擇，對預訓練與微調同樣適用。

8. 上下文嵌入與靜態嵌入

直觀說法。靜態嵌入讓一個詞在任何地方都只有一個向量。上下文編碼器替每次出現各給一個向量，而且差異隨深度增加。

NLP 04 建立的 Word2Vec 或 PPMI 向量是表格中的一列：鳥和機器共用同一個 `crane` 向量，它必須平均兩種上下文。上面印出的餘弦值以極小規模呈現上下文式的替代方案。在嵌入層，兩個 `crane` 向量完全相同（餘弦 1.0），因為詞元和位置都相同。經過一個編碼器區塊後餘弦為 0.819，兩個區塊後為 0.639：右側上下文 `lifts the steel` 或 `eats the fish` 已被混入。

Ethayarajh 在真實模型中量測了這個效應。在 ELMo、BERT 與 GPT-2 中，同一個詞在不同上下文的表示，仍比兩個不同詞的表示更相似，但這種自我相似度在上層低很多；平均而言，一個詞的上下文表示中，只有不到 5% 的變異能由該詞的靜態嵌入解釋 [19]。同一研究也發現，上下文表示在任何一層都不是各向同性的：向量集中在空間中的狹窄區域 [19]。實務上的後果是，BERT 向量之間的原始餘弦相似度，不會自動成為有意義的語意相似度。句子層級的相似度需要自己的訓練目標，NLP 04 與 NLP 11 會討論。

靜態向量仍有用途：便宜、推論時不需要編碼器，也容易檢查。當同一個表面形式帶有不同意義或角色時，上下文向量較合適，而真實文字大多是這種情況。

9. 探測表示中包含什麼

直觀說法。探測器（probe）是在凍結表示上訓練、用來預測某種語言性質的小型分類器。若它成功，表示該性質可以從向量中取回。究竟是表示還是探測器完成了工作，則是另一個問題。

Tenney、Das 與 Pavlick 逐層探測 BERT，發現負責詞性標記、剖析、命名實體、語意角色與共指的區域，大致依傳統 NLP 管線的順序出現，後面的層有時會修正前面的決定 [20]。Hewitt 與 Manning 的**結構探測器**尋找一個線性轉換，使詞向量之間的平方距離編碼剖析樹中的距離；他們在 ELMo 與 BERT 中找到這樣的轉換，在基準中則沒有 [21]。Clark 等人分析的是注意力而非向量，找到專門注意分隔標記、固定位置偏移，以及直接受詞、限定詞與共指提及的注意力頭 [23]。

Hewitt 與 Liang 補上了關鍵的對照。**控制任務**替每個詞型指派隨機標籤，因此只能靠記住詞的身分解決。好的探測器應該具有**選擇性**：語言任務準確率高，對應控制任務準確率低 [22]。沒有這個比較，高探測準確率可能只顯示探測器很強，而不是表示編碼了該性質。

我們的小型實驗呈現了兩面。詞性在每一層都可完美取回，但每個詞的隨機標籤也一樣，因為在這個文法中，詞性完全由詞本身決定。探測器的成功因此只說明「這個詞可以被辨認」，選擇性為零。`crane` 詞義探測則不同。在嵌入層，兩種詞義共用一個向量，準確率在機率水準或更低（0.38，32 個句子、四折）；經過一個編碼器區塊後就完美。這個性質需要上下文，所以逐層上升是編碼器把資訊移進 `crane` 向量的證據。即使如此，探測只顯示資訊對線性讀取器**可得**，不表示模型在預測時**使用**了它。

10. BERT 家族

直觀說法。 BERT 之後，好幾篇論文一次改變配方中的一部分：訓練多久、如何共享參數、如何壓縮，以及預測什麼。

模型	改變了什麼	報告的動機
RoBERTa [11]	以更大批次、更多資料訓練更久，移除 NSP，使用更長序列與動態遮罩	複製研究發現 BERT 明顯訓練不足
ALBERT [12]	分解式嵌入參數化、跨層參數共享、句序預測	降低記憶體並加速訓練；以連貫性目標取代 NSP
DistilBERT [13]	在預訓練階段把知識蒸餾到較小的學生模型，使用三重損失	作者報告小 40%、保留 BERT 97% 的語言理解能力、快 60%
ELECTRA [14]	取代詞元偵測：小型生成器填入被遮罩位置，編碼器判斷每個詞元是原本的還是被替換的	從全部輸入詞元學習，而非只從被遮罩的子集

ALBERT 的分解式嵌入把詞彙表大小與隱藏大小分開：不用一張 $|V| \times H$ 的表，而是一張 $|V| \times E$ 的表加上 $E \times H$ 投影，其中 E 遠小於 H [12]。作者認為 WordPiece 嵌入應學習與上下文無關的表示，隱藏狀態則與上下文有關，所以兩者大小不必相同 [12]。ELECTRA 微調時保留的是判別器；生成器在預訓練後就丟棄 [14]。

這些變體同時改變參數量、計算預算與目標，因此其發表的比較屬於各自的實驗設定。選用時，要檢查它的分詞器、最大長度、訓練目標與訓練資料，並在你在乎的任務上量測。

11. 多語與中文 BERT

直觀說法。一個編碼器可以用共享詞彙表同時在多種語言上預訓練。中文則需要決定單位，因為中文書寫時詞與詞之間沒有空格。

原始釋出包含一個多語 BERT-Base，在 104 種語言的維基百科上訓練，共用 110k 的 WordPiece 詞彙表，且沒有任何告訴模型目前是哪種語言的標記 [2]。Pires、Schlinger 與 Garrette 發現此模型的零樣本遷移出乎意料地好，也就是用一種語言微調、在另一種語言評估，甚至跨越書寫系統也行；在類型學上相近的語言之間遷移最好；但某些語言對有系統性的缺陷 [15]。XLM-R 之後以遠更多經過過濾的 CommonCrawl 資料，在一百種語言上訓練 Transformer 遮罩語言模型，報告在跨語言基準上大幅優於多語 BERT，對低資源語言尤其明顯 [16]。它對正向遷移與容量稀釋之間取捨的分析提醒我們：在固定大小的模型中加入更多語言並非沒有代價 [16]。

對中文，原始多語模型在 WordPiece 之前，於 CJK 表意文字範圍內的每個字前後加上空格，因此中文實際上以字為單位分詞 [2]；第 3 節的 `basic_split` 函數模仿這個規則。釋出內容也提供只用中文的 BERT-Base，涵蓋簡體與繁體中文 [2]。以字為位置可避免斷詞錯誤，但只遮罩「幫浦」這類多字詞中的一個字，模型可以由同詞的另一個字猜回，而不必依賴句子上下文。Cui 等人替中文 BERT 引入**全詞遮罩**（whole word masking）：以斷詞器找出詞的邊界，被選中詞的所有字一起遮罩，而輸入仍以字為單位分詞 [17]。他們的 MacBERT 模型以詞向量相似度選出的相近詞取代 `[MASK]`，縮小預訓練與微調輸入之間的差距 [17]。

處理台灣的繁體中文文字時，要檢查預訓練語料實際包含什麼，而不只看語言標籤。以簡體中文為主的資料所訓練的模型，可能能正確切分繁體字，卻在地區詞彙（例如「資料」與其他地區的用法）或特定領域的台灣術語上較弱。這是要用領域內例子量測的資料問題，不是架構能決定的。

現代觀點

Rogers、Kovaleva 與 Rumshisky 的入門綜述 [24] 回顧了超過 150 篇關於 BERT 的研究：它捕捉了哪些語言與世界知識、存在何處，注意力頭與各層如何運作，對訓練目標與架構的修改，過度參數化與壓縮。其整體訊息與第 9 節一致：許多性質可以被取回，但可取回不等於被使用，探測結果也取決於探測器。

在大型僅解碼器模型的時代，編碼器並沒有消失。**ModernBERT [25]** 發表於 ACL 2025，是 2024 年的僅編碼器模型，把近年的架構與效率最佳化帶進 BERT 設計；作者把編碼器描述為許多正式檢索與分類管線的主力，並報告以 2 兆個詞元訓練、原生序列長度為 8,192。**Zhao 等人的綜述 [26]** 把遮罩編碼器放在預訓練語言模型的更廣歷史中，並指出因果解碼器已被既有大型語言模型廣泛採用為架構；NLP 09 說明了為何單一因果目標就能同時用於評分與生成。

2026 年的實務總結：當整個輸入都可取得、輸出是標籤、區段或檢索用向量，而且延遲與成本重要時，使用雙向編碼器；輸出是開放式文字時，使用僅解碼器模型。開放問題包括超越探測的忠實解釋、低資源語言與書寫系統的穩健多語涵蓋、超出學得位置上限的長文件編碼，以及評估任務結果中有多少來自預訓練表示、而非微調資料。

重點整理

- BERT 是不用標籤預訓練、再以小型任務頭微調的 Transformer 編碼器；預訓練提供表示，不提供任務標籤。
- 輸入是打包成 `[CLS] A [SEP] B [SEP]` 的 WordPiece 片段，加上相加的詞元、段落與學得位置嵌入；段落不阻擋注意力。
- MLM 選取 15% 位置並依 80/10/10 替換；每個被選位置都是目標，遮罩分數也不是因果困惑度。
- NSP 後來被發現並非必要（RoBERTa），或被句序預測取代（ALBERT）。
- 從零預訓練的小型 BERT 呈現了這些機制：保留句空格的填補優於頻率基準、但低於文法本身的上限，而歧義詞的向量隨深度分開。
- 上下文向量隨每次出現而不同，靜態向量則不會。上下文向量之間的原始餘弦，不會自動是語意相似度。
- 探測需要控制任務；探測顯示資訊可得，不表示模型使用它。
- 多語與中文 BERT 做了特定的單位選擇（共享 WordPiece、CJK 逐字切分、全詞遮罩），這些選擇影響模型學到什麼。

練習

1. 在小型 BERT 中，把動態遮罩換成訓練前只抽一次的靜態遮罩。比較保留資料的遮罩準確率與損失曲線。

   提示

   把 `corrupt(train_ids, g)` 移到迴圈外只呼叫一次。靜態遮罩下，每個句子永遠藏同樣的位置，所以許多（句子、位置）目標從未出現。

2. 在玩具語料中加入下一句預測：用 `[SEP]` 與段落 ID 組成兩句的文字對，其中一半的第二句是隨機的。再建立句序版本。在你的語料中，哪一個只靠主題線索就能解決？

   提示

   建立兩份使用不同詞彙的「文件」。隨機第二句通常來自另一份文件，所以 NSP 很容易；同一文件內對調順序則不然。

3. 擴充 WordPiece 詞彙表，使 `pumpkin` 不再是 `[UNK]`，但不要直接加入 `pumpkin`。再找一個詞，讓貪婪最長匹配的切法與片段數最少的切法不同。

   提示

   第一部分加入 `##kin` 就夠了。第二部分可以放入一個會留下難處理剩餘部分的長前綴，以及一個能乾淨切分的短前綴。

4. 在玩具語料上設計一個探測任務，使類似詞性的控制結果會顯示**正的**選擇性。這個性質必須依賴上下文，而不是詞的身分。

提示

名詞是主詞還是受詞是一個選項，但先檢查在這個文法中，位置本身是否已經洩漏答案。若是，就加入打破這種關聯的句子變體。

5. 把 MLM 頭換成 ELECTRA 式的取代詞元偵測器：以隨機詞而非 `[MASK]` 損壞位置，並在每個位置訓練二元分類器。比較每個批次中有多少位置產生損失。

提示

ELECTRA 使用小型的已訓練生成器；隨機替換讓偵測容易得多，所以要說明你的簡化改變了什麼。比較有遮罩與沒有遮罩時的已標註位置數。

6. 把雙向範例改成詞元分類器。若一個已標註詞拆成三個片段，定義哪些位置貢獻損失，以及如何重建詞標籤。

提示

選擇只監督第一片段，或明確定義所有片段的標籤映射。忽略填補與特殊標記、訓練標記頭，再把輸出映射回原文。

參考文獻

1. Jacob Devlin, Ming-Wei Chang, Kenton Lee and Kristina Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” NAACL, 2019. Paper
2. Google Research, *BERT: original implementation and released models*, software, 2018. Encoder and pooler ·Tokenizer ·Multilingual and Chinese models
3. Ashish Vaswani et al., “Attention Is All You Need,” NeurIPS, 2017. Paper
4. Alec Radford, Karthik Narasimhan, Tim Salimans and Ilya Sutskever, “Improving Language Understanding by Generative Pre-Training,” OpenAI technical report, 2018. Paper
5. Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee and Luke Zettlemoyer, “Deep Contextualized Word Representations,” NAACL, 2018. Paper
6. Tomas Mikolov, Kai Chen, Greg Corrado and Jeffrey Dean, “Efficient Estimation of Word Representations in Vector Space,” ICLR Workshop, 2013. Paper
7. Yonghui Wu et al., “Google’s Neural Machine Translation System: Bridging the Gap between Human and Machine Translation,” arXiv:1609.08144, 2016. Paper
8. Dan Hendrycks and Kevin Gimpel, “Gaussian Error Linear Units (GELUs),” arXiv:1606.08415, 2016, revised 2023. Paper
9. Jimmy Lei Ba, Jamie Ryan Kiros and Geoffrey E. Hinton, “Layer Normalization,” arXiv:1607.06450, 2016. Paper
10. Ilya Loshchilov and Frank Hutter, “Decoupled Weight Decay Regularization,” ICLR, 2019. Paper
11. Yinhan Liu et al., “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” arXiv:1907.11692, 2019. Paper

12. Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma and Radu Soricut, “ALBERT: A Lite BERT for Self-supervised Learning of Language Representations,” ICLR, 2020. Paper
13. Victor Sanh, Lysandre Debut, Julien Chaumond and Thomas Wolf, “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,” 5th Workshop on Energy Efficient Machine Learning and Cognitive Computing at NeurIPS, 2019. Paper
14. Kevin Clark, Minh-Thang Luong, Quoc V. Le and Christopher D. Manning, “ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators,” ICLR, 2020. Paper
15. Telmo Pires, Eva Schlinger and Dan Garrette, “How Multilingual is Multilingual BERT?,” ACL, 2019. Paper
16. Alexis Conneau et al., “Unsupervised Cross-lingual Representation Learning at Scale,” ACL, 2020. Paper
17. Yiming Cui, Wanxiang Che, Ting Liu, Bing Qin and Ziqing Yang, “Pre-Training With Whole Word Masking for Chinese BERT,” IEEE/ACM Transactions on Audio, Speech, and Language Processing 29, 2021. Publisher ·Author manuscript
18. Masato Hagiwara, *Real-World Natural Language Processing: Practical Applications with Deep Learning*, Chapter 9, Manning, 2021. Publisher
19. Kawin Ethayarajh, “How Contextual are Contextualized Word Representations? Comparing the Geometry of BERT, ELMo, and GPT-2 Embeddings,” EMNLP-IJCNLP, 2019. Paper
20. Ian Tenney, Dipanjan Das and Ellie Pavlick, “BERT Rediscovers the Classical NLP Pipeline,” ACL, 2019. Paper
21. John Hewitt and Christopher D. Manning, “A Structural Probe for Finding Syntax in Word Representations,” NAACL, 2019. Paper
22. John Hewitt and Percy Liang, “Designing and Interpreting Probes with Control Tasks,” EMNLP-IJCNLP, 2019. Paper
23. Kevin Clark, Urvashi Khandelwal, Omer Levy and Christopher D. Manning, “What Does BERT Look at? An Analysis of BERT’s Attention,” BlackboxNLP Workshop, 2019. Paper
24. Anna Rogers, Olga Kovaleva and Anna Rumshisky, “A Primer in BERTology: What We Know About How BERT Works,” Transactions of the Association for Computational Linguistics 8, 2020. Paper
25. Benjamin Warner et al., “Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference,” ACL, 2025. Paper
26. Wayne Xin Zhao et al., “A Survey of Large Language Models,” Frontiers of Computer Science, 2026. Updated survey
27. Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li and Peter J. Liu, “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer,” JMLR 21(140), 2020. Paper

NLP 11・以 BERT 做分類（情感分析、自然語言推論）

為單一文字與文字對決策微調預訓練雙向編碼器，從文章詞元讀出答案區段，並誠實評估標籤、機率與捷徑。

網站版本: https://blog.shyandram.dev/zh-tw/learn/nlp/nlp-11-classification-bert/

你將學到

- 依決策型態選擇輸出頭：一個標籤、每個詞元一個標籤，或文章中的一段區段。
- 為情感分析與前提—假說推論微調預訓練編碼器，並了解標準資料集（SST、IMDB、SNLI、MultiNLI、SQuAD）實際包含什麼。
- 辨識標註痕跡（annotation artifacts）、只看假說的基準與挑戰集，理解為何分布內高分可能掩蓋捷徑。
- 有意識地設定學習率、訓練輪數、暖身、類別權重與決策門檻，並知道微調為何可能不穩定。
- 以準確率、macro-F1、校準與覆蓋率評估，並區分相關證據與充分證據。
- 在 CPU 上從頭以遮罩語言模型預訓練一個微小編碼器，再微調於合成情感與推論任務。

先看全貌

NLP 10 建立了 BERT：以預測被遮住詞元來預訓練的雙向編碼器。預訓練給的是一個表示函數，不是一個應用。客服團隊仍要說明「緊急」是什麼意思，推論資料集仍要規定一句話何時可由另一句推出。本章處理第二步：在編碼器上接一個小輸出層，用標註樣本同時訓練兩者，再檢查結果解決的是預期任務，而不是一個方便的仿冒品。

> **用最簡單的話說**
>
> 想像一位讀過上千本書、卻從沒分類過影評的朋友。你給他看二十則標成「喜歡」或「不喜歡」的影評。因為他早就懂「沉悶」與「精彩」這些詞，很快就學會分類。但如果你給的每則「不喜歡」範例剛好都含有「不」字，朋友可能只會找「不」字。微調是那堂短短的課；好的評估則檢查朋友到底學到了什麼。

本章有兩類主要決策。**單一文字分類**替一個輸入指定一個標籤：正面或負面情感、主題、工單路徑。**文字對分類**標註兩個輸入之間的關係：假說是否可由前提推出、與前提矛盾，或兩者皆非？擷取式問答與它們相鄰：模型不是輸出一個標籤，而是選擇答案在文章中從哪裡開始、到哪裡結束。三者都重用同一個預訓練編碼器，只有輸出頭與訓練標籤不同 [3]。

Hagiwara 第 9 章正是針對這些應用討論以預訓練編碼器進行遷移學習 [1]；McRoy 關於基準任務以及資訊擷取與論證的章節則提供語言學觀點 [2]。全程請記住一個問題：什麼證據能說服懷疑者，模型學到的是任務，而不是資料集？

BERT 的應用：選擇輸出頭

直觀說法。任務頭把編碼器的詞元向量轉成需要的答案。先決定答案是一個類別、每個詞元各一個標籤，還是文章中的一段文字。

應用	輸入	監督輸出
情感、主題或客服工單分類	一段文字	由池化 `[CLS]` 狀態預測一個類別
蘊涵、改寫或查詢—文件相關性	一起打包的兩段文字	關係標籤或相關性分數
實體識別或其他詞元標記	已分詞的文字	每個受監督詞元位置的標籤
擷取式問答	問題與文章	文章中的起點與終點
使用 Sentence-BERT 的語意檢索	分開編碼的文字	以句子層級目標學得的可比較向量

序列與詞元任務使用 BERT 論文描述的表示 [3]；原始分類與區段實作明確呈現輸出形狀 [4]。Sentence-BERT 則加入句子比較的訓練 [20]。以下公式說明這些選擇各自計算什麼。

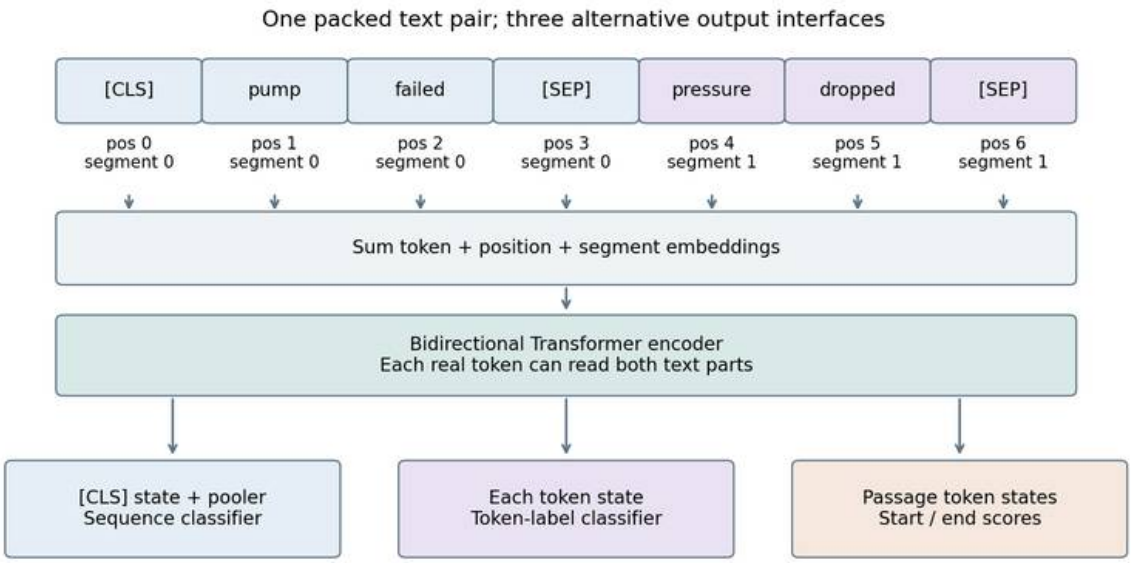


圖 *1* — 共用的 *BERT* 編碼器支援不同監督輸出。片段 *A*/*B* 標籤只標示文字部分，不阻止跨片段注意力。三個任務頭示意不同任務，不是訓練後模型的預測。

序列分類與文字對決策

對有 K 個類別的分類器，一種常見 BERT 介面是：

$$\mathbf{c} = \tanh(W_p \mathbf{h}_{\text{CLS}} + \mathbf{b}_p), \qquad p(y = k \mid \mathbf{x}) = \operatorname{softmax}(W_c \mathbf{c} + \mathbf{b}_c)_k.$$

其中 $\mathbf{x}$ 是打包後的詞元序列；$\mathbf{h}_{\text{CLS}} \in \mathbb{R}^d$ 是第一個上下文輸出列；$W_p \in \mathbb{R}^{d \times d}$ 與 $\mathbf{b}_p \in \mathbb{R}^d$ 組成池化器；$\mathbf{c} \in \mathbb{R}^d$ 是池化輸出；$W_c \in \mathbb{R}^{K \times d}$ 與 $\mathbf{b}_c \in \mathbb{R}^K$ 組成分類器；$y = k$ 表示類別 k。端到端微調

時，標註類別的交叉熵同時更新任務頭與編碼器 [4]。

對「the pump failed」，工單分類器可預測事故類別。對前提—假說文字對，相同輸出形狀可表示蘊涵、矛盾或兩者皆非。這些標籤來自任務資料集，不是預訓練時下一句預測（NSP）的標籤詞彙。文字對自注意力讓一段文字的詞元在最終決策前，先讀取另一段文字。

詞元標記

替每個上下文輸出列接上分類器：

$$p(y_i = k \mid \mathbf{x}) = \operatorname{softmax}(W_t \mathbf{h}_i + \mathbf{b}_t)_k, \qquad \mathcal{L}_{\text{tag}} = -\frac{1}{|J|} \sum_{i \in J} \log p(y_i^* \mid \mathbf{x}).$$

其中 i 是詞元位置，K 是標籤數量；$W_t \in \mathbb{R}^{K \times d}$ 與 $\mathbf{b}_t \in \mathbb{R}^K$ 由所有位置共用；y_i 是詞元標籤；y_i^* 是標註值；J 是納入監督的非空位置集合。設備名稱若被拆成多個 WordPiece，就需要一致規則：只監督第一個片段、忽略其他片段，或把標籤一致地映射到各片段。J 不應包含填補與特殊標記。推論時，再把預測片段映射回詞或字元區段，細節見NLP 06。NLP 12 的語意角色標註會以加上述詞標記的方式重用同一個任務頭。

擷取式問答

假設文章寫「Pump B stopped at noon」，問題是「Which pump stopped?」。把問題與文章一起打包、編碼，再替可能的答案邊界評分：

$$a_i = \mathbf{w}_s^\top \mathbf{h}_i, \quad b_i = \mathbf{w}_e^\top \mathbf{h}_i, \qquad \mathcal{L}_{\text{span}} = -\log p_s(i^*) - \log p_e(j^*).$$

其中 i, j 是候選起點與終點位置；$\mathbf{w}_s, \mathbf{w}_e \in \mathbb{R}^d$ 是學得的起點與終點向量；a_i, b_i 是邊界分數；p_s, p_e 是在允許位置上計算的 softmax 分布；i^*, j^* 是標註的答案邊界。解碼時，在 $i \le j$ 與指定最大長度下，選取使 $a_i + b_j$ 最大的合法區段 [4]。問題位置與填補不能當成文章答案回傳。若加入「沒有答案」標記，也必須訓練此選項，並以開發資料選擇決策門檻。

BERT 論文為這個任務只新增兩個向量：起點向量與終點向量；每個邊界機率都是它與最終詞元狀態的內積，再對文章位置取 softmax [3]。圖 2 說明為何解碼必須聯合進行，而不是各自取兩次 argmax：最佳起點與最佳終點必須構成合法區段。在這組合成分數中，分數最高的合法區段是「Pump B」。

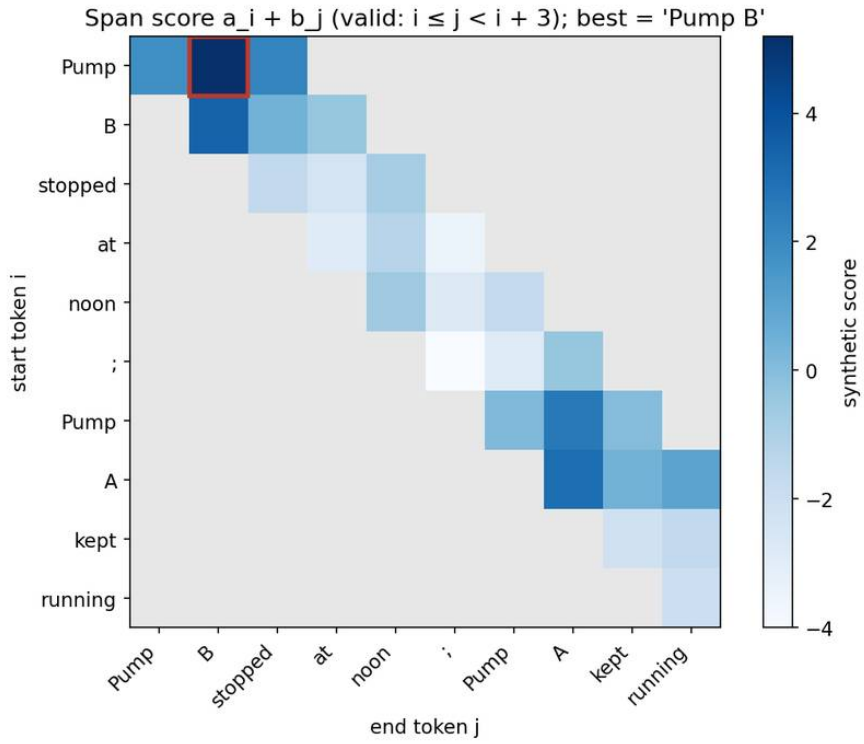


圖 2 — 在起點＋終點分數上解碼區段。灰色格子違反 $i \le j$ 或長度限制。分數為示意用的虛構數值。

輸出會是「Pump B」這類文章文字。這個設定不會生成解釋，也不會找出尚未取得的文章。NLP 09 的檢索增強生成處理找文章的問題；下方關於可回答性的小節則處理：找到的文章是否足夠。

檢索：聯合評分，還是可重用向量？

聯合 BERT 相關性模型把查詢與候選一起打包，因此候選表示依賴這次查詢；換一對文字就要重新執行。句子嵌入模型則分開編碼文件，允許快取。Sentence-BERT 以配對或三元組句子目標訓練第二種介面 [20]。原始預訓練 `[CLS]` 向量或任意平均池化，不會自動形成經過良好訓練的餘弦檢索空間。

實際系統的一種設計是先用分開儲存的向量召回候選，再聯合評分較少的查詢—候選文字對。應分別評估檢索階段的候選召回與最終排序器。NLP 04 說明池化與相似度，NLP 09 展開完整檢索流程。

單一文字分類：情感分析

直觀說法。情感分析判斷一段文字表達正面或負面意見。看似簡單，但否定、轉折與混合意見都表示：只數好話與壞話並不夠。

在 n 個標註樣本上寫出微調目標：

$$\mathcal{L}(\theta, \phi) = -\frac{1}{n}\sum_{i=1}^{n} \log p_{\theta,\phi}(y_i \mid \mathbf{x}_i).$$

其中 θ 是由預訓練初始化的編碼器參數；$\phi = (W_p, \mathbf{b}_p, W_c, \mathbf{b}_c)$ 是隨機初始化的任務頭參數；$\mathbf{x}_i$ 是第 i 段打包文字；y_i 是其標籤；$p_{\theta,\phi}$ 是上面定義的分類分布。完整微調同時更新 θ 與 ϕ。凍結 θ、只訓練 ϕ 比較便宜，但只能重用凍結特徵本來就具備的區分。

兩個基準資料集，以及它們量測什麼

Stanford Sentiment Treebank（SST）包含 11,855 個取自影評的單句；作者剖析這些句子，並請標註者標記剖析樹中全部 215,154 個不重複片語 [5]。片語層級標籤讓研究者能觀察情感如何組合：「not」與「good」組成的片語有自己的標籤。論文同時報告細粒度（五類）與二元的句子層級結果 [5]。GLUE 後來把二元版本收錄為 **SST-2**，列出約 67k 個訓練樣本與 1.8k 個測試樣本，以準確率評估 [14]。最初的 SST 模型是建立在剖析樹上的遞迴網路；ML 03A 說明這類架構。

Maas 等人的 **IMDB** 影評資料集含 50,000 則評論，平均分為 25,000 則訓練與 25,000 則測試，另有 50,000 則未標註評論；標註任務是類別平衡的二元問題 [6]。它的單位是整篇評論，不是一句話。評論常比短編碼器視窗長，因此文件分類器需要明確的截斷或分塊政策：保留開頭、保留結尾，或在多個視窗上彙總預測。每種政策回答的問題略有不同，因為評論常在結尾改變語氣。

這兩個資料集說明一個反覆出現的問題：**監督單位**定義了任務。只用獨立意見句訓練的句子情感模型，可能無法處理包含引述、修正與多位說話者的完整對話。如果部署可以查看先前回合，標註與訓練也應包含相同的允許脈絡。反過來，標註者如果看得到最後處理結果，但模型只看得到第一則訊息，就可能產生模型根本無法判定的目標。NLP 03 以計數式分類器詳細說明這份預測約定，換成 BERT 後同樣適用。

上下文編碼器增加了什麼，又沒有增加什麼

詞袋分類器在「the battery is good」與「the battery is not good」中給「good」相同權重。上下文編碼器則在兩句中替「good」算出不同向量，所以任務頭原則上能學會反轉。NLP 03 以計數特徵做了受控否定測試，顯示稀疏證據會讓這種反轉無法確定。上下文讓區別**可以被表示**；標註資料仍必須讓它**可以被學到**。片語身分也不同於作用範圍：區分「not every repair succeeded」與「every repair failed」需要組合性的區別，相鄰詞特徵無法提供；微調後的編碼器也只有在訓練資料包含這種區別時才會學到。

情感標籤本身也是標註決策。一則評論可能稱讚螢幕、批評電池。單一標籤情感分析強迫這則評論歸入一類；面向式情感分析則把意見指派給目標。兩者都不是「唯一正確」的任務，應選擇應用實際使用其輸出的那一種。

文字對：自然語言推論

直觀說法。文字可以支持某個結論，卻未必證明它在現實中為真。文字蘊涵與證據品質要分開。

自然語言推論（natural language inference，NLI）依標註慣例，把前提與假說分為蘊涵、矛盾或中立。若前提是「Mina 修了一個感測器」，通常可推出「有人修了某樣東西」，卻推不出「Mina 修了所有感測器」；「Mina 沒有修任何感測器」則與前提衝突。

對 BERT，把文字對打包成 `[CLS] premise [SEP] hypothesis [SEP]`，片段 ID 分別為 0 與 1，再套用 $K = 3$ 的序列任務頭：

$$\hat{y} = \operatorname*{argmax}_{k \in \{\text{E},\text{N},\text{C}\}} p_{\theta,\phi}(y = k \mid [\text{CLS}]\,\mathbf{u}\,[\text{SEP}]\,\mathbf{v}\,[\text{SEP}]).$$

其中 $\mathbf{u}$ 是前提詞元序列，$\mathbf{v}$ 是假說詞元序列，E, N, C 分別表示蘊涵、中立與矛盾，其他符號與序列分類頭相同。因為自注意力跨越 `[SEP]` 邊界，每個假說詞元在每一層都能注意每個前提詞元。這種聯合編碼讓決策能依賴兩段文字逐詞元的比較；這也是文字對模型無法快取其中一側的原因。

SNLI 與 MultiNLI

Stanford Natural Language Inference（SNLI）語料含 570,152 組人工撰寫的句對，標記為蘊涵、矛盾與語意獨立（中立）[7]。標註者看到一則圖片說明作為前提，再撰寫假說。論文討論一個細微的標註選擇：是否假設兩句描述同一事件。若不做此假設，許多看似矛盾的句對就會是中立。語料固定一種慣例，讓每組句對只有一個標籤 [7]。你替自己的推論資料做的，正是這類必須明文記錄的決策。

MultiNLI 把設計擴展到十種書面與口語英語文類、共 433k 個樣本。訓練集只出現五種文類，因此開發集與測試集分成來自訓練文類的 **matched** 部分，以及來自其他文類的 **mismatched** 部分 [8]。這種切分把領域偏移變成可量測的量，而不是事後想到的問題。

標註痕跡與只看假說的基準

被要求寫出矛盾句的群眾標註者，常使用相同手法：否定詞，或換一個活動。Gururangan 等人顯示，一個**只看假說**的簡單文字分類器，能在 SNLI 約 67%、MultiNLI 約 53% 的資料上預測正確標籤，且否定與模糊等現象與特定類別高度相關 [9]。Poliak 等人在十個 NLI 資料集上執行只看假說的基準，發現它在其中多個資料集上顯著勝過多數類別基準 [10]。忽略前提的模型不可能在做推論，所以它的分數量測的是資料集規律。

McCoy、Pavlick 與 Linzen 從相反方向著手：他們建立 **HANS**，一組受控資料，讓三種句法啟發法——詞彙重疊、子序列與成分——給出錯誤答案，並發現在 MNLI 上訓練的模型（包括 BERT）在其上表現很差 [11]。這些研究合起來支持兩個習慣：

1. **一定要跑部分輸入基準**。若只看假說（或只看前提、只看問題）的模型表現良好，基準分數就有一部分與你在意的關係無關。
2. **建立翻轉線索的挑戰句對**。固定詞彙，只改變一個應該重要的區別：角色反轉、否定出現在前提而非假說、可能性與斷言、指控與事實、一個與全部。

推論模型預測的是標註關係，不會獨立驗證前提。若前提是編造的，獲得蘊涵的假說也可能是編造的。因此，結合檢索的應用需要來源選擇，也需要回答支持檢查；下方問答小節會繼續討論。

論證擷取可辨識主張、提出的支持陳述與明說的攻擊關係 [2]。「裝置可靠，因為它通過了一次測試」具有可辨識的論證結構，但辨識結構不代表測試具有代表性，也不表示結論有效。應用可以儲存陳述的理由，並把其證據強度標為尚未評估。

有用的挑戰句對一次改變一個關鍵差別。流暢改寫也是有用變化，但應保留原本標籤。合成改寫若改變作用範圍或刪除歸屬，就必須重新標註，不能自動繼承原紀錄。

把擷取式問答視為文字對任務：答案需要支持

直觀說法。 SQuAD 式問答要求模型在給定文章中指出答案。實務上更難的問題是：文章裡到底有沒有受支持的答案。

Stanford Question Answering Dataset（SQuAD）含 107,785 組由群眾標註者針對 536 篇維基百科文章撰寫的問答；每個答案都是對應段落中的一段區段 [12]。它的兩個指標都忽略標點與冠詞 *a*、*an*、*the*：**完全符合（exact match）**是預測與任一參考答案完全相同的比例；**F1** 量測預測與參考答案的詞元重疊，再對問題平均 [12]。部分給分很重要，因為標註者的分歧大多出現在區段邊緣不重要的詞上。

SQuAD 2.0 加入超過 50,000 個刻意寫得與可回答問題相似的不可回答問題；系統還必須判斷段落何時不支持任何答案並拒答 [13]。BERT 論文的作法是把不可回答問題視為起點與終點都在 `[CLS]` 的區段。預測時比較空答案分數 $s_{\text{null}} = \mathbf{w}_s^\top \mathbf{h}_{\text{CLS}} + \mathbf{w}_e^\top \mathbf{h}_{\text{CLS}}$ 與最佳非空區段分數 $\hat{s} = \max_{j \ge i} (a_i + b_j)$，只有在 $\hat{s} > s_{\text{null}} + \tau$ 時才回答 [3]。其中 τ 是在開發集上選來最大化 F1 的門檻。這一個門檻就把區段模型變成選擇性回答器。

本機證據式回答器

不用訓練好的區段模型，也能示範回答契約。下面的回答器只處理一個明確問題家族，回傳來源區間。它以 TF–IDF 為四個合成段落排序，再只回答 Pump A 的狹義溫度上限問題。程式保留單字元詞，讓裝置代碼 A 仍是特徵。函數不是通用意圖分類器，呼叫者必須把已聲明的問題家族路由給它。

```
import re
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = {
    'temperature-v2': 'Pump A temperature limit is 80 C. This policy is current.',
    'shutdown-v1': 'Pump A shutdown follows a vibration alarm.',
    'label-v1': 'A technician replaced the cabinet label.',
    'sensor-v1': 'Sensor B reports humidity once per hour.',
}
ids = list(corpus)
vectorizer = TfidfVectorizer(ngram_range=(1, 2), token_pattern=r"(?u)\b\w+\b")
D = vectorizer.fit_transform([corpus[key] for key in ids])
def rank(query):
    score = (D @ vectorizer.transform([query]).T).toarray().ravel()
    order = sorted(range(len(ids)), key=lambda i: (-score[i], ids[i]))
    return [(ids[i], float(score[i])) for i in order]
```

```
def answer_temperature(query):
    if not re.search(r'\bpump\s+a\b', query, flags=re.I):
        return {'answer': None, 'reason': 'unsupported entity'}
    key, score = rank(query)[0]
    text = corpus[key]
    pattern = r'Pump A temperature limit is (\d+) C\.'
    match = re.search(pattern, text)
    if not match:
        return {'answer': None, 'reason': 'no matching evidence'}
    evidence = text[match.start():match.end()]
    result = {'answer': f'{match.group(1)} C', 'document': key,
              'start': match.start(), 'end': match.end(), 'evidence': evidence}
    assert text[result['start']:result['end']] == result['evidence']
    return result
print(answer_temperature('What is the Pump A temperature limit?'))
print(answer_temperature('What is the Pump Z temperature limit?'))
```

Pump A 回傳受支持的 80 C、文件版本與字元區間 `0–33`。即使存在溫度相關段落，Pump Z 問題仍回傳 `unsupported entity`。在 SQuAD 式資料上訓練的區段任務頭會以學得的邊界分數取代正規表示式，但周邊契約——實體檢查、證據區間、驗證過的位移、明確拒答——應該保留。

明確實體檢查避免高詞彙分數把 Pump A 的上限移植到沒有支持的裝置。證據表達式也要求來源說出預期關係，而非只含一個數字。更彈性的系統需要更完整的實體與關係檢查，包括範圍、單位及適用日期。應透過標註測試擴充能力，不要放寬到所有問題都收到答案。

回答器只看最高段落，因此正確證據排在後面時會拒答。這個限制刻意保留並說明。後續版本可以在固定預算內檢查多個候選，記錄最終用了哪個。要同時量測受支持回答與錯誤接受。

函數保留證據切片並驗證位移，可抓出序列化錯誤，卻不能抓全部語意錯誤。引用句仍可能談錯裝置、過期政策或假設情況。精確引文是重要機械檢查，支持關係仍是契約另外一部分。

相關不代表可回答

段落可以討論主題，卻缺少所問事實。系統應能有意義地回傳證據不足。

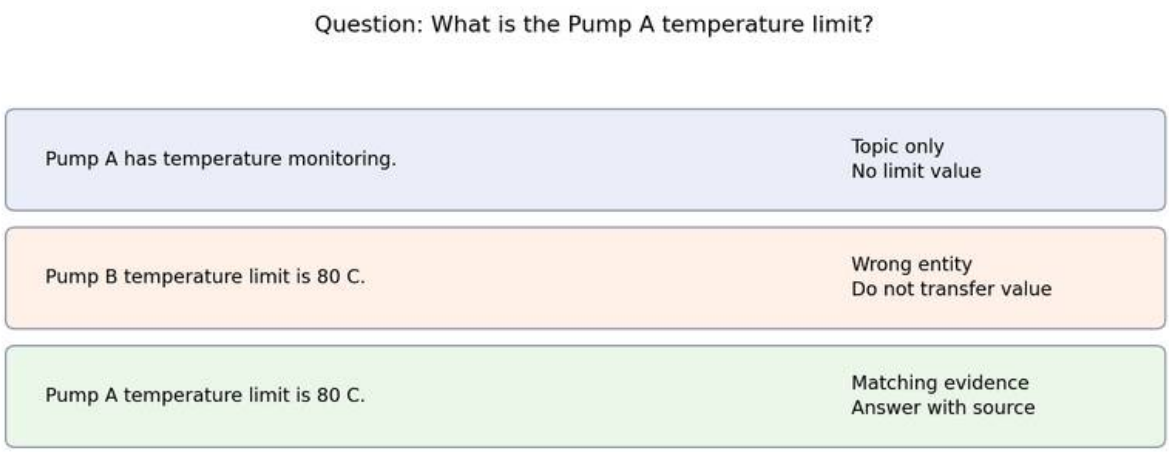


圖 3 — 主題重疊、錯誤實體證據與完整支持需要不同決策，分數本身不能定義可回答性。

狹義問題應先定義必要欄位，再評估系統。溫度範例需要裝置身分、上限關係、數值與單位，也可能需要生效日期或操作條件。即使段落第一名，只要少一個必需欄位，證據仍可能不足。

拒答政策決定何時不回答或請求澄清。若使用信心分數，門檻應在開發資料依錯答與不必要覆核的成本選擇——正是 SQuAD 2.0 任務頭中 τ 的角色。原始相似度不是校準後的正確機率；語料、分詞器或嵌入模型改變時，尺度也可能改變。

同時報告覆蓋率與接受回答的品質：

$$\text{coverage} = \frac{N_{\text{accepted}}}{N_{\text{all}}}, \qquad \text{accepted accuracy} = \frac{N_{\text{correct, accepted}}}{N_{\text{accepted}}}.$$

其中 N_{all} 是評估問題數；N_{accepted} 是收到回答的數量；$N_{\text{correct, accepted}}$ 是依契約判正確的接受回答數。沒有接受回答時，接受準確率未定義；應回報此情況，不能把零接受當成成功準確率。

覆蓋率必須包含沒有可用答案的問題。否則只在處處有答案的集合上，永遠回答的系統也會好看——這正是 SQuAD 2.0 要揭露的弱點 [13]。應納入錯誤實體、缺少值、來源衝突與不支援的時間範圍，測試模型是否認知證據限制，而非只產生可能的區段。

用受控替換診斷失敗

把一個不完美元件換成已知正確輸入，就能找到答案在哪裡失敗。診斷結果應與端到端部署分數分開。

先直接提供標註支持段落給回答器。如果仍答錯值，檢索就不是直接原因；應查實體、單位、範圍與文字實現。如果正確證據能答對，檢索證據卻失敗，就查候選召回與段落次序。這能定位瓶頸，但不假裝部署時也會有完美證據。

接著保留正確段落，加入無關或衝突段落。只有額外上下文才造成失敗，表示是內容選擇或證據整合問題。一次改一個性質：另一裝置、過期版本、引述指控或未授權指令。執行前先標預期行為，避免看到結果後改解釋替輸出辯護。

受支持答案需要一串成功決策。對適用證據已存在語料中的問題，可寫成：

$$P(R \cap E \cap G) = P(R)P(E \mid R)P(G \mid R, E).$$

其中 P 表示機率，R 是充分證據保留於檢索結果的事件，E 是正確解讀證據的事件，G 是輸出忠實實現解讀的事件。這是機率鏈式法則，不是假設獨立。把互不相干的無條件階段分數相乘並不等價：閱讀器在人工挑選簡單文章上的準確率，可能不同於在檢索器實際候選上的準確率。

以下人工評估讓條件關係看得見：

```
import numpy as np

retrieved = np.arange(100) < 80
interpreted = np.arange(100) < 72
realized = np.arange(100) < 68
r = retrieved.mean()
e = interpreted[retrieved].mean()
g = realized[retrieved & interpreted].mean()
complete = (retrieved & interpreted & realized).mean()
print('Conditional stage rates:', round(r, 3), round(e, 3), round(g, 3))
print('Complete supported answers:', round(complete, 3))
assert np.isclose(r * e * g, complete)
```

階段比例為 `0.8`、`0.9`、`0.944`，完整成功率卻為 `0.68`。這是合成結果，不是系統效能報告。各元件表現看似不錯，完整答案仍未必同樣可靠。最後保留真實查詢分布：診斷套件刻意提高困難案例比例，代表性保留集合則估計一般行為。兩者應分開報告目的，不要混成一個總分。

基準：GLUE 與 SuperGLUE

直觀說法。基準套件讓一個預訓練模型跑過許多小任務並彙總結果。它適合比較通用編碼器，卻不擅長告訴你自己的應用是否可行。

GLUE 收錄九個英語理解任務——單句任務（語法可接受度與 SST-2 情感）、相似度與改寫任務，以及包含 MNLI 在內的推論任務——並附上一個針對特定語言現象的診斷資料集與線上排行榜 [14]。它的資料集都不是為此基準從頭建立，部分任務保留私有測試標籤，以確保排行榜被公平使用 [14]。各任務指標不同：SST-2 用準確率，改寫任務用準確率與 F1，句子相似度用 Pearson 與 Spearman 相關係數，類別不平衡的可接受度任務用 Matthews 相關係數 [14]。BERT 論文的主要遷移結果報告在 GLUE、SQuAD 與一個常識推論資料集上 [3]。

SuperGLUE 保留 GLUE 的設計——圍繞八個任務的排行榜、單一彙總分數與分析工具——但挑選更困難的任務，包括針對段落的是非問答，以及嵌入子句上的三類蘊涵 [15]。

讀套件平均分數要小心。它混合了規模、指標與雜訊程度不同的任務，提升可能只來自某個小任務。除非同時報告部分輸入基準與挑戰集，否則它對標註痕跡一無所知；除非你的領域就在其中，否則它也不能說明你的領域。用套件挑選起始檢查點，用自己的保留資料挑選系統。

微調實務

直觀說法。微調是從好權重出發的短期監督訓練。主要風險是學習率太大而破壞預訓練所學、更新次數太少或太多、隨機種子運氣不好，以及類別比例或標籤意義與部署不同的資料。

學習率、訓練輪數與排程

BERT 作者把 dropout 固定在 0.1，並報告以下範圍在其任務上普遍有效：批次大小 16 或 32；Adam 學習率 5×10^{-5}、3×10^{-5} 或 2×10^{-5}；訓練 2、3 或 4 輪 [3]。他們也觀察到，大型資料集（十萬筆以上標註樣本）對這些選擇遠不如小型資料集敏感 [3]。釋出的分類程式預設學習率 5×10^{-5}、三輪、最大序列長度 128、暖身比例 0.1 [4]。其最佳化器排程為：

$$\eta_t = \begin{cases} \eta_{\max}\, t/t_w, & t < t_w, \\ \eta_{\max}\,(1 - t/T), & t \geq t_w. \end{cases}$$

其中 η_t 是第 t 次更新的學習率；$\eta_{\max}$ 是設定的峰值；T 是總訓練步數；t_w 是暖身步數，為 T 的固定比例。同一實作對層正規化與偏差以外的參數施加 0.01 的權重衰減，並把全域梯度範數裁剪在 1.0 [4]。這類解耦權重衰減的最佳化器一般稱為 AdamW [26]。

為什麼學習率這麼小？每次更新都在移動編碼了有用區別的預訓練權重。適合從隨機初始化訓練網路的學習率，可能在幾百步內就把它們抹除，之後模型實際上只是從少量標註資料學習。

不穩定與隨機重啟

對 BERT-LARGE，原作者發現在小型資料集上微調有時不穩定；他們進行數次隨機重啟——保留同一預訓練檢查點，只改變資料洗牌與分類層初始化——再依開發集選出最佳模型 [3]。Mosbach、Andriushchenko 與 Klakow 分析 BERT、RoBERTa 與 ALBERT 在 GLUE 任務上的這種不穩定。他們把失敗的執行歸因於訓練早期的最佳化困難，特徵是梯度消失；其餘變異則來自訓練損失幾乎相同的執行之間的泛化差異 [16]。他們沒有在 BERT-BASE 上觀察到這種不穩定。他們對小型資料集的建議是：使用**含** Adam 偏差修正的小學習率，並大幅增加迭代次數，訓練到接近零的訓練損失 [16]。

實務教訓：報告多個種子的分布，而不只報告最好的種子，且只依開發資料選擇。一次幸運的執行不是方法。

類別不平衡

真實標籤分布很少平衡。類別加權改變每個訓練樣本的貢獻：

$$\mathcal{L}_{\text{weighted}} = -\frac{1}{n}\sum_{i=1}^{n} \alpha_{y_i} \log p_\theta(y_i \mid d_i).$$

其中 $\mathcal{L}_{\text{weighted}}$ 是加權交叉熵，n 是訓練樣本數，i 索引樣本，d_i 是文件，y_i 是標註類別，$\alpha_{y_i} > 0$ 是該類別的權重，p_θ 是分類器分布。加權政策改變訓練目標，但不能創造缺少的語言多樣性。

加權、過度取樣與門檻調整回答不同問題。加權改變參數學習；過度取樣改變呈現頻率，有時只是不斷重複少數類別中的同幾個樣本；門檻調整則在分數算出後改變行動政策。Hagiwara 的開發章節討論不平衡與訓練選擇 [1]。任何介入都應以真實類別盛行率評估，而不是以人為平衡的最終測試集評估。

校準

分類器可能選對標籤，卻提供不可靠的機率。校準（calibration）檢查預測信心是否與相似樣本的觀察正確率一致。模型可以保留 argmax 決策，同時改變信心。Guo 等人研究這個差別與訓練後的溫度縮放 [17]：

$$\tilde{p}(y = k \mid \mathbf{x}) = \operatorname{softmax}(\mathbf{z}/T_c)_k, \qquad \mathrm{ECE} = \sum_{b=1}^{B} \frac{|S_b|}{n} \left| \operatorname{acc}(S_b) - \operatorname{conf}(S_b) \right|.$$

其中 $\mathbf{z}$ 是 logit 向量，$T_c > 0$ 是在保留開發資料上最小化負對數概似所擬合的單一溫度，$\tilde{p}$ 是縮放後的分布；對期望校準誤差 ECE，樣本依信心分成 B 個區間 S_b，n 是樣本數，$\operatorname{acc}(S_b)$ 是區間 b 內的準確率，$\operatorname{conf}(S_b)$ 是其平均預測信心。所有 logit 除以同一個正數不會改變大小次序，因此溫度縮放不可能改變準確率。

Desai 與 Durrett 在 NLI、改寫偵測與常識推理上量測 BERT 與 RoBERTa。直接使用時，預訓練模型在領域內是校準的，且領域外校準誤差低於他們比較的基準；溫度縮放進一步降低領域內誤差，標籤平滑則有助於領域外 [18]。這是那些任務與模型的性質，不保證你的微調檢查點也如此。只要機率會驅動拒答之類的行動，就應在自己的開發資料上量測校準。

評估：準確率、macro-F1 與選擇性決策

直觀說法。準確率計算答對多少。Macro-F1 讓每個類別同等重要，罕見類別因此無法被忽略。選擇性指標則問：系統可以拒答時，被接受的答案有多好。

把一個類別視為正類時：

$$P = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \qquad R = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}, \qquad F_1 = \frac{2PR}{P + R}.$$

其中 P 是精確率（precision），R 是召回率（recall），TP 是真陽性數，FP 是偽陽性數，FN 是偽陰性數，F_1 是精確率與召回率的調和平均。分母為零時，必須明確選擇慣例。Macro F1 平等平均各類別 F1；依樣本數加權的平均則讓常見類別影響較大。指標文件說明這些選擇 [19]。

對 SST-2 或 IMDB 這類平衡任務，準確率已足夠 [6]、[14]。三類 NLI 也應報告各類別 F1：模型可能準確率不錯，卻混淆中立與矛盾。對偏斜任務，macro-F1 或 Matthews 相關係數讓罕見類別的失敗比較容易看見，但兩者都不代表失敗的實際成本。漏掉緊急客服案件與誤送一般收據，可能有不同後果。應另外寫出成本政策，而不是把方便計算的分數當成完整效用。

選擇性預測可寫成：

$$\mathrm{coverage} = \frac{1}{n} \sum_{i=1}^{n} a_i, \qquad \mathrm{risk}_{\mathrm{accepted}} = \frac{\sum_{i=1}^{n} a_i \mathbf{1}[\hat{y}_i \neq y_i]}{\sum_{i=1}^{n} a_i}.$$

其中 n 是評估樣本數，$a_i \in \{0,1\}$ 表示是否接受第 i 個樣本，$\hat{y}_i$ 是預測，y_i 是參考標籤，$\mathbf{1}[\cdot]$ 在條件成立時為一，否則為零。如果沒有接受任何樣本，接受風險便未定義。風險與涵蓋率曲線呈現不同門檻下的取捨；部署門檻應依驗證資料選擇，而不是依最終測試標籤選擇。

接著閱讀錯誤。建立錯誤表，包含文字、標準標籤、預測標籤、機率、語言、長度、來源群組與分析者的解釋。實用錯誤類型包括標註歧義、缺少脈絡、否定、未見詞彙、多主題輸入、引述，以及確實超出範圍的文字。不同類型需要不同修正。更多訓練樣本可能改善未見詞彙，卻無法補上預測時永遠拿不到的對話回合。ML 10 涵蓋一般的評估與監測工具。

把模型與前處理產物一起保存。分詞器與詞彙表、最大長度與截斷政策、標籤順序、校準溫度與決策門檻，共同構成推論套件。以另一個分詞器載入權重，會默默改變意義。套件應保存有版本的標籤結構與幾個確定性介面範例，讓服務可以檢查假設。

可執行實驗：先預訓練，再微調

直觀說法。我們在一個微小的未標註語料上，以遮罩語言模型從頭預訓練一個兩層編碼器，再為情感任務與 NLI 任務微調。資料是合成的，因此實驗示範的是機制，不是基準效能。

實驗設計分離出三個問題：

1. **預訓練能遷移嗎？**未標註語料包含所有形容詞，每個都出現在 `love` 或 `hate` 旁邊。情感訓練集從未包含 `superb` 或 `terrible`；測試集只包含這兩個形容詞，有否定也有不否定。隨機初始化的編碼器從未在任何脈絡中見過它們。
2. **只看假說的基準能揭露捷徑嗎？** NLI 訓練資料中，`not` 只出現在矛盾假說裡——一個刻意製造的標註痕跡。
3. **挑戰集能揭露它嗎？**挑戰集讓 `not` 出現在蘊涵中（兩句都否定），並建立假說沒有 `not` 的矛盾（改為前提被否定）。

編碼器以縮小版重現NLP 10 的 BERT 配方：詞元＋位置＋片段嵌入、層正規化、兩個 Transformer 編碼器區塊、遮住 15% 的位置，以及微調時接在 `[CLS]` 上、含 tanh 層的池化任務頭。最佳化器為 AdamW [26]。

```
import itertools, random
import torch
from torch import nn
from torch.nn import functional as F
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score

torch.manual_seed(0); random.seed(0); torch.set_num_threads(1)
POS, NEG = ['good', 'great', 'fine', 'superb'], ['bad', 'awful', 'poor', 'terrible']
ITEMS, NAMES = ['battery', 'screen', 'camera', 'speaker'], ['mina', 'ravi', 'lena']
VERBS = {'repaired': 'repair', 'cleaned': 'clean', 'tested': 'test'}
OBJS = ['the pump', 'the valve', 'the sensor']
words = set(' '.join(POS + NEG + ITEMS + NAMES + OBJS).split()) | set(VERBS) \
    | set(VERBS.values()) | set('the is not i love hate it someone did'.split())
vocab = ['[PAD]', '[CLS]', '[SEP]', '[MASK]'] + sorted(words)
```

```
idx, L, D = {w: i for i, w in enumerate(vocab)}, 16, 32

def batch(examples):                       # each example: (text,) or (text_a, text_b)
    ids, segs = [], []
    for ex in examples:
        toks, seg = ['[CLS]'], [0]
        for s, text in enumerate(ex):
            part = text.split() + ['[SEP]']
            toks += part; seg += [s] * len(part)
        ids.append([idx[t] for t in toks] + [0] * (L - len(toks)))
        segs.append(seg + [0] * (L - len(seg)))
    return torch.tensor(ids), torch.tensor(segs)

class TinyBERT(nn.Module):
    def __init__(self):
        super().__init__()
        self.tok, self.pos, self.seg = nn.Embedding(len(vocab), D), nn.Embedding(L, D),
         ↪  nn.Embedding(2, D)
        self.norm = nn.LayerNorm(D)
        block = nn.TransformerEncoderLayer(D, 2, 64, dropout=0.0, activation='gelu',
         ↪  batch_first=True)
        self.encoder = nn.TransformerEncoder(block, 2)
        self.mlm = nn.Linear(D, len(vocab))
    def forward(self, ids, seg):
        h = self.tok(ids) + self.pos(torch.arange(ids.shape[1])) + self.seg(seg)
        return self.encoder(self.norm(h), src_key_padding_mask=ids.eq(0))

# 1) Masked-LM pretraining on unlabeled text (no sentiment or NLI labels).
corpus = [(f'i love it the {i} is {a}',) for i in ITEMS for a in POS] \
    + [(f'i hate it the {i} is {a}',) for i in ITEMS for a in NEG] \
    + [(f'{n} {v} {o}',) for n in NAMES for v in VERBS for o in OBJS]
model = TinyBERT()
ids, seg = batch(corpus)
opt = torch.optim.AdamW(model.parameters(), lr=3e-3)
for step in range(400):
    pick = (torch.rand(ids.shape) < 0.15) & ids.gt(3)    # 15% of real tokens
    logits = model.mlm(model(ids.masked_fill(pick, idx['[MASK]']), seg))
    loss = F.cross_entropy(logits.flatten(0, 1), ids.masked_fill(~pick, -100).flatten())
    opt.zero_grad(); loss.backward(); opt.step()
print('final MLM loss:', round(loss.item(), 3))
pretrained_state = {k: v.clone() for k, v in model.state_dict().items()}

# 2) Fine-tune encoder + [CLS] head end to end.
def finetune(encoder, train, n_classes, epochs=60, lr=1e-3):
    torch.manual_seed(1)
    head = nn.Sequential(nn.Linear(D, D), nn.Tanh(), nn.Linear(D, n_classes))
    opt = torch.optim.AdamW([*encoder.parameters(), *head.parameters()], lr=lr)
    x, y = batch([ex for ex, _ in train]), torch.tensor([lab for _, lab in train])
    for _ in range(epochs):
        loss = F.cross_entropy(head(encoder(*x)[:, 0]), y)
        opt.zero_grad(); loss.backward(); opt.step()
    return lambda exs: head(encoder(*batch(exs))[:, 0]).argmax(-1).tolist()

def report(name, data, predict):
```

```
    gold = [lab for _, lab in data]
    with torch.no_grad():
        pred = predict([ex for ex, _ in data])
    print(f'{name:46s} acc={accuracy_score(gold, pred):.2f} '
          f'macro-F1={f1_score(gold, pred, average="macro"):.2f}')

# Task A: sentiment with negation. "superb" and "terrible" never get a label.
def review(item, adj, negated):
    return ((f'the {item} is {"not " * negated}{adj}',), int(adj in POS) ^ negated)
labeled = [a for a in POS + NEG if a not in ('superb', 'terrible')]
train_a = [review(i, a, n) for i in ITEMS for a in labeled for n in (0, 1)]
test_a = [review(i, a, n) for i in ITEMS for a in ('superb', 'terrible') for n in (0, 1)]
report('sentiment, pretrained encoder', test_a, finetune(model, train_a, 2))
report('sentiment, random-init encoder', test_a, finetune(TinyBERT(), train_a, 2))

# Task B: NLI (0 entailment, 1 neutral, 2 contradiction) with a negation shortcut.
def pair(n, v, o, kind):
    p, neg = f'{n} {v} {o}', f'{n} did not {VERBS[v]} {o}'
    other = list(VERBS)[(list(VERBS).index(v) + 1) % 3]
    return {'ent': ((p, f'someone {v} {o}'), 0), 'neu': ((p, f'{n} {other} {o}'), 1),
            'con': ((p, neg), 2),                  # training: "not" only in contradictions
            'neg-ent': ((neg, neg), 0),            # challenge: "not" in an entailment
            'neg-con': ((neg, p), 2)}[kind]        # challenge: contradiction without "not"
triples = list(itertools.product(NAMES, VERBS, OBJS)); random.shuffle(triples)
train_b = [pair(*t, k) for t in triples[:18] for k in ('ent', 'neu', 'con')]
test_b = [pair(*t, k) for t in triples[18:] for k in ('ent', 'neu', 'con')]
challenge = [pair(*t, k) for t in triples[18:] for k in ('neg-ent', 'neu', 'neg-con')]

vec = CountVectorizer(token_pattern=r'\S+').fit([ex[1] for ex, _ in train_b])
hyp_only = LogisticRegression(C=10, max_iter=1000).fit(
    vec.transform([ex[1] for ex, _ in train_b]), [lab for _, lab in train_b])
hyp_predict = lambda exs: hyp_only.predict(vec.transform([ex[1] for ex in exs])).tolist()
model.load_state_dict(pretrained_state)
pair_predict = finetune(model, train_b, 3)
for split, data in [('in-distribution', test_b), ('challenge', challenge)]:
    report(f'NLI {split}, hypothesis-only', data, hyp_predict)
    report(f'NLI {split}, pair encoder', data, pair_predict)
counter = [pair(*t, k) for t in triples[:18] for k in ('neg-ent', 'neg-con')]
model.load_state_dict(pretrained_state)
report('NLI challenge, pair encoder + counterexamples', challenge,
       finetune(model, train_b + counter, 3))
```

在單一 CPU 執行緒上只需數秒，輸出為：

```
final MLM loss: 0.621
sentiment, pretrained encoder                 acc=1.00 macro-F1=1.00
sentiment, random-init encoder                acc=0.25 macro-F1=0.20
NLI in-distribution, hypothesis-only          acc=1.00 macro-F1=1.00
NLI in-distribution, pair encoder             acc=1.00 macro-F1=1.00
NLI challenge, hypothesis-only                acc=0.33 macro-F1=0.22
NLI challenge, pair encoder                   acc=0.67 macro-F1=0.56
NLI challenge, pair encoder + counterexamples  acc=1.00 macro-F1=1.00
```

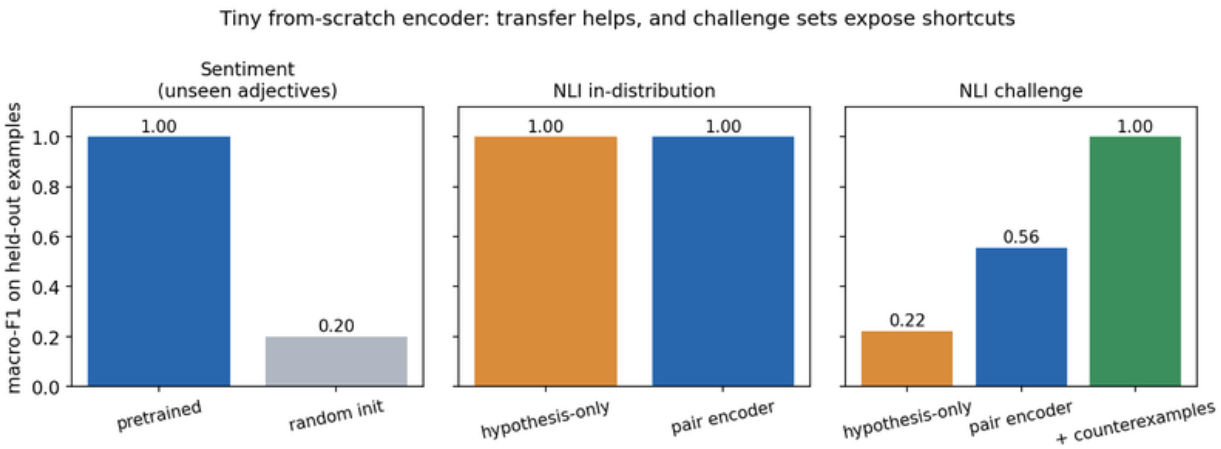


圖 *4* — 上述執行的 *macro-F1*，由圖表腳本重新產生。情感測試句 *16* 句，分布內與挑戰 *NLI* 句對各 *27* 組；全部為合成資料。

遷移。預訓練編碼器正確標記全部十六個測試句，包括否定句，儘管 `superb` 與 `terrible` 從未帶著標籤出現。預訓練期間，它們分別與 `love` 及 `hate` 共享脈絡，因此它們的表示早已靠近同極性的已標註形容詞。隨機初始化的編碼器對這兩個詞只有隨機嵌入，微調集也沒有任何東西能把它們連到標籤。這是縮小版的 BERT 論點：未標註文字可以提供少量標註資料無法提供的區別。它也顯示了依賴性：遷移之所以成功，是因為未標註語料包含相關的共現。用不相干的文字預訓練並不會有幫助。

捷徑。在分布內資料上，只看假說的邏輯斯迴歸是完美的。這正是警訊：從不讀前提的模型不可能在做推論，所以這份測試集並不要求推論。微調後的文字對編碼器在分布內也是完美的，單看這點無法知道它用了什麼。

挑戰。在挑戰集上，只看假說的基準掉到機率水準。檢查預測可發現，文字對編碼器正確分類了每個前提被否定的矛盾句對與每個中立句對，卻把九個雙重否定的蘊涵全部標成矛盾——它部分學到了「假說有 `not` 就是矛盾」。加入十八個兩種挑戰類型的反例（以訓練用的人名、動詞與物件建立）後，挑戰集就被修正。最後一行要小心解讀：反例與挑戰集共用樣板，所以滿分說明的是：只要資料要求，模型就能學會這個區別；並不表示它能泛化到新的捷徑。

這裡一切都刻意縮小：40 個詞的詞彙、最多十幾個詞元的句子、一個種子。在我們試過的四個種子中，定性模式相同，但隨機初始化與挑戰集的確切數字會變動。練習 5 請你量測這個分布。

現代觀點

綜述與其分類地圖。 Minaee 等人在 ACM Computing Surveys 的回顧涵蓋超過 150 個深度學習文字分類模型與超過 40 個資料集，橫跨情感分析、新聞分類、問答與自然語言推論 [21]。它依架構分組——前饋、循環、卷積、膠囊、注意力、記憶增強、圖、孿生、混合，以及基於 Transformer 的預訓練語言模型——核心結論是：以任務標籤微調的預訓練語言模型，已在許多文字分類任務上創下新的最佳成績。Chang 等人的評估綜述區分評估什麼、在哪裡（哪些基準）以及如何（自動、人工、程序）[24]；Zhao 等人的綜述則把分類放在預訓練、調適、使用與能力評估的整體流程中 [25]。合起來看，它們主張依任務擬定評估計畫，而不是只看一個排行榜數字。

LLM 時代的編碼器（2023–2026）。大型僅解碼器模型可以透過提示進行分類，不需任務專屬權重。Zhang 等人在 26 個資料集、13 個情感任務上，把 LLM 與在領域資料上訓練的小型模型比較。LLM 在較簡單的任務上表現令人滿意，但在需要結構化情感資訊或對特定現象更深入理解的任務上落後；在少樣本設定中則明顯勝過小型模型 [22]。實務解讀：標籤稀少時，提示是強力選項；有標註資料且重視延遲時，微調後的編碼器仍具競爭力。編碼器本身也仍在演進。發表於 ACL 2025 的 ModernBERT 把近年的架構與訓練最佳化帶入僅編碼器模型，以 2 兆個詞元、原生 8,192 詞元序列長度訓練，並在分類與檢索評估上報告強勁結果，同時兼顧速度與記憶體效率 [23]。本章的微調介面——接在 `[CLS]` 或詞元狀態上的任務頭——可以原封不動沿用。

沒有改變的事。標註痕跡 [9]、[10] 與啟發式捷徑 [11] 是資料集與訓練訊號的性質，不是某一種架構的性質。更大或更新的模型仍可能學到它們，提示式模型也可能帶來自己的捷徑。部分輸入基準、挑戰集、校準檢查 [18] 與拒答政策 [13] 仍是找出問題的工具。

開放問題。領域偏移下的穩健性（MultiNLI 的 matched ／ mismatched 切分 [8] 是早期且狹窄的版本）、長文與多段落輸入的校準拒答、多語與語碼轉換的情感、隨政策改變的標籤，以及評估集洩漏進預訓練語料。每一項都更適合在大型模型旁邊放一個小而透明的基準來研究。

重點整理

- 編碼器是共用的；任務頭與標籤定義任務。先選序列、詞元或區段輸出。
- 文字對任務把兩段文字打包成一個序列，讓注意力逐詞元比較；代價是任一側都無法快取。
- 監督單位——句子、評論、對話——是任務定義的一部分。
- 只看假說（部分輸入）的基準若表現良好，表示該基準並不完全要求你在意的關係。
- 一次翻轉一個線索的挑戰集，能揭露分布內準確率所掩蓋的捷徑。
- 以小學習率、暖身與衰減微調；報告多個種子的變異，並只依開發資料選擇。
- 把決策（準確率、macro-F1）、機率（校準）與拒答（覆蓋率、接受風險）當作不同性質分別評估。
- 相關不代表可回答：區段答案需要支持它的段落，以及明確說「未提及」的方式。

練習

1. **截斷政策**。你要分類的評論常比編碼器視窗長。比較保留開頭詞元、保留結尾詞元，以及在重疊視窗上平均預測三種作法。每種政策會誤判哪些評論？

   提示

   寫三則簡短的合成評論：結論出現在開頭、出現在結尾，以及在中途反轉（「一開始我很喜歡……但是」）。在選擇政策前，先決定標註準則想要的標籤。

2. **部分輸入稽核**。對實驗中的 NLI 資料，再訓練一個只看前提的基準。執行前先預測它的準確率，再解釋結果。

   提示

   在訓練樣板中，每個前提都會與三種標籤同時出現。只看前提的模型不可能優於多數類別；把它與只看假說的結果比較。

3. **SQuAD 2.0 門檻**。給定可回答與不可回答問題在開發集上的分數 s_{null} 與 $\hat{s}$，說明如何選擇 τ，以及 τ 增大時覆蓋率與接受準確率如何變化。

   提示

   在觀察到的分數差 $\hat{s} - s_{\text{null}}$ 上掃描 τ。τ 越大，回答的問題越少。畫出覆蓋率對接受準確率的曲線，再依錯答的既定成本選點。

4. **校準**。為實驗中的情感分類器加入溫度縮放：在保留切分上以最小化負對數概似擬合一個溫度。說明準確率為何不會改變，並量測平均信心如何移動。

   提示

   把 logit 除以正的純量會保留其次序。改為最佳化 $\log T$，可讓溫度保持為正。

5. **種子變異**。以五個種子執行預訓練與微調，報告每一行的平均與範圍，並判斷哪些結論仍然成立。

   提示

   兩個 `torch.manual_seed` 呼叫都要改。把方向性的主張（預訓練優於隨機初始化；挑戰集低於分布內）與確切數值的主張分開。

6. **推論挑戰句對**。建立六組 NLI 句對，分別測試角色反轉、前提中的否定、情態、歸屬、量詞範圍與時間。在評估任何模型前，先寫下預期標籤與假設。

   提示

   前提提供的資訊太少時，要保留中立選項。測試歸屬時，比較「Mina said the pump failed」與「The pump failed」。

參考文獻

1. Masato Hagiwara, *Real-World Natural Language Processing: Practical Applications with Deep Learning*, Chapters 9–11, Manning, 2021. Publisher
2. Susan McRoy, *Principles of Natural Language Processing*, Chapters 6 and 8, open textbook, 2021. Open textbook
3. Jacob Devlin, Ming-Wei Chang, Kenton Lee and Kristina Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” NAACL-HLT, 2019. Paper
4. Google Research, *BERT: original implementation*, software, 2018. Classification, Question answering, Optimizer
5. Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng and Christopher Potts, “Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank,” EMNLP, 2013. Paper

6. Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng and Christopher Potts, “Learning Word Vectors for Sentiment Analysis,” ACL-HLT, 2011. Paper
7. Samuel R. Bowman, Gabor Angeli, Christopher Potts and Christopher D. Manning, “A large annotated corpus for learning natural language inference,” EMNLP, 2015. Paper
8. Adina Williams, Nikita Nangia and Samuel R. Bowman, “A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference,” NAACL-HLT, 2018. Paper
9. Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman and Noah A. Smith, “Annotation Artifacts in Natural Language Inference Data,” NAACL-HLT, 2018. Paper
10. Adam Poliak, Jason Naradowsky, Aparajita Haldar, Rachel Rudinger and Benjamin Van Durme, “Hypothesis Only Baselines in Natural Language Inference,” *SEM, 2018. Paper
11. R. Thomas McCoy, Ellie Pavlick and Tal Linzen, “Right for the Wrong Reasons: Diagnosing Syntactic Heuristics in Natural Language Inference,” ACL, 2019. Paper
12. Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev and Percy Liang, “SQuAD: 100,000+ Questions for Machine Comprehension of Text,” EMNLP, 2016. Paper
13. Pranav Rajpurkar, Robin Jia and Percy Liang, “Know What You Don’t Know: Unanswerable Questions for SQuAD,” ACL, 2018. Paper
14. Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy and Samuel R. Bowman, “GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding,” ICLR, 2019. Paper
15. Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy and Samuel R. Bowman, “SuperGLUE: A Stickier Benchmark for General-Purpose Language Understanding Systems,” NeurIPS, 2019. Paper
16. Marius Mosbach, Maksym Andriushchenko and Dietrich Klakow, “On the Stability of Fine-tuning BERT: Misconceptions, Explanations, and Strong Baselines,” ICLR, 2021. Paper
17. Chuan Guo, Geoff Pleiss, Yu Sun and Kilian Q. Weinberger, “On Calibration of Modern Neural Networks,” ICML, 2017. Paper
18. Shrey Desai and Greg Durrett, “Calibration of Pre-trained Transformers,” EMNLP, 2020. Paper
19. scikit-learn developers, “Classification metrics,” scikit-learn documentation, current online edition. Documentation
20. Nils Reimers and Iryna Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” EMNLP-IJCNLP, 2019. Paper
21. Shervin Minaee, Nal Kalchbrenner, Erik Cambria, Narjes Nikzad, Meysam Chenaghlu and Jianfeng Gao, “Deep Learning–based Text Classification: A Comprehensive Review,” ACM Computing Surveys 54(3), 2021. Survey
22. Wenxuan Zhang, Yue Deng, Bing Liu, Sinno Jialin Pan and Lidong Bing, “Sentiment Analysis in the Era of Large Language Models: A Reality Check,” Findings of NAACL, 2024. Paper
23. Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini et al., “Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference,” ACL, 2025. Paper
24. Yupeng Chang et al., “A Survey on Evaluation of Large Language Models,” ACM Transactions on Intelligent Systems and Technology, 2024. Survey
25. Wayne Xin Zhao et al., “A Survey of Large Language Models,” Frontiers of Computer Science, 2026. Updated survey

26. Ilya Loshchilov and Frank Hutter, “Decoupled Weight Decay Regularization,” ICLR, 2019. Paper

NLP 12・句法與語意任務

以 CKY 剖析成分、以轉移或生成樹剖析依存，標註語意角色、解決共指、概覽 AMR，並以各自的指標評估每種結構。

網站版本: https://blog.shyandram.dev/zh-tw/learn/nlp/nlp-12-syntactic-semantic-tasks/

你將學到

- 定義上下文無關文法，用 CKY 圖表辨識並列舉剖析，再以機率文法或神經跨度分數在其中選擇。
- 以轉移系統（arc-standard）與圖式解碼建立依存樹，並解釋為何逐詞獨立選中心詞需要樹限制。
- 標註述詞—論元結構（語意角色），在擷取的關係中保留作用範圍與來源，並讀懂一個小型 AMR 圖。
- 以先行詞模型把提及連成實體，並在脈絡不足時保留歧義。
- 用各自的指標評估每種結構：PARSEVAL 括號、UAS ／ LAS、角色 F1、MUC ／ B^3 ／ CEAF 與 CoNLL F1，以及 Smatch。
- 說明預訓練編碼器與大型語言模型如何改變這些任務，以及哪些事沒有改變。

先看全貌

語言表面是線性的，解讀卻有結構。句子以有順序的詞序列輸入，但讀者會把詞分組、辨識彼此的關係、判斷誰對誰做了什麼，並把一句中的「it」連到另一句的「the pump」。「We saw robots with cameras」可能描述帶著相機的機器人，也可能描述觀察時使用的器材。單靠詞清單無法選定解讀。

> **用最簡單的話說**
>
> 句子像一組建構玩具。詞是零件，文法說明怎樣連接。剖析顯示連接方式；角色標註說明哪個零件是動手的、哪個是被處理的；共指則發現兩個零件其實是同一個玩具被看了兩次。有時相同零件能做出兩種合理建構。好的解釋應呈現兩者，再詢問哪個符合情境。

本章收集自然語言處理中經典的**結構化預測**任務。在NLP 06 中，輸出是每個位置一個標籤；這裡的輸出則是一棵樹、一張圖或提及的分割，而且某處的選擇會限制其他地方：一個詞只能有一個中心詞、一個跨度只能屬於述詞的一個論元、共指連結必須形成一致的群組。McRoy 關於語法、語意與篇章的章節，以及另附的圖表剖析附錄，提供語言學觀點 [1]；Jurafsky 與 Martin 關於成分剖析、依存剖析、語意角色與共指的章節，提供演算法觀點 [2]。成分、依存、量詞範圍等語言學類別本身，在NLP 02 介紹。

這些結構有助擷取誰做了什麼、定位修飾語，並建立區分流暢度與結構理解的測試。它們無法處理所有問題：合文法的句子可能是假的，不完整的訊息也可能清楚傳達意思。我們把每種結構視為範圍明確的表示，而不是理解的定義。

成分與上下文無關文法

直觀說法。片語結構把詞組成能遞迴結合的單位。文法規定允許的組合；剖析則是某個句子的推導。

上下文無關文法（context-free grammar）可定義為：

$$G = (\mathcal{N}, \Sigma, \mathcal{R}, S).$$

其中 G 是文法，$\mathcal{N}$ 是非終結類別集合，Σ 是終結詞彙表，$\mathcal{R}$ 是改寫規則集合，$S \in \mathcal{N}$ 是起始類別。非終結符代表名詞片語等結構，終結符則是觀察到的詞。上下文無關規則替換一個非終結符，而不檢查周圍符號。

原創教學文法可使用 $S \to NP\ VP$、$VP \to V\ NP$、$PP \to P\ NP$ 與 $NP \to NP\ PP$。其中 S 表示句子，NP 表示名詞片語，VP 表示動詞片語，PP 表示介系詞片語，V 表示動詞類別，P 表示介系詞類別。這是教學文法的符號選擇，不是完整英文文法。Penn Treebank 等樹庫提供大量以這類樹標註的句子，文法與剖析器即由此學習 [4]。

遞迴規則能以有限規格產生無界的句子家族，但同樣的遞迴也可能讓天真的搜尋反覆做相同工作或無限循環，因此剖析器需要搜尋策略與緊湊的部分分析表示。

應區分涵蓋範圍與偏好。規則系統可能接受句子卻給出多個分析，也可能因缺少詞彙條目而無法剖析完全合理的句子。剖析失敗只表示所實作的文法與搜尋沒有找到分析，不表示使用者寫了無意義的內容。明確文法的好處是可檢查性：能顯示哪個規則連結了跨度，以及歧義出現在哪裡 [1]。

依跨度進行圖表剖析：CKY 演算法

由上而下的假設與由下而上的證據

直觀說法。剖析器可以從句子預期包含什麼開始，也可以從觀察詞已支持什麼開始。兩個方向都需要避免重複工作。

由上而下搜尋從起始類別展開產生式，形成預期子節點。它能排除不符合預期的詞彙分析，但不好的展開選擇可能導致很長的失敗路徑；若遞迴產生式把第一個子節點展成同一類別，就可能在還沒消耗輸入前反覆遇到同一個未解預期。由下而上搜尋從觀察詞建立跨度支持的類別，可能產生許多不屬於任何完整句子的片語，但中間結果也可能被多個成功分析共用。

因此，設計問題不只是選哪個方向「比較好」，而是部分工作如何表示，以及兩個部分分析何時算同一搜尋狀態。圖表儲存完成跨度，另一條路徑抵達既有狀態時不必重新計算。不過記憶化本身不會解決歧義：辨識只需要每個跨度與類別一個布林值，偏好的剖析需要分數與回溯指標，所有選項則需要以森林保留不同推導連結。

跨度遞迴式

直觀說法。儲存每個連續跨度可能的類別。把短跨度結合成長跨度，並讓所有完整剖析共用結果。

Cocke–Kasami–Younger（CKY）演算法是這個想法的標準動態規劃 [2]。它需要二元規則：喬姆斯基正規形（Chomsky normal form）的文法只有 $A \to B\,C$ 與 $A \to a$ 兩種規則，而任何不含空產生式的上下文無關文法都能轉成此形式 [2]。採用半開邊界：跨度 $[i, j)$ 包含位置 i 到 $j-1$ 的詞元。辨識可寫成：

$$C[i,j,A] = \bigvee_{\substack{A \to BC \in \mathcal{R} \\ i<k<j}} \left(C[i,k,B] \wedge C[k,j,C] \right).$$

其中 $C[i,j,A]$ 是類別 A 能涵蓋跨度 $[i,j)$ 的布林敘述，$\mathcal{R}$ 是二元規則集合，$A \to BC$ 結合左類別 B 與右類別 C，k 是內部切分邊界，$\wedge$ 表示兩個子節點都必須存在，$\bigvee$ 表示至少一組規則與切分成功。字母 C 同時出現在圖表符號與右子節點類別中，可由中括號與類別引數分辨角色。

先以詞彙規則初始化長度為一的格子，再依跨度長度、起點、切分點與相容規則迭代。子節點比父節點短，因此已經完成。一元規則與空產生式需要額外處理；本章可執行文法刻意不包含兩者。

這個演算法能辨識輸入，但辨識本身無法解釋樹。應以回溯指標或緊湊森林儲存成功組合。多個父節點可以共享子跨度，而不必複製完整子樹。高度歧義句子若展開所有完整樹，即使辨識是多項式時間，展開仍可能需要指數時間。固定二元文法下，跨度與切分巢狀迴圈對詞元長度呈立方關係；圖表有平方數量的跨度，每個跨度包含可能類別。

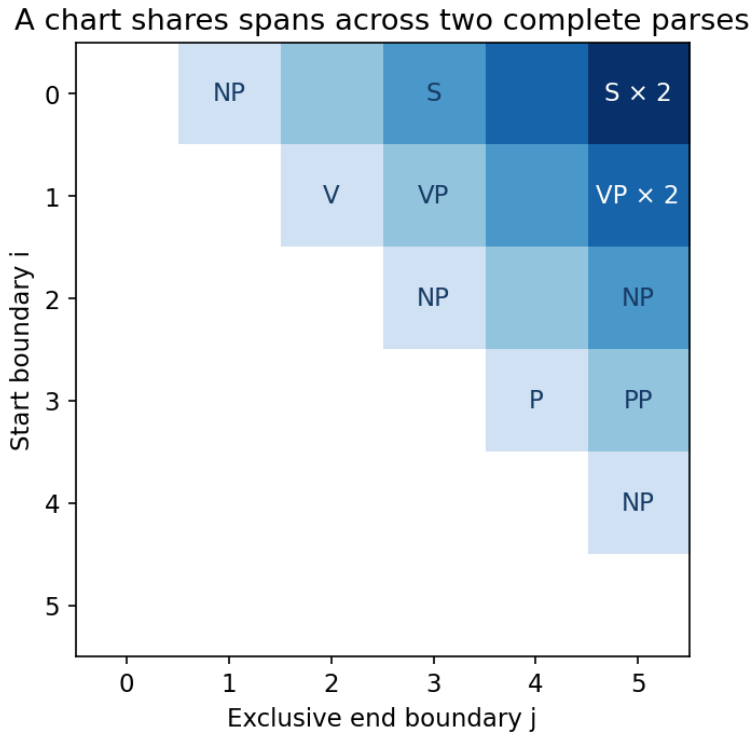


圖 *1* — 圖表儲存可重用的跨度；完整句子格可以包含不只一種分析。

保留兩種分析的剖析器

文法允許兩種附接時，就回報兩種選項。剖析器不應只因某個選項先被加入，就丟掉歧義。以下獨立剖析器在圖表格子中儲存完整小樹。它刻意比緊湊森林實作簡單；適合五個詞的示範，而前面幾段已說明為何明確列舉不適合大型歧義輸入。

```
from collections import defaultdict

words = "we saw robots with cameras".split()
lexical = {
    "we": ["NP"], "saw": ["V"], "robots": ["NP"],
    "with": ["P"], "cameras": ["NP"],
}
rules = [
    ("S", "NP", "VP"),
    ("VP", "V", "NP"),
    ("VP", "VP", "PP"),
    ("NP", "NP", "PP"),
    ("PP", "P", "NP"),
]
chart = defaultdict(lambda: defaultdict(list))
for i, word in enumerate(words):
    for category in lexical.get(word, []):
        chart[i, i + 1][category].append((category, word))
for length in range(2, len(words) + 1):
    for start in range(len(words) - length + 1):
        end = start + length
        for split in range(start + 1, end):
            for parent, left, right in rules:
                for left_tree in chart[start, split].get(left, []):
                    for right_tree in chart[split, end].get(right, []):
                        tree = (parent, left_tree, right_tree)
                        if tree not in chart[start, end][parent]:
                            chart[start, end][parent].append(tree)

def bracket(tree):
    if len(tree) == 2:
        return f"({tree[0]} {tree[1]})"
    return f"({tree[0]} {bracket(tree[1])} {bracket(tree[2])})"

parses = chart[0, len(words)].get("S", [])
assert len(parses) == 2
for tree in parses:
    print(bracket(tree))
print("complete parses:", len(parses))
```

```
(S (NP we) (VP (V saw) (NP (NP robots) (PP (P with) (NP cameras)))))
(S (NP we) (VP (VP (V saw) (NP robots)) (PP (P with) (NP cameras))))
```

```
complete parses: 2
```

一棵樹把介系詞片語放進受詞名詞片語，另一棵則讓它修飾動詞片語。兩者都按順序使用每個詞，並符合文法。要選擇其中之一，可能需要周圍場景、先前篇章，或超出目前規則的詞彙偏好。

加入未知詞，某個詞彙格子可能變空。正確做法是診斷涵蓋範圍。正式服務可以使用備援標記器、允許明確未知類別，或要求澄清，但每個備援都會改變接受的語言，應記錄並測試。

結構上的機率

直觀說法。文法說明哪些分析存在；機率模型可以偏好部分分析，並彙整各選項的不確定性。

機率式上下文無關文法（PCFG）為每個產生式配置機率，並在相同父類別的規則間正規化。一個推導具有：

$$p(\mathcal{T}) = \prod_{r \in \mathcal{T}} p(r), \qquad \hat{\mathcal{T}} = \operatorname*{argmax}_{\mathcal{T} \in \mathcal{D}(\mathbf{x})} p(\mathcal{T}).$$

其中 $\mathcal{T}$ 是完整推導樹，r 索引樹中每次產生式使用，$p(r)$ 是規則機率，$\mathcal{D}(\mathbf{x})$ 是終結輸出為輸入 $\mathbf{x}$ 的允許剖析集合，$\hat{\mathcal{T}}$ 是機率最高的剖析。同一規則重複使用時，乘積會分別計入。

把圖表中的選項改成加總，就能計算各推導的總機率質量（inside 演算法）；改成最大值，就能計算最佳推導（機率式 CKY）。兩者回答不同問題：具有許多中等機率剖析的句子，總機率可能高於只有一個強剖析的句子，最佳樹也不一定占後驗質量的大部分。以下程式對同樣五個詞同時計算兩者。

```
import math
from collections import defaultdict

words = "we saw robots with cameras".split()
lexical = {"we": "NP", "saw": "V", "robots": "NP", "with": "P", "cameras": "NP"}

def parse(p_np_pp):
    # Binary-rule probabilities. Both trees use the same five lexical rules,
    # so their probabilities are a shared factor and are omitted (set to 1).
    rules = {("S", "NP", "VP"): 1.0, ("PP", "P", "NP"): 1.0,
             ("VP", "V", "NP"): 0.6, ("VP", "VP", "PP"): 0.4,
             ("NP", "NP", "PP"): p_np_pp}
    best = defaultdict(dict)    # (i, j) -> {category: (log prob, tree)}
    inside = defaultdict(lambda: defaultdict(float))
    for i, w in enumerate(words):
        best[i, i + 1][lexical[w]] = (0.0, f"({lexical[w]} {w})")
        inside[i, i + 1][lexical[w]] = 1.0
    n = len(words)
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length
            for k in range(i + 1, j):
```

```
                for (a, b, c), p in rules.items():
                    if b in best[i, k] and c in best[k, j]:
                        score = math.log(p) + best[i, k][b][0] + best[k, j][c][0]
                        if score > best[i, j].get(a, (-math.inf,))[0]:
                            best[i, j][a] = (score, f"({a} {best[i, k][b][1]} {best[k,
                             ↪  j][c][1]})")
                        inside[i, j][a] += p * inside[i, k][b] * inside[k, j][c]
    score, tree = best[0, n]["S"]
    return tree, math.exp(score), inside[0, n]["S"]

for p in (0.2, 0.5):
    tree, p_best, p_total = parse(p)
    print(f"P(NP -> NP PP) = {p}: best = {p_best:.3f}, total = {p_total:.3f}")
    print("  ", tree)
```

```
P(NP -> NP PP) = 0.2: best = 0.240, total = 0.360
   (S (NP we) (VP (VP (V saw) (NP robots)) (PP (P with) (NP cameras))))
P(NP -> NP PP) = 0.5: best = 0.300, total = 0.540
   (S (NP we) (VP (V saw) (NP (NP robots) (PP (P with) (NP cameras)))))
```

動詞附接的機率是 $0.4 \times 0.6 = 0.24$，與名詞規則無關；名詞附接則是 $0.6 \times p(NP \to NP\,PP)$。當該規則機率越過 0.4 時，偏好的樹就翻轉，而 inside 總和永遠同時計入兩棵樹。注意決定附接的是什麼：一個對 `saw`、`robots`、`cameras` 一無所知的規則機率。下一節的神經評分器正是針對這個弱點。

從樹庫估計的規則機率取決於標註慣例與領域。新聞樹庫與客服語料可能有不同片段模式與附接偏好。機率建模提供偏好機制，但不會消除領域依賴。

神經成分剖析：為跨度評分，再用圖表解碼

直觀說法。不再相乘規則機率，而是讓神經編碼器替每個跨度與標籤評分，再用同一張圖表找出分數最高的樹。

跨度式剖析器替每個候選成分評分，並搜尋總分最大的樹：

$$s(\mathcal{T}) = \sum_{(i,j,\ell)\in\mathcal{T}} s(i,j,\ell), \qquad \hat{\mathcal{T}} = \operatorname*{argmax}_{\mathcal{T}} s(\mathcal{T}).$$

其中 $\mathcal{T}$ 是視為帶標籤跨度 (i,j,ℓ) 集合的樹，ℓ 是半開跨度 $[i,j)$ 的標籤，$s(i,j,\ell)$ 是由跨度邊界上下文向量算出的分數，$\hat{\mathcal{T}}$ 是解碼得到的樹。Stern、Andreas 與 Klein 顯示，獨立替標籤與跨度評分可與傳統圖表動態規劃相容，也容許以遞迴切分輸入的貪婪由上而下解碼器 [5]。Kitaev 與 Klein 把該設計中的 LSTM 編碼器換成自注意力，發現分開位置與內容資訊能提升準確度，並報告在 Penn Treebank 上不使用外部資料時 93.55 F1、使用預訓練詞表示時 95.13 F1 [6]。後續研究再加入多語預訓練 [7]。

即使分數來自大型神經網路，輸出仍是一棵樹：圖表保證結構合法，編碼器則提供 PCFG 規則機率無法提供的詞彙證據。

依存結構與語法角色

直觀說法。依存分析直接連結詞，顯示哪個詞依附哪個詞，並標記語法關係。

基本依存樹可用中心詞索引表示：

$$h_t \in \{0, 1, \dots, T\} \setminus \{t\}, \qquad r_t \in \mathcal{D}.$$

其中 h_t 是詞 t 的中心詞索引，0 表示人工根節點，T 是句子長度，r_t 是該詞的關係標籤，$\mathcal{D}$ 是關係集合。有效基本樹還需要明確的根節點規則、連通性與無有向循環。

逐詞獨立選中心詞可能違反全域要求。兩個詞可能互相選為中心而形成循環；數個不連通部分可能沒有唯一句根。樹解碼器應落實預期結構，或由服務驗證並拒絕無效分析。

Universal Dependencies 提供跨語言的詞法、詞性與語法依存標註框架 [8]。類別設計希望兼顧跨語言比較與語言專屬差異。這是標註框架，不表示每種語言具有相同詞序或明示語法標記。

片語結構與依存關係強調不同資訊。成分樹清楚表示跨度分組；依存樹清楚表示詞彙中心與關係。相互轉換需要中心詞、標點與功能詞的約定，不能只當成換一種畫法。

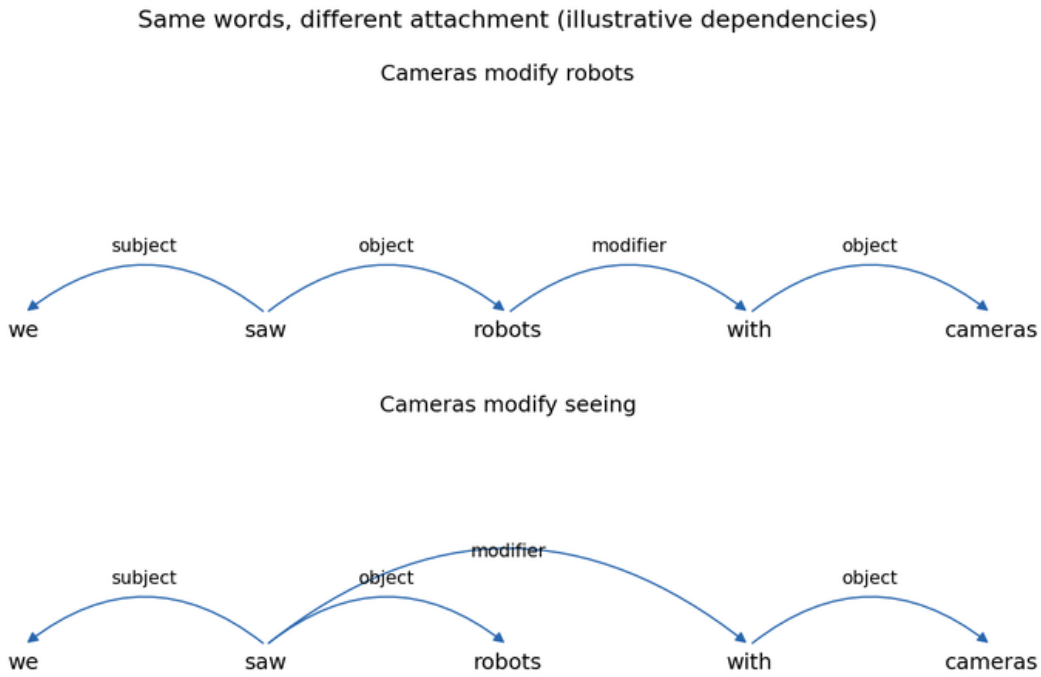


圖 2 — 相同詞的兩種附接。這些示意語法連結用來解釋歧義，並未複製樹庫中的圖。

對擷取而言，語法角色是有用證據，但不能與語意角色互換。被動句改變語法主語位置，卻保留誰對誰做了什麼。下方的語意角色小節會發展這個差別，說明依存箭頭為何不能單獨證明真實世界關係。

附接歧義揭露模型的歸納偏好

兩種相機附接共享詞彙，卻替看見事件配置不同參與者。名詞片語附接描述機器人的性質；動詞片語附接描述看見時的工具或情境。下游擷取器若選定名詞解讀，可能建立機器人—相機持有關係；工具解讀卻不支持該紀錄。附接差異之所以重要，是因為它改變關係，不只是括號長得不同。

如 PCFG 程式所示，文法可以允許兩者，機率模型仍會偏好其一。若只有父類別的規則機率，除非擴充狀態或評分表示，否則模型無法依 `saw`、`robots`、`cameras` 的詞彙身分改變偏好。這是容量限制；對相同分數擴大搜尋，不會創造缺少的詞彙證據。

相反的失敗是搜尋限制：模型可能已有實用分數，卻在形成完整樹之前，透過剪枝移除正確跨度。應分別診斷：參考結構能否表示、是否保留於候選建立階段，以及在留下的候選中是否得最高分。答案分別指向不同改善：擴充表示、調整剪枝或改進評分。

也可能真的缺少資訊：孤立句子中兩種閱讀可能都成立，標準樹只記錄語料選定的解讀。對擷取器而言，先保留兩種可能，等到檢查所需關係再決定，可能比早期定案更合適。

轉移式依存剖析

直觀說法。用一個堆疊由左到右讀句子，每一步從少數動作中選一個：把下一個詞移入堆疊，或把堆疊頂端兩項互相連結。分類器選動作，動作序列就建出樹。

Nivre 把確定性依存剖析器形式化為轉移系統，並分析以堆疊為基礎的變體（arc-standard 與 arc-eager，限於投射樹）以及以串列為基礎的變體，其中之一能處理非投射樹 [9]。**投射（projective）**樹是指弧線能畫在句子上方而不交叉的樹。在 Chen 與 Manning 使用的 arc-standard 系統中，組態 $c = (\sigma, \beta, A)$ 包含堆疊 σ、緩衝區 β 與弧集合 A，並有三種轉移 [10]：

$$\begin{aligned}
\textsc{Shift}&: \ (\sigma,\ w|\beta,\ A) \Rightarrow (\sigma|w,\ \beta,\ A),\\
\textsc{Left-Arc}_r&: \ (\sigma|s_2|s_1,\ \beta,\ A) \Rightarrow (\sigma|s_1,\ \beta,\ A \cup \{s_1 \xrightarrow{r} s_2\}),\\
\textsc{Right-Arc}_r&: \ (\sigma|s_2|s_1,\ \beta,\ A) \Rightarrow (\sigma|s_2,\ \beta,\ A \cup \{s_2 \xrightarrow{r} s_1\}).
\end{aligned}$$

其中 s_1 是堆疊頂端，s_2 是其下一項，w 是緩衝區第一個詞，$\sigma|x$ 表示把 x 放在堆疊 σ 頂端，$h \xrightarrow{r} d$ 是從中心詞 h 指向依附詞 d、標籤為 r 的弧，$\Rightarrow$ 讀作「變成」。剖析開始時堆疊只有根、所有詞都在緩衝區；緩衝區清空且只剩根時結束。每個詞被移入一次、被連結一次，因此 n 個詞的句子恰好需要 $2n$ 次轉移。

訓練分類器時，把每棵標準樹轉成能產生它的轉移序列，稱為**靜態神諭（static oracle）**。以下規則是一種標準選擇：堆疊第二項的標準中心詞是頂端時向左連結；頂端的標準中心詞是第二項且頂端已收齊自己所有依附詞時向右連結；否則移入。

```
words = ["ROOT", "we", "saw", "robots", "with", "cameras"]
# Gold heads for the instrument reading: "with cameras" attaches to "saw".
gold = {1: 2, 2: 0, 3: 2, 4: 5, 5: 2}

def oracle(gold):
    stack, buffer, arcs, steps = [0], list(range(1, len(words))), {}, []
```

```
    def done(h):  # has h already collected all of its dependents?
        return all(d in arcs for d, head in gold.items() if head == h)
    while buffer or len(stack) > 1:
        if len(stack) >= 2 and gold.get(stack[-2]) == stack[-1]:
            dependent = stack.pop(-2)
            arcs[dependent] = stack[-1]; steps.append("LEFT-ARC")
        elif len(stack) >= 2 and gold[stack[-1]] == stack[-2] and done(stack[-1]):
            dependent = stack.pop()
            arcs[dependent] = stack[-1]; steps.append("RIGHT-ARC")
        else:
            stack.append(buffer.pop(0)); steps.append("SHIFT")
    return steps, arcs

steps, arcs = oracle(gold)
print(" ".join(steps))
print({words[d]: words[h] for d, h in sorted(arcs.items())})
assert arcs == gold and len(steps) == 2 * (len(words) - 1)
```

```
SHIFT SHIFT LEFT-ARC SHIFT RIGHT-ARC SHIFT SHIFT LEFT-ARC RIGHT-ARC RIGHT-ARC
{'we': 'saw', 'saw': 'ROOT', 'robots': 'saw', 'with': 'cameras', 'cameras': 'saw'}
```

五個詞用了十次轉移，與預測一致。`done` 檢查很重要：如果 `robots` 在 `cameras` 到達前就被連到 `saw` 並彈出，之後就不可能建出名詞附接的樹。歸約的順序編碼了樹。

剖析時由分類器取代神諭。Chen 與 Manning 把堆疊與緩衝區固定位置上的詞、詞性與弧標籤嵌入，輸入一個使用立方活化函數（cube activation）的單隱藏層網路，再以 softmax 選擇轉移；他們的貪婪剖析器很快，但如他們所說，會受到錯誤傳遞影響，因為早期錯誤動作無法撤銷 [10]。Kiperwasser 與 Goldberg 則為每個詞元計算一個與剖析器共同訓練的雙向 LSTM 向量，再串接少數幾個向量作為特徵；同一種表示同時適用於貪婪轉移剖析器與全域最佳化的圖式剖析器 [12]。轉移剖析器的時間與句長成線性；代價是局部、由左到右的決策程序。

圖式依存剖析

直觀說法。一次替所有可能的中心詞—依附詞弧評分，再選出能構成樹、總分最高的一組弧。

McDonald 等人把加權依存剖析形式化為：在以詞加上根為節點的有向圖中，尋找最大生成樹 [11]：

$$\hat{\mathbf{h}} = \operatorname*{argmax}_{\mathbf{h} \in \mathcal{Y}(\mathbf{x})} \sum_{t=1}^{T} s(h_t, t).$$

其中 $\mathbf{h} = (h_1, \dots, h_T)$ 為每個詞指定一個中心詞，$\mathcal{Y}(\mathbf{x})$ 是對句子 $\mathbf{x}$ 構成有效樹的中心詞指派集合，$s(h, t)$ 是從中心詞 h 指向依附詞 t 的弧分數。在投射樹上，Eisner 演算法以 $O(n^3)$ 時間求解；在包含非投射樹的所有樹上，Chu–Liu–Edmonds 最大生成樹演算法以 $O(n^2)$ 時間求解 [11]。

Dozat 與 Manning 在深層雙向編碼器之上，把弧評分器做成**雙仿射（biaffine）**形式 [13]：

$$s(h, t) = \mathbf{r}_t^{(\text{dep})\top} U \, \mathbf{r}_h^{(\text{head})} + \mathbf{r}_h^{(\text{head})\top} \mathbf{u}.$$

其中 $\mathbf{r}_t^{(\text{dep})}$ 與 $\mathbf{r}_h^{(\text{head})}$ 是依附詞與候選中心詞上下文向量的低維投影，U 是替兩者互動評分的學習矩陣，$\mathbf{u}$ 是讓每個詞具有成為中心詞先驗傾向的學習向量。第二個雙仿射分類器則預測所選弧的關係標籤。評分器一次就產生完整的 $(T+1)\times T$ 弧分數矩陣。

樹限制帶來什麼？以下程式為我們的句子使用虛構分數，先讓每個詞獨立挑選最佳中心詞，再列舉所有單一根的樹——只對極小輸入可行，卻讓解碼器的工作看得見。最後計算下一節定義的 UAS 與 LAS。

```
import itertools
import numpy as np

words = ["ROOT", "we", "saw", "robots", "with", "cameras"]
n = len(words) - 1
# score[h, d]: invented arc scores (head h -> dependent d), not a trained model.
score = np.full((n + 1, n + 1), -2.0)
score[0, 2], score[2, 1], score[2, 3], score[5, 4] = 3.0, 2.0, 2.0, 2.0
score[3, 5], score[2, 5] = 1.5, 1.2                # robots vs saw for "cameras"
score[4, 5] = 1.8                                  # tempting but creates a cycle
np.fill_diagonal(score, -np.inf)

def is_tree(heads):                                # heads[d-1] is the head of d
    if sum(h == 0 for h in heads) != 1:
        return False
    for d in range(1, n + 1):
        seen, node = set(), d
        while node != 0:
            if node in seen:
                return False
            seen.add(node); node = heads[node - 1]
    return True

greedy = [int(np.argmax(score[:, d])) for d in range(1, n + 1)]
total = lambda hs: sum(score[h, d] for d, h in enumerate(hs, start=1))
trees = [hs for hs in itertools.product(range(n + 1), repeat=n)
         if all(h != d for d, h in enumerate(hs, 1)) and is_tree(hs)]
best = max(trees, key=total)
print("greedy heads:", greedy, "valid tree:", is_tree(greedy))
print("best tree:   ", list(best), "score:", round(total(best), 2))
print("trees enumerated:", len(trees))

gold = [2, 0, 2, 5, 2]
labels_gold = ["nsubj", "root", "obj", "case", "obl"]
labels_pred = ["nsubj", "root", "obj", "mark", "nmod"]
uas = np.mean([p == g for p, g in zip(best, gold)])
las = np.mean([p == g and lp == lg for p, g, lp, lg
               in zip(best, gold, labels_pred, labels_gold)])
print(f"UAS = {uas:.2f}, LAS = {las:.2f}")
```

```
greedy heads: [2, 0, 2, 5, 4] valid tree: False
best tree:    [2, 0, 2, 5, 3] score: 10.5
trees enumerated: 625
```

```
UAS = 0.80, LAS = 0.60
```

貪婪選擇讓 `with` 與 `cameras` 互為中心詞，形成與根不連通的兩詞循環（圖 3）。在五個詞的 625 棵單一根樹中，最佳者打破循環，把 `cameras` 連到 `robots`——這是有效的樹，卻是名詞解讀，而標準標註選的是工具解讀。最大生成樹演算法不需列舉就能找到相同答案。

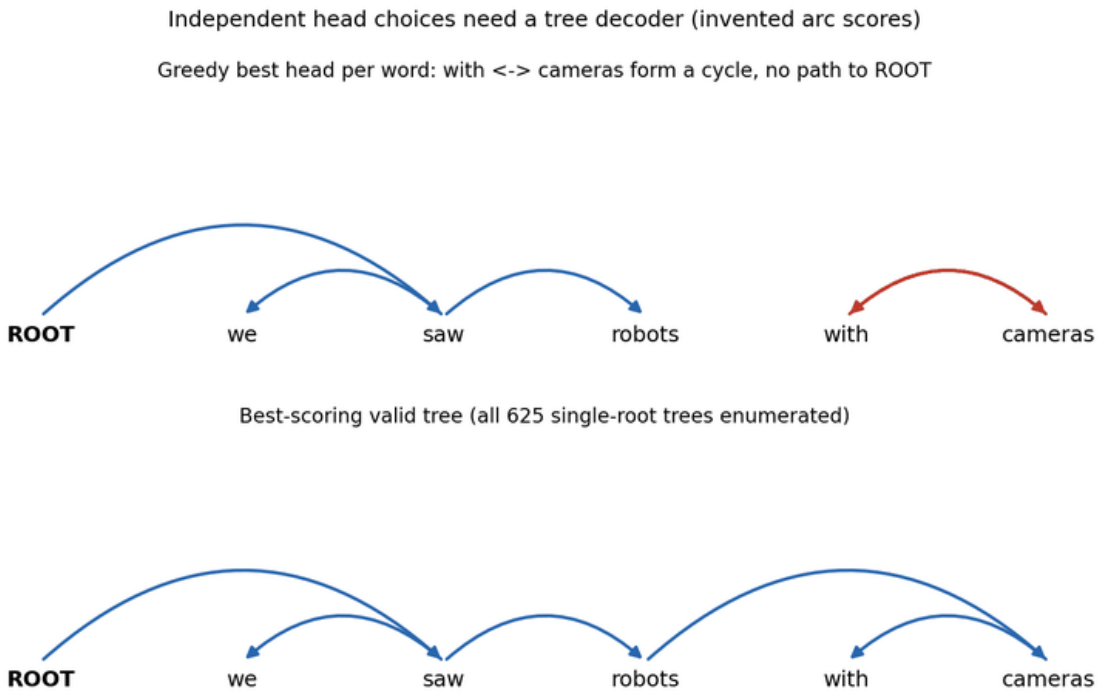


圖 3 — 獨立選出的最佳中心詞可能形成循環；樹解碼器恢復有效結構。分數為虛構，與程式一致。

圖式與轉移式剖析器的取捨不同：前者在弧分數上做全域搜尋，解碼為平方或立方時間；後者做線性時間的局部決策。Kiperwasser 與 Goldberg 的共用 BiLSTM 特徵同時服務兩個家族 [12]。不論用哪一族，結構保證都來自解碼器或轉移系統，而不是編碼器。

評估剖析器：PARSEVAL、UAS 與 LAS

直觀說法。比較剖析器承諾產生的表示，並檢查斷詞、標註或脈絡改變答案的情況。

最初的 PARSEVAL 程序先移除標點等材料，再比較候選剖析與樹庫剖析，計算**交叉括號**分數——候選括號與參考括號交叉的次數——以及**召回率**，即共同括號數除以參考括號數 [3]。今日標準是帶標籤括號的精確率、召回率與 F1 [2]：

$$P_{\text{lab}} = \frac{|\mathcal{B}(\hat{T}) \cap \mathcal{B}(T^{*})|}{|\mathcal{B}(\hat{T})|}, \qquad R_{\text{lab}} = \frac{|\mathcal{B}(\hat{T}) \cap \mathcal{B}(T^{*})|}{|\mathcal{B}(T^{*})|}.$$

其中 $\mathcal{B}(\mathcal{T})$ 是樹 $\mathcal{T}$ 的帶標籤成分 (i, j, ℓ) 集合，$\hat{\mathcal{T}}$ 是預測樹，$\mathcal{T}^*$ 是參考樹，F1 是兩者的調和平均。對我們的兩個剖析，兩棵樹只有一個帶標籤括號不同：名詞附接樹有 $(2, 5, NP)$，動詞附接樹有 $(1, 3, VP)$。計入預終結符時，每棵樹有九個帶標籤成分，因此任一棵對另一棵評分，精確率與召回率都是 8/9。應說明是否包含詞彙預終結符、標點與根；不同納入規則會改變分母。

對依存結構，**無標籤附接分數**（UAS）是中心詞正確的詞所占比例，**帶標籤附接分數**（LAS）還要求關係正確：

$$\mathrm{LAS} = \frac{1}{M} \sum_{t \in \mathcal{J}} \mathbf{1}[(\hat{h}_t, \hat{r}_t) = (h_t, r_t)].$$

其中 LAS 是帶標籤附接分數，$\mathcal{J}$ 是評估詞位置集合，$M = |\mathcal{J}|$ 是集合大小，$\hat{h}_t$ 與 $\hat{r}_t$ 是預測中心詞與關係，h_t 與 r_t 是參考中心詞與關係，$\mathbf{1}[\cdot]$ 是指示函數；UAS 則去掉關係條件。在程式中，`cameras` 的中心詞錯誤（兩個分數都扣分），`with` 的中心詞正確但標籤錯誤（只有 LAS 扣分），因此得到 0.80 與 0.60。

套用這些公式前，必須固定詞元對齊。CoNLL 2018 共享任務要求系統把原始文字剖析成多種語言的 Universal Dependencies，測試輸入不附任何標準標註 [14]。系統因此必須自行切分與斷詞，而預測詞必須先與標準詞對齊，附接分數才有意義。如果標準標註有兩個詞，斷詞器卻輸出一個，就無法正常解讀附接比較。應報告切分不符，並說明如何處理對齊，不要默默刪除困難樣本。

建立小型診斷配對：保持詞彙相近但移動修飾語，加入巢狀子句，把主動句改為被動句，或加入片段與混合語言片語。即使總體分數穩定，這些測試仍能揭露模型結構在什麼條件下改變。句子確實有歧義時，不要把一棵標準樹視為絕對確定。語料需要選定慣例或經裁決的分析，但其他分析可能也合理。對應用而言，應問分歧是否改變擷取事實或行動，讓剖析錯誤與實際後果連結。

語意角色標註：誰對誰做了什麼

直觀說法。同一個事件可以用不同語法表達。應擷取事件角色，不能假定第一個名字永遠是行動者。

簡單的雙參與者述詞可寫成：

$$\mathrm{repair}(\mathrm{Mina}, \mathrm{sensor}).$$

其中 repair 是關係名稱；第一個論元是修理者；第二個是被修理物件。「Mina 修好了感測器」與「感測器被 Mina 修好了」可以共用這種表示，雖然表面的主詞不同。語法提供線索，角色解讀還必須考慮述詞與句型 [1]。

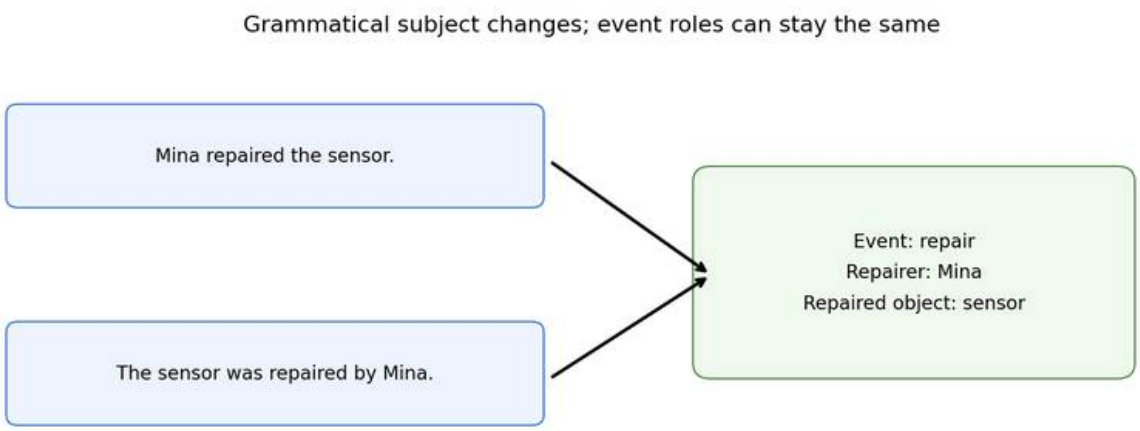


圖 *4* — 表面主詞可以改變，事件角色則保持相同。例句與紀錄皆為合成示例。

並非每個動詞都允許所有變換，也不是每個論元都明說。「感測器修好了」沒有指明修理者。若系統填入前文最近提到的人，就是做了上下文推論，而非直接擷取。這個推論可能合理，但需要另外的信心政策與證據契約。當應用未授權推論時，應保留明確的未知角色。

論元與修飾語也有差別。在「Mina 週二在臺北修好了感測器」中，修理者與被修理物件參與核心事件，地點與時間則補充事件。事件結構可包含四個欄位，但應說明哪些是必填。選填欄位能避免擷取器為滿足過度嚴格的格式而編造細節。

PropBank 角色與 SRL 任務

Gildea 與 Jurafsky 把自動語意角色標註建構為剖析句子上的統計任務：把帶角色的片語對應到剖析成分，從詞與樹抽取特徵，再估計每種可能角色的機率 [15]。**PropBank** 接著為 Penn Treebank 的動詞標註述詞專屬的編號論元：每個動詞義項有一個框架集，其中 `Arg0` 通常是施事般的參與者、`Arg1` 是受事或主題，更高編號則依動詞定義；附加語則用 `ArgM` 標籤，例如表示否定的 `ArgM-NEG` 與表示情態的 `ArgM-MOD` [16]。編號正是重點：在「an amendment offered by Rep. X」中，amendment 雖是語法主詞，仍是 *offer* 的 `Arg1` [16]。CoNLL-2005 共享任務以 PropBank 標準化 SRL 評估：預測論元只有在跨度與角色都正確時才算對，系統以論元層級的精確率、召回率與 F1 比較 [17]。

語意角色能連接不同句型，卻不保證各種資源的標註一致。有些資源使用述詞專屬角色，有些使用通用角色。合併訓練語料前，先比較定義與例子。機械式重新命名標籤，可能掩蓋工具、原因、受益者與隱含參與者的標註差異。

把 SRL 視為以述詞為條件的標記

He 等人把 SRL 寫成每個述詞各做一次的 BIO 序列標記：輸入標出哪個詞是述詞，深層 highway BiLSTM 搭配**受限解碼**——例如禁止沒有前置 `B-` 的 `I-` 標籤——標記論元跨度 [18]。他們的八層集成模型在 CoNLL-2005 測試集上報告 83.2 F1，並發現深層模型能找回長距離依存，卻仍會犯出

乎意料的明顯錯誤，句法資訊仍有幫助空間 [18]。換成預訓練編碼器後，同一設計就是NLP 11 的詞元標記任務頭再加上述詞指示。以下程式把角色跨度轉成標籤序列再轉回來，並拒絕無效序列。

```
tokens = "Mina said the sensor was repaired by Lena".split()
# One frame per predicate: (start, end) token spans, end exclusive.
frames = {
    "said":     {"ARG0": (0, 1), "ARG1": (2, 8)},
    "repaired": {"ARG1": (2, 4), "ARG0": (6, 8)},
}

def to_bio(frame, n):
    tags = ["O"] * n
    for role, (i, j) in frame.items():
        tags[i] = "B-" + role
        tags[i + 1:j] = ["I-" + role] * (j - i - 1)
    return tags

def from_bio(tags):
    spans, start, role = {}, None, None
    for k, tag in enumerate(tags + ["O"]):
        if start is not None and not tag.startswith("I-" + role):
            spans[role] = (start, k); start = None
        if tag.startswith("B-"):
            start, role = k, tag[2:]
        elif tag.startswith("I-") and start is None:
            raise ValueError(f"I- tag without B- at position {k}")
    return spans

for predicate, frame in frames.items():
    tags = to_bio(frame, len(tokens))
    tags[tokens.index(predicate)] = "B-V"
    print(f"{predicate:9s}", " ".join(tags))
    assert {r: s for r, s in from_bio(tags).items() if r != "V"} == frame
try:
    from_bio(["O", "I-ARG1", "I-ARG1"])
except ValueError as error:
    print("invalid:", error)
```

```
said      B-ARG0 B-V B-ARG1 I-ARG1 I-ARG1 I-ARG1 I-ARG1 I-ARG1
repaired  O O B-ARG1 I-ARG1 O B-V B-ARG0 I-ARG0
invalid: I- tag without B- at position 1
```

同一句話產生兩個框架。對 *said*，Mina 是說話者，整個轉述子句是 `ARG1`。對 *repaired*，Lena 是修理者，感測器雖是語法主詞，仍是 `ARG1`。只讀 *repaired* 框架的擷取器，會把完成的修理記成事實；*said* 框架則顯示修理是 Mina 的轉述。這正是下方關係紀錄必須保留的歸屬差別。

語意表示中的作用範圍

直觀說法。「every」、「some」與「not」這類小詞，對陳述的改變可能比名字還大。語意剖析器必須記錄它們修飾什麼，否則輸出回答的是錯誤問題。

NLP 02 以一個實作反模型介紹量詞範圍與否定。這裡看作用範圍對任務輸出代表什麼。「Every technician checked a sensor」有一種閱讀是每位技師可檢查不同感測器，另一種閱讀是有一個感測器被所有人檢查。「Mina did not check every sensor」的形式是

$$\neg \forall z\,[S(z) \Rightarrow C(m,z)] \quad \equiv \quad \exists z\,[S(z) \wedge \neg C(m,z)].$$

其中 z 遍歷實體，$S(z)$ 表示感測器，$C(m,z)$ 表示 Mina 檢查該實體，m 指 Mina，$\neg$ 表示否定，$\equiv$ 表示邏輯等價。一個反例感測器，就足以讓全稱陳述不成立。「Mina checked no sensor」則排除每一次成功檢查。否定的位置決定哪種證據能反駁陳述。

文件資料庫通常沒有完整知識：沒有擷取到檢查紀錄，可能是未報告，不代表沒發生。回答全稱問題時，這個差別很重要。觀察清單可支持「這些感測器已檢查」，但要成立「所有感測器都已檢查」，還需要完整的相關感測器清單與全面涵蓋。反之，找到一個明確未檢查的感測器，就足以反駁完整性。作用範圍推理與證據完整性必須一起考慮。同樣地，「沒有找到故障證據」描述觀察或搜尋結果，不應自動轉成「故障不可能發生」。

範圍有限的擷取器遇到不支援的作用範圍句型，應拒絕判定或送交覆核。能處理「沒有修理」的規則，仍可能誤解「沒有說 Mina 修理了」：系統必須知道被否定的是什麼，而這正是 PropBank 的 `ArgM-NEG` 附在特定述詞上的資訊 [16]。

組合意義是對角色與作用範圍進行計算

理解組合的一個實用方式，是問每個片語還缺什麼。`Mina` 指定一個參與者；`repaired` 要求特定角色的參與者；`repaired a sensor` 用存在條件處理受詞要求，仍留下行動者等待填入；結合主詞後才補完該要求。這種組合順序說明，只辨認相同三個內容詞還不夠，角色必須正確綁定 [1]。

「Mina repaired a sensor」的一種受限邏輯閱讀為：

$$\exists z\,[S(z) \wedge R(m,z)].$$

其中 z 遍歷可能實體，$S(z)$ 表示 z 是感測器，$R(m,z)$ 表示參與者 m 修復 z，m 指 Mina。存在量詞表示至少有一個合適實體，不會識別是哪個感測器，也不表示所有感測器都修好了。這個簡單閱讀刻意省略時態與事件時間；它們是額外語意欄位，不是符號暗中提供的事實。

「Only Mina repaired the sensor」限制其他可能參與者；「Mina tried to repair the sensor」描述嘗試，不保證修復完成；「Mina stopped repairing the sensor」改變事件的時間解讀。只有行動者與對象的紀錄，無法表達這些差異。這提供選擇表示的具體標準：列出必須回答的問題，檢查是否有兩個需要不同答案的來源陳述，卻被壓成同一紀錄。如果嘗試修復與完成修復都變成 `repair(Mina, sensor)`，這個結構就太粗，無法回答完成狀態。序列化已經造成資訊損失時，更強編碼器也無法避免。

關係、事件與來源追溯

直觀說法。關係連結實體，事件則以一次發生為中心組織參與者。兩者都需要證據，也都可能在實體區段全對的情況下出錯。

二元關係紀錄可寫成：

$$q = (r, h, t, s).$$

其中 q 是紀錄；r 是關係類型；h 是頭實體或提及；t 是尾實體或提及；s 指向支持證據。頭與尾只是次序慣例，不是語言學主張。雇用關係中，交換員工與雇主就改變了紀錄。結構描述必須定義方向，評估也必須檢查。

事件擷取在觸發詞或事件類型之外，還包含買方、賣方、物品、金額與時間等角色——也就是限縮在應用結構描述內的語意角色標註。交易句可能提到多個價格，只有其中一個是實際付款金額。共同選擇角色配置能避免部分矛盾，但若來源本身含糊，仍需要更多上下文。應用允許時，可保留多個解讀或送交覆核。

來源追溯應包含文件身分、來源版本與支持區段。沒有版本，位移可能指向後來編輯過的文件；沒有歸屬——就像上面的 *said* 框架——引述的指控會變成直接事實。這些是紀錄管線的性質，不是換大編碼器就會自動解決的問題。

應分階段評估——提及偵測、類別預測、關係選擇與最終受支持紀錄——並且除了正確提及，也要用預測提及評估關係：測試時提供完美輸入，可能掩蓋實際部署管線的失敗率。

篇章、提及與共指

直觀說法。意義會跨越句子延續。共指消解判斷哪些提及指向同一實體，讓關於「the pump」與「it」的事實落在同一筆紀錄上。

篇章（discourse）是跨子句或句子組織的語言。例如「幫浦變熱了。操作員停止了它。」介紹事件、行動者與物件指涉，可能暗示對升溫的回應，卻不一定證明升溫造成停止。比較「The pump stopped **because** the valve closed」、「The pump stopped **after** the valve closed」與「The pump stopped **although** the valve remained open」：第一句提出因果解釋，第二句排列事件先後而不建立因果，第三句標示與預期的對比 [1]。摘要器或擷取器加入「because」，就是在增加主張。關係還有作用範圍：在「Mina said the pump stopped because the valve closed」中，解釋可能屬於 Mina 的報告，而不是獨立建立的發現。

提及（mention）是指向實體、事件或概念的文字區段；先行詞（antecedent）是協助解讀後續提及的較早表達；共指（coreference）把指向相同實體的提及分組。措辭相似不保證身分相同，兩個不同幫浦都可能叫「備用幫浦」。

先行詞排序模型

Lee 等人的區段式共指模型，可替一個提及的可能先行詞評分 [19]。抽象形式為：

$$p(a_i = j \mid d) = \frac{\exp s(i, j, d)}{\sum_{k \in A_i} \exp s(i, k, d)}.$$

其中 d 是文件或對話；i 索引當前提及；A_i 是包含空選項的候選先行詞集合；a_i 是選到的先行詞變數；$s(i,j,d)$ 是學得相容性分數。空選項讓提及可以引入新實體或暫不連結。刪除候選會改變模型能考慮的範圍。Lee 等人的模型把各項貢獻分開 [19]：

$$s(i,j) = s_m(i) + s_m(j) + s_a(i,j) \quad (j \neq \epsilon), \qquad s(i,\epsilon) = 0.$$

其中 i 是目前候選區段，j 是較早候選區段，ϵ 是空先行詞，s_m 評分區段是否為提及，s_a 評分配對相容性。固定空選項分數為零，替真實連結提供共同基準。提及剪枝節省配對計算，卻可能在連結開始前，就移除唯一有效的先行詞。

模型既要偵測提及，也要連結。較早實體區段若已被丟掉，後續只改善剩餘候選分數也找不回它。診斷時應分開偵測召回與連結準確率。

群組讓監督更複雜。如果 `Pump A`、`the pump`、`it` 指同一實體，最後提及連到任一較早提及都正確。任意只獎勵最近一個的損失，會懲罰另一條有效連結。以群組為目標的訓練改為獎勵它們的合計機率：

$$\mathcal{L}_i = -\log \sum_{j \in G_i} p(a_i = j \mid d).$$

其中 $\mathcal{L}_i$ 是候選區段 i 的損失，G_i 是依標註與剪枝規則保留下來、且非空的有效先行詞集合，a_i 是先行詞變數，d 是文件，$p(a_i = j \mid d)$ 是條件先行詞機率。沒有有效較早先行詞時，空選項屬於 G_i。這是對有效連結取邊際，原始模型也使用這種方法 [19]。推論時，沿先行詞連結合併提及，形成實體群組。一條錯誤橋接就可能合併兩個原本各自合理的群組，錯誤合併的群組之後可能讓多個事件歸到錯誤幫浦。

新近性基準與它的失敗

以下手寫基準使用明確相容性與新近性，刻意保持小型以揭露失敗。裝置類別由資料提供，程式不會依姓名或社會刻板印象推測屬性。

```
entities = [
    {"id": "pump_a", "category": "device", "turn": 1},
    {"id": "valve_b", "category": "device", "turn": 2},
    {"id": "operator", "category": "person", "turn": 2},
]
def candidates(category):
    return [e for e in entities if e["category"] == category]
def recent(category):
    choices = candidates(category)
    return max(choices, key=lambda e: e["turn"])["id"] if choices else None
print("Recency baseline for it:", recent("device"))
assert recent("device") == "valve_b"
# In a synthetic annotation, the speaker meant the earlier pump.
intended = "pump_a"
assert recent("device") != intended
# A cautious application preserves ambiguity instead of hiding the choice.
possible = {e["id"] for e in candidates("device")}
resolution = next(iter(possible)) if len(possible) == 1 else None
assert resolution is None and possible == {"pump_a", "valve_b"}
```

基準輸出 `valve_b`，人工設定的說話意圖卻指較早幫浦。線索無法每次都確定身分。「你指 Pump A 還是 Valve B？」這種澄清能明確保留不確定性；NLP 08 會在對話中發展澄清。

上下文還包含指示語（deixis）：這個、這裡、現在等表達依賴說話者、地點、時間或介面選取，因此應把相關情境與話語一起存；系統拒絕輸入無法支持的推論，不應被判錯。引文也帶來限制：操作員引文中的 `I` 指操作員，「另一台幫浦」則預設一個新近性規則無法定義的對比集合。

評估共指：MUC、B^3、CEAF 與 CoNLL F1

直觀說法。共指輸出是把提及分割成實體。不同指標以不同單位——連結、提及或實體——計算兩個分割的一致程度，因此單一指標無法說明全貌。

令 K 為鍵（標準）分割，R 為回應分割。

- **MUC** 以連結為基礎 [20]。對每個鍵實體 S，令 $p(S)$ 為回應在 S 上誘導的分割，未匹配的提及視為單例。召回率為 $\sum_S (|S| - |p(S)|) / \sum_S (|S| - 1)$：回應保留的鍵連結數，除以所需的最少連結數。精確率則交換鍵與回應的角色。
- **B^3** 以提及為基礎。對每個提及 m，比較包含它的鍵實體 K_m 與回應實體 R_m：召回率對提及平均 $|K_m \cap R_m|/|K_m|$，精確率平均 $|K_m \cap R_m|/|R_m|$ [23]。
- **CEAF** 以實體為基礎 [21]。它在鍵實體與回應實體間找最佳一對一對齊，也就是最大二分匹配。以實體相似度 $\phi(K, R) = 2|K \cap R|/(|K| + |R|)$（實體式變體 CEAF_e）計算時，召回率是對齊相似度總和除以鍵實體數，精確率則除以回應實體數。

CoNLL-2012 共享任務以 MUC、B^3 與 CEAF_e 三者 F1 的**未加權平均**評分，理由是每個指標代表不同面向 [23]。Pradhan 等人後來發布參考評分程式，因為不一致的實作——特別是預測提及與標準提及不符時——產生了無法比較的數字 [22]。以下程式針對圖 5 的回應計算三者，其中「it」被錯誤併入閥門群組。

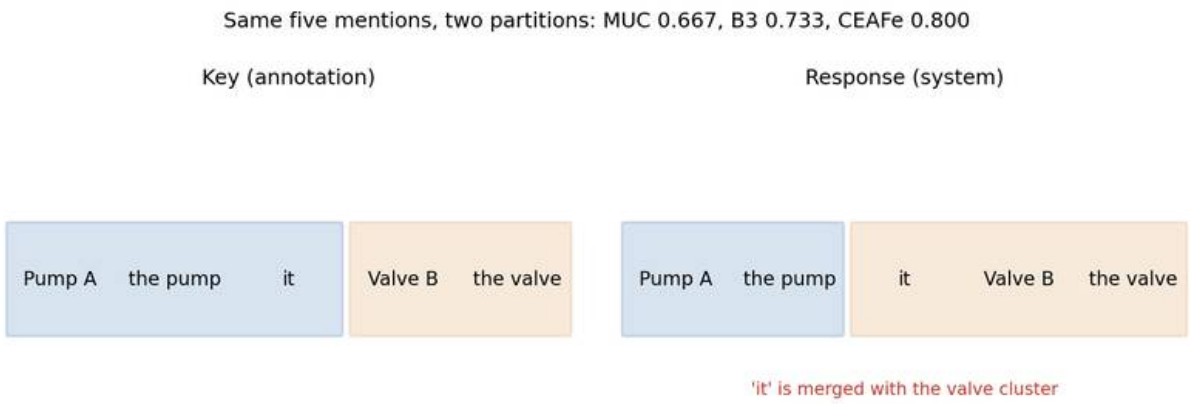


圖 5 — 一條錯誤連結，三種不同扣分。分數由下方程式計算。

```
import numpy as np
from scipy.optimize import linear_sum_assignment
```

```
# Mentions: a="Pump A", b="the pump", c="it", d="Valve B", e="the valve"
key = [{"a", "b", "c"}, {"d", "e"}]
response = [{"a", "b"}, {"c", "d", "e"}]          # "it" wrongly joined to the valve

def f1(p, r):
    return 2 * p * r / (p + r) if p + r else 0.0

def muc(gold, pred):                                  # link-based
    def recall(g, p):
        def parts(s):  # pieces of s cut by p; uncovered mentions are singletons
            covered = set().union(*p)
            return sum(1 for t in p if s & t) + len(s - covered)
        return sum(len(s) - parts(s) for s in g) / sum(len(s) - 1 for s in g)
    return recall(pred, gold), recall(gold, pred)    # precision, recall

def b_cubed(gold, pred):                              # mention-based
    def score(a, b):
        cluster_of = {m: s for s in b for m in s}
        return np.mean([len(s & cluster_of.get(m, set())) / len(s)
                        for s in a for m in s])
    return score(pred, gold), score(gold, pred)

def ceaf_e(gold, pred):                               # entity-based
    phi = np.array([[2 * len(g & p) / (len(g) + len(p)) for p in pred] for g in gold])
    rows, cols = linear_sum_assignment(-phi)
    total = phi[rows, cols].sum()
    return total / len(pred), total / len(gold)

scores = {name: f1(*fn(key, response)) for name, fn in
          [("MUC", muc), ("B3", b_cubed), ("CEAFe", ceaf_e)]}
for name, value in scores.items():
    print(f"{name:6s} F1 = {value:.3f}")
print(f"CoNLL F1 = {np.mean(list(scores.values())):.3f}")
```

```
MUC    F1 = 0.667
B3     F1 = 0.733
CEAFe  F1 = 0.800
CoNLL F1 = 0.733
```

手算 MUC：鍵需要 $2+1=3$ 條連結；回應把幫浦實體切成兩塊、閥門實體保持完整，保留 $1+1=2$ 條連結，因此召回率為 $2/3$，依對稱性精確率也是 $2/3$。B³ 對每個受影響的提及扣分，CEAF_e 則仍讓每個鍵實體對齊到大致正確的回應實體。一個錯誤、三個數字：這正是共享任務取平均的原因，也是應用應該同時檢查使用這些群組之紀錄的原因。

分數只在標註政策下可解讀：是否連結事件與聯想關係、單例是否計分，以及如何標註歧義。

語意剖析與 AMR

直觀說法。語意剖析把句子映射到程式能使用的意義表示：邏輯形式、資料庫查詢或一張圖。抽象意義表示（Abstract Meaning Representation，AMR）是廣泛使用的整句圖格式。

Banarescu 等人為「語意庫」設計 AMR：AMR 是有根、帶標籤的有向圖，便於人閱讀、也便於程式走訪；它試圖抽離句法上的特異性，讓基本意義相同的句子得到相同 AMR，並大量使用 PropBank 框架集 [24]。當一個實體扮演多個角色時，AMR 使用**再進入（re-entrancy）**——一個有多個父節點的節點——因此圖不是樹。在「the operator wants to stop the pump」中，operator 同時是 `want-01` 與 `stop-01` 的 `ARG0`。AMR 刻意不規定圖如何從詞推導出來，因此詞與節點的對齊是另一個問題 [24]。

Smatch 把每張圖視為三元組的合取——`instance(variable, concept)` 或 `relation(variable1, variable2)`——並在使匹配最多的變數對應下，報告匹配三元組的精確率、召回率與 F1，以此評估 AMR 剖析 [25]。找出該對應是 NP 完全問題，因此論文提供整數線性規劃方法，以及帶隨機重啟的爬山啟發法 [25]。對四個變數的圖，我們可以直接嘗試每種對應：

```
import itertools

# "the operator wants to stop the pump" vs a parse that loses who stops.
gold = {("instance", "w", "want-01"), ("instance", "o", "operator"),
        ("instance", "s", "stop-01"), ("instance", "p", "pump"),
        ("ARG0", "w", "o"), ("ARG1", "w", "s"),
        ("ARG0", "s", "o"), ("ARG1", "s", "p")}
pred = {("instance", "a", "want-01"), ("instance", "b", "operator"),
        ("instance", "c", "stop-01"), ("instance", "d", "pump"),
        ("ARG0", "a", "b"), ("ARG1", "a", "c"), ("ARG1", "c", "d")}

def rename(triples, mapping):
    return {(r, mapping.get(x, x), mapping.get(y, y)) for r, x, y in triples}

pred_vars, gold_vars = sorted({t[1] for t in pred}), sorted({t[1] for t in gold})
best = max(len(rename(pred, dict(zip(pred_vars, perm))) & gold)
           for perm in itertools.permutations(gold_vars))
precision, recall = best / len(pred), best / len(gold)
print("matched triples:", best)
print(f"Smatch P={precision:.3f} R={recall:.3f}
 ↪  F1={2*precision*recall/(precision+recall):.3f}")
```

```
matched triples: 7
Smatch P=1.000 R=0.875 F1=0.933
```

預測圖少了一個三元組——說明 operator 也是執行停止者的再進入 `ARG0` 邊。Smatch 仍給 0.933。對安全應用而言，「誰停止幫浦」可能就是全部問題。與本章其他彙總指標一樣，Smatch 應搭配針對關鍵關係的檢查。

預訓練模型如何改變這些任務

直觀說法。預訓練編碼器把剖析器、角色標註器與共指系統變成「編碼器加上小型結構解碼器」。它們沒有取消對解碼器、標註規範或謹慎評估的需求。

有三個明顯轉變。第一，**特徵被上下文向量取代**。Kitaev 與 Klein 的剖析器在使用預訓練詞表示後，從 93.55 提升到 95.13 F1 [6]；Joshi 等人把 BERT 用於共指，在 OntoNotes 上報告比先前最佳成績高 3.9 F1，並指出在文件層級脈絡、對話與提及改寫上仍有弱點 [26]。SpanBERT 則改變預訓練目標本身——遮住連續跨度，並訓練跨度邊界表示去預測跨度內容——報告 OntoNotes 共指 79.6% F1，以及 SQuAD 1.1 與 2.0 上的 94.6 與 88.7 F1 [27]。跨度形狀的任務受益於跨度形狀的預訓練。

第二，**結構有一部分其實存在於編碼器內部**。Hewitt 與 Manning 的結構探測找到一個線性轉換，使詞向量間的平方距離編碼剖析樹中的距離；這對 ELMo 與 BERT 成立，對基準則不成立 [28]。Tenney、Das 與 Pavlick 發現 BERT 以可解釋的順序在各層表示傳統管線的步驟——詞性標記、剖析、命名實體、語意角色，最後是共指——同時有時會依較高層資訊修正較低層的決策 [29]。這些是探測結果：它們顯示資訊可被線性取出，不表示模型像剖析器那樣使用它。

第三，**管線收斂成微調與提示**。編碼器為標記與樹提供證據，大型語言模型也可以依提示回傳結構 [30]。輸出驗證器仍有必要：看起來合理的括號字串，可能漏掉詞元、重排詞，或不形成連通樹。

現代觀點

綜述與其分類。 Zhang 的句法—語意剖析綜述聚焦於兩種最常見的形式化——成分剖析與依存剖析——回顧兩者以及語意依存圖的代表模型，並涵蓋跨領域、跨語言與聯合剖析，以及語料建置 [31]。Poesio 等人對回指計算模型的回顧，從神經網路之前的研究一路追溯到現今的神經模型，整理標註資料集，並討論橋接指涉與篇章指示等較少研究的現象 [32]。Chen 等人的 SRL 綜述已獲 Artificial Intelligence Review 接受，提出四維分類——模型架構、句法特徵建模、應用情境與多模態延伸——分析句法特徵何時有幫助，並探討 LLM 與專門 SRL 系統的互補角色 [34]。Zhao 等人的 LLM 綜述區分模型訓練、調適、使用與能力評估 [30]；結構可在其中扮演不同角色：監督輸出、內部表示探測、生成限制或診斷挑戰。Chang 等人的評估綜述再區分量測任務與使用的基準及程序 [33]。

近期原始研究（2023–2026）。 Tian、Xia 與 Song 發現 LLM 擅長淺層剖析（分塊），卻難以產生正確的完整成分樹，並以先提示分塊、再過濾分塊的方式改善 [35]。Hale 與 Stanojević 跨類型多樣的語言測試一項候選句法普遍性，發現所研究的模型只在部分較高資源語言中遵守它 [36]，說明以英文為主的結構測試不能證明一致的多語言能力。在共指方面，Le 與 Ritter 發現提示指令微調模型可以勝過無監督共指系統，但依賴高品質的提及偵測 [37]；Porada、Zou 與 Cheung 在五個共指系統間控制預訓練語言模型，發現在相同模型規模下，基於編碼器的模型勝過較新的基於解碼器的模型，而控制編碼器後，過去五年報告的 F1 提升大部分（但不是全部）消失 [38]。在語意表示方面，Jin 等人發現把 AMR 放進 LLM 思維鏈提示，在五個任務上大致弊多於利，錯誤集中在多詞表達、命名實體與最後推論步驟 [39]。

區分結構有效與結構泛化

這些研究引出兩種不同測試：第一，系統能否替這個標註規則產生合法結構？第二，選擇是否回應語法差異，而不是熟悉詞序？結構驗證器處理第一個問題，無法證明第二個。總是回傳某種慣常附接的模型，可以通過格式檢查，卻在受控比較中失敗。

建立詞彙幾乎不變、角色相關構造卻改變的配對（主動與被動、修飾語移位、關係子句），再固定構造並更換詞彙。如果不熟悉的名詞改變整個結構，詞彙涵蓋正在干擾；如果語法改變後解讀不變，熟悉詞彙模式主導了結果。最後檢查錯誤是否改變下游所需的語意角色。

較長上下文改變指涉決策，不只增加記憶大小

較長歷史可恢復缺少的先行詞，但也會加入相容候選；在先行詞 softmax 中，即使既有連結分數不變，新增候選仍會改變分母，因此以短歷史校準的門檻未必能沿用。比較短歷史、長但不相關的歷史，以及含另一台同類裝置的長歷史，才能區分有用脈絡與單純更多文字。

未解問題。標註框架各有差異；低資源語言缺少獨立檢查的樹；切分與語法交互影響；自發對話含有片段與修正；長文件與對話中的共指仍然困難；模型也可能學會基準慣例，卻缺乏穩健的組合能力。保留替代分析、測試結構最小配對、比較架構時控制編碼器 [38]，以及量測下游後果，是研究這些限制的具體做法。

重點整理

- 文法無法剖析是涵蓋範圍的結果，不是對說話者的評判；圖表共享跨度，但列舉所有樹仍可能指數成長。
- 最佳樹（最大值）與總機率（加總）回答不同問題；只有規則機率看不到詞彙證據，這正是神經跨度與弧評分器補上的部分。
- 轉移式剖析器做線性時間的局部決策；圖式剖析器替所有弧評分並需要樹解碼器，因為獨立選擇中心詞可能形成循環。
- 每種結構都有自己的指標——帶標籤括號、UAS ／ LAS、論元 F1、MUC ／ B^3 ／ CEAF、Smatch——且都需要說明分母與對齊政策。
- 語意角色在語法變換下保持不變；PropBank 依述詞編號論元，並把否定與情態標為附加語。
- 在擷取紀錄中保留作用範圍、歸屬與來源；過粗的結構描述會丟失任何編碼器事後都找不回的區別。
- 共指是偵測加上連結；一條錯誤橋接就可能合併實體，明確的歧義結果勝過自信的錯誤。
- 預訓練編碼器改進並簡化了這些系統，但有效性仍來自解碼器，評估仍來自標註規範。

練習

1. **延伸文法**。讓圖表剖析器接受「we saw cameras」與一個新介系詞，再加入未見名詞，說明需要哪個詞彙改變。

   提示

   既有二元規則已支援較短句子。終結符的涵蓋與片語組合是不同問題。

2. **辨識與列舉**。把圖表的樹清單改成布林類別，比較剖析器仍能回答什麼，以及丟掉什麼。接著修改 PCFG 程式，加入針對 `robots with` 的詞彙化規則分數，讓名詞附接在原始規則機率下勝出。

提示

不保留樹仍能判斷是否接受；要恢復不同附接，需要回溯指標。詞彙化版本可把 $NP \to NP\ PP$ 的分數乘上一個依中心名詞而定的因子。

3. **結構驗證器與非投射樹**。為基本依存樹檢查詞元輸出、每詞一個中心詞、沒有自我連結、唯一根與無循環。再為一個短句建構弧線會交叉的有效樹，並說明 arc-standard 神諭為何無法產生它。

提示

從每詞追蹤中心詞指標，應抵達根，且不重訪同一詞。arc-standard 只連結堆疊頂端兩項，因此它建立的每條弧都跨越一段連續、已完成的區域。

4. **兩種閱讀，兩筆紀錄**。寫出「A technician inspected every sensor」的兩種解讀。設計能儲存任一種閱讀的關係結構描述，以及從文件集合回答「每個感測器都檢查過了嗎？」的證據規則。

提示

比較存在量詞在全稱量詞外側與相反順序。答案需要感測器清單與涵蓋資訊，而不只是擷取到的檢查事件。

5. **共指指標**。建立一個把圖 5 五個提及全部併成一個實體的回應，以及一個讓五個提及都是單例的回應。執行程式前先算出兩者的 MUC、B^3 與 $CEAF_e$，再說明哪個指標對哪種錯誤最寬容。

提示

MUC 計算連結，所以單一巨大群組會保留每條鍵連結。B^3 對每個群組含錯誤成員的提及扣分。單例完全沒有連結。

6. **雙語指示語**。寫一則含有指示詞、相對日期與間接請求的雙語訊息，例如混用英文與中文。描述兩種會改變解讀的語境，並列出共指或擷取系統需要哪些證據才能區分。

提示

轉寄引述會改變 “I”、“here” 與 “tomorrow” 所依據的視角。能力測驗與維修對話，可以讓相同問句有不同目的。

參考文獻

1. Susan McRoy, *Principles of Natural Language Processing*, Chapters 3–5 and 7 and the chart-parsing appendix, open textbook, 2021. Open textbook
2. Daniel Jurafsky and James H. Martin, *Speech and Language Processing*, 3rd ed. draft (August 19, 2026), Chapters 19, 20, 22 and 24. Book site

3. E. Black, S. Abney, D. Flickenger, C. Gdaniec, R. Grishman, P. Harrison, D. Hindle, R. Ingria, F. Jelinek, J. Klavans, M. Liberman, M. Marcus, S. Roukos, B. Santorini and T. Strzalkowski, "A Procedure for Quantitatively Comparing the Syntactic Coverage of English Grammars," Speech and Natural Language Workshop, 1991. Paper
4. Mitchell P. Marcus, Beatrice Santorini and Mary Ann Marcinkiewicz, "Building a Large Annotated Corpus of English: The Penn Treebank," Computational Linguistics 19(2), 1993. Paper
5. Mitchell Stern, Jacob Andreas and Dan Klein, "A Minimal Span-Based Neural Constituency Parser," ACL, 2017. Paper
6. Nikita Kitaev and Dan Klein, "Constituency Parsing with a Self-Attentive Encoder," ACL, 2018. Paper
7. Nikita Kitaev, Steven Cao and Dan Klein, "Multilingual Constituency Parsing with Self-Attention and Pre-Training," ACL, 2019. Paper
8. Marie-Catherine de Marneffe, Christopher D. Manning, Joakim Nivre and Daniel Zeman, "Universal Dependencies," Computational Linguistics 47(2), 2021. Paper
9. Joakim Nivre, "Algorithms for Deterministic Incremental Dependency Parsing," Computational Linguistics 34(4), 2008. Paper
10. Danqi Chen and Christopher Manning, "A Fast and Accurate Dependency Parser using Neural Networks," EMNLP, 2014. Paper
11. Ryan McDonald, Fernando Pereira, Kiril Ribarov and Jan Hajič, "Non-Projective Dependency Parsing using Spanning Tree Algorithms," HLT-EMNLP, 2005. Paper
12. Eliyahu Kiperwasser and Yoav Goldberg, "Simple and Accurate Dependency Parsing Using Bidirectional LSTM Feature Representations," Transactions of the ACL 4, 2016. Paper
13. Timothy Dozat and Christopher D. Manning, "Deep Biaffine Attention for Neural Dependency Parsing," ICLR, 2017. Paper
14. Daniel Zeman, Jan Hajič, Martin Popel, Martin Potthast, Milan Straka, Filip Ginter, Joakim Nivre and Slav Petrov, "CoNLL 2018 Shared Task: Multilingual Parsing from Raw Text to Universal Dependencies," CoNLL Shared Task, 2018. Paper
15. Daniel Gildea and Daniel Jurafsky, "Automatic Labeling of Semantic Roles," Computational Linguistics 28(3), 2002. Paper
16. Martha Palmer, Daniel Gildea and Paul Kingsbury, "The Proposition Bank: An Annotated Corpus of Semantic Roles," Computational Linguistics 31(1), 2005. Paper
17. Xavier Carreras and Lluís Màrquez, "Introduction to the CoNLL-2005 Shared Task: Semantic Role Labeling," CoNLL, 2005. Paper
18. Luheng He, Kenton Lee, Mike Lewis and Luke Zettlemoyer, "Deep Semantic Role Labeling: What Works and What's Next," ACL, 2017. Paper
19. Kenton Lee, Luheng He, Mike Lewis and Luke Zettlemoyer, "End-to-end Neural Coreference Resolution," EMNLP, 2017. Paper
20. Marc Vilain, John Burger, John Aberdeen, Dennis Connolly and Lynette Hirschman, "A Model-Theoretic Coreference Scoring Scheme," MUC-6, 1995. Paper
21. Xiaoqiang Luo, "On Coreference Resolution Performance Metrics," HLT-EMNLP, 2005. Paper
22. Sameer Pradhan, Xiaoqiang Luo, Marta Recasens, Eduard Hovy, Vincent Ng and Michael Strube, "Scoring Coreference Partitions of Predicted Mentions: A Reference Implementation," ACL, 2014. Paper
23. Sameer Pradhan, Alessandro Moschitti, Nianwen Xue, Olga Uryupina and Yuchen Zhang,

"CoNLL-2012 Shared Task: Modeling Multilingual Unrestricted Coreference in OntoNotes," EMNLP-CoNLL Shared Task, 2012. Paper
24. Laura Banarescu, Claire Bonial, Shu Cai, Madalina Georgescu, Kira Griffitt, Ulf Hermjakob, Kevin Knight, Philipp Koehn, Martha Palmer and Nathan Schneider, "Abstract Meaning Representation for Sembanking," Linguistic Annotation Workshop, 2013. Paper
25. Shu Cai and Kevin Knight, "Smatch: an Evaluation Metric for Semantic Feature Structures," ACL, 2013. Paper
26. Mandar Joshi, Omer Levy, Luke Zettlemoyer and Daniel Weld, "BERT for Coreference Resolution: Baselines and Analysis," EMNLP-IJCNLP, 2019. Paper
27. Mandar Joshi, Danqi Chen, Yinhan Liu, Daniel S. Weld, Luke Zettlemoyer and Omer Levy, "SpanBERT: Improving Pre-training by Representing and Predicting Spans," Transactions of the ACL 8, 2020. Paper
28. John Hewitt and Christopher D. Manning, "A Structural Probe for Finding Syntax in Word Representations," NAACL-HLT, 2019. Paper
29. Ian Tenney, Dipanjan Das and Ellie Pavlick, "BERT Rediscovers the Classical NLP Pipeline," ACL, 2019. Paper
30. Wayne Xin Zhao et al., "A Survey of Large Language Models," Frontiers of Computer Science, 2026; online manuscript updated March 2026. Survey
31. Meishan Zhang, "A survey of syntactic-semantic parsing based on constituent and dependency structures," Science China Technological Sciences 63, 2020. Survey
32. Massimo Poesio, Juntao Yu, Silviu Paun, Abdulrahman Aloraini, Pengcheng Lu, Janosch Haber and Derya Cokal, "Computational Models of Anaphora," Annual Review of Linguistics 9, 2023. Survey
33. Yupeng Chang et al., "A Survey on Evaluation of Large Language Models," ACM Transactions on Intelligent Systems and Technology, 2024. Survey
34. Huiyao Chen, Meishan Zhang, Jing Li, Lilja Øvrelid, Jan Hajič, Hao Fei and Min Zhang, "A Systematic Survey of Semantic Role Labeling in the Era of Pretrained Language Models," arXiv: 2502.08660, 2025; accepted at Artificial Intelligence Review. Survey
35. Yuanhe Tian, Fei Xia and Yan Song, "Large Language Models are No Longer Shallow Parsers," ACL, 2024. Paper
36. John T. Hale and Miloš Stanojević, "Do LLMs learn a true syntactic universal?" EMNLP, 2024. Paper
37. Nghia T. Le and Alan Ritter, "Are Large Language Models Robust Coreference Resolvers?" arXiv:2305.14489, 2023. Paper
38. Ian Porada, Xiyuan Zou and Jackie Chi Kit Cheung, "A Controlled Reevaluation of Coreference Resolution Models," LREC-COLING, 2024. Paper
39. Zhijing Jin, Yuen Chen, Fernando Gonzalez Adauto, Jiarui Liu, Jiayi Zhang, Julian Michael, Bernhard Schölkopf and Mona Diab, "Analyzing the Role of Semantic Representations in the Era of Large Language Models," NAACL, 2024. Paper

篇 IV

圖形識別

PR 01・模式、機率與貝氏決策

建立辨識流程，區分似然與後驗機率，推導最小錯誤率、最小風險與拒絕決策，以及高斯判別函數與 ROC 分析。

網站版本: https://blog.shyandram.dev/zh-tw/learn/pattern-recognition/pr-01-bayes-decisions/

你將學到

- 把辨識問題拆成觀測、特徵、預測與行動，並先想清楚「一筆樣本」代表什麼。
- 區分類別先驗、類別似然與類別後驗，並看懂先驗改變時決策邊界如何移動。
- 推導最小錯誤率規則、任意損失矩陣下的最小風險規則，以及 Chow 的拒絕規則。
- 把分類器寫成判別函數，推導高斯情形下的線性（LDA）與二次（QDA）決策邊界。
- 讀懂 ROC 曲線與精確率–召回率曲線，連結 Neyman–Pearson 準則，並依成本選擇操作點。

先看全貌

相機拍下一個零件。系統量測它的形狀，估計它是否有缺陷，再決定要不要送去人工檢查。這些是不同的階段：拍到的像素是**觀測**（observation）；形狀量測值是**特徵**（feature）；缺陷機率是**預測**（prediction）；送去檢查則是**行動**（action）。貝氏決策理論處理的正是最後兩者之間的連結：在目前的信念之下，應該採取什麼行動？

> **用最簡單的話說**
>
> 分水果的機器看到一顆水果，先猜它熟了沒有。「猜」是一件事，「選籃子」是另一件事。如果箱子裡大部分水果都是熟的，即使這顆看起來有點怪，機器也該偏向「熟了」。如果丟掉一顆好水果的代價，遠大於請人看一眼，機器就應該偶爾說：「我不確定，請人看一下這顆。」貝氏決策理論，就是把這三件事算清楚：每一種有多常見、證據怎麼說、每一種錯誤有多痛。

本系列沿用經典的圖形識別教科書：Duda、Hart 與 Stork 的《Pattern Classification》[1]、Bishop 的《Pattern Recognition and Machine Learning》[2]，以及 Theodoridis 與 Koutroumbas 的《Pattern Recognition》[3]，統計觀點則參考 Hastie、Tibshirani 與 Friedman [4]。各章的範例與推導都是原創的，較長的證明請回到教科書。這些教科書都從貝氏決策理論開始，因為後面所有方法都可以拿它來衡量：一個分類器好不好，取決於它多接近「如果知道真實分布，貝氏規則會做的決定」。

辨識不限於影像。觀測也可以是一段錄音、一串筆畫或一份文件。分類預測類別，迴歸預測數值，分群則在沒有標籤時找出群組。本章談分類，但這套思考方式（先為不確定性建模，再選期望成本最低的行動）適用於三者。

先定義實驗

白話版。在寫任何公式之前，先寫下一筆樣本是什麼，以及資料是怎麼蒐集的。辨識系統許多出人意料的失敗都來自這一步，而不是分類器本身。

一張影像、一位病人、一段影片、影片中的一個裁切區域，是不同的**抽樣單位**（sampling unit）。如果二十個影格來自同一段錄影，把它們當成二十個獨立個案，會讓測試看起來比實際部署容易得多，因為測試影格幾乎就是訓練影格的複本。

精確版。辨識流程可以寫成

$$o \longrightarrow x = \phi(o) \longrightarrow p(c \mid x) \longrightarrow a(x).$$

其中 o 是原始觀測（影像、波形）；ϕ 是前處理與特徵抽取，把觀測映射成特徵向量 $x \in \mathbb{R}^d$；$c \in \{c_1, \dots, c_K\}$ 是 K 個類別之一；$p(c \mid x)$ 是模型對類別的信念；$a(x)$ 是最後採取的行動。學習階段用**訓練**樣本調整 ϕ 與機率模型的參數；**驗證**樣本用來選擇門檻、正則化等設定；保留的**測試**集只用一次，評估整個已選定的程序（PR 09 會仔細討論：驗證與集成方法）。

手工特徵（邊緣直方圖、形狀矩）與學習得到的嵌入（預訓練網路的輸出）佔據同一個位置 ϕ。換掉特徵抽取器，就改變了分類器能拿到的資訊。被裁切、量化或池化移除的資訊，下游任何分類器都救不回來。用決策理論的語言說：問題的貝氏錯誤率是相對於特徵而定義的；差的 ϕ 會抬高任何分類器都無法低於的下限。

貝氏定理：證據如何改變信念

白話版。貝氏定理告訴我們，如何利用「這個量測值在各類別中有多典型」，把「各類別有多常見」更新成「對這個物體而言各類別的可能性」。

精確版。令類別為 $c_1, \dots, c_K$。貝氏定理給出

$$P(c_k \mid x) = \frac{p(x \mid c_k)\, P(c_k)}{\sum_{j=1}^{K} p(x \mid c_j)\, P(c_j)}.$$

- **先驗**（prior）$P(c_k)$ 是看這筆觀測之前類別 c_k 的機率，例如產線上缺陷零件的比例。
- **似然**（likelihood，或類別條件密度）$p(x \mid c_k)$ 描述類別 c_k 內量測值的分布。
- **證據**（evidence）$p(x) = \sum_j p(x \mid c_j) P(c_j)$ 是 x 的整體密度；它對每個類別都相同，只負責正規化。
- **後驗**（posterior）$P(c_k \mid x)$ 是觀測到 x 之後類別 c_k 的機率。

連續量測的似然是密度，數值不一定介於零與一之間，也不是「恰好觀測到這個實數」的機率。類別後驗機率則落在 $[0, 1]$，並且對 k 加總為一。把兩個條件方向 $p(x \mid c)$ 與 $P(c \mid x)$ 搞混，就是經典的**基本率謬誤**（base-rate error），下面的程式範例會具體呈現。

因為證據是共用的，做決策只需要乘積 $p(x \mid c_k) P(c_k)$。圖 1 以兩個一維高斯類別說明其幾何意義：邊界落在兩條乘上先驗後的似然曲線交會之處。當類別 c_1 的先驗從 0.5 降到 0.1，它的加權曲線縮小，邊界往 c_1 的平均值移動：系統現在需要更強的證據，才會判定為較稀少的類別。

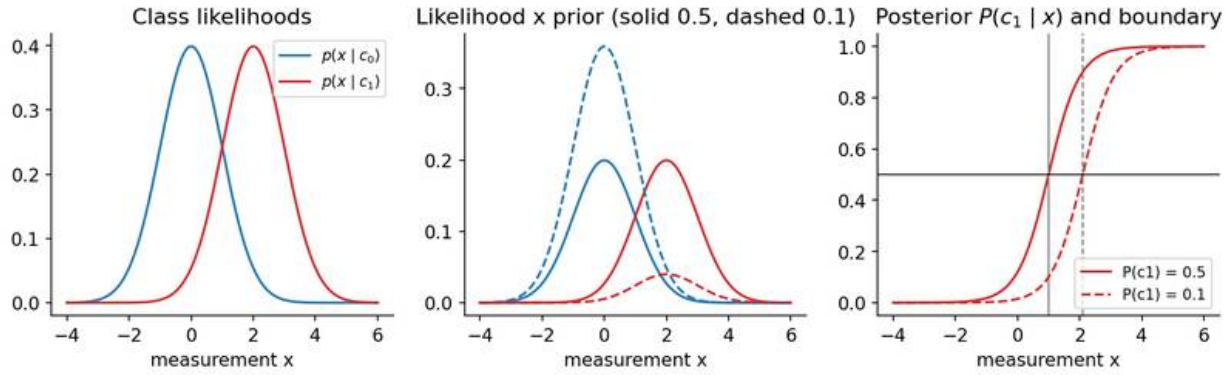


圖 *1* — 先驗改變時，似然（左）不變；乘上先驗的似然（中）與後驗（右）則會改變。$P(c1) = 0.5$ 時邊界在 $x = 1$；$P(c1) = 0.1$ 時移到 $1 + ln(9)/2 \approx 2.10$。

圖 1 的單位變異數情形有封閉解。平均值 $\mu_0 = 0$、$\mu_1 = 2$、變異數為 1 時，加權似然的對數比為 $2x - 2 + \ln \frac{P(c_1)}{P(c_0)}$，在 $x = 1 + \frac{1}{2} \ln \frac{P(c_0)}{P(c_1)}$ 處為零。先驗相等時是 $x = 1$，$P(c_1) = 0.1$ 時則是 $x \approx 2.10$。

程式碼：稀有事件改變後驗

這個 NumPy 範例使用假設的檢測統計：偵測器能標出 90% 的缺陷零件（靈敏度），也會誤標 8% 的良品（偽陽性率），而缺陷零件佔 2%。程式計算偵測器輸出陽性與陰性後的後驗，再比較最小錯誤率與最小風險的行動（後面幾節會推導）。

```
import numpy as np

prior = 0.02
sensitivity = 0.90
false_positive_rate = 0.08
p_positive = (sensitivity * prior /
              (sensitivity * prior + false_positive_rate * (1 - prior)))
p_negative = ((1 - sensitivity) * prior /
              ((1 - sensitivity) * prior + (1 - false_positive_rate) * (1 - prior)))
p = np.array([p_negative, p_positive])
c_fp, c_fn = 1.0, 9.0
threshold = c_fp / (c_fp + c_fn)
risks = np.column_stack([c_fn * p, c_fp * (1 - p)])
decisions = risks.argmin(axis=1)
assert np.array_equal(decisions, (p > threshold).astype(int))
print("posteriors [negative observation, positive observation]:", p)
print("minimum-risk actions:", decisions, "threshold:", threshold)
print("minimum-error actions:", (p > 0.5).astype(int))
```

輸出顯示：陰性後的後驗約為 0.0022，陽性後也只有 0.187。陽性觀測並不代表 90% 的缺陷機率：靈敏度以真實類別為條件，後驗則以觀測為條件；在 2% 的先驗下，大部分陽性標記其實來自數量龐大的良品。成本相同時，兩種觀測都不會判為「缺陷」（$0.187 < 0.5$）。若漏掉缺陷的代價是多做一次檢查的九倍，門檻降到 0.1，陽性標記就會送去檢查。機率模型沒變，變的只有決策規則。

最小錯誤率決策

白話版。如果每種錯誤的代價都一樣，就永遠選擇對這筆觀測而言機率最高的類別。平均而言，沒有其他規則能犯更少的錯。

精確版。決策規則 $a(x)$ 把每個 x 指派到一個類別，因而把特徵空間切成**決策區域** $\mathcal{R}_1, \dots, \mathcal{R}_K$，其中 $\mathcal{R}_k = \{x : a(x) = c_k\}$。錯誤機率為

$$P(\text{error}) = \int p(x)\,\bigl[1 - P(a(x) \mid x)\bigr]\,dx.$$

其中 $P(a(x) \mid x)$ 是所選類別的後驗。被積函數非負，而 $p(x)$ 與規則無關，所以只要在每個 x 上分別讓 $1 - P(a(x) \mid x)$ 最小，整個積分就最小，也就是選

$$a^*(x) = \arg\max_k P(c_k \mid x) = \arg\max_k p(x \mid c_k) P(c_k).$$

這就是**最大後驗（MAP）規則**，也稱**貝氏分類器**。它的錯誤率

$$P^*_{\text{err}} = \int p(x)\Bigl[1 - \max_k P(c_k \mid x)\Bigr]dx$$

稱為**貝氏錯誤率**（Bayes error）：在這組特徵下，任何分類器能達到的最低錯誤率。只有當各類別在特徵空間中完全不重疊時，它才是零。在貝氏錯誤率為 7% 的問題上，測試錯誤率 8% 已接近最佳；同樣的 8%，若貝氏錯誤率是 1%，就還有很大的改進空間。真實任務的貝氏錯誤率通常未知，但這個概念能把兩個問題分開：「特徵的資訊夠不夠？」（貝氏錯誤率），以及「分類器有沒有把特徵中的資訊榨乾？」（與下限之間的差距）。

兩類別時，MAP 規則成為**似然比檢定**（likelihood-ratio test）：

$$\Lambda(x) = \frac{p(x \mid c_1)}{p(x \mid c_0)} \;\gtrless\; \frac{P(c_0)}{P(c_1)}.$$

比值超過右邊的門檻就選 c_1，否則選 c_0。資料只透過 $\Lambda(x)$ 進入決策；先驗以及接下來會看到的成本，只會移動門檻。這就是同一個分數可以服務許多不同決策政策的原因。

損失矩陣與最小風險

白話版。有些錯誤比其他錯誤嚴重。為每一種錯誤訂一個成本，在目前信念下計算每個可能行動的平均成本，然後採取最便宜的行動。

精確版。令可用行動為 $\alpha_1, \dots, \alpha_A$（通常是「判為類別 i」，也可能包含「拒絕」）。**損失矩陣** $\lambda_{ij} = L(\alpha_i, c_j)$ 給出真實類別為 c_j 時採取行動 α_i 的成本。一個行動的**條件風險**（conditional risk）是它在後驗下的期望損失：

$$R(\alpha_i \mid x) = \sum_{j=1}^{K} \lambda_{ij}\, P(c_j \mid x), \qquad a^*(x) = \arg\min_i R(\alpha_i \mid x).$$

整體風險 $R = \int R(a(x) \mid x)\, p(x)\, dx$ 可用與前面相同的逐點論證最小化，所以這條**貝氏決策規則**達到可能的最小期望損失，即**貝氏風險** [1, 2]。在**零一損失**（$i = j$ 時 $\lambda_{ij} = 0$，否則為 1）下，$R(\alpha_i \mid x) = 1 - P(c_i \mid x)$，規則化約為 MAP。

兩類別時，記 λ_{10} 為真實是 c_0 卻判為 c_1 的成本（偽陽性），λ_{01} 為真實是 c_1 卻判為 c_0 的成本（偽陰性）。當 $R(\alpha_1 \mid x) < R(\alpha_0 \mid x)$ 時選 c_1 較便宜，整理後得到

$$\frac{p(x \mid c_1)}{p(x \mid c_0)} > \frac{\lambda_{10} - \lambda_{00}}{\lambda_{01} - \lambda_{11}} \cdot \frac{P(c_0)}{P(c_1)}.$$

成本與先驗在門檻中是相乘的：把偽陰性的成本加倍，對決策的影響和把陽性類別的先驗加倍完全相同。若正確決策不花成本（$\lambda_{00} = \lambda_{11} = 0$），記 $C_{\text{FP}} = \lambda_{10}$、$C_{\text{FN}} = \lambda_{01}$、$p = P(c_1 \mid x)$，規則就成為後驗上的門檻：

$$C_{\text{FP}}(1 - p) < C_{\text{FN}}\, p \quad \Longleftrightarrow \quad p > \frac{C_{\text{FP}}}{C_{\text{FP}} + C_{\text{FN}}}.$$

漏掉缺陷的代價是多做一次檢查的九倍時，門檻是 0.1，而不是 0.5（圖 2 右）。

實務上有兩點很重要。第一，損失矩陣是一個建模決定，常常由機器學習團隊以外的人（臨床醫師、安全工程師、產品負責人）決定；請明確寫下來，讓人可以審查。第二，後驗門檻只有在後驗**經過校準**（calibrated）時才有意義，也就是說，被給予機率 0.1 的個案中，真的大約有 10% 是陽性。許多現代分類器並非如此，這是下方「現代觀點」的主題之一。

拒絕選項

白話版。系統不確定時，可以把案例交給人處理。拒絕判斷有固定成本；做出判斷的期望成本，則在後驗分散時很高。當判斷的期望成本高於拒絕成本時，就拒絕。

精確版。加入一個行動 α_{reject}，不論真實類別為何，損失都是常數 C_R；K 個分類行動仍用零一損失。最佳分類行動的風險為 $1 - \max_k P(c_k \mid x)$，因此貝氏規則成為 **Chow 規則** [5]：

$$a^*(x) = \begin{cases} \arg\max_k P(c_k \mid x) & \text{if } \max_k P(c_k \mid x) \geq 1 - C_R, \\ \text{reject} & \text{otherwise.} \end{cases}$$

若拒絕成本為 0.2，系統只在最大後驗至少為 0.8 時才接受（圖 2 左）。兩個極端有助於建立直覺：若 $C_R = 0$（請人看完全免費），系統會拒絕所有不確定的案例；若 $C_R \geq 1 - 1/K$，系統永遠不拒絕，因為最大後驗至少是 $1/K$。

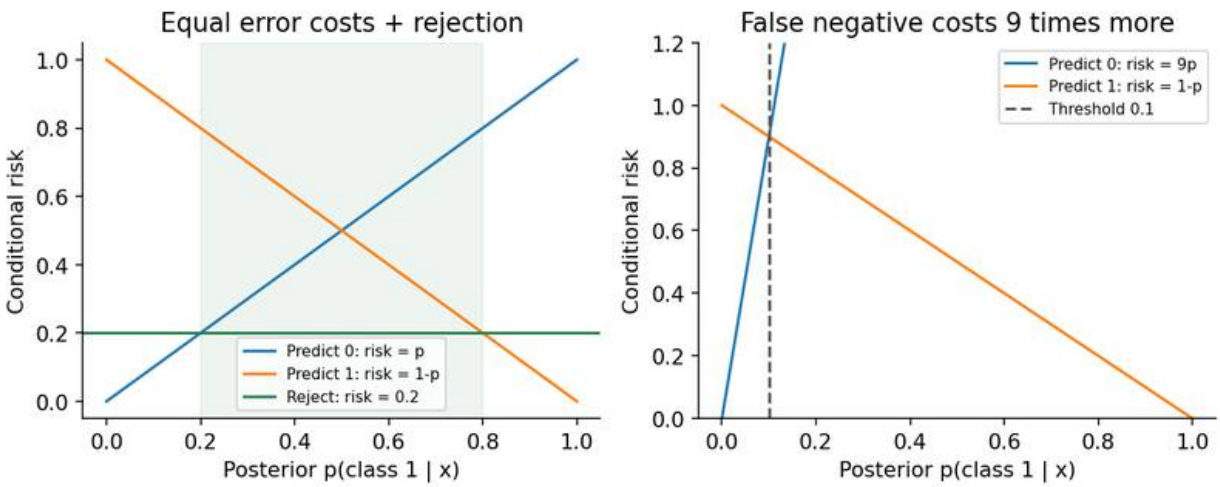


圖 *2* — 條件風險隨後驗變化。左：成本相同，再加上成本 *0.2* 的拒絕行動；後驗介於 *0.2* 與 *0.8* 時拒絕最便宜。右：偽陰性成本是偽陽性的九倍，決策門檻移到 *0.1*。

Chow 指出，掃過門檻會畫出一條**錯誤–拒絕權衡曲線**：每多拒絕一個百分點的樣本，就能降低剩餘樣本的錯誤，但效益遞減 [5]。圖 3 以三個重疊的高斯類別示範，它們的真實後驗是已知的。不拒絕時約有四分之一樣本被分錯；拒絕最不確定的 40% 後，全部樣本中的錯誤降到約 8%。

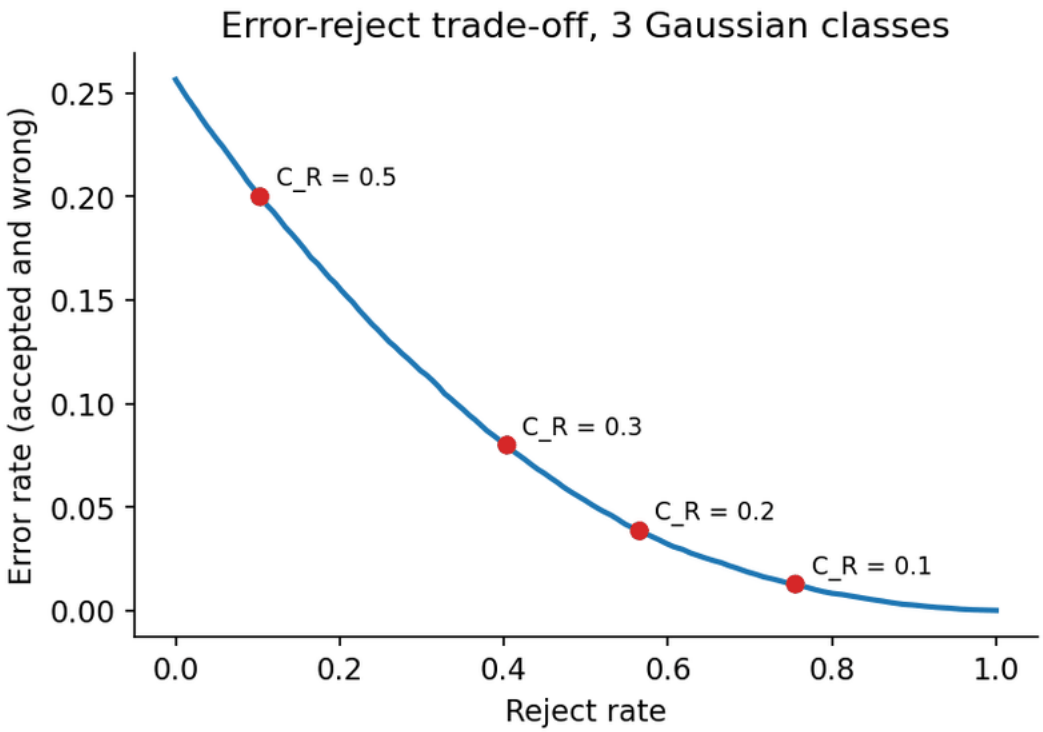


圖 *3* — 三個二維高斯類別（後驗已知，*20,000* 筆合成樣本）的錯誤–拒絕權衡。每個點對應一個拒絕成本 *C_R* 下的 *Chow* 規則；縱軸計算被接受且分錯的樣本比例。

程式碼：Chow 規則

```
import numpy as np

rng = np.random.default_rng(2)
means = np.array([[0, 0], [2, 0], [1, 1.7]])
y = rng.integers(0, 3, 20000)
X = means[y] + rng.normal(0, 1, (len(y), 2))

# exact posteriors: equal priors, identity covariance
logits = -0.5 * ((X[:, None, :] - means[None]) ** 2).sum(-1)
post = np.exp(logits - logits.max(1, keepdims=True))
post /= post.sum(1, keepdims=True)
pred, pmax = post.argmax(1), post.max(1)

for c_r in [0.5, 0.3, 0.2, 0.1]:
    accept = pmax >= 1 - c_r                    # Chow's rule
    reject_rate = 1 - accept.mean()
    err_accepted = np.mean(pred[accept] != y[accept])
    risk = np.mean(np.where(accept, pred != y, c_r))
    print(f"C_R={c_r}: reject {reject_rate:.3f}, "
          f"error on accepted {err_accepted:.3f}, average loss {risk:.3f}")
```

在這個亂數種子下，$C_R = 0.3$ 約拒絕 40% 的樣本，被接受的樣本中錯誤約為 13.5%；$C_R = 0.1$ 約拒絕 76%，剩下約 4.4% 的錯誤。平均損失隨 C_R 下降而下降，因為拒絕本身變便宜了。取指數之前先減去最大 logit（`logits.max`），是讓 softmax 數值穩定的標準技巧。

只有當不確定性估計有意義時，拒絕才有幫助。Chow 規則處理的是**模糊性**（ambiguity）：落在已知類別之間的觀測。它處理不了**新穎性**（novelty）：訓練時從未見過的物體，可能離每個類別都很遠，卻仍得到很有自信的後驗，因為後驗只在已知類別之間正規化。偵測這類輸入是開放集辨識與分布外偵測的課題（見「現代觀點」）。

判別函數與決策區域

白話版。分類器可以寫成一組評分函數，每個類別一個：算出所有分數，選最大的。許多不同的分數會給出相同的決策。

精確版。分類器由**判別函數**（discriminant function）$g_1(x), \dots, g_K(x)$ 以及規則「若對所有 $j \neq k$ 都有 $g_k(x) > g_j(x)$，就把 x 判給 c_k」定義 [1]。取 $g_k(x) = P(c_k \mid x)$ 就得到貝氏最小錯誤率分類器，但對所有判別函數套用任何**單調遞增**函數 f，每個決策都不變。因此以下三種選擇等價：

$$g_k(x) = P(c_k \mid x), \qquad g_k(x) = p(x \mid c_k)P(c_k), \qquad g_k(x) = \ln p(x \mid c_k) + \ln P(c_k).$$

實務上用的是對數形式：它把許多小密度的乘積變成加總，避免數值下溢。最小風險時則用 $g_k(x) = -R(\alpha_k \mid x)$。

c_i 與 c_j 之間的**決策邊界**是 $g_i(x) = g_j(x)$、且兩者都大於其他所有分數的點集合。兩類別時只需要一個函數，稱為**二分器**（dichotomizer）：

$$g(x) = g_1(x) - g_0(x) = \ln \frac{p(x \mid c_1)}{p(x \mid c_0)} + \ln \frac{P(c_1)}{P(c_0)}, \qquad \text{decide } c_1 \text{ if } g(x) > 0.$$

這是對數似然比加上對數先驗勝算。它也等於後驗的 **logit**，因為 $P(c_1 \mid x) = \sigma(g(x))$，其中 $\sigma(z) = 1/(1+e^{-z})$ 是邏輯斯 sigmoid 函數。請記住這個恆等式：在 PR 03 中，邏輯斯迴歸直接把 $g(x)$ 建模成線性函數，而不是從密度推導。

判別函數的觀點很有用，因為它把分類器計算什麼（邊界的形狀）和它如何得到（密度、直接擬合、間隔）分開。線性判別函數 $g_k(x) = w_k^\top x + w_{k0}$ 產生超平面邊界與凸的決策區域；二次判別函數在二維中產生圓錐曲線，一般情形則是超二次曲面。高斯情形正好說明各自何時出現。

高斯情形：LDA 與 QDA 邊界

白話版。如果每個類別都像一團多變量高斯形狀的「雲」，貝氏規則就是在量測 x 與每團雲中心的距離，而且單位會依照雲的形狀調整。所有雲形狀相同時，邊界是直的；形狀不同時，邊界會彎曲。

精確版。假設 $p(x \mid c_k) = \mathcal{N}(x; \mu_k, \Sigma_k)$，平均值 $\mu_k \in \mathbb{R}^d$，共變異數矩陣 $\Sigma_k \in \mathbb{R}^{d \times d}$（對稱、正定）：

$$\mathcal{N}(x; \mu, \Sigma) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\!\Big(-\tfrac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)\Big),$$

其中 $|\Sigma|$ 是行列式。去掉所有類別共用的常數 $-\frac{d}{2}\ln 2\pi$ 後，對數判別函數為

$$g_k(x) = -\tfrac{1}{2}\,(x-\mu_k)^\top \Sigma_k^{-1}(x-\mu_k) - \tfrac{1}{2}\ln|\Sigma_k| + \ln P(c_k).$$

二次式 $\Delta_k^2(x) = (x-\mu_k)^\top \Sigma_k^{-1}(x-\mu_k)$ 是 x 到 μ_k 的**馬氏距離**（Mahalanobis distance）平方：先把類別「白化」成單位球，再量歐氏距離。圖 4 的三種情形涵蓋了所有可能 [1, 4]。

情形 1：所有 k 都有 $\Sigma_k = \sigma^2 I$。特徵互不相關，變異數皆為 σ^2。展開 $\|x-\mu_k\|^2$ 並丟掉與類別無關的 $x^\top x$，得到線性判別函數

$$g_k(x) = w_k^\top x + w_{k0}, \qquad w_k = \frac{\mu_k}{\sigma^2}, \qquad w_{k0} = -\frac{\mu_k^\top \mu_k}{2\sigma^2} + \ln P(c_k).$$

類別 i 與 j 的邊界是超平面 $w^\top(x - x_0) = 0$，其中 $w = \mu_i - \mu_j$，且

$$x_0 = \tfrac{1}{2}(\mu_i + \mu_j) - \frac{\sigma^2}{\|\mu_i - \mu_j\|^2} \ln \frac{P(c_i)}{P(c_j)}\,(\mu_i - \mu_j).$$

邊界垂直於兩平均值的連線。先驗相等時它通過中點，分類器正是IP 13 的**最小距離（最近平均值）分類器**。先驗不等時，平面會往遠離較可能類別平均值的方向滑動。

情形 2：所有類別共用 $\Sigma_k = \Sigma$。二次項 $x^\top \Sigma^{-1} x$ 同樣是共用的而被消去，剩下

$$w_k = \Sigma^{-1}\mu_k, \qquad w_{k0} = -\tfrac{1}{2}\mu_k^\top \Sigma^{-1}\mu_k + \ln P(c_k).$$

邊界仍是超平面 $w^\top(x - x_0) = 0$，其中 $w = \Sigma^{-1}(\mu_i - \mu_j)$，但不再垂直於平均值差：它會傾斜，順著共用的相關結構（圖 4 中）。這就是機率形式的**線性判別分析**（linear discriminant analysis,

LDA）。先驗相等時，它是最小馬氏距離分類器。方向 $\Sigma^{-1}(\mu_1 - \mu_0)$ 與 Fisher 在 1936 年由投影準則推得的方向相同，PR 03 會重新推導。

情形 3：任意 Σ_k。沒有任何項會被消去：

$$g_k(x) = x^\top W_k x + w_k^\top x + w_{k0}, \quad W_k = -\tfrac{1}{2}\Sigma_k^{-1}, \quad w_k = \Sigma_k^{-1}\mu_k, \quad w_{k0} = -\tfrac{1}{2}\mu_k^\top\Sigma_k^{-1}\mu_k - \tfrac{1}{2}\ln|\Sigma_k| + \ln P(c_k).$$

邊界是**超二次曲面**：在二維中可能是直線、拋物線、橢圓或雙曲線。這就是**二次判別分析**（quadratic discriminant analysis, QDA）。在圖 4（右）中，緊密的藍色類別被一個橢圓包住，因為在任何方向上離藍色平均值很遠的點，都更能由分布寬廣的紅色類別解釋。決策區域甚至可以不連通，這在線性判別函數中不會發生。

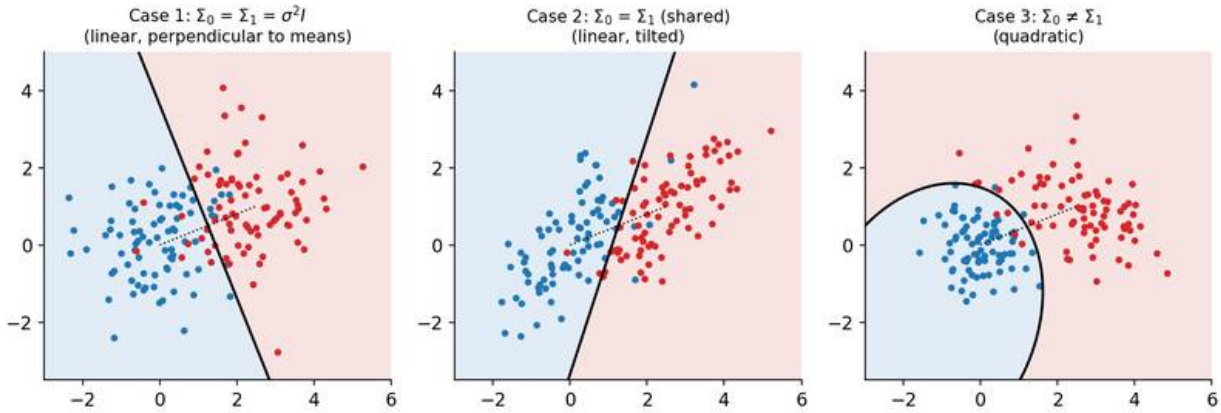


圖 *4* — 先驗相等的兩個高斯類別的貝氏邊界（黑線）。情形 *1*：共用球形共變異數，得到垂直平分線。情形 *2*：共用且相關的共變異數讓直線傾斜（*LDA*）。情形 *3*：共變異數不同，得到二次邊界（*QDA*）。點線連接兩個平均值。

兩類別共用共變異數且先驗相等時，貝氏錯誤率有封閉解：

$$P^*_{\text{err}} = \Phi\!\left(-\frac{\Delta}{2}\right), \qquad \Delta = \sqrt{(\mu_1 - \mu_0)^\top\Sigma^{-1}(\mu_1 - \mu_0)},$$

其中 Φ 是標準常態累積分布函數，Δ 是兩平均值之間的馬氏距離。即使兩類別相距三個標準差（$\Delta = 3$），仍有 $\Phi(-1.5) \approx 6.7\%$ 無法避免的錯誤。在高斯玩具資料上訓練分類器時，這條公式是很好的合理性檢查。

該用哪一種情形？這是偏差與變異數的取捨。每個類別的完整共變異數有 $d(d+1)/2$ 個自由元素；$d = 100$ 時，每類別就是 5,050 個數字，所需的樣本遠多於許多資料集能提供的量。共用一個共變異數（LDA）、限制為對角矩陣（即高斯樸素貝氏），或往簡單目標收縮，都是以彈性換取穩定。如何估計這些參數，是下一章PR 02：密度估計的主題。

程式碼：從零實作高斯判別函數

這個範例擬合高斯類別模型，用 Cholesky 分解計算對數判別函數（絕不顯式求反矩陣），並在圖 4 情形 3 的資料上，確認決策與 scikit-learn 的 `LinearDiscriminantAnalysis`、`QuadraticDiscriminantAnalysis` 一致。

```
import numpy as np
from sklearn.discriminant_analysis import (LinearDiscriminantAnalysis,
                                           QuadraticDiscriminantAnalysis)

rng = np.random.default_rng(0)
means = [np.array([0.0, 0.0]), np.array([2.5, 1.0])]
covs = [np.array([[0.4, 0.0], [0.0, 0.4]]), np.array([[2.0, -0.6], [-0.6, 1.0]])]
X = np.vstack([rng.multivariate_normal(m, S, 300) for m, S in zip(means, covs)])
y = np.repeat([0, 1], 300)

def fit_gaussians(X, y, shared):
    params = []
    pooled = sum(np.cov(X[y == k].T, bias=True) * (y == k).sum()
                 for k in (0, 1)) / len(y)
    for k in (0, 1):
        Xk = X[y == k]
        S = pooled if shared else np.cov(Xk.T, bias=True)
        params.append((Xk.mean(0), S, len(Xk) / len(y)))
    return params

def discriminants(X, params):
    g = []
    for mu, S, prior in params:
        L = np.linalg.cholesky(S)               # S = L L^T
        z = np.linalg.solve(L, (X - mu).T)      # whitened residuals
        maha = (z ** 2).sum(0)                  # squared Mahalanobis distance
        logdet = 2 * np.log(np.diag(L)).sum()
        g.append(-0.5 * maha - 0.5 * logdet + np.log(prior))
    return np.stack(g, axis=1)

for shared, ref in [(True, LinearDiscriminantAnalysis(store_covariance=True)),
                    (False, QuadraticDiscriminantAnalysis())]:
    mine = discriminants(X, fit_gaussians(X, y, shared)).argmax(1)
    theirs = ref.fit(X, y).predict(X)
    name = "LDA (shared)" if shared else "QDA (separate)"
    print(f"{name}: training error {np.mean(mine != y):.3f}, "
          f"agrees with scikit-learn on {np.mean(mine == theirs):.1%} of points")
```

兩個手寫分類器在每一個點上都與 scikit-learn 一致。這組資料的真實共變異數不同，QDA 的訓練錯誤率（4.3%）低於 LDA（5.5%）。不過這是訓練錯誤率：在高維且樣本少時，保留資料上的排序經常反過來。

ROC 分析與 Neyman–Pearson 觀點

白話版。一個兩類別分數可以在許多門檻上切開。每個門檻都在「抓到更多陽性」與「發出更多假警報」之間取捨。ROC 曲線一次畫出所有取捨，讓你等到成本或先驗確定之後再挑門檻。

精確版。令 $s(x)$ 為任何「越大越偏向 c_1」的分數（對數似然比、後驗、網路的 logit），當 $s(x) > t$ 時判為 c_1。對每個門檻 t 定義

$$\mathrm{TPR}(t) = P(s(x) > t \mid c_1), \qquad \mathrm{FPR}(t) = P(s(x) > t \mid c_0).$$

真陽性率（靈敏度、召回率）是被抓到的陽性比例；偽陽性率（1− 特異度）是被誤標的陰性比例。**接收者操作特徵**（receiver operating characteristic, ROC）曲線在 t 從 $+\infty$ 掃到 $-\infty$ 時，畫出 TPR 對 FPR 的關係 [7]。無用的分數落在對角線上；完美的分數會通過左上角。因為兩個比率都以真實類別為條件，類別先驗改變時，ROC 曲線不會改變。

Neyman–Pearson 準則。有時成本很難說清楚，但限制條件很容易講：「假警報最多 1%」。Neyman–Pearson 引理 [6] 指出：在所有滿足 $\mathrm{FPR} \le \alpha$ 的決策規則中，TPR 最高的是似然比檢定 $\Lambda(x) > \eta$，其中 η 選成讓偽陽性率恰好等於 α（必要時在邊界上隨機化）。所以似然比的 ROC 曲線是可達到的最佳曲線；其他任何分數的 ROC 都在它之下或與之重合。貝氏規則與 Neyman–Pearson 規則都是似然比檢定，差別只在門檻怎麼選。

依成本選擇操作點。在門檻 t 下，每筆樣本的期望成本為

$$\mathbb{E}[\text{cost}] = C_{\mathrm{FP}}\, P(c_0)\, \mathrm{FPR}(t) + C_{\mathrm{FN}}\, P(c_1)\, (1 - \mathrm{TPR}(t)).$$

在 ROC 空間中，成本相同的點落在斜率為 $C_{\mathrm{FP}}P(c_0)/(C_{\mathrm{FN}}P(c_1))$ 的**等成本直線**上；最佳操作點是這種直線與曲線相切之處 [7]。對似然比分數而言，切點正是先前推導的貝氏門檻。

曲線下面積。 AUC 用一個數字概括整條曲線。它等於隨機抽一個陽性樣本與一個陰性樣本，陽性得分較高的機率（平手算一半）[7]。對兩個相距 d' 個標準差、單位變異數的高斯分數分布，$\mathrm{AUC} = \Phi(d'/\sqrt{2})$：圖 5 中 $d' = 2$ 得到 0.921。AUC 忽略成本、先驗與校準，所以兩個 AUC 相同的分類器，在實際部署的門檻上成本可能差很多。

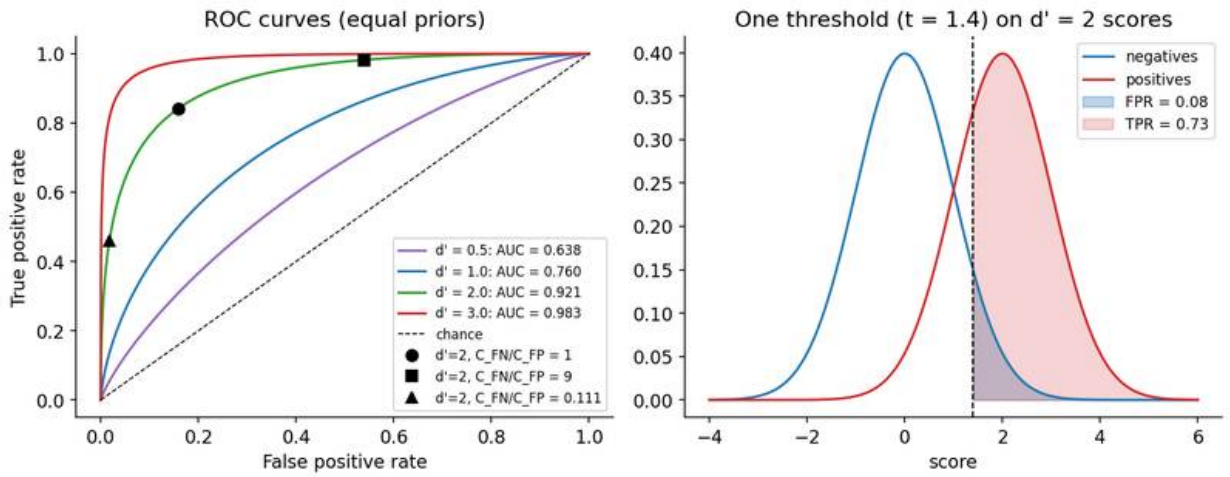


圖 5 — 左：等變異數高斯分數的 *ROC* 曲線；標記為先驗相等時，$d' = 2$ 曲線上三種成本比的貝氏操作點。右：在 $d' = 2$ 的分數上取一個門檻，以及它產生的 *FPR* 與 *TPR*。

精確率–召回率曲線。當陽性很稀少（詐欺、缺陷、偵測中的小物體）時，即使 FPR 很小，假警報的絕對數量仍可能很多，因為陰性實在太多。精確率 $= \mathrm{TP}/(\mathrm{TP} + \mathrm{FP})$ 取決於先驗，能把這件事暴露出來。Davis 與 Goadrich [8] 證明：一條曲線在 ROC 空間中支配另一條，若且唯若它在精確率–召回率空間中也支配；他們也指出 PR 點之間不能用直線內插。Saito 與 Rehmsmeier [9] 以模擬和已發表的研究說明，在不平衡資料上 PR 圖比 ROC 更有資訊量。物件偵測基準因此報告平均精確率（average precision）（見DLCV 03）。

程式碼：ROC、AUC 與成本最佳門檻

```
import numpy as np
from sklearn.metrics import roc_auc_score, roc_curve

rng = np.random.default_rng(1)
n_neg, n_pos = 9000, 1000                       # prevalence 10%
scores = np.r_[rng.normal(0, 1, n_neg), rng.normal(2, 1, n_pos)]
labels = np.r_[np.zeros(n_neg), np.ones(n_pos)]

fpr, tpr, thr = roc_curve(labels, scores)
auc = roc_auc_score(labels, scores)
# Mann-Whitney view of AUC: P(score_pos > score_neg)
pos, neg = scores[labels == 1], scores[labels == 0]
auc_pairs = (pos[:, None] > neg[None, :]).mean()
print(f"AUC = {auc:.4f}, pairwise estimate = {auc_pairs:.4f}")

# choose the operating point with minimum expected cost per sample
c_fp, c_fn, prior = 1.0, 9.0, n_pos / (n_pos + n_neg)
cost = c_fp * fpr * (1 - prior) + c_fn * (1 - tpr) * prior
i = np.argmin(cost)
print(f"best threshold {thr[i]:.2f}: FPR {fpr[i]:.3f}, TPR {tpr[i]:.3f}, "
      f"expected cost {cost[i]:.3f}")
# Bayes threshold from the true model: predict 1 when the likelihood ratio
# exceeds c_fp (1 - prior) / (c_fn prior); for unit Gaussians, log LR = 2s - 2
t_bayes = (np.log(c_fp * (1 - prior) / (c_fn * prior)) + 2) / 2
print(f"Bayes threshold from the true densities: {t_bayes:.2f}")
```

經驗 AUC（0.9166）與成對排序估計完全一致，也接近理論值 $\Phi(2/\sqrt{2}) \approx 0.921$。先驗為 10%、成本比為 9:1 時，先驗勝算與成本比互相抵消，貝氏門檻是 $t = 1$；經驗搜尋找到 0.99。實際專案中，門檻搜尋必須在驗證資料上進行，絕不能用測試集。

假設在哪裡失效

貝氏規則的最佳性，是相對於假設的分布與指定的損失而言。部署時，每一項都可能出錯。

先驗偏移。訓練集與部署環境的類別比例可能不同，例如兩座工廠之間，或平衡的研究資料集與真實診所之間。若類別條件分布 $p(x \mid c)$ 不變（**標籤偏移**，label shift），不必重新訓練就能修正後驗：把每個後驗乘上新舊先驗的比值，再重新正規化，

$$P_{\text{new}}(c_k \mid x) \propto P_{\text{old}}(c_k \mid x)\, \frac{P_{\text{new}}(c_k)}{P_{\text{old}}(c_k)}.$$

新先驗未知時，Saerens、Latinne 與 Decaestecker [10] 在未標註的部署資料上用 EM 程序估計；Lipton、Wang 與 Smola 的黑箱偏移估計（black-box shift estimation）[11] 則從分類器的混淆矩陣估計。若新相機改變的是似然本身（**共變量偏移**或**概念偏移**），只修正先驗並不夠。

後驗未校準。本章每個門檻都假設 $P(c \mid x)$ 名副其實。SVM、提升樹或深度網路的分數常有系統性的過度自信或自信不足；在保留資料上重新校準（Platt scaling、保序迴歸、溫度縮放）是標準修正方式 [12, 14]。

標籤雜訊與代理標籤。標籤記錄的可能是一次不完美的檢查，而非物理真相。這時模型學到的是檢查員的後驗，而不是世界的後驗。

單一數字的摘要。兩個準確率相同的分類器，偽陰性率可能不同，期望損失也可能差很多。請一併報告混淆矩陣、操作點與成本。

現代觀點

貝氏決策理論已有半世紀以上的歷史，而它的詞彙（後驗、風險、拒絕、操作點）如今正是可信賴深度學習的詞彙。2017 年以來的研究，大多在處理一件事：當分類器是大型網路時，如何讓餵給貝氏規則的輸入值得信賴。

校準。 Guo 等人 [12] 發現現代深度網路比十年前較淺的網路校準差得多。他們把這與深度、寬度、權重衰減和批次正規化連結起來，並證明**溫度縮放**（temperature scaling，把所有 logit 除以一個在驗證資料上擬合的純量，即單參數版的 Platt scaling）出奇有效。Minderer 等人 [13] 在 2021 年前後的影像分類器上重新檢視這個問題，發現最新的模型，尤其是不使用卷積的模型，屬於校準最好的一群；早期觀察到的「模型越大、或分布偏移時校準越差」的趨勢，在新架構上不那麼明顯。Silva Filho 等人的綜述 [14] 是最好的入門：涵蓋適當評分規則（proper scoring rules）與校準指標、可靠度圖，以及二元與多類別的事後校準方法的完整整理。Gawlikowski 等人 [23] 回顧神經網路不確定性這個更大的領域，區分可降低的模型不確定性與不可降低的資料不確定性，並涵蓋貝氏神經網路、集成與測試時資料擴增。

拒絕與選擇性分類。 Chow 的拒絕規則 [5] 以**選擇性分類**（selective classification）之名被重新發現。Geifman 與 El-Yaniv [16] 示範如何在訓練好的網路外加上拒絕門檻，使被接受樣本上的風險以高機率低於使用者指定的水準，用覆蓋率換取有保證的準確率。Hendrickx 等人的綜述 [15] 形式化兩種拒絕：**模糊拒絕**（Chow 的情形）與**新穎拒絕**（與訓練資料不同的輸入），回顧如何同時評估模型的預測品質與拒絕品質，並分類學習含拒絕選項模型的架構。

不依賴分布的保證。共形預測（conformal prediction）把單一標籤換成一個標籤集合，在「校準資料與測試資料可交換」這個唯一假設下，保證以指定機率（例如 90%）包含真實類別。Angelopoulos 與 Bates [17] 提供附程式碼的實作導論。預測集合很大時，扮演的就是拒絕的角色：模型在告訴你它無法縮小範圍。

開放集辨識與分布外偵測。在 K 個已知類別上正規化的後驗，沒辦法說「以上皆非」。Geng、Huang 與 Chen [18] 綜述開放集辨識（open-set recognition）：定義、模型家族、資料集與評估方式，以及它和含拒絕選項的分類、零樣本與少樣本學習的關係。Yang 等人 [19] 把異常偵測、新穎偵測、開放集辨識、分布外（OOD）偵測與離群值偵測統一為**廣義 OOD 偵測**框架下的子任務。有兩個結果與本章直接相關。Hendrycks 與 Gimpel [20] 指出，**最大 softmax 機率**（也就是 Chow 的統計量）是偵測誤分類與分布外輸入的強大基準。Vaze 等人 [21] 發現分類器說「以上皆非」的能力，與它的封閉集準確率高度相關，因此改善一般分類器就能改善開放集表現。Lee 等人 [22] 在預訓練網路的特徵上擬合**共用共變異數的類別條件高斯**（正是上面的情形 2 ／ LDA），並以馬氏距離作為信心分數。一個 1970 年代風格的生成式分類器，放在學習到的特徵上，至今仍是有競爭力的 OOD 偵測器。

開放問題。成本很少能精確得知，而且會隨時間改變；在一個分布上量到的校準，不一定能移轉到另一個分布；共形覆蓋率這類保證是邊際的（對輸入平均），而非針對特定輸入或子群體的條件保證。損失矩陣應該是什麼、由誰決定，仍是沒有任何估計器能回答的人類問題。

重點整理

- 辨識系統把觀測映射到特徵、特徵映射到信念、信念映射到行動；抽樣單位與資料切分是問題定義的一部分。
- 先驗、似然與後驗回答不同的問題。稀有類別需要強證據：盛行率 2% 時，靈敏度 90% 的檢測得到的後驗低於 20%。
- 最大後驗使錯誤率最小；最小條件風險可處理任何損失矩陣。兩類別時兩者都是似然比檢定，門檻取決於先驗乘上成本。
- 當最大後驗低於 $1 - C_R$ 時，Chow 規則拒絕；它處理模糊性，不處理新穎性。
- 判別函數在單調轉換下等價。高斯類別共用共變異數時邊界為線性（LDA），共變異數不同時為二次（QDA）。
- ROC 曲線把分數與門檻分開；Neyman–Pearson 引理使似然比成為最佳分數；陽性稀少時，PR 曲線更有資訊量。
- 這一切都仰賴校準良好的機率與穩定的先驗，而校準、選擇性分類、共形預測與 OOD 偵測等現代研究，正是在設法確保這兩件事。

練習

1. **不對稱成本**。偽陽性成本為四、偽陰性成本為一時，推導後驗 $p = P(c_1 \mid x)$ 上的二元決策門檻。這條規則現在偏向哪個類別？為什麼？

   提示

   當 $4(1-p) < p$ 時選 c_1，所以門檻是 $p > 0.8$。現在假警報才是昂貴的錯誤，規則需要強證據才會說「陽性」。

2. **拒絕與否**。後驗為 $(0.55, 0.35, 0.10)$、拒絕成本 $C_R = 0.3$，在零一分類損失下選出最小風險行動。C_R 為哪些值時答案會改變？

   提示

   最佳分類風險為 $1 - 0.55 = 0.45 > 0.3$，所以拒絕。當 $C_R \geq 0.45$ 時，系統會分類（判為 c_1）。

3. **移動的邊界**。兩個一維類別 $p(x \mid c_0) = \mathcal{N}(0, 1)$、$p(x \mid c_1) = \mathcal{N}(3, 1)$。求先驗相等與 $P(c_1) = 0.2$ 時的最小錯誤率邊界；再求先驗相等、偽陰性成本為偽陽性四倍時的最小風險邊界。

   提示

   對數似然比為 $3x - 4.5$。先驗相等：$x = 1.5$。$P(c_1) = 0.2$：$3x - 4.5 = \ln 4$，所以 $x \approx 1.96$。$C_{\text{FN}} = 4C_{\text{FP}}$：$3x - 4.5 = \ln(1/4)$，所以 $x \approx 1.04$。先驗與成本透過同一個門檻作用，這裡方向相反。

4. **馬氏距離與歐氏距離**。兩類別共用 $\Sigma = \begin{bmatrix} 4 & 0 \\ 0 & 1 \end{bmatrix}$，平均值 $\mu_0 = (0, 0)$、$\mu_1 = (2, 1)$，先驗相等。分別用歐氏最近平均值規則與貝氏（LDA）規則分類 $x = (2, 0)$，並解釋為何不一致。

提示

歐氏距離平方：到 μ_0 為 4，到 μ_1 為 1，所以最近平均值規則判為 c_1。馬氏距離平方：到 μ_0 為 $(2^2/4)+0=1$，到 μ_1 為 $0+1=1$，平手：x 恰好在 LDA 邊界上。第一個特徵變異很大，所以沿它偏移 2 個單位只是弱證據；歐氏規則高估了它。

5. **從零計算 ROC**。只用 NumPy，把 ROC 程式範例中的分數排序後計算 ROC 曲線與 AUC，並與 `roc_auc_score` 比對。接著把盛行率改成 1%，說明 ROC、AUC 與固定門檻下的精確率中，哪些會改變。

提示

把分數由大到小排序；標籤的累積和給出 TP 數，1− 標籤的累積和給出 FP 數；用梯形法把 TPR 對 FPR 積分。ROC 與 AUC 只取決於兩個類別條件分數分布，因此（除抽樣雜訊外）不變；固定門檻下的精確率會大幅下降，因為偽陽性現在來自大得多的陰性群體。

6. **抽樣單位**。某測試集包含與訓練所用影片相同影片中的相鄰影格。說明它的準確率估計的是什麼，並提出能估計新影片、新相機或新受試者表現的切分方式。

提示

它估計的是在訓練影格近似複本上的準確率（內插任務）。請以錄影為單位切分（分組切分）；如果部署時會遇到新相機或新的人，也要保留整台相機或整位受試者。

參考文獻

1. R. O. Duda, P. E. Hart and D. G. Stork, *Pattern Classification*, 2nd ed., Wiley, 2001 (Ch. 2, Bayesian decision theory). publisher page
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006 (Sections 1.2 and 1.5). official page and free PDF
3. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2008 (Ch. 1–2). publisher page
4. T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, 2009 (Section 4.3, linear discriminant analysis). official page
5. C. K. Chow, "On optimum recognition error and reject tradeoff," *IEEE Transactions on Information Theory*, vol. 16, no. 1, pp. 41–46, 1970. doi
6. J. Neyman and E. S. Pearson, "On the problem of the most efficient tests of statistical hypotheses," *Philosophical Transactions of the Royal Society of London, Series A*, vol. 231, pp. 289–337, 1933. doi
7. T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006. doi
8. J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in *Proc. International Conference on Machine Learning (ICML)*, 2006. doi

9. T. Saito and M. Rehmsmeier, “The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets,” *PLOS ONE*, vol. 10, no. 3, e0118432, 2015. doi
10. M. Saerens, P. Latinne and C. Decaestecker, “Adjusting the outputs of a classifier to new a priori probabilities: a simple procedure,” *Neural Computation*, vol. 14, no. 1, pp. 21–41, 2002. doi
11. Z. C. Lipton, Y.-X. Wang and A. Smola, “Detecting and correcting for label shift with black box predictors,” in *Proc. ICML*, PMLR 80, 2018. paper
12. C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. ICML*, PMLR 70, 2017. paper
13. M. Minderer, J. Djolonga, R. Romijnders, F. Hubis, X. Zhai, N. Houlsby, D. Tran and M. Lucic, “Revisiting the calibration of modern neural networks,” in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2021. paper
14. T. Silva Filho, H. Song, M. Perello-Nieto, R. Santos-Rodriguez, M. Kull and P. Flach, “Classifier calibration: a survey on how to assess and improve predicted class probabilities,” *Machine Learning*, vol. 112, pp. 3211–3260, 2023. doi
15. K. Hendrickx, L. Perini, D. Van der Plas, W. Meert and J. Davis, “Machine learning with a reject option: a survey,” *Machine Learning*, vol. 113, pp. 3073–3110, 2024. doi
16. Y. Geifman and R. El-Yaniv, “Selective classification for deep neural networks,” in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2017. paper
17. A. N. Angelopoulos and S. Bates, “Conformal prediction: a gentle introduction,” *Foundations and Trends in Machine Learning*, vol. 16, no. 4, pp. 494–591, 2023. doi
18. C. Geng, S.-J. Huang and S. Chen, “Recent advances in open set recognition: a survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 10, pp. 3614–3631, 2021. doi
19. J. Yang, K. Zhou, Y. Li and Z. Liu, “Generalized out-of-distribution detection: a survey,” *International Journal of Computer Vision*, vol. 132, pp. 5635–5662, 2024. doi
20. D. Hendrycks and K. Gimpel, “A baseline for detecting misclassified and out-of-distribution examples in neural networks,” in *Proc. International Conference on Learning Representations* (*ICLR*), 2017. arXiv
21. S. Vaze, K. Han, A. Vedaldi and A. Zisserman, “Open-set recognition: a good closed-set classifier is all you need,” in *Proc. ICLR*, 2022. arXiv
22. K. Lee, K. Lee, H. Lee and J. Shin, “A simple unified framework for detecting out-of-distribution samples and adversarial attacks,” in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2018. paper
23. J. Gawlikowski et al., “A survey of uncertainty in deep neural networks,” *Artificial Intelligence Review*, vol. 56, suppl. 1, pp. 1513–1589, 2023. doi

PR 02・密度估計與貝氏學習

估計貝氏分類器所需的密度：最大似然、MAP 與完整貝氏估計；直方圖、Parzen 窗與核頻寬選擇；k-NN 密度與 k-NN 分類器；以及維度詛咒。

網站版本: https://blog.shyandram.dev/zh-tw/learn/pattern-recognition/pr-02-density-estimation/

你將學到

- 為什麼貝氏分類器需要密度估計，以及參數式與非參數式估計的差別。
- 如何推導高斯分布的最大似然（ML）估計、為何樣本少時共變異數會奇異，以及收縮（shrinkage）如何解決。
- MAP 與完整貝氏估計如何使用參數的先驗，並以高斯平均值與 Bernoulli 比率實際推導。
- 直方圖、Parzen 窗（核密度估計）與 k 近鄰估計如何由同一條公式推出，以及如何選擇頻寬或 k。
- k-NN 分類器如何從 k-NN 密度估計推得、Cover–Hart 上界說了什麼，以及為何所有局部方法都受維度詛咒所苦。

先看全貌

PR 01 說明了最佳分類器需要類別先驗 $P(c_k)$ 與類別條件密度 $p(x \mid c_k)$。實務上沒有人會把這些分布交給我們；我們只有一組有限的標註樣本。本章談的是如何把樣本變成密度。有了 $\hat{p}(x \mid c_k)$ 與 $\hat{P}(c_k)$，就能代入貝氏規則。這是分類的**生成式**（generative）路線。下一章則走**判別式**（discriminative）路線，直接擬合決策邊界。

> **用最簡單的話說**
>
> 每次在公園看到一隻鳥，就在地圖上貼一張貼紙。現在猜猜下一隻鳥會出現在哪裡。你可以在大部分貼紙外面畫一**個大橢圓**（參數式模型：只要決定橢圓的中心和形狀）。也可以在地圖上鋪**格子**，數每一格有幾張貼紙（直方圖）。或者在每張貼紙上堆一**小座沙丘**，看沙子在哪裡堆得最高（核估計）。每一種都是不同的猜法，也各有失敗的可能：如果鳥喜歡兩個不同的池塘，一個橢圓就太簡單；如果只有五張貼紙，沙丘又會太凹凸不平。

估計方法分成兩大類 [1, 2, 3]：

- **參數式**（parametric）方法假設一種函數形式（例如高斯），並估計固定數量的參數。形式正確時需要的資料很少，形式錯誤時則會系統性地出錯。
- **非參數式**（non-parametric）方法讓資料決定形狀，複雜度隨樣本數增加。它們很有彈性，但需要多得多的資料，在高維時尤其如此。

兩類方法都面對同一個核心問題：手上的資料能支撐多少結構？一個有彈性的估計器可以把訓練資料描述得很漂亮，卻依然對新資料預測得很差。下面每個方法都有一個「平滑旋鈕」（參數數量、先驗、組距、頻寬、k），把它調好比選對方法更重要。

最大似然估計

白話版。選擇讓觀測到的資料出現機率最高的參數值。

精確版。令 $D = \{x_1, \dots, x_N\}$ 為從密度 $p(x \mid \theta)$ 獨立抽出的樣本，參數向量 θ 未知。θ 的**似然**是 $p(D \mid \theta) = \prod_n p(x_n \mid \theta)$，ML 估計最大化它的對數：

$$\hat{\theta}_{\mathrm{ML}} = \arg\max_{\theta} \ell(\theta), \qquad \ell(\theta) = \sum_{n=1}^{N} \log p(x_n \mid \theta).$$

獨立性假設談的是抽樣（不同觀測互不影響），而不是同一個特徵向量內各座標彼此獨立。做分類器時，我們為每個類別各擬合一個模型，只用該類別的樣本，並以類別頻率 $\hat{P}(c_k) = N_k/N$ 估計先驗。

高斯情形

d 維高斯 $\mathcal{N}(\mu, \Sigma)$ 的對數似然為

$$\ell(\mu, \Sigma) = -\frac{N}{2}\log|\Sigma| - \frac{1}{2}\sum_{n}(x_n - \mu)^{\top}\Sigma^{-1}(x_n - \mu) + \text{const}.$$

令對 μ 的梯度為零，得到 $\Sigma^{-1}\sum_n (x_n - \mu) = 0$。對 Σ^{-1} 微分（利用 $\partial \log|A|/\partial A = A^{-\top}$ 與 $\partial\, a^{\top}Aa/\partial A = aa^{\top}$），得到估計值

$$\hat{\mu} = \frac{1}{N}\sum_{n} x_n, \qquad \hat{\Sigma} = \frac{1}{N}\sum_{n}(x_n - \hat{\mu})(x_n - \hat{\mu})^{\top}.$$

樣本平均是不偏的。ML 共變異數則略有**偏差**：$\mathbb{E}[\hat{\Sigma}] = \frac{N-1}{N}\Sigma$，因為 $\hat{\mu}$ 是用同一批資料擬合的，比真實平均更貼近這些資料。常見的樣本共變異數除以 $N-1$ 來消除這個偏差。N 很大時，兩者差異可以忽略。

ML 失效之時：參數太多

完整共變異數有 $d(d+1)/2$ 個獨立元素。$d = 64$ 個特徵時，每個類別就是 2,080 個數字，平均值還要再加 64 個。更糟的是，$\hat{\Sigma}$ 是以 $\hat{\mu}$ 為中心的 N 個秩一矩陣之和，所以秩最多為 $N-1$。**當一個類別的樣本數不超過維度時，它的 ML 共變異數是奇異的**：某些方向上變異數為零，$\log|\hat{\Sigma}| = -\infty$，PR 01 的高斯判別函數無法定義。即使 N 只比 d 稍大，最小的特徵值也會被系統性低估，而反矩陣 $\hat{\Sigma}^{-1}$ 會大幅放大這些方向上的雜訊。

正則化把共變異數往較簡單的目標**收縮**：

$$\tilde{\Sigma} = (1 - \alpha)\hat{\Sigma} + \alpha\tau I, \qquad \tau = \operatorname{tr}(\hat{\Sigma})/d, \qquad 0 \le \alpha \le 1.$$

其中 τI 是平均變異數相同的球形矩陣，α 是收縮強度。收縮把大的特徵值往下拉、小的往上推，用一點偏差換來條件數好得多的估計。Ledoit 與 Wolf [14] 推導出使期望平方誤差最小的 α 的封閉式估計，因此不需要交叉驗證。其他常見的限制有**對角**共變異數（忽略相關；即下文的高斯樸素貝氏），以及各類別**共用**共變異數（匯集證據；即 PR 01 的 LDA）。這些是不同的建模假設，不是可以互換的數值技巧。

程式碼：ML 共變異數何時失效

```
import numpy as np
from scipy.stats import multivariate_normal
from sklearn.covariance import EmpiricalCovariance, LedoitWolf

rng = np.random.default_rng(0)
d = 40
A = rng.normal(size=(d, d))
true_cov = A @ A.T / d + 0.5 * np.eye(d)          # a well-conditioned truth
test = rng.multivariate_normal(np.zeros(d), true_cov, 2000)

for n in [30, 60, 200, 1000]:
    train = rng.multivariate_normal(np.zeros(d), true_cov, n)
    mu = train.mean(0)
    S_ml = EmpiricalCovariance().fit(train).covariance_   # divisor n
    S_lw = LedoitWolf().fit(train)
    rank = np.linalg.matrix_rank(S_ml)
    ll_ml = (multivariate_normal(mu, S_ml, allow_singular=True).logpdf(test).mean()
             if rank == d else -np.inf)
    ll_lw = multivariate_normal(mu, S_lw.covariance_).logpdf(test).mean()
    print(f"n={n:4d}: rank(S_ML)={rank:2d}, test log-lik ML={ll_ml:8.2f}, "
          f"Ledoit-Wolf={ll_lw:8.2f} (shrinkage {S_lw.shrinkage_:.2f})")
```

$d = 40$、$n = 30$ 時，ML 共變異數的秩是 29，幾乎對每個測試點都給出零密度（對數似然 $-\infty$）。$n = 60$ 時它可逆但很差（測試對數似然 -94.6，Ledoit–Wolf 為 -63.0）。自動選出的收縮強度隨 n 增加從 0.65 降到 0.08，$n = 1000$ 時兩種估計幾乎重合。經驗法則：每個參數分到的樣本越少，就該收縮得越多。

MAP 與貝氏估計

白話版。最大似然完全相信資料。貝氏估計則從對參數的先驗信念出發，讓資料逐步推翻它。資料少時先驗很重要；資料多時幾乎無關緊要。

精確版。把 θ 視為隨機變數，給它**參數先驗** $p(\theta)$。貝氏定理給出參數後驗

$$p(\theta \mid D) = \frac{p(D \mid \theta)\, p(\theta)}{\int p(D \mid \theta')\, p(\theta')\, d\theta'} \;\propto\; p(D \mid \theta)\, p(\theta).$$

參數先驗與 PR 01 的類別先驗不同。類別先驗說的是某個類別有多常見；參數先驗說的是在看到這份訓練集之前，哪些模型設定是合理的。後驗常見的用法有三種：

- **MAP 估計**只保留眾數：$\hat{\theta}_{\text{MAP}} = \arg\max_\theta \left[\log p(D \mid \theta) + \log p(\theta)\right]$。對數先驗扮演正則項；權重上的高斯先驗正好是 L_2（ridge）懲罰，PR 03 會用到。
- **完整貝氏預測**對所有參數值做平均，而不是只挑一個：

$$p(x_{\text{new}} \mid D) = \int p(x_{\text{new}} \mid \theta)\, p(\theta \mid D)\, d\theta.$$

這個**後驗預測**分布把參數的不確定性帶進預測。

- **代入式預測**（plug-in）使用 $p(x_{\text{new}} \mid \hat{\theta})$，其中 $\hat{\theta}$ 來自 ML 或 MAP。它忽略參數不確定性，因此資料稀少時會過度自信。

有一個細節：MAP 對重新參數化不具不變性。σ 上密度的眾數，不等於對應的 σ^2 上密度眾數的平方根，因為變數變換時密度會多出 Jacobian 因子。ML 估計與完整後驗沒有這個問題。

實例：高斯的平均值

令 $x_n \sim \mathcal{N}(\mu, \sigma^2)$，$\sigma^2$ 已知，並給未知平均值高斯先驗 $\mu \sim \mathcal{N}(m_0, s_0^2)$。把似然與先驗相乘，對 μ 配方後可知後驗仍是高斯，$\mu \mid D \sim \mathcal{N}(m_N, s_N^2)$，其中 [1, 2]

$$\frac{1}{s_N^2} = \frac{1}{s_0^2} + \frac{N}{\sigma^2}, \qquad m_N = s_N^2 \left(\frac{m_0}{s_0^2} + \frac{N\bar{x}}{\sigma^2} \right),$$

$\bar{x}$ 是樣本平均。有兩種讀法能讓這條公式好記。**精度相加**：後驗精度 $1/s_N^2$ 等於先驗精度加上 N 倍單一觀測的精度。**平均值是精度加權平均**：先驗平均與樣本平均依精度加權。當 $N \to \infty$，$m_N \to \bar{x}$（ML 估計）且 $s_N \to 0$。因為後驗是高斯，眾數等於平均，所以這裡 $\hat{\mu}_{\text{MAP}} = m_N$。後驗預測分布為 $\mathcal{N}(m_N,\ \sigma^2 + s_N^2)$：比代入式的 $\mathcal{N}(m_N, \sigma^2)$ 寬，多出的部分就是對 μ 剩餘的不確定性。

後驗與先驗屬於同一分布族的先驗稱為**共軛**（conjugate）先驗。共軛配對（高斯–高斯、Beta–Bernoulli、Dirichlet–類別分布、平均與共變異數的 Normal–inverse-Wishart）都有封閉式更新；其他先驗則需要PR 08 的近似推論方法。

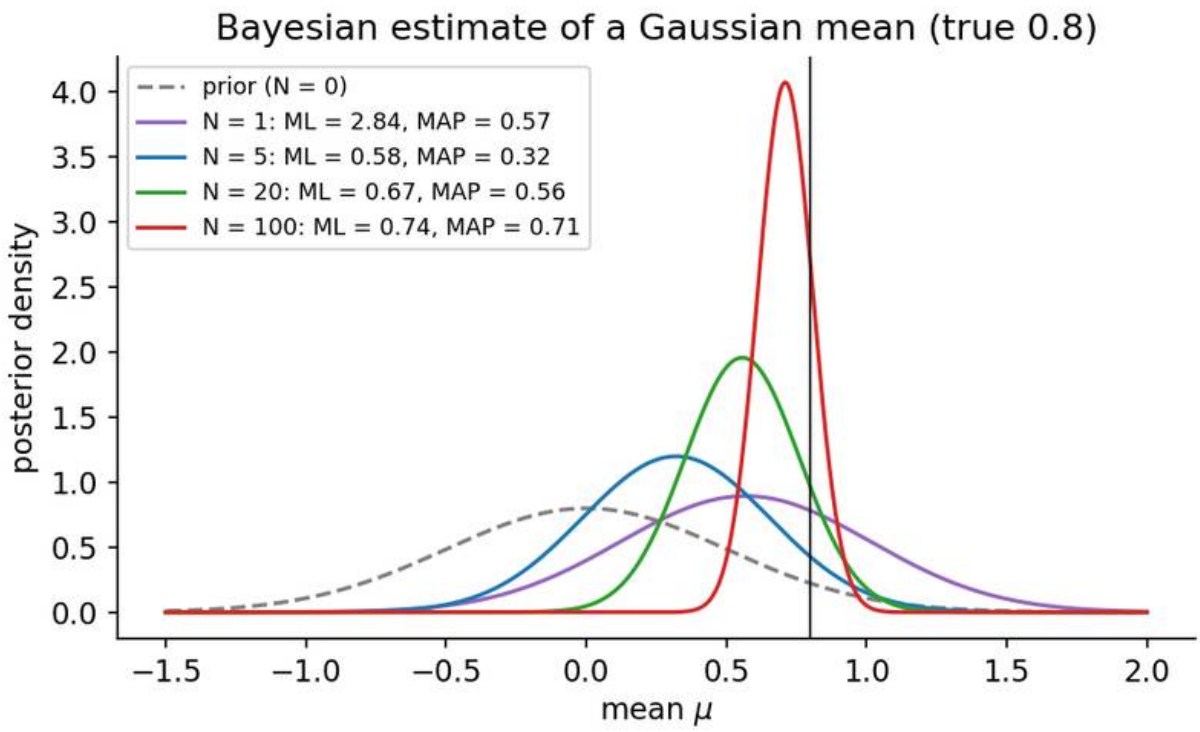


圖 *1* — 高斯平均值的貝氏估計（真值 *0.8*，先驗 $N(0,\,0.5^2)$）。只有一筆樣本時，*ML* 估計（*2.84*）偏得很遠，後驗平均被拉回到 *0.57*；有 *100* 筆樣本時，*ML*（*0.74*）與 *MAP*（*0.71*）幾乎一致，後驗也很窄。

實例：Bernoulli 比率

對 Bernoulli 比率 q（例如產線的缺陷率）給定先驗 $\mathrm{Beta}(a, b)$，在 N 次試驗中觀測到 s 次成功後，

$$q \mid D \sim \mathrm{Beta}(a+s,\; b+N-s), \qquad p(x_{\text{new}}=1 \mid D) = \frac{a+s}{a+b+N}.$$

先驗就像 a 次假想的成功與 b 次假想的失敗。一次成功、零次失敗時，ML 預測機率為一，這顯然荒謬。均勻先驗 $\mathrm{Beta}(1,1)$ 預測 $2/3$，這就是 Laplace 的接續法則（rule of succession）。同樣的概念稱為**加法平滑**或 **Laplace 平滑**，它能避免樸素貝氏文字分類器只因某個字在訓練時從未與某類別一起出現，就給整份文件零機率。

程式碼：ML、MAP 與後驗預測

```
import numpy as np

rng = np.random.default_rng(3)
sigma = 1.0                 # known noise standard deviation
m0, s0 = 0.0, 0.5           # prior on the mean: N(m0, s0^2)
```

```
x = rng.normal(0.8, sigma, 5)
n = len(x)

mu_ml = x.mean()
post_var = 1 / (1 / s0**2 + n / sigma**2)
post_mean = post_var * (m0 / s0**2 + x.sum() / sigma**2)   # = MAP for a Gaussian
pred_var = sigma**2 + post_var                              # predictive variance

# brute-force check on a grid: posterior is proportional to likelihood x prior
grid = np.linspace(-3, 3, 200001)
log_post = (-0.5 * ((x[:, None] - grid) ** 2).sum(0) / sigma**2
            - 0.5 * (grid - m0) ** 2 / s0**2)
w = np.exp(log_post - log_post.max())
w /= w.sum()
print(f"ML {mu_ml:.3f} | MAP/posterior mean {post_mean:.3f} "
      f"(grid {np.sum(w * grid):.3f}) | posterior sd {np.sqrt(post_var):.3f}")
print(f"predictive sd {np.sqrt(pred_var):.3f} vs plug-in sd {sigma:.3f}")
```

五筆樣本時，ML 平均為 0.577，後驗平均為 0.320（網格積分結果一致），後驗標準差為 0.333。預測標準差 1.054 比代入值寬 5%：這裡不大，但資料越少或維度越高時會迅速變大。

由密度建立分類器

白話版。為每個類別擬合一個密度，乘上類別頻率，選乘積最大的類別。每個密度怎麼建模，決定了分類器能學到什麼。

高斯代入式分類器。估計 $\hat{\mu}_k$、$\hat{\Sigma}_k$ 與 $\hat{P}(c_k)$，代入 PR 01 的高斯判別函數。各類別共變異數分開就是 QDA，共用（匯集）共變異數就是 LDA，邊界的幾何形狀與 PR 01 推導的一樣。計算時請用 Cholesky 分解或解線性方程組，不要顯式求反矩陣。

樸素貝氏。假設特徵**在給定類別下條件獨立**：

$$p(x \mid c_k) = \prod_{j=1}^{d} p(x_j \mid c_k).$$

每個類別只需要 d 個一維密度，而不是一個 d 維密度，因此很少的資料就能估計。它並不假設特徵在整體母體中獨立；它假設的是類別解釋了特徵之間所有的相依性。高斯樸素貝氏為每個座標在每個類別中各給一個一維高斯；當各類別的變異數不同時，邊界是二次的。當特徵高度相關（或重複）時，樸素貝氏會把同一份證據計算好幾次，所以即使決策仍算合理，後驗也會過度自信。

生成式與判別式。 Ng 與 Jordan [15] 比較了樸素貝氏與它的判別式對應方法邏輯斯迴歸。他們證明：在資料無限多的極限下，邏輯斯迴歸的錯誤率較低（或相等）；但樸素貝氏以快得多的速度逼近它自己較高的漸近錯誤率，所需樣本數與維度呈對數關係，而非線性關係。所以樣本少時生成式模型常勝出，樣本多時判別式模型勝出。這是理解兩條路線取捨的好心智模型。

非參數估計：一條公式，三種估計器

白話版。要估計某一點的密度，就看它周圍的一個小區域，數有多少樣本落在裡面。密度就是「個數除以（樣本總數乘上區域大小）」。你可以固定區域去數個數，也可以固定個數去擴大區域。

精確版。樣本落在 x 周圍小區域 $\mathcal{R}$ 內的機率為 $P = \int_{\mathcal{R}} p(x')\,dx' \approx p(x)\,V$，其中 V 是 $\mathcal{R}$ 的體積。若 N 個樣本中有 k 個落在裡面，$k \approx NP$，所以 [1, 2]

$$\hat{p}(x) = \frac{k}{N\,V}.$$

這裡藏了兩個近似，而且方向相反。$P \approx p(x)V$ 需要小區域（否則 $\hat{p}$ 是 p 的模糊平均：**偏差**）。$k \approx NP$ 需要區域裡有很多樣本（否則計數充滿雜訊：**變異數**）。Duda、Hart 與 Stork [1] 證明：當 $N \to \infty$ 時，若區域縮小（$V_N \to 0$）而計數增加（$k_N \to \infty$），但增加速度比 N 慢（$k_N/N \to 0$），估計就會收斂到 $p(x)$。有兩種滿足方式：

- **固定** V、**計數** k：直方圖與 Parzen 窗（核密度估計）；
- **固定** k、**量測** V：k 近鄰密度估計。

直方圖

把軸切成寬 w 的組，對落在第 i 組（計數為 n_i）的 x，令 $\hat{p}(x) = n_i/(Nw)$。直方圖簡單又快，但有三個弱點，見圖 2。估計在組界處不連續。它除了組距之外還取決於網格的**起點**：把所有組平移 0.3，圖形就變了。而在 d 維中，組數以 $(1/w)^d$ 成長，遠在 d 到達十之前，大部分的組就是空的。Scott 的分析 [8] 顯示，使平均積分平方誤差最小的組距只以 $N^{-1/3}$ 的速度縮小，所以更多資料只能慢慢換來更細的組。

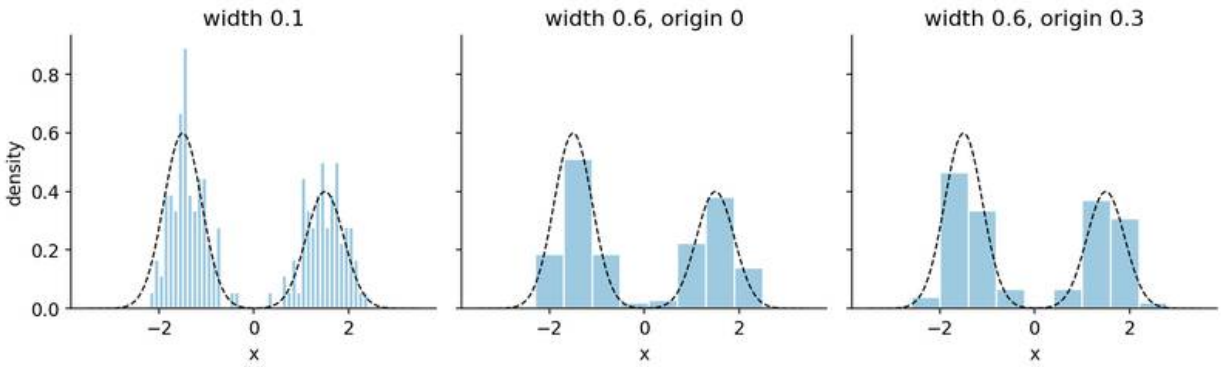


圖 2 — 同樣的 *180* 筆樣本（虛線為真實密度）。組距窄則雜訊多；組距寬則取決於網格從哪裡開始。右邊兩張只差在組的起點。

Parzen 窗與核密度估計

Parzen 窗或**核密度估計**（kernel density estimate, KDE）由 Rosenblatt [5] 提出、Parzen [6] 加以分析，它不固定網格，而是在每個樣本上放一個窗：

$$\hat{p}_h(x) = \frac{1}{N h^d} \sum_{n=1}^{N} K\!\left(\frac{x - x_n}{h}\right).$$

其中 K 是**核**，滿足 $K(u) \geq 0$ 且 $\int K(u)\,du = 1$（所以 $\hat{p}_h$ 是合法的密度），$h > 0$ 是**頻寬**（bandwidth），扮演組距的角色。用盒狀核（在單位立方體內 $K = 1$）時，這條公式字面上就是「數出 x 周圍邊長 h 的立方體內有幾個樣本」，也就是上面固定 V 的估計器。高斯核 $K(u) = (2\pi)^{-d/2} e^{-\|u\|^2/2}$ 把硬性計數換成平滑權重，得到平滑的估計（圖 3 左）。實務上，核的形狀遠不如頻寬重要。

偏差與變異數。對一維的平滑密度，KDE 的偏差約隨 h^2（乘上曲率 $p''(x)$）成長，變異數約隨 $1/(Nh)$ 成長。兩者之和最小時，最佳頻寬與 $N^{-1/5}$ 成正比，平均積分平方誤差以 $N^{-4/5}$ 下降，比形式正確的參數模型的 N^{-1} 慢 [7, 8]。在 d 維中，最佳頻寬按 $N^{-1/(d+4)}$ 縮放，誤差按 $N^{-4/(d+4)}$ 縮放。隨 d 變慢，正是維度詛咒的一個面向。

選擇頻寬。這是實務上最重要的決定。常見選項：

- **經驗法則**。 Silverman 法則 $h = 0.9\,\min(\hat{\sigma},\ \mathrm{IQR}/1.34)\,N^{-1/5}$ [7]，其中 $\hat{\sigma}$ 是樣本標準差、IQR 是四分位距；資料為高斯時它是最佳的，對多峰資料則會過度平滑。
- **交叉驗證**。選擇讓保留樣本對數似然最大的 h（似然交叉驗證），或讓積分平方誤差的不偏估計最小的 h（最小平方交叉驗證，Scott 與 Terrell [9] 有分析）。
- **代入法**。 Sheather 與 Jones [10] 從資料估計最佳頻寬公式中未知的曲率項並代入；他們的選擇器是一維情形下廣泛使用、可靠的預設值。

無論用哪一種規則，都不要看最終測試資料來選 h。圖 4 說明選擇為何重要：在雙峰樣本上，$h = 0.08$ 產生虛假的小峰，$h = 1.2$ 把兩個峰合而為一，驗證對數似然則正確地偏好 $h = 0.2$。

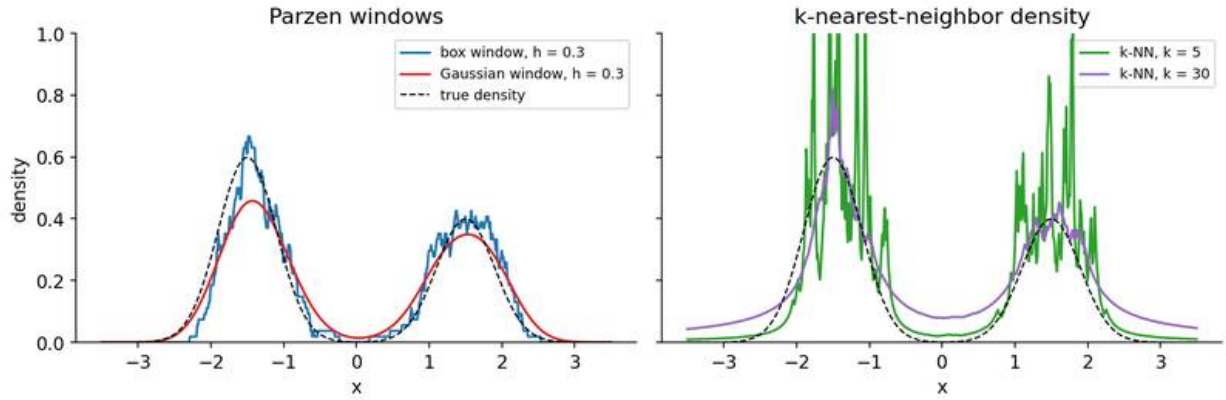


圖 3 — 左：盒狀窗得到鋸齒狀估計，同寬的高斯窗則得到平滑估計。右：*k-NN* 密度估計會依資料調整寬度，但 k 小時很尖刺，而且尾巴衰減得太慢。

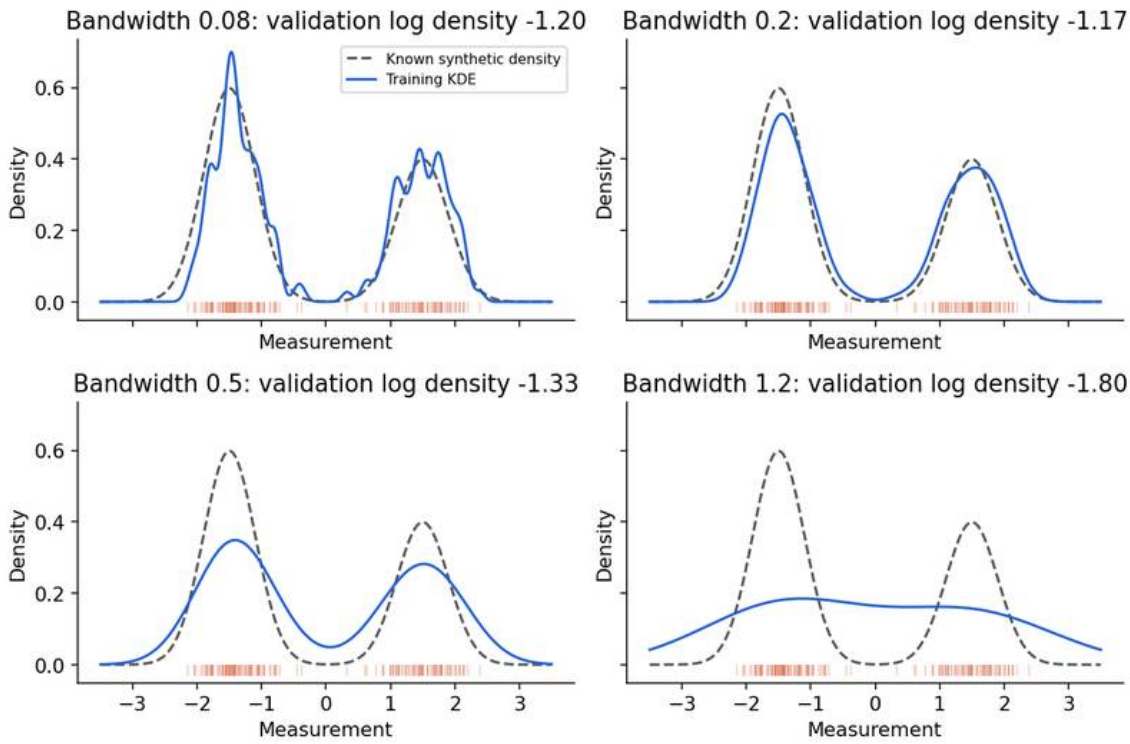


圖 *4* — 在下方程式碼所用的訓練樣本上，以四種頻寬做 *KDE*。窄頻寬顯露抽樣波動；寬頻寬把峰模糊掉。每張標題中的平均驗證對數密度在 $h = 0.2$ 時最高。

程式碼：頻寬選擇與 Beta 後驗

這個範例在合成的雙峰分布上使用 scikit-learn 的 `KernelDensity` [26]，搭配獨立的驗證樣本、用於比較的 Silverman 法則，以及上一節的封閉式 Beta 後驗。

```
import numpy as np
from scipy.stats import beta
from sklearn.neighbors import KernelDensity

rng = np.random.default_rng(4)
def sample(n):
    means = np.where(rng.random(n) < 0.6, -1.5, 1.5)
    return (means + rng.normal(0, 0.4, n)).reshape(-1, 1)
train, validation = sample(180), sample(100)
bandwidths = [0.08, 0.2, 0.5, 1.2]
scores = []
for h in bandwidths:
    model = KernelDensity(bandwidth=h).fit(train)
    scores.append(model.score(validation) / len(validation))
best = bandwidths[int(np.argmax(scores))]

# Silverman's rule of thumb for comparison
```

```
x = train.ravel()
iqr = np.subtract(*np.percentile(x, [75, 25]))
h_silverman = 0.9 * min(x.std(ddof=1), iqr / 1.34) * len(x) ** (-1 / 5)

posterior = beta(2, 1)  # uniform prior + one success
print("validation mean log densities:",
      {h: round(float(s), 3) for h, s in zip(bandwidths, scores)})
print("selected bandwidth:", best, "| Silverman's rule:", round(h_silverman, 3))
print("posterior mean:", posterior.mean())
print("95% credible interval:", posterior.ppf([0.025, 0.975]))
assert np.isclose(posterior.mean(), 2 / 3)
```

驗證選出 $h = 0.2$，Silverman 法則則給出 $h \approx 0.475$。這條法則假設資料是單一高斯；而這份樣本的分散主要來自兩峰之間的間隔，所以法則過度平滑。均勻先驗下觀測一次成功後，Beta 後驗的平均為 $2/3$，95% 可信區間約為 $[0.16, 0.99]$：一筆觀測能告訴我們的非常有限。

可信區間描述的是在假設的先驗與似然下的不確定性，並不自動等於校準過的頻率學派覆蓋率保證。同樣地，密度分數好，也不保證分類邊界好：模型描述的某些變異可能與標籤無關。

k 近鄰密度估計

改為固定 k 而非 V：以 x 為中心擴大一個球，直到剛好包含 k 個樣本，令 $V_k(x)$ 為球的體積。則 [11]

$$\hat{p}_k(x) = \frac{k}{N\, V_k(x)}.$$

Loftsgaarden 與 Quesenberry [11] 證明，當 $k \to \infty$ 且 $k/N \to 0$ 時這個估計是一致的。它的頻寬會自動調整：資料密集處窄、稀疏處寬。但圖 3（右）顯示了它的缺點。k 小時非常尖刺，因為樣本聚集處的 V_k 很小。在尾部，一維中 V_k 只隨距離線性成長，所以 $\hat{p}_k$ 以 $1/|x|$ 衰減，估計**積分不為一**（積分發散）。因此它是很差的密度估計，但正如下一節所示，它是很好的分類基礎。

k 近鄰分類器

白話版。要分類一個新點，找出離它最近的 k 個訓練點，然後多數決。不需要訓練模型；訓練集本身就是模型。

精確版。在 x 周圍畫一個總共包含 k 個樣本的球，其中 k_c 個屬於類別 c。對每個類別分別套用 $k/(NV)$ 估計，類別 c 有 N_c 個樣本，得到

$$\hat{p}(x \mid c) = \frac{k_c}{N_c V}, \qquad \hat{P}(c) = \frac{N_c}{N}, \qquad \hat{p}(x) = \frac{k}{NV} \quad \Longrightarrow \quad \hat{P}(c \mid x) = \frac{\hat{p}(x \mid c)\hat{P}(c)}{\hat{p}(x)} = \frac{k_c}{k}.$$

體積消去了，後驗估計就是各類別鄰居所佔的比例。於是貝氏規則成為**多數決**。$k = 1$ 時就是**最近鄰規則**。

它有多好？ Cover 與 Hart [12] 證明，當 $N \to \infty$ 時，1-NN 規則的錯誤率 $P_{1\mathrm{NN}}$ 滿足

$$P^* \le P_{1\text{NN}} \le P^*\Big(2 - \frac{K}{K-1}P^*\Big) \le 2P^*,$$

其中 P^* 是貝氏錯誤率，K 是類別數。在資料無限多時，最佳分類所需資訊的一半，就包含在單一個最近鄰裡。隨著 k 增加（且 $k/N \to 0$），k-NN 的錯誤率會逼近 P^* 本身。這是漸近的敘述；資料有限且維度很多時，情況就沒那麼樂觀。

選擇 k。 k 小則偏差低、變異數高：圖 5 中 1-NN 的邊界會繞著個別的雜訊點打轉。k 大則邊界平滑，但最後會忽略真正的結構。請用交叉驗證選 k。兩個實務細節：兩類別時用奇數 k 以避免平手，並且先**標準化特徵**，因為歐氏距離會被數值範圍最大的特徵主導。PR 04 會把核方法與最近鄰與其他非線性分類器比較，學習式距離度量則在該章與後續章節討論。

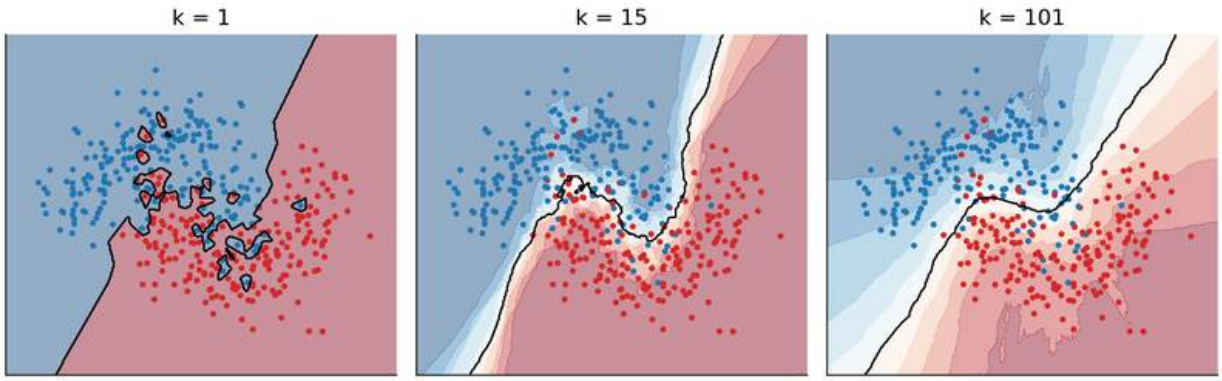


圖 *5* — 在 *400* 個帶雜訊的雙月形點上做 *k-NN*。色階表示投票比例 k_c/k；黑色曲線是 *0.5* 邊界。k *= 1* 過度擬合個別點，k *= 101* 開始把月形拉直。

程式碼：k-NN 與高斯分類器

```
import numpy as np
from sklearn.datasets import make_moons
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis

X, y = make_moons(1000, noise=0.3, random_state=0)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.5, random_state=0)

search = GridSearchCV(KNeighborsClassifier(),
                      {"n_neighbors": [1, 3, 5, 9, 15, 31, 61, 101]}, cv=5)
search.fit(Xtr, ytr)                      # k chosen on training folds only
print("cross-validated k:", search.best_params_["n_neighbors"])
models = {"1-NN": KNeighborsClassifier(1),
          f"{search.best_params_['n_neighbors']}-NN": search.best_estimator_,
          "Gaussian naive Bayes": GaussianNB(),
          "QDA": QuadraticDiscriminantAnalysis()}
for name, m in models.items():
    m.fit(Xtr, ytr)
```

```
    print(f"{name:22s} test error {1 - m.score(Xte, yte):.3f}")
```

交叉驗證選出 $k = 31$，測試錯誤率 10.6%；1-NN 為 12.4%。高斯模型的類別形狀只是一個橢圓，跟不上月形，表現較差（樸素貝氏 17.4%，QDA 17.6%）。在這個二維問題上，非參數方法勝出，因為參數形式是錯的。下一節說明為何維度升高時這個優勢會縮小。

維度詛咒

白話版。局部方法需要「近」的鄰居。在高維空間裡，沒有所謂的近：即使只想蒐集一小部分資料，也得橫跨大半個空間，而最近點與最遠點的距離幾乎一樣。

精確版。「維度詛咒」（curse of dimensionality）這個說法涵蓋幾個相關的效應；Bishop [2, Section 1.4] 有幾何上的說明。其中三個很容易驗證（圖 6）：

1. **鄰域必須很大**。在單位立方體 $[0,1]^d$ 內均勻分布的資料中，包含比例 r 資料的子立方體，邊長為 $r^{1/d}$。要涵蓋 10% 的資料，邊長在二維是 0.32、十維是 0.79、一百維是 0.98。「局部」鄰域幾乎跨越每個特徵的全部範圍，估計就不再是局部的了。
2. **體積都在角落與外殼**。單位球體積除以外接立方體 $[-1,1]^d$ 的體積，會迅速趨近於零；而球的體積幾乎全集中在靠近表面的薄殼中。高維高斯的樣本與平均值的距離幾乎相同，而不是聚在平均值附近。
3. **距離集中**。 Beyer 等人 [13] 證明，在資料分布滿足廣泛條件下，最遠鄰與最近鄰距離的比值隨 d 增加趨近於一，「最近鄰」因而失去意義。在下面的模擬中，最近與最遠距離之比從二維的 0.02 升到一百維的約 0.7。

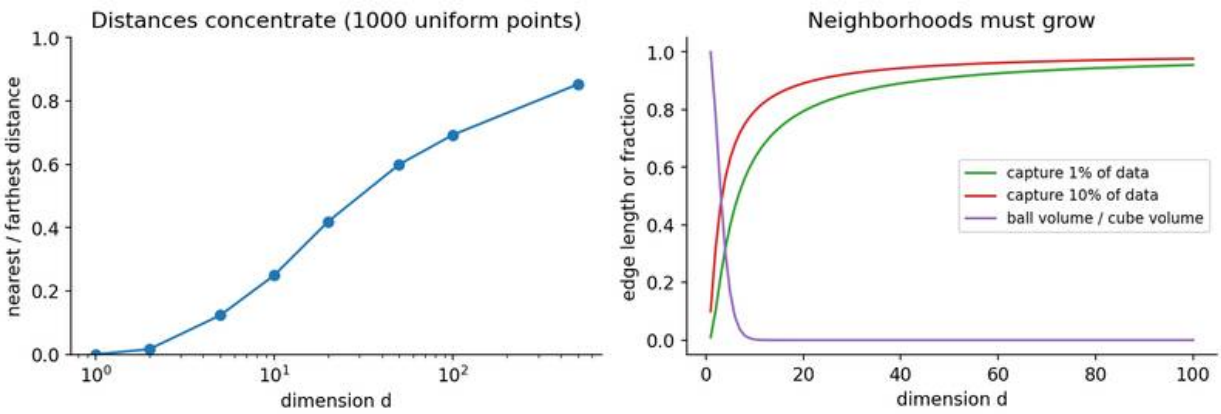


圖 *6* — 左：*1,000* 個均勻點與 *50* 個查詢點的距離集中現象。右：涵蓋均勻資料 *1%* 或 *10%* 所需的子立方體邊長，以及單位球佔立方體 *[-1, 1]^d* 的體積比例。

程式碼：距離集中

```
import numpy as np
```

```
rng = np.random.default_rng(0)
for d in [1, 2, 10, 100, 1000]:
    pts = rng.random((1000, d))          # 1000 uniform points in [0, 1]^d
    q = rng.random((50, d))              # 50 query points
    dist = np.sqrt(((q[:, None, :] - pts[None]) ** 2).sum(-1))
    ratio = (dist.min(1) / dist.max(1)).mean()
    edge = 0.1 ** (1 / d)                # sub-cube edge holding 10% of the volume
    print(f"d={d:5d}: nearest/farthest = {ratio:.3f}, "
          f"edge for 10% of data = {edge:.3f}")
```

最近／最遠距離比在二維是 0.017，十維 0.27，一百維 0.70，一千維 0.89。

那為什麼高維時還有方法行得通？真實資料並非均勻分布。影像、聲音與文字都貼近巨大環境空間中的低維結構（流形、群集），有資訊的方向遠少於原始座標數。因此實務上的防禦是：先降維或學習表徵（PR 05：特徵與維度）、限制模型（共用或對角共變異數、樸素貝氏），以及正則化。最近鄰方法今天之所以表現很好，正因為它們用在學習到的嵌入上，而不是原始像素上，如「現代觀點」所述。

現代觀點

自教科書寫成以來，密度估計分成兩股潮流：傳統統計學精進了 KDE 與共變異數估計；深度學習則建立了能擴展到影像的密度模型，而最近鄰搜尋成了基礎設施。

重新檢視核密度估計。 Chen 的教學文 [16] 是溫和的現代入門：收斂速率、頻寬選擇、信賴帶及其偏差處理，以及用 KDE 估計密度的幾何與拓撲特徵（峰值、等高集、脊線）的近期研究。Wang 與 Scott [17] 回顧為高維資料設計的非參數密度估計器，包括峰值分群（modal clustering）等應用，並列出開放的研究方向。Scott 的專書 [8] 與 Silverman 的經典 [7] 仍是理論的標準參考。

高維共變異數。本章的奇異共變異數問題本身就是一整個研究領域。Fan、Liao 與 Liu [18] 回顧大型共變異數矩陣與精度（反共變異數）矩陣的估計方法，著重於基於秩的方法與基於因子模型的方法；Ledoit–Wolf 收縮 [14] 至今仍是函式庫中的預設正則化方式。

深度密度模型。正規化流（normalizing flow）把簡單的基底分布送過一連串可逆、可微的映射，建構出有彈性的密度；變數變換公式則給出精確的對數似然。Papamakarios 等人 [19] 與 Kobyzev、Prince、Brubaker [20] 是兩篇標準綜述：前者強調設計原則、表達能力與計算取捨，後者著重流的建構方式與開放問題。自迴歸模型、VAE 與擴散模型是學習高維密度的其他方法；見ML 06：生成模型與ML 07：擴散與流匹配。Murphy 的兩冊教科書 [4] 以同一套記號涵蓋本章的傳統估計器與這些現代模型。

大規模最近鄰。 k-NN 如今無所不在，只是用在學習到的嵌入上，而非原始特徵。Johnson、Douze 與 Jégou [21] 示範如何在 GPU 上進行十億規模的相似度搜尋；Faiss 函式庫論文 [22] 描述向量搜尋的取捨空間（準確度、速度、記憶體）以及現代向量資料庫背後的索引方法。Papernot 與 McDaniel 的**深度 k-NN**（deep k-NN）[23] 在每一層的表徵中把測試輸入與它的訓練鄰居比較，以鄰居間標籤的不一致作為信心與穩健性的訊號。在分布外偵測上，Sun 等人 [24] 直接使用特徵空間中第 k 個最近鄰的距離，不做任何分布假設，並報告相對於參數式馬氏距離基準的大幅改進。那個參數式基準來自 Lee 等人 [25]，它在網路特徵上擬合共用共變異數的類別條件高斯，也就是 PR 01 的 LDA。本章參數式與非參數式之間的抉擇，正在深度特徵之上重演。

開放問題。高維中可靠的密度估計依然困難。Nalisnick 等人 [27] 發現，在某個影像資料集上訓練的流模型、VAE 與 PixelCNN，可能給另一個資料集更高的似然（例如在日常物體資料集上訓練的模型，給門牌號碼照片資料集更高的分數），所以單靠似然並不是安全的新穎性分數。嵌入空間中頻寬或 k 的選擇，以及非參數估計的校準不確定性，都仍是活躍的研究問題。

重點整理

- 生成式分類器需要 $\hat{p}(x \mid c_k)$ 與 $\hat{P}(c_k)$；估計器的假設決定了邊界的形狀。
- 高斯的 ML 估計是樣本平均與除以 N 的共變異數；樣本數 $N \le d$ 時共變異數奇異，需要收縮（例如 Ledoit–Wolf）或共用／對角結構。
- MAP 保留後驗眾數；完整貝氏預測對參數積分，資料稀少時比代入式預測更寬。共軛先驗有封閉解：高斯平均的精度相加，Beta–Bernoulli 比率的計數相加。
- 非參數估計都源自 $\hat{p} = k/(NV)$：直方圖與 Parzen ／ KDE 固定 V，k-NN 固定 k。頻寬或 k 比核的形狀重要；用交叉驗證或有根據的規則選擇，絕不用測試資料。
- k-NN 分類器估計 $P(c \mid x) \approx k_c/k$；在漸近意義下，1-NN 的錯誤率不超過貝氏錯誤率的兩倍。
- 高維時鄰域不再局部、距離會集中；局部方法實務上只在低維或學習到的表徵上才有效。

練習

1. **計算參數數量**。計算一個 64 維高斯的平均值與共變異數參數數量。解釋為何用 30 筆樣本擬合會失敗，並提出兩種修正方式。

   提示

   64 個平均值加上 $64 \cdot 65/2 = 2{,}080$ 個共變異數元素。30 筆樣本時，$\hat{\Sigma}$ 的秩最多為 29，所以是奇異的。修正方式：往 τI 收縮（Ledoit–Wolf）、對角共變異數（樸素貝氏）、各類別共用共變異數，或先降維。

2. **Beta–Bernoulli 更新**。從 $\mathrm{Beta}(2,3)$ 開始，觀測到三次成功、兩次失敗，計算後驗、MAP 估計與成功的後驗預測機率。

   提示

   後驗 $\mathrm{Beta}(5,5)$。MAP（眾數）$= (5-1)/(5+5-2) = 0.5$。預測機率 $= 5/10 = 0.5$。與 ML 的 $3/5 = 0.6$ 比較。

3. **精度加權平均**。高斯平均的先驗為 $\mathcal{N}(0,1)$、雜訊變異數 $\sigma^2 = 4$，要幾筆樣本後資料權重才等於先驗權重？此時後驗標準差是多少？

   提示

   先驗精度為 1；每筆樣本增加 1/4。$N = 4$ 時權重相等。後驗精度為 2，所以 $s_N = 1/\sqrt{2} \approx 0.71$。

4. **重複的特徵**。在高斯樸素貝氏中複製一個特徵。預測準確率與後驗信心會怎麼變，再用 scikit-learn 在 `load_breast_cancer` 上驗證。

提示

多出的座標沒有新資訊，但分解後的似然把它算兩次，相當於把它的似然比平方。決策可能稍有改變；後驗會更靠近 0 或 1（更過度自信）。比較前後 `predict_proba` 的直方圖與對數損失。

5. **k-NN 密度不是密度。**證明在一維中，k-NN 估計 $\hat{p}_k(x)$ 在遠離資料處以 $1/|x|$ 衰減，並由此推論 $\int \hat{p}_k(x)\,dx$ 發散。為什麼這不影響 k-NN 分類器？

提示

遠離所有樣本時，到第 k 個鄰居的距離約為 $|x|$，所以 $V_k \approx 2|x|$、$\hat{p}_k \approx k/(2N|x|)$，其積分以對數方式發散。分類器只在同一點使用比例 k_c/k，共同的體積會消去。

6. **頻寬規則。**從 $\mathcal{N}(-2, 0.5^2)$ 與 $\mathcal{N}(2, 0.5^2)$ 的混合分布產生 500 筆樣本。比較 Silverman 法則、似然交叉驗證（對 `KernelDensity` 做 `GridSearchCV`），以及（若有安裝 `statsmodels`）代入式選擇器。哪一個過度平滑？為什麼？

提示

Silverman 法則使用整體分散程度（這裡約 2.1），它主要來自兩峰之間的間隔，因此給出的頻寬比每個峰所需的大好幾倍。交叉驗證似然選出的值接近單峰內的最佳值。

參考文獻

1. R. O. Duda, P. E. Hart and D. G. Stork, *Pattern Classification*, 2nd ed., Wiley, 2001 (Ch. 3, maximum-likelihood and Bayesian estimation; Ch. 4, non-parametric techniques). publisher page
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006 (Sections 1.4, 2.1–2.3, 2.5 and 4.2). official page and free PDF
3. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2008 (Ch. 2, density estimation and Bayesian classification). publisher page
4. K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022. official page
5. M. Rosenblatt, "Remarks on some nonparametric estimates of a density function," *The Annals of Mathematical Statistics*, vol. 27, no. 3, pp. 832–837, 1956. doi
6. E. Parzen, "On estimation of a probability density function and mode," *The Annals of Mathematical Statistics*, vol. 33, no. 3, pp. 1065–1076, 1962. doi
7. B. W. Silverman, *Density Estimation for Statistics and Data Analysis*, Chapman and Hall, 1986 (reissued by Routledge, 2018). doi
8. D. W. Scott, *Multivariate Density Estimation: Theory, Practice, and Visualization*, 2nd ed., Wiley, 2015. doi
9. D. W. Scott and G. R. Terrell, "Biased and unbiased cross-validation in density estimation," *Journal of the American Statistical Association*, vol. 82, no. 400, pp. 1131–1146, 1987. doi
10. S. J. Sheather and M. C. Jones, "A reliable data-based bandwidth selection method for kernel density estimation," *Journal of the Royal Statistical Society, Series B*, vol. 53, no. 3, pp. 683–690, 1991. doi

11. D. O. Loftsgaarden and C. P. Quesenberry, “A nonparametric estimate of a multivariate density function,” *The Annals of Mathematical Statistics*, vol. 36, no. 3, pp. 1049–1051, 1965. doi
12. T. M. Cover and P. E. Hart, “Nearest neighbor pattern classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967. doi
13. K. Beyer, J. Goldstein, R. Ramakrishnan and U. Shaft, “When is ‘nearest neighbor’ meaningful?,” in *Database Theory — ICDT’99*, Lecture Notes in Computer Science, Springer, 1999. doi
14. O. Ledoit and M. Wolf, “A well-conditioned estimator for large-dimensional covariance matrices,” *Journal of Multivariate Analysis*, vol. 88, no. 2, pp. 365–411, 2004. doi
15. A. Y. Ng and M. I. Jordan, “On discriminative vs. generative classifiers: a comparison of logistic regression and naive Bayes,” in *Advances in Neural Information Processing Systems* (*NIPS*), 2001. paper
16. Y.-C. Chen, “A tutorial on kernel density estimation and recent advances,” *Biostatistics & Epidemiology*, vol. 1, no. 1, pp. 161–187, 2017. doi
17. Z. Wang and D. W. Scott, “Nonparametric density estimation for high-dimensional data — algorithms and applications,” *WIREs Computational Statistics*, vol. 11, no. 4, 2019. doi
18. J. Fan, Y. Liao and H. Liu, “An overview of the estimation of large covariance and precision matrices,” *The Econometrics Journal*, vol. 19, no. 1, pp. C1–C32, 2016. doi
19. G. Papamakarios, E. Nalisnick, D. J. Rezende, S. Mohamed and B. Lakshminarayanan, “Normalizing flows for probabilistic modeling and inference,” *Journal of Machine Learning Research*, vol. 22, no. 57, pp. 1–64, 2021. arXiv
20. I. Kobyzev, S. J. D. Prince and M. A. Brubaker, “Normalizing flows: an introduction and review of current methods,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 11, pp. 3964–3979, 2021. doi
21. J. Johnson, M. Douze and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021. doi
22. M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini and H. Jégou, “The Faiss library,” arXiv:2401.08281, 2024. arXiv
23. N. Papernot and P. McDaniel, “Deep k-nearest neighbors: towards confident, interpretable and robust deep learning,” arXiv:1803.04765, 2018. arXiv
24. Y. Sun, Y. Ming, X. Zhu and Y. Li, “Out-of-distribution detection with deep nearest neighbors,” in *Proc. International Conference on Machine Learning* (*ICML*), PMLR 162, 2022. paper
25. K. Lee, K. Lee, H. Lee and J. Shin, “A simple unified framework for detecting out-of-distribution samples and adversarial attacks,” in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2018. paper
26. scikit-learn developers, “Density estimation,” scikit-learn user guide. documentation
27. E. Nalisnick, A. Matsukawa, Y. W. Teh, D. Gorur and B. Lakshminarayanan, “Do deep generative models know what they don’t know?,” in *Proc. International Conference on Learning Representations* (*ICLR*), 2019. arXiv

PR 03・迴歸與分類的線性模型

從基本原理推導線性判別：最小平方法、Fisher 判別、感知器、邏輯斯迴歸，以及最大間隔 SVM 與其對偶問題，並以損失函數統一它們。

網站版本: https://blog.shyandram.dev/zh-tw/learn/pattern-recognition/pr-03-linear-models/

你將學到

- 看懂線性判別函數的幾何（權重向量、偏差、帶號距離），以及用兩類別分類器組成多類別分類器的陷阱。
- 推導最小平方法、ridge 迴歸及其貝氏詮釋，並理解為何最小平方法是脆弱的分類器。
- 推導 Fisher 線性判別，並連結到 PR 01 的高斯 LDA。
- 訓練感知器、證明它的收斂定理，並說明它為何可能停在任何一條分隔線上。
- 用牛頓法擬合邏輯斯迴歸，並解釋可分資料為何會把權重推向無限大。
- 推導最大間隔 SVM、它的 Lagrange 對偶問題與支援向量，並把這些方法都看成同一個線性分數上的不同損失。

先看全貌

線性模型是每門圖形識別課程中第一個真正的分類器，也一直是面對新表徵時第一個該試的方法。如果一條直線就能在特徵空間中分開兩個類別，大型非線性分類器可能只增加成本，而不增加有用的準確率。如果線性基準失敗了，它的錯誤能幫助我們找出缺少的特徵或非線性結構。在現代深度學習中，幾乎每個分類器的最後一層都是學習特徵上的線性模型，而「線性分類器在這些特徵上表現如何？」正是評估表徵的標準測試。

> **用最簡單的話說**
>
> 每個量測值都有一票。把每個量測值乘上它的投票力道，全部加起來，再和一個門檻比較：高於門檻就是「是」，低於就是「否」。本章所有方法用的都是這台投票機，差別只在**怎麼決定投票力道**：讓總分盡量接近正確答案（最小平方法）、把兩群拉開（Fisher）、一次修正一個錯誤（感知器）、預測誠實的機率（邏輯斯迴歸），或在兩群之間留下最寬的空隙（SVM）。

線性分數是 $f(x) = w^\top \phi(x) + b$。**特徵映射** ϕ 可以是原始強度、平方值、梯度，或凍結的神經網路嵌入。「線性」指的是參數 w 與 b，不一定是原始量測 x：取 $\phi(x) = (x, x^2)$ 時，線性模型在 x 上畫出的是彎曲的邊界。本章依循 Duda、Hart 與 Stork [1, Ch. 5]、Bishop [2, Ch. 3–4, 7] 以及 Theodoridis 與 Koutroumbas [3, Ch. 3]，統計觀點參考 Hastie 等人 [4]。讓 ϕ 變成隱式且非常豐富的核方法，是PR 04 的主題。

線性判別函數

白話版。兩類別線性分類器是穿過特徵空間的一刀平切：二維中是直線，三維中是平面，一般情形是超平面。權重向量決定這一刀面向哪裡，偏差決定它切在哪裡。

精確版。兩類別時，若 $g(x) = w^\top x + b > 0$ 判為 c_1，否則判為 c_0。決策邊界 $g(x) = 0$ 是超平面 H。由初等幾何可得三個事實 [1, 2]：

- w 是 H 的**法向量**：對 H 上任兩點 x_a, x_b，$w^\top(x_a - x_b) = 0$。
- 任一點 x 到 H 的**帶號距離**為 $r = g(x)/\|w\|$。所以 $g(x)$ 正比於 x 離邊界多遠、在哪一側。
- 原點到 H 的距離是 $-b/\|w\|$；偏差只平移平面，不旋轉它。

把輸入**擴增**一個常數來吸收偏差很方便：$\tilde{x} = (x, 1)$、$\tilde{w} = (w, b)$，於是 $g(x) = \tilde{w}^\top \tilde{x}$。程式中我們默默這麼做。標籤取 $y \in \{-1, +1\}$ 時，樣本被正確分類的充要條件是它的**函數間隔**（functional margin）$y\,g(x)$ 為正。下面每個方法都能用這個量來表示。

超過兩個類別。兩類別分類器有兩種直覺的組合方式：一**對其餘**（one-versus-rest，K 個分類器，各自把一類和其他所有類分開），以及一**對一**（one-versus-one，每對類別一個，共 $K(K-1)/2$ 個，再投票）。兩者都可能在輸入空間留下投票結果模糊的區域 [2, Section 4.1.2]。乾淨的解法是**線性機**（linear machine）：每個類別一個判別函數 $g_k(x) = w_k^\top x + b_k$，把 x 判給分數最大的類別。它的決策區域一定是凸且單連通的，因為若 x_a 與 x_b 都是 g_k 最大，兩點之間每一點也是如此（每個 g_j 沿線段都是線性的）。PR 01 的高斯 LDA、多類別邏輯斯迴歸，以及神經網路的最後一層，都是線性機。

最小平方法與 ridge 迴歸

白話版。要預測一個數值，就選讓預測誤差平方最小的權重。如果權重比資料能支撐的多，再加上對大權重的懲罰。

精確版。令**設計矩陣** $\Phi \in \mathbb{R}^{N \times M}$ 的第 n 列是特徵向量 $\phi(x_n)^\top$（含截距用的常數座標），$t \in \mathbb{R}^N$ 是目標值。最小平方法最小化

$$\mathcal{L}(w) = \frac{1}{2}\|\Phi w - t\|_2^2,$$

其梯度 $\Phi^\top(\Phi w - t)$ 在**正規方程** $\Phi^\top \Phi w = \Phi^\top t$ 處為零。不需要顯式計算 $(\Phi^\top\Phi)^{-1}$，那樣會使條件數平方；基於 QR 或 SVD 的解法（`np.linalg.lstsq`）處理近乎相依的欄位更可靠。若假設目標值帶有獨立、同變異數的高斯雜訊，這個目標函數正好是負對數似然，因此最小平方法就是最大似然估計（PR 02）。雜訊假設說明了統計詮釋，但不代表平方誤差適合所有目標。

Ridge 迴歸 [8] 加上懲罰 $\frac{\lambda}{2}\|w\|^2$，得到

$$(\Phi^\top\Phi + \lambda I)\, w = \Phi^\top t.$$

只要 $\lambda > 0$，左邊的矩陣一定可逆，即使 $M > N$ 也是。截距通常不納入懲罰。特徵縮放很重要，因為同樣的懲罰對不同單位的座標效果不同；縮放統計量只能在訓練資料上擬合。

貝氏連結

給權重高斯先驗 $w \sim \mathcal{N}(0, \alpha^{-1} I)$（精度 α）與精度為 β 的高斯雜訊，權重的後驗是高斯分布，其共變異數與平均值為 [2, Section 3.3]

$$S = (\alpha I + \beta \Phi^\top \Phi)^{-1}, \qquad m = \beta S \Phi^\top t.$$

對新的特徵向量 ϕ_*，預測分布為

$$p(t_* \mid x_*, D) = \mathcal{N}(t_* \mid m^\top \phi_*,\ \beta^{-1} + \phi_*^\top S \phi_*).$$

第一個變異數項是觀測雜訊；第二項表達對權重的不確定性，在訓練資料很少涵蓋的特徵方向上會比較大。後驗平均 m 等於 $\lambda = \alpha/\beta$ 的 ridge 解：ridge 是 MAP 估計，和 PR 02 的高斯平均是同一個故事。

用最小平方法做分類

白話版。可以把類別標籤當成數字（+1 與 −1，或 one-hot 向量），再用最小平方法擬合。它很快、有封閉解，但工具不對：它會懲罰「太正確」的點。

精確版。對 K 個類別，把目標編碼成 **one-hot** 向量 $t_n \in \{0,1\}^K$，用最小平方法擬合線性機 $g(x) = W^\top \tilde{x}$，$W = (\tilde{X}^\top \tilde{X})^{-1} \tilde{X}^\top T$，再依最大輸出分類。兩類別時若選用適當的目標值（類別 1 為 N/N_1，類別 0 為 $-N/N_0$），最小平方的方向正好是下一節推導的 Fisher 方向 [2, Section 4.1.5]，所以這個方法並非毫無道理。

它有兩個眾所周知的失敗模式：

1. **對遠在正確一側的點很敏感**。對目標 +1 而言，平方誤差懲罰 $g(x) = 5$ 和懲罰 $g(x) = -3$ 一樣重。圖 1 加入二十個已被正確分類、但位置很遠的紅點；最小平方法為了減少它們的「誤差」而旋轉邊界，開始把藍點分錯。邏輯斯迴歸幾乎不動。
2. **三類以上時的遮蔽（masking）**。當類別大致排成一直線時，中間類別的輸出可能處處被鄰居壓過，於是中間類別永遠不會被預測出來 [4, Section 4.2]。

從機率的角度看，最小平方法背後的高斯雜訊模型對二元目標根本是錯的。解法是使用為分類設計的損失，本章其餘部分就在做這件事。

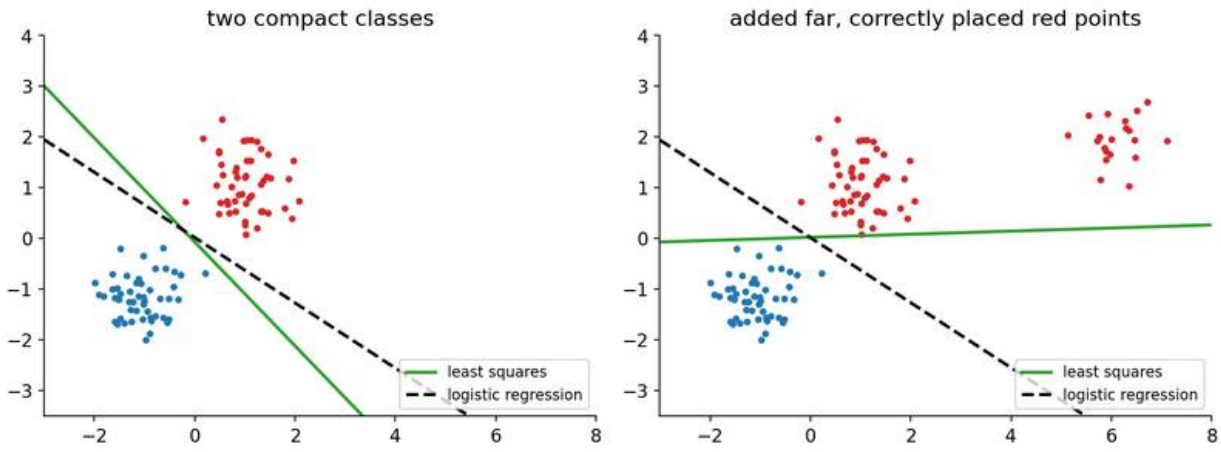


圖 *1* — 最小平方法與邏輯斯迴歸。加入早已被正確分類的紅點（右）會把最小平方邊界（綠線）拉進藍色類別；邏輯斯迴歸（虛線）不受影響。

Fisher 線性判別

白話版。把所有點投影到一條線上，看兩個類別的一維直方圖。好的線會讓兩類平均值離得遠，而且每類都很窄。Fisher 方向就是讓這個比值最大的那條線。

精確版。 Fisher [5] 提議選擇投影 $z = w^\top x$，使投影後類別平均值的分離程度相對於各類內部的分散最大。令 $m_k = \frac{1}{N_k}\sum_{n\in c_k} x_n$ 為類別平均。投影後的平均是 $w^\top m_k$，類別 k 投影後的類內散布為 $\sum_{n\in c_k}(w^\top x_n - w^\top m_k)^2 = w^\top S_k w$，其中 $S_k = \sum_{n\in c_k}(x_n - m_k)(x_n - m_k)^\top$。**Fisher 準則**為

$$J(w) = \frac{(w^\top m_1 - w^\top m_0)^2}{w^\top S_0 w + w^\top S_1 w} = \frac{w^\top S_B w}{w^\top S_W w},$$

其中

$$S_B = (m_1 - m_0)(m_1 - m_0)^\top, \qquad S_W = S_0 + S_1$$

分別是**類間**與**類內散布矩陣**。J 與 w 的長度無關，只與方向有關。

推導。令比值的梯度為零，得到

$$(w^\top S_B w)\, S_W w = (w^\top S_W w)\, S_B w.$$

括號中的兩項都是純量。此外 $S_B w = (m_1 - m_0)\,[(m_1 - m_0)^\top w]$ 永遠指向 $m_1 - m_0$。因此 $S_W w \propto m_1 - m_0$，又因為只有方向重要，

$$w^\star \propto S_W^{-1}(m_1 - m_0).$$

若 S_W 是單位矩陣的倍數（等向、不相關的類別），Fisher 方向就是平均值差。否則 S_W^{-1} 會把它從類別分散較大的方向轉開。圖 2 顯示兩個細長、相關的類別的效果：投影到平均值差上，直方圖嚴重重疊（$J = 0.70$）；投影到 Fisher 方向，兩者分得很開（$J = 3.18$）。

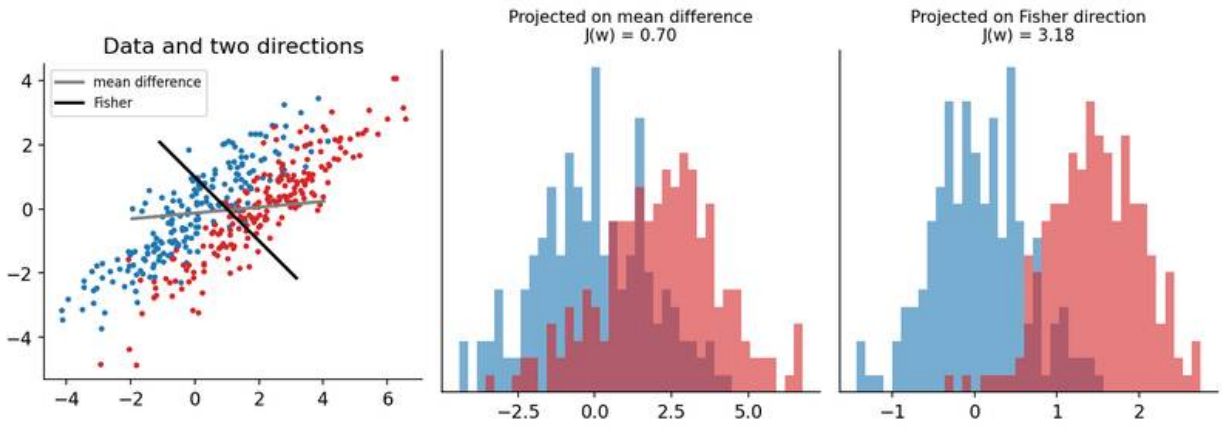


圖 2 — 兩個相關的類別分別投影到平均值差（中，$J = 0.70$）與 *Fisher* 方向（右，$J = 3.18$）。*S_W* 使用各類別的共變異數；類別大小相同時這只會縮放 J。

與高斯 LDA 的關係。 Fisher 準則不做任何分布假設；它是表徵的準則。但當 S_W 正比於匯集的類內共變異數時，它的方向與PR 01 的高斯 LDA 方向 $\Sigma^{-1}(\mu_1 - \mu_0)$ 相同。兩者只差在門檻：Fisher 不指定投影軸上的切點，高斯 LDA 則由類別平均與先驗決定切點。所以兩者都稱為「LDA」。（在主題模型中，LDA 也指 latent Dirichlet allocation，與此無關。）Fisher 投影本身不提供校準過的機率。

超過兩個類別。 K 個類別、整體平均為 m 時，定義 $S_W = \sum_k S_k$ 與 $S_B = \sum_k N_k (m_k - m)(m_k - m)^\top$。使多類別準則最大的投影 $W \in \mathbb{R}^{d \times (K-1)}$，由 $S_W^{-1} S_B$ 的前幾個特徵向量張成 $[1, 2]$。因為 S_B 是受整體平均約束的 K 個秩一項之和，秩最多為 $K - 1$，所以 Fisher 最多只能產生 $K - 1$ 個有用的特徵。這就是PR 05 中的監督式降維。

程式碼：Fisher 方向

```
import numpy as np
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

rng = np.random.default_rng(2)
cov = np.array([[3.0, 2.4], [2.4, 2.4]])
X0 = rng.multivariate_normal([0, 0], cov, 200)
X1 = rng.multivariate_normal([2, 0], cov, 200)
m0, m1 = X0.mean(0), X1.mean(0)
S_W = np.cov(X0.T) + np.cov(X1.T)       # within-class scatter (per-class covariances)
w = np.linalg.solve(S_W, m1 - m0)                          # Fisher direction

def J(v):                        # Fisher criterion for two classes
    return (v @ (m1 - m0)) ** 2 / (v @ S_W @ v)

lda = LinearDiscriminantAnalysis().fit(np.vstack([X0, X1]), np.repeat([0, 1], 200))
cos = w @ lda.coef_[0] / np.linalg.norm(w) / np.linalg.norm(lda.coef_[0])
angles = np.linspace(0, np.pi, 3601)
best = max(J(np.array([np.cos(a), np.sin(a)])) for a in angles)
print(f"J(mean difference) = {J(m1 - m0):.4f}, J(Fisher) = {J(w):.4f}, "
```

```
      f"grid maximum = {best:.4f}")
print(f"cosine between Fisher direction and LDA coef_: {cos:.6f}")
```

封閉解達到網格搜尋的最大值（$J = 3.1841$），超過平均值差（0.6953）的四倍；它的方向與 scikit-learn 高斯 LDA 的係數在小數點後六位一致（餘弦 1.000000）。

感知器

白話版。從任意一條線開始。逐一檢查訓練點；每當有點落在錯誤的一側，就把線往能正確分類它的方向推一點。如果類別可以被一條線分開，這個過程保證會停下來。

精確版。 Rosenblatt 的感知器（perceptron）[6] 使用標籤 $y_n \in \{-1, +1\}$ 與擴增特徵 $\tilde{x}_n$。它最小化**感知器準則**

$$J_P(w) = -\sum_{n \in \mathcal{M}} y_n \, w^\top \tilde{x}_n,$$

其中 $\mathcal{M}$ 是被分錯的樣本集合（$y_n w^\top \tilde{x}_n \le 0$ 者）。每一項都非負，且 $J_P = 0$ 的充要條件是所有樣本都被正確分類。每次對一個分錯的樣本做隨機梯度下降，得到更新規則

$$w \leftarrow w + \eta \, y_n \tilde{x}_n.$$

以零初始化 w 時，學習率 $\eta > 0$ 只會縮放 w，不會改變決策序列，所以常取 $\eta = 1$。IP 13 的感知器一節以影像圖樣的語言介紹同一條規則，並連結到多層網路。

收斂定理。假設資料**以間隔** $\gamma > 0$ **線性可分**：存在單位向量 u，使所有 n 都滿足 $y_n u^\top \tilde{x}_n \ge \gamma$。令 $R = \max_n \|\tilde{x}_n\|$。那麼感知器（從 $w_0 = 0$ 開始）最多更新 $(R/\gamma)^2$ 次。證明只要三行。經過 t 次更新後：

- 每次更新讓 $w^\top u$ 增加 $y_n \tilde{x}_n^\top u \ge \gamma$，所以 $w_t^\top u \ge t\gamma$；
- 每次更新都發生在錯誤上，此時 $y_n w^\top \tilde{x}_n \le 0$，所以 $\|w_t\|^2 = \|w_{t-1}\|^2 + 2 y_n w_{t-1}^\top \tilde{x}_n + \|\tilde{x}_n\|^2 \le \|w_{t-1}\|^2 + R^2$，因此 $\|w_t\|^2 \le t R^2$；
- 由 Cauchy–Schwarz 不等式，$t\gamma \le w_t^\top u \le \|w_t\| \le \sqrt{t}\, R$，所以 $t \le R^2/\gamma^2$。

這個上界與樣本數和維度都無關，只取決於類別之間的空隙相對於資料大小有多「寬」。

限制。如果類別重疊，感知器永遠不會停止，權重會一直循環（實務上常見的修正是保留目前看過最好的權重）。即使收斂了，它也停在第一條碰到的分隔超平面上，而這取決於初始化與資料順序（圖 3 左）。它的分數也不是機率。從損失的角度看，感知器使用 $\max(0, -y\,g(x))$，任何正確分類（無論多勉強）的損失都是零。邏輯斯迴歸與 SVM 都要求比「剛好正確」更多，藉此修正這一點。

程式碼：感知器更新次數與上界

```
import numpy as np
from sklearn.svm import SVC
```

```
rng = np.random.default_rng(1)
X = np.vstack([rng.normal([-1.0, -0.7], 0.6, (30, 2)),
               rng.normal([1.0, 0.7], 0.6, (30, 2))])
y = np.r_[-np.ones(30), np.ones(30)]
Phi = np.c_[X, np.ones(len(X))]          # augmented features (bias as last weight)

def perceptron(Phi, y, rng, max_epochs=1000):
    w, updates = np.zeros(Phi.shape[1]), 0
    for epoch in range(max_epochs):
        errors = 0
        for i in rng.permutation(len(y)):
            if y[i] * (w @ Phi[i]) <= 0:  # misclassified or on the boundary
                w += y[i] * Phi[i]        # learning rate 1 (scale is irrelevant)
                errors += 1
        updates += errors
        if errors == 0:
            return w, updates
    return w, updates

runs = [perceptron(Phi, y, np.random.default_rng(s)) for s in range(200)]
updates = np.array([u for _, u in runs])
print(f"updates over 200 random orders: mean {updates.mean():.1f}, "
      f"max {updates.max()}")
print("all runs separate the data:",
      all(np.all(np.sign(Phi @ w) == y) for w, _ in runs))

# convergence bound (R / gamma)^2, with gamma from the max-margin separator
svm = SVC(kernel="linear", C=1e6).fit(X, y)
u = np.r_[svm.coef_[0], svm.intercept_[0]]
gamma = np.min(y * (Phi @ u)) / np.linalg.norm(u)
R = np.max(np.linalg.norm(Phi, axis=1))
print(f"bound (R/gamma)^2 = {(R / gamma) ** 2:.1f}")
```

在 200 種隨機呈現順序下，感知器平均需要 2.7 次更新（最多 10 次），且每次執行都能分開資料。最壞情況上界是 691：它是保證，不是預測。上界需要一個有效的間隔 γ；我們從硬間隔 SVM（下方 SVM 一節的主題）取得它。由於 SVM 不懲罰偏差，擴增空間中真正的最大間隔可能稍大一些，會給出稍緊的上界。

邏輯斯迴歸

白話版。不要硬性的是／否，而是把線性分數通過一條 S 形曲線壓成機率。擬合權重，讓真實標籤被賦予的機率盡可能高。

精確版。標籤取 $y \in \{0, 1\}$，二元邏輯斯迴歸 [7] 的模型為

$$P(y = 1 \mid x) = \sigma(w^\top \tilde{x}), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

分數 $w^\top \tilde{x}$ 是**對數勝算** $\ln \frac{P(y=1|x)}{P(y=0|x)}$。這個形式並非隨意選的。PR 01 證明了共用共變異數的高斯

類別，其後驗正好是 $\sigma(w^\top x + w_0)$，其中 $w = \Sigma^{-1}(\mu_1 - \mu_0)$；許多其他類別條件分布族也是如此。邏輯斯迴歸保留這個後驗的函數形式，但直接擬合 w，只有 $d+1$ 個參數，而非高斯模型的 $2d + d(d+1)/2 + 1$ 個。Ng 與 Jordan [18] 分析了這種生成式與判別式之間的取捨（PR 02）。

訓練。 Bernoulli 模型的負對數似然是**交叉熵**

$$E(w) = -\sum_{n=1}^{N} \Big[y_n \ln p_n + (1 - y_n) \ln(1 - p_n) \Big], \qquad p_n = \sigma(w^\top \tilde{x}_n).$$

因為 $\sigma'(z) = \sigma(z)(1 - \sigma(z))$，它的梯度與 Hessian 矩陣出奇簡單 [2, Section 4.3]：

$$\nabla E = \sum_{n} (p_n - y_n)\, \tilde{x}_n = \tilde{X}^\top (p - y), \qquad H = \tilde{X}^\top R\, \tilde{X}, \quad R = \operatorname{diag}(p_n(1 - p_n)).$$

梯度和最小平方法一樣是「預測減目標，乘上輸入」的形式。H 是半正定的，所以 E 是凸函數，沒有虛假的局部最小值。牛頓法 $w \leftarrow w - H^{-1}\nabla E$ 可以改寫成每次迭代權重 R 都會改變的加權最小平方問題：**迭代重新加權最小平方法**（iteratively reweighted least squares, IRLS）。程式中請從 logit 以穩定的公式計算損失，例如 $\ln(1 + e^z) =$ `np.logaddexp(0, z)`，而不是對捨入後的機率取對數。

可分資料。如果有超平面能分開訓練類別，把 w 放大會讓每個 p_n 更接近它的標籤，所以 E 一直下降趨近於零，未正則化的解並不存在：$\|w\| \to \infty$。實務上請一定要正則化，例如加上 $\frac{\lambda}{2}\|w\|^2$（scikit-learn 的 `C` 就是 $1/\lambda$）。值得注意的是，權重如何發散是明確定義的：在可分資料上，單純的梯度下降會在方向上收斂到最大間隔 SVM 的解 [19]，「現代觀點」會再回到這一點。

多類別。把 sigmoid 換成 **softmax** $P(y = k \mid x) = \exp(w_k^\top \tilde{x}) / \sum_j \exp(w_j^\top \tilde{x})$，交叉熵換成 K 類別版本。這是有機率輸出的線性機，也正是標準神經網路分類器的最後一層加損失函數。在標籤雜訊或分布偏移下，機率輸出不會自動保持校準（PR 01）。

程式碼：牛頓法（IRLS）與可分資料

```
import numpy as np
from sklearn.linear_model import LogisticRegression

rng = np.random.default_rng(0)
X = np.vstack([rng.normal(-1, 1, (100, 2)), rng.normal(1, 1, (100, 2))])
y = np.r_[np.zeros(100), np.ones(100)]
Phi = np.c_[np.ones(len(X)), X]

def nll(w, lam):
    z = Phi @ w
    # log(1 + e^z) - y z, written stably with logaddexp
    return np.sum(np.logaddexp(0, z) - y * z) + 0.5 * lam * np.sum(w[1:] ** 2)

# Newton's method (iteratively reweighted least squares)
lam, w = 1.0, np.zeros(3)
for it in range(20):
    p = 1 / (1 + np.exp(-(Phi @ w)))
    grad = Phi.T @ (p - y) + lam * np.r_[0, w[1:]]
    H = Phi.T @ (Phi * (p * (1 - p))[:, None]) + lam * np.diag([0, 1, 1])
```

```
    step = np.linalg.solve(H, grad)
    w -= step
    if np.linalg.norm(step) < 1e-10:
        break
sk = LogisticRegression(C=1 / lam, tol=1e-10, max_iter=1000).fit(X, y)
print(f"Newton converged in {it + 1} iterations, loss {nll(w, lam):.4f}")
print("Newton:      ", np.round(w, 4))
print("scikit-learn:", np.round(np.r_[sk.intercept_, sk.coef_[0]], 4))
```

牛頓法在 8 次迭代內收斂，得到與 scikit-learn 相同的權重（截距、w_1、w_2）$= (0.0777, 1.7205, 1.5300)$。懲罰不包含截距，與 scikit-learn 的慣例一致。

下一段程式在可被一條通過原點的直線分開的資料上，對未正則化的邏輯斯損失執行單純的梯度下降，並追蹤權重範數，以及 w 與最大間隔方向之間的餘弦。

```
import numpy as np
from sklearn.svm import LinearSVC

rng = np.random.default_rng(1)
X = np.vstack([rng.normal([2.0, 1.0], 0.6, (40, 2)),
               rng.normal([-2.0, -1.0], 0.6, (40, 2))])
y = np.r_[np.ones(40), -np.ones(40)]              # separable through the origin

# max-margin direction without bias (hard margin approximated by large C)
svm = LinearSVC(fit_intercept=False, loss="hinge", C=1e4, max_iter=200000).fit(X, y)
u = svm.coef_[0] / np.linalg.norm(svm.coef_[0])

w, lr = np.zeros(2), 0.1
for t in range(1, 100001):
    m = y * (X @ w)
    grad = -(X * (y / (1 + np.exp(m)))[:, None]).mean(0)   # logistic loss gradient
    w -= lr * grad
    if t in (10, 100, 1000, 10000, 100000):
        print(f"step {t:6d}: ||w|| = {np.linalg.norm(w):6.2f}, "
              f"cos(w, SVM) = {w @ u / np.linalg.norm(w):.5f}")
```

範數無上限地增長，但很慢（10 步後 0.68，10^5 步後 8.64），而與 SVM 方向的餘弦從 0.87 緩慢爬升到 0.98。這正是 Soudry 等人 [19] 預測的、以對數速度收斂到最大間隔方向的現象。

支援向量機：最大間隔

白話版。在所有能分開類別的直線中，選周圍那條空「馬路」最寬的一條。只有站在馬路邊緣的點才重要；其他點只要不越過馬路，怎麼移動答案都不變。

精確版。使用標籤 $y_n \in \{-1, +1\}$ 與分數 $f(x) = w^\top x + b$（這裡偏差分開寫）。對分隔超平面而言，x_n 到邊界的距離是 $y_n f(x_n)/\|w\|$。因為 (w, b) 與 (cw, cb) 定義同一個平面，我們固定尺度，使最近的點滿足 $y_n f(x_n) = 1$（**標準形式**）。於是每一側的間隔是 $1/\|w\|$，馬路寬度是 $2/\|w\|$。最大化它等價於凸二次規劃 [9, 10]

$$\min_{w,b} \frac{1}{2}\|w\|^2 \quad \text{subject to} \quad y_n(w^\top x_n + b) \geq 1, \quad n = 1, \dots, N.$$

Lagrange 函數與對偶。為每個限制式引入乘數 $a_n \geq 0$：

$$\mathcal{L}(w, b, a) = \frac{1}{2}\|w\|^2 - \sum_n a_n [y_n(w^\top x_n + b) - 1].$$

令 $\partial\mathcal{L}/\partial w = 0$、$\partial\mathcal{L}/\partial b = 0$，得到

$$w = \sum_n a_n y_n x_n, \qquad \sum_n a_n y_n = 0.$$

代回後消去 w 與 b，剩下**對偶問題**

$$\max_a \sum_n a_n - \frac{1}{2}\sum_n \sum_m a_n a_m y_n y_m x_n^\top x_m \quad \text{subject to} \quad a_n \geq 0, \ \sum_n a_n y_n = 0.$$

有三個重要的推論 [2, 11]：

- **支援向量**。Karush–Kuhn–Tucker（KKT）條件包含**互補鬆弛性** $a_n[y_n f(x_n) - 1] = 0$。所以只有恰好落在間隔上的點才有 $a_n > 0$。這些點就是**支援向量**（support vector）；其他點的 $a_n = 0$，拿掉也不會改變解。偏差可由任一支援向量 s 求得：$b = y_s - w^\top x_s$（為了數值穩定，對所有支援向量取平均）。
- **預測的稀疏性**。$f(x) = \sum_{n \in \mathcal{S}} a_n y_n x_n^\top x + b$ 只取決於支援向量集合 $\mathcal{S}$。
- **只有內積**。資料在對偶問題與預測中只以內積 $x_n^\top x_m$ 出現。把內積換成核函數 $k(x_n, x_m)$，就得到非線性 SVM，而完全不必計算 ϕ。這就是**核技巧**，在PR 04 中發展。

軟間隔與鉸鏈損失。真實類別會重疊，所以限制式不可能全部成立。Cortes 與 Vapnik [10] 加入**鬆弛變數** $\xi_n \geq 0$、限制式 $y_n f(x_n) \geq 1 - \xi_n$ 與懲罰 $C\sum_n \xi_n$。在最佳解處 $\xi_n = \max(0, 1 - y_n f(x_n))$，所以軟間隔 SVM 等價於無限制問題

$$\min_{w,b} \frac{1}{2}\|w\|^2 + C\sum_n \max(0,\ 1 - y_n f(x_n)).$$

第二項是**鉸鏈損失**（hinge loss）：超出間隔的點為零，在間隔內與錯誤一側則線性增加。對偶問題中唯一的改變是盒狀限制 $0 \leq a_n \leq C$。所以線性 SVM 就是「ridge 正則化的鉸鏈損失最小化」，可以直接和正則化的邏輯斯迴歸比較。C、核頻寬與特徵縮放的作用，以及 SVM 分數不是機率這件事，都在 PR 04 討論。

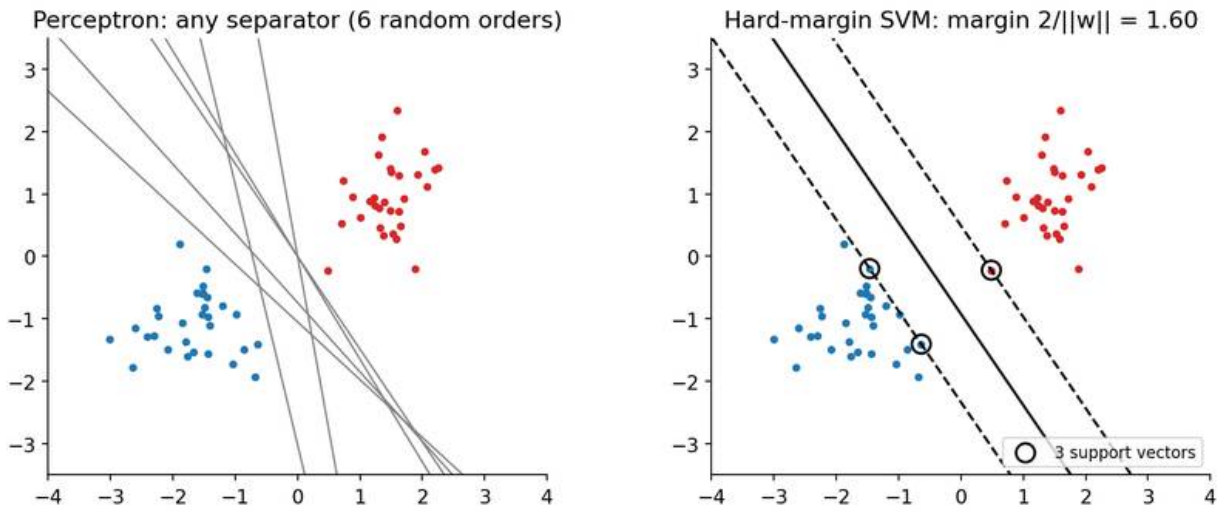


圖 3 — 左：感知器停在它最先碰到的分隔線上（六種隨機順序）。右：硬間隔 *SVM* 選出間隔最寬的唯一直線；只有三個被圈起的支援向量決定它。

程式碼：實際驗證對偶關係

這段檢查擬合一個（近乎）硬間隔的線性 SVM，並以數值驗證對偶關係：$w = \sum_n a_n y_n x_n$、$\sum_n a_n y_n = 0$，以及支援向量上 $y_n f(x_n) = 1$。

```
import numpy as np
from sklearn.svm import SVC

rng = np.random.default_rng(7)
X = np.vstack([rng.normal([-1.5, -1.0], 0.6, (30, 2)),
               rng.normal([1.5, 1.0], 0.6, (30, 2))])
y = np.r_[-np.ones(30), np.ones(30)]

svm = SVC(kernel="linear", C=1e6).fit(X, y)          # huge C ~ hard margin
alpha_y = svm.dual_coef_[0]                          # alpha_n * y_n for SVs
w_dual = alpha_y @ svm.support_vectors_              # w = sum_n alpha_n y_n x_n
f = X @ svm.coef_[0] + svm.intercept_[0]

print("support vectors:", len(svm.support_), "of", len(y))
print("w from dual:", np.round(w_dual, 4), " coef_:", np.round(svm.coef_[0], 4))
print("sum_n alpha_n y_n =", round(float(alpha_y.sum()), 8))
print("y f(x) on support vectors:", np.round(y[svm.support_] * f[svm.support_], 4))
print("smallest y f(x) over all points:", round(float(np.min(y * f)), 4))
print("margin width 2/||w|| =", round(2 / np.linalg.norm(svm.coef_[0]), 4))
```

60 個點中只有 3 個是支援向量（圖 3 中被圈起的點）。由對偶係數重建的權重向量與 `coef_` 完全一致，對偶係數總和為零，支援向量落在 $yf(x) \approx 1$（在求解器容許誤差內），沒有點落在間隔之內，間隔寬度為 1.60。scikit-learn 的 `SVC` 包裝了 LIBSVM [16]；對大型稀疏問題，`LinearS`

VC 與 `LogisticRegression(solver="liblinear")` 使用 LIBLINEAR [14]，而 Pegasos [15] 這類隨機次梯度求解器可擴展到非常大的資料集。

一張圖看懂：線性分類器就是損失函數

白話版。本章所有方法都用同一個線性函數為點打分數。差別在於對每個間隔值 $y\,g(x)$ 收多少費用：只要正確就不收（感知器）、安全地正確才不收（SVM）、平滑遞減的費用（邏輯斯），或以恰好 1 為中心的拋物線（最小平方）。

精確版。標籤取 $y \in \{-1, +1\}$、間隔 $m = y\,g(x)$，每個方法都在最小化某個**代理損失**（surrogate loss）$\varphi(m)$ 的（可能正則化的）平均值，用來代替非凸的零一損失 $\mathbb{1}[m \le 0]$（圖 4）：

方法	損失 $\varphi(m)$	行為
感知器	$\max(0, -m)$	任何正確的點都為零；與尺度無關，所以 $w = 0$ 是平凡的最小解
鉸鏈（SVM）	$\max(0, 1 - m)$	超出間隔即為零；解是稀疏的（支援向量）
邏輯斯	$\ln(1 + e^{-m})$	平滑、永不為零；能給出機率
平方	$(1 - m)^2 = (y - g(x))^2$	懲罰 $m > 1$：「太正確」的點會拉動邊界

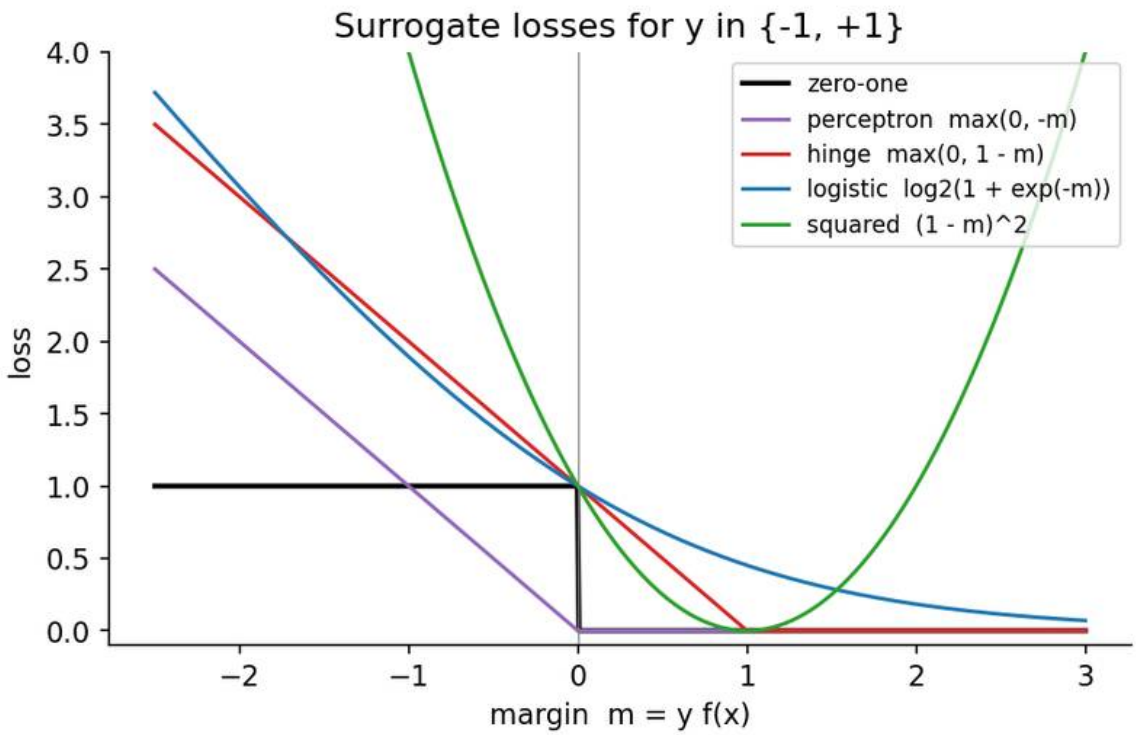


圖 *4* — 以間隔 $m = y\,f(x)$ 表示的代理損失。邏輯斯損失乘上 $1/\ln 2$，使它通過 $(0, 1)$。只有平方損失在 $m > 1$ 時增加，這解釋了圖 *1*。

在資料足夠時，哪些代理損失會得到貝氏最佳分類器？Bartlett、Jordan 與 McAuliffe [12] 給出精確的答案：凸代理損失是**分類校準的**（classification-calibrated，最小化其期望值會得到貝氏決策 $\operatorname{sign}(2\eta(x)-1)$，其中 $\eta(x)=P(y=1\mid x)$），若且唯若它在 0 可微且 $\varphi'(0)<0$。鉸鏈、邏輯斯、平方與指數損失都符合；感知器損失不符合。他們也把代理風險的超額量與分類風險的超額量連結起來。Rosasco 等人 [13] 比較不同凸損失所隱含的收斂速率，結論是對分類而言，鉸鏈損失收斂得和邏輯斯損失差不多快，比平方損失快得多。不過這些損失估計的東西仍不同：逐點最小化期望邏輯斯損失得到 $g(x)=\ln\frac{\eta(x)}{1-\eta(x)}$，所以它的分數是對數勝算（經 sigmoid 即為機率）；而期望鉸鏈損失的逐點最小解是 $\operatorname{sign}(2\eta(x)-1)$，不帶任何機率資訊。這就是 SVM 分數需要另外校準的原因。

程式碼：在 digits 上比較線性分類器

內建的 digits 資料集（8×8 影像、10 類）不需下載。這只是一次可重現的保留集示範，不是基準排名。每個縮放器都在管線內、只用訓練資料擬合。

```
from sklearn.datasets import load_digits
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression, Perceptron, RidgeClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

X, y = load_digits(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=7)
models = {
    "least squares (ridge)": make_pipeline(StandardScaler(), RidgeClassifier(alpha=1.0)),
    "perceptron": make_pipeline(StandardScaler(),
                                Perceptron(max_iter=1500, random_state=7)),
    "logistic": make_pipeline(StandardScaler(),
                              LogisticRegression(C=1, max_iter=1500)),
    "linear SVM": make_pipeline(StandardScaler(),
                                LinearSVC(C=0.1, max_iter=20000, random_state=7)),
    "Gaussian LDA": LinearDiscriminantAnalysis(solver="lsqr", shrinkage="auto"),
}
for name, model in models.items():
    model.fit(Xtr, ytr)
    print(f"{name:22s} test accuracy: {accuracy_score(yte, model.predict(Xte)):.4f}")
```

在這次切分上，邏輯斯迴歸達到 97.1%，線性 SVM 96.4%，感知器 95.1%，ridge 正則化最小平方法與收縮 LDA 各為 94.7%。五者都是同樣 64 個像素上的線性機；差異完全來自損失與正則化。看過這些測試結果後再調整正則化，會讓這個測試集變成驗證集。真正的模型選擇需要獨立的驗證切分，或在訓練資料內做交叉驗證（PR 09）。

現代觀點

線性分類器沒有被深度學習取代；它們移到了深度學習的最上層。三條近期的研究脈絡說明它們為何仍然重要，而經典的綜述文獻對方法本身仍很有用。

綜述與求解器。 Burges 的教學文 [11] 仍是 SVM 最易讀的推導，從 VC 維度的動機一路講到對偶與核。Cervantes 等人 [17] 綜述 2020 年時的 SVM 分類：各種變形、應用，以及大型資料集上的訓練成本、類別不平衡與模型選擇等挑戰。計算方面，LIBLINEAR [14] 讓擁有數百萬稀疏特徵的線性 SVM 與邏輯斯迴歸成為例行工作；LIBSVM [16] 是 scikit-learn `SVC` 背後的核 SVM 參考實作；Pegasos [15] 則證明在 SVM 原始問題上做隨機次梯度下降，對線性核而言達到指定準確度所需的執行時間不直接取決於訓練集大小，這與用 SGD 訓練神經網路是同一個概念。

線性探針：衡量表徵的尺。 Alain 與 Bengio [21] 提議在網路的每一層獨立訓練線性分類器（「探針」，probe），並觀察到線性可分性隨深度單調增加。Kornblith、Shlens 與 Le [22] 在 12 個資料集上比較 16 個網路，發現把網路當固定特徵抽取器、上接邏輯斯迴歸分類器時，ImageNet 準確率與遷移準確率高度相關；但有些讓 ImageNet 準確率略為提升的正則化方式，會讓倒數第二層的特徵差很多。CLIP [23] 把它的零樣本分類器與在凍結特徵上訓練的邏輯斯迴歸探針做比較；DINOv2 [24] 也主要以凍結骨幹上的線性分類器評估其自監督特徵。Kumar 等人 [25] 證明完整微調可能扭曲好的預訓練特徵，在分布外資料上表現不如線性探針，並提出先擬合線性探針再微調（LP-FT）。換句話說：在凍結的基礎模型上，訓練一個良好正則化的邏輯斯迴歸，是很強的基準，往往也是正確的第一個實驗。

隱式偏差：邏輯斯迴歸偷偷找到了 SVM。 Soudry 等人 [19] 證明，在線性可分資料上對未正則化的邏輯斯（或交叉熵）損失做梯度下降，會在方向上收斂到硬間隔 SVM 的解；他們也把結果推廣到其他指數尾部的損失、多類別問題，以及受限設定下的深度網路。收斂速度只有對數級，這有助於解釋為何在訓練錯誤率為零後繼續訓練，仍能持續改善間隔。上面的梯度下降程式在二維中呈現了這個效果。

神經崩塌：訓練末期的 Fisher 圖像。 Papyan、Han 與 Donoho [20] 在三種架構與七個資料集上觀察到：在訓練的終端階段（訓練錯誤率為零之後），每個類別最後一層的特徵會崩塌到類別平均；類別平均排列成最大分離的對稱構型（單純形等角緊框架，simplex equiangular tight frame）；分類器權重與類別平均對齊；決策規則變成最近類別平均。用本章的語言說，類內散布 S_W 縮向零，Fisher 準則無限增大，網路最後一層變成 PR 01 的最小距離分類器。

開放問題。線性探針的結果取決於正則化與資料量，我們何時能相信它是表徵品質的衡量？隱式偏差的結果如何推廣到真實的、不可分的、有資料擴增的訓練？線性輸出層的機率在分布偏移下何時能保持校準？這些問題把本系列最古老的分類器和當前研究連在一起。

重點整理

- 線性判別函數 $g(x) = w^\top x + b$ 是以 w 為法向量的超平面；$g(x)/\|w\|$ 是帶號距離。多類別問題請每類用一個判別函數（線性機）。
- 最小平方法與 ridge 有封閉解與貝氏 MAP 詮釋，但最小平方法是差勁的分類器：它懲罰「太正確」的點，也可能遮蔽類別。
- Fisher 方向 $w \propto S_W^{-1}(m_1 - m_0)$ 最大化類間對類內的分散比；它等於高斯 LDA 的方向，後者再由平均與先驗加上門檻。

- 在可分資料上，感知器最多更新 $(R/\gamma)^2$ 次後收斂，但會停在任意一條分隔線上，在重疊資料上則會循環。
- 邏輯斯迴歸以凸的交叉熵最小化（牛頓法／IRLS）擬合對數勝算；沒有正則化時，在可分資料上權重會發散，方向趨向最大間隔解。
- 硬間隔 SVM 最大化間隔 $2/\|w\|$；它的對偶顯示只有支援向量重要，而且資料只以內積出現，為核方法（PR 04）開啟大門。軟間隔就是 ridge 正則化的鉸鏈損失。
- 這些方法都是同一個線性分數配上不同的代理損失；鉸鏈、邏輯斯與平方損失都是分類校準的，但只有邏輯斯損失能估計機率。

練習

1. **對參數線性**。證明以最小平方法擬合 $g(x) = w_0 + w_1 x + w_2 x^2$ 是線性迴歸問題，並描述若把同樣的特徵用在兩類別線性分類器上，x 上的決策區域長什麼樣子。

   提示

   建立設計矩陣的列 $(1, x, x^2)$；模型對 (w_0, w_1, w_2) 是線性的。x 的二次式最多變號兩次，所以一個類別的區域可以是一段區間，另一類則是它的補集（兩條半直線）。

2. **單位與 ridge**。兩個特徵量測同一段距離，一個以公尺、一個以毫米表示。說明沒有特徵縮放時 ridge 迴歸會怎麼做，以及為何標準化能解決問題。

   提示

   描述同樣效果時，毫米特徵的係數可以小 1,000 倍，因此受到的懲罰遠小得多；ridge 會把幾乎所有權重放到毫米特徵上。標準化之後，兩個座標在同樣的尺度上受懲罰。

3. **手算 Fisher**。兩類別的平均為 $m_0 = (0, 0)$、$m_1 = (2, 0)$，類內散布 $S_W = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$。計算 Fisher 方向並與平均值差比較。兩個方向何時會重合？

   提示

   $S_W^{-1} = \frac{1}{3}\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$，所以 $w \propto S_W^{-1}(2, 0)^\top = \frac{1}{3}(4, -2) \propto (2, -1)$，因為類別沿 $(1, 1)$ 方向拉長，所以方向從平均值差 $(1, 0)$ 轉開。當 $m_1 - m_0$ 是 S_W 的特徵向量時兩方向恰好重合，例如 $m_1 = (2, 2)$。

4. **永不停止的感知器**。建構一個讓感知器永遠更新下去的一維資料集，並說明收斂定理的哪個假設不成立。

   提示

   在同一座標放兩個標籤相反的點（或任何重疊的類別）。不存在 $\gamma > 0$ 的 u，所以上界不適用；每次修正一個點的更新，都會弄壞另一個點。

5. **手算 SVM**。求兩點 $x_1 = (1, 1)$、$y_1 = +1$ 與 $x_2 = (-1, -1)$、$y_2 = -1$ 的硬間隔 SVM。給出 w、b、對偶變數與間隔寬度，並檢查 KKT 條件。

   提示

   由對稱性 $b = 0$、$w = c(1, 1)$。限制式 $y_1 w^\top x_1 = 1$ 給出 $2c = 1$，所以 $w = (\tfrac{1}{2}, \tfrac{1}{2})$，寬度 $2/\|w\| = 2\sqrt{2}$，正是兩點間的距離。由 $w = a_1 x_1 - a_2 x_2$ 與 $a_1 = a_2$ 得 $a_1 = a_2 = \tfrac{1}{4}$。兩點都是支援向量。

6. **邏輯斯與鉸鏈**。用 digits 管線，在訓練集上以 5 折交叉驗證，為多個 `C` 值擬合 `LogisticRegression` 與 `LinearSVC`。比較兩者最佳的驗證準確率，再比較它們的分數：哪一個能給出有意義的機率？另一個要如何得到機率？

   提示

   準確率通常很接近，正如損失分析所預測。只有邏輯斯迴歸的 `predict_proba` 估計後驗；對 SVM，要在保留資料上擬合校準映射（Platt scaling 或保序迴歸，例如 `CalibratedClassifierCV`）。

參考文獻

1. R. O. Duda, P. E. Hart and D. G. Stork, *Pattern Classification*, 2nd ed., Wiley, 2001 (Ch. 5, linear discriminant functions). publisher page
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006 (Sections 3.1–3.3, 4.1–4.3 and 7.1). official page and free PDF
3. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2008 (Ch. 3, linear classifiers). publisher page
4. T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, 2009 (Ch. 3–4 and 12). official page
5. R. A. Fisher, "The use of multiple measurements in taxonomic problems," *Annals of Eugenics*, vol. 7, no. 2, pp. 179–188, 1936. doi
6. F. Rosenblatt, "The perceptron: a probabilistic model for information storage and organization in the brain," *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958. doi
7. D. R. Cox, "The regression analysis of binary sequences," *Journal of the Royal Statistical Society, Series B*, vol. 20, no. 2, pp. 215–232, 1958. doi
8. A. E. Hoerl and R. W. Kennard, "Ridge regression: biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970. doi
9. B. E. Boser, I. M. Guyon and V. N. Vapnik, "A training algorithm for optimal margin classifiers," in *Proc. 5th Annual Workshop on Computational Learning Theory (COLT)*, pp. 144–152, 1992. doi
10. C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995. doi
11. C. J. C. Burges, "A tutorial on support vector machines for pattern recognition," *Data Mining and Knowledge Discovery*, vol. 2, no. 2, pp. 121–167, 1998. doi
12. P. L. Bartlett, M. I. Jordan and J. D. McAuliffe, "Convexity, classification, and risk bounds," *Journal of the American Statistical Association*, vol. 101, no. 473, pp. 138–156, 2006. doi

13. L. Rosasco, E. De Vito, A. Caponnetto, M. Piana and A. Verri, "Are loss functions all the same?," *Neural Computation*, vol. 16, no. 5, pp. 1063–1076, 2004. doi
14. R.-E. Fan, K.-W. Chang, C.-J. Hsieh, X.-R. Wang and C.-J. Lin, "LIBLINEAR: a library for large linear classification," *Journal of Machine Learning Research*, vol. 9, pp. 1871–1874, 2008. paper
15. S. Shalev-Shwartz, Y. Singer, N. Srebro and A. Cotter, "Pegasos: primal estimated sub-gradient solver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011. doi
16. C.-C. Chang and C.-J. Lin, "LIBSVM: a library for support vector machines," *ACM Transactions on Intelligent Systems and Technology*, vol. 2, no. 3, article 27, 2011. doi
17. J. Cervantes, F. Garcia-Lamont, L. Rodríguez-Mazahua and A. Lopez, "A comprehensive survey on support vector machine classification: applications, challenges and trends," *Neurocomputing*, vol. 408, pp. 189–215, 2020. doi
18. A. Y. Ng and M. I. Jordan, "On discriminative vs. generative classifiers: a comparison of logistic regression and naive Bayes," in *Advances in Neural Information Processing Systems* (*NIPS*), 2001. paper
19. D. Soudry, E. Hoffer, M. S. Nacson, S. Gunasekar and N. Srebro, "The implicit bias of gradient descent on separable data," *Journal of Machine Learning Research*, vol. 19, no. 70, pp. 1–57, 2018. paper
20. V. Papyan, X. Y. Han and D. L. Donoho, "Prevalence of neural collapse during the terminal phase of deep learning training," *Proceedings of the National Academy of Sciences*, vol. 117, no. 40, pp. 24652–24663, 2020. doi
21. G. Alain and Y. Bengio, "Understanding intermediate layers using linear classifier probes," arXiv:1610.01644, 2016. arXiv
22. S. Kornblith, J. Shlens and Q. V. Le, "Do better ImageNet models transfer better?," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), 2019. paper
23. A. Radford et al., "Learning transferable visual models from natural language supervision," in *Proc. International Conference on Machine Learning* (*ICML*), PMLR 139, 2021. paper
24. M. Oquab et al., "DINOv2: learning robust visual features without supervision," *Transactions on Machine Learning Research*, 2024. arXiv
25. A. Kumar, A. Raghunathan, R. Jones, T. Ma and P. Liang, "Fine-tuning can distort pretrained features and underperform out-of-distribution," in *Proc. International Conference on Learning Representations* (*ICLR*), 2022. arXiv

PR 04・近鄰、核與非線性分類器

從最近鄰與核技巧，到軟間隔 SVM、RBF 網路、決策樹與多層感知器：分類器如何讓決策邊界彎曲，以及每一種彎法的代價。

網站版本: https://blog.shyandram.dev/zh-tw/learn/pattern-recognition/pr-04-kernels-nonlinear-classifiers/

你將學到

- 最近鄰規則為什麼不必擬合任何參數就能產生非線性決策區域，以及距離度量為何和 k 一樣重要。
- 核技巧（kernel trick）是什麼、什麼樣的函數才是有效的（Mercer）核，以及如何在有限的 Gram 矩陣上檢查。
- 軟間隔支持向量機（soft-margin SVM）：原始問題與對偶問題、支持向量，以及超參數 C 與 γ 如何塑造邊界。
- 高斯過程與相關向量機如何以機率的角度重複利用核。
- 另外三條通往非線性的路：徑向基函數（RBF）網路、以不純度（impurity）準則生長的決策樹（CART），以及多層感知器。
- 如何公平比較這些方法家族，以及它們在 2026 年的位置。

先看全貌

PR 03 建立的分類器，決策邊界都是直線、平面或超平面。許多真實問題並非如此：環內的點無法用任何一條直線和環外的點分開；手寫的「1」與「7」在像素空間中沿著彎曲、扭轉的區域互相重疊。我們需要會彎的邊界。

讓邊界彎曲的基本做法只有幾種：

1. **和儲存的樣本比較**。找出與新點最相似的訓練點，參考它們的標籤（最近鄰）。
2. **先換表徵，再維持線性**。把每個輸入 x 映射成新特徵 $\phi(x)$，讓超平面在新空間中可行。核方法隱式地做這件事；RBF 網路與多層感知器則以固定或學習得到的基底函數明確地做。
3. **把空間切成小塊**。決策樹提出一連串是非題，每條回答路徑最後落在一個帶有自己標籤的小方塊。

用最簡單的話說

想像你在公園裡替撿到的葉子分類。你可以把新葉子放在已貼好標籤的收藏旁，照抄最像的那幾片的標籤（近鄰）。你可以戴上一副特殊眼鏡，讓差異一下子跳出來，於是簡單的規則又能用了（核與特徵）。你也可以玩「二十個問題」：「比我的拇指寬嗎？邊緣有鋸齒嗎？」問到只剩一種葉子為止（決策樹）。

本章每一種方法都有同一個前提：非線性分類器只能利用量測中已有的資訊。如果兩個類別在你記錄的特徵裡看起來一模一樣，再怎麼彎曲邊界也分不開。特徵設計是PR 05 的主題。本章的教科書背景是 Theodoridis 與 Koutroumbas 第 2、4 章 [1]，以及 Bishop 第 2.5.2、5.1–5.3、6.1–6.4、7.1–7.2 與 14.4 節 [2]。

最近鄰分類

白話版。要分類一個新點，就找出離它最近的 k 個訓練點，讓它們投票。除了把資料存起來，什麼都沒有「學」，但得到的邊界可以順著資料呈現任何形狀。

規則與它的保證

給定訓練資料對 (x_n, y_n)，$n = 1, \dots, N$，以及查詢點 x，令 $N_k(x)$ 為在某個距離 d 之下最接近 x 的 k 個訓練點索引。k 近鄰（k-NN）規則預測

$$\hat{y}(x) = \arg\max_{c} \sum_{n \in N_k(x)} w_n \, \mathbf{1}[y_n = c],$$

其中 c 走遍所有類別；$\mathbf{1}[\cdot]$ 在括號內條件成立時為 1，否則為 0；w_n 是投票權重：單純多數決時 $w_n = 1$，距離加權投票時則取 $d(x, x_n)$ 的遞減函數。

各類票數比例是類別後驗機率 $P(c \mid x)$ 的粗略估計：k-NN 其實是一種換了外衣的無母數密度方法（PR 02），鄰域大小由 k 決定，而非固定的頻寬 [2]。Cover 與 Hart 證明了 1-NN 規則的經典結果：當訓練樣本數無限增加時，它的錯誤率至多是貝氏錯誤率的兩倍 [3]。這是漸近的敘述；在有限資料下，尤其是高維度時，k-NN 可能離這個界限很遠。

度量方式是一種建模決策

歐氏距離把每個座標上的一單位差異視為同樣重要。如果一個特徵以公釐量測、另一個以公里量測，前者會主宰每個鄰域。把特徵標準化、學習一個 Mahalanobis 度量 $d(x, z)^2 = (x - z)^\top M (x - z)$（$M$ 為半正定矩陣），或以 cosine 相似度比較正規化後的嵌入（embedding），都會改變誰算是鄰居。

對於已正規化為單位長度的向量 u 與 v，

$$\|u - v\|^2 = \|u\|^2 + \|v\|^2 - 2u^\top v = 2 - 2u^\top v.$$

因此歐氏距離最近的鄰居，恰好就是 cosine 相似度最大的鄰居：兩個準則對候選點的排序剛好相反，選出的鄰居卻相同。這就是為什麼在正規化的深度特徵上做最近鄰搜尋（ML 05）時，兩者可以互換。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

選擇 k，以及預測的成本

小的 k（例如 1）會給出鋸齒狀的邊界，緊跟每個帶雜訊的訓練點：偏差低、變異高。大的 k 在較大區域內平均，使邊界平滑，但可能把小類別整個抹掉。k、度量方式與尺度都應該只用訓練資料，以交叉驗證來選擇。

k-NN 把所有成本都推到預測階段。單純的查詢要把 x 和全部 N 個訓練點比較，對 D 維輸入的成本是 $O(ND)$。樹狀索引與近似最近鄰搜尋在實務上可以降低這個成本。在高維度時還會出現另一個問題：當許多座標帶有無關的變異，隨機點之間的距離會集中在某個共同值附近，「最近」便不再代表「最相似」。本章稍後圖 3 的右圖顯示二維玩具問題上的 7-NN 邊界，那裡不會出現這些問題。

核：核技巧與 Mercer 條件

白話版。許多線性演算法只透過內積 $x^\top z$ 使用輸入。如果把每個內積換成某個「相似度函數」$k(x, z)$，而它恰好等於某個更豐富特徵空間中的內積，我們就能用線性演算法的價格得到非線性演算法，而且從頭到尾不必算出那些豐富的特徵。

把資料抬升

看圖 1 左邊的環狀資料。沒有任何直線能把內圈的圓盤和外環分開。加入第三個特徵：半徑平方 $r^2 = x_1^2 + x_2^2$。在新的三維空間中，內圈點位置低、外環點位置高，一個平面就能完美分開它們。抬升後空間中的線性分類器，在原空間中就是一個圓。

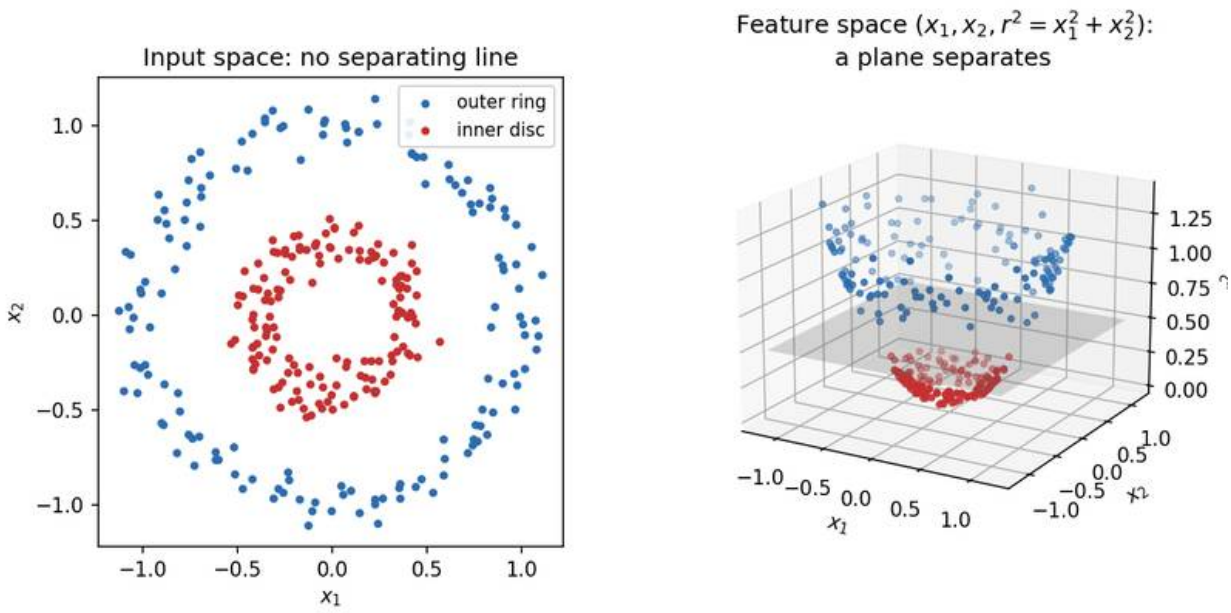


圖 *1* — 抬升到特徵空間。加入特徵 $r^2 = {x_1}^2 + {x_2}^2$ 後，圓形邊界變成一個平面。

明確抬升很快就變得昂貴。D 個變數、次數不超過 m 的所有單項式共有 $\binom{D+m}{m}$ 個；對 16×16 的影像（$D=256$）且 $m=3$，大約是 290 萬個特徵。核方法可以避免把它們建出來。

核技巧

核是一個函數 $k(x,z)$，存在某個映射到內積空間的特徵映射 ϕ，使得

$$k(x,z)=\phi(x)^{\top}\phi(z).$$

其中 x 與 z 是輸入，$\phi(x)$ 是 x 的（可能非常長、甚至無限維的）特徵向量，右式是它們的內積。舉例來說，在二維中令 $x=(a,b)$，

$$(x^{\top}z+1)^2=\phi(x)^{\top}\phi(z),\qquad \phi(x)=(1,\ \sqrt{2}a,\ \sqrt{2}b,\ a^2,\ b^2,\ \sqrt{2}ab).$$

計算左式只需一次內積與一次平方；右式每個點需要六個特徵。在 D 維、次數 m 時，左式的成本仍是 $O(D)$。

核技巧指的是：任何完全以資料點之間內積寫成的演算法，都可以把 $x_i^{\top}x_j$ 換成 $k(x_i,x_j)$ 而「核化」[7]。SVM 對偶問題、核嶺迴歸、核 PCA（PR 05）與高斯過程都屬於這一類。所有核值收集在 **Gram 矩陣** $K\in\mathbb{R}^{N\times N}$，$K_{ij}=k(x_i,x_j)$。

下面的程式以數值驗證這個等式，接著示範一個常見、卻不是有效核的「相似度」。

```
import numpy as np

rng = np.random.default_rng(0)
x, z = rng.normal(size=2), rng.normal(size=2)

def phi(v):  # explicit feature map of (v.w + 1)^2 in 2-D
    a, b = v
    s = np.sqrt(2)
    return np.array([1, s * a, s * b, a * a, b * b, s * a * b])

lhs = (x @ z + 1) ** 2
rhs = phi(x) @ phi(z)
print("kernel:", round(lhs, 6), " explicit:", round(rhs, 6))
assert np.isclose(lhs, rhs)

X = rng.normal(size=(40, 2))
K_sig = np.tanh(2.0 * X @ X.T - 1.0)        # "sigmoid kernel"
print("sigmoid Gram min eigenvalue:", np.linalg.eigvalsh(K_sig).min().round(3))
```

兩個數字相同。tanh「sigmoid 核」的 Gram 矩陣有明顯為負的特徵值（此例約為 -15），所以在這組參數下，不存在任何特徵映射 ϕ 能重現它。

哪些函數是核？Mercer 條件

函數 k 是有效的核，若且唯若它**對稱**，即 $k(x,z)=k(z,x)$，而且**半正定**：對任何有限點集 $x_1,\dots,x_N$ 與任何實數向量 $a\in\mathbb{R}^N$，

$$a^{\top} K a = \sum_{i=1}^{N}\sum_{j=1}^{N} a_i a_j\, k(x_i, x_j) \;\geq\; 0.$$

換句話說，每個 Gram 矩陣的特徵值都非負。只有對稱是不夠的，sigmoid 的例子就說明了這點。Mercer 定理 [4] 給出連續版本。對緊緻定義域 $\mathcal{X}$ 上連續且對稱的 k，若它在積分意義下半正定，

$$\int_{\mathcal{X}}\int_{\mathcal{X}} g(x)\,k(x,z)\,g(z)\,dx\,dz \geq 0 \quad \text{對所有平方可積的} g,$$

則可以展開成 $k(x,z)=\sum_{i=1}^{\infty}\lambda_i\, e_i(x)\, e_i(z)$，其中特徵值 $\lambda_i \geq 0$，e_i 為正交歸一的特徵函數。此時特徵映射就是 $\phi(x)=(\sqrt{\lambda_1}e_1(x), \sqrt{\lambda_2}e_2(x), \dots)$。用現代的說法，每個半正定核都定義一個函數構成的**再生核希爾伯特空間**（reproducing kernel Hilbert space, RKHS），而表示定理（representer theorem）指出：在這個空間中做正則化擬合，解必定是有限組合 $f(x)=\sum_{n}\alpha_n k(x_n, x)$ [7]。

有效的核可以由有效的核組合出來。若 k_1 與 k_2 是核，那麼 k_1+k_2、$c\,k_1$（$c>0$）、$k_1 k_2$、$\exp(k_1)$，以及對任意函數 f 的 $f(x)k_1(x,z)f(z)$ 也都是核 [2]。常見的核如下：

核	公式	參數	說明
線性	$x^{\top}z$	無	回到線性模型
多項式	$(\gamma x^{\top}z + r)^m$	$\gamma>0$，$r>0$，整數 $m\geq 1$	包含次數至多 m 的所有單項式（$r=0$ 時只有 m 次項）
高斯 RBF	$\exp(-\gamma\lVert x-z\rVert^2)$	$\gamma>0$	無限維特徵空間；局部相似度
Laplacian	$\exp(-\gamma\lVert x-z\rVert_1)$	$\gamma>0$	函數比高斯核粗糙
Sigmoid	$\tanh(\gamma x^{\top}z + r)$	γ, r	一般而言不是半正定

高斯 RBF 核是預設的非線性選擇。參數 γ（有時寫成 $1/(2\sigma^2)$，σ 為頻寬）決定相似度能延伸多遠。γ 大時，每個訓練點只影響極小的鄰域；γ 小時，核在資料範圍內幾乎是常數，模型趨近非常平滑、近乎線性的函數。由於 γ 乘在距離平方上，它和特徵的尺度綁在一起：使用 RBF 核之前一定要先標準化。

核嶺迴歸：一行看懂核技巧

使用特徵 ϕ 的嶺迴歸最小化 $\sum_n (t_n - w^{\top}\phi(x_n))^2 + \lambda\lVert w\rVert^2$。它的解可以寫成 $w=\sum_n \alpha_n \phi(x_n)$，於是

$$\alpha = (K + \lambda I)^{-1} t, \qquad f(x) = k_x^{\top}\alpha,$$

其中 $t=(t_1,\dots,t_N)^{\top}$ 是目標值，$\lambda>0$ 是正則化強度，I 是 $N\times N$ 單位矩陣，$k_x=(k(x_1,x),\dots,k(x_N,x))^{\top}$ 是查詢點 x 與各訓練點的相似度。原本 D 維特徵空間中的求解，變成 $N\times N$ 樣本空間中的求解：特徵多、樣本少時很划算；N 很大時則很昂貴（時間 $O(N^3)$，記憶體 $O(N^2)$）。

```
import numpy as np
from sklearn.kernel_ridge import KernelRidge
from sklearn.metrics.pairwise import rbf_kernel

rng = np.random.default_rng(1)
```

```
X = rng.uniform(-3, 3, size=(60, 1))
t = np.sin(X[:, 0]) + 0.1 * rng.normal(size=60)
lam, gamma = 0.1, 0.5

K = rbf_kernel(X, X, gamma=gamma)
alpha = np.linalg.solve(K + lam * np.eye(len(X)), t)
Xq = np.linspace(-3, 3, 5)[:, None]
f_ours = rbf_kernel(Xq, X, gamma=gamma) @ alpha
f_skl = KernelRidge(alpha=lam, kernel="rbf", gamma=gamma).fit(X, t).predict(Xq)
print(np.round(f_ours, 4))
assert np.allclose(f_ours, f_skl)
```

手寫的求解與 scikit-learn 的 `KernelRidge` 結果一致，五個查詢點上的預測也貼合 $\sin x$。

支持向量機

白話版。在所有能分開兩類的邊界中，挑出中間那條空白「街道」最寬的一條。只有位在街道邊緣或街道內的點才重要，它們就是支持向量。允許少數點以代價 C 進入街道，方法就能處理互相重疊的資料；把內積換成核，街道就能彎曲。

硬間隔與軟間隔

使用標籤 $y_n \in \{-1, +1\}$ 與分數 $f(x) = w^\top \phi(x) + b$，$w$ 是權重向量，b 是偏差；預測取 f 的正負號。若資料可分，可以調整 w 與 b 的尺度，使最近的點滿足 $y_n f(x_n) = 1$。兩條線 $f = \pm 1$ 之間的距離，也就是**間隔**（margin），等於 $2/\|w\|$。最大化間隔，等於在所有 n 都滿足 $y_n f(x_n) \geq 1$ 的條件下最小化 $\frac{1}{2}\|w\|^2$：這是 Boser、Guyon 與 Vapnik 的硬間隔 SVM [5]。

真實資料會重疊，因此 Cortes 與 Vapnik 引入鬆弛變數 $\xi_n \geq 0$，衡量每個點違反間隔的程度 [6]。**軟間隔 SVM** 求解

$$\min_{w,b,\xi} \; \frac{1}{2}\|w\|^2 + C\sum_{n=1}^{N} \xi_n \quad \text{subject to} \quad y_n f(x_n) \geq 1 - \xi_n, \;\; \xi_n \geq 0.$$

$\xi_n = 0$ 的點位在正確的一側，且間隔至少為 1；$0 < \xi_n \leq 1$ 的點落在街道內但仍分類正確；$\xi_n > 1$ 則表示分類錯誤。常數 $C > 0$ 是違反間隔的價格。消去 ξ 可得等價的無約束形式

$$\min_{w,b} \; \frac{1}{2}\|w\|^2 + C\sum_{n} \max\left(0,\; 1 - y_n f(x_n)\right),$$

也就是加上正則化的**鉸鏈損失**（hinge loss）。滿足 $y_n f(x_n) \geq 1$ 的點完全不貢獻損失，這正是 SVM 稀疏性的來源。（有些教科書把損失取平均、把常數放在正則項，寫成 $\frac{\lambda}{2}\|w\|^2 + \frac{1}{N}\sum_n \text{hinge}$；兩者在 $C = 1/(\lambda N)$ 時相同。）

對偶問題與支持向量

對每個間隔約束引入拉格朗日乘數 $\alpha_n \geq 0$，再消去 w、b 與 ξ，就得到**對偶問題** [2], [6]：

$$\max_{\alpha} \sum_{n=1}^{N} \alpha_n - \frac{1}{2} \sum_{n=1}^{N} \sum_{m=1}^{N} \alpha_n \alpha_m y_n y_m \, k(x_n, x_m) \quad \text{subject to} \quad 0 \le \alpha_n \le C, \ \sum_{n=1}^{N} \alpha_n y_n = 0.$$

這是 N 個乘數 α_n 的凸二次規劃問題，而資料只透過核值 $k(x_n, x_m)$ 出現，核技巧就是從這裡進入。最佳權重向量是 $w = \sum_n \alpha_n y_n \phi(x_n)$，所以預測也只需要核：

$$f(x) = \sum_{n=1}^{N} \alpha_n y_n \, k(x_n, x) + b.$$

Karush–Kuhn–Tucker（KKT）最佳性條件把訓練點分成三組：

- $\alpha_n = 0$：點在街道外（$y_n f(x_n) \ge 1$），對 f 沒有影響；
- $0 < \alpha_n < C$：點恰好落在間隔上（$y_n f(x_n) = 1$），由這些點決定 b；
- $\alpha_n = C$：點在街道內或被分錯（$y_n f(x_n) \le 1$）。

$\alpha_n > 0$ 的點就是**支持向量**。刪掉任何其他點再重新訓練，得到的分類器完全相同。支持向量有多少取決於資料：在嚴重重疊的問題上，訓練集中很大一部分都可能成為支持向量，預測就會和最近鄰搜尋一樣昂貴。

C 與 γ 如何塑造 RBF-SVM

使用高斯核時，一切由兩個數字控制。C 決定訓練錯誤相對於寬間隔的代價；γ 決定核有多局部。圖 2 在同一份帶雜訊資料上掃過兩者。

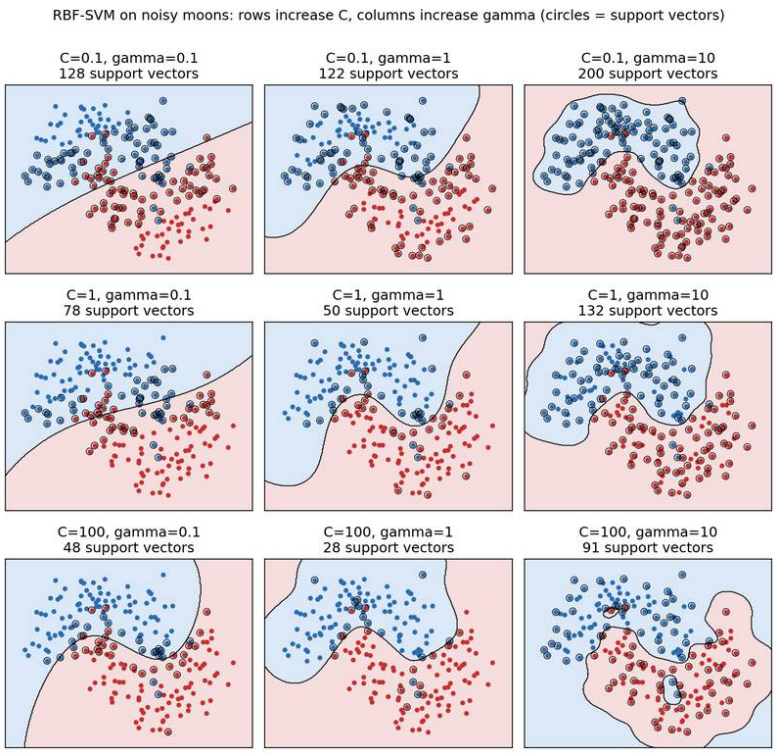


圖 2 — 帶雜訊雙月形資料（已標準化）上的 *RBF-SVM*。γ 小時邊界近乎線性；γ 大時會在個別點周圍形成小島。C 小時容許許多間隔違反（支持向量多）；C 大時更用力擬合訓練點。由 *scripts/figures/pr_04_kernels_nonlinear_classifiers_extra.py* 產生。

這張格狀圖可以這樣讀。往下走（C 變大），邊界會追著個別訓練點跑，而支持向量數變少，因為允許進入間隔的點變少了。往右走（γ 變大），每個點的影響範圍縮小；到 $\gamma = 10$ 時分類器開始畫出小島，這就是過擬合（overfitting）。好的區域是一條對角帶：γ 變大時，通常需要較小的 C。實務上，會在對數尺度的網格上用交叉驗證搜尋 C 與 γ。

```
import numpy as np
from sklearn.datasets import make_moons
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = make_moons(n_samples=500, noise=0.18, random_state=2)
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=2)
models = {
    "linear SVM": SVC(kernel="linear", C=1),
    "RBF SVM": SVC(kernel="rbf", C=3, gamma=0.7),
    "7-neighbor": KNeighborsClassifier(n_neighbors=7),
}
for name, estimator in models.items():
    model = make_pipeline(StandardScaler(), estimator).fit(Xtr, ytr)
    print(name, "test accuracy:", round(model.score(Xte, yte), 4))

search = GridSearchCV(
    make_pipeline(StandardScaler(), SVC(kernel="rbf")),
    {"svc__C": [0.1, 1, 10, 100], "svc__gamma": [0.01, 0.1, 1, 10]},
    cv=5)
search.fit(Xtr, ytr)
best = search.best_estimator_
print("best:", search.best_params_,
      "CV acc:", round(search.best_score_, 4),
      "test acc:", round(best.score(Xte, yte), 4),
      "support vectors:", best[-1].n_support_.sum(), "of", len(Xtr))
```

在這個切分上，線性 SVM 的測試準確率是 0.827，手選參數的 RBF-SVM 與 7-NN 都是 0.96。網格搜尋選出 $C = 1$、$\gamma = 1$（交叉驗證準確率 0.971），350 個訓練點中有 59 個是支持向量，測試準確率 0.953。調參後的模型並沒有任何實質意義上的「比較差」：150 個測試點中，一個點就等於 0.0067 的準確率，兩者只差一個點。單一切分上這種大小的差異應視為雜訊（PR 09）。標準化器放在 pipeline 裡，才會在每個訓練摺上重新擬合。

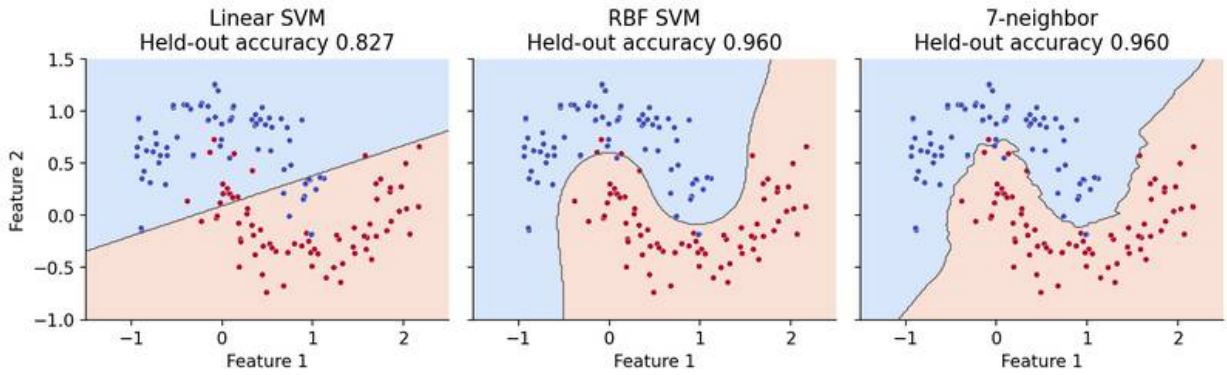


圖 3 — 與上面程式相同切分下的線性 *SVM*、*RBF-SVM* 與 *7-NN*。標準化器只以訓練資料擬合；圖中的點與分數都來自保留的測試資料。這是合成範例，不是一般性的基準測試。

分數不是機率

SVM 的輸出 $f(x)$ 是一個帶正負號、類似距離的分數，不是 $P(y=+1 \mid x)$；它的訓練目標從未要求它經過校準。若需要機率（例如拒絕選項，PR 01，或是要結合多個模型），應在保留資料上另外擬合一個由分數到機率的單調映射，並用可靠度圖檢查（PR 09）。多類別 SVM 通常由二元 SVM 組成，採用一對其餘（one-versus-rest）或一對一（one-versus-one）；scikit-learn 的 `SVC` 使用一對一。

高斯過程與相關向量

白話版。高斯過程使用同樣的核，但它不只回傳一個最佳函數，而是回傳一整個「合理函數」的分布，所以每個預測都附帶誤差棒。

高斯過程（Gaussian process, GP）是函數 f 上的先驗，使得任何有限組函數值 $(f(x_1),\dots,f(x_N))$ 都服從聯合高斯分布，平均（通常）為零、共變異數為 $K_{ij}=k(x_i,x_j)$ [8]。因此核必須是有效的共變異數，也就是半正定。對於觀測雜訊為高斯、變異數 σ^2 的迴歸，查詢點 x 的後驗是高斯分布，

$$\mu(x)=k_x^\top (K+\sigma^2 I)^{-1}t, \qquad s^2(x)=k(x,x)-k_x^\top (K+\sigma^2 I)^{-1}k_x,$$

其中 $\mu(x)$ 是預測平均，$s^2(x)$ 是潛在函數的變異數。平均值正好就是 $\lambda=\sigma^2$ 的核嶺迴歸。變異數是新的資訊：它在訓練輸入附近很小，遠離訓練輸入時回升到先驗變異數 $k(x,x)$。核的超參數（如 γ 與 σ^2）可以用最大化邊際概似（marginal likelihood）來選擇，而不必用交叉驗證 [2], [8]。

有兩點要注意。不確定度是以所選的核為條件；核選錯時，GP 可能很有自信地犯錯。另外，GP 分類比較困難：在高斯潛在函數上接伯努利概似，會讓後驗不再是高斯，因此需要 Laplace、期望傳播（expectation propagation）或變分近似（PR 08）。

相關向量機（relevance vector machine, RVM）由 Tipping 提出 [9]，並收錄於 Bishop 第 7.2 節 [2]。它使用和 SVM 相同的函數形式 $f(x)=\sum_n w_n k(x_n,x)+b$，但為每個權重 w_n 設定一個平均為零的高斯先驗，各自帶有自己的精度超參數。最大化邊際概似會把許多精度推向無窮大，等於把對應的基底函數關掉。存活下來的訓練點稱為「相關向量」，數量通常遠少於 SVM 的支持向

量；輸出具有機率意義；基底函數也不必滿足 Mercer 條件。代價是訓練問題不再是凸的。因此「稀疏」有好幾種意思：儲存的基底函數少（SVM、RVM）、非零的輸入權重少（lasso），或輸入資料本身稀疏。請說清楚你指的是哪一種。

徑向基函數網路

白話版。在輸入空間中幾個有代表性的位置放上高斯「小山丘」，量測新點讓每座山丘被激發的程度，再把這些激發值交給線性分類器。這就是圖 1 的抬升想法，只是改用一小組明確的特徵。

RBF 網路計算

$$f(x) = \sum_{j=1}^{M} w_j \, \exp\!\left(-\gamma \lVert x - \mu_j \rVert^2\right) + w_0,$$

其中 $\mu_1, \dots, \mu_M$ 是 M 個**中心**，γ 決定每座山丘的寬度，w_j 是輸出權重，w_0 是偏差。Moody 與 Darken 提出經典的兩階段訓練 [10]：先不使用標籤放置中心，例如用 k-means 分群（PR 07）；中心固定後，輸出權重就是一般的線性最小平方或邏輯斯迴歸問題。Park 與 Sandberg 證明 RBF 網路是萬能近似器：只要中心夠多，就能在緊緻集合上任意逼近任何連續函數 [11]。

和前面的核機器比較：核 SVM 等於在每個支持向量上放一座山丘，由間隔準則決定；RBF 網路則使用 $M \ll N$ 座山丘，位置來自分群（若所有參數一起訓練，則來自梯度下降），所以預測很便宜。它的弱點是非監督地放置中心時忽略了標籤，中心可能浪費在兩類根本不相接的地方。

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X, y = make_moons(n_samples=500, noise=0.18, random_state=2)
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=2)
scaler = StandardScaler().fit(Xtr)
Xtr, Xte = scaler.transform(Xtr), scaler.transform(Xte)

M, gamma, lam = 12, 1.0, 1e-3
centers = KMeans(M, n_init=10, random_state=0).fit(Xtr).cluster_centers_

def design(X):  # N x (M+1): Gaussian bumps plus a bias column
    d2 = ((X[:, None, :] - centers[None, :, :]) ** 2).sum(-1)
    return np.c_[np.exp(-gamma * d2), np.ones(len(X))]

Phi = design(Xtr)
t = 2.0 * ytr - 1.0                       # targets in {-1, +1}
w = np.linalg.solve(Phi.T @ Phi + lam * np.eye(M + 1), Phi.T @ t)
pred = (design(Xte) @ w > 0).astype(int)
print("RBF network test accuracy:", round((pred == yte).mean(), 4))
```

十二座山丘加上最小平方輸出層，在雙月形切分上得到 0.953 的測試準確率，和保留數十個支持向量的 RBF-SVM 相當。圖 5 左邊是同樣的網路改用邏輯斯輸出層的結果。

決策樹

白話版。決策樹和資料玩「二十個問題」。每個問題只看一個特徵，問「它是否不超過這個門檻？」每個答案把點送往左邊或右邊，直到抵達一片帶有標籤的葉節點。樹是貪婪地長出來的：每一步都挑出能讓分出來的兩群盡可能「純」的那個問題。

不純度準則

設某個節點中的訓練點類別比例為 $p_1, \dots, p_C$（節點中屬於各類別 c 的點所占比例）。**不純度**函數在某一類占據全部點時為零，在各類均勻混合時最大。三種標準選擇為

$$\text{Gini:}\quad G = 1 - \sum_{c=1}^{C} p_c^2 = \sum_{c} p_c (1 - p_c),$$

$$\text{Entropy:}\quad H = -\sum_{c=1}^{C} p_c \log_2 p_c,$$

$$\text{Misclassification:}\quad E = 1 - \max_c p_c.$$

Gini 指標是從節點中隨機（取後放回）抽兩個點、它們標籤不同的機率。熵（entropy）是標籤在資訊理論意義下的不確定度，單位是位元。誤分類率是節點以多數類別作為預測時的錯誤率。圖 4 左邊畫出兩類情況下的三者。

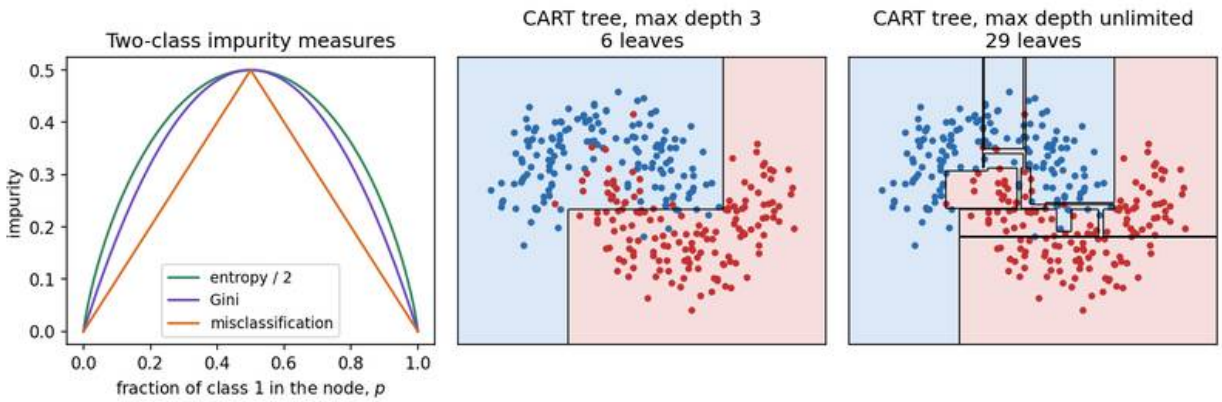


圖 4 — 左：兩類的不純度準則（熵乘以 ½ 以共用座標軸）。中：深度 3 的 *CART* 樹給出粗糙的軸對齊階梯。右：深度不限的樹在雜訊點周圍切出細小方塊，對訓練資料過擬合。

Gini 與熵都是嚴格凹函數，因此切分節點永遠不會讓它們的加權平均變大。誤分類率是分段線性的，當一次切分讓子節點更純、卻沒有改變任何子節點的多數類別時，它常常無法給予獎勵。這就是為什麼生長樹時用 Gini 或熵，評估時才用錯誤率。

長一棵樹：CART 演算法

Breiman、Friedman、Olshen 與 Stone 的 CART 程序 [12] 由上而下生長二元樹。在點集為 S 的節點，它考慮每個特徵 j 與相鄰觀測值之間的每個門檻 τ，把 S 分成 $S_L = \{x \in S : x_j \le \tau\}$ 與 $S_R = S \setminus S_L$，並選擇不純度下降量最大的切分：

$$\Delta i(j,\tau) = i(S) - \frac{|S_L|}{|S|}\, i(S_L) - \frac{|S_R|}{|S|}\, i(S_R),$$

其中 $i(\cdot)$ 是所選的不純度，$|S|$ 是集合中的點數。接著對兩個子節點遞迴，直到停止規則成立（節點已純、節點太小、達到最大深度）。Quinlan 的 ID3 [13] 是近親：它使用熵，把 Δi 稱為**資訊增益**（information gain），並對類別型特徵做多路切分。

下面的程式以暴力法找出最佳根節點切分，並和 scikit-learn 比對；scikit-learn 的 `DecisionTreeClassifier` 實作的是最佳化過的 CART。

```
import numpy as np
from sklearn.datasets import make_moons
from sklearn.tree import DecisionTreeClassifier

def gini(labels):
    p = np.bincount(labels, minlength=2) / len(labels)
    return 1.0 - (p ** 2).sum()

def best_split(X, y):
    best = (np.inf, None, None)
    for j in range(X.shape[1]):
        values = np.unique(X[:, j])
        for thr in (values[:-1] + values[1:]) / 2:
            left = X[:, j] <= thr
            n_l, n_r = left.sum(), (~left).sum()
            score = (n_l * gini(y[left]) + n_r * gini(y[~left])) / len(y)
            if score < best[0]:
                best = (score, j, thr)
    return best

X, y = make_moons(n_samples=300, noise=0.25, random_state=1)
score, j, thr = best_split(X, y)
root = DecisionTreeClassifier(max_depth=1).fit(X, y).tree_
print(f"ours: feature {j}, threshold {thr:.4f}, weighted Gini {score:.4f}")
print(f"sklearn: feature {root.feature[0]}, threshold {root.threshold[0]:.4f}")
assert j == root.feature[0] and np.isclose(thr, root.threshold[0])
```

兩者都選擇特徵 1、門檻 0.0547，把不純度從根節點的 0.5 降到加權後的 0.287。

剪枝與樹的性格

一棵長到每片葉子都純的樹會把訓練集背下來（圖 4 右）。CART 的解法是**成本複雜度剪枝**（cost-complexity pruning）[12]：先長一棵大樹 T_0，再對每個由合併內部節點得到的子樹 T 計算

$$R_\alpha(T) = R(T) + \alpha\,|\widetilde{T}|,$$

其中 $R(T)$ 是訓練誤分類率，$|\widetilde{T}|$ 是葉節點數，$\alpha \geq 0$ 是複雜度的價格。增加 α 會得到一串巢狀、越來越小的樹；再以交叉驗證選擇 α（scikit-learn 中的 `ccp_alpha`）。

樹有鮮明的性格。它不需要特徵標準化，能處理數值與類別混合的輸入，若某個無關特徵從未幫上切分就會被忽略，還能產生人看得懂的規則。它的邊界是軸對齊的階梯，所以斜的邊界要花很多次切分。它也不穩定：資料的小變動可能改變根節點的切分，進而改變整棵樹。把許多隨機化的樹平均起來（bagging、隨機森林），或依序加入（boosting），可以解決不穩定的問題，這是PR 09 的主題。

多層感知器作為非線性分類器

白話版。多層感知器（multilayer perceptron, MLP）就像把 RBF 網路的山丘換成學習得到的「斜坡」，而且斜坡本身是從標籤學來的。每個隱藏單元畫一條線；下一層把這些線組合成彎曲的區域。

對 C 個類別，單一隱藏層的 MLP 計算

$$h = \sigma(W_1 x + b_1), \qquad P(y = c \mid x) = \operatorname{softmax}(W_2 h + b_2)_c,$$

其中 W_1, b_1 把 D 個輸入映射到 H 個隱藏單元，σ 是逐元素的非線性函數（ReLU $\max(0,\cdot)$、logistic 或 tanh），W_2, b_2 把隱藏單元映射到各類分數。所有權重以梯度下降最小化交叉熵、共同訓練，梯度由反向傳播計算 [14]。和前面所有方法不同，特徵（h）與分類器都從標籤學來。Cybenko [15] 與 Hornik [16] 的萬能近似定理保證：只要單元夠多，單一隱藏層就能在緊緻集合上逼近任何連續函數；但定理沒有說需要多少單元，也沒有說梯度下降一定找得到它們。從神經元、激活函數到反向傳播與訓練技巧的完整內容在ML 02；影像分類的版本在IP 13。

從幾何上看，使用 ReLU 時每個隱藏單元以一個超平面切開輸入空間，網路輸出在切出的各個區塊上是分段線性的，所以圖 5 中 MLP 的邊界由直線段組成。

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = make_moons(n_samples=500, noise=0.18, random_state=2)
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=2)
for hidden in [(2,), (16,), (16, 16)]:
    mlp = make_pipeline(
        StandardScaler(),
        MLPClassifier(hidden_layer_sizes=hidden, alpha=1e-3,
                      max_iter=3000, random_state=0))
    mlp.fit(Xtr, ytr)
    print(hidden, "test accuracy:", round(mlp.score(Xte, yte), 4))
```

只有兩個隱藏單元時，網路只能把邊界彎一次，準確率 0.833。十六個單元達到 0.947，再加第二層 16 個單元在這個簡單的二維問題上沒有幫助。深度在影像這類高維輸入上才顯出價值，因為它讓網路能層層建構特徵（ML 03）。

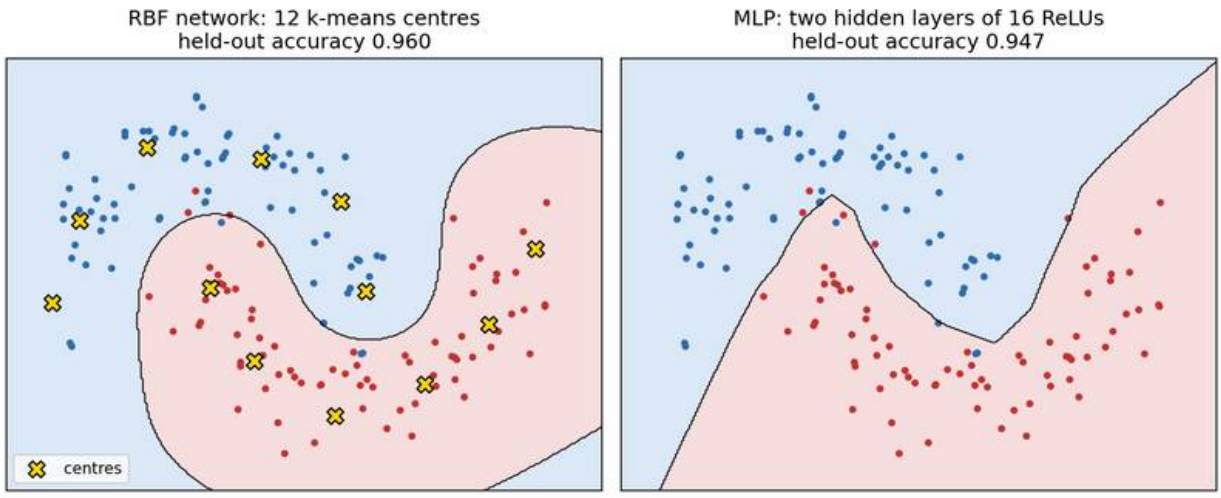


圖 5 — 兩種明確的特徵映射。左：*12* 個 *k-means* 中心加上邏輯斯輸出層的 *RBF* 網路，給出平滑、局部的曲線。右：使用 *ReLU* 單元的 *MLP*，給出分段線性的邊界。圖中為與程式相同切分的保留資料。

如何在非線性分類器之間選擇

方法	邊界形狀	訓練成本	預測成本	主要超參數	典型優點
k-NN	任意；順著資料	無（儲存資料）	每次查詢 $O(ND)$	k、度量、尺度	簡單的基準；搭配好的嵌入時效果好
核 SVM	平滑（RBF）	約 $O(N^2)$–$O(N^3)$	$O(N_{\text{sv}}D)$	C、核、γ	中小型資料、理論完整
高斯過程	平滑，附不確定度	精確解 $O(N^3)$	平均 $O(N)$、變異數 $O(N^2)$	核；以邊際概似設定	校準良好的不確定度（迴歸）
RBF 網路	平滑、局部	分群 + 線性求解	$O(MD)$	M、γ	預測便宜
決策樹	軸對齊方塊	每層 $O(DN \log N)$	$O(\text{depth})$	深度、葉大小、α	可解釋；可處理混合型特徵
MLP	分段線性（ReLU）	迭代式 SGD	$O(\text{weights})$	寬度、深度、權重衰減、學習率	可擴展到大型資料；自行學習特徵

其中 N_{sv} 是支持向量數，M 是 RBF 中心數。複雜度欄位是數量級，不是精確計數。沒有任何一列在所有情況下都勝出。請在同一套驗證流程下比較候選方法，並記住網格搜尋範例的教訓：單一切分上不到一個百分點的差異，很少是真的。

現代觀點

從 1990 年代末到 2012 年左右深度網路接手大規模感知任務之前，核方法主導了圖形識別。它們並沒有消失，而是換了角色。以下六條線索勾勒出目前的樣貌。

經典領域的綜述。 Hofmann、Schölkopf 與 Smola 發表於 *Annals of Statistics* 的綜述 [7] 是核方法的標準地圖：正定核與其 RKHS、表示定理、為字串與圖等結構化資料建構核，以及大間隔背後的統計學習理論。Cervantes 等人 [19] 從應用面綜述 SVM 分類：在基本 SVM 表現不佳之處發展出的變體（非常大的資料集、多類別、類別不平衡）、與演化式參數搜尋的結合、大量應用案例，以及方法本身的限制。關於決策樹，Costa 與 Pedreira [20] 圍繞任何預測學習器的三個目標來回顧近年的研究：擬合訓練資料、推廣到訓練資料之外，以及保持可解釋性。

讓核方法擴展。 Gram 矩陣 $O(N^2)$ 的記憶體需求曾是主要障礙。Rahimi 與 Recht 的隨機特徵 [17] 以一個明確的隨機映射 $z(x) \in \mathbb{R}^R$ 近似 RBF 這類平移不變核，使 $z(x)^\top z(x') \approx k(x, x')$，於是在 R 個隨機特徵上的線性模型就能代替核機器。Liu 等人的綜述 [18] 整理了十年來的後續研究：隨機特徵的取樣方式、達到給定精度需要多少特徵的理論，以及與深度網路的關聯。把這類近似與 GPU 線性代數結合，FALKON 系列研究能在數十億個點上訓練核嶺迴歸 [23]；GPyTorch [24] 則以迭代式的矩陣–矩陣求解，讓精確與近似的高斯過程推論在 GPU 上變得實用。

以核的角度看深度學習。 Jacot、Gabriel 與 Hongler 證明：在適當尺度下的無限寬度極限中，以梯度下降訓練神經網路等價於使用固定**神經正切核**（neural tangent kernel）的核迴歸 [21]。Belkin、Ma 與 Mandal 指出，深度學習中令人困惑的現象，例如完全擬合訓練資料卻仍能推廣，在核機器中同樣會出現，因此理解核是理解深度網路的前提 [22]。真實的有限寬度網路確實會學習特徵，這是固定的核做不到的，所以這個類比是理論工具，而不是替代品。

樹仍然稱霸表格資料。對於表格資料（由異質量測構成的資料列，而非像素或文字），Grinsztajn、Oyallon 與 Varoquaux 在 45 個中型資料集上比較樹集成與深度架構，發現樹模型仍然領先，並把差距歸因於神經網路對無資訊特徵較敏感、具有旋轉不變性，以及偏好平滑函數 [25]。最新的挑戰者是**表格基礎模型**：TabPFN 是一個在數百萬個合成資料集上訓練的 transformer，只要一次前向傳播就能對新的小型資料集做預測。作者報告它在樣本數至多 10,000 的資料集上勝過先前的方法，包括經過調參的梯度提升樹集成，而且訓練時間少得多 [26]。

在學習到的特徵上找鄰居。最近鄰規則作為大型預訓練嵌入之上的分類器，重新受到重視。一旦特徵夠好，k-NN 或 cosine 相似度分類器往往是最簡單又強的基準，而且不必重新訓練就能加入新類別。IP 13 與ML 05 討論了這種做法。

未解問題。精確的核方法與 GP 在 N 上的擴展性仍然不佳，近似品質與速度之間的取捨仍是活躍的研究方向 [18]。為結構化資料選擇核大多仍靠人工。單棵樹依然不穩定，而全域最佳的樹計算代價很高 [20]。至於表格資料，基礎模型式的方法能否在小型資料集以外取代調好參數的樹集成，到 2026 年仍是開放問題。

重點整理

- 非線性邊界來自三個想法：和儲存樣本比較（k-NN）、抬升到線性規則可行的特徵空間（核、RBF 網路、MLP），或切割空間（樹）。
- k-NN 的行為由度量與尺度決定的程度不亞於 k；對單位向量而言，歐氏與 cosine 鄰居相同。

- 一個核有效，若且唯若每個 Gram 矩陣都對稱半正定（Mercer 條件）。核技巧在任何以內積寫成的演算法中，把內積換成核值。
- 軟間隔 SVM 最小化 $\frac{1}{2}\|w\|^2$ 加上 C 倍的鉸鏈損失。它的對偶問題只透過核依賴資料，只有支持向量（$\alpha_n > 0$）影響預測。在標準化的特徵上，以交叉驗證同時調整 C 與 γ。
- GP 給出附帶不確定度的核預測；RVM（Tipping）給出稀疏的機率式核模型。SVM 分數不是機率。
- RBF 網路使用少數明確的高斯山丘；樹依不純度下降（Gini 或熵）貪婪生長，需要剪枝；MLP 自行學習特徵，詳見 ML 02。
- 沒有哪個家族處處勝出：中小型平滑問題用核，表格資料用樹及其集成，原始感知資料用深度網路，好的嵌入之上用近鄰。

練習

1. **距離與 cosine**。推導單位向量的 $\|u - v\|^2 = 2 - 2u^\top v$，並證明歐氏距離下的 1-NN 等於 cosine 相似度下的 1-NN。若向量沒有正規化，結論還成立嗎？

   提示

   展開 $\|u - v\|^2 = u^\top u - 2u^\top v + v^\top v$。單位長度時，距離是 $u^\top v$ 的遞減函數，所以一者的最小值就是另一者的最大值。沒有正規化時長度會介入：一個方向稍有不同的長向量，在歐氏距離上可能比方向完全一致的短向量更近。

2. **建構核**。寫出 $\mathbb{R}^2$ 中 $k(x, z) = (x^\top z)^2$ 的特徵映射，藉此證明它是有效的核。再用組合規則論證高斯核 $\exp(-\gamma\|x - z\|^2)$ 是有效的。

   提示

   $\phi(x) = (x_1^2, x_2^2, \sqrt{2}x_1x_2)$。對高斯核，寫成 $\exp(-\gamma\|x - z\|^2) = \exp(-\gamma\|x\|^2)\,\exp(2\gamma x^\top z)\,\exp(-\gamma\|z\|^2)$。中間的因子是有效（縮放過的線性）核取 exp，外側兩個因子構成 $f(x)\,k(x,z)\,f(z)$ 的形式。

3. **γ 的極限**。對固定的有限資料集，當 $\gamma \to 0$ 與 $\gamma \to \infty$ 時，RBF Gram 矩陣分別趨近什麼？把兩個極限分別對應到圖 2 最左與最右的欄。

   提示

   $\gamma \to 0$ 時每個元素都趨近 1（全為 1 的矩陣）：所有點看起來都一樣，邊界變得近乎線性或常數。$\gamma \to \infty$ 時非對角元素趨近 0，$K \to I$：每個點只和自己相似，SVM 可以背下每個訓練標籤，也就是「小島」的情形。

4. **支持向量與成本**。一個在 100,000 張影像上訓練的 RBF-SVM，保留了 90% 的影像作為支持向量。這代表資料有什麼特性？預測時間會如何？請提出兩個可以嘗試的做法。

   提示

   許多點落在間隔內或被分錯：類別在這個特徵空間中嚴重重疊，或是 C 太小、γ 選得不好。每次預測需要約 90,000 次核計算。可以重新調整 C 與 γ、改善特徵、改用隨機特徵加線性 SVM，或換一個模型家族。

5. **手算不純度**。某節點有 40 個 A 類點與 40 個 B 類點。切分 1 得到子節點 (30 A, 10 B) 與 (10 A, 30 B)；切分 2 得到 (40 A, 20 B) 與 (0 A, 20 B)。計算兩種切分的誤分類率下降量與 Gini 下降量。兩種準則各偏好哪一個切分？

   提示

   根節點 Gini $= 0.5$，錯誤率 $= 0.5$。切分 1：兩個子節點的錯誤率都是 0.25，Gini 都是 $2 \cdot 0.75 \cdot 0.25 = 0.375$；加權錯誤率 0.25，加權 Gini 0.375。切分 2：左子節點錯誤率 1/3，Gini $2 \cdot \frac{2}{3} \cdot \frac{1}{3} = 4/9$；右子節點是純的。加權錯誤率 $\frac{60}{80} \cdot \frac{1}{3} = 0.25$，加權 Gini $\frac{60}{80} \cdot \frac{4}{9} = 1/3$。錯誤率無法區分兩者（都是 0.25）；Gini 偏好切分 2，因為它產生了純節點。

6. **設計 RBF 網路**。在 RBF 網路的程式中，改變 $M \in \{2, 4, 12, 50\}$ 與 $\gamma \in \{0.1, 1, 10\}$。哪些組合欠擬合、哪些過擬合？為什麼 M 小而 γ 大的組合表現很差？

   提示

   中心少而 γ 大時，山丘很窄，大多數點幾乎不激發任何一座，設計矩陣在遠離中心處近乎為零，分類器只能退回偏差項。中心多且 γ 大時，行為開始像正則化不足的核機器。

參考文獻

1. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2009, Chapters 2 and 4. publisher page
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006, Sections 2.5.2, 5.1–5.3, 6.1–6.4, 7.1–7.2 and 14.4. official book page and PDF
3. T. Cover and P. Hart, “Nearest Neighbor Pattern Classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967. doi:10.1109/TIT.1967.1053964
4. J. Mercer, “Functions of Positive and Negative Type, and Their Connection with the Theory of Integral Equations,” *Philosophical Transactions of the Royal Society of London, Series A*, vol. 209, pp. 415–446, 1909. doi:10.1098/rsta.1909.0016
5. B. E. Boser, I. M. Guyon, and V. N. Vapnik, “A Training Algorithm for Optimal Margin Classifiers,” *Proceedings of the Fifth Annual Workshop on Computational Learning Theory (COLT)*, pp. 144–152, 1992. doi:10.1145/130385.130401
6. C. Cortes and V. Vapnik, “Support-Vector Networks,” *Machine Learning*, vol. 20, pp. 273–297, 1995. doi:10.1007/BF00994018
7. T. Hofmann, B. Schölkopf, and A. J. Smola, “Kernel Methods in Machine Learning,” *The Annals of Statistics*, vol. 36, no. 3, 2008. doi:10.1214/009053607000000677
8. C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*, MIT Press, 2006. book page
9. M. E. Tipping, “Sparse Bayesian Learning and the Relevance Vector Machine,” *Journal of Machine Learning Research*, vol. 1, pp. 211–244, 2001. JMLR
10. J. Moody and C. J. Darken, “Fast Learning in Networks of Locally-Tuned Processing Units,” *Neural Computation*, vol. 1, no. 2, pp. 281–294, 1989. doi:10.1162/neco.1989.1.2.281

11. J. Park and I. W. Sandberg, “Universal Approximation Using Radial-Basis-Function Networks,” *Neural Computation*, vol. 3, no. 2, pp. 246–257, 1991. doi:10.1162/neco.1991.3.2.246
12. L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*, Chapman and Hall/CRC, 2017 edition. doi:10.1201/9781315139470
13. J. R. Quinlan, “Induction of Decision Trees,” *Machine Learning*, vol. 1, pp. 81–106, 1986. doi:10.1007/BF00116251
14. D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning Representations by Back-Propagating Errors,” *Nature*, vol. 323, pp. 533–536, 1986. doi:10.1038/323533a0
15. G. Cybenko, “Approximation by Superpositions of a Sigmoidal Function,” *Mathematics of Control, Signals, and Systems*, vol. 2, pp. 303–314, 1989. doi:10.1007/BF02551274
16. K. Hornik, “Approximation Capabilities of Multilayer Feedforward Networks,” *Neural Networks*, vol. 4, no. 2, pp. 251–257, 1991. doi:10.1016/0893-6080(91)90009-T
17. A. Rahimi and B. Recht, “Random Features for Large-Scale Kernel Machines,” *Advances in Neural Information Processing Systems 20 (NIPS)*, 2007. paper
18. F. Liu, X. Huang, Y. Chen, and J. A. K. Suykens, “Random Features for Kernel Approximation: A Survey on Algorithms, Theory, and Beyond,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 10, pp. 7128–7148, 2022. doi:10.1109/TPAMI.2021.3097011
19. J. Cervantes, F. Garcia-Lamont, L. Rodríguez-Mazahua, and A. Lopez, “A Comprehensive Survey on Support Vector Machine Classification: Applications, Challenges and Trends,” *Neurocomputing*, vol. 408, pp. 189–215, 2020. doi:10.1016/j.neucom.2019.10.118
20. V. G. Costa and C. E. Pedreira, “Recent Advances in Decision Trees: An Updated Survey,” *Artificial Intelligence Review*, vol. 56, pp. 4765–4800, 2023. doi:10.1007/s10462-022-10275-5
21. A. Jacot, F. Gabriel, and C. Hongler, “Neural Tangent Kernel: Convergence and Generalization in Neural Networks,” *NeurIPS*, 2018. arXiv:1806.07572
22. M. Belkin, S. Ma, and S. Mandal, “To Understand Deep Learning We Need to Understand Kernel Learning,” arXiv:1802.01396, 2018. arXiv
23. G. Meanti, L. Carratino, L. Rosasco, and A. Rudi, “Kernel Methods Through the Roof: Handling Billions of Points Efficiently,” *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020. paper
24. J. R. Gardner, G. Pleiss, D. Bindel, K. Q. Weinberger, and A. G. Wilson, “GPyTorch: Blackbox Matrix-Matrix Gaussian Process Inference with GPU Acceleration,” *NeurIPS*, 2018. arXiv:1809.11165
25. L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?,” *Advances in Neural Information Processing Systems 35 (NeurIPS), Datasets and Benchmarks Track*, 2022. paper
26. N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, et al., “Accurate Predictions on Small Data with a Tabular Foundation Model,” *Nature*, vol. 637, pp. 319–326, 2025. doi:10.1038/s41586-024-08328-6

PR 05・特徵選擇、轉換與降維

選擇與建構特徵：過濾式、包裝式與嵌入式選擇，類別可分性準則，PCA、Fisher LDA、核 PCA、ICA 與流形學習（Isomap、LLE、t-SNE、UMAP），以及如何在不洩漏資訊的前提下驗證特徵流程。

網站版本: https://blog.shyandram.dev/zh-tw/learn/pattern-recognition/pr-05-features-dimensionality/

你將學到

- 為什麼較少但較好的特徵常常勝過更多特徵，以及特徵**選擇**、特徵**萃取**與特徵**學習**的差別。
- 過濾式、包裝式與嵌入式選擇，以及最常見的錯誤：在交叉驗證之外選特徵。
- 如何為特徵與特徵子集評分：Fisher 比值、散佈矩陣準則、Bhattacharyya 距離與互資訊。
- 線性萃取：PCA（含白化）與 Fisher 線性判別分析的推導與比較。
- 非線性與非高斯的萃取：核 PCA、ICA（與 NMF），以及以 Isomap、LLE、t-SNE、UMAP 進行的流形學習，包括它們的圖能證明什麼、不能證明什麼。
- 經典的人工設計影像與訊號描述子，以及這一切在 2026 年的位置。

先看全貌

兩個辨識系統可以使用完全相同的分類器，行為卻大不相同，因為它們描述輸入的方式不同。紋理直方圖丟掉了「東西在哪裡」；形狀描述子可能忽略顏色；從網路照片學來的嵌入（embedding）可能悄悄編碼了背景。每一種表徵都保留部分變異、捨棄其餘部分，而分類器只能使用被保留的部分。

> **用最簡單的話說**
>
> 你要向朋友描述一個玩具，讓他能從玩具箱裡挑出來。「紅色、圓的、跟柳丁一樣大」很好用；「它是原子組成的」雖然沒錯卻沒用；把你看到的 500 件事全講一遍又記不住。好的描述很簡短，而且剛好保留了讓這個玩具和其他玩具區分開來的線索。

本章談的就是如何建立這種簡短而有用的描述。有三種做法：

- **選擇**保留原始量測的一個子集，例如從 5,000 個基因表現量中保留 20 個。保留的特徵仍然可以解釋。
- **萃取**以所有量測的函數計算出新座標，例如投影到前幾個主成分。
- **學習**從資料估計萃取規則，常用深度網路（ML 05）。PCA 本身就已是（非常簡單的）學習得到的萃取規則；這幾類是互相重疊的。

如果彈性很大的分類器可以使用全部特徵，為什麼還要費事？有三個理由。在訓練樣本數固定時，加入資訊量很弱的特徵可能讓訓練出的分類器變差，因為每多一維就多了要估計的參數與過

擬合的空間：這就是 Theodoridis 與 Koutroumbas 所討論的「維度詛咒」與峰值現象（peaking phenomenon）[1]。特徵越少，量測、儲存與計算越便宜。而且一組小而有意義的特徵，能幫助人理解分類器依賴的是什麼。Guyon 與 Elisseeff 在特徵選擇的經典導論中列出的也是這三個動機 [3]。本章的教科書背景是 Theodoridis 與 Koutroumbas 第 5–7 章 [1]，以及 Bishop 第 4.1.4–4.1.6 與 12.1–12.4 節 [2]。人工設計的影像特徵另請參考IP 12。

特徵選擇

白話版。特徵選擇問的是：在已有的量測中，我該保留哪些？你需要一種為特徵（或特徵組合）評分的方法，以及一種搜尋方法，因為不可能把每個子集都試一遍。

D 個特徵共有 $2^D - 1$ 個非空子集：$D = 100$ 時約為 10^{30} 個。除了極小的 D，窮舉搜尋都不可行。各種方法的差別在於如何評分與如何搜尋 [3], [22], [23]。

過濾式方法

過濾式（filter）方法直接由資料計算某個準則，為每個特徵（或子集）評分，與最終的分類器無關，再保留最好的。典型的準則包括變異數（剔除近乎常數的特徵）、類別分離統計量（例如 ANOVA F 統計量或下一節的 Fisher 比值）、與標籤的相關係數，以及互資訊（mutual information）

$$I(x_j; y) = \sum_{v} \sum_{c} p(x_j = v, y = c) \, \log \frac{p(x_j = v, y = c)}{p(x_j = v)\, P(y = c)},$$

其中 x_j 是第 j 個特徵（離散化為數值 v），y 是取值為 c 的類別標籤，p、P 是由訓練集估計的聯合與邊際機率。$I(x_j; y) = 0$ 若且唯若該特徵與標籤在統計上獨立。

過濾式方法快速，且與分類器無關，但單變量過濾有兩個盲點。**冗餘**：同一個好特徵的兩份複本都會得高分，但第二份毫無貢獻。Peng、Long 與 Ding 的 mRMR 準則處理這個問題：獎勵與標籤的相關性，同時懲罰與已選特徵之間的互資訊 [6]。**交互作用**：兩個特徵可能各自毫無用處、合在一起卻具決定性。在互斥或（XOR）型態 $y = \mathbf{1}[x_1 > 0] \oplus \mathbf{1}[x_2 > 0]$ 中，每個特徵單獨看都和 y 獨立，所以任何單變量過濾都會把兩者都丟掉。

包裝式方法

包裝式（wrapper）方法以實際的分類器在候選子集上訓練，並以驗證表現作為該子集的分數 [4]。由於窮舉不可行，包裝式方法使用啟發式搜尋，例如：

- **循序前向選擇**：從空集合開始，每次加入最能提升驗證分數的特徵；
- **循序後向刪除**：從全部特徵開始，每次移除最沒用的一個；
- 允許撤回先前步驟的浮動（floating）變體 [1]。

包裝式方法透過分類器的眼睛看見交互作用與冗餘，但成本高（每個候選都要完整訓練一次），而且因為它在同一份驗證資料上嘗試許多子集，可能會對驗證集本身過擬合。提出這個名詞的 Kohavi 與 John 論文，至今仍是討論這些取捨的標準參考 [4]。

嵌入式方法

嵌入式（embedded）方法在訓練過程中順帶完成特徵選擇。最有名的例子是 lasso，它在線性模型上加入 ℓ_1 懲罰 [5]：

$$\hat{w} = \arg\min_{w} \frac{1}{2N} \sum_{n=1}^{N} \left(t_n - w^\top x_n\right)^2 + \lambda \|w\|_1, \qquad \|w\|_1 = \sum_{j=1}^{D} |w_j|,$$

其中 t_n 為目標值，x_n 為輸入，$\lambda \geq 0$ 為懲罰強度。ℓ_1 球的尖角會讓 $\hat{w}$ 的許多座標恰好為零，存活下來的座標就是被選中的特徵。同樣的想法也適用於分類用的邏輯斯迴歸。決策樹及其集成（PR 04、PR 09）也內建選擇：從未幫上任何切分的特徵就不會被使用。

選擇偏差的陷阱

監督式選擇會用到標籤。如果你先在完整資料集上選特徵，再只對分類器做交叉驗證，那麼保留的測試摺早已影響了選出哪些特徵，誤差估計就會過於樂觀。Ambroise 與 McLachlan 指出，這個錯誤讓已發表的基因表現分類器看起來近乎完美；把選擇放進每個交叉驗證摺內執行後，誤差就不再是零 [7]。下面的程式用純雜訊特徵把這個效應具體呈現出來。

```
import numpy as np
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline

rng = np.random.default_rng(0)
X = rng.normal(size=(60, 5000))          # 60 samples, 5000 pure-noise features
y = rng.integers(0, 2, size=60)          # labels unrelated to X

# Wrong: select on all data, then cross-validate the classifier only.
X_sel = SelectKBest(f_classif, k=20).fit_transform(X, y)
wrong = cross_val_score(LogisticRegression(max_iter=1000), X_sel, y, cv=5)

# Right: selection is part of the model and is refitted in every fold.
model = make_pipeline(SelectKBest(f_classif, k=20),
                      LogisticRegression(max_iter=1000))
right = cross_val_score(model, X, y, cv=5)
print(f"selection outside CV: {wrong.mean():.2f}")
print(f"selection inside CV:  {right.mean():.2f}")
```

標籤是隨機的，所以沒有任何方法真正能超過 50% 的準確率。在交叉驗證之外做選擇，回報的是 0.98；把選擇器放進 pipeline，回報 0.58，扣掉 60 個樣本估計的雜訊後就是隨機水準。這個規則可以推廣：每一個會看標籤的步驟（特徵選擇、LDA 這類監督式投影、決定保留幾個特徵）都屬於訓練程序的一部分。即使是 PCA 與標準化這類非監督步驟，也應該只在訓練摺上擬合，讓你評估的程序就是你要部署的程序。

評估特徵與特徵子集

白話版。好的特徵會讓各類別彼此離得遠，相對於每個類別本身的分散程度而言。有好幾個公式把這個想法變成一個數字，從單一特徵的簡單比值，到整個子集的準則。

單一特徵的 **Fisher 判別比值**比較類別平均之間的分散與類別之內的分散：

$$\mathrm{FDR}_j = \frac{\sum_c P_c\,(\mu_{cj} - \mu_j)^2}{\sum_c P_c\,\sigma_{cj}^2},$$

其中 P_c 是類別 c 的比例，μ_{cj} 與 σ_{cj}^2 是特徵 j 在類別 c 中的平均與變異數，μ_j 是整體平均 [1]。（兩類時，Theodoridis 與 Koutroumbas 寫成 $(\mu_1-\mu_2)^2/(\sigma_1^2+\sigma_2^2)$，概念相同。）

對特徵的**子集**，則使用散佈矩陣。令 x 為限制在該子集上的特徵向量，μ_c、μ 分別為類別平均與整體平均向量，

$$S_W = \sum_c P_c\,\Sigma_c, \qquad S_B = \sum_c P_c\,(\mu_c-\mu)(\mu_c-\mu)^{\top}, \qquad J = \operatorname{tr}(S_W^{-1} S_B),$$

其中 Σ_c 是類別 c 的共變異數矩陣，S_W 是類內散佈，S_B 是類間散佈，tr 是跡。當類別平均相對於類內分散相距很遠（在真正重要的方向上量測）時，J 就大 [1]。和把各特徵比值相加不同，J 考慮了特徵之間的相關。

另外兩個準則來自機率。兩個高斯類別 $\mathcal{N}(\mu_1,\Sigma_1)$ 與 $\mathcal{N}(\mu_2,\Sigma_2)$ 之間的 **Bhattacharyya 距離**為

$$B = \frac{1}{8}(\mu_1-\mu_2)^{\top}\Big(\frac{\Sigma_1+\Sigma_2}{2}\Big)^{-1}(\mu_1-\mu_2) + \frac{1}{2}\ln\frac{\big|\tfrac{1}{2}(\Sigma_1+\Sigma_2)\big|}{\sqrt{|\Sigma_1|\,|\Sigma_2|}},$$

$|\cdot|$ 為行列式。第一項衡量平均的分離，第二項衡量共變異數的差異。在先驗相等時，它給出貝氏錯誤率的上界 $P_e \le \tfrac{1}{2}e^{-B}$ [1]。前面介紹的、與標籤之間的**互資訊**，則是不依賴分布假設的對應版本。

下面的程式在 13 個特徵的葡萄酒（wine）資料上，比較單一特徵分數與子集準則。

```
import numpy as np
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler

X, y = load_wine(return_X_y=True)
X = StandardScaler().fit_transform(X)
classes = np.unique(y)
mu = X.mean(axis=0)

# Per-feature Fisher ratio: between-class over within-class variance.
between = sum((y == c).mean() * (X[y == c].mean(0) - mu) ** 2 for c in classes)
within = sum((y == c).mean() * X[y == c].var(0) for c in classes)
fdr = between / within
print("top-3 features by Fisher ratio:", np.argsort(fdr)[::-1][:3],
      np.round(np.sort(fdr)[::-1][:3], 2))

def J(cols):  # scatter-matrix criterion trace(Sw^-1 Sb) on a feature subset
```

```
    Z = X[:, cols]
    m = Z.mean(0)
    Sw = sum((y == c).mean() * np.cov(Z[y == c].T, bias=True).reshape(len(cols), -1)
             for c in classes)
    Sb = sum((y == c).mean() * np.outer(Z[y == c].mean(0) - m, Z[y == c].mean(0) - m)
             for c in classes)
    return np.trace(np.linalg.solve(Sw, Sb))

top2 = list(np.argsort(fdr)[::-1][:2])
print("J(top-2 by ratio) =", round(J(top2), 2))
best_pair = max(((i, j) for i in range(13) for j in range(i + 1, 13)),
                key=lambda p: J(list(p)))
print("best pair by J:", best_pair, "J =", round(J(list(best_pair)), 2))
```

特徵 6、12、11 的個別比值最高（2.67、2.38、2.17）。然而依 J 選出的最佳配對是特徵 6 與 9（$J = 5.39$），而不是個別最好的前兩名（$J = 4.93$）：特徵 12 的資訊和特徵 6 重疊，特徵 9 則與它互補。這就是單變量過濾的冗餘問題，以數字呈現。

這些準則都只是代理指標。特徵集合的最終裁判，是整條流程在不洩漏資訊的情況下估計出的交叉驗證表現（PR 09）。

主成分分析

白話版。 PCA 旋轉座標系，讓第一個軸指向資料變化最大的方向，第二個軸指向剩餘變化最大的方向，依此類推。只保留前幾個軸，就得到最小平方意義下最好的低維摘要。它從不看標籤。

推導

設 $x_1, \dots, x_N \in \mathbb{R}^D$ 的平均為 $\bar{x}$，共變異數為 $\Sigma = \frac{1}{N-1}\sum_n (x_n - \bar{x})(x_n - \bar{x})^\top$。資料投影到單位向量 u 上的變異數為 $u^\top \Sigma u$。在 $u^\top u = 1$ 的約束下以拉格朗日乘數 λ 最大化它，得到

$$\Sigma u = \lambda u,$$

所以最佳方向是 Σ 的特徵向量，它捕捉到的變異數就是對應的特徵值 λ。第一主方向是特徵值最大的特徵向量；之後的主方向是其餘特徵向量，依特徵值遞減排列，且彼此正交 [2], [8], [9]。保留前 r 個方向，同時也使平均平方重建誤差最小，誤差等於被捨棄的特徵值總和 $\sum_{i>r} \lambda_i$ [2]。

實務上，PCA 以中心化資料矩陣 $X_c \in \mathbb{R}^{N \times D}$（第 n 列為 $x_n - \bar{x}$）的奇異值分解（SVD）來計算：

$$X_c = U S V^\top, \qquad \lambda_i = \frac{s_i^2}{N-1}, \qquad Z = X_c V_r,$$

其中 U 與 V 的行向量正交歸一，$S = \mathrm{diag}(s_1, s_2, \dots)$ 依遞減順序存放奇異值，V_r 是 V 的前 r 行（主方向），$Z \in \mathbb{R}^{N \times r}$ 是**分數**（scores）。比值 $\lambda_i / \sum_k \lambda_k$ 是第 i 個成分的**解釋變異比例**。重建時計算 $\hat{x} = \bar{x} + V_r z$。

白化

白化（whitening）把每個分數縮放為單位變異數：

$$z_i^{\text{white}} = \frac{z_i}{\sqrt{\lambda_i + \epsilon}},$$

其中 z_i 是第 i 個成分上的分數，λ_i 是由訓練資料估計的該成分變異數，$\epsilon \geq 0$ 是避免極小特徵值造成爆量的小常數。白化後訓練資料的共變異數（近似）為單位矩陣。這讓所有方向一樣「大聲」，有助於某些演算法（下文的 ICA、k-means、某些神經網路），但也會放大那些原本因為主要是雜訊而很小的方向。白化是一種建模選擇，不是免費的改進。

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA

X, _ = load_digits(return_X_y=True)          # 1797 images of 8x8 = 64 pixels
Xc = X - X.mean(axis=0)
U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
eigvals = S ** 2 / (len(X) - 1)               # variances along each direction
ratio = eigvals / eigvals.sum()
r = int(np.searchsorted(np.cumsum(ratio), 0.90) + 1)
print("components for 90% variance:", r)

Z = Xc @ Vt[:r].T                             # scores, N x r
X_hat = Z @ Vt[:r] + X.mean(axis=0)           # reconstruction
print("relative reconstruction error:",
      round(np.linalg.norm(X - X_hat) / np.linalg.norm(Xc), 3))

pca = PCA(n_components=r).fit(X)
assert np.allclose(pca.explained_variance_, eigvals[:r])
assert np.allclose(np.abs(pca.components_), np.abs(Vt[:r]))   # equal up to sign
Zw = Z / np.sqrt(eigvals[:r])                 # whitened scores
print("whitened covariance is identity:",
      np.allclose(np.cov(Zw.T), np.eye(r)))
```

64 個像素維度中，有 21 維就包含了數字影像 90% 的變異，以它們重建的相對誤差為 0.31。手寫的 SVD 與 scikit-learn 的結果除了每個成分的正負號外完全一致（特徵向量的正負號是任意的）。圖 1 顯示累積曲線與重建結果。

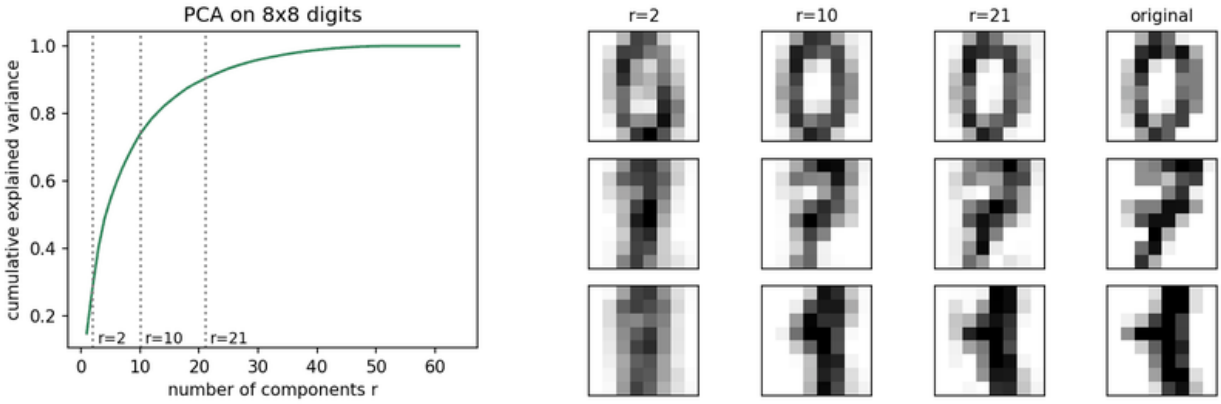


圖 *1* — *scikit-learn digits* 資料上的 *PCA*。兩個成分只得到模糊的團塊；十個成分已看得出數字；*21* 個成分（*90%* 的變異）已接近原圖。

變異不等於類別資訊

PCA 依變異數為方向排序，而變異數和對分類的用處並不相同。在圖 2 中，一個很大的干擾變異（想像成光照）主宰了第一主成分，而類別差異位在一個變異很小的方向。投影到 PC 1 保留了 96% 的變異，卻幾乎丟掉了全部的類別資訊。監督式投影（下一節的 Fisher LDA）則立刻找到有用的方向。

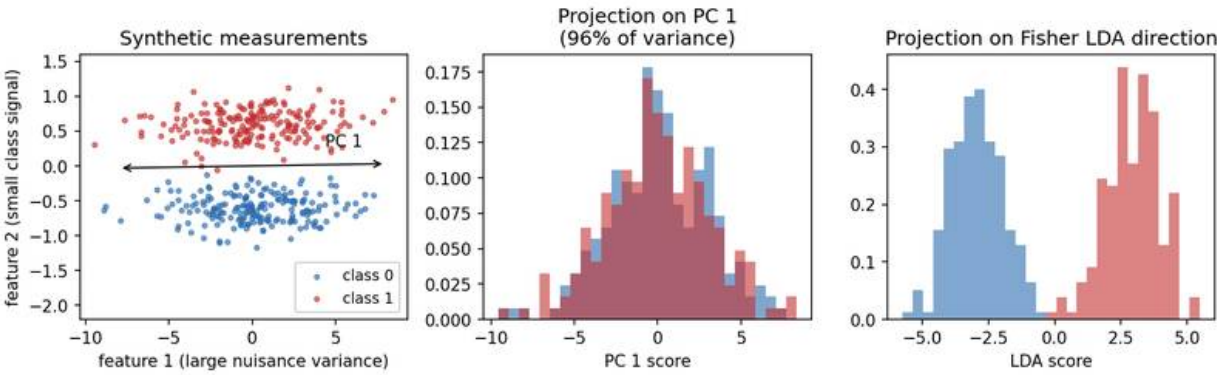


圖 *2* — 變異不等於類別資訊。*PC 1*（*96%* 的變異）沿著干擾軸，把兩類混在一起；*Fisher LDA* 方向則把它們分開。合成資料；由 *scripts/figures/pr_05_features_dimensionality_extra.py* 產生。

Jolliffe 與 Cadima 的綜述 [10] 說明了 PCA 能做與不能做的事，以及它的許多變體（例如稀疏 PCA 與穩健 PCA）。把 PCA 變成潛在變數模型的機率 PCA 與因素分析，在 Bishop 第 12 章 [2] 與ML 05A 中有完整推導。

Fisher 線性判別分析

白話版。 LDA 尋找這樣的方向：在這些方向上，類別平均相距很遠，相對於各類別本身的分散程度。它使用標籤，所以找到的方向對分類有意義；但它最多只能產生比類別數少一個的方向。

投影方向 v 的 Fisher 準則是沿 v 的類間散佈與類內散佈之比 [11]：

$$J(v) = \frac{v^\top S_B v}{v^\top S_W v},$$

S_W 與 S_B 的定義同上一節。令梯度為零得到**廣義特徵值問題**

$$S_B v = \lambda S_W v,$$

最佳方向是特徵值 λ 最大的特徵向量（當 S_W 可逆時，等同於 $S_W^{-1} S_B$ 的特徵向量）[1], [2]。兩類時有封閉解 $v \propto S_W^{-1}(\mu_1 - \mu_2)$，它在PR 03 中以高斯 LDA 分類器的形式再次出現。

為什麼最多 $K-1$ 個方向。 S_B 是 K 個秩為一的矩陣 $(\mu_c - \mu)(\mu_c - \mu)^\top$ 的加權和。向量 $\mu_c - \mu$ 線性相依，因為 $\sum_c P_c(\mu_c - \mu) = 0$。所以 S_B 的秩至多為 $K-1$，非零特徵值至多 $K-1$ 個。不論輸入維度多高，十個數字類別最多給出九個判別方向。

當 S_W 奇異時。樣本數少於特徵數時（影像中很常見），S_W 無法求逆。標準做法是先用 PCA、再在降維後的空間做 LDA，或把 S_W 往單位矩陣的倍數收縮（正則化 LDA）。

```
from sklearn.datasets import load_digits
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

X, y = load_digits(return_X_y=True)
lda = LDA().fit(X, y)
print("discriminant directions for 10 classes:", lda.transform(X).shape[1])
for name, reducer in [("PCA", PCA(n_components=2)),
                      ("LDA", LDA(n_components=2))]:
    pipe = make_pipeline(StandardScaler(), reducer,
                         KNeighborsClassifier(n_neighbors=5))
    acc = cross_val_score(pipe, X, y, cv=5).mean()
    print(f"{name} to 2-D, then 5-NN: CV accuracy {acc:.3f}")
```

一如預測，十個類別的 LDA 回傳九個方向。壓縮到二維後，在 LDA 座標上的 5-NN 分類器交叉驗證準確率為 0.624，在前兩個主成分上則為 0.512。因為 LDA 使用標籤，它被放在 pipeline 內，在每個摺中重新擬合。圖 3 說明原因：LDA 把整個數字類別拉開，PCA 則主要呈現像素變化最大的方向。

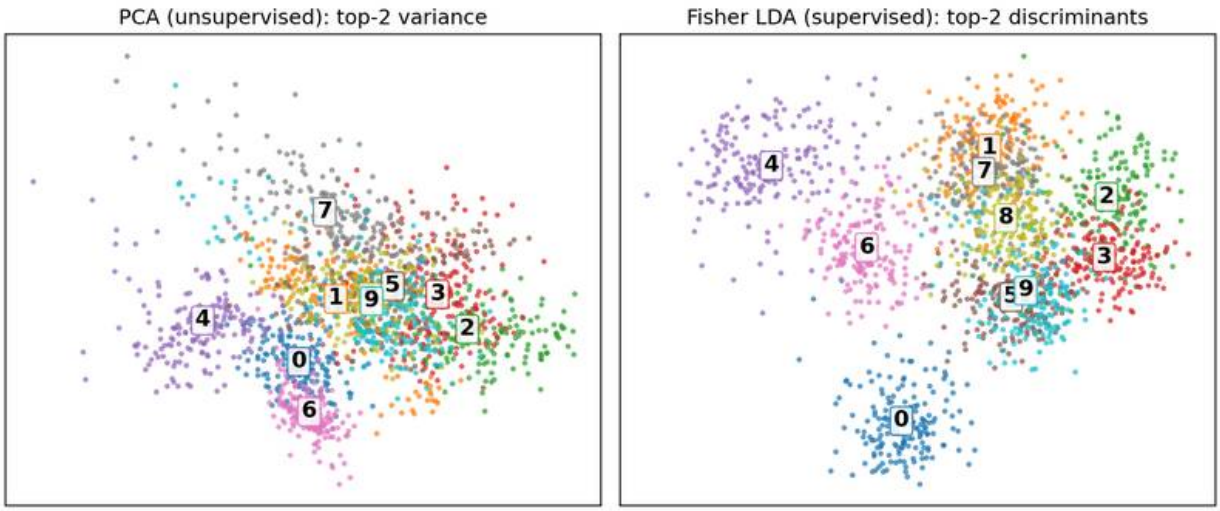


圖 3 — 數字資料投影到二維。左：前兩個主成分（未使用標籤）。右：前兩個 *Fisher* 判別方向（使用標籤）。數字標記各類別的平均位置。

核 PCA

白話版。核 PCA 就是在核的豐富特徵空間中做 PCA（PR 04），而且從不需要算出那些特徵。它能把任何線性投影都攤不開的結構攤開。

Schölkopf、Smola 與 Müller 證明，在特徵空間 $\phi(x)$ 中做 PCA 只需要核矩陣 $K_{ij} = k(x_i, x_j)$ [12]。先在特徵空間中將它中心化：

$$\tilde{K} = K - \mathbf{1}_N K - K\mathbf{1}_N + \mathbf{1}_N K \mathbf{1}_N,$$

其中 $\mathbf{1}_N$ 是每個元素皆為 $1/N$ 的 $N \times N$ 矩陣。對係數向量 $a_i \in \mathbb{R}^N$ 解特徵值問題 $\tilde{K} a_i = N\lambda_i a_i$，並正規化使 $N\lambda_i \, a_i^\top a_i = 1$。點 x 的第 i 個核主成分為

$$z_i(x) = \sum_{n=1}^{N} a_{in} \, \tilde{k}(x_n, x),$$

$\tilde{k}$ 是以同樣方式中心化的核。成本是一個 $N \times N$ 的特徵值問題，新點則需要 N 次核計算。

核 PCA 仍然是非監督的：它依特徵空間中的變異數為成分排序，而不是依類別分離程度。在雙環資料上，前兩個 RBF 核成分描述的是繞著環的角度，把兩類混在一起；第三個成分描述的是半徑，能把兩類完全分開（圖 4）。

```
import numpy as np
from sklearn.datasets import make_circles
from sklearn.decomposition import PCA, KernelPCA
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
```

```
X, y = make_circles(n_samples=400, factor=0.4, noise=0.07, random_state=0)
Z = KernelPCA(n_components=4, kernel="rbf", gamma=2).fit_transform(X)
print("|corr(component, label)|:",
      np.round([abs(np.corrcoef(Z[:, i], y)[0, 1]) for i in range(4)], 3))

for name, reducer in [("linear PCA, 2 comps", PCA(n_components=2)),
                      ("kernel PCA, 3 comps", KernelPCA(n_components=3,
                                                        kernel="rbf", gamma=2))]:
    acc = cross_val_score(make_pipeline(reducer, LogisticRegression()),
                          X, y, cv=5).mean()
    print(f"{name} -> logistic regression: {acc:.3f}")
```

只有第三個成分和標籤相關（0.982；其餘都低於 0.05）。在兩個線性主成分上，邏輯斯迴歸無法勝過隨機猜測（0.432）；在三個核成分上則完全正確（1.000）。

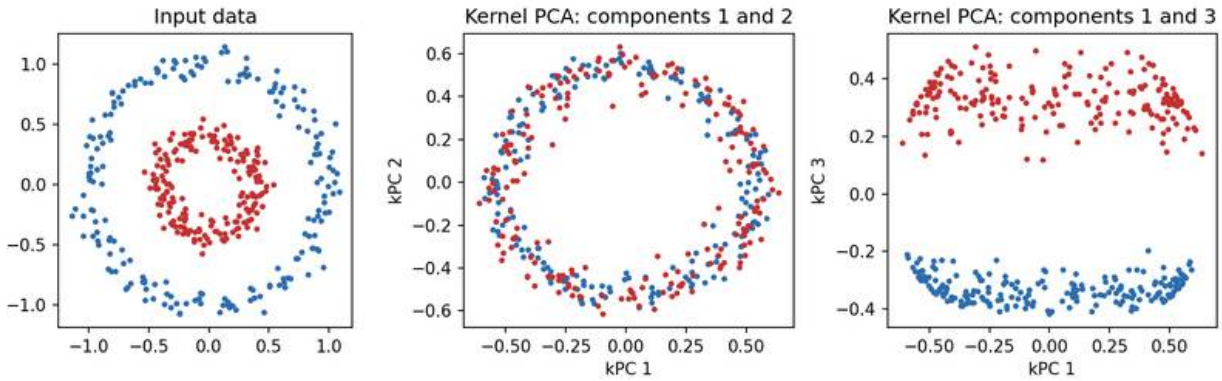


圖 4 — 雙環資料上的核 *PCA*（*RBF*，$\gamma = 2$）。前面的成分捕捉角度並混合兩類；成分 3 捕捉半徑並把它們分開。特徵空間中的高變異仍然不代表高類別資訊。

獨立成分分析

白話版。在派對上，兩支麥克風各錄到兩個人聲的混合。ICA 試圖只從混合訊號中還原出各別的人聲，所依靠的是這些人聲彼此統計獨立而且不是高斯分布。

ICA 假設觀測向量是獨立訊號源的線性混合 [13]：

$$x = As,$$

其中 $s = (s_1, \dots, s_M)$ 是統計獨立、非高斯的訊號源，A 是未知的混合矩陣，x 是觀測到的訊號。目標是找到解混矩陣 $W \approx A^{-1}$，使 $\hat{s} = Wx$ 的各成分彼此獨立。為什麼要非高斯？根據中央極限定理，混合訊號往往比訊號源「更像高斯」，所以最大化 $w^\top x$ 的非高斯性就能找到一個訊號源。FastICA 以負熵（negentropy）的近似衡量非高斯性，並在資料白化後執行固定點迭代 [13]。有兩種模糊性永遠無法消除：訊號源的順序與尺度（包括正負號）。

和 PCA 的對比正是重點。PCA 只讓成分不相關（二階統計量）；ICA 使用高階統計量，讓成分獨

立。如果訊號源是高斯的，不相關就已經等於獨立，白化後資料的任何旋轉都一樣好，ICA 就會失敗。

```
import numpy as np
from sklearn.decomposition import FastICA

t = np.linspace(0, 8, 2000)
S = np.c_[np.sin(3 * t),                      # source 1: sine
          np.sign(np.sin(5 * t)),             # source 2: square wave
          ((2 * t) % 2) - 1]                  # source 3: sawtooth
A = np.array([[1.0, 0.5, 0.3],
              [0.6, 1.0, 0.4],
              [0.4, 0.3, 1.0]])               # unknown mixing matrix
X = S @ A.T                                   # what the sensors record

S_hat = FastICA(n_components=3, whiten="unit-variance",
                random_state=0).fit_transform(X)
corr = np.abs(np.corrcoef(S.T, S_hat.T)[:3, 3:])
print(np.round(corr, 3))                      # one entry near 1 per row
```

每個真實訊號源都恰好對應一個還原出的成分，相關係數絕對值為 0.999 或 1.000，其餘配對都低於 0.05。圖 5 顯示，對同樣的混合訊號做 PCA，得到的是互不相關、但仍然是混合的訊號。

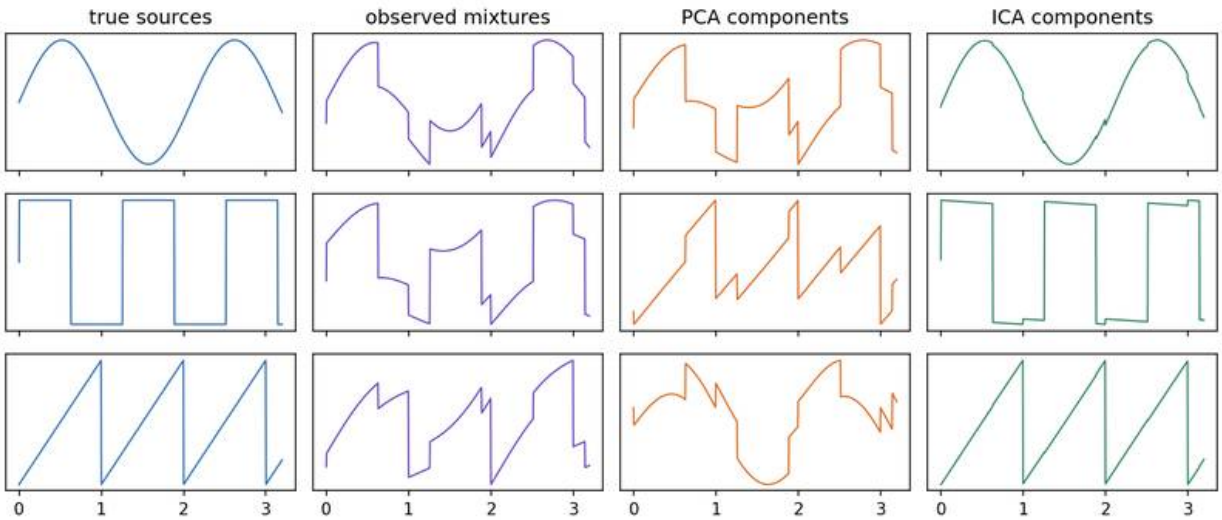


圖 5 — 盲訊號源分離。*PCA* 讓混合訊號去相關，卻沒有把它們分離；*FastICA* 在順序與尺度之外還原出正弦、方波與鋸齒波。

非負矩陣分解（non-negative matrix factorization, NMF）是適用於非負資料（例如像素強度或字詞計數）的相關分解：它把資料寫成 $X \approx WH$，其中 W, H 的每個元素都 ≥ 0。由於不能做減法，分解出的因子傾向於是可相加的「部件」，Lee 與 Seung 在人臉影像上展示了這一點 [14]。

流形學習

白話版。有時高維資料落在一張彎曲的低維薄片上，就像一張捲起來的海報。流形學習試著把薄片攤開：沿著薄片相鄰的點要保持靠近，而且不能被那些只是因為海報捲起來、才在三維中靠近的點騙過。

圖 6 的瑞士捲（Swiss roll）是標準例子：一個二維長方形在三維中捲起來。PCA 以投影把它壓平，讓捲的不同層疊在一起。以鄰域為基礎的方法表現較好：

- **Isomap** [15] 建立 k 近鄰圖，以圖上的最短路徑長度估計沿流形的測地距離，再找出歐氏距離與這些測地距離相符的低維座標（經典多維尺度法）。
- **局部線性嵌入**（locally linear embedding, LLE）[16] 把每個點寫成鄰居的加權組合 $x_n \approx \sum_{m \in \mathcal{N}(n)} W_{nm} x_m$，且 $\sum_m W_{nm} = 1$，再找出能以同一組權重重建的低維點 y_n。
- **t-SNE** [17] 把距離轉換成輸入空間中的鄰居機率 p_{ij}（高斯，頻寬由「perplexity」參數決定）與嵌入空間中的 q_{ij}（重尾的 Student-t），並最小化 Kullback–Leibler 散度 $\sum_{i \neq j} p_{ij} \log(p_{ij}/q_{ij})$。重尾讓分得很開的群可以散開。
- **UMAP** [18] 建立加權的 k 近鄰圖（一種「模糊」的拓撲表示），再以交叉熵目標最佳化低維配置。它通常比 t-SNE 快，也常用於相同目的。

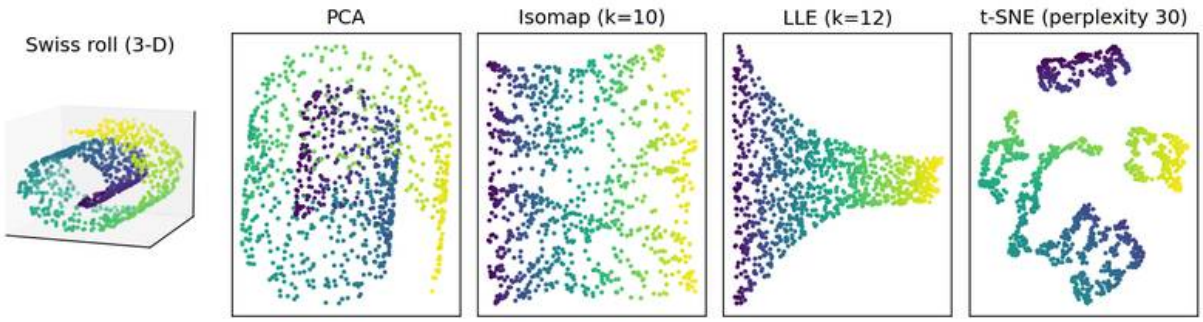


圖 6 — 瑞士捲與四種二維嵌入，依沿捲位置著色。*Isomap* 與 *LLE* 把它攤開；*PCA* 做不到；*t-SNE* 保持鄰居相鄰，卻把連續的薄片撕成分開的島嶼。

一種定量檢查是**可信度**（trustworthiness）：嵌入中每個點的 k 個最近鄰，在輸入空間中也是近鄰的程度（1.0 為完美）。

```
from sklearn.datasets import make_swiss_roll
from sklearn.decomposition import PCA
from sklearn.manifold import Isomap, LocallyLinearEmbedding, TSNE, trustworthiness

X, t = make_swiss_roll(n_samples=1000, noise=0.05, random_state=0)
methods = {
    "PCA": PCA(n_components=2),
    "Isomap": Isomap(n_neighbors=10, n_components=2),
    "LLE": LocallyLinearEmbedding(n_neighbors=12, n_components=2,
                                  random_state=0),
    "t-SNE": TSNE(n_components=2, perplexity=30, init="pca",
                  random_state=0),
```

```
}
for name, method in methods.items():
    Y = method.fit_transform(X)
    print(f"{name:7s} trustworthiness (k=10): "
          f"{trustworthiness(X, Y, n_neighbors=10):.3f}")
```

PCA 得 0.905；Isomap 與 t-SNE 得 0.999；LLE 得 0.996。注意這個數字看不到的東西：即使圖 6 顯示 t-SNE 把捲撕成了島嶼，它的分數仍和 Isomap 一樣高。局部鄰域保留了，全域結構卻沒有。

嵌入圖能證明什麼、不能證明什麼

這些方法主要是用來觀察資料的工具。從它們的運作方式可以推出三點提醒 [19]：

- **群的大小與間距預設沒有意義。** t-SNE 會把局部密度正規化，所以緊密的群和鬆散的群可能看起來一樣大，群與群之間的距離也可能說明不了什麼。
- **結果取決於設定。** perplexity、鄰居數、初始化與隨機種子都會改變圖形。Wattenberg、Viégas 與 Johnson 的互動式文章展示了同一份資料產生非常不同的 t-SNE 圖 [19]。
- **二維可能不夠。**投影到二維必然扭曲高維距離；圖中分得很開的島嶼並不能證明存在分開的族群。

在分類上還有一個實務缺口：t-SNE 與標準 LLE 只為訓練點學出座標，沒有自然的規則能嵌入新點。Isomap、核 PCA 與 UMAP 提供樣本外轉換，但如果非線性嵌入是分類器的一部分，就必須像其他步驟一樣在訓練摺內擬合。

影像與訊號描述子

白話版。在特徵可以學習之前，人們手工設計特徵，精確量測他們認為重要的性質：紋理、邊緣、形狀、頻率內容。它們至今仍是有用的基準與可解釋的檢查工具。

Theodoridis 與 Koutroumbas 以兩章篇幅介紹以轉換為基礎與針對應用的特徵 [1]；IP 12 詳細討論影像描述子。想一想每一種保留了什麼、對什麼敏感：

描述子	保留	典型的敏感因素
強度直方圖	數值的分布	完全失去空間排列
共生矩陣（GLCM）統計量 [20]	某一位移下成對強度的關係	量化階數與位移方向
方向梯度直方圖（HOG）[21]	格狀小區塊內的局部邊緣方向	裁切、尺度與對齊
面積、周長、緊緻度	區域幾何	分割錯誤與解析度
Fourier/DCT 係數	頻率內容	對齊方式與保留哪些頻率
小波係數	變化的位置與尺度	基底選擇與邊界處理

對量化影像 I 與位移 δ，灰階共生矩陣計算所有像素位置 p 上的配對 $(I(p), I(p+\delta))$；正規化為機率 $P(i,j)$ 後，Haralick 的紋理特徵包括對比 $\sum_{i,j}(i-j)^2 P(i,j)$ 與能量 $\sum_{i,j} P(i,j)^2$ [20]。務必註明位移與灰階數，因為數值取決於兩者。對面積為 A、周長為 P 的分割物件，緊緻度 $4\pi A/P^2$ 在理想連續圓時等於 1，形狀越不規則越小；以像素估計周長會讓小物件的數值失真。Dalal 與 Triggs

的 HOG 描述子由小區塊內正規化的梯度方向直方圖構成，最初用於行人偵測，後來成為最廣泛使用的人工設計影像特徵之一 [21]。

對音訊與其他一維訊號，特徵是在訊號近似穩定的短時窗內計算：各頻帶的頻譜能量、過零率等。時窗長度在時間解析度與頻率解析度之間取捨。

程式碼：同一個分類器，三種特徵集合

這個例子在內建的 digits 資料上，以同一個分類器比較固定的特徵策略。

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression

X, y = load_digits(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=9)
transforms = {
    "all features": "passthrough",
    "24 PCA coordinates": PCA(n_components=24, random_state=9),
    "24 selected coordinates": SelectKBest(f_classif, k=24),
}
for name, transform in transforms.items():
    model = make_pipeline(
        StandardScaler(), transform, LogisticRegression(max_iter=1500))
    model.fit(Xtr, ytr)
    print(name, "test accuracy:", round(model.score(Xte, yte), 4))
```

全部 64 個像素的測試準確率為 0.969，24 個主成分為 0.956，ANOVA F 分數最高的 24 個像素為 0.953。在這裡降到約三分之一的維度，準確率損失很小。數字影像有些邊緣像素是常數，所以 F 檢定會對常數特徵發出警告；這個警告反映的是變異數為零，不是資訊洩漏，那些像素只是得不到有意義的分數。

現代觀點

人工的特徵工程大多已被學習得到的表徵取代，但本章的概念遍布現代流程：深度特徵同樣會被選擇、投影、白化、視覺化，並用相同的工具評估。以下是綜述文獻對這個領域的描繪。

特徵選擇。 Guyon 與 Elisseeff 的導論 [3] 確立了詞彙（過濾式、包裝式、嵌入式），討論了特徵建構、排序與驗證，並強調一個特徵可能單獨無用、組合起來卻有用。Chandrashekar 與 Sahin [23] 提供一份依同樣的過濾式／包裝式／嵌入式架構整理的精簡綜述。Li 等人 [22]「從資料的角度」重新檢視特徵選擇：他們把傳統資料的演算法分為基於相似度、基於資訊理論、基於稀疏學

習與基於統計四類，加入結構化、異質與串流資料的方法，並釋出開源的評測資料庫。Ambroise 與 McLachlan 關於資訊洩漏的警告 [7]，仍是最重要的單一實務教訓。

PCA 與其後繼者。 Jolliffe 與 Cadima 的綜述 [10] 把 PCA 說明為一種由資料定義的自適應技術，並整理了針對不同資料型態的變體。在現代深度學習中，PCA 以診斷工具（某一層的激活中，有多少方向承載了大部分變異）以及前處理與初始化步驟的形式延續，例如 t-SNE 建議採用的 PCA 初始化 [26]。

用於視覺化的降維。 Espadoto 等人 [24] 對投影技術做了大規模的定量綜述：他們以多種定量指標（例如鄰域與距離的保留程度）刻畫資料集、技術與投影品質，量測結果如何隨技術與資料變化，並把觀察整理成在特定情境下選擇投影方法的指引，所有程式與結果都以開放基準的形式釋出。Wang、Huang、Rudin 與 Shaposhnik [25] 分析了 t-SNE、UMAP 與 TriMAP 為何會有各自的行為，聚焦於保留局部結構與全域結構之間的取捨，並提出兼顧兩者的 PaCMAP。

二維嵌入的陷阱。 在 t-SNE 與 UMAP 已成例行工具的單細胞生物學中，出現了最尖銳的批評。Kobak 與 Berens [26] 提出讓 t-SNE 圖更忠實的流程：PCA 初始化、較高的學習率與多尺度相似度。Kobak 與 Linderman [27] 指出，常被歸功於 UMAP 的較佳全域結構，大部分來自它的初始化；使用同樣有資訊的初始化時，t-SNE 也能同樣好地保留全域結構。Chari 與 Pachter [28] 更進一步主張：極端地降到二維必然扭曲高維資料，這類圖不應用來做定量結論。

再談變異與有用性。 圖 2 的教訓在自監督學習中以更大的規模重現。Balestriero 與 LeCun [29] 認為，以重建輸入為目標訓練表徵，會強調資料中高變異的成分，而這些成分對感知任務的資訊量往往不如低變異的細節；因此以重建訓練的特徵需要額外步驟（例如微調）才能用於分類。這正是 PCA 的限制，換成了深度學習的場景。

未解問題。 在特徵高度相關、樣本又少的情況下，選擇結果依然不穩定：不同子集可能表現一樣好，讓解釋變得困難。評估嵌入仍缺乏能同時涵蓋局部與全域忠實度的公認指標 [24], [25]。至於學習得到的特徵，如何判斷哪些方向承載的是與任務相關、而不只是高變異的資訊，是活躍的研究問題 [29]。

重點整理

- 特徵選擇保留原始量測；萃取建立新座標；學習則從資料擬合萃取規則。三者都決定了分類器看得到什麼。
- 過濾式方法快速，但會漏掉交互作用與冗餘；包裝式方法看得見，但成本較高且可能對驗證資料過擬合；嵌入式方法（lasso、樹）在訓練中完成選擇。
- 任何使用標籤的步驟（特徵選擇、LDA、決定特徵數量）都必須在每個交叉驗證摺內重新擬合。在完整資料上選擇，可能在純雜訊上回報 98% 的準確率。
- Fisher 比值、$J = \mathrm{tr}(S_W^{-1} S_B)$、Bhattacharyya 距離與互資訊可為特徵評分；由於冗餘，子集必須整體評分。
- PCA 保留最大變異（共變異數的特徵向量，以 SVD 計算）；白化讓各方向變異相等。變異不等於類別資訊。
- Fisher LDA 解 $S_B v = \lambda S_W v$，最多給出 $K - 1$ 個方向。核 PCA、ICA 與 NMF 把線性萃取延伸到非線性、獨立或非負的結構。
- Isomap、LLE、t-SNE 與 UMAP 主要是視覺化工具：它們的圖取決於設定，群與群之間的間隙不能證明族群分離。

練習

1. **相同直方圖、不同影像**。構造兩張 4×4 的二值影像，強度直方圖相同但紋理不同。表中哪一種描述子能分辨它們？

   提示

   用八個值為 1 的像素，一張排成棋盤格，另一張填滿左半邊。兩者直方圖相同。水平位移 $(0, 1)$ 的共生矩陣可以分辨：棋盤格只有不相等的鄰居配對，左半填滿的影像則大多是相等的配對。

2. **$K-1$ 的上限**。證明 K 個類別的類間散佈 S_B 秩至多為 $K-1$。這對兩類問題上的 LDA 代表什麼？

   提示

   $S_B = \sum_c P_c d_c d_c^\top$，其中 $d_c = \mu_c - \mu$，且 $\sum_c P_c d_c = 0$，所以 K 個向量 d_c 至多張成 $K-1$ 維。兩類時 LDA 只給出一個方向 $v \propto S_W^{-1}(\mu_1 - \mu_2)$。

3. **99% 的變異也不夠時**。描述一個實際的成像情境，其中保留 99% 的 PCA 變異仍會丟掉類別訊號。你要如何偵測這個問題？又該怎麼做？

   提示

   大幅且獨立的光照或姿態變化，加上一個小而一致的瑕疵標記（換了外衣的圖 2）。比較有無 PCA 時的交叉驗證準確率，或逐一檢查各成分的類別分離程度（Fisher 比值）即可偵測。改用監督式投影、先把干擾因素（例如光照）正規化，或乾脆不用 PCA。

4. **XOR 陷阱**。產生在 $[-1, 1]$ 上均勻分布的 x_1, x_2 與八個無關的雜訊特徵，令 $y = \mathbf{1}[x_1 > 0] \oplus \mathbf{1}[x_2 > 0]$。分別以 `f_classif` 與 `mutual_info_classif` 為十個特徵排序，再以決策樹做循序前向選擇。哪些方法找得到 x_1 與 x_2？

   提示

   兩種單變量過濾給 x_1、x_2 的分數都接近零，和雜訊特徵一樣，因為每一個單獨都與 y 獨立。一次加一個特徵的包裝式方法，第一步也可能因為同樣的理由而卡住；能評估特徵配對的樹狀包裝式方法，或從全部特徵開始的後向刪除，則找得到它們。

5. **量化資訊洩漏**。在選擇偏差的程式中，改變雜訊特徵數（100、1,000、5,000）與樣本數（30、60、200）。「錯誤做法」的準確率如何變化？解釋這個趨勢。

   提示

   樂觀程度隨候選特徵數增加、隨樣本數減少：雜訊特徵越多，在小樣本中碰巧與標籤相關的就越多，而挑出最好的 20 個，挑中的正是這些。

6. **解讀 t-SNE 圖**。在 digits 資料上以 perplexity 5、30、100 執行 t-SNE，每種各用兩個隨機種子。圖中哪些特徵是穩定的（哪些數字彼此相鄰？），哪些會改變（群的大小、間隙、位置）？從單張這種圖，你願意宣稱什麼？

提示

局部鄰域的成員相當穩定；絕對位置、群的面積與群間距離則不穩定。站得住腳的說法是「這些點在輸入空間中局部相似」，而不是「這兩群相距很遠」或「這一群的多樣性是另一群的兩倍」。

參考文獻

1. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2009, Chapters 5–7. publisher page
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006, Sections 4.1.4–4.1.6 and 12.1–12.4. official book page and PDF
3. I. Guyon and A. Elisseeff, "An Introduction to Variable and Feature Selection," *Journal of Machine Learning Research*, vol. 3, pp. 1157–1182, 2003. JMLR
4. R. Kohavi and G. H. John, "Wrappers for Feature Subset Selection," *Artificial Intelligence*, vol. 97, pp. 273–324, 1997. doi:10.1016/S0004-3702(97)00043-X
5. R. Tibshirani, "Regression Shrinkage and Selection via the Lasso," *Journal of the Royal Statistical Society, Series B*, vol. 58, no. 1, pp. 267–288, 1996. doi:10.1111/j.2517-6161.1996.tb02080.x
6. H. Peng, F. Long, and C. Ding, "Feature Selection Based on Mutual Information: Criteria of Max-Dependency, Max-Relevance, and Min-Redundancy," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 8, pp. 1226–1238, 2005. doi:10.1109/TPAMI.2005.159
7. C. Ambroise and G. J. McLachlan, "Selection Bias in Gene Extraction on the Basis of Microarray Gene-Expression Data," *Proceedings of the National Academy of Sciences*, vol. 99, no. 10, pp. 6562–6566, 2002. doi:10.1073/pnas.102102699
8. K. Pearson, "On Lines and Planes of Closest Fit to Systems of Points in Space," *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, vol. 2, no. 11, pp. 559–572, 1901. doi:10.1080/14786440109462720
9. H. Hotelling, "Analysis of a Complex of Statistical Variables into Principal Components," *Journal of Educational Psychology*, vol. 24, no. 6, pp. 417–441, 1933. doi:10.1037/h0071325
10. I. T. Jolliffe and J. Cadima, "Principal Component Analysis: A Review and Recent Developments," *Philosophical Transactions of the Royal Society A*, vol. 374, 20150202, 2016. doi:10.1098/rsta.2015.0202
11. R. A. Fisher, "The Use of Multiple Measurements in Taxonomic Problems," *Annals of Eugenics*, vol. 7, no. 2, pp. 179–188, 1936. doi:10.1111/j.1469-1809.1936.tb02137.x
12. B. Schölkopf, A. Smola, and K.-R. Müller, "Nonlinear Component Analysis as a Kernel Eigenvalue Problem," *Neural Computation*, vol. 10, no. 5, pp. 1299–1319, 1998. doi:10.1162/089976698300017467
13. A. Hyvärinen and E. Oja, "Independent Component Analysis: Algorithms and Applications," *Neural Networks*, vol. 13, no. 4–5, pp. 411–430, 2000. doi:10.1016/S0893-6080(00)00026-5
14. D. D. Lee and H. S. Seung, "Learning the Parts of Objects by Non-Negative Matrix Factorization," *Nature*, vol. 401, pp. 788–791, 1999. doi:10.1038/44565

15. J. B. Tenenbaum, V. de Silva, and J. C. Langford, "A Global Geometric Framework for Nonlinear Dimensionality Reduction," *Science*, vol. 290, no. 5500, pp. 2319–2323, 2000. doi:10.1126/science.290.5500.2319
16. S. T. Roweis and L. K. Saul, "Nonlinear Dimensionality Reduction by Locally Linear Embedding," *Science*, vol. 290, no. 5500, pp. 2323–2326, 2000. doi:10.1126/science.290.5500.2323
17. L. van der Maaten and G. Hinton, "Visualizing Data using t-SNE," *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008. JMLR
18. L. McInnes, J. Healy, and J. Melville, "UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction," arXiv:1802.03426, 2018. arXiv
19. M. Wattenberg, F. Viégas, and I. Johnson, "How to Use t-SNE Effectively," *Distill*, 2016. doi:10.23915/distill.00002
20. R. M. Haralick, K. Shanmugam, and I. Dinstein, "Textural Features for Image Classification," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-3, no. 6, pp. 610–621, 1973. doi:10.1109/TSMC.1973.4309314
21. N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, vol. 1, pp. 886–893, 2005. doi:10.1109/CVPR.2005.177
22. J. Li, K. Cheng, S. Wang, F. Morstatter, R. P. Trevino, J. Tang, and H. Liu, "Feature Selection: A Data Perspective," *ACM Computing Surveys*, vol. 50, no. 6, 2018. doi:10.1145/3136625
23. G. Chandrashekar and F. Sahin, "A Survey on Feature Selection Methods," *Computers & Electrical Engineering*, vol. 40, no. 1, pp. 16–28, 2014. doi:10.1016/j.compeleceng.2013.11.024
24. M. Espadoto, R. M. Martins, A. Kerren, N. S. T. Hirata, and A. C. Telea, "Toward a Quantitative Survey of Dimension Reduction Techniques," *IEEE Transactions on Visualization and Computer Graphics*, vol. 27, no. 3, pp. 2153–2173, 2021. doi:10.1109/TVCG.2019.2944182
25. Y. Wang, H. Huang, C. Rudin, and Y. Shaposhnik, "Understanding How Dimension Reduction Tools Work: An Empirical Approach to Deciphering t-SNE, UMAP, TriMAP, and PaCMAP for Data Visualization," *Journal of Machine Learning Research*, vol. 22, 2021. arXiv:2012.04456
26. D. Kobak and P. Berens, "The Art of Using t-SNE for Single-Cell Transcriptomics," *Nature Communications*, vol. 10, 2019. doi:10.1038/s41467-019-13056-x
27. D. Kobak and G. C. Linderman, "Initialization Is Critical for Preserving Global Data Structure in Both t-SNE and UMAP," *Nature Biotechnology*, vol. 39, pp. 156–157, 2021. doi:10.1038/s41587-020-00809-z
28. T. Chari and L. Pachter, "The Specious Art of Single-Cell Genomics," *PLOS Computational Biology*, vol. 19, e1011288, 2023. doi:10.1371/journal.pcbi.1011288
29. R. Balestriero and Y. LeCun, "Learning by Reconstruction Produces Uninformative Features for Perception," arXiv:2402.11337, 2024. arXiv

PR 06・模板比對與結構化辨識

以相關、正規化互相關與動態時間扭曲比對圖形，以字串、文法與圖描述結構，並用圖模型、HMM（前向演算法、Viterbi）與 CRF 建模脈絡。

網站版本: https://blog.shyandram.dev/zh-tw/learn/pattern-recognition/pr-06-templates-structured-models/

你將學到

- 模板比對如何以差平方和、互相關與正規化互相關（NCC）運作，以及各自能容忍哪些干擾變化。
- 動態時間扭曲（dynamic time warping, DTW）如何對齊以不同速度展開的序列，以及如何加以約束。
- 句法與結構式圖形識別：把圖形描述為字串、文法與圖，並以編輯距離與圖匹配加以比較。
- 有向與無向圖模型如何分解聯合分布，以及為什麼這讓推論變得有效率。
- 隱藏式馬可夫模型（HMM）：前向演算法、前向–後向平滑、Viterbi 解碼與 Baum–Welch 學習，附經過測試的程式。
- 條件隨機場（CRF）作為 HMM 的判別式對應版本，以及這些概念在 2026 年如何延續。

先看全貌

PR 03 與PR 04 的分類器一次只看一個固定長度的特徵向量，並單獨決定它的類別。許多辨識問題並非如此：

- 要在影像中找到某個零件，你把參考圖片在影像上滑動，尋找最吻合的位置（**模板**）。
- 一個口說的詞或一個簽名隨時間展開，而不同的人產生它的速度不同（**對齊**）。
- 一個字元、一個分子或一張道路網，最好用它的部件以及部件如何連接來描述（**結構**）。
- 單獨一小塊影像可能是道路也可能是屋頂；潦草的手寫字母可能是「c」也可能是「e」。相鄰的區域或字母能消除這種歧義（**脈絡**）。

> **用最簡單的話說**
>
> 讀潦草的手寫字時，你不會盯著每個字母單獨看。你會把它的形狀和你認識的字母比較（模板），允許某些字母被拉長或壓扁（對齊），知道筆畫如何相連構成一個字母（結構），並用相鄰的字母猜出整個字（脈絡）。本章把這些習慣一一變成演算法。

整章有一條共同的主線：**動態規劃**。DTW、編輯距離、HMM 前向演算法、Viterbi 解碼與 CRF 推論，用的都是同一個技巧：把對所有對齊方式或標籤序列的龐大搜尋，拆成可以重複利用的小子問題，成本就從指數降到多項式。

開頭先提醒一點：只有當脈絡的假設符合現實時，脈絡才有幫助。過強的「鄰居要一致」規則可能抹掉小物件或短暫的事件，所以脈絡並非越多越好。本章的教科書背景是 Theodoridis 與 Koutroumbas 第 8、9 章 [1]，以及 Bishop 第 8.1–8.4 與 13.1–13.3 節 [2]。

模板比對

白話版。保留一張小的參考影像，也就是模板。把它在場景上滑動，在每個位置量測底下那塊影像有多相似，分數最好的位置就是匹配處。有趣的問題在於如何量測相似度，才不會被無關的變化（例如曝光變亮或變暗）騙過。

三種相似度

把模板寫成由 M 個像素值組成的向量 t，模板下方的影像區塊寫成同樣長度 M 的 x。

差平方和（SSD）。$\mathrm{SSD}(x,t) = \|x-t\|^2 = \sum_i (x_i - t_i)^2$，越小越好。這是最自然的距離，但場景中任何亮度偏移或對比變化都會讓它變大。

互相關。把平方展開，$\|x-t\|^2 = \|x\|^2 - 2x^\top t + \|t\|^2$。模板能量 $\|t\|^2$ 是常數，所以如果區塊能量 $\|x\|^2$ 也大致固定，最小化 SSD 就等同於最大化相關 $x^\top t$。實務上 $\|x\|^2$ 變化很大，單純的相關會被明亮區域吸引。

正規化互相關（NCC）。先減去平均，再除以範數：

$$\mathrm{NCC}(x,t) = \frac{(x-\bar{x}\mathbf{1})^\top (t-\bar{t}\mathbf{1})}{\|x-\bar{x}\mathbf{1}\|\,\|t-\bar{t}\mathbf{1}\|},$$

其中 $\bar{x}$ 與 $\bar{t}$ 是區塊與模板的平均像素值，$\mathbf{1}$ 是全為 1 的向量。NCC 是兩個減去平均後的向量夾角的餘弦，所以落在 $[-1,1]$；它等於 1 若且唯若對某個 $a>0$ 與任意 b 有 $x = a\,t + b\mathbf{1}$。因此它對正的對比變化（a）與亮度偏移（b）不變。模板或區塊為常數時 NCC 沒有定義（分母為零），各實作會回傳 0 或 NaN 之類的約定值。NCC 就是IP 13 中的「以相關進行比對」。

有效率地計算

在 $N \times N$ 影像的每個位置以 $m \times m$ 模板計算 NCC，成本約為 $N^2 m^2$ 次運算。分子是一個相關運算，可以用 FFT 一次算出所有位置（相關定理，IP 04）。分母中的局部平均與能量隨位置變化；Lewis 說明如何由影像及其平方的累加表（積分影像）便宜地取得它們，讓 NCC 幾乎和單純的相關一樣快 [3]。

```
import cv2
import numpy as np
from skimage import data

img = data.camera().astype(np.float32)          # 512 x 512 grayscale
r0, c0, h, w = 300, 200, 40, 40
template = img[r0:r0 + h, c0:c0 + w].copy()
```

```
def ncc_at(window, t):
    a, b = window - window.mean(), t - t.mean()
    return float((a * b).sum() / (np.linalg.norm(a) * np.linalg.norm(b)))

# The scene is captured again under different exposure: darker and lower contrast.
scene = 0.5 * img + 20.0
ncc = cv2.matchTemplate(scene, template, cv2.TM_CCOEFF_NORMED)
ssd = cv2.matchTemplate(scene, template, cv2.TM_SQDIFF)
r, c = np.unravel_index(ncc.argmax(), ncc.shape)
print("NCC best (row, col):", (int(r), int(c)), "score", round(float(ncc.max()), 4))
r, c = np.unravel_index(ssd.argmin(), ssd.shape)
print("SSD best (row, col):", (int(r), int(c)))
print("hand-computed NCC at true location:",
      round(ncc_at(scene[r0:r0 + h, c0:c0 + w], template), 4))
```

NCC 在真實位置 (300, 200) 找到模板，分數為 1.0，手算值也和 OpenCV 的 `TM_CCOEFF_NORMED` 一致。SSD 被曝光變化騙過，選了 (130, 413)：這是一塊天空，變暗後的強度（平均約 129）比變暗後的真實區塊（平均約 92）更接近模板原本的數值（平均約 143）。

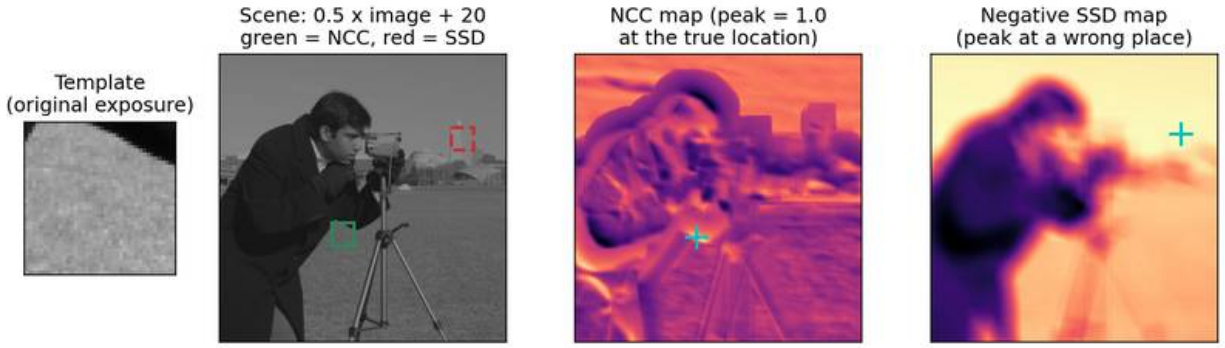


圖 *1* — 曝光變化下的模板比對。*NCC* 對增益 *0.5* 與偏移 *20* 不變，找到了真實位置；*SSD* 則否。由 *scripts/figures/pr_06_templates_structured_models_extra.py* 產生。

模板處理不了的事

NCC 只消除亮度與對比，其他一概不管。旋轉、尺度變化、透視、遮蔽與非剛性形變都會降低分數。經典的補救方法是儲存一整組模板（旋轉與縮放後的副本）、在影像金字塔上搜尋，或先比對小部件再檢查它們的幾何排列，後者就通往下文的結構式方法。當外觀受到嚴格控制時（例如工業檢測），模板比對仍是正確的工具，它也仍是追蹤與影像對位的核心步驟 [4]。

動態時間扭曲

白話版。兩個人說同一個詞，一個慢、一個快。逐點比較兩段錄音會失敗，因為聲音在時間上對不齊。DTW 拉長或壓縮其中一條序列的時間軸，讓它與另一條盡可能對齊，再量測剩下的差異。

遞迴式

令 $x = (x_1, \dots, x_n)$ 與 $t = (t_1, \dots, t_m)$ 為兩條序列，$d(x_i, t_j)$ 為局部成本，例如 $|x_i - t_j|$，或向量樣本的 $\|x_i - t_j\|^2$。**扭曲路徑**是一串索引對 $(i_1, j_1), \dots, (i_L, j_L)$，從 $(1,1)$ 開始、在 (n,m) 結束，每次只走 $(1,0)$、$(0,1)$ 或 $(1,1)$ 其中一步，因此在兩條序列中都不會倒退。DTW 是所有扭曲路徑中總成本最小者，以填表方式計算：

$$D(i,j) = d(x_i, t_j) + \min\{D(i-1,j),\ D(i,j-1),\ D(i-1,j-1)\},$$

其中 $D(0,0) = 0$，且對 $i, j \geq 1$ 有 $D(i,0) = D(0,j) = \infty$。於是 $\mathrm{DTW}(x,t) = D(n,m)$。每一格只需要它的三個鄰格，所以時間成本是 $O(nm)$。回溯每一格的最小值來自哪個鄰格，就能還原最佳對齊。這就是 Sakoe 與 Chiba 為口說詞辨識所最佳化的演算法 [5]。

約束與注意事項

不加約束的路徑可能把一條序列的單一樣本對應到另一條序列的一長段（「病態扭曲」），讓不相關的序列看起來相似。兩種標準的解法是：

- **全域約束**，例如 Sakoe–Chiba 帶 $|i - j| \leq w$，禁止遠離對角線的格子，也把成本降為 $O(nw)$ [5]；
- **斜率約束**或步伐權重，懲罰過長的水平或垂直移動 [5]。

帶寬 w 必須容得下真實的時間差。在程式與圖 2 中，3 個樣本的帶寬比真實的位移（約 6 個樣本）窄，所以加了帶的 DTW 無法完全對齊兩條正弦波。

實務上還有兩件事很重要。DTW 不是度量：它可能違反三角不等式，所以假設度量性質的資料結構要小心使用。此外，序列在比較前通常會先做 **z 正規化**（平均為零、變異數為一），讓 DTW 比較的是形狀，而不是偏移與尺度。

```
import numpy as np

def dtw(x, t, band=None):
    """DTW distance between 1-D sequences x (length n) and t (length m)."""
    n, m = len(x), len(t)
    D = np.full((n + 1, m + 1), np.inf)
    D[0, 0] = 0.0
    for i in range(1, n + 1):
        lo, hi = 1, m
        if band is not None:                    # Sakoe-Chiba band
            lo, hi = max(1, i - band), min(m, i + band)
        for j in range(lo, hi + 1):
            cost = abs(x[i - 1] - t[j - 1])
            D[i, j] = cost + min(D[i - 1, j], D[i, j - 1], D[i - 1, j - 1])
    return D[n, m]

s = np.linspace(0, 2 * np.pi, 60)
a = np.sin(s)
b = np.sin(s - 0.6)                             # same shape, shifted in time
c = 0.25 * np.sign(np.sin(3 * s))               # a different shape
```

```
print("lock-step a-b:", round(np.abs(a - b).sum(), 2),
      " a-c:", round(np.abs(a - c).sum(), 2))
print("DTW       a-b:", round(dtw(a, b), 2),
      " a-c:", round(dtw(a, c), 2))
print("DTW a-b with band 3:", round(dtw(a, b, band=3), 2))
```

兩種比較都使用同樣的局部成本 $|x_i - t_j|$。逐點（lock-step）比較時，平移後的副本 b 成本為 22.76，不同形狀的 c 為 35.18。DTW 把平移副本的成本降到 5.05，而不同形狀仍維持 34.57：扭曲吸收的是時間差，而不是形狀差。加上 3 個樣本的帶之後，平移副本的成本升到 12.76，因為對約 6 個樣本的位移而言，這個帶太窄了。

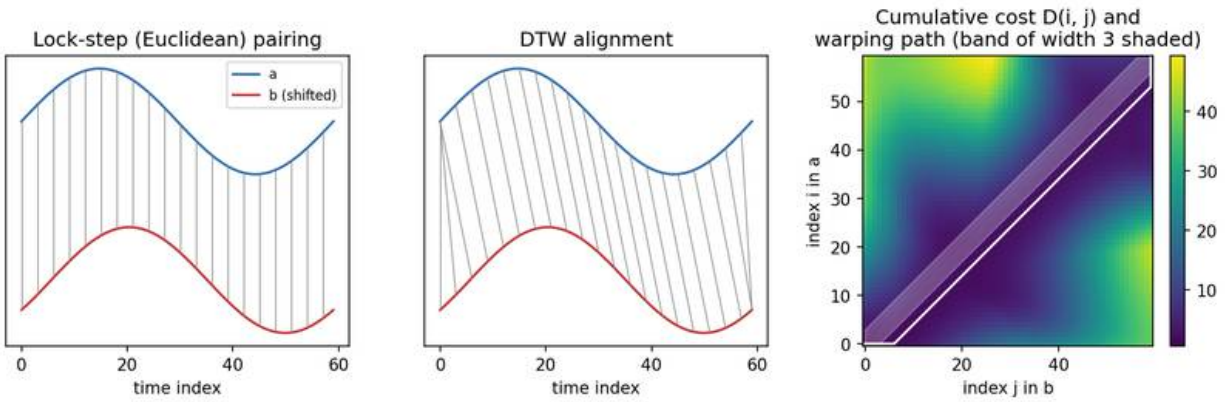


圖 2 — 動態時間扭曲。逐點比較把相同的時間索引配成一對；*DTW* 則把兩條曲線中相對應的部分配對。在成本矩陣中，最佳路徑偏離對角線約 *6* 個樣本，落在寬度 *3* 的帶外。

以 DTW 距離做 1-最近鄰分類，多年來一直是時間序列分類中強而難以超越的基準 [16]。結合便宜的下界與及早放棄無望的候選之後，在極長的序列中以 DTW 搜尋最近鄰才變得可行 [6]。

句法與結構式圖形識別

白話版。有些圖形最好不要用一串數字描述，而是用它的部件以及部件如何組合來描述，就像句子是依文法排列的字詞。結構式圖形識別把圖形描述成由基本部件組成的字串、樹或圖，再比較或剖析這些結構。

字串與文法

先選一小組**基本元素**（primitives）構成字母表，例如沿物件邊界走一步的四個方向。於是一個形狀就變成一個字串。4 方向**鏈碼**（chain code）在追蹤邊界時，向右一步寫 0、向上寫 1、向左寫 2、向下寫 3（參見IP 12）。圖 3 顯示三個輪廓與它們的鏈碼。

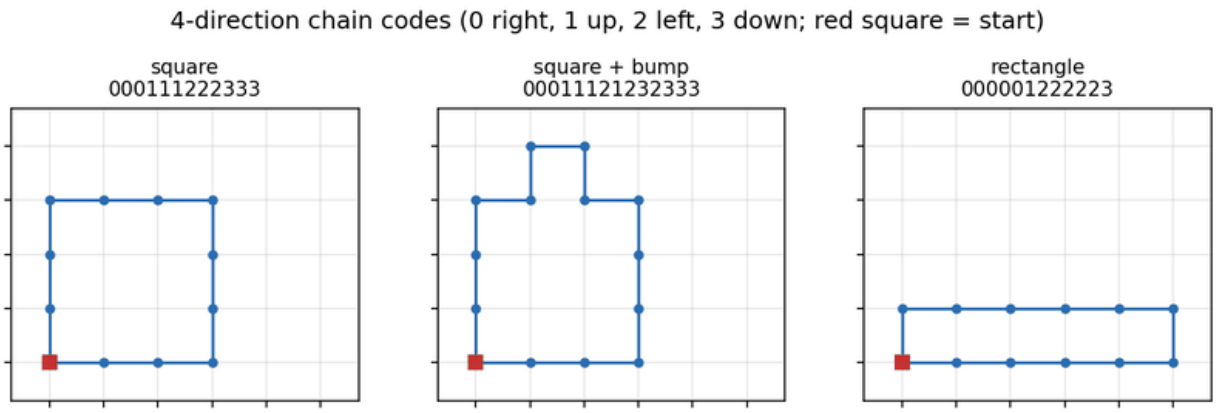


圖 3 — 把形狀寫成字串。每個輪廓都從紅色起點逆時針追蹤，邊界每走一步寫下一個符號。

在**句法**（syntactic）圖形識別中，每個類別由一套形式文法描述：一組改寫規則，恰好產生該類別所有合法的字串。辨識就是**剖析**：若某類別的文法能產生這個字串，字串就屬於該類別。例如規則 $S \to 0S2 \mid 1$ 產生字串 $0^k 1\, 2^k$，這個「類別」中每段向右的步數都有等長的向左步數與之對應。文法可以表達遞迴結構（巢狀部件、重複的子形狀），這是固定長度特徵向量做不到的。它的弱點是：真實資料有雜訊，單一個錯誤的基本元素就會讓字串不合文法；而且文法很難從範例中學出來。為規則附上機率的隨機文法可以處理第一個問題；下文的隱藏式馬可夫模型就是最簡單的隨機文法 [1]。

編輯距離

一種更寬容的做法，是用「把一個字串變成另一個的最低成本」來比較字串。**編輯距離**（Levenshtein 距離）計算最少需要幾次插入、刪除與替換。Wagner 與 Fischer 給出了動態規劃解法 [7]：

$$E(i,j) = \min\{E(i-1,j)+1,\; E(i,j-1)+1,\; E(i-1,j-1)+\mathbf{1}[s_i \neq u_j]\},$$

其中 $E(i,0)=i$、$E(0,j)=j$，s（長度 n）與 u（長度 m）為兩個字串，答案是 $E(n,m)$。它的結構和 DTW 完全相同，只是把「拉長」換成「插入」、「刪除」與「替換」三種操作。成本可以加權，例如讓相鄰方向之間的替換比相反方向便宜。

```
import numpy as np

def edit_distance(s, t):
    """Levenshtein distance: unit-cost insertions, deletions, substitutions."""
    n, m = len(s), len(t)
    D = np.zeros((n + 1, m + 1), dtype=int)
    D[:, 0] = np.arange(n + 1)
    D[0, :] = np.arange(m + 1)
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            sub = 0 if s[i - 1] == t[j - 1] else 1
            D[i, j] = min(D[i - 1, j] + 1,          # delete s[i-1]
                          D[i, j - 1] + 1,          # insert t[j-1]
```

```
                           D[i - 1, j - 1] + sub)    # match or substitute
    return D[n, m]

# 4-direction chain codes (0=right, 1=up, 2=left, 3=down) of three outlines
square = "000111222333"           # 3 x 3 square, traced counter-clockwise
square_bump = "00011121232333"   # the same square with a one-pixel bump on top
rectangle = "000001222223"       # a flat 5 x 1 rectangle
for name, code in [("square + bump", square_bump), ("rectangle", rectangle)]:
    print(f"edit distance(square, {name}) = {edit_distance(square, code)}")
```

有凸起的正方形和正方形相差 2 次編輯（插入一個「上」與一個「下」）；扁平的長方形相差 4 次。以這個距離在字串上做最近鄰分類，就是 k-NN 的結構式版本。鏈碼取決於起點與旋轉；把它們正規化（例如選擇使字串最小的起點，並改用相鄰方向之間的差）是表徵設計的一部分。

圖與圖匹配

當部件之間的連接不只是一條鏈時，圖形就成為一張**帶屬性的圖**：節點是部件（筆畫、區域、原子），帶有大小或顏色等屬性；邊是關係（相接、在上方、鍵結）。辨識於是成為**圖匹配**：找出兩張圖的節點之間的對應關係。精確匹配（圖同構或子圖同構）不容許任何差異，一般而言計算困難；容錯匹配，例如**圖編輯距離**，以節點與邊的插入、刪除、替換推廣字串編輯距離，計算代價更高。Conte、Foggia、Sansone 與 Vento 綜述了圖形識別中三十年的圖匹配研究，從精確演算法到不精確演算法及其應用 [8]；Foggia、Percannella 與 Vento 則以十年間的圖匹配與圖學習方法更新了這幅圖像 [9]。

圖模型描述相依關係

白話版。圖模型是一張「哪些變數直接影響哪些變數」的圖。這張圖告訴你龐大的聯合機率如何拆成小的局部片段，而小片段正是讓計算變得可行的關鍵。

有向無環圖（貝氏網路）把變數 $z_1, \dots, z_M$ 的聯合分布分解為

$$p(z_1, \dots, z_M) = \prod_{i=1}^{M} p(z_i \mid \mathrm{pa}(z_i)),$$

其中 $\mathrm{pa}(z_i)$ 是 z_i 在圖中的父節點 [2]。**無向**圖（馬可夫隨機場）在它的團（clique，即兩兩相連的節點子集）C 上使用非負的位能函數 ψ_C：

$$p(z) = \frac{1}{Z} \prod_{C} \psi_C(z_C), \qquad Z = \sum_{z} \prod_{C} \psi_C(z_C).$$

位能是相容程度的分數，不一定是機率。**配分函數** Z 把乘積正規化；它對所有聯合組態求和，往往是最難計算的量。

條件獨立要依圖的規則判讀（有向圖用 d-分離，無向圖用圖分離），不能只看哪裡沒有箭頭。一個經典的意外是：兩個獨立的原因，在以它們共同的結果為條件後就會變得相依（「解釋消除」，explaining away）[2]。**因子圖**把變數與因子畫成兩種節點，讓局部的訊息傳遞變得明確；在樹

上，訊息傳遞（sum-product 演算法）能精確算出所有邊際分布，而下文的 HMM 演算法就是它在鏈上的特例 [2]。

在影像標記中，一種常見的無向模型是由一元資料項與成對平滑項組成的**能量**：

$$E(z; x) = \sum_{i} U_i(z_i; x) + \lambda \sum_{(i,j)\in\mathcal{E}} \mathbf{1}[z_i \neq z_j],$$

其中 z_i 是像素 i 的標籤，U_i 是在影像 x 下該標籤的成本（例如像素分類器輸出的負對數機率），$\mathcal{E}$ 是相鄰像素對的集合，$\lambda \geq 0$ 是相鄰像素標籤改變的代價（Potts 模型）。對應的分布是 $p(z \mid x) \propto \exp(-E(z; x))$。它鼓勵相鄰標籤一致，能清除雜訊，但 λ 太大時也會抹掉細長的結構。在一般的格點上精確最小化很困難，近似推論見PR 08。

隱藏式馬可夫模型

白話版。一個系統在幾個隱藏狀態之間移動（靜音、母音、子音；走路、跑步），每個時間步發出一個只取決於當下狀態的觀測。你看得到觀測，卻永遠看不到狀態。HMM 讓你能計算一串觀測有多可能、它大概是由哪些隱藏狀態產生的，以及如何從資料中學出模型的機率。

模型

有 K 個隱藏狀態的 HMM，包含初始機率 $\pi_j = p(z_1 = j)$、**轉移矩陣** $A_{ij} = p(z_t = j \mid z_{t-1} = i)$，以及**發射**機率（或密度）$B_j(x_t) = p(x_t \mid z_t = j)$。狀態序列 $z_{1:T} = (z_1, \dots, z_T)$ 與觀測 $x_{1:T}$ 的聯合機率為

$$p(z_{1:T}, x_{1:T}) = \pi_{z_1} B_{z_1}(x_1) \prod_{t=2}^{T} A_{z_{t-1}, z_t} B_{z_t}(x_t).$$

其中內建了兩個假設：隱藏狀態是一階馬可夫的（未來只透過當下狀態依賴過去），且每個觀測只取決於當下狀態。觀測本身不是馬可夫的：把隱藏狀態積分掉後，觀測之間會出現長程相依。Rabiner 的教學文章至今仍是標準入門，它把 HMM 的計算整理成三個問題 [10]：**評估**（計算 $p(x_{1:T})$）、**解碼**（找出隱藏狀態）與**學習**（估計 π, A, B）。圖 4 對照 HMM 的有向圖與稍後討論的 CRF。

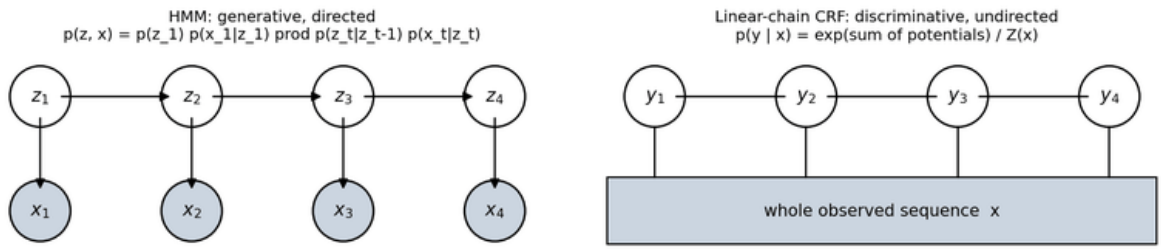


圖 4 — *HMM* 與線性鏈 *CRF*。*HMM* 是狀態與觀測的有向生成式模型；*CRF* 是以整條觀測序列為條件的標籤無向模型。

評估：前向演算法

對全部 K^T 條狀態序列加總聯合機率，在實際的 T 下毫無可能。定義**前向變數** $\alpha_t(j) = p(x_{1:t}, z_t = j)$。由於 z_t 總結了過去能告訴未來的一切，它滿足

$$\alpha_1(j) = \pi_j B_j(x_1), \qquad \alpha_t(j) = B_j(x_t) \sum_{i=1}^{K} \alpha_{t-1}(i) A_{ij},$$

且 $p(x_{1:T}) = \sum_j \alpha_T(j)$。成本是 $O(TK^2)$，而不是 $O(TK^T)$。由於 α_t 是許多機率的乘積，長序列會造成數值下溢。標準解法是每一步都正規化：記錄 $c_t = \sum_j \tilde{\alpha}_t(j)$ 與 $\hat{\alpha}_t = \tilde{\alpha}_t / c_t$，其中 $\tilde{\alpha}_t$ 是未正規化的更新值。這時 $\hat{\alpha}_t(j) = p(z_t = j \mid x_{1:t})$ 恰好就是**濾波**狀態機率，$c_t = p(x_t \mid x_{1:t-1})$，而且

$$\log p(x_{1:T}) = \sum_{t=1}^{T} \log c_t.$$

評估直接給出一個分類器：每個類別訓練一個 HMM（例如每個口說詞一個），新序列就歸給「其 HMM 概似乘以先驗」最高的類別。

平滑：前向–後向演算法

濾波只用過去與現在計算 $p(z_t \mid x_{1:t})$，這是線上系統在時間 t 所能知道的一切。**平滑**計算 $p(z_t \mid x_{1:T})$，連之後的觀測也一起使用。後向遞迴 $\beta_t(i) = p(x_{t+1:T} \mid z_t = i)$，

$$\beta_T(i) = 1, \qquad \beta_t(i) = \sum_{j=1}^{K} A_{ij}\, B_j(x_{t+1})\, \beta_{t+1}(j),$$

和前向變數結合，得到 $\gamma_t(j) = p(z_t = j \mid x_{1:T}) \propto \alpha_t(j)\beta_t(j)$ [2], [10]。加入尺度化時，每個後向步驟除以 c_{t+1}，於是 $\gamma_t = \hat{\alpha}_t \odot \hat{\beta}_t$（逐元素乘積）恰好成立。

解碼：Viterbi 演算法

我們常常想要的是機率最高的單一狀態序列 $\arg\max_{z_{1:T}} p(z_{1:T} \mid x_{1:T})$。Viterbi 演算法 [11] 把前向遞迴中的加總換成取最大值。在對數空間中，

$$\delta_1(j) = \log \pi_j + \log B_j(x_1), \qquad \delta_t(j) = \log B_j(x_t) + \max_i \{\delta_{t-1}(i) + \log A_{ij}\},$$

其中 $\delta_t(j)$ 是在時間 t 結束於狀態 j 的最佳路徑的對數機率。記錄使其最大的前一個狀態 $\psi_t(j) = \arg\max_i\{\cdots\}$，選出最佳的最終狀態，再沿指標往回追溯。成本同樣是 $O(TK^2)$。

機率最高的路徑，不等於在每個時間點各自挑選平滑機率 $\gamma_t(j)$ 最高的狀態。逐步挑選最大化的是「預期正確的狀態數」，但可能把模型認定不能相連的狀態串在一起。下面的程式在一個狀態必須依 $0 \to 1 \to 2 \to 0$ 循環的模型上展示這一點，其中轉移 $0 \to 2$ 的機率為零。

```
import itertools
import numpy as np
```

```
pi = np.array([1 / 3, 1 / 3, 1 / 3])           # initial state probabilities
A = np.array([[0.5, 0.5, 0.0],                 # cyclic model: 0 -> 1 -> 2 -> 0
              [0.0, 0.5, 0.5],
              [0.5, 0.0, 0.5]])
B = np.array([[0.7, 0.3],                      # P(symbol | state)
              [0.5, 0.5],
              [0.3, 0.7]])
obs = np.array([1, 0, 1, 0, 0])
T, K = len(obs), len(pi)

# Forward pass with scaling: alpha[t] = p(z_t | x_1..t), c[t] = p(x_t | x_1..t-1).
alpha, c = np.zeros((T, K)), np.zeros(T)
alpha[0] = pi * B[:, obs[0]]
c[0] = alpha[0].sum(); alpha[0] /= c[0]
for t in range(1, T):
    alpha[t] = B[:, obs[t]] * (alpha[t - 1] @ A)
    c[t] = alpha[t].sum(); alpha[t] /= c[t]
log_lik = np.log(c).sum()

# Backward pass (scaled) and smoothed marginals gamma[t] = p(z_t | x_1..T).
beta = np.ones((T, K))
for t in range(T - 2, -1, -1):
    beta[t] = A @ (B[:, obs[t + 1]] * beta[t + 1]) / c[t + 1]
gamma = alpha * beta

# Viterbi in log space (log 0 = -inf marks forbidden transitions).
with np.errstate(divide="ignore"):
    lA, lB, lpi = np.log(A), np.log(B), np.log(pi)
delta = lpi + lB[:, obs[0]]
back = []
for t in range(1, T):
    cand = delta[:, None] + lA
    back.append(cand.argmax(axis=0))
    delta = cand.max(axis=0) + lB[:, obs[t]]
path = [int(delta.argmax())]
for b in reversed(back):
    path.append(int(b[path[-1]]))
path = path[::-1]

# Brute force over all K**T paths as an independent check.
def joint(z):
    p = pi[z[0]] * B[z[0], obs[0]]
    for t in range(1, T):
        p *= A[z[t - 1], z[t]] * B[z[t], obs[t]]
    return p
paths = list(itertools.product(range(K), repeat=T))
assert np.isclose(log_lik, np.log(sum(joint(z) for z in paths)))
assert tuple(path) == max(paths, key=joint)

marginal = gamma.argmax(axis=1)
print("log p(x):", round(log_lik, 4))
print("Viterbi path:         ", path, " p =", round(joint(path), 5))
print("per-step argmax gamma:", marginal.tolist(),
```

```
      " p =", round(joint(marginal), 5))
```

尺度化的前向計算給出 $\log p(x) = -3.5449$，而概似與 Viterbi 路徑都和全部 $3^5 = 243$ 條路徑的暴力列舉一致。Viterbi 回傳 $1 \to 1 \to 2 \to 0 \to 0$，聯合機率為 0.00179。逐步挑選機率最高的狀態得到 $2, 0, 2, 0, 0$，其中包含被禁止的跳躍 $0 \to 2$，在模型下的機率恰好為零。哪一種解碼才對取決於損失：逐步準確率偏好邊際分布；要得到合法、一致的序列則需要 Viterbi。

圖 5 在一條較長的模擬序列上展示同樣的計算：濾波機率只對過去做出反應；平滑機率使用整條序列，因而更有把握；Viterbi 路徑除了觀測有歧義的地方之外，都跟得上真實狀態。

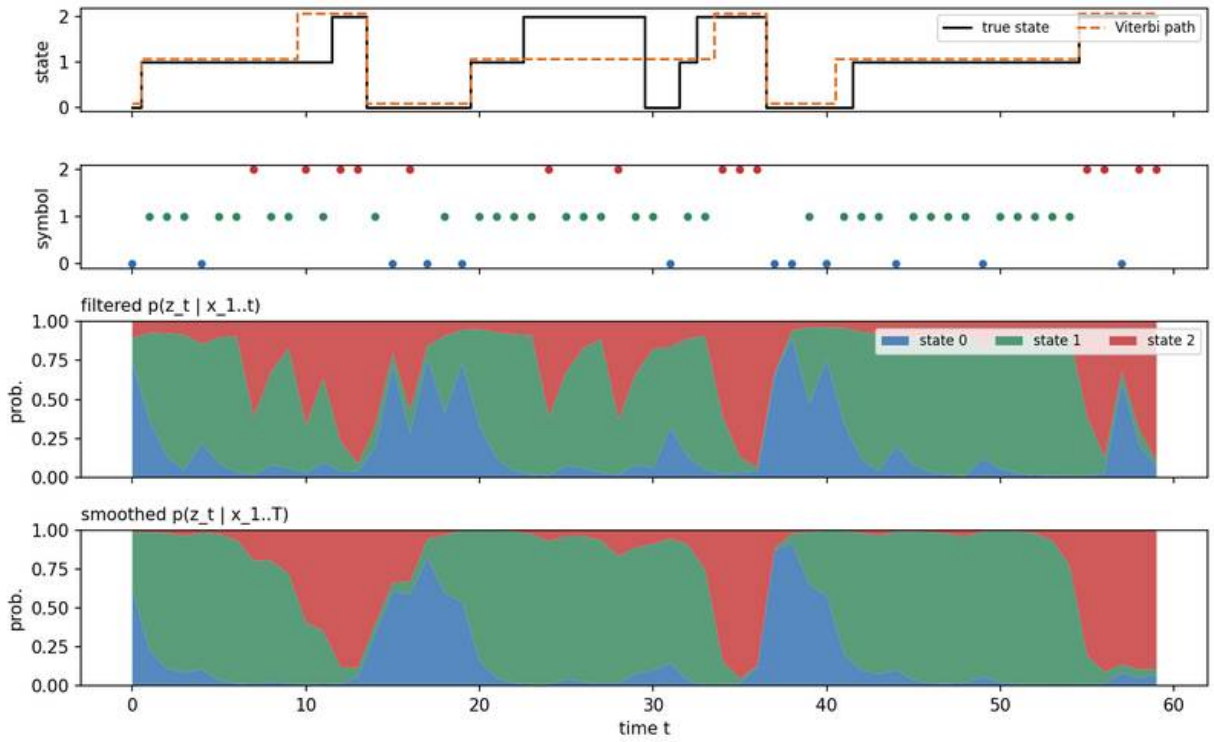


圖 *5* — 模擬的 *3* 狀態循環 *HMM*。濾波使用到時間 t 為止的觀測；平滑使用整條序列。在這次執行中，*Viterbi* 路徑在 *78%* 的時間步與真實狀態相符，逐步挑選平滑機率為 *77%*，逐步挑選濾波機率為 *73%*。

關於評估的提醒。如果系統必須線上做決策，請回報濾波（或固定延遲）下的表現。平滑結果用到了決策當下尚不存在的觀測，會高估部署後系統的能力。

學習：Baum–Welch

當隱藏狀態沒有標註時，參數以期望值最大化（EM）估計，在 HMM 中稱為 **Baum–Welch** 演算法 [12]。E 步驟執行前向–後向，得到預期的狀態佔用 $\gamma_t(j)$ 與預期的轉移 $\xi_t(i,j) = p(z_{t-1} = i, z_t = j \mid x_{1:T})$。M 步驟重新估計

$$\hat{A}_{ij} = \frac{\sum_{t=2}^{T} \xi_t(i,j)}{\sum_{t=2}^{T} \sum_{k} \xi_t(i,k)}, \qquad \hat{\pi}_j = \gamma_1(j),$$

並以 $\gamma_t(j)$ 加權的觀測估計發射參數（例如高斯發射的加權平均與共變異數）。每次迭代都不會降低概似，但結果取決於初始化，可能停在局部極大值；PR 07 介紹一般形式的 EM。如果隱藏狀態是連續的，而且一切都是線性高斯，同樣的前向遞迴就成為 Kalman 濾波器 [13]。以學習得到的向量取代離散狀態的神經遞迴模型，見ML 03A。

條件隨機場

白話版。 HMM 建模的是狀態如何產生觀測，這迫使它對觀測做簡單的假設。如果你只需要「給定觀測下的標籤」，CRF 就直接建模這件事，而且可以使用整個輸入的任何特徵，不論彼此是否重疊。

HMM 是**生成式**的：它建模 $p(z, x)$，包括觀測如何產生。這要求發射模型 $B_j(x_t)$ 完整描述 x_t；當特徵很多、彼此重疊又相依時（字詞本身、它的字尾、大小寫、相鄰的字詞……），這就很彆扭。Lafferty、McCallum 與 Pereira 提出條件隨機場，直接建模給定觀測 x 時標籤序列 $y = (y_1, \dots, y_T)$ 的**條件**分布 [14]。**線性鏈 CRF** 為

$$p(y \mid x) = \frac{1}{Z(x)} \exp\Big(\sum_{t=1}^{T} \sum_{k} \theta_k f_k(y_{t-1}, y_t, x, t) \Big), \qquad Z(x) = \sum_{y'} \exp\Big(\sum_{t,k} \theta_k f_k(y'_{t-1}, y'_t, x, t) \Big),$$

其中 f_k 是特徵函數，可以看當前與前一個標籤，以及輸入序列的任何部分；θ_k 是學習得到的權重；$Z(x)$ 針對這一個輸入，對所有標籤序列 y' 做正規化 [14], [15]。把 HMM 寫成這種形式，它就是特例：特徵只有「前一個標籤、當前標籤」與「當前標籤、當前觀測」兩類指示函數。CRF 可以加入任意的輸入特徵，而不必建模它們的分布。

推論使用和 HMM 相同的動態規劃：前向演算法（以 log-sum-exp 取代加總）計算 $Z(x)$ 與邊際分布，Viterbi 找出最佳標記。訓練時最大化條件對數概似 $\sum_n \log p(y^{(n)} \mid x^{(n)})$，通常加上 ℓ_2 懲罰。對線性鏈 CRF 而言，這個目標是凹函數，因此沒有不好的局部最佳解；它的梯度是「觀測到的特徵計數減去預期的特徵計數」，預期值由前向–後向算出 [15]。下面的程式以暴力法檢查 $\log Z$ 的前向計算。

```
import itertools
import numpy as np

rng = np.random.default_rng(0)
T, K = 5, 3
U = rng.normal(size=(T, K))       # unary scores  theta . f(y_t, x, t)
P = rng.normal(size=(K, K))       # transition scores  theta . f(y_{t-1}, y_t)

def score(y):
    return U[np.arange(T), y].sum() + sum(P[y[t - 1], y[t]] for t in range(1, T))

# log Z by the forward recursion in log space (log-sum-exp instead of sum).
log_alpha = U[0].copy()
for t in range(1, T):
```

```
    m = log_alpha[:, None] + P
    log_alpha = U[t] + np.logaddexp.reduce(m, axis=0)
log_Z = np.logaddexp.reduce(log_alpha)

brute = np.logaddexp.reduce([score(np.array(y))
                             for y in itertools.product(range(K), repeat=T)])
print("log Z forward:", round(log_Z, 6), " brute force:", round(brute, 6))
assert np.isclose(log_Z, brute)
```

兩者都得到 $\log Z = 6.173403$。這裡的分數是代替 $\sum_k \theta_k f_k$ 的隨機數；在真正的 CRF 中，它們來自 x 的特徵，在現代系統中則來自神經網路。

在影像上，同樣的想法得到像素上的格狀 CRF：一元項來自像素分類器，成對項類似上面的 Potts 能量。在有迴圈的格點上已無法做精確推論，因此使用近似方法（PR 08）。

現代觀點

模板、對齊、文法與馬可夫鏈，在深度網路旁邊看起來有點過時。事實上它們都延續了下來，有時就藏在網路內部。以下是綜述文獻與近期研究中的主要脈絡。

模板比對。 Hashemi 等人 [4] 回顧了比對的基本概念、不變特徵與可變形模型等進展，以及包括醫學影像分析在內的應用。他們的結論是：模板比對與其他辨識步驟結合時效果最好，例如在 NCC 之後接一個穩健的決策階段。當外觀受控時，以相關為基礎的比對快速、簡單又精確，至今仍是工業檢測、影像對位的標準做法，也是追蹤器的基本組件。當外觀會變化時，比對就轉移到學習得到的特徵空間：「模板」現在是一張特徵圖，相關運算在深度特徵之間進行，但運算本身是一樣的。

時間序列分類與 DTW。 Bagnall 等人的「bake off」[16] 在同一個框架中重新實作了 18 個時間序列分類演算法，並在 85 個 UCR 資料集上各做 100 次重抽樣比較。只有九個演算法顯著優於兩個基準（其中之一是 DTW 的 1-NN），而一個由轉換式集成組成的方法（COTE）勝過其他所有方法。Middlehurst、Schäfer 與 Bagnall 的「bake off redux」[17] 在擴充為 112 個資料集的資料庫與 30 個新問題上重做這項比較，在分類體系中加入以卷積、以特徵與深度學習為基礎的類別，並發現兩個較新的演算法 MultiROCKET+Hydra 與 HIVE-COTE 2 顯著優於其餘方法。Ismail Fawaz 等人 [18] 以統一的分類體系回顧了時間序列分類中的深度學習，並在同一個開放框架中於 97 個資料集上訓練了 8,730 個網路。與此同時，DTW 本身成為一個基本組件：soft-DTW [19] 把 DTW 遞迴中的最小值換成平滑的 soft-minimum，使它可微分，因此能作為訓練網路的損失，也能用來平均時間序列。

神經網路內部的對齊。連結時序分類（connectionist temporal classification, CTC）[20] 讓遞迴網路在沒有逐幀對齊的情況下輸出標籤序列，方法是以一個和 HMM 密切相關的前向–後向遞迴，對所有對齊方式加總。動態規劃留了下來，發射模型則變成了網路。

結合深度特徵的 CRF。 Sutton 與 McCallum 的教學 [15] 至今仍是 CRF 建模、推論與訓練的標準參考。在視覺領域，具高斯邊位能的全連接 CRF 讓所有像素對上的平均場推論變得有效率 [21]，並成為語意分割常用的精修步驟；Zheng 等人接著把這個平均場推論寫成一個遞迴網路層，和 CNN 一起端到端訓練 [22]。在語言領域，Lample 等人的 BiLSTM-CRF [23] 在雙向 LSTM 上加了一層 CRF 輸出，用於命名實體辨識，讓標籤轉移被明確建模，而特徵則由學習得到。在這兩個

領域，後來具有大感受野或注意力機制的架構往往已建模足夠的脈絡，使明確的 CRF 幫助有限；但在標籤約束很重要的場合，結構化輸出層仍在使用。

結構與圖。 Conte 等人 [8] 與 Foggia 等人 [9] 的圖匹配綜述，描繪了從精確與容錯匹配，走向圖核、圖嵌入與以學習為基礎之方法的轉變。它們的現代後代，圖神經網路，以訊息傳遞學習節點與整張圖的表徵，正是因子圖局部計算的直接推廣。

狀態空間模型。「一個精簡的隱藏狀態總結了過去」這個概念，是 HMM 與 Kalman 濾波器 [13] 的核心，如今在序列建模中再度成為焦點。Mamba [24] 這類選擇性狀態空間模型使用與輸入相關的線性遞迴，執行時間與序列長度成線性關係，被提出作為長序列上注意力機制的替代方案。

未解問題。 DTW 與編輯距離每次比較都是平方成本，所以在大型資料集中搜尋必須依靠界限與剪枝 [6]。圖編輯距離的精確計算仍然昂貴。HMM 與 CRF 的結果取決於圖中內建的獨立性假設；當假設錯誤時，平滑可能很有自信地抹掉短暫事件。而對任何線上系統來說，濾波與平滑評估之間的區別都很容易弄錯。

重點整理

- NCC 減去平均並除以範數，因此對亮度偏移與正的對比變化不變；SSD 與單純的相關則否。兩者都無法處理旋轉、尺度或形變。
- DTW 以動態規劃在 $O(nm)$ 內找出成本最低的單調對齊；Sakoe–Chiba 帶限制病態扭曲並降低成本，但必須比真實的時間差更寬。
- 結構式辨識把圖形描述為字串、文法或圖；編輯距離以和 DTW 相同的動態規劃模式比較字串。
- 圖模型把聯合分布分解成局部片段；在鏈與樹上的訊息傳遞既精確又有效率。
- HMM 演算法：前向（$p(x)$ 與濾波）、前向–後向（平滑）、Viterbi（最佳路徑）、Baum–Welch（學習），成本都是 $O(TK^2)$。以尺度化或對數運算避免下溢。
- Viterbi 路徑與逐步最可能狀態回答的是不同的問題；後者可能是不可能出現的序列。
- CRF 直接建模 $p(y \mid x)$，可使用任意重疊的輸入特徵，與 HMM 共用推論演算法，而在線性鏈情況下訓練目標是凹函數。

練習

1. **NCC 與負片。**證明把區塊的每個像素換成 $ax_i + b$（$a > 0$）時 NCC 不變。$a < 0$ 時會怎樣？為什麼 NCC 無法告訴你 b 的值？

   提示

   減去平均會完全消除 b，所以關於它的資訊一點都不剩。因子 a 出現在分子，並以 $|a|$ 出現在分母；$a > 0$ 時互相抵消，$a < 0$ 時正負號翻轉（完美匹配的負片得分為 -1）。

2. **手算 DTW。**以成本 $|x_i - t_j|$ 計算 $x = (0, 1, 2)$ 與 $t = (0, 0, 1, 2)$ 的 DTW 距離。寫出 3×4 的表格與最佳路徑。逐點距離是多少？為什麼在這裡它根本沒有定義？

提示

最佳路徑把 x_1 同時對應到兩個 0，接著 $x_2 \leftrightarrow t_3$、$x_3 \leftrightarrow t_4$，總成本為 0。逐點比較需要等長，而這兩條序列不等長；DTW 自然地處理不等長的情況。

3. **被禁止的轉移**。自己構造一個兩狀態或三狀態的 HMM，把其中一個轉移機率設為零，並找出一串觀測，使 γ_t 的逐步 argmax 成為不可能的狀態序列。用上面的程式驗證。

提示

你需要兩個相鄰時間步，它們各自最可能的狀態之間恰好是被禁止的轉移。發射機率接近 0.5（有歧義）的對稱或循環模型很容易做到；接著檢查 `joint(marginal)` 是否為 0。

4. **線上與離線**。某活動辨識器以錄好的序列、根據平滑狀態機率回報 95% 的準確率。它將在手機上執行，每秒都必須做決定。這個數字包含了什麼資訊上的優勢？你應該改為回報什麼？

提示

平滑用到了每次決策之後的觀測。應回報濾波準確率，或應用可以容忍之延遲下的固定延遲平滑結果（例如延遲 2 秒的決策可以使用到 $t+2$ 為止的觀測）。

5. **從 HMM 到 CRF**。把 HMM 的 $\log p(z, x)$ 寫成特徵函數乘以權重的總和，即線性鏈 CRF 的形式。你得到哪些特徵與權重？在詞性標記中，CRF 可以加入哪些 HMM 難以使用的額外特徵？

提示

使用指示特徵 $\mathbf{1}[z_{t-1} = i, z_t = j]$，權重為 $\log A_{ij}$；$\mathbf{1}[z_t = j, x_t = v]$，權重為 $\log B_j(v)$；以及 $\mathbf{1}[t = 1, z_1 = j]$，權重為 $\log \pi_j$。CRF 可以加入「下一個字以 -ly 結尾」或「當前的字首字母大寫」這類特徵，它們和字詞本身重疊且相依。

6. **加權編輯距離**。修改 `edit_distance`，使相差 90 度的兩個方向替換成本為 1、相差 180 度為 2。重新計算鏈碼範例中的距離。排名會改變嗎？

提示

替換成本是方向碼之間的循環差 $\min(|a-b|, 4-|a-b|)$。有凸起的正方形只需要插入，所以距離仍是 2；請檢查長方形最便宜的編輯序列用到了哪些替換，以判斷它的距離是否改變。

參考文獻

1. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2009, Chapters 8 and 9. publisher page
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006, Sections 8.1–8.4 and 13.1–13.3. official book page and PDF

3. J. P. Lewis, "Fast Normalized Cross-Correlation," *Vision Interface*, 1995. paper
4. N. S. Hashemi, R. B. Aghdam, A. S. B. Ghiasi, and P. Fatemi, "Template Matching Advances and Applications in Image Analysis," arXiv:1610.07231, 2016. arXiv
5. H. Sakoe and S. Chiba, "Dynamic Programming Algorithm Optimization for Spoken Word Recognition," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 26, no. 1, pp. 43–49, 1978. doi:10.1109/TASSP.1978.1163055
6. T. Rakthanmanon, B. Campana, A. Mueen, G. Batista, B. Westover, Q. Zhu, J. Zakaria, and E. Keogh, "Searching and Mining Trillions of Time Series Subsequences under Dynamic Time Warping," *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 262–270, 2012. doi:10.1145/2339530.2339576
7. R. A. Wagner and M. J. Fischer, "The String-to-String Correction Problem," *Journal of the ACM*, vol. 21, no. 1, pp. 168–173, 1974. doi:10.1145/321796.321811
8. D. Conte, P. Foggia, C. Sansone, and M. Vento, "Thirty Years of Graph Matching in Pattern Recognition," *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 18, no. 3, pp. 265–298, 2004. doi:10.1142/S0218001404003228
9. P. Foggia, G. Percannella, and M. Vento, "Graph Matching and Learning in Pattern Recognition in the Last 10 Years," *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 28, no. 1, 1450001, 2014. doi:10.1142/S0218001414500013
10. L. R. Rabiner, "A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition," *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, 1989. doi:10.1109/5.18626
11. A. J. Viterbi, "Error Bounds for Convolutional Codes and an Asymptotically Optimum Decoding Algorithm," *IEEE Transactions on Information Theory*, vol. 13, no. 2, pp. 260–269, 1967. doi:10.1109/TIT.1967.1054010
12. L. E. Baum, T. Petrie, G. Soules, and N. Weiss, "A Maximization Technique Occurring in the Statistical Analysis of Probabilistic Functions of Markov Chains," *The Annals of Mathematical Statistics*, vol. 41, no. 1, pp. 164–171, 1970. doi:10.1214/aoms/1177697196
13. R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960. doi:10.1115/1.3662552
14. J. D. Lafferty, A. McCallum, and F. C. N. Pereira, "Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data," *Proceedings of the 18th International Conference on Machine Learning (ICML)*, pp. 282–289, 2001. paper page
15. C. Sutton and A. McCallum, "An Introduction to Conditional Random Fields," *Foundations and Trends in Machine Learning*, vol. 4, no. 4, pp. 267–373, 2012. doi:10.1561/2200000013
16. A. Bagnall, J. Lines, A. Bostrom, J. Large, and E. Keogh, "The Great Time Series Classification Bake Off: A Review and Experimental Evaluation of Recent Algorithmic Advances," *Data Mining and Knowledge Discovery*, vol. 31, pp. 606–660, 2017. doi:10.1007/s10618-016-0483-9
17. M. Middlehurst, P. Schäfer, and A. Bagnall, "Bake Off Redux: A Review and Experimental Evaluation of Recent Time Series Classification Algorithms," *Data Mining and Knowledge Discovery*, vol. 38, pp. 1958–2031, 2024. doi:10.1007/s10618-024-01022-1
18. H. Ismail Fawaz, G. Forestier, J. Weber, L. Idoumghar, and P.-A. Muller, "Deep Learning for Time Series Classification: A Review," *Data Mining and Knowledge Discovery*, vol. 33, pp. 917–963, 2019. doi:10.1007/s10618-019-00619-1
19. M. Cuturi and M. Blondel, "Soft-DTW: A Differentiable Loss Function for Time-Series," *ICML*, 2017. arXiv:1703.01541

20. A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks," *Proceedings of the 23rd International Conference on Machine Learning* (*ICML*), pp. 369–376, 2006. doi:10.1145/1143844.1143891
21. P. Krähenbühl and V. Koltun, "Efficient Inference in Fully Connected CRFs with Gaussian Edge Potentials," *NIPS*, 2011. arXiv:1210.5644
22. S. Zheng, S. Jayasumana, B. Romera-Paredes, V. Vineet, Z. Su, D. Du, C. Huang, and P. H. S. Torr, "Conditional Random Fields as Recurrent Neural Networks," *ICCV*, 2015. arXiv:1502.03240
23. G. Lample, M. Ballesteros, S. Subramanian, K. Kawakami, and C. Dyer, "Neural Architectures for Named Entity Recognition," *NAACL*, 2016. arXiv:1603.01360
24. A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," arXiv:2312.00752, 2023. arXiv

PR 07・分群、混合模型與 EM

以 k-means 與 k-means++、階層式連結、DBSCAN 與譜分群為無標籤資料分組；推導高斯混合模型的 EM；用 BIC 與有效性指標選擇群數。

網站版本: https://blog.shyandram.dev/zh-tw/learn/pattern-recognition/pr-07-clustering-mixtures/

你將學到

- 分群演算法實際上在最佳化什麼，以及為什麼特徵、尺度與距離的選擇，在任何演算法執行之前就已決定了答案。
- Lloyd 的 k-means 如何運作、為何一定收斂卻只到局部最佳，以及 k-means++ 播種如何給出可證明的 $O(\log K)$ 保證。
- 階層式分群（single、complete、average 與 Ward 連結）、DBSCAN 與譜分群各自如何定義「一群」。
- 如何從頭推導高斯混合模型期望值最大化（expectation–maximization, EM）的 E 步與 M 步、為何 EM 不會讓似然下降，以及它會在哪裡失敗。
- 如何用 BIC 與 AIC 選擇群數，以及如何解讀內部指標（silhouette、Davies–Bouldin、Calinski–Harabasz）、外部指標（ARI、NMI）與穩定度。

先看全貌

到目前為止，這個系列中的每一筆訓練樣本都附有標籤。分群（clustering）把標籤拿掉，問的是：資料裡有沒有自然的群組？這個問題的答案從來不只由資料本身決定。同一批影像可以依物體、依顏色、依背景，或依拍攝的相機來分組；每一種分法在它自己的相似性定義下都是「對的」。因此，一個分群演算法其實是兩件事：精確說明我們指的是哪一種相似性，再加上一個在該定義下尋找最佳分組的搜尋程序。

> **用最簡單的話說**
>
> 把一大盒鈕扣倒在地上，你想把它們整理成幾堆。你可以按顏色、按大小，或按扣眼數量來分。每一種分法都很整齊，但哪一種有用，要看你接下來要做什麼。分群方法就是一套「堆鈕扣的規則」。**k-means** 說：「選幾個堆的中心，每顆鈕扣放到最近的那一堆。」**DBSCAN** 說：「擠在一起的鈕扣屬於同一堆，孤零零的就當作剩下的。」**高斯混合模型**則說：「每一堆都是一團模糊的雲，一顆鈕扣可以有 70% 把握屬於這團、30% 屬於那團。」

為什麼這對辨識很重要：分群可以建立視覺詞彙（把局部影像描述子用 k-means 量化成「視覺詞」）、初始化與壓縮模型（向量量化、調色盤）、在已標註的類別中找出子類別、找出離群值、分割影像（像素分群，見IP 10），而在現代自監督學習中，還能產生用來訓練深度網路的偽標籤。高斯混合模型也是**潛在變數模型**（latent-variable model）的標準範例，而 EM 是我們在PR 08 會遇到的許多推論演算法的原型。

本章依循 Theodoridis 與 Koutroumbas 書中的分群部分（第 11–16 章）[1]，以及 Bishop 第 9 章的混合模型與 EM [2]。我們會整理主要的演算法家族，完整推導其中最重要的兩個，最後討論如何判斷一個分群結果好不好。

相似性、尺度與「群」的意義

白話版。在選演算法之前，先決定什麼叫做「近」。換了特徵或單位，群就會跟著改變。

精確版。大多數方法都從特徵向量 $x, x' \in \mathbb{R}^D$ 之間的相異度 $d(x, x')$ 出發。常見選擇有

$$d_2(x,x') = \|x - x'\|_2, \qquad d_M(x,x') = \sqrt{(x-x')^\top S^{-1}(x-x')}, \qquad d_{\cos}(x,x') = 1 - \frac{x^\top x'}{\|x\|\,\|x'\|},$$

其中 d_2 是歐氏距離；d_M 是以正定矩陣 S（通常是共變異數估計）定義的馬氏距離（Mahalanobis distance）；$d_{\cos}$ 是餘弦相異度，它忽略向量長度，常用於深度網路的嵌入向量（embedding）。

由此有三個實務上的後果。

1. **尺度是一個建模決策**。若一個座標以公釐計、另一個以公里計，歐氏距離就會被前者主導。把每個特徵標準化為單位變異數可以消除這個問題，但也會抹掉真正有意義的離散程度差異。做分類時可以交給驗證集決定；分群沒有標籤，這個選擇只能來自領域知識。
2. **硬指派與軟指派**。硬分割把每個點 x_n 指派到恰好一個群 $z_n \in \{1, \dots, K\}$。軟指派則保存隸屬程度 $r_{nk} \geq 0$，且 $\sum_k r_{nk} = 1$。在高斯混合模型中，這些是後驗機率；在模糊分群 [1] 中，它們是由目標函數定義的隸屬度，不應被解讀為機率。
3. **群不是類別**。群的編號是事後任意附上的。這一次的「第 3 群」下一次可能變成「第 1 群」，因此任何與類別標籤的比較都必須對排列不變，而這正是後面介紹的 ARI 與 NMI 的性質。

Theodoridis 與 Koutroumbas 把演算法分成序列式、階層式、以目標函數為基礎（k-means、混合模型、模糊分群），以及以圖或密度為基礎等家族 [1]。我們大致依照這個順序。

K-means 與 Lloyd 演算法

白話版。選 K 個中心。重複兩個步驟：把每個點交給最近的中心，再把每個中心移到它所屬點的平均位置。直到沒有任何變化為止。

目標函數

令 one-hot 指標 $z_{nk} \in \{0, 1\}$（若第 n 點屬於第 k 群則 $z_{nk} = 1$，且 $\sum_k z_{nk} = 1$），中心為 $\mu_1, \dots, \mu_K \in \mathbb{R}^D$。k-means 最小化群內平方和

$$J(z, \mu) = \sum_{n=1}^{N} \sum_{k=1}^{K} z_{nk} \, \|x_n - \mu_k\|^2.$$

其中 N 是點的數量，D 是維度。J 也稱為 *inertia* 或失真（distortion）。

兩個交替步驟

Lloyd 演算法 [3]（原本是為脈衝碼調變中的訊號量化而發展）以**區塊座標下降**（block coordinate descent）最小化 J：

- **指派步驟**。固定 μ。每一項只牽涉一個 z_n，因此對 $k = \arg\min_j \|x_n - \mu_j\|^2$ 設 $z_{nk} = 1$。
- **更新步驟**。固定 z。J 對每個 μ_k 都是凸二次函數；令梯度為零，

$$\frac{\partial J}{\partial \mu_k} = -2 \sum_n z_{nk}(x_n - \mu_k) = 0 \quad \Longrightarrow \quad \mu_k = \frac{\sum_n z_{nk} x_n}{\sum_n z_{nk}},$$

也就是目前屬於第 k 群各點的平均。「k-means」這個名稱由 MacQueen [4] 提出；他的版本在每個點到來後立即更新中心：$\mu_k \leftarrow \mu_k + (x_n - \mu_k)/N_k$，其中 N_k 是第 k 群到目前為止的點數。

為什麼會收斂，又為什麼只是局部

每一步只會讓 J 下降或維持不變：指派步驟為目前的 μ 挑出最佳的 z，更新步驟為目前的 z 挑出最佳的 μ。把 N 個點分成 K 組的分割方式是有限的，而且（只要平手時採用固定規則）演算法不會回到 J 更高的分割，因此在有限次迭代後一定停止。停下來的固定點是**局部**最小值：任何單一步驟都無法再改善的分割。不同的初始中心可能導致差異很大的固定點。

實作上有兩個細節。**空群**（沒有任何點被指派）沒有平均值；常見作法是保留舊中心，或在目前誤差最大的點重新播種。另外每次迭代需要 $O(NKD)$ 次距離計算，這促成了後面的小批次版本。

k-means 的假設

由於目標是各點到中心的歐氏距離平方和，兩群之間的邊界就是兩個中心的垂直平分線：k-means 把空間切成 **Voronoi 區域**。它適合緊密、大致呈球形且大小相近的群；它會切開細長的群、把靠近大群的小群併進去，也無法沿著彎曲形狀分群（圖 1，中間）。

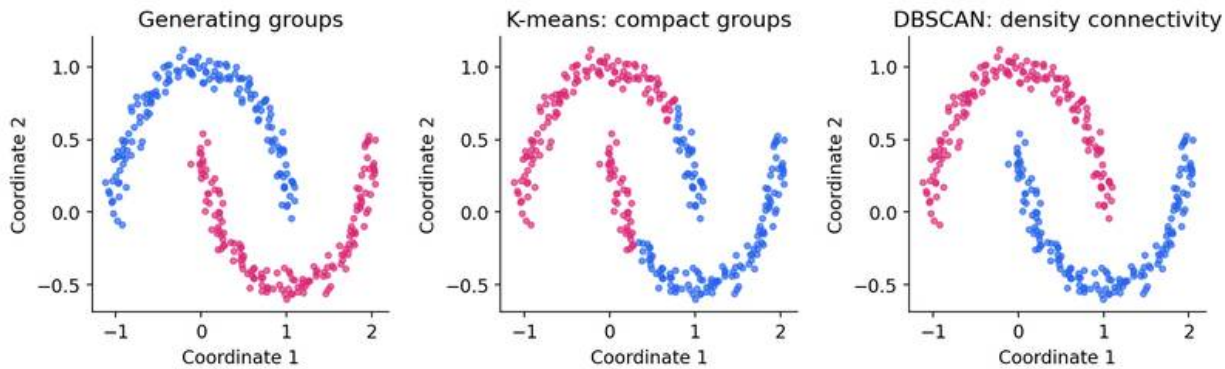


圖 *1* — 同一份雙月形資料，依生成標籤（左）、*k-means*（中）與 *DBSCAN*（右）分組。*k-means* 與 *DBSCAN* 都在沒有標籤的情況下擬合；顏色只是任意的群編號。

k-means++ 播種

白話版。先隨機選一個中心，之後每個新中心都盡量選離現有中心遠的點，但保留隨機性，免得離群值每次都被選中。

Arthur 與 Vassilvitskii [5] 提出 **D² 播種**：

1. 從資料中均勻隨機選出第一個中心。
2. 對每個點計算 $D(x_n)$，即它到目前已選中心中最近者的距離。
3. 以機率 $D(x_n)^2 / \sum_m D(x_m)^2$ 選出 x_n 作為下一個中心。
4. 重複步驟 2–3 直到選滿 K 個中心，再執行 Lloyd 演算法。

他們的主要定理指出，光是播種本身，在期望值意義下就已達到

$$\mathbb{E}[J] \le 8(\ln K + 2)\, J_{\mathrm{OPT}},$$

其中 J_{OPT} 是最佳的 k-means 目標值；之後的 Lloyd 迭代只會讓 J 更低。均勻隨機播種沒有這樣的保證。這個上界是最壞情況的敘述；實務上的好處主要表現在災難性的結果變少了。函式庫會把 k-means++ 與多次重新啟動結合（scikit-learn 的 `n_init`），保留 J 最低的一次。

程式：從頭實作 k-means++ 與 Lloyd

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

def kmeans_pp_init(X, k, rng):
    """D^2 seeding: each new center is drawn with probability proportional
    to the squared distance to the nearest center chosen so far."""
    centers = [X[rng.integers(len(X))]]
    for _ in range(k - 1):
        d2 = np.min(((X[:, None, :] - np.array(centers)[None]) ** 2).sum(-1), axis=1)
        centers.append(X[rng.choice(len(X), p=d2 / d2.sum())])
    return np.array(centers)

def lloyd(X, centers, max_iter=100):
    history = []
    for _ in range(max_iter):
        d2 = ((X[:, None, :] - centers[None]) ** 2).sum(-1)   # N x K
        z = d2.argmin(axis=1)                                 # assignment step
        history.append(d2[np.arange(len(X)), z].sum())        # objective J
        new = np.array([X[z == k].mean(axis=0) if np.any(z == k) else centers[k]
                        for k in range(len(centers))])        # update step
        if np.allclose(new, centers):
            break
        centers = new
    return centers, z, history

X, _ = make_blobs(n_samples=600, centers=4, cluster_std=1.8, random_state=7)
rng = np.random.default_rng(0)
best = None
```

```
for restart in range(10):                       # several seeds, keep the best J
    c, z, hist = lloyd(X, kmeans_pp_init(X, 4, rng))
    if best is None or hist[-1] < best[2][-1]:
        best = (c, z, hist)
c, z, hist = best
print("J per iteration:", np.round(hist, 1))
assert all(a >= b - 1e-9 for a, b in zip(hist, hist[1:]))   # monotone
ref = KMeans(n_clusters=4, n_init=10, random_state=0).fit(X)
print("from scratch J:", round(hist[-1], 2), " sklearn inertia:", round(ref.inertia_, 2))

# single runs (no restarts): uniform random seeding vs D^2 seeding, 15 clusters
X2, _ = make_blobs(n_samples=1500, centers=15, cluster_std=0.6, random_state=2)
for name, init in [("random", lambda r: X2[r.choice(len(X2), 15, replace=False)]),
                   ("k-means++", lambda r: kmeans_pp_init(X2, 15, r))]:
    finals = [lloyd(X2, init(np.random.default_rng(s)))[2][-1] for s in range(20)]
    print(f"{name:9s} final J over 20 runs: median {np.median(finals):7.1f}, "
          f"worst {np.max(finals):7.1f}")
```

輸出（NumPy 2.0、scikit-learn 1.6）：

```
J per iteration: [5865.6 3633.4 3625. ]
from scratch J: 3625.0  sklearn inertia: 3625.0
random    final J over 20 runs: median  1305.1, worst  4750.8
k-means++ final J over 20 runs: median  1147.5, worst  1525.2
```

從頭寫的版本與 scikit-learn 達到相同的目標值，而且 J 單調下降。在 15 群的問題上，均勻播種有時會把兩個種子放進同一團、另一團卻一個也沒有；Lloyd 演算法無法修正這種情況，最差的一次大約差了三倍。圖 2 顯示兩種播種各自最差的一次。

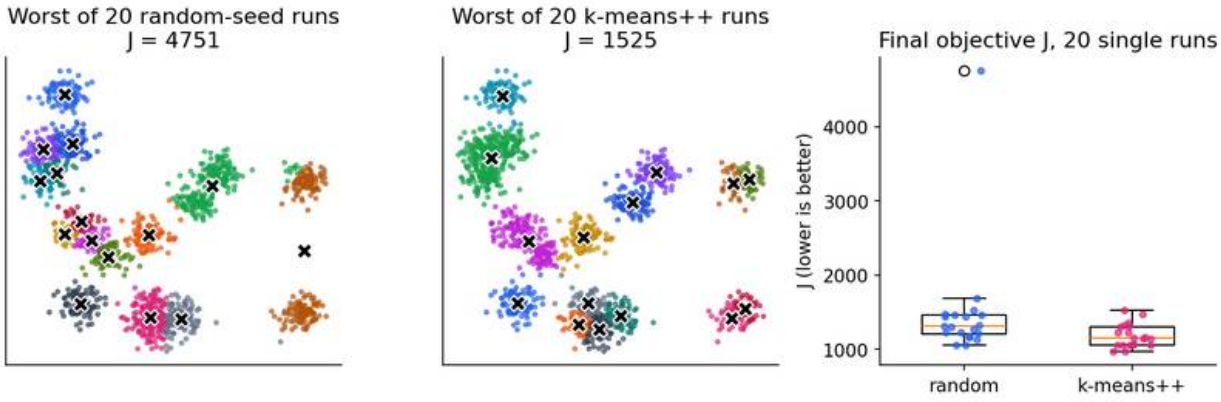


圖 2 — 播種很重要。*15* 團合成資料，每種播種規則各執行 *20* 次單一 *Lloyd*（不重新啟動）。隨機播種會讓某些執行卡在合併與切開的團；D^2（*k-means++*）播種的目標值較低、變異也較小。黑色叉號為最終中心。

小批次 k-means

面對數百萬個點時，Sculley 的小批次 k-means [6] 用小的隨機批次來更新中心：每個中心朝被指派給它的批次點移動，每個中心的學習率依 1/(已見過的點數) 遞減。它以略高的最終目標值換取大幅加速，在 scikit-learn 中為 `MiniBatchKMeans`，是建立大型視覺詞彙時的常見選擇。

階層式分群

白話版。一開始每個點自成一群，接著反覆把最近的兩群合併。合併紀錄形成一棵樹；在某個高度切開它，就得到各群。

聚合式演算法與連結方式

聚合式（agglomerative）分群需要定義兩個群 A 與 B 之間的距離。四種經典的**連結**（linkage）為

$$
\begin{aligned}
d_{\text{single}}(A,B) &= \min_{a\in A,\,b\in B} d(a,b),\\
d_{\text{complete}}(A,B) &= \max_{a\in A,\,b\in B} d(a,b),\\
d_{\text{average}}(A,B) &= \frac{1}{|A|\,|B|}\sum_{a\in A}\sum_{b\in B} d(a,b),\\
\Delta_{\text{Ward}}(A,B) &= \frac{|A|\,|B|}{|A|+|B|}\,\|\bar{a}-\bar{b}\|^2,
\end{aligned}
$$

其中 $|A|$ 是 A 中的點數，$\bar{a},\bar{b}$ 是兩群的平均。Ward 準則 [7] 是合併 A 與 B 所造成的群內平方和 J 的增量，所以 Ward 聚合是一種由下而上、貪婪地讓 J 保持很小的方法，目標與 k-means 相同。

每種連結都有自己的個性（圖 3）：

- **Single linkage** 只要任何兩點夠近就合併。它能沿著任意形狀走，但一條細細的點「橋」就可能把兩群串在一起。它等價於切斷最小生成樹（minimum spanning tree）中最長的邊，這也是 [1] 中的圖論方法之一。
- **Complete linkage** 要求所有點對都夠近，會產生緊密且直徑相近的群，對離群值敏感。
- **Average linkage** 介於兩者之間。
- **Ward** 產生緊密、大小相近、大致呈球形的群，很像 k-means。

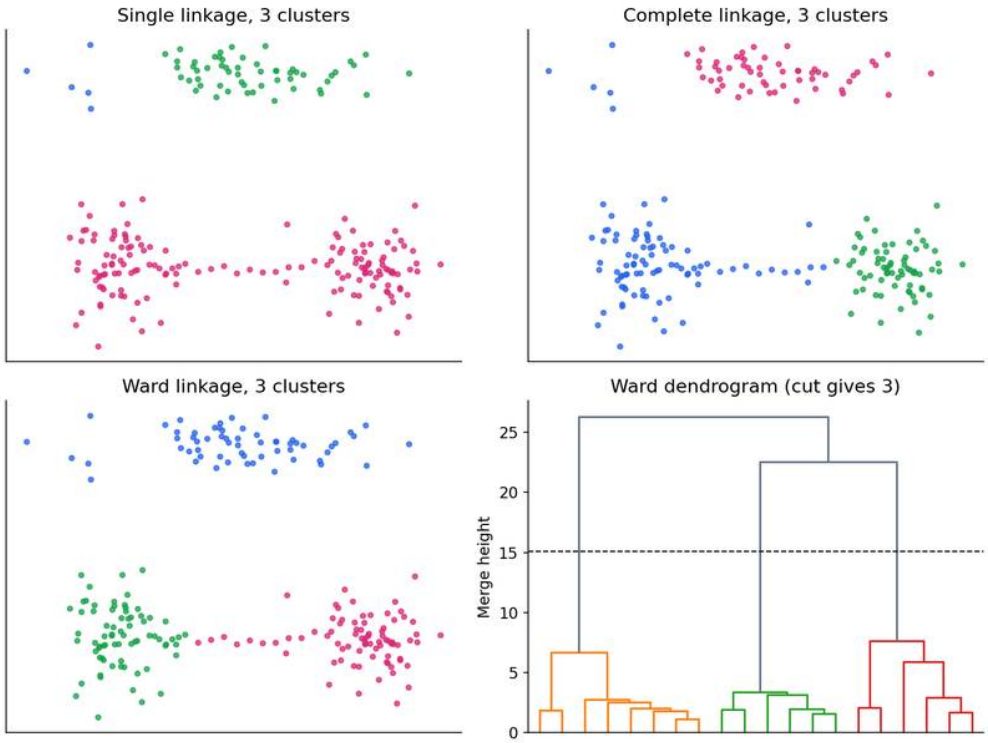


圖 3 — 同一份資料、三種連結。*Single linkage* 沿著細橋串接，把兩團併成一群，並把一群浪費在四個零散點上；*complete* 與 *Ward* 連結則讓兩團保持分開，把細橋分給兩側。右下：*Ward* 樹狀圖，在虛線處切開得到三群。

樹狀圖

合併歷程是一棵二元樹，稱為**樹狀圖**（dendrogram）；每個接點的高度就是合併當下的連結值。在高度 h 切開樹，就得到該層的分割；相鄰合併高度之間若有很大的落差，往往暗示一個自然的群數。不過切割高度仍然是一個選擇。

實務上有兩個限制：合併一旦發生就不會撤銷，早期的錯誤會一路傳下去；標準演算法需要儲存所有點對距離，記憶體為 $O(N^2)$，使它只適用於數萬個點以內。分裂式（由上而下）階層方法也存在，但較少使用。

以密度為基礎的分群：DBSCAN

白話版。一群就是一塊擁擠的區域。擁擠處的點讓群長大；空曠處的點被標為雜訊。

DBSCAN（Density-Based Spatial Clustering of Applications with Noise）[8] 有兩個參數：半徑 ε 與點數 MinPts。令 $N_\varepsilon(x) = \{x' : d(x, x') \le \varepsilon\}$ 為 x 的 ε 鄰域（包含 x 本身）。則

- 若 $|N_\varepsilon(x)| \ge$ MinPts，x 是**核心點**（core point）；
- 若 x 不是核心點，但落在某個核心點的 ε 鄰域內，它是**邊界點**（border point）；

- 其餘的點都是**雜訊**（noise）。

若存在一串核心點，相鄰兩者距離都不超過 ε，最後抵達某點的鄰域，則該點從起始核心點密度可達。一群就是彼此密度相連的點所構成的極大集合。DBSCAN 能找出任意形狀的群、不需要事先指定 K，而且明確拒絕指派孤立的點。

如何選 ε。原論文建議使用排序後的 **k 距離圖**：對每個點計算它到第 k 個最近鄰的距離（$k = \text{MinPts}$），排序後在曲線急遽上升的「膝點」附近選 ε（圖 4 左）。圖中膝點左側那些距離很大的點，就是雜訊的候選。scikit-learn 的 `min_samples` 會把點本身算進去，與上面的定義一致。

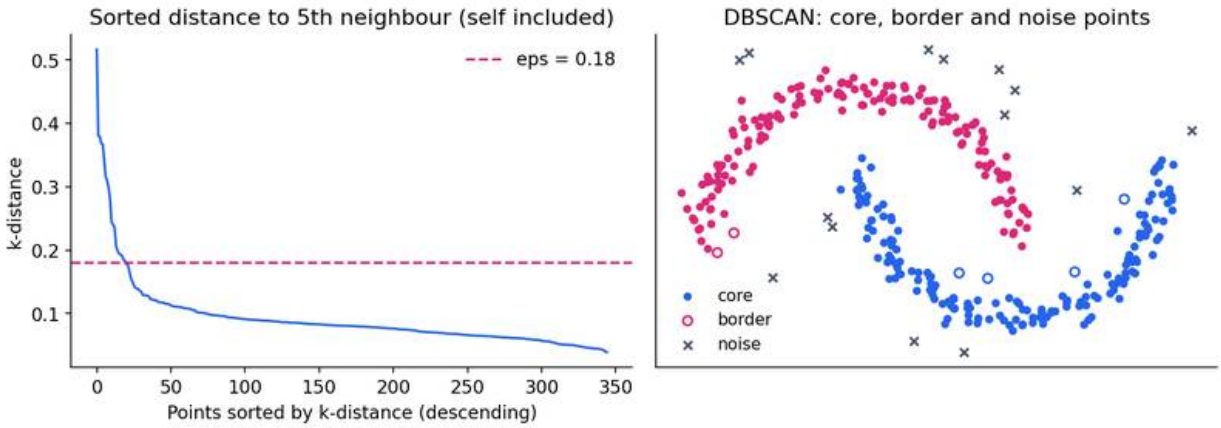


圖 *4* — 在雙月形資料加上 *25* 個均勻離群點上執行 *DBSCAN*。左：排序後的第 *5* 近鄰距離曲線；所選的 $\varepsilon = 0.18$ 位於曲線開始上升之處。右：核心點（實心）、邊界點（空心）以及被判為雜訊的點（叉號）。

限制。單一 ε 意味著單一密度水準。若一群很密、另一群很稀，就沒有任何 ε 能同時把兩者分開。在高維空間中距離會趨於集中，鄰域點數失去對比。**HDBSCAN** [9] 針對第一個問題，建立涵蓋所有密度水準的階層，並取出最穩定的群，以「最小群大小」取代 ε；scikit-learn 中提供 `HDBSCAN`。

一頁看懂譜分群

白話版。建一張把相近點連起來的圖，再找出切斷最少強連結的切法。

給定親和矩陣 W（$W_{ij} \geq 0$，相似的點數值大，例如 k 近鄰圖或 $W_{ij} = \exp(-\|x_i - x_j\|^2 / 2\sigma^2)$）與對角度數矩陣 $D_{ii} = \sum_j W_{ij}$，一種正規化圖拉普拉斯矩陣為

$$L_{\text{sym}} = I - D^{-1/2} W D^{-1/2}.$$

若圖有 K 個連通分量，L_{sym} 的特徵值 0 重數為 K，其特徵向量指出各分量。若圖接近不連通，最小 K 個特徵值對應的特徵向量仍近似指出各群。譜分群把這些特徵向量排成矩陣的行、把每一列正規化，再對各列執行 k-means [10]。結果可以沿著在原空間中會打敗 k-means 的彎曲形狀走，因為特徵向量衡量的是連通性，而不是到某個中心的距離。

圖本身就是模型的一部分。鄰居數或頻寬 σ 都必須選擇；一張意外不連通的圖（例如有一個孤立節點）會產生一個主導結果的平凡特徵向量。von Luxburg 的教學文章 [10] 解釋了圖切割的詮釋（ratio cut 與 normalized cut），也提供如何選圖的實務建議。

程式：同一種幾何，不同的假設

下面的雙月形資料是帶標籤生成的，但標籤只在事後用來以調整蘭德指數（adjusted Rand index, ARI；1 表示在重新編號下完全一致，約 0 表示與隨機相當）評分。

```
import numpy as np
from sklearn.datasets import make_moons
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import (KMeans, AgglomerativeClustering, DBSCAN,
                             HDBSCAN, SpectralClustering)
from sklearn.mixture import GaussianMixture
from sklearn.metrics import adjusted_rand_score

X, truth = make_moons(n_samples=320, noise=0.06, random_state=5)

# k-distance curve for choosing DBSCAN's eps (k = min_samples)
dist, _ = NearestNeighbors(n_neighbors=5).fit(X).kneighbors(X)
kdist = np.sort(dist[:, -1])                    # 5th neighbour, counting the point itself
print("k-distance quantiles 50/90/99%:", np.round(np.quantile(kdist, [0.5, 0.9, 0.99]), 3))

models = {
    "k-means": KMeans(n_clusters=2, n_init=10, random_state=5),
    "single linkage": AgglomerativeClustering(n_clusters=2, linkage="single"),
    "Ward linkage": AgglomerativeClustering(n_clusters=2, linkage="ward"),
    "Gaussian mixture": GaussianMixture(n_components=2, n_init=3, random_state=5),
    "DBSCAN": DBSCAN(eps=0.18, min_samples=5),
    "HDBSCAN": HDBSCAN(min_cluster_size=15),
    "spectral": SpectralClustering(n_clusters=2, affinity="nearest_neighbors",
                                   n_neighbors=20, assign_labels="kmeans",
                                   random_state=5),
}
for name, model in models.items():
    labels = model.fit_predict(X)
    print(f"{name:17s} ARI = {adjusted_rand_score(truth, labels):.3f}"
          f"   noise points = {int((labels == -1).sum())}")
```

```
k-distance quantiles 50/90/99%: [0.079 0.11  0.144]
k-means           ARI = 0.260   noise points = 0
single linkage    ARI = 1.000   noise points = 0
Ward linkage      ARI = 0.242   noise points = 0
Gaussian mixture  ARI = 0.488   noise points = 0
DBSCAN            ARI = 1.000   noise points = 0
HDBSCAN           ARI = 0.988   noise points = 2
spectral          ARI = 1.000   noise points = 0
```

圖 1 顯示其中兩個結果。以中心或橢圓為基礎的方法（k-means、Ward、兩個成分的高斯混合）把每個月形切成兩半；以連通性為基礎的方法（single linkage、DBSCAN、HDBSCAN、譜分群）則找回兩個月形。所選的 $\varepsilon = 0.18$ 高於 k 距離的第 99 百分位數，因此沒有任何月形上的點被當成雜訊。鄰居太少的 k 近鄰圖可能不連通，此時 scikit-learn 會警告譜嵌入可能不可靠；這裡用 20 個鄰居就避開了這個問題。但這並不代表以連通性為基礎的方法「比較好」：在相互重疊的高斯團上，single linkage 會把一切串在一起，而 k-means 勝出。比較方法時必須改變幾何、密度、雜訊與維度。

高斯混合模型與 EM 演算法

白話版。假裝每個點的產生方式是：先擲一顆 K 面骰子選一團雲，再從那團高斯雲中取樣。我們永遠看不到骰子。EM 會先為每個點猜測各團雲產生它的可能性（E 步），再用這些猜測當作權重重新擬合每團雲（M 步），然後重複。

模型

具有 K 個成分的高斯混合模型（Gaussian mixture model, GMM）的密度為

$$p(x \mid \theta) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}(x \mid \mu_k, \Sigma_k), \qquad \pi_k \geq 0, \quad \sum_{k=1}^{K} \pi_k = 1,$$

其中 π_k 是**混合權重**（成分 k 的先驗機率），$\mu_k \in \mathbb{R}^D$ 是其平均，Σ_k 是其 $D \times D$ 共變異數矩陣，$\theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1}^{K}$ 收集所有參數，且

$$\mathcal{N}(x \mid \mu, \Sigma) = \frac{1}{(2\pi)^{D/2} |\Sigma|^{1/2}} \exp\Big(-\tfrac{1}{2} (x-\mu)^{\top} \Sigma^{-1} (x-\mu) \Big).$$

引入一個**潛在**的 one-hot 變數 z_n，令 $p(z_{nk} = 1) = \pi_k$、$p(x_n \mid z_{nk} = 1) = \mathcal{N}(x_n \mid \mu_k, \Sigma_k)$。把 z_n 加總消去，就回到混合密度。給定資料 $X = \{x_1, \dots, x_N\}$，對數似然為

$$\ln p(X \mid \theta) = \sum_{n=1}^{N} \ln \sum_{k=1}^{K} \pi_k \, \mathcal{N}(x_n \mid \mu_k, \Sigma_k).$$

「和的對數」把所有參數綁在一起，令梯度為零沒有封閉解。EM 是繞過這個困難的標準方法 [11]、[2]。

步驟 1：完整資料對數似然

如果我們知道指派 $Z = \{z_n\}$，(X, Z) 的對數似然就是

$$\ln p(X, Z \mid \theta) = \sum_{n=1}^{N} \sum_{k=1}^{K} z_{nk} \Big[\ln \pi_k + \ln \mathcal{N}(x_n \mid \mu_k, \Sigma_k) \Big].$$

現在對數只作用在單一高斯上，問題拆成 K 個各自獨立的加權高斯擬合：很容易。

步驟 2（E 步）：完整資料對數似然的期望值

我們不知道 Z，但在目前參數 θ^{old} 下，我們知道它的後驗。由貝氏定理，

$$r_{nk} \equiv p(z_{nk}=1 \mid x_n, \theta^{\text{old}}) = \frac{\pi_k \, \mathcal{N}(x_n \mid \mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \, \mathcal{N}(x_n \mid \mu_j, \Sigma_j)},$$

在 θ^{old} 下計算。r_{nk} 是成分 k 對點 n 的**責任值**（responsibility），每一列加總為 1。由於完整資料對數似然對 z_{nk} 是線性的，在這個後驗下取期望，只要把 z_{nk} 換成 r_{nk}：

$$Q(\theta, \theta^{\text{old}}) = \sum_{n}\sum_{k} r_{nk}\Big[\ln \pi_k - \tfrac{D}{2}\ln 2\pi - \tfrac{1}{2}\ln|\Sigma_k| - \tfrac{1}{2}(x_n-\mu_k)^{\top}\Sigma_k^{-1}(x_n-\mu_k)\Big].$$

步驟 3（M 步）：以封閉形式最大化 Q

定義**有效點數** $N_k = \sum_n r_{nk}$。

混合權重。只有 $\sum_n \sum_k r_{nk}\ln\pi_k$ 與 π 有關，且受限於 $\sum_k \pi_k = 1$。引入拉格朗日乘數 λ，

$$\frac{\partial}{\partial \pi_k}\Big[\sum_{n,k} r_{nk}\ln\pi_k + \lambda\Big(\sum_k \pi_k - 1\Big)\Big] = \frac{N_k}{\pi_k} + \lambda = 0 \;\Rightarrow\; \pi_k = -\frac{N_k}{\lambda}.$$

對 k 加總得 $\lambda = -N$，所以 $\pi_k = N_k/N$。（限制 $\pi_k \geq 0$ 自動成立。）

平均。對 μ_k 的梯度為 $\sum_n r_{nk}\Sigma_k^{-1}(x_n-\mu_k)$。令其為零並乘上 Σ_k，

$$\mu_k = \frac{1}{N_k}\sum_n r_{nk}\, x_n.$$

共變異數。令 $\Lambda_k = \Sigma_k^{-1}$，則 $-\tfrac{1}{2}\ln|\Sigma_k| = \tfrac{1}{2}\ln|\Lambda_k|$，二次項為 $-\tfrac{1}{2}\operatorname{tr}\big(\Lambda_k(x_n-\mu_k)(x_n-\mu_k)^{\top}\big)$。利用 $\partial \ln|\Lambda|/\partial\Lambda = \Lambda^{-1}$，以及對對稱矩陣 A 有 $\partial \operatorname{tr}(\Lambda A)/\partial\Lambda = A^{\top}$，

$$\frac{\partial Q}{\partial \Lambda_k} = \frac{1}{2}\sum_n r_{nk}\Big[\Sigma_k - (x_n-\mu_k)(x_n-\mu_k)^{\top}\Big] = 0 \;\Rightarrow\; \Sigma_k = \frac{1}{N_k}\sum_n r_{nk}(x_n-\mu_k)(x_n-\mu_k)^{\top}.$$

這些正是單一高斯熟悉的最大似然公式（見PR 02），只是每個點都依其責任值加權。EM 交替執行 E 步與 M 步，直到對數似然不再改善（圖 5）。

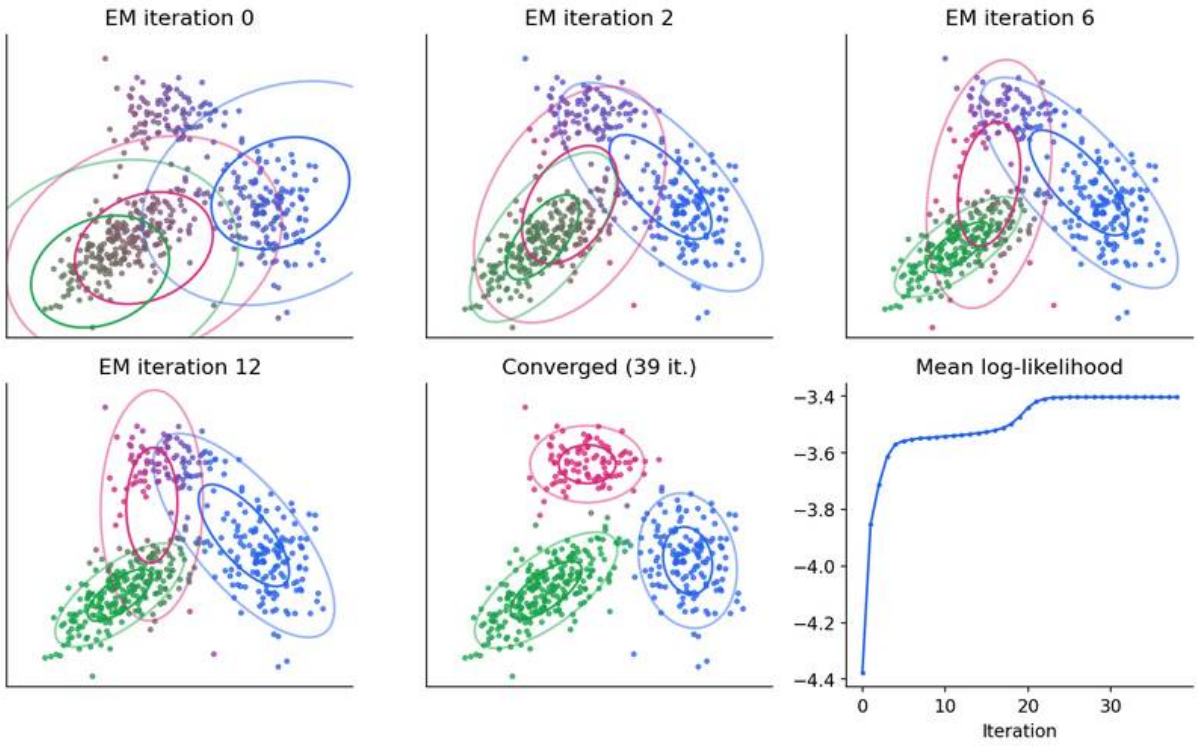


圖 5 — 三成分高斯混合的 *EM*。各成分從隨機資料點出發、使用整體共變異數；點的顏色依責任值混合各成分的顏色。右下：平均對數似然從不下降。

為什麼 EM 不會讓似然下降

對潛在指派上的任何分布 $q(Z)$，

$$\ln p(X \mid \theta) = \underbrace{\sum_{Z} q(Z) \ln \frac{p(X, Z \mid \theta)}{q(Z)}}_{\mathcal{L}(q,\theta)} + \underbrace{\sum_{Z} q(Z) \ln \frac{q(Z)}{p(Z \mid X, \theta)}}_{\mathrm{KL}(q \,\|\, p(Z \mid X,\theta)) \,\geq\, 0}.$$

因此 $\mathcal{L}(q,\theta)$ 是對數似然的下界，稱為**證據下界**（evidence lower bound, ELBO）。E 步令 $q(Z) = p(Z \mid X, \theta^{\text{old}})$，使 KL 項為零、下界在 θ^{old} 處貼合。M 步對 θ 最大化 $\mathcal{L}$；由於 $\mathcal{L}(q,\theta) = Q(\theta, \theta^{\text{old}}) +$ 常數，這等同於最大化 Q。新的似然至少等於新的下界，新的下界至少等於舊的下界，而舊的下界等於舊的似然。因此 $\ln p(X \mid \theta^{\text{new}}) \geq \ln p(X \mid \theta^{\text{old}})$ [11]、[2]。當精確後驗難以計算時，我們把 q 限制在一個可處理的分布族中，就得到**變分推論**（variational inference），這是PR 08 的主題。

單調不代表全域：EM 收斂到一個取決於初始化的駐點。實務上的實作會先以 k-means 初始化，並多次重新啟動。

奇異解與如何避免

不受限 GMM 的似然是**無界**的。把某個成分的平均正好放在一個資料點上，$\mu_j = x_n$，並令 $\Sigma_j = \sigma^2 I$。這一項對 $p(x_n)$ 的貢獻為 $\pi_j (2\pi\sigma^2)^{-D/2}$，當 $\sigma \to 0$ 時趨於無限大，而其他成分仍讓其餘點的似然保持有限。因此最大似然估計並不存在；EM 找到的是一個避開崩塌的良好局部最大值。補救方法包括：在共變異數對角線加上一個小常數（scikit-learn 的 `reg_covar`）、限制共變異數形式（對角、球形、共用）、對參數加上先驗（MAP-EM 或貝氏混合模型），或重新啟動正在崩塌的成分。

可識別性與標籤交換

把成分的編號互換，密度不變，因此有 $K!$ 個等價的最大值。「成分 2」沒有固有意義，要把成分編號與類別標籤比較，必須先有一個配對規則。

k-means 是 EM 的極限

對所有 k 固定 $\pi_k = 1/K$ 與 $\Sigma_k = \epsilon I$。責任值變成

$$r_{nk} = \frac{\exp(-\|x_n - \mu_k\|^2 / 2\epsilon)}{\sum_j \exp(-\|x_n - \mu_j\|^2 / 2\epsilon)}.$$

當 $\epsilon \to 0$，距離最小的那一項主導，因此最近的中心 $r_{nk} \to 1$，其餘為 0：E 步變成 k-means 的指派步驟，μ_k 的 M 步變成 k-means 的更新 [2]。所以一般的 GMM 是在 k-means 上加入軟指派、不等大小（π_k）與橢圓形狀（Σ_k）的延伸。

程式：從頭實作 EM，並與 scikit-learn 對照

```
import numpy as np
from scipy.special import logsumexp
from scipy.stats import multivariate_normal
from sklearn.mixture import GaussianMixture

rng = np.random.default_rng(3)
true_means = np.array([[0.0, 0.0], [4.0, 1.0], [1.0, 4.0]])
true_covs = [np.array([[1.0, 0.6], [0.6, 0.8]]),
             np.array([[0.5, -0.2], [-0.2, 1.2]]),
             np.array([[0.7, 0.0], [0.0, 0.3]])]
counts = [250, 150, 100]
X = np.vstack([rng.multivariate_normal(m, S, n)
               for m, S, n in zip(true_means, true_covs, counts)])
N, D = X.shape
K, reg = 3, 1e-6                              # reg: small covariance floor

# initialise with K random points, shared data covariance, equal weights
mu = X[rng.choice(N, K, replace=False)]
Sigma = np.array([np.cov(X.T) for _ in range(K)])
```

```
pi = np.full(K, 1.0 / K)
loglik = []
for it in range(200):
    # E step: log of pi_k N(x_n | mu_k, Sigma_k), normalised with log-sum-exp
    log_joint = np.column_stack([np.log(pi[k]) +
                                 multivariate_normal(mu[k], Sigma[k]).logpdf(X)
                                 for k in range(K)])
    log_norm = logsumexp(log_joint, axis=1)
    R = np.exp(log_joint - log_norm[:, None])     # responsibilities r_nk
    loglik.append(log_norm.sum())
    # M step: closed-form weighted updates
    Nk = R.sum(axis=0)
    pi = Nk / N
    mu = (R.T @ X) / Nk[:, None]
    for k in range(K):
        diff = X - mu[k]
        Sigma[k] = (R[:, k, None] * diff).T @ diff / Nk[k] + reg * np.eye(D)
    if it > 0 and loglik[-1] - loglik[-2] < 1e-8:
        break

assert np.all(np.diff(loglik) > -1e-6)           # EM never decreases log-likelihood
print("iterations:", len(loglik), " final mean log-lik:", round(loglik[-1] / N, 4))
print("weights:", np.round(np.sort(pi), 3))
gm = GaussianMixture(3, covariance_type="full", n_init=5, random_state=0).fit(X)
print("sklearn mean log-lik:", round(gm.score(X), 4))
print("sklearn weights:", np.round(np.sort(gm.weights_), 3))
```

```
iterations: 40  final mean log-lik: -3.4032
weights: [0.202 0.299 0.499]
sklearn mean log-lik: -3.4033
sklearn weights: [0.203 0.3   0.497]
```

E 步在對數空間中計算：高維的密度值會向下溢位，而 `logsumexp` 能穩定地計算 $\ln \sum_k e^{a_k}$。估得的權重與生成比例 $0.5, 0.3, 0.2$ 及 scikit-learn 的解相符；微小差異來自 scikit-learn 自己的停止容忍度。

選擇群數

白話版。群越多，對訓練資料的擬合總是越好。模型選擇準則對每個多出來的參數收取代價，最佳的 K 在擬合與代價之間取得平衡。

混合模型的 BIC 與 AIC

對一個最大化似然為 $\hat{L}$、有 p 個自由參數、N 個資料點的模型，

$$\mathrm{BIC} = -2\ln \hat{L} + p \ln N, \qquad \mathrm{AIC} = -2\ln \hat{L} + 2p,$$

越小越好。貝氏資訊準則（Bayesian information criterion, BIC）[12] 來自對數邊際似然（模型證據，見PR 08）的大樣本近似；赤池資訊準則（Akaike information criterion, AIC）[13] 估計的是預期的預測對數損失。由於 $N \geq 8$ 時 $\ln N > 2$，BIC 對複雜度的懲罰較重，傾向選擇較少的成分。

對 D 維、完整共變異數的 GMM，

$$p = \underbrace{(K-1)}_{\text{權重}} + \underbrace{KD}_{\text{平均}} + \underbrace{K\,\frac{D(D+1)}{2}}_{\text{共變異數}}.$$

在高維時共變異數占了大部分：當 $D = 128$（類似 SIFT 的描述子），每個完整共變異數有 8,256 個自由數值，這就是為什麼大型描述子通常使用對角共變異數。因此共變異數的形式也是模型選擇的一部分。

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.mixture import GaussianMixture

X, _ = make_blobs(n_samples=600, centers=4, cluster_std=1.0, random_state=7)
X = X @ np.array([[1.0, 0.0], [0.6, 0.5]])        # shear: make clusters elliptical
results = []
for cov in ["spherical", "diag", "full"]:
    for K in range(1, 9):
        gm = GaussianMixture(K, covariance_type=cov, n_init=3, random_state=0).fit(X)
        results.append((gm.bic(X), gm.aic(X), cov, K))
for bic, aic, cov, K in sorted(results)[:5]:
    print(f"BIC={bic:8.1f}  AIC={aic:8.1f}  {cov:9s} K={K}")

# check the parameter count used by BIC for a full-covariance model
gm = GaussianMixture(4, covariance_type="full", random_state=0).fit(X)
D, K = X.shape[1], 4
p = (K - 1) + K * D + K * D * (D + 1) // 2
manual = -2 * gm.score(X) * len(X) + p * np.log(len(X))
print("free parameters:", p, " manual BIC:", round(manual, 1), " sklearn:", round(gm.bic(X),
 ↪  1))
```

```
BIC=  4301.1  AIC=  4200.0  full      K=4
BIC=  4320.7  AIC=  4193.2  full      K=5
BIC=  4357.6  AIC=  4203.7  full      K=6
BIC=  4388.4  AIC=  4208.2  full      K=7
BIC=  4414.9  AIC=  4243.4  diag      K=8
free parameters: 23  manual BIC: 4301.1  sklearn: 4301.1
```

BIC 在完整共變異數下選出真正的 $K = 4$（剪切變換讓各群變成橢圓，所以球形與對角模型需要額外的成分）。AIC 稍微偏好 $K = 5$：它較輕的懲罰讓第五個成分可以用小小的似然增益「買」到位置。一般而言兩個準則都不是「正確」的；它們都假設模型族足夠合適，而對非高斯的群，BIC 會很樂意用好幾個高斯來描述一個彎曲的群。若目標是密度估計，這沒有問題；若每個成分應該對應一個語意群，就有問題了。

其他經驗法則

手肘法（elbow method）畫出 k-means 的 J 對 K 的曲線，尋找彎折處；但 J 總是遞減，彎折處的判斷很主觀。下一節的 silhouette 與穩定度方法提供了其他選擇。貝氏作法也可以先放比需要更多的成分，再讓混合權重上的先驗把不需要的成分關掉；Bishop 以變分高斯混合模型完整示範了這一點 [2]。

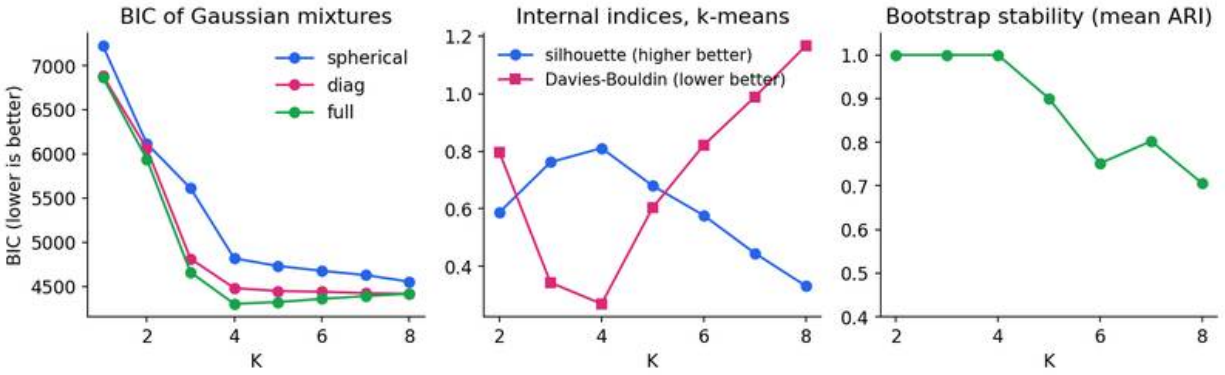


圖 6 — 在合成的四團資料上選擇 K 的三種方式。左：*BIC*（剪切後的資料）。中：*k-means* 的 *silhouette*（越高越好）與 *Davies–Bouldin*（越低越好）。右：自助法穩定度在 $K=3$ 與 $K=4$ 時都完美，因此光靠穩定度無法決定。

分群有效性：分群結果好不好？

白話版。沒有標籤時，「好」必須先被定義：緊密且分得開（內部指標）、與某個可信的參考一致（外部指標），或在資料受擾動時能重現（穩定度）。Theodoridis 與 Koutroumbas 用第 16 章專門討論這些問題 [1]；PR 09 會在監督式情境下回到驗證的議題。

內部指標

Silhouette [14]。對群 C 中的點 i，令 $a(i)$ 為它到 C 中其他點的平均距離，$b(i)$ 為它到任何其他群各點平均距離中的最小值。則

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1].$$

$s(i)$ 接近 1 表示 i 穩穩位於所屬群內；接近 0 表示在邊界上；負值表示它可能被分錯群。所有點的平均 silhouette 概括了整個分割，而逐點的 silhouette 圖能指出哪些群比較弱。

Davies–Bouldin [15]。令 s_i 為群 i 各點到其質心 c_i 的平均距離，$d_{ij} = \|c_i - c_j\|$。則

$$\mathrm{DB} = \frac{1}{K} \sum_{i=1}^{K} \max_{j \neq i} \frac{s_i + s_j}{d_{ij}},$$

也就是每一群與另一群之間「最壞情況相似度」的平均。越低越好。

Calinski–Harabasz [16]。以群間散佈 $B = \sum_k N_k (c_k - \bar{x})(c_k - \bar{x})^\top$ 與群內散佈 $W = \sum_k \sum_{n \in C_k} (x_n - c_k)(x_n - c_k)^\top$（$\bar{x}$ 為整體平均），

$$\mathrm{CH} = \frac{\operatorname{tr}(B)/(K-1)}{\operatorname{tr}(W)/(N-K)},$$

這是一種變異數比統計量。越高越好。

這三個指標都偏好在所選幾何下緊密、凸、分得開的群。它們會給 k-means 在雙月形資料上的錯誤答案比正確的彎曲答案更高的分數。它們衡量的是與某種假設的群形狀是否一致，而不是語意上是否正確。

外部指標

當有參考標籤時（用於基準測試，或來自另外保留的標註），以對排列不變的指標比較分割。

調整蘭德指數（ARI） [17]。蘭德指數計算兩個分割在多少比例的點對上一致（同為「同一群」或同為「不同群」）。ARI 減去在群大小相同的隨機標記下的期望值，再重新縮放：

$$\mathrm{ARI} = \frac{\mathrm{RI} - \mathbb{E}[\mathrm{RI}]}{\max \mathrm{RI} - \mathbb{E}[\mathrm{RI}]},$$

因此隨機分割的分數約為 0，（在重新編號下）完全相同的分割得 1。

正規化互資訊（NMI）。把兩組標記 U 與 V 視為定義在點上的隨機變數，則

$$\mathrm{NMI}(U, V) = \frac{I(U;V)}{\frac{1}{2}\left(H(U) + H(V)\right)},$$

其中 I 為互資訊，H 為熵（這裡示範的算術平均正規化是 scikit-learn 的預設，也有其他正規化方式）。Vinh、Epps 與 Bailey [18] 指出，當群數相對於 N 很大時，未調整的 NMI 會向上偏差，並建議使用經過機率校正的版本，例如調整互資訊（adjusted mutual information, AMI）。

如果參考標籤曾被用來調整演算法或 K，它們就不再是獨立的檢驗。

穩定度

在自助法樣本或擾動後的資料上重新分群，並在相同的點上以 ARI 比較新分割與原分割。產生不穩定分割的 K 值得懷疑。但穩定是必要條件而非充分條件：把兩團相近的資料合併是完全可重現的（圖 6 右，$K = 3$），而一個穩定的分割仍可能反映的是相機或光線等無關因素。

程式：有效性指標與穩定度

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import (silhouette_score, davies_bouldin_score,
                             calinski_harabasz_score, adjusted_rand_score,
```

```
                            normalized_mutual_info_score)

X, truth = make_blobs(n_samples=600, centers=4, cluster_std=1.0, random_state=7)
print(" K  silhouette(+)  Davies-Bouldin(-)  Calinski-Harabasz(+)")
for K in range(2, 8):
    z = KMeans(K, n_init=10, random_state=0).fit_predict(X)
    print(f"{K:2d}  {silhouette_score(X, z):12.3f}  {davies_bouldin_score(X, z):17.3f}"
          f"  {calinski_harabasz_score(X, z):20.1f}")

z = KMeans(4, n_init=10, random_state=0).fit_predict(X)
print("external ARI:", round(adjusted_rand_score(truth, z), 3),
      " NMI:", round(normalized_mutual_info_score(truth, z), 3))
perm = np.array([2, 0, 3, 1])[z]                       # relabel clusters
print("ARI after relabelling:", round(adjusted_rand_score(truth, perm), 3))

# stability: refit on bootstrap samples, compare on the original points
rng = np.random.default_rng(1)
for K in [3, 4, 5, 6]:
    base = KMeans(K, n_init=10, random_state=0).fit(X)
    scores = []
    for b in range(10):
        idx = rng.integers(0, len(X), len(X))
        boot = KMeans(K, n_init=10, random_state=b).fit(X[idx])
        scores.append(adjusted_rand_score(base.predict(X), boot.predict(X)))
    print(f"K={K}: bootstrap stability (mean ARI) = {np.mean(scores):.3f}")
```

```
 K  silhouette(+)  Davies-Bouldin(-)  Calinski-Harabasz(+)
 2         0.588              0.796                850.9
 3         0.762              0.343               2235.5
 4         0.811              0.270               7641.8
 5         0.680              0.603               6341.0
 6         0.577              0.821               5620.2
 7         0.446              0.988               5304.4
external ARI: 1.0  NMI: 1.0
ARI after relabelling: 1.0
K=3: bootstrap stability (mean ARI) = 1.000
K=4: bootstrap stability (mean ARI) = 1.000
K=5: bootstrap stability (mean ARI) = 0.858
K=6: bootstrap stability (mean ARI) = 0.699
```

在這些分得很開的團上，所有內部指標都同意 $K = 4$，外部指標也確認不論群如何編號都完全吻合。真實資料很少這麼友善：各指標常常意見不一，而這種不一致本身就是資訊，它告訴你在所選尺度下，資料並沒有唯一明確的群結構。

現代觀點

白話版。本章的演算法已有數十年歷史，至今仍無所不在。改變的是表示法：今天我們通常對學習得到的嵌入向量分群，而不是對原始像素或手工描述子；深度網路有時甚至是藉由分群來訓練的。

值得一讀的綜述

- **Xu 與 Wunsch，“Survey of clustering algorithms”（2005）** [19] 是從圖型識別與類神經網路觀點出發的經典廣泛綜述。它先回顧鄰近度量，再把演算法整理為階層式、平方誤差（k-means 家族）、混合密度、圖論、模糊、類神經（自組織映射、自適應共振理論）、核方法、序列資料、大規模與高維方法，最後討論分群有效性。它是古典分群領域的一張好地圖。
- **Jain，“Data clustering: 50 years beyond K-means”（2010）** [20] 是一篇較短、觀點鮮明的概論。它的主要訊息至今仍適用：沒有放諸四海皆準的最佳演算法；表示法與「群」的定義比演算法更重要；群數與有效性仍是開放問題；附加資訊（成對限制、半監督分群）與分群集成是有價值的方向。
- **Ikotun 等人，“K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of big data”（2023）** [21] 回顧了眾多 k-means 變體：初始化（包括 k-means++）、自動選擇 K、處理混合型或高維資料，以及可擴展與分散式的實作。
- **Zhou 等人，“A comprehensive survey on deep clustering: Taxonomy, challenges, and future directions”（ACM Computing Surveys，2024）** [22] 依「表示學習模組」與「分群模組」如何互動來整理深度分群：多階段方法（先學特徵再分群）、迭代方法（偽標籤與特徵學習交替進行）、生成式方法（如具混合先驗的自編碼器等潛在變數模型），以及同時最佳化方法（兩者聯合最佳化）。它也討論了基準、評估指標與應用。
- **Ren 等人，“Deep clustering: A comprehensive survey”（IEEE TNNLS，2025）** [23] 依資料來源與先驗知識整理此領域：單視角、半監督、多視角與遷移式深度分群。與 [22] 一起閱讀，可以看出深度分群的設計空間已變得多麼寬廣。

從視覺詞到自我標註

兩篇具影響力的視覺論文展示了古典分群如何變成訓練訊號。**DeepCluster** [24] 交替進行兩件事：對卷積網路的特徵執行 k-means，以及訓練該網路以群指派作為偽標籤來預測；在整個資料集上反覆進行，就能從無標籤影像學到有用的特徵。**SCAN** [25] 則把表示學習與分群分開：先用自監督的前置任務（pretext task）學特徵，再在該特徵空間中為每張影像找出最近鄰，訓練一個分群頭讓鄰居被分到同一群，最後以自我標註加以精煉。兩者都是把本章的概念（k-means 作為量化、鄰域作為相似性的證據）應用到學習得到的表示上；ML 05 說明了這些表示從何而來。

實務現況（2023–2026）與開放問題

在目前的實務中，為影像分組的常見作法是：從預訓練（通常是自監督）的編碼器取得嵌入向量，視需要降維，再執行 k-means、高斯混合或 HDBSCAN，並在一小部分已標註資料上驗證。本章的提醒依然完全適用：

- **表示法定義了相似性**。為某個目的訓練的嵌入向量，可能依風格、背景或相機分群，而不是依你在意的類別。
- **K 與有效性仍是開放問題**。資訊準則假設了模型族；內部指標假設了形狀；外部指標需要標籤。上述綜述沒有任何一篇宣稱有通用解法。
- **深度分群的評估通常仰賴有標籤的基準**，以及對排列不變的分數，例如把群與類別做最佳配對後的分群準確率、NMI 與 ARI [22]。這些分數衡量的是與某一種選定標記的一致程度，而不

是「真正的」結構。
- **規模與不平衡**。長尾資料、大量小群與串流資料，同時考驗古典與深度方法；[21] 與 [22] 都把它們列為活躍的研究問題。

重點整理

- 分群由它的相似性概念所定義。特徵、尺度與距離是在任何演算法執行前就做出的建模決策。
- K-means（Lloyd）交替進行最近中心指派與平均更新；J 單調下降至局部最小值。k-means++ 播種給出 $\mathbb{E}[J] \le 8(\ln K + 2) J_{\mathrm{OPT}}$，並大幅減少糟糕的結果；但仍應多次重新啟動。
- 連結方式定義了階層式分群：single linkage 會沿著細橋串接，complete 與 Ward 偏好緊密的群。樹狀圖仍然需要被切開。
- DBSCAN 把群定義為密度相連的區域，能找出任意形狀並標出雜訊；ε 由 k 距離曲線決定，HDBSCAN 能處理不同的密度。
- 高斯混合模型的 EM 由完整資料對數似然推出：E 步計算責任值，M 步計算加權平均與共變異數，並以拉格朗日乘數得到 $\pi_k = N_k/N$。ELBO 分解證明了單調改善；奇異解與標籤交換需要明確處理；k-means 是硬指派、等向的極限。
- $\mathrm{BIC} = -2\ln \hat{L} + p \ln N$ 與 $\mathrm{AIC} = -2\ln \hat{L} + 2p$ 在擬合與參數數量之間取捨；共變異數形式也是選擇的一部分。
- 內部指標（silhouette、Davies–Bouldin、Calinski–Harabasz）獎勵某種假設的幾何；外部指標（ARI、NMI）需要獨立的標籤；穩定度是必要但不充分的條件。

練習

1. **Lloyd 演算法不會增加 J**。證明指派步驟與更新步驟都不會增加 J，並據此論證演算法一定會停止。如果平手時隨機決定，終止論證哪裡會出問題？

   提示

   指派步驟分別最小化每一項 $\|x_n - \mu_{z_n}\|^2$；更新步驟利用 $\sum_{n \in C_k} \|x_n - \mu\|^2 = \sum_{n \in C_k} \|x_n - \bar{x}_k\|^2 + N_k \|\bar{x}_k - \mu\|^2$。分割最多有 K^N 種。若平手時隨機決定，兩個 J 相等的分割可能永遠來回切換；固定規則（例如平手時留在目前的群）可以避免。

2. **手算 k-means++**。一維資料為 $\{0, 1, 2, 10, 11\}$，$K = 2$，第一個種子為 0。計算每個點成為第二個種子的機率，並與均勻抽樣第二個種子比較。

   提示

   D^2 值為 $0, 1, 4, 100, 121$，總和 226。因此 $P(10) = 100/226$、$P(11) = 121/226$，合計約 0.98。若均勻抽樣，第二個種子落在右側那一群的機率只有 2/5（若排除第一個種子則為 2/4）。

3. **一維混合模型的 M 步**。對變異數為 σ_1^2, σ_2^2 的一維兩成分混合，不用矩陣微積分，直接從 Q 推導 σ_k^2 的 M 步更新。接著證明若某成分的責任值集中在單一點 x_n 上（該點 $r_{nk} \approx 1$、其餘點 ≈ 0），更新會把 σ_k^2 推向 0。scikit-learn 的 `reg_covar` 怎麼處理這件事？

提示

對 $\sum_n r_{nk}[-\frac{1}{2}\ln\sigma_k^2-(x_n-\mu_k)^2/(2\sigma_k^2)]$ 關於 σ_k^2 微分，得到 $\sigma_k^2=\sum_n r_{nk}(x_n-\mu_k)^2/N_k$。若所有權重都在同一點上，$\mu_k=x_n$，總和為零。`reg_covar` 在每次 M 步後於對角線加上一個小常數，讓變異數不會趨近於零。

4. **計算參數數量**。對 $D=64$ 的局部描述子與 $K=32$ 個成分，分別計算完整、對角與球形共變異數 GMM 的自由參數數量。若有 $N=10^5$ 個描述子，各情況的 BIC 懲罰 $p\ln N$ 有多大？

提示

完整：$31+32\cdot 64+32\cdot 64\cdot 65/2=31+2048+66560=68{,}639$。對角：$31+2048+2048=4{,}127$。球形：$31+2048+32=2{,}111$。再乘上 $\ln 10^5\approx 11.51$。完整模型必須讓 $-2\ln\hat{L}$ 比對角模型改善大約 7.4×10^5 才能打平。

5. **當指標誤導你**。在比較程式的雙月形資料上，以 $K=2$ 執行 k-means 並執行 DBSCAN。計算兩個分割的 silhouette 分數，以及它們相對於生成標籤的 ARI。用各指標衡量的內容解釋兩者的分歧。

提示

Silhouette 以歐氏距離計算，偏好凸且分得開的群，因此 k-means 的切法通常比兩個月形得到更高分，儘管它的 ARI 約為 0.26，而 DBSCAN 為 1.0。內部指標隱含了形狀假設；這裡的「正確」答案是由連通性定義的。

6. **穩定不等於真相**。產生四團高斯資料，其中兩團非常接近（例如中心為 $(0,0)$ 與 $(1.2,0)$，標準差 0.5）。計算 $K=3$ 與 $K=4$ 的自助法穩定度。哪一個比較穩定？哪一個符合生成過程？還需要什麼額外資訊才能下定論？

提示

把相近的兩團合併是非常可重現的，因此 $K=3$ 的穩定度可能與 $K=4$ 一樣高甚至更高。生成過程有四群，但它們是否應該被分開取決於任務。領域知識、一小部分已標註資料，或以 GMM 的 BIC 這類以密度為基礎的準則，都能提供額外證據。

參考文獻

1. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2009, Ch. 11–16 (clustering basic concepts, sequential, hierarchical, function-optimization and graph-based algorithms, cluster validity). doi
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006, Ch. 9 (k-means, mixtures of Gaussians, EM, the general EM algorithm). book page
3. S. P. Lloyd, "Least squares quantization in PCM," *IEEE Transactions on Information Theory*, vol. 28, no. 2, pp. 129–137, 1982. doi

4. J. MacQueen, “Some methods for classification and analysis of multivariate observations,” in *Proc. Fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1, pp. 281–297, 1967. Project Euclid
5. D. Arthur and S. Vassilvitskii, “k-means++: The advantages of careful seeding,” in *Proc. 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2007, pp. 1027–1035. pdf
6. D. Sculley, “Web-scale k-means clustering,” in *Proc. 19th International Conference on World Wide Web (WWW)*, 2010, pp. 1177–1178. doi
7. J. H. Ward, “Hierarchical grouping to optimize an objective function,” *Journal of the American Statistical Association*, vol. 58, pp. 236–244, 1963. doi
8. M. Ester, H.-P. Kriegel, J. Sander and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *Proc. 2nd International Conference on Knowledge Discovery and Data Mining (KDD)*, 1996. pdf
9. R. J. G. B. Campello, D. Moulavi and J. Sander, “Density-based clustering based on hierarchical density estimates,” in *Advances in Knowledge Discovery and Data Mining (PAKDD)*, LNCS, 2013, pp. 160–172. doi
10. U. von Luxburg, “A tutorial on spectral clustering,” *Statistics and Computing*, vol. 17, pp. 395–416, 2007. doi
11. A. P. Dempster, N. M. Laird and D. B. Rubin, “Maximum likelihood from incomplete data via the EM algorithm,” *Journal of the Royal Statistical Society: Series B*, vol. 39, no. 1, pp. 1–22, 1977. doi
12. G. Schwarz, “Estimating the dimension of a model,” *The Annals of Statistics*, vol. 6, no. 2, 1978. doi
13. H. Akaike, “A new look at the statistical model identification,” *IEEE Transactions on Automatic Control*, vol. 19, no. 6, pp. 716–723, 1974. doi
14. P. J. Rousseeuw, “Silhouettes: A graphical aid to the interpretation and validation of cluster analysis,” *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987. doi
15. D. L. Davies and D. W. Bouldin, “A cluster separation measure,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-1, no. 2, pp. 224–227, 1979. doi
16. T. Caliński and J. Harabasz, “A dendrite method for cluster analysis,” *Communications in Statistics*, vol. 3, no. 1, pp. 1–27, 1974. doi
17. L. Hubert and P. Arabie, “Comparing partitions,” *Journal of Classification*, vol. 2, pp. 193–218, 1985. doi
18. N. X. Vinh, J. Epps and J. Bailey, “Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance,” *Journal of Machine Learning Research*, vol. 11, pp. 2837–2854, 2010. JMLR
19. R. Xu and D. Wunsch II, “Survey of clustering algorithms,” *IEEE Transactions on Neural Networks*, vol. 16, no. 3, pp. 645–678, 2005. doi
20. A. K. Jain, “Data clustering: 50 years beyond K-means,” *Pattern Recognition Letters*, vol. 31, no. 8, pp. 651–666, 2010. doi
21. A. M. Ikotun, A. E. Ezugwu, L. Abualigah, B. Abuhaija and J. Heming, “K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of big data,” *Information Sciences*, vol. 622, pp. 178–210, 2023. doi
22. S. Zhou, H. Xu, Z. Zheng, J. Chen, Z. Li, J. Bu, J. Wu, X. Wang, W. Zhu and M. Ester, “A comprehensive survey on deep clustering: Taxonomy, challenges, and future directions,” *ACM*

Computing Surveys, vol. 57, no. 3, pp. 1–38, 2024 (arXiv:2206.07579). doi
23. Y. Ren, J. Pu, Z. Yang, J. Xu, G. Li, X. Pu, P. S. Yu and L. He, "Deep clustering: A comprehensive survey," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 4, pp. 5858–5878, 2025 (arXiv:2210.04142). doi
24. M. Caron, P. Bojanowski, A. Joulin and M. Douze, "Deep clustering for unsupervised learning of visual features," in *Proc. European Conference on Computer Vision (ECCV)*, LNCS, 2018, pp. 139–156. doi
25. W. Van Gansbeke, S. Vandenhende, S. Georgoulis, M. Proesmans and L. Van Gool, "SCAN: Learning to classify images without labels," in *Proc. European Conference on Computer Vision (ECCV)*, LNCS, 2020, pp. 268–285. doi

PR 08・近似推論與貝氏模型複雜度

後驗積分為何困難，以及兩大類解法：取樣（Monte Carlo、重要性取樣、Metropolis–Hastings、Gibbs、HMC）與最佳化（變分推論、Laplace、EP），並連結到 VAE 與擴散模型。

網站版本: https://blog.shyandram.dev/zh-tw/learn/pattern-recognition/pr-08-approximate-inference/

你將學到

- 為什麼貝氏後驗在差一個常數的意義下很容易「計算」，卻很難「積分」；以及為什麼網格法在幾個維度之後就失效。
- 普通 Monte Carlo、拒絕取樣（rejection sampling）與重要性取樣（importance sampling）如何把積分變成平均，以及如何判斷權重已經崩潰。
- 馬可夫鏈 Monte Carlo（MCMC）如何運作：細緻平衡（detailed balance）、Metropolis–Hastings（含非對稱提議的修正項）、Gibbs 取樣、Hamiltonian Monte Carlo，以及讓你保持誠實的診斷工具（$\hat{R}$、有效樣本數）。
- 變分推論（variational inference）如何把積分變成最佳化：用兩種方法推導證據下界（ELBO）、mean-field 座標上升、反向與正向 KL 的差別，以及搭配重參數化技巧（reparameterization trick）的隨機梯度。
- Laplace 近似與期望傳播（expectation propagation）的定位、模型證據如何懲罰複雜度，以及同樣的想法如何驅動 VAE 與擴散模型。

先看全貌

前面的章節都在找一組「最好」的參數：最大概似、MAP，或混合模型的 EM。貝氏辨識系統問的是更大的問題：看過資料之後，每一組參數各有多可信？把它們全部考慮進來，平均會預測什麼？回答這個問題需要對所有未知量積分，而幾乎所有實用模型的積分都沒有閉式解。本章介紹兩大類替代方案：**抽樣**，產生行為像後驗的樣本；或**擬合**一個較簡單、但盡量貼近後驗的分布。

> **用最簡單的話說**
>
> 想像一大片漆黑的田地，到處埋著多寡不一的寶藏，而你想知道寶藏的總量。找到最富的那一點很容易：一直往上坡走就好，這就是最佳化。要知道總量就難多了。一種做法是隨機但聰明地四處走動，在寶藏多的地方多停留，邊走邊記錄，這就是**取樣**。另一種做法是在田上蓋一條簡單平滑的毯子，盡量讓毯子的形狀像藏寶圖，然後改量毯子，這就是**變分推論**。走路慢但老實；毯子快，卻可能把它沒被設計來表現的起伏藏起來。

為什麼對辨識很重要：不確定性告訴系統何時該拒絕判斷（見PR 01）；積分後的證據可以在沒有保留驗證集的情況下比較模型；而本章的工具，尤其是 ELBO 與 Langevin 式取樣，正是現代生成模型所用的方法。本章依循 Bishop 關於近似推論與取樣的章節 [1]；Theodoridis 與 Koutroumbas 則提供估計與最佳化的背景（第 2 章與附錄 C），並非與本章方法一一對應 [2]。

為什麼精確推論很難

白話版。貝氏定理說：後驗等於「先驗乘以概似，再除以一個正規化的數」。分子對任何單一猜測通常都好算；分母卻是對所有可能猜測的總和，麻煩就從這裡開始。

精確版。把所有未知量（潛在變數與參數）合記為 z，觀測資料記為 D，則

$$p(z \mid D) = \frac{p(D, z)}{p(D)}, \qquad p(D) = \int p(D, z)\, dz,$$

其中 $p(D, z) = p(D \mid z)\, p(z)$ 是聯合密度（概似乘以先驗），$p(D)$ 是**證據**（evidence），又稱邊際概似。我們想要的幾乎都是後驗下的期望值：

$$\mathbb{E}_{p(z \mid D)}[f(z)] = \int f(z)\, p(z \mid D)\, dz,$$

例如後驗平均（$f(z) = z$）、預測機率（$f(z) = p(y^* \mid x^*, z)$），或某參數為正的機率（$f(z) = \mathbf{1}[z > 0]$）。以下記 $\tilde{p}(z) = p(D, z)$ 為**未正規化**後驗：逐點計算很便宜，但正規化常數 $Z = p(D)$ 未知。

積分困難有三個原因：

1. **維度**。每軸 G 個點的網格在 d 維需要 G^d 次計算。$G = 201$、$d = 50$ 時約為 10^{115}，遠超任何電腦的能力。
2. **集中**。資料多時，後驗只占廣大空間中極小的區域，隨機猜測幾乎永遠落不進去。
3. **離散結構**。有 K 個成分、N 個點的混合模型，潛在指派有 K^N 種組合；即使每一項都很簡單，也無法精確加總。

MAP 估計藉由最大化 $\tilde{p}$ 繞過這些困難，但最大值不是積分。後驗的最高點可能位於一個很窄、質量很少的尖峰上，而單一點完全不帶有分散程度的資訊。PR 07 的 EM 介於兩者之間：它在 E step 對潛在指派積分（高斯混合可以精確做到），但最後仍回傳參數的點估計。

實例：貝氏邏輯斯迴歸

考慮二類問題，輸入 $x_n \in \mathbb{R}^2$（一個偏差特徵與一個量測值），標籤 $y_n \in \{0, 1\}$，權重 w，模型為

$$p(y_n = 1 \mid x_n, w) = \sigma(w^\top x_n), \qquad \sigma(s) = \frac{1}{1 + e^{-s}}, \qquad p(w) = \mathcal{N}(0, 4I).$$

對數聯合密度為 $\sum_n \left[y_n w^\top x_n - \log(1 + e^{w^\top x_n}) \right] - \frac{1}{8}\|w\|^2$ 加上常數。高斯先驗與邏輯斯概似不共軛，因此後驗不是高斯，正規化常數也沒有公式。只有兩個權重時，我們仍可以用網格暴力計算，得到一個可以拿來檢驗所有近似方法的標準答案。

```
import numpy as np

rng = np.random.default_rng(0)
n = 40
X = np.c_[np.ones(n), rng.normal(size=n)]
w_true = np.array([-0.5, 1.5])
y = rng.random(n) < 1 / (1 + np.exp(-X @ w_true))
```

```
def log_joint(w):                       # log p(y | X, w) + log p(w), w ~ N(0, 2^2 I)
    s = X @ w
    return np.sum(y * s - np.logaddexp(0, s)) - 0.5 * np.sum(w**2) / 4.0

# Brute-force normalization on a grid: fine in 2-D, hopeless in 50-D.
g = np.linspace(-4, 4, 201)
W0, W1 = np.meshgrid(g, g, indexing="ij")
logp = np.array([[log_joint(np.array([a, b])) for b in g] for a in g])
cell = (g[1] - g[0])**2
log_evidence = np.log(np.sum(np.exp(logp - logp.max())) * cell) + logp.max()
post = np.exp(logp - log_evidence)
mean = np.array([np.sum(W0 * post), np.sum(W1 * post)]) * cell
sd = np.sqrt(np.array([np.sum((W0 - mean[0])**2 * post),
                       np.sum((W1 - mean[1])**2 * post)]) * cell)
print("log evidence:", round(log_evidence, 3))
print("posterior mean:", mean.round(3), " sd:", sd.round(3))
print("grid points for 201 per axis in 50-D:", f"{201.0**50:.2e}")
```

輸出的對數證據為 -22.117，後驗平均為 $(-0.303, 1.631)$，標準差為 $(0.366, 0.578)$。注意取指數前先減去 `logp.max()`：真實資料的對數密度是很大的負數，在對數空間運算（log-sum-exp 技巧）是每位實作者的第一個習慣。稍後我們會用同一個模型來評估 Laplace 與變分近似。

Monte Carlo：把積分變成平均

白話版。如果能從某個分布抽出隨機樣本，那麼某個量在這些樣本上的平均，就能估計它的期望值。樣本越多估計越準，而誤差縮小的速度可以預測，而且與維度無關。

精確版。給定獨立樣本 $z^{(1)}, \dots, z^{(S)} \sim p$，$I = \mathbb{E}_p[f(z)]$ 的 Monte Carlo 估計量為

$$\hat{I}_S = \frac{1}{S}\sum_{s=1}^{S} f(z^{(s)}), \qquad \mathbb{E}[\hat{I}_S] = I, \qquad \operatorname{Var}[\hat{I}_S] = \frac{\operatorname{Var}_p[f(z)]}{S}.$$

其中 S 是樣本數，$\operatorname{Var}_p[f]$ 是 f 在 p 下的變異數。這個估計量是**不偏**的，標準誤以 $1/\sqrt{S}$ 下降：誤差要減半，樣本要變四倍。最驚人的是式中沒有 d。Monte Carlo 把隨維度指數成長的網格，換成只取決於 f 變異數的成本。

問題在於「從 p 抽樣」這個假設。後驗通常無法直接抽樣。本章取樣部分的其餘內容，就是在製造這樣的樣本，或修正從錯誤分布抽出的樣本。

拒絕取樣

假設我們能從簡單的**提議分布**（proposal）$q(z)$ 抽樣，並知道常數 M 使得對所有 z 都有 $M\,q(z) \geq \tilde{p}(z)$。抽 $z \sim q$ 與 $u \sim \text{Uniform}(0,1)$，若 $u < \tilde{p}(z)/(Mq(z))$ 就保留 z。保留下來的點正是來自 p 的精確樣本，因為包絡 Mq 底下被接受的區域形狀恰好就是 $\tilde{p}$。接受機率為 Z/M，其中 $Z = \int \tilde{p}$。在高維空間，可用的包絡必須比目標大上許多，幾乎所有樣本都會被拒絕。因此拒絕取樣是處理一維條件分布（例如在 Gibbs 取樣器內部）的積木，而不是通用的後驗引擎 [1]。

重要性取樣

白話版。從一個方便的分布抽樣，再給每個樣本一個權重，說明它在你真正關心的分布下比較可能（或比較不可能）多少。當方便的分布涵蓋重要區域時效果很好；涵蓋不到時，它會默默地失敗。

精確版。對任何在 $p(z)f(z) \neq 0$ 之處都滿足 $q(z) > 0$ 的 q，

$$\mathbb{E}_p[f(z)] = \int f(z)\,\frac{p(z)}{q(z)}\,q(z)\,dz \approx \frac{1}{S}\sum_{s=1}^{S} w_s f(z^{(s)}), \qquad w_s = \frac{p(z^{(s)})}{q(z^{(s)})}, \quad z^{(s)} \sim q.$$

比值 w_s 稱為**重要性權重**。若只有 $\tilde{p}$ 可用，把權重正規化就能消去未知的 Z：

$$\hat{I}_{\mathrm{SN}} = \sum_{s=1}^{S} \bar{w}_s f(z^{(s)}), \qquad \bar{w}_s = \frac{\tilde{w}_s}{\sum_t \tilde{w}_t}, \qquad \tilde{w}_s = \frac{\tilde{p}(z^{(s)})}{q(z^{(s)})}.$$

這個**自我正規化**（self-normalized）估計量在 S 有限時略有偏差，但當 $S \to \infty$ 時一致。附帶的好處是 $\frac{1}{S}\sum_s \tilde{w}_s$ 是證據 Z 的不偏估計，這就是重要性取樣用於模型比較的方式。

估計的品質取決於權重有多不均勻。常用的摘要是權重的**有效樣本數**（effective sample size, ESS）：

$$\mathrm{ESS} = \frac{1}{\sum_s \bar{w}_s^2},$$

所有權重相等時為 S，一個權重獨占全部時為 1。有兩種失敗模式值得注意：

- **提議分布太窄或放錯位置**。p 有質量而 q 很小的區域，會產生罕見但巨大的權重。估計被少數樣本主導，變異數甚至可能是無限大。
- **尾部太輕**。比值 p/q 在尾部必須有界。經驗法則是讓 q 的尾部比 p 更重（用 Student-t 取代高斯，或用更寬的高斯）。

```
import numpy as np
from scipy.stats import norm

rng = np.random.default_rng(1)
N = 10_000
exact = norm.sf(3.0)                       # P(X > 3) for X ~ N(0, 1)

# Plain Monte Carlo: most samples never reach the tail.
x = rng.normal(0, 1, N)
plain = np.mean(x > 3)

# Importance sampling with a proposal centered on the tail, q = N(3, 1).
z = rng.normal(3, 1, N)
w = norm.pdf(z) / norm.pdf(z, loc=3)       # p(z) / q(z)
is_est = np.mean(w * (z > 3))
print(f"exact {exact:.6f}  plain MC {plain:.6f}  importance {is_est:.6f}")

# Self-normalized IS for an unnormalized target, with effective sample size.
```

```
def log_p_tilde(t):                       # N(1.5, 0.7^2) without its constant
    return -0.5 * ((t - 1.5) / 0.7) ** 2

for scale in [1.0, 3.0, 0.3]:
    t = rng.normal(0, scale, N)
    log_w = log_p_tilde(t) - norm.logpdf(t, scale=scale)
    wn = np.exp(log_w - log_w.max())
    wn /= wn.sum()
    ess = 1.0 / np.sum(wn**2)
    print(f"proposal N(0,{scale}^2): mean {np.sum(wn * t):.3f}  ESS {ess:7.1f} of {N}")
```

稀有事件的估計說明了重要性取樣存在的理由：精確的尾端機率是 0.001350；10,000 個樣本的普通 Monte Carlo 給出 0.001900（只有 19 個樣本落在尾端），平移後的提議分布則給出 0.001372。第二部分的目標平均為 1.5。兩個寬的提議分布給出 1.509 與 1.492，ESS 約為 1,870 與 2,830；窄的提議分布 $\mathcal{N}(0, 0.3^2)$ 給出 0.921，ESS 只有 13，看起來很有把握，答案卻是錯的。ESS 極小就是警鈴。

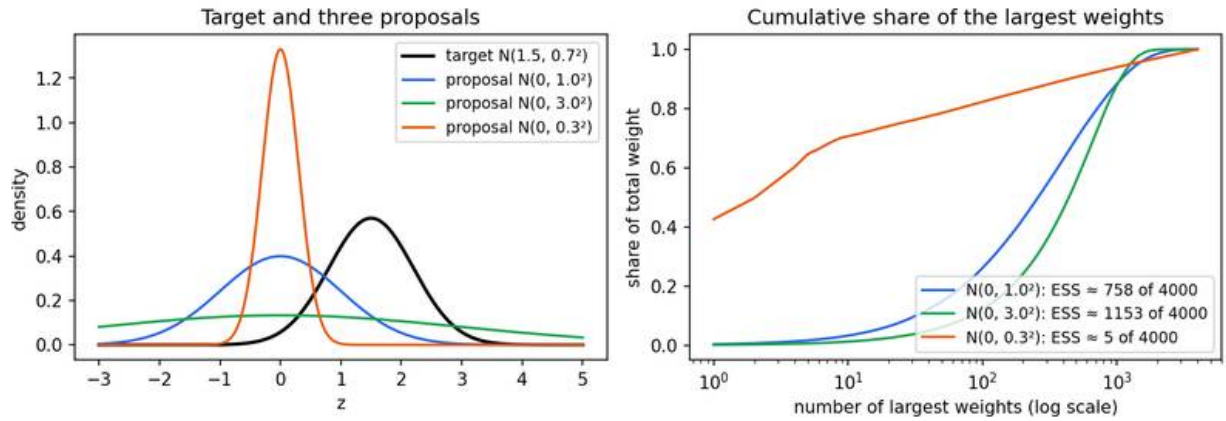


圖 *1* — 每個提議分布抽 *4,000* 個樣本的重要性取樣。窄的提議分布（橘色）幾乎碰不到目標，因此最大的單一權重就占了總權重的 *40%* 以上，*ESS* 約為 *5*。寬的提議分布則把權重分散到數百個樣本。由本篇的圖形腳本計算。

即使不是主要引擎，重要性取樣仍然很重要：它能把舊模型的樣本重新加權到新模型、能估計證據，也是**序列 Monte Carlo**（粒子濾波器）的核心；在序列 Monte Carlo 中，權重隨資料到來逐步更新並重新取樣。

馬可夫鏈 Monte Carlo

白話版。不再抽獨立樣本，而是進行一場隨機漫步，並設計規則讓它長期下來在每個區域停留的時間比例，恰好等於該區域的後驗機率。這樣沿路的時間平均就能估計後驗期望值。相鄰步驟彼此相關，所以一段長漫步的價值少於同樣數量的獨立樣本。

精確版。若轉移核 $T(z' \mid z)$ 滿足

$$\int T(z' \mid z)\, p(z)\, dz = p(z'),$$

則 p 是此馬可夫鏈的**穩態**（不變）分布。一個簡單的充分條件是**細緻平衡**（可逆性）：

$$p(z)\,T(z' \mid z) = p(z')\,T(z \mid z') \quad \text{for all } z, z'.$$

兩邊對 z 積分即得穩態條件。若鏈還能到達每個區域（不可約）且不會週期循環（非週期），則無論從哪裡出發，狀態的分布都會收斂到 p，遍歷平均 $\frac{1}{S}\sum_s f(z^{(s)})$ 也會收斂到 $\mathbb{E}_p[f]$。重點是細緻平衡只涉及比值 $p(z')/p(z)$，因此未知的正規化常數會消去。

Metropolis–Hastings

Metropolis 演算法 [3] 經 Hastings 推廣 [4]，可以從任何提議分布 $q(z' \mid z)$ 建構出這樣的轉移核：

1. 從目前狀態 z 提議 $z' \sim q(z' \mid z)$。
2. 以下列機率接受：

$$A(z', z) = \min\left(1,\ \frac{\tilde{p}(z')\,q(z \mid z')}{\tilde{p}(z)\,q(z' \mid z)}\right).$$

3. 若接受就移到 z'；否則**停在 z，並再記錄一次 z**。

其中 $q(z \mid z')$ 是提議反向移動的密度。因子 $q(z \mid z')/q(z' \mid z)$ 稱為 **Hastings 修正**：它抵銷提議分布對某些方向的偏好。對稱提議（例如高斯隨機漫步）滿足 $q(z' \mid z) = q(z \mid z')$，規則就化簡為 Metropolis 原本的 $\min(1, \tilde{p}(z')/\tilde{p}(z))$。

為什麼行得通？取 $z' \neq z$，並假設 $\tilde{p}(z')q(z \mid z') \le \tilde{p}(z)q(z' \mid z)$。則 $p(z)T(z' \mid z) = p(z)\,q(z' \mid z)\,\frac{p(z')q(z|z')}{p(z)q(z'|z)} = p(z')q(z \mid z')$；反方向的接受率為 1，所以 $p(z')T(z \mid z') = p(z')q(z \mid z')$。兩邊相等，細緻平衡成立。

忘了第 3 步（只記錄被接受的狀態）是經典錯誤：它會高估鏈容易穿過的區域，讓所有估計都產生偏差。

已知動差的高斯目標可以用來檢查隨機漫步鏈。斷言刻意設得寬鬆：這是數值上的基本檢查，不是收斂證明。

```
import numpy as np

rng = np.random.default_rng(12)
def log_target(x):
    return -0.5 * ((x - 1.5) / 0.7)**2
x, accepted = -3.0, 0
samples = []
for step in range(18000):
    proposal = x + rng.normal(0, 0.8)
    if np.log(rng.random()) < min(0.0, log_target(proposal) - log_target(x)):
        x = proposal
        accepted += 1
    if step >= 2000:                      # discard warm-up, keep repeats
        samples.append(x)
samples = np.asarray(samples)
```

```
rho1 = np.corrcoef(samples[:-1], samples[1:])[0, 1]
print("mean, std:", samples.mean(), samples.std())
print("acceptance:", accepted / 18000, "lag-one correlation:", rho1)
assert abs(samples.mean() - 1.5) < 0.12
assert abs(samples.std() - 0.7) < 0.12
```

這條鏈得到平均 1.50、標準差 0.69，接受率 0.67，lag-1 自相關 0.74。步長是主要的調整旋鈕，而光看接受率無法判斷步長好不好。極小的步長幾乎全被接受，卻像在爬行；極大的步長大多被拒絕，鏈停在原地。兩者都會產生高度相關的樣本。

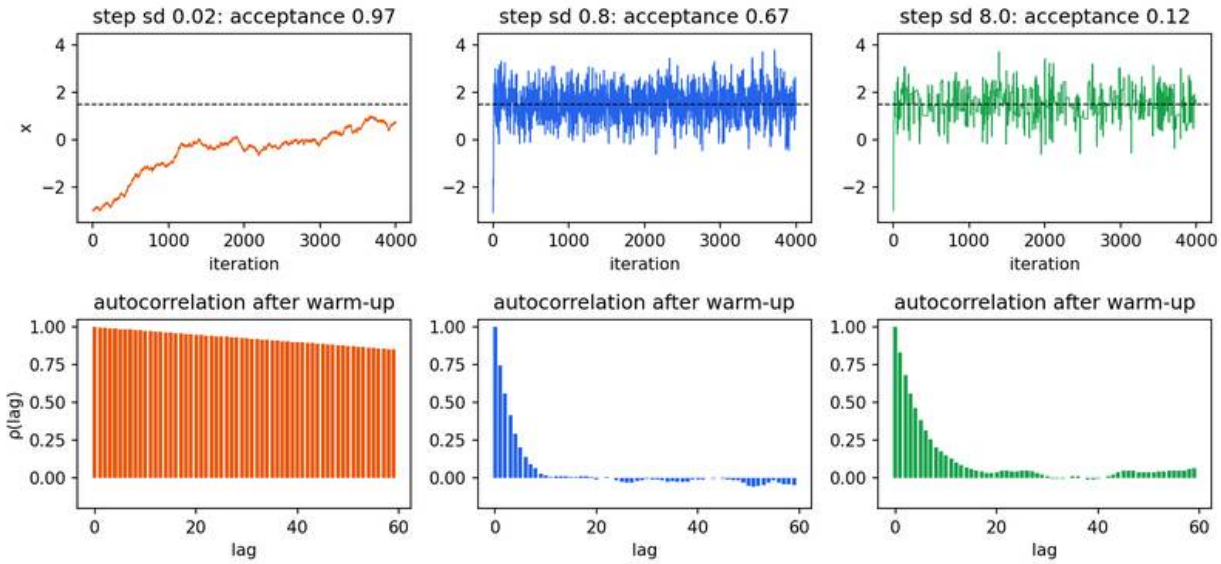


圖 2 — 對 $N(1.5, 0.7^2)$ 執行隨機漫步 *Metropolis*，起點為 -3。步長 *0.02*（接受率 *0.97*）在 *4,000* 步後仍未抵達目標，自相關幾乎不衰減。步長 *0.8* 混合良好。步長 *8*（接受率 *0.12*）長時間停在同一處。由本篇的圖形腳本計算。

Gibbs 取樣

白話版。一次只更新一個變數（或一個區塊），從「給定其他所有變數」的分布中抽它。如果這些條件分布很容易抽（共軛模型常是如此），每一步都會被接受。

精確版。 Gibbs 取樣由 Geman 與 Geman 為了以馬可夫隨機場做影像復原而提出 [5]。它依序走過區塊 $j = 1, \dots, J$，並抽

$$z_j \sim p(z_j \mid z_{-j}, D),$$

其中 z_{-j} 表示除了 j 以外的所有區塊。它是 Metropolis–Hastings 的特例：提議為 $q(z' \mid z) = p(z'_j \mid z_{-j})$ 且 $z'_{-j} = z_{-j}$，代入接受比恰好等於 1 [1]。以高斯混合為例，可以交替地依 responsibility 為每個點抽成分標籤，再從共軛後驗抽成分參數，這是 EM 的隨機版親戚。

對平均為零、變異數為一、相關係數為 ρ 的二維高斯，條件分布為 $z_1 \mid z_2 \sim \mathcal{N}(\rho z_2,\, 1 - \rho^2)$，$z_2$ 亦同。下面的程式從四個分散的角落各跑一條鏈，並計算下一小節介紹的 **split-$\hat{R}$** 收斂診斷。

```
import numpy as np

rng = np.random.default_rng(3)
rho = 0.85                      # target: zero-mean bivariate Gaussian, unit variances
sd = np.sqrt(1 - rho**2)        # conditional standard deviation

def gibbs(n_steps, start):
    z = np.array(start, dtype=float)
    out = np.empty((n_steps, 2))
    for t in range(n_steps):
        z[0] = rng.normal(rho * z[1], sd)   # z1 | z2 ~ N(rho z2, 1 - rho^2)
        z[1] = rng.normal(rho * z[0], sd)   # z2 | z1 ~ N(rho z1, 1 - rho^2)
        out[t] = z
    return out

def split_rhat(chains):         # chains: (m, n) draws of one scalar quantity
    m, n = chains.shape
    halves = chains[:, : n // 2 * 2].reshape(2 * m, n // 2)
    W = halves.var(axis=1, ddof=1).mean()          # within-chain variance
    B = (n // 2) * halves.mean(axis=1).var(ddof=1) # between-chain variance
    var_plus = (n // 2 - 1) / (n // 2) * W + B / (n // 2)
    return np.sqrt(var_plus / W)

starts = [(-4, -4), (4, 4), (-4, 4), (4, -4)]
chains = np.stack([gibbs(3000, s)[500:] for s in starts])   # drop warm-up
draws = chains.reshape(-1, 2)
print("sample correlation:", np.corrcoef(draws.T)[0, 1].round(3))
print("sample variances:", draws.var(axis=0).round(3))
lag1 = np.mean([np.corrcoef(c[:-1, 0], c[1:, 0])[0, 1] for c in chains])
print("lag-1 autocorrelation of z1:", lag1.round(3), " (theory rho^2 =", rho**2, ")")
print("split R-hat for z1:", split_rhat(chains[:, :, 0]).round(4))
```

輸出：樣本相關 0.849，變異數約 0.98，lag-1 自相關 0.717（理論值 $\rho^2 = 0.7225$），split-$\hat{R}$ 為 1.0009。自相關是沿座標軸移動的代價：目標是一條細長的斜脊時，每個座標只能走一小段就撞到「牆」。當 $\rho \to 1$，鏈會慢到幾乎不動。重新參數化（旋轉或白化變數），或把相關的變數當成一個區塊一起更新，就能解決這個問題。

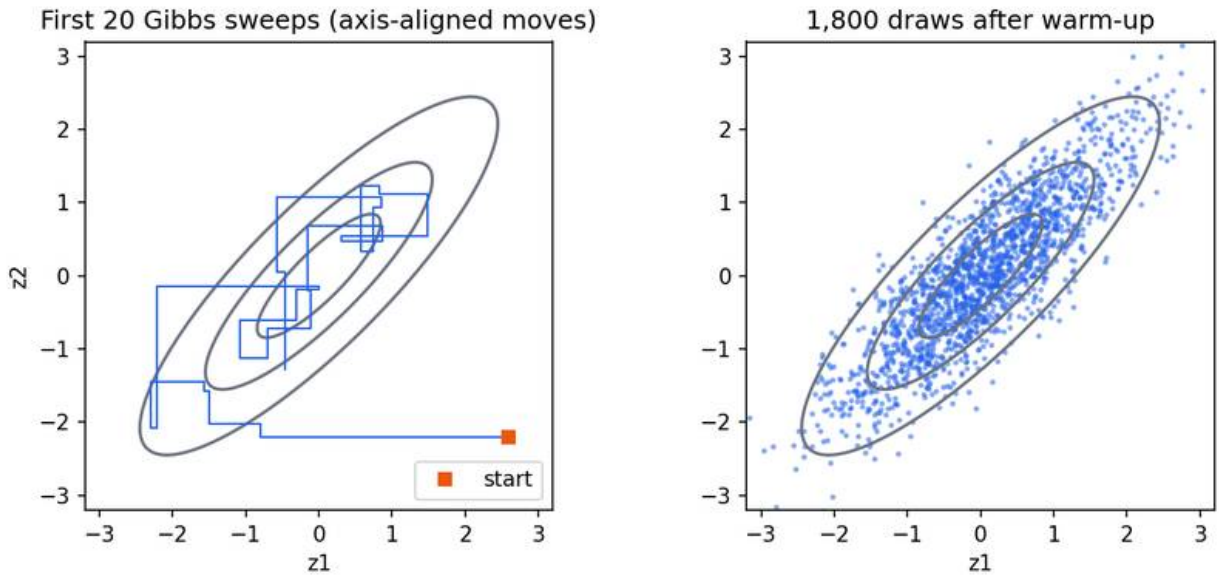


圖 3 — 對相關係數 *0.85* 的高斯進行 *Gibbs* 取樣。每半步都平行於座標軸移動（左），所以沿斜脊前進很慢；但暖機之後，樣本仍正確地填滿目標（右）。由本篇的圖形腳本計算。

Hamiltonian Monte Carlo

白話版。隨機漫步把力氣浪費在來回踱步上。Hamiltonian Monte Carlo（HMC）利用對數後驗的梯度，像一顆在曲面上無摩擦滑行的冰球一樣沿著分布滑行，做出又長又有方向的移動，同時仍以很高的機率被接受。

精確版。定義位能 $U(z) = -\log \tilde{p}(z)$，加入輔助動量 $r \sim \mathcal{N}(0, M)$（M 為質量矩陣），組成 Hamiltonian

$$H(z, r) = U(z) + \tfrac{1}{2} r^{\top} M^{-1} r.$$

Hamiltonian 動力學保持 H 守恆並保持體積。實作上以步長 ϵ 的 L 個 **leapfrog** 步驟模擬：

$$r \leftarrow r - \tfrac{\epsilon}{2} \nabla U(z), \qquad z \leftarrow z + \epsilon M^{-1} r, \qquad r \leftarrow r - \tfrac{\epsilon}{2} \nabla U(z).$$

leapfrog 積分器可逆且保持體積，但不完全守恆能量，因此終點 (z', r') 以 $\min\left(1, \exp(H(z,r) - H(z',r'))\right)$ 的機率被接受，也就是在聯合空間上的一步 Metropolis [6]。軌跡順著目標的幾何走，因此 HMC 能穿越會困住 Gibbs 或隨機漫步的相關斜脊。它的調整參數 ϵ 與 L 很敏感；**No-U-Turn Sampler**（NUTS）在路徑開始折返時停止，自動決定軌跡長度，並在暖機期間調整 ϵ [7]。Betancourt 的概念性導論以**典型集合**（typical set，後驗質量實際所在的那層薄殼）解釋 HMC 為何能擴展到高維 [8]。HMC 需要可微、連續的目標；離散潛在變數必須先加總掉，或用其他移動方式處理。

一個近親是 **Langevin** 更新

$$z \leftarrow z + \tfrac{\epsilon}{2} \nabla \log \tilde{p}(z) + \sqrt{\epsilon}\, \xi, \qquad \xi \sim \mathcal{N}(0, I),$$

它相當於重新抽動量後只走一步 leapfrog。沒有接受/拒絕步驟時，有限的 ϵ 只是近似正確；加上 Metropolis 修正就成為 Metropolis-adjusted Langevin algorithm。請記住這個更新，它會在擴散模型中再次出現。

診斷：這條鏈說的是實話嗎？

有限次的執行不可能證明收斂，診斷只能揭露問題。標準做法：

- **從分散的起點跑多條鏈**，並捨棄每條鏈開頭的暖機部分。
- **用 $\hat{R}$ 比較各鏈**。有 m 條（切成兩半的）長度為 n 的鏈，令 W 為鏈內變異數的平均、B/n 為各鏈平均的變異數，則

$$\hat{R} = \sqrt{\frac{\frac{n-1}{n}W + \frac{1}{n}B}{W}}.$$

若所有鏈探索的是同一個分布，$\hat{R} \approx 1$；明顯大於 1 表示各鏈意見不一。Vehtari 等人指出傳統版本會漏掉重尾與尺度差異，並提出秩正規化（rank-normalized）、折疊的 split-$\hat{R}$，以及 bulk 與 tail 有效樣本數；這就是目前軟體所回報的版本 [9]。

- **回報有效樣本數**。對於穩態鏈，若目標量的 lag-ℓ 自相關為 ρ_ℓ，

$$N_{\text{eff}} \approx \frac{N}{1 + 2\sum_{\ell \geq 1} \rho_\ell},$$

當估計的自相關變得雜亂時就截斷總和。Monte Carlo 標準誤大約是 $\text{sd}/\sqrt{N_{\text{eff}}}$，而不是 $\text{sd}/\sqrt{N}$。對上面的 Gibbs 鏈，$\rho_\ell = \rho^{2\ell}$，所以 $N_{\text{eff}}/N = (1-\rho^2)/(1+\rho^2) \approx 0.16$：儲存的 10,000 個樣本約只值 1,600 個獨立樣本。

- **檢查軌跡圖**；對梯度式取樣器，也要注意 divergence 警告，它標示出積分器跟不上的高曲率區域。

不同的量混合速度不同：平均看起來沒問題，尾端分位數卻可能還沒收斂。而且即使鏈混合得完美，它描述的也只是你寫下的模型的後驗，完全無法說明模型本身是否正確。

變分推論

白話版。選一族簡單的分布（例如彼此獨立的高斯），在其中尋找最接近後驗的那一個。積分變成最佳化，速度快、能處理巨量資料，代價是一種無法靠跑更久來消除的偏差。

精確版。選擇一族可處理的密度 $\mathcal{Q}$，以**變分參數** ϕ 編號。變分推論（VI）[10] 求解

$$q^* = \arg\min_{q \in \mathcal{Q}} D_{\text{KL}}(q(z) \,\|\, p(z \mid D)), \qquad D_{\text{KL}}(q\|p) = \mathbb{E}_q\left[\log \frac{q(z)}{p(z)}\right].$$

這個 KL 含有未知的 $p(D)$，無法直接計算。訣竅就是 ELBO。

用兩種方法推導 ELBO

推導一：分解。把 $\log p(z \mid D) = \log p(D, z) - \log p(D)$ 代入 KL：

$$D_{\mathrm{KL}}(q \| p(\cdot \mid D)) = \mathbb{E}_q[\log q(z)] - \mathbb{E}_q[\log p(D, z)] + \log p(D).$$

移項得到

$$\log p(D) = \mathcal{L}(q) + D_{\mathrm{KL}}(q(z) \,\|\, p(z \mid D)), \qquad \mathcal{L}(q) = \mathbb{E}_q[\log p(D, z)] - \mathbb{E}_q[\log q(z)].$$

$\log p(D)$ 與 q 無關，所以最大化 $\mathcal{L}$ 恰好等於最小化 KL。KL 非負，所以 $\mathcal{L}(q) \le \log p(D)$：它是**證據下界**（evidence lower bound），簡稱 ELBO。

推導二：Jensen 不等式。在證據積分中同乘同除 q，再利用 log 的凹性：

$$\log p(D) = \log \mathbb{E}_q\left[\frac{p(D, z)}{q(z)}\right] \ge \mathbb{E}_q\left[\log \frac{p(D, z)}{q(z)}\right] = \mathcal{L}(q).$$

Jensen 不等式的差距正是推導一中的 KL。ELBO 有一個好用的讀法：$\mathcal{L} = \mathbb{E}_q[\log p(D \mid z)] - D_{\mathrm{KL}}(q(z) \| p(z))$，也就是要擬合資料，但不要離先驗太遠。EM 是一個特例：在 E step 把潛在變數的 q 設為精確後驗（讓下界變緊），再於 M step 對參數最大化下界 [1]。

Mean field 與座標上升（CAVI）

最常見的分布族是 **mean field**：$q(z) = \prod_j q_j(z_j)$，即 z 各區塊上彼此獨立的因子。固定 q_j 以外的所有因子，對 q_j 最大化 ELBO，得到

$$\log q_j^*(z_j) = \mathbb{E}_{q_{-j}}[\log p(D, z)] + \text{const},$$

其中 $\mathbb{E}_{q_{-j}}$ 對其他所有因子取平均，常數讓 q_j^* 正規化。（推導方式：把 $\mathcal{L}$ 寫成只含 q_j 的函數，它等於 $-D_{\mathrm{KL}}(q_j \| \tilde{q}_j)$ 加上常數，其中 $\log \tilde{q}_j = \mathbb{E}_{q_{-j}}[\log p(D, z)]$。）依序輪流更新各個 j，就是**座標上升變分推論**（CAVI）。每次更新都不會降低 ELBO，所以 CAVI 會收斂，但只收斂到局部最佳 [11]。對條件共軛的模型，每個 q_j^* 會與先驗同族，更新有閉式解。

實例：平均與精確度都未知的高斯。觀測 $x_n \sim \mathcal{N}(\mu, \tau^{-1})$，先驗 $\mu \mid \tau \sim \mathcal{N}(\mu_0, (\lambda_0 \tau)^{-1})$、$\tau \sim \mathrm{Gamma}(a_0, b_0)$。精確後驗已知（Normal–Gamma），因此可以為近似打分數。取 $q(\mu, \tau) = q(\mu) q(\tau)$，mean-field 更新規則給出 $q(\mu) = \mathcal{N}(\mu_N, \lambda_N^{-1})$ 與 $q(\tau) = \mathrm{Gamma}(a_N, b_N)$，其中 [1]

$$\mu_N = \frac{\lambda_0 \mu_0 + N \bar{x}}{\lambda_0 + N}, \qquad \lambda_N = (\lambda_0 + N)\, \mathbb{E}[\tau],$$
$$a_N = a_0 + \frac{N + 1}{2}, \qquad b_N = b_0 + \tfrac{1}{2}\, \mathbb{E}_\mu\Big[\textstyle\sum_n (x_n - \mu)^2 + \lambda_0 (\mu - \mu_0)^2\Big],$$

$\bar{x}$ 為樣本平均，$\mathbb{E}[\tau] = a_N / b_N$。兩個因子透過 $\mathbb{E}[\tau]$ 以及 $\mathbb{E}[\mu]$、$\mathbb{E}[\mu^2]$ 互相依賴，所以要反覆迭代。

```
import numpy as np
```

```
# Data x_n ~ N(mu, 1/tau); prior mu | tau ~ N(mu0, 1/(lam0 tau)), tau ~ Gamma(a0, b0).
rng = np.random.default_rng(4)
x = rng.normal(2.0, 0.5, size=30)
N, xbar = len(x), x.mean()
mu0, lam0, a0, b0 = 0.0, 1.0, 1.0, 1.0

# Mean field q(mu, tau) = q(mu) q(tau) with q(mu) = N(mu_N, 1/lam_N), q(tau) = Gamma(a_N,
 ↪  b_N).
E_tau = a0 / b0
for it in range(50):
    mu_N = (lam0 * mu0 + N * xbar) / (lam0 + N)
    lam_N = (lam0 + N) * E_tau
    a_N = a0 + (N + 1) / 2
    E_mu, E_mu2 = mu_N, mu_N**2 + 1 / lam_N
    b_N = b0 + 0.5 * (np.sum(x**2) - 2 * E_mu * np.sum(x) + N * E_mu2
                      + lam0 * (E_mu2 - 2 * mu0 * E_mu + mu0**2))
    E_tau = a_N / b_N

# Exact posterior is Normal-Gamma: compare the marginal moments of mu.
lam_post = lam0 + N
mu_post = (lam0 * mu0 + N * xbar) / lam_post
a_post = a0 + N / 2
b_post = b0 + 0.5 * (np.sum((x - xbar) ** 2) + lam0 * N * (xbar - mu0) ** 2 / lam_post)
var_mu_exact = b_post / ((a_post - 1) * lam_post)       # Student-t marginal variance
print("E[mu]   VI %.4f  exact %.4f" % (mu_N, mu_post))
print("Var[mu] VI %.5f  exact %.5f" % (1 / lam_N, var_mu_exact))
print("E[tau]  VI %.3f   exact %.3f" % (a_N / b_N, a_post / b_post))
```

平均一致（皆為 1.8668），$\mathbb{E}[\tau]$ 也一致（2.146），但 μ 的變異數在 VI 下是 0.01503，精確值是 0.01603：小了約 6%。在精確後驗中，μ 與 τ 相依（μ 的分散程度隨 $1/\tau$ 縮放），其邊際分布是尾部較重的 Student-t；因子化的高斯無法表達這兩種性質。

這種低估是系統性的。對精確度矩陣為 Λ 的高斯目標，最佳的 mean-field 因子 q_j 的變異數是 $1/\Lambda_{jj}$，也就是條件變異數，永遠不大於邊際變異數。圖 4 顯示極端情況。

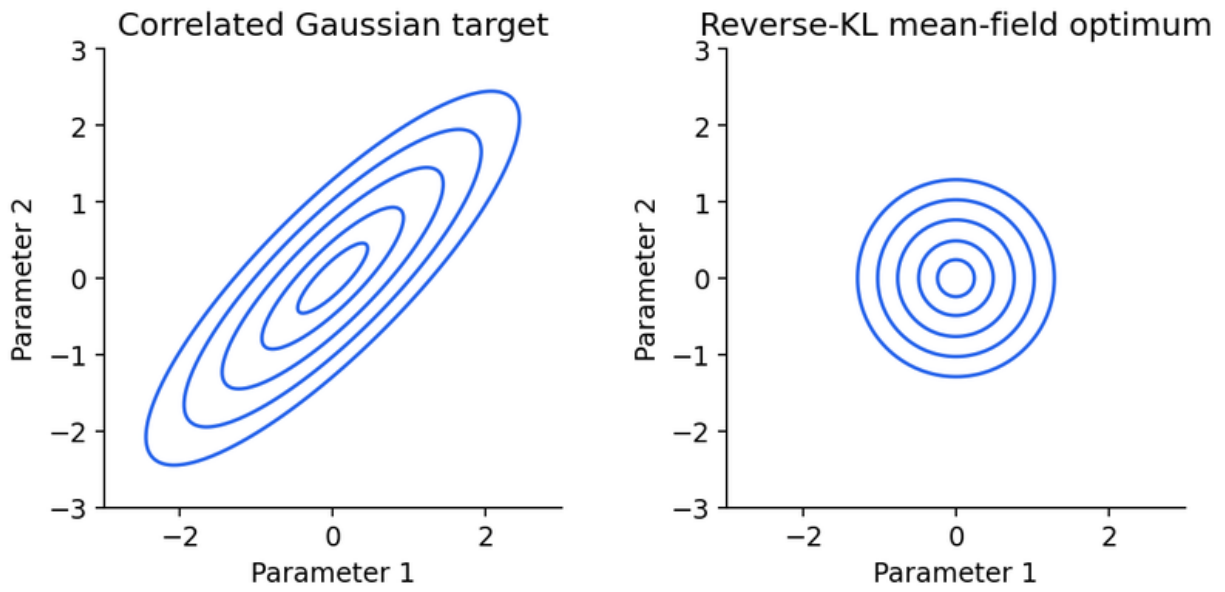


圖 *4* — 原創解析示意：目標的邊際變異數為一，相關係數為 *0.85*。在 *reverse KL* 下，最佳高斯 *mean-field* 的各因子變異數為 $1 - 0.85^2 \approx 0.28$，展示相關性的遺失與邊際不確定性的低估。等高線表示相對密度，不是模型效能。

用哪個 KL？找峰與覆蓋質量

KL 不對稱，方向不同答案就不同。標準 VI 使用的**反向 KL** $D_{\mathrm{KL}}(q\|p)$ 在 q 下取期望：它重罰把 q 的質量放在 p 極小的地方，卻不在意 q 忽略 p 的某些部分。結果是**找峰**（mode seeking）且**分散不足**。**正向 KL** $D_{\mathrm{KL}}(p\|q)$ 在 p 下取期望：它重罰漏掉 p 有質量的任何區域，因此 q 會伸展開來涵蓋一切，即**覆蓋質量**（mass covering），也可能把質量放在峰與峰之間的低谷。在高斯這類指數族之內，最小化正向 KL 恰好就是**動差匹配**：q 取 p 的平均與共變異數。

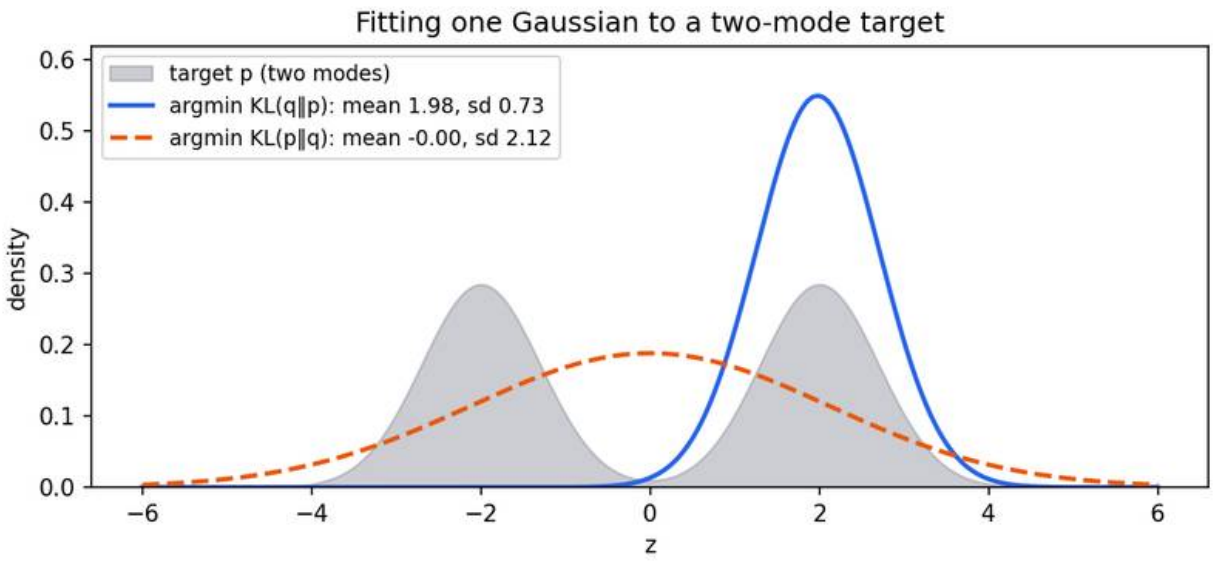


圖 5 — 以單一高斯擬合 $N(-2, 0.7^2)$ 與 $N(2, 0.7^2)$ 的等權混合。最小化反向 $KL(q\|p)$ 會鎖定其中一個峰（藍色；鎖定哪個峰取決於初始值）。最小化正向 $KL(p\|q)$ 則匹配混合分布的平均與變異數、涵蓋兩個峰，卻把峰值放在目標幾乎沒有質量之處（橘色虛線）。由本篇的圖形腳本以數值計算。

兩個方向都不是「唯一正確」的。反向 KL 好計算，答案銳利好用，但低估不確定性。正向 KL 是期望傳播背後的目標，也是以最大概似把模型擬合到資料時的目標；當漏掉一個峰的代價很高時，它比較安全。

隨機與黑箱 VI

CAVI 需要共軛性，而且每次更新都要掃過整個資料集。現代 VI 改用 ELBO 的**隨機梯度**，幾乎適用於任何模型，也能透過小批次（minibatch）抽樣處理大量資料 [11, 12]。$\nabla_\phi \mathcal{L}(\phi)$ 主要有兩種梯度估計量：

- **Score function（REINFORCE）估計量**，黑箱 VI 採用此法 [13]：

$$\nabla_\phi \mathcal{L} = \mathbb{E}_{q_\phi}\Big[\nabla_\phi \log q_\phi(z)\,\big(\log p(D,z) - \log q_\phi(z)\big)\Big].$$

它只需要計算 $\log p(D,z)$，也適用於離散的 z，但變異數高，需要控制變量（control variates）。

- **重參數化（pathwise）估計量**。把 z 寫成 $z = g_\phi(\varepsilon)$，其中雜訊 ε 來自固定分布，例如 $z = m + \sigma \odot \varepsilon$、$\varepsilon \sim \mathcal{N}(0, I)$。則

$$\nabla_\phi \mathcal{L} = \mathbb{E}_{\varepsilon}\Big[\nabla_\phi\big(\log p(D, g_\phi(\varepsilon)) - \log q_\phi(g_\phi(\varepsilon))\big)\Big].$$

它需要 $\log p$ 可微、z 連續，但變異數通常低得多。VAE 讓這個估計量廣為人知 [15]。

對參數為 $\phi = (m, s)$、$\sigma = e^s$ 的對角高斯，熵項等於 $\sum_j s_j$ 加上常數，所以它對 s 的梯度就是 1。套用到邏輯斯迴歸後驗：

```
import numpy as np

rng = np.random.default_rng(0)
n = 40
X = np.c_[np.ones(n), rng.normal(size=n)]
y = (rng.random(n) < 1 / (1 + np.exp(-X @ np.array([-0.5, 1.5])))).astype(float)
prior_var = 4.0

def grad_log_joint(W):                  # W: (S, 2) -> gradients (S, 2)
    p = 1 / (1 + np.exp(-W @ X.T))      # (S, n)
    return (y - p) @ X - W / prior_var

# Laplace: Newton's method to the MAP, then invert the Hessian.
w = np.zeros(2)
for _ in range(25):
    p = 1 / (1 + np.exp(-X @ w))
    H = X.T @ (X * (p * (1 - p))[:, None]) + np.eye(2) / prior_var
    w = w + np.linalg.solve(H, grad_log_joint(w[None])[0])
print("Laplace mean:", w.round(3), " sd:", np.sqrt(np.diag(np.linalg.inv(H))).round(3))

# Mean-field Gaussian VI with reparameterization: w = m + exp(s) * eps.
m, s = np.zeros(2), np.full(2, -1.0)
lr, S = 0.02, 64
for step in range(3000):
    eps = rng.normal(size=(S, 2))
    W = m + np.exp(s) * eps
    g = grad_log_joint(W)
    grad_m = g.mean(axis=0)                                  # d ELBO / d m
    grad_s = (g * eps).mean(axis=0) * np.exp(s) + 1.0        # + entropy term
    m += lr * grad_m
    s += lr * grad_s
print("VI mean:     ", m.round(3), " sd:", np.exp(s).round(3))
```

與網格的標準答案比較：平均 (−0.303, 1.631)、標準差 (0.366, 0.578)。程式輸出 Laplace 平均 (−0.291, 1.500)、標準差 (0.355, 0.556)；VI 平均 (−0.303, 1.630)、標準差 (0.358, 0.562)。VI 幾乎完全找回後驗平均，因為它對偏斜的後驗取了平均；Laplace 回報的是眾數，後驗偏斜時與平均不同。兩者都略為低估分散程度。40 個資料點時後驗接近高斯，差異不大；資料更少或類別可分時，差異會迅速擴大。

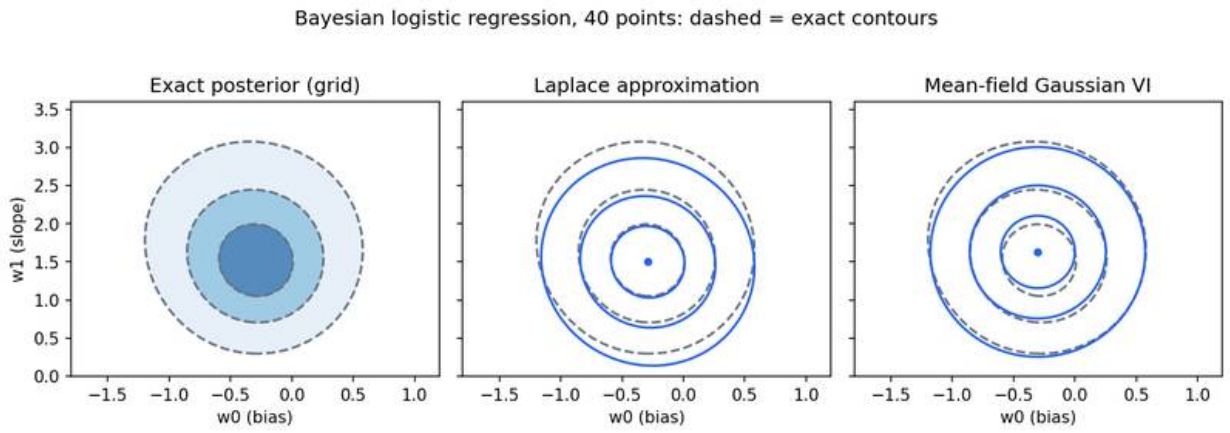


圖 *6* — 以 *40* 個合成資料點做兩權重的貝氏邏輯斯迴歸。灰色虛線：網格算出的精確後驗等高線。藍色：*Laplace*（以 *MAP* 為中心，往較小的斜率偏移）與 *mean-field* 高斯 *VI*（中心接近真正的後驗平均，略微太窄）。由本篇的圖形腳本計算。

Laplace 近似與期望傳播

Laplace：在峰頂放一個高斯

白話版。找到後驗的最高點，然後擬合一個在該處曲率相同的高斯。

精確版。在梯度為零的眾數 $\hat{z}$ 附近，把 $\log \tilde{p}(z)$ 展開到二階。令 $H = -\nabla^2 \log \tilde{p}(\hat{z})$ 為負對數後驗的 Hessian，

$$q(z) = \mathcal{N}(z \mid \hat{z},\, H^{-1}), \qquad \log p(D) \approx \log \tilde{p}(\hat{z}) + \frac{d}{2}\log 2\pi - \frac{1}{2}\log |H|,$$

其中 d 是 z 的維度。第二個式子是 **Laplace 證據近似**；只保留隨 N 成長的項，就得到PR 07 中用來選擇混合成分數的 BIC 準則 [1]。Laplace 很便宜（一次最佳化加一個 Hessian），而且資料越多越準確，因為正則模型的後驗會越來越接近高斯。它在下列情況會失敗：後驗偏斜、眾數位於邊界上（例如變異數趨近零）、Hessian 奇異，以及多峰後驗（它只描述其中一個峰）。現代的「深度網路 Laplace」方法只對最後一層使用 Laplace，或採用結構化的 Hessian 近似，為訓練好的神經網路提供便宜的不確定性估計。

期望傳播

期望傳播（EP）[14] 處理可分解為 $p(z \mid D) \propto \prod_i f_i(z)$ 的後驗，例如每個資料點一個因子。它用指數族中的簡單**站點**（site）$\tilde{f}_i(z)$ 近似每個因子，使 $q(z) \propto \prod_i \tilde{f}_i(z)$，並逐一修正站點：

1. 移除站點 i，形成**空腔**（cavity）分布 $q^{\setminus i}(z) \propto q(z)/\tilde{f}_i(z)$。
2. 形成**傾斜**（tilted）分布 $\hat{p}_i(z) \propto f_i(z)\, q^{\setminus i}(z)$，即空腔乘上精確的因子。
3. 把新的 q 設為分布族中與 $\hat{p}_i$ 動差相同的成員，也就是最小化正向 KL $D_{\mathrm{KL}}(\hat{p}_i \| q)$。
4. 更新站點：$\tilde{f}_i \propto q^{\text{new}}/q^{\setminus i}$。

由於每一步在局部使用正向 KL，EP 傾向於覆蓋質量，變異數往往比 mean-field VI 校準得更好，例如在高斯過程分類中。代價是它沒有保證會改善的全域目標函數，可能不收斂，也可能產生無效（負變異數）的站點，需要阻尼（damping）[1, 14]。

證據與複雜度：貝氏版奧坎剃刀

白話版。一個什麼都能解釋的模型，把先驗的賭注分散得很薄，所以解釋你實際看到的資料時拿不到多少功勞。一個押注在這類資料上的較簡單模型，反而能拿到較多功勞。

精確版。模型 $\mathcal{M}$ 的證據是把概似對先驗積分：

$$p(D \mid \mathcal{M}) = \int p(D \mid \theta, \mathcal{M})\, p(\theta \mid \mathcal{M})\, d\theta.$$

彈性大的模型可以達到很高的峰值概似，但如果它大部分的先驗質量都落在擬合很差的參數上，積分仍然很小。Laplace 公式讓這一點一目了然：證據等於峰值乘上一個比較後驗體積與先驗體積的「奧坎因子」（Occam factor）。由此有兩點提醒。第一，證據取決於先驗，甚至取決於幾乎不影響後驗的面向，例如模糊先驗有多寬。第二，它回答的是「哪個模型最能預料到這份資料？」，這與「哪個模型最能預測新資料？」相關但不相同，後者由保留驗證回答（PR 09）。跨模型比較 ELBO 也要小心：每個 ELBO 與其證據的差距是不同且未知的 KL。

從 ELBO 到 VAE 與擴散模型

本章的工具是 ML 系列中兩類深度生成模型的骨幹。

VAE 是攤銷式（amortized）變分推論。變分自編碼器以潛在碼 z、先驗 $p(z)$ 與神經網路解碼器 $p_\theta(x \mid z)$ 為影像 x 建模。它不為每張影像各自最佳化一組變分參數，而是讓編碼器網路直接輸出這些參數：$q_\phi(z \mid x) = \mathcal{N}(\mu_\phi(x), \operatorname{diag} \sigma^2_\phi(x))$。這就是**攤銷式推論**：一個網路學會對任何輸入做推論。每張影像的目標函數正是 ELBO：

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q_\phi(z \mid x)}[\log p_\theta(x \mid z)] - D_{\mathrm{KL}}(q_\phi(z \mid x) \,\|\, p(z)),$$

並以重參數化梯度對 θ 與 ϕ 聯合訓練 [15, 16]。它已知的弱點正是上面談過的：**攤銷差距**（編碼器的答案比逐張最佳化差）、反向 KL 的分散不足，以及會造成模糊的高斯解碼器。完整說明與程式碼見ML 06。

擴散模型以 ELBO 訓練，並以 Langevin 式動力學取樣。去噪擴散機率模型固定一個逐步加入高斯雜訊的前向過程 $x_0 \to x_1 \to \dots \to x_T$，並學習反向過程 $p_\theta(x_{t-1} \mid x_t)$。前向的含雜訊狀態扮演潛在變數的角色，前向過程則是固定的變分後驗；訓練目標是 $-\log p_\theta(x_0)$ 的變分上界，常用的雜訊預測損失函數則是此上界的重新加權版本 [17]。Score-based 觀點 [18] 改為學習 $s_\theta(x, t) \approx \nabla_x \log p_t(x)$，也就是含雜訊資料對數密度的梯度，並透過積分反向時間的隨機微分方程來產生樣本；其 predictor–corrector 取樣器明確地穿插 Langevin MCMC 步驟，以 s_θ 取代 $\nabla \log \tilde{p}$。與本章前面的 Langevin 更新相比，唯一的差別是 score 是從資料學來的，而不是由公式算出來的。見ML 07。

如何選擇方法

方法	回傳什麼	優點	主要失敗模式
網格／數值積分	精確數值	標準答案	隨維度指數成長
重要性取樣	加權樣本、證據	簡單、可平行、能給出 $p(D)$	q 漏掉 p 時權重崩潰
隨機漫步 MH	相關樣本	只需要 $\tilde{p}$	高維時很慢；需調參
Gibbs	相關樣本	條件分布容易時不需調參	變數相關時很慢
HMC ／ NUTS	相關樣本	能擴展到大量連續參數	需要梯度；離散變數；divergence
Mean-field VI	分布 q	快速、可擴展、適合小批次	分散不足；局部最佳
Laplace	眾數上的高斯	便宜；可估計證據	偏斜、邊界、多峰
EP	高斯／指數族	變異數常較準	可能不收斂

合理的工作流程是：先用最佳化方法（MAP 或 VI）求速度，在較小版本的問題上用謹慎的取樣器對照檢查，再用已知參數模擬資料，確認整條流程能還原它應該還原的東西。

現代觀點

值得一讀的綜述。 Blei、Kucukelbir 與 McAuliffe 寫給統計學者的綜述 [11] 是 VI 的標準入門：它推導 ELBO 與 CAVI、完整走過高斯混合的例子、介紹隨機 VI，並明確指出 VI 以犧牲 MCMC 的漸近精確性換取速度，且傾向低估後驗變異數。Zhang、Bütepage、Kjellström 與 Mandt [12] 把後續文獻整理成四個方向：可擴展的 VI（隨機與分散式）、適用於非共軛模型的通用 VI（黑箱與重參數化梯度）、更準確的 VI（反向 KL 以外的散度、更豐富的分布族），以及攤銷式 VI（推論網路、VAE）。Jordan、Ghahramani、Jaakkola 與 Saul [10] 是圖模型變分方法的經典導論，至今仍是凸對偶觀點最清楚的來源。取樣方面，Neal 的專章 [6] 是 HMC 的標準推導，Betancourt [8] 提供 HMC 在高維為何有效的幾何圖像，Hoffman 與 Gelman [7] 描述 NUTS，也就是現代機率程式系統的預設取樣器。Vehtari 等人 [9] 訂定了目前收斂診斷的標準。Kingma 與 Welling 的專書 [16] 把 VAE 當作攤銷式 VI 加以綜述，Yang 等人 [19] 則綜述擴散模型，包括其變分與 score-based 表述。

2023–2026 的現況。有三個明顯趨勢。

1. 機器學習進入貝氏計算。 Winter、Campbell、Lin、Srivastava 與 Dunson [20] 回顧了正規化流（normalizing flows）、貝氏 coreset、分散式推論與變分方法如何被用來加速後驗計算，以及這些方法的理論保證在哪些地方仍弱於傳統 MCMC。
2. 擴散的想法被用於取樣，而不只是生成。即使只有未正規化的密度（沒有資料），仍然可以訓練擴散式的取樣器。Denoising Diffusion Samplers [21] 學習反轉一個加雜訊的過程，使其終點落在只知道差一個常數的目標上，得到近似樣本與正規化常數的估計；它與最佳控制及 Schrödinger bridge 有所連結，是快速成長的學習式取樣器家族中具代表性的一員。
3. 重視工作流程而非單一演算法。實務上已轉向預設執行 NUTS、回報秩正規化 $\hat{R}$ 與 bulk ／ tail ESS [9] 的機率程式系統，並把 VI 或 Laplace 當作快速的第一步，而不是最終答案。

開放問題。為非常大的模型提供可靠的不確定性仍然困難：全批次 MCMC 太昂貴，而便宜的近似（mean-field VI、最後一層 Laplace、集成）彼此結論不一。多峰後驗會同時擊敗局部 VI 與單

一鏈。在沒有標準答案時評估近似的品質，大體上仍靠經驗法則；透過 ELBO 比較模型，也會受到未知近似差距的干擾。

重點整理

- 後驗在差一個常數的意義下容易計算、卻難以積分；網格成本是 G^d，超過幾個維度就非近似不可。
- Monte Carlo 誤差以 $1/\sqrt{S}$ 下降且與維度無關，但前提是能從正確的分布抽樣。重要性取樣修正錯誤的分布，權重的 ESS 告訴你修正何時已經崩潰。
- Metropolis–Hastings 只需要 $\tilde{p}$ 的比值，非對稱提議則需加上 Hastings 修正。Gibbs 是接受率為 1 的 MH。HMC 利用梯度走得更遠。一定要記錄被拒絕時的狀態、跑多條鏈，並檢查 $\hat{R}$ 與 ESS。
- VI 最大化 ELBO，等同最小化對後驗的反向 KL。它快速、可擴展、會找峰，並傾向低估變異數，在 mean field 下尤其如此。
- Laplace 在眾數上擬合高斯，提供便宜的證據估計（以及 BIC）；EP 在局部匹配動差，變異數常較準，但可能不收斂。
- 證據實現了奧坎剃刀，但取決於先驗，回答的問題也與保留資料上的預測不同。
- VAE 是搭配重參數化梯度的攤銷式 VI；擴散模型以變分上界訓練，並以學習到的 score 驅動類似 Langevin 的動力學來取樣。

練習

1. **手推 ELBO**。從 $D_{\mathrm{KL}}(q(z) \| p(z \mid D))$ 出發，推導 $\log p(D) = \mathcal{L}(q) + D_{\mathrm{KL}}$。再證明 $q(z) = p(z \mid D)$ 時下界是緊的，並說明這如何讓 EM 成為最大化 ELBO 的特例。

   提示

   代入 $\log p(z \mid D) = \log p(D, z) - \log p(D)$，並因為 $\log p(D)$ 與 z 無關而把它移出期望值。q 等於後驗時 KL 為零。EM 的 E step 把 q 設為目前參數下潛在變數的後驗（下界變緊）；M step 固定 q，對參數最大化 $\mathcal{L}$。

2. **Hastings 修正**。你對一個正值參數使用對數常態隨機漫步提議：$z' = z e^{\eta}$，$\eta \sim \mathcal{N}(0, s^2)$。計算 $q(z' \mid z)$ 與比值 $q(z \mid z')/q(z' \mid z)$。若直接使用 Metropolis 原本的規則，會出什麼問題？

   提示

   $q(z' \mid z) = \frac{1}{z' s\sqrt{2\pi}} \exp(-(\log z' - \log z)^2/(2s^2))$。指數部分對稱，所以比值為 z'/z。省略它會讓鏈從正比於 $\tilde{p}(z)/z$ 的密度取樣，偏向較小的值。

3. **步長與混合**。在 Metropolis 程式中，把提議標準差改為 0.02，再改為 8。對每種情況回報接受率、lag-1 自相關，並用診斷小節的公式估計 N_{eff}（在第一個負的自相關處截斷總和）。哪個步長每次迭代得到的有效樣本最多？

提示

對照圖 2。極小的步長接受率接近 1，但自相關也接近 1，所以 N_{eff} 非常小，鏈甚至可能還沒到達目標。極大的步長接受率低，會長時間重複同一個值。中間的步長（約為目標的標準差）能得到多得多的有效樣本。

4. **斜脊上的 Gibbs**。以 $\rho = 0.99$ 重新執行 Gibbs 取樣器。先預測 z_1 的 lag-1 自相關與 N_{eff}/N，再用模擬驗證。提出並實作一個修正方法。

提示

z_1 鏈是係數為 $\rho^2 \approx 0.98$ 的自迴歸過程，所以 $N_{\text{eff}}/N = (1-\rho^2)/(1+\rho^2) \approx 0.01$。修正方法：把 (z_1, z_2) 當成一個區塊一起抽，或旋轉到 $u = (z_1+z_2)/\sqrt{2}$、$v = (z_1-z_2)/\sqrt{2}$，兩者在目標下彼此獨立。

5. **Mean field 低估變異數**。對高斯目標 $\mathcal{N}(0, \Sigma)$，精確度 $\Lambda = \Sigma^{-1}$，用 mean-field 更新規則證明最佳的 q_j 是變異數為 $1/\Lambda_{jj}$ 的高斯。證明 $1/\Lambda_{jj} \le \Sigma_{jj}$，並在變異數為一、相關係數 0.85 時計算兩者。

提示

$\log p = -\tfrac{1}{2} z^\top \Lambda z +$ 常數；對 q_{-j} 取期望，保留含 z_j 的項：$-\tfrac{1}{2}\Lambda_{jj} z_j^2 - z_j \sum_{k \neq j} \Lambda_{jk} \mathbb{E}[z_k]$，這是精確度為 Λ_{jj} 的高斯。$1/\Lambda_{jj}$ 是給定其餘變數時 z_j 的條件變異數，由 Schur 補數可知它等於 Σ_{jj} 減去一個非負項。$\rho = 0.85$ 時：約 0.28 對 1。

6. **用重要性取樣估計證據**。對兩權重邏輯斯迴歸，分別以 (a) 先驗作為提議分布、(b) 共變異數放大 1.5 倍的 Laplace 高斯作為提議分布，以重要性取樣估計 $\log p(D)$。把兩者與網格值 -22.117 及權重 ESS 比較。在 50 維時你會信任哪一個？為什麼？

提示

估計值為 $\log \frac{1}{S}\sum_s \tilde{p}(z_s)/q(z_s)$，用 log-sum-exp 計算。先驗比後驗寬得多，大部分樣本權重微不足道，ESS 只占 S 的一小部分；以 Laplace 為基礎的提議分布落在後驗上，ESS 大得多。在高維時，先驗與後驗的落差呈指數擴大，以先驗為提議分布會變得毫無用處，而擬合良好、尾部略重的提議分布仍然可行。

參考文獻

1. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006, Sections 4.4 (Laplace), 10.1–10.2 and 10.7 (variational inference, EP) and 11.1–11.5 (sampling). 官方書籍頁面與 PDF
2. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2009, Chapter 2 (Bayes decision theory and parameter estimation) and Appendix C (cost function optimization)。這些是先備知識，並非與本章內容完全相同。doi
3. N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller and E. Teller, “Equation of state calculations by fast computing machines,” *The Journal of Chemical Physics*, vol. 21, no. 6, pp. 1087–1092, 1953. doi

4. W. K. Hastings, "Monte Carlo sampling methods using Markov chains and their applications," *Biometrika*, vol. 57, no. 1, pp. 97–109, 1970. doi
5. S. Geman and D. Geman, "Stochastic relaxation, Gibbs distributions, and the Bayesian restoration of images," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-6, no. 6, pp. 721–741, 1984. doi
6. R. M. Neal, "MCMC using Hamiltonian dynamics," in S. Brooks, A. Gelman, G. Jones and X.-L. Meng (eds.), *Handbook of Markov Chain Monte Carlo*, Chapman and Hall/CRC, 2011 (arXiv: 1206.1901). arXiv
7. M. D. Hoffman and A. Gelman, "The No-U-Turn Sampler: Adaptively setting path lengths in Hamiltonian Monte Carlo," *Journal of Machine Learning Research*, vol. 15, pp. 1593–1623, 2014. JMLR
8. M. Betancourt, "A conceptual introduction to Hamiltonian Monte Carlo," arXiv:1701.02434, 2017. arXiv
9. A. Vehtari, A. Gelman, D. Simpson, B. Carpenter and P.-C. Bürkner, "Rank-normalization, folding, and localization: An improved $\hat{R}$ for assessing convergence of MCMC (with discussion)," *Bayesian Analysis*, vol. 16, no. 2, 2021. doi
10. M. I. Jordan, Z. Ghahramani, T. S. Jaakkola and L. K. Saul, "An introduction to variational methods for graphical models," *Machine Learning*, vol. 37, no. 2, pp. 183–233, 1999. doi
11. D. M. Blei, A. Kucukelbir and J. D. McAuliffe, "Variational inference: A review for statisticians," *Journal of the American Statistical Association*, vol. 112, no. 518, pp. 859–877, 2017. doi
12. C. Zhang, J. Bütepage, H. Kjellström and S. Mandt, "Advances in variational inference," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, no. 8, pp. 2008–2026, 2019. doi
13. R. Ranganath, S. Gerrish and D. M. Blei, "Black box variational inference," in *Proc. International Conference on Artificial Intelligence and Statistics* (*AISTATS*), PMLR vol. 33, 2014. PMLR
14. T. P. Minka, "Expectation propagation for approximate Bayesian inference," in *Proc. Conference on Uncertainty in Artificial Intelligence* (*UAI*), 2001 (arXiv:1301.2294). arXiv
15. D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," arXiv:1312.6114, 2013. arXiv
16. D. P. Kingma and M. Welling, "An introduction to variational autoencoders," *Foundations and Trends in Machine Learning*, vol. 12, no. 4, pp. 307–392, 2019. arXiv
17. J. Ho, A. Jain and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2020 (arXiv:2006.11239). arXiv
18. Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon and B. Poole, "Score-based generative modeling through stochastic differential equations," in *Proc. International Conference on Learning Representations* (*ICLR*), 2021. arXiv
19. L. Yang, Z. Zhang, Y. Song, S. Hong, R. Xu, Y. Zhao, W. Zhang, B. Cui and M.-H. Yang, "Diffusion models: A comprehensive survey of methods and applications," *ACM Computing Surveys*, vol. 56, no. 4, 2023. doi
20. S. Winter, T. Campbell, L. Lin, S. Srivastava and D. B. Dunson, "Machine learning and the future of Bayesian computation," arXiv:2304.11251, 2023. arXiv
21. F. Vargas, W. Grathwohl and A. Doucet, "Denoising diffusion samplers," in *Proc. International Conference on Learning Representations* (*ICLR*), 2023 (arXiv:2302.13834). arXiv

PR 09・集成、驗證與分群有效性

推導偏差–變異數分解、bagging、隨機森林、AdaBoost 與梯度提升，再以交叉驗證、bootstrap、巢狀與群組隔離驗證及成對統計檢定，誠實估計並比較辨識器。

網站版本: https://blog.shyandram.dev/zh-tw/learn/pattern-recognition/pr-09-ensembles-validation/

你將學到

- 推導平方誤差的偏差–變異數分解（bias–variance decomposition），並用它說明為什麼平均能幫助不穩定的學習器。
- 說明決策樹如何切分特徵空間，以及 bagging 與隨機森林（random forest）如何降低其變異數，包括 out-of-bag 估計。
- 從指數損失推導 AdaBoost，連結到梯度提升（gradient boosting），並區分 boosting、投票、貝氏模型平均與 stacking。
- 以保留法（hold-out）、k 折、群組隔離與巢狀交叉驗證，以及 bootstrap（.632 與.632+），估計整個辨識程序的表現。
- 以 McNemar 檢定、5×2cv 與修正重抽樣 t 檢定，以及 Friedman/Nemenyi 程序比較兩個以上的分類器，並知道各自何時成立。
- 同時評估決策與機率，並把分群有效性視為一種模型選擇問題。

先看全貌

單一辨識器犯錯的原因有兩種，性質截然不同。有時它太簡單，表達不了真正的決策規則；有時它太有彈性，把這份訓練集的偶然細節也背了下來。集成方法組合許多模型，分別對付第二種問題（bagging、隨機森林）或第一種問題（boosting）。驗證則是故事的另一半：一旦我們能建立大量模型、調整大量旋鈕，就需要可信的方法估計「被選中的程序」在新資料上的表現，並判斷某個程序是否真的比另一個好。

> **用最簡單的話說**
>
> 想像大家一起猜罐子裡有幾顆糖。一位朋友的猜測可能差很多；但如果十位朋友**各自獨立**猜，再取平均，離譜的猜測會互相抵消，平均通常更接近。可是如果十位朋友都只是照抄最大聲那位的答案，平均就完全沒有幫助。現在假設你想知道誰最會猜：你必須拿一個他們**沒看過**的新罐子來考；用他們已經研究過的罐子來考，每個人看起來都會很厲害。

為什麼重要：樹集成至今仍是表格型特徵（包括手工設計的影像描述子）上最強的方法之一 [23]；深度網路集成是取得不確定性估計的標準作法 [24]；而幾乎每一篇比較辨識器的論文，都建立在交叉驗證與統計檢定之上。Theodoridis 與 Koutroumbas 在第 4 章討論決策樹、分類器組合與 boosting，第 10 章（「Supervised Learning: The Epilogue」）討論誤差估計與比較，第 16 章討論分群有效性 [1]。Bishop 第 14 章區分了數種數學上不同的模型組合方式，第 3.2 節則推導偏差–變異數分解 [2]。分類器本身已在PR 03 與PR 04 介紹過。

偏差、變異數與雜訊

白話版。想像用同一個學習演算法，在許多份來自同一來源的不同訓練集上重新訓練。**平均**預測仍可能偏離真值，這個系統性的偏離就是**偏差**（bias）。個別預測會在平均值周圍散開，這個散布就是**變異數**（variance）。即使是完美的預測器，也無法消除標籤本身的隨機性，那就是**雜訊**（noise）。

精確版。令目標為 $y = f(x) + \varepsilon$，其中 f 是未知的迴歸函數，ε 是雜訊，滿足 $\mathbb{E}[\varepsilon] = 0$、$\mathrm{Var}(\varepsilon) = \sigma^2$，且與其他一切獨立。學習演算法把隨機的訓練集 D 映射成預測器 $\hat{f}_D$。記 $\bar{f}(x) = \mathbb{E}_D[\hat{f}_D(x)]$ 為對訓練集平均後的預測。在固定輸入 x 上，對訓練集與測試雜訊取期望的平方誤差為

$$\begin{aligned}\mathbb{E}_{D,\varepsilon}\big[(y - \hat{f}_D(x))^2\big] &= \mathbb{E}\big[(\varepsilon + f(x) - \bar{f}(x) + \bar{f}(x) - \hat{f}_D(x))^2\big] \\ &= \underbrace{\sigma^2}_{\text{noise}} + \underbrace{\big(f(x) - \bar{f}(x)\big)^2}_{\text{bias}^2} + \underbrace{\mathbb{E}_D\big[(\hat{f}_D(x) - \bar{f}(x))^2\big]}_{\text{variance}}.\end{aligned}$$

三個交叉項都會消失：ε 期望為零且與 D 獨立；$f(x) - \bar{f}(x)$ 是常數；而依 $\bar{f}$ 的定義，$\mathbb{E}_D[\bar{f}(x) - \hat{f}_D(x)] = 0$。再對輸入分布 $p(x)$ 取平均，就得到期望測試誤差。Geman、Bienenstock 與 Doursat 把這個張力稱為**偏差／變異數兩難**（bias/variance dilemma）[3]：讓模型更有彈性，通常會降低偏差、提高變異數，最佳複雜度在兩者之間取得平衡。

下面的程式以暴力法，對雜訊正弦曲線上不同深度的迴歸樹估計每一項。程式抽出 200 份各含 60 點的訓練集；200 棵擬合樹的經驗平均與散布，給出偏差平方與變異數。

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor

rng = np.random.default_rng(0)
f = lambda x: np.sin(2 * np.pi * x)          # true regression function
sigma = 0.3                                   # noise standard deviation
x_test = np.linspace(0, 1, 200)[:, None]

def bias_variance(depth, n_sets=200, n=60):
    preds = np.empty((n_sets, len(x_test)))
    for s in range(n_sets):                   # many independent training sets
        x = rng.random((n, 1))
        y = f(x[:, 0]) + rng.normal(0, sigma, n)
        tree = DecisionTreeRegressor(max_depth=depth, random_state=0)
        preds[s] = tree.fit(x, y).predict(x_test)
    mean_pred = preds.mean(axis=0)
    bias2 = np.mean((mean_pred - f(x_test[:, 0])) ** 2)
    var = np.mean(preds.var(axis=0))
    return bias2, var

for depth in [1, 2, 3, 5, 8, None]:
    b2, v = bias_variance(depth)
    print(f"depth={str(depth):>4}  bias^2={b2:.4f}  variance={v:.4f}  "
          f"expected test MSE~{b2 + v + sigma**2:.4f}")
```

在我們的執行中，深度 1 的偏差平方約 0.074、變異數約 0.035；深度 3 的總和最低（偏差平方

約 0.003、變異數約 0.045）；完全長成的樹幾乎沒有偏差，但變異數約 0.092，和雜訊變異數 $\sigma^2 = 0.09$ 本身一樣大。圖 1 以視覺方式呈現同一個實驗。

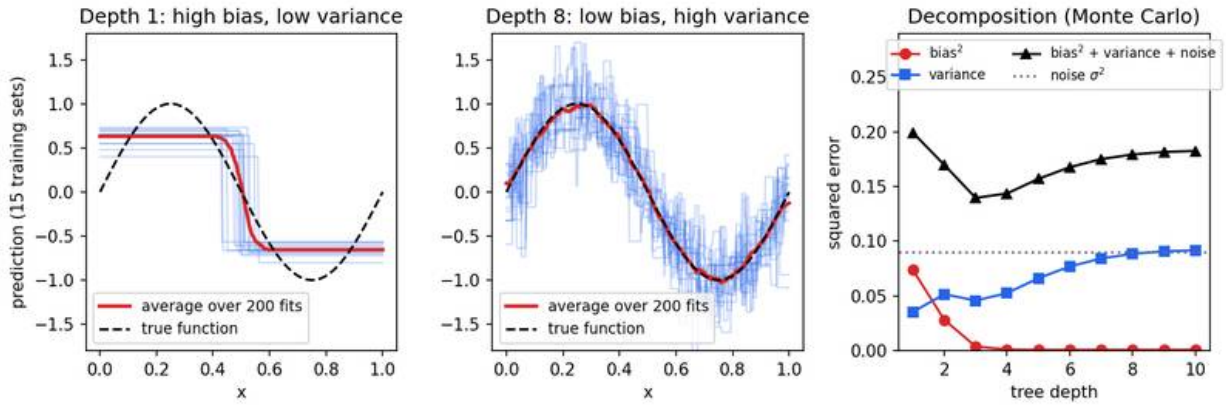


圖 *1* — 迴歸樹在 $y = sin(2\pi x)$ + 雜訊（$\sigma = 0.3$，每份訓練集 *60* 點，共 *200* 份訓練集）上的偏差–變異數分解。淺樹總是錯在同一個方向（偏差）；深樹平均起來是對的，但每一棵都很吵（變異數）。由本章的腳本產生。

有三點提醒讓這張圖保持誠實。第一，乾淨的分解只適用於平方誤差；對 0–1 分類損失，偏差與變異數有好幾種互相競爭的定義，而且兩者可能以非加法方式交互作用，因此對分類器而言，請把分解當作直覺，而不是精確的帳目。第二，「複雜度」不只是參數數量：樹深、正則化強度、boosting 回合數與提早停止（early stopping）都會讓模型在偏差–變異數軸上移動。第三，極大的過參數化模型，其測試誤差曲線可能在容量超過內插點後先升再降，也就是 Belkin 等人記錄的「雙重下降」（double descent）形狀 [25]。分解在每一點上仍然成立；改變的是變異數隨容量增長的行為。

決策樹與空間切分

白話版。決策樹對特徵問一連串是非題，例如「特徵 3 是否小於 0.7？」，每個最終答案格（葉節點）存放一個預測。它是低偏差、高變異數學習器的經典例子，這也正是它成為集成方法最愛用的積木的原因。

精確版。決策樹以 $x_j \le b$ 等條件遞迴分割觀測。每個葉節點估計一個類別分布（分類）或一個常數（迴歸）。對類別比例為 p_k 的分類節點，**Gini 不純度**為

$$G = 1 - \sum_k p_k^2,$$

熵則為 $H = -\sum_k p_k \log p_k$。兩者在純節點為零，在均勻混合時最大。把父節點 P 分成子節點 L 與 R，分別含有父節點 n_P 個樣本中的 n_L 與 n_R 個，此分割以不純度下降量評分：

$$\Delta = G(P) - \frac{n_L}{n_P} G(L) - \frac{n_R}{n_P} G(R),$$

學習器選擇 Δ 最大的特徵與閾值。這是局部貪婪選擇：最好的單一分割不一定通往最好的樹。深樹能捕捉交互作用與非線性、與座標軸平行的邊界，但少數訓練點改變就可能讓整棵樹大幅改

變；這種不穩定，正是圖 1 中看到的高變異數。

深度限制、最小葉節點樣本數與成本–複雜度剪枝可以控制複雜度。由於分割只取決於每個特徵的排序，樹對單調尺度轉換不變，因此比距離模型較不受特徵尺度影響；但缺失值、特徵定義，以及只由訓練資料擬合的轉換，仍需要仔細處理。Theodoridis 與 Koutroumbas（第 4 章）與 Bishop（第 14.4 節）都把樹視為非線性分類器的一個家族 [1], [2]。

Bagging 與隨機森林

白話版。 Bagging（bootstrap aggregating）以「放回抽樣」從原始資料建立許多份訓練集，在每份上訓練一個不穩定的模型，再平均或投票。隨機森林再加入一個隨機來源：每次分割時只看隨機挑出的一部分特徵，讓樹之間更不一致，平均效果也就更好。

精確版。 Breiman 提出 bagging，作為改善**不穩定**程序的方法，也就是訓練集稍有擾動、輸出就大幅改變的程序 [4]。給定 B 份大小皆為 n 的 bootstrap 樣本 $D^{(1)}, \dots, D^{(B)}$，bagging 迴歸器為 $\hat{f}_{\text{bag}}(x) = \frac{1}{B}\sum_{b} \hat{f}_{D^{(b)}}(x)$，bagging 分類器則採多數決（或平均類別機率）。

為什麼平均有幫助？假設在固定的 x 上，M 個純量預測器的變異數都是 σ^2，任兩者的相關係數都是 ρ。展開總和的變異數：

$$\operatorname{Var}\Big(\frac{1}{M}\sum_{m=1}^{M} f_m\Big) = \frac{1}{M^2}\Big[M\sigma^2 + M(M-1)\rho\sigma^2\Big] = \sigma^2\left[\rho + \frac{1-\rho}{M}\right].$$

第二項隨 M 增大而消失，第一項 $\rho\sigma^2$ 卻不會：彼此相關的錯誤形成一個下限（圖 2 左）。平均基本上不改變偏差，所以 bagging 對深樹這類低偏差、高變異數的學習器幫助最大，對線性判別式或大 k 的 k 近鄰這類穩定學習器幫助很小。這個示意公式說明了相關錯誤為何限制改善幅度；它並不保證每個分類集成都會提高準確率。

隨機森林直接降低 ρ [5]。在每份 bootstrap 樣本上長樹時，每次分割只考慮隨機選出的 m_{try} 個特徵（分類問題常見的預設是 d 個特徵中取 $\sqrt{d}$ 個）。Breiman 證明森林的泛化誤差可以用個別樹的**強度**與樹之間的**相關性**來界定 [5]，這和變異數公式是同一個取捨：較弱但相關性較低的樹，可能組成更強的森林。

Out-of-bag（OOB）估計。 在 n 次放回抽樣中，某一筆觀測**沒有**被抽到的機率是 $(1-1/n)^n \to e^{-1} \approx 0.368$。因此每棵樹大約漏掉 37% 的資料，而每筆觀測都可以由大約三分之一沒見過它的樹來預測。彙整這些預測，幾乎不必額外成本就能得到 OOB 誤差估計 [5]。在圖 2 右，樹夠多之後，OOB 估計就會貼近測試誤差。但一般的 bootstrap 是對「列」重抽樣：如果好幾列屬於同一位受試者、書寫者或同一次錄製，一棵樹可能在某人的一張影像上訓練，又在同一人的另一張影像上做 OOB「測試」。這時對新群組部署而言，OOB 分數就會過度樂觀。

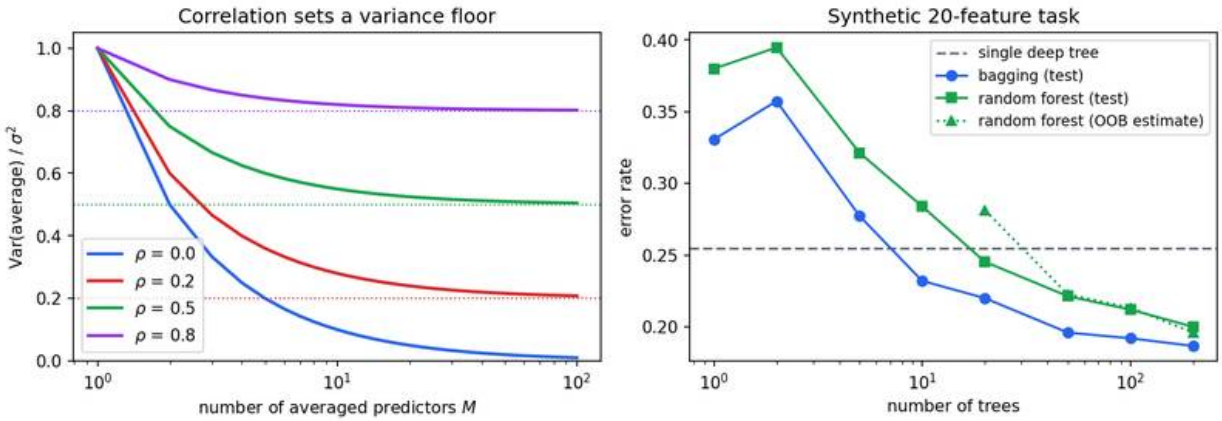


圖 2 — 左：M 個等相關預測器平均後的解析變異數。右：一個合成的 *20* 特徵任務（訓練 *750*、測試 *750* 筆）。兩種集成在有數十棵樹之後都勝過單一深樹；在這個特定任務上，單純 *bagging* 略優於隨機森林，提醒我們隨機特徵子集不保證一定有幫助。樹非常少（*1–2* 棵）時投票不穩定，也可能平手。

Boosting 改變下一個學習器的工作

白話版。 Bagging 的成員彼此獨立、可以平行訓練。Boosting 則**依序**訓練：每個新的弱學習器專注在目前集成仍然答錯的樣本上，最後的預測是加權總和。Bagging 主要降低變異數；boosting 主要降低偏差，把許多「只比亂猜好一點」的規則變成強分類器。

精確版。 Boosting 建立加法模型

$$F_M(x) = \sum_{m=1}^{M} \alpha_m h_m(x),$$

其中每個 h_m 是弱學習器（常用深度 1 的樹，即**樹樁** stump），α_m 是它的權重。對標籤 $y \in \{-1, +1\}$，類別預測為 $\operatorname{sign} F_M(x)$。

AdaBoost 演算法

Freund 與 Schapire 的 AdaBoost [6] 對訓練資料 (x_i, y_i)，$i = 1, \dots, n$，運作如下：

1. 從均勻權重 $w_i^{(1)} = 1/n$ 開始。
2. 對 $m = 1, \dots, M$：
   - 以權重 $w^{(m)}$ 擬合 h_m；
   - 計算加權誤差 $\epsilon_m = \sum_i w_i^{(m)} \mathbf{1}[h_m(x_i) \neq y_i] / \sum_i w_i^{(m)}$；
   - 令 $\alpha_m = \tfrac{1}{2} \log \frac{1-\epsilon_m}{\epsilon_m}$；
   - 更新 $w_i^{(m+1)} \propto w_i^{(m)} \exp\left(-\alpha_m y_i h_m(x_i)\right)$ 並重新正規化。
3. 輸出 $\operatorname{sign} \sum_m \alpha_m h_m(x)$。

被分錯的點滿足 $y_i h_m(x_i) = -1$，權重乘上 e^{α_m}；分對的點則乘上 $e^{-\alpha_m}$。$\epsilon_m < 0.5$ 的學習器得到正的投票權，而且越準，票越重。

從指數損失推導 AdaBoost

Friedman、Hastie 與 Tibshirani 證明 AdaBoost 就是以**指數損失** $L(y, F) = \exp(-yF)$ 進行的**前向逐步加法建模**（forward stagewise additive modeling）[7]。假設 F_{m-1} 已固定，我們加入一項 αh。訓練損失為

$$\sum_{i=1}^{n} \exp\big(-y_i[F_{m-1}(x_i) + \alpha h(x_i)]\big) = \sum_{i=1}^{n} w_i^{(m)} \exp\big(-\alpha y_i h(x_i)\big), \qquad w_i^{(m)} = \exp\big(-y_i F_{m-1}(x_i)\big).$$

權重 $w_i^{(m)}$ 與 α、h 都無關。把總和拆成分對與分錯的點：

$$\sum_i w_i^{(m)} e^{-\alpha y_i h(x_i)} = e^{-\alpha} \sum_{y_i = h(x_i)} w_i^{(m)} + e^{\alpha} \sum_{y_i \neq h(x_i)} w_i^{(m)} = (e^{\alpha} - e^{-\alpha}) \sum_i w_i^{(m)} \mathbf{1}[y_i \neq h(x_i)] + e^{-\alpha} \sum_i w_i^{(m)}.$$

第 *1* 步，選 h。對任何 $\alpha > 0$，第一個係數為正，所以最佳的 h 就是最小化**加權訓練誤差**的那一個，正好是「以權重 $w^{(m)}$ 擬合 h_m」。

第 *2* 步，選 α。除以 $\sum_i w_i^{(m)}$，並以 ϵ 表示正規化後的加權誤差。我們要最小化 $g(\alpha) = e^{-\alpha}(1-\epsilon) + e^{\alpha}\epsilon$。令 $g'(\alpha) = -e^{-\alpha}(1-\epsilon) + e^{\alpha}\epsilon = 0$，得到 $e^{2\alpha} = (1-\epsilon)/\epsilon$，也就是

$$\alpha_m = \frac{1}{2} \log \frac{1 - \epsilon_m}{\epsilon_m}.$$

第 *3* 步，更新權重。下一輪權重為 $w_i^{(m+1)} = \exp(-y_i F_m(x_i)) = w_i^{(m)} \exp(-\alpha_m y_i h_m(x_i))$，正是 AdaBoost 的更新式。演算法的每一步都能從指數損失的貪婪最小化推回來。

由此得到兩個結果。因為 $\mathbf{1}[yF < 0] \le e^{-yF}$，訓練誤差以平均指數損失為上界；採用最佳 α_m 時，只要 $\epsilon_m < 0.5$，每一回合都會把這個上界乘上 $2\sqrt{\epsilon_m(1-\epsilon_m)} < 1$，所以只要每個弱學習器都以某個差距勝過亂猜，訓練誤差就會指數下降 [6]。另外，期望指數損失的最小化者是 $F^*(x) = \frac{1}{2}\log\frac{P(y=1\mid x)}{P(y=-1\mid x)}$，即對數勝算（log-odds）的一半 [7]，因此 $\sigma(2F_M(x))$ 是一個機率估計（不一定校準良好）。指數權重也解釋了一個已知弱點：標錯的點會得到指數成長的權重，所以 AdaBoost 可能對標籤雜訊與離群值很敏感。

下面的程式用加權樹樁從零實作 AdaBoost，並與 scikit-learn 的 `AdaBoostClassifier` 比對。對兩類問題，scikit-learn 的 SAMME 變體使用 $\alpha = \log\frac{1-\epsilon}{\epsilon}$，是我們的兩倍；這會把 F 放大，但每個符號、以及重新正規化後的每個權重都不變。

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier

X, y01 = make_classification(n_samples=600, n_features=10, n_informative=4,
                             flip_y=0.05, random_state=1)
y = 2 * y01 - 1                                   # labels in {-1, +1}
```

```
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=1)

def adaboost_fit(X, y, M=100):
    n = len(y)
    w = np.full(n, 1 / n)                          # step 1: uniform weights
    stumps, alphas = [], []
    for m in range(M):
        h = DecisionTreeClassifier(max_depth=1, random_state=0)
        h.fit(X, y, sample_weight=w)               # step 2a: weighted fit
        pred = h.predict(X)
        err = np.sum(w * (pred != y)) / np.sum(w) # step 2b: weighted error
        err = np.clip(err, 1e-12, 1 - 1e-12)
        alpha = 0.5 * np.log((1 - err) / err)      # step 2c: learner weight
        w = w * np.exp(-alpha * y * pred)          # step 2d: reweight
        w /= w.sum()
        stumps.append(h); alphas.append(alpha)
    return stumps, np.array(alphas)

def adaboost_score(stumps, alphas, X):
    return sum(a * h.predict(X) for h, a in zip(stumps, alphas))

stumps, alphas = adaboost_fit(Xtr, ytr)
F_tr = adaboost_score(stumps, alphas, Xtr)
F_te = adaboost_score(stumps, alphas, Xte)
print("from scratch: train err", np.mean(np.sign(F_tr) != ytr),
      " test err", np.mean(np.sign(F_te) != yte))
print("exp-loss bound on train err:", np.mean(np.exp(-ytr * F_tr)))

ref = AdaBoostClassifier(DecisionTreeClassifier(max_depth=1, random_state=0),
                         n_estimators=100, random_state=0).fit(Xtr, ytr)
agree = np.mean(ref.predict(Xte) == np.sign(F_te))
print("sklearn test err", np.mean(ref.predict(Xte) != yte),
      " agreement with ours:", agree)
```

在 scikit-learn 1.6 下，我們的執行印出訓練誤差 0.076、指數損失上界 0.384，兩種實作的測試誤差都是 0.156，測試預測 100% 一致。圖 3 顯示 300 回合的曲線，以及樹樁在二維中拼出的階梯狀邊界。

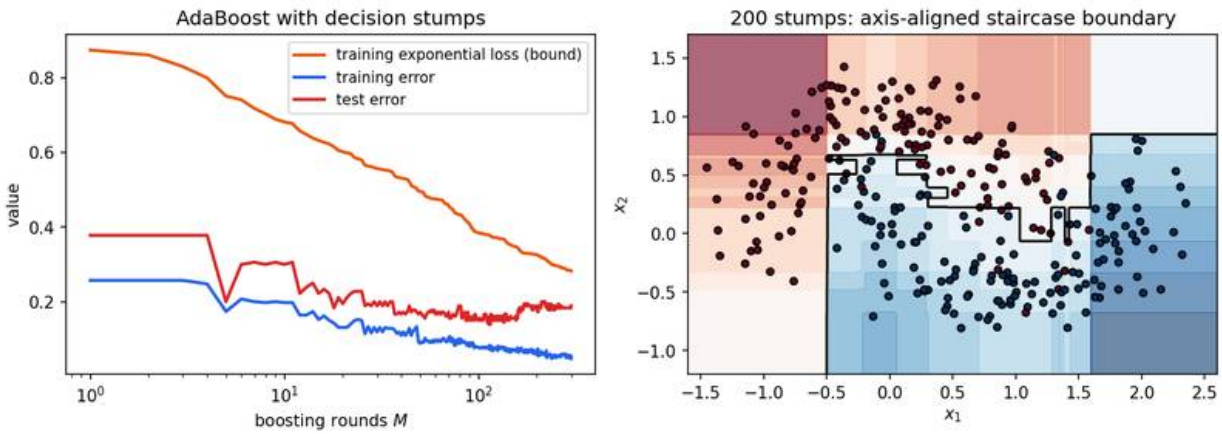


圖 3 — 以決策樹樁實作的 *AdaBoost*（我們從零寫的版本）。左：指數損失是訓練誤差的上界，而且持續縮小；測試誤差趨於平穩，在有標籤雜訊時可能稍微回升。右：把與座標軸平行的樹樁相加，在雙月形資料上得到階梯狀的決策邊界。

梯度提升

白話版。梯度提升把 AdaBoost 的想法推廣到任何可微分的損失：每一回合先算出每個訓練預測應該往哪個方向移動才能降低損失，再用一棵小樹去擬合這些方向。

精確版。 Friedman 的梯度提升機（gradient boosting machine）[8] 是在函數空間中做梯度下降。在第 m 回合，計算**偽殘差**（pseudo-residuals）

$$r_{im} = -\left.\frac{\partial L(y_i, F)}{\partial F}\right|_{F=F_{m-1}(x_i)},$$

以最小平方法把迴歸樹 h_m 擬合到 (x_i, r_{im})，用線搜尋選步長 ρ_m（或每個葉節點各取一個最佳常數，即 Friedman 的 TreeBoost），再以**學習率**（收縮，shrinkage）$0 < \nu \le 1$ 更新 $F_m = F_{m-1} + \nu \rho_m h_m$。對平方誤差 $L = \tfrac{1}{2}(y-F)^2$，偽殘差就是一般殘差 $y_i - F_{m-1}(x_i)$。對 $y \in \{0,1\}$、F 為對數勝算的二元對數損失，偽殘差是 $y_i - \sigma(F_{m-1}(x_i))$：標籤減去**預測機率**，而不是減去預測類別。若使用指數損失並搭配線搜尋，這個程序會化約為一種 AdaBoost。

學習率、回合數與樹的大小會互相影響：較小的 ν 需要更多回合，但通常泛化得更好；回合數以驗證資料上的提早停止來決定，絕不用最終測試集。XGBoost [9] 以正則化目標函數、二階（牛頓法）葉節點值、能處理稀疏資料的分割搜尋與加權分位數草圖（weighted quantile sketch），讓梯度提升樹變得快速且可擴展；scikit-learn 的 `HistGradientBoostingClassifier` 等以直方圖為基礎的實作也遵循相同原則 [26]。

平均不一定是同一種運算

許多方法都在「組合模型」，但它們回答的是不同問題。

方法	組合由什麼決定？	主要注意事項
投票或機率平均（bagging、森林）	對獨立訓練的預測器做固定、等權重的組合	錯誤彼此相關、校準不一致
Boosting	依序擬合；權重來自損失函數	標籤雜訊；需要提早停止
貝氏模型平均	模型假設的後驗機率	證據與先驗必須有意義
Stacking	學習一個以基底模型輸出為輸入的預測器	必須用 out-of-fold 預測來訓練
條件式混合（混合專家）	隨輸入改變的閘門或責任值	專家與閘門可能一起過擬合

貝氏模型平均（Bayesian model averaging，Bishop 第 14.1 節 [2]）使用

$$p(y \mid x, D) = \sum_m p(y \mid x, D, M_m)\, p(M_m \mid D),$$

其中 $p(M_m \mid D) \propto p(D \mid M_m) p(M_m)$ 是模型 M_m 的後驗機率。這些權重是後驗模型機率，不是任意的驗證準確率；資料充足時，它們往往集中在單一模型上。BMA 表達的是「資料是**哪一個**模

型產生的」這種不確定性；委員會或集成則是用幾個部件組成一個更豐富的模型 [2]。模型證據 $p(D \mid M_m)$ 與PR 08 的近似推論工具相連。

Stacking（堆疊泛化，stacked generalization，Wolpert [10]）則是**學習**一個從基底模型預測到目標的映射。它的訓練特徵必須來自沒有用這些樣本訓練過的基底模型；否則 stacker 學到的是不切實際、過擬合的簡單預測。標準做法是：把訓練集分成 K 折，每個基底模型在 $K-1$ 折上擬合並預測保留的那一折，收集這些 out-of-fold 預測當作特徵，在其上擬合一個簡單的後設學習器（常用正則化邏輯斯迴歸），最後再用全部訓練資料重新擬合各基底模型。

下面在 scikit-learn 內建的乳癌資料集（569 筆、30 個特徵）上比較各方法，所有方法都在同一個分層 5 折交叉驗證中執行，因此數字可以互相比較。`StackingClassifier` 會在每個外層訓練折內部，自行建立內層的 out-of-fold 預測（`cv=5`）。

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (BaggingClassifier, RandomForestClassifier,
                              HistGradientBoostingClassifier, StackingClassifier)

X, y = load_breast_cancer(return_X_y=True)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
logreg = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))
models = {
    "single tree": DecisionTreeClassifier(random_state=0),
    "bagged trees": BaggingClassifier(DecisionTreeClassifier(), n_estimators=100,
                                      random_state=0),
    "random forest": RandomForestClassifier(n_estimators=100, random_state=0),
    "hist. gradient boosting": HistGradientBoostingClassifier(random_state=0),
    "logistic regression": logreg,
}
models["stacking"] = StackingClassifier(
    estimators=[("rf", models["random forest"]),
                ("gb", models["hist. gradient boosting"]),
                ("lr", logreg)],
    final_estimator=LogisticRegression(max_iter=2000),
    cv=5, stack_method="predict_proba")
for name, model in models.items():
    s = cross_val_score(model, X, y, cv=cv, scoring="accuracy")
    print(f"{name:>24}: {s.mean():.3f} +/- {s.std():.3f}")
```

我們的執行得到平均準確率：單一決策樹 0.926、bagging 0.960、隨機森林 0.965、梯度提升 0.970、邏輯斯迴歸 0.979、stacking 0.979，各折標準差介於 0.009 到 0.022。由此得到兩個教訓。集成明顯修好了不穩定的單一決策樹。但在這個小而近乎線性可分的資料集上，一個正則化良好的**線性**模型已經和 stack 一樣好，而前四名之間的差距都在一個折標準差之內。它們之中是否有誰**顯著**較好，要交給本章稍後的統計檢定，而不是用眼睛看平均值。

驗證評估的是整個程序

白話版。只有當測試資料完全沒有參與模型的建立，測試分數才值得信任。我們看著資料時做的每一個選擇，包括特徵、超參數、閾值，甚至用哪個演算法，都是「模型」的一部分，都必須在不使用評分資料的情況下決定。

訓練擬合參數。驗證選擇超參數、特徵、閾值與校準方式。測試評估被選中的程序。每一個適應性選擇，都會消耗它所檢視資料中的資訊。嚴格來說，交叉驗證估計的是一個**程序**（演算法加上調參規則）在給定大小的資料集上訓練後的表現，而不是你最後要部署的那一個擬合模型的誤差；Bates、Hastie 與 Tibshirani 把這一點說得很精確，並指出天真的交叉驗證信賴區間可能涵蓋率不足 [19]。

保留法、k 折與留一法

- **保留法（hold-out）**。只切一次訓練與測試。簡單又便宜，但資料少時估計的變異數高、依賴切分方式，而且模型使用的訓練資料比手上擁有的少。
- **k 折交叉驗證**。把資料分成 k 折；在 $k-1$ 折上訓練、在剩下一折上評分，輪流讓每一折都被保留一次（圖 4 上）。CV 估計為 $\widehat{\mathrm{Err}}_{\mathrm{CV}} = \frac{1}{n}\sum_{i} L(y_i, \hat{f}^{(-\kappa(i))}(x_i))$，其中 $\kappa(i)$ 是樣本 i 所在的折，$\hat{f}^{(-k)}$ 是不含第 k 折訓練出的模型。$k = 5$ 或 10 是常見的折衷：每個模型看到 80–90% 的資料，悲觀偏差小，成本是 k 次擬合。
- **留一法（leave-one-out, LOO）**。即 $k = n$ 的情形。對大小為 n 的程序幾乎不偏，但需要 n 次擬合（除非有封閉解，例如線性平滑器或 k-NN），而且因為 n 份訓練集幾乎相同，變異數可能很高。
- **分層（stratification）**讓各折的類別比例相近，對稀有類別很重要。它無法解決樣本之間的相依性。

Arlot 與 Celisse 回顧了這些選擇在估計風險與選擇模型兩方面的理論 [18]。

群組與時間

當觀測共享同一位病患、書寫者、相機或錄製時段時，要把**整個群組**放在一起（圖 4 中）。否則同一個人的近似重複影像會落在切分的兩側，分數衡量的是對那個人的記憶，而不是對新的人的辨識。依時間排序的資料需要依時間切分（用過去訓練、用未來測試）。原則是：切分方式要對應部署時的問題。「已知書寫者的新影像」與「新書寫者的影像」是不同問題，應該用不同的切分。

Bootstrap、.632 與.632+

白話版。不用折，改抽許多份 bootstrap 樣本，在每份上訓練，再用該份漏掉的點測試。

精確版。以放回抽樣抽出 B 份大小為 n 的 bootstrap 樣本。對每一份擬合模型，並在沒被抽到的點（約 36.8% 的資料）上評估。其平均即留一 bootstrap 誤差 $\widehat{\mathrm{Err}}^{(1)}$。由於每個 bootstrap 模型只用到約 63.2% 的**相異**點訓練，$\widehat{\mathrm{Err}}^{(1)}$ 偏悲觀；而再代入（訓練）誤差 $\overline{\mathrm{err}}$ 偏樂觀。Efron 的 **.632 估計量**把兩者混合：

$$\widehat{\mathrm{Err}}^{(.632)} = 0.368\,\overline{\mathrm{err}} + 0.632\,\widehat{\mathrm{Err}}^{(1)}.$$

它對 1-最近鄰這類嚴重過擬合、訓練誤差為零的學習器會失效。Efron 與 Tibshirani 的 **.632+ 估計量** [11] 在過擬合越嚴重時，把權重越往 $\widehat{\mathrm{Err}}^{(1)}$ 移動，藉此修正。令 $\hat{\gamma}$ 為**無資訊錯誤率**（no-information error rate，假如輸入與標籤彼此獨立時的錯誤率：$\hat{\gamma} = \sum_{k} \hat{p}_k (1 - \hat{q}_k)$，其中 $\hat{p}_k$ 是類別 k 的觀測比例，$\hat{q}_k$ 是預測為 k 的比例），並定義相對過擬合率 $\hat{R} = (\widehat{\mathrm{Err}}^{(1)} - \overline{\mathrm{err}})/(\hat{\gamma} - \overline{\mathrm{err}})$，截斷到 $[0,1]$。則

$$\hat{w} = \frac{0.632}{1 - 0.368\hat{R}}, \qquad \widehat{\mathrm{Err}}^{(.632+)} = (1-\hat{w})\,\overline{\mathrm{err}} + \hat{w}\,\widehat{\mathrm{Err}}^{(1)}.$$

$\hat{R} = 0$（沒有過擬合）時它化約為.632；$\hat{R} = 1$ 時它變成 $\widehat{\mathrm{Err}}^{(1)}$。

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score, KFold

X, y = make_classification(n_samples=200, n_features=6, n_informative=3,
                           flip_y=0.1, random_state=3)
model = KNeighborsClassifier(n_neighbors=1)          # resubstitution error is 0
rng = np.random.default_rng(3)
n, B = len(y), 200
resub = np.mean(model.fit(X, y).predict(X) != y)
oob_errs = []
for b in range(B):
    idx = rng.integers(0, n, n)                       # draw n with replacement
    out = np.setdiff1d(np.arange(n), idx)             # ~36.8% left out
    model.fit(X[idx], y[idx])
    oob_errs.append(np.mean(model.predict(X[out]) != y[out]))
err_boot1 = np.mean(oob_errs)                          # leave-one-out bootstrap
err_632 = 0.368 * resub + 0.632 * err_boot1
cv10 = 1 - cross_val_score(model, X, y,
                           cv=KFold(10, shuffle=True, random_state=3)).mean()
print(f"resubstitution {resub:.3f}  OOB bootstrap {err_boot1:.3f}  "
      f".632 {err_632:.3f}  10-fold CV {cv10:.3f}")

# .632+ : shrink toward the out-of-bootstrap error when overfitting is large.
pred = model.fit(X, y).predict(X)
gamma = sum(np.mean(y == k) * (1 - np.mean(pred == k)) for k in np.unique(y))
R = np.clip((err_boot1 - resub) / (gamma - resub), 0, 1)
w = 0.632 / (1 - 0.368 * R)
print(f"no-information rate {gamma:.3f}  R {R:.3f}  "
      f".632+ {(1 - w) * resub + w * err_boot1:.3f}")
```

對 1-NN，再代入誤差為 0.000，bootstrap 外誤差 0.332，.632 估計只有 0.210（被沒有意義的零拉低），.632+ 估計為 0.278，10 折 CV 為 0.325。這裡.632 估計量嚴重過度樂觀；.632+ 修補了大部分差距。實務上，10 折 CV（可重複多次）與.632+ 都合理；bootstrap 在樣本極少時以及建立信賴區間時特別有吸引力。

巢狀交叉驗證

白話版。如果你用交叉驗證調超參數，然後回報同一個交叉驗證分數，你回報的是**最幸運**那組設定的分數，會過度樂觀。巢狀 CV 把整個調參過程放進外層迴圈裡。

巢狀交叉驗證有負責選擇的**內層迴圈**，以及評估這個選擇程序的**外層迴圈**（圖 4 下）。在每個外層訓練集內，以內層 CV 選出超參數；選中的設定在整個外層訓練集上重新擬合，再在內層從未見過的外層保留折上評分。Varma 與 Simon 證明，回報內層（調參用）CV 分數可能有相當大的偏差，而巢狀 CV 給出幾乎不偏的估計 [12]。Cawley 與 Talbot 更進一步指出：對模型選擇準則本身過擬合，造成的損失可能與學習演算法之間的差距一樣大，因此低變異數的選擇準則與不偏的準則同樣重要 [13]。前處理（標準化、特徵選擇、PCA）要在每個內層折中重新擬合，通常把它放進 `Pipeline` 即可。回報外層各折之間的變異，但不要把彼此相關的折分數當成獨立測量，去算天真的信賴區間。

估計完程序之後，在開發資料上做選擇，最後在從未動過的測試集上評估一次。反覆查看測試誤差並重新設計模型，會把測試集變成另一個驗證集。

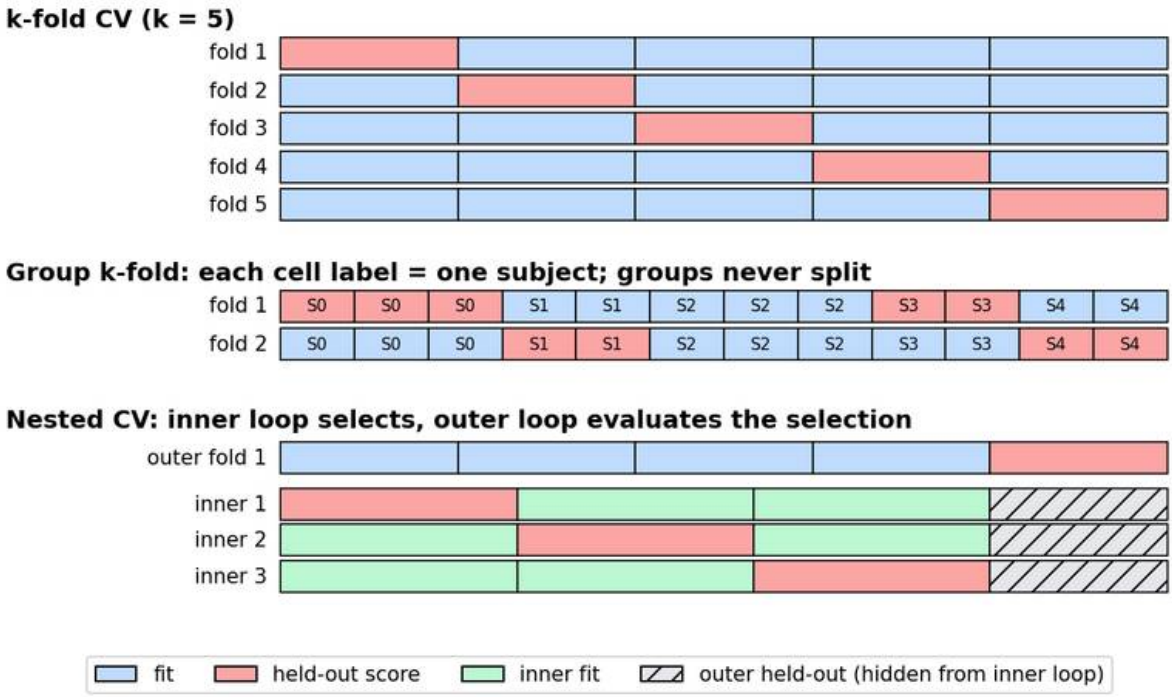


圖 *4* — 三種驗證配置。上：*5* 折 *CV*。中：群組 *k* 折讓每位受試者（*S0–S4*）的所有觀測在每次切分中都只落在同一側。下：巢狀 *CV*；內層迴圈只用外層訓練資料調參，外層保留折（斜線）在調好的模型評分前都保持隱藏。

程式碼：沒有群組洩漏的巢狀選模

這個 CPU 範例為每位合成受試者產生三筆帶雜訊的重複測量。群組標籤描述的是受試者，不是類別。外層迴圈估計所選森林程序的表現；另一份群組不重疊的測試集在最終回報前都不會被動

到。

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, GroupKFold, GridSearchCV
from sklearn.metrics import balanced_accuracy_score, log_loss

rng = np.random.default_rng(9)
subject_x, subject_y = make_classification(
    n_samples=180, n_features=8, n_informative=5, n_redundant=1,
    class_sep=1.0, random_state=9)
groups = np.repeat(np.arange(180), 3)
X = subject_x[groups] + rng.normal(0, 0.03, (len(groups), 8))
y = subject_y[groups]
dev_groups, test_groups = train_test_split(
    np.arange(180), test_size=0.25, stratify=subject_y, random_state=9)
dev = np.flatnonzero(np.isin(groups, dev_groups))
test = np.flatnonzero(np.isin(groups, test_groups))
assert not set(groups[dev]) & set(groups[test])

def search():
    return GridSearchCV(
        RandomForestClassifier(n_estimators=60, random_state=9, n_jobs=1),
        {"max_depth": [3, None], "min_samples_leaf": [1, 3]},
        cv=GroupKFold(3), scoring="balanced_accuracy")

outer_scores = []
for fit, valid in GroupKFold(3).split(X[dev], y[dev], groups[dev]):
    chosen = search().fit(X[dev[fit]], y[dev[fit]], groups=groups[dev[fit]])
    pred = chosen.predict(X[dev[valid]])
    outer_scores.append(balanced_accuracy_score(y[dev[valid]], pred))
final = search().fit(X[dev], y[dev], groups=groups[dev])
p = final.predict_proba(X[test])
pred = final.predict(X[test])
positive = p[:, np.flatnonzero(final.classes_ == 1)[0]]
print("Nested outer balanced accuracy:", np.round(outer_scores, 3))
print("Selected parameters:", final.best_params_)
print("Independent group test:", balanced_accuracy_score(y[test], pred))
print("Test log loss:", log_loss(y[test], p, labels=final.classes_))
print("Test Brier:", np.mean((positive - y[test]) ** 2))
```

我們的執行印出外層平衡準確率 0.869、0.784 與 0.861，選中 `max_depth=None, min_samples_leaf=3`，並在獨立群組測試集上得到平衡準確率 0.853、對數損失 0.378 與 Brier 分數 0.118。外層分數的分散（0.78 到 0.87）提醒我們，單一切分會有多嘈雜。這些結果描述的是合成受試者，不代表在影像上的表現。若把 `GroupKFold` 換成逐列打亂的折，同一受試者的近似複本就會出現在兩側，分數因而虛高（練習 3）。

以統計方法比較分類器

白話版。如果分類器 A 得 97.9%、B 得 96.5%，A 真的比較好，還是只是剛好在這份資料上運氣好？統計檢定問的是：如果兩者其實一樣好，觀察到這樣的差異有多令人意外。麻煩的是，教科書上常見的 t 檢定假設各次測量彼此獨立，而交叉驗證分數並不獨立。

Dietterich 對近似檢定的研究 [14] 是經典的起點；Raschka 的教學文章整理了主要選項並附有程式碼 [20]。

單一測試集：McNemar 檢定

兩個分類器各訓練一次，並在**同一個**測試集上評估。計算 n_{01}：A 對而 B 錯的測試點數；n_{10}：反過來的點數。兩者都對或都錯的點，對「誰比較好」沒有提供資訊。在錯誤率相等的虛無假設下，每一個不一致的點偏向哪一方的機會相等，因此 $n_{01} \sim \text{Binomial}(n_{01}+n_{10}, \tfrac{1}{2})$。連續性修正後的統計量

$$\chi^2 = \frac{(|n_{01} - n_{10}| - 1)^2}{n_{01} + n_{10}}$$

近似服從自由度 1 的 χ^2 分布；計數小時改用精確二項檢定。Dietterich 發現 McNemar 檢定的型一錯誤率可接受，並建議在演算法只能執行一次時使用它 [14]。它沒有納入訓練集選擇所造成的變異。

重複切分：為什麼天真的 t 檢定會失敗

一個誘人的替代做法是重複 J 次隨機訓練／測試切分，記錄誤差差值 d_j，再做成對 t 檢定。Dietterich 證明這種**重抽樣 t 檢定**的型一錯誤率嚴重膨脹 [14]：各訓練集高度重疊，所以 d_j 之間呈正相關，一般的標準誤 $\hat{\sigma}/\sqrt{J}$ 遠遠太小。常見的修正有兩種。

- **5×2cv 成對 t 檢定** [14]。做五次 2 折 CV。在第 i 次中，令 $p_i^{(1)}, p_i^{(2)}$ 為兩折上的誤差差值，$\bar{p}_i$ 為其平均，$s_i^2 = (p_i^{(1)} - \bar{p}_i)^2 + (p_i^{(2)} - \bar{p}_i)^2$。統計量 $t = p_1^{(1)} / \sqrt{\tfrac{1}{5}\sum_{i=1}^{5} s_i^2}$ 與自由度 5 的 t 分布比較。
- **修正重抽樣 t 檢定**（Nadeau 與 Bengio [15]）。保留 J 次隨機切分，每次有 n_{test} 個測試點與 n_{train} 個訓練點，但把變異數放大以反映重疊：

$$t = \frac{\bar{d}}{\sqrt{\left(\dfrac{1}{J} + \dfrac{n_{\text{test}}}{n_{\text{train}}}\right)\hat{\sigma}_d^2}},$$

其中 $\bar{d}$ 與 $\hat{\sigma}_d^2$ 是 d_j 的平均與樣本變異數，並與自由度 $J-1$ 的 t 分布比較。多出來的 $n_{\text{test}}/n_{\text{train}}$ 項不會隨 J 增大而縮小，這正是重點：對同一份資料重抽再多次，也無法創造更多資訊。

```
import numpy as np
from scipy import stats
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.model_selection import train_test_split, ShuffleSplit
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

X, y = load_breast_cancer(return_X_y=True)
A = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))
B = RandomForestClassifier(n_estimators=200, random_state=0)

# 1) McNemar's test on one held-out test set (paired predictions).
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.4, stratify=y,
                                      random_state=0)
ca = A.fit(Xtr, ytr).predict(Xte) == yte
cb = B.fit(Xtr, ytr).predict(Xte) == yte
n01 = int(np.sum(ca & ~cb))      # A right, B wrong
n10 = int(np.sum(~ca & cb))      # A wrong, B right
p_exact = stats.binomtest(n01, n01 + n10, 0.5).pvalue if n01 + n10 else 1.0
chi2 = (abs(n01 - n10) - 1) ** 2 / (n01 + n10) if n01 + n10 else 0.0
print(f"McNemar: n01={n01} n10={n10} chi2_cc={chi2:.3f} "
      f"p_chi2={stats.chi2.sf(chi2, 1):.3f} p_exact={p_exact:.3f}")

# 2) Corrected resampled t-test (Nadeau & Bengio) over J random splits.
J, test_frac = 30, 0.2
d = []
for tr, te in ShuffleSplit(J, test_size=test_frac, random_state=0).split(X):
    ea = np.mean(A.fit(X[tr], y[tr]).predict(X[te]) != y[te])
    eb = np.mean(B.fit(X[tr], y[tr]).predict(X[te]) != y[te])
    d.append(ea - eb)
d = np.array(d)
naive_se = np.sqrt(d.var(ddof=1) / J)
corr_se = np.sqrt(d.var(ddof=1) * (1 / J + test_frac / (1 - test_frac)))
for name, se in [("naive", naive_se), ("corrected", corr_se)]:
    t = d.mean() / se
    print(f"{name:>9} t={t:.3f}  p={2 * stats.t.sf(abs(t), J - 1):.3f}")
```

我們的執行得到 $n_{01} = 9$、$n_{10} = 5$，McNemar $p \approx 0.42$：在一個含 228 筆案例的測試集上，看不出差異的證據。在 30 次隨機 80/20 切分中，邏輯斯迴歸的誤差平均約低 1.8 個百分點。天真的 t 檢定給出 $t \approx -5.4$、$p < 0.001$；修正後的檢定給出 $t \approx -1.85$、$p \approx 0.075$（圖 5）。同一份資料，在 5% 水準下卻得到相反的結論：天真的檢定把 30 次高度重疊的重抽樣誤當成 30 次獨立實驗。

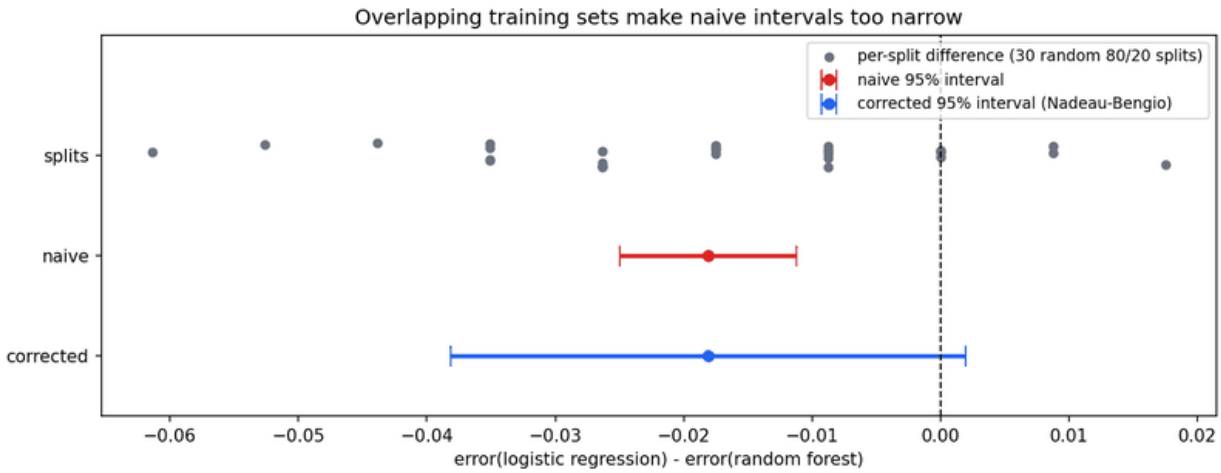


圖 5 — 乳癌資料上 30 次隨機 80/20 切分的誤差差值（邏輯斯迴歸減隨機森林）。天真的成對 t 區間不含零；考慮訓練集重疊的 *Nadeau–Bengio* 修正區間則含零。

多個分類器、多個資料集

在 N 個資料集上比較 k 個演算法時，Demšar 建議使用基於排名的無母數檢定 [16]：兩個分類器用 Wilcoxon 符號等級檢定；更多個則用 **Friedman 檢定**。在每個資料集上為演算法排名（1 為最佳），令 R_j 為演算法 j 的平均排名。在「全部等價」的虛無假設下，

$$\chi_F^2 = \frac{12N}{k(k+1)}\left[\sum_{j=1}^{k} R_j^2 - \frac{k(k+1)^2}{4}\right]$$

近似服從自由度 $k-1$ 的 χ^2 分布（Demšar 也給出一個較不保守、服從 F 分布的變體）。若虛無假設被拒絕，**Nemenyi 事後檢定**在兩個演算法的平均排名差距超過臨界差 $\mathrm{CD} = q_\alpha\sqrt{k(k+1)/(6N)}$ 時判定兩者不同，其中 q_α 來自 Studentized range 統計量除以 $\sqrt{2}$；Demšar 的**臨界差圖**（critical difference diagram）能精簡地呈現結果 [16]。這裡彼此獨立的單位是資料集，而不是折。

Benavoli、Corani、Demšar 與 Zaffalon 主張更進一步，以**貝氏分析**取代虛無假設顯著性檢定，直接回報一個分類器在實務上較好、在實務等價區間（region of practical equivalence）內等價、或較差的機率 [17]。無論用哪種檢定，都要記得 p 值完全不說明差異**有多大**；也要回報效果量與區間。

同時量測決策與機率

準確率衡量答對的比例；**平衡準確率**（balanced accuracy）平均各類別的召回率，因此不會被多數類別主導。**Macro-F1** 在平均各類 F1 時給每個類別相同權重。混淆矩陣顯示發生了哪些錯誤。對稀有的正類，要在實際使用的閾值上檢查精確率與召回率；在閾值尚未決定時，檢查完整的精確率–召回率曲線或 ROC 曲線。

對數損失與 Brier 分數評估機率預測：

$$\text{LogLoss} = -\frac{1}{N}\sum_{n} \log p_n(y_n), \qquad \text{Brier} = \frac{1}{N}\sum_{n} (p_n - y_n)^2,$$

其中 $p_n(y_n)$ 是模型給第 n 筆樣本真實類別的機率；在 Brier 分數（二元情形）中，p_n 是預測為正類的機率，$y_n \in \{0, 1\}$。兩者都是**適當評分規則**（proper scoring rule）：回報真實機率時，其期望值最佳。**可靠度圖**（reliability diagram）在各個區間中比較預測機率與觀察到的頻率。好的排序、好的準確率與好的校準是不同的性質：隨機森林平均投票，傾向把機率推向中間；回合數很多的 boosting 可能把機率推向兩端；模型也可能 ROC AUC 很好，分數卻校準得很差。

擬合校準器時（scikit-learn 提供 sigmoid 與 isotonic 兩種映射 [26]），要使用沒有參與對應基底模型擬合的資料所產生的預測。若觀測有群組，校準用的折也必須遵守群組。校準不會自動修復領域偏移（domain shift）。

把分群有效性當作模型選擇

分群沒有標籤，所以「驗證」的意義不同。Theodoridis 與 Koutroumbas 以整個第 16 章討論分群有效性，因為整齊的分割不一定能服務任務 [1]。完整的指標（silhouette、Davies–Bouldin、Calinski–Harabasz、gap statistic、混合模型的 BIC）在PR 07 中隨演算法一起介紹；這裡把它們放進本章的驗證框架。

內部準則只使用特徵與指派結果。點 i 的 silhouette 比較它到自己群的平均距離 $a(i)$，以及到其他群的最小平均距離 $b(i)$：

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \in [-1, 1].$$

它偏好在所選幾何中分離良好的群。以最大化內部指標來選群數，是一個模型選擇步驟，和調超參數完全一樣；分數高並不代表各群對應到想要的語意類別。

外部準則把指派結果與獨立的參考標籤比較。調整蘭德指數（adjusted Rand index, ARI）對成對一致性做機率校正，而且對標籤排列不變，所以哪一群叫「1」並不重要。如果用標籤來選演算法或群數，那些標籤就不再是未被碰過的外部評估；選擇與評估分離的原則在這裡同樣適用。

穩定性比較重抽樣或擾動後的分割，很像 bootstrap：對多個子樣本分群，並在它們共有的觀測上衡量一致性（例如 ARI），同時處理標籤排列。穩定的群仍可能只反映與任務無關的相機或擷取假影。

現代觀點

集成學習綜述。 Dong、Yu、Cao、Shi 與 Ma 對集成學習做了廣泛的綜述 [21]：他們回顧主流的集成方法、依其特性分類、列出各方法的挑戰與研究方向，並討論集成與深度學習、強化學習的結合。Ganaie、Hu、Malik、Tanveer 與 Suganthan 回顧了**集成深度學習** [22]，把方法整理成以 bagging、boosting、stacking 為基礎的深度集成、負相關學習（negative correlation learning）、顯式與隱式集成（例如共享權重的 snapshot 或 dropout 式集成）、同質與異質成員，以及決策融合策略。在所有這些類別中，實務問題都和前面變異數公式提出的一樣：如何便宜地取得**多樣**的成員。

表格資料上的樹集成。儘管深度網路興起，梯度提升樹（XGBoost [9] 及其以直方圖為基礎的後繼者）與隨機森林 [5] 仍是中等規模表格問題的預設選擇；在辨識中，這類問題包括手工描述子、詮釋資料，以及其他模型的輸出。Grinsztajn、Oyallon 與 Varoquaux 在 45 個資料集上評比樹模型與深度架構，發現在約一萬筆樣本的規模下，樹模型仍是最先進的方法，並把差距歸因於神經網路對無資訊特徵敏感、具有旋轉不變性，以及難以學習不規則函數 [23]。

用集成估計不確定性。 Lakshminarayanan、Pritzel 與 Blundell 證明，**深度集成**（deep ensembles，以不同隨機初始化並搭配適當評分規則訓練的多個網路）在他們的實驗中，給出的預測不確定性與近似貝氏方法相當或更好，而且在分布外輸入上退化得較平緩 [24]。這把 bagging 的直覺（意見分歧代表變異數）連到PR 08 的貝氏模型平均，也是新的不確定性方法都要比較的基準。

重新檢視偏差–變異數。 Belkin、Hsu、Ma 與 Mandal 記錄了**雙重下降** [25]：當容量超過模型能內插訓練資料的那一點後，測試誤差可能再次下降。這並沒有推翻分解，但提醒我們「參數越多、變異數越大」不是定律，而經典的 U 形曲線只描述了容量範圍的一部分。

交叉驗證到底在估計什麼。 Arlot 與 Celisse 的綜述 [18] 仍是 CV 在風險估計與模型選擇方面理論的參考文獻。Bates、Hastie 與 Tibshirani 證明，CV 估計的是在同樣大小的其他訓練集上擬合的模型的平均誤差，而不是手上這個特定模型的誤差，且標準的 CV 信賴區間涵蓋率可能很差；他們提出以巢狀 CV 估計變異數，得到涵蓋率較好的區間 [19]。隨著自動化超參數搜尋在同一組折上比較成千上萬種設定，Cawley 與 Talbot 關於「對選擇準則過擬合」的警告 [13] 變得更加重要。

比較演算法。實務已從天真的重抽樣 t 檢定，轉向單一資料集上的 McNemar 與修正檢定 [14], [15]、跨資料集時以排名為基礎並搭配臨界差圖的 Friedman/Nemenyi 檢定 [16]，以及具實務等價區間的貝氏比較 [17]。Raschka 的教學文章 [20] 是涵蓋以上所有方法的實用指南。

開放問題。在分布偏移（新相機、新場域或新族群）下的誠實評估，因為沒有任何隨機切分能對應部署情境；大型預訓練模型的預訓練資料可能包含測試影像所造成的洩漏；在推論時便宜的集成如何提供可靠的不確定性；以及只有單一大型基準時的統計比較。ML 10 會在深度視覺模型上延續這些主題。

重點整理

- 對平方誤差而言，期望測試誤差 = 雜訊 + 偏差平方 + 變異數；深樹這類有彈性的學習器以偏差換取變異數。
- 平均 M 個成對相關為 ρ 的預測器，剩下的變異數是 $\sigma^2[\rho + (1-\rho)/M]$：bagging 與隨機森林對不穩定的學習器有幫助，而降低成員間相關性（隨機特徵子集）是關鍵槓桿。
- AdaBoost 是對指數損失的貪婪前向逐步最小化；梯度提升把樹擬合到任意可微分損失的負梯度，收縮與提早停止是主要的正則化手段。
- 投票、boosting、貝氏模型平均與 stacking 以不同方式組合模型；stacking 必須用 out-of-fold 預測訓練。
- 驗證估計的是**程序**：每個適應性選擇都屬於迴圈內部，切分必須遵守群組與時間，而要回報調過參的模型表現需要巢狀 CV。
- Bootstrap 的.632+ 估計量修正了.632 對過擬合學習器的過度樂觀；out-of-bag 分數忽略了群組結構。
- 不要對重疊的 CV 折做天真的 t 檢定；改用 McNemar、5×2cv、修正重抽樣 t 檢定，或跨資料

集的 Friedman/Nemenyi，並回報效果量。
- 內部分群指標、外部一致性（ARI）與穩定性回答不同的問題；用它們選 K 就是模型選擇。

練習

1. **相關性下限**。展開總和的變異數，推導平均變異數公式。當 $\rho = 0.8$ 時，需要多少個預測器才能讓變異數落在極限值的 1% 以內？這對 bagging 非常相似的模型有什麼啟示？

   提示

   總和有 M 個變異數項，以及 $M(M-1)$ 個有序共變異數項 $\rho\sigma^2$。極限是 $\rho\sigma^2 = 0.8\sigma^2$；超出的部分是 $(1-\rho)\sigma^2/M = 0.2\sigma^2/M$。要求 $0.2/M \le 0.008$ 得到 $M \ge 25$。超過幾十個高度相關的模型後幾乎沒有收穫；要再降低，唯一的方法是降低 ρ。

2. **手算 AdaBoost**。五個點的權重都是 0.2。第一個樹樁恰好分錯其中一點。計算 ϵ_1、α_1 與新的正規化權重。在新權重下，同一個樹樁的加權誤差是多少？為什麼這不是巧合？

   提示

   $\epsilon_1 = 0.2$，$\alpha_1 = \frac{1}{2}\log 4 = \log 2 \approx 0.693$。未正規化權重：分錯的點為 $0.2 \cdot 2 = 0.4$，其餘為 $0.2/2 = 0.1$；總和 0.8，所以新權重是 0.5 與 0.125（四個）。該樹樁的新加權誤差恰好是 0.5：採用最佳 α 時，AdaBoost 總是重新加權到讓上一個學習器不比亂猜好，迫使下一個學習器去找新的東西。

3. **會洩漏的折**。在巢狀 CV 程式中，把每個 `GroupKFold` 換成 `KFold(3, shuffle=True, random_state=0)`（並移除 `groups` 參數）。比較外層分數與群組不重疊的測試分數，並說明每種切分回答的是哪一個部署問題。

   提示

   使用逐列的折時，同一受試者三筆幾乎相同的測量中，常有兩筆在訓練集、第三筆被拿來驗證，所以森林可以認出受試者而不是類別；內外層分數應該會高於獨立群組測試。逐列的折回答「已知受試者的新測量」；群組折回答「新的受試者」。

4. **穩定學習器的 bootstrap 與 CV**。用 `KNeighborsClassifier(n_neighbors=15)` 重跑 bootstrap 程式。再代入誤差、bootstrap 外誤差、.632、.632+ 與 10 折 CV 現在如何比較？為什麼.632 與.632+ 的差距縮小了？

   提示

   15-NN 分類器的訓練誤差不再為零，所以相對過擬合率 $\hat{R}$ 小得多，$\hat{w}$ 接近 0.632，兩個估計量幾乎一致。所有估計值應該都落在彼此幾個百分點之內。

5. **選擇檢定**。你有 (a) 一個固定的基準切分，以及兩個偵測器的預測；(b) 一個小資料集，以及足以把每個模型重新訓練十次的計算資源；(c) 五個演算法在十二個公開資料集上的評估結果。每種情況適合哪個檢定？各自彼此獨立的單位是什麼？

提示

(a) 對成對不一致使用 McNemar 檢定（或精確二項檢定）；單位是測試影像，訓練集造成的變異沒有被納入。(b) 5×2cv t 檢定或修正重抽樣 t 檢定；單位是重抽樣，並針對重疊做修正。(c) Friedman 檢定加 Nemenyi 事後檢定（或貝氏階層比較）；單位是資料集。

6. **意義錯誤的群**。構造（在紙上或用 `make_blobs`）一個物體影像資料集，它依相機強烈分群，而類別標籤描述的是物體身分。哪些有效性指標會揭露這種不一致？哪些不會？

提示

silhouette 等內部指標會給相機群很高的分數，因為它們分離得很好。對物體標籤的外部 ARI 會很低，而對相機詮釋資料的 ARI 會很高。穩定性也會很高，所以單靠穩定性無法揭露這個分群在語意上是錯的。

參考文獻

1. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2009. 第 4 章（決策樹、分類器組合、boosting）、第 10 章（「Supervised Learning: The Epilogue」）與第 16 章（「Cluster Validity」）。doi
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006. 第 1.3 與 3.2 節（模型選擇、偏差–變異數）及第 14.1–14.5 節（貝氏模型平均、委員會、boosting、樹、條件式混合）。官方書籍與 PDF
3. S. Geman, E. Bienenstock and R. Doursat, “Neural networks and the bias/variance dilemma,” *Neural Computation*, vol. 4, no. 1, pp. 1–58, 1992. doi
4. L. Breiman, “Bagging predictors,” *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996. doi
5. L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. doi
6. Y. Freund and R. E. Schapire, “A decision-theoretic generalization of on-line learning and an application to boosting,” *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997. doi
7. J. Friedman, T. Hastie and R. Tibshirani, “Additive logistic regression: A statistical view of boosting (with discussion),” *The Annals of Statistics*, vol. 28, no. 2, 2000. doi
8. J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *The Annals of Statistics*, vol. 29, no. 5, 2001. doi
9. T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016. doi
10. D. H. Wolpert, “Stacked generalization,” *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992. doi
11. B. Efron and R. Tibshirani, “Improvements on cross-validation: The .632+ bootstrap method,” *Journal of the American Statistical Association*, vol. 92, no. 438, pp. 548–560, 1997. doi
12. S. Varma and R. Simon, “Bias in error estimation when using cross-validation for model selection,” *BMC Bioinformatics*, vol. 7, 91, 2006. doi

13. G. C. Cawley and N. L. C. Talbot, “On over-fitting in model selection and subsequent selection bias in performance evaluation,” *Journal of Machine Learning Research*, vol. 11, pp. 2079–2107, 2010. JMLR
14. T. G. Dietterich, “Approximate statistical tests for comparing supervised classification learning algorithms,” *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998. doi
15. C. Nadeau and Y. Bengio, “Inference for the generalization error,” *Machine Learning*, vol. 52, no. 3, pp. 239–281, 2003. doi
16. J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006. JMLR
17. A. Benavoli, G. Corani, J. Demšar and M. Zaffalon, “Time for a change: A tutorial for comparing multiple classifiers through Bayesian analysis,” *Journal of Machine Learning Research*, vol. 18, no. 77, pp. 1–36, 2017. JMLR
18. S. Arlot and A. Celisse, “A survey of cross-validation procedures for model selection,” *Statistics Surveys*, vol. 4, pp. 40–79, 2010. doi
19. S. Bates, T. Hastie and R. Tibshirani, “Cross-validation: What does it estimate and how well does it do it?,” *Journal of the American Statistical Association*, vol. 119, no. 546, pp. 1434–1445, 2024. doi
20. S. Raschka, “Model evaluation, model selection, and algorithm selection in machine learning,” arXiv:1811.12808, 2018. arXiv
21. X. Dong, Z. Yu, W. Cao, Y. Shi and Q. Ma, “A survey on ensemble learning,” *Frontiers of Computer Science*, vol. 14, no. 2, pp. 241–258, 2020. doi
22. M. A. Ganaie, M. Hu, A. K. Malik, M. Tanveer and P. N. Suganthan, “Ensemble deep learning: A review,” *Engineering Applications of Artificial Intelligence*, vol. 115, 105151, 2022. doi
23. L. Grinsztajn, E. Oyallon and G. Varoquaux, “Why do tree-based models still outperform deep learning on typical tabular data?,” in *Advances in Neural Information Processing Systems* (*NeurIPS*) *Datasets and Benchmarks Track*, 2022 (arXiv:2207.08815). arXiv
24. B. Lakshminarayanan, A. Pritzel and C. Blundell, “Simple and scalable predictive uncertainty estimation using deep ensembles,” in *Advances in Neural Information Processing Systems* (*NIPS*), 2017 (arXiv:1612.01474). arXiv
25. M. Belkin, D. Hsu, S. Ma and S. Mandal, “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *Proceedings of the National Academy of Sciences*, vol. 116, no. 32, pp. 15849–15854, 2019. doi
26. scikit-learn developers, *scikit-learn User Guide*：集成方法、交叉驗證、機率校準與分群效能評估。

PR 10・辨識專題：拒絕、漂移與學習式特徵

以 scikit-learn 手寫數字資料建立端到端、可重現的辨識專題：定義任務、鎖住測試集、以巢狀驗證比較基準與分類器、做錯誤分析，再檢驗拒絕、未知類別與分布漂移。

網站版本: https://blog.shyandram.dev/zh-tw/learn/pattern-recognition/pr-10-recognition-capstone/

你將學到

- 如何把模糊的目標（「辨識手寫數字」）寫成一份實驗協定：觀測單位、抽樣單位、損失、拒絕選項，以及每個資料切分各自能用來做什麼。
- 如何用同一條管線（pipeline）與巢狀交叉驗證，公平比較本系列介紹過的分類器（樣板、單純貝氏、k-NN、LDA、邏輯斯迴歸、核 SVM、隨機森林）。
- 如何用校正後的重抽樣 t 檢定（corrected resampled t-test）判斷兩個模型的差距是否真實，而不是只看各折平均。
- 如何在開發資料上做錯誤分析、解讀學習曲線，並且只打開鎖住的測試集一次。
- 如何區分封閉集合準確率、選擇性分類、未知類別拒絕與分布漂移下的穩健性，以及這些原則如何延伸到預訓練神經網路特徵。

先看全貌

前九章各自研究辨識器的一個部分：決策與風險（PR 01）、密度估計（PR 02）、線性與核分類器（PR 03、PR 04）、特徵（PR 05）、樣板（PR 06）、分群（PR 07）、近似推論（PR 08），以及集成與驗證（PR 09）。真實的專題需要同時用上它們，順序要對，而且每個決策能看到哪些資料都必須有紀律。

> **用最簡單的話說**
>
> 想像你在教朋友把一大罐鈕扣分到十個盒子裡。你們先講好規則：什麼算鈕扣、有哪些盒子、遇到奇怪的東西（例如一枚硬幣）該怎麼辦。接著你偷偷把一小把鈕扣放進口袋，練習時完全不拿出來。朋友用其餘的鈕扣練習、嘗試各種分類技巧，並仔細檢查每一次失誤。直到最後，你才把口袋裡的鈕扣拿出來，數一數有幾顆被放進正確的盒子。如果練習時偷看了口袋裡的鈕扣，最後的分數就會讓朋友看起來比實際更厲害。

為什麼重要：多數令人失望的辨識系統，並不是因為選錯了分類器，而是因為評估回答的問題和部署時的問題不同：測試集洩漏進模型選擇、切分時把同一個人的樣本分到兩邊、系統從沒被問過遇到沒見過的東西該怎麼辦，或者沒有人檢查影像平移一個像素時會發生什麼事。Theodoridis 與 Koutroumbas 把系統設計與評估視為圖形識別本身的一部分 [1]，Bishop 則把最後一步表述為在損失函數下、帶有明確拒絕選項的決策 [2]。本章圍繞這些想法，完成一個完整的專題。

專題總覽

白話版。我們依序走九個步驟。前七步只使用開發資料；第 8 步打開鎖住的測試集一次；第 9 步提出比「準確率多少」更難的問題。

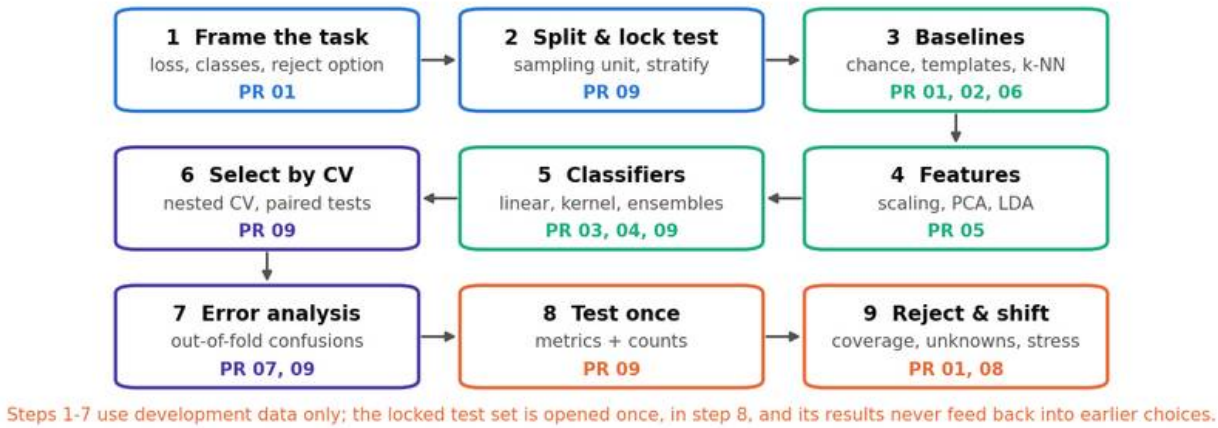


圖 *1* — 專題流程與各步驟所依據的章節。第 *1–7* 步只看開發資料；測試集只在第 *8* 步打開一次。

步驟	回答的問題	主要工具	對應章節
1 定義	什麼是一筆觀測、一個類別、一次錯誤？	損失矩陣、拒絕選項	PR 01
2 切分	哪些資料可以影響哪些決策？	分層、分組切分	PR 09
3 基準	在這裡「好」是什麼意思？	機率水準、類別平均、k-NN、單純貝氏	PR 01、02、06
4 特徵	分類器看到什麼表徵？	標準化、PCA、LDA	PR 05
5 分類器	哪些模型家族值得一試？	線性、核方法、集成	PR 03、04、09
6 選模	選哪個設定？差距是真的嗎？	巢狀交叉驗證、校正 t 檢定	PR 09
7 分析	為何失敗？更多資料有用嗎？	折外混淆、學習曲線	PR 07、09
8 測試	凍結後的流程表現如何？	含區間的準確率、各類別報告	PR 09
9 壓力測試	不確定、新類別與漂移怎麼辦？	涵蓋率–風險、未知類別、擾動	PR 01、08

以下所有程式都能在筆電 CPU 上於一分鐘內跑完，只使用 scikit-learn 內建資料 [4]。第 2 到第 8 步的程式片段構成同一支腳本：請在同一個 Python 工作階段中依序執行。

步驟 1 — 定義任務

白話版。動到模型之前，先寫下一筆樣本是什麼、可能的答案有哪些、犯錯的代價是多少，以及系統何時可以說「我不知道」。

一份書面協定

協定是在實驗之前寫下的簡短文件，能防止你在看到答案後才改問題。任何辨識專題至少要回答：

1. **觀測**。一筆輸入是什麼？一張影像、一個裁切區塊、一段影片，還是一整份文件？
2. **抽樣單位**。哪些觀測來自同一個來源（同一個人、同一台相機、同一位病人、同一天）？同源觀測彼此不獨立，在每一次切分中都必須留在同一側。
3. **標籤空間**。訓練時已知哪些類別？其他類別的輸入應如何處理？
4. **損失**。所有錯誤都一樣嚴重嗎？可以棄權嗎？棄權的代價是多少？
5. **資料角色**。訓練、驗證、測試資料各自可以拿來做什麼？
6. **部署條件**。評估必須涵蓋哪些裝置、書寫者、光照或時間區段？

我們想逼近的決策規則

PR 01 說明在 0–1 損失下，貝氏分類器會選擇後驗機率最大的類別。若加入代價為 λ_r 的拒絕選項（正確決策代價為 0，錯誤代價為 1），就得到 Bishop 在決策理論一節所介紹的拒絕規則 [2]：

$$\hat{y}(x) = \begin{cases} \arg\max_k P(\omega_k \mid x) & \text{if } \max_k P(\omega_k \mid x) \geq 1 - \lambda_r, \\ \text{reject} & \text{otherwise.} \end{cases}$$

其中 x 是特徵向量，ω_k 是第 k 類，$P(\omega_k \mid x)$ 是它的後驗機率，$0 \leq \lambda_r \leq 1$ 是拒絕相對於錯誤的代價。若 $\lambda_r = 0$，拒絕不必付出代價，只要不是百分之百確定就拒絕；若 K 個類別時 $\lambda_r \geq 1 - 1/K$，則永遠不拒絕。之後的一切（選分類器、驗證、設定門檻）都是在用有限資料逼近這條規則。請記得它的限制：這條規則只處理已知類別之間的不確定。一個全新的類別仍可能得到很高的後驗機率，所以第 9 步會另外處理未知類別。

本專題與它的資料卡

我們使用 `sklearn.datasets.load_digits` 辨識手寫數字 [4]。為資料寫一張簡短的「資料卡」也是定義任務的一部分（這正是資料集資料表 datasheets for datasets 的精神 [24]）：

- **來源**。這份資料是 Alpaydin 與 Kaynak 在 UCI 發布的「Optical Recognition of Handwritten Digits」資料集中測試部分的複本 [3]。
- **觀測**。共 1,797 張影像。每張原本是 32×32 的二值點陣圖，切成不重疊的 4×4 區塊；得到的 8×8 每一格計算區塊內「亮」的像素數，所以數值是 0 到 16 的整數 [3]。
- **類別**。十個數字，大致平衡（每類 174 到 183 張）。
- **來源者**。依資料集說明，這部分由 13 個人書寫，與書寫 UCI 訓練部分的 30 人不同 [3]。**資料沒有提供書寫者編號**。

最後一點限制了我們能做的所有宣稱。因為無法依書寫者分組，隨機切分會讓同樣這 13 個人的字同時出現在兩側。因此我們報告的測試準確率，估計的是這些書寫者的新影像上的表現，而不是新書寫者上的表現。UCI 的訓練部分由不同的人書寫，可以做到書寫者不重疊的測試，但它沒有內建在 scikit-learn 中，本章也不使用。坦白寫出這個限制，是誠實結果的一部分。

步驟 2 — 切分資料並鎖住測試集

白話版。一開始就把四分之一的資料放進「上鎖的抽屜」。在第 8 步之前，所有決策都不能用到它。

```
import numpy as np
import warnings
warnings.filterwarnings("ignore", message=".*within_class_std_dev_.*")
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

digits = load_digits()
X, y = digits.data, digits.target          # 1797 x 64, integer intensities 0..16
print("shape:", X.shape, "range:", X.min(), X.max())
print("class counts:", np.bincount(y))

idx = np.arange(len(y))
dev, test = train_test_split(idx, test_size=0.25, stratify=y, random_state=0)
assert not set(dev) & set(test)
print("development:", len(dev), "test (locked):", len(test))
```

輸出：`shape: (1797, 64) range: 0.0 16.0`，各類別數量介於 174 到 183，`development: 1347 test (locked): 450`。（warnings 過濾器只是關掉 `NearestCentroid` 的一則無害提示：某些邊緣像素在同一類別內恆為常數。）

為什麼要分層？測試集只有 450 張，隨機切分很容易讓某個數字偏少。分層（stratification）讓兩邊的類別比例一致。它不能解決觀測之間的相依性；只有依抽樣單位分組才能做到（PR 09）。

資料洩漏：最常見的自欺方式

Kapoor 與 Narayanan 整理了以機器學習為基礎的科學研究中的再現性失敗，發現資料洩漏（data leakage）遍布許多領域；他們提出的分類包括：訓練與測試沒有乾淨分離、在全部資料上擬合前處理、跨切分的重複樣本、不正當的特徵，以及測試集與目標母體不符 [5]。在辨識專題中，最常見的形式是：

- 在切分前，就用全部資料擬合標準化、PCA 或特徵選擇；
- 看著測試分數選擇超參數、特徵或門檻；
- 近乎重複的影像（連續影格、擴增後的副本、同一個人）落在切分兩側；
- 測試集來自與部署不同的母體（例如部署時面對新書寫者，測試卻只有同一群書寫者）。

解法是從結構上著手：把每一個需要擬合的轉換都放進 scikit-learn 的 `Pipeline`，讓交叉驗證在每一折的訓練部分重新擬合；並且在第 8 步之前，任何函式呼叫都不碰測試索引。

步驟 3 — 觀察資料並建立基準

白話版。訓練任何巧妙的模型之前，先看看樣本，並計算簡單方法能做到多好。一個花俏模型若只比極簡模型好一點點，就不算成功。

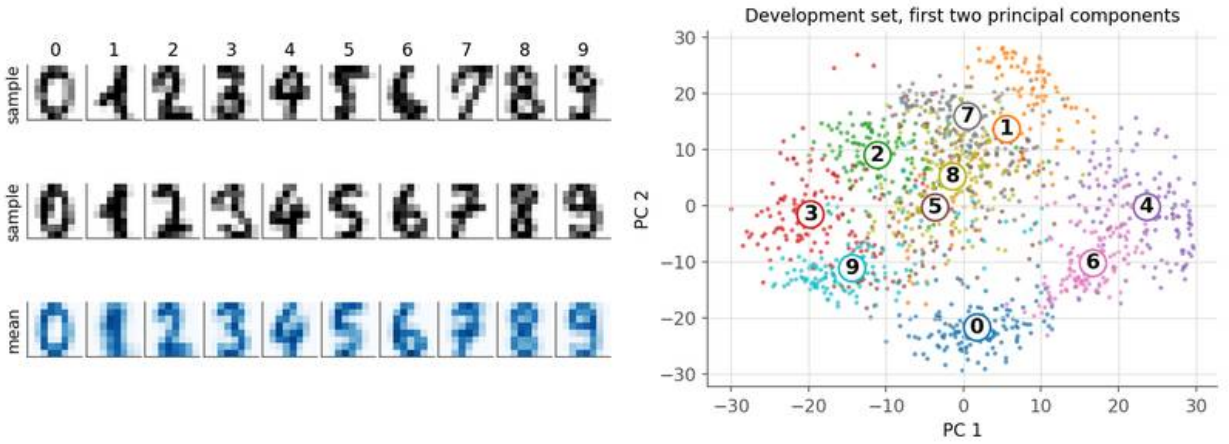


圖 2 — 左：每個類別兩張開發集影像與該類別的平均影像（也就是最近類別平均分類器的樣板）。右：開發集投影到前兩個主成分。數字 *0*、*4*、*6* 已大致分開；*1*、*7*、*8*、*5*、*9* 在這個二維視圖中嚴重重疊。

圖 2 已透露出難度的結構。類別平均看起來像乾淨的數字，所以樣板比對（PR 06）應該有不錯的效果。在兩個主成分中，有些類別分得很開，有些重疊；PR 05 解釋過，變異大的方向不一定最具鑑別力，因此兩個成分中的重疊，並不代表 64 維中也重疊。

我們使用四個基準，各自對應一章：

- **多數類別**。永遠預測最常見的數字。十個平衡類別下，準確率約 10%。這是下限。
- **最近類別平均**。把影像和每個類別平均比較，選最接近的：最簡單的樣板比對器。
- **高斯單純貝氏**。一種貝氏分類器（PR 01），類別條件密度為對角共變異數的高斯分布。像素之間高度相關，獨立假設在這裡嚴重不成立，我們預期它表現較差。
- **k 最近鄰**。一種以密度為基礎的無母數規則（PR 02）：由最接近的 k 張訓練影像投票。

步驟 4 — 決定特徵

白話版。分類器只看得到數字。要決定用哪些數字、如何縮放、要不要壓縮，而且這些轉換只能在訓練折上擬合。

我們的原始特徵是 64 個區塊計數。有三種常見轉換值得比較：

標準化。減去每個特徵在訓練資料上的平均，再除以標準差：

$$z_j = \frac{x_j - \bar{x}_j}{s_j},$$

其中 x_j 是第 j 個像素，$\bar{x}_j$ 與 s_j 是它在訓練折上的平均與標準差，z_j 是標準化後的值。當特徵單位差異很大時，這對以距離或梯度為基礎的方法很重要。但這裡所有特徵本來就是同一個單位（0 到 16 的計數）。更糟的是，邊緣像素幾乎總是 0，s_j 很小：標準化會把偶爾出現的雜散筆畫放大成很大的值。我們會看到這個選擇明顯拖累核 SVM。

主成分分析。 PCA 把資料投影到變異最大的方向（PR 05）。保留 d' 個成分時，表徵為 $u = W^\top(x - \bar{x})$，其中 W 存放訓練共變異數前 d' 個特徵向量。它能移除雜訊方向並加速核方法，但變異不等於類別資訊。

線性鑑別分析。 LDA（PR 03、PR 05）找出最多 $K - 1 = 9$ 個方向，使類間散布相對於類內散布最大。我們把它當作帶收縮（shrinkage）的分類器使用，收縮可正則化類內共變異數的估計。

> **特徵也是超參數**
>
> 要不要標準化、PCA 保留幾個成分、要加入哪些手工特徵，都是用資料做出的選擇。請像對待正則化常數一樣對待它們：放進管線，讓驗證來決定。在交叉驗證之前就用全部資料擬合一次，就是洩漏。

對真實影像而言，這一步也是加入工程特徵（HOG、LBP、矩等，見特徵擷取一章）或預訓練嵌入的地方。在 8×8 的解析度下，原始計數是合理的起點。

步驟 5 — 比較候選分類器

白話版。讓每個候選模型都跑過開發集上同一組交叉驗證切分，比較它們的分數分布，而不是只看最好的那一折。

```
from sklearn.model_selection import RepeatedStratifiedKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.dummy import DummyClassifier
from sklearn.neighbors import NearestCentroid, KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier

candidates = {
    "majority class (PR 01)": DummyClassifier(strategy="most_frequent"),
    "nearest class mean (PR 06)": NearestCentroid(),
    "Gaussian naive Bayes (PR 01)": GaussianNB(),
    "k-NN, k=3 (PR 02)": KNeighborsClassifier(n_neighbors=3),
    "LDA, shrinkage (PR 03/05)": LinearDiscriminantAnalysis(
        solver="lsqr", shrinkage="auto"),
    "logistic regression (PR 03)": make_pipeline(
        StandardScaler(), LogisticRegression(C=1.0, max_iter=3000)),
    "PCA-30 + RBF SVM (PR 04/05)": make_pipeline(
        StandardScaler(), PCA(n_components=30, random_state=0),
        SVC(C=10, gamma="scale")),
```

```
    "random forest (PR 09)": RandomForestClassifier(
        n_estimators=300, random_state=0, n_jobs=1),
}
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=2, random_state=0)
cv_scores = {}
for name, model in candidates.items():
    s = cross_val_score(model, X[dev], y[dev], cv=cv, scoring="accuracy")
    cv_scores[name] = s
    print(f"{name:30s} {s.mean():.3f} +/- {s.std():.3f}")
```

輸出（10 折的平均 ± 標準差）：

候選模型	交叉驗證平均準確率	標準差
多數類別	0.100	0.000
高斯單純貝氏	0.827	0.017
最近類別平均	0.897	0.011
帶收縮的 LDA	0.951	0.007
邏輯斯迴歸（標準化）	0.966	0.012
隨機森林，300 棵樹	0.975	0.010
標準化 + PCA-30 + RBF SVM	0.976	0.010
3 最近鄰	0.985	0.007

已經可以看出幾個教訓：

- **簡單模型的排名反映了它們的假設**。單純貝氏的獨立假設被相鄰像素間的相關性打破，是最弱的真正模型。每類只有一個樣板（最近平均）無法涵蓋同一數字的不同寫法。LDA 建模共享的完整共變異數，表現好得多。
- **未調參模型中最好的，是最簡單的無母數模型**。 3-NN 勝過所有未調參的有母數模型。1,347 張小影像、只有 13 位書寫者，許多影像在訓練折中都有近乎雙胞胎的樣本，這正是最近鄰最擅長的情況。這也提醒我們，這種切分衡量的是「已知書寫者的新影像」。
- **預設值不是公平的比較**。 SVM 與邏輯斯迴歸帶著標準化步驟與接近預設的設定。下結論前，強的候選模型值得調參，而且要在不會讓比較產生偏誤的協定下進行。

步驟 6 — 調參與選模，不自欺

白話版。調參是方法的一部分。要估計「含調參的 SVM」有多好，必須在每一個外層折內重做整個調參。然後再問：與次佳模型的剩餘差距，是否大於雜訊？

巢狀交叉驗證

內層迴圈選超參數，外層迴圈評估整個流程。如果直接回報內層最佳分數，等於回報許多雜訊數字中的最大值，必然偏樂觀。Varma 與 Simon 以實驗展示了交叉驗證選模的這種偏誤 [6]；Cawley 與 Talbot 則指出，過度擬合選模準則所造成的效能差異，可能與不同學習演算法之間的差異一樣大 [7]。Raschka 的教學式回顧整理了實務做法 [9]。

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.pipeline import Pipeline

svm = Pipeline([("scale", StandardScaler()), ("pca", PCA(random_state=0)),
                ("svc", SVC())])
grid = {"scale": [StandardScaler(), "passthrough"],
        "pca": [PCA(30, random_state=0), "passthrough"],
        "svc__C": [1, 10],
        "svc__gamma": ["scale", 0.001]}
inner = StratifiedKFold(5, shuffle=True, random_state=1)
outer = StratifiedKFold(5, shuffle=True, random_state=2)
search = GridSearchCV(svm, grid, cv=inner, scoring="accuracy")
nested = cross_val_score(search, X[dev], y[dev], cv=outer)
print("nested-CV accuracy of the tuning procedure:",
      nested.round(3), "mean", nested.mean().round(3))
search.fit(X[dev], y[dev])
print("chosen on all development data:", search.best_params_)
```

輸出：外層各折準確率 `[0.993 0.993 0.985 0.985 0.993]`，平均 `0.99`。在全部開發資料上選出的設定是**不標準化、PCA 保留 30 個成分**、$C = 10$、$\gamma = 0.001$。

兩點觀察。第一，網格本身包含了前處理的決定（`"passthrough"` 表示「略過這一步」），而驗證結果拒絕了標準化，印證了步驟 4 的論點。第二，巢狀估計值（0.990）描述的是「執行這個網格搜尋，再重新擬合」這個流程，也就是我們要部署的東西；它不是某一組超參數的性質。

贏家真的比較好嗎？

調參後的 SVM（巢狀 0.990）與 3-NN（0.985）相差半個百分點。這是真的嗎？對各折分數做一般的成對 t 檢定，會把各折當成獨立樣本；但不同折的訓練集高度重疊，各折分數正相關，一般檢定因此過度自信。Nadeau 與 Bengio 為重抽樣設計提出了校正後的變異數 [8]：

$$t = \frac{\bar{d}}{\sqrt{\left(\frac{1}{k} + \frac{n_{\text{te}}}{n_{\text{tr}}}\right)\hat{\sigma}_d^2}},$$

其中 d_i 是兩個模型在第 i 次重抽樣上的分數差，$\bar{d}$ 與 $\hat{\sigma}_d^2$ 是這 k 個差值的平均與樣本變異數，n_{tr} 與 n_{te} 是每次重抽樣的訓練與測試筆數。額外的 $n_{\text{te}}/n_{\text{tr}}$ 項放大變異數，以反映重疊。統計量與自由度 $k-1$ 的 t 分布比較。PR 09 推導了這個檢定，並討論 McNemar 檢定等替代方案。

```
from scipy import stats

def corrected_t(a, b, n_train, n_test):
    """Nadeau-Bengio corrected resampled t-test on paired fold scores."""
    d = np.asarray(a) - np.asarray(b)
    k = len(d)
    var = d.var(ddof=1) * (1 / k + n_test / n_train)
    t = d.mean() / np.sqrt(var)
```

```
    return t, 2 * stats.t.sf(abs(t), df=k - 1)

n_test_fold = len(dev) // 5
tuned = cross_val_score(search.best_estimator_, X[dev], y[dev], cv=cv)
t, p = corrected_t(tuned,
                   cv_scores["k-NN, k=3 (PR 02)"],
                   n_train=len(dev) - n_test_fold, n_test=n_test_fold)
print(f"tuned SVM vs k-NN: corrected t = {t:.2f}, p = {p:.3f}")
```

輸出：`tuned SVM vs k-NN: corrected t = 1.03, p = 0.331`。

在這份資料上，兩者的差異無法在統計上區分。（嚴格來說，調參後的設定本身就是在這些開發資料上選出的，比較對 SVM 略為有利；即便如此仍不顯著。）在真實專題中，這是一個決策點，而不是失敗。兩者都合理，這時就由其他準則決定：SVM 只儲存支撐向量，每個支撐向量做一次核函數計算；k-NN 要儲存並搜尋所有訓練影像；SVM 的決策值可以校準成機率；k-NN 只有一個容易解釋的超參數。我們選擇調參後的 SVM 管線，因為它的巢狀估計較高、模型較精簡，並記錄 3-NN 是統計上平手的替代方案。

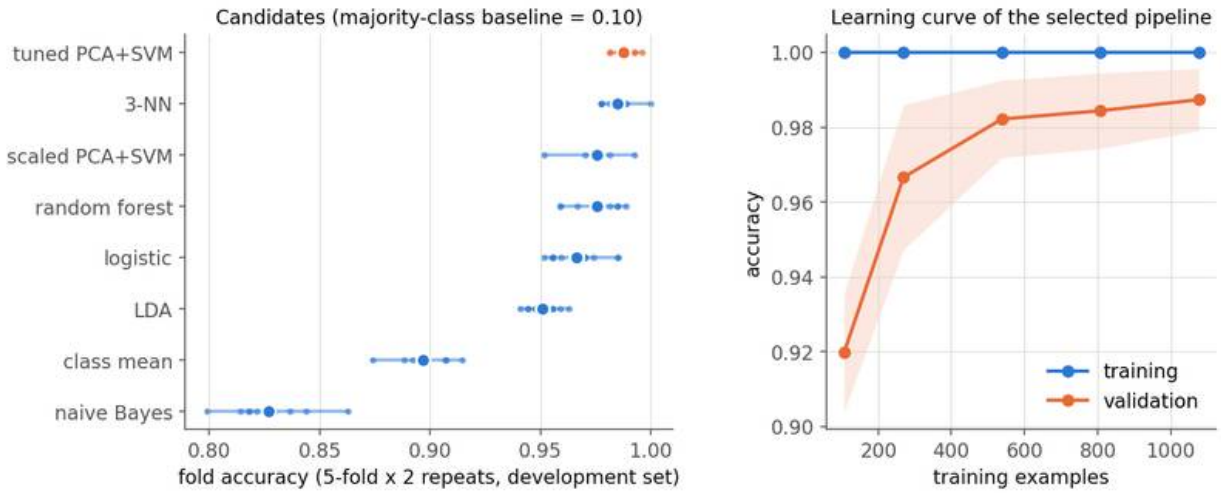


圖 *3* — 左：*10* 個開發折上的準確率（小點）、平均（大點）與範圍（橫條）。調參後的管線（橘色）使用搜尋選出的設定。右：所選管線的學習曲線；陰影為各折間 ± 1 個標準差。

步驟 7 — 在開發資料上做錯誤分析

白話版。去看錯誤，但要看開發資料上的錯誤。一旦因為測試錯誤而重新設計系統，測試集就不再是測試集了。

折外預測

`cross_val_predict` 會為每張開發影像回傳一個「沒有用它訓練過」的模型所做的預測。這些折外（out-of-fold）預測是研究錯誤的正當場所。

```
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import confusion_matrix

chosen = search.best_estimator_
oof = cross_val_predict(chosen, X[dev], y[dev],
                        cv=StratifiedKFold(5, shuffle=True, random_state=3))
cm_dev = confusion_matrix(y[dev], oof)
off = cm_dev.copy()
np.fill_diagonal(off, 0)
order = np.argsort(off.ravel())[::-1][:5]
print("out-of-fold errors:", int(off.sum()), "of", len(dev))
for t_, p_ in zip(*np.unravel_index(order, off.shape)):
    print(f"  true {t_} -> predicted {p_}: {off[t_, p_]}")
dev_errors = dev[oof != y[dev]]
print("first misclassified dataset indices:", dev_errors[:8])
```

輸出：1,347 張中有 17 個錯誤。最常見的混淆是 9 → 8（3 次）、5 → 9（3 次）、8 → 1（2 次），其次是 9 → 5、6 → 8 等單次錯誤。

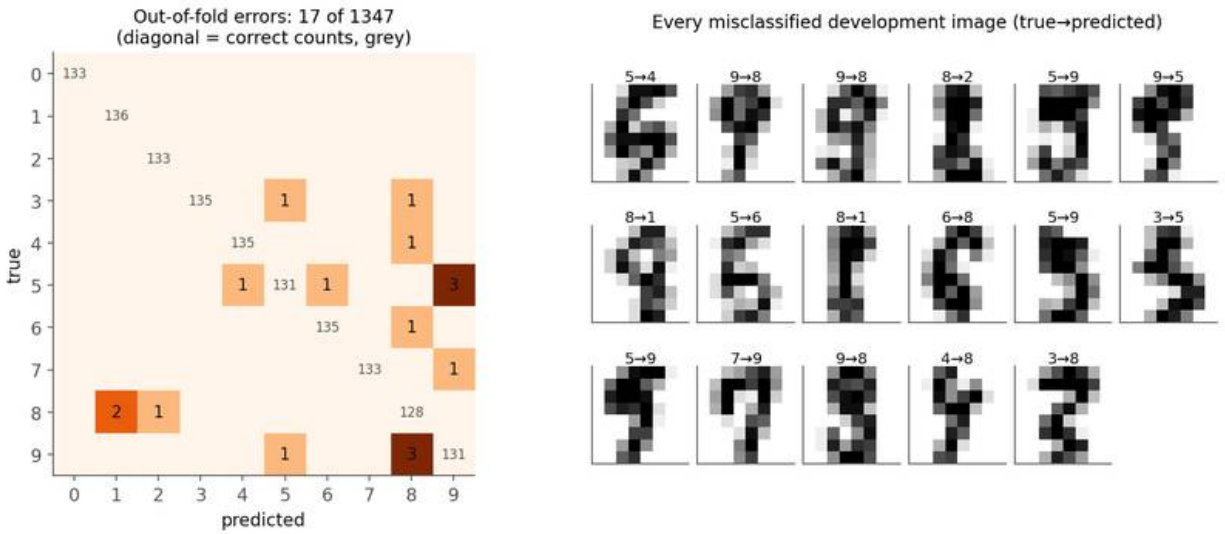


圖 4 — 所選管線的折外錯誤分析。左：非對角線計數（對角線以小字顯示正確數量）。右：每一張分錯的開發集影像，標示真實 → 預測。

解讀圖 4 時，就需要領域知識：

- **系統性混淆**。錯誤集中在上方有封閉圈的數字（5、8、9）與細長直筆畫的數字（1、8）。在 8×8 解析度下，區分 9 與 8 的缺口可能只有一格。
- **標籤或品質問題**。有些分錯的影像連人都難以判斷。在較大的專題中，你會逐一檢視，判斷它們是標註錯誤（要修正）、本質上模稜兩可的輸入（第 9 步拒絕的候選），還是一般失誤（以特徵或資料改善）。
- **把錯誤分群**。錯誤有上百個時，無法逐一檢查。常見技巧是在特徵空間中對分錯的樣本分群（PR 07），再檢查每群的幾個成員；一個群集通常對應一種失敗模式，例如某種書寫風格或擷取

問題。

好的錯誤分析最後會得到假設，每個假設都搭配一個能在開發資料上執行的實驗，例如：「讓模型對一格平移不敏感，可能改善 5/9 混淆；試著在訓練折中加入平移後的副本，再重跑巢狀比較。」

更多資料有用嗎？學習曲線

```
from sklearn.model_selection import learning_curve

sizes, tr, va = learning_curve(
    chosen, X[dev], y[dev], cv=StratifiedKFold(5, shuffle=True, random_state=3),
    train_sizes=[0.1, 0.25, 0.5, 0.75, 1.0])
for n, a, b in zip(sizes, tr.mean(1), va.mean(1)):
    print(f"n={n:4d}  train={a:.3f}  validation={b:.3f}")
```

訓練影像數	訓練準確率	驗證準確率
107	1.000	0.920
269	1.000	0.967
538	1.000	0.982
807	1.000	0.984
1,077	1.000	0.987

各種訓練量下訓練準確率都是滿分，驗證準確率在右端仍緩慢上升（圖 3 右）。以偏差–變異的語言（PR 09）來說，剩下的誤差主要是變異：模型已經夠有彈性，更多（或更多樣）的訓練資料可能還有一點幫助。若兩條曲線一起停在低處，建議就相反：需要表達力更強的模型或更好的特徵。

步驟 8 — 打開測試集，只打開一次

白話版。凍結所有選擇，把最終管線用在鎖住的測試影像上，並連同不確定性與適用範圍一起報告結果。

```
from sklearn.metrics import (accuracy_score, balanced_accuracy_score,
                             classification_report)

final = search.best_estimator_            # already refit on all of dev
pred = final.predict(X[test])
print("TEST accuracy:", round(accuracy_score(y[test], pred), 4),
      "balanced:", round(balanced_accuracy_score(y[test], pred), 4),
      "errors:", int((pred != y[test]).sum()), "of", len(test))
cm = confusion_matrix(y[test], pred)
print(cm)
print(classification_report(y[test], pred, digits=3))

k, n, z = int((pred == y[test]).sum()), len(test), 1.96   # Wilson interval
```

```
phat = k / n
centre = (phat + z**2 / (2 * n)) / (1 + z**2 / n)
half = z * np.sqrt(phat * (1 - phat) / n + z**2 / (4 * n**2)) / (1 + z**2 / n)
print(f"95% Wilson interval for accuracy: [{centre - half:.3f}, {centre + half:.3f}]")
```

輸出：測試準確率 **0.9933**（450 張中 3 個錯誤：3 → 7、8 → 1、9 → 5 各一次），平衡準確率 0.9933，各類別召回率介於 0.977 到 1.000，95% Wilson 區間為 **[0.981, 0.998]**。

如何解讀與報告：

- **報告數量，而不只是百分比**。「450 張中錯 3 張」能讓讀者立刻知道小數點第三位有多可信。
- **報告區間**。準確率接近 1 時，常用的常態近似 $\hat{p} \pm 1.96\sqrt{\hat{p}(1-\hat{p})/n}$ 可能超過 1，並不可靠；上面使用的 Wilson 分數區間會留在 $[0, 1]$ 之內，在邊界附近表現較好。若有 n 筆測試樣本、k 筆正確、$\hat{p} = k/n$，其中心為 $(\hat{p} + z^2/2n)/(1 + z^2/n)$，95% 涵蓋時 $z = 1.96$。
- **與開發估計比較**。測試結果（0.993）與巢狀估計（0.990）一致。若低很多，可能原因是開發過程有洩漏，或測試母體與開發不同，兩者都值得追查；但若因此修改系統，就需要一份新的未碰過的測試集。
- **說明範圍**。這是同樣 13 位書寫者的新影像上的準確率（步驟 1），不能證明對新書寫者、新掃描器或新前處理的表現。

程式中的協定註解很重要：`search.best_estimator_` 在測試前已在全部開發資料上重新擬合一次。若再用開發加測試資料重新擬合，然後報告測試分數，就毫無意義。部署時可以用全部有標籤的資料重新擬合凍結的流程，但報告的數字必須來自上面這次執行。

步驟 9 — 超越封閉集合準確率

一個在已知數字的平衡測試上達到 99% 的辨識器，若會自信地把一個英文字母標成數字，或在掃描器把影像平移時崩潰，依然可能毫無用處。這一步檢驗三個封閉集合數字無法回答的問題。

四種容易混淆的任務

任務	系統必須做什麼	評估必須包含什麼
封閉集合分類	在訓練過的類別中選一個	已知類別的獨立樣本
選擇性分類	對困難的已知樣本棄權	已知樣本的涵蓋率與被接受樣本的錯誤率
開放集合辨識	分類已知類別，並拒絕未知類別	保留的未知類別，以及已知樣本
異常偵測	標出偏離參考母體的樣本	定義清楚的正常母體與相關的偏離

信心低的已知樣本不一定是未知類別；不熟悉的類別可能得到很高的信心；一個有效的已知物件若拍得很差，也可能被拒絕。這些任務需要不同的資料與指標。

選擇性分類：涵蓋率與風險

白話版。讓分類器略過最沒把握的輸入，並量測兩件事：它還回答了多少輸入，以及這些回答有多常出錯。

設信心分數為 $q(x)$、接受門檻為 τ，在 N 筆已知類別測試樣本上，預測為 $\hat{y}_n$、標籤為 y_n：

$$\text{coverage}(\tau) = \frac{1}{N}\sum_{n=1}^{N} \mathbf{1}[q(x_n) \geq \tau],$$

$$\text{selective risk}(\tau) = \frac{\sum_n \mathbf{1}[q(x_n) \geq \tau]\,\mathbf{1}[\hat{y}_n \neq y_n]}{\sum_n \mathbf{1}[q(x_n) \geq \tau]}.$$

這裡 $\mathbf{1}[\cdot]$ 在條件成立時為 1，否則為 0。沒有任何樣本被接受時，選擇性風險沒有定義。掃描 τ、畫出風險對涵蓋率的關係，就得到風險–涵蓋率曲線；Geifman 與 El-Yaniv 說明了如何為深度網路選擇門檻，使其以高機率保證達到指定的風險 [11]。有用的門檻必須在驗證資料上選定，再只在測試資料上報告。

開放集合辨識：保留的類別

白話版。訓練時假裝某些數字不存在，再檢查它們出現時系統是否會拒絕。

Scheirer 等人將開放集合辨識形式化：分類器必須限制它標為已知的「開放空間」，否則一個以 K 類訓練的模型，會把世上任何輸入都歸到其中一類 [13]。最簡單的基準分數是最大 softmax（或類別）機率，由 Hendrycks 與 Gimpel 提出，用來偵測分錯的樣本與分布外樣本 [12]。

我們以縮小的標籤空間重做實驗。數字 0–5 為**已知**；數字 6–7 為開發階段可以看的**代理未知**；數字 8–9 為保留到最後評估的**測試未知**。驗證在兩條預先定義的管線之間做選擇，拒絕門檻設為驗證信心的第 5 百分位數，目標是接受約 95% 的已知數字。所有選擇在最後區塊之前就凍結。這支腳本是獨立的。

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import balanced_accuracy_score, confusion_matrix, roc_auc_score

digits = load_digits()
X, y = digits.data, digits.target
known = np.flatnonzero(y < 6)
train, rest = train_test_split(known, test_size=0.4,
                               stratify=y[known], random_state=10)
valid, test = train_test_split(rest, test_size=0.5,
                               stratify=y[rest], random_state=10)
proxy_unknown = np.flatnonzero((y == 6) | (y == 7))
```

```
test_unknown = np.flatnonzero(y >= 8)
assert not set(train) & (set(valid) | set(test))
assert not set(proxy_unknown) & set(test_unknown)

candidates = {
    "linear": make_pipeline(
        StandardScaler(), LogisticRegression(C=1.0, max_iter=1500)),
    "PCA + RBF": make_pipeline(
        StandardScaler(), PCA(n_components=24, random_state=10),
        SVC(C=4.0, gamma=0.02, probability=True, random_state=10)),
}
scores = {}
for name, model in candidates.items():
    model.fit(X[train], y[train])
    validation_p = model.predict_proba(X[valid])
    validation_pred = model.classes_[validation_p.argmax(axis=1)]
    scores[name] = balanced_accuracy_score(y[valid], validation_pred)
name = max(scores, key=scores.get)
model = candidates[name]
threshold = np.quantile(model.predict_proba(X[valid]).max(axis=1), 0.05)
proxy_conf = model.predict_proba(X[proxy_unknown]).max(axis=1)
print("Validation candidates:", scores, "selected:", name)
print("Fixed threshold:", threshold)
print("Proxy-unknown acceptance:", np.mean(proxy_conf >= threshold))

# Final evaluation begins here; do not revise choices from these results.
known_p = model.predict_proba(X[test])
known_conf = known_p.max(axis=1)
known_pred = model.classes_[known_p.argmax(axis=1)]
accept = known_conf >= threshold
unknown_conf = model.predict_proba(X[test_unknown]).max(axis=1)
print("Known test balanced accuracy:",
      balanced_accuracy_score(y[test], known_pred))
print("Known coverage:", accept.mean())
print("Accepted known accuracy:",
      np.mean(known_pred[accept] == y[test][accept]) if accept.any() else np.nan)
print("Unknown-test acceptance:", np.mean(unknown_conf >= threshold))
is_unknown = np.r_[np.zeros(len(test)), np.ones(len(test_unknown))]
unknown_score = 1 - np.r_[known_conf, unknown_conf]
print("Unknown detection AUROC:", roc_auc_score(is_unknown, unknown_score))
print("Known-class confusion matrix:\n", confusion_matrix(y[test], known_pred))
```

依輸出順序：驗證平衡準確率 0.982（線性）對 0.995（PCA + RBF），因此選 RBF 管線；門檻 0.741；**代理未知接受率 0.083**；已知測試平衡準確率 0.981；已知涵蓋率 0.949；被接受的已知樣本準確率 0.995；**測試未知接受率 0.350**；未知偵測 AUROC 0.944。

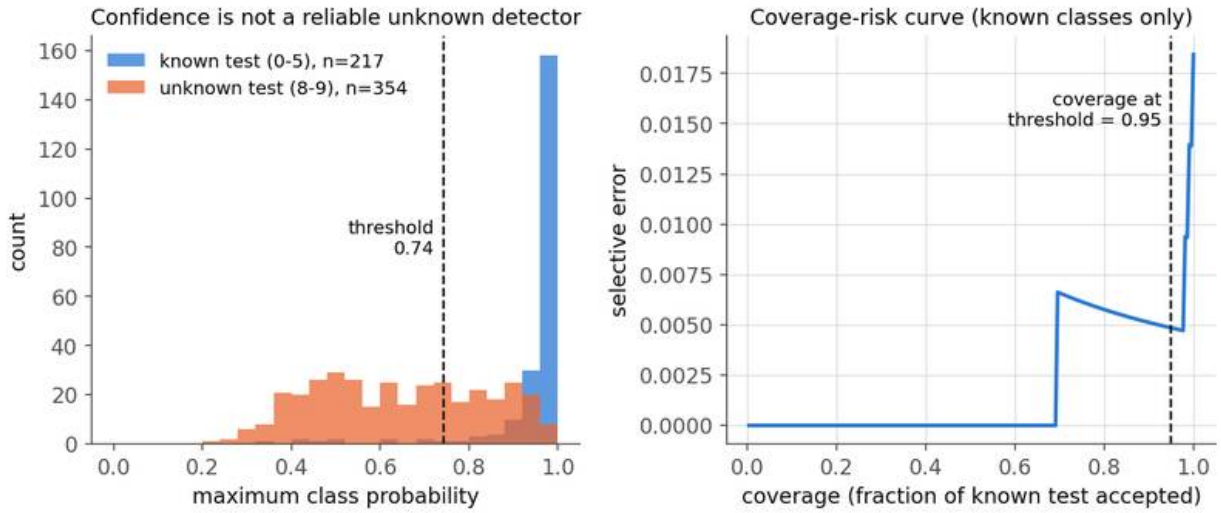


圖 *5* — 左：已知測試數字（*0–5*）與未知測試數字（*8–9*）的最大類別機率，門檻在驗證資料上固定。右：已知測試數字的風險–涵蓋率曲線；虛線為固定門檻下的涵蓋率。

這些數字道出開放集合評估的主要教訓：

- 在已知數字上，拒絕機制如預期運作：涵蓋率 95%，被接受的樣本準確率 99.5%。
- 代理未知（6 與 7）有 92% 被拒絕，這可能讓人以為信心門檻「解決了」未知類別。
- 真正的測試未知（8 與 9）**有 35% 被接受**。8 看起來像多了筆畫的 3 或 5，9 看起來像 5 或 3；分類器很有信心地把它們對應到這些已知數字。
- AUROC 0.944 概括的是對這幾種特定未知類別的排序能力，它不會決定操作門檻，也不代表其他種類的新奇輸入。

在一組代理未知上的表現，不會轉移到其他未知類別。請報告保留了哪些類別，也絕不要宣稱能拒絕「所有不熟悉的輸入」。Vaze 等人發現，較強的封閉集合分類器往往也是較好的開放集合辨識器，兩者在各方法間高度相關 [16]；因此改進基礎模型與使用更好的分數（特徵距離、能量或 logit 分數）是互補的。

另一類分數直接建模參考母體：單類 SVM（one-class SVM）估計包含大部分訓練資料的區域 [17]；孤立森林（Isolation Forest）則標出能被隨機軸向切分迅速孤立的點 [18]。「離群值」可能是擷取錯誤、罕見但有效的案例，或是新類別；統計上的不尋常並不會告訴你是哪一種。

分布漂移與壓力測試

白話版。部署之後，世界會改變。用受控的擾動提出「如果……」的問題，更好的做法是用來自新條件的真實資料。

令 $p_{\text{train}}(x, y)$ 與 $p_{\text{deploy}}(x, y)$ 分別為訓練與部署時特徵與標籤的聯合分布。Moreno-Torres 等人整理了分類問題中資料集漂移的術語 [19]。最重要的三種情況是：

- **先驗（標籤）漂移：**類別頻率 $p(y)$ 改變，$p(x \mid y)$ 不變。例如新表單版面讓某個數字變得常見許多。

- **共變量漂移：** $p(x)$ 改變，$p(y \mid x)$ 不變。例如新掃描器讓筆畫變粗。
- **概念漂移：** $p(y \mid x)$ 本身改變。例如寫 7 的新慣例多了一橫。

這些是建模假設，不是能從資料名稱讀出的診斷。換相機可能改變類別條件特徵；加入新類別則會徹底打破標籤空間。

我們無法在這裡蒐集新的手寫資料，但可以對測試影像施加受控擾動來探測敏感度。模型不重新訓練；擾動方式在看結果之前就決定好。

```
rng = np.random.default_rng(0)
def shift_right(X, k=1):
    imgs = X.reshape(-1, 8, 8)
    out = np.zeros_like(imgs)
    out[:, :, k:] = imgs[:, :, :-k]
    return out.reshape(len(X), 64)

conditions = {
    "clean": X[test],
    "noise sd=2": np.clip(X[test] + rng.normal(0, 2, X[test].shape), 0, 16),
    "noise sd=4": np.clip(X[test] + rng.normal(0, 4, X[test].shape), 0, 16),
    "shift 1 px right": shift_right(X[test], 1),
    "contrast x0.5": X[test] * 0.5,
}
for name, Xc in conditions.items():
    print(f"{name:18s} accuracy = {accuracy_score(y[test], final.predict(Xc)):.3f}")
```

（請接在步驟 8 的程式之後執行，而不是開放集合腳本之後，因為它使用十類版本的 `X`、`test` 與 `final`。）

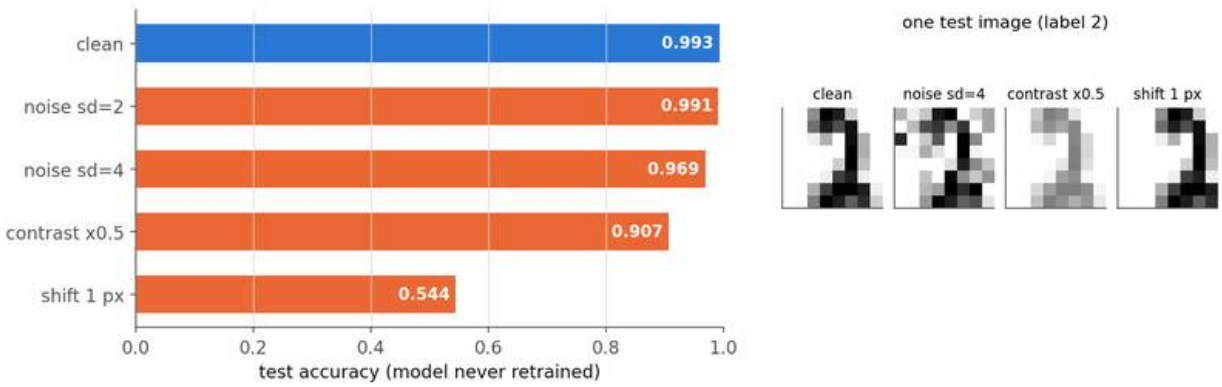


圖 *6* — 凍結管線在訓練時從未見過的擾動下的測試準確率，以及一張測試影像在各條件下的樣子。強度維持在資料集的 *0–16* 範圍內。

條件	測試準確率
乾淨	0.993
加性雜訊，標準差 2	0.991
加性雜訊，標準差 4	0.969

條件	測試準確率
對比減半	0.907
向右平移一格	**0.544**

加性雜訊幾乎沒有影響。對比減半損失約 9 個百分點，因為 RBF 核直接比較原始強度，而 γ 是針對全對比筆畫調的。**平移一格**幾乎讓準確率減半：以像素向量為輸入的分類器，並不知道平移後的 3 仍然是 3。一格相當於原始 32×32 點陣圖中 4 個像素的區塊，也就是影像寬度的八分之一，對裁切不精確的情況而言是很實際的錯位。這是把平移容忍度內建進模型的經典理由：透過資料擴增，或透過卷積與池化，LeCun 等人在文件辨識中展示了這一點 [10]（見ML 03）。

合成擾動是便宜的初步檢查，無法取代新條件下的真實資料。WILDS 等基準蒐集了跨醫院、相機與地區的真實分布漂移，顯示從分布內到分布外有明顯的效能下降 [21]；Recht 等人依原始流程重建 CIFAR-10 與 ImageNet 測試集，即使沒有刻意的漂移，許多模型的準確率仍然下降 [20]。請依有意義的條件（書寫者、裝置、場域、時間）分別報告結果，而不只是彙總。

名副其實的機率

如果下游使用者會依據信心值行動，這些值就必須校準：信心 0.8 的預測中，約 80% 應該是對的。Guo 等人指出現代深度網路常常過度自信，而在驗證資料上擬合單一純量溫度（temperature scaling）是強而有效的修正 [22]。對 SVM 而言，`SVC(probability=True)` 會在內部以交叉驗證擬合 Platt 縮放；`CalibratedClassifierCV` 則提供明確的控制。校準必須在沒有用來訓練基礎模型的資料上擬合，而且它無法修復分布漂移。

銜接學習式與基礎模型特徵

白話版。把手工特徵換成預訓練網路的特徵，本章其餘的一切都不變。

用預訓練編碼器取代特徵映射：

$$x = \phi_{\text{pretrained}}(I), \qquad \hat{y} = g_\theta(x),$$

其中 I 是原始影像，$\phi_{\text{pretrained}}$ 是凍結的網路（例如 DINOv2 這類自監督視覺 Transformer [25]），x 是它的嵌入，g_θ 是以你的標籤訓練、參數為 θ 的分類器。**線性探測**（linear probe，在凍結嵌入上做邏輯斯迴歸）是標準基準；請在固定切分與指標的前提下，將它與嵌入上的 k-NN、正則化的非線性分類頭以及微調比較。ML 05 說明這類特徵如何學得，ML 09 說明如何調適它們。

協定可以自然延伸：

- 記錄檢查點、輸入縮放、正規化、池化方式與嵌入維度；
- 下游前處理（標準化、PCA）只在訓練嵌入上擬合；
- 檢查點、提示詞或層的選擇都算模型選擇，必須在驗證資料上進行；
- 若評估影像可能出現在預訓練資料中，請說明你掌握了哪些關於重疊的證據；
- 以距離為基礎的未知分數（到訓練嵌入的 k-NN 距離）在好的嵌入上往往比在原始像素上有效得多，但仍需要保留的未知類別來評估。

表徵學習改變的是量測的取得方式，它不會改變最小風險決策、誠實的驗證，以及把已知類別準確率與拒絕、漂移分開看待的原則；這正是本系列的框架 [1], [2]。

要繳交什麼

一個完成的辨識專題，至少應包含：

1. **任務定義：**觀測、標籤空間、損失、拒絕政策。
2. **資料卡：**來源、授權、抽樣單位、類別數量、已知限制（本例：13 位書寫者、沒有書寫者編號）[24]。
3. **切分說明：**每個切分的用途、切分單位、亂數種子。
4. **基準與候選模型：**試過的所有模型，而不只是贏家，附上開發分數與其變異。
5. **選模證據：**所選流程的巢狀估計，以及與第二名的統計比較。
6. **錯誤分析：**混淆結構、範例、假設、學習曲線。
7. **最終測試結果：**數量、區間、各類別結果、範圍說明。
8. **壓力測試結果：**拒絕（涵蓋率與風險）、保留的未知類別、漂移或擾動測試；若使用機率，還要有校準。
9. **可重現性：**環境與函式庫版本、完整的管線物件、類別順序；若重要，還有執行時間與記憶體。

模型卡（model cards）將這份清單中面向部署的部分形式化（預期用途、評估條件、在相關族群上的表現、注意事項）[23]。如果原本的測試集影響了任何重新設計，修改後的系統就需要一份新的受保護測試集。

現代觀點

本章流程的原則由來已久，實務仍在演進。以下回顧與研究是很好的入門。

Raschka，〈Model evaluation, model selection, and algorithm selection in machine learning〉（2018）[9]。一篇長篇教學式回顧，把評估分成三個不同目標：估計泛化效能、選擇超參數與模型、比較演算法。內容涵蓋保留法、自助法與交叉驗證、巢狀交叉驗證，以及統計檢定（McNemar、5×2cv 等），並依資料量給出實務建議。它是最接近本章步驟 2 與步驟 6 的單一參考文獻。

Kapoor 與 Narayanan，〈Leakage and the reproducibility crisis in machine-learning-based science〉（*Patterns*，2023）[5]。蒐集了許多科學領域中已發表的洩漏案例，提出洩漏類型的分類，並設計「模型資訊表」（model info sheets），要求作者逐項說明為何不存在各類洩漏。主要結論是：洩漏很常見，往往在報告的數字中看不出來，最好透過協定預防，而不是事後偵測。

Geng、Huang 與 Chen，〈Recent advances in open set recognition: a survey〉（TPAMI，2021）[14]。區分已知的已知、已知的未知、未知的已知與未知的未知等類別，並把開放集合方法分為判別式（傳統機器學習與深度網路）與生成式（實例生成與非實例生成）兩大家族。它也說明開放集合辨識與零樣本、單樣本以及帶拒絕的分類之間的關係，並回顧常用的資料集與評估準則。

Yang、Zhou、Li 與 Liu，〈Generalized out-of-distribution detection: a survey〉（IJCV，2024）

[15]。把異常偵測、新奇偵測、開放集合辨識、分布外偵測與離群值偵測統一在同一個框架中，依漂移屬於語意或共變量、以及分布內任務是什麼來區分。決定你的專題屬於「四種任務」表的哪一列時，應該使用這套詞彙。

Vaze、Han、Vedaldi 與 Zisserman，〈Open-set recognition: a good closed-set classifier is all you need?〉（ICLR，2022）[16]。顯示封閉集合準確率與開放集合表現高度相關，訓練良好的基準搭配最大 logit 分數即可與專門方法競爭；並提出「語意漂移基準」（Semantic Shift Benchmark），把語意上的新奇與低階漂移分開。

Koh 等人的 WILDS（ICML，2021）[21] 與 Recht 等人（ICML，2019）[20]。這是評估漂移的兩種互補視角：WILDS 整理跨領域的真實部署漂移；Recht 等人則顯示，即使小心地「從同一分布」重新蒐集測試集，準確率仍可能下降。兩者共同主張依條件分別報告結果，並把單一的獨立同分布測試分數視為上限。

Guo 等人，〈On calibration of modern neural networks〉（ICML，2017）[22] 與 **Geifman 與 El-Yaniv（NeurIPS，2017）[11]** 是深度模型信心品質與選擇性預測的起點。

最新進展（2023–2026）。實務上，辨識專題越來越常從凍結或輕度微調的基礎模型特徵（例如 DINOv2 [25]）加上線性或 k-NN 分類頭開始，把省下的力氣投入資料品質、評估設計與文件記錄。分布外評估則朝向統一的基準，以及 [15] 所提出的語意與共變量區分發展。資料表 [24] 與模型卡 [23] 提供了現成的範本，可與這些結果一起記錄資料與模型。

開放問題。

- **未知的未知**。任何保留的「未知」類別都只是一個樣本；沒有通用的方法能保證拒絕所有新奇輸入。
- **漂移下的評估**。為每一種部署條件蒐集有標籤的資料代價高昂，而合成擾動並不能可靠地預測真實漂移。
- **預訓練重疊**。在網路規模的預訓練下，越來越難保證測試影像從未被看過，這讓「鎖住測試集」的原則更加複雜。
- **小樣本比較**。許多應用專題只有數百筆而非數百萬筆有標籤樣本；此時本章每個估計的變異都很大，統計比較不可或缺。

重點整理

- 先寫協定：觀測、抽樣單位、標籤空間、損失、拒絕政策與每個切分的角色。多數評估失敗其實是協定失敗。
- 在任何建模之前鎖住測試集，把每個需要擬合的轉換放進管線，並把特徵與前處理當成超參數。
- 在相同的折上比較所有候選模型；基準告訴你什麼叫「好」，而在這份數字資料上，3-NN 基準很難被打敗。
- 用巢狀交叉驗證估計含調參的流程，並在宣稱某模型較好之前使用校正後的重抽樣檢定。
- 在開發資料的折外預測上做錯誤分析；用學習曲線判斷該加資料還是換更強的模型。
- 只打開測試集一次，並報告數量、區間與範圍（本例：同樣 13 位書寫者的新影像）。
- 封閉集合準確率、選擇性風險、未知類別拒絕與漂移下的穩健性是不同的性質：一個 99% 準確的數字分類器，仍接受了 35% 的未見數字，且在平移一格時失去將近一半的準確率。

練習

1. **以距離做拒絕**。在開放集合實驗中，把最大類別機率換成測試影像在「以訓練資料擬合的 PCA 空間」中到最近訓練影像的距離。只用驗證集的已知數字選門檻，再在測試集上報告已知涵蓋率與未知接受率。它是否勝過信心門檻？

   提示

   只在 `X[train]` 上擬合 `StandardScaler` 與 `PCA`，再對轉換後的訓練集使用 `NearestNeighbors(n_neighbors=1)`。分數就是距離（越大越像「未知」）；把門檻設在驗證距離的第 95 百分位數，讓約 95% 的已知驗證數字被接受。兩個數字都要報告；看到測試結果後不得更改門檻。

2. **不作弊的風險–涵蓋率曲線**。以十類調參 SVM 的決策值作為信心，先用開發資料的折外預測畫出風險–涵蓋率曲線，再（僅供報告）畫出測試集上的曲線。說明為什麼選擇性準確率可能提高，而有用的涵蓋率卻崩潰；涵蓋率為零時又會怎樣？

   提示

   依信心由高到低排序樣本；每個截斷點的涵蓋率是保留比例，風險是保留樣本中的錯誤率。涵蓋率很低時只剩少數樣本，風險很不穩定；涵蓋率為零時風險沒有定義。在開發曲線上選定任一操作點，再讀出它在測試上的值。

3. **修補平移弱點**。在每一個訓練折中，加入影像向左、右、上、下各平移一格的副本。重跑步驟 6 的巢狀比較與步驟 9 的壓力測試。哪些準確率改變了？為什麼擴增必須在折內進行？

   提示

   寫一個小型轉換器或自訂迴圈，只擴增每次切分中的訓練部分；在切分前擴增，會把驗證影像的平移副本放進訓練，這就是洩漏。預期平移測試的準確率會大幅上升，而乾淨資料的準確率維持相近或略有變化。

4. **以設計達成書寫者不重疊的評估**。 UCI 資料集的訓練部分由 30 人書寫，測試部分由另外 13 人書寫 [3]。（在紙上）設計一個衡量新書寫者泛化能力的實驗。你預期本章哪些數字會改變？只靠內建資料，有哪些結論就無法再下？

   提示

   在一群書寫者上訓練與選模，在另一群上測試。預期 k-NN 的準確率特別會下降，因為它的優勢來自同一書寫者的近似重複樣本。由於同一部分內沒有書寫者編號，開發階段仍無法做分組交叉驗證，所以驗證依然混合了書寫者。

5. **顯著嗎？**使用步驟 5 的 10 個折分數，對邏輯斯迴歸與隨機森林、以及 LDA 與邏輯斯迴歸套用校正後的重抽樣 t 檢定。再計算不含 $n_{\text{te}}/n_{\text{tr}}$ 項的一般成對 t 檢定。在哪種情況下，兩種檢定會得到不同結論？

提示

5 折交叉驗證下 $n_{\text{te}}/n_{\text{tr}} \approx 0.25$，遠大於 $1/k = 0.1$，因此校正後的變異數約為一般變異數的 3.5 倍，t 統計量約縮小 $\sqrt{3.5} \approx 1.9$ 倍。差異小但一致的情況，正是兩者結論分歧之處。

6. **凍結特徵對微調**。針對你選擇的真實影像資料集，設計一個比較「凍結預訓練特徵上的線性探測」與「完整微調」的實驗。列出每一個可能洩漏測試資訊的決策。

提示

包括檢查點與層的選擇、影像縮放與正規化、資料擴增、學習率與訓練輪數的選擇（提前停止）、門檻與校準的擬合、若使用視覺–語言模型則還有提示詞設計，以及預訓練資料與測試影像之間可能的重疊。

參考文獻

1. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press (Elsevier), 2009; Ch. 1 “Introduction”, Ch. 5–7 on feature selection and generation, Ch. 10 “Supervised Learning: The Epilogue”. doi
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006; Sec. 1.5 (decision theory and the reject option), Sec. 1.3 (model selection), Ch. 14 (combining models). official page and PDF
3. E. Alpaydin and C. Kaynak, “Optical Recognition of Handwritten Digits,” UCI Machine Learning Repository, 1998. doi
4. F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. JMLR; dataset loader: load_digits
5. S. Kapoor and A. Narayanan, “Leakage and the reproducibility crisis in machine-learning-based science,” *Patterns*, vol. 4, no. 9, 2023. doi
6. S. Varma and R. Simon, “Bias in error estimation when using cross-validation for model selection,” *BMC Bioinformatics*, vol. 7, 91, 2006. doi
7. G. C. Cawley and N. L. C. Talbot, “On over-fitting in model selection and subsequent selection bias in performance evaluation,” *Journal of Machine Learning Research*, vol. 11, pp. 2079–2107, 2010. JMLR
8. C. Nadeau and Y. Bengio, “Inference for the generalization error,” *Machine Learning*, vol. 52, pp. 239–281, 2003. doi
9. S. Raschka, “Model evaluation, model selection, and algorithm selection in machine learning,” arXiv:1811.12808, 2018. arXiv
10. Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. doi
11. Y. Geifman and R. El-Yaniv, “Selective classification for deep neural networks,” in *Advances in Neural Information Processing Systems 30* (*NIPS*), 2017. proceedings
12. D. Hendrycks and K. Gimpel, “A baseline for detecting misclassified and out-of-distribution examples in neural networks,” in *Proc. ICLR*, 2017. arXiv

13. W. J. Scheirer, A. de Rezende Rocha, A. Sapkota and T. E. Boult, “Toward open set recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 7, pp. 1757–1772, 2013. doi
14. C. Geng, S.-J. Huang and S. Chen, “Recent advances in open set recognition: A survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 10, 2021. doi
15. J. Yang, K. Zhou, Y. Li and Z. Liu, “Generalized out-of-distribution detection: A survey,” *International Journal of Computer Vision*, 2024. doi
16. S. Vaze, K. Han, A. Vedaldi and A. Zisserman, “Open-set recognition: A good closed-set classifier is all you need?,” in *Proc. ICLR*, 2022. arXiv
17. B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola and R. C. Williamson, “Estimating the support of a high-dimensional distribution,” *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001. doi
18. F. T. Liu, K. M. Ting and Z.-H. Zhou, “Isolation forest,” in *Proc. IEEE International Conference on Data Mining (ICDM)*, 2008. doi
19. J. G. Moreno-Torres, T. Raeder, R. Alaiz-Rodríguez, N. V. Chawla and F. Herrera, “A unifying view on dataset shift in classification,” *Pattern Recognition*, vol. 45, no. 1, pp. 521–530, 2012. doi
20. B. Recht, R. Roelofs, L. Schmidt and V. Shankar, “Do ImageNet classifiers generalize to ImageNet?,” in *Proc. ICML*, PMLR 97, pp. 5389–5400, 2019. PMLR
21. P. W. Koh, S. Sagawa, H. Marklund et al., “WILDS: A benchmark of in-the-wild distribution shifts,” in *Proc. ICML*, PMLR 139, pp. 5637–5664, 2021. PMLR
22. C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. ICML*, 2017. arXiv
23. M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji and T. Gebru, “Model cards for model reporting,” in *Proc. Conference on Fairness, Accountability, and Transparency (FAT*)*, 2019. doi
24. T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III and K. Crawford, “Datasheets for datasets,” *Communications of the ACM*, vol. 64, no. 12, 2021. doi
25. M. Oquab, T. Darcet, T. Moutakanni et al., “DINOv2: Learning robust visual features without supervision,” arXiv:2304.07193, 2023. arXiv

篇 V

機器學習

ML 01・機器學習入門

電腦如何從資料中學習：學習範式、線性迴歸與邏輯迴歸、成本函數、梯度下降、過擬合與模型評估，全部用 NumPy 從頭實作。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-01-introduction/

你將學到

- 「從資料中學習」是什麼意思，以及它和手寫規則有什麼不同；全篇以手寫數字的小影像當作貫穿的例子。
- 主要的學習範式：監督式學習（迴歸與分類）、非監督式學習（分群、降維、異常偵測）、自監督式學習，以及強化學習，包括「基於人類回饋的強化學習」（RLHF）如何調校聊天模型。
- 一個學習問題的基本詞彙：模型 $f_{w,b}(x)$、參數、特徵、目標值、預測值、訓練／驗證／測試集，以及成本函數。
- 以均方誤差為成本的線性迴歸，以及用來最小化成本的梯度下降：更新規則、學習率、收斂，以及批次、小批次與隨機梯度下降的差別。
- 為什麼要做特徵縮放；邏輯迴歸如何用 sigmoid 函數與交叉熵損失把一條直線變成分類器；如何用驗證資料發現過擬合，並用 L2 正則化修正。
- 如何用準確率、精確率、召回率與 MSE 誠實地回報結果。

先看全貌

大多數程式都像食譜：有人把問題想清楚，再把每一個步驟寫下來。機器學習把這件事反過來。我們寫一個**有彈性**、裡面有很多可調整數字的程式，給它看大量「正確完成任務」的例子，再讓一個演算法去調整這些數字，直到程式把任務做好。範例取代了食譜。

用最簡單的話說

想想你小時候怎麼學會分辨貓和狗。沒有人給你一本規則手冊（「尖耳朵加鬍鬚、不會汪汪叫，就是貓」）。大人只是一次又一次指著說「貓」、「狗」、「貓」，你猜錯了就被糾正，直到你「就是知道」。機器學習也是這樣：我們給電腦看很多附上正確答案的例子，它先猜，我們告訴它錯了多少，它每次把猜測稍微修正一點，直到很少出錯。

為什麼重要：幾乎所有現代電腦視覺系統，從手機的臉部解鎖到醫學影像分析，都是學習出來的模型，而不是手寫的程式。本篇的幾個概念：帶有參數的模型、衡量誤差的成本、讓成本下降的梯度下降，以及用另外的資料檢查結果，正是今天訓練最大型神經網路所用的同一套想法，只是模型的規模不同。把這幾個概念弄清楚之後，本系列接下來的內容（ML 02 與各篇深度學習教學）主要就是在談「用**哪一種**模型」與「用**哪一種**成本」。

關於本篇

本篇是由我先前的筆記〈Introduction to Machine Learning〉[1] 擴寫而成的正式教學（原文為英文）。那份筆記是跟著吳恩達（Andrew Ng）的 Machine Learning Specialization [4] 與《Deep Learning》一書 [3] 整理的。

什麼是機器學習？

白話版。機器學習系統藉由觀察資料來把任務做得更好，而不是由程式設計師寫下每一個決定。

用學習取代寫程式

這個想法由來已久。1959 年，Arthur Samuel 描述了一個學會下西洋跳棋（checkers）、而且下得比他本人還好的程式：它下了很多盤棋，並據此調整自己評估棋局的方式 [2]。他的重點是：一台能從經驗中學習的電腦，可以省去這項任務原本需要的大量細部手寫程式。那句著名的一行定義，把機器學習說成「讓電腦不需明確寫程式就能學習的研究領域」，其實是後人對這個想法的轉述，通常被歸於 Samuel，但並不是他論文裡的原句。

教科書中常見一個更具操作性的定義，包含三個部分：一個**任務**（例如辨識數字）、一個**效能指標**（辨識正確的比例），以及**經驗**（一批有標註的數字影像）。如果程式在這個任務上的效能隨著經驗增加而提升，我們就說它在**學習** [3]、[7]。

影像的例子：規則與學習

我們貫穿全篇的例子是 scikit-learn 的 digits 資料集 [5]、[6]：1,797 張手寫數字的灰階影像，每張只有 8×8 個像素，數值從 0 到 16。就像〈Images as Arrays〉介紹的，每張影像只是一個小小的數字陣列，攤平之後就是 64 個數字組成的向量。

假設我們只需要分辨 **0** 和 **1**。程式設計師可能會寫一條規則：「1 很細、0 很寬，所以數一數有幾欄有墨跡；不超過五欄就是 1。」這條規則容易理解，對大多數直立的 1 也是對的。但人們寫 1 的方式各不相同，有的傾斜、有的帶襯線或底座，而傾斜的 1 佔的欄數和 0 一樣多（圖 1）。這條規則大約只對了 79%。想修好它就得加更多規則（「除非它是斜的 ……」），而每條新規則又會帶來新的例外。

機器學習的做法則**不寫任何**針對數字的規則。它挑一個有彈性的函數家族（這裡是邏輯迴歸，本篇後面會從頭實作），再讓演算法從範例中選出其中的 65 個數字。在沒看過的影像上，它把 0 和 1 完全分對；而且一模一樣的程式碼，不加任何新規則就能處理全部十個數字：

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

digits = load_digits()                      # 1,797 images, 8x8 pixels, values 0..16
X, y = digits.data, digits.target           # X: (1797, 64) flattened pixels; y: 0..9

# Hand-written rule for "0 versus 1": a 1 is thin, a 0 is wide.
```

```
keep = (y == 0) | (y == 1)
imgs, labels = digits.images[keep], y[keep]
width = (imgs.max(axis=1) > 4).sum(axis=1)   # how many columns contain ink
rule = np.where(width <= 5, 1, 0)
print("hand rule, 0 vs 1:", round((rule == labels).mean(), 3))          # 0.789

# Learned rule for the same task, judged on images it never saw.
X01 = imgs.reshape(len(imgs), -1)
Xa, Xb, ya, yb = train_test_split(X01, labels, test_size=0.3, random_state=0)
print("learned, 0 vs 1:", LogisticRegression(max_iter=1000).fit(Xa, ya).score(Xb, yb))   #
 ↪  1.0

# The same recipe scales to all ten digits with no new rules.
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=0)
print("learned, 10 digits:",
      round(LogisticRegression(max_iter=5000).fit(X_tr, y_tr).score(X_te, y_te), 3))  # 0.953
```

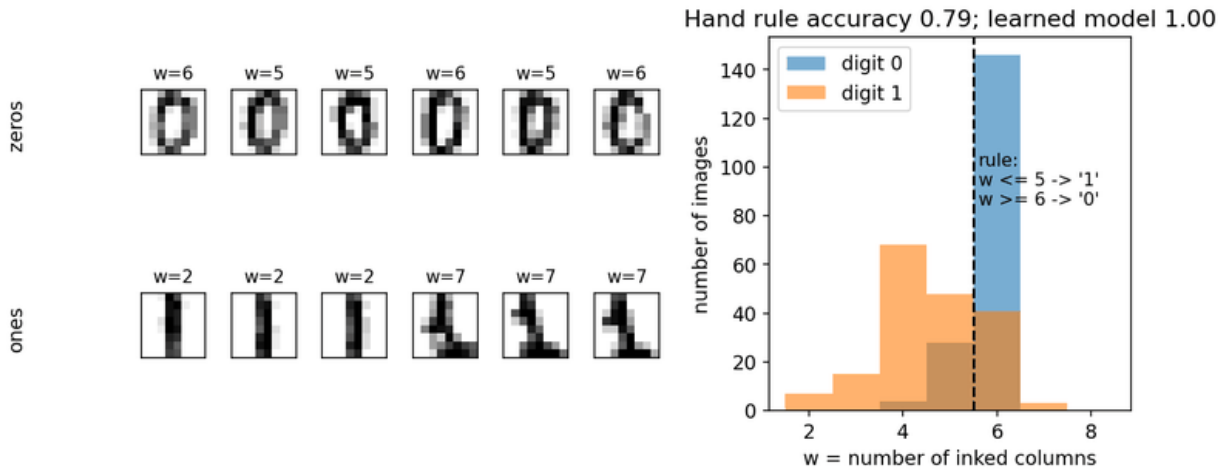


圖 *1* — 為什麼手寫規則很脆弱。左：*digits* 資料集中的 *0* 與 *1*；傾斜的 *1* 和 *0* 一樣寬。右：「數有墨跡的欄數」這個特徵在兩類之間大量重疊，所以任何門檻都會犯很多錯。學習出來的模型則同時使用全部 *64* 個像素。

這個小實驗包含了支持機器學習的全部理由：當規則難以用文字說清楚、範例卻很容易取得時，就從範例中學出規則。它也顯示了代價：學到的模型是一串 65 個數字，比「數欄數」難讀得多。

學習範式

白話版。各種學習問題的主要差別在於**學習者得到什麼樣的回饋**：每個範例都有正確答案、完全沒有答案、可以從資料本身製造答案，或是只偶爾得到一個「做得好不好」的分數。

範式	資料長什麼樣子	學到什麼	典型例子
監督式	成對的 (x, y)：輸入與「正確答案」	從 x 到 y 的函數	數字影像 → 數字標籤
非監督式	只有輸入 x	結構：群組、方向、離群值	把相似的數字分在一起
自監督式	只有輸入 x，但把 x 的一部分藏起來當作目標	通用的表示（representation）	預測被遮住的字詞或影像區塊
強化學習	代理人的動作，以及動作換來的獎勵	策略：在每種情況下該採取哪個動作	玩遊戲、調校聊天模型

我原本的筆記列了第一、第二和第四種；自監督式學習後來已成為預訓練大型模型的主要方式，所以這裡讓它單獨佔一列 [9]、[10]。

監督式學習

在監督式學習（supervised learning）中，學習者會拿到正確答案。資料是一組範例 $\{(x^{(i)}, y^{(i)})\}_{i=1}^{m}$，其中 $x^{(i)}$ 是輸入，$y^{(i)}$ 是期望的輸出（稱為**標籤**或**目標值**），m 是範例數。目標是找到一個函數 f，使 $f(x)$ 接近 y，**而且對資料中沒有的新輸入也一樣**。最後這一點稱為**泛化**（generalization），它才是真正的目標；把範例背起來很容易。

非監督式學習

在非監督式學習（unsupervised learning）中沒有答案，只有輸入 $\{x^{(i)}\}$。目標是從資料中找出有意思的東西：相似範例組成的群組、精簡的描述，或是格格不入的範例。由於沒有唯一「正確」的輸出，這類方法很適合用來探索資料，但也比較難評估。

自監督式學習

自監督式學習（self-supervised learning）使用沒有標註的資料，但把一部分藏起來、要模型預測被藏的部分，藉此把問題變成監督式問題。在文字上，遮住一個字詞，從句子的其餘部分預測它。在影像上，遮住大部分的區塊再把它們重建出來：遮罩自編碼器（masked autoencoder）會遮住一張影像 75% 的區塊，訓練網路把它們補回來 [12]。LeCun 與 Misra 認為，自監督式學習讓模型能從原始觀察中累積廣泛的背景知識，因為標籤是免費得到的 [9]。

重點不在「填空」本身，而在於：一個很會填空的模型，必定已經學到很多關於資料結構的知識。這個學到的內部**表示**可以再用很少的標籤，調整去做許多其他任務。這正是**基礎模型**（foundation model）背後的做法：在大規模、廣泛的資料上訓練（通常是自監督式），再調整去做各式各樣的下游任務 [10]。GPT-3 只是被訓練來在大量文字上預測下一個字，卻能憑提示中給的幾個例子完成許多語言任務，不需要再訓練 [11]。

強化學習

在強化學習（reinforcement learning, RL）中，一個**代理人**（agent）與**環境**互動。每一步它觀察一個**狀態**、選一個**動作**，然後得到一個**獎勵**（一個數字）和新的狀態。沒有人告訴它正確的動作；

它必須自己發現哪些動作能在長期累積較高的總獎勵，而獎勵常常在造成它的動作之後很久才出現。學到的是一個**策略**（policy），也就是在每個狀態下選擇動作的規則 [8]。Samuel 的跳棋程式就是這種情境的早期例子 [2]。

今天，強化學習最為人所知的用途是調校大型語言模型。在 Ouyang 等人描述的**基於人類回饋的強化學習**（reinforcement learning from human feedback, RLHF）[13] 中，先用人寫的示範答案對預訓練語言模型做微調（監督式學習）。接著由人對模型針對同一個提示所產生的幾個答案排序，再訓練一個獨立的**獎勵模型**來預測這些排序。最後用強化學習最佳化語言模型，讓它產生獎勵模型評分高的答案。作者回報，標註人員偏好他們 13 億參數的調校模型，勝過 1,750 億參數的 GPT-3 [13]。更近期，DeepSeek-R1 用大規模強化學習，搭配可自動檢查的獎勵（例如數學答案是否正確），來提升逐步推理的能力 [14]。

注意 RLHF 串起了三種範式：自監督式預訓練、監督式微調，以及強化學習。真實的系統會自由混用各種範式；上面那張表是一張地圖，而不是一道道牆。

監督式學習：迴歸與分類

白話版。監督式學習主要有兩種。**迴歸**預測一個數字；**分類**預測一個類別。

迴歸

在迴歸（regression）中，目標值 y 是一個實數，所以可能的輸出有無限多個。例如：預測股價、用之前的考試成績預測學生的期末成績，或從臉部影像預測一個人的年齡。最後一個例子說明：輸入可以是一張影像，輸出仍然只是一個數字。

分類

在分類（classification）中，目標值是一小組固定類別中的某一個。例如：垃圾郵件或非垃圾郵件（兩類，二**元分類**）、臉型的類別，或是腫瘤類型，如良性、惡性第 1 型、惡性第 2 型（**多類別分類**）。類別是名稱，不是數量：「第 2 類」並不是「第 1 類」的兩倍。分類器通常會對每個類別輸出一個機率，再挑最可能的那一個。

```
from sklearn.datasets import make_regression, make_classification
from sklearn.linear_model import LinearRegression, LogisticRegression

# Regression: the target is a real number.
X_r, y_r = make_regression(n_samples=100, n_features=1, noise=10.0, random_state=0)
reg = LinearRegression().fit(X_r, y_r)
print(reg.predict(X_r[:3]).round(1), y_r[:3].round(1))   # numbers vs numbers

# Classification: the target is one of a few labels.
X_c, y_c = make_classification(n_samples=100, n_features=2, n_redundant=0,
                               n_clusters_per_class=1, random_state=0)
clf = LogisticRegression().fit(X_c, y_c)
```

```
print(clf.predict(X_c[:5]), y_c[:5])                    # labels vs labels
print(clf.predict_proba(X_c[:2]).round(3))              # class probabilities
```

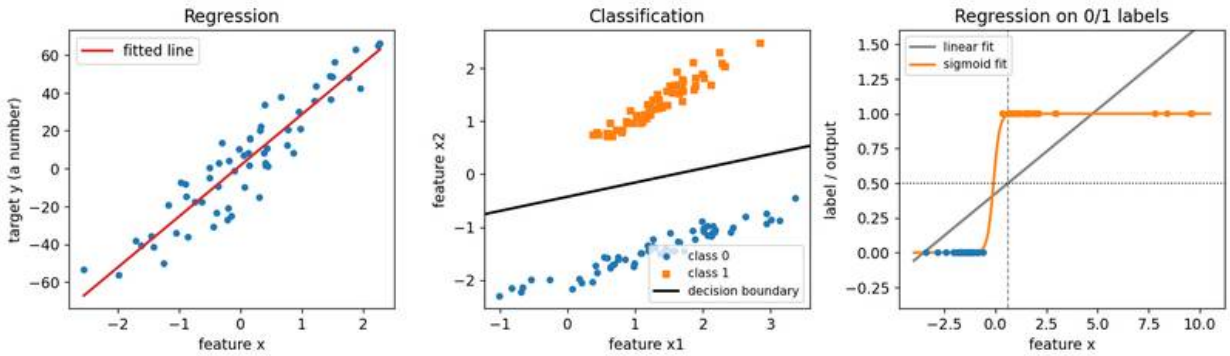


圖 2 — 左：迴歸讓曲線穿過數值目標。中：分類用決策邊界分開有標籤的點。右：對 *0/1* 標籤擬合一條迴歸直線（灰色），輸出會超出 $[0, 1]$，而且遠處的點會拉動它跨過 *0.5* 的位置；*sigmoid*（邏輯迴歸，橘色）才是合適的工具。

為什麼不用迴歸來做分類？

把類別編成 0 和 1、擬合一條迴歸直線、大於 0.5 就判為 1，這個做法很誘人。在簡單的資料上它也許可行，但通常表現很差，原因有兩個，都能在圖 2（右）看到。第一，直線會輸出 −0.4 或 1.7 這類數值，它們不是機率。第二，平方誤差會懲罰那些**已經分對**、只是離邊界很遠的點，所以少數極端但容易的範例會把直線拉過去，讓它跨過 0.5 的位置移到錯誤的地方。本篇後面的邏輯迴歸同時解決了這兩個問題。

非監督式學習

白話版。即使沒有答案，我們仍然可以問：哪些範例屬於同一群？怎樣用更少的數字描述每個範例？哪些範例很奇怪？

分群

分群（clustering）把相似的範例歸在一起。經典的演算法是 **k-means** [15]。先決定群數 K，從 K 個中心點出發，然後重複兩個步驟直到不再變化：把每個範例指派給最近的中心，再把每個中心移到被指派給它的範例的平均位置。每一步都只會讓範例到其中心的總平方距離

$$\sum_{i=1}^{m} \lVert x^{(i)} - \mu_{c^{(i)}} \rVert^2$$

變小或不變，其中 μ_k 是第 k 群的中心，$c^{(i)}$ 是範例 i 被指派到的群。所以演算法一定會停下來，但可能停在不好的局部解，這也是為什麼要從好幾個隨機起點各跑一次。Jain 回顧五十年分群研

究的文章 [15] 強調，一般而言沒有「最好」的分群演算法：「正確」的群組取決於「相似」對這份資料代表什麼意思。

一個常見的混淆：**k 近鄰法**（k-nearest neighbors, KNN）**不是**分群方法。KNN 對一個新範例分類的方式，是讓 *k* 個最相似的**有標籤**訓練範例投票，所以它是監督式方法 [5]。k-means 的 k 是群數；KNN 的 k 是鄰居數。

降維

降維（dimensionality reduction）用更少的數字描述每個範例，同時盡量保留有用的資訊。**主成分分析**（principal component analysis, PCA）找出資料變化最大的幾個方向，再把資料投影到其中前幾個方向上；新的座標彼此不相關，並依序盡可能捕捉最多的變異 [16]。兩個 PCA 座標就能把 64 個像素的數字影像變成平面上的點，讓我們看得到（圖 3 中）。

異常偵測

異常偵測（anomaly detection）找出與其他資料模式不符的範例：詐騙交易、瑕疵零件、損壞的影像。Chandola、Banerjee 與 Kumar 的綜述 [17] 依照各技術對「正常長什麼樣子」所做的假設，把眾多技術加以分類。一個簡單的版本使用 PCA：在資料上擬合幾個主成分，用它們重建每個範例，再標出重建誤差大的範例，因為主成分只描述了常見的部分。

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

X, y = load_digits(return_X_y=True)             # y is used only to check, never to fit

# Clustering: ten groups, found without looking at y.
km = KMeans(n_clusters=10, n_init=10, random_state=0).fit(X)
purity = sum(np.bincount(y[km.labels_ == k]).max() for k in range(10)) / len(y)
print("cluster purity:", round(purity, 3))      # 0.792: most clusters are one digit

# Dimensionality reduction: 64 numbers per image -> 2 numbers.
Z = PCA(n_components=2).fit_transform(X)        # shape (1797, 2)

# Anomaly detection: images that a 20-component PCA rebuilds badly are unusual.
pca20 = PCA(n_components=20).fit(X)
err = ((X - pca20.inverse_transform(pca20.transform(X))) ** 2).sum(axis=1)
print("most unusual images:", np.argsort(err)[-5:])
```

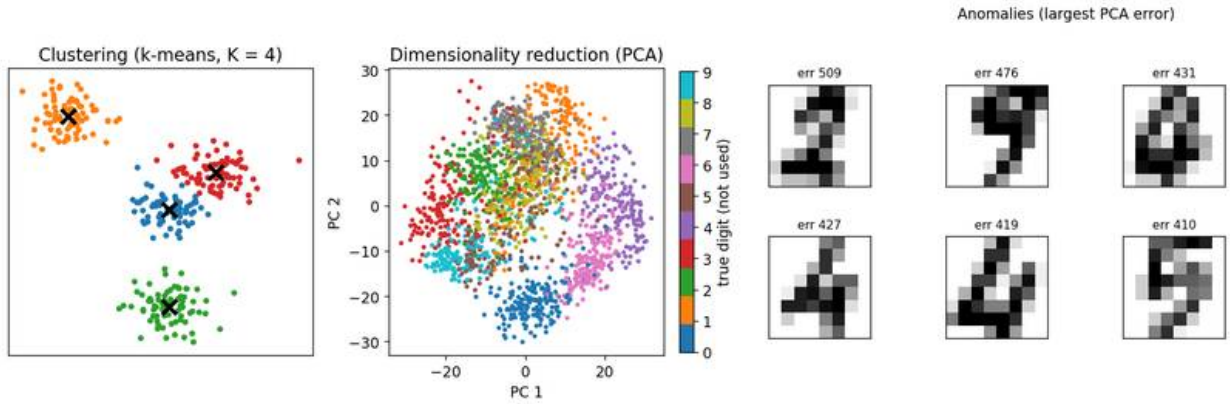


圖 3 — 非監督式學習。左：*k-means* 把點聚集在 K 個中心周圍。中：*PCA* 把每個 64 像素的數字壓縮成兩個數字（顏色是 *PCA* 從沒看過的真實數字）。右：*PCA* 重建誤差最大的數字，正是看起來最奇怪的那些。

程式中的純度（purity）檢查只在分群**之後**才用到標籤，用來看找出的群組是否和數字吻合。大約 79% 的影像落在以它自己的數字為主的群裡，即使 k-means 從沒看過任何標籤。

學習問題的詞彙

白話版。在談數學之前，我們需要替各部分取名字：什麼進去、什麼出來、正確答案是什麼、模型是什麼，以及怎麼衡量它的錯誤。

輸入、輸出與目標值

- **特徵**（features，輸入）x：描述一個範例的數字。對數字影像來說是 64 個像素值；對一棟房子來說是坪數和房間數。有 n 個特徵時寫成 $x = (x_1, \dots, x_n)$，x_j 是第 j 個特徵。
- **目標值**（target）y：正確的輸出，也叫標籤或**真值**（ground truth）。只有監督式學習有目標值。
- **預測值**（prediction）$\hat{y}$（讀作 y-hat）：模型對某個輸入的輸出。帽子代表「估計的」。
- **訓練範例** i：成對的 $(x^{(i)}, y^{(i)})$。括號中的上標是索引，不是次方。$x_j^{(i)}$ 是範例 i 的第 j 個特徵。

模型與參數

模型是把特徵對應到預測值的函數。我們寫成 $f_{w,b}$，表示它取決於稱為**參數**的可調整數字，這裡是 w（**權重**，weights）與 b（**偏差**，bias）：

$$\hat{y} = f_{w,b}(x).$$

選擇 f 的**形式**（直線、曲線、神經網路）是設計上的決定；選擇 w 和 b 的**數值**則是學習要做的事。由我們手動設定的數字，例如學習率或多項式的次數，稱為**超參數**（hyperparameters），以和學習出來的參數區分。

成本函數

損失（loss）$L(\hat{y}, y)$ 衡量一個預測錯了多少。**成本函數**（cost function）$J(w, b)$ 把損失對整個訓練集取平均，因此它是一個數字，說明在某組參數下整個模型錯了多少。模型是 f；成本是 J。把兩者分清楚可以避免很多混淆：$f_{w,b}(x)$ 吃一個輸入、吐出預測值；$J(w, b)$ 吃一組參數、吐出誤差。

於是訓練的**目標**就很明確：找出讓 J 盡可能小的參數，使所有訓練範例的 $\hat{y}^{(i)}$ 都接近 $y^{(i)}$。

訓練集、驗證集與測試集

資料會切成三份，各有不同的工作：

- **訓練集**用來擬合參數；
- **驗證集**在開發過程中用來比較模型、選擇超參數；
- **測試集**只在最後使用一次，用來估計最終模型在新資料上的表現。

測試集絕不能影響任何決定，否則估計值會過於樂觀。我們會在過擬合那一節回來說明**為什麼**要這樣切。

線性迴歸

白話版。在資料中畫一條直線。這條線有兩個旋鈕：斜率和高度。學習就是轉動旋鈕，直到直線盡可能貼近所有的點。

模型

只有一個特徵時，線性迴歸（linear regression）模型是

$$f_{w,b}(x) = wx + b,$$

其中 x 是特徵，w 是斜率（x 增加 1 時預測值改變多少），b 是截距（$x = 0$ 時的預測值）。改變 w 會讓直線傾斜；改變 b 會讓直線上下移動（圖 4 左）。

有 n 個特徵時，同一個模型是加權總和：

$$f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b = \sum_{j=1}^{n} w_j x_j + b,$$

其中 $\mathbf{w} = (w_1, \ldots, w_n)$ 為每個特徵各存一個權重。常見的技巧是加上一個常數特徵 $x_0 = 1$，偏差就變成另一個權重 w_0，模型縮寫成 $\mathbf{w} \cdot \mathbf{x}$。無論哪種寫法，「線性」指的都是對**參數**是線性的；之後我們會餵給它平方或三次方的特徵，它仍然是線性迴歸。

成本函數：均方誤差

對每個範例，**誤差**（殘差）是 $f_{w,b}(x^{(i)}) - y^{(i)}$。取平方讓誤差變成正數，並讓大誤差比小誤差受到更重的懲罰。對 m 個訓練範例取平均，就得到**均方誤差**（mean squared error, MSE）成本：

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right)^2,$$

其中 m 是訓練範例數，$(x^{(i)}, y^{(i)})$ 是第 i 個範例。多出來的 $\frac{1}{2}$ 只是為了方便：微分平方項時出現的 2 會和它抵消，而且它不會改變最小值的位置。

把成本想像成參數上的一片地形。如果固定 b、只改變 w，J 是一條拋物線：太平或太陡的直線誤差都很大，介於中間的某個斜率最好（圖 4 中）。同時考慮 w 和 b 時，J 是一個碗，我們把它畫成由橢圓組成的等高線圖（圖 4 右）。最好的直線就在碗底。

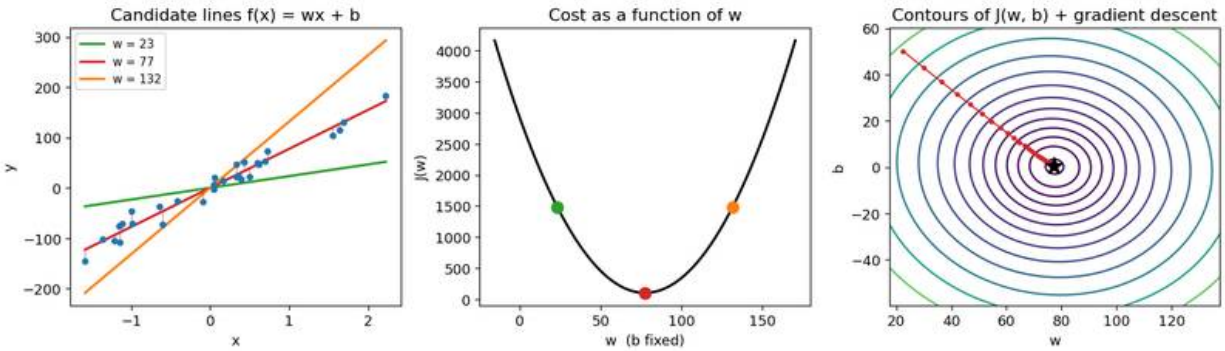


圖 4 — 從直線到成本。左：三條候選直線；垂直線段是最佳直線的殘差。中：固定 b 時，$J(w)$ 是拋物線，每條直線都是上面的一個點。右：$J(w,b)$ 是一個碗（等高線），梯度下降沿著下坡走到碗底。

為什麼這個碗只有一個底

對線性迴歸來說，MSE 成本是 (w,b) 的**凸函數**（convex）：曲面上任兩點之間的直線段都位於曲面上方或曲面上。凸函數沒有其他的局部最小值；每個最小值都是全域最小值。這是一個特別友善的情況。對許多其他模型，包括ML 02 的神經網路，成本地形有許多山谷，最佳化方法可能停在不是最低的那一個。原本的筆記正是在講這一點：一般而言，我們能期待的最好結果是找到一個夠好、接近全域最小值的局部最小值。

對線性迴歸，我們甚至可以用公式直接解出碗底（正規方程式，`np.polyfit` 就是這麼做的）。我們仍然使用梯度下降，因為在沒有公式可用時，它依然有效。

梯度下降

白話版。你在濃霧中站在成本地形的某處。感覺一下地面往哪邊傾斜，往下坡走一小步，再重複。等地面平了，你就到底了。

更新規則

導數 $\partial J/\partial w$ 告訴我們 w 稍微增加時 J 會怎麼變：正值表示「往右是上坡」，負值表示「往左是上坡」。它的正負號給出方向，大小說明坡有多陡。梯度下降（gradient descent）讓每個參數**逆著**它的導數走一小步：

$$w \leftarrow w - \alpha \frac{\partial J(w,b)}{\partial w}, \qquad b \leftarrow b - \alpha \frac{\partial J(w,b)}{\partial b},$$

其中 $\alpha > 0$ 是**學習率**（步長），$\leftarrow$ 表示「以右邊取代」。兩個導數都要在**目前**的 (w,b) 計算完之後才更新任何一個參數，這叫做**同步更新**。所有偏導數組成的向量就是**梯度** ∇J，它指向上升最快的方向，所以這個更新是朝下降最快的方向移動 [3]。

對線性迴歸的 MSE 成本，連鎖律給出

$$\frac{\partial J}{\partial w} = \frac{1}{m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right)x^{(i)}, \qquad \frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right).$$

用文字讀：w 的梯度是「平均誤差乘以特徵」；b 的梯度是「平均誤差」。如果預測值平均而言太高，b 就往下調。如果預測值主要在 x 很大的地方太高，w 就往下調。有 n 個特徵時，$\partial J/\partial w_j$ 用 $x_j^{(i)}$ 取代 $x^{(i)}$。

從頭實作

```
import numpy as np
from sklearn.datasets import make_regression

X, y = make_regression(n_samples=100, n_features=1, noise=10.0, random_state=0)
x = X[:, 0]                                   # one feature, shape (m,)
m = len(x)

def cost(w, b):
    return ((w * x + b - y) ** 2).sum() / (2 * m)

w, b, alpha = 0.0, 0.0, 0.1
for step in range(200):
    err = w * x + b - y                       # f_wb(x_i) - y_i for every i
    dj_dw = (err * x).sum() / m
    dj_db = err.sum() / m
    w, b = w - alpha * dj_dw, b - alpha * dj_db   # simultaneous update
    if step % 50 == 0:
        print(f"step {step:3d}  J = {cost(w, b):8.2f}")

print("gradient descent:", round(w, 3), round(b, 3))   # 42.619 -0.814
print("closed form     :", np.polyfit(x, y, 1).round(3))  # [42.619 -0.814]
```

成本在 50 步內從大約 800 降到 57，之後就不再變化。梯度下降和精確公式的結果在小數點後三位都一致。剩下的成本不是零，因為資料中含有任何直線都無法解釋的雜訊。

收斂：導數為零的地方

「重複直到收斂」需要一個定義。在最小值處地形是平的，所以那裡的**導數為零**，更新 $w - \alpha \cdot 0$ 不會改變 w。要注意變成零的是**導數**，不是成本：最小值處的成本只是能達到的最小數值，在上面的例子中是 57。實務上，當兩次迭代之間成本下降得比一個小容許值還少、梯度的大小低於某個門檻，或是用完固定的步數時，就停止。

一個有用的推論：越接近最小值，導數越小，所以即使 α **固定**，步伐 $\alpha\,\partial J/\partial w$ 也會自動變小。梯度下降在接近谷底時會自然放慢。

選擇學習率

學習率是梯度下降最重要的超參數（圖 5）：

- **太小：**每一步都很小。演算法確實在下坡，但可能需要非常多步。
- **剛好：**成本快速下降並穩定下來。
- **太大：**一步跨過谷底跳到山谷另一側，甚至可能比起點還高。參數來回彈跳，可能永遠停不下來（**越過頭**，overshoot），甚至一路飛向無限大（**發散**，divergence）。

標準的診斷工具是**學習曲線**：把 J 對迭代次數作圖。學習率合適時，J 每一步都會下降。如果 J 曾經**上升**，就是 α 太大（或梯度的程式有錯）。實用的做法是以大約 3 倍的間隔嘗試數值，例如 0.001、0.003、0.01、0.03、0.1、0.3，然後挑曲線仍平穩下降的最大那一個。

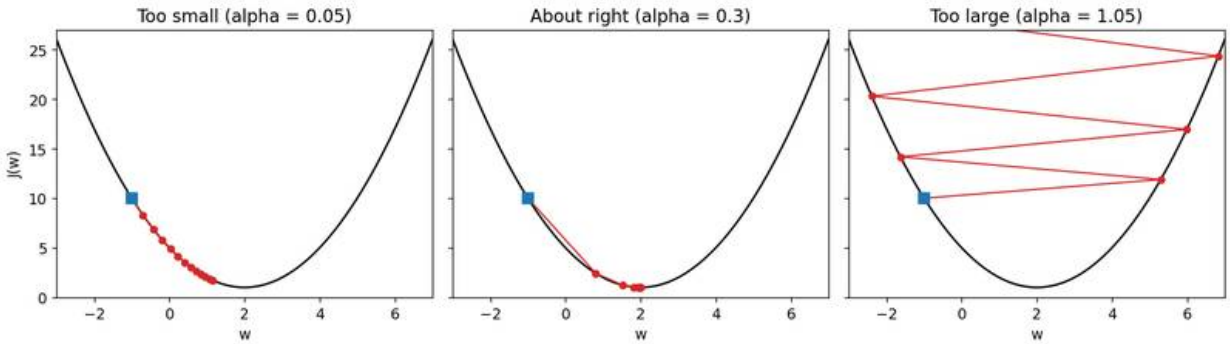


圖 5 — 同一個成本 $J(w)$、同一個起點、三種學習率。太小：進展緩慢。剛好：快速收斂。太大：每一步越過得更遠，迭代發散。

許多最佳化方法會在訓練過程中調整步長：用一個**排程**（schedule）隨時間降低 α，或像 Adam 那樣，用每個參數過去梯度的移動統計量來縮放它的步伐 [19]。這讓「接近最小值時步伐變小」成為明確的設計，而不只是副作用。

批次、小批次與隨機梯度下降

梯度公式要對全部 m 個範例加總。100 個範例沒問題，一億個就是麻煩了。關鍵的洞見是：梯度是一個**平均值**，也就是對資料的期望值，而平均值可以從樣本估計。[3] 對此有清楚的說明，Bottou、Curtis 與 Nocedal [18] 則做了深入的分析。

- **批次梯度下降**（batch gradient descent）每次更新都使用**整個**訓練集。每一步都精確，但也昂貴。
- **小批次梯度下降**（mini-batch gradient descent）先打亂資料，每次更新只用一個隨機的小子集（**小批次**），大小為 B 個範例，通常是 32 到幾百。每一步是對完整梯度帶雜訊但不偏的估計，成本只有 B/m。
- **隨機梯度下降**（stochastic gradient descent, SGD）在嚴格意義上每次更新只用一個範例（$B = 1$）。實務上「SGD」也常指小批次的版本。

完整走過訓練資料一遍稱為一個 **epoch**。注意程式中的命名：深度學習函式庫中的 `batch_size` 參數指的是**小批次**的大小，而不是批次梯度下降的「整個資料集」。

小批次梯度的雜訊不只可以容忍，往往還有幫助：在相同的 epoch 數內，小批次下降比批次下降多做了非常多次更新。Bottou 等人認為，這正是隨機方法主導大規模學習的原因：它們用遠少於精確方法的計算量，就能達到有用的準確度 [18]。

```
import numpy as np
from sklearn.datasets import make_regression

X, y = make_regression(n_samples=10_000, n_features=5, noise=10.0, random_state=0)
m, n = X.shape
rng = np.random.default_rng(0)

def minibatch_gd(batch_size, alpha=0.05, epochs=5):
    w, b = np.zeros(n), 0.0
    for epoch in range(epochs):
        order = rng.permutation(m)                    # shuffle once per epoch
        for start in range(0, m, batch_size):
            idx = order[start:start + batch_size]
            err = X[idx] @ w + b - y[idx]
            w -= alpha * X[idx].T @ err / len(idx)    # gradient estimated on the batch
            b -= alpha * err.mean()
    return ((X @ w + b - y) ** 2).mean() / 2

for bs in [m, 64, 1]:                                 # batch, mini-batch, stochastic
    print(f"batch size {bs:5d}: J = {minibatch_gd(bs, alpha=0.05 if bs > 1 else 0.005):.2f}")
# batch size 10000: J = 6127.00   (only 5 updates in 5 epochs)
# batch size    64: J = 49.93     (785 updates)
# batch size     1: J = 50.64     (50,000 noisy updates)
```

同樣走過資料五遍，批次下降只走了五步，離谷底還很遠；小批次下降則基本上已經收斂（這份資料的雜訊程度讓可達到的最佳 J 大約是 50）。單一範例的 SGD 也能到達，但因為梯度雜訊較大，需要較小的學習率，而且無法有效利用向量化運算。

特徵縮放

白話版。如果一個特徵以千為單位、另一個以小數為單位，成本地形會變成又長又窄的山谷，梯度下降會左右來回鋸齒狀前進。把所有特徵放到相近的尺度，山谷就會變成圓圓的碗，很容易走到底。

為什麼尺度很重要

假設特徵 x_1（例如以平方英尺計的房屋面積）的範圍是好幾千，特徵 x_2（房間數）的範圍是 1 到 5。w_1 稍微改變一點，預測值就變很多；w_2 做同樣的改變卻幾乎沒影響。所以 J 沿著 w_1 方向非常陡，沿著 w_2 方向非常平：等高線是細長的橢圓。學習率若小到不會在陡的方向發散，在平的方向上就幾乎沒有進展（圖 6 左）。

標準化與最小–最大縮放

最常見的解法是**標準化**（standardization，z 分數正規化）：

$$x'_j = \frac{x_j - \mu_j}{\sigma_j},$$

其中 μ_j 與 σ_j 是特徵 j **在訓練集上計算**的平均值與標準差。縮放後每個特徵的平均為 0、標準差為 1。另一個做法是**最小–最大縮放**（min–max scaling），$x'_j = (x_j - \min_j)/(\max_j - \min_j)$，把訓練集的範圍對應到 $[0, 1]$，這也是像素強度的常用選擇（8 位元影像除以 255）。兩者在 scikit-learn 中分別是 `StandardScaler` 與 `MinMaxScaler` [5]。

兩條規則可以避免隱微的錯誤。只在訓練集上計算 μ_j, σ_j，再把**同一組**數字套用到驗證、測試和未來的資料。並且記得縮放會改變學到的權重的意義：權重現在代表特徵每增加一個標準差時預測值改變多少。

```
import numpy as np
from sklearn.datasets import make_regression

X, y = make_regression(n_samples=200, n_features=2, noise=5.0, random_state=1)
X = X * np.array([1000.0, 1.0])               # feature 1 now has a ~1000x wider range

def gd(X, y, alpha, steps=500):
    m, n = X.shape
    w, b = np.zeros(n), 0.0
    for _ in range(steps):
        err = X @ w + b - y
        w -= alpha * X.T @ err / m
        b -= alpha * err.mean()
    return ((X @ w + b - y) ** 2).mean() / 2

print("raw,    alpha=1e-7:", round(gd(X, y, 1e-7), 2))    # 158.01; alpha=3e-6 diverges
mu, sigma = X.mean(axis=0), X.std(axis=0)
```

```
Xs = (X - mu) / sigma                          # z-score standardization
print("scaled, alpha=0.1 :", round(gd(Xs, y, 0.1), 2))    # 13.47
```

在原始特徵上，任何大於約 2×10^{-6} 的學習率都會發散；用安全的學習率跑 500 步，成本仍卡在 158 左右（跑 5,000 步也還是 158 左右），因為第二個權重幾乎不動。標準化之後，$\alpha = 0.1$ 輕鬆到達最小值（大約 13，也就是雜訊的程度）。

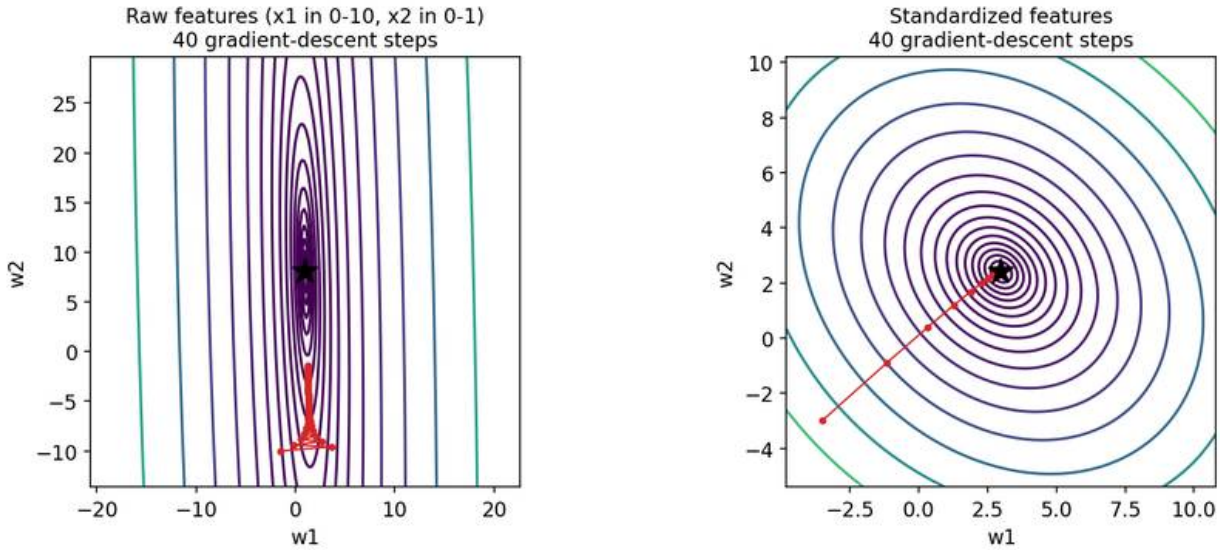


圖 *6* — 特徵縮放重塑了成本地形。左：尺度差異很大的特徵形成狹窄的山谷，梯度下降呈鋸齒狀前進。右：標準化之後，等高線接近圓形，同樣的步數就能到達最小值。

用於分類的邏輯迴歸

白話版。保留線性迴歸的加權總和，但把輸出壓進 0 到 1 之間，讓它可以被解讀成機率。然後選擇權重，讓真實類別的機率越高越好。

Sigmoid 函數

Sigmoid（logistic）函數是

$$g(z) = \frac{1}{1 + e^{-z}},$$

其中 z 是任意實數。它把 $-\infty$ 對應到 0、$+\infty$ 對應到 1、0 恰好對應到 0.5，中間是一條 S 形曲線（圖 7 左）。它還有對稱性 $g(-z) = 1 - g(z)$，以及方便的導數 $g'(z) = g(z)\,(1 - g(z))$。

模型與決策邊界

邏輯迴歸（logistic regression）把線性模型送進 sigmoid：

$$f_{\mathbf{w},b}(\mathbf{x}) = g(\mathbf{w}\cdot\mathbf{x}+b) = \frac{1}{1+e^{-(\mathbf{w}\cdot\mathbf{x}+b)}},$$

並把輸出解讀為標籤為 1 的估計機率，$f_{\mathbf{w},b}(\mathbf{x}) = P(y=1 \mid \mathbf{x})$。名字雖然叫「迴歸」，它其實是分類方法。做決定時，只要這個機率至少 0.5 就預測為 1。由於 $g(z) \geq 0.5$ 恰好在 $z \geq 0$ 時成立，規則就是

$$\hat{y} = 1 \quad \text{if} \quad \mathbf{w}\cdot\mathbf{x}+b \geq 0, \qquad \hat{y} = 0 \text{ otherwise.}$$

滿足 $\mathbf{w}\cdot\mathbf{x}+b=0$ 的集合就是**決策邊界**（decision boundary）。有兩個特徵時它是一條直線；一般而言它是一個平的超平面，而 $\mathbf{w}$ 與它垂直。離邊界很遠的點，機率接近 0 或 1；靠近邊界的點，機率接近 0.5。若想得到彎曲的邊界，就餵給模型非線性的特徵，例如 x_1^2 或 x_1x_2；模型對參數仍然是線性的。

成本：交叉熵（對數損失）

為什麼不沿用平方誤差？和 sigmoid 結合之後，平方誤差成本對 $(\mathbf{w},b)$ **不是**凸的：它會出現平坦的高原和多個山谷。所以邏輯迴歸改用**對數損失**（log loss），也稱為**二元交叉熵**（binary cross-entropy）：

$$L(f,y) = -y\log f \;-\; (1-y)\log(1-f),$$

其中 $f = f_{\mathbf{w},b}(\mathbf{x})$ 是預測機率，$y \in \{0,1\}$ 是真實標籤。每個範例只會用到兩項中的一項。若 $y=1$，損失是 $-\log f$：$f=1$ 時為零，$f \to 0$ 時無限增大。若 $y=0$，損失是 $-\log(1-f)$，正好是鏡像（圖 7 中）。信心滿滿卻答錯會受到很重的懲罰；信心滿滿且答對則幾乎沒有成本。

成本是整個訓練集上的平均：

$$J(\mathbf{w},b) = \frac{1}{m}\sum_{i=1}^{m}\Big[-y^{(i)}\log f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - (1-y^{(i)})\log\big(1-f_{\mathbf{w},b}(\mathbf{x}^{(i)})\big)\Big].$$

這個選擇並不隨意。如果把每個標籤看成一次擲硬幣，出現 1 的機率是 $f_{\mathbf{w},b}(\mathbf{x}^{(i)})$，那麼 J 正好是訓練標籤的負對數概似（negative log-likelihood）除以 m。最小化它就是**最大概似估計**（maximum likelihood estimation）[3]。而且搭配 sigmoid 時，這個成本是凸的，所以梯度下降又能找到唯一的全域最小值。

再一次梯度下降

利用 $g' = g(1-g)$，對數損失的導數會化簡得非常漂亮：

$$\frac{\partial J}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m}\big(f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - y^{(i)}\big)\,x_j^{(i)}, \qquad \frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}\big(f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - y^{(i)}\big).$$

它和線性迴歸的梯度**形式完全相同**：平均誤差乘以特徵。唯一的差別在 f 的內部。這不是巧合；只要輸出函數和它相配的概似函數成對使用，整個模型家族都有這個性質，這也是梯度下降程式碼這麼容易重複使用的原因之一。

```
import numpy as np
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=200, n_features=2, n_redundant=0,
                           n_clusters_per_class=1, class_sep=1.5, random_state=3)

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

def log_loss(p, y, eps=1e-12):
    p = np.clip(p, eps, 1 - eps)              # avoid log(0)
    return -(y * np.log(p) + (1 - y) * np.log(1 - p)).mean()

w, b, alpha = np.zeros(2), 0.0, 0.5
for step in range(1000):
    p = sigmoid(X @ w + b)                    # predicted P(y = 1 | x)
    w -= alpha * X.T @ (p - y) / len(y)       # same form as linear regression
    b -= alpha * (p - y).mean()

p = sigmoid(X @ w + b)
print("log loss:", round(log_loss(p, y), 3))  # 0.06
print("accuracy:", ((p >= 0.5) == y).mean())  # 0.99
print("boundary: %.2f*x1 + %.2f*x2 + %.2f = 0" % (w[0], w[1], b))
```

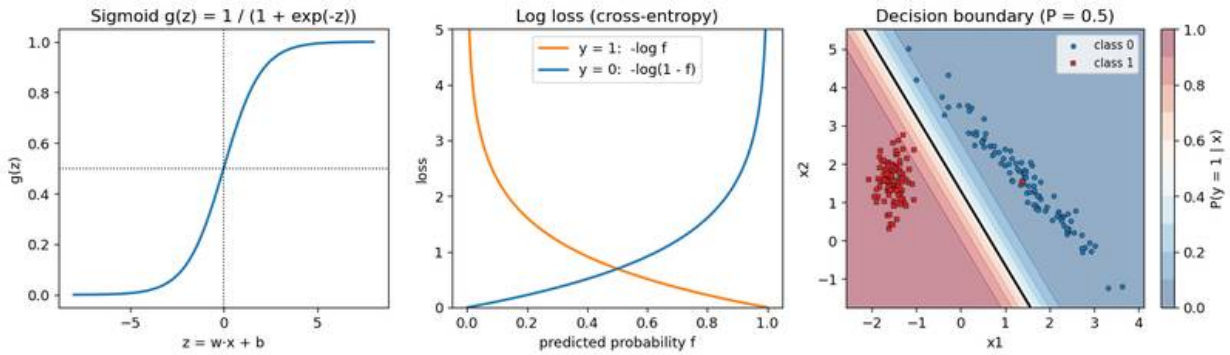


圖 7 — 邏輯迴歸。左：*sigmoid* 把任何分數壓進 $(0, 1)$。中：$y = 1$ 與 $y = 0$ 的對數損失，對信心滿滿的錯誤懲罰很重。右：從頭訓練的模型；底色是預測機率，黑線是決策邊界 $w{\cdot}x + b = 0$。

超過兩個類別

當類別數 $K > 2$ 時，標準的推廣方式是讓每個類別有自己的權重向量和分數 $z_k = \mathbf{w}_k \cdot \mathbf{x} + b_k$，再用 **softmax** 函數把分數轉成總和為 1 的機率：$P(y = k \mid \mathbf{x}) = e^{z_k} / \sum_{l=1}^{K} e^{z_l}$。成本同樣是真實類別的負對數機率的平均（類別交叉熵）[3]。本篇開頭 scikit-learn 的 `LogisticRegression` 處理十個數字時做的就是這件事，而它也正是ML 02 中大多數分類網路的輸出層。

過擬合、欠擬合與正則化

白話版。太簡單的模型抓不到模式（**欠擬合**，underfitting）。太有彈性的模型會把訓練範例連同雜訊一起背下來，遇到新範例就失敗（**過擬合**，overfitting）。我們用模型沒訓練過的資料來分辨兩者，並用懲罰大權重的方式馴服過於有彈性的模型。

親眼看它發生

取 25 個帶雜訊的平滑曲線樣本，擬合不同次數的多項式，也就是在特徵 $x, x^2, \dots, x^d$ 上做線性迴歸（圖 8）：

- **1 次（欠擬合，高偏差）**。直線跟不上曲線。不論在訓練集**還是**新資料上，誤差都很大。
- **3 次（擬合良好）**。曲線跟著趨勢走，忽略雜訊。兩邊的誤差都很小。
- **15 次（過擬合，高變異）**。曲線幾乎精確地穿過每個訓練點，所以訓練誤差接近零。但在點與點之間和點的範圍之外，它劇烈擺盪，新資料上的誤差暴增。

光看訓練誤差無法分辨好模型和過擬合的模型；它甚至偏好過擬合的那個。只有模型沒看過的資料才能揭露差別。這就是我們保留驗證集的原因。

使用驗證集與測試集

流程是：

1. 在**訓練集**上擬合每個候選模型（每種次數、每種正則化強度）。
2. 在**驗證集**上比較候選模型，保留最好的那個。
3. 在**測試集**上回報所選模型的誤差，只做一次。

為什麼要三份而不是兩份？因為在驗證集上從眾多候選中挑出最好的，本身就是一種擬合：勝出者有一部分只是剛好在那些特定範例上運氣好。沒動過的測試集才能給出不偏的最終估計。資料很少時，可以用**交叉驗證**（cross-validation）取代單一的驗證集：把訓練資料切成 k 份，訓練 k 次，每次用不同的一份做驗證，再取平均。

要留意的模式：如果訓練誤差和驗證誤差都很高，模型欠擬合（換更有彈性的模型或更好的特徵）。如果訓練誤差很低、驗證誤差卻高得多，模型過擬合（取得更多資料、簡化模型，或加上正則化）。

L2 正則化

正則化（regularization）保留有彈性的模型，但不鼓勵極端的參數值。15 次多項式那條狂野的曲線，需要非常大、又彼此精細抵消的權重。**L2 正則化**在成本上加入權重平方大小的懲罰：

$$J_{\text{reg}}(\mathbf{w}, b) = J(\mathbf{w}, b) + \frac{\lambda}{2m} \sum_{j=1}^{n} w_j^2,$$

其中 J 是原本的成本（MSE 或對數損失），$\lambda \geq 0$ 是**正則化強度**（一個超參數），加總的範圍是所有權重，但慣例上不包括偏差 b。$\lambda = 0$ 時回到原本的模型；λ 越大，權重越被拉向零，擬合出的函數越平滑；λ 太大則會欠擬合。對線性迴歸來說，這就是**脊迴歸**（ridge regression），由 Hoerl 與 Kennard 提出，用來在特徵高度相關時穩定最小平方估計 [20]。

從梯度下降的更新可以看出懲罰項做了什麼：

$$w_j \leftarrow w_j \left(1 - \alpha \frac{\lambda}{m}\right) - \alpha \frac{\partial J}{\partial w_j}.$$

在每次一般的更新之前，每個權重都先乘上一個略小於 1 的數。這就是為什麼 L2 正則化在神經網路訓練中也被稱為**權重衰減**（weight decay）[3]。

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import mean_squared_error

rng = np.random.default_rng(0)
x = rng.uniform(0, 1, 25)
y = np.sin(2 * np.pi * x) + rng.normal(0, 0.25, 25)      # true curve + noise
X = x[:, None]

# 60% train, 20% validation, 20% test
X_tmp, X_te, y_tmp, y_te = train_test_split(X, y, test_size=0.2, random_state=0)
X_tr, X_va, y_tr, y_va = train_test_split(X_tmp, y_tmp, test_size=0.25, random_state=0)

def fit(degree, lam=0.0):
    reg = Ridge(alpha=lam) if lam > 0 else LinearRegression()   # sklearn calls lambda
     ↪  "alpha"
    model = make_pipeline(PolynomialFeatures(degree), StandardScaler(), reg)
    return model.fit(X_tr, y_tr)

for degree, lam in [(1, 0), (3, 0), (15, 0), (15, 1e-2)]:
    model = fit(degree, lam)
    tr = mean_squared_error(y_tr, model.predict(X_tr))
    va = mean_squared_error(y_va, model.predict(X_va))
    print(f"degree {degree:2d}, lambda {lam:g}: train MSE {tr:.3f}, val MSE {va:.3g}")

best = fit(15, 1e-2)                                    # chosen on validation only
print("test MSE:", round(mean_squared_error(y_te, best.predict(X_te)), 3))
```

輸出說明了一切：1 次的訓練／驗證 MSE 是 0.144 ／ 0.292（欠擬合），3 次是 0.031 ／ 0.052（良好），沒有正則化的 15 次訓練 MSE 只有 0.003，驗證 MSE 卻高達數百萬，因為有些驗證點落在訓練點之間很寬的空隙中（例如 $x \approx 0.27$ 到 $x \approx 0.61$ 之間），多項式在那裡劇烈擺盪。對同一個 15 次模型加上溫和的 L2 懲罰（$\lambda = 0.01$），得到 0.041 ／ 0.035，測試 MSE 為 0.075。

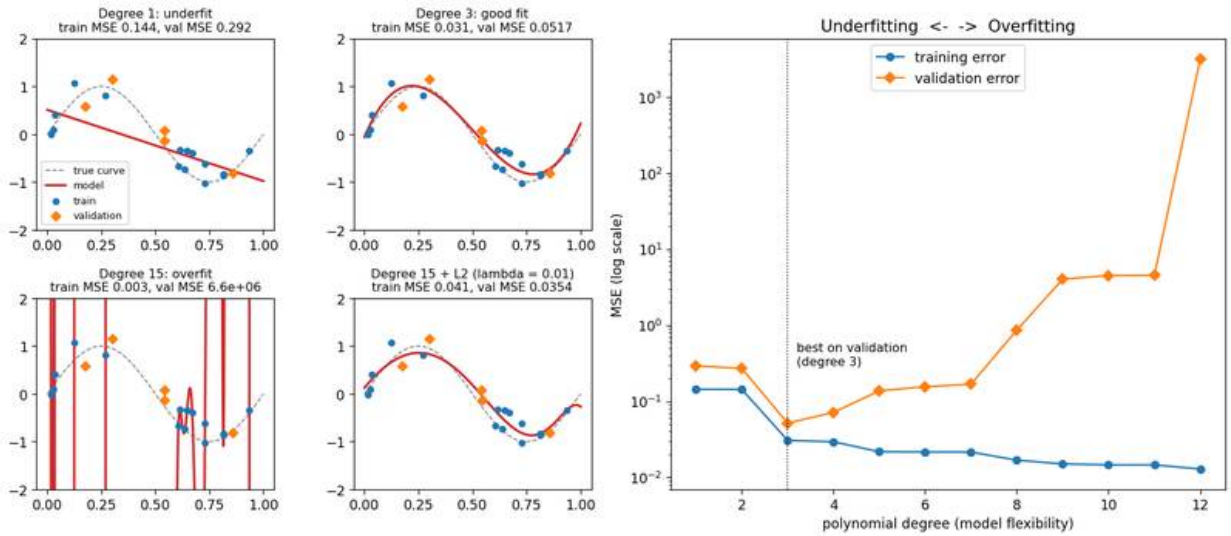


圖 *8* — 欠擬合、良好擬合、過擬合，以及被 *L2* 正則化馴服的過擬合（藍色為訓練點，橘色為驗證點）。右：次數增加時，訓練誤差持續下降，驗證誤差則先降後升；驗證曲線的最低點就是該選的模型。

評估模型

白話版。單一個數字永遠說不完整個故事。選一個符合「犯錯代價」的指標，而且一定要和簡單的基準比較。

迴歸指標

迴歸最常用的指標是 MSE，$\frac{1}{m}\sum_i (\hat{y}^{(i)} - y^{(i)})^2$，這裡不再有只是為了訓練方便的 $\frac{1}{2}$。它的平方根 **RMSE** 和 y 的單位相同，比較好解讀（「平均大約差 3 分」）。**平均絕對誤差**（mean absolute error），$\frac{1}{m}\sum_i |\hat{y}^{(i)} - y^{(i)}|$，對少數大誤差比較不敏感 [5]。

分類指標

分類中，**準確率**（accuracy）是預測正確的比例。它容易理解，也容易騙人。在 digits 資料中，大約 10% 的影像是 9。一個永遠回答「不是 9」的「模型」有 90% 的準確率，卻完全沒用。

混淆矩陣（confusion matrix）計算二元問題的四種結果，「陽性」代表我們在意的那一類：

	預測為陽性	預測為陰性
實際為陽性	真陽性（TP）	偽陰性（FN）
實際為陰性	偽陽性（FP）	真陰性（TN）

由此得到

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \qquad \text{precision} = \frac{TP}{TP + FP}, \qquad \text{recall} = \frac{TP}{TP + FN}.$$

精確率（precision）回答「模型說是陽性時，有多常是對的？」**召回率**（recall）回答「所有真正的陽性中，模型找到了多少？」兩者互相拉扯：把邏輯迴歸的 0.5 決策門檻調低，會找到更多陽性（召回率較高），但也會發出更多假警報（精確率較低）。哪個比較重要取決於每種錯誤的代價：癌症篩檢不該漏掉腫瘤（高召回率），垃圾郵件過濾器則不該把真正的郵件藏起來（高精確率）。**F1 分數**是精確率與召回率的調和平均，把兩者合成一個數字 [5]。

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score

X, y = load_digits(return_X_y=True)
y = (y == 9).astype(int)                  # "is it a 9?": only about 10% positives
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)

lazy = np.zeros_like(y_te)                # always answers "not a 9"
model = LogisticRegression(max_iter=5000).fit(X_tr, y_tr)
pred = model.predict(X_te)

print("lazy accuracy :", round(accuracy_score(y_te, lazy), 3), " recall:",
 ↪  recall_score(y_te, lazy))
print("model accuracy:", round(accuracy_score(y_te, pred), 3),
      " precision:", round(precision_score(y_te, pred), 3),
      " recall:", round(recall_score(y_te, pred), 3))
print(confusion_matrix(y_te, pred))       # rows: true 0/1, columns: predicted 0/1
# lazy accuracy : 0.9   recall: 0.0
# model accuracy: 0.974  precision: 0.87  recall: 0.87
```

偷懶的基準有 90% 的準確率，召回率卻是零。真正的模型準確率 97.4%，聽起來只好一點點，但 0.87 的召回率顯示它確實找到了大部分的 9。一定要回報基準；類別不平衡時，一定要看準確率以外的指標。同樣的指標會在 DIP 的圖樣分類一章再次出現，並更仔細地處理門檻與多類別的問題。

現代觀點

本篇的機制（帶參數的模型、成本、以梯度為基礎的最佳化，以及保留資料的評估）幾十年來沒有改變，下面談到的每個系統也都還在使用它們。改變的是**特徵從哪裡來**、**標籤從哪裡來**，以及**模型有多大**。以下幾篇回顧文章把這些轉變描繪得很好。

機器學習這個領域。 Jordan 與 Mitchell 發表在《Science》的回顧 [7] 以一個問題來界定機器學習：如何打造能透過經驗自動進步的電腦；並把它定位在電腦科學與統計學的交會處、人工智慧與資料科學的核心。他們把機器學習在科學、醫療、製造、教育與金融等領域的快速擴散，歸因於三件事同時到來：新的學習演算法與理論、可取得資料量的爆炸性成長，以及便宜的運算。這個框架是對抗炒作的好解藥：方法的好壞，取決於它得到的資料與經驗。

從特徵工程到表示學習。很長一段時間，傳統模型的成敗主要取決於人工設計的特徵，也就是由專家來做圖 1 中「數有墨跡的欄數」這一步：邊緣直方圖、紋理描述子、關鍵點描述子（見IP 12）。Bengio、Courville 與 Vincent [21] 認為這種依賴是一個弱點，因為正確的表示決定了任務的難易，並主張直接從資料學習表示本身。他們的回顧涵蓋了做到這件事的工具（機率模型、自編碼器、流形學習與深度網路），以及讓表示變好的通用先驗，例如把資料背後各自獨立的解釋因子分離開來。用本篇的語言來說，深度網路仍然是「特徵加上邏輯迴歸」，只是特徵由前面的層產生，而這些層的權重也是用同樣的梯度下降學出來的。這就是ML 02 的主題。

自監督式學習。學習表示需要資料，而標籤是昂貴的部分。Jing 與 Tian 關於自監督式視覺特徵學習的綜述 [22] 把不需要標籤的「前置任務」（pretext task）分成四類：以生成為基礎（例如重建影像或為影像上色）、以上下文為基礎（利用空間或時間結構，例如影像區塊的排列）、以免費語意標籤為基礎（使用自動得到的標籤，例如來自繪圖引擎或傳統方法），以及跨模態（判斷兩個訊號，例如影片與聲音，是否屬於同一事件）。它也確立了至今仍是標準的評估流程：用表示遷移到分類、偵測、分割等下游任務的表現來評判它。Gui 等人較新的綜述 [23] 則把這個領域重新整理為：以上下文為基礎的方法、對比學習（利用負例、自蒸餾或特徵去相關）、以遮罩影像建模為主的生成式方法，以及結合對比與生成目標的混合方法，反映出遮罩自編碼器 [12] 之後轉向遮罩預測的趨勢。對實務工作者，Balestriero 等人的《A Cookbook of Self-Supervised Learning》[24] 把現代方法分成深度度量學習家族、自蒸餾家族、典型相關分析家族與遮罩影像建模，並坦白指出結果有多依賴資料擴增與訓練時使用的小型「投影器」（projector）網路等細節。

基礎模型。 Bommasani 等人的報告 [10] 為由此產生的範式命名：在大規模、廣泛的資料上訓練（通常是自監督式），再調整去做許多下游任務的模型。報告強調了兩個後果。**湧現**（emergence）：像是憑提示中的幾個例子就能遵循指示 [11] 這類能力，是隨著規模出現的，而不是被設計出來的。**同質化**（homogenization）：許多應用如今都由同樣少數幾個基礎模型調整而來，這帶來巨大的槓桿效果，但也代表基礎模型的缺陷會被建立在它之上的一切所繼承。報告也強調，我們對這些模型如何運作、何時會失敗的理解仍非常有限。

基於人類回饋的強化學習。強化學習已經從遊戲走到語言模型訓練的最後階段 [13]、[14]。Casper 等人 [25] 整理了可能出錯的地方。他們依照 RLHF 的三個步驟（收集人類回饋、擬合獎勵模型、最佳化策略），逐一列出每一步的問題：人的回饋帶有雜訊、前後不一致，而且可能被操弄；獎勵模型只是「人們真正想要的東西」的不完美替代；而針對這個替代品用力最佳化的策略，可能會鑽它的漏洞。他們區分了在 RLHF 框架內用更好的方法就能解決的問題，以及需要換一種方法才能克服的根本限制，並主張提高透明度、把 RLHF 與其他安全措施結合，而不是只依賴它。

大規模最佳化。 Bottou、Curtis 與 Nocedal [18] 解釋了為什麼成為機器學習主力的，是樸素的隨機梯度法，而不是傳統數值分析中更精巧的最佳化方法：資料充足時，便宜而帶雜訊的一步勝過昂貴而精確的一步。Adam [19] 等自適應方法則在其上加入每個參數各自的步長。本節提到的每個模型，無論多大，都是用本篇小批次迴圈的後代訓練出來的。

未解問題。儘管規模驚人，這篇入門教學中的幾個問題仍然是最困難的。**泛化**：非常大的模型幾乎完美地擬合訓練資料，卻仍能泛化，這和圖 8 中簡單的過擬合圖像不符，其原因尚未完全被理解。**評估**：當模型用了相當大比例的公開網路資料來訓練，就很難保證測試集沒有出現在訓練資料中，因此誠實評估的基礎，乾淨的訓練／測試分離，變得難以驗證。**目標**：成本函數是模型唯一會最佳化的東西，而 RLHF 顯示出要寫出一個真正反映人們需求的成本有多難。還有**資料**：模型能學到什麼，受限於它看到的資料與標籤，以及其中所有的缺口與偏差。

針對電腦視覺，傳統路線（人工特徵加上簡單分類器）在 DIP 的圖樣分類一章中有完整發展；學

習的路線則延續到ML 02 與〈Deep Learning for Image Processing〉。

重點整理

- 機器學習用一個參數由範例擬合而來的彈性模型，取代手寫規則；當規則難以說清楚、範例卻容易取得時，它最能發揮。
- 各範式的差別在於回饋：監督式（有正確答案；數字用迴歸、類別用分類）、非監督式（沒有答案；分群、降維、異常偵測）、自監督式（答案由資料本身製造），以及強化學習（動作換來獎勵）。KNN 是監督式；k-means 是分群。
- 把模型 $f_{w,b}(x)$ 和成本 $J(w,b)$ 分開：模型把輸入對應到預測值；成本把參數對應到誤差。訓練就是最小化 J。
- 梯度下降重複 $w \leftarrow w - \alpha\, \partial J/\partial w$。在最小值處，**導數**為零。學習率必須小到不會越過頭，又大到能有進展；小批次讓每一步都很便宜。
- 做梯度下降前先縮放特徵，而且只用訓練集的統計量。
- 邏輯迴歸對線性分數套上 sigmoid，並最小化交叉熵；它的決策邊界是 $\mathbf{w}\cdot\mathbf{x}+b=0$，梯度形式和線性迴歸相同。
- 用模型沒訓練過的資料評判它：用驗證集做選擇、在測試集上只回報一次、模型過擬合時加上（L2）正則化，類別不平衡時要看準確率以外的指標。

練習

1. **手算導數**。對單一特徵的 MSE 成本 $J(w,b)=\frac{1}{2m}\sum_i (wx^{(i)}+b-y^{(i)})^2$，推導 $\partial J/\partial w$ 與 $\partial J/\partial b$。接著對三個點 $(1,2),(2,4),(3,6)$，從 $w=0, b=0$ 出發，以 $\alpha=0.1$ 計算一步梯度下降。

   提示

   對每個平方項用連鎖律得到 $2(wx^{(i)}+b-y^{(i)})\cdot x^{(i)}$，2 和 $\frac{1}{2}$ 抵消，就得到梯度下降一節中的公式。在 $w=b=0$ 時誤差是 $-2,-4,-6$。所以 $\partial J/\partial w = (-2\cdot 1-4\cdot 2-6\cdot 3)/3 = -28/3$，$\partial J/\partial b = -12/3 = -4$。一步之後 $w = 0.1\cdot 28/3 \approx 0.933$，$b = 0.4$。

2. **找出發散的門檻**。對 $J(w)=\frac{1}{2m}\sum_i (wx^{(i)}-y^{(i)})^2$（沒有偏差），證明一步梯度下降會把到最佳值 w^* 的距離乘上 $1-\alpha s$，其中 $s=\frac{1}{m}\sum_i (x^{(i)})^2$。$\alpha$ 在什麼範圍內梯度下降會收斂？用特徵縮放一節中的 `gd` 函數檢查你的答案。

   提示

   因為 J 是曲率為 s、最小值在 w^* 的拋物線，$\partial J/\partial w = s\,(w-w^*)$。所以 $w_{\text{new}} - w^* = (1-\alpha s)(w-w^*)$。當 $|1-\alpha s|<1$，也就是 $0<\alpha<2/s$ 時收斂；$\alpha = 1/s$ 時一步就落在最小值上。有多個特徵時，s 換成 $\frac{1}{m}X^\top X$ 的最大特徵值，這就是為什麼一個尺度巨大的特徵會逼得 α 必須非常小。

3. **用迴歸做分類**。用 `make_classification(n_features=1, n_informative=1, n_redundant=0, n_clusters_per_class=1)` 產生一維資料，再加上五個 x 非常大的第 1 類範例。分別擬合 `LinearRegression`（以 0.5 為門檻）與 `LogisticRegression`。比較加入額外點前後兩者的準確率，並解釋差異。

提示

額外的點很容易分類，但平方誤差仍然希望直線經過它們附近。直線的斜率下降，跨過 0.5 的位置往第 1 類移動，一些靠近邊界的普通第 1 類點因此被分錯。分類正確且離得很遠的點，對數損失已經接近 0，所以邏輯迴歸幾乎不受影響。

4. **縮放 digits**。用本篇從頭實作的邏輯迴歸，在 digits 資料上訓練「是 9 或不是 9」，一次用原始像素（0 到 16），一次用除以 16 的像素。使用相同的步數，找出各自可用的最大學習率。縮放改變的是最終準確率、速度，還是兩者都有？

提示

這裡所有像素的單位相同，所以效果比圖 6 溫和，但原始輸入大了 16 倍，梯度的大小和地形的曲率也隨之放大。原始像素的安全學習率大約小 $16^2 = 256$ 倍。學習率都調好時，兩者能達到相近的準確率；縮放過的版本只是比較容易調。

5. **選擇 lambda**。用多項式的例子，固定 15 次，嘗試 $\lambda \in \{10^{-6}, 10^{-4}, 10^{-2}, 1, 100\}$。以對數橫軸畫出訓練與驗證 MSE 對 λ 的變化。哪一端欠擬合、哪一端過擬合？你會選哪個 λ？為什麼不能用測試集來做這個選擇？

提示

很小的 λ 等於沒有正則化（過擬合：訓練誤差低、驗證誤差高）。很大的 λ 把曲線壓成接近常數（欠擬合：兩種誤差都高）。選驗證誤差最低的 λ。如果測試集參與了選擇，它就不再是新資料上效能的不偏估計。

6. **哪一種範式？**把每個問題歸類為監督式（迴歸或分類）、非監督式、自監督式或強化學習，並說出目標值或獎勵：(a) 用附有實際結果的使用紀錄，估計手機剩餘的電池時間；(b) 把一百萬張沒有標籤的商品照片分組以便瀏覽；(c) 藉由預測每張照片缺少的四分之一來預訓練影像模型；(d) 教機械手臂堆積木，只有在積木堆穩時才給分數；(e) 在沒有任何故障標註的情況下，標出工廠感測器資料流中的異常讀數。

提示

(a) 監督式迴歸；目標值是剩餘時數。(b) 非監督式分群。(c) 自監督式；目標值是被藏起來的像素，由資料本身產生。(d) 強化學習；獎勵是積木是否堆穩，要到最後才給。(e) 非監督式異常偵測。

接下來的機器學習／深度學習路線

若數學符號還不熟悉，先讀線性代數、機率與資訊理論 和數值計算，再進入類神經網路。

後續從影像濾波與 CNN 連接循環／遞迴網路、Transformer 和表示學習。接著由因子模型與自編碼器進入 VAE、GAN、正規化流、擴散／ flow matching、自迴歸與可重用的基礎模型，最後用評估與部署 完成一個可靠實驗。統計決策與推論的觀點見 圖形識別，具體視覺任務則見 視覺深度學習。

參考文獻

1. Shyandram, “Introduction to Machine Learning,” Medium post, 2024; updated 2026. link
2. A. L. Samuel, “Some studies in machine learning using the game of checkers,” *IBM Journal of Research and Development*, vol. 3, no. 3, pp. 210–229, 1959. doi
3. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016. book site
4. A. Ng, E. Shyu, A. Bagul and G. Ladwig, “Machine Learning Specialization,” DeepLearning.AI and Stanford Online, Coursera. course page
5. F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. link
6. E. Alpaydin and C. Kaynak, “Optical Recognition of Handwritten Digits,” UCI Machine Learning Repository, 1998. doi
7. M. I. Jordan and T. M. Mitchell, “Machine learning: Trends, perspectives, and prospects,” *Science*, vol. 349, no. 6245, pp. 255–260, 2015. doi
8. R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018. book page
9. Y. LeCun and I. Misra, “Self-supervised learning: The dark matter of intelligence,” Meta AI blog, 2021. link
10. R. Bommasani, D. A. Hudson, E. Adeli et al., “On the opportunities and risks of foundation models,” arXiv:2108.07258, 2021. arXiv
11. T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan et al., “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems* (*NeurIPS*), vol. 33, 2020. link
12. K. He, X. Chen, S. Xie, Y. Li, P. Dollár and R. Girshick, “Masked autoencoders are scalable vision learners,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), pp. 15979–15988, 2022. arXiv
13. L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright et al., “Training language models to follow instructions with human feedback,” in *Advances in Neural Information Processing Systems* (*NeurIPS*), vol. 35, 2022. link
14. DeepSeek-AI, “DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning,” *Nature*, vol. 645, pp. 633–638, 2025. arXiv
15. A. K. Jain, “Data clustering: 50 years beyond K-means,” *Pattern Recognition Letters*, vol. 31, no. 8, pp. 651–666, 2010. doi
16. I. T. Jolliffe and J. Cadima, “Principal component analysis: A review and recent developments,” *Philosophical Transactions of the Royal Society A*, vol. 374, no. 2065, 20150202, 2016. doi
17. V. Chandola, A. Banerjee and V. Kumar, “Anomaly detection: A survey,” *ACM Computing Surveys*, vol. 41, no. 3, 2009. doi
18. L. Bottou, F. E. Curtis and J. Nocedal, “Optimization methods for large-scale machine learning,” *SIAM Review*, vol. 60, no. 2, pp. 223–311, 2018. doi
19. D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. International Conference on Learning Representations* (*ICLR*), 2015. arXiv
20. A. E. Hoerl and R. W. Kennard, “Ridge regression: Biased estimation for nonorthogonal problems,” *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970. doi
21. Y. Bengio, A. Courville and P. Vincent, “Representation learning: A review and new perspectives,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 8, pp. 1798–

1828, 2013. doi
22. L. Jing and Y. Tian, "Self-supervised visual feature learning with deep neural networks: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 11, pp. 4037–4058, 2021. doi
23. J. Gui, T. Chen, J. Zhang, Q. Cao, Z. Sun, H. Luo and D. Tao, "A survey on self-supervised learning: Algorithms, applications, and future trends," arXiv:2301.05712, 2023. arXiv
24. R. Balestriero, M. Ibrahim, V. Sobal, A. Morcos, S. Shekhar et al., "A cookbook of self-supervised learning," arXiv:2304.12210, 2023. arXiv
25. S. Casper, X. Davies, C. Shi et al., "Open problems and fundamental limitations of reinforcement learning from human feedback," *Transactions on Machine Learning Research*, 2023. TMLR ·arXiv

ML 01A・學習所需的線性代數

向量、矩陣與張量、範數、特徵分解、奇異值分解、偽反矩陣、跡與行列式，並一步步推導主成分分析，附經過測試的 NumPy 程式碼。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-01a-linear-algebra/

你將學到

- 純量、向量、矩陣與張量如何表示影像、批次與學習到的轉換，以及如何把它們的形狀弄清楚。
- 矩陣乘法的四種讀法，以及單位矩陣、反矩陣、生成空間、秩與零空間對線性方程組的意義。
- 範數如何量測大小、特殊矩陣（對角、對稱、正交）如何簡化計算，以及內積如何定義相似度。
- 特徵分解與奇異值分解（SVD）揭露了矩陣的哪些性質，以及為什麼 SVD 給出最佳的低秩近似。
- Moore–Penrose 偽反矩陣如何解最小平方與欠定問題，以及條件數和嶺正則化為什麼重要。
- 跡與行列式的用法，以及如何只靠這些工具，從頭推導主成分分析（PCA）。

先看全貌

神經網路裡幾乎每個運算，都是「線性映射」接上一個簡單的非線性函數。全連接層是乘上權重矩陣；卷積是有結構的矩陣乘法；注意力機制先計算向量之間的內積，再用一個權重矩陣把向量混合起來。線性代數就是描述這一切的語言。它告訴我們一個映射保留了什麼、丟掉了什麼、拉長了什麼，以及怎樣計算才不會損失精確度。

> **用最簡單的話說**
>
> 向量是一串測量值，例如一個人的身高、體重和年齡。矩陣像一台機器：把一串數字放進去，它會混合出一串新的數字。有的機器只是把這串數字「轉個方向」（旋轉），有的把某些部分放大、某些部分縮小（拉伸），有的則把某些部分壓扁，資訊從此就找不回來。線性代數就是這些機器的說明書：它告訴你每台機器朝哪個方向拉、拉多少，以及能不能把動作復原。

本篇對應 Goodfellow、Bengio 與 Courville 所著《Deep Learning》的「應用數學」部分 [1]，以及《Mathematics for Machine Learning》的線性代數章節 [2]。想多做練習，Strang 的教科書 [3] 是最平易近人的完整課程；Golub 與 Van Loan [4]、Trefethen 與 Bau [5] 則是「如何在電腦上可靠地完成這些計算」的標準參考書。如果你還不熟悉矩陣符號，建議在ML 02 之前先讀本篇。姊妹篇是ML 01B・機率與資訊理論 與ML 01C・數值計算。

本篇有一條主線。我們從物件與形狀開始，接著談乘法與解方程式，再用範數量測大小。特徵分解與 SVD 說明一個矩陣的「個性」；偽反矩陣、跡與行列式則是建立在它們之上的工具。最後，我們用上述每一項工具推導 PCA——這個經典的資料壓縮方法——推導方式與 [1] 相同。

純量、向量、矩陣與張量

白話版。機器學習裡的每個數字都住在一個有命名軸的陣列裡。做任何乘法之前，先說清楚每個軸代表什麼。

純量（scalar）是單一數字，寫成 $s \in \mathbb{R}$。**向量**（vector）是 n 個數字的有序清單：$x \in \mathbb{R}^n$，分量為 $x_1, \dots, x_n$；依慣例，向量是一個行向量（直的）。**矩陣**（matrix）是二維的數字格子：$A \in \mathbb{R}^{m \times n}$ 有 m 列（row）與 n 行（column），A_{ij} 是第 i 列、第 j 行的元素。**張量**（tensor）則是任意多個軸的陣列。

這些物件可以直接對應到資料：

- 高 H、寬 W 的灰階影像是 $\mathbb{R}^{H \times W}$ 的矩陣；攤平後就是向量 $x \in \mathbb{R}^{HW}$（見影像即陣列）。
- n 筆樣本、每筆 d 個特徵，每列放一筆樣本，就得到**設計矩陣**（design matrix）$X \in \mathbb{R}^{n \times d}$。
- 權重 $W \in \mathbb{R}^{d \times m}$、偏差 $b \in \mathbb{R}^m$ 的全連接層一次處理整個批次：$Y = XW + \mathbf{1}b^\top \in \mathbb{R}^{n \times m}$，其中 $\mathbf{1} \in \mathbb{R}^n$ 是全為 1 的向量。
- PyTorch 的影像批次通常是形狀 $B \times C \times H \times W$（批次、通道、高、寬）的四維張量；Transformer 的 token 嵌入批次通常是 $B \times N \times D$（批次、序列長度、嵌入維度）。

轉置（transpose）$A^\top$ 把列與行對調：$(A^\top)_{ij} = A_{ji}$。它滿足 $(A^\top)^\top = A$ 與 $(AB)^\top = B^\top A^\top$——注意順序反過來了。

廣播

寫 $\mathbf{1}b^\top$ 在數學上很誠實，但很繁瑣，所以程式庫允許你直接把向量加到矩陣的每一列。這叫**廣播**（broadcasting）：兩個陣列形狀不同時，長度為 1 的軸（或缺少的前導軸）會被延展成相同大小。廣播很方便，也很危險，因為形狀錯了程式往往照樣執行。最常見的錯誤是把形狀 $(n, 1)$ 的行向量和形狀 $(n,)$ 的一維陣列相減，結果悄悄變成 $n \times n$ 的矩陣，而不是 n 個殘差。

```
import numpy as np

rng = np.random.default_rng(0)
X = rng.normal(size=(32, 64))      # batch of 32 examples, 64 features
W = rng.normal(size=(64, 10))      # linear map 64 -> 10
b = rng.normal(size=10)            # one bias per output
Y = X @ W + b                      # b broadcasts over the 32 rows
assert Y.shape == (32, 10)

# A classic broadcasting bug: (32,) - (32, 1) silently becomes (32, 32)
y_true = rng.normal(size=32)
y_pred = rng.normal(size=(32, 1))
print("buggy residual shape:", (y_true - y_pred).shape)
print("fixed residual shape:", (y_true - y_pred[:, 0]).shape)

# Four equivalent views of the same matrix product
A, B = rng.normal(size=(3, 4)), rng.normal(size=(4, 2))
entry = np.array([[A[i] @ B[:, j] for j in range(2)] for i in range(3)])
cols = np.stack([A @ B[:, j] for j in range(2)], axis=1)
```

```
outer = sum(np.outer(A[:, k], B[k]) for k in range(4))
assert np.allclose(A @ B, entry) and np.allclose(A @ B, cols) and np.allclose(A @ B, outer)
print("all views agree")
```

錯誤的那一行印出 `(32, 32)`，修正後是 `(32,)`。對 32×32 陣列取平均仍然只得到一個數字，所以用這種方式算出的損失函數可以順利訓練，卻在最佳化錯誤的目標。凡是重要的程式碼，都替形狀加上 `assert`。

矩陣與向量的乘法

白話版。矩陣乘法把兩台線性機器串成一台。輸出的每個數字，是第一個矩陣的某一列與第二個矩陣的某一行的內積。

對 $A \in \mathbb{R}^{m\times n}$ 與 $B \in \mathbb{R}^{n\times p}$，**矩陣乘積** $C = AB \in \mathbb{R}^{m\times p}$ 為

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}.$$

其中 i 是 A（也是 C）的列索引，j 是 B（也是 C）的行索引，k 走遍共同的內部維度 n。內部維度必須相同——遇到形狀錯誤時，這是第一個要檢查的地方。

同一個乘積有四種讀法，各有用處：

1. **元素觀點**。C_{ij} 是 A 的第 i 列與 B 的第 j 行的內積。注意力分數 $QK^\top$ 就是這樣讀：每個 query 和每個 key 比較。
2. **行觀點**。C 的第 j 行等於 A 乘上 B 的第 j 行；C 的每一行都是 A 各行的線性組合。
3. **列觀點**。C 的第 i 列等於 A 的第 i 列乘上 B。在 $Y = XW$ 中，每筆輸出樣本就是它的輸入列通過同一組權重。
4. **外積觀點**。$C = \sum_k a_{:k} b_{k:}$，是 n 個秩一矩陣的總和，每個由 A 的第 k 行與 B 的第 k 列構成。這個觀點會在本篇後面解釋低秩近似與 LoRA。

上面的程式碼驗證了四種讀法結果相同。

矩陣乘法滿足**結合律** $(AB)C = A(BC)$ 與**分配律** $A(B + C) = AB + AC$，但**不滿足交換律**：一般而言 $AB \neq BA$，而且其中一個乘積常常根本沒有定義。**逐元素**（Hadamard）乘積 $A \odot B$ 則是把對應元素相乘，需要形狀相同。在 NumPy 與 PyTorch 中，`A @ B` 是矩陣乘法，`A * B` 是逐元素乘法。兩個向量 $x, y \in \mathbb{R}^n$ 的**內積**是特例 $x^\top y = \sum_i x_i y_i$，而且對稱：$x^\top y = y^\top x$。

單位矩陣、反矩陣與線性方程組

m 條方程式、n 個未知數的線性方程組可以簡寫成

$$Ax = b, \qquad A \in \mathbb{R}^{m\times n},\ x \in \mathbb{R}^n,\ b \in \mathbb{R}^m.$$

A 的每一列是一條方程式的係數，x 是未知數，b 是等號右邊的值。

單位矩陣 I_n 對角線為 1、其餘為 0，不改變任何向量：$I_n x = x$。若方陣 A 存在矩陣 A^{-1} 使 $A^{-1}A = AA^{-1} = I$，就稱 A **可逆**，此時方程組恰有一解 $x = A^{-1}b$。

這個公式很適合推導，卻不適合計算。直接算出 A^{-1}，比用 LU、Cholesky 或 QR 分解直接求解更費時、也更不精確 [4], [5]。在 NumPy 中請寫 `np.linalg.solve(A, b)`，而不是 `np.linalg.inv(A) @ b`。本篇中的 A^{-1} 是思考工具。

線性相依、生成空間與秩

白話版。秩計算一個矩陣真正能產生多少個不同方向。重複或多餘的行不會增加新東西。

向量 $v^{(1)}, \dots, v^{(k)}$ 的**線性組合**是 $\sum_i c_i v^{(i)}$，c_i 為純量。一組向量的**生成空間**（span）是它們所有線性組合構成的集合。由於 $Ax = \sum_j x_j a_{:j}$，向量 Ax 是 A 各行 $a_{:j}$ 的線性組合。因此 $Ax = b$ 有解，若且唯若 b 落在各行的生成空間裡，這個空間稱為 A 的**行空間**（column space，或值域）。

若一組向量中沒有任何一個是其他向量的線性組合——等價地說，$\sum_i c_i v^{(i)} = 0$ 只在所有 $c_i = 0$ 時成立——就稱它們**線性獨立**。A 的**秩**（rank）是線性獨立的行數，而它永遠等於線性獨立的列數。對 $A \in \mathbb{R}^{m \times n}$，$\operatorname{rank}(A) \le \min(m, n)$；等號成立時稱 A **滿秩**。

A 的**零空間**（null space）是所有滿足 $Az = 0$ 的向量 z，也就是被 A 摧毀的輸入方向。**秩–零度定理**說

$$\operatorname{rank}(A) + \dim \operatorname{null}(A) = n,$$

其中 n 是行數，dim 是零空間中獨立方向的個數。這一條等式就解釋了 $Ax = b$ 的三種情況：

形狀與秩	會發生什麼	典型的機器學習情境
方陣、滿秩	恰有一解	良好設定的座標轉換
高瘦（$m > n$）、行滿秩	通常沒有精確解；改求最小平方解	樣本數多於特徵數的迴歸
矮胖（$m < n$）或秩不足	若有解則有無限多解	參數多於限制條件，例如深度網路

對方陣來說，「可逆」「滿秩」「各行線性獨立」「零空間只有零向量」是同一件事；不滿足的方陣稱為**奇異**（singular）矩陣。

範數：量測大小

白話版。範數把一個向量變成一個非負數，表示它有多大。不同的範數對「大」有不同看法。

範數是滿足以下條件的函數 f：$f(x) = 0 \Rightarrow x = 0$；$f(x + y) \le f(x) + f(y)$（三角不等式）；對任何純量 α，$f(\alpha x) = |\alpha| f(x)$。最常見的是 L^p 範數：

$$\|x\|_p = \Big(\sum_i |x_i|^p \Big)^{1/p}, \qquad p \ge 1.$$

- $\|x\|_2$ 是**歐幾里得**範數，也就是一般的長度，常簡寫為 $\|x\|$。它的平方 $\|x\|_2^2 = x^\top x$ 更好微分，出現在均方誤差損失與權重衰減中。

- $\|x\|_1 = \sum_i |x_i|$ 在每個地方——包括靠近零的地方——都以相同速率成長。當「恰好為零」和「稍微不為零」的差別很重要時它特別有用，這也是 L^1 懲罰會鼓勵稀疏解的原因。
- $\|x\|_\infty = \max_i |x_i|$ 是**最大範數**，即最大元素的大小。對抗強健性的研究常用它限制擾動。
- 非零元素的個數有時被稱為「L^0 範數」，但它不是範數：把 x 乘以 2 並不會改變它。

內積透過夾角與歐幾里得範數相連：$x^\top y = \|x\|_2 \|y\|_2 \cos\theta$，$\theta$ 是 x 與 y 的夾角。**餘弦相似度**

$$\cos(x, y) = \frac{x^\top y}{\|x\|_2 \|y\|_2}$$

只對非零向量有定義，並且忽略尺度。這對某些嵌入很有用（對比學習正因如此會把特徵正規化），但當大小本身有意義時——例如影像亮度——就不適用。

矩陣範數

矩陣最常用兩種範數。**Frobenius 範數**把矩陣當成一條長向量：

$$\|A\|_F = \sqrt{\sum_{i,j} A_{ij}^2}.$$

譜範數（spectral norm，即算子 2-範數）量測 A 能把單位向量拉長的最大倍數：

$$\|A\|_2 = \max_{\|x\|_2 = 1} \|Ax\|_2 = \sigma_{\max}(A),$$

其中 $\sigma_{\max}(A)$ 是最大奇異值，定義見下方 SVD 一節。若以全部奇異值 $\sigma_1 \ge \sigma_2 \ge \dots$ 表示，則 $\|A\|_F^2 = \sum_i \sigma_i^2$，而 $\|A\|_2 = \sigma_1$。所以許多小奇異值可以讓 Frobenius 範數變大，卻不改變最大放大倍數。譜範數限制了一層網路最多能把輸入擾動放大多少，因此出現在強健性與 GAN 穩定性的研究中 [17]。

```
import numpy as np

x = np.array([3.0, -4.0, 0.0, 1.0])
for p in [1, 2, np.inf]:
    print(f"L{p} norm:", np.linalg.norm(x, p))
print("L0 'norm' (count of nonzeros):", np.count_nonzero(x))

A = np.array([[3.0, 0.0], [4.0, 5.0]])
s = np.linalg.svd(A, compute_uv=False)
print("singular values:", s.round(4))
print("Frobenius:", np.linalg.norm(A, "fro"), "=", np.sqrt((s**2).sum()))
print("spectral :", np.linalg.norm(A, 2), "=", s[0])
assert np.isclose(np.linalg.norm(A, "fro")**2, np.trace(A.T @ A))
```

對 $x = (3, -4, 0, 1)$，程式印出 $L^1 = 8$、$L^2 = \sqrt{26} \approx 5.10$、$L^\infty = 4$，以及三個非零元素。這個 2×2 矩陣的奇異值是 $3\sqrt{5} \approx 6.71$ 與 $\sqrt{5} \approx 2.24$，因此 $\|A\|_F = \sqrt{50} \approx 7.07$、$\|A\|_2 \approx 6.71$。

特殊的矩陣與向量

白話版。有些矩陣比其他矩陣好處理得多。學會辨認它們，可以省下計算量，也能理解許多設計選擇。

- **對角矩陣** $D = \operatorname{diag}(v)$ 只有主對角線上有非零元素。乘上它等於把每個座標分別縮放：$\operatorname{diag}(v)x = v \odot x$。當每個 $v_i \neq 0$ 時，它的反矩陣是 $\operatorname{diag}(1/v_1, \dots, 1/v_n)$。正規化層中逐通道的縮放，其實就是對角矩陣。長方形的對角矩陣也存在，會出現在 SVD 中。
- **對稱矩陣**滿足 $A = A^\top$。共變異數矩陣、Gram 矩陣 $X^\top X$、平滑函數的 Hessian 與距離矩陣都是對稱的。
- **單位向量**滿足 $\|x\|_2 = 1$。若 $x^\top y = 0$，兩向量**正交**；若它們同時也是單位向量，就是**單範正交**（orthonormal）。
- **正交矩陣** Q 是各列、各行皆單範正交的方陣，因此 $Q^\top Q = QQ^\top = I$ 且 $Q^{-1} = Q^\top$——反矩陣不花成本。它保持長度與角度不變，$\|Qx\|_2 = \|x\|_2$，幾何上是旋轉、鏡射或兩者的組合。

若 $Q \in \mathbb{R}^{n \times k}$ 有 $k < n$ 個單範正交的行，則 $Q^\top Q = I_k$，但 $QQ^\top \neq I_n$。矩陣 $P = QQ^\top$ 是投影到 Q 各行生成空間的**正交投影**。投影 Px 是該子空間中離 x 最近的點，殘差 $x - Px$ 與 Q 的每一行都垂直。投影兩次不會再改變任何東西：$P^2 = P$。下面的 PCA 正是在尋找最好的這種 Q。

特徵分解

白話版。大多數向量被矩陣作用後會改變方向。特徵向量是那些只被拉長或縮短的特殊方向，特徵值則告訴你拉長或縮短了多少。

方陣 $A \in \mathbb{R}^{n \times n}$ 的**特徵向量**是滿足

$$Av = \lambda v$$

的非零向量 v，純量 λ 是對應的**特徵值**。若 v 是特徵向量，任何 $c \neq 0$ 的 cv 也是，所以通常把特徵向量正規化成單位長度。

假設 A 有 n 個線性獨立的特徵向量 $v^{(1)}, \dots, v^{(n)}$，特徵值為 $\lambda_1, \dots, \lambda_n$。把特徵向量當成 V 的各行，把特徵值放進 $\Lambda = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$，就有 $AV = V\Lambda$，因此

$$A = V \Lambda V^{-1}.$$

這就是**特徵分解**。它的意思是：先換到特徵向量座標（V^{-1}），每個座標各自縮放（Λ），再換回來（V）。不是每個矩陣都有特徵分解。剪切矩陣 $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ 只有一個獨立的特徵向量；旋轉 90° 的矩陣特徵值是 $\pm i$，是複數，因為沒有任何實數方向能保持不轉。

對稱的情形

對稱矩陣是好相處的情形，也是機器學習最常遇到的。**譜定理**指出，每個實對稱矩陣都可以寫成

$$A = Q \Lambda Q^\top,$$

其中 Q 是正交矩陣（各行是單範正交的特徵向量），Λ 是實數對角矩陣 [3], [4]。因為 $Q^{-1} = Q^{\top}$，完全不需要求反矩陣。幾何上，A 沿著方向 $q^{(i)}$（Q 的第 i 行）把空間拉伸 λ_i 倍。除了特徵值的排列順序，以及重複特徵值內部基底的選擇之外，這個分解是唯一的；慣例上把特徵值由大到小排列。

對對稱的 A，把**二次型** $f(x) = x^{\top}Ax$ 限制在單位向量上，其值被最大與最小特徵值夾住：

$$\lambda_{\min} \le x^{\top}Ax \le \lambda_{\max} \qquad \text{當} \|x\|_2 = 1,$$

最大值出現在最大特徵值的特徵向量，最小值出現在最小特徵值的特徵向量。比值 $x^{\top}Ax / x^{\top}x$ 稱為 **Rayleigh 商**。這個事實是後面 PCA 推導的引擎。

正定矩陣

特徵值的正負號可以為對稱矩陣分類：

名稱	特徵值	等價條件
正定（PD）	全部 $\lambda_i > 0$	對所有 $x \neq 0$，$x^{\top}Ax > 0$
半正定（PSD）	全部 $\lambda_i \ge 0$	對所有 x，$x^{\top}Ax \ge 0$
負定（半負定）	全部 $\lambda_i < 0$（或 ≤ 0）	不等式反向
不定	正負都有	$x^{\top}Ax$ 正負皆可能

每個共變異數矩陣與每個 Gram 矩陣 $X^{\top}X$ 都是半正定的，因為 $v^{\top}X^{\top}Xv = \|Xv\|_2^2 \ge 0$。高斯密度需要正定的共變異數矩陣才能求反。損失函數的 Hessian 在嚴格局部最小值處是正定的，在鞍點處則是不定的；ML 01C 會用這一點解釋最佳化為什麼可能很慢。正定矩陣有 **Cholesky 分解** $A = LL^{\top}$（L 為下三角矩陣），這是用它解方程組、以及從高斯分佈抽樣最快又穩定的方式 [4]。

```
import numpy as np

rng = np.random.default_rng(1)
M = rng.normal(size=(4, 4))
S = M @ M.T + 0.1 * np.eye(4)        # symmetric positive definite
lam, Q = np.linalg.eigh(S)           # ascending eigenvalues, orthonormal Q
assert np.allclose(Q @ np.diag(lam) @ Q.T, S)
assert np.allclose(Q.T @ Q, np.eye(4))
print("eigenvalues:", lam.round(3), "all positive:", bool((lam > 0).all()))

# The quadratic form x^T S x is bounded by the extreme eigenvalues
x = rng.normal(size=4); x /= np.linalg.norm(x)
print("Rayleigh quotient in range:", lam[0] <= x @ S @ x <= lam[-1])

# A non-symmetric matrix can have complex eigenvalues
R = np.array([[0.0, -1.0], [1.0, 0.0]])  # 90-degree rotation
print("rotation eigenvalues:", np.linalg.eigvals(R))
```

對稱矩陣請用 `np.linalg.eigh`：它比較快，回傳由小到大排列的實數特徵值與單範正交的特徵向量。`np.linalg.eig` 處理一般情形，可能回傳複數，如旋轉矩陣所示。

奇異值分解

白話版。任何形狀的矩陣，都依序做三件事：旋轉、沿著互相垂直的軸拉伸、再旋轉一次。SVD 告訴你這兩次旋轉與各個拉伸倍數。

特徵分解需要方陣，而且不一定存在。**奇異值分解**（singular value decomposition, SVD）則對每個實矩陣 $A \in \mathbb{R}^{m\times n}$ 都存在：

$$A = U\Sigma V^{\top}.$$

- $U \in \mathbb{R}^{m\times m}$ 是正交矩陣，各行是**左奇異向量**，屬於輸出空間的方向。
- $\Sigma \in \mathbb{R}^{m\times n}$ 是長方形對角矩陣，對角元素 $\sigma_1 \ge \sigma_2 \ge \dots \ge 0$ 是**奇異值**。
- $V \in \mathbb{R}^{n\times n}$ 是正交矩陣，各行是**右奇異向量**，屬於輸入空間的方向。

由右往左讀，$Ax = U\Sigma V^{\top}x$ 先把 x 表示成 V 座標（旋轉或鏡射），再把第 i 個座標乘上 σ_i，最後沿著 U 的方向擺放。圖 1 以 2×2 矩陣示範：單位圓變成一個橢圓，半軸長分別是 σ_1 與 σ_2。

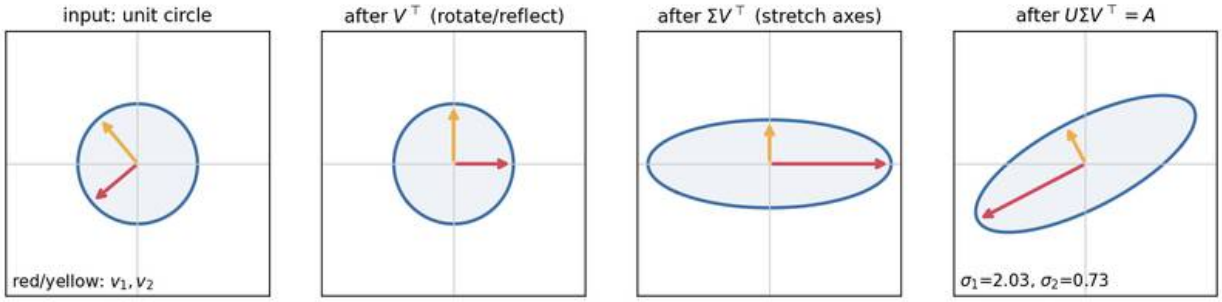


圖 *1 — SVD* 就是「旋轉、拉伸、旋轉」。右奇異向量 *v1*、*v2*（紅、黃）先被 V 的轉置轉到座標軸上，再依奇異值拉伸，最後由 U 旋轉。最終的橢圓就是單位圓經過 A 映射後的像。

SVD 與特徵分解關係密切。由於 $A^{\top}A = V\Sigma^{\top}\Sigma V^{\top}$ 且 $AA^{\top} = U\Sigma\Sigma^{\top}U^{\top}$，右奇異向量是 $A^{\top}A$ 的特徵向量，左奇異向量是 $AA^{\top}$ 的特徵向量，而 $\sigma_i = \sqrt{\lambda_i(A^{\top}A)}$。對於對稱半正定矩陣，SVD 與特徵分解一致。

非零奇異值的個數等於秩 r。只保留它們，就得到**精簡**（thin）SVD：$A = U_r\Sigma_r V_r^{\top} = \sum_{i=1}^{r}\sigma_i u^{(i)}v^{(i)\top}$，也就是 r 個秩一矩陣的總和——又是矩陣乘法的外積觀點。

最佳低秩近似

把這個總和截斷在第 k 項，得到 $A_k = \sum_{i=1}^{k}\sigma_i u^{(i)}v^{(i)\top}$。**Eckart–Young 定理** [6] 指出，A_k 是 A 在 Frobenius 範數下的最佳秩 k 近似：

$$\min_{\operatorname{rank}(B)\le k}\|A-B\|_F^2 = \|A-A_k\|_F^2 = \sum_{i>k}\sigma_i^2.$$

同一個 A_k 在譜範數下也是最佳的，誤差為 $\|A-A_k\|_2 = \sigma_{k+1}$ [4]。換句話說：丟掉最小的那些奇異值，損失的能量剛好就是丟掉的部分。儲存 A_k 只需要 $k(m+n+1)$ 個數字，而不是 mn 個。

圖 2 把這套方法用在一張 256 × 256 的灰階照片上。自然影像的奇異值衰減得很快，所以秩 50 的近似已經保留了主要結構。低秩時的條紋狀瑕疵，來自每個秩一項都是一個行樣式與一個列樣式的外積。

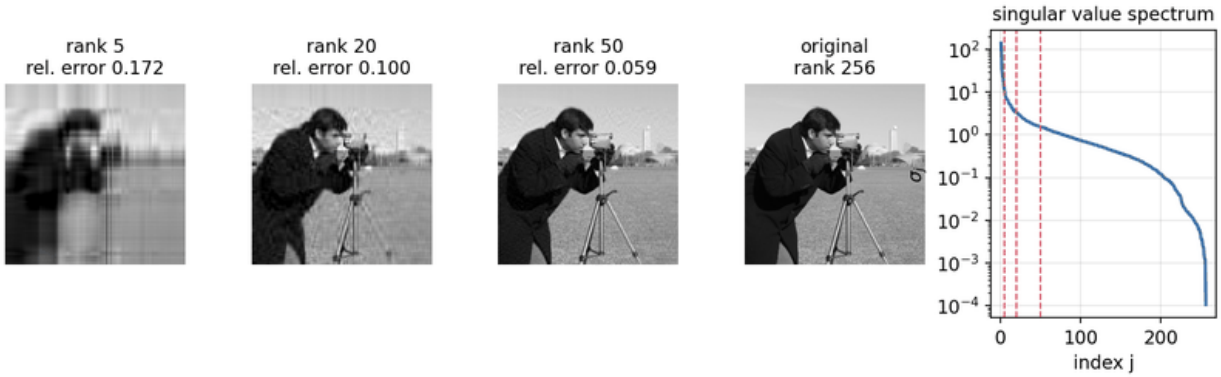


圖 *2 — scikit-image* 中 *256×256*「*camera*」影像的秩 *k* 近似，標題標示相對 *Frobenius* 誤差。右圖：對數尺度下的奇異值，虛線標出 $k = 5$、20、50。每個近似的誤差由該虛線右邊的奇異值決定。

```
import numpy as np

rng = np.random.default_rng(0)
A = rng.normal(size=(12, 8))
U, s, Vt = np.linalg.svd(A, full_matrices=False)
assert np.allclose(U @ np.diag(s) @ Vt, A)
k = 3
Ak = (U[:, :k] * s[:k]) @ Vt[:k]
err_F = np.linalg.norm(A - Ak, "fro")**2
err_2 = np.linalg.norm(A - Ak, 2)
assert np.isclose(err_F, (s[k:]**2).sum()) and np.isclose(err_2, s[k])
print("rank-k squared Frobenius error:", round(err_F, 3))
print("rank-k spectral error = sigma_{k+1}:", round(err_2, 3), round(s[k], 3))
# Eigenvalues of A^T A are squared singular values of A
assert np.allclose(np.sort(np.linalg.eigvalsh(A.T @ A))[::-1], s**2)
print("storage: full", A.size, "numbers; rank-3 factors", k * (12 + 8 + 1))
```

`full_matrices=False` 回傳精簡 SVD，這幾乎總是你要的。`(U[:, :k] * s[:k])` 用廣播縮放 U 的各行，不必建立對角矩陣。兩條 Eckart–Young 等式都在機器精確度內成立。

一個提醒：奇異值最大的方向承載最多能量，卻不一定承載最有用的資訊。對分類而言，變異數小的方向可能正是把類別分開的那個方向。PR 05 正是為此比較了 PCA 與判別式方法。

Moore–Penrose 偽反矩陣

白話版。矩陣沒有反矩陣時，偽反矩陣給出次佳的答案：盡可能貼近資料的解，而在同樣好的解之中，選最小的那一個。

只有滿秩方陣才有反矩陣。對任意 $A \in \mathbb{R}^{m\times n}$，**Moore–Penrose 偽反矩陣**（pseudo-inverse）$A^{+} \in \mathbb{R}^{n\times m}$ 是唯一滿足以下四個 Penrose 條件的矩陣 [7]：

$$AA^{+}A = A, \quad A^{+}AA^{+} = A^{+}, \quad (AA^{+})^{\top} = AA^{+}, \quad (A^{+}A)^{\top} = A^{+}A.$$

用 SVD 計算最容易。若 $A = U\Sigma V^{\top}$，則

$$A^{+} = V\Sigma^{+}U^{\top},$$

其中 $\Sigma^{+} \in \mathbb{R}^{n\times m}$ 是把 Σ 轉置、並把每個非零奇異值 σ_i 換成 $1/\sigma_i$（零維持為零）。Goodfellow 等人給出等價的極限形式 $A^{+} = \lim_{\alpha\searrow 0}(A^{\top}A + \alpha I)^{-1}A^{\top}$ [1]，這已經暗示了嶺迴歸。

在秩的表格的每一種情況下，偽反矩陣都給出正確答案：

- **高瘦、行滿秩**（$m > n$）。$Ax = b$ 通常沒有精確解。此時 $x = A^{+}b = (A^{\top}A)^{-1}A^{\top}b$ 是**最小平方**解：它使 $\|Ax - b\|_2^2$ 最小。
- **矮胖、列滿秩**（$m < n$）。解有無限多個。此時 $x = A^{+}b = A^{\top}(AA^{\top})^{-1}b$ 是 $\|x\|_2$ 最小的解，稱為**最小範數**解。其他任何解都多加了一個零空間向量，只會讓範數變大。
- **秩不足**。$A^{+}b$ 同時兼具兩者：在所有最小平方解之中，範數最小的那一個。

最小範數性質是通往一個現代問題的小窗口。對欠定的最小平方問題從零開始做梯度下降，會收斂到最小範數解，因為每個梯度 $A^{\top}(Ax - b)$ 都落在 A 的列空間中。過參數化的網路不是線性的，但思考「在眾多完美擬合中，訓練演算法會選哪一個」時，這個簡單情形通常是起點。

最小平方、正規方程式與條件數

對迴歸設計矩陣 $X \in \mathbb{R}^{n\times d}$ 與目標 $y \in \mathbb{R}^{n}$，最小平方法最小化 $\|Xw - y\|_2^2$。令梯度為零，得到**正規方程式**

$$X^{\top}Xw = X^{\top}y.$$

它們是對的，但在數值上有風險。線性問題的敏感度用**條件數** $\kappa_2(A) = \sigma_{\max}/\sigma_{\min}$ 量測。粗略地說，資料中的相對誤差在解中最多可能被放大 κ 倍 [5]。形成 $X^{\top}X$ 會把條件數平方：$\kappa(X^{\top}X) = \kappa(X)^2$，於是輕微病態的問題變成嚴重病態。程式庫的最小平方求解器（`np.linalg.lstsq`、`torch.linalg.lstsq`）直接對 X 做 QR 或 SVD 分解，避免了這個平方 [4]。

當兩個特徵幾乎相同時，$\sigma_{\min}$ 非常小，即使預測沒問題，個別係數也會極度敏感。**嶺迴歸**（ridge regression）[8] 加上懲罰項 $\lambda\|w\|_2^2$，$\lambda > 0$：

$$w_{\text{ridge}} = (X^{\top}X + \lambda I)^{-1}X^{\top}y = \sum_{i} \frac{\sigma_i}{\sigma_i^2 + \lambda}\,(u^{(i)\top}y)\,v^{(i)}.$$

右邊的 SVD 形式說明了嶺迴歸在做什麼。一般最小平方使用係數 $1/\sigma_i$，在 σ_i 很小時爆炸；嶺迴歸使用 $\sigma_i/(\sigma_i^2 + \lambda)$，上限是 $1/(2\sqrt{\lambda})$。大 σ 的方向幾乎不受影響，小 σ 的方向被壓抑，代價是引入偏差。在神經網路中，同一個懲罰項稱為**權重衰減**（weight decay）。

```
import numpy as np

rng = np.random.default_rng(0)
```

```
# Over-determined: more equations than unknowns -> least squares
x = np.linspace(-1, 1, 40)
X = np.c_[np.ones_like(x), x]
y = 2 + 3*x + rng.normal(0, .05, len(x))
w, residuals, rank, sv = np.linalg.lstsq(X, y, rcond=None)
print("intercept and slope:", w.round(3), "rank:", rank)
assert np.allclose(w, np.linalg.pinv(X) @ y)

# Under-determined: fewer equations than unknowns -> minimum-norm solution
A = rng.normal(size=(2, 5)); b = rng.normal(size=2)
w_min = np.linalg.pinv(A) @ b
null = np.linalg.svd(A)[2][-1]             # a direction with A @ null = 0
w_other = w_min + 0.7 * null
print("both solve Aw=b:", np.allclose(A @ w_min, b), np.allclose(A @ w_other, b))
print("norms:", np.linalg.norm(w_min).round(3), "<", np.linalg.norm(w_other).round(3))

# Near-collinear features: condition number and ridge
z = rng.normal(size=200)
F = np.c_[z, z + 1e-4 * rng.normal(size=200)]
t = z + 0.01 * rng.normal(size=200)
print("cond(F) = %.1e, cond(F^T F) = %.1e" % (np.linalg.cond(F), np.linalg.cond(F.T @ F)))
w_ls = np.linalg.lstsq(F, t, rcond=None)[0]
lam = 1e-2
w_ridge = np.linalg.solve(F.T @ F + lam * np.eye(2), F.T @ t)
print("least squares:", w_ls.round(2), " ridge:", w_ridge.round(3))
```

直線擬合得到截距 $\approx$ 2.00、斜率 $\approx$ 3.01，而 `pinv` 與 `lstsq` 結果一致。欠定系統有兩個精確解，偽反矩陣給的那一個範數較小（0.885 對 1.128）。在共線的例子中，$\kappa(F) \approx 2 \times 10^4$，而 $\kappa(F^\top F) \approx 4 \times 10^8$，正好是它的平方。最小平方回傳誇張的係數 $(-2.63, 3.63)$，但兩者相加仍約為 1，所以預測沒有問題；嶺迴歸則回傳穩定的 $(0.5, 0.5)$。第一個例子中的截距行讓直線可以有非零偏移。在實際流程中，置中與縮放只能在訓練集上擬合，再原封不動套用到驗證與測試資料。

跡運算子

白話版。跡就是把對角線加起來。它看起來簡單到不值一提，卻能把麻煩的總和改寫成矩陣運算式，並讓你在式子裡搬動矩陣。

方陣的**跡**（trace）是

$$\operatorname{Tr}(A) = \sum_i A_{ii}.$$

它有這些好用的性質：

- **Frobenius 範數**。$\|A\|_F = \sqrt{\operatorname{Tr}(AA^\top)} = \sqrt{\operatorname{Tr}(A^\top A)}$。
- **轉置**。$\operatorname{Tr}(A) = \operatorname{Tr}(A^\top)$。
- **循環性質**。只要乘積有定義，$\operatorname{Tr}(ABC) = \operatorname{Tr}(CAB) = \operatorname{Tr}(BCA)$，即使矩陣不是方陣、中間乘積形狀不同也成立。特別地，對 $A \in \mathbb{R}^{m\times n}$、$B \in \mathbb{R}^{n\times m}$，$\operatorname{Tr}(AB) = \operatorname{Tr}(BA)$。可以循環，但不能任意重排：一般而言 $\operatorname{Tr}(ABC) \neq \operatorname{Tr}(ACB)$。

- **純量**。純量等於自己的跡，$a = \mathrm{Tr}(a)$。這能把二次型變成跡：$x^\top Ax = \mathrm{Tr}(x^\top Ax) = \mathrm{Tr}(Axx^\top)$。
- **特徵值**。$\mathrm{Tr}(A) = \sum_i \lambda_i$。對共變異數矩陣而言，這就是總變異數，也就是沿每個主方向的變異數總和。

許多期望值就是用這個純量技巧算出來的。若隨機向量 x 的平均為零、共變異數為 Σ，則 $\mathbb{E}[x^\top Ax] = \mathbb{E}[\mathrm{Tr}(Axx^\top)] = \mathrm{Tr}(A\Sigma)$，因為跡與期望值都是線性的。

行列式

白話版。方陣的行列式說明它把面積（二維）或體積（更高維）放大了多少倍。若行列式為零，矩陣就把空間壓扁了，無法復原。

方陣的**行列式** $\det(A)$ 是一個數字，具有以下性質：

- $\det(A) = \prod_i \lambda_i$，即特徵值的乘積。
- $|\det(A)|$ 是 A 縮放體積的倍數：單位立方體被映射成體積為 $|\det(A)|$ 的平行多面體。
- 正負號記錄方向（定向）。行列式為負表示 A 包含鏡射。
- $\det(A) = 0$ 恰好表示 A 是奇異的：某個方向被壓成零，資訊遺失。
- $\det(AB) = \det(A)\det(B)$、$\det(A^\top) = \det(A)$、$\det(A^{-1}) = 1/\det(A)$。

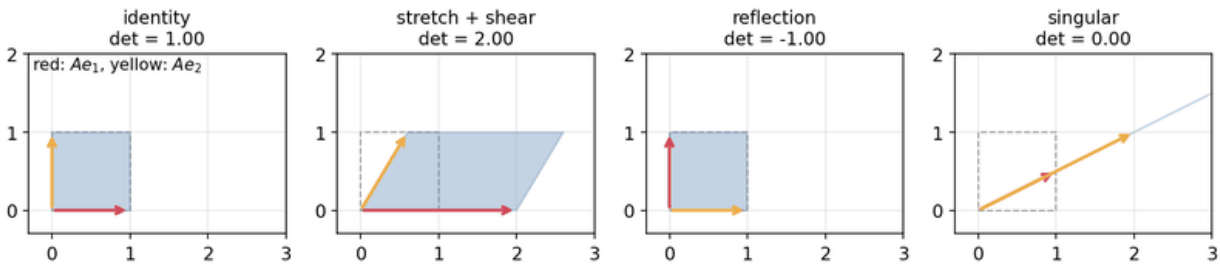


圖 3 — 行列式就是帶正負號的面積。虛線：單位正方形；陰影：它的像，由 *Ae1*（紅）與 *Ae2*（黃）張成。剪切不改變面積；鏡射讓正負號翻轉；奇異矩陣把正方形壓成一條線（$det = 0$）。

行列式在機器學習中主要出現在兩個地方。多變量高斯密度中有 $\det(\Sigma)^{-1/2}$，是共變異數的體積正規化；ML 01B 會用到它。正規化流（normalizing flow，見ML 06）需要 Jacobian 矩陣 J 的 $\log|\det J|$，來記錄可逆網路如何拉伸機率質量。這兩種情況都需要對數，絕不要直接算原始值：一個對角元素皆為 10^{-3} 的 500×500 對角矩陣，行列式是 10^{-1500}，在 float64 中會下溢成零。

```
import numpy as np

rng = np.random.default_rng(2)
A, B, C = rng.normal(size=(3, 4)), rng.normal(size=(4, 5)), rng.normal(size=(5, 3))
print("cyclic trace:", np.isclose(np.trace(A @ B @ C), np.trace(C @ A @ B)),
      np.isclose(np.trace(A @ B @ C), np.trace(B @ C @ A)))

S = rng.normal(size=(4, 4))
lam = np.linalg.eigvals(S)
print("trace = sum of eigenvalues:", np.isclose(np.trace(S), lam.sum().real))
```

```
print("det = product of eigenvalues:", np.isclose(np.linalg.det(S), np.prod(lam).real))

# log-determinant of a covariance without overflow/underflow
Sigma = np.diag(np.full(500, 1e-3))
print("det underflows:", np.linalg.det(Sigma))
sign, logdet = np.linalg.slogdet(Sigma)
print("slogdet:", sign, round(logdet, 2))
L = np.linalg.cholesky(Sigma)
print("via Cholesky:", round(2 * np.log(np.diag(L)).sum(), 2))
```

直接計算行列式印出 `0.0`，而 `slogdet` 與 Cholesky 兩種做法都得到 $\log\det = 500\log 10^{-3} \approx -3453.88$。對正定矩陣，若 $A = LL^\top$，則 $\det(A) = \prod_i L_{ii}^2$，所以 $\log\det A = 2\sum_i \log L_{ii}$。

實例：推導主成分分析

白話版。PCA 要找出資料點雲最貼近的那個平坦子空間。把資料投影上去，每個點就能壓縮成少數幾個數字，同時損失最少。

PCA 可追溯到 Pearson 的「最貼近點集的直線與平面」[9] 與 Hotelling 對「主成分」的分析 [10]；Jolliffe 與 Cadima 提供了現代的綜述 [11]。我們依照 [1] 的方式，只用上面介紹的工具來推導。

設定。有 n 個點 $x^{(1)}, \dots, x^{(n)} \in \mathbb{R}^d$，想讓每個點只用 $l < d$ 個數字儲存。先從每個點減去平均 $\mu = \frac{1}{n}\sum_i x^{(i)}$，因此可以假設資料已置中。把置中後的點當作 $X \in \mathbb{R}^{n\times d}$ 的各列。

步驟 1：選定編碼的形式。使用線性**解碼器** $g(c) = Dc$，$D \in \mathbb{R}^{d\times l}$。為了讓問題良好設定，要求 D 的各行單範正交：$D^\top D = I_l$。（否則可以無止境地縮小 c、放大 D。）

步驟 2：固定解碼器，找最佳編碼。對給定的 x，選擇 c 使重建誤差最小：

$$c^\ast = \arg\min_c \|x - Dc\|_2^2 = \arg\min_c \left(-2x^\top Dc + c^\top D^\top Dc \right) = \arg\min_c \left(-2x^\top Dc + c^\top c \right).$$

我們展開平方、丟掉與 c 無關的 $x^\top x$，並使用 $D^\top D = I$。對 c 的梯度是 $-2D^\top x + 2c$，令其為零就得到**編碼器**

$$f(x) = D^\top x.$$

所以編碼就是一次矩陣–向量乘法，而重建 $r(x) = g(f(x)) = DD^\top x$ 正是 x 到 D 生成空間的正交投影。

步驟 3：找最佳解碼器。現在選擇 D，使所有點的總重建誤差最小：

$$D^\ast = \arg\min_D \|X - XDD^\top\|_F^2 \quad \text{subject to } D^\top D = I_l.$$

先看 $l = 1$，此時 D 是單一單位向量 d。把 Frobenius 範數寫成跡並展開：

$$\begin{aligned} \|X - Xdd^\top\|_F^2 &= \operatorname{Tr}\left((X - Xdd^\top)^\top (X - Xdd^\top)\right) \\ &= \operatorname{Tr}(X^\top X) - 2\operatorname{Tr}(X^\top Xdd^\top) + \operatorname{Tr}(dd^\top X^\top Xdd^\top) \\ &= \operatorname{Tr}(X^\top X) - 2\,d^\top X^\top Xd + d^\top X^\top Xd\,(d^\top d) \\ &= \operatorname{Tr}(X^\top X) - d^\top X^\top Xd. \end{aligned}$$

第二項與第三項用循環性質把 $d^\top$ 移到最前面，最後一行用 $d^\top d = 1$ 化簡。第一項與 d 無關，所以最小化誤差等同於

$$d^* = \arg\max_{d} \; d^\top X^\top X d \quad \text{subject to } d^\top d = 1.$$

這是對稱半正定矩陣 $X^\top X$ 的 Rayleigh 商。由特徵分解一節可知，它在 $X^\top X$ **最大特徵值對應的特徵向量**處達到最大。

步驟 4：多個主成分。對一般的 l 重複這個論證（例如用歸納法，要求每個新方向與先前的方向正交），答案是：D 的各行是 $X^\top X$ 前 l 大特徵值對應的特徵向量 [1]。最小誤差等於被捨棄的特徵值總和。

步驟 5：變異數觀點與 SVD 的連結。把 $X^\top X$ 除以 n，得到樣本共變異數 $S = \frac{1}{n} X^\top X$（有些教科書用 $n-1$）。$d^\top S d$ 是投影後資料 Xd 的變異數。所以「最小化重建誤差」與「最大化保留的變異數」選出的是同樣的方向——Pearson 與 Hotelling 的觀點一致。最後，若置中資料矩陣的精簡 SVD 為 $X = U\Sigma V^\top$，則 $X^\top X = V\Sigma^2 V^\top$：主方向就是右奇異向量 V，特徵值是 σ_i^2，編碼是 $XV_l = U_l\Sigma_l$。實務上 PCA 是用 X 的 SVD 計算，而不是先形成共變異數矩陣，理由與最小平方法中的條件數相同。

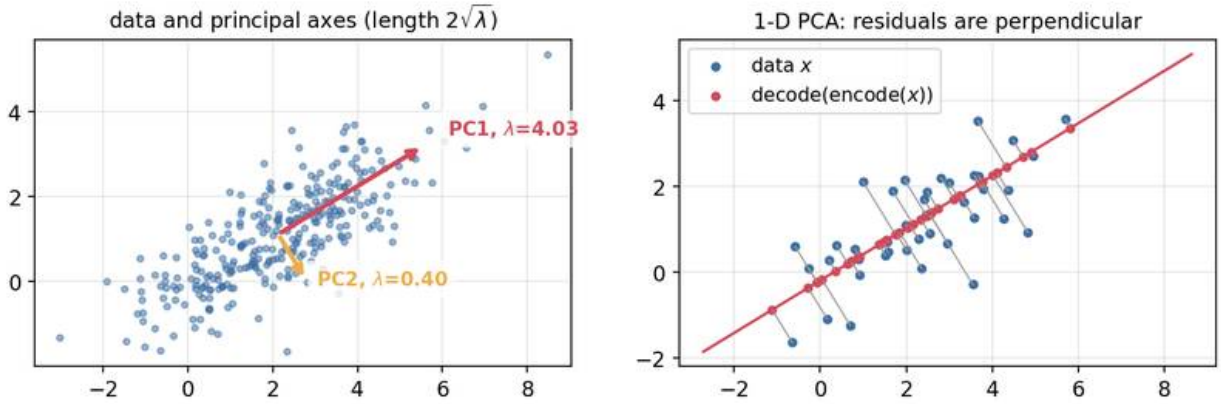


圖 4 — 對 300 個相關二維資料點做 PCA。左：由樣本共變異數特徵分解得到的主軸，長度畫為 $2\sqrt{\lambda}$。右：一維 PCA 的重建 $decode(encode(x))$；每條灰色殘差都與紅線垂直，因為重建就是正交投影。

```
import numpy as np

rng = np.random.default_rng(0)
n, d, l = 500, 5, 2
Z = rng.normal(size=(n, l))
X = Z @ rng.normal(size=(l, d)) + 0.1 * rng.normal(size=(n, d)) + 3.0

mu = X.mean(axis=0)
Xc = X - mu                                  # centre first
S = Xc.T @ Xc / n                            # sample covariance (d x d)
lam, Q = np.linalg.eigh(S)
lam, Q = lam[::-1], Q[:, ::-1]               # descending
D = Q[:, :l]                                 # decoder matrix, orthonormal columns
```

```
codes = Xc @ D                                  # encode: f(x) = D^T (x - mu)
X_hat = codes @ D.T + mu                        # decode: g(c) = D c + mu
mse = np.mean(np.sum((X - X_hat)**2, axis=1))
print("explained variance ratio:", (lam[:l].sum() / lam.sum()).round(4))
print("reconstruction MSE:", mse.round(5), " sum of discarded eigenvalues:",
 ↪  lam[l:].sum().round(5))
assert np.isclose(mse, lam[l:].sum())

# Same subspace from the SVD of the centred data matrix
U, s, Vt = np.linalg.svd(Xc, full_matrices=False)
assert np.allclose(s[:l]**2 / n, lam[:l])
assert np.allclose(np.abs(Vt[:l] @ D), np.eye(l), atol=1e-8)
print("SVD and eigendecomposition agree")
```

這份合成資料是五維的點，實際上落在一個二維平面附近，再加上小雜訊。兩個主成分解釋了 99.36% 的變異數，而平均平方重建誤差剛好等於被捨棄的三個特徵值總和，與步驟 4 的預測一致。SVD 的做法找到同樣的方向（差一個正負號，特徵向量的正負號本來就是任意的）。

三個實務提醒。一定要先置中，否則第一個「主成分」只會指向平均值。若特徵的單位無法比較，要先標準化，因為 PCA 不具尺度不變性。平均與方向都只能在訓練資料上擬合。PCA 是自編碼器的線性祖先；ML 05A 會發展它的機率版本與非線性後繼者。

你會遇到的矩陣微積分

白話版。訓練就是沿著梯度走，而機器學習中的大多數梯度，都是少數幾條矩陣公式反覆使用。

對向量 $x \in \mathbb{R}^n$ 的純量函數 $f(x)$，**梯度** $\nabla_x f \in \mathbb{R}^n$ 收集所有偏導數 $\partial f / \partial x_i$。對向量函數 $f : \mathbb{R}^n \to \mathbb{R}^m$，**Jacobian 矩陣** $J \in \mathbb{R}^{m \times n}$ 的元素為 $J_{ij} = \partial f_i / \partial x_j$。最常用的規則如下：

函數	梯度	備註
$a^\top x$	a	線性函數
$x^\top A x$	$(A + A^\top)x$	A 對稱時為 $2Ax$
$\tfrac{1}{2}\lVert Xw - y \rVert_2^2$	$X^\top(Xw - y)$	最小平方損失
$\lVert x \rVert_2^2$	$2x$	權重衰減
$\operatorname{Tr}(AB)$ 對 A	$B^\top$	搭配上面的跡技巧

每一條都可以把運算式寫成對索引的總和、逐項微分來驗證 [2]。向量函數的**連鎖律**是把 Jacobian 相乘：若 $y = g(x)$，而 $L = f(y)$ 是純量，則 $\nabla_x L = J_g^\top \nabla_y L$。反向傳播逐層套用這條規則，把轉置的局部 Jacobian 乘上傳進來的梯度，而不必真的建出完整的 Jacobian（那會大得驚人）。ML 02 會詳細建構這個過程，ML 01C 則加入 Hessian，也就是二階導數矩陣。

令最小平方梯度為零，就回到前一節的正規方程式。這個損失的 Hessian 是 $X^\top X$，為半正定，所以損失是凸函數，每個駐點都是全域最小值。用有限差分快速檢查公式：

```
import numpy as np
```

```
rng = np.random.default_rng(3)
X, y = rng.normal(size=(30, 4)), rng.normal(size=30)
w = rng.normal(size=4)
L = lambda w: 0.5 * np.sum((X @ w - y)**2)
grad = X.T @ (X @ w - y)                    # analytic gradient
eps = 1e-6
num = np.array([(L(w + eps*e) - L(w - eps*e)) / (2*eps) for e in np.eye(4)])
print("max gradient mismatch:", np.abs(grad - num).max())
# Setting the gradient to zero gives the normal equations
w_star = np.linalg.lstsq(X, y, rcond=None)[0]
print("gradient at solution ~ 0:", np.abs(X.T @ (X @ w_star - y)).max() < 1e-10)
```

解析梯度與數值梯度相差約 10^{-8}，正是有限差分捨入誤差預期的大小。

現代觀點

本篇的定義已有百年歷史，但它們在機器學習中的用法仍不斷改變。以下四條研究方向值得認識。

隨機化數值線性代數。大型矩陣的精確 SVD 很昂貴，大約需要 $O(mn\min(m,n))$ 次運算。Halko、Martinsson 與 Tropp 在 SIAM Review 的綜述 [12] 讓一個簡單的想法成為標準做法：用一個細長的隨機矩陣 Ω 乘上 A，「草描」出它的值域；把結果正交化得到基底 Q；再對小矩陣 $Q^{\top}A$ 做精確 SVD。加上少量過取樣與幾次冪迭代，對奇異值會衰減的矩陣就能得到準確的結果。Martinsson 與 Tropp 在 *Acta Numerica* 的綜述 [13] 涵蓋理論與大量演算法：範數估計、隨機嵌入、迴歸、低秩近似、半正定矩陣的 Nyström 近似、串流演算法，以及機器學習中的核矩陣。Murray 等人 [14]（LAPACK Working Note 299，2023）從軟體角度切入：他們指出 RandNLA 方法已進入 MATLAB、NAG Library、NVIDIA 的 cuSOLVER 與 scikit-learn，並主張建立標準的「RandBLAS」與「RandLAPACK」程式庫。scikit-learn 的隨機化 PCA 求解器就是直接的後代。核心演算法只要幾行：

```
import numpy as np

def randomized_svd(A, k, oversample=10, power_iters=2, seed=0):
    rng = np.random.default_rng(seed)
    Omega = rng.normal(size=(A.shape[1], k + oversample))  # random test matrix
    Y = A @ Omega                                          # sample the range of A
    for _ in range(power_iters):                           # sharpen a slow spectrum
        Y = A @ (A.T @ Y)
    Q, _ = np.linalg.qr(Y)                                 # orthonormal basis
    B = Q.T @ A                                            # small (k+p) x n matrix
    Ub, s, Vt = np.linalg.svd(B, full_matrices=False)
    return (Q @ Ub)[:, :k], s[:k], Vt[:k]

rng = np.random.default_rng(1)
m, n, r = 2000, 500, 20
A = rng.normal(size=(m, r)) @ rng.normal(size=(r, n)) + 0.01 * rng.normal(size=(m, n))
U, s, Vt = randomized_svd(A, k=r)
s_exact = np.linalg.svd(A, compute_uv=False)[:r]
```

```
print("max relative error in top singular values:", np.max(np.abs(s - s_exact) / s_exact))
rel = np.linalg.norm(A - (U * s) @ Vt) / np.linalg.norm(A)
print("relative reconstruction error:", round(rel, 5))
```

對這個近似秩 20 的矩陣，前 20 個奇異值與精確值的相對誤差約 10^{-15}，而秩 20 重建的誤差（約 0.2%）由加入的雜訊決定。冪迭代是奇異值衰減緩慢時仍能成功的關鍵；若矩陣的奇異值完全平坦，任何秩 k 的方法——隨機或非隨機——都做不好。

大型模型中的低秩結構。矩陣乘法的外積觀點，已經成為調整大型網路的實用工具。LoRA [15] 凍結預訓練權重 $W_0 \in \mathbb{R}^{d\times k}$，只學習更新量 $\Delta W = BA$，其中 $B \in \mathbb{R}^{d\times r}$、$A \in \mathbb{R}^{r\times k}$，且 $r \ll \min(d,k)$。可訓練的數字只有 $r(d+k)$ 個，而不是 dk 個；以 $d = k = 4096$、$r = 8$ 為例，每個矩陣是 65,536 個，而不是約 1,680 萬個。B 初始化為零，所以訓練恰好從預訓練模型出發。GaLore [16]（ICML 2024）則讓所有權重都可訓練，但把梯度投影到低秩子空間，以縮小最佳化器的記憶體用量；作者認為像 LoRA 那樣把權重更新本身限制在低秩子空間，可能不如全秩訓練。兩者都依賴 Eckart–Young 的直覺：矩陣中有用的內容大多集中在少數幾個奇異方向。這在特定任務上是否成立，是實證問題，不是定理。

控制譜範數。網路對輸入擾動的敏感度，上限是各層譜範數的乘積（在 ReLU 這類 1-Lipschitz 激勵函數下）。譜正規化（spectral normalization）[17] 把每個權重矩陣除以最大奇異值的估計值，而這個估計在每個訓練步驟只需一次冪迭代，成本很低。它原本是為了穩定 GAN 的判別器而提出（見ML 06），凡是需要 Lipschitz 上界的地方都能看到同樣的想法。

超越矩陣。卷積核與影片特徵天生就是三軸以上的張量。Kolda 與 Bader 在 SIAM Review 的綜述 [18] 是張量分解的標準入門，涵蓋 CP 分解（秩一張量的總和，推廣了外積觀點）與 Tucker 分解（一個小的核心張量沿每個模態乘上一個矩陣，推廣了 SVD）。與矩陣不同，張量的秩一般而言很難計算，最佳低秩近似也不一定存在，Eckart–Young 那幅清楚的圖像無法直接沿用。這些分解被用來壓縮網路層，以及分析多向資料。

開放問題。為什麼訓練後的權重矩陣常有快速衰減的奇異值？這和泛化有什麼關係？調整用的秩可以多低才不損失品質？這取決於任務還是模型？隨機化演算法要如何與低精度硬體結合——那裡的捨入誤差遠大於 float64？這些問題都還沒有定論，而本篇的工具正是閱讀相關論文所需要的。若想找把線性代數與深度學習連起來的完整教科書，可參考 [1]、[2] 與 [19]。

重點整理

- 為每個軸命名。大多數無聲的錯誤都是形狀或廣播錯誤，`assert` 能抓到它們。
- 矩陣乘法有四種讀法（元素、行、列、外積總和）；選擇讓問題最清楚的那一種。
- 秩與零空間決定 $Ax = b$ 有零個、一個或無限多個解。程式中不要形成 A^{-1}，改用求解器。
- 對稱矩陣有實數特徵值與單範正交的特徵向量；特徵值的正負號決定二次型是碗、鞍還是倒扣的碗。
- 每個矩陣都有 SVD：旋轉、拉伸、旋轉。截斷 SVD 給出最佳低秩近似，誤差等於被丟掉的奇異值。
- 偽反矩陣給出最小平方解與最小範數解。小奇異值會放大雜訊；條件數量測放大多少，嶺正則化則能抑制它。
- 跡與行列式分別是特徵值的和與積。推導時用跡的循環性質，程式中用對數行列式。

- PCA 是投影到共變異數前幾大特徵向量上的投影，等價於置中資料的前幾個右奇異向量。最小化重建誤差與最大化變異數得到相同答案。

練習

1. **形狀與偏差**。一個批次形狀為 32×64，權重矩陣形狀為 64×10。偏差應該是什麼形狀？若改用形狀 $(32, 1)$ 的偏差，會出什麼問題？

   提示

   形狀 $(10,)$ 的偏差會沿 32 列廣播：每個輸出單元一個偏移量，由所有樣本共用。$(32, 1)$ 的偏差則沿 10 行廣播，變成每個樣本一個偏移量。程式照樣能跑，但「偏差」現在取決於樣本在批次中的位置，這沒有意義，而且會隨批次大小改變。

2. **投影**。設 $Q \in \mathbb{R}^{n \times k}$ 的各行單範正交，$P = QQ^\top$。證明 $P^2 = P$、$P^\top = P$，且 $x - Px$ 與 Q 的每一行正交。P 的特徵值是什麼？

   提示

   $P^2 = Q(Q^\top Q)Q^\top = QIQ^\top = P$。正交性：計算 $Q^\top(x - Px) = Q^\top x - Q^\top QQ^\top x = 0$。由 $P^2 = P$，任何特徵值滿足 $\lambda^2 = \lambda$，所以 $\lambda \in \{0, 1\}$：特徵值 1 的重數為 k（子空間本身），0 的重數為 $n - k$（它的正交補空間）。因此 $\operatorname{Tr}(P) = k$。

3. **手算 SVD**。透過計算 $A^\top A$ 的特徵值，求 $A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$ 的奇異值。驗證 $\|A\|_F^2 = \sigma_1^2 + \sigma_2^2$ 以及 $|\det A| = \sigma_1 \sigma_2$。

   提示

   $A^\top A = \begin{bmatrix} 25 & 20 \\ 20 & 25 \end{bmatrix}$ 的特徵值是 45 與 5，所以 $\sigma_1 = 3\sqrt{5}$、$\sigma_2 = \sqrt{5}$。於是 $\sigma_1^2 + \sigma_2^2 = 50 = 9 + 16 + 25$，且 $\sigma_1 \sigma_2 = 15 = |\det A| = |3 \cdot 5 - 0 \cdot 4|$。一般而言，方陣的 $|\det A| = \prod_i \sigma_i$，因為 $|\det U| = |\det V| = 1$。

4. **條件數與嶺迴歸**。從最小平方的程式碼出發，把第二個特徵換成第一個特徵的完全複本。`np.linalg.lstsq` 會回傳什麼？為什麼它是最小範數解？接著推導嶺迴歸解的 SVD 形式，並證明每個係數 $\sigma/(\sigma^2 + \lambda)$ 最多是 $1/(2\sqrt{\lambda})$。

   提示

   兩行完全相同時秩為 1，解集合是一條直線 $w_1 + w_2 = c$。`lstsq` 使用 SVD，回傳偽反矩陣解，也就是平分權重 $w_1 = w_2 = c/2$，因為直線上離原點最近的就是這一點。至於上界，最大化 $g(\sigma) = \sigma/(\sigma^2 + \lambda)$：$g'(\sigma) = 0$ 發生在 $\sigma = \sqrt{\lambda}$，此時 $g = 1/(2\sqrt{\lambda})$。

5. **PCA 的誤差**。利用本篇的推導，證明 l 個主成分的 PCA 總重建誤差等於 $\sum_{i>l} \lambda_i(X^\top X)$，而保留的變異數比例是 $\sum_{i \le l} \lambda_i / \sum_i \lambda_i$。接著對 `sklearn.datasets.load_digits()` 做 PCA，找出保留 90% 變異數所需的最小 l。

提示

寫成 $\|X - XDD^\top\|_F^2 = \mathrm{Tr}(X^\top X) - \mathrm{Tr}(D^\top X^\top X D)$（使用 $D^\top D = I$ 與循環性質）。當 D 是前 l 個特徵向量時，第二個跡等於 $\sum_{i \le l} \lambda_i$，而 $\mathrm{Tr}(X^\top X) = \sum_i \lambda_i$。手寫數字資料：把 1797×64 的矩陣置中，計算奇異值，再對 σ_i^2 取累積和。請回報你自己算出的數字，不要相信別人引用的數字。

6. **行列式與體積**。某個 2×2 矩陣把單位正方形映射成頂點為 0、$(2,1)$、$(1,3)$、$(3,4)$ 的平行四邊形。寫出這樣的矩陣，計算它的行列式，並說明為什麼正規化流必須記錄每一層的 $\log|\det J|$。

提示

把兩條邊向量放進各行：$A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$，$\det A = 5$，所以面積放大 5 倍。若一個映射把體積放大 5 倍，機率密度就必須縮小同樣倍數，總機率才能維持為 1。疊加多層時這些倍數相乘，取對數後變成相加，這就是正規化流逐層加總對數行列式的原因。

參考文獻

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 2 “Linear Algebra”. Book
2. M. P. Deisenroth, A. A. Faisal and C. S. Ong, *Mathematics for Machine Learning*, Cambridge University Press, 2020. Book
3. G. Strang, *Introduction to Linear Algebra*, 6th ed., Wellesley-Cambridge Press, 2023. Book site
4. G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press, 2013. DOI
5. L. N. Trefethen and D. Bau III, *Numerical Linear Algebra*, SIAM, 1997. DOI
6. C. Eckart and G. Young, “The Approximation of One Matrix by Another of Lower Rank,” *Psychometrika*, vol. 1, pp. 211–218, 1936. DOI
7. R. Penrose, “A Generalized Inverse for Matrices,” *Mathematical Proceedings of the Cambridge Philosophical Society*, vol. 51, pp. 406–413, 1955. DOI
8. A. E. Hoerl and R. W. Kennard, “Ridge Regression: Biased Estimation for Nonorthogonal Problems,” *Technometrics*, vol. 12, pp. 55–67, 1970. DOI
9. K. Pearson, “On Lines and Planes of Closest Fit to Systems of Points in Space,” *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, vol. 2, pp. 559–572, 1901. DOI
10. H. Hotelling, “Analysis of a Complex of Statistical Variables into Principal Components,” *Journal of Educational Psychology*, vol. 24, pp. 417–441, 1933. DOI
11. I. T. Jolliffe and J. Cadima, “Principal Component Analysis: A Review and Recent Developments,” *Philosophical Transactions of the Royal Society A*, vol. 374, 20150202, 2016. DOI
12. N. Halko, P.-G. Martinsson and J. A. Tropp, “Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions,” *SIAM Review*, vol. 53, no. 2, pp. 217–288, 2011. DOI ·arXiv
13. P.-G. Martinsson and J. A. Tropp, “Randomized Numerical Linear Algebra: Foundations and Algorithms,” *Acta Numerica*, vol. 29, pp. 403–572, 2020. DOI ·arXiv

14. R. Murray, J. Demmel, M. W. Mahoney, N. B. Erichson, M. Melnichenko, O. A. Malik, L. Grigori, P. Luszczek, M. Dereziński, M. E. Lopes, T. Liang, H. Luo and J. Dongarra, "Randomized Numerical Linear Algebra: A Perspective on the Field With an Eye to Software," LAPACK Working Note 299, 2023. arXiv
15. E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," in *Proc. International Conference on Learning Representations* (*ICLR*), 2022. arXiv
16. J. Zhao, Z. Zhang, B. Chen, Z. Wang, A. Anandkumar and Y. Tian, "GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection," *ICML*, 2024. arXiv
17. T. Miyato, T. Kataoka, M. Koyama and Y. Yoshida, "Spectral Normalization for Generative Adversarial Networks," *ICLR*, 2018. arXiv
18. T. G. Kolda and B. W. Bader, "Tensor Decompositions and Applications," *SIAM Review*, vol. 51, no. 3, pp. 455–500, 2009. DOI
19. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. Book

ML 01B・機率與資訊理論

隨機變數、常見分布、貝氏定理、期望值與共變異數、概似、熵、交叉熵、KL 散度與結構化模型，並連結到神經網路的訓練與評估。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-01b-probability-information/

你將學到

- 機器學習為什麼需要機率，以及解讀機率的兩種常見方式（頻率與信念程度）。
- 隨機變數、機率質量函數與機率密度函數如何運作，以及邊際、條件與聯合機率如何透過加法規則、乘法規則與連鎖規則彼此連結。
- 期望值、變異數、共變異數與相關係數如何描述一個分布，以及為什麼共變異數為零不代表獨立。
- 深度學習裡反覆出現的分布——Bernoulli、類別分布、高斯、指數、Laplace、Dirac、經驗分布與混合分布——以及用來參數化它們的 sigmoid 與 softplus 函數。
- 貝氏定理、變數變換公式、最大概似估計與 MAP 估計如何運作，並附上實際測試過的 NumPy 程式碼。
- 熵、交叉熵、KL 散度與互資訊各自衡量什麼，為什麼分類損失就是交叉熵，以及結構化（圖）模型如何把龐大的聯合分布拆成小因子。

先看全貌

神經網路永遠看不到整個世界。它只看到有限的訓練集——由不完美的感測器量測、由忙碌的人標註——接著卻要回答從沒看過的影像。這個過程的每一步都牽涉**不確定性**。機率論就是描述不確定性的數學：它告訴我們如何表達自己不知道的事、如何結合證據，以及如何做出承認懷疑的預測。與它關係密切的資訊理論，則衡量不確定性有多少，以及使用錯誤分布要付出多少代價。兩者合起來，幾乎可以解釋本站 ML 與 DL 系列中的每一個損失函數。

> **用最簡單的話說**
>
> 想像一個看不到內部的彈珠袋。**機率**是你對下一顆彈珠的最佳猜測：「大概是紅色，也許是藍色，幾乎不可能是綠色。」每摸出一顆彈珠，你就更了解這個袋子，並更新你的猜測——這個更新就是**貝氏定理**。**熵**是你預期會有多驚訝：全是紅色彈珠的袋子永遠不會讓你驚訝；十種顏色各占一樣多的袋子則常常讓你驚訝。機器學習模型是一個很厲害的猜測者，而訓練就是讓它對真正摸出的彈珠越來越不驚訝。

本篇依循 Goodfellow、Bengio 與 Courville《Deep Learning》中的「應用數學」部分 [1]，並補充 Bishop 與 Bishop [2]、Murphy [3] 以及《Mathematics for Machine Learning》[4] 的內容。它位於ML 01A・線性代數與ML 01C・數值計算之間。如果你接下來要讀圖形識別系列，PR 01・貝氏決策會直接延伸這裡的貝氏定理一節。

為什麼需要機率？

白話版。電腦科學通常處理確定的事實：一個位元不是 0 就是 1，程式不是停止就是不停止。機器學習沒辦法這樣運作，因為資料、世界和模型都不完美。

Goodfellow 等人 [1] 列出三種不確定性的來源：

1. **系統本身的隨機性**。有些系統本來就是隨機的——光子抵達相機感測器的時間（散粒雜訊）、骰子的點數，或行人踏下人行道的確切時刻。
2. **觀測不完整**。即使系統是決定性的，只要我們看不到它所有的變數，它看起來就是隨機的。一張汽車照片無法告訴你煞車燈是不是快要亮起。
3. **建模不完整**。模型必須捨棄資訊才有用處。只看 32×32 縮圖的分類器無法找回降取樣時遺失的細節，所以它對細粒度類別的預測必然帶有不確定性。

使用帶有不確定性的簡單規則（「大多數鳥會飛」），往往比使用永遠正確但極其複雜的規則（「鳥會飛，但企鵝、鴕鳥、受傷的鳥……除外」）更實際。

解讀機率的兩種方式

機率可以有兩種解讀，而兩者遵守相同的數學規則。

- **頻率學派機率**描述可重複事件的長期頻率：「這枚硬幣擲很多次，大約 70% 會出現正面」。
- **貝氏機率**描述對某件不一定能重複的事的信念程度：「看了這張 X 光片，我有 40% 的把握病人罹病」。

只要要求信念程度滿足幾項符合常識的一致性性質，就能證明它們必須遵守與頻率完全相同的規則，因此同一套計算可以同時服務兩種解讀 [1], [3]。實務上，深度學習混用兩者：資料被視為從某個分布抽出的樣本（頻率觀點），而模型的輸出常被解讀成對標籤的信念（貝氏觀點）。

隨機變數與機率分布

白話版。隨機變數是事先不知道值的量；機率分布說明每個值出現的可能性有多大。

[1] 用正體字母（例如 x）表示隨機變數；本篇用 X 表示變數、x 表示它的一個取值。隨機變數可以是**離散的**（有限或可數個值，例如 $\{1, \dots, K\}$ 中的類別標籤），也可以是**連續的**（實數值，例如被視為實數的像素強度）。

機率質量函數

離散變數有**機率質量函數**（probability mass function, PMF）$P(X = x)$，常簡寫為 $P(x)$。它必須滿足

$$0 \le P(x) \le 1 \text{ for every } x, \qquad \sum_{x} P(x) = 1.$$

這裡的總和涵蓋 X 所有可能的值 x。第一個條件說沒有事件比「確定」更確定；第二個條件說總有某件事會發生。聯合 PMF $P(X = x, Y = y)$ 則是兩個事件同時發生的機率。

機率密度函數

連續變數有**機率密度函數**（probability density function, PDF）$p(x)$，必須滿足

$$p(x) \geq 0, \qquad \int p(x)\,dx = 1.$$

密度不是機率。X 落在區間 $[a, b]$ 的機率是密度曲線下的面積：

$$P(a \leq X \leq b) = \int_a^b p(x)\,dx.$$

任何單一精確值的機率都是零。因為只有面積必須不超過一，密度本身可以大於一：標準差為 0.25 的窄高斯，峰值約為 1.6（圖 1 右上）。把密度值當成「機率 1.6」回報，是常見錯誤。

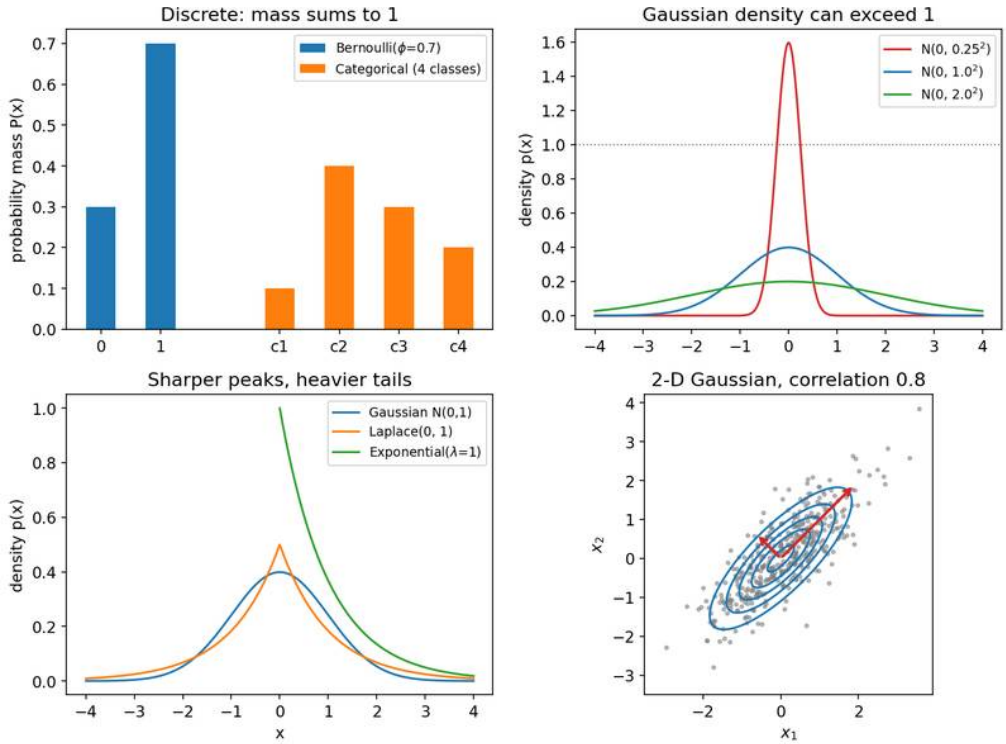


圖 *1* — 常見分布。左上：離散機率質量總和為一。右上：密度可以超過一，因為只有面積必須等於一。左下：*Laplace* 密度峰值比高斯尖、尾巴比高斯厚；指數分布只定義在 $x \geq 0$。右下：相關係數 *0.8* 的二維高斯；紅色箭頭是共變異數矩陣的特徵向量，長度為兩倍標準差。

邊際機率

白話版。如果知道兩件事如何一起變化，就可以把其中一件「加總掉」，得到另一件事單獨的變化方式。

給定 X 與 Y 的聯合分布，X 的**邊際**分布為

$$P(x) = \sum_{y} P(x, y) \quad \text{(discrete)}, \qquad p(x) = \int p(x, y)\, dy \quad \text{(continuous)}.$$

這就是**加法規則**（sum rule）。如果把聯合分布寫成表格，列代表 x、欄代表 y，那麼 x 的邊際分布就是每一列的總和——過去習慣寫在紙張的邊緣（margin），因此得名。

條件機率

白話版。條件機率回答的是：「在已經知道那件事的情況下，這件事有多可能？」

$$P(y \mid x) = \frac{P(x, y)}{P(x)}, \qquad P(x) > 0.$$

$P(y \mid x)$ 是觀察到 $X = x$ 之後 $Y = y$ 的機率，只有在條件真的可能發生時才有定義。移項後得到**乘法規則**（product rule）$P(x, y) = P(y \mid x) P(x)$。

不要把條件化和介入混為一談。「有人帶傘的情況下街道是濕的」機率很高，但強迫大家帶傘並不會讓街道變濕。關於介入的問題屬於因果推論，光有聯合分布是不夠的。

機率的連鎖規則

反覆套用乘法規則，可以把任何 n 個變數的聯合分布分解成

$$P(x^{(1)}, \dots, x^{(n)}) = P(x^{(1)}) \prod_{i=2}^{n} P(x^{(i)} \mid x^{(1)}, \dots, x^{(i-1)}).$$

這裡 $x^{(i)}$ 是第 i 個變數的值。這個恆等式是精確的——不需要任何假設。它是自迴歸模型的骨幹：語言模型或自迴歸影像模型，會根據前面所有的 token 或像素，一次預測一個（ML 08、NLP 01）。建模的選擇在於如何參數化每一個條件分布，而不在分解本身。

獨立與條件獨立

如果

$$p(x, y) = p(x)\, p(y) \quad \text{for all } x, y,$$

就稱兩個變數**獨立**，記為 $X \perp Y$。如果

$$p(x, y \mid z) = p(x \mid z)\, p(y \mid z) \quad \text{for all } x, y, z,$$

就稱它們在給定 Z 下**條件獨立**，記為 $X \perp Y \mid Z$。

冰淇淋銷量和曬傷人數彼此相關，但在給定天氣之後，兩者大致獨立。條件獨立是讓高維模型變得可計算的主要工具——單純貝氏（naive Bayes）假設特徵在給定類別下彼此獨立；隱藏式馬可夫模型假設給定目前狀態後，未來與過去獨立。這些都是假設 [3]：應該由問題本身來支持，而不是因為能讓代數變簡單就採用。

程式碼：聯合機率表、邊際與條件機率

下面的例子為天氣與帶傘建立一張小型聯合機率表，計算邊際與條件機率，並檢查是否獨立。它也展示了兩個變數明顯相依、共變異數卻（幾乎）為零的情況。

```
import numpy as np

# Joint table P(weather, umbrella): rows = sunny/rainy, columns = no/yes
P = np.array([[0.56, 0.14],
              [0.06, 0.24]])
assert np.isclose(P.sum(), 1)

p_weather = P.sum(axis=1)            # marginal over umbrella
p_umbrella = P.sum(axis=0)           # marginal over weather
p_umb_given_w = P / p_weather[:, None]   # each row sums to 1
print("P(weather):", p_weather)
print("P(umbrella):", p_umbrella)
print("P(umbrella | weather):\n", p_umb_given_w)
print("independent?", np.allclose(P, np.outer(p_weather, p_umbrella)))

# Zero covariance does not imply independence
rng = np.random.default_rng(0)
x = rng.uniform(-1, 1, 200_000)
y = x**2                              # y is a deterministic function of x
print("cov(x, y) ~", np.cov(x, y)[0, 1].round(4))
```

輸出：邊際分布為 $[0.7, 0.3]$ 與 $[0.62, 0.38]$；條件機率表為 $[[0.8, 0.2], [0.2, 0.8]]$；兩個變數不獨立；而 x 與 x^2 的樣本共變異數約為 -0.0007，幾乎是零——即使 y 完全由 x 決定。

期望值、變異數與共變異數

白話版。期望值是長期平均；變異數是數值通常偏離平均多遠；共變異數則說明兩個量是否傾向一起上升、一起下降。

函數 f 在某個分布下的**期望值**為

$$\mathbb{E}_{x\sim P}[f(x)] = \sum_{x} P(x) f(x), \qquad \mathbb{E}_{x\sim p}[f(x)] = \int p(x) f(x)\, dx.$$

下標 $x \sim P$ 指出對哪個分布取平均；意思明確時可以省略。期望值是**線性的**：對常數 α, β，$\mathbb{E}[\alpha f(x) + \beta g(x)] = \alpha \mathbb{E}[f(x)] + \beta \mathbb{E}[g(x)]$。幾乎每一個訓練損失都是對資料分布的期望值，並用小批次（mini-batch）的平均來估計。

變異數衡量數值在平均 $\mu = \mathbb{E}[f(x)]$ 周圍的分散程度：

$$\mathrm{Var}(f(x)) = \mathbb{E}\big[(f(x) - \mu)^2\big] = \mathbb{E}[f(x)^2] - \mu^2.$$

它的平方根是**標準差**，單位與 $f(x)$ 相同。

兩個量的**共變異數**衡量它們如何一起變化：

$$\mathrm{Cov}(f(x), g(y)) = \mathbb{E}\big[(f(x) - \mathbb{E}[f(x)])\,(g(y) - \mathbb{E}[g(y)])\big].$$

共變異數為正，代表兩者傾向同時高於（或低於）各自的平均；為負，代表一個高時另一個傾向低。它的大小取決於單位，所以我們常把它正規化成**相關係數**

$$\rho = \frac{\mathrm{Cov}(X, Y)}{\sqrt{\mathrm{Var}(X)\,\mathrm{Var}(Y)}} \in [-1, 1].$$

獨立意味著共變異數為零，但**共變異數為零不代表獨立**。共變異數只衡量線性相依。在上面的程式碼中，x 在 $[-1, 1]$ 上均勻分布、$y = x^2$：知道 x 就能確切知道 y，但因為關係對稱且非線性，共變異數為零。稍後介紹的互資訊能捕捉任何形式的相依。

共變異數矩陣

對平均為 $\boldsymbol{\mu} = \mathbb{E}[\mathbf{x}]$ 的隨機向量 $\mathbf{x} \in \mathbb{R}^n$，**共變異數矩陣**為

$$\Sigma = \mathrm{Cov}(\mathbf{x}) = \mathbb{E}\big[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top\big], \qquad \Sigma_{ij} = \mathrm{Cov}(x_i, x_j).$$

對角線是各分量的變異數 $\Sigma_{ii} = \mathrm{Var}(x_i)$。$\Sigma$ 是對稱半正定矩陣，因此（見ML 01A）具有非負實特徵值與正交特徵向量。特徵向量是資料雲的主軸，特徵值是沿主軸的變異數——也就是圖 1 中的紅色箭頭——這正是 PCA 所計算的東西。對仿射映射 $\mathbf{y} = A\mathbf{x} + \mathbf{b}$，有 $\mathbb{E}[\mathbf{y}] = A\boldsymbol{\mu} + \mathbf{b}$ 與 $\mathrm{Cov}(\mathbf{y}) = A\Sigma A^\top$。

常見機率分布

白話版。有一小群分布會一再出現。選擇其中一個，就是對資料或目標做出一個假設。

Bernoulli 分布

單一二元變數 $X \in \{0, 1\}$，參數 $\phi \in [0, 1]$：

$$P(X = 1) = \phi, \quad P(X = 0) = 1 - \phi, \quad P(X = x) = \phi^x (1 - \phi)^{1 - x}.$$

平均為 ϕ，變異數為 $\phi(1 - \phi)$。輸出層使用 sigmoid 的二元分類器，就是在預測標籤之 Bernoulli 分布的參數 ϕ。

類別分布（multinoulli）

具有 K 個狀態的單一離散變數，以機率向量 $\mathbf{p} \in [0,1]^K$（$\sum_k p_k = 1$）為參數：

$$P(X = k) = p_k.$$

若標籤以 one-hot 編碼為 $\mathbf{y} \in \{0,1\}^K$，則 $P(\mathbf{y}) = \prod_k p_k^{y_k}$。softmax 分類器輸出的正是這樣的向量。（多項分布計算多次抽取的結果次數；只抽一次時就退化成類別分布，Goodfellow 等人稱之為「multinoulli」[1]。）

高斯（常態）分布

最常用的連續分布：

$$\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\!\Big(-\frac{(x-\mu)^2}{2\sigma^2}\Big).$$

μ 是平均（也是峰值位置），σ^2 是變異數。有時改用**精確度**（precision）$\beta = 1/\sigma^2$，在變異數經常改變時計算起來比較省。

高斯之所以成為預設選擇有兩個原因。**中央極限定理**說，許多變異數有限的獨立變數相加，會趨近高斯分布，所以由許多微小因素組成的量測雜訊往往接近高斯。而在實數線上、平均與變異數固定的所有分布中，高斯的**熵最大**：它加入的額外結構最少。

在 n 維中，

$$\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \Sigma) = \frac{1}{\sqrt{(2\pi)^n \det \Sigma}} \exp\!\Big(-\tfrac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{x}-\boldsymbol{\mu})\Big),$$

其中 $\boldsymbol{\mu} \in \mathbb{R}^n$ 是平均向量，$\Sigma \in \mathbb{R}^{n \times n}$ 是對稱正定的共變異數矩陣。**精確度矩陣** $\Lambda = \Sigma^{-1}$ 有個漂亮的意義：$\Lambda_{ij} = 0$ 恰好代表在給定其他所有座標下，x_i 與 x_j 條件獨立。**等向**（isotropic）高斯的 $\Sigma = \sigma^2 I$——這正是擴散模型使用的雜訊分布（ML 07）。

它和你會用到的損失函數有一個根本的連結：如果迴歸網路預測一個固定變異數之高斯的平均，那麼最大化概似就等於最小化**均方誤差**。

指數分布與 Laplace 分布

有時候我們想在特定位置放一個尖峰。

$$p(x; \lambda) = \lambda\, \mathbf{1}_{x \ge 0}\, e^{-\lambda x}, \qquad \text{Laplace}(x; \mu, \gamma) = \frac{1}{2\gamma} \exp\!\Big(-\frac{|x-\mu|}{\gamma}\Big).$$

$\lambda > 0$ 是速率，$\mathbf{1}_{x \ge 0}$ 是指示函數（$x \ge 0$ 時為 1，否則為 0），μ 是位置，$\gamma > 0$ 是尺度。和高斯相比，Laplace 密度的峰值更尖、尾巴更厚（圖 1 左下）。用最大概似預測 Laplace 分布的位置，等同於最小化**絕對誤差**（L_1 損失），這就是 L_1 比 L_2 更能容忍離群值的原因；而對權重使用 Laplace 先驗，就得到 L_1 正則化（稀疏性）。

Dirac 分布與經驗分布

Dirac delta $\delta(x-\mu)$ 把所有質量放在點 μ：它在其他地方都是零，積分卻等於一。它是一種廣義函數，由它在積分中的作用來定義：$\int \delta(x-\mu) f(x)\,dx = f(\mu)$。

訓練集 $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ 定義了**經驗分布**

$$\hat{p}(\mathbf{x}) = \frac{1}{m}\sum_{i=1}^{m} \delta(\mathbf{x}-\mathbf{x}^{(i)}),$$

在 m 個樣本上各放 $1/m$ 的質量。把損失對整個資料集取平均來訓練模型，就等於在 $\hat{p}$ 下取期望值。經驗分布也是讓訓練資料概似最大的分布——這正是只最小化訓練損失會過擬合（overfitting）的原因。

混合分布與潛在變數

白話版。混合分布先隨機選一個「類型」，再從那個類型的分布中抽樣。

$$P(\mathbf{x}) = \sum_{k=1}^{K} P(z=k)\, P(\mathbf{x} \mid z=k).$$

z 是一個類別型的**潛在變數**（latent variable）——我們從未直接觀察到的變數——以機率 $\pi_k = P(z=k)$ 選出，$P(\mathbf{x} \mid z=k)$ 是第 k 個成分。**高斯混合模型**（Gaussian mixture model, GMM）使用高斯成分 $\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \Sigma_k)$；只要成分夠多，它可以把任何平滑密度近似到任意指定的非零誤差 [1]。給定一個觀測 $\mathbf{x}$，貝氏定理給出**後驗責任值** $P(z=k \mid \mathbf{x})$：該點由第 k 個成分產生的可能性。用期望最大化（EM）擬合 GMM 是PR 07 的主題。

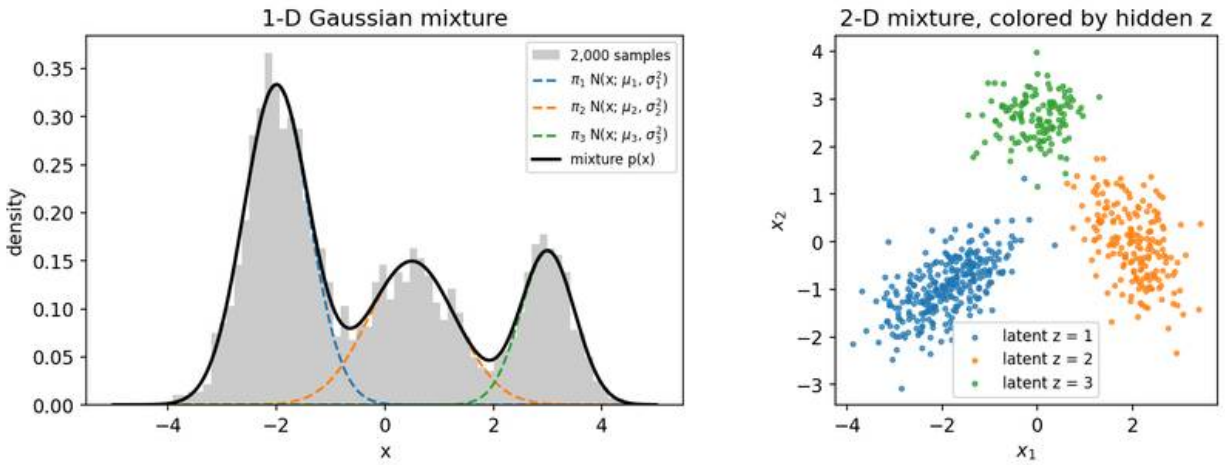


圖 2 — 高斯混合。左：黑色密度是三個成分的加權總和；灰色直方圖是先選成分、再從該成分抽樣得到的 2,000 個樣本。右：在二維中，顏色代表隱藏成分 z——真實資料集並不會告訴你這個資訊。

潛在變數是現代生成模型的核心。VAE（ML 06）粗略地說，就是潛在變數 $\mathbf{z}$ 為連續、且 $p(\mathbf{x} \mid \mathbf{z})$ 由神經網路表示的混合模型 [8]；此時邊際分布 $p(\mathbf{x}) = \int p(\mathbf{x} \mid \mathbf{z}) p(\mathbf{z})\, d\mathbf{z}$ 無法直接計算，只能近

似。

```
import numpy as np

rng = np.random.default_rng(0)
pi = np.array([0.5, 0.3, 0.2])      # mixing weights P(z = k)
mu = np.array([-2.0, 0.5, 3.0])     # component means
sd = np.array([0.6, 0.8, 0.5])      # component standard deviations

# Ancestral sampling: first the hidden component z, then x given z
z = rng.choice(3, size=100_000, p=pi)
x = rng.normal(mu[z], sd[z])

def log_normal(x, m, s):
    return -0.5*((x - m)/s)**2 - np.log(s) - 0.5*np.log(2*np.pi)

def mixture_logpdf(x):
    # log sum_k pi_k N(x; mu_k, sd_k^2), computed with log-sum-exp
    a = np.log(pi) + log_normal(x[:, None], mu, sd)
    m = a.max(axis=1, keepdims=True)
    return (m + np.log(np.exp(a - m).sum(axis=1, keepdims=True))).ravel()

grid = np.linspace(-6, 6, 4001)
print("density integrates to",
 ↪  (np.exp(mixture_logpdf(grid)).sum()*(grid[1]-grid[0])).round(4))
print("sample mean", x.mean().round(3), "vs exact", (pi*mu).sum().round(3))
# Posterior responsibility P(z | x) for one observation
a = np.log(pi) + log_normal(np.array([0.0])[:, None], mu, sd)
resp = np.exp(a - a.max()); resp /= resp.sum()
print("P(z | x=0):", resp.round(3))
```

密度積分為 1.0，樣本平均（-0.255）與精確平均 $\sum_k \pi_k \mu_k = -0.25$ 相符，而點 $x = 0$ 以 0.99 的機率被歸給中間的成分。這裡用到的 log-sum-exp 技巧在ML 01C 中說明。

實用函數：sigmoid 與 softplus

白話版。 sigmoid 把任何實數壓進 0 到 1 之間，變成一個機率；softplus 是 $\max(0, x)$ 的平滑版本，輸出永遠為正。

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \qquad \zeta(x) = \log\left(1 + e^{x}\right).$$

logistic sigmoid σ 產生 Bernoulli 分布的參數 ϕ；當 $|x|$ 很大時它會**飽和**（變平），使那裡的梯度很小。**softplus** ζ 產生正數，例如標準差 σ 或精確度 β；它的名字來自它是「正部」$x^{+} = \max(0, x)$ 的柔化版本。

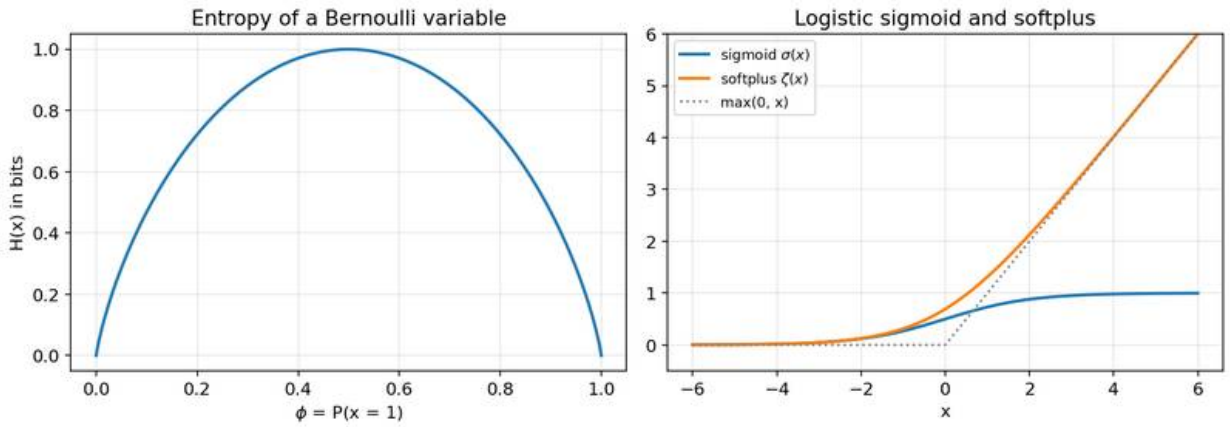


圖 3 — 左：*Bernoulli* 變數的熵，在 $\varphi = 0.5$ 時最大（1 位元），結果確定時為零。右：*logistic sigmoid* $\sigma(x)$ 與 *softplus* $\zeta(x)$，後者是 $max(0, x)$ 的平滑版本。

下列恆等式值得記住，因為每次化簡損失函數時都會用到 [1]：

$$1 - \sigma(x) = \sigma(-x), \qquad \frac{d}{dx}\sigma(x) = \sigma(x)(1 - \sigma(x)),$$

$$\log \sigma(x) = -\zeta(-x), \qquad \frac{d}{dx}\zeta(x) = \sigma(x),$$

$$\sigma^{-1}(p) = \log \frac{p}{1-p}, \qquad \zeta(x) - \zeta(-x) = x.$$

σ^{-1} 是 **logit**（對數勝算）函數。恆等式 $\log \sigma(x) = -\zeta(-x)$ 說明了為什麼二元交叉熵應該從 logit 直接計算：$-\log \sigma(x) = \zeta(-x)$，而 softplus 可以在不溢位的情況下求值。

```
import numpy as np

def sigmoid(x):
    return 1/(1 + np.exp(-x))

def softplus(x):
    return np.logaddexp(0, x)          # log(1 + exp(x)) without overflow

x = np.linspace(-30, 30, 13)
assert np.allclose(1 - sigmoid(x), sigmoid(-x))
assert np.allclose(np.log(sigmoid(x)), -softplus(-x))
assert np.allclose(softplus(x) - softplus(-x), x)
h = 1e-6
assert np.allclose((softplus(x+h) - softplus(x-h))/(2*h), sigmoid(x), atol=1e-6)
p = sigmoid(x[4:9])
assert np.allclose(np.log(p/(1-p)), x[4:9])   # logit inverts sigmoid
print("naive log(1+exp(1000)):", np.log1p(np.exp(np.float64(1000))))
print("stable softplus(1000):", softplus(1000.0))
```

所有恆等式都通過檢查。直接計算的寫法傳回 `inf`（並出現溢位警告），而 `np.logaddexp` 傳回 1000.0。

貝氏定理

白話版。貝氏定理把「假設成立時，看到這個證據的可能性有多大？」轉換成「看到證據之後，這個假設有多可能？」——並且堅持你必須考慮這個假設原本有多常見。

把乘法規則從兩個方向寫出來，$P(a, b) = P(a \mid b)P(b) = P(b \mid a)P(a)$，於是

$$P(a \mid b) = \frac{P(b \mid a)\,P(a)}{P(b)}, \qquad P(b) = \sum_{a'} P(b \mid a')\,P(a').$$

- $P(a)$ 是**先驗**（prior）：看到證據之前 a 的可信度。
- $P(b \mid a)$ 是在假設 a 下證據 b 的**概似**（likelihood）。
- $P(a \mid b)$ 是**後驗**（posterior）：更新後的可信度。
- $P(b)$ 是**證據**（evidence，或稱邊際概似），對所有假設 a' 加總得到的正規化常數。

實例：基本比率效應

考慮一個假想的瑕疵偵測器，**靈敏度** $P(+ \mid \text{defect}) = 0.9$、**偽陽性率** $P(+ \mid \text{no defect}) = 0.05$，而**盛行率**（基本比率）$P(\text{defect}) = 0.01$。被判為陽性的物品真的有瑕疵的機率是

$$P(\text{defect} \mid +) = \frac{0.9 \cdot 0.01}{0.9 \cdot 0.01 + 0.05 \cdot 0.99} \approx 0.154.$$

即使偵測器能抓到 90% 的瑕疵，警報中只有約 15% 是真的。用數量來想就很清楚：在 10,000 件物品中，有 100 件有瑕疵，其中 90 件觸發警報；但另外 9,900 件正常品中，也有 495 件觸發警報（圖 4 右）。同一個偵測器在 10% 盛行率下則得到 $0.09/(0.09 + 0.045) = 2/3$。這是數學示範，不是任何真實系統的效能宣稱。它的教訓適用於每一個部署中的分類器：**光有靈敏度與偽陽性率，無法決定陽性預測有多可信；部署環境的盛行率同樣關鍵**。 ML 10 在討論分布偏移下的精確率時會再回到這一點。

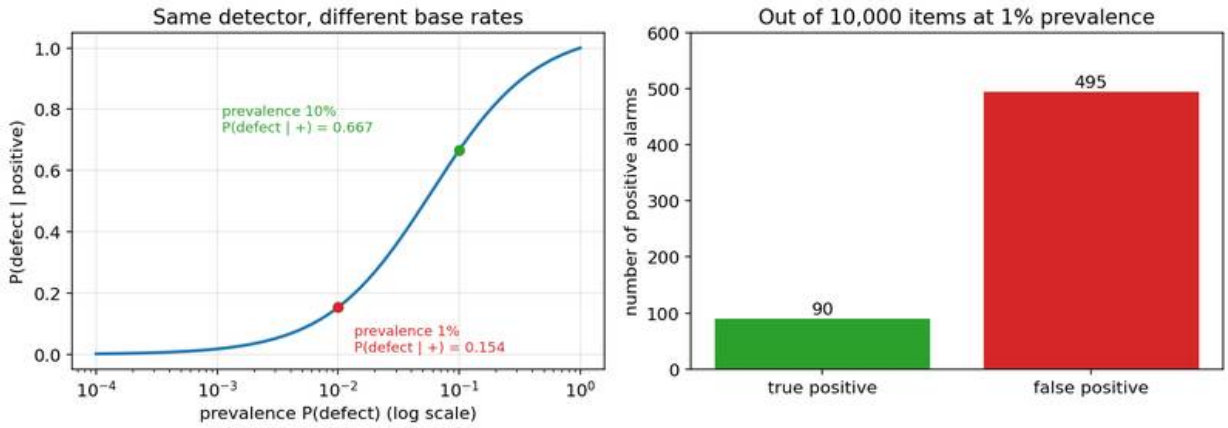


圖 4 — 假想偵測器（靈敏度 *0.9*、偽陽性率 *0.05*）的基本比率效應。左：後驗 $P(\textit{defect} \mid +)$ 強烈依賴盛行率。右：在 *1%* 盛行率下，假警報比真警報多出五倍以上。

```
import numpy as np

def posterior_positive(prevalence, sensitivity=0.9, false_pos=0.05):
    evidence = sensitivity*prevalence + false_pos*(1 - prevalence)   # P(+)
    return sensitivity*prevalence / evidence                          # P(defect | +)

for prev in [0.001, 0.01, 0.1, 0.5]:
    print(f"prevalence {prev:>5}: P(defect | +) = {posterior_positive(prev):.3f}")

# Simulation check with 1,000,000 synthetic items
rng = np.random.default_rng(0)
defect = rng.random(1_000_000) < 0.01
alarm = np.where(defect, rng.random(defect.size) < 0.9, rng.random(defect.size) < 0.05)
print("simulated:", defect[alarm].mean().round(3))
```

公式對四種盛行率分別給出 0.018、0.154、0.667 與 0.947，而一百萬件物品的模擬重現了 0.154。

貝氏定理也是**貝氏分類器**的基礎：預測後驗 $P(y = k \mid \mathbf{x}) \propto p(\mathbf{x} \mid y = k) P(y = k)$ 最大的類別 k，可以讓期望錯誤率最小。PR 01 會深入發展這套決策理論。

密度的變數變換

白話版。拉伸一張橡皮布，上面的墨水會變淡。隨機變數經過變換後，它的密度也必須依照變換把空間拉伸了多少來重新縮放。

一個誘人的錯誤：若 $y = g(x)$、x 的密度為 p_x，就以為 $p_y(y) = p_x(g^{-1}(y))$。這是錯的，因為它的積分不等於一。對可逆且可微的 g，一維的正確規則是

$$p_y(y) = p_x(g^{-1}(y)) \left| \frac{d\, g^{-1}(y)}{dy} \right|.$$

小區間內的機率必須守恆：$|p_y(y)\, dy| = |p_x(x)\, dx|$。在 n 維中，若 $\mathbf{y} = g(\mathbf{x})$，則

$$p_x(\mathbf{x}) = p_y(g(\mathbf{x})) \left| \det \frac{\partial g(\mathbf{x})}{\partial \mathbf{x}} \right|,$$

其中 $\partial g / \partial \mathbf{x}$ 是 g 的 $n \times n$ **Jacobian 矩陣**（第 (i, j) 個元素為 $\partial y_i / \partial x_j$），$|\det(\cdot)|$ 是其行列式的絕對值——也就是 g 放大小體積的倍率（行列式見ML 01A）。這個公式是**正規化流**（normalizing flows）的全部基礎：它堆疊 Jacobian 行列式容易計算的可逆網路，得到精確的概似 [9]。

```
import numpy as np

rng = np.random.default_rng(0)
x = rng.normal(0, 1, 500_000)        # x ~ N(0, 1)
y = np.exp(x)                        # y = g(x), invertible on (0, inf)

def p_x(x):
```

```
    return np.exp(-0.5*x**2)/np.sqrt(2*np.pi)

def p_y(y):
    # p_y(y) = p_x(g^{-1}(y)) * |d g^{-1}(y) / dy| = p_x(log y) / y
    return p_x(np.log(y))/y

counts, edges = np.histogram(y, bins=np.linspace(0.05, 4, 80), density=False)
centers = 0.5*(edges[:-1] + edges[1:])
empirical = counts/(y.size*np.diff(edges))
print("max |histogram - formula|:", np.abs(empirical - p_y(centers)).max().round(3))
print("wrong formula (no Jacobian) error:", np.abs(empirical -
 ↪  p_x(np.log(centers))).max().round(3))
```

加上 Jacobian 因子 $1/y$ 後，直方圖與公式的差距在 0.011 以內（抽樣雜訊）；少了它，誤差達到 0.46。

概似、最大概似與 MAP

白話版。固定已觀察到的資料，問哪些參數值最能讓這些資料顯得合理。如果你對參數另有先驗知識，也把它一併考慮進去。

假設模型 $p_{\boldsymbol{\theta}}(\mathbf{x})$ 的參數為 $\boldsymbol{\theta}$，並觀察到由獨立同分布（i.i.d.）樣本組成的資料集 $\mathcal{D} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$。**概似**為

$$L(\boldsymbol{\theta}) = p_{\boldsymbol{\theta}}(\mathcal{D}) = \prod_{i=1}^{m} p_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}).$$

作為 $\boldsymbol{\theta}$ 的函數，它不是參數上的機率分布：它對 $\boldsymbol{\theta}$ 的積分不必等於一。**最大概似估計**（maximum likelihood estimate, MLE）為

$$\boldsymbol{\theta}_{\mathrm{ML}} = \arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{m} \log p_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) = \arg\min_{\boldsymbol{\theta}} \; \mathbb{E}_{\mathbf{x}\sim\hat{p}}[-\log p_{\boldsymbol{\theta}}(\mathbf{x})].$$

我們取對數，因為許多小於一的數相乘會下溢，而對數相加在數值上很安全，且最大值位置不變。右邊的形式說明 MLE 是在經驗分布 $\hat{p}$ 下**最小化平均負對數概似**（negative log-likelihood, NLL）。對分類器 $p_{\boldsymbol{\theta}}(y \mid \mathbf{x})$，同樣的想法給出條件 NLL $-\log p_{\boldsymbol{\theta}}(y \mid \mathbf{x})$，也就是下面會看到的交叉熵損失。

兩個經典結果：n 次擲硬幣得到 k 次正面時，$\phi_{\mathrm{ML}} = k/n$；對高斯而言，μ_{ML} 是樣本平均，$\sigma^2_{\mathrm{ML}} = \frac{1}{m}\sum_i (x^{(i)} - \mu_{\mathrm{ML}})^2$——這是略帶偏差的估計（改除以 $m-1$ 可以消除偏差）。

MAP 估計

貝氏推論把 $\boldsymbol{\theta}$ 本身視為隨機變數，給它一個**先驗** $p(\boldsymbol{\theta})$，再計算**後驗**

$$p(\boldsymbol{\theta} \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta})}{p(\mathcal{D})} \;\propto\; p(\mathcal{D} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta}).$$

完整的後驗往往難以計算，常見的捷徑是取它的眾數，即**最大後驗**（maximum a posteriori, MAP）估計：

$$\boldsymbol{\theta}_{\text{MAP}} = \arg\max_{\boldsymbol{\theta}} \left[\log p(\mathcal{D} \mid \boldsymbol{\theta}) + \log p(\boldsymbol{\theta}) \right].$$

若使用高斯先驗 $p(\boldsymbol{\theta}) = \mathcal{N}(\mathbf{0}, \frac{1}{\lambda} I)$，則 $-\log p(\boldsymbol{\theta})$ 等於 $\frac{\lambda}{2}\|\boldsymbol{\theta}\|_2^2$ 加上常數：**權重衰減就是使用高斯先驗的 MAP**。Laplace 先驗則給出 L_1 正則化。這把機率假設和ML 02 中的正則化項連結起來。但 MAP 點估計仍然只是一個參數向量，無法告訴你參數有多不確定。近似整個後驗是PR 08 與變分推論 [10] 的主題。

```
import numpy as np

rng = np.random.default_rng(0)
flips = rng.random(10) < 0.7          # 10 coin flips, true P(heads) = 0.7
k, n = flips.sum(), flips.size

phi = np.linspace(1e-3, 1 - 1e-3, 999)
log_lik = k*np.log(phi) + (n - k)*np.log(1 - phi)
a, b = 5.0, 5.0                        # Beta(5, 5) prior, centered on 0.5
log_prior = (a - 1)*np.log(phi) + (b - 1)*np.log(1 - phi)

print(f"{k} heads in {n} flips")
print("MLE  grid:", phi[log_lik.argmax()].round(3), " closed form:", k/n)
print("MAP  grid:", phi[(log_lik + log_prior).argmax()].round(3),
      " closed form:", round((k + a - 1)/(n + a + b - 2), 3))

# Gaussian MLE: sample mean and the 1/m (biased) variance
x = rng.normal(2.0, 3.0, 50)
print("Gaussian MLE mean, var:", x.mean().round(3), x.var().round(3),
      " unbiased var:", x.var(ddof=1).round(3))
```

在這個隨機種子下，10 次擲出 6 次正面。MLE 為 0.6；Beta(5, 5) 先驗把 MAP 估計拉向 0.5，得到 0.556，與閉合解 $(k + a - 1)/(n + a + b - 2)$ 一致。資料越多，概似就越占主導，兩種估計會逐漸接近。高斯的 MLE 變異數（7.71）小於不偏估計（7.87），符合預期。

資訊理論

白話版。資訊理論衡量驚訝程度。罕見的事件一旦發生，帶來大量資訊；確定的事件則毫無資訊。把驚訝程度取平均就是熵；使用錯誤分布所多出的驚訝，就是 KL 散度。

Claude Shannon 為了研究透過有雜訊的通道傳送訊息需要多少位元而創立了這個領域 [5]；標準教科書是 Cover 與 Thomas [7]。對機器學習而言，最有用的概念是量化兩個分布有多不同。

自資訊與熵

結果 x 的**自資訊**（self-information）為

$$I(x) = -\log P(x).$$

使用自然對數時單位為 **nat**；使用 $\log_2$ 時單位為**位元**（bit）（$1\ \text{nat} = 1/\ln 2 \approx 1.443$ 位元）。確定事件（$P = 1$）的資訊量為零；不太可能的事件資訊量很大；而獨立事件的資訊可以相加，因為對數把乘積變成總和。

Shannon 熵是自資訊的期望值：

$$H(P) = \mathbb{E}_{x\sim P}[I(x)] = -\sum_{x} P(x)\log P(x).$$

我們採用 $0\log 0 = 0$ 的約定，理由是 $p \to 0$ 時 $p\log p \to 0$。熵是編碼從 P 抽出之結果所需的最少平均位元數（或 nat 數）。決定性結果的熵為零；K 個結果的均勻分布熵最大，為 $\log K$。參數為 ϕ 的 Bernoulli 變數，$H = -\phi\log\phi - (1-\phi)\log(1-\phi)$，在 $\phi = 0.5$ 時達到 1 位元的峰值（圖 3 左）。

對連續變數，$h(p) = -\int p(x)\log p(x)\,dx$ 稱為**微分熵**。使用時要小心：它可以是負的（$[0, \tfrac{1}{2}]$ 上的均勻分布 $h = -\log 2$），而且座標尺度一改它就跟著改。它並不是離散位元數的極限版本。

KL 散度

Kullback–Leibler 散度 [6] 衡量分布 Q 與參考分布 P 有多不同：

$$D_{\mathrm{KL}}(P\,\|\,Q) = \mathbb{E}_{x\sim P}\Big[\log\frac{P(x)}{Q(x)}\Big] = \sum_{x} P(x)\log\frac{P(x)}{Q(x)}.$$

用編碼的語言說，它是：當編碼是為 Q 設計、卻用來編碼 P 的樣本時，額外需要的 nat 數。它的主要性質：

- **非負**：$D_{\mathrm{KL}}(P\|Q) \geq 0$，等號成立若且唯若 $P = Q$（離散情況；連續情況為「幾乎處處」相等）。這來自 Jensen 不等式，因為 $-\log$ 是凸函數。
- **在無支撐處為無限大**：若某個 x 使 $P(x) > 0$ 但 $Q(x) = 0$，散度為無限大。
- **不對稱**：一般而言 $D_{\mathrm{KL}}(P\|Q) \neq D_{\mathrm{KL}}(Q\|P)$，而且不滿足三角不等式。因此 KL 不是距離度量，儘管它常被描述成「分布之間的距離」。

正向與反向 KL

當近似用的分布族太簡單時，這種不對稱會產生看得見的後果。圖 5 用單一高斯 q 去擬合有兩個分得很開之峰的目標 p。

- 最小化**正向 KL** $D_{\mathrm{KL}}(p\|q)$ 時，若 q 在 p 有質量的地方很小，就會受到重罰。因此最佳高斯會**涵蓋兩個峰**，把平均放在兩峰之間（$\mu = 0$、$\sigma \approx 2.62$），並把大量質量放在空蕩的谷底。對高斯 q 而言，正向 KL 的最佳解就是匹配 p 的平均與變異數。
- 最小化**反向 KL** $D_{\mathrm{KL}}(q\|p)$ 時，若 q 在 p 很小的地方很大，就會受到重罰。因此最佳高斯會**鎖定其中一個峰**（$\mu \approx 2.49$、$\sigma \approx 0.81$），忽略另一個。

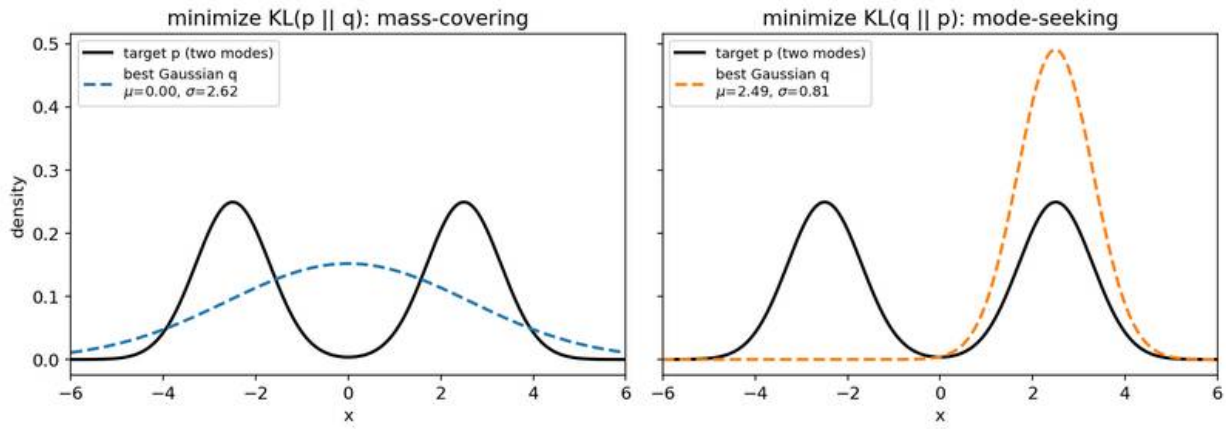


圖 5 — *KL* 不對稱。用單一高斯 q 擬合雙峰目標 p：正向 $KL(p\|q)$ 得到以兩峰中間為中心、涵蓋質量的擬合；反向 $KL(q\|p)$ 得到只落在一個峰上、尋找眾數的擬合（落在另一個峰上是同樣好的鏡像解）。

要最小化哪一個方向，是設計上的選擇。**最大概似最小化的是從資料分布到模型的正向 KL**，因此以概似訓練的模型（自迴歸模型、正規化流、VAE）傾向把機率分散到所有資料上，有時會產生模糊的樣本。**變分推論通常最小化從近似分布到真實後驗的反向 KL** [10]，因此傾向低估後驗的不確定性。

交叉熵

Q 相對於 P 的**交叉熵**為

$$H(P,Q) = -\mathbb{E}_{x\sim P}[\log Q(x)] = H(P) + D_{\mathrm{KL}}(P\,\|\,Q).$$

因為 $H(P)$ 與 Q 無關，對 Q 最小化交叉熵就等於最小化正向 KL。在分類中，P 是經驗標籤分布——每個樣本是一個 one-hot 向量——而 Q 是模型的 softmax 輸出 $\hat{\mathbf{y}}$。對真實類別為 y 的單一樣本，

$$H(\mathbf{e}_y, \hat{\mathbf{y}}) = -\sum_k [\mathbf{e}_y]_k \log \hat{y}_k = -\log \hat{y}_y,$$

其中 $\mathbf{e}_y$ 是類別 y 的 one-hot 向量。對整個資料集取平均，這正是條件負對數概似。所以「交叉熵損失」、「對數損失」與「類別模型的負對數概似」是同一個目標的三個名字。任何由負對數概似構成的損失，都是經驗分布與模型之間的交叉熵——這也是為什麼均方誤差同樣是高斯模型的「交叉熵」[1]。

互資訊

互資訊衡量知道一個變數能減少多少對另一個變數的不確定性：

$$I(X;Y) = D_{\mathrm{KL}}(p(x,y)\,\|\,p(x)p(y)) = H(X) - H(X\mid Y) = H(X) + H(Y) - H(X,Y).$$

其中 $H(X \mid Y) = -\mathbb{E}_{x,y}[\log p(x \mid y)]$ 是條件熵，$H(X,Y)$ 是聯合熵。$I(X;Y) \geq 0$，等號成立若且唯若 X 與 Y 獨立——和共變異數不同，它能偵測任何形式的相依。它對 X 與 Y 是對稱的。

程式碼：資訊量

```
import numpy as np

p = np.array([.7, .2, .1])
q = np.array([.5, .3, .2])
entropy = -(p*np.log(p)).sum()
cross_entropy = -(p*np.log(q)).sum()
kl = (p*np.log(p/q)).sum()
reverse_kl = (q*np.log(q/p)).sum()
assert np.allclose(cross_entropy - entropy, kl) and kl >= 0
print("H(p), H(p,q), KL(p||q), KL(q||p) in nats:",
      np.round([entropy, cross_entropy, kl, reverse_kl], 4))
print("H(p) in bits:", (entropy/np.log(2)).round(4))

# Mutual information from a joint table (same weather/umbrella table)
P = np.array([[0.56, 0.14],
              [0.06, 0.24]])
px, py = P.sum(1), P.sum(0)
mi = (P*np.log(P/np.outer(px, py))).sum()
H = lambda t: -(t[t > 0]*np.log(t[t > 0])).sum()
assert np.isclose(mi, H(px) + H(py) - H(P.ravel()))
print("I(weather; umbrella) =", mi.round(4), "nats")

# Cross-entropy loss on one-hot labels equals mean -log q(y)
probs = np.array([[0.7, 0.2, 0.1], [0.1, 0.8, 0.1], [0.3, 0.3, 0.4]])
labels = np.array([0, 1, 2])
print("mean cross-entropy:", (-np.log(probs[np.arange(3), labels])).mean().round(4))
```

輸出：$H(p) = 0.8018$、$H(p,q) = 0.8869$、$D_{\mathrm{KL}}(p\|q) = 0.0851$、$D_{\mathrm{KL}}(q\|p) = 0.0920$ nats——兩個方向的值不同。$H(p)$ 為 1.1568 位元。天氣與帶傘的互資訊為 0.1637 nats，兩種公式算出的結果一致。三筆預測的平均交叉熵為 0.4987 nats。在真正的訓練程式中，請用融合函數直接從 logit 計算交叉熵，而不是先算機率（見ML 01C）。

結構化機率模型

白話版。多個變數的聯合分布大到無法寫成表格。結構化模型畫一張圖，標明哪些變數直接影響哪些變數，聯合分布就能拆成許多小塊。

100 個二元變數的聯合機率表需要 $2^{100} - 1$ 個數字。**結構化機率模型**，又稱**圖模型**（graphical models），利用條件獨立把聯合分布分解成小因子 [1], [3]。主要有兩類。

有向模型（貝氏網路）使用有向無環圖 $\mathcal{G}$。每個變數只依賴它的父節點 $Pa_{\mathcal{G}}(x_i)$：

$$p(\mathbf{x}) = \prod_i p(x_i \mid Pa_{\mathcal{G}}(x_i)).$$

例如在 $a \to b \to c$ 且 $a \to c$ 時，$p(a,b,c) = p(a)\,p(b \mid a)\,p(c \mid a,b)$；刪掉邊 $a \to c$ 就是宣稱 $c \perp a \mid b$，最後一個因子縮短為 $p(c \mid b)$。有向圖很適合描述生成故事（「先選類別，再產生像素」），而混合模型 $z \to \mathbf{x}$ 是最簡單的例子。

無向模型（馬可夫隨機場）使用無向圖。每個**團**（clique）$C^{(i)}$（彼此全部相連的節點集合）有一個非負的**因子** $\phi^{(i)}$，且

$$p(\mathbf{x}) = \frac{1}{Z} \prod_i \phi^{(i)}(C^{(i)}), \qquad Z = \sum_{\mathbf{x}} \prod_i \phi^{(i)}(C^{(i)}).$$

配分函數 Z 用來正規化乘積；計算它往往是最困難的部分。無向模型適合對稱的交互作用，例如影像先驗中相鄰像素的關係，或用於分割的條件隨機場。

圖是對分布的一種描述，不是另一種分布：任何分布都可以用兩種方式寫出，只是其中一種可能精簡得多。深度學習常隱含地使用這些概念——自迴歸模型是全連接的有向模型，VAE 則是條件分布由神經網路表示的雙節點有向模型 $\mathbf{z} \to \mathbf{x}$ [8]。

現代觀點

本篇的機率與資訊理論是古典內容。深度學習時代改變的是它如何被使用，以及實務上會在哪裡出錯。本節綜覽相關綜述文獻與近年方向；只整理它們的框架，不轉述基準數字。

校準：預測機率誠實嗎？

如果分類器在所有預測信心為 0.8 的輸入中，大約有 80% 是正確的，就稱它是**校準良好**的（calibrated）。Guo 等人 [11]（ICML 2017）發現現代深度網路常常**過度自信**，並指出深度、寬度、權重衰減與批次正規化都會影響校準。他們主要推薦的**溫度縮放**（temperature scaling）——把所有 logit 除以一個在驗證集上擬合的純量 T——是 Platt scaling 的單參數版本；因為它只改變信心、不改變預測類別，至今仍是強力的基準方法。Wang 的綜述 [12] 把校準方法分為四大類：事後校準、訓練期間的正則化、不確定性估計，以及這些方法的組合；其修訂版也涵蓋大型語言模型的校準。從機率的角度看，教訓是：softmax 輸出是 $P(y \mid \mathbf{x})$ 的一個模型，和任何模型一樣，它的不確定性也可能是錯的，而不只是 argmax 會錯。

不確定性量化

兩篇主要綜述描繪了深度學習中不確定性研究的全貌。Abdar 等人 [13]（*Information Fusion*，2021）回顧了貝氏近似、集成方法以及它們在許多領域的應用。Gawlikowski 等人 [14]（*Artificial Intelligence Review*，2023）區分了**可降低的模型（認知，epistemic）不確定性**與**不可降低的資料（偶然，aleatoric）不確定性**——分別對應本篇開頭所說的第三與第一種不確定性來源——並比較了確定性單一網路、貝氏方法、集成與測試時資料擴增，以及校準的量測方式。

有兩篇原始研究特別有影響力。**深度集成**（deep ensembles）[15] 從不同的隨機初始化訓練多個網路，再平均它們的預測分布；這個簡單、可平行化的方法與近似貝氏神經網路表現相當，並且在分布外（out-of-distribution）輸入上表現出較高的不確定性。Ovadia 等人 [16]（NeurIPS 2019）

在**資料集偏移**下評測不確定性方法，發現以分布內驗證資料做的事後校準難以延續，而對多個模型做邊際化的方法（如集成）表現較穩定。

另一種假設較少的互補方法是**共形預測**（conformal prediction）。Angelopoulos 與 Bates [17] 提供了一份淺顯的入門：利用一組保留的校準資料，共形方法能把任何預訓練模型的分數轉成預測集合，並以使用者指定的機率包含真實標籤，前提是校準資料與測試資料可交換（exchangeable）。它不要求模型機率本身已經校準，因此對被當成黑盒子使用的視覺與語言系統很有吸引力。

表示學習中的互資訊

互資訊曾是自監督學習（ML 05）中熱門的目標。對比預測編碼（Contrastive Predictive Coding）[18] 提出了 **InfoNCE** 損失：它在多個負樣本中辨認出正樣本對，可以證明是在最大化互資訊的一個下界。Poole 等人 [19]（ICML 2019）統一了許多變分互資訊界，並指出當真實互資訊很大時這些界會退化，在高偏差與高變異數之間取捨。Tschannen 等人 [20]（ICLR 2020）主張這些方法的成功「不能只歸因於互資訊本身的性質」：編碼器架構與估計器的參數化至少同樣重要。現在的共識是把 InfoNCE 類的目標視為有資訊理論動機、行為良好的對比損失，而不是精確的互資訊估計器。

生成模型中的散度與潛在變數

生成建模就是應用機率。VAE [8] 最大化證據下界（evidence lower bound），它結合了重建概似與一個 KL 項；Blei 等人 [10] 回顧了變分推論，即在一個容易處理的分布族中最小化（反向）KL 來近似難解的後驗。正規化流 [9] 完全建立在變數變換公式上。擴散模型與流匹配模型（ML 07）結合了高斯雜訊、條件期望值與概似界。選擇哪一種散度（正向 KL、反向 KL 或其他）決定了模型是涵蓋全部資料，還是集中在少數眾數上——與圖 5 是同一幅圖像，只是規模大得多。

尚待解決的問題

- **分布偏移下的校準**。在驗證集上成立的校準，換到新的相機、醫院或提示詞時可能失效 [16]；如何偵測這種情況仍是開放問題。
- **區分不確定性類型**。把預測不確定性拆成認知與偶然兩部分，在概念上很清楚，但對大型網路很難可靠地做到 [14]。
- **高維資訊量的估計**。變分互資訊界有根本的偏差–變異數限制 [19]，因此對深度表示做資訊理論宣稱時需要格外謹慎。
- **基礎模型的不確定性**。把校準與共形方法應用到輸出開放式內容的大型視覺–語言模型與語言模型，是活躍的研究方向 [12], [17]。

重點整理

- 機率描述來自隨機性、部分觀測與簡化模型的不確定性；頻率學派與貝氏學派的解讀遵守同一套規則。
- PMF 給出機率；PDF 給出密度，而密度可以大於一。連續事件的機率是面積。

- 加法規則（邊際化）、乘法規則（條件化）與連鎖規則都是精確的；獨立與條件獨立則是建模假設。
- 共變異數只衡量線性共變——共變異數為零不代表獨立；互資訊能偵測任何相依。
- 選擇分布就是選擇損失：Bernoulli／類別分布給出交叉熵，高斯給出平方誤差，Laplace 給出絕對誤差；高斯與 Laplace 先驗分別給出 L_2 與 L_1 正則化。
- 貝氏定理把證據與基本比率結合；同一個偵測器在某個盛行率下可信，在另一個盛行率下卻可能誤導。
- 變換隨機變數時，密度要乘上 Jacobian 行列式的絕對值來重新縮放。
- 交叉熵等於熵加 KL；最小化它就是最大概似。KL 不對稱：正向 KL 涵蓋各峰，反向 KL 只追一個峰。

練習

1. **單位與確定性**。 (a) 把 1 nat 的損失換算成位元。(b) 分別計算公平硬幣、確定結果與公平八面骰的熵（以位元為單位）。

   提示

   (a) 除以 $\ln 2$：約 1.443 位元。(b) 1 位元、0 位元與 $\log_2 8 = 3$ 位元。K 個結果的均勻分布熵為 $\log K$，是 K 個結果所能達到的最大值。

2. **盛行率偏移**。使用貝氏定理一節中的偵測器，找出讓陽性結果恰好像擲硬幣、即 $P(\text{defect} \mid +) = 0.5$ 的盛行率。

   提示

   令盛行率為 π，解 $0.9\pi = 0.05(1-\pi)$，得 $\pi = 0.05/0.95 \approx 0.053$，約 5.3%。低於這個值時，大多數警報都是假的。

3. **相依但不相關**。令 X 以相同機率取 $-1, 0, 1$，且 $Y = X^2$。證明 $\operatorname{Cov}(X, Y) = 0$，並以位元計算 $I(X; Y)$。

   提示

   $\mathbb{E}[X] = 0$ 且 $\mathbb{E}[XY] = \mathbb{E}[X^3] = 0$，所以共變異數為零。因為 Y 是 X 的函數，$H(Y \mid X) = 0$，於是 $I(X;Y) = H(Y)$。$Y = 1$ 的機率為 2/3、$Y = 0$ 的機率為 1/3，所以 $I \approx 0.918$ 位元。

4. **高斯 MLE 等於平方誤差**。證明在 σ 固定時，對 $\boldsymbol{\theta}$ 最大化 $\prod_i \mathcal{N}(y^{(i)}; f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), \sigma^2)$ 等同於最小化均方誤差。如果 σ 也由網路預測，會有什麼改變？

   提示

   負對數概似為 $\sum_i \frac{(y^{(i)} - f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}))^2}{2\sigma^2} + m \log \sigma + \text{const}$。$\sigma$ 固定時，只有平方誤差與 $\boldsymbol{\theta}$ 有關。如果網路也預測 $\sigma(\mathbf{x})$（例如透過 softplus），每個樣本的誤差會被 $1/\sigma^2$ 加權，而 $\log \sigma$ 懲罰項會防止變異數無限制地變大——這就是異質變異（heteroscedastic）迴歸模型。

5. **變數變換**。若 $X \sim \mathcal{U}(0,1)$（均勻分布）且 $Y = -\frac{1}{\lambda}\log X$，$\lambda > 0$，$Y$ 的密度是什麼？用 NumPy 抽樣驗證你的答案。

   提示

   $g^{-1}(y) = e^{-\lambda y}$ 且 $|dg^{-1}/dy| = \lambda e^{-\lambda y}$，而在 $(0,1)$ 上 $p_X = 1$。所以當 $y \geq 0$ 時 $p_Y(y) = \lambda e^{-\lambda y}$：指數分布。這就是反轉換抽樣（inverse-transform sampling）的技巧。

6. **KL 的方向**。對 $p = (0.5, 0.5, 0)$ 與 $q = (0.4, 0.4, 0.2)$，計算 $D_{\mathrm{KL}}(p\|q)$ 與 $D_{\mathrm{KL}}(q\|p)$。哪一個是無限大？這對以最大概似訓練模型有什麼啟示？

   提示

   $D_{\mathrm{KL}}(p\|q) = \log(0.5/0.4) \approx 0.223$ nats（第三項是 $0 \log 0 = 0$）。$D_{\mathrm{KL}}(q\|p) = \infty$，因為 q 在 p 沒有質量的地方放了質量。最大概似最小化的是 $D_{\mathrm{KL}}(p_{\text{data}}\|q_{\text{model}})$；只要模型對任何一筆觀察到的樣本給出零機率，它就會變成無限大——所以以概似訓練的模型必須在資料可能出現的每個地方都分配一些機率。

參考文獻

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 3 “Probability and Information Theory”. Book
2. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. Book
3. K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022. Book
4. M. P. Deisenroth, A. A. Faisal and C. S. Ong, *Mathematics for Machine Learning*, Cambridge University Press, 2020. Book
5. C. E. Shannon, “A Mathematical Theory of Communication,” *Bell System Technical Journal*, vol. 27, pp. 379–423, 1948. DOI
6. S. Kullback and R. A. Leibler, “On Information and Sufficiency,” *The Annals of Mathematical Statistics*, vol. 22, pp. 79–86, 1951. DOI
7. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006. DOI
8. D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *Proc. International Conference on Learning Representations (ICLR)*, 2014. arXiv
9. G. Papamakarios, E. Nalisnick, D. J. Rezende, S. Mohamed and B. Lakshminarayanan, “Normalizing Flows for Probabilistic Modeling and Inference,” *Journal of Machine Learning Research*, vol. 22, no. 57, pp. 1–64, 2021. arXiv
10. D. M. Blei, A. Kucukelbir and J. D. McAuliffe, “Variational Inference: A Review for Statisticians,” *Journal of the American Statistical Association*, vol. 112, no. 518, pp. 859–877, 2017. DOI
11. C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, “On Calibration of Modern Neural Networks,” *ICML*, 2017. arXiv
12. C. Wang, “Calibration in Deep Learning: A Survey of the State-of-the-Art,” arXiv:2308.01222, 2023 (revised 2025). arXiv
13. M. Abdar et al., “A Review of Uncertainty Quantification in Deep Learning: Techniques, Applications and Challenges,” *Information Fusion*, vol. 76, pp. 243–297, 2021. DOI

14. J. Gawlikowski et al., “A Survey of Uncertainty in Deep Neural Networks,” *Artificial Intelligence Review*, vol. 56, pp. 1513–1589, 2023. DOI
15. B. Lakshminarayanan, A. Pritzel and C. Blundell, “Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles,” *NeurIPS (NIPS)*, 2017. arXiv
16. Y. Ovadia et al., “Can You Trust Your Model’s Uncertainty? Evaluating Predictive Uncertainty Under Dataset Shift,” *NeurIPS*, 2019. arXiv
17. A. N. Angelopoulos and S. Bates, “A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification,” arXiv:2107.07511, 2021. arXiv
18. A. van den Oord, Y. Li and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” arXiv:1807.03748, 2018. arXiv
19. B. Poole, S. Ozair, A. van den Oord, A. A. Alemi and G. Tucker, “On Variational Bounds of Mutual Information,” *ICML*, 2019. arXiv
20. M. Tschannen, J. Djolonga, P. K. Rubenstein, S. Gelly and M. Lucic, “On Mutual Information Maximization for Representation Learning,” *ICLR*, 2020. arXiv

ML 01C・數值計算與梯度檢查

浮點數、溢位與下溢、穩定的 softmax、條件數、梯度、Jacobian 與 Hessian、牛頓類方法、KKT 條件與混合精度：讓數學模型在真實計算中站得住腳。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-01c-numerical-computation/

你將學到

- 電腦如何儲存實數、機器 epsilon 的意義，以及 float16、bfloat16、float32、float64 為何表現不同。
- 正確的公式為何會溢位、下溢或遺失有效位數，以及如何穩定地計算 softmax、log-sum-exp 與 logistic 損失。
- 條件數衡量什麼，以及「問題條件不良」與「演算法不穩定」的差別。
- 梯度、Jacobian 與 Hessian 如何描述損失曲面，梯度下降為何會鋸齒前進，牛頓法與擬牛頓法又改變了什麼。
- 如何用投影、廣義拉格朗日函數與 KKT 條件處理限制條件。
- 如何檢查導數、除錯非有限值，並安全地使用混合精度。

先看全貌

機器學習用實數的語言寫成，但電腦沒有真正的實數。它只有有限個浮點數，每一次算術運算都會把結果捨入到其中一個。大多數時候這種捨入看不見；有時卻是災難：指數溢位成無限大、極小的機率變成剛好 0 而它的對數變成負無限大，或是兩個幾乎相等的數相減，絕大部分有效位數就此消失。

故事的另一半是最佳化。訓練模型就是最小化一個有數百萬甚至數十億個參數的函數，而我們只能用數值方法做到：依照導數的指引走很多小步。這些步伐前進得多快、多可靠，取決於損失曲面的形狀（斜率與曲率），也取決於我們怎麼走。本篇以 Goodfellow、Bengio 與 Courville 的「Numerical Computation」一章 [1] 為骨架，補上現代低精度訓練所依賴的浮點細節，並用小型 NumPy 實驗串起每個概念。

> **用最簡單的話說**
>
> 想像一台只能顯示七位數的計算機。你問它一百萬加 0.1，它顯示一百萬：0.1 從螢幕上掉下去了。接著再減掉一百萬，你得到 0 而不是 0.1。好的數值程式會安排計算步驟，讓重要的位數永遠不會掉出螢幕。最佳化則是另一種遊戲：你蒙著眼站在起伏的田野上，想走到最低點。你能用腳感覺坡度（梯度），多花點力氣還能感覺地面怎麼彎（Hessian）。數值計算要做的，就是在計算機不騙你的前提下，安全地走下山。

如果 softmax 與交叉熵對你還很陌生，請先讀ML 01B；特徵值與奇異值分解則可隨時回頭參考ML 01A。之後的ML 02 會用到本篇的所有內容來訓練神經網路。

浮點數

白話版。浮點數儲存一個正負號、幾位有效數字和一個 2 的次方。相鄰浮點數之間的相對間距大致固定，所以大數的絕對捨入誤差大，小數的絕對捨入誤差小。

浮點數如何儲存

IEEE 754 標準 [3] 定義了幾乎所有 CPU 與 GPU 使用的二進位浮點格式。一個正規（非特殊值）數字儲存為

$$x = (-1)^{s} \times \left(1 + \sum_{i=1}^{t} b_i \, 2^{-i}\right) \times 2^{e},$$

其中 $s \in \{0,1\}$ 是正負號位元，$b_1, \dots, b_t \in \{0,1\}$ 是有效數（significand）中儲存的 t 個小數位元（開頭的 1 是隱含的），e 是限制在固定範圍內的整數指數。最重要的兩個數：

- **機器 epsilon** $\varepsilon_{\text{mach}} = 2^{-t}$：1 與下一個可表示的較大數之間的間距。
- **單位捨入誤差**（unit roundoff）$u = \varepsilon_{\text{mach}}/2$：「捨入到最近值」的相對誤差上界。對正規範圍內的任意實數 x，儲存值為 $\mathrm{fl}(x) = x(1+\delta)$，且 $|\delta| \le u$。

標準也要求每個基本運算 $(+, -, \times, \div, \sqrt{\cdot})$ 都像先精確計算再捨入一次那樣：$\mathrm{fl}(a \circ b) = (a \circ b)(1+\delta)$，$|\delta| \le u$。這條規則就是捨入誤差分析的基礎 [2]、[4]。

比最小正規數更小的範圍，格式改用**次正規數**（subnormal），犧牲前導精度、讓數值逐步降到 0。特殊值則補足整體：結果太大時為 $\pm\infty$；無效運算如 $0/0$、$\infty - \infty$、$\infty \times 0$ 則得到 **NaN**（not a number）。NaN 一旦出現，幾乎所有碰到它的運算都會回傳 NaN，這就是為什麼單一個壞值就能毀掉整次訓練。

你會遇到的四種格式

下表以 `numpy.finfo` 計算（bfloat16 因 NumPy 不支援，改用 `torch.finfo`）。「位元」欄列出正負號／指數／小數位元數。

格式	位元	$\varepsilon_{\text{mach}}$	最大有限值	最小正規數	最小次正規數
float16（IEEE 半精度）	1/5/10	9.77×10^{-4}	65504	6.10×10^{-5}	5.96×10^{-8}
bfloat16	1/8/7	7.81×10^{-3}	3.39×10^{38}	1.18×10^{-38}	9.18×10^{-41}
float32（單精度）	1/8/23	1.19×10^{-7}	3.40×10^{38}	1.18×10^{-38}	1.40×10^{-45}
float64（雙精度）	1/11/52	2.22×10^{-16}	1.80×10^{308}	2.23×10^{-308}	4.94×10^{-324}

這張表要從兩個方向讀。**指數位元**決定範圍：float16 超過 65504 就溢位，而 bfloat16 保留了 float32 的 8 個指數位元，所以範圍與 float32 相同。**小數位元**決定精度：float32 約有 7 位有效十進位數字，float64 約 16 位，float16 約 3 位，bfloat16 只有約 2 位。bfloat16「範圍寬、精度粗」與 float16「範圍窄、精度較細」的取捨，正是兩種半精度格式需要不同訓練做法的原因（見後文混合精度一節與 [5]、[12]）。

```
import numpy as np

for dtype in (np.float16, np.float32, np.float64):
    info = np.finfo(dtype)
    print(f"{info.dtype}: eps={float(info.eps):.3e}  max={float(info.max):.3e}  "
          f"tiny={float(info.tiny):.3e}  subnormal={float(info.smallest_subnormal):.3e}")

# Rounding: 1 + eps/2 rounds back to 1 in float32
e = np.finfo(np.float32).eps
print(np.float32(1) + np.float32(e/2) == np.float32(1))   # True
print(np.float32(1) + np.float32(e) == np.float32(1))     # False

# Addition is not associative
a, b, c = np.float32(1e8), np.float32(-1e8), np.float32(1.0)
print((a + b) + c, a + (b + c))                           # 1.0 0.0
```

最後一行值得停下來想。精確算術下兩種順序都得到 1；但在 float32 中，10^8 附近相鄰浮點數的間距是 8，所以 $-10^8 + 1$ 會捨入回 -10^8，那個 1 就不見了。實務上的後果是：用不同順序加總同一組數字（例如換了 GPU 數量，或換了歸約〔reduction〕核心），最後幾位數就可能不同。這是每次執行結果不完全一致的主要來源之一，而且不是你程式的錯 [13]。

溢位、下溢與消去

白話版。溢位是結果大到存不下；下溢是結果小到變成 0（或以次正規數形式失去精度）。消去（cancellation）是兩個幾乎相等的數相減：捨入誤差留了下來，真正的訊號卻被丟掉。

溢位與下溢

在機器學習中，最常見的元兇是指數函數。只要 $z > \ln(3.40\times10^{38}) \approx 88.7$，$\exp(z)$ 就超過 float32 的最大值，因此 logit 為 100 時在 float32 中會溢位（在 float16 中，logit 約 11.1 就會溢位，因為 $\ln 65504 \approx 11.09$）。另一方向，$\exp(-110) \approx 1.7 \times 10^{-48}$ 比 float32 最小的次正規數還小，結果剛好是 0。這兩種情況預設都不會報錯，只會產生 `inf` 與 `0`，而下一個運算就把它們變成 `nan` 或 $-\infty$。

```
import numpy as np

with np.errstate(over="ignore", under="ignore", invalid="ignore"):
    print(np.exp(np.float32(100.0)))        # inf: overflow
    print(np.exp(np.float32(-110.0)))       # 0.0: underflow
    big = np.exp(np.float32(100.0))
    print(big / big)                        # nan: inf / inf

# Catastrophic cancellation and its cure
x = 1e-10
print(np.log(1 + x), np.log1p(x))           # roundoff vs accurate
print(np.exp(x) - 1, np.expm1(x))
```

```
# Variance: the one-pass formula cancels badly when the mean is large
rng = np.random.default_rng(0)
data = (1e4 + rng.normal(0, 1, 100_000)).astype(np.float32)
one_pass = np.mean(data**2) - np.mean(data)**2
two_pass = np.mean((data - data.mean())**2)
print("one-pass:", one_pass, " two-pass:", two_pass)
```

在這台機器上，單趟（one-pass）變異數得到 −8.0——負的變異數——而雙趟（two-pass）公式給出正確的約 1.0。資料平均值為 10^4、變異數為 1，所以 $\mathbb{E}[X^2] \approx 10^8$、$(\mathbb{E}X)^2 \approx 10^8$；在這個量級上 float32 只能分辨約 8 的差異，真正的差值 1 完全淹沒在雜訊裡。

為什麼減法是危險的運算

假設 $\hat{a} = a(1+\delta_a)$、$\hat{b} = b(1+\delta_b)$ 的儲存相對誤差滿足 $|\delta_a|, |\delta_b| \le u$。即使減法本身是精確的，

$$\frac{|(\hat{a}-\hat{b}) - (a-b)|}{|a-b|} = \frac{|a\delta_a - b\delta_b|}{|a-b|} \le u\,\frac{|a|+|b|}{|a-b|}.$$

其中 a, b 是真實值，u 是單位捨入誤差。當 $a \approx b$ 時，因子 $(|a|+|b|)/|a-b|$ 非常大：減法並沒有製造誤差，而是把兩數共有的位數消掉，暴露出輸入原本就帶有的誤差。解法是代數上的：改寫公式，讓這個減法根本不發生。你會用到的例子：

- `log1p(x)` 計算 $\log(1+x)$、`expm1(x)` 計算 $e^x - 1$，在 x 很小時都很精確，天真寫法則幾乎失去所有位數（見上方輸出）。
- 變異數：先減去平均值（雙趟），或使用 Welford 的串流更新。
- Softmax 與 log-sum-exp：先減去最大值，下一節說明。
- 距離：$\|x-y\|^2 = \|x\|^2 + \|y\|^2 - 2x^\top y$ 很快，但在 $x \approx y$ 時嚴重消去；需要精確時，把結果截在 0 以上，或直接計算 $x - y$。

穩定的 softmax 與 log-sum-exp

白話版。取指數前先減去一個共同常數；對數機率要直接計算，而不是先產生極小的機率再取對數。

Softmax 把 logit 向量 $z \in \mathbb{R}^K$ 轉成機率：

$$\operatorname{softmax}(z)_i = \frac{\exp(z_i)}{\sum_{j=1}^{K} \exp(z_j)}, \qquad i = 1, \dots, K.$$

把同一個常數 c 加到每個 logit 上，softmax 不變，因為因子 $\exp(c)$ 在分子與分母間抵銷。取 $c = -m$，其中 $m = \max_i z_i$：

$$p_i = \frac{\exp(z_i - m)}{\sum_j \exp(z_j - m)}.$$

現在每個指數都 ≤ 0，不會溢位；最大的一項剛好是 $\exp(0) = 1$，所以分母至少為 1，也不會下溢成 0。同樣的技巧也能穩定 **log-sum-exp** 函數：

$$\mathrm{LSE}(z) = \log \sum_j \exp(z_j) = m + \log \sum_j \exp(z_j - m), \qquad \log p_i = z_i - \mathrm{LSE}(z).$$

LSE 是最大值的平滑版本：$\max_i z_i \le \mathrm{LSE}(z) \le \max_i z_i + \log K$；它的梯度恰好就是 softmax，$\partial\,\mathrm{LSE}/\partial z_i = p_i$。

對目標類別 y，交叉熵損失為 $-\log p_y = \mathrm{LSE}(z) - z_y$，它對 logit 的梯度是那個著名的簡單形式 $p - e_y$，其中 e_y 是目標的 one-hot 向量。先算 `softmax` 再取 `log` 仍可能失敗：若 p_y 下溢成 0，它的對數就是 $-\infty$，即使 $\mathrm{LSE}(z) - z_y$ 是個完全正常的有限數。這就是框架提供接受 logit 的融合函數 `log_softmax` 與 `cross_entropy` 的原因。

程式：會讓天真指數運算溢位的 logit

這個獨立的 NumPy 範例檢查穩定的機率、對數機率，以及平移 logit 的不變性。

```
import numpy as np

def logsumexp(z):
    m = np.max(z)
    return m + np.log(np.exp(z-m).sum())
def logsoftmax(z):
    return z-logsumexp(z)

z = np.array([1000., 1001., 999.], dtype=np.float64)
logp = logsoftmax(z)
p = np.exp(logp)
assert np.allclose(p.sum(), 1)
assert np.allclose(logsoftmax(z+12345), logp)
print("probabilities:", p)
print("cross-entropy for class 1:", -logp[1])
with np.errstate(over="ignore", invalid="ignore"):
    naive = np.exp(z) / np.exp(z).sum()
print("naive softmax:", naive)
```

穩定版輸出機率 $[0.245, 0.665, 0.090]$，類別 1 的交叉熵為 0.408 nats；天真版輸出 `[nan nan nan]`，因為 $\exp(1000)$ 即使在 float64 也會溢位（float64 的上限是 $\exp(709.78)$）。

穩定的 logistic（二元交叉熵）損失

二元分類以另一種形式遇到同樣的問題。設 logit 為 z、標籤 $y \in \{0, 1\}$、sigmoid 為 $\sigma(z) = 1/(1+e^{-z})$，損失 $-y \log \sigma(z) - (1-y)\log(1-\sigma(z))$ 可化簡為

$$\ell(z, y) = \mathrm{softplus}(z) - yz, \qquad \mathrm{softplus}(z) = \log(1 + e^z) = \max(z, 0) + \log\!\left(1 + e^{-|z|}\right).$$

右邊的寫法從不對正數取指數，`log1p` 則讓小修正項保持精確。請優先使用 `binary_cross_entropy_with_logits`，不要自己先算 sigmoid 再取對數。

```
import numpy as np

def bce_naive(z, y):
    s = 1 / (1 + np.exp(-z))
    return -(y*np.log(s) + (1-y)*np.log(1-s))

def bce_stable(z, y):
    # softplus(z) - y*z, with softplus(z) = max(z,0) + log1p(exp(-|z|))
    return np.maximum(z, 0) + np.log1p(np.exp(-np.abs(z))) - y*z

z = np.array([-800., -30., 0., 30., 800.])
y = np.array([1., 1., 0., 0., 0.])
with np.errstate(over="ignore", divide="ignore", invalid="ignore"):
    print("naive :", bce_naive(z, y))
print("stable:", bce_stable(z, y))
```

天真版對兩個「信心十足的錯誤」（$z = \pm 800$）回傳 `inf`，在 $z = 30$ 時回傳 30.001 而非 30.000，因為 $1 - \sigma(30)$ 是靠消去算出來的。穩定版回傳 800、30、$\log 2$、30、800，全部正確。

條件不良

白話版。有些問題天生敏感：輸入的微小改變會造成答案的巨大改變，不論你算得多小心。**條件數**（condition number）就是衡量這種敏感度的量。

線性系統的條件數

考慮對可逆方陣 A 解 $Ax = b$。若資料變成 $b + \delta b$，解變成 $x + \delta x$，則

$$\frac{\|\delta x\|}{\|x\|} \le \kappa(A)\,\frac{\|\delta b\|}{\|b\|}, \qquad \kappa(A) = \|A\|\,\|A^{-1}\| = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)} \ \text{(2-norm)}.$$

其中 $\|\cdot\|$ 是歐氏向量範數及其誘導矩陣範數，$\sigma_{\max}, \sigma_{\min}$ 是 A 的最大與最小奇異值。對稱正定矩陣的奇異值等於特徵值，因此 $\kappa = \lambda_{\max}/\lambda_{\min}$。Goodfellow 等人用特徵分解對函數 $f(x) = A^{-1}x$ 陳述同一件事：它的條件數是 $\max_{i,j} |\lambda_i/\lambda_j|$ [1]。實用的經驗法則：$\kappa \approx 10^k$ 時，相對於輸入的精度，大約會損失 k 位十進位精度 [4]。

```
import numpy as np

A = np.array([[1.0, 1.0],
              [1.0, 1.0001]])
print("condition number:", np.linalg.cond(A))

b = np.array([2.0, 2.0001])
```

```
x = np.linalg.solve(A, b)
db = np.array([0.0, 0.0001])                  # tiny change in the data
x2 = np.linalg.solve(A, b + db)
rel_in = np.linalg.norm(db) / np.linalg.norm(b)
rel_out = np.linalg.norm(x2 - x) / np.linalg.norm(x)
print("x:", x, " x after perturbation:", x2)
print(f"relative change in b: {rel_in:.1e}, in x: {rel_out:.1e}, "
      f"amplification: {rel_out/rel_in:.0f}")
```

A 的兩欄幾乎相同，$\kappa(A) \approx 4\times10^4$；$b$ 只有 3.5×10^{-5} 的相對改變，解卻從 $(1,1)$ 移到 $(0,2)$——百分之百的改變，放大約 28,000 倍，仍在 κ 的上界之內。沒有任何計算出錯，是這個問題本身條件不良。

條件不良的問題 vs. 不穩定的演算法

請把兩個概念分開：

- **條件不良的問題**會放大輸入的小變化。任何演算法都無法修正；你需要更好的資料、正則化（它改變了問題）或重新表述問題。
- **不穩定的演算法**會替一個可能條件良好的問題加入可避免的誤差。天真的 softmax 與單趟變異數就是不穩定的演算法，換演算法就能解決。

最小平方法同時展示兩者。如果特徵幾乎共線，係數本身就是條件不良的（問題）。解正規方程 $X^\top X w = X^\top y$ 會讓情況比必要的更糟，因為 $\kappa(X^\top X) = \kappa(X)^2$（演算法）；基於 QR 或 SVD 的求解器則直接處理 X。Ridge 迴歸加入 $\lambda\|w\|^2$，把問題改成 $(X^\top X + \lambda I)w = X^\top y$，其條件數 $(\sigma_{\max}^2 + \lambda)/(\sigma_{\min}^2 + \lambda)$ 較小——ML 01A 會進一步發展這個主題。

以梯度為基礎的最佳化

白話版。要最小化一個函數，就看它的斜率、往下坡走。梯度指向上升最陡的方向，所以往反方向走；步伐要夠小，讓你剛量到的斜率大致仍然有效。

導數、梯度與方向導數

我們要最小化**目標函數**（也稱成本、損失或誤差函數）$f:\mathbb{R}^n\to\mathbb{R}$。單變數時，導數 $f'(x)$ 給出一階近似 $f(x+\epsilon)\approx f(x)+\epsilon f'(x)$，所以往 $f'(x)$ 的反號方向走一小步就能讓 f 下降。多變數時，**梯度** $\nabla_x f(x)\in\mathbb{R}^n$ 收集所有偏導數 $\partial f/\partial x_i$。單位方向 u 上的**方向導數**是沿 u 的斜率：

$$\frac{\partial}{\partial \alpha} f(x+\alpha u)\Big|_{\alpha=0} = u^\top \nabla_x f(x) = \|\nabla_x f(x)\|_2 \cos\theta,$$

其中 θ 是 u 與梯度的夾角。當 $\theta=\pi$，也就是 u 指向 $-\nabla_x f(x)$ 時，它最負。這就是**最陡下降法**，也就是**梯度下降**：

$$x \leftarrow x - \eta \nabla_x f(x),$$

其中 $\eta > 0$ 是**學習率**（步長）。η 可以取一個小常數、可以試幾個值並保留目標函數最低者（**線搜尋**，line search），也可以依照排程變化。當梯度的每個分量都（接近）0 時，梯度下降就收斂了。

臨界點：極小值、極大值與鞍點

滿足 $\nabla_x f(x) = 0$ 的點稱為**臨界點**或**駐點**。**局部極小值**比所有鄰近點都低，**局部極大值**比所有鄰近點都高，**鞍點**兩者皆非：它沿某些方向是極小、沿另一些方向是極大。**全域極小值**是整個定義域上的最低值；一個函數可能有好幾個全域極小值，也可能有許多不是全域極小的局部極小值。

對深度網路，我們很少找到真正的全域極小值，也不需要；只要參數讓損失夠低而且泛化良好就行。在高維空間中，對許多「類隨機」函數而言，鞍點比差勁的局部極小值更常見，而且因為梯度在鞍點附近變小，它們會拖慢以梯度為基礎的最佳化器 [8]。圖 1 顯示三種臨界點。

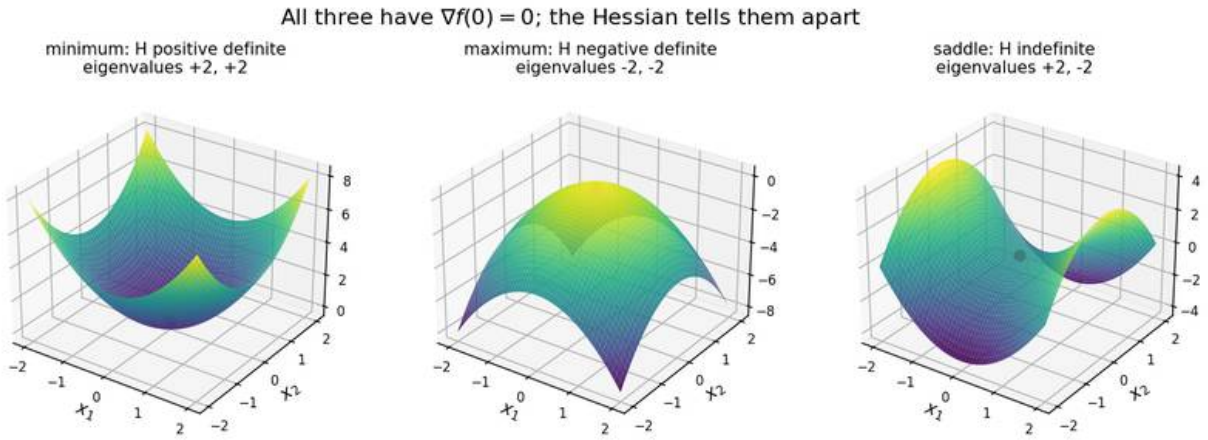


圖 *1* — 三個在原點梯度為零的函數。*Hessian* 的特徵值可以區分它們：全正（極小值）、全負（極大值）、正負皆有（鞍點）。

Jacobian 與 Hessian 矩陣

白話版。 Jacobian 收集向量值函數的所有一階導數。Hessian 收集純量函數的所有二階導數：它告訴你斜率本身如何變化，也就是曲率。

定義

對函數 $f : \mathbb{R}^m \to \mathbb{R}^n$，**Jacobian 矩陣** $J \in \mathbb{R}^{n \times m}$ 的元素為

$$J_{ij} = \frac{\partial f_i(x)}{\partial x_j}, \qquad i = 1, \dots, n,\ j = 1, \dots, m.$$

第 i 列是輸出 f_i 的梯度。反向傳播就是不斷乘上 Jacobian；反向模式自動微分計算向量–Jacobian 乘積 $v^\top J$，卻從不顯式建出 J，這就是為什麼一次梯度的成本只是一次函數求值的小常數倍 [11]。

對純量函數 $f:\mathbb{R}^n\to\mathbb{R}$，**Hessian 矩陣** $H(f)(x)\in\mathbb{R}^{n\times n}$ 是梯度的 Jacobian：

$$H(f)(x)_{ij}=\frac{\partial^2 f(x)}{\partial x_i\,\partial x_j}.$$

只要二階偏導數連續，微分順序就不影響結果，所以 $H_{ij}=H_{ji}$：Hessian 是對稱矩陣。實對稱矩陣有實數特徵值 $\lambda_1,\dots,\lambda_n$ 與一組正交歸一的特徵向量 $v_1,\dots,v_n$。沿單位方向 d 的二**階方向導數**是 $d^\top Hd$。若 d 是特徵向量 v_i，它等於 λ_i；其他方向則是特徵值的加權平均 $\sum_i \lambda_i(d^\top v_i)^2$，權重介於 0 與 1 之間且總和為 1。所以 $\lambda_{\max}$ 是任一方向上的最大曲率，$\lambda_{\min}$ 是最小曲率。

一步梯度到底做了什麼：二階泰勒展開

令 $g=\nabla_x f(x^{(0)})$、$H=H(f)(x^{(0)})$。在 $x^{(0)}$ 附近的二階泰勒近似為

$$f(x)\approx f(x^{(0)})+(x-x^{(0)})^\top g+\tfrac{1}{2}(x-x^{(0)})^\top H(x-x^{(0)}).$$

代入一步梯度 $x=x^{(0)}-\eta g$ 得到

$$f(x^{(0)}-\eta g)\approx f(x^{(0)})-\eta\,g^\top g+\tfrac{1}{2}\eta^2\,g^\top Hg.$$

三項分別是目前的值、斜率預測的改善量，以及曲率的修正。若 $g^\top Hg\le 0$，η 越大，近似值就越小（直到近似本身失效為止）。若 $g^\top Hg>0$，曲率項最終會勝出，近似值在下式取得最小：

$$\eta^*=\frac{g^\top g}{g^\top Hg}.$$

最壞情況下 g 與 $\lambda_{\max}$ 的特徵向量同向，此時 $\eta^*=1/\lambda_{\max}$。曲率大，就只能走小步。

二階導數檢定

在臨界點（$g=0$），泰勒展開只剩二次項，所以由 Hessian 決定這是哪一種點：

- H **正定**（所有 $\lambda_i>0$）：局部極小值。
- H **負定**（所有 $\lambda_i<0$）：局部極大值。
- H 至少有一個正特徵值與一個負特徵值：**鞍點**。
- 其他情況（有些 $\lambda_i=0$，其餘同號）：檢定無法判斷。

一維中不可能有鞍點；但在 n 維中，只要有一個特徵值的符號「不對」就是鞍點，所以隨著 n 變大，鞍點變得非常普遍 [8]。

Hessian 的條件數：梯度下降為何鋸齒前進

在二次函數 $f(w)=\frac{1}{2}w^\top Hw$（$H$ 對稱正定）上，梯度下降是線性的：$w\leftarrow(I-\eta H)w$。把 w 用 H 的特徵向量展開，沿 v_i 的分量每一步都乘上 $(1-\eta\lambda_i)$。因此：

- 只有當每個 i 都滿足 $|1-\eta\lambda_i|<1$，也就是 $0<\eta<2/\lambda_{\max}$ 時，迭代才**穩定**。

- **最慢**的是平坦方向，每步只縮小 $|1-\eta\lambda_{\min}|$ 倍。
- 最佳固定步長 $\eta = 2/(\lambda_{\max}+\lambda_{\min})$ 讓誤差每步縮小 $(\kappa-1)/(\kappa+1)$ 倍，其中 $\kappa = \lambda_{\max}/\lambda_{\min}$ 是 Hessian 的條件數。

所以條件數大，表示某個方向要求極小的步伐（否則發散），另一個方向卻需要很多步才有進展。梯度下降把時間浪費在狹窄山谷的陡峭兩壁間來回彈跳——圖 2 中的鋸齒——同時沿著谷底緩慢前進。輸入特徵尺度不一致正好造成這種情況，這也是標準化與正規化層能加快訓練的原因之一。

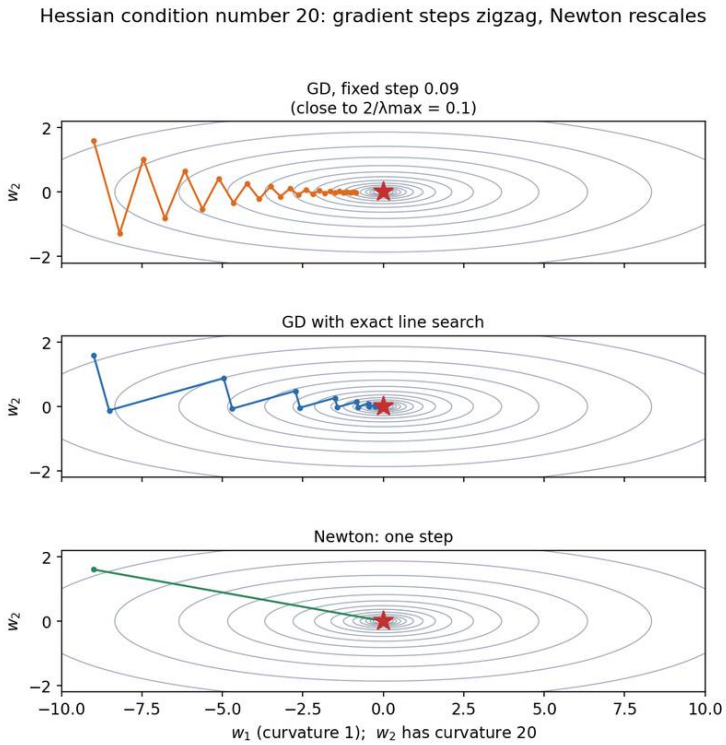


圖 2 — 最小化 *Hessian* 條件數為 *20* 的二次函數。步長接近穩定極限的梯度下降在陡峭方向來回振盪；精確線搜尋使相鄰兩步互相垂直；牛頓法以 *Hessian* 的反矩陣重新縮放，對二次函數一步就到達極小值。

圖 3 以數值呈現穩定極限。低於 $2/\lambda_{\max}$ 時損失下降；稍高於極限時，陡峭分量每步放大 $|1-\eta\lambda_{\max}|>1$ 倍，損失隨之爆炸。

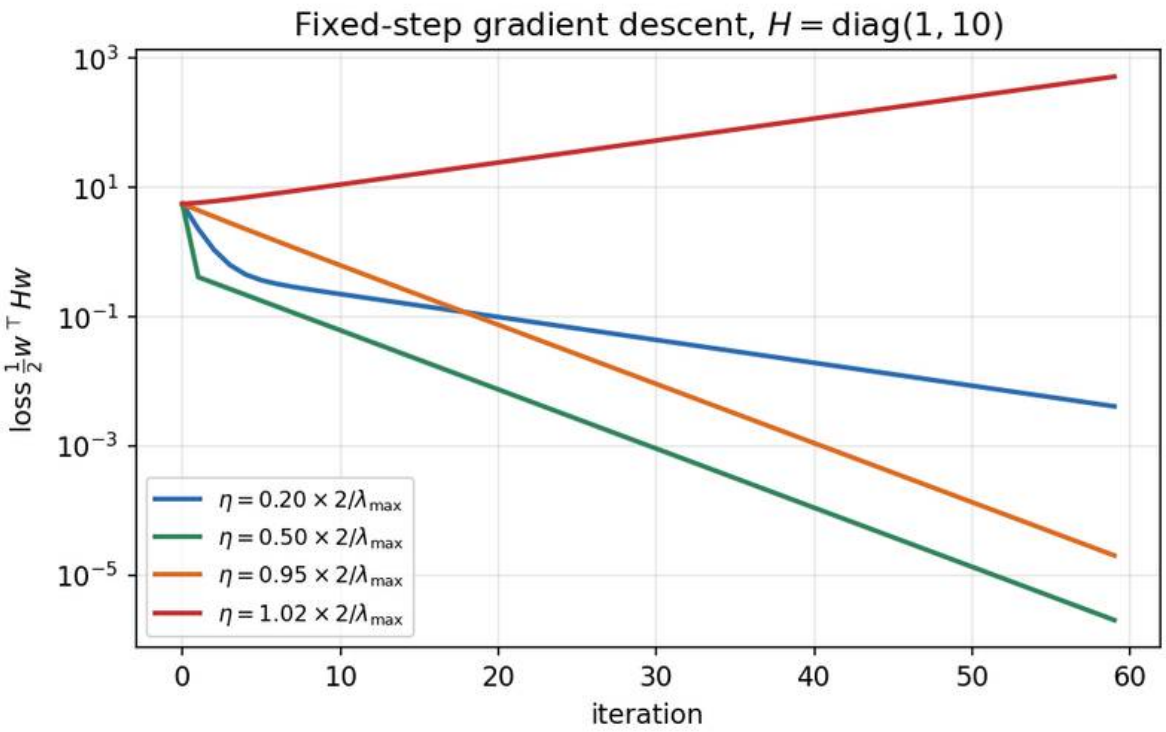


圖 3 — 在特徵值為 1 與 10 的二次函數上做固定步長梯度下降。小於 $2/\lambda max$ 的步長都會收斂（在 $2/(\lambda max+\lambda min)$ 附近最快）；只比極限大 2% 的步長就會發散。

```
import numpy as np

H = np.diag([1.0, 25.0])                    # Hessian: condition number 25
f = lambda w: 0.5 * w @ H @ w
grad = lambda w: H @ w

def run_gd(eta, steps=50):
    w = np.array([10.0, 1.0])
    for _ in range(steps):
        w = w - eta * grad(w)
    return f(w)

lam_max = np.linalg.eigvalsh(H).max()
for eta in (0.01, 0.07, 0.079, 0.081):       # stability limit 2/25 = 0.08
    print(f"eta={eta:<6} loss after 50 steps: {run_gd(eta):.3e}")

# Exact line search along -g for a quadratic: eps* = g'g / g'Hg
w = np.array([10.0, 1.0])
g = grad(w)
eps_star = (g @ g) / (g @ H @ g)
print("optimal step along the gradient:", round(eps_star, 4))

# One Newton step lands on the minimizer of a quadratic
w_newton = w - np.linalg.solve(H, g)
```

```
print("after one Newton step:", w_newton)
```

輸出說明了一切：$\eta = 0.01$ 穩定但慢（50 步後損失 18.3），$\eta = 0.07$ 降到 0.035，$\eta = 0.079$ 勉強穩定但陡峭分量仍在振盪（損失約 1.0），$\eta = 0.081$ 發散（損失 148）。沿第一個梯度的最佳步長是 0.046，而一步牛頓法正好落在 $(0, 0)$。

二階方法

白話版。不只跟著斜率走，也利用曲率：依照函數在每個方向彎得多急來縮放步伐，陡的方向走小步、平的方向走大步。

牛頓法

令二階泰勒近似的梯度為零，得到**牛頓步**：

$$x^* = x^{(0)} - H(f)(x^{(0)})^{-1}\,\nabla_x f(x^{(0)}).$$

在正定二次函數上，這一步直接跳到極小值（圖 2 下）。對局部接近正定二次函數的一般函數，反覆套用這個更新比梯度下降收斂快得多——在標準假設下，於解附近呈二次收斂 [6]。代價是 Hessian：儲存它需要 $O(n^2)$ 記憶體，用它求解需要 $O(n^3)$ 時間，對一個有 10^8 個參數的網路根本不可能。

對非凸目標還有更根本的問題。牛頓法找的是梯度為零的點，它對待鞍點和極小值一視同仁：在鞍點附近，牛頓步會指向鞍點。在充滿鞍點的地形上，單純的牛頓法正好被我們想逃離的點吸引。Dauphin 等人提出「saddle-free Newton」，改用 Hessian 特徵值的絕對值來縮放，以避免這個問題 [8]。相對地，梯度下降不會以同樣方式被鞍點吸引，雖然它在鞍點附近可能走得很慢。

擬牛頓法與 L-BFGS

擬牛頓法（quasi-Newton）只用梯度建立 $B_k \approx H$（或其反矩陣）的近似。走了一步 $s_k = x_{k+1} - x_k$、梯度變化為 $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$ 之後，它要求**割線條件** $B_{k+1} s_k = y_k$，意思是模型沿這一步的曲率要與觀察到的斜率變化一致。BFGS 是這類更新中最常用的；**L-BFGS**（有限記憶 BFGS）只保存最近 m 組 (s_k, y_k)，以 $O(mn)$ 時間隱式套用反 Hessian 的近似 [6]、[10]。L-BFGS 很適合平滑、確定性、中等規模的問題（logistic 迴歸、單張影像的風格轉換、擬合幾千個參數）；但當梯度是有雜訊的小批次估計時，它的曲率估計會受影響。

Hessian–向量乘積

許多利用曲率的方法其實不需要 H 本身，只需要乘積 Hv。這可以用大約幾次梯度計算的成本精確算出，因為 $Hv = \nabla_x(\nabla_x f(x)^\top v)$ 是某個純量的梯度；Pearlmutter 說明了如何用自動微分做到 [9]。有限差分版本 $Hv \approx [\nabla f(x + rv) - \nabla f(x - rv)]/(2r)$ 是很實用的檢查。以 Hessian–向量乘積做冪迭代（power iteration）可以估計 $\lambda_{\max}$，研究者正是用這個方法追蹤訓練中損失的尖銳度（sharpness）。

```
import numpy as np

rng = np.random.default_rng(0)
X = rng.normal(size=(50, 4))
y = (rng.random(50) < 0.5).astype(float)

def grad(w):                                    # logistic regression, mean loss
    p = 1 / (1 + np.exp(-(X @ w)))
    return X.T @ (p - y) / len(y)

def hessian(w):
    p = 1 / (1 + np.exp(-(X @ w)))
    return (X.T * (p * (1 - p))) @ X / len(y)

w = rng.normal(size=4)
v = rng.normal(size=4)
r = 1e-5
hvp_fd = (grad(w + r*v) - grad(w - r*v)) / (2*r)   # never forms H
print("Hv (finite diff):", hvp_fd.round(6))
print("Hv (explicit)   :", (hessian(w) @ v).round(6))
print("eigenvalues of H:", np.linalg.eigvalsh(hessian(w)).round(4))
```

兩種乘積在小數點後六位都相同，而且 Hessian 的特徵值全為正：logistic 迴歸是凸的，它的 Hessian 處處半正定。

Lipschitz 連續與凸性

深度學習的目標函數幾乎沒有什麼保證，但有兩個性質能解釋大部分理論。若對所有 x, y 都有 $|f(x) - f(y)| \le L\|x - y\|_2$，則函數是常數為 L 的 **Lipschitz 連續**：它的變化率有上界。若梯度是 L-Lipschitz（函數「L-平滑」），曲率就以 L 為上界，而步長 $\eta \le 1/L$ 的梯度步保證不會讓函數變大：

$$f(x - \eta \nabla f(x)) \le f(x) - \frac{\eta}{2}\|\nabla f(x)\|_2^2 \qquad (0 < \eta \le 1/L).$$

這是前面「$\eta < 2/\lambda_{\max}$」規則的一般形式，也是大多數收斂證明的起點 [7]、[14]。

若函數（二次可微時）的 Hessian 處處半正定，它就是**凸函數**。凸函數沒有鞍點，也沒有虛假的局部極小值：每個局部極小值都是全域極小值。最小平方、logistic 迴歸與支援向量機都是凸的；深度網路不是。凸最佳化是許多工具的來源，例如對偶性與下面的 KKT 條件，深度學習也借用了它們 [7]。

有限制條件的最佳化

白話版。有時答案必須落在允許的區域內——範數有上限的權重、總和為一的機率、必須為正的物理量。要嘛走一步之後投影回區域內，要嘛為每個限制條件加上一個懲罰項，並讓它的強度自動決定。

投影梯度下降

我們要在可行集合 $\mathbb{S} \subseteq \mathbb{R}^n$ 上最小化 $f(x)$。如果投影到 $\mathbb{S}$ 很容易，就先走一般的梯度步，再回到最近的可行點：

$$x \leftarrow \operatorname{Proj}_{\mathbb{S}}\left(x - \eta \nabla f(x)\right), \qquad \operatorname{Proj}_{\mathbb{S}}(v) = \arg\min_{x \in \mathbb{S}} \|x - v\|_2 .$$

對歐氏球 $\{x : \|x\|_2 \le r\}$，投影為 $v \cdot \min(1, r/\|v\|_2)$：太長就縮短。權重的最大範數限制、把參數截在一個方盒內、投影到機率單形（simplex）都屬於這個模式。另一種做法是重新參數化，讓限制條件自動成立，例如用 $x = r\,(\cos\theta, \sin\theta)$ 表示圓上的向量，或用 softmax 產生機率。

廣義拉格朗日函數

更一般的做法是用 m 個等式限制 $g^{(i)}(x) = 0$ 與 k 個不等式限制 $h^{(j)}(x) \le 0$ 描述可行集合：

$$\mathbb{S} = \{x \mid g^{(i)}(x) = 0 \ \forall i,\ h^{(j)}(x) \le 0 \ \forall j\}.$$

為每個限制條件引入一個 **KKT 乘數**，等式用 λ_i、不等式用 α_j，並定義**廣義拉格朗日函數**

$$L(x, \lambda, \alpha) = f(x) + \sum_{i=1}^{m} \lambda_i\, g^{(i)}(x) + \sum_{j=1}^{k} \alpha_j\, h^{(j)}(x).$$

原本有限制的問題等價於沒有限制的 min–max 問題

$$\min_{x} \max_{\lambda} \max_{\alpha,\ \alpha \ge 0} L(x, \lambda, \alpha).$$

為什麼可行？若 x 違反某個限制，內層最大化可以選一個很大的乘數，把 L 推到 $+\infty$，所以不可行點永遠不會被選上。若 x 可行，內層最大化最多只能讓這些項為零（等式本來就是 0；對 $h^{(j)}(x) \le 0$，最好的非負 α_j 會使 $\alpha_j h^{(j)}(x) = 0$），因此值恰好是 $f(x)$ [1]。

KKT 條件

在解 x^* 處（在稱為「限制條件規範性」〔constraint qualification〕的溫和正則條件下），存在乘數滿足 **Karush–Kuhn–Tucker（KKT）條件** [1]、[7]：

1. **穩定性（stationarity）：** $\nabla_x L(x^*, \lambda^*, \alpha^*) = 0$。
2. **原始可行性：** x^* 滿足所有限制條件。
3. **對偶可行性：** $\alpha^* \ge 0$。
4. **互補鬆弛性：**對每個 j，$\alpha_j^*\, h^{(j)}(x^*) = 0$。

互補鬆弛性是說，每個不等式要嘛**有效**（active，$h^{(j)}(x^*) = 0$：解落在邊界上，限制條件以 $\alpha_j \ge 0$ 的力量把它推回來），要嘛**無效**（inactive，$h^{(j)}(x^*) < 0$：限制條件在局部不起作用，$\alpha_j = 0$）。從幾何上看，穩定性條件是說：在有效限制處，下坡方向 $-\nabla f$ 正好垂直指出可行集合之外，與 ∇h 平行；任何可行的移動都會往上坡走（圖 4）。對凸問題，KKT 條件也是最佳性的充分條件 [7]。

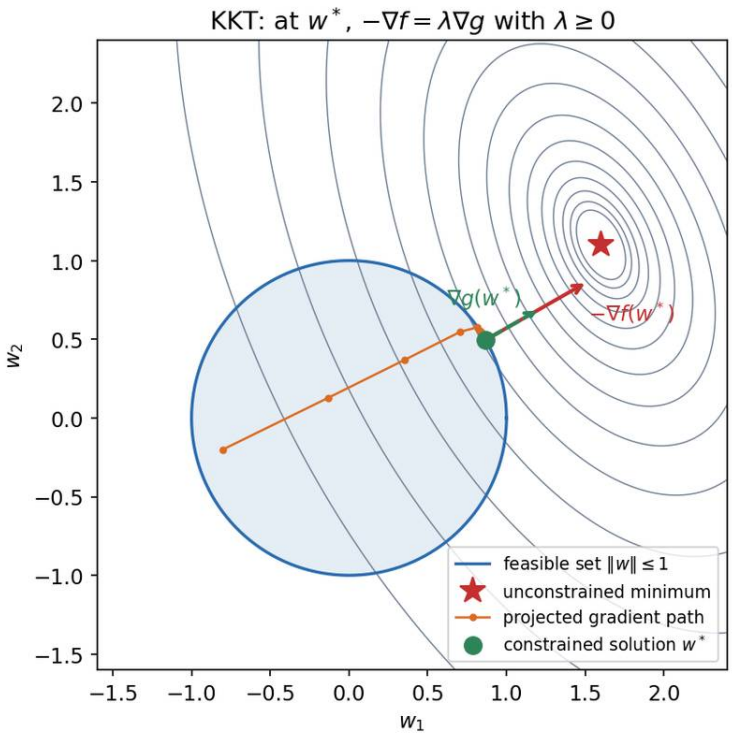


圖 4 — 在單位圓盤上最小化二次函數。無限制的極小值（星號）不可行。投影梯度下降停在邊界上的 w^*，此處 $-\nabla f$ 與限制條件的外法向量 ∇g 平行：這正是乘數為正時的 *KKT* 穩定性條件。

範例：帶範數限制的最小平方

依照 [1] 的例子，在限制 $h(w) = w^\top w - r^2 \leq 0$ 下最小化平方誤差 $f(w) = \|Aw - b\|_2^2$，其中 $A \in \mathbb{R}^{N \times n}$、$b \in \mathbb{R}^N$。拉格朗日函數為

$$L(w, \alpha) = \|Aw - b\|_2^2 + \alpha\,(w^\top w - r^2).$$

穩定性條件給出 $2A^\top(Aw - b) + 2\alpha w = 0$，所以

$$w(\alpha) = (A^\top A + \alpha I)^{-1} A^\top b.$$

這就是懲罰係數為 α 的 ridge 迴歸。若一般最小平方解 $w(0)$ 已經滿足 $\|w\| \leq r$，限制條件無效，$\alpha^* = 0$，問題解決。否則限制條件有效，就增大 α 直到 $\|w(\alpha)\| = r$；因為 $\|w(\alpha)\|$ 隨 α 單調遞減，用二分法就能找到。這也正是對 α 做內層最大化的行為：限制被違反時乘數變大，剛好滿足時停下。下面的程式用兩種方法求解，並檢查全部四個 KKT 條件。

```
import numpy as np

# minimize ||Aw - b||^2  subject to  ||w||^2 <= r^2
rng = np.random.default_rng(1)
A = rng.normal(size=(30, 5))
```

```
b = rng.normal(size=30)
r = 0.2

def w_of(lam):                                  # stationarity of the Lagrangian
    return np.linalg.solve(A.T @ A + lam * np.eye(5), A.T @ b)

w_ls = w_of(0.0)
print("unconstrained norm:", np.linalg.norm(w_ls).round(3))

lo, hi = 0.0, 1e6                               # find lambda with ||w|| = r
for _ in range(200):
    mid = 0.5 * (lo + hi)
    if np.linalg.norm(w_of(mid)) > r:
        lo = mid
    else:
        hi = mid
lam = hi
w = w_of(lam)
print("lambda:", round(lam, 3), " ||w||:", np.linalg.norm(w).round(6))

# KKT checks
stationarity = 2 * A.T @ (A @ w - b) + 2 * lam * w
print("stationarity residual:", np.abs(stationarity).max())
print("primal feasible:", w @ w <= r**2 + 1e-9, " dual feasible:", lam >= 0)
print("complementary slackness:", lam * (w @ w - r**2))

# Projected gradient descent reaches the same point
L = 2 * np.linalg.eigvalsh(A.T @ A).max()     # Lipschitz constant of the gradient
v = np.zeros(5)
for _ in range(5000):
    v = v - (1 / L) * 2 * A.T @ (A @ v - b)
    n = np.linalg.norm(v)
    if n > r:
        v = v * (r / n)                         # projection onto the ball
print("projected GD matches KKT solution:", np.allclose(v, w, atol=1e-6))
```

無限制解的範數為 0.219，大於 $r = 0.2$，所以限制條件有效。二分法找到 $\alpha^* \approx 1.81$，穩定性殘差約 10^{-15}，互補鬆弛性成立，而步長 $1/L$ 的投影梯度下降收斂到同一個向量。請注意，梯度的 Lipschitz 常數 L 決定了安全的步長，正如上一節所說。

檢查導數

白話版。檢查程式算出的梯度，是否與「輕推參數時損失實際的變化」一致。

中央有限差分估計偏導數：

$$\frac{\partial f}{\partial x_j} \approx \frac{f(x + \epsilon e_j) - f(x - \epsilon e_j)}{2\epsilon},$$

其中 e_j 是第 j 個標準基底向量，$\epsilon > 0$ 是小步長。它的**截斷誤差**（忽略高階泰勒項造成）是 $O(\epsilon^2)$，優於前向差分 $[f(x + \epsilon e_j) - f(x)]/\epsilon$ 的 $O(\epsilon)$。但分子是兩個幾乎相等的數相減，所以**捨入誤差**

以 $u|f|/\epsilon$ 的速度增長。總誤差在中等步長時最小——當 f 與其導數為單位尺度時，中央差分約在 $\epsilon \approx u^{1/3}$、前向差分約在 $u^{1/2}$。圖 5 顯示這種 V 形曲線，也說明為什麼 float32 的梯度檢查遠不如 float64 精確。

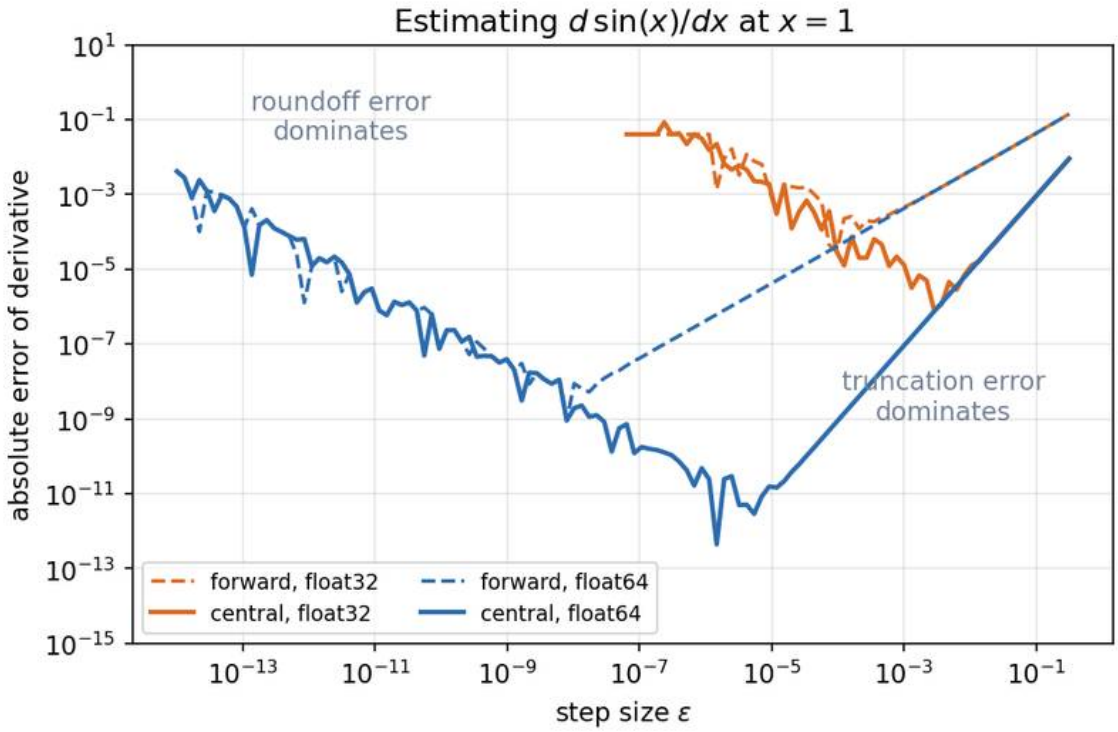


圖 *5* — 在 *x=1* 處以有限差分估計 $sin(x)$ 導數的誤差。最低點右側由截斷誤差主導（前向差分斜率 *1*、中央差分斜率 *2*）；左側由捨入誤差主導。中央差分在 *float32* 最低約 *1e−6*，在 *float64* 可低到約 *1e−11* 或更小。

實用原則：用 float64 做檢查；嘗試多個步長；用相對誤差比較，例如 $|a-n|/\max(|a|,|n|,\text{tiny})$；並在平滑的點上測試。ReLU 在 0 處不可微，那裡沒有單一導數可以比對。參數很多時，**方向檢查**更省：選一個隨機單位向量 v，比較 $\nabla f(w)^\top v$ 與沿 v 的有限差分——只需要一對函數求值，而不是 $2n$ 次。

```
import numpy as np

rng = np.random.default_rng(0)
X = rng.normal(size=(20, 5))
y = rng.normal(size=20)
w = rng.normal(size=5)
v = rng.normal(size=5)
v /= np.linalg.norm(v)
def loss(w):
    return .5*np.mean((X@w-y)**2)
grad = X.T@(X@w-y)/len(y)
eps = 1e-5
numeric = (loss(w+eps*v)-loss(w-eps*v))/(2*eps)
```

```
analytic = grad@v
assert np.allclose(numeric, analytic, rtol=1e-6, atol=1e-8)
print("directional derivative:", numeric, analytic)
```

兩個數字大約有九位有效數字相同。但要記得「檢查通過」代表什麼：自動微分計算的是被記錄下來的程式的導數，而不是你心中想寫的程式 [11]。被 detach 的張量、錯誤的歸約方式、意外的廣播（broadcasting）或目標洩漏（target leakage），都會產生一個完全可微卻錯誤的目標函數，而梯度檢查會很樂意地確認它的梯度。請在已知答案的情況下，直接檢查損失值本身。

混合精度

白話版。大部分運算用 16 位元數字，可以讓記憶體傳輸量減半並使用高速硬體單元；但 16 位元不夠應付所有事情，少數敏感的量要保留在 32 位元。

Micikevicius 等人提出的混合精度訓練 [12] 結合三個想法：

1. 保留一份 **float32 主權重**（master weights）。當更新量小於所加權重的約 2^{-11} 倍時，在 float16 中加上去會被完全捨去（與上文 $1 + \varepsilon_{\text{mach}}/2 = 1$ 是同一個現象），而很多更新就是這麼小。
2. float16 要用**損失縮放**（loss scaling）：反向傳播前把損失乘上因子 S，讓小梯度不會下溢到 float16 最小次正規數（6×10^{-8}）以下，更新前再把梯度除以 S。動態損失縮放在出現溢位（inf 或 NaN）時降低 S，並跳過該步。
3. 即使輸入是 float16，內積與歸約也以 float32 **累加**。

bfloat16 的指數與 float32 一樣有 8 位元，因此很少需要損失縮放；Kalamkar 等人發現，bfloat16 訓練在多種模型上都能達到 float32 的結果，而且不需要調整超參數 [5]。代價是精度：$\varepsilon_{\text{mach}} \approx 0.0078$ 的 bfloat16 無法區分 $1 + 0.003$ 與 1，所以長串加總與權重更新仍需要 float32 累加。

損失縮放保護的是 float16 的小梯度；它無法修正錯誤的目標函數，也無法修正網路其他地方的溢位（例如注意力 logit 或正規化統計量）。改變精度也可能改變最佳化軌跡與輸出，特別是歸約運算與長時間訓練。請在目標硬體上驗證；較小的資料型別不保證每個運算都會變快 [13]。

非有限值的除錯順序

損失變成 `nan` 或 `inf` 時，別急著把所有東西都截斷。先找出第一個產生非有限值的運算，再判斷它屬於本篇哪一種失效模式。實用的順序：

1. **輸入：**檢查形狀、範圍、資料型別與缺值。一個 NaN 像素或一張沒有正規化的 0–255 影像，就會改變下游的一切。
2. **前向傳播：**檢查激活值與損失是否有限。留意 `exp`、`log`、除法、`sqrt`，以及變異數接近 0 的正規化層。
3. **損失：**使用以 logit 為輸入的損失（`cross_entropy`、`binary_cross_entropy_with_logits`），而不是先算機率再取 `log`。
4. **梯度：**在小模型上做梯度檢查；查看各層的梯度範數。

5. **更新：**比較更新量與權重的大小；學習率超過曲率極限 $2/\lambda_{\max}$ 時，即使算術完全精確，也會振盪甚至發散。
6. **精度：**先在 float32（或 float64）中重現，再加入混合精度或編譯。

先讓模型過擬合一個極小的批次，確認整條流程能學習。梯度範數截斷（gradient-norm clipping）限制的是傳給最佳化器的梯度，並不直接限制自適應最佳化器的更新量，也無法讓 NaN 梯度變得有意義。ML 02 會更詳細地介紹最佳化器與正規化。

現代觀點

本篇的核心概念已有數十年歷史，但因為模型更大、訓練更久、硬體不斷把精度往下壓，它們仍站在當前研究的中心。

大規模學習的最佳化理論。 Bottou、Curtis 與 Nocedal 發表在 SIAM Review 的綜述 [14] 是這個領域的標準地圖。它解釋為什麼隨機梯度法主導大規模機器學習（資料有冗餘時，便宜而有雜訊的步伐勝過昂貴而精確的步伐），在類似上文 Lipschitz 條件的平滑性假設下分析其收斂，並把改進方法分成兩大類：降低雜訊（更大或逐漸增大的批次、梯度聚合／變異數縮減、迭代平均）與二階方法（Hessian-free 牛頓法、Gauss–Newton、擬牛頓法、自然梯度與對角縮放）。Nocedal 與 Wright [6]、Boyd 與 Vandenberghe [7] 仍是確定性與凸理論的標準教科書；Baydin 等人 [11] 則綜述了讓梯度變得便宜的自動微分。

損失曲面實際長什麼樣子。 Dauphin 等人 [8] 主張，高維非凸最佳化的主要障礙是鞍點，而不是差勁的局部極小值。後來 Cohen 等人 [15] 觀察到，神經網路上的全批次梯度下降通常運作在「穩定性邊緣」（edge of stability）：Hessian 的最大特徵值會上升到剛好略高於 $2/\eta$——正是上文推導的古典穩定極限——而損失在長時間尺度上仍持續下降，只是不再單調。古典的二次分析預測此時會發散，所以如何解釋這個狀態仍是開放的理論問題。

考慮曲率的最佳化器。完整的牛頓法在大規模下不可行，但結構化的近似很實用。K-FAC [16] 以每層小矩陣的 Kronecker 乘積近似 Fisher 資訊矩陣（一種半正定的曲率矩陣，與 Hessian 的 Gauss–Newton 近似密切相關），讓求反矩陣變得便宜。Shampoo [17] 為張量的每個維度各維護一個預條件（preconditioning）矩陣。更近期的 Muon 對每個權重矩陣的更新量做近似正交化；Liu 等人 [18] 報告，在加入權重衰減並仔細調整每個參數的更新尺度後，它在其實驗中以約兩倍於所比較之自適應最佳化器基準的計算效率訓練大型語言模型。這些方法都在重新回答本篇用 Hessian 提出的問題：一步更新應該如何依方向重新縮放？

更低的精度。格式的演進從搭配損失縮放的 float16 [12]、bfloat16 [5]，一路走到 8 位元浮點數。Micikevicius 等人 [19] 提出兩種 FP8 編碼：E4M3（精度較高）與 E5M2（範圍較大），並在影像與語言模型上展示了與 16 位元基準相當的訓練結果。上方表格中的範圍–精度取捨，正是這些設計要平衡的東西，而它們放寬的是 IEEE 754 的規則 [3]。逐張量或逐區塊的縮放因子，扮演著 float16 時代損失縮放的角色。

大規模下的穩定 softmax。減去最大值的技巧有串流版本。Milakov 與 Gimelshein [20] 證明，只要維護一個執行中的最大值，並在最大值改變時重新縮放累加和，就能一趟算出 softmax 的正規化因子。FlashAttention [21] 以此為基礎，在高速的晶片上記憶體中逐區塊計算精確的注意力，從不實際建出完整的注意力矩陣；FlashAttention-3 [22] 進一步延伸到 FP8，並以區塊量化與「incoherent processing」降低量化誤差。這個線上技巧的核心只要幾行：

```
import numpy as np

def online_logsumexp(chunks):
    m, s = -np.inf, 0.0                 # running max and rescaled running sum
    for c in chunks:
        m_new = max(m, c.max())
        s = s * np.exp(m - m_new) + np.exp(c - m_new).sum()
        m = m_new
    return m + np.log(s)

rng = np.random.default_rng(0)
z = rng.normal(0, 50, size=10_000)
chunks = np.array_split(z, 37)          # seen one block at a time
m = z.max()
reference = m + np.log(np.exp(z - m).sum())
print(online_logsumexp(chunks), reference)
assert np.isclose(online_logsumexp(chunks), reference)
```

兩個值都印出 174.0918752579…，在顯示的位數內完全相同，雖然線上版本從未同時看到所有 logit。ML 04 介紹注意力本身，ML 10 介紹部署時的量化。

開放問題。梯度方法為何能在穩定性邊緣成功；如何讓低精度（8 位元及以下）訓練像 float32 一樣穩健，而不必手動調整縮放；如何讓考慮曲率的最佳化器夠便宜、夠穩健，足以取代一階方法成為預設；以及在平行歸約改變加總順序時，如何得到可重現的結果。

重點整理

- 浮點數有固定的相對精度（$\varepsilon_{\text{mach}}$）與固定的範圍；指數位元決定範圍，小數位元決定精度。
- 溢位、下溢與消去都是可預期的。用代數方式修正：`exp` 之前先減最大值、使用 `log1p` ／ `expm1`、使用以 logit 為輸入的損失、平方之前先減平均值。
- 條件數是問題的性質；穩定性是演算法的性質。條件數 10^k 大約會讓你損失 k 位數字。
- Hessian 的特徵值給出曲率。梯度下降只有在 $\eta < 2/\lambda_{\max}$ 時穩定，並會依條件數 $\lambda_{\max}/\lambda_{\min}$ 變慢。
- 牛頓法依曲率重新縮放，在極小值附近收斂很快，但昂貴且會被鞍點吸引；擬牛頓法與 Hessian–向量乘積是實用的折衷。
- 限制條件可以用投影或廣義拉格朗日函數處理；KKT 條件刻畫了解的性質。
- 梯度檢查需要 float64、合理的步長與平滑的測試點，而且它只檢查你實際寫出的程式。
- 混合精度需要 float32 主權重與累加，float16 還需要損失縮放；要驗證，不要假設行為完全相同。

練習

1. **Softmax 平移**。證明 softmax 在 $z_i \mapsto z_i + c$ 下不變，並說明取 $c = -\max_i z_i$ 時分母落在 $[1, K]$ 之間。

提示

共同因子 $\exp(c)$ 在分子與分母間抵銷。平移後每一項 $\exp(z_j - m)$ 都在 $(0, 1]$ 之間，最大的一項等於 1，所以 K 項的總和介於 1 與 K 之間，既不會溢位，也不會下溢成 0。

2. **有限差分掃描**。對 $f(x) = e^x$ 在 $x = 0$ 處，以 $\epsilon = 10^{-1}, \dots, 10^{-12}$ 分別在 float32 與 float64 中計算前向與中央差分。各自在哪個 ϵ 誤差最小？與 $u^{1/2}$、$u^{1/3}$ 比較。

提示

float64 的 $u \approx 1.1 \times 10^{-16}$，所以 $u^{1/2} \approx 10^{-8}$（前向）、$u^{1/3} \approx 5 \times 10^{-6}$（中央）。float32 的 $u \approx 6 \times 10^{-8}$，對應約 2×10^{-4} 與 4×10^{-3}。觀察到的最佳步長應與這些值相差不到一個數量級，最低誤差則大約分別是 $u^{1/2}$ 與 $u^{2/3}$。

3. **穩定極限**。對 $f(w) = \frac{1}{2} w^\top H w$，$H = \mathrm{diag}(2, 50)$，求最大的穩定固定學習率，以及使最壞情況收縮因子最小的步長。用這個最佳步長，需要幾次迭代才能把誤差縮小 10^6 倍？

提示

穩定需要 $\eta < 2/50 = 0.04$。最佳步長為 $2/(\lambda_{\max} + \lambda_{\min}) = 2/52 \approx 0.0385$，收縮因子為 $(\kappa - 1)/(\kappa + 1) = 24/26 \approx 0.923$。解 $0.923^k \le 10^{-6}$：$k \ge \ln 10^6 / (-\ln 0.923) \approx 173$ 次。牛頓法只要一步。

4. **判斷臨界點**。找出 $f(x, y) = x^3 - 3x + y^2$ 的臨界點並加以分類。

提示

$\nabla f = (3x^2 - 3,\ 2y) = 0$ 給出 $(1, 0)$ 與 $(-1, 0)$。Hessian 為 $\mathrm{diag}(6x,\ 2)$：在 $(1, 0)$ 是 $\mathrm{diag}(6, 2)$，正定，為局部極小值；在 $(-1, 0)$ 是 $\mathrm{diag}(-6, 2)$，不定，為鞍點。兩者都不是全域極小值，因為 $x \to -\infty$ 時 $f \to -\infty$。從 $(-1, 0)$ 附近開始的牛頓法會收斂到鞍點。

5. **單形上的 KKT**。在 $\sum_i x_i = 1$ 且 $x_i \ge 0$ 的限制下，對 $c = (0.8, 0.6, -0.4)$ 最小化 $f(x) = \frac{1}{2}\|x - c\|_2^2$。寫出拉格朗日函數與 KKT 條件，並求出解。

提示

$L = \frac{1}{2}\|x - c\|^2 + \lambda(\sum_i x_i - 1) - \sum_i \alpha_i x_i$，$\alpha_i \ge 0$。穩定性給出 $x_i = c_i - \lambda + \alpha_i$；互補鬆弛性給出 $x_i = \max(c_i - \lambda, 0)$。猜測只有前兩個為正：$(0.8 - \lambda) + (0.6 - \lambda) = 1$，所以 $\lambda = 0.2$、$x = (0.6, 0.4, 0)$。驗證：$c_3 - \lambda = -0.6 < 0$，所以 $x_3 = 0$，且 $\alpha_3 = 0.6 \ge 0$。這就是到機率單形的歐氏投影。

6. **錯誤損失的正確梯度**。在梯度檢查的範例中，把 mean 歸約改成 sum。檢查還會通過嗎？自動微分錯了嗎？訓練上還有什麼會改變？

提示

如果同時一致地修改損失與解析梯度，檢查會通過：對新的目標函數而言，導數是正確的。自動微分沒有錯。但梯度變成原來的 N 倍，所以最大的穩定學習率縮小為 $1/N$；在多項損失中，各項之間的平衡也會改變。梯度檢查無法告訴你目標函數選錯了。

參考文獻

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 4 (Numerical Computation) and ch. 8. Book
2. D. Goldberg, "What Every Computer Scientist Should Know About Floating-Point Arithmetic," *ACM Computing Surveys*, vol. 23, no. 1, pp. 5–48, 1991. DOI
3. IEEE, "IEEE Standard for Floating-Point Arithmetic," IEEE Std 754-2019, 2019. DOI
4. N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM, 2002. DOI
5. D. Kalamkar et al., "A Study of BFLOAT16 for Deep Learning Training," arXiv:1905.12322, 2019. arXiv
6. J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., Springer, 2006. DOI
7. S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. Book
8. Y. N. Dauphin, R. Pascanu, C. Gulcehre, K. Cho, S. Ganguli and Y. Bengio, "Identifying and Attacking the Saddle Point Problem in High-Dimensional Non-Convex Optimization," *NeurIPS*, 2014. Paper
9. B. A. Pearlmutter, "Fast Exact Multiplication by the Hessian," *Neural Computation*, vol. 6, no. 1, pp. 147–160, 1994. DOI
10. D. C. Liu and J. Nocedal, "On the Limited Memory BFGS Method for Large Scale Optimization," *Mathematical Programming*, vol. 45, pp. 503–528, 1989. DOI
11. A. G. Baydin, B. A. Pearlmutter, A. A. Radul and J. M. Siskind, "Automatic Differentiation in Machine Learning: a Survey," *Journal of Machine Learning Research*, vol. 18, no. 153, pp. 1–43, 2018. arXiv
12. P. Micikevicius et al., "Mixed Precision Training," *ICLR*, 2018. arXiv
13. PyTorch documentation, "Numerical Accuracy." Documentation
14. L. Bottou, F. E. Curtis and J. Nocedal, "Optimization Methods for Large-Scale Machine Learning," *SIAM Review*, vol. 60, no. 2, pp. 223–311, 2018. DOI
15. J. M. Cohen, S. Kaur, Y. Li, J. Z. Kolter and A. Talwalkar, "Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability," *ICLR*, 2021. arXiv
16. J. Martens and R. Grosse, "Optimizing Neural Networks with Kronecker-factored Approximate Curvature," *ICML*, 2015. Paper
17. V. Gupta, T. Koren and Y. Singer, "Shampoo: Preconditioned Stochastic Tensor Optimization," *ICML*, 2018. Paper
18. J. Liu, J. Su, X. Yao et al., "Muon is Scalable for LLM Training," arXiv:2502.16982, 2025. arXiv
19. P. Micikevicius et al., "FP8 Formats for Deep Learning," arXiv:2209.05433, 2022. arXiv
20. M. Milakov and N. Gimelshein, "Online Normalizer Calculation for Softmax," arXiv:1805.02867, 2018. arXiv
21. T. Dao, D. Y. Fu, S. Ermon, A. Rudra and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," *NeurIPS*, 2022. Paper
22. J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani and T. Dao, "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision," *NeurIPS*, 2024. Paper

ML 02・類神經網路：從感知器到深度網路

從單一感知器到深度網路：激活函數、正規化層、輸出層與損失函數、反向傳播、最佳化器，以及讓網路訓練得起來又能泛化的實務習慣。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-02-neural-networks/

你將學到

- 感知器（perceptron）是什麼、它的學習規則如何運作，以及它和邏輯迴歸（logistic regression）到底差在哪裡。
- 把層疊起來如何得到多層感知器（multilayer perceptron, MLP）、通用近似定理（universal approximation theorem）保證了什麼、沒保證什麼，以及「深」究竟是什麼意思。
- 目前常用的激活函數（sigmoid、tanh、softmax、ReLU、Leaky ReLU、GELU、SiLU/Swish、SwiGLU）與正規化層（BatchNorm、LayerNorm、RMSNorm），連同公式與導數。
- 如何讓輸出層搭配正確的損失函數，以及反向傳播（backpropagation）如何算出每一個梯度：我們會為兩層網路親手推導，並用 NumPy 做數值檢查。
- 實務上怎麼訓練：初始化、SGD、momentum、Adam 與 AdamW、學習率排程、batch size、梯度裁剪（gradient clipping），以及怎麼讀損失曲線。
- 如何診斷欠擬合（underfitting）與過擬合（overfitting）、對付過擬合的正則化工具，以及殘差連接（residual connection）如何一路通往 Transformer。

先看全貌

類神經網路是一個帶有許多可調旋鈕（也就是**參數**）的函數。它接收輸入，例如影像的像素或平面上一個點的兩個座標，然後產生輸出，例如類別標籤。所謂訓練，就是每次把旋鈕轉動一點點，讓模型在已知範例上的輸出越來越接近正確答案。本篇的所有內容都在回答三個問題之一：這個函數長什麼樣子（架構）、怎麼衡量「更接近」（損失函數），以及怎麼把旋鈕轉好（最佳化）。

> **用最簡單的話說**
>
> 想像一排人在傳紙條。第一個人看著一張圖片，寫下幾個簡單的觀察（「這裡有一條弧線」、「這一塊很亮」）。下一個人讀了這些紙條，寫出稍微聰明一點的結論（「這看起來像一隻眼睛」）。最後一個人說「貓」或「狗」。答錯的時候，老師從隊伍尾巴往回走，告訴每個人他的紙條要負多少責任，每個人就把自己的習慣改一點點。往回走的那一趟就是**反向傳播**，重複幾千幾萬次就是**訓練**。

為什麼重要：今天在影像、語音與語言領域的幾乎所有系統，都是由本篇介紹的零件組成的。卷積網路、Transformer 和擴散模型（diffusion model）用的是同一套材料：線性層、非線性激活函數、正規化、殘差連接、損失函數、反向傳播，以及自適應最佳化器。把這些弄清楚之後，深度學習系列談的就只是如何為影像排列這些零件。

關於本篇

本篇由作者先前的學習筆記〈機器學習及類神經網路筆記〉[1] 擴充改寫而成。閱讀前建議先熟悉ML 01 的觀念：資料、特徵、損失函數與梯度下降。

感知器

白話版。感知器是最簡單的人工神經元。它把每個輸入乘上一個權重，加總後再加上偏差（bias），總和為正就回答「是」，否則回答「否」。它只在答錯時才微調權重，藉此學習。

模型

Rosenblatt 在 1958 年提出感知器，作為大腦如何儲存與辨識圖樣的模型 [4]。對輸入向量 $\mathbf{x} \in \mathbb{R}^d$，它計算

$$z = \mathbf{w}^\top \mathbf{x} + b, \qquad \hat{y} = \mathbf{1}[z > 0],$$

其中 $\mathbf{w} \in \mathbb{R}^d$ 是權重向量，$b \in \mathbb{R}$ 是偏差，z 是**激活前的值**（pre-activation，即加權和），$\mathbf{1}[\cdot]$ 是指示函數（條件成立時為 1，否則為 0），$\hat{y} \in \{0, 1\}$ 是預測類別。這個指示函數就是**階梯函數**（step function）。從幾何上看，$\mathbf{w}^\top \mathbf{x} + b = 0$ 是一個超平面（在二維中是一條直線），感知器依照點落在超平面的哪一側來分類。

感知器學習規則

把訓練範例 $(\mathbf{x}_i, y_i)$ 一個一個拿來看。每看一個，就算出 $\hat{y}_i$ 並更新

$$\mathbf{w} \leftarrow \mathbf{w} + \eta\,(y_i - \hat{y}_i)\,\mathbf{x}_i, \qquad b \leftarrow b + \eta\,(y_i - \hat{y}_i),$$

其中 $\eta > 0$ 是**學習率**（步長），$y_i \in \{0, 1\}$ 是真實標籤。預測正確時 $y_i - \hat{y}_i = 0$，什麼都不變。如果感知器答 0 但正確答案是 1，權重會往 $\mathbf{x}_i$ 的方向移動，下次遇到這個輸入時 z 就會變大；反方向的錯誤則讓權重遠離它。當兩個類別可以被一個超平面分開（資料**線性可分**）時，這個規則在有限次更新後就不會再犯錯；如果不可分，它永遠不會停下來。

```
import numpy as np

def perceptron_train(X, y, epochs=20, eta=1.0):
    """X: (N, d)，y 為 0 或 1。回傳 w, b。"""
    w, b = np.zeros(X.shape[1]), 0.0
    for _ in range(epochs):
        mistakes = 0
        for x_i, y_i in zip(X, y):
            y_hat = float(w @ x_i + b > 0)       # 階梯函數
            if y_hat != y_i:                     # 只在答錯時更新
                w += eta * (y_i - y_hat) * x_i
                b += eta * (y_i - y_hat)
                mistakes += 1
```

```
        if mistakes == 0:
            break
    return w, b

# AND 線性可分；XOR 不是
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
for name, y in [("AND", [0, 0, 0, 1]), ("XOR", [0, 1, 1, 0])]:
    w, b = perceptron_train(X, np.array(y, float))
    print(name, (X @ w + b > 0).astype(int))
# AND [0 0 0 1]
# XOR [1 1 0 0]   <- 錯了：沒有任何一條直線能做到
```

感知器與邏輯迴歸的差別

這兩個模型很容易混淆，因為第一步完全相同，都是 $z = \mathbf{w}^\top \mathbf{x} + b$。但它們是不同的方法：

	感知器	邏輯迴歸
輸出函數	階梯：$\hat{y} = \mathbf{1}[z > 0]$	sigmoid：$\hat{p} = \sigma(z) = \dfrac{1}{1+e^{-z}}$
輸出意義	0/1 的硬決策	機率 $P(y=1 \mid \mathbf{x})$
訓練方式	感知器規則，只在答錯時更新	以梯度下降最小化交叉熵（最大概似）
資料不可分時	永不收斂	收斂到定義明確的最佳解

邏輯迴歸對單一範例的損失梯度是 $(\hat{p}_i - y_i)\,\mathbf{x}_i$，所以梯度下降的一步是 $\mathbf{w} \leftarrow \mathbf{w} + \eta\,(y_i - \hat{p}_i)\,\mathbf{x}_i$。這看起來幾乎和感知器規則一模一樣，但 $\hat{p}_i$ 是介於 0 與 1 之間的機率，因此**每一個**範例都有貢獻，而且非常有把握又答對的範例貢獻很小。正是這種平滑性，讓邏輯迴歸可以用梯度下降訓練，也讓它能被疊成更大的可微分網路。相對地，階梯函數幾乎處處導數為零，梯度根本無法通過它。

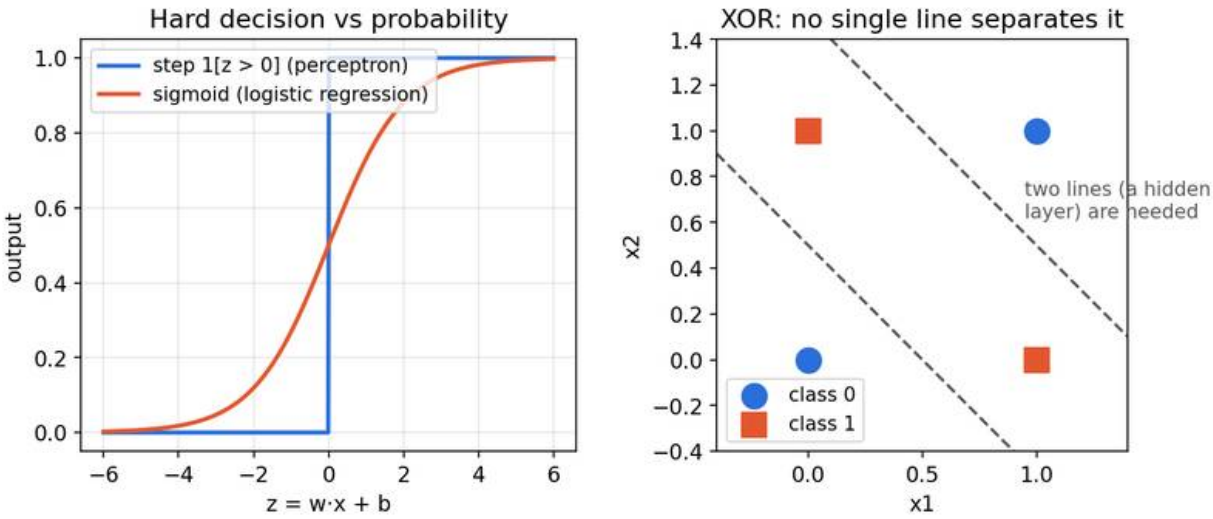


圖 1 — 左：感知器的階梯輸出與邏輯迴歸的 *sigmoid* 機率。右：*XOR*，任何單一線性邊界都無法解決的經典問題；需要兩條邊界（也就是一個隱藏層）。

多層感知器（MLP）

白話版。一個神經元只能畫一條直線。把很多神經元並排就有很多條直線，再把它們的輸出送進下一層組合起來，組合的結果可以彎曲，網路就能切出彎曲的區域。

把層疊起來

只有一個隱藏層的 MLP 計算

$$\mathbf{h} = f(W_1\mathbf{x} + \mathbf{b}_1),$$
$$\hat{\mathbf{y}} = g(W_2\mathbf{h} + \mathbf{b}_2),$$

其中 $\mathbf{x} \in \mathbb{R}^d$ 是輸入，$W_1 \in \mathbb{R}^{m\times d}$ 與 $\mathbf{b}_1 \in \mathbb{R}^m$ 是第一層的權重與偏差，m 是**寬度**（隱藏單元數），f 是逐元素作用的非線性**激活函數**（activation function），$\mathbf{h} \in \mathbb{R}^m$ 是隱藏表示，$W_2 \in \mathbb{R}^{K\times m}$ 與 $\mathbf{b}_2 \in \mathbb{R}^K$ 把它映射到 K 個輸出，g 則是依任務選擇的輸出函數（見後文）。每個隱藏單元都是一個「類似邏輯迴歸」的神經元；「多層感知器」這個名字是歷史沿用，現代 MLP 用的是平滑的激活函數，不是階梯函數。

非線性 f 是關鍵。少了它，$W_2(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (W_2W_1)\mathbf{x} + (W_2\mathbf{b}_1 + \mathbf{b}_2)$ 又只是一個線性映射：再多層的線性層都會塌縮成一層，網路依然只能畫一個超平面。以 XOR 為例，兩個隱藏單元可以各畫出圖 1 中的一條虛線，輸出單元再回答「是否落在兩條線之間」。

通用近似

Cybenko 在 1989 年證明：只要隱藏層夠寬，一**個**由 sigmoid 型單元組成的隱藏層，就能在有界的方塊（單位超立方體）上以任意精度逼近任何連續函數 [5]。後續研究把結果推廣到其他激活函數，包括 ReLU。這個定理令人安心，但要仔細讀：

- 它講的是**存在性**。它只說合適的權重存在，完全沒說梯度下降找不找得到。
- 它沒說「夠寬」到底要**多寬**。對某些函數，所需的寬度會隨輸入維度極快地成長。
- 它沒談**泛化**：把訓練點擬合得很好，不代表能準確預測新的點。

所以這個定理說明了 MLP 在原理上夠有彈性，而本篇其餘部分談的，正是它留下來的那些實務問題。

深度神經網路：深度與寬度

白話版。「深」只是代表依序疊了很多層。並沒有一個官方的層數門檻，超過就算深。

深度神經網路（deep neural network, DNN）是具有多個隱藏層的 MLP 或其他層狀網路：$\mathbf{h}_\ell = f(W_\ell\mathbf{h}_{\ell-1} + \mathbf{b}_\ell)$，$\ell = 1, \dots, L$，且 $\mathbf{h}_0 = \mathbf{x}$。「深」沒有公認的門檻。常見的非正式說法是隱藏層超過一層就算深，但各教材計算總層數或隱藏層數的方式不同，所以「三層」在不同書裡指的東西也不一樣。重要的是概念而不是數字：深度網路是把很多簡單步驟**組合**（composition）起來得到答案。

增加模型能力有兩種方式：**寬度**（每層更多單元）與**深度**（更多層）。兩者並不等價。加寬是多放幾塊只組合一次的零件；加深則讓網路重複利用前面各層的特徵，建構出更抽象的特徵。LeCun、Bengio 與 Hinton 的回顧論文 [2] 正是以此解釋深度學習的成功：每一層都把前一層的表示轉換成稍微抽象一點的表示，疊很多這樣的層，網路就能直接從原始資料學到非常複雜的函數，而且特徵是學出來的，不是人工設計的。

從歷史來看，1990 年代以**淺層**方法為主流。支持向量機（SVM）是當時的主要工具，它是一種核方法（kernel method），並不是很寬的類神經網路；那時深度網路很難訓練。從 2000 年代中期開始，更多的資料、GPU 與更好的訓練技巧讓深度變得可行，深度網路開始在一個又一個基準測試中勝出 [2]。不過深度並不是全部：單純疊更多普通的層，最後反而會讓訓練**變差**，這個問題要靠殘差連接才得以解決（見最後一個主要章節）。

圖 2 以 scikit-learn 的雙月形（two moons）資料集展示寬度與深度的作用。一層只有 2 或 8 個隱藏單元時，邊界只由少數幾段直線組成，無法沿著月牙走。更寬或更深的網路能把邊界彎曲到貼合資料；最寬又最深的那個甚至繞著個別的雜訊點打轉，這是過擬合的第一個徵兆。

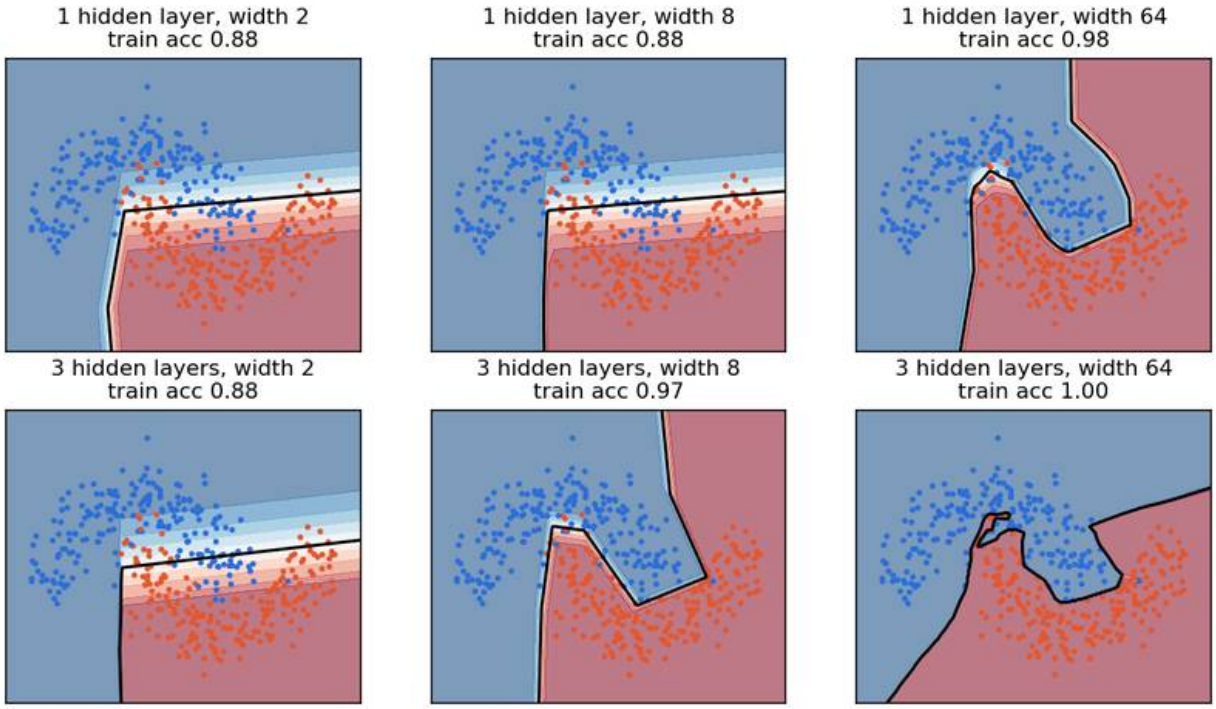


圖 *2 — ReLU MLP 在 make_moons*（*400* 點、雜訊 *0.2*）上的決策邊界，欄為寬度、列為深度，每個模型都以 *Adam* 做 *1,500* 步全批次訓練。黑色曲線是機率 *0.5* 的等高線。

激活函數

白話版。激活函數是每次加權和之後加上的「彎曲」。沒有它，整個網路就只是直線模型。不同的彎曲方式在平滑度、速度，以及梯度能否順利通過很多層之間取捨。

圖 3 畫出常見的選擇與它們的導數。導數和函數本身一樣重要，因為反向傳播會一層一層地把導數乘起來（見訓練章節）。

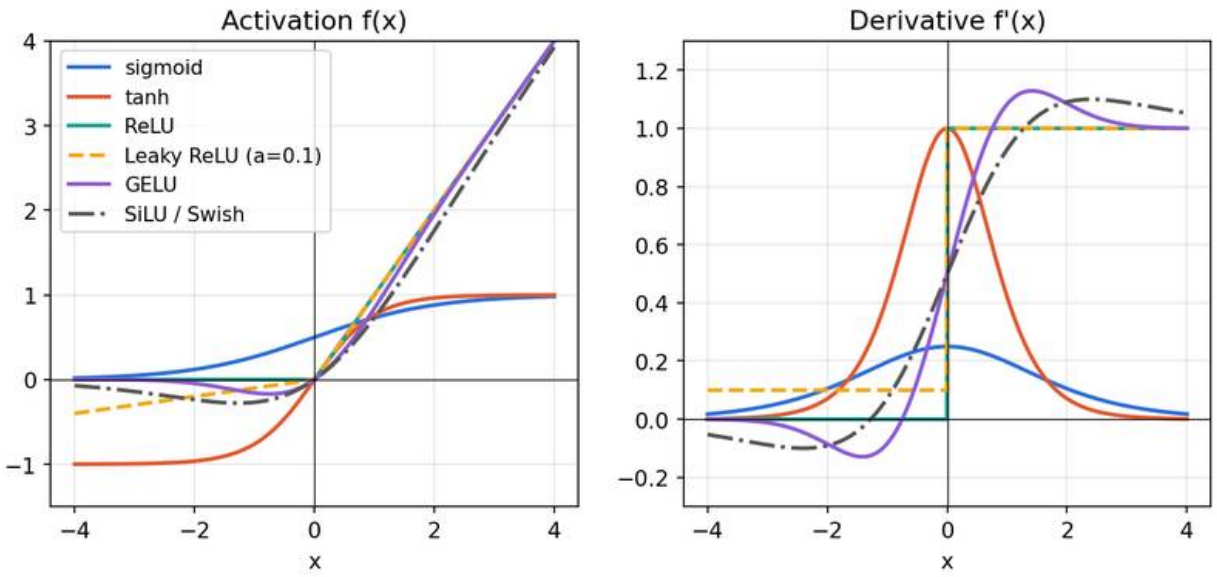


圖 3 — 常見激活函數（左）與其導數（右）。注意 *sigmoid* 的導數處處都很小（最多 *0.25*），而 *sigmoid* 和 *tanh* 在輸入很大時都變得非常平坦。

Sigmoid 與 tanh

$$\sigma(x) = \frac{1}{1+e^{-x}}, \quad \sigma'(x) = \sigma(x)(1-\sigma(x)); \qquad \tanh(x) = 2\sigma(2x) - 1, \quad \tanh'(x) = 1 - \tanh^2(x).$$

Sigmoid 把任何實數映射到 $(0,1)$；tanh 映射到 $(-1,1)$，並以零為中心。兩者都會**飽和**(saturate)：$|x|$ 很大時曲線很平，導數幾乎為零，梯度幾乎過不去。Sigmoid 的導數最大只有 $1/4$（在 $x=0$）。這兩件事讓深層 sigmoid 網路很難訓練，Glorot 與 Bengio 對此有詳細分析 [17]。今天 sigmoid 主要用在二元分類器的**輸出**與門控機制（例如下面的 SwiGLU），tanh 則出現在某些遞迴網路中。

Softmax

Softmax 把 K 個實數分數組成的向量 $\mathbf{z}$（稱為 **logits**）轉成機率分布：

$$p_k = \mathrm{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad \frac{\partial p_k}{\partial z_j} = p_k(\delta_{kj} - p_j),$$

其中 $k = j$ 時 δ_{kj} 為 1，否則為 0。每個 p_k 都是正數，而且總和為 1。和本節其他函數不同，softmax 作用在**整個向量**上，而不是逐元素作用，所以幾乎只用在多類別分類器的輸出層（以及 attention 內部），不當作隱藏層的激活函數。寫程式時，先減去 $\max_j z_j$ 再取指數；結果不變，但可以避免溢位。

ReLU 與 Leaky ReLU

$$\mathrm{ReLU}(x) = \max(0, x), \quad \mathrm{ReLU}'(x) = \mathbf{1}[x > 0]; \qquad \mathrm{LeakyReLU}_a(x) = \max(ax, x), \quad 0 < a < 1.$$

整流線性單元（rectified linear unit, ReLU）讓正的輸入原封不動通過，其餘歸零。它對所有正輸入的導數恰好是 1，所以梯度通過活躍單元時不會縮小，而且計算成本極低。ReLU 是讓深層監督式網路不必靠非監督式預訓練也能訓練起來的改變之一 [2]。它的弱點是**死亡 ReLU**（dying ReLU）：如果某個單元對所有範例的輸入都是負的，它的梯度就是零，從此不再學習。Leaky ReLU 在負側保留一個小斜率 a（例如 0.01），讓一些梯度永遠能流過。Dubey、Singh 與 Chaudhuri 的激活函數綜述 [7] 把這些以及後來的許多變體歸為 ReLU 家族。

GELU 與 SiLU/Swish

這些是 ReLU 的平滑版本，會讓小的負值通過：

$$\mathrm{GELU}(x) = x\,\Phi(x), \qquad \mathrm{GELU}'(x) = \Phi(x) + x\,\varphi(x),$$

其中 Φ 是標準常態分布的累積分布函數（CDF），φ 是它的密度函數 [8]。GELU 不像 ReLU 依正負號硬切，而是依輸入的大小來加權：$\Phi(x)$ 是標準常態隨機變數小於 x 的機率。常用的快速近似是 $0.5x(1 + \tanh[\sqrt{2/\pi}\,(x + 0.044715x^3)])$ [8]。GELU 是許多 Transformer 模型的預設激活函數。

$$\mathrm{Swish}_\beta(x) = x\,\sigma(\beta x), \qquad \mathrm{SiLU}(x) = x\,\sigma(x), \qquad \mathrm{SiLU}'(x) = \sigma(x)(1 + x(1 - \sigma(x))).$$

sigmoid 加權線性單元（SiLU）由 Elfwing、Uchibe 與 Doya 為強化學習提出 [9]。Ramachandran、Zoph 與 Le 以自動搜尋找到同樣的形狀，β 可固定或可學習，並稱之為 Swish [10]；$\beta = 1$ 時 Swish 就是 SiLU。GELU 和 SiLU 長得非常像（圖 3）。兩者都不是單調函數：對中等大小的負輸入，它們會稍微低於零。

SwiGLU：帶門控的前饋區塊

門控線性單元（gated linear unit, GLU）把兩個線性投影逐元素相乘，其中一個先通過一個像閘門一樣的非線性函數。Shazeer 在 Transformer 的前饋（feed-forward）區塊中測試了幾種變體 [11]。SwiGLU 版本以論文的列向量寫法、且省略偏差，寫成

$$\mathrm{FFN}_{\mathrm{SwiGLU}}(\mathbf{x}) = \left(\mathrm{Swish}_1(\mathbf{x}W) \odot \mathbf{x}V\right) W_2,$$

其中 $\mathbf{x}$ 是大小為 d_{model} 的列向量，W 與 V 是 $d_{\text{model}} \times d_{\text{ff}}$ 矩陣，W_2 是 $d_{\text{ff}} \times d_{\text{model}}$ 矩陣，$\odot$ 是逐元素相乘。在 Shazeer 的實驗中，包括 SwiGLU 與 GEGLU 在內的 GLU 變體，表現都優於該區塊的 ReLU 與 GELU 版本。由於 SwiGLU 有三個權重矩陣而不是兩個，d_{ff} 要縮小為原本的 2/3，參數量才會相同 [11]。例如 LLaMA 就以 SwiGLU 取代 ReLU，隱藏維度用 $\frac{2}{3} \cdot 4d$ [12]；SwiGLU 已成為大型語言模型的常見預設。

激活函數	值域	平滑	目前的典型用途
sigmoid	$(0, 1)$	是	二元／多標籤輸出、門控
tanh	$(-1, 1)$	是	某些遞迴網路
softmax	機率向量	是	多類別輸出、attention 權重

激活函數	值域	平滑	目前的典型用途
ReLU / Leaky ReLU	$[0,\infty)$ / $\mathbb{R}$	否（0 處有折角）	CNN、MLP，安全的預設
GELU、SiLU	約 $[-0.17,\infty)$、$[-0.28,\infty)$	是	Transformer、現代 CNN
SwiGLU	$\mathbb{R}$	是	Transformer 的前饋區塊

正規化層

白話版。訓練進行中，網路內部流動的數值可能飄到很大或很小的尺度，讓學習變得不穩定。正規化層（normalization）在每一步把它們重新縮放到標準大小，再讓網路自己選擇偏好的最終尺度。

三種常見的正規化層共用同一個模板。對一組數值 $x_1, \dots, x_n$，

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad \sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i-\mu)^2, \qquad y_i = \gamma\,\frac{x_i-\mu}{\sqrt{\sigma^2+\epsilon}} + \beta,$$

其中 μ 與 σ^2 是平均與變異數，ϵ 是為了數值安全的小常數，γ, β 是可學習的縮放與平移（如果有幫助，網路可以藉此還原正規化）。三者的差別在於**哪些數值**被放在同一組（圖 4）。

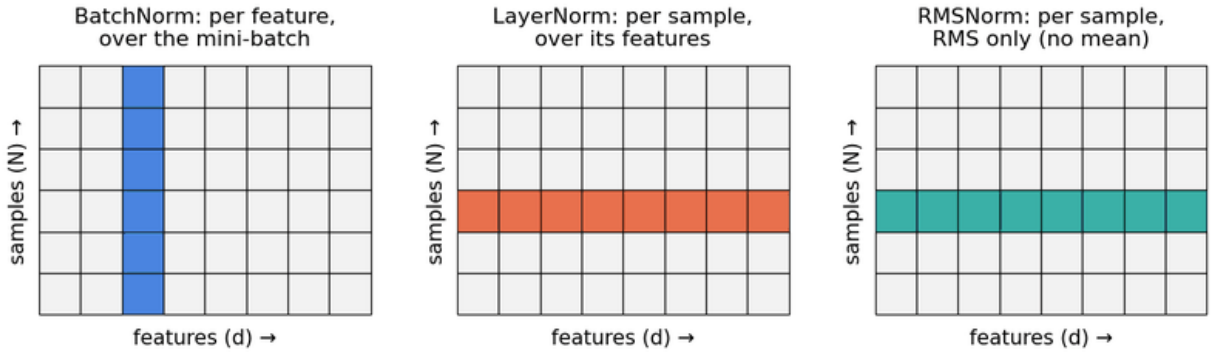


圖 4 — 哪些元素共用統計量。*BatchNorm* 對每個特徵在 *mini-batch* 上正規化；*LayerNorm* 與 *RMSNorm* 對每個樣本在它自己的特徵上正規化。

- **BatchNorm**（Ioffe 與 Szegedy [13]）對每個特徵（在 CNN 中是每個通道）**跨 mini-batch** 計算 μ, σ^2。它能加快訓練，成為 CNN 的標準配置。由於統計量取決於 batch，它在訓練與推論時的行為不同：推論時改用訓練期間累積的移動平均。batch 很小時統計量變得很吵，效果也會變差。
- **LayerNorm**（Ba、Kiros 與 Hinton [14]）**對單一樣本的所有特徵**計算 μ, σ^2。它與 batch 無關，訓練和測試時行為一致，也很自然地適用於序列。它是 Transformer 的標準正規化方式。
- **RMSNorm**（Zhang 與 Sennrich [15]）省略減去平均的步驟，只除以均方根：$y_i = \gamma_i\, x_i / \sqrt{\frac{1}{n}\sum_j x_j^2 + \epsilon}$。作者假設 LayerNorm 的重新置中（re-centering）是可有可無的，並報告品質相當而執行時間更短 [15]。LLaMA 採用 RMSNorm，並把它放在每個 Transformer 子層的**輸入端**（pre-normalization），而不是輸出端 [12]。

```
import torch

x = torch.randn(4, 8) * 5 + 3                          # 4 個樣本、8 個特徵
ln = torch.nn.LayerNorm(8, elementwise_affine=False)
bn = torch.nn.BatchNorm1d(8, affine=False)
rms = x / torch.sqrt(x.pow(2).mean(dim=1, keepdim=True) + 1e-6)

print(ln(x).mean(dim=1))     # 每個樣本（列）約為 0
print(bn(x).mean(dim=0))     # 每個特徵（欄）約為 0（訓練模式）
print(rms.pow(2).mean(dim=1))  # 每個樣本約為 1；RMS 為 1，但平均沒有被移除
```

Huang 等人 [16] 的綜述整理了這個家族，並把任何正規化方法拆成三個選擇：哪些數值放在一起（正規化區域）、做什麼運算（標準化、置中、縮放或白化），以及之後如何還原表示（可學習的仿射步驟）。BatchNorm、LayerNorm 與 RMSNorm 主要差在前兩項。

設計輸出層與選擇損失函數

白話版。最後一層必須說答案的語言：數量就輸出任意實數，是非題就輸出機率，「K 選一」就輸出機率分布。每種輸出都有一個天生搭配的損失函數，下表就是該用的組合。

任務	輸出層	輸出範圍	損失函數
迴歸	線性（不加激活函數）	$\mathbb{R}$	均方誤差（MSE，L2）或平均絕對誤差（MAE，L1）
二元分類	1 個單元 + sigmoid	$(0,1)$	二元交叉熵（binary cross-entropy）
多標籤（多個是非題）	K 個單元，各接 sigmoid	$(0,1)^K$	每個標籤各算二元交叉熵
多類別（K 選一）	K 個單元 + softmax	機率向量	交叉熵（cross-entropy）

常見的錯誤是在迴歸輸出接 sigmoid。sigmoid 的值域是開區間 $(0,1)$，它的工作是為二元或多標籤分類輸出機率。迴歸的輸出層通常是**線性**的，才能輸出任何實數。只有當目標值本身已知落在 0 到 1 之間（例如正規化後的像素強度或比例）時，迴歸輸出接 sigmoid 才合理。

對單一範例，損失函數為

$$\begin{aligned} \text{MSE：} \quad & \ell = \tfrac{1}{2}\|\hat{\mathbf{y}} - \mathbf{y}\|^2, \\ \text{二元交叉熵：} \quad & \ell = -[\, y \log \hat{p} + (1-y)\log(1-\hat{p}) \,], \\ \text{交叉熵：} \quad & \ell = -\sum_{k=1}^{K} y_k \log p_k = -\log p_c, \end{aligned}$$

其中 $\mathbf{y}$ 是目標，$\hat{p} = \sigma(z)$，$\mathbf{p} = \mathrm{softmax}(\mathbf{z})$，$y_k$ 是 one-hot 標籤，c 是正確類別。（MSE 的 $\frac{1}{2}$ 只是為了方便；PyTorch 的 `MSELoss` 取平均且不含這個因子。）MAE 即 $\|\hat{\mathbf{y}} - \mathbf{y}\|_1$，比 MSE 更不受離群值影響。每一組搭配都是最大概似估計：MSE 對應目標上的高斯雜訊，二元交叉熵對應 Bernoulli 標籤，交叉熵對應類別（categorical）標籤 [3]。

這些搭配還有一個漂亮的實務理由。三種情況下，損失對輸出層**激活前的值**的梯度都只是

$$\frac{\partial \ell}{\partial \mathbf{z}} = \hat{\mathbf{y}} - \mathbf{y} \quad \text{（預測減目標）},$$

這裡的 $\hat{\mathbf{y}}$ 分別指線性輸出、$\sigma(z)$ 或 softmax$(\mathbf{z})$。會飽和的 sigmoid 或 softmax 導數被抵消掉了，所以即使模型非常有把握地答錯，梯度仍然很大。（練習 3 請你為 softmax 驗證這件事。）這也是為什麼框架會把最後的激活函數併進損失函數：PyTorch 的 `nn.CrossEntropyLoss` 與 `nn.BCEWithLogitsLoss` 接收的是 **logits**，所以模型本身在訓練時不能以 softmax 或 sigmoid 結尾。

訓練（一）：前向傳遞與反向傳播

白話版。前向傳遞（forward pass）從輸入到輸出逐層算出預測與損失。反向傳播再從損失往回走到輸入，用連鎖律（chain rule）算出每個權重對誤差的貢獻。它會重複利用中間結果，所以算出所有梯度的成本大約只等於多做一次前向傳遞。

Rumelhart、Hinton 與 Williams 讓反向傳播成為訓練多層網路的普及方法 [6]。它其實就是微積分的連鎖律，只是安排得讓任何計算都不重複。圖 5 畫出我們要微分的網路的計算圖。

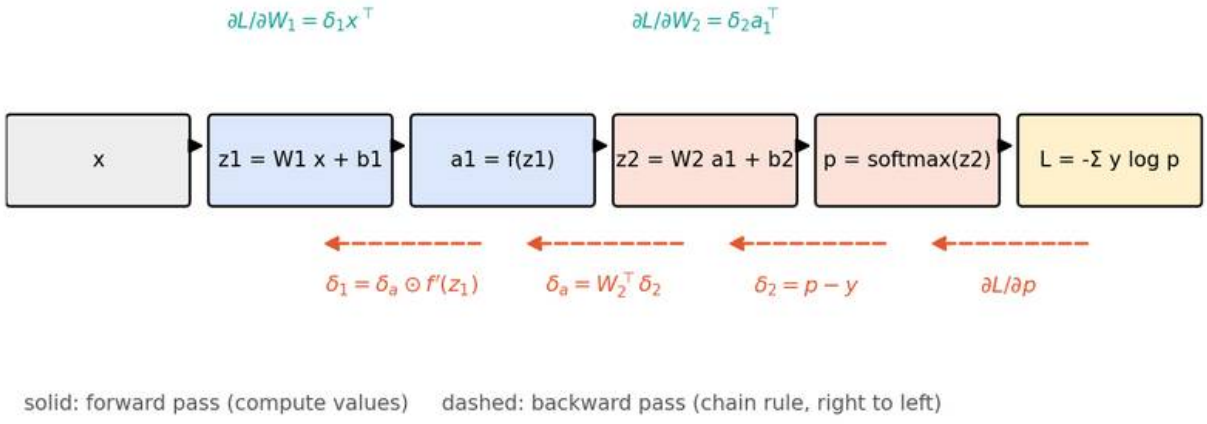


圖 5 — 搭配 *softmax* 交叉熵的兩層 *MLP* 的計算圖。實線：前向傳遞。虛線：反向傳遞，把誤差訊號 δ 從損失往輸入方向傳回去。

兩層 MLP 的推導

取一個範例 $(\mathbf{x}, \mathbf{y})$，$\mathbf{x} \in \mathbb{R}^d$，$\mathbf{y} \in \{0,1\}^K$ 是 one-hot 標籤。前向傳遞為

$$\begin{aligned} \mathbf{z}_1 &= W_1\mathbf{x} + \mathbf{b}_1 \in \mathbb{R}^m, \quad \mathbf{a}_1 = f(\mathbf{z}_1) \in \mathbb{R}^m, \\ \mathbf{z}_2 &= W_2\mathbf{a}_1 + \mathbf{b}_2 \in \mathbb{R}^K, \quad \mathbf{p} = \mathrm{softmax}(\mathbf{z}_2), \qquad L = -\textstyle\sum_k y_k \log p_k, \end{aligned}$$

其中 $W_1 \in \mathbb{R}^{m\times d}$、$W_2 \in \mathbb{R}^{K\times m}$，$f$ 是逐元素的激活函數。定義某一層的**誤差訊號**為損失對該層激活前的值的梯度：$\delta_\ell = \partial L/\partial \mathbf{z}_\ell$。

步驟 1：輸出層。由上一節可知，softmax 搭配交叉熵給出

$$\delta_2 = \frac{\partial L}{\partial \mathbf{z}_2} = \mathbf{p} - \mathbf{y} \in \mathbb{R}^K.$$

步驟 2：輸出層權重。因為 $z_{2,k} = \sum_j W_{2,kj}\, a_{1,j} + b_{2,k}$，所以 $\partial z_{2,k}/\partial W_{2,kj} = a_{1,j}$，於是

$$\frac{\partial L}{\partial W_2} = \delta_2\, \mathbf{a}_1^\top \in \mathbb{R}^{K\times m}, \qquad \frac{\partial L}{\partial \mathbf{b}_2} = \delta_2.$$

一個權重的梯度等於（它輸出端的誤差）乘以（它輸入端的激活值）。形狀和 W_2 相同，這是很好用的檢查。

步驟 3：往回穿過線性層。每個隱藏激活值 $a_{1,j}$ 都影響所有輸出，所以它的梯度要對所有輸出加總：$\partial L/\partial a_{1,j} = \sum_k \delta_{2,k} W_{2,kj}$，寫成矩陣形式為

$$\frac{\partial L}{\partial \mathbf{a}_1} = W_2^\top \delta_2 \in \mathbb{R}^m.$$

步驟 4：往回穿過激活函數。因為 f 逐元素作用，它的 Jacobian 是對角矩陣，連鎖律就變成逐元素相乘：

$$\delta_1 = \frac{\partial L}{\partial \mathbf{z}_1} = (W_2^\top \delta_2) \odot f'(\mathbf{z}_1).$$

步驟 5：第一層權重。和步驟 2 完全一樣，

$$\frac{\partial L}{\partial W_1} = \delta_1\, \mathbf{x}^\top \in \mathbb{R}^{m\times d}, \qquad \frac{\partial L}{\partial \mathbf{b}_1} = \delta_1.$$

更深的網路只是重複步驟 3 到 5：$\delta_\ell = (W_{\ell+1}^\top \delta_{\ell+1}) \odot f'(\mathbf{z}_\ell)$，以及 $\partial L/\partial W_\ell = \delta_\ell\, \mathbf{a}_{\ell-1}^\top$。對 N 個範例組成的 mini-batch，損失取平均，所以每個梯度也是各範例梯度的平均。

用 NumPy 從零實作，並做梯度檢查

下面的程式把範例存成**列**（`X` 的形狀是 $N \times d$），這是程式中常見的慣例。於是 `X @ W1`（`W1` 形狀為 $d\times m$）扮演數學式中 $W_1\mathbf{x}$ 的角色，上面每個公式在程式裡都以轉置的形式出現：例如 `dW2 = A1.T @ dZ2` 就是把 $\delta_2\mathbf{a}_1^\top$ 沿 batch 疊起來。**梯度檢查**把一個解析導數和中央差分 $(L(w+\epsilon)-L(w-\epsilon))/2\epsilon$ 比較；如果兩者在很多位數上都一致，反向傳遞就是對的。只要親手寫梯度，就一定要做這個檢查。

```
import numpy as np
from sklearn.datasets import make_moons

rng = np.random.default_rng(0)
X, y = make_moons(n_samples=200, noise=0.2, random_state=0)
N, d, h, K = X.shape[0], 2, 16, 2
Y = np.eye(K)[y]                                  # one-hot 目標，(N, K)

# He 式初始化（標準差 = sqrt(2 / fan_in)）
W1 = rng.normal(0, np.sqrt(2 / d), (d, h)); b1 = np.zeros(h)
W2 = rng.normal(0, np.sqrt(2 / h), (h, K)); b2 = np.zeros(K)
```

```
def forward(X, W1, b1, W2, b2):
    Z1 = X @ W1 + b1                              # (N, h)
    A1 = np.maximum(Z1, 0)                        # ReLU
    Z2 = A1 @ W2 + b2                             # (N, K) logits
    Z2 = Z2 - Z2.max(axis=1, keepdims=True)       # 數值穩定
    P = np.exp(Z2) / np.exp(Z2).sum(axis=1, keepdims=True)
    loss = -np.mean(np.sum(Y * np.log(P + 1e-12), axis=1))
    return Z1, A1, P, loss

def backward(X, Z1, A1, P, W2):
    dZ2 = (P - Y) / N                             # 輸出層的 delta
    dW2 = A1.T @ dZ2;  db2 = dZ2.sum(axis=0)
    dA1 = dZ2 @ W2.T
    dZ1 = dA1 * (Z1 > 0)                          # ReLU'(z) = 1[z > 0]
    dW1 = X.T @ dZ1;   db1 = dZ1.sum(axis=0)
    return dW1, db1, dW2, db2

# 對 W1 的一個元素做梯度檢查（中央差分）
Z1, A1, P, loss = forward(X, W1, b1, W2, b2)
dW1, db1, dW2, db2 = backward(X, Z1, A1, P, W2)
eps = 1e-6; E = np.zeros_like(W1); E[0, 3] = eps
num = (forward(X, W1 + E, b1, W2, b2)[3] - forward(X, W1 - E, b1, W2, b2)[3]) / (2 * eps)
print(f"analytic {dW1[0, 3]:.8f}  numeric {num:.8f}")

# 單純的梯度下降
lr = 0.5
for step in range(2000):
    Z1, A1, P, loss = forward(X, W1, b1, W2, b2)
    dW1, db1, dW2, db2 = backward(X, Z1, A1, P, W2)
    W1 -= lr * dW1; b1 -= lr * db1; W2 -= lr * dW2; b2 -= lr * db2
acc = (P.argmax(axis=1) == y).mean()
print(f"final loss {loss:.3f}, train accuracy {acc:.3f}")
# analytic -0.01941581  numeric -0.01941581
# final loss 0.063, train accuracy 0.970
```

實務上你永遠不必自己寫反向傳遞：PyTorch 等框架會記錄前向計算，在你呼叫 `loss.backward()` 時自動執行完全相同的程序。但親手寫過一次，仍是理解這個呼叫在做什麼、以及為什麼計算圖中某處導數為零時它會失靈的最好方法。

訓練（二）：讓最佳化真正可行

白話版。只知道梯度還不夠。你還需要一個合理的起點（初始化）、把梯度變成步伐的規則（最佳化器）、隨時間改變的步長（排程）、每一步看多少範例的選擇（batch size），以及防止偶發超大步伐的保護（裁剪）。

梯度消失與梯度爆炸

把遞迴式 $\delta_\ell = (W_{\ell+1}^\top \delta_{\ell+1}) \odot f'(\mathbf{z}_\ell)$ 沿 L 層展開，第一層的梯度會包含 L 個權重矩陣與 L 個激活函數導數的**連乘**。如果典型的因子小於 1，連乘會隨深度指數縮小，前面的層幾乎學不到東西：這就是**梯度消失**（vanishing gradient）。用 sigmoid 時每個 f' 最多 0.25，十層就可能把訊號縮小到 $0.25^{10} \approx 10^{-6}$。如果典型因子大於 1，例如初始權重太大，連乘會指數成長，更新變得巨大，損失變成 `inf` 或 `NaN`：這就是**梯度爆炸**（exploding gradient）。

下面介紹的補救方法，從不同方向處理這個連乘：

- 在大範圍內導數為 1 的激活函數（ReLU 與它的平滑親戚）；
- 讓訊號大小逐層保持不變的初始化；
- 重新標準化激活值的正規化層；
- **殘差連接**，給梯度一條因子為 1 的通道（最後一個主要章節）；
- **梯度裁剪**，對付爆炸的標準緊急煞車。

有時也會列出其他做法，例如微調（fine-tuning）預訓練模型或改用較小的網路；它們讓整體訓練變得比較容易，但並不直接解決梯度問題。

權重初始化

如果所有權重一開始都相同，每個隱藏單元算的東西都一樣、拿到的梯度也一樣，它們就永遠一模一樣。所以權重要隨機初始化。尺度很重要：我們希望激活值（前向）與梯度（反向）的變異數在層與層之間大致維持不變。對有 n_{in} 個輸入、n_{out} 個輸出的線性層：

- **Xavier（Glorot）初始化** [17] 為 tanh 等對稱激活函數平衡前向與反向條件：$\mathrm{Var}(W_{ij}) = \dfrac{2}{n_{\text{in}} + n_{\text{out}}}$，例如在 $[-\sqrt{6/(n_{\text{in}}+n_{\text{out}})}, \sqrt{6/(n_{\text{in}}+n_{\text{out}})}]$ 上均勻分布。
- **He（Kaiming）初始化** [18] 考慮到 ReLU 會把約一半的輸入歸零，使每層的變異數減半，因此把尺度加倍：$\mathrm{Var}(W_{ij}) = \dfrac{2}{n_{\text{in}}}$，例如 $W_{ij} \sim \mathcal{N}(0, 2/n_{\text{in}})$。

偏差通常從零開始。He 等人證明這種初始化能讓非常深的 ReLU 網路從零開始訓練 [18]。上面的 NumPy 範例就是用它。

最佳化器

下面每個最佳化器都是**隨機梯度下降**（stochastic gradient descent, SGD）的變體：在一個小的隨機 mini-batch 上估計梯度，往反方向走一步，然後重複。記 $\mathbf{g}_t = \nabla_\theta L_{\text{batch}}(\theta_t)$ 為第 t 步的 mini-batch 梯度，θ 為所有參數，η 為學習率。

SGD。 $\theta_{t+1} = \theta_t - \eta\, \mathbf{g}_t$。簡單、不需額外記憶體，但在狹長的山谷中會來回鋸齒狀前進，在平坦方向上又走得很慢。

SGD with momentum。維持一個移動的速度：$\mathbf{v}_{t+1} = \mu\, \mathbf{v}_t + \mathbf{g}_t$，$\theta_{t+1} = \theta_t - \eta\, \mathbf{v}_{t+1}$，$\mu$ 約為 0.9。就像一顆沉重的球滾下山坡，它把鋸齒平均掉，並在方向一致時累積速度。

Adam（Kingma 與 Ba [19]）為每個參數維持兩個移動平均，分別是梯度與梯度平方：

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\,\mathbf{g}_t, \quad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\,\mathbf{g}_t^2,$$
$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t}, \qquad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t}, \qquad \theta_t = \theta_{t-1} - \eta\,\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t}+\epsilon},$$

其中平方、開根號與除法都是逐元素運算，$\beta_1 \approx 0.9$、$\beta_2 \approx 0.999$ 是衰減率，ϵ 是小常數。第一個平均就是 momentum。除以第二個平均的平方根，讓每個參數有自己的步長，這是 Adam 所借鏡的 RMSprop 的概念 [19]：梯度一直很大的參數走小步，很少被更新的參數走大步。**偏差修正**（bias correction）$1/(1-\beta^t)$ 是因為兩個平均都從零開始，若不修正，在最初幾步會太小。

AdamW（Loshchilov 與 Hutter [20]）。權重衰減（weight decay）在每一步把權重往零縮。對單純的 SGD，這等同於在損失中加入 L2 懲罰 $\frac{\lambda}{2}\|\theta\|^2$。對 Adam 則不然：L2 項會進入 $\mathbf{g}_t$，再被 $\sqrt{\hat{\mathbf{v}}_t}$ 除掉，於是梯度大的權重反而被正則化得**比較少**。AdamW 把衰減和自適應步伐**解耦**：

$$\theta_t = \theta_{t-1} - \eta\left(\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t}+\epsilon} + \lambda\,\theta_{t-1}\right).$$

作者證明這能改善 Adam 的泛化能力 [20]，AdamW 現在是訓練 Transformer 與大型語言模型最常用的最佳化器。在 PyTorch 中，`torch.optim.Adam(..., weight_decay=λ)` 是耦合的 L2 版本，`torch.optim.AdamW` 才是解耦版本。

較新的最佳化器（簡述）。有兩個近年的選項值得認識。**Lion** 是透過自動化的程式搜尋找到的 [21]：它只保留一個 momentum 緩衝區（所以比 Adam 省記憶體），並以內插後 momentum 的**正負號**（sign）更新每個參數，因此所有更新的大小都相同；它需要比 AdamW 更小的學習率。**Muon** [22] 作用在隱藏層的二維權重矩陣上：它取 momentum 更新，用幾次 Newton–Schulz 迭代把它近似正交化；嵌入層、輸出層、偏差以及其他純量或向量參數仍以 AdamW 訓練。Liu 等人報告，在加入權重衰減與逐參數更新尺度調整之後，Muon 在他們的運算最適（compute-optimal）大型語言模型訓練實驗中，達到約 AdamW 兩倍的運算效率 [23]。這些都還是活躍的研究主題；AdamW 仍是安全的預設。

學習率排程

學習率通常是最重要的超參數。訓練初期合適的學習率，到後期常常太大，因為模型那時需要小而謹慎的步伐，所以學習率會隨時間改變（圖 6）。

- **階梯衰減**（step decay）：在固定的 epoch 把學習率乘上 0.1 之類的因子。以 SGD 訓練 CNN 的傳統做法。
- **餘弦退火**（cosine annealing）：$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t/T))$，在 T 步內從 $\eta_{\max}$ 平滑降到 $\eta_{\min}$；SGDR [24] 把它和週期性的熱重啟（warm restart）結合。
- **暖身**（warmup）：在最初幾百或幾千步內，把學習率從接近零線性升高。訓練初期 Adam 的二階動差估計很吵，權重也離任何好區域都很遠，直接用全尺寸的步伐可能造成不穩定。

以一個已發表的大型模型為具體例子：LLaMA 使用 AdamW（$\beta_1 = 0.9$、$\beta_2 = 0.95$）、權重衰減 0.1、梯度裁剪 1.0、2,000 步暖身，以及結束於峰值 10% 的餘弦排程 [12]。

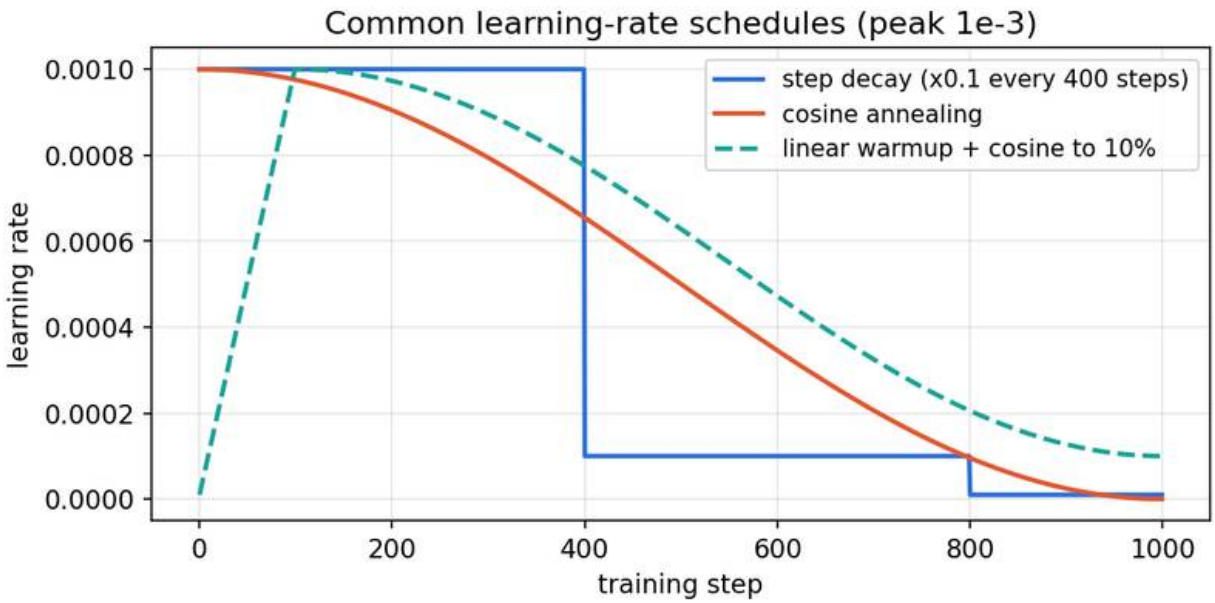


圖 *6* — 三種常見的學習率排程，共 *1,000* 步，峰值 *1e-3*。

Mini-batch、iteration 與 epoch

Mini-batch（常簡稱 batch）是用來做一次梯度估計的 B 個隨機範例。一次 **iteration**（迭代）是一次參數更新，使用一個 mini-batch。一個 **epoch** 是把整個訓練集完整走過一遍，對 N 個範例需要 $\lceil N/B \rceil$ 次 iteration。例如 $N = 50{,}000$、$B = 128$ 時，每個 epoch 有 391 次 iteration。

Batch size B 同時影響速度與結果，而且**並非越大越好**：

- **小 batch** 的梯度估計很吵（mini-batch 平均的變異數大約以 $1/B$ 下降），所以損失曲線跳動，每個 epoch 要走很多步，硬體利用率也差。
- **大 batch** 的梯度穩定，在 GPU 上平行效率高，但每個 epoch 的更新次數較少，而且在其他設定不變時往往泛化較差。Keskar 等人發現，大 batch 訓練傾向收斂到**尖銳極小值**（sharp minima），也就是解附近損失急遽上升的地方；小 batch 訓練則因為梯度雜訊，傾向找到泛化較好的**平坦極小值**（flat minima）[25]。

實務上，選一個能舒服放進記憶體、而且仍然訓練得好的最大 batch；改變 B 時要重新調整學習率，兩者交互作用很強。

梯度裁剪

即使網路設計得很好，偶爾某個 batch 也會產生巨大的梯度。**梯度範數裁剪**（gradient-norm clipping）在梯度範數超過門檻 c 時把整個梯度縮小：

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1, \frac{c}{\|\mathbf{g}\|}\right),$$

其中 $\|\mathbf{g}\|$ 是把所有參數梯度串接起來後的範數。它保留方向、限制步伐大小，是對付梯度爆炸的標準防線，在遞迴網路與 Transformer 中尤其常用 [3]。門檻 1.0 是常見的選擇。

在 PyTorch 中組合起來

下面的程式以 mini-batch、AdamW、餘弦排程與梯度裁剪，在 `make_moons` 上訓練一個小型 MLP。在 CPU 上幾秒鐘就跑完。

```
import torch, torch.nn as nn
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

torch.manual_seed(0)
X, y = make_moons(n_samples=1000, noise=0.25, random_state=0)
Xtr, Xva, ytr, yva = train_test_split(X, y, test_size=0.3, random_state=0)
Xtr, Xva = torch.tensor(Xtr, dtype=torch.float32), torch.tensor(Xva, dtype=torch.float32)
ytr, yva = torch.tensor(ytr), torch.tensor(yva)

model = nn.Sequential(
    nn.Linear(2, 64), nn.ReLU(),
    nn.Linear(64, 64), nn.ReLU(),
    nn.Linear(64, 2),               # 輸出 logits：這裡不加 softmax
)
loss_fn = nn.CrossEntropyLoss()     # 內部會做 log-softmax
opt = torch.optim.AdamW(model.parameters(), lr=1e-2, weight_decay=1e-2)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=200)

for epoch in range(200):
    model.train()
    perm = torch.randperm(len(Xtr))
    for i in range(0, len(Xtr), 64):                # 每個 mini-batch 64 筆
        idx = perm[i:i + 64]
        loss = loss_fn(model(Xtr[idx]), ytr[idx])
        opt.zero_grad()
        loss.backward()                              # 反向傳播
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        opt.step()
    sched.step()

model.eval()
with torch.no_grad():
    acc = (model(Xva).argmax(dim=1) == yva).float().mean().item()
print(f"validation accuracy: {acc:.3f}")   # 約 0.94
```

注意 `model.train()` 與 `model.eval()`：它們讓 dropout、BatchNorm 等層在訓練行為與推論行為之間切換。測試時忘了 `eval()` 是經典的 bug。

泛化：欠擬合、過擬合與正則化

白話版。目標不是在訓練資料上表現好，而是在模型從沒看過的資料上表現好。模型有兩種相反的失敗方式：太弱，連訓練資料都學不會（欠擬合）；或者把訓練資料連同雜訊一起背下來，遇到新資料就失敗（過擬合）。

模型能力與兩種失敗模式

模型的**能力**（capacity）粗略地說就是它能擬合多複雜的函數；它隨參數量、深度、寬度與訓練時間增加。能力提高時，訓練誤差持續下降，驗證誤差則先降後升，因為模型開始擬合雜訊。兩者之間的差距稱為**泛化差距**（generalization gap）[3]。

- **欠擬合**：**訓練**誤差本身就很高。模型或訓練程序抓不到資料的規律。
- **過擬合**：訓練誤差很低，但驗證或測試誤差高得多。

處理欠擬合，依序是：

1. **先檢查程式與資料**。 bug、輸入與標籤沒對齊、前處理錯誤，或損失套在錯誤的張量上，看起來都像欠擬合。
2. **提高模型能力**：更寬或更深的模型，或更好的架構。
3. **改善最佳化**：訓練更久、調整學習率與排程、換最佳化器、檢查初始化與正規化。

把模型**變小**永遠不能解決欠擬合，那只會讓能力更低。在準確率已經足夠之後，為了速度或記憶體選擇較小的模型是合理的，但那是效率上的考量，不是解方。

處理過擬合：用更多或更多樣的資料、加入正則化（見下文）、降低能力，以及提早停止。也值得確認訓練集和驗證集真的來自同一個分布；兩者不一致時看起來也像過擬合，但需要的是修正資料，而不是正則化。

讀懂損失曲線

把訓練損失與驗證損失都對 iteration 或 epoch 作圖。下面四種情況涵蓋大部分狀況：

- **A. 兩條曲線一開始就是平的**。如果損失很高，模型根本沒在學：懷疑 bug、學習率太小或太大，或模型與資料不匹配。如果損失已經很低，可能是任務非常簡單，或每個記錄點之間隔了太多 iteration（一次記錄涵蓋的資料太多），把曲線的形狀藏起來了；請更頻繁地記錄。
- **B. 訓練損失持續下降，驗證損失轉而上升**。典型的過擬合。保留驗證損失最低時的參數（提早停止），並加入正則化。
- **C. 兩條曲線都下降，並在彼此接近處持平**。可能模型已接近最佳，這在訓練與驗證資料非常相似時很常見；或兩者都還很高，那就是欠擬合：訓練更久或提高能力。
- **D. 驗證損失高於訓練損失，但兩者都穩定下降**。正常、健康的情況。繼續訓練，並保留驗證損失最低的檢查點。

正則化工具

圖 7 刻意製造過擬合，並展示兩種解法。一個很寬的 MLP（兩個各 256 單元的隱藏層）只用 100 個有雜訊的雙月形資料點訓練。沒有正則化時，訓練損失降向零，驗證損失則很早就到達最低點，接著急遽上升。權重衰減與 dropout 都大幅減緩了這個上升。

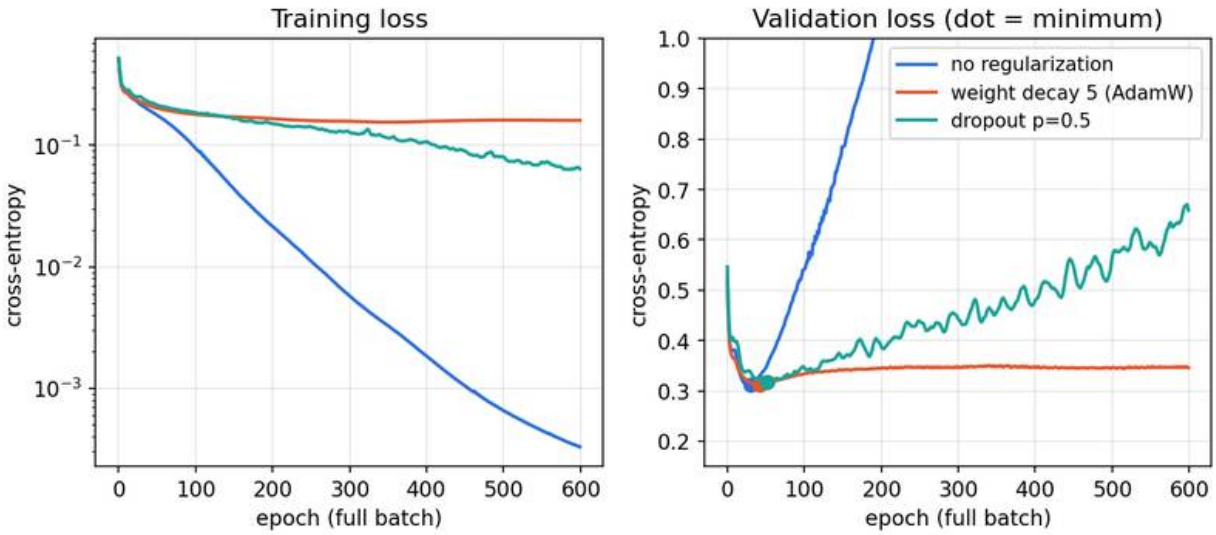


圖 7 — 以 *make_moons*（雜訊 *0.35*）的 *100* 個訓練點製造過擬合，並以 *2,000* 個驗證點評估。全批次 *AdamW*，學習率 *3e-3*。圓點標示每次執行的最低驗證損失，也就是提早停止會保留模型的位置。

權重衰減（L2 正則化）。在損失中加入 $\frac{\lambda}{2}\|\theta\|^2$ 來懲罰大權重；使用 Adam 時則改用解耦的衰減（AdamW）。大權重讓網路能圍著個別資料點畫出尖銳、彎曲的邊界；讓權重保持小，就偏好較平滑的函數。在圖 7 中，$\lambda = 5$ 的解耦衰減讓驗證損失在最低點之後幾乎持平。

Dropout（Srivastava 等人 [27]）。訓練時，每個隱藏單元以機率 p 被設為零，每個範例、每一步都獨立抽樣，存活的單元再乘上 $1/(1-p)$，讓激活值的期望不變（「inverted dropout」）。測試時使用所有單元，不做縮放。於是每一步訓練的都是一個不同的「變瘦」子網路；單元無法依賴特定的夥伴，必須學出本身就有用的特徵，而測試時的完整網路就像許多瘦網路的平均 [27]。這也是 `model.eval()` 很重要的原因。

資料擴增（data augmentation）。用不改變標籤的轉換產生新的訓練範例：對影像做翻轉、裁切、小角度旋轉、色彩抖動與加雜訊。它常常是影像領域最有效的單一正則化方法，因為它直接教模型該有的不變性。但它必須符合任務：水平翻轉對貓沒問題，對辨識文字就不行。

提早停止（early stopping）。訓練時監看驗證損失，保留它最低那個 epoch 的參數；如果連續若干個 epoch（「patience」）都沒有改善就停止。它不花成本、不必修改模型，並藉由限制參數能離初始值多遠而發揮正則化作用 [3]。

更多資料與更好的策略。原始筆記列出的其他手段值得再強調：更多訓練資料是對付過擬合最可靠的方法；更強的架構（更深，或結構更好以利收斂）在模型欠擬合時有幫助；更好的訓練策略（最佳化器、學習率排程、batch size）則兩者都有幫助。

殘差連接與通往 Transformer 之路

白話版。疊很多層時，每一層不但要改進訊號，還得把它完整傳下去，這其實很難。殘差連接給每個區塊一條快速道路：區塊只需要學一個加在輸入上的修正量。

He 等人觀察到，在已經很深的網路上再加普通的層，連**訓練**誤差都會變差；這是最佳化的問題，不是過擬合 [28]。他們的**殘差區塊**計算

$$\mathbf{y} = \mathbf{x} + F(\mathbf{x}),$$

其中 $\mathbf{x}$ 是區塊的輸入，F 是一小疊層（在 ResNet 中是卷積加上 BatchNorm 與 ReLU）。如果一個區塊最好的做法是什麼都不做，它只需要把 F 推向零，這很容易。反向傳遞說明了為什麼它好訓練：

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = I + \frac{\partial F}{\partial \mathbf{x}},$$

其中 I 是單位矩陣。單位項讓梯度直接穿過每一個區塊，所以許多區塊的連乘不再必然縮到零。靠著殘差連接，He 等人在 ImageNet 上訓練了多達 152 層的網路，並贏得 ILSVRC 2015 的多個項目 [28]。

Transformer（Vaswani 等人 [29]）結合了本篇的每一個概念。每個區塊有兩個子層：多頭自注意力（multi-head self-attention，用 softmax 在不同位置之間混合資訊）與逐位置的 MLP，每個子層都包在殘差連接與 LayerNorm 裡。原始論文是在相加之後才正規化；許多近期模型（包括 LLaMA）改為正規化子層的**輸入**（pre-normalization），並使用 RMSNorm 與 SwiGLU [12]：

$$\mathbf{h} = \mathbf{x} + \mathrm{Attention}\,(\,\mathrm{Norm}(\mathbf{x})), \qquad \mathbf{y} = \mathbf{h} + \mathrm{FFN}\,(\,\mathrm{Norm}(\mathbf{h})).$$

自 2017 年起，這個設計從機器翻譯擴展到語言模型、影像、語音等領域，也是今日大型語言模型的基礎。它的訓練方法同樣來自本篇：AdamW、暖身加餘弦衰減、梯度裁剪與權重衰減 [12]。深度學習系列會從這裡接著談用於影像的卷積網路與 Transformer，第一個任務是影像分類。

現代觀點

上述的基本工具已經穩定好幾年了；研究持續探討它們為什麼有效，以及每個零件更好的版本。以下幾篇回顧從不同角度提供了很好的導覽。

全貌：LeCun、Bengio 與 Hinton [2]（Nature，2015）。這篇短回顧至今仍是「深度學習是什麼」最清楚的說明：以多層進行表示學習（representation learning），每一層把前一層的表示轉成更抽象的表示，特徵以反向傳播從資料中學得，而不是人工設計。它介紹了以 SGD 進行的監督式學習、ReLU 的角色、處理影像的卷積網路、處理序列的遞迴網路，以及分散式表示（distributed representation），最後預測非監督式學習，以及表示學習與推理的結合，在未來會更加重要。若要以教科書的方式學習本篇的每個主題（包括損失函數的最大概似觀點、正則化與最佳化），Goodfellow、Bengio 與 Courville 的書 [3] 仍是標準參考。

最佳化：Sun [26]。Sun 的綜述問的是「類神經網路何時、為什麼能被成功訓練？」，並把答案分成三部分：梯度的問題（爆炸、消失）以及透過謹慎的初始化與正規化來解決；實用的演算法（SGD、Adam 等自適應方法，以及分散式訓練）與它們的收斂理論；以及損失地形（loss

landscape）的整體圖像，包括不好的局部極小值、模式連通性（mode connectivity）、樂透彩券假說（lottery ticket hypothesis）與無限寬度極限。它是從上述實務走向理論的好橋樑。

正規化：Huang 等人 [16]（IEEE TPAMI，2023）。這篇綜述從最佳化的角度給出正規化方法的統一觀點，並提出一套分類法，把任何方法拆成正規化區域、正規化運算與表示還原三部分。它涵蓋從 BatchNorm 開始的發展史、正規化為何能加速訓練並幫助泛化的分析，以及在不同領域的應用。讀它可以把 BatchNorm、LayerNorm、分組式與以權重為主的正規化，看成同一個設計空間中的不同點。

激活函數：Dubey、Singh 與 Chaudhuri [7]（Neurocomputing，2022）。這篇綜述把激活函數分為 logistic sigmoid ／ tanh 類、ReLU 類、ELU 類與可學習類，依值域、單調性與平滑性等性質加以描述，並在不同網路架構與資料類型上評測其中 18 種。它的實務訊息是：沒有哪一種激活函數處處最好，選擇會和架構與資料互相影響。

現況（2023–2026）。從已發表的模型訓練配方可以看到幾個趨同的方向：

- **架構**。採用 pre-normalization 與殘差連接的 Transformer 區塊，是語言領域的主流架構，在影像領域也越來越普遍；RMSNorm 與 SwiGLU（如 LLaMA [12]）是大型語言模型的常見預設。
- **最佳化**。 AdamW [20] 搭配線性暖身、餘弦（或類似的）衰減與梯度裁剪是標準配方。不只做逐座標縮放的最佳化器是活躍的研究方向：Lion [21] 使用正負號更新、較省記憶體；Muon [22] 把矩陣更新正交化，並有在大規模下提升運算效率的報告 [23]。這些增益在不同任務與規模下有多穩健，仍在研究中，因此 AdamW 依然是安全的選擇。
- **泛化**。尖銳與平坦極小值的觀點 [25] 是有用的直覺，而不是完整的理論；高度過度參數化（over-parameterized）的網路為什麼還能泛化得這麼好，至今仍是開放問題，Sun 的綜述 [26] 整理了主要的研究方向。

開放問題。深度網路可靠的泛化理論；同時選擇學習率、batch size 與排程的原則性規則，特別是在放大模型規模時；正規化、殘差連接與自適應最佳化器之間究竟如何交互作用；以及不靠目前配方中那些大量小技巧，也能在極大規模下穩定訓練。

重點整理

- 感知器是線性分數接上階梯函數，以「答錯才更新」的規則訓練；邏輯迴歸則用 sigmoid 機率與交叉熵。結構相似但方法不同，而且只有平滑的那一個能被疊起來並以梯度訓練。
- 具有非線性激活函數的隱藏層，才讓網路不只是線性模型。一個夠寬的隱藏層在原理上能逼近任何連續函數，但那只是存在性結果；深度讓有用的函數更容易被表示與學習。「深」沒有固定的層數定義。
- ReLU 家族（ReLU、Leaky ReLU、GELU、SiLU）讓梯度保持暢通；sigmoid 與 softmax 主要放在輸出層；SwiGLU 是 Transformer 前饋層的現代選擇。BatchNorm 沿 batch 正規化，LayerNorm 與 RMSNorm 沿單一樣本的特徵正規化。
- 輸出層要配合任務：迴歸用線性加 MSE，二元或多標籤用 sigmoid 加二元交叉熵，多類別用 softmax 加交叉熵。每種情況的輸出梯度都是預測減目標。
- 反向傳播是從損失往回套用連鎖律：$\delta_\ell = (W_{\ell+1}^\top \delta_{\ell+1}) \odot f'(\mathbf{z}_\ell)$，$\partial L/\partial W_\ell = \delta_\ell \mathbf{a}_{\ell-1}^\top$。親手寫的梯度要做數值檢查。
- 實務訓練：He 或 Xavier 初始化、AdamW 搭配暖身與衰減排程、梯度裁剪，以及和學習率一起選擇的 batch size，而不是「越大越好」。

- 欠擬合代表訓練誤差高，需要更多模型能力或更好的最佳化，絕不是更小的模型；過擬合則需要更多資料、資料擴增、權重衰減、dropout 與提早停止。
- 殘差連接讓非常深的網路可以訓練，而 Transformer 正是這些概念的集大成。

練習

1. **手算感知器**。對 AND 資料執行感知器規則，$\eta = 1$、$\mathbf{w} = \mathbf{0}$、$b = 0$，依 $(0,0), (0,1), (1,0), (1,1)$ 的順序拜訪各點。寫下每次更新後的 $\mathbf{w}$ 與 b，直到某個 epoch 完全沒有錯誤。接著用一句話解釋，為什麼同樣的程序用在 XOR 上永遠不會停。

   提示

   在嚴格判斷 $z > 0$ 下，第一個點 $(0,0)$ 的 $z = 0$，預測為 0，是正確的。第一個錯誤發生在 $(1,1)$，得到 $\mathbf{w} = (1,1)$、$b = 1$。繼續做下去：在標籤為 0 的點上犯錯就減去 $\mathbf{x}$ 和 1，在標籤為 1 的點上犯錯就加上它們。以這個順序，五個 epoch 內共更新十次，第六個 epoch 完全正確，此時 $\mathbf{w} = (2,1)$、$b = -2$，也就是直線 $2x_1 + x_2 = 2$。換一個拜訪順序會得到另一條同樣有效的直線。對 XOR 而言，沒有任何 $(\mathbf{w}, b)$ 能把四個點全部分對，所以永遠有某個點是錯的，也就永遠會更新。

2. **sigmoid 的梯度消失得多快？**證明 $\sigma'(x) = \sigma(x)(1 - \sigma(x)) \le 1/4$。接著考慮一條由 20 個 sigmoid 層組成、所有權重都等於 1 的鏈。梯度在往回傳的路上，最多會被乘上多大的因子？對活躍單元的 ReLU 重做一次推理。

   提示

   $s(1-s)$ 在 $s \in (0,1)$ 上是開口向下的拋物線，在 $s = 1/2$ 有最大值 $1/4$。二十個最多 $1/4$ 的因子相乘，最多是 $4^{-20} \approx 9 \times 10^{-13}$。對活躍的 ReLU 單元，$f' = 1$，所以激活函數完全不會造成縮小；只有權重會，這就是初始化重要的原因。

3. 「**預測減目標**」**梯度**。利用 $\partial p_k / \partial z_j = p_k(\delta_{kj} - p_j)$，證明當 $\mathbf{y}$ 為 one-hot 時，$L = -\sum_k y_k \log p_k$ 滿足 $\partial L / \partial z_j = p_j - y_j$。再對 sigmoid 搭配二元交叉熵證明同樣的結果。

   提示

   $\partial L / \partial z_j = -\sum_k (y_k / p_k)\, p_k(\delta_{kj} - p_j) = -\sum_k y_k \delta_{kj} + p_j \sum_k y_k = -y_j + p_j$，因為 $\sum_k y_k = 1$。對 sigmoid，$\partial \ell / \partial \hat{p} = -y/\hat{p} + (1-y)/(1-\hat{p})$，且 $\partial \hat{p} / \partial z = \hat{p}(1-\hat{p})$；相乘並化簡即得 $\hat{p} - y$。

4. **擴充梯度檢查**。在 NumPy 範例中，把 `W2`、`b1`、`b2` 的每個元素（而不只是 `W1` 的一個元素）都和有限差分比較，並回報最大相對誤差 $|a - n| / \max(|a|, |n|, 10^{-12})$。然後故意放一個 bug，例如拿掉 `* (Z1 > 0)` 這個因子，看看誤差會變多大。

   提示

   用 `np.ndindex(param.shape)` 跑過每個元素，把它擾動 $\pm 10^{-6}$ 並重新計算損失。正確的實作相對誤差約在 10^{-7} 或更小（少數緊鄰 ReLU 折角的元素可能較大）。拿掉 ReLU 導數後，`W1` 與 `b1` 會出現量級為 1 的誤差，而 `W2` 與 `b2` 仍然正確，因為 bug 只在隱藏層以下的反向路徑上。

5. **Iteration、epoch 與 batch size**。某資料集有 60,000 張訓練影像。(a) $B = 32$ 與 $B = 1{,}024$ 時，一個 epoch 各有多少次 iteration？(b) 兩者都訓練 20 個 epoch，各做了多少次參數更新？(c) 根據 batch size 的討論，舉出兩個理由說明在相同學習率下，$B = 1{,}024$ 的執行可能得到較差的驗證準確率，並提出一個可能有幫助的調整。

提示

(a) $\lceil 60000/32 \rceil = 1875$ 與 $\lceil 60000/1024 \rceil = 59$。(b) 37,500 次對 1,180 次更新。(c) 同樣的 epoch 數下更新次數少很多；梯度雜訊也較少，而尖銳極小值的研究把這和較差的泛化連在一起。常見的調整是針對新的 batch size 重新調學習率（通常調大，並加上暖身），以及訓練更多 epoch。

6. **診斷曲線**。同事的網路訓練損失一開始下降很快，接著停在一個很高的值不動；驗證損失和它幾乎貼在一起。同事提議把隱藏單元數減半，「讓它收斂得快一點」。發生了什麼事？你會先檢查什麼？你會改提出什麼建議？

提示

兩條損失都很高而且彼此接近：這是欠擬合（數值偏高的情況 C），不是過擬合。先檢查 bug：標籤是否與輸入對齊、損失是否作用在 logits 上、資料是否已正規化、學習率是否合理。接著提高模型能力或改善最佳化（更寬或更深的模型、學習率排程、訓練更久）。把模型縮小會降低能力，只會讓欠擬合更嚴重。

參考文獻

1. Shyandram, “機器學習及類神經網路筆記,” Medium post, 2024; updated 2026. link
2. Y. LeCun, Y. Bengio and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015. doi
3. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016. online book
4. F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, pp. 386–408, 1958. doi
5. G. Cybenko, “Approximation by superpositions of a sigmoidal function,” *Mathematics of Control, Signals and Systems*, vol. 2, pp. 303–314, 1989. doi
6. D. E. Rumelhart, G. E. Hinton and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986. doi
7. S. R. Dubey, S. K. Singh and B. B. Chaudhuri, “Activation functions in deep learning: A comprehensive survey and benchmark,” *Neurocomputing*, vol. 503, pp. 92–108, 2022. arXiv
8. D. Hendrycks and K. Gimpel, “Gaussian error linear units (GELUs),” arXiv:1606.08415, 2016. arXiv
9. S. Elfwing, E. Uchibe and K. Doya, “Sigmoid-weighted linear units for neural network function approximation in reinforcement learning,” *Neural Networks*, vol. 107, pp. 3–11, 2018. arXiv
10. P. Ramachandran, B. Zoph and Q. V. Le, “Searching for activation functions,” arXiv:1710.05941, 2017. arXiv
11. N. Shazeer, “GLU variants improve Transformer,” arXiv:2002.05202, 2020. arXiv

12. H. Touvron, T. Lavril, G. Izacard et al., “LLaMA: Open and efficient foundation language models,” arXiv:2302.13971, 2023. arXiv
13. S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. International Conference on Machine Learning* (*ICML*), PMLR 37, pp. 448–456, 2015. PMLR
14. J. L. Ba, J. R. Kiros and G. E. Hinton, “Layer normalization,” arXiv:1607.06450, 2016. arXiv
15. B. Zhang and R. Sennrich, “Root mean square layer normalization,” in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2019. arXiv
16. L. Huang, J. Qin, Y. Zhou, F. Zhu, L. Liu and L. Shao, “Normalization techniques in training DNNs: Methodology, analysis and application,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 8, pp. 10173–10196, 2023. arXiv
17. X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *Proc. International Conference on Artificial Intelligence and Statistics* (*AISTATS*), PMLR 9, pp. 249–256, 2010. PMLR
18. K. He, X. Zhang, S. Ren and J. Sun, “Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification,” in *Proc. IEEE International Conference on Computer Vision* (*ICCV*), pp. 1026–1034, 2015. arXiv
19. D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. International Conference on Learning Representations* (*ICLR*), 2015. arXiv
20. I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. International Conference on Learning Representations* (*ICLR*), 2019. arXiv
21. X. Chen, C. Liang, D. Huang, E. Real, K. Wang, H. Pham, X. Dong, T. Luong, C.-J. Hsieh, Y. Lu and Q. V. Le, “Symbolic discovery of optimization algorithms,” in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2023. arXiv
22. K. Jordan, “Muon: An optimizer for hidden layers in neural networks,” Medium post, December 2024. link
23. J. Liu, J. Su, X. Yao et al., “Muon is scalable for LLM training,” arXiv:2502.16982, 2025. arXiv
24. I. Loshchilov and F. Hutter, “SGDR: Stochastic gradient descent with warm restarts,” in *Proc. International Conference on Learning Representations* (*ICLR*), 2017. arXiv
25. N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy and P. T. P. Tang, “On large-batch training for deep learning: Generalization gap and sharp minima,” in *Proc. International Conference on Learning Representations* (*ICLR*), 2017. arXiv
26. R. Sun, “Optimization for deep learning: Theory and algorithms,” arXiv:1912.08957, 2019. arXiv
27. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014. JMLR
28. K. He, X. Zhang, S. Ren and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition* (*CVPR*), pp. 770–778, 2016. arXiv
29. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2017. arXiv

ML 03・卷積、影像濾波與 CNN

從模糊與 Sobel 卷積核到學習式濾波器：卷積與互相關的差別、邊界處理、填補、步幅、膨脹、池化、感受野、卷積的反向傳播，以及經典 CNN 架構。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-03-convolutional-networks/

筆記本: https://github.com/Shyandram/cv-blog/blob/05b94ec/notebooks/image-processing/convolution.ipynb

你將學到

- 什麼是卷積核（kernel）、它如何在影像上滑動，以及為什麼同一種運算可以模糊、銳化或找出邊緣。
- **互相關**（correlation）與**卷積**（convolution）的確切差別，以及 NumPy、SciPy、OpenCV、PyTorch 實際計算的是哪一種。
- 如何用 NumPy 從零寫出 2-D 濾波，在 OpenCV、SciPy 與 PyTorch 中重現同樣結果，並有意識地處理影像邊界。
- 卷積層如何推廣固定濾波器：通道、權重共享、填補、步幅、膨脹、1×1 卷積與池化。
- 感受野如何隨深度成長，以及梯度如何反向穿過一個卷積。
- 經典架構（LeNet、AlexNet、VGG、GoogLeNet、ResNet、U-Net、MobileNet、EfficientNet、ConvNeXt）各自的貢獻，以及 CNN 在 2026 年的定位。

先看全貌

影像是一個數字網格（見影像就是陣列）。我們想對影像做的許多事都是**局部的**：一個像素是在邊緣上、在平滑區域內，還是在雜訊斑塊裡，大多取決於它的鄰居。**卷積**正是把「看一小塊鄰域、用固定權重組合、然後在每個位置重複」這件事形式化的運算。它是古典影像處理的主力工具，也是**卷積神經網路**（convolutional neural network, CNN）的基本積木；差別在於 CNN 從資料中學習權重，而不是由人手動設計。

> **用最簡單的話說**
>
> 想像你有一張開了九個小洞的硬紙板，每個洞後面寫著一個數字。把紙板放在圖畫上，把每個洞看到的顏色乘上洞後面的數字，全部加起來，把答案寫在一張新紙上。接著把紙板滑一格，再做一次，直到蓋過整張圖。如果九個數字都一樣，新的圖就是模糊的平均。如果左邊的數字是負的、右邊是正的，只有在圖從暗變亮的地方才會得到大答案：這就是邊緣偵測器。CNN 就是一台同時滑動成千上萬片這種紙板，並且**自己學會**洞後面該寫什麼數字的機器。

本篇分成兩半。前半（濾波）是古典工具箱，只需要 NumPy。後半（CNN）建立在ML 02 之上，你在那裡已經認識了網路層、損失函數與反向傳播。若之後想更深入固定濾波器，IP 03 介紹空間濾波，IP 04 則透過卷積定理說明頻率域觀點。深度學習方面，DLCV 02 詳細探討現代分類骨幹

網路，ML 03A 轉向序列資料，ML 04 則介紹卷積的主要替代方案：注意力機制。標準教科書可參考 Goodfellow 等人 [1] 的第 9 章與 Prince [10] 的第 10 章。

第一部分・用固定卷積核濾波

滑動的加權和

白話版。每個輸出像素都是輸入影像在相同位置附近像素的加權和。那一小格權重稱為**卷積核**（kernel，也稱為濾波器 filter 或遮罩 mask）。

精確版。令 I 為灰階影像，$I(x, y)$ 是第 x 欄、第 y 列的強度。令 K 為大小 $(2a+1) \times (2b+1)$ 的卷積核，垂直索引從 $-a$ 到 a、水平索引從 $-b$ 到 b，因此 $K(0,0)$ 是它的中心。濾波後的影像 G 為

$$G(x, y) = \sum_{i=-a}^{a} \sum_{j=-b}^{b} K(i, j)\, I(x + j,\; y + i).$$

其中 i 索引卷積核的列、j 索引卷積核的行，$G(x, y)$ 是相同位置的輸出，雙重總和恰好走訪卷積核的每個係數一次。以 3×3 卷積核為例，$a = b = 1$，每個輸出像素需要九次乘法與八次加法。

圖 1 手算了一個輸出值。橘色視窗蓋住填補後輸入左上角的 3×3 鄰域。與卷積核逐元素相乘再相加：$1 \cdot 1 + 0 \cdot 2 + (-1) \cdot 0 + 2 \cdot 0 + 0 \cdot 1 + (-2) \cdot 3 + 1 \cdot 2 + 0 \cdot 0 + (-1) \cdot 1 = -4$。這個數字就成為 G 在視窗中心的值。把視窗往右滑一格再重複，就填出下一個輸出。

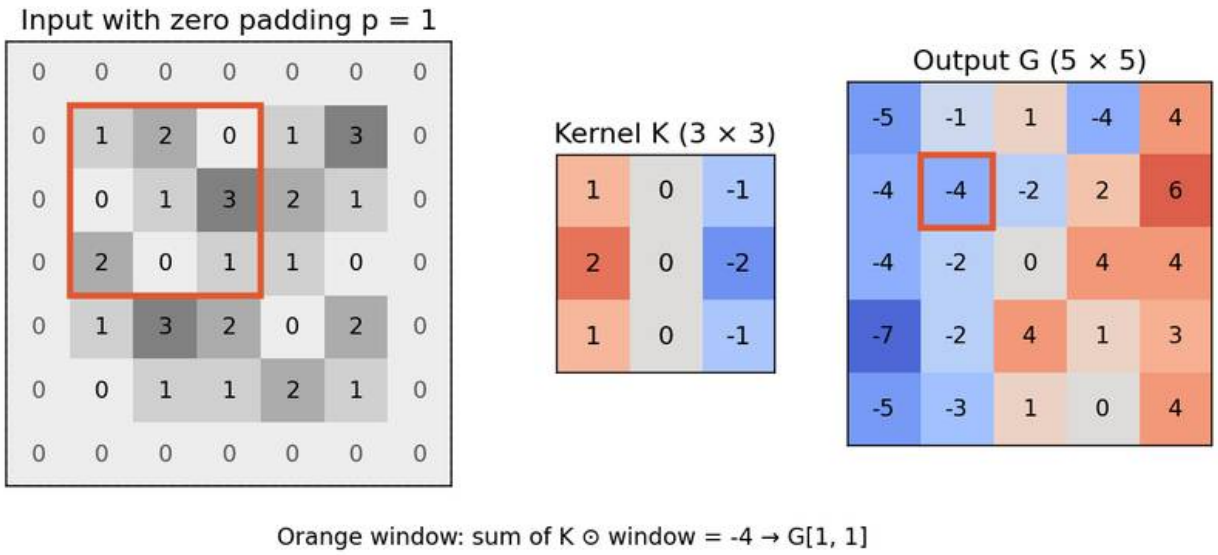


圖 *1* — 滑動加權和的一步。橘色 3×3 視窗與 K 逐元素相乘再加總，在對應的輸出位置得到 -4。零填補（灰色）讓輸出維持與輸入相同的大小。

有兩個觀察對後面所有內容都很重要。第一，同樣的九個權重用在每一個位置：這個運算具有**平移不變性**（shift-invariant，也稱為空間不變）。第二，這個運算是**線性的**：對 $\alpha I_1 + \beta I_2$ 濾波會得

到 $\alpha G_1 + \beta G_2$。線性平移不變（linear shift-invariant, LSI）運算恰好就是卷積能表達的那一類，這也是卷積在訊號處理中如此核心的原因。

互相關與卷積

白話版。上面的公式在技術上是**互相關**（cross-correlation）。真正的**卷積**做同樣的事，只是卷積核先旋轉 180 度。對稱的卷積核兩者結果相同；不對稱的則不同。

精確版。數學上的卷積寫成 $K * I$：

$$(K * I)(x, y) = \sum_{i=-a}^{a} \sum_{j=-b}^{b} K(i, j)\, I(x - j,\ y - i) = \sum_{i,j} K(-i, -j)\, I(x + j,\ y + i).$$

第二種寫法顯示，卷積等於用翻轉後的卷積核 $K(-i, -j)$ 做互相關。看出差別最乾淨的方法，是對單一亮點（**脈衝**，impulse）濾波。互相關會留下一個旋轉 180 度的卷積核副本；卷積則留下卷積核的原樣副本。這個副本就是**脈衝響應**（impulse response），也是訊號處理偏好卷積的原因：任何影像的輸出，都是一堆經過平移、縮放的脈衝響應之和。

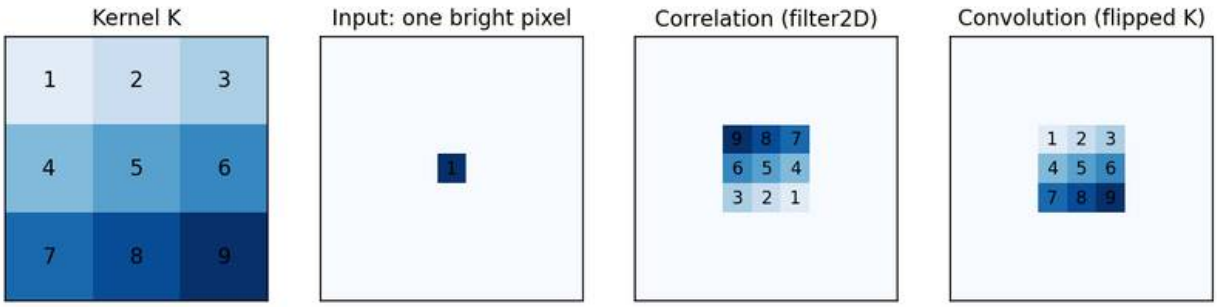


圖 2 — 對脈衝濾波。互相關（*cv2.filter2D* 的計算方式）印出旋轉 *180°* 的卷積核；卷積印出卷積核本身。

卷積具有互相關所沒有的代數性質。它滿足**交換律**（$K * I = I * K$）與**結合律**（$K_2 * (K_1 * I) = (K_2 * K_1) * I$），因此連續套用兩個濾波器，等於套用一個較大的濾波器。例如連續做兩次 3×3 方框模糊，就等於一個 5×5 卷積核。結合律也解釋了**可分離**（separable）卷積核：$k \times k$ 的高斯核等於一個 $k \times 1$ 的行濾波器再接一個 $1 \times k$ 的列濾波器，每個像素的運算量從 k^2 降到 $2k$。**卷積定理**指出，空間中的卷積等於傅立葉域中的乘法；IP 04 會發展這個觀點。

各函式庫計算的是哪一種？

函式庫呼叫	計算內容	預設邊界
`cv2.filter2D`	互相關 [11]	reflect-101（`BORDER_DEFAULT`）
`scipy.ndimage.correlate` / `convolve`	互相關 / 卷積	`reflect`（重複邊緣樣本）
`scipy.signal.convolve2d`	卷積	補零
`torch.nn.functional.conv2d`、`nn.Conv2d`	互相關 [31]	零填補

對於手工設計的不對稱卷積核（Sobel、浮雕、運動模糊），翻轉與否會改變結果的正負號或方向，所以需要真正的卷積時，要刻意翻轉，例如用 `cv2.flip(K, -1)`。對於學習出來的卷積核，這個差別就消失了：一個「本來應該」使用翻轉卷積核的網路，會直接把翻轉後的版本學起來。這就是為什麼深度學習框架把互相關稱為「卷積」也無傷大雅。

模糊、邊緣與銳化

白話版。總和為一的正權重會平均鄰居而產生模糊。總和為零的權重會抵消平坦區域、只留下變化，因此能找出邊緣。把一點點「影像減去其模糊版本」加回影像，就能銳化。

方框模糊（box blur）讓每個鄰居權重相等；3×3 版本有九個 $1/9$。**高斯模糊**取樣

$$K(i,j)=\frac{1}{Z}\exp\!\left(-\frac{i^2+j^2}{2\sigma^2}\right),$$

其中 σ（以像素為單位）決定模糊寬度，Z 將權重正規化使總和為一。近的像素比遠的像素重要，因此能避免方框模糊的塊狀瑕疵。實務上常用約 3σ 的卷積核半徑。

權重總和告訴你在每個像素都等於 c 的平坦區域會發生什麼：輸出為 $c\sum K$。總和為一的模糊保持亮度；總和為二的卷積核會讓亮度加倍（並使 8 位元影像飽和）；總和為零的卷積核則輸出零。

Sobel 卷積核估計水平與垂直方向的導數，同時在垂直方向上做平滑：

$$S_x=\begin{bmatrix}-1&0&1\\-2&0&2\\-1&0&1\end{bmatrix}=\begin{bmatrix}1\\2\\1\end{bmatrix}\begin{bmatrix}-1&0&1\end{bmatrix},\qquad S_y=S_x^{\top}.$$

S_x 對沿 x 方向的強度變化有反應，所以它凸顯**垂直**邊界；S_y 凸顯**水平**邊界。輸出 G_x、G_y 帶有正負號：由暗到亮與由亮到暗的邊緣正負相反。**梯度強度**（gradient magnitude）$\sqrt{G_x^2+G_y^2}$ 是非負的邊緣強度，$\operatorname{atan2}(G_y,G_x)$ 則是邊緣方向。上面的分解顯示 Sobel 是可分離的：一個 $[1,2,1]^{\top}$ 平滑行向量乘上一個 $[-1,0,1]$ 中央差分。

拉普拉斯（Laplacian）卷積核 $\begin{bmatrix}0&1&0\\1&-4&1\\0&1&0\end{bmatrix}$ 近似 $\partial^2 I/\partial x^2+\partial^2 I/\partial y^2$；它對細線以及邊緣的兩側都有反應。**反銳化遮罩**（unsharp masking）以下式銳化：

$$I_{\text{sharp}}=I+\lambda\,(I-G_\sigma * I),$$

其中 $G_\sigma * I$ 是高斯模糊後的副本，$I-G_\sigma * I$ 是高頻細節，$\lambda>0$ 控制強度。λ 太大會放大雜訊，並在邊緣兩側產生亮暗光暈。

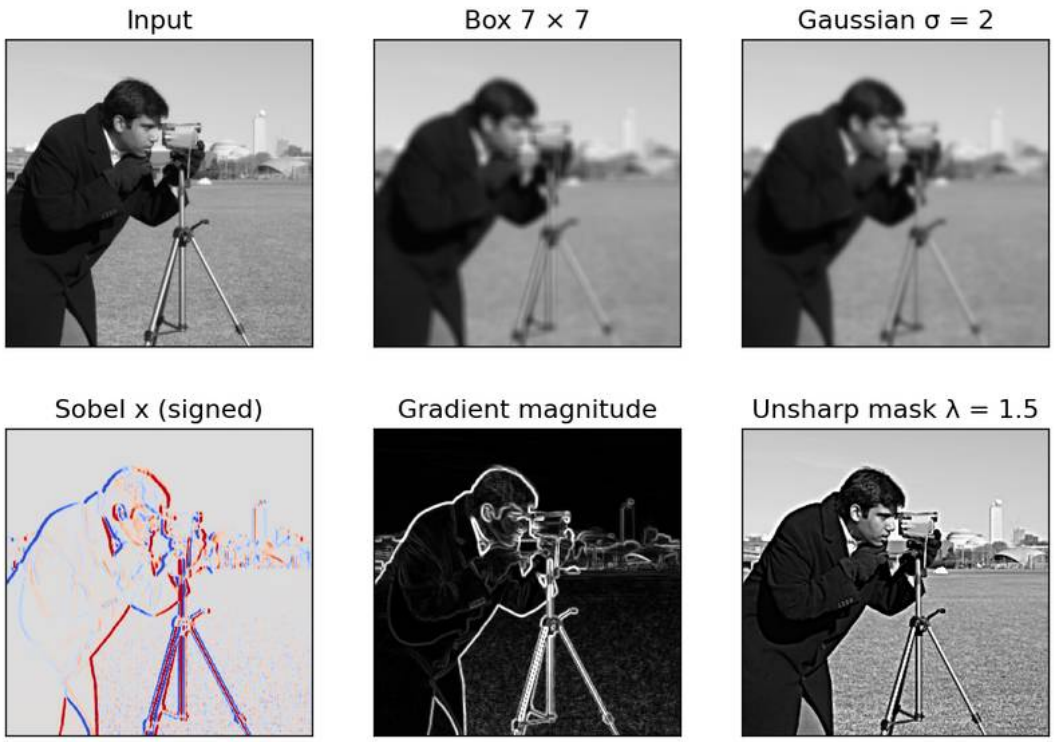


圖 3 — 同一張影像經過六種卷積核。方框與高斯模糊壓抑細節；*Sobel x* 帶有正負號（紅 = 由暗到亮，藍 = 由亮到暗）；梯度強度勾勒出物體輪廓；反銳化遮罩把放大的細節加回去。

動手試試：卷積核實驗室

編輯下方的卷積核格子，觀察輸出即時更新。先從 Sobel x 開始，然後把它轉置，改為偵測水平邊緣而非垂直邊緣。把模糊預設的每個權重都調大，你會看到平坦區域變亮，正如權重總和規則的預測。試試拉普拉斯與浮雕預設，並在看結果之前先預測它們的卷積核總和。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

下方的滑桿比較一張合成測試影像（圓盤、正方形、棋盤格與條紋）與它經過 5×5 高斯模糊（$\sigma = 1.4$）的結果。細條紋與棋盤格最先消失，因為模糊是一種**低通濾波器**：它移除快速的強度變化，保留緩慢的變化。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

程式碼：從零開始，再到 OpenCV、SciPy 與 PyTorch

下面的迴圈就是整個演算法。它不逐一走訪每個像素（在 Python 裡很慢），而是走訪卷積核的每個係數，每個係數加上一份平移過的填補影像，讓 NumPy 做向量化計算。接著程式檢查：只要

對齊定義（互相關或卷積）與邊界規則，四個函式庫的結果就會一致。需要 numpy、scipy、opencv-python 與 torch。

```
import numpy as np
import cv2
import torch
import torch.nn.functional as F
from scipy import ndimage, signal

def correlate2d(img, K, mode="edge"):
    """Cross-correlation of a 2-D float image with an odd-sized kernel.
    mode is any np.pad mode: "constant" (zeros), "edge", "reflect", "wrap"."""
    kh, kw = K.shape
    ph, pw = kh // 2, kw // 2
    padded = np.pad(img, ((ph, ph), (pw, pw)), mode=mode)
    H, W = img.shape
    out = np.zeros((H, W), dtype=np.float64)
    for i in range(kh):              # loop over kernel taps, not pixels:
        for j in range(kw):          # each step adds one shifted copy of the image
            out += K[i, j] * padded[i:i + H, j:j + W]
    return out

def convolve2d(img, K, mode="edge"):
    return correlate2d(img, K[::-1, ::-1], mode)     # convolution = correlation with
     ↪  flipped K

rng = np.random.default_rng(0)
img = rng.random((32, 40))
K = rng.standard_normal((3, 5))                      # deliberately asymmetric

ours_corr = correlate2d(img, K, "reflect")
ours_conv = convolve2d(img, K, "reflect")
# OpenCV: filter2D computes correlation; BORDER_REFLECT_101 is np.pad's "reflect"
cv_corr = cv2.filter2D(img, cv2.CV_64F, K, borderType=cv2.BORDER_REFLECT_101)
# SciPy: ndimage.correlate / ndimage.convolve with mode="mirror" (= reflect-101)
sp_corr = ndimage.correlate(img, K, mode="mirror")
sp_conv = ndimage.convolve(img, K, mode="mirror")
# PyTorch: conv2d is correlation; pad first with the same rule
t = torch.tensor(img)[None, None]
t = F.pad(t, (2, 2, 1, 1), mode="reflect")
pt_corr = F.conv2d(t, torch.tensor(K)[None, None])[0, 0].numpy()

for name, a in [("OpenCV", cv_corr), ("SciPy", sp_corr), ("PyTorch", pt_corr)]:
    print(f"{name:8s} correlation matches:", np.allclose(ours_corr, a))
print("SciPy    convolution matches:", np.allclose(ours_conv, sp_conv))
print("conv == corr for this kernel?", np.allclose(ours_conv, ours_corr))
# zero padding: our convolution equals signal.convolve2d with mode="same"
same = signal.convolve2d(img, K, mode="same", boundary="fill")
print("zero-padded 'same' convolution matches:", np.allclose(convolve2d(img, K, "constant"),
 ↪  same))
```

除了 conv == corr 因為隨機卷積核不對稱而印出 False 之外，所有檢查都印出 True。注意命

名陷阱：NumPy 的 `"reflect"` 填補（`dcb|abcd|cba`，不重複邊緣樣本）對應 OpenCV 的 `BORDER_REFLECT_101` 與 SciPy 的 `"mirror"`；而 SciPy 的 `"reflect"` 會重複邊緣樣本（`dcba|abcd|dcba`）。

第二段程式碼展示了初學者處理真實影像時最常遇到的兩個錯誤：整數飽和與卷積核總和規則。

```
import numpy as np
import cv2
from skimage import data

img_u8 = data.camera()                                  # 512 x 512 uint8 test image
img = img_u8.astype(np.float32) / 255.0
sobel_x = np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]], np.float32)

gx = cv2.filter2D(img, -1, sobel_x)                     # float in, float out: negatives kept
gy = cv2.filter2D(img, -1, sobel_x.T)
mag = np.hypot(gx, gy)
print("float Sobel x range:", gx.min().round(3), gx.max().round(3))

gx_u8 = cv2.filter2D(img_u8, -1, sobel_x)               # uint8 in, uint8 out: saturated
print("uint8 Sobel x range:", gx_u8.min(), gx_u8.max(),
      "| pixels clipped to 0:", f"{(gx_u8 == 0).mean():.0%}")

blur = cv2.GaussianBlur(img, (5, 5), 1.4)               # 5 x 5 Gaussian, sigma 1.4
print("mean brightness kept by blur:", np.isclose(img.mean(), blur.mean(), atol=1e-3))
K2 = 2 * np.ones((3, 3), np.float32) / 9                # weights sum to 2
print("mean after sum-2 kernel:", round(float(cv2.filter2D(img, -1, K2).mean() /
 ↪  img.mean()), 3))
```

在浮點數下，Sobel 響應大約介於 -3.4 到 3.3。若輸入與輸出都是 `uint8`，所有負的響應都會被截成零（這裡超過一半的像素），因此所有由亮到暗的邊緣都消失了。請一律用 `float32` 濾波，只在顯示時才轉回整數。

邊界處理

白話版。在影像邊緣附近，卷積核有一部分會懸在影像外面。你必須決定要假裝那裡有什麼值。

常見選擇有：**零填補**（常數填補），假裝外面的世界是黑色；**複製**（replicate），重複邊緣像素；**反射**（reflect），在邊界處鏡射影像；**環繞**（wrap），把影像視為週期性的，離散傅立葉轉換隱含的就是這種假設。OpenCV 的預設 `BORDER_REFLECT_101` 會鏡射但不重複邊緣像素 [11]。PyTorch 的 `nn.Conv2d` 預設使用零填補，除非你把 `padding_mode` 設為 `'reflect'`、`'replicate'` 或 `'circular'` [31]。

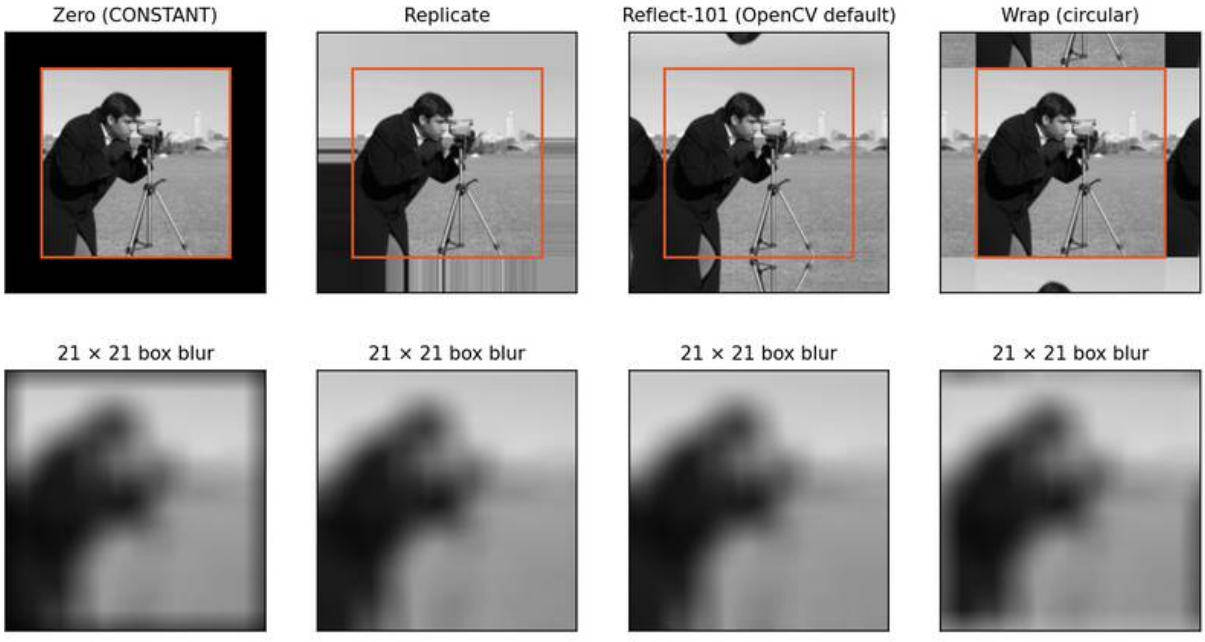


圖 *4* — 四種邊界規則及其對大型 *21 × 21* 方框模糊的影響。零填補讓每個邊框變暗；環繞混合了相對的邊；複製與反射讓平坦的邊框維持平坦。

零填補會讓模糊影像的邊框變暗，因為黑色像素被平均進去了。環繞則把影像頂部混進底部。對照片而言，複製與反射通常最不顯眼。卷積核小的時候，差別只影響一圈細框；但在深層 CNN 中，每一層都會加上自己的一圈：經過許多零填補層之後，特徵圖中很大一部分都「看過」人工邊界，這讓網路能從填補中推斷出絕對位置。

第二部分・卷積神經網路

從固定濾波器到學習式網路層

白話版。卷積層是一組濾波器，其權重以梯度下降學習。每個濾波器同時看所有輸入通道，產生一個輸出通道，稱為**特徵圖**（feature map）。

精確版。令輸入為 $X \in \mathbb{R}^{C_{\text{in}} \times H \times W}$（通道、高、寬），權重為 $K \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k \times k}$，偏差為 $b \in \mathbb{R}^{C_{\text{out}}}$。步幅為一、零填補為 p 的網路層計算

$$Y_{o,u,v} = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} K_{o,c,i,j}\, X_{c,\,u+i-p,\,v+j-p}.$$

o 索引輸出通道（濾波器），c 索引輸入通道，(u, v) 索引輸出位置，(i, j) 索引卷積核係數；影像外的位置讀取填補值。接著逐元素套用 ReLU 等非線性函數。一個 RGB 濾波器有三片 $k \times k$，每

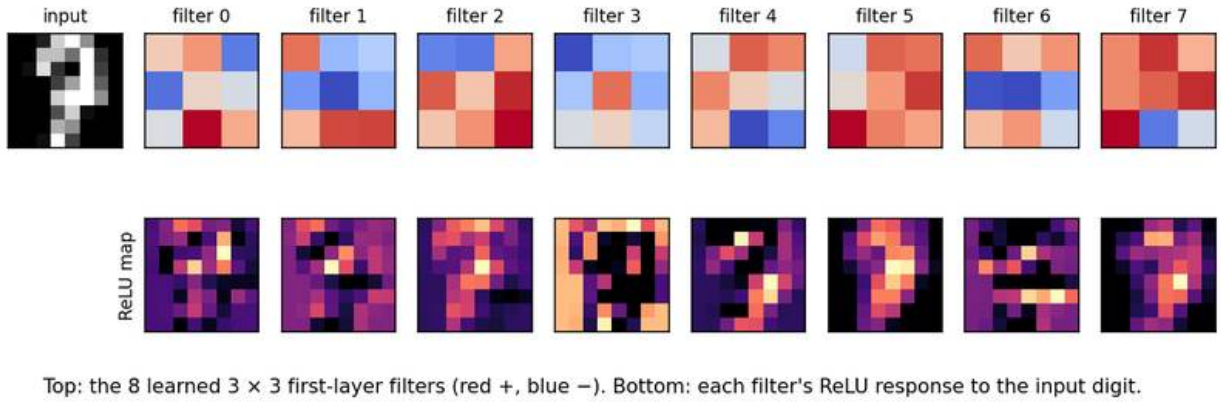
個顏色通道一片，它們的響應會**加總**成每個位置的一個數字：一個濾波器不會產生三張分開的影像。

這個公式內建了兩個結構假設：

- **局部連接**（local connectivity）：每個輸出只取決於一個 $k \times k$ 鄰域，而不是整張影像。
- **權重共享**（weight sharing，又稱參數共享）：同一個 K 用在每個位置 (u, v)。

兩者合起來就是 CNN 的**歸納偏差**（inductive bias），也就是對哪些函數比較可能的假設。它的回報是效率。一個 $C_{\text{in}} = 3$、$C_{\text{out}} = 32$、$k = 3$ 的網路層有 $32 \times (3 \cdot 9 + 1) = 896$ 個參數，與影像大小無關。若改用全連接層，把攤平的 $224 \times 224 \times 3$ 影像映射到區區 32 個數字，就需要 $150{,}528 \times 32 \approx 4.8$ 百萬個權重，而且還得在每個位置分別學會同一個邊緣偵測器。

Goodfellow 等人 [1] 把動機總結為稀疏交互作用、參數共享與等變性。圖 5 展示一個小型 CNN 在後面分類器程式碼所用的 8×8 數字資料集上訓練後，第一層學到了什麼。學到的 3×3 濾波器看起來像有方向的導數與斑點偵測器：網路自己重新發現了類似 Sobel 的卷積核，因為它們有用，而不是因為有人告訴它。Zeiler 與 Fergus [30] 透過視覺化大型 ImageNet 網路的濾波器與特徵圖，在大尺度上展示了同樣的現象。



Top: the 8 learned 3 × 3 first-layer filters (red +, blue −). Bottom: each filter's ReLU response to the input digit.

圖 *5* — 在 *8 × 8* 數字上學到的第一層濾波器（上）以及它們對一個測試數字的 *ReLU* 響應（下）。許多看起來像有方向的邊緣或筆畫偵測器。

填補、步幅與膨脹

白話版。填補（padding）控制邊界處的行為以及輸出是否維持大小。步幅（stride）跳過一些位置以縮小輸出。膨脹（dilation）把卷積核係數分散開來，不增加權重就能看得更遠。

對輸入大小 n（高或寬）、卷積核大小 k、每邊填補 p、步幅 s 與膨脹 d，輸出大小為 [12]

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - d(k-1) - 1}{s} \right\rfloor + 1.$$

$d(k-1)+1$ 是**有效卷積核大小**：膨脹 d 會在係數之間插入 $d-1$ 個空隙。三種填補慣例有名稱：**valid**（$p = 0$，輸出縮小 $k-1$）、**same**（奇數 k 且 $s = 1$ 時 $p = (k-1)/2$，輸出維持大小）、**full**

($p = k - 1$，計入所有部分重疊，輸出增加 $k - 1$）。步幅二大致把每個空間維度減半，是池化之外常見的下採樣方式。

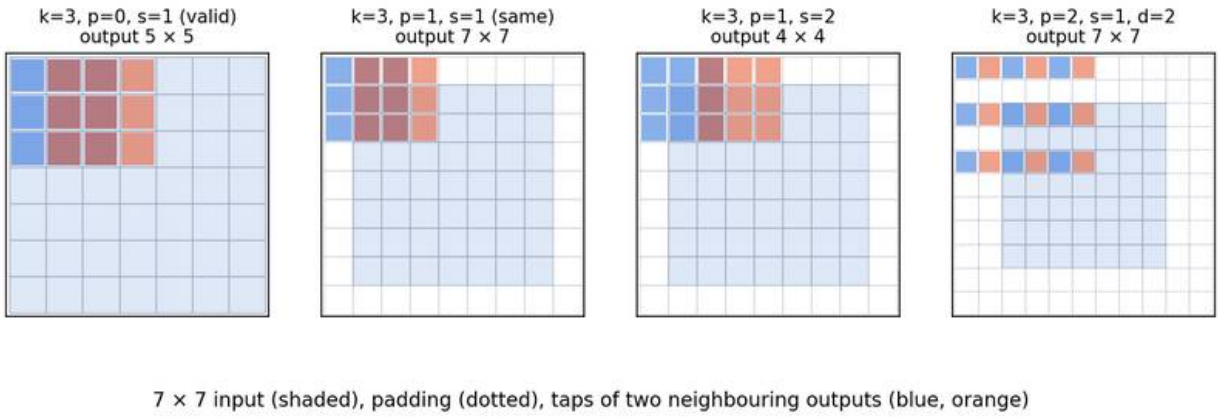


圖 *6* — 四種設定下兩個相鄰輸出（藍與橘；重疊處呈褐色）的係數位置。步幅讓視窗移動 s；膨脹讓係數相隔 d。

膨脹（atrous）卷積由 Yu 與 Koltun [18] 推廣到密集預測：堆疊膨脹率為 1、2、4、… 的網路層，可以在維持完整解析度的同時，讓視野呈指數成長。單一膨脹層只看到稀疏的輸入格點，因此混合膨脹率時要小心，以免產生「網格」（gridding）瑕疵。**轉置卷積**（transposed convolution）把帶步幅卷積的前向映射反過來執行以上採樣；它會在下面的反向傳播小節再次出現。Dumoulin 與 Visin 的卷積算術指南 [12] 為這些設定的每種組合都畫了圖解。

下面的程式碼用 PyTorch 檢查公式、計算參數量、說明 1×1 卷積就是逐像素的線性層，並量測平移等變性：

```
import torch
from torch import nn

def out_size(n, k, p=0, s=1, d=1):
    return (n + 2 * p - d * (k - 1) - 1) // s + 1

x = torch.randn(1, 3, 64, 64)                          # one RGB 64 x 64 image
for k, p, s, d in [(3, 1, 1, 1), (5, 2, 2, 1), (3, 2, 1, 2), (1, 0, 1, 1)]:
    conv = nn.Conv2d(3, 16, k, stride=s, padding=p, dilation=d)
    y = conv(x)
    n_params = sum(q.numel() for q in conv.parameters())
    assert y.shape[-1] == out_size(64, k, p, s, d)
    assert n_params == 16 * (3 * k * k + 1)
    print(f"k={k} p={p} s={s} d={d}: output {tuple(y.shape[1:])}, parameters {n_params}")

# A 1x1 convolution is the same linear layer applied at every pixel
conv1 = nn.Conv2d(16, 4, kernel_size=1)
lin = nn.Linear(16, 4)
lin.weight.data = conv1.weight.data[:, :, 0, 0].clone()
lin.bias.data = conv1.bias.data.clone()
z = torch.randn(2, 16, 8, 8)
same = torch.allclose(conv1(z), lin(z.permute(0, 2, 3, 1)).permute(0, 3, 1, 2), atol=1e-6)
```

```
print("1x1 conv == per-pixel Linear:", same)

# Translation equivariance away from borders (stride 1, no padding effects in the centre)
conv = nn.Conv2d(1, 4, 3, padding=1, bias=False)
img = torch.zeros(1, 1, 32, 32); img[..., 10:14, 8:12] = 1
shift = lambda t: torch.roll(t, shifts=(3, 5), dims=(-2, -1))
with torch.no_grad():
    a, b = conv(shift(img)), shift(conv(img))
print("shift then conv == conv then shift:", torch.allclose(a, b, atol=1e-6))
conv2 = nn.Conv2d(1, 4, 3, padding=1, stride=2, bias=False)
with torch.no_grad():
    y0, y1 = conv2(img), conv2(torch.roll(img, 1, dims=-1))   # shift by one pixel
print("stride 2, 1-pixel shift: max output change =", round((y0 - y1).abs().max().item(), 3))
```

它印出的輸出形狀為 $16 \times 64 \times 64$、$16 \times 32 \times 32$、$16 \times 64 \times 64$ 與 $16 \times 64 \times 64$，參數量分別為 448、1216、448 與 64；1×1 與等變性檢查都通過；而平移一個像素會讓步幅二的輸出產生明顯非零的變化（確切數值取決於隨機初始化的權重）。

通道與 1×1 卷積

白話版。通道是特徵圖的「是什麼」維度；高與寬是「在哪裡」維度。1×1 卷積在每個像素混合通道，不看鄰居。

具有 C_{in} 個輸入與 C_{out} 個輸出的 1×1 卷積，會把每個像素的通道向量乘上同一個 $C_{\text{out}} \times C_{\text{in}}$ 矩陣，正如上面程式碼所驗證。Lin 等人的 Network in Network [17] 把這個想法當作迷你的逐像素 MLP，並提出**全域平均池化**（global average pooling）作為分類器頭部。GoogLeNet [16] 在昂貴的 3×3 與 5×5 分支之前用 1×1 網路層減少通道數，ResNet 的瓶頸區塊 [5] 則把 3×3 層夾在 1×1 縮減層與 1×1 擴張層之間。動機都相同：空間濾波的成本是通道混合的 k^2 倍，所以便宜的部分交給 1×1 網路層，昂貴的部分保持窄小。

池化與下採樣

白話版。池化（pooling）用一個數字（通常是最大值或平均值）概括特徵圖的每個小區塊。它縮小特徵圖，並讓表示對小幅平移較不敏感。

視窗 2×2、步幅二的**最大池化**保留每個區塊中的最大值：「這附近有沒有出現這個特徵？」**平均池化**保留平均值。**全域平均池化**把每個通道在所有位置上平均，不論輸入大小，每個通道都產生一個數字，這就是現代分類器能接受不同影像大小的原因。池化沒有可學習的參數。它的梯度很簡單：最大池化把整個上游梯度交給每個視窗中勝出的輸入；平均池化則把梯度平均分給視窗內所有輸入。

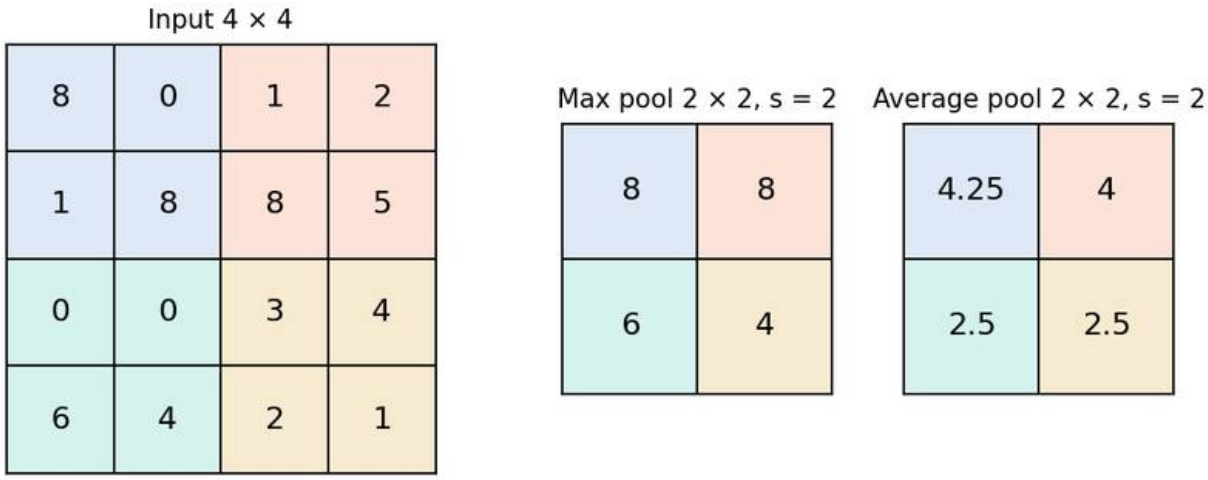


圖 7 — 視窗 2×2、步幅 2 的最大池化與平均池化。每個彩色區塊變成一個數字。

無論用池化或帶步幅的卷積，下採樣都能節省運算並擴大後續的感受野，但它也是一種沒有適當低通濾波的**次取樣**（subsampling）。高頻會產生混疊（aliasing），輸入只移動一個像素，輸出就可能明顯改變，正如上面程式碼所量測。Zhang [20] 證明在次取樣前插入一個小模糊（「抗混疊」或「blur pooling」），能讓標準 CNN 的平移不變性顯著提升：這是古典訊號處理規則修好深度網路的漂亮例子。

等變性不等於不變性

白話版。平移輸入，卷積特徵圖也會跟著平移。但這不等於最終預測保持不變。

令 T_Δ 把影像平移 Δ 個像素。在遠離邊界處，步幅為一的卷積具有**平移等變性**（translation-equivariant）：$f(T_\Delta X) = T_\Delta f(X)$。分類器要的是**不變性**（invariance）：$g(T_\Delta X) = g(X)$。CNN 藉由堆疊等變的網路層再加上池化（特別是會捨棄位置資訊的全域平均池化），得到近似的不變性。填補、步幅、裁切與有限的影像大小都會破壞精確的等變性。旋轉與尺度等變性**並不是**內建的；必須靠資料擴增或特別設計的群等變（group-equivariant）網路層來提供。

這個區別在實務上很重要。偵測器必須是等變的：人移動了，預測框也要跟著移動。分類器應該是不變的：人移動了，標籤不該改變。同一個骨幹網路可以服務兩者，只是頭部不同。

感受野

白話版。前面的網路層看到影像的小碎片；較深的網路層把這些碎片組合起來，看到較大的區域。

一個單元的**感受野**（receptive field）是能影響它的輸入像素集合。令 r_ℓ 為第 ℓ 層之後的感受野寬度，j_ℓ 為**跳距**（jump），即第 ℓ 層相鄰單元之間相隔的輸入像素數。從 $r_0 = j_0 = 1$ 開始，卷積核大小 k_ℓ、膨脹 d_ℓ、步幅 s_ℓ 的網路層給出

$$r_\ell = r_{\ell-1} + (d_\ell(k_\ell - 1))\, j_{\ell-1}, \qquad j_\ell = s_\ell\, j_{\ell-1}.$$

三個堆疊的步幅一 3×3 網路層看到 7×7 個像素，面積與單一 7×7 網路層相同，但權重是 $3 \cdot 9C^2 = 27C^2$ 而不是 $49C^2$，並且有三個非線性而非一個；這正是 VGG [15] 的核心設計論點。步幅二的網路層會讓跳距加倍，因此之後每一層的感受野都以兩倍速度成長。

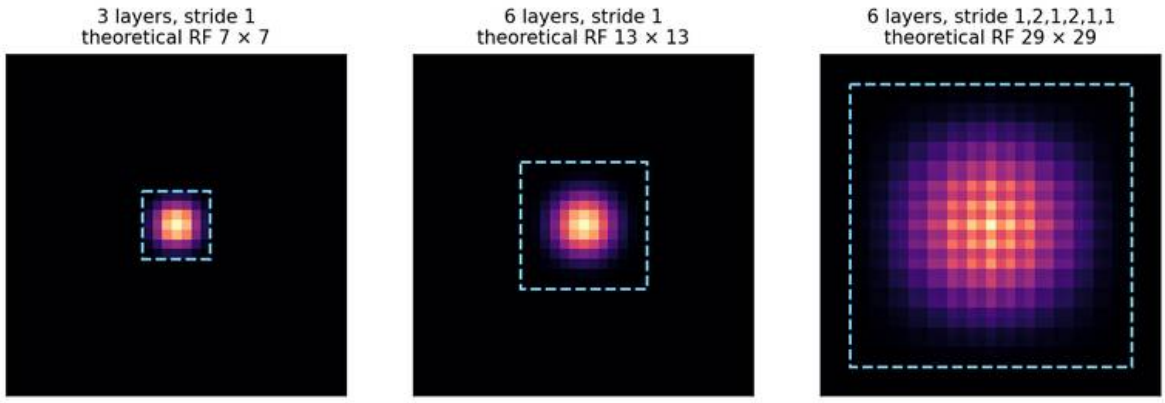


圖 *8* — 理論感受野（虛線）與實際影響力的比較，實際影響力以一個中央輸出穿過 3×3 平均網路層堆疊後的輸入梯度量測。影響力呈類高斯分布並集中在中央；步幅則加上棋盤狀圖樣。

理論感受野說明哪些像素**可能**有影響。Luo 等人 [3] 透過輸入梯度量測**有效感受野**（effective receptive field），如圖 8 所示，發現它大致呈高斯形狀，只占理論感受野的一小部分，而且對均勻的網路層堆疊而言，大約隨深度的平方根成長。由此得到兩個實務教訓：網路可能需要比理論感受野所暗示的更多上下文；而下採樣、膨脹與大型卷積核，都是不必無限加深就能擴大有效感受野的方法。

卷積的反向傳播

白話版。卷積是線性的，所以它的反向傳播也是卷積：對卷積核的梯度是把輸入與上游梯度做互相關，對輸入的梯度則是把上游梯度與卷積核做卷積。

精確版。取單一通道、無填補、步幅一：$Y_{u,v} = \sum_{i,j} K_{i,j} X_{u+i,v+j}$。令 L 為損失，$G_{u,v} = \partial L / \partial Y_{u,v}$ 為反向傳播傳來的**上游梯度**，如ML 02 所述。每個權重 $K_{i,j}$ 用在每個輸出位置，所以依連鎖律，它的梯度是對這些位置的加總：

$$\frac{\partial L}{\partial K_{i,j}} = \sum_{u,v} G_{u,v}\, X_{u+i,v+j}.$$

這是輸入 X 與 G 的 valid 互相關，也說明了為什麼權重共享不會讓訓練變複雜：共享的權重只是把每個位置的梯度累加起來。每個輸入像素 $X_{m,n}$ 會貢獻給所有視窗涵蓋它的輸出，對應係數為 $(i,j) = (m-u, n-v)$：

$$\frac{\partial L}{\partial X_{m,n}} = \sum_{i,j} G_{m-i,n-j}\, K_{i,j},$$

其中 G 在其範圍外視為零。這是 G 與 K 的 **full 卷積**，等價於把零填補後的 G 與翻轉的卷積核做互相關。這正是**轉置卷積**所計算的東西，轉置卷積也因此得名。偏差的梯度就是每個輸出通道的 $\sum_{u,v} G_{u,v}$。若有多個通道，輸入梯度還要對輸出通道加總，卷積核梯度還要對批次加總。

```
import torch
import torch.nn.functional as F

torch.manual_seed(0)
X = torch.randn(1, 1, 6, 7, requires_grad=True)     # input
K = torch.randn(1, 1, 3, 3, requires_grad=True)     # kernel
Y = F.conv2d(X, K)                                   # 'valid' correlation: 4 x 5
G = torch.randn_like(Y)                              # pretend upstream gradient dL/dY
(Y * G).sum().backward()                             # so that dL/dY = G exactly

# 1) dL/dK: correlate the input with the upstream gradient
dK = F.conv2d(X.detach(), G)
# 2) dL/dX: 'full' convolution of G with K = correlation of zero-padded G with flipped K
dX = F.conv2d(F.pad(G, (2, 2, 2, 2)), K.detach().flip(-1, -2))
# 3) the same thing is what a transposed convolution computes
dX_t = F.conv_transpose2d(G, K.detach())

print("dL/dK matches autograd:", torch.allclose(dK, K.grad, atol=1e-5))
print("dL/dX matches autograd:", torch.allclose(dX, X.grad, atol=1e-5))
print("conv_transpose2d gives the same:", torch.allclose(dX_t, X.grad, atol=1e-5))
```

三行都印出 `True`。`(Y * G).sum().backward()` 這個技巧讓 autograd 使用我們指定的上游梯度 G，是測試任何手推反向傳播的好方法。

經典架構速覽

白話版。每個知名的 CNN 都是卷積、非線性與下採樣的堆疊。這些年改變的是深度、如何讓深層堆疊可以訓練，以及如何聰明地分配運算量。

模型（年份）	關鍵想法	帶來的啟示
LeNet-5（1998）[13]	卷積 → 次取樣 → 卷積 → 次取樣 → 全連接，端到端訓練	學習出來的卷積特徵在數字辨識上勝過手工特徵
AlexNet（2012）[14]	更深更寬、ReLU、dropout、資料擴增、GPU 訓練	大量標註資料加上 GPU 讓 CNN 贏下 ImageNet
VGG（2014）[15]	只用 3×3 卷積，16–19 層	以小卷積核加深既簡單又有效
GoogLeNet / Inception（2014）[16]	平行的 1×1、3×3、5×5 分支，搭配 1×1 縮減	以受控成本取得多尺度特徵
ResNet（2015）[5]	殘差連接 $y = x + F(x)$、BatchNorm	非常深的網路（100 層以上）變得可以訓練
U-Net（2015）[4]	編碼器–解碼器，在每個解析度加上跳接	用少量影像得到密集、像素對齊的輸出
MobileNetV2（2018）[7]	深度可分離卷積、反轉殘差、線性瓶頸	適合手機的高準確率模型
EfficientNet（2019）[8]	深度、寬度、解析度的複合縮放	各維度要一起縮放，而不是一次一個

模型（年份）	關鍵想法	帶來的啟示
ConvNeXt（2022）[9]	7×7 深度卷積核、LayerNorm、反轉瓶頸、Transformer 時代的訓練方法	現代化的純 CNN 可與視覺 Transformer 競爭

從 LeNet 到 ResNet 的模式是深度穩定增加。AlexNet [14] 證明，用 GPU、ReLU 與 dropout 在 ImageNet 上訓練的深層 CNN 能大幅勝過傳統流程。VGG [15] 主張使用一致的 3×3 網路層堆疊，如感受野小節所計算。GoogLeNet [16] 用 1×1 縮減讓寬度變便宜。接著深度撞上了最佳化之牆：層數更多的普通網路反而有**更高的訓練誤差**，這不是過擬合（overfitting），而是最佳化失敗。ResNet [5] 用恆等捷徑解決了這個問題。DLCV 02 會沿著這條路線繼續探討訓練方法、遷移學習與評估。

殘差區塊與正規化

白話版。不要求一個區塊計算整個輸出，而是要它計算對輸入的修正量。

殘差區塊計算 $y = x + F_\theta(x)$，其中 F_θ 通常是兩到三個卷積，搭配正規化與 ReLU。它的雅可比矩陣為 $\partial y/\partial x = I + \partial F_\theta/\partial x$：即使 F_θ 貢獻很小，恆等路徑仍能把訊號往前送、把梯度往回傳。若 $F_\theta \approx 0$ 就是一個區塊所能做到最好的，它也能輕易學會，所以增加區塊不再傷害最佳化。當形狀改變（更多通道、步幅二）時，捷徑變成投影 $P(x)$，通常是帶步幅的 1×1 卷積。

批次正規化（batch normalization）[19] 在訓練時用批次平均與變異數標準化每個通道，再套用學習出來的縮放與平移；評估時則使用移動平均。它讓訓練更快、對初始化較不敏感，但在批次非常小或相關性高時表現不佳，此時**群組正規化**（GroupNorm）[6]（在每個樣本內對通道群組計算統計量）較可靠。驗證之前一定要呼叫 `model.eval()`，讓 BatchNorm 使用移動統計量。

```
import torch
from torch import nn

count = lambda m: sum(p.numel() for p in m.parameters())

C = 64
dense = nn.Conv2d(C, C, 3, padding=1, bias=False)
separable = nn.Sequential(
    nn.Conv2d(C, C, 3, padding=1, groups=C, bias=False),   # depthwise: one 3x3 per channel
    nn.Conv2d(C, C, 1, bias=False))                        # pointwise: mix channels
print("dense:", count(dense), "| depthwise+pointwise:", count(separable),
      "| ratio:", round(count(separable) / count(dense), 4), "vs 1/C + 1/9 =",
      round(1 / C + 1 / 9, 4))

class BasicBlock(nn.Module):
    """ResNet-style block: y = relu(x + F(x)), with a 1x1 projection when shapes change."""
    def __init__(self, c_in, c_out, stride=1):
        super().__init__()
        self.f = nn.Sequential(
            nn.Conv2d(c_in, c_out, 3, stride, 1, bias=False), nn.BatchNorm2d(c_out),
            ↪  nn.ReLU(),
            nn.Conv2d(c_out, c_out, 3, 1, 1, bias=False), nn.BatchNorm2d(c_out))
        self.proj = (nn.Identity() if stride == 1 and c_in == c_out else
```

```
                    nn.Sequential(nn.Conv2d(c_in, c_out, 1, stride, bias=False),
                                  nn.BatchNorm2d(c_out)))
    def forward(self, x):
        return torch.relu(self.proj(x) + self.f(x))

x = torch.randn(2, 64, 32, 32)
print("same shape :", tuple(BasicBlock(64, 64)(x).shape))
print("downsample :", tuple(BasicBlock(64, 128, stride=2)(x).shape))
```

輸出顯示密集網路層有 36,864 個權重，可分離組合則有 4,672 個，比例 0.1267 與下面的公式一致；區塊輸出形狀分別為 $64 \times 32 \times 32$ 與 $128 \times 16 \times 16$。

高效率 CNN：把空間濾波與通道混合分開

白話版。「看周圍」（空間濾波）與「組合特徵」（通道混合）不必在同一個昂貴的運算中完成。

密集的 $k \times k$ 卷積有 $k^2 C_{\text{in}} C_{\text{out}}$ 個權重。**深度**（depthwise）卷積分別濾波每個通道（$k^2 C_{\text{in}}$ 個權重，PyTorch 中為 `groups=C`）；接著的**逐點**（pointwise）1×1 卷積混合通道（$C_{\text{in}} C_{\text{out}}$ 個權重）。當 $C_{\text{in}} = C_{\text{out}} = C$ 時，

$$\frac{\text{depthwise} + \text{pointwise}}{\text{dense}} = \frac{k^2 C + C^2}{k^2 C^2} = \frac{1}{C} + \frac{1}{k^2}.$$

當 $k = 3$ 且 C 很大時，這個比例趨近 $1/9$。

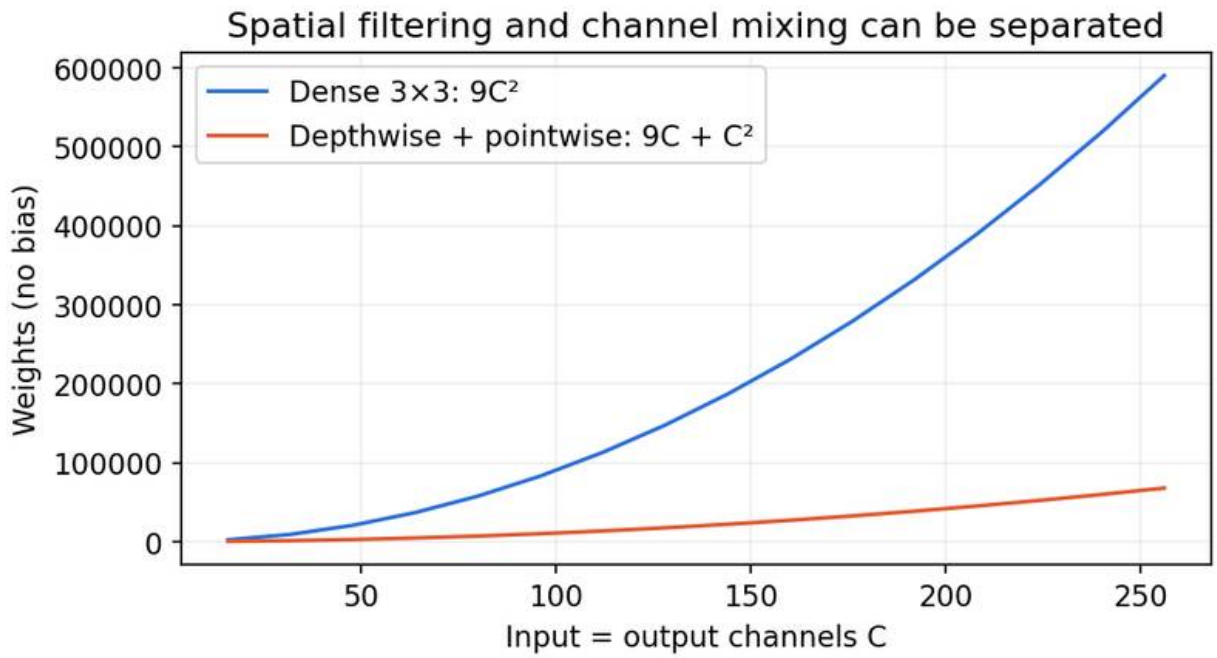


圖 *9* — 輸入與輸出通道數相同時的權重數解析計算。權重較少不代表延遲一定較低：深度卷積的運算強度低，在某些硬體上可能跑得很慢。

MobileNetV2 [7] 的區塊先用 1×1 網路層擴張通道，以深度 3×3 濾波，再以線性（無 ReLU）1×1 瓶頸投影回去，並在兩個窄端之間加上殘差連接。EfficientNet [8] 用單一複合係數同時縮放深度、寬度與輸入解析度。ConvNeXt [9] 從 ResNet 出發，一次一項地套用與視覺 Transformer 相關的設計選擇：patchify 起始層、7×7 深度卷積核、反轉瓶頸、較少的激勵函數、LayerNorm，以及搭配大量資料擴增的較長訓練。結果與相近大小的 Swin Transformer 相當，顯示 Transformer 的優勢有很大一部分來自訓練方法與巨觀設計，而不只是注意力本身。

程式碼：完整的小影像分類器

這個範例需要 `torch`、`numpy` 與 `scikit-learn`。它使用內建的 8×8 數字資料集，因此不需要下載或 GPU。分層切分得到彼此獨立的訓練、驗證與測試集：驗證集用來挑選最佳的訓練週期，測試集只在最後評估一次。這是教學用的基準，而不是 ImageNet 的訓練方法。

```
import copy
import numpy as np
import torch
from torch import nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0)
torch.set_num_threads(1)
d = load_digits()
X = torch.tensor(d.images[:, None] / 16.0, dtype=torch.float32)
y = torch.tensor(d.target, dtype=torch.long)
train, test = train_test_split(np.arange(len(y)), test_size=.2,
                               stratify=d.target, random_state=0)
train, val = train_test_split(train, test_size=.2,
                              stratify=d.target[train], random_state=1)
model = nn.Sequential(
    nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(),
    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(),
    nn.MaxPool2d(2),                     # B × 32 × 4 × 4
    nn.Flatten(), nn.Linear(32 * 4 * 4, 10),
)
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
best_loss, best_state = float("inf"), None
for epoch in range(12):
    model.train()
    order = torch.tensor(train)[torch.randperm(len(train))]
    for idx in order.split(64):
        loss = nn.functional.cross_entropy(model(X[idx]), y[idx])
        opt.zero_grad()
        loss.backward()
        opt.step()
    model.eval()
    with torch.no_grad():
        val_loss = nn.functional.cross_entropy(model(X[val]), y[val]).item()
    if val_loss < best_loss:
        best_loss = val_loss
```

```
        best_state = copy.deepcopy(model.state_dict())
model.load_state_dict(best_state)
model.eval()
with torch.no_grad():
    accuracy = (model(X[test]).argmax(1) == y[test]).float().mean().item()
print("parameters:", sum(p.numel() for p in model.parameters()))
print("best validation loss:", round(best_loss, 4))
print("held-out accuracy:", round(accuracy, 4))
```

在我們的 CPU 上執行，它回報 9,930 個參數，保留測試集準確率約 96%（確切數字會因 PyTorch 版本略有差異）。交叉熵接收的是 **logits**，前面不接 softmax。頭部把 $32 \times 4 \times 4$ 特徵圖攤平，而不是做全域平均池化，因為在很小的數字影像上，筆畫的位置本身就帶有資訊；對於大小不一的影像，自適應或全域池化能提供固定大小的頭部。

要檢查錯誤，而不只看分數。哪些數字配對容易混淆？把每個測試數字平移一個像素會不會變差？在把所有進步都歸功於卷積之前，先與參數量相當的 MLP 以及古典 SVM 比較，如IP 13 所示。

現代觀點

綜述文獻。有幾篇綜述描繪了 CNN 的全貌，各有不同重點。Gu 等人 [23]（*Pattern Recognition*，2018）依元件整理 CNN 的進展：卷積變體、池化、激勵函數、損失函數、正規化與最佳化，接著是應用。Khan 等人 [22]（*Artificial Intelligence Review*，2020）依主要創新提出架構分類法：空間利用（卷積核大小）、深度、多路徑設計（捷徑、Inception 分支）、寬度、特徵圖利用（通道注意力）以及基於注意力的設計。Li 等人 [21]（*IEEE TNNLS*，2022）涵蓋從 LeNet 開始的歷史，比較一維、二維與多維卷積，並綜覽輕量化網路與應用，給出選擇超參數的經驗法則。Alzubaidi 等人 [2]（*Journal of Big Data*，2021）以 CNN 架構為中心，對深度學習做了廣泛、應用導向的回顧，並討論資料稀少、遷移學習與硬體。這些綜述都早於 ConvNeXt，因此請把它們當作 2012–2020 年代的地圖來讀。

Transformer 之後的 CNN（2022–2026）。視覺 Transformer（ML 04）促使學界重新檢視卷積，主要發現是兩者的差距比早期比較所顯示的要小：

- **現代化的 CNN**。 ConvNeXt [9] 證明經過細心現代化的 ResNet 能與 Swin 相當。ConvNeXt V2 [24]（CVPR 2023）加入以稀疏卷積實作的遮罩自編碼器預訓練，以及全域響應正規化層，把自監督預訓練（見ML 05）帶進純 CNN。
- **大型卷積核**。 RepLKNet [26]（CVPR 2022）使用大到 31×31 的深度卷積核，並以重新參數化的小卷積核降低最佳化難度，藉此擴大有效感受野。UniRepLKNet [27]（CVPR 2024）把大卷積核設計延伸到音訊、影片、點雲與時間序列。
- **動態與可變形卷積**。 InternImage [25]（CVPR 2023）把取樣位置由輸入預測的可變形卷積，擴展成用於偵測與分割的大型基礎骨幹網路。
- **規模**。 Smith 等人 [29]（2023）在大型 JFT-4B 影像資料集上預訓練 NFNet CNN，發現在可比較的預算下，其效能遵循與視覺 Transformer 相似的運算量縮放趨勢，主張在大規模下主導結果的是運算量與資料，而不是架構。
- **邊緣裝置**。 MobileNetV4 [28]（ECCV 2024）設計了在 CPU、GPU、DSP 與加速器上都有良好延遲的區塊與神經架構搜尋，承認光看參數量無法預測速度。

CNN 在 2026 年的定位。卷積仍是下列情境的預設選擇：受延遲限制與裝置端的推論、高解析度的密集預測（復原、分割、醫學影像，見DLCV 04）、局部性偏差有幫助的小資料集，以及 CNN–Transformer 混合模型的起始層或前段。大型視覺基礎模型多半以 Transformer 為主，但它們的 patch 嵌入、上採樣解碼器與擴散 U-Net 中經常包含卷積。

開放問題。下採樣下的精確平移等變性（混疊 [20]）；以低成本內建旋轉與尺度對稱性；理解一個任務真正需要多大的有效感受野 [3]；以及架構之間的公平比較，這需要先對齊資料、資料擴增、解析度、訓練排程與運算量，才能把差異歸因於卷積或注意力。

重點整理

- 卷積核在影像上滑動，並在每個位置計算加權和。函式庫通常計算的是**互相關**；真正的卷積會翻轉卷積核，這只對不對稱的手工卷積核有影響。
- 卷積核權重的總和決定它對平坦區域的響應：總和為 1 時模糊但保持亮度；總和為 0 時偵測變化。請用浮點數濾波，並刻意選擇邊界規則。
- 卷積層是一組具有**局部連接**與**權重共享**的學習式濾波器。它的參數量 $C_{\text{out}}(C_{\text{in}}k^2 + 1)$ 與影像大小無關。
- 填補、步幅與膨脹決定輸出大小 $\lfloor (n + 2p - d(k-1) - 1)/s \rfloor + 1$。$1 \times 1$ 卷積混合通道；池化與步幅能下採樣，但可能產生混疊。
- 卷積具有平移**等變性**；不變性來自池化。感受野隨深度與步幅成長，但有效感受野遠小於理論感受野。
- 卷積的反向傳播本身也是卷積：卷積核梯度是輸入與上游梯度的互相關，輸入梯度則是轉置卷積。
- 架構史是一段關於深度（LeNet → VGG → ResNet）、有效率地使用通道（1×1、深度可分離）與更好訓練方法的故事。在預算相當時，現代化的 CNN 仍能與 Transformer 競爭。

練習

1. **設計並驗證**。寫出一個只偵測水平邊緣的 3×3 卷積核，以及另一個只偵測從左上到右下對角邊緣的卷積核。在卷積核實驗室中，或用 `cv2.filter2D` 在一張含有水平線、垂直線與對角線的合成影像上驗證兩者。

   提示

   水平邊緣是沿 y 方向的變化，所以使用 $S_y = S_x^{\top}$（各列為 $-1,-2,-1$ / $0,0,0$ / $1,2,1$）。對角邊緣則把導數放在跨越對角線的方向，例如 $\begin{bmatrix} 0 & 1 & 2 \\ -1 & 0 & 1 \\ -2 & -1 & 0 \end{bmatrix}$。它的權重總和為零，所以平坦區域輸出為零。

2. **卷積核總和**。為什麼模糊卷積核的權重總和應該為一？先預測再檢查：一張中灰色的 `uint8` 影像經過權重都等於 $2/9$ 的 3×3 卷積核濾波後會怎樣？若改用 `float32` 濾波又有什麼不同？

   提示

數值為 c 的平坦區域會映射到 $c\sum K$。總和為二時，灰階 128 會變成 256，在 `uint8` 中飽和為 255；所有比中灰亮的區域都變成白色。在 `float32` 中數值加倍但不會被截斷，之後可以再重新縮放。

3. **組合**。證明連續做兩次 3×3 方框模糊等於一個 5×5 卷積核，並寫出這個卷積核。在遠離邊界處以數值驗證，並解釋為什麼在邊界處等式可能不成立。

   提示

   依結合律，組合後的卷積核是方框核與自己的卷積：$[1,2,3,2,1]$ 與自己的外積再除以 81。在邊界處，第一次濾波已經編造了填補值，第二次再對那個結果填補；而單次 5×5 濾波填補的是原始影像，所以兩者在寬度為二的外框中不同。

4. **網路層算術**。對 64×64 的 RGB 輸入，以及一個有 16 個濾波器、$k=5$、$p=2$、$s=2$ 的網路層：寫出輸出形狀與參數數量。接著計算三個步幅分別為 1、2、1 的 3×3 網路層之後的 (r,j)，並與所有步幅都為一的情況比較。

   提示

   $\lfloor (64+4-4-1)/2\rfloor +1=32$，所以輸出為 $16\times 32\times 32$，參數為 $16(3\cdot 25+1)=1216$。感受野：$(3,1)\to(5,2)\to(9,2)$。步幅全為一則得到 $(7,1)$。

5. **手算梯度**。對 1×4 的輸入 $X=[x_1,x_2,x_3,x_4]$ 與卷積核 $K=[k_1,k_2]$ 做 valid 互相關，寫出 Y，再以 $G=\partial L/\partial Y$ 推導 $\partial L/\partial K$ 與 $\partial L/\partial X$。用反向傳播程式碼中的 autograd 技巧檢查你的公式。

   提示

   $Y=[k_1x_1+k_2x_2,\ k_1x_2+k_2x_3,\ k_1x_3+k_2x_4]$。於是 $\partial L/\partial k_1=G_1x_1+G_2x_2+G_3x_3$、$\partial L/\partial k_2=G_1x_2+G_2x_3+G_3x_4$（$X$ 與 G 的互相關）。$\partial L/\partial X=[G_1k_1,\ G_1k_2+G_2k_1,\ G_2k_2+G_3k_1,\ G_3k_2]$，即 G 與 K 的 full 卷積。

6. **數字分類的消融實驗**。在分類器中，把 `Flatten + Linear(512, 10)` 換成全域平均池化 + `Linear(32, 10)`。比較參數量、驗證損失與測試準確率，再用平移一個像素的測試數字評估兩個模型。解釋你觀察到的取捨。

   提示

   使用 `nn.AdaptiveAvgPool2d(1)` 再接 `nn.Flatten()`。池化移除了大部分頭部參數與位置資訊，可能有助於對平移的穩健性，但在筆畫位置很重要的 8×8 數字上可能變差。其他設定保持不變，只用驗證集做選擇，並同時回報準確率與參數量。

參考文獻

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 9. Book

2. L. Alzubaidi et al., “Review of deep learning: concepts, CNN architectures, challenges, applications, future directions,” *Journal of Big Data*, 2021. Review
3. W. Luo, Y. Li, R. Urtasun and R. Zemel, “Understanding the Effective Receptive Field in Deep Convolutional Neural Networks,” NeurIPS, 2016. Paper
4. O. Ronneberger, P. Fischer and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” MICCAI, 2015. Paper
5. K. He, X. Zhang, S. Ren and J. Sun, “Deep Residual Learning for Image Recognition,” CVPR, 2016. Paper
6. Y. Wu and K. He, “Group Normalization,” ECCV, 2018. Paper
7. M. Sandler et al., “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” CVPR, 2018. Paper
8. M. Tan and Q. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” ICML, 2019. Paper
9. Z. Liu et al., “A ConvNet for the 2020s,” CVPR, 2022. Paper
10. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023, ch. 10. Book and notebooks
11. OpenCV, “Image Filtering” (`filter2D`, border types), documentation, version 4.x. Documentation
12. V. Dumoulin and F. Visin, “A guide to convolution arithmetic for deep learning,” arXiv:1603.07285, 2016. arXiv
13. Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, 1998. DOI
14. A. Krizhevsky, I. Sutskever and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” NeurIPS, 2012. Paper
15. K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” ICLR, 2015. arXiv
16. C. Szegedy et al., “Going Deeper with Convolutions,” CVPR, 2015. arXiv
17. M. Lin, Q. Chen and S. Yan, “Network In Network,” ICLR, 2014. arXiv
18. F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” ICLR, 2016. arXiv
19. S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” ICML, 2015. arXiv
20. R. Zhang, “Making Convolutional Networks Shift-Invariant Again,” ICML, 2019. arXiv
21. Z. Li, F. Liu, W. Yang, S. Peng and J. Zhou, “A Survey of Convolutional Neural Networks: Analysis, Applications, and Prospects,” *IEEE Transactions on Neural Networks and Learning Systems*, 2022. DOI
22. A. Khan, A. Sohail, U. Zahoora and A. S. Qureshi, “A Survey of the Recent Architectures of Deep Convolutional Neural Networks,” *Artificial Intelligence Review*, 2020. arXiv
23. J. Gu et al., “Recent Advances in Convolutional Neural Networks,” *Pattern Recognition*, 2018. arXiv
24. S. Woo et al., “ConvNeXt V2: Co-designing and Scaling ConvNets with Masked Autoencoders,” CVPR, 2023. arXiv
25. W. Wang et al., “InternImage: Exploring Large-Scale Vision Foundation Models with Deformable Convolutions,” CVPR, 2023. arXiv
26. X. Ding et al., “Scaling Up Your Kernels to 31x31: Revisiting Large Kernel Design in CNNs”

(RepLKNet), CVPR, 2022. arXiv
27. X. Ding et al., "UniRepLKNet: A Universal Perception Large-Kernel ConvNet for Audio, Video, Point Cloud, Time-Series and Image Recognition," CVPR, 2024. arXiv
28. D. Qin et al., "MobileNetV4: Universal Models for the Mobile Ecosystem," ECCV, 2024. arXiv
29. S. L. Smith, A. Brock, L. Berrada and S. De, "ConvNets Match Vision Transformers at Scale," arXiv:2310.16764, 2023. arXiv
30. M. D. Zeiler and R. Fergus, "Visualizing and Understanding Convolutional Networks," ECCV, 2014. arXiv
31. PyTorch, `torch.nn.Conv2d` documentation. Documentation

ML 03A ・循環與遞迴神經網路

從序列與樹結構學習：共享的循環狀態、時間反向傳播、梯度消失與爆炸、LSTM 與 GRU 閘門、雙向與編碼器–解碼器模型、作為通往 Transformer 橋梁的注意力機制、遞迴網路，以及它們在電腦視覺中的應用。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-03a-recurrent-recursive-networks/

你將學到

- 循環網路如何在每個時間步重複使用同一組權重，因而能讀取任意長度的序列。
- 如何用時間反向傳播（backpropagation through time, BPTT）訓練它，以及為什麼跨越長時間的梯度會消失或爆炸。
- LSTM 與 GRU 的閘門如何建立一條受保護的記憶路徑；我們逐條寫出方程式，並與 PyTorch 的實作比對。
- 雙向、堆疊與編碼器–解碼器（序列到序列）模型如何組裝，以及注意力機制如何移除編碼器的瓶頸，進而催生 Transformer。
- 遞迴網路如何沿著樹狀結構、而不是鏈狀結構組合表示。
- 循環結構在電腦視覺中的用途（影像描述、影片、時空預測），以及 Mamba 與 Mamba-2 如何使用選擇性狀態空間、平行掃描與固定大小的串流記憶。

先看全貌

ML 03 對影像只用了一個假設：相鄰像素彼此相關，而且同一個圖樣可能出現在任何位置，所以同一個濾波器可以在**空間**上重複使用。序列也適用同樣的技巧，只是換成在**時間**上重複使用。影片是一連串影格，圖說是一連串字詞，筆跡是一連串座標。**循環神經網路**（recurrent neural network, RNN）一次讀入序列中的一個元素，把一份稱為**隱藏狀態**（hidden state）的摘要從這一步帶到下一步，並在每一步使用相同的權重 [1]。**遞迴神經網路**（recursive neural network）則把這個想法從鏈推廣到樹：它反覆套用同一條共享的組合規則，把局部拼成整體。

中文文獻常把這兩個英文名詞混用；本篇固定以「循環神經網路」指 recurrent neural network、以「遞迴神經網路」指 recursive neural network，以免混淆。

> **用最簡單的話說**
>
> 聽故事時，你不會記住每一個字，而是在腦中保留一幅「到目前為止發生了什麼」的小圖畫，每聽到新的一句就更新這幅圖。循環網路就是這樣做的：它有一本小筆記本（隱藏狀態），一次讀一個字或一張影格，每讀完一個就用同一套規則改寫筆記。LSTM 是一本附了橡皮擦和筆的筆記本，網路會學著小心使用它們，讓開頭的重要筆記到了最後也不會被塗糊。遞迴網路則像玩 LEGO：把兩塊小積木扣成一塊大的，再把大的扣成更大的，每次都用同一種扣法。

既然 Transformer 已經主宰大規模的語言與視覺任務，為什麼還要學 RNN？有三個理由。第一，現代序列建模的每一個概念——教師強制、編碼器–解碼器、注意力、遮罩、因果式生成——都是在 RNN 上發展出來的，而把 Transformer 理解成「對 RNN 局限的回答」是最容易的理解方式。第二，循環結構很適合**串流**處理：每一步只需要固定大小的狀態，適合機器人、感測器和裝置端的影片。第三，循環結構回來了：Mamba 等現代狀態空間模型本質上就是參數化方式很巧妙的循環式，而ML 04 會把它們應用到影像上。

循環狀態

白話版。在每一步，把目前的輸入和前一步的狀態結合成新的狀態，每次都使用相同的權重。需要輸出時，就從狀態讀出來。

精確版。設輸入序列為 $x_1, \dots, x_T$，其中 $x_t \in \mathbb{R}^d$；隱藏狀態為 $h_t \in \mathbb{R}^m$，起始狀態 h_0 固定（通常是零）。簡單（Elman [9]）RNN 為

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b), \qquad \hat{y}_t = g\big(W_y h_t + c\big).$$

$W_x \in \mathbb{R}^{m\times d}$ 把輸入映射到狀態空間，$W_h \in \mathbb{R}^{m\times m}$ 是把前一步資訊帶過來的**循環**權重矩陣，b 與 c 是偏差，W_y 把狀態映射成輸出，g 是輸出函數，例如 softmax。關鍵在於這些參數都不依賴 t。這就是**跨時間的參數共享**，相當於 CNN 權重共享在時間上的版本：不論序列有 5 步還是 5,000 步，模型的參數數量都相同，而在第 3 步學到的圖樣也能在第 300 步被辨認出來。

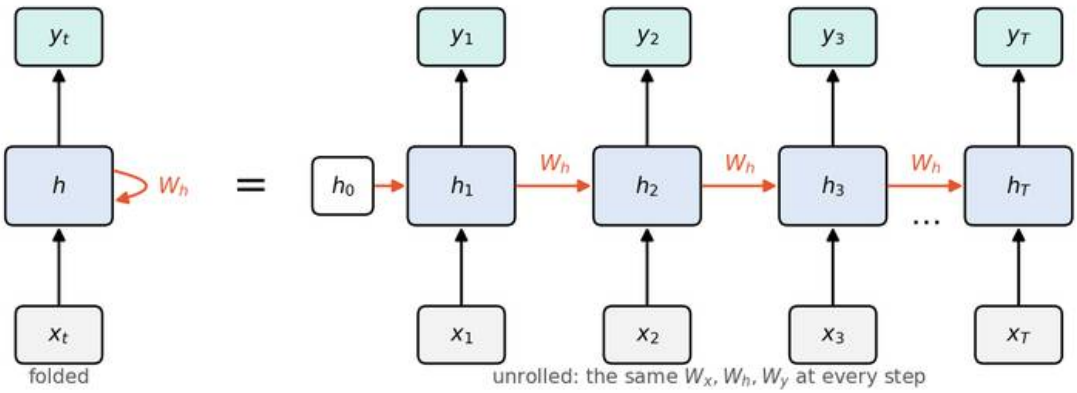


圖 *1 — RNN* 的折疊與展開兩種畫法。展開後，迴圈變成一個很深的前饋網路，而且每一層共用同一組權重。

展開（unrolling，圖 1）是最關鍵的思考步驟。把迴圈寫開 T 步，就得到一個 T 層深的普通前饋網路，而每一層的權重都相同。你在ML 02 學到的反向傳播，全部都能套用在這張展開後的計算圖上。

同一個循環式可以支援好幾種輸入–輸出型態：

型態	例子	從哪裡讀取輸出
多對一	分類一段影片；判斷一句話的情感	最後的狀態 h_T（或池化後的狀態）

型態	例子	從哪裡讀取輸出
多對多、對齊	為每張影格貼標籤；為每個字詞標註詞性	每一個 h_t
一對多	從一個影像向量生成圖說	一個把自己的輸出再餵回去的解碼器
多對多、不對齊	翻譯一句話；描述一段影片	**編碼器–解碼器**（稍後介紹）

由 $x_{\le t}$ 預測 x_{t+1} 的 RNN 是一個**自迴歸**（autoregressive）模型，從中取樣就能生成新的序列；但當作編碼器使用的 RNN 並不會自動變成生成器。決定任務的是輸出頭與損失函數。

程式碼：用 NumPy 寫出 RNN 與它的反向傳播

下面的程式碼手動實作前向傳遞與時間反向傳播，損失為 $L = v^\top h_T$，再用有限差分檢查其中一個梯度元素。請注意 `dWh += …`這一行：因為 W_h 在每一步都被使用，它的梯度是所有步貢獻的**總和**。

```
import numpy as np

rng = np.random.default_rng(0)
d, m, T = 3, 5, 7                                  # input size, state size, length
Wx = rng.normal(0, 0.5, (m, d))
Wh = rng.normal(0, 0.5, (m, m))
b = np.zeros(m)

def rnn_forward(xs, h0):
    hs = [h0]
    for x in xs:                                    # the SAME Wx, Wh, b at every step
        hs.append(np.tanh(Wx @ x + Wh @ hs[-1] + b))
    return hs

def rnn_backward(xs, hs, dhT):
    """BPTT for the loss L = dhT . h_T; returns dL/dWh and dL/dx_t for every t."""
    dWh, dxs, dh = np.zeros_like(Wh), [None] * len(xs), dhT
    for t in reversed(range(len(xs))):
        da = dh * (1 - hs[t + 1] ** 2)              # through tanh
        dWh += np.outer(da, hs[t])                  # shared weight: gradients ADD up
        dxs[t] = Wx.T @ da
        dh = Wh.T @ da                              # pass the gradient one step back
    return dWh, dxs

xs = rng.normal(size=(T, d))
hs = rnn_forward(xs, np.zeros(m))
dhT = rng.normal(size=m)
dWh, dxs = rnn_backward(xs, hs, dhT)

# finite-difference check of one entry of dL/dWh
eps, (i, j) = 1e-6, (2, 3)
Wh[i, j] += eps; up = dhT @ rnn_forward(xs, np.zeros(m))[-1]
Wh[i, j] -= 2 * eps; down = dhT @ rnn_forward(xs, np.zeros(m))[-1]
Wh[i, j] += eps
```

```
print("BPTT vs finite difference:", np.isclose(dWh[i, j], (up - down) / (2 * eps)))
print("|dL/dx_t| from first to last step:", np.round([np.linalg.norm(g) for g in dxs], 3))
```

它會印出 `True`，而輸入梯度的範數為 $[0.009, 0.015, 0.006, 0.029, 0.383, 0.212, 0.433]$：即使序列只有七步，早期輸入收到的梯度也比後期小了大約十到五十倍。這就是縮小版的梯度消失問題。

時間反向傳播

白話版。把網路展開，在展開後的計算圖上執行普通的反向傳播，再把每個共享權重的梯度加總起來。

精確版。把激活前的值寫成 $a_t = W_x x_t + W_h h_{t-1} + b$，因此 $h_t = \tanh(a_t)$。假設損失是各步的總和 $L = \sum_t L_t(h_t)$。從 $t = T$ 往回推，抵達 h_t 的總梯度為

$$\frac{\partial L}{\partial h_t} = \frac{\partial L_t}{\partial h_t} + \left(\frac{\partial h_{t+1}}{\partial h_t}\right)^{\top} \frac{\partial L}{\partial h_{t+1}}, \qquad \frac{\partial h_{t+1}}{\partial h_t} = \operatorname{diag}(1 - h_{t+1}^2)\, W_h.$$

第一項是第 t 步的局部損失；第二項是 h_t 影響到的所有後續部分，往回傳了一步。令 $\delta_t = (1 - h_t^2) \odot \partial L / \partial h_t$（激活前的梯度），參數梯度就是對時間的總和：

$$\frac{\partial L}{\partial W_h} = \sum_{t=1}^{T} \delta_t\, h_{t-1}^{\top}, \qquad \frac{\partial L}{\partial W_x} = \sum_{t=1}^{T} \delta_t\, x_t^{\top}, \qquad \frac{\partial L}{\partial b} = \sum_{t=1}^{T} \delta_t.$$

這個演算法稱為**時間反向傳播（BPTT）**，Werbos [8] 對它有清楚的闡述。它的計算量與 T 成線性，但記憶體也一樣：每個 h_t 都必須保存下來，供反向傳遞使用。

截斷式 BPTT

對非常長的資料流（一小時的影片、連續的感測紀錄）而言，保存所有狀態是不可能的。**截斷式 BPTT**（truncated BPTT）把資料流切成每段 k 步的視窗來處理。它會把隱藏狀態從一個視窗帶到下一個視窗，所以模型仍然「看得到」完整的過去；但它在每個視窗邊界把狀態從計算圖上**切斷**（detach），所以梯度最多只往回傳 k 步。因此，模型只有在視窗內的梯度訊號促使它學會保存長距離資訊時，才能利用這些資訊。

```
import torch
from torch import nn

torch.manual_seed(0)
rnn, head = nn.GRU(1, 16, batch_first=True), nn.Linear(16, 1)
params = list(rnn.parameters()) + list(head.parameters())
opt = torch.optim.Adam(params, lr=1e-2)
t = torch.arange(0, 2000) * 0.05
stream = torch.sin(t) + 0.1 * torch.randn_like(t)   # one long noisy signal
x, y = stream[:-1, None][None], stream[1:, None][None]   # predict the next sample

h, window = None, 50
```

```
for start in range(0, x.shape[1] - window, window):
    out, h = rnn(x[:, start:start + window], h)
    loss = nn.functional.mse_loss(head(out), y[:, start:start + window])
    opt.zero_grad()
    loss.backward()                       # gradients flow back at most `window` steps
    nn.utils.clip_grad_norm_(params, 1.0)
    opt.step()
    h = h.detach()                        # keep the state, cut the graph
print("final window MSE:", round(loss.item(), 4), "(noise variance is 0.01)")
```

把資料流走過一遍之後，最後一個視窗的均方誤差約為 0.014，接近加入雜訊所設定的 0.01 下限。如果忘了 `h.detach()`，第二個視窗會試圖反向傳播進一張已經被釋放的計算圖，PyTorch 會丟出錯誤；反過來，若在每個視窗都把 `h = None` 重設，就等於把上下文丟掉了。

梯度消失與梯度爆炸

白話版。從第 t 步傳回第 s 步的梯度，在每一步都會乘上大致相同的矩陣。反覆相乘不是把它縮到接近零，就是把它放大到爆掉，所以單純的 RNN 很難學到超過幾十步的依賴關係。

精確版。把每一步的 Jacobian 串起來，得到

$$\frac{\partial h_t}{\partial h_s} = \prod_{k=s+1}^{t} \frac{\partial h_k}{\partial h_{k-1}} = \prod_{k=s+1}^{t} \operatorname{diag}\!\left(1 - h_k^2\right) W_h,$$

這是 $t-s$ 個矩陣依序相乘的乘積（矩陣乘法不可交換，所以順序很重要）。因為 $|\tanh'| \le 1$，

$$\left\| \frac{\partial h_t}{\partial h_s} \right\| \le \left(\sigma_{\max}(W_h) \right)^{t-s},$$

其中 $\sigma_{\max}$ 是最大奇異值。若 $\sigma_{\max}(W_h) < 1$，來自遠處步驟的梯度會以指數速度**消失**。若 W_h 有很大的奇異值、而且神經元沒有飽和，梯度就可能**爆炸**。Bengio、Simard 與 Frasconi [6] 在 1994 年證明，這不是某種特定訓練方法的缺陷：一個能在雜訊下穩健保存資訊的系統，必然處於長距離依賴梯度會縮小的狀態。Pascanu、Mikolov 與 Bengio [7] 用動態系統的工具重新分析這個問題，並提出處理梯度爆炸的實用方法。

用純量線性 RNN $h_t = \lambda h_{t-1} + x_t$ 最容易看出直覺：此時 $\partial h_t / \partial h_s = \lambda^{t-s}$；當 $\lambda = 0.9$ 時是 $0.9^{80} \approx 2 \times 10^{-4}$，當 $\lambda = 1.1$ 時是 $1.1^{80} \approx 2 \times 10^{3}$。圖 2 顯示在真實網路中量測到的相同行為。

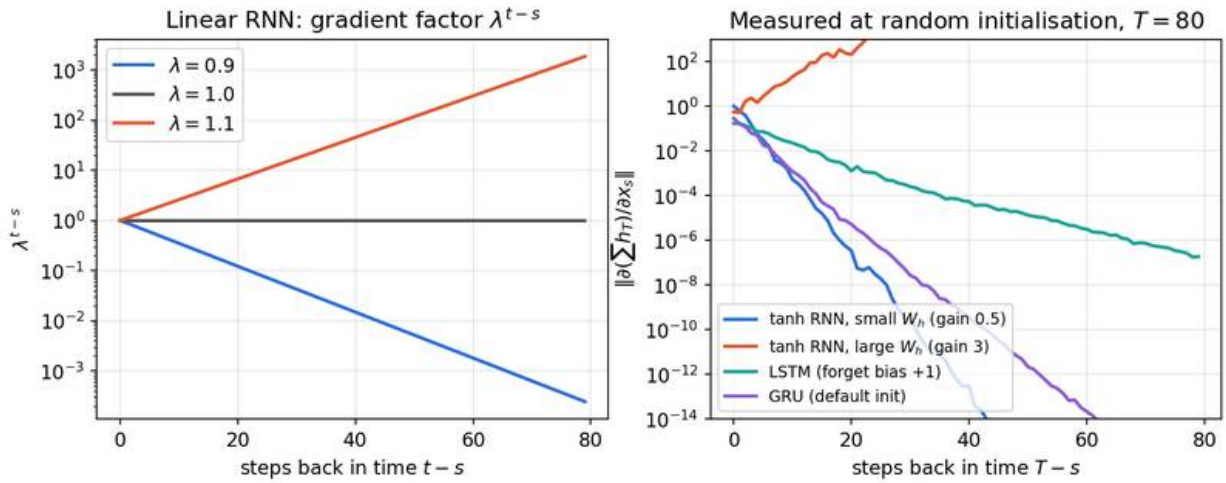


圖 2 — 左：純量因子 $\lambda^{(t-s)}$。右：在隨機初始化下量測最後狀態對每個輸入的梯度（$T = 80$）。單純的 *tanh RNN* 不是消失就是爆炸；*LSTM* 的加法記憶路徑衰減得慢得多。

對策。梯度爆炸用**梯度裁剪**（gradient clipping）[7] 處理：若梯度範數 $\|g\|$ 超過門檻 τ，就把它縮放成 $g \leftarrow \tau\, g/\|g\|$。這樣能保留更新方向、同時限制步長。PyTorch 的 `clip_grad_norm_` 做的正是這件事。梯度消失就比較難處理：需要換一種架構，讓狀態更新接近恆等映射，梯度才能穿過許多步。這種架構就是 LSTM。

長短期記憶（LSTM）

白話版。給網路一條獨立的記憶通道，也就是**細胞狀態**（cell state），它透過加法而不是反覆的矩陣乘法來更新，並讓學習得到的**閘門**（gate）決定要擦掉什麼、寫入什麼、顯露什麼。

精確版。以 $[x_t, h_{t-1}]$ 表示串接、σ 表示 logistic sigmoid，標準的 LSTM 一步計算為

$$
\begin{aligned}
f_t &= \sigma\big(W_f [x_t, h_{t-1}] + b_f\big) && \text{forget gate} \\
i_t &= \sigma(W_i [x_t, h_{t-1}] + b_i) && \text{input gate} \\
o_t &= \sigma(W_o [x_t, h_{t-1}] + b_o) && \text{output gate} \\
\tilde{c}_t &= \tanh(W_c [x_t, h_{t-1}] + b_c) && \text{candidate memory} \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t && \text{cell update} \\
h_t &= o_t \odot \tanh(c_t) && \text{exposed state}
\end{aligned}
$$

每個閘門都是一個介於 0 到 1 之間的數值向量，而 $\odot$ 是逐元素相乘，所以每個閘門就像一排調光開關，每個記憶單元各一個。遺忘閘（forget gate）f_t 決定保留多少舊記憶，輸入閘（input gate）i_t 決定寫入多少候選記憶 $\tilde{c}_t$，輸出閘（output gate）o_t 決定把多少記憶顯露成 h_t。

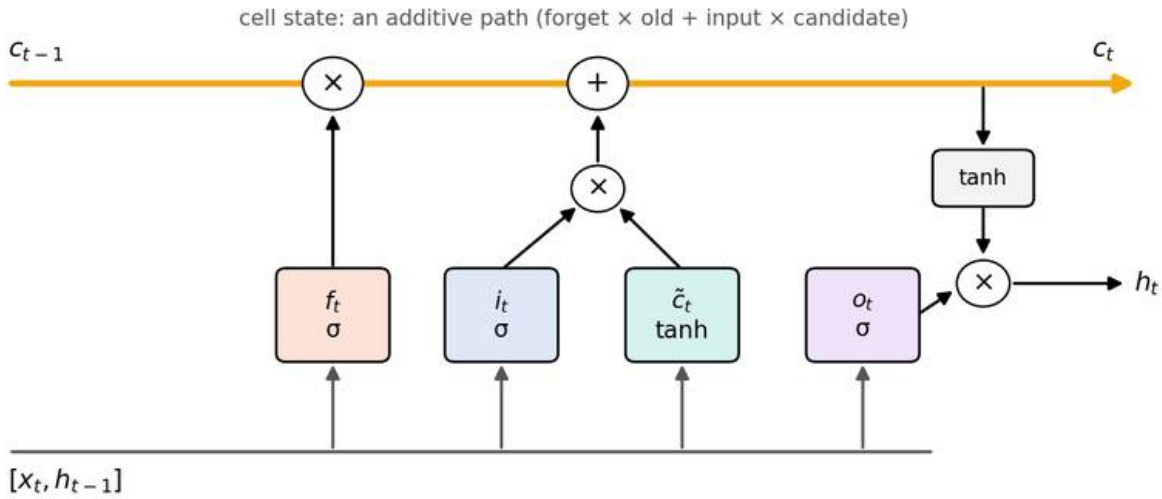


圖 *3* — *LSTM* 細胞的內部。細胞狀態（金色）只經過逐元素縮放與加法，這給了梯度一條穿越時間的受保護路徑。

為什麼有效。沿著細胞狀態路徑，$\partial c_t / \partial c_{t-1} = \mathrm{diag}(f_t)$（忽略經過閘門的較小間接路徑）。這條路徑上沒有反覆出現的權重矩陣，也沒有壓縮型的非線性函數。只要遺忘閘維持在接近 1，梯度就能幾乎不變地穿過許多步。這就是圖 2 中 LSTM 曲線衰減得這麼慢的原因，也是常見的初始化會把遺忘閘偏差設成正值（例如 +1）的原因：讓網路一開始就傾向於記住。但這並不保證無限長的記憶：遺忘閘能夠、也確實會學著關閉，而長距離的功勞分配（credit assignment）仍然會隨距離變得更困難。

歷史與出處。 Hochreiter 與 Schmidhuber 在 1997 年提出 LSTM [2]，當時用的是「恆定誤差旋轉木馬」(constant-error carousel)：細胞狀態原封不動地往下傳，**沒有遺忘閘**。Gers、Schmidhuber 與 Cummins 在 2000 年加入遺忘閘 [5]，讓細胞能學會重設記憶，例如在連續資料流中相互獨立的片段之間。上面這組含遺忘閘的方程式，才是實務上與所有主流框架使用的版本。Greff 等人 [19] 在語音、手寫與音樂任務上比較了八種 LSTM 變體，發現沒有一種顯著勝過標準架構，而遺忘閘與輸出激活函數是其中最關鍵的元件。

參數量。四個閘門區塊各有一個 $m \times (d + m)$ 的權重矩陣和一個偏差，所以一層 LSTM 大約有 $4m(d + m + 1)$ 個參數，是相同狀態大小的簡單 RNN 的四倍。（PyTorch 把輸入偏差和循環偏差分開存，所以是 $4m(d + m + 2)$。）

程式碼：從零寫出一步 LSTM，並與 PyTorch 比對

PyTorch 依照輸入閘、遺忘閘、候選細胞（g）、輸出閘的順序把四個閘門區塊堆疊在一起。親手實作一步，再和 `nn.LSTMCell` 比較，是確認自己沒讀錯方程式的最好方法：

```
import torch
from torch import nn

torch.manual_seed(0)
```

```
d, m = 4, 6
cell = nn.LSTMCell(d, m)                # PyTorch stacks gate weights in order i, f, g, o

def lstm_step(x, h, c):
    z = x @ cell.weight_ih.T + cell.bias_ih + h @ cell.weight_hh.T + cell.bias_hh
    i, f, g, o = z.chunk(4, dim=-1)
    i, f, o = torch.sigmoid(i), torch.sigmoid(f), torch.sigmoid(o)
    g = torch.tanh(g)                   # candidate memory, often written c~_t
    c = f * c + i * g                   # additive cell update
    h = o * torch.tanh(c)
    return h, c

x = torch.randn(2, d)
h0, c0 = torch.randn(2, m), torch.randn(2, m)
h_ref, c_ref = cell(x, (h0, c0))
h_ours, c_ours = lstm_step(x, h0, c0)
print("matches nn.LSTMCell:", torch.allclose(h_ours, h_ref, atol=1e-6),
      torch.allclose(c_ours, c_ref, atol=1e-6))
print("parameters:", sum(p.numel() for p in cell.parameters()), "= 4 * m * (d + m + 2) =",
      4 * m * (d + m + 2))
```

它會印出 `True True` 以及 `288 = ⋯ = 288`。

閘控循環單元（GRU）

白話版。 GRU 是精簡版的 LSTM：它把細胞狀態和隱藏狀態合而為一，並用兩個閘門取代三個。

精確版。依照 Cho 等人 [3] 以及 PyTorch 的 `nn.GRU`：

$$
\begin{aligned}
r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r) && \text{reset gate} \\
z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z) && \text{update gate} \\
\tilde{h}_t &= \tanh(W x_t + r_t \odot (U h_{t-1}) + b) && \text{candidate} \\
h_t &= z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t && \text{interpolation}
\end{aligned}
$$

重設閘（reset gate）r_t 決定在形成候選狀態時使用多少舊狀態；更新閘（update gate）z_t 在保留舊狀態（$z_t \approx 1$）與替換成新狀態（$z_t \approx 0$）之間內插。這個內插扮演了 LSTM 加法細胞路徑的角色。有兩點值得注意。有些論文與教科書會把 z_t 與 $1 - z_t$ 的角色對調；兩種模型等價，但閘門的意義正好相反。此外，原始論文把重設閘放在循環矩陣之前（$U(r_t \odot h_{t-1})$），而 PyTorch 與 cuDNN 則像上式一樣放在之後。在替閘門賦予意義之前，務必先查閱框架的文件。

GRU 有三個閘門區塊而不是四個，所以在相同寬度下參數比 LSTM 少約 25%。兩者都沒有在所有任務上穩定勝出；當資料或運算資源有限時，GRU 是合理的預設選擇。

程式碼：能推廣到新序列的循環分類器

這個完整的 CPU 範例學習判斷序列總和的正負號，必須把全部十二步的資訊累積起來才辦得到。一組全新的測試資料用來確認模型不是只把訓練序列背下來。

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
rnn = nn.GRU(input_size=1, hidden_size=24, batch_first=True)
head = nn.Linear(24, 2)
opt = torch.optim.Adam(list(rnn.parameters())+list(head.parameters()), lr=.003)
def batch(n):
    x = torch.randn(n, 12, 1)
    y = (x.sum(dim=(1, 2)) > 0).long()
    return x, y
for step in range(400):
    x, y = batch(64)
    _, h = rnn(x)
    loss = F.cross_entropy(head(h[-1]), y)
    opt.zero_grad(); loss.backward()
    nn.utils.clip_grad_norm_(list(rnn.parameters())+list(head.parameters()), 1.)
    opt.step()
rnn.eval(); head.eval()
with torch.no_grad():
    x, y = batch(512)
    _, h = rnn(x)
    acc = (head(h[-1]).argmax(1) == y).float().mean()
    print("fresh-sequence accuracy:", acc.item())
```

它在長度 12 的新序列上達到約 99% 的準確率。即使設定了 `batch_first=True`，回傳的隱藏狀態張量形狀仍是 `layers × batch × hidden`；這個旗標只影響輸入與輸出序列的排列方式。用更長的序列測試是在問另一個問題——長度推廣（length generalization）——模型可能在長度 12 成功、卻在長度 100 失敗（見練習 2）。

深層、雙向與串流 RNN

堆疊。深層 RNN 把某一層循環網路輸出的隱藏狀態序列，餵給另一層循環網路（PyTorch 中的 `num_layers`）。較低的層可以捕捉短距離的圖樣，較高的層則捕捉較長距離的圖樣。常見的是二到四層；更深的堆疊通常需要在層與層之間加入殘差連接。

雙向。 Schuster 與 Paliwal [10] 提出讓一個 RNN 沿時間往前跑、另一個往後跑，再把兩者的狀態串接起來：$h_t = [\vec{h}_t; \overleftarrow{h}_t]$。如此每個位置都同時總結了它的過去與未來。這非常適合整段輸入都已取得的**離線**任務（標註錄好的影片、編碼一個句子），但它**不具因果性**：不能用在必須在未來影格抵達之前就做出預測的即時任務。把離線雙向模型的準確率拿來報告線上應用，會高估部署後系統實際能掌握的資訊。

變動長度。一個批次裡的序列長度不一，會被填補（padding）到相同長度。若不小心處理，填補值會滲入狀態與損失。請使用每條序列的實際長度（PyTorch 的 `pack_padded_sequence`）或遮罩，讓填補的步驟既不更新狀態、也不計入損失。

串流。對即時資料流，要在多次呼叫之間保留狀態，只在彼此獨立的資料流之間重設。不小心把某位使用者、某份文件或某段影片的狀態帶到下一個，會洩漏資訊並造成無法重現的結果。

編碼器–解碼器模型與教師強制

白話版。要把一個序列轉換成另一個長度不同的序列（把句子翻譯成另一種語言、把影片轉成描述），就用一個 RNN 把輸入讀成摘要，再用第二個 RNN 根據這份摘要，一次一個符記地寫出輸出。

精確版。在 Sutskever、Vinyals 與 Le [11] 的**序列到序列**（sequence-to-sequence, seq2seq）模型，以及 Cho 等人 [3] 密切相關的編碼器–解碼器中，**編碼器** RNN 讀入 $x_1, \dots, x_T$，其最終狀態成為上下文向量 $c = h_T$。以 c 初始化的**解碼器** RNN 建模

$$p(y_1, \dots, y_U \mid x) = \prod_{u=1}^{U} p(y_u \mid y_{<u}, c), \qquad s_u = \mathrm{RNN}(s_{u-1}, y_{u-1}),$$

其中 s_u 是解碼器狀態，$p(y_u \mid \cdot)$ 是在詞彙表上的 softmax。當解碼器輸出序列結束符記時生成就停止，所以輸入與輸出的長度互不相關。

教師強制（teacher forcing）。訓練時，解碼器在第 u 步的輸入是**真實答案**中的前一個符記 y_{u-1}，而不是模型自己的預測。這讓所有步驟都能針對已知目標平行訓練，大幅穩定學習。然而在測試時，模型必須吃進自己的預測，而這些預測可能含有訓練時從未見過的錯誤。這種訓練與測試之間的落差稱為**曝光偏差**（exposure bias）。排程取樣（scheduled sampling）[25] 在訓練過程中逐步用模型的取樣取代真實輸入，以減輕這個問題。解碼時通常使用貪婪搜尋或集束搜尋（beam search），而不是隨機取樣。

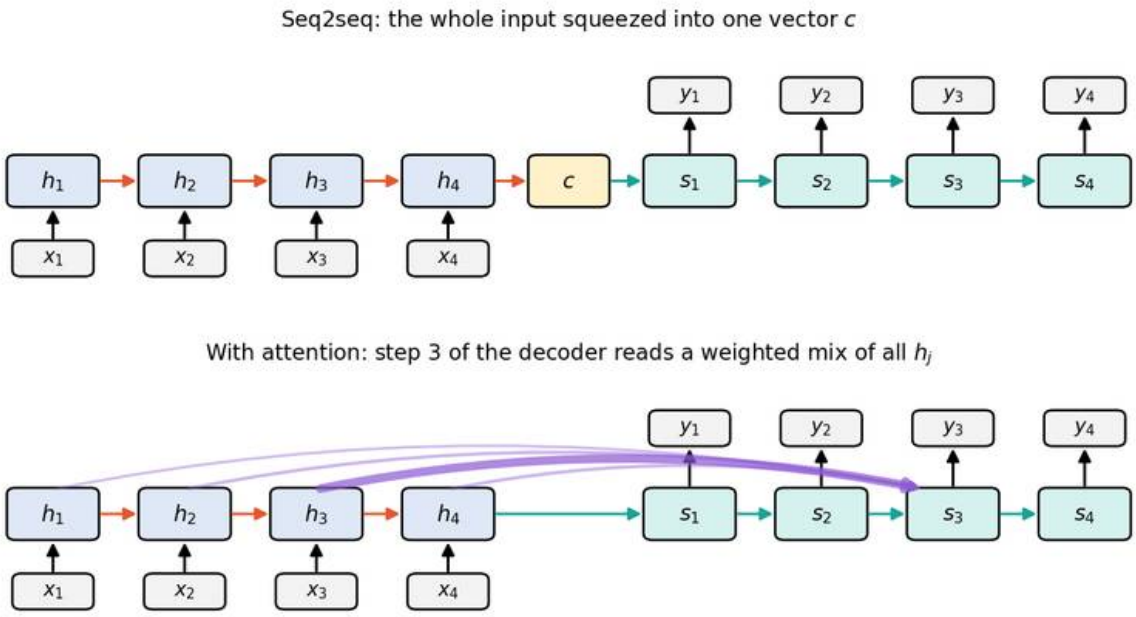


圖 4 — 上：單純的 *seq2seq* 把整段輸入擠進一個向量 c。下：加上注意力後，解碼器的每一步都讀

取所有編碼器狀態經學習得到的加權組合。

圖 4（上）清楚顯示了弱點：解碼器對輸入所知道的一切，都必須通過一個固定大小的向量 c，不論輸入有多長。單純 seq2seq 模型的翻譯品質在長句子上會下降。Sutskever 等人發現，單純把**來源句子反轉**就有幫助，因為這縮短了對應字詞之間的距離——這暗示問題出在瓶頸與過長的路徑 [11]。

注意力：通往 Transformer 的橋梁

白話版。與其強迫解碼器只依賴一個摘要向量，不如讓它在每一步都回頭看**所有**編碼器狀態，並自行決定此刻哪些最相關。

精確版。 Bahdanau、Cho 與 Bengio [12] 保留了每一個編碼器狀態 $h_1, \dots, h_T$（來自雙向編碼器），並在解碼器的每一步 u 計算

$$e_{u,j} = v^\top \tanh\!\left(W s_{u-1} + U h_j\right), \qquad \alpha_{u,j} = \frac{\exp(e_{u,j})}{\sum_{k=1}^{T} \exp(e_{u,k})}, \qquad c_u = \sum_{j=1}^{T} \alpha_{u,j}\, h_j .$$

$e_{u,j}$ 是解碼器當下需求（s_{u-1}）與編碼器位置 j 之間經學習得到的**對齊分數**（alignment score）；W、U 與 v 是可學習參數。softmax 把分數轉成皆為正、總和為一的權重 $\alpha_{u,j}$，而**上下文向量** c_u 就是用這些權重對編碼器狀態做的加權平均。解碼器接著把 c_u 與自己的狀態一起使用。這種形式稱為**加法注意力**（additive attention）。因為 c_u 在每一步都會改變，而且能直接存取每一個輸入位置，任何輸入到任何輸出之間的路徑長度都變成一步，與序列長度無關。

```
import torch
from torch import nn

class AdditiveAttention(nn.Module):
    """Bahdanau-style scores e_j = v^T tanh(W s + U h_j), then softmax over j."""
    def __init__(self, d_dec, d_enc, d_att):
        super().__init__()
        self.W = nn.Linear(d_dec, d_att, bias=False)
        self.U = nn.Linear(d_enc, d_att, bias=False)
        self.v = nn.Linear(d_att, 1, bias=False)

    def forward(self, s, H, mask=None):
        # s: B x d_dec (decoder state), H: B x T x d_enc (all encoder states)
        e = self.v(torch.tanh(self.W(s)[:, None] + self.U(H))).squeeze(-1)   # B x T
        if mask is not None:                        # True where the input is padding
            e = e.masked_fill(mask, float("-inf"))
        alpha = e.softmax(dim=-1)                   # attention weights, rows sum to 1
        context = (alpha[:, :, None] * H).sum(1)    # B x d_enc weighted average
        return context, alpha

torch.manual_seed(0)
att = AdditiveAttention(d_dec=16, d_enc=32, d_att=8)
H = torch.randn(2, 5, 32)
s = torch.randn(2, 16)
pad = torch.tensor([[False] * 5, [False, False, False, True, True]])   # 2nd sequence has
 ↪  length 3
```

```
ctx, alpha = att(s, H, pad)
print("context:", tuple(ctx.shape), "| weights sum to 1:",
      torch.allclose(alpha.sum(-1), torch.ones(2)))
print("weights on padding:", alpha[1, 3:].tolist())
```

它會印出形狀為 `(2, 32)` 的上下文，確認權重總和為一，並顯示填補位置的權重恰好為零：在 softmax 之前用 $-\infty$ 遮罩，是讓注意力忽略填補（以及在ML 04 中忽略未來）的標準做法。

注意力權重也可以解讀成一種軟性的**對齊**（alignment）。圖 5 顯示一個帶有這種注意力的小型 GRU 編碼器–解碼器，在 CPU 上訓練不到一分鐘，學習把隨機數字序列反轉（在教師強制下，對新序列的逐符記準確率約 96%）。沒有人告訴它，它卻學會了在寫第 u 個輸出時去看第 $T+1-u$ 個輸入位置：也就是反對角線。

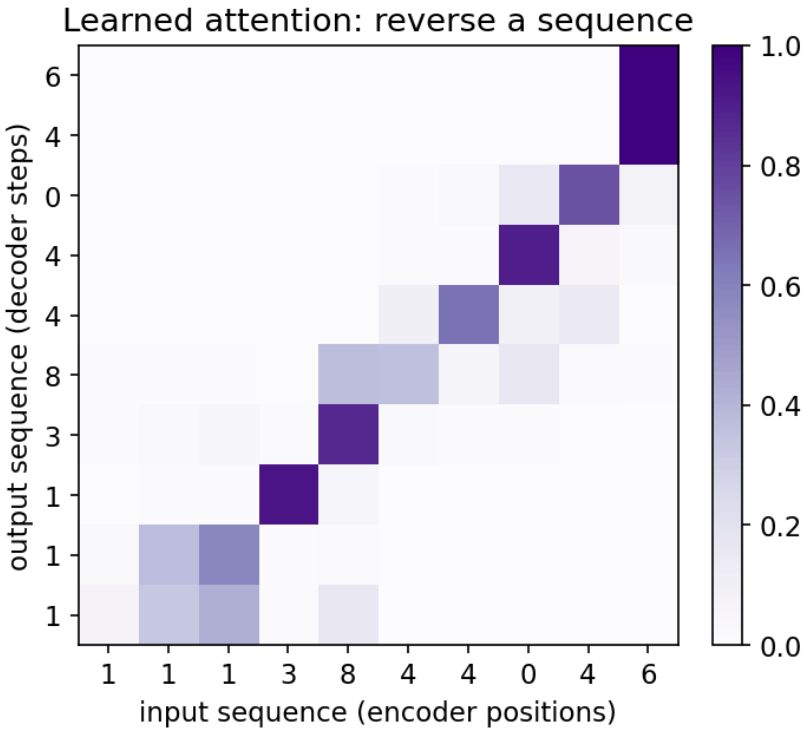


圖 5 — 訓練來反轉序列的玩具編碼器–解碼器的注意力權重（由圖片腳本產生）。每個輸出步驟都集中在鏡射位置的輸入上。

從這裡走向 Transformer。注意力解決了瓶頸，但編碼器與解碼器仍然是循環式的，所以處理長度 T 的序列需要 T 個無法在訓練時平行化的循序步驟。Vaswani 等人 [28] 問：如果完全拿掉循環，**在編碼器與解碼器內部也使用注意力**（自注意力，self-attention），再用位置編碼恢復順序，會怎麼樣？答案就是 Transformer：它能在所有位置上平行訓練，成為語言領域的主流架構，並且隨著 ViT 成為視覺領域的主流架構。ML 04 會從縮放點積注意力（scaled dot-product attention）接續這個故事，它以 $q^\top k/\sqrt{d_k}$ 取代加法分數。

循環網路在電腦視覺中的應用

影像描述（image captioning）。Show and Tell [13]（Vinyals 等人）是一個直接的影像到序列模型：一個 CNN（ML 03）把影像編碼成一個向量，用它初始化一個 LSTM 語言模型，再由這個語言模型逐字生成圖說，並以教師強制在影像–圖說配對上訓練。Show, Attend and Tell [14]（Xu 等人）保留 CNN 的空間特徵網格而不是單一向量，讓 LSTM 在生成每個字時對影像區域做注意力，所產生的注意力圖常常會凸顯正在被說出名稱的物體。這是最早廣為人知的視覺注意力展示之一。

影片理解。長期循環卷積網路（Long-term Recurrent Convolutional Networks, LRCN）[15]（Donahue 等人）對每張影格套用 CNN、跨影格套用 LSTM，並端到端訓練，用於動作辨識、影像描述與影片描述。CNN 處理空間，RNN 處理時間：這是處理任何逐影格特徵序列的常見模式，例如姿態軌跡，或物件追蹤中的逐影格偵測結果。

時空預測。當狀態本身就應該是一張影像時，全連接的閘門會浪費空間結構。Shi 等人的 **ConvLSTM** [16] 把 LSTM 閘門中的矩陣乘法換成卷積，$i_t = \sigma(W_{xi} * X_t + W_{hi} * H_{t-1} + \dots)$，所以輸入、狀態與閘門都是三維張量（通道 × 高 × 寬）。它最初是為了從雷達圖做降雨即時預報而提出，如今廣泛用於影片預測與其他網格上的預測任務。

發生了什麼改變。在大規模的影像描述與影片分類上，Transformer 與視覺–語言模型（見ML 09）已大致取代 LSTM，因為它們能平行訓練，而且隨著資料增加擴展得更好。循環與卷積–循環模型仍常見於影格逐張抵達、記憶體必須維持固定的場合：線上影片處理、追蹤、機器人與低功耗裝置。

遞迴網路：沿著樹組合

白話版。遞迴網路沿著一棵樹，套用同一條共享的組合規則，把兩個子節點的表示合併成父節點，直到抵達根節點。

精確版。對一棵葉節點帶有向量（詞嵌入、區域特徵）的二元樹，每個內部節點計算

$$h_{\text{parent}} = \tanh\!\Big(W\,[h_{\text{left}}; h_{\text{right}}] + b\Big),$$

每個節點都使用**同一個** $W \in \mathbb{R}^{D \times 2D}$ 與 b。根節點向量代表整個結構，可以拿來分類；每個節點也可以有自己的標籤。訓練使用**結構反向傳播**（backpropagation through structure）：和 BPTT 是同一個想法，只是沿著樹的邊進行。長度為 n 的鏈是完全不平衡的樹這個特例；平衡樹把任一葉節點到根節點的路徑從 $n-1$ 步縮短到大約 $\log_2 n$ 步（圖 6），縮短了梯度要走的路。

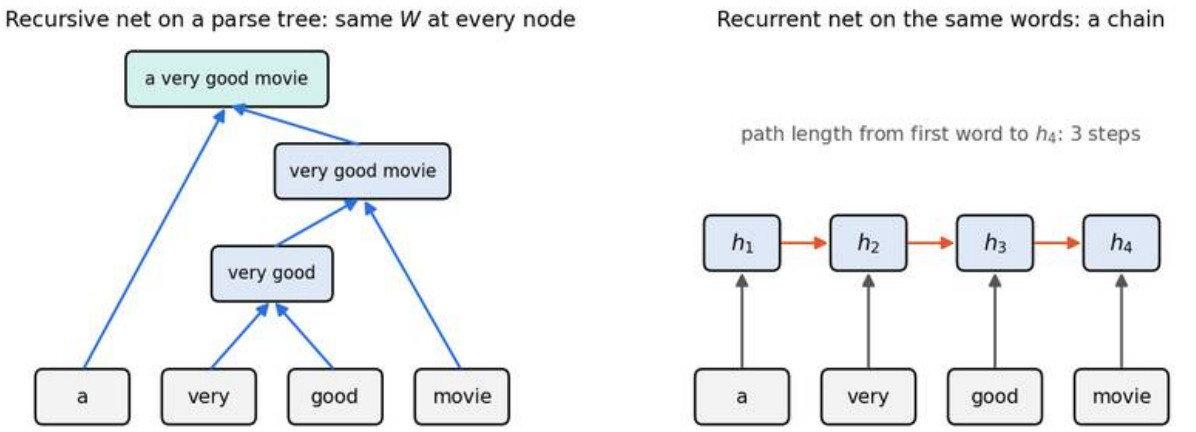


圖 *6* — 遞迴網路沿著樹前進（左）；循環網路沿著鏈前進（右）。兩者都共享權重，但樹反映了輸入本身的結構。

Socher 等人 [18] 最早是在視覺情境中使用遞迴網路：他們把影像切成超像素，接著同時學習把相鄰區域合併成場景剖析樹、以及計算區域表示，並用同一套方法剖析句子。他們後來的遞迴神經張量網路（Recursive Neural Tensor Network）與 Stanford Sentiment Treebank [17] 顯示，沿著剖析樹組合能捕捉否定（“not very good”）之類的現象。Tai、Socher 與 Manning 的 **Tree-LSTM** [4] 為樹狀組合加入 LSTM 式的閘門，每個子節點各有一個遺忘閘，讓父節點能有選擇地保留來自每棵子樹的資訊。

```
import torch
from torch import nn

torch.manual_seed(0)
D = 8
vocab = {"a": 0, "very": 1, "good": 2, "movie": 3}
emb = nn.Embedding(len(vocab), D)
compose = nn.Linear(2 * D, D)            # ONE weight matrix shared by every internal node
classify = nn.Linear(D, 2)               # e.g. negative / positive sentiment

def encode(tree):
    """tree is a word (leaf) or a pair (left, right)."""
    if isinstance(tree, str):
        return emb(torch.tensor(vocab[tree]))
    left, right = tree
    return torch.tanh(compose(torch.cat([encode(left), encode(right)])))

tree = ("a", (("very", "good"), "movie"))          # (a ((very good) movie))
root = encode(tree)
loss = nn.functional.cross_entropy(classify(root)[None], torch.tensor([1]))
loss.backward()                                     # backpropagation through structure
print("root vector:", tuple(root.shape), "| loss:", round(loss.item(), 4))
print("gradient reached the shared W:", compose.weight.grad.abs().sum().item() > 0)
swapped = encode((("movie", ("good", "very")), "a"))
print("swapping children changes the root:", not torch.allclose(root, swapped))
```

遞迴就是一般的 Python 遞迴呼叫，而 autograd 會即時建立樹狀的計算圖。交換子節點會改變結果，因為串接讓左右兩個位置扮演不同的角色。

實務上的主要限制在於樹本身：得有人提供它（剖析器、分割階層），而且要有效率地把形狀各異的樹組成批次並不容易。同時學習結構與表示是可行的，但更困難。在目前的實務中，自注意力能在沒有明確樹結構的情況下，學到柔性、隨輸入而變的分組，這也是遞迴網路如今少見的原因之一；不過階層式組合的想法仍延續在階層式視覺骨幹網路與場景圖（scene graph）中。

Mamba：選擇性狀態空間與掃描

有用的記憶應該能在干擾之間保留重要觀察，並在情境改變時覆寫舊資訊。Mamba [21] 讓記憶更新取決於目前輸入，同時維持更新對前一個狀態的仿射形式。這個差別讓模型保有循環的解讀方式，又能用平行前綴掃描（parallel prefix scan）計算整段序列。

從連續動態到選擇性更新

先從小型的連續時間記憶開始。穩定的轉移使儲存的資訊逐漸衰減；輸入投影寫入新資訊；輸出投影讀出記憶：

$$\frac{dh(\tau)}{d\tau} = Ah(\tau) + Bu(\tau), \qquad y(\tau) = Ch(\tau).$$

其中 τ 是連續時間，$u(\tau) \in \mathbb{R}$ 是單一輸入通道，$h(\tau) \in \mathbb{R}^N$ 是該通道的 N 維狀態，$A \in \mathbb{R}^{N\times N}$ 是轉移矩陣，$B \in \mathbb{R}^{N\times 1}$ 負責寫入輸入，$C \in \mathbb{R}^{1\times N}$ 負責讀取狀態，$y(\tau) \in \mathbb{R}$ 是輸出。完整的神經網路區塊會以可學習投影混合多個通道，並加入局部卷積與閘控 [21]。這個方程式只抽出記憶機制。

在每個區間內保持輸入與寫入矩陣不變。精確的零階保持（zero-order hold）離散化得到

$$\bar{A}_t = \exp(\Delta_t A), \qquad \bar{B}_t = \left(\int_0^{\Delta_t} \exp(sA)\, ds \right) B_t,$$
$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t u_t, \qquad y_t = C_t h_t.$$

其中 t 是離散 token 的索引，$\Delta_t > 0$ 是區間長度，s 是積分變數，u_t 是固定的輸入，B_t, C_t 是目前的寫入與讀取矩陣，上橫線表示離散係數。即使 A 是奇異矩陣，積分形式仍然成立；使用反矩陣的公式則需要額外假設。若 A 是對角矩陣，這些運算可以逐元素執行。

選擇性表示從目前 token 預測 Δ_t, B_t, C_t。例如，可學習的仿射投影接上 $\mathrm{softplus}(z) = \log(1 + e^z)$，就能產生正的區間長度。底層可學習的 A 可以保持固定，但 $\bar{A}_t$ 仍會透過 Δ_t 改變。當對角元素為負時，較長的區間會降低舊狀態的保留比例。選擇性的寫入與讀取也決定哪些內容進入或離開記憶。因此，即使每次狀態更新都是仿射的，整體映射對輸入仍是非線性的。圖 7 使用合成係數單獨呈現保留效果，並未量測訓練好的模型。

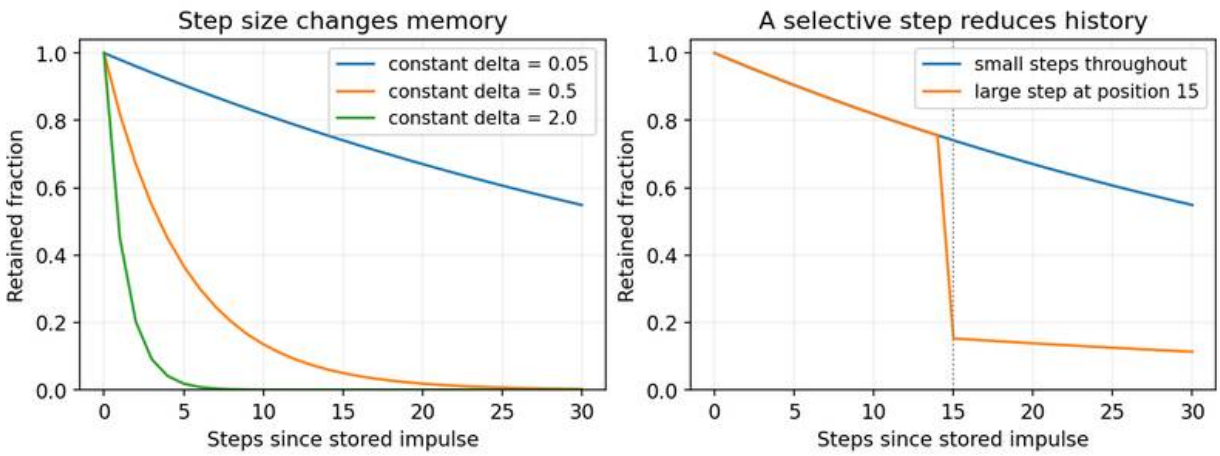


圖 7 — 純量轉移 $A = -0.4$ 時，歷史保留比例是 $exp(-0.4\ delta)$ 的連乘。選定的大區間會明顯降低舊記憶，小區間則能保留得更久。

為什麼這種循環可以掃描

輸入相依的係數算好後，組合兩次仿射更新仍然只需要一次仿射更新：

$$(a_2, b_2) \circ (a_1, b_1) = (a_2 a_1,\ a_2 b_1 + b_2).$$

其中配對 (a_i, b_i) 表示映射 $h \mapsto a_i h + b_i$，$i = 1, 2$ 標示連續的兩次更新，$\circ$ 表示先套用第一次，再套用第二次。組合具有結合律；對角轉移的乘法是逐元素運算。因此，前綴掃描可以把運算重新分組成樹狀結構，卻保留時間順序造成的效果。一般 LSTM ／ GRU 的閘門取決於前一個隱藏狀態，通常無法在狀態尚未算出之前先準備這些係數。

以下可獨立執行的 CPU 範例使用精確的對角離散化，以及手動指定的選擇性係數，檢查因果性與分塊之間的狀態連續性。這是教學用的循環，並非預訓練 Mamba 實作或速度基準。

```
import numpy as np

rng = np.random.default_rng(44)
u = rng.normal(size=24)
A = -np.array([0.1, 0.8])  # two stable diagonal entries

def selective_scan(values, state=None):
    h = np.zeros(2) if state is None else state.copy()
    outputs = []
    for x in values:
        delta = np.logaddexp(0.0, 0.5 * x)
        B = np.array([1.0, np.tanh(x)])
        C = np.array([1.0, 0.5 + 0.1 * np.tanh(x)])
        a = np.exp(delta * A)
        b = np.expm1(delta * A) / A * B * x
        h = a * h + b
        outputs.append(C @ h)
    return np.array(outputs), h
```

```
y, final_state = selective_scan(u)
changed = u.copy()
changed[12:] += 10.0
changed_y, _ = selective_scan(changed)
assert np.allclose(y[:12], changed_y[:12])
left, boundary = selective_scan(u[:12])
right, resumed_state = selective_scan(u[12:], boundary)
assert np.allclose(y, np.r_[left, right])
assert np.allclose(final_state, resumed_state)
reset_right, _ = selective_scan(u[12:])
assert not np.allclose(right, reset_right)
print('causal prefix and carried-state chunk checks passed')
```

輸出確認兩項檢查都通過。在批次邊界把狀態歸零會改變計算；反過來，在獨立範例之間共用狀態則會洩漏資訊。串流實作必須追蹤每個狀態屬於哪段序列，並在真正的序列邊界重設。

Mamba-2 與注意力的連結

Mamba-2 [29] 同時改變內部序列混合器與外圍區塊。其結構化狀態空間對偶性（structured state-space duality）在每個頭內使用「純量乘上單位矩陣」的轉移，並把計算切成適合加速器硬體執行密集矩陣乘法的區塊。它也重新安排參數投影與正規化。這比在原始 Mamba 區塊中加入注意力更為具體。

初始狀態為零時，純量轉移的循環可以展開為

$$y_t = \sum_{j=1}^{t} \left[C_t \left(\prod_{k=j+1}^{t} a_k \right) B_j \right] u_j.$$

其中 j 索引先前輸入，k 索引中間的轉移，a_k 是乘在單位矩陣上的離散純量轉移，B_j 是離散寫入列向量，C_t 是讀取行向量，空的連乘等於一。括號內是具有結構的因果混合係數。這提供類似注意力的矩陣觀點，卻不需要一般 softmax 注意力，也不必儲存完整矩陣。Hu 等人 [30]（ICML 2026）把對偶性延伸到對角轉移，並釐清為何無法直接延伸到標準 softmax 注意力。

在固定寬度與狀態維度下，循環推論只攜帶固定大小的狀態；一般具有快取的因果注意力則保留 key 與 value，儲存量隨上下文長度增加。壓縮能省下記憶體，但精確回想許多互不相關細節的能力也受到考驗。序列成本呈線性並不代表訓練記憶體固定：活化值、梯度與重新計算策略都會影響結果。應一起評估檢索品質、上下文長度與裝置實測延遲。影像特有的排序問題接著在ML 04討論。

現代觀點

綜述文獻。 Goodfellow 等人書中的第 10 章 [1] 至今仍是循環與遞迴網路、回聲狀態（echo-state）想法與長期依賴問題最清楚的統一論述。Lipton、Berkowitz 與 Elkan [20] 在 2015 年寫了一篇 RNN 序列學習的批判性回顧：以時間順序、自成一體地交代三十年來通往 LSTM 與雙向 RNN 的研究，統一了互相衝突的記號與名稱，並涵蓋影像描述、翻譯與手寫辨識等應用。Greff 等人

[19] 與其說是綜述，不如說是大型實證研究，但它是「LSTM 哪些元件重要」這個問題的參考依據。Tiezzi 等人近年的兩篇回顧，則為 Transformer 時代重新定位循環結構：一篇詳盡的綜述 [26] 把 Transformer–RNN 混合模型、具循環形式的注意力變體，以及深度狀態空間模型（一類特殊的線性 RNN）統一放在「循環」這個大傘之下，並強調從可能無限長的資料流中線上學習、而非處理固定長度序列所帶來的研究機會；另一篇刊登於 *Nature Machine Intelligence* 的較短觀點文章 [27]，則重新檢視位於 Transformer 與狀態空間模型交會處的循環處理。

循環的回歸（2023–2026）。 Transformer 處理所有位置兩兩之間的關係，記憶體與運算量隨長度呈平方成長。一系列模型重新採用循環結構，以固定大小的狀態取得線性時間的推論：

- **線性循環單元。** Orvieto 等人 [24]（ICML 2023）證明，一個經過精心參數化的**線性**循環式——對角的複數狀態矩陣、穩定的初始化與正規化——在長距離基準測試上可媲美深度狀態空間模型，而且非線性可以放在循環式之外。
- **選擇性狀態空間。** Mamba [21] 讓狀態空間的參數成為輸入的函數，所以模型能像 LSTM 的閘門那樣選擇要儲存或忽略什麼，同時以考慮硬體特性的平行掃描（parallel scan）維持訓練效率。視覺上的變體，以及它們對影像是否真有必要的爭論，會在ML 04 討論。
- **復興的 LSTM 與 GRU。** xLSTM [22]（NeurIPS 2024）為 LSTM 加入指數型閘門與矩陣形式的記憶，並把它擴展到語言模型的規模。Feng 等人 [23]（2024）移除 LSTM 與 GRU 閘門對隱藏狀態的依賴（“minLSTM”、“minGRU”），使訓練時能使用平行掃描，並報告了與近期序列模型相當的結果。

共同的教訓是：為 LSTM 發明的**閘控、加法式狀態更新**是對的；限制經典 RNN 的主要是循序訓練。讓循環式對狀態呈線性（因此能用平行掃描計算），就能恢復平行訓練，同時保留便宜的串流推論。

開放問題。超出訓練長度範圍的長度推廣；在非常長的資料流上、不用完整 BPTT 的功勞分配；固定大小的狀態相較於注意力不斷成長的快取究竟能記住多少——對需要精確回想遠處細節的任務而言，這是實際存在的限制；以及為二維影像與三維影片選擇掃描順序，因為它們沒有唯一自然的序列順序。

重點整理

- RNN 在每一步套用同一個共享更新 $h_t = f(x_t, h_{t-1})$，所以能用固定數量的參數處理任意長度的序列。展開後，它就是一個權重綁定的深層網路。
- BPTT 就是在展開後的計算圖上做普通的反向傳播；共享權重的梯度對時間加總。截斷式 BPTT 會把狀態往前帶，但限制梯度往回傳的距離。
- 長距離梯度是 Jacobian 的連乘，傾向於消失或爆炸。裁剪能處理爆炸；具有近似恆等、加法式狀態路徑的架構能處理消失。
- LSTM 的閘門（遺忘、輸入、輸出）與 GRU 的閘門（重設、更新）控制一條受保護的記憶路徑。遺忘閘是 2000 年才加入 1997 年 LSTM 的。務必確認框架的閘門慣例。
- 雙向編碼器使用未來的上下文，不具因果性；串流模型必須小心管理狀態。
- Seq2seq 以教師強制把序列對應到序列；注意力移除了它單一向量的瓶頸，而不含循環的自注意力就成了 Transformer。

- 遞迴網路以一條共享規則沿著樹組合。在電腦視覺中，循環結構推動了影像描述、影片模型與 ConvLSTM 預測，而閘控線性循環以狀態空間模型的形式重新回歸。
- Mamba 從輸入預測寫入、讀取與區間係數，同時讓更新對狀態維持仿射形式；Mamba-2 透過結構化對偶性高效率地分塊計算。壓縮省下快取記憶體，也限制精確回想能力。

練習

1. **因果性**。你的團隊報告一個使用雙向 LSTM 的動作辨識模型，逐影格準確率達 92%，並提議把它部署成即時警報系統。這個計畫有什麼問題？你要如何公平地評估可部署的替代方案？執行選擇性掃描範例，再加上反向掃描：哪個因果性檢查應該失敗？

   提示

   第 t 張影格上的反向 LSTM 已經讀過 t 之後的影格，而即時系統還沒看到這些影格。請重新訓練一個單向（因果）模型，或允許固定的前瞻延遲，並以完全相同的延遲做評估。把準確率表示成延遲的函數來報告。反向掃描讀過被改動的未來，因此即使正向前綴不變，合併後的前綴通常仍會改變。

2. **長度推廣**。用長度 12 訓練 GRU 總和分類器，接著在不重新訓練的情況下，用長度 6、24、100 與 400 的新序列評估。畫出準確率對長度的圖。再用 LSTM、以及用長度 6–24 混合訓練，重複一次。

   提示

   每種長度都要產生新的資料。總和的尺度隨 $\sqrt{T}$ 成長，所以即使邏輯正確，狀態在長輸入上也可能飽和。混合長度訓練通常比更換細胞種類更有幫助。在長度 12 上成功，本身並不能說明長度 400 的表現。

3. **梯度上界**。對純量 RNN $h_t = \tanh(w h_{t-1} + x_t)$，證明 $|\partial h_t / \partial h_s| \le |w|^{t-s}$。梯度在 w 取哪些值時可能爆炸？為什麼 tanh 的飽和會抑制爆炸？

   提示

   每個因子都是 $w(1 - h_k^2)$，且 $0 \le 1 - h_k^2 \le 1$。爆炸需要 $|w| > 1$，但很大的 $|w|$ 也會把 h_k 推向 ± 1，使 $1 - h_k^2$ 變小。梯度只會在停留於非飽和區域的軌跡上爆炸，這就是實務上爆炸往往是突發性的原因。

4. **LSTM 的記憶**。在 LSTM 方程式中，假設從 s 到 t 的每一步都有 $f_t = 1$ 且 $i_t = 0$。c_t 用 c_s 表示是多少？沿著細胞路徑的 $\partial c_t / \partial c_s$ 是多少？如果像 1997 年原始設計那樣，把遺忘閘永遠固定為 1，在一個永不結束的輸入流上會發生什麼事？

   提示

   $c_t = c_s$ 且 $\partial c_t / \partial c_s = I$：完美的記憶與完美的梯度流動。若 $f_t \equiv 1$ 而寫入不為零，細胞狀態會在無止盡的資料流上無限累積，這正是 Gers 等人讓網路學會遺忘所解決的問題。

5. **注意力的形狀與遮罩**。在加法注意力的程式碼中，一個批次的編碼器狀態形狀為 $8 \times 40 \times 256$（其中有些序列經過填補），解碼器狀態形狀為 8×512。請給出 e、α 與上下文的形狀，並說明若用 0 而不是 $-\infty$ 遮罩填補位置，會出什麼問題。

   提示

   e 與 α 是 8×40；上下文是 8×256。分數 0 只是一個普通的分數：$e^0 = 1$ 仍會給填補位置正的權重，上下文就會混入沒有意義的填補狀態。

6. **遞迴與循環**。對一棵有 $n = 1024$ 個葉節點的平衡二元樹，葉節點與根節點之間有幾個組合步驟？與一個由左到右讀取同樣 1024 個輸入的 RNN 比較，並把答案與梯度消失的上界連結起來。

   提示

   $\log_2 1024 = 10$ 步，鏈則最多 1023 步。梯度上界的衰減方式是（因子）$^{\text{path length}}$，所以從每個輸入到輸出的路徑越短，越有利於最佳化。注意力把這一點推到極致：路徑長度為一。

參考文獻

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 10. Book
2. S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997. DOI
3. K. Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014. arXiv
4. K. S. Tai, R. Socher and C. D. Manning, “Improved Semantic Representations From Tree-Structured Long Short-Term Memory Networks,” ACL, 2015. arXiv
5. F. A. Gers, J. Schmidhuber and F. Cummins, “Learning to Forget: Continual Prediction with LSTM,” *Neural Computation*, 2000. DOI
6. Y. Bengio, P. Simard and P. Frasconi, “Learning long-term dependencies with gradient descent is difficult,” *IEEE Transactions on Neural Networks*, 1994. DOI
7. R. Pascanu, T. Mikolov and Y. Bengio, “On the difficulty of training Recurrent Neural Networks,” ICML, 2013. arXiv
8. P. J. Werbos, “Backpropagation through time: what it does and how to do it,” *Proceedings of the IEEE*, 1990. DOI
9. J. L. Elman, “Finding Structure in Time,” *Cognitive Science*, 1990. DOI
10. M. Schuster and K. K. Paliwal, “Bidirectional recurrent neural networks,” *IEEE Transactions on Signal Processing*, 1997. DOI
11. I. Sutskever, O. Vinyals and Q. V. Le, “Sequence to Sequence Learning with Neural Networks,” NeurIPS, 2014. arXiv
12. D. Bahdanau, K. Cho and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” ICLR, 2015. arXiv
13. O. Vinyals, A. Toshev, S. Bengio and D. Erhan, “Show and Tell: A Neural Image Caption Generator,” CVPR, 2015. arXiv

14. K. Xu et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” ICML, 2015. arXiv
15. J. Donahue et al., “Long-term Recurrent Convolutional Networks for Visual Recognition and Description,” CVPR, 2015. arXiv
16. X. Shi et al., “Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting,” NeurIPS, 2015. arXiv
17. R. Socher et al., “Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank,” EMNLP, 2013. Paper
18. R. Socher, C. C.-Y. Lin, A. Y. Ng and C. D. Manning, “Parsing Natural Scenes and Natural Language with Recursive Neural Networks,” ICML, 2011. Paper
19. K. Greff, R. K. Srivastava, J. Koutník, B. R. Steunebrink and J. Schmidhuber, “LSTM: A Search Space Odyssey,” *IEEE Transactions on Neural Networks and Learning Systems*, 2017. arXiv
20. Z. C. Lipton, J. Berkowitz and C. Elkan, “A Critical Review of Recurrent Neural Networks for Sequence Learning,” arXiv:1506.00019, 2015. arXiv
21. A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” COLM, 2024. arXiv
22. M. Beck et al., “xLSTM: Extended Long Short-Term Memory,” NeurIPS, 2024. Paper
23. L. Feng et al., “Were RNNs All We Needed?,” arXiv:2410.01201, 2024. arXiv
24. A. Orvieto et al., “Resurrecting Recurrent Neural Networks for Long Sequences,” ICML, 2023. arXiv
25. S. Bengio, O. Vinyals, N. Jaitly and N. Shazeer, “Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks,” NeurIPS, 2015. arXiv
26. M. Tiezzi, M. Casoni, A. Betti, T. Guidi, M. Gori and S. Melacci, “On the Resurgence of Recurrent Models for Long Sequences: Survey and Research Opportunities in the Transformer Era,” arXiv:2402.08132, 2024. arXiv
27. M. Tiezzi, M. Casoni, A. Betti, T. Guidi, M. Gori and S. Melacci, “Back to recurrent processing at the crossroad of transformers and state-space models,” *Nature Machine Intelligence*, 2025. DOI
28. A. Vaswani et al., “Attention Is All You Need,” NeurIPS, 2017. arXiv
29. T. Dao and A. Gu, “Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality,” ICML, PMLR 235, 2024. Paper
30. J. Y.-C. Hu, X. Zhang, A. Elsheikh, W. Wu and H. Liu, “On Structured State-Space Duality,” ICML, PMLR 306, 2026. Paper

ML 04・注意力與視覺 Transformer

從 query、key、value 建立自注意力；加入多頭、位置編碼與 Transformer 區塊；用 ViT 把影像切成 patch token；再認識 DeiT、Swin、CNN 與 Transformer 混合模型、高效率注意力與狀態空間模型。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-04-vision-transformers/

你將學到

- 自注意力（self-attention）如何算出一個隨輸入而變的加權平均；query、key、value 各自扮演什麼角色；為什麼分數要除以 $\sqrt{d_k}$。
- 多頭注意力（multi-head attention）、因果遮罩（causal mask）與交叉注意力（cross-attention）怎麼建構，並用程式碼與 PyTorch 逐一比對。
- 為什麼注意力需要位置資訊，以及正弦、可學習、相對與旋轉（rotary）位置編碼如何提供它。
- Transformer 區塊如何組合注意力、MLP、LayerNorm 與殘差連接。
- Vision Transformer（ViT）如何把影像變成 patch token、為什麼它需要那麼多資料，以及 DeiT、Swin 與 CNN–Transformer 混合設計如何回應這個問題。
- 注意力為什麼昂貴；FlashAttention、視窗與 token 合併各自改變了什麼；Mamba 與 Mamba-2 和注意力的差異，以及影像掃描順序如何影響全域上下文與方向偏差。

先看全貌

CNN（ML 03）讓資訊透過固定的局部鄰域傳遞：一個像素要經過很多層，才能「聽到」遠處像素的消息。**注意力**（attention）則讓每個位置根據自己目前的內容，自行決定要讀取哪些其他位置，而且一步就到。要辨認一隻鳥，含有鳥頭的 patch 可以直接參考遠處含有翅膀的 patch。

> **用最簡單的話說**
>
> 想像一間教室，每個小朋友手上拿著一片拼圖。每個人都對自己那一片提出一個問題（「我這片有藍色的邊，誰有天空？」），其他人則各自舉起一張卡片，描述自己手上那一片。每個小朋友會最專心聽那些卡片最符合自己問題的同學，然後更新自己對這片拼圖的想法。接著大家用更新後的想法再來一輪。注意力就是這個「聽誰說話」的規則；Transformer 則是重複很多輪聆聽，中間再穿插一點自己思考的時間。

注意力的想法來自用循環網路做的機器翻譯：ML 03A 說明了它如何移除編碼器–解碼器 RNN 的單一向量瓶頸。Vaswani 等人的 Transformer [1] 走得更遠，完全拿掉循環結構，編碼器與解碼器都只由注意力和逐 token 的小型網路組成。Dosovitskiy 等人的 Vision Transformer（ViT）[2] 則幾乎原封不動地把 Transformer 編碼器套用到影像 patch 序列上。Bishop 與 Bishop [10] 提供了現代的教科書式推導，而現代觀點一節討論的綜述 [3]、[11]、[12] 則整理了眾多視覺變體。讀

完本篇之後，DLCV 02 會在實務上比較 CNN 與 Transformer 骨幹在分類上的表現，ML 05 則說明這些骨幹如何在沒有標註的情況下預訓練。

自注意力：query、key 與 value

白話版。每個 token 產生三個向量：**query**（「我在找什麼？」）、**key**（「我含有什麼？」）以及 **value**（「如果你聽我的，我要傳出什麼？」）。一個 token 的輸出，就是所有 value 的平均，權重取決於它的 query 與每個 key 的吻合程度。

精確版。把 N 個 token 疊成矩陣 $X \in \mathbb{R}^{N\times D}$ 的各列，其中 D 是模型寬度。三個可學習的矩陣將它們投影：

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V,$$

其中 $W_Q, W_K \in \mathbb{R}^{D\times d_k}$、$W_V \in \mathbb{R}^{D\times d_v}$。**縮放點積注意力**（scaled dot-product attention）[1] 為

$$\mathrm{Attention}(Q,K,V) = \mathrm{softmax}_{\mathrm{row}}\!\left(\frac{QK^\top}{\sqrt{d_k}}\right)V = AV.$$

$QK^\top$ 是一個 $N\times N$ 的**分數**矩陣：第 (i,j) 項是 query i 與 key j 的點積，衡量 token j 對 token i 有多相關。逐列的 softmax 把每一列轉成非負且總和為一的權重，得到**注意力矩陣** A。輸出 $Y = AV \in \mathbb{R}^{N\times d_v}$ 每個 token 一列：第 i 列是 $\sum_j A_{ij} v_j$，也就是 value 向量的加權平均。

為什麼要三個不同的投影？如果 query 和 key 是同一個向量，每個 token 都會和自己最吻合，而且相關性被迫是對稱的。分開的 W_Q 與 W_K 讓「我要找的東西」可以不同於「我能提供的東西」（眼睛的 patch 可以去找鼻子，而不必自己就是鼻子）；分開的 W_V 則讓「我能提供的東西」不同於「我實際傳出的內容」。

為什麼要除以 $\sqrt{d_k}$？若 query 與 key 的各分量互相獨立、平均為零、變異數為一，則點積 $q\cdot k = \sum_{l=1}^{d_k} q_l k_l$ 的平均為零、變異數為 d_k。不縮放的話，頭越寬分數就越大，softmax 會飽和（幾乎所有權重集中在一個 token 上），梯度也會變得極小。除以 $\sqrt{d_k}$ 讓變異數回到一。這是一種尺度慣例，並不是把每個 query 正規化成單位長度。

決定誰跟誰交談的是內容，而不是位置。卷積的權重綁定在相對位置上；注意力的權重則是在執行時由資料算出來的。由此衍生一個性質：自注意力是**排列等變**（permutation-equivariant）的。把輸入 token 打亂，輸出也會以相同方式被打亂，因為公式裡沒有任何東西涉及 token 的順序。下面的程式碼同時檢查形狀與這個對稱性。

```
import torch

torch.manual_seed(0)
X = torch.randn(5, 8)                    # N=5, D=8
Wq, Wk, Wv = [torch.randn(8, 4) for _ in range(3)]
def attention(x):
    q, k, v = x @ Wq, x @ Wk, x @ Wv
    a = (q @ k.T / q.shape[-1]**.5).softmax(dim=-1)
    return a @ v, a
```

```
y, a = attention(X)
perm = torch.tensor([3, 0, 4, 1, 2])
yp, _ = attention(X[perm])
assert torch.allclose(a.sum(-1), torch.ones(5), atol=1e-6)
assert torch.allclose(yp, y[perm], atol=1e-5)
print("output shape:", tuple(y.shape))
```

這個對稱性很適合處理集合，但影像不是一堆 patch 的集合：眼睛長在嘴巴下面的臉，就不是一張臉。下面介紹的位置資訊會打破這個對稱性。

多頭注意力

白話版。平行執行好幾個注意力運算，每個都有自己較小的投影，讓不同的頭可以追蹤不同的關係（鄰近紋理、同一個物體、對稱的部位）。最後把它們的輸出串接起來再混合。

精確版。有 h 個頭時，第 r 個頭有自己的 $W_Q^{(r)}, W_K^{(r)}, W_V^{(r)} \in \mathbb{R}^{D\times d_k}$，通常取 $d_k = d_v = D/h$：

$$\text{head}_r = \text{Attention}\big(XW_Q^{(r)}, XW_K^{(r)}, XW_V^{(r)}\big), \qquad \text{MHA}(X) = [\,\text{head}_1, \dots, \text{head}_h\,]\,W_O,$$

其中方括號把 h 個寬度為 D/h 的輸出串接回寬度 D，$W_O \in \mathbb{R}^{D\times D}$ 再把它們混合。由於每個頭寬度只有 D/h，多頭注意力的成本與一個全寬度的頭差不多，所有投影合計 $4D^2$ 個權重（另加偏差）。各個頭不保證會學到人類看得懂的角色；有些頭是多餘的，可以被剪除。

下面的實作把投影切成多個頭、逐頭計算分數、串接後再投影，結果與 `nn.MultiheadAttention` 完全一致。它也示範了**因果遮罩**：在 softmax 之前把禁止的分數設為 $-\infty$，這些位置的權重就會恰好為零。

```
import torch
from torch import nn

torch.manual_seed(0)
B, N, D, h = 2, 6, 16, 4
mha = nn.MultiheadAttention(D, num_heads=h, batch_first=True, bias=False)
X = torch.randn(B, N, D)

def multi_head(x):
    Wq, Wk, Wv = mha.in_proj_weight.chunk(3)          # each D x D (rows = outputs)
    q, k, v = x @ Wq.T, x @ Wk.T, x @ Wv.T              # B x N x D
    split = lambda t: t.view(B, N, h, D // h).transpose(1, 2)   # B x h x N x d_k
    q, k, v = split(q), split(k), split(v)
    scores = q @ k.transpose(-2, -1) / (D // h) ** 0.5  # B x h x N x N
    A = scores.softmax(dim=-1)
    heads = A @ v                                       # B x h x N x d_k
    concat = heads.transpose(1, 2).reshape(B, N, D)     # put the heads side by side
    return concat @ mha.out_proj.weight.T, A

ours, A = multi_head(X)
ref, _ = mha(X, X, X)
print("matches nn.MultiheadAttention:", torch.allclose(ours, ref, atol=1e-5))
```

```
print("attention tensor:", tuple(A.shape), "| parameters:",
      sum(p.numel() for p in mha.parameters()), "= 4 * D^2 =", 4 * D * D)

# Causal mask: query i may only read keys j <= i
causal = torch.triu(torch.ones(N, N, dtype=torch.bool), diagonal=1)   # True = forbidden
out_c, w_c = mha(X, X, X, attn_mask=causal)
print("no weight on the future:", torch.all(w_c[:, causal] == 0).item())
```

輸出為 `True`、形狀 `(2, 4, 6, 6)` 的注意力張量（批次、頭、query、key）、`1024 = 4 * D^2 = 1024`，以及因果檢查的 `True`。正式的程式碼請呼叫 `torch.nn.functional.scaled_dot_product_attention`，它計算的是同一件事，並會在裝置、資料型別與形狀允許時，自動改用融合（fused）、省記憶體的核心 [7]。

連接 token 的三種方式

白話版。一個 query 可以讀取哪些 token，是模型設計的一部分，以分數矩陣上的遮罩來表達。

機制	query 可讀取	典型用途
雙向自注意力	同一序列中的所有 token	ViT 及其他影像編碼器
因果（遮罩）自注意力	只有自己與更早的位置	下一個 token 的生成、自迴歸影像模型
交叉注意力	來自另一個序列的 token	解碼器讀取編碼器；影像 token 讀取文字

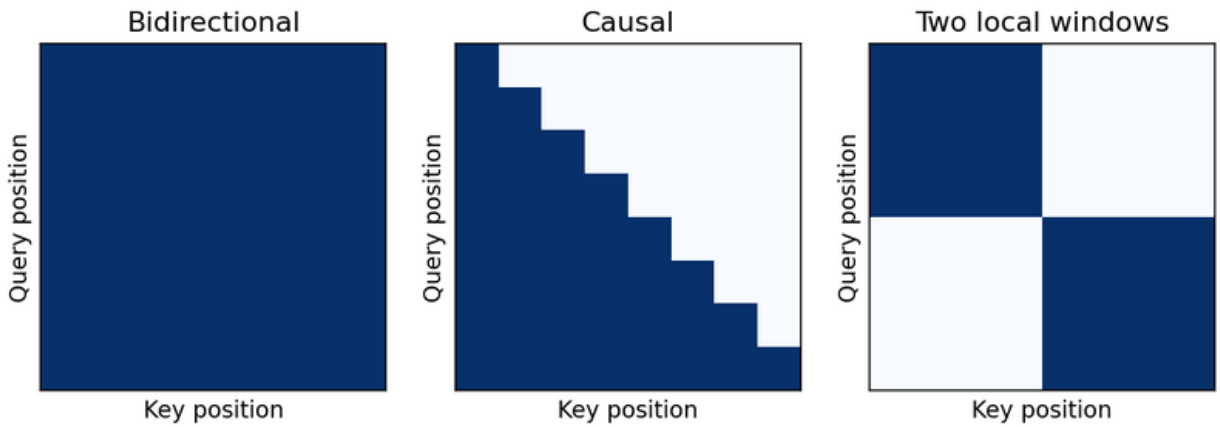


圖 *1* — 注意力遮罩：藍色表示允許，白色表示禁止。因果生成還會把輸入平移一個位置，讓目標無法被直接複製。

在**交叉注意力**中，query 來自一個序列 X，key 與 value 則來自另一個序列 Z：$Q = XW_Q$、$K = ZW_K$、$V = ZW_V$。分數矩陣的形狀是 $N_X \times N_Z$，輸出每個 query 一列。原始 Transformer 的解碼器就是這樣讀取編碼器；影像描述模型這樣讀取影像特徵；文字生成影像的擴散模型也這樣讀取文字提示（ML 07），而不需要在影像 patch 上做自迴歸。

注意力權重是計算的一部分，並不自動等於一種解釋。很大的權重可能乘上一個毫無資訊的 value，而殘差與 MLP 路徑也會影響輸出。

位置資訊

白話版。注意力不理會順序，所以我們必須告訴模型每個 token 在哪裡。最簡單的方法是在第一層之前，為每個 token 加上一個與位置有關的向量；更精緻的方法則把相對位置直接注入注意力分數。

正弦位置編碼。原始 Transformer [1] 在位置 pos 的 token 上加一個固定向量 $p_{pos} \in \mathbb{R}^D$：

$$p_{pos,2i} = \sin\!\left(\frac{pos}{10000^{2i/D}}\right), \qquad p_{pos,2i+1} = \cos\!\left(\frac{pos}{10000^{2i/D}}\right), \qquad i = 0, \dots, \tfrac{D}{2} - 1.$$

每一對維度 $(2i, 2i+1)$ 都是一個以角頻率 $\omega_i = 10000^{-2i/D}$ 繞圓旋轉的點，頻率從很快（每個位置一弧度）到非常慢都有。由三角函數的和角公式可得兩個性質。第一，對任一固定位移 k，都有 $p_{pos+k} = p_{pos} R_k$，其中 R_k 是一個與 pos 無關的區塊對角旋轉矩陣；因此一個線性層就能學會「往後看 k 個位置」。第二，$p_m \cdot p_n = \sum_i \cos\left(\omega_i (m-n)\right)$ 只與位移 $m-n$ 有關（圖 2 中）。

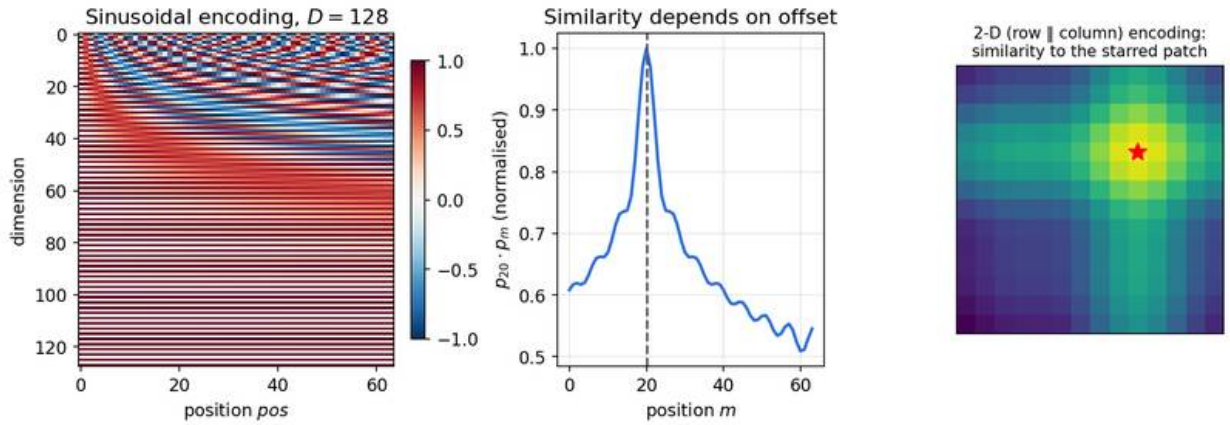


圖 2 — 正弦位置編碼。左：每一對維度以各自的頻率振盪。中：編碼之間的相似度只取決於位移。右：把列與行的編碼串接起來，就得到 14×14 *patch* 網格的二維編碼。

```
import numpy as np

def sinusoidal(n_pos, D):
    pos = np.arange(n_pos)[:, None]
    omega = 1.0 / 10000 ** (np.arange(0, D, 2) / D)    # one frequency per sin/cos pair
    pe = np.zeros((n_pos, D))
    pe[:, 0::2] = np.sin(pos * omega)
    pe[:, 1::2] = np.cos(pos * omega)
    return pe, omega

pe, omega = sinusoidal(100, 16)
k = 7                                                  # a fixed offset
```

```
# block-diagonal rotation that maps p_pos to p_{pos+k}, the same for every pos
R = np.zeros((16, 16))
for i, w in enumerate(omega):
    c, s = np.cos(w * k), np.sin(w * k)
    R[2 * i:2 * i + 2, 2 * i:2 * i + 2] = [[c, -s], [s, c]]
print("p[pos+k] == p[pos] @ R for all pos:", np.allclose(pe[:-k] @ R, pe[k:]))
print("dot products depend only on offset:",
      np.isclose(pe[10] @ pe[17], pe[50] @ pe[57]))
```

兩項檢查都印出 `True`。

其他做法。

- **可學習的絕對位置嵌入**：每個位置一個可訓練的向量，ViT [2] 就是如此。ViT 依光柵（raster）順序編號的一維可學習嵌入，效果與明確的二維嵌入差不多；訓練後，同一列或同一行的 patch 嵌入會變得相似，代表模型自己學會了網格結構。換到新的解析度時，先用二維內插調整嵌入網格的大小，再進行微調。
- **相對位置表示**（Shaw 等人 [17]）在 token i 與 j 的分數上，加一個取決於位移 $j-i$ 的可學習項。Swin [5] 則在每個視窗的分數上加一張可學習的**相對位置偏差**（relative position bias）表。
- **旋轉位置嵌入**（rotary position embedding, RoPE；Su 等人 [16]）把每個 query 與 key 依其位置旋轉一個成比例的角度，使用的正是上面程式碼中的那種旋轉。於是分數 $q_m^\top k_n$ 只透過 $m-n$ 依賴位置。RoPE 已是大型語言模型的標準配備，在視覺中也越來越常見，做法是對影像的兩個軸分別旋轉。
- **不加明確編碼**：網路內部的卷積（如 CvT [19]），或邊界的零填補，本身就可能洩漏足夠的位置資訊。

Transformer 區塊

白話版。一個區塊交替做兩件事：注意力讓 token 互相交換資訊；接著一個小型 MLP 分別處理每個 token。殘差連接與正規化讓整個堆疊保持穩定。

精確版。 ViT 與大多數現代模型使用的**前置正規化**（pre-norm）區塊為

$$U = X + \mathrm{MHA}(\mathrm{LN}(X)), \qquad Y = U + \mathrm{MLP}(\mathrm{LN}(U)), \qquad \mathrm{MLP}(u) = W_2\,\mathrm{GELU}(W_1 u + b_1) + b_2.$$

LN 是**層正規化**（layer normalization）[14]：它把每個 token 的 D 個特徵標準化為平均零、變異數一，再套用可學習的縮放與平移；與 BatchNorm 不同，它不依賴同一批次中的其他樣本。MLP 把每個 token 從 D 擴展到通常 $4D$ 個隱藏單元，套用 GELU 非線性 [26]（$\mathrm{GELU}(z) = z\,\Phi(z)$，$\Phi$ 為標準常態分布的累積分布函數），再投影回去。殘差相加代表每個子層只需學習一個修正量，與 ResNet 完全相同。

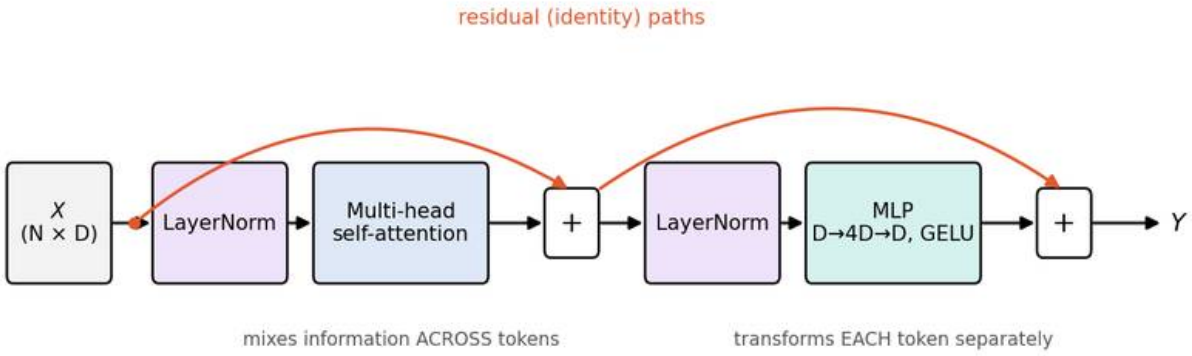


圖 3 — 前置正規化的 *Transformer* 區塊。注意力在 *token* 之間混合資訊；*MLP* 分別轉換每個 *token*；殘差路徑（橘色）把輸入繞過每個分支。

原始 Transformer 把 LayerNorm 放在每次殘差相加**之後**（後置正規化，post-norm）。Xiong 等人 [15] 證明，使用後置正規化時，初始化階段靠近輸出的梯度很大，這正是它需要學習率暖身（warm-up）的原因；前置正規化則讓梯度保持穩定，可以不用暖身就訓練。所以只說「一個 Transformer」並不完整，還得說明正規化放在哪裡。每個區塊的權重數：注意力 $4D^2$、MLP $8D^2$，合計約 $12D^2$，其中三分之二在 MLP。

Vision Transformer（ViT）

白話版。把影像切成方形 patch，把每個 patch 變成向量，加上位置資訊，然後送進標準的 Transformer 編碼器。答案從一個額外的特殊 token 讀出。

切成 patch。對一張 $H \times W$ 像素、C 個通道的影像，若 patch 大小 P 能整除 H 與 W，就有 $N = HW/P^2$ 個 patch。把每個 patch 攤平成 $x_i^{\text{patch}} \in \mathbb{R}^{P^2C}$，再做線性投影：

$$z_0 = [\, x_{\text{cls}};\ x_1^{\text{patch}}E;\ \ldots;\ x_N^{\text{patch}}E \,] + E_{\text{pos}},$$

其中 $E \in \mathbb{R}^{P^2C \times D}$ 是可學習的 **patch 嵌入**，$x_{\text{cls}} \in \mathbb{R}^D$ 是加在序列最前面的可學習**類別 token**（class token），$E_{\text{pos}} \in \mathbb{R}^{(N+1)\times D}$ 存放可學習的位置嵌入。經過 L 個區塊後，對類別 token 的輸出套用一個 LayerNorm 與線性分類頭，就得到 logits。把所有 patch 的輸出取平均（mean pooling）也是同樣常見的替代做法。

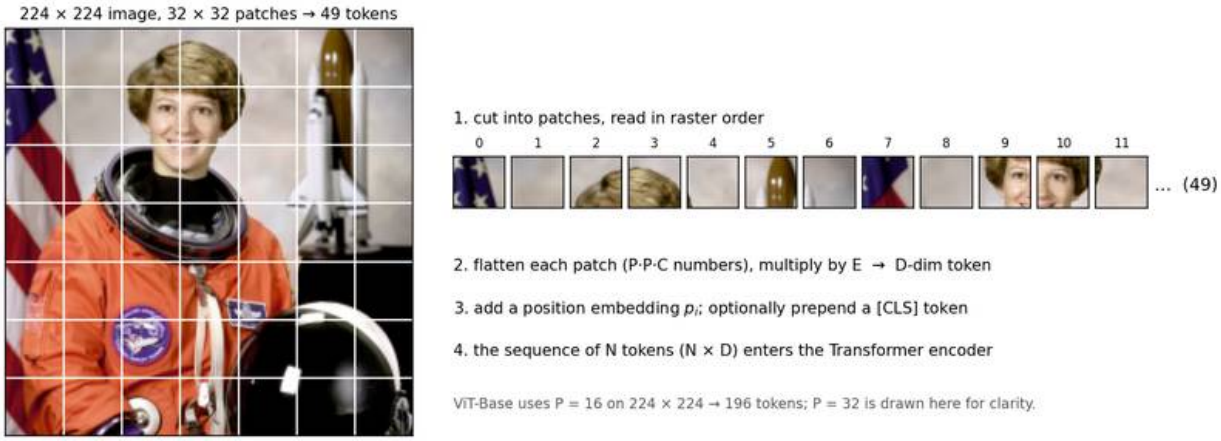


圖 *4* — 切成 *patch*。為了清楚起見這裡畫的是 *32* 像素的格子；*ViT-Base* 使用 *16 × 16* 的 *patch*，把 *224 × 224* 的影像變成 *196* 個 *token*。

對不重疊的 $P \times P$ patch 套用線性映射，恰好等於核大小與步幅都是 P 的卷積，實作上也正是這樣寫：

```
import torch
from torch import nn

torch.manual_seed(0)
B, C, H, W, P, D = 2, 3, 32, 32, 8, 48
x = torch.randn(B, C, H, W)

# 1) explicit: cut into P x P patches, flatten each, multiply by E
patches = x.unfold(2, P, P).unfold(3, P, P)                 # B, C, H/P, W/P, P, P
patches = patches.permute(0, 2, 3, 1, 4, 5).reshape(B, -1, C * P * P)   # B, N, P*P*C
E = torch.randn(C * P * P, D) * 0.02
tokens_explicit = patches @ E                                # B, N, D

# 2) the same linear map written as a convolution with kernel = stride = P
conv = nn.Conv2d(C, D, kernel_size=P, stride=P, bias=False)
conv.weight.data = E.T.reshape(D, C, P, P)
tokens_conv = conv(x).flatten(2).transpose(1, 2)             # B, N, D

print("tokens:", tuple(tokens_conv.shape), "| identical:",
      torch.allclose(tokens_explicit, tokens_conv, atol=1e-5))
```

輸出為 16 個寬度 48 的 token，以及 `True`。

模型大小。標準設定有 ViT-Base（12 個區塊、$D = 768$、12 個頭、約 8,600 萬個參數）、ViT-Large（24 個區塊、$D = 1024$、16 個頭）與 ViT-Huge（32 個區塊、$D = 1280$、16 個頭）[2]。「ViT-B/16」代表使用 16×16 patch 的 Base 模型。**patch 大小是最主要的運算量旋鈕**：P 減半，N 變成四倍，注意力分數的數量變成十六倍。小 patch 保留細節；大 patch 便宜，但比 patch 還細的資訊都會遺失，而且後面任何一層都救不回來。

資料的教訓。 Dosovitskiy 等人 [2] 發現，只在 ImageNet-1k（約 130 萬張影像）上訓練的 ViT，表現略遜於規模相當的 ResNet；但若先在 ImageNet-21k（1,400 萬張）或 JFT-300M（3 億張）上預訓練再微調，它就能追平甚至超越 ResNet，而且預訓練成本更低。他們的解釋是**歸納偏置**（inductive bias）：CNN 在每一層都假設局部性與平移等變性，而 ViT 只在 patch 嵌入內是局部的，空間關係必須從資料中**學出來**。Raghu 等人 [18] 比較了 ViT 與 ResNet 的內部表徵，發現 ViT 各層的表徵更加一致、較低層同時兼顧局部與全域的注意力，而且要在早期層學到局部注意力，需要大規模的預訓練。

程式碼：在手寫數字上訓練一個小型 ViT

這個完整範例在與ML 03 CNN 相同的 8×8 手寫數字資料上訓練一個小型 ViT，使用 2×2 的 patch（16 個 token）、一個類別 token、可學習的位置嵌入，以及四個前置正規化區塊。在 CPU 上約半分鐘即可跑完。

```
import copy
import numpy as np
import torch
from torch import nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0)
torch.set_num_threads(1)

class TinyViT(nn.Module):
    def __init__(self, img=8, patch=2, chans=1, D=64, depth=4, heads=4, classes=10):
        super().__init__()
        N = (img // patch) ** 2
        self.patch = nn.Conv2d(chans, D, kernel_size=patch, stride=patch)   # patchify + E
        self.cls = nn.Parameter(torch.zeros(1, 1, D))                       # [CLS] token
        self.pos = nn.Parameter(torch.randn(1, N + 1, D) * .02)             # learned
         ↪  positions
        layer = nn.TransformerEncoderLayer(D, heads, dim_feedforward=4 * D, dropout=0.1,
                                           activation="gelu", batch_first=True,
                                           norm_first=True)                 # pre-norm block
        self.blocks = nn.TransformerEncoder(layer, depth, enable_nested_tensor=False)
        self.norm, self.head = nn.LayerNorm(D), nn.Linear(D, classes)

    def forward(self, x):
        t = self.patch(x).flatten(2).transpose(1, 2)                         # B, N, D
        t = torch.cat([self.cls.expand(len(t), -1, -1), t], dim=1) + self.pos
        return self.head(self.norm(self.blocks(t))[:, 0])                    # read [CLS]

d = load_digits()
X = torch.tensor(d.images[:, None] / 16.0, dtype=torch.float32)
y = torch.tensor(d.target)
train, test = train_test_split(np.arange(len(y)), test_size=.2, stratify=d.target,
                               random_state=0)
train, val = train_test_split(train, test_size=.2, stratify=d.target[train], random_state=1)
model = TinyViT()
```

```
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.05)
best = (float("inf"), None)
for epoch in range(30):
    model.train()
    for idx in torch.tensor(train)[torch.randperm(len(train))].split(64):
        loss = nn.functional.cross_entropy(model(X[idx]), y[idx])
        opt.zero_grad(); loss.backward(); opt.step()
    model.eval()
    with torch.no_grad():
        v = nn.functional.cross_entropy(model(X[val]), y[val]).item()
    if v < best[0]:
        best = (v, copy.deepcopy(model.state_dict()))
model.load_state_dict(best[1]); model.eval()
with torch.no_grad():
    acc = (model(X[test]).argmax(1) == y[test]).float().mean().item()
print("parameters:", sum(p.numel() for p in model.parameters()))
print("best validation loss:", round(best[0], 4), "| held-out accuracy:", round(acc, 4))
```

在我們的執行中，它約有 202,000 個參數，保留測試集準確率約 95%；而 ML 03 的 CNN 在相同的資料切分下，只用不到 10,000 個參數就達到約 96%。在小型資料集上，CNN 內建的局部性大約值二十倍的參數量。這是「資料的教訓」的縮影，而不是對大規模效能的定論。

訓練配方：DeiT 及其後續

白話版。早期 ViT 與 CNN 在 ImageNet 規模資料上的差距，很大一部分其實來自訓練方式，而不是架構。

DeiT（Touvron 等人 [4]）**只用 ImageNet-1k** 就把 ViT 訓練到具競爭力，靠的是強力的資料擴增（RandAugment、Mixup、CutMix、隨機抹除）、大量正則化（stochastic depth、label smoothing）以及較長的訓練排程。它還引入了**蒸餾 token**（distillation token）：一個類似類別 token 的額外可學習 token，其輸出被訓練去符合一個預訓練 CNN 教師模型的預測。CNN 的歸納偏置因此透過它的預測轉移過來。後來的 DeiT III [27] 證明，一個更簡單但仔細調校的監督式配方（輕量的「3-Augment」擴增、簡單的裁切，以及二元交叉熵損失函數）能讓純 ViT 更強。Steiner 等人 [25] 對資料、擴增與正則化做了大規模研究（「AugReg」），發現擴增、正則化與運算量的適當組合，可以追上用大約十倍資料訓練的模型。

一般性的教訓與ML 03 討論 ConvNeXt 時相同：只有在資料、擴增、解析度、訓練排程與運算量都對齊的情況下，比較架構才有意義。

Swin 與階層式 Transformer

白話版。對數千個 token 做全域注意力很昂貴，而且純 ViT 從頭到尾維持同一個粗解析度，不利於偵測與分割。Swin 把注意力限制在局部視窗內，在層與層之間平移視窗讓資訊仍能擴散，並合併 patch 來建立類似 CNN 的金字塔。

視窗。 Swin [5] 把 token 網格切成互不重疊、每個 $M \times M$ 個 token 的視窗（通常 $M = 7$），只在每個視窗內部計算自注意力。對 $h \times w$ 的 token 網格、C 個通道，論文給出的成本為

$$\Omega(\text{global MSA}) = 4hwC^2 + 2(hw)^2C, \qquad \Omega(\text{window MSA}) = 4hwC^2 + 2M^2hwC,$$

其中 $4hwC^2$ 是四個投影的成本，第二項則是計算分數與加權的乘法。全域成本對 token 數 hw 是二次方；固定 M 時，視窗成本是線性的。

平移視窗。若視窗固定不動，不同視窗的 token 永遠不會互動。Swin 讓一般視窗與在兩個方向各平移 $\lfloor M/2 \rfloor$ 個 token 的視窗交替出現。在第 ℓ 層位於視窗邊界附近的 token，到第 $\ell+1$ 層會與相鄰視窗的 token 同在一個視窗，資訊因此得以跨越邊界（圖 5）。高效率的實作會把特徵圖做循環平移，並用遮罩避免從對側邊緣繞過來的 token 互相注意。

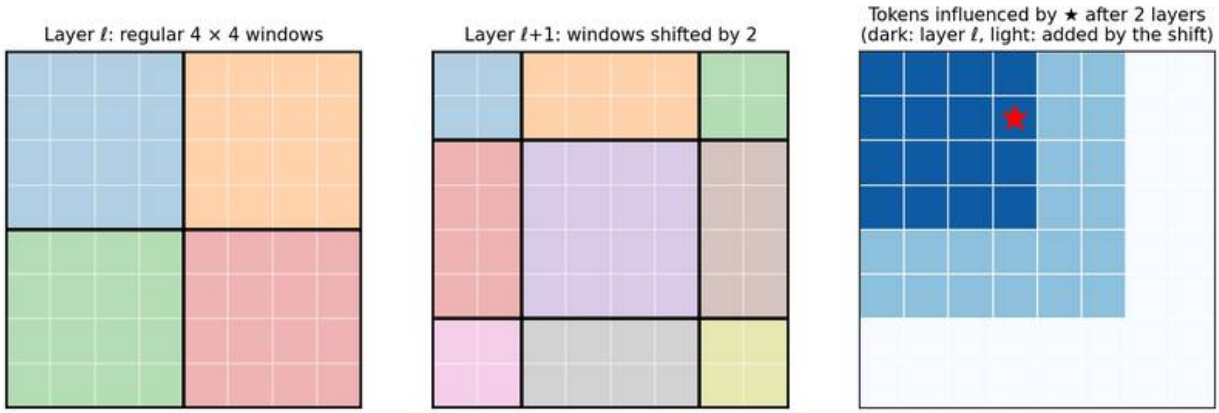


圖 *5 — Swin* 在 *8 × 8 token* 網格上的一般視窗與平移視窗。兩層之後，星號 *token* 的資訊會到達深色視窗（第 ℓ 層），再加上平移視窗帶來的淺色帶狀區域（第 ℓ*+1* 層）。

階層結構。在各階段之間，**patch 合併**（patch merging）把每 2×2 組相鄰 token 串接後做線性投影，使解析度減半、通道數加倍，就像步幅卷積一樣。四個階段分別產生輸入解析度 1/4、1/8、1/16、1/32 的特徵圖，與 ResNet 的金字塔相同，因此 Swin 可以在偵測與分割框架中直接取代 CNN 骨幹。每個視窗的分數也會加上可學習的**相對位置偏差**。Swin V2 [32] 用殘差後置正規化、縮放餘弦注意力（scaled cosine attention），以及連續、對數間隔的位置偏差來擴大規模，後者在不同視窗大小與解析度之間的遷移效果更好。

CNN–Transformer 混合模型

白話版。卷積擅長在資料少時處理局部細節；注意力擅長長距離互動。混合模型讓兩者各自在適合的地方發揮。

- **卷積起始層（stem）**。原始 ViT 論文就已測試過一種「混合」版本，其 patch 取自 ResNet 的特徵圖而不是原始像素 [2]；在較小的運算預算下有幫助。
- **在注意力裡放卷積**。 CvT [19] 在每個階段使用卷積式 token 嵌入，並用深度可分離卷積產生 query、key、value 的投影，藉此內建局部性，也因此可以省去明確的位置嵌入。
- **依階段堆疊**。 CoAtNet [20] 在前期放深度卷積階段（此時解析度高、局部模式為主），後期放相對注意力階段；它在各種資料規模下都報告了強勁的結果，因為卷積階段在資料有限時提供了歸納偏置。

- **行動裝置混合模型**。 MobileViT [28] 在 MobileNet 風格的網路中插入小型 Transformer 區塊，以行動裝置等級的成本加入全域脈絡。
- **反方向**。 ConvNeXt（見ML 03）把 Transformer 時代的設計選擇引進純 CNN，並追平了 Swin。

成本與高效率注意力

白話版。全域注意力要把每個 token 和其他每個 token 比較，所以影像邊長加倍，工作量就變成十六倍。高效率方法要嘛讓精確注意力跑得更快，要嘛限制哪些 token 互動，要嘛減少 token 數量。

對 N 個寬度為 D 的 token，每層分數矩陣約需 N^2D 次乘加，投影與 MLP 約需 $12ND^2$ 次。單純的實作會為每個頭儲存一個 $N \times N$ 矩陣。固定 patch 大小時，影像高與寬各加倍，N 變成四倍，成對分數變成十六倍。

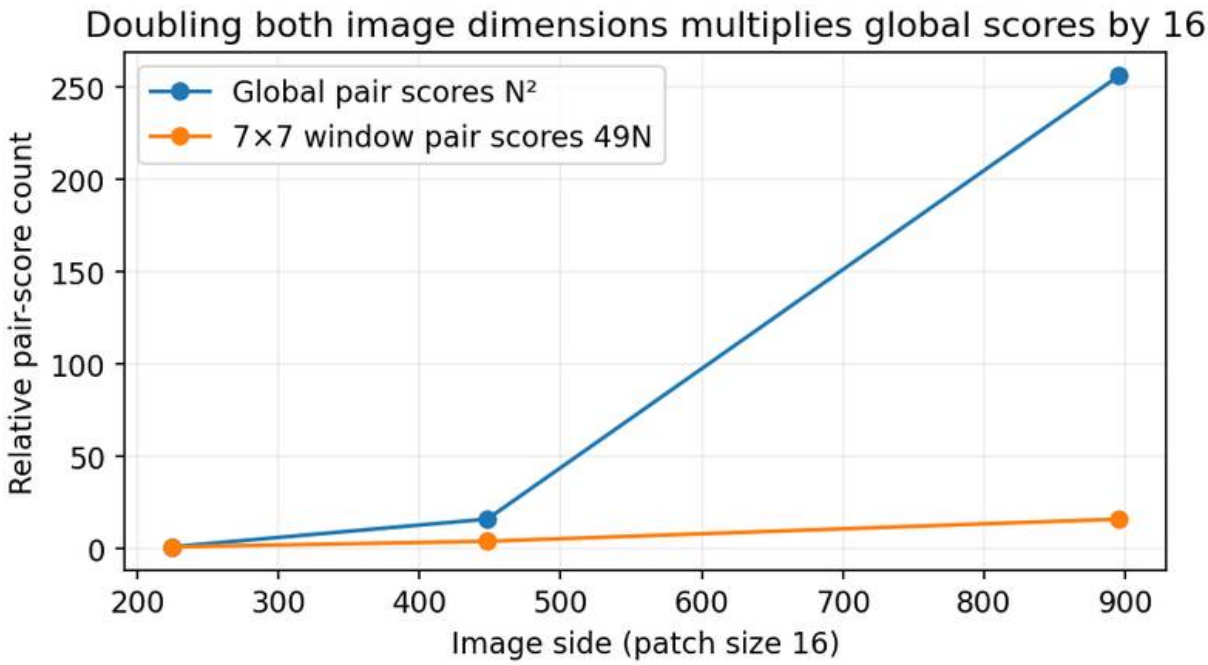


圖 *6* — 成對分數的成長趨勢，並非實測的執行時間。投影、*MLP*、記憶體存取與核心實作品質也會影響延遲。

主要的方法家族：

- **精確但更快**。 FlashAttention [6] 把計算切成區塊（tiling），讓 query、key、value 的區塊留在快速的晶片內記憶體中，並且從不把完整的 $N \times N$ 矩陣寫到較慢的 GPU 記憶體，藉此計算**精確**的注意力；FlashAttention-2 [29] 進一步改善了工作切分方式。算術量仍是二次方，但記憶體用量變成線性，實際執行時間也大幅下降。PyTorch 的 `scaled_dot_product_attention` 在可行時會自動選用這類核心 [7]。
- **限制連接方式**。視窗（Swin）或其他稀疏模式減少了成對的數量，代價是需要好幾層才能達成全域溝通。

- **減少 token**。 Token Merging（ToMe）[21] 在預訓練好的 ViT 內，以快速的配對步驟逐步合併相似的 token，不需重新訓練就能提高吞吐量。較大的 patch、token 剪枝與池化階段則以更粗略的方式達到相同效果。
- **更小的模型**。 Papa 等人的綜述 [13] 把高效率 ViT 分為精簡架構、剪枝、知識蒸餾與量化四類，並提出「Efficient Error Rate」指標，用來在參數量、位元數、FLOPs 與模型大小之間比較模型。

狀態空間模型與混合模型

白話版。不必比較每一對 token，而是像循環網路（ML 03A）一樣，讓一個精簡的狀態沿著序列傳遞。成本隨 token 數線性成長。

對 token 序列，一個簡化的狀態空間遞迴為

$$h_t = \bar{A}_t\, h_{t-1} + \bar{B}_t\, x_t, \qquad y_t = C_t\, h_t,$$

其中 h_t 是固定大小的隱藏狀態，$\bar{A}_t$、$\bar{B}_t$、C_t 是（離散化後的）狀態空間矩陣。在 Mamba 這類**選擇性**（selective）模型中，寫入／讀取係數與步長取決於輸入 x_t；底層連續轉移可以保持固定，離散化後的係數卻會改變。輸入選擇與仿射狀態更新使平行掃描（parallel scan）成為可能 [33]。影像沒有天然的一維順序，因此視覺版本必須選擇**掃描順序**：Vision Mamba（Vim）[30] 加上位置嵌入，對 patch 序列做正向與反向掃描；VMamba [31] 則沿著二維網格的四個方向掃描。Visual Mamba 綜述 [8] 回顧了 200 多篇論文，並指出掃描方式是視覺應用中最關鍵的設計決策。

MambaOut [9]（CVPR 2025）是一個有用的反例。作者主張 Mamba 適合長序列與因果（自迴歸）結構的任務，並在保留閘控卷積區塊的同時拿掉狀態空間元件，建立了 MambaOut 模型。他們的模型在 ImageNet 分類上勝過視覺 Mamba 模型，顯示 Mamba 在這裡並非必要；而對於涉及長 token 序列的偵測與分割，他們認為狀態空間模型仍值得探索。更廣泛的重點在於方法論：要把某個結果歸功於某種機制（注意力、掃描、卷積）之前，必須先把它與閘控、正規化、訓練配方和參數預算分開來看。

影像掃描是一種歸納偏置

Mamba [33] 提供高效率的序列運算，但不會替我們決定影像的幾何結構。Vim [30] 使用 patch 序列、位置嵌入與正向／反向處理；VMamba [31] 則把四條路徑的選擇性掃描放進階層式視覺骨幹中。單靠模型名稱，無法判斷感受野的影響強度、解析度行為或執行時間。狀態維度、掃描路徑、局部運算與降取樣階段都會影響結果。

在固定掃描順序下，影像中相鄰的兩個像素，可能在序列中相隔許多步。反過來，一列最後的像素與下一列第一個像素，雖然在空間上相距甚遠，卻在序列中連續。局部卷積可以先提供鄰近證據，再由掃描把上下文傳得更遠。多條路徑提供額外的傳遞方式，但不會自動恢復所有幾何對稱性。位置嵌入也能編碼有用的位置，卻讓移轉到不同網格成為另一個需要處理的設計問題。

可以用數學明確呈現這個選擇：

$$Y^{(r)} = P_r^{-1}\, \mathcal{S}_r(P_r X), \qquad Y = \frac{1}{m}\sum_{r=1}^{m} Y^{(r)}.$$

其中 X 包含展平後的影像 token，r 索引 m 條路徑中的一條，P_r 把 token 排列成該路徑的順序，$\mathcal{S}_r$ 執行該路徑的序列循環，P_r^{-1} 還原影像座標，Y 對產生的特徵取平均。這是概念性的路徑融合模型。可學習的視覺區塊可以使用不同投影或融合規則；這裡選擇平均，是為了讓實驗容易解讀。

CPU 實驗：量測方向偏差

下一個範例使用單一純量狀態，每條路徑的衰減係數都相同。先比較正向／反向的逐列掃描，以及加入逐欄掃描的四條路徑；接著把輸入旋轉 90 度、處理後再把輸出轉回來。全部使用合成的 6×6 脈衝，不下載權重或影像。這個實驗單獨檢查排序，並不重現訓練好的 Vim 或 VMamba。

```
import numpy as np

def scan_image(image, four=False):
    ids = np.arange(image.size).reshape(image.shape)
    row, col = ids.ravel(), ids.T.ravel()
    orders = [row, row[::-1]]
    if four:
        orders += [col, col[::-1]]
    outputs = []
    for order in orders:
        h = 0.0
        out = np.zeros(image.size)
        for i in order:
            h = 0.8 * h + 0.2 * image.ravel()[i]
            out[i] = h
        outputs.append(out.reshape(image.shape))
    return np.mean(outputs, axis=0)

image = np.zeros((6, 6))
image[1, 4] = 1.0
errors = []
for four in [False, True]:
    original = scan_image(image, four=four)
    rotated_back = np.rot90(scan_image(np.rot90(image), four=four), -1)
    error = np.max(np.abs(original - rotated_back))
    errors.append(error)
    print('four routes:', four, 'rotation discrepancy:', round(error, 6))
assert 0.0 < errors[1] < errors[0]
```

兩條路徑的最大差異為 `0.063622`，四條路徑則為 `0.026738`。在這個範例中，增加路徑減少方向偏差，但剩餘差異並非零。旋轉逐列路徑時，其中一個遍歷軸會反向；變換後的路徑未必屬於原本四條路徑的集合。因此，即使係數共用且以平均融合，也不保證旋轉等變性。可學習的方向投影、位置資訊與局部運算又引入更多選擇。應直接測試需要的性質，而不是從「二維掃描」推定它存在。

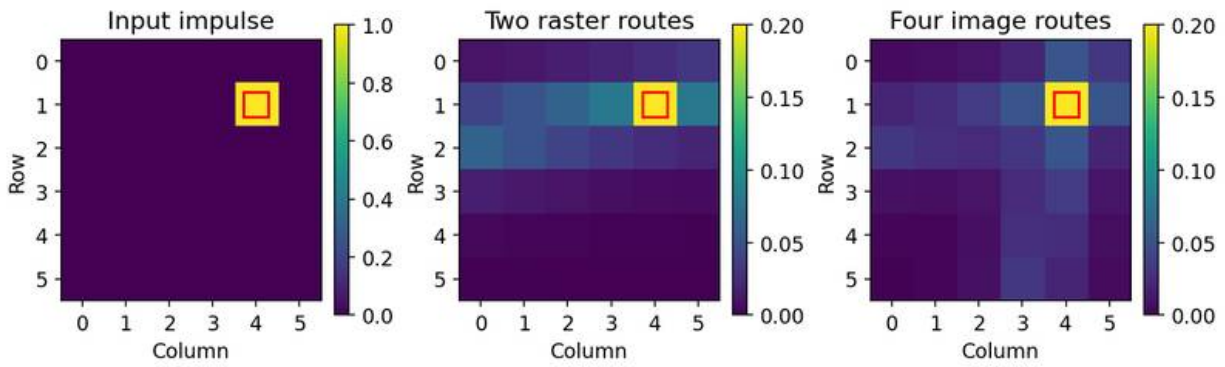


圖 7 — 紅色外框標示輸入脈衝。兩條逐列路徑不均勻地傳遞上下文；加入逐欄路徑後，影響圖會改變。這些合成響應使用範例中的相同循環，並非學得的注意力圖或訓練好的 *VMamba* 活化值。

資訊能沿著完整掃描傳遞，因此循環可以具有全域感受野。這表示依賴關係可能存在，並不代表它在所有距離都保持強大影響。純量衰減為 0.8 時，攜帶的貢獻每經過一步就再乘上 0.8。ML 03A 推導的選擇性轉移可以改變這種保留方式，但訓練仍須學會哪些資訊值得保存。即使可以讀取遠處上下文，狀態壓縮或降取樣仍可能遺失細緻邊界與小物件。

Mamba-2 改變了什麼，又該如何評估

Mamba-2 [34] 以結構化狀態空間對偶性與高效率的分塊計算，重新安排選擇性序列混合器。它改變轉移結構與區塊參數投影，並沒有讓任意影像 patch 序列都等同於 Transformer。具有結構的因果係數矩陣可以用類似注意力的方式解讀，但標準 softmax 注意力仍保有不同的表達與計算性質。應用到影像時，仍須指定方向與融合方式。因此，對偶性回答的是計算問題，並未消除幾何問題。

公平的視覺比較應區分三種實驗：在共用區塊中只替換混合器；在對齊的訓練預算下比較完整骨幹；以及在目標硬體上量測部署模型。固定資料、資料增強、token 解析度與下游預測頭。除了參數量，也記錄狀態大小，並在不同影像解析度與批次大小下量測延遲及記憶體峰值。token 數的線性項可以同時伴隨昂貴的投影、路徑排列與小型運算核心的額外成本。上面的 Python 迴圈無法支持 GPU 吞吐量的宣稱。

對密集任務，檢查邊界附近、小型結構與不同長寬比下的錯誤；對影片，區分離線雙向編碼器與因果即時處理。除了分布內分類，也應納入旋轉／裁切敏感度與更長網格的評估。MambaOut [9] 提供重要的移除混合器消融實驗。結果支持依任務比較，並不是證明所有影像問題都不需要掃描的定理。Xu 等人 [8] 依骨幹、掃描機制，以及影像／影片／點雲／多模態應用整理視覺 Mamba。在已核實的紀錄中，這篇綜述仍是 2024 年預印本。其分類架構有助於選擇控制變因的實驗；提出的變體數量並不能確立普遍的贏家。

現代觀點

綜述文獻。 Khan 等人 [3]（*ACM Computing Surveys*，2022）從基礎（自注意力、大規模預訓練、雙向編碼）談起，接著綜覽 Transformer 在辨識、生成模型、視覺問答等多模態任務、影

片、低階視覺與三維分析上的應用，最後列出開放問題。Han 等人 [11]（*IEEE TPAMI*，2023）依任務整理視覺 Transformer（骨幹、偵測與分割等中高階視覺、低階視覺、影片），也涵蓋高效率 Transformer 方法。Liu 等人 [12]（*IEEE TNNLS*，2024）回顧了一百多個用於影像、點雲與視覺–語言資料之分類、偵測與分割的視覺 Transformer，並提出三個未來方向。在效率方面，Papa 等人 [13]（*IEEE TPAMI*，2024）最有系統。狀態空間視覺模型請參考 Xu 等人 [8]。教科書式的推導則見 Bishop 與 Bishop [10]。

2023–2026 年的最新發展。純 ViT 已成為預設的**基礎模型**（foundation model）骨幹。ViT-22B [24]（ICML 2023）把 ViT 擴大到 220 億個參數，並需要一些架構修改（例如對 query 與 key 做正規化）才能在這個規模下穩定訓練。DINOv2 [23]（TMLR 2024）以自監督方式在大型精選影像集合上訓練 ViT，得到的通用特徵在凍結狀態下就能用於分類、分割與深度估計（ML 05）。Darcet 等人 [22]（ICLR 2024）發現大型 ViT 會把資訊量低的背景 patch 挪作內部暫存空間，在注意力圖中產生高範數的假影 token；加入少數額外的**暫存器 token**（register tokens）就能消除這些假影，並改善密集預測。視覺–語言基礎模型（ML 09）與擴散 Transformer（ML 07）都建立在同樣的區塊上。與此同時，階層式與混合設計在高解析度的密集任務上仍很常見；在預算對齊時，現代化的 CNN 依然具有競爭力（ML 03）；而狀態空間與線性注意力模型仍持續被探索，用於影片這類非常長的 token 序列。

開放問題。高解析度與長影片下的二次方成本；純 ViT 對資料與運算量的龐大需求；能在不同解析度與長寬比之間泛化的位置編碼；如何解讀注意力而不過度詮釋；以及在注意力、卷積與狀態空間機制之間，做出運算量對齊的公平比較。

重點整理

- 自注意力是一個可學習、隨輸入而變的加權平均：$\mathrm{softmax}(QK^\top/\sqrt{d_k})V$。除以 $\sqrt{d_k}$ 讓分數維持單位變異數，softmax 才不會飽和。
- 多頭注意力以約等於一個頭的成本，平行執行 h 個較窄的頭；遮罩（因果、視窗）與交叉注意力決定誰讀取誰。
- 注意力是排列等變的，因此必須注入位置資訊：正弦、可學習絕對、相對偏差或旋轉編碼。
- 前置正規化的 Transformer 區塊先做 $X + \mathrm{MHA}(\mathrm{LN}(X))$，再做 $U + \mathrm{MLP}(\mathrm{LN}(U))$；注意力混合 token，MLP 轉換每個 token。
- ViT = 切成 patch（步幅為 P 的卷積）+ 位置嵌入 + 類別 token + Transformer 編碼器。它的歸納偏置弱，需要大規模預訓練或強力的訓練配方（DeiT、AugReg）。
- Swin 透過平移視窗與 patch 合併找回局部性與階層結構；CvT、CoAtNet、MobileViT 等混合模型結合了卷積與注意力。
- FlashAttention 讓精確注意力更快，但不改變其二次方的算術量；視窗、token 合併與狀態空間掃描則改變了實際計算的內容。比較機制時一定要對齊預算。
- Vision Mamba 與 VMamba 把選擇性狀態空間應用到影像序列。多條路徑會改變影響傳遞方式，但不自動保證旋轉等變性，也不保證保留所有遠處細節。

練習

1. **Token 預算**。對一張 512×512 的影像，分別計算 patch 大小為 16 與 8 時的 token 數，以及每個頭的注意力分數個數。patch 大小減半時，分數個數增加幾倍？

   提示

   $N = (512/16)^2 = 1024$ 與 $(512/8)^2 = 4096$ 個 token；$N^2 = 1{,}048{,}576$ 與 $16{,}777{,}216$ 個分數。patch 大小減半，N 乘以 4，分數乘以 16。

2. **縮放檢查**。對 $d_k = 4, 64, 1024$，隨機產生各分量為獨立標準常態的 query 與 key。分別量測有無除以 $\sqrt{d_k}$ 時 $q \cdot k$ 的變異數，以及在 100 個 key 上 softmax 的最大權重。不縮放時，softmax 會發生什麼事？

   提示

   原始點積的變異數接近 d_k；縮放後接近 1。不縮放時，在 $d_k = 1024$ 下 softmax 幾乎把所有權重都放在單一個 key 上（近乎 one-hot），流經其他 key 的梯度因此消失。

3. **移除位置資訊**。在 TinyViT 程式碼中，把位置嵌入設為零、拿掉類別 token 改用平均池化，然後在送進區塊之前打亂 patch token 的順序，並在評估模式下比較 logits。解釋結果，並說明這對一個沒有位置資訊的影像分類器意味著什麼。

   提示

   logits 在浮點誤差範圍內相同，因為每個區塊都是排列等變的，而平均池化是排列不變的。記得關掉 dropout（`eval()`），並打亂 token 而不是 patch 內的像素。這樣的模型無法分辨一張臉和用相同 patch 拼湊出來的亂序臉。

4. **交叉注意力的形狀**。在一個文字生成影像的模型中，256 個寬度 320 的影像 token 讀取 77 個寬度 768 的文字 token，使用 8 個寬度 40 的頭。寫出 W_Q、W_K、W_V、單一個頭的分數矩陣以及輸出的形狀。

   提示

   $W_Q \in \mathbb{R}^{320 \times 320}$（8 個頭 × 40），$W_K, W_V \in \mathbb{R}^{768 \times 320}$。每個頭的分數為 256×77。輸出跟隨 query：經過輸出投影後為 256×320。key 與 value 的長度必須彼此相同，而不必與 query 長度相同。

5. **Swin 的算術**。利用成本公式，比較 56×56 的 token 網格、$C = 96$、$M = 7$ 時的全域注意力與視窗注意力。兩種情況各由哪一項主導？

   提示

   $hw = 3136$。全域：$4 \cdot 3136 \cdot 96^2 \approx 1.16 \times 10^8$ 加上 $2 \cdot 3136^2 \cdot 96 \approx 1.89 \times 10^9$，由二次方項主導。視窗：同樣的 1.16×10^8 加上 $2 \cdot 49 \cdot 3136 \cdot 96 \approx 2.95 \times 10^7$，由投影主導。此例中視窗版本約便宜 14 倍。

6. **公平比較**。你有 2,000 張標註影像，以及預訓練好的 CNN、ViT 與視覺 Mamba 骨幹。設計一個實驗來決定要部署哪一個骨幹，並列出必須固定的條件。納入移除混合器的消融實驗，以及解析度／旋轉測試。

提示

先在凍結的特徵上比較線性探測（linear probe），再在驗證集上以對齊的預算、為各模型分別調校學習率來做微調。記錄預訓練資料、輸入解析度、參數量、FLOPs，以及在目標裝置上實測的延遲。拿預訓練的 ViT 去比從頭訓練的 CNN，測到的是預訓練，而不是架構。對掃描骨幹，記錄狀態大小，並測試保留閘控與局部卷積、移除混合器後，優勢是否仍存在。解析度與旋轉測試要使用一致的前處理。

參考文獻

1. A. Vaswani et al., “Attention Is All You Need,” NeurIPS, 2017. arXiv
2. A. Dosovitskiy et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” ICLR, 2021. arXiv
3. S. Khan, M. Naseer, M. Hayat, S. W. Zamir, F. S. Khan and M. Shah, “Transformers in Vision: A Survey,” *ACM Computing Surveys*, 2022. arXiv
4. H. Touvron et al., “Training data-efficient image transformers & distillation through attention,” ICML, 2021. arXiv
5. Z. Liu et al., “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows,” ICCV, 2021. arXiv
6. T. Dao, D. Y. Fu, S. Ermon, A. Rudra and C. Ré, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” NeurIPS, 2022. arXiv
7. PyTorch, `scaled_dot_product_attention` and `TransformerEncoderLayer` documentation. Attention, Encoder layer
8. R. Xu et al., “Visual Mamba: A Survey and New Outlooks,” arXiv:2404.18861, 2024. arXiv
9. W. Yu and X. Wang, “MambaOut: Do We Really Need Mamba for Vision?,” CVPR, 2025. Paper
10. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024, ch. 12. Book
11. K. Han et al., “A Survey on Vision Transformer,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023. DOI
12. Y. Liu et al., “A Survey of Visual Transformers,” *IEEE Transactions on Neural Networks and Learning Systems*, 2024. DOI
13. L. Papa, P. Russo, I. Amerini and L. Zhou, “A Survey on Efficient Vision Transformers: Algorithms, Techniques, and Performance Benchmarking,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024. DOI
14. J. L. Ba, J. R. Kiros and G. E. Hinton, “Layer Normalization,” arXiv:1607.06450, 2016. arXiv
15. R. Xiong et al., “On Layer Normalization in the Transformer Architecture,” ICML, 2020. arXiv
16. J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, 2024. DOI
17. P. Shaw, J. Uszkoreit and A. Vaswani, “Self-Attention with Relative Position Representations,” NAACL, 2018. arXiv

18. M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang and A. Dosovitskiy, “Do Vision Transformers See Like Convolutional Neural Networks?,” NeurIPS, 2021. arXiv
19. H. Wu et al., “CvT: Introducing Convolutions to Vision Transformers,” ICCV, 2021. arXiv
20. Z. Dai, H. Liu, Q. V. Le and M. Tan, “CoAtNet: Marrying Convolution and Attention for All Data Sizes,” NeurIPS, 2021. arXiv
21. D. Bolya et al., “Token Merging: Your ViT But Faster,” ICLR, 2023. arXiv
22. T. Darcet, M. Oquab, J. Mairal and P. Bojanowski, “Vision Transformers Need Registers,” ICLR, 2024. arXiv
23. M. Oquab et al., “DINOv2: Learning Robust Visual Features without Supervision,” *Transactions on Machine Learning Research*, 2024. arXiv
24. M. Dehghani et al., “Scaling Vision Transformers to 22 Billion Parameters,” ICML, 2023. arXiv
25. A. Steiner et al., “How to train your ViT? Data, Augmentation, and Regularization in Vision Transformers,” *Transactions on Machine Learning Research*, 2022. arXiv
26. D. Hendrycks and K. Gimpel, “Gaussian Error Linear Units (GELUs),” arXiv:1606.08415, 2016. arXiv
27. H. Touvron, M. Cord and H. Jégou, “DeiT III: Revenge of the ViT,” ECCV, 2022. arXiv
28. S. Mehta and M. Rastegari, “MobileViT: Light-weight, General-purpose, and Mobile-friendly Vision Transformer,” ICLR, 2022. arXiv
29. T. Dao, “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning,” ICLR, 2024. arXiv
30. L. Zhu et al., “Vision Mamba: Efficient Visual Representation Learning with Bidirectional State Space Model,” ICML, 2024. arXiv
31. Y. Liu et al., “VMamba: Visual State Space Model,” NeurIPS, 2024. arXiv
32. Z. Liu et al., “Swin Transformer V2: Scaling Up Capacity and Resolution,” CVPR, 2022. arXiv
33. A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” COLM, 2024. Author version
34. T. Dao and A. Gu, “Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality,” ICML, PMLR 235, 2024. Paper

ML 05・表示學習與自監督

什麼是表示、為什麼分散式特徵能遷移，以及 SimCLR、MoCo、BYOL、SimSiam、DINO、MAE、JEPA 與 CLIP 如何在沒有標記的情況下學到特徵，並用線性探測與微調誠實評估。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-05-representation-learning/

你將學到

- 什麼是**表示**（representation）、為什麼**分散式**（distributed）表示比 one-hot 編碼更能泛化，以及好的表示應該（和不應該）具備哪些不變性。
- **遷移學習**（transfer learning）如何運作：預訓練一次編碼器，再透過 k-NN、線性探測（linear probe）或微調（fine-tuning）重複使用。
- 主要的**自監督學習**（self-supervised learning）家族如何從無標記資料中造出訓練訊號：對比式（SimCLR、MoCo，以及 InfoNCE 損失函數）、沒有負例的教師–學生法（BYOL、SimSiam、DINO）、冗餘抑制（Barlow Twins、VICReg）、遮罩建模（BEiT、MAE）與特徵預測（I-JEPA、V-JEPA 2）。
- 為什麼「讓兩個視圖一致」可能會**塌縮**（collapse），以及對齊性與均勻性如何把這件事說清楚。
- **CLIP** 如何把圖說當作監督訊號，並做到零樣本（zero-shot）分類。
- 如何誠實地評估一個表示，包括為什麼微調可能輸給線性探測，以及 LP-FT 如何改善。

先看全貌

假設你有一百萬張沒有標記的影像，但真正關心的任務只有 500 個標記樣本。只用這 500 個樣本從頭訓練一個大型網路，等於把幾乎所有可用的資料都丟掉了。**表示學習**把工作拆成兩步。第一步，從一個不需要你的標記的廣泛訊號學出編碼器 $h = f_\theta(x)$。第二步，用手上少量的標記，在 h 之上訓練一個小型的任務頭（head）。

> **用最簡單的話說**
>
> 小孩在學會每種鳥的名字之前，就已經會注意翅膀、鳥嘴、顏色和動作。後來爸媽指著三隻知更鳥和兩隻烏鴉，這樣就夠了：從此小孩就能分辨知更鳥和烏鴉。「注意」這件事是免費學來的，只是靠著看這個世界。表示就是這種可以重複使用的觀察方式，而自監督學習就是「只是看」的那個部分。

為什麼重要：幾乎所有現代視覺系統都從一個預訓練的編碼器出發。影像分類器、物件偵測器、分割模型、影像–文字模型，以及生成模型裡的影像編碼器，都建立在用這種方式學到的表示之上。起點的好壞，往往比任務頭的細節更重要。

本章先談什麼是表示、為什麼遷移有效，接著逐一介紹實務上主流的自監督目標，最後討論如何衡量一個表示到底好不好。本章假設你已經熟悉神經網路（ML 02）、CNN（ML 03）與 Vision Transformer（ML 04）。線性因子模型與自編碼器是學習表示的古典路線，另有專章：ML 05A。

什麼是表示？

白話版。表示就是模型用來描述一個輸入的那串數字。好的表示讓你關心的東西容易讀出來，並把你不關心的東西藏起來。

精確版。編碼器是一個函數 $f_\theta : \mathcal{X} \to \mathbb{R}^D$，把輸入 x（這裡是影像）對應到特徵向量 $h = f_\theta(x)$，其中 θ 是可學習的參數，D 是特徵維度。任務頭 g 把 h 對應到預測，所以完整模型是 $g \circ f_\theta$。如果一個簡單的任務頭（例如線性分類器）只用少量標記樣本就能從 h 解決任務，我們就說 h 是這個任務的好表示。Bengio、Courville 與 Vincent 的綜述 [1] 是這個觀點的經典陳述；它列出讓表示有用的「先驗」（priors），例如平滑性、多個解釋因子、由淺到深逐漸抽象的特徵階層，以及跨任務共享的因子。

局部表示與分散式表示

類別最簡單的表示是 **one-hot**：每個概念一個單元，且恰好只有一個單元是開啟的。它容易讀，卻無法泛化。知道「紅色汽車」完全不會告訴你任何關於「紅色卡車」的事，因為它們沒有共同的開啟單元。**分散式**表示則用許多特徵來描述每個輸入，而每個特徵又被許多輸入共享：「紅」、「藍」、「汽車」、「卡車」。於是「紅色卡車」就能重複使用從紅色汽車學來的「紅」這個單元。

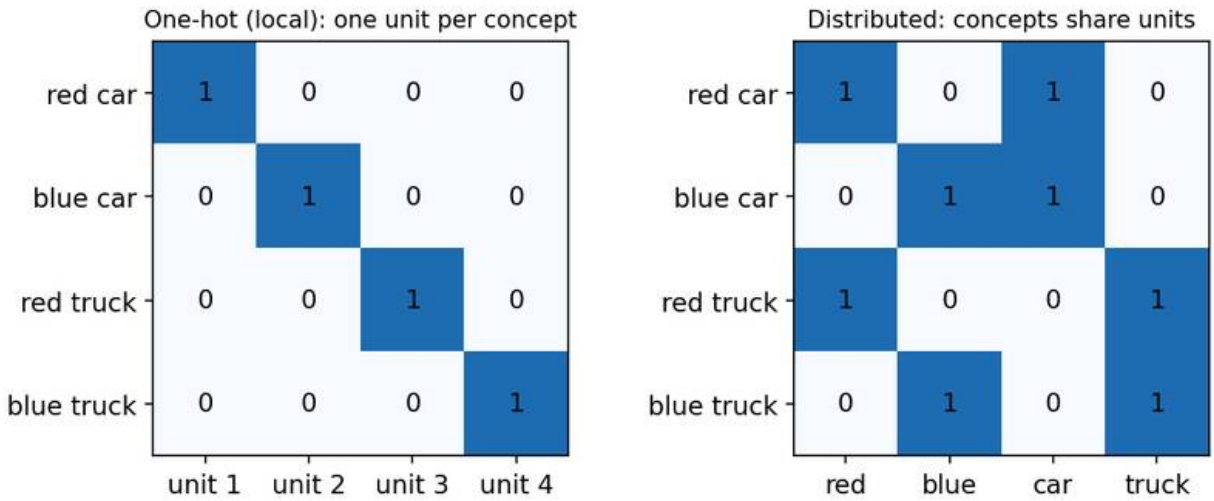


圖 *1* — 四個概念的局部（*one-hot*）編碼與分散式編碼。在分散式編碼下，像紅色卡車這樣的新組合可以重複使用從其他樣本學到的特徵。

計數論證很驚人。用 D 個二元特徵，one-hot 編碼只能分辨 D 個概念，而分散式編碼原則上能分辨 2^D 種組態 [1]。深度網路會自動學出分散式編碼：每個隱藏單元對許多輸入有反應，每個輸入也會啟動許多單元。表示學習大部分的實際價值都來自這種共享。

不變性與等變性

有用的表示會保留與任務相關的差異，並壓抑會造成干擾的差異。

- 若 $f(T(x)) = f(x)$，特徵就對轉換 T 具有**不變性**（invariance）：把一隻貓平移兩個像素，不應該改變「這是一隻貓」。
- 若 $f(T(x)) = T'(f(x))$，特徵就具有**等變性**（equivariance），也就是以可預測的方式跟著改變：平移那隻貓，牠的分割遮罩也應該平移同樣的距離。

「有用」永遠取決於下游任務。顏色不變性有助於物體辨識，卻會傷害水果成熟度的估計。旋轉不變性對紋理有幫助，卻會搞混 6 和 9。密集預測（分割、深度）需要空間上等變的區塊（patch）特徵，而不是每張影像一個不變的向量。後面會看到，自監督目標主要是透過所使用的**資料增強**（data augmentation）來選擇不變性，所以這個選擇是建模決策，而不是細節。

學習距離：度量學習

PCA 在ML 01 介紹過，並在ML 05A 深入討論，它保留變異最大的方向。但變異大的方向可能來自光照，而不是物體身分。監督學習依照標記來塑造特徵；**度量學習**（metric learning）則依照應該配對的樣本來塑造距離。給定錨點（anchor）a、正例 p（同一身分）與負例 n（不同身分），三元組損失（triplet loss）是

$$\mathcal{L}_{\text{triplet}} = \max\left(0,\ \|h_a - h_p\|^2 - \|h_a - h_n\|^2 + m\right),$$

其中 h_a, h_p, h_n 是三個特徵向量，$m > 0$ 是邊界（margin）。當正例比負例近至少 m 時，損失為零。太容易的負例損失為零，什麼也學不到，所以挑選有資訊量的（「困難的」）負例很重要。下面的對比式自監督學習，就是一種用資料增強而不是標記來產生正例的度量學習。

遷移學習：預訓練一次，重複使用很多次

白話版。在一個大任務上訓練網路，保留它的特徵擷取器，再拿去用在小任務上。你可以凍結特徵、只訓練新的任務頭，也可以讓整個網路繼續學習。

精確版。預訓練在來源資料集或目標上找出編碼器參數 $\theta^\star$。遷移到一個有標記集合 $\{(x_i, y_i)\}_{i=1}^{n}$ 的目標任務時，有三種常見做法：

1. k **最近鄰（k-NN）：**不訓練。用測試特徵 $f_{\theta^\star}(x)$ 最相似的 k 個訓練特徵的標記來分類。
2. **線性探測：**凍結 $\theta^\star$，在 $h = f_{\theta^\star}(x)$ 上擬合線性分類器 $\hat{y} = \text{softmax}(Wh + b)$，其中 W 與 b 是唯一被訓練的參數。
3. **微調：**從 $\theta^\star$ 出發，連同任務頭一起更新編碼器，通常使用較小的學習率。

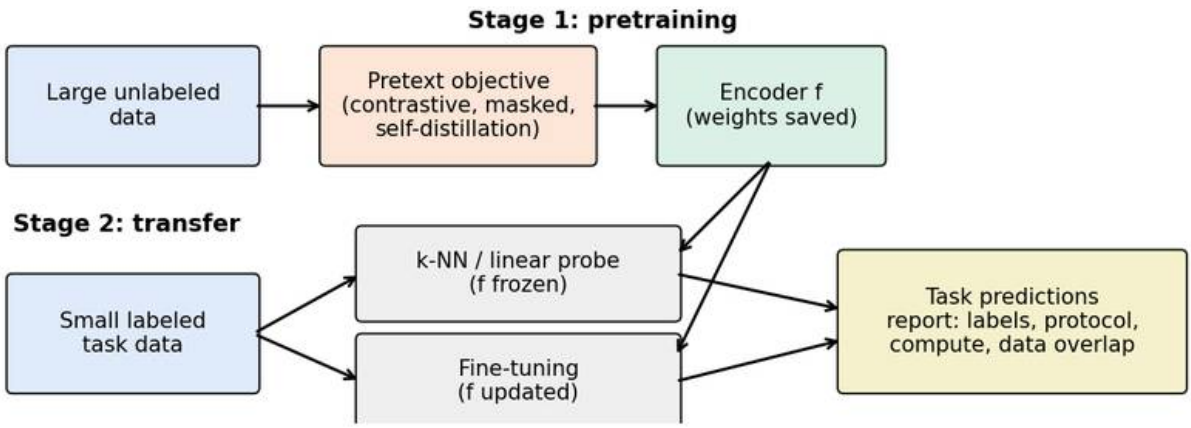


圖 2 — 表示學習的兩階段模式。預訓練昂貴但只做一次；遷移便宜且每個任務各做一次。

遷移為什麼會有效？Yosinski 等人 [2] 在 ImageNet 訓練的 CNN 上逐層量測。第一層特徵（很像 Gabor 濾波器的邊緣與顏色偵測器）是通用的：在任務之間遷移得很好。越高層的特徵越專屬於來源任務，而且來源任務與目標任務差異越大，可遷移性就越低。他們也發現，從遷移來的特徵初始化再微調，即使在目標任務上，也比從隨機初始化訓練有更好的泛化。

多年來，預訓練的預設來源是有監督的 ImageNet 分類。可是標記很昂貴，而且像「狗」這樣的標記，完全沒有告訴網路那隻狗的姿勢、草地或光線。自監督學習讓資料自己監督自己，移除了標記的瓶頸。

自監督學習：讓資料自己產生標記

白話版。發明一個答案可以從資料本身算出來的任務：「這兩個裁切來自同一張照片嗎？」、「這個遮罩下面原本是什麼？」、「哪一句圖說配這張影像？」。解這種**代理任務**（pretext task），會迫使網路學到有用的特徵。

精確版。自監督目標是只從無標記輸入（或像影像–圖說這種天然配對的資料）計算出來的損失 $\mathcal{L}(\theta; x)$。Liu 等人 [3] 與 Gui 等人 [4] 的綜述，以及 Balestriero 等人實用的「食譜」[5]，把這個領域整理成下表的幾個家族：

家族	訓練訊號	代表方法	主要風險
對比式	拉近正配對，推開其他樣本	SimCLR、MoCo	需要大量負例；假負例
自蒸餾（無負例）	學生在另一個視圖上模仿教師	BYOL、SimSiam、DINO	缺少不對稱或中心化時會塌縮
冗餘抑制	一致性加上去相關、非常數的特徵維度	Barlow Twins、VICReg	正則項帶來額外超參數
遮罩建模	預測被遮住的像素或符元	BEiT、MAE	特徵可能偏重低階細節
特徵預測	預測被遮住區域的特徵	I-JEPA、V-JEPA 2	需要像蒸餾一樣的防塌縮設計
語言監督	讓影像與其圖說配對	CLIP、SigLIP	繼承網路資料的偏誤與雜訊

這些家族是學習目標，而不是架構：每一種都可以用來訓練 CNN 或 Transformer。現代系統常

常結合好幾種。以下逐一介紹。

對比式學習與 InfoNCE 損失

白話版。讓同一張影像的兩個增強版本彼此一致，並且和批次中其他所有影像不一致。這是一道選擇題：「在這 N 個候選中，哪一個是我的搭檔？」

InfoNCE

van den Oord、Li 與 Vinyals 在對比預測編碼（Contrastive Predictive Coding, CPC）[6] 中提出 **InfoNCE** 損失。取一個查詢 q、一個「屬於」它的正例鍵 k^+，以及 $N-1$ 個負例鍵 $k_1^-, \dots, k_{N-1}^-$。給定相似度分數 $s(\cdot,\cdot)$ 與溫度 $\tau > 0$，

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp\left(s(q,k^+)/\tau\right)}{\exp\left(s(q,k^+)/\tau\right) + \sum_{j=1}^{N-1} \exp\left(s(q,k_j^-)/\tau\right)}.$$

這正是一個 N 類分類問題的 softmax 交叉熵，正確類別就是正例。由此得到兩個結果。

- **互資訊的下界**。CPC 證明 $I(q;k^+) \geq \log N - \mathcal{L}_{\text{InfoNCE}}$，其中 I 是互資訊（mutual information）。最小化損失會提高這個下界，而負例越多（N 越大）下界可以越高。實務上這個界很鬆，特徵品質也高度依賴編碼器與資料增強，所以請把它當成動機，而不是特徵為什麼好的解釋。
- **溫度控制難度**。除以較小的 τ 會讓 softmax 更尖銳，使損失集中在與查詢最相似的負例（困難負例）上。

SimCLR

SimCLR [7] 是最簡單形式的 InfoNCE。取一批 B 張影像，各產生兩個增強視圖（隨機裁切並縮放、顏色扭曲、模糊），共 $2B$ 個視圖。骨幹網路 f 把每個視圖對應到 h，一個小型 MLP **投影頭**（projection head）g 再把 h 對應到 $z = g(h)$，相似度用餘弦相似度 $s(z_i, z_j) = z_i^\top z_j / (\|z_i\|\|z_j\|)$。對於搭檔為 j 的視圖 i，

$$\ell_{i,j} = -\log \frac{\exp(z_i^\top z_j/\tau)}{\sum_{k \neq i} \exp(z_i^\top z_k/\tau)},$$

其中 z 已經做過 ℓ_2 正規化，總和遍及其他所有 $2B-1$ 個視圖。批次裡其他每張影像都是負例，因此不需要記憶庫，但這個方法受益於大批次。總損失對每個正配對的兩種順序取 $\ell_{i,j}$ 的平均；論文稱之為 NT-Xent 損失（normalized temperature-scaled cross-entropy）。

SimCLR 論文有三個值得記住的發現 [7]。資料增強的組合非常重要，其中隨機裁切加上顏色扭曲特別關鍵。投影頭有幫助：投影頭之前的特徵（h）比之後的（z）更能遷移，所以預訓練後會丟掉投影頭。而且對比式學習比監督學習更受益於大批次與更長的訓練。

程式：檢查損失與塌縮時的值

這段獨立的 PyTorch 範例用合成的配對向量驗證損失、自身遮罩與梯度。它不訓練編碼器。

```
import torch
from torch.nn import functional as F

torch.manual_seed(0)
B, D = 8, 16
base = torch.randn(B, D)
views = torch.cat([base + .1*torch.randn_like(base),
                   base + .1*torch.randn_like(base)]).requires_grad_()
z = F.normalize(views, dim=-1)
logits = z @ z.T / .2
self_mask = torch.eye(2*B, dtype=torch.bool)
logits = logits.masked_fill(self_mask, float("-inf"))
positive = (torch.arange(2*B) + B) % (2*B)
loss = F.cross_entropy(logits, positive)
loss.backward()
assert torch.isfinite(views.grad).all()
print("paired loss:", round(loss.item(), 4))

collapsed = F.normalize(torch.ones(2*B, D), dim=-1)
scores = (collapsed @ collapsed.T / .2).masked_fill(self_mask, float("-inf"))
collapsed_loss = F.cross_entropy(scores, positive)
print("collapsed loss:", round(collapsed_loss.item(), 4))
print("expected:", round(torch.log(torch.tensor(2*B-1.)).item(), 4))
```

它印出配對損失 0.1423 與塌縮損失 2.7081，正好等於 $\log 15$。如果所有表示都相同，正例得到的機率是 $1/(2B-1)$，損失就是 $\log(2B-1)$，也就是均勻亂猜的最差值。因此對比式的鑑別讓完全塌縮成為一個很差的解，雖然最佳化與負例選擇仍然可能很困難。

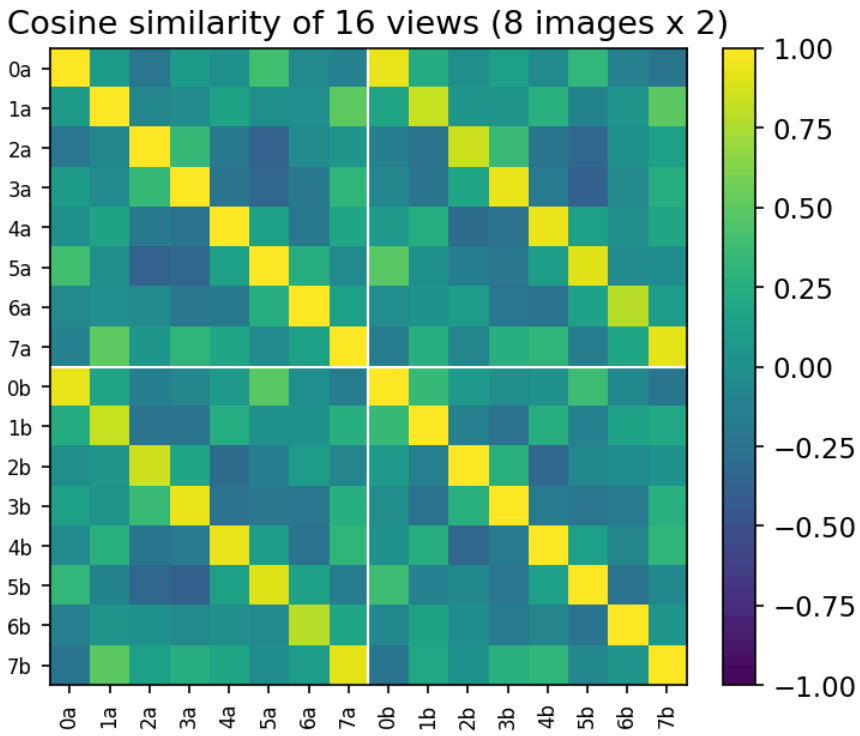


圖 *3* — 用本章稍後的小型 *SimCLR* 編碼器訓練後，*16* 個視圖（*8* 張數字影像，各兩個增強）之間的餘弦相似度。明亮的副對角線就是正配對；*InfoNCE* 把它們往上推，把其他所有元素往下壓。

MoCo：負例佇列與動量編碼器

InfoNCE 偏好大量負例，但 SimCLR 只能從當前批次取得負例。**動量對比**（Momentum Contrast, MoCo）[8] 用兩個想法把負例數量和批次大小脫鉤。

1. **佇列**。最近幾個批次編碼出來的鍵被存進長度為 K 的先進先出佇列，重複當作負例使用。無論批次多小，查詢都能看到 K 個負例。
2. **動量編碼器**。佇列裡的鍵是由稍早版本的網路產生的。如果鍵編碼器變化太快，新舊鍵就會不一致。因此 MoCo 用另一個編碼器計算鍵，它的參數 θ_k 緩慢地跟隨查詢編碼器的參數 θ_q：

$$\theta_k \leftarrow m\,\theta_k + (1-m)\,\theta_q,$$

其中 $m \in [0,1)$ 是動量，接近 1（論文預設 0.999）。只有 θ_q 接收梯度。這種**指數移動平均**（exponential moving average, EMA）教師的想法，在 BYOL、DINO 與 JEPA 中都會再次出現。

```
import torch
from torch.nn import functional as F

torch.manual_seed(0)
B, D, K = 8, 16, 32          # batch, embedding size, queue length
```

```
# --- MoCo-style InfoNCE: one positive key per query, negatives from a queue
q = F.normalize(torch.randn(B, D), dim=1)                 # queries (online encoder)
k = F.normalize(q + 0.1 * torch.randn(B, D), dim=1)       # keys (momentum encoder)
queue = F.normalize(torch.randn(K, D), dim=1)             # keys from past batches
l_pos = (q * k).sum(1, keepdim=True)                      # B x 1
l_neg = q @ queue.T                                       # B x K
logits = torch.cat([l_pos, l_neg], dim=1) / 0.07
loss_moco = F.cross_entropy(logits, torch.zeros(B, dtype=torch.long))
queue = torch.cat([k, queue])[:K]                         # enqueue new, drop oldest
print("MoCo logits:", tuple(logits.shape), "loss:", round(loss_moco.item(), 3))

# momentum (EMA) update of the key encoder's parameters
theta_q, theta_k, m = torch.randn(5), torch.zeros(5), 0.999
theta_k = m * theta_k + (1 - m) * theta_q
print("key params moved by:", round((theta_k.norm() / theta_q.norm()).item(), 4))
```

每個查詢面對一個 $1 + K$ 類的分類問題，正確答案永遠在第 0 欄，所以目標全為零。輸出顯示 logits 的形狀是 `(8, 33)`，而 EMA 一步只讓鍵編碼器往查詢編碼器移動了 0.1%（從零開始、走一步）。實際實作中，q 與 k 來自兩個編碼器分別作用在同一批影像的兩個增強版本上。

對齊性與均勻性

Wang 與 Isola [9] 從幾何角度清楚說明對比式損失在單位球面上做了什麼。在正規化特徵下，損失在漸近意義上最佳化兩個性質：

$$\mathcal{L}_{\text{align}} = \mathbb{E}_{(x,x^+)} \|f(x) - f(x^+)\|_2^2, \qquad \mathcal{L}_{\text{unif}} = \log \mathbb{E}_{x,y}\, e^{-t\|f(x)-f(y)\|_2^2},$$

其中 (x, x^+) 是正配對，x, y 是獨立樣本，$t > 0$（常用 $t = 2$）。正配對被映射到相近的點時，**對齊性**（alignment）就低。特徵在球面上分散開來時，**均勻性**（uniformity）就低，也就是盡可能保留輸入的資訊。塌縮的編碼器有完美的對齊性（0）和最差的均勻性（0）；有用的編碼器兩者都要小。

```
import torch
from torch.nn import functional as F

def alignment(z1, z2):            # mean squared distance between positive pairs
    return (z1 - z2).square().sum(1).mean()

def uniformity(z, t=2.0):         # log of mean Gaussian potential over all pairs
    return torch.pdist(z).square().mul(-t).exp().mean().log()

torch.manual_seed(0)
n, D = 512, 16
collapsed = F.normalize(torch.ones(n, D), dim=1)
spread = F.normalize(torch.randn(n, D), dim=1)
noisy_view = F.normalize(spread + 0.1 * torch.randn(n, D), dim=1)
print("collapsed: align", alignment(collapsed, collapsed).item(),
      "uniform", round(uniformity(collapsed).item(), 3))
```

```
print("spread:    align", round(alignment(spread, noisy_view).item(), 3),
      "uniform", round(uniformity(spread).item(), 3))
```

塌縮的集合得到對齊性 0.0、均勻性 0.0；分散的集合大約是 0.138 與 −3.51。圖 4 追蹤下面訓練數字編碼器時的這兩個量。在第 0 步，隨機網路把每個視圖都映射到幾乎同一點，所以對齊性接近零只是因為一切都塌縮了。訓練先把特徵分散開來（均勻性急遽下降），再逐步把正例拉回彼此身邊。

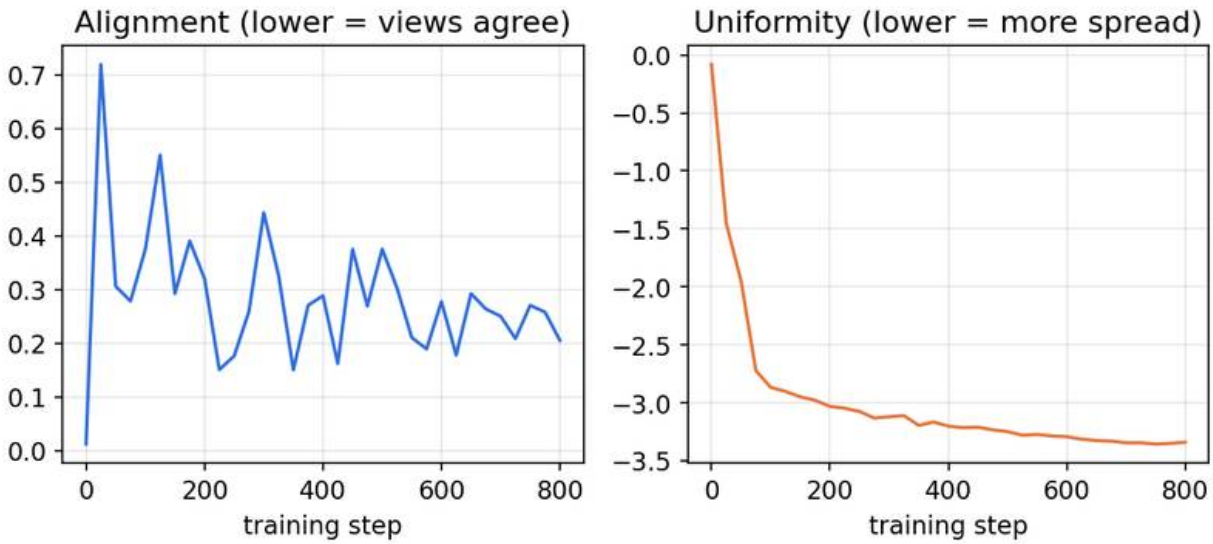


圖 *4* — 在 *8×8* 數字上以 *SimCLR* 訓練時，投影特徵的對齊性與均勻性。單獨的低對齊性沒有意義（常數編碼器也有）；有用的狀態是低對齊性與低均勻性同時成立。

實作範例：在 8×8 數字上跑 SimCLR

白話版。完全不使用標記，在 scikit-learn 的數字資料集上訓練一個迷你 SimCLR 編碼器，再檢查它的特徵能不能讓最近鄰分類器更穩健。

這裡的資料增強是最多一個像素的隨機平移、隨機亮度變化與像素雜訊。因此我們預期學到的特徵對小平移大致不變，這是原始像素做不到的。評估時，每一種特徵都使用同一組有標記的訓練集，並分別在乾淨的數字與最多平移一個像素的數字上測試。

```
import copy
import numpy as np
import torch
from torch import nn
from torch.nn import functional as F
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
```

```
torch.manual_seed(0); np.random.seed(0); torch.set_num_threads(1)
d = load_digits()
Xtr, Xte, ytr, yte = train_test_split(d.data / 16., d.target, test_size=.3,
                                      random_state=0, stratify=d.target)
Xtr_t = torch.tensor(Xtr, dtype=torch.float32)

def augment(x):                       # the augmentations define the invariances
    img = F.pad(x.view(-1, 1, 8, 8), (1, 1, 1, 1))
    i, j = np.random.randint(0, 3, size=2)               # shift by up to 1 px
    img = img[:, :, i:i + 8, j:j + 8]
    img = img * (0.7 + 0.6 * torch.rand(len(img), 1, 1, 1))  # brightness
    return (img + 0.1 * torch.randn_like(img)).reshape(len(x), 64)  # noise

backbone = nn.Sequential(nn.Unflatten(1, (1, 8, 8)),
                         nn.Conv2d(1, 32, 3, padding=1), nn.ReLU(),
                         nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(),
                         nn.AdaptiveAvgPool2d(2), nn.Flatten())  # h: 256-d
random_backbone = copy.deepcopy(backbone)                    # baseline
head = nn.Sequential(nn.ReLU(), nn.Linear(256, 64))          # projection g
opt = torch.optim.Adam([*backbone.parameters(), *head.parameters()], lr=1e-3)

def nt_xent(z1, z2, tau=0.2):
    z = F.normalize(torch.cat([z1, z2]), dim=-1)
    n = len(z1)
    logits = (z @ z.T / tau).fill_diagonal_(float("-inf"))
    return F.cross_entropy(logits, (torch.arange(2 * n) + n) % (2 * n))

for step in range(800):                                      # about a minute on CPU
    x = Xtr_t[torch.randint(len(Xtr_t), (256,))]
    loss = nt_xent(head(backbone(augment(x))), head(backbone(augment(x))))
    opt.zero_grad(); loss.backward(); opt.step()

def shift(x, dy, dx):                    # move one 8x8 image, fill with zeros
    img = np.zeros((10, 10)); img[1:9, 1:9] = x.reshape(8, 8)
    return img[1 - dy:9 - dy, 1 - dx:9 - dx].ravel()

moves = np.random.default_rng(0).integers(-1, 2, size=(len(Xte), 2))
Xte_shift = np.stack([shift(x, *m) for x, m in zip(Xte, moves)])
enc = lambda net: (lambda A: net(torch.tensor(A, dtype=torch.float32)).detach().numpy())
pca = PCA(32, random_state=0).fit(Xtr)
for name, f in [("raw pixels", lambda A: A), ("PCA-32", pca.transform),
                ("random CNN", enc(random_backbone)), ("SimCLR CNN", enc(backbone))]:
    knn = KNeighborsClassifier(5, metric="cosine").fit(f(Xtr), ytr)
    print(f"{name:11s} clean {knn.score(f(Xte), yte):.3f}  "
          f"shifted {knn.score(f(Xte_shift), yte):.3f}")
```

在我們的機器上，輸出為：

```
raw pixels  clean 0.978  shifted 0.524
PCA-32      clean 0.978  shifted 0.507
random CNN  clean 0.974  shifted 0.524
```

```
SimCLR CNN  clean 0.987  shifted 0.919
```

確切數字會因 PyTorch 版本與硬體略有不同，重點是模式。在乾淨的數字上，原始像素已經非常好：這個資料集很簡單，花俏的編碼器幫助不大。在平移過的數字上，原始像素、PCA 以及同架構但未訓練的 CNN 都掉到 50% 左右，而自監督特徵保住了大部分的準確率。網路學到了我們透過資料增強要求的不變性，而且僅止於此：如果改用旋轉過的數字測試（資料增強從未出現過旋轉），就不應期待同樣的穩健性。

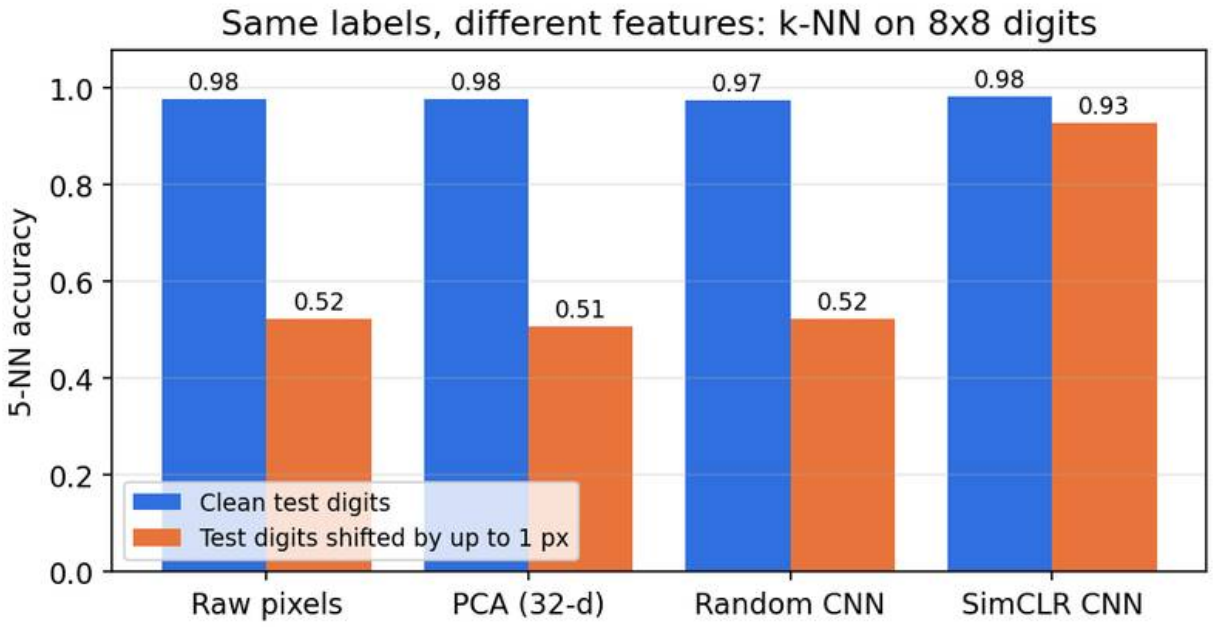


圖 *5* — 相同的標記、相同的分類器、不同的特徵。繪圖腳本使用另一串亂數，所以數字與上面印出的結果最多相差 *0.01*。隨機 *CNN* 排除了「光靠卷積就能帶來穩健性」這個解釋。

沒有負例：為什麼只求一致會塌縮

白話版。如果唯一的指令是「讓兩個輸出一致」，那麼對每張影像都輸出同一個常數就完美達成了。沒有負例的方法必須靠別的機制阻止這件事。

對兩個視圖 v_1, v_2 最小化 $\|f_\theta(v_1) - f_\theta(v_2)\|^2$，有一個平凡解 $f_\theta(x) = c$，其中 c 是常數。光是一致，並不要求特徵攜帶任何資訊。**完全塌縮**把所有東西映射到同一點；**維度塌縮**（dimensional collapse）比較隱微：特徵會變化，但只沿著少數幾個方向，浪費了 D 維中的大部分。

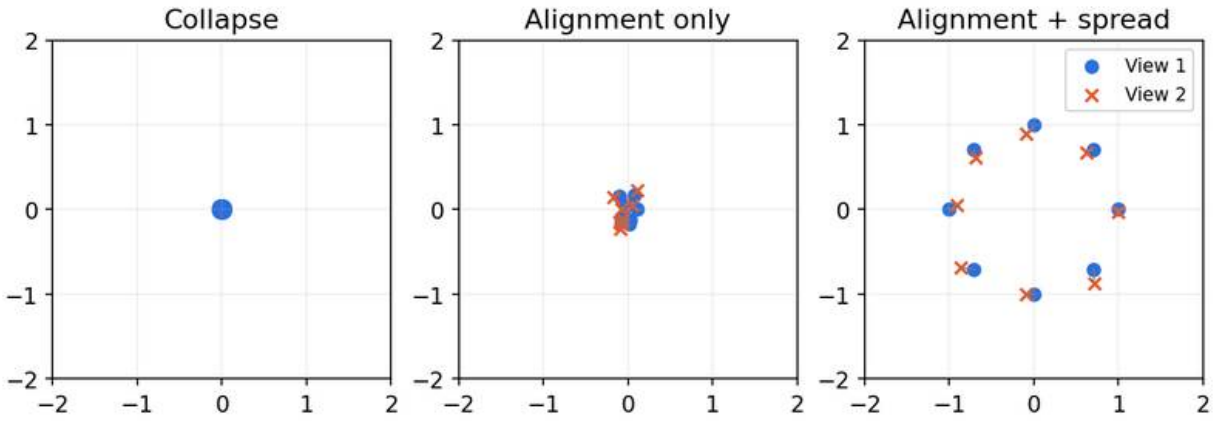


圖 6 — 塌縮、只有對齊沒有分散，以及對齊且分散的示意圖。這些點是刻意構造的說明，不是已發表模型學到的嵌入。

BYOL：線上網路追逐緩慢的目標

BYOL [10] 使用兩個網路。**線上**（online）網路（編碼器、投影器，再加一個**預測器** q_ϕ）被訓練去預測**目標**（target）網路對同一張影像另一個視圖的投影。目標網路不靠梯度訓練，而是線上權重的 EMA，和 MoCo 一樣：

$$\theta_{\text{target}} \leftarrow m\,\theta_{\text{target}} + (1-m)\,\theta_{\text{online}}.$$

損失是正規化後的預測與正規化後的目標投影之間的平方距離，並對兩個視圖做對稱化。目標輸出被當作常數（**停止梯度**，stop-gradient）。沒有負例，BYOL 實務上卻不會塌縮。不對稱性（一邊有預測器，另一邊有停止梯度與 EMA）以及完整的訓練配方都很重要；光有 EMA 並不能證明不可能塌縮。

SimSiam：停止梯度是關鍵成分

Chen 與 He 的 SimSiam [11] 把 BYOL 再進一步精簡：兩個視圖共用一個編碼器，一個分支上有預測器 h，**沒有**動量編碼器、**沒有**負例，也不需要大批次。令 $p_1 = h(f(v_1))$、$z_2 = f(v_2)$，

$$\mathcal{L} = \tfrac{1}{2}\,\mathcal{D}(p_1, \text{sg}(z_2)) + \tfrac{1}{2}\,\mathcal{D}(p_2, \text{sg}(z_1)), \qquad \mathcal{D}(p, z) = -\frac{p^\top z}{\|p\|\,\|z\|},$$

其中 sg 是停止梯度，$\mathcal{D}$ 是負的餘弦相似度。論文實驗顯示，拿掉停止梯度會讓模型很快塌縮，而預測器也是必要的。作者把這個方法詮釋為一種交替式、類似期望最大化（EM）的最佳化。實務上的教訓是：非常小的架構不對稱，就決定了這些方法是在學習還是在塌縮。

DINO：帶中心化與銳化的自蒸餾

DINO [12] 把一致性問題變成帶軟標記的分類。教師與學生都透過帶溫度的 softmax，在 K 個「原型」維度上輸出一個機率向量。看到視圖 x' 的學生，被訓練去符合教師在另一個視圖 x 上的

分布：

$$\mathcal{L} = -\sum_{k=1}^{K} P_t(x)_k \, \log P_s(x')_k, \qquad P_t(x) = \mathrm{softmax}\Big(\frac{g_t(x) - c}{\tau_t}\Big),$$

其中 g_t 是教師網路（學生的 EMA），c 是教師輸出的移動平均（**中心化**，centering），τ_t 是較低的教師溫度（**銳化**，sharpening）。中心化防止某一維獨大；銳化防止輸出變成均勻分布。兩者各自單獨使用都會以不同方式塌縮；合在一起才會平衡。DINO 也使用**多重裁切**（multi-crop）：教師只看大的全域裁切，學生還會看到小的局部裁切，因此學生必須從局部證據推斷全域內容。搭配 Vision Transformer 時，DINO 的注意力圖在完全沒有分割標記的情況下就能凸顯物體邊界，這點很受矚目。

Barlow Twins 與 VICReg：直接規範統計量

另一條路是直接在損失中禁止塌縮。**Barlow Twins** [13] 計算兩個視圖（在批次上標準化後）嵌入之間的 $D \times D$ 互相關矩陣 C，並把它推向單位矩陣：

$$\mathcal{L}_{\mathrm{BT}} = \sum_i (1 - C_{ii})^2 + \lambda \sum_i \sum_{j \neq i} C_{ij}^2,$$

第一項讓每一維在兩個視圖間一致，第二項（權重 $\lambda > 0$）讓不同維度去相關，避免攜帶重複的資訊。**VICReg** [14] 把同樣的想法拆成三個有名字的項：**不變性**（兩個嵌入間的均方距離）、**變異數**（用 hinge 損失讓每一維的標準差保持在門檻之上，直接禁止常數輸出）與**共變異數**（懲罰非對角的共變異數元素）。「沒有負例」不代表「沒有防塌縮設計」；這些方法只是把設計放進損失，而不是放進架構。

遮罩建模：從填空中學習

白話版。把輸入的一部分藏起來，預測缺了什麼。BERT 就是這樣學會語言的；只要選對遮住的比例，這招對影像也有效。

BEiT [15] 把 BERT 式的預訓練帶進 Vision Transformer。影像被切成區塊，部分區塊被遮住，網路為被遮住的位置預測離散的**視覺符元**（visual token），這些符元來自另外訓練好的影像符元化器（tokenizer）。

MAE [16] 則直接預測原始像素。它遮住很大比例的區塊，只把可見的區塊送進編碼器，再用一個輕量的解碼器重建缺失的像素。令被遮住的區塊索引集合為 $\mathcal{M}$，

$$\mathcal{L}_{\mathrm{mask}} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \|\hat{x}_i - x_i\|^2,$$

其中 x_i 是區塊 i 的真實像素向量，$\hat{x}_i$ 是預測值。MAE 的設計是刻意的。高遮罩比例（預設 75%）讓任務夠難，網路無法只靠鄰居內插。編碼器只處理可見區塊，降低預訓練成本。用每個目標區塊自身的平均與變異數做正規化，可以改善特徵。預訓練完成後，解碼器就被丟掉。

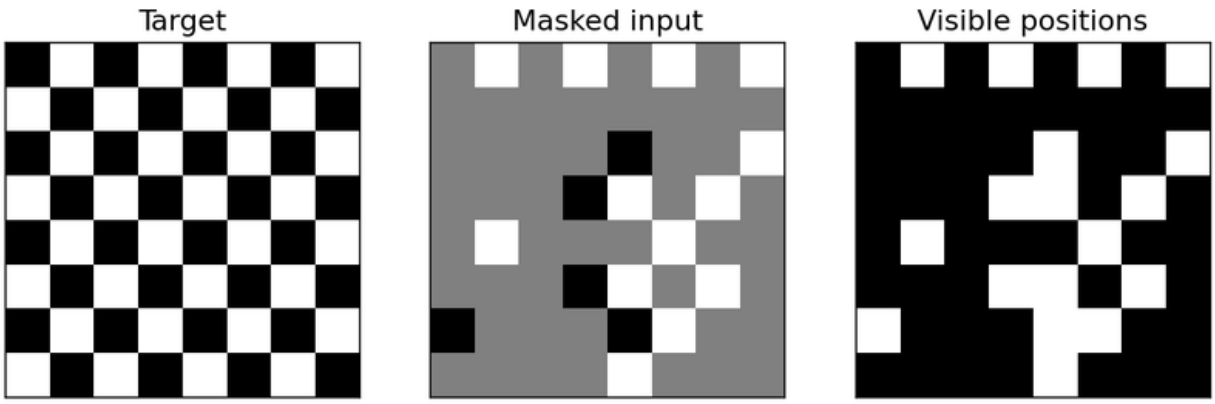


圖 7 — 遮罩預測從輸入本身造出目標。棋盤格只是說明遮罩操作，並不代表語意理解。

重建不等於生成。從可見像素預測被遮住像素的模型，並沒有被訓練成從零開始取樣出完整影像。低重建損失也不保證得到最好的語意特徵：背景紋理可能主導像素誤差。遮罩建模的特徵在線性探測下通常不如對比式或蒸餾式特徵，但微調之後很強，這也是為什麼評估方式必須講清楚。

預測特徵而不是像素：JEPA

白話版。與其把每個不確定的像素都畫出來，不如預測被遮住區域的描述。

I-JEPA [17]（影像聯合嵌入預測架構，Image Joint-Embedding Predictive Architecture）編碼一個可見的**脈絡**（context）區塊，並從中預測數個**目標**（target）區塊的編碼器特徵。目標編碼器是脈絡編碼器的 EMA，預測器則接收脈絡特徵以及目標的位置。簡化後的目標函數是

$$\mathcal{L}_{\text{pred}} = \left\| g_\phi\big(f_\theta(x_{\text{context}}), \text{positions}\big) - \text{sg}\left(f_{\bar{\theta}}(x_{\text{target}})\right) \right\|^2,$$

其中 f_θ 是脈絡編碼器，$f_{\bar{\theta}}$ 是 EMA 目標編碼器，g_ϕ 是預測器，sg 是停止梯度。這個式子只是概括想法；遮罩策略（大的、語意尺度的目標區塊）與架構都是方法的一部分，不是可有可無的細節。

V-JEPA 2 [18] 把特徵預測大規模延伸到影片。它的報告把無動作的影片預訓練，與之後用少量機器人資料訓練、用於規劃的動作條件模型區分開來。預測未來的潛在狀態、辨識動作、生成逼真的未來畫面，是不同的能力；一個「世界模型」的主張應該說清楚它指的是哪一種。

以語言作為監督：CLIP

白話版。網路上到處都是附有圖說的影像。訓練一個影像編碼器和一個文字編碼器，讓每張影像落在自己的圖說旁邊。之後你只要把類別名稱寫成句子，就能分類影像。

CLIP [19] 用從網路收集的影像–文字配對來訓練。在一批 N 個配對中，影像編碼器產生正規化嵌入 $u_1, \dots, u_N$，文字編碼器產生 $v_1, \dots, v_N$。$N \times N$ 的相似度矩陣 $S_{ij} = u_i^\top v_j / \tau$（$\tau$ 是可學習的溫度）的對角線就是正確配對。損失是兩個 InfoNCE 損失的平均，一個沿著列（哪句圖說配這張影像？），一個沿著行（哪張影像配這句圖說？）：

$$\mathcal{L}_{\text{CLIP}} = \frac{1}{2}\Big[\frac{1}{N}\sum_i -\log\frac{e^{S_{ii}}}{\sum_j e^{S_{ij}}} + \frac{1}{N}\sum_j -\log\frac{e^{S_{jj}}}{\sum_i e^{S_{ij}}}\Big].$$

零樣本分類隨之而來。為每個類別嵌入一句提示，例如「a photo of a dog」，再把影像分到提示嵌入最相似的類別。不需要任何任務專屬的標記，不過提示的措辭會影響準確率。

SigLIP [20] 把 softmax 換成對每一個影像–文字配對各自獨立的 **sigmoid** 損失：矩陣中的每個元素都變成一個是非題：「這兩個配對嗎？」這樣就不需要對整個批次做正規化，簡化了大規模分散式訓練。

```
import torch
from torch.nn import functional as F

torch.manual_seed(0)
D = 16

# --- CLIP-style symmetric loss on N matched image-text pairs
N = 6
img = F.normalize(torch.randn(N, D), dim=1)
txt = F.normalize(img + 0.3 * torch.randn(N, D), dim=1)   # caption i matches image i
logit_scale = torch.tensor(1 / 0.07)
sim = logit_scale * img @ txt.T                           # N x N, diagonal = matches
labels = torch.arange(N)
loss_clip = (F.cross_entropy(sim, labels) + F.cross_entropy(sim.T, labels)) / 2
print("CLIP loss:", round(loss_clip.item(), 3))

# zero-shot classification: embed one prompt per class, pick the most similar
class_prompts = txt[:3]                                   # stand-ins for "a photo of a
 ↪  {class}"
pred = (img @ class_prompts.T).argmax(1)
print("zero-shot predictions for images 0-2:", pred[:3].tolist())

# SigLIP-style alternative: an independent yes/no decision for every pair
t, b = 10.0, -10.0                                        # learnable in practice
signs = 2 * torch.eye(N) - 1                              # +1 on matches, -1 elsewhere
loss_sig = -F.logsigmoid(signs * (t * img @ txt.T + b)).sum() / N
print("sigmoid loss:", round(loss_sig.item(), 3))
```

合成的圖說是影像嵌入加上雜訊的副本，所以 CLIP 損失很小（0.054），零樣本步驟也找回了正確的「類別」`[0, 1, 2]`。在真正的 CLIP 中，兩個編碼器分別是 Vision Transformer 或 CNN 與文字 Transformer，而困難的部分在資料：圖說描述了什麼、遺漏了什麼，決定了表示知道什麼。sigmoid 損失的值（3.473）不能和 softmax 損失直接比較；它的單位不同，還多了偏置項 b。

DINOv2 與 DINOv3：把特徵當成產品

白話版。只要有足夠的精選資料與算力，預訓練就能產出一個可以凍結、並重複用在許多不同任務上的編碼器，從分類到深度估計都行。

DINOv2 [21] 結合影像層級的自蒸餾（如 DINO）與區塊層級的遮罩目標，並且非常重視**資料策展**（data curation）：從未經整理的影像中，透過與精選影像比對檢索，建出一個大型、多樣且去除重複的預訓練集。成果是一系列凍結後就能搭配簡單任務頭、全域與區塊特徵都好用的編碼器。2025 年發表的 DINOv3 [22] 進一步擴大規模，並加入 **Gram anchoring**，以對抗長時間訓練中密集（區塊）特徵品質下降的問題。它不只是讓每個特徵各自匹配，而是約束區塊特徵之間的相似度矩陣，使其接近較早期模型的矩陣。報告也把一個非常大的教師模型蒸餾成較小的模型以便部署。

實務上的問題是：凍結的全域或密集特徵能不能解決**你的**任務。一個擅長語意檢索的模型，可能會漏掉製造零件上的一道小刮痕。提高輸入解析度並使用區塊特徵可能有幫助，但這會改變運算預算與比較的方式。

誠實地評估遷移

白話版。只有在沒被用來建立或挑選它的資料與任務上也有幫助，一個表示才稱得上好。你怎麼測，會改變答案。

評估方式	訓練什麼	衡量什麼
k-NN	不訓練；與有標記參考特徵的相似度	表示的局部結構
線性探測	線性任務頭；編碼器凍結	簡單讀出就能取得的資訊
微調	任務頭加上部分或全部編碼器層	在給定標記與運算預算下的可調適性
密集遷移	在區塊特徵圖上的空間任務頭	是否適合像素或區域層級的輸出

線性探測與微調

微調的自由度較高，所以當目標資料充足、且與預訓練資料相似時，它通常會贏。但它不一定比較好。Kumar 等人 [23] 從理論與實驗上證明，完整微調可能會**扭曲**好的預訓練特徵：當隨機初始化的任務頭還是錯的時候，它的大梯度流進編碼器，把特徵改往適合訓練分布、卻傷害**分布外**（out-of-distribution, OOD）輸入的方向。線性探測不會扭曲特徵。他們的對策 **LP-FT** 分兩步：先凍結編碼器訓練線性任務頭，再從這個任務頭出發微調全部參數。

我們在數字範例上重現了這個定性效應。編碼器是上面的 SimCLR CNN；有標記的訓練數字是乾淨的；測試數字則是平移過的，屬於輕微的分布偏移。每種標記預算都對三次隨機抽取的標記樣本取平均。

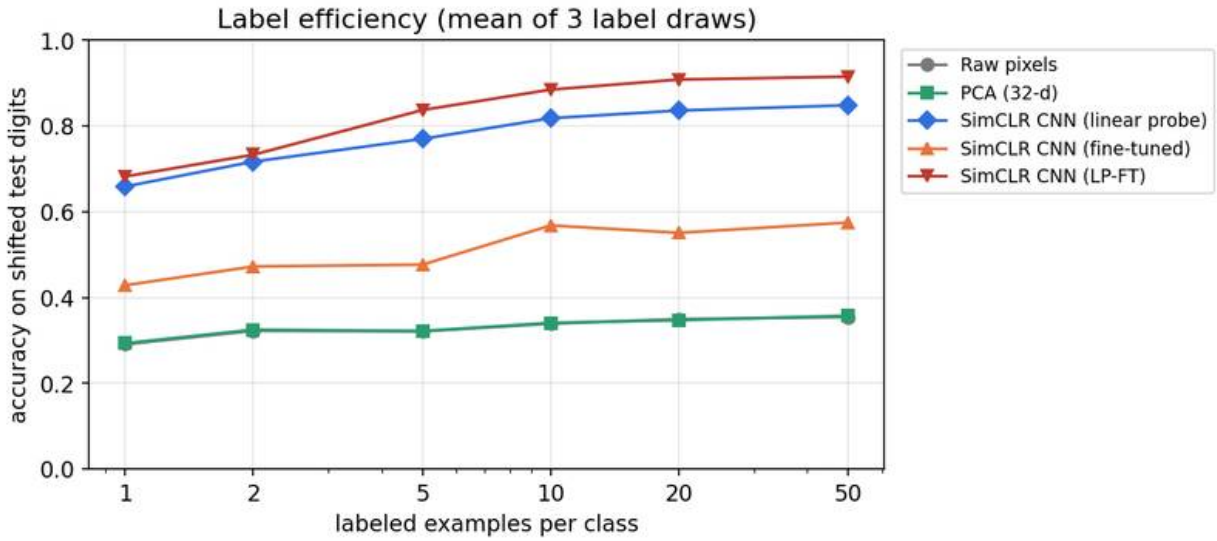


圖 *8* — 平移測試數字上的標記效率。自監督特徵比像素或 *PCA* 更省標記。在固定的一套訓練設定下，從隨機任務頭開始的一般微調降低了平移測試的準確率，而 *LP-FT* 則比探測更好。這是玩具實驗的結果：調好學習率或有更多標記時，一般微調可能表現更好。

請看圖 8 的形狀，而不是確切數值。凍結的自監督特徵搭配線性任務頭，每類只要少數幾個標記，就能勝過每類 50 個標記的原始像素。在這裡，從隨機初始化任務頭開始的一般微調輸給了凍結的探測，而 LP-FT 兩者都贏，這和 Kumar 等人在分布偏移下回報的順序相同。微調需要自己的超參數搜尋，任務頭的初始化也很重要。

防止資料洩漏的規則

- 做相似度比較時，特徵要一致地正規化（餘弦 *k*-NN 需要正規化向量）。
- 標準化、PCA 與分類器的超參數，只能在訓練與驗證資料上擬合。
- 凍結的編碼器仍然有訓練歷史：預訓練資料與評估資料的重疊可能灌水結果，網路規模的模型尤其如此。
- 回報標記數量、骨幹網路、檢查點、輸入解析度、資料增強、任務頭、調參預算與評估方式。在測試集上從許多編碼器中挑最好的，就是測試資料洩漏，即使沒有任何一個被微調。

現代觀點

綜述與回顧論文。四篇綜述從互補的角度描繪這個領域。

- **Bengio、Courville 與 Vincent（TPAMI 2013）[1]** 為整件事定下框架。他們問一個表示憑什麼比另一個好，並以一組通用先驗回答：分散式編碼、深度與抽象、解釋因子的解糾纏（disentangling）、流形與自然分群。他們回顧了機率模型、自編碼器與流形學習。這篇早於現代自監督學習，但仍是「我們為什麼要學表示」最好的陳述。

- **Liu 等人（TKDE 2023）[3]** 把橫跨視覺、語言與圖的自監督方法分成**生成式**（自編碼與自迴歸重建）、**對比式**（脈絡–實例與實例–實例）與**生成–對比式**（對抗式）三組，並討論當時對比式目標為何在視覺基準上勝出。
- **Gui 等人（TPAMI 2024）[4]** 對演算法做了較新的整理，涵蓋對比式、自蒸餾、冗餘抑制與遮罩影像建模方法，並討論應用與開放問題，例如理論、代理任務的選擇與多模態自監督學習。
- **Balestriero 等人（2023）[5]** 是由領域內許多作者合寫的實務指南。它把方法分成深度度量學習家族（SimCLR 及其相關方法）、自蒸餾家族（BYOL、SimSiam、DINO）、典型相關分析家族（VICReg、Barlow Twins）以及遮罩影像建模，並收集了論文常常埋在細節裡的「訣竅」（資料增強、投影器、EMA 排程、批次大小的影響）。Prince 的教科書 [24] 則以平易、圖文並茂的方式並附上 notebook 介紹同樣的概念。

實務現況（2023–2026）。從上面引用的論文中可以看到以下趨勢：

- **結合多種目標**。領先的影像編碼器把影像層級的蒸餾與區塊層級的遮罩預測結合起來（DINOv2 [21]、DINOv3 [22]），而不是只依賴一種代理任務。
- **資料是一級設計選擇**。 DINOv2 的策展流程，以及 CLIP [19] 與 SigLIP [20] 背後網路規模的配對收集，都顯示預訓練吃進什麼資料和損失函數一樣重要。
- **密集特徵**。分割、深度與對應關係需要好的區塊特徵，這促成了像 Gram anchoring [22] 這樣的目標。
- **在潛在空間中預測，並延伸到影片**。 I-JEPA [17] 與 V-JEPA 2 [18] 預測特徵而不是像素，並延伸到影片與動作條件預測。
- **與語言對齊的編碼器成為多模態模型的預設骨幹**（見ML 09）。

開放問題。我們仍然缺少一套能預測「哪種代理任務適合哪種下游任務」的理論；防止塌縮靠的多半是配方而不是保證；評估結果高度依賴評估方式，以及預訓練與測試資料之間的重疊；網路規模的預訓練也繼承了資料中的偏誤與缺口。我們對 [3]、[4]、[5]、[21]、[22] 的解讀是：資料與目標的設計值得和架構一樣多的關注。這是對文獻的詮釋，不是「某個目標到處都贏」的定理。

接下來的章節換一個問題：與其描述一張影像，模型要如何描述並**取樣**整個分布？自編碼器見ML 05A，生成模型見ML 06。

重點整理

- 如果簡單的任務頭只用少量標記就能從表示中解決任務，這個表示就是有用的。分散式編碼能泛化，因為特徵在樣本之間共享。
- 你想要的不變性取決於任務。在自監督學習中，是資料增強在選擇不變性，平移數字的例子就說明了這點。
- 對比式方法（SimCLR、MoCo）就是 InfoNCE 分類：在負例中找出你的正例。塌縮會讓它們付出 $\log(2B-1)$ 的代價。
- 沒有負例的方法（BYOL、SimSiam、DINO）靠不對稱避免塌縮：預測器、停止梯度、EMA 教師、中心化與銳化。Barlow Twins 與 VICReg 則把防塌縮項放進損失。
- 遮罩建模（BEiT、MAE）重建被遮住的內容；JEPA 預測被遮住的特徵。CLIP 讓影像與圖說對齊，做到零樣本分類。
- k-NN、線性探測與微調回答的是不同的問題。在分布偏移下，微調可能扭曲特徵；LP-FT 是簡單的解法。

練習

1. **溫度與塌縮**。在第一段程式中，把溫度從 0.2 改成 0.05 和 1.0。塌縮損失會變嗎？配對損失呢？

   提示

   塌縮損失仍是 $\log 15$：所有非自身的 logits 都相等，所以除以任何 τ 之後 softmax 仍是均勻的。配對損失會改變，因為溫度縮放的是分數之間的差距；有真正的差距時，τ 越小 softmax 越尖銳。

2. **資料增強就是假設**。你想要的特徵用於 (a) 顯微影像中的細胞計數、(b) 鳥種辨識、(c) 讀取路標文字。各舉一個你會拿掉的標準 SimCLR 資料增強，並說明原因。

   提示

   (a) 隨機裁切可能改變視野中的細胞數量，所以「相同」的標記其實不再相同。(b) 強烈的顏色抖動可能抹去羽色，而羽色是重要的鳥種線索。(c) 水平翻轉會把文字變成鏡像文字。先決定什麼必須保留，再選擇不變性。

3. **MAE 的運算量**。一張影像有 196 個區塊，MAE 遮住 75%。有多少個區塊進入編碼器？編碼器中兩兩注意力分數的數量縮小為幾分之幾？

   提示

   49 個可見區塊進入編碼器。注意力分數隨符元數量的平方成長：$(49/196)^2 = 1/16$。解碼器仍然要處理全部 196 個位置，但它很小。

4. **破壞數字實驗**。修改 SimCLR 數字程式，讓 `augment` 不做平移。重新訓練，並比較平移測試的 k-NN 準確率與原本的結果。這告訴你什麼？

   提示

   把隨機的 `i, j` 換成 `i, j = 1, 1`。在我們的機器上，平移測試的準確率從 0.919 掉到 0.743，而乾淨測試的準確率維持在 0.981。少了平移增強，目標中就沒有任何東西要求平移不變性；剩下的穩健性來自訓練過的卷積加池化編碼器本來就會學到的東西（主範例中未訓練的那個完全沒有）。決定學到什麼的主要是資料增強，而不是「自監督」這個名詞。

5. **紙上的 SimSiam 消融實驗**。在 SimSiam 中，假設你拿掉停止梯度，讓兩個分支都接收梯度。用兩句話說明為什麼常數輸出會變成很容易達到的最小值，以及為什麼光有預測器不足以阻止它。

   提示

   兩邊都有梯度時，只要 p_1 和 z_2 指向同一方向，損失 $-\cos(p_1, z_2)$ 就達到最小，而一個常數編碼器加上一個輸出同樣常數方向的預測器，對每個輸入都做得到。停止梯度改變了最佳化動態，讓目標在每一步中保持固定；SimSiam 論文回報，沒有它就會迅速塌縮。

6. **公平的探測**。設計一個實驗，在每類 10、50、100 個標記下，比較凍結的預訓練編碼器與 PCA 加邏輯斯迴歸。列出三件必須固定的事，以及兩件必須回報的事。

提示

必須固定：有標記的子集（兩者使用相同樣本）、保留的測試切分，以及選擇正則化強度的程序（只用驗證資料）。必須回報：預訓練編碼器的訓練資料（以及與你測試集的任何重疊），以及隨機抽取標記的次數與其變異範圍。

參考文獻

1. Y. Bengio, A. Courville and P. Vincent, "Representation Learning: A Review and New Perspectives," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 8, 2013. Paper ·arXiv
2. J. Yosinski, J. Clune, Y. Bengio and H. Lipson, "How transferable are features in deep neural networks?" NeurIPS, 2014. Paper
3. X. Liu et al., "Self-supervised Learning: Generative or Contrastive," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, 2023 (online 2021). Paper ·arXiv
4. J. Gui et al., "A Survey on Self-Supervised Learning: Algorithms, Applications, and Future Trends," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024. Paper ·arXiv
5. R. Balestriero et al., "A Cookbook of Self-Supervised Learning," arXiv:2304.12210, 2023. Paper
6. A. van den Oord, Y. Li and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," arXiv:1807.03748, 2018. Paper
7. T. Chen, S. Kornblith, M. Norouzi and G. Hinton, "A Simple Framework for Contrastive Learning of Visual Representations," ICML, 2020. Paper
8. K. He, H. Fan, Y. Wu, S. Xie and R. Girshick, "Momentum Contrast for Unsupervised Visual Representation Learning," CVPR, 2020. Paper
9. T. Wang and P. Isola, "Understanding Contrastive Representation Learning through Alignment and Uniformity on the Hypersphere," ICML, 2020. Paper
10. J.-B. Grill et al., "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning," NeurIPS, 2020. Paper
11. X. Chen and K. He, "Exploring Simple Siamese Representation Learning," CVPR, 2021. Paper
12. M. Caron et al., "Emerging Properties in Self-Supervised Vision Transformers," ICCV, 2021. Paper
13. J. Zbontar, L. Jing, I. Misra, Y. LeCun and S. Deny, "Barlow Twins: Self-Supervised Learning via Redundancy Reduction," ICML, 2021. Paper
14. A. Bardes, J. Ponce and Y. LeCun, "VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning," ICLR, 2022. Paper
15. H. Bao, L. Dong, S. Piao and F. Wei, "BEiT: BERT Pre-Training of Image Transformers," ICLR, 2022. Paper
16. K. He et al., "Masked Autoencoders Are Scalable Vision Learners," CVPR, 2022. Paper
17. M. Assran et al., "Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture," CVPR, 2023. Paper

18. M. Assran et al., “V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning,” technical report, 2025. Report
19. A. Radford et al., “Learning Transferable Visual Models From Natural Language Supervision,” ICML, 2021. Paper
20. X. Zhai, B. Mustafa, A. Kolesnikov and L. Beyer, “Sigmoid Loss for Language Image Pre-Training,” ICCV, 2023. Paper
21. M. Oquab et al., “DINOv2: Learning Robust Visual Features without Supervision,” *Transactions on Machine Learning Research*, 2024. Paper
22. O. Siméoni et al., “DINOv3,” technical report, 2025. Report
23. A. Kumar, A. Raghunathan, R. Jones, T. Ma and P. Liang, “Fine-Tuning can Distort Pretrained Features and Underperform Out-of-Distribution,” ICLR, 2022. Paper
24. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. Book and notebooks

ML 05A・線性因子模型與自編碼器

PCA、機率 PCA、因子分析、ICA、稀疏編碼、NMF 與自編碼器（欠完備、稀疏、去噪、收縮）：壓縮、雜訊假設與流形觀點如何一路通往 VAE。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-05a-factor-models-autoencoders/

你將學到

- 少數幾個隱藏的**因子**（factor）如何解釋大量彼此相關的量測值，以及為什麼這個想法同時支撐了古典統計與現代深度生成模型。
- PCA、機率 PCA 與因子分析的差別，以及它們各自對雜訊做了什麼假設。
- 獨立成分分析（ICA）、稀疏編碼（sparse coding）與非負矩陣分解（NMF）如何藉由不同的額外假設，挑出不同的因子。
- 線性與非線性自編碼器（autoencoder）和這些模型的關係，以及稀疏、去噪、收縮懲罰如何改變自編碼器學到的東西。
- **流形觀點**（manifold view）對高維資料說了什麼，以及它為何自然地把我們從自編碼器帶到變分自編碼器（VAE）。

先看全貌

一張 8×8 的數字影像有 64 個像素值，一張手機照片則有數百萬個。然而，一批這類影像中真正有意義的變化，只取決於少得多的潛在量：是哪個數字、筆畫粗細、傾斜角度、位置、光照。**因子模型**試圖用少數幾個隱藏（潛在）座標來解釋每一筆觀測。**自編碼器**則是一種神經網路，它學著把輸入擠進這樣的座標，再展開還原。

> **用最簡單的話說**
>
> 想像你在電話裡向別人描述朋友的長相。你不會逐一唸出每顆雀斑，而是說「圓臉、短髮、笑容很大、光從左邊打過來」。這幾個詞就是因子：聽的人靠它們就能在腦中拼出一張不錯的畫面。因子模型要找的，是能同時解釋最多張臉的那幾個詞。自編碼器則是一對小幫手：一個把圖片變成幾個詞，另一個把詞變回圖片。兩人一起練習，直到還原的圖片看起來夠像為止。

為什麼要用一整章談這個？因為現代表示學習與生成模型裡幾乎每個想法，都能在這裡找到祖先。用於視覺化的降維、去噪、壓縮、異常偵測、把影像轉成緊湊編碼供擴散模型使用的「tokenizer」，以及 VAE 的機率架構，全都從同一個問題出發：哪一小組原因可能產生了這些觀測？從線性因子模型到自編碼器的脈絡，依循 Goodfellow、Bengio 與 Courville [1, 第 13–14 章] 以及 Murphy [2]；Bengio、Courville 與 Vincent 的回顧論文 [3] 則說明了學習到的表示為何重要。

一個範本，多種模型

本章每一個方法都套用同一個範本。**編碼器**（encoder）把觀測 $x \in \mathbb{R}^d$ 映射成編碼 $z \in \mathbb{R}^k$，**解碼器**（decoder）再把編碼映回重建結果 $\hat{x}$。各模型的差別只在三個選擇：

選擇	本章出現的選項
編碼器／解碼器的形式	線性（PCA、因子分析、ICA、NMF、稀疏編碼）或非線性（深度自編碼器）
編碼的大小	**欠完備**（undercomplete，$k < d$：瓶頸迫使模型壓縮）或**過完備**（overcomplete，$k \geq d$：必須靠其他機制阻止模型直接複製）
額外假設或懲罰	高斯雜訊（PPCA、FA）、獨立且非高斯（ICA）、稀疏（稀疏編碼、稀疏自編碼器）、非負（NMF）、對破壞具穩健性（去噪）、對微小輸入變化不敏感（收縮）

請記住這張表。兩個方法結論不同時，幾乎總是因為它們在某一列做了不同的選擇，而不是其中一個「錯了」。

PCA：最佳的線性重建子空間

白話版。找幾個彼此正交的方向，使得每筆資料只保留在這些方向上的座標時，損失的資訊越少越好。

令 $X \in \mathbb{R}^{n \times d}$ 的每一列是一筆已中心化（減去各欄平均）的資料，共 n 筆。PCA 選擇一個欄向量互相正交且長度為 1 的矩陣 $W \in \mathbb{R}^{d \times k}$，使平方重建誤差最小：

$$\min_{W} \; \|X - XWW^{\top}\|_F^2 \qquad \text{subject to} \qquad W^{\top}W = I_k.$$

其中 $\|\cdot\|_F$ 是 Frobenius 範數（所有元素平方和的平方根），$Z = XW \in \mathbb{R}^{n \times k}$ 是**編碼**（也稱分數），$\hat{X} = ZW^{\top}$ 是重建結果。編碼器是 $x \mapsto W^{\top}x$，解碼器是 $z \mapsto Wz$：PCA 本身就是一個自編碼器，只是線性且權重共用。

解從哪裡來

兩種等價的觀點會得到同一個答案。

最大變異數。取一個單位向量 w。投影後資料 Xw 的變異數是 $w^{\top}Sw$，其中 $S = \frac{1}{n-1}X^{\top}X$ 是樣本共變異數矩陣。用拉格朗日乘數 λ，在 $w^{\top}w = 1$ 的限制下最大化 $w^{\top}Sw$：

$$\nabla_w (w^{\top}Sw - \lambda(w^{\top}w - 1)) = 0 \quad \Longrightarrow \quad Sw = \lambda w.$$

所以 w 必須是 S 的特徵向量，而它捕捉到的變異數就是對應的特徵值 λ。最好的方向是特徵值最大的特徵向量；在與第一個方向正交的限制下，次好的方向是第二個特徵向量，依此類推。

最小重建誤差。由於 W 的欄向量正交歸一，誤差可以拆成「總變異數減去保留的變異數」。因此最小化誤差等同於最大化保留的變異數，而殘差恰好是被捨棄的特徵值總和（乘上 $n-1$）。

Eckart–Young 定理用矩陣語言說了同一件事：把奇異值分解 $X = U\Sigma V^{\top}$ 截斷到最大的 k 個奇異值，就得到 Frobenius 範數下最佳的秩 k 近似。PCA 方向就是前 k 個右奇異向量（V 的欄），而共變異數的特徵值為 $\sigma_j^2/(n-1)$。

程式：用 SVD 實作 PCA

下面的程式在 scikit-learn 內建 digits 資料集的訓練部分上擬合 PCA，在保留的測試影像上評估重建，並與 scikit-learn 的實作互相核對。

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split

X = load_digits().data / 16.
train, test = train_test_split(X, test_size=.2, random_state=0)

mean = train.mean(axis=0)
U, s, Vt = np.linalg.svd(train - mean, full_matrices=False)
var = s**2 / (len(train) - 1)              # eigenvalues of the covariance
ratio = np.cumsum(var) / var.sum()
for k in (2, 10, 20):
    W = Vt[:k].T                           # d x k, orthonormal columns
    rec = (test - mean) @ W @ W.T + mean
    print(f"k={k:2d}  explained={ratio[k-1]:.3f}  "
          f"test MSE={np.mean((test - rec)**2):.4f}")

sk = PCA(n_components=10).fit(train)
print("matches sklearn:", np.allclose(np.abs(sk.components_), np.abs(Vt[:10])))
```

輸出：

```
k= 2  explained=0.291  test MSE=0.0546
k=10  explained=0.740  test MSE=0.0202
k=20  explained=0.896  test MSE=0.0084
matches sklearn: True
```

最後一行的 `np.abs` 並不是投機取巧：每個主方向本來就只確定到正負號，因為 w 與 $-w$ 張成同一條直線。也請注意，平均值只在訓練資料上估計，再套用到測試資料；若用測試資料估計平均值，就會洩漏資訊。

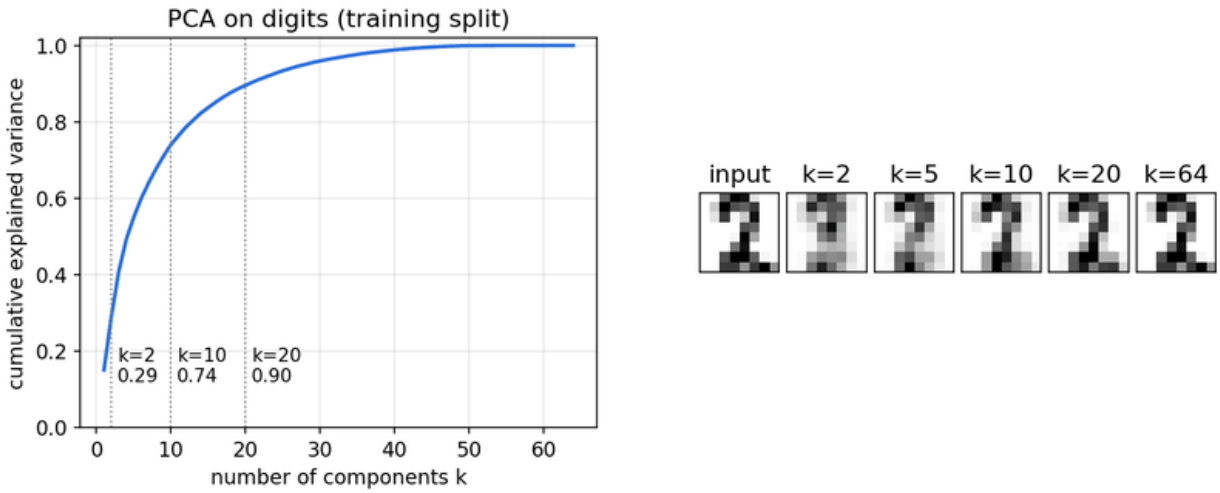


圖 *1* — 在內建 *digits* 資料集上的 *PCA*。左：累積解釋變異數（在訓練資料上擬合）。右：一張保留的測試數字用 k 個成分重建；$k = 64$ 時可完整還原輸入。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

PCA 沒有承諾的事

從目標函數可以直接得出三個提醒。

- **中心化與尺度會改變答案**。沒有中心化時，第一個「成分」大多指向平均值。若把每個特徵標準化為單位變異數，每個像素的權重就相同，因此原始像素 PCA 與標準化 PCA 解的是不同問題。
- **變異數不等於相關性**。變異數最大的方向可能是光照或背景，而不是你關心的類別。PCA 是非監督式的；它最大化保留的變異數，而不是類別分離度（那是線性判別分析等方法的工作）。
- **PCA 是線性的**。若資料落在彎曲的曲面上，平坦的子空間可能需要很多維度，才能描述其實是低維的形狀。我們會在流形一節回到這一點。

機率 PCA 與因子分析

白話版。與其只找一個子空間，不如寫下一個資料如何產生的故事：先抽出幾個隱藏因子，把它們線性混合，再加上雜訊。接著用最大概似法擬合這個故事。

兩個模型共用同一條生成方程式：

$$x = \mu + Wz + \epsilon, \qquad z \sim \mathcal{N}(0, I_k),$$

其中 $\mu \in \mathbb{R}^d$ 是平均值，$W \in \mathbb{R}^{d \times k}$ 是**負荷矩陣**（loading matrix，描述每個因子如何出現在每個量測座標上），z 是潛在因子向量，ϵ 是與 z 獨立的雜訊。兩者只差在雜訊。

機率 PCA（PPCA）

在 PPCA [4] 中，雜訊是等向的，$\epsilon \sim \mathcal{N}(0, \sigma^2 I_d)$：每個座標的雜訊變異數都相同。把 z 積分掉，得到高斯邊際分布

$$x \sim \mathcal{N}(\mu,\, C), \qquad C = WW^{\top} + \sigma^2 I_d.$$

這個共變異數有清楚的解讀：低秩部分 $WW^{\top}$ 由因子解釋，再加上一個因子解釋不了的固定「底噪」σ^2。Tipping 與 Bishop 證明最大概似解有封閉形式 [4]：

$$W_{\mathrm{ML}} = U_k (\Lambda_k - \sigma^2 I)^{1/2} R, \qquad \sigma^2_{\mathrm{ML}} = \frac{1}{d-k} \sum_{j=k+1}^{d} \lambda_j,$$

其中 U_k 是樣本共變異數前 k 個特徵向量，Λ_k 是對應的特徵值 $\lambda_1, \dots, \lambda_k$，$R$ 是任意 $k \times k$ 正交矩陣。換句話說：負荷矩陣恰好張成 PCA 子空間，而雜訊變異數就是 PCA 丟掉的變異數的平均。因此 PPCA 就是 PCA 再加上一個概似函數。這個概似函數很有用：你可以在保留資料上比較不同 k 的模型、用期望最大化（EM）演算法處理缺值，也能建立 PPCA 的混合模型。

給定一筆觀測，其因子的後驗分布是高斯分布，平均為

$$\mathbb{E}[z \mid x] = M^{-1} W^{\top} (x - \mu), \qquad M = W^{\top} W + \sigma^2 I_k.$$

這就是機率式的編碼器。當 $\sigma^2 \to 0$ 時，它趨近一般的 PCA 投影；當 $\sigma^2 > 0$ 時，它會把編碼往零收縮，因為看到的東西有一部分被歸因於雜訊。

因子分析（FA）

因子分析（factor analysis）保留相同結構，但允許對角的雜訊共變異數，$\epsilon \sim \mathcal{N}(0, \Psi)$，$\Psi = \mathrm{diag}(\psi_1, \dots, \psi_d)$。每個觀測座標各有自己的殘差雜訊水準 ψ_i，而潛在因子負責解釋座標之間**共享**的共變異數。邊際分布為 $x \sim \mathcal{N}(\mu, WW^{\top} + \Psi)$，因子的後驗平均為 $\mathbb{E}[z \mid x] = W^{\top}(WW^{\top} + \Psi)^{-1}(x - \mu)$。

FA 沒有封閉解，因此用 EM 或直接最佳化概似函數來擬合。當量測來自雜訊程度不一的感測器，或某些像素幾乎恆定（例如 digits 影像邊框上永遠空白的像素）時，這份額外的彈性就很重要。PCA 會毫不猶豫地把成分花在任何高變異數的座標上；FA 則可以把個別雜訊很大的座標解釋成「雜訊」，把因子留給座標之間的相關性。

旋轉問題

兩個模型都有同一種不確定性。對任意正交矩陣 R，把 W 換成 WR、z 換成 $R^{\top} z$，$WW^{\top}$ 不變，因此 x 的分布也不變，因為 $R^{\top} z$ 仍是標準常態。子空間可以被確定，個別的座標軸卻不行。這就是因子分析文獻使用 varimax 等旋轉來挑選較好解讀的座標軸的原因，也說明了因子不會自動等同於物理上的成因。若要確定個別因子，我們必須在高斯以外再假設些什麼，而 ICA 正是這麼做的。

獨立成分分析（ICA）

白話版。一個房間裡好幾個人同時講話，幾支麥克風各自錄到不同比例的混音。ICA 試著找出統計上盡可能獨立、而且盡可能不像高斯分布的組合，藉此還原每個人的聲音。

無雜訊的 ICA 模型為

$$x = As, \qquad s = (s_1, \dots, s_k) \text{ with independent components,}$$

其中 $x \in \mathbb{R}^d$ 是觀測到的混合訊號，$A \in \mathbb{R}^{d \times k}$ 是未知的**混合矩陣**，來源 s_j 彼此獨立且**非高斯**。目標是找一個解混矩陣 $B \approx A^{-1}$（或其偽逆），使 $\hat{s} = Bx$ 還原出來源 [5]。

為什麼關鍵在於非高斯

上一節的旋轉問題說明了 ICA 為何需要非高斯來源。先把資料**白化**（whitening）：轉換 x 使其共變異數為單位矩陣（先做 PCA，再把每個成分縮放到單位變異數即可）。白化之後，剩下的未知量只是一個旋轉。如果來源是高斯，獨立單位高斯的任何旋轉仍然是獨立單位高斯，於是沒有任何旋轉比較好。ICA 的可識別性定理說的是反方向：只要**至多一個**來源是高斯，來源就能被還原，只剩三種無傷大雅的不確定性 [5]：

- **順序**（我們無法得知哪個來源是「第一個」），
- **正負號**，以及
- **尺度**（大聲的來源配上較小的混合係數，看起來一模一樣）。

中央極限定理提供了找到正確旋轉的直覺：獨立變數的總和往往比各部分更像高斯。所以 x 的投影中最不像高斯的那些，最可能是單一來源而不是混合。因此 ICA 演算法會最大化某種非高斯程度的量度。一個有理論依據的量度是**負熵**（negentropy）：

$$J(y) = H(y_{\text{gauss}}) - H(y) \;\geq 0,$$

其中 H 是微分熵，y_{gauss} 是與 y 變異數相同的高斯變數。在變異數固定的所有分布中，高斯分布的熵最大，所以 $J(y) = 0$ 恰好發生在 y 為高斯時。由於熵很難估計，**FastICA** 演算法用 $G(u) = \log\cosh u$ 等對比函數近似負熵，並以快速的不動點迭代找出每個解混方向 [5]。最大化非高斯程度，與最小化各估計成分之間的互資訊、以及 ICA 模型的最大概似估計密切相關。

程式：把三個訊號解混

這個玩具問題用了一個正弦波、一個方波與一個鋸齒波，全都是非高斯，並以固定矩陣混合。

```
import numpy as np
from sklearn.decomposition import FastICA, PCA

rng = np.random.default_rng(0)
t = np.linspace(0, 8, 2000)
S = np.c_[np.sin(2*t),                  # smooth sine
          np.sign(np.sin(3*t)),         # square wave
```

```
          2*(t % 1) - 1]               # sawtooth
S += 0.05 * rng.normal(size=S.shape)
A = np.array([[1.0, 1.0, 1.0],
              [0.5, 2.0, 1.0],
              [1.5, 1.0, 2.0]])        # unknown mixing matrix
X = S @ A.T                            # three "microphones"

def best_match(est):
    C = np.abs(np.corrcoef(S.T, est.T)[:3, 3:])
    return np.round(C.max(axis=1), 3)

ica = FastICA(n_components=3, whiten="unit-variance", random_state=0)
print("ICA |corr| with each source:", best_match(ica.fit_transform(X)))
print("PCA |corr| with each source:", best_match(PCA(3).fit_transform(X)))
```

輸出：

```
ICA |corr| with each source: [0.997 0.999 0.997]
PCA |corr| with each source: [0.689 0.811 0.735]
```

我們用相關係數的絕對值並取最佳配對來比較，因為 ICA 回傳的來源順序、正負號與尺度都是任意的。PCA 只做去相關：它找的是高變異數的正交方向，在這裡仍然是混合訊號。

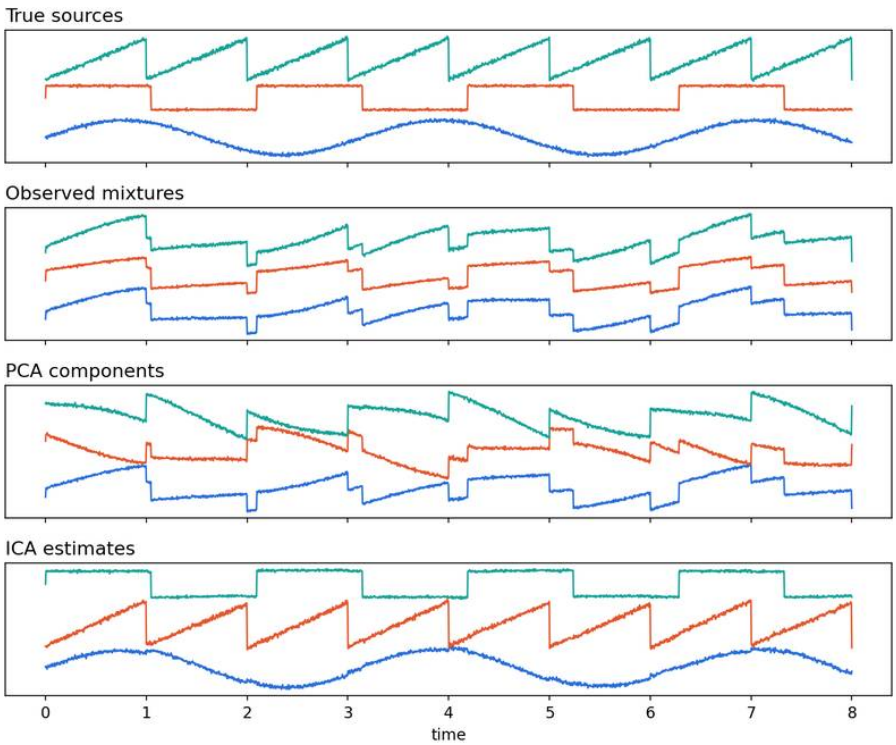


圖 2 — 盲訊號分離。*PCA* 成分仍是混合訊號；*FastICA* 在順序、正負號與尺度的不確定性之內，還原出正弦波、方波與鋸齒波。

ICA 廣泛用於分離腦電圖（EEG）等感測紀錄中的雜訊與偽影。它的假設也劃出了它的極限：它需要真正獨立、非高斯的來源，以及接近線性且瞬時的混合過程。在影像區塊（patch）上，ICA 往往學到局部化、有方向性的邊緣狀濾波器，這也引出一個密切相關的想法。

稀疏編碼

白話版。準備一本很大的小圖樣字典（「原子」，atom），每次描述一個輸入時只用其中少數幾個。稀疏編碼學習一個字典 $D = [d_1, \dots, d_m] \in \mathbb{R}^{d \times m}$，並為每筆資料 x_i 學一個稀疏編碼 $\alpha_i \in \mathbb{R}^m$，求解

$$\min_{D,\,\{\alpha_i\}} \sum_{i=1}^{n} \left(\tfrac{1}{2} \|x_i - D\alpha_i\|_2^2 + \lambda \|\alpha_i\|_1 \right) \qquad \text{subject to} \qquad \|d_j\|_2 \le 1.$$

第一項要求準確重建；ℓ_1 範數 $\|\alpha\|_1 = \sum_j |\alpha_j|$ 把大多數編碼元素推成恰好為零；$\lambda > 0$ 決定兩者的取捨。原子的範數限制防止模型作弊，也就是把原子放得很大、編碼縮得很小。字典常是**過完備**的（$m > d$）：原子比像素維度多，但對任何一個輸入只有少數幾個會啟動。

Olshausen 與 Field [6] 在自然影像的區塊上訓練這類模型，發現學到的原子是局部化、有方向性、帶通的，非常像初級視覺皮質中簡單細胞的感受野。這個結果影響深遠，因為它顯示：只靠稀疏性、不需要任何標籤，就能解釋生物視覺的一項已知特性。

有兩個實務重點讓稀疏編碼不同於本章稍後的自編碼器：

- **推論本身是一個最佳化問題**。給定 D，要為新輸入找出 α，就得解一個 lasso 問題。這裡沒有固定的編碼器網路；「編碼器」是一個演算法。訓練時交替更新編碼（固定 D）與字典（固定編碼）。
- **稀疏性是一種先驗**。ℓ_1 懲罰對應於編碼上的 Laplace 先驗，所以稀疏編碼可以看成：在一個具有稀疏、非高斯先驗的線性因子模型中做最大後驗（MAP）推論，而這正是 ICA 所屬的同一個家族。

程式：稀疏程度與重建精確度

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.decomposition import DictionaryLearning
from sklearn.model_selection import train_test_split

X = load_digits().data / 16.
train, test = train_test_split(X, test_size=.2, random_state=0)
for alpha in (0.1, 1.0):
    dl = DictionaryLearning(n_components=32, alpha=alpha, max_iter=30,
                            transform_algorithm="lasso_lars",
                            transform_alpha=alpha, random_state=0)
    dl.fit(train[:600])
    codes = dl.transform(test)
    rec = codes @ dl.components_
    active = np.mean(np.abs(codes) > 1e-8, axis=1).mean()
```

```
    print(f"alpha={alpha}: active atoms per image={active*32:.1f}/32, "
          f"test MSE={np.mean((test - rec)**2):.4f}")
```

輸出（在筆電 CPU 上約需半分鐘）：

```
alpha=0.1: active atoms per image=17.9/32, test MSE=0.0075
alpha=1.0: active atoms per image=3.0/32, test MSE=0.0401
```

提高 λ（這裡的 `alpha`）可以換到稀疏得多的描述，每張影像約啟動 3 個原子而不是 18 個，代價是重建誤差變高。兩種設定都沒有「對錯」：壓縮編解碼器、去噪器與可解釋性工具各自會做不同的選擇。請注意，這個字典沒有減去平均值，所以它的誤差不能直接與上面的 PCA 數字比較。

非負矩陣分解（NMF）

白話版。把每筆資料描述成非負部件的加法組合：你可以加上筆畫，但永遠不能減掉筆畫。

對非負資料 $X \geq 0$（像素強度、詞頻、光譜），NMF 尋找 $Z \geq 0$ 與 $H \geq 0$，使下列重建準則最小：

$$\min_{Z \geq 0,\, H \geq 0} \|X - ZH\|_F^2,$$

其中 $H \in \mathbb{R}^{k \times d}$ 的每一列是一個非負部件，$Z \in \mathbb{R}^{n \times k}$ 的每一列是非負的混合權重。因為沒有東西可以互相抵消，這些部件往往看起來像物體的局部，而不是 PCA 那種有正有負的「幽靈」圖樣。Lee 與 Seung [7] 在人臉與文字上展示了這種以部件為基礎的行為，並推廣了簡單的乘法更新規則來擬合模型。

兩點提醒。第一，以部件為基礎是一種傾向，而不是保證：在某些資料集上，因子並不是乾淨的部件。第二，目標函數對 Z 與 H 聯合而言是非凸的，所以初始化很重要，不同次執行可能得到不同的因子。scikit-learn 的 `init="nndsvda"` 用以 SVD 為基礎的起點讓結果更穩定。

程式：比較各因子模型的重建

這個 CPU 範例只在訓練用的數字上擬合各模型，並在保留的影像上評估重建。它比較的是不同的模型假設，並不是宣稱某個方法在所有任務上都勝出。

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA, FactorAnalysis, NMF

X = load_digits().data / 16.
train, test = train_test_split(X, test_size=.2, random_state=0)
models = {
    "PCA": PCA(n_components=12),
    "factor analysis": FactorAnalysis(n_components=12, random_state=0),
```

```
    "NMF": NMF(n_components=12, init="nndsvda", max_iter=1000,
               random_state=0),
}
for name, model in models.items():
    model.fit(train)
    z = model.transform(test)
    if name == "factor analysis":
        reconstructed = z @ model.components_ + model.mean_
    else:
        reconstructed = model.inverse_transform(z)
    mse = np.mean((test-reconstructed)**2)
    print(name, "held-out reconstruction MSE:", round(mse, 5))
```

輸出：

```
PCA held-out reconstruction MSE: 0.01652
factor analysis held-out reconstruction MSE: 0.02501
NMF held-out reconstruction MSE: 0.02207
```

PCA 在平方重建誤差上勝出，這是必然的：在所有秩 12 的線性重建中，對這個準則而言 PCA 子空間就是最佳解（只差在訓練／測試切分）。但這不代表它是最好的模型。上面因子分析的重建是後驗平均重建，它刻意把每個像素的一部分歸因於雜訊；而 FA 真正的產出——一個帶有逐像素雜訊的完整機率模型——完全無法用 MSE 概括。NMF 則用一些誤差換來非負、可加的部件。

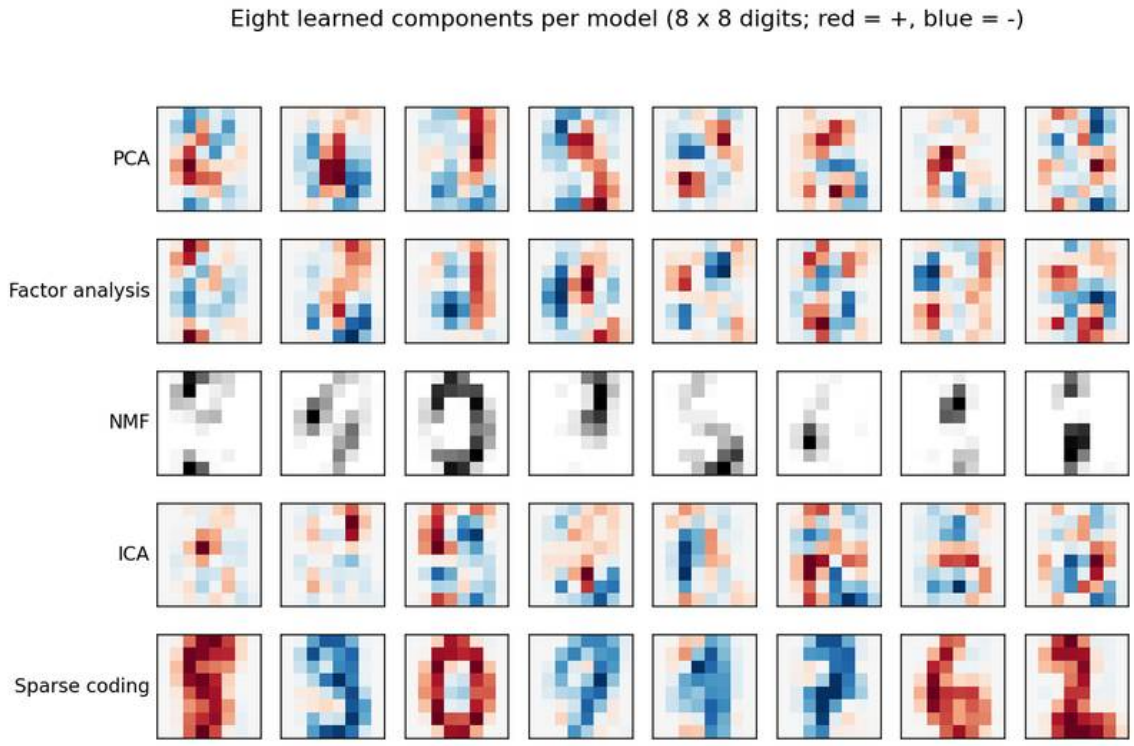


圖 3 — 各線性因子模型在同一批數字上學到的八個成分。*PCA* 與 *FA* 給出有正負號的全域圖樣；*NMF* 給出非負的筆畫；*ICA* 給出有正負號但較局部的圖樣；稀疏編碼的原子像是原型，每次只組合少數幾個。

線性自編碼器與 PCA

白話版。如果你訓練一個完全沒有非線性的神經網路去壓縮並重建輸入，它最後做的就是 PCA，只是座標可能被旋轉或拉伸。

對中心化的輸入，線性自編碼器計算

$$z = W_e x, \qquad \hat{x} = W_d z, \qquad \mathcal{L}(W_e, W_d) = \frac{1}{n}\sum_{i} \|x_i - W_d W_e x_i\|^2,$$

其中編碼器 $W_e \in \mathbb{R}^{k\times d}$、解碼器 $W_d \in \mathbb{R}^{d\times k}$，且 $k < d$。乘積 $W_d W_e$ 的秩至多為 k，所以由 Eckart–Young 定理，最好的乘積就是投影到前 k 個主成分子空間。Baldi 與 Hornik [8] 證明了更多：對這種平方誤差的線性網路，每個局部極小值都是全域極小值，而在全域極小值處，網路投影到主成分子空間。然而個別的權重矩陣並不唯一。對任意可逆的 $k \times k$ 矩陣 G，把 W_e 換成 GW_e、W_d 換成 $W_d G^{-1}$，乘積不變，所以編碼座標可以是 PCA 分數的任何可逆線性轉換。

程式：線性自編碼器重新找到 PCA 子空間

```
import numpy as np, torch
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from scipy.linalg import subspace_angles

torch.manual_seed(0)
X = load_digits().data / 16.
X = X - X.mean(axis=0)
Xt = torch.tensor(X, dtype=torch.float32)
k = 5
enc = torch.nn.Linear(64, k, bias=False)
dec = torch.nn.Linear(k, 64, bias=False)
opt = torch.optim.Adam([*enc.parameters(), *dec.parameters()], lr=1e-2)
for step in range(3000):
    loss = (dec(enc(Xt)) - Xt).square().mean()
    opt.zero_grad(); loss.backward(); opt.step()

Wd = dec.weight.detach().numpy()            # 64 x k decoder columns
P = PCA(n_components=k).fit(X).components_.T
angles = np.degrees(subspace_angles(Wd, P))
print("largest angle to PCA subspace (deg):", round(angles.max(), 3))
print("decoder columns orthonormal?",
      np.allclose(Wd.T @ Wd, np.eye(k), atol=1e-2))
```

輸出：

```
largest angle to PCA subspace (deg): 0.003
decoder columns orthonormal? False
```

子空間夾角（兩個 k 維子空間之間的最大夾角）基本上是零：網路找到了 PCA 子空間。但它的解碼器欄向量並不正交歸一，所以它們不是主成分本身。這就是座標不確定性的實際表現，也是本章反覆出現的主題的第一個線索：光靠重建，只能確定保留了什麼，無法確定如何標記它。

非線性自編碼器

白話版。把線性映射換成神經網路，編碼就能描述任何平坦子空間都無法捕捉的彎曲結構。

一般的自編碼器有編碼器 f_ϕ 與解碼器 g_θ，兩者都是神經網路：

$$z = f_\phi(x), \qquad \hat{x} = g_\theta(z), \qquad \min_{\phi,\theta} \frac{1}{n}\sum_i \ell(x_i,\, g_\theta(f_\phi(x_i))),$$

其中 ℓ 是重建損失函數，例如實數值用平方誤差，$[0,1]$ 之間的值用二元交叉熵。ℓ 的選擇本身就是一個假設：平方誤差對應以 $\hat{x}$ 為中心的高斯概似，交叉熵對應 Bernoulli 概似。

Hinton 與 Salakhutdinov [9] 在 2006 年展示：先以逐層貪婪預訓練初始化、再端到端微調的深度自編碼器，在相同維度下找到的低維編碼，重建影像的效果比 PCA 好得多。今天，更好的初始化、正規化層與最佳化器已讓預訓練步驟變得不必要，但核心教訓依然成立：當資料是彎曲的，非線性編碼器可以比線性編碼器壓縮得更徹底。

欠完備還不夠，過完備需要規則

欠完備自編碼器（$k < d$）被迫壓縮。這很有用，但並不保證能學到好特徵。夠強大的編碼器與解碼器在原理上可以把每個訓練樣本映到它自己的任意編碼（例如它的索引），再把解碼記下來，在沒學到任何可重用結構的情況下達到很低的訓練誤差 [1, 第 14 章]。**過完備**自編碼器（$k \geq d$）則可以直接學成恆等函數。兩種情況下，讓表示有用的都不是瓶頸本身，而是歸納偏誤（inductive bias）：有限的容量、架構（例如卷積），以及明確的懲罰項。下一節介紹三種經典懲罰，合稱**正則化自編碼器**（regularized autoencoders）。

正則化自編碼器：稀疏、去噪與收縮

每一種懲罰都用不同方式回答「編碼應該忽略什麼？」這個問題。

稀疏自編碼器

白話版。允許有很多編碼單元，但要求對任何一個輸入只有少數幾個啟動。

損失函數在編碼 $h = f_\phi(x)$ 上加一個稀疏懲罰：

$$\mathcal{L} = \ell(x,\, g_\theta(h)) + \lambda\,\Omega(h), \qquad \Omega(h) = \sum_j |h_j|.$$

其他做法包括：懲罰每個單元平均啟動率偏離一個小目標值的程度，或只保留最大的 k 個啟動值（「top-k」或 k-稀疏自編碼器）。稀疏自編碼器是稀疏編碼的「攤銷式」（amortized）表親：編碼器網路一次前向傳遞就預測出稀疏編碼，而不必為每個輸入解一次 lasso。Goodfellow 等人把這個懲罰解讀為編碼上的近似先驗，而不是一般的權重正則化 [1, 第 14 章]。正如現代觀點一節會說明的，稀疏自編碼器近年已成為解讀大型神經網路內部的關鍵工具。

去噪自編碼器

白話版。給網路看一個受損的輸入，要它重建出乾淨的版本。要做到這點，它必須學會乾淨的資料長什麼樣子。

去噪自編碼器（denoising autoencoder，DAE）先抽出一個受破壞的副本 $\tilde{x} \sim C(\tilde{x} \mid x)$，再最小化

$$\mathcal{L} = \mathbb{E}_{x}\, \mathbb{E}_{\tilde{x}\sim C(\cdot\mid x)} \big[\, \ell(x,\, g_\theta(f_\phi(\tilde{x}))) \,\big],$$

其中 C 是事先選定的破壞過程：加性高斯雜訊、遮罩雜訊（把隨機一部分輸入設為零），或椒鹽雜訊 [10], [11]。Vincent 等人在 2008 年提出這個想法 [10]，並在 2010 年證明把 DAE 層堆疊起來，能為分類器提供有用的特徵 [11]。由於輸入與目標不同，即使編碼是過完備的，直接複製也不再是解。

破壞過程決定了模型學會對什麼具有穩健性。用高斯雜訊訓練的 DAE 沒有理由去補齊缺失的區塊，而用遮罩雜訊訓練的 DAE 則會學到補全。現代的遮罩影像建模，例如ML 05 中的遮罩自編碼器（MAE），就是遮罩雜訊 DAE 的直系後代。

這裡還有更深一層的幾何解讀。對微小的高斯破壞，向量 $g(f(\tilde{x})) - \tilde{x}$ 會從受損的點指回資料密度較高的區域；它近似估計了**分數**（score）$\nabla_x \log p(x)$，也就是對數密度的梯度 [1, 第 14 章]。這個關聯是分數式與擴散生成模型的根源之一，詳見ML 07。

程式：在數字上訓練去噪自編碼器

這個獨立的 PyTorch 範例在訓練用的數字上訓練一個小型非線性 DAE，並評估從加了雜訊的保留輸入重建乾淨影像的效果。

```
import torch
from torch import nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0); torch.set_num_threads(1)
train, test = train_test_split(load_digits().data/16., test_size=.2, random_state=0)
X = torch.tensor(train, dtype=torch.float32)
Xt = torch.tensor(test, dtype=torch.float32)
encoder = nn.Sequential(nn.Linear(64, 32), nn.ReLU(), nn.Linear(32, 12))
decoder = nn.Sequential(nn.Linear(12, 32), nn.ReLU(), nn.Linear(32, 64), nn.Sigmoid())
model = nn.Sequential(encoder, decoder)
```

```
opt = torch.optim.Adam(model.parameters(), lr=.003)
for step in range(2000):
    clean = X[torch.randint(len(X), (64,))]
    noisy = (clean + .2*torch.randn_like(clean)).clamp(0, 1)
    loss = (model(noisy)-clean).square().mean()
    opt.zero_grad(); loss.backward(); opt.step()
model.eval()
with torch.no_grad():
    noisy = (Xt + .2*torch.randn_like(Xt)).clamp(0, 1)
    print("noisy-input MSE:", (noisy-Xt).square().mean().item())
    print("reconstructed MSE:", (model(noisy)-Xt).square().mean().item())
```

輸出（含雜訊輸入約 0.0248，經過 DAE 後約 0.0197；最後幾位數依 PyTorch 版本而異）：

```
noisy-input MSE: 0.024781296029686928
reconstructed MSE: 0.01968955807387829
```

改善是真實的，但幅度不大，圖 4 說明了原因：在 12 維的編碼下，DAE 去除了斑點雜訊，但也把一些筆畫細節抹平了。去噪與忠實壓縮往不同方向拉扯；更寬的編碼或卷積架構會改變兩者的平衡。

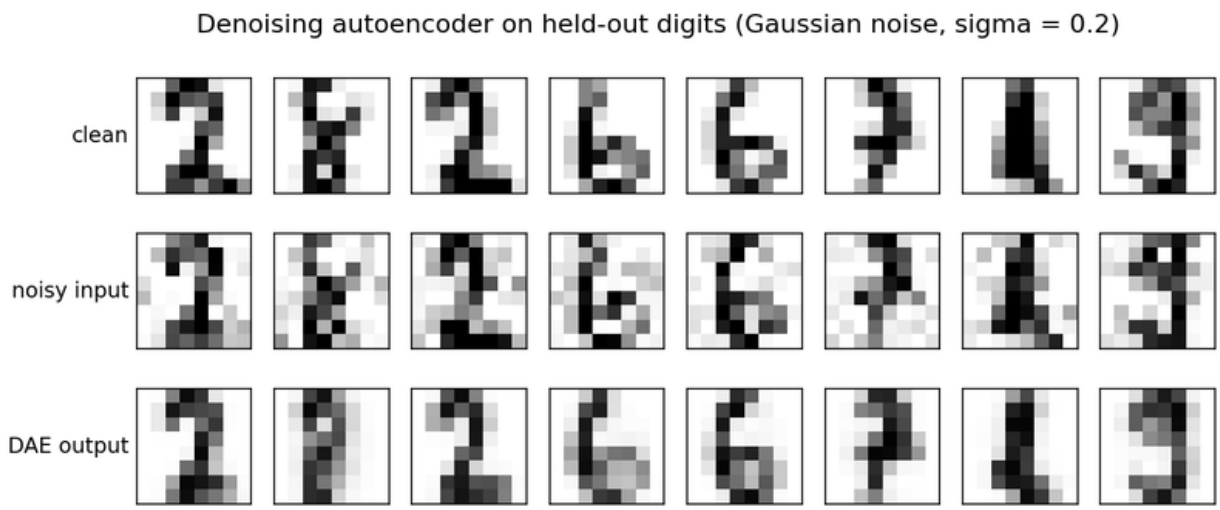


圖 *4* — 去噪自編碼器在保留數字上的輸出。背景斑點被去除；部分細微筆畫細節因 *12* 個單元的瓶頸而遺失。

收縮自編碼器

白話版。只要輸入的微小變化造成編碼的大幅變動，就懲罰編碼器。編碼應該只對重要的變化有反應。

Rifai 等人 [12] 加上編碼器 Jacobian 矩陣的 Frobenius 範數平方：

$$\mathcal{L} = \ell(x,\, g_\theta(f_\phi(x))) + \lambda \left\| \frac{\partial f_\phi(x)}{\partial x} \right\|_F^2, \qquad \left\| \frac{\partial f_\phi(x)}{\partial x} \right\|_F^2 = \sum_{j,i} \left(\frac{\partial h_j}{\partial x_i} \right)^2 .$$

其中 $h = f_\phi(x)$，Jacobian 元素 $\partial h_j / \partial x_i$ 衡量輸入座標 i 變動時，編碼單元 j 移動多少。對單層 sigmoid 編碼器 $h = \sigma(Wx + b)$，該元素為 $h_j(1 - h_j)W_{ji}$，所以懲罰項很容易計算：

$$\left\| \frac{\partial h}{\partial x} \right\|_F^2 = \sum_j \left(h_j(1 - h_j) \right)^2 \sum_i W_{ji}^2 .$$

重建項希望編碼隨輸入改變（否則無法重建不同的輸入），懲罰項則希望編碼盡量少變。折衷的結果是：編碼只沿著訓練資料實際變化的少數方向保持敏感。DAE 以隱含方式達到相關的效果：對微小高斯雜訊，去噪訓練懲罰的是整個重建函數的敏感度 [1, 第 14 章]；而 CAE 則明確地只懲罰編碼器。

程式：收縮懲罰的實際效果

```
import torch
from torch import nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

train, test = train_test_split(load_digits().data/16., test_size=.2, random_state=0)
X = torch.tensor(train, dtype=torch.float32)
Xt = torch.tensor(test, dtype=torch.float32)

def fit(lam):
    torch.manual_seed(0)
    W = nn.Linear(64, 16)                    # encoder h = sigmoid(Wx + b)
    dec = nn.Linear(16, 64)
    opt = torch.optim.Adam([*W.parameters(), *dec.parameters()], lr=3e-3)
    for step in range(1500):
        x = X[torch.randint(len(X), (64,))]
        h = torch.sigmoid(W(x))
        rec = (dec(h) - x).square().sum(1).mean()
        # ||J||_F^2 = sum_j (h_j(1-h_j))^2 * ||W_j||^2 for a sigmoid layer
        jac = ((h*(1-h))**2 * W.weight.square().sum(1)).sum(1).mean()
        loss = rec + lam*jac
        opt.zero_grad(); loss.backward(); opt.step()
    with torch.no_grad():
        h = torch.sigmoid(W(Xt))
        mse = (dec(h) - Xt).square().mean().item()
        jac = ((h*(1-h))**2 * W.weight.square().sum(1)).sum(1).mean().item()
    return mse, jac

for lam in (0.0, 0.5):
    mse, jac = fit(lam)
    print(f"lambda={lam}: test MSE={mse:.4f}, mean ||J||_F^2={jac:.2f}")
```

輸出：

```
lambda=0.0: test MSE=0.0141, mean ||J||_F^2=4.79
lambda=0.5: test MSE=0.0194, mean ||J||_F^2=0.70
```

懲罰項把編碼器的平均敏感度降低了將近七倍，代價是重建誤差略為上升。這筆交易值不值得，取決於下游用途：較平滑的編碼通常有助於最近鄰檢索或少標籤分類，而壓縮編解碼器則會偏好未加懲罰的模型。

流形觀點

白話版。真實資料並不會填滿它們所在的高維空間。它們聚集在一個又薄又彎曲的曲面附近，就像一張捲起來放在房間裡的紙。好的表示描述的是你在這張紙上的位置。

流形假說（manifold hypothesis）認為，影像等自然資料集中在嵌入高維輸入空間中的低維流形附近 [3]。一個隨機的 64 像素陣列幾乎不可能像一個數字；合理數字的集合是一個極小而有結構的區域。在任何資料點附近，流形局部看起來像一個平面，即**切空間**（tangent space），由少數幾個能讓你保持「真實」的移動方向張成（筆畫變粗、稍微傾斜、小幅平移）。與切空間正交的方向則會離開流形、進入雜訊。

這個觀點為本章所有內容提供了單一的視角：

- **PCA** 擬合一張全域的平面紙。流形接近平坦時有效，彎曲時就失效。
- **PPCA 或 FA 的混合模型**擬合許多張局部平面紙。
- **非線性自編碼器**學到流形的彎曲座標圖：編碼器給出紙上的座標，解碼器再把座標映回房間裡。
- **去噪**教模型把掉出流形的點移回流形上。
- **收縮**懲罰讓編碼器對垂直於流形的方向不敏感，而重建項讓它沿著切方向保持敏感 [1, 第 14 章], [12]。

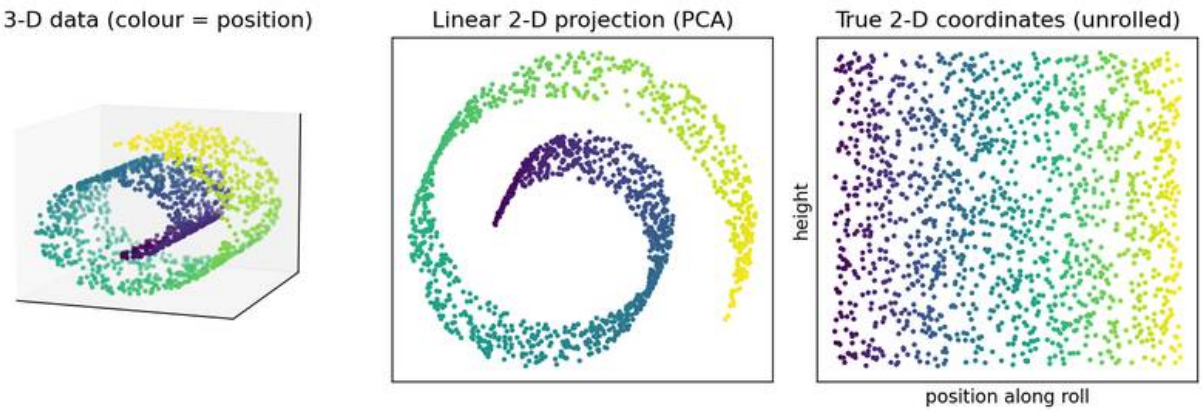


圖 *5* — 瑞士捲是一張捲在三維空間裡的二維紙。線性投影（*PCA*）把它壓成螺旋，無法保留沿紙面的位置；真實的內在座標則是一個平坦的矩形。

圖 5 讓線性方法的限制變得具體。瑞士捲有兩個內在維度——沿捲方向的位置與高度——但沒有任何二維線性投影能把它攤開：PCA 的視圖保留了螺旋，紙面上相距很遠的點並沒有依序排開。非線性編碼器原則上可以學到攤開後的座標。實務上能否做到，取決於容量、正則化與資料量；高維空間中的流形學習至今仍然困難。

從自編碼器到 VAE

白話版。一般的自編碼器學會壓縮與解壓縮，卻沒有提供憑空創造新編碼的規則。VAE 加上了這條規則。

假設訓練好的自編碼器把每個訓練用的數字映射到 $\mathbb{R}^{12}$ 中的一個編碼。為了生成新數字，我們想隨機挑一個編碼再解碼。但要挑哪個隨機編碼？訓練編碼可能占據一個奇怪、成團的區域，而它們之間的隨機點解碼後可能是一團亂。重建損失函數從來沒有要求編碼空間平滑、填滿，或符合任何已知分布。所以極佳的重建並不代表良好的生成。

變分自編碼器（variational autoencoder）[13] 的解法是回到 PPCA 與 FA 的機率因子模型故事，只是改用神經網路解碼器：

$$z \sim p(z) = \mathcal{N}(0, I), \qquad x \sim p_\theta(x \mid z),$$

其中 $p_\theta(x \mid z)$ 是一個分布，其參數由解碼器網路產生。由於真正的後驗 $p_\theta(z \mid x)$ 此時已無法計算，就由第二個網路——編碼器——輸出近似後驗 $q_\phi(z \mid x)$，通常是對角高斯分布。訓練時最大化 $\log p_\theta(x)$ 的**證據下界**（evidence lower bound，ELBO）：

$$\log p_\theta(x) \;\geq\; \underbrace{\mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big]}_{\text{reconstruction}} \;-\; \underbrace{D_{\mathrm{KL}}\big(q_\phi(z \mid x) \,\|\, p(z)\big)}_{\text{keep codes near the prior}}.$$

對照前面所有內容來讀這兩項：

- **重建**項（reconstruction）正是自編碼器的損失，只是現在在編碼中注入了雜訊。高斯解碼器下它變成平方誤差，Bernoulli 解碼器下則是二元交叉熵。
- **KL** 項（讓編碼靠近先驗）是一個正則化項，就像稀疏與收縮懲罰，但它的設計讓所有訓練樣本的編碼合起來填滿先驗分布。這正是從 $z \sim \mathcal{N}(0, I)$ 取樣之所以有意義的原因。
- 若使用**線性**高斯解碼器，這個模型就是 PPCA。它的精確後驗是高斯分布，可以被線性高斯編碼器完全匹配，所以下界變成等號，最佳解與 PPCA 的最大概似解一致。VAE 是我們一開始那些因子模型的非線性推廣。

Kingma 與 Welling 的導論 [14] 詳細發展了這個關聯。把 KL 項乘上 $\beta > 1$ 就得到 **β-VAE** [15]，它的提出是為了以重建品質為代價、鼓勵學到解耦（disentangled）的因子；請注意當 $\beta \neq 1$ 時，目標函數就不再是 ELBO。ELBO 的完整推導、重參數化技巧，以及後驗崩塌（posterior collapse）等失效模式，請見ML 06。

重建誤差能告訴你什麼、不能告訴你什麼

離開自編碼器之前，先給一個實務上的警告。重建誤差常被當作異常分數：用正常資料訓練的自編碼器，應該能把正常輸入重建得很好、把異常輸入重建得很差。這有時行得通，但當自編碼器

泛化得太好、連異常輸入也重建得不錯，或異常只存在於瓶頸本來就會丟掉的細節時，就會失敗。同樣地，重建得好的編碼可能不利於分類，而分類效果好的編碼可能丟掉了影像復原任務所需的像素細節。壓縮、去噪、下游效用與生成，請分開評估。

現代觀點

白話版。本章的古典模型並沒有消失，而是變成了零件：生成模型的壓縮階段、視覺的預訓練目標，以及觀察大型網路內部的顯微鏡。

值得一讀的綜述

- **Bengio、Courville 與 Vincent（2013）** [3] 是表示學習的標準回顧。它用一串通用先驗來界定好的表示（平滑性、多重解釋因子、概念的階層、跨任務共享的因子、流形假說、稀疏性等），並在這個框架下串起自編碼器、機率模型與流形學習。想知道本章那些懲罰項為什麼存在，它是很好的地圖。
- **Bank、Koenigstein 與 Giryes（2020）** [16] 是一篇精簡的自編碼器綜述章節。它依序介紹正則化變體（稀疏、去噪、收縮）、VAE 與解耦變體、各種應用（生成建模、分類、分群、異常偵測、推薦系統與降維），以及結合對抗式訓練的混合方法，是本篇很好的補充讀物。
- **Kingma 與 Welling（2019）** [14] 是作者親自撰寫的 VAE 延伸導論，涵蓋 ELBO、重參數化技巧、更豐富的後驗分布與階層模型，並把 VAE 放進其他深度潛在變數模型的脈絡中。
- **Goodfellow、Bengio 與 Courville** [1, 第 13–14 章] 與 **Murphy** [2] 仍是線性因子模型、DAE 與分數的關聯，以及流形詮釋最清楚的教科書說明。

解耦：一個冷靜的結果

一個反覆出現的期望是：非監督式因子模型能找回那組真正的潛在因子，每個編碼單元對應一個（「解耦」，disentanglement）。Locatello 等人 [17] 從理論上證明，若對模型與資料都沒有歸納偏誤，非監督式地學習解耦表示在根本上是不可能的，這在更大的尺度上呼應了因子分析的旋轉不確定性。他們在一個大規模、可重現的研究中也發現：沒有監督就無法辨認出解耦良好的模型，而且更高的解耦程度並不會可靠地降低下游任務的樣本複雜度。實務教訓與 ICA 相同：可識別性永遠來自額外的假設，所以請把假設講清楚。

自編碼器作為 tokenizer

自編碼器在現代最重要的角色，是大型生成模型的第一階段。**VQ-VAE** [18] 把連續編碼換成一本學習出來的離散向量碼本，使影像變成離散 token 的網格，供自迴歸模型預測（見ML 08）。**潛在擴散**（latent diffusion）[19] 先訓練一個做感知壓縮的自編碼器，再在它的潛在空間、而不是像素空間中執行擴散模型，大幅降低運算量。兩者都受到自編碼器重建品質的上限限制：生成器無法還原編碼器已經丟掉的細節。如何設計這些 tokenizer——在壓縮、重建，以及潛在空間是否容易被生成器建模之間取得平衡——在 2026 年仍是活躍的研究領域。

用於可解釋性的稀疏自編碼器（2023–2026）

稀疏自編碼器找到了一個嶄新且非常活躍的應用：大型語言與視覺模型的**機制可解釋性**（mechanistic interpretability）。出發點是一個觀察：單一神經元往往是多義的（polysemantic），會對好幾個不相關的概念產生反應。Cunningham 等人 [20] 訓練過完備的稀疏自編碼器來重建語言模型的內部啟動值，找到一些稀疏啟動的特徵，自動化評估認為它們比神經元或其他分解方式更具可解釋性、更單義。Gao 等人 [21] 使用 *k*-稀疏（top-*k*）自編碼器直接控制稀疏程度，報告了自編碼器大小與稀疏程度的乾淨縮放定律，提出評估特徵品質的新指標，並在 GPT-4 的啟動值上訓練了一個有 1,600 萬個潛在單元的自編碼器。概念上，這就是把稀疏編碼（Olshausen 與 Field 的想法）套用到網路的啟動值，而不是影像區塊。

開放問題

- **評估**。重建誤差、稀疏程度與下游準確率彼此不一致，而且對於學到的特徵是否「正確」——無論是解耦還是可解釋性——都沒有公認的衡量方式。
- **可識別性**。什麼時候可以相信學到的因子對應真實的成因？非線性 ICA 與因果表示學習研究的是哪些額外假設（時間結構、輔助變數、介入）能讓這件事成立。
- **Tokenizer 設計**。要壓縮多少、編碼該是離散還是連續、如何讓潛在表示既利於重建又容易建模，仍是開放的工程與理論問題。

重點整理

- 本章所有模型共用一個範本：編碼器映到編碼、解碼器映回來，再加上一個決定偏好哪種編碼的假設。
- PCA 最小化線性重建誤差；其方向是共變異數的特徵向量，可由中心化資料的 SVD 求得。
- PPCA 加入等向高斯雜訊，並以最大概似找回 PCA 子空間；因子分析允許每個座標有各自的雜訊水準。兩者都能確定子空間，但無法確定座標軸。
- ICA 假設來源獨立且非高斯，藉此確定座標軸（只剩順序、正負號與尺度的不確定性）；稀疏編碼與 NMF 則以稀疏性與非負性挑選因子。
- 線性自編碼器找回的是 PCA 子空間，而不是唯一的 PCA 權重。非線性自編碼器需要瓶頸及／或懲罰（稀疏、去噪、收縮）才能學到有用的結構。
- 流形觀點把這一切統一起來：好的編碼沿著資料的低維曲面移動，並忽略離開曲面的方向。
- 重建本身不提供取樣器。VAE 加入先驗與近似後驗；若解碼器是線性的，它就退化為 PPCA。

練習

1. **雜訊假設**。你記錄了 20 個感測器，其中 5 個的雜訊遠大於其他感測器，但它們全都反映同樣幾個潛在成因。你會從 PPCA 還是因子分析開始？PCA 會怎麼處理那些雜訊大的感測器？

   提示

選因子分析：它的對角 Ψ 可以把較大的殘差變異數分配給那 5 個雜訊大的感測器，而不必為它們花費因子。PCA（以及只有單一共享 σ^2 的 PPCA）可能會把成分花在高變異數的雜訊感測器上，因為 PCA 眼中只有變異數。

2. **潛在不確定性**。對可逆的 $k \times k$ 矩陣 G，證明把 W_e 換成 GW_e、W_d 換成 $W_d G^{-1}$，線性自編碼器的重建結果不變。哪些矩陣 G 也能保持 PPCA 的概似函數不變？

提示

乘積為 $W_d G^{-1} G W_e = W_d W_e$。對 PPCA，概似函數取決於 $WW^\top$；把 W 換成 WG 得到 $WGG^\top W^\top$，當 $GG^\top = I$，也就是 G 為正交矩陣時不變。

3. **為什麼 ICA 對高斯來源失效**。產生兩個獨立的標準高斯來源，用任意可逆的 2×2 矩陣混合，再用不同隨機種子執行 FastICA 數次。你觀察到什麼？為什麼？

提示

白化之後，兩個獨立單位高斯的任何旋轉仍是兩個獨立單位高斯，所以沒有「正確」的旋轉可找。不同種子會給出不同但同樣合理的解混方向，與真實來源的相關係數每次都不一樣。

4. **稀疏編碼與 PCA 編碼**。使用稀疏編碼的程式，比較每張測試影像的非零編碼元素數量，與相同維度（32）的 PCA 編碼。接著把 `alpha` 設為 0.05、0.1、0.3、1.0，畫出稀疏編碼的測試 MSE。

提示

PCA 編碼是稠密的：32 個元素基本上全都非零。隨著 `alpha` 增大，啟動的原子數下降、誤差上升，畫出一條稀疏程度與精確度的取捨曲線。記得字典沒有平均值項，所以請比較曲線，而不是拿單一數字與 PCA 比。

5. **去噪的不匹配**。用高斯雜訊訓練本篇的 DAE，再拿隨機一個 3×3 區塊被設為零的輸入來測試。接著用這種遮罩破壞訓練第二個 DAE，並把兩個模型在兩種破壞下都測一遍。

提示

這是兩種不同的破壞過程。用高斯雜訊訓練的模型從沒被要求補齊缺失的結構，所以通常會留下空洞；用遮罩訓練的模型則學會補全。請列出一張 2×2 的表（訓練破壞 × 測試破壞），而不是從其中一種的成功推論出對另一種的穩健性。

6. **手算收縮懲罰**。對單一 sigmoid 單元 $h = \sigma(w^\top x + b)$，推導 $\|\partial h / \partial x\|^2$。它何時最大？這說明收縮自編碼器允許編碼在什麼地方保持敏感？

提示

$\partial h / \partial x = h(1-h)w$，所以平方範數為 $h^2(1-h)^2\|w\|^2$。它在 $h = 1/2$（sigmoid 中段最陡處）最大，單元飽和時則幾乎為零。這個懲罰把單元推向對大多數輸入都飽和的狀態，只在資料確實需要變化的地方留在敏感區間。

參考文獻

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 13–14. Book
2. K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022. Book
3. Y. Bengio, A. Courville and P. Vincent, "Representation Learning: A Review and New Perspectives," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 8, 2013. Paper
4. M. E. Tipping and C. M. Bishop, "Probabilistic Principal Component Analysis," *Journal of the Royal Statistical Society: Series B*, vol. 61, no. 3, 1999. Paper
5. A. Hyvärinen and E. Oja, "Independent component analysis: algorithms and applications," *Neural Networks*, vol. 13, no. 4–5, 2000. Paper
6. B. A. Olshausen and D. J. Field, "Emergence of simple-cell receptive field properties by learning a sparse code for natural images," *Nature*, vol. 381, 1996. Paper
7. D. D. Lee and H. S. Seung, "Learning the parts of objects by non-negative matrix factorization," *Nature*, vol. 401, 1999. Paper
8. P. Baldi and K. Hornik, "Neural networks and principal component analysis: Learning from examples without local minima," *Neural Networks*, vol. 2, no. 1, 1989. Paper
9. G. E. Hinton and R. R. Salakhutdinov, "Reducing the Dimensionality of Data with Neural Networks," *Science*, vol. 313, 2006. Paper
10. P. Vincent, H. Larochelle, Y. Bengio and P.-A. Manzagol, "Extracting and composing robust features with denoising autoencoders," ICML, 2008. Paper
11. P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio and P.-A. Manzagol, "Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion," *JMLR*, vol. 11, 2010. Paper
12. S. Rifai, P. Vincent, X. Muller, X. Glorot and Y. Bengio, "Contractive Auto-Encoders: Explicit Invariance During Feature Extraction," ICML, 2011. Paper
13. D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," ICLR, 2014. Paper
14. D. P. Kingma and M. Welling, "An Introduction to Variational Autoencoders," *Foundations and Trends in Machine Learning*, vol. 12, no. 4, 2019. Paper
15. I. Higgins et al., "β-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework," ICLR, 2017. Paper
16. D. Bank, N. Koenigstein and R. Giryes, "Autoencoders," survey chapter, arXiv:2003.05991, 2020. Survey
17. F. Locatello et al., "Challenging Common Assumptions in the Unsupervised Learning of Disentangled Representations," ICML, 2019. Paper
18. A. van den Oord, O. Vinyals and K. Kavukcuoglu, "Neural Discrete Representation Learning," NeurIPS, 2017. Paper
19. R. Rombach, A. Blattmann, D. Lorenz, P. Esser and B. Ommer, "High-Resolution Image Synthesis with Latent Diffusion Models," CVPR, 2022. Paper
20. H. Cunningham, A. Ewart, L. Riggs, R. Huben and L. Sharkey, "Sparse Autoencoders Find Highly Interpretable Features in Language Models," arXiv:2309.08600, 2023. Paper
21. L. Gao et al., "Scaling and evaluating sparse autoencoders," arXiv:2406.04093, 2024. Paper

ML 06・生成模型：自編碼器、VAE、GAN 與正規化流

從學習標籤走向學習分布：概似式與隱式模型、ELBO 與重參數化、對抗訓練與 Wasserstein GAN、正規化流，以及生成模型該如何評估。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-06-generative-models/

你將學到

- 建模一個分布 $p(x)$ 和預測標籤有什麼不同，以及為什麼最大概似等同於最小化 KL 散度。
- 兩大家族：寫得出（或能界定）$\log p(x)$ 的**概似式模型（likelihood-based）**，以及只提供取樣器的**隱式模型（implicit）**。
- VAE 如何把無法計算的積分變成可訓練的下界（ELBO），以及重參數化技巧為什麼讓它可以微分。
- GAN 如何透過判別器學習、原始目標函數其實在最小化什麼、模式崩塌（mode collapse）為何發生，以及 Wasserstein GAN 改變了什麼。
- 正規化流如何用變數變換公式算出精確概似，以及能量模型、自迴歸模型與擴散模型在地圖上的位置。
- 如何用概似、Inception Score、FID 與精確率／召回率評估生成器，以及每個指標**無法**告訴你什麼。

先看全貌

分類器問：「這張影像裡有什麼？」**生成模型（generative model）**問：「可能出現哪些影像？」如果它學到有用的分布，我們就能取樣出新的例子、補上缺失區域、壓縮資料、偵測異常輸入，或依文字、類別標籤、另一張影像來條件式地生成。

> **用最簡單的話說**
>
> 會認房子的小朋友，看到圖可以說「房子」。懂得房子怎麼畫的小朋友，則能自己畫出一棟有門、有窗、有屋頂的新房子。生成模型想學的是例子背後的「食譜」，好煮出新的菜——而且好廚師每次端出來的菜都不一樣，但都好吃，而不是同一道菜一直重複。

學分布比學標籤難得多。標籤只是一個數字；一張影像有成千上萬個像素，彼此以複雜的方式相互依賴。本章的每個家族，都是對同一個工程問題的不同回答：要怎麼表示一個極高維資料上的分布，才能既從例子**訓練**它，之後又能從它**取樣**？

本章採用 Bishop 與 Bishop [1] 以及 Prince [2] 發展的機率觀點。Bond-Taylor 等人的比較綜述 [3] 依照我們將遇到的取捨來整理這些家族——訓練速度、樣本品質、多樣性，以及是否能得到概似；Papamakarios 等人的正規化流綜述 [4] 則特別有助於區分「學到的分布」與「方便的取樣程序」。擴散模型與自迴歸模型各有專章（ML 07 與ML 08），因為它們的訓練與取樣值得仔細討論；這裡先把它們放上地圖。

用分布描述資料

白話版。找一個機率模型，讓觀察到的例子在它之下看起來「很可能」，再用這個模型產生看起來像出自同一來源的新例子。

最大概似

假設資料 $x_1, \dots, x_n$ 是從未知分布 p_{data} 獨立抽出的樣本，p_θ 是參數為 θ 的模型。最大概似估計最小化平均**負對數概似（negative log-likelihood，NLL）**：

$$\mathcal{L}_{\text{NLL}}(\theta) = -\frac{1}{n}\sum_{i=1}^{n} \log p_\theta(x_i).$$

其中 n 是訓練樣本數，$\log p_\theta(x_i)$ 是模型給第 i 個樣本的對數密度。

為什麼這是合理的目標？當 n 變大，平均值會趨近於對資料分布的期望值，而且

$$\mathbb{E}_{x\sim p_{\text{data}}}[-\log p_\theta(x)] = D_{\text{KL}}(p_{\text{data}} \,\|\, p_\theta) + H(p_{\text{data}}).$$

$D_{\text{KL}}(p\|q) = \mathbb{E}_p[\log p - \log q]$ 是 Kullback–Leibler 散度，只有兩個分布相同時才為零；$H(p_{\text{data}})$ 是資料的熵，與 θ 無關。因此**最大概似就等於最小化從資料到模型的前向 KL 散度**。

KL 的方向很重要。前向 KL 會重罰「在真實資料所在之處給出接近零的密度」的模型，所以概似訓練是「涵蓋質量（mass-covering）」的：它寧可把機率攤到每一個模態（mode）上，即使因此在模態之間也放了一些機率。請記住這點；稍後會看到 GAN 的行為不同。

密度、離散像素與每維位元數

對連續值而言，$p_\theta(x)$ 是**密度**，不是「恰好觀察到 x」的機率；密度可以大於 1。然而真實影像存的是 0–255 這類整數。直接用連續密度去擬合整數是病態問題：模型可以在整數格點上放無限高的尖峰，讓概似趨於無窮大。標準解法是**去量化（dequantization）**——在擬合連續密度之前，對每個整數值加上 $[0, 1)$ 的均勻雜訊。Theis 等人 [5] 證明，連續模型在去量化資料上的對數概似，會是對應離散模型對數概似的下界，這讓不同論文的數字可以相互比較。另一種作法是直接使用離散概似（對 256 個值做 softmax，或像下方程式碼一樣對二值像素用 Bernoulli 分布）。

影像的概似通常以**每維位元數（bits per dimension）**回報：

$$\text{bpd} = \frac{\mathcal{L}_{\text{NLL}}}{D \ln 2},$$

其中 $\mathcal{L}_{\text{NLL}}$ 以每張影像的 nat 為單位，D 是像素值的個數（高 × 寬 × 通道數）。每維位元數就是「以此模型為基礎的無損壓縮器」平均每個子像素需要的位元數，因此概似與壓縮是一體兩面。

條件式與無條件生成

判別式分類器建模 $p(y \mid x)$。條件式生成器建模 $p(x \mid c)$，其中 c 可以是文字、類別、低解析度影像或已觀察到的像素。貝氏定理把相關的量連在一起，但分類器本身並不提供影像取樣器。

條件化也解釋了一個常見的失敗。最小化像素 MSE 會預測**條件平均** $\mathbb{E}[x \mid c]$。如果一個模糊的輸入有好幾個合理輸出——例如一張灰階照片裡的襯衫可能是紅色也可能是藍色——平均值會把它們混成褐色。生成模型則嘗試表示整個分布 $p(x \mid c)$，一次取樣出一個合理的答案。

兩大家族：概似式與隱式

以下每個模型都需要兩種能力：從資料**訓練**的方法，以及**取樣**的方法。它們的差別在於能不能計算 $p_\theta(x)$。

- **概似式（顯式）模型**寫下密度或其界限，再加以最大化。自迴歸模型與正規化流給出精確概似；VAE（以及從變分角度看的擴散模型）最佳化一個下界；能量模型只定義到一個未知正規化常數為止的密度。
- **隱式模型**只定義一個取樣器——「把雜訊推過一個網路」——而沒有 $p_\theta(x)$ 的公式。GAN 是主要例子。因為沒有概似可以最大化，它們必須用別的方式比較樣本與資料來訓練。

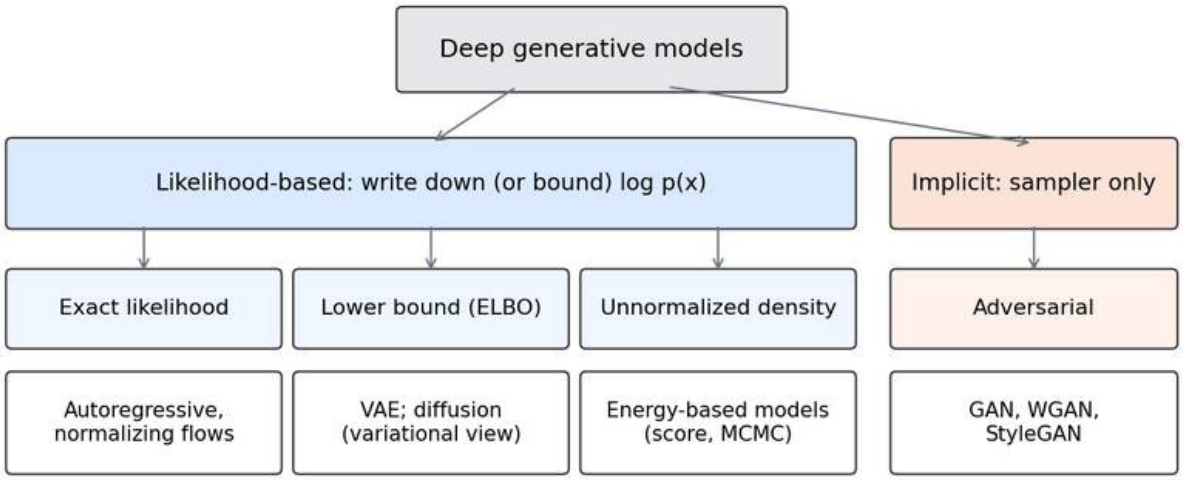


圖 *1* — 生成模型的教學用分類，採用比較綜述 [3] 的顯式／隱式劃分。實際系統常結合多個分支。

一般自編碼器：有壓縮、沒有先驗

白話版。把一個例子編碼成精簡的潛在碼，再解碼回來。這對壓縮與表示學習非常好，但它本身並不告訴你怎麼「發明」新的潛在碼。

編碼器 $z = f_\phi(x)$ 與解碼器 $\hat{x} = g_\theta(z)$ 最小化 $\|x - \hat{x}\|^2$ 之類的重建損失。自編碼器、它們的線性親戚（PCA、因子分析）及各種正則化變體，在ML 05A 中有完整介紹。本章要強調的只有一點：一般自編碼器並不要求潛在碼**落在哪裡**。訓練資料的潛在碼可能只佔據潛在空間中一塊又薄又怪的區域。若你接著取樣 $z \sim \mathcal{N}(0, I)$ 再解碼，可能落在解碼器從沒見過的空白區域，得到一團垃圾。因此重建再好，也不保證無條件生成的品質。

VAE 透過「給潛在碼一個先驗分布」，把自編碼器變成機率模型來解決這個問題。

VAE：潛在變數機率模型

白話版。想像每張影像的產生過程是：先從一個簡單分布隨機挑幾個隱藏的「設定」（姿態、筆畫粗細、風格），再根據這些設定加上一點雜訊畫出影像。VAE 同時學習這個「繪製器」（解碼器），以及一個快速的「猜測器」（編碼器）：讀一張影像，提出它大概是由哪些設定產生的。

模型

選一個潛在向量 $z \in \mathbb{R}^k$ 的先驗 $p(z)$，通常是標準高斯 $\mathcal{N}(0, I)$，以及解碼器 $p_\theta(x \mid z)$——一個輸出影像分布參數的神經網路（二值像素用 Bernoulli 機率，或高斯平均值）。模型對影像的密度是邊際分布

$$p_\theta(x) = \int p_\theta(x \mid z)\, p(z)\, dz.$$

取樣很容易：先抽 $z \sim p(z)$，再抽 $x \sim p_\theta(x \mid z)$。計算 $p_\theta(x)$ 則很難：積分要跑過所有可能的潛在碼，而解碼器是非線性網路，沒有封閉解。真正的後驗 $p_\theta(z \mid x) = p_\theta(x \mid z)p(z)/p_\theta(x)$ 也因此無法計算。

推導證據下界

Kingma 與 Welling [6] 引入一個近似後驗的**編碼器**（或稱推論網路）$q_\phi(z \mid x)$。以下逐步推導。因為 $\log p_\theta(x)$ 與 z 無關，我們可以對任何 z 的分布取期望而不改變它的值：

$$\begin{aligned}
\log p_\theta(x) &= \mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x)\big] \\
&= \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p_\theta(x,z)}{p_\theta(z \mid x)}\right] \\
&= \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p_\theta(x,z)}{q_\phi(z \mid x)}\right] + \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{q_\phi(z \mid x)}{p_\theta(z \mid x)}\right] \\
&= \mathrm{ELBO}(x) + D_{\mathrm{KL}}(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)).
\end{aligned}$$

第二行用了 $p_\theta(x) = p_\theta(x,z)/p_\theta(z \mid x)$；第三行在對數裡同乘、同除 $q_\phi(z \mid x)$。因為 KL 散度永遠不為負，第一項是 $\log p_\theta(x)$ 的下界：**證據下界（evidence lower bound，ELBO）**。下界與真實對數概似之間的**差距**，恰好是近似後驗與真實後驗之間的 KL 散度。編碼器越好，下界越緊；完美的編碼器會讓差距歸零。

把 $p_\theta(x,z) = p_\theta(x \mid z)p(z)$ 展開，得到程式中使用的形式：

$$\mathrm{ELBO}(x) = \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)]}_{\text{reconstruction}} - \underbrace{D_{\mathrm{KL}}(q_\phi(z \mid x) \,\|\, p(z))}_{\text{stay close to the prior}}.$$

第一項（重建）獎勵能讓解碼器重現 x 的潛在碼；第二項（貼近先驗）懲罰分布偏離先驗太遠的潛在碼。用 Jensen 不等式（$\log \mathbb{E} \geq \mathbb{E} \log$）一行就能得到同樣的下界，但上面的分解更有用，因

為它明確指出了差距是什麼。訓練時對 θ 與 ϕ 同時最大化資料上的平均 ELBO，也就是最小化**負 ELBO**。

當編碼器是對角高斯 $q_\phi = \mathcal{N}(\mu, \mathrm{diag}(\sigma^2))$、先驗是標準高斯時，KL 項有封閉解：

$$D_{\mathrm{KL}}(q_\phi \| p) = \frac{1}{2} \sum_{j=1}^{k} \left(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1 \right),$$

其中 μ_j 與 σ_j^2 是編碼器在第 j 個潛在座標上的平均值與變異數。每一項只有在 $\mu_j = 0$ 且 $\sigma_j^2 = 1$ 時才為零。

重參數化

重建項是對 $z \sim q_\phi(z \mid x)$ 的期望值，而我們需要它對 ϕ 的梯度。照字面寫的取樣無法微分：隨機抽樣是個黑盒子。**重參數化技巧（reparameterization trick）** [6] 把隨機性移到一個固定的雜訊變數上：

$$\epsilon \sim \mathcal{N}(0, I), \qquad z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon.$$

$\odot$ 是逐元素相乘。現在 z 是 ϕ 與獨立雜訊 ϵ 的確定性、可微函數，一般的反向傳播就能透過 μ 與 σ 給出低變異的梯度估計。程式中編碼器預測 `logvar` $= \log \sigma^2$，再用 `exp(.5 * logvar)` 當標準差，不需任何限制就能保證為正。

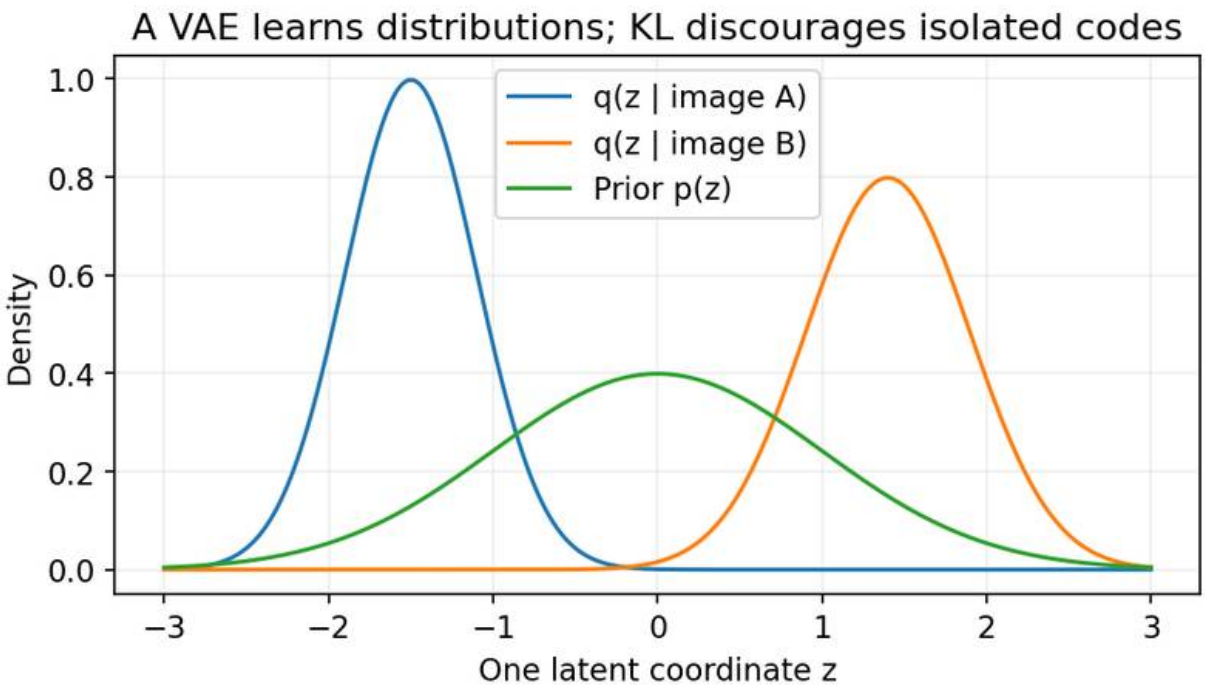


圖 2 — *VAE* 以分布表示潛在碼的不確定性，並鼓勵它與共用的先驗相容。曲線為示意。

程式：二值手寫數字的小型 VAE

這個完整的 CPU 範例使用 scikit-learn 內建的 8×8 手寫數字，並先二值化，讓 Bernoulli 解碼器成為明確的概似模型。重建項對像素加總、KL 對潛在維度加總，然後兩者都對樣本取平均；若只偷偷對其中一項取平均，就會改變兩者的比重。

```
import torch
from torch import nn
from torch.nn import functional as F
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0)
torch.set_num_threads(1)
d = load_digits()
train, test = train_test_split(d.data, test_size=.2, random_state=0,
                               stratify=d.target)
X = torch.tensor((train > 8).astype("float32"))
Xt = torch.tensor((test > 8).astype("float32"))
enc = nn.Sequential(nn.Linear(64, 64), nn.ReLU(), nn.Linear(64, 16))
dec = nn.Sequential(nn.Linear(8, 64), nn.ReLU(), nn.Linear(64, 64))
opt = torch.optim.Adam(list(enc.parameters()) + list(dec.parameters()), lr=.002)

def terms(x):
    mu, logvar = enc(x).chunk(2, dim=-1)
    z = mu + (.5*logvar).exp() * torch.randn_like(mu)
    recon = F.binary_cross_entropy_with_logits(dec(z), x, reduction="none").sum(1)
    kl = .5*(mu.square() + logvar.exp() - logvar - 1).sum(1)
    return recon, kl

for step in range(400):
    x = X[torch.randint(len(X), (64,))]
    recon, kl = terms(x)
    loss = (recon + kl).mean()
    opt.zero_grad()
    loss.backward()
    opt.step()
enc.eval(); dec.eval()
with torch.no_grad():
    recon, kl = terms(Xt)
    samples = dec(torch.randn(4, 8)).sigmoid().reshape(4, 8, 8)
    print("test negative ELBO estimate:", round((recon + kl).mean().item(), 3))
    print("reconstruction / KL:", round(recon.mean().item(), 3),
          round(kl.mean().item(), 3))
    print("sample probability maps:", tuple(samples.shape))
```

在我們的機器上，它印出測試集負 ELBO 約為每張影像 20.5 nats，其中重建約 17.0 nats、KL 約 3.5 nats。因為 ELBO 是對數概似的下界，20.5 nats 就是測試 NLL 的**上界**。輸出是 Bernoulli 機率圖；若要得到二值樣本，就依這些機率對每個像素取樣。測試估計帶有隨機性，多取幾次潛在樣本再平均會更穩定。這裡沒有用測試集挑選檢查點。

圖 3 顯示同樣資料上、潛在空間只有**兩維**的 VAE（由圖檔腳本訓練更久）學到了什麼。即使從沒用到標籤，相同類別的數字仍落在彼此附近；對一個規則網格上的 z 解碼，數字形狀會平滑地漸變。這正是一般自編碼器缺少的性質：先驗主體附近的每一點，解碼出來都像個數字。

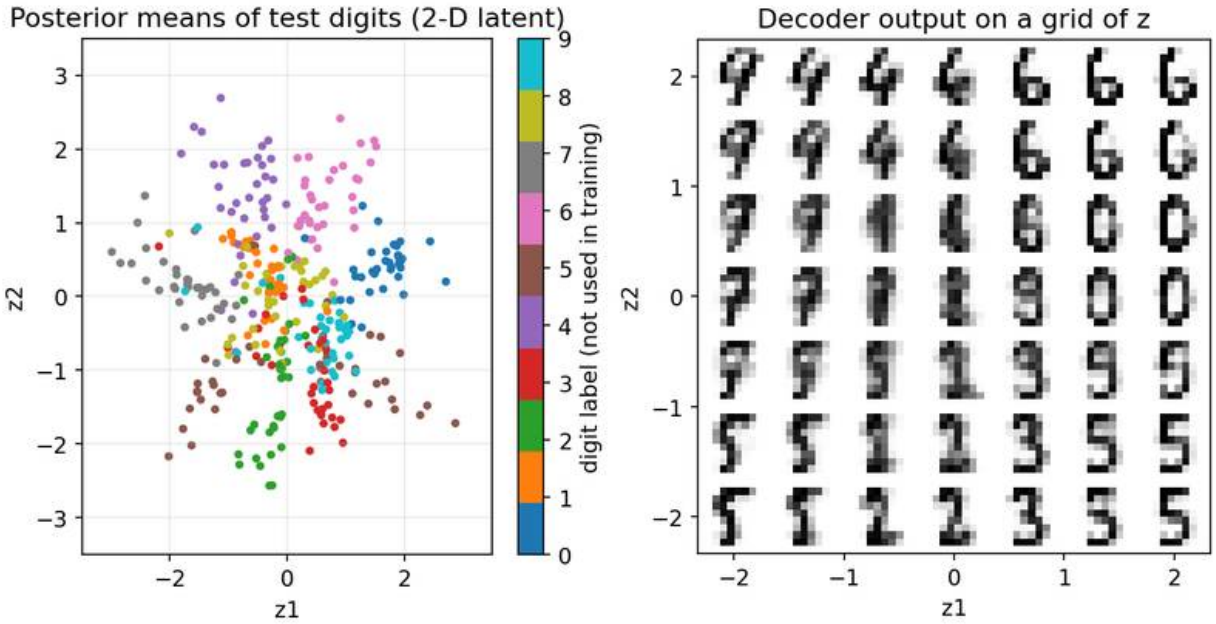


圖 3 — 在二值化 8×8 數字上訓練、潛在空間為 2 維的 *VAE*。左：測試影像的編碼器平均值（標籤只用於著色）。右：在潛在值網格上的解碼器輸出。

後驗崩塌、β-VAE 與更深的 VAE

如果解碼器非常強大（例如本身就能建模像素的自迴歸解碼器），提高 ELBO 最便宜的方式可能是完全忽略 z：令 $q_\phi(z \mid x) = p(z)$ 使 KL 項為零，讓解碼器包辦一切。這就是**後驗崩塌（posterior collapse）**。Bowman 等人 [7] 在文字 VAE 上觀察到這個現象，並提出 **KL 退火（KL annealing）**：一開始給 KL 項很小的權重，訓練中逐步升到 1，讓模型在懲罰生效前先學會使用潛在碼。監看每個潛在維度的 KL 是最簡單的診斷——KL 接近零的維度不攜帶任何資訊。

Higgins 等人 [8] 則反其道而行：**β-VAE** 把 KL 項乘上 $\beta > 1$，以犧牲重建為代價，迫使模型學出更分解、更「解纏（disentangled）」的潛在碼。要精確理解這件事：當 $\beta \neq 1$，目標函數就不再是 ELBO，它的值也不再是 $\log p_\theta(x)$ 的下界；Kingma 與 Welling 的教學綜述 [9] 討論了這點以及許多其他變體。最後，單層、因子化高斯的 VAE 容易產生模糊影像，部分原因是高斯解碼器會對不確定性取平均。**階層式 VAE** 疊了好幾層潛在變數；NVAE [10] 顯示，經過精心設計、搭配穩定化正則化的深層階層，可以在高達 256×256 像素的自然影像上產生清晰樣本。

GAN：與對手一起學習

白話版。兩個網路在玩遊戲。**生成器（generator）**把隨機雜訊變成假的例子；**判別器（discriminator）**看一個例子並猜它是真是假。生成器的勝利條件是騙過判別器，這逼得它的輸出越來越像真實資料。

極小極大遊戲

令 G 把雜訊 $z \sim p(z)$ 映到樣本 $G(z)$，其分布記為 p_g。令 $D(x) \in (0,1)$ 是判別器估計 x 為真的機率。Goodfellow 等人 [11] 的原始 GAN 目標是

$$\min_G \max_D \; V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p(z)}[\log(1 - D(G(z)))].$$

判別器最大化 V：這就是一般的二元交叉熵，真資料標 1、假資料標 0。生成器最小化 V：它希望 $D(G(z))$ 越大越好。

這個遊戲在最小化什麼

固定生成器，把 V 寫成對 x 的積分：$V = \int [p_{\text{data}}(x)\log D(x) + p_g(x)\log(1-D(x))]dx$。對每個 x 分別來看，當 $a, b > 0$，函數 $a\log D + b\log(1-D)$ 在 $D = a/(a+b)$ 有最大值（令導數 $a/D - b/(1-D)$ 為零即可）。所以**最佳判別器**是

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}.$$

它就是「若兩者事前機率相同，一個點來自資料而非生成器」的機率。把 D^* 代回並加減 $\log 4$，得到

$$V(D^*, G) = -\log 4 + 2\,\mathrm{JSD}(p_{\text{data}} \,\|\, p_g),$$

其中 $\mathrm{JSD}(p\|q) = \tfrac{1}{2}D_{\mathrm{KL}}(p\|m) + \tfrac{1}{2}D_{\mathrm{KL}}(q\|m)$、$m = \tfrac{1}{2}(p+q)$，稱為 **Jensen–Shannon 散度**。因此在完美判別器之下，生成器是在最小化資料與模型間的一個對稱散度，全域最佳解是 $p_g = p_{\text{data}}$，此時處處 $D^* = 1/2$ [11]。注意整個過程從沒計算過概似：判別器扮演一個學出來、會自我調整的「這看起來像資料嗎？」檢定。

非飽和損失與實作方式

訓練初期生成器很差，$D(G(z)) \approx 0$，$\log(1 - D(G(z)))$ 幾乎是平的，生成器的梯度消失。實務上生成器改為最小化 $-\mathbb{E}\log D(G(z))$，也就是同一篇論文 [11] 提出的**非飽和（non-saturating）**損失：正好在生成器落後時給出強梯度。它的固定點相同，但動態不同。

兩者交替更新。更新 D 時要把生成樣本 **detach**，讓梯度不會流到 G；更新 G 時讓梯度穿過 D 流進 G，但只執行 G 的最佳化器。用 logits 搭配 `binary_cross_entropy_with_logits`，在數值上比先套 sigmoid 再取 log 更安全。

程式：在八個高斯上訓練 GAN

測試模態涵蓋的經典場景，是二維平面上排成一圈的八個緊密高斯群。它小到能在 CPU 上一分鐘內訓練完，而且你可以**親眼看到**每個模態有沒有被涵蓋。

```
import math
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
angles = torch.arange(8) * (2 * math.pi / 8)
centers = 2.0 * torch.stack([angles.cos(), angles.sin()], dim=1)

def sample_data(n):
    idx = torch.randint(8, (n,))
    return centers[idx] + 0.05 * torch.randn(n, 2)

def mlp(i, o):
    return nn.Sequential(nn.Linear(i, 128), nn.ReLU(),
                         nn.Linear(128, 128), nn.ReLU(), nn.Linear(128, o))

G, D = mlp(8, 2), mlp(2, 1)
opt_g = torch.optim.Adam(G.parameters(), lr=1e-3, betas=(.5, .999))
opt_d = torch.optim.Adam(D.parameters(), lr=1e-3, betas=(.5, .999))
for step in range(3000):
    real = sample_data(256)
    fake = G(torch.randn(256, 8))
    # Discriminator: logits, real -> 1, fake -> 0; detach so G is untouched.
    d_loss = (F.binary_cross_entropy_with_logits(D(real), torch.ones(256, 1))
              + F.binary_cross_entropy_with_logits(D(fake.detach()),
                                                   torch.zeros(256, 1)))
    opt_d.zero_grad(); d_loss.backward(); opt_d.step()
    # Generator: non-saturating loss -log D(G(z)).
    g_loss = F.binary_cross_entropy_with_logits(D(fake), torch.ones(256, 1))
    opt_g.zero_grad(); g_loss.backward(); opt_g.step()

with torch.no_grad():
    x = G(torch.randn(5000, 8))
    dist = torch.cdist(x, centers)
    nearest = dist.min(1)
    good = nearest.values < 0.15          # within 3 std of some mode
    counts = torch.bincount(nearest.indices[good], minlength=8)
print("fraction near a mode:", round(good.float().mean().item(), 3))
print("modes with >= 1% of samples:", int((counts >= 50).sum()))
print("samples per mode:", counts.tolist())
```

輸出為：

```
fraction near a mode: 0.896
```

```
modes with >= 1% of samples: 7
samples per mode: [590, 14, 710, 276, 651, 731, 863, 646]
```

約 90% 的樣本落在某個模態上，但 45° 方向的模態在 5,000 個樣本中只分到 14 個，另一個模態也明顯偏少。這就是當場抓到的**部分模式崩塌**。圖 4 呈現生成器的演變：早期是一團沒有結構的點雲，接著是一個涵蓋所有模態、但也涵蓋模態之間空隙的圓環，最後是更銳利的群聚，卻幾乎放棄了一個模態。精確數字會隨亂數種子與套件版本改變，但這個定性現象很典型。

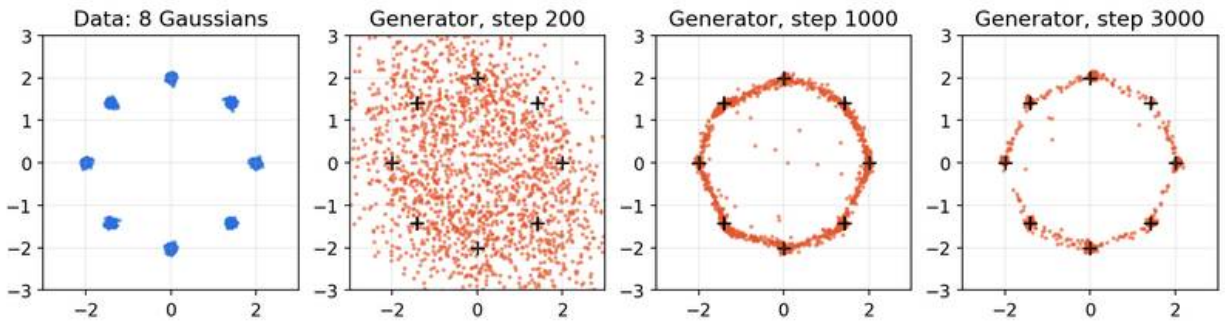


圖 *4* — 小型 *GAN* 學習八個高斯組成的圓環（十字標記真實模態）。到第 *3000* 步時樣本很清晰，但有一個模態幾乎被丟掉：清晰度與涵蓋度是兩回事。

模式崩塌及其成因

當生成器產生的輸出種類太少，就發生了**模式崩塌**。生成器得到的獎勵來自「目前的判別器接受的樣本」，而不是多樣性。如果它找到一塊判別器喜歡的區域，就可以把樣本都堆在那裡；判別器接著學會拒絕那塊區域，生成器再跳到另一塊，雙方可能互相追逐而不收斂。與最大概似不同，生成器的逐樣本損失中，沒有任何一項會直接懲罰**漏掉**某個模態。Salimans 等人 [12] 提出了特徵匹配（feature matching）與小批次判別（minibatch discrimination）等早期對策，讓判別器看整批樣本的統計量，於是多樣性不足就能被察覺。

所以一組清晰、逼真的影像，仍可能只涵蓋資料分布的一小部分。品質與涵蓋度一定要分開量測——我們會在評估一節回到這點。

Wasserstein GAN

當 p_{data} 與 p_g 分布在又薄、又互不重疊的集合上（影像在像素空間中正是如此），JSD 會卡在最大值 $\log 2$，無法提供「兩者**相距多遠**」的有用梯度。Arjovsky 等人 [13] 改用 **Wasserstein-1（推土機）距離**：把一個分布的機率質量搬到另一個分布的最小成本。根據 Kantorovich–Rubinstein 定理，它的對偶形式是

$$W(p_{\text{data}}, p_g) = \sup_{\|f\|_L \le 1} \mathbb{E}_{x\sim p_{\text{data}}}[f(x)] - \mathbb{E}_{x\sim p_g}[f(x)],$$

上確界取遍所有 1-Lipschitz 函數 f（輸入從 x' 變到 x 時，輸出變化不超過 $\|x - x'\|$）。網路 f 這時改稱為**評論器（critic）**：它輸出無上下界的分數，而不是機率。即使兩個支撐集不重疊，生成器

樣本越靠近資料，Wasserstein 距離也會平滑地縮小。

難處在於如何落實 Lipschitz 限制。原始 WGAN 把評論器的每個權重裁剪到一個小區間，作法粗糙，可能讓評論器過於簡單。**WGAN-GP** [14] 改為加上梯度懲罰

$$\lambda\, \mathbb{E}_{\hat{x}}[(\|\nabla_{\hat{x}} f(\hat{x})\|_2 - 1)^2],$$

其中 $\hat{x}$ 是在真實樣本與生成樣本配對之間的直線上均勻抽取，λ 是權重（論文使用 10）。另外兩個廣泛使用的穩定技巧直接作用在判別器上：**頻譜正規化（spectral normalization）** [15] 把每個權重矩陣除以它最大的奇異值，逐層限制 Lipschitz 常數；Mescheder 等人 [16] 分析的 **R1 懲罰**則在真實資料上懲罰 $\frac{\gamma}{2}\|\nabla_x D(x)\|^2$，他們證明這能帶來局部收斂，而未正則化的 GAN 訓練可能會震盪。

架構，以及 GAN 現在用在哪裡

StyleGAN [17] 說明架構與損失同樣重要：它把雜訊映到一個中介潛在空間，再把這個「風格」注入每一個解析度，得到可控制、高品質的人臉合成。GAN 只需一次生成器前向傳遞就能取樣，推論很快。對抗損失也常作為其他系統的**元件**出現——例如在影像 tokenizer 中用來重建清晰細節。但這不代表每個使用這種 tokenizer 的模型都是 GAN 生成器；分布可能完全由別的東西建模。

正規化流：可逆轉換

白話版。從簡單的雜訊（例如標準高斯）開始，把它推過一連串可逆轉換。因為每一步都能還原，任何影像都可以倒推回「會產生它的那個雜訊」——同時精確記錄每一步把空間拉伸或擠壓了多少，於是得到精確的概似。

變數變換

令 $z \sim p_0(z)$ 是簡單的基底分布，$x = f_\theta(z)$ 是同維度、可逆且可微的映射。則

$$\log p_\theta(x) = \log p_0(f_\theta^{-1}(x)) - \log \left| \det \frac{\partial f_\theta}{\partial z} \right|_{z = f_\theta^{-1}(x)}.$$

$\partial f_\theta / \partial z$ 是映射的 Jacobian 矩陣，它的行列式衡量映射把 z 附近的小體積放大了多少。機率質量守恆，所以若一塊區域被拉伸成兩倍體積，它的密度就必須減半。在一維中，$x = 2z$ 讓每個區間加倍，對數密度加上 $-\log 2$。組合映射 $f = f_K \circ \cdots \circ f_1$ 只要把各步的對數行列式加起來。

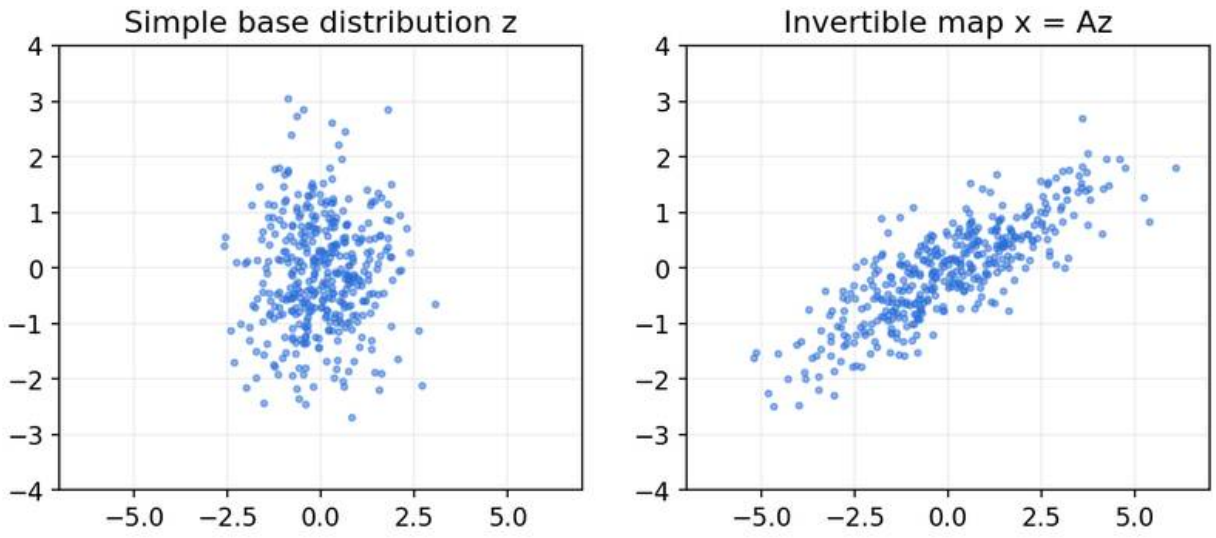


圖 5 — 變數變換保持總機率不變，同時改變密度。這個仿射映射的行列式可以解析計算。

耦合層

公式是精確的，但一般 $D \times D$ 行列式的成本是 $O(D^3)$——對影像來說毫無希望。流的層因此被設計成 Jacobian 為三**角矩陣**，行列式就只是對角線的乘積。RealNVP [18] 的仿射耦合層把 x 分成兩半 (x_1, x_2)，計算

$$y_1 = x_1, \qquad y_2 = x_2 \odot \exp\big(s(x_1)\big) + t(x_1),$$

其中 s 與 t 是任意神經網路。Jacobian 是區塊三角矩陣：第一塊是單位矩陣，第二塊是 $\operatorname{diag}(\exp s(x_1))$，所以 $\log|\det J| = \sum_j s_j(x_1)$。反函數一樣便宜：$x_2 = (y_2 - t(y_1)) \odot \exp(-s(y_1))$。關鍵在於 s 與 t 永遠不需要求逆，所以可以任意複雜。交替切分方式堆疊多層，就能讓每個座標都被轉換。Glow [19] 則在耦合層之間加入學習得到的可逆 1×1 卷積，作為廣義的通道排列。

程式：驗證耦合層的對數行列式

下面的程式建立一個仿射耦合層，並把它便宜算出的對數行列式，與 autograd 算出的完整 Jacobian 做比對。

```
import torch
from torch import nn

torch.manual_seed(0)

class AffineCoupling(nn.Module):
    """RealNVP-style layer: keep x1, scale-and-shift x2 using x1."""
    def __init__(self, dim, hidden=32):
        super().__init__()
```

```
        self.d = dim // 2
        self.net = nn.Sequential(nn.Linear(self.d, hidden), nn.Tanh(),
                                 nn.Linear(hidden, 2 * (dim - self.d)))

    def forward(self, x):
        x1, x2 = x[..., :self.d], x[..., self.d:]
        s, t = self.net(x1).chunk(2, dim=-1)
        s = torch.tanh(s)                          # keep scales moderate
        y2 = x2 * s.exp() + t
        return torch.cat([x1, y2], -1), s.sum(-1)   # output, log|det J|

    def inverse(self, y):
        y1, y2 = y[..., :self.d], y[..., self.d:]
        s, t = self.net(y1).chunk(2, dim=-1)
        s = torch.tanh(s)
        return torch.cat([y1, (y2 - t) * (-s).exp()], -1)

layer = AffineCoupling(4)
x = torch.randn(4)
y, logdet = layer(x)
J = torch.autograd.functional.jacobian(lambda v: layer(v)[0], x)
print("log|det J| from layer:   ", round(logdet.item(), 5))
print("log|det J| from autograd:", round(torch.linalg.slogdet(J)[1].item(), 5))
print("upper-right block is zero:", bool(torch.allclose(J[:2, 2:], torch.zeros(2, 2))))
print("inverse error:", (layer.inverse(y) - x).abs().max().item() < 1e-6)

# Log density of y under a standard-normal base after one layer:
base = torch.distributions.Normal(0., 1.)
z = layer.inverse(y)
log_px = base.log_prob(z).sum() - layer(z)[1]
print("log p(y):", round(log_px.item(), 4))
```

兩個對數行列式都印出 0.14867，Jacobian 右上區塊恰好為零（三角結構），反函數也還原了輸入。最後一行就是變數變換公式的實際運作。訓練一個流模型，就是用一疊這樣的層，在資料上最大化這個 `log p`。

優點、限制與連續流

流模型提供精確概似、精確反演（適合影像編輯，也能把資料映到性質良好的潛在空間），以及一次前向傳遞的取樣。代價在於架構：每一層都必須可逆且行列式便宜，而且潛在空間必須與資料**同維度**，這會把容量浪費在表示所有像素方向上的雜訊。Rezende 與 Mohamed [20] 最初提出流，是為了讓 VAE 的近似後驗更有彈性，這說明同一個工具既能用於推論，也能用於生成。

連續正規化流把離散的層疊換成常微分方程 $\mathrm{d}x/\mathrm{d}t = v_\theta(x, t)$。Neural ODE [21] 指出，這時對數密度是透過 v_θ 的**散度**（Jacobian 的跡）而非行列式來演化，於是解除了架構限制，但計算概似需要解一次 ODE。ML 07 介紹的**流匹配（flow matching）** [22]，則用簡單的迴歸而非最大概似來訓練這種向量場。所以「流」這個詞指的是相關的轉換，但訓練目標不一定是概似。

能量模型

白話版。與其描述怎麼產生影像，不如給每一張可能的影像打一個分數——「能量」——能量越低代表越「像真的」。能量低的地方，機率就高。

能量模型（energy-based model，EBM）定義

$$p_\theta(x) = \frac{\exp(-E_\theta(x))}{Z_\theta}, \qquad Z_\theta = \int \exp(-E_\theta(x))\, dx,$$

其中 E_θ 是任何輸出純量的網路，Z_θ 是讓密度積分為一的**配分函數（partition function）**。EBM 極有彈性——任何架構都行——但在高維中 Z_θ 無法計算，所以概似與精確樣本都無法直接取得。Goodfellow、Bengio 與 Courville [23] 涵蓋了經典工具（配分函數與近似推論等章節），Song 與 Kingma [24] 則整理了現代的訓練途徑：搭配 MCMC 樣本（例如 Langevin 動力學）的最大概似；**分數匹配（score matching）**，它擬合 $\nabla_x \log p_\theta(x) = -\nabla_x E_\theta(x)$，因而完全不需要 Z_θ；以及雜訊對比類方法。分數匹配正是通往擴散模型的橋樑。

自迴歸模型（簡介）

白話版。一次產生影像的一小塊，每一塊都以目前已產生的全部內容為條件，就像一個字一個字地寫句子。

對 x 的 D 個元素採用任何順序，機率的連鎖律都是精確的：

$$p_\theta(x) = \prod_{i=1}^{D} p_\theta(x_i \mid x_1, \ldots, x_{i-1}).$$

每個因子都是一般的分類或迴歸問題，所以訓練就是簡單、穩定、具精確概似的最大概似。PixelRNN 與 PixelCNN [25] 就是這樣對原始像素建模，每個子像素用一個 256 類的 softmax。缺點是取樣必須循序進行：D 個元素要 D 次網路計算。現代系統改為預測壓縮後的 token、一組組的 token 或整個尺度。ML 08 會詳細介紹這個家族。

擴散模型（簡介）

白話版。教一個網路從帶雜訊的影像中去掉一點點雜訊。然後從純雜訊出發，反覆套用這個去雜訊器，慢慢雕出一張影像。

擴散模型定義一個固定的前向過程，逐步對資料加入高斯雜訊，直到只剩雜訊；再學習一個逐步還原的反向過程。Ho 等人 [26] 證明，訓練（在權重之外）可以化約成預測被加入的雜訊——一個簡單的迴歸——而且這個目標是重新加權的變分下界，這也是圖 1 把擴散模型放在 VAE 旁邊的原因。與分數匹配的連結（見能量模型一節）提供了第二個等價觀點。Yang 等人的綜述 [27] 整理了眾多變體。擴散模型與流匹配如今主導高品質影像生成，是ML 07 的主題。

家族地圖

家族	基本訓練訊號	取樣機制	概似	主要限制
VAE	變分下界（ELBO）	取樣潛在變數再解碼（一次傳遞）	下界	下界取決於近似推論；樣本可能模糊
GAN	對抗判別	生成器前向傳遞（一次）	無	遊戲不穩定；模式崩塌
正規化流	變數變換概似	可逆轉換（一次）	精確	可逆性與同維度限制
自迴歸	條件概似	循序條件取樣	精確	生成時的相依性可能代價高
擴散	去雜訊／分數目標	反向雜訊過程（多步）	下界或基於 ODE	品質取決於模型與取樣器
流匹配	速度場迴歸	對學到的向量場積分	需額外計算	積分成本
能量模型	能量學習（MCMC、分數匹配）	常需迭代取樣	未正規化	正規化與混合可能困難

Bond-Taylor 等人 [3] 把這些取捨總結成一個很難同時滿足的三角：快速取樣、高樣本品質、良好的模態涵蓋。GAN 快又清晰，但可能漏掉模態；概似式模型涵蓋模態，但過去較慢或較模糊；擴散模型以多步取樣為代價，達到高品質與高涵蓋。這張表是教學用的整理，不是互斥的分類。實際模型常結合多個家族：VAE 壓縮影像、Transformer 建模它的潛在分布，再用擴散目標訓練那個 Transformer。

評估生成模型

白話版。生成器可以在不同面向上「好」：給真實資料高機率、產生個別看來逼真的樣本、涵蓋資料中的所有多樣性，或遵循它的條件。沒有任何單一數字能涵蓋全部，所以要用多個指標，並清楚每個指標漏掉了什麼。

Theis 等人 [5] 把這點講得很透徹：在高維中，平均對數概似、樣本品質，以及 Parzen 窗估計之類的舊代理指標，彼此可以**大致獨立**。模型可能概似很好但樣本很差，反之亦然。請依你實際的用途來評估。

概似

對概似式模型而言，保留資料上的 NLL（或每維位元數）有理論根據，在去量化與前處理一致時也可以相互比較。GAN 則無法計算它。而且高概似不代表能可靠地偵測分布外資料：Nalisnick 等人 [28] 發現，在 CIFAR-10 上訓練的流模型、VAE 與 PixelCNN，可能給不相關的 SVHN 門牌數字影像**更高**的概似，因為概似主要受平滑度這類低階統計量主導，而非語意內容。

Inception Score

Salimans 等人 [12] 提出 **Inception Score（IS）**。把每張生成影像 x 送進預訓練的 ImageNet 分類器（Inception-v3），得到標籤分布 $p(y \mid x)$，並令 $p(y) = \mathbb{E}_x[p(y \mid x)]$ 為對樣本的平均。則

$$\mathrm{IS} = \exp\Big(\mathbb{E}_{x \sim p_g} D_{\mathrm{KL}}(p(y \mid x) \,\|\, p(y))\Big).$$

當每張影像都被有信心地分類（$p(y \mid x)$ 尖銳）、整組樣本又涵蓋許多類別（$p(y)$ 分散）時，IS 就高。它從不看真實資料，所以無法判斷樣本是否像**你的**資料集，而且只對類似 ImageNet 的內容有意義。

Fréchet Inception Distance

Heusel 等人 [29] 提出的 **Fréchet Inception Distance（FID）**在特徵空間中比較真實與生成影像。對兩組影像擷取 Inception-v3 特徵，各自擬合一個高斯——平均值 μ_r, μ_g 與共變異數 Σ_r, Σ_g——再計算兩個高斯之間的 Fréchet 距離：

$$\mathrm{FID} = \|\mu_r - \mu_g\|_2^2 + \mathrm{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2}\right).$$

越低越好。FID 使用真實資料，對品質與多樣性都敏感，因此成為標準指標。但它不是逐張影像的真實度量：它取決於特徵擷取器、影像縮放方式，特別是**樣本數**（有限樣本下它有偏差，所以只能比較樣本數相同的 FID），而且它把品質與涵蓋度混成一個數字。

精確率與召回率

為了把兩者分開，Sajjadi 等人 [30] 定義了**分布的精確率與召回率**：精確率問的是生成樣本中有多少比例落在真實資料所在之處（品質），召回率問的是真實分布有多少比例被生成器涵蓋（多樣性）。Kynkäänniemi 等人 [31] 提出一個簡單又穩健的估計方法，以下就來實作。用一組球的聯集近似每組樣本的流形，每顆球的半徑等於該點到同組中第 k 個最近鄰的距離。於是

- **精確率** = 落在某個真實樣本的球內的生成樣本比例；
- **召回率** = 落在某個生成樣本的球內的真實樣本比例。

在影像上，這些點是深度特徵而非像素。在我們的二維圓環上，可以直接用座標：

```
import numpy as np

rng = np.random.default_rng(0)
angles = np.arange(8) * 2 * np.pi / 8
centers = 2 * np.stack([np.cos(angles), np.sin(angles)], 1)

def ring(n, modes):
    idx = rng.choice(modes, n)
    return centers[idx] + .05 * rng.standard_normal((n, 2))

def knn_radius(a, k):
    d = np.linalg.norm(a[:, None] - a[None], axis=-1)
    return np.sort(d, axis=1)[:, k]          # column 0 is the point itself

def coverage(query, ref, k=3):
    """Fraction of query points inside some reference point's k-NN ball."""
    r = knn_radius(ref, k)
    d = np.linalg.norm(query[:, None] - ref[None], axis=-1)
    return np.mean((d <= r[None]).any(axis=1))
```

```
real = ring(1000, range(8))
fakes = {
    "all 8 modes": ring(1000, range(8)),
    "only 2 modes": ring(1000, [0, 4]),
    "8 modes, blurry": ring(1000, range(8)) + .4 * rng.standard_normal((1000, 2)),
}
for name, fake in fakes.items():
    precision = coverage(fake, real)   # are fake samples on the real manifold?
    recall = coverage(real, fake)      # is the real manifold covered by fakes?
    print(f"{name:16s} precision={precision:.2f} recall={recall:.2f}")
```

```
all 8 modes      precision=0.97 recall=0.96
only 2 modes     precision=0.97 recall=0.26
8 modes, blurry  precision=0.07 recall=0.99
```

模式崩塌的生成器維持很高的精確率——它的每個樣本都很完美——但召回率掉到約 2/8。模糊的生成器恰好相反：它涵蓋每個模態，但大多數樣本偏離資料。像 FID 這樣的單一純量會把兩者都判為「比較差」，卻說不出**為什麼**。圖 6 呈現這三種情況。

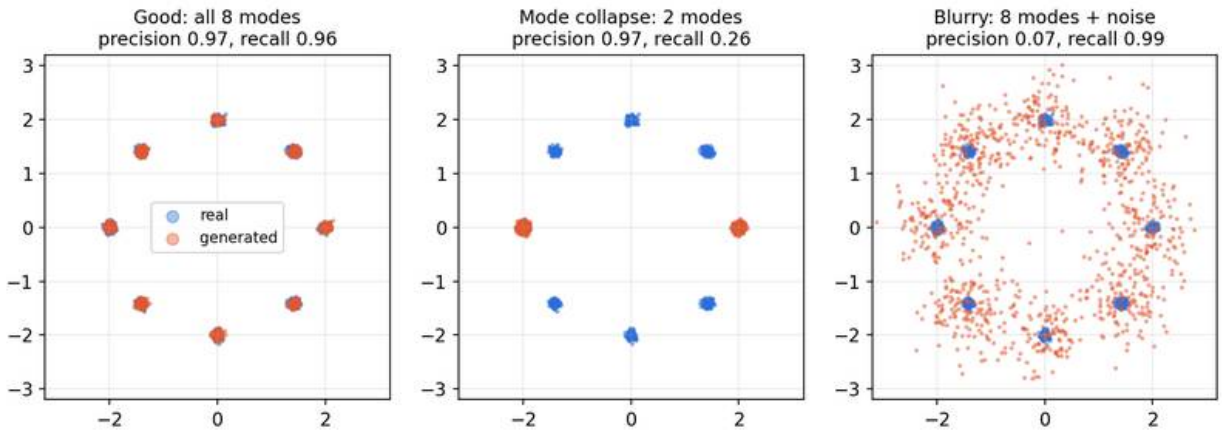


圖 *6* — 玩具圓環上的 k 近鄰精確率與召回率：模式崩塌降低召回率，模糊降低精確率。

現行指標的已知限制

評估本身就是活躍的研究領域。Stein 等人 [32] 進行了大規模人類實驗，發現沒有任何現有自動指標與人類的真實感判斷有強相關；特別是以 Inception 特徵計算的 FID，低估了擴散模型的感知真實度。他們建議 FID 與精確率／召回率之類的指標改用更豐富的自監督特徵空間（DINOv2 ViT-L/14）。Jayasumana 等人 [33] 主張 FID 的常態假設與 Inception 特徵不適合現代文字生成影像模型，並提出 **CMMD**：在 CLIP 嵌入上計算的最大平均差異（maximum mean discrepancy），不需要高斯假設。實務建議：回報多個互補指標並附上確切協定（特徵擷取器、樣本數、解析度、參考集）、展示未經挑選的樣本，而當主張涉及「看起來的品質」時，進行人類評估。

條件式任務還要加上**對齊度**的量測（影像是否符合文字或低解析度輸入？）。在影像復原中，一個漂亮的生成紋理可能沒有觀測資料支持；「看起來合理的樣本」與「量測到的證據」之間的區別，是DLCV 04 的核心議題。

現代觀點

值得一讀的綜述。四篇綜述從不同角度涵蓋本章內容。

- **Bond-Taylor 等人 [3]**（IEEE TPAMI 2022）是 VAE、GAN、正規化流、能量模型與自迴歸模型最好的單篇比較綜述。它依核心技術與已知病徵整理每個家族，並從訓練速度、取樣速度、樣本品質、多樣性與概似加以比較，結論是每個家族都在某處用弱點換來優點，而混合方法是自然的前進方向。
- **Papamakarios 等人 [4]**（JMLR 2021）是正規化流的統一論述：表達能力、耦合式／自迴歸式／殘差式／連續式流的設計空間，以及在密度估計、推論與模擬上的用途。
- **Kingma 與 Welling [9]**（Foundations and Trends in ML 2019）是 VAE 發明者撰寫的權威教學綜述：推導、重參數化估計量、更緊的下界、更有彈性的後驗（包括流）、階層式模型，以及加權 β 的變體。
- **Yang 等人 [27]**（ACM Computing Surveys 2023）綜述擴散模型——取樣加速、概似改進、結構化資料，以及與 VAE、GAN、流和 EBM 的關聯——是通往下一章的橋樑。

Bishop 與 Bishop [1]、Prince [2] 的教科書對所有家族都有一致的推導，Prince 的 notebook 更提供可執行的配套程式。

領域現況（2023–2026）。擴散與流匹配模型——通常在學到的潛在空間中運作——如今產出了大部分最先進的影像與影片生成結果。這讓**自編碼器**再次成為承重元件：VQ-VAE [34] 展示了如何學習離散潛在碼，讓另一個先驗模型去建模；潛在擴散 [35] 則證明，把生成模型放在壓縮後的自編碼器潛在空間中執行，能在保留細節的同時大幅降低成本。tokenizer 必須在壓縮率、重建品質，以及潛在空間是否容易建模之間取得平衡；請先評估它的重建，因為下游先驗無法還原編碼器已經丟掉的細節。

GAN 並沒有消失。GigaGAN [36] 把 GAN 擴展到文字生成影像，並凸顯 GAN 仍然最擅長的事：快到足以互動使用的單次傳遞取樣、高解析度上取樣，以及平滑、可編輯的潛在空間。Huang 等人 [37] 重新檢視訓練目標本身，推導出具局部收斂保證的正則化相對式（relativistic）GAN 損失，並顯示一個不依賴傳統技巧大全的極簡現代基準（R3GAN）在標準基準上能超越 StyleGAN2。對抗損失也繼續存在於其他系統中，例如用來讓 tokenizer 的重建更銳利。

在評估方面，2023–2024 年對 FID 的批評 [32]、[33] 推動領域轉向更豐富的特徵擷取器、分開的精確率／召回率式指標與人類評估，但至今仍沒有公認的單一指標。

開放問題。（1）與人類一致、且能區分品質、多樣性與記憶的可靠評估。（2）偵測模型何時在複製訓練影像而非泛化。（3）真正能用於分布外偵測的校準概似 [28]。（4）速度、品質與涵蓋度的三角取捨：在不失去模態的前提下，以 GAN 的速度達到擴散模型的品質。（5）學出同時具備壓縮性、忠實性又容易建模的潛在空間。

重點整理

- 生成需要一個分布或一個明確定義的取樣程序。最大概似等於最小化 $D_{\mathrm{KL}}(p_{\text{data}} \| p_\theta)$，這會鼓勵涵蓋每一個模態。
- 概似式模型（自迴歸、流、VAE、EBM、擴散的下界）寫下或界定 $\log p(x)$；隱式模型（GAN）只提供取樣器。
- 一般自編碼器提供重建，卻沒有潛在碼的先驗，所以解碼隨機潛在碼不一定得到逼真的樣本。
- ELBO 等於 $\log p_\theta(x)$ 減去 $D_{\mathrm{KL}}(q_\phi(z \mid x) \| p_\theta(z \mid x))$；重參數化讓它能用反向傳播訓練。
- 在最佳判別器下，原始 GAN 最小化 $2\,\mathrm{JSD} - \log 4$；非飽和損失、Wasserstein 評論器、梯度懲罰、頻譜正規化與 R1 讓這個遊戲可以訓練，但模式崩塌仍必須另外量測。
- 流模型透過變數變換公式計算精確概似，耦合層的三角 Jacobian 讓它變得便宜。
- 概似、IS、FID 與精確率／召回率量測的是不同的東西；請回報多個指標，並附上確切協定。

練習

1. **KL 健全性檢查**。把 $\mu = 0$、$\sigma^2 = 1$ 代入高斯 KL。接著計算一維中 $\mu = 1, \sigma^2 = 1$ 的 KL。

   提示

   $\mu = 0, \sigma^2 = 1$ 時每一項都是零，因為後驗與先驗是同一個分布。$\mu = 1, \sigma^2 = 1$ 時得到 $\frac{1}{2}(1+1-0-1) = 0.5$ nats：把平均值移動一個標準差，代價是半個 nat。

2. **損失縮放**。把 VAE 的重建項改成對像素取平均而不是加總。若 KL 保持不變，會發生什麼事？

   提示

   相對於 KL，重建項變小了 64 倍。這等同於對加總損失使用 $\beta = 64$ 的 β-VAE：它實際上改變了正則化強度（往往導向後驗崩塌），而不只是改變回報的單位。

3. **最佳判別器**。在 $p_{\text{data}}(x) = 0.3$、$p_g(x) = 0.1$ 的點 x 上，$D^*(x)$ 是多少？當 $p_g = p_{\text{data}}$ 時，D^* 處處是多少？此時遊戲的值是多少？

   提示

   $D^*(x) = 0.3/(0.3 + 0.1) = 0.75$。兩個分布相同時，處處 $D^* = 1/2$，$V = \log \frac{1}{2} + \log \frac{1}{2} = -\log 4$，與 $\mathrm{JSD} = 0$ 一致。

4. **仿射流**。對 $x = Az + b$、$z \sim \mathcal{N}(0, I)$，推導密度修正項，並說明為何奇異的 A 會使它失效。再用已知的高斯 $\mathcal{N}(b, AA^\top)$ 檢查你的答案。

   提示

   減去 $\log |\det A|$。因為 $\det(AA^\top)^{1/2} = |\det A|$，這與 $\mathcal{N}(b, AA^\top)$ 的正規化常數一致。奇異映射不可逆，會把體積壓扁，所以同維度的變數變換公式不適用。

5. **辨認崩塌**。分別為 VAE 後驗崩塌與 GAN 模式崩塌提出不同的檢查方法。接著把 GAN 程式改成 WGAN-GP（評論器不加 sigmoid、梯度懲罰 $\lambda = 10$、每次生成器更新前做多次評論器更新），並比較各模態的樣本數。

   提示

   VAE：檢查每個潛在維度的 KL，以及改變 z 時解碼輸出是否跟著變。GAN：統計每個模態的樣本數（如我們的程式）、計算召回率，並在固定條件下找重複的樣本。這是兩種不同的失敗。WGAN-GP：取樣 $\hat{x} = \alpha x_{\text{real}} + (1-\alpha)x_{\text{fake}}$、$\alpha \sim U(0,1)$，呼叫 `torch.autograd.grad` 並設 `create_graph=True`；若訓練震盪就降低學習率。下結論前請比較多個亂數種子。

6. **指標的盲點**。利用精確率／召回率程式，建構一組生成樣本，使它的精確率與召回率和好的生成器差不多，但一半的樣本只集中在四個模態。這告訴你這個指標有什麼限制？

   提示

   模態權重不均仍可能得到高召回率，因為召回率只問每個真實點附近是否有**某個**生成點，而不問模態的頻率是否相符。精確率／召回率量測的是支撐集的重疊，而不是密度；請搭配 FID 這類分布層級的指標或逐模態直方圖。

參考文獻

1. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. Book
2. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. Book and notebooks
3. S. Bond-Taylor, A. Leach, Y. Long and C. G. Willcocks, "Deep Generative Modelling: A Comparative Review of VAEs, GANs, Normalizing Flows, Energy-Based and Autoregressive Models," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022. Paper
4. G. Papamakarios et al., "Normalizing Flows for Probabilistic Modeling and Inference," *JMLR*, 2021. Review
5. L. Theis, A. van den Oord and M. Bethge, "A note on the evaluation of generative models," ICLR, 2016. Paper
6. D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," ICLR, 2014. Paper
7. S. R. Bowman et al., "Generating Sentences from a Continuous Space," CoNLL, 2016. Paper
8. I. Higgins et al., "β-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework," ICLR, 2017. Paper
9. D. P. Kingma and M. Welling, "An Introduction to Variational Autoencoders," *Foundations and Trends in Machine Learning*, 2019. Review
10. A. Vahdat and J. Kautz, "NVAE: A Deep Hierarchical Variational Autoencoder," NeurIPS, 2020. Paper
11. I. Goodfellow et al., "Generative Adversarial Nets," NeurIPS, 2014. Paper
12. T. Salimans et al., "Improved Techniques for Training GANs," NeurIPS, 2016. Paper
13. M. Arjovsky, S. Chintala and L. Bottou, "Wasserstein GAN," ICML, 2017. Paper
14. I. Gulrajani et al., "Improved Training of Wasserstein GANs," NeurIPS, 2017. Paper

15. T. Miyato et al., “Spectral Normalization for Generative Adversarial Networks,” ICLR, 2018. Paper
16. L. Mescheder, A. Geiger and S. Nowozin, “Which Training Methods for GANs do actually Converge?” ICML, 2018. Paper
17. T. Karras, S. Laine and T. Aila, “A Style-Based Generator Architecture for Generative Adversarial Networks,” CVPR, 2019. Paper
18. L. Dinh, J. Sohl-Dickstein and S. Bengio, “Density estimation using Real NVP,” ICLR, 2017. Paper
19. D. P. Kingma and P. Dhariwal, “Glow: Generative Flow with Invertible 1x1 Convolutions,” NeurIPS, 2018. Paper
20. D. J. Rezende and S. Mohamed, “Variational Inference with Normalizing Flows,” ICML, 2015. Paper
21. R. T. Q. Chen, Y. Rubanova, J. Bettencourt and D. Duvenaud, “Neural Ordinary Differential Equations,” NeurIPS, 2018. Paper
22. Y. Lipman et al., “Flow Matching for Generative Modeling,” ICLR, 2023. Paper
23. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 18–20. Book
24. Y. Song and D. P. Kingma, “How to Train Your Energy-Based Models,” arXiv, 2021. Paper
25. A. van den Oord, N. Kalchbrenner and K. Kavukcuoglu, “Pixel Recurrent Neural Networks,” ICML, 2016. Paper
26. J. Ho, A. Jain and P. Abbeel, “Denoising Diffusion Probabilistic Models,” NeurIPS, 2020. Paper
27. L. Yang et al., “Diffusion Models: A Comprehensive Survey of Methods and Applications,” *ACM Computing Surveys*, 2023. Survey
28. E. Nalisnick et al., “Do Deep Generative Models Know What They Don’t Know?” ICLR, 2019. Paper
29. M. Heusel et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” NeurIPS, 2017. Paper
30. M. S. M. Sajjadi, O. Bachem, M. Lucic, O. Bousquet and S. Gelly, “Assessing Generative Models via Precision and Recall,” NeurIPS, 2018. Paper
31. T. Kynkäänniemi, T. Karras, S. Laine, J. Lehtinen and T. Aila, “Improved Precision and Recall Metric for Assessing Generative Models,” NeurIPS, 2019. Paper
32. G. Stein et al., “Exposing flaws of generative model evaluation metrics and their unfair treatment of diffusion models,” NeurIPS, 2023. Paper
33. S. Jayasumana et al., “Rethinking FID: Towards a Better Evaluation Metric for Image Generation,” CVPR, 2024. Paper
34. A. van den Oord, O. Vinyals and K. Kavukcuoglu, “Neural Discrete Representation Learning,” NeurIPS, 2017. Paper
35. R. Rombach et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” CVPR, 2022. Paper
36. M. Kang et al., “Scaling up GANs for Text-to-Image Synthesis,” CVPR, 2023. Paper
37. Y. Huang, A. Gokaslan, V. Kuleshov and J. Tompkin, “The GAN is dead; long live the GAN! A Modern GAN Baseline,” NeurIPS, 2024. Paper

ML 07・擴散模型與流匹配

推導雜訊、乾淨影像與速度預測，以 JiT 和表徵自編碼器比較像素與潛在生成，並理解預訓練影像／影片的擴散與流生成流程。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-07-diffusion-flow-matching/

你將學到

- 一個「已知的加噪過程」如何把「生成一張影像」變成普通的監督式迴歸問題。
- DDPM 的簡化雜訊預測損失如何一步一步從變分下界推導出來。
- 雜訊、乾淨影像與速度預測如何在代數上換算，為什麼損失與有限容量網路不同，以及分數估計與 SDE 觀點如何連結它們。
- DDPM、DDIM 與快速 ODE 求解器如何取樣，以及為什麼步數是一個設計選擇。
- 分類器引導（classifier guidance）與無分類器引導（classifier-free guidance）如何操控取樣器，代價又是什麼。
- 像素與潛在擴散、DiT、JiT、表徵自編碼器、流匹配與少步生成如何連結，以及預訓練影像／影片流程如何評估。

先看全貌

擴散模型學的是「把受控的破壞復原」。訓練時，我們拿一個真實樣本，加上已知強度的 Gaussian 雜訊，然後請神經網路告訴我們加了什麼雜訊。因為雜訊是我們自己加的，正確答案永遠拿得到：不需要標籤，也不需要對手，就是迴歸而已。生成時，我們從純雜訊出發，反覆套用學到的「去掉一點雜訊」步驟，直到一個樣本浮現出來。

> **用最簡單的話說**
>
> 想像你慢慢用鉛筆在一張畫上亂塗。你把畫在各種塗鴉程度下都留一份：幾條線、很多線、多到原本的畫完全看不見。模型只練習一件小事：看一張被塗過的紙，猜出哪些線是後來加上去的。練習夠多之後，你給它一張「只有」塗鴉的紙。它擦掉一點、再看一次、再擦掉一點，一張全新的畫就慢慢出現了——而且這張畫從來不在那疊紙裡。

為什麼重要？從 2021 年起，擴散類模型在影像、影片與音訊合成上拿下了許多最先進的結果，同一套機制也用於影像修補、超解析度、影像編輯、3D 生成與科學資料。擴散模型也解決了ML 06中較早模型家族的兩個痛點：GAN 需要脆弱的雙人賽局，VAE 的樣本則容易模糊。擴散模型的訓練是穩定的迴歸，而因為生成被拆成許多小而容易的步驟，樣本也很清晰。

代價是取樣成本：經典取樣器每產生一張影像要呼叫網路數百次。2021 年以來的大量研究——DDIM、快速 ODE 求解器、蒸餾、一致性模型、流匹配、整流流——都是在維持品質的同時減少網路呼叫次數。

本章依照幾篇主要綜述的架構安排。Yang 等人 [1] 把領域分成高效取樣、改進概似（likelihood）與特殊結構資料三個方向；Croitoru 等人 [27] 依公式化方式（DDPM、雜訊條件分數網路、SDE）分類；Lipman 等人的流匹配指南 [2] 與 Lai 等人的專書 [29] 則把擴散與流視為同一族「學習出來的傳輸」。Prince 的教科書 [14] 是推導時很好的入門參考。全文**每一節只使用一種明確的時間方向慣例**，換慣例時會特別說明——混用慣例是這個領域最常見的正負號錯誤來源。

前向擴散有封閉解

白話版。每一步保留影像的大部分，再加上一點獨立的 Gaussian 雜訊。經過足夠多步之後，影像什麼都不剩——只剩分布已知的雜訊。

這個想法可追溯到 Sohl-Dickstein 等人 [15]：用緩慢的前向擴散破壞結構，再學習把它反轉；DDPM [3] 讓它變得實用。令 $x_0 \in \mathbb{R}^d$ 為一筆資料（把影像攤平成 d 個數字，大致縮放到 $[-1, 1]$）。選一組**雜訊排程（noise schedule）** $\beta_1, \dots, \beta_T$，其中 $0 < \beta_t < 1$，並記 $\alpha_t = 1 - \beta_t$。前向（加噪）過程是一條馬可夫鏈

$$q(x_t \mid x_{t-1}) = \mathcal{N}\big(\sqrt{\alpha_t}\, x_{t-1},\ \beta_t I\big), \qquad t = 1, \dots, T,$$

其中 $\mathcal{N}(\mu, \Sigma)$ 表示平均為 μ、共變異數為 Σ 的 Gaussian，I 是 $d \times d$ 單位矩陣。係數 $\sqrt{\alpha_t}$ 把訊號稍微縮小，讓總變異數維持有界而不會一路變大；這也是這一族被稱為**變異數保持（variance preserving, VP）**的原因。

推導一次到位的公式

把一步寫成 $x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1-\alpha_t}\,\epsilon_t$，其中 $\epsilon_t \sim \mathcal{N}(0, I)$。代入前一步，

$$\begin{aligned} x_t &= \sqrt{\alpha_t}\left(\sqrt{\alpha_{t-1}} x_{t-2} + \sqrt{1-\alpha_{t-1}}\,\epsilon_{t-1}\right) + \sqrt{1-\alpha_t}\,\epsilon_t \\ &= \sqrt{\alpha_t \alpha_{t-1}}\, x_{t-2} + \underbrace{\sqrt{\alpha_t(1-\alpha_{t-1})}\,\epsilon_{t-1} + \sqrt{1-\alpha_t}\,\epsilon_t}_{\text{獨立 Gaussian 之和}}. \end{aligned}$$

兩個雜訊項是獨立、平均為零的 Gaussian，所以相加後仍是 Gaussian，變異數為 $\alpha_t(1-\alpha_{t-1}) + (1-\alpha_t) = 1 - \alpha_t\alpha_{t-1}$。把同樣的論證一路推回 x_0，令 $\bar\alpha_t = \prod_{s=1}^{t} \alpha_s$，得到

$$q(x_t \mid x_0) = \mathcal{N}\big(\sqrt{\bar\alpha_t}\, x_0,\ (1-\bar\alpha_t) I\big) \quad \Longleftrightarrow \quad x_t = \sqrt{\bar\alpha_t}\, x_0 + \sqrt{1-\bar\alpha_t}\,\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

這裡 $\bar\alpha_t$ 是到第 t 步仍保留的訊號變異數比例，ϵ 是一個標準 Gaussian 向量。這條公式讓訓練很便宜：我們可以直接跳到任何雜訊程度，不必模擬前面的每一步。

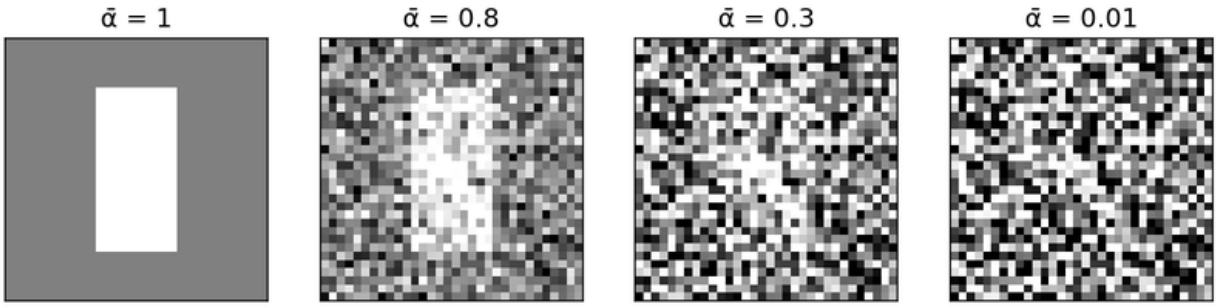


圖 *1* — 封閉形式前向過程在幾個訊號程度下的樣子。這裡共用同一組雜訊以便看出效果；實際訓練時每筆資料、每一步都抽新的雜訊。

雜訊排程與訊雜比

第 t 步的**訊雜比（signal-to-noise ratio, SNR）**為

$$\mathrm{SNR}(t) = \frac{\bar{\alpha}_t}{1-\bar{\alpha}_t},$$

即訊號變異數與雜訊變異數之比（假設資料變異數為 1）。它從 $t=1$ 時非常大，單調下降到 $t=T$ 時接近零。排程有兩個要求：$\bar{\alpha}_T$ 必須接近 0，讓 x_T 實際上就是標準 Gaussian（取樣器從 $\mathcal{N}(0, I)$ 出發）；中間各步則應把訓練資源花在網路真的有東西可學的雜訊程度上。

DDPM [3] 使用 $T = 1000$ 與**線性排程**，β_t 從 10^{-4} 線性增加到 0.02。Nichol 與 Dhariwal [19] 觀察到這樣會太快摧毀資訊：鏈的相當一大段裡，影像幾乎已經是純雜訊。他們的**餘弦排程**定義 $\bar{\alpha}_t = f(t)/f(0)$，其中 $f(t) = \cos^2\!\left(\frac{t/T + s}{1+s} \cdot \frac{\pi}{2}\right)$，小偏移量 $s = 0.008$，並把每個 β_t 截在 0.999。圖 2 比較兩者。Karras 等人 [20] 後來主張：排程、網路輸入輸出的縮放，以及訓練時雜訊程度的分布，是應該分開調整的獨立設計選擇。

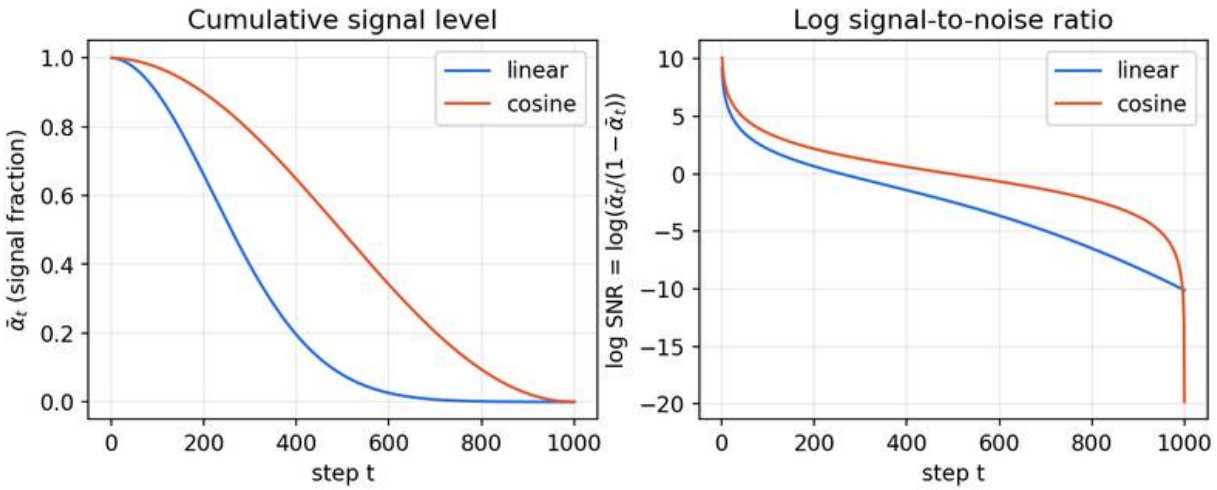


圖 *2* — *$T = 1000$* 時的線性排程與餘弦排程。餘弦排程讓訊號保留得更久，對數訊雜比在各步之間分布得更平均。

反向過程與變分下界

白話版。要生成，就需要反向的一步：「給一個有雜訊的樣本，稍微乾淨一點的樣本長什麼樣？」對真實資料我們無法直接算出來，但如果同時知道乾淨樣本，就算得出來——而這正好給出一個簡單的訓練損失。

模型與下界

生成模型是一條反向馬可夫鏈，從 $p(x_T) = \mathcal{N}(0, I)$ 出發，使用學習出來的 Gaussian 轉移

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(\mu_\theta(x_t, t),\, \sigma_t^2 I\big),$$

其中 μ_θ 是參數為 θ 的神經網路，σ_t^2 是我們稍後要選的變異數。這是一個以 $x_{1:T}$ 為潛在變數的模型，就像一個編碼器 q 固定而非學習的深層 VAE（見ML 06）。同樣的 Jensen 不等式論證給出證據下界（ELBO）。依照 [3]、[15]，負的下界可以整理成一串 KL 散度的和：

$$-\log p_\theta(x_0) \;\le\; \mathbb{E}_q\Big[\underbrace{D_{\mathrm{KL}}(q(x_T \mid x_0)\,\|\,p(x_T))}_{L_T} + \sum_{t=2}^{T} \underbrace{D_{\mathrm{KL}}(q(x_{t-1} \mid x_t, x_0)\,\|\,p_\theta(x_{t-1} \mid x_t))}_{L_{t-1}} \underbrace{-\log p_\theta(x_0 \mid x_1)}_{L_0}\Big].$$

D_{KL} 是 Kullback–Leibler 散度。L_T 沒有可學參數（當 $\bar\alpha_T \approx 0$ 時很小）；L_0 是最後一步的解碼項；每個 L_{t-1} 則把模型的反向一步與**前向後驗分布** $q(x_{t-1} \mid x_t, x_0)$ 相比——也就是「如果知道乾淨樣本 x_0，反向一步會是什麼」的精確答案。

前向後驗是 Gaussian

由貝氏定理，$q(x_{t-1} \mid x_t, x_0) \propto q(x_t \mid x_{t-1})\, q(x_{t-1} \mid x_0)$。兩個因子對 x_{t-1} 都是 Gaussian，所以乘積也是 Gaussian。把 x_{t-1} 的二次項與一次項收集起來（配方），得到

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}\big(\tilde\mu_t(x_t, x_0),\, \tilde\beta_t I\big),$$

$$\tilde\mu_t = \frac{\sqrt{\bar\alpha_{t-1}}\,\beta_t}{1-\bar\alpha_t}\, x_0 + \frac{\sqrt{\alpha_t}\,(1-\bar\alpha_{t-1})}{1-\bar\alpha_t}\, x_t, \qquad \tilde\beta_t = \frac{1-\bar\alpha_{t-1}}{1-\bar\alpha_t}\,\beta_t.$$

後驗平均是乾淨樣本與目前雜訊樣本的加權平均；後驗變異數 $\tilde\beta_t$ 比 β_t 略小。你不必相信代數：下面的程式抽樣前向鏈，並以 x_t 對 x_{t-1} 做迴歸。

```
import numpy as np

rng = np.random.default_rng(0)
beta = np.linspace(1e-4, 0.02, 1000)
alpha = 1 - beta
abar = np.cumprod(alpha)
t, x0, n = 300, 1.3, 400_000           # check q(x_{t-1} | x_t, x_0) at step t

# Sample the forward chain: x_{t-1} from q(x_{t-1}|x_0), then one more step.
x_prev = np.sqrt(abar[t-1])*x0 + np.sqrt(1-abar[t-1])*rng.normal(size=n)
```

```
x_t = np.sqrt(alpha[t])*x_prev + np.sqrt(beta[t])*rng.normal(size=n)

# Empirical regression of x_{t-1} on x_t (the posterior mean is linear in x_t).
slope, intercept = np.polyfit(x_t, x_prev, 1)
resid_var = np.var(x_prev - (slope*x_t + intercept))

# Closed-form posterior from the derivation.
c_x0 = np.sqrt(abar[t-1])*beta[t] / (1-abar[t])
c_xt = np.sqrt(alpha[t])*(1-abar[t-1]) / (1-abar[t])
var = beta[t]*(1-abar[t-1]) / (1-abar[t])
print(f"slope     {slope:.4f}  vs  {c_xt:.4f}")
print(f"intercept {intercept:.4f}  vs  {c_x0*x0:.4f}")
print(f"variance  {resid_var:.6f}  vs  {var:.6f}")
```

輸出在小數點後三到四位吻合（斜率 0.9934 對 0.9930，變異數 0.006050 對 0.006052）；微小差距是蒙地卡羅誤差。

從下界到簡化損失

現在把前向公式反過來用：$x_0 = (x_t - \sqrt{1-\bar\alpha_t}\,\epsilon)/\sqrt{\bar\alpha_t}$。代入 $\tilde\mu_t$，再用 $\bar\alpha_t = \alpha_t\bar\alpha_{t-1}$ 化簡，

$$\tilde\mu_t = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar\alpha_t}}\,\epsilon\right).$$

取樣時網路看得到 x_t，卻看不到 ϵ。所以 DDPM 讓模型平均採用完全相同的形式，只是把未知的雜訊換成預測值 $\epsilon_\theta(x_t, t)$：

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar\alpha_t}}\,\epsilon_\theta(x_t, t)\right).$$

兩個共變異數同為固定 $\sigma_t^2 I$ 的 Gaussian，其 KL 散度就是兩個平均之間平方距離的縮放，因此

$$L_{t-1} = \frac{1}{2\sigma_t^2}\|\tilde\mu_t - \mu_\theta\|^2 + C = \frac{\beta_t^2}{2\sigma_t^2\,\alpha_t\,(1-\bar\alpha_t)}\,\|\epsilon - \epsilon_\theta(x_t, t)\|^2 + C,$$

其中 C 與 θ 無關。下界的每一項因此都是**加權的雜訊預測誤差**。Ho 等人 [3] 發現，拿掉權重並均勻抽樣 t，訓練效果反而更好：

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{x_0,\,t\sim\mathcal{U}\{1,\dots,T\},\,\epsilon\sim\mathcal{N}(0,I)}\left\|\epsilon - \epsilon_\theta\big(\sqrt{\bar\alpha_t}x_0 + \sqrt{1-\bar\alpha_t}\,\epsilon,\;t\big)\right\|^2.$$

把它當成演算法來讀：挑一筆訓練資料、挑一個隨機步數、挑隨機雜訊，用一行算出 x_t，然後迴歸這個雜訊。時間 t 會餵給網路（通常透過正弦嵌入），因為同樣的像素值在不同雜訊程度下代表不同意義。

有兩點容易弄錯。第一，$\mathcal{L}_{\text{simple}}$ 不是概似下界：拿掉權重相對降低了低雜訊步的比重，這有助於樣本品質，卻會讓報告的對數概似變差。Improved DDPM [19] 用混合損失學習變異數 σ_t^2（在 β_t 與 $\tilde\beta_t$ 之間內插），找回了好的概似。第二，同一個網路也給出**乾淨估計**

$$\hat{x}_0(x_t, t) = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}},$$

每個現代取樣器內部都會用到它。當 $\bar{\alpha}_t$ 非常小時，這個除法會放大預測誤差，所以高雜訊時的 $\hat{x}_0$ 是一個模糊的平均，而不是一個樣本。

雜訊、乾淨資料、速度與分數

白話版。預測雜訊、預測乾淨影像、預測「往哪個方向走會讓這個樣本更典型」，是表達同一份知識的三種方式。差別在縮放，而縮放會改變訓練損失對不同雜訊程度的加權。

三種參數化

因為 $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$ 是固定的線性關係，網路可以輸出下列任何一個：

- 雜訊 ϵ（DDPM [3]），
- 乾淨樣本 x_0，
- **速度** $v = \sqrt{\bar{\alpha}_t}\,\epsilon - \sqrt{1-\bar{\alpha}_t}\,x_0$，由 Salimans 與 Ho [22] 為了蒸餾而提出，在排程兩端都維持良好的尺度。

給定 x_t 與 t，三者可以彼此精確換算。但是對 ϵ、x_0 或 v 做不加權的 MSE，對各雜訊程度的加權並不相同，所以這個選擇會影響訓練。Karras 等人 [20] 用輸入輸出的「預條件（preconditioning）」係數把這件事講清楚：讓網路在每個雜訊程度下的輸入與目標都維持單位變異數。注意，這裡擴散模型的「速度」是訊號與雜訊的一種重新參數化；它與本章後面流匹配的傳輸速度相關，但不相同。

精確轉換不代表有限容量網路等價

對保留變異數的路徑，令 $a = \sqrt{\bar{\alpha}_t}$ 與 $b = \sqrt{1-\bar{\alpha}_t}$，因此 $a^2 + b^2 = 1$，而 $x_t = ax_0 + b\epsilon$。擴散速度 $v_{\mathrm{VP}} = a\epsilon - bx_0$ 與前向狀態共同構成一個旋轉：

$$x_0 = ax_t - bv_{\mathrm{VP}}, \qquad \epsilon = bx_t + av_{\mathrm{VP}},$$
$$\hat{x}_0^{(\epsilon)} = (x_t - b\hat{\epsilon})/a, \quad \hat{\epsilon}^{(x)} = (x_t - a\hat{x}_0)/b.$$

這裡 a, b 是已知的排程係數；v_{VP} 是 [22] 使用的目標；帽號表示估計值；上標指出網路直接輸出的量。最後兩種轉換的除數必須非零。即使內部時間的代數精確，端點附近仍有數值與統計上的不穩定問題。

雜訊誤差 δ_ϵ 會變成乾淨影像誤差 $-(b/a)\delta_\epsilon$。因此，固定雜訊程度下互相對應的預測滿足：

$$\|\hat{x}_0 - x_0\|^2 = \frac{b^2}{a^2}\|\hat{\epsilon} - \epsilon\|^2 = \frac{1}{\mathrm{SNR}(t)}\|\hat{\epsilon} - \epsilon\|^2, \qquad \|\hat{x}_0^{(v)} - x_0\|^2 = b^2\|\hat{v}_{\mathrm{VP}} - v_{\mathrm{VP}}\|^2.$$

$\mathrm{SNR}(t) = a^2/b^2$ 是訊雜比；第二個恆等式由前述旋轉得到。相同形式的無加權損失因此會強調不同時間。改變目標卻未記錄時間抽樣與損失權重，就混入了其他實驗變因 [20]、[22]。

另外還有容量問題。若網路直接輸出乾淨影像，最後的表徵可以捨棄干擾性的雜訊。直接輸出雜訊的網路可能必須保留輸入中更多座標，或透過跳接取得那些座標。事後轉換某個網路的輸出，不會讓它的架構等同於直接學習另一種目標的網路。對大像素區塊與高維潛在表徵，這個差異尤其值得檢查；後面的 JiT 討論提供近期證據 [42]。這不代表雜訊預測普遍失敗：前面的 DDPM 範例與密集影像去雜訊網路就是反例。

```
import numpy as np

rng = np.random.default_rng(7)
x0 = rng.normal(size=32)
eps = rng.normal(size=32)
a, b = np.sqrt(.2), np.sqrt(.8)
xt = a*x0 + b*eps
v_vp = a*eps - b*x0
assert np.allclose(a*xt - b*v_vp, x0)
assert np.allclose(b*xt + a*v_vp, eps)

# A deliberately imperfect noise estimate: compare equivalent losses.
eps_hat = eps + .03*rng.normal(size=32)
x_hat = (xt - b*eps_hat)/a
noise_mse = np.mean((eps_hat - eps)**2)
clean_mse = np.mean((x_hat - x0)**2)
assert np.allclose(clean_mse/noise_mse, b*b/(a*a))
print("Clean/noise MSE ratio:", round(clean_mse/noise_mse, 3))

# A straight flow uses a different path and a different velocity.
t = .35
zt = (1-t)*eps + t*x0
u = x0 - eps
x_from_flow = zt + (1-t)*u
u_from_clean = (x0 - zt)/(1-t)
assert np.allclose(x_from_flow, x0)
assert np.allclose(u_from_clean, u)
print("VP velocity equals straight-flow target:", np.allclose(v_vp, u))
```

範例印出損失比例 4.0，並以 `False` 指出兩種速度目標不同。文獻有時都用字母 v；檢查點的排程器必須指出真正的路徑、正規化與時間方向。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

什麼是分數？為什麼去雜訊能估計它？

密度 p 的**分數（score）**是 $\nabla_x \log p(x)$：一個指向更高機率方向的向量場。如果知道分數，就能用 **Langevin 動力學**取樣：沿著分數走一小步、加一點雜訊，反覆進行。Song 與 Ermon [17] 在多個雜訊程度上訓練「雜訊條件分數網路（NCSN）」，並以退火 Langevin 動力學取樣——這是一條與擴散平行的路線，後來證明是同一個想法。

連接兩者的橋梁是**去雜訊分數匹配（denoising score matching）**（Vincent [16]）。對 Gaussian 破壞而言，（在均方誤差意義下）最好的雜訊預測器是一個條件期望值 $\epsilon^*(x_t, t) = \mathbb{E}[\epsilon \mid x_t]$，而 **Tweedie 公式** [40] 把它連到雜訊邊際分布 p_t 的分數：

$$\nabla_{x_t} \log p_t(x_t) = -\frac{\mathbb{E}[\epsilon \mid x_t]}{\sqrt{1-\bar{\alpha}_t}} \quad \Longrightarrow \quad s_\theta(x_t, t) \approx -\frac{\epsilon_\theta(x_t, t)}{\sqrt{1-\bar{\alpha}_t}},$$

其中 s_θ 是學到的分數。同一個公式用「乾淨」單位寫成 $\mathbb{E}[x_0 \mid x_t] = (x_t + (1-\bar{\alpha}_t)\nabla \log p_t(x_t))/\sqrt{\bar{\alpha}_t}$。兩個提醒：這是第 t 層加噪後的邊際分布的分數，不是任何單張訓練影像的梯度；而且它對最佳預測器成立——訓練出來的網路只是近似。

你可以在一切都已知的情況下驗證 Tweedie 公式：一維 Gaussian 資料，此時最佳雜訊預測器對 x_t 是線性的。

```
import numpy as np

rng = np.random.default_rng(0)
s, abar, n = 0.7, 0.4, 500_000     # data x0 ~ N(0, s^2); one noise level
x0 = s*rng.normal(size=n)
eps = rng.normal(size=n)
xt = np.sqrt(abar)*x0 + np.sqrt(1-abar)*eps

# "Train" the best linear noise predictor eps_hat = w * x_t by least squares.
w = (xt @ eps) / (xt @ xt)
score_from_eps = -w / np.sqrt(1-abar)       # s(x_t) = -eps_hat / sqrt(1-abar)

# Exact: x_t ~ N(0, abar s^2 + 1 - abar), so grad log p_t(x) = -x / var.
exact = -1 / (abar*s**2 + 1 - abar)
print(f"score slope from noise predictor: {score_from_eps:.4f}")
print(f"exact score slope:                {exact:.4f}")
```

它印出 -1.2560 對 -1.2563：對雜訊做迴歸就能找回分數。

SDE 觀點

Song 等人 [4] 讓步數趨於無窮。用連續時間 $t \in [0, 1]$ 與速率 $\beta(t)$，VP 前向過程變成**隨機微分方程（SDE）**

$$dx = -\tfrac{1}{2}\beta(t)\, x\, dt + \sqrt{\beta(t)}\, dw,$$

其中 w 是標準 Wiener 過程（布朗運動）。一個經典結果指出：這個過程倒著時間跑，仍然是一條 SDE，而其中唯一未知的量就是分數：

$$dx = \big[-\tfrac{1}{2}\beta(t)\, x - \beta(t)\, \nabla_x \log p_t(x) \big] dt + \sqrt{\beta(t)}\, d\bar{w},$$

此處 dt 為負，$\bar{w}$ 是反向時間的 Wiener 過程。此外，還存在一條確定性的**機率流 ODE（probability-flow ODE）**，其解在每個時間點的邊際分布 p_t 都相同：

$$\frac{dx}{dt} = -\tfrac{1}{2}\beta(t)\Big[x + \nabla_x \log p_t(x)\Big].$$

這把整個領域統一起來。DDPM 的祖先取樣（ancestral sampling）是反向 SDE 的離散化；NCSN [17] 對應到「變異數爆炸（variance exploding）」SDE；而這條 ODE 把擴散模型變成連續正規化流，因此可以用瞬時變數變換公式算出精確的對數概似 [4]。這條 ODE 也解釋了 DDIM 這類確定性取樣器為什麼有效，以及為什麼可以套用一般的 ODE 求解器。

取樣是一個反向過程

白話版。在單一隨機雜訊程度上算出的損失並不是生成器。要生成，必須從純雜訊出發，以一致的排程串起許多反向步驟——而你可以選擇要走幾步、每步注入多少新的隨機性。

DDPM 祖先取樣

DDPM 沿著學到的反向鏈取樣：

$$x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z, \qquad z \sim \mathcal{N}(0, I), \qquad \sigma_t^2 \in \{\beta_t,\ \tilde{\beta}_t\},$$

最後一步不再加入雜訊。Ho 等人 [3] 報告 $\sigma_t^2 = \beta_t$ 與 $\sigma_t^2 = \tilde{\beta}_t$ 的結果相近；學習出的變異數 [19] 在步數較少時幫助最大。

程式：二維分布上的 DDPM

這個完整的 CPU 範例（約 10 秒）使用兩個中心在 $x = \pm 1.5$、標準差 0.25 的 Gaussian 群集。它包含影像模型的每個運算——排程、隨機時間訓練、雜訊預測、反向平均——卻不需要影像等級的計算量。在 100 步與這個排程下，到 $t = T$ 時幾乎沒有訊號殘留。

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
T = 100
beta = torch.linspace(.001, .15, T)
alpha = 1-beta
abar = alpha.cumprod(0)
previous = torch.cat([torch.ones(1), abar[:-1]])
post_var = beta * (1-previous) / (1-abar)
net = nn.Sequential(nn.Linear(3, 64), nn.SiLU(),
                    nn.Linear(64, 64), nn.SiLU(), nn.Linear(64, 2))
opt = torch.optim.Adam(net.parameters(), lr=.002)
def data(n):
    side = torch.randint(2, (n, 1))*2-1
    return torch.cat([side*1.5, torch.zeros(n, 1)], 1) + .25*torch.randn(n, 2)
for step in range(3000):
```

```
    x0 = data(128)
    t = torch.randint(T, (len(x0),))
    eps = torch.randn_like(x0)
    a = abar[t, None]
    xt = a.sqrt()*x0 + (1-a).sqrt()*eps
    predicted = net(torch.cat([xt, t[:, None].float()/(T-1)], 1))
    loss = F.mse_loss(predicted, eps)
    opt.zero_grad(); loss.backward(); opt.step()
net.eval()
with torch.no_grad():
    x = torch.randn(512, 2)
    for t in reversed(range(T)):
        time = torch.full((len(x), 1), t/(T-1))
        eps = net(torch.cat([x, time], 1))
        mean = (x-beta[t]/(1-abar[t]).sqrt()*eps)/alpha[t].sqrt()
        x = mean + post_var[t].sqrt()*torch.randn_like(x) if t else mean
    assert torch.isfinite(x).all()
    print("terminal signal:", round(abar[-1].item(), 6))
    left = x[x[:, 0] < 0]
    print("fraction in left cluster:", round((x[:, 0] < 0).float().mean().item(), 3))
    print("left cluster: mean x", round(left[:, 0].mean().item(), 2),
          "std", round(left[:, 0].std().item(), 2), "(data: -1.5 and 0.25)")
```

在我們的機器上，它印出終端訊號 0.000349、左群集比例 0.441，左群集平均與標準差為 -1.47 與 0.25。群集的形狀是對的；44/56 的分配則顯示這個小網路沒有精確學到各模態的權重。這就是為什麼單一摘要數字永遠不夠：一定要把樣本畫出來，並對照新抽的資料檢查模態涵蓋、模態內離散程度與離群值。

DDIM：同樣的訓練，不同的取樣器

Song、Meng 與 Ermon [5] 注意到 $\mathcal{L}_{\text{simple}}$ 只依賴邊際分布 $q(x_t \mid x_0)$，而不依賴產生它們的馬可夫鏈。許多非馬可夫的前向過程擁有相同的邊際分布，每一個都給出不同的反向取樣器——而且全部都能用同一個訓練好的 ϵ_θ。對任何遞減的步數子序列 $\tau_1 > \tau_2 > \cdots$，**DDIM** 的更新是

$$x_{\tau_{i+1}} = \sqrt{\bar\alpha_{\tau_{i+1}}}\,\hat{x}_0 + \sqrt{1-\bar\alpha_{\tau_{i+1}}-\sigma^2}\,\epsilon_\theta(x_{\tau_i},\tau_i) + \sigma z, \qquad \sigma = \eta\sqrt{\frac{1-\bar\alpha_{\tau_{i+1}}}{1-\bar\alpha_{\tau_i}}\Big(1-\frac{\bar\alpha_{\tau_i}}{\bar\alpha_{\tau_{i+1}}}\Big)},$$

其中 $\hat{x}_0$ 是在 x_{τ_i} 的乾淨估計，$\eta \ge 0$ 控制隨機性。讀法是：「先估計乾淨樣本，再沿著預測的雜訊方向，把它重新加噪到下一個較低的雜訊程度。」當 $\eta = 1$ 且步數連續時，它還原 DDPM 的後驗變異數；當 $\eta = 0$ 時，取樣器是**確定性**的，也是機率流 ODE 的一種離散化。確定性有實際影響：同樣的起始雜訊永遠得到同樣的輸出，所以可以在雜訊空間內插，或把真實影像反推回它的雜訊（DDIM inversion），這是許多影像編輯方法的基礎。

以下獨立範例重複相同的小型訓練設定，讓程式能單獨執行。在同一工作階段實驗時，可直接重用已訓練的 `net`、`abar` 與 `T`：

```
import torch
from torch import nn
```

```
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
T = 100
beta = torch.linspace(.001, .15, T)
alpha = 1-beta
abar = alpha.cumprod(0)
previous = torch.cat([torch.ones(1), abar[:-1]])
post_var = beta * (1-previous) / (1-abar)
net = nn.Sequential(nn.Linear(3, 64), nn.SiLU(),
                    nn.Linear(64, 64), nn.SiLU(), nn.Linear(64, 2))
opt = torch.optim.Adam(net.parameters(), lr=.002)
def data(n):
    side = torch.randint(2, (n, 1))*2-1
    return torch.cat([side*1.5, torch.zeros(n, 1)], 1) + .25*torch.randn(n, 2)
for step in range(3000):
    x0 = data(128)
    t = torch.randint(T, (len(x0),))
    eps = torch.randn_like(x0)
    a = abar[t, None]
    xt = a.sqrt()*x0 + (1-a).sqrt()*eps
    predicted = net(torch.cat([xt, t[:, None].float()/(T-1)], 1))
    loss = F.mse_loss(predicted, eps)
    opt.zero_grad(); loss.backward(); opt.step()
net.eval()

def ddim_sample(n, steps, eta=0.0, seed=1):
    g = torch.Generator().manual_seed(seed)
    taus = torch.linspace(T-1, 0, steps).round().long()
    x = torch.randn(n, 2, generator=g)
    with torch.no_grad():
        for i, t in enumerate(taus):
            time = torch.full((n, 1), t.item()/(T-1))
            eps = net(torch.cat([x, time], 1))
            a_t = abar[t]
            a_prev = abar[taus[i+1]] if i+1 < steps else torch.tensor(1.)
            x0_hat = (x - (1-a_t).sqrt()*eps) / a_t.sqrt()
            sigma = eta*((1-a_prev)/(1-a_t)*(1-a_t/a_prev)).sqrt()
            x = (a_prev.sqrt()*x0_hat + (1-a_prev-sigma**2).sqrt()*eps
                 + sigma*torch.randn(n, 2, generator=g))
    return x

for steps in [5, 20, 100]:
    s = ddim_sample(2000, steps)
    left = s[s[:, 0] < 0]
    print(f"{steps:3d} steps: left fraction {(s[:, 0] < 0).float().mean():.3f}, "
          f"left-cluster mean x {left[:, 0].mean():.2f}, std {left[:, 0].std():.2f}")
```

輸出呈現典型的規律。用 5 步時，左群集中心在 −1.11、標準差 0.39——樣本被拉向中間並被抹開，因為大步長是用直線去走一條彎曲的軌跡。用 20 步時是 −1.37 與 0.30；100 步時是 −1.42 與 0.24，接近資料。改變步數完全不需要重新訓練。

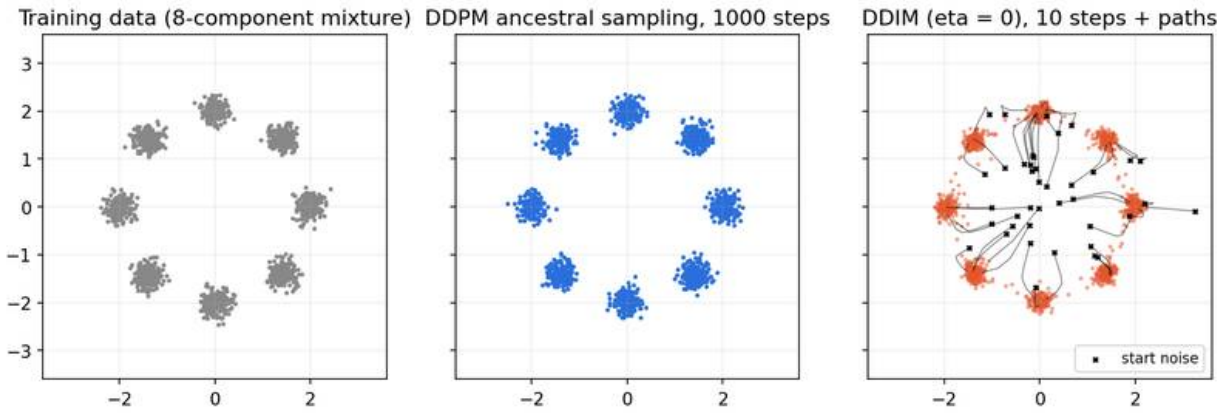


圖 3 — 取樣器都由八成分混合的精確去雜訊器驅動，所以只有取樣規則不同。*1000* 步 *DDPM* 重現了資料；*10* 步確定性 *DDIM* 落在模態附近，但群集略寬、被抹開。黑線是從起始雜訊（×）出發的 *DDIM* 軌跡。

更快的求解器

一旦取樣等同於「解一條 ODE」，數值分析就派得上用場。Karras 等人（EDM）[20] 使用二階 Heun 求解器搭配仔細選擇的雜訊程度間距，並證明取樣器、排程、預條件與訓練雜訊分布可以分開調整。DPM-Solver [21] 利用擴散 ODE 的半線性結構——線性部分可以精確積分——在大約 10 次網路評估內達到良好品質。隨機取樣器在每一步加一點新雜訊；這需要更多步，但可以修正累積的誤差。無論用哪種求解器，都要報告**函數評估次數（number of function evaluations, NFE）**：網路被呼叫的次數才是真正的取樣成本，而不是「步數」。

操控生成：條件與引導

白話版。想生成「一隻貓」而不是「任何東西」，就把條件交給網路。引導則像一個旋鈕：更嚴格地遵循條件，代價是多樣性。

條件化

條件模型學習 $\epsilon_\theta(x_t, t, c)$，其中 c 是條件。類別標籤透過嵌入加到時間嵌入上；文字透過**交叉注意力（cross-attention）**，讓影像特徵去關注文字編碼器的 token [7]；影像（用於超解析度、修補、編輯）則串接到輸入通道，或經由額外的編碼器送入。

分類器引導

對加噪後的邊際分布套用貝氏定理，得到 $\nabla \log p_t(x_t \mid y) = \nabla \log p_t(x_t) + \nabla \log p_t(y \mid x_t)$。Dhariwal 與 Nichol [18] 在加噪影像上另外訓練一個分類器 $p_\phi(y \mid x_t)$，把它的梯度乘上 w 加到無條件模型上。以雜訊單位表示，

$$\hat{\epsilon} = \epsilon_\theta(x_t, t) - w\sqrt{1-\bar{\alpha}_t}\,\nabla_{x_t}\log p_\phi(y \mid x_t).$$

尺度 $w > 1$ 大致是從 $p_t(x_t)\,p_t(y \mid x_t)^w$ 取樣——一個被銳化的分布，用多樣性換取擬真度。加上架構改進，這篇論文展示了擴散模型在 ImageNet [35] 的樣本品質上勝過 GAN [18]。缺點是需要額外的分類器，而且必須在每個雜訊程度的雜訊輸入上訓練。

無分類器引導（CFG）

Ho 與 Salimans [6] 拿掉了分類器。訓練時，隨機一部分樣本的條件被換成「空」標記，於是同一個網路同時學到 $\epsilon_\theta(x_t, t, c)$ 與 $\epsilon_\theta(x_t, t, \varnothing)$。貝氏定理告訴我們，分類器梯度等於條件分數與無條件分數之差，所以上式的「隱式分類器」版本是

$$\epsilon_{\text{guided}} = \epsilon_\theta(x_t, t, \varnothing) + w(\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \varnothing)).$$

在這個慣例下，$w = 0$ 是無條件，$w = 1$ 是普通的條件模型，$w > 1$ 則外插超過它。（有些論文把同一件事寫成 $(1 + w')\epsilon_c - w'\epsilon_u$，因此他們的「尺度」是我們記號中的 $w - 1$；比較數字之前一定要先確認慣例。）

下面的程式使用四團混合分布的精確雜訊預測器，因此任何奇怪的行為都來自引導本身，而不是網路沒訓練好。類別 1 是 $x > 0$ 的那兩團。

```
import numpy as np

# Data: four Gaussian blobs at the corners of a square; class c is the sign of x.
MU = np.array([[-1.5, -1.5], [-1.5, 1.5], [1.5, -1.5], [1.5, 1.5]])
SIG = 0.2
CLASS = np.array([0, 0, 1, 1])

def eps_exact(x, abar, comps):
    """Exact E[eps | x_t] for a Gaussian mixture restricted to `comps`."""
    m = np.sqrt(abar)*MU[comps]
    var = abar*SIG**2 + 1 - abar
    logw = -((x[:, None] - m[None])**2).sum(-1) / (2*var)
    w = np.exp(logw - logw.max(1, keepdims=True))
    w /= w.sum(1, keepdims=True)
    score = -(w[..., None]*(x[:, None] - m[None])).sum(1) / var
    return -np.sqrt(1-abar)*score

abar = np.cumprod(1 - np.linspace(1e-4, 0.02, 1000))
taus = np.linspace(999, 0, 50).round().astype(int)
everything, class1 = np.arange(4), np.where(CLASS == 1)[0]
print("data: mean distance to its mode", round(SIG*np.sqrt(np.pi/2), 3))
for w in [0, 1, 3, 8]:
    x = np.random.default_rng(0).normal(size=(4000, 2))
    for i, t in enumerate(taus):                    # deterministic DDIM
        e_u, e_c = eps_exact(x, abar[t], everything), eps_exact(x, abar[t], class1)
        e = e_u + w*(e_c - e_u)                     # classifier-free guidance
        x0 = (x - np.sqrt(1-abar[t])*e) / np.sqrt(abar[t])
        a_prev = abar[taus[i+1]] if i+1 < len(taus) else 1.0
        x = np.sqrt(a_prev)*x0 + np.sqrt(1-a_prev)*e
    dist = np.linalg.norm(x[:, None] - MU[None], axis=-1).min(1)
```

```
    print(f"w={w}: class-1 fraction {(x[:, 0] > 0).mean():.3f}, "
          f"mean distance to nearest mode {dist.mean():.3f}")
```

資料到其模態的平均距離是 0.251。$w = 0$ 時約一半樣本（0.490）屬於類別 1；$w = 1$ 時全部屬於類別 1，距離為 0.225（50 步 DDIM 略微低估離散程度）。$w = 3$ 時距離為 0.259，$w = 8$ 時增加到 0.446：強引導不只是「讓它更像類別 1」——它把樣本推超過資料，這是影像模型在非常高引導下出現過飽和色彩與變形的玩具版本。圖 4 在八模態混合上呈現同樣的效應。

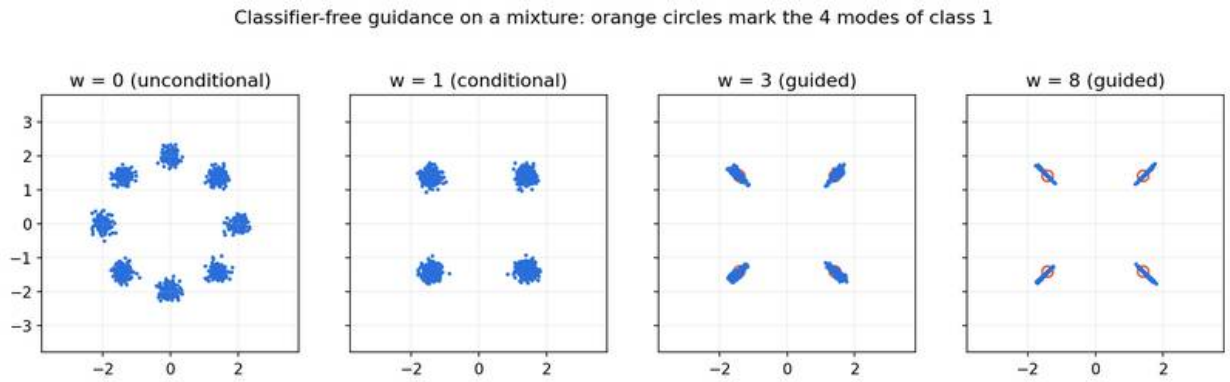


圖 *4* — 在八模態混合上的精確無分類器引導，其中類別 *1* 有 *4* 個模態（橘色圓圈）。$w = 1$ 已經把樣本限制在類別 *1*；更大的 w 讓群集在某些方向收縮、在其他方向拉長——樣本不再服從任何資料分布。

引導也有計算成本：每一步都需要一次條件預測和一次無條件預測，所以 30 步的引導取樣器需要 60 次網路評估，除非兩者被合併計算。而且 w 是取樣的旋鈕，不是影像與提示詞相符程度的校準指標。

潛在擴散與擴散 Transformer

白話版。像素很昂貴。先用自編碼器把影像壓縮成較小的特徵網格，在這個壓縮空間裡做擴散，最後再解碼。另外，去雜訊網路本身也可以是 Transformer，而不是卷積式 U-Net [37]。

潛在擴散

Rombach 等人 [7] 先訓練一個自編碼器：編碼器 $z = \mathcal{E}(x)$ 把 $H \times W \times 3$ 影像映射成 $h \times w \times c$ 的潛在表示，空間降取樣倍率為 $f = H/h$；解碼器 $\mathcal{D}$ 再映射回來。自編碼器以重建、感知與對抗損失，加上輕量的正則化（朝向 Gaussian 的小 KL 項，或向量量化）訓練，讓潛在空間保持良好性質。接著在 z 上照原樣訓練擴散模型，生成最後以 $\hat{x} = \mathcal{D}(z_0)$ 結束。當 $f = 8$ 時，512×512 影像變成 64×64 的潛在表示：每次網路呼叫的空間位置少了 64 倍。

取捨是重建瓶頸：編碼器丟棄的資訊無法由該代碼忠實找回，雖然解碼器可能合成合理細節。潛在表示的尺度也必須與擴散模型訓練時一致；實作上會把潛在表示重新縮放到大約單位變異數。[7] 的文字條件化在 U-Net 中使用交叉注意力層，這也是 Stable Diffusion 系列背後的設計。

擴散 Transformer

DiT（Peebles 與 Xie [8]）把 U-Net 換成在潛在 patch 上運作的 Vision Transformer [41]（見ML 04）。大小為 $h \times w$ 的潛在表示被切成 $p \times p$ 的 patch，得到 hw/p^2 個 token；p 越小，token 越多、計算量越大、品質越好。時間步與類別標籤透過**自適應層正規化（adaptive layer norm）**為每個區塊提供條件：它們預測每個 LayerNorm 的縮放與平移；在「adaLN-Zero」變體中還預測一個初始化為零的閘控係數，讓每個殘差區塊一開始等於恆等映射。這篇論文的主要發現是一個縮放趨勢：在不同模型大小與 patch 大小之間，Transformer 計算量（GFLOPs）越多，FID 就穩定地越低 [8]。

把三個選擇分開來看很有幫助。**架構**（U-Net 或 Transformer）是函數近似器。**目標**（雜訊、x_0、v 或流速度）是它要迴歸的對象。**取樣器**（DDPM、DDIM、ODE 求解器）則決定預測如何變成影像。Stable Diffusion 3 [9] 是一種新組合的例子：一個潛在 Transformer，影像與文字 token 串流使用各自的權重但共同做注意力（「MM-DiT」），以整流流目標訓練，並讓訓練時間步的分布偏向中間雜訊程度。

對影片而言，潛在表示與注意力必須同時涵蓋時間與空間，因此壓縮與注意力模式成為主要的工程問題。Melnik 等人的綜述 [31] 回顧了影片擴散的架構、時間一致性機制與評估。在影片上，逐格品質不夠：物體恆存、動作合理性與長時間一致性都需要各自的評估。

像素空間、潛在空間與表徵自編碼器

白話版。生成器需要一個反覆更新的空間。它可以是影像本身、緊湊的重建代碼，也可以是視覺表徵模型學到的特徵。加入潛在階段，是選擇特定的計算與統計取捨，而不是生成模型必備的條件。

為什麼像素要變成潛在表徵，再變回像素？

訓練影像以像素形式進來，輸出也必須是可觀看的像素。在兩端之間，編碼器可以去除冗餘，產生適合反覆更新、成本較低的網格 [7]。先用重建訓練編碼器與解碼器，再對編碼後的訓練影像學習生成分布。兩項工作不同：重建保留某個指定樣本；生成學習哪些編碼樣本合理。

訓練與取樣路徑如下：

$$\text{training: } x \xrightarrow{\mathcal{E}} z_{\text{data}} \xrightarrow{\text{corrupt}} z_t \xrightarrow{\text{prediction loss}} \theta,$$

$$\text{generation: } z_{\text{noise}} \xrightarrow{\text{learned sampler}} \hat{z} \xrightarrow{\mathcal{D}} \hat{x},$$

$$\text{reconstruction check: } x \xrightarrow{\mathcal{E}} \mathcal{E}(x) \xrightarrow{\mathcal{D}} \tilde{x}.$$

這裡 x 是訓練影像；$\mathcal{E}$ 與 $\mathcal{D}$ 是編碼器與解碼器；z_{data} 是編碼後的影像；z_t 是其受污染狀態；θ 是生成預測器的參數；$\hat{z}$ 是生成的代碼；$\hat{x}$ 是生成影像；$\tilde{x}$ 是重建影像。純文字到影像取樣**不需要**先編碼一張輸入影像。編輯與影像到影像生成則可以編碼實際輸入，從部分污染的代碼開始。

主要節省來自去雜訊器或流預測器會反覆執行，而編碼訓練資料與解碼樣本的次數相對少。空間縮減因子 f 使網格位置減少 f^2 倍，但潛在通道數同樣重要。三通道影像與 c 通道潛在表徵的純量數量縮減比例是 $3f^2/c$。不能把網格縮減比例直接說成所有浮點數量或總執行時間的縮減比例。

對 Transformer，還必須區分詞元數與詞元寬度。完整注意力儲存的矩陣元素數與 N^2 成正比，其中 N 是詞元數；特徵投影也受詞元寬度影響 [41]。像素與潛在空間都能以區塊大小降低 N。這些數量本身不能預測實際速度，因為運算核心、記憶體傳輸、解碼器與條件化也有成本。

```
# Synthetic shape bookkeeping, not a benchmark or a specific checkpoint.
H = W = 512
channels, spatial_factor, patch = 4, 8, 2
pixel_values = H*W*3
h = H//spatial_factor
w = W//spatial_factor
latent_values = h*w*channels
pixel_tokens = (H//patch)*(W//patch)
latent_tokens = (h//patch)*(w//patch)
print("Spatial positions reduction:", (H*W)//(h*w))
print("Scalar values reduction:", pixel_values//latent_values)
print("Dense attention entries reduction:", pixel_tokens**2//latent_tokens**2)
assert pixel_values//latent_values == 48
assert pixel_tokens//latent_tokens == 64
assert pixel_tokens**2//latent_tokens**2 == 4096
```

此人工例子得到空間位置少 64 倍、純量少 48 倍、注意力矩陣元素少 4096 倍，條件是**兩個座標空間都採用數值相同的區塊大小**。改變區塊大小，比較結果就會改變。像素 Transformer 可以使用大得多的區塊，得到與潛在 Transformer 相同的詞元數。

代價是資訊流失。若兩張影像具有相同代碼，確定性的解碼器無法還原各自不同的精確細節。解碼器可以補出合理紋理，但那沒有找回原始影像的資訊。因此，生成訓練之前應獨立評估重建。檢查細線、文字與跨影格變化，而不只看平均影像誤差。

下例的簡單編碼器對區塊取平均，解碼器重複這些平均值。它示範瓶頸，不宣稱重現學習式 VAE 或 RAE。

```
import numpy as np

y, x = np.mgrid[:64, :64]
checker = ((x+y) % 2).astype(float)
smooth = x/63
image = .6*smooth + .4*checker
f = 4
latent = image.reshape(64//f, f, 64//f, f).mean(axis=(1, 3))
reconstruction = np.repeat(np.repeat(latent, f, axis=0), f, axis=1)
print("Synthetic reconstruction MSE:", round(np.mean((image-reconstruction)**2), 6))
assert np.mean((image-reconstruction)**2) > .04
# Recovering the exact latent does not recover the missing checkerboard.
assert np.allclose(reconstruction[0, :f], reconstruction[0, 0])
```

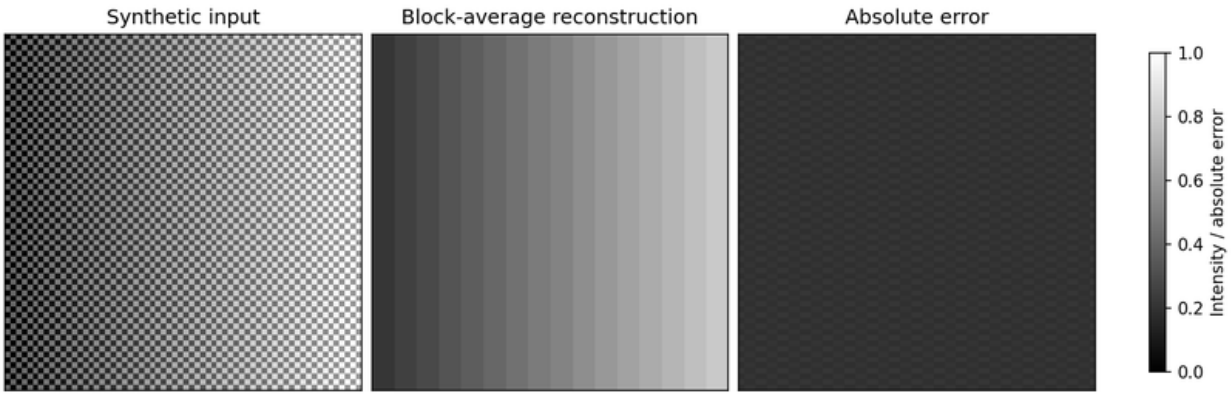


圖 5 — 潛在瓶頸可能在生成模型訓練前就移除細節。區塊平均範例用來解釋資訊流失，並非學習式自編碼器的評估。

在編輯時，反覆編碼與解碼結果可能累積失真。若流程允許，可將中間編輯保留在相容的潛在空間，並檢查最後的像素輸出。優良的無條件生成器仍可能不擅長重建。自編碼器的上限指的是對編碼資訊的忠實還原，並不表示生成解碼器永遠無法合成新的細緻紋理。

重建代碼與語意表徵代碼

VAE 編碼器以重建目標與分布正則化訓練，其代碼設計是為了支援解碼與取樣 [7]。預訓練辨識編碼器則學習有用的視覺特徵。兩者目標可以一致，但辨識所需的不變性也可能捨棄忠實重建需要的細節。解碼器與保留下來的特徵結構，決定表徵是否適合成為生成空間。

Zheng、Ma、Tong 與 Xie 的**表徵自編碼器（Representation Autoencoders，RAEs）**[43]，把凍結的預訓練表徵編碼器搭配可訓練解碼器。其 ICLR 2026 研究以語意視覺特徵作為生成模型的空間，檢查高維詞元特徵、依維度調整的雜訊排程、解碼器穩健性，以及淺而寬的擴散輸出頭。這改變了模型生成的表徵，而不只是對傳統潛在預測器加入特徵對齊損失。

不要把「潛在」等同於「每一維都更小」。空間稀疏的特徵網格仍可能有很多通道。RAE 也不會自動成為變分模型：凍結特徵擷取器加上解碼器，本身並未定義 Gaussian 後驗、KL 正則化或概似下界。應分別測量重建、表徵品質與生成品質。

Tong 等人 [44] 在 2026 年將此方法延伸到自由形式的文字到影像生成。研究在更大規模比較 RAE 與 VAE 空間，並檢查解碼器資料組成。結果顯示，依維度調整的雜訊排程仍重要，而某些小規模架構與解碼器調整在其較大規模設定下幫助有限。這提醒我們在規模與資料變化時重新測試配方，不能把 ImageNet 結果當成普遍設計規則。

對新應用，檢查所選編碼器是否保留重要內容：文件的細小文字、檢測影像的小缺陷，或影片的影格間差異。用保留的真實特徵與生成特徵分別評估解碼器。只用精確訓練代碼學習的解碼器，可能對取樣代碼的小幅分布外誤差敏感。依訓練配方正規化特徵，並將編碼器版本與生成檢查點一起記錄。

JiT：在像素上直接預測乾淨影像

Li 與 He 的 **Just image Transformers（JiT）**[42] 在 2025 年發布預印本，並於 CVPR 2026 發表，重新檢查直接操作像素的大區塊 Transformer。網路預測乾淨影像，再由該預測推得取樣用的流速度。其控制實驗支持以流形解釋的觀點：瓶頸能保留有結構的乾淨影像資訊，卻可能讓直接預測高維雜訊或速度變困難。該實驗設定不使用預訓練潛在詞元化器或額外表徵損失。這是特定架構與訓練條件的證據，不是像素模型總能勝過潛在生成器的證明。

關鍵是區分**網路直接輸出**與**損失或求解器使用的量**。對後面定義的直線路徑，使用 $y_t = (1-t)\epsilon + tx$，$t = 0$ 是雜訊，$t = 1$ 是乾淨資料。直接預測 $\hat{x}_\theta(y_t, t)$ 的網路可以提供：

$$\hat{u}_\theta(y_t, t) = \frac{\hat{x}_\theta(y_t, t) - y_t}{1-t}, \qquad \|\hat{u}_\theta - (x-\epsilon)\|^2 = \frac{\|\hat{x}_\theta - x\|^2}{(1-t)^2}.$$

這裡 x 是乾淨影像；ϵ 是抽出的雜訊；y_t 是兩者內插；$\hat{x}_\theta$ 是直接預測；$\hat{u}_\theta$ 是對應的傳輸速度。恆等式適用於 $t < 1$ 且尚未進行數值截限的情況。因此，輸出 x 的網路可以用速度空間的損失訓練。稱為「乾淨預測」不代表損失是無加權的乾淨影像 MSE。JiT 的釋出實作清楚呈現此差異 [42]。

$t = 1$ 附近，除以 $1 - t$ 必須有端點慣例與數值保護。就內部時間的代數而言，固定乾淨影像誤差越靠近端點，放大程度越高。取樣時應使用檢查點的時間分布、雜訊縮放與求解器配方。只改直接輸出的標籤，無法重現方法。

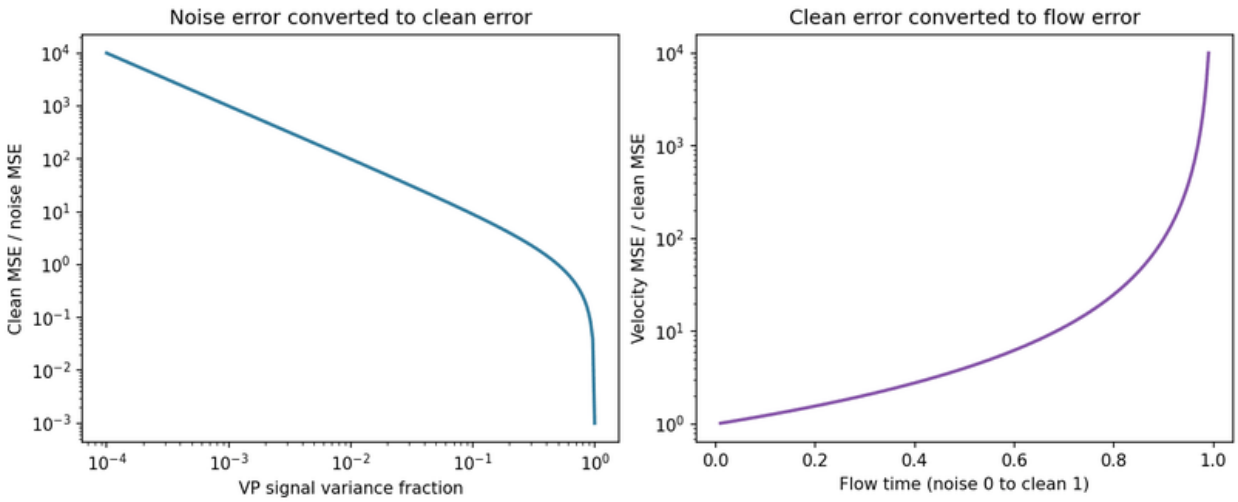


圖 *6* — 等價預測使用不同單位。左圖是低訊號下的雜訊誤差放大；右圖是接近乾淨端點時的速度誤差放大。曲線是代數恆等式，而非模型效能測量。

有意義的比較應固定訓練資料、計算量、解析度、詞元數與取樣器，再逐一改變表徵或目標。依重建需求、生成品質與總成本，比較像素 Transformer、緊湊 VAE 潛在 Transformer 與表徵潛在 Transformer。記錄編碼器／解碼器成本與表徵預訓練。即使生成階段從零開始，使用預訓練視覺編碼器的模型仍承接了先前的訓練。

流匹配：學習一個傳輸速度

白話版。先把雜訊排程放一邊。抽一個雜訊點和一個資料點，用直線連起來，訓練網路預測沿著這類直線上每一點的行進方向。生成時，丟下一個新的雜訊點，跟著預測的方向走。

時間慣例改變。本節的時間從 $t = 0$ **的雜訊走到** $t = 1$ **的資料**，與前面擴散各節相反（前面 t 越大代表雜訊越多）。

條件流匹配目標

連續正規化流用一條 ODE $dx_t/dt = v(x_t, t)$ 移動點。如果速度場 v 正確，它會把雜訊分布 $p_0 = \mathcal{N}(0, I)$ 傳輸到資料分布 p_1。Lipman 等人 [10]（以及同時期的 Liu 等人 [11]、Albergo 與 Vanden-Eijnden 的隨機內插 [34]）展示了如何在完全不模擬 ODE 的情況下訓練 v。

把雜訊 $z \sim \mathcal{N}(0, I)$ 與資料 $x \sim p_1$ 配對，定義直線的**條件路徑**

$$x_t = (1 - t)\, z + t\, x, \qquad \frac{dx_t}{dt} = u_t(x_t \mid z, x) = x - z.$$

每一對都以等速沿直線移動。我們真正想要的邊際速度，是通過該點的所有配對之條件速度的平均：

$$u_t(y) = \mathbb{E}[\, x - z \mid x_t = y \,].$$

我們算不出這個平均，但其實也不需要。比較「理想」損失與我們算得出來的損失：

$$\mathcal{L}_{\mathrm{FM}} = \mathbb{E}_{t, x_t} \| v_\theta(x_t, t) - u_t(x_t) \|^2, \qquad \mathcal{L}_{\mathrm{CFM}} = \mathbb{E}_{t, z, x} \| v_\theta(x_t, t) - (x - z) \|^2,$$

其中 $t \sim \mathcal{U}[0, 1]$。把兩個平方展開。$\| v_\theta \|^2$ 項完全相同。交叉項也相等，因為由重複期望值定律，$\mathbb{E}[v_\theta(x_t, t)^\top (x - z)] = \mathbb{E}[v_\theta(x_t, t)^\top \, \mathbb{E}[x - z \mid x_t]] = \mathbb{E}[v_\theta^\top u_t(x_t)]$。剩下的項都不含 θ。因此

$$\nabla_\theta \, \mathcal{L}_{\mathrm{CFM}} = \nabla_\theta \, \mathcal{L}_{\mathrm{FM}},$$

也就是說，迴歸每一對的速度 $x - z$ 就能訓練出邊際速度場。這和擴散模型是同一個技巧：一個對單一樣本而言充滿雜訊的目標，其條件平均卻是對的。生成時用任何 ODE 求解器，從 $t = 0$ 到 $t = 1$ 解 $dx/dt = v_\theta(x, t)$。

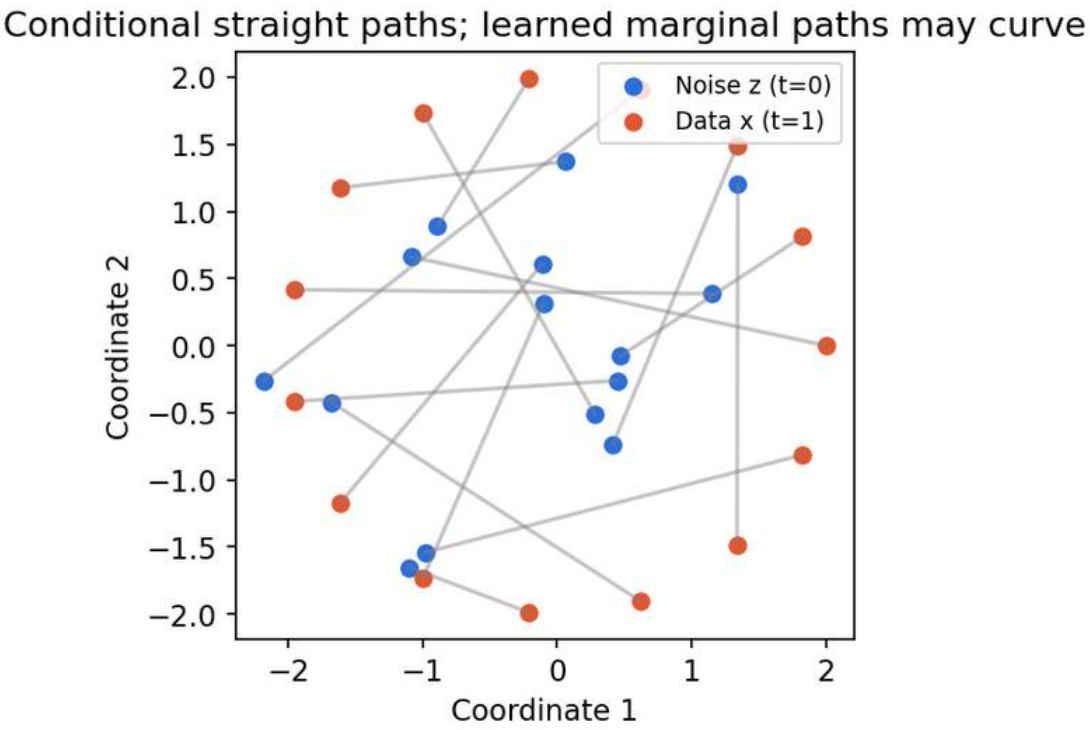


圖 7 — 條件訓練路徑是直線。不同配對的路徑會交叉，所以交叉點的邊際速度是平均值，學到的軌跡可能彎曲。

流匹配與擴散的關係

線性路徑是一條 Gaussian 路徑：給定 x，$x_t \sim \mathcal{N}(t\,x,\ (1-t)^2 I)$。記 $\hat{z} = \mathbb{E}[z \mid x_t]$（一個雜訊預測），並利用 $x = (x_t - (1-t)z)/t$，

$$u_t(x_t) = \frac{x_t - \hat{z}}{t}, \qquad \nabla \log p_t(x_t) = -\frac{\hat{z}}{1-t}, \qquad \text{因此} \qquad u_t(x_t) = \frac{x_t + (1-t)\,\nabla \log p_t(x_t)}{t}.$$

所以對這條路徑而言，流匹配速度、雜訊預測器與分數仍然可以互相換算；不同的是路徑（直線或 VP 曲線）、損失對雜訊程度的加權，以及時間慣例。擴散模型的機率流 ODE 本身就是沿著某條特定 Gaussian 路徑的流。Lipman 等人的指南 [2] 與隨機內插框架 [34] 都把兩者視為同一個建構的特例。實務上的建議與參數化時相同：這些目標是靠推導彼此關聯，而不是改個名字就能互換。

程式：兩個群集上的流匹配

這個獨立範例用線性條件路徑學習與 DDPM 程式相同的兩群集分布，再用步長 $1/N$ 的 Euler 法取樣。

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
net = nn.Sequential(nn.Linear(3, 64), nn.SiLU(),
                    nn.Linear(64, 64), nn.SiLU(), nn.Linear(64, 2))
opt = torch.optim.Adam(net.parameters(), lr=.002)
for step in range(3000):
    z = torch.randn(128, 2)
    side = torch.randint(2, (128, 1))*2-1
    x = torch.cat([1.5*side, torch.zeros(128, 1)], 1) + .25*torch.randn(128, 2)
    t = torch.rand(128, 1)
    xt = (1-t)*z+t*x
    loss = F.mse_loss(net(torch.cat([xt, t], 1)), x-z)
    opt.zero_grad(); loss.backward(); opt.step()
net.eval()
def euler_sample(n, steps, seed=1):
    x = torch.randn(n, 2, generator=torch.Generator().manual_seed(seed))
    with torch.no_grad():
        for k in range(steps):
            t = torch.full((n, 1), k/steps)
            x = x + net(torch.cat([x, t], 1))/steps
    return x

for steps in [1, 4, 50]:
    s = euler_sample(2000, steps)
    assert torch.isfinite(s).all()
    left = s[s[:, 0] < 0]
    print(f"{steps:2d} Euler steps: left fraction {(s[:, 0] < 0).float().mean():.3f}, "
          f"left-cluster mean x {left[:, 0].mean():.2f}, std {left[:, 0].std():.2f}")
```

對 1、4、50 步，它印出左群集平均 -0.18、-1.20、-1.45，標準差 0.12、0.42、0.29。一步 Euler 幾乎落在整體資料平均：在 $t = 0$ 時邊際速度是 $\mathbb{E}[x] - z$，所以一整步會把每一個雜訊點都送到 $\mathbb{E}[x]$。直線的條件路徑並不會讓學到的邊際流變直。圖 8 用八模態混合的精確速度呈現同樣的效應。

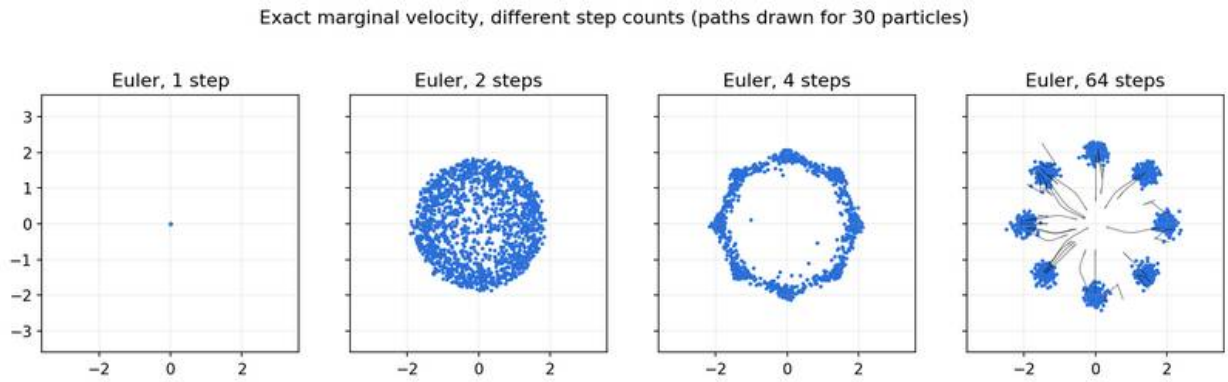


圖 *8* — 即使使用精確的邊際速度，*Euler* 取樣仍需要許多步：一步會把所有樣本塌縮到資料平均，*64* 步的軌跡（黑線）在路徑後段才轉向各模態。

整流流與 reflow

Liu、Gong 與 Liu [11] 使用同樣的直線內插（「整流流」），並加上 **reflow**：訓練後，從每個 z 解學到的 ODE 產生配對 $(z, \hat{x})$，再用這些已耦合的配對（而非獨立配對）重新訓練。因為新配對本來就由流相連，它們的直線較少交叉，學到的軌跡變得更直，需要的 Euler 步數也更少；夠直的流還可以再蒸餾成一步模型。Stable Diffusion 3 [9] 證明了整流流訓練的大型潛在 Transformer 在文字生成影像的規模上可行。

預訓練影像與影片生成器是完整流程

白話版。預訓練生成器是一組相容元件。只載入去雜訊器權重，還不能確定表徵、文字條件、預測目標與取樣排程正確。從影像跨到影片，也會改變重建必須保留的內容。

執行檢查點前，先閱讀元件

實務流程結合文字編碼器、可選的影像／影片編碼器、時間條件預測器、取樣器，以及可選的解碼器。初次生成之後，可能還有獨立的細化模型。每個元件都預期特定張量形狀與縮放。排程器宣告 `v_prediction` 時，可能指 VP 速度；流排程器則可能預期直線內插的導數。應檢查訓練路徑，而不是從設定字串猜測。

以下已核實的例子呈現架構差異。它們是固定的研究基準，不是最新產品排行榜。參考文獻包含論文或技術報告，以及官方釋出文件。

基準	生成表徵與預測器	預訓練流程需要檢查的項目
SDXL [45]	重建潛在表徵與卷積去雜訊器	文字條件、潛在縮放、可選的基礎／細化模型組合與相容取樣設定
Stable Diffusion 3 [9]	潛在整流流與聯合文字／影像 Transformer	流的時間慣例、文字串流、時間抽樣與自編碼器相容性
FLUX.1 [46]	潛在整流流 Transformer	確切釋出版本、引導行為、文字條件、自編碼器與釋出條款
JiT [42]	直接在像素上做類別條件乾淨影像預測	影像正規化、區塊／解析度設定、推得的速度與取樣配方；不是自由文字到影像檢查點
HunyuanVideo [47]	壓縮影片潛在表徵與流匹配 Transformer	影片自編碼器、影像／影片聯合訓練、時間維度與文字條件元件
Wan2.1 [48]	影片擴散 Transformer 與時空自編碼器	文字到影片或影像到影片版本、因果影片壓縮、時間長度與解碼快取

SDXL [45] 說明較大的潛在影像生成架構與可選細化階段。Stable Diffusion 3 [9] 示範相同的大方向如何改用整流流與聯合文字／影像注意力。FLUX.1 的官方模型卡 [46] 描述釋出的整流流 Transformer，`.1-dev` 版本採用引導蒸餾。這些特性不代表檢查點可以互換。一般雙次推論的 CFG 與蒸餾後的引導輸入，可能有不同含義與成本。

影像到影像生成會編碼輸入，並加入部分雜訊，在保留原始內容與改變能力間取捨。實際雜訊程度由排程決定，不只是介面上「strength」這類標籤。比較編輯時，保留相同輸入，記錄初始時間、種子、提示、引導與重建基準。否則可能把重建偽影誤當成文字要求的編輯。

影片加入時間壓縮與一致性問題

令影片有 F 個影格、高度 H 與寬度 W。假想編碼器以 f_t 壓縮時間，以 f_s 壓縮兩個空間軸，約得到 $(F/f_t) \times (H/f_s) \times (W/f_s)$ 個潛在位置。確切形狀取決於填補與首影格的特殊處理。切分區塊後，跨所有位置的密集注意力成本隨剩餘詞元數平方成長；分別壓縮時間與空間，因此可能大幅改變成本。

HunyuanVideo 技術報告 [47] 結合 3D 影片自編碼器、流匹配 Transformer 與分階段影像／影片訓練。Wan 報告 [48] 說明因果影片 VAE，包括首影格特殊處理、空間與時間壓縮，以及分塊編碼／解碼的特徵快取。自編碼器具因果性，不代表整個影片生成器就以自回歸方式取樣；仍須另外檢查生成注意力與取樣程序。

單一影格可能很漂亮，但整段影片可能改變物體身分、突然反轉運動，或忘記被遮蔽的物體。生成前先測試保留影片的重建，再評估外觀、運動、提示對應與跨影格一致性。Melnik 等人 [31] 的綜述提供這些影片特有問題的閱讀地圖。單張影像指標無法診斷時間行為。

預訓練影片生成器也不會自動成為可靠的世界模型。預測合理影片與預測指定動作的後果不同。世界模型章節以狀態、動作、展開預測與評估進一步說明此差異。使用實際的條件化介面，並評估系統宣稱要解決的任務。

可重現的第一個實驗

先記錄流程清單：檢查點版本、編碼器／解碼器版本、文字編碼器、表徵形狀與縮放、預測目標、時間方向、求解器、評估次數、引導、資料型別與種子。若有自編碼器，接著做重建檢查，再對固定的小組提示與種子生成結果。一次比較一項變動。報告潛在方法成本時，納入解碼延遲與記憶體峰值。

本章 CPU 範例不下載權重，用來檢驗基本恆等式與失敗情況。完整解析度的預訓練生成，應依官方釋出說明 [42]、[45]–[48]，在另外配置資源的實驗中執行。實際測量、論文已發表結果與簡化計算，是三種不同證據。

少步與一步生成

白話版。如果取樣器需要 50 次網路呼叫，就訓練第二個模型，用幾次呼叫沿著軌跡跳躍——或者從頭訓練一個本來就設計成會跳的模型。

主要家族有：

- **漸進式蒸餾** [22]：學生模型學著用一步匹配老師的兩個確定性步驟；重複這個程序，每一輪把步數減半。
- **一致性模型** [12]：學習一個函數 $f_\theta(x_t, t)$，把機率流 ODE 軌跡上的任何一點映射到該軌跡的起點（乾淨點）。它必須滿足邊界條件：在最小時間 ϵ 時 $f_\theta(x, \epsilon) = x$，作法是寫成 $f_\theta(x,$

$t) = c_{\text{skip}}(t)\,x + c_{\text{out}}(t)\,F_\theta(x,t)$，並令 $c_{\text{skip}}(\epsilon)=1$、$c_{\text{out}}(\epsilon)=0$。訓練要求同一條軌跡上相鄰兩點的輸出一致。在一致性蒸餾中，相鄰點來自預訓練擴散模型的一步 ODE 求解；在一致性訓練中，則來自同一筆資料在兩個雜訊程度下的加噪版本，不需要老師。取樣只需一步，或幾步「跳到乾淨、重新加噪、再跳」。改進的訓練方法 [32] 簡化了連續時間一致性模型，並在 ImageNet 512 × 512 上把它擴展到 15 億參數、以兩步取樣。

- **對抗式蒸餾**：ADD [33] 結合來自大型預訓練擴散老師的分數蒸餾與對抗損失，讓學生模型在一到四步內產生清晰影像。
- **平均速度模型**：MeanFlow [13] 學習一段區間上的平均速度 $u(x_t,r,t) = \frac{1}{t-r}\int_r^t v(x_s,s)\,ds$，而不是瞬時速度 v。把 $(t-r)\,u = \int_r^t v\,ds$ 對 t 微分，得到恆等式 $u = v - (t-r)\frac{d}{dt}u$，它不需要老師就能提供訓練目標。之後只要對整個區間呼叫一次網路就能生成一個樣本；論文報告在 ImageNet 256 × 256 上從頭訓練、一次評估的 FID 為 3.43 [13]。

少步數是學出來的能力，而不是換個漂亮方程式就免費得到的結果。比較時要計算 NFE（包含引導評估）、潛在模型中解碼器的成本、解析度、品質以及多樣性——也要記得，一步網路每次呼叫可能比多步網路更大、更慢。

評估生成模型

白話版。「看起來好不好？」至少藏了三個問題：單一樣本是否逼真（擬真度）、樣本集合是否涵蓋資料的多樣性（多樣性）、樣本是否符合條件（對齊）。沒有任何單一數字能同時回答三者。

Fréchet Inception Distance

FID（Heusel 等人 [23]）用 Inception-v3 網路 [36] 嵌入真實與生成影像，對兩組特徵各擬合一個 Gaussian，再計算兩個 Gaussian 之間的 Fréchet（2-Wasserstein）距離：

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \operatorname{tr}\!\left(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2}\right),$$

其中 μ_r, Σ_r 與 μ_g, Σ_g 是真實集合與生成集合的特徵平均與共變異數，tr 是矩陣的跡。越低越好。下面的程式把公式套用在二維「特徵」上。

```
import numpy as np
from scipy import linalg

def frechet_distance(a, b):
    """Frechet distance between Gaussians fitted to feature sets a and b."""
    mu1, mu2 = a.mean(0), b.mean(0)
    c1, c2 = np.cov(a, rowvar=False), np.cov(b, rowvar=False)
    covmean = linalg.sqrtm(c1 @ c2).real
    return float(((mu1 - mu2)**2).sum() + np.trace(c1 + c2 - 2*covmean))

rng = np.random.default_rng(0)
centers = np.array([[-1.5, 0.], [1.5, 0.]])
real = centers[rng.integers(2, size=5000)] + .25*rng.normal(size=(5000, 2))
good = centers[rng.integers(2, size=5000)] + .25*rng.normal(size=(5000, 2))
one_mode = centers[[0]] + .25*rng.normal(size=(5000, 2))      # mode collapse
```

```
blurry = .6*rng.normal(size=(5000, 2))                          # mass in the gap
print("good     :", round(frechet_distance(real, good), 3))
print("one mode :", round(frechet_distance(real, one_mode), 3))
print("blurry   :", round(frechet_distance(real, blurry), 3))
small = [frechet_distance(real[:50], good[i*50:(i+1)*50]) for i in range(20)]
print("good, 50 samples: mean", round(np.mean(small), 3), "std", round(np.std(small), 3))
```

正確的取樣器得到約 0.001，只會產生左邊模態的取樣器得到 3.9，把機率質量放在兩個模態之間的「模糊」取樣器得到 0.976。最後一行顯示一個實務陷阱：只用 50 個樣本時，即使完美的取樣器也得到 0.077 ± 0.073，因為樣本數少時 FID 有向上的偏差。只有在樣本數、參考集合、特徵擷取器與前處理都相同時，才能比較 FID。

FID 之外

- **精確率與召回率**（Kynkäänniemi 等人 [24]）把擬真度與多樣性分開：在特徵空間用 k 近鄰球估計真實與生成資料的「流形」；精確率是落在真實流形內的生成樣本比例，召回率則是落在生成流形內的真實樣本比例。我們的「單一模態」取樣器會有高精確率、低召回率。
- **特徵擷取器很重要**。Stein 等人 [25] 做了大規模的人類評估，發現沒有任何常用指標與人類判斷強烈相關；尤其擴散模型的感知真實度沒有反映在 FID 上。他們建議以 DINOv2-ViT-L/14 [39] 的特徵取代 Inception，並指出現有指標無法把記憶（memorization）與欠擬合或模態收縮區分開來。
- **CMMD**（Jayasumana 等人 [26]）把 Inception 特徵換成 CLIP [38] 嵌入，並把 Gaussian 假設換成使用 Gaussian RBF 核的最大平均差異（MMD），得到一個不偏且更省樣本的估計量。
- **概似**。透過機率流 ODE，擴散模型可以報告精確的對數概似（以每維位元數表示）[4]、[19]。概似衡量的東西與視覺品質不同，兩者可能不一致。
- **條件模型**需要對齊檢查（影像是否呈現提示詞所說的內容？），通常靠人工評分或視覺語言模型；也需要對訓練集做記憶檢查。
- **成本**。報告 NFE、引導評估次數、解析度，以及在註明硬體上的實際時間。

評估章節ML 10 會再回到樣本數效應與誠實比較。

現代觀點

綜述與參考書。文獻很多，以下幾篇回顧是很好的地圖。

- Yang 等人 [1]（*ACM Computing Surveys*，2023）把擴散研究組織成三個改進方向——高效取樣、改進概似估計、特殊結構資料（流形、離散資料、不變性）——並涵蓋與 VAE、GAN、流的結合，以及跨領域應用。
- Croitoru 等人 [27]（*IEEE TPAMI*，2023）聚焦於視覺。他們指出三種通用公式化——DDPM、雜訊條件分數網路與 SDE——並將視覺應用分類，例如生成、超解析度、修補、編輯、轉換與分割。
- Cao 等人 [28]（*IEEE TKDE*，2024）從方法面切入：基礎公式化、演算法改進（取樣加速、目標設計、條件化），以及從影像到生物與醫療的應用。

- Lipman 等人 [2]（2024）是一份附程式碼的流匹配指南，涵蓋 Gaussian 與一般機率路徑、與擴散的關係，以及延伸到離散與黎曼資料。
- Lai、Song、Kim、Mitsufuji 與 Ermon [29]（2025）是一本專書長度的著作，把變分、分數與流三種觀點呈現為通往同一個時間相依速度場的三條路，接著討論引導、快速求解器與流映射（少步）模型。
- Shen 等人 [30]（*TMLR*，2025）從演算法層面（訓練、取樣、壓縮）、系統層面（硬體與服務部署）與框架層面綜述效率問題。
- Melnik 等人 [31]（*TMLR*，2024）綜述影片擴散：架構、時間一致性機制、資料集與評估。

2023–2026 的現況。綜合這些回顧與本章的原始論文，可以看到幾個趨勢。(1) 潛在空間中的 *Transformer*：DiT 類主幹 [8] 在許多大型系統中取代了 U-Net，而以整流流目標訓練的文字–影像聯合 Transformer [9] 成為文字生成影像的標準配方之一。(2) 流匹配成為預設目標：直線路徑的流匹配 [10]、[11] 現在與雜訊預測並用或取而代之，擴散則被理解為機率路徑的一種選擇 [2]、[29]。(3) 更少的評估次數：蒸餾 [22]、[33]、一致性模型 [12]、[32] 與平均速度模型 [13] 把取樣降到一到四次網路呼叫，品質逼近多步取樣器。(4) 更好的評估：社群正從以 Inception 為基礎的 FID 轉向更豐富的特徵空間、人類研究與對齊指標 [25]、[26]。(5) 影像之外：影片、3D、音訊與科學資料都重用同一套機制，其中影片的時間一致性與成本是主要的開放挑戰 [31]。

表徵與預測，2025–2026。JiT [42] 測試像素上的直接乾淨影像預測；RAE [43] 測試凍結語意特徵空間中的生成；後續文字到影像研究 [44] 檢查更大規模的行為。這些研究區分模型操作的空間與網路直接預測的目標。前述預訓練例子 [45]–[48] 說明實際影像／影片流程為何需要元件相容性與時間重建。比較時應匹配計算量與資料，並納入表徵預訓練。

開放問題。高解析度與長影片的取樣成本；不犧牲多樣性、也不造成過飽和的有原則的引導；與人類判斷一致且能偵測記憶的評估指標；理解泛化（為什麼模型會產生新樣本而不是複製訓練資料）；有保證的可控性與編輯；以及資料來源、授權同意與濫用——這些是社會問題，而不只是技術問題。下一章ML 08 介紹主要的另一條路線：在像素與 token 上的自迴歸生成，以及它與擴散的混合方法。

重點整理

- 前向過程有封閉解 $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$，所以訓練就是「挑一個隨機雜訊程度，迴歸雜訊」。
- 變分下界的每一項都是加權的雜訊預測誤差；$\mathcal{L}_{\text{simple}}$ 拿掉權重，樣本變好，但它不是概似。
- 在指定路徑下，雜訊、乾淨影像、VP 速度、分數與流速度可以換算；時間方向、損失加權、端點穩定性與網路容量都重要。JiT 直接輸出乾淨像素，再推得訓練與取樣用的速度。
- 取樣就是解反向 SDE 或 ODE：DDPM、DDIM 與高階求解器用同一個訓練好的網路，在步數與精確度之間取捨；要計算 NFE，而不是「步數」。
- 引導用多樣性換取遵循度，CFG 讓每步成本加倍；非常大的權重會把樣本推離資料分布。
- 像素、重建潛在表徵與 RAE 特徵是不同生成空間。潛在壓縮節省反覆更新的成本，但可能流失資訊；預訓練影像／影片流程需要相容的編碼器、預測器、引導與取樣器。
- 條件流匹配透過迴歸每一對的直線速度來訓練邊際速度，但直線的條件路徑不會讓學到的流變直——少步生成需要 reflow、蒸餾、一致性或平均速度訓練。
- 沒有單一指標能衡量品質：FID 依賴樣本數與特徵擷取器；要搭配精確率／召回率、人類或對齊研究，以及成本。

練習

1. **找回乾淨估計**。從 $x_t = \sqrt{\bar\alpha_t}x_0 + \sqrt{1-\bar\alpha_t}\epsilon$ 出發，由雜訊預測推導 $\hat x_0$。如果 ϵ_θ 有誤差 δ，在 $\bar\alpha_t = 0.25$ 與 $\bar\alpha_t = 10^{-4}$ 時，$\hat x_0$ 的誤差有多大？接著由乾淨預測推導直線流速度，說明 JiT 為何能輸出 x 卻使用速度損失。

   提示

   $\hat x_0 = (x_t - \sqrt{1-\bar\alpha_t}\,\epsilon_\theta)/\sqrt{\bar\alpha_t}$，所以誤差是 $\sqrt{1-\bar\alpha_t}\,\delta/\sqrt{\bar\alpha_t}$：在 $\bar\alpha_t = 0.25$ 時約 $1.7\,\delta$，在 10^{-4} 時約 $100\,\delta$。高雜訊時，$\hat x_0$ 是許多可能影像的平均，而不是一張影像。

2. **後驗平均的兩種形式**。把 $x_0 = (x_t - \sqrt{1-\bar\alpha_t}\,\epsilon)/\sqrt{\bar\alpha_t}$ 代入 $\tilde\mu_t$，證明它等於 $\frac{1}{\sqrt{\alpha_t}}(x_t - \frac{\beta_t}{\sqrt{1-\bar\alpha_t}}\epsilon)$。接著把後驗程式中的 `t` 改成 50 與 900，確認公式仍然吻合。

   提示

   使用 $\bar\alpha_t = \alpha_t\bar\alpha_{t-1}$ 與 $\beta_t = 1-\alpha_t$。x_t 的係數變成 $\frac{\beta_t + \alpha_t(1-\bar\alpha_{t-1})}{\sqrt{\alpha_t}(1-\bar\alpha_t)} = \frac{1}{\sqrt{\alpha_t}}$，因為分子等於 $1-\bar\alpha_t$。

3. **計算成本**。一個潛在文字生成影像模型用 30 步 DDIM 取樣，並以分開的條件與無條件前向計算做無分類器引導，最後跑一次解碼器。總共需要幾次去雜訊器評估？若換成一個已把引導「內建」的 4 步蒸餾模型，次數如何變化？對人工形狀範例，把潛在通道改成 16、像素區塊大小改成 16，重新計算純量與注意力矩陣元素數。哪些節省仍成立，哪些取決於區塊切分？

   提示

   60 次去雜訊器評估加一次解碼器；把兩個 CFG 分支批次處理可以減少實際時間，但不會減少計算量。內建引導的 4 步蒸餾模型需要 4 次評估加解碼器。要比較品質與多樣性，而不只是次數。

4. **時間慣例**。擴散各節使用「t 大＝雜訊多」；流匹配一節使用「$t=0$ 是雜訊、$t=1$ 是資料」。用 $s = 1-t$ 重新參數化這個流。dx/ds 是什麼？如果只改時間輸入的標籤，會出什麼錯？

   提示

   $dx/ds = -v(x, 1-s)$。如果把 s 餵給一個以 t 訓練的網路，卻沒有翻轉正負號與時間輸入，就會朝錯誤方向積分，把樣本推離資料。

5. **引導掃描**。在 CFG 程式中加入 $w \in \{0.5, 1.5, 5\}$，並記錄類別 1 樣本中落得比較靠近錯誤類別模態的比例。畫出到最近模態的平均距離對 w 的關係。引導從哪裡開始不再有幫助？

   提示

   在 $w = 1$ 時，精確的條件模型就已把每個樣本放進類別 1，所以更大的 w 在這裡無法提升類別正確率——只會讓群集變形。對一個學習出來、並不完美的條件模型而言，適度的 $w > 1$ 可以提升遵循度，這也是實務上使用它的原因。

6. **取樣器消融實驗**。固定訓練好的流匹配網路。把 1、2、5、20、100 步的 Euler 與二階 Heun 步驟（先用 Euler 預測，再平均兩端的速度）比較。使用相同的起始雜訊，測量群集平均與離散程度。在相同網路呼叫次數下，2 步 Heun 取樣器是否勝過 4 步 Euler？

提示

Heun 每步需要兩次網路呼叫。要畫出樣本，而不只看數字。對一個以瞬時速度訓練的模型做一步 Euler，並不能檢驗另外訓練的一步模型（例如一致性模型或 MeanFlow）。

參考文獻

1. L. Yang, Z. Zhang, Y. Song, S. Hong, R. Xu, Y. Zhao, W. Zhang, B. Cui and M.-H. Yang, "Diffusion Models: A Comprehensive Survey of Methods and Applications," *ACM Computing Surveys*, 2023. arXiv
2. Y. Lipman, M. Havasi, P. Holderrieth, N. Shaul, M. Le, B. Karrer, R. T. Q. Chen, D. Lopez-Paz, H. Ben-Hamu and I. Gat, "Flow Matching Guide and Code," arXiv:2412.06264, 2024. arXiv
3. J. Ho, A. Jain and P. Abbeel, "Denoising Diffusion Probabilistic Models," *NeurIPS*, 2020. arXiv
4. Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon and B. Poole, "Score-Based Generative Modeling through Stochastic Differential Equations," *ICLR*, 2021. arXiv
5. J. Song, C. Meng and S. Ermon, "Denoising Diffusion Implicit Models," *ICLR*, 2021. arXiv
6. J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications; arXiv:2207.12598, 2022. arXiv
7. R. Rombach, A. Blattmann, D. Lorenz, P. Esser and B. Ommer, "High-Resolution Image Synthesis with Latent Diffusion Models," *CVPR*, 2022. arXiv
8. W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," *ICCV*, 2023. arXiv
9. P. Esser, S. Kulal, A. Blattmann et al., "Scaling Rectified Flow Transformers for High-Resolution Image Synthesis," *ICML*, PMLR 235, 2024. PMLR
10. Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel and M. Le, "Flow Matching for Generative Modeling," *ICLR*, 2023. arXiv
11. X. Liu, C. Gong and Q. Liu, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow," *ICLR*, 2023. arXiv
12. Y. Song, P. Dhariwal, M. Chen and I. Sutskever, "Consistency Models," *ICML*, 2023. arXiv
13. Z. Geng, M. Deng, X. Bai, J. Z. Kolter and K. He, "Mean Flows for One-step Generative Modeling," *NeurIPS*, 2025. arXiv
14. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. book site
15. J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan and S. Ganguli, "Deep Unsupervised Learning using Nonequilibrium Thermodynamics," *ICML*, PMLR 37, 2015. PMLR
16. P. Vincent, "A Connection Between Score Matching and Denoising Autoencoders," *Neural Computation*, vol. 23, no. 7, pp. 1661–1674, 2011. doi
17. Y. Song and S. Ermon, "Generative Modeling by Estimating Gradients of the Data Distribution," *NeurIPS*, 2019. arXiv
18. P. Dhariwal and A. Nichol, "Diffusion Models Beat GANs on Image Synthesis," *NeurIPS*, 2021. NeurIPS

19. A. Q. Nichol and P. Dhariwal, “Improved Denoising Diffusion Probabilistic Models,” *ICML*, PMLR 139, 2021. PMLR
20. T. Karras, M. Aittala, T. Aila and S. Laine, “Elucidating the Design Space of Diffusion-Based Generative Models,” *NeurIPS*, 2022. arXiv
21. C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li and J. Zhu, “DPM-Solver: A Fast ODE Solver for Diffusion Probabilistic Model Sampling in Around 10 Steps,” *NeurIPS*, 2022. arXiv
22. T. Salimans and J. Ho, “Progressive Distillation for Fast Sampling of Diffusion Models,” *ICLR*, 2022. arXiv
23. M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler and S. Hochreiter, “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” *NeurIPS* (*NIPS*), 2017. arXiv
24. T. Kynkäänniemi, T. Karras, S. Laine, J. Lehtinen and T. Aila, “Improved Precision and Recall Metric for Assessing Generative Models,” *NeurIPS*, 2019. arXiv
25. G. Stein, J. C. Cresswell, R. Hosseinzadeh et al., “Exposing Flaws of Generative Model Evaluation Metrics and Their Unfair Treatment of Diffusion Models,” *NeurIPS*, 2023. arXiv
26. S. Jayasumana, S. Ramalingam, A. Veit, D. Glasner, A. Chakrabarti and S. Kumar, “Rethinking FID: Towards a Better Evaluation Metric for Image Generation,” *CVPR*, 2024. CVF
27. F.-A. Croitoru, V. Hondru, R. T. Ionescu and M. Shah, “Diffusion Models in Vision: A Survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023. doi
28. H. Cao, C. Tan, Z. Gao, Y. Xu, G. Chen, P.-A. Heng and S. Z. Li, “A Survey on Generative Diffusion Models,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 2814–2830, 2024. doi
29. C.-H. Lai, Y. Song, D. Kim, Y. Mitsufuji and S. Ermon, “The Principles of Diffusion Models,” monograph, arXiv:2510.21890, 2025. arXiv
30. H. Shen, J. Zhang, B. Xiong et al., “Efficient Diffusion Models: A Survey,” *Transactions on Machine Learning Research*, 2025. arXiv
31. A. Melnik, M. Ljubljanac, C. Lu, Q. Yan, W. Ren and H. Ritter, “Video Diffusion Models: A Survey,” *Transactions on Machine Learning Research*, 2024. arXiv
32. C. Lu and Y. Song, “Simplifying, Stabilizing and Scaling Continuous-Time Consistency Models,” *ICLR*, 2025. arXiv
33. A. Sauer, D. Lorenz, A. Blattmann and R. Rombach, “Adversarial Diffusion Distillation,” *ECCV*, 2024. arXiv
34. M. S. Albergo and E. Vanden-Eijnden, “Building Normalizing Flows with Stochastic Interpolants,” *ICLR*, 2023. arXiv
35. J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, “ImageNet: A Large-Scale Hierarchical Image Database,” *CVPR*, 2009, pp. 248–255. doi
36. C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens and Z. Wojna, “Rethinking the Inception Architecture for Computer Vision,” *CVPR*, 2016, pp. 2818–2826. arXiv
37. O. Ronneberger, P. Fischer and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” *MICCAI*, LNCS 9351, pp. 234–241, 2015. arXiv
38. A. Radford, J. W. Kim, C. Hallacy et al., “Learning Transferable Visual Models From Natural Language Supervision,” *ICML*, 2021. arXiv
39. M. Oquab, T. Darcet, T. Moutakanni et al., “DINOv2: Learning Robust Visual Features without Supervision,” *Transactions on Machine Learning Research*, 2024. arXiv
40. B. Efron, “Tweedie’s Formula and Selection Bias,” *Journal of the American Statistical Association*,

vol. 106, no. 496, pp. 1602–1614, 2011. doi
41. A. Dosovitskiy, L. Beyer, A. Kolesnikov et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” *ICLR*, 2021. arXiv
42. T. Li and K. He, “Back to Basics: Let Denoising Generative Models Denoise,” *CVPR*, 2026 (preprint 2025). paper, publication record, official JiT code.
43. B. Zheng, N. Ma, S. Tong and S. Xie, “Diffusion Transformers with Representation Autoencoders,” *ICLR*, 2026 (preprint 2025). proceedings, manuscript.
44. S. Tong, B. Zheng, Z. Wang, B. Tang, N. Ma, E. Brown, J. Yang, R. Fergus, Y. LeCun and S. Xie, “Scaling Text-to-Image Diffusion Transformers with Representation Autoencoders,” *CVPR Findings Track*, 2026. manuscript, author publication record.
45. D. Podell, Z. English, K. Lacey, A. Blattmann, T. Dockhorn, J. Müller, J. Penna and R. Rombach, “SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis,” *ICLR*, 2024. proceedings, official model card.
46. Black Forest Labs, “FLUX.1 [dev],” official pretrained-model card, 2024. model card.
47. W. Kong, Q. Tian, Z. Zhang et al., “HunyuanVideo: A Systematic Framework For Large Video Generative Models,” technical report, arXiv:2412.03603, 2024. report, official release.
48. Team Wan, A. Wang, B. Ai, B. Wen et al., “Wan: Open and Advanced Large-Scale Video Generative Models,” technical report, arXiv:2503.20314, 2025. report, official Wan2.1 release.

ML 08・視覺自迴歸：像素、Tokens 與尺度

把影像分布拆成一連串條件機率：PixelCNN、VQ-VAE 與 VQGAN tokenizer、GPT 式 token transformer、next-scale 預測、遮罩生成模型、連續 token，以及它們與擴散模型的比較。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-08-autoregressive-vision/

你將學到

- 機率的鏈鎖法則（chain rule）如何把「建模整張影像」變成一連串「預測下一塊」的問題，以及如何用每維位元數（bits per dimension）衡量結果。
- 為什麼取樣必須逐步進行，訓練卻可以平行；因果遮罩（causal mask）與遮罩卷積（masked convolution）如何讓預測器看不到自己的答案。
- PixelRNN 與 PixelCNN 如何直接對像素建模，以及過長的序列為什麼讓領域轉向學習出來的 token。
- VQ-VAE 與 VQGAN tokenizer 如何把影像轉成一小格整數，包括 codebook 損失、commitment 損失與直通（straight-through）梯度。
- GPT 式 token transformer（Image GPT、Parti、LlamaGen）、next-scale 預測（VAR）、遮罩生成模型（MaskGIT、Muse）與連續 token 模型（MAR）如何改變「預測單位」。
- 取樣控制（temperature、top-k、top-p、classifier-free guidance、KV cache）如何運作，以及自迴歸生成器和擴散模型的比較。

先看全貌

自迴歸（autoregressive, AR）生成器一塊一塊地建立樣本。每一步，它看著目前為止產生的所有內容，預測下一塊的機率分布，從中抽樣，再把結果接到序列後面。這裡的「一塊」可以是一個像素值、學習出來的 codebook 中的一個整數 token、一個連續特徵向量，甚至是下一個解析度的整張圖。

> **用最簡單的話說**
>
> 想像一格一格地畫漫畫。每一格新的畫面都要和前面已經畫好的格子接得上。練習的時候，你臨摹真正的漫畫，所以永遠知道前面幾格長什麼樣子。自己創作新漫畫時，就只能接受自己先前畫的格子，即使有點歪也一樣。自迴歸影像模型就是這樣：用真實影像練習，再一塊一塊地創造新影像。

這為什麼重要？自迴歸正是大型語言模型背後的配方：用 transformer 做下一個 token 預測。把影像帶進這套配方，就能用同一種架構讀寫文字與圖片、沿用語言模型的部署工程，並用精確的似然（likelihood）衡量進展。代價是影像本來就不是序列，必須有人決定**什麼是 token**，以及**token 用什麼順序產生**。本章大部分內容都在討論這兩個決定。

Xiong 等人的綜述 [1] 依照預測單位，把視覺 AR 模型分成三大類：**以像素為基礎**、**以 token 為基礎**、**以尺度為基礎**。我們沿用這張地圖，再加上遮罩式（非由左至右）與連續 token 模型，它

們模糊了 AR 與擴散之間的界線。前面關於生成模型與擴散與 flow matching的章節提供本章用到的 VAE 與擴散背景；Prince 的教科書 [9] 則以教材層級涵蓋一般的機率基礎。

鏈鎖法則不需要獨立假設

白話版：任何聯合分布都可以寫成一連串「給定前面元素，下一個元素的分布」的乘積。這是機率論的事實，而不是建模假設。

對於有順序的序列 $x = (x_1, \dots, x_N)$，

$$p_\theta(x) = \prod_{i=1}^{N} p_\theta(x_i \mid x_{<i}), \qquad \mathcal{L}_{\text{NLL}}(\theta) = -\sum_{i=1}^{N} \log p_\theta(x_i \mid x_{<i}).$$

其中 x_i 是所選順序中的第 i 個元素，$x_{<i} = (x_1, \dots, x_{i-1})$ 是它之前的所有元素，θ 是網路參數，$\mathcal{L}_{\text{NLL}}$ 是單一訓練樣本的負對數似然（實務上會對整個資料集取平均）。

鏈鎖法則對**任何**順序都精確成立。近似只來自模型本身：有限的網路、有限的上下文視窗，或對每個條件分布形式的限制。離散值的條件分布通常是 softmax；二值像素可用 Bernoulli 分布；連續特徵則需要密度函數或另一個條件生成器（MAR 一節會看到一個例子）。

順序是計算上的選擇，不是對物理世界的主張。**光柵順序（raster order）**（由左到右、由上到下）很方便，但對二維影像來說是任意的：你下方的像素和右邊的像素一樣相關。不同順序會產生不同的條件分布，而有限的模型可能覺得某些順序比較容易學。圖 1 展示本章用到的三種預測單位。

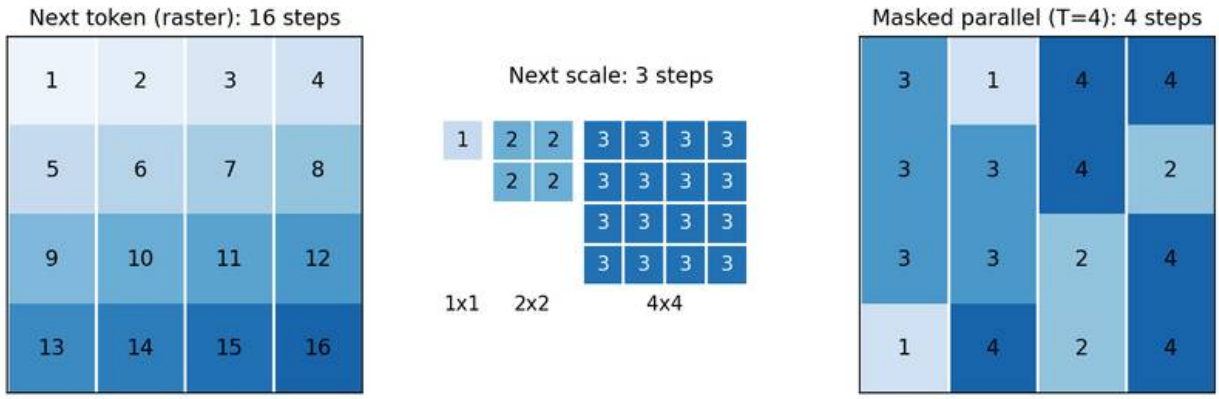


圖 *1* — 以三種生成方式產生同樣 *16* 個最終 *token*。數字代表產生該 *token* 的循序步驟：光柵自迴歸需要 *16* 步，*next-scale* 預測需要 *3* 步（每個尺度一步），$T = 4$ 的遮罩平行解碼需要 *4* 步。僅為示意圖，沒有訓練任何模型。

衡量 AR 模型：每維位元數

因為鏈鎖法則給出精確的 $\log p_\theta(x)$，AR 模型通常用**每維位元數（bits per dimension, bpd）**表示的負對數似然來比較：

$$\text{bpd}(x) = \frac{-\log_2 p_\theta(x)}{D},$$

其中 D 是影像中離散值的個數（例如 32×32 的 RGB 影像 $D = 3 \cdot 32 \cdot 32$），$\log_2$ 把 nat 換成位元。如果模型對每個 8 位元值都給均勻機率，分數恰好是 8 bpd；任何更低的值都代表模型學到了結構。bpd 在保留（held-out）影像上量測，因此同時能偵測過擬合（overfitting）與欠擬合。

有兩點要注意。第一，bpd 和視覺品質可能不一致：模型可能把大部分似然花在肉眼看不到的雜訊上，卻在整體構圖上犯奇怪的錯。第二，對 **token** 模型而言，似然是定義在 codebook 索引上，而非像素上。有損的 tokenizer 會丟掉資訊，所以 token 層級的似然不是精確的像素空間似然，不能直接和像素模型的 bpd 比較。

Teacher forcing、因果遮罩與曝光偏差

白話版：訓練時，模型永遠看到真實的歷史；取樣時，它看到的是自己產生的歷史。遮罩確保訓練過程中沒有任何位置能偷看自己的答案。

使用 **teacher forcing** 時，訓練損失中的每個條件分布都以真實的前文計算。由於整張訓練影像都在手上，只要禁止位置 i 讀取位置 $\geq i$，所有 N 個條件分布就能在一次前向傳遞中算完。因果 transformer 用注意力遮罩做到這一點；PixelCNN 則用遮罩過的卷積核。

常見的輸入慣例有兩種，遮罩必須與慣例相符：

- 在**位移輸入（shifted-input）**的解碼器中，輸入 `[BOS, x1, x2, ...]`，預測 `[x1, x2, x3, ...]`。這時包含對角線的下三角遮罩是正確的，因為位置 i 的輸入是前一個元素。
- 在接收**未位移**值的模型中，依賴關係必須**嚴格**在前。若包含對角線，網路就能直接複製它該預測的值。

推論時，先前抽樣的錯誤會進入後續步驟的上下文。這種訓練與推論的落差常稱為**曝光偏差（exposure bias）**。它不會讓最大似然變成錯的（分解依然精確），但對不完美的模型和長序列確實有實務影響：影像版面一開始的小錯，可能把後續步驟推進訓練中從未出現過的情境。

像素模型：PixelRNN 與 PixelCNN

白話版：把每個像素的強度當成 256 類的分類問題，輸入是它上方和左方的所有像素。

PixelRNN 與 PixelCNN 由 van den Oord、Kalchbrenner 與 Kavukcuoglu 在同一篇論文中提出 [2]。兩者都用 **256 類 softmax** 而不是高斯分布來建模每個 8 位元值，因此分布可以是多峰的（像素可以「要嘛黑、要嘛白，但不是灰」）。色彩則在每個像素內部採固定順序：先紅、再綠、再藍，讓綠色可以依賴紅色，藍色可以依賴兩者。PixelRNN 使用二維 LSTM 層（Row LSTM 與 Diagonal BiLSTM），依賴範圍可能沒有上限；PixelCNN 則以堆疊的遮罩卷積取代，感受野有限，但因為所有位置可以平行計算，訓練快得多。

遮罩卷積

遮罩卷積把卷積核乘上一個二值遮罩，將「未來」位置的權重設為零。在 [2] 中，**遮罩 A** 只用在第一層：它排除中心像素，所以第一層永遠看不到它要預測的值（彩色時，也只看得到已預測的通道）。**遮罩 B** 用在之後的所有層：它允許中心位置，因為第一層之後，中心的特徵已經只匯總了較早的像素。正式地說，對中心在 (c, c)、$c = \lfloor k/2 \rfloor$ 的 $k \times k$ 卷積核，

$$M_{uv} = \begin{cases} 1 & u < c, \\ 1 & u = c \text{ and } v < c \text{ (A) or } v \le c \text{ (B)}, \\ 0 & \text{otherwise}, \end{cases}$$

其中 u 是卷積核的列索引，v 是行索引。下面的程式建立兩種遮罩、堆疊五層，並用梯度找出 9×9 影像中，哪些輸入像素能影響中心位置的輸出。

```
import torch
from torch import nn
from torch.nn import functional as F

class MaskedConv2d(nn.Conv2d):
    """PixelCNN-style masked convolution for one channel (raster order)."""
    def __init__(self, cin, cout, k, mask_type):
        super().__init__(cin, cout, k, padding=k//2, bias=False)
        mask = torch.ones(k, k)
        c = k//2
        mask[c, c + (mask_type == "B"):] = 0   # centre (type A) and pixels to its right
        mask[c+1:] = 0                         # all rows below
        self.register_buffer("mask", mask)
    def forward(self, x):
        return F.conv2d(x, self.weight*self.mask, padding=self.padding)

torch.manual_seed(0)
layers = [MaskedConv2d(1, 8, 3, "A"), nn.Tanh()]
for _ in range(4):                             # 5 layers: reach of 5 pixels
    layers += [MaskedConv2d(8, 8, 3, "B"), nn.Tanh()]
net = nn.Sequential(*layers, MaskedConv2d(8, 1, 1, "B"))
print(net[0].mask.int())
x = torch.rand(1, 1, 9, 9, requires_grad=True)
net(x)[0, 0, 4, 4].backward()                  # logit for pixel (row 4, col 4)
dep = (x.grad[0, 0].abs() > 0).int()
print(dep)
later = torch.zeros(9, 9, dtype=torch.bool)
later[4, 4:] = True; later[5:] = True
assert dep[later].sum() == 0                   # never reads itself or later pixels
print("earlier pixels never seen (blind spot):", int(((dep == 0) & ~later).sum()))
```

assert 通過：目標位置本身及之後的像素都不會影響它的 logit，所以模型是合法的自迴歸分解。但輸出還揭露了另一件事：有六個在光柵順序中較早的像素，在目標右上方排成階梯狀，梯度為零。這就是堆疊遮罩卷積的**盲點（blind spot）**（圖 2）：每一層的感受野往左上方擴張，右上方的區域卻永遠到不了。模型依然正確，只是得到的資訊比它能得到的少。

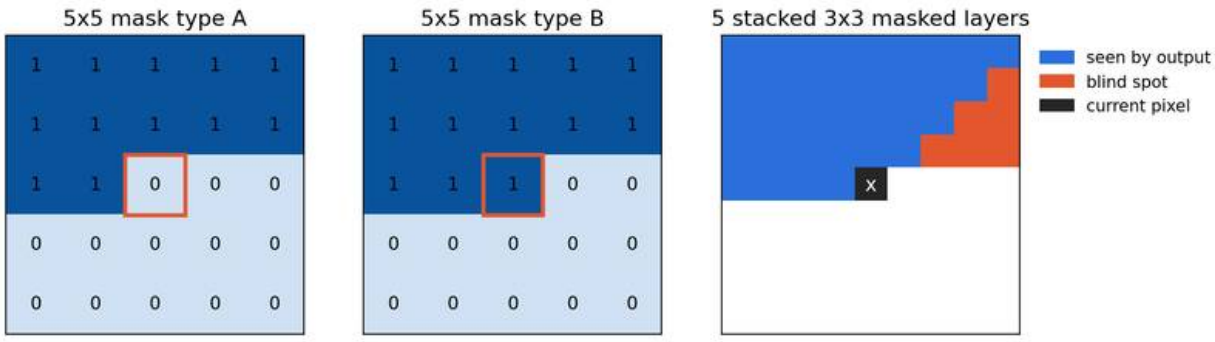


圖 2 — 左、中：遮罩 *A* 排除中心權重，遮罩 *B* 保留中心權重。右：疊了五層 3×3 遮罩卷積後，用梯度量出哪些輸入像素會影響深色格子的預測。橘色格子在光柵順序中較早，卻永遠看不到，也就是盲點。

van den Oord 等人的 Gated PixelCNN [3] 把網路拆成兩個堆疊來消除盲點：**垂直堆疊**不受限制地看到目前這列以上的所有列，**水平堆疊**看到目前這列中直到目前像素為止的部分，兩者在每層之後合併。它也把 ReLU 換成閘控單元 $y = \tanh(W_f * x) \odot \sigma(W_g * x)$，其中 $*$ 是卷積，σ 是 logistic sigmoid，$\odot$ 是逐元素乘法；並示範如何以類別標籤或嵌入向量為條件。加上這些改變後，PixelCNN 在 ImageNet [23] 上達到與 PixelRNN 相當的似然，計算成本卻低得多 [3]。

程式：最小的自迴歸模型

遮罩卷積只是強制因果性的一種方式，也不一定需要 transformer。以下完整的 CPU 實驗產生 4×4 二值影像，每一列都重複同樣四個隨機位元。一個帶遮罩的線性層，用嚴格下三角遮罩排除目前像素與之後所有像素，直接學習條件分布。

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
class BinaryAR(nn.Module):
    def __init__(self, n=16):
        super().__init__()
        self.weight = nn.Parameter(torch.zeros(n, n))
        self.bias = nn.Parameter(torch.zeros(n))
        self.register_buffer("mask", torch.tril(torch.ones(n, n), diagonal=-1))
    def forward(self, x):
        return F.linear(x, self.weight*self.mask, self.bias)

model = BinaryAR()
opt = torch.optim.Adam(model.parameters(), lr=.04)
for step in range(600):
    first_row = torch.randint(2, (128, 4)).float()
    x = first_row.repeat(1, 4)
    loss = F.binary_cross_entropy_with_logits(model(x), x)
    opt.zero_grad(); loss.backward(); opt.step()
```

```
with torch.no_grad():
    a = torch.rand(2, 16)
    b = a.clone(); b[:, 8:] = torch.rand(2, 8)
    assert torch.allclose(model(a)[:, :9], model(b)[:, :9])
    samples = torch.zeros(8, 16)
    for i in range(16):
        p = model(samples)[:, i].sigmoid()
        samples[:, i] = torch.bernoulli(p)
    rows = samples.reshape(-1, 4, 4)
    print("one generated image:\n", rows[0].int())
    print("repeat agreement:", (rows[:, 1:] == rows[:, :1]).float().mean().item())
```

assert 檢查因果性：改變第 8 個位置以後的輸入，不會改變位置 0 到 8 的 logit，因為這些輸出只讀較早的輸入。訓練之後，後面各列與第一列的一致率達到 1.0，而第一列仍保持隨機（模型不應該背下某一列）。

真實分布有 16 張機率相同的影像，所以熵是每張 $\log_2 16 = 4$ 位元，也就是每像素 0.25 位元。後面幾列不含任何新的不確定性。若模型把 16 個像素都當成獨立的公平硬幣，每張影像需要 16 位元。16 與 4 位元之間的差距，正是自迴歸模型能捕捉、而獨立像素模型捕捉不到的結構。

為什麼像素模型擴展不下去

一張 256×256 的 RGB 影像有 $3 \cdot 256 \cdot 256 = 196{,}608$ 個值，所以像素層級的取樣需要大約二十萬次循序的網路計算。大部分似然也被花在細微、局部容易預測的細節上，而人們真正注意的整體構圖只占位元的一小部分。Image GPT [11] 證明，在低解析度影像上以自迴歸方式預測像素的純序列 transformer，學到的特徵在線性探測（linear probing）下遷移效果很好；但在較高解析度下，像素路線依然太昂貴。下一步就是學習更短、更有意義的序列。

離散視覺 token：VQ-VAE 與 VQGAN

白話版：另外訓練一個網路，把影像壓縮成一小格來自學習字典的「單字」，然後對單字做自迴歸，而不是對像素。

VQ-VAE

VQ-VAE [4] 有一個編碼器，在降取樣網格的每個位置輸出特徵向量 $z_e(x) \in \mathbb{R}^d$；一個由 K 個學習向量組成的 **codebook** $\{e_1, \dots, e_K\} \subset \mathbb{R}^d$；以及一個解碼器。每個特徵都被替換成最近的 code：

$$k^* = \arg\min_k \|z_e(x) - e_k\|_2^2, \qquad z_q(x) = e_{k^*}.$$

影像於是由 k^* 值組成的整數網格表示。訓練目標是

$$\mathcal{L}_{\text{VQ}} = \underbrace{\|x - D(z_q)\|^2}_{\text{reconstruction}} + \underbrace{\|\text{sg}[z_e] - e_{k^*}\|^2}_{\text{codebook}} + \beta \underbrace{\|z_e - \text{sg}[e_{k^*}]\|^2}_{\text{commitment}},$$

其中 D 是解碼器，sg[·] 是**停止梯度（stop-gradient）**運算子（前向傳遞時是恆等函數，反向傳遞時梯度為零），β 是 commitment 項的小權重（下面的程式用 0.25）。這裡把重建項寫成平方誤差；也可以換成任何重建似然。

三項各司其職。**codebook** 項把選中的 code 拉向編碼器輸出，就像 k-means 的更新。**commitment** 項把編碼器拉向它的 code，讓編碼器輸出不會離 codebook 太遠。`argmin` 沒有有用的梯度，所以重建損失的梯度要透過**直通估計器（straight-through estimator）**傳到編碼器：前向傳遞使用 z_q，反向傳遞則把 z_q 的梯度原封不動複製給 z_e，彷彿量化是恆等函數。在程式中只要一行：`z_q = z_e + (e - z_e).detach()`。

下面的玩具例子在四個群集上訓練一個二維「編碼器」、codebook 與解碼器，共有八個 code 可用：

```
import torch
from torch import nn
from torch.nn import functional as F

def quantize(z_e, codebook, beta=0.25):
    """z_e: (n, d) encoder outputs; codebook: (K, d). Returns z_q with straight-through
     ↪  gradient."""
    dist = torch.cdist(z_e, codebook)            # (n, K) Euclidean distances
    idx = dist.argmin(1)                         # nearest code index per vector
    e = codebook[idx]
    codebook_loss = F.mse_loss(e, z_e.detach())  # moves codes toward encoder outputs
    commit_loss = F.mse_loss(z_e, e.detach())    # keeps the encoder near its code
    z_q = z_e + (e - z_e).detach()               # forward: e; backward: identity
    return z_q, idx, codebook_loss + beta*commit_loss

torch.manual_seed(0)
torch.set_num_threads(1)
centres = torch.tensor([[2., 0.], [-2., 0.], [0., 2.], [0., -2.]])
encoder = nn.Linear(2, 2)                        # toy "encoder" and "decoder"
decoder = nn.Linear(2, 2)
codebook = nn.Parameter(torch.randn(8, 2)*.1)    # K = 8 codes, 4 are enough
opt = torch.optim.Adam([*encoder.parameters(), *decoder.parameters(), codebook], lr=.03)
for step in range(800):
    x = centres[torch.randint(4, (256,))] + .1*torch.randn(256, 2)
    z_q, idx, vq_loss = quantize(encoder(x), codebook)
    loss = F.mse_loss(decoder(z_q), x) + vq_loss
    opt.zero_grad(); loss.backward(); opt.step()

with torch.no_grad():
    x = centres.repeat_interleave(100, 0) + .1*torch.randn(400, 2)
    z_q, idx, _ = quantize(encoder(x), codebook)
    print("reconstruction MSE:", round(F.mse_loss(decoder(z_q), x).item(), 4))
    print("codes used:", idx.unique().numel(), "of", len(codebook))
    print("codes per cluster:", [idx[i*100:(i+1)*100].unique().tolist() for i in range(4)])
z = torch.randn(3, 2, requires_grad=True)
z_q, _, _ = quantize(z, codebook.detach())
z_q.sum().backward()
print("straight-through gradient:", z.grad[0].tolist())
```

輸出顯示重建 MSE 約 0.0096，只用了 **8 個 code 中的 4 個**（每個群集一個），直通梯度是 `[1.0, 1.0]`。這些數字藏著兩個教訓。重建誤差幾乎正好等於群集內的雜訊變異數 $0.1^2 = 0.01$：量化保留了「屬於哪一群」，丟掉了「在群內哪個位置」。另外，一半的 codebook 是**死碼（dead code）**：從未被選到的 code 收不到 codebook 梯度，永遠不會移動（圖 3）。在真正的 tokenizer 中，codebook 使用率低會浪費容量，是需要監控的典型失敗。

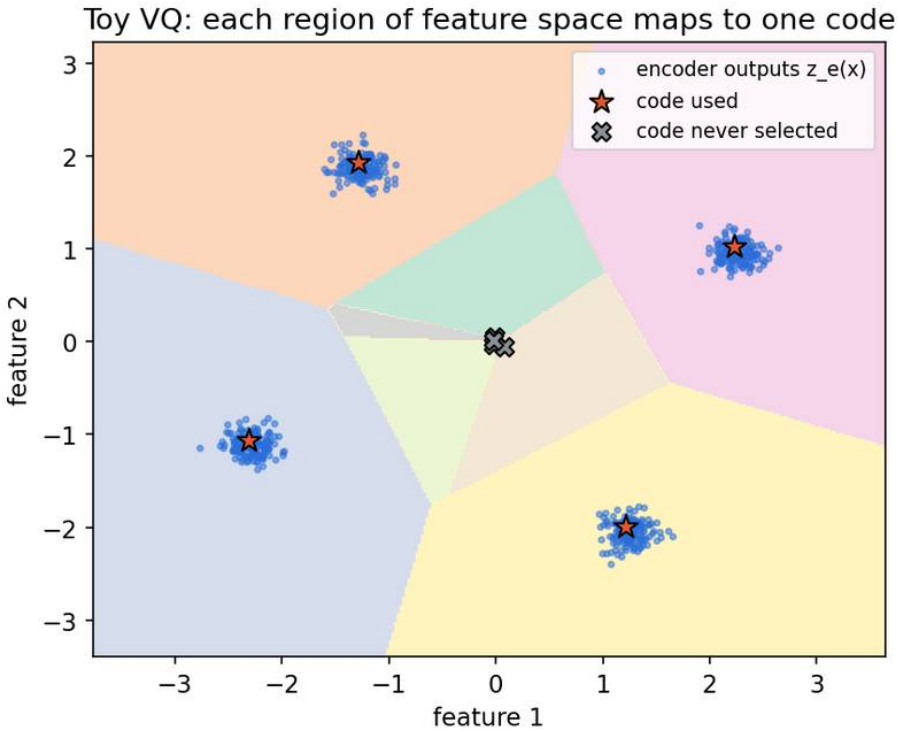


圖 3 — 玩具向量量化。每個色塊是會被映射到同一個 *code* 的特徵集合。四個 *code*（星形）被使用，每群一個；另外四個（叉號）從未被選中，停留在初始化位置附近。

兩個階段：先 tokenizer，再先驗

VQ-VAE 本身只是自編碼器，並不會告訴你哪些 token 網格比較可能出現。因此生成分成**兩個階段**（圖 4）。第一階段訓練 tokenizer。第二階段把它凍結，把每張訓練影像轉成索引網格，再對 M 個索引訓練自迴歸模型 $p_\psi(k_1, \dots, k_M)$。取樣時，先從先驗抽出索引網格，再解碼。[4] 中的先驗是建立在 code 上的 PixelCNN；VQ-VAE-2 [10] 使用兩層 code 圖（較粗的上層與較細的下層）搭配 PixelCNN 式先驗，產生了多樣且高保真度的 ImageNet 樣本。

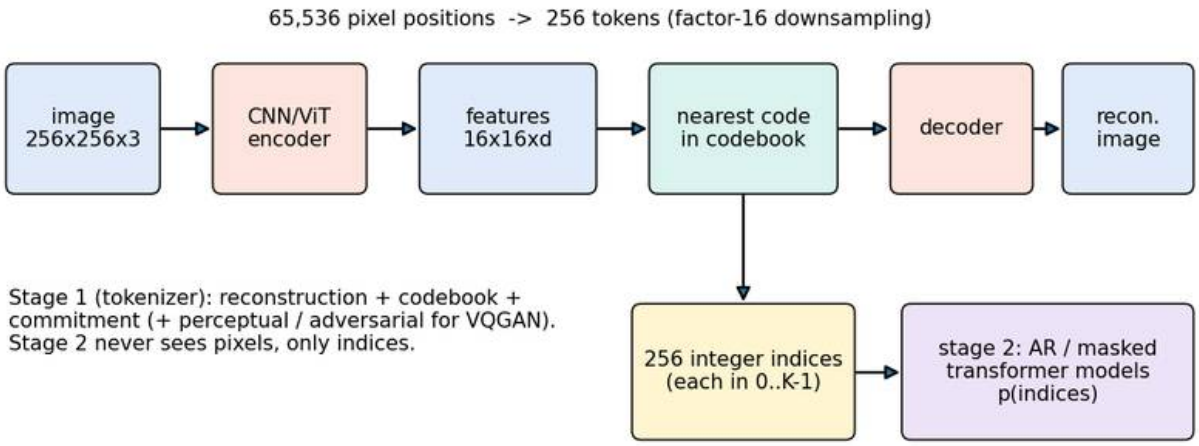


圖 *4* — 兩階段生成。*tokenizer*（上排）為重建而訓練；先驗（右下）只看得到整數索引。以 *16* 倍降取樣，一張 *256×256* 影像變成 *256* 個 *token*，而不是 *65,536* 個像素位置。

簡單的算術說明了它的吸引力。若 tokenizer 把 256×256 影像在每個方向降取樣 16 倍，先驗看到的是 $16 \times 16 = 256$ 個 token，而不是 65,536 個像素位置（或 196,608 個色彩值）。自注意力的成本隨序列長度平方成長，這就是可行與不可行之間的差別。

VQGAN：感知導向的 token

Esser、Rombach 與 Ommer 的 VQGAN [5] 保留兩階段設計，但訓練 tokenizer 時除了重建損失，還加上**感知損失（perceptual loss）**與**以區塊為單位的對抗損失（patch-based adversarial loss）**，並用 transformer 對 token 網格建模。對抗與感知項讓解碼器能從很少的 token 產生清晰、逼真的紋理，因為 token 不再需要精確編碼每個像素，只需要編碼足夠的資訊，讓解碼器產生一個合理的版本。這就是 VQGAN 式 tokenizer 成為 token 式生成標準前端的原因。注意分工：**tokenizer** 使用對抗訓練，**先驗**則是用最大似然訓練的普通自迴歸模型。

Codebook 的問題與替代方案

真實的 tokenizer 反覆遇到三個問題：死碼（如玩具例子）、tokenizer 能重建的內容與先驗能建模的內容之間的落差，以及文字、細線等小細節的遺失。幾種設計分別回應這些問題：

- **ViT-VQGAN** [12] 使用 Vision Transformer 的編碼器與解碼器，在低維的**分解（factorized）**空間中查找 code，並對 code 與特徵做 ℓ_2 **正規化**，改善了 codebook 使用率與重建品質。
- **RQ-VAE** [15] 使用**殘差量化（residual quantization）**：每個位置由一小疊 code 表示，每個 code 量化前面幾個 code 沒表示到的部分。這讓空間網格可以更粗（序列更短），保真度卻相近。
- **FSQ** [16]（有限純量量化，finite scalar quantization）把特徵投影到少數幾維，每一維四捨五入到少數固定的值。隱含的 codebook 大小是各維層級數的乘積；沒有會崩潰的學習 codebook，也不需要 commitment 損失。

壓縮一定會造成瓶頸。請**分別**量測 codebook 使用率、重建品質與生成器品質：生成器無法產生 tokenizer 表示不了的細節。

建立在 token 上的自迴歸 transformer

白話版：一旦影像變成整數序列，生成它就和生成文字是同一個問題：用僅解碼器（decoder-only）的 transformer 預測下一個 token。

從像素到 token

Image GPT [11] 把 GPT 式 transformer 直接用在低解析度像素序列上。它的主要發現是關於**表徵學習**：只訓練來預測下一個像素的模型，其特徵在上面訓練線性分類器時遷移得很好。改用 token 而不是像素，序列短到足以在更高解析度下生成。VQGAN transformer [5] 是經典例子：一個在 VQGAN 索引上以光柵順序運作的類 GPT 模型，讓 transformer 能合成若以像素序列表示會長得無法處理的高解析度影像。

把文字生成影像當成翻譯：Parti

Parti [13] 把文字生成影像視為機器翻譯。transformer **編碼器**讀取文字提示；transformer **解碼器**以它為條件，自迴歸地生成 ViT-VQGAN 影像 token [12]。報告把這個序列到序列（sequence-to-sequence）模型擴展到 200 億參數，並提出 PartiPrompts：一個涵蓋多種內容類別、超過 1,600 個英文提示的基準。

原封不動的語言模型架構：LlamaGen

LlamaGen [14] 問的是：沒有任何影像專屬歸納偏置的標準語言模型架構，擴大規模後能不能把影像生成好？它在 16 倍降取樣、$K = 16{,}384$ 個 8 維 code 的 VQ tokenizer 上，使用 Llama 式 [24] 解碼器（RMSNorm 前置正規化、SwiGLU 前饋層、二維旋轉位置編碼）。類別標籤或投影後的文字嵌入作為**前綴（prefix）**token 輸入，之後以光柵順序生成。論文釋出 1.11 億到 31 億參數的類別條件模型，並指出標準的 LLM 推論服務軟體（vLLM）能加速推論。它更廣的訊息是架構重用：除了 tokenizer，影像生成器幾乎可以和文字模型共用一切。

自迴歸模型的 classifier-free guidance

Classifier-free guidance（CFG）[18] 原本是為擴散模型提出的，但可以直接搬到 AR 的 logit 上。訓練時，條件 c 會隨機被替換成學習出來的**空（null）**嵌入，讓同一個網路同時學會條件與無條件預測。取樣的每一步計算兩組 logit 再組合：

$$\ell_{\mathrm{g}} = \ell_{\mathrm{u}} + s\,(\ell_{\mathrm{c}} - \ell_{\mathrm{u}}),$$

其中 ℓ_{c} 是有條件時的下一個 token logit，ℓ_{u} 是空條件時的 logit，s 是 guidance 強度；$s = 1$ 就回到普通的條件模型。LlamaGen 使用的正是這個形式 [14]。較大的 s 通常提升對提示或類別的貼

合度與樣本銳利度，代價是多樣性。除非把兩次計算合併成一個批次，否則它也會讓每步計算量加倍，所以只看步數會低估成本。

Next-scale 預測（VAR）

白話版：不再依固定閱讀順序預測下一個 token，而是在所有較粗版本的條件下，預測整張影像下一個更清晰的版本。

光柵順序會造成奇怪的依賴：影像左上角在完全不知道右下角的情況下被生成。VAR（Visual AutoRegressive modeling）[6] 主張這種單向順序違背影像 token 之間的雙向相關與空間局部性，因此改用解析度遞增的 token 圖 $r_1, \dots, r_K$ 上的**由粗到細**順序：

$$p(r_1, \dots, r_K) = \prod_{k=1}^{K} p(r_k \mid r_1, \dots, r_{k-1}),$$

其中 r_k 是 $h_k \times w_k$ 的 token 索引網格，r_1 是 1×1，r_K 是完整的潛在解析度。在較粗的圖給定之下，r_k 的所有 token **平行**預測，所以循序步數等於尺度數，而不是 token 數。

tokenizer 是方法的一部分。VAR 的多尺度 VQ 編碼器以**殘差**方式運作：從編碼器特徵圖 f 開始，對 $k = 1, \dots, K$，把 f 降取樣到 $h_k \times w_k$，量化得到 r_k（所有尺度共用一個 codebook），把查到的 code 升取樣回完整解析度，經過一個小卷積 ϕ_k，再從 f 減掉，然後進入下一個尺度 [6]。因此每個尺度編碼的是較粗尺度還沒解釋的部分。單純把普通 token 圖縮放並不能重現這個方法。

訓練時，**區塊式因果（block-wise causal）**注意力遮罩讓 r_k 的每個 token 可以注意 $r_{\le k}$ 的所有 token，但看不到更細的尺度；推論時以 KV cache 儲存較粗的尺度。VAR 也使用 CFG。論文報告在類別條件的 ImageNet 256×256 上，VAR 把其 AR 基準的 FID 從 18.65 改善到 1.73，推論也更快，而且模型呈現類似語言模型的冪律擴展趨勢 [6]。VAR 獲得 NeurIPS 2024 最佳論文獎。和往常一樣，這些數字只適用於論文的資料、解析度與訓練設定。

尺度內的平行預測本身就是一個假設：r_k 中的 token 在給定 $r_{<k}$ 時被獨立抽樣。遮罩模型一節會在玩具問題上展示這個假設的代價。

遮罩生成模型（MaskGIT 與 Muse）

白話版：從一格全是空白的網格開始。每一輪同時猜所有空格，只保留最有把握的猜測，其餘的再塗回空白。

MaskGIT [8] 使用與 AR 模型相同的 VQ token，但改用能看到所有位置的**雙向** transformer。訓練目標是遮罩 token 預測：選一個遮罩比例，把這個比例的 token 換成特殊的 `[MASK]` token，然後在被遮住的位置上最小化交叉熵，

$$\mathcal{L}_{\text{mask}} = -\mathbb{E}\Big[\sum_{i \in \mathcal{M}} \log p_\theta(y_i \mid y_{\bar{\mathcal{M}}}) \Big],$$

其中 y 是 token 網格，$\mathcal{M}$ 是被遮住的位置集合，$y_{\bar{\mathcal{M}}}$ 是沒被遮住的 token。遮罩比例取為 $\gamma(r)$，其中 $r \sim \mathcal{U}(0,1)$，**遮罩排程（mask schedule）**γ 從 1 遞減到 0。

生成進行 T 次迭代。第 t 次迭代時，模型平行預測並抽樣所有被遮住的 token；每個樣本的機率就是它的**信心（confidence）**。這次迭代後仍保持遮住的 token 數為

$$n_t = \lceil \gamma(t/T)\, N \rceil,$$

N 是 token 總數；信心最低的 n_t 個新 token 會被重新遮住，其餘的則固定下來。MaskGIT 比較了數種排程，選擇**餘弦**排程 $\gamma(r) = \cos(\pi r/2)$：早期（模型所知甚少時）只固定少數 token，後期才固定較多；它以 16×16 的 token 網格和 1,024 個條目的 codebook，在 8 次迭代內生成 256×256 影像 [8]。Muse [17] 把同樣的想法擴展到文字生成影像，以預訓練大型語言模型的文字嵌入作為遮罩 token transformer 的條件。

MaskGIT **不是**一般由左到右的鏈鎖法則自迴歸：順序是依信心即時決定的，而且同一步會抽樣好幾個 token。這些同時產生的 token 在目前上下文給定下是**獨立**抽樣的。下面的玩具例子顯示這何時會造成影響。它在前面那個重複列的 4×4 影像上訓練一個小型雙向網路，並用餘弦排程解碼。

```
import math
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
N, MASK = 16, 2                                  # 16 binary tokens; state 2 = [MASK]
gamma = lambda r: math.cos(math.pi*r/2)          # cosine schedule: fraction still masked
net = nn.Sequential(nn.Flatten(), nn.Linear(3*N, 128), nn.ReLU(), nn.Linear(128, N))

def data(n):                                     # 4x4 images whose rows repeat
    return torch.randint(2, (n, 4)).repeat(1, 4)

def logits(tokens):                              # sees every position: bidirectional
    return net(F.one_hot(tokens, 3).float())

opt = torch.optim.Adam(net.parameters(), lr=3e-3)
for step in range(1500):
    x = data(256)
    ratio = torch.tensor([gamma(r) for r in torch.rand(256).tolist()])
    n_mask = (ratio*N).ceil().clamp(min=1)
    masked = torch.rand(256, N).argsort(1) < n_mask[:, None]   # random positions
    inp = torch.where(masked, MASK, x)
    loss = F.binary_cross_entropy_with_logits(logits(inp)[masked], x[masked].float())
    opt.zero_grad(); loss.backward(); opt.step()

@torch.no_grad()
def generate(n, T=4):
    tokens = torch.full((n, N), MASK)
    for t in range(1, T+1):
        p = logits(tokens).sigmoid()
        sample = torch.bernoulli(p).long()
        conf = torch.where(sample == 1, p, 1-p)              # prob. of the drawn value
        still = tokens == MASK
        conf = torch.where(still, conf, torch.inf)           # never re-mask fixed tokens
```

```
        proposal = torch.where(still, sample, tokens)
        n_keep_masked = math.floor(gamma(t/T)*N)              # 0 after the last step
        order = conf.argsort(1)                               # least confident first
        remask = torch.zeros_like(still)
        remask.scatter_(1, order[:, :n_keep_masked], True)
        tokens = torch.where(remask, MASK, proposal)
    return tokens

for T in (1, 2, 4, 8, 16):
    rows = generate(1000, T).reshape(-1, 4, 4)
    agree = (rows[:, 1:] == rows[:, :1]).float().mean().item()
    valid = (rows == rows[:, :1]).all(2).all(1).float().mean().item()
    print(f"T={T:2d}  repeat agreement={agree:.3f}  valid images={valid:.3f}")
```

（程式用 `floor` 而不是論文中的上取整，好讓這個小例子在 $t = T$ 時剩餘數量剛好是零。）輸出為：

```
T= 1  repeat agreement=0.511  valid images=0.000
T= 2  repeat agreement=0.812  valid images=0.100
T= 4  repeat agreement=0.908  valid images=0.360
T= 8  repeat agreement=0.998  valid images=0.982
T=16  repeat agreement=1.000  valid images=0.999
```

$T = 1$ 時，每個 token 都從全遮罩的上下文獨立抽樣，所以各列幾乎從不一致。迭代次數增加後，前幾個 token 早早被固定，後面的 token 可以以它們為條件；到 $T = 8$ 時幾乎每個樣本都合法。在自然影像上，同一輪解碼的 token 之間依賴較弱，所以少數幾次迭代就夠用；但**較少步數**與**較多獨立假設**之間的取捨是真實存在的（圖 5）。

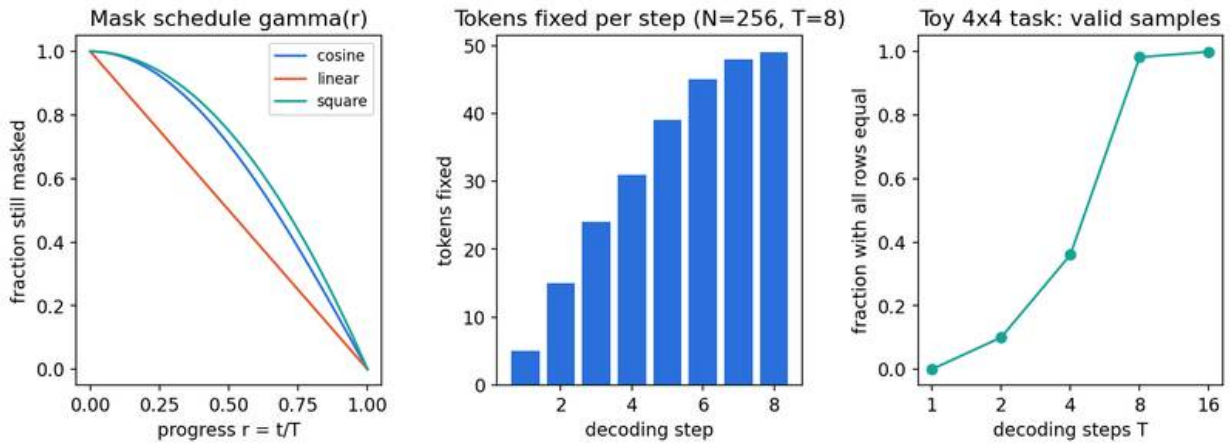


圖 5 — 左：候選的遮罩排程 $\gamma(r)$。中：在餘弦排程、256 個 *token*、$T = 8$ 時，早期只固定少數 *token*，後期固定較多。右：在重複列的玩具任務中，合法樣本比例隨解碼迭代次數增加，因為同一步中被獨立抽樣的 *token* 變少了。

連續 token 與擴散條件分布（MAR）

白話版：自迴歸決定接下來是什麼；一個小型擴散模型可以決定怎麼抽樣它，因此 token 不必再是整數。

向量量化有損，而且不好訓練。Li 等人 [7] 指出，自迴歸需要的是一種建模 $p(x_i \mid x_{<i})$ 的方法，不一定是類別分布。他們讓 AR（或遮罩 AR）transformer 為每個連續值 token x_i 輸出條件向量 z_i，並以**擴散損失（diffusion loss）**訓練一個小型 MLP 去噪器 ϵ_θ：

$$\mathcal{L}(z_i, x_i) = \mathbb{E}_{\epsilon,t}\|\epsilon - \epsilon_\theta(x_{i,t} \mid t, z_i)\|^2, \qquad x_{i,t} = \sqrt{\bar{\alpha}_t}\, x_i + \sqrt{1-\bar{\alpha}_t}\, \epsilon,$$

其中 $\epsilon \sim \mathcal{N}(0, I)$，$t$ 是擴散時間步，$\bar{\alpha}_t$ 是ML 07 中的雜訊排程。梯度也會經由 z_i 流進 transformer。要抽樣一個 token，就以 z_i 為條件執行小型去噪器的反向擴散過程；temperature 用來縮放注入的雜訊。論文的主要變體 **MAR**（masked autoregressive）採隨機順序、每步預測一組 token，把 MaskGIT 的外層結構和內層的連續取樣器結合在一起 [7]。

因此，「AR 對擴散」並不是二選一。一個系統可以用自迴歸決定 token 的順序與分組，再用擴散抽樣每一組。同樣地，語言解碼器可以自迴歸地生成描述影像的文字，而完全不生成像素；統一這些方法的是分解的概念，而不是某種特定的輸出類型。

取樣、temperature 與快取

白話版：訓練好的模型在每一步給出一個分布；取樣器決定抽樣時要多大膽，快取則避免重算過去。

對 K 個可能值的 logit ℓ_k，**temperature** T 會重新縮放分布：

$$p_T(k) = \frac{\exp(\ell_k / T)}{\sum_{j=1}^{K} \exp(\ell_j / T)}.$$

較低的 T 讓機率集中在最高的幾個值；較高的 T 則讓機率分散（圖 6）。**Top-k** 取樣只保留最高的 k 個 logit；**top-p**（nucleus）取樣保留機率總和達到 p 的最小集合。兩者都會把剩下的部分重新正規化。

```
import torch

def filtered_probs(logits, temperature=1.0, top_k=None, top_p=None):
    """Return the (renormalized) distribution actually sampled from."""
    logits = logits / temperature
    if top_k is not None:                                  # keep the k largest logits
        kth = logits.topk(top_k).values[..., -1, None]
        logits = logits.masked_fill(logits < kth, -torch.inf)
    if top_p is not None:                                  # smallest set with mass >= p
        sorted_logits, order = logits.sort(descending=True)
        cum = sorted_logits.softmax(-1).cumsum(-1)
        drop = cum - sorted_logits.softmax(-1) >= top_p  # mass before this token already >=
         ↪  p
        sorted_logits = sorted_logits.masked_fill(drop, -torch.inf)
```

```
        logits = torch.empty_like(logits).scatter(-1, order, sorted_logits)
    return logits.softmax(-1)

logits = torch.tensor([2.0, 1.0, 0.0, -1.0])
for name, kw in [("T=1", {}), ("T=0.5", {"temperature": .5}), ("T=2", {"temperature": 2.}),
                 ("top-k=2", {"top_k": 2}), ("top-p=0.8", {"top_p": .8})]:
    p = filtered_probs(logits, **kw)
    entropy = -(p[p > 0]*p[p > 0].log2()).sum().item()
    print(f"{name:10s}", [round(v, 3) for v in p.tolist()], f"entropy={entropy:.2f} bits")
```

模型本身的分布（$T = 1$）熵為 1.37 位元；$T = 0.5$ 把它降到 0.66 位元，$T = 2$ 提高到 1.80 位元。在這個例子中，top-$k = 2$ 與 top-$p = 0.8$ 都保留最可能的兩個值（第一個值的機率 0.644 小於 0.8，所以還需要第二個）。這些設定全都改變了實際抽樣的分布，因此不要把取樣器的視覺品質和模型的似然混為一談。

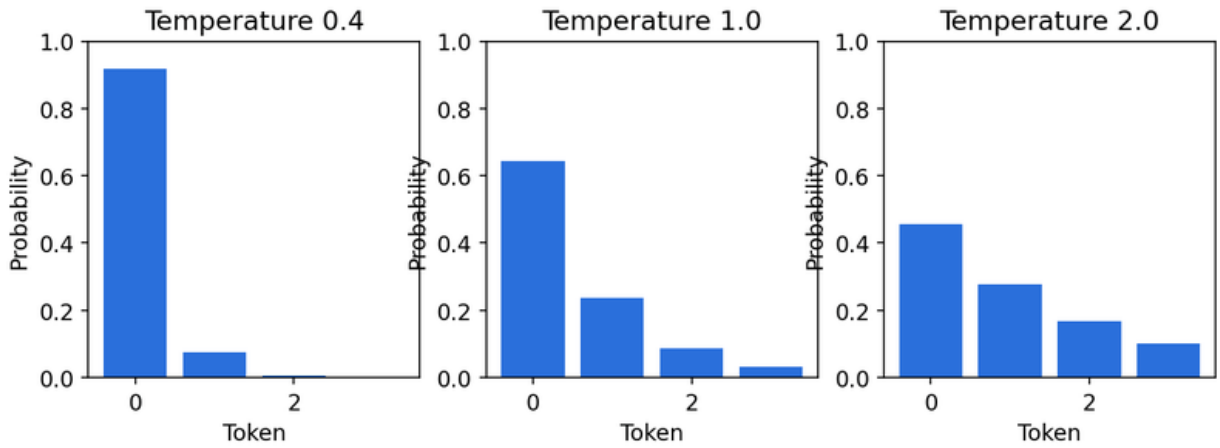


圖 6 — *temperature* 改變每個條件分布的多樣性，但無法找回 *tokenizer* 已經丟掉的資訊。

KV cache

在因果 transformer 中，新增一個 token 時，較早 token 的 key 與 value 不會改變。**KV cache** 把它們存起來，因此每一步只需計算最新 token 的 query、key 與 value，再對快取做注意力：

```
import torch
from torch.nn import functional as F

torch.manual_seed(0)
d, n = 8, 6
Wq, Wk, Wv = (torch.randn(d, d) for _ in range(3))
x = torch.randn(n, d)                                   # embeddings of n tokens

# Full causal attention over the whole sequence (what training computes).
q, k, v = x@Wq, x@Wk, x@Wv
mask = torch.ones(n, n).tril().bool()
```

```
scores = (q@k.T/d**.5).masked_fill(~mask, -torch.inf)
full = scores.softmax(-1)@v

# Incremental decoding: each step computes q, k, v for the newest token only.
K_cache, V_cache, outputs = [], [], []
for i in range(n):
    K_cache.append(x[i]@Wk); V_cache.append(x[i]@Wv)
    K, V = torch.stack(K_cache), torch.stack(V_cache)
    att = F.softmax((x[i]@Wq)@K.T/d**.5, -1)        # attends to tokens 0..i
    outputs.append(att@V)
assert torch.allclose(full, torch.stack(outputs), atol=1e-5)
print("cached decoding matches full causal attention")
print("dot products per step:", [i+1 for i in range(n)], "total:", n*(n+1)//2)
```

assert 確認快取解碼的結果與訓練時的因果輸出完全相同（這裡只有一層；真實模型會快取每一層、每個 head）。快取省去重算，卻沒有消除循序依賴，而且會隨序列長度線性成長。第 i 步仍要對 i 個快取的 key 做注意力，所以長度 n 的序列總注意力工作量是 $n(n+1)/2$，對 n 呈平方成長。這就是更短的序列（更好的 tokenizer）與更少的循序步驟（next-scale 或遮罩解碼）如此重要的原因。

對影片而言，token 可以依空間與時間排序，或分組成區塊。這時快取、壓縮與時間順序主導了成本，而物件一致性與長時間漂移需要另外評估。

自迴歸與擴散的比較

白話版：兩個家族都把一個困難的生成問題拆成許多比較簡單的問題。AR 把影像切成小塊；擴散則把雜訊程度切成多個步驟。

面向	自迴歸（token／尺度）	遮罩生成	擴散／ flow（ML 07）
每步生成什麼	一個 token（或一個尺度）	依信心選出的一組 token	對整個樣本的一次修正
循序步數	token 數或尺度數	固定且通常很少（T）	求解器步數（NFE）
常見表示	離散 code（或搭配擴散頭的連續 token [7]）	離散 code	連續的像素或潛在表示
訓練目標	精確 NLL（交叉熵）	遮罩交叉熵	去噪／速度回歸（與 ELBO 相關）
似然	在 token 上精確	不是單一精確的鏈鎖法則似然	下界或基於 ODE 的估計
天然優勢	與 LLM 共用架構與部署；容易與文字混合	步數少；修補與編輯容易	影像品質強；guidance 彈性大
常見弱點	循序解碼長；tokenizer 瓶頸；錯誤累積	同一步內的獨立假設	除非蒸餾，否則網路計算次數多

有三點讓這張表保持公允。第一，家族之間會混合：MAR [7] 在 AR 迴圈中放了一個擴散模型，而潛在擴散也依賴一個類似 tokenizer 的學習自編碼器。第二，某個家族「勝過」另一個的說法（例如 LlamaGen 的論文標題 [14]，或 VAR 與 DiT 的比較 [6]），只在特定基準、tokenizer、模型大小與 guidance 設定下成立，並非普遍成立。第三，成本應該以網路計算次數、guidance 計算次數、序列長度與實際延遲比較，不能只看「步數」。

現代觀點

這個領域已經從原始像素走向學習出來的表示，從固定的光柵順序走向彈性的預測單位，從獨立的影像生成器走向能同時讀寫文字與影像的模型。以下幾篇綜述描繪了這片版圖：

- **Xiong 等人 [1]**（TMLR）是最直接的視覺自迴歸綜述。它依表示方式分類（以像素、token、尺度為基礎的模型），也就是本章採用的分類，並涵蓋影像、影片、3D、多模態生成，以及具身智慧與醫學等應用。
- **Bond-Taylor 等人 [19]**（TPAMI 2022）並列比較 VAE、GAN、正規化流（normalizing flow）、能量模型與自迴歸模型，包括執行時間、樣本多樣性與架構限制上的取捨。它把 AR 模型放進更大的生成模型版圖中：精確的似然與穩定的訓練，代價是緩慢的循序取樣。
- **Li 等人 [20]**（TPAMI 2026）綜述多模態 LLM 的離散 tokenization，整理向量量化的各種變體，以及它們在傳統系統、單一模態 LLM 與多模態 LLM 中的應用，並討論 codebook 崩潰與梯度估計不穩定等問題。
- **Jia 等人 [22]** 綜述離散 tokenizer 在生成、理解、推薦與檢索中的應用，先拆解 tokenizer 的子模組與運作機制，再依應用整理方法，並列出現有限制與未來方向。
- **Zhao 等人 [21]** 綜述統一的多模態理解與生成模型，把它們分成以擴散為基礎、以自迴歸為基礎，以及 AR 與擴散混合三類，這正是 2025–2026 年大部分研究活動所在。

最新進展（2023–2026）。在經過查證的文獻中，有三條主線特別突出。(i) 在更好的 *tokenizer* 上擴展單純的 *AR transformer*：LlamaGen [14] 顯示，幾乎沒有影像專屬設計的語言模型架構，只要 tokenizer 與訓練夠好，就能在類別條件 ImageNet 上具競爭力。(ii) 改變預測單位：VAR 的 next-scale 預測 [6] 與 MaskGIT、Muse 的遮罩解碼 [8], [17] 減少了循序步數；MAR [7] 用每個 token 一個擴散頭，去掉了向量量化。(iii) 統一的多模態模型：同一套下一個 token 預測機制在單一模型中生成文字與影像 token，tokenizer 也被設計成同時服務理解與生成 [20], [21]。我們的解讀是：選擇**表示方式與預測單位**，如今至少和選擇骨幹網路一樣重要。

開放問題。這些綜述反覆指出以下幾點：

- *Tokenizer* 品質。重建能力決定了生成品質的上限；小字、人臉與精細幾何通常是最先受損的部分。如何在不丟細節的前提下提升 codebook 使用率與語意結構，仍是活躍的研究主題 [20], [22]。
- 離散與連續 *token*。離散 token 適合 LLM 的機制；連續 token 重建得更好，但需要連續的條件模型，例如擴散頭 [7]。
- 速度。循序解碼受延遲限制。next-scale、遮罩與平行解碼以獨立假設換取更少步數。
- 評估。token 似然、FID 類指標、提示貼合度與人類偏好衡量的是不同的東西；有損 tokenizer 的 token 似然並不是像素似然。
- 影片與長上下文。token 數量隨時間暴增，時間一致性、物件恆存性與漂移都需要專門的評估 [1]。

重點整理

- 自迴歸是一種分解方式，不是「transformer」的同義詞：鏈鎖法則對任何順序都精確成立，模型的容量與上下文決定每個條件分布的好壞。

- 在 teacher forcing 下訓練是平行的，取樣則是循序的。位移輸入與因果遮罩（注意力遮罩或 PixelCNN 的遮罩 A ／ B）必須一起設計。
- 像素模型精確但序列很長；學習出來的 tokenizer（VQ-VAE、VQGAN、RQ-VAE、FSQ）縮短序列，代價是生成器無法超越的重建瓶頸。
- Token transformer（Image GPT、VQGAN transformer、Parti、LlamaGen）重用語言模型的機制，包括 logit 上的 classifier-free guidance 與 KV cache。
- Next-scale（VAR）、遮罩（MaskGIT、Muse）與連續 token（MAR）模型改變了預測的**單位**；循序步數越少，通常代表條件獨立假設越多。
- Temperature、top-k 與 top-p 重塑抽樣分布，在多樣性與保真度之間取捨，但無法補回 tokenizer 丟掉的資訊。
- AR 與擴散是互補的分解方式，而且越來越常結合；比較它們時，要在相同的資料、tokenizer、guidance 與計算量下進行。

練習

1. **洩漏答案**。把 `BinaryAR` 的遮罩改成包含對角線（`diagonal=0`）。說明為什麼訓練損失可以降到接近零，取樣卻會失敗。

   提示

   每個輸出現在都能讀到自己的目標並直接複製，所以訓練損失崩落。取樣時，計算目前位置的 logit 時它的值還是 0（尚未抽樣），學到的「複製」規則因此產生無意義的結果。程式中的因果性 assert 也會失敗。

2. **每維位元數**。對重複列的 4×4 二值資料，計算以下情況的每像素位元數：(a) 真實分布；(b) 把每個像素當成獨立公平硬幣的模型；(c) 完美學會第一列、但把後面各列當成獨立公平硬幣的模型。

   提示

   (a) 4 位元 / 16 像素 = 0.25。(b) 16 / 16 = 1.0。(c) 每張 4 + 12 = 16 位元，同樣是每像素 1.0 位元：學會第一列的邊際分布沒有幫助，因為它的位元本來就是均勻的。所有的改善都來自建模後面各列對第一列的依賴。

3. **Tokenizer 預算**。一張 512×512 影像以 8 倍或 16 倍降取樣做 tokenization。分別計算序列長度，以及完整自注意力的成本比。用 16 倍時可能失去什麼？

   提示

   $64 \times 64 = 4096$ 對 $32 \times 32 = 1024$ 個 token。注意力成本隨長度平方成長，所以比值是 16。用 16 倍時，每個 token 要描述一個 16×16 區塊，除非 codebook 或解碼器能補償，否則小字和細線更容易遺失。

4. **盲點**。把遮罩卷積的程式改成 7 層、11×11 輸入、目標在中心。有多少較早的像素永遠看不到？接著畫出垂直堆疊加水平堆疊如何消除它們。

提示

盲點是目標右上方、往左上擴張的感受野永遠到不了的階梯區域；計算光柵順序中較早、梯度為零的格子，答案是 10。垂直堆疊看到上方完整的列（沒有左右限制），並把結果送進只處理目前這列的水平堆疊，於是每個較早的像素都變得可達。

5. **遮罩解碼步數**。用 $n_t = \lceil \gamma(t/T)N \rceil$、餘弦排程、$N = 256$、$T = 8$，計算每一步固定多少 token。為什麼排程一開始只固定少數 token？

提示

剩餘遮罩數為 252、237、213、182、143、98、50、0，所以每步固定 4、15、24、31、39、45、48、50 個 token（圖中使用 floor，數字最多差一）。在精確算術中 $\gamma(1) = 0$；但浮點數的 $\cos(\pi/2)$ 是極小的正數，直接取上整數會留下一個被遮住的 token，這也是程式改用 floor 的另一個原因。一開始模型的上下文很少，平行猜測最不可靠、也最可能互相矛盾；只固定少數 token，能讓後面的步驟以它們為條件。

6. **為方法分類**。對 PixelCNN、VQGAN transformer、Parti、LlamaGen、VAR、MaskGIT 與 MAR，寫出預測單位、表示方式（像素、離散 token、連續 token、多尺度圖）、條件分布類型，以及 16×16 token 網格所需的循序步數。

提示

PixelCNN：光柵順序的像素，256 類 softmax，每個值一步。VQGAN transformer、Parti、LlamaGen：光柵順序的離散 token，codebook 上的 softmax，256 步。VAR：多尺度離散圖，每個 token 一個 softmax、尺度內平行，每個尺度一步。MaskGIT：離散 token，雙向遮罩預測，T 步（論文中為 8）。MAR：隨機順序分組的連續 token，每個 token 一個擴散頭，步數等於組數（每組另有內層擴散步驟）。

參考文獻

1. J. Xiong, G. Liu, L. Huang et al., "Autoregressive Models in Vision: A Survey," *Transactions on Machine Learning Research* (*TMLR*), 2025 (arXiv:2411.05902, 2024). arXiv
2. A. van den Oord, N. Kalchbrenner and K. Kavukcuoglu, "Pixel Recurrent Neural Networks," in *Proc. ICML*, PMLR 48, 2016. PMLR
3. A. van den Oord, N. Kalchbrenner, O. Vinyals, L. Espeholt, A. Graves and K. Kavukcuoglu, "Conditional Image Generation with PixelCNN Decoders," in *NeurIPS*, 2016. NeurIPS
4. A. van den Oord, O. Vinyals and K. Kavukcuoglu, "Neural Discrete Representation Learning," in *NeurIPS*, 2017. NeurIPS
5. P. Esser, R. Rombach and B. Ommer, "Taming Transformers for High-Resolution Image Synthesis," in *Proc. IEEE/CVF CVPR*, 2021. CVF
6. K. Tian, Y. Jiang, Z. Yuan, B. Peng and L. Wang, "Visual Autoregressive Modeling: Scalable Image Generation via Next-Scale Prediction," in *NeurIPS*, 2024 (Best Paper Award). arXiv
7. T. Li, Y. Tian, H. Li, M. Deng and K. He, "Autoregressive Image Generation without Vector Quantization," in *NeurIPS*, 2024. arXiv

8. H. Chang, H. Zhang, L. Jiang, C. Liu and W. T. Freeman, "MaskGIT: Masked Generative Image Transformer," in *Proc. IEEE/CVF CVPR*, 2022. CVF
9. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. Book
10. A. Razavi, A. van den Oord and O. Vinyals, "Generating Diverse High-Fidelity Images with VQ-VAE-2," in *NeurIPS*, 2019. NeurIPS
11. M. Chen, A. Radford, R. Child, J. Wu, H. Jun, D. Luan and I. Sutskever, "Generative Pretraining From Pixels," in *Proc. ICML*, PMLR 119, 2020. PMLR
12. J. Yu, X. Li, J. Y. Koh et al., "Vector-quantized Image Modeling with Improved VQGAN," in *ICLR*, 2022. arXiv
13. J. Yu, Y. Xu, J. Y. Koh et al., "Scaling Autoregressive Models for Content-Rich Text-to-Image Generation," *Transactions on Machine Learning Research*, 2022. arXiv
14. P. Sun, Y. Jiang, S. Chen, S. Zhang, B. Peng, P. Luo and Z. Yuan, "Autoregressive Model Beats Diffusion: Llama for Scalable Image Generation," arXiv:2406.06525, 2024. arXiv
15. D. Lee, C. Kim, S. Kim, M. Cho and W.-S. Han, "Autoregressive Image Generation using Residual Quantization," in *Proc. IEEE/CVF CVPR*, 2022. CVF
16. F. Mentzer, D. Minnen, E. Agustsson and M. Tschannen, "Finite Scalar Quantization: VQ-VAE Made Simple," in *Proc. International Conference on Learning Representations* (*ICLR*), 2024. arXiv
17. H. Chang, H. Zhang, J. Barber et al., "Muse: Text-To-Image Generation via Masked Generative Transformers," in *Proc. ICML*, PMLR 202, 2023. PMLR
18. J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," arXiv:2207.12598, 2022 (short version at the NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications). arXiv
19. S. Bond-Taylor, A. Leach, Y. Long and C. G. Willcocks, "Deep Generative Modelling: A Comparative Review of VAEs, GANs, Normalizing Flows, Energy-Based and Autoregressive Models," *IEEE TPAMI*, vol. 44, no. 11, 2022. doi
20. J. Li, Y. Fu, J. Liu, L. Cao, W. Ji, M. Yang, I. King and M.-H. Yang, "Discrete Tokenization for Multimodal LLMs: A Comprehensive Survey," *IEEE TPAMI*, 2026 (arXiv:2507.22920). doi
21. S. Zhao, X. Zhang, J. Guo et al., "Unified Multimodal Understanding and Generation Models: Advances, Challenges, and Opportunities," arXiv:2505.02567, 2025; revised 2026. arXiv
22. J. Jia, J. Gao, B. Xue et al., "From Principles to Applications: A Comprehensive Survey of Discrete Tokenizers in Generation, Comprehension, Recommendation, and Information Retrieval," arXiv:2502.12448, 2025. arXiv
23. J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, "ImageNet: A Large-Scale Hierarchical Image Database," *CVPR*, 2009, pp. 248–255. doi
24. H. Touvron, T. Lavril, G. Izacard et al., "LLaMA: Open and Efficient Foundation Language Models," arXiv:2302.13971, 2023. arXiv

ML 09・基礎模型與視覺語言學習

什麼樣的模型稱為基礎模型：縮放定律、預訓練目標、CLIP 與 SigLIP、DINOv2 特徵、提示式分割、多模態語言模型、以 linear probe 與 LoRA 適配，以及如何評估風險。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-09-foundation-models/

你將學到

- 什麼樣的模型稱為基礎模型（foundation model）；為什麼這個詞描述的是訓練與重用方式，而不是某種架構，也不一定要能生成文字。
- 縮放定律（scaling laws）說了什麼（帶有下限的冪律、計算最佳配置），以及沒有說什麼。
- 主要的預訓練目標——監督式、影像文字對比、遮罩影像建模、自蒸餾與生成式——以及各自傾向產生什麼樣的特徵。
- CLIP、SigLIP、DINOv2/DINOv3 與 Segment Anything 系列如何作為可重用的視覺元件。
- 多模態大型語言模型如何把視覺編碼器接到語言模型，以及 LLaVA 式指令微調如何運作。
- 如何適配預訓練模型——linear probe、微調、LP-FT、adapter、prompt tuning、LoRA、提示——以及如何誠實地評估結果，包括偏誤、幻覺與測試集污染。

先看全貌

在電腦視覺的大部分歷史裡，每個任務各有一個模型：一個網路分類花朵、另一個偵測汽車、再一個分割腫瘤，每個都從頭訓練，或從 ImageNet [43] 分類器開始。**基礎模型**則在廣泛的資料上、以大規模訓練一次，接著**適配**到許多下游任務 [1]。這個詞描述的是模型如何被訓練與重用，而不是一種新的網路層。DINOv2 [15] 或 DINOv3 [16] 影像編碼器、CLIP 式影像文字模型 [3]、SAM [5] 這類提示式分割器、多模態語言模型，以及廣泛預訓練的影像生成器，都可以扮演這個角色。

> **用最簡單的話說**
>
> 與其每換一份工作就從零教一個新學生，不如找一位已經看過幾百萬張圖、讀過幾百萬句圖說的人。他已經知道邊緣、輪子、臉和狗長什麼樣子。遇到新工作時，你只需要給他幾個例子、一句簡短的說明，或一件小小的新工具——不必重新上一整套學校。

為什麼重要？有三個理由。第一，**標註很昂貴**，而預訓練把大部分學習移到不需要人工標籤的資料上（圖說、影像本身）。第二，**一個骨幹網路服務許多任務**，讓工程投入更集中。第三，它改變了**介面**：很多時候不必重新訓練，只要用一段文字提示、一次點擊或幾個範例就能指定任務。Bommasani 等人把這種方法的集中稱為同質化（homogenization），並提醒它同時也集中了失敗：共用骨幹的一個缺陷，會被建立在它之上的每個系統繼承 [1]。

本章延續ML 05・表示學習（什麼樣的特徵可以重用）、ML 04・Vision Transformer（常用骨幹），以及生成模型章節ML 07 與ML 08。Bishop 與 Bishop 的教科書 [14] 對遷移學習與表示有系統性

的說明，可作為參考。本章沿用 Awais 等人的視覺基礎模型綜述 [39] 與 Caffagni 等人的多模態綜述 [2] 所畫出的架構地圖，並引用為 2026 年 10 月版本查證過的代表性報告。這些例子用來說明模型家族，不是在替商業產品排名，也不宣稱某個 checkpoint 在所有情況都最好。

什麼樣的模型稱為「基礎」模型

白話版。基礎模型是在廣泛資料上、以通用目標預訓練的大型模型，它的內部表示或使用介面可以重用在從未專門訓練過的任務上。

Bommasani 等人 [1] 在 2021 年提出這個名稱，並強調兩個性質：

- **湧現（emergence）**。能力來自規模與訓練目標，而不是被明確設計出來。例如 CLIP 只被訓練去配對影像與圖說，卻能把影像分類到用文字命名的類別 [3]；DINO 完全沒有使用標籤，它的注意力圖卻能勾勒出物體輪廓 [24]。
- **同質化（homogenization）**。許多應用收斂到少數幾個預訓練骨幹與訓練配方。

使用「湧現」時要小心。本章中它只表示一種不是訓練目標的能力，並不代表無法預測或神秘的跳躍；某個能力看起來是否「突然」出現，常常取決於用來衡量它的指標。

形式上，令 f_θ 為在上游資料集 $\mathcal{D}_{\text{up}}$ 上以目標 $\mathcal{L}_{\text{pre}}$ 預訓練的模型：

$$\theta^\star = \arg\min_\theta \; \mathbb{E}_{x \sim \mathcal{D}_{\text{up}}}[\mathcal{L}_{\text{pre}}(f_\theta; x)].$$

下游任務 k 提供一個（通常很小的）資料集 $\mathcal{D}_k$ 與任務損失 ℓ_k。適配就是選一小組新的或被修改的參數 ϕ_k，並求解

$$\phi_k^\star = \arg\min_{\phi_k} \sum_{(x,y)\in\mathcal{D}_k} \ell_k(h_{\phi_k}(f_{\theta^\star}, x), y),$$

其中 h_{ϕ_k} 是我們放在預訓練模型上方或內部的任何東西：線性分類頭、adapter、LoRA 更新、提示，或什麼都不放（純提示，$\phi_k = \emptyset$）。整章都圍繞上面兩行：$\mathcal{L}_{\text{pre}}$ 與 $\mathcal{D}_{\text{up}}$ 應該是什麼，以及 ϕ_k 可以多小。

基礎模型**不一定**要產生文字。一個凍結的影像編碼器，若它的特徵供一百個不同分類器使用，就是基礎模型。反過來，為單一狹窄任務訓練的大模型，不論多大都不算。

縮放定律

白話版。用更多計算量訓練更大的模型、餵更多資料，測試損失通常會平滑且可預測地下降——在對數–對數圖上大致是一條直線——直到接近某個下限。正是這種可預測性，讓研究團隊能用小規模實驗規劃大型訓練。

Kaplan 等人 [17] 訓練了多組 Transformer 語言模型，報告損失與模型大小、資料量及計算量之間呈冪律關係，部分趨勢跨越超過七個數量級；在測試範圍內，寬度與深度的影響很小。對模型大小 N 的冪律寫成

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N},$$

其中 L 是保留資料上的交叉熵損失，N 是不含嵌入層的參數數量，N_c 是與 N 同單位的擬合常數，$\alpha_N > 0$ 是擬合指數。取對數得 $\log L = \alpha_N \log N_c - \alpha_N \log N$：斜率為 $-\alpha_N$ 的直線。

真實曲線會彎曲，因為有一部分損失永遠消不掉（資料本身就有雜訊或歧義）。因此 Hoffmann 等人 [18] 擬合了帶有不可約項的形式：

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}},$$

其中 D 是訓練 token 數，E 是不可約損失，A, B, α, β 是擬合常數。對稠密 Transformer，訓練計算量常用的近似是 $C \approx 6ND$ 次浮點運算 [17]（每個參數、每個 token，前向約 2 次、反向約 4 次）。固定 C，在 N 上最小化 L（令 $D = C/6N$），就得到**計算最佳**（compute-optimal）的模型大小。Hoffmann 等人訓練了超過 400 個語言模型，結論是：在計算最佳的訓練下，模型大小與訓練 token 數應等比例擴大——模型大小每加倍，token 數也應加倍 [18]。這和 Kaplan 等人較早「用相對適量的資料訓練非常大的模型」的建議不同 [17]；這個分歧提醒我們，擬合出的指數取決於實驗流程（學習率排程、納入哪些實驗、計算量怎麼算）。

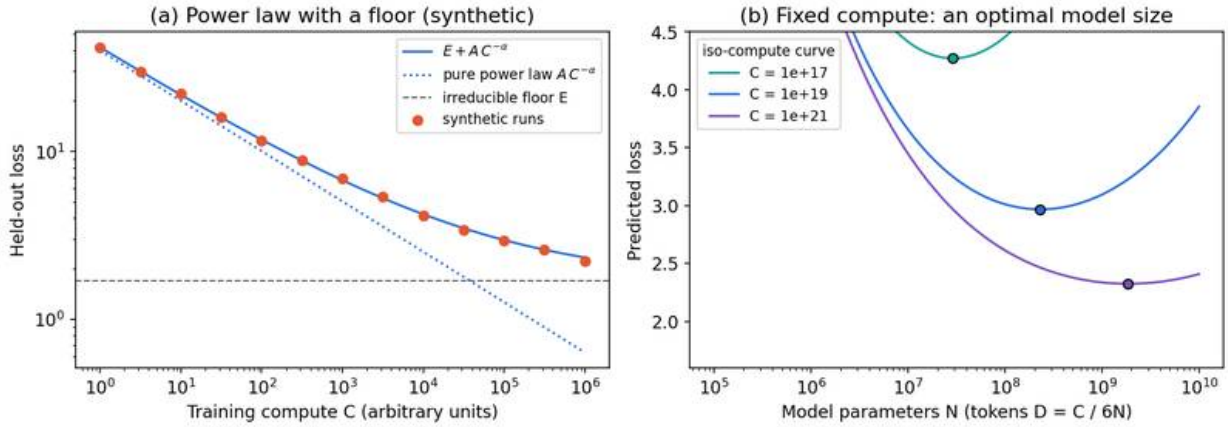


圖 *1* — (*a*) 帶有不可約下限 E 的冪律：純冪律（點線）會一直下降，在大規模時過度樂觀。(*b*) 固定計算量時，模型太小會欠擬合，太大則看到的 *token* 太少，因此每個預算都有最佳大小。所有常數皆為合成。

視覺領域也有自己的縮放研究。Zhai 等人 [19] 針對在大型標註資料上訓練的 Vision Transformer [21]，刻畫錯誤率、資料量與計算量的關係，並擴大到二十億參數的模型。Cherti 等人 [20] 在公開的 LAION 資料集 [34] 上訓練 CLIP 式模型，發現零樣本分類、檢索、linear probing 與微調都呈冪律縮放——但也發現 OpenAI 的 CLIP 與 OpenCLIP 在架構相同、配方相近的情況下，縮放行為仍不同，因為兩者的**訓練資料分布**不同。資料不是一個純量。

程式：擬合縮放定律並外插

下面的程式在 13 個合成訓練實驗中，只用較便宜的那幾個擬合 $L(C) = E + A\,C^{-\alpha}$，再預測昂貴的那幾個。所有資料都是合成的；重點在流程。

```
import numpy as np
from scipy.optimize import curve_fit

rng = np.random.default_rng(0)
# Synthetic "runs": loss = E + A * C^(-alpha), with 2% multiplicative noise.
E_true, A_true, alpha_true = 1.7, 40.0, 0.3
compute = np.logspace(0, 6, 13)            # arbitrary compute units
loss = (E_true + A_true * compute**-alpha_true) * np.exp(.02 * rng.normal(size=13))

def law(c, E, A, alpha):
    return E + A * c**-alpha

# Fit only the cheaper runs, then extrapolate to the larger ones.
small = compute <= 1e4
(E, A, alpha), _ = curve_fit(law, compute[small], loss[small], p0=(1, 10, .5))
print(f"fit: E={E:.2f}  A={A:.1f}  alpha={alpha:.3f}")
for c, l in zip(compute[~small], loss[~small]):
    print(f"C={c:9.0f}  observed {l:.3f}  predicted {law(c, E, A, alpha):.3f}")

# Naive straight line in log-log space ignores the floor E.
slope = np.polyfit(np.log(compute[small]), np.log(loss[small]), 1)[0]
print(f"pure power-law slope without floor: {slope:.3f}")
```

輸出：擬合得到 $E \approx 1.73$、$A \approx 40.0$、$\alpha \approx 0.300$；外插的損失與含雜訊的觀測值相差約 7% 以內；若在對數–對數空間畫一條不含下限的直線，斜率約為 -0.25，指數是錯的。有兩個教訓可以帶到實務：一定要納入下限；跨越多個數量級的外插應視為帶誤差範圍的預測。

縮放定律沒告訴你的事

- 它預測的是預訓練分布上的**平均損失**，不是你的任務準確率。下游指標可能平滑改善、飽和，或呈階梯式變化。
- 它只在**固定配方與資料分布**下成立 [20]。更好的資料過濾或不同的目標都會改變常數。
- 它完全不談**推論成本**。用更多資料訓練的較小模型，在相同損失下部署可能更便宜，這也是許多上線模型會訓練超過計算最佳點的原因。
- 資料是有限的。高品質資料用完後，不論再加多少計算量，B/D^{β} 這一項都不會再縮小。

預訓練目標：目標值從哪裡來？

白話版。每種預訓練方法都會發明一個預測任務。差別在於「答案」從哪裡來：人工標籤、圖說、同一張影像被遮住的部分、另一個網路，或下一個 token。

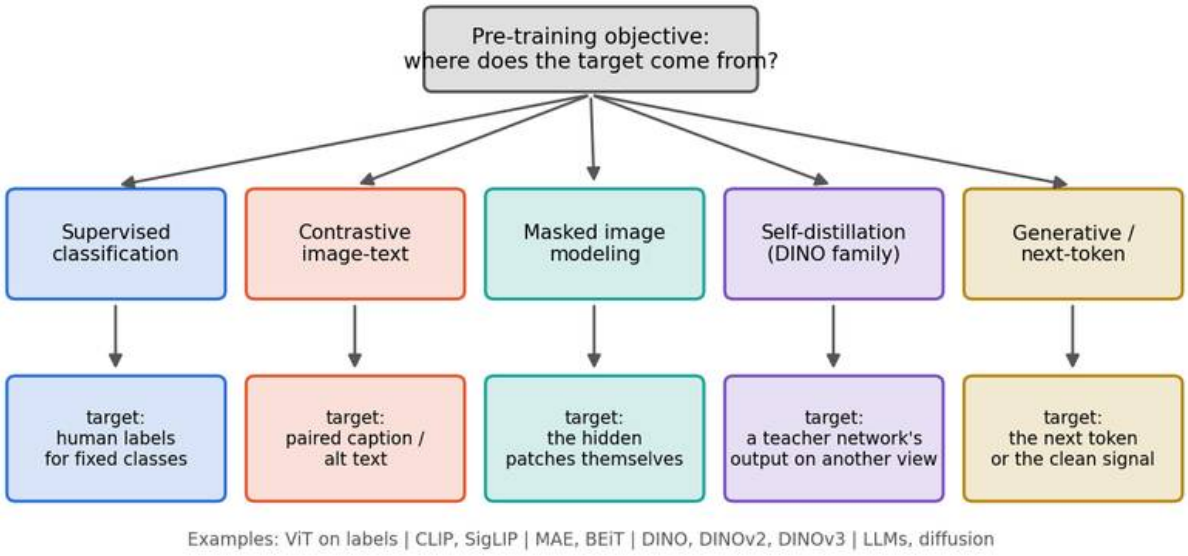


圖 2 — 依訓練目標值的來源，把預訓練目標分成五個家族。

目標	目標值來源	代表模型	典型強項
監督式分類	人工標籤	在大型標註資料上訓練的 ViT [19], [21]	在標籤空間內有強的全域語意
影像文字對比	成對的圖說／ alt text	CLIP [3]、ALIGN [25]、SigLIP [4]	以文字定義的辨識、檢索
遮罩影像建模	同一張影像被遮住的 patch	BEiT [23]、MAE [22]	微調後很強；保留局部細節
自蒸餾	教師網路在另一個視角上的輸出	DINO [24]、DINOv2 [15]、DINOv3 [16]	凍結特徵、密集對應
生成式／下一個 token	下一個 token 或乾淨訊號	LLM、擴散（ML 07）、自迴歸（ML 08）	生成；語言介面

監督式預訓練

在大型標註資料集上用交叉熵訓練分類器，然後拿掉最後一層。這是十年來的 ImageNet 配方，在大規模下仍然有效 [19], [21]。它的弱點是特徵被固定的標籤集合塑形：標籤不需要的區分可能會被丟掉。

影像文字對比學習

網路影像本來就附有圖說，因此 CLIP [3] 與 ALIGN [25] 同時訓練影像編碼器與文字編碼器，讓批次中配對正確的組合分數高於配錯的組合。ALIGN 顯示，只做最少過濾、雜訊很多的大型 alt-text 語料，仍能得到強的表示 [25]。損失函數的細節在下一節。

遮罩影像建模

把影像大部分遮住，再預測缺少的部分。BEiT [23] 遮住 patch 後預測它們的離散視覺 token，借用 BERT [44] 的遮罩語言建模概念。MAE [22] 直接預測原始像素，並觀察到遮住高比例（例如 75%）的 patch 會形成一個不平凡的任務；它的編碼器只看可見的 patch，再由輕量解碼器重建其餘部分，使訓練很有效率。令 $\mathcal{M}$ 為被遮住的 patch 索引集合，x_p 為 patch p 的（正規化）像素，$\hat{x}_p$ 為重建結果，

$$\mathcal{L}_{\text{MAE}} = \frac{1}{|\mathcal{M}|} \sum_{p \in \mathcal{M}} \|\hat{x}_p - x_p\|_2^2.$$

損失只在被遮住的 patch 上計算：預測可見的 patch 只是複製，沒有意義。遮罩建模的特徵在微調後常常非常好，但作為 linear probe 的凍結特徵時，通常比對比式或自蒸餾的特徵弱。

自蒸餾：DINO 家族

DINO [24]——作者形容為「沒有標籤的自蒸餾」——訓練一個**學生**網路 g_s，讓它在同一張影像的不同增強視角（裁切）上去配合**教師**網路 g_t。兩者都輸出 K 維分數，再用溫度 τ_s 與 τ_t 的 softmax 轉成機率：

$$P_s(x') = \text{softmax}\left(\frac{g_s(x')}{\tau_s}\right), \qquad P_t(x) = \text{softmax}\left(\frac{g_t(x) - c}{\tau_t}\right), \qquad \mathcal{L} = -\sum_{k=1}^{K} P_t(x)_k \log P_s(x')_k.$$

其中 x 與 x' 是兩個視角，c 是教師輸出的移動平均（**置中**，centering），較小的 τ_t 讓教師分布變得尖銳（**銳化**，sharpening）。置中避免某一維獨大；銳化避免輸出變成均勻分布；兩者合起來防止**崩塌**（collapse）——也就是每張影像都對應到相同輸出的平凡解。教師不接收梯度，它的權重是學生權重的指數移動平均（動量編碼器）：

$$\theta_t \leftarrow \lambda\, \theta_t + (1 - \lambda)\, \theta_s, \qquad 0 < \lambda < 1.$$

DINO 也使用多重裁切（multi-crop）訓練：學生看幾個小的局部裁切，教師看全域裁切，迫使模型學會局部與全域的對應。作者報告，所得到的 ViT 特徵含有明確的物體分割資訊，這在監督式 ViT 或卷積網路中沒有那麼明顯；而且這些特徵很適合作為 k 最近鄰分類器 [24]。

DINOv2 [15] 把這個配方擴大，關鍵是整理資料：用自動化流程建立專門、多樣且經過整理的影像資料集，而不是使用未整理的網路影像；訓練一個 10 億參數的 ViT，再蒸餾成較小的模型。作者報告這些凍結特徵在他們大多數影像層級與像素層級的基準上超越 OpenCLIP [15]。DINOv3 [16] 進一步擴大資料與模型，並提出 **Gram anchoring**，針對一個已知問題：長時間訓練時，密集（逐 patch）特徵圖的品質會退化。它也加入事後（post-hoc）策略，提升對解析度、模型大小與文字對齊的彈性。

為什麼密集特徵重要。分類器每張影像只需要一個向量；分割、深度估計、對應與追蹤則需要**每個 patch** 都有好的向量。自蒸餾編碼器之所以成為這些密集任務常用的凍結骨幹，正是因為它們的 patch 特徵不需要微調就保有語意。

生成式預訓練

預測下一個 token（語言模型、ML 08 中的自迴歸影像模型）或去雜訊（ML 07 的擴散模型）也是預訓練目標。對語言來說，它產生最有彈性的介面——用自然語言下指令——這也是多模態模型把視覺接到預訓練語言模型上，而不是反過來的原因。

影像與文字的共享空間：CLIP 與 SigLIP

白話版。訓練一個影像編碼器與一個文字編碼器，讓對應的圖片與圖說指向相近的方向。之後任何一句話都可以變成一個「類別」。

CLIP [3] 把影像嵌入 v_i 與文字嵌入 t_j 正規化成單位長度，並以可學習的溫度 τ 形成分數 $s_{ij} = v_i^\top t_j / \tau$。對一個含 B 組配對樣本的批次，

$$\mathcal{L}_{\text{CLIP}} = \frac{1}{2}\left[\frac{1}{B}\sum_i -\log \frac{\exp(s_{ii})}{\sum_j \exp(s_{ij})} + \frac{1}{B}\sum_j -\log \frac{\exp(s_{jj})}{\sum_i \exp(s_{ij})}\right].$$

每張影像應在 B 個圖說中選出配對的那一個（第一項，逐列 softmax），每個圖說也應選出配對的影像（第二項，逐欄）。批次中其他組合都當作負樣本。訓練使用成對的文字監督，通常來自既有的圖說或 alt text；下游的「零樣本」使用並不代表模型訓練時沒有監督。

推論時，把「a photo of a dog」這類候選描述編碼，再與影像比較。文字嵌入取代了學習式分類器的類別權重：

$$\hat{y} = \arg\max_k v(x)^\top t(\text{prompt}_k).$$

候選集合與措辭都會影響結果；把多個模板（「a photo of a k」「a close-up of a k」）的嵌入平均，是降低敏感度的常見作法。零樣本的意思是在所述流程下沒有針對任務訓練參數；它不保證這些類別、甚至測試影像本身，不曾出現在預訓練資料中（見〈風險與評估〉）。

程式：配對損失與檢索機制

以下是**合成嵌入**，用來讓計算一目了然。它們不是預訓練模型的輸出，配對結果也不能證明真實世界的零樣本準確率。

```
import torch
from torch.nn import functional as F

images = F.normalize(torch.tensor([[1., .1, 0.], [.1, 1., 0.],
                                   [0., .1, 1.]]), dim=-1)
texts = F.normalize(torch.eye(3), dim=-1)
scores = images @ texts.T / .1
labels = torch.arange(3)
loss = (F.cross_entropy(scores, labels) +
        F.cross_entropy(scores.T, labels))/2
print("matching text:", scores.argmax(1).tolist())
```

```
print("paired loss:", round(loss.item(), 4))
ambiguous = F.normalize(torch.tensor([[.5, .5, .2]]), dim=-1)
print("cosines:", (ambiguous @ texts.T).tolist())
```

它會印出 `matching text: [0, 1, 2]`、約 `0.0002` 的配對損失，以及模糊影像約 `0.68, 0.68, 0.27` 的餘弦值：它與兩個描述一樣接近，任何 softmax 都無法告訴你哪一個才對。

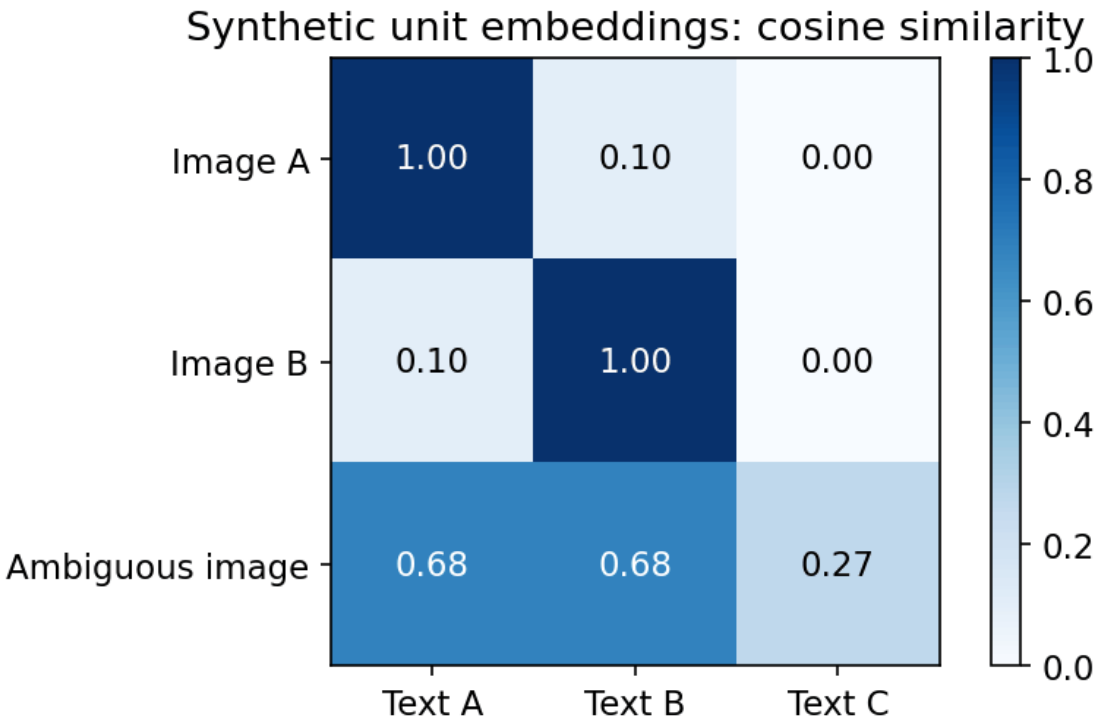


圖 3 — 餘弦相似度可以為候選排序，但它不是概念存在與否的校準機率。

SigLIP：用 sigmoid 取代 softmax

SigLIP [4] 把批次正規化的 softmax 換成對每一組影像文字配對各自做二元判斷。令配對正確時 $y_{ij} = +1$、否則 -1，並有可學習的尺度 a 與偏差 b，

$$\mathcal{L}_{\text{SigLIP}} = -\frac{1}{B}\sum_{i=1}^{B}\sum_{j=1}^{B}\log \sigma(y_{ij}(a\, v_i^{\top} t_j + b)),$$

其中 $\sigma(u) = 1/(1+e^{-u})$，因此每一項等於 $\log(1+\exp(-y_{ij}(a\, v_i^{\top} t_j + b)))$。因為沒有任何一項需要對整個批次求和，這個損失不需要跨裝置的全域正規化，讓大批次訓練更簡單。由於正樣本只有 B 個、負樣本有 $B^2 - B$ 個，偏差 b 的初始值會讓初始預測強烈偏向負類。

共享嵌入支援檢索與開放詞彙辨識，但細粒度計數、否定（「一隻**沒有**項圈的狗」）、空間關係與罕見的領域詞彙都是已知弱點。這些需求要直接評估，不要從亮眼的整體分類分數推論。

提示式分割：Segment Anything 系列

白話版。提示告訴模型你想要哪個區域或概念，輸出則是遮罩。

SAM [5] 結合一個較重的影像編碼器（每張影像跑一次）、輕量的提示編碼器與輕量的遮罩解碼器，因此同一張影像上的許多提示都能很快回應。空間提示——點、框或粗略遮罩——不需要固定的類別詞彙就能選取物體。因為單次點擊有歧義（點在襯衫上可能指襯衫，也可能指整個人），SAM 會預測幾個候選遮罩並附上信心分數。它在 SA-1B 上訓練：超過 10 億個遮罩、1100 萬張具授權且尊重隱私的影像，透過「模型在迴圈中」的資料引擎蒐集 [5]。

SAM 2 [6] 把提示式分割延伸到影片，加入串流記憶：模型儲存過去影格與被提示物體的特徵，並沿時間傳遞遮罩；在之後影格上的修正點擊會更新記憶。

SAM 3 [7] 於 2025 年發布，報告在 2026 年 3 月修訂，提出**提示式概念分割**（promptable concept segmentation）：給定一個短名詞片語（例如「yellow school bus」）、影像範例或兩者，它會回傳**所有**符合實例的遮罩與身分。它的影像層級偵測器與基於記憶的影片追蹤器共用一個骨幹，並以 **presence head** 把「這個概念是否出現」的辨識與「實例在哪裡」的定位分開 [7]。

這些介面提供的資訊不同：空間點說明在哪裡，名詞片語說明是什麼，範例說明長什麼樣子。請報告建立提示的成本，以及提示來自人、真值標註或自動偵測器。用真值框當提示得到的「零樣本分割」分數，其實隱藏了大量的標註資訊。

多模態大型語言模型

白話版。把視覺特徵轉成語言模型讀得懂的「字」，再教它遵循視覺指令。

常見的設計是

$$\text{image/video} \xrightarrow{\text{vision encoder}} h \xrightarrow{\text{connector}} z \xrightarrow{\text{language decoder with text}} \text{answer.}$$

這裡 h 是視覺特徵網格（對 ViT 而言，每個 patch 一個向量），z 是位於語言模型 token 嵌入空間中的向量序列，解碼器把 z 當成一般文字的嵌入來處理。

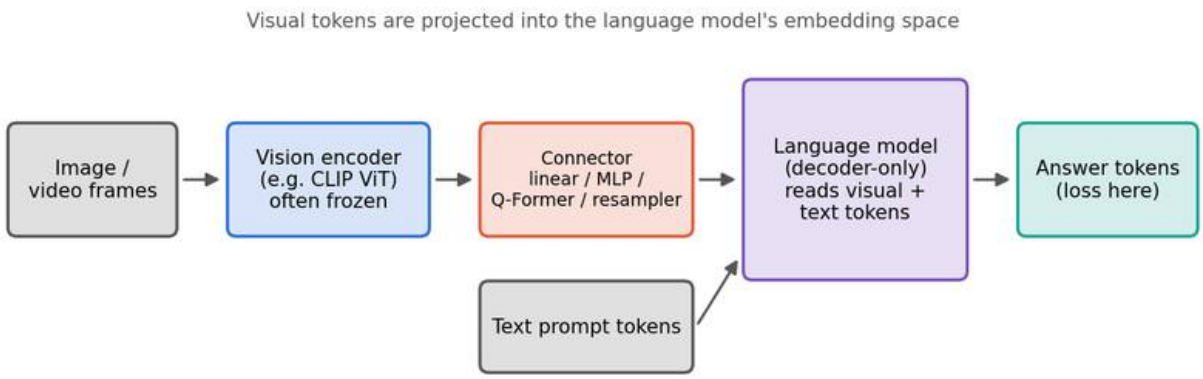


圖 4 — 典型的多模態 *LLM* 流程。視覺特徵被轉換成語言模型嵌入空間中的 *token*。

連接器

- **線性或 MLP 投影。** LLaVA [8] 用單一投影把 CLIP 特徵映射到語言模型的嵌入空間；LLaVA-1.5 [28] 報告簡單的 MLP 連接器（搭配 336 像素解析度的 CLIP ViT-L 與學術 VQA 資料）出奇地強大且節省資料。每個 patch 變成一個視覺 token。
- **可學習查詢。** BLIP-2 [26] 在凍結的影像編碼器與凍結的語言模型之間插入輕量的 Querying Transformer（Q-Former）。一組固定的可學習查詢向量對影像特徵做交叉注意力，因此語言模型永遠只看到少量、固定數目的視覺 token。BLIP-2 分兩階段訓練 Q-Former：先對凍結的影像編碼器做表示學習，再對凍結的語言模型做生成學習。
- **重取樣與交叉注意力。** Flamingo [27] 橋接凍結的視覺與語言模型，並能處理任意交錯的影像文字序列，使模型能從直接寫在提示中的範例做少樣本學習。

取捨在於 token 數與細節：每個 patch 一個 token 保留細節，但序列很長（多張影像或影片時很昂貴）；少數可學習查詢很便宜，但可能遺失小字。

視覺指令微調

LLaVA [8] 用只處理語言的 GPT-4 [45]，根據影像的圖說與框生成多模態指令遵循資料，再分兩階段訓練：先對齊連接器（特徵對齊），再在指令資料上微調連接器與語言模型。損失是一般的下一個 token 交叉熵，只算在**答案** token 上：

$$\mathcal{L} = -\sum_{i \in \mathcal{A}} \log p_\theta(w_i \mid z,\, w_{<i}),$$

其中 w_i 是文字 token，$\mathcal{A}$ 是屬於助理回答的位置集合，z 是視覺 token。指令與提示 token 作為上下文可見，但不計入損失。

Qwen3-VL 的 2025 年報告 [9] 呈現目前的方向：一系列稠密與混合專家（mixture-of-experts）模型，原生支援最長 256K token 的影像、影片與文字交錯上下文。它是代表性報告，不代表每個任務都會因更長的上下文或更多生成的推理而受益。

示意圖沒畫出的事

閱讀文件可能需要以切塊（tiling）或動態解析度保留小字；影片問題需要時間戳與時間順序。這些細節會改變 token 數與前處理；通用的「視覺編碼器 + LLM」示意圖只是起點。流暢的回答也可能提到影像中根本不存在的物體（**物體幻覺**），或數錯數量。請區分物體定位、OCR、空間推理、事實問答與風格上的指令遵循——它們需要不同的證據與指標（見〈風險與評估〉）。整個設計空間的綜述可參考 Yin 等人 [40] 與 Caffagni 等人 [2]。

適配基礎模型

白話版。用能解決目標任務的最便宜適配方式，並在看起來像實際部署的資料上檢查。

策略	可訓練元件	適合的起點
提示／上下文範例	無	模型介面本來就符合任務
凍結特徵 + 線性分類頭（linear probe）	小型監督式讀出層	標籤少、特徵遷移良好
Prompt tuning ／ visual prompt tuning	少數可學習的輸入 token	許多任務共用同一個凍結骨幹
Adapter ／ LoRA	小型插入模組或低秩模組	記憶體有限下的領域適配
部分微調	選定的區塊加分類頭	想做一些適配，又不想負擔完整最佳化器成本
完整微調（可搭配 LP-FT）	所有模型參數	資料與計算足夠；單靠遷移不夠

Linear probe

凍結編碼器 f，先算好特徵，再擬合線性分類器 $\hat{y} = \text{softmax}(Wf(x) + b)$。它便宜、不容易過擬合，也是衡量表示品質的標準**量尺**：如果線性讀出就能成功，代表資訊在特徵中是線性可取得的。

程式：為什麼預訓練特徵在標籤少時最有幫助

這個玩具實驗模擬整個故事。觀測值是 300 維，混合了 8 個**語意**因子（決定類別）、8 個很大的**干擾**因子（想像成光線與姿態）以及雜訊。「預訓練」看到 50 個上游類別，標籤充足，並學到一個 8 維投影（線性判別分析，代替深度編碼器）。下游有四個**新**類別，每類只有少數標籤。

```
import numpy as np
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression

rng = np.random.default_rng(0)
D, k_sem, k_nui = 300, 8, 8
sem_map = rng.normal(size=(k_sem, D))   # semantic factors -> pixels
nui_map = rng.normal(size=(k_nui, D))   # nuisance factors (light, pose...)

def sample(centers, n):
    y = rng.integers(len(centers), size=n)
    z = centers[y] + 0.7 * rng.normal(size=(n, k_sem))
    u = 4.0 * rng.normal(size=(n, k_nui))          # large nuisance variation
    x = z @ sem_map + u @ nui_map + rng.normal(size=(n, D))
    return x, y

# 1) "Pre-training": many upstream classes with plenty of labels.
up_centers = rng.normal(size=(50, k_sem))
x_up, y_up = sample(up_centers, 10_000)
encoder = LinearDiscriminantAnalysis(n_components=k_sem).fit(x_up, y_up)

# 2) Downstream: four NEW classes, few labels, linear probe only.
new_centers = rng.normal(size=(4, k_sem))
x_te, y_te = sample(new_centers, 2_000)
f_te = encoder.transform(x_te)          # frozen features, computed once
```

```
for shots in [1, 5, 20, 100]:
    raw, feat = [], []
    for repeat in range(10):
        x_tr, y_tr = sample(new_centers, 4 * shots)
        raw.append(LogisticRegression(max_iter=3000)
                   .fit(x_tr, y_tr).score(x_te, y_te))
        feat.append(LogisticRegression(max_iter=3000)
                    .fit(encoder.transform(x_tr), y_tr).score(f_te, y_te))
    print(f"{shots:>3} labels/class | raw pixels {np.mean(raw):.3f}"
          f" | frozen features {np.mean(feat):.3f}")
```

在我們的機器上，它印出的準確率大約是：每類 1 個標籤時 0.29 對 0.71、5 個時 0.62 對 0.95、20 個時 0.96 對 0.99、100 個時 0.98 對 0.99（原始輸入對凍結特徵；隨機猜測為 0.25）。預訓練投影已經丟掉干擾方向，所以少數標籤就夠了；標籤夠多時，原始輸入上的 probe 也會自己學會忽略干擾，差距隨之消失。上游類別與下游類別不同——特徵之所以能遷移，是因為兩者共享相同的語意因子。如果下游任務依賴的正是預訓練丟掉的因子（這裡是干擾因子），那麼不論加多少標籤，凍結特徵都會失敗。這就是實務上要測試的失敗模式。

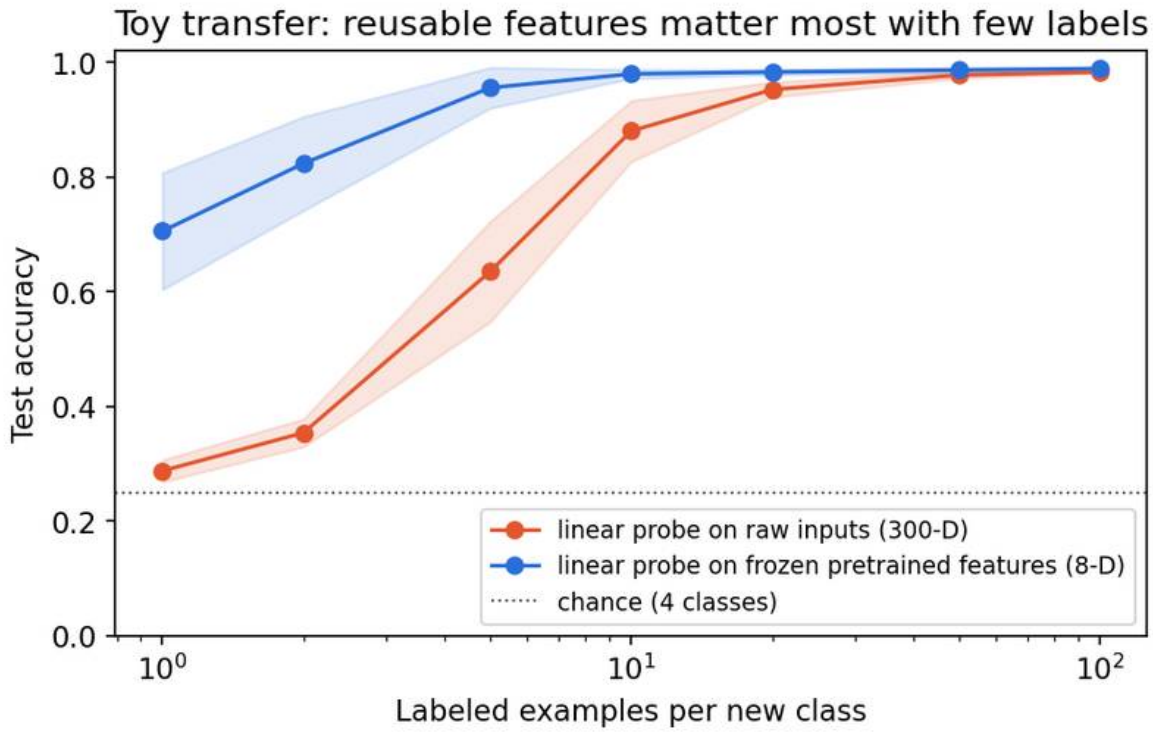


圖 *5* — 在更多標籤預算下的玩具遷移實驗（*10* 次抽樣的平均 ± 一個標準差）。標籤稀少時，可重用特徵最重要。

微調、LP-FT 與穩健微調

完整微調會更新所有權重，資料足夠時通常有最好的分布內準確率。但 Kumar 等人 [32] 指出它可能扭曲好的預訓練特徵：在他們的基準上，微調平均而言分布內準確率較高，分布外準確率卻比 linear probing 低。他們的解釋是，隨機初始化的分類頭在訓練初期把又大又雜的梯度送進骨幹。**LP-FT**——先擬合 linear probe，再從這個分類頭開始微調全部參數——減少了這種扭曲，並在他們的實驗中勝過兩種單獨作法 [32]。**WiSE-FT** [33] 走另一條路：在權重空間線性內插零樣本模型與微調模型，$\theta_\gamma = (1-\gamma)\theta_{\text{zero-shot}} + \gamma\,\theta_{\text{ft}}$，對 CLIP 類模型在分布偏移下提升了穩健性，同時保持強的目標任務準確率。兩者都很便宜，在斷言「微調傷害了穩健性」之前值得先試。

參數高效微調（PEFT）

Adapter [29] 在每個 Transformer 區塊內插入小型瓶頸 MLP，只訓練它們，每個任務只增加少量參數，原網路保持不變。**Prompt tuning** 學習幾個額外的輸入 token；**Visual Prompt Tuning（VPT）** [30] 把這個作法用在 ViT 上，在凍結骨幹的 patch 序列前面加上可學習的 token。Xu 等人 [42] 綜覽 PEFT，提出加法式（additive）、部分（partial）、重參數化（reparameterized）、混合（hybrid）與統一（unified）微調的分類，並實驗比較參數與記憶體效率。

LoRA [10] 把凍結權重 $W_0 \in \mathbb{R}^{d_{\text{out}}\times d_{\text{in}}}$ 的更新重參數化為

$$W = W_0 + \frac{\alpha}{r} BA, \qquad A \in \mathbb{R}^{r\times d_{\text{in}}}, \quad B \in \mathbb{R}^{d_{\text{out}}\times r},$$

其中秩 $r \ll \min(d_{\text{in}}, d_{\text{out}})$，$\alpha$ 是縮放常數。凍結 W_0，只學 A, B。可訓練權重從 $d_{\text{out}}d_{\text{in}}$ 降到 $r(d_{\text{in}} + d_{\text{out}})$。把 B 初始化為 0，使初始更新為零；A 隨機初始化，學習才能開始。訓練後 BA 可以合併回 W_0，因此推論不增加成本。QLoRA [31] 以 4 位元精度儲存凍結的基礎權重，並穿過它們反向傳播到 LoRA adapter，大幅減少記憶體，讓非常大的模型也能在單張 GPU 上微調。

程式：凍結基礎權重上的低秩更新

這個獨立範例在合成輸入上訓練一步 adapter。它展示參數化方式與初始行為，並沒有重現完整的預訓練模型微調流程。

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
class LoRALinear(nn.Module):
    def __init__(self, din, dout, rank=4, alpha=4):
        super().__init__()
        self.base = nn.Linear(din, dout, bias=False)
        self.base.weight.requires_grad_(False)
        self.A = nn.Parameter(torch.randn(rank, din) * .02)
        self.B = nn.Parameter(torch.zeros(dout, rank))
        self.scale = alpha/rank
    def forward(self, x):
```

```
        return self.base(x) + self.scale * F.linear(F.linear(x, self.A), self.B)

layer = LoRALinear(32, 16)
x = torch.randn(8, 32)
assert torch.allclose(layer(x), layer.base(x))
opt = torch.optim.Adam([layer.A, layer.B], lr=.01)
loss = F.mse_loss(layer(x), torch.randn(8, 16))
opt.zero_grad(); loss.backward(); opt.step()
assert layer.base.weight.grad is None
print("trainable:", sum(p.numel() for p in layer.parameters() if p.requires_grad))
```

它印出 `trainable: 192`，也就是 $r(d_{\text{in}} + d_{\text{out}}) = 4 \times (32 + 16)$，完整矩陣則是 512。第一個 assertion 檢查初始化為零的 B 在一開始不改變模型；第二個檢查凍結的基礎權重沒有收到梯度。

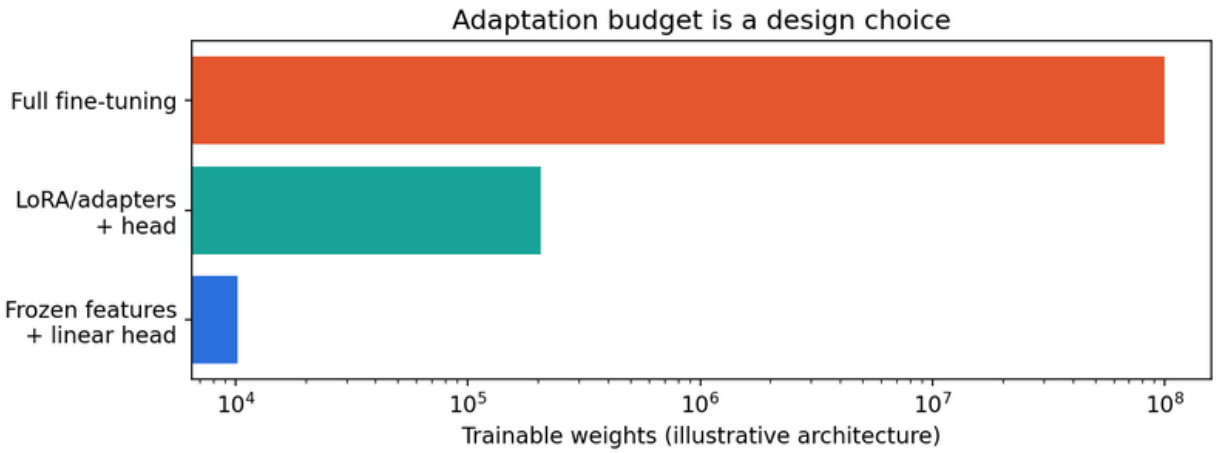


圖 *6* — 示意架構（特徵維度 *1024*、*10* 類、*12* 個矩陣上 *rank 8*、總共 *1* 億參數）的可訓練權重預算解析計算。活化記憶體、凍結權重儲存與最佳化器狀態必須另外計算。

LoRA 是一種參數化方式，不保證不會過擬合或遺忘。Adapter 仍需要領域驗證。凍結的 BatchNorm 統計量、影像正規化、輸出標籤對應，以及各 checkpoint 特有的前處理，也都需要刻意處理。

提示與上下文學習

最便宜的適配不改任何權重：為 CLIP 寫更好的文字提示、給 SAM 一個框，或像 Flamingo 那樣把幾個已解好的範例放進多模態模型的上下文 [27]。Zhang 等人的視覺語言綜述 [41] 比較了提示、特徵適配與微調在各種視覺任務上的表現。提示迭代很快，但很脆弱：措辭稍微改變就可能影響結果，因此要固定並報告提示流程。

影片、幾何與世界模型

白話版。除了辨認單一影格，模型還可以預測隨時間的變化、推論三維結構，或支援動作規劃。

V-JEPA 2 [11] 研究在特徵空間（而非像素空間）做預測的自監督影片模型，並以條件於動作的延伸版本進行規劃。VGGT [12] 以一次前饋，從一個或多個視角預測相機參數、深度、點雲圖（point maps）與軌跡。它們代表超越分類的兩條路：時間狀態建模與幾何推論。

看起來逼真的生成影片不一定遵守物理限制；場景重建即使不產生寫實像素也可能很有用。請依所宣稱的能力，量測多視角一致性、幾何、條件於動作的結果與時間行為。DLCV 07・世界模型會更詳細地討論部分可觀測性、學習動態與規劃。

風險與評估

白話版。用網路上一大片資料訓練出的模型，會繼承那片資料的偏誤與缺口，可能自信地描述不存在的東西，也可能早就看過你的「測試」集。評估必須針對這些事實來設計。

資料來源、偏誤與隱私

網路規模的影像文字資料集大到無法人工檢查。LAION-5B [34] 公開了這樣的資料集（58.5 億組經 CLIP 過濾的配對），因而促成 OpenCLIP 這類開放重現 [20]；Birhane 等人 [35] 對其前身 LAION-400M 的稽核，則記錄了影像文字配對中露骨與刻板印象的內容。下游系統會繼承這些關聯。授權與同意也很重要：訓練資料、模型權重與生成輸出可能各有不同條款；個人資料（人臉、姓名、醫療影像）帶來的隱私義務，不會因為資料是公開的就消失。SAM 的 SA-1B 是一個在說明中強調具授權且尊重隱私影像的資料集例子 [5]；部署每個模型時都應記錄這類性質。

幻覺

多模態 LLM 可能提到影像中不存在的物體。POPE [37] 以輪詢方式衡量物體幻覺：針對特定物體是否存在提出是非題（包括常見物體與經常共同出現的干擾物體），使量測比評分自由生成的描述更不受模型生成風格影響。POPE 錯誤率低並不涵蓋計數、屬性或關係；這些要另外測試。

污染與「零樣本」

如果基準影像（或近似重複的影像）在預訓練資料中，「零樣本」分數有一部分其實是在測記憶。Mayilvahanan 等人 [36] 探討 CLIP 的泛化是否主要來自訓練與測試的高相似度：他們把 LAION 中與常見分布外基準太相似的影像剪除後重新訓練，部分基準的表現下降，但整體仍然很強。教訓是雙向的：相似度解釋了一部分報告的表現，但不是全部——所以要實際量測，不要預設任何一方。實務檢查包括：把測試集與可取得的訓練資料做去重複、在模型資料截止日期之後蒐集的資料上報告結果，並完整說明零樣本流程（類別名稱、模板、候選集合）。

基準

MMMU [38] 這類綜合性多模態基準——從考試、測驗與教科書蒐集 11.5K 個大學程度問題，橫跨六大學科，包含圖表、示意圖、地圖、表格等多種影像——測試知識與推理，但單一數字混合了感知、OCR、領域知識與答案格式。部署時，請建立一個小型、針對任務的評估集並納入已知

的失敗案例，報告不同隨機種子或提示措辭下的變異，並與簡單基準比較（凍結特徵上的 linear probe 常常強得令人意外）。ML 10・評估與部署 會深入討論校準、資料切分與部署。

現代觀點

值得閱讀的綜述。

- **Bommasani 等人 [1]**——為基礎模型命名的報告。內容涵蓋能力（語言、視覺、機器人、推理）、應用（法律、醫療、教育）、技術（建模、訓練、適配、評估、資料）與社會影響。它的核心觀點——湧現、同質化與被繼承的失敗——至今仍是這個領域的框架。
- **Awais 等人 [39]**——電腦視覺綜述，依架構設計、訓練目標（對比式與生成式）、資料集、微調機制與提示方式整理基礎模型，並列出評估、實際應用、對抗穩健性與可解釋性等開放挑戰。
- **Zhang 等人 [41]**——TPAMI 上關於視覺任務的視覺語言模型綜述：網路架構、預訓練目標、下游任務、資料集、遷移學習與知識蒸餾，並對所回顧的方法做基準比較。
- **Yin 等人 [40]** 與 **Caffagni 等人 [2]**——多模態大型語言模型綜述：架構（編碼器、連接器、LLM）、訓練策略與資料（預訓練、指令微調、對齊）、評估與幻覺。
- **Han 等人 [13]**——把地圖擴展到多模態生成模型的綜述，依生成類別與技術組織。
- **Xu 等人 [42]**——參數高效微調的批判性回顧，把方法分成加法式、部分、重參數化、混合與統一五個家族，並實驗比較代表性方法。

2023–2026 年的現況。有四個趨勢特別明顯。(1) **更好的凍結特徵**：DINOv2 [15] 與 DINOv3 [16] 讓自監督編碼器強到許多密集任務直接從凍結的 patch 特徵開始。(2) **概念層級的提示**：分割從「點一個物體」（SAM [5]）走到影片記憶（SAM 2 [6]），再到「分割這個片語的每一個實例」（SAM 3 [7]）。(3) **長且交錯的多模態上下文**：Qwen3-VL [9] 等模型可同時讀取多張影像、影片與文字。(4) **超越辨識**：用於規劃的影片預測（V-JEPA 2 [11]）與前饋式幾何（VGGT [12]）把基礎模型當成世界與場景的估計器。

開放問題。

- 有依據的可靠性：減少幻覺，並讓模型在適當時候說出「看不到」[37]。
- 在污染與基準飽和下做誠實的評估 [36]。
- 資料：把整理資料當成一個縮放軸 [15], [20]，以及來源、授權與同意 [34], [35]。
- 有效率的適配與推論：高解析度影像與影片的 token 預算；能保留穩健性的 PEFT [32], [33]。
- 讓密集、三維與時間理解達到影像層級模型的語意水準。

面對任何新的「基礎」模型，請記錄它的輸入／輸出介面、訓練目標、資料來源、支援的適配方式、checkpoint 授權、前處理與任務層級的評估。這能避免把模型大小或開放式聊天介面當成具備通用能力的證據。

重點整理

- 「基礎」描述的是廣泛預訓練加上重用，而不是特定架構；這類模型不一定要生成文字。
- 縮放定律描述帶有下限、近似冪律的平滑損失改善；計算最佳的訓練會平衡模型大小與資料量；常數取決於資料與配方。

- 預訓練目標決定特徵擅長什麼：對比式適合以文字定義的辨識，遮罩建模適合微調，自蒸餾適合凍結的密集特徵，生成式提供有彈性的介面。
- CLIP 式對齊支援檢索與以文字定義的分類；餘弦相似度不是校準過的信心。
- 空間、概念與語言提示提供不同種類的資訊；請報告提示由誰產生。
- 多模態 LLM = 視覺編碼器 + 連接器 + 語言模型，以只算答案的下一個 token 損失訓練；連接器決定細節與 token 數的取捨。
- 適配從便宜的開始（提示、linear probe），再到 PEFT（LoRA、adapter），最後才是完整微調；LP-FT 與權重內插有助於保留穩健性。
- 明確評估偏誤、幻覺與污染；單一綜合基準分數不足以證明通用能力。

練習

1. **候選集合的影響**。在合成 CLIP 範例中加入第四個描述。影像不變時，softmax 的「信心」會不會改變？

   提示

   會：softmax 的分母改變了。影像與某個固定描述的相似度不變，但正規化後的機率取決於候選集合。試著加入一個與模糊影像嵌入很接近的文字向量。

2. **計算最佳配置**。利用參數化定律 $L(N, D) = E + A/N^{\alpha} + B/D^{\beta}$ 與 $C = 6ND$，證明最佳模型大小的成長為 $N^{\star} \propto C^{\beta/(\alpha+\beta)}$。$\alpha = \beta$ 代表什麼？

   提示

   代入 $D = C/(6N)$，對 N 微分並令其為零：$\alpha A N^{-\alpha-1} = \beta B (6N/C)^{\beta}/N$。解得 $N^{\alpha+\beta} \propto C^{\beta}$。當 $\alpha = \beta$ 時，$N^{\star} \propto C^{1/2}$，因此 $D^{\star} \propto C^{1/2}$：參數與 token 等比例擴大，與 Hoffmann 等人的結論一致。

3. **讓遷移失效**。在 linear probe 實驗中，讓下游類別取決於干擾因子而非語意因子。隨著標籤增加，凍結特徵上的 probe 會怎樣？這對選擇基礎模型有什麼啟示？

   提示

   改為對 u 抽類別中心，而不是 z。凍結的 8 維特徵被訓練成丟掉干擾方向，所以不論加多少標籤都接近隨機猜測，而原始輸入上的 probe 會進步。骨幹只有在預訓練保留了你的任務所需資訊時才有用——投入適配之前，先用 linear probe 檢查。

4. **LoRA 預算**。一個 ViT-B 區塊有四個 768×768 的注意力投影，以及兩個大小分別為 3072×768 與 768×3072 的 MLP 矩陣。計算這六個矩陣完整微調與全部使用 rank-8 LoRA 時的可訓練權重數。

   提示

   完整：$4 \cdot 768^2 + 2 \cdot 3072 \cdot 768 = 2{,}359{,}296 + 4{,}718{,}592 = 7{,}077{,}888$。LoRA：$8 \cdot [4 \cdot (768 + 768) + 2 \cdot (3072 + 768)] = 8 \cdot (6144 + 7680) = 110{,}592$，約為完整數量的 1.6%。

5. **提示的公平性**。設計一個能把遮罩品質與提示品質分開的 SAM 評估。

提示

固定提示流程並計算人工互動次數。把自動提議與使用理想真值提示得到的遮罩分開評估，兩者都要報告。對 SAM 3，在同一批影像上比較名詞片語提示與範例提示。

6. **污染稽核**。有一個新的影像文字模型宣稱在某個公開基準上有很高的零樣本準確率。在相信這個數字之前，列出你會做的三項檢查。

提示

(a) 以嵌入相似度等方式，在可取得的訓練資料中搜尋測試影像的近似重複；(b) 在資料截止日期之後新蒐集的資料上評估；(c) 檢查對提示模板與候選類別清單的敏感度；也可以與 linear probe 及簡單基準比較。

參考文獻

1. R. Bommasani et al., "On the Opportunities and Risks of Foundation Models," Stanford CRFM report, arXiv:2108.07258, 2021; revised 2022. arXiv
2. D. Caffagni et al., "The Revolution of Multimodal Large Language Models: A Survey," *Findings of ACL*, 2024. arXiv
3. A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision," *ICML*, 2021. arXiv
4. X. Zhai, B. Mustafa, A. Kolesnikov and L. Beyer, "Sigmoid Loss for Language Image Pre-Training," *ICCV*, 2023. arXiv
5. A. Kirillov et al., "Segment Anything," *ICCV*, 2023. arXiv
6. N. Ravi et al., "SAM 2: Segment Anything in Images and Videos," *ICLR*, 2025. arXiv
7. N. Carion et al., "SAM 3: Segment Anything with Concepts," arXiv:2511.16719, 2025; revised March 2026. arXiv
8. H. Liu, C. Li, Q. Wu and Y. J. Lee, "Visual Instruction Tuning," *NeurIPS*, 2023. arXiv
9. S. Bai et al., "Qwen3-VL Technical Report," arXiv:2511.21631, 2025. arXiv
10. E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," *ICLR*, 2022. arXiv
11. M. Assran et al., "V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning," arXiv:2506.09985, 2025. arXiv
12. J. Wang et al., "VGGT: Visual Geometry Grounded Transformer," *CVPR*, 2025. arXiv
13. L. Han et al., "A Survey of Generative Categories and Techniques in Multimodal Generative Models," arXiv:2506.10016, 2025. arXiv
14. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. Book
15. M. Oquab et al., "DINOv2: Learning Robust Visual Features without Supervision," *Transactions on Machine Learning Research*, 2024. arXiv
16. O. Siméoni et al., "DINOv3," arXiv:2508.10104, 2025. arXiv
17. J. Kaplan et al., "Scaling Laws for Neural Language Models," arXiv:2001.08361, 2020. arXiv

18. J. Hoffmann et al., "Training Compute-Optimal Large Language Models," arXiv:2203.15556, 2022; published as "An empirical analysis of compute-optimal large language model training," *NeurIPS*, 2022. arXiv
19. X. Zhai, A. Kolesnikov, N. Houlsby and L. Beyer, "Scaling Vision Transformers," *CVPR*, 2022. arXiv
20. M. Cherti et al., "Reproducible Scaling Laws for Contrastive Language-Image Learning," *CVPR*, 2023. arXiv
21. A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," *ICLR*, 2021. arXiv
22. K. He, X. Chen, S. Xie, Y. Li, P. Dollár and R. Girshick, "Masked Autoencoders Are Scalable Vision Learners," *CVPR*, 2022. arXiv
23. H. Bao, L. Dong, S. Piao and F. Wei, "BEiT: BERT Pre-Training of Image Transformers," *ICLR*, 2022. arXiv
24. M. Caron et al., "Emerging Properties in Self-Supervised Vision Transformers," *ICCV*, 2021. arXiv
25. C. Jia et al., "Scaling Up Visual and Vision-Language Representation Learning With Noisy Text Supervision," *ICML*, 2021. arXiv
26. J. Li, D. Li, S. Savarese and S. Hoi, "BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models," *ICML*, 2023. PMLR
27. J.-B. Alayrac et al., "Flamingo: a Visual Language Model for Few-Shot Learning," *NeurIPS*, 2022. arXiv
28. H. Liu, C. Li, Y. Li and Y. J. Lee, "Improved Baselines with Visual Instruction Tuning," *CVPR*, 2024. arXiv
29. N. Houlsby et al., "Parameter-Efficient Transfer Learning for NLP," *ICML*, 2019. PMLR
30. M. Jia et al., "Visual Prompt Tuning," *ECCV*, 2022. arXiv
31. T. Dettmers, A. Pagnoni, A. Holtzman and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," *NeurIPS*, 2023. arXiv
32. A. Kumar, A. Raghunathan, R. Jones, T. Ma and P. Liang, "Fine-Tuning can Distort Pretrained Features and Underperform Out-of-Distribution," *ICLR*, 2022. arXiv
33. M. Wortsman et al., "Robust Fine-Tuning of Zero-Shot Models," *CVPR*, 2022. arXiv
34. C. Schuhmann et al., "LAION-5B: An Open Large-Scale Dataset for Training Next Generation Image-Text Models," *NeurIPS Datasets and Benchmarks Track*, 2022. arXiv
35. A. Birhane, V. U. Prabhu and E. Kahembwe, "Multimodal Datasets: Misogyny, Pornography, and Malignant Stereotypes," arXiv:2110.01963, 2021. arXiv
36. P. Mayilvahanan, T. Wiedemer, E. Rusak, M. Bethge and W. Brendel, "Does CLIP's Generalization Performance Mainly Stem from High Train-Test Similarity?," *ICLR*, 2024. arXiv
37. Y. Li et al., "Evaluating Object Hallucination in Large Vision-Language Models," *EMNLP*, 2023. arXiv
38. X. Yue et al., "MMMU: A Massive Multi-discipline Multimodal Understanding and Reasoning Benchmark for Expert AGI," *CVPR*, 2024. arXiv
39. M. Awais et al., "Foundational Models Defining a New Era in Vision: A Survey and Outlook," arXiv:2307.13721, 2023. arXiv
40. S. Yin et al., "A Survey on Multimodal Large Language Models," *National Science Review*, 2024. arXiv

41. J. Zhang, J. Huang, S. Jin and S. Lu, “Vision-Language Models for Vision Tasks: A Survey,” *IEEE TPAMI*, vol. 46, no. 8, 2024. arXiv
42. L. Xu et al., “Parameter-Efficient Fine-Tuning Methods for Pretrained Language Models: A Critical Review and Assessment,” arXiv:2312.12148, 2023. arXiv
43. J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, “ImageNet: A Large-Scale Hierarchical Image Database,” *CVPR*, 2009, pp. 248–255. doi
44. J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *NAACL-HLT*, 2019, pp. 4171–4186. doi
45. OpenAI, “GPT-4 Technical Report,” arXiv:2303.08774, 2023. arXiv

ML 10・評估、效率與部署

不洩漏地切分資料、做交叉驗證、為分數加上信賴區間、檢查校準與分布偏移下的穩健性，誠實量測 FLOPs、記憶體與延遲，再把模型匯出、壓縮、監控並做到可重現。

網站版本: https://blog.shyandram.dev/zh-tw/learn/ml/ml-10-evaluation-deployment/

你將學到

- 如何建立基準模型（baseline）與資料切分，讓結果真正支持你想提出的主張，以及資料洩漏（leakage）如何悄悄破壞這一切。
- *K* 折、群組、依時間順序與巢狀交叉驗證（cross-validation）的運作方式，以及各自適用的時機。
- 如何為測試分數加上不確定性：標準誤、成對 bootstrap 與 McNemar 檢定。
- 校準（calibration）與準確率有何不同、如何量測（Brier 分數、NLL、ECE），以及溫度縮放能修正其中哪一部分。
- 分布偏移（distribution shift）是什麼（共變量、標籤、概念偏移）、穩健性基準如何探測它，以及部署後如何監控。
- 如何正確計算 FLOPs、記憶體與延遲，以及匯出、量化、剪枝、蒸餾和 ONNX Runtime、TensorRT 等推論引擎實際帶來什麼。
- 如何讓實驗可重現並留下文件。

先看全貌

訓練模型只是專案的中段，不是終點。在任何人依賴它之前，你必須回答四個問題：它在真正會遇到的案例上有效嗎？這個數字有多可靠？它知道自己什麼時候可能出錯嗎？它符合要執行的裝置、延遲與預算嗎？上線之後還有第五個問題：它現在還有效嗎？

> **用最簡單的話說**
>
> 一個背下去年考卷的學生，考去年的考卷當然滿分，但這什麼也說明不了。公平的考試要用新題目（乾淨的資料切分）。一次考試可能運氣好，所以你想知道換一天考分數會晃動多少（信賴區間）。好學生遇到可能答錯的題目也會說「我不太確定」（校準）。如果正式考場很吵，你也要在吵雜環境下測試（穩健性）。如果是限時考試，答對但太慢也不行（延遲）。學生開始工作後，還是要有人不時抽查他的成果（監控）。

這一章是 ML 系列的最後一章。它整理 Goodfellow 等人 [1, ch. 11] 的實務方法論與 James 等人 [2] 的統計學習觀點，再補上把訓練好的網路變成可靠服務所需的工程步驟。視覺任務專屬的指標（top-1、mAP、PSNR、FID 等）在DLCV 02 與DLCV 03 中說明；本章聚焦所有任務共通的部分。

先建立有意義的基準模型

白話版。在投入更多算力之前，先看看簡單方法在完全相同的切分上表現如何。基準模型告訴你複雜模型到底多換到了什麼。

影像任務常見的好基準包括：在原始像素或手工特徵上做邏輯迴歸；最近鄰或 SVM 分類器；以及在預訓練模型凍結特徵上接一個小型線性分類頭（見ML 09）。邏輯迴歸學習線性邊界；支援向量機（support vector machine, SVM）最大化帶正則化的間隔；最近鄰直接比較樣本；樹集成則切分特徵空間。傳統圖形識別章節IP 13 有更完整的推導。

作為參考，二元線性 SVM 最小化

$$\frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\max(0,\,1 - y_i(w^\top x_i + b)), \qquad y_i \in \{-1, +1\},$$

其中 w 與 b 是權重與偏差，x_i 是第 i 個特徵向量，$C > 0$ 在間隔寬度與訓練錯誤之間取捨。$\max(0,\,1 - y_i f(x_i))$ 稱為**鉸鏈損失（hinge loss）**：位於正確一側且間隔至少為 1 的點損失為零，其餘則線性增加。

程式碼：把前處理放進 pipeline 的基準模型

這個例子只使用內建資料。模型選擇用驗證集；選出的模型只在測試集上評估一次。標準化與 PCA 都在各自的訓練 pipeline 內擬合，絕不在整份資料上擬合。

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, balanced_accuracy_score

X, y = load_digits(return_X_y=True)
Xa, Xt, ya, yt = train_test_split(X, y, test_size=.2, stratify=y, random_state=0)
Xtr, Xv, ytr, yv = train_test_split(Xa, ya, test_size=.25, stratify=ya, random_state=1)
models = {      # listed from simplest to most complex
    "linear": make_pipeline(StandardScaler(),
                            LogisticRegression(max_iter=2000)),
    "PCA + RBF SVM": make_pipeline(StandardScaler(), PCA(n_components=30),
                                   SVC(C=10, gamma="scale")),
}
scores = {}
for name, model in models.items():
    model.fit(Xtr, ytr)
    scores[name] = accuracy_score(yv, model.predict(Xv))
best = max(scores, key=scores.get)   # tie -> first listed, i.e. the simpler model
pred = models[best].predict(Xt)
print("validation:", scores)
print("selected:", best)
```

```
print("test accuracy:", accuracy_score(yt, pred))
print("test balanced accuracy:", balanced_accuracy_score(yt, pred))
```

在這個切分上，兩個候選模型的驗證準確率剛好都是 0.9639（360 張中答對 347 張）。「平手時選較簡單的模型」這條規則是事先定好的，因此選出線性模型，它在測試集上的準確率為 0.9639、平衡準確率為 0.9636。這個小實驗藏著兩個教訓。第一，平手規則是評估協定的一部分，應該在看結果之前寫下來。第二，驗證集只有 360 張時，一張影像就值 0.28 個百分點，所以小實驗中許多看似「勝出」的差距其實只是雜訊；後面的信賴區間一節會把這點說得更精確。

資料切分與資料洩漏

白話版。測試集必須和部署時的資料「一樣新」。如果測試影像和訓練影像來自同一位病人、同一段影片、同一個場景或同一段時間，測試分數量到的是記憶，而不是泛化。

依主張決定切分單位

同一位病人、同一段影片、同一批產品或同一台相機的影像彼此相關。把相鄰影格隨機切分，會讓幾乎相同的畫面同時出現在訓練與測試中。先問部署時「新」代表什麼，再依該單位切分：模型會遇到新病人就依病人切，會遇到新影片就依影片切，要裝到新醫院就依院區切，要預測未來就依時間切。

圖 *1* — 依泛化主張選擇切分單位。以群組分開可以避免幾乎重複的情境跨越切分邊界。

分層抽樣（stratification）讓各切分的類別比例相近，但它不能取代群組切分。切分前先去除重複，並把完全或近似重複的樣本放在同一側。若部署母體會隨時間漂移，依時間切分（用過去訓練、用最近一段時間測試）才是誠實的做法。

資料洩漏的各種面貌

資料洩漏是指任何讓「預測當下不該取得的資訊」流進模型或模型選擇過程的途徑。Kapoor 與 Narayanan 發現資料洩漏錯誤影響了 17 個使用 ML 的科學領域、共 329 篇論文，並提出洩漏類型的分類，其中包括缺少訓練／測試分離、在整份資料上做前處理、重複樣本、時間洩漏與不正當特徵 [4]。scikit-learn 文件也列出最常見的程式錯誤 [3]。實用檢查表：

洩漏	例子	修正
在全部資料上前處理	在訓練＋測試上擬合 `StandardScaler`、PCA 或詞彙表	把前處理放進只在訓練折上擬合的 `Pipeline`
相關的單位	同一段影片的影格、同一次掃描的切片、同一張照片的裁切	依群組切分
重複樣本	同一張影像被爬取兩次	切分前先用雜湊或嵌入去重
時間	用下週的資料預測本週	依時間順序切分
目標代理變數	在結果**之後**才記錄的特徵（例如「是否已接受治療」）	稽核每個特徵的記錄時間
重複使用測試集	一邊看測試分數一邊調超參數	決策用驗證集，測試只做一次

最後一列最隱微。每次看了測試分數再修改，測試集就變得更像驗證集一點。公開基準在整個社群的規模上都面臨這個問題。

交叉驗證

白話版。不只做一次訓練／驗證切分，而是輪流：把資料分成 K 份，訓練 K 次，每次留出不同的一份來評估，再取平均。每個樣本都恰好被評估一次，也能看到分數的變動幅度。

K 折交叉驗證與其變形

把索引分成 K 個互斥的折 $F_1, \dots, F_K$。對每個 k，用除了 F_k 以外的所有折訓練出 $\hat{f}^{(-k)}$，並在 F_k 上計算誤差。交叉驗證估計值為

$$\widehat{\mathrm{Err}}_{\mathrm{CV}} = \frac{1}{K}\sum_{k=1}^{K} \frac{1}{|F_k|} \sum_{i \in F_k} \ell\big(y_i, \hat{f}^{(-k)}(x_i)\big),$$

其中 ℓ 是單一樣本的損失（例如 0–1 錯誤），$|F_k|$ 是該折的大小。常見選擇為 $K = 5$ 或 $K = 10$ [2]。各種變形只差在如何組成各折：

- **分層 K 折**讓各折的類別比例相同；類別不平衡時很有用。
- **群組 K 折**把每個群組（病人、影片、院區）只分到一個折，因此沒有群組同時出現在訓練與留出資料中。
- **時間序列切分**永遠用較早的資料訓練、用較晚的資料驗證。
- **留一群組**是每折只放一個群組的群組 K 折；群組很少時有用。

各折分數並不獨立（訓練集互相重疊），所以各折之間的標準差會低估真正的不確定性。把它當成穩定性的粗略指標，而不是正式的信賴區間。

程式碼：隨機 K 折對群組資料做了什麼

我們模擬 40 位病人、每人 25 張影像。同一位病人的影像共享一個「指紋」（病人特有的外觀），而標籤以病人為單位固定。真正的類別訊號很弱。

```
import numpy as np
from sklearn.model_selection import KFold, GroupKFold, cross_val_score
from sklearn.ensemble import RandomForestClassifier

rng = np.random.default_rng(0)
n_groups, per_group = 40, 25                    # e.g. 40 patients, 25 images each
groups = np.repeat(np.arange(n_groups), per_group)
y = np.repeat(rng.integers(0, 2, n_groups), per_group)   # label fixed per patient
identity = rng.normal(size=(n_groups, 20))[groups]       # patient "fingerprint"
signal = 0.3 * y[:, None] * np.ones((1, 20))             # weak real class signal
X = identity + signal + 0.3 * rng.normal(size=(len(y), 20))

model = RandomForestClassifier(n_estimators=200, random_state=0)
random_cv = cross_val_score(model, X, y, cv=KFold(5, shuffle=True, random_state=0))
group_cv = cross_val_score(model, X, y, cv=GroupKFold(5), groups=groups)
print(f"random K-fold accuracy: {random_cv.mean():.3f} +/- {random_cv.std():.3f}")
print(f"group  K-fold accuracy: {group_cv.mean():.3f} +/- {group_cv.std():.3f}")
```

輸出：

```
random K-fold accuracy: 0.999 +/- 0.002
group  K-fold accuracy: 0.657 +/- 0.158
```

隨機切分回報近乎完美的準確率，因為隨機森林認出了每位病人的指紋，直接抄該病人的標籤。群組 K 折符合「對新病人有效」這個主張，回報的是 0.657，而且各折差異很大。兩個數字在算術上都沒錯，但只有一個回答了真正的問題。

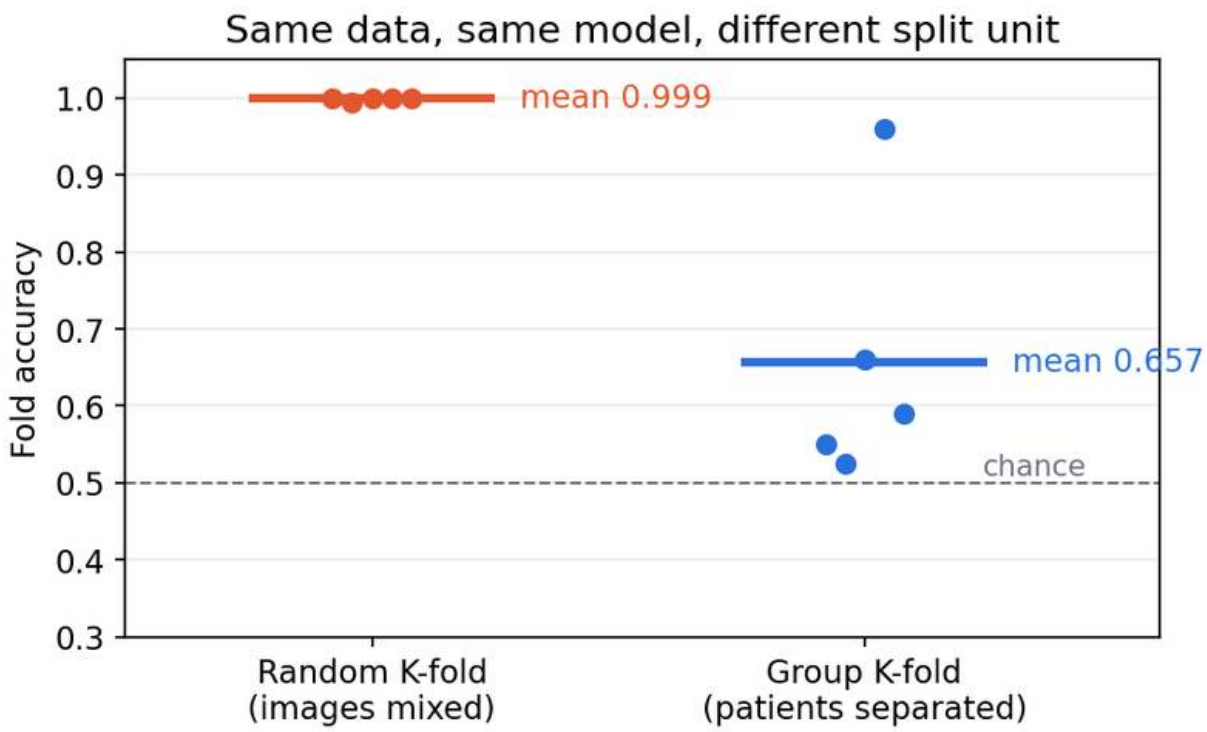


圖 2 — 上方程式碼的輸出，每個點代表一折。造成 *34* 個百分點差距的是切分單位，不是模型。

巢狀交叉驗證

如果同時用交叉驗證來**選**超參數又**回報**表現，回報的數字會偏樂觀：你挑的是剛好在那些折上看起來最好的設定。Varma 與 Simon 用兩類在統計上完全相同的「虛無」資料示範了這點：用交叉驗證調整 SVM 後，在 38% 的模擬資料集上 CV 誤差估計低於 30%，而調好的分類器在獨立測試資料上並不比亂猜好 [5]；Cawley 與 Talbot 則顯示，對模型選擇準則的過擬合可能和不同學習演算法之間的差距一樣大 [6]。

巢狀交叉驗證用兩層迴圈解決這個問題。外層留出一折做評估；在每個外層訓練集內，由內層交叉驗證選擇超參數。外層折只用來為內層選出的設定打分數。在 scikit-learn 中，這只是把 `GridSearchCV` 放進 `cross_val_score`。得到的結果估計的是整個**程序**（包含調參）的表現，也就是你重新訓練時實際會執行的東西。

對深度網路而言，完整的巢狀交叉驗證通常太昂貴。常見的折衷是固定一組訓練／驗證／測試切分，為每個被比較的方法訂明調參預算，並只打開測試集一次。

指標回顧

白話版。先決定一個錯誤的代價，再選數字。單一純量很少能說完整個故事。

任務	有用的量測	常見陷阱
分類	各類別精確率／召回率、macro-F1、平衡準確率、AUROC/AUPRC	準確率隱藏罕見失敗；PR 曲線受盛行率影響
偵測／分割	AP、IoU/Dice、邊界誤差	比較不同 IoU 門檻或評估協定的結果
檢索	Recall@k、在明確相關集合下的 mAP	圖庫中的重複樣本灌高匹配率
影像復原	PSNR/SSIM、感知指標、與觀測的一致性	看似合理的紋理不一定是真實細節
生成	FID/KID、涵蓋率、條件對齊、人工評估	特徵層面的真實感不代表忠於提示
影片／幾何	時間一致性、任務準確率、幾何誤差	單張影格好看可能掩蓋結構不一致

分類指標（混淆矩陣、top-k、ROC/PR 曲線）的定義與陷阱在DLCV 02 中詳細說明，偵測指標（IoU、AP、COCO mAP）則在DLCV 03。對生成模型而言，FID 比較特徵分布的高斯擬合（平均與共變異數）；KID 則使用無偏的核最大平均差異估計，因此在樣本數少時表現較穩定 [7]。兩者都不是人類偏好的通用量度。務必寫明特徵擷取器、參考集合、樣本數、解析度與取樣設定。

有兩個經典事實值得記住。正例只有 1% 時，永遠預測負例的分類器有 99% 準確率而召回率為零，所以對罕見事件而言，單看準確率毫無意義。此外，精確率取決於盛行率：同一個偵測器在事件更罕見的母體中精確率較低，但它的召回率與偽陽性率不變。

統計顯著性與信賴區間

白話版。測試分數是從有限樣本算出來的，換一批樣本就會有些不同。要回報一個範圍而不只是一個點，並在**相同**樣本上比較兩個模型。

準確率的標準誤

若 n 個測試樣本彼此獨立，模型真正的準確率為 p，則答對的個數服從二項分布，觀測準確率 $\hat{p}$ 的標準誤為

$$\mathrm{SE}(\hat{p}) = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}, \qquad 95\%\ \text{interval} \approx \hat{p} \pm 1.96\,\mathrm{SE}.$$

這裡 $\hat{p}$ 是答對比例，1.96 是標準常態分布的第 97.5 百分位數。當 $\hat{p} = 0.972$、$n = 540$ 時，$\mathrm{SE} \approx 0.0071$，區間大約是 ± 1.4 個百分點。當 $\hat{p}$ 非常接近 0 或 1、或 n 很小時，這個常態近似很差，應改用 Wilson 或精確二項區間。若測試樣本有群組結構（每位病人多張影像），有效樣本數比較接近群組數，此時必須以群組為單位重抽樣來計算區間。

Bootstrap

Bootstrap 以「從觀測資料中放回抽樣」來估計任意統計量的抽樣分布 [9]。抽出 B 組大小為 n 的重抽樣，在每組上重新計算統計量，並取第 2.5 與第 97.5 百分位數作為近似的 95% 區間（**百分位數區間**）。它適用於沒有簡單公式的指標，例如 macro-F1、AUROC 或 mAP。

比較兩個模型時，要用**成對** bootstrap：每次只抽一次樣本索引，並在同一組重抽樣上評估兩個模型。因為兩個模型通常在許多相同的困難樣本上失敗，配對能消除這部分共同變異，使差值的區間比兩個獨立區間緊得多。

McNemar 檢定

兩個分類器在同一個測試集上評估時，只有它們意見不同的樣本才帶有「誰比較好」的資訊。令 n_{01} 為模型 A 答對而 B 答錯的樣本數，n_{10} 為相反情況。在「兩個模型錯誤率相同」的虛無假設下，每個不一致樣本落在兩邊的機率相等，所以 $n_{10} \sim \text{Binomial}(n_{01}+n_{10}, \frac{1}{2})$。精確 McNemar 檢定就由此計算雙尾二項 p 值。Dietterich 比較了五種比較學習演算法的檢定，發現對只能執行一次的演算法（昂貴模型的典型情況），McNemar 檢定是唯一第一型錯誤可以接受的方法；若能負擔執行十次，他建議使用 5×2 交叉驗證 t 檢定 [8]。

程式碼：成對 bootstrap 與 McNemar

```
import numpy as np
from scipy.stats import binomtest
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC

X, y = load_digits(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
a = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)).fit(Xtr, ytr)
b = make_pipeline(StandardScaler(), SVC(C=10, gamma="scale")).fit(Xtr, ytr)
ca, cb = a.predict(Xte) == yte, b.predict(Xte) == yte    # per-example correctness

# 1) Percentile bootstrap: resample test examples with replacement.
rng = np.random.default_rng(0)
idx = rng.integers(0, len(yte), size=(10_000, len(yte)))
diff = cb[idx].mean(axis=1) - ca[idx].mean(axis=1)       # paired: same resample
lo, hi = np.percentile(diff, [2.5, 97.5])
print(f"accuracy A={ca.mean():.4f}  B={cb.mean():.4f}  n={len(yte)}")
print(f"B - A = {cb.mean() - ca.mean():+.4f}, 95% bootstrap CI [{lo:+.4f}, {hi:+.4f}]")

# 2) Exact McNemar test: only the discordant examples carry information.
n01 = int(np.sum(ca & ~cb))      # A right, B wrong
n10 = int(np.sum(~ca & cb))      # A wrong, B right
p = binomtest(n10, n01 + n10, 0.5).pvalue
print(f"discordant pairs: A-only={n01}, B-only={n10}, exact McNemar p={p:.4f}")
```

輸出：

```
accuracy A=0.9722  B=0.9852  n=540
```

```
B - A = +0.0130, 95% bootstrap CI [-0.0019, +0.0278]
discordant pairs: A-only=5, B-only=12, exact McNemar p=0.1435
```

SVM 高出 1.3 個百分點，但區間包含零，McNemar 的 p 值是 0.14。540 張中只有 17 張能區分兩個模型，而 12 比 5 在公平硬幣下並不罕見。以這個測試集而言，誠實的結論是：「B 大概至少一樣好；說它更好的證據很薄弱。」

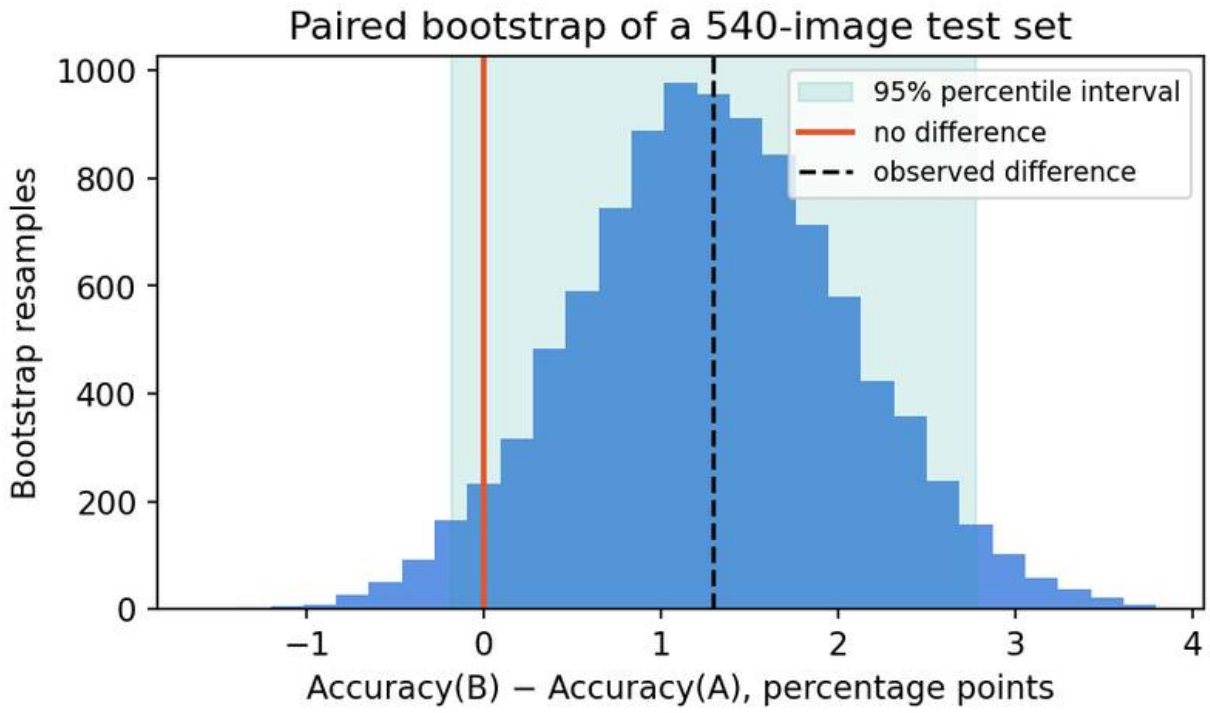


圖 3 — 上方程式碼中 $Accuracy(B) - Accuracy(A)$ 的成對 *bootstrap* 分布。*95%* 區間跨過零。

其他變異來源

測試樣本不是唯一的隨機來源。對神經網路而言，隨機種子會改變初始化、資料順序與資料擴增；資料切分本身也是隨機的；某些 GPU 核心是非確定性的。Bouthillier 等人為整個基準評測過程建模，發現資料取樣、參數初始化與超參數選擇造成的變異會明顯影響結論，並建議在重複實驗中盡可能隨機化這些來源 [10]。實務原則：

- 每個設定用多個種子執行（常見 3–5 個），回報平均與分散程度。
- 用成對檢定，在相同樣本與切分上比較方法。
- 若比較很多模型或很多資料集，總會有一些純屬偶然地「勝出」；要做多重比較校正，或在新資料上確認。
- 回報效果大小與區間。統計顯著的差異在實務上可能無關緊要；實務上很大的差異，在小資料集上也可能尚未被統計證實。

校準：知道自己可能出錯

白話版。模型的信心應該和它答對的頻率相符。在所有以 80% 信心做出的預測中，大約 80% 應該是對的。準確率與校準是兩種不同的性質。

量測機率品質

令 p_{ik} 為樣本 i 屬於類別 k 的預測機率，$y_{ik} \in \{0,1\}$ 為 one-hot 標籤。兩個**合適評分規則**（proper scoring rule，在期望上由真實機率達到最小）為：

$$\mathrm{NLL} = -\frac{1}{n}\sum_{i=1}^{n} \log p_{i,y_i}, \qquad \mathrm{Brier} = \frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} (p_{ik} - y_{ik})^2,$$

其中 y_i 是樣本 i 的真實類別，K 是類別數。NLL 是負對數概似（negative log-likelihood）。有些函式庫會把 Brier 分數再除以 K；請說明你使用哪種慣例。

期望校準誤差（expected calibration error, ECE）依最高類別信心把預測分到 M 個區間 $B_1, \dots, B_M$，並在每個區間比較平均信心與準確率：

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \left| \mathrm{acc}(B_m) - \mathrm{conf}(B_m) \right|,$$

其中 $\mathrm{acc}(B_m)$ 是第 m 個區間的答對比例，$\mathrm{conf}(B_m)$ 是該區間的平均信心。**可靠度圖**（reliability diagram）逐區間畫出 acc 對 conf；完美校準的模型落在對角線上。ECE 不是合適評分規則，且取決於區間數與分箱方式，因此應與 NLL 或 Brier 分數及可靠度圖一起回報。

溫度縮放

Guo 等人觀察到，比起數十年前的網路大得多的現代深度網路常常過度自信，而一個純量修正就出奇地有效 [11]。**溫度縮放**（temperature scaling）在 softmax 之前把 logits ℓ 除以 $T > 0$：

$$\hat{p}_k = \frac{\exp(\ell_k / T)}{\sum_j \exp(\ell_j / T)}, \qquad T^\star = \arg\min_{T>0} \ \mathrm{NLL}_{\mathrm{cal}}(T).$$

$T > 1$ 會軟化過度自信的預測；$T < 1$ 則讓信心不足的預測更尖銳。因為把所有 logits 除以同一個正數不會改變誰最大，預測類別（也就是準確率）不變。T 必須在留出的校準集上擬合，絕不能在測試集上擬合。後來 Minderer 等人發現，最新的影像分類器，尤其是 Vision Transformer 這類不使用卷積的模型，屬於校準最好的一群，而且「校準隨分布偏移而衰退」等舊趨勢在它們身上較不明顯 [12]。

程式碼：溫度縮放，以及偏移對它的影響

```
import numpy as np
from scipy.optimize import minimize_scalar
from scipy.special import log_softmax, softmax
```

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

X, y = load_digits(return_X_y=True)
X = X / 16.0
Xa, Xte, ya, yte = train_test_split(X, y, test_size=0.4, stratify=y, random_state=0)
Xtr, Xcal, ytr, ycal = train_test_split(Xa, ya, test_size=0.3, stratify=ya, random_state=0)
net = MLPClassifier(hidden_layer_sizes=(128,), alpha=1e-6, max_iter=300,
                    random_state=0).fit(Xtr, ytr)

def logits(X):                          # log-probabilities = logits up to a constant
    return np.log(np.clip(net.predict_proba(X), 1e-12, 1.0))

def ece(prob, y, bins=10):
    conf, pred = prob.max(1), prob.argmax(1)
    edges = np.linspace(0, 1, bins + 1)
    total = 0.0
    for lo, hi in zip(edges[:-1], edges[1:]):
        m = (conf > lo) & (conf <= hi)
        if m.any():
            total += m.mean() * abs((pred[m] == y[m]).mean() - conf[m].mean())
    return total

def nll(T, z, y):
    return -log_softmax(z / T, axis=1)[np.arange(len(y)), y].mean()

T = minimize_scalar(nll, bounds=(0.05, 20), method="bounded",
                    args=(logits(Xcal), ycal)).x
print(f"fitted temperature T = {T:.2f}")
noise = np.random.default_rng(0).normal(size=Xte.shape)
for sigma in [0.0, 0.3, 0.6]:           # Gaussian pixel noise = a simple covariate shift
    Xs = np.clip(Xte + sigma * noise, 0, 1)
    z = logits(Xs)
    p_raw, p_ts = softmax(z, axis=1), softmax(z / T, axis=1)
    print(f"noise {sigma:.1f}: acc={(p_raw.argmax(1) == yte).mean():.3f}  "
          f"ECE raw={ece(p_raw, yte):.3f}  ECE scaled={ece(p_ts, yte):.3f}  "
          f"NLL raw={nll(1, z, yte):.3f}  NLL scaled={nll(T, z, yte):.3f}")
```

輸出（scikit-learn 可能印出收斂警告，在這裡無害）：

```
fitted temperature T = 1.19
noise 0.0: acc=0.972  ECE raw=0.007  ECE scaled=0.015  NLL raw=0.108  NLL scaled=0.110
noise 0.3: acc=0.794  ECE raw=0.088  ECE scaled=0.063  NLL raw=0.693  NLL scaled=0.639
noise 0.6: acc=0.435  ECE raw=0.380  ECE scaled=0.347  NLL raw=2.998  NLL scaled=2.598
```

有三點觀察。在乾淨資料上，這個小網路本來就校準得很好，而用 324 張校準影像擬合 T 反而讓 ECE 略微**變差**：在小資料上擬合的修正可能引入雜訊。加入像素雜訊後，模型嚴重過度自信：在 $\sigma = 0.6$ 時它只有 43.5% 答對，信心卻高得多。在乾淨資料上擬合的溫度縮放，在偏移下只幫上一點忙。這與 Ovadia 等人的大型研究一致：他們發現許多不確定性方法的校準會隨偏移增加而惡化 [16]。

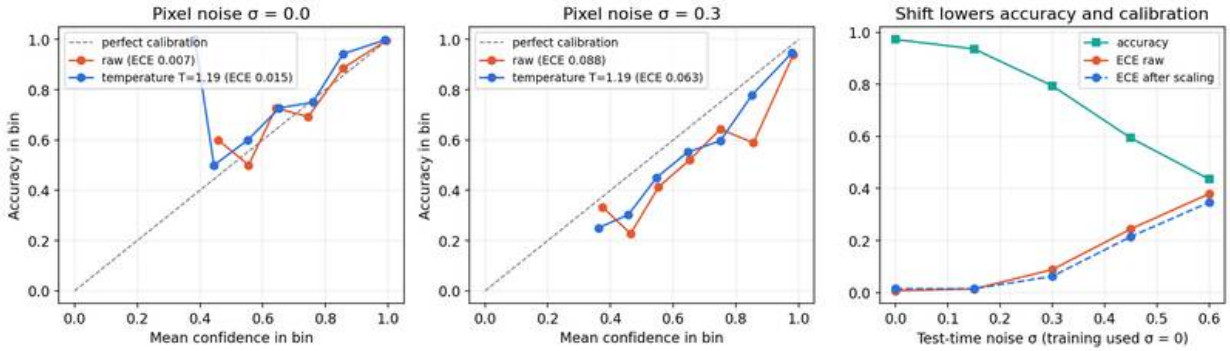


圖 4 — 上述網路的可靠度圖與偏移掃描（使用同一組雜訊樣式，依 σ 縮放）。點落在對角線下方代表過度自信。

穩健性與分布偏移

白話版。部署後的世界很少和訓練集一模一樣。相機會換、光線會變、使用者族群會變。穩健的模型會優雅地退化；脆弱的模型則無聲地失敗。

三種偏移

把聯合分布寫成 $p(x, y) = p(y \mid x)\,p(x) = p(x \mid y)\,p(y)$，其中 x 是輸入、y 是標籤。偏移依訓練時（p_{tr}）與部署時（p_{te}）哪一部分改變來命名：

- **共變量偏移（covariate shift）**：$p_{\text{te}}(x) \neq p_{\text{tr}}(x)$，但 $p(y \mid x)$ 不變。新相機、新光線、新掃描儀。
- **標籤（先驗）偏移（label shift）**：$p_{\text{te}}(y) \neq p_{\text{tr}}(y)$，但 $p(x \mid y)$ 不變。疫情爆發時某種疾病變得更常見。
- **概念偏移（concept shift）**：$p(y \mid x)$ 本身改變。「垃圾郵件」的定義隨時間演變。

在純共變量偏移下，期望測試損失可改寫成加權的訓練損失：$\mathbb{E}_{\text{te}}[\ell] = \mathbb{E}_{\text{tr}}[w(x)\,\ell]$，重要性權重 $w(x) = p_{\text{te}}(x)/p_{\text{tr}}(x)$。對影像而言這些權重很難估計，但這個等式說明了：測試輸入落在訓練資料稀少的區域時，偏移的影響最大。在標籤偏移下，Lipton 等人證明，只要有固定黑箱分類器在留出訓練資料上的混淆矩陣，加上它在未標註測試資料上的預測頻率，就足以估計新的類別先驗並重新加權模型 [17]。

探測偏移的基準

- **ImageNet-C** 對 ImageNet 驗證影像施加 15 種損壞（雜訊、模糊、天氣、數位），並分多個嚴重程度；**ImageNet-P** 則加入擾動序列。兩者量測的是對常見損壞的穩健性，而不是對抗攻擊 [13]。我們的像素雜訊掃描就是一個迷你版本。
- **ImageNetV2** 盡可能依照原始收集流程重新建立 ImageNet 測試集。許多模型的準確率下降了 11–14%，但在原始測試集上的進步轉換成在新測試集上更大的進步，所以這個下降反映的是影

像稍微更難，而不是對重複使用的測試集過擬合 [14]。即使是小心的重新收集，本身也是一種偏移。
- **WILDS** 收集了十個具有自然偏移的資料集，包括來自不同醫院的腫瘤組織、新地點的自動相機以及來自新國家的衛星影像，並報告分布內與分布外表現之間有很大落差，而現有的緩解方法都沒能補上 [15]。

實務教訓：一定要加入至少一個依照你預期的部署變化（新院區、新裝置、較晚的時間）建立的**偏移**評估集，並把分布內與偏移後的結果並列回報。

效率：FLOPs、記憶體與延遲

白話版。「快」與「小」各有好幾種意思。運算量較少的模型不一定比較快，參數較少的模型也不一定比較省記憶體。要在目標裝置上量測。

計算運算量

一次**乘加運算**（multiply–accumulate, MAC）是一次乘法加一次加法，所以 1 MAC ≈ 2 個浮點運算（FLOPs）。對有 d_{in} 個輸入、d_{out} 個輸出的全連接層，以及有 C_{in} 個輸入通道、C_{out} 個輸出通道、$k \times k$ 卷積核與 $H' \times W'$ 輸出的卷積層：

$$\text{FLOPs}_{\text{linear}} \approx 2\, d_{\text{in}} d_{\text{out}}, \qquad \text{FLOPs}_{\text{conv}} \approx 2\, C_{\text{in}} k^2 C_{\text{out}} H' W'$$

以上是每個樣本的運算量（偏差與激活函數的成本很小）。各函式庫回報的是 MAC 還是 FLOPs 並不一致，請寫明。

為什麼 FLOPs 不等於速度

Dehghani 等人指出，常見的成本指標（參數量、FLOPs、吞吐量、延遲）可能彼此矛盾，只回報其中一個會讓效率看起來失真 [18]。主要原因：

- **記憶體流量**。若某層必須從記憶體讀取權重，卻只做很少的運算，它受限於記憶體頻寬而非運算。對單一樣本套用線性層時，要讀取 $4d_{\text{in}}d_{\text{out}}$ 位元組的 fp32 權重，只做 $2d_{\text{in}}d_{\text{out}}$ 個 FLOPs：每位元組 0.5 個 FLOPs，遠不足以讓現代加速器忙起來。批次化可以提高這個比率（稱為**運算強度**，arithmetic intensity）。
- **平行度**。深度可分離卷積與許多小層的 FLOPs 很少，卻無法充分利用硬體。
- **核心支援**。在目標執行環境上沒有最佳化核心的運算，無論 FLOPs 多少都可能很慢。

記憶體帳

推論時，記憶體要放權重（參數量 × 每個值的位元組數）、目前仍在使用的各層激活值、核心的工作空間，對自迴歸 Transformer 還有會隨上下文長度與批次大小成長的 **KV 快取**。訓練時還要加上梯度與最佳化器狀態：Adam 每個參數多存兩個值，所以使用 Adam 的 fp32 模型在計入激活值之前，就大約需要權重記憶體的 4×。

延遲量測協定

實際延遲取決於硬體、執行環境、批次大小、輸入解析度、數值精度與執行緒設定，而且前幾次呼叫因為一次性的初始化會比較慢。可靠的協定：

1. 固定並回報裝置、執行環境版本、精度、批次大小、輸入尺寸與執行緒數。
2. 先執行暖機迭代並丟棄結果。
3. 計時多次並回報百分位數（p50 與 p90 或 p99），而不只是平均：使用者感受到的是尾端。
4. 在 GPU 上，停止計時前要先同步裝置，因為核心是非同步啟動的。
5. 除了網路本身，也要量測端到端時間（前處理、資料傳輸、解碼）。

柏拉圖前緣（Pareto frontier）包含那些「沒有其他候選在每個面向（準確率、延遲、記憶體）都至少一樣好、且至少在一個面向嚴格更好」的候選模型。呈現前緣能讓取捨一目了然，而不必捏造一個唯一的「最佳模型」。

部署：匯出、壓縮與執行環境

白話版。要上線一個模型，你會把它凍結成可攜的檔案，通常再讓它更小或更省，然後用為目標硬體設計的推論引擎執行。每一步都可能改變輸出，所以每一步都需要重新評估。

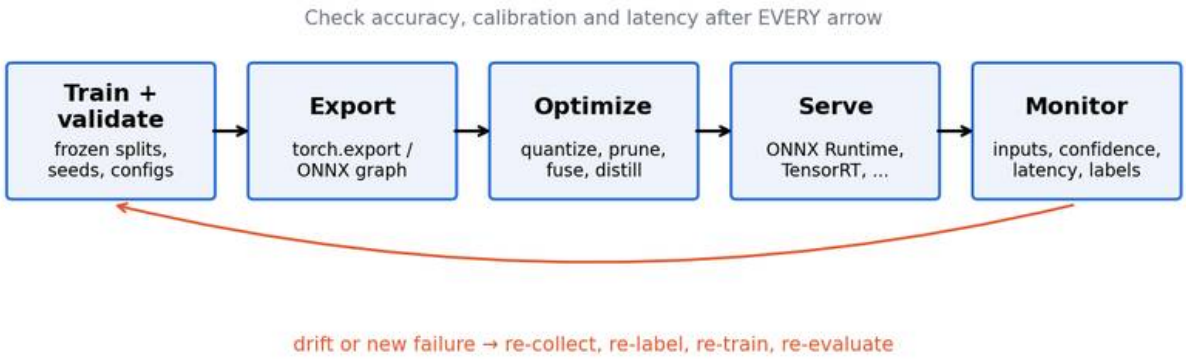


圖 5 — 部署迴圈。每一步之後都應重新檢查準確率、校準與延遲，因為匯出、壓縮與執行環境都會改變數值。

匯出與推論執行環境

訓練程式碼寫在彈性高、由 Python 驅動的框架中。部署則偏好靜態的計算描述：

- `torch.export` 以預先（ahead-of-time）的方式把 PyTorch 模組追蹤成 `ExportedProgram`，也就是一張張量運算圖，可以序列化，並在沒有原始 Python 類別的情況下用不同輸入執行 [19]。
- **ONNX**（Open Neural Network Exchange）是機器學習模型的開放標準檔案格式，讓在一個框架中訓練的模型能被其他工具執行 [20]。

- **ONNX Runtime** 是 ONNX 模型的跨平台推論引擎。它把運算圖切成子圖，交給各個硬體專屬的**執行提供者**（execution provider，例如 CUDA、TensorRT、OpenVINO 或 CoreML）[21]。
- **NVIDIA TensorRT** 是一套 SDK，把模型（來自各框架或 ONNX）編譯成 NVIDIA GPU 上的最佳化引擎，使用層融合與 FP16、BF16、FP8、INT8 等較低精度 [22]。

這些執行環境靠融合運算、為確切的張量形狀挑選快速核心以及降低精度來加速。因此同一個模型在不同執行環境上的輸出可能略有差異。請在一批驗證資料上比較匯出後與原始模型的輸出，並在轉換後重新量測準確率。

量化

量化（quantization）用較少的位元來儲存與計算。在常見的**仿射（非對稱）均勻**量化中，實數值 x 會被映射成整數 q，再映射回來：

$$q = \mathrm{clip}\Big(\mathrm{round}\big(\tfrac{x}{s}\big) + z,\ 0,\ 2^{b}-1\Big), \qquad \hat{x} = s\,(q - z),$$

其中 b 是位元寬度，$s > 0$ 是**尺度**（可表示值之間的間距），z 是整數**零點**，選擇它使實數 0 能被精確表示，$\hat{x}$ 是反量化後的近似值。對包含 0 的範圍 $[x_{\min}, x_{\max}]$，$s = (x_{\max} - x_{\min})/(2^{b}-1)$、$z = \mathrm{round}(-x_{\min}/s)$。**對稱**變體固定 $z = 0$ 並使用有號範圍。Jacob 等人的純整數運算方案說明了如何用 8 位元整數矩陣乘法執行整個網路，以及如何為此進行訓練 [23]。

誤差有兩個來源：範圍內的**捨入**（每個值最多 $s/2$）與範圍外的**截斷**。兩者都取決於尺度的**粒度**：每個張量一個尺度、每個輸出通道一個，或每一小組權重一個。Nagel 等人提供實用指南，涵蓋這些選擇、**訓練後量化**（post-training quantization, PTQ：訓練後用一小份具代表性的資料校準範圍）與**量化感知訓練**（quantization-aware training, QAT：訓練時模擬捨入，讓權重去適應）[24]。Gholami 等人則綜述了更廣的設計空間 [25]。

```
import numpy as np

def quantize(x, bits=8, axis=None):
    """Asymmetric (affine) uniform quantization, per tensor or per channel."""
    qmin, qmax = 0, 2**bits - 1
    lo = x.min(axis=axis, keepdims=True) if axis is not None else x.min()
    hi = x.max(axis=axis, keepdims=True) if axis is not None else x.max()
    lo, hi = np.minimum(lo, 0), np.maximum(hi, 0)      # keep 0 exactly representable
    s = (hi - lo) / (qmax - qmin)                       # scale
    z = np.round(qmin - lo / s)                         # zero point (integer)
    q = np.clip(np.round(x / s) + z, qmin, qmax)        # stored integers
    return s * (q - z)                                  # dequantized values

rng = np.random.default_rng(0)
# 64 output channels whose weight magnitudes differ by up to 100x
W = rng.normal(size=(64, 256)) * np.logspace(-2, 0, 64)[:, None]
for bits in [8, 4]:
    e_t = np.abs(quantize(W, bits) - W).mean()
    e_c = np.abs(quantize(W, bits, axis=1) - W).mean()
    print(f"{bits}-bit mean |error|: per-tensor {e_t:.5f}  per-channel {e_c:.5f}")
print("zero maps back to:", quantize(np.array([-1.0, 0.0, 3.0]))[1])
```

輸出：

```
8-bit mean |error|: per-tensor 0.00632  per-channel 0.00124
4-bit mean |error|: per-tensor 0.06718  per-channel 0.02112
zero maps back to: 0.0
```

當各通道的範圍差異很大時，共用一個尺度會把大部分格點浪費在最大的通道上，讓小通道被粗糙地捨入。在這裡，逐通道尺度讓 8 位元時的誤差降到約五分之一。在大型語言模型中，少數離群的激活通道會在更大規模上造成同樣的問題，這正是 GPTQ（逐層、補償誤差的權重捨入）與 AWQ（依激活找出並保護重要權重通道）等方法在 3–4 位元純權重量化中處理的問題 [42], [43]。

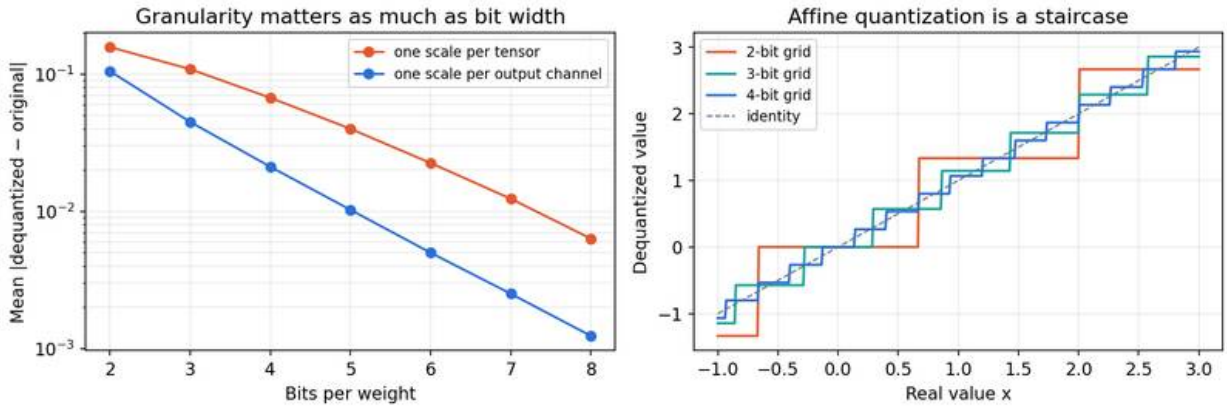


圖 *6* — 每多一個位元，量化誤差大約減半；尺度粒度的選擇則讓整條曲線上下平移。

剪枝

剪枝（pruning）移除貢獻很小的權重、通道、注意力頭或區塊。**非結構化**剪枝把個別權重設為零；它可以移除很大比例的參數，例如 Deep Compression 結合剪枝、以量化實現的權重共享與 Huffman 編碼，大幅縮小網路 [26]。但零星分布的零只有在執行環境具備該稀疏樣式的核心時才會省時間。**結構化**剪枝移除整個通道或注意力頭，產生較小的稠密張量，任何執行環境都能利用。Blalock 等人彙整了 81 篇剪枝論文，發現基準與指標不一致，使人難以判斷哪些方法真的有效，並提出標準化的評測框架 [27]。Hoefler 等人深入綜述了訓練與推論中的稀疏性 [28]。

知識蒸餾

蒸餾（distillation）訓練一個小的**學生**模型去模仿大的**教師**模型。Hinton 等人提出讓學生匹配教師軟化後的輸出分布 [29]。設溫度為 T，教師與學生的 logits 分別為 ℓ^t 與 ℓ^s，軟化機率為 $p^{(T)} = \mathrm{softmax}(\ell/T)$，權重 $\lambda \in [0,1]$：

$$\mathcal{L} = (1-\lambda)\,\mathcal{L}_{\mathrm{CE}}(y,\, p^{s,(1)}) + \lambda\, T^2\, D_{\mathrm{KL}}(p^{t,(T)} \,\|\, p^{s,(T)}).$$

軟化的教師分布帶有「暗知識」：哪些錯誤類別彼此相似。軟項的梯度按 $1/T^2$ 縮放，所以乘上 T^2 能在改變 T 時保持兩項的平衡 [29]。Gou 等人依**傳遞什麼**（輸出、中間特徵、樣本間關係）與**如**

何傳遞（離線、線上、自蒸餾）整理了後續研究 [30]。學生會繼承教師的能力，也會繼承它的錯誤與偏差。

程式碼：純權重 int8、延遲與匯出

這個範例把一個小 MLP 的權重以 int8 儲存（每個輸出通道一個尺度），並在 CPU 上量測檔案大小與延遲。只用到 PyTorch 核心功能。

```
import io, time
import numpy as np
import torch
from torch import nn

torch.manual_seed(0)
model = nn.Sequential(nn.Linear(512, 2048), nn.ReLU(),
                      nn.Linear(2048, 2048), nn.ReLU(),
                      nn.Linear(2048, 10)).eval()
x = torch.randn(32, 512)

class Int8WeightLinear(nn.Module):
    """Weight-only int8 storage, symmetric per output channel."""
    def __init__(self, lin):
        super().__init__()
        w = lin.weight.detach()
        scale = w.abs().amax(dim=1, keepdim=True) / 127
        self.register_buffer("q", torch.round(w / scale).to(torch.int8))
        self.register_buffer("scale", scale)
        self.register_buffer("bias", lin.bias.detach().clone())
    def forward(self, x):                       # dequantize, then fp32 matmul
        return nn.functional.linear(x, self.q.float() * self.scale, self.bias)

int8 = nn.Sequential(*[Int8WeightLinear(m) if isinstance(m, nn.Linear) else m
                       for m in model]).eval()

def size_mb(m):
    buf = io.BytesIO()
    torch.save(m.state_dict(), buf)
    return buf.getbuffer().nbytes / 1e6

def latency_ms(m, x, warmup=10, runs=100):
    with torch.inference_mode():
        for _ in range(warmup):                 # first calls pay one-off costs
            m(x)
        times = []
        for _ in range(runs):
            t0 = time.perf_counter()
            m(x)
            times.append((time.perf_counter() - t0) * 1e3)
    return np.percentile(times, [50, 90]).round(2)

with torch.inference_mode():
    gap = (model(x) - int8(x)).abs().max().item()
```

```
    agree = (model(x).argmax(1) == int8(x).argmax(1)).float().mean().item()
print(f"fp32 file: {size_mb(model):.1f} MB  p50/p90 latency {latency_ms(model, x)} ms")
print(f"int8 file: {size_mb(int8):.1f} MB  p50/p90 latency {latency_ms(int8, x)} ms")
print(f"max |logit difference| = {gap:.4f}, top-1 agreement = {agree:.3f}")

# Ahead-of-time graph capture: a portable program, independent of this Python class.
exported = torch.export.export(model, (x,))
ops = [n.target for n in exported.graph.nodes if n.op == "call_function"]
print(type(exported).__name__, "->", [str(o).split(".")[1] for o in ops])
```

在 CPU 上（四個執行緒）執行一次的輸出：

```
fp32 file: 21.1 MB  p50/p90 latency [2.74 4.49] ms
int8 file: 5.3 MB  p50/p90 latency [ 9.23 11.06] ms
max |logit difference| = 0.0018, top-1 agreement = 1.000
ExportedProgram -> ['linear', 'relu', 'linear', 'relu', 'linear']
```

你的延遲數字會不同；重點在於趨勢。檔案小了四倍，輸出幾乎沒變。但 int8 版本反而**更慢**，因為它每次呼叫都把權重轉回 fp32，再執行同樣的 fp32 矩陣乘法。只有在執行環境具備該運算的整數（或低精度）核心時，或工作負載受記憶體頻寬限制、讀取較少位元組成為主因時，較低精度才會變快。這正是「壓縮了」與「加速了」之間的落差，也是 TensorRT、ONNX Runtime 以及 PyTorch 自家的 `torchao` 函式庫 [46] 要彌補的地方。這個 MLP 的運算量為每個樣本 $2 \times (512 \cdot 2048 + 2048 \cdot 2048 + 2048 \cdot 10) \approx 10.5$ MFLOPs，兩個版本完全相同。

部署後的監控

白話版。即使程式碼完全沒變，模型也可能變差，因為資料變了。要觀察輸入與輸出，盡可能收集新的標籤，並事先決定什麼情況會觸發調查。

Sculley 等人描述 ML 系統如何累積隱藏的「技術債」：糾纏的特徵、未申報的預測使用者、模型輸出影響未來訓練資料的回饋迴圈，以及依賴會在無預警下改變的外部資料 [33]。Paleyes 等人收集已發表的部署案例研究，發現部署流程的每個階段都會出問題 [39]。有用的訊號，由便宜到最有資訊量：

- **系統健康**：延遲百分位數、錯誤率、記憶體、吞吐量。
- **輸入統計**：影像亮度、解析度、裝置類型、缺失欄位；與訓練時的參考時段比較。
- **輸出統計**：預測類別與信心的分布。這些不需要標籤。
- **延遲到來的標籤**：定期標註一小部分線上資料，是直接估計準確率的唯一方式。

要比較線上時段與參考時段，雙樣本檢定是簡單的起點。對經驗累積分布函數分別為 F_{ref} 與 F_{live} 的一維樣本，**Kolmogorov–Smirnov 統計量**為

$$D = \sup_t |F_{\text{ref}}(t) - F_{\text{live}}(t)|,$$

也就是兩條曲線之間最大的垂直差距。Rabanser 等人比較了多種偏移偵測方法，發現先用預訓練分類器降維（例如取其 softmax 輸出）再做雙樣本檢定，在他們研究的偏移中表現最好 [32]。

```
import numpy as np
from scipy.stats import ks_2samp
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

X, y = load_digits(return_X_y=True)
X = X / 16.0
Xtr, Xlive, ytr, ylive = train_test_split(X, y, test_size=0.5, random_state=0)
Xtr, Xref, ytr = Xtr[:600], Xtr[600:], ytr[:600]   # reference window from training time
clf = LogisticRegression(max_iter=2000).fit(Xtr, ytr)
ref_conf = clf.predict_proba(Xref).max(1)     # label-free signal: top-class confidence

rng = np.random.default_rng(0)
for name, noise in [("week 1 (clean)", 0.0), ("week 2 (new camera)", 0.25)]:
    batch = rng.choice(len(Xlive), 200, replace=False)
    Xb = np.clip(Xlive[batch] + noise * rng.normal(size=Xlive[batch].shape), 0, 1)
    conf = clf.predict_proba(Xb).max(1)
    stat, p = ks_2samp(ref_conf, conf)
    acc = (clf.predict(Xb) == ylive[batch]).mean()   # usually NOT available live
    print(f"{name}: mean confidence {conf.mean():.3f}, KS={stat:.3f}, p={p:.2g}, "
          f"(hidden) accuracy {acc:.3f}")
```

輸出：

```
week 1 (clean): mean confidence 0.846, KS=0.089, p=0.28, (hidden) accuracy 0.950
week 2 (new camera): mean confidence 0.757, KS=0.277, p=1.3e-08, (hidden) accuracy 0.885
```

信心分布在沒有任何標籤的情況下就標記出第 2 週，而隱藏的準確率證實確實發生了變化。偵測器告訴你資料**有沒有**改變，而不是準確率**是否**下降：無害的偏移也會觸發它，而在很大的時段下，即使微小的變化也會變得「顯著」。用它來觸發調查與標註，而不是自動重新訓練。

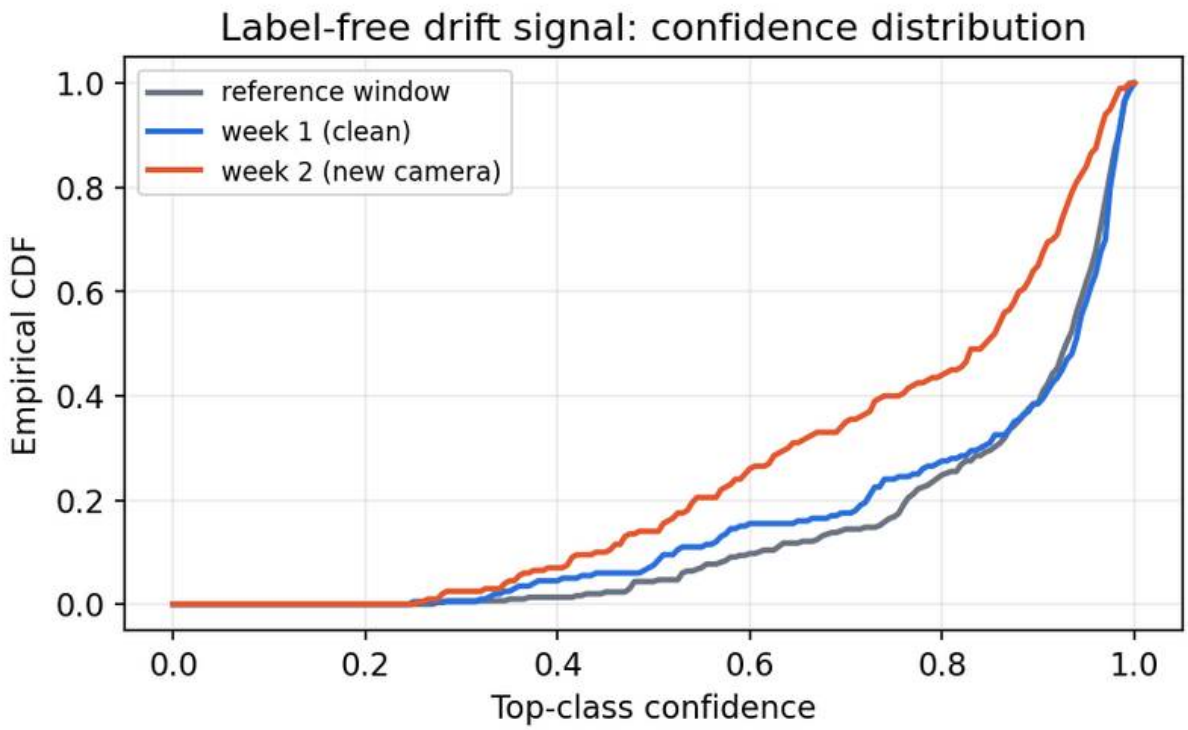


圖 7 — *Kolmogorov–Smirnov* 統計量就是參考曲線與線上時段曲線之間最大的垂直差距。

可重現性與文件

白話版。別人（包括六個月後的你）應該能重跑你的實驗並得到相同的數字，讀者也應該能看出這個模型是做什麼用的、哪些場合不該使用。

讓實驗可重複

為每個函式庫固定並記錄隨機種子，但要知道它的極限：PyTorch 文件明言，不同版本、不同 commit 或不同平台之間無法保證完全可重現的結果，並建議在需要逐位元一致時設定種子、呼叫 `torch.use_deterministic_algorithms(True)` 並關閉 cuDNN benchmark [34]。除了種子，還要記錄：

- 資料集版本、切分 ID（實際的樣本 ID 清單）與去重規則；
- 前處理與資料擴增程式碼、預訓練檢查點及其來源；
- 完整設定（最佳化器、學習率排程、停止規則）、軟體環境與硬體；
- 每一個驗證階段的決策，以及唯一一次的最終測試評估。

Pineau 等人報告的 NeurIPS 2019 可重現性計畫，推出了程式碼提交政策、社群可重現性挑戰，以及正好涵蓋上述項目的檢查表 [37]。

為模型與資料寫文件

模型卡（model cards）是隨訓練好的模型附上的簡短文件，說明預期用途、評估資料、依相關群體與條件細分的表現，以及已知限制 [35]。**資料集說明書**（datasheets for datasets）要求資料集建立者記錄動機、組成、收集過程、前處理、建議用途與維護方式 [36]。兩者都讓本章的評估決策，對依賴它們的人變得透明。

結業實驗

選一個具備適當規模、可合法使用資料集的任務。定義部署母體與切分單位。比較三個候選：一個傳統基準、凍結預訓練特徵加小分類頭，以及一個微調過的網路。對每一個，回報附 bootstrap 區間的測試指標、與最佳基準的成對比較、可靠度圖、一個偏移集上的結果，以及目標裝置上的延遲百分位數。寫一頁模型卡。最終測試集只打開一次；若它揭露了失敗而你修改了模型，就把舊結果視為開發證據，並為修改後的主張取得新的評估。

現代觀點

評估與部署的工程本身已經成為一個研究領域。以下幾篇綜述與回顧勾勒了它的樣貌。

資料洩漏與可重現性。 Kapoor 與 Narayanan 對以 ML 為基礎之科學的回顧，記錄了 17 個領域中 329 篇論文的洩漏問題，並提出「模型資訊表」（model info sheets），要求作者論證其訓練／測試分離為何成立 [4]。結合 Bouthillier 等人的變異研究 [10] 與 NeurIPS 可重現性計畫 [37]，共識是：多數過度樂觀的結果來自評估協定的錯誤，而不是什麼特殊的統計問題。

不確定性與校準。 Gawlikowski 等人綜述深度網路的不確定性估計：他們區分可降低的模型（認知）不確定性與不可降低的資料（偶然）不確定性，把方法分為確定性網路、貝氏網路、集成與測試時資料擴增，並回顧校準方法及其實務限制 [38]。Ovadia 等人則補上實證警告：事後校準在偏移下不夠用，而對多個模型做邊際化的方法（例如集成）表現較好 [16]。

實務部署。 Paleyes 等人綜述已發表的案例研究，把挑戰對應到部署流程的各個階段，並顯示實務工作者在每個階段都會遇到問題 [39]，延伸了 Sculley 等人的技術債觀點 [33]。

模型壓縮。 Cheng 等人提供了 CNN 剪枝、低秩分解、緊湊卷積與蒸餾的經典綜覽 [31]；Gholami 等人 [25] 與 Nagel 等人 [24] 聚焦量化；Hoefler 等人聚焦稀疏性 [28]；Gou 等人聚焦蒸餾 [30]。對大型語言模型，Zhu 等人綜述了針對數十億參數模型的量化、剪枝、蒸餾與低秩方法 [41]。2023–2026 年一個明顯的趨勢是：4 位元純權重量化（GPTQ [42]、AWQ [43]）與 FP8 等低精度格式，已經從論文走進生產工具：TensorRT 列出 FP8、INT8、FP4 與 INT4 等支援精度 [22]，而 PyTorch 的量化工具正整合到 `torchao` 函式庫中 [46]。

全面評估。對基礎模型，HELM 同時在許多情境與多個指標上評估許多模型（準確率、校準、穩健性、公平性、偏差、毒性與效率），並主張單一排行榜數字會掩蓋這些取捨 [40]。同樣的多指標習慣也適用於視覺模型。

規模化。規模定律（scaling laws）擬合模型大小、資料、算力與損失之間的經驗關係。Hoffmann 等人發現在固定算力預算下，語言模型的參數量與訓練 token 數應大致等比例擴增 [44]；Zhai 等人研究 ViT 準確率如何隨模型大小、資料與算力擴展 [45]。一個領域擬合出的常數不應直接搬到另一個領域；可以遷移的教訓是：每個結果都要報告背後的算力與資料。

開放問題。評估資料混入網路規模的預訓練資料（資料污染）很難排除；未知偏移下的校準仍未解決；延遲數字很少能在不同硬體世代之間轉移；生成式與互動式系統的評估（見DLCV 07）仍缺乏公認的協定。

重點整理

- 基準模型與乾淨的切分決定結果是否有意義。依部署時會「新」的單位切分，並把所有前處理放在訓練折之內擬合。
- 交叉驗證在小資料上給出更穩定的估計；樣本相關時用群組或依時間順序的折，在同一份資料上既調參又回報時用巢狀交叉驗證。
- 每個測試分數都需要不確定性。用成對 bootstrap 或 McNemar 檢定，在相同樣本上比較模型。
- 準確率、校準與穩健性是不同的性質。溫度縮放能修正分布內的過度自信，但修不了偏移造成的過度自信。
- FLOPs、參數量、記憶體與延遲可能互相矛盾。要在目標硬體上，以暖機與百分位數量測延遲。
- 量化、剪枝與蒸餾只有在執行環境能利用時才會節省資源；每次匯出或壓縮後都要重新評估準確率。
- 部署後要監控輸入、輸出與少量新標籤，並為模型與資料寫文件，讓別人能重現與判斷。

練習

1. **洩漏稽核。**同事先在全部 1,797 張數字影像上擬合 PCA，再切分訓練與測試並回報測試準確率。說明洩漏在哪裡，以及為什麼 PCA 從未看過標籤卻仍有洩漏。你會如何修正程式碼？

   提示

   主成分方向是用測試影像算出來的，所以表示法被調整得貼近測試分布。把 `PCA` 放進 `Pipeline`，使它只在每個訓練折上擬合。對 PCA 而言這個偏差常常不大，但對使用標籤的監督式前處理（例如依標籤做特徵選擇），偏差可能很大。

2. **罕見的正例。**某資料集只有 1% 是正例影像。永遠預測負例的分類器，準確率、正例召回率與平衡準確率各是多少？你會向醫師回報哪個指標？為什麼？

   提示

   準確率 99%、召回率 0、平衡準確率 0.5（正例召回率 0 與負例召回率 1 的平均）。應回報在臨床選定操作點下的召回率（敏感度）與特異度或精確率，並附上區間，因為真正重要的是漏掉一個正例的代價。

3. **測試集要多大？**你預期準確率約 90%，希望 95% 信賴區間約為 ± 1 個百分點。用常態近似，大約需要多少個獨立測試樣本？如果影像來自 50 位病人，會有什麼不同？

   提示

   解 $1.96\sqrt{0.9 \cdot 0.1/n} = 0.01$，得 $n \approx 3{,}460$。若只有 50 位病人，樣本並不獨立；有效樣本數比較接近 50 而非影像張數，所以區間會寬得多。請用以病人為單位重抽樣的 bootstrap。

4. **手算溫度**。一個三類別模型輸出 logits $(4, 1, 0)$。計算 $T = 1$ 與 $T = 2$ 時最高類別的機率。預測類別會改變嗎？什麼時候擬合出的 T 會小於 1？

   提示

   $T = 1$：$e^4/(e^4 + e^1 + e^0) \approx 0.936$。$T = 2$：$e^2/(e^2 + e^{0.5} + 1) \approx 0.736$。argmax 不變。當模型信心不足時（例如強正則化或標籤平滑之後），擬合出的 $T < 1$。

5. **壓縮的宣稱**。一個剪枝後的模型參數只剩原本的一半，但在你的 GPU 上延遲完全相同。提出三種可能解釋，以及一個能區分它們的實驗。

   提示

   非結構化的零仍被稠密核心拿去相乘；剩下的成本主要來自激活值、記憶體流量或未改變的層；或批次太小，使核心啟動開銷成為主因。逐層剖析時間，並與一個張量真正變小的結構化剪枝模型比較。

6. **設計一個監控方案**。你的瑕疵偵測器在工廠產線上執行；人工檢驗的標籤要兩週後才會回來。設計監控方案：記錄什麼、哪些統計量要與哪個參考比較、什麼門檻會觸發警報，以及警報後要做什麼。

   提示

   每天記錄延遲、影像統計（亮度、模糊程度）、預測瑕疵率與信心直方圖；以 KS 或族群穩定度指標與驗證時的參考時段比較；在持續性變化時才發出警報，而不是單日波動。警報後立即標註一批樣本、計算附區間的準確率，然後才決定要重新校準、微調還是回滾。

參考文獻

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 11 “Practical Methodology”. book
2. G. James, D. Witten, T. Hastie, R. Tibshirani and J. Taylor, *An Introduction to Statistical Learning: with Applications in Python*, Springer, 2023. book
3. scikit-learn developers, “Common pitfalls and recommended practices,” scikit-learn documentation. docs
4. S. Kapoor and A. Narayanan, “Leakage and the reproducibility crisis in machine-learning-based science,” *Patterns*, vol. 4, no. 9, 2023. doi
5. S. Varma and R. Simon, “Bias in error estimation when using cross-validation for model selection,” *BMC Bioinformatics*, vol. 7, art. 91, 2006. doi
6. G. C. Cawley and N. L. C. Talbot, “On over-fitting in model selection and subsequent selection bias in performance evaluation,” *Journal of Machine Learning Research*, vol. 11, pp. 2079–2107, 2010. paper
7. M. Bińkowski, D. J. Sutherland, M. Arbel and A. Gretton, “Demystifying MMD GANs,” ICLR, 2018. arXiv

8. T. G. Dietterich, “Approximate statistical tests for comparing supervised classification learning algorithms,” *Neural Computation*, vol. 10, no. 7, 1998. doi
9. B. Efron, “Bootstrap methods: another look at the jackknife,” *The Annals of Statistics*, vol. 7, no. 1, 1979. doi
10. X. Bouthillier et al., “Accounting for variance in machine learning benchmarks,” MLSys, 2021. arXiv
11. C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, “On calibration of modern neural networks,” ICML, 2017. arXiv
12. M. Minderer et al., “Revisiting the calibration of modern neural networks,” NeurIPS, 2021. arXiv
13. D. Hendrycks and T. Dietterich, “Benchmarking neural network robustness to common corruptions and perturbations,” ICLR, 2019. arXiv
14. B. Recht, R. Roelofs, L. Schmidt and V. Shankar, “Do ImageNet classifiers generalize to ImageNet?,” ICML, 2019. paper
15. P. W. Koh et al., “WILDS: A benchmark of in-the-wild distribution shifts,” ICML, 2021. paper
16. Y. Ovadia et al., “Can you trust your model’s uncertainty? Evaluating predictive uncertainty under dataset shift,” NeurIPS, 2019. arXiv
17. Z. C. Lipton, Y.-X. Wang and A. Smola, “Detecting and correcting for label shift with black box predictors,” ICML, 2018. arXiv
18. M. Dehghani, A. Arnab, L. Beyer, A. Vaswani and Y. Tay, “The efficiency misnomer,” ICLR, 2022. arXiv
19. PyTorch, “torch.export,” PyTorch documentation. docs
20. ONNX, “Open Neural Network Exchange: the open standard for machine learning interoperability.” site
21. ONNX Runtime, “Welcome to ONNX Runtime,” documentation. docs
22. NVIDIA, “NVIDIA TensorRT documentation.” docs
23. B. Jacob et al., “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” CVPR, 2018. doi
24. M. Nagel et al., “A white paper on neural network quantization,” arXiv:2106.08295, 2021. arXiv
25. A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney and K. Keutzer, “A survey of quantization methods for efficient neural network inference,” book chapter in *Low-Power Computer Vision*; arXiv:2103.13630, 2021. arXiv
26. S. Han, H. Mao and W. J. Dally, “Deep compression: compressing deep neural networks with pruning, trained quantization and Huffman coding,” ICLR, 2016. arXiv
27. D. Blalock, J. J. Gonzalez Ortiz, J. Frankle and J. Guttag, “What is the state of neural network pruning?,” MLSys, 2020. arXiv
28. T. Hoefler, D. Alistarh, T. Ben-Nun, N. Dryden and A. Peste, “Sparsity in deep learning: pruning and growth for efficient inference and training in neural networks,” *Journal of Machine Learning Research*, vol. 22, 2021. arXiv
29. G. Hinton, O. Vinyals and J. Dean, “Distilling the knowledge in a neural network,” NIPS Deep Learning Workshop, 2014; arXiv:1503.02531. arXiv
30. J. Gou, B. Yu, S. J. Maybank and D. Tao, “Knowledge distillation: a survey,” *International Journal of Computer Vision*, vol. 129, 2021. doi
31. Y. Cheng, D. Wang, P. Zhou and T. Zhang, “Model compression and acceleration for deep

neural networks: the principles, progress, and challenges," *IEEE Signal Processing Magazine*, vol. 35, no. 1, 2018. doi arXiv
32. S. Rabanser, S. Günnemann and Z. C. Lipton, "Failing loudly: an empirical study of methods for detecting dataset shift," NeurIPS, 2019. arXiv
33. D. Sculley et al., "Hidden technical debt in machine learning systems," NeurIPS, 2015. paper
34. PyTorch, "Reproducibility," PyTorch documentation. docs
35. M. Mitchell et al., "Model cards for model reporting," ACM FAT*, 2019. arXiv
36. T. Gebru et al., "Datasheets for datasets," *Communications of the ACM*, vol. 64, no. 12, 2021. doi
37. J. Pineau et al., "Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program)," *Journal of Machine Learning Research*, vol. 22, 2021. paper
38. J. Gawlikowski et al., "A survey of uncertainty in deep neural networks," *Artificial Intelligence Review*, vol. 56, 2023. doi
39. A. Paleyes, R.-G. Urma and N. D. Lawrence, "Challenges in deploying machine learning: a survey of case studies," *ACM Computing Surveys*, vol. 55, no. 6, 2022. doi
40. P. Liang et al., "Holistic evaluation of language models," *Transactions on Machine Learning Research*, 2023. arXiv
41. X. Zhu, J. Li, Y. Liu, C. Ma and W. Wang, "A survey on model compression for large language models," *Transactions of the Association for Computational Linguistics*, vol. 12, 2024. doi
42. E. Frantar, S. Ashkboos, T. Hoefler and D. Alistarh, "GPTQ: accurate post-training quantization for generative pre-trained transformers," ICLR, 2023. arXiv
43. J. Lin et al., "AWQ: activation-aware weight quantization for LLM compression and acceleration," MLSys, 2024. arXiv
44. J. Hoffmann et al., "Training compute-optimal large language models," arXiv:2203.15556, 2022; NeurIPS 2022 version titled "An empirical analysis of compute-optimal large language model training." arXiv
45. X. Zhai, A. Kolesnikov, N. Houlsby and L. Beyer, "Scaling vision transformers," CVPR, 2022. doi
46. PyTorch, "torchao documentation." docs

篇 VI

電腦視覺深度學習

DLCV 01・深度學習的影像處理與電腦視覺

深度學習如何重新整理影像處理、影像分析與電腦視覺：從傳統圖形識別流程，到 CNN、Vision Transformer、自監督骨幹網路與可提示的基礎模型。

網站版本: https://blog.shyandram.dev/zh-tw/learn/deep-learning/dl-01-deep-learning-for-image-processing/

你將學到

- 傳統影像工作的三個層次（影像處理、影像分析、電腦視覺），如何對應到深度學習實際使用的兩大類：**低階**（low-level）與**高階**（high-level）視覺任務。
- 為什麼教科書上的圖形識別流程（感測器 → 特徵 → 分類器 → 驗證）會被**端對端學習**（end-to-end learning）取代，以及它又如何以「預訓練骨幹＋輕量 head」的形式悄悄回歸。
- 卷積層為什麼就是一組**學習而來的空間濾波器**，以及卷積核大小、步幅、填補與池化如何決定**感受野**（receptive field）與特徵階層。
- 經典 CNN（LeNet-5、AlexNet、VGG、GoogLeNet、ResNet）背後的設計理由，以及用於密集預測的編碼器–解碼器（FCN、U-Net）。
- **Vision Transformer** 怎麼運作、為什麼需要大規模預訓練，以及 Swin、ConvNeXt 與自監督骨幹網路（DINO、MAE、DINOv2/v3）各自如何回應它。
- 現今任務與代表模型的全貌：分類、偵測、分割（SAM 系列）、復原與增強、生成（擴散模型、潛在擴散、DiT），以及視覺–語言模型（CLIP、LLaVA）。

先看全貌

這篇是深度學習系列的入口。它不會深入任何單一任務，而是先給你一張地圖：影像問題有哪些種類、哪些網路設計能解決它們、這些設計又為什麼長成現在這樣。後續的教學會再分別深入影像分類、物件偵測、影像復原、低光源增強與物件追蹤。

> **用最簡單的話說**
>
> 想像一間照相館裡有兩種員工。**修圖師傅**拿到一張刮傷、模糊或太暗的照片，交回一張比較乾淨的照片：照片進、照片出。**偵探**看著一張照片，告訴你裡面**有什麼**：「一隻貓，坐在沙發上，在左邊。」以前的電腦視覺，是由專家幫這兩位員工準備好一套固定的工具。深度學習則讓他們看過幾百萬個例子之後，自己打造工具。現在甚至出現了一位「超級員工」，看過數十億張圖，你只要用幾個字告訴他要做什麼就好。

為什麼重要：幾乎所有現代相機的處理流程、醫學影像產品、駕駛輔助系統與修圖 App，都是由本篇介紹的元件組成。看懂這張地圖，你就能為新問題挑一個合理的起點、帶著批判的眼光讀論文，也能看出數位影像處理系列裡的哪些傳統想法仍活在網路裡面（其實很多）。

關於本篇

本篇改寫並擴充自作者先前的文章〈深度學習的數位影像處理介紹〉[1]，保留原文的架構與觀點，並補上細節、數學與程式碼。

影像任務的三個層次

白話版。廣義的「影像處理」包含三種工作。有些操作把一張影像變成一張更好的影像；有些操作從影像中取出某個部分或某種性質；有些則試著像人一樣**理解**影像。

精確版。依照 Gonzalez 與 Woods [2] 的分法，可以用「輸入什麼、輸出什麼」把影像工作分成三個層次。

層次	輸入 → 輸出	典型操作	傳統例子
影像處理（狹義；低階）	影像 → 影像	逐像素或鄰域的基本步驟	直方圖等化、濾波、傅立葉轉換（IP 03、IP 04）
影像分析（中階）	影像 → 屬性或被標示出的區域	分割、特徵擷取、描述，運用先驗知識	人臉影像中的膚色區域、X 光影像中的血管區域（IP 10、IP 12）
電腦視覺（高階；理解）	影像 → 意義	辨識、詮釋、判斷	分類、「畫面中有沒有行人？」（IP 13）

這三者的界線並不嚴格，實際系統往往三者都會用到。例如一個血管分割工具，會先去雜訊（低階）、再擷取血管圖（中階），最後可能標記出狹窄處（高階）。這三個層次描述的是**計算的種類**，而不是三個分開的部門。

深度學習如何重畫這張地圖

在深度學習的論文裡，你幾乎看不到「中階」這個詞。這個領域依照**輸出的形式**，只把任務分成兩大類 [1]：

- **低階視覺**：輸出是一張和輸入（大致）同樣大小的影像，目標是逐像素準確的訊號。例如去雜訊、去模糊、超解析度、除霧、低光源增強、影像到影像轉換。
- **高階視覺**：輸出是**語意**描述，可能是一個標籤、一組帶標籤的方框，或是每個像素一個標籤。

原本的「影像分析」被併入兩者之一。分割的輸出是逐像素的圖，**看起來**像影像到影像的任務；但它的數值是需要辨識才能得到的類別標籤，所以被歸為高階視覺。至於輸出一張更乾淨的亮度影像的邊緣或血管增強，則歸為低階視覺。

這樣分的理由很實際。兩類任務在損失函數（像素迴歸 vs. 分類）、架構（全解析度的編碼器–解碼器 vs. 逐步降採樣的骨幹網路）、訓練資料（劣化/乾淨影像對 vs. 人工標註）與評估指標（保真度 vs. 正確率）上都不同。圖 1 畫出了這個重新整理。

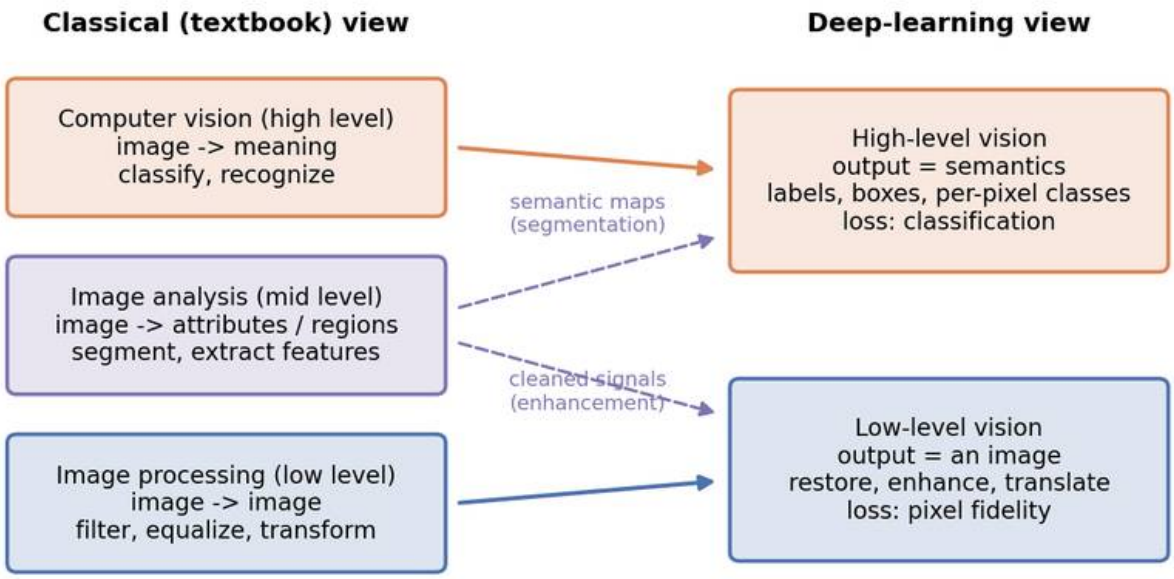


圖 1 — 傳統的三個層次（左）與深度學習使用的兩大類（右）。中階的影像分析依輸出是「清理過的訊號」還是「語意圖」而被分到兩邊。

從圖形識別流程到端對端學習

白話版。在深度學習之前，辨識系統像一條生產線：先有相機，再由人設計的步驟量出有用的性質，最後由分類器做決定。深度學習把中間的步驟融合成一個網路，讓它自己學會要量什麼。

精確版。傳統流程 [3] 有四個階段：

1. **感測器**（sensor）。擷取圖形（相機、掃描器、顯微鏡）。
2. **特徵生成與選擇**（feature generation & selection）。以人工設計的映射 ϕ（動差、紋理統計、邊緣直方圖、關鍵點描述子），從影像 I 計算出向量 $\mathbf{x} = \phi(I) \in \mathbb{R}^d$，再保留最有資訊量的分量。
3. **分類器設計**（classifier）。學習決策規則 $\hat{y} = g(\mathbf{x}; \theta)$，$\theta$ 為參數，例如貝氏分類器、最近鄰規則或支援向量機。
4. **系統驗證**（evaluation）。在保留的資料上量測錯誤率，太高就回到第 2 或第 3 步。

被學習的只有 θ。特徵映射 ϕ 由工程師固定，所以整個系統的上限就是 ϕ 的品質。特徵擷取（IP 12）那一章可以看到，這些映射背後花了多少巧思。

端對端學習讓特徵映射也變成可學習的。深度網路計算

$$\hat{y} = g(\phi(I; \mathbf{w}); \theta),$$

其中 $\phi(\cdot\,; \mathbf{w})$ 是權重為 $\mathbf{w}$ 的多層網路，$g(\cdot\,; \theta)$ 是一個小的 head，$\mathbf{w}$ 與 θ 在標註資料 (I, y) 上以梯度下降與反向傳播共同最小化損失 $\mathcal{L}(\hat{y}, y)$。輸入就是原始像素；除了架構與損失函數之外，沒有任何東西是人工設計的。

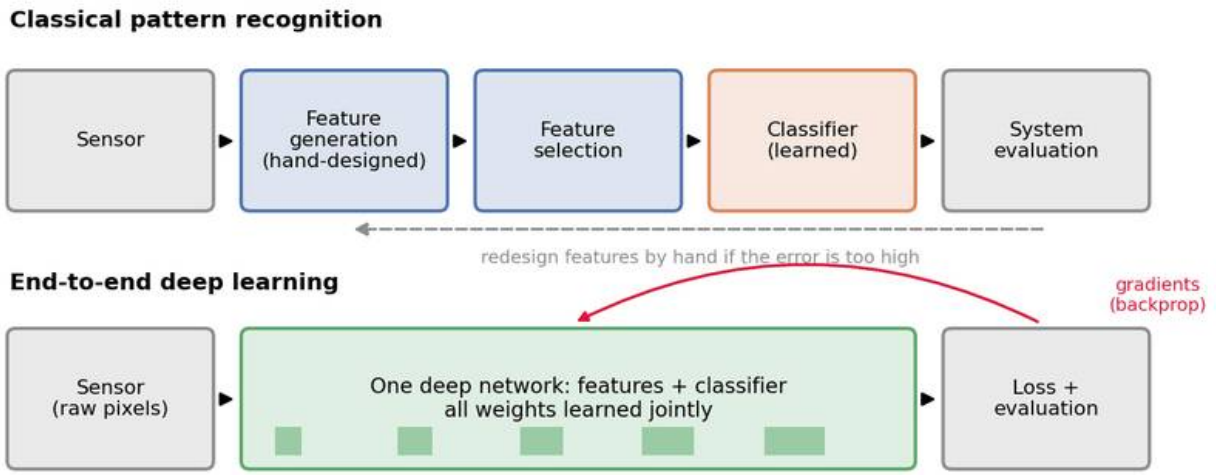


圖 2 — 傳統圖形識別（上）與端對端學習（下）。深度網路裡「特徵」與「分類器」的功能區分仍然存在，但兩者之間沒有清楚的分界。

網路內部的功能區分其實還在：前面的層像特徵擷取器，最後幾層像分類器。但實務上，你通常無法指著某一層說「特徵到這裡結束」[1]。這帶來兩個結果：

- **資料取代了設計**。端對端系統需要大量標註資料；人工設計的 ϕ 原本承載的先驗知識，現在得由網路從資料中學到。
- **分工以更大的規模回歸**。大約從 2021 年起，標準作法是用一個**預訓練骨幹**（在巨大的資料集上訓練一次，常常不需要標註）加上一個在你的小資料集上訓練的**輕量任務 head**。這又是「特徵擷取器＋分類器」，只是特徵擷取器是學來的，而不是設計出來的。我們會在自監督骨幹那一節回到這一點。

卷積神經網路

白話版。卷積層拿一個小視窗在影像上滑動，在每個位置計算加權和，跟IP 03 的空間濾波器一模一樣。差別在於：CNN 會**學習**濾波器的權重、一次使用很多個濾波器，並把數十層這樣的層與非線性函數交錯疊起來。

卷積層是一組學習而來的濾波器

精確版。設輸入特徵圖為 $X \in \mathbb{R}^{C_{\text{in}} \times H \times W}$（通道、高、寬）。一個有 C_{out} 個 $k \times k$ 濾波器的卷積層計算

$$Y_o(x,y) = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{i=-r}^{r} \sum_{j=-r}^{r} W_{o,c}(i,j)\, X_c(x+i, y+j), \qquad o = 1, \dots, C_{\text{out}},$$

其中奇數 k 時 $r = (k-1)/2$，$W_{o,c}(i,j)$ 是第 o 個濾波器在輸入通道 c、位移 (i,j) 的權重，b_o 是偏差。嚴格來說這是**相關**（correlation，不翻轉卷積核），如卷積教學所述；由於權重是學出來的，

翻不翻轉沒有差別。之後接一個逐元素的非線性函數，例如 $\mathrm{ReLU}(z) = \max(0, z)$。如果沒有它，一疊卷積層會塌縮成一個大的線性濾波器 [1]。

有兩個性質讓這種層特別適合影像：

- **局部連接**（local connectivity）。每個輸出只依賴一個 $k \times k$ 鄰域，因為相鄰像素彼此高度相關，距離遠的像素則關係弱得多。
- **權重共享**（weight sharing）。同一個濾波器用在每一個位置。這讓卷積層具有**平移等變性**（translation equivariance：輸入平移，輸出也跟著同樣平移），也大幅減少參數量。

省下的量非常可觀。從 $224 \times 224 \times 3$ 的影像連到 64 個神經元的全連接層需要 $224 \cdot 224 \cdot 3 \cdot 64 + 64 \approx 960$ 萬個參數。從 3 個通道到 64 個通道的 3×3 卷積只需要 $3 \cdot 3 \cdot 3 \cdot 64 + 64 = 1{,}792$ 個，而且輸出的是 64 張**全解析度**的特徵圖，而不是 64 個數字。

下面的程式把 PyTorch 卷積層的兩個濾波器手動設成 Sobel 卷積核，讓這一層變成邊緣偵測器。訓練則會改由梯度下降來決定這些數字。

```
import torch, torch.nn as nn, torch.nn.functional as F
from skimage import data, img_as_float

x = torch.tensor(img_as_float(data.camera()), dtype=torch.float32)[None, None]  # (1, 1,
 ↪  512, 512)
conv = nn.Conv2d(1, 2, kernel_size=3, padding=1, bias=False)
sobel_x = torch.tensor([[-1., 0., 1.], [-2., 0., 2.], [-1., 0., 1.]])
with torch.no_grad():
    conv.weight[0, 0] = sobel_x        # 通道 0：對垂直邊緣有反應
    conv.weight[1, 0] = sobel_x.T      # 通道 1：對水平邊緣有反應
    y = F.relu(conv(x))                # (1, 2, 512, 512)：兩張特徵圖
```

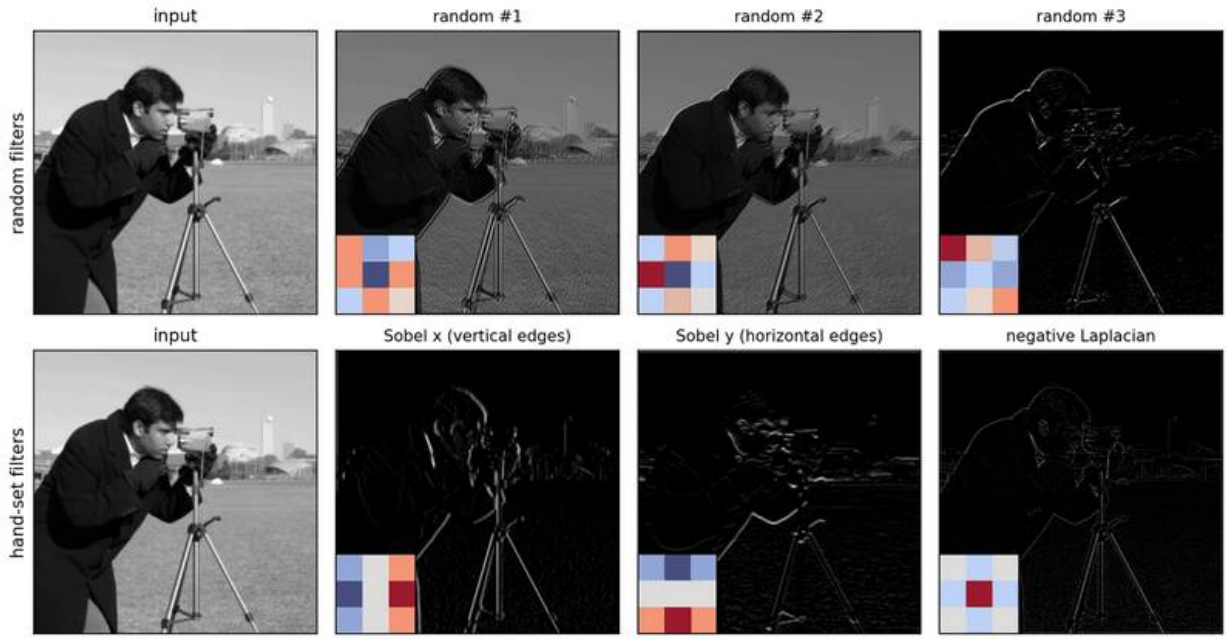


圖 *3* — 攝影師影像經 *3×3* 卷積（並經過 *ReLU*）的特徵圖。上：隨機初始權重，得到平滑與邊緣混雜的結果。下：手動設定的 *Sobel* 與 *Laplacian* 權重。訓練會把隨機濾波器推向有用的濾波器。

訓練好的第一層濾波器長什麼樣子？我們可以用 scikit-learn 內建的 8×8 手寫數字 [36]，在 CPU 上幾秒鐘就訓練完一個極小的 CNN。

```
import torch, torch.nn as nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0)
X, y = load_digits(return_X_y=True)                    # 1,797 張 8x8 影像
X = torch.tensor(X / 16.0, dtype=torch.float32).reshape(-1, 1, 8, 8)
Xtr, Xte, ytr, yte = train_test_split(X, torch.tensor(y), test_size=0.25, random_state=0)

net = nn.Sequential(
    nn.Conv2d(1, 8, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),     # 8x8 -> 4x4
    nn.Conv2d(8, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),    # 4x4 -> 2x2
    nn.Flatten(), nn.Linear(16 * 2 * 2, 10),
)
opt = torch.optim.Adam(net.parameters(), lr=1e-2)
for epoch in range(30):
    for i in range(0, len(Xtr), 64):
        loss = nn.functional.cross_entropy(net(Xtr[i:i + 64]), ytr[i:i + 64])
        opt.zero_grad(); loss.backward(); opt.step()
print((net(Xte).argmax(1) == yte).float().mean())       # 約 0.97
filters = net[0].weight.detach()[:, 0]                   # (8, 3, 3) 學到的卷積核
```

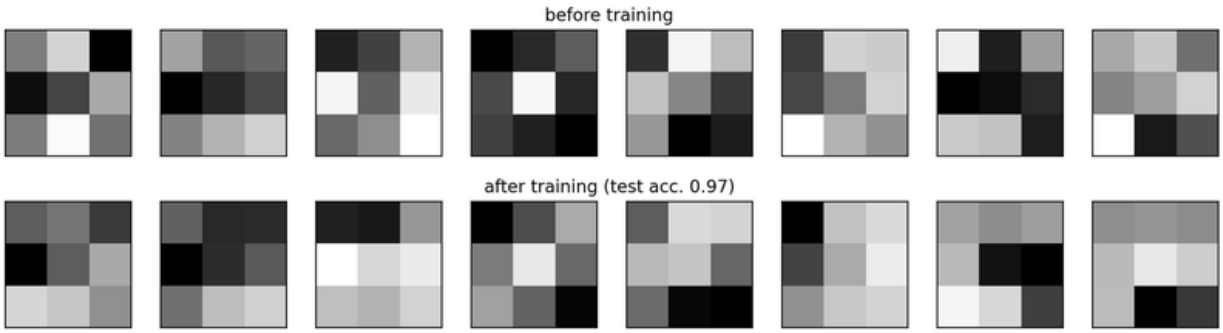


圖 *4* — 小型數字 *CNN* 的八個第一層 *3×3* 濾波器，訓練前（上）與訓練後（下）。其中幾個朝有方向性的明暗轉換移動，也就是人工設計者會挑選的類微分圖樣；沒有人告訴網路要這麼做。（*8×8* 的小輸入與 *3×3* 卷積核讓效果較不明顯；在自然影像上訓練的大型網路會更清楚地呈現這個現象。）

步幅、填補與輸出大小

白話版。填補（padding）在影像外圍加一圈邊框，讓視窗可以停在邊緣像素上。步幅（stride）讓視窗一次跳不只一個像素，因此輸出會變小。

精確版。對寬度為 W 的輸入、卷積核大小 k、每邊填補 p 個像素、步幅 s，輸出寬度為

$$W_{\text{out}} = \left\lfloor \frac{W + 2p - k}{s} \right\rfloor + 1.$$

$k = 3$、$p = 1$、$s = 1$ 時大小不變（“same” 填補）；$s = 2$ 時減半。預設是補零；但在低階網路中，常偏好鏡射或複製邊界的填補（與IP 03 討論的邊界處理相同），因為補零會在影像邊緣產生暗色瑕疵。

池化

池化（pooling）用一個數字概括每個小視窗，通常取最大值（max pooling）或平均值（average pooling），一般使用 2×2 視窗、步幅 2。它把解析度減半、讓表示對小幅平移較不敏感，並以很低的成本擴大感受野。許多現代網路改用步幅卷積取代池化，讓網路自己學會怎麼降採樣。在分類網路的最後，**全域平均池化**（global average pooling）把 $C \times H \times W$ 的特徵圖變成給分類 head 用的 C 維向量。

感受野

白話版。一個神經元的感受野，是輸入影像中能影響它的那一塊區域。一層 3×3 看得到 3×3 個像素；疊兩層，每個輸出看得到 5×5。降採樣會讓它長得快得多。

精確版。將各層編號為 $\ell = 1, \dots, L$，卷積核大小 k_ℓ、步幅 s_ℓ。令 j_ℓ 為**跳距**（jump），也就是第 ℓ 層兩個相鄰單元在輸入影像上相距多少像素；r_ℓ 為感受野大小；輸入層 $r_0 = 1$、$j_0 = 1$。則

$$r_\ell = r_{\ell-1} + (k_\ell - 1)\, j_{\ell-1}, \qquad j_\ell = j_{\ell-1}\, s_\ell.$$

每一層增加 $k_\ell - 1$ 個輸入單元，而每個輸入單元相當於 $j_{\ell-1}$ 個像素。若處處步幅為 1，L 層 3×3 得到 $r_L = 2L + 1$：線性成長。每一次步幅 2 都讓跳距加倍，後面的層讓感受野長得兩倍快，因此隨深度大致呈指數成長。

```
def receptive_field(layers):
    r, j = 1, 1                       # 感受野與跳距，以輸入像素為單位
    for k, s in layers:               # 每一層的（卷積核大小, 步幅）
        r, j = r + (k - 1) * j, j * s
    return r

print(receptive_field([(3, 1)] * 3))                                          # 7
print(receptive_field([(3, 1), (3, 1), (2, 2), (3, 1), (3, 1), (2, 2), (3, 1)])) # 24
```

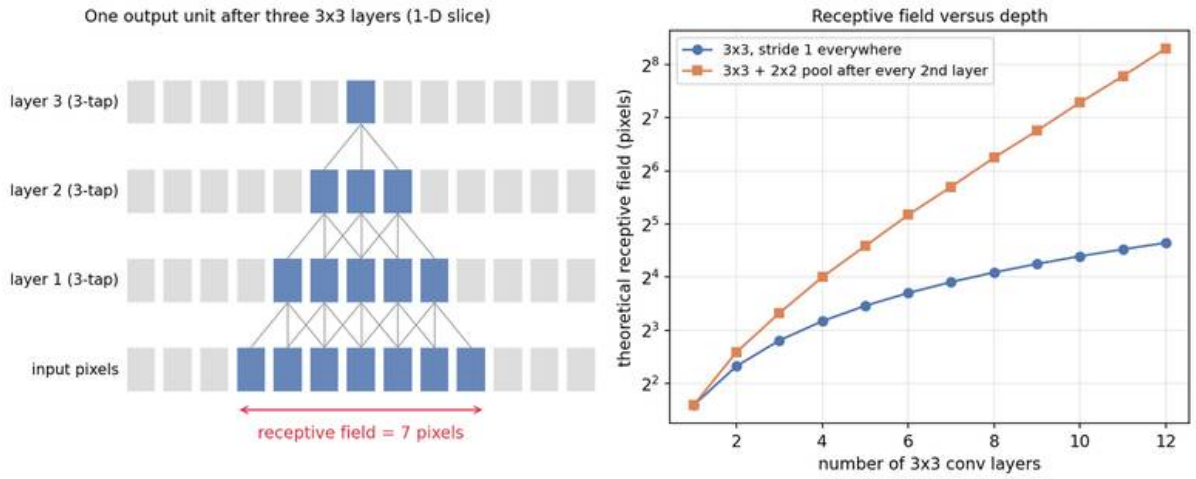


圖 *5* — 左：疊三層 *3* 抽頭的層，讓一個輸出擁有 *7* 個像素的感受野。右：理論感受野對深度的關係，比較處處步幅 *1* 與每兩層之後做一次 *2* 倍降採樣。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

實務上還有兩點要注意。第一，上面的公式給的是**理論**感受野。Luo 等人 [12] 發現，以「每個輸入像素對輸出梯度實際影響多少」來量測的**有效**感受野（effective receptive field）大致呈高斯分佈，只佔理論感受野的一小部分，中心附近的像素佔主導地位。第二，感受野決定網路**能**用到什麼：感受野比雜訊相關長度還小的去雜訊器，無法去除那種雜訊；最後一層只看得到物體一部分的分類器，也無法從上下文辨識它。

特徵階層

由於感受野隨深度變大、解析度變小，CNN 會建立一個**特徵階層**：前面的層對邊緣與色塊有反應（圖 4），中間的層對紋理與部件有反應，後面的層則對大範圍的物體層級圖樣有反應。典型的骨幹網路分成好幾個**階段**（stage），每個階段結束時做 2 倍降採樣並把通道數加倍，所以特徵從「高解析度、通道少、局部」一路變成「低解析度、通道多、語意」。高階任務讀取後面的階段；密集預測與低階任務還需要前面高解析度的階段，這正是編碼器–解碼器存在的原因（見下文）。

經典 CNN 架構

白話版。 2012 到 2016 年間，一系列一個比一個深的網路贏得了 ImageNet 大規模視覺辨識挑戰賽（ILSVRC）[6]。每一個都帶來一個至今仍在使用的設計想法。

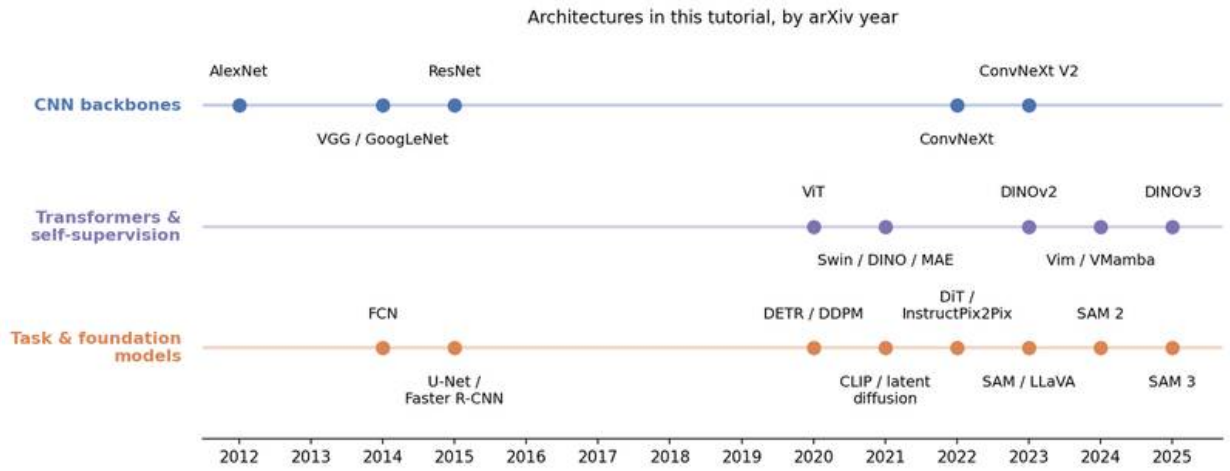


圖 *6* — 本篇提到的架構時間軸（依 *arXiv* 年份）。*LeNet-5*（*1998*）與 *1980* 年代末更早的卷積網路位於這條軸的左側之外。

LeNet-5，以及 CNN 真正的起點

原文的標題寫著「CNN（1998~）」。1998 年是 LeCun、Bottou、Bengio 與 Haffner 發表 LeNet-5 [5] 的年份，並不是 CNN 的起點：以反向傳播訓練的卷積網路，早在 1989 年就已經用來辨識手寫郵遞區號 [4]。

LeNet-5 是一個以網路形式呈現的標準圖形識別系統 [1]。C1 到 C5 是**特徵擷取器**：卷積層（C1、C3、C5）與子取樣層（S2、S4）交錯，把 32×32 的數字影像變成 120 維的特徵向量；F6 與輸出層則是**分類器**。包含特徵在內的所有東西，都以分類損失透過梯度下降訓練。端對端的想法其實早已到位，只是還需要再等十五年的資料量與運算力，才成為主流。

AlexNet（2012）

AlexNet [7] 有五個卷積層與三個全連接層（共八個可學習的層，約 6,000 萬個參數）。它的元件個別來看並不新，新的是組合：訓練速度遠快於飽和型 sigmoid 的 ReLU 激活函數、高效率的 GPU 實作、大量的資料擴增，以及全連接層中的 dropout。它以很大的差距贏得 ILSVRC 2012，這也是為什麼 2012 年通常被視為視覺深度學習時代的起點。

VGG（2014）

Simonyan 與 Zisserman [8] 只問了一個問題：如果只用 3×3 卷積，並把網路加深，會怎樣？他們最好的配置有 16 與 19 個權重層。3×3 之所以足夠，答案就在上面的感受野公式：兩層 3×3 看得到 5×5，三層看得到 7×7。輸入、輸出都是 C 個通道時，三層 3×3 需要 $3 \cdot 9C^2 = 27C^2$ 個權重，而一層 7×7 需要 $49C^2$ 個，而且前者還多插入了兩個非線性函數。VGG 整齊一致的設計，讓它多年來都是預設的預訓練特徵擷取器。

GoogLeNet / Inception（2014）

22 層深的 GoogLeNet [9] 贏得 ILSVRC 2014。它的 **Inception 模組**平行執行 1×1、3×3、5×5 卷積與一個池化分支，再把輸出串接起來，讓每個模組同時看到好幾種尺度。為了負擔得起，便宜的 1×1 卷積會先減少通道數。1×1 卷積在每個像素上混合通道而不看鄰居，相當於逐像素的全連接層，至今仍是到處可見的基本元件。

ResNet（2015），以及殘差連接為什麼有效

照理說越深越好，但實際上並非如此：He 等人 [10] 觀察到，一個普通的 56 層網路，**訓練誤差**反而比 20 層的高。這不是過擬合（overfitting），而是最佳化失敗：較深的網路理論上可以複製淺網路、再把多出來的層設成恆等映射，但梯度下降找不到那個解。

解法是**殘差區塊**（residual block，圖 7）。與其讓一疊層直接學習映射 $H(\mathbf{x})$，不如讓它學殘差 $F(\mathbf{x})=H(\mathbf{x})-\mathbf{x}$，再把輸入加回來：

$$\mathbf{y}=\sigma\big(\mathbf{x}+F(\mathbf{x};W)\big),$$

其中 $\mathbf{x}$ 是區塊輸入，F 通常是 conv–BN–ReLU–conv–BN（BN 為批次正規化，batch normalization），權重為 W，σ 為 ReLU。當通道數或解析度改變時，捷徑改用 1×1 卷積取代恆等映射。

為什麼有效：

- **恆等映射變得容易**。如果多出來的深度沒有用處，區塊只需要把 F 推向零，這比讓一疊非線性層模仿恆等映射容易得多。把 F 最後一個 BN 的縮放參數初始化為零，每個區塊一開始就是恆等映射。
- **梯度有一條高速公路**。忽略最後的非線性，$\partial\mathbf{y}/\partial\mathbf{x}=I+\partial F/\partial\mathbf{x}$。恆等項把梯度從損失直接帶到前面的層，不必穿過每一個權重矩陣，所以不會隨深度消失。
- **集成的觀點**。把 L 個區塊展開，會得到許多長短不一的路徑之和，因此一個很深的 ResNet 在某種程度上就像許多較淺網路的集合。

```
class ResidualBlock(nn.Module):
    """ 基本區塊：y = ReLU(x + F(x))，F = conv-BN-ReLU-conv-BN。"""
    def __init__(self, c):
        super().__init__()
        self.f = nn.Sequential(
            nn.Conv2d(c, c, 3, padding=1, bias=False), nn.BatchNorm2d(c), nn.ReLU(),
            nn.Conv2d(c, c, 3, padding=1, bias=False), nn.BatchNorm2d(c),
        )
    def forward(self, x):
        return torch.relu(x + self.f(x))

blk = ResidualBlock(16)
nn.init.zeros_(blk.f[4].weight)                  # 把最後一個 BN 的縮放設為零，使 F(x) = 0
x = torch.randn(2, 16, 32, 32)
print(torch.allclose(blk(x), torch.relu(x)))     # True：區塊一開始就是恆等映射（再接 ReLU）
```

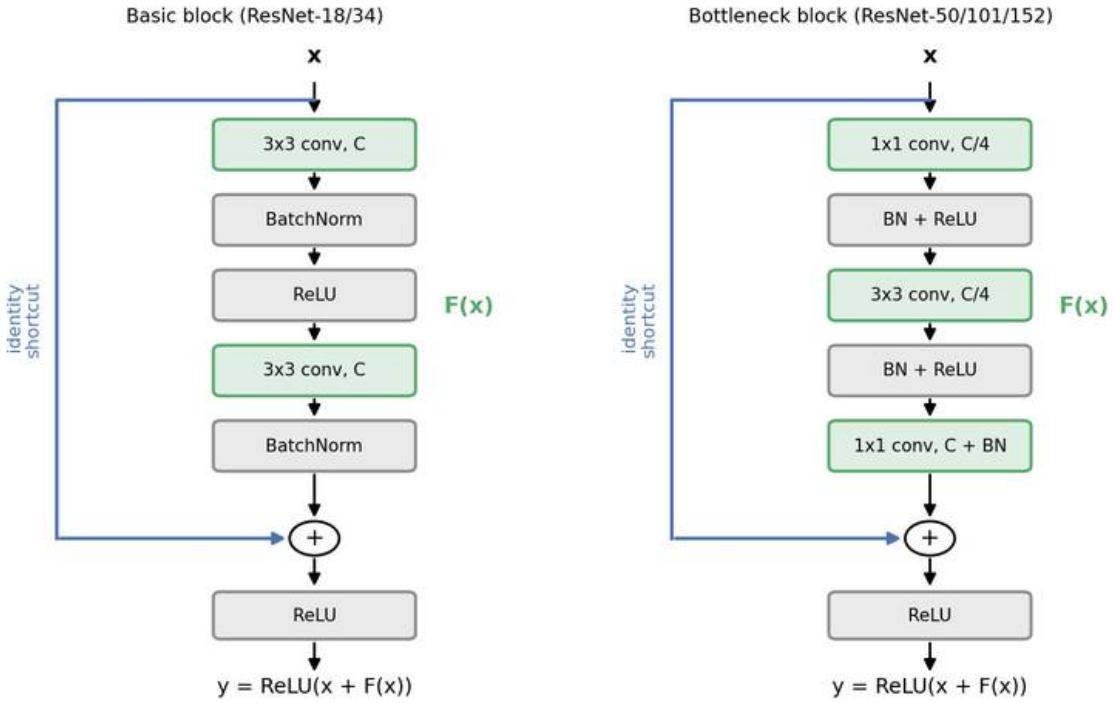


圖 *7* — 左：基本殘差區塊。右：*ResNet-50/101/152* 使用的瓶頸區塊，以 *1×1* 卷積在便宜的 *3×3* 卷積前後縮減並恢復通道數。

論文在 ImageNet 上訓練了 **18、34、50、101 與 152 層**的 ResNet，其中較深的三種使用瓶頸區塊。有名的 **1202 層**網路只出現在論文的 CIFAR-10 實驗中 [10]、[11]（32×32 的小影像）；它可以順利訓練，但過擬合，測試結果反而比 110 層的版本差。ResNet 家族贏得 ILSVRC 2015，而殘差連接如今幾乎存在於所有深度網路中，Transformer 也不例外。

用於密集預測的編碼器–解碼器

白話版。分類骨幹為了換取語意而丟掉解析度。分割或復原需要在每個像素上都給出答案，所以要再加上後半段，把解析度找回來。

精確版。 Long 等人提出的**全卷積網路**（fully convolutional network, FCN）[13] 把分類器中的全連接層換成卷積，讓網路對任意大小的輸入輸出一張粗略的類別分數圖，再以學習而來的轉置卷積上採樣，並與前面較精細層的分數融合。**U-Net** [14] 把這個想法做成對稱的形式：收縮的**編碼器**（卷積區塊與降採樣）與擴張的**解碼器**（上採樣與卷積區塊），中間以**跳接**（skip connection）把每個編碼器階段的特徵圖串接到同一解析度的解碼器階段。編碼器提供「是什麼」，跳接提供「在哪裡」。U-Net 原本是為標註很少、需要大量資料擴增的生醫影像設計的；同樣的形狀如今撐起了醫學影像分割、DLCV 04 介紹的幾乎所有復原網路，以及許多擴散模型內部的去雜訊器。

Vision Transformer

白話版。 Transformer [15] 原本是為句子設計的：它把輸入當成一串 token，讓每個 token 都能看到其他所有 token。Vision Transformer（ViT）[16] 把影像切成小方塊（patch），把每個 patch 當成一個「字」，再把這串序列送進一個沒有修改過的 Transformer 編碼器。

切塊、嵌入、注意力

精確版。對影像 $I \in \mathbb{R}^{C\times H\times W}$ 與 patch 大小 P：

1. **切塊**。把 I 切成 $N = HW/P^2$ 個不重疊的 patch，並把每一塊攤平成向量 $\mathbf{p}_i \in \mathbb{R}^{CP^2}$。對 224×224 的 RGB 影像與 $P = 16$，$N = 196$，每個向量有 768 個元素。
2. **嵌入**。把每個 patch 線性映射成 D 維 token，$\mathbf{z}_i = E\,\mathbf{p}_i$，$E \in \mathbb{R}^{D\times CP^2}$。在最前面加上一個可學習的類別 token $\mathbf{z}_{\text{cls}}$，並加上可學習的**位置嵌入**（position embedding）$\mathbf{e}_i$，因為注意力本身不知道 token 來自哪裡：$Z^{(0)} = [\mathbf{z}_{\text{cls}}; \mathbf{z}_1; \dots; \mathbf{z}_N] + [\mathbf{e}_0; \dots; \mathbf{e}_N]$。
3. **編碼**。套用 L 個 Transformer 編碼器區塊，每個區塊包含多頭自注意力（MSA）子層與 MLP 子層，兩者都搭配層正規化（LN）與殘差連接：

$$
\begin{aligned}
Z' &= Z^{(\ell-1)} + \mathrm{MSA}(\mathrm{LN}(Z^{(\ell-1)})),\\
Z^{(\ell)} &= Z' + \mathrm{MLP}(\mathrm{LN}(Z')).
\end{aligned}
$$

4. **Head**。把最後的類別 token（或所有 token 的平均）送進 MLP head，它扮演的就是圖形識別中分類器的角色 [1]。

對 token 矩陣 $Z \in \mathbb{R}^{(N+1)\times D}$，單一注意力頭計算

$$
\mathrm{Attn}(Z) = \mathrm{softmax}\!\left(\frac{QK^{\top}}{\sqrt{d}}\right)V, \qquad Q = ZW_Q,\; K = ZW_K,\; V = ZW_V,
$$

其中 $W_Q, W_K, W_V \in \mathbb{R}^{D\times d}$ 是學習而來的投影，d 是每個頭的維度。softmax 矩陣的第 i 列表示 token i 從其他每個 token 汲取多少資訊。和卷積相比，這裡的「濾波器」是依內容計算出來的，而且從第一層起就涵蓋整張影像。代價是計算量隨 token 數呈平方成長，$O(N^2 d)$，這也是為什麼使用 patch 而不是像素。

```
img = torch.randn(1, 3, 224, 224)
P, D = 16, 768
patches = img.unfold(2, P, P).unfold(3, P, P)                    # (1, 3, 14, 14, 16, 16)
patches = patches.permute(0, 2, 3, 1, 4, 5).reshape(1, -1, 3 * P * P)
print(patches.shape)                                              # (1, 196, 768)
embed = nn.Conv2d(3, D, kernel_size=P, stride=P)                  # 切塊＋線性嵌入，一步完成
tokens = embed(img).flatten(2).transpose(1, 2)                    # (1, 196, 768)
```

最後兩行說明了一個實用的等式：patch 嵌入**就是**一個卷積核大小與步幅都等於 P 的卷積。

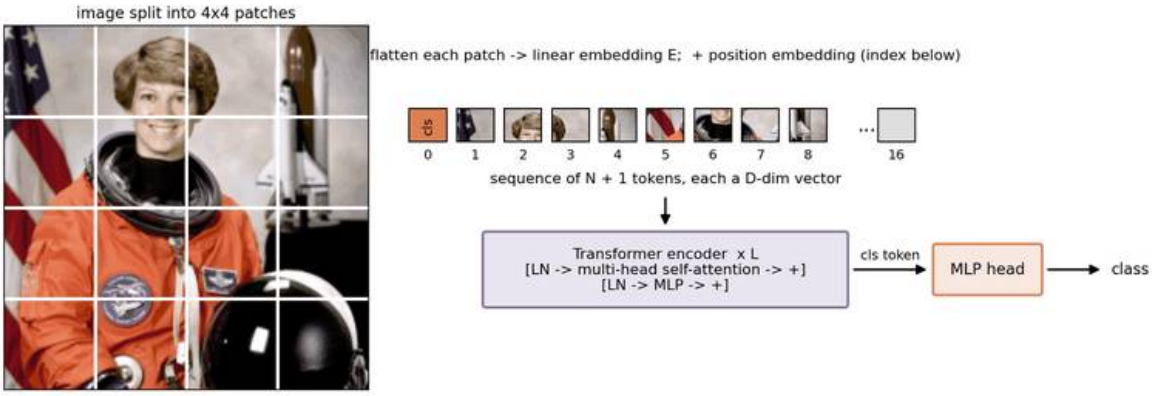


圖 *8* — 一張圖看懂 *ViT*：切塊、線性嵌入、加上位置嵌入與類別 *token*、經過 *Transformer* 編碼器，再由類別 *token* 做分類。（為了清楚起見只畫 *4×4*；*ViT-B/16* 在 *224×224* 輸入上使用 *14×14* 個 *patch*。）

歸納偏置與規模的需求

原文提到 ViT「更難收斂」。更精確的說法 [1]、[16] 與**歸納偏置**（inductive bias）有關，也就是架構在看到資料之前就內建的假設。CNN 假設局部性與權重共享，所以它已經「知道」相鄰像素要一起看，也知道同一個圖樣出現在影像中任何位置都代表同一件事。ViT 幾乎沒有任何假設：除了切 patch 之外，所有空間結構，甚至位置嵌入的二維排列，都必須自己學。

這是一種取捨，而不是缺陷。資料少的時候，CNN 的假設是寶貴的先發優勢，只在 ImageNet-1k 上從頭訓練的 ViT 表現不如同等的 ResNet。資料夠多時，這些假設反而成為限制；在非常大的資料集上預訓練的 ViT，微調後可以追平甚至超越最好的 CNN [16]。實務上的結論是：ViT 都是**預訓練後**才使用，預訓練資料可能是巨大的標註資料集，也越來越常是自監督學習。

ViT 之後的骨幹網路

白話版。 ViT 之後，研究朝三個方向走：讓 Transformer 更適合影像（Swin）、證明現代化之後的 CNN 也能追上（ConvNeXt），以及在沒有標註的情況下預訓練，讓一個骨幹就能服務所有任務（DINO、MAE、DINOv2/v3）。

Swin Transformer：局部性回歸

Swin [17] 只在不重疊的局部視窗（例如 7×7 個 token）內計算注意力，讓計算量對影像大小呈線性。在交替的區塊中，視窗格線會**位移**半個視窗，讓資訊跨越視窗邊界。在階段之間，相鄰的 token 會被合併，使解析度減半。結果是一個階層式、多尺度的特徵金字塔，形狀就像 CNN 骨幹，正好是偵測與分割 head 所需要的。Swin 把 CNN 的歸納偏置，也就是局部性與階層性，放回了 Transformer 裡。

ConvNeXt：CNN 的反擊

Liu 等人 [18] 從 ResNet-50 出發，一步一步採用 Transformer 的訓練方法與設計選擇來將它現代化：「切塊式」的 stem（步幅 4 的 4×4 卷積）、大的 7×7 逐通道卷積（depthwise convolution）、倒置瓶頸的 MLP、較少的激活與正規化層，以及層正規化。得到的純 CNN，**ConvNeXt**，在分類、偵測與分割上的準確度與擴展性都與 Swin 打平。結論是：ViT 的優勢有很大一部分來自訓練方法與宏觀設計，而不是注意力本身。**ConvNeXt V2** [18] 接著加入了為卷積改寫的遮罩自編碼器預訓練（全卷積遮罩自編碼器，FCMAE）以及全域響應正規化（global response normalization）層。如今大型基礎模型多以 ViT 為骨幹，但在低階任務與邊緣裝置上，卷積仍然很常見 [1]。

自監督預訓練：DINO 與 MAE

自監督學習（self-supervised learning）從影像本身產生訓練訊號，因此不需要標註。主流的作法有兩種，而且常常被混為一談。

DINO：自蒸餾（self-distillation）[19]。學生網路 g_s 與教師網路 g_t 架構相同，各自輸出一個 K 維的機率向量。對一張影像做好幾個隨機裁切：兩個大的「全域」視圖與數個小的「局部」視圖。教師只看全域視圖，學生看所有視圖，並被訓練去符合教師的輸出：

$$\min_{\theta_s} \sum_{x\in\{x_1^g,x_2^g\}} \sum_{\substack{x'\in V\\ x'\neq x}} H(P_t(x),\,P_s(x')), \qquad H(a,b) = -a^\top \log b,$$

其中 V 是所有視圖的集合，P_t、P_s 是教師與學生的 softmax 輸出，θ_s 是學生的權重。教師不經由梯度訓練，它的權重是學生權重的指數移動平均，$\theta_t \leftarrow \lambda\theta_t + (1-\lambda)\theta_s$，$\lambda$ 接近 1；教師的輸出還會經過置中（centering）與銳化（sharpening），避免訓練塌縮成常數輸出。讓「小裁切」去符合「整張圖」，會迫使網路從局部辨識出物體。一個令人驚訝的湧現性質是：DINO ViT 的注意力圖在從未看過任何遮罩的情況下，就能分割出主要物體。

MAE：遮罩預測（masked prediction）[20]。遮罩自編碼器（masked autoencoder）隨機遮住大部分的 patch（75 % 效果最好），只用 ViT 編碼看得見的 patch，再讓一個輕量解碼器重建被遮住的像素：

$$\mathcal{L}_{\text{MAE}} = \frac{1}{|\mathcal{M}|}\sum_{i\in\mathcal{M}} \|\hat{\mathbf{p}}_i - \mathbf{p}_i\|_2^2,$$

其中 $\mathcal{M}$ 是被遮住的 patch 索引集合，$\mathbf{p}_i$ 是 patch i（正規化後）的像素，$\hat{\mathbf{p}}_i$ 是重建結果。這種「遮住再預測」的遊戲，相當於語言模型中遮字預測的影像版本；相對地，DINO 是教師–學生蒸餾法 [1]。MAE 的編碼器只處理四分之一的 token，所以很省計算，微調效果也非常好；DINO 類的特徵則更適合**凍結**使用，直接接一個線性 head。

DINOv2 與 DINOv3：凍結即用的通用特徵

DINOv2 [21] 把 DINO 式的自蒸餾擴展到 10 億參數的 ViT，在一個自動**篩選**（curated）過的大型影像集合上訓練，再蒸餾成較小的模型。它凍結後的特徵，只需要一個輕量 head，就能同時勝任影像層級任務（分類、檢索）與像素層級任務（分割、深度估計）。**DINOv3** [21] 進一步擴大資

料與模型規模，並提出 **Gram anchoring**，這是一種正則化方法，能防止密集的 patch 特徵在非常長的訓練過程中退化，讓同一個凍結骨幹提供高品質的密集特徵。

這讓我們回到了圖形識別流程。「特徵擷取器＋分類器」以**預訓練凍結骨幹＋輕量 head** 的形式回來了，只是特徵擷取器現在是在極大規模、沒有標註的情況下學一次就好。

狀態空間骨幹

Vision Mamba（Vim）與 **VMamba** [22] 以選擇性狀態空間模型（selective state-space model）取代注意力，用與序列長度呈線性的成本掃描 token 序列。影像不是序列，所以這些模型會沿不只一條路徑掃描 patch 網格（Vim 是雙向的；VMamba 的二維選擇性掃描會走好幾條路線）。它們對高解析度輸入很有吸引力，但目前仍偏向研究路線，還不是預設的骨幹。

高階視覺任務

白話版。高階任務的差別在於答案要多精確地指出**位置**：整張圖一個標籤、每個物體一個方框，或是每個像素一個標籤。

任務	輸出	典型損失	典型指標	代表模型
分類	一個標籤（或機率向量）	交叉熵	top-1 / top-5 正確率	ResNet、ViT、ConvNeXt [10]、[16]、[18]
物件偵測	一組（方框、類別、分數）	分類＋方框迴歸，或集合匹配	多個 IoU 門檻下的平均精確度（AP）	Faster R-CNN、DETR [23]
語意分割	每個像素一個類別	逐像素交叉熵、Dice（重疊損失，見下文）	mean IoU	FCN、U-Net [13]、[14]
實例分割	每個物體一個遮罩與類別	偵測＋遮罩損失	mask AP	可提示式：SAM 系列 [24]

分類

為整張影像指定一個標籤。網路是骨幹＋全域池化＋線性層，以交叉熵 $\mathcal{L} = -\log \hat{p}_y$ 訓練，其中 $\hat{p}_y$ 是真實類別 y 的預測機率。在自監督學習出現之前，分類也是產生大多數骨幹網路的**預訓練任務**。DLCV 02 會深入介紹。

物件偵測

找出每個物體、它的類別，以及它的位置；位置以軸對齊的方框表示（感興趣區域，ROI，由角點座標或中心與長寬決定）[1]。有兩條設計路線可以說明這個領域 [23]。**Faster R-CNN** 是兩階段式：區域提議網路（region proposal network）從共享的 CNN 特徵中提出候選方框，第二個 head 再對每個候選分類並微調；重複的框由非極大值抑制（non-maximum suppression）移除。**DETR** 把偵測視為**集合預測**：Transformer 解碼器輸出固定數量的預測，再以預測與真實物體之間的二

分匹配定義損失，省去了人工設計的錨框與非極大值抑制。方框品質以交併比（intersection over union）衡量：

$$\mathrm{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|},$$

其中 A、B 分別是預測與真實區域。與它密切相關的 Dice 係數 $2|A \cap B|/(|A| + |B|)$，以可微分的軟性形式作為分割損失。平均精確度則在一個或多個 IoU 門檻下概括精確率–召回率曲線。細節請見DLCV 03。

語意分割與實例分割

這裡的感興趣區域是物體確切的像素集合，而不是方框 [1]。

- **語意分割**（semantic segmentation）為每個像素標上類別。兩個重疊的人會變成同一塊「人」的區域。
- **實例分割**（instance segmentation）還要把同一類別的不同物體分開。兩個人會變成「人 1」與「人 2」。

語意分割是上面編碼器–解碼器 [13]、[14] 的本行。實例分割則再加上一個類似偵測的元件，為每個物體提出一個遮罩。

可提示的分割：SAM、SAM 2、SAM 3

分割領域如今有了**可提示的基礎模型** [24]。**SAM**（Segment Anything）接收一張影像與一個提示（點、方框或粗略遮罩），回傳被指定物體的遮罩。笨重的 ViT 影像編碼器每張影像只跑一次，輕量的提示編碼器與遮罩解碼器則每個提示跑一次，所以互動很快。它在 SA-1B 上訓練；SA-1B 包含 1,100 萬張影像上超過 10 億個遮罩，是以「模型在迴圈中」的資料引擎建立的。**SAM 2** 把這個能力擴展到影片，以串流記憶（streaming memory）把物體一幀一幀帶下去，並在新的影片資料集（SA-V）上訓練。**SAM 3** 加入了**可提示的概念分割**（promptable concept segmentation）：給它一個短名詞片語，例如「黃色校車」，或是影像範例，或兩者並用，它會找出、分割並（在影片中）追蹤**所有**符合的實例，而不只是點擊指向的那一個物體。到了 SAM 3，偵測、實例分割與追蹤之間的界線已經變得很薄；追蹤請見DLCV 06。

低階視覺任務

白話版。低階任務輸入一張影像，回傳同一個場景經過改善或轉換的影像。輸出以逐像素的方式評判，或以看起來有多自然來評判。

一般的形式是一個反問題。劣化的觀測 $\mathbf{y}$ 被建模為

$$\mathbf{y} = \mathcal{D}(\mathbf{x}) + \mathbf{n},$$

其中 $\mathbf{x}$ 是乾淨影像，$\mathcal{D}$ 是劣化運算子（模糊、降採樣、霧、曝光不足），$\mathbf{n}$ 是雜訊，與IP 05 的模型相同。網路 f_θ 在成對資料上訓練，最小化例如 $\|f_\theta(\mathbf{y}) - \mathbf{x}\|_1$。由於真實的成對資料很少，通常會

對乾淨影像套用一個建模好的劣化來**合成**成對資料，而合成劣化與真實劣化之間的落差，正是這個領域的核心難題。

- **影像復原**（image restoration）找回乾淨影像，又可稱為「清晰化」：去雜訊、去模糊、超解析度（小圖變大圖）、除霧（霧濛濛的影像變清楚）、去雨。架構是全解析度的編碼器–解碼器（類 U-Net），或不降採樣的殘差網路，現在也常加入視窗注意力。請見DLCV 04。
- **影像增強**（image enhancement）讓內容更容易看清楚，但沒有唯一的「正確答案」：低光源增強（讓暗影像中的細節變清楚，某種程度上是聰明的調亮；如果是把黑夜變白天，就屬於轉譯了）與水下影像增強。請見DLCV 05。
- **影像對齊**（image alignment，或稱影像配準）估計把一張影像對應到另一張影像的幾何轉換或稠密位移場，是連拍融合、HDR 與醫學影像比對的前置步驟。
- **影像到影像轉換**（image-to-image translation）把影像從一個領域或風格轉到另一個：白天變黑夜、夏天變冬天、草圖變照片。換臉的「深偽」（deepfake）就是同一個想法的（被濫用的）延伸應用。自從擴散模型出現後，轉換與編輯常改由一個以輸入影像為條件的生成模型完成，見下一節。

生成與視覺–語言模型

白話版。有兩類較新的模型不太能放進舊地圖：一類從文字**創造**影像，另一類能**談論**影像。兩者都依賴大規模預訓練，以及把影像和語言連結起來。

擴散模型：從雜訊到影像

去雜訊擴散機率模型（denoising diffusion probabilistic model, DDPM）[27] 定義了一個前向過程，在 T 步內逐漸對影像 $\mathbf{x}_0$ 加入高斯雜訊。寫成封閉形式為

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, I),$$

其中 $t \in \{1, \dots, T\}$ 是步數，$\bar{\alpha}_t \in (0,1)$ 是固定的排程，從接近 1 遞減到接近 0。網路 $\boldsymbol{\epsilon}_\theta$ 被訓練來預測加入的雜訊：

$$\mathcal{L} = \mathbb{E}_{\mathbf{x}_0, t, \boldsymbol{\epsilon}} \| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \|_2^2,$$

生成時則反向執行這個過程：從純雜訊開始，一步一步去雜訊。換句話說，生成器就是**反覆套用許多次的學習式去雜訊器**，這讓生成直接和上面的復原問題連在一起。

- **潛在擴散**（latent diffusion）[28] 不在像素上、而是在預訓練自編碼器的壓縮潛在空間中進行擴散，大幅降低成本，使高解析度的文字生成影像變得可行；文字條件透過交叉注意力（cross-attention）加入。它是 Stable Diffusion 的基礎。
- **DiT**（Diffusion Transformer）[28] 把 U-Net 去雜訊器換成作用在潛在 patch 上的 Transformer，並顯示品質會隨模型計算量穩定提升。
- **InstructPix2Pix** [29] 依照文字指令（「把它變成冬天」）編輯給定的影像，讓傳統的影像到影像轉換成為條件生成的一個特例。

視覺–語言模型

視覺問答（visual question answering, VQA），也就是針對一張影像回答自由形式的問題，以前需要串起影像特徵、影像描述（captioning）與文字問答的流程 [1]。兩個進展讓它變成單一模型。

CLIP [25] 以**對比式**（contrastive）目標，在 4 億組影像–文字對上訓練影像編碼器 f 與文字編碼器 g。在一個包含 B 組配對的批次中，令正規化後的嵌入為 $\mathbf{u}_i = f(I_i)$、$\mathbf{v}_j = g(T_j)$，溫度為 τ，影像到文字的損失為

$$\mathcal{L}_{I\to T} = -\frac{1}{B}\sum_{i=1}^{B} \log \frac{\exp(\mathbf{u}_i^\top \mathbf{v}_i/\tau)}{\sum_{j=1}^{B}\exp(\mathbf{u}_i^\top \mathbf{v}_j/\tau)},$$

總損失則與對稱的文字到影像項平均。配對的組合被拉近，批次中其他所有組合被推開。由於標籤空間變成了「任何一句話」，CLIP 可以做零樣本（zero-shot）分類：把「a photo of a cat」、「a photo of a dog」……等提示嵌入，再挑出與影像最接近的那一個。

LLaVA [26] 透過一個投影層，把 CLIP 視覺編碼器接上大型語言模型（LLM）；投影層把視覺 token 映射到 LLM 的詞嵌入空間，再以關於影像的指令跟隨對話進行微調。目前大多數多模態 LLM 都沿用這種**視覺編碼器＋投影層＋ LLM** 的架構，所以 VQA、影像描述與許多辨識任務，現在都可以透過對同一個模型下提示來完成。

現代觀點

貫穿本篇的趨勢，是從「一個任務一個模型」走向「一個基礎模型＋提示或輕量 head」。以下的綜述論文，是理解這個領域如何走到今天的好地圖。

CNN 架構。 Li 等人 [30] 回顧了 CNN 的歷史、卷積運算本身、經典與進階架構（LeNet 到 ResNet 一脈，以及輕量化與加入注意力的網路）、一維／二維／多維卷積與各種應用。他們也整理了每個元件的改進（層設計、激活與損失函數、正則化、最佳化、快速計算），透過實驗歸納出挑選函數與超參數的經驗法則，最後討論開放問題與有潛力的方向。若想一次看到 CNN 的設計空間被逐個元件攤開，這是最好的單一參考。

Vision Transformer。 Khan 等人 [31]（ACM Computing Surveys）依任務整理 Transformer 的研究：先是基礎（自注意力、大規模預訓練、雙向編碼），接著是辨識（分類、偵測、分割、動作辨識）、生成模型、多模態任務（VQA、視覺推理、視覺定位）、影片、低階視覺（超解析度、增強、上色）與三維點雲。他們的核心對比與本篇一致：Transformer 以極少的歸納偏置建模長距離依賴，因此需要大規模預訓練或架構上的先驗（視窗、階層）才能有好的資料效率。

自監督視覺學習。 Jing 與 Tian [32]（TPAMI）綜述了 2020 年以前那一代的自監督方法，依前置任務（pretext task）分為生成式、上下文式、免費語意標籤式與跨模態式四類，並說明以遷移到下游任務來評估特徵的標準流程。Balestriero 等人 [33] 則以實用的「食譜」形式涵蓋新一代方法，把它們分成四個家族：深度度量學習（對比式）、自蒸餾（DINO 家族）、典型相關分析類方法，以及遮罩影像建模（MAE）。他們強調許多隱藏的「旋鈕」（資料擴增、投影 head、防止塌縮的機制），這也解釋了為什麼自監督結果難以重現。

視覺基礎模型。 Awais 等人 [34]（TPAMI）依模態組合（視覺搭配文字、音訊、深度）、訓練目標（對比式 vs. 生成式）與**提示類型**（文字提示；視覺提示，例如 SAM 的點、框、遮罩；以及異

質混合提示）為視覺基礎模型分類。他們列出的開放挑戰包括評估與基準測試、真實世界理解與上下文推理、偏差、對抗強健性與可解釋性。Zhang 等人 [35]（TPAMI）聚焦於用於視覺辨識的視覺–語言模型：架構與預訓練目標、遷移方法（提示微調、特徵轉接器），以及把知識蒸餾到偵測器與分割器。他們主張網路規模的影像–文字資料降低了對群眾標註的依賴，並讓一個模型服務許多任務。

2024–2026 年的最新進展。不引用任何基準數字，大致的局面如下：

- **骨幹網路**。以自監督或影像–文字資料預訓練的大型 ViT 是高階任務的預設選擇，凍結的 DINOv2/DINOv3 特徵 [21] 是密集任務的強基準。現代 CNN（ConvNeXt V2 [18]）仍具競爭力，在延遲、記憶體或高解析度是主要考量時更受青睞。狀態空間骨幹 [22] 是活躍的替代路線。
- **分割與追蹤**已收斂到可提示模型：影片物體用 SAM 2，開放詞彙概念用 SAM 3 [24]。
- **生成**已轉向以 Transformer 為去雜訊器的潛在擴散 [28]，編輯則轉向以指令為條件的擴散 [29]。
- **理解**越來越多交給 LLaVA 架構的多模態 LLM [26]。
- **低階視覺**仍大量依賴在合成劣化上訓練的任務專用編碼器–解碼器，並越來越常結合生成式（擴散）先驗；見 DLCV 04 與 DLCV 05。

開放問題。 (1) 資料與評估：基準測試趨於飽和，也可能與網路規模的預訓練資料重疊，難以區分泛化與記憶 [34]。(2) 真實世界的劣化：在合成雜訊或模糊上訓練的模型，遇到真實相機流程仍會失敗。(3) 保真度與幻覺：生成式先驗產生銳利、但有時是捏造的細節，這在醫學或鑑識用途中無法接受。(4) 效率：注意力對 token 數呈平方成長，基礎模型也很難在裝置端執行。(5) 大規模下的密集特徵：DINOv3 之所以需要 Gram anchoring [21]，正是因為密集特徵品質可能隨訓練規模擴大而退化。(6) 以未篩選網路資料訓練的模型，其偏差、強健性與可解釋性 [34]。

重點整理

- 深度學習把傳統的三個層次縮減為兩類：**低階**（影像 → 影像，以保真度評判）與**高階**（影像 → 語意，以正確率評判）。中階分析則依輸出的形式歸入其中之一。
- **端對端學習**以學習而來的特徵取代人工設計的特徵；而分工以**預訓練骨幹＋輕量 head** 的形式在更大規模下回歸。
- 卷積層是一組具局部連接與權重共享的**學習式空間濾波器**。步幅、填補與池化決定輸出大小；**感受野**依 $r_\ell = r_{\ell-1} + (k_\ell - 1) j_{\ell-1}$ 成長，而有效感受野比理論值小。
- LeNet-5（1998）不是第一個 CNN；以反向傳播訓練的卷積網路可追溯到 1989 年。ResNet 在 ImageNet 上訓練的是 **18/34/50/101/152** 層；**1202** 層只出現在 CIFAR-10。殘差連接讓恆等映射變得容易，也給梯度一條直達的路徑。
- 帶跳接的編碼器–解碼器（**FCN**、**U-Net**）是密集預測與影像復原的標準形狀。
- **ViT** 的歸納偏置很少，因此需要大規模預訓練。Swin 把局部性加回去；ConvNeXt 則證明現代化的 CNN 可以與之打平。
- **DINO** 是教師–學生自蒸餾；**MAE** 是遮罩預測。DINOv2/v3 提供凍結即用的通用特徵。
- 各種任務越來越收斂到可提示或與語言連結的**基礎模型**：分割用 SAM 1/2/3，生成用潛在擴散與 DiT，視覺–語言用 CLIP 與 LLaVA。

練習

1. **手算感受野**。一個網路依序為：7×7 卷積、步幅 2；3×3 最大池化、步幅 2；接著四個 3×3、步幅 1 的卷積。計算每一層之後的理論感受野與跳距，並用 `receptive_field` 函式驗證。

   提示

   從 $r=1$、$j=1$ 開始。第一個卷積之後：$r=7$、$j=2$。池化之後：$r=7+2\cdot 2=11$、$j=4$。之後每一層 3×3 增加 $2\cdot 4=8$，所以 $r=19, 27, 35, 43$，$j=4$。

2. **參數預算**。輸入與輸出通道都是 $C=256$ 時，比較（忽略偏差）一層 5×5 卷積、兩層疊起來的 3×3 卷積，以及一個瓶頸結構（1×1 降到 64 通道、64 通道的 3×3、1×1 回到 256）的權重數。哪些的感受野相同？

   提示

   $25C^2\approx 164$ 萬；$2\cdot 9C^2\approx 118$ 萬；瓶頸 $256\cdot 64+9\cdot 64^2+64\cdot 256\approx 7$ 萬。前兩者都看得到 5×5；瓶頸只看得到 3×3，但便宜得多，這就是 ResNet-50 以上使用它的原因。

3. **沒有捷徑時，恆等映射為什麼很難**。在網路內部，區塊的輸入通常來自前一個 ReLU，因此是非負的。證明對這樣的輸入，$F\equiv 0$ 的殘差區塊輸出恰好是 $\mathbf{y}=\mathbf{x}$。接著說明普通區塊（conv–BN–ReLU–conv–BN–ReLU）要輸出 $\mathbf{y}=\mathbf{x}$ 必須學到什麼。這對非常深的普通網路代表什麼？

   提示

   對 $\mathbf{x}\geq 0$，$\mathrm{ReLU}(\mathbf{x}+0)=\mathbf{x}$。普通區塊則必須學到恰好抵消兩次卷積與兩次正規化的權重，這是一個精確又脆弱的解，梯度下降很少找得到。殘差區塊只需要 $F=0$，而權重衰減與零初始化本來就把它往那裡推。因此很深的普通網路很難讓多出來的層「什麼都不做」，這與 ResNet 作者觀察到的退化現象一致。

4. **ViT 的 token 數與計算量**。對 512×512 的輸入，計算 patch 大小為 32、16、8 時的 token 數，以及一層全域注意力的相對成本（$\propto N^2$）。Swin 使用 7×7 視窗的視窗注意力，會如何改變這個成長趨勢？

   提示

   $N=(512/P)^2$：256、1024、4096 個 token；成本比為 $1:16:256$。視窗注意力每層成本為 $N\cdot 49$（每個 token 只關注 49 個 token），所以隨 N 線性成長。

5. **學來的與手動設定的濾波器**。修改數字 CNN，讓第一層凍結為四個手動設定的卷積核（兩個 Sobel、一個 Laplacian、一個平均模糊）加上四個隨機卷積核，只訓練其餘部分。與完全學習的網路比較測試正確率；再解凍隨機卷積核，觀察它們會變成什麼樣子。

提示

設定好權重之後執行 `conv.weight.requires_grad_(False)`，並只把其餘參數交給最佳化器。在這個簡單的資料集上兩者正確率相近；重點在於手動設定的微分濾波器本來就是合理的第一層，而解凍後的隨機卷積核往往會朝有方向性、類微分的圖樣移動（可對照圖 4）。

6. **選擇配方**。你有 2,000 張標註過的顯微鏡影像，需要 (a) 每張影像的品質標籤、(b) 逐像素的細胞遮罩、(c) 去雜訊後的影像。根據本篇的地圖，為每一項提出骨幹、head 與訓練策略，並說明哪一項最能受益於凍結的自監督骨幹。

提示

(a) 凍結的 DINOv2/v3，或微調的 ConvNeXt ＋線性 head。(b) U-Net（可搭配預訓練編碼器），或以點提示 SAM 再微調。(c) 全解析度的復原網路，在符合該顯微鏡雜訊特性的合成雜訊/乾淨影像對上訓練。凍結的語意特徵對 (a) 與 (b) 幫助最大；(c) 更依賴逐像素準確的統計特性，而不是語意。

參考文獻

1. Shyandram, “深度學習的數位影像處理介紹,” Medium post, 2024; updated 2026. link
2. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018. publisher page
3. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2008. publisher page
4. Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel, “Backpropagation applied to handwritten zip code recognition,” *Neural Computation*, vol. 1, no. 4, pp. 541–551, 1989. doi:10.1162/neco.1989.1.4.541
5. Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. doi:10.1109/5.726791
6. O. Russakovsky, J. Deng, H. Su, et al., “ImageNet large scale visual recognition challenge,” *International Journal of Computer Vision*, vol. 115, no. 3, pp. 211–252, 2015. doi:10.1007/s11263-015-0816-y ·arXiv:1409.0575
7. A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” *NeurIPS*, 2012. proceedings
8. K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *ICLR*, 2015. arXiv:1409.1556
9. C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, “Going deeper with convolutions,” *CVPR*, 2015. doi:10.1109/CVPR.2015.7298594 ·arXiv:1409.4842
10. K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” *CVPR*, 2016. doi:10.1109/CVPR.2016.90 ·arXiv:1512.03385
11. A. Krizhevsky, “Learning multiple layers of features from tiny images,” technical report, University of Toronto, 2009 (the CIFAR-10/100 datasets). dataset page
12. W. Luo, Y. Li, R. Urtasun, and R. Zemel, “Understanding the effective receptive field in deep convolutional neural networks,” *NeurIPS*, 2016. arXiv:1701.04128

13. J. Long, E. Shelhamer, and T. Darrell, “Fully convolutional networks for semantic segmentation,” *CVPR*, 2015. arXiv:1411.4038
14. O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” *MICCAI*, 2015. arXiv:1505.04597
15. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *NeurIPS*, 2017. ML anthology
16. A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” *ICLR*, 2021. arXiv:2010.11929
17. Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, “Swin Transformer: Hierarchical vision Transformer using shifted windows,” *ICCV*, 2021. doi:10.1109/ICCV48922.2021.00986 ·arXiv:2103.14030
18. Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, “A ConvNet for the 2020s,” *CVPR*, 2022, arXiv:2201.03545; and S. Woo, S. Debnath, R. Hu, X. Chen, Z. Liu, I. S. Kweon, and S. Xie, “ConvNeXt V2: Co-designing and scaling ConvNets with masked autoencoders,” *CVPR*, 2023, arXiv:2301.00808.
19. M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, “Emerging properties in self-supervised vision Transformers,” *ICCV*, 2021. doi:10.1109/ICCV48922.2021.00951 ·arXiv:2104.14294
20. K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, “Masked autoencoders are scalable vision learners,” *CVPR*, 2022. doi:10.1109/CVPR52688.2022.01553 ·arXiv:2111.06377
21. M. Oquab, T. Darcet, T. Moutakanni, et al., “DINOv2: Learning robust visual features without supervision,” *Transactions on Machine Learning Research*, 2024, arXiv:2304.07193; and O. Siméoni, H. V. Vo, M. Seitzer, F. Baldassarre, M. Oquab, et al., “DINOv3,” arXiv preprint, 2025, arXiv:2508.10104.
22. L. Zhu, B. Liao, Q. Zhang, X. Wang, W. Liu, and X. Wang, “Vision Mamba: Efficient visual representation learning with bidirectional state space model,” *ICML*, 2024, arXiv:2401.09417; and Y. Liu, Y. Tian, Y. Zhao, H. Yu, L. Xie, Y. Wang, Q. Ye, J. Jiao, and Y. Liu, “VMamba: Visual state space model,” *NeurIPS*, 2024, arXiv:2401.10166.
23. S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” *NeurIPS*, 2015, arXiv:1506.01497; and N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, “End-to-end object detection with Transformers,” *ECCV*, 2020, arXiv:2005.12872.
24. A. Kirillov, E. Mintun, N. Ravi, et al., “Segment anything,” *ICCV*, 2023, arXiv:2304.02643; N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, et al., “SAM 2: Segment anything in images and videos,” *ICLR*, 2025, arXiv:2408.00714; and N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, et al., “SAM 3: Segment anything with concepts,” *ICLR*, 2026, arXiv:2511.16719.
25. A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, et al., “Learning transferable visual models from natural language supervision,” *ICML*, 2021. arXiv:2103.00020
26. H. Liu, C. Li, Q. Wu, and Y. J. Lee, “Visual instruction tuning,” *NeurIPS*, 2023. arXiv:2304.08485
27. J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” *NeurIPS*, 2020. arXiv:2006.11239
28. R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” *CVPR*, 2022, arXiv:2112.10752; and W. Peebles and S. Xie,

"Scalable diffusion models with Transformers," *ICCV*, 2023, arXiv:2212.09748.
29. T. Brooks, A. Holynski, and A. A. Efros, "InstructPix2Pix: Learning to follow image editing instructions," *CVPR*, 2023. arXiv:2211.09800
30. Z. Li, F. Liu, W. Yang, S. Peng, and J. Zhou, "A survey of convolutional neural networks: Analysis, applications, and prospects," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, 2022. doi:10.1109/TNNLS.2021.3084827 ·arXiv:2004.02806
31. S. Khan, M. Naseer, M. Hayat, S. W. Zamir, F. S. Khan, and M. Shah, "Transformers in vision: A survey," *ACM Computing Surveys*, vol. 54, no. 10s, 2022. doi:10.1145/3505244 ·arXiv:2101.01169
32. L. Jing and Y. Tian, "Self-supervised visual feature learning with deep neural networks: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, 2021. doi:10.1109/TPAMI.2020.2992393 ·arXiv:1902.06162
33. R. Balestriero, M. Ibrahim, V. Sobal, et al., "A cookbook of self-supervised learning," arXiv preprint, 2023. arXiv:2304.12210
34. M. Awais, M. Naseer, S. Khan, R. M. Anwer, H. Cholakkal, M. Shah, M.-H. Yang, and F. S. Khan, "Foundation models defining a new era in vision: A survey and outlook," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 47, 2025. doi:10.1109/TPAMI.2024.3506283 ·arXiv:2307.13721
35. J. Zhang, J. Huang, S. Jin, and S. Lu, "Vision-language models for vision tasks: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, 2024. doi:10.1109/TPAMI.2024.3369699 ·arXiv:2304.00685
36. scikit-learn developers, "sklearn.datasets.load_digits" (a copy of the test set of the UCI optical recognition of handwritten digits dataset). docs

DLCV 02・深度影像分類

深度網路如何判斷一張影像裡是什麼：損失函數與評估指標、資料集、CNN 與 Transformer 架構背後的設計理由、現代訓練配方、預訓練、穩健性、校準與可解釋性。

網站版本: https://blog.shyandram.dev/zh-tw/learn/deep-learning/dl-02-image-classification/

你將學到

- 影像分類的精確定義：單標籤、多標籤與細粒度（fine-grained）任務；logits、softmax 與交叉熵（cross-entropy）；top-1 ／ top-5 準確率；混淆矩陣與由它衍生的指標（精確率、召回率、F1、MCC、巨觀與微觀平均、ROC-AUC、平均精確度、mAP）；以及分類器「校準良好」是什麼意思。
- 標準訓練用資料集（MNIST、CIFAR、ImageNet、iNaturalist）各自的用途；哪些評估基準在測什麼（ImageNet-ReaL、ImageNetV2、穩健性測試集、細粒度與遷移測試組、長尾與零樣本評估）；以及如何執行誠實的評估流程。
- 主要骨幹網路（backbone）背後的設計理由：從 AlexNet、VGG、Inception、ResNet，到 MobileNet、EfficientNet、RegNet、ConvNeXt、ViT、DeiT 與 Swin，並附上可以自己驗算的參數量與計算量。
- 現代訓練配方中哪些部分真正重要（資料擴增、Mixup ／ CutMix、標籤平滑、AdamW、暖身＋餘弦排程、隨機深度、EMA），以及為什麼光靠配方就能讓 ResNet-50 提升好幾個百分點。
- 預訓練如何改變了這份工作：線性探測與微調、對比式與遮罩式自監督學習、DINOv2 式的凍結特徵，以及 CLIP 的零樣本（zero-shot）分類。
- 如何誠實地評估：分佈偏移、對抗樣本、捷徑學習、溫度縮放、效率指標與 Grad-CAM。

先看全貌

影像分類對整張影像只問一個問題：這些標籤裡，哪一個最符合？它是最早的深度學習基準任務，到今天仍是大多數骨幹網路誕生的地方。物體偵測、分割或影像復原網路，通常都從一個先以影像分類訓練、或至少以影像分類評估過的骨幹網路出發。

> **用最簡單的話說**
>
> 想像一個小朋友看著圖畫書學認動物。一開始他只看到一團一團的顏色。翻過很多頁之後，他學會先注意耳朵，再注意臉，最後說出「這是貓」。深度分類器也是一層一層這樣做：前面的層注意邊緣和顏色，中間的層注意眼睛、輪子這類零件，最後一層投票選出標籤。訓練就是爸媽在旁邊說了幾千次「不對，那是狗」。這篇教學其餘的部分在講：怎麼做一本更好的圖畫書（資料）、一顆更好的腦袋（架構）、一套更好的教法（訓練配方），以及更好的考試（評估）。因為一個只看過「貓坐在沙發上」的小朋友，可能會以為「沙發」就代表「貓」。

本篇假設你已經知道卷積、CNN 的層與反向傳播。分類的古典面向（貝氏分類器、感知器、用 NumPy 寫的小型 CNN、手推 softmax 與交叉熵）在IP 13。深度學習用於影像的整體歷史

在DLCV 01。這裡我們專注、深入地談分類本身。若想快速複習激活函數與最佳化器，可以參考ML 02・類神經網路。CNN 這一側的發展，Li 等人 [1] 的綜述是很好的單篇入門。

分類問題的定義

白話版。網路看一張影像，對每個類別輸出一個分數。我們把分數轉成機率，與真正的標籤比較，再輕輕推動網路，讓正確類別下一次得到更高的機率。

單標籤、多標籤與細粒度

令 $\mathbf{x} \in \mathbb{R}^{H\times W\times 3}$ 為一張 RGB 影像，$\mathcal{Y} = \{1, \dots, K\}$ 為 K 個類別的集合。參數為 θ 的網路 f_θ 把影像映射到一個 **logits** 向量 $\mathbf{z} = f_\theta(\mathbf{x}) \in \mathbb{R}^K$，每個類別一個未正規化的分數。

- **單標籤（多類別）**分類假設恰好有一個正確類別 $y \in \mathcal{Y}$。以 softmax 把 logits 轉成機率向量：

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, \dots, K,$$

其中 p_k 是模型對 $P(y = k \mid \mathbf{x})$ 的估計。訓練時最小化**交叉熵**，也就是正確類別機率的負對數，在一個小批次上取平均：

$$\mathcal{L}_{\text{CE}} = -\frac{1}{B}\sum_{i=1}^{B} \log p_{y_i}(\mathbf{x}_i),$$

其中 B 是批次大小，y_i 是影像 $\mathbf{x}_i$ 的標籤。它對 logits 的梯度（每張影像）是 $\partial \mathcal{L}/\partial \mathbf{z} = \mathbf{p} - \mathbf{e}_y$，$\mathbf{e}_y$ 是正確類別的 one-hot 向量；推導見 DIP 第 13 章。

- **多標籤**分類允許任意個標籤同時成立（一張街景可能同時有「汽車」、「行人」、「腳踏車」）。每個類別各自接一個 sigmoid $\sigma(z_k) = 1/(1+e^{-z_k})$ 與二元交叉熵 $-\sum_k [t_k \log \sigma(z_k) + (1-t_k)\log(1-\sigma(z_k))]$，其中 $t_k \in \{0,1\}$ 表示類別 k 是否出現。評估指標也要換：以各類別的平均精確度（mean average precision）取代準確率。
- **細粒度**分類要分開外觀非常相近的子類別，例如 200 種鳥或 1,000 種飛機型號。損失函數與單標籤相同，但訊號藏在小地方（鳥喙形狀、翅膀上的條紋），類別通常呈長尾分佈，而且非專家標註者造成的標籤雜訊很常見。iNaturalist [6] 是標準的大型細粒度基準。

評估指標：top-1、top-5 與其他

Top-1 準確率是最高分類別恰好正確的影像比例。**Top-k 準確率**則只要正確類別落在前 k 高分之中就算對。ImageNet 挑戰賽回報 top-5 錯誤率，因為很多影像裡有好幾個物體，而有些類別幾乎無法區分 [2]。但 top-5 會掩蓋真正的錯誤，所以現代論文大多回報 top-1。資料不平衡時，請回報**平衡準確率**（各類別召回率的平均）或逐類別結果，因為分類器只要忽略稀有類別，也能拿到很高的準確率。

```
import torch
import torch.nn.functional as F

logits = torch.tensor([[2.0, 1.0, 0.1, -1.0],    # 2 images, 4 classes
                       [0.5, 2.5, 2.4, 0.0]])
y = torch.tensor([0, 2])

p = logits.softmax(dim=1)
ce = F.cross_entropy(logits, y)                       # mean of -log p[i, y_i]
ce_ls = F.cross_entropy(logits, y, label_smoothing=0.1)
top1 = (logits.argmax(1) == y).float().mean()
top2 = (logits.topk(2, dim=1).indices == y[:, None]).any(1).float().mean()
print(p.round(decimals=3))
print(f"CE {ce:.3f}  CE+LS {ce_ls:.3f}  top-1 {top1:.2f}  top-2 {top2:.2f}")

# multi-label: one independent sigmoid per class, binary cross-entropy
targets = torch.tensor([[1., 0., 1., 0.], [0., 1., 1., 0.]])
print(f"BCE {F.binary_cross_entropy_with_logits(logits, targets):.3f}")
```

第二張影像在類別 1 和 2 之間幾乎平手。Top-1 算它錯，top-2 算它對。支持與反對 top-5 的理由，都在這一個例子裡。

混淆矩陣與由它衍生的指標

白話版。準確率把所有錯誤壓成一個數字。混淆矩陣（confusion matrix）則把錯誤分開來記：對每一個真實類別，數一數模型分別回答了什麼。幾乎所有其他分類指標，都是這張表裡某些格子的比值，而每一種比值回答的是不同的問題。

二元情況：TP、FP、FN、TN

把某一類當作**陽性**（positive，例如「生病」），其餘當作**陰性**（negative）。每一個預測都會落在四個格子之一：

	預測為陽性	預測為陰性
實際為陽性	真陽性（true positive, TP）	偽陰性（false negative, FN）
實際為陰性	偽陽性（false positive, FP）	真陰性（true negative, TN）

用最簡單的話說

煙霧警報器就是一個分類器。真的失火、它響了，是真陽性。你只是把吐司烤焦、它也響了，是偽陽性：很吵，但沒有人受傷。真的失火、它卻沒響，是偽陰性：這才是危險的那一種。什麼事都沒有、它也安靜，是真陰性。好的警報器不只是「大部分時間都對」（它幾乎每天都安靜，要做到這點太容易了），而是要在失火時響、在烤吐司時安靜。不同的指標，對這兩個目標的看重程度不一樣。

設測試樣本共 $N = \text{TP} + \text{FP} + \text{FN} + \text{TN}$ 個，標準的比值為

$$\text{accuracy} = \frac{\text{TP}+\text{TN}}{N}, \quad \text{precision} = \frac{\text{TP}}{\text{TP}+\text{FP}}, \quad \text{recall (TPR)} = \frac{\text{TP}}{\text{TP}+\text{FN}},$$
$$\text{specificity (TNR)} = \frac{\text{TN}}{\text{TN}+\text{FP}}, \qquad \text{FPR} = \frac{\text{FP}}{\text{FP}+\text{TN}} = 1 - \text{TNR}.$$

- **精確率（precision）**回答「模型說是陽性的時候，有多常是對的？」當誤報代價很高時它最重要：會把正常信件藏起來的垃圾郵件過濾器、自動觸發的產品召回。
- **召回率（recall）**，又稱**敏感度**（sensitivity）或**真陽性率**（true positive rate, TPR），回答「所有真正的陽性中，我們抓到了多少？」當漏抓代價很高時它最重要：癌症篩檢、產線上的瑕疵偵測。
- **特異度**（specificity），即**真陰性率**（true negative rate, TNR），是陰性類別的召回率。**偽陽性率**（false positive rate, FPR）是它的補數，也就是被錯誤標記的陰性樣本比例。

精確率與召回率互相拉扯：降低決策門檻會標記更多樣本，召回率上升，精確率通常下降。把兩者合成一個數字的指標是：

$$F_1 = \frac{2\,\text{P}\,\text{R}}{\text{P}+\text{R}} = \frac{2\,\text{TP}}{2\,\text{TP}+\text{FP}+\text{FN}}, \qquad F_\beta = \frac{(1+\beta^2)\,\text{P}\,\text{R}}{\beta^2\,\text{P}+\text{R}},$$

其中 P 是精確率，R 是召回率，$\beta > 0$ 表示召回率比精確率重要幾倍（F_2 偏重召回率，$F_{0.5}$ 偏重精確率）。F_1 是調和平均，所以只有兩者都高時它才會高。也請注意 F_1 忽略了什麼：真陰性完全沒有出現在式子裡。

另外兩個總結指標用到了全部四格：

$$\text{balanced accuracy} = \frac{\text{TPR}+\text{TNR}}{2}, \qquad \text{MCC} = \frac{\text{TP}\cdot\text{TN} - \text{FP}\cdot\text{FN}}{\sqrt{(\text{TP}+\text{FP})(\text{TP}+\text{FN})(\text{TN}+\text{FP})(\text{TN}+\text{FN})}}.$$

平衡準確率（balanced accuracy）是各類別召回率的平均，所以一個忽略某一類的分類器，不論那一類多稀有，都只能拿到 0.5。**Matthews 相關係數**（Matthews correlation coefficient, MCC）是真實與預測二元標籤之間的 Pearson 相關係數：+1 表示完美，0 表示不比隨機好，−1 表示完全相反。Chicco 與 Jurman [57] 主張在二元評估中應優先使用 MCC，而不是準確率與 F_1，因為只有分類器在四個格子上都表現好時，它才會高。Sokolova 與 Lapalme [58] 系統性地比較了這些指標，以及它們各自如何隨混淆矩陣的變化而反應。

為什麼 99:1 時準確率會誤導。假設 1,000 張掃描影像中有 10 張是生病的。一個永遠回答健康的「模型」可以拿到 99.0% 的準確率。模型 B 標記了 30 張，抓到 10 張中的 8 張；它的準確率較低（97.6%），卻是唯一有用的那一個：

```
import numpy as np
from sklearn.metrics import (accuracy_score, balanced_accuracy_score, confusion_matrix,
                             f1_score, matthews_corrcoef, precision_score, recall_score)

# 1,000 scans, 10 sick (label 1). Model A says "healthy" to everyone.
# Model B flags 30 scans and catches 8 of the 10 sick ones.
y = np.r_[np.ones(10), np.zeros(990)].astype(int)
pred_a = np.zeros(1000, dtype=int)
pred_b = np.r_[np.ones(8), np.zeros(2), np.ones(22), np.zeros(968)].astype(int)
```

```
for name, p in [("all-negative", pred_a), ("model B", pred_b)]:
    tn, fp, fn, tp = confusion_matrix(y, p).ravel()
    print(f"{name:12s} TP={tp} FP={fp} FN={fn} TN={tn}  "
          f"acc={accuracy_score(y, p):.3f} bal-acc={balanced_accuracy_score(y, p):.3f} "
          f"P={precision_score(y, p, zero_division=0):.3f} R={recall_score(y, p):.3f} "
          f"F1={f1_score(y, p, zero_division=0):.3f} MCC={matthews_corrcoef(y, p):.3f}")
# all-negative TP=0 FP=0 FN=10 TN=990  acc=0.990 bal-acc=0.500 P=0.000 R=0.000 F1=0.000
 ↪  MCC=0.000
# model B      TP=8 FP=22 FN=2 TN=968  acc=0.976 bal-acc=0.889 P=0.267 R=0.800 F1=0.400
 ↪  MCC=0.454
```

平衡準確率、F_1 與 MCC 都把模型 B 排在第一；準確率卻把它排在最後。經驗法則：只有在類別大致平衡、所有錯誤代價相同時才用準確率；當某一種錯誤比較嚴重時，回報精確率與召回率（或依實際代價選定 β 的 F_β）；需要一個能反映不平衡的單一總結數字時，用平衡準確率或 MCC。

多類別混淆矩陣

有 K 個類別時，混淆矩陣 $C \in \mathbb{N}^{K\times K}$ 的元素 C_{ij} 是**真實**類別為 i（列）、**預測**類別為 j（行）的測試影像數；這是 scikit-learn [62] 的慣例。對角線是預測正確的數量。把每一列除以該列總和，得到**依列正規化**的矩陣：它的對角線就是各類別的召回率，非對角元素則顯示哪些類別被搞混。圖 1 是一個只用 scikit-learn 8×8 手寫數字中 200 張訓練的邏輯迴歸所得到的結果：大部分錯誤是 3 → 8、4 → 1 與 7 → 9，都是筆畫相似的組合。

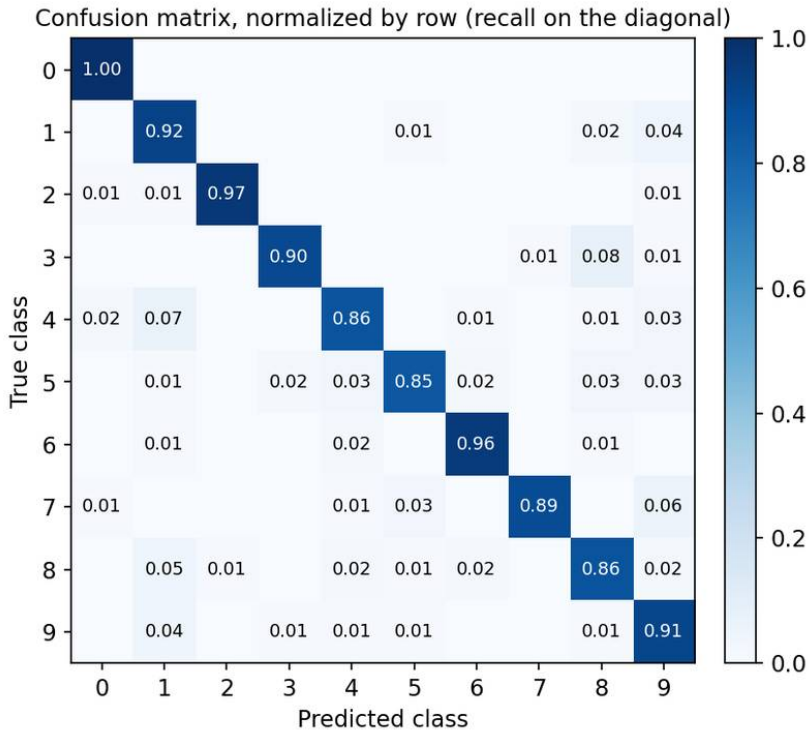


圖 *1* — 以 *200* 張數字訓練、在 *899* 張上測試的邏輯迴歸之依列正規化混淆矩陣（準確率 *91.1%*）。每一列總和為 *1*；對角線是各類別的召回率，較亮的非對角格指出值得修正的混淆。

輪流把每個類別 k 當作「陽性」、其他類別當作「陰性」，就得到逐類別的計數：$\mathrm{TP}_k = C_{kk}$，$\mathrm{FP}_k = \sum_{i \neq k} C_{ik}$（第 k 行的其餘部分），$\mathrm{FN}_k = \sum_{j \neq k} C_{kj}$（第 k 列的其餘部分）。因此逐類別精確率 $\mathrm{P}_k = C_{kk} / \sum_i C_{ik}$，召回率 $\mathrm{R}_k = C_{kk} / \sum_j C_{kj}$。把 K 個逐類別數字合成一個，常見的有三種做法：

- **巨觀平均**（macro average）：對類別直接取平均，$\mathrm{F}_1^{\text{macro}} = \frac{1}{K} \sum_k \mathrm{F}_{1,k}$。每個類別權重相同，所以稀有類別和常見類別一樣重要。
- **微觀平均**（micro average）：先把各類別的 TP_k、FP_k、FN_k 加總，再算一次指標。在單標籤多類別問題中，每一個錯誤都同時是一個 FP（對預測的類別）和一個 FN（對真實的類別），所以微觀精確率 = 微觀召回率 = 微觀 F_1 = 準確率。
- **加權平均**（weighted average）：以各類別的**支持數**（support）n_k（真實樣本數）為權重，對逐類別數值取平均。它很接近微觀平均，幾乎一樣會掩蓋稀有類別的失敗。

計算範例。 100 張測試影像，三個類別的支持數分別為 80（A）、15（B）、5（C）：

真實 ＼ 預測	A	B	C
A（80）	76	3	1
B（15）	4	10	1
C（5）	3	0	2

各行總和為 83、13、4，所以精確率為 $76/83 = 0.916$、$10/13 = 0.769$、$2/4 = 0.500$；召回率為 $76/80 = 0.950$、$10/15 = 0.667$、$2/5 = 0.400$；逐類別 F_1 為 0.933、0.714、0.444。於是

$$\mathrm{F}_1^{\text{macro}} = \tfrac{1}{3}(0.933 + 0.714 + 0.444) = 0.697, \qquad \mathrm{F}_1^{\text{micro}} = \text{accuracy} = \tfrac{76+10+2}{100} = 0.880,$$

而以支持數加權的 F_1 為 $0.80 \cdot 0.933 + 0.15 \cdot 0.714 + 0.05 \cdot 0.444 = 0.875$。微觀與巨觀之間 18 個百分點的差距就是訊號：模型在常見類別上很好，在稀有類別上很差。當每個類別都重要時（罕見疾病、稀有物種）回報巨觀平均；當每一張影像同等重要時，回報微觀平均／準確率。

Cohen's kappa [59] 把觀察到的一致程度，扣掉純靠機率就會有的一致程度：

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \qquad p_o = \frac{1}{N} \sum_k C_{kk}, \qquad p_e = \frac{1}{N^2} \sum_k \Big(\sum_j C_{kj} \Big) \Big(\sum_i C_{ik} \Big),$$

其中 p_o 是準確率，p_e 是一個與輸入無關、只依觀察到的比例隨機預測各類別的分類器所能得到的準確率。它原本是為兩位人類評分者之間的一致性設計的，在比較模型與標註者時仍很常見。

```
from sklearn.datasets import load_digits
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (average_precision_score, classification_report,
 ↪  cohen_kappa_score,
                             confusion_matrix, precision_recall_fscore_support,
                              ↪  roc_auc_score)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import label_binarize
```

```
X, y = load_digits(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.5, stratify=y, random_state=0)
clf = LogisticRegression(max_iter=2000).fit(X_tr[:200], y_tr[:200])   # only 200 labels
pred, prob = clf.predict(X_te), clf.predict_proba(X_te)

cm = confusion_matrix(y_te, pred)                      # rows = true, columns = predicted
cm_norm = confusion_matrix(y_te, pred, normalize="true")   # each row sums to 1 (Figure 1)
print(classification_report(y_te, pred, digits=3))    # per-class P, R, F1, support +
 ↪  averages
for avg in ["macro", "micro", "weighted"]:
    p, r, f, _ = precision_recall_fscore_support(y_te, pred, average=avg)
    print(f"{avg:8s} P={p:.3f} R={r:.3f} F1={f:.3f}")
print(f"kappa {cohen_kappa_score(y_te, pred):.3f}")
print(f"ROC-AUC (one-vs-rest, macro) {roc_auc_score(y_te, prob, multi_class='ovr'):.4f}")
Y = label_binarize(y_te, classes=range(10))            # one binary column per class
print(f"mean AP over classes {average_precision_score(Y, prob, average='macro'):.4f}")
# macro    P=0.914 R=0.911 F1=0.912
# micro    P=0.911 R=0.911 F1=0.911
# weighted P=0.915 R=0.911 F1=0.912
# kappa 0.901
# ROC-AUC (one-vs-rest, macro) 0.9958
# mean AP over classes 0.9742
```

這裡巨觀與微觀平均幾乎相同，因為手寫數字的各類別是平衡的；換成上面 A ／ B ／ C 的例子就不會如此。

不依賴門檻的評估：ROC 曲線與精確率–召回率曲線

上面每一個指標都需要一條決策規則（argmax，或「分數超過 t 就是陽性」）。會輸出分數的分類器，可以一次在所有門檻上評估：把 t 從高掃到低，每個門檻記下一個點。

- **ROC 曲線**畫的是 TPR 對 FPR。曲線下面積 **ROC-AUC** 等於「隨機取一個陽性樣本，它的分數高於隨機取的一個陰性樣本」的機率；0.5 是隨機，1.0 是完美分開。Fawcett [54] 是標準的入門文獻。
- **精確率–召回率（PR）曲線**畫的是精確率對召回率。它的總結指標**平均精確度**（average precision, AP），是在每次多找回一個陽性樣本的召回率位置上，對精確率取平均：

$$\mathrm{AP} = \sum_{n} (\mathrm{R}_n - \mathrm{R}_{n-1})\, \mathrm{P}_n,$$

其中 P_n 與 R_n 是依分數由高到低排序後、第 n 個門檻的精確率與召回率（這是 scikit-learn [62] 的定義；PASCAL VOC [61] 等基準用過內插的變體，所以要確認論文回報的是哪一種）。

為什麼嚴重不平衡時要看 PR。 FPR 的分母是陰性樣本數。當陰性樣本是陽性的 99 倍時，即使有上千個誤報，FPR 也幾乎不動，所以 ROC 曲線看起來很漂亮，精確率卻很差。精確率的分母是被標記的樣本數，誤報會直接打擊它，而 AP 的隨機基準線等於陽性比例，而不是 0.5。Davis 與 Goadrich [55] 證明：一條曲線在 ROC 空間中占優勢，若且唯若它在 PR 空間中也占優勢；並指出最佳化 ROC-AUC 並不等於最佳化 AP。Saito 與 Rehmsmeier [56] 以模擬與文獻回顧顯示，在

不平衡資料上，PR 圖比 ROC 圖更有資訊量。圖 2 把這件事具體化：在陽性只占 1% 的任務上，兩個 ROC-AUC 相同（0.95）的分類器，AP 卻明顯不同（0.45 對 0.36）。

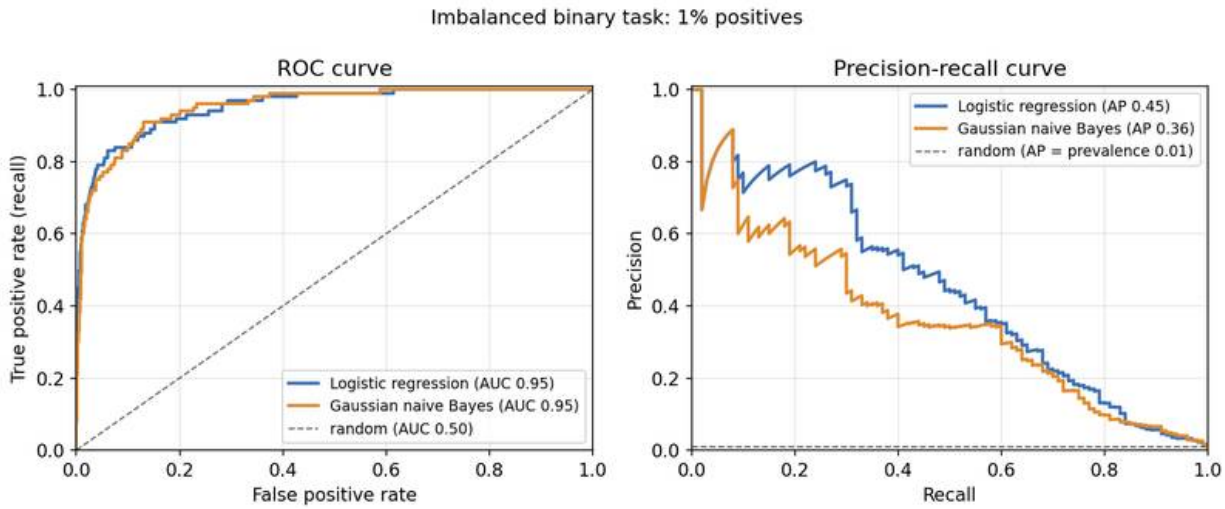


圖 2 — 在陽性只占 *1%* 的合成任務上（*10,000* 個測試點中有 *100* 個陽性），兩個分類器的 *ROC* 曲線（左）與精確率–召回率曲線（右）。兩者 *ROC-AUC* 都是 *0.95*，但邏輯迴歸在每個召回率下的精確率都高得多（*AP 0.45* 對 *0.36*）。在這個任務上，一個全部判為陰性的分類器可以拿到 *99%* 的準確率。

當類別數 $K > 2$ 時，兩種曲線都以**一對其餘**（one-vs-rest）的方式計算：類別 k 為陽性、其他全部為陰性，使用分數 p_k。再把 K 個面積做巨觀平均（`roc_auc_score(..., multi_class="ovr")`），或把所有（類別, 影像）配對合併後做微觀平均。

多標籤指標

多標籤分類中，每張影像有一個二元目標向量 $\mathbf{t} \in \{0,1\}^K$，模型則輸出一個分數向量。每個標籤都是自己的一個二元問題，所以自然的排序指標是**逐類別 AP** 及其對類別的平均 **mAP**；它隨 PASCAL VOC 分類任務 [61] 成為標準。需要門檻的指標分成兩大類，Zhang 與 Zhou [60] 有完整回顧：

- **以標籤為基礎**（label-based）的指標：對每個標籤算一個二元指標，再對標籤做巨觀或微觀平均，與多類別的情況完全相同。
- **以樣本為基礎**（example-based）的指標：對每一張影像算一個分數，再對影像取平均。以樣本為基礎的 F_1 為 $\frac{1}{N}\sum_i \frac{2|\hat{Y}_i \cap Y_i|}{|\hat{Y}_i| + |Y_i|}$，其中 Y_i 與 $\hat{Y}_i$ 是影像 i 的真實與預測標籤集合。
- **Hamming loss** 是（影像, 標籤）位元中錯誤的比例，$\frac{1}{NK}\sum_i |\hat{Y}_i \,\triangle\, Y_i|$，其中 $\triangle$ 是對稱差。它很好解讀，但當稀有標籤很多時，什麼都不預測看起來也會很好。

```
import numpy as np
from sklearn.metrics import average_precision_score, f1_score, hamming_loss

# 4 street photos, 3 labels: car, person, bicycle
```

```
T = np.array([[1, 1, 0], [1, 0, 0], [0, 1, 1], [1, 1, 1]])            # ground truth
S = np.array([[.9, .6, .2], [.8, .4, .3], [.3, .7, .4], [.6, .2, .7]])  # sigmoid scores
P = (S >= 0.5).astype(int)                                             # threshold 0.5

print("per-class AP:", average_precision_score(T, S, average=None).round(3))
print(f"mAP {average_precision_score(T, S, average='macro'):.3f}")
print(f"example-based F1 {f1_score(T, P, average='samples'):.3f}  "
      f"macro F1 {f1_score(T, P, average='macro'):.3f}  micro F1 {f1_score(T, P,
       ↪  average='micro'):.3f}")
print(f"Hamming loss {hamming_loss(T, P):.3f}")
# per-class AP: [1.    0.917 1.   ]
# mAP 0.972
# example-based F1 0.867  macro F1 0.822  micro F1 0.857
# Hamming loss 0.167
```

校準：機率說的是實話嗎？

準確率只檢查 argmax。**校準**（calibration）檢查信心值是否名副其實：在所有信心為 0.8 的預測中，應該大約有 80% 是對的。正式地說，若對每個信心值 c 都有 $P(\hat{y} = y \mid \hat{p} = c) = c$，分類器就是校準良好的，其中 $\hat{y}$ 是預測類別，$\hat{p} = \max_k p_k$ 是它的機率。常用的總結指標是**期望校準誤差**（expected calibration error, ECE）：把預測依信心分到 M 個區間 B_m，計算

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \left| \mathrm{acc}(B_m) - \mathrm{conf}(B_m) \right|,$$

其中 n 是測試影像數，$\mathrm{acc}(B_m)$ 是第 m 個區間內的準確率，$\mathrm{conf}(B_m)$ 是區間內的平均信心。Guo 等人 [47] 發現，用交叉熵長時間訓練的現代深度網路傾向**過度自信**，而只用一個純量就能修正得相當好。我們會在穩健性一節回來談。

資料集與它們的角色

白話版。基準資料集是一份凍結的考卷。它讓大家能公平比較方法；但當整個領域都在準備同一份考卷，分數衡量的就不只是能力，也包括這份考卷本身。把兩種角色分開會比較清楚：用來訓練模型的資料集，以及只用來評估模型的資料集。

訓練用資料集

資料集	影像數	類別數	解析度	角色
MNIST [3]	訓練 6 萬／測試 1 萬	10 個數字	28×28 灰階	健全性檢查、教學
CIFAR-10 / CIFAR-100 [4]	訓練 5 萬／測試 1 萬	10 ／ 100（20 個超類別）	32×32 RGB	小規模的架構與配方研究
ImageNet-1k（ILSVRC 2012）[2]	約 128 萬訓練／ 5 萬驗證	1,000	不固定，通常以 224×224 訓練	骨幹網路的標準基準
ImageNet-21k [5]	約 1,300–1,400 萬（版本不同）	約 2.1 萬	不固定	更大規模的監督式預訓練

資料集	影像數	類別數	解析度	角色
iNaturalist [6]	數十萬，依年份而異	數千個物種	不固定	細粒度、長尾

MNIST 已經被解決，現在只用來抓程式錯誤。CIFAR 小到可以跑大量受控實驗，所以正則化論文特別愛用。ImageNet-1k 在 2012 年挑戰賽之後成為參考標準 [2], [10]：架構通常以「ImageNet-1k 驗證集、224×224 的 top-1」來回報，模型庫裡幾乎每個預訓練骨幹都附有一個 ImageNet 分數。ImageNet-21k 是 ImageNet-1k 所取自的更大階層；Ridnik 等人 [5] 以清理過、考慮語意階層的配方，讓在它上面預訓練變得實際可行。

評估用基準資料集

白話版。訓練資料負責教，評估資料負責考。只考過一份考卷的模型，可能只是學會了那份考卷。所以現代論文會用一整組測試集來評估骨幹網路，每一個都設計來探測一件事：原始分佈、從同一分佈重新抽的樣本、損壞或風格化的影像、不尋常的視角、細粒度類別、很少的標註預算，或稀有類別。

下表列出最常見的評估集。大小以所引用的論文或官方頁面為準。

基準	衡量什麼	大小／類別數	常回報的指標
ImageNet-1k 驗證集 [2]	分佈內準確率；因為 ILSVRC 測試集的標籤保留在評估伺服器上，它成了實質上的測試集	50,000 張，1,000 類	top-1（top-5）
ImageNet-ReaL [8]	同一批驗證影像，重新標註為多標籤的「重新評估」標籤	46,837 張影像、57,553 個標籤（3,163 張沒有任何標籤的影像被捨棄）	ReaL 準確率：top-1 預測只要落在標籤集合內就算對
ImageNetV2 [7]	依原始收集流程重做一次所得到的新樣本上的泛化能力	每個版本 10,000 張（每類 10 張）；三種抽樣版本，通常使用 MatchedFrequency	top-1
ImageNet-C [42]	對合成損壞的穩健性	驗證影像 × 15 種損壞 × 5 個嚴重程度	mean corruption error
ImageNet-R [43]	各種表現形式：藝術、卡通、雕塑、玩具等	30,000 張，200 類	200 類上的 top-1
ImageNet-A [44]	經篩選、專門騙倒 ResNet-50 的自然照片	7,500 張，200 類	200 類上的 top-1
ImageNet-Sketch [63]	轉換到黑白素描領域	50,000 張，1,000 類（每類 50 張）	top-1
ObjectNet [64]	控制偏差的照片，刻意變化旋轉、背景與視角；只有測試集	50,000 張，313 類（其中 113 類與 ImageNet 重疊）	重疊類別上的 top-1
ImageNet-LT [73]	長尾分佈的訓練、平衡的測試	11.58 萬張訓練影像，1,000 類，每類 5 到 1,280 張；平衡的測試集	top-1，常依類別頻率分組回報
iNaturalist [6]	天然長尾、細粒度的物種	見訓練用資料集表	top-1

細粒度與遷移基準。骨幹網路也要看它的特徵能多好地遷移到小型、專門的資料集上，方式是在凍結特徵上做線性探測，或是微調（見下文的遷移學習一節）。

資料集	領域	大小／類別數	常回報的指標
CUB-200-2011 [65]	鳥類物種，附部位與屬性標註	11,788 張，200 類	top-1
Stanford Cars [66]	汽車品牌、型號與年份	196 類；訓練 8,144 ／測試 8,041	top-1
FGVC-Aircraft [67]	三層階層的飛機型號	10,000 張，100 種型號	各類別平均準確率
Oxford-IIIT Pets [68]	貓狗品種	37 類，每類約 200 張	各類別平均準確率
Oxford Flowers-102 [69]	花卉物種	102 類，每類 40 到 258 張；訓練 2,040 ／測試 6,149	各類別平均準確率
Food-101 [70]	菜餚；訓練標籤刻意保留雜訊	101,000 張，101 類（每類訓練 750 ／測試 250）	top-1
Caltech-101 [71]	一般物體	101 類加背景類，每類約 40 到 800 張	各類別平均準確率

Cars 與 Flowers 的訓練／測試大小依 Kornblith 等人 [33] 所列的切分。注意其中好幾個基準回報的是**各類別平均準確率**（mean per-class accuracy，即巨觀平均的召回率），而不是一般的準確率，因為它們的測試集並不平衡；在同一個比較裡混用兩者是常見的錯誤。

有兩個**測試組**把這類資料集綁在一起，用一個數字總結遷移能力：

- Kornblith 等人 [33] 使用了 **12 個資料集**：Food-101、CIFAR-10、CIFAR-100、Birdsnap、SUN397、Stanford Cars、FGVC-Aircraft、PASCAL VOC 2007 分類、Describable Textures、Oxford-IIIT Pets、Caltech-101 與 Oxford Flowers-102。依資料集不同，回報 top-1 準確率、各類別平均準確率或 11 點 mAP（VOC 2007），並比較線性探測、微調與從頭訓練三種設定。
- **VTAB** [72] 包含 19 個任務，分成三組：7 個自然（一般相機影像）、4 個專門（衛星與醫學影像）、8 個結構（場景幾何，例如計數或距離，大多來自模擬環境）。在 **VTAB-1k** 中，每個任務只提供 1,000 個標註訓練樣本（800 個訓練、200 個驗證），因此它測試的是：一個表徵能否從很少的標籤，適應到與預訓練資料很不一樣的任務。

零樣本評估。 CLIP [40] 這類影像–文字模型在評估時，完全不在目標資料集上訓練：把類別名稱寫成文字提示，再把影像歸到最相似的提示（見下文的 CLIP 一節）。CLIP 論文用這種方式在 30 多個既有資料集上做了評估，回報零樣本 CLIP 在 27 個資料集中的 16 個上勝過以 ResNet-50 特徵做的線性探測，並發現對 ImageNet 集成 80 個提示模板，比單一預設提示多了 3.5 個百分點。這帶來兩點提醒。第一，提示與類別名稱的寫法都是超參數；在測試集上調整它們，和在測試集上調學習率一樣，會洩漏測試資訊。第二，預訓練資料是網路規模、而且大多未公開，所以「零樣本」並不保證測試影像（或近乎重複的影像）從未被看過。

這些數字有多可靠？

有三條研究路線改變了我們解讀 ImageNet 準確率的方式。

- **重現。** Recht 等人 [7] 盡可能照原始收集流程，為 CIFAR-10 和 ImageNet 重建了新的測試集（ImageNetV2）。所有模型都掉分：CIFAR-10 掉 3–15%，ImageNet 掉 11–14%。但排名大致保持，舊測試集上的進步也轉移到新測試集上。他們的解讀是：掉分來自新舊樣本之間難度的細微偏移，而不是整個社群反覆使用測試集而過擬合（overfitting）。

- **標籤**。 Beyer 等人 [8] 為 ImageNet 驗證集重新標註、允許一張影像有多個標籤，發現近年的部分「進步」其實是在擬合原始單一標籤標註的特殊習慣。Northcutt 等人 [9] 估計常見測試集中普遍存在標籤錯誤，而且這些錯誤足以改變哪個模型看起來最好。
- **偏差**。每個資料集在視角、背景、拍攝者風格上都有自己的「指紋」。DIP 第 13 章回顧了資料集偏差的研究；實務上的教訓是：只要可能，就用獨立收集的資料來測試。

把基準分數當成關於「這個基準」的陳述

「81.0% top-1」的意思是：在這 50,000 張特定的驗證影像上、用這套前處理、這組標籤所得到的結果。請回報解析度、裁切數量、是否用了額外資料或蒸餾，並在差異小於一個百分點時回報不同隨機種子間的變異。

評估流程

基準分數有多誠實，取決於背後的評估流程。

- **三種切分，三種任務**。在**訓練集**上擬合參數。在**驗證集**上（或在訓練資料內做交叉驗證）選擇超參數、提前停止點、溫度、門檻與提示。**測試集**只在最後碰一次。若基準只公開驗證集（如 ImageNet-1k），就從訓練集中切出一部分用來調整。
- **絕不在測試集上調整**。每一個看過測試結果之後才做的決定（多加一種擴增、多訓練一個 epoch、換一個檢查點），都會讓模型稍微擬合測試集。整個領域的論文累積起來，就是**測試集重複使用**（test-set reuse），也就是對基準的適應性過擬合。ImageNetV2 [7] 發現在新樣本上排名大致保持，這令人安心；但同一份研究也顯示絕對數字無法直接轉移，而新的測試集是唯一直接的檢驗方法。
- **回報變異**。用多個隨機種子訓練（至少三個，小資料集要更多），回報平均 $\pm$ 標準差，或信賴區間。在 10,000 張測試影像上，ImageNetV2 [7] 給出的 95% 信賴區間約為 $\pm 1\%$；在這樣的測試集上進步 0.3 個百分點，算不上結果。
- **回報完整配方**。解析度、裁切數、額外資料、蒸餾、指標的哪一種變體（top-1 或各類別平均準確率、哪一種 AP 定義），以及標籤集的哪一個版本（原始或 ReaL）。

CNN 架構與其背後的理由

白話版。每一個著名的 CNN 都修正了前一代的某個具體問題：訓練太慢、參數太多、太深就難以最佳化、在手機上太昂貴。知道每個架構解決了什麼問題，比背它們的層表更有用。

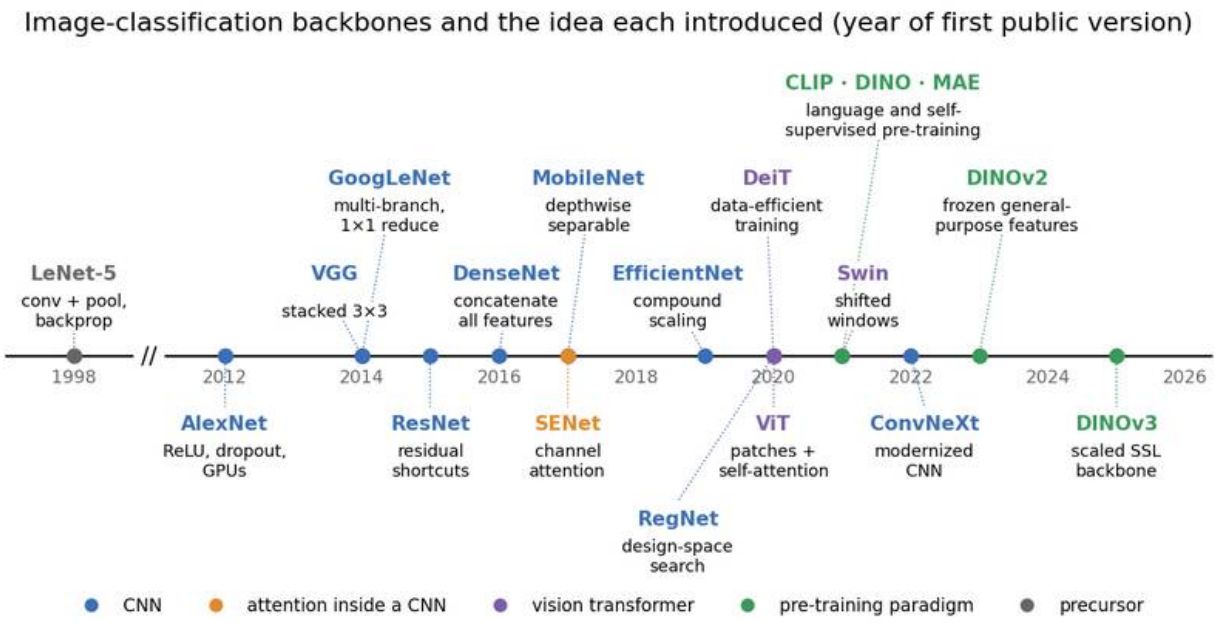


圖 3 — 影像分類骨幹網路與各自引入的主要想法。年份為首次公開版本（通常是 *arXiv*）的年份。顏色區分純 *CNN*、*CNN* 內的注意力機制、視覺 *Transformer*，以及預訓練典範。

AlexNet：深度變得可訓練

2012 年以前，最好的 ImageNet 系統使用手工特徵（SIFT、Fisher 向量）加線性分類器。AlexNet [10] 是一個五層卷積、三層全連接的網路，在 ImageNet 上端到端訓練。讓它能在這個規模上成功的有三個要素。**ReLU** 激活函數 $\max(0, z)$ 對正輸入不會飽和，梯度不會像 tanh 那樣消失，論文回報訓練速度快了好幾倍。**Dropout** 用在大型全連接層上對抗過擬合，那些層占了大部分參數。**GPU** 讓這一切可行：模型被拆在兩張 GPU 上。大量的資料擴增（隨機裁切與翻轉、顏色擾動）是第四個、常被忘記的要素。設計上的教訓是：只要資料和算力足夠，學出來的特徵勝過設計出來的特徵。

VGG：小卷積核，疊起來

VGG [11] 問了一個更乾淨的問題：如果只用 3×3 卷積，然後一直加深（16–19 個有權重的層），會怎樣？兩層 3×3 疊起來看到的是 5×5 區域，三層是 7×7。對 C 個輸入與輸出通道而言，

$$\underbrace{2 \cdot 3^2 C^2}_{\text{two } 3\times 3} = 18C^2 \;<\; \underbrace{5^2 C^2}_{\text{one } 5\times 5} = 25C^2,$$

（忽略偏差項）。所以同樣的感受野，疊層的參數更少，而且兩層之間多了一個非線性。VGG 的一致設計讓它多年來都是熱門的特徵擷取器。它的弱點是成本：全連接頭與寬大的前幾層讓它很笨重。

GoogLeNet ／ Inception：寬度、分支與 1×1 瓶頸

GoogLeNet [12] 則改攻成本。每個 **Inception 模組**平行執行 1×1、3×3、5×5 卷積與池化，再把輸出串接起來，讓下一層自己挑選適合的尺度。在很多通道上跑大卷積核很昂貴，所以先用 1×1 **卷積**減少通道數。1×1 卷積就是在每個像素上做一次跨通道的線性映射：它混合通道而不看鄰居，是改變寬度最便宜的方法。GoogLeNet 也用**全域平均池化**取代龐大的全連接頭。後續的 Inception-v3 論文 [13] 把大卷積核分解成小卷積的堆疊，或 $1\times n$ 與 $n\times1$ 的組合，並提出了**標籤平滑**（label smoothing），我們會在訓練一節再見到它。

ResNet：學習殘差

問題。一直疊加普通的層，最後網路反而變差，連訓練集上都一樣。這種「退化」是最佳化問題，不是過擬合：更深的網路原則上可以複製較淺的網路，再把多出來的層設成恆等映射，但 SGD 就是找不到這個解。

想法。 He 等人 [14] 讓一個區塊學習殘差 $\mathcal{F}$，再把輸入加回去：

$$\mathbf{y} = \mathcal{F}(\mathbf{x}; \{W_i\}) + \mathbf{x},$$

其中 $\mathbf{x}$ 是區塊輸入，$\mathcal{F}$ 是權重為 W_i 的兩到三層卷積，$\mathbf{y}$ 是輸出。如果最佳映射接近恆等，區塊只需要把 $\mathcal{F}$ 推向零，這很容易。當形狀改變（通道變多、步幅為 2）時，捷徑改成 1×1 投影。

為什麼好訓練。把 L 個預激活（pre-activation）形式的區塊 $\mathbf{x}_{l+1} = \mathbf{x}_l + \mathcal{F}(\mathbf{x}_l)$ 展開，如後續關於恆等映射的論文 [15] 所分析：

$$\mathbf{x}_L = \mathbf{x}_l + \sum_{i=l}^{L-1} \mathcal{F}(\mathbf{x}_i), \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{x}_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L}\Big(\mathbf{I} + \frac{\partial}{\partial \mathbf{x}_l}\sum_{i=l}^{L-1}\mathcal{F}(\mathbf{x}_i)\Big),$$

其中 $\mathcal{L}$ 是損失函數，$\mathbf{I}$ 是單位矩陣。恆等項把梯度從頂端直接送到任何一個區塊，所以它不會因為許多小 Jacobian 連乘而消失。該論文也發現，讓捷徑保持純粹的恆等（把 BN 與 ReLU 移進殘差分支，即「預激活」）可以更容易地訓練更深的網路。

瓶頸結構。對更深的 ResNet（50 層以上），每個區塊變成 1×1（降維）$\to 3\times3 \to 1\times1$（升維），如圖 4(c)。在 256 通道時，兩層 3×3 的基本區塊有 $2\cdot9\cdot256^2 \approx 118$ 萬個權重；降到 64 通道的瓶頸區塊則有 $256\cdot64 + 9\cdot64^2 + 64\cdot256 \approx 7$ 萬個。昂貴的 3×3 跑在很薄的張量上，這正是 50–152 層負擔得起的原因。

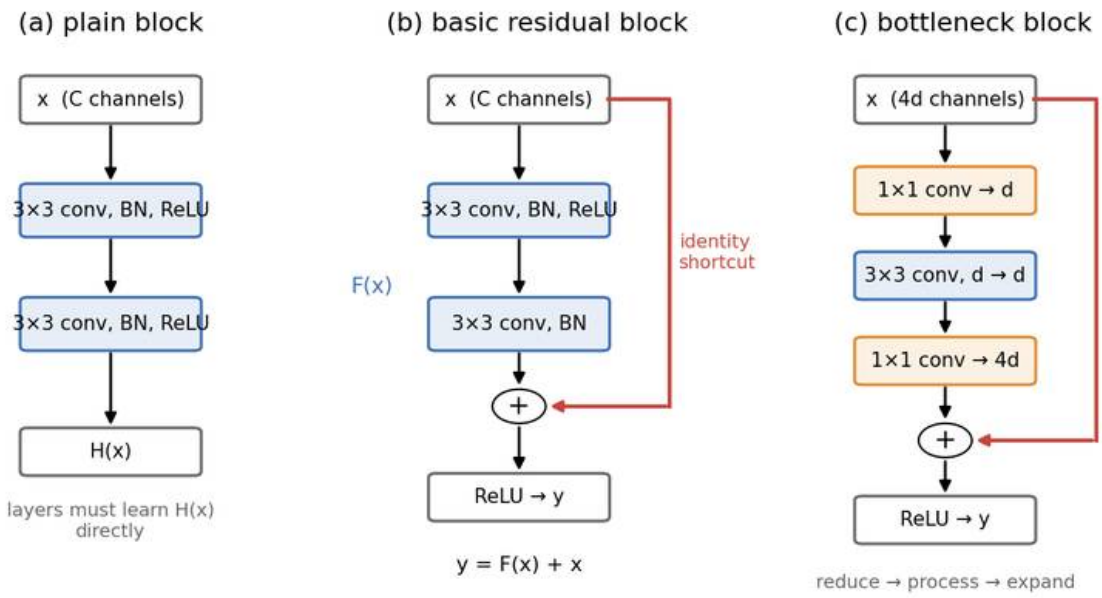


圖 4 — (*a*) 普通區塊必須學習完整的映射 $H(x)$。(*b*) 殘差區塊學習 $F(x) = H(x) - x$，再透過恆等捷徑把輸入加回去。(*c*) 瓶頸區塊在縮減後的寬度 d 上做 3×3 卷積，再升回 $4d$。

本篇實驗所用的基本區塊，以 PyTorch 寫成：

```
import torch.nn as nn
import torch.nn.functional as F

class BasicBlock(nn.Module):
    def __init__(self, cin, cout, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(cin, cout, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(cout)
        self.conv2 = nn.Conv2d(cout, cout, 3, 1, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(cout)
        self.short = nn.Identity()
        if stride != 1 or cin != cout:   # projection shortcut when shapes change
            self.short = nn.Sequential(nn.Conv2d(cin, cout, 1, stride, bias=False),
             ↪  nn.BatchNorm2d(cout))

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        return F.relu(out + self.short(x))   # y = F(x) + x
```

DenseNet：把所有東西都再利用

DenseNet [16] 把特徵再利用推得更遠：在一個密集區塊內，第 ℓ 層接收所有前面特徵圖的**串接**，$\mathbf{x}_\ell = H_\ell([\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{\ell-1}])$，其中 $[\cdot]$ 表示通道串接，H_ℓ 是 BN–ReLU–卷積。每層只新增少量通道

(「成長率」)，所以參數效率高，每一層到損失函數都有一條短路徑。代價是記憶體：串接的激活值隨深度增長，使 DenseNet 在實務上比參數量所暗示的還慢。這是之後會一再出現的教訓：參數量、FLOPs 與實際執行時間是三件不同的事。

MobileNet：把卷積拆開

MobileNet [17] 的目標是手機。在 $H \times W$ 的特徵圖上，從 M 個通道到 N 個通道的標準 $K \times K$ 卷積需要 HWK^2MN 次乘加。**深度可分離**（depthwise separable）卷積把它拆成兩步：**逐通道**（depthwise）$K \times K$ 卷積，每個通道各自濾波（HWK^2M）；以及**逐點**（pointwise）1×1 卷積，負責混合通道（$HWMN$）。兩者的成本比為

$$\frac{HWK^2M + HWMN}{HWK^2MN} = \frac{1}{N} + \frac{1}{K^2},$$

所以當 $K = 3$、輸出通道很多時，可分離版本大約便宜 8–9 倍。它押注的是：空間濾波與通道混合不需要一起學。這個賭注押得很成功，深度卷積後來在 EfficientNet 與 ConvNeXt 中再次出現。

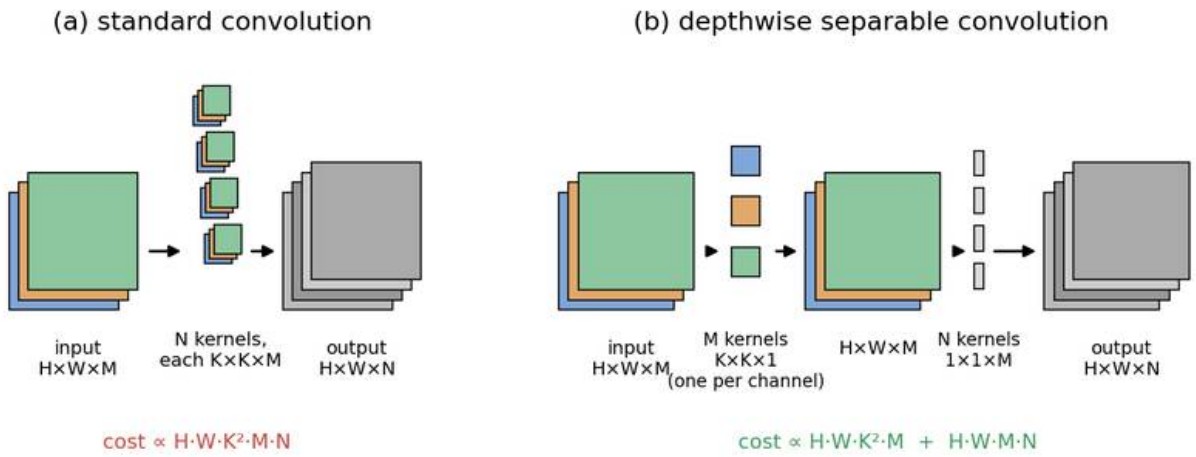


圖 *5* — 標準卷積在同一步裡完成濾波與通道混合。深度可分離卷積先對每個通道各自濾波（*depthwise*），再用 *1×1* 卷積核混合通道（*pointwise*）。

```
import torch.nn as nn

def n_params(m):
    return sum(p.numel() for p in m.parameters())

M, N, K = 128, 128, 3
standard = nn.Conv2d(M, N, K, padding=1, bias=False)
separable = nn.Sequential(nn.Conv2d(M, M, K, padding=1, groups=M, bias=False),   # depthwise
                          nn.Conv2d(M, N, 1, bias=False))                          #
                           ↪  pointwise
print("standard:", n_params(standard), " separable:", n_params(separable),
```

```
      f" ratio {n_params(separable) / n_params(standard):.3f} vs 1/N + 1/K^2 = {1/N +
      ↪  1/K**2:.3f}")
# standard: 147456  separable: 17536  ratio 0.119 vs 1/N + 1/K^2 = 0.119
```

EfficientNet：深度、寬度、解析度一起放大

手上有一個好的小網路，要怎麼把它變大？加深、加寬，還是提高輸入解析度？Tan 與 Le [18] 主張這三者互相影響：解析度更高的影像需要更深（更大的感受野）與更寬（更細的圖樣）。**複合縮放**（compound scaling）用一個係數 ϕ 同時放大三者：

$$d = \alpha^\phi, \quad w = \beta^\phi, \quad r = \gamma^\phi, \qquad \text{s.t. } \alpha\beta^2\gamma^2 \approx 2, \;\; \alpha, \beta, \gamma \geq 1,$$

其中 d、w、r 分別是深度、寬度與解析度的倍率。FLOPs 大約與 $d\,w^2 r^2$ 成正比，所以 ϕ 每加 1，成本大約加倍。常數是在基礎網路上做小型網格搜尋得到的（基礎網路本身由架構搜尋找出，使用帶深度卷積與 squeeze-and-excitation 的反向瓶頸區塊）。「平衡地放大」這個想法，比那些特定常數活得更久。

RegNet：設計空間，而不是單一設計

Radosavovic 等人 [19] 研究的是網路的族群，而不是單一贏家。他們從一個廣泛的類 ResNet 網路空間出發，反覆加上限制（例如共用瓶頸比例、各階段寬度遞增），並檢查錯誤率的分佈是否變好。最後留下的空間 RegNet，用一條簡單的量化線性規則描述各階段寬度。這是方法論上的教訓：好的架構遵循簡單的規律，而比較模型的分佈，比拿一個調好的模型去比另一個更可靠。

ConvNeXt：一步一步現代化的 CNN

在視覺 Transformer 拿下排行榜之後，Liu 等人 [20] 問：它們的優勢有多少來自注意力機制，又有多少來自其他一切？他們從一個用 Transformer 風格配方訓練的 ResNet-50 出發，一次只改一件事：仿照 Swin 的階段比例、「patchify」主幹（4×4、步幅 4 的卷積）、用更大的 7×7 深度卷積、反向瓶頸、更少的激活與正規化層、GELU 與 LayerNorm。結果 ConvNeXt 在 ImageNet 與下游任務上，追平了相近成本的 Swin 類 Transformer。教訓是：架構家族的差異，沒有伴隨而來的許多小設計與訓練選擇那麼重要。

注意力機制與 Transformer

白話版。卷積只用固定的權重看一個小鄰域。注意力機制讓每個位置根據內容，自己決定要聽哪些其他位置的話。視覺 Transformer 把這個機制套用到影像小塊上。

Squeeze-and-excitation：CNN 裡的注意力

SE 區塊 [21] 為任何 CNN 加上便宜、依內容而定的**通道**權重。它先用全域平均池化把每個通道「擠壓」成一個數字，經過一個小型兩層 MLP，再用 sigmoid 閘門乘回每個通道：

$$\mathbf{s} = \sigma(W_2\,\delta(W_1\,\bar{\mathbf{u}})), \qquad \tilde{\mathbf{u}}_c = s_c\,\mathbf{u}_c,$$

其中 $\mathbf{u}_c$ 是特徵圖的第 c 個通道，$\bar{\mathbf{u}}$ 是各通道平均值組成的向量，δ 是 ReLU，W_1 以比例 r 降維，W_2 再升回原維度。網路因此能在整體脈絡顯示「動物」時，加強「毛皮」相關的通道。SE 區塊是 EfficientNet 與許多行動裝置網路的一部分。

```
import torch
import torch.nn as nn

class SE(nn.Module):
    """Squeeze-and-excitation: reweight channels using global context."""
    def __init__(self, c, r=16):
        super().__init__()
        self.fc = nn.Sequential(nn.Linear(c, c // r), nn.ReLU(), nn.Linear(c // r, c),
        ↪  nn.Sigmoid())

    def forward(self, x):                       # x: (B, C, H, W)
        s = x.mean(dim=(2, 3))                  # squeeze: (B, C)
        w = self.fc(s)[:, :, None, None]        # excitation: (B, C, 1, 1) in (0, 1)
        return x * w

print(SE(64)(torch.randn(2, 64, 8, 8)).shape)
```

ViT：把影像當成一串小塊

視覺 Transformer（Vision Transformer, ViT）[22] 幾乎完全拿掉卷積。影像 $\mathbf{x} \in \mathbb{R}^{H\times W\times C}$ 被切成 $N = HW/P^2$ 個不重疊的 $P\times P$ 小塊（patch）。每個小塊攤平後線性投影成 D 維的 **token**。前面再加上一個可學習的**類別 token**，並加上可學習的**位置嵌入**：

$$\mathbf{z}_0 = [\,\mathbf{x}_{\text{class}};\ \mathbf{x}_p^1\mathbf{E};\ \dots;\ \mathbf{x}_p^N\mathbf{E}\,] + \mathbf{E}_{\text{pos}},$$

其中 $\mathbf{x}_p^i \in \mathbb{R}^{P^2C}$ 是第 i 個攤平的小塊，$\mathbf{E} \in \mathbb{R}^{P^2C\times D}$ 是小塊嵌入矩陣，$\mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1)\times D}$ 是位置嵌入。接著由 L 層標準 Transformer 編碼器（多頭自注意力與 MLP，各自帶 LayerNorm 與殘差連接）處理這個序列，最後由一個小型分類頭讀取類別 token 的最終狀態，輸出 logits。單一個注意力頭的自注意力為

$$\text{Attn}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{softmax}\!\Big(\frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d}}\Big)\mathbf{V},$$

其中 $\mathbf{Q} = \mathbf{Z}\mathbf{W}_Q$、$\mathbf{K} = \mathbf{Z}\mathbf{W}_K$、$\mathbf{V} = \mathbf{Z}\mathbf{W}_V$ 是 token 矩陣 $\mathbf{Z}$ 的線性投影，d 是 key 的維度。注意力矩陣大小為 $(N+1)\times(N+1)$：從第一層開始，每個小塊就能注意到所有其他小塊，所以感受野一開始就是全域的。

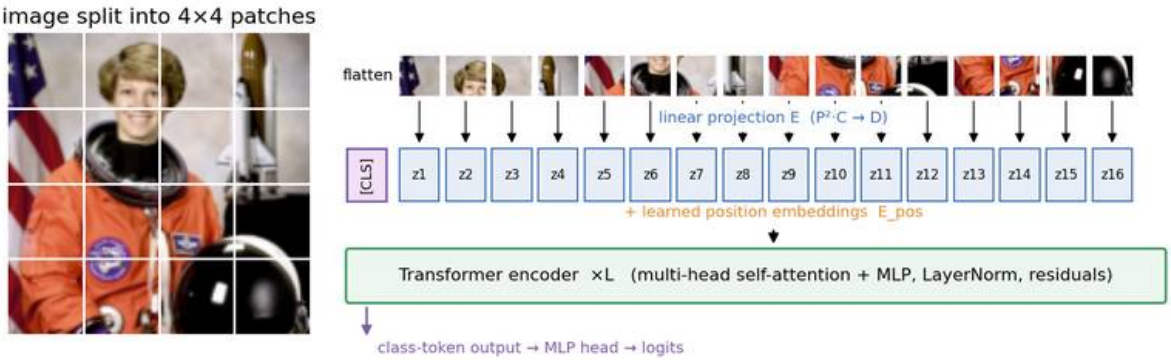


圖 *6 — ViT* 的輸入流程。所有小塊共用同一個線性投影變成 *token*（等價於核大小 $P\times P$、步幅 P 的卷積）。加上類別 *token* 與可學習的位置嵌入後，用類別 *token* 的最終狀態來分類。

```
import torch
import torch.nn as nn

B, C, H, W, P, D = 2, 3, 32, 32, 8, 64
img = torch.randn(B, C, H, W)

# 1) patchify: (B, C, H, W) -> (B, N, P*P*C) with N = (H/P)*(W/P)
patches = img.unfold(2, P, P).unfold(3, P, P)                  # (B, C, H/P, W/P, P, P)
patches = patches.permute(0, 2, 3, 1, 4, 5).reshape(B, -1, C * P * P)
N = patches.shape[1]

# 2) linear patch embedding + class token + learned position embedding
embed = nn.Linear(C * P * P, D)          # same as nn.Conv2d(C, D, kernel_size=P, stride=P)
cls = nn.Parameter(torch.zeros(1, 1, D))
pos = nn.Parameter(torch.randn(1, N + 1, D) * 0.02)
z = torch.cat([cls.expand(B, -1, -1), embed(patches)], dim=1) + pos   # (B, N+1, D)

# 3) one head of scaled dot-product self-attention
Wq, Wk, Wv = (nn.Linear(D, D, bias=False) for _ in range(3))
q, k, v = Wq(z), Wk(z), Wv(z)
A = (q @ k.transpose(1, 2) / D ** 0.5).softmax(dim=-1)         # (B, N+1, N+1), rows sum to 1
out = A @ v
print("tokens:", N + 1, " attention:", tuple(A.shape), " class-token output:", tuple(out[:,
 ↪  0].shape))
```

為什麼 ViT 需要大量資料。CNN 內建了一些假設，稱為**歸納偏差**（inductive bias）：局部性（一個像素主要和鄰居有關）與平移等變性（同一個濾波器用在每個位置）。ViT 幾乎沒有這些；除了切小塊與位置嵌入之外，空間結構都得從資料中學。ViT 論文 [22] 回報：只在 ImageNet-1k 上訓練時，ViT 落後於規模相近的 ResNet；但在大得多的資料集上預訓練（ImageNet-21k，或 3 億張影像的 JFT）之後，就能追平甚至超越。弱先驗在小規模時要付出資料的代價，到了大規模就不再是限制。同一篇論文也嘗試了**混合**（hybrid）模型：把 ResNet 主幹的特徵圖、而不是原始小塊，送進 Transformer；混合模型在計算預算小時較有幫助，模型變大後差距就消失了。

DeiT：只用 ImageNet 訓練 ViT

DeiT [23] 證明「ViT 需要巨量資料」這個結論，有一部分其實是關於訓練配方。用強力的資料擴增（RandAugment、Mixup、CutMix、random erasing）、隨機深度、重複擴增與 AdamW，只在 ImageNet-1k 上訓練的 ViT-B 就達到 83.1% top-1。DeiT 還加入了**蒸餾 token**：第二個可學習的 token，和類別 token 一樣，但它的輸出被訓練去匹配教師網路的預測（他們發現卷積網路當教師效果最好），而類別 token 則以真實標籤訓練。這是知識蒸餾 [51] 的一種 Transformer 專屬形式。測試時把兩個頭的輸出結合起來。

Swin：視窗、位移與階層

全域注意力的成本隨 token 數量呈平方成長，對高解析度輸入與密集預測任務是個問題。Swin [24] 只在不重疊的 $M \times M$ 視窗內計算注意力，並在相鄰層之間把視窗網格位移半個視窗，讓資訊能跨越視窗邊界。各階段之間合併 2×2 相鄰小塊來減少 token 數，形成步幅為 4、8、16、32 的類 CNN 金字塔。對 $h \times w$ 個 token、C 個通道，論文給出

$$\Omega(\text{MSA}) = 4hwC^2 + 2(hw)^2C, \qquad \Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC,$$

所以在視窗大小 M 固定時，視窗注意力對 token 數 hw 是線性的。Swin 帶回了兩個 CNN 先驗：局部性與階層性，成為偵測與分割也通用的骨幹。視覺 Transformer 的整體版圖可參考 Khan 等人的綜述 [25]。

訓練配方

白話版。同一個網路，訓練方法不同，準確率可以差好幾個百分點。現代訓練配方是一組小技巧，每一個都讓「死背」更難，或讓最佳化更平順。

資料擴增

- **隨機縮放裁切**（random resized crop）隨機取一塊占原圖面積一定比例、長寬比隨機的區域，再縮放到訓練解析度。它教會網路尺度與位置不變性，是 ImageNet 訓練的預設做法。**水平翻轉**只在不改變標籤時使用（大多數照片可以，文字或數字不行）。
- **Mixup** [26] 混合兩張影像與它們的標籤：$\tilde{\mathbf{x}} = \lambda \mathbf{x}_i + (1-\lambda)\mathbf{x}_j$、$\tilde{\mathbf{y}} = \lambda \mathbf{y}_i + (1-\lambda)\mathbf{y}_j$，其中 $\lambda \sim \text{Beta}(\alpha, \alpha)$，$\mathbf{y}$ 為 one-hot。它鼓勵模型在訓練樣本之間呈線性行為。
- **CutMix** [27] 把影像 j 的一個隨機矩形貼到影像 i 上，並依貼上的面積比例混合標籤。和 Mixup 不同，每個像素都仍是自然的，網路必須從局部視野辨認物體。
- **RandAugment** [28] 用兩個數字取代學出來的擴增策略：隨機套用 N 個操作（旋轉、剪切、色階分離、對比……），共用同一個強度 M。論文回報這個極小的搜尋空間就能追平或超越先前的自動化策略，而合適的強度取決於模型與資料集大小。

```
import numpy as np
import torch
import torch.nn.functional as F
```

```
def mixup(x, y, num_classes, alpha=0.2):
    lam = float(np.random.beta(alpha, alpha))
    idx = torch.randperm(x.size(0))
    y1 = F.one_hot(y, num_classes).float()
    return lam * x + (1 - lam) * x[idx], lam * y1 + (1 - lam) * y1[idx]

def cutmix(x, y, num_classes, alpha=1.0):
    lam = float(np.random.beta(alpha, alpha))
    idx = torch.randperm(x.size(0))
    H, W = x.shape[-2:]
    h, w = int(H * np.sqrt(1 - lam)), int(W * np.sqrt(1 - lam))
    cy, cx = np.random.randint(H), np.random.randint(W)
    y0, y1_, x0, x1_ = max(cy - h // 2, 0), min(cy + h // 2, H), max(cx - w // 2, 0), min(cx
    ↪  + w // 2, W)
    x = x.clone()
    x[..., y0:y1_, x0:x1_] = x[idx][..., y0:y1_, x0:x1_]
    lam = 1 - (y1_ - y0) * (x1_ - x0) / (H * W)            # actual area kept
    y1 = F.one_hot(y, num_classes).float()
    return x, lam * y1 + (1 - lam) * y1[idx]

def soft_ce(logits, soft_targets):
    return -(soft_targets * logits.log_softmax(1)).sum(1).mean()
```

標籤平滑

以 one-hot 為目標的交叉熵會永遠把正確 logit 往上推；只有在間距無限大時，損失才會到零。**標籤平滑** [13] 把目標換成

$$\mathbf{q} = (1-\varepsilon)\,\mathbf{e}_y + \frac{\varepsilon}{K}\mathbf{1},$$

其中 ε（常用 0.1）是平滑強度，$\mathbf{e}_y$ 是 one-hot 向量，$\mathbf{1}$ 是全 1 向量。此時最佳解是有限的 logit 間距，對網路有正則化效果，通常也會減少過度自信。在 PyTorch 裡只是一個參數：`F.cross_entropy(logits, y, label_smoothing=0.1)`。

最佳化器、排程與正則化

- **AdamW** [29] 把權重衰減與自適應梯度步驟解耦：不是把 $\lambda\theta$ 加到梯度上（Adam 會再對每個參數重新縮放），而是直接縮小權重，$\theta \leftarrow \theta - \eta(\hat{\mathbf{m}}/(\sqrt{\hat{\mathbf{v}}}+\epsilon) + \lambda\theta)$，其中 η 是學習率，$\hat{\mathbf{m}}, \hat{\mathbf{v}}$ 是 Adam 的動差估計，λ 是衰減係數。它是 Transformer 與現代 CNN 配方的預設選擇。偏差與正規化層的參數通常不做衰減。
- **暖身＋餘弦排程**。訓練最初幾步，正規化統計量與 Adam 的動差估計都還不準，大學習率會不穩定，所以先讓學習率在幾個 epoch 內線性上升，接著沿半個餘弦曲線降到接近零，$\eta_t = \frac{1}{2}\eta_{\max}(1+\cos(\pi t/T))$，$t$ 是暖身後的步數，T 是剩餘步數；這個想法由 SGDR [30] 推廣開來。
- **隨機深度**（stochastic depth）[31] 在訓練時隨機跳過整個殘差分支（區塊變成恆等映射），跳過機率隨深度增加。它縮短了訓練時的有效深度，效果類似一群較淺網路的集成。

- **權重的 EMA** 維護一個指數移動平均 $\bar{\theta} \leftarrow \mu\bar{\theta} + (1-\mu)\theta$（$\mu$ 接近 1，例如 0.999），評估時使用 $\bar{\theta}$。它把最後幾步 SGD 的雜訊平滑掉。

```
import math
import torch

def warmup_cosine(step, total, warmup):
    if step < warmup:
        return (step + 1) / warmup                      # linear warmup
    p = (step - warmup) / max(1, total - warmup)
    return 0.5 * (1 + math.cos(math.pi * p))            # cosine decay to 0

model = torch.nn.Linear(10, 3)
decay = [p for n, p in model.named_parameters() if p.ndim > 1]      # weights
no_decay = [p for n, p in model.named_parameters() if p.ndim <= 1]  # biases, norm scales
opt = torch.optim.AdamW([{"params": decay, "weight_decay": 0.05},
                         {"params": no_decay, "weight_decay": 0.0}], lr=1e-3)
total = 1000
sched = torch.optim.lr_scheduler.LambdaLR(opt, lambda s: warmup_cosine(s, total, warmup=50))
ema = torch.optim.swa_utils.AveragedModel(model,
 ↪  multi_avg_fn=torch.optim.swa_utils.get_ema_multi_avg_fn(0.999))
for step in range(total):
    loss = model(torch.randn(16, 10)).pow(2).mean()
    opt.zero_grad(); loss.backward(); opt.step(); sched.step(); ema.update_parameters(model)
```

一個小實驗

為了看這些元件一起運作，我們在 scikit-learn 的 8×8 手寫數字上，訓練了一個 7.8 萬參數的 ResNet（「TinyResNet」：一個主幹、三個通道數分別為 16、32、64 的基本區塊、全域平均池化）。影像先放大到 32×32，讓有步幅的階段有空間運作。為了讓過擬合看得見，我們只用 **100 張訓練影像**，另外 500 張用於校準、1,197 張用於測試；使用 AdamW（權重衰減 0.05）、批次大小 32、40 個 epoch，先暖身兩個 epoch 再做餘弦衰減。一組完全不做擴增；另一組使用隨機旋轉（$\pm 15°$）、縮放與平移（類似隨機縮放裁切的仿射變換；不翻轉，因為翻轉後的數字是另一個符號），並搭配標籤平滑 0.1。所有程式碼都在 `scripts/figures/dl_02_image_classification.py`，在 CPU 上大約一分鐘跑完。

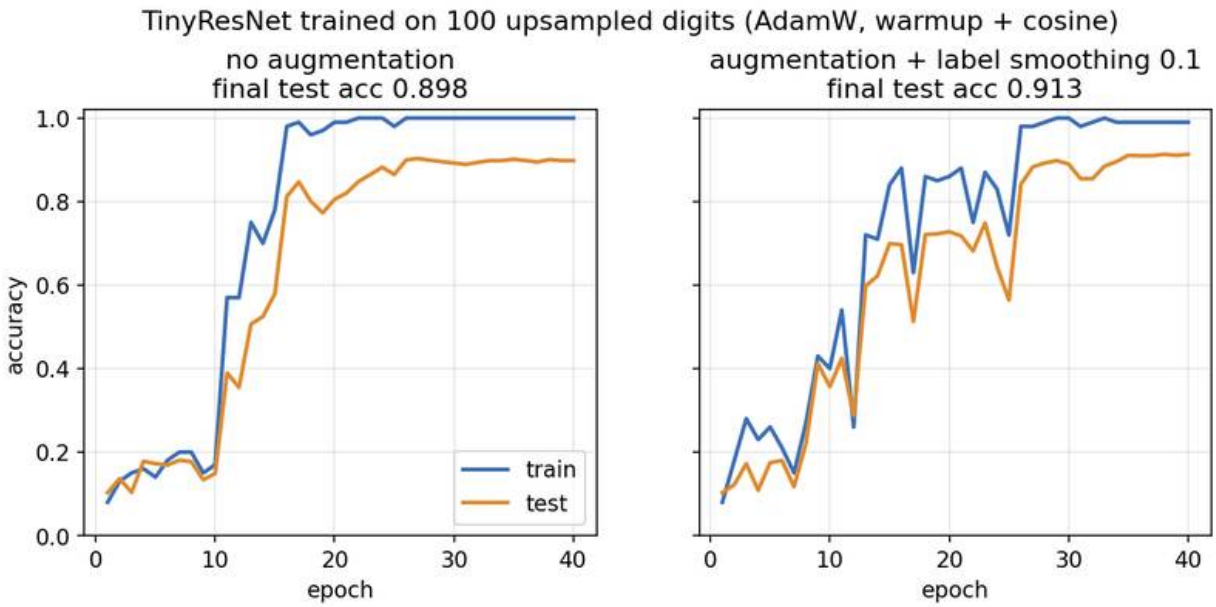


圖 7 — 以 *100* 張數字影像訓練的學習曲線。沒有擴增（左）時，網路背下了整個訓練集，測試準確率 *89.8%*。加上擴增與標籤平滑（右）後達到 *91.3%*。在這麼簡單的任務上差距不大；在自然影像上，配方的影響大得多。

沒有擴增的模型大約二十個 epoch 後就達到 100% 訓練準確率：只有 100 張影像，背起來很容易。有擴增的模型每個 epoch 看到的都是稍微不同的數字，學得比較慢，泛化稍好（91.3% 對 89.8%）。單一隨機種子上 1.5 個百分點不必過度解讀；重點是機制。

「ResNet strikes back」

配方重要性最驚人的證據，來自 Wightman、Touvron 與 Jégou [32]。他們保持 2015 年的 ResNet-50 架構不變，用現代配方重新訓練：隨機縮放裁切與翻轉、RandAugment、Mixup 與 CutMix、LAMB 最佳化器配餘弦排程、二元交叉熵損失，以及更長的訓練。原汁原味的 ResNet-50 在 224×224 的 ImageNet 上，不用額外資料或蒸餾就達到 **80.4% top-1**，而他們比較的標準 torchvision 權重是 76.1%。這帶來兩個結論。第一，許多「新架構打敗 ResNet-50」的宣稱，其實是拿用新配方訓練的新模型，去比用舊配方訓練的舊模型。第二，比較架構時，要用相同且調好的配方訓練它們，就像 ConvNeXt [20] 與 RegNet [19] 的做法。

遷移學習與微調

白話版。從頭訓練一個骨幹網路需要大量資料。比較好的做法是：從一個已經在大型資料集上訓練好的網路出發，再把它調整到你的任務上。你可以凍結它的特徵、只訓練新的最後一層，也可以溫和地重新訓練所有參數。

令 g_ϕ 為一個預訓練骨幹，把影像映射成特徵向量 $\mathbf{h} = g_\phi(\mathbf{x}) \in \mathbb{R}^D$，$\mathbf{W} \in \mathbb{R}^{K\times D}$ 為新的線性分類頭。

- **線性探測**（linear probe）。凍結 ϕ，只訓練 $\mathbf{W}$（等於在固定特徵上做邏輯迴歸）。它便宜、需要的標籤少，也是衡量表徵好壞的標準方法。
- **完整微調**（full fine-tuning）。同時更新 ϕ 與 $\mathbf{W}$，通常骨幹的學習率比分類頭小 10–100 倍，搭配短暫的暖身，有時還會用逐層學習率衰減（越前面的層學習率越小）。它讓特徵適應新的領域，標註資料夠多時通常勝出。
- 介於兩者之間的，有部分微調（只調最後幾個階段），以及在凍結骨幹上加入小型可訓練模組的參數高效方法。

Kornblith 等人 [33] 量測了 ImageNet 準確率能多好地預測遷移表現。對微調而言，ImageNet 上越好的模型通常遷移得越好，相關性很強。對固定特徵而言，相關性較弱，而且取決於正則化選擇：有些提升 ImageNet 準確率的設定（例如標籤平滑與 dropout），反而產生了在其他任務上更差的凍結特徵。好的分類器與好的特徵擷取器相關，但不是同一件事。

```
import torch
import torch.nn as nn

backbone = nn.Sequential(nn.Conv2d(3, 32, 3, 2, 1), nn.ReLU(), nn.Conv2d(32, 64, 3, 2, 1),
 ↪  nn.ReLU(),
                          nn.AdaptiveAvgPool2d(1), nn.Flatten())          # stand-in for a
                           ↪  pre-trained model
head = nn.Linear(64, 5)

# linear probe: freeze the backbone, train only the head
for p in backbone.parameters():
    p.requires_grad = False
probe_opt = torch.optim.AdamW(head.parameters(), lr=1e-3)

# full fine-tuning: unfreeze, use a smaller learning rate for the backbone than for the head
for p in backbone.parameters():
    p.requires_grad = True
ft_opt = torch.optim.AdamW([{"params": backbone.parameters(), "lr": 1e-5},
                            {"params": head.parameters(), "lr": 1e-3}], weight_decay=0.05)
```

自監督學習與基礎模型分類器

白話版。標籤很貴，影像很便宜。自監督學習自己發明一個答案就藏在影像裡的任務（這兩塊裁切是不是來自同一張照片？被遮住的地方原本是什麼？），透過解這個任務學到特徵，最後才用少量標籤。

對比式學習：SimCLR 與 MoCo

對比式方法把同一張影像的兩個擴增**視圖**的嵌入拉近，把不同影像的視圖推遠。SimCLR [34] 取一個 B 張影像的批次，為每張產生兩個隨機視圖，並對每一對正樣本 (i, j) 最小化 InfoNCE（NT-Xent）損失：

$$\ell_{i,j} = -\log \frac{\exp(\mathrm{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2B} \mathbb{1}_{[k \neq i]} \exp(\mathrm{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)},$$

其中 $\mathbf{z}$ 是投影後的嵌入，sim 是餘弦相似度，τ 是溫度，批次中其他 $2B-2$ 個視圖當作負樣本。這其實是一個 $2B-1$ 類的 softmax 分類：「這些之中哪一個是我的搭檔？」SimCLR 發現強力擴增（特別是裁切加上顏色扭曲）、非線性投影頭與大批次都不可或缺。MoCo [35] 則維護一個由緩慢更新的**動量編碼器**（其權重是主編碼器的 EMA）編碼的負樣本**佇列**，避免了對超大批次的需求。

DINO 與 DINOv2：自我蒸餾

DINO [36] 不再使用明確的負樣本。學生網路看幾個裁切（包括小的局部裁切），被訓練去匹配看全域裁切的教師網路的輸出分佈。教師是學生的 EMA；對教師輸出做中心化與銳化，可以防止崩塌成常數。用 ViT 時，DINO 特徵展現出驚人的性質：類別 token 的注意力圖不需任何分割標籤就能把物體分割出來，而在凍結特徵上用簡單的 k-NN 分類器效果就很好。DINOv2 [38] 以自動篩選的資料集與一個大型 ViT（再蒸餾成較小的模型）擴大規模，產生通用的**凍結特徵**，不需微調就能用於分類、分割與深度估計。DINOv3 [39] 繼續擴大規模，並提出「Gram anchoring」，避免密集的小塊特徵在長時間訓練中劣化。

MAE：遮罩影像建模

MAE [37] 借用了遮罩語言模型的想法。它遮掉大比例的小塊（論文中為 75%），只把看得見的小塊送進 ViT 編碼器，再讓一個輕量解碼器重建被遮掉的像素。因為編碼器跳過被遮掉的 token，預訓練很便宜。MAE 特徵的線性可分性不如對比式特徵（線性探測較弱），但**微調**效果非常好。這是一再出現的模式：不同的預訓練目標，適合不同的適應方式。

CLIP：用語言做零樣本分類

CLIP [40] 用數億組影像與說明文字，以對稱的對比損失同時訓練影像編碼器 g 與文字編碼器 h，讓配對的影像與文字有高的餘弦相似度。之後**不需要任何訓練影像**就能建立分類器：為每個類別寫一句提示，例如 `a photo of a dog.`，將它嵌入，然後預測

$$\hat{y} = \arg\max_k \frac{g(\mathbf{x})^\top h(t_k)}{\|g(\mathbf{x})\| \, \|h(t_k)\|},$$

其中 t_k 是類別 k 的提示。文字嵌入扮演線性分類器權重的角色，只是它們是從文字算出來的。論文回報零樣本 CLIP 的準確率追平了一個用 ImageNet 128 萬張標註影像訓練的 ResNet-50，而平均多個提示模板（「a photo of a ...」、「a blurry photo of a ...」）會有幫助。零樣本準確率高度依賴提示的用字、類別名稱在視覺上是否有意義，以及網路資料對該領域的涵蓋程度。

穩健性與評估陷阱

白話版。一個在測試集上拿 90% 的分類器，碰到不同相機拍的照片、有人刻意加上肉眼看不見的雜訊，或它學到的「簡單線索」（背景、浮水印）不見時，可能會錯得一塌糊塗。而且就算答對

了，它的信心也可能不對。

分佈偏移：ImageNet-C、-R 與 -A

Hendrycks 等人為 ImageNet 分類器建立了三個廣泛使用的偏移測試集。**ImageNet-C** [42] 套用 15 種演算法產生的損壞（雜訊、模糊、天氣、數位瑕疵），各有五個嚴重程度，並以相對於基準模型的錯誤率總結（mean corruption error）。**ImageNet-R** [43] 收集了 200 個 ImageNet 類別的各種表現形式（藝術、卡通、雕塑、素描、玩具），測試模型理解的是形狀與概念，還是照片的紋理。**ImageNet-A** [44] 收錄未經修改的自然照片，這些照片經過篩選、專門騙倒 ResNet-50；標準模型在上面表現非常差。這三個測試集揭露不同的弱點，改善其中一個不保證改善其他。大規模預訓練（如 CLIP 與 DINOv2）對這類自然偏移的穩健性提升，往往遠大於在相同資料下更換架構。

對抗樣本：FGSM

Goodfellow 等人 [45] 用網路在高維空間中的**線性**來解釋對抗樣本：對每一個輸入維度做一點點、與權重方向一致的改變，累加起來就是輸出的巨大變化。**快速梯度符號法**（fast gradient sign method, FGSM）一步就完成擾動：

$$\mathbf{x}_{\text{adv}} = \text{clip}\left(\mathbf{x} + \varepsilon \, \text{sign}(\nabla_{\mathbf{x}} \mathcal{L}(f_\theta(\mathbf{x}), y))\right),$$

其中 ε 限制每個像素的改變量（ℓ_∞ 範數），$\mathcal{L}$ 是損失函數，clip 把像素值限制在有效範圍內。

```
import torch.nn.functional as F

def fgsm(model, x, y, eps):
    x = x.clone().requires_grad_(True)
    F.cross_entropy(model(x), y).backward()
    return (x + eps * x.grad.sign()).clamp(-1, 1).detach()
```

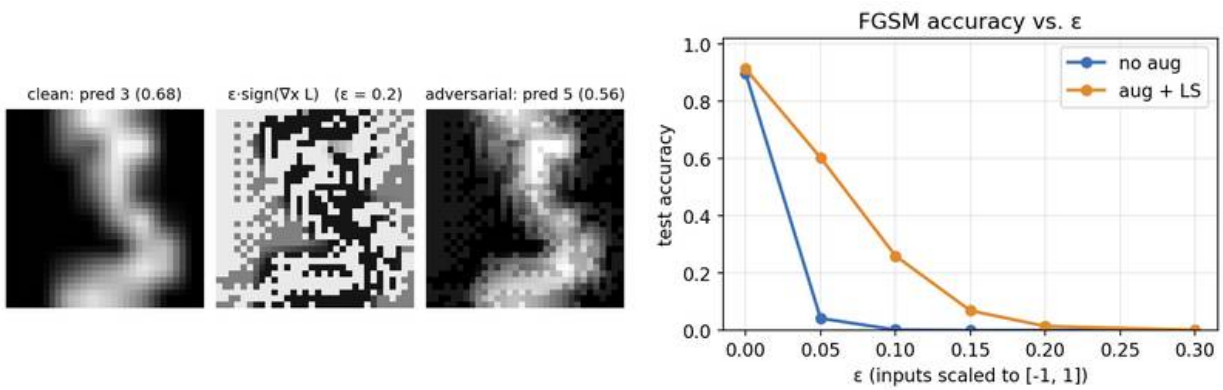


圖 *8* — 在 *TinyResNet* 模型上的 *FGSM*（輸入範圍為 $[-1, 1]$）。$\varepsilon = 0.05$ 時，未擴增模型的測試準確率從 *89.8%* 掉到 *4.2%*，擴增＋標籤平滑模型從 *91.3%* 掉到 *60.3%*；$\varepsilon = 0.2$ 時兩者都接近零。

關於圖 8 有兩點要注意。第一，擴增模型在小 ε 時看起來比較穩健，但已知標籤平滑會讓訓練點附近的損失曲面變平，可能削弱單步梯度攻擊，卻沒有帶來真正的穩健性；正確的檢驗是更強的迭代攻擊。第二，在 $[-1, 1]$ 範圍內 $\varepsilon = 0.05$ 對 8×8 的數字來說是看得見的；在自然影像上，肉眼無法察覺的擾動就足以騙過標準的 ImageNet 模型。

捷徑學習

Geirhos 等人 [46] 把許多失敗案例歸在同一個名字下：**捷徑學習**（shortcut learning）。網路會用最容易的線索來最小化訓練損失，而那常常不是我們想要的線索：看背景而不是動物、看醫院的標記而不是病灶、看紋理而不是形狀。捷徑在獨立同分佈的測試集上看起來沒問題，因為測試集裡也有同樣的捷徑；一遇到分佈偏移就會失敗。對策是：刻意在分佈外測試、設計能打破可疑捷徑的測試，並用 Grad-CAM（見下文）這類可解釋性工具檢查模型在看哪裡。

校準與溫度縮放

溫度縮放（temperature scaling）[47] 在 softmax 之前把所有 logits 除以同一個純量 $T > 0$：$\mathbf{p} = \mathrm{softmax}(\mathbf{z}/T)$。$T$ 是在保留的**校準**集（絕不是測試集）上最小化負對數概似而得。$T > 1$ 讓過度自信的預測變柔和；$T < 1$ 讓信心不足的預測變銳利。因為除以 T 不改變 argmax，準確率不變。

```
import torch
import torch.nn.functional as F

def fit_temperature(logits, y, iters=300):
    """Learn one scalar T > 0 on a held-out calibration split."""
    log_t = torch.zeros(1, requires_grad=True)
    opt = torch.optim.Adam([log_t], lr=0.05)
    for _ in range(iters):
        loss = F.cross_entropy(logits / log_t.exp(), y)
        opt.zero_grad(); loss.backward(); opt.step()
    return log_t.exp().item()

def ece(probs, y, n_bins=10):
    conf, pred = probs.max(1)
    correct = (pred == y).float()
    edges = torch.linspace(0, 1, n_bins + 1)
    total = 0.0
    for lo, hi in zip(edges[:-1], edges[1:]):
        m = (conf > lo) & (conf <= hi)
        if m.any():
            total += m.float().mean() * (correct[m].mean() - conf[m].mean()).abs()
    return float(total)

# toy check: an over-confident "model" = true logits multiplied by 3
torch.manual_seed(0)
true_logits = torch.randn(4000, 10)
y = torch.distributions.Categorical(logits=true_logits).sample()
over = 3 * true_logits
```

```
T = fit_temperature(over[:2000], y[:2000])
print(f"T = {T:.2f}  ECE {ece(over[2000:].softmax(1), y[2000:]):.3f} -> "
      f"{ece((over[2000:] / T).softmax(1), y[2000:]):.3f}")
# T = 2.90  ECE 0.345 -> 0.018
```

在這個玩具例子裡，logits 被放大了三倍，擬合出的溫度（2.90）找回了這個倍數。而在我們真正的 TinyResNet（未擴增的那個模型）上，校準誤差的方向相反。

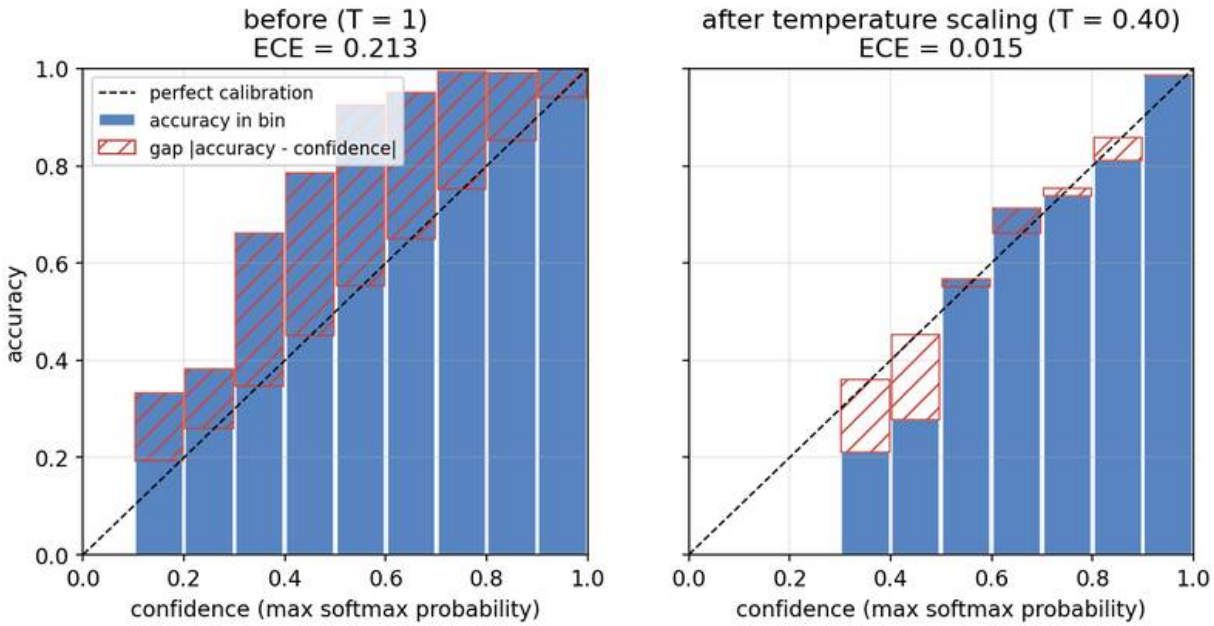


圖 *9* — 未擴增 *TinyResNet* 在 *1,197* 張測試數字上的可靠度圖。縮放前（左）多數區間的準確率高於信心：模型信心不足（*ECE 0.213*）。在另外 *500* 張校準影像上擬合出的溫度 $T = 0.40$，讓 *ECE* 降到 *0.015*，測試 *NLL* 從 *0.515* 降到 *0.298*，而且沒有改變任何一個預測。

為什麼信心不足？我們的小模型訓練時間短、權重衰減又強，使 logits 保持很小。Guo 等人 [47] 觀察到的是大型、長時間訓練的現代網路過度自信，並指出深度、寬度、批次正規化與較小的權重衰減都是成因。校準誤差沒有固定的方向；要量測它。另外也要注意，校準本身在分佈偏移下很脆弱：在分佈內資料上擬合的溫度，不保證在 ImageNet-C 式的偏移上仍然校準良好。

效率

白話版。模型必須放得進裝置、跑得進時間預算。參數量、計算量與實際速度是三種不同的量測，而使用者感受到的只有最後一種。

- **參數量**決定模型在磁碟與記憶體中的大小。一個 float32 參數占 4 個位元組。
- **FLOPs**（或乘加運算 MACs；一次 MAC 通常算作兩個 FLOPs）衡量每張影像的運算量。對核大小 $K \times K$、輸入 C_{in} 通道、輸出 C_{out} 通道、輸出特徵圖為 $H' \times W'$ 的卷積，MACs $= H'W'K^2C_{\text{in}}C_{\text{out}}$。計算量隨解析度的平方成長。

- **延遲與吞吐量**要在目標硬體上量測。深度卷積的 FLOPs 很少，但運算密度低，所以在 GPU 上常比 FLOPs 所暗示的還慢；DenseNet 的串接則耗費記憶體頻寬。一定要實測。
- **量化**（quantization）把權重與激活值存成 8 位元整數，而不是 32 位元浮點數。Jacob 等人 [50] 描述了只用整數運算的推論，搭配量化感知訓練：在訓練時模擬捨入，讓網路學會容忍它。用少量校準資料做訓練後量化，則是較便宜的替代方案。
- **蒸餾** [51] 訓練一個小的學生網路去匹配大型教師網路的軟預測，就像 DeiT 用蒸餾 token 所做的那樣。

可解釋性：CAM 與 Grad-CAM

白話版。疊在影像上的熱圖，顯示哪些區域把某個類別的分數推高了。它是健全性檢查，不是推理過程的證明。

類別激活映射（class activation mapping, CAM）[48] 適用於以全域平均池化加一層線性層結尾的網路。若 A^k 是最後一個卷積層的第 k 個特徵圖，w_k^c 是它連到類別 c 的權重，則類別分數為 $\sum_k w_k^c \cdot \mathrm{mean}(A^k)$，而 $\sum_k w_k^c A^k$ 這張圖顯示分數來自哪裡。**Grad-CAM** [49] 用空間平均的梯度取代這些權重，把方法推廣到任何可微分的架構：

$$\alpha_k^c = \frac{1}{Z}\sum_{i,j}\frac{\partial y^c}{\partial A_{ij}^k}, \qquad L^c_{\text{Grad-CAM}} = \mathrm{ReLU}\Big(\sum_k \alpha_k^c A^k\Big),$$

其中 y^c 是類別 c 的 logit，A_{ij}^k 是通道 k 在位置 (i,j) 的激活值，Z 是位置總數。ReLU 只保留對該類別有正面影響的區域。粗糙的熱圖（我們的網路中是 8×8）再上採樣到影像大小。

```
import torch.nn.functional as F

def grad_cam(model, x, target=None):
    model.eval()
    feats = model.features(x)                          # (1, K, h, w) last conv stage
    feats.retain_grad()
    logits = model.fc(feats.mean(dim=(2, 3)))
    c = logits.argmax(1).item() if target is None else target
    logits[0, c].backward()
    alpha = feats.grad.mean(dim=(2, 3), keepdim=True)  # channel importance weights
    cam = F.relu((alpha * feats).sum(1, keepdim=True))
    cam = F.interpolate(cam, size=x.shape[-2:], mode="bilinear", align_corners=False)
    return (cam / (cam.max() + 1e-8))[0, 0].detach(), c
```

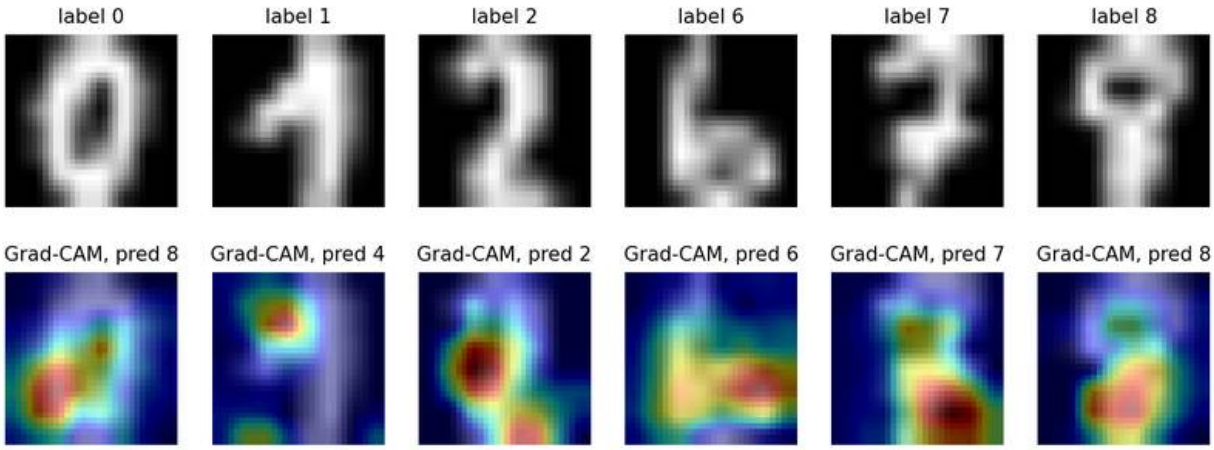


圖 *10* — 以擴增版 *TinyResNet*、對六個數字各自的第一個測試樣本，畫出預測類別的 *Grad-CAM*。其中兩個分錯了：*0* 被判成 *8*，熱區落在圓圈的左下部分；*1* 被判成 *4*，熱區落在它左上方的勾上。熱圖指出了誤導性證據被找到的位置。

解釋方法在模型出錯時最有用，就像圖 10：它們顯示了哪些筆畫看起來像錯誤的類別。但對顯著圖要謹慎。它們很粗糙（受限於最後一層特徵圖的解析度），而且只解釋單一輸入的單一類別分數，不是模型的一般推理方式。

現代觀點

值得一讀的綜述。 Li 等人 [1] 從 CNN 的歷史與基本構成元件談起，介紹經典與進階模型並點出各自的關鍵想法，再以實驗比較激活函數、損失函數與最佳化器，整理出選擇時的經驗法則；最後討論一維、二維與多維卷積的應用，以及一系列開放問題。Khan 等人 [25] 綜述各種任務上的視覺 Transformer。在分類方面，他們把模型分成：單一尺度模型（ViT、DeiT）、多尺度階層模型（如 Swin）、結合卷積的混合模型，以及自監督 Transformer（如 DINO），並強調它們對大規模預訓練的依賴，以及平方複雜度注意力的成本。Gui 等人 [41] 綜述自監督學習，把它分成：以脈絡為基礎的方法（旋轉預測、拼圖等前置任務）、對比式方法（使用負樣本對、像 DINO 的自我蒸餾，或特徵去相關）、生成式方法（像 MAE 的遮罩影像建模），以及結合對比與生成目標的混合方法，並討論與半監督、多模態學習的結合。在基準可靠性方面，ImageNetV2 研究 [7]、Beyer 等人的重新標註研究 [8] 與 Northcutt 等人的標籤錯誤分析 [9] 共同主張：使用多個測試集、多標籤評估，並回報變異。

2023–2026 年的樣貌。五個趨勢定義了當前的實務。

1. **預訓練一次，便宜地適應。**大多數新的分類器都不是從頭訓練。它們從一個以自監督（DINOv2 [38]、DINOv3 [39]）、影像–文字對比（CLIP [40] 及其開源重現）或遮罩建模（MAE [37]）訓練的大型骨幹出發，再以線性探測、*k*-NN 規則或短暫微調來適應。
2. **零樣本與開放詞彙分類。**影像–文字模型把標籤集合變成文字。類別清單可以在測試時更換、不需重新訓練，模糊了分類與檢索之間的界線。
3. **視覺的縮放定律。** Zhai 等人 [52] 把 ViT 擴大到數十億參數，刻畫了錯誤率如何隨計算量、資料量與模型大小下降，並觀察到較大的模型在少樣本遷移中更有樣本效率。Cherti 等人 [53]

在公開資料上為開源 CLIP 訓練擬合出可重現的冪律，發現縮放行為取決於訓練資料分佈與下游任務。

4. **架構趨於收斂**。ConvNeXt [20] 與 Swin [24] 顯示，在相同配方與規模下，卷積與注意力會落在相近的位置。現在的選擇更多取決於硬體、解析度與下游任務（密集預測偏好階層式設計），而不是 top-1 準確率。
5. **評估不再只看 ImageNet-1k top-1**。穩健性測試組（ImageNet-C/R/A [42]–[44] 與 ImageNetV2 [7]）、校準，以及遷移到大量下游資料集，現在都是骨幹論文的標準內容。

開放問題。對自然分佈偏移的穩健性仍落後於乾淨資料上的準確率，而穩健訓練方法帶來的進步往往只針對它所瞄準的那種偏移。沒有反事實測試，捷徑學習 [46] 很難被發現。分佈偏移下的校準仍未解決。長尾與細粒度辨識依然困難，因為稀有類別的樣本最少。基準飽和與標籤雜訊，讓 ImageNet-1k 上的微小準確率差異失去意義。而主流的基礎模型是用網路規模的資料訓練的，其組成、授權與偏差都很難稽核，這直接影響「零樣本」分類器能認出什麼、認不出什麼。

重點整理

- 分類器輸出 logits；單標籤任務用 softmax ＋交叉熵訓練，多標籤任務用獨立 sigmoid ＋二元交叉熵訓練。回報 top-1 準確率，必要時也回報校準（ECE）與逐類別結果。
- 每個里程碑 CNN 都修正了一個瓶頸：ReLU ／ GPU（AlexNet）、疊加的小卷積核（VGG）、1×1 降維與多分支模組（Inception）、為最佳化而設的恆等捷徑（ResNet）、為成本而設的深度可分離卷積（MobileNet）、平衡縮放（EfficientNet）。
- ViT 用卷積的歸納偏差換取彈性；它需要更多資料或更強的配方（DeiT），而視窗加階層（Swin）讓它成為實用的骨幹。ConvNeXt 顯示配方解釋了大部分的差距。
- 訓練配方和架構一樣重要：資料擴增（隨機縮放裁切、Mixup、CutMix、RandAugment）、標籤平滑、AdamW、暖身＋餘弦、隨機深度與 EMA，讓架構不變的 ResNet-50 達到 80.4% top-1。
- 預訓練骨幹是預設的起點。線性探測衡量表徵；微調讓表徵適應任務。CLIP 把類別名稱變成分類器權重，實現零樣本辨識。
- 準確率會掩蓋模型犯的是哪一種錯。請看混淆矩陣；類別不平衡時回報精確率／召回率、平衡準確率或 MCC，並優先使用精確率–召回率曲線與 AP，而不是 ROC-AUC。說清楚平均是巨觀（每個類別同等）還是微觀（每張影像同等；在單標籤任務中等於準確率）。
- 把訓練資料與評估資料分開：用一整組測試集評估骨幹網路（ImageNet 驗證集與 ReaL、ImageNetV2、ImageNet-C/R/A/Sketch、ObjectNet、細粒度與遷移測試組），只在驗證資料上調整，並回報多個隨機種子的平均 $\pm$ 標準差。
- 基準準確率很脆弱：在偏移資料上測試、小心捷徑與對抗輸入、在保留資料上擬合溫度，並檢查 Grad-CAM 熱圖，特別是在出錯的樣本上。

練習

1. **感受野與參數量**。證明三層疊加的 3×3 卷積（步幅 1）感受野為 7×7。在輸入與輸出都是 C 通道、無偏差項的情況下，與單一 7×7 卷積比較參數量。

提示

每層 3×3 讓感受野增加 2 個像素：$3\to5\to7$。參數量：$3\cdot9C^2=27C^2$ 對 $49C^2$，少了約 45%，還多了兩個非線性。

2. **瓶頸區塊的算術**。一個 ResNet 瓶頸區塊輸入有 $4d$ 個通道，降到 d。以 d 表示 1×1–3×3–1×1 分支的權重數（無偏差），並與寬度 $4d$、兩層 3×3 的基本區塊比較。比值是多少？

   提示

   瓶頸：$4d\cdot d+9d^2+d\cdot4d=17d^2$。寬度 $4d$ 的基本區塊：$2\cdot9\cdot16d^2=288d^2$。比值為 $17/288\approx0.059$，權重少了約 17 倍。

3. **標籤平滑的最佳解**。對一個有 K 個類別、平滑強度 ε 的樣本，假設所有錯誤類別的 logit 相等。證明平滑目標下的交叉熵在「正確類別與每個錯誤類別的 logit 差」為 $\log\left((1-\varepsilon+\varepsilon/K)\,/\,(\varepsilon/K)\right)$ 時最小。代入 $K=1000$、$\varepsilon=0.1$ 計算。

   提示

   交叉熵對 $\mathbf{p}$ 的最小值發生在 $\mathbf{p}=\mathbf{q}$。所以 $p_y=1-\varepsilon+\varepsilon/K$，其他每個 $p_k=\varepsilon/K$，logit 差就是兩者比值的對數：$\log(0.9001/0.0001)\approx9.1$。沒有平滑時，最佳間距是無限大。

4. **ViT 的 token 數**。一個使用 16×16 小塊的 ViT 收到 224×224 的影像。有多少 token 進入編碼器？若以 384×384 微調，token 數是多少？注意力矩陣大小成長幾倍？位置嵌入又該怎麼處理？

   提示

   $(224/16)^2=196$ 個小塊加一個類別 token：197。在 384 時：$24^2+1=577$。注意力矩陣成長 $(577/197)^2\approx8.6$ 倍。學到的位置嵌入是 14×14 的網格，要以二維內插放大到 24×24；ViT 論文在較高解析度微調時正是這麼做的。

5. **溫度不改變準確率**。證明對任何 $T>0$，$\arg\max_k\operatorname{softmax}(\mathbf{z}/T)_k=\arg\max_k z_k$。接著用 `scripts/figures/dl_02_image_classification.py`，改在測試集上而不是校準集上擬合溫度，比較回報的 ECE。為什麼那個數字過於樂觀？

   提示

   除以正的常數與指數函數都是嚴格遞增的，而 softmax 的分母是共用的。在測試集上擬合 T 等於用測試標籤調整參數，因此回報的 ECE 不再是對新資料表現的誠實估計。

6. **製造一條捷徑**。修改數字實驗，讓訓練集中每個「3」的左上角都有一個亮點，但測試集沒有。訓練 TinyResNet，回報測試集上 3 的準確率，並看幾個訓練用 3 的 Grad-CAM 熱圖。你預期會看到什麼？如果你事先不知道有這條捷徑，要怎麼發現這個問題？

   提示

   網路光看角落的亮點就能分類訓練集中的 3，所以測試集上 3 的準確率會下降、訓練準確率卻仍完美，而 Grad-CAM 往往會亮在角落。不知道捷徑時，可以找熱區落在背景上的熱圖、在獨立收集的資料上評估，並做反事實測試（抹除或移動可疑區域，看預測是否改變）。

7. **巨觀與微觀 F1**。一個三類別模型在 100 張影像上測試。列為真實類別、行為預測：類別 A（90 張）→ 88 A、1 B、1 C；類別 B（5 張）→ 3 A、2 B、0 C；類別 C（5 張）→ 4 A、0 B、1 C。計算逐類別的精確率、召回率與 F_1，再算巨觀 F_1 與微觀 F_1。如果報告的重點是能否偵測稀有的 B 與 C，你會放哪一個數字？為什麼？

提示

各行總和為 95、3、2。精確率：88/95 = 0.926、2/3 = 0.667、1/2 = 0.500。召回率：88/90 = 0.978、2/5 = 0.400、1/5 = 0.200。用 $F_1 = 2\text{TP}/(2\text{TP} + \text{FP} + \text{FN})$：176/185 = 0.951、4/8 = 0.500、2/7 = 0.286。巨觀 $F_1 = (0.951 + 0.500 + 0.286)/3 = 0.579$。微觀 F_1 等於準確率 91/100 = 0.910。微觀被類別 A 主導；巨觀則揭露 B 與 C 大多被漏掉，所以對稀有類別而言，巨觀（加上逐類別召回率）才是誠實的數字。可以用 `precision_recall_fscore_support(..., average="macro")` 驗算。

參考文獻

1. Z. Li, F. Liu, W. Yang, S. Peng, and J. Zhou, "A Survey of Convolutional Neural Networks: Analysis, Applications, and Prospects," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 12, pp. 6999–7019, 2022. doi:10.1109/TNNLS.2021.3084827
2. O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, et al., "ImageNet Large Scale Visual Recognition Challenge," *International Journal of Computer Vision*, vol. 115, pp. 211–252, 2015. doi:10.1007/s11263-015-0816-y
3. Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. doi:10.1109/5.726791
4. A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," technical report, University of Toronto, 2009. dataset page
5. T. Ridnik, E. Ben-Baruch, A. Noy, and L. Zelnik-Manor, "ImageNet-21K Pretraining for the Masses," *NeurIPS*, 2021. arXiv
6. G. Van Horn, O. Mac Aodha, Y. Song, Y. Cui, C. Sun, A. Shepard, et al., "The iNaturalist Species Classification and Detection Dataset," *CVPR*, 2018. arXiv
7. B. Recht, R. Roelofs, L. Schmidt, and V. Shankar, "Do ImageNet Classifiers Generalize to ImageNet?," *ICML*, PMLR vol. 97, 2019. paper
8. L. Beyer, O. J. Hénaff, A. Kolesnikov, X. Zhai, and A. van den Oord, "Are We Done with ImageNet?," arXiv:2006.07159, 2020. arXiv
9. C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks," *NeurIPS*, 2021. arXiv
10. A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *Advances in Neural Information Processing Systems 25 (NIPS)*, 2012. paper
11. K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," *ICLR*, 2015. arXiv
12. C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going Deeper with Convolutions," *CVPR*, 2015. doi:10.1109/CVPR.2015.7298594
13. C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception Architecture for Computer Vision," *CVPR*, 2016. doi:10.1109/CVPR.2016.308

14. K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *CVPR*, 2016. doi:10.1109/CVPR.2016.90
15. K. He, X. Zhang, S. Ren, and J. Sun, "Identity Mappings in Deep Residual Networks," *ECCV*, 2016. arXiv
16. G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely Connected Convolutional Networks," *CVPR*, 2017. arXiv
17. A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv:1704.04861, 2017. arXiv
18. M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," *ICML*, 2019. arXiv
19. I. Radosavovic, R. P. Kosaraju, R. Girshick, K. He, and P. Dollár, "Designing Network Design Spaces," *CVPR*, 2020. arXiv
20. Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," *CVPR*, 2022. arXiv
21. J. Hu, L. Shen, S. Albanie, G. Sun, and E. Wu, "Squeeze-and-Excitation Networks," arXiv:1709.01507, 2017 (journal version in *IEEE TPAMI*). arXiv
22. A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," *ICLR*, 2021. arXiv
23. H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training Data-Efficient Image Transformers & Distillation through Attention," *ICML*, PMLR vol. 139, 2021. paper
24. Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows," *ICCV*, 2021. doi:10.1109/ICCV48922.2021.00986
25. S. Khan, M. Naseer, M. Hayat, S. W. Zamir, F. S. Khan, and M. Shah, "Transformers in Vision: A Survey," *ACM Computing Surveys*, 2022. arXiv
26. H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond Empirical Risk Minimization," *ICLR*, 2018. arXiv
27. S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, "CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features," *ICCV*, 2019. arXiv
28. E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, "RandAugment: Practical Automated Data Augmentation with a Reduced Search Space," *CVPR Workshops*, 2020. doi:10.1109/CVPRW50498.2020.00359
29. I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," *ICLR*, 2019. arXiv
30. I. Loshchilov and F. Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts," *ICLR*, 2017. arXiv
31. G. Huang, Y. Sun, Z. Liu, D. Sedra, and K. Q. Weinberger, "Deep Networks with Stochastic Depth," *ECCV*, 2016. doi:10.1007/978-3-319-46493-0_39
32. R. Wightman, H. Touvron, and H. Jégou, "ResNet Strikes Back: An Improved Training Procedure in timm," arXiv:2110.00476, 2021. arXiv
33. S. Kornblith, J. Shlens, and Q. V. Le, "Do Better ImageNet Models Transfer Better?," *CVPR*, 2019. arXiv
34. T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A Simple Framework for Contrastive Learn-

ing of Visual Representations," *ICML*, 2020. arXiv
35. K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum Contrast for Unsupervised Visual Representation Learning," *CVPR*, 2020. arXiv
36. M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, "Emerging Properties in Self-Supervised Vision Transformers," *ICCV*, 2021. doi:10.1109/ICCV48922.2021.00951
37. K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked Autoencoders Are Scalable Vision Learners," *CVPR*, 2022. doi:10.1109/CVPR52688.2022.01553
38. M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, et al., "DINOv2: Learning Robust Visual Features without Supervision," *Transactions on Machine Learning Research*, 2024. arXiv
39. O. Siméoni, H. V. Vo, M. Seitzer, F. Baldassarre, M. Oquab, C. Jose, et al., "DINOv3," arXiv:2508.10104, 2025. arXiv
40. A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, et al., "Learning Transferable Visual Models From Natural Language Supervision," *ICML*, PMLR vol. 139, 2021. paper
41. J. Gui, T. Chen, J. Zhang, Q. Cao, Z. Sun, H. Luo, and D. Tao, "A Survey on Self-Supervised Learning: Algorithms, Applications, and Future Trends," *IEEE Transactions on Pattern Analysis and Machine Intelligence* (arXiv:2301.05712, 2023). arXiv
42. D. Hendrycks and T. Dietterich, "Benchmarking Neural Network Robustness to Common Corruptions and Perturbations," *ICLR*, 2019. arXiv
43. D. Hendrycks, S. Basart, N. Mu, S. Kadavath, F. Wang, E. Dorundo, et al., "The Many Faces of Robustness: A Critical Analysis of Out-of-Distribution Generalization," *ICCV*, 2021. arXiv
44. D. Hendrycks, K. Zhao, S. Basart, J. Steinhardt, and D. Song, "Natural Adversarial Examples," *CVPR*, 2021. arXiv
45. I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and Harnessing Adversarial Examples," in *Proc. International Conference on Learning Representations* (*ICLR*), 2015. arXiv
46. R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, "Shortcut Learning in Deep Neural Networks," *Nature Machine Intelligence*, 2020. arXiv
47. C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," *ICML*, 2017. arXiv
48. B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, and A. Torralba, "Learning Deep Features for Discriminative Localization," *CVPR*, 2016. doi:10.1109/CVPR.2016.319
49. R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization," *ICCV*, 2017. doi:10.1109/ICCV.2017.74
50. B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," *CVPR*, 2018. doi:10.1109/CVPR.2018.00286
51. G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," arXiv:1503.02531, 2015. arXiv
52. X. Zhai, A. Kolesnikov, N. Houlsby, and L. Beyer, "Scaling Vision Transformers," *CVPR*, 2022. arXiv
53. M. Cherti, R. Beaumont, R. Wightman, M. Wortsman, G. Ilharco, C. Gordon, et al., "Reproducible Scaling Laws for Contrastive Language-Image Learning," *CVPR*, 2023. arXiv

54. T. Fawcett, “An Introduction to ROC Analysis,” *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006. doi:10.1016/j.patrec.2005.10.010
55. J. Davis and M. Goadrich, “The Relationship Between Precision-Recall and ROC Curves,” *Proc. 23rd International Conference on Machine Learning* (*ICML*), pp. 233–240, 2006. doi:10.1145/1143844.1143874
56. T. Saito and M. Rehmsmeier, “The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets,” *PLOS ONE*, vol. 10, no. 3, e0118432, 2015. doi:10.1371/journal.pone.0118432
57. D. Chicco and G. Jurman, “The Advantages of the Matthews Correlation Coefficient (MCC) over F1 Score and Accuracy in Binary Classification Evaluation,” *BMC Genomics*, vol. 21, article 6, 2020. doi:10.1186/s12864-019-6413-7
58. M. Sokolova and G. Lapalme, “A Systematic Analysis of Performance Measures for Classification Tasks,” *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009. doi:10.1016/j.ipm.2009.03.002
59. J. Cohen, “A Coefficient of Agreement for Nominal Scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960. doi:10.1177/001316446002000104
60. M.-L. Zhang and Z.-H. Zhou, “A Review on Multi-Label Learning Algorithms,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 26, no. 8, pp. 1819–1837, 2014. doi:10.1109/TKDE.2013.39
61. M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The Pascal Visual Object Classes (VOC) Challenge,” *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303–338, 2010. doi:10.1007/s11263-009-0275-4
62. F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. paper
63. H. Wang, S. Ge, E. P. Xing, and Z. C. Lipton, “Learning Robust Global Representations by Penalizing Local Predictive Power,” *NeurIPS*, 2019 (introduces ImageNet-Sketch). arXiv
64. A. Barbu, D. Mayo, J. Alverio, W. Luo, C. Wang, D. Gutfreund, J. Tenenbaum, and B. Katz, “ObjectNet: A Large-Scale Bias-Controlled Dataset for Pushing the Limits of Object Recognition Models,” *NeurIPS*, 2019. dataset page
65. C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie, “The Caltech-UCSD Birds-200-2011 Dataset,” Technical Report CNS-TR-2011-001, California Institute of Technology, 2011. dataset page
66. J. Krause, M. Stark, J. Deng, and L. Fei-Fei, “3D Object Representations for Fine-Grained Categorization,” *IEEE International Conference on Computer Vision Workshops* (*ICCVW*), pp. 554–561, 2013 (introduces Stanford Cars). doi:10.1109/ICCVW.2013.77
67. S. Maji, E. Rahtu, J. Kannala, M. Blaschko, and A. Vedaldi, “Fine-Grained Visual Classification of Aircraft,” arXiv:1306.5151, 2013. arXiv
68. O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar, “Cats and Dogs,” *CVPR*, 2012 (introduces Oxford-IIIT Pets). dataset page
69. M.-E. Nilsback and A. Zisserman, “Automated Flower Classification over a Large Number of Classes,” *Indian Conference on Computer Vision, Graphics and Image Processing* (*ICVGIP*), 2008 (introduces Oxford Flowers-102). dataset page
70. L. Bossard, M. Guillaumin, and L. Van Gool, “Food-101 – Mining Discriminative Components

with Random Forests," *ECCV*, 2014. dataset page
71. L. Fei-Fei, R. Fergus, and P. Perona, "One-Shot Learning of Object Categories," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2006 (the reference given for Caltech-101). dataset page
72. X. Zhai, J. Puigcerver, A. Kolesnikov, et al., "A Large-scale Study of Representation Learning with the Visual Task Adaptation Benchmark," arXiv:1910.04867, 2019. arXiv
73. Z. Liu, Z. Miao, X. Zhan, J. Wang, B. Gong, and S. X. Yu, "Large-Scale Long-Tailed Recognition in an Open World," *CVPR*, 2019 (introduces ImageNet-LT). arXiv

DLCV 03・物件偵測：從 R-CNN 到 DETR

深度網路如何找出並標記影像中的每一個物件：邊界框迴歸、IoU、錨框、NMS 與特徵金字塔；二階段、一階段、無錨框與 Transformer 偵測器；開放詞彙模型；以及偵測該如何評估。

網站版本: https://blog.shyandram.dev/zh-tw/learn/deep-learning/dl-03-object-detection/

你將學到

- 偵測器輸出什麼（邊界框、類別、信心分數），以及偵測和分類、語意分割、實例分割的差別。
- 所有現代偵測器共用的基本元件，包含數學與程式碼：邊界框參數化與迴歸目標、IoU 及 GIoU/DIoU/CIoU 等變體、錨框與標籤指派、非極大值抑制、特徵金字塔，以及焦點損失。
- 四大家族如何運作、又為何被發明：二階段（R-CNN 到 Cascade R-CNN）、一階段（YOLO、SSD、RetinaNet）、無錨框（CornerNet、CenterNet、FCOS），以及用 Transformer 做集合預測（DETR 到 RT-DETR）。
- 開放詞彙與定位（grounding）偵測器（OWL-ViT、Grounding DINO、YOLO-World）以及可提示的基礎模型，如何改變「類別清單」的意義。
- 偵測如何評估（精確率–召回率、AP、VOC 與 COCO 協定、AP50/AP75/APs/m/l）、哪些資料集重要，以及部署時速度受什麼限制。

先看全貌

分類器只回答一個關於整張影像的問題：**這是什麼？**偵測器要回答更難的問題：**什麼東西在哪裡？**它必須找出每一個感興趣的物件，在每個物件外面畫一個緊貼的矩形（**邊界框**，bounding box），寫上名稱，並說明自己有多確定。物件的數量事先不知道：可能是零個、一個，也可能是三百個。

> **用最簡單的話說**
>
> 想像一頁「威利在哪裡？」。**分類器**只會告訴你：「這一頁裡有威利。」**偵測器**則會在威利身上貼一張貼紙，再在每一隻狗、每一顆氣球上各貼一張，並在每張貼紙上寫「這是什麼、有多確定」（「狗，蠻確定的」）。難的地方跟你自己找的時候一樣：遠處的小東西、躲在別的東西後面的東西，還有不要在同一隻狗身上貼兩張貼紙。

偵測是多數「真實世界」視覺系統的入口。自駕車偵測行人與車輛；手機在對焦之前先偵測人臉；工廠偵測瑕疵；而追蹤器（見文末連結的追蹤章節）大多是在**把偵測結果隨時間串起來**。這個領域也產生了許多如今遍布深度學習的想法：多任務損失函數、特徵金字塔、焦點損失，以及以二元匹配做集合預測。

本篇假設你已經熟悉 CNN 與影像分類（DLCV 02），以及IP 12的傳統特徵概念。有兩篇綜述可以框住整個故事：Zou 等人的《Object Detection in 20 Years》[1]，以及 Liu 等人發表於 IJCV 的深度通用物件偵測綜述 [2]。我們會在「現代觀點」回到這兩篇。

偵測問題的定義

白話地說：輸入是一張影像，輸出是一小串（邊界框、類別、分數）三元組。訓練資料則是人工畫好、標上類別名稱的邊界框。

精確地說，給定影像 $I \in \mathbb{R}^{H\times W\times 3}$，偵測器輸出一個集合

$$\hat{Y} = \{(\hat{b}_i, \hat{c}_i, \hat{s}_i)\}_{i=1}^{N},$$

其中 $\hat{b}_i \in \mathbb{R}^4$ 是一個邊界框，$\hat{c}_i \in \{1,\dots,K\}$ 是 K 個類別之一，$\hat{s}_i \in [0,1]$ 是信心分數，而 N 隨影像不同而改變。真值（ground truth）是 M 個標註物件構成的集合 $Y = \{(b_j, c_j)\}_{j=1}^{M}$。常見的邊界框格式有兩種：角點 (x_1, y_1, x_2, y_2) 與中心–尺寸 (c_x, c_y, w, h)。邊界框是**與座標軸對齊**的；旋轉框是另一種變體，用於空拍影像與文字偵測。

有三個性質讓偵測比分類困難：

1. **輸出大小不固定**。網路必須決定要輸出**幾個**東西。本篇的每一種架構，本質上都是對這個問題的不同回答。
2. **同時定位與辨識**。模型對物件外觀的變化要具有不變性，對物件的**位置**卻要很敏感；這股張力形塑了每一種偵測頭（head）的設計。
3. **極度不平衡**。一張典型影像只有少數幾個物件，卻有上萬個候選位置與尺寸，幾乎全是背景。

偵測與相鄰任務的比較

圖 1 在同一個合成場景上比較四種經典辨識任務。**分類**每張影像給一個標籤。**偵測**每個物件給一個邊界框與一個標籤。**語意分割**為每個像素標上類別，因此兩顆相碰的球會融成一團。**實例分割**為每個物件給出各自的像素遮罩，也就是「偵測＋每個框一張遮罩」；稍後會看到，Mask R-CNN [16] 是取得它的標準方法。

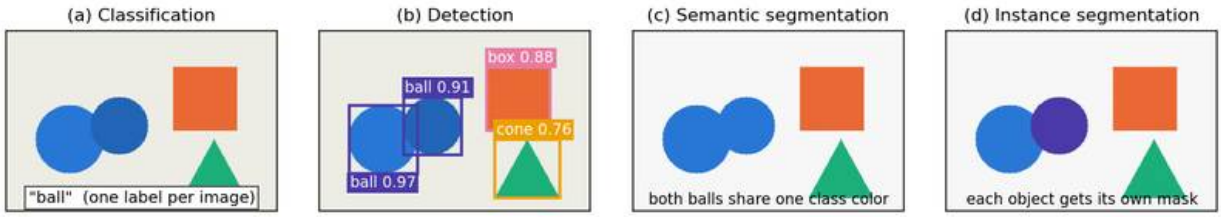


圖 *1* — 同一個場景在四種任務下的樣子：(*a*) 分類、(*b*) 偵測、(*c*) 語意分割、(*d*) 實例分割。

傳統偵測：滑動視窗與手工特徵

白話地說：在深度學習之前，偵測就是「把分類放進迴圈」：在每個位置、每個尺寸切出一個視窗，問分類器裡面有沒有物件，留下回答「有」的視窗。

滑動視窗。在縮放比例為 λ 的影像金字塔上，以步幅 s 滑動固定大小 $w \times h$ 的視窗，視窗數量大約是

$$N_{\text{win}} \approx \sum_{k \ge 0} \frac{(H\lambda^{-k} - h)(W\lambda^{-k} - w)}{s^2},$$

其中 k 是金字塔的層索引。對一張 640×480 的影像、64×128 的視窗、$s = 8$、$\lambda = 1.2$，單一長寬比就有約 10^4 個視窗，每一個都得分類。這就是IP 13的樣板比對（template matching）放大版，也有同樣的弱點：只有一種樣板形狀、要搜尋很多尺度，而且大量運算浪費在背景上。

Viola–Jones（2001）[5] 以三個技巧讓即時人臉偵測成為可能。**Haar-like 特徵**（相鄰矩形區域總和之差）可以用**積分影像**（integral image）$S(x,y) = \sum_{x' \le x, y' \le y} I(x',y')$ 在常數時間內算出，任何矩形總和只需四次查表。**AdaBoost** 從龐大的特徵池中挑出一小組有資訊量的特徵。**注意力串接**（attentional cascade）由一連串越來越複雜的分類器組成，只需少數幾次特徵運算就能拒絕大多數背景視窗。這是對抗不平衡問題的第一個大想法：只在可能有用的地方花運算。

HOG ＋線性 SVM（2005）[6] 用**方向梯度直方圖**（histograms of oriented gradients）取代原始灰階值，它是IP 12中梯度方向描述子的近親。偵測視窗被切成小單元（cell），每個單元以梯度大小加權，統計梯度方向的直方圖；相鄰單元組成彼此重疊的區塊（block）並做對比正規化；串接後的向量由線性 SVM 評分。HOG 對小幅平移與光照變化很穩健，讓行人偵測變得實用。

可變形部件模型（deformable part models, DPM）[7] 加入了結構。一個物件由一個粗略的**根**（root）HOG 濾波器，加上數個解析度較高的**部件**（part）濾波器組成；部件可以相對於根移動，但要付出二次的形變代價：

$$\text{score}(p_0) = F_0 \cdot \phi(p_0) + \sum_{i=1}^{n} \max_{p_i} \left[F_i \cdot \phi(p_i) - d_i \cdot \psi(p_i - p_0) \right] + \beta,$$

其中 p_0 是根的位置，p_i 是第 i 個部件的位置，F_i 是學到的濾波器，$\phi(\cdot)$ 是某位置的 HOG 特徵，ψ 是位移特徵 (dx, dy, dx^2, dy^2)，d_i 是學到的形變權重，β 是偏差。部件位置是潛在變數，以**潛在 SVM**（latent SVM）訓練。DPM 及其變體一直稱霸 PASCAL VOC 基準 [3]，直到 2012 年。

這個時代為何結束。傳統偵測器**慢**（要在位置、尺度、長寬比上窮舉掃描，每個視窗都經過手工設計的流程評分），而且**脆弱**（只有一種手工特徵、少數幾個固定樣板，換到新類別、新視角或雜亂場景就表現不佳），也無法在類別之間共享特徵。深度偵測器保留了有用的想法，例如在特徵金字塔中掃描、串接、部件與困難負樣本挖掘（hard-negative mining），並把手工特徵換成學到的特徵。

核心基本元件

接下來的每一種偵測器都由同一組零件組成。我們仔細定義每一個，並附上可執行的程式碼。

邊界框參數化與迴歸目標

白話地說：網路很不擅長直接輸出絕對像素座標，卻很擅長輸出小幅修正。因此我們從一個**參考框**（候選區域、錨框或一個點）出發，預測該怎麼微調它。

給定中心–尺寸形式的參考框 $P = (P_x, P_y, P_w, P_h)$ 與其匹配的真值框 $G = (G_x, G_y, G_w, G_h)$，R-CNN [8] 提出的標準目標為

$$t_x = \frac{G_x - P_x}{P_w}, \qquad t_y = \frac{G_y - P_y}{P_h}, \qquad t_w = \log \frac{G_w}{P_w}, \qquad t_h = \log \frac{G_h}{P_h}.$$

把位移除以參考框尺寸，讓目標具有**尺度不變性**（10 個像素的誤差，對 20 像素的框比對 400 像素的框嚴重得多）。寬高取**對數**，讓縮放變得對稱（放大兩倍與縮小一半分別是 $+\log 2$ 與 $-\log 2$），並保證解碼後的尺寸為正：$\hat{G}_w = P_w \exp(\hat{t}_w)$。實務上，目標常再除以固定的標準差，例如 $(0.1, 0.1, 0.2, 0.2)$，讓四個量的大小相近。

```
import numpy as np

def to_cxcywh(b):
    return np.stack([(b[:, 0] + b[:, 2]) / 2, (b[:, 1] + b[:, 3]) / 2,
                     b[:, 2] - b[:, 0], b[:, 3] - b[:, 1]], axis=1)

def to_xyxy(c):
    return np.stack([c[:, 0] - c[:, 2] / 2, c[:, 1] - c[:, 3] / 2,
                     c[:, 0] + c[:, 2] / 2, c[:, 1] + c[:, 3] / 2], axis=1)

def encode(anchors, gt, std=(0.1, 0.1, 0.2, 0.2)):
    """Regression targets (tx, ty, tw, th) of gt relative to anchors (both xyxy)."""
    P, G = to_cxcywh(anchors), to_cxcywh(gt)
    t = np.stack([(G[:, 0] - P[:, 0]) / P[:, 2],
                  (G[:, 1] - P[:, 1]) / P[:, 3],
                  np.log(G[:, 2] / P[:, 2]),
                  np.log(G[:, 3] / P[:, 3])], axis=1)
    return t / np.array(std)          # normalize so all four have similar scale

def decode(anchors, t, std=(0.1, 0.1, 0.2, 0.2)):
    P = to_cxcywh(anchors)
    t = t * np.array(std)
    c = np.stack([P[:, 0] + t[:, 0] * P[:, 2],
                  P[:, 1] + t[:, 1] * P[:, 3],
                  P[:, 2] * np.exp(t[:, 2]),
                  P[:, 3] * np.exp(t[:, 3])], axis=1)
    return to_xyxy(c)

anchors = np.array([[0, 0, 32, 32], [100, 100, 228, 164]], float)
gt      = np.array([[4, 2, 40, 30], [90, 110, 250, 170]], float)
t = encode(anchors, gt)
print(np.round(t, 3))
print(np.allclose(decode(anchors, t), gt))   # round trip
```

```
[[ 1.875  0.     0.589 -0.668]
 [ 0.469  1.25   1.116 -0.323]]
True
```

這些目標通常以 Fast R-CNN [9] 的 **smooth L1**（Huber）損失函數訓練：

$$\text{smooth}_{L_1}(x) = \begin{cases} 0.5\,x^2 & \text{if } |x| < 1, \\ |x| - 0.5 & \text{otherwise,} \end{cases}$$

逐一套用在每個座標差 $x = \hat{t} - t$ 上。誤差小時它像 L2（梯度平滑），誤差大時像 L1（在訓練初期對離群值穩健）。它的弱點是把四個座標當成彼此獨立的數字，而評估指標 IoU 卻把邊界框視為一個整體。這個落差催生了以 IoU 為基礎的損失函數。

IoU 及其變體

白話地說：IoU 問的是，在兩個框各自覆蓋的總面積中，有多少比例是兩者都覆蓋到的？完全吻合時為 1，兩框不相碰時為 0。

對兩個框 A 與 B，

$$\mathrm{IoU}(A,B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|},$$

其中 $|\cdot|$ 代表面積。IoU（交並比，intersection over union）具有尺度不變性，是評估的標尺（若一個偵測結果與某個尚未被匹配的真值框的 IoU 超過門檻，就算正確），也用於標籤指派與 NMS。圖 2 顯示這些門檻有多嚴格：把正方形沿兩軸各平移邊長的約 18%，IoU 就掉到 0.5。

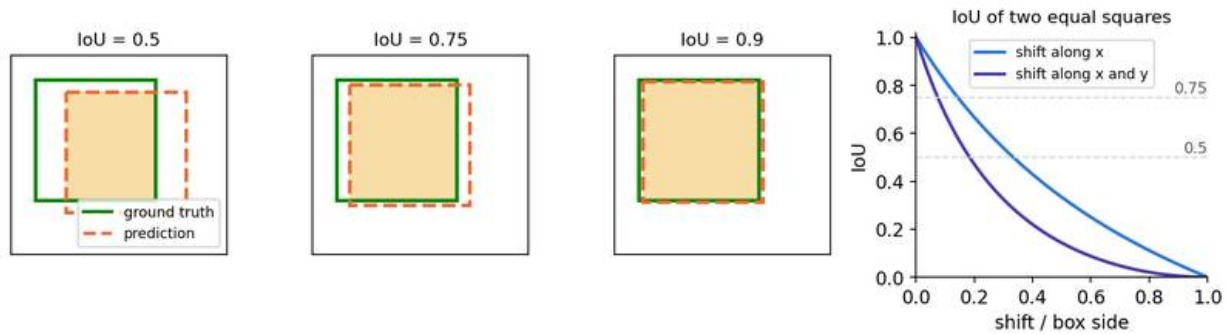


圖 2 — *IoU* 門檻長什麼樣子。左：*IoU* 為 *0.5*、*0.75*、*0.9* 的預測框與真值框。右：兩個相同正方形之一沿 x 軸、或沿 x 與 y 軸平移時的 *IoU*。

拿 $1 - \mathrm{IoU}$ 當損失函數有個缺陷：兩框不重疊時，不論相距多遠 IoU 都是 0，梯度也就是 0。常用的修正有三種：

- **GIoU** [11] 加入一個以同時包住兩框的最小框 C 為基礎的懲罰項：$\mathrm{GIoU} = \mathrm{IoU} - \frac{|C \setminus (A \cup B)|}{|C|}$，值域為 $(-1, 1]$，兩個不重疊的框離得越遠，它就繼續下降。
- **DIoU** [12] 直接懲罰中心點距離：$\mathrm{DIoU} = \mathrm{IoU} - \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2}$，其中 ρ 是兩框中心 $\mathbf{b}$ 與 $\mathbf{b}^{gt}$ 的歐氏距離，c 是 C 的對角線長度。它比 GIoU 收斂得更快，因為即使一個框包住另一個框（此時 GIoU 的懲罰項消失），它仍會把中心點拉近。
- **CIoU** [12] 再加入長寬比項：$\mathrm{CIoU} = \mathrm{DIoU} - \alpha v$，其中 $v = \frac{4}{\pi^2}\left(\arctan\frac{w^{gt}}{h^{gt}} - \arctan\frac{w}{h}\right)^2$ 衡量長寬比的差異，$\alpha = \frac{v}{(1-\mathrm{IoU})+v}$ 是權衡權重。

對應的損失函數分別是 $1 - \mathrm{GIoU}$、$1 - \mathrm{DIoU}$ 與 $1 - \mathrm{CIoU}$。

```
import numpy as np

def iou_family(a, b, eps=1e-9):
    """a, b: (N, 4) boxes as (x1, y1, x2, y2). Returns IoU, GIoU, DIoU, CIoU."""
    ix1, iy1 = np.maximum(a[:, 0], b[:, 0]), np.maximum(a[:, 1], b[:, 1])
    ix2, iy2 = np.minimum(a[:, 2], b[:, 2]), np.minimum(a[:, 3], b[:, 3])
    inter = np.clip(ix2 - ix1, 0, None) * np.clip(iy2 - iy1, 0, None)
    wa, ha = a[:, 2] - a[:, 0], a[:, 3] - a[:, 1]
    wb, hb = b[:, 2] - b[:, 0], b[:, 3] - b[:, 1]
    union = wa * ha + wb * hb - inter
    iou = inter / (union + eps)
    # smallest enclosing box C
    cx1, cy1 = np.minimum(a[:, 0], b[:, 0]), np.minimum(a[:, 1], b[:, 1])
    cx2, cy2 = np.maximum(a[:, 2], b[:, 2]), np.maximum(a[:, 3], b[:, 3])
    area_c = (cx2 - cx1) * (cy2 - cy1)
    giou = iou - (area_c - union) / (area_c + eps)
    # center distance over enclosing-box diagonal
    rho2 = ((a[:, 0] + a[:, 2]) - (b[:, 0] + b[:, 2])) ** 2 / 4 \
         + ((a[:, 1] + a[:, 3]) - (b[:, 1] + b[:, 3])) ** 2 / 4
    c2 = (cx2 - cx1) ** 2 + (cy2 - cy1) ** 2
    diou = iou - rho2 / (c2 + eps)
    v = 4 / np.pi**2 * (np.arctan(wb / hb) - np.arctan(wa / ha)) ** 2
    alpha = v / (1 - iou + v + eps)
    ciou = diou - alpha * v
    return iou, giou, diou, ciou

gt = np.array([[10, 10, 50, 50]] * 3, float)
pred = np.array([[20, 20, 60, 60],    # overlapping, shifted
                 [60, 10, 100, 50],   # disjoint, nearby
                 [200, 10, 240, 50]], float)  # disjoint, far away
for name, v in zip(["IoU", "GIoU", "DIoU", "CIoU"], iou_family(pred, gt)):
    print(f"{name:5s}", np.round(v, 3))
```

```
IoU   [0.391 0.    0.   ]
GIoU  [ 0.311 -0.111 -0.652]
DIoU  [ 0.351 -0.258 -0.662]
CIoU  [ 0.351 -0.258 -0.662]
```

兩個不重疊的預測框 IoU 都是 0，但 GIoU 與 DIoU 正確地指出比較遠的那個比較差。這裡 CIoU 等於 DIoU，因為所有框都是正方形，沒有長寬比差異。

錨框與標籤指派

白話地說：密集偵測器不自由搜尋，而是在特徵圖的每個格子放一組固定的參考框，也就是**錨框**（anchor）。每個錨框都在問：「這裡有沒有一個大致像我的物件？我該怎麼移動才能貼合它？」

在步幅為 s 的特徵圖上（每個格子涵蓋輸入影像的 $s\times s$ 像素），錨框的中心位於 $((i+\tfrac{1}{2})s, (j+\tfrac{1}{2})s)$，並有幾種尺度 σ 與長寬比 $r = h/w$，寬為 $w = \sigma/\sqrt{r}$、高為 $h = \sigma\sqrt{r}$，因此面積維持 σ^2。Faster R-CNN 在每個位置使用 3 種尺度 × 3 種長寬比 = 9 個錨框 [10]（圖 3 左）。

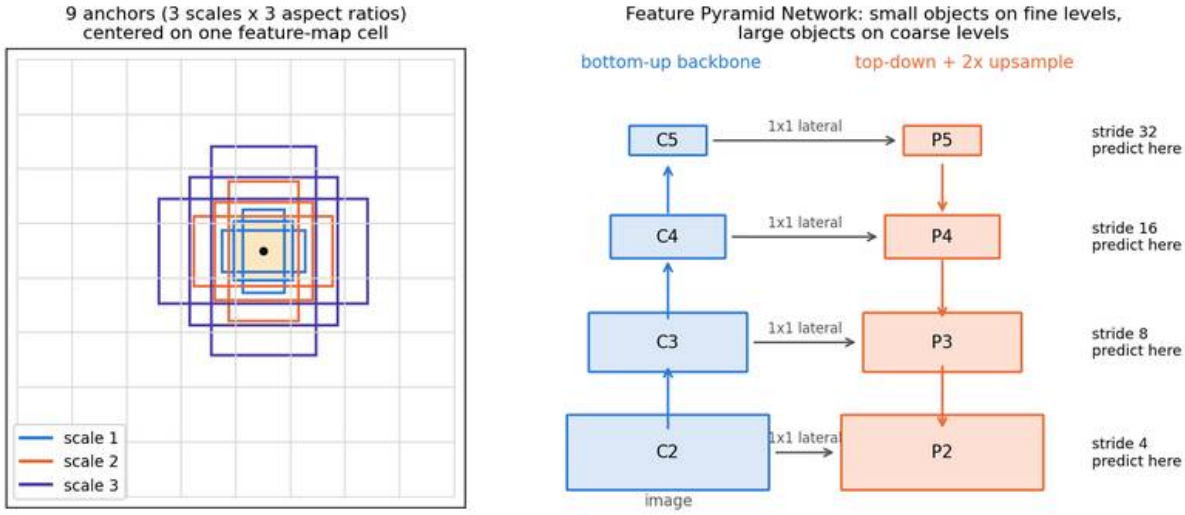


圖 3 — 左：特徵圖某一格上的九個錨框（3 種尺度 × 3 種長寬比）。右：特徵金字塔網路；每一層負責預測對應尺寸範圍的物件。

標籤指派（label assignment）決定哪個錨框要學習哪個物件。經典規則以 IoU 為準：在 Faster R-CNN 的區域提議網路中，若某錨框與某真值框的 IoU 至少 0.7（或它是該真值框最匹配的錨框），就是**正樣本**；若它與所有真值框的 IoU 都低於 0.3，就是**負樣本**；其餘忽略 [10]。RetinaNet 則使用 0.5 與 0.4 [15]。每個正樣本錨框以上述目標朝其匹配的框迴歸。後來發現，指派方式和架構一樣重要：無錨框偵測器（FCOS [29]）改用**位置**與物件尺寸來指派，而 DETR [30] 則以學習出的一對一匹配取代所有固定規則。

錨框也有實際成本：它帶來必須針對每個資料集調整的超參數（尺度、長寬比、門檻），也讓預測數量倍增。YOLO9000 [19] 用 $1 - \text{IoU}$ 當距離，對訓練框做 k-means 分群來決定錨框形狀。

非極大值抑制（NMS）與 Soft-NMS

白話地說：密集偵測器會在同一個物件上反應很多次（相鄰的錨框都看得到它）。**非極大值抑制**（non-maximum suppression, NMS）留下最有信心的框，刪掉與它重疊太多的其他框。

貪婪 NMS（逐類別進行）：依分數排序；反覆取出剩下分數最高的框 M，保留它，並移除所有 $\text{IoU}(M, b_i) > N_t$ 的框 b_i（N_t 通常為 0.5 到 0.7）。它的弱點在擁擠場景：兩個站得很近的真人可能重疊超過 N_t，分數較低的那個就被刪掉，召回率因而下降。

Soft-NMS [13] 把「刪除」改成「降低分數」。以高斯版本為例，

$$s_i \leftarrow s_i \exp\!\left(-\frac{\text{IoU}(M, b_i)^2}{\sigma}\right),$$

其中 s_i 是框 b_i 的分數，σ 控制重疊框被懲罰的速度。重疊很多時分數仍會大幅下降，但真正相鄰的物件會以較低的分數留下來，而不是直接消失。

```
import numpy as np
```

```
def iou_one_to_many(box, boxes):
    x1 = np.maximum(box[0], boxes[:, 0]); y1 = np.maximum(box[1], boxes[:, 1])
    x2 = np.minimum(box[2], boxes[:, 2]); y2 = np.minimum(box[3], boxes[:, 3])
    inter = np.clip(x2 - x1, 0, None) * np.clip(y2 - y1, 0, None)
    area = lambda b: (b[..., 2] - b[..., 0]) * (b[..., 3] - b[..., 1])
    return inter / (area(box) + area(boxes) - inter)

def nms(boxes, scores, thr=0.5):
    order = np.argsort(-scores)
    keep = []
    while order.size:
        i = order[0]
        keep.append(i)
        ious = iou_one_to_many(boxes[i], boxes[order[1:]])
        order = order[1:][ious <= thr]          # drop everything that overlaps too much
    return np.array(keep)

def soft_nms(boxes, scores, sigma=0.5, score_thr=0.001):
    scores = scores.copy()
    idx = np.arange(len(scores))
    keep = []
    while idx.size:
        j = np.argmax(scores[idx]); i = idx[j]
        keep.append(i)
        idx = np.delete(idx, j)
        ious = iou_one_to_many(boxes[i], boxes[idx])
        scores[idx] *= np.exp(-ious**2 / sigma)  # decay instead of delete
        idx = idx[scores[idx] > score_thr]
    return np.array(keep), scores

# two people standing close together + duplicates
boxes = np.array([[10, 10, 60, 110], [14, 12, 62, 112], [8, 6, 58, 104],
                  [40, 15, 90, 115], [43, 18, 92, 118]], float)
scores = np.array([0.95, 0.90, 0.80, 0.85, 0.70])
print("greedy NMS keeps:", nms(boxes, scores, 0.5))
keep, s = soft_nms(boxes, scores)
print("Soft-NMS order:", keep, "rescored:", np.round(s[keep], 3))
```

```
greedy NMS keeps: [0 3]
Soft-NMS order: [0 3 2 4 1] rescored: [0.95  0.761 0.183 0.145 0.058]
```

貪婪 NMS 為每個人留下一個框。Soft-NMS 保留所有框，但把重複的框排在兩個真實偵測結果之後很遠的位置；之後的分數門檻或 AP 計算會把它們當成低信心的猜測。圖 4 在合成場景上呈現同樣的效果。

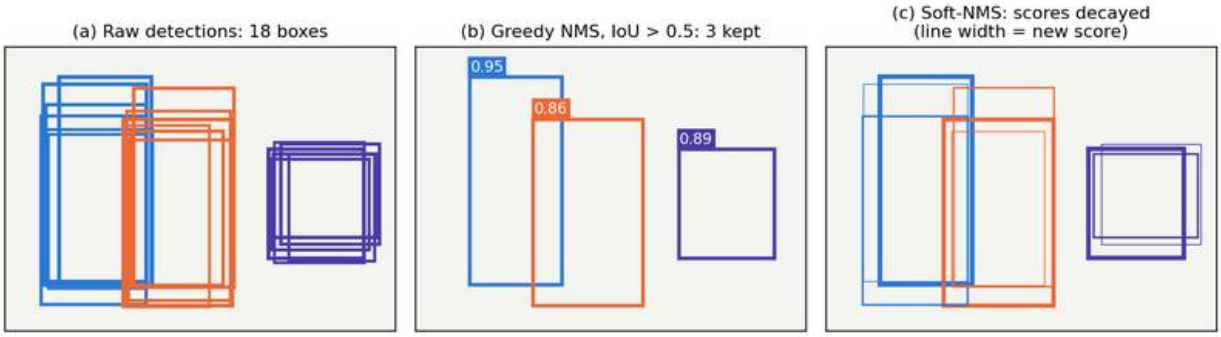


圖 *4* — 在合成偵測結果上做 *NMS*。(*a*) 三個物件周圍的 *18* 個原始框。(*b*) *IoU* 門檻 *0.5* 的貪婪 *NMS* 為每個物件留下一個框。(*c*) *Soft-NMS* 保留所有框，但降低重疊框的分數（線越細，分數越低）。

NMS 還有三個問題：它有一個要手動調整的門檻；它是循序、依資料而定的步驟，不容易在加速器上執行；而且它不屬於訓練的一部分，所以網路從來沒被教過要避免重複。去掉 NMS 是 DETR 與 YOLOv10 [24] 的主要動機之一。

特徵金字塔（FPN）

白話地說：小物件需要高解析度的特徵圖；大物件需要看過大量上下文的特徵。特徵金字塔讓每個尺度都同時擁有兩者。

CNN 骨幹網路本來就會產生步幅為 $4, 8, 16, 32$ 的金字塔 $\{C_2, C_3, C_4, C_5\}$，但淺層的語意較弱。**特徵金字塔網路**（Feature Pyramid Network, FPN）[14] 加上一條由上而下的路徑：從最粗的一層開始，上採樣 2 倍，再加上經 1×1 投影的骨幹橫向連接，

$$P_l = \mathrm{Conv}_{3\times3}\big(\mathrm{Conv}_{1\times1}(C_l) + \mathrm{Up}_{2\times}(P_{l+1})\big),$$

因此每個輸出層 P_l 的通道數相同，語意也都很強。在搭配 FPN 的二階段偵測器中，大小為 $w \times h$ 的感興趣區域從第 $l = \lfloor l_0 + \log_2(\sqrt{wh}/224) \rfloor$ 層擷取特徵（[14] 中 $l_0 = 4$），所以區域大一倍就往上移一層；密集偵測器也使用類似的「尺寸對應層級」規則。後來的頸部（neck）設計（額外的由下而上路徑、加權融合）都在精煉同一個想法；2017 年後幾乎每個偵測器都用上了它（圖 3 右）。

焦點損失與類別不平衡

白話地說：如果 99.9% 的候選都是背景，一般的損失函數會被數百萬個簡單的「這裡沒東西」樣本主導。焦點損失把模型已經答對的樣本「調小音量」。

令 $p \in [0,1]$ 為前景類別的預測機率，$y \in \{0,1\}$ 為標籤。若 $y = 1$，定義 $p_t = p$，否則 $p_t = 1 - p$。**焦點損失**（focal loss）[15] 為

$$\mathrm{FL}(p_t) = -\alpha_t (1 - p_t)^{\gamma} \log(p_t),$$

其中 $\gamma \geq 0$ 是聚焦參數，α_t 是類別平衡權重（前景為 α，背景為 $1 - \alpha$）。$\gamma = 0$ 時它就是加權交叉熵。以論文預設的 $\gamma = 2$、$\alpha = 0.25$，一個 $p_t = 0.99$ 的簡單負樣本相對於交叉熵被降權

$(0.01)^2 = 10^{-4}$ 倍，而困難樣本幾乎保留全部權重。損失會再除以正樣本錨框的數量。二階段偵測器用別的方法避開這個問題（提議階段先移除大部分背景，再以固定的前景：背景比例取樣），而 SSD [26] 使用**困難負樣本挖掘**（只保留損失最高的負樣本，比例約 3:1）。焦點損失第一次讓**密集**的一階段偵測器和二階段偵測器一樣準確。

二階段偵測器：R-CNN 家族

白話地說：先提出幾千個「可能有東西」的地方，再仔細看每一個，決定它**是什麼**以及邊緣**確切在哪裡**。

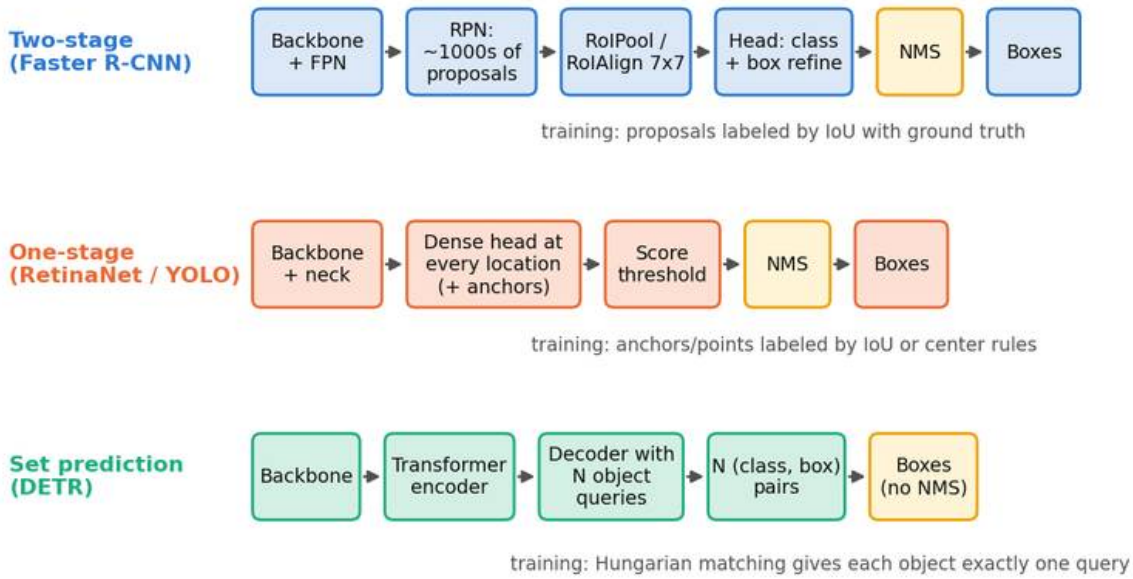


圖 5 — 三種偵測器流程。對二階段與一階段偵測器而言，*NMS*（黃色）是後處理步驟；*DETR* 式的集合預測透過學習一對一指派而把它移除。

R-CNN（2014） [8] 先用一個由下而上的分組演算法，為每張影像產生約 2,000 個與類別無關的候選區域（region proposal），把每個區域變形成固定大小，送進以分類預訓練的 CNN，以每類別一個的線性 SVM 為特徵評分，再用上述目標微調邊界框。它把 PASCAL VOC 2012 的 mean AP 相對於先前最佳結果提升了 30% 以上，並確立了「先在分類上預訓練，再在偵測上微調」這個至今仍是標準的做法。它非常慢，因為 CNN 要對每個候選區域各跑一次。

Fast R-CNN（2015） [9] 對整張影像只跑一次 CNN，再用 **RoI pooling** 從共享的特徵圖上切出每個候選區域：把區域投影到特徵圖上，分成固定的 $H \times W$ 網格（例如 7×7），每格做最大池化，所以不論形狀為何，每個候選區域都得到相同大小的特徵。整個網路以多任務損失訓練：

$$L = L_{\text{cls}}(p, u) + \lambda \, [u \geq 1] \, L_{\text{loc}}(t^u, v),$$

其中 p 是 $K+1$ 類的 softmax，u 是真實類別（背景為 0），$[u \geq 1]$ 只在前景候選區域為 1，t^u 是針對類別 u 預測的框偏移量，v 是目標偏移量，L_{loc} 是 smooth L1 損失。

Faster R-CNN（2015） [10] 讓候選區域本身也由學習產生。一個小型的**區域提議網路**（Region Proposal Network, RPN）在共享特徵圖上滑動，為每個位置的 9 個錨框預測「是否為物件」的分數與框偏移量。最好的候選區域（經 NMS 後）再送進 Fast R-CNN 的偵測頭。由於兩個階段共用骨幹，候選區域幾乎不花成本，整個偵測器也成為一個端到端訓練的網路。

Mask R-CNN（2017） [16] 加上一個小型全卷積分支，在每個 RoI 內為每個類別預測一張二值遮罩，因而得到實例分割。它的關鍵細節是 **RoIAlign**：RoI pooling 會把區域邊界和格子邊緣捨入到特徵網格上，使特徵最多偏移半個步幅。RoIAlign 保留實數座標，並以雙線性內插取樣每一格。這聽起來是小事，但像素級精確的遮罩需要它，高 IoU 下的框準確度也會提升。

Cascade R-CNN（2018） [17] 處理一個微妙的問題。以 IoU ≥ 0.5 為正樣本訓練的偵測頭，學到的是「在 0.5 夠好」的框，在 0.75 卻不夠好。單純提高門檻會讓正樣本太少而過擬合（overfitting）。Cascade R-CNN 串接數個以遞增 IoU 門檻（0.5、0.6、0.7）訓練的偵測頭，每一級都精煉前一級的框，所以每一級拿到的訓練分布品質都更高。它在嚴格的 COCO 指標上帶來大幅提升。

二階段偵測器準確、有彈性（RoI 偵測頭可以預測遮罩、關鍵點或屬性），至今仍是許多實例分割任務的預設選擇，但每個 RoI 一次的偵測頭與 NMS 都要花時間。

一階段偵測器：YOLO、SSD 與 RetinaNet

白話地說：跳過提議步驟。在一次前向傳遞中，對特徵圖的每個位置做一次密集預測，再用 NMS 清理。

YOLO v1（2016） [18] 把影像切成 $S \times S$ 網格（$S = 7$）。每格預測 $B = 2$ 個框（各帶一個信心值）以及一組 C 個類別機率，因此輸出是單一個 $S \times S \times (5B + C)$ 張量（PASCAL VOC 的 20 類對應 $7 \times 7 \times 30$）。物件中心所在的格子負責該物件。它能即時執行，而且因為看得到整張影像，背景誤判比 Fast R-CNN 少；但它定位較不精確，也難以處理小物件與成群的物件，因為每格只能預測固定數量的框與一個類別。

YOLO 家族之後快速演進。下表只列出我們能查證論文的版本；許多廣泛使用的版本是以軟體套件形式維護、沒有論文，本篇不予涵蓋。

版本	年份	主要想法（依論文）
YOLO [18]	2016	單次網格迴歸；即時
YOLO9000 / v2 [19]	2016（arXiv）	以 IoU 距離做 k-means 選出錨框；批次正規化；較高解析度訓練
YOLOv3 [20]	2018（技術報告）	更深的骨幹；在三個尺度預測；各類別獨立的 logistic 分數
YOLOv4 [21]	2020（arXiv）	系統性研究訓練技巧（bag of freebies）與低成本的架構附加元件（bag of specials）
YOLOv7 [22]	2023（CVPR）	可訓練的 bag-of-freebies：重參數化模組、改良的標籤指派、高效的層聚合
YOLOv9 [23]	2024（ECCV）	可程式化梯度資訊（PGI）讓資訊穿過深層而不流失；GELAN 骨幹
YOLOv10 [24]	2024（NeurIPS）	以一致的雙重指派（一對多與一對一）做免 NMS 訓練；以效率為導向的設計

版本	年份	主要想法（依論文）
YOLOv12 [25]	2025（arXiv）	以注意力為核心、同時維持即時速度的設計

可以看出一個反覆出現的主題：「密集網格＋ NMS」的核心想法不變，改變的是標籤指派、頸部、訓練配方，以及最近的注意力機制與免 NMS 偵測頭。

SSD（2016） [26] 在**多個**不同解析度的特徵圖上接上小型卷積預測器，每層都有自己一組多種長寬比的預設框（default box，即錨框），處理尺度的能力遠勝 YOLO v1。它以困難負樣本挖掘與大量資料擴增訓練。

RetinaNet（2017） [15] 結合 FPN 骨幹、密集錨框（每個位置 9 個），以及分類與框迴歸兩個小型子網路，並以焦點損失訓練。論文的分析顯示，一階段偵測器先前落後二階段偵測器的主因是類別不平衡，而不是架構。

無錨框偵測器：用點取代框

白話地說：錨框是對物件長相的猜測。無錨框偵測器拿掉它們，改從**點**預測邊界框：角點、中心點，或物件內部的每一個像素。

CornerNet（2018） [27] 把物件偵測成一對關鍵點：一張左上角熱圖、一張右下角熱圖，以及每個角點的**嵌入向量**（embedding）。同一物件的兩個角點被訓練成嵌入相近，因此以嵌入距離來配對。由於角點常落在物件外面，論文提出**角點池化**（corner pooling），沿著列與行收集角點的證據。

CenterNet，〈Objects as Points〉（2019） [28] 以中心點代表每個物件。網路輸出一張熱圖 $\hat{Y} \in [0,1]^{\frac{W}{R}\times\frac{H}{R}\times K}$（輸出步幅 R，每類一個通道），以及每個位置的尺寸 (w,h) 與次像素偏移。真值中心以高斯函數散佈在熱圖上，熱圖以逐像素的焦點損失訓練。推論時，峰值就是局部最大值，用 3×3 最大池化比較即可找到，因此**不需要以 IoU 為基礎的 NMS**。同樣的設計只要在中心點迴歸更多量，就能延伸到 3D 框與人體姿態。

FCOS（2019） [29] 把偵測當成分割來做。特徵圖上每個落在真值框內的位置都是正樣本，並迴歸到框四邊的距離：

$$l^* = x - x_1, \qquad t^* = y - y_1, \qquad r^* = x_2 - x, \qquad b^* = y_2 - y,$$

其中 (x,y) 是該位置的影像座標，(x_1,y_1,x_2,y_2) 是框。每一層 FPN 只處理最大迴歸距離落在自己範圍內的物件，這解決了框重疊時大部分的歧義。離物件中心遠的位置預測出的框品質差，因此 FCOS 加入**中心度**（centerness）分支：

$$\text{centerness} = \sqrt{\frac{\min(l^*,r^*)}{\max(l^*,r^*)}\cdot\frac{\min(t^*,b^*)}{\max(t^*,b^*)}},$$

它在中心為 1，往邊緣遞減；測試時乘上類別分數，讓偏離中心的框在 NMS 前排得較後面。FCOS 去掉了所有錨框超參數，表現追平甚至超越以錨框為基礎的 RetinaNet。

以 Transformer 做集合預測：DETR 及其後繼者

白話地說：與其預測上千個候選再刪掉重複的，不如直接預測一小**組**答案，並在訓練時讓每個真實物件剛好被一個預測認領。

DETR（2020） [30] 使用 CNN 骨幹、作用在特徵圖上的 Transformer 編碼器，以及一個 Transformer 解碼器；解碼器接收 N 個學習得到的**物件查詢**（object query，論文中為 100 個），平行輸出 N 組（類別、框）預測。沒有錨框、沒有候選區域，也沒有 NMS。由於 N 大於物件數，真值會用特殊的「無物件」類別 $\varnothing$ 補齊到 N 個。

關鍵在於以二**元匹配**（bipartite matching）訓練。令 $y = \{y_j\}$ 為補齊後的真值，$\hat{y} = \{\hat{y}_i\}_{i=1}^{N}$ 為預測。找出使總匹配成本最低的 N 元素排列 $\hat{\sigma}$：

$$\hat{\sigma} = \arg\min_{\sigma \in \mathfrak{S}_N} \sum_{j=1}^{N} \mathcal{L}_{\text{match}}(y_j, \hat{y}_{\sigma(j)}), \qquad \mathcal{L}_{\text{match}} = -\mathbb{1}_{\{c_j \neq \varnothing\}}\, \hat{p}_{\sigma(j)}(c_j) + \mathbb{1}_{\{c_j \neq \varnothing\}}\, \mathcal{L}_{\text{box}}(b_j, \hat{b}_{\sigma(j)}),$$

其中 $\mathfrak{S}_N$ 是所有排列的集合，c_j 與 b_j 是目標 j 的類別與框，$\hat{p}_{\sigma(j)}(c_j)$ 是被匹配的查詢對類別 c_j 的預測機率，而 $\mathcal{L}_{\text{box}} = \lambda_{L1}\|b_j - \hat{b}_{\sigma(j)}\|_1 + \lambda_{\text{iou}}(1 - \text{GIoU}(b_j, \hat{b}_{\sigma(j)}))$。這個指派問題可以用**匈牙利演算法**（Hungarian algorithm）[31] 在多項式時間內精確求解。接著，訓練損失（**集合損失**，set loss）使用匹配好的配對：

$$\mathcal{L}_{\text{Hungarian}} = \sum_{j=1}^{N} \Big[-\log \hat{p}_{\hat{\sigma}(j)}(c_j) + \mathbb{1}_{\{c_j \neq \varnothing\}}\, \mathcal{L}_{\text{box}}(b_j, \hat{b}_{\hat{\sigma}(j)}) \Big],$$

當 $c_j = \varnothing$ 時，對數機率項的權重降為十分之一，以平衡大量的「無物件」名額。因為每個物件只會匹配到一個查詢，重複的預測會被當成錯誤的「無物件」而受到**懲罰**，解碼器的自注意力因此學會抑制重複，這就是不需要 NMS 的原因。

用 SciPy 寫匹配步驟只需幾行。下面的成本以 DETR 的權重結合三項；`linear_sum_assignment` 可直接處理 $N \times M$ 的長方形矩陣（DETR 的 L1 項是在正規化的 (c_x, c_y, w, h) 上計算，這裡為了簡潔使用角點格式）。

```
import numpy as np
from scipy.optimize import linear_sum_assignment

def giou_matrix(a, b):
    x1 = np.maximum(a[:, None, 0], b[None, :, 0]); y1 = np.maximum(a[:, None, 1], b[None, :,
     ↪  1])
    x2 = np.minimum(a[:, None, 2], b[None, :, 2]); y2 = np.minimum(a[:, None, 3], b[None, :,
     ↪  3])
    inter = np.clip(x2 - x1, 0, None) * np.clip(y2 - y1, 0, None)
    area = lambda b: (b[:, 2] - b[:, 0]) * (b[:, 3] - b[:, 1])
    union = area(a)[:, None] + area(b)[None, :] - inter
    cx1 = np.minimum(a[:, None, 0], b[None, :, 0]); cy1 = np.minimum(a[:, None, 1], b[None,
     ↪  :, 1])
    cx2 = np.maximum(a[:, None, 2], b[None, :, 2]); cy2 = np.maximum(a[:, None, 3], b[None,
     ↪  :, 3])
    c = (cx2 - cx1) * (cy2 - cy1)
    return inter / union - (c - union) / c
```

```
rng = np.random.default_rng(1)
N, C = 6, 3                                      # 6 queries, 3 real classes (+ "no object")
logits = rng.normal(size=(N, C + 1))
prob = np.exp(logits) / np.exp(logits).sum(1, keepdims=True)
pred_boxes = np.array([[.10, .10, .40, .50], [.55, .20, .90, .60], [.12, .08, .42, .48],
                       [.30, .60, .60, .95], [.00, .00, .20, .20], [.50, .50, .70, .70]])
gt_boxes  = np.array([[.10, .10, .40, .50], [.32, .62, .62, .93], [.56, .22, .88, .62]])
gt_labels = np.array([0, 2, 1])

w_cls, w_l1, w_giou = 1.0, 5.0, 2.0              # weights used by DETR
cost = (-w_cls * prob[:, gt_labels]                          # (N, M) class term
        + w_l1 * np.abs(pred_boxes[:, None] - gt_boxes[None]).sum(-1)
        - w_giou * giou_matrix(pred_boxes, gt_boxes))
rows, cols = linear_sum_assignment(cost)         # works on rectangular N x M matrices
print("query -> gt:", dict(zip(rows.tolist(), cols.tolist())))
print("unmatched queries (trained as 'no object'):", sorted(set(range(N)) -
 ↪  set(rows.tolist())))
```

```
query -> gt: {0: 0, 1: 2, 3: 1}
unmatched queries (trained as 'no object'): [2, 4, 5]
```

查詢 2 幾乎是查詢 0 的複本，卻沒有被匹配，之後會被訓練成「無物件」。這正是取代 NMS 的機制。

代價。 DETR 需要非常長的訓練（數百個 epoch，Faster R-CNN 大約只要十幾個），小物件表現也差，因為在高解析度特徵圖上做全域注意力太昂貴，只能用單一一層粗特徵。後繼者逐一解決了這些問題。

- **Deformable DETR（2021）** [32] 以**可變形注意力**（deformable attention）取代密集注意力：每個查詢只關注參考點附近少數幾個學習得到的取樣點，並橫跨多個特徵層級。這讓多尺度特徵變得負擔得起；依論文所述，它以少 10 倍的訓練 epoch 達到比 DETR 更好的準確度（小物件尤其明顯）。
- **DINO（2022）** [33]（不要和同名的自監督方法混淆）結合了**對比去雜訊訓練**（把加了雜訊的真值框當成額外查詢送入並要求還原，雜訊較大的則必須被拒絕）、**混合查詢選擇**（以編碼器特徵初始化錨點位置），以及 **look-forward-twice** 的框更新。它報告在 COCO 上以標準的 50 層 CNN 骨幹訓練 12 個 epoch 得到 49.4 AP、24 個 epoch 得到 51.3 AP；以大型骨幹並在 Objects365 預訓練後，在 COCO val2017 上達到 63.2 AP。
- **RT-DETR（2024）** [34] 鎖定即時應用。**高效混合編碼器**只在最粗的一層使用注意力，並以卷積融合不同尺度；**IoU 感知的查詢選擇**挑出更好的初始查詢。論文報告其較小與較大版本在 COCO 上分別為 53.1 / 54.3 AP，在 T4 GPU 上達 108 / 74 FPS，在當時比尺寸相當的 YOLO 模型更快也更準，而且不需 NMS。

如今 DETR 路線與 YOLO 路線正在匯流：YOLOv10 借用一對一指派來去掉 NMS，即時 DETR 則借用高效的卷積頸部。

開放詞彙與定位偵測器

白話地說：封閉集合的偵測器只能找出它訓練過的 80 或 365 個類別。開放詞彙偵測器在**測試時以文字**接收類別清單，所以你可以直接要它找「紅色消防栓」或「堆高機」，不必重新訓練。

技巧在於把最後的分類層（一個由 K 個類別權重向量組成的固定矩陣）換成**文字嵌入**。對區域特徵 $\mathbf{z}_i$ 與提示詞（例如 “a photo of a forklift”）的文字嵌入 $\mathbf{e}_k$，類別分數是縮放後的餘弦相似度：

$$s_{ik} = \tau \cdot \frac{\mathbf{z}_i^\top \mathbf{e}_k}{\|\mathbf{z}_i\| \, \|\mathbf{e}_k\|},$$

其中 τ 是學習得到的溫度參數。任何能嵌入的片語都能變成一個類別。視覺端必須學會把區域與語言對齊，這需要遠比任何框標註資料集更大量的影像–文字資料。

- **OWL-ViT（2022）** [35] 從以影像–文字對比預訓練的 Transformer 影像編碼器與文字編碼器出發，移除最後的池化，在每個輸出 token 上接上輕量的框與類別偵測頭；類別偵測頭就是上面的文字嵌入。它支援零樣本（文字查詢）與單樣本（影像範例查詢）偵測，論文也顯示模型越大效果越穩定提升。
- **Grounding DINO（2024）** [36] 把語言深度融入 DINO 偵測器：影像與文字互相做交叉注意力的特徵增強器、語言引導的查詢選擇，以及跨模態解碼器。它在偵測、定位（片語對應區域）與圖說資料上預訓練，報告在 COCO 零樣本遷移（完全不用 COCO 訓練資料）上達 52.5 AP，在一組多樣化真實世界偵測資料集組成的基準上平均 AP 為 26.1。
- **YOLO-World（2024）** [37] 以視覺–語言路徑聚合網路（RepVL-PAN）與區域–文字對比預訓練，把開放詞彙帶進即時偵測器。它的**先提示、後偵測**（prompt-then-detect）模式會事先離線編碼使用者的詞彙，因此推論成本與封閉集合的 YOLO 相當。論文報告在 LVIS 上達 35.4 AP，在 V100 GPU 上 52.0 FPS。

可提示的基礎模型更進一步。SAM 3 [38]（2025）接受一個簡短名詞片語或影像範例作為**概念提示**（concept prompt），為影像與影片中**所有**符合的實例輸出遮罩與身分；作者建立了一個資料引擎，產生含有 400 萬個不重複概念標籤的資料集，並報告在其可提示概念分割基準上的準確度是既有系統的兩倍。在這類系統中，偵測成為一個通用「找出這個概念」模型的其中一種輸出，而不是單獨訓練的網路。

小物件與密集場景

白話地說：極小的物件只有很少的像素，步幅 32 的特徵可能把它壓縮到不到一個格子。人群之所以困難，是因為物件互相遮擋，而 NMS 會把相鄰的物件誤認為重複。

小物件。 COCO 把面積小於 32^2 像素的物件定義為「小」[40]。經過步幅 16 的骨幹網路後，這樣的物件最多只占約 2×2 個特徵格，外觀大多在降採樣中流失。IoU 也變得嚴苛：在 10×10 的框上偏差 2 個像素，IoU 就可能跌破 0.7。常見的對策有高解析度特徵層（步幅 4 的 P_2）、特徵金字塔、推論時把影像切塊、提高輸入解析度、複製貼上與尺度擴增，以及不會讓小框缺少正樣本的指派規則。Cheng 等人的綜述與基準 [39] 整理了這些策略，並提出兩個大型小物件基準：SODA-D（24,828 張交通影像、278,433 個實例、9 個類別）與 SODA-A（2,513 張高解析度空拍影像、872,069 個實例、9 個類別）。

密集與遮擋場景。在人群中，真實物件彼此大量重疊，固定的 NMS 門檻只能在召回率與重複之間取捨。Soft-NMS [13] 有幫助；集合預測偵測器則透過學習哪些預測是重複的，完全避開門檻。遮擋也讓框的標註變得模糊（要標完整範圍，還是只標看得見的部分？），評估結果因此取決於協定。這些問題在追蹤中會再次出現，因為物件會消失又重新出現。

偵測器的評估

白話地說：把所有偵測結果依信心排序，由上往下走：每個偵測結果要不是命中（它和一個尚未被認領的真實物件夠吻合），就是誤報。記錄精確率與召回率如何變化；這條曲線下的面積就是平均精確率（Average Precision, AP）。

精確率、召回率與 AP

對單一類別與單一 IoU 門檻 θ，把測試集中所有偵測結果依分數由高到低排序。若某偵測結果與同類別、尚未被匹配的真值框 IoU 至少為 θ，就是**真陽性**（TP）；否則是**偽陽性**（FP）。每個真值框只能被匹配一次，重複的偵測就是這樣被懲罰的。取前 k 個偵測結果時，

$$\text{precision}(k) = \frac{\text{TP}(k)}{k}, \qquad \text{recall}(k) = \frac{\text{TP}(k)}{M},$$

其中 M 是該類別真值物件的數量。為了讓曲線單調，精確率以其上包絡線**內插**：$p_{\text{interp}}(r) = \max_{r' \ge r} p(r')$。接著：

- **VOC 11 點 AP**（PASCAL VOC 到 2009 年為止 [3]）：$\text{AP} = \frac{1}{11} \sum_{r \in \{0, 0.1, \dots, 1\}} p_{\text{interp}}(r)$。
- **VOC 全點 AP**（VOC 2010 起 [41]）：內插曲線下的精確面積，$\text{AP} = \sum_n (r_{n+1} - r_n)\, p_{\text{interp}}(r_{n+1})$，對所有不同的召回值加總。
- **COCO AP** [40]：在 $0, 0.01, \dots, 1$ 共 101 個召回點上取 p_{interp} 的平均。

圖 6 顯示一條 PR 曲線、它的包絡線，以及 AP 面積。

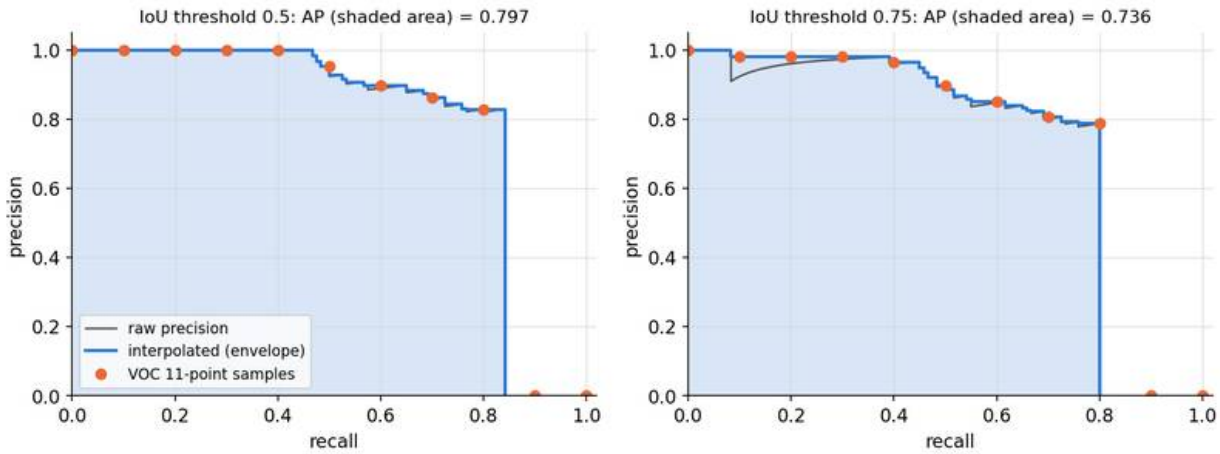


圖 *6* — 合成偵測結果在兩個 *IoU* 門檻下的精確率–召回率。內插包絡線下的陰影面積即全點 *AP*；橘點是 *VOC 11* 點取樣。精確率在召回率終止處降為 *0*，因為有些物件從未被偵測到。

COCO 協定

COCO 的主要指標寫作 **AP** 或 **mAP@[.5:.95]**，對 10 個 IoU 門檻 $\theta \in \{0.50, 0.55, \dots, 0.95\}$ 及全部 80 個類別取平均 [40]：

$$\mathrm{AP}_{\mathrm{COCO}} = \frac{1}{10\,K} \sum_{\theta} \sum_{k=1}^{K} \mathrm{AP}_k(\theta).$$

它獎勵的是精確定位，而不只是找到物件。配套指標有 AP_{50}（VOC 式指標）、AP_{75}（嚴格），以及針對小（面積 $< 32^2$）、中（32^2 到 96^2）、大（$> 96^2$）物件的 AP_S、AP_M、AP_L。每張影像最多只計分 100 個偵測結果。每張影像 1、10、100 個偵測結果下的平均召回率（AR）也會一併報告。注意 COCO 把類別平均後的指標稱為「AP」；論文中的「mAP」通常指同一件事。

下面的程式碼在合成預測上從頭計算所有指標（真值加上抖動、約 15% 漏偵測，再加上一些隨機的誤報）。

```
import numpy as np

def box_iou(a, b):
    """Pairwise IoU between (N,4) and (M,4) xyxy boxes -> (N, M)."""
    x1 = np.maximum(a[:, None, 0], b[None, :, 0]); y1 = np.maximum(a[:, None, 1], b[None, :,
     ↪  1])
    x2 = np.minimum(a[:, None, 2], b[None, :, 2]); y2 = np.minimum(a[:, None, 3], b[None, :,
     ↪  3])
    inter = np.clip(x2 - x1, 0, None) * np.clip(y2 - y1, 0, None)
    area = lambda b: (b[:, 2] - b[:, 0]) * (b[:, 3] - b[:, 1])
    return inter / (area(a)[:, None] + area(b)[None, :] - inter)

def match(dets, gts, thr):
    """dets: list over images of (boxes, scores); gts: list of gt boxes.
    Returns scores, TP flags (sorted by score), and number of GT."""
    recs = []
    for img, ((db, ds), gb) in enumerate(zip(dets, gts)):
        for k in range(len(ds)):
            recs.append((ds[k], img, k))
    recs.sort(key=lambda r: -r[0])                       # global ranking by confidence
    used = [np.zeros(len(g), bool) for g in gts]
    tp = np.zeros(len(recs))
    for n, (s, img, k) in enumerate(recs):
        gb = gts[img]
        if len(gb) == 0:
            continue
        ious = box_iou(dets[img][0][k:k + 1], gb)[0]
        ious[used[img]] = -1                             # each GT can be matched once
        j = ious.argmax()
        if ious[j] >= thr:
            tp[n] = 1; used[img][j] = True
```

```
    return tp, sum(len(g) for g in gts)

def pr_curve(tp, n_gt):
    ctp, cfp = np.cumsum(tp), np.cumsum(1 - tp)
    return ctp / n_gt, ctp / (ctp + cfp)                # recall, precision

def ap(rec, prec, mode="all"):
    r = np.concatenate([[0], rec, [1]]); p = np.concatenate([[0], prec, [0]])
    p = np.maximum.accumulate(p[::-1])[::-1]            # monotone envelope
    if mode == "all":                                   # VOC2010+: exact area
        i = np.where(r[1:] != r[:-1])[0]
        return np.sum((r[i + 1] - r[i]) * p[i + 1])
    pts = np.linspace(0, 1, 11 if mode == "voc11" else 101)
    return np.mean([p[r >= t].max() if np.any(r >= t) else 0 for t in pts])

# synthetic data: 50 images, 1-4 objects each; detector = jittered GT + false positives
rng = np.random.default_rng(0)
gts, dets = [], []
for _ in range(50):
    n = rng.integers(1, 5)
    xy = rng.uniform(0, 200, (n, 2)); wh = rng.uniform(20, 80, (n, 2))
    g = np.hstack([xy, xy + wh]); gts.append(g)
    keep = rng.random(n) > 0.15                         # miss ~15% of objects
    jit = rng.normal(0, 0.06, (n, 4)) * np.hstack([wh, wh])
    db = (g + jit)[keep]; ds = rng.uniform(0.4, 1.0, keep.sum())
    m = rng.integers(0, 3)                              # 0-2 false positives
    fxy = rng.uniform(0, 200, (m, 2)); fwh = rng.uniform(20, 80, (m, 2))
    db = np.vstack([db, np.hstack([fxy, fxy + fwh])]); ds = np.concatenate([ds,
     ↪  rng.uniform(0, 0.7, m)])
    dets.append((db, ds))

for thr in (0.5, 0.75):
    tp, n_gt = match(dets, gts, thr)
    rec, prec = pr_curve(tp, n_gt)
    print(f"IoU {thr}: VOC11 {ap(rec, prec, 'voc11'):.3f}  all-point {ap(rec, prec):.3f}"
          f"  COCO-101 {ap(rec, prec, 'coco'):.3f}")
aps = [ap(*pr_curve(*match(dets, gts, t)), "coco") for t in np.arange(0.5, 0.96, 0.05)]
print("AP@[.5:.95] =", round(float(np.mean(aps)), 3))
```

```
IoU 0.5: VOC11 0.776  all-point 0.797  COCO-101 0.797
IoU 0.75: VOC11 0.749  all-point 0.736  COCO-101 0.738
AP@[.5:.95] = 0.532
```

可以看出三個教訓。三種內插規則得到相近但**不相同**的數字，所以不同協定算出的 AP 絕對不能互相比較。AP@[.5:.95] 比 AP50 低得多，因為抖動的框通不過嚴格門檻。而這只是單一類別；真正的 mAP 會對類別取平均，所以稀有類別和常見類別占同樣的份量。

AP 沒告訴你的事

AP 總結的是整個測試集上的排序品質，因此它不會替你選定操作門檻，也不會區分錯誤的**類型**：定位錯誤、與相似類別混淆、重複偵測、背景誤報，在 AP 裡看起來都一樣。診斷模型時，要依錯誤類型與物件大小拆開分析，並且實際去看影像。

資料集與基準

白話地說：偵測的進展由少數幾個公開資料集推動；每一個都改變了「好」的定義。

資料集	類別	規模（依論文）	標註	重要性
PASCAL VOC [3]	20	以今日標準來看很小	框（＋部分遮罩）	2005–2012 年的基準；AP50
MS COCO [4]	標註 91 種；偵測用 80 種	32.8 萬張影像、250 萬個標註實例	框＋實例遮罩	雜亂的日常場景、大量小物件；AP@[.5:.95]
Objects365 [42]	365	大規模、高品質的框	框	強大偵測器常用的預訓練資料集
Open Images V4 [43]	600 個有框類別	920 萬張影像；1,540 萬個框	框、影像標籤、關係	非常大；影像層級有 1.98 萬個概念
LVIS [44]	超過 1000	16.4 萬張影像中約 200 萬張遮罩	實例遮罩	長尾詞彙；稀有類別評估

有兩個趨勢很明顯。類別數持續增加（20、80，到數百乃至上千），隨之而來的是**長尾**問題：在 LVIS 中，許多類別只有少數幾個訓練樣本，忽略頻率的損失函數或取樣器在稀有類別上就會失敗。另外，最大的模型如今都先在 Objects365 或影像–文字資料上預訓練，再於 COCO 上微調，使得「只用 COCO 訓練」與「使用額外資料」成為兩個分開的排行榜。

速度、準確度與部署

白話地說：實務上你是在一條曲線上選一個點：更準通常代表更多運算。真正重要的是在**你的**硬體上、端到端的延遲。

- **FLOPs 不等於延遲**。 FLOPs 計算的是算術量，但執行時間還取決於記憶體存取、運算子支援與平行度。深度可分離卷積與注意力在 FLOPs 上可能很便宜，在某些加速器上卻很慢。
- **端到端量測**。回報延遲時要包含前處理、網路**以及**後處理。NMS 的成本隨候選框數量增加（最壞情況與數量的平方成正比）、依資料而定，而且常在 CPU 上執行。RT-DETR 的論文 [34] 正是分析了這一點，這也是 RT-DETR 與 YOLOv10 [24] 等免 NMS 設計對部署很有吸引力的原因之一。
- **輸入解析度**是最強的調整旋鈕：邊長減半，運算量約減為四分之一，但受傷最重的是小物件。
- **量化**（quantization，例如 8 位元整數的權重與活化值）與**重參數化**（訓練時用多分支區塊，之後折疊成單一卷積，如 YOLOv7 [22] 所用）能在小幅損失準確度下降低延遲。框迴歸輸出與分數頭通常對量化最敏感，所以要用真實資料校準，並且檢查 AP75，而不只是 AP50。
- **批次大小 1** 是邊緣裝置的常見情境；以大批次量測的吞吐量數字不能直接套用。

一個你可以自己訓練的小型偵測器

為了親眼看到邊界框迴歸與 IoU 損失端到端地運作，下面的 PyTorch 程式訓練一個小 CNN，在帶雜訊的 32×32 影像中定位一個矩形或橢圓。這是單一物件偵測器：網路透過 sigmoid 直接迴歸 $[0,1]$ 範圍內的 (c_x, c_y, w, h)，以 L1 加上 GIoU 損失訓練，和 DETR 對匹配到的查詢所做的一樣。

```
import time, torch, torch.nn as nn
torch.manual_seed(0)
S = 32
torch.set_num_threads(4)

def make_batch(n):
    """One white rectangle or ellipse per 32x32 noisy image; target = (cx, cy, w, h) in [0,
     ↪  1]."""
    img = 0.1 * torch.randn(n, 1, S, S)
    wh = torch.rand(n, 2) * 0.4 + 0.15
    c = wh / 2 + torch.rand(n, 2) * (1 - wh)
    yy, xx = torch.meshgrid(torch.arange(S), torch.arange(S), indexing="ij")
    xx, yy = (xx[None] + 0.5) / S, (yy[None] + 0.5) / S
    dx = (xx - c[:, 0, None, None]) / (wh[:, 0, None, None] / 2)
    dy = (yy - c[:, 1, None, None]) / (wh[:, 1, None, None] / 2)
    rect = (dx.abs() <= 1) & (dy.abs() <= 1)
    ell = dx**2 + dy**2 <= 1
    shape = torch.where((torch.rand(n) < 0.5)[:, None, None], rect, ell)
    return img + shape[:, None].float(), torch.cat([c, wh], 1)

def xyxy(b):
    return torch.cat([b[:, :2] - b[:, 2:] / 2, b[:, :2] + b[:, 2:] / 2], 1)

def giou(p, g):
    p, g = xyxy(p), xyxy(g)
    lt, rb = torch.max(p[:, :2], g[:, :2]), torch.min(p[:, 2:], g[:, 2:])
    inter = (rb - lt).clamp(min=0).prod(1)
    area = lambda b: (b[:, 2:] - b[:, :2]).prod(1)
    union = area(p) + area(g) - inter
    c = (torch.max(p[:, 2:], g[:, 2:]) - torch.min(p[:, :2], g[:, :2])).prod(1)
    return inter / union, inter / union - (c - union) / c

net = nn.Sequential(
    nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),   # 16
    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),  # 8
    nn.Conv2d(32, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),  # 4
    nn.Flatten(), nn.Linear(32 * 4 * 4, 128), nn.ReLU(),
    nn.Linear(128, 4), nn.Sigmoid())                               # (cx, cy, w, h) in [0, 1]
opt = torch.optim.Adam(net.parameters(), 2e-3)

t0 = time.time()
for step in range(500):
    x, y = make_batch(64)
    pred = net(x)
    iou, g = giou(pred, y)
    loss = (pred - y).abs().sum(1).mean() + (1 - g).mean()        # L1 + GIoU loss
    opt.zero_grad(); loss.backward(); opt.step()
```

```
with torch.no_grad():
    x, y = make_batch(1000)
    iou, _ = giou(net(x), y)
print(f"train time {time.time() - t0:.1f}s | test mean IoU {iou.mean():.3f}"
      f" | IoU>=0.5: {(iou >= 0.5).float().mean():.1%} | IoU>=0.75: {(iou >=
      ↪ 0.75).float().mean():.1%}")
```

```
train time 21.1s | test mean IoU 0.885 | IoU>=0.5: 100.0% | IoU>=0.75: 97.8%
```

（訓練時間視你的 CPU 而定。）因為恰好只有一個物件，所以沒有指派問題，也不需要 NMS。練習 6 會請你把它擴充到多個物件，而本篇所有困難的問題都會在那裡重新出現。

現代觀點

圖 7 以每篇引用文獻的發表年份，把本篇的方法放在時間軸上。

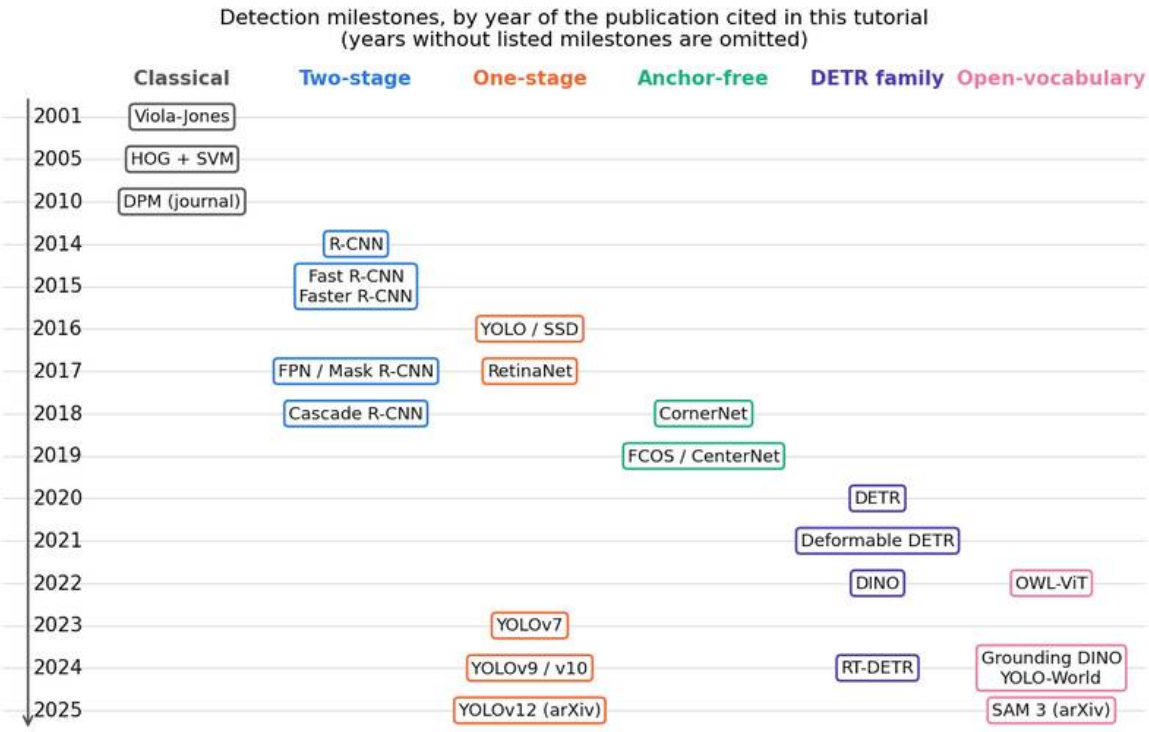


圖 7 — 依本篇引用文獻發表年份排列、按家族分組的里程碑。沒有列出里程碑的年份已省略。

綜述怎麼說

《**Object Detection in 20 Years**》（Zou 等人，*Proceedings of the IEEE*，2023）[1] 是涵蓋最廣的歷史回顧。它涵蓋從傳統時代（Viola–Jones、HOG、DPM）到深度二階段與一階段方法的里程碑偵測器、偵測資料集與指標、基本元件（多尺度偵測、上下文、困難負樣本挖掘、NMS、框迴歸）、加速技術，以及近期最先進的方法。它的主要訊息是：大多數「新」想法都有更早的根源——多尺度掃描變成特徵金字塔，串接變成串接式偵測頭，困難負樣本挖掘變成焦點損失，而 NMS 被重新思考了好幾次。

《**Deep Learning for Generic Object Detection: A Survey**》（Liu 等人，*IJCV*，2020）[2] 深入回顧了 300 多篇研究，並依偵測框架、物件特徵表示、候選區域產生、上下文建模、訓練策略與評估指標加以整理。它是 Transformer 之前 CNN 時代最好的單一參考文獻，並強調進展同樣來自訓練策略、資料與評估，而不只是架構。

《**Object Detection with Transformers: A Review**》（Shehzadi 等人，*Sensors*，2025）[45] 分析了 21 項對 DETR 的近期改進，依骨幹、查詢設計與注意力機制的修改分類，並在共同基準上比較。它的分類法對應到上面描述的修正：用多尺度與稀疏注意力改善小物件與速度，用更好的查詢初始化與去雜訊改善收斂。

《**A Survey on Open-Vocabulary Detection and Segmentation: Past, Present, and Future**》（Zhu 與 Chen，*IEEE TPAMI*，2024）[46] 依照如何利用弱監督，把開放詞彙方法分成六個家族：視覺–語意空間映射、新類別視覺特徵合成、區域感知訓練、偽標籤、知識蒸餾與遷移學習。它涵蓋 2D、3D 與影片任務，並指出預訓練資料與測試資料在詞彙上的重疊，讓「新類別」的評估變得複雜。

《**Towards Large-Scale Small Object Detection**》（Cheng 等人，*IEEE TPAMI*，2023）[39] 聚焦在最頑強的失敗模式，依類別回顧各種對策，並提出前面介紹的 SODA 基準。它顯示即使是強大的偵測器，小物件的準確度仍遠低於較大的物件。

2023–2026 年的最先進技術

- **即時封閉集合偵測**如今是 YOLO 路線（YOLOv9 [23]、YOLOv10 [24]、YOLOv12 [25]）與即時 DETR（RT-DETR [34]）之間的競賽。兩條路線互相借鏡：YOLO 引入一對一指派與免 NMS 偵測頭，DETR 引入卷積頸部與混合編碼器。在即時預算下使用注意力已經負擔得起。
- **高準確度的封閉集合偵測**由 DETR 式模型主導，例如搭配大型骨幹與大規模預訓練（Objects365 [42] 及更多）的 DINO [33]。COCO 上的進步幅度已經很小，對最大的模型而言這個基準已接近飽和，因此注意力轉向更難的基準：LVIS [44]、開放詞彙與跨領域評估。
- **開放詞彙與定位偵測**（OWL-ViT [35]、Grounding DINO [36]、YOLO-World [37]）正走進實務：在許多應用中，你現在是去**提示**一個偵測器，而不是訓練一個，必要時再微調。
- **可提示的基礎模型**（SAM 3 [38]）以概念提示統一了偵測、分割與追蹤。

尚待解決的問題

1. **定位品質與信心分數**。分類分數與框品質分開訓練，兩者的相關性常常不高，所以「最好」的框可能被一個更有信心、卻更差的框抑制掉。
2. **小、擁擠與被遮擋的物件**仍遠未解決 [39]。

3. **長尾與開放世界辨識**。稀有類別與真正未見過的物件，以及知道模型**何時**不知道。
4. **評估**。COCO AP 隱藏了錯誤類型，在頂端已經飽和；開放詞彙的評估則取決於提示詞，以及預訓練資料裡原本就包含了什麼 [46]。
5. **在真實硬體上的效率**，包括免 NMS 設計、利於量化的偵測頭，以及公平的延遲報告。

重點整理

- 偵測輸出一個大小不定的（框、類別、分數）三元組集合；每種架構都是對「要輸出幾個、誰負責哪個物件？」的不同回答。
- 邊界框以相對於參考框、經尺度正規化的偏移量來迴歸（t_x, t_y 與對數尺度的 t_w, t_h）；以 IoU 為基礎的損失（GIoU、DIoU、CIoU）讓訓練和以 IoU 為基礎的指標一致。
- 錨框與指派規則決定哪個預測學習哪個物件；NMS 事後移除重複，Soft-NMS 緩和它在人群中的失敗。
- FPN 處理尺度；焦點損失處理讓密集偵測器變弱的前景–背景不平衡。
- 二階段（R-CNN、Fast、Faster、Mask、Cascade）精煉候選區域；一階段（YOLO、SSD、RetinaNet）一次密集預測；無錨框（CornerNet、CenterNet、FCOS）從點預測；DETR 以匈牙利匹配預測集合，不需要 NMS。
- 開放詞彙偵測器把類別清單變成文字提示，可提示的基礎模型則融合了偵測、分割與追蹤。
- 用正確的協定量測：VOC 11 點、VOC 全點與 COCO 101 點的 AP 各不相同；COCO AP 對 IoU 0.50–0.95 取平均，APs/m/l 則揭露尺度上的弱點。

練習

1. **手算編碼**。一個錨框中心為 $(100, 100)$、大小為 64×32。一個真值框中心為 $(110, 96)$、大小為 80×40。在不做正規化的情況下計算 (t_x, t_y, t_w, t_h)，再把它們解碼回去。

   提示

   $t_x = 10/64 = 0.156$，$t_y = -4/32 = -0.125$，$t_w = \log(80/64) = 0.223$，$t_h = \log(40/32) = 0.223$。解碼：$c_x = 100 + 0.156 \cdot 64 = 110$，$w = 64e^{0.223} = 80$，$y$ 與 h 同理。

2. **IoU 與平移**。證明兩個相同的 $a \times a$ 正方形沿單一軸平移 d 時，$\text{IoU} = (a - d)/(a + d)$。平移多少時 IoU 為 0.5 與 0.75？再對兩軸等量平移的情況重做一次（圖 2 右）。

   提示

   交集為 $a(a - d)$，聯集為 $2a^2 - a(a - d) = a(a + d)$。IoU 0.5 時 $d = a/3$；IoU 0.75 時 $d = a/7$。斜向平移時 $\text{IoU} = (a - d)^2/(2a^2 - (a - d)^2)$，所以 IoU 0.5 時 $d = a(1 - \sqrt{2/3}) \approx 0.18a$。

3. **貪婪 NMS 何時失敗**。修改 NMS 範例，讓兩個不同的人以 IoU 0.6 重疊，然後分別以門檻 0.5 與 0.7 執行貪婪 NMS，以及 Soft-NMS。哪一種能找回兩個人？高門檻又帶來什麼新問題？

提示

門檻 0.5 時，分數較低的那個人被刪掉。門檻 0.7 時兩人都留下，但同一個人 IoU 介於 0.5 到 0.7 的重複框也會留下，增加誤報。Soft-NMS 以降低後的分數保留第二個人（原分數乘上 $e^{-0.36/0.5} \approx 0.49$），在 AP 中仍會計入召回。

4. **焦點損失的數字**。在 $\gamma = 2$、$\alpha = 0.25$ 下，對一個背景錨框計算 $p = 0.01$（簡單）與 $p = 0.6$（困難）時的焦點損失與一般交叉熵。兩者各自差了多少倍？

提示

背景時 $p_t = 1-p$，$\alpha_t = 0.75$。簡單：$\mathrm{CE} = -\log 0.99 = 0.01005$；$\mathrm{FL} = 0.75 \cdot 0.01^2 \cdot 0.01005 \approx 7.5\times10^{-7}$，約小了 1.3×10^4 倍。困難：$\mathrm{CE} = -\log 0.4 = 0.916$；$\mathrm{FL} = 0.75 \cdot 0.6^2 \cdot 0.916 \approx 0.247$，只小了約 3.7 倍。

5. **協定之間的差異**。使用 AP 程式碼，（藉由改變亂數種子或抖動量）找出 VOC 11 點 AP **高於**全點 AP 的情況，以及低於的情況。說明為什麼兩者沒有固定的大小關係。

提示

11 點規則在固定的召回值取樣包絡線，所以若包絡線剛好在取樣點右側很高、之後馬上下降，就會高估；反之則低估。上面的輸出已經同時呈現兩種情形：IoU 0.5 時 11 點的值較低，0.75 時較高。

6. **從一個物件到多個物件**。把小型 PyTorch 偵測器擴充到最多三個形狀的影像。方案 A：像 YOLO v1 一樣預測 8×8 的（物件分數、框）網格，把每個物件指派給其中心所在的格子。方案 B：預測 5 個（類別、框）名額，並像 DETR 一樣以匈牙利匹配訓練。比較兩者的失敗模式。

提示

方案 A 在兩個中心落在同一格時會失敗，也需要 NMS 去除相鄰格子的重複框。方案 B 不需要 NMS，但訓練較慢，而且訓練初期匹配不穩定（同一個物件可能在不同步驟被指派給不同名額）。對 detach 後的成本矩陣使用 `scipy.optimize.linear_sum_assignment`。

偵測結果是追蹤的原料。下一步——在影片影格之間串連邊界框並維持身分——請見DLCV 06・物件追蹤。

參考文獻

1. Z. Zou, K. Chen, Z. Shi, Y. Guo and J. Ye, “Object detection in 20 years: A survey,” *Proceedings of the IEEE*, vol. 111, no. 3, 2023. doi
2. L. Liu, W. Ouyang, X. Wang, P. Fieguth, J. Chen, X. Liu and M. Pietikäinen, “Deep learning for generic object detection: A survey,” *International Journal of Computer Vision*, 2020. doi
3. M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn and A. Zisserman, “The PASCAL Visual Object Classes (VOC) challenge,” *International Journal of Computer Vision*, vol. 88, 2010. doi
4. T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick et al., “Microsoft COCO: Common objects in context,” in *Proc. ECCV*, 2014. doi

5. P. Viola and M. Jones, “Rapid object detection using a boosted cascade of simple features,” in *Proc. IEEE CVPR*, 2001. doi
6. N. Dalal and B. Triggs, “Histograms of oriented gradients for human detection,” in *Proc. IEEE CVPR*, 2005. doi
7. P. F. Felzenszwalb, R. B. Girshick, D. McAllester and D. Ramanan, “Object detection with discriminatively trained part-based models,” *IEEE TPAMI*, 2010. doi
8. R. Girshick, J. Donahue, T. Darrell and J. Malik, “Rich feature hierarchies for accurate object detection and semantic segmentation,” in *Proc. IEEE CVPR*, 2014. arXiv
9. R. Girshick, “Fast R-CNN,” in *Proc. IEEE ICCV*, 2015. arXiv
10. S. Ren, K. He, R. Girshick and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” *IEEE TPAMI*, 2017 (conference version 2015). doi
11. H. Rezatofighi, N. Tsoi, J. Gwak, A. Sadeghian, I. Reid and S. Savarese, “Generalized intersection over union: A metric and a loss for bounding box regression,” in *Proc. IEEE/CVF CVPR*, 2019. arXiv
12. Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye and D. Ren, “Distance-IoU loss: Faster and better learning for bounding box regression,” in *Proc. AAAI*, 2020. arXiv
13. N. Bodla, B. Singh, R. Chellappa and L. S. Davis, “Soft-NMS — Improving object detection with one line of code,” in *Proc. IEEE ICCV*, 2017. arXiv
14. T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan and S. Belongie, “Feature pyramid networks for object detection,” in *Proc. IEEE CVPR*, 2017. doi
15. T.-Y. Lin, P. Goyal, R. Girshick, K. He and P. Dollár, “Focal loss for dense object detection,” in *Proc. IEEE ICCV*, 2017. doi
16. K. He, G. Gkioxari, P. Dollár and R. Girshick, “Mask R-CNN,” in *Proc. IEEE ICCV*, 2017. doi
17. Z. Cai and N. Vasconcelos, “Cascade R-CNN: Delving into high quality object detection,” in *Proc. IEEE/CVF CVPR*, 2018. doi
18. J. Redmon, S. Divvala, R. Girshick and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE CVPR*, 2016. doi
19. J. Redmon and A. Farhadi, “YOLO9000: Better, faster, stronger,” arXiv:1612.08242, 2016. arXiv
20. J. Redmon and A. Farhadi, “YOLOv3: An incremental improvement,” tech. report, arXiv:1804.02767, 2018. arXiv
21. A. Bochkovskiy, C.-Y. Wang and H.-Y. M. Liao, “YOLOv4: Optimal speed and accuracy of object detection,” arXiv:2004.10934, 2020. arXiv
22. C.-Y. Wang, A. Bochkovskiy and H.-Y. M. Liao, “YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors,” in *Proc. IEEE/CVF CVPR*, 2023. doi
23. C.-Y. Wang, I-H. Yeh and H.-Y. M. Liao, “YOLOv9: Learning what you want to learn using programmable gradient information,” in *Proc. ECCV*, 2024. doi
24. A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin et al., “YOLOv10: Real-time end-to-end object detection,” in *Proc. NeurIPS*, 2024. arXiv
25. Y. Tian, Q. Ye and D. Doermann, “YOLOv12: Attention-centric real-time object detectors,” arXiv:2502.12524, 2025. arXiv
26. W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu and A. C. Berg, “SSD: Single shot multibox detector,” in *Proc. ECCV*, 2016. arXiv
27. H. Law and J. Deng, “CornerNet: Detecting objects as paired keypoints,” in *Proc. ECCV*, 2018. doi

28. X. Zhou, D. Wang and P. Krähenbühl, "Objects as points," arXiv:1904.07850, 2019. arXiv
29. Z. Tian, C. Shen, H. Chen and T. He, "FCOS: Fully convolutional one-stage object detection," in *Proc. IEEE/CVF ICCV*, 2019. arXiv
30. N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. ECCV*, 2020. doi
31. H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, 1955. doi
32. X. Zhu, W. Su, L. Lu, B. Li, X. Wang and J. Dai, "Deformable DETR: Deformable transformers for end-to-end object detection," in *Proc. ICLR*, 2021. arXiv
33. H. Zhang, F. Li, S. Liu, L. Zhang, H. Su, J. Zhu, L. M. Ni and H.-Y. Shum, "DINO: DETR with improved denoising anchor boxes for end-to-end object detection," arXiv:2203.03605, 2022. arXiv
34. Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang et al., "DETRs beat YOLOs on real-time object detection," in *Proc. IEEE/CVF CVPR*, 2024. doi
35. M. Minderer, A. Gritsenko, A. Stone, M. Neumann, D. Weissenborn et al., "Simple open-vocabulary object detection with vision transformers," in *Proc. ECCV*, 2022. arXiv
36. S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang et al., "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," in *Proc. ECCV*, 2024. doi
37. T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang et al., "YOLO-World: Real-time open-vocabulary object detection," in *Proc. IEEE/CVF CVPR*, 2024. doi
38. N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu et al., "SAM 3: Segment anything with concepts," arXiv:2511.16719, 2025. arXiv
39. G. Cheng, X. Yuan, X. Yao, K. Yan, Q. Zeng et al., "Towards large-scale small object detection: Survey and benchmarks," *IEEE TPAMI*, vol. 45, no. 11, 2023. arXiv
40. COCO Consortium, "COCO detection evaluation," official evaluation description (12 metrics, 101-point interpolation). link
41. M. Everingham, S. M. A. Eslami, L. Van Gool, C. K. I. Williams, J. Winn and A. Zisserman, "The PASCAL Visual Object Classes challenge: A retrospective," *International Journal of Computer Vision*, vol. 111, 2015. doi
42. S. Shao, Z. Li, T. Zhang, C. Peng et al., "Objects365: A large-scale, high-quality dataset for object detection," in *Proc. IEEE/CVF ICCV*, 2019. doi
43. A. Kuznetsova, H. Rom, N. Alldrin, J. Uijlings, I. Krasin et al., "The Open Images Dataset V4: Unified image classification, object detection, and visual relationship detection at scale," *International Journal of Computer Vision*, 2020. arXiv
44. A. Gupta, P. Dollár and R. Girshick, "LVIS: A dataset for large vocabulary instance segmentation," in *Proc. IEEE/CVF CVPR*, 2019. doi
45. T. Shehzadi, K. A. Hashmi, M. Liwicki, D. Stricker and M. Z. Afzal, "Object detection with transformers: A review," *Sensors*, vol. 25, no. 19, 2025. doi
46. C. Zhu and L. Chen, "A survey on open-vocabulary detection and segmentation: Past, present, and future," *IEEE TPAMI*, 2024. doi

DLCV 04・深度影像復原：低階視覺

同一個退化模型、許多個任務：深度網路如何去除模糊、雜訊、低解析度、霧、雨、雪與暗光，從 SRCNN、DnCNN 一路到 Restormer、all-in-one 模型與擴散先驗。

網站版本: https://blog.shyandram.dev/zh-tw/learn/deep-learning/dl-04-image-restoration/

你將學到

- 什麼是「低階視覺」(low-level vision)，以及為什麼超解析度、去雜訊、去模糊、除霧、除雨、除雪與低光增強其實是**同一個問題**：反轉一個退化 $y = \mathcal{D}(x) + n$。
- 深度影像復原在實務上怎麼設定：成對資料、**合成退化流程** (synthetic degradation pipeline)，以及它與真實世界（盲）退化之間的落差。你會用 PyTorch 實作一條退化流程，並訓練一個小小的殘差去雜訊網路。
- 各任務的里程碑與目前主流家族，從 SRCNN、DnCNN、DehazeNet 與 GoPro 去模糊基準，到 SwinIR、Restormer、NAFNet、MambaIR 與以擴散模型為基礎的超解析度(SUPIR、OSEDiff)。
- **All-in-one** 模型（AirNet、TransWeather、PromptIR、AdaIR、DA-CLIP、InstructIR）如何用一個網路處理未知或混合的退化。
- 該用哪些損失函數與評估指標、為什麼 PSNR 和「看起來真實」會互相矛盾（**感知–失真權衡**，perception–distortion trade-off），以及 NTIRE 挑戰賽如何追蹤這個領域。

先看全貌

低階視覺處理的是像素本身。在系統能辨認人臉、讀出路牌或偵測車輛之前，影像得先清楚到「看得見」。影像復原模型就是流程中負責這一步的部分：拿到一張受損的照片（太小、雜訊多、模糊、起霧、下雨、太暗），輸出對乾淨場景更好的估計。

> **用最簡單的話說**
>
> 想像有人拿給你一張被洗衣機洗過的照片：糊掉了、有斑點、也褪色了。你這輩子看過成千上萬張正常的照片，所以你猜得出它**原本**長什麼樣子：人臉有兩隻眼睛、建築物的邊是直的、天空是平滑的。復原網路做的是同一件事。它看過幾百萬組「洗之前／洗之後」的照片，學會了乾淨照片長什麼樣、洗衣機又會怎麼破壞它，再同時用這兩種知識把損傷還原。

為什麼重要：每支手機相機在你看到照片之前，都已經做過去雜訊、去馬賽克（demosaicking）與超解析度。衛星、顯微鏡、醫學與監視影像在量測之前都要先復原。影音串流服務會把低位元率的畫面放大。自駕車需要偵測器在雨霧中照常運作。從研究的角度看，許多架構想法也是先在復原任務上被驗證的：殘差學習、通道注意力、高效率 Transformer 與擴散先驗，都有知名的復原版本。

關於本篇

本篇改寫並擴充自作者先前的文章〈Low-Level Vision Task-Image Restoration 簡介〉[1]。保留原文的順序與觀點，把 2026 年的更新與更正直接寫進正文，並補上數學、程式碼與綜述文獻。

本篇假設你已熟悉 CNN 與反向傳播（DLCV 01），以及傳統影像復原（IP 05），後者詳細介紹了雜訊模型、Wiener 濾波與退化模型。這裡會直接連結過去，不再重複。

什麼是低階視覺？

白話版。高階視覺問的是「圖裡有什麼？」（分類、偵測）；低階視覺問的是「這張圖逐個像素來看，實際上長什麼樣？」前者關於**理解**，後者關於**感知**。

作者在原文 [1] 中的比喻，來自人與外界互動的方式：我們先**感知**（光打在視網膜上，看見邊緣、顏色與對比），再**理解**（那是一個杯子）。兩者缺一不可。如果感知很差（杯子很模糊），理解也會跟著受影響。低階視覺任務就是為了讓感知重新變好而存在：當我們說一個東西「很模糊」，代表感官對它的判定並不靈敏，於是就有了把它變「清晰」的去模糊任務。

精確版。低階視覺任務把一張影像映射成內容（通常）相同的另一張影像：

$$f_\theta : \mathbb{R}^{H\times W\times C} \to \mathbb{R}^{H'\times W'\times C'},$$

輸出是逐像素的估計，而不是一個標籤。H, W, C 是輸入的高、寬與通道數，H', W', C' 是輸出的（超解析度倍率為 s 時 $H' = sH$，其他情況通常相等）。這帶來三個影響整個領域的特性：

1. **輸出密集且高解析度**。每個像素都重要，所以架構不能大幅降採樣，必須保留細節，這讓高解析度下的運算成本很高。
2. **監督訊號很容易合成**。拿一張乾淨影像，用已知的方式把它弄壞，就得到一組輸入–目標配對。這是相對於辨識任務（需要人工標註）的巨大優勢，也是這個領域最大弱點的來源：合成的損傷終究不是真實的損傷。
3. 「**正確答案**」**並不唯一**。很多張乾淨影像都和同一張受損影像相容。哪一張「最好」，取決於你重視的是忠於真實還是看起來自然，後面的評估章節會詳細討論。

這類任務範圍很廣：去模糊、超解析度、去雜訊、修補（inpainting）、色彩校正、紅外線與可見光融合等等。過去大家常在 Papers with Code 上依任務瀏覽分類與排行榜，但它已停止營運，原網址現在會直接轉址到 Hugging Face 的 Trending Papers 頁面，依任務分類的 SOTA 頁面已無法瀏覽。現在追蹤進展的可靠方式，是每年的 **NTIRE** 挑戰賽報告（New Trends in Image Restoration and Enhancement，CVPR 的一個 workshop），本篇接近結尾時會介紹 [2], [3]。

一個問題、許多面貌：退化模型

白話版。本篇的每一個任務都可以寫成：「相機看到乾淨的場景 x，某種東西破壞了它，我們拿到的是 y。」改變的只有那個「破壞」。只要能把破壞寫下來，就能說出這是哪個任務。

統一的觀點

$$y = \mathcal{D}(x) + n,$$

其中 x 是未知的乾淨影像，$\mathcal{D}$ 是退化運算子（可能是非線性的，也可能未知），n 是雜訊，y 是觀測到的影像。復原就是從 y 估計 x。以下是原文 [1] 與本篇後面用到的具體形式，圖 1 把每一種套用在同一張照片上。

模糊（課本上的模型）。和IP 05 一樣，

$$g(u,v) = (h * f)(u,v) + n(u,v),$$

其中 f 是清晰影像，h 是點擴散函數（模糊核），$*$ 是二維卷積，n 是加性雜訊，g 是觀測影像，(u, v) 是像素座標。去雜訊是 $h = \delta$（單位脈衝，也就是沒有模糊）的特例。

超解析度。低解析度影像是高解析度影像經過模糊、次取樣再加上雜訊的結果：

$$y = (x * k)\downarrow_s + n,$$

其中 k 是模糊（抗鋸齒）核，$\downarrow_s$ 表示每個方向每 s 個像素取一個，s 是放大倍率（2、3、4.....）。經典的「bicubic」基準設定則把這一切換成 MATLAB 風格的雙三次降採樣，並且不加雜訊。

壓縮。 $y = \mathrm{JPEG}_q(x)$，q 是品質因子。JPEG 會量化 8×8 區塊的 DCT 係數（見IP 08），產生區塊效應與振鈴。它既非加性也非線性，所以通常直接當成黑盒子放進退化流程。

霧（大氣散射模型）。

$$I(p) = J(p)\,t(p) + A\,(1 - t(p)), \qquad t(p) = e^{-\beta d(p)},$$

其中 p 是像素位置，I 是有霧影像，J 是無霧的場景輻射，A 是全域大氣光（霧的顏色），$t \in (0, 1]$ 是透射率，β 是散射係數，d 是場景深度。第一項是被衰減的場景；第二項是被散射進相機的「空氣光」（airlight）。越遠的物體（d 大）t 越小，逐漸融入 A。

雨。雨線（rain streak）常用加性模型：

$$I = B + R,$$

其中 B 是無雨背景，R 是雨層。附著在鏡頭或擋風玻璃上的雨滴（raindrop）行為不同：它會遮住並折射背景，所以常用的模型是 $I = (1 - M) \odot B + R$，其中 M 是雨滴的二值遮罩，$\odot$ 是逐元素乘積，R 是穿過雨滴的光 [39]。

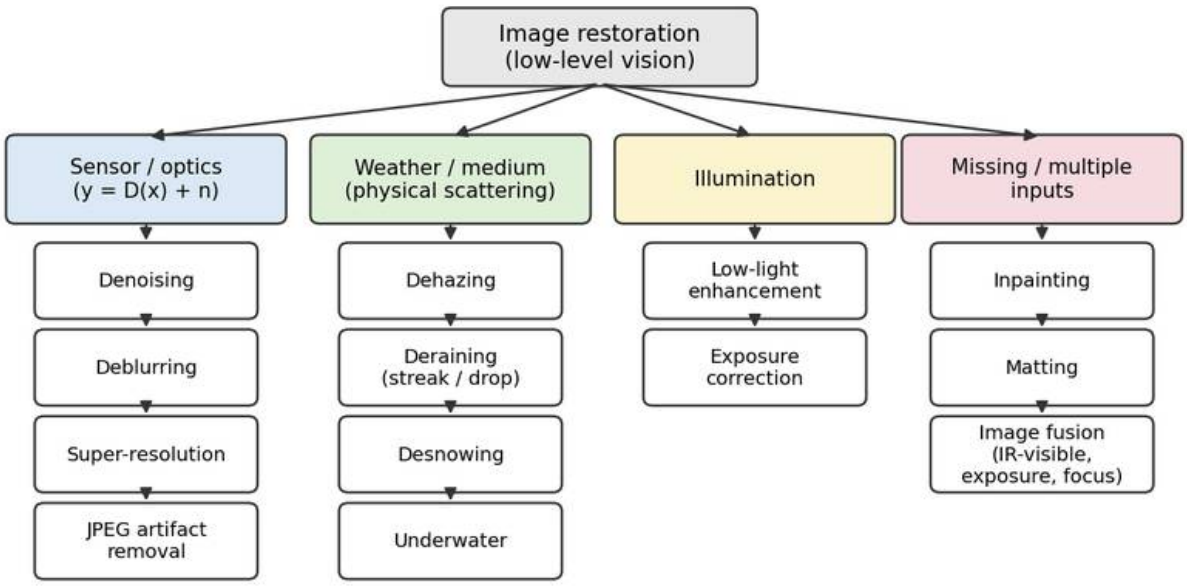
低光（Retinex）。 Retinex 理論 [5] 把影像分解成反射率與光照：

$$S = R \cdot L,$$

其中 S 是觀測影像，R 是反射率（物體表面本身的顏色與紋理，這是我們要的），L 是光照，$\cdot$ 是逐元素乘積。暗照片的 L 很小；增強就是估計 L 並把它提亮，同時原本藏在暗處的雜訊也會被放大。

圖 *1* — 同一張乾淨影像 x 經過本節各種退化模型。每個復原任務都想還原其中一種（或幾種）退化。

這些模型都不完全精確，卻有相同的形狀：乾淨影像進去，運算子加上雜訊出來。這正是原文 [1] 的核心觀點：雖然不是很嚴謹，但透過退化模型來看，它們本質上都可以用同一種方式「理解」，**屬於同一種問題**，同一套工具（而且常常是同一個網路）都適用。圖 2 依照損傷的來源把它們分類。



All-in-one models try to cover several boxes with one network

圖 2 — 依退化來源整理的低階視覺任務分類。*All-in-one* 模型試圖用單一網路涵蓋多個方框。

為什麼困難：病態問題

上面幾乎每一種 $\mathcal{D}$，都有許多不同的 x 會產生相同的 y。降採樣 4 倍丟掉每 16 個像素中的 15 個；模糊把高頻壓到幾乎為零；$t \approx 0$ 的濃霧會抹去遠處的物體。這個反問題是**病態的**（ill-posed），我們需要一個關於乾淨影像長相的**先驗**（prior）。用貝氏（MAP）形式寫：

$$\hat{x} = \arg\min_{x} \; \tfrac{1}{2}\|y - \mathcal{D}(x)\|_2^2 + \lambda\, \Phi(x),$$

第一項（資料保真項）要求 $\hat{x}$ 必須能解釋觀測，Φ 是編碼先驗的正則項（自然的影像 Φ 小），$\lambda > 0$ 平衡兩者。整個領域的歷史，就是一部尋找更好先驗 Φ 的歷史。

深度學習之前：手工先驗的時代

在深度學習興起之前，每個任務都有自己精心推導的先驗，成敗取決於對問題是否有足夠充分的物理或統計知識（prior knowledge）。去雜訊用局部平滑、非局部自相似與稀疏性；去模糊用 Wiener 與限制最小平方濾波，加上重尾的梯度先驗；除霧用暗通道先驗 [25]；超解析度用插值與以範例為基礎的字典；低光增強用直方圖等化與 Retinex 分解。IP 05 涵蓋了這套工具的核心（雜訊模型、平均／中位數／自適應濾波、反濾波、Wiener 與 CLS 濾波），結尾也簡短回顧了走向學習式去雜訊的歷史。作者在 [1] 中的看法是：深度學習逐漸解決並超越這些模型能做到的程度之後，關鍵不再是為每個任務建立強大的理論先驗，而是強大的**學習機制與特徵擷取方法**。退化模型並沒有消失，它從求解器搬進了**資料產生器**。

深度學習的問題設定

白話版。做出大量（受損, 乾淨）的影像對，訓練網路把受損的變成乾淨的。大部分的苦工都在讓損傷看起來夠真實。

以成對資料做監督式訓練

給定乾淨影像集合 $\{x_i\}$ 與一個退化取樣器，建立配對 (y_i, x_i)，其中 $y_i = \mathcal{D}_i(x_i) + n_i$，然後求

$$\theta^\star = \arg\min_{\theta} \; \frac{1}{N} \sum_{i=1}^{N} \ell(f_\theta(y_i),\, x_i),$$

其中 f_θ 是參數為 θ 的復原網路，N 是訓練對數量，ℓ 是損失函數（L1、L2、感知、對抗……見損失函數一節）。訓練時使用隨機裁切的小塊（patch，例如 64×64 到 256×256）並搭配翻轉與旋轉，因為卷積與視窗注意力網路具有平移等變性，而全解析度影像也放不進 GPU 記憶體。

有一個事實可以解釋後面很多現象：用 L2 損失時，最佳網路輸出的是**後驗平均** $f^\star(y) = \mathbb{E}[x \mid y]$。當許多清晰影像都與 y 相容時，它們的平均是模糊的。這就是為什麼以 MSE 訓練的超解析度看起來很平滑，也是後來引入 GAN 與擴散模型的原因。

合成退化流程

配對資料有三種來源。(1) **合成**：用已知模型（雙三次降採樣、高斯雜訊、高斯模糊、JPEG）弄壞乾淨影像。便宜且無限量，但受限於模型有多真實。(2) **實拍**：同一個場景拍兩次，例如短曝光與長曝光，或把連拍平均成乾淨參考（如真實雜訊資料集 SIDD [22]），或把高速攝影的連續影格平均，得到真實的動態模糊（GoPro [44]）。真實但昂貴，且侷限於特定相機。(3) **非成對／自監督**：只用受損影像學習，靠 GAN、循環一致性或雜訊統計。今天大多數強模型都用豐富退化流程產生的合成配對訓練，再同時在合成與實拍資料上評估。

典型的「經典」流程是模糊 → 降採樣 → 雜訊 → JPEG，模仿相機與圖片分享網站依序破壞照片的順序：

```
import numpy as np, cv2, torch, torch.nn.functional as F
from skimage import data, img_as_float32

def gaussian_kernel(sigma, size=21):
    ax = torch.arange(size) - size // 2
    g = torch.exp(-ax**2 / (2 * sigma**2))
    k = torch.outer(g, g)
    return k / k.sum()

def degrade(x, scale=4, sigma_blur=1.6, sigma_noise=0.03, jpeg_q=40, rng=None):
    """x: float tensor (C,H,W) in [0,1]. Returns the low-quality image y = D(x) + n."""
    rng = rng or np.random.default_rng(0)
    C = x.shape[0]
    k = gaussian_kernel(sigma_blur).to(x)[None, None].repeat(C, 1, 1, 1)
    y = F.conv2d(F.pad(x[None], (10,) * 4, mode="reflect"), k, groups=C)   # 1) 模糊  h * x
    y = y[..., ::scale, ::scale]                                           # 2) 降採樣  (.)↓s
    y = y + sigma_noise * torch.from_numpy(rng.standard_normal(y.shape).astype(np.float32))
     ↪  # 3) 雜訊 n
    y = y.clamp(0, 1)[0]
    img = (y.permute(1, 2, 0).numpy() * 255).round().astype(np.uint8)      # 4) JPEG
     ↪  編碼再解碼
    ok, buf = cv2.imencode(".jpg", img[..., ::-1], [cv2.IMWRITE_JPEG_QUALITY, jpeg_q])
    img = cv2.imdecode(buf, cv2.IMREAD_COLOR)[..., ::-1]
    return torch.from_numpy(img.copy()).permute(2, 0, 1).float() / 255

x = torch.from_numpy(img_as_float32(data.astronaut())).permute(2, 0, 1)    # (3, 512, 512)
y = degrade(x)
print(x.shape, "->", y.shape)        # torch.Size([3, 512, 512]) -> torch.Size([3, 128, 128])
```

實際訓練時，每個參數（模糊核的寬度與形狀、雜訊強度與種類、JPEG 品質，甚至順序）都會**對每張影像隨機取樣**，讓網路看到的是一整個退化分布，而不是單一一種。

真實世界與盲退化

只用一種固定 $\mathcal{D}$（例如雙三次 $\times 4$）訓練的模型，遇到真實照片就會失效，因為真實退化是未知的、隨位置變化的，而且是多種效應的混合。這就是**盲**（blind，或稱真實世界）復原問題。大致有兩種策略：

- **把合成分布做得夠寬，寬到能涵蓋真實情況**。 Real-ESRGAN [4] 把經典流程**連續套用兩次**（「高階」退化，每一輪都隨機取樣模糊核、縮放、雜訊與 JPEG），並加入 sinc 濾波器來模擬振鈴與過衝。即使只用合成資料訓練，它在真實低品質影像上的泛化也比雙三次訓練的模型好得多。
- **引入強大的生成先驗**。預訓練的文字生成影像擴散模型已經知道自然影像長什麼樣。DiffBIR [16]、SUPIR [17] 與 OSEDiff [18] 都以受損輸入作為條件來驅動這類模型，讓先驗「補上」合理的細節。這是真實世界超解析度近年的主要趨勢，後面會再討論。

第三種互補的策略是**估計退化本身**（雜訊強度、模糊核、退化種類），再餵給網路，例如 FFDNet 的雜訊強度圖 [21] 與各種 all-in-one 模型。在盲超解析度中，未知的主要是 $y = (x * k)\downarrow_s + n$ 裡的模糊核 k，這條路線發展出自己的一系列方法：

- **以模糊核作為網路的條件**。 SRMD [87] 把退化當作額外輸入交給單一 SR 網路：模糊核先降成低維向量，連同雜訊強度一起展開成與低解析度影像同尺寸的圖，再與影像串接。這樣一個網路就能處理許多種模糊核與雜訊強度，前提是有人提供這些資訊。USRNet [91] 更進一步採用**深度展開**（deep unfolding）：把半二次分裂（half-quadratic splitting）的 MAP 求解器展開成網路，其中資料模組以封閉形式使用給定的模糊核、倍率與雜訊強度，先驗模組則是學習出的去雜訊器，所以一個模型就能處理任意已知的模糊核。
- **估計模糊核，再修正它**。 IKC [88] 觀察到錯誤的模糊核會造成特徵性的瑕疵（太大會過度銳化，太小會過度平滑），並利用這些瑕疵迭代修正初始的模糊核估計，同時由一個以模糊核為條件的 SR 網路（SFTMD）還原影像。DAN [90] 把這個交替過程展開成一個端到端網路：估計器（estimator）同時根據低解析度輸入與目前的 SR 結果預測模糊核，還原器（restorer）用目前的模糊核做超解析度，兩者輪流進行，和上面去模糊一節的傳統迴圈是同一個想法。
- **只從影像本身估計模糊核**。 KernelGAN [89] 在測試時只用那一張測試影像訓練一個小 GAN：生成器學著把影像縮小，讓判別器分不出縮小後影像的區塊與原圖的區塊。由於自然影像的區塊會跨尺度重複出現，學到的縮小運算就是這張影像自己的 SR 模糊核估計，再交給非盲 SR 方法使用。

明確估計模糊核的方法具可解釋性，而且在退化真的是「模糊、降採樣、雜訊」且模糊核大致均勻時很準確。若不是如此（JPEG、相機處理流程、隨位置變化的模糊），它們就很脆弱，而且模糊核的估計誤差會傳到結果裡。這促成了**隱式**的路線：BSRGAN [92] 用一個實用的退化模型訓練，模糊（等向與非等向高斯核）、降採樣、雜訊與 JPEG 等步驟以隨機順序組合；Real-ESRGAN [4] 用它的高階退化流程；兩者在測試時都不估計模糊核。擴散先驗方法 [16]–[18] 也屬於隱式。Liu 等人的綜述 [93] 依照如何建模退化、以及使用哪些資料（外部資料集或單張測試影像）來整理盲 SR 方法，並在合成與真實影像上比較它們。

超解析度（Super-Resolution, SR）

白話版。把小圖變大**而且**變清楚，也就是預測缺少的高解析度像素，而不只是把現有的像素拉伸開來。

問題設定

單張影像超解析度（SISR）在 $y = (x * k)\downarrow_s + n$ 之下，從 $y \in \mathbb{R}^{H\times W\times C}$ 估計 $x \in \mathbb{R}^{sH\times sW\times C}$。傳統做法是**插值法**（interpolation）：最近鄰（nearest neighbor）、雙線性（bilinear）與雙三次（bicubic）。其中雙三次插值以 4×4 鄰域擬合三次多項式，最為知名，至今仍被廣泛使用，既是基準方法，也是產生低解析度測試輸入的標準方式。插值只會重新分配既有資訊，無法重建被 $\downarrow_s$ 丟掉的高頻。

網路怎麼「變大」：三種上採樣策略

輸出比輸入大，所以每個 SR 網路都得決定解析度**在哪裡**提升。

a. 先上採樣（先插值）。 SRCNN [6] 先用雙三次插值把輸入放大到目標尺寸，再接三層卷積（區塊擷取、非線性映射、重建）。簡單直觀，但每一層都在高解析度上運算，速度慢，網路還得清理插值造成的瑕疵。

b. 轉置卷積（反卷積，deconvolution）。 FSRCNN [7] 在低解析度上運算，只在最後用學習得到的轉置卷積放大，速度比 SRCNN 快很多。步長無法整除核大小的轉置卷積會產生不均勻的重疊，容易出現**棋盤狀瑕疵**（checkerboard artifact）[9]，所以現在很少被當作 SR 的最終上採樣器；不過在其他類型的神經網路中（例如分割與生成模型的解碼器）仍然相當常見。

c. 子像素卷積（pixel shuffle）。 ESPCN [8] 在低解析度上計算 Cs^2 個通道，再重新排列成 s 倍大的影像：

$$\mathrm{PS}(T)_{c,\,sh+i,\,sw+j} = T_{cs^2+si+j,\,h,\,w}, \qquad 0 \le i,j < s,$$

其中 T 是有 Cs^2 個通道的低解析度特徵張量，(h,w) 是低解析度位置，(i,j) 是子像素偏移，c 是輸出通道。所有學習都在低解析度進行，重新排列本身沒有參數。這是現代 SR 網路的主流選擇（`torch.nn.PixelShuffle`）。

里程碑

- **SRCNN**（2014/2016）[6]：第一個端到端的 SR CNN，在雙三次放大的輸入上接三層卷積。
- **FSRCNN**（2016）[7] 與 **ESPCN**（2016）[8]：後上採樣，達到即時速度。
- **EDSR**（2017）[10]：拿掉批次正規化（BN 會削弱復原任務所需的數值範圍彈性）的深層殘差網路，用殘差縮放穩定訓練，並有多尺度版本。它贏得 NTIRE 2017 SR 挑戰賽。
- **RCAN**（2018）[11]：非常深的「殘差中的殘差」結構，加上**通道注意力**（channel attention），用全域統計量重新加權特徵通道，讓網路專注於高頻資訊。
- **SRGAN**（2017）[12] 與 **ESRGAN**（2018）[13]：對抗損失與感知（VGG 特徵）損失產生銳利、逼真的紋理，而不是 MSE 的模糊平均。ESRGAN 改良了生成器（不含 BN 的 Residual-in-Residual Dense Block）、使用相對判別器（relativistic discriminator），並在激活前的特徵上計算感知損失。

大約在 2018 年，這些模型的結果在人眼看來已經遠勝插值，領域也開始質疑該怎麼**評判** SR，評判標準本身的界定引起更多疑慮（見感知–失真一節）。

Transformer、真實世界 SR 與擴散模型（目前的主流家族）

- **Transformer**。 SwinIR [14]（2021）以殘差 Swin Transformer 區塊（在位移的局部視窗內做自注意力）處理 SR、去雜訊與 JPEG 去壓縮瑕疵。HAT [15]（CVPR 2023）結合通道注意力、視窗自注意力與重疊交叉注意力模組，讓每個輸出「啟動更多像素」。兩者至今仍是常見的比較基準。
- **狀態空間模型**。 MambaIR [49] 與 MambaIRv2 [50] 以選擇性狀態空間（Mamba）掃描取代注意力，以線性成本取得全域感受野；見骨幹網路一節。
- **真實世界（盲）SR**。前面介紹過的 Real-ESRGAN [4] 與其高階退化流程。
- **擴散先驗**。 DiffBIR [16] 先去除退化，再用預訓練的潛在擴散模型重新生成細節；SUPIR [17] 把這個做法放大，使用大型擴散骨幹、兩千萬張附文字描述的訓練影像，以及負向品質提示（negative-quality prompt）；OSEDiff [18] 把過程蒸餾成**單一步**擴散，直接從低品質影像出發，並以變分分數蒸餾（variational score distillation）維持輸出的真實感。這些方法在真實照片上能產生更有說服力的紋理，但也可能捏造出場景中不存在的細節。

資料集與指標

標準訓練集是 **DIV2K** [19]（為 NTIRE 2017 推出的 2K 解析度影像）。評估使用幾個經典的小型測試集（自然、都市與漫畫影像），在亮度（Y）通道上報告 PSNR 與 SSIM，通常會先裁掉 s 個邊界像素。真實世界 SR 則以感知與無參考指標評估（LPIPS、NIQE 等，見評估指標一節）。

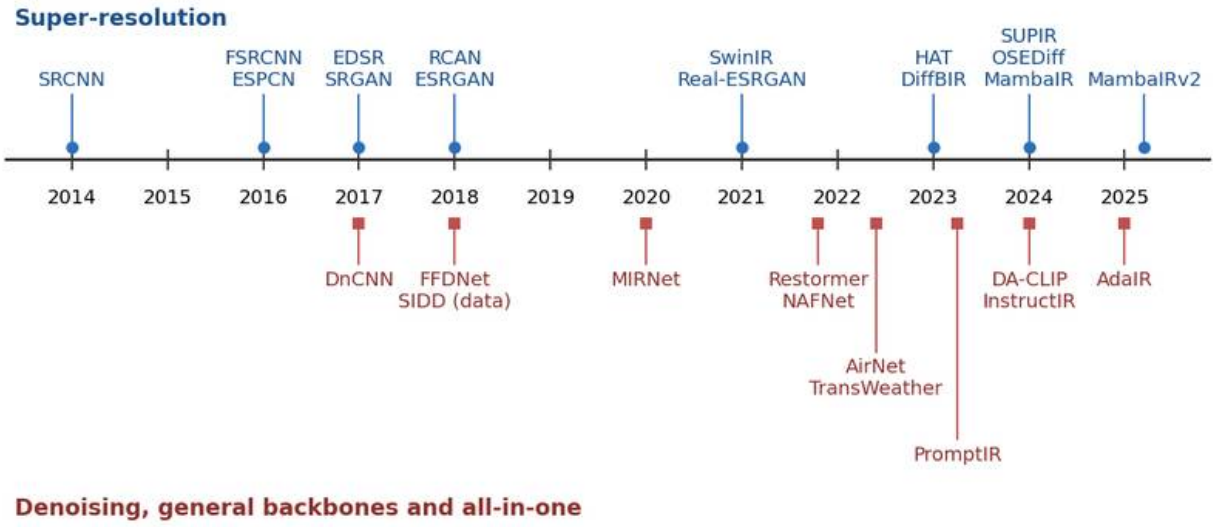


圖 3 — 超解析度（上）以及去雜訊、通用骨幹與 *all-in-one* 復原（下）的里程碑，依發表年份排列（同一年內的位置只是為了方便閱讀）。

去雜訊（Denoising）

白話版。把照片上隨機的顆粒去掉，同時不要把真正的紋理也抹平。雜訊是最單純的退化，所以去雜訊成了大多數深度復原想法最先出現的試驗場。

問題設定

$y = x + n$。合成設定中 $n \sim \mathcal{N}(0, \sigma^2 I)$（加性白高斯雜訊，AWGN，標準差 σ 通常以 0–255 的尺度表示，例如 $\sigma = 25$）。真實相機的雜訊不一樣：它和訊號相關（光子散粒雜訊隨亮度增加），去馬賽克後在空間上具相關性，還會被相機的影像處理流程改變形狀。

里程碑

DnCNN [20] 做了兩個後來成為標準的選擇。第一是**殘差學習**（residual learning）：網路 $\mathcal{R}_\theta$ 不直接預測 x，而是預測雜訊，輸出為

$$\hat{x} = y - \mathcal{R}_\theta(y),$$

其中 $\mathcal{R}_\theta(y) \approx n$。殘差（雜訊）比乾淨影像「更容易學」，使得搭配批次正規化的深層純 CNN 能訓練得很好。單一 DnCNN 也能針對一段範圍的雜訊強度訓練（盲高斯去雜訊），而且同一個網路也能處理 SR 與 JPEG 去區塊。

FFDNet [21] 把**雜訊強度圖** M（與影像同尺寸，每個像素的值為 σ）和降採樣後的輸入一起餵給網路。測試時調整 M，可以在去雜訊與保留細節之間取捨；非均勻的 M 則能處理隨位置變化的雜訊。它的速度快到可以在 CPU 上執行。

真實雜訊。 **SIDD** 資料集 [22] 提供真實的智慧型手機雜訊影像，乾淨參考影像則透過仔細拍攝連拍並加以處理而得。只用 AWGN 訓練的模型在它上面表現很差，這推動了整個領域走向真實的雜訊合成與實拍訓練資料。

通用骨幹。 MIRNet [30]、**Restormer** [23] 與 **NAFNet** [24] 是通用的復原架構，在 SIDD 以及去模糊等任務上都報告了最先進的結果，會在骨幹網路一節介紹。

程式碼：一個小小的殘差去雜訊網路

下面的程式在四張 `skimage` 影像隨機切出的 32×32 小塊上訓練一個五層、DnCNN 風格的網路，每一步都加上新的高斯雜訊，然後在一張從沒看過的影像上測試。在筆電 CPU 上大約一分鐘內就能跑完（我們在負載很重的機器上量到約一分鐘）。

```
import time, numpy as np, torch, torch.nn as nn
from skimage import data, img_as_float32
from skimage.color import rgb2gray

torch.manual_seed(0); torch.set_num_threads(2)
rng = np.random.default_rng(0)
sigma = 25 / 255                                    # 雜訊強度（[0,1] 尺度）
```

```
train_imgs = [img_as_float32(im) for im in (data.camera(), data.coins(), data.moon(),
 ↪  data.page())]
test_img = img_as_float32(rgb2gray(data.astronaut()))   # 測試用，不參與訓練

def random_patches(n=32, p=32):
    batch = []
    for _ in range(n):
        im = train_imgs[rng.integers(len(train_imgs))]
        i, j = rng.integers(0, im.shape[0] - p), rng.integers(0, im.shape[1] - p)
        batch.append(im[i:i + p, j:j + p])
    return torch.from_numpy(np.stack(batch))[:, None]   # (n, 1, p, p)

class TinyDnCNN(nn.Module):
    """conv-ReLU, (depth-2) x conv-BN-ReLU, conv。預測的是雜訊，不是影像。"""
    def __init__(self, depth=5, ch=32):
        super().__init__()
        layers = [nn.Conv2d(1, ch, 3, padding=1), nn.ReLU(inplace=True)]
        for _ in range(depth - 2):
            layers += [nn.Conv2d(ch, ch, 3, padding=1), nn.BatchNorm2d(ch),
             ↪  nn.ReLU(inplace=True)]
        layers += [nn.Conv2d(ch, 1, 3, padding=1)]
        self.body = nn.Sequential(*layers)
    def forward(self, y):
        return y - self.body(y)                         # x_hat = y - R(y)：殘差學習

net = TinyDnCNN()
opt = torch.optim.Adam(net.parameters(), lr=2e-3)
t0 = time.time()
for step in range(300):
    x = random_patches()
    y = x + sigma * torch.randn_like(x)                 # 即時產生合成配對
    loss = nn.functional.l1_loss(net(y), x)
    opt.zero_grad(); loss.backward(); opt.step()
print(f"trained in {time.time() - t0:.0f} s, last L1 = {loss.item():.4f}")

net.eval()
with torch.no_grad():
    x = torch.from_numpy(test_img)[None, None]
    y = x + sigma * torch.randn_like(x)
    x_hat = net(y).clamp(0, 1)
mse = lambda a, b: torch.mean((a - b) ** 2).item()
print(f"PSNR noisy {10*np.log10(1/mse(y.clamp(0,1), x)):.2f} dB -> denoised
 ↪  {10*np.log10(1/mse(x_hat, x)):.2f} dB")
# 例如 PSNR noisy 20.74 dB -> denoised 28.64 dB
```

即使是這個只用四張圖、訓練 300 個小步驟的玩具模型，在沒看過的照片上也能提升約 8 dB，而且勝過手動調整的高斯濾波（圖 4）。拖動滑桿，比較雜訊輸入與小網路的輸出。

雜訊（$\sigma = 25$）

小型殘差 *CNN*

滑桿比較：顆粒明顯的灰階太空人臉部，與小型 CNN 去雜訊後乾淨但略顯平滑的同一區塊。

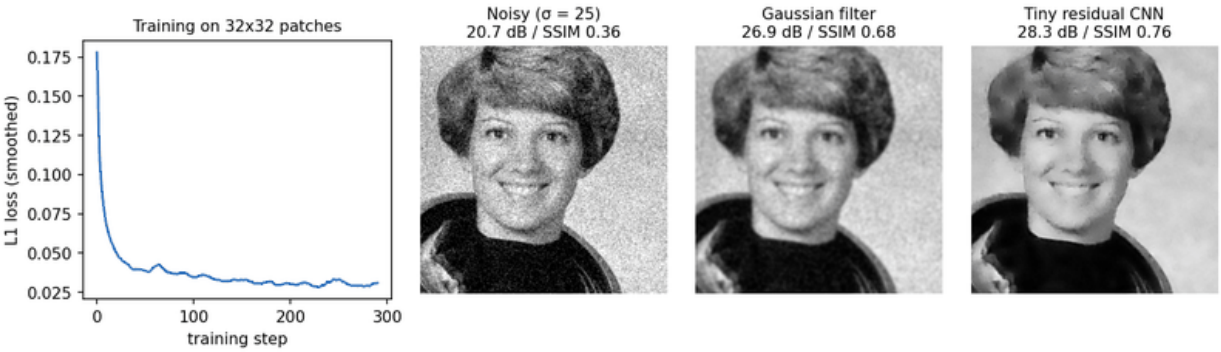


圖 *4* — 小型殘差去雜訊網路。*PSNR* 與 *SSIM* 是在完整的 *512* × *512* 測試影像上計算。*CNN* 去掉的雜訊比高斯濾波多，邊緣也更銳利，但同時把細緻紋理壓平了，這是 *MSE* ／ *L1* 訓練的典型行為。不同次執行、不同機器的數值會略有差異。

除霧（Dehazing）

白話版。去掉霧或霾造成的灰白色面紗，找回對比與顏色，遠處物體尤其明顯。用原文的說法，它是去雜訊與去模糊的一種延伸，只是多了一個物理模型。

傳統做法與暗通道先驗

傳統方法先估計 $I = Jt + A(1-t)$ 中的未知量（透射率圖 t 與大氣光 A），再代回公式求解。最困難的部分，就是想出能把這些未知量確定下來的可靠關係。最知名的是何凱明等人的**暗通道先驗**（dark channel prior）[25]：在無霧的戶外影像中，大多數局部區塊裡至少有一個像素在某個顏色通道上非常暗（陰影、深色或鮮豔的物體）：

$$J^{\text{dark}}(p) = \min_{c\in\{r,g,b\}} \Big(\min_{q\in\Omega(p)} J^c(q) \Big) \approx 0,$$

其中 $\Omega(p)$ 是以 p 為中心的小區塊，J^c 是顏色通道 c。對霧的模型做同樣的雙重取最小值，再利用 $J^{\text{dark}} \approx 0$，就得到透射率估計

$$\tilde{t}(p) = 1 - \omega \min_c \Big(\min_{q\in\Omega(p)} \frac{I^c(q)}{A^c} \Big),$$

其中 $\omega < 1$（論文中取 0.95）刻意保留一點霧，以維持深度感。接著 $\hat{J} = (I-A)/\max(\tilde{t}, t_0) + A$，小小的 t_0 用來避免除以零。這個先驗在大片與大氣光相近的明亮區域（白牆、天空）會失效，而以區塊為單位得到的粗糙 t 也必須再精修，才能貼合邊緣。

深度除霧：兩個流派

原文 [1] 把深度除霧分成兩派，這個分法至今仍很貼切。

1. 物理引導（「套公式」）。學出未知參數，再反轉模型。**DehazeNet** [26] 用 CNN（搭配 Maxout 單元與有界 ReLU）從有霧區塊迴歸出透射率 t。**AOD-Net** [27] 進一步簡化大氣散射模型，把 t 與 A 合併成單一函數 K：

$$J(p) = K(p)\,I(p) - K(p) + b, \qquad K(p) = \frac{\frac{1}{t(p)}(I(p) - A) + (A - b)}{I(p) - 1},$$

其中 b 是常數偏移。一個輕量 CNN 預測 K，於是整個網路是端到端的，還可以接在偵測器前面一起訓練。

2. 強大的通用架構（「超級模型」）。跳過物理，直接用強大的網路學 $I \mapsto J$。**GridDehazeNet** [28] 使用以注意力為基礎的多尺度網格；**FFA-Net** [29] 堆疊通道注意力與像素注意力區塊，並做自適應特徵融合。這類設計的學習能力很強，換句話說，很有機會也適用於其他退化任務，這正是 MIRNet [30] 等通用復原網路走的路。之後出現了 Transformer 除霧模型：**DehazeFormer** [31] 修改正規化層、激活函數與空間聚合方式，把 Swin Transformer 調整成適合除霧。

資料集

- **RESIDE** [32]：大型基準，包含以深度與散射模型合成的室內外有霧影像，以及用於任務導向評估的真實有霧影像。它的 SOTS 測試集是最常被引用的除霧基準。
- **O-HAZE** [33] 與 **Dense-Haze** [34]：以專業造霧機拍攝的真實有霧／無霧配對，分別是戶外場景與濃霧場景。規模小，但能突顯合成與真實之間的落差。
- **KITTI** [35] **不是**除霧資料集。它是在 Karlsruhe 一帶錄製的自駕車情境視覺基準（立體視覺、光流、里程計、3D 物件偵測、追蹤）。部分除霧研究會在它的影像上合成霧氣（它有光達與立體視覺提供的深度），或用它評估除霧對下游偵測的影響。

除雪（Desnowing）

白話版。去掉飄落的雪。看起來像除雨的親戚，但行為上也很像除霧。

除雪比看起來困難。雪花的大小與透明度差異極大，從不透明的白色團塊到半透明的條紋都有，而大雪還會造成類似霧的**朦朧效應**（veiling effect）。所以除雪模型不只要處理呈現為白點的「雪」，也得處理朦朧的霧氣。**DesnowNet** [36] 推出 **Snow100K** 資料集（在真實影像上合成雪，另附真實雪景影像），以及一個同時估計雪的半透明程度與其殘留的網路。**JSTASR** [37] 明確建模雪的大小與透明度，並加入去除朦朧效應的步驟。現在除雪通常被放進 TransWeather [52] 這類「惡劣天氣」all-in-one 模型中一起研究。

除雨（Deraining）

白話版。把照片裡的雨去掉。「雨」其實指兩種很不一樣的東西：在空中落下的細雨線，以及停在鏡頭或擋風玻璃上的雨滴。

雨線與雨滴

- **雨線**（rain streak）是疊加在場景上、細而亮、有方向性的線條，模型為 $I = B + R$。遠處的雨線累積起來會形成類似霧的面紗。方法的目標是把兩層分開。一個早期具影響力的深度方法同時偵測並去除雨 [38]，並提供了被廣泛使用的輕雨與大雨合成資料集。
- **雨滴**（raindrop）附著在玻璃表面，像一個個小透鏡：遮住並折射背景、常常失焦，而且沒有一致的方向。Qian 等人的注意力 GAN [39] 用注意力圖（雨滴在哪裡？）同時引導生成器與判別器，並推出一個透過玻璃拍攝、有無水滴成對的雨滴資料集。

走向真實場景

大多數除雨資料集是合成的，用合成雨線訓練的模型面對真實的雨常常失效。**NTIRE 2025 Day and Night Raindrop Removal for Dual-Focused Images** 挑戰賽 [40] 反映了研究重心轉向真實資料：它使用真實拍攝的 **Raindrop Clarity** 資料集，涵蓋白天與夜晚，以及對焦在雨滴與對焦在背景兩種情況（訓練 14,139 張、驗證 240 張、測試 731 張）。對焦設定很關鍵：相機對焦在雨滴上時，背景也是模糊的，於是去除雨滴有一部分變成了去模糊。

低光增強（簡介）

白話版。讓暗照片看起來曝光正常、細節可見、色調自然，同時不要把雜訊放大。

訓練時對比暗影像與同一場景正常曝光的參考影像，例如隨 **RetinexNet** [41] 推出、包含低光／正常光配對的 **LOL** 資料集。傳統方法以對比度調整（直方圖等化）與 Retinex 模型為主（較少見的還有反轉大氣散射模型）。深度方法有兩條路線：受 Retinex 啟發、分解 $S = R \cdot L$ 再調整 L 的網路（RetinexNet [41]），以及特別設計的通用網路。**Retinexformer** [42]（ICCV 2023）結合兩者：單階段的 Retinex 公式加上光照引導的注意力，作者稱它是第一個以 Transformer 為基礎的低光增強方法。低光增強本身的內容（零參考方法、曲線估計、雜訊、色彩、影片）多到值得獨立一章：見DLCV 05。

影像融合（簡介）

白話版。把同一場景、各有所長的幾張影像組合成一張，讓結果同時擁有每一張的優點，資訊更豐富多樣。

原文 [1] 提到三種常見設定：**紅外線與可見光**融合（紅外線提供熱目標，可見光相機提供紋理與色彩）、**多重曝光**融合（結合曝光不足與過度照片的細節，得到類似 HDR 的結果），以及**多焦點**融合（由對焦在不同深度的照片合成全景深影像）。與其他任務不同，融合通常沒有真實的「正確

融合影像」，所以深度融合依賴保留來源影像梯度與強度的非監督式損失，或使用 GAN。Zhang 等人的綜述 [43] 依這些情境與學習策略整理了深度融合方法。

去模糊（Deblurring）

白話版。從被手震、物體移動或失焦弄糊的影像中，還原出清晰的影像。

問題設定

去模糊就是 $g = h * f + n$ 的直接應用。模糊核 h 已知時稱為**非盲**（non-blind）去模糊（DIP 第 5 章中 Wiener 與 CLS 濾波的設定，以及學習式非盲求解器）；h 未知時稱為**盲**（blind）去模糊。動態場景中的真實動態模糊更麻煩：不同物體的運動不同、深度也不同，所以根本沒有單一的全域 h。

盲去模糊：估計模糊核

白話版。如果你確切知道相機是怎麼晃的，把模糊還原回去就是一個研究得很透徹的濾波問題。盲去模糊的麻煩在於你不知道相機怎麼晃，所以必須同時猜「怎麼晃」和「清晰的照片」。訣竅是輪流猜：先用目前猜的晃動把影像變清晰，再用得到的最清楚的邊緣去改進對晃動的猜測，然後重複。

聯合問題。對空間上均勻的模糊，盲反卷積（blind deconvolution）要解

$$(\hat{f}, \hat{h}) = \arg\min_{f,h} \|h * f - g\|_2^2 + \lambda_f\, \phi(f) + \lambda_h\, \psi(h) \quad \text{subject to} \quad h \geq 0,\ \ \textstyle\sum_{u,v} h(u,v) = 1,$$

其中 g 是模糊影像，f 是未知的清晰影像，h 是未知的模糊核（只在一個小視窗內非零，手震通常是 15×15 到 75×75 像素），ϕ 是影像先驗（通常是稀疏梯度懲罰 $\sum_p |\nabla f(p)|^\alpha$，$0 < \alpha \leq 1$，因為自然影像大多是平坦區域加上少數強邊緣），$\psi$ 是模糊核先驗（例如 $\|h\|_2^2$ 或 $\|h\|_1$，偏好緊湊、稀疏的運動軌跡），$\lambda_f, \lambda_h > 0$ 是它們的權重。限制條件表示模糊核只是把光重新分配：它非負，而且保持總亮度不變。這也消除了一種不確定性，因為對任何 $c > 0$，$(c\,h,\ f/c)$ 和 (h, f) 一樣能解釋 g。另一種不確定性則無法消除：把 f 往一個方向平移、h 往反方向平移，$h * f$ 不變，所以模糊核只能還原到差一個平移的程度。

為什麼這麼病態。未知數（f 和 h）遠多於方程式，很多組解都能完全解釋 g。更糟的是，最明顯的那一組是錯的：$h = \delta$（沒有模糊）且 $f = g$ 完美符合資料。Levin 等人 [74] 指出，在常用的稀疏梯度先驗下，模糊影像的先驗代價往往比清晰影像更低，因為模糊會讓梯度變小，而 ϕ 偏好小梯度。所以對 (f, h) 做天真的聯合 MAP 估計，常會得到「沒有模糊」的解。他們的分析指向一個較適定的目標：只估計模糊核，把影像積分掉，

$$\hat{h} = \arg\max_h\ p(h \mid g) = \arg\max_h\ p(h) \int p(g \mid f, h)\, p(f)\, df,$$

其中 $p(g \mid f, h)$ 是由資料項給出的高斯似然，$p(f)$ 和 $p(h)$ 是 ϕ 與 ψ 背後的先驗，積分則是對所有清晰影像取平均。模糊核的未知數比像素觀測少得多，所以它的後驗遠比整張影像的後驗確定。一旦知道 $\hat{h}$，再用非盲反卷積就能還原 f。

傳統的解法。實務上有效的方法，不是近似上述的邊際化，就是改變先驗，讓清晰的解勝出：

- **變分貝氏（variational Bayes）。** Fergus 等人 [75] 用重尾的混合高斯模型描述影像梯度，以變分貝氏近似模糊核的後驗，並由粗到細估計模糊核；影像則再用標準的非盲方法還原。
- **邊緣預測與挑選。** Shan 等人 [76] 以自然影像梯度先驗和仔細設計的雜訊模型輪流估計模糊核與影像，以抑制振鈴。Cho 與 Lee [77] 讓這個交替過程變快：他們用雙邊濾波與衝擊濾波（shock filter）從目前的潛在影像預測清晰邊緣，並以 FFT 在梯度域估計模糊核。Xu 與 Jia [78] 發現比模糊核還細的邊緣會誤導估計，所以第一階段只挑選有資訊量的邊緣，第二階段再精修模糊核的支撐範圍。
- **偏好清晰影像的先驗。** Krishnan 等人 [79] 把 $\|\nabla f\|_1$ 換成正規化稀疏度 $\|\nabla f\|_1 / \|\nabla f\|_2$，它與尺度無關，而且會隨模糊增加，所以不再偏好模糊的解。Pan 等人 [80] 使用暗通道（與 He 等人除霧先驗 [25] 相同的量）：模糊會把暗像素和周圍的亮像素平均在一起，使暗通道變得不那麼稀疏，因此對暗通道加上 L_0 懲罰就會偏好清晰影像。

標準流程。幾乎所有傳統盲去模糊方法都遵循同一個迴圈。在影像金字塔最粗的一層，從 delta 模糊核開始，交替進行

$$f^{(t+1)} = \arg\min_{f} \|h^{(t)} * f - g\|_2^2 + \lambda_f \, \phi(f),$$

$$h^{(t+1)} = \arg\min_{h \geq 0,\ \sum h = 1} \|h * \nabla \tilde{f}^{(t+1)} - \nabla g\|_2^2 + \lambda_h \|h\|_2^2,$$

其中 t 是迭代次數，∇ 是影像梯度，$\tilde{f}$ 是經過邊緣預測或挑選後的潛在影像（銳化過，弱梯度設為零）。模糊核這一步在梯度域進行，因為卷積與微分可以交換，$\nabla(h * f) = h * \nabla f$，而模糊的資訊主要藏在梯度裡；搭配二次懲罰時，它在頻域有封閉解，之後再投影回限制條件。迭代幾次後，把模糊核放大到金字塔的下一層（**由粗到細**，coarse-to-fine），因為大模糊在粗尺度上只有幾個像素寬，這樣可以避開不好的局部極小值。最後固定 $\hat{h}$，用**非盲反卷積**產生輸出影像：可以是IP 05 的 Wiener 或限制最小平方濾波，也可以是超拉普拉斯（hyper-Laplacian）先驗或學習式的非盲求解器。

深度學習與模糊核。神經網路以三種方式進入盲去模糊。

- **先預測模糊核（或濾波器），再反卷積。** Sun 等人 [81] 訓練 CNN，把每個區塊的局部模糊核分類到一組線性運動之一，用馬可夫隨機場（MRF）平滑得到的非均勻運動場，再做反卷積。Schuler 等人 [82] 建構一個深層網路，各階段模仿傳統迴圈（特徵擷取、模糊核估計、影像估計），並在合成模糊上端到端訓練。Chakrabarti [83] 訓練網路為每個區塊預測反卷積濾波器的傅立葉係數，用結果估計一個全域模糊核，最後以標準非盲反卷積收尾。
- **自監督、逐張影像估計。** Deep Image Prior [84] 指出，未經訓練的生成網路，其結構本身就是有用的影像先驗。SelfDeblur [85] 用兩個生成網路：一個編碼器–解碼器負責影像，一個輸出接 softmax 的全連接網路負責模糊核（softmax 自動滿足 $h \geq 0$ 與 $\sum h = 1$），兩者一起擬合單張模糊影像。不需要訓練資料，但每張影像都要跑一次最佳化。
- **不估計模糊核、端到端。** Nah 等人 [44] 的多尺度網路、DeblurGAN [86]（條件式 GAN 加上內容損失），以及通用骨幹 Restormer 與 NAFNet，都直接把模糊影像映射成清晰影像，從頭到尾不產生 h。

哪一種比較好？以模糊核為基礎的方法適合**空間均勻**的模糊（拍攝遠處靜態場景時的手震），提供**可解釋**的中間結果，可以檢查也可以重複使用（例如用來處理同一次曝光的其他影格），而且只要物理模型成立，就能泛化到新的模糊核；但它們較慢，遇到隨位置變化的模糊或邊緣很少的影像就會失效。不估計模糊核的網路能處理物體運動與深度造成的**非均勻**模糊，測試時很快，也是

GoPro 等動態場景基準上目前最好的方法，但它們只能泛化到訓練時看過的模糊分布，也不給出明確的模糊核。Zhang 等人的綜述 [69] 整理了這個轉變。

程式碼：一個交替式盲反卷積的玩具範例。下面的程式用一個彎曲的 15 × 15 手震模糊核把 256 × 256 的 `camera` 影像弄糊，然後交替進行封閉形式的影像步驟（帶二次梯度懲罰、類似 CLS 的濾波器）與模糊核步驟（只用經衝擊濾波後最強的 2% 梯度做最小平方，再投影到 $h \ge 0$、$\sum h = 1$）。在 CPU 上約一秒跑完。

```
import numpy as np
from numpy.fft import fft2, ifft2
from scipy.ndimage import convolve, gaussian_filter, laplace
from skimage import data, img_as_float, transform

rng = np.random.default_rng(0)
f_true = transform.resize(img_as_float(data.camera()), (256, 256), anti_aliasing=True)
K = 15                                              # 模糊核大小 K x K
h_true = np.zeros((K, K))
for t in np.linspace(0, 1, 40):                     # 一條彎曲的「手震」軌跡
    h_true[round(3 + 8 * t), round(2 + 10 * t + 2 * np.sin(3 * t))] += 1
h_true = gaussian_filter(h_true, 0.5); h_true /= h_true.sum()
g = convolve(f_true, h_true, mode="wrap") + 0.005 * rng.standard_normal(f_true.shape)

def otf(h, shape):                                  # 置中的模糊核 -> 轉移函數
    p = np.zeros(shape); p[:h.shape[0], :h.shape[1]] = h
    return fft2(np.roll(p, (-(h.shape[0] // 2), -(h.shape[1] // 2)), (0, 1)))

G = fft2(g)
Dx, Dy = otf(np.array([[0, -1, 1]]), g.shape), otf(np.array([[0], [-1], [1]]), g.shape)

def image_step(h, lam):        # f = argmin ||h*f - g||^2 + lam ||grad f||^2（封閉解，類似
↪   CLS）
    H = otf(h, g.shape)
    return np.real(ifft2(np.conj(H) * G / (np.abs(H)**2 + lam * (np.abs(Dx)**2 +
     ↪  np.abs(Dy)**2))))

def shock(f, n=5, dt=0.5):     # 邊緣預測：衝擊濾波把柔和的邊緣變成階梯
    f = gaussian_filter(f, 0.7)
    for _ in range(n):
        gy, gx = np.gradient(f)
        f = f - dt * np.sign(laplace(f)) * np.hypot(gx, gy)
    return f

def kernel_step(f, keep=0.02, beta=1e-3):   # 梯度域最小平方，只用強邊緣
    num = den = 0
    for D in (Dx, Dy):
        fx = np.real(ifft2(D * fft2(f)))
        fx[np.abs(fx) < np.quantile(np.abs(fx), 1 - keep)] = 0
        Fx = fft2(fx)
        num, den = num + np.conj(Fx) * D * G, den + np.abs(Fx)**2
    h = np.roll(np.real(ifft2(num / (den + beta))), (K // 2, K // 2), (0, 1))[:K, :K]
    h = np.maximum(h, 0)                            # 投影到 h >= 0 ……
    return h / h.sum()                              # ……以及 sum(h) = 1
```

```
def best_shift(fn):            # 盲反卷積只能還原到差一個小平移的 h（與 f）
    return min(fn(dy, dx) for dy in range(-3, 4) for dx in range(-3, 4))
kernel_error = lambda h: best_shift(lambda dy, dx: np.linalg.norm(np.roll(h, (dy, dx), (0,
 ↪  1)) - h_true)) / np.linalg.norm(h_true)
psnr = lambda f: -best_shift(lambda dy, dx: 10 * np.log10(np.mean((np.clip(np.roll(f, (dy,
 ↪  dx), (0, 1)), 0, 1) - f_true)[8:-8, 8:-8] ** 2)))

h = np.zeros((K, K)); h[K // 2, K // 2] = 1
h = gaussian_filter(h, 1.0); h /= h.sum()          # 從接近 delta 的模糊核（「沒有模糊」）開始
print(f"kernel error at start: {kernel_error(h):.2f}")
for _ in range(30):                                # 交替：先更新影像，再更新模糊核
    h = kernel_step(shock(image_step(h, lam=0.02)))
print(f"kernel error after 30 alternations: {kernel_error(h):.2f}")
print(f"PSNR  blurry {psnr(g):.2f} dB | estimated kernel {psnr(image_step(h, 1e-2)):.2f} dB"
      f" | true kernel {psnr(image_step(h_true, 1e-2)):.2f} dB")
# kernel error at start: 1.19
# kernel error after 30 alternations: 0.57
# PSNR  blurry 22.59 dB | estimated kernel 27.11 dB | true kernel 29.67 dB
```

模糊核的相對誤差大約減半，用估計的模糊核做最後的非盲步驟，也補回了模糊輸入與「真實模糊核」結果之間大部分的差距。但要誠實看待這個玩具範例：模糊是合成的、均勻的、循環的（所以 FFT 模型完全成立），雜訊很低，參數（`lam`、`keep`、衝擊濾波的次數）也是針對這張影像調的。程式沒有影像金字塔，所以更大的模糊核會失敗；面對真實照片時，還需要處理邊界、飽和像素，並使用穩健的損失。兩種誤差都是在最佳的小平移之後才量測，原因就是上面說的平移不確定性。

資料與里程碑

Nah 等人 [44]（CVPR 2017）用 **GoPro** 資料集處理這個問題：他們以高速攝影機拍攝清晰的影片，把連續影格平均起來，得到真實的模糊影像，並以中間那一格作為清晰目標。他們的網路是**多尺度**的：先還原粗略版本，再逐步在更細的尺度上精修，類似傳統盲去模糊由粗到細的策略，但完全不需要估計模糊核。GoPro 至今仍是最常用的去模糊基準。

今天最常被比較的基準是通用骨幹 **Restormer** [23] 與 **NAFNet** [24]。NAFNet 有一個驚人的發現：常用的非線性激活函數（ReLU、GELU、Sigmoid）其實不是必要的，可以用兩半特徵的逐元素相乘（「SimpleGate」）取代，或直接移除，得到一個「無非線性激活」（nonlinear activation free）的網路，不但更簡單，在 GoPro 與 SIDD 上也更強。Zhang 等人的去模糊綜述 [69] 涵蓋了更廣的範圍（盲與非盲、影片、失焦、損失函數）。

影像修補（Inpainting，簡介）

白話版。把缺失或破損的區域（刮痕、被移除的物體、破洞）補上合理的內容。

修補是最接近影像**生成**的復原任務：在大破洞裡沒有東西可以「還原」，只能創造出和周圍相符的內容。它的退化是一個遮罩，$y = m \odot x$，其中 m 在破洞處為 0、其他地方為 1。前陣子相當知名的**遮罩自編碼器**（masked autoencoder, MAE）[45] 也可以歸類於此，可視為把修補當作預訓練

任務：遮住 75% 的影像區塊，訓練 ViT 把它們重建回來，結果學到了很強的視覺表徵。實務上的修補方面，**LaMa** [46] 使用具備全影像感受野的快速傅立葉卷積，並以大型遮罩訓練，能在高解析度下填補大破洞；如今，以擴散模型為基礎的修補是大範圍、語意層級編輯的預設做法。

去背（Matting，簡介）

白話版。把前景（一個人、他的頭髮）連同柔和、半透明的邊緣一起摳出來，好貼到新背景上。

去背假設每個像素是前景與背景的混合：

$$I = \alpha F + (1 - \alpha)B,$$

其中 $\alpha \in [0, 1]$ 是不透明度（alpha matte），F 是前景顏色，B 是背景顏色。每個像素有七個未知數（α 與兩組 RGB 顏色），卻只有三個觀測值，嚴重不足，所以傳統方法要求使用者提供三**色圖**（trimap：確定前景、確定背景、不確定區域）。它也可以視為分割的一種軟性變形。**Background Matting** [47] 用一張不含主體的背景照片取代三色圖，先在合成影像上訓練網路，再以對抗損失（判斷新合成圖看起來是否自然）在真實影像上訓練。

通用骨幹網路

白話版。在上述所有任務中，總是那幾種網路設計反覆勝出。它們是通用的「影像到影像引擎」，可以拿來訓練上面任何一種退化。

骨幹	核心想法	為什麼適合復原
U-Net [48]	編碼器–解碼器，每個尺度都有跳躍連接	降採樣路徑提供大感受野，跳躍連接保留細節
SwinIR [14]	在全解析度上堆疊位移視窗自注意力的殘差群組	權重隨內容自適應；SR、去雜訊、JPEG 表現強
Restormer [23]	由 Transformer 區塊組成的 U-Net；注意力在**通道**維度計算（轉置注意力），搭配門控前饋網路	成本與像素數呈線性，可擴展到大影像
NAFNet [24]	由簡單卷積區塊組成的 U-Net，使用 LayerNorm、SimpleGate 與簡化通道注意力；沒有非線性激活	每單位 FLOP 的準確度極高；預設的「簡單基準」
MambaIR / MambaIRv2 [49], [50]	選擇性狀態空間（Mamba）掃描，加上局部增強與通道注意力；v2 加入「注意力式」非因果掃描	以線性成本取得全域感受野

反覆出現的設計理由。（1）**全域殘差**：預測 $x - y$（或 x 減去上採樣的 y），讓網路只學缺少的部分。（2）自 EDSR [10] 以來，大多數 SR／復原網路**不用批次正規化**：訓練小塊與完整測試影像的 BN 統計量不同，會損害色彩與數值範圍的保真度。（3）以可控成本取得**大感受野**：多尺度 U-Net、視窗或通道注意力、狀態空間掃描。復原既需要上下文（分辨紋理與雜訊），也需要全解析度的細節。（4）用**注意力或門控**做內容自適應處理：通道注意力（RCAN）、門控（NAFNet、Restormer）。原文的「Extension」清單預言了 ViT、Flow、GAN、Diffusion 等新架構；到了 2026 年，狀態空間模型也加入 CNN 與 Transformer，成為第三種骨幹選項 [49], [50]。

All-in-one 與通用復原

白話版。不要為雨、霧、雜訊各訓練一個模型，而是訓練**一個**模型，讓它自己判斷影像哪裡出了問題，再把它修好。

動機

原文的「Extension」一節把 all-in-one 復原列為 2023 年最熱門的題目之一，大約從 2021 年開始。動機很實際：在實際部署的系統中，我們事先不知道某張影像有哪一種退化（或哪幾種混合），各種天氣也可能隨時出現或同時發生。一整組任務專屬模型需要前面再接一個可靠的分類器，還浪費記憶體；一個能自我調整的模型方便得多。困難在於**任務干擾**：不同退化需要不同、甚至相反的操作（去雜訊是平滑，去模糊是銳化），所以直接做多任務訓練的效果會輸給專用模型。

代表方法

- **AirNet** [51]（CVPR 2022），也就是原文提到的 airnet：以對比學習訓練的退化編碼器，在沒有標籤的情況下擷取影像「**如何**」受損的表徵，再用它引導復原網路。它用單一模型處理未知類型的退化；它的雜訊＋雨＋霧設定後來成為標準的「三任務」基準。
- **TransWeather** [52]（CVPR 2022）：單一 Transformer 編碼器–解碼器處理雨、霧、雪，解碼器中有可學習的**天氣類型查詢**（weather-type query）。
- **PromptIR** [53]（NeurIPS 2023）：Restormer 風格的網路加上輕量的**提示區塊**（prompt block）。可學習的提示元件依照輸入特徵預測出的權重組合，注入解碼器，編碼退化相關的資訊。在去雜訊、除雨、除霧上測試。
- **AdaIR** [54]（ICLR 2025）：觀察到不同退化影響的是**不同的頻帶**，於是從輸入中挖掘低頻與高頻特徵並自適應地調變。涵蓋五種退化：雜訊、霧、雨、動態模糊與低光。
- **DA-CLIP** [55]（ICLR 2024）：以一個訓練好的控制器調整視覺語言模型 CLIP，讓它的影像編碼器能從受損影像同時產生乾淨內容的嵌入與退化的嵌入，再透過交叉注意力引導復原網路。
- **InstructIR** [56]（ECCV 2024）：使用者寫一句自然語言指令（「把雨去掉」、「調亮一點」），文字編碼器把它轉成嵌入，再用這個嵌入調變復原網路。

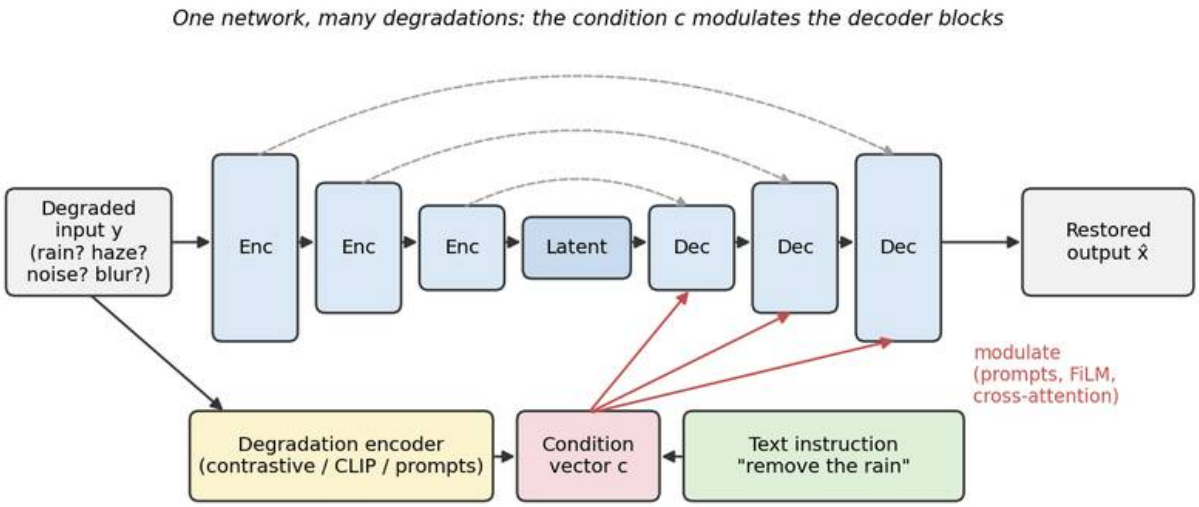


圖 5 — *All-in-one* 復原背後的共同樣板：編碼器–解碼器骨幹，加上一個能感知退化的條件 c。c 可以從影像推論（*AirNet*、*PromptIR*、*AdaIR*、*DA-CLIP*），也可以由文字給定（*InstructIR*），用來調變解碼器。

原文的動機仍是核心；改變的是「**由誰**決定退化類型」。它從固定的任務標籤，轉移到學習出的提示、視覺語言嵌入，最後到人類的指令，背後往往有大型預訓練模型支撐。All-in-one 綜述 [72] 對此有詳細討論。

損失函數

白話版。損失函數決定了什麼叫「好」。像素損失獎勵平均而言接近真實；感知與對抗損失獎勵看起來真實；頻率損失獎勵把細節做對。

像素損失。令 $\hat{x} = f_\theta(y)$，

$$\ell_2 = \frac{1}{P}\sum_p (\hat{x}_p - x_p)^2, \qquad \ell_1 = \frac{1}{P}\sum_p |\hat{x}_p - x_p|, \qquad \ell_\varepsilon = \frac{1}{P}\sum_p \sqrt{(\hat{x}_p - x_p)^2 + \varepsilon^2},$$

其中 P 是像素值的總數，ε 是一個小常數，讓最後一個成為平滑、可微分的 L1 版本。L2 直接最佳化 PSNR，但對大誤差懲罰很重，而且會收斂到後驗平均。Zhao 等人 [57] 證明在架構固定的情況下，光是換損失函數就能明顯改變結果，而且 L1 與受感知啟發的損失（包括和 SSIM 的組合）常常勝過 L2，甚至在與 L2 相關的指標上也是。L1 是現代大多數 SR 與復原論文的預設選擇。

感知（特徵）損失。在預訓練分類網路 ϕ（通常是 VGG）的特徵空間中比較影像 [58]：

$$\ell_{\text{feat}} = \frac{1}{C_j H_j W_j}\|\phi_j(\hat{x}) - \phi_j(x)\|_2^2,$$

其中 ϕ_j 是第 j 層的特徵圖，有 C_j 個通道、尺寸為 $H_j \times W_j$。它能容忍小的位移，獎勵的是邊緣與紋理吻合，而不是數值完全相同。

對抗損失。判別器 D_ψ 學習分辨復原影像與真實影像，復原網路則學著騙過它，例如 $\ell_{\text{adv}} = -\log D_\psi(\hat{x})$。這會把輸出推向自然影像的流形上（SRGAN [12]、ESRGAN [13]、Real-ESRGAN [4]），代價是可能出現憑空捏造的紋理。實務上以 GAN 為基礎的復原網路會使用加權和，例如 $\ell = \ell_1 + \lambda_p \ell_{\text{feat}} + \lambda_a \ell_{\text{adv}}$，權重為 λ_p, λ_a。

頻率損失。網路傾向先擬合低頻，對細節擬合不足。在傅立葉轉換 $\mathcal{F}$ 上加一項損失，例如對複數值或振幅計算 $\|\mathcal{F}(\hat{x}) - \mathcal{F}(x)\|_1$，就能直接懲罰缺少的高頻。**焦點頻率損失**（focal frequency loss）[59] 更進一步，自適應地加重目前最難重建的頻率成分。頻率項現在在去模糊與 all-in-one 模型中很常見。

評估指標

白話版。「復原結果是否接近真實？」與「它看起來像不像真的照片？」是兩個不同的問題，由不同的指標回答，而且你無法同時把兩者做到最好。

全參考指標

PSNR（峰值訊雜比）：

$$\text{PSNR}(x, \hat{x}) = 10 \log_{10} \frac{\text{MAX}^2}{\text{MSE}(x, \hat{x})},$$

其中 MAX 是像素可能的最大值（1 或 255），MSE 是均方誤差。單位是 dB；提升 1 dB 約等於 MSE 降低 21%。

SSIM（結構相似度）[60] 比較局部平均、變異數與共變異數：

$$\text{SSIM}(a, b) = \frac{(2\mu_a\mu_b + C_1)(2\sigma_{ab} + C_2)}{(\mu_a^2 + \mu_b^2 + C_1)(\sigma_a^2 + \sigma_b^2 + C_2)},$$

在兩張影像的局部視窗 a、b 上計算再取平均，其中 μ 是局部平均，σ^2 是局部變異數，σ_{ab} 是局部共變異數，C_1, C_2 是維持數值穩定的小常數。它和人眼感受到的結構相關性比 PSNR 好，但仍然偏好平滑的結果。

LPIPS [61] 衡量深度網路特徵之間的距離（依人類相似度判斷校準）。越低越好，它會獎勵 PSNR 所懲罰的逼真紋理。

```
import numpy as np
from scipy import ndimage as ndi
from skimage import data, img_as_float32
from skimage.metrics import peak_signal_noise_ratio, structural_similarity

x = img_as_float32(data.camera())
y = np.clip(x + np.random.default_rng(0).normal(0, 0.1, x.shape), 0, 1).astype(np.float32)
x_hat = ndi.gaussian_filter(y, 1.0)

def psnr(ref, est, peak=1.0):
```

```
    mse = np.mean((ref.astype(np.float64) - est) ** 2)
    return 10 * np.log10(peak ** 2 / mse)

print(f"PSNR  ours {psnr(x, x_hat):.2f} dB | skimage {peak_signal_noise_ratio(x, x_hat,
 ↪  data_range=1):.2f} dB")
print(f"SSIM  {structural_similarity(x, x_hat, data_range=1):.3f}")
# PSNR  ours 27.20 dB | skimage 27.20 dB
# SSIM  0.639
```

一定要寫清楚計算慣例（RGB 或 Y 通道、邊界裁切、數值範圍、是否先四捨五入成 8 位元）：光是這些差異，就足以讓論文中的 PSNR 相差零點幾 dB。

無參考指標

白話版。無參考（no-reference，又稱盲，blind）指標只看一張影像，就猜測人會給它幾分，沒有「標準答案」可以對照。這就像老師沒有參考答案也要批改作文：只能依靠「好作文通常長什麼樣子」的經驗。影像復原需要這類指標，因為我們最想修復的照片（真實的模糊、雜訊、下雨、昏暗照片）根本沒有乾淨的版本。

全參考、部分參考與無參考

影像品質評估（image quality assessment, IQA）方法依照它們對原始影像 x 知道多少來分類：

- **全參考（full-reference, FR）**：完整的 x 可用，指標逐像素或逐特徵比較 $\hat{x}$ 與 x（PSNR、SSIM [60]、LPIPS [61]）。合成基準用的就是這一類。
- **部分參考（reduced-reference, RR）**：只有 x 的少數摘要特徵可用（例如隨壓縮視訊一起傳送的統計量），指標在 $\hat{x}$ 上計算同樣的特徵再比較。
- **無參考（no-reference, NR，盲）**：只有 $\hat{x}$ 可用。指標必須自帶一套「什麼叫好」的模型。

復原需要 NR 指標有三個理由。(1) **真實測試影像沒有真值**：真實世界 SR、去雨、除霧都在沒有人擁有乾淨版本的照片上評估。(2) **參考影像本身可能不完美，或只是眾多合理答案之一**：GAN 或擴散模型的輸出可能看起來非常好，PSNR 卻很低，因為它的紋理與參考影像不同（見下方的感知–失真權衡）。(3) **NR 分數可以在執行時引導系統**，例如在沒有參考影像時，從多個輸出或參數中挑一個。代價是：NR 指標是人類意見的模型，而每個模型都有盲點。

如何驗證 NR 指標：與人類意見的相關性

白話版。只有當一個 NR 指標對影像的排序和人一致時，我們才信任它。所以研究者收集大量影像的大量人類評分，再檢查指標分數與評分有多一致。

平均意見分數。在主觀實驗中，每張影像 i 由多位觀察者評分，平均分就是它的**平均意見分數**（mean opinion score），$\mathrm{MOS}_i = \frac{1}{R}\sum_{r=1}^{R} s_{ir}$，其中 s_{ir} 是評分者 r 給的分數，R 是評分者人數。有些資料集公布的是**差異**分數（DMOS），以失真影像與其參考影像評分之差為基礎；DMOS 通常越低越好。

SRCC（Spearman 等級相關係數）衡量指標是否把影像排成和人相同的順序：

$$\mathrm{SRCC} = 1 - \frac{6\sum_{i=1}^{n} d_i^2}{n\,(n^2-1)},$$

其中 n 是測試影像數，d_i 是影像 i 依預測分數的名次與依 MOS 的名次之差（這個簡式假設沒有同名次；有同名次時改算名次的 Pearson 相關）。SRCC 對分數做任何單調轉換都不變，因此它衡量的是**預測的單調性**。

PLCC（Pearson 線性相關係數）衡量分數之間有多線性地一致：

$$\mathrm{PLCC} = \frac{\sum_{i=1}^{n}(q_i-\bar{q})(m_i-\bar{m})}{\sqrt{\sum_{i=1}^{n}(q_i-\bar{q})^2}\,\sqrt{\sum_{i=1}^{n}(m_i-\bar{m})^2}},$$

其中 q_i 是指標給影像 i 的分數，m_i 是它的 MOS，$\bar{q}$、$\bar{m}$ 是兩者的平均。由於各指標的尺度是任意的，常見做法是先用一個擬合出的單調函數（通常是 logistic 函數）轉換 q_i，再在轉換後的分數上算 PLCC，因此 PLCC 衡量的是校準後的**預測準確度**。兩者都落在 $[-1,1]$。對 NIQE 這種「越低越好」的指標，與 MOS 的原始相關係數是負的，論文會報告其絕對值。指標通常在資料集的一部分上訓練、在其餘部分測試，而跨資料集的相關性才是對泛化能力更誠實的檢驗。

人類評分資料集。 NR 模型在 IQA 資料集上訓練與測試。這些資料集的差別在於失真從哪裡來，這點對復原非常重要：

資料集	失真來源	內容
LIVE（Release 2）[104]	合成	五種失真類型（JPEG、JPEG2000、高斯模糊、白雜訊、JPEG2000 位元錯誤）的影像，附 DMOS
TID2013 [105]	合成	25 張參考影像 × 24 種失真 × 5 個等級 = 3,000 張失真影像；分數來自五個國家觀察者的 971 次實驗
KADID-10k [106]	合成	81 張原始影像 × 25 種失真 × 5 個等級 = 10,125 張，每張 30 個群眾外包評分
KonIQ-10k [107]	真實（in the wild）	10,073 張照片，約 120 萬個來自 1,459 位群眾外包工作者的評分
SPAQ [108]	真實（智慧型手機）	66 支智慧型手機拍攝的 11,125 張照片，附品質、屬性（亮度、色彩豐富度、對比、雜訊程度、銳利度）與場景標籤，以及 EXIF 資料
PIPAL [109]	復原輸出	影像復原演算法（包含 GAN 方法）的輸出，超過 113 萬個人類判斷，以 Elo 系統轉成分數

合成資料集能精確控制失真，但只有「教科書式」的損壞。真實資料集有真正的相機問題，卻沒有復原輸出。只有 PIPAL 這類資料集，才包含復原論文真正需要評分的那種影像。

第一類：自然場景統計（NSS）模型

白話版。乾淨的照片有一些隱藏的統計習慣。把照片的局部亮度與局部對比去掉之後，剩下的部分會呈現某種典型形狀的鐘形直方圖，而且相鄰的剩餘值彼此只有微弱的關聯。模糊、雜訊與壓

縮會以各自特有的方式改變這個形狀。NSS 指標就是量測這個形狀，再問它有多不尋常。

MSCN 係數。對灰階影像 I，**去均值對比正規化**（mean-subtracted contrast-normalized, MSCN）係數為 [94]

$$\hat{I}(i,j) = \frac{I(i,j) - \mu(i,j)}{\sigma(i,j) + C}, \qquad \mu(i,j) = \sum_{k,l} w_{k,l} I(i+k, j+l), \qquad \sigma(i,j) = \sqrt{\sum_{k,l} w_{k,l} \big(I(i+k,j+l) - \mu(i,j)\big)^2},$$

其中 (i,j) 是像素位置，w 是權重總和為 1 的小型圓對稱高斯視窗，μ 與 σ 是局部加權平均與標準差，C 是一個小常數（8 位元影像取 1），避免在平坦區域除以零。這是一種除法正規化，大致模仿人類早期視覺中的對比增益控制。

廣義高斯擬合。$\hat{I}$ 的直方圖以零均值的廣義高斯分布（generalized Gaussian distribution, GGD）建模：

$$f(t; \alpha, \sigma^2) = \frac{\alpha}{2\beta\,\Gamma(1/\alpha)} \exp\!\Big(-(|t|/\beta)^{\alpha}\Big), \qquad \beta = \sigma\sqrt{\frac{\Gamma(1/\alpha)}{\Gamma(3/\alpha)}},$$

其中 t 是一個 MSCN 值，$\alpha > 0$ 是**形狀**參數（$\alpha = 2$ 為高斯，$\alpha = 1$ 為拉普拉斯；α 越小直方圖越尖、尾巴越重，越大則頂部越平），σ^2 是變異數，β 是對應的尺度，Γ 是 gamma 函數。估計 α 的簡單方法是**動差匹配**（moment matching）：比值 $\mathbb{E}[t^2]/(\mathbb{E}|t|)^2 = \Gamma(1/\alpha)\,\Gamma(3/\alpha)/\Gamma(2/\alpha)^2$ 只與 α 有關，所以先從資料算出左邊，再搜尋能對上的 α。

BRISQUE [94]（Blind/Referenceless Image Spatial Quality Evaluator）計算 MSCN 係數的 GGD 形狀與變異數，再加上對相鄰 MSCN 係數乘積（水平、垂直與兩個對角方向）擬合非對稱 GGD 得到的參數，並在兩個影像尺度上重複。一個以合成失真影像的人類評分訓練的支持向量迴歸器（SVR），把這些特徵對應成品質分數。越低越好。它是**需要意見分數的**（opinion-aware）：訓練時需要人類評分。

NIQE [62]（Natural Image Quality Evaluator）則**不需要意見分數**（opinion-unaware）：它從未看過人類評分或失真影像。它在清晰的區塊上計算類似的 NSS 特徵，對一組原始影像的特徵擬合一個均值 ν_1、共變異數 Σ_1 的多變量高斯（multivariate Gaussian, MVG），對測試影像的區塊擬合另一個 MVG (ν_2, Σ_2)，然後報告

$$D(\nu_1, \nu_2, \Sigma_1, \Sigma_2) = \sqrt{(\nu_1 - \nu_2)^{\top} \Big(\frac{\Sigma_1 + \Sigma_2}{2}\Big)^{-1} (\nu_1 - \nu_2)},$$

也就是兩個高斯之間類似 Mahalanobis 距離的量（$^{\top}$ 為轉置）。越低越好；零代表「統計上和原始影像集一樣」。

PIQE [96]（Perception-based Image Quality Evaluator）也使用 MSCN 係數，但**逐區塊**運作：把影像切成 16×16 的區塊，標記出現區塊效應或雜訊的區塊，再把被標記區塊的失真彙整成一個分數。它不需要用意見分數訓練，還能給出影像哪裡看起來失真的局部圖。在常見的 MATLAB 實作中，分數介於 0 到 100，越低越好。

Ma 等人的分數與感知指數。Ma 等人 [95] 對超解析影像做了人類評分實驗，並用這些分數學出一個 SR 專用的無參考指標：使用空間域與頻率域的統計特徵，以及兩階段迴歸；他們的分數（「Ma」，越高越好）大致介於 0 到 10。2018 年 PIRM 感知超解析挑戰賽 [63] 把它與 NIQE 合成**感知指數**（Perceptual Index）：

$$\mathrm{PI} = \tfrac{1}{2}\big((10 - \mathrm{Ma}) + \mathrm{NIQE}\big),$$

PI 越低代表感知品質越好。PIRM 在感知–失真平面上依 RMSE 分成三個區域（RMSE 分別不超過 11.5、12.5 與 16），在每個區域內以 PI 排名，因此是在固定的失真預算下比較真實感。許多感知 SR 論文至今仍報告 PI。

程式：實際看看 MSCN 統計量。下面的程式在 `camera` 及其模糊、加雜訊、過度銳化的版本上，計算 MSCN 係數、以動差匹配估計的 GGD 形狀、MSCN 變異數，以及水平相鄰係數的相關（BRISQUE 成對特徵的簡化版）：

```
import numpy as np
from scipy.ndimage import gaussian_filter
from scipy.special import gamma
from skimage import data, img_as_float

def mscn(img, sigma=7/6, C=1.0):
    """Mean-subtracted contrast-normalized coefficients of a grayscale image in [0, 255]."""
    mu = gaussian_filter(img, sigma)
    sd = np.sqrt(np.abs(gaussian_filter(img * img, sigma) - mu * mu))
    return (img - mu) / (sd + C)

def ggd_shape(x):
    """Moment-matching estimate of the generalized Gaussian shape alpha (2 = Gaussian)."""
    alphas = np.arange(0.2, 10.0, 0.001)
    r_theory = gamma(1 / alphas) * gamma(3 / alphas) / gamma(2 / alphas) ** 2
    r_data = np.mean(x ** 2) / np.mean(np.abs(x)) ** 2
    return alphas[np.argmin((r_theory - r_data) ** 2)], np.mean(x ** 2)

rng = np.random.default_rng(0)
clean = img_as_float(data.camera()) * 255
versions = {
    "clean": clean,
    "blur (sigma=2)": gaussian_filter(clean, 2),
    "noise (sigma=20)": np.clip(clean + rng.normal(0, 20, clean.shape), 0, 255),
    "over-sharpened": np.clip(clean + 2.0 * (clean - gaussian_filter(clean, 2)), 0, 255),
}
for name, img in versions.items():
    m = mscn(img)
    alpha, var = ggd_shape(m)
    rho = np.mean(m[:, :-1] * m[:, 1:]) / var           # correlation of horizontal neighbors
    print(f"{name:17s} GGD shape {alpha:4.2f} | MSCN variance {var:4.2f} | neighbor corr.
     ↪  {rho:+.2f}")
# clean             GGD shape 1.56 | MSCN variance 0.28 | neighbor corr. +0.12
# blur (sigma=2)    GGD shape 0.87 | MSCN variance 0.04 | neighbor corr. +0.89
# noise (sigma=20)  GGD shape 3.12 | MSCN variance 0.68 | neighbor corr. -0.10
# over-sharpened    GGD shape 2.35 | MSCN variance 0.43 | neighbor corr. +0.09
```

每種失真都把特徵推往自己的方向（圖 6）。**模糊**去掉局部對比，大部分 MSCN 值縮向零：變異數很小、直方圖非常尖（α 小）、相鄰值高度相關。**雜訊**加入彼此獨立的能量：變異數變大、直方圖頂部變平（α 大於 2）、相鄰相關接近零（略為負值，因為減去局部平均會讓相鄰值彼此牽連）。**過度銳化**則把特徵推往和雜訊相同的方向。這正是 NSS 指標用於復原時的核心弱點：它分不出復原出來的紋理和憑空加上的高頻能量。另外要注意，絕對數值取決於影像內容與 C（乾淨的 `cam`

era 影像並不是單位高斯），這就是為什麼 BRISQUE 需要訓練迴歸器、NIQE 需要原始參考影像集，而不是使用固定門檻。

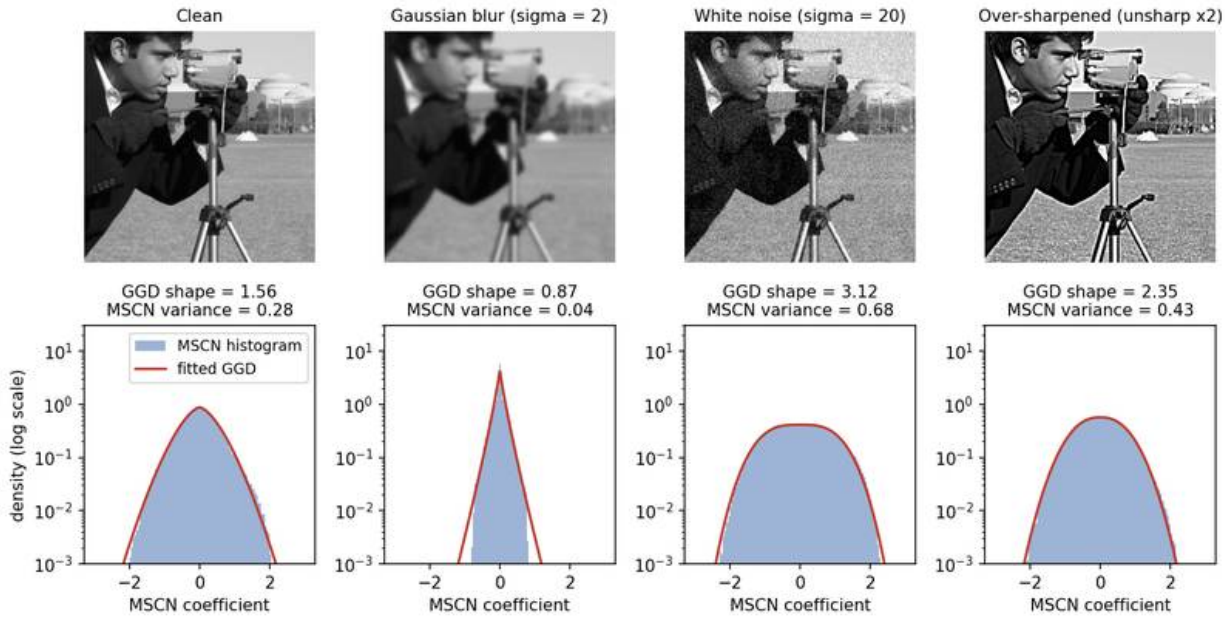


圖 *6 — MSCN* 統計量，*BRISQUE*、*NIQE* 與 *PIQE* 的基礎。模糊讓直方圖變窄變尖；雜訊與過度銳化都讓它變寬變平，因此 *NSS* 指標會把兩者看成相似的變化。

第二類：學習式深度模型

白話版。不用手工設計的統計量，而是訓練神經網路直接預測人類評分。深度特徵更能掌握內容、語意與複雜的瑕疵，但模型只懂它訓練時看過的影像與意見。

- **NIMA** [97] 用 CNN 預測人類評分的整個分布，而不是單一數字。令 p_k 為預測的分數 k 的機率（1–10 分制），報告的分數是平均值 $\hat{s} = \sum_k k\, p_k$，分布的離散程度則代表人們可能多不一致。以美學評分訓練的權重判斷的是「這是不是一張好看的照片」，而不是「這是不是忠實的復原」。
- **HyperIQA** [98] 針對真實環境（in the wild）中的失真。它把評估拆成三個階段：骨幹網路先理解內容，接著一個**超網路**（hyper network）依據內容產生品質預測網路的權重（「感知規則」隨畫面內容調整，因為同樣的模糊出現在天空上和臉上，給人的感受不同），最後由產生出來的網路預測分數。
- **MUSIQ** [99] 是多尺度 transformer，以**原始解析度與長寬比**輸入影像，外加縮小的版本，並使用基於雜湊的 2-D 空間與尺度嵌入，因此不會因縮放或裁切而改變正要被評估的瑕疵。
- **MANIQA** [100] 結合 vision transformer 特徵、轉置(通道)注意力區塊與尺度 Swin transformer 區塊，再用雙分支、區塊加權的輸出頭預測分數。它贏得 NTIRE 2022 感知 IQA 挑戰賽的無參考賽道，該賽事資料包含復原與 GAN 演算法的輸出，因此比用相機失真訓練的指標更接近復原的情境。
- **TOPIQ** [101] 採用**由上而下**（top-down）的想法：高階語意特徵決定該看哪裡，再透過跨尺度注意力機制用它們加權低階、對失真敏感的特徵（網路稱為 CFANet，骨幹為 ResNet-50）。同

一設計可用於 FR 與 NR 評估，且比基於 transformer 的 IQA 模型輕量。

這些模型都是**需要意見分數的**，輸出「越高越好」的分數，而且都依賴訓練資料集：同一架構在 KonIQ-10k 上訓練與在 PIPAL 上訓練，可能會給復原方法不同的排名。

第三類：視覺–語言與大型多模態模型

白話版。用大量圖文資料訓練出的模型，已經「知道」好照片和壞照片長什麼樣子。你可以直接問它們：把影像和文字提示比較，或乾脆請一個聊天機器人式的模型打分數。

CLIP-IQA [102] 比較影像的 CLIP 嵌入與一對反義提示（例如「Good photo.」與「Bad photo.」）的嵌入。令 $\mathbf{x}$ 為影像嵌入，$\mathbf{t}_1, \mathbf{t}_2$ 為兩個文字嵌入，$s_i = \mathbf{x}^\top \mathbf{t}_i / (\|\mathbf{x}\| \, \|\mathbf{t}_i\|)$ 為它們的餘弦相似度，分數為

$$\text{CLIP-IQA} = \frac{e^{s_1}}{e^{s_1} + e^{s_2}} \in [0, 1],$$

越高越好。基本版本是**零樣本**（zero-shot）的（完全不需要意見分數），換用其他提示對（「Noisy photo.」/「Clean photo.」、「Sharp photo.」/「Blurry photo.」）可以探測單一屬性。提示的措辭會改變分數。

Q-Align [103] 微調一個大型多模態模型，讓它像人類評分者一樣回答：給出**文字評分等級**（bad、poor、fair、good、excellent），而不是數字。測試時，令 X_{l_i} 為模型對等級 l_i 的 logit，分數是只在這五個字上做 softmax 後的期望等級：

$$S = \sum_{i=1}^{5} i \cdot \frac{e^{X_{l_i}}}{\sum_{j=1}^{5} e^{X_{l_j}}},$$

其中 $i = 1, \ldots, 5$ 依序代表從 bad 到 excellent 的等級。這類模型跨資料集表現強，但運算量大，也會繼承基礎模型的偏誤。

分布層級的指標

有些指標評的是一組輸出而非單張影像。**FID** [64] 比較復原結果集合與一組真實影像在 Inception 特徵上的分布；它原本是為生成模型設計的，現在用於擴散 SR 這類生成式復原，但它需要大量影像，而且無法告訴你任何單張影像的好壞。原文也提到的 **IS**（Inception Score）衡量類別預測的信心與多樣性，比較適合類別條件式生成而非復原，如今在復原任務中很少使用。

常見陷阱

- **指標可以被刻意迎合**。一個會加顆粒或銳化邊緣的復原模型可以改善 NSS 分數（上面的程式就顯示了這點），卻沒有變得更忠實。任何可微分的指標（以及任何學習式 IQA 網路）一旦被拿來當訓練損失函數或用於模型選擇，都可能被直接最佳化，網路學會的是討好指標而不是討好人。如果一個方法在損失函數中用了 CLIP 或某個 IQA 網路，再用同一個模型評估它就不是獨立的評估。

- **領域不匹配**。大多數需要意見分數的指標是在合成失真（LIVE、TID2013、KADID-10k）或相機照片（KonIQ-10k、SPAQ）上訓練的。復原輸出的瑕疵不一樣：憑空生成的紋理、過度平滑、GAN 特有的圖樣。這種不匹配正是 PIPAL [109] 明確的動機：其作者發現既有的 IQA 方法無法公平評估以 GAN 為基礎的復原。低光源的問題則在於曝光與色彩，而大多數 NSS 特徵會把它們正規化掉；低光源專屬的討論請見DLCV 05。
- **指標彼此不一致**。 NSS、學習式與視覺–語言指標常把同一批方法排出不同的順序，而且都可能與人的判斷不同。分數也取決於實作、權重檔、色彩空間、輸入尺寸以及影像是否被縮放，所以從不同論文抄來的數字往往無法直接比較。
- 「**NR 品質較高**」**不等於**「**較正確**」。 NR 指標無法知道一張清晰的臉或一行文字是否和真實場景相符。憑空生成的車牌也可能拿到滿分。

建議做法

1. 有參考影像時，以 FR 指標（PSNR、SSIM、LPIPS）為主，NR 指標作為補充；並在感知–失真平面上標出你的方法（見下一小節）。
2. 在沒有參考影像的真實影像上，報告**多個來自不同類別的 NR 指標**（例如 NIQE 或 PI、一兩個學習式模型如 MUSIQ、MANIQA 或 TOPIQ，以及一個視覺–語言指標如 CLIP-IQA 或 Q-Align），並註明每個指標是越高還是越低越好。
3. 使用同一套實作並寫明版本與權重檔。開源的 **IQA-PyTorch** 工具箱（`pyiqa`）[110] 以統一介面重新實作了本節大多數的指標（NIQE、BRISQUE、PIQE、Ma 分數與 PI、NIMA、HyperIQA、MUSIQ、MANIQA、TOPIQ、CLIP-IQA、Q-Align，以及 FR 指標）。
4. 不要對著裁判調參：至少保留一個完全沒參與訓練或模型選擇的指標。
5. 重要的主張要有**使用者研究**佐證（平均意見分數或成對比較，盲測且隨機化）；DLCV 05 介紹了 MOS 與 Bradley–Terry 成對比較的流程。指標之間或指標與人不一致時，要明白說出來。

總覽表

指標	類別	需要訓練嗎？	方向	主要對什麼敏感
NIQE [62]	NSS	只需原始影像（不需意見分數）	越低越好	局部統計量的偏離：模糊、雜訊、壓縮
BRISQUE [94]	NSS + 迴歸	需要，合成失真上的人類評分	越低越好	同樣的統計量，對應到人類評分
PIQE [96]	NSS，逐區塊	不需意見分數	越低越好	局部區塊中的區塊效應與雜訊；可輸出失真圖
Ma [95]	手工特徵 + 迴歸	需要，SR 輸出上的人類評分	越高越好	SR 瑕疵、紋理與銳利度
PI [63]	NSS + 學習式	透過其中的 Ma 部分	越低越好	SR 的感知品質；獎勵紋理，包含被放大的雜訊
NIMA [97]	學習式 CNN	需要，評分分布	越高越好	技術品質或美學品質，視權重檔而定
HyperIQA [98]	學習式 CNN + 超網路	需要	越高越好	真實失真，並依內容脈絡判斷

指標	類別	需要訓練嗎？	方向	主要對什麼敏感
MUSIQ [99]	學習式 transformer	需要	越高越好	原始解析度下的多尺度品質
MANIQA [100]	學習式 transformer	需要	越高越好	復原與 GAN 瑕疵（以這類資料訓練）
TOPIQ [101]	學習式 CNN，由上而下注意力	需要	越高越好	語意重要區域中的失真
CLIP-IQA [102]	視覺–語言	不需要（零樣本版本）	越高越好	整體「好照片」印象；取決於提示
Q-Align [103]	大型多模態模型	需要，評分轉成文字等級	越高越好	類似人類的整體品質判斷
FID [64]	分布層級	使用預訓練 Inception 特徵	越低越好	一組輸出與真實影像之間的分布差距

感知–失真權衡

Blau 與 Michaeli [65] 把這個矛盾形式化。令 $\Delta(x, \hat{x})$ 為任意失真度量（MSE、$1 - $ SSIM……），$d(p_X, p_{\hat{X}})$ 為真實影像分布與復原影像分布之間的散度（感知品質：d 越小，輸出在統計上越無法與真實影像區分）。**感知–失真函數**定義為

$$P(D) = \min_{p_{\hat{X}|Y}} d(p_X, p_{\hat{X}}) \quad \text{subject to} \quad \mathbb{E}[\Delta(X, \hat{X})] \le D,$$

其中最小值取遍所有（可能帶隨機性的）復原方法。他們證明 $P(D)$ 是非遞增的，而且（當散度對第二個引數為凸時）是凸函數：原點附近有一塊無法達到的區域。一個方法不可能同時擁有最小失真與完美的真實感。特別是 MMSE 估計 $\mathbb{E}[x \mid y]$ 的失真最低，但輸出分布模糊、不真實；想看起來真實，就必須接受較高的 MSE。GAN（以及現在的擴散模型）是沿著這條邊界移動的原則性方法。

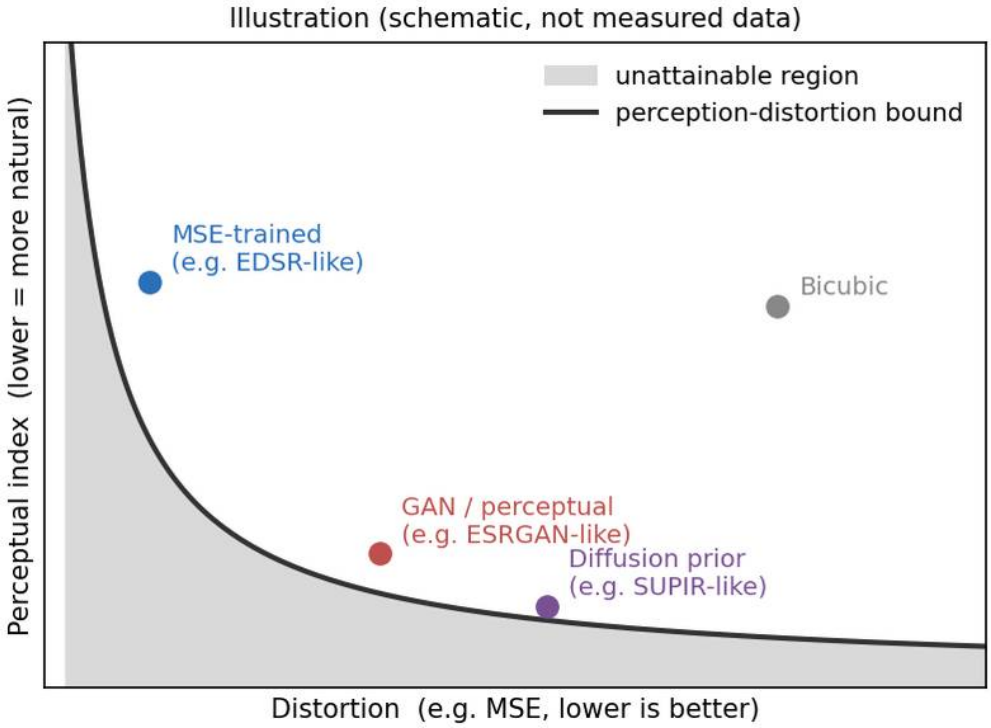


圖 7 — 感知–失真權衡的示意圖 [*65*]（非實測資料）。邊界是凸且遞減的；各方法在保真度（左）與真實感（下）之間取捨。

這正是原文評估指標一節的更新：擴散模型能「生成」看似合理但不一定忠於原圖的細節，保真度（PSNR ／ SSIM）與感知品質之間的落差因此更受重視。**NTIRE 2025** ×4 SR 挑戰賽就分成兩個賽道：以 PSNR 排名的 restoration track，以及以感知分數排名的 perceptual track [3]；2024 年的版本則以 PSNR 排名 [2]。實務建議：同時報告一個失真指標與一個感知指標，並說明你的方法瞄準權衡曲線上的哪個位置。

NTIRE 挑戰賽

白話版。每年 CVPR 的一個 workshop 會在固定資料集上舉辦公開競賽，並發表報告說明勝出的想法：這是了解「現在什麼有用」最簡單的方式。

既然依任務分類的排行榜已經消失，NTIRE 挑戰賽報告就是追蹤復原領域的好管道。每份報告都會說明任務、資料、評估方式，以及名列前茅隊伍的方法。本篇引用的例子：

- **NTIRE 2017 SR** [19]：推出 DIV2K；EDSR [10] 奪冠。
- **NTIRE 2024** ×4 **SR** [2]：在 DIV2K 上做雙三次 ×4，以 PSNR 排名。
- **NTIRE 2025** ×4 **SR** [3]：restoration（PSNR）與 perceptual 兩個賽道分開評比。
- **NTIRE 2025 雨滴去除** [40]：真實的白天與夜晚雨滴影像，並有雙重對焦設定。

近年的報告有兩個共同模式：勝出的作品通常是上述骨幹（HAT、Restormer、NAFNet 或

Mamba 風格，有時再加上擴散先驗）的集成或大型混合模型，並使用大量資料與算力訓練；而挑戰賽也越來越常使用真實拍攝的資料與感知賽道。

現代觀點

原文最後列出了延伸方向（醫學影像、多模態、非監督、新架構、all-in-one、低階結合高階、即時與低運算、水下）。其中大多數現在都是活躍的研究領域。以下依任務整理綜述文獻，再談 2023–2026 年的最新進展與尚未解決的問題。

綜述論文

- **Wang、Chen 與 Hoi，〈Deep Learning for Image Super-resolution: A Survey〉（TPAMI 2021）** [66]。把 SR 分成監督式、非監督式與特定領域 SR；在監督式 SR 中，再依上採樣位置（前、後、漸進、迭代上下採樣）、上採樣運算（插值、轉置卷積、子像素）、網路設計（殘差、遞迴、密集、注意力、多路徑）、損失函數與訓練策略來拆解模型。重點：大部分 SR 的進展都能描述成在這些軸上的選擇，而非監督／真實世界 SR 被點名為主要的開放方向，領域後來也確實往這裡走。
- **Tian 等人，〈Deep Learning on Image Denoising: An Overview〉（Neural Networks 2020）** [67]。把 CNN 去雜訊方法分成加性白雜訊、真實雜訊、盲去雜訊，以及混合了模糊或低解析度的雜訊四類，並在公開基準上比較。重點：瓶頸往往是雜訊模型的真實程度，而不是網路。
- **Elad、Kawar 與 Vaksman，〈Image Denoising: The Deep Learning Revolution and Beyond〉（SIAM J. Imaging Sciences 2023）** [68]。回顧去雜訊從傳統先驗到深度網路的歷史，接著主張去雜訊器如今是其他反問題（plug-and-play、regularization by denoising）以及擴散式影像生成的基本元件，並主張復原應該接受多個合理解（後驗取樣），而不是只給一個答案。重點：前沿已經從「更好的 MMSE 去雜訊器」移到「從後驗分布取樣」。
- **Zhang 等人，〈Deep Image Deblurring: A Survey〉（IJCV 2022）** [69]。涵蓋模糊成因、資料集與指標，以及依架構、損失函數與應用（人臉、文字、立體）分類的 CNN 去模糊方法，包含盲與非盲設定。重點：動態場景去模糊已從估計模糊核，轉向以 GoPro 這類實拍配對訓練的端到端多尺度網路。
- **Gui 等人，〈A Comprehensive Survey and Taxonomy on Single Image Dehazing Based on Deep Learning〉（ACM Computing Surveys 2023）** [70]。把方法分成監督式、半監督式與非監督式，並在各類中依是否及如何使用散射模型細分；也整理資料集、網路模組、損失函數與指標，並實驗比較基準方法。重點：物理引導與無物理的設計並存，合成到真實的落差是主要未解問題。
- **Yang 等人，〈Single Image Deraining: From Model-Based to Data-Driven and Beyond〉（TPAMI 2021）** [71]。回顧雨的外觀模型、以模型為基礎的方法（先驗與最佳化），以及資料驅動的方法（架構、限制條件、損失函數、資料集）。重點：真實的雨模型與真實世界的評估，比在合成資料上繼續微調架構更重要。
- **Jiang 等人，〈A Survey on All-in-One Image Restoration: Taxonomy, Evaluation and Future Trends〉（TPAMI 2025）** [72]。依架構設計、學習範式與核心創新（退化表徵、提示、混合專家、語言引導）整理 all-in-one 方法，並彙整資料集與評估協定。重點：各論文的任務組合不一致，讓比較變得困難；對未見過與混合退化的泛化仍是開放問題。

- **Li 等人，〈Diffusion Models for Image Restoration and Enhancement: A Comprehensive Survey〉（IJCV 2025）** [73]。依學習範式（監督式 vs. 零樣本使用預訓練先驗）、條件策略、框架設計與建模策略分類以擴散模型為基礎的復原，聚焦於 SR、去模糊、修補與盲／真實世界復原。重點：擴散模型的感知品質最好，但運算成本高，也有保真度與捏造細節的疑慮；蒸餾成少步驟與更好的評估方式是關鍵方向。

2023–2026 年的最新進展

1. **All-in-one 與盲復原成為主流設定**，不再是小眾題目。方法從學習式退化表徵（AirNet）發展到提示（PromptIR）、頻率感知的自適應（AdaIR）與語言引導（DA-CLIP、InstructIR）。
2. **以預訓練擴散模型作為生成先驗**，大幅改變了真實世界 SR 與盲復原（DiffBIR、SUPIR），一步式變體（OSEDiff）則把推論成本從數十、數百步降到一步。
3. **狀態空間模型**（MambaIR、MambaIRv2）加入 CNN 與 Transformer，成為以線性成本取得全域感受野的骨幹選項。
4. **評估方式跟上了生成能力**：挑戰賽把保真度賽道與感知賽道分開 [3]，真實拍攝的資料集（SIDD、O-HAZE、Dense-Haze、Raindrop Clarity）也和合成基準一起成為標準。

尚未解決的問題

- **真實世界的泛化**。合成流程仍然漏掉真實相機的影像處理、隨位置變化的模糊與混合天氣；模型遇到沒見過的組合時表現會下滑。
- **保真度 vs. 捏造**。生成先驗可能憑空創造文字、人臉或紋理。在醫學、鑑識與科學用途中這是不能接受的，而我們還缺少衡量「忠實又清晰」的好指標。
- **評估**。 PSNR ／ SSIM 獎勵模糊；無參考指標可以被刻意迎合；人類評測成本高。對生成式復原而言，指標與人類偏好的一致性仍然很弱。
- **效率**。在手機與相機上做高解析度、即時（以及影片）復原仍是很強的限制，對 Transformer 與擴散模型尤其如此。
- **All-in-one 模型的任務干擾與擴展**，以及如何在不重新訓練的情況下加入新的退化。
- **低階結合高階**。讓復原幫助下游偵測或分割，而不是給人看，是另一種目標（任務導向評估，例如 RESIDE [32]）。

作者的觀點

原文以兩個至今仍成立的觀察作結。第一，由於每個任務都簡單明瞭，現階段 SOTA 的比拼大多靠特殊的架構取勝，也就代表大量的算力與複雜的實驗設計；但這些任務在應用上都相當實用且容易上手，所以在**應用面**與**任務的組合**上還有很多著墨空間（all-in-one 復原正是往這裡發展）。第二，相較於高階任務，這個領域的學習成本不高：入門時很容易想像模型的架構與樣貌（受損的影像進去、更乾淨的影像出來），進而接觸更進一步的內容，是進入深度學習研究的好入口。

重點整理

- 低階視覺關注的是把像素**感知**好；幾乎每個任務都是 $y = \mathcal{D}(x) + n$ 的一個特例，差別只在 $\mathcal{D}$（模糊、降採樣、JPEG、散射、雨層、Retinex 光照、遮罩）。
- 深度學習把退化模型從求解器搬進了**資料產生器**：真實、隨機化的合成流程（以及實拍配對）和網路一樣重要。
- SR 的上採樣方式從雙三次先放大（SRCNN），經過轉置卷積（FSRCNN），演進到今天標準的子像素卷積（ESPCN）。
- 少數幾種通用骨幹主導了這個領域：U-Net 風格的 CNN（NAFNet）、Transformer（SwinIR、Restormer、HAT）與狀態空間模型（MambaIR）；常見的設計習慣是全域殘差、不用 BN、大感受野，以及門控／注意力。
- All-in-one 模型（AirNet、TransWeather、PromptIR、AdaIR、DA-CLIP、InstructIR）透過以學習、提示或文字給定的退化描述作為條件，讓一個網路處理未知的退化。
- L2 ／ L1 損失得到忠實但平滑的結果；感知、對抗與擴散式方法得到真實但較不忠實的結果。**感知–失真權衡**告訴我們在極限上兩者不可兼得，所以兩類指標都要報告。
- 盲去模糊與盲 SR 除了影像，還必須估計**模糊核**。天真的聯合 MAP 會偏好「沒有模糊」的解 [74]；有效的方法會把影像邊際化、使用邊緣預測或偏好清晰影像的先驗、由粗到細交替更新影像與模糊核，最後做非盲反卷積。深度方法則是預測模糊核或以它為條件（SRMD、IKC、DAN、KernelGAN、SelfDeblur），或乾脆完全略過它（以 GoPro 訓練的網路、BSRGAN、Real-ESRGAN）。
- KITTI 是自駕車基準，不是除霧資料集；除霧請用 RESIDE，並搭配 O-HAZE、Dense-Haze 這類真實配對。
- 無參考指標是人類意見的模型，以與平均意見分數的 SRCC ／ PLCC 來驗證。NSS 指標（BRISQUE、NIQE、PIQE、PI）量測 MSCN 統計量的偏離；學習式（NIMA、HyperIQA、MUSIQ、MANIQA、TOPIQ）與視覺–語言（CLIP-IQA、Q-Align）模型則從評分中學習。它們都可能被銳化或雜訊迎合，也都受訓練失真與復原瑕疵之間落差的影響，所以要同時報告多個 NR 指標、FR 指標與使用者研究。

練習

1. **退化的順序**。用 `degrade` 函式，在相同參數下比較 (a) 模糊 → 降採樣 → 雜訊 → JPEG 與 (b) 雜訊 → 模糊 → 降採樣 → JPEG。從視覺上，以及在平坦區塊量測雜訊標準差，說明為什麼順序會影響網路要學的東西。

   提示

   在 (b) 中，模糊與降採樣會過濾雜訊：標準差變小，而且在空間上變得相關（變成平滑的斑塊，而不是逐像素的顆粒）。在 (a) 中，雜訊在低解析度下仍然是白雜訊。只用 (a) 訓練的網路，測試時遇到相關雜訊會去除不足。這也是 Real-ESRGAN 隨機化並重複各個階段的原因之一。

2. **手算 pixel shuffle**。一個特徵張量的形狀為 $(Cs^2, H, W) = (4, 2, 2)$，$s = 2$、$C = 1$，第 k 個通道全部填入常數 k。用 SR 一節的公式寫出 pixel shuffle 的 4×4 輸出，再用 `torch.nn.PixelShuffle(2)` 驗證。

提示

輸出像素 $(2h+i, 2w+j)$ 取自通道 $2i+j$。所以每個 2×2 區塊都是 $\begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}$，$4\times 4$ 的輸出就是把這個區塊重複四次。每個輸出區塊都從同一個低解析度位置上的四個通道各取一個值。

3. **殘差還是直接預測？**把 `TinyDnCNN.forward` 改成回傳 `self.body(y)`（直接預測乾淨影像），以相同的隨機種子與步數訓練兩個版本，比較 PSNR 與損失曲線。再用 $\sigma = 5/255$ 重做一次。殘差學習在什麼情況下幫助最大？

提示

在訓練步數很少時，殘差版本通常收斂更快、PSNR 更高，因為它在初始化時就已經大約輸出 y（PSNR 不差），只需要學一個小修正。低雜訊時直接預測的版本最吃力，因為它必須透過好幾層 ReLU 學出一個近乎恆等的映射。

4. **用數字算暗通道**。某個有霧像素 $I = (0.75, 0.78, 0.80)$，其區塊內所有通道 I^c/A^c 的最小值為 0.70，$A = (0.9, 0.9, 0.9)$。取 $\omega = 0.95$、$t_0 = 0.1$，計算這個像素的 $\tilde{t}$ 與還原後的 $\hat{J}$。

提示

$\tilde{t} = 1 - 0.95 \times 0.70 = 0.335$。接著 $\hat{J}^c = (I^c - 0.9)/0.335 + 0.9$，約為 $(0.452, 0.542, 0.601)$。輸出比輸入更暗、飽和度更高，這正是去掉霧的面紗後該有的樣子。

5. **感知 vs. 失真**。把小型去雜訊網路的輸出再加上少量新的高斯雜訊（$\sigma = 3/255$），計算加入前後的 PSNR 與 SSIM。雖然兩個指標都下降了，影像看起來卻可能**更**自然（比較不像塑膠）。把這個現象和感知–失真函數 $P(D)$ 連結起來。

提示

近似 MMSE 的輸出位於低失真的位置，但離自然影像分布很遠（太平滑）。加入顆粒會讓輸出分布更接近真實照片（d 變小），代價是 MSE 變高：這是沿著（或朝向）凸邊界的移動，而不是越過它。

6. **設計一個 all-in-one 實驗**。你要為雜訊（$\sigma \in [0, 50]$）、雨線與霧建立單一模型。說明訓練資料的混合方式、如何為網路提供條件（測試時沒有標籤）、要和哪些基準比較，以及如何偵測任務干擾。

提示

每個任務各自取資料集並平衡取樣；加入一個與網路一起訓練的退化編碼器（如 AirNet 的對比學習，或 PromptIR 的提示）。與三個使用相同骨幹、相同大小的單任務模型比較，也和不加條件的版本比較。任務干擾的跡象是：all-in-one 模型在某個任務上落後單任務模型，在其他任務上卻打平；另外也要在訓練時沒見過的混合退化（雨＋霧）上測試。

7. **動手估計模糊核**。在盲反卷積的程式中，(a) 拿掉 `shock(...)` 並設定 `keep=1.0`（使用全部梯度），比較模糊核誤差與最後的 PSNR；(b) 改從 3×3 的方框模糊核開始，而不是接近 delta 的模糊核；(c) 把 `h_true` 換成 31×31 視窗中一條長 25 像素的直線運動軌跡。用 Levin 等人 [74] 的討論與由粗到細的策略解釋每個結果。

提示

(a) 使用全部梯度時，平滑的潛在影像中大量的弱梯度會主導最小平方擬合，估計不再改善（我們的執行結果中，模糊核誤差不降反升，最後的 PSNR 也停留在模糊輸入的水準）。只挑選少數強而且預測良好的邊緣，模糊核步驟才有資訊量，這正是實務上對付天真聯合 MAP「沒有模糊」偏差的方法。(b) 這個迴圈是非凸的交替最佳化，起點很重要；不同的起點可能收斂到不同的局部極小值。(c) 在全解析度下，從 delta 起點很難找到大的模糊核：我們的執行結果中誤差下降得少很多，也只補回一小部分「真實模糊核」的 PSNR。先在縮小的影像上估計模糊核（那時它只有幾個像素長），再把結果放大，正是影像金字塔的用途。

8. **驗證一個 NR 指標**。五張復原影像的平均意見分數為 MOS $= (4.1, 3.2, 2.5, 1.8, 3.9)$，NIQE 分數為 $(3.4, 4.2, 5.1, 6.3, 3.0)$。(a) 手算 SRCC，並解釋它的正負號。(b) 如果改報告 $\log(\text{NIQE})$，SRCC 會改變嗎？PLCC 呢？(c) 把 MSCN 程式用在一張去雜訊後再過度銳化的影像上，解釋為什麼 NSS 指標可能給它比單純去雜訊的輸出更好的分數。

提示

(a) 由小到大排名時，影像 1–5 的 MOS 名次為 $(5, 3, 2, 1, 4)$，NIQE 名次為 $(2, 3, 4, 5, 1)$。名次差為 $(3, 0, -2, -4, 3)$，所以 $\sum d_i^2 = 38$，$\text{SRCC} = 1 - 6 \cdot 38/(5 \cdot 24) = -0.9$。符號為負，是因為 NIQE 越低代表品質越好；報告時取 $|\text{SRCC}| = 0.9$。只有影像 1 與影像 5 的順序對調。(b) SRCC 只依賴名次，任何像 log 這樣的單調轉換都不會改變它；PLCC 則會改變，這也是為什麼 PLCC 通常在擬合單調對應之後才計算。(c) 去雜訊會留下平滑的影像，其 MSCN 統計量類似輕微模糊（變異數低、直方圖尖）。銳化把高頻能量加回來，把統計量推回接近自然影像的樣子，不論加上的細節是不是真的。

參考文獻

1. Shyandram, “Low-Level Vision Task-Image Restoration 簡介,” Medium post, 2023; updated 2026. link
2. Z. Chen, Z. Wu, E. Zamfir, K. Zhang, Y. Zhang, R. Timofte et al., “NTIRE 2024 Challenge on Image Super-Resolution (×4): Methods and Results,” in *Proc. IEEE/CVF CVPR Workshops*, 2024. arXiv
3. Z. Chen, K. Liu, J. Gong, J. Wang, L. Sun et al., “NTIRE 2025 Challenge on Image Super-Resolution (×4): Methods and Results,” in *Proc. IEEE/CVF CVPR Workshops*, 2025. arXiv
4. X. Wang, L. Xie, C. Dong and Y. Shan, “Real-ESRGAN: Training Real-World Blind Super-Resolution with Pure Synthetic Data,” in *Proc. IEEE/CVF ICCV Workshops*, 2021. arXiv
5. E. H. Land and J. J. McCann, “Lightness and Retinex Theory,” *Journal of the Optical Society of America*, vol. 61, no. 1, pp. 1–11, 1971. doi
6. C. Dong, C. C. Loy, K. He and X. Tang, “Image Super-Resolution Using Deep Convolutional Networks,” *IEEE TPAMI*, 2016 (conference version ECCV 2014). arXiv
7. C. Dong, C. C. Loy and X. Tang, “Accelerating the Super-Resolution Convolutional Neural Network,” in *Proc. ECCV*, 2016. arXiv
8. W. Shi, J. Caballero, F. Huszár, J. Totz, A. P. Aitken, R. Bishop, D. Rueckert and Z. Wang, “Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel Convolutional Neural Network,” in *Proc. IEEE CVPR*, 2016. arXiv

9. A. Odena, V. Dumoulin and C. Olah, "Deconvolution and Checkerboard Artifacts," *Distill*, 2016. link
10. B. Lim, S. Son, H. Kim, S. Nah and K. M. Lee, "Enhanced Deep Residual Networks for Single Image Super-Resolution," in *Proc. IEEE CVPR Workshops*, 2017. arXiv
11. Y. Zhang, K. Li, K. Li, L. Wang, B. Zhong and Y. Fu, "Image Super-Resolution Using Very Deep Residual Channel Attention Networks," in *Proc. ECCV*, 2018. arXiv
12. C. Ledig, L. Theis, F. Huszár, J. Caballero, A. Cunningham, A. Acosta, A. Aitken, A. Tejani, J. Totz, Z. Wang and W. Shi, "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *Proc. IEEE CVPR*, 2017. arXiv
13. X. Wang, K. Yu, S. Wu, J. Gu, Y. Liu, C. Dong, C. C. Loy, Y. Qiao and X. Tang, "ESRGAN: Enhanced Super-Resolution Generative Adversarial Networks," in *Proc. ECCV Workshops*, 2018. arXiv
14. J. Liang, J. Cao, G. Sun, K. Zhang, L. Van Gool and R. Timofte, "SwinIR: Image Restoration Using Swin Transformer," in *Proc. IEEE/CVF ICCV Workshops*, 2021. arXiv
15. X. Chen, X. Wang, J. Zhou, Y. Qiao and C. Dong, "Activating More Pixels in Image Super-Resolution Transformer," in *Proc. IEEE/CVF CVPR*, 2023. arXiv
16. X. Lin, J. He, Z. Chen, Z. Lyu, B. Dai, F. Yu, W. Ouyang, Y. Qiao and C. Dong, "DiffBIR: Towards Blind Image Restoration with Generative Diffusion Prior," arXiv:2308.15070, 2023. arXiv
17. F. Yu, J. Gu, Z. Li, J. Hu, X. Kong, X. Wang, J. He, Y. Qiao and C. Dong, "Scaling Up to Excellence: Practicing Model Scaling for Photo-Realistic Image Restoration In the Wild," in *Proc. IEEE/CVF CVPR*, 2024. arXiv
18. R. Wu, L. Sun, Z. Ma and L. Zhang, "One-Step Effective Diffusion Network for Real-World Image Super-Resolution," in *Proc. NeurIPS*, 2024. arXiv
19. E. Agustsson and R. Timofte, "NTIRE 2017 Challenge on Single Image Super-Resolution: Dataset and Study," in *Proc. IEEE CVPR Workshops*, 2017. doi
20. K. Zhang, W. Zuo, Y. Chen, D. Meng and L. Zhang, "Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising," *IEEE Transactions on Image Processing*, vol. 26, no. 7, pp. 3142–3155, 2017. arXiv
21. K. Zhang, W. Zuo and L. Zhang, "FFDNet: Toward a Fast and Flexible Solution for CNN-Based Image Denoising," *IEEE Transactions on Image Processing*, 2018. arXiv
22. A. Abdelhamed, S. Lin and M. S. Brown, "A High-Quality Denoising Dataset for Smartphone Cameras," in *Proc. IEEE/CVF CVPR*, 2018. doi
23. S. W. Zamir, A. Arora, S. Khan, M. Hayat, F. S. Khan and M.-H. Yang, "Restormer: Efficient Transformer for High-Resolution Image Restoration," in *Proc. IEEE/CVF CVPR*, 2022. arXiv
24. L. Chen, X. Chu, X. Zhang and J. Sun, "Simple Baselines for Image Restoration," in *Proc. ECCV*, 2022. arXiv
25. K. He, J. Sun and X. Tang, "Single Image Haze Removal Using Dark Channel Prior," in *Proc. IEEE CVPR*, 2009. doi
26. B. Cai, X. Xu, K. Jia, C. Qing and D. Tao, "DehazeNet: An End-to-End System for Single Image Haze Removal," *IEEE Transactions on Image Processing*, 2016. arXiv
27. B. Li, X. Peng, Z. Wang, J. Xu and D. Feng, "AOD-Net: All-in-One Dehazing Network," in *Proc. IEEE ICCV*, 2017. doi ·arXiv
28. X. Liu, Y. Ma, Z. Shi and J. Chen, "GridDehazeNet: Attention-Based Multi-Scale Network for Image Dehazing," in *Proc. IEEE/CVF ICCV*, 2019. arXiv

29. X. Qin, Z. Wang, Y. Bai, X. Xie and H. Jia, "FFA-Net: Feature Fusion Attention Network for Single Image Dehazing," in *Proc. AAAI*, 2020. arXiv
30. S. W. Zamir, A. Arora, S. Khan, M. Hayat, F. S. Khan, M.-H. Yang and L. Shao, "Learning Enriched Features for Real Image Restoration and Enhancement," in *Proc. ECCV*, 2020. arXiv
31. Y. Song, Z. He, H. Qian and X. Du, "Vision Transformers for Single Image Dehazing," *IEEE Transactions on Image Processing*, vol. 32, 2023. arXiv
32. B. Li, W. Ren, D. Fu, D. Tao, D. Feng, W. Zeng and Z. Wang, "Benchmarking Single-Image Dehazing and Beyond," *IEEE Transactions on Image Processing*, 2019. arXiv
33. C. O. Ancuti, C. Ancuti, R. Timofte and C. De Vleeschouwer, "O-HAZE: A Dehazing Benchmark with Real Hazy and Haze-Free Outdoor Images," in *Proc. IEEE/CVF CVPR Workshops*, 2018. doi
34. C. O. Ancuti, C. Ancuti, M. Sbert and R. Timofte, "Dense-Haze: A Benchmark for Image Dehazing with Dense-Haze and Haze-Free Images," in *Proc. IEEE ICIP*, 2019. doi
35. A. Geiger, P. Lenz and R. Urtasun, "Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite," in *Proc. IEEE CVPR*, 2012. project page
36. Y.-F. Liu, D.-W. Jaw, S.-C. Huang and J.-N. Hwang, "DesnowNet: Context-Aware Deep Network for Snow Removal," *IEEE Transactions on Image Processing*, 2018. arXiv
37. W.-T. Chen, H.-Y. Fang, J.-J. Ding, C.-C. Tsai and S.-Y. Kuo, "JSTASR: Joint Size and Transparency-Aware Snow Removal Algorithm Based on Modified Partial Convolution and Veiling Effect Removal," in *Proc. ECCV*, 2020. doi
38. W. Yang, R. T. Tan, J. Feng, J. Liu, Z. Guo and S. Yan, "Deep Joint Rain Detection and Removal from a Single Image," in *Proc. IEEE CVPR*, 2017. arXiv
39. R. Qian, R. T. Tan, W. Yang, J. Su and J. Liu, "Attentive Generative Adversarial Network for Raindrop Removal from a Single Image," in *Proc. IEEE/CVF CVPR*, 2018. arXiv
40. X. Li, Y. Jin, X. Jin, Z. Wu, B. Li, Y. Wang et al., "NTIRE 2025 Challenge on Day and Night Raindrop Removal for Dual-Focused Images: Methods and Results," in *Proc. IEEE/CVF CVPR Workshops*, 2025. arXiv
41. C. Wei, W. Wang, W. Yang and J. Liu, "Deep Retinex Decomposition for Low-Light Enhancement," in *Proc. BMVC*, 2018. arXiv
42. Y. Cai, H. Bian, J. Lin, H. Wang, R. Timofte and Y. Zhang, "Retinexformer: One-stage Retinex-based Transformer for Low-light Image Enhancement," in *Proc. IEEE/CVF ICCV*, 2023. arXiv
43. H. Zhang, H. Xu, X. Tian, J. Jiang and J. Ma, "Image Fusion Meets Deep Learning: A Survey and Perspective," *Information Fusion*, vol. 76, pp. 323–336, 2021. doi
44. S. Nah, T. H. Kim and K. M. Lee, "Deep Multi-scale Convolutional Neural Network for Dynamic Scene Deblurring," in *Proc. IEEE CVPR*, 2017. doi ·arXiv
45. K. He, X. Chen, S. Xie, Y. Li, P. Dollár and R. Girshick, "Masked Autoencoders Are Scalable Vision Learners," in *Proc. IEEE/CVF CVPR*, 2022. arXiv
46. R. Suvorov, E. Logacheva, A. Mashikhin, A. Remizova, A. Ashukha, A. Silvestrov, N. Kong, H. Goka, K. Park and V. Lempitsky, "Resolution-robust Large Mask Inpainting with Fourier Convolutions," in *Proc. IEEE/CVF WACV*, 2022. arXiv
47. S. Sengupta, V. Jayaram, B. Curless, S. Seitz and I. Kemelmacher-Shlizerman, "Background Matting: The World is Your Green Screen," in *Proc. IEEE/CVF CVPR*, 2020. arXiv
48. O. Ronneberger, P. Fischer and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Proc. MICCAI*, 2015. arXiv

49. H. Guo, J. Li, T. Dai, Z. Ouyang, X. Ren and S.-T. Xia, "MambaIR: A Simple Baseline for Image Restoration with State-Space Model," in *Proc. ECCV*, 2024. arXiv
50. H. Guo, Y. Guo, Y. Zha, Y. Zhang, W. Li, T. Dai, S.-T. Xia and Y. Li, "MambaIRv2: Attentive State Space Restoration," in *Proc. IEEE/CVF CVPR*, 2025. arXiv
51. B. Li, X. Liu, P. Hu, Z. Wu, J. Lv and X. Peng, "All-In-One Image Restoration for Unknown Corruption," in *Proc. IEEE/CVF CVPR*, 2022. code & paper
52. J. M. J. Valanarasu, R. Yasarla and V. M. Patel, "TransWeather: Transformer-based Restoration of Images Degraded by Adverse Weather Conditions," in *Proc. IEEE/CVF CVPR*, 2022. arXiv
53. V. Potlapalli, S. W. Zamir, S. Khan and F. S. Khan, "PromptIR: Prompting for All-in-One Image Restoration," in *Proc. NeurIPS*, 2023. arXiv
54. Y. Cui, S. W. Zamir, S. Khan, A. Knoll, M. Shah and F. S. Khan, "AdaIR: Adaptive All-in-One Image Restoration via Frequency Mining and Modulation," in *Proc. ICLR*, 2025. arXiv
55. Z. Luo, F. K. Gustafsson, Z. Zhao, J. Sjölund and T. B. Schön, "Controlling Vision-Language Models for Multi-Task Image Restoration," in *Proc. ICLR*, 2024. arXiv
56. M. V. Conde, G. Geigle and R. Timofte, "InstructIR: High-Quality Image Restoration Following Human Instructions," in *Proc. ECCV*, 2024. arXiv
57. H. Zhao, O. Gallo, I. Frosio and J. Kautz, "Loss Functions for Image Restoration with Neural Networks," *IEEE Transactions on Computational Imaging*, 2017 (arXiv title: "Loss Functions for Neural Networks for Image Processing"). arXiv
58. J. Johnson, A. Alahi and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *Proc. ECCV*, 2016. doi ·arXiv
59. L. Jiang, B. Dai, W. Wu and C. C. Loy, "Focal Frequency Loss for Image Reconstruction and Synthesis," in *Proc. IEEE/CVF ICCV*, 2021. arXiv
60. Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004. doi
61. R. Zhang, P. Isola, A. A. Efros, E. Shechtman and O. Wang, "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *Proc. IEEE/CVF CVPR*, 2018. arXiv
62. A. Mittal, R. Soundararajan and A. C. Bovik, "Making a 'Completely Blind' Image Quality Analyzer," *IEEE Signal Processing Letters*, 2013. doi
63. Y. Blau, R. Mechrez, R. Timofte, T. Michaeli and L. Zelnik-Manor, "The 2018 PIRM Challenge on Perceptual Image Super-resolution," in *Proc. ECCV Workshops*, 2018. arXiv
64. M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler and S. Hochreiter, "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *Proc. NeurIPS*, 2017. arXiv
65. Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *Proc. IEEE/CVF CVPR*, 2018. arXiv
66. Z. Wang, J. Chen and S. C. H. Hoi, "Deep Learning for Image Super-Resolution: A Survey," *IEEE TPAMI*, 2021. arXiv
67. C. Tian, L. Fei, W. Zheng, Y. Xu, W. Zuo and C.-W. Lin, "Deep Learning on Image Denoising: An Overview," *Neural Networks*, vol. 131, pp. 251–275, 2020. arXiv
68. M. Elad, B. Kawar and G. Vaksman, "Image Denoising: The Deep Learning Revolution and Beyond — A Survey Paper," *SIAM Journal on Imaging Sciences*, vol. 16, no. 3, pp. 1594–1654, 2023. doi ·arXiv
69. K. Zhang, W. Ren, W. Luo, W.-S. Lai, B. Stenger, M.-H. Yang and H. Li, "Deep Image Deblurring:

A Survey," *International Journal of Computer Vision*, 2022. arXiv
70. J. Gui, X. Cong, Y. Cao, W. Ren, J. Zhang, J. Zhang, J. Cao and D. Tao, "A Comprehensive Survey and Taxonomy on Single Image Dehazing Based on Deep Learning," *ACM Computing Surveys*, 2023. doi ·arXiv
71. W. Yang, R. T. Tan, S. Wang, Y. Fang and J. Liu, "Single Image Deraining: From Model-Based to Data-Driven and Beyond," *IEEE TPAMI*, 2021. doi ·arXiv
72. J. Jiang, Z. Zuo, G. Wu, K. Jiang and X. Liu, "A Survey on All-in-One Image Restoration: Taxonomy, Evaluation and Future Trends," *IEEE TPAMI*, 2025. arXiv
73. X. Li, Y. Ren, X. Jin, C. Lan, X. Wang, W. Zeng, X. Wang and Z. Chen, "Diffusion Models for Image Restoration and Enhancement: A Comprehensive Survey," *International Journal of Computer Vision*, 2025. arXiv
74. A. Levin, Y. Weiss, F. Durand and W. T. Freeman, "Understanding and Evaluating Blind Deconvolution Algorithms," in *Proc. IEEE CVPR*, 2009, pp. 1964–1971. doi
75. R. Fergus, B. Singh, A. Hertzmann, S. T. Roweis and W. T. Freeman, "Removing Camera Shake from a Single Photograph," *ACM Transactions on Graphics* (Proc. SIGGRAPH), vol. 25, no. 3, pp. 787–794, 2006. doi
76. Q. Shan, J. Jia and A. Agarwala, "High-quality Motion Deblurring from a Single Image," *ACM Transactions on Graphics* (Proc. SIGGRAPH), vol. 27, no. 3, 2008. doi
77. S. Cho and S. Lee, "Fast Motion Deblurring," *ACM Transactions on Graphics* (Proc. SIGGRAPH Asia), vol. 28, no. 5, 2009. doi
78. L. Xu and J. Jia, "Two-Phase Kernel Estimation for Robust Motion Deblurring," in *Proc. ECCV*, 2010, pp. 157–170. doi
79. D. Krishnan, T. Tay and R. Fergus, "Blind Deconvolution Using a Normalized Sparsity Measure," in *Proc. IEEE CVPR*, 2011, pp. 233–240. doi
80. J. Pan, D. Sun, H. Pfister and M.-H. Yang, "Blind Image Deblurring Using Dark Channel Prior," in *Proc. IEEE CVPR*, 2016, pp. 1628–1636. doi
81. J. Sun, W. Cao, Z. Xu and J. Ponce, "Learning a Convolutional Neural Network for Non-uniform Motion Blur Removal," in *Proc. IEEE CVPR*, 2015. arXiv
82. C. J. Schuler, M. Hirsch, S. Harmeling and B. Schölkopf, "Learning to Deblur," *IEEE TPAMI*, vol. 38, no. 7, pp. 1439–1451, 2016. doi ·arXiv
83. A. Chakrabarti, "A Neural Approach to Blind Motion Deblurring," in *Proc. ECCV*, 2016. arXiv
84. D. Ulyanov, A. Vedaldi and V. Lempitsky, "Deep Image Prior," in *Proc. IEEE/CVF CVPR*, 2018, pp. 9446–9454. doi ·arXiv
85. D. Ren, K. Zhang, Q. Wang, Q. Hu and W. Zuo, "Neural Blind Deconvolution Using Deep Priors," in *Proc. IEEE/CVF CVPR*, 2020. arXiv
86. O. Kupyn, V. Budzan, M. Mykhailych, D. Mishkin and J. Matas, "DeblurGAN: Blind Motion Deblurring Using Conditional Adversarial Networks," in *Proc. IEEE/CVF CVPR*, 2018. arXiv
87. K. Zhang, W. Zuo and L. Zhang, "Learning a Single Convolutional Super-Resolution Network for Multiple Degradations," in *Proc. IEEE/CVF CVPR*, 2018. arXiv
88. J. Gu, H. Lu, W. Zuo and C. Dong, "Blind Super-Resolution With Iterative Kernel Correction," in *Proc. IEEE/CVF CVPR*, 2019. arXiv
89. S. Bell-Kligler, A. Shocher and M. Irani, "Blind Super-Resolution Kernel Estimation using an Internal-GAN," in *Proc. NeurIPS*, 2019. link ·arXiv
90. Z. Luo, Y. Huang, S. Li, L. Wang and T. Tan, "Unfolding the Alternating Optimization for Blind

Super Resolution," in *Proc. NeurIPS*, 2020. arXiv
91. K. Zhang, L. Van Gool and R. Timofte, "Deep Unfolding Network for Image Super-Resolution," in *Proc. IEEE/CVF CVPR*, 2020. arXiv
92. K. Zhang, J. Liang, L. Van Gool and R. Timofte, "Designing a Practical Degradation Model for Deep Blind Image Super-Resolution," in *Proc. IEEE/CVF ICCV*, 2021. arXiv
93. A. Liu, Y. Liu, J. Gu, Y. Qiao and C. Dong, "Blind Image Super-Resolution: A Survey and Beyond," *IEEE TPAMI*, 2022 (early access). doi ·arXiv
94. A. Mittal, A. K. Moorthy and A. C. Bovik, "No-Reference Image Quality Assessment in the Spatial Domain," *IEEE Transactions on Image Processing*, vol. 21, no. 12, pp. 4695–4708, 2012. doi
95. C. Ma, C.-Y. Yang, X. Yang and M.-H. Yang, "Learning a No-Reference Quality Metric for Single-Image Super-Resolution," *Computer Vision and Image Understanding*, vol. 158, pp. 1–16, 2017. doi ·arXiv
96. N. Venkatanath, D. Praneeth, M. C. Bh, S. S. Channappayya and S. S. Medasani, "Blind Image Quality Evaluation Using Perception Based Features," in *Proc. Twenty First National Conference on Communications (NCC)*, 2015, pp. 1–6. doi ·MATLAB documentation
97. H. Talebi and P. Milanfar, "NIMA: Neural Image Assessment," *IEEE Transactions on Image Processing*, vol. 27, no. 8, pp. 3998–4011, 2018. doi ·arXiv
98. S. Su, Q. Yan, Y. Zhu, C. Zhang, X. Ge, J. Sun and Y. Zhang, "Blindly Assess Image Quality in the Wild Guided by a Self-Adaptive Hyper Network," in *Proc. IEEE/CVF CVPR*, 2020, pp. 3664–3673. doi
99. J. Ke, Q. Wang, Y. Wang, P. Milanfar and F. Yang, "MUSIQ: Multi-scale Image Quality Transformer," in *Proc. IEEE/CVF ICCV*, 2021, pp. 5128–5137. doi ·arXiv
100. S. Yang, T. Wu, S. Shi, S. Lao, Y. Gong, M. Cao, J. Wang and Y. Yang, "MANIQA: Multi-dimension Attention Network for No-Reference Image Quality Assessment," in *Proc. IEEE/CVF CVPR Workshops*, 2022, pp. 1190–1199. doi ·arXiv
101. C. Chen, J. Mo, J. Hou, H. Wu, L. Liao, W. Sun, Q. Yan and W. Lin, "TOPIQ: A Top-Down Approach From Semantics to Distortions for Image Quality Assessment," *IEEE Transactions on Image Processing*, vol. 33, pp. 2404–2418, 2024. doi ·arXiv
102. J. Wang, K. C. K. Chan and C. C. Loy, "Exploring CLIP for Assessing the Look and Feel of Images," in *Proc. AAAI Conference on Artificial Intelligence*, 2023. arXiv
103. H. Wu, Z. Zhang, W. Zhang, C. Chen, L. Liao, C. Li, Y. Gao, A. Wang, E. Zhang, W. Sun, Q. Yan, X. Min, G. Zhai and W. Lin, "Q-Align: Teaching LMMs for Visual Scoring via Discrete Text-Defined Levels," in *Proc. ICML*, PMLR 235, pp. 54015–54029, 2024. link ·arXiv
104. H. R. Sheikh, M. F. Sabir and A. C. Bovik, "A Statistical Evaluation of Recent Full Reference Image Quality Assessment Algorithms," *IEEE Transactions on Image Processing*, vol. 15, no. 11, pp. 3440–3451, 2006 (LIVE Image Quality Assessment Database). doi ·database
105. N. Ponomarenko, L. Jin, O. Ieremeiev, V. Lukin, K. Egiazarian, J. Astola, B. Vozel, K. Chehdi, M. Carli, F. Battisti and C.-C. J. Kuo, "Image Database TID2013: Peculiarities, Results and Perspectives," *Signal Processing: Image Communication*, vol. 30, pp. 57–77, 2015. doi ·database
106. H. Lin, V. Hosu and D. Saupe, "KADID-10k: A Large-scale Artificially Distorted IQA Database," in *Proc. Eleventh International Conference on Quality of Multimedia Experience (QoMEX)*, 2019. doi ·database
107. V. Hosu, H. Lin, T. Sziranyi and D. Saupe, "KonIQ-10k: An Ecologically Valid Database for

Deep Learning of Blind Image Quality Assessment," *IEEE Transactions on Image Processing*, vol. 29, pp. 4041–4056, 2020. arXiv

108. Y. Fang, H. Zhu, Y. Zeng, K. Ma and Z. Wang, "Perceptual Quality Assessment of Smartphone Photography," in *Proc. IEEE/CVF CVPR*, 2020, pp. 3674–3683. doi
109. J. Gu, H. Cai, H. Chen, X. Ye, J. Ren and C. Dong, "PIPAL: A Large-Scale Image Quality Assessment Dataset for Perceptual Image Restoration," in *Proc. ECCV*, 2020. arXiv
110. C. Chen and J. Mo, "IQA-PyTorch: PyTorch Toolbox for Image Quality Assessment," software (pip package `pyiqa`), 2022. GitHub

DLCV 05・低光影像增強

從直方圖等化、Retinex 到曲線估計、Retinex Transformer 與擴散模型：如何讓暗照片變亮，又不放大雜訊、不造成色偏、也不被評估指標騙了。

網站版本: https://blog.shyandram.dev/zh-tw/learn/deep-learning/dl-05-low-light-enhancement/

你將學到

- 為什麼低光影像增強（low-light image enhancement, LLIE）其實是兩個問題的合體：**色調強化**與**細節（含雜訊）還原**，以及為什麼「直接調亮」行不通。
- 傳統工具箱如何運作：HSV、HSI、YCbCr、Lab 中的亮度通道；直方圖等化與 CLAHE；gamma 曲線；Retinex（SSR、MSR、LIME）；以及「反轉後去霧」的技巧。
- 深度 LLIE 方法如何依**訓練訊號**分類（監督式、展開式、非配對、零參考、生成式），並推導 Retinex-Net 分解損失函數與 Zero-DCE 曲線的數學。
- 如何誠實地評估增強結果：全參考指標、無參考指標（NIQE、BRISQUE、PI、明度順序誤差、學習式評估器）與使用者研究、**GT-mean** 陷阱，以及以任務為導向的評估。
- 常用資料集與挑戰賽（LOL、LOL-v2、LSRW、SDSD、SID、FiveK、ExDark、NTIRE 2024/2025）各自用來做什麼，以及還有哪些問題沒解決。

先看全貌

在暗處拍到的照片，並不只是「把亮度調低的正常照片」。感測器收到的光子非常少，所以影像又暗、又有雜訊，還常常帶著色偏。低光影像增強的目標，是產生「如果當時光線充足，你會拍到的那張照片」：曝光正確、顏色自然、細節乾淨，而且不引入新的假影。

> **用最簡單的話說**
>
> 想像你在幾乎關燈的房間裡畫畫。畫出來又淡又髒，你也搞不清楚哪一筆是哪個顏色。事後開燈、把這張畫「影印得亮一點」是沒用的：髒污也跟著變亮，畫錯的顏色還是錯的。好的低光增強器，就像一位朋友看著你那張淡淡的畫，把它重新畫一遍，畫成白天看起來的樣子：同一張圖、正確的顏色，而且沒有髒污。

為什麼重要：手機每天都在暗處拍照，車輛與機器人必須在夜間看得見，監視系統、科學影像與醫療內視鏡也都在跟同樣的物理限制奮戰。低光也會傷害下游模型：用白天照片訓練的偵測器，到了晚上準確率會下降。因此 LLIE 既是畫質問題，也是電腦視覺的強健性問題。

> **關於本篇**
>
> 本篇由作者先前的 LLIE 文章 [1] 擴寫而成（原文在此），依照原文的大綱，補上數學、程式碼、圖表與文獻綜述。

目的與難點

白話版。我們想把一張暗的影像對應成一張曝光正常的影像。難的地方在於「變亮」和「修復」是兩件不同的工作：變亮改變的是色調（每個區域多亮多暗），修復則是找回被黑暗與雜訊破壞的細節與乾淨的顏色。

精確版。 LLIE 和影像轉換（image-to-image translation）不同：後者會改變影像的屬性（夏天變冬天、照片變畫作），LLIE 則應該完整保留場景與內容，只把光線不足造成的破壞還原。原文 [1] 把這些破壞分成兩個難點：

1. **色調強化**。在全域與局部提高曝光，同時不壓垮亮部、也不把對比壓平。燈具這類亮區必須維持明亮，暗區要被打開，而且區域之間的明暗順序應該保留。
2. **細節還原**。找回埋在雜訊與量化誤差裡的紋理與顏色，同時抑制雜訊而不是放大它。

在深度學習裡，只要網路被訓練成輸出乾淨的影像，細節還原本來就是它在做的事，因此多數方法的設計重點放在色調（要把哪些影像性質或物理模型放進網路）與學習機制（用什麼監督方式與損失函數），而不是另外設計一個專門的細節模組。

為什麼不是單純調亮

線性增益 $y = c\,x$（$c > 1$）會在四個地方失敗，全都可以在圖 1 看到：

- **雜訊**。光子到達是隨機的。光子很少時，訊噪比很低，而增益會把訊號和雜訊一起放大。影像變亮了，卻沒有變乾淨。
- **色偏**。低光環境的光常帶有顏色（鎢絲燈、鈉燈路燈），相機的白平衡與三個色彩通道的雜訊程度也不同。三通道等比例放大會保留色偏；分別放大又可能產生奇怪的顏色。
- **飽和與過曝**。已經接近上限的像素（燈、窗戶、螢幕）會在 1 被截斷。大到足以照亮暗部的全域增益，會毀掉亮部。
- **量化**。一張暗的 8 位元影像可能只用到 20 到 40 個灰階。把它們拉開，就會出現明顯的色階斷層（banding）。

低光拍攝的模型

明確寫出退化模型很有幫助，一方面便於理解，另一方面可用來**合成訓練資料**。請在線性光（數值正比於光子數）上操作，而不是在經過 gamma 編碼的像素上（gamma 請見IP 03）。常用的是 Foi 等人的 Poisson–Gaussian 模型 [7]：

$$z = \frac{1}{\eta}\,\mathcal{P}(\eta\,k\,x) + n, \qquad n \sim \mathcal{N}(0, \sigma_r^2),$$

其中 $x \in [0,1]$ 是乾淨的線性影像，$k \in (0,1]$ 是到達感測器的光量比例，η 是把強度換算成光子數的尺度（η 越大代表光子越多、相對雜訊越小），$\mathcal{P}(\cdot)$ 是模擬**散粒雜訊**（shot noise）的 Poisson 隨機變數，n 是標準差 σ_r 的高斯**讀出雜訊**（read noise）。之後相機再套上色調曲線並量化。z 的變異數約為 $kx/\eta + \sigma_r^2$，會隨訊號增加，也就是說雜訊是與訊號相關的；這就是為什麼「加高斯雜訊」很難代表真實的低光。

```
import numpy as np
from skimage import data, img_as_float

rng = np.random.default_rng(0)
clean = img_as_float(data.coffee())            # H x W x 3, sRGB-encoded, in [0, 1]

def synth_low_light(x, k=0.05, peak=2000.0, read_std=8e-4, gamma=2.2):
    lin = k * x ** gamma                           # 1) less light, in linear units
    lin = rng.poisson(lin * peak) / peak           # 2) shot noise (Poisson)
    lin = lin + rng.normal(0, read_std, lin.shape) # 3) read noise (Gaussian)
    out = np.clip(lin, 0, 1) ** (1 / gamma)        # 4) re-encode for display
    return np.round(out * 255) / 255               # 5) 8-bit quantization

dark = synth_low_light(clean)
print(clean.mean(), dark.mean())                   # about 0.39 -> 0.10
```

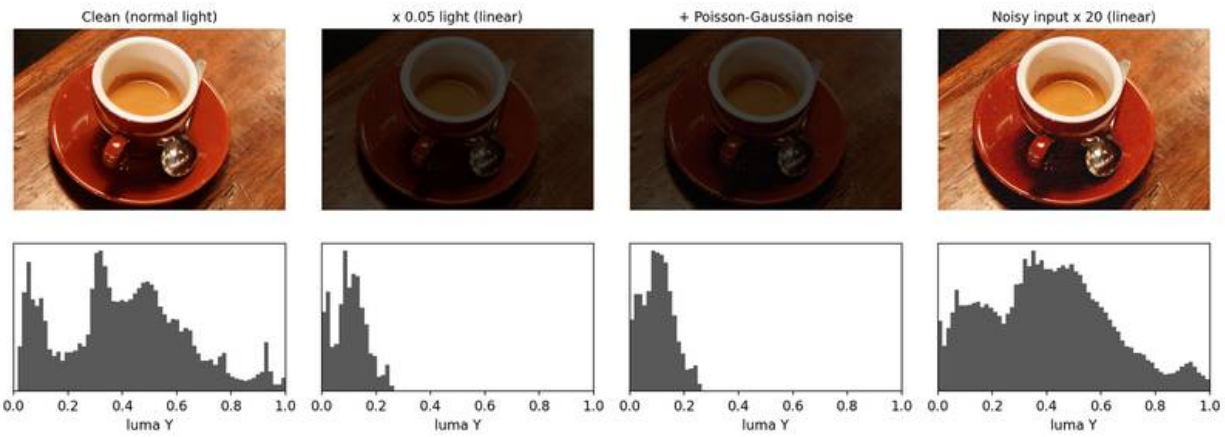


圖 *1* — 合成的低光影像。拿掉 *95%* 的光，直方圖被擠進最下方四分之一。在暗影像裡很難看到雜訊，但用完美的增益把曝光還原後（最右），雜訊就現形了：調亮與去噪是兩個獨立的問題。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

最右邊那張是本篇最重要的一張圖：即使是理想的亮度校正，雜訊依然存在。這就是為什麼 LLIE 是一個復原問題（請見DLCV 04・影像復原），而不只是色調對應問題。

非深度學習的 LLIE 與低光影像的特性

白話版。在深度學習之前，主要有兩大流派：**直方圖**類方法重新分配灰階；**Retinex** 類方法把影像拆成「光照」和「物體」兩部分，再修正光照。第三種想法借自去霧，可以歸進 Retinex 家族；統計方法則延伸了以上各類。

低光影像在色彩空間上的特性

許多色彩空間會把「有多亮」的通道和「是什麼顏色」的通道分開。不同空間稱它為明度（value）、強度（intensity）、亮度（luma）或明亮度（lightness），定義也各不相同。設 $R, G, B \in [0, 1]$（色彩空間本身請見IP 06）：

$$\begin{aligned} \text{HSV:} \quad & V = \max(R, G, B), \\ \text{HSI:} \quad & I = \tfrac{1}{3}(R + G + B), \\ \text{YCbCr (BT.601):} \quad & Y' = 0.299\,R' + 0.587\,G' + 0.114\,B', \\ \text{CIELAB:} \quad & L^* = 116\,f(Y_{\text{lin}}/Y_n) - 16. \end{aligned}$$

其中 R', G', B' 是經 gamma 編碼的值；Y_{lin} 是線性光的亮度，Y_n 是參考白的亮度；$f(t)$ 在一個小門檻以上為 $t^{1/3}$，以下為線性。幾點說明：

- **V 就是「亮通道」（bright channel）**。它是逐像素的最大值，與去霧中使用的暗通道（dark channel，逐區塊的最小值）[14] 相對。在暗影像中，V 是被黑暗破壞最少的通道，這也是 LIME [17] 用它來初始化光照的原因。
- **亮度權重**。 ITU-R BT.601 [8] 定義亮度為 $Y' = 0.299R' + 0.587G' + 0.114B'$，這個全範圍（full range）形式就是 JPEG/JFIF 使用的版本。8 位元的有限範圍（limited range，又稱 studio range）影片中，BT.601 把黑色放在 16、白色放在 235：$Y = 16 + 219\,Y'$。若 $R', G', B' \in [0, 255]$，就變成 $Y = 16 + 0.257R' + 0.504G' + 0.098B'$，因為 $219/255 \times (0.299, 0.587, 0.114) \approx (0.257, 0.504, 0.098)$。兩個公式描述的是同一個亮度，只差在範圍與偏移量。
- L^* 被設計成感知上均勻，相同的數值差在人眼看來差不多大，因此很適合拿來做對比增強。

實務上的結論是：既然亮度可以和顏色拆開，就可以**只增強亮度通道**並保留色度。圖 2 顯示，選哪個通道影響很大。

```
R, G, B = dark[..., 0], dark[..., 1], dark[..., 2]
V = dark.max(axis=2)                          # HSV value
I = dark.mean(axis=2)                         # HSI intensity
Y = 0.299 * R + 0.587 * G + 0.114 * B         # BT.601 luma, full range (JPEG/JFIF)
Y_limited = 16 + 219 * Y                      # 8-bit limited ("studio") range, 16..235
print(np.allclose(Y_limited, 16 + (65.481 * R + 128.553 * G + 24.966 * B)))  # True
```

圖 2 — 同一條 *gamma* 曲線套用在不同色彩空間。提高 V 或 I 等於三個通道乘上同一個倍數，色相不變但飽和度增強（非常橘）；固定色度只提高 Y 或 L^*，結果則會褪色。兩者都不是「正確答案」：色調與色彩是耦合的。

圖中可以看到兩種效應。以共同倍數縮放 RGB（HSV、HSI）會保留色相與通道比例，於是本來就飽和的物體變得非常飽和；只改 Y' 或 L^*、固定 Cb/Cr 或 a*/b*，則是只加亮度不加顏色，影像看起來就褪色了。色調與色彩的這種耦合，正是後來 HVI-CIDNet [27] 想用學習出來的色彩空間處理的問題。

直方圖等化與 CLAHE

白話版。直方圖等化（histogram equalization, HE）把影像的灰階攤開，讓每個亮度值被使用的機會差不多。暗影像的值擠在 0 附近，等化後就會被拉滿整個範圍。

精確版。對 L 個灰階、正規化直方圖 $p_r(r_k) = n_k/(MN)$ 的影像，HE 以縮放後的累積分布函數（CDF）作為對應：

$$s_k = T(r_k) = (L-1)\sum_{j=0}^{k} p_r(r_j), \qquad k = 0, \dots, L-1,$$

其中 n_k 是灰階 r_k 的像素數，MN 是影像大小。T 的斜率正比於直方圖，所以像素很多的灰階會被拉開。推導請見IP 03。對彩色影像，常見做法是先轉到有亮度通道的色彩空間，只等化該通道，再轉回來，例如 RGB → Lab → 等化 L^* → RGB。

全域 HE 在低光下有兩個問題：它會過度拉伸大片平坦區域（連同其中的雜訊），而且單一對應無法同時照顧暗角與明亮的燈。**自適應 HE**（adaptive HE）為每個區塊計算對應；**CLAHE**（contrast-limited AHE）在計算 CDF 前先把每個區塊的直方圖在某個上限截斷，藉此限制 T 的斜率，也就限制了雜訊增益，最後在相鄰區塊間內插以避免接縫 [9], [10]。

Gamma 校正

冪次律 $s = r^\gamma$ 在 $\gamma < 1$ 時對暗部的提升大於亮部；它的斜率 $\gamma r^{\gamma-1}$ 在 $r = 0$ 附近很大。它便宜又單調，但屬於全域操作：整張圖只有一個 γ、不會去噪，而且依套用的空間不同，同樣面臨上述「褪色或過飽和」的選擇。

```
from skimage import color, exposure

gamma_out = dark ** 0.4                                    # power-law curve on every
 ↪  channel
lab = color.rgb2lab(dark)
L = lab[..., 0] / 100                                      # L* scaled to [0, 1]
he = lab.copy();    he[..., 0] = 100 * exposure.equalize_hist(L)
clahe = lab.copy(); clahe[..., 0] = 100 * exposure.equalize_adapthist(L, clip_limit=0.01)
he_out, clahe_out = color.lab2rgb(he), color.lab2rgb(clahe)
for name, im in [("gamma", gamma_out), ("HE", he_out), ("CLAHE", clahe_out)]:
    print(f"{name:6s} mean {im.mean():.3f}   std {im.std():.3f}")
```

圖 3 — 傳統增強方法作用在圖 *1* 的有雜訊輸入上。全部都變亮了，但沒有一個去除雜訊；在輸入最暗的地方，*Retinex* 的輸出出現強烈的彩色雜訊。

Retinex 理論

白話版。 Retinex 認為，眼睛（或相機）接收到的影像是「物體本身的顏色」乘上「照在它上面的光」。如果能估計出光，就能把它除掉，看到物體真正的樣子，再換上一個更好的光。

精確版。 Land 的 Retinex 理論 [11] 把觀察到的影像 S 表示為

$$S(\mathbf{x}) = R(\mathbf{x}) \circ L(\mathbf{x}),$$

其中 $\mathbf{x}$ 是像素位置，$R \in [0,1]$ 是**反射率**（reflectance，物體表面的性質，假設在任何光照下都不變），L 是**照度**（illumination，照在場景上的光，除了陰影邊界之外在空間上平滑變化），$\circ$ 是逐元素相乘。請注意用詞：L 是照度，不是 luminance；luminance 通常指色彩空間裡的亮度通道，例如上面的 Y。

從一個乘積解出兩個未知數是病態問題（ill-posed），因此每個 Retinex 方法都會加上先驗。最經典的是「照度是平滑的」。

單尺度 Retinex（SSR） [12] 用高斯模糊 G_σ 估計 L，並在對數域運算，讓乘法變成加法：

$$\log R_c(\mathbf{x}) = \log S_c(\mathbf{x}) - \log\left(G_\sigma * S_c\right)(\mathbf{x}),$$

對每個色彩通道 c 計算，$*$ 為卷積，σ 為周圍（surround）尺度。σ 小，局部對比強但邊緣出現光暈（halo）；σ 大，色調較好但細節較弱。**多尺度 Retinex（MSR）** [13] 以權重 w_n 平均多個尺度（通常三個）的 SSR 輸出：

$$\log R_c^{\mathrm{MSR}} = \sum_{n=1}^{N} w_n \left[\log S_c - \log(G_{\sigma_n} * S_c)\right].$$

由於 SSR 與 MSR 逐通道運算，顏色容易變灰。**MSRCR**（帶色彩還原的 MSR）[13] 將結果乘上一個色彩還原因子，依據是各通道佔 $R+G+B$ 的比例；其他變體則只對強度通道做 MSR，並保留輸入的色度。

```
from scipy import ndimage as ndi

def ssr(img, sigma=40.0, eps=1e-3):
    log_s = np.log(img + eps)
    log_l = np.log(ndi.gaussian_filter(img, sigma=(sigma, sigma, 0)) + eps)  # L ~ Gaussian
     ↪  blur
    r = log_s - log_l                                          # log R = log S - log L
    lo, hi = np.percentile(r, [1, 99])                         # robust stretch for display
    return np.clip((r - lo) / (hi - lo), 0, 1)

msr = np.mean([ssr(dark, s) for s in (15, 80, 250)], axis=0)   # MSR, equal weights
 ↪  (simplified)
```

做增強時，我們通常不只要 R，因為單獨的 R 看起來平板又不自然。我們要的是把 L 調亮再組合回去：$\hat{S} = R \circ \hat{L}$，例如 $\hat{L} = L^{\gamma}$、$\gamma < 1$。這個「分解、調整 L、重組」的模式，就是大多數 Retinex 深度網路的範本。

圖 4 — *LIME* 風格的 *Retinex* 分解。照度以 $max(R, G, B)$ 初始化後再平滑；反射率為 S / L。把 L 以 *gamma 0.35* 調亮再乘回去，得到自然的結果；單獨的 R 則顯得平板而過度飽和。

LIME：把 Retinex 寫成最佳化問題

LIME [17] 只估計照度。它以亮通道 $\hat{T}(\mathbf{x}) = \max_c S_c(\mathbf{x})$ 初始化，再解一個保留結構的平滑問題來精修：讓 T 接近 $\hat{T}$，但除了影像有強結構的地方之外都要平滑。增強後的影像為 S/T^{γ}；由於除法會放大雜訊，LIME 可以選擇在之後去噪。LIME 論文的影像（常稱 LIME 資料集）成為標準測試集，而這個方法正是傳統配方的好例子：一個物理模型、一個手工設計的先驗、再加上最佳化。

另一個模型：對反轉影像去霧

Retinex 並不是唯一的成像模型。去霧使用的是**大氣散射模型**（atmospheric scattering model）：

$$I(\mathbf{x}) = J(\mathbf{x})\, t(\mathbf{x}) + A\,(1 - t(\mathbf{x})),$$

其中 I 是有霧的觀測影像，J 是無霧的場景輻射，$t \in (0, 1]$ 是透射率（場景光有多少能留下來），A 是全域大氣光。這個模型不能直接套用在暗影像上，因為暗影像看起來並不像有霧的影像。

Dong 等人觀察到，**反轉**後的低光影像 $1 - S$ 確實像有霧：暗區變成明亮、低對比的區域，就像霧一樣。因此他們先反轉影像，套用去霧演算法（精神上與暗通道先驗 [14] 相同），再把結果反轉回來 [15], [16]。以上面的符號表示：

$$I_{\text{haze}} = 1 - S_{\text{Retinex}}, \qquad J_{\text{haze}} = 1 - \hat{S}_{\text{enhanced}}.$$

從「哪些資訊可靠」的角度看：去霧假設暗通道代表無霧的內容；但在低光影像裡，暗通道大多是被破壞的像素，所以重點轉向亮通道，也就是受黑暗影響最少的區域。LIME 論文 [17] 分析了這種反轉做法，並說明它可以改寫成 Retinex 的形式，所以兩個模型本質上是同一個假設，只是 Retinex 的表述較簡單。

統計方法

許多傳統方法在這些模型之上再加入統計量或先驗：保持場景明暗順序的自然度約束（NPE [18]）、分層或二維直方圖模型（DICM 資料集背後的方法 [59]），以及融合多張合成曝光。它們都是手工設計、不需要訓練資料，泛化行為可預期；弱點則是雜訊，通常被忽略或交給另一個去噪器處理。

深度學習 LLIE

白話版。深度網路從資料中學習「暗到亮」的對應。問題不在於要不要用網路，而在於**用什麼訓練訊號**：成對的暗/亮照片、非配對的照片、完全沒有參考影像，或者是一個描述「亮影像長什麼樣子」的生成模型。物理知識（通常是 Retinex）可以放進以上任何一種。

深度學習本身就是一個龐大的最佳化過程，而它的黑盒子特性讓我們能省掉許多手工假設。常見的第一種分法是**以理論為基礎**（通常是 Retinex）與**不以理論為基礎**（學習能力強的通用網路）。實務上兩者常交織在一起，所以不能用二元方式一刀切。圖 5 主要改以訓練訊號分組，參考了綜述論文 [2], [3] 的做法。

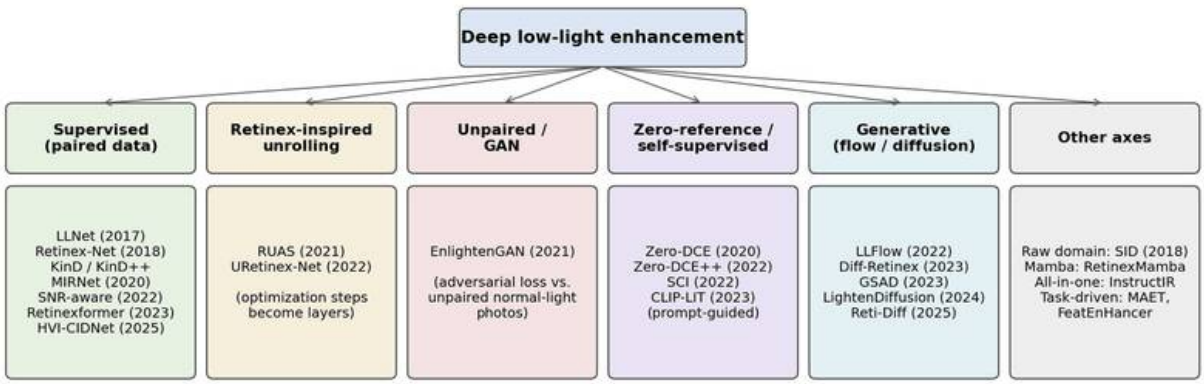


圖 5 — 主要依訓練訊號分類的深度 *LLIE*（*flow* ／擴散那一欄則是依模型家族）。*Retinex* 分解是一種架構上的選擇，出現在好幾欄裡，而不是獨立的一欄。

監督式方法（成對資料）

給定低光與正常光影像對 (x_i, y_i)，訓練 f_θ 最小化重建損失，通常是 $\mathcal{L} = \|f_\theta(x) - y\|_1$ 再加上感知或結構項。成對資料取得成本高（同一靜態場景拍兩種曝光，或用合成的），而且網路會學到參考影像特定的亮度與色彩風格，這一點在後面談評估時很重要。

- **LLNet** [19]（2017）是最早的深度 LLIE 模型之一：一個堆疊稀疏去噪自編碼器（stacked sparse denoising autoencoder），用人工變暗並加雜訊的小區塊訓練，同時學習調亮與去噪。
- **CNN + 亮通道先驗**（Tao 等人 [20]，2017）依序處理兩個難點：先用 CNN 去噪，再以亮通道驅動、類似大氣散射模型的方式增強對比，是把學習與上述反轉想法結合的早期例子。
- **MIRNet** [24]（2020）是一個通用的復原骨幹網路：保留一條全解析度的支線，並行多條低解析度支線、讓它們彼此交換資訊，再用 selective kernel 注意力融合；論文除了增強之外，也在去噪與超解析度上評估。
- **SNR-aware** [25]（2022）估計逐像素的訊噪比圖。訊噪比高的地方，局部卷積就夠了；訊噪比很低的地方，局部鄰域不可靠，模型改用長距離的 Transformer 注意力，從影像其他地方借用資訊。
- **HVI-CIDNet** [27]（2025）處理的是圖 2 的色彩空間問題：sRGB 對顏色非常敏感，容易產生色偏與亮度假影；HSV 解決了亮度問題，卻帶來紅色與黑色的雜訊假影。作者提出 HVI 色彩空間，由極化的色相/飽和度圖與可學習的強度構成，並搭配一個把色彩與強度解耦的網路。

以 Retinex 為基礎的網路

Retinex-Net [21]（2018）把目標從「估計 R」改成「**輸入與輸出各自有自己的 R 和 L**」。流程是：分解低光與正常光影像、把 L 增強成 $\hat{L}$、再與 R 重組。它也提出了 LOL 資料集（見後文）。它的分解網路沒有 R 或 L 的真值，只靠一致性來訓練，損失函數為

$$\mathcal{L}_{\text{decom}} = \mathcal{L}_{\text{recon}} + \lambda_{ir}\,\mathcal{L}_{ir} + \lambda_{is}\,\mathcal{L}_{is},$$

$$\mathcal{L}_{\text{recon}} = \sum_{i \in \{\text{low},\text{normal}\}} \sum_{j \in \{\text{low},\text{normal}\}} \lambda_{ij} \left\| R_i \circ I_j - S_j \right\|_1,$$

$$\mathcal{L}_{ir} = \|R_{\text{low}} - R_{\text{normal}}\|_1, \qquad \mathcal{L}_{is} = \sum_i \left\| \nabla I_i \circ \exp\!\left(-\lambda_g \nabla R_i\right) \right\|.$$

論文的記號是 $S = R \circ I$，所以這裡的 I 就是照度（我們記號中的 L）。S_j 是輸入影像 j，R_i 與 I_i 是影像 i 分解出的反射率與照度，∇ 是水平加垂直的梯度。各項的意義：

- $\mathcal{L}_{\text{recon}}$：每組分解都必須重建出自己的影像（$\lambda_{ii} = 1$），而且把兩張影像的反射率與照度交叉組合也應該大致成立（$i \neq j$ 時 $\lambda_{ij} = 0.001$）。
- $\mathcal{L}_{ir}$：兩張影像拍的是同一個場景，所以反射率應該相同；網路正是藉此學到「反射率」是什麼。
- $\mathcal{L}_{is}$：照度應該平滑，除了反射率有強邊緣的地方。權重 $\exp(-\lambda_g \nabla R)$ 在物體邊界上放寬平滑懲罰。這是 LIME 結構感知先驗的學習版。

論文使用 $\lambda_{ir} = 0.001$、$\lambda_{is} = 0.1$、$\lambda_g = 10$。增強網路則以 $\|R_{\text{low}} \circ \hat{I} - S_{\text{normal}}\|_1$ 加上同樣的平滑項訓練。

這個模式很快就流行起來，許多方法的名字都帶有「Retinex」：

- **KinD** [22]（2019）把工作拆成三個子網路：分解、反射率還原（因為暗影像的反射率帶有雜訊與色偏，所以放寬了「R 不變」的假設），以及可由使用者控制亮度倍率的照度調整。**KinD++** [23]（2021）進一步改良設計以減少視覺缺陷。
- **Retinexformer** [26]（2023）指出拍攝與調亮都會破壞分解出的成分，於是把模型改寫為帶擾動項的形式：$S = (R + \hat{R}) \circ (L + \hat{L})$，其中 $\hat{R}$ 與 $\hat{L}$ 模擬雜訊、假影與色彩失真。它採**單階段**訓練（取代 Retinex-Net 的多階段訓練），使用 L1 損失；它的**照度引導多頭自注意力**（illumination-guided multi-head self-attention）用照度估計來引導注意力，讓曝光程度不同的區域彼此交流。它後來成為常見的比較基準。
- **RetinexMamba** [42]（2024 年預印本）沿用 Retinex Transformer 的架構，把注意力換成 Mamba 狀態空間（state space）模組，其計算量隨像素數線性成長。「Retinex 分解＋新骨幹」多年來一直是很穩定的設計模式。

另一條較少見的路線是以大氣散射模型、而非單純的 Retinex 為基礎。CPGA-Net [28] 結合通道先驗與 gamma 校正，建構一個同時處理全域與局部資訊的輕量網路；CPGA-Net+ [29] 則重新檢視理論上的照度模型，以提升效率。這條研究路線也觀察到：不同功能模組學到的特徵，會對應到先驗的物理角色；這表示受模型引導的網路，並不是完全無法理解的黑盒子。

Retinex 啟發的展開式網路

展開（unrolling，或 unfolding）是把 Retinex 問題的迭代最佳化演算法，每一次迭代變成一個帶有可學習參數的網路階段。網路架構繼承了演算法的結構與可解釋性，而訓練取代了手工調整的先驗。

- **RUAS** [30]（2021）展開照度估計與去噪的最佳化模型，搜尋先驗模組的架構，並以協同式的無參考（reference-free）策略訓練，所以它同時也屬於非監督那一欄。它非常小也非常快。
- **URetinex-Net** [31]（2022）把 Retinex 分解的最佳化展開成網路，包含初始化、R 與 L 的展開更新，以及照度調整等學習模組。

非配對訓練：EnlightenGAN

EnlightenGAN [32]（2021）使用生成對抗網路（GAN），只需要一組暗影像和另一組不相關的明亮影像。生成器是 U-Net，並以輸入照度推得的自正則化注意力圖引導，產生增強影像。**全域**判別器評估整張影像，**局部**判別器評估隨機小區塊，讓輸出在整體與細節上都像光線充足的照片。由於沒有真值，**自特徵保持**（self-feature-preserving）損失比較輸入與輸出的 VGG 特徵，讓內容維持不變。

零參考方法與曲線估計

白話版。 Zero-DCE 不直接輸出新影像，而是替每個像素輸出一條調亮曲線，套用它，再用不需要任何參考照片的損失來評分：「曝光好不好？顏色平不平衡？曲線平不平滑？局部對比有沒有保留？」

LE 曲線。 Zero-DCE [33]（2020）定義了一條二次的**光增強曲線**（light-enhancement curve）

$$\mathrm{LE}(x;\alpha) = x + \alpha\, x\,(1-x), \qquad \alpha \in [-1,1],$$

其中 $x \in [0,1]$ 是像素值。這條曲線有三個好性質：$\mathrm{LE}(0)=0$ 且 $\mathrm{LE}(1)=1$，範圍不變、不需要截斷；當 $|\alpha| \le 1$ 時單調，因為 $\mathrm{d}\,\mathrm{LE}/\mathrm{d}x = 1+\alpha(1-2x) \ge 0$，明暗順序得以保留；而且可微分，能用反向傳播訓練。單一條二次曲線不夠靈活，所以要**迭代**套用：

$$\mathrm{LE}_n(\mathbf{x}) = \mathrm{LE}_{n-1}(\mathbf{x}) + \mathcal{A}_n(\mathbf{x})\,\mathrm{LE}_{n-1}(\mathbf{x})\big(1-\mathrm{LE}_{n-1}(\mathbf{x})\big),$$

其中 LE_0 是輸入，$n = 1,\dots,8$，而 $\mathcal{A}_n$ 是**逐像素、逐通道**的參數圖，而非單一的 α。一個七層的 CNN（DCE-Net，每層 32 個通道、約 79k 個參數、沒有降取樣）以 tanh 輸出 $8\times 3 = 24$ 張參數圖。每個像素因此都有自己的高階曲線，局部調整就是這樣從看似全域的公式中產生的。

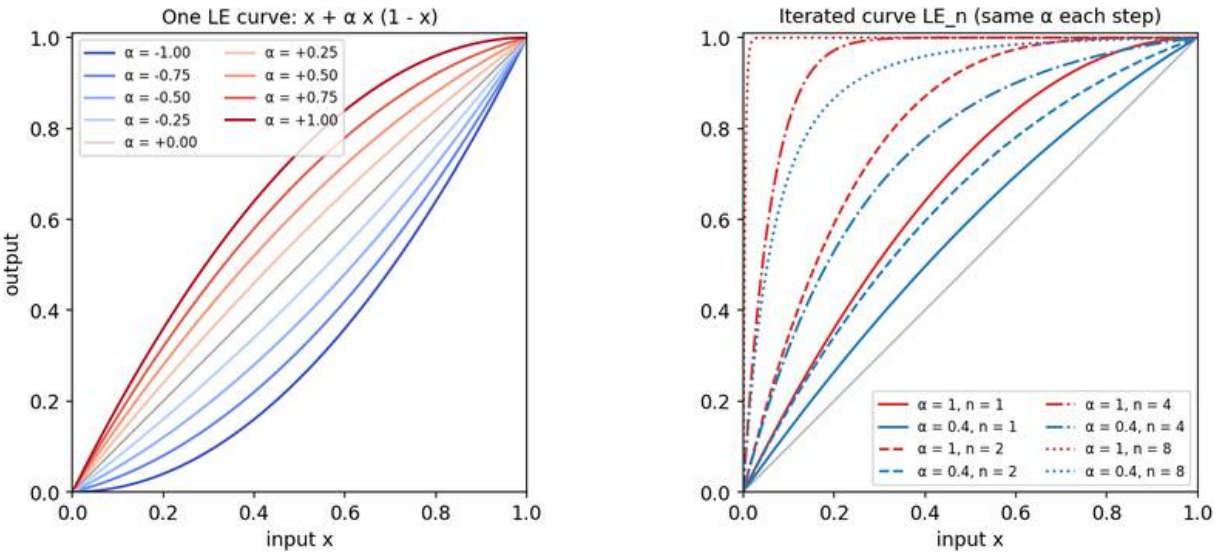


圖 *6 — LE 曲線。左：α 從 -1 到 1 的單次迭代。右：迭代讓曲線變陡；$\alpha=1$、$n=8$ 時，0.02 這個值已經被對應到將近 1，所以暗部裡微小的雜訊值會被極度放大。*

```
def le_iterate(x, alphas):
    """Zero-DCE curve: LE_n = LE_{n-1} + a_n * LE_{n-1} * (1 - LE_{n-1})."""
    y = x.copy()
    for a in alphas:              # a can be a scalar or a per-pixel map
        y = y + a * y * (1 - y)
    return y

x = np.array([0.0, 0.02, 0.1, 0.3, 0.6, 1.0])
print(le_iterate(x, [1.0] * 8).round(3))   # [0.  0.994 1.  1.  1.  1.]
print(le_iterate(x, [0.4] * 8).round(3))   # [0.  0.245 0.695 0.93 0.985 1.]
```

第一行輸出顯示了危險之處：對很小的 x，每一步大約乘上 $1+\alpha$，所以 $\alpha=1$ 迭代八次的增益接近 $2^8 = 256$。暗部裡的雜訊也會被放大同樣的倍數。

無參考損失。 Zero-DCE 用四個不需要真值的損失函數訓練：

$$\mathcal{L}_{spa} = \frac{1}{K}\sum_{i=1}^{K}\sum_{j\in\Omega(i)}\left(|Y_i - Y_j| - |I_i - I_j|\right)^2,$$

$$\mathcal{L}_{exp} = \frac{1}{M}\sum_{k=1}^{M}|Y_k - E|, \qquad \mathcal{L}_{col} = \sum_{(p,q)\in\varepsilon}(J^p - J^q)^2,$$

$$\mathcal{L}_{tv_A} = \frac{1}{N}\sum_{n=1}^{N}\sum_{c\in\{R,G,B\}}\left(|\nabla_x A_n^c| + |\nabla_y A_n^c|\right)^2.$$

在 $\mathcal{L}_{spa}$（**空間一致性**）中，K 是 4×4 區域的數量，$\Omega(i)$ 是區域 i 的四個鄰居，Y 與 I 分別是增強影像與輸入影像中該區域的平均強度：相鄰區域之間的對比應該保留。在 $\mathcal{L}_{exp}$（**曝光控制**）中，Y_k 是 M 個不重疊 16×16 區域中第 k 個的平均強度，$E = 0.6$ 是目標的良好曝光程度。在 $\mathcal{L}_{col}$（**色彩恆常**）中，J^p 是通道 p 在整張影像上的平均，$\varepsilon = \{(R,G),(R,B),(G,B)\}$：這是灰色世界（gray-world）假設，認為場景的平均顏色是灰色。在 $\mathcal{L}_{tv_A}$（**照度平滑**）中，N 是迭代次數，∇_x, ∇_y 是水平與垂直梯度：相鄰像素應該得到相似的曲線。總損失為 $\mathcal{L}_{spa} + \mathcal{L}_{exp} + W_{col}\mathcal{L}_{col} + W_{tv_A}\mathcal{L}_{tv_A}$，論文中 $W_{col} = 0.5$、$W_{tv_A} = 20$；訓練資料是 SICE [56] 的多重曝光影像，同時提供曝光不足與過曝的輸入。

後續方法改良了這個想法：

- **Zero-DCE++** [34]（2022）改用深度可分離卷積（depthwise separable convolution），只預測 **3** 張曲線參數圖並在所有迭代中重複使用，而且在降取樣後的輸入上估計，再縮放回原尺寸。模型因此縮小到約 10k 個參數，即使在 CPU 上也跑得相當快。
- **SCI** [35]（2022）以權重共享的多階段串接學習照度，並用**自校正**（self-calibrated）模組讓各階段的輸出在訓練時收斂到相同狀態，因此測試時只需要一**個**基本模組。它以非監督損失訓練。
- **CLIP-LIT** [36]（2023）針對逆光影像。它使用視覺-語言模型 CLIP，學習一對文字提示（prompt），一個代表逆光、一個代表光線良好，再迭代地精修提示與增強網路，讓輸出與「光線良好」提示的 CLIP 相似度成為訓練訊號。

用 PyTorch 寫一個零參考小玩具

為了看看這些損失函數如何運作，下面的程式碼在 CPU 上訓練一個迷你的 Zero-DCE 風格網路約半分鐘。它用四張 `skimage` 影像的隨機裁切訓練，每塊都在線性光中以隨機倍率變暗；測試則用訓練時從沒看過的咖啡杯影像。全程不使用任何真值。（損失權重與論文不同，因為各項的正規化方式不同：曝光誤差用平方而非絕對值，梯度用平均而非加總。）

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from skimage import data, img_as_float
from skimage.transform import resize
```

```
torch.manual_seed(0); rng = np.random.default_rng(0)
darken = lambda x, k: (k * x ** 2.2) ** (1 / 2.2)          # less light, in linear units
train = [resize(img_as_float(f()), (256, 256), anti_aliasing=True)
         for f in (data.astronaut, data.chelsea, data.rocket, data.immunohistochemistry)]

def batch(n=4, size=64):                                    # random crops, random darkness
    xs = []
    for _ in range(n):
        img = train[rng.integers(len(train))]
        i, j = rng.integers(0, 256 - size, 2)
        xs.append(darken(img[i:i + size, j:j + size], rng.uniform(0.02, 0.3)))
    return torch.tensor(np.stack(xs), dtype=torch.float32).permute(0, 3, 1, 2)

class TinyDCE(nn.Module):
    def __init__(self, n_iter=8, w=16):
        super().__init__()
        self.n_iter = n_iter
        self.body = nn.Sequential(
            nn.Conv2d(3, w, 3, padding=1), nn.ReLU(), nn.Conv2d(w, w, 3, padding=1),
            ↪  nn.ReLU(),
            nn.Conv2d(w, w, 3, padding=1), nn.ReLU(), nn.Conv2d(w, 3 * n_iter, 3,
            ↪  padding=1), nn.Tanh())
    def forward(self, x):
        alphas = self.body(x).chunk(self.n_iter, dim=1)    # 8 maps of shape B x 3 x H x W
        y = x
        for a in alphas:
            y = y + a * y * (1 - y)                         # the LE curve
        return y, torch.cat(alphas, 1)

def L_exp(y, E=0.6, p=16):   # mean brightness of 16x16 patches should be near E
    return (F.avg_pool2d(y.mean(1, keepdim=True), p) - E).pow(2).mean()
def L_col(y):                # gray-world: channel means should agree
    r, g, b = y.mean((2, 3)).unbind(1)
    return ((r - g) ** 2 + (r - b) ** 2 + (g - b) ** 2).mean()
def L_tv(a):                 # curve parameters should vary smoothly in space
    return (a[..., 1:, :] - a[..., :-1, :]).pow(2).mean() + (a[..., :, 1:] - a[..., :,
     ↪  :-1]).pow(2).mean()
def L_spa(y, x, p=4):        # keep the contrast between neighbouring 4x4 regions
    Y, X = (F.avg_pool2d(t.mean(1, keepdim=True), p) for t in (y, x))
    return sum(((torch.diff(Y, dim=d) - torch.diff(X, dim=d)) ** 2).mean() for d in (2, 3))

net = TinyDCE()
opt = torch.optim.Adam(net.parameters(), lr=1e-3)
for step in range(600):                                     # ~20-40 s on a laptop CPU
    x = batch()
    y, a = net(x)
    loss = L_spa(y, x) + 10 * L_exp(y) + 5 * L_col(y) + 200 * L_tv(a)
    opt.zero_grad(); loss.backward(); opt.step()

test = darken(img_as_float(data.coffee()), 0.05)           # an image never used in training
with torch.no_grad():
    out, _ = net(torch.tensor(test, dtype=torch.float32).permute(2, 0, 1)[None])
print(f"mean brightness: {test.mean():.3f} -> {out.mean():.3f}")   # about 0.10 -> 0.50
```

拖動滑桿，比較暗的輸入與小玩具的輸出：

暗的輸入

零參考小玩具

暗的咖啡杯影像與小網路的輸出：輸出亮得多，木紋清晰可見，顏色比原照片淺、略偏綠且飽和度較低。

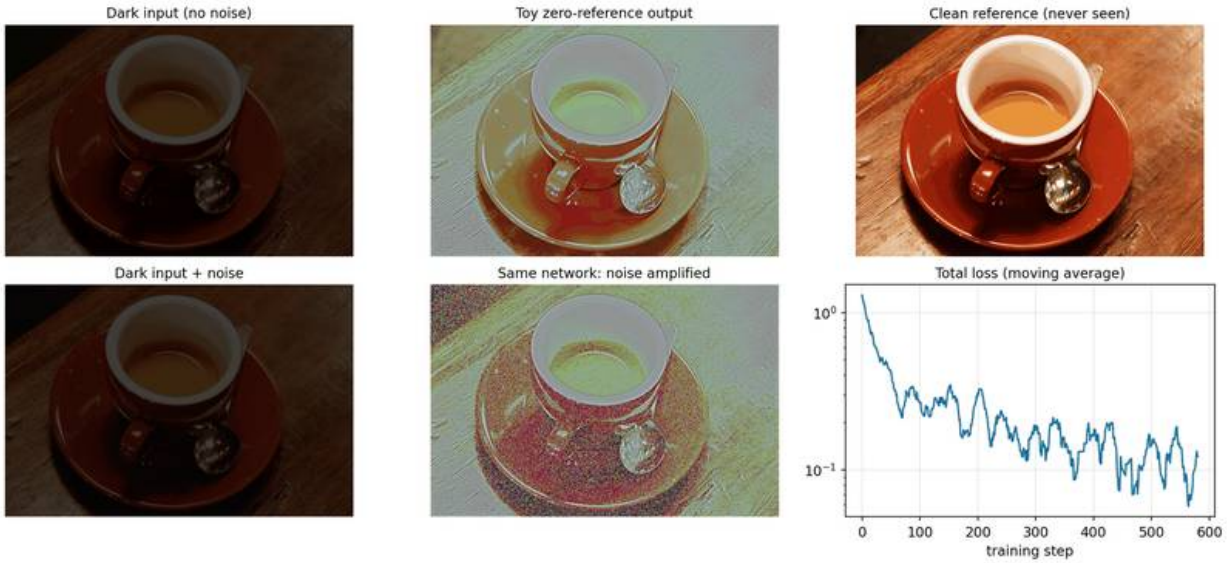


圖 7 — 零參考損失做得到與做不到的事。曝光損失給出好的亮度，但灰色世界的色彩損失把紅褐色場景往灰色拉（上排中間對比上排右邊）；遇到真實雜訊時，逐像素曲線會把雜訊放大（下排中

間）。

這個小玩具在一張圖裡同時展示了零參考想法的優點與極限。沒有任何一張參考影像，網路就學到了合理的調亮方式，而且能遷移到沒看過的照片。但每一個損失都是一種先驗，而先驗可能是錯的：場景本來就偏紅或偏褐時，灰色世界假設就失效；固定的曝光目標 E 會把刻意保留的暗區拉平；而損失中沒有任何一項要求去噪。真正的零參考方法用多得多的影像訓練，有些還加入了明確的雜訊處理，但這些失敗模式在已發表的結果中同樣看得到。

正規化流與擴散模型

LLFlow [37]（2022）觀察到：同一張暗影像有許多種可接受的明亮版本，而逐像素損失會把它們平均成一個模糊的折衷。它學習一個條件式正規化流（normalizing flow）：一個可逆網路，在以暗輸入特徵為條件下，把正常光影像對應到高斯分布。這些特徵由呼應傳統方法的先驗引導：輸入的直方圖等化版本、色彩圖（接近 Retinex 的反射率），以及雜訊圖。

2023 年起，**擴散模型**（diffusion model）大量進入 LLIE。它們以暗影像為條件，從隨機雜訊逐步去噪來生成輸出，因此能合成回歸式網路會模糊掉的合理細節：

- **Diff-Retinex** [38]（2023）先用 Retinex Transformer 分解影像，再用條件式擴散模型生成復原後的反射率與照度。
- **GSAD** [39]（2023）加入全域結構感知的正則化，以影像結構引導擴散軌跡上以曲率為基礎的約束，並對困難區域加入不確定性引導項。
- **LightenDiffusion** [40]（2024）在潛在空間中做 Retinex 分解，並以非配對資料訓練。
- **Reti-Diff** [41]（2025）以潛在擴散模型產生精簡的 Retinex 先驗，再用它引導一個復原 Transformer，降低在全解析度上跑擴散的成本。

代價是運算量與保真度：多步去噪讓推論變慢，生成的細節可能看起來對，卻不一定真實。Adhikarla 等人的綜述 [6] 把這個子領域分成六類：內在分解、頻譜與潛在空間、加速、引導式、多模態，以及自主式。

Raw 域的低光

以上方法都作用在處理過的 sRGB 影像上，而相機的處理流程（去馬賽克、白平衡、去噪、色調曲線、壓縮）早已丟掉部分資訊。**Learning to See in the Dark** [44]（2018）改把 **raw** 感測器資料送進 U-Net，用它取代整條處理流程。它的 SID 資料集有 5,094 張短曝光 raw 影像，對應 424 張不同的長曝光參考影像，來自 Sony α7S II（Bayer 感測器）與 Fujifilm X-T2（X-Trans 感測器），曝光比為 ×100、×250 與 ×300。Raw 資料保留光子數與數值之間的線性關係，位元數也更多，因此在 sRGB 中毫無希望的極暗場景也能被救回來。代價是模型綁定在單一感測器上。

All-in-one 復原

低光增強也是通用復原模型中的一項任務。**InstructIR** [43]（2024）接受自然語言指令（「把這張照片調亮」、「去除雜訊」）並據此復原影像，在一個網路中涵蓋去噪、去模糊、去霧與低光增強。

面向下游任務的低光處理

如果目標是偵測而非好看的照片，常見的基準做法是「先增強、再偵測」（請見DLCV 03・物件偵測）。但為人眼調校的增強，不一定是偵測器需要的。**MAET** [45]（2021）以物理方式模擬低光退化，並訓練偵測器同時預測退化參數，讓特徵能感知退化。**FeatEnHancer** [46]（2023）則在網路內部增強階層式特徵，直接用下游任務的損失訓練，而不是增強像素。

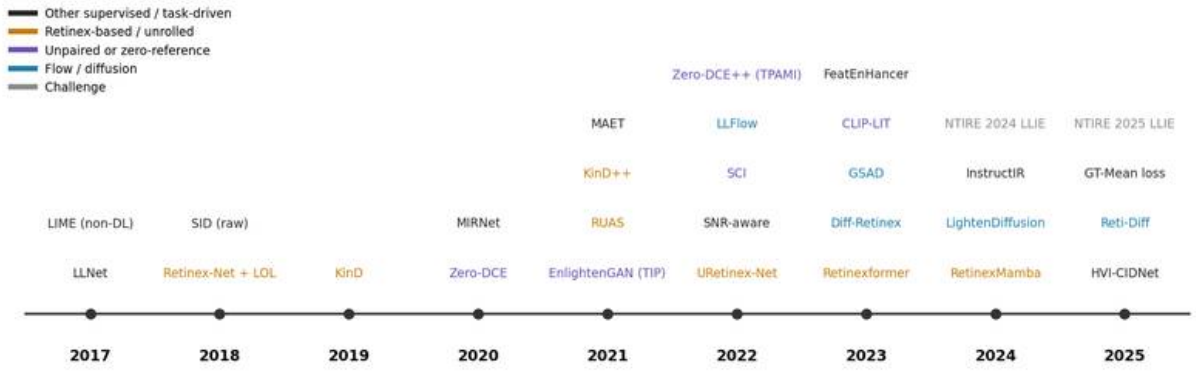


圖 *8* — 本篇提到的方法時間軸（依發表年份）。顏色標示以 *Retinex* 為基礎或展開式的設計、非配對或零參考訓練、*flow* 與擴散模型，以及挑戰賽。

評估

白話版。分數有兩類。**全參考**指標把輸出和一張真值照片比較；**無參考**指標只看輸出本身。第三種做法則是問：增強後的影像有沒有幫助下游任務？每一種回答的問題都不同，而且每一種都可能被鑽漏洞。

全參考指標

給定參考影像 y 與輸出 $\hat{y}$（數值在 $[0,1]$），**PSNR** 為

$$\mathrm{PSNR} = 10\log_{10}\frac{1}{\frac{1}{n}\|\hat{y}-y\|_2^2},$$

其中 n 是數值的個數。它是純粹的像素誤差。**SSIM** [47] 比較局部的亮度、對比與結構統計量，與人眼感知的相關性較高。**LPIPS** [48] 衡量兩張影像深度特徵之間的距離，越低越好，對 PSNR 忽略的紋理與模糊很敏感。

無參考指標：為什麼低光評估離不開它

白話版。無參考（no-reference, NR）指標只看一張影像，就猜它的品質好不好，沒有「標準答案」可以比對。低光論文需要這類指標，因為最貼近真實的測試照片根本沒有標準答案；就算有，那

也只是眾多可接受亮度中的其中一種。

精確版。有兩個事實把 LLIE 推向無參考評估：

1. **真實測試集是非配對的**。 LIME [17]、MEF [58]、NPE [18]、DICM [59] 與 VV [60] 都是小規模的真實照片集，常有不均勻的光照，沒有真值。LOL [21] 這類成對資料集雖然存在，但它們是在受控條件下拍攝的（靜態場景、兩種曝光），模型可能在上面表現很好卻無法泛化（即 [2] 所報告的泛化落差）。因此幾乎每篇 LLIE 論文都會展示非配對測試集上的結果，而能替這些結果打分數的，只有無參考指標與人的判斷。
2. **「正確」的亮度本身就有歧義**。即使有參考影像，正常光照的照片也只是某位攝影師對曝光與色彩的一種選擇。稍微亮一點或暖一點的輸出可能一樣好看，PSNR 卻會掉好幾 dB（這正是下面 GT-mean 協定出現的原因）。無參考指標至少不會懲罰一個不同但合理的曝光。

代價是：無參考指標本身是一個人類偏好的模型，而每個模型都有盲點。無參考影像品質評估（image quality assessment, IQA）的一般理論，包括 NIQE、BRISQUE、MUSIQ 與 CLIP-IQA 的建構方式，請見DLCV 04・影像復原。以下各小節摘要每一類指標量測的是什麼，並聚焦在它們面對增強後的低光影像時的表現。

經典的自然場景統計指標：NIQE、BRISQUE 與 PI

白話版。自然照片有共同的統計規律：把局部亮度與對比去掉之後，剩下的數值會呈現特定形狀的鐘形分布。模糊、雜訊與壓縮會改變這個形狀。自然場景統計（natural scene statistics, NSS）指標量測的，就是一張影像的統計量離乾淨照片的統計量有多遠。

MSCN 係數。 NIQE 與 BRISQUE 都從灰階影像 I 的**去均值、對比正規化**（mean-subtracted contrast-normalized, MSCN）係數出發 [50]：

$$\hat{I}(i,j) = \frac{I(i,j) - \mu(i,j)}{\sigma(i,j) + C}, \qquad \mu(i,j) = \sum_{k,l} w_{k,l}\, I(i+k, j+l), \qquad \sigma(i,j) = \sqrt{\sum_{k,l} w_{k,l}\, (I(i+k,j+l) - \mu(i,j))^2},$$

其中 (i,j) 是像素位置，w 是總和為 1 的小型圓對稱高斯視窗，μ 與 σ 是局部加權平均與標準差，C 是避免在平坦區域除以零的小常數（8 位元影像取 $C = 1$）。對原始高品質影像而言，$\hat{I}$ 的直方圖接近單位高斯；失真會讓它變得更尖（模糊）或尾巴更厚（雜訊、區塊效應）。相鄰 MSCN 係數在四個方向上的乘積，則捕捉剩下的結構。

BRISQUE [50] 對 $\hat{I}$ 擬合廣義高斯分布、對相鄰乘積擬合非對稱廣義高斯分布（兩個尺度），再把擬合得到的形狀與變異數參數，送進一個以人類對合成失真的主觀評分訓練的支援向量迴歸器。越低越好。

NIQE [49] 不需要主觀評分。它挑選清晰的區塊、計算類似的 NSS 特徵，對一組原始高品質影像擬合平均 ν_1、共變異數 Σ_1 的多變量高斯（multivariate Gaussian, MVG），再對測試影像擬合另一個 MVG (ν_2, Σ_2)，回報

$$D(\nu_1, \nu_2, \Sigma_1, \Sigma_2) = \sqrt{(\nu_1 - \nu_2)^\top \left(\frac{\Sigma_1 + \Sigma_2}{2} \right)^{-1} (\nu_1 - \nu_2)},$$

也就是兩個高斯之間類似馬氏距離（Mahalanobis distance）的距離。越低越好：0 代表「統計上與高品質影像集一樣」。

PI。感知指標（Perceptual Index）是為 2018 年 PIRM 超解析度挑戰賽提出的 [63]：

$$\mathrm{PI} = \tfrac{1}{2}((10 - \mathrm{Ma}) + \mathrm{NIQE}),$$

其中 Ma 是 Ma 等人 [64] 學習得到的無參考分數（越高越好，大約介於 0 到 10，以人類對超解析度影像的評分訓練）。PI 越低越好。例如 Zero-DCE [33] 就在非配對測試集上回報 PI，並搭配使用者研究。

它們獎勵什麼，以及在曝光問題上如何誤導。這三個指標都是為結構性失真（模糊、雜訊、壓縮、振鈴）設計的，不是為曝光或色彩：

- MSCN 正規化在定義上就把局部平均與對比除掉了，所以整體亮度的改變幾乎不會影響特徵。增強不足的影像與曝光良好的影像可能拿到相近的分數。
- 灰階特徵忽略色偏與飽和度，而這正是圖 2 裡的錯誤。
- 雜訊與過度銳化會增加高頻能量，對這些統計量來說，一張雜訊被放大、紋理「豐富」的輸出，可能比一張乾淨但略柔的輸出更「自然」。強烈局部對比的 HE 與 CLAHE 輸出就是典型例子。
- 用來定義「高品質」的影像集與訓練用的失真裡，很少有暗景、夜景或逆光場景，所以「自然」其實等於「像白天」。

低光專用的量測：明度順序、色彩豐富度與雜訊

白話版。有些量測直接針對增強時會出錯的地方：方法有沒有把「哪裡比較亮」的關係顛倒？顏色是鮮豔還是褪色？調亮之後露出了多少雜訊？

明度順序誤差（Lightness Order Error, LOE）。 NPE 論文 [18] 主張，自然的增強應該保留場景中明度的相對順序：如果燈原本比牆亮，增強後也應該如此，因為這個順序承載了光的方向與深度感。LOE 計算兩個像素的明暗順序被翻轉的次數。令 $L(x) = \max_{c\in\{R,G,B\}} I_c(x)$ 為輸入的明度，$L_e(x)$ 為增強後影像的明度，

$$\mathrm{LOE} = \frac{1}{m}\sum_{x=1}^{m} \mathrm{RD}(x), \qquad \mathrm{RD}(x) = \sum_{y=1}^{m} U(L(x), L(y)) \oplus U(L_e(x), L_e(y)),$$

其中 m 是像素數，$p \geq q$ 時 $U(p,q) = 1$、否則為 0，$\oplus$ 是互斥或（XOR），因此 $\mathrm{RD}(x)$ 就是相對於 x 改變了順序的像素個數。越低越好。實務上有三點要注意：

- 計算量是 $O(m^2)$，所以實作會在降取樣後的影像上計算 LOE。原始數值會隨 m 增加，只有在相同的降取樣設定下才能比較，而許多論文並未說明自己的設定。
- 比較的基準是**輸入**，而不是乾淨影像。任何在最大值通道上單調的色調曲線都會得到 LOE = 0，包括什麼都不做。而且暗輸入裡的雜訊早已打亂相鄰像素的順序，所以去除雜訊的方法反而會被懲罰。
- LOE 完全不管結果夠不夠亮。它是防止不自然局部色調映射的護欄，不是品質分數。

色彩豐富度（colorfulness）。 Hasler 與 Süsstrunk [65] 以 8 位元 RGB 上的對立色軸，擬合出一個與主觀感受相符的簡單色彩豐富度量：

$$rg = R - G, \qquad yb = \tfrac{1}{2}(R + G) - B, \qquad M = \sqrt{\sigma_{rg}^2 + \sigma_{yb}^2} + 0.3\sqrt{\mu_{rg}^2 + \mu_{yb}^2},$$

其中 μ 與 σ 是整張影像上的平均與標準差，權重 0.3 來自作者的擬合。越高代表色彩越豐富。它適合當作描述性統計量，用來偵測褪色（只增強亮度通道，圖 2）或過飽和，但它不知道什麼才是正確的顏色，而色度通道裡的雜訊會灌大 σ_{rg} 與 σ_{yb}。

輸出的雜訊程度。既然調亮會讓雜訊現形（圖 1），回報輸出的雜訊程度就很有幫助。Immerkær 的快速估計法 [66] 用一個能抵消平滑結構、保留雜訊的遮罩對影像濾波：

$$N = \begin{bmatrix} 1 & -2 & 1 \\ -2 & 4 & -2 \\ 1 & -2 & 1 \end{bmatrix}, \qquad \hat{\sigma} = \sqrt{\frac{\pi}{2}}\,\frac{1}{6\,(W-2)(H-2)} \sum_{i,j} \lvert (I * N)(i,j) \rvert,$$

其中 $W \times H$ 是影像大小，總和只取內部像素。它假設雜訊是白高斯雜訊，所以遇到紋理豐富的影像會高估，遇到空間相關的雜訊（真實相機管線與許多網路產生的正是這種）則會低估。請只用它比較同一張輸入的不同輸出，不要跨場景比較。

下面的程式碼重建圖 1 的暗咖啡杯影像，比較五種「增強」結果與乾淨照片。

```
import numpy as np
from scipy import ndimage as ndi
from skimage import color, data, exposure, img_as_float

rng = np.random.default_rng(0)
clean = img_as_float(data.coffee())

def synth_low_light(x, k=0.05, peak=2000.0, read_std=8e-4, gamma=2.2):   # 與圖 1 相同
    lin = k * x ** gamma
    lin = rng.poisson(lin * peak) / peak
    lin = lin + rng.normal(0, read_std, lin.shape)
    return np.round(np.clip(lin, 0, 1) ** (1 / gamma) * 255) / 255

def loe(ref, out, step=8):
    """Lightness Order Error on a strided subsample (m = 50 x 75 = 3750 pixels here)."""
    L, Le = (im.max(axis=2)[::step, ::step].ravel() for im in (ref, out))
    U, Ue = L[:, None] >= L[None, :], Le[:, None] >= Le[None, :]
    return (U ^ Ue).sum(axis=1).mean()                  # mean over x of RD(x)

def colorfulness(im):
    """Hasler-Suesstrunk colorfulness, RGB scaled to 0..255."""
    R, G, B = (255 * im[..., c] for c in range(3))
    rg, yb = R - G, 0.5 * (R + G) - B
    return np.hypot(rg.std(), yb.std()) + 0.3 * np.hypot(rg.mean(), yb.mean())

def noise_sigma(im):
    """Immerkaer's noise estimate on BT.601 luma, in 8-bit units."""
    Y = 255 * (0.299 * im[..., 0] + 0.587 * im[..., 1] + 0.114 * im[..., 2])
    N = np.array([[1, -2, 1], [-2, 4, -2], [1, -2, 1]])
    H, W = Y.shape
    return np.sqrt(np.pi / 2) / (6 * (W - 2) * (H - 2)) * np.abs(ndi.convolve(Y, N)[1:-1,
     ↪  1:-1]).sum()

dark = synth_low_light(clean)
lab = color.rgb2lab(dark); L = lab[..., 0] / 100
he = lab.copy(); he[..., 0] = 100 * exposure.equalize_hist(L)
```

```
cl = lab.copy(); cl[..., 0] = 100 * exposure.equalize_adapthist(L, clip_limit=0.01)
outputs = {
    "dark input": dark,
    "gain x20 (linear)": np.clip(20 * dark ** 2.2, 0, 1) ** (1 / 2.2),
    "gamma 0.4": dark ** 0.4,
    "HE on L*": np.clip(color.lab2rgb(he), 0, 1),
    "CLAHE on L*": np.clip(color.lab2rgb(cl), 0, 1),
    "clean reference": clean,
}
for name, im in outputs.items():
    print(f"{name:18s} mean {im.mean():.2f}  LOE {loe(dark, im):6.1f}  "
          f"colorfulness {colorfulness(im):5.1f}  noise {noise_sigma(im):5.2f}")
# dark input         mean 0.10  LOE    0.0  colorfulness  23.1  noise  3.32
# gain x20 (linear)  mean 0.39  LOE    0.3  colorfulness  90.0  noise 12.93
# gamma 0.4          mean 0.34  LOE    0.0  colorfulness  61.6  noise 11.19
# HE on L*           mean 0.48  LOE  334.7  colorfulness  28.3  noise 16.17
# CLAHE on L*        mean 0.40  LOE  657.6  colorfulness  27.1  noise 23.06
# clean reference    mean 0.39  LOE  322.0  colorfulness  76.9  noise  4.69
```

圖 *9* — 更亮不代表更好。*HE* 是最亮的輸出，但明度順序被破壞的程度僅次於 *CLAHE*，而且幾乎沒有顏色；有雜訊的線性增益比乾淨照片更「鮮豔」；乾淨照片本身的 *LOE* 也很高，因為 *LOE* 是相對於有雜訊的輸入計算的。

這些數字要一起看，不能各看各的：

- **只看亮度會排錯順序**。 HE 的平均最高（0.48），但 LOE 也很大、雜訊第二高，顏色幾乎都沒

了。CLAHE 的局部重新映射則是破壞明度順序最嚴重的。

- **色彩豐富度會被雜訊騙**。線性增益得到 90，高於乾淨照片的 77，部分原因是被放大的色度雜訊拉高了 σ_{rg} 與 σ_{yb}。
- **LOE 懲罰復原**。乾淨照片明明是理想答案，相對於有雜訊的輸入卻得到 LOE 322；什麼都不做反而是 0。低 LOE 是自然色調曲線的必要條件，而不是品質的充分條件。
- **只有雜訊估計把乾淨照片**（4.7）與明亮但多雜訊的輸出（11 到 23）區分開來。沒有任何單一的無參考數字能同時涵蓋「曝光良好、顏色自然、乾淨無雜訊」。

學習式無參考評估器

白話版。不再使用手工設計的統計量，而是訓練一個神經網路去預測人類評分，或直接問大型視覺語言模型一張照片看起來好不好。這類模型更能捕捉人們在意的東西，包括內容與美感，但也會繼承訓練資料的偏差。

主要的類別如下，都可以套用在 LLIE 的輸出上（也有共用的實作，例如 IQA-PyTorch 工具箱 [75]）：

- **NIMA** [67]（2018）用 CNN 預測人類評分在各分數區間上的分布，而不是單一數字。回報的分數是該分布的平均 $\hat{s} = \sum_k k\, p_k$，p_k 是預測為分數 k 的機率。以美感評分訓練的版本量測的是「這是不是一張好看的照片」，和「這是不是忠實的增強」並不相同。
- **MUSIQ** [68]（2021）是多尺度 Transformer，以影像的原始解析度與長寬比輸入，並加入縮小版本，使用以雜湊為基礎的 2D 位置與尺度嵌入。不做縮放對 LLIE 很重要，因為雜訊與細紋理正是縮放會藏起來的東西。越高越好。
- **CLIP-IQA** [69]（2023）利用視覺語言模型 CLIP 與一對反義提示詞「Good photo.」和「Bad photo.」。令 $\mathbf{x}$ 為影像嵌入、$\mathbf{t}_1, \mathbf{t}_2$ 為兩個文字嵌入，$s_i = \mathbf{x}^\top \mathbf{t}_i / (\|\mathbf{x}\|\,\|\mathbf{t}_i\|)$，分數為 $e^{s_1}/(e^{s_1}+e^{s_2})$，介於 $[0,1]$，越高越好。基本版本是零樣本的，不需要主觀評分。換成其他提示詞對（「Bright photo.」/「Dark photo.」、「Noisy photo.」/「Clean photo.」）可以探測單一屬性，對曝光評估很方便，但用字不同分數就會不同。同樣的 CLIP 相似度想法也被 CLIP-LIT [36] 當作訓練訊號，所以 CLIP 類指標不應該是評判以 CLIP 訓練之增強器的唯一裁判。
- **MANIQA** [70]（2022）結合 Vision Transformer、轉置（通道）注意力與 Swin Transformer 區塊，再以雙分支、依區塊加權的方式產生分數。它贏得 NTIRE 2022 感知 IQA 挑戰賽，而該挑戰的影像包含復原演算法與 GAN 演算法的輸出，因此比用相機失真訓練的指標更貼近「演算法輸出」的情境。
- **Q-Align** [71]（2024）微調一個大型多模態模型，讓它以文字評分等級（bad、poor、fair、good、excellent）作答，模仿人類評分者的方式。令 X_{l_i} 為模型對等級 l_i 的 logit，分數為 $S = \sum_{i=1}^{5} i \cdot e^{X_{l_i}} / \sum_{j=1}^{5} e^{X_{l_j}}$，也就是只在這五個詞上做 softmax 後的期望等級。
- **低光專用的評估器**。Kannan 等人 [72]（2023）針對不同 LLIE 方法復原的影像收集人類主觀評分，以此評測現有的無參考 IQA 模型，並提出一個不需主觀評分（opinion-unaware）的模型：以自監督對比學習，學出對低光復原影像失真敏感的特徵。這類資料集才是決定「在增強輸出上該信任哪個指標」的正確測試平台。

低光情境下的注意事項。學習式評估器多半以白天照片訓練。它們傾向給更亮、更銳利、更飽和的影像高分，因而偏好過度增強；一張刻意保留暗調的夜景，即使是最忠實的結果，也可能被扣分。分數取決於權重檔（用哪個訓練集）、輸入尺寸與實作，這些都應該回報。另外，當一個方法

在損失函數中使用了某個學習式指標或相同的骨幹網路時，那個指標就不再是獨立的裁判。

使用者研究與成對比較

白話版。好不好看，最終由人決定。兩種標準的問法是：請人在量表上打分數，或請人從兩張中選出比較好的一張。

平均意見分數（mean opinion score, MOS）。N 位評分者各自給一張影像一個固定量表上的分數 s_n（常見為 1 到 5），$\mathrm{MOS} = \frac{1}{N}\sum_{n=1}^{N} s_n$。ITU-R BT.500 [73] 為這類測試訂定標準：觀看條件、評分量表、單刺激與雙刺激以及比較式方法，還有篩除不可靠評分者的程序。例如 Zero-DCE [33] 請 15 位受試者以 1–5 分評分 NPE、LIME、MEF、DICM 與 VV 共 202 張影像的結果，並指示他們留意過曝或曝光不足、色彩偏差，以及不自然的紋理或雜訊。

成對比較。絕對量表在不同評分者之間會漂移；從兩張中選一張比較容易，也比較一致。**Bradley–Terry** 模型 [74] 把大量這樣的選擇轉換成每個方法的分數：每個方法 i 有一個強度 $\pi_i > 0$，且

$$P(i \text{ 勝過 } j) = \frac{\pi_i}{\pi_i + \pi_j},$$

再以最大概似估計從觀察到的勝負擬合各個 π_i。EnlightenGAN [32] 就採用這個協定：9 位受試者在 23 張影像上，比較五種方法輸出的配對，判斷是否有可見雜訊、曝光假影，以及不真實的顏色或紋理，再以 Bradley–Terry 擬合得到排名。

良好的做法：隱藏方法名稱、隨機化順序與左右位置、同時顯示輸入以提供脈絡、使用足夠多的影像與評分者、回報信賴區間或顯著性，並說明顯示器與觀看條件。只有少數評分者與精挑細選影像的研究，只能算展示，不能算證據。

無參考指標的選擇與回報

指標	類型	需要訓練資料？	方向	敏感於	低光情境的注意事項
NIQE [49]	NSS	只需高品質影像（不需主觀評分）	越低越好	灰階結構中的模糊、雜訊、壓縮	幾乎看不到整體曝光與色彩；可能偏好多雜訊、高對比的輸出
BRISQUE [50]	NSS + 迴歸	需要，合成失真的主觀評分	越低越好	與 NIQE 相同的特徵，映射到人類分數	以白天失真訓練；忽略色偏
PI [63], [64]	NSS + 學習式	Ma 分數以超解析度評分訓練	越低越好	紋理與銳利度	為超解析度設計；獎勵細節，包括被放大的雜訊
LOE [18]	任務專用	不需要	越低越好	相對於輸入的明度順序翻轉	取決於降取樣；什麼都不做得 0；懲罰去噪
色彩豐富度 [65]	任務專用（描述性）	不需要（常數已擬合）	視情況而定	對立色的分散程度與偏移	不知道正確顏色；會被色度雜訊灌大

指標	類型	需要訓練資料？	方向	敏感於	低光情境的注意事項
雜訊估計 [66]	任務專用（描述性）	不需要	越低越好	高頻殘差	假設白雜訊；會被紋理與相關雜訊混淆
NIMA [67]	學習式	需要，評分分布	越高越好	美感或技術品質，依權重檔而定	美感模型獎勵好看而非忠實的結果
MUSIQ [68]	學習式	需要，主觀評分	越高越好	原始解析度下的多尺度品質	多以白天照片訓練；依權重檔而定
CLIP-IQA [69]	學習式（視覺語言）	零樣本版本不需要	越高越好	語意上「好照片」的印象；依提示詞而定	提示詞用字有影響；對以 CLIP 訓練的增強器不獨立
MANIQA [70]	學習式	需要，主觀評分	越高越好	復原與 GAN 假影	並非專門針對曝光錯誤訓練
Q-Align [71]	學習式（大型多模態模型）	需要，以文字等級表示的主觀評分	越高越好	整體、類似人類的品質判斷	運算量大；繼承基礎模型的偏差；依版本而定
MOS ／成對比較 [73], [74]	人類	需要人類評分者	MOS 越高／強度越高	人們真正偏好的結果	成本高；小規模研究雜訊大；必須回報協定

實務建議：

1. **在成對資料集上**，以標準協定下的全參考指標（PSNR、SSIM、LPIPS）為主，無參考分數只作為補充。
2. **在非配對資料集上**，回報多個來自不同類別的無參考指標（一個 NSS、一到兩個學習式、LOE），說明實作、權重檔與影像尺寸，並展示大量視覺範例，包括失敗案例。
3. **預期會有分歧**。NSS 與學習式指標對方法的排名經常不同，兩者也都可能與人的判斷不一致；Li 等人的綜述 [2] 指出指標分數與視覺品質常常不符，Liu 與 Fan [5] 則發現畫質與下游任務表現之間還有另一道落差。指標意見不一時，請直接說明，而不是只挑對自己有利的那一個。
4. **不要最佳化裁判**。如果方法的損失函數用了某個指標或相同的骨幹網路（例如 CLIP），就要用別的東西來評估它。
5. **用人來支撐主張**。對於新方法，設計良好的使用者研究或成對比較，是最有力的無參考證據。

GT-mean 陷阱

白話版。在成對基準上，整體亮度的一點點差異就會大幅改變 PSNR。有些論文在評分前，會把每張輸出重新縮放，讓它的平均亮度與真值一致。但實際使用時，你永遠拿不到這個資訊。

精確版。「GT-mean」評估把輸出乘上 $\lambda = \mathbb{E}[y]/\mathbb{E}[f(x)]$，再對 $\lambda f(x)$ 計算指標，$\mathbb{E}[\cdot]$ 為影像平均。由於 LLIE 的「正確」亮度本來就有歧義，這樣做可以大幅提高 PSNR。Retinexformer 的作者在程式碼中提供了這個模式，以便與 LLFlow、KinD 及近期擴散模型等方法比較，但明確表示不建議使用，因為它用真值來修正輸出 [26]。Liao 等人 [51]（2025）研究輸出與參考之間的亮度不匹配，提出用於訓練的 GT-mean 損失函數，並同時回報標準與 GT-mean 指標。讀表格時：先確認使用哪一種設定，絕不混用兩者，並優先選擇在標準協定下表現好的方法。

以任務為導向的評估

由於 LLIE 常是前處理步驟，另一種檢驗方式是在增強後的影像上跑偵測或分割模型（例如在 ExDark [57] 上）並回報準確率。Liu 與 Fan 2025 年的綜述 [5] 發現兩者脫節：監督式方法常產生感知品質高的影像，對視覺任務的提升卻有限；零樣本（zero-shot）方法雖然畫質分數較低，在各種任務上的提升反而更一致。Liu 等人的基準 [4] 也評估了低光下的人臉偵測。完整的評估應該回報保真度、無參考品質、任務表現與執行時間。

基準資料集

白話版。資料集的差別在於：有沒有成對、暗是真實的還是合成的、是 raw 還是 sRGB，以及有沒有偵測用的標註。

資料集	類型	內容與用途
LOL (v1) [21]	成對、真實、sRGB	改變曝光拍攝的 500 組低光/正常光影像（485 組訓練／15 組測試），400×600。最常用的 LLIE 基準；測試集很小。
LOL-v2 [52]	成對，真實＋合成子集	較大的 *Real* 子集（實拍成對）與合成成對資料的 *Synthetic* 子集。兩個子集要分開回報。
LSRW [53]	成對、真實	5,650 組，來自 Nikon D7500（3,170 組）與 Huawei P40 Pro（2,480 組）；5,600 組訓練、50 組測試。測試跨裝置泛化。
SDSD [54]	成對影片	以機電系統對齊、拍攝兩次的動態室內與室外場景；常拆成影格做影像測試。
SID [44]	成對、raw	5,094 張短曝光 raw 影像，搭配 424 張長曝光參考（Sony 與 Fuji）。極低光、raw 到 sRGB。
MIT-Adobe FiveK [55]	成對、修圖	5,000 張 raw 照片，每張由五位專家修圖。用於色調與色彩增強（照片修圖）。
SICE [56]	多重曝光序列	每個場景從曝光不足到過曝的多張拍攝與一張參考；用來訓練曝光校正（例如 Zero-DCE）。
VE-LOL [4]	成對＋偵測	VE-LOL-L 用於增強，VE-LOL-H 附有人臉標註，用於高階任務評估。
ExDark [57]	非配對、有標註	7,363 張低光影像、12 個物件類別、10 種光照條件。以任務為導向的評估（偵測）。
LIME [17]、MEF [58]、NPE [18]、DICM [59]、VV [60]	非配對測試集	小型的真實低光或光照不均照片集，用於視覺比較與無參考指標。

挑戰賽：NTIRE 2024 與 2025

CVPR 的 NTIRE workshop 連續兩年舉辦 LLIE 挑戰賽。**NTIRE 2024** 挑戰賽 [61] 涵蓋超高解析度（4K 以上）、不均勻照明、逆光、極暗與夜景等情境，共有 428 人註冊、22 隊有效提交。**NTIRE 2025** 挑戰賽 [62] 有 762 人註冊、28 隊有效提交。這些報告整理了勝出隊伍的設計，是「在固定且公平的協定下什麼方法有效」的一個好切面。

現代觀點

這個領域在不到十年內從手工先驗走到學習模型，但原文 [1] 提出的核心問題依然相同：如何描述光、如何處理雜訊，以及如何判斷輸出好不好。下面幾篇綜述是最好的入門點。

Li 等人，TPAMI 2022 [2]。引用最多的深度 LLIE 綜述。它依學習策略（監督式、強化學習、非監督式、零樣本與半監督式學習）、網路結構、是否使用 Retinex 模型、資料格式（sRGB 或 raw）、損失函數，以及訓練與測試資料來整理方法。它也提出了 LLIV-Phone：一個由多款手機相機在各種光照下拍攝的影像與影片資料集，以及一個比較方法的線上平台。主要結論是泛化落差（許多方法在手機資料上表現不佳），以及指標分數與視覺品質經常不一致。

Zheng 等人，"Low-Light Image and Video Enhancement: A Comprehensive Survey and Beyond" [3]。以類似的分類回顧傳統與深度方法，並在資料上更進一步：提出 SICE_Grad 與 SICE_Mix（在單張影像中混合曝光不足與過曝），以及 Night Wenzhou（空拍與街景的夜間影片集），用來測試現有基準沒涵蓋的情況。原文的概觀圖也出自這篇。

Liu 等人，IJCV 2021 [4]。"Benchmarking low-light image enhancement and beyond" 與其說是綜述，不如說是基準：VE-LOL 資料集提供成對資料與人臉標註，以全參考、無參考與語意指標進行比較，並提出一個增強加偵測的聯合模型。它讓以任務為導向的評估成為主流。

Liu 與 Fan，Neurocomputing 2025 [5]。這篇綜述追問哪種增強真正幫得上下游視覺任務，並發現前述畫質指標與任務表現之間的脫節，其中零樣本方法帶來最一致的任務提升。

Adhikarla 等人，2025 [6]。一篇關於擴散模型 LLIE 的綜述，提出六類分類法並做效能分析。它的訊息是：擴散模型帶來真實感與彈性，但推論成本很高，因此加速、引導與潛在空間設計是目前的熱門方向。

NTIRE 報告 [61], [62]。它們不是綜述，但呈現了頂尖隊伍在共同協定、高解析度與困難光照下的實際做法。

目前的最先進技術（2023–2026）。主要有三個方向。第一，**Retinex 引導的骨幹網路**：保留「光與反射率」結構的 Transformer（Retinexformer [26]）與狀態空間模型（RetinexMamba [42]）。第二，**生成模型**：擴散模型（Diff-Retinex [38]、GSAD [39]、LightenDiffusion [40]、Reti-Diff [41]）與 flow（LLFlow [37]）。第三，**更好的表示法與評估協定**：學習出來的色彩空間（HVI-CIDNet [27]）、考慮亮度不匹配的訓練與評估（GT-Mean loss [51]）、語言引導與 all-in-one 復原（CLIP-LIT [36]、InstructIR [43]），以及任務感知的增強（MAET [45]、FeatEnHancer [46]）。

未解決的問題。

- **真實雜訊**。多數 sRGB 方法使用的訓練資料，其雜訊與經過相機處理流程後的真實感測器雜訊不相符。更好的雜訊模型 [7]、raw 域訓練 [44]，或增強與去噪的聯合處理仍是核心課題。

- **影片一致性**。逐影格增強會閃爍。成對的影片資料（SDSD [54]）稀少，時間一致性依然困難，對生成模型尤其如此。
- **評估**。亮度目標本身是病態的、GT-mean 協定、無參考指標對曝光不可靠，以及感知指標與任務指標的落差，都讓比較結果很脆弱。
- **邊緣裝置上的效率**。手機與車輛需要即時、低功耗的模型。曲線方法（Zero-DCE++ [34]、SCI [35]）與輕量的先驗式網路 [28] 證明了極小的模型也能運作，而擴散模型對這類用途仍然太慢。
- **泛化**。在某個資料集或相機上訓練的模型，換到另一個常常失效（[2] 所報告的泛化落差）；混合過曝與曝光不足、逆光，以及有色人工光源仍然困難。

重點整理

- LLIE 是**色調強化加上復原**：完美的亮度校正之後，雜訊、色偏與量化誤差依然存在（圖 1）。
- 亮度可以在獨立通道（V、I、Y'、L^*）中增強，但這個選擇會牽動飽和度。BT.601 亮度為 $0.299R' + 0.587G' + 0.114B'$；$16 + 0.257R' + 0.504G' + 0.098B'$ 是同一個亮度在 8 位元有限範圍下的寫法。
- Retinex 把影像表示為反射率乘上**照度**，$S = R \circ L$；SSR、MSR、LIME 以平滑先驗估計 L，而反轉影像去霧的做法本質上建立在同一個假設上。
- 深度方法最好依**訓練訊號**分類：監督式、展開式、非配對（EnlightenGAN）、零參考（Zero-DCE、SCI、CLIP-LIT）與生成式（LLFlow、擴散模型）。Retinex 分解橫跨其中各類。
- Zero-DCE 的 LE 曲線 $x + \alpha x(1 - x)$ 保留範圍與順序；它的無參考損失是先驗（曝光、灰色世界、平滑、對比），遇到真實雜訊與色彩鮮豔的場景時可能失效。
- 同時回報全參考、無參考與任務導向的結果，**絕不混用 GT-mean 與標準數字**，並記住 PSNR 較高不代表偵測更好。
- 每個無參考指標都只看到一個面向：NIQE、BRISQUE、PI 看得到結構，卻幾乎看不到曝光與色彩；LOE 看得到明度順序，卻看不到亮度與雜訊；色彩豐富度會被色度雜訊灌大；學習式評估器則繼承了白天照片的偏差。**更亮不代表更好**（圖 9）：請回報多個來自不同類別的無參考指標，再加上設計良好的使用者研究。

練習

1. **單調曲線**。證明 $\mathrm{LE}(x;\alpha) = x + \alpha x(1 - x)$ 在 $[0, 1]$ 上單調不減，若且唯若 $|\alpha| \le 1$。$\alpha = 1.5$ 時會出什麼問題？

   提示

   導數為 $1 + \alpha(1 - 2x)$，對 x 是線性的，所以在 $[0, 1]$ 上的最小值出現在端點：$x = 0$ 時為 $1 + \alpha$，$x = 1$ 時為 $1 - \alpha$。兩者都 ≥ 0 恰好等價於 $|\alpha| \le 1$。$\alpha = 1.5$ 時，$x = 1$ 處的斜率為 -0.5，接近白色的像素會隨輸入變亮而變暗，明暗順序反轉，在亮部造成假影。

2. **迭代曲線的增益**。對很小的輸入 x，證明以固定 α 迭代八次大約會把 x 乘上 $(1 + \alpha)^8$。用這個結果解釋為什麼零參考曲線方法會放大暗部雜訊，並提出一個能減輕問題的損失函數或架構修改。

提示

x 很小時 $x(1-x) \approx x$，所以每一步都是 $x \mapsto (1+\alpha)x$。$\alpha = 1$ 時就是 $2^8 = 256$，標準差為 σ 的雜訊變成 256σ。可能的修正：加入去噪分支或考慮雜訊的損失、在估計訊噪比低的地方限制 α（類似 SNR-aware 的精神），或懲罰輸出中、平滑後的輸入所沒有的高頻能量。

3. **兩個亮度公式**。一個 8 位元像素為 $(R', G', B') = (200, 100, 50)$。計算它的 BT.601 全範圍亮度與有限範圍亮度，並說明兩者編碼的是同一個量。

提示

全範圍：$0.299 \cdot 200 + 0.587 \cdot 100 + 0.114 \cdot 50 = 124.2$。有限範圍：$16 + 0.257 \cdot 200 + 0.504 \cdot 100 + 0.098 \cdot 50 \approx 122.7$。換算回去：$(122.7 - 16) \cdot 255/219 \approx 124.2$。唯一的差別是比例 219/255 與偏移量 16。

4. **沒有真值的 Retinex 損失**。在 Retinex-Net 中，為什麼需要 $\mathcal{L}_{ir} = \|R_{\text{low}} - R_{\text{normal}}\|_1$？描述一個能讓 $\mathcal{L}_{\text{recon}}$ 的自我重建項與 $\mathcal{L}_{is}$ 都變成零、卻毫無用處的分解，並說明 $\mathcal{L}_{ir}$ 如何排除它。

提示

令 $I_i \equiv 1$（完全平滑）且 $R_i = S_i$。每張影像都能被完美重建，平滑損失也為零，但根本沒有做任何分解。由於 $S_{\text{low}} \neq S_{\text{normal}}$，這會讓 $R_{\text{low}} \neq R_{\text{normal}}$，所以 $\mathcal{L}_{ir}$ 很大。要求兩者反射率相等，就迫使兩張影像之間的亮度差異被放進照度裡。

5. **手算 GT-mean**。你的模型輸出平均為 0.40，真值平均為 0.50，除了這個整體倍率之外輸出完美無缺。GT-mean 的縮放因子是多少？如果影像是常數，縮放前後的 PSNR 各是多少？為什麼縮放後的數字無法公平地與正常評估的方法比較？

提示

$\lambda = 0.50/0.40 = 1.25$。縮放前：MSE $= 0.1^2 = 0.01$，PSNR $= 20$ dB。縮放後輸出等於參考，誤差為 0，PSNR 趨於無窮大。實際使用時方法永遠拿不到真值的平均，所以 GT-mean 數字獎勵的是正確的結構，卻忽略了任務中最難的部分：決定亮度。

6. **更換色彩先驗**。在 PyTorch 小玩具中，把灰色世界損失換成保留輸入色度的損失，例如 $\|y/\sum_c y_c - x/\sum_c x_c\|_1$。重新訓練，並把咖啡杯的結果與圖 7 比較。兩種先驗各在什麼情況下較好？

提示

分母要加上一個小 epsilon。保留輸入色度能讓紅褐色的杯子維持紅色，但也會保留光源的色偏（例如鈉燈）。灰色世界能去除色偏，卻會讓真正色彩鮮豔的場景褪色。實際的方法常結合較弱的色彩先驗，與從資料學到的色彩校正。

7. **手算 LOE**。一張三像素的輸入，明度為 $L = (0.1, 0.2, 0.3)$，某個增強器輸出 $L_e = (0.5, 0.4, 0.9)$。計算每個像素的 RD(x) 與 LOE。接著解釋為什麼在圖 9 中，乾淨參考影像的 LOE（322）反而高於 gamma 曲線（0），以及這對「只用 LOE 比較一個會去噪的增強器與單純的色調曲線」意味著什麼。

提示

只有像素對 $(1,2)$ 的順序改變：輸入中 $0.1 < 0.2$，輸出中卻是 $0.5 > 0.4$。這同時翻轉了 $U(L_1, L_2)$ 與 $U(L_2, L_1)$，所以 RD $= (1,1,0)$（像素和自己比較永遠不會翻轉），LOE $= 2/3$。逐通道套用的 gamma 曲線是單調的，會完全保留 $\max(R,G,B)$ 的順序，所以得到 0。乾淨影像沒有暗輸入裡的雜訊，而這些雜訊早已打亂許多相鄰、數值幾乎相等的像素順序，因此乾淨影像在這些像素對上與輸入不一致。換句話說，LOE 獎勵保留輸入的雜訊、懲罰真正的復原；它必須搭配雜訊或品質量測一起看，絕不能單獨使用。

參考文獻

1. Shyandram，〈簡介 Deep Learning Low-Light Image Enhancement (LLIE)〉，Medium 文章，2024；2026 年更新。連結
2. C. Li, C. Guo, L. Han, J. Jiang, M.-M. Cheng, J. Gu and C. C. Loy, "Low-light image and video enhancement using deep learning: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 12, pp. 9396–9416, 2022. arXiv
3. S. Zheng, Y. Ma, J. Pan, C. Lu and G. Gupta, "Low-light image and video enhancement: A comprehensive survey and beyond," arXiv:2212.10772, 2022. arXiv
4. J. Liu, D. Xu, W. Yang, M. Fan and H. Huang, "Benchmarking low-light image enhancement and beyond," *International Journal of Computer Vision*, vol. 129, no. 4, pp. 1153–1184, 2021. doi project
5. F. Liu and L. Fan, "A review of advancements in low-light image enhancement using deep learning," *Neurocomputing*, vol. 652, art. 131052, 2025. arXiv
6. E. Adhikarla, Y. Liu and B. D. Davison, "Diffusion models for low-light image enhancement: A multi-perspective taxonomy and performance analysis," arXiv:2510.05976, 2025. arXiv
7. A. Foi, M. Trimeche, V. Katkovnik and K. Egiazarian, "Practical Poissonian-Gaussian noise modeling and fitting for single-image raw-data," *IEEE Transactions on Image Processing*, vol. 17, no. 10, pp. 1737–1754, 2008. doi
8. ITU-R, "Recommendation BT.601-7: Studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios," 2011. ITU
9. S. M. Pizer, E. P. Amburn, J. D. Austin, et al., "Adaptive histogram equalization and its variations," *Computer Vision, Graphics, and Image Processing*, vol. 39, no. 3, pp. 355–368, 1987. doi
10. K. Zuiderveld, "Contrast limited adaptive histogram equalization," in *Graphics Gems IV*, Academic Press, 1994, pp. 474–485. doi
11. E. H. Land, "The retinex theory of color vision," *Scientific American*, vol. 237, no. 6, pp. 108–128, 1977. doi
12. D. J. Jobson, Z. Rahman and G. A. Woodell, "Properties and performance of a center/surround retinex," *IEEE Transactions on Image Processing*, vol. 6, pp. 451–462, 1997. doi
13. D. J. Jobson, Z. Rahman and G. A. Woodell, "A multiscale retinex for bridging the gap between color images and the human observation of scenes," *IEEE Transactions on Image Processing*, vol. 6, no. 7, pp. 965–976, 1997. doi
14. K. He, J. Sun and X. Tang, "Single image haze removal using dark channel prior," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 12, pp. 2341–2353, 2011. doi

15. X. Dong, Y. Pang and J. Wen, “Fast efficient algorithm for enhancement of low lighting video,” *ACM SIGGRAPH 2010 Posters*, 2010. SIGGRAPH history doi
16. X. Dong, G. Wang, Y. Pang, W. Li, J. Wen, W. Meng and Y. Lu, “Fast efficient algorithm for enhancement of low lighting video,” in *Proc. IEEE International Conference on Multimedia and Expo (ICME)*, 2011. doi
17. X. Guo, Y. Li and H. Ling, “LIME: Low-light image enhancement via illumination map estimation,” *IEEE Transactions on Image Processing*, vol. 26, no. 2, pp. 982–993, 2017. doi
18. S. Wang, J. Zheng, H.-M. Hu and B. Li, “Naturalness preserved enhancement algorithm for non-uniform illumination images,” *IEEE Transactions on Image Processing*, vol. 22, no. 9, pp. 3538–3548, 2013. doi
19. K. G. Lore, A. Akintayo and S. Sarkar, “LLNet: A deep autoencoder approach to natural low-light image enhancement,” *Pattern Recognition*, vol. 61, pp. 650–662, 2017. doi
20. L. Tao, C. Zhu, J. Song, T. Lu, H. Jia and X. Xie, “Low-light image enhancement using CNN and bright channel prior,” in *Proc. IEEE International Conference on Image Processing (ICIP)*, 2017, pp. 3215–3219. doi
21. C. Wei, W. Wang, W. Yang and J. Liu, “Deep retinex decomposition for low-light enhancement,” in *Proc. British Machine Vision Conference (BMVC)*, 2018. arXiv project
22. Y. Zhang, J. Zhang and X. Guo, “Kindling the darkness: A practical low-light image enhancer,” in *Proc. ACM International Conference on Multimedia (ACM MM)*, 2019, pp. 1632–1640. doi
23. Y. Zhang, X. Guo, J. Ma, W. Liu and J. Zhang, “Beyond brightening low-light images,” *International Journal of Computer Vision*, vol. 129, no. 4, pp. 1013–1037, 2021. doi
24. S. W. Zamir, A. Arora, S. Khan, et al., “Learning enriched features for real image restoration and enhancement,” in *Proc. European Conference on Computer Vision (ECCV)*, 2020, pp. 492–511. arXiv
25. X. Xu, R. Wang, C.-W. Fu and J. Jia, “SNR-aware low-light image enhancement,” in *Proc. IEEE/CVF CVPR*, 2022, pp. 17693–17703. doi
26. Y. Cai, H. Bian, J. Lin, H. Wang, R. Timofte and Y. Zhang, “Retinexformer: One-stage Retinex-based transformer for low-light image enhancement,” in *Proc. IEEE/CVF ICCV*, 2023. arXiv code
27. Q. Yan, Y. Feng, C. Zhang, et al., “HVI: A new color space for low-light image enhancement,” in *Proc. IEEE/CVF CVPR*, 2025. arXiv
28. S.-E. Weng, S.-G. Miaou and R. Christanto, “A lightweight low-light image enhancement network via channel prior and gamma correction,” *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 39, no. 12, art. 2554013, 2025. doi code
29. S.-E. Weng, C.-Y. Hsiao, L.-W. Lu, et al., “Rethinking theoretical illumination for efficient low-light image enhancement” (CPGA-Net+), arXiv:2409.05274, 2024. arXiv
30. R. Liu, L. Ma, J. Zhang, X. Fan and Z. Luo, “Retinex-inspired unrolling with cooperative prior architecture search for low-light image enhancement,” in *Proc. IEEE/CVF CVPR*, 2021, pp. 10556–10565. arXiv
31. W. Wu, J. Weng, P. Zhang, X. Wang, W. Yang and J. Jiang, “URetinex-Net: Retinex-based deep unfolding network for low-light image enhancement,” in *Proc. IEEE/CVF CVPR*, 2022, pp. 5891–5900. doi
32. Y. Jiang, X. Gong, D. Liu, et al., “EnlightenGAN: Deep light enhancement without paired supervision,” *IEEE Transactions on Image Processing*, vol. 30, pp. 2340–2349, 2021. arXiv

33. C. Guo, C. Li, J. Guo, et al., “Zero-reference deep curve estimation for low-light image enhancement,” in *Proc. IEEE/CVF CVPR*, 2020, pp. 1777–1786. arXiv
34. C. Li, C. Guo and C. C. Loy, “Learning to enhance low-light image via zero-reference deep curve estimation,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 8, 2022. arXiv
35. L. Ma, T. Ma, R. Liu, X. Fan and Z. Luo, “Toward fast, flexible, and robust low-light image enhancement,” in *Proc. IEEE/CVF CVPR*, 2022, pp. 5627–5636. arXiv
36. Z. Liang, C. Li, S. Zhou, R. Feng and C. C. Loy, “Iterative prompt learning for unsupervised backlit image enhancement,” in *Proc. IEEE/CVF ICCV*, 2023. arXiv
37. Y. Wang, R. Wan, W. Yang, H. Li, L.-P. Chau and A. C. Kot, “Low-light image enhancement with normalizing flow,” in *Proc. AAAI Conference on Artificial Intelligence*, vol. 36, no. 3, pp. 2604–2612, 2022. arXiv
38. X. Yi, H. Xu, H. Zhang, L. Tang and J. Ma, “Diff-Retinex: Rethinking low-light image enhancement with a generative diffusion model,” in *Proc. IEEE/CVF ICCV*, 2023. arXiv
39. J. Hou, Z. Zhu, J. Hou, H. Liu, H. Zeng and H. Yuan, “Global structure-aware diffusion process for low-light image enhancement,” in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2023. arXiv
40. H. Jiang, A. Luo, X. Liu, S. Han and S. Liu, “LightenDiffusion: Unsupervised low-light image enhancement with latent-Retinex diffusion models,” in *Proc. European Conference on Computer Vision* (*ECCV*), 2024. arXiv
41. C. He, C. Fang, Y. Zhang, et al., “Reti-Diff: Illumination degradation image restoration with Retinex-based latent diffusion model,” in *Proc. International Conference on Learning Representations* (*ICLR*), 2025. proceedings
42. J. Bai, Y. Yin, Q. He, Y. Li and X. Zhang, “RetinexMamba: Retinex-based Mamba for low-light image enhancement,” arXiv:2405.03349, 2024. arXiv
43. M. V. Conde, G. Geigle and R. Timofte, “InstructIR: High-quality image restoration following human instructions,” in *Proc. European Conference on Computer Vision* (*ECCV*), 2024. arXiv
44. C. Chen, Q. Chen, J. Xu and V. Koltun, “Learning to see in the dark,” in *Proc. IEEE/CVF CVPR*, 2018, pp. 3291–3300. arXiv
45. Z. Cui, G.-J. Qi, L. Gu, S. You, Z. Zhang and T. Harada, “Multitask AET with orthogonal tangent regularity for dark object detection,” in *Proc. IEEE/CVF ICCV*, 2021. arXiv
46. K. A. Hashmi, G. Kallempudi, D. Stricker and M. Z. Afzal, “FeatEnHancer: Enhancing hierarchical features for object detection and beyond under low-light vision,” in *Proc. IEEE/CVF ICCV*, 2023. arXiv
47. Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004. doi
48. R. Zhang, P. Isola, A. A. Efros, E. Shechtman and O. Wang, “The unreasonable effectiveness of deep features as a perceptual metric,” in *Proc. IEEE/CVF CVPR*, 2018. arXiv
49. A. Mittal, R. Soundararajan and A. C. Bovik, “Making a ‘completely blind’ image quality analyzer,” *IEEE Signal Processing Letters*, vol. 20, no. 3, pp. 209–212, 2013. doi
50. A. Mittal, A. K. Moorthy and A. C. Bovik, “No-reference image quality assessment in the spatial domain,” *IEEE Transactions on Image Processing*, vol. 21, no. 12, pp. 4695–4708, 2012. doi
51. J. Liao, S. Hao, R. Hong and M. Wang, “GT-Mean loss: A simple yet effective solution for

brightness mismatch in low-light image enhancement," in *Proc. IEEE/CVF ICCV*, 2025. arXiv
52. W. Yang, W. Wang, H. Huang, S. Wang and J. Liu, "Sparse gradient regularized deep Retinex network for robust low-light image enhancement," *IEEE Transactions on Image Processing*, vol. 30, pp. 2072–2086, 2021. doi
53. J. Hai, Z. Xuan, R. Yang, et al., "R2RNet: Low-light image enhancement via Real-low to Real-normal Network," *Journal of Visual Communication and Image Representation*, vol. 90, art. 103712, 2023. arXiv
54. R. Wang, X. Xu, C.-W. Fu, et al., "Seeing dynamic scene in the dark: A high-quality video dataset with mechatronic alignment," in *Proc. IEEE/CVF ICCV*, 2021, pp. 9680–9689. doi code
55. V. Bychkovsky, S. Paris, E. Chan and F. Durand, "Learning photographic global tonal adjustment with a database of input/output image pairs," in *Proc. IEEE CVPR*, 2011, pp. 97–104. dataset
56. J. Cai, S. Gu and L. Zhang, "Learning a deep single image contrast enhancer from multi-exposure images," *IEEE Transactions on Image Processing*, vol. 27, no. 4, pp. 2049–2062, 2018. doi
57. Y. P. Loh and C. S. Chan, "Getting to know low-light images with the Exclusively Dark dataset," *Computer Vision and Image Understanding*, vol. 178, pp. 30–42, 2019. arXiv
58. K. Ma, K. Zeng and Z. Wang, "Perceptual quality assessment for multi-exposure image fusion," *IEEE Transactions on Image Processing*, vol. 24, no. 11, pp. 3345–3356, 2015. doi
59. C. Lee, C. Lee and C.-S. Kim, "Contrast enhancement based on layered difference representation of 2D histograms," *IEEE Transactions on Image Processing*, vol. 22, no. 12, pp. 5372–5384, 2013. doi
60. V. Vonikakis, "Datasets" (source of the VV test images), personal web page. link
61. X. Liu, Z. Wu, A. Li, et al., "NTIRE 2024 challenge on low light image enhancement: Methods and results," in *Proc. IEEE/CVF CVPR Workshops*, 2024. arXiv
62. X. Liu, Z. Wu, F.-A. Vasluianu, et al., "NTIRE 2025 challenge on low light image enhancement: Methods and results," in *Proc. IEEE/CVF CVPR Workshops*, 2025. arXiv
63. Y. Blau, R. Mechrez, R. Timofte, T. Michaeli and L. Zelnik-Manor, "The 2018 PIRM challenge on perceptual image super-resolution," in *Proc. European Conference on Computer Vision (ECCV) Workshops*, 2018. arXiv
64. C. Ma, C.-Y. Yang, X. Yang and M.-H. Yang, "Learning a no-reference quality metric for single-image super-resolution," *Computer Vision and Image Understanding*, vol. 158, pp. 1–16, 2017. doi arXiv
65. D. Hasler and S. E. Süsstrunk, "Measuring colorfulness in natural images," in *Proc. SPIE 5007, Human Vision and Electronic Imaging VIII*, 2003. doi
66. J. Immerkær, "Fast noise variance estimation," *Computer Vision and Image Understanding*, vol. 64, no. 2, pp. 300–302, 1996. doi
67. H. Talebi and P. Milanfar, "NIMA: Neural image assessment," *IEEE Transactions on Image Processing*, 2018. arXiv
68. J. Ke, Q. Wang, Y. Wang, P. Milanfar and F. Yang, "MUSIQ: Multi-scale image quality transformer," in *Proc. IEEE/CVF ICCV*, 2021. arXiv
69. J. Wang, K. C. K. Chan and C. C. Loy, "Exploring CLIP for assessing the look and feel of images," in *Proc. AAAI Conference on Artificial Intelligence*, 2023. arXiv
70. S. Yang, T. Wu, S. Shi, et al., "MANIQA: Multi-dimension attention network for no-reference

image quality assessment," in *Proc. IEEE/CVF CVPR Workshops*, 2022, pp. 1190–1199. doi arXiv
71. H. Wu, Z. Zhang, W. Zhang, C. Chen, et al., "Q-Align: Teaching LMMs for visual scoring via discrete text-defined levels," in *Proc. International Conference on Machine Learning (ICML)*, PMLR 235, 2024. arXiv
72. V. Kannan, S. Malik, N. C. Babu and R. Soundararajan, "Quality assessment of low-light restored images: A subjective study and an unsupervised model," *IEEE Access*, vol. 11, pp. 68216–68230, 2023. doi arXiv
73. ITU-R, "Recommendation BT.500-15: Methodologies for the subjective assessment of the quality of television images," 2023. ITU
74. R. A. Bradley and M. E. Terry, "Rank analysis of incomplete block designs: The method of paired comparisons," *Biometrika*, vol. 39, no. 3–4, pp. 324–345, 1952. doi
75. C. Chen and J. Mo, "IQA-PyTorch: PyTorch toolbox for image quality assessment," software (pip package `pyiqa`). GitHub

DLCV 06・物件追蹤：從單一物件到多物件

電腦視覺如何在影片中跟住物體：重識別、從相關濾波器到 Transformer 與 SAM 2 的單物件追蹤，以及結合卡爾曼濾波器、匈牙利演算法、端到端模型與 MOTA ／ IDF1 ／ HOTA 指標的多物件追蹤。

網站版本: https://blog.shyandram.dev/zh-tw/learn/deep-learning/dl-06-object-tracking/

你將學到

- 追蹤在「分類 → 偵測」之後的位置，以及每個追蹤器都要做的兩件事：感知跨影格的運動、維持每個物件的身分。
- 一套清楚的分類：重識別（re-identification）與時序追蹤；單物件追蹤（SOT ／ VOT）與多物件追蹤（MOT）；以及多目標跨攝影機追蹤（MTMCT）。
- 單物件追蹤的演進：相關濾波器（MOSSE、KCF）、孿生網路（Siamese；SiamFC、SiamRPN）、單流 Transformer（OSTrack、SeqTrack），以及影片分割模型（SAM 2、SAMURAI、SAM 3）。
- 深入多物件追蹤：「先偵測再追蹤」（tracking-by-detection）流程（偵測器、卡爾曼濾波器、外觀嵌入、匈牙利演算法）、SORT 家族（SORT、DeepSORT、ByteTrack、OC-SORT、BoT-SORT）、聯合偵測與嵌入（JDE、FairMOT）、以 query 為基礎的端到端追蹤器（MOTR、MOTRv2、MOTIP），以及開放詞彙追蹤。
- 怎麼評估追蹤器：Re-ID 的 CMC 與 mAP、SOT 的 success 與 precision、MOT 的 MOTA、IDF1 與 HOTA，以及 HOTA 為什麼會被提出。
- 一個可以直接執行的 NumPy 追蹤器，並在合成場景上實際量測 ID 切換與各項指標。

先看全貌

偵測器看一張圖，告訴你「這裡有三個人」。追蹤器看一段影片，告訴你「這個人和一秒前那個是**同一個**，而那一位剛走到柱子後面，等一下會再出來」。追蹤就是在偵測之上加入時間與記憶。

用最簡單的話說

想像你和朋友一起看足球賽，各自挑一位最喜歡的球員。在一張照片裡找出場上所有球員，是**偵測**。眼睛一直盯著**你的**球員，看她跑、轉身、被別人擋住一下又出現，是**單物件追蹤**。幫**每一位**球員都貼上號碼貼紙，而且就算兩個人撞在一起，貼紙也絕對不會貼錯，是**多物件追蹤**。電視切到另一台攝影機時，你還認得出你的球員，就是**重識別**。

為什麼重要：自駕車要靠追蹤才知道行人是**正走向**馬路，而不只是站在路邊；運動分析要靠追蹤才算得出每位球員跑了多遠；倉儲要靠追蹤來清點堆高機和機器人；影片剪輯要靠追蹤讓遮罩黏在移動的物體上。這些應用不只要知道東西**在哪裡**，還要知道**誰是誰**、隨時間怎麼變化。

關於本篇

本篇由作者先前在 Medium 發表的文章〈物件追蹤 Object Tracking 簡介〉[1] 擴充而成，保留原文架構，並補上技術細節、可執行的追蹤器與最新的文獻回顧。原文請見這裡。

從分類、偵測到追蹤

白話版。分類回答「圖裡**是什麼**」；偵測回答「是什麼、**在哪裡**」；追蹤回答「是什麼、在哪裡、**是哪一個**，而且隨時間持續」。

[1] 的這條路線，正好是電腦視覺任務的一張地圖：

1. **影像分類**把影像 I 對應到一個標籤 y。
2. **物件偵測**把影像對應到一組帶有類別與信心分數的框，$\mathcal{D} = \{(b_j, c_j, s_j)\}_{j=1}^{N}$，其中 b_j 是邊界框，c_j 是類別，$s_j \in [0,1]$ 是分數。偵測模仿的是人類視覺判斷畫面中目標位置的能力，詳細內容請見DLCV 03・物件偵測。
3. **物件追蹤**輸入影格序列 $I_1, \dots, I_T$，輸出**軌跡**：對每個物件 k，給出它在可見影格集合 $\mathcal{T}_k$ 上的一串框（或遮罩）$(b_t^k)_{t \in \mathcal{T}_k}$，而且全部帶著同一個身分 k。

新加入的關鍵元素是身分標籤。偵測是無狀態的：第 t 格與第 $t+1$ 格各自獨立處理。追蹤則是有狀態的：它要把資訊帶到下一格。

追蹤在做什麼？

白話版。追蹤模仿人類視覺對運動物體的感知：我們會預測東西接下來往哪裡去、往那裡看，並在它外觀改變或短暫被擋住時，依然把它當成同一個東西。

更精確地說，每個追蹤器都在解兩個互相耦合的子問題：

- **隨時間定位**（運動感知）：估計每個目標在每一格中的位置，最好能用過去預測未來。這讓追蹤器能撐過模糊、漏偵測與短暫遮擋。
- **維持身分**（資料關聯，data association）：決定第 t 格的哪個觀測屬於第 $t-1$ 格的哪個目標。這能防止兩個交錯而過的人互換標籤。

單物件追蹤因為只有一個身分，主要力氣花在第一個問題；多物件追蹤主要在處理第二個問題；重識別則是**完全沒有**運動資訊時的第二個問題。

[1] 對各類方法列出的困難，都來自這兩件事：外觀變化（姿態、光照、尺度、視角）、被其他物體或場景遮擋、長得很像的干擾物、快速或非線性的運動、攝影機本身的移動，以及目標進出畫面。

追蹤的分類

白話版。追蹤器可以依照**追什麼**（一個或多個物件、一台或多台攝影機）來分，也可以依照**怎麼維持身分**（只靠外觀，或是逐格跟著運動）來分。

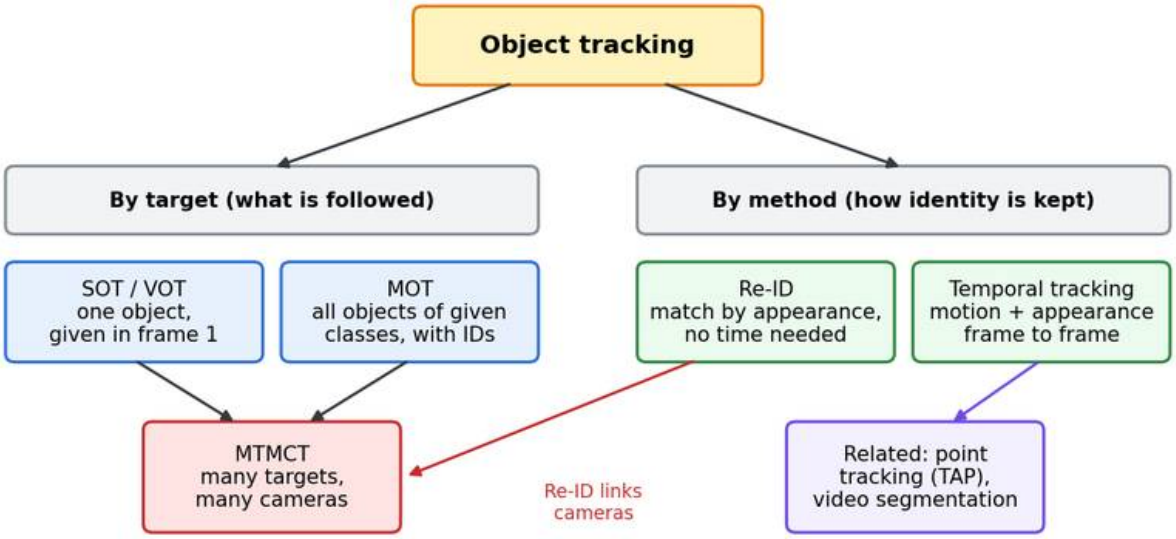


圖 *1* — 追蹤的分類。依目標：單物件（*SOT*，又稱 *VOT*）與多物件（*MOT*）。依方法：只靠外觀的重識別，以及時序追蹤。兩者結合後最困難的情境是多目標跨攝影機追蹤（*MTMCT*）。

依照 [1]：

- **依作法**。重識別（*Re-ID*）只靠外觀（有時加上姿態），在時間相隔很遠、或來自不同攝影機的影像之間比對同一個目標。時序物件追蹤則在同一段影片中逐格跟隨目標，可以利用運動的連續性。
- **依目標**。視覺物件追蹤（*visual object tracking, VOT*）又稱單物件追蹤（*single-object tracking, SOT*），追蹤第一格中以框指定的任意一個物件。多物件追蹤（*multi-object tracking, MOT*）則在**連續影格**中追蹤指定類別的所有物件（例如每一位行人），並為每個物件維持一致的 ID。
- **終極目標**。多目標跨攝影機追蹤（*Multi-Target Multi-Camera Tracking, MTMCT*）在整個攝影機網路中追蹤多個目標，當一個人離開某台攝影機、出現在另一台時，仍維持相同身分。它結合了單一攝影機內的 MOT 與攝影機之間的 Re-ID。

傳統文獻與大多數基準資料集都以**行人**為主要目標 [1]，因為監控與自駕車讓「人」成為最有價值的追蹤類別。新的基準則加入了舞者、運動員、車輛與開放詞彙類別。

重識別（Re-ID）

白話版。 Re-ID 是一個搜尋問題：給你某台攝影機拍到的一個人，請在其他攝影機拍到的照片中找出同一個人，並由最像到最不像排序。

問題定義

Re-ID 被定義成**檢索**（retrieval）問題 [7]。查詢影像（query）q 與由其他攝影機偵測出的行人圖庫（gallery）$\mathcal{G} = \{g_1, \dots, g_M\}$，都經由網路 f_θ 映射成嵌入向量，再依距離排序圖庫，通常使用餘弦距離

$$d(q, g) = 1 - \frac{f_\theta(q)^\top f_\theta(g)}{\|f_\theta(q)\| \, \|f_\theta(g)\|},$$

其中 $f_\theta(\cdot) \in \mathbb{R}^D$ 是參數為 θ 的網路輸出的 D 維嵌入。訓練的目標是讓同一身分的嵌入靠近、不同身分的嵌入遠離。常見做法是把身分分類損失（對 C 個訓練身分做交叉熵）與三**元組損失**（triplet loss）結合：

$$\mathcal{L}_{\text{tri}} = \left[\, d(a, p) - d(a, n) + m \,\right]_+,$$

其中 a 是錨點影像，p 是正樣本（同一身分），n 是負樣本（不同身分），$m > 0$ 是邊界（margin），$[x]_+ = \max(x, 0)$。測試時的身分從未在訓練中出現，所以 Re-ID 是開放集合（open-set）問題：網路必須學到一般性的「是不是同一個人」。

挑戰

[1] 把困難分成**資料因素**與**環境因素**。資料因素：標註身分數量有限、偵測器給出的框有雜訊、身體只拍到一部分、解析度低。環境因素：視角與各攝影機色彩響應不同、光照、遮擋、雜亂背景，以及換了衣服的人。Re-ID 也帶來明顯的**隱私**疑慮，這也是一些監控型資料集被撤下的原因之一（見下文）。原文把城市級監控系統列為主要應用；如今 Re-ID 同樣常見於 MOT 追蹤器裡的外觀模組，以及零售與運動分析。

代表模型

- **強基線（strong baseline）**。 Luo 等人 [14] 證明：一個普通的 ResNet-50，搭配一組訓練「技巧」——學習率暖身（warm-up）、隨機擦除（random erasing）資料增強、標籤平滑（label smoothing）、最後一層 stride 設為 1 以得到更大的特徵圖，以及 **BNNeck**（一層批次正規化，把三元組損失用的特徵與分類用的特徵分開）——就是一個強大且可重現的基線。
- **質心三元組損失（centroid triplet loss, CTL）**。原文提到的模型（ResNet-50，輸入 256×128）[15]，把「樣本對樣本」的三元組換成「樣本對類別質心」的距離，檢索時也是拿查詢影像與各身分的質心比較，而不是與每一張圖庫影像比較。
- **CLIP-ReID**。 Li 等人 [16] 把視覺語言模型 CLIP 用在沒有文字標籤的 Re-ID 上。第一階段凍結編碼器，學習每個身分專屬的文字 token；第二階段再以這些文字特徵作為額外監督，微調影像編碼器。

資料集

資料集	內容	狀態
Market-1501 [12]	1,501 個身分、6 台攝影機、偵測器裁切的框	可取得
MSMT17 [13]	4,101 個身分、15 台室內外攝影機	可取得
DukeMTMC [10]（及 DukeMTMC-reID）	8 台同步的校園攝影機	**2019 年已撤下**

DukeMTMC 與跨攝影機追蹤的 IDF1 指標一起被提出 [10]，其中的 Re-ID 子集也被廣泛使用。由於在校園拍攝行人引發的隱私與倫理疑慮，作者已於 2019 年撤下該資料集，不應再下載或使用；Peng 等人 [11] 追蹤了它在撤下後仍以衍生版本流傳的情形，並主張資料集需要更好的管理（stewardship）。

指標：CMC rank-k 與 mAP

累積匹配特性（cumulative matching characteristic, CMC）在 rank k 的值，是「第一個正確匹配出現在排序後圖庫前 k 名」的查詢比例：

$$\mathrm{CMC}(k) = \frac{1}{|Q|}\sum_{q \in Q} \mathbb{1}\left[\, r_q \le k \,\right],$$

其中 Q 是查詢集合，r_q 是查詢 q 的第一個正確匹配的名次。*Rank-1* 準確率就是 $\mathrm{CMC}(1)$。由於一個查詢在圖庫中通常有好幾個正確匹配，只看 rank-k 會忽略**其他**正確匹配排得好不好。**平均精確度均值**（mean average precision, mAP）補上了這一點：

$$\mathrm{AP}(q) = \frac{1}{G_q}\sum_{i=1}^{G_q} \frac{i}{r_{q,i}}, \qquad \mathrm{mAP} = \frac{1}{|Q|}\sum_{q} \mathrm{AP}(q),$$

其中 G_q 是 q 在圖庫中的正確匹配數，$r_{q,i}$ 是第 i 個正確匹配的名次，所以 $i / r_{q,i}$ 就是找到第 i 個正確匹配那一刻的精確度。

```
import numpy as np

def cmc_map(dist, q_ids, g_ids, max_rank=5):
    """dist: (Q, G) distances between query and gallery embeddings."""
    order = np.argsort(dist, axis=1)                # closest gallery items first
    match = g_ids[order] == q_ids[:, None]          # (Q, G) True where same person
    cmc = (np.cumsum(match, axis=1) > 0).mean(axis=0)[:max_rank]
    aps = []
    for row in match:
        hits = np.flatnonzero(row)                  # ranks (0-based) of true matches
        precision_at_hits = np.arange(1, len(hits) + 1) / (hits + 1)
        aps.append(precision_at_hits.mean())
    return cmc, np.mean(aps)

rng = np.random.default_rng(0)
centres = rng.normal(size=(50, 64))                 # 50 identities, 64-D embeddings
g_ids = np.repeat(np.arange(50), 4)                 # 4 gallery images per identity
```

```
q_ids = np.arange(50)
gallery = centres[g_ids] + 1.6 * rng.normal(size=(200, 64))
query = centres[q_ids] + 1.6 * rng.normal(size=(50, 64))
norm = lambda x: x / np.linalg.norm(x, axis=1, keepdims=True)
dist = 1 - norm(query) @ norm(gallery).T            # cosine distance
cmc, mAP = cmc_map(dist, q_ids, g_ids)
print("rank-1 %.2f  rank-5 %.2f  mAP %.2f" % (cmc[0], cmc[4], mAP))
```

在這個合成圖庫（每個身分 4 張）上，程式輸出 `rank-1 0.76  rank-5 0.92  mAP 0.57`：大多數查詢都能在前面找到一**張**正確影像，但另外三張散落在後面，只有 mAP 看得出來。

單物件追蹤

白話版。你在第一格框住某個東西，追蹤器必須在接下來整段影片中一直把框放在同一個東西上，而且事先不知道它是哪一類物體。

問題定義

給定影格 $I_1, \dots, I_T$ 與初始框 b_1，單物件追蹤器輸出 $\hat{b}_2, \dots, \hat{b}_T$。目標**與類別無關**：可能是人、鳥或咖啡杯。因此 SOT 是一種 *one-shot* 學習：目標唯一的範例就是第一格的影像塊，也就是**模板**（template）z。在每個新影格，追蹤器在上一個位置附近的搜尋區域 x 中尋找目標。[1] 列出的挑戰是：複雜的外觀變化、移動模式，以及光照、遮擋等環境變化。

傳統方法：相關濾波器

在深度學習之前，最快的追蹤器會線上學習一個**相關濾波器**（correlation filter）。MOSSE [17] 在頻率域學習濾波器 H，讓它與訓練影像塊 F_i 做相關後，在目標中心產生尖銳的高斯峰 G_i：

$$\min_{H^*} \sum_i |F_i \odot H^* - G_i|^2 \quad \Rightarrow \quad H^* = \frac{\sum_i G_i \odot F_i^*}{\sum_i F_i \odot F_i^*},$$

其中大寫字母代表二維離散傅立葉轉換，$\odot$ 是逐元素相乘，* 是複數共軛，除法也是逐元素進行。因為相關運算在頻率域變成乘法（見IP 04），訓練與偵測只需要幾次 FFT，在 CPU 上就能達到每秒數百格。KCF [18] 指出，用影像塊的**所有循環位移**來訓練，會得到一個可以被 DFT 對角化的循環矩陣，於是核嶺迴歸（kernel ridge regression）變成逐元素運算，也能使用 HOG 等多通道特徵。它們的弱點是搜尋視窗固定、循環假設造成的邊界效應，以及人工設計的特徵。

孿生網路追蹤器

白話版。離線用大量影片訓練一個網路，只回答一個問題：「模板出現在這張搜尋影像的哪裡？」測試時直接凍結使用。

SiamFC [19] 把**同一個**全卷積網路 φ 同時套用在模板 z 與搜尋區域 x 上，再把兩張特徵圖做互相關：

$$f(z, x) = \varphi(z) \star \varphi(x) + b\,\mathbb{1},$$

其中 $\star$ 是互相關（模板特徵扮演卷積核），b 是可學習的偏差，$\mathbb{1}$ 是全為 1 的圖。輸出是一張分數圖，峰值位置就是新位置。訓練時使用邏輯損失（logistic loss），真實中心附近標為正、其他位置標為負。由於不做線上更新，它速度快、也不容易漂移，但只能預測位置，尺度則靠搜尋一個小型影像金字塔來處理。

SiamRPN [20] 在孿生特徵之後加上**區域提議網路**（region proposal network）：一個分支判斷錨框是目標還是背景，另一個分支迴歸框的偏移量。追蹤因此變成以模板定義類別的「one-shot 偵測」，能輸出長寬比會變化的精確框。

Transformer 單流追蹤器

2022 年後，主流從孿生 CNN 轉向 Transformer。關鍵差別在於模板與搜尋區域**在哪裡**互動（圖 2）。雙流（two-stream）設計中，兩者分別編碼，最後才比較一次。在 OSTrack [21] 這類**單流**（one-stream）設計中，z 與 x 的影像 patch 被串成同一個 token 序列，由同一個 Vision Transformer 處理，因此每一層注意力都同時在擷取特徵、也在建立模板與搜尋區域之間的關係。OSTrack 也會在前幾層丟掉明顯是背景的搜尋 token 以節省計算。SeqTrack [22] 更進一步簡化：由編碼器—解碼器 Transformer 以自迴歸方式，把框的四個座標當成離散 token 序列逐一生成，只用一般的交叉熵訓練——與語言模型的做法相同。

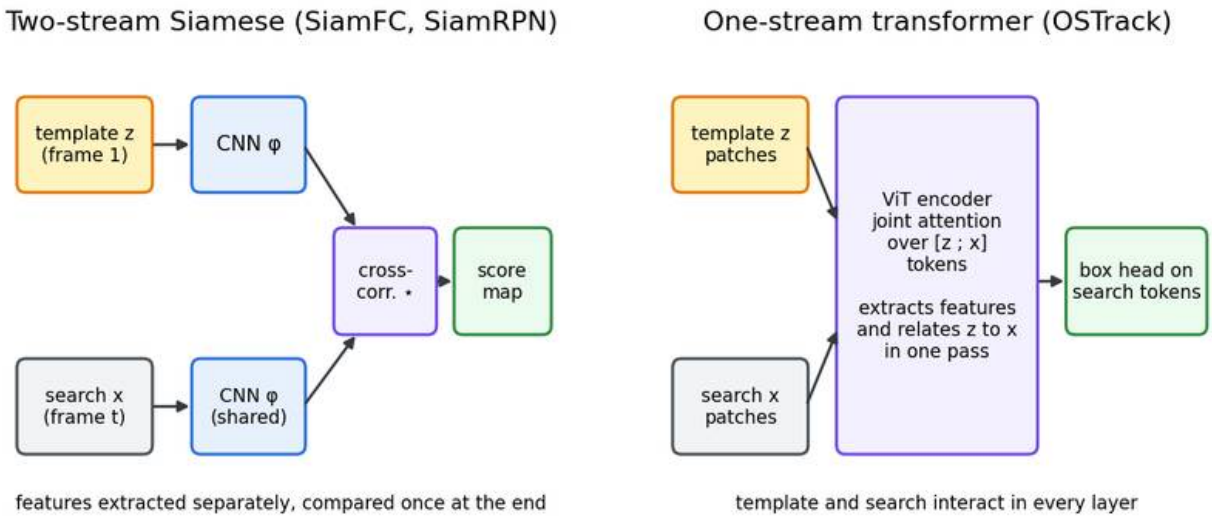


圖 2 — 雙流孿生追蹤器只在最後比較一次模板與搜尋特徵；單流 *Transformer* 追蹤器讓兩者在每一層都互動。

用影片分割模型做追蹤

白話版。新一代「影片中分割一切」的模型，只要你點一下說「就是這個」，就能用像素遮罩在整段影片中跟住它。這**就是**單物件追蹤，只是輸出遮罩而不是框。

SAM 2 [23] 把 Segment Anything 模型延伸到影片，加入**串流記憶**（streaming memory）：過去影格的特徵與預測遮罩，連同被提示的影格，都存在記憶庫中，供目前影格做注意力。只要在一格中用點、框或遮罩提示，就能得到貫穿整段影片的「masklet」，行為就像輸出遮罩的 SOT 追蹤器。SAMURAI [24] 不需重新訓練就把 SAM 2 改造成追蹤器：它加入卡爾曼濾波器運動模型，從 SAM 2 的多個候選遮罩中挑選，並用考慮運動的規則決定哪些過去影格進入記憶，這在有相似干擾物的擁擠場景特別有幫助。SAM 3 [25] 加入**可提示概念分割**（promptable concept segmentation）：給一個名詞片語，例如「黃色校車」，或一張範例影像，它就能在影片中偵測、分割並追蹤**所有**符合的實例，並給每個實例各自的身分。這已經是一種開放詞彙 MOT，SOT 與 MOT 的界線也因此更加模糊。

基準資料集與指標

基準	特色
OTB [26]	早期的標準基準，每段序列都有屬性標記（遮擋、快速運動等）；提出 success plot 與 precision plot
VOT challenge [53]	自 2013 年起每年舉辦的競賽，採用失敗後重新初始化的評估協定；現為 VOTS，同時評估追蹤**與**分割
LaSOT [27]	長時追蹤：1,400 段序列、超過 350 萬格，每一格都由人工標註
GOT-10k [28]	one-shot 協定：訓練與測試的物件類別互不重疊
TrackingNet [29]	大規模、真實場景影片，以獨立的評估伺服器評分

兩個經典指標來自 OTB [26]：

$$\text{Success}(\tau) = \frac{1}{T}\sum_{t=1}^{T} \mathbb{1}[\,\text{IoU}(\hat{b}_t, b_t) > \tau], \qquad \text{AUC} = \int_0^1 \text{Success}(\tau)\, d\tau,$$

$$\text{Precision}(\delta) = \frac{1}{T}\sum_{t=1}^{T} \mathbb{1}[\|\hat{c}_t - c_t\|_2 \le \delta],$$

其中 $\hat{b}_t, b_t$ 是預測框與真實框，τ 是 IoU 門檻，$\hat{c}_t, c_t$ 是兩者的中心，δ 是像素距離，慣例取 20。success plot 的 AUC 是 OTB 與 LaSOT 的主要數字。GOT-10k 則報告**平均重疊**（average overlap，即平均 IoU），以及 $\tau = 0.5$ 與 0.75 時的成功率。

```
import numpy as np

def iou_xywh(a, b):
    """IoU between rows of a and b, boxes as [x, y, w, h] (top-left corner)."""
    x1 = np.maximum(a[:, 0], b[:, 0]); y1 = np.maximum(a[:, 1], b[:, 1])
    x2 = np.minimum(a[:, 0] + a[:, 2], b[:, 0] + b[:, 2])
```

```
    y2 = np.minimum(a[:, 1] + a[:, 3], b[:, 1] + b[:, 3])
    inter = np.clip(x2 - x1, 0, None) * np.clip(y2 - y1, 0, None)
    return inter / (a[:, 2] * a[:, 3] + b[:, 2] * b[:, 3] - inter)

def success_auc(pred, gt, thresholds=np.linspace(0, 1, 21)):
    ious = iou_xywh(pred, gt)
    curve = np.array([(ious > t).mean() for t in thresholds])
    return curve.mean()                                  # area under the success plot

def precision_at(pred, gt, px=20):
    c_pred = pred[:, :2] + pred[:, 2:] / 2; c_gt = gt[:, :2] + gt[:, 2:] / 2
    return (np.linalg.norm(c_pred - c_gt, axis=1) <= px).mean()

rng = np.random.default_rng(0)
gt = np.c_[np.linspace(50, 400, 300), np.full(300, 80.0), np.full(300, 40.0), np.full(300,
 ↪  60.0)]
pred = gt + rng.normal(0, 4, gt.shape)
pred[200:230, :2] += 120                                 # 30 frames lost on a distractor
print("success AUC %.3f   precision@20px %.3f" % (success_auc(pred, gt), precision_at(pred,
 ↪  gt)))
```

輸出為 `success AUC 0.647   precision@20px 0.900`：跟到干擾物上的 30 格讓 precision 掉了 10 個百分點；而其他影格上的小幅抖動對 AUC 的影響大得多，因為 AUC 也會獎勵貼得很緊的框。

多物件追蹤

白話版。在每一格找出所有感興趣的物件，並給每個物件一個號碼，只要它還在畫面中，號碼就不變——即使物件彼此交錯、互相遮擋，或離開後又回來。

MOT 是在**連續影格**中同時追蹤多個目標，並為每個目標維持一致的 ID；單一畫格本身只能做偵測。[1] 列出的挑戰包括：頻繁遮擋、軌跡何時開始與結束、目標之間外觀相似，以及目標之間的相互影響（人群中的人會一起移動、彼此遮擋）。

形式上，給定每一格的偵測 $\mathcal{D}_t$，MOT 要找出一組軌跡 $\mathcal{T} = \{\tau_k\}$，把真實的偵測分組、丟掉誤報，並補上漏掉的偵測。離線（offline）方法對整段影片一起求解，常寫成圖或網路流問題；本節著重的**線上**（online）方法，必須只用第 $1..t$ 格在第 t 格就做出決定，這正是即時應用所需要的。

「先偵測再追蹤」流程

十年來的主流設計是**先偵測再追蹤**（tracking-by-detection）[2]、[4]：每一格都跑偵測器，再把偵測結果跨時間連起來（圖 3）。每一格經過四個步驟：

1. **偵測**。偵測器（Faster R-CNN、YOLO 系列、DETR 系列；見DLCV 03）輸出框與分數。
2. **預測**。運動模型把每條現有軌跡往前推到現在應該在的位置。
3. **描述**。（可選）用 Re-ID 網路把每個偵測框裁切出的影像轉成嵌入向量。

4. **關聯**。在預測的軌跡與偵測之間建立代價矩陣，解指派問題。匹配到的軌跡被更新；沒匹配到的偵測可能開啟新軌跡（**出生**）；太久沒匹配到的軌跡被刪除（**死亡**）。

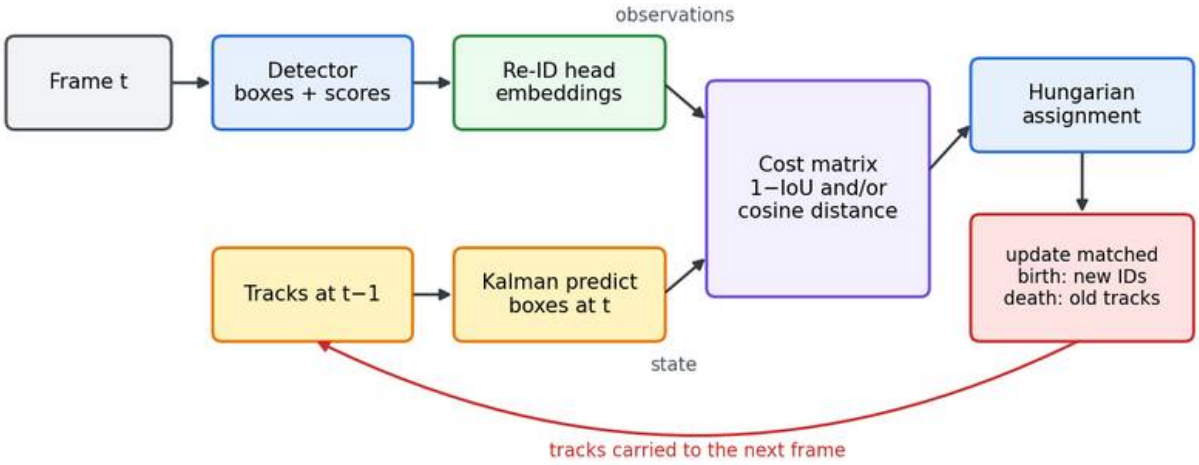


圖 3 — 線上的先偵測再追蹤流程。觀測（框、嵌入）與狀態（預測的軌跡）在代價矩陣中會合，指派結果再更新下一格要用的狀態。

運動模型：卡爾曼濾波器

白話版。先根據物體上一刻的位置與速度猜它現在在哪；再把這個猜測往偵測結果拉一點，而且比較不確定的那一方，就比較不被相信。

幾乎所有先偵測再追蹤的系統，都使用等速模型的**線性卡爾曼濾波器**（Kalman filter）[30]。在我們的程式中，一條軌跡的狀態是

$$\mathbf{x} = [\, c_x,\, c_y,\, w,\, h,\, v_x,\, v_y \,]^\top,$$

其中 (c_x, c_y) 是框的中心，w, h 是寬與高，(v_x, v_y) 是中心的速度（像素／格）。（SORT 使用中心、面積與長寬比，而不是 w, h [32]；BoT-SORT 又改回 w 與 h [36]。）模型分兩步。

預測（時間更新）：

$$\hat{\mathbf{x}}_{t|t-1} = \mathbf{F}\,\hat{\mathbf{x}}_{t-1|t-1},$$
$$\mathbf{P}_{t|t-1} = \mathbf{F}\,\mathbf{P}_{t-1|t-1}\,\mathbf{F}^\top + \mathbf{Q},$$

其中 $\mathbf{F}$ 是 6×6 的狀態轉移矩陣（單位矩陣再加上幾個 1，把 v_x, v_y 加到 c_x, c_y 上，因為 $\Delta t = 1$ 格），$\mathbf{P}$ 是狀態共變異數（我們的不確定性），$\mathbf{Q}$ 是過程雜訊共變異數（真實運動可以偏離等速多少）。

更新（量測更新），使用匹配到的偵測 $\mathbf{z}_t = [c_x, c_y, w, h]^\top$：

$$\mathbf{S}_t = \mathbf{H}\,\mathbf{P}_{t|t-1}\,\mathbf{H}^\top + \mathbf{R}, \qquad \mathbf{K}_t = \mathbf{P}_{t|t-1}\,\mathbf{H}^\top\,\mathbf{S}_t^{-1},$$
$$\hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1} + \mathbf{K}_t(\mathbf{z}_t - \mathbf{H}\,\hat{\mathbf{x}}_{t|t-1}), \qquad \mathbf{P}_{t|t} = (\mathbf{I} - \mathbf{K}_t\mathbf{H})\,\mathbf{P}_{t|t-1},$$

其中 $\mathbf{H} = [\,\mathbf{I}_4 \;\; \mathbf{0}\,]$ 取出狀態中可觀測的部分，$\mathbf{R}$ 是偵測器的量測雜訊共變異數，$\mathbf{z}_t - \mathbf{H}\hat{\mathbf{x}}_{t|t-1}$ 是**創新量**（innovation，也就是「意外程度」），$\mathbf{S}_t$ 是它的共變異數，$\mathbf{K}_t$ 是**卡爾曼增益**。若預測非常不確定（$\mathbf{P}$ 很大），更新會幾乎把估計值拉到偵測結果上；若偵測器雜訊很大（$\mathbf{R}$ 很大），就比較相信預測。偵測缺漏時，追蹤器只做預測步驟（稱為「滑行」，coasting），$\mathbf{P}$ 會持續變大（圖 4）。

```
import numpy as np
from scipy.optimize import linear_sum_assignment

# ---------- 1. constant-velocity Kalman filter ----------
class KalmanBox:
    """State x = [cx, cy, w, h, vx, vy]; measurement z = [cx, cy, w, h]."""
    def __init__(self, z, q=1.0, r=4.0):
        self.x = np.r_[z, 0.0, 0.0]
        self.P = np.diag([r, r, r, r, 100.0, 100.0])    # unknown velocity: large variance
        self.F = np.eye(6); self.F[0, 4] = self.F[1, 5] = 1.0   # dt = 1 frame
        self.H = np.eye(4, 6)
        self.Q = q * np.diag([1, 1, 0.1, 0.1, 0.5, 0.5])
        self.R = r * np.eye(4)

    def predict(self):
        self.x = self.F @ self.x
        self.P = self.F @ self.P @ self.F.T + self.Q
        return self.x[:4]

    def update(self, z):
        S = self.H @ self.P @ self.H.T + self.R          # innovation covariance
        K = self.P @ self.H.T @ np.linalg.inv(S)          # Kalman gain
        self.x = self.x + K @ (z - self.H @ self.x)
        self.P = (np.eye(6) - K @ self.H) @ self.P
```

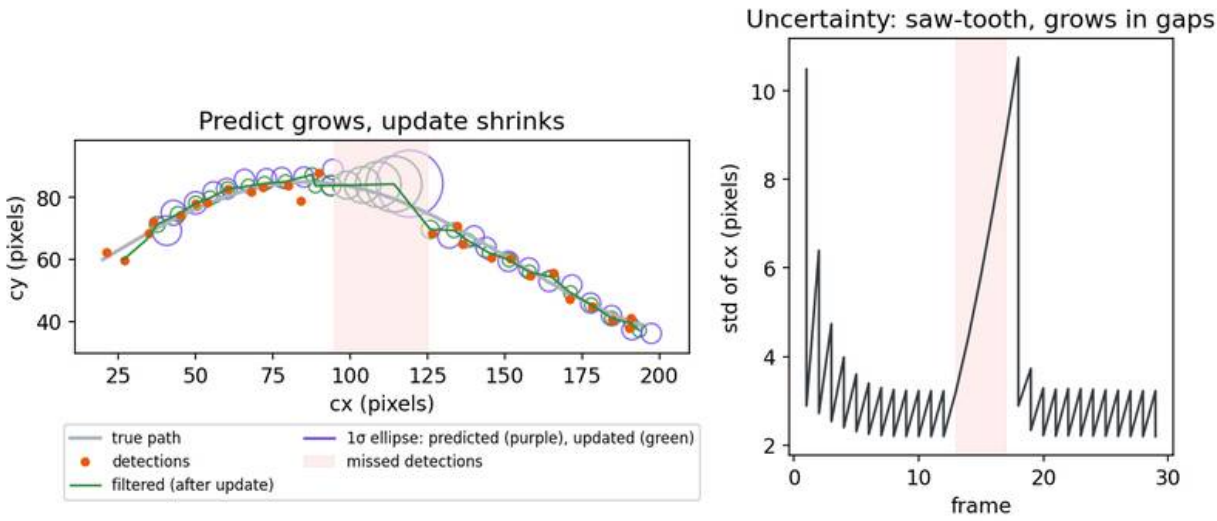


圖 4 — 在合成軌跡上的卡爾曼預測與更新。每次預測都讓不確定性膨脹，每次更新都讓它縮小。連續五格漏偵測時（粉紅色區域），濾波器沿直線滑行，不確定性持續上升——身分錯誤正是在這

種時候發生。

在模型假設下，創新量 $\boldsymbol{\nu}$ 的平方**馬氏距離**（Mahalanobis distance）$d_M^2 = \boldsymbol{\nu}^\top \mathbf{S}^{-1}\boldsymbol{\nu}$ 服從自由度 4 的 χ^2 分布，所以 DeepSORT [33] 用它來**閘控**（gating，直接禁止不合理的匹配）。等速假設也是最大的弱點：舞者、運動員，以及任何由移動攝影機拍攝的畫面，都會違反它。

外觀嵌入

Re-ID 網路（見上一節）把每個偵測裁切影像轉成單位向量 $\mathbf{e}_j$。每條軌跡保有外觀記憶，例如指數移動平均 $\mathbf{m}_k \leftarrow \alpha\,\mathbf{m}_k + (1-\alpha)\,\mathbf{e}_j$ 後再重新正規化，其中 $\alpha \in [0,1)$（我們用 0.9）。外觀代價就是餘弦距離 $1 - \mathbf{m}_k^\top \mathbf{e}_j$。在長時間遮擋後運動預測已經沒用時，正是靠外觀把身分找回來。

關聯：代價矩陣與匈牙利演算法

白話版。為每一組「軌跡 k 配上偵測 j」打一個「有多不合適」的分數，做成一張表，再選出一對一配對中總分最小的那一組。

有 K 條預測軌跡與 N 個偵測時，建立代價矩陣 $\mathbf{C} \in \mathbb{R}^{K\times N}$，例如

$$C_{kj} = \lambda\,(1 - \mathrm{IoU}(\hat{b}_k, b_j)) + (1-\lambda)\,(1 - \mathbf{m}_k^\top \mathbf{e}_j),$$

其中 $\hat{b}_k$ 是軌跡 k 的預測框，b_j 是偵測 j 的框，$\lambda \in [0,1]$ 是運動與外觀的權重（$\lambda = 1$ 就是 SORT 的純 IoU）。接著解**線性指派問題**

$$\min_{\mathbf{X}\in\{0,1\}^{K\times N}} \sum_{k,j} C_{kj} X_{kj} \quad \text{s.t.} \quad \sum_j X_{kj} \le 1,\ \sum_k X_{kj} \le 1,$$

讓每條軌跡最多拿一個偵測，反之亦然。**匈牙利演算法**（Hungarian algorithm）[31] 能在多項式時間內精確求解（$n = \max(K,N)$ 時為 $O(n^3)$）；SciPy 的 `linear_sum_assignment` 實作了一個快速的變形。最後，代價超過門檻的配對會被拒絕，實務上的閘控就是這樣做的。

```
# ---------- 2. IoU and Hungarian association ----------
def iou_matrix(a, b):
    """a: (N,4), b: (M,4) boxes as [cx, cy, w, h] -> (N,M) IoU."""
    a1, a2 = a[:, None, :2] - a[:, None, 2:] / 2, a[:, None, :2] + a[:, None, 2:] / 2
    b1, b2 = b[None, :, :2] - b[None, :, 2:] / 2, b[None, :, :2] + b[None, :, 2:] / 2
    wh = np.clip(np.minimum(a2, b2) - np.maximum(a1, b1), 0, None)
    inter = wh[..., 0] * wh[..., 1]
    area = lambda x: x[..., 2] * x[..., 3]
    return inter / (area(a)[:, None] + area(b)[None, :] - inter + 1e-9)

def associate(cost, max_cost):
    """Hungarian assignment; pairs whose cost exceeds max_cost are rejected."""
    if cost.size == 0:
        return [], list(range(cost.shape[0])), list(range(cost.shape[1]))
    rows, cols = linear_sum_assignment(cost)
    pairs = [(r, c) for r, c in zip(rows, cols) if cost[r, c] <= max_cost]
    mr = {r for r, _ in pairs}; mc = {c for _, c in pairs}
```

```
    return (pairs, [r for r in range(cost.shape[0]) if r not in mr],
            [c for c in range(cost.shape[1]) if c not in mc])
```

在合成場景上的完整追蹤器

為了看清楚每個想法帶來什麼，我們產生一個 80 格、5 位行人的場景，刻意放入幾種經典的失敗情境：

- 物件 0 與 1 **交錯而過**，重疊時位在後方的物件 1 只得到**低信心**偵測；
- 物件 2 與 3 相遇後**反彈回去**（非線性運動，像舞者），重疊時後方那位同樣只有低信心偵測；
- 物件 4 走到**柱子後方** 8 格，這段期間偵測都是低信心；
- 每個偵測都有 5% 機率漏掉，另外偶爾會出現低分的誤報。

每個偵測也帶有一個 16 維外觀向量：所屬物件固定的單位向量加上雜訊。

```
# ---------- 3. synthetic scene ----------
def make_scene(T=80, seed=0):
    rng = np.random.default_rng(seed)
    W, H = 30.0, 70.0
    t = np.arange(T, dtype=float)
    tr = {}
    # pair 1: plain crossing, B is behind A (occluded while they overlap)
    tr[0] = np.c_[80 + 5 * t, np.full(T, 120.0)]
    tr[1] = np.c_[480 - 5 * t, np.full(T, 128.0)]
    # pair 2: they meet and bounce back (non-linear motion, like dancers)
    m = 40
    x2 = np.where(t < m, 120 + 4 * t, 120 + 4 * m - 4 * (t - m))
    x3 = np.where(t < m, 440 - 4 * t, 440 - 4 * m + 4 * (t - m))
    tr[2] = np.c_[x2 + 10, np.full(T, 260.0)]
    tr[3] = np.c_[x3 - 10, np.full(T, 266.0)]
    # walker passing behind a pole (frames 30-37): detections become low-score
    tr[4] = np.c_[60 + 3 * t, 40 + 1.0 * t]
    gt = []                                     # per frame list of (id, box)
    dets = []                                   # per frame list of (box, score, gt_id)
    feats = {k: (lambda v: v / np.linalg.norm(v))(rng.normal(size=16)) for k in tr}
    for f in range(T):
        g, d = [], []
        boxes = {k: np.r_[tr[k][f], W, H] for k in tr}
        for k, b in boxes.items():
            g.append((k, b))
            score = rng.uniform(0.6, 0.95)
            if k == 1 and iou_matrix(b[None], boxes[0][None])[0, 0] > 0.2:
                score = rng.uniform(0.15, 0.45)       # occluded by object 0
            if k == 3 and iou_matrix(b[None], boxes[2][None])[0, 0] > 0.2:
                score = rng.uniform(0.15, 0.45)       # occluded by object 2
            if k == 4 and 30 <= f <= 37:
                score = rng.uniform(0.15, 0.45)       # behind the pole
            if rng.random() < 0.05:
                continue                               # random missed detection
            noisy = b + np.r_[rng.normal(0, 2.0, 2), rng.normal(0, 1.0, 2)]
```

```
            e = feats[k] + 0.15 * rng.normal(size=16)
            d.append((noisy, score, k, e / np.linalg.norm(e)))
        if rng.random() < 0.3:                          # low-score false positive
            fp = np.r_[rng.uniform(50, 550), rng.uniform(50, 320), W, H]
            e = rng.normal(size=16)
            d.append((fp, rng.uniform(0.1, 0.4), -1, e / np.linalg.norm(e)))
        gt.append(g); dets.append(d)
    return gt, dets
```

下面的追蹤器有三個開關。全部關閉時就是 **SORT 風格**：卡爾曼預測、IoU 代價、只對高分偵測（分數 ≥ 0.5）做匈牙利匹配，連續漏 2 格就刪除軌跡。`use_low=True` 加入 **BYTE** 風格的第二輪：沒配到的軌跡再只用 IoU 去配低分偵測。`use_app=True` 在第一輪加入**外觀**代價，並加上第三輪：純靠外觀，把遺失 30 格以內的軌跡重新識別回來。

```
# ---------- 4. SORT-style tracker with optional BYTE and appearance ----------
class Track:
    def __init__(self, tid, z, emb):
        self.id, self.kf, self.emb = tid, KalmanBox(z), emb
        self.lost = 0

def run_tracker(dets, use_low=False, use_app=False, hi=0.5, max_age=2,
                app_age=30, lam=0.5):
    tracks, out, next_id = [], [], 0
    for d in dets:
        for tk in tracks:
            tk.kf.predict()
        boxes = np.array([x[0] for x in d]).reshape(-1, 4)
        scores = np.array([x[1] for x in d])
        embs = np.array([x[3] for x in d]).reshape(-1, 16)
        high = np.where(scores >= hi)[0]
        low = np.where((scores < hi) & (scores >= 0.1))[0]
        active = [tk for tk in tracks if tk.lost <= max_age]
        pred = np.array([tk.kf.x[:4] for tk in active]).reshape(-1, 4)
        # stage 1: high-score detections vs active tracks
        cost = 1 - iou_matrix(pred, boxes[high])
        gate = 0.7
        if use_app and len(active) and len(high):
            tE = np.array([tk.emb for tk in active])
            app = 1 - tE @ embs[high].T            # cosine distance
            cost = np.where(cost > 0.9, 9.0, lam * cost + (1 - lam) * app)
            gate = 0.6
        pairs, ut, ud = associate(cost, gate)
        matched = [(active[r], high[c]) for r, c in pairs]
        rest_tracks = [active[r] for r in ut]
        rest_high = [high[c] for c in ud]
        # stage 2 (BYTE): remaining tracks vs low-score detections, IoU only
        if use_low and rest_tracks and len(low):
            p2 = np.array([tk.kf.x[:4] for tk in rest_tracks])
            pairs2, ut2, _ = associate(1 - iou_matrix(p2, boxes[low]), 0.5)
            matched += [(rest_tracks[r], low[c]) for r, c in pairs2]
            rest_tracks = [rest_tracks[r] for r in ut2]
        # stage 3 (appearance only): long-lost tracks vs leftover high-score dets
```

```
        if use_app and rest_high:
            lost = [tk for tk in tracks if max_age < tk.lost <= app_age]
            if lost:
                app = 1 - np.array([tk.emb for tk in lost]) @ embs[rest_high].T
                pairs3, _, ud3 = associate(app, 0.3)
                matched += [(lost[r], rest_high[c]) for r, c in pairs3]
                rest_high = [rest_high[c] for c in ud3]
        for tk, j in matched:
            tk.kf.update(boxes[j]); tk.lost = 0
            tk.emb = 0.9 * tk.emb + 0.1 * embs[j]; tk.emb /= np.linalg.norm(tk.emb)
        hit = {id(tk) for tk, _ in matched}
        for tk in tracks:
            if id(tk) not in hit:
                tk.lost += 1
        for j in rest_high:                            # birth: unmatched high-score
            tracks.append(Track(next_id, boxes[j], embs[j])); next_id += 1
        tracks = [tk for tk in tracks if tk.lost <= (app_age if use_app else max_age)]
        out.append([(tk.id, tk.kf.x[:4].copy()) for tk in tracks if tk.lost == 0])
    return out
```

指標程式（MOTA、IDF1、HOTA，下文說明）與實驗：

```
# ---------- 5. metrics: MOTA, IDF1, HOTA ----------
def frame_match(g, p, thr):
    if not g or not p:
        return []
    iou = iou_matrix(np.array([b for _, b in g]), np.array([b for _, b in p]))
    r, c = linear_sum_assignment(-iou)
    return [(g[i][0], p[j][0]) for i, j in zip(r, c) if iou[i, j] >= thr]

def mota(gt, hyp, thr=0.5):
    fn = fp = idsw = 0; ngt = 0; last = {}
    for g, p in zip(gt, hyp):
        m = frame_match(g, p, thr)
        ngt += len(g); fn += len(g) - len(m); fp += len(p) - len(m)
        for gi, pi in m:
            if gi in last and last[gi] != pi:
                idsw += 1
            last[gi] = pi
    return 1 - (fn + fp + idsw) / ngt, idsw

def idf1(gt, hyp, thr=0.5):
    gids = sorted({i for g in gt for i, _ in g}); pids = sorted({i for p in hyp for i, _ in
    ↪  p})
    gi = {k: n for n, k in enumerate(gids)}; pi = {k: n for n, k in enumerate(pids)}
    ov = np.zeros((len(gids), len(pids)))
    for g, p in zip(gt, hyp):
        if g and p:
            iou = iou_matrix(np.array([b for _, b in g]), np.array([b for _, b in p]))
            for a in range(len(g)):
                for b in range(len(p)):
                    ov[gi[g[a][0]], pi[p[b][0]]] += iou[a, b] >= thr
```

```
    r, c = linear_sum_assignment(-ov)
    idtp = ov[r, c].sum()
    n_gt = sum(len(g) for g in gt); n_p = sum(len(p) for p in hyp)
    return 2 * idtp / (n_gt + n_p)

def hota(gt, hyp, alphas=np.arange(0.05, 0.96, 0.05)):
    res = []
    for a in alphas:
        pairs = []; n_gt = n_p = 0
        for g, p in zip(gt, hyp):
            pairs += frame_match(g, p, a); n_gt += len(g); n_p += len(p)
        tp = len(pairs)
        if tp == 0:
            res.append((0, 0, 0)); continue
        cnt = {}
        for q in pairs:
            cnt[q] = cnt.get(q, 0) + 1
        gcount = {}; pcount = {}
        for g in gt:
            for i, _ in g: gcount[i] = gcount.get(i, 0) + 1
        for p in hyp:
            for i, _ in p: pcount[i] = pcount.get(i, 0) + 1
        ass = sum(cnt[q] / (gcount[q[0]] + pcount[q[1]] - cnt[q]) for q in pairs) / tp
        det = tp / (n_gt + n_p - tp)
        res.append((np.sqrt(det * ass), det, ass))
    return np.array(res).mean(axis=0)   # HOTA, DetA, AssA averaged over alpha

gt, dets = make_scene(seed=3)
for name, kw in [("IoU only", {}), ("+ low-score pass", dict(use_low=True)),
                 ("+ appearance", dict(use_low=True, use_app=True))]:
    hyp = run_tracker(dets, **kw)
    m, sw = mota(gt, hyp)
    print(f"{name:18s} MOTA {m:.2f}  IDF1 {idf1(gt, hyp):.2f}  "
          f"HOTA {hota(gt, hyp)[0]:.2f}  ID switches {sw}")
```

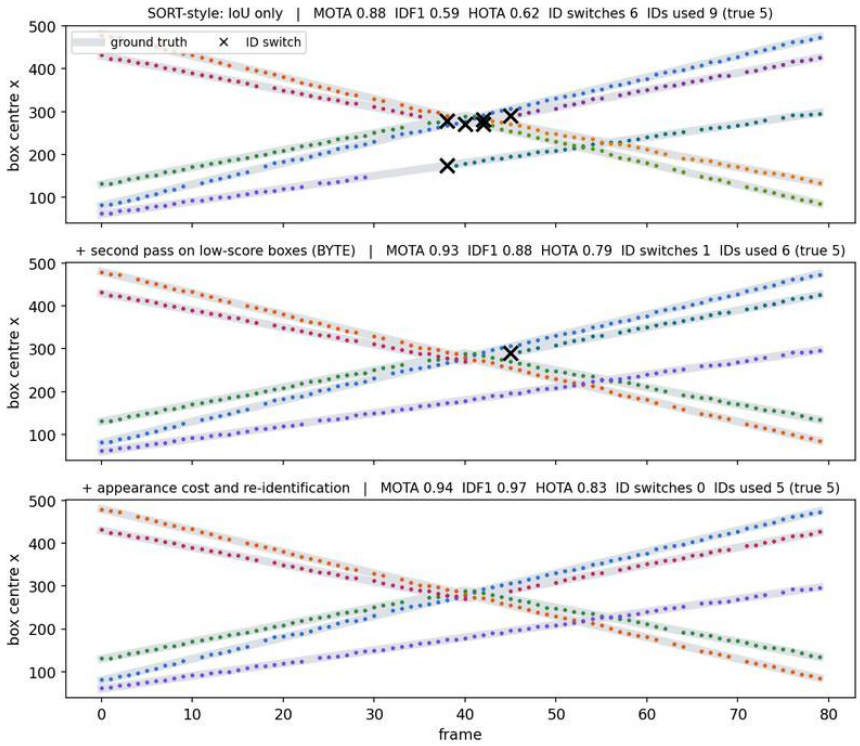


圖 *5* — 同一組合成偵測，用三種方式追蹤。顏色代表預測 *ID*，黑色叉號標示 *ID* 切換。低分關聯消除了遮擋造成的斷裂；外觀則修正了反彈情境，因為等速預測此時指向錯誤的方向。

程式輸出：

追蹤器	MOTA	IDF1	HOTA	ID 切換	使用的 ID 數（真實：5）
SORT 風格，只用 IoU	0.88	0.59	0.62	6	9
+ BYTE 低分偵測第二輪	0.93	0.88	0.79	1	6
+ 外觀與重新識別	0.94	0.97	0.83	0	5

仔細看這張表。只用 IoU 的追蹤器會弄丟被遮擋的物件，因為它們的偵測分數低於門檻，軌跡死掉，再出現時就換了**新 ID**（軌跡斷裂，fragmentation）。低分偵測的第二輪讓這些軌跡活下來，幾乎修好了所有問題，但修不了反彈：卡爾曼濾波器預測兩位舞者會繼續往前走，所以重疊之後其中一位沒被配到，稍後以新 ID 重新出現。只有外觀能解決這個情況。也請注意 MOTA 的變化（0.88 → 0.94）比 IDF1（0.59 → 0.97）小得多：MOTA 主要是在衡量偵測，我們會在指標一節再回來談。

SORT 家族

這些想法正好對應到最常被引用的線上追蹤器：

追蹤器	核心想法
SORT [32]	卡爾曼濾波器 + IoU 代價 + 匈牙利指派；刻意極簡、速度非常快。證明了偵測器的品質主導追蹤品質。
DeepSORT [33]	加入在行人 Re-ID 資料集上訓練的 CNN 外觀描述子、馬氏距離閘控，以及優先處理最近才看到之軌跡的**匹配級聯**（matching cascade）。能撐過更長的遮擋，ID 切換更少。
ByteTrack [34]	關聯**每一個**偵測框：先配高分框，再把剩下的軌跡只用 IoU 去配**低分**框。被遮擋的物件常常分數偏低，因此能被救回而不是被丟掉；而低分的背景框配不到軌跡，就被捨棄。
OC-SORT [35]	「以觀測為中心」：遺失的軌跡被重新找到時，沿著上一次與這一次觀測之間的虛擬軌跡重跑卡爾曼更新（修正滑行期間累積的誤差），並在代價中加入運動方向一致性項。在 DanceTrack 這類非線性運動上特別強。
BoT-SORT [36]	ByteTrack 式的關聯，再加上攝影機運動補償（全域影像對齊）、以 w, h 取代長寬比的卡爾曼狀態，以及融合 IoU 與 Re-ID 的代價。

這個家族的教訓是：在強偵測器之上**精心設計的關聯規則**，在行人基準上非常難被超越。Adžemović 2025 年的綜述 [4] 也得到同樣的結論：在密集、運動大致線性的場景，啟發式方法領先；運動複雜時，學習式關聯表現較好。

聯合偵測與嵌入，以及以偵測器追蹤

對每個裁切框都另外跑一次 Re-ID 網路很昂貴。**JDE** [37] 在單階段偵測器上加一個嵌入頭，一次前向傳遞就同時輸出框與外觀向量。**FairMOT** [38] 指出這種共享對 Re-ID 並「不公平」：以錨框為基礎的偵測器產生的特徵對身分而言很模糊，而且訓練被偵測任務主導。它改用無錨框、CenterNet 風格的偵測器，在高解析度特徵圖上放兩個對等的分支（偵測與 Re-ID）。原文也提到的 **Tracktor** [39]，則在大多數情況下省去顯式關聯：把上一格的框送進偵測器的框迴歸頭，讓每個框被拉到物件的新位置，並保留原本的身分。

以 query 追蹤：端到端 MOT

白話版。不再手寫「預測、比較、配對」，而是給 Transformer 每個物件一個記憶格，讓它自己學會每一格怎麼更新這個記憶格。

DETR 風格的偵測器（見DLCV 03）用 **query** 表示物件。MOTR [40] 把這延伸到影片：每個偵測到的物件的輸出嵌入，變成一個**軌跡 query**（track query）傳到下一格，持續預測同一個物件；新的**偵測 query** 則負責找出新出現的物件。身分由 query 隱式攜帶，推論時沒有卡爾曼濾波器，也沒有匈牙利匹配。訓練時使用考慮軌跡的標籤指派（tracklet-aware label assignment）與在片段上平均的損失。它的弱點是偵測品質：同一個解碼器既要偵測又要關聯，兩個任務互相衝突。MOTRv2 [41] 以另一個預訓練偵測器（YOLOX）的提議框作為錨點來引導 MOTR，緩解了這個衝突。MOTIP [42]（CVPR 2025）把關聯重新定義為 **ID 預測**：給定到目前為止的軌跡，每條軌跡帶有一個從可學習 ID 字典中取出的 ID 標籤，解碼器把每個新偵測分類成既有的某個 ID 或「新物件」。這是 in-context、可端到端訓練的，而且只需要物件層級的特徵。

開放詞彙 MOT

傳統 MOT 只能追蹤訓練集中出現過的類別。**OVTrack** [43]（CVPR 2023）透過從視覺語言模型（CLIP）蒸餾知識來做分類與關聯，並用擴散模型**幻想生成**（hallucinate）的影像對來學習穩健的外觀特徵，藉此追蹤任意類別。SAM 3 的概念提示 [25] 則從分割這一側朝同一個目標前進。

資料集

資料集	領域	考驗什麼
MOT17 [44]	行人，固定與移動攝影機	標準行人基準；以 MOT16 的序列重新發布為 MOT17
MOT20 [45]	非常擁擠的行人場景	嚴重遮擋、小目標
DanceTrack [46]	群舞	外觀一致、運動多變且非線性；無法靠外觀區分人
SportsMOT [47]	籃球、排球、足球（240 段序列）	快速且速度多變的運動，外觀相似但仍可區分
BDD100K MOT [48]	行車影片	從移動車輛上做多類別追蹤（汽車、行人、自行車騎士等）

從 MOT17 轉向 DanceTrack 與 SportsMOT，改變了這個領域的重點：在 MOT17 上，好的偵測器加上 IoU 匹配就能走很遠；但在 DanceTrack 上，運動模型與學習式關聯才是關鍵。

指標：MOTA、IDF1、HOTA

白話版。追蹤器有兩種失敗方式：漏掉物件或憑空多出物件（偵測錯誤），或是把誰是誰搞混（關聯錯誤）。不同指標對這兩者的權重不同。

三個指標都先在每一格、以某個 IoU 門檻把預測與真實值配對。令 GT_t、FN_t、FP_t 與 IDSW_t 分別是第 t 格的真實物件數、漏偵測數、誤報數與身分切換數。所謂身分切換，是指某個真實物件這次配到的預測 ID，與它上一次配到的不同。

MOTA（CLEAR MOT [9]）：

$$\mathrm{MOTA} = 1 - \frac{\sum_t (\mathrm{FN}_t + \mathrm{FP}_t + \mathrm{IDSW}_t)}{\sum_t \mathrm{GT}_t}.$$

它的範圍是 $-\infty$ 到 1。由於偵測數量通常遠多於切換次數，FN 與 FP 主導了 MOTA：它主要是偵測分數。而且切換只在發生的那一刻被計算一次，所以一個把兩個 ID 換掉後維持 1,000 格的追蹤器，與一個 1 格後就換回來的追蹤器，付出的代價相同。

IDF1 [10] 在真實軌跡與預測軌跡之間做**全域**的一對一匹配（同樣用匈牙利演算法），最大化兩者一致的影格數，然後

$$\mathrm{IDF1} = \frac{2\,\mathrm{IDTP}}{2\,\mathrm{IDTP} + \mathrm{IDFP} + \mathrm{IDFN}},$$

其中 IDTP 計算那些配到「與其真實身分對應之預測軌跡」的偵測，IDFP、IDFN 則是其餘的預測與真實偵測。IDF1 衡量追蹤器能維持正確身分多久，所以主要是關聯分數——但因為匹配是全域的，它的行為有時並不直觀，例如偵測變好時 IDF1 反而下降。

HOTA（Higher Order Tracking Accuracy [8]）的提出，正是因為上述兩個指標既不平衡、也無法拆解。對一個定位門檻 α，以每格配對得到真陽性（TP）、FN 與 FP：

$$\mathrm{DetA}_\alpha = \frac{|\mathrm{TP}|}{|\mathrm{TP}| + |\mathrm{FN}| + |\mathrm{FP}|}, \qquad \mathrm{AssA}_\alpha = \frac{1}{|\mathrm{TP}|} \sum_{c \in \mathrm{TP}} \frac{|\mathrm{TPA}(c)|}{|\mathrm{TPA}(c)| + |\mathrm{FNA}(c)| + |\mathrm{FPA}(c)|},$$

$$\mathrm{HOTA}_\alpha = \sqrt{\mathrm{DetA}_\alpha \cdot \mathrm{AssA}_\alpha}, \qquad \mathrm{HOTA} = \frac{1}{19} \sum_{\alpha \in \{0.05, 0.10, \ldots, 0.95\}} \mathrm{HOTA}_\alpha.$$

對一個配對 c（真實 ID g、預測 ID p），$\mathrm{TPA}(c)$ 是整段影片中同樣把 g 配到 p 的所有 TP；$\mathrm{FNA}(c)$ 是 g 被配到其他 ID 或漏掉的偵測；$\mathrm{FPA}(c)$ 是 p 被配到其他物件或沒配到任何物件的偵測。幾何平均代表追蹤器必須在偵測與關聯**兩方面**都好才能得高分；DetA 與 AssA（以及定位準確度 LocA）也可以分開報告，用來診斷問題。對 α 取平均也會獎勵精確的框。（我們的程式用的是簡化版：每格最大化 IoU 的匹配；正式定義使用的匹配分數還會偏好過去已經關聯過的配對。）

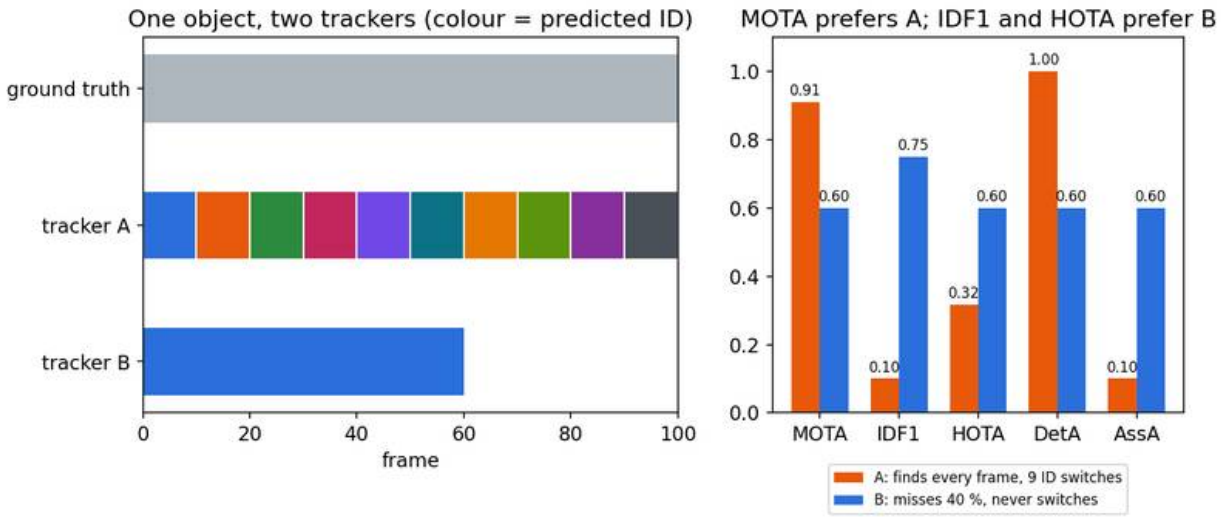


圖 *6 — HOTA 為什麼被提出。追蹤器 A 每格都偵測到，但換了九次身分；追蹤器 B 漏掉 40% 的影格，但從不換身分。MOTA 認為 A 遠遠領先；IDF1 與 HOTA 認為 B 較好，而 DetA ／ AssA 正好說明原因。*

圖 6 是用上面的函式算出來的：追蹤器 A 的 MOTA $= 1 - 9/100 = 0.91$，AssA $= 0.10$，所以 HOTA $= \sqrt{1.0 \times 0.10} \approx 0.32$；追蹤器 B 的 MOTA、DetA、AssA 與 HOTA 都是 0.60，IDF1 $= 2 \cdot 60/(100 + 60) = 0.75$。如今 MOT17、MOT20、DanceTrack 與 SportsMOT 上的論文會同時報告 HOTA、IDF1 與 MOTA，而 HOTA 通常是主要的排名指標。

點追蹤：相關的新趨勢

白話版。不追整個物件，而是追單一個**點**——臉頰上的一顆雀斑、箱子的一個角——貫穿整段影片，即使它短暫被擋住也一樣。

追蹤任意點（tracking any point, TAP）的問題是：給定第 t_0 格的查詢點 (x, y)，求它在其他每一格的位置與是否可見。它把光流（兩格之間）推廣到長時間、能處理遮擋的軌跡，也是IP 12傳統關鍵點追蹤的自然延伸。TAPIR [49] 先在每一格獨立比對查詢點的特徵得到粗略軌跡，再用時間卷積加以精修。CoTracker [50] 用 Transformer **聯合**追蹤許多點，讓點與點互相幫忙：被遮擋的點可以從還看得見的鄰近點推斷出來。CoTracker3 [51] 簡化了架構，並改用由既有追蹤器產生偽標籤的真實影片來訓練，而不只靠合成資料。點追蹤器現在被用作物件追蹤器中的運動線索，也用在影片編輯與機器人上。

多目標跨攝影機追蹤（MTMCT）

白話版。很多台攝影機看著同一個地方。先在每台攝影機裡追蹤每個人，再判斷離開 3 號攝影機的那位女士，就是走進 5 號攝影機的同一位。

MTMCT 就是 [1] 所說的終極目標。典型的流程是：先在每台攝影機內做 MOT 得到**短軌跡**（tracklet），用 Re-ID 特徵描述每條短軌跡，再利用外觀、攝影機拓樸（哪些攝影機相連、彼此之間的移動時間），以及在已校正攝影機下的地面 3D 位置，把不同攝影機的短軌跡分群。IDF1 原本就是為這個情境提出的 [10]。**AI City Challenge** 是主要的常態性基準。第九屆（2025）[52] 的 Track 1 是倉儲型環境中的多類別 **3D** 跨攝影機追蹤，類別包括人、人形機器人、自主移動機器人與堆高機，並提供攝影機校正資料與 3D 框標註。從影像上的框走向多視角、已校正的 3D 軌跡，正是 MTMCT 前進的方向。

現代觀點

綜述論文怎麼說

- **Luo 等人〈Multiple object tracking: A literature review〉[3]**（*Artificial Intelligence*，2021；2014 年首次公開）。經典參考文獻。它形式化定義 MOT 問題，依初始化方式（以偵測為基礎或不需偵測）、處理模式（線上或離線）與輸出型態（確定性或機率性）分類方法，並分析外觀模型、運動模型、互動模型、互斥限制與遮擋處理等元件，以及各種推論方法。想不依賴任何網路去理解問題本身，就讀這篇。
- **Ciaparrone 等人〈Deep learning in video multi-object tracking: A survey〉[2]**（*Neurocomputing*，2020）。把深度 MOT 整理成四個階段——偵測、特徵擷取／運動預測、親和度計算與關聯——並回顧深度學習如何進入每個階段。從 MOTChallenge 上的比較得到的主要結論是：大部分進步來自偵測品質與學習式外觀特徵。
- **Adžemović〈Deep learning-based multi-object tracking: A comprehensive survey from foundations to state-of-the-art〉[4]**（arXiv，2025）。把先偵測再追蹤分成聯合偵測與嵌入、啟發式、以運動為主、親和度學習與離線方法五類，並與端到端追蹤器比較。它指出 2022 年（ByteTrack 與 MOTR）是加速的轉捩點，並總結：啟發式追蹤器在擁擠、運動大致線性的場景領先，

學習式關聯則在複雜運動上勝出。

- **Marvasti-Zadeh 等人〈Deep learning for visual tracking: A comprehensive survey〉[5]**（*IEEE T-ITS*，2022）。針對 SOT。從九個面向（網路架構、網路運用、訓練方式、目標函數、輸出、與相關濾波器的結合、空拍視角、長時追蹤與線上追蹤）分析深度追蹤器，並在 OTB、VOT、LaSOT 與空拍基準上比較。
- **Thangavel（Kugarajeevan）等人〈Transformers in single object tracking: An experimental survey〉[6]**（*IEEE Access*，2023）。把 Transformer 追蹤器分成 CNN-Transformer 混合、雙流兩階段，以及單流單階段全 Transformer 三類，並實驗評估其穩健性與效率；單流設計（OSTrack 類）成為主流模式。
- **Ye 等人〈Deep learning for person re-identification: A survey and outlook〉[7]**（*IEEE TPAMI*，2022）。把封閉世界 Re-ID（特徵學習、度量學習、排序最佳化）與開放世界 Re-ID（異質資料、原始影像、雜訊標籤、非監督與開放集合設定）分開討論，提出 AGW 基線與 mINP 指標，後者衡量找到**所有**正確匹配所需的代價。
- **Luiten 等人〈HOTA〉[8]**（*IJCV*，2020）。不是綜述，但它重新檢視了整個領域的評估方式，以實例與使用者研究說明 MOTA 與 IDF1 的偏差，也是目前排行榜以 HOTA 排名的原因。

2024–2026 的技術現況

- **SOT** 由 OSTrack ／ SeqTrack 一脈的單流 Transformer 追蹤器主導，而分割基礎模型的角色越來越重：加上運動感知記憶的 SAM 2（SAMURAI），不需針對追蹤訓練就有很強的 zero-shot 追蹤能力 [23]、[24]。
- **MOT** 兩個陣營並存。啟發式的先偵測再追蹤（ByteTrack、OC-SORT、BoT-SORT 及其後繼者）仍是實務上的預設選擇：簡單、快速、模組化，大部分工作由強大的偵測器完成。端到端追蹤器（MOTR、MOTRv2、MOTIP）則在運動複雜、外觀模糊的情境領先，例如 DanceTrack 與 SportsMOT [4]、[42]。
- **開放詞彙與可提示追蹤**正進入主流模型：OVTrack 建立了這個任務，SAM 3 則能依文字概念偵測、分割並追蹤帶有身分的實例 [25]、[43]。
- **點追蹤**（TAPIR、CoTracker、CoTracker3）已成熟為通用的運動基本元件 [49]–[51]。
- **MTMCT** 正走向已校正、多類別的 3D 追蹤（AI City Challenge 2025）[52]。

尚待解決的問題

- **長時間遮擋與重新進場**。幾秒之後運動模型就沒用了，只能靠外觀維持身分；但在服裝一致（DanceTrack、運動比賽）或跨攝影機時，外觀並不可靠。
- **非線性運動與攝影機運動**。等速卡爾曼濾波器仍是預設，也是舞蹈、運動與行車資料上的弱點；學習式運動模型還沒有在所有情境都明顯更好。
- **端到端與啟發式關聯之爭**。端到端追蹤器優雅、在複雜運動上勝出，但需要大量影片訓練資料與計算，在擁擠行人場景上仍輸給簡單的啟發式方法。如何設計出兩邊都最好的統一架構，仍是開放問題。
- **開放詞彙與長尾追蹤**。稀有類別，以及能推廣到訓練中從未見過之類別的外觀特徵。
- **評估**。 HOTA 解決了很多問題，但基準仍以行人為主，而跨數小時影片與多台攝影機的身分層級評估依然困難。**隱私與資料集倫理**（DukeMTMC 的教訓）也限制了能蒐集哪些資料。

重點整理

- 追蹤 = 偵測 + 時間：每個追蹤器都要同時跟住運動與維持身分，兩者的比重定義了 SOT、MOT 與 Re-ID。
- Re-ID 是以學習嵌入進行的檢索；請同時報告 CMC rank-k 與 mAP；DukeMTMC 已於 2019 年撤下，不應再使用。
- SOT 從相關濾波器（快速、人工特徵）走到孿生網路比對（離線訓練），再到單流 Transformer；SAM 2 類的影片分割如今能以遮罩做 SOT，SAM 3 更能依文字提示擴展到多個實例。
- 線上 MOT 是一個迴圈：偵測 → 卡爾曼預測 → 代價矩陣 → 匈牙利指派 → 更新／出生／死亡。實務上的進步大多來自代價矩陣裡放了什麼，以及哪些偵測被允許參與匹配。
- ByteTrack 的低分偵測第二輪修正遮擋造成的斷裂；外觀修正非線性運動與長時間缺漏；端到端 query 追蹤器則直接學習關聯。
- MOTA 主要衡量偵測，IDF1 主要衡量關聯；HOTA = $\sqrt{\text{DetA} \cdot \text{AssA}}$（再對 IoU 門檻取平均）兼顧並能拆解兩者。
- 目前的前沿是長時間身分維持、非線性運動、開放詞彙類別，以及跨攝影機的 3D 追蹤。

練習

1. **卡爾曼增益的直覺**。對一維等位置模型，先驗變異數為 P、量測雜訊為 R，證明卡爾曼增益為 $K = P/(P+R)$，且後驗變異數為 $PR/(P+R)$。若每次預測都讓 P 增加 Q，連續五格沒有偵測之後 K 會怎麼變？

   提示

   $H = 1$ 時，$S = P + R$，$K = P/S$。後驗變異數為 $(1-K)P = PR/(P+R)$，比 P 與 R 都小。滑行五步後先驗變成 $P + 5Q$，K 往 1 靠近：濾波器會非常相信下一個偵測，這也是為什麼遮擋後一次錯誤的匹配會讓軌跡大幅移動。

2. **手算匈牙利演算法**。軌跡 A、B 與偵測 1、2 的預測 IoU 為 $\begin{bmatrix} 0.6 & 0.5 \\ 0.55 & 0.0 \end{bmatrix}$。貪婪匹配（IoU 最高者優先）的結果是什麼？以 $1 - \text{IoU}$ 為代價的匈牙利演算法結果又是什麼？哪一個的 IoU 總和較大？

   提示

   貪婪法先選 A–1（0.6），剩下 B–2 的 IoU 為 0（被拒絕），總和 0.6。匈牙利演算法選 A–2 與 B–1，總和 $0.5 + 0.55 = 1.05$。貪婪法讓軌跡 B 沒配到，並為偵測 2 開一個新 ID——這正是 ID 切換出現的方式之一。

3. **把追蹤器弄壞**。在合成場景中修改 `make_scene` 裡的反彈，讓物件 2 與 3 直接穿過彼此（速度不反轉）。三個追蹤器中哪一個進步最多？為什麼？

   提示

運動變成線性後，等速預測在重疊期間都是正確的。以 `seed=3` 來說，低分偵測追蹤器的 IDF1 從 0.88 提升到約 0.96，也用了正確的 5 個 ID，幾乎追上外觀追蹤器（同樣約 0.96）；只用 IoU 的追蹤器仍會讓被遮擋的物件斷裂。外觀最有幫助的時候，正是運動非線性的時候，這也是 DanceTrack 的設計初衷。

4. **指標計算**。一段影片中只有一個物件，共 200 格。追蹤器每一格都偵測到它，但第 1–100 格用 ID 1，第 101–200 格用 ID 2。假設框完全正確，在單一門檻下計算 MOTA、IDF1、DetA、AssA 與 HOTA。

   提示

   MOTA $= 1 - 1/200 = 0.995$。最佳全域 ID 匹配涵蓋 100 格：IDF1 $= 2 \cdot 100/(200 + 200) = 0.5$。DetA $= 1$。每個 TP 的 $|\text{TPA}| = 100$、$|\text{FNA}| = 100$、$|\text{FPA}| = 0$，所以 AssA $= 0.5$，HOTA $= \sqrt{0.5} \approx 0.71$。一次切換幾乎不影響 MOTA，卻讓 AssA 減半。

5. **Re-ID 指標**。在 Re-ID 程式中，把雜訊從 1.6 改成 2.2，再改成 1.0。rank-1 與 mAP 怎麼變？哪一個比較敏感？請用 AP 的定義解釋。

   提示

   雜訊變大時 mAP 下降得比 rank-1 快，因為 rank-1 只需要最容易的那一個正確匹配排在第一，而 AP 會懲罰每一個被往後推的正確匹配。每個查詢有 4 個正確匹配時，最難的那一個通常決定了 AP 損失多少。

6. **設計題**。你要在倉庫中用 12 台已校正的攝影機追蹤堆高機與工作人員。用本篇介紹的元件畫出一個流程，並說明你會報告哪個指標、為什麼。

   提示

   每台攝影機做偵測，搭配 ByteTrack ／ BoT-SORT 風格的追蹤器；利用校正參數把框投影到地面；跨攝影機關聯先看 3D 位置與時間，再看 Re-ID 外觀（工作人員可能穿著一樣的背心）；兩個類別分開處理。報告 HOTA（平衡，且可拆成 DetA ／ AssA）與 IDF1（跨攝影機的長時身分），與 AI City Challenge 的設定一致。

參考文獻

1. Shyandram，〈物件追蹤 Object Tracking 簡介〉，Medium 文章，2023；2026 年更新。連結
2. G. Ciaparrone, F. Luque Sánchez, S. Tabik, L. Troiano, R. Tagliaferri and F. Herrera, “Deep learning in video multi-object tracking: A survey,” *Neurocomputing*, 2020. arXiv:1907.12740
3. W. Luo, J. Xing, A. Milan, X. Zhang, W. Liu and T.-K. Kim, “Multiple object tracking: A literature review,” *Artificial Intelligence*, vol. 293, 2021 (arXiv 2014). arXiv:1409.7618
4. M. Adžemović, “Deep learning-based multi-object tracking: A comprehensive survey from foundations to state-of-the-art,” arXiv:2506.13457, 2025. arXiv

5. S. M. Marvasti-Zadeh, L. Cheng, H. Ghanei-Yakhdan and S. Kasaei, "Deep learning for visual tracking: A comprehensive survey," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, 2022. arXiv:1912.00535
6. J. Thangavel (Kugarajeevan), T. Kokul, A. Ramanan and S. Fernando, "Transformers in single object tracking: An experimental survey," *IEEE Access*, vol. 11, 2023. arXiv:2302.11867
7. M. Ye, J. Shen, G. Lin, T. Xiang, L. Shao and S. C. H. Hoi, "Deep learning for person re-identification: A survey and outlook," *IEEE TPAMI*, vol. 44, 2022. arXiv:2001.04193
8. J. Luiten, A. Ošep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixé and B. Leibe, "HOTA: A higher order metric for evaluating multi-object tracking," *International Journal of Computer Vision*, 2020. arXiv:2009.07736
9. K. Bernardin and R. Stiefelhagen, "Evaluating multiple object tracking performance: The CLEAR MOT metrics," *EURASIP Journal on Image and Video Processing*, 2008. doi
10. E. Ristani, F. Solera, R. S. Zou, R. Cucchiara and C. Tomasi, "Performance measures and a data set for multi-target, multi-camera tracking," *ECCV Workshop on Benchmarking Multi-Target Tracking*, 2016. arXiv:1609.01775
11. K. Peng, A. Mathur and A. Narayanan, "Mitigating dataset harms requires stewardship: Lessons from 1000 papers," arXiv:2108.02922, 2021. arXiv
12. L. Zheng, L. Shen, L. Tian, S. Wang, J. Wang and Q. Tian, "Scalable person re-identification: A benchmark," *ICCV*, 2015. doi
13. L. Wei, S. Zhang, W. Gao and Q. Tian, "Person transfer GAN to bridge domain gap for person re-identification," *CVPR*, 2018. arXiv:1711.08565
14. H. Luo, Y. Gu, X. Liao, S. Lai and W. Jiang, "Bag of tricks and a strong baseline for deep person re-identification," *CVPR Workshops*, 2019. arXiv:1903.07071
15. M. Wieczorek, B. Rychalska and J. Dąbrowski, "On the unreasonable effectiveness of centroids in image retrieval," arXiv:2104.13643, 2021. arXiv
16. S. Li, L. Sun and Q. Li, "CLIP-ReID: Exploiting vision-language model for image re-identification without concrete text labels," *AAAI*, 2023. arXiv:2211.13977
17. D. S. Bolme, J. R. Beveridge, B. A. Draper and Y. M. Lui, "Visual object tracking using adaptive correlation filters," *CVPR*, 2010. doi
18. J. F. Henriques, R. Caseiro, P. Martins and J. Batista, "High-speed tracking with kernelized correlation filters," *IEEE TPAMI*, vol. 37, 2015. arXiv:1404.7584
19. L. Bertinetto, J. Valmadre, J. F. Henriques, A. Vedaldi and P. H. S. Torr, "Fully-convolutional Siamese networks for object tracking," arXiv:1606.09549, 2016 (ECCV 2016 workshops). arXiv
20. B. Li, J. Yan, W. Wu, Z. Zhu and X. Hu, "High performance visual tracking with Siamese region proposal network," *CVPR*, 2018. doi
21. B. Ye, H. Chang, B. Ma, S. Shan and X. Chen, "Joint feature learning and relation modeling for tracking: A one-stream framework" (OSTrack), *ECCV*, 2022. arXiv:2203.11991
22. X. Chen, H. Peng, D. Wang, H. Lu and H. Hu, "SeqTrack: Sequence to sequence learning for visual object tracking," *CVPR*, 2023 (the arXiv entry was later extended as SeqTrackv2). arXiv:2304.14394
23. N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma et al., "SAM 2: Segment anything in images and videos," *ICLR*, 2025. arXiv:2408.00714
24. C.-Y. Yang, H.-W. Huang, W. Chai, Z. Jiang and J.-N. Hwang, "SAMURAI: Adapting segment anything model for zero-shot visual tracking with motion-aware memory," arXiv:2411.11922,

2024. arXiv
25. N. Carion, L. Gustafson, Y.-T. Hu et al., "SAM 3: Segment anything with concepts," *ICLR*, 2026. arXiv:2511.16719
26. Y. Wu, J. Lim and M.-H. Yang, "Object tracking benchmark," *IEEE TPAMI*, vol. 37, 2015. doi
27. H. Fan, L. Lin, F. Yang, P. Chu, G. Deng, S. Yu et al., "LaSOT: A high-quality benchmark for large-scale single object tracking," *CVPR*, 2019. arXiv:1809.07845
28. L. Huang, X. Zhao and K. Huang, "GOT-10k: A large high-diversity benchmark for generic object tracking in the wild," *IEEE TPAMI*, vol. 43, 2021. arXiv:1810.11981
29. M. Müller, A. Bibi, S. Giancola, S. Al-Subaihi and B. Ghanem, "TrackingNet: A large-scale dataset and benchmark for object tracking in the wild," arXiv:1803.10794, 2018 (ECCV 2018). arXiv
30. R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960. doi
31. H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, pp. 83–97, 1955. doi
32. A. Bewley, Z. Ge, L. Ott, F. Ramos and B. Upcroft, "Simple online and realtime tracking," *ICIP*, 2016. arXiv:1602.00763
33. N. Wojke, A. Bewley and D. Paulus, "Simple online and realtime tracking with a deep association metric," *ICIP*, 2017. doi
34. Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," *ECCV*, 2022. arXiv:2110.06864
35. J. Cao, J. Pang, X. Weng, R. Khirodkar and K. Kitani, "Observation-centric SORT: Rethinking SORT for robust multi-object tracking," *CVPR*, 2023. arXiv:2203.14360
36. N. Aharon, R. Orfaig and B.-Z. Bobrovsky, "BoT-SORT: Robust associations multi-pedestrian tracking," arXiv:2206.14651, 2022. arXiv
37. Z. Wang, L. Zheng, Y. Liu, Y. Li and S. Wang, "Towards real-time multi-object tracking" (JDE), *ECCV*, 2020. arXiv:1909.12605
38. Y. Zhang, C. Wang, X. Wang, W. Zeng and W. Liu, "FairMOT: On the fairness of detection and re-identification in multiple object tracking," *IJCV*, 2021. arXiv:2004.01888
39. P. Bergmann, T. Meinhardt and L. Leal-Taixé, "Tracking without bells and whistles" (Tracktor), *ICCV*, 2019. arXiv:1903.05625
40. F. Zeng, B. Dong, Y. Zhang, T. Wang, X. Zhang and Y. Wei, "MOTR: End-to-end multiple-object tracking with transformer," *ECCV*, 2022. arXiv:2105.03247
41. Y. Zhang, T. Wang and X. Zhang, "MOTRv2: Bootstrapping end-to-end multi-object tracking by pretrained object detectors," *CVPR*, 2023. arXiv:2211.09791
42. R. Gao, J. Qi and L. Wang, "Multiple object tracking as ID prediction" (MOTIP), *CVPR*, 2025. arXiv:2403.16848
43. S. Li, T. Fischer, L. Ke, H. Ding, M. Danelljan and F. Yu, "OVTrack: Open-vocabulary multiple object tracking," *CVPR*, 2023. arXiv:2304.08408
44. A. Milan, L. Leal-Taixé, I. Reid, S. Roth and K. Schindler, "MOT16: A benchmark for multi-object tracking," arXiv:1603.00831, 2016 (MOT16/MOT17). arXiv
45. P. Dendorfer, H. Rezatofighi, A. Milan, J. Shi, D. Cremers, I. Reid, S. Roth, K. Schindler and L. Leal-Taixé, "MOT20: A benchmark for multi object tracking in crowded scenes," arXiv:2003.09003, 2020. arXiv

46. P. Sun, J. Cao, Y. Jiang, Z. Yuan, S. Bai, K. Kitani and P. Luo, “DanceTrack: Multi-object tracking in uniform appearance and diverse motion,” *CVPR*, 2022. arXiv:2111.14690
47. Y. Cui, C. Zeng, X. Zhao, Y. Yang, G. Wu and L. Wang, “SportsMOT: A large multi-object tracking dataset in multiple sports scenes,” *ICCV*, 2023. arXiv:2304.05170
48. F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan and T. Darrell, “BDD100K: A diverse driving dataset for heterogeneous multitask learning,” *CVPR*, 2020. arXiv:1805.04687
49. C. Doersch, Y. Yang, M. Vecerik, D. Gokay, A. Gupta, Y. Aytar et al., “TAPIR: Tracking any point with per-frame initialization and temporal refinement,” *ICCV*, 2023. arXiv:2306.08637
50. N. Karaev, I. Rocco, B. Graham, N. Neverova, A. Vedaldi and C. Rupprecht, “CoTracker: It is better to track together,” *ECCV*, 2024. arXiv:2307.07635
51. N. Karaev, I. Makarov, J. Wang, N. Neverova, A. Vedaldi and C. Rupprecht, “CoTracker3: Simpler and better point tracking by pseudo-labelling real videos,” arXiv:2410.11831, 2024. arXiv
52. Z. Tang, S. Wang, D. C. Anastasiu et al., “The 9th AI City Challenge,” *ICCV Workshops*, 2025. arXiv:2508.13564
53. VOT Challenge organizers, “The Visual Object Tracking (VOT) challenge series,” website, 2013–present. votchallenge.net

DLCV 07・世界模型：影片、潛在動態與規劃

世界模型是什麼、模型式強化學習如何使用它（World Models、Dreamer 系列、MuZero）、為什麼影片生成器被當成世界模擬器、潛在動態與 JEPA 式預測表示如何運作，以及如何評估它們，並附上完整的 CPU 實驗。

網站版本: https://blog.shyandram.dev/zh-tw/learn/deep-learning/dl-07-world-models/

你將學到

- 世界模型是什麼，以及觀測、隱藏狀態、信念（belief）與動作的差別。
- 模型式強化學習如何以三種方式使用學到的模型：線上規劃、在想像中學習策略，以及用價值等價模型做樹搜尋；並以 World Models、PlaNet、Dreamer 系列、TD-MPC2 與 MuZero 為例。
- 為什麼影片預測與生成被視為「世界模擬器」、為什麼平方誤差預測器會產生模糊，以及 Genie 這類互動式生成器能證明什麼、不能證明什麼。
- 循環狀態空間模型與 JEPA 式預測表示如何在不重建（或重建）像素的情況下學習潛在動態。
- 如何擬合一個小型動態模型、量測多步誤差、用它規劃，並偵測規劃器何時在利用模型錯誤，全部在 CPU 上完成。
- 如何評估世界模型，以及哪些問題仍然開放。

先看全貌

追蹤問的是物體去了哪裡。世界模型問的是接下來可能發生什麼，尤其是**如果代理人採取某個動作**會發生什麼。機器人靠近杯子時，光有物體標籤不夠：它需要預測夾爪和杯子會怎麼移動、哪些結果不確定，以及哪個動作較可能達成目標。

> **用最簡單的話說**
>
> 推玩具車之前，你會先想像它會跑到哪裡。你可以想像輕推和用力推，挑比較好的那一種，推下去，再看看實際發生了什麼。世界模型就是這種想像力的學習版本。規劃器則是在想像中試推、挑出一個動作的那部分。每推一次就重新看一眼，能讓想像中的小錯誤不會越積越多。

「世界模型」一詞涵蓋好幾個研究傳統。Ding 等人依兩種功能整理文獻：建立能捕捉世界運作方式的內部表示，以及預測未來狀態以模擬並引導決策 [1]。本章採用一個操作型定義：**世界模型是一個學習得到的模型，它表示足夠多的環境資訊，能在指定條件（通常包括代理人自己的動作）下預測相關的變化**。它的用處取決於任務。用來渲染場景的模型、用來學習表示的模型，以及用來控制機器人的模型，可能需要非常不同的資訊。

觀測、狀態、信念與動作

白話版。相機影像不是世界本身，只是世界的一個局部視角。世界模型必須記住目前看不到的東西，例如球的速度，或被遮住的物體。

令 s_t 表示時間 t 的真實環境狀態，o_t 為觀測，a_t 為執行的動作：

$$s_{t+1} \sim p(s_{t+1} \mid s_t, a_t), \qquad o_t \sim p(o_t \mid s_t).$$

第一個分布是**轉移**（動態）模型，第二個是**觀測**模型。狀態可能包含物體位置、速度與接觸條件。影像只呈現一個投影，帶有遮擋與感測器雜訊；Szeliski 的教科書涵蓋這個觀測過程中的幾何與運動 [3]。兩張完全相同的影像可能有不同的未來：同一個位置的球，可能正往左也可能正往右。

因此，學習得到的模型會從歷史建立表示：

$$z_t = E_\theta(o_{0:t}, a_{0:t-1}), \qquad p_\theta(z_{t+1} \mid z_t, a_t),$$

其中 E_θ 是參數為 θ 的編碼器，$o_{0:t}$ 是到時間 t 為止的觀測序列，$p_\theta(z_{t+1} \mid z_t, a_t)$ 是學到的潛在轉移。編碼器不會自動讓 z_t 成為充分的狀態。若它丟掉了速度、被遮住的物體或某個相關的材質性質，轉移模型就無法找回這些資訊。循環記憶或一小段影片上下文會有幫助，但必須在實際的遮擋與延遲下測試。

當狀態只能被部分觀測時，有原則的對象是**信念**：在已知歷史下，對可能狀態的一個機率分布。Bishop 對序列潛在變數模型（隱藏馬可夫模型與線性動態系統）的說明提供了機率背景 [2]。一個緊湊的潛在向量可以近似這個分布中有用的部分；它不必一對一地對應到人類看得懂的物理變數。

預測的分解

對一個候選的動作序列，未來觀測的機率可分解為

$$p_\theta(o_{t+1:t+H} \mid o_{\le t}, a_{0:t+H-1}) = \int p_\theta(z_t \mid o_{\le t}, a_{<t}) \prod_{k=t}^{t+H-1} p_\theta(z_{k+1} \mid z_k, a_k)\, p_\theta(o_{k+1} \mid z_{k+1})\, dz_{t:t+H},$$

其中 H 是預測視野。三個因子分別是**推論**（給定過去，目前的潛在狀態是什麼？）、**轉移**（它在這些動作下如何演變？）與**渲染**（相機會看到什麼？）。如果目標與成本定義在潛在空間中，規劃器可以直接在 z 上運作，省略像素解碼器。本章許多設計選擇，其實就是決定要學哪些因子、怎麼學。

模型式強化學習

白話版。強化學習（reinforcement learning, RL）靠嘗試錯誤學會行動。如果代理人也學會世界如何回應它的動作，它就能在腦中練習，大幅減少真實嘗試的次數。

問題設定

RL 代理人以離散步驟與環境互動。每一步它觀測環境，依**策略** $\pi(a_t \mid s_t)$ 選擇 a_t，得到獎勵 r_t，環境隨之改變。目標是最大化期望**回報**

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k},$$

其中 $\gamma \in [0,1)$ 是折扣因子，讓近期獎勵比遠期獎勵更重要。**價值** $V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$ 是從狀態 s 出發、依循 π 時的期望回報。

無模型（model-free）方法直接從真實轉移學習策略或價值。**模型式**（model-based）方法另外學習轉移與獎勵的模型，並用它減少真實互動。這個想法由來已久：Sutton 的 Dyna 架構（1991）就已經把從真實經驗學習與在學到的模型所模擬的經驗上規劃結合起來，兩者使用同樣的價值更新 [4]。Moerland 等人的綜述沿兩個軸整理這個領域：如何學習動態模型（處理隨機性、不確定性、部分可觀測性與時間抽象），以及規劃與學習如何整合（從哪裡開始規劃、給規劃多少預算、怎麼規劃，以及如何把結果回饋到行動與學習中）[5]。

使用模型的三種方式

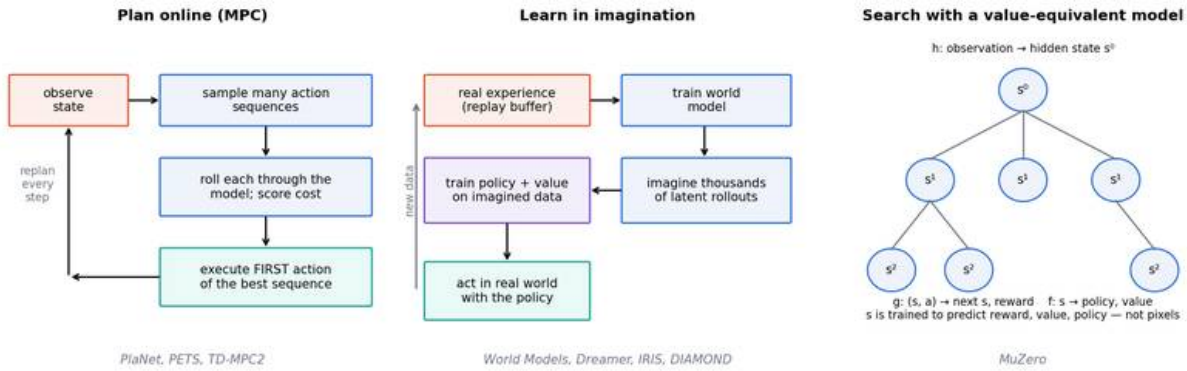


圖 *1* — 使用學到的模型的三種方式。同一個模型可以扮演不只一種角色；差別在於決策是在哪裡算出來的。

1. **線上規劃（模型預測控制）**。每一個真實步驟，都在模型中模擬候選動作序列以尋找好的序列，只執行第一個動作，觀測，再重複。PlaNet [7]、PETS [29] 與 TD-MPC2 [11] 屬於這一類。
2. **在想像中學習**。用模型產生的軌跡訓練一個策略（通常還有價值函數），執行時直接用策略行動，不再搜尋。World Models [6]、Dreamer 系列 [8], [9], [10]、IRIS [12] 與 DIAMOND [13] 屬於這一類。
3. **用價值等價模型搜尋**。學一個只預測規劃所需量（獎勵、價值、策略）、不預測觀測的模型，並在它的潛在空間中做樹搜尋。MuZero [14] 是代表。

三者有一個共同的危險：模型總會在某些地方出錯，而最佳化器非常擅長找到正好是那些地方。我們會在「不確定性與模型被利用」一節回到這點。

為什麼誤差會累積

假設學到的轉移 $\hat{f}$ 的單步誤差最多為 ε，而真實動態 f 是常數為 L 的 Lipschitz 函數（狀態的小變化最多讓下一個狀態變化 L 倍）。從同一個狀態出發、施加相同動作，k 步後預測與真實狀態的差距 e_k 滿足 $e_{k+1} \le L e_k + \varepsilon$，因此

$$e_H \le \varepsilon \sum_{k=0}^{H-1} L^k = \varepsilon \, \frac{L^H - 1}{L - 1} \quad (L \neq 1).$$

當 $L > 1$ 時，這個上界隨視野 H 指數成長；當 $L < 1$ 時，它飽和於 $\varepsilon/(1-L)$。這是最壞情況的上界，但它解釋了兩個常見設計：頻繁重新規劃（MPC）與使用短的模型推演。Janner 等人把第二個想法具體化：他們的 MBPO 演算法從真實狀態分岔出短的模型推演，而不是從頭想像長軌跡，並從理論與實驗兩方面論證這能保留模型資料的好處、同時限制其偏差 [30]。

World Models：壓縮、預測、控制

白話版。先學會用幾個數字概括每張畫面，再學會預測下一個概括，最後在上面訓練一個很小的控制器。控制器甚至可以完全在模型的「夢境」裡訓練。

Ha 與 Schmidhuber 的 *World Models*（以「Recurrent World Models Facilitate Policy Evolution」發表於 NeurIPS 2018，另有互動式線上版本）讓這個想法在深度學習中廣為人知 [6]。他們的代理人有三個部分：

- **V（視覺）**：一個變分自編碼器，把每張 64×64 的畫面壓縮成小的潛在向量 z_t。
- **M（記憶）**：一個帶混合密度輸出的循環網路，在給定 z_t、動作與隱藏狀態 h_t 時，預測 z_{t+1} 的分布。
- **C（控制器）**：一個刻意設計得極小的線性策略 $a_t = W_c[z_t; h_t] + b_c$，以演化策略而非反向傳播訓練。

在賽車任務中，使用 V 與 M 特徵的控制器解決了任務；在 VizDoom 任務中，控制器完全在 M 模擬的環境中訓練，再轉移到真實遊戲 [6]。這個設計把**感知**、**預測**與**決策**分開，並顯示當表示已經帶有相關動態時，控制器可以非常小。它也揭露了核心風險：在不完美夢境中訓練的控制器，可能學會利用夢境的錯誤。

潛在動態：從 PlaNet 到 Dreamer 系列

白話版。不預測未來的影像，而是預測一個緊湊的隱藏狀態，只在訓練時才把影像解碼出來。一個同時包含確定性記憶與隨機部分的隱藏狀態，可以表示代理人記得的東西，也能表示它不確定的東西。

循環狀態空間模型

Hafner 等人的 PlaNet 從影像學習環境動態，並在潛在空間中以快速線上規劃選擇動作；它的關鍵建模想法是同時包含確定性與隨機成分的潛在動態模型，即**循環狀態空間模型**（recurrent state-space model, RSSM）[7]。RSSM 保留一個確定性記憶 h_t 與一個隨機狀態 z_t：

$$h_t = f_\theta(h_{t-1}, z_{t-1}, a_{t-1}), \qquad z_t \sim p_\theta(z_t \mid h_t) \text{（prior）}, \qquad z_t \sim q_\theta(z_t \mid h_t, o_t) \text{（posterior）}.$$

這裡 f_θ 是一個循環單元；**先驗**（prior）在沒看到目前影像的情況下預測 z_t（想像時就用它）；**後驗**（posterior）還會看到影像 o_t（訓練時以及過濾真實觀測時使用）。多個輸出頭讀取模型狀態 (h_t, z_t)，預測影像 $\hat{o}_t$、獎勵 $\hat{r}_t$ 與延續旗標（回合是否繼續）。

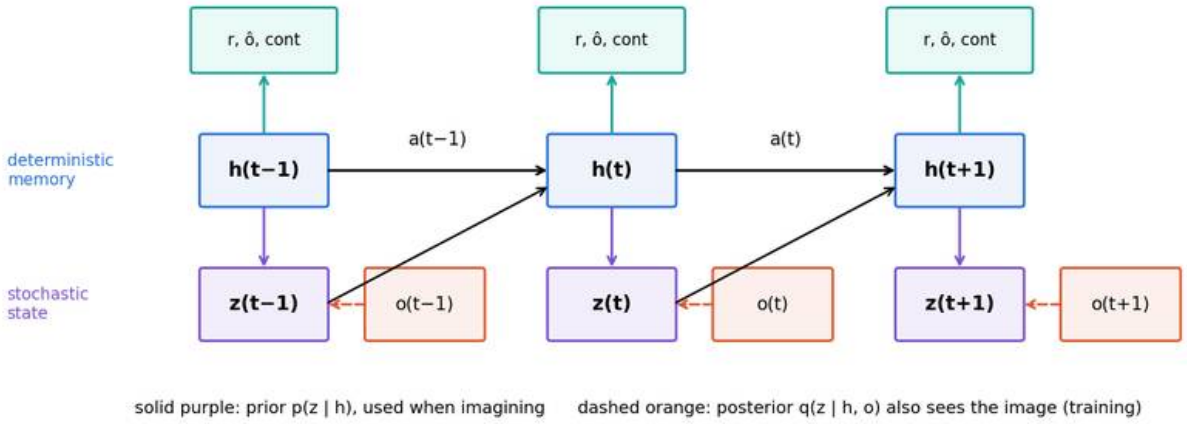


圖 2 — 展開的 *RSSM*。想像時只用先驗路徑（實線）；訓練時 z 還會以真實影像為條件（虛線）。

訓練時最大化觀測序列概似的變分下界。一個簡化的單步損失為

$$\mathcal{L}_t = -\log p_\theta(o_t \mid h_t, z_t) - \log p_\theta(r_t \mid h_t, z_t) + \beta\, D_{\mathrm{KL}}\big(q_\theta(z_t \mid h_t, o_t) \,\|\, p_\theta(z_t \mid h_t)\big),$$

其中 $\beta > 0$ 是 KL 項的權重。重建項讓狀態帶有資訊；KL 項則教先驗去預測後驗將會看到的東西，這正是想像所需要的能力。實際實作還會加入損失平衡與其他穩定化技巧，所以請把這個式子當成教學大綱，而不是任何一篇論文的精確損失。

Dreamer：在想像中學習行為

Dreamer（2020）保留 RSSM，但把線上規劃換成一個完全在想像出的潛在軌跡上訓練的 actor–critic。它的關鍵做法是把學到的狀態價值的解析梯度，沿著在緊湊潛在空間中想像出的軌跡反向傳播 [8]。代理人從由真實資料推得的潛在狀態出發，用 actor 想像 H 步，用學到的 critic 評分，並以自舉目標（例如 λ-回報）訓練 critic：

$$V_t^\lambda = \hat{r}_t + \gamma\, \hat{c}_t \big[(1-\lambda)\, v_\psi(s_{t+1}) + \lambda\, V_{t+1}^\lambda \big],$$

其中 $\hat{r}_t$ 與 $\hat{c}_t$ 是預測的獎勵與延續，v_ψ 是 critic，$\lambda \in [0,1]$ 在單步自舉（$\lambda = 0$）與較長的想像回報（$\lambda = 1$）之間取捨。actor 則被更新以提高這些價值。

這條路線持續發展：

- **DreamerV2**（2021）改用**離散**（類別型）潛在變數，成為第一個藉由在獨立訓練的世界模型中學習行為、在 55 款遊戲的 Atari 基準上達到人類水準的代理人 [9]。
- **DreamerV3**（Nature，2025）使用以正規化、平衡與轉換為基礎的穩健化技巧，使單一設定就能在超過 150 個任務上勝過專門方法，並且是第一個在沒有人類資料或課程設計的情況下、從零開始在 Minecraft 中收集到鑽石的演算法 [10]。

另外兩條相關路線說明 RSSM 只是眾多選擇之一。**TD-MPC2** 在一個**隱式**、沒有解碼器的世界模型（不重建影像）的潛在空間中規劃，以單一組超參數在 104 個線上 RL 任務上取得強勁結果，並訓練出一個能執行 80 個任務的 317M 參數代理人 [11]。**IRIS** 以離散自編碼器加上對影像 token 的自迴歸 Transformer 取代 RSSM，只用相當於兩小時的 Atari 遊戲經驗學習 [12]。**DIAMOND** 更進一步走向像素：它讓代理人在**擴散**世界模型中訓練，主張離散潛在變數可能丟掉對控制重要的視覺細節，並展示了這個模型可作為 3D 遊戲的互動式神經遊戲引擎 [13]。

MuZero：只預測規劃需要的東西

白話版。要下好一盤棋，你不需要預測畫面會長什麼樣子。你只需要預測局面有多好、哪些著法看起來有希望，以及會得到哪些獎勵。MuZero 正是學這些，然後進行搜尋。

AlphaZero 式的樹搜尋需要一個遊戲規則的模擬器。MuZero 移除了這個需求：它學一個反覆套用時能預測與規劃最直接相關之量的模型，也就是獎勵、動作選擇策略與價值 [14]。它有三個函數：

$$s^0 = h_\theta(o_1, \dots, o_t), \qquad (r^k, s^k) = g_\theta(s^{k-1}, a^k), \qquad (\mathbf{p}^k, v^k) = f_\theta(s^k),$$

其中 h_θ（表示函數）把觀測歷史編碼成隱藏狀態 s^0，g_θ（動態函數）把隱藏狀態與假設動作 a^k 映射到預測獎勵 r^k 與下一個隱藏狀態，f_θ（預測函數）輸出策略向量 $\mathbf{p}^k$ 與價值 v^k。蒙地卡羅樹搜尋展開 g_θ 來選擇動作。訓練時沿著真實軌跡把模型展開 K 步，並最小化

$$\mathcal{L} = \sum_{k=0}^{K} \Big[\ell^r(u_{t+k}, r^k) + \ell^v(z_{t+k}, v^k) + \ell^p(\boldsymbol{\pi}_{t+k}, \mathbf{p}^k) \Big] + c\|\theta\|^2,$$

其中 u 是觀測到的獎勵，z 是價值目標（自舉回報），$\boldsymbol{\pi}$ 是搜尋產生的造訪次數分布，ℓ^r, ℓ^v, ℓ^p 是各量的損失，c 是 L2 懲罰的權重。

隱藏狀態 s^k 從不被要求重建觀測。它只需要**價值等價**：用它規劃，必須得到與在真實環境中規劃相同的獎勵、價值與策略。以這個設計，MuZero 在 57 款 Atari 遊戲上創下新的最佳成績，並在沒有被告知規則的情況下，於圍棋、西洋棋與將棋上追平 AlphaZero 的超人表現 [14]。對視覺而言的教訓是：世界模型的預測目標應依用途而定；預測每個像素只是一種選擇，不是必要條件。

影片預測與生成：作為世界模擬器

白話版。如果模型能生成逼真的未來影片，也許它學會了世界如何運作。有時候確實如此；但更常見的是，它只學會了影片通常看起來像什麼。判斷是哪一種，才是難處所在。

以動作為條件的影片預測

Finn 等人在一個含 59,000 次機器人推動互動的資料集上，訓練了**以動作為條件的影片預測**模型；由於模型預測的是像素運動而非原始像素，它對沒見過的物體有部分泛化能力，而以不同的未來動作為條件，就產生了對不同未來的「視覺想像」[15]。這是最字面意義上的世界模型：影像進、動作進、影像出。

為什麼平方誤差預測器會模糊

真實的未來是不確定的。假設從同樣的過去出發，球有 $\frac{1}{2}$ 的機率往左、$\frac{1}{2}$ 的機率往右。對平方誤差而言，最佳的單一預測是**條件平均**：

$$\hat{o}^\star = \arg\min_{\hat{o}} \ \mathbb{E}[\|o_{t+1} - \hat{o}\|^2 \mid o_{\le t}] = \mathbb{E}[o_{t+1} \mid o_{\le t}],$$

在這裡就是兩顆半亮的球，一張永遠不會發生的影像。一個從兩種未來中**取樣**的生成模型會產生逼真的影像，但平均而言平方誤差**更高**。

```
import numpy as np

x = np.arange(64)
def frame(center):                      # a 1-D "image" with one bright ball
    return np.exp(-0.5 * ((x - center) / 2.0) ** 2)

# Same past, two equally likely futures: the ball bounces left or right.
futures = np.stack([frame(20), frame(44)])
mean_pred = futures.mean(axis=0)        # what an MSE-trained predictor converges to
sample_pred = futures[0]                # what a good generative sampler might output

def expected_mse(pred):                 # average over the two possible futures
    return np.mean([(pred - f) ** 2 for f in futures])

print(f"peak brightness: true future 1.00, mean prediction {mean_pred.max():.2f}")
print(f"expected MSE of mean prediction:   {expected_mse(mean_pred):.4f}")
print(f"expected MSE of a sharp sample:    {expected_mse(sample_pred):.4f}")
```

輸出：

```
peak brightness: true future 1.00, mean prediction 0.50
expected MSE of mean prediction:   0.0277
expected MSE of a sharp sample:    0.0554
```

清晰的樣本的期望誤差恰好是模糊平均的兩倍。這就是為什麼像素 MSE 會獎勵模糊、為什麼會出現隨機式影片模型，以及為什麼只用一個平方誤差數字評估世界模型，可能選錯模型。

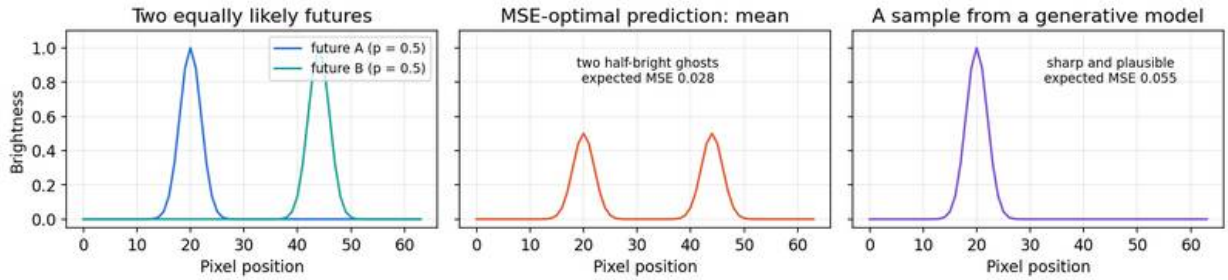


圖 3 — 上方程式碼的輸出。*MSE* 最佳的預測把互不相容的未來平均在一起；取樣結果合理，但 *MSE* 分數較差。

Babaeizadeh 等人的 SV2P 為潛在變數的每個樣本預測一個不同的可能未來，結果比沒有隨機性的同一模型好得多 [16]。現代影片生成器在大得多的規模上使用同樣的原理，採用自迴歸 token 模型或擴散與流模型（見ML 07 與ML 08）。

大型生成式世界模型

- **GAIA-1** 把駕駛世界建模視為對影片、文字與動作離散 token 的下一個 token 預測，能產生逼真的駕駛情境，並可細部控制自車行為與場景特徵 [17]。
- **UniSim** 結合在不同面向上豐富的資料集（影像中的物體、機器人資料中的密集動作、導航資料中的運動），學習真實世界互動的「通用模擬器」；它能模擬高階指令與低階控制，而完全在其中訓練的策略可零樣本部署到真實機器人上 [18]。
- **Genie** 是一個在未標註的網路影片上、不使用動作標籤訓練出的生成式互動環境。它由影片 tokenizer、自迴歸動態模型與一個從畫面變化推斷動作的**潛在動作模型**組成，因此即使訓練時沒有動作標籤，使用者仍能在生成的世界中逐格行動 [19]。這類潛在動作不一定對應到經過校正的實體致動器。
- **Genie 3** 由 Google DeepMind 於 2025 年 8 月發布，能從文字生成可即時導覽的世界，每秒 24 格、720p，並維持數分鐘的一致性。公告列出了限制：動作空間受限、難以模擬多個獨立代理人、真實地點不精確，以及互動只能持續數分鐘而非數小時 [20]。
- **Cosmos** 是 NVIDIA 的開放「世界基礎模型」平台：包含預訓練影片模型、影片 tokenizer、資料整理流程與後訓練範例，目的是微調成機器人與車輛專用的世界模型 [21]。

逼真的影片就代表理解物理嗎？

兩項仔細的研究給出的答案是：不一定。Motamed 等人的 **Physics-IQ** 基準測試流體力學、光學、固體力學、磁學與熱力學；在目前的影片模型（包括 Sora、Runway、Pika、Lumiere、Stable Video Diffusion 與 VideoPoet）中，物理理解嚴重不足，且與視覺真實感無關 [22]。Kang 等人在由古典力學支配的 2D 模擬器上訓練擴散影片模型，發現分布內完美泛化、組合式泛化可觀察到隨規模改善，但分布外則失敗；模型的行為像是在找最相近的訓練樣本，而不是抽象出規則 [23]。

用於機器人控制時，也要問「動作」指的是什麼：力、關節目標、夾爪指令、相機移動，還是推斷出的 token。單位錯誤、時間偏移或控制頻率不同，都會改變條件預測問題，即使影片看起來很有說服力。

預測式表示：JEPA 家族

白話版。預測每一個像素，會把力氣浪費在不重要的細節上，例如樹葉的紋理。不如預測缺失或未來部分的抽象**描述**。如果描述夠好，這個預測就對規劃有用。

聯合嵌入預測架構（joint-embedding predictive architecture, JEPA）用編碼器分別編碼可見的**上下文** x 與被隱藏的**目標** y，並訓練一個預測器把上下文嵌入映射到目標嵌入：

$$\mathcal{L} = \| P_\phi(E_\theta(x),\, m) - \mathrm{sg}[\bar{E}(y)] \|^2,$$

其中 E_θ 是上下文編碼器，$\bar{E}$ 是目標編碼器，P_ϕ 是以目標位置資訊 m（例如在影像或時間中的位置）為條件的預測器，sg（stop-gradient）阻止梯度流入目標分支。沒有任何東西在像素空間中重建。危險在於**崩塌**（collapse）：如果兩個編碼器都輸出常數，損失就是零。實務上用不對稱設計（只有一側有預測器、stop-gradient，以及不以同一梯度訓練的目標編碼器）來避免崩塌；ML 05 更詳細地討論這些自監督機制。

- **I-JEPA** 從一個資訊豐富的上下文區塊，預測同一張影像中數個大型目標區塊的表示，並發現遮罩策略（大而具語意的目標、空間上分散的上下文）是在不使用手工資料擴增的情況下學到語意特徵的關鍵 [24]。
- **V-JEPA** 把特徵預測當成獨立的目標應用到影片上，在 200 萬支影片上訓練，不使用預訓練影像編碼器、文字、負樣本或重建，所得的凍結特徵在動作類與外觀類任務上都表現良好 [25]。
- **V-JEPA 2** 先在超過一百萬小時的網路影片上預訓練不含動作的 JEPA，再用不到 62 小時的未標註機器人影片，後訓練出一個潛在的**以動作為條件**的世界模型 V-JEPA 2-AC；以影像目標進行規劃後，它在兩個實驗室的 Franka 機械手臂上實現零樣本的拾取與放置，且沒有使用來自這些環境的資料、任務專屬訓練或獎勵 [26]。被動影片與動作資料的分工，讓本章所說的觀測／動作區別變得具體。
- **V-JEPA 2.1** 加入可見與被遮罩 token 都參與的密集預測損失、跨中間層的深層自監督，以及適用影像與影片的多模態 tokenizer，目標是空間上有結構、時間上一致的密集特徵 [27]。
- **DINO-WM** 用現成的編碼器展示同樣的想法：它從離線軌跡預測未來的 DINOv2 patch 特徵，並把動作序列最佳化到接近目標影像的特徵來規劃，不重建像素，也不學獎勵模型 [28]。

特徵空間中的目標需要小心。距離 $\|z - z_{\text{goal}}\|^2$ 只會獎勵表示中保留下來的東西。如果表示忽略了夾爪是否閉合，規劃器可能到達一個看起來相似的狀態，卻沒有真的抓住物體。單憑特徵品質更好，並不能證明編碼器能預測任意動作的後果。

各家族之間的關係

家族	預測目標	主要好處	要檢驗的問題
潛在動態（RSSM、TD-MPC2）	緊湊狀態、獎勵、價值	便宜、可重複推演	潛在狀態是否保留任務相關動態？
價值等價（MuZero）	只有獎勵、價值、策略	不在無關細節上花力氣	獎勵改變時還能遷移嗎？
自迴歸影片（GAIA-1、Genie、IRIS）	下一張畫面或 token	明確的序列分解	長推演時誤差會漂移嗎？
擴散／流影片（DIAMOND、Cosmos）	一起生成的畫面或片段	表達力強、能表示不確定的未來	能否在控制延遲內以動作操控結果？

家族	預測目標	主要好處	要檢驗的問題
預測式嵌入（JEPA、DINO-WM）	目標特徵	不需像素重建就能學到結構	動作是否被表示？目標是否有意義？

這些選擇彼此相容。Transformer 可以參數化其中任何一種；潛在世界模型可以透過擴散解碼；影片生成器可以外層用自迴歸、內層用擴散。架構、預測目標、時間分解與訓練目標是各自獨立的設計決策。

從資料學習動態

訓練樣本應保留順序與動作對齊：

$$(o_t,\, a_t,\, o_{t+1},\, r_t,\, \text{done}_{t+1}),$$

其中 done_{t+1} 標記回合結束。記錄每個動作發生在每個相機時間戳之前或之後。依完整回合、環境實例與相關受試者切分；隨機切分相鄰畫面會洩漏幾乎相同的場景，讓轉移預測器看起來比實際更好（資料洩漏的一般討論見ML 10）。

資料收集策略很重要。在緩慢、輕柔互動上訓練的模型，可能在劇烈運動時失敗。當資料中沒有動作時，觀察到的相關性特別麻煩：模型可能學到手靠近時杯子會動，卻完全沒有證據知道其他推法會怎樣。這就是被動影片與介入式資料的差別。

單步損失衡量從真實歷史出發的預測。**多步損失**展開模型自己的預測並懲罰累積誤差。展開會改變訓練難度與梯度路徑；它並不能消除觀測互動範圍之外的分布偏移。

完整 CPU 實驗：擬合、預測、規劃

我們使用二維狀態 $x_t = (\text{position}, \text{velocity})$（位置與速度）、純量動作 u_t，以及一個有阻尼的線性系統：

$$x_{t+1} = Ax_t + Bu_t + \epsilon_t,$$

其中 A 是 2×2 狀態轉移矩陣，B 是動作輸入向量，ϵ_t 是微小的高斯雜訊。這個系統刻意設計成完全可觀測，以便在加入影像編碼器與部分可觀測性之前，先單獨檢視動態與規劃。訓練與測試回合各自獨立產生。我們只在訓練轉移上擬合，並用相同的保留動作同時評估單步預測與開迴路推演。

規劃器使用**交叉熵方法**（cross-entropy method, CEM）：從高斯分布取樣候選動作序列，保留模型下成本最低的「菁英」序列，把高斯的平均與分散重新擬合到它們，然後重複。它只執行一個動作，取得新的真實狀態後重新規劃。這就是模型預測控制。

```
import numpy as np

rng = np.random.default_rng(7)
A = np.array([[1.0, 0.1], [0.0, 0.94]])
B = np.array([0.005, 0.1])
```

```
def collect(n, steps=40):
    x = np.zeros((n, steps + 1, 2))
    x[:, 0] = rng.normal(size=(n, 2)) * [0.8, 0.3]
    u = rng.uniform(-1, 1, size=(n, steps))
    for t in range(steps):
        x[:, t + 1] = (x[:, t] @ A.T + u[:, t, None] * B
                       + rng.normal(scale=0.002, size=(n, 2)))
    return x, u

train, train_u = collect(120)
test, test_u = collect(80)
phi = np.column_stack([train[:, :-1].reshape(-1, 2), train_u.ravel()])
target = train[:, 1:].reshape(-1, 2)
W = np.linalg.lstsq(phi, target, rcond=None)[0]
W_no_action = np.linalg.lstsq(phi[:, :2], target, rcond=None)[0]

def predict(x, u):
    return np.concatenate([x, np.asarray(u)[..., None]], axis=-1) @ W

one_step = predict(test[:, :-1], test_u)
rollout = np.zeros_like(test)
rollout[:, 0] = test[:, 0]
for t in range(test_u.shape[1]):
    rollout[:, t + 1] = predict(rollout[:, t], test_u[:, t])
rmse = lambda a, b: np.sqrt(np.mean((a - b) ** 2))
print("One-step:", rmse(one_step, test[:, 1:]))
print("Without actions:", rmse(test[:, :-1] @ W_no_action, test[:, 1:]))
print("Final-horizon rollout:", rmse(rollout[:, -1], test[:, -1]))

def plan(x, mean, horizon=20, population=256):
    std = np.ones(horizon)
    for _ in range(4):
        candidates = np.clip(rng.normal(mean, std, (population, horizon)), -1, 1)
        imagined = np.broadcast_to(x, (population, 2)).copy()
        costs = np.zeros(population)
        for t in range(horizon):
            imagined = predict(imagined, candidates[:, t])
            costs += (imagined[:, 0] ** 2 + 0.15 * imagined[:, 1] ** 2
                      + 0.02 * candidates[:, t] ** 2)
        costs += 20 * imagined[:, 0] ** 2
        elite = candidates[np.argsort(costs)[:32]]
        mean = elite.mean(axis=0)
        std = elite.std(axis=0) + 0.05
    return mean

real = np.array([1.5, 0.0])
mean = np.zeros(20)
for _ in range(40):
    sequence = plan(real, mean)
    real = A @ real + B * sequence[0]  # real system; never used inside plan()
    mean = np.r_[sequence[1:], 0.0]
print("Controlled final state:", real)
assert rmse(one_step, test[:, 1:]) < 0.01
```

```
assert abs(real[0]) < 0.1
```

輸出：

```
One-step: 0.0019866110218121832
Without actions: 0.04106321931941812
Final-horizon rollout: 0.013772759824274517
Controlled final state: [-0.00453985 -0.0210205 ]
```

單步 RMSE（0.0020）落在雜訊水準；拿掉動作輸入後增加到二十倍（0.0411），可見動作確實重要。即使是這個幾乎完美的線性模型，視野 40 的開迴路推演誤差（0.0138）也約為單步誤差的七倍，是上面推導的誤差累積的溫和例子。MPC 把位置從 1.5 帶到約 −0.0045，而零動作時位置停在 1.5。這些是本腳本的合成結果，不是已發表的比較。

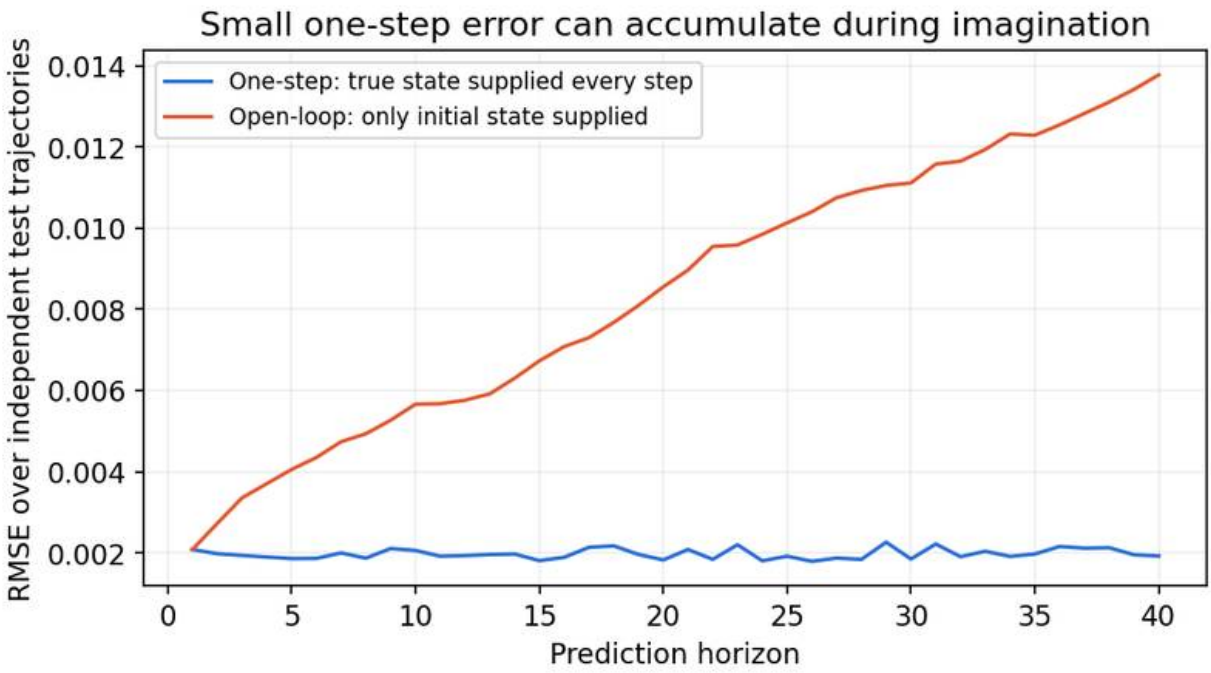


圖 4 — 獨立的合成測試回合。單步評估每一步都拿到真實狀態；開迴路評估只拿到初始狀態與保留的動作。

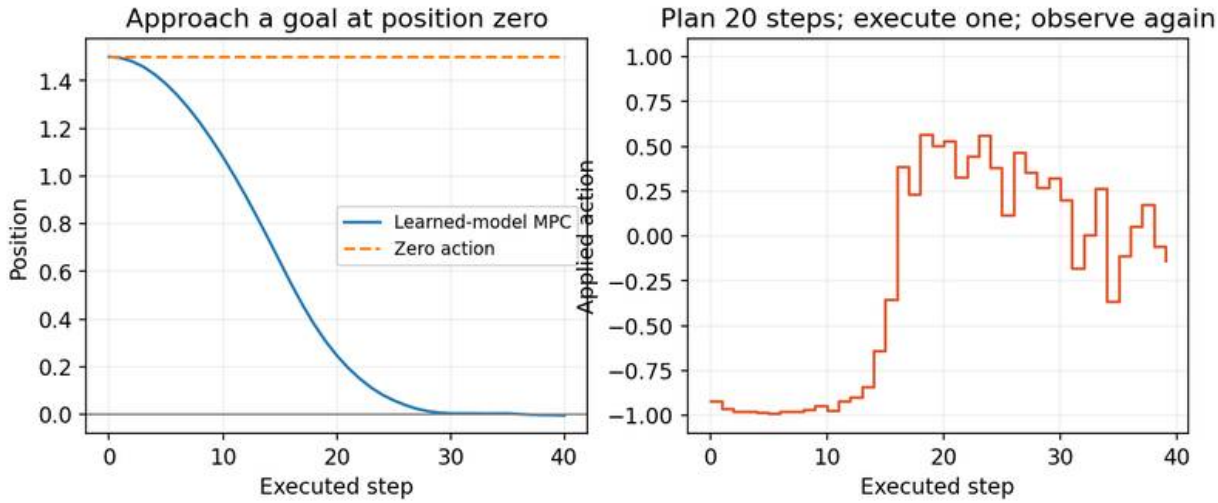


圖 5 — *CEM* 使用學到的轉移模型規劃。在合成環境中只執行第一個動作，再從新的觀測重新規劃。

把 x_t 換成影像嵌入，並不能重現 Dreamer 或 V-JEPA 2-AC：那會多出編碼器學習問題、潛在不確定性，以及需要一個有效的目標或獎勵。

規劃、不確定性與模型被利用

白話版。規劃器會去找模型最喜歡的動作。如果模型在某處是錯的，規劃器會很樂意往那裡跑。知道模型**在哪裡**不確定，才能讓它保持誠實。

有限視野的規劃目標可寫成

$$a^{*}_{t:t+H-1} = \arg\min_{a_{t:t+H-1}} \mathbb{E}_{\theta}\Big[\sum_{k=1}^{H} c(z_{t+k}, a_{t+k-1}) + \lambda_{\mathrm{T}}\, c_{\mathrm{T}}(z_{t+H})\Big],$$

其中 c 是每步成本，c_{T} 是以 λ_{T} 加權的終端成本，期望值針對模型預測的潛在狀態 z。**模型預測控制**執行第一個動作、重新觀測、再重新規劃。回饋可以修正漂移，但無法挽救一個其直接後果就被嚴重誤判的動作。

兩種不確定性

偶然（aleatoric）不確定性是本質上的隨機性（擲硬幣、可能往左也可能往右的球）；資料再多也消除不了。**認知**（epistemic）不確定性來自資料不足（模型從沒見過這種情況）；更多資料可以消除它。對生成模型取樣可以表示偶然不確定性，但樣本多樣不代表已校準。認知不確定性常以**集成**（ensemble）近似：在 bootstrap 重抽樣上或以不同初始化訓練的多個模型，在資料密集處意見一致，在資料稀少處則傾向分歧。Chua 等人的 PETS 把機率式網路（處理偶然雜訊）組成集成（處理認知不確定性），搭配以取樣為基礎的不確定性傳遞與 CEM 規劃，在數個基準上以少得多的樣本追平強大無模型方法的最終表現 [29]。

程式碼：規劃器利用學到的模型

代理人只有動作落在 $[-1, 1]$ 的記錄資料，但規劃器可以選擇 $[-3, 3]$ 之間的任何動作。

```
import numpy as np
from sklearn.neural_network import MLPRegressor

rng = np.random.default_rng(0)
true_reward = lambda a: np.sin(3 * a) * np.exp(-0.3 * a ** 2)   # unknown to the agent

# Logged interactions only cover actions in [-1, 1].
a_data = rng.uniform(-1, 1, 200)
r_data = true_reward(a_data) + 0.05 * rng.normal(size=200)

ensemble = []
for k in range(5):                          # bootstrap + different initialization
    idx = rng.integers(0, 200, 200)
    m = MLPRegressor(hidden_layer_sizes=(64, 64), max_iter=3000, random_state=k)
    ensemble.append(m.fit(a_data[idx, None], r_data[idx]))

grid = np.linspace(-3, 3, 601)
preds = np.stack([m.predict(grid[:, None]) for m in ensemble])
mean, std = preds.mean(0), preds.std(0)

for name, score in [("single model", preds[0]),
                    ("ensemble mean", mean),
                    ("pessimistic: mean - 2 std", mean - 2 * std)]:
    a_star = grid[np.argmax(score)]       # the "planner": pick the best-looking action
    print(f"{name:26s} picks a = {a_star:+.2f}, predicted {score.max():.2f}, "
          f"true {true_reward(a_star):.2f}")
print(f"mean std inside data {std[np.abs(grid) <= 1].mean():.3f}, "
      f"outside {std[np.abs(grid) > 1.5].mean():.3f}")
```

輸出：

```
single model               picks a = -3.00, predicted 1.79, true -0.03
ensemble mean              picks a = -3.00, predicted 1.58, true -0.03
pessimistic: mean - 2 std  picks a = +0.43, predicted 0.87, true 0.91
mean std inside data 0.025, outside 0.321
```

單一模型與集成平均都在資料範圍外外插出一條上升直線，規劃器於是衝到動作空間的邊緣，而那裡真實獎勵約為零。集成在資料外的分歧是資料內的十三倍；**悲觀**分數（平均減兩倍標準差）讓規劃器留在資料範圍內，找到真實獎勵為 0.91 的動作。限制動作、縮短視野、懲罰分歧以及用真實回饋測試，是實務上的對策。不過，集成成員意見一致並不保證正確：五個模型可能共享同樣的偏差。

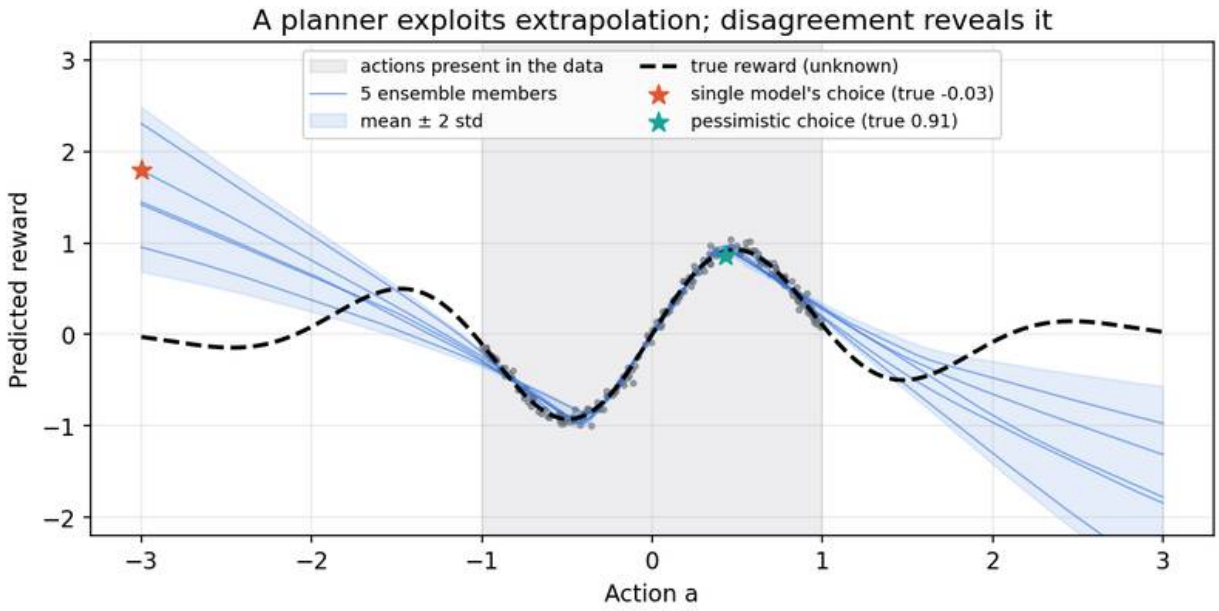


圖 *6* — 上方程式碼的輸出。對學到的模型做最佳化，會把規劃器推向模型最不可靠的地方；集成的分散程度能揭露這些區域。

如何評估世界模型

問題	量測	重要的控制條件
能預測已知動態嗎？	單步與依視野索引的狀態或特徵誤差	獨立回合；固定動作序列
外觀能保持一致嗎？	重建與影片品質指標、身分一致性	比較相機運動與遮擋
動作的作用正確嗎？	相同初始上下文、指定不同動作	真實的以動作為條件的參考軌跡
遵守物理限制嗎？	碰撞、物體恆存、守恆；Physics-IQ 等基準 [22]	任務專屬容許誤差；分布外案例 [23]
對決策有幫助嗎？	成功率、回報、人工介入次數、控制延遲	在真實或可信的環境中測試

像素保真度與決策用處可能不一致，模糊的例子已經說明了這點。平均出來的預測可能平方誤差很低，卻呈現一個不可能的場景；準確的潛在預測器可能省略視覺上無關的紋理。每個指標都要一起報告預測目標、視野、條件與輸出解析度。

比較時，請納入持續性基準（預測「什麼都不變」）、無動作預測器，以及一個適合任務的簡單動態模型。分別評估保留的物體、視角、動作範圍與環境性質。除了想像中的結果，也要報告閉迴路控制結果，並計入編碼器與取樣器的延遲：每個動作要花十秒規劃的規劃器，對以 10 Hz 運作的機器人可能毫無用處。

現代觀點

綜述。 Moerland 等人對模型式 RL 提供了最系統化的整理：動態模型學習（隨機性、不確定性、部分可觀測性、時間抽象）、規劃與學習整合方式的分類，以及隱式（價值等價、MuZero 式）方法 [5]。Ding 等人（ACM Computing Surveys）依功能整理更廣泛的「世界模型」文獻，也就是理解世界與預測未來兩類，並回顧在生成式遊戲、自動駕駛、機器人與社會模擬上的應用 [1]。Zhu 等人從影片生成的角度綜述「通用世界模型」，涵蓋影片生成器、自動駕駛世界模型以及自主代理人中的世界模型，並討論其限制 [31]。Guan 等人則提供了專門針對自動駕駛世界模型的初步綜述 [32]。

2023–2026 年的現況。

- **模型式 RL 趨於成熟**。 DreamerV3 展示了單一設定在超過 150 個任務上都能運作，並於 2025 年發表在 Nature [10]；TD-MPC2 展示了無解碼器的潛在規劃能擴展到 317M 參數的多任務代理人 [11]；IRIS 與 DIAMOND 顯示 Transformer 與擴散世界模型在 Atari 100k 基準上具競爭力 [12], [13]。
- **影片生成器被重新定位為模擬器**。 GAIA-1、UniSim、Genie、Genie 3 與 Cosmos 在帶有動作、文字或推斷出之潛在動作的影片上訓練大型模型 [17], [18], [19], [20], [21]。它們能幫助代理人的最強證據，是 UniSim 把在模擬器中訓練的策略零樣本轉移到真實世界 [18]；最強的警告則是：真實感不代表懂物理 [22], [23]。
- **預測式表示走向控制**。 JEPA 系列從影像（I-JEPA）走到影片（V-JEPA），再到真實機器人上以動作為條件的規劃（V-JEPA 2-AC）與更密集的特徵（V-JEPA 2.1）[24], [25], [26], [27]；DINO-WM 顯示凍結的自監督特徵已能支援零樣本規劃 [28]。

開放問題。

- **長視野一致性**。誤差會累積；生成的世界會漂移；物體身分與畫面外的狀態很難維持數分鐘，更別說數小時 [20]。
- **因果與物理正確性**。在被動影片上訓練的模型學到的是相關性；分布外的物理會失敗 [22], [23]。介入式（動作）資料稀少又昂貴。
- **動作介面**。從影片推斷的潛在動作、機器人關節指令與文字指令是不同的東西；可靠地在它們之間轉換仍未解決。
- **被利用與不確定性**。在想像中訓練的規劃器與策略會利用模型錯誤；大型影片模型大多缺乏校準過的認知不確定性。
- **評估**。目前沒有公認的基準能跨領域把預測品質與後續決策品質連結起來；多數論文只報告影片指標或任務回報其中之一，很少兩者都有。
- **成本**。對大型影片模型取樣可能遠慢於即時，這與閉迴路控制相衝突。

重點整理

- 世界模型預測環境如何變化，通常是在代理人的動作之下；它的預測目標與動作介面決定了它能支援什麼。
- 觀測不等於狀態。部分可觀測性需要記憶或信念估計。
- 模型可以用來線上規劃（MPC）、在想像中訓練策略（World Models、Dreamer），或用價值等

價的潛在狀態做搜尋（MuZero）。
- RSSM 結合確定性記憶與隨機潛在變數；KL 項訓練先驗去想像後驗將會看到的東西。
- 平方誤差的影片預測會把可能的未來平均成模糊；隨機與生成式模型以較高的 MSE 換取逼真的樣本，但真實感不等於理解物理。
- JEPA 式模型預測特徵而不是像素；它們需要防止崩塌的設計，以及在特徵空間中有意義的目標。
- 單步誤差會在長視野中累積，最佳化器會利用模型錯誤；集成、悲觀估計、短推演與重新規劃是標準的防禦手段。
- 最有力的證據，是把受控的動作介入與保留資料上的預測，連結到在真實環境中的成功行為。

練習

1. **動作消融**。在 CPU 實驗中拿掉動作欄位。比較單步誤差、視野 40 的誤差與控制行為。為什麼一個不看動作的短期預測器仍可能看起來還不錯？

   提示

   想像的動作不再影響無動作預測器，所以規劃器完全無法比較它們的後果：每個候選都得到相同的預測成本。在一步之內，有阻尼的動態只讓狀態改變一點點，因此類似「保持不變」的預測仍能得到中等分數，這也是為什麼只看單步誤差是很弱的測試。

2. **部分觀測**。隱藏速度，擬合「位置到位置」的動態。然後改用最後兩個位置與前一個動作作為特徵。這段歷史何時有幫助？觀測雜訊在哪裡造成傷害？

   提示

   兩個位置的差可以估計速度（有限差分），所以雙位置模型能恢復大部分狀態。每個位置上的雜訊在除以時間步長時會被放大，因此觀測雜訊越大，好處越小。

3. **規劃視野**。比較視野 5、20 與 40，並把初始平均調整成對應長度。在多個獨立種子上量測最終距離、動作力度與執行時間。

   提示

   保持成本定義不變，並計算所有模型呼叫次數。太短的視野「看不到」提早煞車可以避免衝過頭；太長的視野成本更高，也累積更多模型誤差，不過在這個線性系統中誤差很小。

4. **領域偏移**。以速度阻尼 0.94 收集訓練回合，然後在阻尼為 0.75 的環境中執行控制。比較開迴路執行一個計畫與每一步重新規劃。找出一個訓練資料涵蓋不足的動作範圍。

   提示

   把學到的動態（以 0.94 擬合）與真實執行的動態（0.75）分開；不要在評估轉移上重新擬合。重新規劃使用新的觀測，所以能部分修正有偏差的模型；開迴路執行則做不到。

5. **平均與樣本**。把模糊的例子延伸成三個同樣可能的未來，位置在 10、32 與 54。計算平均預測與單一清晰樣本的期望 MSE。再提出一個會偏好樣本的指標。

提示

平均現在有三個三分之一高度的峰；期望 MSE 約為 0.037（平均）與 0.074（樣本），又是兩倍。這個倍數是普遍的：若 X 與 Y 是同一分布的獨立抽樣，則 $\mathbb{E}\|X-Y\|^2 = 2\,\mathbb{E}\|X-\mathbb{E}X\|^2$，所以完美的取樣器的 MSE 永遠是平均的兩倍。分布層級的指標（例如比較生成樣本集合與真實未來集合，就像 FID 類指標對影像所做的）或多個樣本中取最佳的 best-of-N 誤差，會獎勵清晰且合理的預測。

6. **悲觀的取捨**。在集成程式碼中，把「mean − 2 std」的係數 2 換成 0、0.5、1 與 4。畫出每種情況下選到的動作與其真實獎勵。係數太大時會出什麼問題？

提示

係數為 0 時規劃器會利用外插；係數增大時它會留在資料範圍內。過度悲觀會讓規劃器避開任何有些微分歧的動作，可能錯過資料範圍內的最佳動作，而在 RL 情境中還會停止探索。

參考文獻

1. J. Ding et al., "Understanding world or predicting future? A comprehensive survey of world models," *ACM Computing Surveys*, vol. 58, no. 3, 2025. doi arXiv
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006, ch. 13 "Sequential Data". book
3. R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed., Springer, 2022. book
4. R. S. Sutton, "Dyna, an integrated architecture for learning, planning, and reacting," *ACM SIGART Bulletin*, vol. 2, no. 4, 1991. doi
5. T. M. Moerland, J. Broekens, A. Plaat and C. M. Jonker, "Model-based reinforcement learning: a survey," *Foundations and Trends in Machine Learning*, vol. 16, no. 1, 2023. doi arXiv
6. D. Ha and J. Schmidhuber, "Recurrent world models facilitate policy evolution," NeurIPS, 2018; interactive version "World Models," arXiv:1803.10122. paper interactive
7. D. Hafner et al., "Learning latent dynamics for planning from pixels," ICML, 2019. paper
8. D. Hafner, T. Lillicrap, J. Ba and M. Norouzi, "Dream to control: learning behaviors by latent imagination," ICLR, 2020. arXiv
9. D. Hafner, T. Lillicrap, M. Norouzi and J. Ba, "Mastering Atari with discrete world models," ICLR, 2021. arXiv
10. D. Hafner, J. Pasukonis, J. Ba and T. Lillicrap, "Mastering diverse control tasks through world models," *Nature*, vol. 640, 2025. doi arXiv
11. N. Hansen, H. Su and X. Wang, "TD-MPC2: scalable, robust world models for continuous control," ICLR, 2024. arXiv
12. V. Micheli, E. Alonso and F. Fleuret, "Transformers are sample-efficient world models," ICLR, 2023. arXiv
13. E. Alonso et al., "Diffusion for world modeling: visual details matter in Atari," NeurIPS, 2024. arXiv

14. J. Schrittwieser et al., “Mastering Atari, Go, chess and shogi by planning with a learned model,” *Nature*, vol. 588, 2020. doi arXiv
15. C. Finn, I. Goodfellow and S. Levine, “Unsupervised learning for physical interaction through video prediction,” NIPS, 2016. arXiv
16. M. Babaeizadeh, C. Finn, D. Erhan, R. H. Campbell and S. Levine, “Stochastic variational video prediction,” arXiv:1710.11252, 2017. arXiv
17. A. Hu et al., “GAIA-1: a generative world model for autonomous driving,” technical report, arXiv:2309.17080, 2023. arXiv
18. S. Yang et al., “Learning interactive real-world simulators,” arXiv:2310.06114, 2023. arXiv
19. J. Bruce et al., “Genie: generative interactive environments,” arXiv:2402.15391, 2024. arXiv
20. Google DeepMind, “Genie 3: a new frontier for world models,” blog post, August 5, 2025. blog
21. NVIDIA (N. Agarwal et al.), “Cosmos world foundation model platform for physical AI,” arXiv:2501.03575, 2025. arXiv
22. S. Motamed, L. Culp, K. Swersky, P. Jaini and R. Geirhos, “Do generative video models understand physical principles?,” arXiv:2501.09038, 2025. arXiv
23. B. Kang et al., “How far is video generation from world model: a physical law perspective,” ICML, 2025. arXiv
24. M. Assran et al., “Self-supervised learning from images with a joint-embedding predictive architecture,” CVPR, 2023. doi arXiv
25. A. Bardes et al., “Revisiting feature prediction for learning visual representations from video,” arXiv:2404.08471, 2024. arXiv
26. M. Assran et al., “V-JEPA 2: self-supervised video models enable understanding, prediction and planning,” arXiv:2506.09985, 2025. arXiv
27. L. Mur-Labadia et al., “V-JEPA 2.1: unlocking dense features in video self-supervised learning,” arXiv:2603.14482, 2026. arXiv
28. G. Zhou, H. Pan, Y. LeCun and L. Pinto, “DINO-WM: world models on pre-trained visual features enable zero-shot planning,” arXiv:2411.04983, 2024. arXiv
29. K. Chua, R. Calandra, R. McAllister and S. Levine, “Deep reinforcement learning in a handful of trials using probabilistic dynamics models,” NeurIPS, 2018. arXiv
30. M. Janner, J. Fu, M. Zhang and S. Levine, “When to trust your model: model-based policy optimization,” NeurIPS, 2019. arXiv
31. Z. Zhu et al., “Is Sora a world simulator? A comprehensive survey on general world models and beyond,” arXiv:2405.03520, 2024. arXiv
32. Y. Guan et al., “World models for autonomous driving: an initial survey,” arXiv:2403.02622, 2024. arXiv

篇 VII

最佳化

OPT 01・決策建模與凸集合

把工程決策轉成變數、目標與限制，理解凸幾何、投影、分離，以及數值解的保證範圍。

網站版本: https://blog.shyandram.dev/zh-tw/learn/optimization/opt-01-modeling-convex-sets/

你將學到

- 把決策轉成變數、目標、限制與固定資料。
- 區分可行、最佳、不可行、無界與未達到下確界的問題。
- 理解仿射集合、凸組合、凸包與錐。
- 辨認半空間、多面體、範數球、橢球與半正定錐。
- 使用保留凸性的運算、投影與分離超平面。
- 檢查數值解對模型與實際應用各自提供什麼證據。

先看全貌

影像復原系統選擇輸出像素，排程系統選擇資源分配，擬合程序選擇係數。三者都有許多可選答案、比較答案好壞的方式，以及允許答案的規則。最佳化把這三個要素寫清楚。

本軌以 Stephen Boyd 與 Lieven Vandenberghe 的《Convex Optimization》[1] 為主軸，學習辨認、建立與求解具有全域保證的問題。前幾章建立集合、函數、標準問題與對偶；應用連到擬合、統計估計與幾何設計；最後討論數值演算法與現代延伸。

> **用最簡單的話說**
>
> 想像你在調配果昔，希望好喝，又有固定預算，不能用已經沒庫存的材料。材料用量是可以選的變數，口味是想改善的目標，預算與庫存是規則。若混合任意兩種合規配方仍合規，這些配方就形成凸集合。

比喻有其限制：口味不一定是凸函數，材料也可能只能整包購買。第一個能力是辨認實際決策的哪些部分適合凸模型、哪些不適合。合理建模，比對錯誤方程式使用快速求解器更重要。

凸最佳化也能出現在非凸學習流程中，例如在固定特徵上擬合線性分類器、計算受限預測、擬合校準曲線，或替困難問題建立下界。但它不會讓任意神經網路訓練自動變成凸問題。保證只屬於實際求解的那個數學模型。

從一句話到最佳化模型

白話版。先決定可以改什麼、已知什麼、什麼答案比較好，以及必須遵守哪些規則，再選演算法。

一般純量最佳化可寫成

$$\begin{aligned} \text{minimize}_{x\in\mathbb{R}^n} \quad & f_0(x) \\ \text{subject to} \quad & f_i(x) \le 0, \quad i = 1,\dots,m, \\ & h_j(x) = 0, \quad j = 1,\dots,p. \end{aligned}$$

x 是含 n 個決策變數的向量；f_0 是要最小化的純量目標；f_i 是 m 個不等式限制函數；h_j 是 p 個等式限制函數。固定量測與設計參數出現在函數中，但除非模型明確指定，否則不是決策變數。

例如兩種作業的資源用量為 x_1, x_2，容量限制可以是 $2x_1 + x_2 \le 5$，且 $x_1, x_2 \ge 0$。若固定目標配置為 b，$\|x - b\|_2^2$ 偏好接近目標。這些數字定義合成模型，不是產線量測。

單位也是模型的一部分。把金錢成本與影像重建誤差相加，需要權重慣例。換算尺度的係數應有可解釋含意，並經過檢查。影像從零到 255 改成零到一，其他項不變時，正則化的相對強度也可能改變。

建立簡短的建模紀錄：每個變數的意義與單位、固定輸入的來源、每個限制的理由，以及目標的解釋。這能避免把方便的公式誤認成唯一合理的任務描述。

限制與懲罰代表不同承諾

硬限制表示答案不被允許；懲罰表示某種偏差不理想，但可以換取其他好處。即使某次答案相似，兩者的含意仍不同。

例如強制 $x \ge 0$，所有可行影像都非負；對負值加平方懲罰，仍可能容許少量負值，以換取較好的資料項。增加懲罰係數不自動等於精確限制，有限精度下尤其如此。

預算限制設定明確上限，成本懲罰則表示與係數有關的偏好。係數選擇是應用決策，凸性不會替你決定。對偶章會推導乘子如何描述放寬硬限制的敏感度。

算例：容量限制下的目標配置

目標配置未必符合現有容量。最佳化找最近的容許配置，而不是把每個座標減少相同數量。

取合成目標 $b = (3, 3)$，在 $x \ge 0$ 與 $2x_1 + x_2 \le 5$ 下最小化 $\frac{1}{2}\|x - b\|_2^2$。目標消耗九單位容量，因此不可行。最佳解的容量邊界為作用中限制，其法向量為 $a = (2, 1)$，所以最短修正沿著 a：

$$x^\star = b - \alpha a, \qquad \alpha = \frac{a^\top b - 5}{a^\top a} = \frac{4}{5}, \qquad x^\star = \left(\frac{7}{5}, \frac{11}{5}\right).$$

其中 $x^\star$ 是候選配置、α 是修正尺度、a 是容量限制的法向量。代入可得 $2x_1^\star + x_2^\star = 5$，而且兩座標都為正，所以非負限制不會改變這個投影。

為什麼它是最近的點？對任何可行 x，都有 $a^\top(x - x^\star) \le 0$。展開目標差：

$$\frac{1}{2}\|x - b\|_2^2 - \frac{1}{2}\|x^\star - b\|_2^2 = \frac{1}{2}\|x - x^\star\|_2^2 - \alpha a^\top(x - x^\star) \ge 0.$$

其中 x 是任意可行競爭點，其餘符號如上定義。兩項都非負，因此候選點為全域最小解，目標值為 8/5。這個計算在使用求解器前，就連起變數、作用中規則、修正方向與最佳值 [1]。

可行性與最佳值

白話版。被允許的答案不一定是最佳答案。有時沒有任何答案符合規則，有時目標喜歡的是越跑越遠的選擇。

$$\mathcal{F} = \{x : f_i(x) \le 0 \text{ for every } i,\ h_j(x) = 0 \text{ for every } j,\ x \text{ in all required domains}\}.$$

$\mathcal{F}$ 是同時滿足所有限制與函數定義域的可行集合。集合中的點稱為可行點。最佳值為

$$p^* = \inf_{x \in \mathcal{F}} f_0(x).$$

p^* 是所有可達目標值的最大下界。最佳解 x^* 是達到 $f_0(x^*) = p^*$ 的可行點。下確界存在，不代表存在達到它的點。

要區分四種情況：可行且達到有限最佳值；可行集合為空；目標向負無限延伸；有限下確界卻沒有點達到。

例如最小化 x 且要求 $x > 0$，最佳值為零，但零不允許。最小化 $-x$ 且 $x \ge 0$，問題向下無界。同時要求 $x \ge 1$ 與 $x \le 0$，則不可行。求解器都可能沒有回傳有限最佳解，但原因完全不同。

非空緊緻可行集合上的連續目標會達到最小值。閉而無界的集合，若目標在無限遠成長得足夠，也可能存在最小值。解是否存在，是開始執行演算法前就屬於模型的問題。

數值狀態是有限精度計算的證據

求解器通常把限制殘差、最佳條件或原始—對偶 gap 與容忍值比較。成功狀態不表示實數意義下精確可行，更不保證量測、目標與應用限制都正確。

若應用必須真正守住安全上限，需說明數值容忍值如何對應該上限。$x \le 1$ 與 $10^6 x \le 10^6$ 數學等價，但未正規化殘差的尺度不同。

記錄殘差單位、尺度與容忍值。不同目標定義的數值不要直接比較。若停止原因是迭代上限，就如實報告；不能把點幾乎不動當成最佳性的證明。

仿射集合與凸集合

白話版。仿射集合包含任意兩點之間的整條直線；凸集合只需要包含它們之間的線段。

對 $u, v \in \mathbb{R}^n$：

$$x = \theta u + (1 - \theta) v.$$

u, v 為端點向量，x 為組合，純量 θ 控制直線位置。允許所有實數 θ，得到整條線；限制 $0 \le \theta \le 1$，得到線段。

包含所有仿射組合的集合是仿射集合；包含所有非負且係數總和一的凸組合，則是凸集合。所以仿射集合必為凸，但圓盤是凸而不是仿射。

有限多點的凸組合：

$$x = \sum_{i=1}^{r} \theta_i v_i, \qquad \theta_i \ge 0, \qquad \sum_{i=1}^{r} \theta_i = 1.$$

v_i 是 r 個端點，θ_i 是權重，組合是加權平均。非負很重要；外插可以離開凸集合。

凸性本身不要求平滑邊界、有界或閉。開球是凸，無界直線也是凸，單點也是凸。空集合慣例上也算凸，因為沒有違反定義的端點。

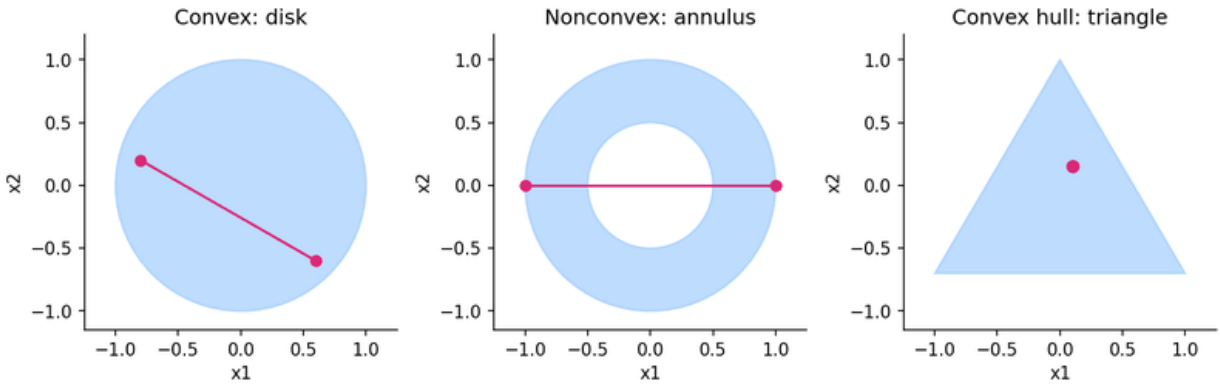


Figure 1 — 原創合成幾何。圓盤與填滿的三角形是凸集合；圓環不是，因為兩個允許端點的連線穿過被排除的孔洞。

反例能否證，取樣不能證明

要否證凸性，只需找到集合內兩點，使其中點或線段上其他點落在集合外。圓環可用相反端點平均成中心。

要證明凸性，使用定義或保留規則。隨機測很多點對能揭露實作錯誤，卻不能保證所有端點都成立。很小的非凸缺口可能完全沒有被抽到。

這與數值梯度檢查相似：它們是實作診斷，而數學證明處理全稱敘述。兩者用途不同，應標示清楚。

程式碼：凸組合與非凸反例

以下自成一體，驗證特定點。圓盤凸性的證明來自三角不等式，不是只有 assertion。

```
import numpy as np

u = np.array([0.8, 0.1])
v = np.array([-0.2, 0.7])
theta = np.linspace(0, 1, 101)
segment = theta[:, None] * u + (1 - theta[:, None]) * v
assert np.linalg.norm(segment, axis=1).max() <= 1 + 1e-12
ring_u, ring_v = np.array([1.0, 0.0]), np.array([-1.0, 0.0])
midpoint = (ring_u + ring_v) / 2
```

```
assert not (0.5 <= np.linalg.norm(midpoint) <= 1.2)
print("disk sampled-segment maximum norm:", np.linalg.norm(segment, axis=1).max())
print("ring midpoint:", midpoint, "norm:", np.linalg.norm(midpoint))
```

凸包與鬆弛

白話版。凸包填入原有選擇的所有加權平均。它能簡化離散決策，但新加入的點未必真的可執行。

$$\operatorname{conv}(S) = \left\{ \sum_{i=1}^{r} \theta_i v_i : r \text{ finite}, \; v_i \in S, \; \theta_i \geq 0, \; \sum_i \theta_i = 1 \right\}.$$

S 是原集合，r 是有限組合中的點數；$\operatorname{conv}(S)$ 是包含 S 的最小凸集合。

若必須在三個配置中選一個，凸包也容許混合。若混合代表時間分配，可能是實際可操作的選擇；若必須買一台不可分割的機器，分數配置只是鬆弛，不是原問題可行答案。

最小化問題擴大可行集合，不會提高最佳值，所以凸鬆弛可給離散問題下界。把鬆弛解變成真實離散選擇，還需額外步驟；捨入可能違反限制或傷害目標。

不可把鬆弛問題的精確最佳解說成原問題的精確最佳解。原可行集、鬆弛集合、下界與實際達成目標應分開。應用章會再討論。

常用凸集合詞彙

白話版。多數有用模型由幾種可重用幾何形狀組成。學會辨認後，陌生方程式也比較容易理解。

超平面、半空間與多面體

$$H = \{x : a^\top x = b\}, \qquad P = \{x : a^\top x \leq b\}.$$

$x \in \mathbb{R}^n$ 是點，$a \neq 0$ 是法向量，b 是純量偏移。超平面是仿射；閉半空間包含邊界與其中一側。

多面體是有限個半空間與超平面的交集，常寫 $Ax \leq b$, $Ex = d$。A, E 放係數，b, d 放界限，不等式逐座標比較。多面體可有界、無界、低維或空。

盒狀集合只限制各座標上下界；機率單純形再加入非負與總和一。這說明機率向量與有界像素常適合凸限制。

範數球與橢球

$$B = \{x : \|x - c\| \leq r\}.$$

c 是中心，$r \geq 0$ 是半徑，$\|\cdot\|$ 是範數。三角不等式給出凸性。二範數球是圓形，無限範數球是盒，一範數球帶有角點。

$$\mathcal{E} = \{c + Au : \|u\|_2 \le 1\}.$$

A 把單位二範數球映射成橢球，u 是輔助座標，c 是中心。A 可逆時可用正定形狀矩陣寫二次描述；A 奇異時得到低維影像，仍然凸。

錐與矩陣不等式

凸錐對非負組合封閉：

$$\alpha u + \beta v \in K \quad \text{for } u, v \in K,\ \alpha, \beta \ge 0.$$

K 是錐，u, v 是其中的點，α, β 是非負純量；權重不需要總和一。非負 orthant 是例子。

二階錐包含 $\|z\|_2 \le t$ 的 (t, z)，其中 t 是純量，z 是向量，可表達許多範數限制。

半正定錐包含滿足下式的對稱矩陣：

$$v^\top X v \ge 0 \quad \text{for every } v \in \mathbb{R}^n.$$

$X \in \mathbb{R}^{n \times n}$ 對稱，v 是任意測試向量。記為 $X \succeq 0$，不等於每個矩陣元素非負。共變異數矩陣自然落在此錐中。

廣義不等式用錐定義順序：$x \preceq_K y$ 表示 $y - x \in K$。Orthant 對應逐座標順序，對稱矩陣則對應半正定順序，必須說明使用哪個錐。

保留凸性的運算

白話版。已知一個集合凸後，可以透過特定運算得到另一個凸集合。這些規則比一直畫圖更可靠。

凸集合的交集仍凸：兩端點的線段會同時留在每個集合內。因此加入更多凸限制很直接。

仿射映射與仿射逆像保留凸性。C 凸時，$\{Ax + b : x \in C\}$ 與 $\{x : Ax + b \in C\}$ 都凸；A, b 是固定矩陣與向量。一般非線性轉換沒有此保證。

Cartesian product 與集合和保留凸性。取部分座標的投影是仿射映射，因此消去輔助變數後仍凸。這是 extended formulation 的基礎：增加變數反而可能更容易描述。

聯集通常不凸，兩個分離的球就示範失敗。限制敘述中的「且」改成「或」，即使每個分支都凸，也可能根本改變問題。

矩陣變數也一樣。X, Y 半正定時，$0 \le \theta \le 1$ 下的 $\theta X + (1 - \theta) Y$ 半正定，對每個向量檢查二次型即可。

兩個可重用的短證明

凸性證明通常追蹤一條線段如何通過集合構造。假設 $u, v \in C_1 \cap C_2$，且 C_1, C_2 都凸。對 $0 \le \theta \le 1$，混合點 $\theta u + (1 - \theta) v$ 同時屬於兩集合，因此屬於交集。任意多個凸集合的交集也可用相同推理。

對仿射映射，令 $y_1 = Au + b$、$y_2 = Av + b$，其中 $u, v \in C$。兩映射點的混合為

$$\theta y_1 + (1-\theta) y_2 = A[\theta u + (1-\theta) v] + b.$$

其中 C 是凸集合、A, b 是固定轉換資料、y_1, y_2 是映射後的點。中括號內的點屬於 C，所以混合點也在映射集合內。這就是消去輔助座標仍保留凸可行集合的原因 [1，第 2.3 節]。

聯集的對應證明會失敗，因為兩端點可能來自不同集合。例如 $-1 \in [-2,-1]$、$1 \in [1,2]$，但中點零不屬於任何一個區間。兩區間各自凸，聯集卻不凸。對建模而言，同時滿足凸規則保留凸性；二選一規則則可能破壞它。

投影與分離超平面

白話版。投影找離任意點最近的允許點。從投影指向原點的方向，也能描述可行集合位於邊界哪一側。

非空閉凸集合 C 的二範數投影為

$$\Pi_C(y) = \arg\min_{x \in C} \frac{1}{2} \|x - y\|_2^2.$$

y 是查詢點，x 是可行候選，$\Pi_C(y)$ 是唯一最小點。平方二範數的嚴格凸性確保唯一；其他距離或非凸集合未必如此。

令 $z = \Pi_C(y)$：

$$(y - z)^\top (x - z) \le 0 \quad \text{for every } x \in C.$$

z 是投影，x 遍歷所有可行點。從 z 朝任意可行 x 移動一小步，平方距離不能下降，就得到此條件。

若 $y \notin C$，$y - z \neq 0$，它定義通過 z 的超平面，使可行集合與 y 分處兩側，給出具體分離構造。

更一般的分離定理比較不相交凸集合 [1，第 2.5 節]。沒有附加假設時，分離未必有正 margin；閉性、緊緻性與集合距離都影響嚴格分離。

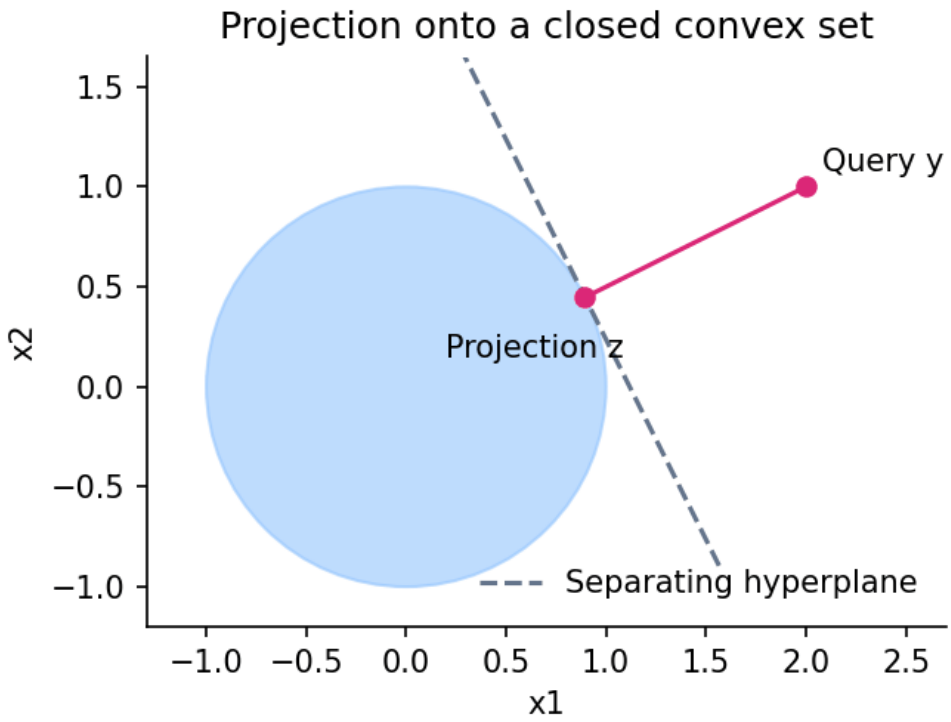


Figure 2 — 原創解析示意：外部點投影至單位圓盤，投影方向提供分離超平面的法向量。

程式碼：盒與球的投影

盒投影逐座標裁切；球投影只對外部點縮放。[1，第 8.1 節] 討論這些幾何應用。

```
import numpy as np

y = np.array([2.0, -1.5])
lower, upper = np.array([0.0, -1.0]), np.array([1.0, 1.0])
box_projection = np.clip(y, lower, upper)
radius = 1.0
ball_projection = y * min(1.0, radius / np.linalg.norm(y))
rng = np.random.default_rng(2)
points = rng.normal(size=(1000, 2))
points /= np.maximum(1.0, np.linalg.norm(points, axis=1))[:, None]
inequality = (points - ball_projection) @ (y - ball_projection)
assert inequality.max() <= 1e-12
assert np.all(box_projection >= lower) and np.all(box_projection <= upper)
print("box projection:", box_projection)
print("ball projection:", ball_projection)
```

取樣只展示投影不等式，不能替代所有可行點的證明。複雜集合的投影本身可能需要最佳化，所以「投影梯度」不代表投影便宜。

推導投影檢查條件

投影不等式來自檢查每個可行方向，而不是畫一條看起來合理的切線。令 $z = \Pi_C(y)$，任取 $x \in C$。凸性使 $z + \theta(x - z)$ 在 $0 \le \theta \le 1$ 時可行。因為 z 最小化平方距離，其單側方向導數必須非負：

$$\frac{d}{d\theta}\frac{1}{2}\|z + \theta(x - z) - y\|_2^2\Big|_{\theta=0} = (z - y)^\top (x - z) \ge 0.$$

其中 y 是查詢點、z 是其投影、x 是可行比較點、θ 是線段參數。這便得到前面的分離不等式。它也充分：把 x 處距離展開，就得到 z 處距離，加上一個非負內積與 $\frac{1}{2}\|x - z\|_2^2$。

唯一性也很直接。若不同的 z_1, z_2 都最小化平方距離，可行中點的值會嚴格更小。封閉性則確保存在：最小化序列留在有界的距離次水平集合內，因此有收斂子序列，而極限仍在 C。非空、封閉與凸性各自扮演明確角色 [1]。

內部、相對內部與隱藏等式

白話版。集合可能在自己的平面內有活動空間，卻在完整周圍空間沒有內部。

機率向量座標總和一，位於超平面，完整向量空間中的內部是空。但座標嚴格正的點，仍可位於此超平面中的相對內部。

仿射包是包含 C 的最小仿射集合；相對內部是在仿射包中量測的內部。這與可行性條件和對偶有關，不能要求完整維度內部而排除有用的等式模型。

不等式也可能隱藏等式：同時要求 $a^\top x \le b$ 與 $a^\top x \ge b$，就強制等號。辨認它能改善解釋與數值處理。

嚴格可行不是「算出很小違反」。它要求相關不等式嚴格成立，同時滿足等式。對偶與內點章會說明何時需要此條件。

凸問題需要集合與函數都合適

白話版。允許選擇是凸，只完成一半；目標形狀與等式也必須符合條件。

凸最小化使用凸目標、凸不等式函數與仿射等式 [1]。下一章定義凸函數並推導辨認規則。

凸可行集合本身不夠：在盒上最小化有多個谷地的函數仍非凸。凸目標加不連通離散限制，也非凸。

整體問題凸時，每個局部最小點都是全域最小點。但解未必唯一，線性目標可能在整個面上相同。可行集合上的嚴格凸性，可在存在最小值時保證唯一。

全域性與唯一性不保證所有尺度都容易。巨大密集問題可能超過記憶體，病態問題的緊容忍值可能難求。凸性提供結構優勢，但計算量與實作仍重要。

為什麼局部最小解必須是全域最小解

凸性把較好的遠處答案連到目前點，排除局部陷阱。假設 x 是凸可行集合 C 上凸函數 f 的局部最小解，但某個可行 y 有 $f(y) < f(x)$。對任意 $0 < \theta \le 1$，定義 $z_\theta = (1-\theta)x + \theta y$，則

$$f(z_\theta) \le (1-\theta)f(x) + \theta f(y) < f(x).$$

其中 z_θ 是可行線段點、θ 是混合權重。任意小的正 θ 都能讓該點任意靠近 x，而且改善目標，與局部最小性矛盾。證明不需要可微性 [1]。

它給全域性，卻不給唯一性。例如在 $0 \le x_1, x_2 \le 1$ 上最小化 x_1，每個 $(0, x_2)$ 都是全域最小解。嚴格凸目標會排除不同的同值最小解，因為可行中點的成本會更低。存在性仍獨立：在 $x > 0$ 下最小化 x 是凸問題，卻沒有最小解。

這三個問題決定對求解器的期待。凸性使局部最小解全域；嚴格曲率可選出唯一答案；封閉性與目標成長可確保答案存在。數值程序還需要適當收斂論證，不能只靠以上三點。

現代觀點

白話版。現代凸最佳化仍從建模與幾何開始；新方法改變如何存取可行集合，以及花多少成本取得準確證書。

Grant、Boyd 與 Ye 的 disciplined convex programming [2] 以已驗證的組合規則組織模型。這是建模方法，不保證任意方便公式都凸。代數上凸的式子，也可能需要改寫，讓規則系統看得出凸性。

Pokutta 的 2024 Frank–Wolfe 綜述 [3] 區分投影、barrier 與線性最小化 oracle 等集合存取方式。容易描述的集合，投影仍可能很貴；線性最小化有時較便宜，凸組合則維持可行。後續章節會推導更新與條件。

Applegate 等人的 PDLP 報告 [4] 於 2025 首次公開、2026 修訂，研究實用的一階線性規劃求解器，連結原始—對偶模型、尺度調整、restart 與可行性修整。這是原創求解器研究，不是綜述；它說明即使熟悉的凸問題，實作與需求精度仍重要。

目前挑戰包含以不過度膨脹的輔助變數表達複雜限制、利用稀疏矩陣、讓尺度下的容忍值有意義，以及把求解器嵌入較大流程時維持證書。學習式特徵可能非凸，後續分配問題卻凸，各階段保證應分開描述。

這是經挑選的閱讀路線，不宣稱檢閱所有最佳化綜述。核心理論依 [1]，現代觀點明確指出每個來源支持哪種延伸。

重點整理

- 變數、固定資料、目標、限制與單位必須明確。
- 可行、有界、達到最佳值與數值停止，是不同問題。
- 凸集合包含所有非負且權重總和一的平均。
- 仿射集合、多面體、範數球與錐是可重用積木。
- 交集與仿射轉換保留凸性，聯集通常不會。

- 閉凸集合投影唯一，並提供幾何證書。
- 凸最佳化保證針對建立的問題及其假設。

練習

1. 建立三種非負資源的預算配置模型，以平方距離接近固定目標，說明單位、變數與資料。

   提示
   向量 x、非負、總和限制與固定目標。

2. 直接證明兩個凸集合的交集凸，並提出聯集反例。

   提示
   線段留在兩者中；分離區間否證聯集。

3. 三種配置擇一時，何時凸混合能實際操作，何時只是鬆弛？

   提示
   時間分配與不可分硬體。

4. 推導中心在原點的二範數球投影，說明內部點的情況。

   提示
   外部點沿徑向落在邊界。

5. 提出最佳值有限卻未達到的凸問題，說明缺少哪個存在條件。

   提示
   開區間排除極限點。

6. 限制殘差為 10^{-6}，為何沒有尺度與應用單位就不能判斷是否足夠？

   提示
   同一不等式縮放後，原始殘差會改變。

參考文獻

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004。**閱讀：第 1–2 章；第 4.1–4.2 與 8.1 節補充問題術語及投影**。官方書籍頁、官方免費 PDF。
2. Michael Grant, Stephen Boyd and Yinyu Ye, “Disciplined Convex Programming,” in *Global Optimization: From Theory to Implementation*, Springer, 2006。作者出版頁。
3. Sebastian Pokutta, “The Frank-Wolfe Algorithm: A Short Introduction,” *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 126, 2024。開放綜述。
4. David Applegate, Mateo Díaz, Oliver Hinder, Haihao Lu, Miles Lubin, Brendan O’Donoghue and Warren Schudy, “PDLP: A Practical First-Order Method for Large-Scale Linear Programming,” arXiv preprint, 2025，2026 修訂。已驗證論文記錄。

OPT 02・凸函數、組合規則與共軛

以弦線、支撐平面與曲率辨認凸目標，建立有效組合，理解 perspective、共軛與擬凸性。

網站版本: https://blog.shyandram.dev/zh-tw/learn/optimization/opt-02-convex-functions/

你將學到

- 用定義域、弦線與上圖集合定義凸性。
- 在不假設處處可微時使用一階與二階條件。
- 辨認重要純量、範數與矩陣函數。
- 以加總、仿射代入、最大值、組合與 perspective 建模。
- 推導共軛，連結線性下界。
- 區分凸、強凸、擬凸與對數凹結構。

先看全貌

上一章描述允許的選擇，本章描述分數如何改變。凸目標有一種一致性：混合兩個選擇後，分數不高於兩端分數的同樣混合；局部支撐平面提供全域下界。

但熟悉的損失不一定對每個變數都凸。平方誤差對預測值與線性係數凸，對多層神經網路的全部參數通常不凸。必須說明正在最佳化的變數與定義域。

> **用最簡單的話說**
> 想像一個碗，用直棒連接碗上兩點，碗保持在棒下方。一塊從下方碰到碗的小平板，也一直在碗下方。這就是凸弦線與支撐平面的圖像。

Boyd 與 Vandenberghe 第 3 章 [1] 提供多種辨認方式。目標不是背大量公式，而是用少量基本函數與有效轉換規則，讓複雜目標有可檢查的凸性理由。

凸性包括定義域。某公式在所有可微處曲率都正，若完整定義域不連通，仍不是該集合上的凸函數。定義域需要和導數同樣仔細檢查。

弦線、定義域與上圖集合

白話版。連接兩個函數值的直線，在凸函數的所有中間輸入，都不低於真正函數值。

凸定義域 C 上的 $f : C \to \mathbb{R}$：

$$f(\theta x + (1-\theta)y) \le \theta f(x) + (1-\theta)f(y), \qquad 0 \le \theta \le 1.$$

$x, y \in C$ 是任意輸入，θ 是混合權重，所有選擇都成立時 f 為凸。反向不等式定義凹性；仿射函數等號成立，同時凸與凹。

此條件同時比較輸入與輸出。定義域凸，確保混合輸入可用；不能只對方便的部分端點驗證。

上圖集合（epigraph）：

$$\mathrm{epi} f = \{(x, t) : x \in C,\, f(x) \le t\}.$$

t 是純量高度，(x, t) 位於函數圖形上方或圖形上。f 凸，若且唯若其 epigraph 是凸集合，連結前章幾何與本章目標。

引入 t，最小化它且限制 $f(x) \le t$，就是 epigraph formulation。精確改寫不改變最佳值，也可把非平滑目標變成較容易表達的限制。

Jensen 不等式與期望

$$f\left(\sum_i \theta_i x_i\right) \le \sum_i \theta_i f(x_i), \qquad \theta_i \ge 0, \quad \sum_i \theta_i = 1.$$

x_i 是輸入，θ_i 是權重。隨機向量 X 若符合可積分與定義域條件，則 $f(\mathbb{E}[X]) \le \mathbb{E}[f(X)]$。

因此先平均再算凸成本，可能比個別成本平均小。但這不表示平均任意模型參數都改善預測，還需要預測損失對那些參數凸。

非線性轉換後的期望，通常不等於期望後的轉換。偏向哪一側取決於凸或凹，不是「平均一定有代表性」的直覺。

支撐平面：一階條件

白話版。可微凸函數的切線近似是全域下界，不只是附近的草圖。

開凸定義域上，可微凸性等價於

$$f(y) \ge f(x) + \nabla f(x)^\top (y - x).$$

x 是近似位置，y 是任意定義域點，$\nabla f(x)$ 是對最佳化變數的梯度。右側仿射式從下方支撐函數。

對 $x + \theta(y - x)$ 使用弦線條件，整理、除以正的 θ，再令其趨近零，可得到此方向導數界。

若 $\nabla f(x) = 0$，不等式給所有 y 都有 $f(y) \ge f(x)$，因此無限制可微凸問題的駐點全域最佳，但未必唯一。

有限制時，非零梯度也可能最佳，因為改善方向被禁止。對偶與受限演算法章會明確表示此事；小梯度不是每種受限問題的合適停止規則。

非平滑支撐向量

角點不破壞凸性。次梯度（subgradient）g 在 x 滿足

$$f(y) \geq f(x) + g^{\top}(y - x) \quad \text{for every } y.$$

g 是提供全域仿射下界的向量，所有這類向量形成次微分 $\partial f(x)$。可微點只有梯度；絕對值在零的次梯度為 $[-1, 1]$。

絕對殘差、最大值與稀疏懲罰會用到此概念。次梯度不是在角點任選導數，必須滿足全域支撐條件。後面會連到 proximal 方法。

無限制凸函數的 $0 \in \partial f(x)$ 保證全域最佳。零梯度與零屬於一組次梯度，都是同一支撐觀念。

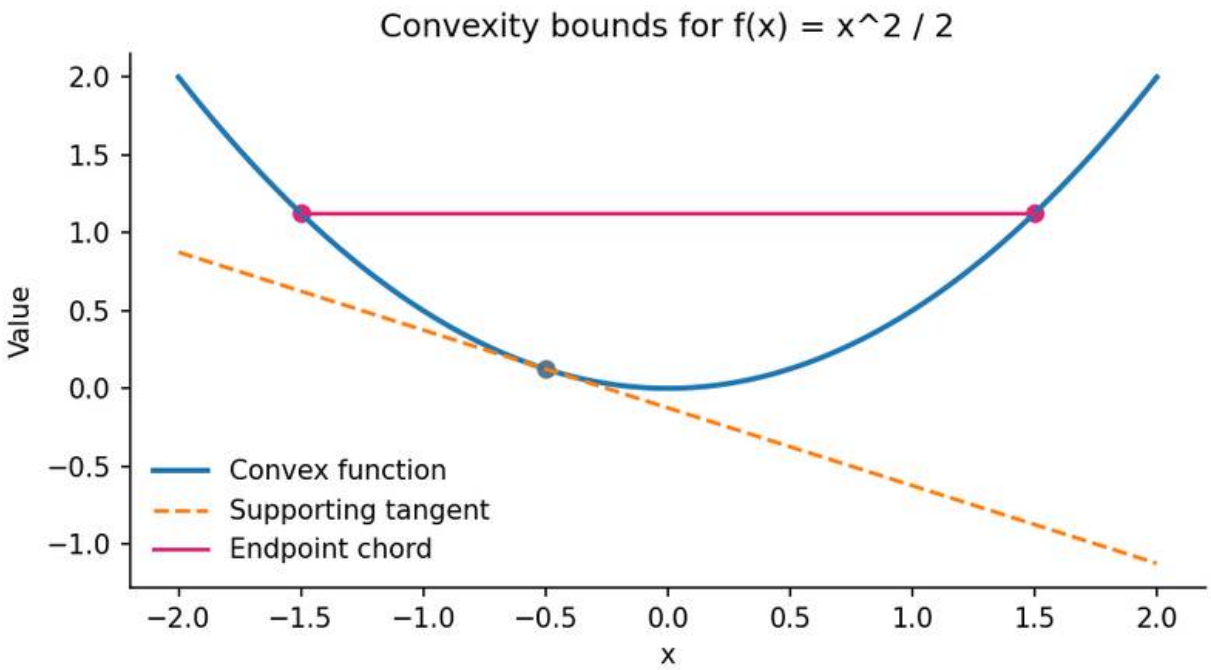


Figure 1 — 原創解析示意。凸函數位於端點弦線下方，指定位置的切線則提供全域仿射下界。

曲率：二階條件

白話版。平滑函數必須在所有方向都向上彎，才是凸；只看各座標可能漏掉向下彎的斜向方向。

開凸定義域上，二次連續可微 f 的凸性等價於

$$\nabla^2 f(x) \succeq 0 \quad \text{for every } x.$$

$\nabla^2 f(x)$ 是對稱 Hessian，半正定表示每個 $v^{\top}\nabla^2 f(x) v$ 都非負。

只有對角元素非負不夠。$f(x_1, x_2) = x_1 x_2$ 的座標二階導數為零，但 Hessian 不定，兩條對角方向分別向上與向下彎。

這是完整定義域上的全域條件，抽查十個位置的特徵值只是診斷。解析 Hessian 的半正定分解才可對所有位置建立證明。

嚴格凸與強凸

嚴格凸要求不同端點與嚴格介於零一的權重下，弦線不等式嚴格成立。凸集合上最小值存在時，嚴格凸可保證唯一。

二範數下的強凸要求某 $\mu > 0$：

$$f(y) \geq f(x) + \nabla f(x)^\top (y - x) + \frac{\mu}{2}\|y - x\|_2^2.$$

μ 是統一曲率下界。二次可微時對應 $\nabla^2 f(x) \succeq \mu I$，$I$ 為單位矩陣。強凸比嚴格凸更強；嚴格凸未必有正的統一下界。

正二次正則化常在受懲罰座標提供強凸。未懲罰截距或限制定義域仍須分別分析，不能由一個正則化項推論全部參數強凸。

Hessian 條件實際量測什麼

多變數曲率測試可以化成每條直線上的純量測試。固定點 x 與方向 v，在仍屬於定義域的區間令 $q(t) = f(x + tv)$。微分兩次得到

$$q''(t) = v^\top \nabla^2 f(x + tv) v.$$

其中 t 是直線座標、q 是限制在直線上的純量函數、v 是任意方向。半正定 Hessian 讓每個直線限制都凸。每條線段都屬於某條直線，因此可推出弦線定義。反過來，每條直線都凸，就要求其二階導數非負。這說明測試為何檢查所有方向的二次型，而不是個別矩陣元素 [1]。

嚴格凸與強凸可以用具體例子區分。純量函數 $f(x) = x^4$ 在 $\mathbb{R}$ 上嚴格凸，因為導數 $4x^3$ 嚴格遞增。但 $f''(0) = 0$，所以不存在正的全域 μ 滿足強凸 Hessian 下界。其最小解唯一，卻不能使用 OPT 06 的強凸停止上界。加入 $\frac{\mu}{2}x^2$，其中選定 $\mu > 0$，可補足曲率下界，同時也改變目標。

有用函數的基本目錄

白話版。少數基本函數涵蓋很多實務目標，使用方式取決於其定義域與正負號。

仿射 $a^\top x + b$ 同時凸與凹。範數由齊次性與三角不等式建立凸性；平方二範數平滑，二範數本身在原點不平滑。

$$f(x) = \frac{1}{2} x^\top Q x + c^\top x + d.$$

Q 是對稱曲率矩陣，c 是線性係數向量，d 是純量。此函數凸若且唯若 $Q \succeq 0$；正定 Q 的最小特徵值提供強凸界。

指數處處凸，負對數在正輸入凸。冪次必須說清楚指數與定義域，「所有冪次都凸」是錯的；平方根在非負輸入凹，其負值則凸。

最大座標或仿射分數的最大值凸且通常非平滑；仿射函數的最小值凹，不普遍凸。正負號不能忽略。

矩陣定義域

對正定對稱 X，$-\log\det X$ 是凸函數 [1，第 3.1 節]。變數是矩陣，定義域排除奇異矩陣。

對稱擾動 H 的二階方向導數：

$$\operatorname{tr}(X^{-1}HX^{-1}H) \geq 0.$$

H 是對稱矩陣方向，tr 為 trace。Congruence 轉換後成為平方 Frobenius 範數，說明曲率非負。

對稱矩陣最大特徵值凸：

$$\lambda_{\max}(X) = \sup_{\|v\|_2=1} v^{\top}Xv.$$

$\lambda_{\max}$ 表示最大特徵值，v 遍歷單位向量。固定 v 時二次型對 X 元素仿射，逐點上確界因此凸，展示「誰是變數」的重要性。

程式碼：二次函數與反例的 Hessian

以下 Hessian 固定，因此特徵值檢查適用所有輸入，並驗證一次支撐平面不等式。

```
import numpy as np

Q = np.array([[3.0, 1.0], [1.0, 2.0]])
bad = np.array([[0.0, 1.0], [1.0, 0.0]])
assert np.linalg.eigvalsh(Q).min() > 0
assert np.linalg.eigvalsh(bad).min() < 0
x, y = np.array([0.4, -0.6]), np.array([-1.0, 0.8])
f = lambda z: 0.5 * z @ Q @ z
support = f(x) + (Q @ x) @ (y - x)
gap = f(y) - support
assert np.isclose(gap, 0.5 * (y - x) @ Q @ (y - x))
print("quadratic Hessian eigenvalues:", np.linalg.eigvalsh(Q))
print("product Hessian eigenvalues:", np.linalg.eigvalsh(bad))
print("supporting-plane gap:", gap)
```

加總、代入與逐點最大值

白話版。以保留凸性的運算組合已知函數，運算規則就是證明。

凸函數的非負加權和凸，負係數通常翻轉或破壞性質。固定、與決策變數無關的分布下，定義良好的積分或期望也保留凸性。

仿射代入：

$$g(x) = f(Ax + b).$$

f 凸，A, b 固定。g 定義域要求變換後落在 f 的定義域。此規則建立平方殘差、範數殘差與線性估計正則化的凸性。

f_i 凸時，共同定義域上的 $\max_i f_i(x)$ 凸。引入 t 並限制所有 $f_i(x) \le t$，可建模最壞殘差、最壞情境與最大座標。

共同凸的輔助變數部分最小化也保留凸性：

$$g(x) = \inf_y F(x, y).$$

F 必須對 (x, y) 共同凸，y 是被消去變數；分別對各區塊凸不夠。這連結 epigraph 的座標投影。

哪些運算不自動保留凸性

凸函數相乘、相除，通常不保證凸；取最小值可能形成多個谷地。

x^2y^2 不會因為是兩個非負凸式的乘積就共同凸。Reciprocal 或指數外層需要正確單調性的組合論證。

固定參數乘變數是仿射，兩個決策變數相乘是 bilinear，通常非凸。把待學或未知量誤當固定資料，是常見建模錯誤。

組合與單調性

白話版。外層凸函數若以適當方向回應內層參數，就可保留凸性。

$$F(x) = h(g_1(x), \dots, g_k(x)).$$

h 是外層，g_i 是純量內層。充分規則是：h 凸，各參數上若內層凸則非遞減，若內層凹則非遞增，或內層為仿射。還須納入定義域與 extended-value 條件 [1，第 3.2 節]。

指數凸且遞增，所以指數作用在凸內層保留凸。負對數凸且遞減，作用在正的凹內層得到凸。

平方不能隨意作用在凸式：負輸入時它遞減，正輸入時遞增。非負凸內層平方是有效組合，任意正負則需要另證。

這些是充分辨認規則，不是判定所有代數式的完整程序。例如 $\sqrt{1 + x^2}$ 改寫成二範數，就能直接看出凸性。

從導數看組合規則

對純量平滑函數，鏈鎖律能看清單調性要求從哪裡出現。令 $F(x) = h(g(x))$，則

$$F''(x) = h''(g(x))[g'(x)]^2 + h'(g(x))g''(x).$$

其中撇號表示純量導數、g 是內層函數、h 是外層函數。若 h 凸且非遞減、g 凸，兩項都非負。若 h 凸且非遞增、g 凹，第二項仍非負。組合規則也適用非平滑函數，但這個計算揭露了正負號來源 [1]。

取凸內層式 $g(x) = x^2 - 1$ 與凸外層函數 $h(t) = t^2$。組合為 $(x^2 - 1)^2$，二階導數為 $12x^2 - 4$，在零點為負，所以組合不凸。內層輸出在零附近為負，平方函數在該區域遞減，因此所需符號配合失敗。

若定義域為凸，且凸內層式在整個定義域非負，平方就合法，因為外層在該區域非遞減。因此平方歐氏殘差範數凸，但「任意凸損失式都可再平方」不是一般有效規則。

Log-sum-exp 與穩定計算

白話版。 Log-sum-exp 平滑地從上方近似最大分數，也產生正規化權重。

$$f(x) = \log\left(\sum_{i=1}^{n} e^{x_i}\right), \qquad p_i = \frac{e^{x_i}}{\sum_j e^{x_j}}.$$

x_i 為純量座標，p_i 是正且總和一的權重。梯度為 p，Hessian 為

$$\nabla^2 f(x) = \operatorname{diag}(p) - pp^\top.$$

$\operatorname{diag}(p)$ 把權重放在對角。任意方向 v 的二次型是 v_i 的加權變異數，因此非負，證明凸。

所有座標加相同常數，函數也只平移該常數。因此全一方向曲率零，在完整空間不強凸。

溫度 $\tau > 0$ 下，$f_\tau(x) = \tau \log \sum_i e^{x_i/\tau}$：

$$\max_i x_i \le f_\tau(x) \le \max_i x_i + \tau \log n.$$

τ 控制平滑與近似差；越小越接近最大值，但可能使數值最佳化更病態。先扣除最大縮放座標，再做指數，能穩定計算。

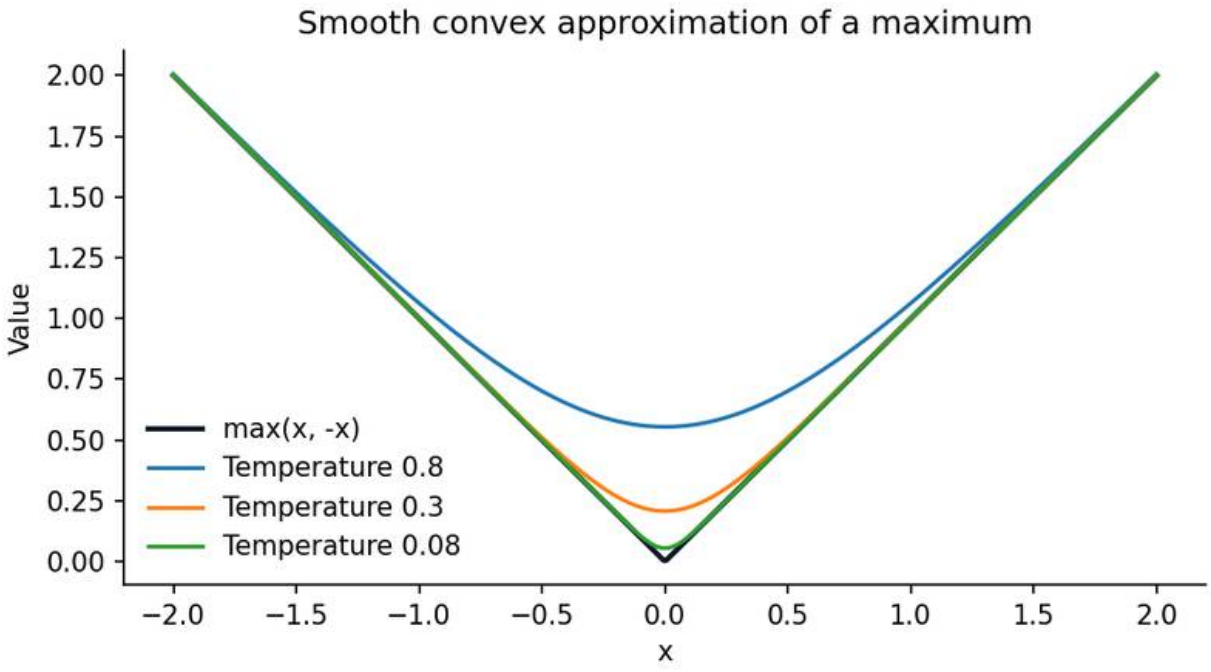


Figure 2 — 原創解析示意。溫度縮放 *log-sum-exp* 位於最大分數上方；較小溫度更接近，但轉折更尖。

程式碼：穩定計算與 Jensen 檢查

完整範例避免大輸入溢位，驗證 Hessian 並展示一次有限 Jensen 比較。

```
import numpy as np
from scipy.special import logsumexp

x = np.array([1000.0, 999.0, 997.0])
value = logsumexp(x)
p = np.exp(x - value)
H = np.diag(p) - np.outer(p, p)
assert np.linalg.eigvalsh(H).min() >= -1e-12
assert np.isclose(p.sum(), 1)
u, v = np.array([1., -1., 0.]), np.array([-2., 2., 1.])
theta = 0.3
left = logsumexp(theta * u + (1 - theta) * v)
right = theta * logsumexp(u) + (1 - theta) * logsumexp(v)
assert left <= right + 1e-12
print("stable value:", value, "weights:", p)
print("Jensen gap:", right - left)
```

Perspective：正規化變數並保留凸性

白話版。 Perspective 把正規化輸入的成本，和使用的尺度相乘，產生看似比值的有用凸式。

$$g(x,t) = tf(x/t), \qquad t > 0, \quad x/t \in \operatorname{dom} f.$$

f 凸，x 是向量，t 是正尺度，g 對 (x,t) 共同凸。正尺度定義域不可漏。

$f(u) = \|u\|_2^2$ 給 $g(x,t) = \|x\|_2^2/t$，稱 quadratic-over-linear。任意相除不保證凸，但此特定結構凸。

混合 (x_1,t_1) 與 (x_2,t_2) 後，正規化輸入的權重與 θt_1、$(1-\theta)t_2$ 成比例。對 f 使用凸性，再乘混合尺度，即可證明。

此形式用於正規化速率、分配時間或正質量，也可揭露精確錐表示。未仔細定義閉延伸前，不能任意允許零或負尺度。

Perspective 證明：正規化後權重會改變

普通平均與除以變動尺度不能直接交換。Perspective 仍然凸，是因為正規化產生另一組仍合法的權重。

取兩個定義域點 $(x_1,t_1),(x_2,t_2)$，其中 $t_1,t_2>0$，混合權重為 $\theta \in [0,1]$。定義

$$\bar{x} = \theta x_1 + (1-\theta)x_2, \qquad \bar{t} = \theta t_1 + (1-\theta)t_2,$$
$$\beta = \frac{\theta t_1}{\bar{t}}, \qquad \frac{\bar{x}}{\bar{t}} = \beta\frac{x_1}{t_1} + (1-\beta)\frac{x_2}{t_2}.$$

其中 $\bar{x}, \bar{t}$ 是混合後的分子與尺度；$\beta \in [0,1]$ 是正規化權重。對最後一行套用 f 的凸性，再乘上 $\bar{t} > 0$。係數變成 $\bar{t}\beta = \theta t_1$ 與 $\bar{t}(1-\beta) = (1-\theta)t_2$，恰好得到 $g(x,t) = tf(x/t)$ 的凸弦線界 [1]。

對二次除線性例子，取 $(x_1, t_1) = (2,1)$、$(x_2, t_2) = (2,3)$，且 $\theta = 1/2$。兩成本為 4 與 4/3，混合點 $(2,2)$ 的成本為 2，低於平均 8/3。其正規化輸入為 1：第一個正規化輸入 2 的權重為 $\beta = 1/4$，第二個 2/3 的權重為 3/4。依尺度調整權重是關鍵；直接平均正規化輸入會得到不同結果。

共軛與線性下界

白話版。共軛問某個線性分數最多能超過函數多少，答案決定該斜率下支撐線的最佳截距。

$$f^*(y) = \sup_{x \in \operatorname{dom} f} \{y^\top x - f(x)\}.$$

y 是斜率向量，x 是原變數，$f^*(y)$ 是差值上確界。有限值給下界：

$$f(x) \geq y^\top x - f^*(y).$$

這直接來自定義，是 Fenchel 不等式 [1，第 3.3 節]，用於對偶與 splitting 演算法。

$f(x) = \frac{1}{2} x^\top Q x$ 且 Q 正定時，最大差的點滿足 $Qx = y$：

$$f^*(y) = \frac{1}{2} y^\top Q^{-1} y.$$

Q 是曲率矩陣。程式用求解 $Qx = y$，不用明算反矩陣。

集合 indicator 的共軛是 support function。Extended-value indicator 在允許集合為零、其他地方正無限，可把限制併入目標，但不是有限懲罰。

Proper closed convex 函數取兩次共軛可回到原函數。缺少條件時，double conjugate 對應適當的閉凸包絡，不能任意假設重現原公式。

程式碼：共軛證書

二次 Fenchel 界在 $y = Qx$ 等號成立；範例檢查等號與其他斜率的非負 gap。

```
import numpy as np

Q = np.array([[4., 1.], [1., 2.]])
x = np.array([0.5, -0.8])
y = np.array([1.2, 0.3])
f = lambda z: 0.5 * z @ Q @ z
conjugate = lambda slope: 0.5 * slope @ np.linalg.solve(Q, slope)
gap = f(x) + conjugate(y) - x @ y
matching = Q @ x
assert gap >= -1e-12
assert np.isclose(f(x) + conjugate(matching), x @ matching)
print("Fenchel gap:", gap)
print("matching slope:", matching)
```

具有明確定義域的非平滑共軛

共軛能把非平滑懲罰轉成簡單的斜率限制。令 $f(x) = \lambda\|x\|_1$，其中固定 $\lambda > 0$。在定義中分開各座標，得到

$$f^*(y) = \sum_i \sup_{x_i \in \mathbb{R}} (y_i x_i - \lambda|x_i|) = \begin{cases} 0, & \|y\|_\infty \le \lambda, \\ +\infty, & \|y\|_\infty > \lambda. \end{cases}$$

其中 x_i, y_i 是原變數與斜率的座標；$\|y\|_\infty = \max_i |y_i|$。當 $|y_i| \le \lambda$，每個座標的式子至多為零，在 $x_i = 0$ 達到。只要某個斜率量值超過門檻，讓 x_i 同號並增加量值，就能使式子無界。

因此共軛的定義域是無限範數球。對應下界 $y^\top x \le \lambda\|x\|_1$ 在非零座標具有匹配斜率 $y_i = \lambda\,\mathrm{sign}(x_i)$ 時取等號。一個計算便連起範數幾何、次梯度與對偶可行性 [1，第 3.3 節]。OPT 08 會用同樣的門檻結構推導近端更新。

擬凸性與對數結構

白話版。有些函數不凸，但低於任何門檻的輸入集合都凸；另一些函數取對數後凸或凹。各自有不同保證。

擬凸（quasiconvex）要求所有下水平集合凸：

$$C_\alpha = \{x : f(x) \le \alpha\}.$$

α 是任意實數門檻，C_α 是其下水平集合。等價地，混合值不超過兩端較大值，比弦線條件弱。

凸函數都擬凸。區間上的單調純量函數，即使曲率非向上，也擬凸。適當形式下，可對門檻二分，每步解凸可行問題 [1，第 3.4、4.2 節]。

正函數 f 的 log-convex 表示 $\log f$ 凸，log-concave 表示 $\log f$ 凹。正性與定義域都是定義的一部分。密度的 log density 凹，不代表所有參數化下的估計問題都凸。

某些 geometric program 可對正變數取對數，精確轉成凸問題。這只適用指定類型，不是對任意目標和限制取對數的許可。下一章會推導。

公式的凸性與實作的凸性

數學凸表示式仍可能算得不好。指數溢位、矩陣邊界附近捨入，或意外陣列廣播，都可能使算出的量無效，原本函數卻依然凸。

應把符號曲率論證與數值評估檢查，作為不同證據。少數點的 Hessian 取樣可能找到反例，卻不能證明處處非負曲率。穩定實作也不會證明指定公式凸。

求解時參數固定，凸性宣稱針對的是該參數值下的決策變數。係數改變符號可能使組合規則失效。應記錄哪些參數範圍保留保證，不要無條件稱整個參數化族為凸。

學得函數也一樣。對後來當成決策變數的輸入為凸，不表示對訓練中的權重為凸。曲率規則一次描述一個最佳化問題。

現代觀點

白話版。資料學習函數時，仍可限制它對決策輸入凸，但訓練參數本身未必凸。

Disciplined convex programming [2] 將組合規則形式化，區分仿射、凸與凹，追蹤單調性與定義域。語法規則失敗可能只需要改寫，不一定否證數學凸性。

Schaller、Bemporad 與 Boyd 的 2025 稿件 [3] 擬合參數化凸函數，要求對有效參數值，函數在決策變數上仍凸且符合 disciplined programming。它連結資料學習與保持凸模型，是原創研究，不是綜述，也不保證任意神經損失。

Pokutta 的 2024 綜述 [4] 依幾何存取組織受限平滑方法，使用凸、平滑與強凸分析更新。全域凸性、統一下曲率與上平滑界有不同角色，應分別檢查。

開放問題包含結構函數類別的表達能力、雜訊參數下的定義域證明、不可微解映射，以及學習模型的泛化。最佳化證書證明擬合目標的性質，不證明它符合未來觀測。

重點整理

- 凸性同時包含凸定義域與弦線不等式。
- 切線與次梯度提供全域下界。
- 半正定 Hessian 建立所有方向的平滑凸性。
- 嚴格凸、強凸與平滑是不同假設。
- 組合規則需要單調性、正負號與定義域。
- Perspective 與共軛揭露比值和限制的結構。
- 擬凸與對數凸形式需要自己的推理。

練習

1. 從可微函數的弦線定義推導支撐平面不等式。

   提示

   整理弦線、除權重，再令其趨零。

2. 用二變數例子說明 Hessian 對角非負不保證凸。

   提示

   用 $x_1 x_2$ 的對角方向。

3. 把 $\sqrt{1 + x^2}$ 改寫成可辨認凸性的範數。

   提示

   向量 $(1, x)$ 的二範數。

4. 推導平方二範數的 perspective，說明定義域。

提示

代入 x/t 再乘正 t。

5. 推導 $\frac{1}{2}\|x\|_2^2$ 的共軛與 Fenchel 等號條件。

提示

微分二次差。

6. 提出區間上擬凸卻非凸的函數，驗證下水平集合。

提示

正區間上的平方根單調且凹，下水平集合仍為區間。

參考文獻

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004。**閱讀：第 3 章；第 4.2 節的擬凸可行性**。官方書籍頁、官方免費 PDF。
2. Michael Grant, Stephen Boyd and Yinyu Ye, “Disciplined Convex Programming,” in *Global Optimization: From Theory to Implementation*, Springer, 2006。作者出版頁。
3. Michael Schaller, Alberto Bemporad and Stephen Boyd, “Learning Parametric Convex Functions,” manuscript, June 2025。已驗證稿件頁。
4. Sebastian Pokutta, “The Frank-Wolfe Algorithm: A Short Introduction,” *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 126, 2024。開放綜述。

OPT 03・線性、二次、錐與幾何規劃

辨認標準凸問題，推導精確 epigraph 與錐改寫，轉換幾何規劃，檢查尺度與多目標權衡。

網站版本: https://blog.shyandram.dev/zh-tw/learn/optimization/opt-03-convex-programs/

你將學到

- 辨認線性、二次與凸二次限制問題。
- 精確改寫絕對值、最大值與範數限制。
- 以正確矩陣順序表達二階錐與半正定限制。
- 對正變數取對數，把幾何規劃轉成凸問題。
- 區分精確改寫、凸鬆弛與近似。
- 選求解器前，說明尺度、可行檢查與多目標權衡。

先看全貌

模型可能凸，寫法卻把凸性藏起來。絕對殘差和看似非平滑，加入輔助變數就成為線性規劃；範數界變成錐限制；正變數設計可在對數座標下凸。

Boyd 與 Vandenberghe 第 4 章 [1] 整理這些問題家族，提供可重用模式，也幫助選擇利用結構的數值方法。類型由方程式與定義域決定，不由應用名稱決定。

> **用最簡單的話說**
>
> 不同拼圖有不同積木盒。有時拼圖看起來陌生，只是積木轉了方向，改寫規則就能認出熟悉類型。但增加新積木會改變拼圖，所以要分清楚改寫與鬆弛。

本章用合成小問題展示精確數學關係與數值檢查，不是 benchmark。程式使用既有數值函式庫，不新增建模系統依賴。

凸標準形式

白話版。目標、不等式與等式必須一起符合條件，只看目標凸不夠。

$$\begin{aligned} \text{minimize}_x \quad & f_0(x) \\ \text{subject to} \quad & f_i(x) \le 0, \quad i = 1, \dots, m, \\ & Ax = b. \end{aligned}$$

$x \in \mathbb{R}^n$ 是決策向量；$f_0, \dots, f_m$ 在共同有效定義域凸；A, b 是固定等式係數與目標。凸下水平集合與仿射集合相交，得到凸可行集合。

凹函數可放另一側：g 凹時，$g(x) \geq a$ 等價於凸不等式 $a - g(x) \leq 0$。非線性凸函數等號通常不凸；$\|x\|_2^2 = 1$ 是球面，$\|x\|_2^2 \leq 1$ 才是凸球。

把等式拆成兩個不等式也不能修好，因為其中一側翻轉曲率。若物理問題真的要求球面，改成球就是鬆弛，可能改變答案。

精確改寫與改變問題

精確改寫在消去輔助變數或反轉一對一變換後，保留原決策與目標。鬆弛擴大允許集合，近似則修改函數或限制。

最小化 t 且 $f(x) \leq t$ 精確，因為目標偏好最小允許 t。但把相同不等式放進別的模型，若目標不偏好較小 t，未必達到等號。

增加變數時應證明兩方向：原可行點可延伸成新可行點，新點也映回原可行決策，且目標關係正確。

線性規劃

白話版。線性規劃的成本與規則都是直線，可行多面體具有角、面與可能延伸至無限的方向。

$$\begin{aligned} \text{minimize}_x \quad & c^\top x \\ \text{subject to} \quad & Gx \leq h, \qquad Ax = b. \end{aligned}$$

c 是固定成本向量，G, h 定義不等式，A, b 定義等式。座標界可另列或加入 G。

線性目標同時凸與凹，解未必唯一；若平行某可行面，整個面都可能最佳。

非負標準形式為 $Ax = b,\ x \geq 0$。加入非負 slack 將不等式轉成等式，自由純量可拆成兩個非負純量差。

這些改寫精確，但會增大問題與改變 conditioning。求解器若直接支援界與不等式，就不必為了形式手動展開。

絕對殘差變成線性限制

$$\min_x \|Ax - b\|_1.$$

A 是固定設計矩陣，x 是係數，b 是量測。每個殘差加入 t_i：

$$-t \leq Ax - b \leq t, \qquad \min_{x,t} \mathbf{1}^\top t.$$

t 限制殘差幅度，$\mathbf{1}$ 是全一向量。可行 t_i 不小於絕對殘差，目標加總使界緊。成對不等式已推得非負，也可明列。

Minimax 只加一個純量 t，限制 $-t\mathbf{1} \leq Ax - b \leq t\mathbf{1}$ 並最小化 t，控制最大絕對殘差，與總和代表不同偏好。

程式碼：小型配置 LP

SciPy 介面見 [2]。先檢查狀態，再獨立檢查可行性。

```
import numpy as np
from scipy.optimize import linprog

c = np.array([-3.0, -2.0])
G = np.array([[1.0, 1.0], [2.0, 1.0]])
h = np.array([4.0, 5.0])
result = linprog(c, A_ub=G, b_ub=h, bounds=[(0, None), (0, None)])
assert result.success, result.message
x = result.x
assert np.all(G @ x <= h + 1e-8)
assert np.all(x >= -1e-8)
assert np.allclose(x, [1, 3], atol=1e-7)
print("allocation:", x, "objective:", c @ x)
print("slacks:", h - G @ x)
```

負成本把最大化正收益轉為最小化。容量限制使非負選擇有界；移除關鍵界可能無界，是建模結果而非求解器缺陷。

二次規劃與凸 QCQP

白話版。二次規劃把直線成本換成碗狀成本，規則仍線性。向上彎的二次限制也能加入。

凸 QP 目標為

$$f_0(x) = \frac{1}{2} x^\top Q x + c^\top x + d, \qquad Q \succeq 0.$$

Q 對稱半正定，c 是線性係數，d 是不影響最佳解的常數，限制為仿射。

最小平方展開後 $Q = A^\top A$, $c = -A^\top b$。二次正則化加正對角曲率。設計矩陣 rank 不足時，未正則化解未必唯一。

凸 QCQP 允許

$$\frac{1}{2} x^\top Q_i x + a_i^\top x + r_i \le 0, \qquad Q_i \succeq 0.$$

Q_i 是限制曲率，a_i 是線性係數，r_i 是偏移。任意二次限制不保證凸；不定二次限制即使搭配強凸目標，也可能非凸。

數值結構很重要

無限制正定 QP 解 $Qx = -c$。受限最佳點可能在邊界，不能通常先解無限制再裁切。

一般二次函數耦合座標，調一個座標會改變其他座標最佳值。只有特定幾何，如普通二範數的盒投影，逐座標裁切才正確。

Q 大小與稀疏性影響計算。明算 $A^\top A$ 可能破壞稀疏性與放大病態，實作可保留殘差算子。演算法章會討論分解與矩陣向量乘法。

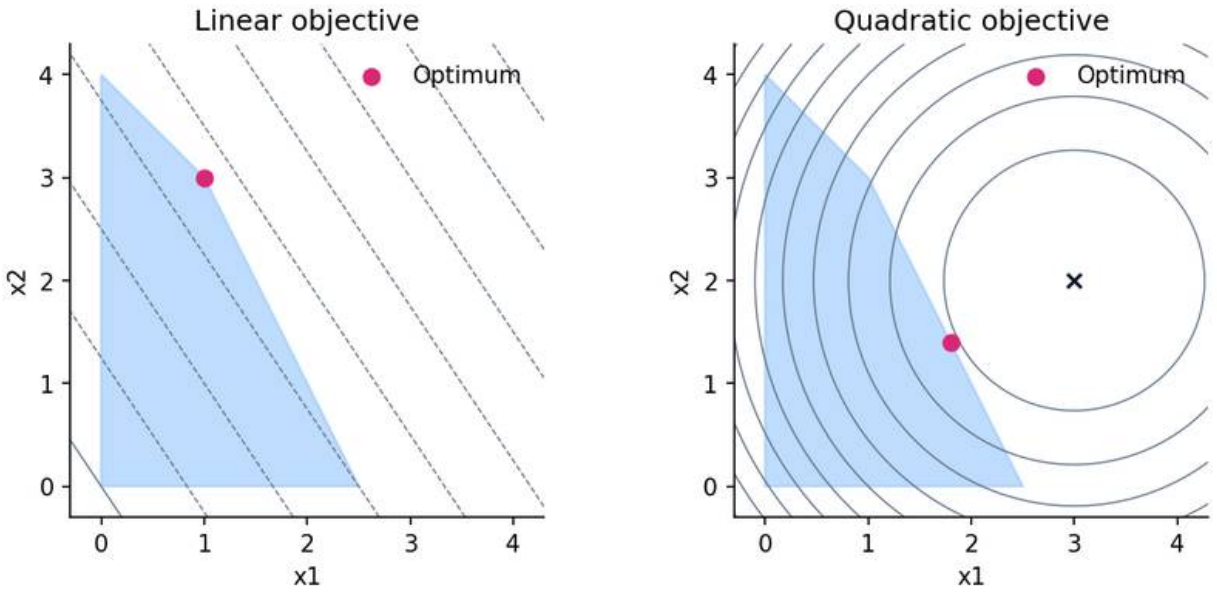


Figure 1 — 原創解析比較。本例線性成本偏好角點，二次成本選離目標最近的可行點；兩者使用相同凸可行集合。

反例算解：為什麼 QP 不能直接裁切

耦合曲率讓一個座標改變時，另一個座標的最佳值也跟著移動。考慮凸二次函數

$$f(x) = \frac{1}{2}(x-a)^\top Q(x-a), \qquad Q = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \quad a = \begin{bmatrix} 2 \\ 0 \end{bmatrix}, \quad x_1 \le 1.$$

其中 $x \in \mathbb{R}^2$ 是變數、a 是無限制目標、Q 是正定曲率矩陣。直接裁切 a 得到 $(1,0)$，目標值為一。但固定 $x_1 = 1$ 後，不受限制的第二座標之駐點條件給出 $(x_1 - 2) + 2x_2 = 0$，因此 $x_2 = 1/2$。

候選點 $(1, 1/2)$ 的目標值為 $3/4$，梯度為 $(-3/2, 0)$，由 $x_1 - 1 \le 0$ 的乘子 $3/2$ 平衡。可行性、乘子符號、駐點條件與互補鬆弛因此透過 OPT 04 的條件驗證最佳解。座標裁切忽略了 Q 的非對角元素；歐氏盒投影之所以可裁切，是因為其二次函數可按座標分離。

二階錐規劃

白話版。二階錐以範數小於純量表示弧形限制，形成標準幾何形式。

$$\|A_i x + b_i\|_2 \le c_i^\top x + d_i.$$

A_i, b_i 定義範數中的向量，c_i, d_i 定義仿射純量界，x 是決策。右側自動必須非負。

不能直接平方後漏掉右側正負條件，否則可能新增右側負值的點。錐表示正確保留正負。

常見 rotated second-order cone 要求 $u, v \geq 0$, $2uv \geq \|z\|_2^2$。係數二是慣例，需配合介面。令 $u = t$, $v = 1/2$, $z = Ax - b$，即可表達平方殘差不超過 t。

不同縮放數學等價，數值行為卻可能不同。

Robust 線性限制

係數不確定時，令 $a = \bar{a} + Eu$, $\|u\|_2 \leq 1$。要求所有允許 u 都滿足 $a^\top x \leq b$，等價於

$$\bar{a}^\top x + \|E^\top x\|_2 \leq b.$$

$\bar{a}$ 是名義係數，E 映射球狀不確定性，b 是固定界。範數來自單位球上 $u^\top E^\top x$ 的最大值。

此為指定不確定集合的精確 robust counterpart，不保護集合以外所有實際擾動。集合較大可能保守，集合不合理可能漏掉重要變異。

盒狀不確定性導向一範數項，橢球導向轉換後二範數，連到上一章的 support function [1]。

推導穩健對應式，而不是猜測懲罰

穩健範數項是容許擾動的精確最壞結果。展開不確定線性限制：

$$\sup_{\|u\|_2 \leq 1} (\bar{a} + Eu)^\top x = \bar{a}^\top x + \sup_{\|u\|_2 \leq 1} u^\top E^\top x = \bar{a}^\top x + \|E^\top x\|_2.$$

其中 x 是決策、$\bar{a}$ 是名目係數、E 是不確定性映射、u 是容許擾動。最後的等號來自內積範數上界；分母非零時，$u = E^\top x / \|E^\top x\|_2$ 可取等號。分母為零時，所有擾動的貢獻都為零。

對合成限制 $(1 + 0.2u_1)x_1 + (2 + 0.1u_2)x_2 \leq 4$，其中 $\|u\|_2 \leq 1$，精確穩健規則為 $x_1 + 2x_2 + \sqrt{0.04x_1^2 + 0.01x_2^2} \leq 4$。在 $x = (1, 1)$，左側最壞值為 $3 + \sqrt{0.05}$，因此穩健可行。最大化擾動與 $(0.2, 0.1)$ 成比例。這裡沒有憑空選調整權重；新增項直接來自所指定的不確定性幾何 [1]。

半正定規劃

白話版。 SDP 限制完整矩陣在每個向量方向都非負，不能只檢查元素。

$$F(x) = F_0 + \sum_{i=1}^{n} x_i F_i \succeq 0.$$

$F_0, \dots, F_n$ 是固定對稱矩陣，x_i 是決策純量，$F(x)$ 必須半正定。搭配線性目標與仿射等式得到 SDP。

若矩陣全對角，退化為線性不等式；一般矩陣則編碼元素之間的關係。

$$X = \begin{bmatrix} 1 & r \\ r & 1 \end{bmatrix}.$$

r 是 off-diagonal correlation，此矩陣半正定若且唯若 $-1 \le r \le 1$。負元素可允許，過大正元素反而不定。

Schur complement 揭露結構

正定 P 下：

$$\begin{bmatrix} P & q \\ q^\top & t \end{bmatrix} \succeq 0 \quad \iff \quad t - q^\top P^{-1} q \ge 0.$$

P 是對稱正定區塊，q 是向量，t 是純量。右側是 Schur complement。奇異 P 還需要 range 條件或廣義論證。

區塊對決策仿射時，可把部分反矩陣式轉成矩陣限制。凸性依賴仿射結構，不是只要寫成區塊矩陣就成立。

用配方證明 Schur 條件

區塊矩陣半正定，代表其二次型對每個向量都非負。對 $P \succ 0$、向量 v 與純量 α，

$$\begin{aligned} \begin{bmatrix} v \\ \alpha \end{bmatrix}^\top \begin{bmatrix} P & q \\ q^\top & t \end{bmatrix} \begin{bmatrix} v \\ \alpha \end{bmatrix} &= v^\top P v + 2\alpha q^\top v + \alpha^2 t \\ &= (v + \alpha P^{-1} q)^\top P (v + \alpha P^{-1} q) + \alpha^2 (t - q^\top P^{-1} q). \end{aligned}$$

其中 P 是正定區塊、q 是相容向量、t 是純量區塊、v, α 是任意測試座標。若 $t \ge q^\top P^{-1} q$，兩項都非負。反過來，選 $\alpha = 1$、$v = -P^{-1} q$，第一項消失，強制該純量不等式成立。如此證明等價性的兩個方向 [1]。

它也是建模方式。固定 P 且 $q(x)$ 為仿射向量時，上界 $q(x)^\top P^{-1} q(x) \le t$ 可化成對 (x, t) 仿射的矩陣不等式。若 P 是矩陣決策變數，在正定定義域上，同樣的區塊表示仍可對其元素仿射。因此原反矩陣式雖把結構藏起來，非線性項仍可能具有凸上圖集合。

假設 $P \succ 0$ 才能配方並使用反矩陣。P 奇異時不能默默沿用同一公式；向量的值域條件會成為等價性的一部分。

程式碼：矩陣順序不是逐元素順序

此例檢查已知特徵值，並驗證 Schur 邊界。

```
import numpy as np

for r in [-0.8, 0.8, 1.2]:
    X = np.array([[1., r], [r, 1.]])
    eigenvalues = np.linalg.eigvalsh(X)
    print("r:", r, "eigenvalues:", eigenvalues)
    assert (eigenvalues.min() >= -1e-12) == (abs(r) <= 1)
P = np.array([[2., .3], [.3, 1.]])
```

```
q = np.array([.4, -.2])
t = q @ np.linalg.solve(P, q)
block = np.block([[P, q[:, None]], [q[None, :], np.array([[t]])]])
assert np.linalg.eigvalsh(block).min() >= -1e-12
print("Schur boundary t:", t)
```

這是 spectral 檢查，不是完整 SDP 求解器。小型直接計算說明模型，不暗示通用非線性 optimizer 提供專用錐方法的全部證書。

幾何規劃

白話版。正變數模型在原座標可能非凸，取對數使乘法變加法後，卻可能凸。

Geometric programming 的 monomial：

$$m(x) = c \prod_{i=1}^{n} x_i^{a_i}, \qquad c > 0, \quad x_i > 0.$$

c 正，a_i 可為任意實數，不同於基礎代數中的非負整數指數慣例。

Posynomial 是 monomial 加總：

$$f(x) = \sum_{k=1}^{K} c_k \prod_{i} x_i^{a_{ik}}, \qquad c_k > 0.$$

K 是項數，c_k 正，a_{ik} 是指數，不能有負係數。

GP 最小化 posynomial，限制 posynomial 不超過一、monomial 等於一，且變數正 [1，第 4.5 節]。任意加總、正負或等式不屬於此類。

推導對數轉換

令 $y_i = \log x_i$，則 $x_i = e^{y_i}$。Monomial 的對數仿射，posynomial 的對數：

$$\log f(e^{y}) = \log \sum_{k=1}^{K} \exp(a_k^{\top} y + \log c_k).$$

a_k 是第 k 項指數向量，右側為仿射式的 log-sum-exp，因此凸。

對數嚴格遞增，所以原目標與 log 目標有相同最佳選擇；回轉變數即可。Posynomial 不等式轉為凸不等式，monomial 等式轉為仿射等式。

目標數值已改變，報告應用成本時要轉回。這是正定義域的一對一變換；刪負項或把非 monomial 等式改不等式，則可能改變問題。

程式碼：對數座標的正配置

最小化 $1/x + 1/y$，且 $x + y \le 4$, $x, y > 0$。對稱性與凸目標給 $x = y = 2$，以下數值驗證。

```
import numpy as np
from scipy.optimize import minimize
from scipy.special import logsumexp

objective = lambda z: logsumexp(-z)
constraint = {"type": "ineq",
              "fun": lambda z: np.log(4.) - logsumexp(z)}
result = minimize(objective, np.log([1., 1.]),
                  constraints=[constraint], method="SLSQP",
                  options={"ftol": 1e-12, "maxiter": 100})
assert result.success, result.message
x = np.exp(result.x)
assert x.sum() <= 4 + 1e-7
assert np.allclose(x, [2., 2.], atol=1e-5)
print("positive variables:", x)
print("original objective:", np.sum(1 / x))
print("transformed objective:", result.fun)
```

介面見 [3]。這個小例與解析解比較，不宣稱通用方法在所有尺度可替代專用 GP 求解器。

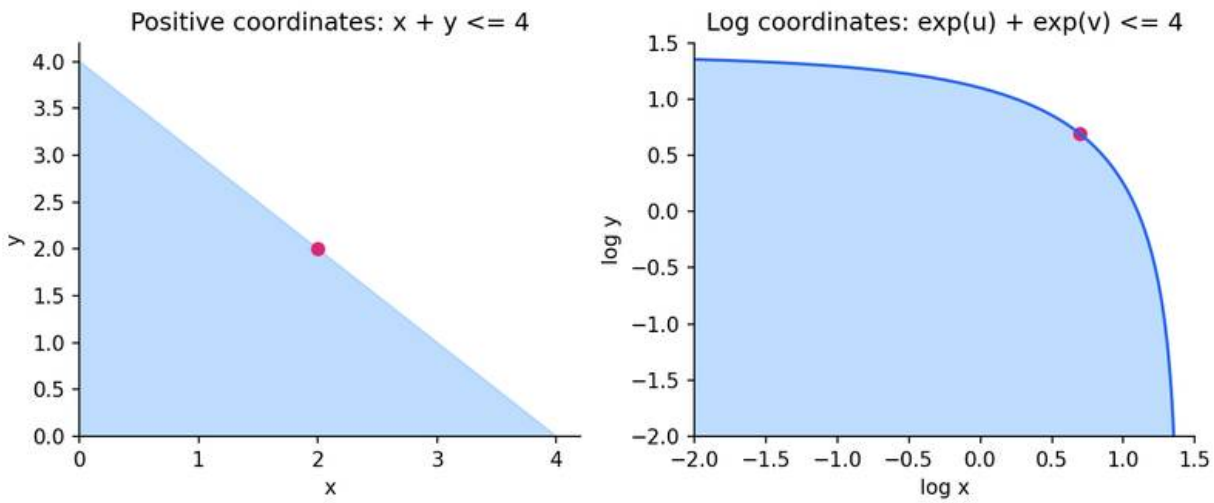


Figure 2 — 原創解析幾何。正變數下 $x + y \le 4$，對數座標變成 *log-sum-exp* $\le \log 4$。兩張圖的軸代表不同變數。

設計算例：非對稱的正配置

即使不能靠對稱性找到答案，對數轉換仍保留最佳解。在 $x + y \le 6$、$x, y > 0$ 下最小化 $4/x + 1/y$。兩項都隨其變數增大而下降，因此最佳解會用完預算。

令 $y = 6 - x$。微分剩下的純量目標，得到

$$-\frac{4}{x^2} + \frac{1}{(6-x)^2} = 0 \quad \Longrightarrow \quad x = 2(6-x) \quad \Longrightarrow \quad (x^*, y^*) = (4, 2).$$

其中 x, y 是正配置量，星號表示最佳解。在 $0 < x < 6$ 上二階導數為正，所以駐點是唯一最小解，原成本為 3/2。

換成對數變數 $s = \log x$、$t = \log y$，目標變為 $\log(4e^{-s} + e^{-t})$，限制變為 $\log(e^s + e^t) \le \log 6$，兩者都是 log-sum-exp。轉換後解為 $(s^*, t^*) = (\log 4, \log 2)$，轉換成本為 $\log(3/2)$。若把 $\log(3/2)$ 回報成原成本，就混淆兩套座標。這個例子提供整個轉換的解析檢查 [1，第 4.5 節]。

以可行性求解擬凸問題

白話版。有些目標非凸，但「此分數能否達到」是凸可行問題。反覆詢問門檻，可找到最佳分數。

若 $\{x : f_0(x) \le \alpha\}$ 對每個固定門檻 α 都能用凸限制描述，可維持最佳分數上下界，檢查中間門檻：可行就降低上界，不可行就提高下界。

每個子問題的門檻是固定資料。若把它也當自由變數，可能產生非凸乘積；各子問題凸，不代表門檻與決策共同凸。

例如正分母下的仿射比值 $(a^\top x + b)/(c^\top x + d)$，固定 α 時限制變成 $a^\top x + b \le \alpha(c^\top x + d)$，對 x 仿射。分母嚴格正是必要條件。

接近最佳門檻時，可行區域可能很窄。近似求解失敗不等於不可行證書，需適當可行程序，報告門檻區間與容忍值。[1] 的擬凸討論擴展非凸目標，但保證仍精確限於凸子問題與外層二分。

求解前的建模稽核

逐條把原需求轉成變數，檢查定義域、正負號、單位，並追蹤輔助變數消去方式。

範數的輔助上界若不在目標中，可能保持鬆；若只要求限制，仍可正確，但不可直接當成實際範數。應自行計算真正範數。

Lifted 矩陣半正定凸，但要求等於向量外積會引入 rank 限制。刪除此限制是鬆弛，矩陣可能提供下界卻沒有對應原向量。

也要檢查邊界。GP 變數不能任意設零，log-determinant 矩陣不能在有限目標下奇異。允許這些點的改寫，需要明確延伸論證。

這些是數學驗證。求解器可能正確最佳化錯誤鬆弛或錯誤正負的模型；與小型解析案例比較，能檢查從文字到陣列再到解釋的整條流程。

多目標與 Pareto 權衡

白話版。兩目標衝突時，最佳化不會替你決定偏好；它提供指定偏好下的答案與權衡。

$F(x) = (f_1(x), \dots, f_k(x))$ 中，k 是目標數。若沒有其他可行點能改善一項而不傷害其他項，則為 Pareto 最佳。

$$\min_x \sum_{i=1}^{k} w_i f_i(x), \qquad w_i > 0.$$

w_i 編碼偏好與單位；達到的最佳解是 Pareto 最佳。零權重可忽略某項，只提供較弱效率敘述。

也可最小化一項，限制其他項不超過指定值。例如在固定記憶體預算下最小化重建誤差，比虛構 bytes 到誤差的通用換算更自然。

Scalarization 仍需符合凸規則。變動權重得到的是此模型的權衡曲線，不涵蓋被排除的離散選擇或上游系統行為。

建模與數值檢查

白話版。文字公式與實際陣列必須描述相同決策，微小錯誤可能悄悄改變限制。

檢查變數預設界。有的介面預設非負，有的自由；忽略預設可把原本自由迴歸係數變成非負。

檢查不等式方向。數學 $g(x) \le 0$ 與介面 $g(x) \ge 0$ 可能同時出現，必須刻意翻譯。

檢查形狀與對稱。非對稱陣列不符合對稱半正定記號；broadcasting 可能把純量限制變成不同向量限制。

縮放需保留與原變數的關係，解與殘差都應轉回原單位檢查。轉換形式與原形式都驗證。

狀態與證書也分開。小例有解析答案時，通用求解器成功提供數值支持；正式凸模型還應依類型檢查原始可行、對偶可行與 gap。

求解器轉換後，寫出的模型仍應可以審閱。應在同一份回報保留變數單位、定義域限制、轉換與所有鬆弛。若回傳轉換解，先映回原座標，再檢查物理限制與應用目標。

對數座標、輔助上圖變數與提升矩陣模型尤其需要這樣做。沒有映射的轉換可行點可能被誤解；若定義限制未準確滿足，輔助目標也可能與原表示式不同。

現代觀點

白話版。標準類型仍適用現代研究，新實作常利用結構改善成本，而非改變數學類型。

Disciplined convex programming [4] 透過曲率與單調性規則，將高層描述精確轉成標準形式。先讓凸性論證明確，再選求解器。

PDLP [5] 於 2025 公開、2026 修訂，以實用一階原始—對偶方法處理大型 LP，加入 preconditioning、restart 與可行修整。結論針對測試問題與精度，並非所有 LP 的通用冠軍。

Schaller、Bemporad 與 Boyd [6] 擬合符合 disciplined programming 的參數化凸函數，讓學習式目標或限制進入凸建模流程，但須維持有效參數與決策變數凸性。

開放問題包括減少不必要展開、利用錐稀疏性、尺度下可靠證明不可行，以及 active-set 變化附近的微分。凸類型是分析基礎，完整系統仍需資料、容忍值與工程驗證。

重點整理

- 凸標準形式需要凸目標與不等式、仿射等式。
- 輔助變數的精確性應證明兩方向。
- LP、QP、SOCP 與 SDP 編碼不同幾何。
- 矩陣順序不同於逐座標順序。
- GP 要求正變數與受限 monomial ／ posynomial。
- 多目標需要明確偏好或界限。
- 狀態、原變數可行性與最佳證書應分別檢查。

練習

1. 將 minimax 絕對殘差擬合改為 LP，證明等價。

   提示
   一個純量界限制所有正負殘差。

2. 範數右側可能負時，為何平方會新增點？

   提示
   範數非負。

3. 推導二乘二、對角一的半正定矩陣中 r 範圍。

   提示
   特徵值 $1+r, 1-r$。

4. 把 $x^{-1}+y^{-1}$ 且 $x+y\le 4$ 轉為 log 座標，再回復原目標。

   提示
   代入指數並對正式取 log。

5. 提出凸函數的非凸等號水平集合。

   提示
   正水平的平方範數。

6. 比較誤差—成本加權和，與固定成本限制下最小誤差的含意。

   提示
   一個是交換率，另一個是硬承諾。

參考文獻

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004。**閱讀：第 4 章，特別 4.3–4.7；第 6.1 節範數擬合**。官方書籍、免費 PDF。
2. SciPy developers, “linprog,” *SciPy API documentation*, accessed 2026。已驗證介面。
3. SciPy developers, “minimize,” *SciPy API documentation*, accessed 2026。最佳化介面。
4. Michael Grant, Stephen Boyd and Yinyu Ye, “Disciplined Convex Programming,” in *Global Optimization: From Theory to Implementation*, Springer, 2006。作者出版頁。
5. David Applegate, Mateo Díaz, Oliver Hinder, Haihao Lu, Miles Lubin, Brendan O’Donoghue and Warren Schudy, “PDLP: A Practical First-Order Method for Large-Scale Linear Programming,” arXiv preprint, 2025，2026 修訂。論文記錄。
6. Michael Schaller, Alberto Bemporad and Stephen Boyd, “Learning Parametric Convex Functions,” manuscript, June 2025。已驗證稿件。

OPT 04 ・對偶性、KKT 條件與證明憑證

推導拉格朗日下界，區分弱對偶與強對偶，以 KKT 驗證最佳性，並解讀乘子、敏感度與不可行性憑證。

網站版本: https://blog.shyandram.dev/zh-tw/learn/optimization/opt-04-duality-kkt/

你將學到

- 從拉格朗日函數推導受限制最小化問題的下界。
- 區分對偶可行性、弱對偶、強對偶與最佳值是否達到。
- 寫出 KKT 條件，辨認它何時能證明全域最佳性。
- 以正確的乘子符號解讀互補鬆弛與敏感度。
- 建立簡單的不可行性憑證，並把對偶序關係延伸至錐。
- 回報殘差與間隙，同時保留數值證據真正支持的範圍。

先看全貌

找到好的可行解，能給最佳成本一個上界。但是，我們怎麼知道其他地方沒有藏著更好的答案？對偶問題會產生下界。當上界與下界相遇，兩者就共同證明最佳值。

Boyd 與 Vandenberghe 的第 5 章 [1] 把這個想法連到幾何、支撐平面、乘子與數值演算法。即使已經會解原問題，對偶仍有用：它能驗證答案、說明有限資源的價值，並揭露另一種計算結構。

> **用最簡單的話說**
>
> 你找到一份花十枚硬幣的食譜。另一個有力的論證指出，每份合乎規定的食譜至少都要十枚硬幣。兩者合在一起，就證明你的食譜最便宜。找到食譜與證明下界，是兩件不同的工作。

保證必須遵守符號與假設。任意選乘子算出的數值，不一定是下界。單靠很小的駐點殘差，也不是完整的證明。並非每個凸問題表示法都自動具備強對偶。

本章保留明確的定義域，使用能檢查精確答案的小例子推導。這套推理可以延伸至大問題，但實作細節與容許誤差仍是數值判斷的一部分。

從限制式到拉格朗日函數

白話說。替每個不等式與等式附上一個價格，再最小化加上價格後的目標，而不直接強制這些限制。

考慮

$$\min_{x\in D} f_0(x) \quad \text{subject to} \quad f_i(x)\le 0,\quad h_j(x)=0.$$

x 是決策向量，D 是最小化時保留的共同定義域，f_0 是目標，f_i 是不等式，h_j 是等式。

拉格朗日函數為

$$L(x,\lambda,\nu)=f_0(x)+\sum_{i=1}^{m}\lambda_i f_i(x)+\sum_{j=1}^{p}\nu_j h_j(x).$$

λ_i 是不等式乘子，ν_j 是等式乘子。若採用 $f_i\le 0$ 的慣例，就要求 $\lambda_i\ge 0$。等式乘子不限制符號，因為任何符號乘上可行點的等式殘差，結果都是零。

在可行點，不等式的貢獻非正，等式的貢獻為零。因此，當 λ 非負時，$L(x,\lambda,\nu)\le f_0(x)$。

把乘子稱為「懲罰」可能造成誤解。這裡的拉格朗日函數，並不是在無限制最小化時替違規加上一個很大的正成本。它的用途是建立一族下界，符號限制也由此而來。

定義對偶函數

$$g(\lambda,\nu)=\inf_{x\in D} L(x,\lambda,\nu).$$

g 是拉格朗日對偶函數，輸入為乘子對。它對 D 中的所有點取下確界，而非只看可行點。

對任一可行的 x 與非負的 λ，

$$g(\lambda,\nu)\le L(x,\lambda,\nu)\le f_0(x).$$

第一個不等式來自下確界，第二個來自原問題可行性與乘子符號。因此，g 是原問題最佳值的下界。

若下確界為負無窮，這仍是正確、但沒有提供資訊的下界。想取得有用的憑證，必須選擇能產生適當有限對偶值的乘子。

為什麼對偶問題是凹的

白話說。下界會隨價格改變。我們選擇能給最強下界的價格；這個最大化問題的目標是凹函數。

固定每個 x 時，L 對乘子都是仿射函數。仿射函數逐點取下確界，會得到凹函數。因此，即使原問題不是凸的，g 仍是凹的。

對偶問題是

$$\begin{aligned}\text{maximize}_{\lambda,\nu}\quad & g(\lambda,\nu)\\ \text{subject to}\quad & \lambda\ge 0.\end{aligned}$$

以 $d^\star$ 表示對偶最佳值。弱對偶（weak duality）指出 $d^\star\le p^\star$，其中 $p^\star$ 是原問題最佳值。這個結果不要求凸性，只需要符號正確的下界構造。

最大化凹函數，符合一般凸最佳化問題的意義。但是，求對偶函數值本身可能很困難，因為它也許要求解一個難解的非凸最小化。「對偶是凹的」不等於自動擁有容易計算的查詢工具。

對偶與原問題的維度也不必相同。每個明確對偶化的限制式各產生一個變數。把部分限制留在 D 中，而不加乘子，會改變寫出的對偶表示，但下界邏輯依然一致。

三行看懂凹性證明

對偶為凹，是因為取下確界後，混合價格的下界不會低於個別下確界的相應混合。令 $z=(\lambda,\nu)$ 收集乘子，取兩組容許價格 z_1,z_2 與 $\theta\in[0,1]$。拉格朗日函數對價格為仿射，因此

$$\begin{aligned} g(\theta z_1+(1-\theta)z_2) &= \inf_{x\in D}\left[\theta L(x,z_1)+(1-\theta)L(x,z_2)\right] \\ &\geq \theta \inf_{x\in D} L(x,z_1)+(1-\theta)\inf_{x\in D} L(x,z_2) \\ &= \theta g(z_1)+(1-\theta)g(z_2). \end{aligned}$$

其中 D 是保留的原問題定義域、g 是對偶函數；此比較考慮有限的對偶值。在同一 x 處，每項都至少等於自己的下確界，因此得到中間不等式。這個步驟完全沒有假設原目標凸 [1]。

計算上必須區分：原問題非凸時，只找到 L 的局部最小值，並不等於計算其下確界。該局部值可能高於 $g(z)$，因而失去合法下界的意義。對偶凹性描述的是精確對偶函數，而不是任何自稱「對偶評估器」的近似程序。

完整的一維推導

白話說。一個小例子就能看清價格、無限制最小化解與下界之間的關係。

在 $x\leq 1$ 下最小化 $\frac{1}{2}(x-2)^2$。x 是純量。受限制最佳解是 $x^*=1$，成本 $p^*=1/2$。

使用乘子 $\lambda\geq 0$，

$$L(x,\lambda)=\frac{1}{2}(x-2)^2+\lambda(x-1).$$

拉格朗日函數的無限制最小化解滿足 $x-2+\lambda=0$，所以 $x_\lambda=2-\lambda$。代回後得到

$$g(\lambda)=\lambda-\frac{1}{2}\lambda^2.$$

x_λ 隨選擇的乘子而變，不一定是原問題可行點。對偶是一條凹拋物線，在 $\lambda^*=1$ 達到最大值 $g(\lambda^*)=1/2$。上界與下界於此相遇。

最佳乘子平衡了目標想增加 x 的力量，以及作用中的上界限制。若 λ 太小，拉格朗日最小化解會落在允許區域之外；若太大，即使價格更高，下界反而更弱。

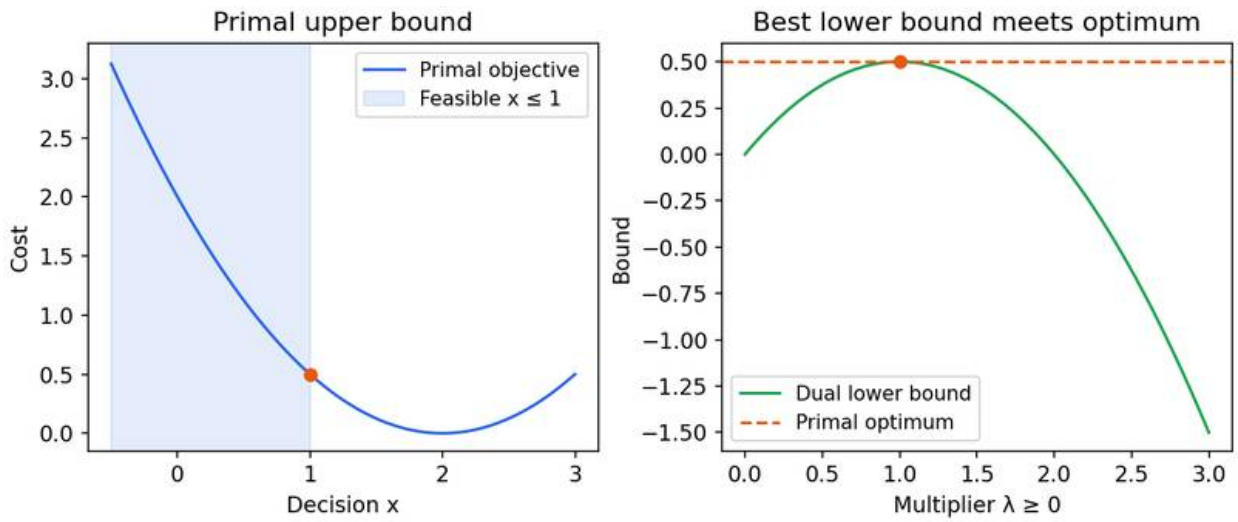


圖 *1* — 在 $x \le 1$ 下最小化 $(x-2)^2 / 2$ 的原創解析推導。可行原問題最佳解與最好的對偶下界，數值同為 $1/2$。

程式：驗證間隙與完整 KKT 條件

以下獨立計算會檢查原問題可行性、乘子符號、駐點條件、互補鬆弛，以及兩個數值相等。

```
import numpy as np

x, multiplier = 1.0, 1.0
objective = 0.5 * (x - 2)**2
dual_value = multiplier - 0.5 * multiplier**2
constraint = x - 1
stationarity = x - 2 + multiplier
complementarity = multiplier * constraint
assert constraint <= 0 and multiplier >= 0
assert abs(stationarity) < 1e-12
assert abs(complementarity) < 1e-12
assert np.isclose(objective, dual_value)
print("primal value:", objective, "dual value:", dual_value)
print("stationarity:", stationarity, "complementarity:", complementarity)
```

強對偶與限制資格條件

白話說。弱對偶永遠給出上下界次序。強對偶則說，最好的下界恰好等於最佳值；這需要額外條件。

強對偶（strong duality）表示 $d^* = p^*$。它描述最佳值，不會自動保證最佳解存在。問題可能只有一串解逼近某個數值，卻沒有任何點真正達到它。

對具有仿射等式的凸原問題，一個標準充分條件是 Slater 條件 [1，第 5.2 節]：

$$\exists x \in \operatorname{relint} D : f_i(x) < 0 \text{ for all } i, \qquad Ax = b.$$

D 是共同函數定義域，$\operatorname{relint} D$ 是它的相對內部，A, b 定義仿射等式限制。意思是存在一個點，位於適當內部、嚴格滿足不等式，並且滿足等式。

在這些假設下，強對偶成立。若最佳值有限，對偶最佳值也會達到。原問題是否達到最佳值，仍可能需要另外討論存在性。

仿射不等式有較弱的精細條件，所以要求它們全部嚴格成立，可能比必要條件更強。沒找到 Slater 點，本身不會推翻強對偶；它只表示這個充分條件尚未證明。

並非每種表示法中，凸性本身就已經足夠。定義域可能退化，限制資格條件也可能不成立。與其說「凸問題的間隙永遠是零」，應說明採用哪個條件，或直接驗證一對有效且相等的憑證。

定義域限制也是對偶的一部分

目標若包含對數或倒數，它的定義域仍要保留在拉格朗日最小化中。把它當成所有實數上都有定義的函數，會悄悄改變問題。

另一種方式是使用延伸值函數，在定義域外給正無窮。數學上這很方便，但數值程式不應把意外產生的無效浮點值當成模型慣例。

錐表示法可以一致地選擇對偶化或保留定義域與錐限制。這會影響對偶公式與符號。應從實際原問題推導下界，不要直接搬用符號不相容的公式。

零間隙也可能沒有最佳乘子

限制資格條件失敗不一定造成正對偶間隙，卻可能使乘子最佳值無法達到。考慮在 $x^2 \le 0$ 下最小化 x，其中純量 $x \in \mathbb{R}$。唯一可行點是零，所以 $p^\star = 0$，而且沒有嚴格可行點。

拉格朗日函數為 $L(x, \lambda) = x + \lambda x^2$。對 x 最小化得到

$$g(\lambda) = \begin{cases} -1/(4\lambda), & \lambda > 0, \\ -\infty, & \lambda = 0. \end{cases}$$

其中 $\lambda \ge 0$ 是二次限制的價格，g 是其對偶下界。對正 λ，最小化點為 $-1/(2\lambda)$。當 λ 無界增大，$g(\lambda)$ 趨近零。因此 $d^\star = p^\star = 0$，但沒有任何有限乘子達到對偶最佳值。

在原問題解處，駐點條件要求 $1 + 2\lambda x = 0$，卻變成 $1 = 0$。這種表示沒有 KKT 乘子。把等價限制改寫成等式 $x = 0$，就有乘子 $\nu = -1$。可行決策不變，正則性與乘子描述卻改善。這正是 OPT 01 的隱藏等式與 OPT 07 的初始化值得重視的原因 [1]。

KKT 最佳性條件

白話說。 KKT 結合允許的原問題選擇、允許的價格、力的平衡，以及「有餘裕的限制不能帶正價格」這條規則。

對可微凸函數 f_0, f_i 與仿射等式 $Ax = b$，條件為

$$
\begin{aligned}
& f_i(x) \le 0, \qquad Ax = b, \\
& \lambda_i \ge 0, \\
& \lambda_i f_i(x) = 0, \\
& \nabla f_0(x) + \sum_i \lambda_i \nabla f_i(x) + A^\top \nu = 0.
\end{aligned}
$$

x 是原問題候選點，λ 是不等式乘子，ν 是等式乘子。最後一列是拉格朗日駐點條件。整組稱為 Karush–Kuhn–Tucker 條件。

對這個凸且可微的問題，全部條件成立，就足以證明原問題與對偶最佳性。非負乘子使拉格朗日函數對 x 為凸；駐點條件使 x 成為它的全域最小化解；可行性與互補鬆弛則使拉格朗日值等於目標值。

在適當資格條件，例如 Slater 條件下，最佳點也會存在滿足 KKT 的乘子。這提供必要性，以及前述充分性。應把兩個邏輯方向分清楚。

非凸問題的拉格朗日駐點，可能是對 x 的局部最大值或鞍點。KKT 在正則條件下可能必要，通常卻不能證明全域最佳性。凹二次函數的內部駐點就是簡單反例。

互補鬆弛提供單向資訊

若 $f_i(x) < 0$，互補鬆弛會迫使 $\lambda_i = 0$。若 $\lambda_i > 0$，限制必須作用中：$f_i(x) = 0$。

作用中的限制仍可能有零乘子。退化或冗餘限制可能使價格不唯一。因此，「作用中」與「正價格」不能互換。

在平滑受限制最佳點，負目標梯度會落在作用中限制法向量與等式法向量、搭配適當符號形成的組合裡。這解釋了駐點條件的幾何意義：可行下降方向無法降低目標。

數值上的接近作用中，依賴容許誤差。很小的餘裕搭配很小的乘子，可能難以做離散分類。若區別重要，應回報實際數值，避免硬給確定的作用中或非作用中標籤。

等式限制二次問題

白話說。二次目標搭配線性等式時，KKT 條件形成一個線性系統，揭露後續受限制演算法需要的代數。

在 $Ax = b$ 下，目標為 $\frac{1}{2}x^\top Qx + c^\top x$，則

$$
\begin{bmatrix} Q & A^\top \\ A & 0 \end{bmatrix}
\begin{bmatrix} x \\ \nu \end{bmatrix}
=
\begin{bmatrix} -c \\ b \end{bmatrix}.
$$

Q 是對稱半正定矩陣，c 是目標係數向量，A 是等式矩陣，b 是目標值，ν 是等式乘子。

即使最佳化問題是凸的，這個區塊矩陣通常仍是不定矩陣。要求正定矩陣的求解器，不能直接套用到完整 KKT 系統。

若 Q 正定，且 A 的列獨立，系統就非奇異。更一般的情況，需要考慮等式零空間上的曲率，以及冗餘限制。偽逆可能給出一個解，但不應掩蓋背後的秩問題。

等式乘子會在可行仿射集合的法向方向，平衡目標梯度。它的符號不受限制。移除限制會同時改變可行集合與乘子的意義。

謹慎推導 LP 對偶

白話說。對偶公式沒有一套通用符號。它的正負號來自實際使用的原問題慣例。

取 LP：在 $Ax = b,\ x \geq 0$ 下最小化 $c^\top x$。把非負限制保留在定義域。拉格朗日函數為 $c^\top x + \nu^\top (Ax - b)$。

對非負 x 取下確界，恰在 $c + A^\top \nu \geq 0$ 時有限。任一係數為負時，該座標可以無限增大，使下確界變成負無窮。

令 $y = -\nu$，對偶變成

$$\max_{y} b^\top y \quad \text{subject to} \quad A^\top y \leq c.$$

y 是符號不受限制的等式價格向量。對任何可行的原問題與對偶解，

$$c^\top x - b^\top y = x^\top (c - A^\top y) \geq 0.$$

間隙是非負原問題座標與非負對偶餘裕的內積。這是特別直觀的弱對偶表示。

改用原問題最大化，或反轉不等式，就會改變對偶符號。推導拉格朗日函數並檢查下界，比記憶不同表格更能防止錯誤。

從範數共軛建立擬合對偶

對偶可把參數懲罰變成殘差限制。考慮稀疏線性擬合，目標為 $\frac{1}{2}\|Ax - b\|_2^2 + \lambda \|x\|_1$，其中 A, b 固定、$\lambda > 0$。引入殘差變數 $r = Ax - b$ 與不受符號限制的等式乘子 y：

$$L(x, r, y) = \frac{1}{2}\|r\|_2^2 + \lambda \|x\|_1 + y^\top (Ax - b - r).$$

其中 x 是參數向量，y 的每個座標對應一筆觀測。對 r 最小化得到 $r = y$，殘差相關項的值為 $-\|y\|_2^2/2$。依 OPT 02 的範數共軛計算，對 x 最小化恰好在 $\|A^\top y\|_\infty \leq \lambda$ 時有限。因此對偶為

$$\text{maximize}_{y} \quad -\frac{1}{2}\|y\|_2^2 - b^\top y \qquad \text{subject to} \quad \|A^\top y\|_\infty \leq \lambda.$$

其中 y 是對偶變數，範數限制是對偶可行性。在最佳解處，它等於擬合殘差，而殘差與設計欄向量的相關量必須留在懲罰門檻內。這連起下界構造與稀疏擬合幾何 [1，第 5–6 章]。

對合成 $A = I$、$b = (2, -0.3, 0.1)$ 與 $\lambda = 0.4$，原問題解為 $x = (1.6, 0, 0)$，對偶候選為 $y = x - b = (-0.4, 0.3, -0.1)$。兩者分別可行，目標值皆為 0.77。上下界相等才證明擬合最佳；只看到零座標並不足夠。

敏感度與限制的價值

白話說。在指定正則條件下，乘子衡量最佳成本如何隨限制變動。它的單位取決於限制的單位。

把不等式改為 $f_i(x) \le u_i$，等式改為 $Ax - b = v$。以 $p^\star(u, v)$ 表示最佳成本。若零擾動時強對偶成立且最佳乘子存在，

$$p^\star(u, v) \ge p^\star(0, 0) - \lambda^{\star\top} u - \nu^{\star\top} v.$$

u, v 是右側擾動；$\lambda^\star, \nu^\star$ 是原問題的最佳乘子。依此慣例，正 u_i 會放寬不等式。

此式給擾動最佳值一個全域仿射下界。若值函數在原問題處可微，它的導數就是 $-\lambda^\star$ 與 $-\nu^\star$。

若不可微，乘子可能描述支撐斜率，而非唯一導數。大的乘子表示模型在這個局部方向敏感，不是普遍適用的物理重要性分數。

上界改變的純量例子

在 $x \le b$ 下最小化 $\tfrac{1}{2}(x - 2)^2$。最佳點是 $\min(2, b)$，最佳值為

$$p^\star(b) = \frac{1}{2}\max(2 - b, 0)^2, \qquad \lambda^\star(b) = \max(2 - b, 0).$$

b 是允許的上界。在可微處，$dp^\star/db = -\lambda^\star(b)$。一旦上界超過無限制目標，再放寬已沒有幫助，乘子就是零。

限制縮放會使乘子反向縮放。把 $x - b \le 0$ 乘上正數，不會改變可行點，但會改變價格。比較敏感度時，應先處理單位與正規化。

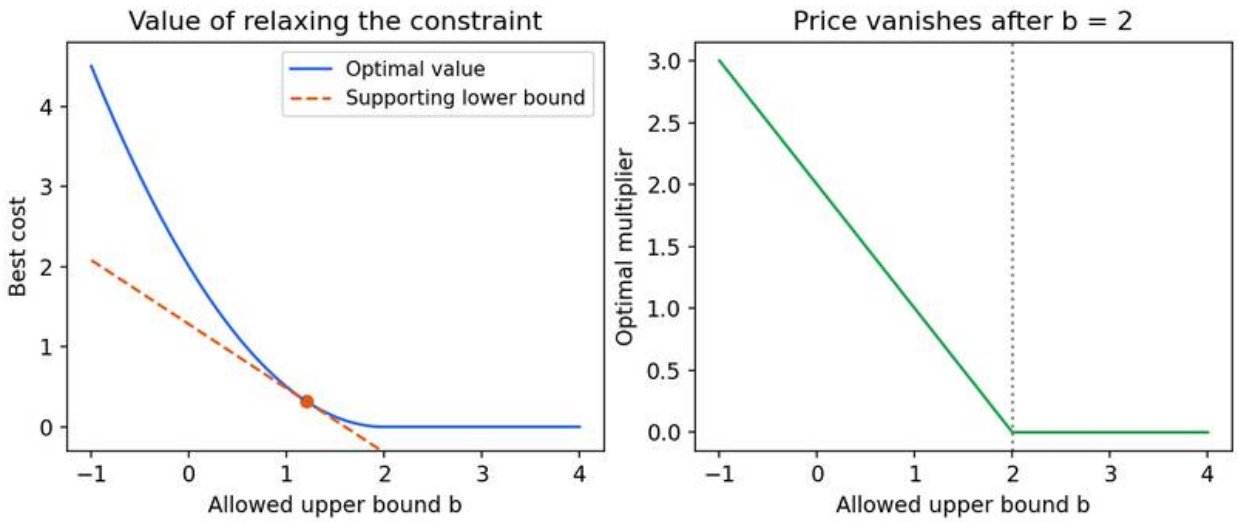


圖 2 — 原創解析敏感度例子。限制仍有效時，放寬上界會降低成本；允許無限制目標之後，最佳值與乘子都為零。

程式：有限差分敏感度檢查

```
import numpy as np

value = lambda b: 0.5 * max(2 - b, 0)**2
b, eps = 1.2, 1e-5
multiplier = max(2 - b, 0)
derivative = (value(b + eps) - value(b - eps)) / (2 * eps)
assert np.isclose(derivative, -multiplier, atol=1e-8)
test_bounds = np.linspace(-1, 4, 50)
lower_bound = value(b) - multiplier * (test_bounds - b)
assert np.all([value(t) for t in test_bounds] >= lower_bound - 1e-12)
print("finite-difference derivative:", derivative)
print("negative multiplier:", -multiplier)
```

從同一個下界推導敏感度

敏感度不等式不需要新原理。把原問題的最佳乘子放進擾動後的拉格朗日函數。對任何滿足 $f_i(x) \le u_i$ 與 $h_j(x) = v_j$ 的點，

$$f_0(x) \ge g(\lambda^*, \nu^*) - {\lambda^*}^{\top} u - {\nu^*}^{\top} v.$$

其中 u, v 是擾動向量，星號乘子對未擾動問題最佳。對擾動可行點取下確界，再使用原問題的強對偶，就得到前述值函數下界。負號來自在拉格朗日函數中扣除擾動後的右側。

對純量上界例子，在 $b = 1$ 時值為 $1/2$、$\lambda^* = 1$。把上界增加 δ，其中 $0 < \delta < 1$，實際值變成 $(1-\delta)^2/2$。實際下降為 $\delta - \delta^2/2$，乘子則預測一階下降 δ。下界預測 $1/2 - \delta$ 保持在實際值下方，差為 $\delta^2/2$。

若限制改寫為 $10(x - b) \le 0$，乘子變成 $1/10$。限制對 b 的導數為 -10，與乘子相乘仍預測斜率 -1。價格依賴表示方式，原成本變動的預測則一致 [1，第 5.6 節]。

不可行性憑證

白話說。合法加權組合限制式，可以證明沒有任何選擇滿足它們。這比演算法沒找到可行點更有力。

對不等式 $Gx \le h$，假設向量 y 滿足

$$y \ge 0, \qquad G^{\top} y = 0, \qquad h^{\top} y < 0.$$

G, h 定義不等式系統，y 是非負憑證向量。若可行 x 存在，加權求和就會得到 $0 = y^{\top} G x \le y^{\top} h < 0$，產生矛盾。

這是 [1，第 5.8 節] 討論的擇一定理形式背後的幾何。它區分數學憑證與找不到解的搜尋。

在浮點數下，等式與嚴格負號需要相對尺度判斷。很小的轉置殘差，加上極小的負憑證值，可能受累積誤差影響而無法下結論。數值認證需要適當容許誤差分析。

程式：互不相容的純量不等式

限制為 $x \le 0$ 與 $x \ge 1$。非負相加就能證明不可能。

```
import numpy as np

G = np.array([[1.], [-1.]])
h = np.array([0., -1.])
certificate = np.array([1., 1.])
assert np.all(certificate >= 0)
assert np.allclose(G.T @ certificate, 0)
assert h @ certificate < 0
print("transpose residual:", G.T @ certificate)
print("certificate contradiction:", h @ certificate)
```

錐、鞍點與數值間隙

白話說。從相應對偶錐選擇價格，就能把下界推理延伸到錐不等式。原問題與對偶解也可視為鞍點。

錐 K 的對偶錐定義為

$$K^* = \{y : y^\top z \ge 0 \text{ for every } z \in K\}.$$

z 遍歷原錐，y 是價格向量。若限制寫成屬於 K，應選擇讓對偶錐內積產生正確下界的符號。正交象限、二階錐與半正定錐，在常用內積下都有熟悉的自對偶結構。

矩陣變數的內積使用跡：對對稱矩陣，$\langle X, Y\rangle = \operatorname{tr}(XY)$。互補鬆弛因此是跡條件，不是逐元素相乘。

在適當原問題與對偶假設下，鞍點滿足

$$L(x^*, \lambda, \nu) \le L(x^*, \lambda^*, \nu^*) \le L(x, \lambda^*, \nu^*).$$

x^* 與 (λ^*, ν^*) 是最佳原問題與對偶點；x 遍歷定義域，$\lambda \ge 0$。左側在可行點最大化價格，右側在最佳價格下最小化決策。

實務上，只有兩點適當可行，且正確計算對偶值時，目標差才是有證明力的間隙。近似可行可能需要修正上下界。相對間隙應使用明確記錄的分母，在接近零或負目標值時仍有意義。

完整 LP 憑證與無效近似解

考慮在 $x_1 + x_2 = 1$、$x \ge 0$ 下最小化 $x_1 + 3x_2$。原問題點 $(1, 0)$ 成本為一。依前面推導的 LP 對偶慣例，純量對偶變數 $y = 1$ 可行，因為 $(y, y) \le (1, 3)$，對偶值也是一。上下界相等，不需枚舉所有可行點就能證明最佳性。

對偶餘裕為 $(0, 2)$，與原問題點的內積為零，直接呈現互補鬆弛。第二座標相對等式價格較昂貴，因此配置為零。此解釋來自係數與指定等式，不是一般特徵重要性結論。

現在取近似點 $(1.01, -0.01)$。它精確滿足等式，目標為看似更好的 0.98，卻違反非負性。與對偶值相減得到負數，不會推翻弱對偶，因為候選點在可行集合外。

只顯示等式殘差與目標的程式，可能因此回報誤導結果。應檢查所有邊界與錐限制，包括透過求解器變數邊界參數編碼、沒有放進明確矩陣列的限制。

裁切負座標得到 $(1.01, 0)$，隨即違反等式。在這個特定例子，再正規化會得到 $(1, 0)$；但裁切與正規化都不是耦合限制的通用有效修補。

近似殘差需要修正推理

對近似可行 LP 點對，代數目標差為

$$c^\top x - b^\top y = x^\top s + y^\top r, \qquad s = c - A^\top y, \quad r = Ax - b.$$

x, y 是原問題與對偶候選向量，s 是對偶餘裕，r 是原問題等式殘差。恆等式是精確代數；要得到非負且有證明力的間隙，還需要必要可行性。

若 $x \geq 0$、$s \geq 0$，但 $r \neq 0$，殘差項可能正或負。小殘差乘上大價格，仍可能重要。因此，單靠絕對等式容許誤差，不是通用目標準確度保證。

若對偶餘裕逐分量只保證不小於 $-\varepsilon$，而 $x \geq 0$，就有 $x^\top s \geq -\varepsilon \|x\|_1$。$\varepsilon \geq 0$ 衡量最壞負餘裕。大原問題向量可能放大微小對偶違反。

除非另外有所有可行原問題點的有效界，這個不等式不會把近似對偶值變成全域下界。無界可行集合尤其難以修正。不要把任意近似乘子描述為精確全域憑證。

應一起回報原問題殘差、負餘裕大小、乘子範數與目標差。若需要嚴格憑證，應依問題結構採有效修補、已驗證誤差上界，或更高精度檢查。

現代觀點

白話說。現代大規模方法直接使用原問題與對偶結構，應用也越來越常對受限制解做微分。兩者都依賴同樣的憑證與正則性觀念。

Applegate 等人的 PDLP 報告 [2] 在 2025 年首次發布，2026 年修訂，討論實用原問題與對偶步驟及可行性修整的大規模 LP 計算。它在此的意義是：有用的系統要同時監測可行性與成本改善。若目標誤差小，卻帶著不可接受的限制違反，仍是另一種結果。

Pokutta 的 2024 年綜述 [3] 說明 Frank–Wolfe 方法的原問題與對偶收斂。對可行集合做線性最小化，能從支撐平面不等式建立一階間隙。這與下界憑證有關，但假設與計算方式不同於一般拉格朗日乘子對偶。

Agrawal 等人的可微凸最佳化層 [4]，把結構化表示法與解的微分連起來。原論文發表於 2019 年，這裡把它作為現今最佳化層研究的基礎，不把它標成 2026 年新成果。退化或作用中集合改變附近，可微性與良好條件數可能失效；正確凸解不會自動保證良好的梯度。

目前問題包括超大規模的可靠憑證、限制由學習取得時的不確定性解讀、非唯一解的微分，以及精度和應用延遲的平衡。最佳化精度、模型有效性與學習泛化，應分別評估。

重點整理

- 可行原問題點給上界，可行對偶點給下界。
- 乘子符號取決於實際不等式慣例。
- 即使原問題非凸，對偶函數仍是凹的。
- 強對偶需要適當條件，或直接驗證相等的有效憑證。
- KKT 對可微凸問題是充分條件，必要性則需要正則性。
- 互補鬆弛與敏感度有明確的單向規則及符號。
- 數值殘差與間隙必須結合尺度、可行性解讀。

練習

1. 在 $x \le b$ 下最小化 $\frac{1}{2}(x-a)^2$，把 a, b 視為固定資料，推導對偶。

   提示
   先對純量最小化拉格朗日函數，再最大化乘子。

2. 給一個在最佳點作用中、乘子卻為零的不等式。

   提示
   無限制最小化點的冗餘邊界即可。

3. 保留非負變數在定義域中，推導 LP 對偶。

   提示
   要讓下確界有限，線性係數必須非負。

4. 說明在 $[-1, 1]$ 最小化 $-x^2$ 時，原點 KKT 駐點條件為何不能證明最佳性。

   提示
   原點是目標最大值。

5. 建立一組非負加權矛盾，證明兩個純量不等式不可行。

   提示
   把低於必要下界的上界加起來。

6. 把不等式乘上十，推導乘子的縮放，並說明原問題敏感度為何保持一致。

   提示
   保留拉格朗日函數中的乘積。

參考文獻

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. **閱讀：第 5 章，涵蓋對偶、KKT、擾動、擇一定理與廣義不等式**。官方書頁，免費 PDF。
2. David Applegate, Mateo Díaz, Oliver Hinder, Haihao Lu, Miles Lubin, Brendan O'Donoghue and Warren Schudy, "PDLP: A Practical First-Order Method for Large-Scale Linear Programming," arXiv preprint, 2025, revised 2026. 已核對論文。
3. Sebastian Pokutta, "The Frank-Wolfe Algorithm: A Short Introduction," *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 126, 2024. 開放綜述。
4. Akshay Agrawal, Brandon Amos, Shane Barratt, Stephen Boyd, Steven Diamond and J. Zico Kolter, "Differentiable Convex Optimization Layers," *NeurIPS*, 2019. 官方出版頁。

OPT 05 ・近似、統計估計與實驗設計

選擇凸殘差損失、正則化與形狀限制，建立概似和偵測器問題，並以最佳實驗設計配置量測資源。

網站版本: https://blog.shyandram.dev/zh-tw/learn/optimization/opt-05-fitting-estimation/

你將學到

- 依重要的誤差選擇殘差範數，而非只沿用習慣。
- 區分參數正則化、穩健殘差損失與不確定量測模型。
- 使用線性不等式，讓擬合函數保持單調或凸。
- 辨認凸的概似與機率估計表示法。
- 設計隨機化偵測器，理解以動差建立的風險上界。
- 使用資訊矩陣與明確準則，配置量測資源。

先看全貌

最佳化模型可以替觀測擬合曲線、估計分布，或決定接下來收集哪些觀測。它們有共同模式：選擇表示方式、說明如何衡量不一致，再指定允許哪些選擇。

Boyd 與 Vandenberghe 的第 6–7 章 [1] 連結近似與統計估計。適合一起閱讀：同一個平方殘差，可以是幾何近似誤差、雜訊模型下的概似，或受限制設計目標的一部分。即使數值公式相同，解釋仍帶有不同假設。

> **用最簡單的話說**
> 你想找一把靠近許多點的尺。一條規則在意平均不準多少；另一條在意最差的點；第三條不允許尺彎得太厲害。找最佳尺以前，要先決定哪種「最好」符合用途。

成功求解器回答的是你寫下的最佳化問題。它不會證明量測假設正確、曲線能預測新資料，或感測器配置可以實際執行。本章把這些問題與數學一起保留。

例子使用合成觀測與小型量測模型，展示各部分如何運作，不在程式裡隱藏下載、預訓練系統或特定應用假設。

殘差範數表達優先順序

白話說。殘差是預測與觀測的差。選擇哪種範數，會決定哪些誤差模式特別昂貴。

線性模型為

$$r = Ax - b, \qquad \min_x \|Ax - b\|.$$

$A \in \mathbb{R}^{m \times n}$ 是固定設計矩陣，$b \in \mathbb{R}^m$ 是觀測，$x \in \mathbb{R}^n$ 是參數，$r \in \mathbb{R}^m$ 是殘差向量。所選範數作用於殘差，不是直接作用於參數。

歐幾里得範數或其平方，透過平方貢獻強調大殘差。絕對值範數加總殘差大小。最大值範數則最小化最大的絕對殘差，適合要求每個量測誤差都不超過一致容許值的場合 [1，第 6.1 節]。

$$\|r\|_2^2 = \sum_i r_i^2, \qquad \|r\|_1 = \sum_i |r_i|, \qquad \|r\|_\infty = \max_i |r_i|.$$

r_i 是第 i 個殘差座標。當 r 對 x 仿射，三者都是凸目標。把非負歐幾里得範數平方不會改變最小化解，但會改變目標值、導數，以及與其他懲罰項的取捨。

絕對值殘差可以引入 $t_i \geq r_i$、$t_i \geq -r_i$，再最小化 $\sum_i t_i$。一致容許值則引入一個純量 t，要求 $-t \leq r_i \leq t$。若額外限制只有仿射式，兩者都是 LP 表示法。

最佳常數的三種意思

只擬合一個數字，就能看出殘差目標的差別。取合成觀測 $b = (0, 1, 8)$，對三者都預測同一個純量 c。對應設計矩陣是一欄全為一的向量。

對平方損失，微分 $c^2 + (c-1)^2 + (c-8)^2$ 得到 $6c - 18 = 0$，所以 $c = 3$。對絕對損失，離開資料點時，斜率等於低於 c 的觀測數減去高於 c 的觀測數。斜率在中位數由負轉正，因此 $c = 1$ 最小化總和。對最大絕對損失，平衡兩端誤差：$c = 8 - c$，因此 $c = 4$。

其中 c 是擬合常數、b 是固定觀測。在各自最佳解處，平方殘差和為 38、絕對殘差和為 8、最大殘差為 4。它們是不同單位的不同量，直接比較數值大小沒有意義。真正要比較的是回傳預測：平均擬合選三、中位數擬合選一、均勻誤差擬合選四。觀測八大幅拉動平均數，卻不會使中位數超過中間觀測 [1]。

實際任務須先問哪種誤差模式昂貴。儀器的統一容許誤差適合最大誤差；指定高斯量測模型可導向平方誤差；受污染的反應值可促使選用影響力有界的殘差損失。這些選擇都不能只由求解器方便與否決定。

單位與加權誤差

不同單位的量測，沒有指定縮放慣例就不能合理相加。一像素殘差與一秒殘差，不會自動具有可比性。

加權最小平方目標可以寫為

$$(Ax - b)^\top W (Ax - b), \qquad W \succeq 0.$$

W 是固定半正定權重矩陣。若 W 是已知正定雜訊共變異數的反矩陣，目標就有統計意義。若權重代表優先順序，則應明確說明那個意義。

非對角元素可以描述相關量測誤差。把完整共變異數換成對角權重，是模型近似，不只是實作細節。以矩陣因子白化殘差，再擬合時，應保留原本指定的二次形式。

大殘差不會自動代表離群值。它可能揭露錯誤模型、縮放不良的特徵、真正罕見的現象或損壞觀測。在降低某些誤差的影響以前，先看資料生成過程。

最小範數解與正則化

白話說。符合觀測可能仍留下許多參數選擇。參數範數能從中選解，或在擬合與複雜度間取捨。

最小範數問題為

$$\min_x \|x\| \quad \text{subject to} \quad Ax = b.$$

x 是參數向量，A, b 固定。這不同於殘差範數問題：此處要求精確一致，目標則選擇小的參數向量 [1，第 6.2 節]。

等式不一致時，問題不可行。把它換成軟殘差項會改變問題。在欠定且一致的系統中，歐幾里得最小範數解可由適當偽逆表示，但仍須關心數值秩與奇異值。

正則化近似則平衡兩個目標：

$$\min_x \frac{1}{2}\|Ax - b\|_2^2 + \lambda R(x), \qquad \lambda \geq 0.$$

R 是凸懲罰，λ 是非負權重。二分之一讓導數較簡潔。改變這個因子卻保持 λ 不動，會改變取捨。

取 $R(x) = \frac{1}{2}\|x\|_2^2$，就是 ridge 或二次正則化模型 [1，第 6.3 節]。正 λ 使 Hessian $A^\top A + \lambda I$ 正定。取 $R(x) = \|x\|_1$，非光滑幾何會鼓勵部分座標精確為零。

截距通常另行處理，因為懲罰它與懲罰特徵效果的意義不同。特徵單位也會改變參數範數。標準化矩陣欄，或採明確縮放的懲罰，可以讓這項選擇可見。

逐方向推導二次收縮

二次正則化減少量測薄弱方向造成的不穩定。對前面定義的 ridge 目標，令梯度為零得到

$$(A^\top A + \lambda I)x^* = A^\top b.$$

其中 x^* 是最佳解、A, b 是固定設計與觀測、I 是單位矩陣、$\lambda > 0$ 是懲罰權重。令 $A = U\Sigma V^\top$ 為奇異值分解，U, V 包含正交奇異向量，Σ 包含奇異值 σ_i。在方向 v_i 上，係數為

$$v_i^\top x^* = \frac{\sigma_i}{\sigma_i^2 + \lambda}\, u_i^\top b.$$

其中 u_i, v_i 是配對奇異向量。沒有正則化時，非零奇異值會把觀測係數除以 σ_i；很小的值可能放大量測誤差。加入正則化後，分母不會低於 λ。在這個所有座標都受懲罰的模型中，零空間方向的係數為零。

對 $A = \mathrm{diag}(1, 0.1)$ 與 $b = (1, 0.1)$，未正則化解為 $(1, 1)$。取 $\lambda = 0.1$ 後，解變為 $(10/11, 1/11)$。量測薄弱的第二方向被收縮得更多。這同時解釋數值穩定性與引入的偏差：正則化沒有學到第二個真實係數很小，而是在觀測對該方向限制較弱時，施加一種偏好 [1，第 6.3 節]。

限制與懲罰相關，但不同

在 $R(x) \le \rho$ 下最小化 $\|Ax - b\|_2^2$，會直接指定可接受的複雜度預算。在適當條件下，它的拉格朗日乘子可以連到懲罰表示。

這不表示特定數值 λ 事先就對應明顯的 ρ。非唯一解與未作用的限制，可能讓對應更複雜。應選擇最能表達真正需求的表示法。

正則化可能改善條件數，但數值優勢與預測改善是不同的事。條件數良好卻有偏差的模型，仍可能預測很差。若目標是預測，應用有效評估切分選擇調整參數，並把擬合目標與最後評估指標分開。

穩健損失與穩健量測模型

白話說。「穩健」可能指減少異常大殘差的影響，也可能指防範量測運算子的變動。兩者是不同模型。

一個凸殘差損失是 Huber 函數：

$$\phi_\delta(r) = \begin{cases} \frac{1}{2}r^2, & |r| \le \delta, \\ \delta|r| - \frac{1}{2}\delta^2, & |r| > \delta. \end{cases}$$

r 是純量殘差，$\delta > 0$ 是轉換尺度。導數在零附近等於 r，外部大小則飽和於 δ。對仿射殘差加總此損失，就得到凸問題 [1，第 6.1 節]。

轉換尺度具有殘差的單位。把所有觀測乘上一個尺度、卻不改 δ，會改變哪些點減少影響。此損失不能抵抗所有污染；設計矩陣中的高槓桿誤差需要另外處理。

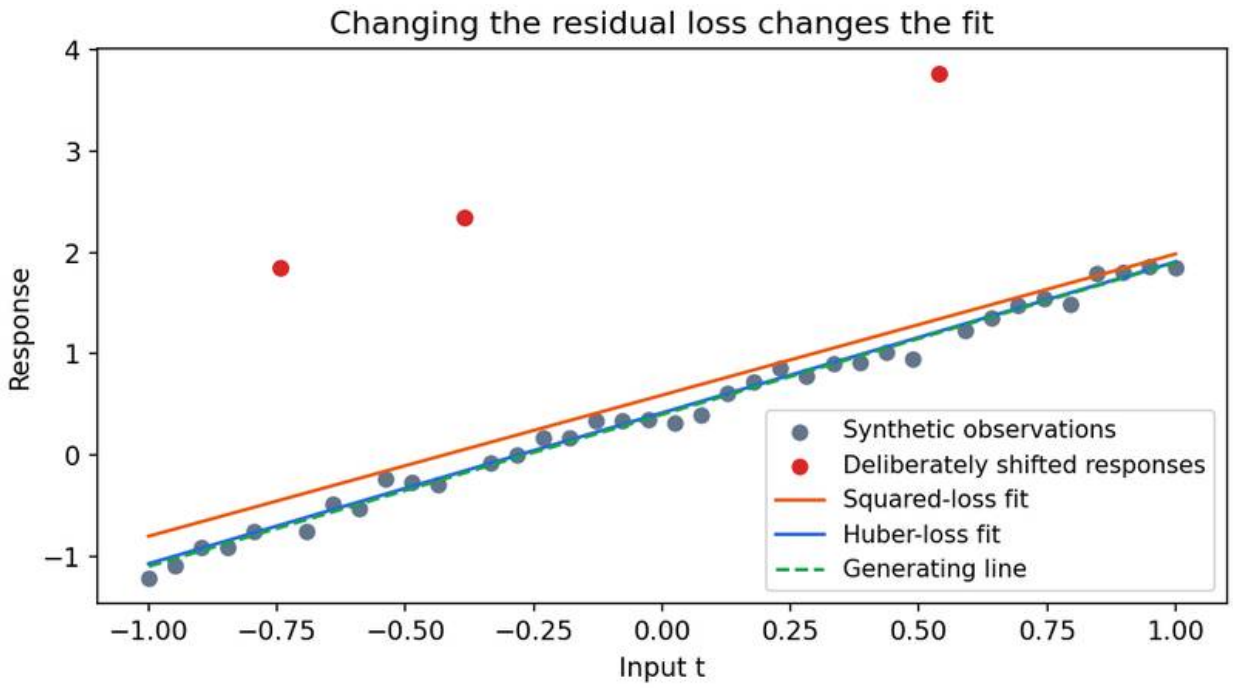


圖 *1* — 原創合成觀測與擬合解。部分反應值刻意向上移動。本示意比較殘差損失，不是基準測試，也不是一般穩健性保證。

程式：用兩種損失擬合直線

資料與偏移在本機生成。平滑最佳化呼叫遵循 SciPy 官方介面 [2]。我們比較解，並驗證穩健目標的梯度，而不宣稱某估計器永遠較好。

```
import numpy as np
from scipy.optimize import minimize

rng = np.random.default_rng(15)
t = np.linspace(-1, 1, 40)
A = np.column_stack([np.ones_like(t), t])
b = 0.4 + 1.5*t + rng.normal(0, 0.08, len(t))
b[[5, 12, 30]] += 2.5
least_squares = np.linalg.lstsq(A, b, rcond=None)[0]
delta = 0.2

def huber_objective(x):
    r = A @ x - b
    loss = np.where(abs(r) <= delta, 0.5*r*r,
                    delta*abs(r) - 0.5*delta**2)
    return loss.sum(), A.T @ np.clip(r, -delta, delta)

fit = minimize(huber_objective, least_squares, jac=True,
               method="BFGS", options={"gtol": 1e-9})
assert np.linalg.norm(huber_objective(fit.x)[1]) < 1e-6
print("squared-loss coefficients:", np.round(least_squares, 4))
print("Huber-loss coefficients:", np.round(fit.x, 4))
print("robust objective:", round(fit.fun, 6))
```

不確定運算子

假設矩陣本身會在指定集合 $\mathcal{A}$ 中變動，最壞情況近似模型是

$$\min_{x} \sup_{A \in \mathcal{A}} \|Ax - b\|.$$

x 是決策，b 是固定目標，$\mathcal{A}$ 是允許的量測矩陣。每個固定矩陣範數目標對 x 為凸，所以逐點上確界仍凸 [1，第 6.4 節]。

凸性不保證容易計算上確界。有限不確定集合可以產生有限個上圖限制。結構化橢球不確定性可能允許錐重寫。模糊描述的「小擾動」，不足以選擇正確對應模型。

期望平方誤差是另一個模型。若 $A = \bar{A} + U$ 且 $\mathbb{E}[U] = 0$，

$$\mathbb{E}\|Ax - b\|_2^2 = \|\bar{A}x - b\|_2^2 + x^{\top} P x, \qquad P = \mathbb{E}[U^{\top} U].$$

$\bar{A}$ 是平均運算子，U 是隨機變動，P 是半正定二階動差矩陣。零均值假設使交叉項消失。此處正則化來自指定的不確定模型，而非任意複雜度偏好。

最壞情況與期望保護回答不同問題。不確定集合不是機率分布；共變異數模型也不保證控制每個可能實現。

具有形狀限制的函數擬合

白話說。我們可以要求擬合函數遞增或凸，同時讓係數最佳化保持凸。

對固定基底函數 $\psi_j(t)$，

$$f(t) = \sum_{j=1}^{n} x_j \psi_j(t).$$

t 是輸入，ψ_j 是已知基底，x_j 是待擬合係數。在固定樣本輸入評估時，預測對 x 仿射，即使基底對 t 非線性。

這個區別很重要：對最佳化變數的凸性，不等於對函數輸入的凸性。多項式迴歸可以是凸係數擬合問題，擬合出的多項式卻既不遞增，也不凸。

在有序節點 $t_1 < \cdots < t_m$，令 $z_i = f(t_i)$ 為擬合值。單調性可要求 $z_{i+1} \geq z_i$。分段線性內插函數的凸性要求

$$\frac{z_{i+1} - z_i}{t_{i+1} - t_i} \leq \frac{z_{i+2} - z_{i+1}}{t_{i+2} - t_{i+1}}.$$

z_i 是變數，節點間距是固定正數。這些斜率順序不等式對擬合值為線性。與取樣任意彈性函數導數不同，它們會保證這個特定內插函數在整段節點區間的形狀。

程式：以 LP 做單調絕對誤差擬合

我們擬合五個有序量測，輔助變數表示絕對殘差上界。SciPy 的 LP 邊界全部明確指定 [3]，包括符號不受限的擬合值。

```
import numpy as np
from scipy.optimize import linprog

observed = np.array([0., 2., 1., 3., 4.])
n = len(observed)
I = np.eye(n)
order = np.zeros((n-1, n))
for i in range(n-1):
    order[i, i], order[i, i+1] = 1, -1
G = np.vstack([np.hstack([I, -I]),
               np.hstack([-I, -I]),
               np.hstack([order, np.zeros((n-1, n))])])
h = np.r_[observed, -observed, np.zeros(n-1)]
fit = linprog(np.r_[np.zeros(n), np.ones(n)], A_ub=G, b_ub=h,
```

```
              bounds=[(None, None)]*n + [(0, None)]*n,
              method="highs")
assert fit.success
z = fit.x[:n]
assert np.all(np.diff(z) >= -1e-9)
assert np.isclose(abs(z-observed).sum(), fit.fun)
print("monotone fitted values:", z)
print("total absolute residual:", fit.fun)
```

絕對誤差解可能不唯一。程式驗證性質與目標，不要求某個特定平台值。加入嚴格凸的次要目標可以選定一解，但那會是另一個指定問題。

觀測區間外的限制需要小心。分段線性內插提供明確區間內函數；外插則是額外規則。形狀限制可以增加合理性，但若真實過程違反它們，也會引入偏差。

概似與凸參數化

白話說。概似模型可以產生凸目標，但是否為凸，取決於所選參數化與定義域。

對自然參數為 θ 的分布，獨立觀測 y_i 的模型為

$$p(y \mid \theta) = \exp(\theta^{\top} T(y) - A(\theta) + c(y)).$$

$T(y)$ 是充分統計量向量，$A(\theta)$ 是對數配分函數，$c(y)$ 是與參數無關的基底項。指數族模型的 A 在自然定義域上為凸。

忽略與 θ 無關的項，負對數概似為

$$\ell(\theta) = N A(\theta) - \theta^{\top} \sum_{i=1}^{N} T(y_i).$$

N 是觀測數。此式對自然參數為凸。任意非線性參數轉換，不一定保留凸性 [1，第 7.1 節]。

對成功機率 $p \in (0,1)$ 的 Bernoulli 觀測，N 次有 k 次成功，負對數概似為 $-k \log p - (N-k)\log(1-p)$。若兩種結果都有出現，內部最佳點為 $p = k/N$。

若所有觀測都成功或都失敗，開放定義域的最佳值可能只在邊界逼近。允許邊界分布或加入先驗，會改變存在性問題。程式若裁切機率，應把裁切記錄成定義域選擇。

最大後驗估計會加入負對數先驗。若負對數先驗在所選座標為凸，整體凸性就保留。「Bayesian」本身不保證凸最佳化，點估計也不代表完整後驗不確定性。

非參數機率

對離散結果，在單純形上估計機率向量 q：

$$q_j \geq 0, \qquad \sum_{j} q_j = 1.$$

q_j 是結果 j 的機率。給定計數 n_j，負對數概似為 $-\sum_j n_j \log q_j$，正計數搭配零機率時，要採適當延伸值慣例。

已知動差形成仿射限制，例如固定結果特徵 a_j 與已知平均值 μ 的 $\sum_j q_j a_j = \mu$。最大化熵，等價於最小化 $\sum_j q_j \log q_j$，可在這些限制下選擇分布 [1，第 7.2 節]。

動差限制應來自與資料相同的母體、使用相同單位。不一致的經驗限制可能使單純形模型不可行。放軟限制，是決定接受多少不確定性的模型選擇，不是修補代數上的麻煩。

以線性規劃設計偵測器

白話說。看見離散證據後的決策，可以直接最佳化。允許隨機決策，會讓可行集合凸化。

假設 p_{kj} 是假說 j 下觀測 k 的已知機率。令 T_{ik} 表示看見 k 後決定假說 i 的機率。

$$T_{ik} \geq 0, \qquad \sum_i T_{ik} = 1, \qquad D_{ij} = \sum_k T_{ik} p_{kj}.$$

D_{ij} 是 j 為真時決定 i 的機率。T 是決策變數，條件機率 p_{kj} 固定。D 的每個元素對 T 都是線性的。

加權錯誤成本、正確偵測下界與錯誤決策上界，都能寫成線性目標和限制 [1，第 7.3 節]。假說先驗可以加權整體期望錯誤，但不應悄悄拿它代替條件錯誤保證。

二元假說的適當最佳偵測器表示法，會產生概似比閾值規則。邊界觀測可能需要隨機化，才能符合精確錯誤預算。確定性規則是受限子集；強制使用它們會引入離散選擇。

最佳偵測器的有效性，取決於條件分布。分布偏移、估計概似與校準不確定性，都應納入判斷。指定機率表下的數值最佳性，不會驗證機率表本身。

偵測器算例：隨機化為什麼有用

偽陽性預算可能落在兩個確定性規則之間。假設觀測為 a 或 b。假設 H_0 下的機率為 $(0.8, 0.2)$，H_1 下為 $(0.2, 0.8)$。這是合成條件機率表，不是量測頻率。

令 $t_a, t_b \in [0, 1]$ 為各觀測後判定 H_1 的機率。在偽陽性預算 0.1 下最大化真陽性機率，形成 LP：

$$\begin{aligned} \text{maximize}_{t_a, t_b} \quad & 0.2 t_a + 0.8 t_b \\ \text{subject to} \quad & 0.8 t_a + 0.2 t_b \leq 0.1, \\ & 0 \leq t_a, t_b \leq 1. \end{aligned}$$

其中目標為 H_1 下的偵測機率，第一限制為 H_0 下的錯誤機率。在 b 花一單位錯誤預算，可得四單位偵測機率；在 a 花同樣預算，只得四分之一單位。因此把預算給 b：$t_a = 0$、$t_b = 1/2$，真陽性機率為 0.4。

在 b 做確定性判定會消耗 0.2，超過預算。在這個離散觀測集合中，任何非平凡的確定性規則都違反預算，因此唯一可行確定性規則是永遠選 H_0，完全不偵測。隨機化填入確定性規則之間的凸空間，達到需要的操作點 [1，第 7.3 節]。這是在已知觀測後隨機決策，不是把未知機率當成決策變數。

動差與指數風險上界

白話說。有時我們知道動差，卻不知道整個分布。明確列出假設後，上界可以保守控制尾端事件。

對均值 μ、有限變異數 σ^2 的純量隨機變數 Y，Chebyshev 不等式為

$$\Pr(|Y-\mu| \ge a) \le \frac{\sigma^2}{a^2}, \qquad a > 0.$$

a 是偏離閾值。這個上界只使用前兩階動差，可能很鬆。大於一的上界可以換成一；它不是估計的事件機率 [1，第 7.4 節]。

對閾值 b，指數或 Chernoff 上界使用

$$\Pr(Y \ge b) \le \inf_{s>0} \exp(-sb)\, \mathbb{E}[\exp(sY)].$$

s 是正輔助參數，使用處的期望必須有限。最佳化其對數，得到 $\inf_{s>0}\{\log \mathbb{E}[\exp(sY)] - sb\}$。

對數動差生成函數在定義域上對 s 為凸。但這不會自動使所有設計與風險的聯合問題，對全部變數為凸。應檢查隨機變數如何依賴決策，以及保留哪些分布假設。

尾端上界可以支援保守設計限制，卻不能代替檢查動差估計是否可靠。厚尾、相依性與有限樣本不確定性，可能讓假定的動差生成函數不適合。

選擇下一批量測

白話說。估計品質不只取決於怎麼擬合，也取決於量測什麼。凸實驗設計先配置連續量測資源，再處理離散取整。

對候選量測向量 $a_j \in \mathbb{R}^n$、獨立單位變異數雜訊與正規化權重 w_j，定義

$$M(w) = \sum_j w_j a_j a_j^{\top}, \qquad w_j \ge 0, \quad \sum_j w_j = 1.$$

M 是每單位正規化量測資源的資訊矩陣。矩陣正定、量測模型正確且計入總資源時，相應線性估計器的共變異數會與其反矩陣成比例。

D-optimal 設計最小化 $-\log\det M$，針對不確定性體積。A-optimal 設計最小化 $\operatorname{tr}(M^{-1})$，針對總座標變異數。E-optimal 設計最大化 M 的最小特徵值，針對最弱量測方向 [1，第 7.5 節]。

它們是不同的純量摘要。單位與座標轉換對它們解釋的影響可能不同。小的總變異數仍可能藏著量測不良的方向；小不確定體積，也不保證每個座標的界。

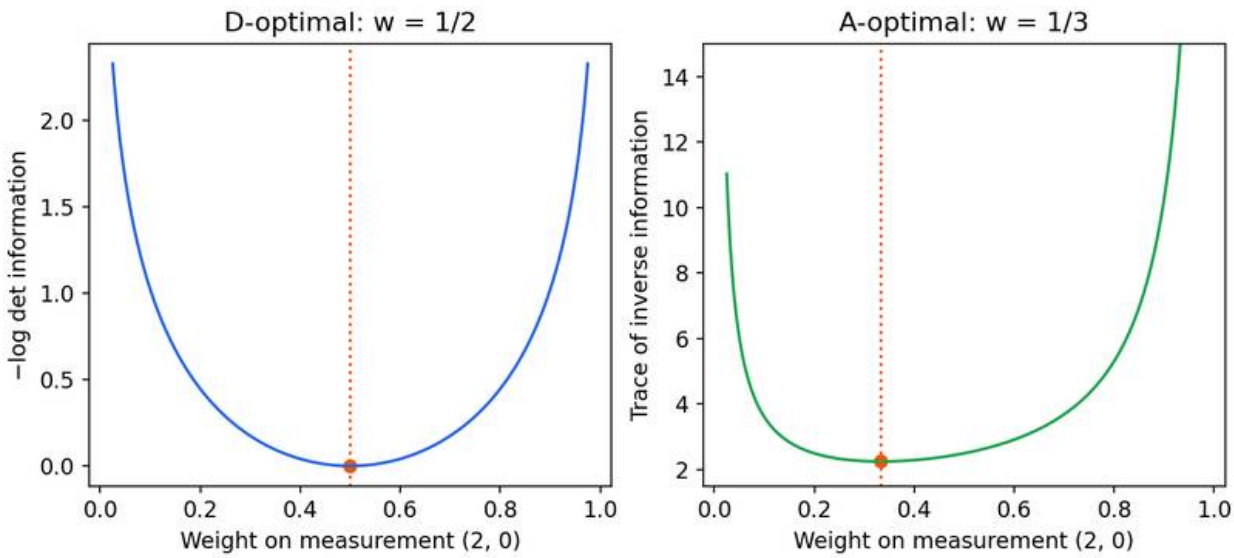


圖 2 — 原創兩方向解析設計。*D-optimal* 與 *A-optimal* 配置不同，因為體積和總座標變異數是不同目標。曲線假設獨立單位變異數雜訊，以及兩方向皆有正資源。

程式：兩種設計準則與精確檢查

候選向量為 $(2,0)$ 與 $(0,1)$，第一個權重 w，第二個 $1-w$。資訊矩陣為對角矩陣 $M = \mathrm{diag}(4w, 1-w)$。任一方向未量測時，兩種準則都發散。

```
import numpy as np
from scipy.optimize import minimize_scalar

def d_loss(w):
    return -np.log(4*w) - np.log(1-w)

def a_loss(w):
    return 1/(4*w) + 1/(1-w)

d = minimize_scalar(d_loss, bounds=(1e-6, 1-1e-6), method="bounded")
a = minimize_scalar(a_loss, bounds=(1e-6, 1-1e-6), method="bounded")
assert np.isclose(d.x, 0.5, atol=1e-5)
assert np.isclose(a.x, 1/3, atol=1e-5)
print("D-optimal first weight:", round(d.x, 6))
print("A-optimal first weight:", round(a.x, 6))
```

實驗總數固定時，實際次數是整數。連續權重是鬆弛，或隨機取樣策略。取整可能破壞預算，或讓某方向完全未量測；轉成排程後應重新檢查矩陣。

若候選向量無法張成參數空間，任何權重都不能使資訊矩陣正定。加入先驗矩陣可以正則化設計，但意義變成先驗加量測資訊。

推導兩種設計準則為什麼配置不同

兩方向實驗在執行數值程式前就可解出。候選向量為 $(2,0)$ 與 $(0,1)$，權重為 $w, 1-w$，資訊矩陣為 $M = \mathrm{diag}(4w, 1-w)$。令 $0 < w < 1$，確保兩方向都有量測。

D-optimal 目標與導數為

$$f_D(w) = -\log(4w) - \log(1-w), \qquad f_D'(w) = -\frac{1}{w} + \frac{1}{1-w}.$$

其中 f_D 是負對數行列式準則。駐點條件給出 $w = 1/2$。強度因子四只讓目標改變一個常數，不影響這個體積準則的配置。

對 A-optimal 設計，

$$f_A(w) = \frac{1}{4w} + \frac{1}{1-w}, \qquad f_A'(w) = -\frac{1}{4w^2} + \frac{1}{(1-w)^2}.$$

其中 f_A 是反資訊矩陣跡準則。分母為正，因此駐點方程化為 $1 - w = 2w$，得到 $w = 1/3$。兩個純量目標的二階導數都為正，而且在端點發散，因此這些駐點為唯一全域最小解 [1，第 7.5 節]。

解讀是：第一種量測本來就更強，因此最小化總變異數時，會把更多資源送往較弱的第二方向。最小化不確定性體積則平衡方向資訊的乘積，因此給相同資源。若量測成本或雜訊變異數不同，須先在資訊矩陣與預算中納入改變，再重新推導。沒有哪種配置永遠最好；每種都回答自己選定的不確定性問題。

擬合與設計的檢核

先從必須準確的應用量開始，決定它是預測、參數分量、最壞殘差或分布機率。選擇數值損失以前，先寫出表示法與單位。

對每個限制，問它是物理必要、統計估計，還是為計算方便引入。這些角色代表不同的不可行性處理方式。物理限制可能要求修正設計，經驗動差可能需要不確定性，冗餘代數可能需要移除。

擬合後驗證定義域、形狀限制與求解器殘差，再以適當獨立程序評估泛化或不確定性。訓練目標最佳解與可靠未來預測，是不同宣稱。

設計時檢查資訊譜、預算、雜訊假設與排程轉換。連同鬆弛目標一起回報最後可執行排程，讓最重要的近似保持可見。

不確定性橢球真正說明什麼

資訊矩陣的反矩陣，只有在相應量測與估計器假設下才能解讀為共變異數。若要轉成具有指定涵蓋機率的區域，還需要分布論證與正確閾值。只由矩陣畫出橢圓，不會建立信賴水準。

對正定共變異數 C，橢球可以寫成 $(x-\hat{x})^{\top} C^{-1} (x-\hat{x}) \le c$。$\hat{x}$ 是中心，$c > 0$ 是閾值。矩陣指定形狀，閾值指定大小；涵蓋率還取決於估計誤差的實際分布。

先驗共變異數、抽樣共變異數與數值 Hessian 反矩陣，不一定具有相同解釋。特定模型下可能一致，但交換使用前應檢查模型。估計器的最佳化容許誤差與抽樣不確定性，也具有不同單位與來源。

重複量測時，獨立等變異數假設可支持簡單的資訊相加。相關重複可能提供比次數所暗示更少的新資訊。忽略相關性的設計，可能把資源自信地配置到冗餘量測。

這也影響停止精度。求解精度遠低於輸入係數的不確定性，可能實際幫助有限；看似微小的硬物理限制違反，卻仍可能不可接受。應依應用尺度設定最佳化容許誤差，而非只看浮點數能提供幾位數。

最後，不確定性視覺化應說明畫的是解析模型、估計共變異數，或重複合成實驗。本章圖形展示示意擬合與目標曲線，沒有呈現經驗校準的涵蓋區域，也沒有宣稱特定信賴機率。

現代觀點

白話說。現今研究把結構化凸估計、大規模一階計算與學得的凸元件結合。凸性保證只屬於明確寫出的變數與定義域。

Pokutta 的 2024 年綜述 [4] 以凸可行集合的線性最小化、收斂保證與實用變體，組織 Frank–Wolfe 方法。單純形限制的實驗設計，是這個觀點的自然場合：極端點搜尋可以避免完整投影。但目標仍須滿足要求的平滑性與正定定義域控制。

Schaller、Bemporad 與 Boyd 的 2025 年手稿 [5] 討論學習參數化凸函數，再用於後續最佳化。它以有紀律的曲率規則連結擬合：對合法固定參數，學得元件可以保持對決策變數的凸性。訓練元件是另一個問題，本身未必凸。

兩來源涵蓋互補問題：如何在結構化可行集合中移動，以及如何建立能放入集合內使用的函數。它們都不會代替不確定性建模、保留資料評估或設計的物理可行性。

目前挑戰包括模型失配下的可靠不確定性、資源限制下有資訊的量測、樣本區間外仍合適的形狀限制，以及觀測逐步到來時的可擴充更新。應區分可證明凸的估計器與完整驗證的統計流程。

重點整理

- 殘差範數、參數範數與評估指標回答不同問題。
- 正則化權重依賴目標正規化與特徵單位。
- 穩健殘差損失與不確定運算子模型防範不同誤差。
- 固定非線性基底仍能產生凸係數擬合。
- 概似凸性依賴參數化與定義域。
- 離散證據的隨機偵測器可以寫成 LP。
- 實驗設計準則不同，連續權重也需要執行方案。

練習

1. 將最大絕對殘差擬合寫成 LP，包含符號不受限的參數。

提示

使用一個非負殘差上界。

2. 把觀測與預測乘以十。求 Huber 轉換尺度與二次正則化權重應如何改變，才能保留指定等價目標。

提示

寫出縮放後目標；答案取決於參數與懲罰單位是否也縮放。

3. 在五節點單調擬合例子加入凸性限制，使用不相等的正節點間距。

提示

排序相鄰斜率。

4. 推導 Bernoulli 最大概似估計，並解釋開放定義域的邊界情況。

提示

令概似導數為零，再看端點。

5. 建立具有明確偽陽性預算的雙假說隨機化偵測器 LP。

提示

使用每個觀測的決策機率，把條件錯誤寫成線性式。

6. 推導程式中兩方向 D-optimal 與 A-optimal 權重，再說明小量測預算取整後為何需要重新檢查。

提示

對純量準則微分；檢查每個取整方向是否有量測。

參考文獻

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. **閱讀：第 6 章第 6.1–6.5 節；第 7 章第 7.1–7.5 節**。 官方書頁，免費 PDF。
2. SciPy developers, “scipy.optimize.minimize” and “scipy.optimize.minimize_scalar,” *SciPy API documentation*, accessed 2026. 多變數介面，純量介面。
3. SciPy developers, “scipy.optimize.linprog,” *SciPy API documentation*, accessed 2026. 官方文件。
4. Sebastian Pokutta, “The Frank-Wolfe Algorithm: A Short Introduction,” *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 126, 2024. 開放綜述。
5. Michael Schaller, Alberto Bemporad and Stephen Boyd, “Learning Parametric Convex Functions,” manuscript, 2025. 官方手稿頁。

OPT 06 ・梯度下降、牛頓步驟與數值可靠性

推導下降方向與線搜尋，比較梯度和牛頓方法，解讀收斂假設，並實作穩定的凸邏輯斯求解器。

網站版本: https://blog.shyandram.dev/zh-tw/learn/optimization/opt-06-gradient-newton/

你將學到

- 區分搜尋方向與沿方向選擇的步長。
- 在 Lipschitz 梯度假設下推導安全梯度步驟。
- 實作具有定義域與迭代保護的 Armijo 回溯。
- 解釋條件數、依範數而異的最陡下降，以及牛頓曲率。
- 解讀牛頓減量與快速局部收斂背後的假設。
- 以 CPU 例子驗證導數、線性求解與停止證據。

先看全貌

凸性告訴我們哪些答案為全域最佳；演算法則告訴我們，如何利用可計算資訊逐步靠近答案。兩者互補：凸問題仍可能尺度不良、昂貴或數值敏感。

Boyd 與 Vandenberghe 的第 9 章 [1] 發展無限制下降方法、梯度下降、最陡下降、牛頓方法、自協調性與實作。本章依這個順序前進，同時保留每步成本與停止判準的意義。

> **用最簡單的話說**
>
> 你正在下山。斜坡箭頭指出下降方向；描繪山坡彎曲的地圖，可以建議更聰明的路線。無論使用哪個方法，都還要檢查下一步安全落地，而且確實降低高度。

例子求解確定性的凸目標。不能把保證悄悄轉移到神經網路的非凸訓練目標、含雜訊導數或非光滑懲罰。實用想法可能能轉移，但證明假設需要重新檢查。

我們先用有精確解的二次函數，再從頭實作平滑正則化邏輯斯目標。兩者使用本機生成資料，在 CPU 執行，除 NumPy 外不需要求解器套件。

一個方向與一段距離

白話說。下降迭代先選往哪裡指，再選走多遠。太早把兩者混在一起，會隱藏步驟為何安全。

迭代寫成

$$x_{k+1} = x_k + t_k d_k.$$

x_k 是目前決策向量，d_k 是搜尋方向，$t_k > 0$ 是步長。對可微目標 f，嚴格下降方向滿足 $\nabla f(x_k)^\top d_k < 0$。

此內積是零步長處的方向導數。可微性表示，在目前點周圍的開放定義域，足夠小的正步長會降低目標，並保留定義域。大步長仍可能提高目標或離開定義域。

方向可能依賴梯度、Hessian、度量或其他結構；線搜尋評估候選距離。只計算接受的迭代，會隱藏許多拒絕的目標評估。

若可微凸函數在開凸定義域的梯度為零，支撐平面不等式就能證明全域最小化解。有約束時，零無限制梯度通常不是正確最佳性條件；下一章會用受限制系統。

精確與非精確線搜尋

精確線搜尋會在允許的正距離上最小化 $f(x + td)$。這個輔助一維問題可能昂貴，也可能不必要。

非精確搜尋接受足夠下降，不精確求解線問題。它也應保留目標定義域。對數項的引數無效時，候選點只是未通過定義域檢查，不是提供了目標證據。

浮點數又帶來另一個區別：數學上下降的步驟，在機器精度下可能與目前值無法區分。不斷縮小到比較偶然通過，不是穩健停止規則。

梯度下降與假設

白話說。梯度下降往梯度反方向走。曲率上界告訴我們，固定步長必須多小。

更新為

$$x_{k+1} = x_k - \eta \nabla f(x_k).$$

$\eta > 0$ 是步長。假設相關區域的梯度具有 Lipschitz 常數 L：

$$\|\nabla f(y) - \nabla f(x)\|_2 \le L\|y - x\|_2.$$

x, y 是連接線段留在該區域的點。L 衡量梯度改變多快，不是正則化係數，也不是損失值。

平滑性不等式為

$$f(x + d) \le f(x) + \nabla f(x)^\top d + \frac{L}{2}\|d\|_2^2.$$

d 是嘗試的位移。代入 $d = -\nabla f(x)/L$ 得到

$$f\left(x - \frac{1}{L}\nabla f(x)\right) \le f(x) - \frac{1}{2L}\|\nabla f(x)\|_2^2.$$

因此，假設與定義域要求成立時，$1/L$ 步長會下降。對二次可微目標，Hessian 最大特徵值上界可以建立此 L [1，第 9.3 節]。

只有凸性與平滑性時，標準固定步長分析通常給次線性目標收斂。強凸性加入正的曲率下界，讓線性幾何收斂成為可能。這些字描述數學速率，不保證每個實作都很快。

從單步下降到收斂速率

下降證明目標改善；收斂速率量化達到指定誤差需要多少工作。設 $\mathbb{R}^n$ 上的可微凸函數具有 L-Lipschitz 梯度，而且最小解 x^* 存在。使用 $x^+ = x - \nabla f(x)/L$。梯度支撐下界給出

$$f(x) - f(x^*) \le \nabla f(x)^\top (x - x^*).$$

其中 x 是目前點、x^+ 是下一迭代點、x^* 是任意最小解。展開更新後的平方距離：

$$\|x^+ - x^*\|_2^2 = \|x - x^*\|_2^2 - \frac{2}{L}\nabla f(x)^\top (x - x^*) + \frac{1}{L^2}\|\nabla f(x)\|_2^2.$$

所有符號保留上述定義。把第一個不等式與前面的平滑下降界結合，得到

$$f(x^+) - f(x^*) \le \frac{L}{2}\left(\|x - x^*\|_2^2 - \|x^+ - x^*\|_2^2\right).$$

對前 k 次更新加總，中間距離逐項抵消。因為目標值下降，最後誤差不大於這 k 個誤差的平均：

$$f(x_k) - f(x^*) \le \frac{L\|x_0 - x^*\|_2^2}{2k}, \qquad k \ge 1.$$

其中 x_0 是起點、x_k 是 k 次更新後的點。這就是 $O(1/k)$ 目標速率的具體意思：依此界把保證誤差減半，通常要把迭代次數加倍。它是這些假設下的最壞情況保證，不是所有實測軌跡都如此 [1]。

強凸性與停止上界

若 f 是 μ 強凸，支撐模型加入 $\frac{\mu}{2}\|y - x\|_2^2$。最小化此下模型得到

$$f(x) - f^* \le \frac{\|\nabla f(x)\|_2^2}{2\mu}.$$

f^* 是全域最小值，$\mu > 0$ 是有效強凸常數。小梯度因此給出單位明確的目標間隙上界。

沒有驗證正 μ 時，同一公式不是憑證。很平坦的目標可能同時有小梯度與很大的解距離；小參數更新也可能只是步長過小。

不要只因數次印出的目標值看似相同就停止。應檢查適當殘差、模型尺度、進度趨勢，以及迭代或時間限制。

條件數改變路徑

白話說。拉長的碗有陡方向與平方向。對陡方向安全的步長，可能讓平方向進展非常慢。

正定二次函數為

$$f(x) = \frac{1}{2}x^\top Q x + c^\top x, \qquad \nabla f(x) = Qx + c.$$

Q 對稱正定，c 固定。Hessian 就是 Q，最小和最大特徵值給 μ 與 L。

比例 $\kappa = L/\mu$ 描述歐幾里得曲率條件數。比例很大時，單一純量步長要照顧非常不同的方向尺度。因此，座標單位會大幅改變梯度下降行為。

標準化變數或選適當度量可以改善路徑，但限制、係數與停止範數都要一致轉換。不能用未轉換的參數距離閾值，公平評估縮放後演算法。

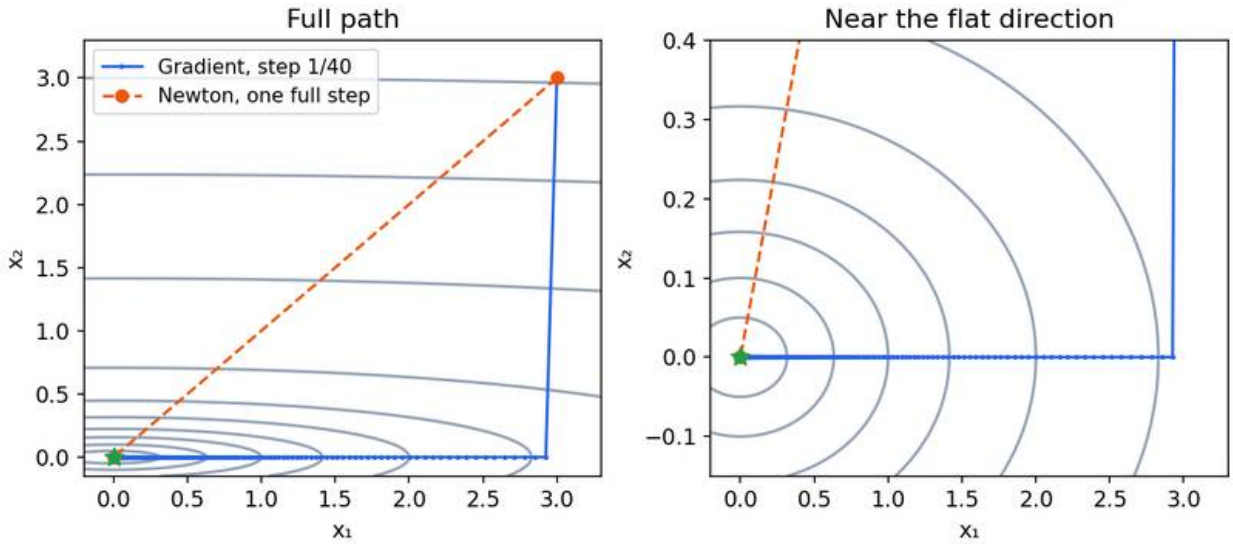


圖 1 — 原創解析二次函數 $f(x) = (x_1^2 + 40x_2^2)/2$。梯度路徑用 $1/40$ 步長；牛頓在一個完整步驟達到精確二次最小值。本例隔離條件數影響，沒有比較一般問題的總執行時間。

程式：比較固定梯度步與牛頓步

目標在零最小。我們比較目標下降，並驗證牛頓線性系統，不使用明確反矩陣。

```
import numpy as np

Q = np.diag([1., 40.])
x = np.array([3., 3.])
start = x.copy()
values = [0.5*x @ Q @ x]
for _ in range(200):
    x = x - (Q @ x)/40
    values.append(0.5*x @ Q @ x)
direction = np.linalg.solve(Q, -(Q @ start))
newton = start + direction
assert np.all(np.diff(values) <= 1e-12)
assert np.allclose(Q @ direction + Q @ start, 0)
assert np.linalg.norm(newton) < 1e-12
print("gradient objective after 200 steps:", round(values[-1], 8))
print("Newton objective after one step:", 0.5*newton @ Q @ newton)
```

一步牛頓解，是精確正定二次函數與準確線性求解的特殊結果。一般平滑函數移動時曲率會變，完整一步可能失敗。

逐特徵向量讀懂二次路徑

二次函數能精確呈現步長影響。對曲率為 Q 的正定二次函數，令 $e_k = x_k - x^*$ 為誤差。梯度下降給出 $e_{k+1} = (I - \eta Q)e_k$。在 Q 的特徵值為 q_i 的特徵向量方向上，誤差係數乘以 $1 - \eta q_i$。

其中 η 是固定步長、I 是單位矩陣、$q_i > 0$ 是方向曲率。所有係數收縮的充分必要條件為 $0 < \eta < 2/L$，其中 L 是最大特徵值。若 $\eta > 2/L$，即使目標凸，最陡方向的誤差量值仍會增大。

對本章的 $Q = \mathrm{diag}(1, 40)$ 與 $\eta = 1/40$，陡座標在一次更新後消失，平坦座標則每次乘以 $39/40$。從 $(3, 3)$ 開始，k 次更新後第一座標誤差為 $3(39/40)^k$。緩慢路徑是方向曲率的結果，而非隨機數值失敗。

對最小特徵值為 μ 的二次函數，取 $\eta = 2/(L + \mu)$ 可平衡極端收縮因子的量值，其最壞值為 $(L - \mu)/(L + \mu)$。相較保守步長，這能改善上界，但需要已知頻譜端點，不能取代任意非二次函數的線搜尋 [1]。

Armijo 回溯

白話說。從候選距離開始，縮小到相對局部斜率有足夠下降。

足夠下降條件為

$$f(x + td) \le f(x) + \alpha t \nabla f(x)^\top d.$$

d 是下降方向，$t > 0$ 是測試距離，$\alpha \in (0, 1)$ 是固定下降比例。回溯反覆把 t 乘上 $\beta \in (0, 1)$，直到定義域與下降測試通過 [1，第 9.2 節]。

牛頓方法常選 $\alpha < 1/2$，讓標準局部分析下，正則最佳點附近能接受完整步。這是考慮假設的選擇，不是通用最佳參數。

實作需要最多試驗次數、最小有意義步長、有限值檢查，以及清楚失敗回報。否則無效方向或錯誤梯度，可能導致無限縮小迴圈。

拒絕步驟告訴我們什麼

拒絕可能表示初始距離太大、遠離解時曲率模型不準、跨越定義域或導數錯誤。單次拒絕本身不能分辨原因。

先檢查方向導數。若非負，一般下降回溯就沒有原本的正當性。可能需要修改方向，也可能候選點已滿足最佳性測試。

最佳點附近反覆拒絕，可能來自浮點抵消。要求低於可表示精度的改善以前，應比較目標與導數尺度。停止時應說明符合哪個數值條件。

最陡下降依賴範數

白話說。「最陡」需要定義一單位移動。不同範數會選出不同方向。

正規化最陡方向求解

$$\min_{\|d\|\le 1} \nabla f(x)^\top d.$$

d 是方向，$\|\cdot\|$ 是選定移動範數，x 是目前點。歐幾里得範數下，非零梯度給方向 $-\nabla f(x)/\|\nabla f(x)\|_2$。

對正定度量矩陣 P，定義 $\|d\|_P = \sqrt{d^\top P d}$。相應未正規化下降方向是

$$d = -P^{-1}\nabla f(x).$$

P 決定方向成本。方向導數為 $-\nabla f(x)^\top P^{-1}\nabla f(x)$，梯度非零時嚴格為負。

選擇 P 類似預條件化：納入已知尺度或曲率，不一定建立真正 Hessian。好處取決於度量系統求解多便宜，以及它多能代表問題。

採座標型移動範數時，最陡方向可能選單一座標，或以不同方式分配移動。書中 [1，第 9.4 節] 說明，最陡下降是一族方法，不是單一與座標無關的程序。

牛頓的局部二次模型

白話說。牛頓以二次函數近似目標，再求該近似的最小化解。

目前點的模型為

$$m(d) = f(x) + g^\top d + \frac{1}{2} d^\top H d, \qquad g = \nabla f(x),\ H = \nabla^2 f(x).$$

d 是位移，g 是梯度，H 是 Hessian。若 H 正定，模型唯一最小化位移滿足

$$Hd = -g.$$

這就是牛頓方向。以線性求解或分解計算，不要建立反矩陣。正定性保證 $g \neq 0$ 時有下降方向。

凸性只給半正定 Hessian，仍可能奇異。因此，完整牛頓步不會自動有定義。正則化 Hessian、受限子空間或其他方向可能合適，但每項都會改變指定數值程序。

牛頓減量

定義

$$\lambda_N(x)^2 = g^\top H^{-1} g = -g^\top d.$$

λ_N 是牛頓減量（Newton decrement），假設 H 正定。二次模型預測，最佳步驟會下降 $\lambda_N^2/2$。

對精確二次函數，預測等於目標間隙。對一般函數，它是局部模型量；嚴格間隙解釋需要額外假設。不要把預測下降標成無條件全域誤差憑證。

導數與求解精確時，在可逆仿射座標轉換下，方向與減量會一致轉換。這是牛頓與歐幾里得梯度下降不同的一個原因，但浮點條件數與實作仍重要。

局部收斂有條件

在適當平滑性、最小化點附近非奇異正定 Hessian，以及合適線搜尋下，牛頓最終會接受完整步，並可二次收斂 [1，第 9.5 節]。

二次收斂表示，在足夠小鄰域中，誤差約以先前誤差平方縮小。它不表示任何初始點的前幾步都如此。

梯度、Hessian 與線性求解誤差可能限制速率。遠離解時，保護措施可能必要；接近解時，要求低於導數準確度的殘差，可能浪費計算或引發誤導的失敗。

牛頓法為什麼尊重座標轉換

牛頓法利用曲率調整方向，所以改變變數單位不會改變其理想的實際步驟。令 $x = Tz + b$，其中 T 可逆、b 為固定位移，定義 $\tilde{f}(z) = f(Tz + b)$。在對應點，鏈鎖律給出

$$\tilde{g} = T^{\top} g, \qquad \widetilde{H} = T^{\top} H T.$$

其中 g, H 是 x 座標下的梯度與正定 Hessian，波浪號表示 z 座標下的對應導數。轉換後的牛頓方程為

$$T^{\top} H T\, \Delta z = -T^{\top} g \quad \Longrightarrow \quad H(T\Delta z) = -g.$$

其中 Δz 是新座標的位移。線性解唯一，因此 $T\Delta z = \Delta x$，等於原牛頓位移。預測下降量也不變：$\tilde{g}^{\top} \widetilde{H}^{-1} \tilde{g} = g^{\top} H^{-1} g$。

普通梯度下降的轉換步驟映回後，卻是 $-\eta T T^{\top} g$，通常不同於 $-\eta g$。這連起座標縮放與最陡下降中的度量。牛頓不變性假設精確導數與求解；條件數不良的座標轉換仍可能傷害浮點實作 [1，第 9.5 節]。

自協調性與受控曲率

白話說。某些凸函數透過本身局部曲率，控制曲率改變速度，提供分析牛頓步的特殊途徑。

標準一維自協調函數滿足

$$|f'''(t)| \le 2\big(f''(t)\big)^{3/2}.$$

t 是純量輸入，撇號表示導數，二階導數非負。多維時，沿定義域中的直線套用此條件 [1，第 9.6 節]。

$t > 0$ 上的對數障礙 $-\log t$ 滿足這個純量關係，連結無限制牛頓分析與下一章內點方法。

自協調性比一般平滑凸性更具體。它允許局部範數控制變動，不必只靠單一全域歐幾里得 Hessian-Lipschitz 常數。理論包含精確的減量區域與複雜度陳述；它們需要正確正規化與障礙假設。

包含對數的函數，不會自動成為每種限制表示的有效障礙。定義域、邊界行為與縮放都要符合問題。即使兩者都有對數，也應區分平滑概似項與限制障礙。

穩定的凸邏輯斯求解器

白話說。完整實作應穩定評估目標、驗證導數，並以最佳性量停止。

對固定列 $a_i^\top$ 與二元結果 $y_i \in \{0,1\}$，使用

$$f(w) = \frac{1}{N}\sum_{i=1}^{N}[\log(1+\exp(a_i^\top w)) - y_i a_i^\top w] + \frac{\rho}{2}\|w\|_2^2.$$

w 是參數向量，N 是觀測數，$\rho > 0$ 是正則化係數。本例正則化每個座標。固定設計邏輯斯概似，遵循 [1，第 7 章] 的凸估計推理。

令 $p_i = 1/(1+\exp(-a_i^\top w))$，導數為

$$g = \frac{1}{N}A^\top(p-y) + \rho w, \qquad H = \frac{1}{N}A^\top \operatorname{diag}(p_i(1-p_i))A + \rho I.$$

A 堆疊量測列，p, y 是觀測向量，I 是單位矩陣。正 ρI 提供有效全域強凸常數 $\mu = \rho$。

使用穩定 softplus 評估，不要先算可能溢位的指數再取對數。下列合成分數大小受控，允許直接 sigmoid；對任意極端分數，也應使用數值穩定的 sigmoid 實作。

推導邏輯斯曲率界限

知道曲率常數，能讓穩定目標更容易最佳化。對單一分數 $s = a_i^\top w$，$\log(1+e^s)$ 的導數為 p_i，二階導數為 $p_i(1-p_i)$。因此扣除線性項 $y_i s$ 後，梯度為 $(p_i - y_i)a_i$，Hessian 為 $p_i(1-p_i)a_i a_i^\top$。

其中 a_i 是固定特徵向量、w 是參數向量、$y_i \in \{0,1\}$ 是觀測標籤、p_i 是 sigmoid 機率。對觀測取平均，再加二次懲罰，即得上面列出的導數。因為 $0 \le p_i(1-p_i) \le 1/4$，

$$\rho I \preceq \nabla^2 f(w) \preceq \left(\rho + \frac{\|A\|_2^2}{4N}\right) I.$$

其中 f 是正則化平均邏輯斯目標、A 是特徵矩陣、N 是觀測數、$\rho > 0$ 是懲罰權重，$\|A\|_2$ 是最大奇異值。下界來自所有座標的正則化；上界可對任意方向 v 檢查 $v^\top A^\top \operatorname{diag}(p_i(1-p_i))Av$ 得出。

因此 $\mu = \rho$ 與 $L = \rho + \|A\|_2^2/(4N)$ 是合法全域常數，能對這個特定模型提供安全梯度步長與目標間隙停止界。若截距不受懲罰，就無法直接推出每個座標都有同樣正下界。數學公式、懲罰實作與停止測試必須一致 [1]。

程式：具有導數與停止檢查的牛頓方法

這是最佳化示範，不是分類基準。求解前先檢查方向導數，最後驗證梯度、下降序列與強凸間隙上界。

```
import numpy as np

rng = np.random.default_rng(16)
A = rng.normal(size=(160, 5))
```

```
truth = np.array([0.8, -0.5, 0.3, 0., 0.4])
prob = 1/(1+np.exp(-(A @ truth)))
y = (rng.random(len(A)) < prob).astype(float)
rho = 0.1

def evaluate(w):
    z = A @ w
    p = 1/(1+np.exp(-z))
    value = np.mean(np.logaddexp(0, z)-y*z) + rho*(w @ w)/2
    gradient = A.T @ (p-y)/len(y) + rho*w
    Hessian = (A.T * (p*(1-p))) @ A/len(y) + rho*np.eye(A.shape[1])
    return value, gradient, Hessian

w = np.zeros(A.shape[1])
direction = rng.normal(size=len(w))
eps = 1e-5
finite_difference = (evaluate(w+eps*direction)[0] -
                     evaluate(w-eps*direction)[0])/(2*eps)
assert np.isclose(finite_difference, evaluate(w)[1] @ direction,
                  atol=1e-8)
history = []
for iteration in range(30):
    value, g, H = evaluate(w)
    history.append(value)
    if np.linalg.norm(g) < 1e-9:
        break
    step_direction = np.linalg.solve(H, -g)
    slope = g @ step_direction
    assert slope < 0
    step = 1.0
    for trial in range(50):
        candidate = w + step*step_direction
        candidate_value = evaluate(candidate)[0]
        if np.isfinite(candidate_value) and candidate_value <= value+0.1*step*slope:
            w = candidate
            break
        step *= 0.5
    else:
        raise RuntimeError("backtracking did not accept a step")
else:
    raise RuntimeError("iteration limit reached")

value, g, H = evaluate(w)
assert np.linalg.norm(g) < 1e-9
assert np.all(np.diff(history) <= 1e-12)
assert np.linalg.eigvalsh(H).min() >= rho-1e-12
print("accepted Newton steps:", len(history)-1)
print("objective:", round(value, 8))
print("gradient norm:", np.linalg.norm(g))
print("objective-gap upper bound:", (g @ g)/(2*rho))
```

導數檢查只測一個方向與一個點，不能證明全部導數正確。修改模型時，應在代表性非零點加入檢查。Hessian 對稱性、特徵值預期與線性求解殘差，提供其他獨立證據。

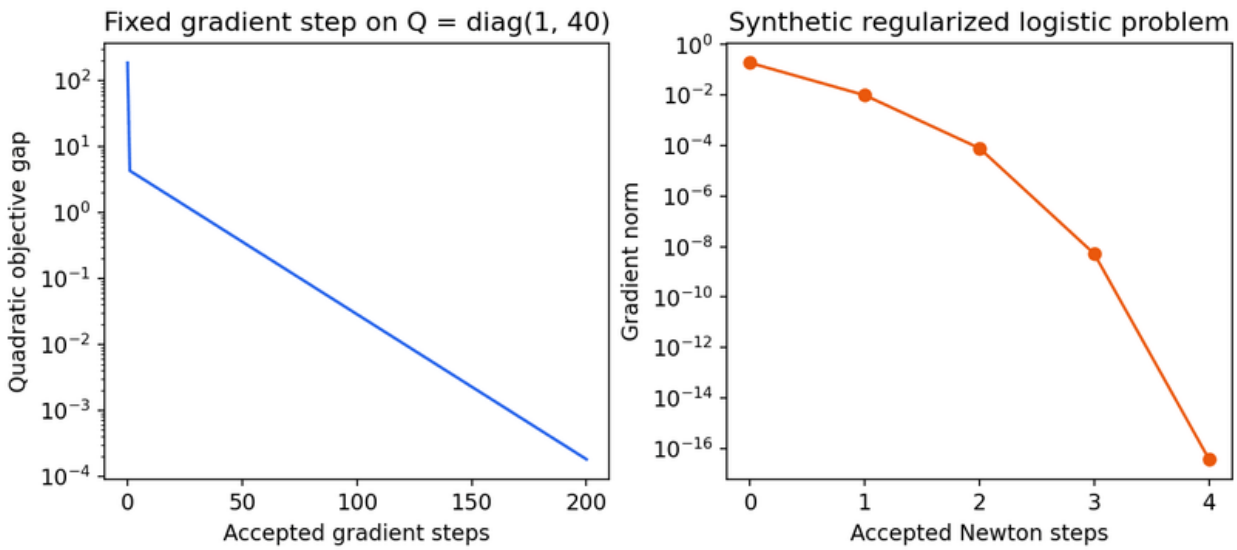


圖 2 — 原創計算收斂曲線。左圖有已知零二次最佳值；右圖是合成邏輯斯目標的梯度範數。接受步數不是執行時間，也不是跨問題基準。

實作與停止回報

白話說。求解器結果應描述為何停止、驗證什麼，以及做了多少計算。

稠密牛頓儲存的矩陣大小，隨變數數量平方增加，分解可能主導成本。梯度方法常只需矩陣向量乘法，因此很多便宜迭代對大問題有吸引力。

稀疏矩陣、低秩項與區塊結構，可能大幅改變比較。利用結構前，要先確認轉換系統等價，且分解填入量可接受 [1，第 9.7 節與附錄 C]。

Hessian 向量乘積可以支援迭代線性求解，不需儲存完整 Hessian。內層求解也有自身殘差與容許誤差。粗略解出的牛頓系統，不一定重現精確牛頓方向與收斂率。

應回報目標值、適當梯度或駐點殘差、定義域檢查、迭代次數，以及指定停止條件。若時間重要，要計入導數評估與線搜尋嘗試，而不只看接受步數。

縮放目標會改變梯度大小。固定絕對梯度容許誤差，不能在任意損失正規化間直接比較。相對判準需要明確尺度，在零附近也應有意義。

迭代限制是資源停止，不是最佳性憑證；線搜尋失敗是演算法診斷，不是不存在解的證明。應保留最後候選點與殘差證據，讓結果可以評估。

精確二次線搜尋揭露取捨

對前述正定二次函數與非零下降方向 d，線上目標為純量二次函數。對距離微分得到

$$t^{*} = -\frac{g^{\top} d}{d^{\top} Q d}.$$

$g = Qx + c$ 是目前梯度，Q 是正定曲率，$t^* > 0$ 是最小化線距離。此公式只對這個特定二次函數給精確線搜尋，不適用於所有平滑目標。

對負梯度方向，分子變成 $g^\top g$，分母為 $g^\top Qg$。距離納入該方向的曲率，可能比保守固定步快，卻不會修正背後拉長的幾何。

連續方向仍可能鋸齒移動。沿一條線精確最小化，不表示線直接指向全域最小值。這清楚區分方向與距離：更好的距離選擇，不能完全取代更好的方向幾何。

牛頓方向使用完整二次曲率系統，在精確算術下以距離一達到最小值。度量近似可能降低系統成本，卻給較不準路線。公平比較須計入度量如何建立與重用，不只看可見路徑長度。

矩陣分解若能對許多右側重用，初始成本就可攤提；若每次迭代矩陣都變，優勢可能縮小。小教材目標評估很便宜，但內含模擬的應用可能很昂貴。宣稱速度前，應說明真正計算原語。

導數檢查需要尺度

中央有限差分把兩個鄰近目標值的差，除以兩倍擾動。縮小擾動，一開始會減少理想截斷誤差，最後卻放大浮點抵消。

應試幾種擾動大小，同時比較相對與絕對差。合理縮放方向、避開定義域邊界，並檢查結果是否在有用區間穩定。一次極小步比較失敗，可能表示數值解析度，而非導數錯誤。

對 Hessian，可比較梯度差與 Hessian 向量乘積。這比只看目標差，測到實作不同層次。目標二次連續可微且預期 Hessian 對稱時，也應檢查對稱性。

自動微分可以減少手動代數，卻仍是對實作表示式微分，而不是對意圖中的模型。錯誤廣播、加總位置不對或意外正則化截距，都可能產生錯誤目標的完全一致導數。

應把檢查與表示法一起記錄。目的是連結符號與陣列，不只是取得通過標記。確認實作符合模型後，數值容許誤差與演算法假設才有意義。

可重現比較也需要起點、目標正規化、停止規則與數值精度。任一項改變，都可能改變表面收斂。合成實驗應保留參數與隨機種子，讓之後的圖能從相同指定問題重新生成。

即使演算法確定性，平台小差異仍可能影響最後幾位數或邊界線搜尋比較。應以有理由的容許誤差驗證性質，不要求完全相同的小數軌跡。本章檢查下降、線性系統一致性與梯度證據，因為它們直接連到表示法。

現代觀點

白話說。現代凸演算法在曲率資訊、投影、近端運算與通訊成本間取捨。適合的選擇取決於結構與要求精度。

Dvurechensky、Shtern 與 Staudigl 的一階綜述 [2] 提供廣泛基礎，包括非歐幾里得近端方法、加速與免投影方法。它發表於 2021 年，是背景地圖，不是 2026 年新結果。

Condat 等人的 2023 年 SIAM Review 綜述 [3]，以原問題與對偶乘積空間和選擇的度量，統一高維非光滑凸問題的近端分裂。它說明加入絕對值懲罰，為何會改變合適計算原語：不能直接讓一般平滑牛頓公式穿過轉折點微分。

Pokutta 的 2024 年 Frank–Wolfe 綜述 [4] 專注受限制集合的線性最小化，以及收斂與憑證結構。它透過詢問能否避開投影或昂貴系統求解，補充梯度與牛頓比較。

這些綜述共同提供實用分類：平滑與非光滑目標、便宜與昂貴可行集合運算、歐幾里得與結構化幾何，以及局部曲率與便宜一階步驟。近期變體不會消除模型差異。

目前議題包括有可靠憑證的自適應方法、大線性系統的精度限制、分散式通訊，以及對求解器輸出微分。應讓演算法符合模型、硬體與要求誤差，並在可比精度下比較總工作量。

重點整理

- 下降方向與步長是兩種不同選擇。
- 固定梯度步需要有效曲率與定義域論證。
- 不良條件數會拖慢純量步長梯度方法。
- 回溯需要保護措施與正確導數符號。
- 牛頓使用線性求解，局部收斂有條件。
- 減量是模型量，除非已建立間隙假設。
- 可靠回報包含停止證據與拒絕工作成本。

練習

1. 推導 $1/L$ 步長的平滑下降不等式。

提示

把負縮放梯度代入二次上模型。

2. 把二次特徵值 40 換成 400，預測並測試固定步長梯度進度如何改變。

提示

平方向收縮取決於特徵值比例。

3. 在 $x > 0$ 上替 $-\log x + x^2$ 實作回溯，每次評估目標前先檢查定義域。

提示

立即拒絕非正候選點。

4. 在非零參數向量，以三個隨機方向驗證邏輯斯梯度。

提示

使用幾種有限差分尺度，比較中央差分與內積。

5. 說明奇異半正定 Hessian 為何不一定定義唯一牛頓方向。

提示

檢查線性系統零空間與一致性。

6. 把邏輯斯目標乘上一百，求如何改變停止量與容許誤差，保留原本要求的目標準確度。

提示

一起轉換梯度、強凸常數與要求間隙。

參考文獻

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. **閱讀：第 9 章第 9.1–9.7 節；線性系統背景看附錄 C；邏輯斯概似連結看第 7 章。** 官方書頁，免費 PDF。
2. Pavel Dvurechensky, Shimrit Shtern and Mathias Staudigl, “First-Order Methods for Convex Optimization,” *EURO Journal on Computational Optimization*, 9, 100015, 2021. 已核對開放手稿，出版文章。
3. Laurent Condat, Daichi Kitahara, Andrés Contreras and Akira Hirabayashi, “Proximal Splitting Algorithms for Convex Optimization: A Tour of Recent Advances, with New Twists,” *SIAM Review*, 65(2), 375–435, 2023. 出版綜述，開放手稿。
4. Sebastian Pokutta, “The Frank-Wolfe Algorithm: A Short Introduction,” *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 126, 2024. 開放綜述。

OPT 07 ・等式限制與內點法

求解受限牛頓系統、追蹤對數中心路徑、區分可行與嚴格可行，並查核原始／對偶憑證。

網站版本: https://blog.shyandram.dev/zh-tw/learn/optimization/opt-07-constrained-interior-point/

你將學到

- 把牛頓法延伸到含仿射等式的凸目標函數。
- 推導對數障礙函數、中心路徑及其對偶間隙。
- 實作具有明確線搜尋的小型可行障礙法求解器。
- 區分可行、嚴格可行與不可行的限制系統。
- 解讀原始／對偶殘差、互補性與不可行性憑證。
- 依數值需求比較現代內點法與一階求解器。

先看全貌

限制式說明解可以做什麼。求解器改善目標時不能忽略它們。等式把每個容許步驟限制在較低維度的空間；不等式則產生必須處理的邊界。內點法（interior-point method）透過一系列問題逼近有用的邊界解，而迭代點保持在邊界內側。

> **用最簡單的話說**
>
> 想像要在有圍籬的花園找到最低的位置。你得留在花園內，地上的線也可能要求你沿特定路徑走。先從離圍籬有一段距離的地方開始；靠近圍籬時，一個想像中的軟墊把你推回來。逐漸減弱軟墊，就能接近圍籬旁的低點而不走出去。

本章搭配 Boyd 與 Vandenberghe 第 10–11 章 [1]：第 10 章處理等式限制下的最小化，第 11 章發展障礙函數、中心路徑、初始化與原始／對偶方法。先讀梯度與牛頓法；憑證看不懂時，回到對偶性與最適條件。Bubeck 的演算法專論 [2] 提供複雜度與求解器家族的另一種觀點。

範例使用低維度合成問題，讓每項主張都能獨立查核。程式是附有斷言、假設透明的教學實作。正式錐規劃求解器還需要尺度調整、稀疏矩陣分解、數值正則化、停止政策與不可行性處理；理解小問題有助於理解這些工程選擇。

1. 等式限制改變容許的方向

目標下降時，仿射等式仍須成立。目前點若滿足 $Ax = b$，步驟 Δx 保持可行的充分必要條件是 $A\Delta x = 0$。先看幾何再看矩陣：容許的步驟位於等式矩陣的零空間。

考慮凸問題：

$$\begin{aligned} \text{minimize}_x \quad & f(x) \\ \text{subject to} \quad & Ax = b. \end{aligned}$$

其中 $x \in \mathbb{R}^n$ 是變數，f 是二次可微的凸目標函數，$A \in \mathbb{R}^{p\times n}$ 與 $b \in \mathbb{R}^p$ 是固定資料。先假設 A 各列獨立，且等式系統有解。拉格朗日乘子 $\nu \in \mathbb{R}^p$ 表示等式限制的影響。

在可行迭代點，沿可行方向最小化二次近似，得到牛頓系統：

$$\begin{bmatrix} H & A^\top \\ A & 0 \end{bmatrix} \begin{bmatrix} \Delta x \\ w \end{bmatrix} = - \begin{bmatrix} \nabla f(x) \\ 0 \end{bmatrix}.$$

其中 $H = \nabla^2 f(x)$ 是 Hessian 矩陣，Δx 是原始變數的候選步驟，w 是局部二次子問題的等式乘子，零區塊大小為 $p \times p$。第二個區塊方程強制 $A\Delta x = 0$。w 解的是局部模型，不一定等於另一個持續維護的乘子估計之增量。

Hessian 只須沿非零可行方向具有正曲率，受限二次模型就能表現良好；被等式排除的方向不必有正曲率。因此完整 Hessian 可能奇異，但受限最小解仍唯一。判斷奇異性是否代表多解前，先檢查真正的可行子空間。

區塊矩陣稱為 Karush–Kuhn–Tucker（KKT）矩陣 [1]。即使 H 正定，整個矩陣通常仍是不定矩陣。不要因原目標凸，就對整個矩陣使用只適合正定矩陣的分解。其結構同時表達原始與對偶方程；小型稠密範例使用一般線性求解即可。

另一種方法是消去等式。取一個可行點 x_0，以及欄向量張成 A 零空間的矩陣 F，寫成 $x = x_0 + Fz$，其中 $z \in \mathbb{R}^{n-p}$ 是降維變數，再解 z 的無限制問題。消去法能簡化小問題，卻可能破壞大問題的稀疏性。兩者幾何相同，計算表示決定成本。

限制子空間曲率與區塊消去

Hessian 奇異不代表受限步驟不明確。在 $x_2 = 3$ 下最小化 $f(x) = x_1^2/2$。Hessian 為 $\operatorname{diag}(1, 0)$，但可行方向只能改變 x_1。唯一最小解為 $(0, 3)$：等式恰好移除平坦方向。

一般而言，令 F 的獨立欄向量張成 A 的零空間。縮減 Hessian 為 $F^\top H F$。這個較小矩陣正定，就能讓可行方向上的局部二次模型有唯一最小解，即使 H 奇異也一樣。其中 H 是目前 Hessian、A 是滿列秩的等式矩陣 [1，第 10 章]。

若 H 本身正定，也可消去原問題方向來推導 KKT 步驟。寫 $g = \nabla f(x)$，利用第一個區塊方程：

$$\begin{aligned} \Delta x &= -H^{-1}(g + A^\top w), \\ (AH^{-1}A^\top)w &= -AH^{-1}g. \end{aligned}$$

其中 Δx 是步驟、w 是局部等式乘子。把第一行代入 $A\Delta x = 0$ 即得第二行。A 各列獨立時，這個較小矩陣為正定：對非零 v，二次型為 $(A^\top v)^\top H^{-1}(A^\top v) > 0$。

這些方程解釋區塊求解，但反矩陣符號只是代數表示。實作應透過 H 的線性求解計算乘積，再比較消去法與直接稀疏 KKT 分解的成本。前面的奇異例子仍能使用零空間表示，卻不能使用這種完整 Hessian 消去。

2. 帶有可查核憑證的受限二次問題

二次目標讓 KKT 方程成為精確條件而非近似。區塊系統產生候選解；檢查等式可行性與駐點條件，才能判斷候選解是否構成這個凸問題的最適性憑證。

對於：

$$f(x) = \tfrac{1}{2} x^\top Q x + c^\top x,$$

$Q \in \mathbb{R}^{n \times n}$ 是對稱半正定矩陣，$c \in \mathbb{R}^n$ 是固定向量，x 是變數。最適配對 (x^*, ν^*) 滿足 $Qx^* + c + A^\top \nu^* = 0$ 及 $Ax^* = b$；星號代表最適值。對可行凸二次問題，這些方程足以保證最適；唯一性還需要可行方向的額外曲率。

```
import numpy as np

Q = np.diag([2.0, 4.0])
c = np.array([-2.0, -1.0])
A = np.array([[1.0, 1.0]])
b = np.array([1.0])
K = np.block([[Q, A.T], [A, np.zeros((1, 1))]])
solution = np.linalg.solve(K, np.concatenate([-c, b]))
x, nu = solution[:2], solution[2:]
primal = A @ x - b
dual = Q @ x + c + A.T @ nu
print("Primal solution:", x)
print("Equality multiplier:", nu)
print("Residual norms:", np.linalg.norm(primal), np.linalg.norm(dual))
assert np.allclose(x, [5/6, 1/6])
assert np.linalg.norm(primal) < 1e-12
assert np.linalg.norm(dual) < 1e-12
```

兩個座標相加為一。無限制最小解的座標則為一與四分之一，違反總量限制。乘子修正駐點方程以反映限制。目標值小不足以證明這項修正；殘差檢查把它明確表達出來。

殘差依賴單位。把等式乘上很大的常數，不改變可行集合，卻改變原始殘差與乘子的尺度。回報原限制的單位及尺度，並用有意義的容許誤差比較殘差。求解器內部的正規化停止條件須轉回應用的量測單位。

不要為了求解而明確計算矩陣反矩陣。線性求解較直接，通常也較穩定。重複求解時，矩陣結構允許才能重用分解。更新線性項與更新 Hessian 或限制矩陣不同；規劃暖啟動與快取時要區分。

非線性目標需要重新計算局部曲率，並採可行線搜尋。完整牛頓步驟即使保持等式，也可能離開目標函數定義域。對數與正定矩陣變數不是處處有定義；等式方程本身無法保護這些定義域。

3. 不等式轉成平滑的內部問題

障礙函數接近邊界時無限增大，只在嚴格可行點有限。如此平滑的受限牛頓法就能把不等式透過目標處理，同時保留明確的仿射等式。

考慮凸不等式 $f_i(x) \le 0$，$i = 1, \dots, m$，以及 $Ax = b$，定義：

$$\phi(x) = -\sum_{i=1}^{m} \log(-f_i(x)), \qquad F_t(x) = t f_0(x) + \phi(x).$$

其中 f_0 是原凸目標，f_i 是各凸不等式函數，m 是純量不等式數目，ϕ 是對數障礙函數，$t > 0$ 控制原目標的相對重要性。定義域要求每個 $f_i(x) < 0$；障礙問題在 $Ax = b$ 下最小化 F_t。

限制函數應以一致單位表示，或取對數前除以正的參考尺度。正數縮放只增加障礙函數的常數項，不改變最小解，但有限精度計算仍可能受影響。數學上的不變性不能當成忽略數值尺度的理由。

二次可微不等式的障礙項，梯度為 $\nabla f_i(x)/(-f_i(x))$，Hessian 為：

$$\frac{\nabla f_i(x) \nabla f_i(x)^{\top}}{f_i(x)^2} + \frac{\nabla^2 f_i(x)}{-f_i(x)}.$$

其中 ∇f_i 是梯度欄向量，$\nabla^2 f_i$ 是 Hessian。因 f_i 凸，嚴格可行定義域內兩項皆半正定。式子也顯示接近活躍邊界、$-f_i(x)$ 變小時，曲率迅速增加。

限制的表示影響路徑。重複同一限制會重複其障礙項，因此有限 t 的近似問題不同，雖然精確可行集合相同。等價限制集合不一定產生相同中間迭代點；標準化與尺度調整是能觀察效果的演算法選擇。

障礙法不能從邊界開始。非負變數起點若為零，$-\log x$ 無定義。自由變數可以用嚴格正值初始化；若等式強制變數為零，則需要重寫問題。任意給限制加一個正偏移會改變問題，不能當作無害的實作修正。

4. 中心路徑連接內點與對偶界限

對每個障礙權重準確求解內部問題，其最小解形成中心路徑（central path）。增加權重會減弱障礙相對原目標的影響，讓解接近可能位於邊界的受限最小解。

令 $x(t)$ 在 $Ax = b$ 下最小化 F_t，定義：

$$\lambda_i(t) = \frac{1}{t[-f_i(x(t))]}, \qquad \nu(t) = \frac{w(t)}{t}.$$

其中 $\lambda_i(t) > 0$ 是不等式乘子，$w(t)$ 是縮放障礙問題的等式乘子，$\nu(t)$ 則是原目標尺度的乘子。t、f_i 與 $x(t)$ 的定義同前。

障礙駐點條件成為原拉格朗日函數的駐點條件；互補性改為 $\lambda_i(t)[-f_i(x(t))] = 1/t$。各純量不等式貢獻相同的正乘積。真正最適點的互補鬆弛要求這些乘積為零；t 增加時，障礙路徑逐漸逼近這項條件。

在完全置中且滿足所述假設的點，其原始／對偶間隙為：

$$f_0(x(t)) - g(\lambda(t), \nu(t)) = \frac{m}{t}.$$

其中 g 是拉格朗日對偶函數，對對偶可行乘子提供最適目標的下界。原始值是上界，對偶值是下界，間隙因此限制目標次最適誤差。這是關於所建立凸問題的敘述，不保證模型符合應用。

等式需要精確置中及有效的對偶構造。駐點殘差很大的數值迭代點，不能只印出 m/t 就宣稱得到精確間隙。以理論值規劃障礙排程，再檢查真實殘差，能計算時也求出真正的對偶下界。如此才不會把外迴圈計數當作憑證。

外層障礙法交替進行兩件事：解固定 t 的內部問題，再增加 t。以前一解作下一起點通常有幫助，因最小解往往相近；但 t 一次改太多會使置中昂貴。依置中實際成本選擇增幅，而不是假設外層迭代越少，總計算就越少。

間隙 m/t 真正從哪裡來

中心路徑間隙是明確價格乘以明確鬆弛量的總和。在精確置中的點，把障礙問題駐點方程除以 t：

$$\nabla f_0(x(t)) + \sum_i \frac{\nabla f_i(x(t))}{t[-f_i(x(t))]} + A^\top \frac{w(t)}{t} = 0.$$

其中 $x(t)$ 最小化障礙問題、$w(t)$ 是其等式乘子、f_0, f_i 是原目標與不等式、$t > 0$ 是障礙權重。代入已定義的 $\lambda_i(t) = 1/[t(-f_i(x(t)))]$ 與 $\nu(t) = w(t)/t$，便恰好得到原拉格朗日函數的駐點方程。

原函數凸且不等式乘子非負，因此拉格朗日函數對 x 凸。駐點條件使 $x(t)$ 成為其全域最小解，所以在此點評估 L 確實算出對偶函數。等式可行性消去等式項，得到

$$g(\lambda(t), \nu(t)) = f_0(x(t)) + \sum_i \lambda_i(t) f_i(x(t)) = f_0(x(t)) - \frac{m}{t}.$$

其中 g 是對偶函數、m 是純量不等式數量。從可行原問題目標減去這個下界，即得間隙。中心點通常不是原最佳解：它在目標改善與正鬆弛量間取捨，但其乘子已提供憑證，而且誤差隨障礙減弱而下降 [1，第 11.2 節]。

若希望理論間隙至多為 $\varepsilon > 0$，選 $t \geq m/\varepsilon$，再把置中問題解得足夠準確。選擇 t 提供目標；駐點求解才是使用公式的依據。只增加 t 卻不置中，就漏掉證明的關鍵等式。

互動示範（網站版）：請開啟本篇教學的網站版本，操作此示範。開啟示範

5. 完整的一維障礙法實驗

使用真正最小解容易查核的二次問題：在 $0 \leq x \leq 1$ 下最小化 $\frac{1}{2}(x-2)^2$。無限制最小點是二，最近的容許點是一，目標值為二分之一。障礙解嚴格位於零與一之間，逐漸接近上端點。

此例為：

$$F_t(x) = \tfrac{t}{2}(x-2)^2 - \log x - \log(1-x).$$

其中 $x \in (0,1)$ 是純量變數，$t > 0$ 是目標權重。一、二階導數為 $F_t'(x) = t(x-2) - 1/x + 1/(1-x)$ 與 $F_t''(x) = t + 1/x^2 + 1/(1-x)^2$；障礙定義域內所有分母皆正。

```
import numpy as np
```

```
def center(t, x):
    def value(z):
        return 0.5*t*(z-2)**2 - np.log(z) - np.log1p(-z)
    for _ in range(100):
        grad = t*(x-2) - 1/x + 1/(1-x)
        hess = t + 1/x**2 + 1/(1-x)**2
        step = -grad/hess
        if grad*grad/hess < 1e-16:
            break
        alpha = 1.0
        for _ in range(100):
            trial = x + alpha*step
            if 0 < trial < 1 and value(trial) <= value(x) + 0.01*alpha*grad*step:
                break
            alpha *= 0.5
        else:
            raise RuntimeError("Line search failed")
        x = trial
    else:
        raise RuntimeError("Centering iteration limit reached")
    return x
x = 0.5
for t in [1., 4., 16., 64., 256., 1024.]:
    x = center(t, x)
    lower, upper = 1/(t*x), 1/(t*(1-x))
    stationarity = x-2-lower+upper
    primal = 0.5*(x-2)**2
    dual = 2-upper-0.5*(-2-lower+upper)**2
    gap = primal-dual
    print(int(t), round(x, 6), round(gap, 7), round(2/t, 7))
    assert 0 < x < 1
    assert abs(stationarity) < 1e-5
    assert abs(gap-2/t) < 1e-8
    assert primal-0.5 <= gap + 1e-8
```

可行回溯檢查在目標比較前執行，避免於定義域外計算對數。充分下降條件使用方向導數，牛頓遞減量則用於置中停止條件。迭代上限讓意外失敗變成明確錯誤，而非無窮迴圈。

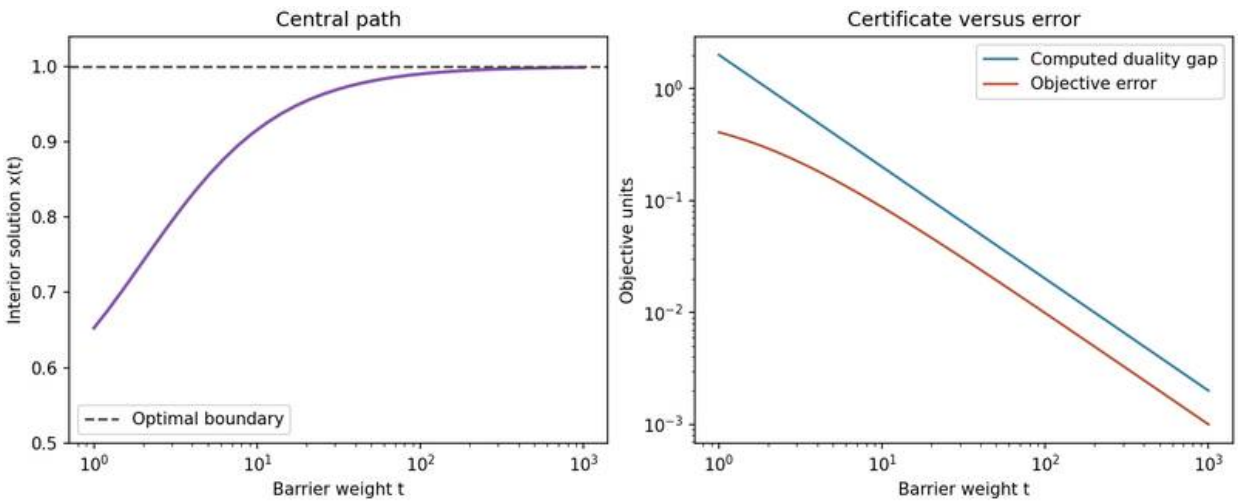


圖 *1* — 合成受限二次問題讓中心路徑憑證可見：解從內部接近一，計算出的對偶間隙限制其目標誤差。

實驗透過純量拉格朗日函數的無限制最小值計算真正對偶值，因此能獨立比較間隙與指定的 $2/t$。大型非線性問題中，對偶函數本身可能需要另一個最佳化問題才能求值。原始／對偶殘差檢查此時有用，但必須說明它在何種正則條件下證明什麼。

接近端點時，1-x 很小，障礙 Hessian 很大。即使數學問題不變，更小的誤差目標仍可能使數值系統更難解。達到應用單位下足夠的準確度即可停止；要求每個可表示位數都準確，可能浪費工作，或暴露應用根本不需處理的條件數問題。

6. 可行性、嚴格可行性與 Phase I

可行解可能存在，卻沒有任何嚴格可行解。原不等式表示的障礙函數此時沒有定義域。這與不可行不同，求解器必須先區分，再嘗試追蹤中心路徑。

簡單的 Phase I（第一階段）構造增加純量 s：

$$\begin{aligned} \text{minimize}_{x,s} \quad & s \\ \text{subject to} \quad & f_i(x) \le s \quad (i = 1, \dots, m), \\ & Ax = b. \end{aligned}$$

其中 s 是限制最大不等式違反量的新增變數。$s < 0$ 的點對原不等式嚴格可行；最適 s 有經證明的正下界，就能證明不可行。最適值零可能代表只有邊界可行；未達到下確界與數值不確定性須另外處理。有限迭代失敗本身不能證明任何一種結果。

矛盾系統 $x \le 0$ 與 $x \ge 1$ 的違反量為 $\max(x, 1 - x)$，在 $x = 1/2$ 達到最小值二分之一。對兩條不等式取相同非負權重，也能產生明確的不可行性憑證。

```
import numpy as np

A = np.array([[1.0], [-1.0]])
```

```
b = np.array([0.0, -1.0])
y = np.array([0.5, 0.5])
print("A transpose y:", A.T @ y)
print("b transpose y:", b @ y)
assert np.all(y >= 0)
assert np.allclose(A.T @ y, 0)
assert b @ y < 0
# If A x <= b existed, multiplying by y would imply 0 <= b@y < 0.
grid = np.linspace(-1, 2, 301)
violation = np.maximum(grid, 1-grid)
assert np.isclose(violation.min(), 0.5)
# A different system is feasible only on its boundary.
A_boundary = np.array([[1.0], [-1.0]])
b_boundary = np.zeros(2)
assert np.all(A_boundary @ np.array([0.0]) <= b_boundary)
```

網格只是已推導最小值的示意，不是任意最佳化問題全域求解的證明。憑證才是關鍵論證：若 $Ax \le b$，乘以 $y \ge 0$ 應得到 $0 = y^\top Ax \le y^\top b < 0$，矛盾。可直接查核資料與乘子。

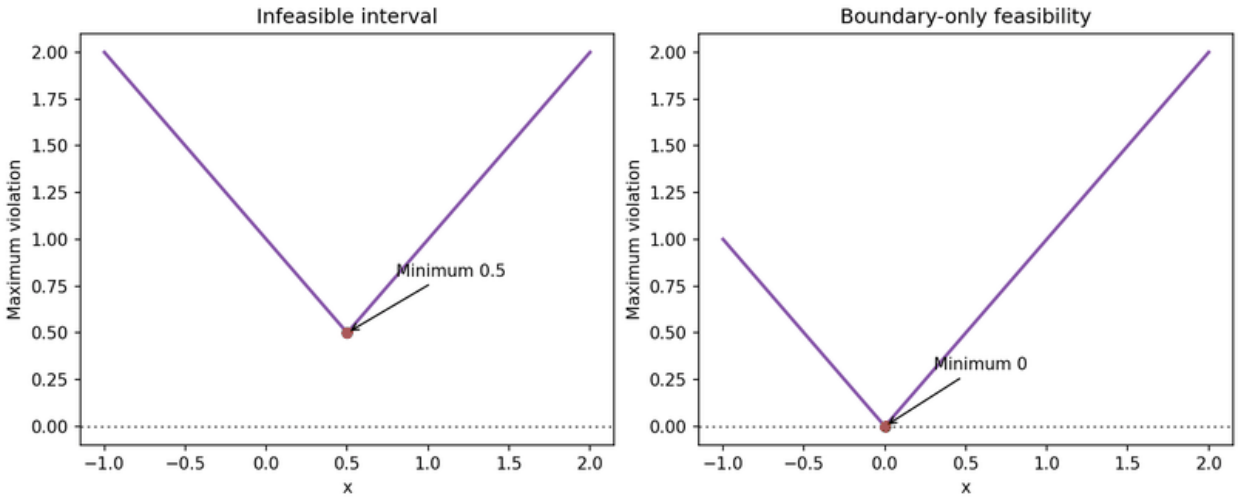


圖 2 — 正的最小違反量證明矛盾區間不可行；第二例的零最小值則指出一個沒有嚴格內部的可行點。

只有邊界可行的系統 $x \le 0$、$-x \le 0$，唯一可行點是零。消去被固定的變數，或把條件寫成等式。起點稍微移動沒有用，因往任一方向移都會違反其中一條。相關內部是把強制等式處理後的相對內部。

Phase I 也依賴 f_i 定義域。放寬不等式值不會自動讓對數、矩陣反矩陣或其他運算有定義。須在函數定義域內初始化，或採合適改寫。函數定義域可行性與仿射、不等式殘差應分開檢查。

Phase I 的對偶價格會正規化

最大違反量形式把初始化連到對偶憑證。對 $f_i(x) - s \le 0$ 附乘子 $\lambda_i \ge 0$，對 $Ax - b = 0$ 附乘子 ν。其拉格朗日函數為

$$L_I(x, s, \lambda, \nu) = \left(1 - \sum_i \lambda_i\right) s + \sum_i \lambda_i f_i(x) + \nu^\top (Ax - b).$$

其中 x, s 是 Phase I 變數、λ, ν 是其乘子、f_i, A, b 是原限制資料。因為 s 不受符號限制，其下確界只在 $\sum_i \lambda_i = 1$ 時有限。因此不等式價格形成正規化非負權重。剩下加權函數的有限下界，會界定原限制的最大違反量 [1，第 11.4 節]。

對 $f_1(x) = x$、$f_2(x) = 1 - x$，取 $\lambda_1 = \lambda_2 = 1/2$。加權和對每個 x 都恰為 $1/2$。因此 Phase I 最佳值至少為 $1/2$，而 $x = 1/2, s = 1/2$ 達到該值。原問題與對偶值相等，證明原不等式不可行。

對只有邊界可行的 $f_1(x) = x$、$f_2(x) = -x$，相同正規化加權和為零。可行 Phase I 點 $x = 0, s = 0$ 匹配它，卻不可能有負的 s。零違反量可證明普通可行性，卻無法提供對數障礙需要的嚴格內部。消去被強制的等式，是適當的下一個建模步驟。

7. 原始／對偶步驟與可靠停止

原始／對偶方法直接處理近似最適方程，不必完全解完每個獨立障礙問題。它維護正的不等式乘子與鬆弛量，再解線性化系統，同時降低駐點、可行性與互補性誤差。

令不等式 i 的鬆弛量為 $s_i = -f_i(x) > 0$，乘子 $\lambda_i > 0$。原問題的殘差包括：

$$r_{\text{dual}} = \nabla f_0(x) + \sum_i \lambda_i \nabla f_i(x) + A^\top \nu,$$

$$r_{\text{pri}} = Ax - b,$$

$$\eta = \sum_i \lambda_i s_i.$$

其中 r_{dual} 是駐點殘差，r_{pri} 是等式殘差，η 是互補乘積總和。函數、矩陣、乘子與鬆弛量定義同前。仿射不等式可以明確加入獨立的鬆弛量方程；非線性表示則可透過 $f_i(x)$ 表達。

η 很小但可行性殘差很大，不是有效的近似解。無限制座標中的梯度很小，也不能證明受限最適。所有相關殘差、乘子符號與定義域條件都要通過。原始點與乘子可行且滿足駐點時，互補乘積總和對應對偶間隙；否則需要限定解讀。

保留邊界距離的線搜尋（fraction-to-boundary）限制步幅，保持鬆弛量與乘子為正。它是演算法定義域的數值保護，不是新增的解限制。不可行起點法可以一開始違反仿射等式，同時保留公式所需的內部條件；其評價函數須減少殘差，而不能只在不可行點比較原目標。

稀疏線性代數往往占主要成本。非零樣式決定分解填入、記憶體及可否重用。少量迭代也可能每一步都很昂貴。比較方法時，除迭代次數外也記錄分解時間與尖峰記憶體。稠密教學實作隱藏這些成本，不宜預測大型問題效能。

標準化求解後，須回到原問題查核解。求解器可能引入輔助變數、錐限制，正規化資料或移除冗餘列。這些操作有助計算，但應用需要原單位限制與可解讀的目標。記錄狀態、殘差、容許誤差、模型版本，以及任何改動回傳點的後處理。

線性化原始／對偶方程

原始／對偶牛頓步驟同時處理最適方程。對 $Q \succeq 0$ 的二次目標、仿射不等式 $Gx + s = h$ 與等式 $Ax = b$，使用正鬆弛向量 s 和正乘子向量 λ。令 c 為目標線性係數，ν 為等式乘子。

定義 $r_d = Qx + c + G^\top \lambda + A^\top \nu$、$r_g = Gx + s - h$、$r_a = Ax - b$。選定互補性目標 $\tau > 0$ 後，線性化得到

$$\begin{aligned} Q\Delta x + G^\top \Delta\lambda + A^\top \Delta\nu &= -r_d, \\ G\Delta x + \Delta s &= -r_g, \\ A\Delta x &= -r_a, \\ \Lambda \Delta s + S\Delta\lambda &= \tau \mathbf{1} - S\lambda. \end{aligned}$$

其中 G, h 指定仿射不等式係數與界限；A, b 指定等式；r_d, r_g, r_a 分別為駐點、不等式可行性與等式可行性殘差，τ 是互補性目標。$\Delta x, \Delta s, \Delta\lambda, \Delta\nu$ 是增量；$S = \operatorname{diag}(s)$、$\Lambda = \operatorname{diag}(\lambda)$，$\mathbf{1}$ 是全一向量。最後一列線性化乘積 $s_i \lambda_i = \tau$，只省略增量的二階乘積。相較完整障礙置中求解，一步就同時降低可行性、駐點與互補性誤差 [1，第 11.7 節]。

若某個鬆弛增量為負，該座標要求步長 $\alpha < -s_i / \Delta s_i$ 才能保持正值。負乘子增量也套用相同規則，取最小容許界限並略留在內側。這便推導出朝邊界比例限制，而非任意裁切規則。仍須加上殘差下降測試：只保持正值並不保證收斂。

8. 在應用邊界解讀憑證

最適性憑證回答指定模型的數學問題。應用決策還取決於模型是否納入正確的限制、量與不確定性。最終報告分開查核兩者：數值正確不能補救遺漏的需求。

假設資源分配模型限制總功率，卻漏掉個別裝置上限。求解器可能得到全域最適且精確證明的分配，但某台裝置無法執行。補上遺漏限制再解，不要把先前結果叫作求解器失敗。相反，限制都已納入，回傳點卻超出容許誤差，問題屬於數值驗收或求解本身。

看到答案前先選容許誤差。溫度上界、預算等式與無因次機率限制需要不同解讀。先用原單位寫出可接受偏差，再連接求解器誤差與尺度設定。後續四捨五入若改變變數，須再次查核可行性；未四捨五入向量的憑證，不會自動證明顯示或實際執行的決策。

符合對偶章節假設時，乘子可解釋敏感度。緊限制的大乘子表示局部小幅放寬可能明顯影響目標；它不是任意大幅變動的預測，尤其活躍限制改變時。小幅擾動、重解問題，並比較實際目標變化與局部預測。

近退化問題須謹慎解讀。幾條幾乎冗餘的限制可能使乘子難以穩定決定，原始決策卻很穩定。乘子變化此時可能誇大決策本身的不確定性。原始與對偶敏感度都要看，影子價格的精度不可超過資料與條件數能支持的程度。

暖啟動也屬於這個邊界。昨天的解可能靠近今天的最小解，卻違反更新後的限制。可行起點障礙法須先找新的內點；原始／對偶或齊次嵌入求解器則可能採其他初始化策略。只有方法支援該可行性狀態才傳入暖啟動；耦合等式存在時，不要逐座標裁切。

有用的結果組合包含模型與資料版本、原目標、原始變數、相關乘子、原單位最大違反量、有效

的間隙或下界，以及停止狀態。效能重要時再附實測時間與記憶體。讀者才能區分可信的近似最小解、不可行性憑證、尚未解決的數值結果，以及需要修正的模型。

現代觀點

內點理論仍有用，現代求解器則在精度、記憶體與硬體上採不同選擇。以下資料提供演算法分類與截至 2026 年 10 月 4 日查核的現代證據。

Bubeck 專論 [2] 依 oracle 提供的資訊、幾何與結構整理一階、結構式及內點方法。比較應同時考慮每步成本，以及 oracle 與假設附帶的保證。本章綜合觀點是依問題中便宜的運算選方法：矩陣向量乘法、投影、近端映射或矩陣分解。

Parikh 與 Boyd 的近端綜述 [3] 用小型凸子問題整理非平滑、受限與分散式演算法，補足障礙觀點：投影或近端運算可以不透過對數內部表示就處理限制。其成本依集合不同而變；數學凸性不決定哪個運算便宜。

Goulart 與 Chen 的 2026 年 Clarabel 論文 [4] 描述使用齊次嵌入、支援多種錐及二次目標的內點求解器，把教科書原理連接到處理不可行性與計算結構的軟體。比較結果屬於所測問題與設定，不代表每個凸模型都有同一個全面贏家。

Applegate 等人的 PDLP 手稿 [5] 於 2025 年首次發布、2026 年修訂，以矩陣向量運算為基礎，加入前處理、尺度調整與自適應策略，發展大型線性規劃的一階方法。它是預印本，效能主張是所報工作負載上的實驗證據。與內點法比較時，納入要求精度、記憶體預算與原單位殘差。

求解器家族	昂貴運算	結構問題	須驗證什麼
等式限制牛頓法	KKT 線性求解	能否重用稀疏結構與分解？	可行性與駐點條件
障礙／內點法	改變的曲率與受限求解	是否有合適內部與錐表示？	定義域、殘差、間隙或憑證
投影／近端方法	投影或近端子問題	限制或非平滑項是否易處理？	迭代殘差與可行輸出
大型一階線性規劃	矩陣向量乘法	其他方法是否受記憶體或分解限制？	指定準確度下的縮放與原始殘差

目前方法論的重點不限於偏好一階或二階。模型表示、前處理、尺度、線性代數與停止政策都是實際演算法行為的一部分。只比較命名的更新公式，會漏掉決定可靠度的許多系統工作。

未解問題包括尺度不良模型的準確憑證、幾乎低維可行區域的穩健性、大型分解的記憶體限制，以及限制改變時的有效暖啟動。學習式預測可以提供起點，但仍須查核可行性與最適性。近似解的誤差若能用應用可容忍的量表示，就有價值。

求解器實驗可建立只改變一項性質的問題族：條件數、活躍限制數、稀疏性或要求誤差。盡可能保留已知最小解或獨立查核的界限。除了時間，也回報失敗與不準確狀態。這類控制問題族比一次預設成功求解更有教學價值。

重點整理

- 等式限制牛頓步驟保持在等式矩陣的零空間。

- 即使目標凸，KKT 矩陣通常仍不定。
- 對數障礙需要相關定義域中的嚴格可行性。
- 完全置中點在所述假設下產生間隙為 m/t 的對偶構造。
- 數值迭代點須檢查真實可行性、駐點條件與互補性。
- 只有邊界可行與不可行不同；Phase I 須用憑證可靠區分。
- 在原問題單位下比較求解成本、精度與記憶體。

練習

1. **受限曲率**。建立完整 Hessian 奇異、但在仿射等式集合上有唯一最小解的半正定二次問題。

   提示

   讓零曲率方向被等式排除，只檢查滿足齊次等式的非零向量之曲率。

2. **等式縮放**。將二次範例的等式乘以 1000，比較解、乘子與殘差尺度，說明哪些量應保持不變。

   提示

   可行集合不變；乘子反向縮放，原始殘差隨限制列縮放。兩個表示都要查核駐點條件。

3. **重複障礙**。複製純量範例的上界不等式，比較有限 t 中心路徑與理論間隙使用的項數。

   提示

   可行集合相同，但上界對數出現兩次，現在有三個純量障礙項。

4. **初始化**。讓純量求解器從零開始，解釋失敗。對此問題及等式強制為零的變數，分別給出有原則的修正。

   提示

   自由非負變數能從嚴格正值開始；被固定為零的變數須消去或改用等式表示，不能只微調數值。

5. **不可行性憑證**。把矛盾區間推廣成 $x \le a$ 與 $x \ge b$，$a < b$。推導最大違反量的最小值及非負憑證。

   提示

   中點平衡兩項違反量。相同權重消掉變數，留下負的右側總和。

6. **停止政策**。求解器回報互補性很小，等式殘差卻非零。寫出接受結果前須加的檢查，解釋只回報目標為何不足。

   提示

   納入原單位等式與不等式殘差、定義域、乘子符號、駐點條件，以及能取得時的有效下界。

參考文獻

1. S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004, Chapters 10–11. Book and official PDF
2. S. Bubeck, "Convex Optimization: Algorithms and Complexity," Foundations and Trends in Machine Learning, 2015. Monograph
3. N. Parikh and S. Boyd, "Proximal Algorithms," Foundations and Trends in Optimization, 2014. Author page and article
4. P. J. Goulart and Y. Chen, "Clarabel: An Interior-Point Solver for Conic Programs with Quadratic Objectives," Mathematical Programming Computation, 2026. Published paper · Author manuscript
5. D. Applegate, M. Díaz, O. Hinder, H. Lu, M. Lubin, B. O'Donoghue and W. Schudy, "PDLP: A Practical First-Order Method for Large-Scale Linear Programming," arXiv preprint, 2025, revised 2026. Paper

OPT 08 ・幾何應用、分裂法與現代方法

把幾何與訊號模型轉成凸問題，選擇近端、ADMM 或條件梯度運算，並以明確憑證評估最佳化流程。

網站版本: https://blog.shyandram.dev/zh-tw/learn/optimization/opt-08-applications-modern-methods/

你將學到

- 用凸幾何建立距離、中心、分離與配置模型。
- 區分凸應用模型與離散或非凸問題的鬆弛。
- 推導非平滑項與限制的近端更新。
- 實作合成訊號去雜訊的分裂法及單純形上的條件梯度法。
- 依昂貴運算選擇方法，查核相應收斂證據。
- 建立可重現的最佳化專題，連接現代綜述與方法。

先看全貌

問題結構同時告訴你如何建模、如何求解時，凸最佳化就有實際價值。幾何模型可能讓投影簡單；訊號模型可能把平滑資料項與非平滑先驗分開；分散式模型可能讓局部求解便宜，但通訊昂貴。合適的演算法運用這些結構。

> **用最簡單的話說**
>
> 想像安排一間工作室：每台機器要有空間、門口要保持暢通，也希望走路距離短。先決定哪些安排被允許，再決定「更好」是什麼。有些規則能讓配置平滑改善；有些則要求一台機器完全在另一台左邊或右邊。這類二選一規則須特別處理。圖畫起來平順，不代表所有配置問題都凸。

幾何閱讀搭配 Boyd 與 Vandenberghe 第 8 章，並連接第 4、6 章的建模與近似 [1]。演算法延伸搭配 Parikh 與 Boyd 的近端綜述 [2]、Boyd 等人的 ADMM 綜述 [3]、Condat 等人的分裂綜述 [4]，以及 Pokutta 的條件梯度導論 [5]。本章把這些運算連成完整的建模、求解與驗證流程，作為單元收尾。

全部實驗使用合成資料、在本機 CPU 執行，目的是讓目標、假設、殘差與失敗模式可查核。數值目標降低，只有在可行集合、資料尺度與評估規則都確認後才有意義。

1. 距離、投影與分離

距離問題問兩個點或集合能在各自限制下靠多近。投影找最近的容許點。這些幾何運算是應用與最佳化演算法的共同積木。

對非空閉凸集合 $C \subseteq \mathbb{R}^n$，定義：

$$\Pi_C(v) = \arg\min_{x \in C} \tfrac{1}{2}\|x - v\|_2^2.$$

其中 $v \in \mathbb{R}^n$ 是固定查詢點，x 是最佳化變數，$\|\cdot\|_2$ 是歐氏範數，$\Pi_C(v)$ 是投影。目標對 x 嚴格凸，因此最小解唯一；閉性確保能達到最小值，非空則確保有可行點。

盒狀集合投影逐座標裁切到區間；仿射集合投影解受限二次問題；半正定矩陣錐投影涉及對稱矩陣的譜分解。它們都是凸投影，成本卻很不同。「投影梯度」只有在理解投影成本後才是有用的演算法描述。

投影滿足 $(v - p)^\top (x - p) \le 0$，對每個 $x \in C$ 成立，其中 $p = \Pi_C(v)$。從投影指向查詢點的位移，背離所有可行方向。它提供最適性檢查，且當查詢點位於集合外時產生分離超平面。

兩個凸集合的距離可寫成最小化 $\|x - y\|_2$，限制為 $x \in C$、$y \in D$，其中 D 是第二個集合。範數對變數配對凸。但任意無界集合的有限下確界不一定能達到；解讀迭代點是否收斂至最近配對前，先查核存在性。緊緻集合較容易處理。

以分離進行分類也有幾何解釋。線性分數 $a^\top x + b$ 定義超平面，a 是法向量、b 是偏移。資料與標籤固定時，間隔限制可以凸；若表示與分類器全部參數共同學習，則可能成為另一個非凸問題。圖形識別單元說明其統計脈絡。

2. 中心、橢球與配置

「中心」有不同意思。最小化到觀測點最大距離的中心，不同於可行區域內最大球的中心；不等式表示的解析中心又是另一個物件。稱一個點為中心前，先說清楚幾何。

對 $a_i^\top x \le b_i$ 描述的多面體，最大內接歐氏球的中心 c 與半徑 r 由下式求得：

$$\begin{aligned} \text{maximize}_{c,r} \quad & r \\ \text{subject to} \quad & a_i^\top c + r\|a_i\|_2 \le b_i \quad \text{for every } i, \\ & r \ge 0. \end{aligned}$$

其中 $a_i \in \mathbb{R}^n$ 與 $b_i \in \mathbb{R}$ 是固定資料，$c \in \mathbb{R}^n$ 與 $r \in \mathbb{R}$ 是變數。長度至多 r 的位移在法向量方向上的最壞貢獻為 $r\|a_i\|_2$。法向量固定，因此每條限制對 (c, r) 都仿射。

此中心以歐氏單位平衡與邊界的距離。把不等式乘正數會同時縮放法向量範數與右側，不改變幾何條件。零法向量須另行檢查，其限制不是永遠成立就是不可能；不可直接除以其範數。

橢球允許各方向不同半徑。寫成 $E = \{c + Bu : \|u\|_2 \le 1\}$，其中 B 是對稱正定形狀矩陣，c 是中心，u 是單位球的輔助座標。包含於半空間的條件變成 $a_i^\top c + \|B^\top a_i\|_2 \le b_i$；最大化 $\log\det B$ 對應最大體積，差一個加法常數 [1]。$B \succ 0$ 定義域不可省略。

置中與包圍的限制不同。內接橢球須位於區域內，包圍橢球則須包含指定點；便利的凸參數化也不同。形狀矩陣看似合理，若不小心引入反矩陣與變數乘積，仍可能得到非凸模型。每項變換都要追溯定義域與凸性規則。

配置問題常最小化可移動點與固定錨點的距離總和。連接與限制固定時，歐氏範數給凸目標；選擇哪些連接存在則引入離散變數。矩形的相對次序先選定後，平面配置子問題可能凸；未知不重疊次序包含析取。原實體問題與凸子問題須分開標示。

中心算例：矩形不一定只有一個答案

最大內切球的中心未必唯一。取矩形 $0 \le x_1 \le 6$、$0 \le x_2 \le 2$。球的限制化為 $r \le c_1 \le 6 - r$ 與 $r \le c_2 \le 2 - r$，並要求 $r \ge 0$。

其中 $c = (c_1, c_2)$ 是中心、r 是半徑。短邊要求 $r \le 1$。最大半徑為一時，所有滿足 $1 \le c_1 \le 5$ 的中心 $(c_1, 1)$ 都最佳。問題凸，最佳解卻不唯一。對稱中心 $(3, 1)$ 只是有效答案之一，不是模型唯一指定的答案。

對內切橢球，對稱正定 B 把單位球映成所需形狀。其體積等於單位球體積乘以 $\det B$，因為線性映射以行列式縮放體積。因此最大化 $\log\det B$ 保留體積最大解。半空間包含規則來自在 $\|u\|_2 \le 1$ 上最大化 $a_i^\top (c + Bu)$，得到 $a_i^\top c + \|B^\top a_i\|_2$。這連起 OPT 03 的支撐函數計算與 OPT 02 的矩陣曲率 [1，第 8 章]。

凸形式是在包含限制下最小化 $-\log\det B$。半空間法向量固定，因此範數項對 B 凸。讓 B 奇異會離開有限對數行列式定義域，並使橢球體積塌縮。

3. 鬆弛需要明確的模型解讀

鬆弛（relaxation）擴大可行集合，讓困難問題較容易處理。它能提供有用界限，或產生稍後需取整的候選點。只有應用允許混合、共享或時間分配時，分數解才可執行。

假設二元配置變數 z_i 必須在 $\{0, 1\}$。改成 $0 \le z_i \le 1$，且其他部分凸時，是離散要求的凸鬆弛。最小化問題擴大可行集合只能降低最適值，因此鬆弛值是原最適值的下界，但不表示鬆弛點是有效的離散決策。

取整後要再次評估原目標與限制。逐座標取整可能違反總預算、連接要求或等式。修復後的可行解提供上界，鬆弛最適值提供下界；兩者差距只有在對應同一原目標，且下方數值是精確鬆弛最適值、有憑證的對偶界或經誤差修正的界時，才有意義。近似的鬆弛原始目標值通常是鬆弛最適值的上界，本身不一定是原離散最適值的下界。

視覺與學習的凸替代模型也一樣。範數懲罰可以鼓勵稀疏，不代表精確解出一般的非零項數最小化問題。凸資料項可能刻意近似物理量測過程。說明模型是統計估計器、計算鬆弛或精確改寫，三種目的支持不同主張。

參數選擇也不在數值憑證內。把正則化問題解得很準，不證明其正則化強度能預測新資料。以正確驗證規則選強度，再於保留資料測試一次。數值最佳化誤差、模型選擇誤差與泛化誤差分開處理。

4. 近端運算處理非平滑結構

非平滑項不必用陡峭平滑函數近似。近端運算（proximal operation）在該項與留在候選點附近之間取捨。許多懲罰與限制有簡單閉式解，適合一階方法 [2]。

對適當、閉、凸函數 g，步幅尺度 $\eta > 0$，定義：

$$\operatorname{prox}_{\eta g}(v) = \arg\min_z \left[g(z) + \frac{1}{2\eta}\|z - v\|_2^2 \right].$$

其中 $v \in \mathbb{R}^n$ 是候選點，$z \in \mathbb{R}^n$ 是子問題變數，g 可取正無窮以表示限制，η 控制二次接近項的尺度。在所述條件下，子問題有唯一最小解。

閉凸集合的指示函數之近端運算就是投影。對 $g(z) = \lambda\|z\|_1$、$\lambda \geq 0$，則是逐座標軟閾值：$\operatorname{sign}(v_i)\max(|v_i| - \eta\lambda, 0)$，其中 i 是座標索引。它能產生精確零；絕對值懲罰在零點不可微，不能直接用普通梯度下降。

最小化 $f(x) + g(x)$、且 f 可微凸時，近端梯度更新為：

$$x^{k+1} = \operatorname{prox}_{\eta g}\left(x^k - \eta \nabla f(x^k)\right).$$

其中 k 是迭代索引，∇f 是平滑梯度，η 依方法假設選擇。梯度若為 L-Lipschitz，保守固定選擇是 $0 < \eta \leq 1/L$。L 限制梯度變化，不是目標值大小。

受限最小平方法變成先梯度再投影；稀疏最小平方法變成先梯度再軟閾值。較複雜的組合，如非平滑函數作用在線性差分算子上，直接近端映射可能昂貴。分裂法引入輔助變數，把便宜運算分開。

有用的殘差是縮放更新 $G_\eta(x) = [x - \operatorname{prox}_{\eta g}(x - \eta\nabla f(x))]/\eta$，其中 G_η 稱近端梯度映射。映射為零時滿足凸組合問題的一階最適條件。其大小是收斂診斷，仍須搭配尺度、可行性及能取得時的目標界限解讀。

在轉角推導軟閾值

近端映射精確處理非平滑轉角。對單一座標，在 $\eta > 0$、$\lambda \geq 0$ 下最小化 $\lambda|z| + (z - v)^2/(2\eta)$。最適條件為

$$0 \in \lambda \partial|z| + \frac{z - v}{\eta}.$$

其中 v 是候選座標、z 是輸出、$\partial|z|$ 是絕對值的次微分集合。當 $z > 0$，次梯度為一，所以 $z = v - \eta\lambda$，只在 $v > \eta\lambda$ 時有效。當 $z < 0$，次梯度為負一，得到 $z = v + \eta\lambda$，只在 $v < -\eta\lambda$ 時有效。在零點，次微分為 $[-1, 1]$，因此零恰好在 $|v| \leq \eta\lambda$ 時最佳 [2]。

三種情況合起來就是閾值公式。對合成向量 $v = (2, -0.3, 0.1)$ 與閾值 $\eta\lambda = 0.4$，輸出為 $(1.6, 0, 0)$。非零輸入能變成精確零，因為零點具有整個區間的支撐斜率。

近端梯度更新來自最小化平滑項的局部上界模型：把 $f(z)$ 換成 $f(x) + \nabla f(x)^\top(z - x) + \|z - x\|_2^2/(2\eta)$，保留精確的 $g(z)$。配方後，模型中心變成 $v = x - \eta\nabla f(x)$。因此平滑成本走梯度步，非平滑成本則做精確小型求解。當 $1/\eta \geq L$，平滑性不等式使模型成為上界，解釋了保守步長條件。

5. 把訊號模型拆成簡單子問題

分段常數訊號可以在符合觀測與懲罰相鄰差異之間取捨估計。二次資料項避免修正太大，絕對差懲罰鼓勵許多鄰近值相同。這個凸估計器適合控制去雜訊實驗，其行為與均勻平滑不同。

令 $y \in \mathbb{R}^n$ 是含雜訊觀測，$D \in \mathbb{R}^{(n-1)\times n}$ 計算相鄰差分，求解：

$$\operatorname{minimize}_x \quad \tfrac{1}{2}\|x - y\|_2^2 + \lambda\|Dx\|_1.$$

其中 $x \in \mathbb{R}^n$ 是估計訊號，$\lambda \geq 0$ 是正則化強度。這是近端與分裂文獻的一維總變異正則化 [2–4]，描述局部變化，不宣稱所有真實訊號都分段常數。

引入 $z = Dx$，使用交替方向乘子法（alternating direction method of multipliers，ADMM）[3]。縮放形式為：

$$x^{k+1} = (I + \rho D^\top D)^{-1}\left(y + \rho D^\top (z^k - u^k)\right),$$
$$z^{k+1} = \operatorname{shrink}(Dx^{k+1} + u^k, \lambda/\rho),$$
$$u^{k+1} = u^k + Dx^{k+1} - z^{k+1}.$$

其中 $\rho > 0$ 是懲罰參數，I 是單位矩陣，u 是縮放對偶變數，shrink 是上述逐座標軟閾值。反矩陣記號只描述數學解，實作使用分解與線性求解。分裂等式在收斂時成立，不必每個中間迭代點都成立。

```
import numpy as np
from scipy.linalg import cho_factor, cho_solve

rng = np.random.default_rng(8)
n = 48
clean = np.r_[np.zeros(16), np.ones(16), np.full(16, 0.3)]
y = clean + rng.normal(0, 0.16, n)
D = np.diff(np.eye(n), axis=0)
lam, rho = 0.4, 4.0
factor = cho_factor(np.eye(n) + rho*D.T @ D)
x, z, u = y.copy(), np.zeros(n-1), np.zeros(n-1)
for k in range(2000):
    x = cho_solve(factor, y + rho*D.T @ (z-u))
    z_old = z.copy()
    v = D @ x + u
    z = np.sign(v) * np.maximum(np.abs(v)-lam/rho, 0)
    u += D @ x-z
    primal = np.linalg.norm(D @ x-z)
    dual = rho*np.linalg.norm(D.T @ (z-z_old))
    if max(primal, dual) < 1e-7:
        break
objective = lambda a: 0.5*np.sum((a-y)**2) + lam*np.abs(D @ a).sum()
print("Iterations and residuals:", k+1, primal, dual)
print("Objectives, raw and estimated:", objective(y), objective(x))
print("Synthetic MSE, raw and estimated:", np.mean((y-clean)**2), np.mean((x-clean)**2))
assert max(primal, dual) < 1e-7
assert objective(x) <= objective(y) + 1e-8
assert np.mean((x-clean)**2) < np.mean((y-clean)**2)
```

最後一項斷言只查核這個合成例子，不是正則化一定改善重建的定理。太強的正則化可能壓平真邊緣或移除窄峰。目標在模型內選取折衷；乾淨訊號誤差則需要真值，實際部署不一定有。

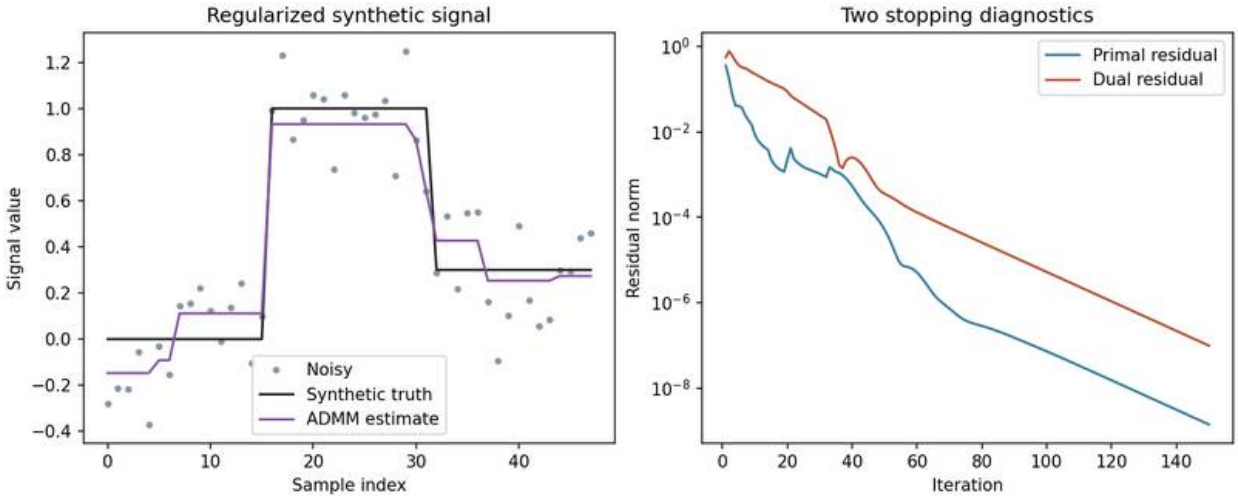


圖 1 — 分裂法分開二次求解與軟閾值。收斂時兩種殘差都趨近零，但不要求每一步單調下降；去雜訊改善屬於這個合成訊號與正則化選擇。

D 與 ρ 固定，所以分解能重用。改正則化強度只改閾值；改 ρ 會改線性系統，須重新分解。若改 ρ 且保留同一未縮放乘子，u 也須一致重縮放，否則更新偷偷改了對偶狀態。

影像中，相鄰差分改成水平與垂直算子，懲罰也有不同合併方式。邊界條件與像素尺度影響估計器。本章一維結果不能當影像復原基準。量測模型請看影像處理，學習式復原比較請看視覺深度學習。

推導拆分更新與殘差

ADMM 方程來自保留明確的差分變數，而不是直接計算困難的組合近端映射。以 $z = Dx$ 建立縮放增廣目標：

$$\frac{1}{2}\|x - y\|_2^2 + \lambda\|z\|_1 + \frac{\rho}{2}\|Dx - z + u\|_2^2 - \frac{\rho}{2}\|u\|_2^2.$$

其中 x 是訊號、y 是觀測、D 是差分矩陣、z 是拆分差分、u 是縮放乘子、$\rho > 0$ 是懲罰參數。最後一項在原問題子步驟中固定。對 x 微分得到 $(I + \rho D^\top D)x = y + \rho D^\top(z - u)$。對 z 最小化就是以 $Dx + u$ 為中心、閾值為 λ/ρ 的軟閾值問題。乘子更新接著累積拆分等式誤差 [3]。

為什麼監測兩個殘差？原問題殘差 $r_p = Dx - z$ 檢查耦合；對偶殘差 $r_d = \rho D^\top(z^{k+1} - z^k)$ 檢查循序更新在駐點條件中留下的不一致。其中 k 是迭代索引；改變正負號慣例不影響殘差範數。

資料項讓這個特定目標對 x 強凸，因此即使相鄰差分在非平滑轉角為零，去雜訊訊號仍唯一。殘差說明拆分演算法是否到達該解，卻不量測與未知乾淨訊號的吻合程度。合成實驗因為知道生成訊號，才能同時檢查兩者。

6. 條件梯度使用線性 oracle

投影可能昂貴，但在集合上最小化線性函數可能簡單。條件梯度法以線性最佳化 oracle 和凸組合更新取代投影；可行初始化後，迭代點保持可行 [5]。

對緊緻凸集合 C 上的可微凸函數 f，求解：

$$v^k \in \arg\min_{v \in C} \nabla f(x^k)^\top v, \qquad x^{k+1} = (1-\gamma_k)x^k + \gamma_k v^k.$$

其中 x^k 是目前可行點，v^k 是線性 oracle 解，$\gamma_k \in [0,1]$ 是步幅權重。新點依凸性可行。Frank–Wolfe 演算法 [5] 就是搭配適當步幅與收斂假設的基本條件梯度構造。

有用的憑證是：

$$g_{\mathrm{FW}}(x) = \nabla f(x)^\top (x - v(x)).$$

其中 $v(x)$ 是在 x 的線性 oracle 解。oracle 精確且目標可微凸時，支撐超平面不等式給 $f(x) - f(x^\ast) \le g_{\mathrm{FW}}(x)$，其中 $x^\ast$ 是 C 內最小解。間隙因此限制目前可行點的目標誤差。近似 oracle 需要額外誤差容許量。

在機率單純形 $C = \{x \ge 0 : \sum_i x_i = 1\}$，線性 oracle 選梯度最小的座標，回傳單純形頂點。混合模型若能便宜選出新成分，此法就有用。每次加入頂點可能形成稀疏迭代表示，但表示與精度折衷仍須檢查。

```
import numpy as np

target = np.array([-0.1, 0.6, 0.5])
def project_simplex(v):
    sorted_v = np.sort(v)[::-1]
    threshold = (np.cumsum(sorted_v)-1)/np.arange(1, len(v)+1)
    active = np.flatnonzero(sorted_v > threshold)[-1]
    return np.maximum(v-threshold[active], 0)
optimum = project_simplex(target)
x = np.full(3, 1/3)
for k in range(1000):
    grad = x-target
    vertex = np.eye(3)[int(np.argmin(grad))]
    gap = float(grad @ (x-vertex))
    error = 0.5*np.sum((x-target)**2)-0.5*np.sum((optimum-target)**2)
    assert error <= gap + 1e-12
    if gap < 1e-4:
        break
    gamma = 2/(k+2)
    x = (1-gamma)*x + gamma*vertex
# Recompute certificate at the returned iterate.
grad = x-target
vertex = np.eye(3)[int(np.argmin(grad))]
gap = float(grad @ (x-vertex))
print("Simplex iterate:", x)
print("Frank-Wolfe gap:", gap)
assert np.all(x >= 0) and np.isclose(x.sum(), 1)
assert gap < 0.01
```

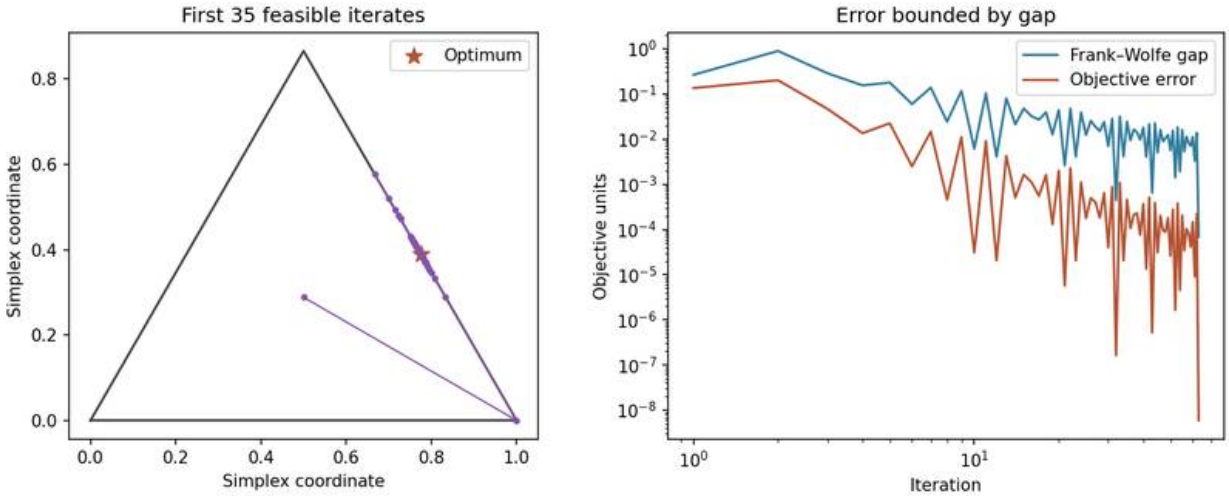


圖 2 — 線性 *oracle* 提供可行頂點與間隙憑證。目標誤差下降時，路徑仍可能來回鋸齒。

範例的參考投影只用於計算精確二次最小解，方便比較；條件梯度本身從不呼叫投影。如此可測間隙界限，而不只看曲線下降便推斷收斂。最後更新後重新計算間隙，讓它描述真正回傳的點。

緊緻可行集合與平滑目標支持通常的收斂分析。無界 oracle 或非平滑目標改變問題。變體能處理額外結構，但不可未檢查假設便沿用基本保證。綜述 [5] 說明選擇與幾何動機。

證明 oracle 間隙，並包含非精確 oracle

條件梯度間隙是最佳線性化可行方向允許的改善。凸性給出 $f(x^\ast) \ge f(x) + \nabla f(x)^\top (x^\ast - x)$。因為精確 oracle 在 C 上最小化該線性分數，

$$f(x) - f(x^\ast) \le \nabla f(x)^\top (x - x^\ast) \le \nabla f(x)^\top (x - v) = g_{\mathrm{FW}}(x).$$

其中 x 是目前可行點、$x^\ast$ 是最佳解、v 是精確線性 oracle 的解。第一個不等式使用目標凸性，第二個使用 oracle 與 $x^\ast$ 的比較。這兩個要素把可計算方向量變成全域誤差上界 [5]。

若可行的近似 oracle 回傳 $\tilde{v}$，滿足 $\nabla f(x)^\top \tilde{v} \le \min_{z \in C} \nabla f(x)^\top z + \delta$，其中已知 $\delta \ge 0$，有效上界就是計算出的間隙加上 δ。其中 δ 是 oracle 的線性目標誤差容許量。因為 $\tilde{v} \in C$，即使間隙須修正，更新仍保持可行。

對單純形梯度 $(2, -1, 3)$，精確 oracle 選第二頂點，分數為 -1。選第一頂點的分數為 2，需要容許量 $\delta = 3$。在 $x = (1/3, 1/3, 1/3)$，精確間隙為 $7/3$，近似 oracle 間隙卻為 $-2/3$。加上三後恢復 $7/3$。此處未修正負間隙代表 oracle 不佳，不代表目標誤差低於零。

7. 依結構、精度與資訊選擇方法

沒有全面最好的更新方程。建模決策決定可取得的資訊及便宜運算。比較名稱或迭代次數前，先比較真正運算成本。

中型平滑問題若能利用曲率，牛頓或內點法可透過較少昂貴步驟取得精確解。巨大稀疏模型可能受分解記憶體限制，一階法便宜的矩陣向量運算更合適。簡單非平滑懲罰可用便宜精確的近端運算；複雜集合若線性 oracle 簡單，條件梯度可避免昂貴投影。

分散式最佳化再加入通訊成本。局部目標可分離，共識限制卻連接變數。ADMM 能暴露局部子問題，但收斂時間須包括通訊輪次與局部求解。數學上分散式不代表實作平行；本章去雜訊範例完全循序執行。

原始／對偶混合梯度法（primal–dual hybrid gradient）以原始與對偶更新處理鞍點表示 [4]，步幅依算子範數與分裂項結構決定。PDLP [6] 等實際線性規劃系統在此結構外加入前處理與自適應技巧。短小理論遞迴與正式求解器是不同描述層次。

要求精度可能反轉比較。快速產生有用近似解的方法，可能要很久才達到極小憑證間隙；另一法前置成本大，但高精度收斂有效。畫出達到幾種指定誤差所需時間，並與間隙一起回報原單位可行性，讓折衷可查核。

記憶體與重複求解也重要。參數掃描在方法支援時能重用資料結構、分解或前一解；稀疏樣式改變可能阻止重用。首次量測納入前處理與設定，暖啟動重複求解另行回報。不要偷偷拿一法暖執行與另一法冷執行比較。

共識：局部求解與共享平均

分散式 ADMM 明確呈現通訊機制。假設 J 個參與者共享向量決策，但各有局部凸成本 f_j。引入局部副本 x_j 與共享變數 z，在所有 $x_j = z$ 下求解 $\min \sum_{j=1}^{J} f_j(x_j)$。

縮放形式的一次共識迭代為

$$x_j^{k+1} = \arg\min_x \left[f_j(x) + \frac{\rho}{2} \|x - z^k + u_j^k\|_2^2 \right],$$

$$z^{k+1} = \frac{1}{J} \sum_{j=1}^{J} (x_j^{k+1} + u_j^k),$$

$$u_j^{k+1} = u_j^k + x_j^{k+1} - z^{k+1}.$$

其中 u_j 是參與者 j 的縮放乘子、$\rho > 0$ 是懲罰參數、k 是迭代索引。第一列為獨立局部求解；第二列對 z 最小化二次共識項之和，其導數使 z 等於平均；第三列累積不一致 [3]。

對純量成本 $f_j(x) = \frac{1}{2}(x - a_j)^2$，局部解為 $x_j^{k+1} = [a_j + \rho(z^k - u_j^k)]/(1 + \rho)$，其中 a_j 是局部目標。兩參與者的目標若為零與四，必須在全域最佳值二達成共識。各自最適的零與四違反共識；平均與乘子更新協調折衷。

因此通訊屬於演算法本身：共享更新需要各個局部量，局部求解則需要最新共享值。平行實作除了局部算術，也須量測訊息成本與同步。若增加共享限制，z 子問題也會改變，常從簡單平均變成受限近端運算。

8. 完整凸最佳化專題

專題須讓模型與證據可重現。選合成訊號估計、幾何配置或資源模型，求解前先寫變數、單位、凸性論證、定義域與可行集合解讀。說明鬆弛及如何把結果轉成可執行決策。

建立一個有解析答案或獨立憑證的小實例，用來檢查符號、形狀、尺度與停止規則。接著一次改一項：雜訊、正則化、條件數、稀疏性、大小或容許誤差。同一實例比較演算法時固定資料種子。

在真正回傳點記錄目標與殘差。分裂法追蹤耦合與對偶殘差；條件梯度報目前間隙與 oracle 精度；障礙法區分外層目標與計算憑證；鬆弛則分開下界與可行取整上界。

數值收斂之外也評估應用。去雜訊器可能準確最小化一個會移除真事件的懲罰；配置求解器可能遵守全部已建模規則，卻漏了門口需求；資源最佳化可能對不確定需求敏感。這些是模型驗證發現，不應模糊數學憑證的意義。

學習流程在選正則化或學前處理前先切分資料。驗證集重複求解、選規則，再回報保留資料結果。ML/DL 評估章說明更廣的學習規則。數值收斂與預測評估回答不同問題，結果表應分不同欄。

現代觀點

閱讀地圖把整理演算法的綜述與改變實務建模／求解的新工作結合。以下來源查核至 2026 年 10 月 4 日，未出版手稿明確標示。

Parikh 與 Boyd [2] 以小凸子問題整理近端法；**Boyd 等人 [3]** 以可分離性、耦合與分散式統計應用整理 ADMM。本章綜合重點是：分裂後哪些子問題變簡單，新增哪些殘差需要控制。分裂能減少總工作才有價值，不是因目標有數項就值得做。

Condat 等人 [4] 透過原始／對偶乘積空間與單調包含統一近端分裂，重訪參數範圍與變體。不同遞迴可依算子與度量連接。用它查明真實假設，尤其平滑二次項可能允許與一般平滑函數不同的選擇。統一觀點避免把任意參數組合當成有保證的方法。

Pokutta 的 2024 年導論 [5] 以線性 oracle 與可行組合整理條件梯度，幫助比較投影法與 oracle 法，也解釋可行集合幾何如何影響行為。先估 oracle 與表示成本，再使用有效間隙。便宜 oracle 仍可能需很多迭代才達應用精度。

Shorinwa 等人的 2024 年多機器人綜述 [7] 區分分散式一階、循序凸規劃與 ADMM 方法，把通訊、協調及實體限制放在數學更新旁。部分機器人模型非凸，或透過一連串凸子問題求解；凸子問題收斂不證明原軌跡或協調問題全域最適。

Applegate 等人的 PDLP 手稿 [6] 於 2025 年發布、2026 年修訂，提供大型一階線性規劃的新實務證據。**Schaller、Bemporad 與 Boyd 的 2025 年手稿 [8]** 則配適相容於規範式程式設計的參數化凸函數。前者關於求解流程，後者關於對決策變數保持凸性的學習函數；兩者都是手稿，也都不會使任意神經網路訓練凸化。

問題	有用家族或閱讀	應蒐集的證據
限制投影便宜嗎？	投影／近端法 [2]	投影成本與最適性殘差
各項能拆成簡單運算嗎？	ADMM、原始／對偶分裂 [3,4]	局部求解成本與耦合殘差

問題	有用家族或閱讀	應蒐集的證據
線性最小化比投影容易嗎？	條件梯度 [5]	oracle 成本、可行點與間隙
通訊是否主導成本？	分散式分類 [7]	輪次、訊息、局部工作與實體可行性
線性模型太大而無法分解嗎？	實務一階線性規劃 [6]	記憶體、準確度與原單位殘差
學習函數能保留決策變數凸性嗎？	參數化凸建模 [8]	結構限制與保留資料上的模型適切性

綜合而言，建模與計算越來越透過可重用結構元件交會：凸子問題、便宜近端映射、學習式凸函數及配合硬體的求解運算。價值仍依明確模型、有效假設與端到端量測。求解器基準不證明泛化，預測基準也不證明可行性。

未解問題包含自動結構辨識、不良尺度下的穩健憑證、通訊效率、活躍限制改變時的可微性，以及學習預測與精確限制的可靠整合。有用研究問題隔離一項議題並保留獨立可查核結果，例如：「同一憑證誤差下，暖啟動是否降低總求解時間？」

本單元已提供完整路徑：凸集合與函數、問題建立、對偶性、估計、數值方法、受限求解器與結構式應用。接著可選一組教科書練習，圍繞假設建立小型原創實驗。核心習慣是寫清楚最佳化什麼、為何凸、演算法計算什麼，以及接受結果的證據。

重點整理

- 幾何建模包含投影、分離、中心、橢球與結構式配置。
- 未知離散選擇與析取可能使原應用非凸。
- 鬆弛下界與可行可執行解扮演不同角色。
- 近端運算直接處理簡單非平滑懲罰與限制。
- ADMM 把耦合拆成易處理運算，須查核原始及對偶殘差。
- 條件梯度透過線性 oracle 保持可行性，並在所述假設下提供間隙。
- 比較原單位準確度、設定、記憶體、重複求解與應用結果。

練習

1. **幾何中心**。推導軸對齊矩形的最大內接球，說明一邊較長時中心為何可能不唯一。

   提示

   較短邊限制半徑；中心可沿長邊移動而仍保留足夠邊界距離。

2. **平面配置**。先選定兩矩形哪個在左，寫凸不重疊限制，解釋次序未知時有何改變。

   提示

   固定次序給仿射分離不等式；未知次序須二選一，是析取而非單一凸半空間。

3. **正則化掃描**。以五種強度執行去雜訊，比較目標、合成重建誤差與近乎零的相鄰差分數。

提示

不同強度的目標是不同函數，不能用原始值公平比較收斂。分開模型品質與求解精度。

4. **懲罰更新**。在實驗中途改 ADMM 懲罰，保持分解與縮放對偶變數一致，再與固定懲罰比較殘差。

提示

未縮放乘子是懲罰乘縮放對偶變數。改懲罰時保持此乘積。

5. **非精確 oracle**。故意使用次最適單純形頂點，找出 oracle 目標誤差如何改憑證界限，以及迭代是否仍可行。

提示

頂點的凸組合保持可行；非精確 oracle 的間隙須加上其線性目標誤差上界。

6. **專題報告**。在一個合成凸應用比較兩種合適演算法，納入模型、凸性證明、停止政策、原單位殘差、時間、記憶體及一個模型失敗。

提示

資料與驗收誤差相同；設定成本與重複求解分開。未解決的數值執行須如實回報，不能算成功求解。

參考文獻

1. S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004, Chapter 8, with Chapters 4 and 6 for modeling and approximation. Book
2. N. Parikh and S. Boyd, “Proximal Algorithms,” Foundations and Trends in Optimization, 2014. Survey
3. S. Boyd, N. Parikh, E. Chu, B. Peleato and J. Eckstein, “Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers,” Foundations and Trends in Machine Learning, 2011. Review
4. L. Condat, D. Kitahara, A. Contreras and A. Hirabayashi, “Proximal Splitting Algorithms for Convex Optimization: A Tour of Recent Advances, with New Twists,” SIAM Review, 2023. Author manuscript
5. S. Pokutta, “The Frank-Wolfe Algorithm: A Short Introduction,” Jahresbericht der Deutschen Mathematiker-Vereinigung, 2024. Survey
6. D. Applegate, M. Díaz, O. Hinder, H. Lu, M. Lubin, B. O’Donoghue and W. Schudy, “PDLP: A Practical First-Order Method for Large-Scale Linear Programming,” arXiv preprint, 2025, revised 2026. Paper
7. O. Shorinwa, T. Halsted, J. Yu and M. Schwager, “Distributed Optimization Methods for Multi-Robot Systems: Part 2—A Survey,” IEEE Robotics & Automation Magazine, 2024. Survey
8. M. Schaller, A. Bemporad and S. Boyd, “Learning Parametric Convex Functions,” manuscript, June 2025. Author page

篇 VIII

資訊理論

IT 01・熵、相對熵與互資訊

從機率建立資訊量，推導鏈式法則與資料處理不等式，並區分精確恆等式和從資料估計資訊量的不確定性。

網站版本: https://blog.shyandram.dev/zh-tw/learn/information-theory/it-01-entropy-mutual-information/

你將學到

- 為什麼對數能把機率轉成可相加的驚訝程度。
- 計算熵、條件熵、相對熵與互資訊。
- 推導鏈式法則，辨認基本不等式何時取等號。
- 解釋處理能丟棄、保留或增加什麼，不把隨機性混同於資訊。
- 用有限機率表與完整 CPU 範例檢查資訊恆等式。
- 區分母體資訊量、經驗估計與學習目標。

先看全貌

感測器傳送一串量測，有些值可預測，有些出乎意料。壓縮器利用可預測性；接收者則需要能減少目標不確定性的量測。這兩種價值相關，但不相同。一串非常隨機的訊號，也可能幾乎沒有關於場景的有用證據。

> **用最簡單的話說**
>
> 想像你在猜盒子裡有什麼。如果每個盒子都裝玩具，「裡面是玩具」幫助不大；「裡面是唯一那台紅色火車」卻可能解決問題。資訊理論依可能結果的機率，量測訊息減少了多少不確定性。

本主線以 Thomas M. Cover 與 Joy A. Thomas 的 *Elements of Information Theory* 第 2 版為核心 [1]。這些資訊量有實際意義：熵連到壓縮，互資訊連到通訊限制，相對熵則把機率失配連到額外描述長度與統計辨別。我們先從有限字母集合開始，讓每個量都能逐項相加檢查。

Stanford 熵講義 [2] 提供易讀的配套說明。本章使用明確合成機率模型，讓定義與數值檢查可以直接逐項確認。

主線如何銜接

教學	核心問題	第 2 版主要章節
IT 01	資訊到底量測什麼？	1–2，以及 17 的基本工具
IT 02	為什麼長序列的整體行為較可預測？	3–4
IT 03	如何縮短描述而不丟失符號？	5、13–14

教學	核心問題	第 2 版主要章節
IT 04	有雜訊的通道能可靠傳多少資訊？	7
IT 05	連續與高斯變數有何不同？	8–9
IT 06	容許失真時需要多少位元率？	10
IT 07	資訊量如何支援推論與決策？	6、11–12、16
IT 08	多使用者、限制與學習帶來什麼改變？	15，以及 17 的進階工具

這種分組涵蓋全書主要主題，但不表示每個定理都能用短推導完整處理。先理解定義，再做數值檢查，最後閱讀證明。計算不清楚時，先明列機率表，再加入神經估計器或通訊系統。

隨機變數、結果與機率模型

隨機變數描述觀測前可能發生什麼。機率來自對實驗的模型。同一個觀測值，在某個模型下罕見，在另一模型下可能很普通。

令 X 的值來自有限字母集合 $\mathcal{X}$：

$$p_X(x) = \Pr(X = x), \qquad p_X(x) \geq 0, \qquad \sum_{x \in \mathcal{X}} p_X(x) = 1.$$

X 是隨機變數，x 是一個可能結果，$\mathcal{X}$ 是結果集合，p_X 是機率質量函數，Pr 表示機率。聯合分布 $p_{XY}(x,y)$ 為成對結果指定機率，邊際分布由加總取得：

$$p_X(x) = \sum_y p_{XY}(x,y), \qquad p_Y(y) = \sum_x p_{XY}(x,y).$$

Y 是另一有限隨機變數，y 是其結果，p_Y 是邊際機率質量函數。若 $p_Y(y) > 0$，條件機率為 $p_{X|Y}(x \mid y) = p_{XY}(x,y)/p_Y(y)$。零機率事件需要另外處理；它不會貢獻以下有限加總。

未特別說明時，對數以二為底，單位是 **bit（位元）**。自然對數對應 **nat**；把 nat 數值除以 $\ln 2$ 可換成 bit。bit 是資訊單位，不一定等於某種檔案格式實際儲存的一個二進位數字。

比較分布時，先固定實驗。一個模型描述灰階值，另一個描述整張影像，兩者熵不能直接比較。改變變數、字母集合、量化或已知條件，都是改變問題。

對數讓驚訝程度可以相加

越罕見的結果，驚訝程度越高。兩個獨立觀測應帶來各自驚訝程度的總和；對數讓加法與機率乘法一致。

定義某結果的自資訊（self-information）：

$$\iota_X(x) = -\log_2 p_X(x).$$

$\iota_X(x)$ 是模型 p_X 下，結果 x 的驚訝程度。獨立結果的機率若為 a 和 b，聯合機率是 ab，因此 $-\log_2(ab) = -\log_2 a - \log_2 b$。等可能結果數加倍，資訊量增加一 bit，而非變成兩倍。

機率一的結果沒有驚訝；機率 1/8 的結果有三 bit。這些描述的是觀測前的機率模型；罕見符號不一定具有較高的實際任務價值。

若模型對實際出現的結果給零機率，自資訊就是無限大。預測應用中，這可能表示支撐集合錯誤：模型宣稱實際可能發生的事件不可能。加入很小的數值常數會改變模型；它可能有用，但不是原分布的計算。

平均驚訝程度就是熵（entropy）：

$$H(X) = -\sum_{x} p_X(x) \log_2 p_X(x) = \mathbb{E}[\iota_X(X)].$$

$H(X)$ 是 X 的熵，$\mathbb{E}$ 依其分布取平均。約定 $0 \log 0 = 0$ 來自極限；從不出現的結果不貢獻平均。

因此熵是分布的性質，不是某個已實現樣本的性質。平均熵很小的分布，也可能偶爾產生高驚訝結果。影像具有顯眼圖案，也仍可能被合適的機率模型輕易預測。

熵的界限與二元例子

結果確定時，不確定性為零。可能結果數固定時，所有結果等機率會使不確定性最大。

若字母集合有 K 個值：

$$0 \le H(X) \le \log_2 K.$$

$K = |\mathcal{X}|$ 是集合大小。集中在單一結果的分布達到下界；均勻分布達到上界。若只有 K_+ 個結果具有正機率，更緊的上界是 $\log_2 K_+$。

二元變數的結果一有機率 p 時：

$$h_2(p) = -p \log_2 p - (1-p) \log_2 (1-p).$$

h_2 是二元熵函數，$p \in [0,1]$。曲線關於 $p = 1/2$ 對稱，最大值是一 bit，接近兩端時趨近零。偏斜來源比公平來源容易預測，即使兩者都只有兩種符號。

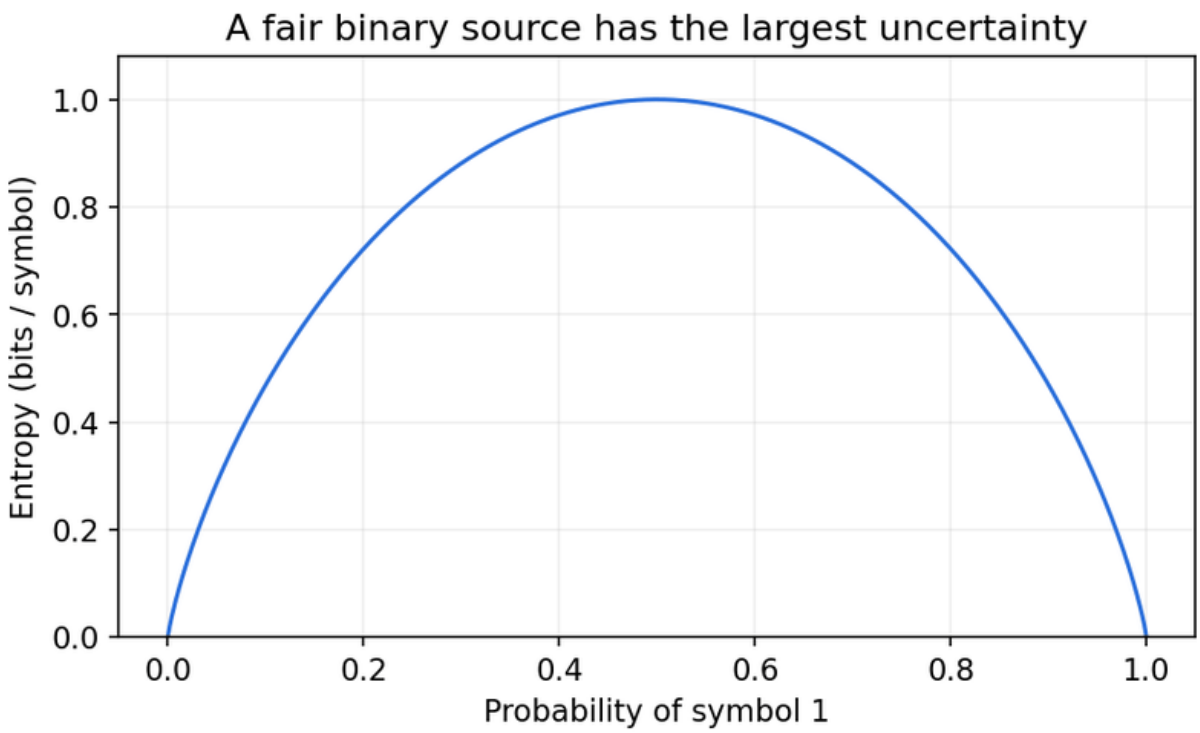


圖 1 — 二元熵的解析曲線。可能符號數不變，只有機率改變。

熵對分布是凹函數。混合兩個來源分布、卻隱藏選了哪個來源，熵不會低於個別熵的加權平均。揭露來源選擇可能消除部分增加的不確定性，因此混合來源常比個別來源更難預測。

最大熵界限也能用與均勻模型的相對熵證明。稍後同一非負性論證，還會說明為什麼條件降低平均熵，以及互資訊為什麼不能為負。

聯合熵與鏈式法則

一起描述兩個變數，可能比各自獨立描述便宜。知道第一個後，共享結構可能讓第二個更容易預測。

聯合熵為

$$H(X,Y) = -\sum_{x,y} p_{XY}(x,y)\log_2 p_{XY}(x,y).$$

$H(X,Y)$ 量測成對變數 (X,Y) 的不確定性。條件熵為

$$H(Y \mid X) = -\sum_{x,y} p_{XY}(x,y)\log_2 p_{Y|X}(y \mid x).$$

$H(Y \mid X)$ 是已知 X 後，對 Y 的平均剩餘不確定性；每個條件分布的熵，依對應 x 的機率加權。

利用 $p_{XY}(x,y) = p_X(x)p_{Y|X}(y \mid x)$，可得

$$H(X, Y) = H(X) + H(Y \mid X).$$

每一項必須來自同一聯合模型。乘積取對數產生兩項，第一項對 y 加總後留下 $p_X(x)$。這說明恆等式的來源，不只是要背的公式。

對序列 $X_1, \dots, X_n$：

$$H(X_1, \dots, X_n) = \sum_{i=1}^{n} H(X_i \mid X_1, \dots, X_{i-1}).$$

n 是變數數量，i 是序列索引；第一項沒有條件變數。此法則不需要獨立假設。若獨立，每個條件熵等於邊際熵，聯合熵就是總和。

影像中，直方圖只描述邊際像素頻率，丟掉空間相依性。兩張影像的直方圖相同，知道鄰近像素後的可預測性卻可能很不同。序列模型決定這些相依性是否能縮短描述。

條件平均有幫助，不代表每種結果都如此

知道另一變數，不會使最佳平均預測變差。但某個特定觀測，可能揭露你正處在特別不確定的情況。

$$H(X \mid Y) \le H(X), \qquad H(X, Y) \le H(X) + H(Y).$$

第一式比較平均條件不確定性與邊際不確定性；第二式由鏈式法則得到。X 和 Y 獨立時取等號。

「平均」很重要。假設大部分日子感測器固定輸出零，少數診斷日輸出公平二元結果。知道今天是診斷日，反而比邊際混合分布更不確定。但平均正常日與診斷日後，知道日子類型仍降低平均不確定性。

加入更多條件時：

$$H(X \mid Y, Z) \le H(X \mid Y).$$

Z 是額外資訊。這比較同一固定聯合分布的熵，並不是說有限神經網路多讀輸入就一定泛化更好。最佳化、資料不足與虛假相關，都可能使學得預測器無法達到最佳條件預測。

有限變數的條件熵非負。後續連續變數的微分熵有不同性質；不能不查假設，就把每個有限字母集合不等式套到連續熵。

用數值檢查條件化

讓診斷日例子更具體：$Y = 1$ 表示診斷日，機率 0.1。平常日 $X = 0$，診斷日的 X 是公平位元。因此 $X = 1$ 的邊際機率為 $0.1 \times 0.5 = 0.05$，得到 $H(X) = h_2(0.05) = 0.286397$ 位元。

知道今天是診斷日，會把不確定性提高至 $H(X \mid Y = 1) = 1$ 位元；但平均所有日子得到 $H(X \mid Y) = 0.9 \times 0 + 0.1 \times 1 = 0.1$ 位元。條件不等式比較的是 0.1 與 0.286397，不是每個個別條件熵與邊際熵。日子標籤平均移除 $I(X; Y) = 0.186397$ 位元。

相對熵量測機率失配

模型預期錯誤結果，可能浪費描述長度。相對熵以真正產生觀測的分布為方向，量測這種失配。

$$D(p\|q) = \sum_x p(x) \log_2 \frac{p(x)}{q(x)}.$$

p 和 q 是同一字母集合上的機率質量函數，$D(p\|q)$ 是相對熵，也稱 Kullback–Leibler divergence [1]。$p(x) = 0$ 的項為零；若 $p(x) > 0$ 而 $q(x) = 0$，相對熵無限大。

非負性可由對數凹性得到：

$$\begin{aligned} -D(p\|q) &= \mathbb{E}_p\left[\log_2 \frac{q(X)}{p(X)}\right] \\ &\le \log_2 \sum_{x:p(x)>0} q(x) \le 0. \end{aligned}$$

$\mathbb{E}_p$ 是依 p 取期望值。第一個不等式是 Jensen 不等式，最後加總不超過一。兩分布一致時取等號。若模型在來源支撐集合外也分配機率，此證明仍成立。

方向重要：$D(p\|q)$ 一般不等於 $D(q\|p)$。它不是度量距離，因為不符合對稱及一般三角不等式等要求。口語稱為「距離」，不能據此套用需要真正距離的演算法。

例如公平來源與一個對結果一給 0.9 機率的模型。模型往某方向過度自信，會為常出現卻未預期的零付代價。反轉分布，描述的是不同實驗及不同平均懲罰。

對均勻分布 $u(x) = 1/K$，展開可得 $D(p\|u) = \log_2 K - H(p)$。非負性給出熵上界，也證明固定有限字母集合上，只有均勻來源達到最大熵。

交叉熵連結預測與編碼

交叉熵用提出的機率模型為觀測打分，包含來源不可避免的不確定性，以及模型不準確的代價。

$$H(p,q) = -\sum_x p(x) \log_2 q(x) = H(p) + D(p\|q).$$

$H(p,q)$ 是交叉熵，$H(p)$ 是真實來源熵，q 提供預測機率。理想機率編碼中，$-\log_2 q(x)$ 是描述長度目標；實際編碼還有整數長度、框架、模型描述與實作成本。

p 固定、參數改變 q 時，最小化母體交叉熵等同最小化相對熵。用資料估計時，經驗平均負對數機率是

$$\widehat{H}_N = -\frac{1}{N}\sum_{j=1}^{N} \log_2 q_\theta(x_j).$$

N 是觀測數，x_j 是第 j 個觀測，θ 是模型參數；帽號表示經驗量。它和母體平均的關係，依觀測取樣方式及是否重用來選模型而定。

分類器對已知特徵的標籤使用條件版本，生成模型則對觀測或序列使用這個觀點。低訓練值描述訓練觀測的擬合，不直接證明未來資料表現。這連到ML/DL 評估規則。

訓練用自然對數時，loss 單位是 nat。乘正常數會改變梯度尺度，但不改變數學最佳解。比較論文中的數值，須統一單位、正規化方式及預測目標。

計算例：一張表，三種資訊問題

以下是母體模型，每格表示觀察到該點對的機率。它描述兩個公平位元，但兩者相同的機會高於不同的機會。

聯合機率	$Y=0$	$Y=1$	列總和
$X=0$	0.4	0.1	0.5
$X=1$	0.1	0.4	0.5
欄總和	0.5	0.5	1.0

各列、欄總和給出邊際機率。兩個邊際都公平，因此 $H(X)=H(Y)=1$ 位元。觀察到 $X=0$ 後，把該列除以總和 0.5，得到 Y 的條件機率 0.8 與 0.2。另一列把兩者對調，但熵相同。

$$H(Y\mid X)=0.5h_2(0.2)+0.5h_2(0.2)=0.721928,$$
$$H(X,Y)=1+0.721928=1.721928,$$
$$I(X;Y)=1-0.721928=0.278072.$$

h_2 是二元熵，$H(Y\mid X)$ 平均兩個條件列，$H(X,Y)$ 描述點對，$I(X;Y)$ 則是知道另一位元後，任一位元減少的不確定性。數值單位都是位元，取至小數第六位；下方程式使用未取整數值計算。

把聯合表與每格都是 0.25 的獨立表比較，也得到同樣互資訊。相同位元格的比值是 $0.4/0.25=1.6$，不同位元格是 $0.1/0.25=0.4$；加權對數比總和為 0.278072。單一格的對數比可以負，平均散度卻仍非負。

接著保留公平單位元來源，但用 0.1、0.9 預測。來源熵仍為一位元，交叉熵則成為 $-0.5\log_2(0.1)-0.5\log_2(0.9)=1.736966$ 位元，其中 0.736966 位元是模型失配。聯合表問題量測相依性，預測問題量測錯誤模型；兩者都用散度，卻比較不同分布。

這張表也能解釋分類。預測 $Y=X$ 的正確機率是 0.8，但條件分布仍有 0.721928 位元不確定性。正確率計算決策是否正確，熵評估整個預測分布，兩者不能互相取代。

程式碼：精確機率表檢查

完整 CPU 範例遮掉零機率項，不把它換成小常數。輔助函式也檢查陣列是否為正規化機率。

```
import numpy as np

def probability(a):
    a = np.asarray(a, dtype=float)
    assert np.all(a >= 0) and np.isclose(a.sum(), 1)
    return a

def entropy(a):
```

```
    a = probability(a)
    positive = a > 0
    return -np.sum(a[positive] * np.log2(a[positive]))

def divergence(p, q):
    p, q = probability(p), probability(q)
    assert p.shape == q.shape
    positive = p > 0
    if np.any(q[positive] == 0):
        return np.inf
    return np.sum(p[positive] * np.log2(p[positive] / q[positive]))

joint = probability([[0.4, 0.1], [0.1, 0.4]])
px, py = joint.sum(axis=1), joint.sum(axis=0)
independent = np.outer(px, py)
hx, hy, hxy = entropy(px), entropy(py), entropy(joint)
information = divergence(joint, independent)
print("H(X), H(Y), H(X,Y):", hx, hy, hxy)
print("H(Y|X), I(X;Y):", hxy - hx, information)
assert np.isclose(information, hx + hy - hxy)
assert np.isclose(divergence(independent, independent), 0)

p, q = probability([0.5, 0.5]), probability([0.1, 0.9])
cross = -np.sum(p * np.log2(q))
print("Source entropy / cross-entropy / mismatch:",
      entropy(p), cross, divergence(p, q))
assert np.isclose(cross, entropy(p) + divergence(p, q))
assert np.isinf(divergence(probability([0.5, 0.5]), probability([0, 1])))
```

這裡的恆等式是在指定母體機率表上檢查。若表來自有限樣本，還多了估計不確定性；代數能精確成立，數值估計卻仍可能不準。

互資訊量測相依性

知道一個變數會改變另一個的預期時，兩者共享資訊。這不只限於線性關係。

$$\begin{aligned} I(X;Y) &= D(p_{XY} \| p_X p_Y) \\ &= H(X) + H(Y) - H(X,Y) \\ &= H(X) - H(X \mid Y). \end{aligned}$$

$I(X;Y)$ 是互資訊（mutual information），$p_X p_Y$ 是邊際分布乘積。它非負、對稱，且只有獨立時為零。上界是兩個邊際熵中較小者。

相對熵形式問實際成對分布和獨立模型差多少；熵形式問一變數移除另一變數多少平均不確定性。兩觀點一致，但凸顯不同應用。

相關係數較狹窄：零相關一般不能證明獨立。非線性相依可能很強，特定線性摘要卻為零。另一方面，高維互資訊通常比相關係數更難估準。

條件互資訊為

$$I(X;Y \mid Z) = H(X \mid Z) - H(X \mid Y,Z).$$

Z 是條件資訊。此量平均各條件分布內的相依性，非負，但可能比無條件 $I(X;Y)$ 大，也可能較小。

令 X 與 Y 是獨立公平二元變數，$Z = X \oplus Y$ 是同位元。無條件時，X 對 Y 沒幫助；已知 Z 後，X 就決定 Y。所以無條件互資訊為零，條件值為一 bit。條件能揭露相依性，不只是減去固定資訊量。

資訊鏈式法則為

$$I(X;Y,Z) = I(X;Y) + I(X;Z \mid Y).$$

(Y,Z) 是聯合觀測。第二項是已有 Y 後，Z 額外提供多少資訊。直接相加兩個無條件互資訊，可能重複計入共享證據，或漏掉聯合效果。

互資訊是平均預測改善

利用 $p_{XY}(x,y) = p_X(x)p_{Y|X}(y \mid x)$ 展開聯合分布比值，邊際因子抵消，得到

$$I(X;Y) = \sum_x p_X(x) D(p_{Y|X=x} \| p_Y).$$

$p_{Y|X=x}$ 是觀察 $X = x$ 後的 Y 條件分布，p_Y 是尚未取得該觀測的邊際預測，散度單位為位元。此式把資訊表達成無條件到條件預測的平均改變。

它也證明等號對應獨立。每個條件散度非負，加權總和為零，必須每個正機率 x 都不改變預測；這正是有限表的獨立條件。計算例中，每個條件列與公平邊際的散度都相同，為 $1 - h_2(0.2)$。

對數預測損失下，最佳無條件預測器付出 $H(Y)$，最佳條件預測器付出 $H(Y \mid X)$，差就是互資訊。這是最佳母體分布的陳述；擬合預測器可能因資料、容量或最佳化不足，無法實現好處。推導辨認可得預測利益，也說明經驗估計仍需要另外驗證。

資料處理、可逆性與額外隨機性

如果處理只讀目前觀測，就不能創造關於原始來源的新證據。它能增加隨機變動，但變動不等於來源資訊。

Markov 關係 $X \to Y \to Z$ 表示

$$p(x,y,z) = p_X(x)p_{Y|X}(y \mid x)p_{Z|Y}(z \mid y).$$

Z 由 Y 產生，不額外讀取 X。這推出資料處理不等式（data-processing inequality）：

$$I(X;Z) \le I(X;Y).$$

兩邊都是同一 X 的資訊。這不表示任意隨機處理都有 $H(Z) \le H(Y)$；加入獨立雜訊可能增加輸出熵，同時降低輸入資訊。

一個有用證明是以兩種順序展開 $I(X; Y, Z)$。Markov 關係使 $I(X; Z \mid Y) = 0$；另一順序包含非負的 $I(X; Y \mid Z)$，所以 $I(X; Y) = I(X; Z) + I(X; Y \mid Z)$。取等號表示 Z 沒有丟掉 Y 原本提供的相關 X 資訊。

確定性轉換滿足 $H(f(X)) \le H(X)$，因為它合併可能結果。在支撐集合上一對一，就不損失不確定性。任一變數做可逆轉換，也保留互資訊；多對一量化則可能丟掉相依性。

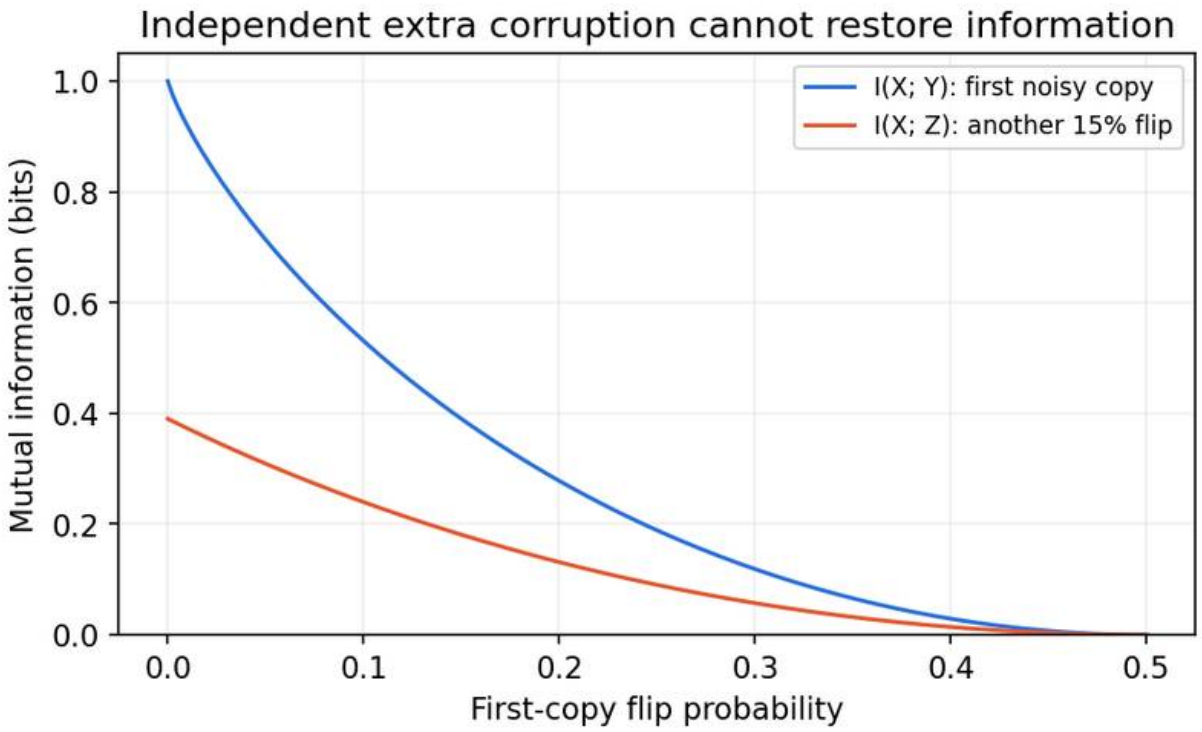


圖 2 — 資料處理的解析例子。第二通道以 *0.15* 機率獨立翻轉輸入；兩條曲線都量測同一原始來源的資訊。

程式碼：處理與同位元

第二個獨立範例建立完整三變數機率表，不使用學得估計器，直接檢查不等式與同位元例子。

```
import numpy as np

def entropy(p):
    p = np.asarray(p, dtype=float)
    assert np.all(p >= 0) and np.isclose(p.sum(), 1)
    q = p[p > 0]
    return -np.sum(q * np.log2(q))

def mutual(joint):
    return entropy(joint.sum(axis=1)) + entropy(joint.sum(axis=0)) - entropy(joint)
```

```
pxy = np.array([[0.4, 0.1], [0.1, 0.4]])
channel = np.array([[0.85, 0.15], [0.15, 0.85]])
pxyz = pxy[:, :, None] * channel[None, :, :]
pxz = pxyz.sum(axis=1)
print("Before / after processing:", mutual(pxy), mutual(pxz))
assert mutual(pxz) <= mutual(pxy) + 1e-12

parity = np.zeros((2, 2, 2))
for x in range(2):
    for y in range(2):
        parity[x, y, x ^ y] = 0.25
pxz, pyz = parity.sum(axis=1), parity.sum(axis=0)
pz = parity.sum(axis=(0, 1))
conditional = entropy(pxz) + entropy(pyz) - entropy(pz) - entropy(parity)
print("Parity I(X;Y) / I(X;Y|Z):",
      mutual(parity.sum(axis=2)), conditional)
assert np.isclose(conditional, 1)
```

估計互資訊違反資料處理時，可能原因包括取樣誤差、不同估計器偏差或 Markov 假設不成立；精確定理本身沒有失效。

估計不只是套恆等式

計算觀測次數容易，判斷未見結果還有多少機率卻不容易。大字母集合與強相依，會讓資訊估計脆弱。

N 個觀測中，結果 x 出現 N_x 次，經驗分布與 plug-in 熵為

$$\hat{p}_N(x) = \frac{N_x}{N}, \qquad \hat{H}_N = -\sum_x \hat{p}_N(x)\log_2 \hat{p}_N(x).$$

N_x 是該結果計數，N 是總數，帽號表示估計。未見結果的經驗機率為零，即使真正機率正值。

固定有限字母集合、獨立取樣且條件適當時，樣本增加會使估計收斂。樣本少時通常低估熵，因為漏掉可能結果，使分布看起來太集中。聯合分布要涵蓋的集合，又比任一邊際大。

量化也定義變數。把連續特徵分成 8 級或 256 級，是不同有限集合問題。更細分格可能提供解析度，也產生更多稀疏格。資訊曲線看起來平滑，不會消除這個取捨。

以下獨立模擬對已知均勻來源做重複合成取樣，檢查 plug-in 估計的平均，不以單次抽樣聲稱信賴程度。

```
import numpy as np

rng = np.random.default_rng(31)
alphabet = 20
truth = np.log2(alphabet)
for n in [40, 400, 4000]:
    estimates = []
    for _ in range(200):
```

```
        sample = rng.integers(alphabet, size=n)
        p = np.bincount(sample, minlength=alphabet) / n
        positive = p > 0
        estimates.append(-np.sum(p[positive] * np.log2(p[positive])))
    print("N:", n, "true entropy:", round(truth, 4),
          "mean estimate:", round(np.mean(estimates), 4))
assert abs(np.mean(estimates) - truth) < 0.02
```

實務上要指定取樣單位、特徵轉換、估計器類型、超參數選擇及不確定性評估。同一錄影的重複影格，不等於獨立場景。先用已知資訊量的分布測估計器，是有用起點。

現代觀點

基本資訊量沒有改變，但現代應用常使用高維學得表示。困難通常是依模型估計或最佳化資訊量，而非寫出定義。

Butakov 等人在 NeurIPS 2024 研究透過 normalizing flow 估互資訊 [3]。方法利用可逆轉換，在假設成立時保留目標資訊、讓估計較容易。換座標能幫助估計器，不等於創造資訊。這是原始方法，不是綜述，也不證明所有神經估計器都無偏。

Aubret、Matignon 與 Hassas 的 2023 綜述 [4] 以**驚訝、新穎性與技能學習**分類內在動機。不可預測觀測、新到訪狀態與可控制技能，是不同目標。本章的解讀是：「最大化熵」必須先指定變數與條件結構，才成為可用設計規則。

El Gamal 2026 的 *Information Theory and Statistical Learning* [5] 連結以散度為基礎的預測及生成學習。它是**建議納入未來第 3 版的草稿章節**，不是已出版的第 3 版，也不取代本主線使用的第 2 版章號。

開放問題包括稀疏高維估計、偏差不同的估計器比較、考慮相依性的評估，以及方便的資訊目標與真正任務之間的關係。方法改善估計下界，不直接證明下游決策改善。

讀這類主張時，可以逐項問：哪些隨機變數？哪個聯合分布？什麼單位？數值是精確量、估計、界限還是最佳化目標？條件獨立假設由什麼支持？這些問題把數學精確性連到有效實驗證據。

重點整理

- 熵是分布下的平均驚訝，不是單一觀測值的性質。
- 鏈式法則處理共享結構，不需要獨立假設。
- 條件降低平均不確定性，個別條件結果可能不同。
- 相對熵有方向、對支撐集合敏感，且非負。
- 交叉熵分開來源不確定性與模型失配。
- 互資訊量測相依，條件與無條件相依可能差很多。
- 資料處理限制固定來源的證據，輸出隨機性卻可能增加。
- 經驗資訊估計繼承取樣、量化與模型假設。

練習

1. 三符號來源機率為 $(1/2, 1/4, 1/4)$，計算各結果驚訝與熵，再比較均勻三符號來源。

提示
用機率加權對數驚訝。

2. 建立均勻邊際、互資訊零的二元聯合表，再建立互資訊一 bit 的表，解釋條件熵。

提示
比較外積與對角表。

3. 展開與 K 符號均勻模型的 $D(p\|u)$，用非負性證明熵上界，寫出取等號條件。

提示
展開為 $\log_2 K - H(p)$。

4. 建立一個具體模型，使某個 y 有 $H(X \mid Y = y) > H(X)$，平均條件不等式仍成立。

提示
用少數公平診斷日和多數確定日。

5. 處理範例再加第三個雜訊通道，逐階檢查互資訊，指出獨立假設。

提示
先乘轉移機率，再消去中間狀態。

6. 固定樣本數，把估計模擬的字母集合放大，區分估計器結論與真正熵結論。

提示
同時記錄未見符號數。

參考文獻

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006，第 1–2、17 章。出版社與目錄。
2. T. Weissman, “Entropy, Relative Entropy, and Mutual Information,” Stanford EE276 Lecture 2 notes, 2024。一手講義。
3. I. D. Butakov, A. D. Tolmachev, S. V. Malanchuk, A. M. Neopryatnaya and A. A. Frolov, “Mutual Information Estimation via Normalizing Flows,” NeurIPS, 2024。會議論文。
4. A. Aubret, L. Matignon and S. Hassas, “An Information-Theoretic Perspective on Intrinsic Motivation in Reinforcement Learning: A Survey,” *Entropy*, 25(2), 327, 2023。已出版綜述。
5. A. El Gamal, “Information Theory and Statistical Learning,” proposed textbook chapter, arXiv preprint, 2026。草稿與出版狀態。

IT 02 ・典型序列、AEP 與熵率

推導長來源區塊為何集中在熵附近，連結典型集與編碼，並區分平穩、獨立、相依與非遍歷來源。

網站版本: https://blog.shyandram.dev/zh-tw/learn/information-theory/it-02-typicality-entropy-rates/

你將學到

- 推導獨立有限字母來源的漸近均分性質。
- 分別限制典型集的機率與大小，不混淆兩者。
- 說明最可能的單一序列為何不一定具有代表性。
- 推導平穩有限狀態 Markov 來源的熵率。
- 用具體反例區分平穩性、獨立性與遍歷性。
- 連結條件典型性、聯合典型性與壓縮及通訊證明。

先看全貌

單次取樣充滿不確定性，長序列的整體行為卻常相當穩定。具有偏差的二元來源，其下一個位元仍無法完全預測，但長區塊的平均描述長度往往接近固定值。典型性把這項觀察寫成精確敘述。

> **用最簡單的話說**
>
> 每次從袋子抽出什麼顏色的珠子都不確定。不過抽很多次之後，大致會有多少種顏色、整串珠子有多令人意外，反而可以預測。多數會遇到的珠串因此能放進比所有可能珠串更小的目錄。

本章依循 Cover 與 Thomas 的第 3–4 章 [1]。Stanford 的 AEP 原始講義 [2] 提供精簡的獨立來源證明。我們會推導結果、說明有限區塊的限制，再延伸至相依來源。後續章節會把存在性論證轉成實際編碼器與解碼器。

典型性永遠相對於指定的來源分布。它不代表外觀普通、語意正確，或對每個族群都有代表性。樣本可能在錯誤的模型下具有高機率，卻仍不符合應用需求。

區塊機率與平均驚訝量

不只問一個符號有多令人意外，而是計算整個區塊的驚訝量，再除以長度。對獨立符號，這就是各符號驚訝量的平均。

設 $X_1, \dots, X_n$ 在有限字母集 $\mathcal{X}$ 上，依機率質量函數 p 獨立同分布。定義區塊 $X^n = (X_1, \dots, X_n)$：

$$p(x^n) = \prod_{i=1}^{n} p(x_i), \qquad J_n = -\frac{1}{n} \log_2 p(X^n) = \frac{1}{n} \sum_{i=1}^{n} -\log_2 p(X_i).$$

其中 n 是區塊長度，x^n 是特定序列，i 是位置，J_n 是以每符號位元數表示的隨機平均驚訝量。等式利用獨立性；其期望為單一來源符號的熵 $H(X)$。

有限字母設定使正機率結果上的驚訝量可積分。字母集中可以包含不可能的符號，但實際取樣區塊不會出現它們。因此大數法則給出

$$J_n \xrightarrow{\Pr} H(X).$$

箭頭表示依機率收斂：對任何固定正容許誤差，J_n 與熵的差超過該誤差的機率會趨近零。這就是**漸近均分性質**（asymptotic equipartition property，AEP）[1,2]。

「均分」是近似且漸近的。它不表示每個區塊都具有相同機率，也不表示所有符號最後都會等頻出現。它表示多數機率質量集中於機率具有近似相同指數尺度的區塊。

對符號一機率為 p 的 Bernoulli 來源，含有 k 個一的區塊滿足

$$-\frac{1}{n}\log_2 p(x^n) = -\frac{k}{n}\log_2 p - \left(1-\frac{k}{n}\right)\log_2(1-p).$$

其中 k 是觀察到的個數，k/n 是經驗比例。當比例接近 p，平均驚訝量接近二元熵 $h_2(p)$。一的個數通常不會接近區塊長度的一半。

典型集：分清機率與計數

多數可能觀察到的結果能放進合理區塊的目錄。目錄內每個區塊的機率可能很低，但加總後涵蓋幾乎全部來源機率。

對容許誤差 $\epsilon > 0$，定義

$$A_\epsilon^{(n)} = \left\{ x^n : \left| -\frac{1}{n}\log_2 p(x^n) - H(X) \right| \le \epsilon \right\}.$$

其中 $A_\epsilon^{(n)}$ 是弱典型集，大括號內的 $|\cdot|$ 表示絕對值，ϵ 是以每符號位元數表示的容許誤差。其元素滿足

$$2^{-n(H(X)+\epsilon)} \le p(x^n) \le 2^{-n(H(X)-\epsilon)}.$$

這限制單一典型區塊的機率。AEP 另外指出 $\Pr(X^n \in A_\epsilon^{(n)})$ 趨近一。

當 n 足夠大，假設典型集機率至少為 $1-\delta$，其中 $\delta > 0$ 是指定的機率容許誤差，則

$$(1-\delta)2^{n(H(X)-\epsilon)} \le |A_\epsilon^{(n)}| \le 2^{n(H(X)+\epsilon)}.$$

其中 $|A_\epsilon^{(n)}|$ 表示典型序列的個數，而非其機率。上界來自每個典型區塊至少具有 $2^{-n(H+\epsilon)}$ 的機率，且總機率不能超過一。下界來自集合至少涵蓋 $1-\delta$ 的機率，而各區塊機率至多為 $2^{-n(H-\epsilon)}$。

把 ϵ 與 δ 分開，可以看清角色：前者限制描述長度的偏差，後者限制被排除的機率質量。教科書可能在選擇足夠大的區塊後，用同一個小參數表示兩者；有限長度計算仍必須區分。

總共有 $|\mathcal{X}|^n$ 種可能序列。若 $H(X) < \log_2 |\mathcal{X}|$，典型目錄的指數尺度就小得多。這是壓縮背後的計數事實：若允許很小的錯誤機率，就不必讓每個可想像的區塊都有短描述。

最可能的區塊不一定典型

單一機率最高的字串可能沒有代表性。許多單獨機率稍低的字串，加起來反而占大多數機率。

對 $p = 0.2$ 的獨立二元來源，全零區塊是單一機率最高的區塊。其平均驚訝量為

$$-\frac{1}{n}\log_2(0.8^n) = -\log_2(0.8).$$

其中 0.8 是零的機率。這個值與 $h_2(0.2)$ 不同。對較窄的容許誤差，全零字串一直在典型集外，即使沒有任何單一字串比它更可能。

它的總機率 0.8^n 趨近零；反之，約五分之一符號為一的區塊有非常多個，加總機率趨近一。機率質量與單一結果排名是不同的對象。

生成模型也有類似區別：最大機率序列不一定像模型的代表性樣本。典型性提供數學例子，卻不保證某個取樣演算法會產生有用語言或影像。

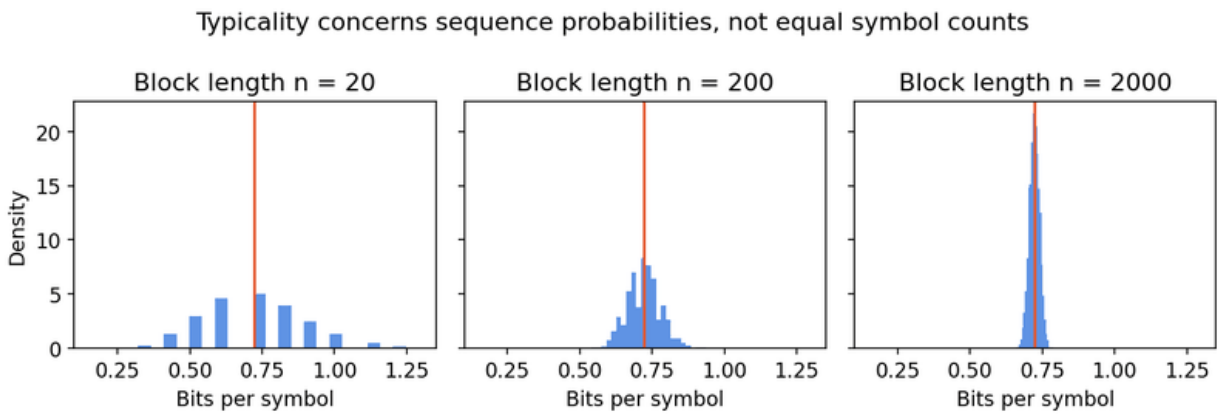


圖 *1* — 每個長度產生 *5,000* 個合成二項取樣區塊。橘線是精確來源熵；橫軸是驚訝量，不是一的比例。

程式：精確典型機率與有限個數

第一個完整例子利用二項計數分布，將典型集機率加總至浮點運算精度。另用小區塊枚舉檢查基數界限。

```
import itertools
import numpy as np
from scipy.stats import binom

p, epsilon = 0.2, 0.1
entropy = -p * np.log2(p) - (1 - p) * np.log2(1 - p)
for n in [20, 200, 2000]:
    k = np.arange(n + 1)
    per_symbol = -(k / n) * np.log2(p) - (1 - k / n) * np.log2(1 - p)
    typical = np.abs(per_symbol - entropy) <= epsilon
```

```
    probability = binom.pmf(k, n, p)[typical].sum()
    print("Length:", n, "typical probability:", probability)

n, epsilon = 10, 0.15
count, mass = 0, 0.0
for block in itertools.product([0, 1], repeat=n):
    k = sum(block)
    probability = p**k * (1 - p)**(n - k)
    surprise = -np.log2(probability) / n
    if abs(surprise - entropy) <= epsilon:
        count += 1
        mass += probability
assert count <= 2**(n * (entropy + epsilon))
assert count >= mass * 2**(n * (entropy - epsilon))
print("Small-block typical count / mass:", count, mass)
```

小區塊的典型機率不一定接近一。「漸近」描述固定容許誤差下，區塊長度持續增加的行為，不代表任何長度都能套用長區塊結論。

計算例：先數字串，再加總機率

使用上方小區塊枚舉的 $n = 10$、$p = 0.2$、$\epsilon = 0.15$。有 k 個一的區塊，其每符號平均驚訝量為 $0.321928 + 0.2k$ 位元，來源熵則為 0.721928。只有 $k = 2$ 符合容許誤差；我們先決定成員，再進行計數。

這些字串有 $\binom{10}{2} = 45$ 個。每個機率為 $0.2^2 \times 0.8^8 = 0.0067108864$，所以典型集機率是 $45 \times 0.0067108864 = 0.301989888$。相較之下，單一全零字串機率為 $0.8^{10} = 0.1073741824$。它比每個典型字串更可能，但比 45 個字串合計的機率低。

這分別回答三個問題：**成員資格**由驚訝量容許誤差決定，**基數**計算接受字串數，**機率質量**加總其機率。45 個字串的目錄需要六個定長索引位元，因為五位元只能區分 32 種選項。但該目錄只涵蓋這個短區塊來源約 30.2% 的機率。因此，即使碼率是每符號 0.6 位元，僅靠長度十目錄仍不是好的幾乎無損編碼。

熵界限沒有失效。固定此容許誤差時，高機率結論需要更長區塊。基數取整與排除機率，是極限論證不會從短實驗中消除的成本。

若改用公平位元模型評分，同一批區塊全部恰好每符號一位元。該模型無法用驚訝量區分全零與兩個一的字串。真正來源未變，改變的是評分模型。計算典型集時，必須指定由哪個分布提供機率。

經驗頻率與組合計數直覺

區塊的符號頻率定義一群具有相同來源機率的字串。計算這一群的大小，就能說明單獨不太可能的字串為何能共同占大多數機率。

對恰好有 k 個一的二元區塊，排列數與該計數的總機率為

$$N_{n,k} = \binom{n}{k}, \qquad \Pr(K_n = k) = \binom{n}{k} p^k (1-p)^{n-k}.$$

其中 $N_{n,k}$ 計算字串個數，K_n 是隨機的一的個數，$\binom{n}{k}$ 是二項係數，p 是來源產生一的機率。同一計數的每種排列機率相同，但排列數差異非常大。

兩個極端分別只有一個全零與一個全一字串；中間則有大量排列。對偏差來源，每種排列的機率與排列數朝不同方向變化，乘積在接近來源期望計數處達到高值。

這也說明經驗頻率典型性與驚訝量典型性的差別。公平二元來源的每個長度 n 字串機率都是 2^{-n}，平均驚訝量恰好一位元。因此任何正容許誤差下，所有可能字串都是弱典型，包括全零。

觀察到全零字串的機率仍趨近零。以頻率定義的典型集可要求一的比例接近一半，從而排除全零。它的機率也趨近一，但成員條件更嚴格。

兩種定義都沒有錯；它們用於不同證明。機率尺度敘述可能只需要弱典型性；需要匹配經驗條件分布的構造，可能必須控制頻率。務必確認證明實際使用哪個定義，尤其從邊際來源編碼轉向聯合輸入輸出關係時。

來源編碼論證與限制

將典型區塊編號，傳送其目錄索引。區塊夠長時，平均每符號使用接近熵的位元數，只在少量被排除的結果上失敗。

典型集至多有 $2^{n(H+\epsilon)}$ 個成員，因此索引大致需要不超過 $n(H+\epsilon)$ 位元，再加取整與區塊框定的成本。解碼器用相同目錄重建所選區塊。

這是**幾乎無損的定長**構造，並未正確編碼所有區塊。可以保留一個額外符號表示失敗，但旗標不能重建非典型區塊。要真正無損，就必須提供足以完整還原它的逸出表示。

簡單的反向論證可說明，遠低於熵的速率為何無法涵蓋多數機率。假設編碼最多正確表示 2^{nR} 個區塊，將它們與典型集取交集。每個典型區塊機率至多為 $2^{-n(H-\epsilon)}$，可接受機率質量至多為

$$2^{nR} 2^{-n(H(X)-\epsilon)} = 2^{-n(H(X)-\epsilon-R)}.$$

其中 R 是以每來源符號位元數表示的定長編碼速率。若 $R < H(X) - \epsilon$，典型集內可接受的機率趨近零。被排除的非典型機率也趨近零，因此編碼無法以高機率重建來源。

論證假設編碼器與解碼器只有有限個不同訊息。這是計數證明，不是有效率的實作；枚舉巨大目錄通常不實際。下一章研究前綴編碼、機率編碼與自適應模型，把理論速率轉成可計算程序。

加上逸出分支，讓目錄真正無損

有時失敗的目錄，可以變成還原每個允許有限區塊的碼。先傳一位元分支旗標；區塊典型時傳定長目錄索引，否則傳逐符號固定寬度的未壓縮表示。

令 $m = \lceil \log_2 |\mathcal{X}| \rceil$，非空典型集的索引寬度為 $\ell_n = \lceil \log_2 |A_\epsilon^{(n)}| \rceil$。期望長度滿足

$$\frac{\mathbb{E}[L_n]}{n} = \frac{1}{n} + \frac{P_T \ell_n}{n} + (1 - P_T) m \le \frac{2}{n} + P_T (H(X) + \epsilon) + (1 - P_T) m.$$

L_n 是區塊總位元長度，m 是原始符號固定寬度，ℓ_n 是典型索引寬度，$P_T = \Pr(X^n \in A_\epsilon^{(n)})$ 是典型機率，$n, \epsilon, H(X)$ 沿用前述定義。等式計入旗標與兩分支；不等式利用典型集基數與向上取整。

解碼器先讀旗標，便知道該讀哪個固定分支長度。典型分支回傳共享目錄項目，逸出分支還原原始符號，沒有區塊被默默省略。小有限區塊的典型集若為空，先只用原始分支，直到有可用目錄。

固定 ϵ 下，AEP 給 $P_T \to 1$，使逸出分支的每符號貢獻消失。旗標與取整成本也隨 n 增長而消失，期望速率因此漸近不超過 $H(X) + \epsilon$。先建立足夠長區塊，再選較小容許誤差，便能任意接近熵。

構造揭露定長幾乎無損與變長完全無損的差別。逸出分支在罕見區塊可能很長，後者結論因此針對**期望長度**，不是每個區塊都有短上界。兩端還須共享目錄與來源模型。這是速率可達證明，明確枚舉通常太昂貴，不是實用壓縮演算法。

有限區塊集中速度取決於來源

長區塊很有用，但需要多長，取決於來源與容許誤差。稀有符號可能造成很大的驚訝量波動。

對驚訝量變異數有限的獨立符號，

$$\operatorname{Var}(J_n) = \frac{V}{n}, \qquad V = \operatorname{Var}[-\log_2 p(X)].$$

其中 V 是來源驚訝量變異數，Var 表示變異數。結果來自獨立變異數相加再除以 n^2。一般變異數界限給出

$$\Pr(|J_n - H(X)| > \epsilon) \le \frac{V}{n\epsilon^2}.$$

其中 ϵ 是前述驚訝量容許誤差。界限可能寬鬆，但明確顯示依賴關係。很小的機率可能提高驚訝量變異數；要求更窄誤差也會增加所需長度。

跨區塊的經驗標準差有用，卻不是通用的有限區塊編碼保證。相依性改變變異數計算；小規模模擬可能漏掉稀有事件；模型不匹配會改變集中位置。

若來源 p 由模型 q 評分，在可積分條件下，獨立區塊平均驚訝量收斂至 $H(p, q)$，而不是 $H(p)$。兩者的差是相對熵。對模型典型與對真實來源典型，是分開的問題。

熵率考慮相依性

鄰近符號常能幫忙互相預測。因此，長序列每符號需要的資訊，可以小於單一邊際符號的熵。

對隨機程序 $X_1, X_2, \ldots$，若極限存在，定義熵率為

$$\bar{H}(X) = \lim_{n \to \infty} \frac{1}{n} H(X_1, \ldots, X_n).$$

其中 $\bar{H}(X)$ 是每符號平均新增的不確定性，n 是區塊長度。定義討論區塊熵，不是擬合模型在某一條已實現序列上的損失。

鏈式法則給出

$$\frac{1}{n}H(X^n) = \frac{1}{n}\sum_{i=1}^{n} H(X_i \mid X^{i-1}).$$

其中 $X^{i-1} = (X_1, \dots, X_{i-1})$ 是先前歷史，$i = 1$ 時為空。相依性使條件項不同於邊際熵。

當所有時間指標共同平移後，每個有限維聯合分布都不改變，程序稱為**平穩**。這比平均值或共變異數固定更強。有限字母的平穩來源，其給定更長過去的條件熵不增加，而且以零為下界，故極限存在；區塊平均熵有相同極限。

兩種極限相等，是因為對收斂的條件熵序列取平均。平穩性允許將固定長度歷史移到共同時間點，而增加過去資訊會降低條件熵。

獨立同分布是特例：每個條件熵都等於 $H(X_1)$。相依的平穩來源可以在每個時間擁有相同邊際分布，但每一步需要的新資訊少得多。

Markov 來源的明確計算

一階 Markov 模型只需目前狀態即可預測下一個狀態。長期不確定性是各轉移列不確定性的加權平均。

設有限 Markov 來源的轉移矩陣為 P：

$$P_{ij} = \Pr(X_{t+1} = j \mid X_t = i), \qquad \pi_j = \sum_i \pi_i P_{ij}.$$

其中 i, j 是來源狀態索引，t 是時間，P_{ij} 是轉移機率，π 是平穩狀態分布。P 每列總和為一；π 非負且總和為一。

若程序以 π 初始化，平穩性與 Markov 性質給出

$$\bar{H}(X) = H(X_{t+1} \mid X_t) = -\sum_i \pi_i \sum_j P_{ij} \log_2 P_{ij}.$$

內部總和是轉移列 i 的熵，外部總和按平穩占有率加權。有限區塊熵為

$$H(X^n) = H(\pi) + (n-1)\bar{H}(X).$$

其中 $H(\pi)$ 是初始狀態熵。鏈式法則貢獻一個初始項與 $n-1$ 個相同轉移項。

對翻轉機率為 a 的對稱二元來源，平穩分布均勻。邊際熵為一位元，熵率則是 $h_2(a)$。當 $a = 0.1$，下一符號多半重複目前符號，因而減少新增不確定性。

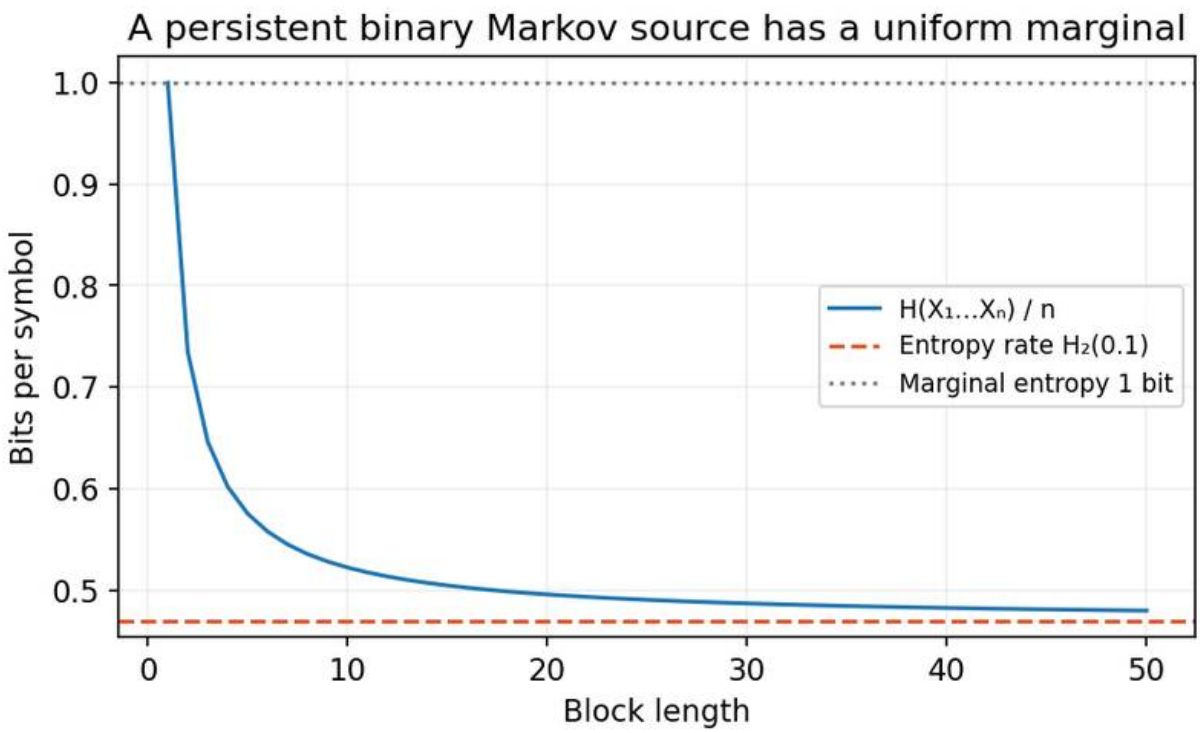


圖 2 — 平穩二元 *Markov* 來源的解析例子。單一符號頻率均勻，不代表來源獨立。

程式：邊際不變，預測性不同

獨立的翻轉事件產生平穩二元 Markov 區塊，再將觀察到的平均驚訝量與精確區塊熵公式比較。

```
import numpy as np

rng = np.random.default_rng(32)
a = 0.1
rate = -a * np.log2(a) - (1 - a) * np.log2(1 - a)
for n in [64, 256, 1024]:
    initial = rng.integers(2, size=1000)
    flips = rng.binomial(1, a, size=(1000, n - 1))
    states = np.column_stack([
        initial, initial[:, None] ^ np.bitwise_xor.accumulate(flips, axis=1)])
    surprise = 1 + (-flips * np.log2(a) - (1 - flips) * np.log2(1 - a)).sum(axis=1)
    expectation = (1 + (n - 1) * rate) / n
    print("Length:", n, "one-frequency:", states.mean(),
          "mean surprise rate:", surprise.mean() / n,
          "exact entropy per symbol:", expectation)
    assert abs(states.mean() - 0.5) < 0.08
    assert abs(surprise.mean() / n - expectation) < 0.02
print("Entropy rate:", rate, "marginal entropy:", 1.0)
```

初始狀態均勻取樣，因此來源平穩。每個區塊固定從零開始會改變有限區塊分布；適當轉移假設下漸近速率仍可能相同，但精確的初始熵項會改變。

邊際公平，為什麼仍容易預測？

翻轉機率 $a = 0.1$ 的對稱 Markov 例子，第一位元有一位元不確定性，後續每個位元貢獻 $h_2(0.1) = 0.468996$ 位元。長度十的區塊熵為 $1 + 9h_2(0.1) \approx 5.220960$ 位元，即每符號 0.522096 位元；十個獨立公平位元則有十位元區塊熵。

可以用初始位元加九個翻轉指示表示整段序列。這個映射可逆：從第一位元開始，依序與翻轉指示做互斥或，就恢復下一狀態。它揭露相依性，沒有捨棄來源資訊。在此特定模型下，指示是有偏的獨立位元，因此比十個邊際公平狀態更容易編碼。

長期速率趨近 0.468996，因為初始化的一位元成本被越來越長的區塊攤提。可預測性來自轉移關係，即使每個時間位置的邊際都公平。影像直方圖也無法揭露利用鄰近像素預測所能獲得的全部節省。

隱藏狀態與觀測對象

轉移模型可能描述內部狀態，接收端卻只看得到帶雜訊的測量。觀測序列因此有自己的預測問題。

設 S_t 是有限隱藏 Markov 狀態，Y_t 是發射觀測，模型可將區塊分解為

$$p(s^n, y^n) = p(s_1) \prod_{t=1}^{n-1} p(s_{t+1} \mid s_t) \prod_{t=1}^{n} p(y_t \mid s_t).$$

其中 s^n 是狀態序列，y^n 是觀測序列，各因子描述轉移與發射。對隱藏序列加總後得到觀測概似 $p(y^n)$。

Y 的熵率通常不只是按平穩分布平均的發射列熵。過去觀測能幫助推論隱藏狀態，剩餘狀態不確定性也影響預測。相關項是 $H(Y_t \mid Y^{t-1})$，而不只是 $H(Y_t \mid S_t)$。

對視覺而言，物件動態模型與相機測量模型描述不同的隨機程序。更多感測雜訊可能提高觀測熵，同時降低關於場景的有用資訊。速率計算必須說明究竟描述哪個程序。

共同典型值需要遍歷性

平穩表示分布不隨時間漂移，卻不保證一條長序列會探索混合族群的全部部分。

在適當的有限字母平穩遍歷假設下，長區塊正規化驚訝量收斂至共同的熵率值。遍歷性排除某個持續存在的隱藏選擇，把程序分成行為不同的平穩成分。

考慮只選一次成分 C：一半機率永遠產生零，一半機率產生獨立公平位元。每個成分平穩，混合也平穩。

條件區塊熵為 $H(X^n \mid C) = n/2$。恆等式

$$H(X^n) = H(X^n \mid C) + I(C; X^n)$$

使用成分標籤 C、區塊 X^n 與互資訊 I。因為 $I(C; X^n) \le H(C) = 1$，區塊平均熵趨近半位元。

然而，確定成分產生的已實現序列，其正規化驚訝量趨近零；公平成分的序列通常趨近一。多數實現不會共享接近一半的單一值。僅有熵率極限，並不足以套用遍歷 AEP 結論。

程式：平穩但非遍歷的反例

這個完整例子從一的個數解析計算混合區塊機率，避免先計算極小公平區塊機率，再取對數。

```
import numpy as np

rng = np.random.default_rng(33)
for n in [128, 1024]:
    component = rng.integers(2, size=2000)
    counts = rng.binomial(n, 0.5, size=2000) * component
    # A nonzero sequence can only come from the fair component:
    # its probability is 0.5 * 2**(-n), so surprise is n + 1.
    all_zero_surprise = -np.log2(0.5 + 0.5 * np.exp2(-float(n)))
    per_symbol = np.where(counts == 0, all_zero_surprise, n + 1) / n
    print("Length:", n, "deterministic mean:", per_symbol[component == 0].mean(),
          "fair mean:", per_symbol[component == 1].mean(),
          "fraction within 0.1 of 0.5:", np.mean(abs(per_symbol - 0.5) <= 0.1))
    assert np.mean(abs(per_symbol - 0.5) <= 0.1) == 0
```

例子描述持續的異質性，而不是時間漂移。資料集包含不同來源類型時，可能需要條件模型或明確混合，不能假設一段長紀錄代表整個族群。

聯合與條件典型性

兩條各自普通的序列，配在一起可能非常異常。通訊證明需要輸入與輸出的關係，不能只看各自的直方圖。

從聯合機率質量函數 p_{XY} 獨立取樣配對 (X_i, Y_i)，考慮聯合與條件平均驚訝量：

$$-\frac{1}{n}\log_2 p(X^n, Y^n) \to H(X, Y), \qquad -\frac{1}{n}\log_2 p(Y^n \mid X^n) \to H(Y \mid X).$$

其中箭頭表示在指定獨立配對模型下依機率收斂。X^n, Y^n 是配對區塊，每對內部允許相依。

聯合典型配對依照來源的聯合關係檢查，只有邊際典型性並不足夠。若 Y 是 X 的帶雜訊副本，兩條無關的公平位元序列可以各自有典型邊際計數，卻具有不符模型的差異頻率。

區塊資訊密度為

$$\imath(x^n; y^n) = \log_2 \frac{p(x^n, y^n)}{p(x^n)p(y^n)}.$$

其中 ι 衡量聯合配對相對於獨立配對模型的證據。在獨立配對取樣及可積分條件下，其正規化平均趨近 $I(X;Y)$。

這連結通道解碼：接收端尋找與收到區塊具有通道預期關係的碼字。它也連結配對與表徵學習，正配對必須仔細定義。從錯誤族群抽取負配對，會改變參照分布。

有限字母的強典型性另外限制各符號或配對的經驗頻率，並尊重不可能事件；弱典型性限制平均對數機率。若證明假設使用特定定義，不能在未說明下把兩者互換。

現代觀點

平均驚訝量與有意義輸出的區別，對現代機率模型尤其重要。長上下文與學習式預測器能描述固定 Markov 階數以外的相依性，損失仍是在指定觀測與詞元化模型下評分。

El Gamal 於 2026 年提出的學習章節 [3]，以散度連結預測與生成模型。它是草稿補充教材，不是已出版的第三版。從來源編碼觀點推論，序列概似應連同預測目標、單位與條件上下文一起報告。

Huang 與 Wu 的 2024 年綜述 [4] 依人類重建與機器分析的區別，整理學習式影像壓縮。這個分類凸顯壓縮表示是否有用，取決於接收者。熵率給出來源描述的限制，卻不單獨指定任務可接受的失真。

許多應用仍面對三個問題：實際區塊長度下，漸近行為多快才有用；擬合來源模型跨族群是否仍有效；以及取樣、詞元化、前處理造成的相依性，如何影響估計速率。

有限評估應比較獨立區塊或取得資料的群組，並保留非典型案例。只報告平均驚訝量，可能掩蓋速率高得多的稀有來源成分。在一個模型下測量概似，不能證明所有來源成分共享同一典型集。

下一章會實際搭起橋梁：編碼器必須定義機率模型、更新規則、終止慣例，以及未預期符號的處理。速率定理引導選擇，完整編碼再解碼的測試則檢查實作是否還原原始資料。

重點整理

- AEP 在指定來源假設下，讓正規化區塊驚訝量集中。
- 典型集機率與基數是不同量，具有不同界限。
- 單一機率最大的區塊，不一定占大多數總機率。
- 典型目錄說明幾乎無損編碼限制，不是有效率的實作。
- 熵率衡量利用相依性後，每一步新增的不確定性。
- 平穩性不保證遍歷性，也不保證共同的已實現驚訝量。
- 聯合典型性檢查邊際典型性可能漏掉的關係。
- 漸近主張必須伴隨有限區塊、模型不匹配與異質性的討論。

練習

1. 對 $p = 0.1$ 的 Bernoulli 來源，計算全零區塊的驚訝量率與來源熵，選一個能在所有長度排除全零的容許誤差。

提示

比較 $-\log_2(0.9)$ 與 $h_2(0.1)$。

2. 以容許誤差 0.05、0.1、0.2 重跑精確典型機率計算，說明較窄容許誤差為何可能需要更長區塊。

提示

平均驚訝量視窗縮窄。

3. 用不同的 ϵ 與 δ 推導典型集基數上下界，指出每一步使用哪個單一區塊機率界限。

提示

以區塊數乘最大或最小允許機率，限制總和。

4. 二元 Markov 鏈轉移矩陣為 $P = \begin{pmatrix} 0.9 & 0.1 \\ 0.3 & 0.7 \end{pmatrix}$。求平穩分布、邊際熵與熵率。

提示

解 $\pi = \pi P$，再加權轉移列熵。

5. 將非遍歷混合例子的成分機率改為四分之一確定來源、四分之三公平來源，求區塊熵率極限與各實現的驚訝量率極限。

提示

持續存在的成分仍阻止共同實現值。

6. 構造兩條邊際典型公平位元序列，使它們在 10% 翻轉通道下不是聯合典型。說明個別符號計數為何無法發現不匹配。

提示

使用差異過多或過少的配對。

參考文獻

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapters 3–4. 出版社書籍與目錄.
2. T. Weissman, “Asymptotic Equipartition Property,” Stanford EE276 Lecture 3 notes, 2024. 原始課程講義.
3. A. El Gamal, “Information Theory and Statistical Learning,” proposed textbook chapter, arXiv preprint, 2026. 草稿與出版狀態.
4. C.-H. Huang and J.-L. Wu, “Unveiling the Future of Human and Machine Coding: A Survey of End-to-End Learned Image Compression,” *Entropy*, 26(5), 357, 2024. 已出版綜述.

IT 03 ・無損編碼與通用壓縮

建立可解碼的 Huffman 與算術編碼例子，推導熵限制，並連結自適應通用編碼、預測與演算法複雜度。

網站版本: https://blog.shyandram.dev/zh-tw/learn/information-theory/it-03-lossless-universal-coding/

你將學到

- 區分單射、唯一可解碼與前綴自由描述。
- 利用 Kraft–McMillan 不等式推導熵下界。
- 建立 Huffman 編碼並測試精確重建。
- 解釋算術編碼區間，實作短訊息的精確算術往返。
- 連結自適應機率預測、通用編碼與模型額外成本。
- 區分可計算壓縮與理想的演算法複雜度。

先看全貌

壓縮用較短、接收者能解讀的表示取代訊息。關鍵是**能解讀**。若串接後的字串產生歧義，只替每個符號分配短字串還不夠。有用的編碼器必須指定訊息如何開始與結束、機率如何取得，以及接收端如何完全反轉表示。

> **用最簡單的話說**
>
> 你和朋友替玩具取綽號，常見玩具用短名字。但如果兩個綽號連在一起，看起來像另一個綽號，朋友就無法正確開箱。壓縮規則既需要短名字，也需要沒有歧義的閱讀方式。

閱讀對照是 Cover 與 Thomas 的第 5 章無損編碼、第 13 章通用來源編碼，以及第 14 章 Kolmogorov 複雜度 [1]。Stanford 的變長編碼與 Huffman 原始講義 [2,3] 提供易讀推導。本章使用原創例子，不重製書中表格或練習。

前章描述幾乎無損的典型集目錄；這裡要求精確重建每個允許訊息。成本包括未預期符號的逸出表示，以及辨識模型所需的資訊。後續率失真理論會明確放寬重建要求，不能將它混入無損主張。

無損編碼必須保留什麼

壓縮表示必須識別原始訊息，包括邊界。兩個不同訊息不能變成相同的傳送物件。

編碼器 f 與解碼器 g 滿足

$$g(f(x)) = x$$

對每個允許的來源訊息 x 成立。其中 $f(x)$ 是表示，g 是重建規則；這要求 f 在允許訊息集合上為單射。

逐符號編碼以串接各符號碼字表示來源字串。**唯一可解碼**的編碼讓不同來源序列的串接結果都不同。**前綴自由**編碼則沒有任何碼字以另一個完整碼字開頭；走到碼樹葉節點時，就能立即解碼。

前綴自由必然唯一可解碼，反向卻不一定成立。孤立符號的單射映射，甚至不保證串接後唯一可解碼。例如三個符號分別使用 0、1、01，第三個符號便無法與第一個接第二個區分。

提供分隔符可以修補歧義，但分隔符也占傳輸空間。提供訊息長度有助於判定終止，長度本身卻不會修補符號剖析歧義。解碼器使用的任何側資訊都屬於通訊協定，必須明確計入。

若來源只有一個已知符號且外部提供長度，空碼字可能成立；它不適用於下方一般多符號碼樹。例子為每個符號至少分配一位元，並假設兩端共享字母集、樹與訊息邊界。

Kraft–McMillan：碼樹的機率預算

很短的碼字占用二元碼樹的大部分空間。短碼字太多，就沒有空間放剩餘符號。

對碼長為整數 ℓ_i 的二元唯一可解碼編碼，

$$K = \sum_i 2^{-\ell_i} \le 1.$$

其中 i 是來源符號索引，ℓ_i 是符號 i 的碼字位元數，K 是 Kraft 總和。反之，滿足不等式的有限非負整數碼長配置，都能實現為前綴碼，單符號情況依通常慣例處理 [1,3]。

在前綴樹中，長度 ℓ 的葉節點，在任何足夠深的層級占全部碼樹的 $2^{-\ell}$。前綴自由使占用後代互不重疊，因此比例總和不能超過一。延伸到唯一可解碼情況，需要跨串接碼字的計數論證，不能直接把樹圖套在非前綴碼上。

不等式測試碼長是否存在，不給出唯一碼樹、位元串流格式，也不保證實際來源下的效率。許多合法碼樹的期望長度仍不必要地偏大。

熵下界

可逆二元描述的平均長度不能短於來源不確定性。看似更好的速率，通常表示成本或假設改變了。

設 p_i 是來源機率，並定義碼長誘導的正規化機率 $q_i = 2^{-\ell_i}/K$，則

$$\mathbb{E}[\ell(X)] = H(p) + D_{\mathrm{KL}}(p\|q) - \log_2 K \ge H(p).$$

其中 X 是隨機來源符號，$\mathbb{E}$ 表示期望，$H(p)$ 是熵，D_{KL} 是以位元表示的相對熵，K 是 Kraft 總和。兩個額外項都非負，因為散度非負且 $K \le 1$。

等式把低效率拆成分布不匹配與未使用的碼樹容量。完整樹滿足 $K = 1$；但 p 與整數碼長誘導的二元冪機率不匹配，仍會造成額外成本。機率符合適當完整二元樹時，才可能達到相等。

選擇 $\ell_i = \lceil -\log_2 p_i \rceil$ 滿足 Kraft 不等式，並給出

$$H(X) \le \mathbb{E}[\ell(X)] < H(X) + 1.$$

上取整符號表示向上取整數。嚴格上界適用於正機率的有限符號及通常非退化慣例。額外位元是每個編碼單位的成本；把長度 n 的來源區塊當單位，每符號取整成本就降至小於 $1/n$。

區塊編碼並非免費：區塊字母集可能指數成長，儲存表格昂貴，延遲也增加。算術編碼不用替每個可能長訊息指定獨立固定樹葉。

Huffman 編碼：合併最不可能的選項

不斷把兩個最不可能的符號合併成暫時符號。逆轉這些合併就得到二元樹，常見符號的路徑比較短。

Huffman 編碼將

$$L = \sum_i p_i \ell_i$$

在指定有限來源分布的二元前綴碼中最小化。其中 L 是期望長度，p_i 是符號機率，ℓ_i 是葉深度。最佳性只針對這個模型與逐符號編碼類別，不涵蓋所有區塊或上下文相依編碼器 [1,3]。

最佳碼樹可以讓兩個最低機率符號位於最深的兄弟葉節點。把較小機率交換到較長路徑，不會增加期望長度。合併這兩個節點後，最佳化轉成較小字母集；遞迴求解就得到貪婪演算法。

相同機率可能產生多棵同樣最佳的樹。若兩端獨立重建樹，確定性的平手規則很重要。實作用遞增計數器，讓堆積比較沒有歧義，也不用比較異質的樹物件。

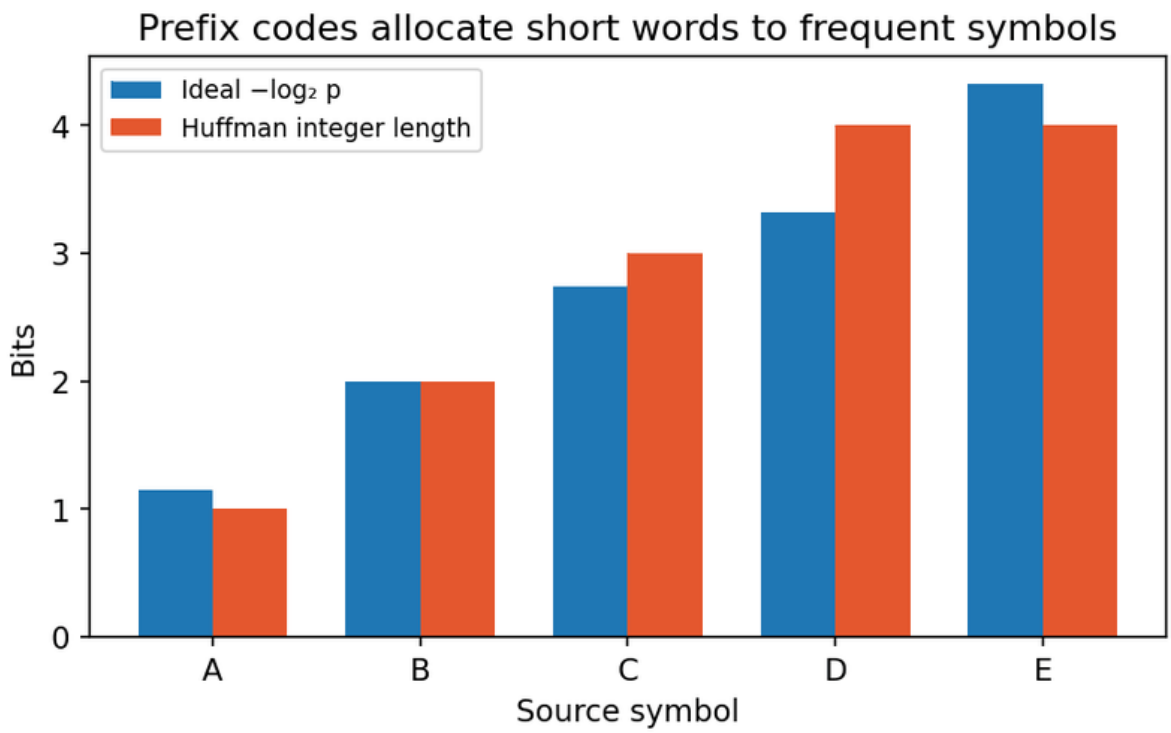


圖 *1* — 機率為 *0.45*、*0.25*、*0.15*、*0.10*、*0.05*。整數符號碼長近似理想驚訝量，但不必逐符號相等。

貪婪合併為什麼保留最佳性？

交換論證可以直接計算成本。若 $p_i \geq p_j$，但符號 i 比 j 深，交換葉標籤後的期望長度變化為

$$\Delta L = (p_i - p_j)(\ell_j - \ell_i) \leq 0.$$

p_i, p_j 是兩機率，$\ell_i > \ell_j$ 是原深度，ΔL 是交換後減交換前的成本。高機率符號放得比低機率深，從來不會有好處。

最佳二元樹可選最深葉為兄弟節點，否則多餘單分支能縮短。交換論證允許把兩個最低機率放到此兄弟葉，不增加成本。將它們替換為機率 $p_i + p_j$ 的暫時符號，深度比原葉少一，因此

$$L_{\text{original}} = L_{\text{merged}} + (p_i + p_j).$$

兩個 L 是合併前後期望長度。增加項固定，不依其餘縮小樹如何安排。因此最佳縮小樹給最佳原樹，假如原樹還有更好解，也會導出更好的縮小樹。反覆使用此論證，才是貪婪演算法的理由，不只是觀察輸出看起來合理 [1,3]。

手算五符號碼樹

機率為 0.45、0.25、0.15、0.10、0.05 時，合併順序容易手算。先合併 0.05 與 0.10，得到 0.15；再把暫時權重與原本的 0.15 合併為 0.30。接著合併 0.25 與 0.30 為 0.55，最後合併 0.45 與 0.55，得到一。

符號碼長等於葉節點到根之間經過的合併次數，結果是 1、2、3、4、4。以下是一個合法配置：

符號	機率	碼字	長度
A	0.45	`0`	1
B	0.25	`10`	2
C	0.15	`110`	3
D	0.10	`1110`	4
E	0.05	`1111`	4

這組碼字前綴自由：讀到 `0` 就確定是 A，讀到 `111` 則還需要一位元才能區分 D 與 E。交換左右子樹會改變碼字，但保留長度與期望成本。

Kraft 總和為 $1/2 + 1/4 + 1/8 + 1/16 + 1/16 = 1$。期望長度為 $0.45 \times 1 + 0.25 \times 2 + 0.15 \times 3 + 0.10 \times 4 + 0.05 \times 4 = 2$ 位元。來源熵約 1.977235 位元，因此逐符號碼每符號付出約 0.022765 位元的取整失配。五個符號也可使用定長三位元碼，但相較此期望長度，每符號多花一位元。

A 的一位元描述比理想驚訝量 $-\log_2(0.45)$ 短，其他符號會補償。定理限制機率加權平均，不要求每個碼字都至少等於自己的驚訝量。下方程式可能為平手節點選不同位元標記，但會得到同樣碼長與兩位元期望。

程式：建樹、編碼、解碼與長度計算

完整例子使用與圖相同的合成機率，檢查每對碼字的前綴自由性，並重建獨立取樣訊息。列印的負載長度排除傳送樹與長度標頭；這些排除明確說明。

```
import heapq
import itertools
import numpy as np

symbols = list("ABCDE")
p = np.array([0.45, 0.25, 0.15, 0.10, 0.05])
counter = itertools.count()
heap = [(float(w), next(counter), s) for s, w in zip(symbols, p)]
heapq.heapify(heap)
while len(heap) > 1:
    wa, _, a = heapq.heappop(heap)
    wb, _, b = heapq.heappop(heap)
    heapq.heappush(heap, (wa + wb, next(counter), (a, b)))
tree = heap[0][2]
code = {}
def visit(node, prefix=""):
    if isinstance(node, str):
        code[node] = prefix or "0"
```

```
    else:
        visit(node[0], prefix + "0")
        visit(node[1], prefix + "1")
visit(tree)
assert all(not b.startswith(a) for i, a in enumerate(code.values())
           for j, b in enumerate(code.values()) if i != j)

rng = np.random.default_rng(34)
message = "".join(rng.choice(symbols, size=2000, p=p))
payload = "".join(code[s] for s in message)
inverse = {v: k for k, v in code.items()}
decoded, prefix = [], ""
for bit in payload:
    prefix += bit
    if prefix in inverse:
        decoded.append(inverse[prefix])
        prefix = ""
assert not prefix and "".join(decoded) == message
entropy = -np.sum(p * np.log2(p))
expected = sum(w * len(code[s]) for s, w in zip(symbols, p))
assert entropy <= expected < entropy + 1
print("Code:", code)
print("Entropy / expected length:", entropy, expected)
print("Payload bits per symbol:", len(payload) / len(message))
```

熵界限討論來源下的期望長度，不是每個有限訊息。特定樣本可比期望速率更短或更長。小訊息的樹傳送成本可能占大部分；跨許多訊息共享穩定字母模型，可以攤提成本。

若由訊息本身估計機率，接收端不能直接使用該估計，除非它被傳送，或可由已解碼符號重現。先擬合直方圖、只報告負載，便隱藏真正的模型成本。

算術編碼：用區間識別整個訊息

從單位區間開始，每個符號挑選與機率成比例的子區間。反覆選擇後，剩下的小區間唯一描述整個訊息。

設 $F_t(s)$ 是符號 s 之前的累積模型機率，$q_t(s)$ 是條件機率，區間界限更新為

$$L_t = L_{t-1} + (U_{t-1} - L_{t-1})F_t(x_t),$$
$$U_t = L_{t-1} + (U_{t-1} - L_{t-1})[F_t(x_t) + q_t(x_t)].$$

其中 $[L_t, U_t)$ 是處理 t 個符號後的區間，x_t 是目前符號，F_t, q_t 在兩端使用相同已解碼歷史，初始區間是 $[0, 1)$。第二行使用舊下界，不是剛更新的下界。

最後寬度為

$$U_n - L_n = \prod_{t=1}^{n} q_t(x_t) = q(x^n).$$

其中 n 是訊息長度，$q(x^n)$ 是模型聯合機率。夠短的二元描述若能指定區間內的點或包含於其中

的二元冪區間，就能識別訊息。理想區間方法只為整個區塊付出有界的終止成本，而不逐符號取整。

實際實作使用有限整數暫存器、重新正規化與明確終止。實數不是可傳送物件，無限精度也不是產品編碼器。下方短例子刻意用精確有理數示範區間不變量，不宣稱建立完整檔案格式。

看懂前三次區間更新

使用例子中有順序的模型：A 機率 1/2，B 為 1/3，C 為 1/6，累積起點分別是 0、1/2、5/6。前綴 ABA 的更新為：

已讀符號	下界	上界	寬度
無	0	1	1
A	0	1/2	1/2
AB	1/4	5/12	1/6
ABA	1/4	1/3	1/12

讀 A 後，可用區間寬 1/2。B 選擇**該區間內**從二分之一到六分之五的部分，所以下界為 $0 + (1/2)(1/2) = 1/4$，上界為 $0 + (1/2)(5/6) = 5/12$。接著 A 選寬度 1/6 區間的下半部。最後寬度為 $1/2 \times 1/3 \times 1/2 = 1/12$，恰好等於 ABA 的模型機率。

解碼時，把標記在目前區間的位置正規化，依相同累積分割選 A、B 或 C，再更新區間並重複。模型與訊息長度讓程序可逆，小區間則解釋理想描述成本。精確有理數標記用來教機制，有限整數實作才決定實際輸出哪些位元。

程式：短訊息的精確區間往返

編碼器輸出有理數標記；解碼器另取得共享模型與訊息長度。分子與分母在此是概念側資訊，其序列化大小不宣稱等於最佳算術碼長。

```
from fractions import Fraction

model = {"A": Fraction(1, 2), "B": Fraction(1, 3), "C": Fraction(1, 6)}
message = "ABACAB"
cumulative = {}
running = Fraction(0)
for symbol, probability in model.items():
    cumulative[symbol] = running
    running += probability
assert running == 1
low, high = Fraction(0), Fraction(1)
joint = Fraction(1)
for symbol in message:
    width = high - low
    old_low = low
    low = old_low + width * cumulative[symbol]
    high = old_low + width * (cumulative[symbol] + model[symbol])
```

```
    joint *= model[symbol]
assert high - low == joint
tag = (low + high) / 2

low_d, high_d = Fraction(0), Fraction(1)
decoded = []
for _ in range(len(message)):
    position = (tag - low_d) / (high_d - low_d)
    symbol = next(s for s in model
                  if cumulative[s] <= position < cumulative[s] + model[s])
    width, old_low = high_d - low_d, low_d
    low_d = old_low + width * cumulative[symbol]
    high_d = old_low + width * (cumulative[symbol] + model[symbol])
    decoded.append(symbol)
assert "".join(decoded) == message
print("Final interval:", low, high, "width:", high - low)
print("Exact decoded message:", "".join(decoded))
```

真實符號的模型機率若為零，區間會消失。編碼器需要正機率逸出路徑，或涵蓋所有合法輸入的字母模型。只在編碼端截斷機率不夠，解碼端也必須使用相同正規化與整數量化。

模型成本是交叉熵，不只是熵

再好的位元封裝，也救不了差的機率模型。模型未預期的符號仍需長描述。

對實際區塊分布 p 與編碼模型 q，

$$\mathbb{E}_p[-\log_2 q(X^n)] = H_p(X^n) + D_{\mathrm{KL}}(p_{X^n} \| q_{X^n}).$$

其中 $\mathbb{E}_p$ 依真實來源取平均，$H_p(X^n)$ 是真實區塊熵，散度比較完整區塊分布。有限編碼器與框定成本還會增加位元。

使用先前符號的模型可以接近熵率，而不只是邊際熵。對持續的 Markov 序列使用獨立模型，就丟掉可預測的重複；更豐富上下文模型也可能過度擬合、需要過多記憶體，或對未見歷史分配脆弱機率。

評估應保留獨立紀錄或資料取得群組，報告實際序列化大小，包括標頭、模型選擇旗標與必要側串流。若預訓練權重永久共享，說明慣例；若每個訊息調適權重，計入更新成本。

有意義的比較，是共同假設下的完整編碼器與解碼器協定。某方法的理想對數機率與另一方法的實際位元組數可供診斷，但不是直接的編碼器比較。

訊息框定與不合法串流

完整實驗也指定訊息邊界如何處理。前綴樹告訴解碼器符號在哪裡結束，卻不一定告訴它整個串流何時結束。長度標頭、訊息結束符號或容器邊界，提供另外這項資訊。

填補到位元組邊界的位元，可能看起來像合法碼字。因此接收端需要有意義的位元數或原始符號數。樹例子的負載是教學用 Python 位元字串，不是省空間的位元組容器。

解碼器應拒絕最後不完整碼字與不可能轉移，不能默默回傳較短結果。完整性機制可能偵測損壞，但偵測檔案損壞與壓縮未損壞檔案是不同要求。若該機制屬於選用協定，傳送大小也必須計入。

這些細節對小訊息與公平比較最重要。漸近敘述可讓固定標頭的每符號成本忽略不計，實際短檔案仍必須支付每個標頭位元。

通用編碼邊解碼邊學習

不知道來源機率時，兩端可依已傳送符號更新相同預測器。更新不用透露未來符號。

序列模型分解為

$$q(x^n) = \prod_{t=1}^{n} q_t(x_t \mid x^{t-1}).$$

其中 x^{t-1} 是先前已解碼歷史，q_t 是正規化條件分布。編碼器觀察目前符號；解碼器從區間知道該符號後，再執行相同狀態更新。

對未知的獨立 Bernoulli 來源，Krichevsky–Trofimov 預測器使用

$$q_t(1 \mid x^{t-1}) = \frac{N_1(t-1) + 1/2}{t}, \qquad q_t(0 \mid x^{t-1}) = 1 - q_t(1 \mid x^{t-1}).$$

其中 $N_1(t-1)$ 計算先前的一，$t-1$ 是先前觀測數。分母為先前計數加一。半計數平滑使任何下一符號都具有正機率，包括第一次觀測 [1]。

這是使用 beta 半計數先驗的 Bernoulli 混合後驗預測分布。適當漸近編碼意義下，它對指定 Bernoulli 家族具有通用性，但不會自動對任意相依來源通用。

定義相對於知道真實來源的參照模型之序列遺憾：

$$\mathcal{R}_n(x^n) = -\log_2 q(x^n) + \log_2 p(x^n).$$

其中 p 是選定參照來源，q 是自適應模型。固定來源下的期望遺憾是區塊散度，非負；單一樣本的實現遺憾卻可為負，因為混合模型可能為該樣本分配比參照來源更高的機率。

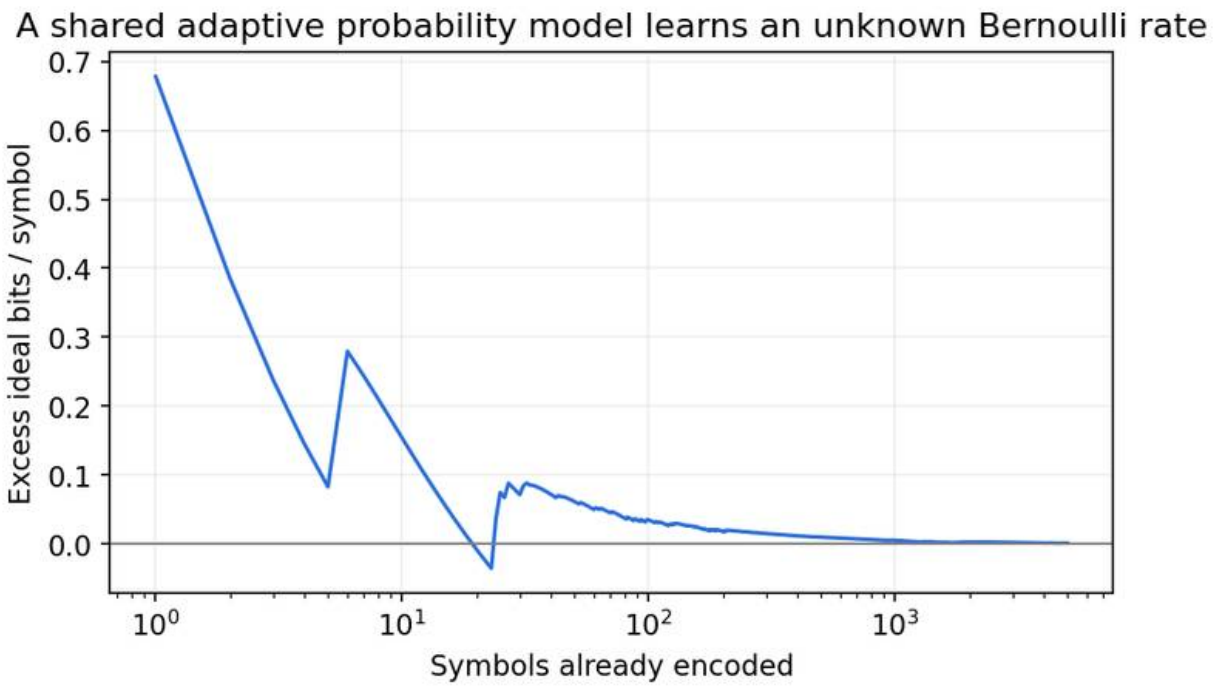


圖 2 — 固定種子的合成 *Bernoulli* 序列。曲線是每符號已實現對數損失差，不是序列化位元組數，也不是單調保證。

程式：調適與突然變動的來源

完整實驗測量理想序列長度，比較固定公平位元模型、匹配真實來源的參照模型，以及半計數調適。第二條序列在中途改變生成機率，顯示平穩計數模型的限制。

```
import numpy as np

rng = np.random.default_rng(23)
def adaptive_length(bits):
    ones, loss = 0, 0.0
    for t, bit in enumerate(bits):
        q = (ones + 0.5) / (t + 1)
        loss -= np.log2(q if bit else 1 - q)
        ones += int(bit)
    return loss

for n in [100, 1000, 5000]:
    bits = rng.binomial(1, 0.2, size=n)
    oracle = -np.sum(bits * np.log2(0.2) + (1 - bits) * np.log2(0.8))
    print("Length:", n, "oracle:", oracle / n,
          "adaptive:", adaptive_length(bits) / n, "fair model:", 1.0)
bits = np.r_[rng.binomial(1, 0.1, size=2500),
             rng.binomial(1, 0.9, size=2500)]
one_model = adaptive_length(bits)
```

```
two_models = adaptive_length(bits[:2500]) + adaptive_length(bits[2500:])
print("Changed source, whole / reset ideal rate:", one_model / 5000, two_models / 5000)
assert two_models < one_model
```

重設比較免費提供變動點。真實編碼器必須識別該位置，或使用兩端共享的預定重設時程。計入重設好處卻省略通知成本，會高估可達節省。

另有兩種不同基準：相對於固定真實機率的遺憾，以及相對於觀察序列事後最佳擬合參數的遺憾。兩者不能互換。即使來源模型正確，有限樣本也可能偏好不同於生成機率的參數。

看因果預測器逐符號學習

尚無觀測時，半計數模型替兩種位元各給 1/2 機率。若已解碼前綴是 `001`，下一個一的機率為 $(1+1/2)/(3+1) = 3/8$，零的機率為 5/8。兩端從相同前綴得到同一狀態，因此不需要傳直方圖。

模型一開始不知道來源偏差，必須支付學習成本。固定 Bernoulli 來源的計數增加時，初期成本被更多符號攤提。但交替字串的總計數公平，卻可由前一符號完全預測；純計數模型漏掉此關係，持續預測接近二分之一。Bernoulli 家族內的通用性指學未知偏差，不是學會所有序列結構。

字典、相依性與通用來源類別

字典編碼器學習重複片語，而不只符號頻率。長重複模式可以用先前片語的參照表示。

Lempel–Ziv 家族使用持續成長的已觀察字串集合建立描述 [1]。漸近通用性結果指定來源類別，例如平穩遍歷有限字母程序，也指定精確演算法；並不表示任何字典實作都在每個有限檔案上勝過每個機率模型。

字典項目、片語邊界與參照索引必須能由解碼器重現。字典增長時，索引需要更多位元。剖析出的片語個數不是壓縮長度，還需要整數編碼規則與終止格式。

不同來源結構適合不同機制。單符號預測器捕捉偏差，上下文預測器捕捉短相依性，片語字典利用重複子字串，學習模型則可利用訓練族群共享的結構。若宣稱無損，仍都需要可逆符號傳送。

通用性相對於基準與極限設定。未知平穩參數是一種問題，變動字母集或任意對抗來源則是另一種。記憶體限制、延遲、短訊息與重設規則可能主導漸近好處。

引用通用速率結果前，應問四件事：涵蓋什麼來源類別、使用何種參照長度、收斂是期望還是實現路徑，以及是否包含模型、字典與框定成本。

Kolmogorov 複雜度：描述的理想極限

想像能印出某字串並停止的最短電腦程式。有結構字串可用短程式，大多數長字串卻不行。

相對於固定通用描述機器 U，定義

$$K_U(x) = \min_{p : U(p) = x} |p|.$$

其中 x 是有限字串，p 是讓機器 U 輸出 x 並終止的程式，$|p|$ 是程式位元長度。某些版本使用前綴自由程式；機器與慣例必須說明 [1]。

不同適當通用機器之間，複雜度只差與機器相關的加法常數。但對短字串，這不表示實際數值與機器無關。更重要的是，一般情況下精確最短程式函數不可計算，不能從定義得到總能找出最佳壓縮的演算法。

把解碼器與框定計入後，無損壓縮器給出演算法描述複雜度的上界；輸出長度不是精確 Kolmogorov 複雜度。某壓縮器不能縮短字串，也不證明字串具演算法隨機性。

計數論證說明，沒有單射壓縮器能縮短每個固定長度輸入：長度 n 的字串，比所有短於 n 的二元描述更多。在誠實邊界慣例下，讓一些輸入更短的任何方法，必須使其他輸入維持原長或變長。

這不妨礙在有結構來源分布上取得優秀壓縮。壓縮利用非均勻機率或重複結構，沒有移除所有可能輸入的計數限制。

現代觀點

Yang、Mandt 與 Theis 的 2023 年已出版專論 [4]，區分無損機率編碼與有損表徵設計。其無損分類包括自迴歸、潛在變數與可逆模型，並強調模型分解與熵編碼器的相容性。我們用這個分類，追問機率如何到達可執行解碼器。

現代自迴歸網路可取代簡單計數預測器，同時保留鏈式協定。下一符號機率必須能由已解碼上下文取得。潛在變數模型需指定傳送潛在資訊的協定；低變分訓練目標不會自動等於最後位元串流長度。

El Gamal 的 2026 年擬議學習章節 [5]，以散度連結壓縮與統計學習。它仍是草稿。古典不匹配恆等式依然有用：更好的預測機率可以降低理想編碼成本，計算與同步則決定節省是否實際可用。

開放問題包括有效調適分布轉移、降低逐步解碼成本、公平比較完整格式，以及讓小訊息負擔得起模型更新。較大預測器可以節省負載，同時增加共享記憶體或計算需求。

具體評估應分開記錄三項：保留資料的負對數機率、包含必要側資訊的實際檔案大小，以及共同硬體假設下的編解碼時間。只有第二項能建立實際壓縮率，只有完整往返斷言能建立精確重建。

下一章轉換角色：來源編碼移除可預測冗餘，通道編碼加入精心設計的冗餘以抵抗雜訊。兩者都使用機率，但解決不同重建問題。

重點整理

- 無損性是具有明確邊界的精確編碼器與解碼器恆等式。
- Kraft–McMillan 限制碼長，熵限制其期望值下界。
- Huffman 對指定二元逐符號前綴問題最佳。
- 算術區間把取整成本攤到整個訊息。
- 模型不匹配先增加散度，再加有限實作額外成本。
- 自適應編碼需要同步因果更新與正機率支撐。
- 通用表現依賴指定來源類別與基準。
- 可計算壓縮長度是理想演算法描述的上界。

練習

1. 在碼字 0、1、01 下，找兩個產生相同串流的不同符號序列。說明其他配置中，提供符號數為何仍可能有歧義。

   提示

   從第三符號與前兩符號開始。

2. 驗證碼長 1、2、3、4、4 的 Kraft 總和，建立實現這些長度的前綴樹，並以例子分布比較期望長度。

   提示

   加總二的冪，配置互不為祖先的葉節點。

3. 將 Huffman 機率改成五符號均勻來源，說明平手為何改變樹而不改變最佳期望長度。

   提示

   保持確定性平手規則。

4. 將精確算術例子改成上下文相依的二元預測器，讓解碼器依已解碼符號更新，驗證最後區間寬度。

   提示

   使用精確半計數分數與相同歷史。

5. 比較半計數調適在常數、交替、獨立偏差二元字串上的結果，指出單一計數模型漏掉哪些來源結構。

   提示

   頻率沒有編碼順序。

6. 學習預測器每符號節省 0.1 理想位元，但需傳送 8,000 位元調適資訊。忽略其他變化，求抵銷額外成本所需訊息長度。

   提示

   總額外成本除以每符號節省。

參考文獻

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapters 5, 13–14. 出版社書籍與目錄.
2. T. Weissman, “Variable Length Lossless Compression,” Stanford EE276 Lecture 4 notes, 2024. 原始課程講義.
3. T. Weissman, “Kraft-McMillan Inequality and Huffman Coding,” Stanford EE276 Lecture 5 notes, 2024. 原始課程講義.
4. Y. Yang, S. Mandt and L. Theis, “An Introduction to Neural Data Compression,” *Foundations and Trends in Computer Graphics and Vision*, 15(2), 113–200, 2023. 已出版專論作者版本.
5. A. El Gamal, “Information Theory and Statistical Learning,” proposed textbook chapter, arXiv preprint, 2026. 草稿與出版狀態.

IT 04 ・通道容量與可靠通訊

推導有限通道容量，說明編碼可達性與 Fano 反向界限，並用 Blahut–Arimoto 計算最佳訊號分布。

網站版本: https://blog.shyandram.dev/zh-tw/learn/information-theory/it-04-channel-capacity/

你將學到

- 定義離散無記憶通道，區分訊息與通道符號。
- 推導二元對稱通道與二元抹除通道的容量。
- 透過聯合典型性與配置直覺理解隨機編碼可達性。
- 利用 Fano 不等式推導可靠速率的反向界限。
- 以 Blahut–Arimoto 計算容量與最佳性差距。
- 區分漸近限制、有限區塊可靠性與任務導向通訊。

先看全貌

雜訊通道不必正確還原每個傳送符號，接收者仍可能恢復訊息。編碼將每個訊息表示成較長且有結構的序列；接收端把帶雜訊觀測與可能序列比較，判斷送出哪個訊息。

> **用最簡單的話說**
>
> 朋友可能聽錯一些字母，傳送精心選擇的較長綽號就能幫忙。額外字母讓不同綽號較難混淆。但在同一個帶雜訊空間塞進太多不同綽號，最後還是會重疊。

來源編碼移除可預測冗餘，通道編碼增加有用冗餘；兩者目標不同。重複位元是簡單通道編碼，卻不一定有效利用通道。核心問題是每次通道使用能傳多少不同訊息，同時讓重建錯誤趨近零。

主要閱讀是 Cover 與 Thomas 第 7 章；書中的計算討論將容量最佳化與 Blahut–Arimoto 方法連結 [1]。Stanford 的通訊與反向定理講義 [2,3] 提供原始推導。我們先建立假設與數值實驗，再說明實際短訊息為何需要額外評估。

先定義通道，再談容量

通道指定輸入符號如何隨機轉成輸出符號。來源分布與通道轉換是不同的對象。

對有限輸入字母集 $\mathcal{X}$ 與輸出字母集 $\mathcal{Y}$，離散無記憶通道由列隨機矩陣表示：

$$W_{xy} = \Pr(Y = y \mid X = x), \qquad p(y^n \mid x^n) = \prod_{t=1}^{n} W_{x_t y_t}.$$

其中 x, y 是符號索引，$W_{xy} \geq 0$ 且每列總和為一。區塊 x^n, y^n 長度為 n，t 索引一次使用。分解定義無記憶性與不隨時間改變的轉移律。

無記憶性描述**給定整個輸入區塊後**的輸出。編碼輸入符號可能相依，因為它們都由同一訊息決定；未條件化的輸出也可能因此相依。證明反向界限時假設輸入獨立，會不必要地限制允許編碼。

單符號計算中，發送者選擇輸入分布 $q_x = \Pr(X = x)$，誘導輸出機率

$$r_y = \sum_x q_x W_{xy}.$$

其中 r_y 是輸出邊際。改變 q 時，通道 W 不變。更改傳播律、接收觀測、資源限制或可用回饋，便定義不同通訊問題。

相同的轉移列無法區分對應輸入。不同列也會因重疊程度而較易或較難區分。輸出字母大小給出粗略資訊上界，實際可利用的區別則由轉移機率決定。

速率與可靠性是操作量

通訊速率是訊息資訊除以通道使用次數。可靠性討論恢復整個訊息，不只是讓多數通道符號正確。

設訊息 M 在 N 個選項中均勻分布，長度 n 編碼把它映至 X^n，再從收到的 Y^n 估計 $\widehat{M}$。定義

$$R_n = \frac{\log_2 N}{n}, \qquad P_e^{(n)} = \Pr(\widehat{M} \neq M).$$

其中 N 是碼本大小，R_n 是每次使用的位元數，$P_e^{(n)}$ 是指定均勻訊息與通道下的平均訊息錯誤機率。它不同於符號錯誤率。

若存在一系列編碼，使漸近速率至少為 R 且訊息錯誤趨近零，則 R 可達。容量是可達速率的上確界：

$$C = \sup\{R : R \text{ is achievable}\} = \max_q I_q(X; Y).$$

其中 q 遍歷有限字母集上的所有輸入分布，I_q 是聯合分布 $q_x W_{xy}$ 下的互資訊。第二個等號是指定有限無記憶通道的通道編碼定理 [1,2]。

標準可達敘述討論嚴格低於容量的速率，不保證有限長度零錯誤，也不提供快速解碼器。容量也不同於**零錯誤容量**：後者要求允許輸出集合永遠不混淆訊息。某通道可以有正的一般容量，但每個有限區塊仍有非零混淆機率。

平均錯誤設定假設均勻訊息。所有訊息的最大錯誤是另一種操作準則；標準有限無記憶結果可移除差的碼字，僅付出漸近可忽略的速率損失，連結兩種容量設定。有限長度可靠性統計仍可能不同。

二元對稱通道

每個位元以相同機率獨立翻轉。公平輸入充分利用兩種輸出選項。

翻轉機率為 e 的二元對稱通道滿足

$$W = \begin{pmatrix} 1-e & e \\ e & 1-e \end{pmatrix}, \qquad C_{\mathrm{BSC}} = 1 - h_2(e).$$

其中 $e \in [0,1]$ 是翻轉機率，h_2 是二元熵，容量單位為每次使用的位元數。任一輸入的條件輸出熵為 $h_2(e)$。因為 $H(Y) \le 1$，互資訊至多 $1 - h_2(e)$；公平輸入達到此界。

$e = 0$ 時無雜訊傳送一位元；$e = 1/2$ 時輸出與輸入獨立，容量為零；$e = 1$ 時每個位元確定翻轉，接收端反轉即可，容量又為一。整個區間的雜訊程度不能只看原始翻轉比例。

圖將翻轉機率限制於零至二分之一，這段範圍自然對應不確定性增加；另一半分支可透過重新標記輸出得到。

二元抹除通道

接收者不是看到正確位元，就是看到明確的缺失標記。知道哪些觀測缺失，本身就有用。

抹除機率為 e 的二元抹除通道，其輸出是零、一與抹除符號：

$$W = \begin{pmatrix} 1-e & 0 & e \\ 0 & 1-e & e \end{pmatrix}, \qquad C_{\mathrm{BEC}} = 1 - e.$$

其中列對應輸入零與一，欄對應兩種正確符號與抹除。抹除事件與輸入獨立。

給定未抹除輸出，輸入已知；給定抹除，不確定性仍為 $H(X)$。因此 $I(X;Y) = (1-e)H(X)$，公平輸入達到每次使用 $1-e$ 位元。

相同數值的事件機率，對翻轉與可見抹除產生不同容量。抹除告訴接收者資訊在哪裡缺失，未標記翻轉卻可能暗示錯誤選項。差異來自不同輸出律，不是所有實體通訊系統的普遍排名。

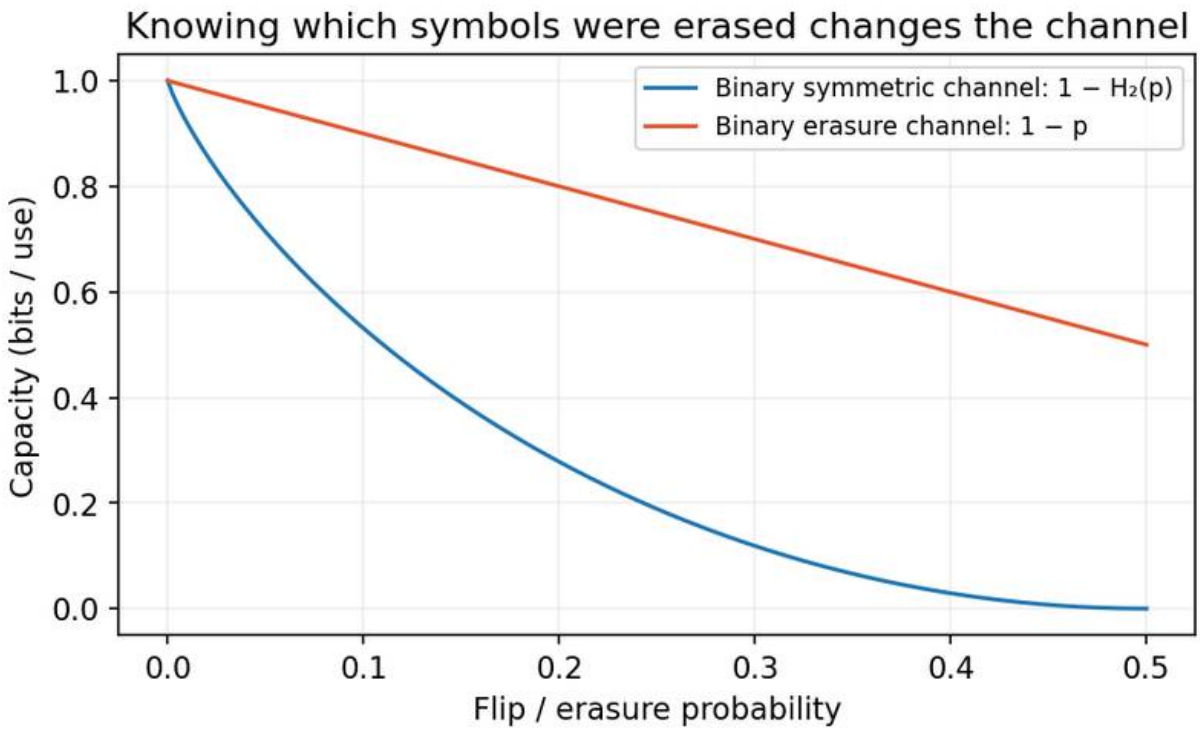


圖 *1* — 每次使用一個二元輸入的解析容量。可見抹除與未標記翻轉是不同通道模型。

不對稱通道需要不對稱訊號分布

一種輸入比另一種可靠時，均勻分布未必最佳。容量最佳化各輸入使用頻率。

考慮零正確到達，而一以機率 a 變成零的 Z 通道：

$$W = \begin{pmatrix} 1 & 0 \\ a & 1-a \end{pmatrix}, \qquad I(q) = h_2(q(1-a)) - q\,h_2(a).$$

其中 a 是一轉零機率，$q = \Pr(X = 1)$ 是純量輸入參數。輸出一機率為 $q(1-a)$，條件輸出熵為 $qh_2(a)$。

第一項鼓勵多樣輸出，第二項計入使用一時付出的雜訊成本。權衡可能使最大值偏離 $q = 1/2$。只傳可靠的零也沒有用：常數輸入不攜帶訊息資訊。

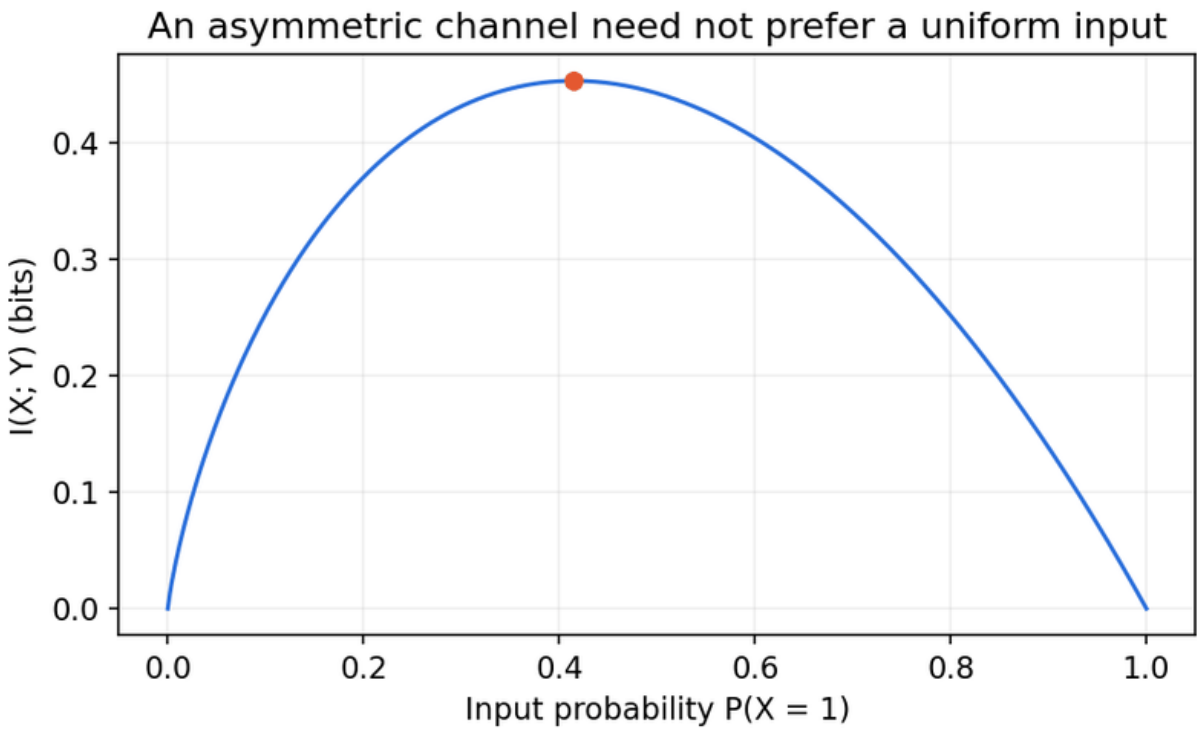


圖 2 — 輸入分布的解析最佳化。某符號較可靠，不代表應專門傳送它。

互資訊對輸入分布為凹函數，因為輸出熵是凹的，條件輸出熵是線性的，因此數值最佳化可行。不同輸入冗餘時，仍可能有平坦最大區域，最佳分布不必唯一。

為何低於容量的速率可達

選許多看起來像合法來源區塊的長碼字。收到的區塊通常符合真正碼字的通道關係，卻很少符合無關碼字。

固定輸入分布 q，從 q^n 獨立產生約 2^{nR} 個碼字。聯合典型解碼器依聯合律 $q_x W_{xy}$，尋找唯一符合收到輸出的碼字 [1]。

主要有兩種錯誤：真正輸入輸出配對未通過典型測試，或另一個碼字通過。第一種機率由聯合 AEP 趨近零；獨立競爭碼字的聯合典型機率，其指數尺度約為 $2^{-nI_q(X;Y)}$。

對競爭碼字使用聯集界限，得到示意尺度

$$P_{\text{competing}} \lesssim 2^{nR} 2^{-n(I_q(X;Y)-\eta)}.$$

其中 $P_{\text{competing}}$ 是出現競爭典型碼字的機率，$\eta > 0$ 吸收典型性容許誤差，$\lesssim$ 省略有限前因子與細節。這是證明直覺，不是精確有限長度效能公式。

若 $R < I_q(X;Y) - \eta$，競爭錯誤界限趨近零。選擇最大化輸入並令容許誤差夠小，便得到低於容量的速率。隨機碼本的平均錯誤很小，表示至少有一個確定碼本錯誤很小。

論證不需要每次訊息傳送隨機種子。碼本在通訊前已固定，兩端皆已知。若實際系統選擇或傳送新碼本，設定協定需另外計算。

隨機編碼證明存在性。逐一搜尋指數大小碼本通常不可行；典型性不是部署解碼器的直接配方。容量定理指出目標，實際結構化編碼另外解決複雜度問題。

Fano 不等式限制接收端推論

接收者經常猜錯，就仍有訊息不確定性。反之，低錯誤機率意味很少剩餘的條件訊息不確定性。

對 $N \geq 2$ 個可能訊息，以及任何根據 Y^n 的估計，

$$H(M \mid Y^n) \leq h_2(P_e) + P_e \log_2(N-1) \leq 1 + P_e \log_2 N.$$

其中 M 是真訊息，Y^n 是收到區塊，$P_e = \Pr(\widehat{M} \neq M)$，$h_2$ 是二元熵。這是 Fano 不等式 [1,3]。

引入錯誤指標便能理解邏輯：描述是否錯誤，至多需要其二元熵。沒有錯誤時，估計識別訊息；有錯誤時，至多剩 $N-1$ 個選項。平均後得到第一個界限。

均勻訊息滿足 $H(M) = \log_2 N$。資料處理與通道無記憶性限制

$$I(M; Y^n) \leq I(X^n; Y^n) \leq \sum_{t=1}^{n} I(X_t; Y_t) \leq nC.$$

其中 X^n 是傳送碼字。中間不等式允許碼符號相依：輸出熵至多為各邊際輸出熵總和，條件輸出熵則在無記憶通道下分離。

合併兩個界限得到

$$(1 - P_e)\log_2 N \leq nC + 1, \qquad P_e \geq 1 - \frac{C}{R_n} - \frac{1}{nR_n}.$$

其中 $R_n = \log_2 N / n$，只有右側為正時下界才有意義。固定速率嚴格超過容量時，錯誤不可能趨近零。

這個基本反向證明建立超過容量的正錯誤底限，單獨並未證明錯誤趨近一。強反向敘述需要額外定理與假設。應精確說明顯示的不等式到底證明什麼。

手算速率、錯誤與來源預算

翻轉機率 $e = 0.1$ 時，BSC 容量為 $1 - h_2(0.1) = 0.531004$ 位元／次使用。同樣抹除機率的 BEC 容量是 0.9。差異來自觀測模型：BEC 會揭露哪些符號遺失。

長度 100、碼率 0.8 的編碼共有 2^{80} 種可能訊息。把 BSC 容量代入基本逆定理：

$$P_e \geq 1 - \frac{0.5310044064}{0.8} - \frac{1}{100 \times 0.8} \approx 0.323744.$$

P_e 是平均整訊息錯誤，100 是通道使用數，0.8 是每次使用的訊息位元。代入容量保留較多小數位，再將答案取整。在指定通道下，無論解碼器如何，這個編碼都不可能達成 1% 訊息錯誤目標。但此界限不決定任何特定編碼的精確錯誤。

碼率改成 0.5 時，同一界限為負，因此只剩平凡下界零。它與低於容量的漸近可達性相容，卻不證明長度 100 已經有足夠可靠性。兩個問題需要不同證據：區塊持續增長的定理，以及有限編碼計算或實驗。

再取一個一的機率為 0.2 的獨立二元來源，每符號熵為 0.721928 位元。漸近分離設定下，要透過此 BSC 幾乎無損傳送，需要每來源符號超過 $0.721928/0.531004 \approx 1.359552$ 次通道使用。一次不足，兩次則有速率餘裕。若先用原始表示，每來源符號需要一位元，再加通道保護；這說明利用來源偏差與抵抗雜訊是不同設計步驟。

頻寬比例不必是整數。長區塊可以分配相應通道使用總數，實現有理數或近似比例；延遲與有限區塊額外成本仍是另外的限制。

Blahut–Arimoto 容量計算

從輸入混合開始，檢查每個通道列與其輸出混合有多能區分，再把機率移向較有資訊的列。重複直到上下界接近。

對目前正輸入機率 q_x 與誘導 r_y，以自然單位定義列散度：

$$d_x = \sum_{y:W_{xy}>0} W_{xy} \ln \frac{W_{xy}}{r_y}, \qquad q_x^{\mathrm{new}} = \frac{q_x e^{d_x}}{\sum_j q_j e^{d_j}}.$$

其中 d_x 是通道列 x 與目前輸出分布的散度，$\ln$ 是自然對數，j 索引輸入。分母使用同一目前分布，更新不會將它換成均勻先驗。自然單位除以 $\ln 2$ 轉成位元。

目前分布的上下界為

$$L(q) = \frac{\sum_x q_x d_x}{\ln 2}, \qquad U(q) = \frac{\max_x d_x}{\ln 2}, \qquad L(q) \le C \le U(q).$$

其中 $L(q)$ 是目前互資訊，$U(q)$ 是以目前輸出混合作候選參照分布得到的對偶形式界限。差距可作為數值停止準則。

這個無限制演算法適用於固定有限通道。輸入成本限制需要修改最佳化，不能先算無限制容量，再宣稱符合功率或符號使用預算。

數值上界為什麼成立？

停止間隙有用的前提，是兩端真的夾住最佳值。固定目前輸出混合 r，考慮**任意**候選輸入分布 u，其輸出混合為 $r_u = uW$。展開對數比得到

$$\sum_x u_x D_{\mathrm{nats}}(W_x \| r) = I_{u,\mathrm{nats}}(X;Y) + D_{\mathrm{nats}}(r_u \| r).$$

W_x 是通道列 x，u_x 是候選使用機率，r 是固定參照輸出，r_u 是候選輸出邊際，$I_{u,\mathrm{nats}}$ 是 u 下的互資訊。整個恆等式使用自然對數。

最後散度非負，因此候選互資訊不超過左側加權總和；該總和又不超過最大的列散度。因為對所有 u 都成立，目前最大列散度就是容量上界。目前輸入本身的互資訊，是單字母最佳化可行值，提供下界。

最佳點分配正機率的輸入，具有相同資訊列散度；未使用輸入也不能提供更高邊際增益。更新把機率移向目前散度較高的列，直到這些條件近乎平衡。因此，間隙衡量此矩陣還可能改善多少，不只是最新參數變動有多小。

程式：驗證解析容量與最佳化 Z 通道

完整實作移除所有輸入都不可能產生的輸出欄，遮罩零列元素，並用穩定對數正規化。回傳輸入分布與其對應下界速率。

```
import numpy as np
from scipy.special import logsumexp
from scipy.optimize import minimize_scalar

def binary_entropy(p):
    if p == 0 or p == 1:
        return 0.0
    return -p * np.log2(p) - (1 - p) * np.log2(1 - p)

def capacity(W, tolerance=1e-10, maximum=20000):
    W = np.asarray(W, dtype=float)
    assert W.ndim == 2 and np.all(W >= 0)
    assert np.allclose(W.sum(axis=1), 1)
    W = W[:, W.sum(axis=0) > 0]
    q = np.full(W.shape[0], 1 / W.shape[0])
    positive = W > 0
    for iteration in range(maximum):
        r = q @ W
        ratio = np.zeros_like(W)
        np.log(np.divide(W, r, out=np.ones_like(W), where=positive),
               out=ratio, where=positive)
        d = (W * ratio).sum(axis=1)
        lower, upper = q @ d / np.log(2), d.max() / np.log(2)
        if upper - lower < tolerance:
            return lower, q, upper - lower, iteration + 1
        log_q = np.log(q) + d
        q = np.exp(log_q - logsumexp(log_q))
    raise RuntimeError("Capacity iteration did not meet the requested gap")

e = 0.1
bsc = np.array([[1-e, e], [e, 1-e]])
bec = np.array([[1-e, 0, e], [0, 1-e, e]])
for W, analytic in [(bsc, 1-binary_entropy(e)), (bec, 1-e)]:
    value, q, gap, steps = capacity(W)
    print("Capacity / analytic / input / gap:", value, analytic, q, gap)
    assert abs(value - analytic) < 1e-8
a = 0.35
z = np.array([[1, 0], [a, 1-a]])
```

```
value, q, gap, steps = capacity(z)
objective = lambda p: binary_entropy(p*(1-a)) - p*binary_entropy(a)
reference = minimize_scalar(lambda p: -objective(p), bounds=(0, 1), method="bounded")
assert abs(value + reference.fun) < 1e-8
print("Z capacity / P(input one) / gap / iterations:", value, q[1], gap, steps)
```

與解析二元例子一致，能檢查單位與矩陣方向；與獨立純量最佳化一致，能檢查不對稱例子。這些測試可發現誤用欄隨機機率、漏掉目前 q 因子，或混用自然與二進位對數。

幾乎冗餘輸入與極端機率可能收斂慢或造成浮點下溢。小差距是計算精度內，對這個有限矩陣的數值證據，不證明矩陣準確描述真實通道。

重複實驗顯示有限區塊權衡

重複一位元能幫助多數決解碼，但每個訊息位元消耗更多使用次數。可靠性較高，不自動代表容量效率好。

位元在獨立 BSC 中重複奇數 k 次，使用多數決，則

$$P_e(k) = \sum_{j=(k+1)/2}^{k} \binom{k}{j} e^j (1-e)^{k-j}, \qquad R = \frac{1}{k}.$$

其中 j 是翻轉副本數，e 是翻轉機率，$P_e(k)$ 是單位元訊息錯誤，R 是碼率。這是精確有限編碼計算，不是容量公式。

```
import numpy as np
from scipy.stats import binom

rng = np.random.default_rng(35)
e = 0.1
for copies in [1, 3, 5, 9]:
    noise = rng.binomial(1, e, size=(50000, copies))
    observed_error = np.mean(noise.sum(axis=1) > copies // 2)
    exact_error = binom.sf(copies // 2, copies, e)
    print("Copies / rate / simulated error / exact error:",
          copies, 1 / copies, observed_error, exact_error)
    assert abs(observed_error - exact_error) < 0.005
```

模擬可假設訊息位元為零，因為對稱編碼對任一傳送位元有相同錯誤機率。更多副本降低此單位元問題的錯誤，同時使速率趨近零。容量則允許碼本以正漸近速率攜帶持續增加的位元數。

有限評估應指定區塊長度、速率、訊息錯誤準則、通道參數與解碼規則。只報告容量，無法描述短編碼的延遲、複雜度或可靠性。

資源限制與不同通道假設

容量計算回答一個精確問題。更改發送者預算、加入記憶或提供不同觀測，問題就改變了。

若輸入 x 的非負成本為 $c(x)$，每次使用的平均預算為 B，有限無記憶通道的限制問題為

$$C(B) = \max_{q:\sum_x q_x c(x) \le B} I_q(X;Y).$$

其中 $c(x)$ 是輸入使用成本，B 是允許平均，q 是滿足限制的機率分布。通道仍是相同矩陣。公式採平均成本慣例；每個符號的嚴格峰值限制定義不同可行集合。

對不對稱二元通道，輸入一的頻率限制可能排除無限制最佳分布。結果應是容量與成本函數，不能將功率標籤貼在無限制數字上。

每秒位元的吞吐量也需要指定每秒通道使用次數。每次使用位元數，不自動等於每個樣本、像素或詞元的位元數。比較調變或表示選擇時，說明每種選擇如何改變使用次數與資源。

回饋允許編碼器先觀察過去通道輸出，再選下一輸入。一般無記憶點對點模型中，無雜訊因果回饋不增加漸近容量 [1,3]，仍可改變有限編碼設計、停止規則與可達錯誤行為。不能未檢查就移用至有記憶或多使用者通道。

持續隱藏雜訊狀態可能造成群聚錯誤，而不是獨立翻轉。合併全部使用估計單一矩陣，會隱藏相依性。上述容量證明使用條件乘積律，不能直接把群聚錯誤當成平均翻轉機率的獨立取樣。

接收端側資訊也會造成類似差異。若側資訊揭露抹除位置、雜訊狀態或部分原訊息，就改變接收觀測。應比較完整觀測模型，不能讓解碼器利用基準沒有取得的資訊，卻把好處全部歸給解碼方法。

通道估計是另一個實驗

數值最佳化可能對錯誤矩陣非常準確。相信速率前，先判斷轉移機率是否描述操作條件。

對帶輸入標籤的輸入輸出觀測，簡單列估計為

$$\widehat{W}_{xy} = \frac{N_{xy}}{\sum_z N_{xz}}.$$

其中 N_{xy} 計算輸入 x 與輸出 y 的配對次數，z 遍歷輸出，$\widehat{W}$ 是經驗通道矩陣。每個輸入列都需觀測；經驗零次數不證明事件不可能。

有限樣本使估計容量與最佳分布都不確定。跨獨立取得資料重複實驗可揭露變異；依已知操作條件分組可揭露不同通道律。最佳化差距衡量一個矩陣的計算收斂，實驗不確定性則討論該最佳化哪個矩陣。

若從資料擬合解碼器或輸入策略，訓練與評估也應使用分開觀測。在同一個極小樣本選擇有利輸入分布並估計表現，容易得到樂觀速率。

有用報告因此包含取樣協定、各輸入次數、操作條件、經驗列機率與保留資料重建結果。稀有輸出需明確處理，因為把它們當成零機率，會扭曲解碼器與資訊估計。

來源資訊如何配合通道資訊

來源編碼把有用不確定性打包成位元，通道編碼可靠地運送位元。兩者速率必須使用相同來源與通道使用預算比較。

若有限無記憶來源每符號熵為 $H(S)$，每來源符號可使用 b 次通道，熟悉的漸近分離條件為

$$H(S) < bC.$$

其中 S 是來源符號，b 是使用比例，C 是點對點容量。在標準長區塊假設下，來源與通道編碼能各自以餘裕接近限制 [1]。

不等式未指定實際區塊長度，也不要求有限延遲下獨立設計最佳；更未宣稱所有網路都有一般來源通道分離。保留假設，才能避免把漂亮公式套至無關限制。

有損重建需要的來源速率改為率失真函數，而不是完整熵；連續振幅通道的容量由實體限制決定。這兩個延伸引出後續章節。

現代觀點

Nguyen 等人的 2025 年已出版綜述 [4]，將語意通訊分成接收者意圖模型、生成模型與深度聯合來源通道編碼。分類依系統想保留什麼、兩端如何協調區分，而非把所有學習系統當成相同編碼器。

這些目標可能改變所需重建。任務接收者也許只需要類別標籤，而非全部原始像素，因此需要的來源資訊可能減少；但在通道與限制不變時，Shannon 容量不會增加。

生成式重建也需要明確準則。合理輸出與原始內容的關係，可能不同於精確解碼訊息。明確評估任務準確度或感知失真很合理，但不能用它們取代位元可靠性主張中的訊息錯誤機率。

從古典模型推論，應分清三層：來源接收者需要什麼、實體通道傳遞什麼，以及編碼如何使用有限資源。聯合最佳化可改善實際匹配，無須推翻對應限制。

開放問題包含通道律變動時的穩健表現、短區塊可靠操作、共享模型同步，以及跨任務與接收者知識的評估。一種任務或通道的綜述結果，不建立通用語意速率。

下一章將字母從有限符號改成連續振幅，先建立微分熵與功率限制，再推導 Gaussian 容量。把連續密度直接代入離散熵公式，會漏掉重要的單位依賴。

重點整理

- 無記憶通道在給定輸入下分解輸出，碼符號不必獨立。
- 容量是可靠訊息速率，在指定假設下等於最佳化互資訊。
- 翻轉與可見抹除有不同通道容量。
- 不對稱通道可能偏好非均勻輸入。
- 隨機編碼以典型性與配置建立存在性。
- Fano 與資料處理限制訊息錯誤趨零的速率。
- Blahut–Arimoto 最佳化有限通道並提供數值差距。

- 有限可靠性與語意目標各需明確評估。

練習

1. 說明翻轉機率 0.9 與 0.1 的 BSC 容量為何相同，描述接收端如何重新標記輸出。

   提示

   反轉每個輸出。

2. 對任意二元輸入機率推導 BEC 互資訊，不預設公平輸入。

   提示

   抹除保留原輸入熵。

3. 將 Z 通道參數改成 0.1 與 0.7，比較最佳輸入機率與二分之一，解釋兩個熵項的競爭。

   提示

   最佳化輸出熵減加權列熵。

4. 長度 100、碼率 0.8 的編碼通過翻轉機率 0.1 的 BSC，計算基本 Fano 訊息錯誤下界，說明它證明與未證明的事。

   提示

   使用 $1 - C/R - 1/(nR)$，負值時保留零下界。

5. 複製通道矩陣某一列並重跑容量，解釋容量為何不變，最佳輸入混合卻可能不再唯一。

   提示

   相同列產生相同條件輸出。

6. 在相同總使用預算下比較重複長度 3 與 9，說明單位元錯誤較小，為何不能單獨判斷系統可靠傳送的資訊較多。

   提示

   同時計算訊息位元與錯誤。

參考文獻

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapter 7 and the computational capacity/rate–distortion discussion. 出版社書籍與目錄.
2. T. Weissman, “Communication and Channel Capacity,” Stanford EE276 Lecture 7 notes, 2024. 原始課程講義.
3. T. Weissman, “Channel Coding Theorem: Converse Part,” Stanford EE276 Lecture 10 notes, 2024. 原始課程講義.
4. L. X. Nguyen, A. D. Raha, P. S. Aung, D. Niyato, Z. Han and C. S. Hong, “A Contemporary Survey on Semantic Communications: Theory of Mind, Generative AI, and Deep Joint Source-Channel Coding,” *IEEE Communications Surveys & Tutorials*, 2025, DOI: 10.1109/COMST.2025.3616973. 已出版論文資料與作者版本.

IT 05 ・微分熵與高斯通道

以明確單位處理連續資訊，推導功率預算下的高斯容量，並用注水法最佳化平行通道。

網站版本: https://blog.shyandram.dev/zh-tw/learn/information-theory/it-05-continuous-gaussian-channels/

你將學到

- 定義微分熵，避免把密度當成機率。
- 追蹤單位與座標改變，同時保留互資訊。
- 從相對熵推導高斯最大熵性質。
- 求平均功率預算下的實數高斯通道容量。
- 以注水法配置平行通道功率。
- 區分解析資訊極限、估計與實作成本。

先看全貌

振幅可以在連續範圍取值，但有雜訊的接收器不能免費區分無限多訊息。更精細的量測需要資源，發送器通常也不能使用任意大的振幅。連續資訊理論會把這些限制明確保留。

> **用最簡單的話說**
> 你可以指向尺上的任何地方，但朋友模糊的視力無法分辨每個細小差異。把位置分遠一點有幫助，可是尺的大小有限。真正問題是：在可用空間內，能放入多少可靠區分的訊號？

本章依 Cover 與 Thomas 第 8–9 章 [1]，並補充 Stanford 的連續變數與高斯通道講義 [2,3]。順序很重要：先定義以密度為基礎的資訊，再證明分布上界，最後用它最佳化功率限制下的通訊。

連續公式也出現在視覺特徵、潛在表示與感測器雜訊模型。即使場景不變，數值仍可能隨單位改變。比較關係時，互資訊是更穩定的量。

密度不是點機率

連續密度描述每單位量測的機率。密度大於一完全可能，並不違反機率規則。

對具有密度 f_X 的實數隨機變數 X，

$$\Pr(X \in A) = \int_A f_X(x)\,dx, \qquad \int_{\mathbb{R}} f_X(x)\,dx = 1.$$

A 是可測區域，x 是所選單位的座標，$f_X(x)$ 是密度，不是精確點的機率。一般連續密度下，每個精確點的機率都為零。

寬度十分之一區間上的均勻變數，密度為十；整段區間的機率仍是一。因此，對密度取負對數可能得到負值。

微分熵（differential entropy）為

$$h(X) = -\int f_X(x) \log_2 f_X(x)\, dx.$$

h 是底數二的微分熵，積分在密度定義處進行。此式假設積分以適當意義存在。它可能負、無窮，有些分布沒有良好定義的有限熵。

與離散熵不同，微分熵本身不是精確表示連續值所需的最少位元數。精確無限精度重建，是另一個通常無界的要求。

符號刻意以小寫 h 表示微分熵、大寫 H 表示離散熵。適當條件下，它們有相似代數關係，解釋與符號限制卻不同。

量化連結到實際符號

真實儀器記錄的是格子，不是無限精確的點。格子越小，描述格子索引需要越多位元。

把純量密度量化成寬度 $\Delta > 0$ 的格子。密度足夠規則且量化夠細時，

$$H(X_\Delta) \approx h(X) + \log_2 \frac{1}{\Delta}.$$

X_Δ 是離散格子索引，Δ 使用與 x 相同的座標單位。這是細格子漸近關係，受規則性與尾端條件限制。

格子機率約等於密度乘格寬，因此負對數機率約等於負對數密度減去對數格寬。平均後就得到此式。

額外項不是小修正。解析度增加時，它會發散。有限微分熵不會讓任意精確值變得便宜。

粗解析度、尖銳不連續或厚尾下，近似可能不好。真實量化器也可能飽和、格子不均勻，或範圍有限。這些細節會改變離散分布，實際編碼實驗應納入。

對感測器，解析度與物理雜訊都重要。儲存數值精度超過量測精度，不一定創造新場景資訊，可能只是多了表示不確定振幅的數字。

程式：座標單位與量化熵

獨立實驗使用單位均勻變數，在不同解析度計算經驗格子索引熵，並對照縮放後解析微分熵變化。

```
import numpy as np

rng = np.random.default_rng(36)
x = rng.uniform(0, 1, size=100000)
def discrete_entropy(values):
    _, counts = np.unique(values, return_counts=True)
```

```
    p = counts / counts.sum()
    return -np.sum(p * np.log2(p))
for bins in [10, 100, 1000]:
    indices = np.floor(x * bins).astype(int)
    estimated = discrete_entropy(indices)
    theoretical = np.log2(bins)
    print("Bins / estimated index entropy / exact:", bins, estimated, theoretical)
    assert abs(estimated - theoretical) < 0.03
scale = 0.1
h_x, h_scaled = 0.0, np.log2(scale)
print("Differential entropy before / after scaling:", h_x, h_scaled)
assert h_scaled < 0
```

負的縮放熵，不是離散訊息的負不確定性；它反映每單位座標的密度。有限格子估計仍非負，而且罕見格子變多時，抽樣偏差會增加。

座標改變會改變微分熵

同樣長度用毫米而非公尺表示，會把數值分布拉伸，微分熵也跟著變。

對非零純量 a 與平移 b，

$$h(aX + b) = h(X) + \log_2 |a|.$$

a 縮放座標，b 平移。平移不改變密度體積，縮放則乘上 $|a|$。對作用於 d 維向量的可逆矩陣 A，

$$h(AX + b) = h(X) + \log_2 |\det A|.$$

A 是非奇異 d 乘 d 矩陣，b 是平移向量，$\det A$ 是行列式。兩側都需要良好定義的熵。

光滑一對一非線性座標映射 T，有相應 Jacobian 修正：

$$h(T(X)) = h(X) + \mathbb{E} \log_2 |\det J_T(X)|.$$

J_T 是 T 的導數矩陣，在隨機 X 處評估；期望平均對數體積變化。恆等式需要適當可微性、可逆性與可積分性。

這解釋潛在密度為何依賴表示座標。可逆轉換可以改變微分熵，而不丟棄資訊。只以原始微分熵比較特徵空間，可能把座標膨脹誤認成資訊品質。

從密度推導尺度修正

這個修正可以直接理解，不必死背。令 $V = aX + b$ 且 $a \neq 0$。座標改變保留機率，因此

$$f_V(v) = \frac{1}{|a|} f_X\left(\frac{v - b}{a}\right).$$

V 是轉換後變數，v 是座標，f_V 是密度，a, b 是尺度與平移。座標區間擴張 $|a|$ 倍，密度就縮小相同比例。因此負對數密度增加常數 $\log_2 |a|$；依未變的事件機率取平均，就得到熵修正。

取寬度 0.1 區間上的均勻來源，微分熵為 $\log_2(0.1) = -3.321928$。分成十個寬度 0.01 的等格，每格機率十分之一，離散熵為 $\log_2(10) = 3.321928$ 位元。密度固定且格子與支撐對齊，因此此處細格關係精確成立。

所有座標乘 100 以改變單位後，區間寬十，微分熵變成 $\log_2(10) = 3.321928$，格寬則變成一。同樣十種結果仍需要 3.321928 位元。縮放使密度熵與格寬修正朝相反方向改變，沒有創造更多可辨事件。只改格子數會改變觀測，改座標單位則不會。

相對熵與互資訊仍有意義

以比值比較密度，會抵消共同座標單位。兩變數關係不應因任一變數換成可逆座標而消失。

對密度 f, g，

$$D_{\mathrm{KL}}(f\|g) = \int f(x)\log_2\frac{f(x)}{g(x)}\,dx, \qquad I(X;Y) = D_{\mathrm{KL}}(f_{XY}\|f_X f_Y).$$

f_{XY} 是聯合密度，$f_X f_Y$ 是獨立乘積參考。散度非負，支撐或可積分性要求時可為無窮。

分別對 X, Y 做可逆座標轉換，密度比與積分中的 Jacobian 因子會抵消，所以互資訊不變。非可逆投影或量化不適用；它們可能丟棄資訊，受到資料處理關係控制。

所有熵有限且相容時，

$$I(X;Y) = h(X) + h(Y) - h(X,Y) = h(Y) - h(Y \mid X).$$

$h(X,Y)$ 是聯合微分熵，$h(Y \mid X)$ 是平均條件微分熵。條件微分熵可能負，但互資訊仍非負。

若熵差會相減無窮，密度比定義更安全。真正連續變數的無雜訊確定性恆等關係，可能有無窮互資訊與奇異聯合法則；一般全維聯合密度公式需要小心。

對相關係數 ρ 的聯合高斯純量對，

$$I(X;Y) = -\frac{1}{2}\log_2(1-\rho^2).$$

ρ 是相關係數，$|\rho| < 1$ 讓共變異數非奇異。只有聯合高斯假設下，相關係數才足以代入此式；一般相依不能只用一個共變異數數值概括。

固定共變異數下，高斯分布有最大熵

在指定共變異數的分布中，高斯以微分熵意義最廣泛地散布不確定性。這是受限制陳述，不是每個來源都高斯的宣稱。

共變異數 Σ 的非奇異 d 維高斯具有

$$h(G) = \frac{1}{2}\log_2\left((2\pi e)^d \det\Sigma\right).$$

G 是高斯向量，d 是維度，Σ 是正定共變異數，e 是自然對數底數。平均值不影響熵。

將另一個具有相同均值與共變異數的密度 f，與高斯密度 g 比較。高斯對數密度是常數加二次形式。因前兩階動差相同，它在 f 下的期望等於在 g 下的期望。因此

$$D_{\mathrm{KL}}(f\|g) = h(G) - h(X) \geq 0.$$

X 具有密度 f，等式假設需要的量有限。它證明最大熵性質，且等號對應高斯分布 [1,2]。

一維固定變異數 σ^2，最大熵為 $\frac{1}{2}\log_2(2\pi e\sigma^2)$。只指定支撐範圍，則有不同極值分布：該有界支撐上的均勻密度。限制決定最大熵分布。

這個證明不只對通訊有用。它讓模型選擇透明：從均值與共變異數採用高斯，在最大熵表示中沒有加入額外形狀限制。但它不會驗證擬合、排除多峰，也不讓高斯概似自動對離群值穩健。

程式：高斯資訊與可逆縮放

完整例子由合成高斯對估計共變異數、計算高斯模型資訊，並驗證可逆縮放改變熵，卻保留資訊公式。

```
import numpy as np

rng = np.random.default_rng(37)
rho = 0.7
covariance = np.array([[1, rho], [rho, 1]])
samples = rng.multivariate_normal([0, 0], covariance, size=100000)
estimated_cov = np.cov(samples, rowvar=False)
def gaussian_entropy(cov):
    sign, logdet = np.linalg.slogdet(np.atleast_2d(cov))
    assert sign > 0
    d = np.atleast_2d(cov).shape[0]
    return 0.5 * (d * np.log(2*np.pi*np.e) + logdet) / np.log(2)
def gaussian_information(cov):
    return (gaussian_entropy([[cov[0, 0]]]) +
            gaussian_entropy([[cov[1, 1]]]) - gaussian_entropy(cov))
estimated = gaussian_information(estimated_cov)
exact = -0.5 * np.log2(1-rho**2)
transform = np.diag([100, 0.2])
scaled_cov = transform @ estimated_cov @ transform.T
assert abs(gaussian_information(scaled_cov) - estimated) < 1e-12
assert abs(gaussian_entropy(scaled_cov) - gaussian_entropy(estimated_cov)
           - np.log2(abs(np.linalg.det(transform)))) < 1e-12
assert abs(estimated - exact) < 0.02
print("Estimated Gaussian-model MI / exact MI:", estimated, exact)
print("Entropy shift:", gaussian_entropy(scaled_cov) - gaussian_entropy(estimated_cov))
```

這是高斯參數計算，不是通用互資訊估計器。非高斯資料集代入共變異數，回答的是高斯模型問題，可能漏掉大量非線性相依。

推導實數高斯通道容量

訊號功率增加可區分輸出的散布；雜訊則讓它們模糊。精確高斯模型下，容量取決於兩者比例。

每次使用傳一個實數振幅：

$$Y = X + Z, \qquad Z \sim \mathcal{N}(0, N), \qquad Z \perp X, \qquad \mathbb{E}[X^2] \le P.$$

X 是輸入，Y 是輸出，Z 是獨立零均值高斯雜訊，$N > 0$ 是雜訊變異數，$P \ge 0$ 是平均輸入功率，$\perp$ 表示獨立。各次使用的雜訊獨立且同分布。

給定 X，只剩平移雜訊，所以 $h(Y \mid X) = h(Z)$。輸出變異數至多 $P + N$。高斯最大熵因此給

$$I(X;Y) \le \frac{1}{2}\log_2(2\pi e(P+N)) - \frac{1}{2}\log_2(2\pi eN) = \frac{1}{2}\log_2\left(1 + \frac{P}{N}\right).$$

P/N 是訊雜比。獨立高斯輸入 $X \sim \mathcal{N}(0, P)$ 讓輸出高斯，達到此單字母上界。相應編碼定理建立

$$C(P) = \frac{1}{2}\log_2\left(1 + \frac{P}{N}\right)$$

在標準平均功率模型下，每次實數通道使用的位元容量 [1,3]。最大化輸入均值為零，因確定性均值消耗功率，卻不增加可區分變動。

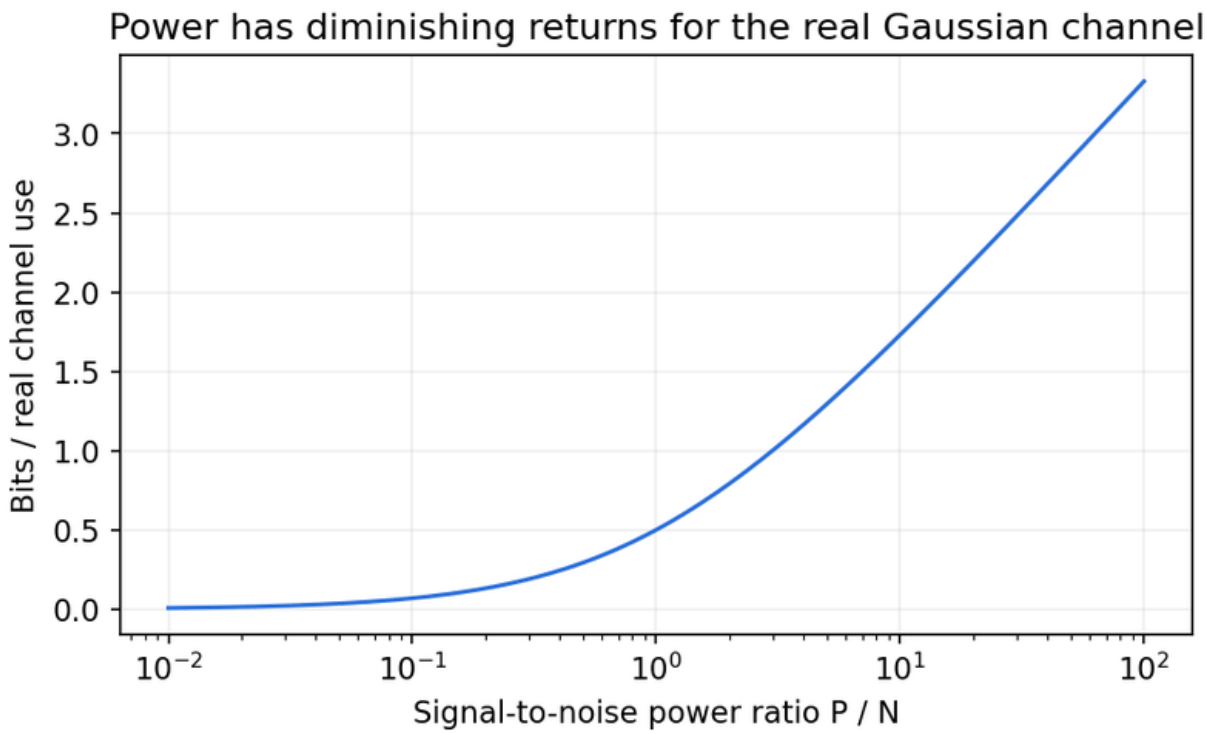


圖 *1* — 每次使用一個實數振幅、獨立高斯雜訊與平均功率限制。二分之一因子屬於此實數慣例。

平均功率限制不會替每個振幅設定嚴格最大值。加入峰值振幅限制，會改變最佳輸入分布，通常也改變容量。有限星座則定義更受限的訊號問題。

類似的複數公式常沒有二分之一，因一次複數使用含兩個實數維度，配合相應功率與雜訊慣例。未定義一次使用就比較公式，可能創造假的兩倍改善。

幾何觀點與限制

多次使用時，雜訊讓觀測形成發送碼字周圍的雲。好的碼字距離夠遠，讓雲通常可區分。

對 n 個獨立高斯雜訊樣本，平方範數集中於 nN 附近：

$$\frac{1}{n}\sum_{t=1}^{n} Z_t^2 \to N.$$

Z_t 是第 t 次雜訊，箭頭表示適當大數法則收斂。功率限制碼字的輸入半徑尺度約 $\sqrt{nP}$，輸出尺度則較大，涉及 $P+N$。

比較高維輸出體積與雜訊雲體積，暗示碼字數尺度為 $(1+P/N)^{n/2}$；正規化二進位對數就是容量式。這是幾何解釋，不是高斯雜訊支撐有界的宣稱。

雜訊有正機率離開任意有限球。可靠通訊要求非典型觀測足夠罕見，不要求所有可能結果的雜訊雲完全不重疊。這區分了誤差趨零容量與零錯誤通訊。

幾何也說明單次使用圖形為何可能誤導。長區塊允許很多座標合作，整體範數集中；短區塊星座則有自己的距離與解碼取捨。

平行通道與注水法

多個獨立通道雜訊不同時，把功率放在資訊邊際增加最多的地方。很吵的通道可能完全不分配功率。

對 m 個雜訊變異數 $N_i > 0$ 的平行實數高斯通道，在總功率 P_{total} 下分配 $P_i \ge 0$：

$$\max_{\sum_i P_i \le P_{\text{total}}} \sum_{i=1}^{m} \frac{1}{2}\log_2\left(1+\frac{P_i}{N_i}\right).$$

i 索引平行維度，m 是數量，目標是所有維度聯合使用一次的位元。獨立高斯輸入達到各分量速率。

作用中功率的導數為 $1/[2\ln 2(N_i+P_i)]$。共同預算下令導數相等，得到

$$P_i = (\mu - N_i)_+, \qquad \sum_i (\mu - N_i)_+ = P_{\text{total}}.$$

μ 是水位，$(u)_+ = \max(u,0)$。符號表示在雜訊地板上加水：作用中通道達到共同訊號加雜訊水位。

此結果來自凹最佳化的最佳性條件。雜訊地板高於水位的通道分配為零；若給正功率，邊際回報比作用中的低雜訊通道差。

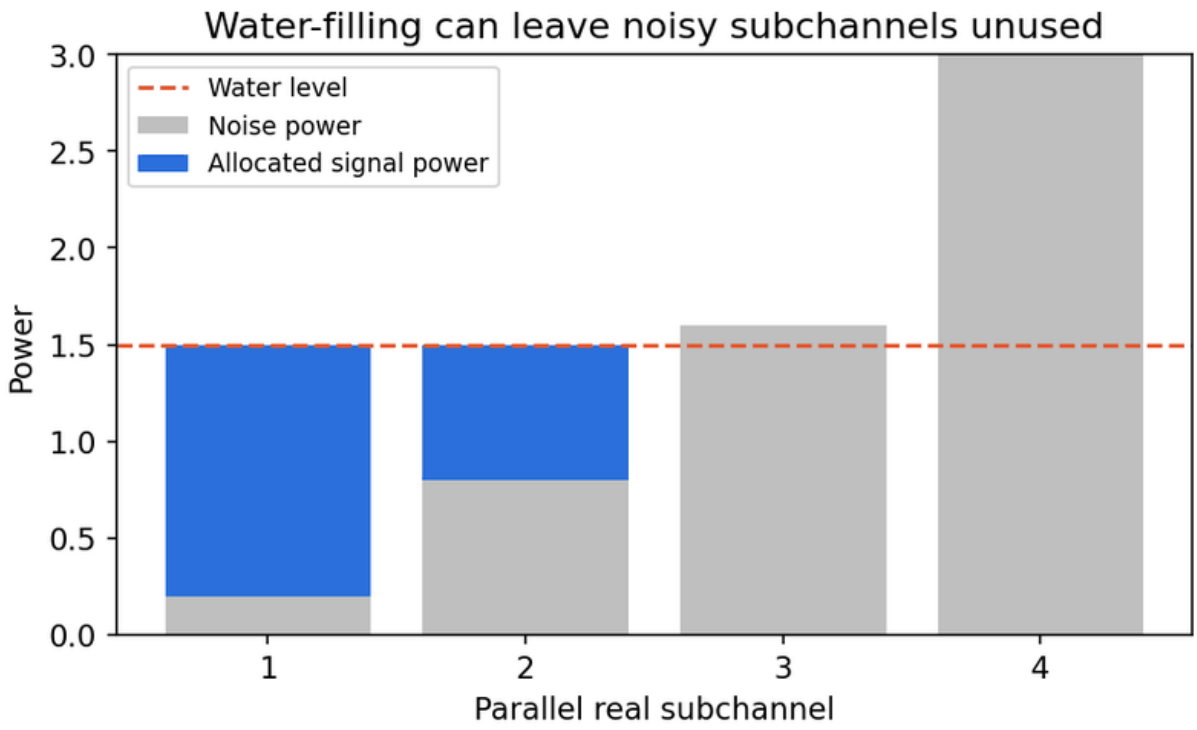


圖 *2* — 四個獨立實數通道的總訊號功率為 *2.0*。注水法是指定模型下的解析資源配置。

手算圖中的功率配置

雜訊地板為 0.2、0.8、1.6、3.0，總訊號功率預算二。先假設只啟用第一通道，總水位會是 $0.2 + 2 = 2.2$，高於第二地板，因此讓第二通道完全不用不可能最佳。

改假設啟用前兩通道，功率總和為 $(\mu - 0.2) + (\mu - 0.8) = 2$，得到 $\mu = 1.5$。它高於兩個作用中地板、低於下一地板 1.6，因此作用集合與假設一致。功率配置為 1.3、0.7、零、零。

聯合使用速率為 $\tfrac{1}{2}\log_2(7.5) + \tfrac{1}{2}\log_2(1.875) = 1.906891$ 位元。每通道平均分配 0.5，約得到 1.561252 位元。兩種設計都使用總功率二，改善來自配置位置，不是增加預算。

第三通道在水位超過 1.6 時啟用，先需要預算 $(1.6 - 0.2) + (1.6 - 0.8) = 2.2$。第四通道在水位超過三時啟用，閾值為 $(3 - 0.2) + (3 - 0.8) + (3 - 1.6) = 6.4$。這些閾值解釋容量與功率曲線中作用集合如何改變。

共同訊號加雜訊水位只適用作用中通道。地板高於水位者維持零訊號功率。這個互補條件說明：若強迫四通道全部有相同總水位，就會得到不可能的負功率。

程式：求水位並比較平均配置

完整例子以二分搜尋求水位，驗證預算與作用中最佳性條件，再比較平均功率速率。

```
import numpy as np

noise = np.array([0.2, 0.8, 1.6, 3.0])
budget = 2.0
low, high = noise.min(), noise.max() + budget
for _ in range(100):
    level = (low + high) / 2
    if np.maximum(level-noise, 0).sum() > budget:
        high = level
    else:
        low = level
level = (low + high) / 2
power = np.maximum(level-noise, 0)
equal = np.full(len(noise), budget / len(noise))
rate = lambda allocation: 0.5*np.log2(1+allocation/noise).sum()
assert abs(power.sum()-budget) < 1e-12
assert np.allclose(noise[power > 0]+power[power > 0], level)
assert rate(power) >= rate(equal)
print("Water level / power allocation:", level, power)
print("Water-filled / equal-allocation joint rate:", rate(power), rate(equal))
```

目標計算每個維度同時使用一次。除以維度數，會得到不同的每維平均；兩者都要先指定使用速率，才能轉成每秒位元。

有色高斯雜訊下，在相容線性輸入與功率假設中，共變異數特徵基底可以建立平行雜訊模式。有通道增益時，有效雜訊地板也依賴增益。零增益不能透過反轉變成有用通道。

向量通道連結幾何與共變異數

多個量測可以對多個輸入座標帶來重疊資訊。矩陣描述混合，共變異數描述雜訊雲與所選訊號散布。

實數線性通道為

$$Y = AX + Z, \qquad Z \sim \mathcal{N}(0, K_Z), \qquad X \sim \mathcal{N}(0, K_X), \qquad Z \perp X.$$

X 是 d 維輸入，Y 是 m 維輸出，A 是 m 乘 d 通道矩陣，K_X 是輸入共變異數，K_Z 是正定雜訊共變異數。輸入矩陣可半正定，允許不使用部分方向。

輸出共變異數為 $AK_XA^{\top} + K_Z$。相減高斯輸出與條件雜訊熵，得到

$$I(X;Y) = \frac{1}{2}\log_2 \frac{\det(AK_XA^{\top} + K_Z)}{\det K_Z}.$$

$A^{\top}$ 是矩陣轉置，行列式使用輸出維度共變異數。即使訊號低秩，正雜訊共變異數也讓分母與輸出共變異數非奇異。

平均總功率慣例下，$\operatorname{tr} K_X \le P$ 限制輸入座標變異數總和。在半正定 K_X 上最大化上式，就是相應高斯向量通道問題 [1]。跡是對角元素總和，P 是每次向量使用總功率。

有用方向同時依賴 A 與 K_Z。很大的原始座標變異數，可能在通道強烈衰減的方向；相關雜訊也可能讓某方向比邊際變異數暗示的更少資訊。相容轉換問題的對角化，揭露能使用相同邊際回報原則的平行模式。

這也說明應區分前處理與物理通道改變。可逆接收器座標轉換，同時改變訊號矩陣與雜訊共變異數，行列式比保持不變。投影會丟棄輸出方向，可能降低資訊；不可逆發送器映射則改變能產生哪些輸入訊號。

數值計算應用穩定分解求對數行列式，不要先算巨大行列式再取對數。行列式可能溢位，對數卻適中。解讀答案前，先檢查正定性與矩陣維度。

條件熵與確定性表示

表示在數值上可以連續，卻只含有限可能狀態。數學分布取決於如何生成，不取決於陣列用浮點數儲存。

微分熵定義假設相對連續座標體積有密度。浮點數儲存的離散標籤仍是離散；有限資料集經確定性映射得到的向量，即使座標為實數，經驗支撐仍有限。

高維特徵空間中的低維流形，相對全維體積具有奇異分布。擬合全維密度隱含加入雜訊或平滑模型。得到的熵屬於該模型，可能強烈依賴尺度。

同樣地，在確定性連續關係加入微小獨立雜訊，會讓密度計算可行，但雜訊大小成為資訊問題的一部分。當大小趨近零，互資訊可能趨無窮。未報告平滑慣例就給有限值，會隱藏此依賴。

比較神經表示時，這些觀察很有用。與離散任務標籤相關的資訊可保持有限，精確確定性連續輸入與表示的資訊卻可能奇異。壓縮或預測目標必須指定哪些變數、雜訊機制與量化器定義速率。

實務做法是先說明觀測模型，再估計資訊：離散格子、明確含雜訊通道、參數密度或固定解析度編碼程序。它們各有正當用途，也回答不同問題。

加入雜訊趨零，資訊為什麼可能發散？

令 $X \sim \mathcal{N}(0,1)$，表示為 $V = X + \varepsilon$，其中獨立雜訊 $\varepsilon \sim \mathcal{N}(0,\tau^2)$ 的變異數 τ^2 為正。這是訊號功率一的高斯通道，因此 $I(X;V) = \frac{1}{2}\log_2(1 + 1/\tau^2)$。$\tau^2 = 1$ 時資訊為半位元，$\tau^2 = 0.01$ 時約 3.329106 位元。雜訊變異數趨零，資訊無界增加。

雜訊恰好零時，$V = X$ 是精確連續副本，點對落在二維聯合空間的一條線上，沒有一般二維聯合密度。因此不能在該端點直接相減邊際與聯合微分熵。有限解析度分格則給出有限離散資訊問題。雜訊或解析度慣例，決定神經表示估計究竟回答哪個問題。

建模與估計：公式證明什麼

高斯公式證明的是高斯問題，不會把每個共變異數估計都變成可靠物理通訊極限。

純量容量推導使用獨立加性高斯雜訊、平均二階動差預算，以及無界實數輸入字母。裁切、量化、訊號相依雜訊或改變中的雜訊分布，可能違反一項以上假設。

有限輸出量化器的資訊，不能超過可能標籤數的對數。量化前算出的實數容量，是另一觀測模型的上界參考，不保證量化接收器能達到。

非高斯來源使用高斯熵近似，在必要條件下給動差上界，而非量測熵值。均值與共變異數吻合，不代表尾端或相依吻合。

維度相對觀測太高時，有限樣本共變異數可能奇異，對數行列式公式可能失效。正則化共變異數會建立修改後統計模型；正則化值影響資訊，應明確說明。

評估時應分開物理雜訊模型、座標慣例、估計參數與觀察到的解碼器表現。合成測試可以驗證代數與實作，假設則須由獨立收集資料驗證。

現代觀點

Butakov 等人的 NeurIPS 2024 論文 [4]，研究透過 normalizing-flow 轉換估計互資訊。它提供利用可逆密度模型協助估計的現代例子，但仍須評估估計器，不能把座標熵解讀成內在資訊。

當代語意通訊文獻的綜述觀點 [5] 區分接收器建模、生成式重建與學得聯合編碼。對連續通道，我們由此推論：應明確回報功率正規化、實數或複數使用慣例、接收器量化與通道失配。

重建影像很好看，仍可能對某個原始細節只有少量忠實資訊。任務目標可能刻意丟棄該細節；這是率失真選擇，不是另一種微分熵慣例。

開放問題包括高維可靠資訊估計、分布偏移下的概似模型評估、兩端一致處理有限精度，以及讓學得系統資源計算符合物理通道。可逆性與高斯假設簡化數學分析，卻不能代替這些檢查。

下一章問：容許部分重建誤差時，來源需要多少資訊？高斯容量處理如何運送資訊，高斯率失真則處理來源需求多少資訊。

重點整理

- 密度是每單位座標的機率，不是點機率。
- 微分熵可負，會隨座標縮放改變。
- 細量化加入與解析度相關的離散描述成本。
- 相對熵與互資訊在可逆座標轉換下保留。
- 高斯最大熵來自固定動差限制。
- 實數高斯容量含明確平均功率預算與二分之一。
- 注水法讓作用中平行通道邊際回報相等。
- 參數資訊公式與經驗解碼可靠性回答不同問題。

練習

1. 求寬度 0.01 的均勻變數微分熵，說明負值為何不允許負長度碼。

提示

用區間寬度對數，另外處理格子編碼。

2. 把單位均勻來源量化成 16、64、256 個等格，求精確離散熵並連到微分熵。

提示
每格等機率。

3. 驗證聯合高斯對分別非零縮放的互資訊不變性。若一變數取整成單一位元，會改變什麼？

提示
可逆尺度的行列式抵消，取整不可逆。

4. 比較訊雜比 0.1、10 時，功率倍增的容量增益。說明絕對速率增加與相對功率增加為何不同。

提示
相減倍增前後容量。

5. 增加注水預算直到第三、第四通道作用，從雜訊地板算出啟用閾值。

提示
把水位設為下一雜訊地板。

6. 說明只有兩輸出標籤的裁切接收器違反哪些假設，並給與訊號功率無關的資訊上界。

提示
輸出熵至多一位元。

參考文獻

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapters 8–9. 出版社書頁與目錄。
2. T. Weissman, “Continuous Random Variables,” Stanford EE276 Lecture 8 notes, 2024. 第一手講義。
3. T. Weissman, “AWGN Channel,” Stanford EE276 Lecture 9 notes, 2024. 第一手講義。
4. I. D. Butakov, A. D. Tolmachev, S. V. Malanchuk, A. M. Neopryatnaya and A. A. Frolov, “Mutual Information Estimation via Normalizing Flows,” *NeurIPS*, 2024. 出版論文。
5. L. X. Nguyen, A. D. Raha, P. S. Aung, D. Niyato, Z. Han and C. S. Hong, “A Contemporary Survey on Semantic Communications: Theory of Mind, Generative AI, and Deep Joint Source-Channel Coding,” *IEEE Communications Surveys & Tutorials*, 2025. 出版資訊與作者版本。

IT 06 ・率失真與有損表示

推導二元與高斯率失真極限，計算有限字母測試通道，並以明確失真評估量化與學得表示。

網站版本: https://blog.shyandram.dev/zh-tw/learn/information-theory/it-06-rate-distortion/

你將學到

- 稱重建可接受之前，先指定失真函數。
- 區分操作速率與資訊論測試通道。
- 推導 Bernoulli ／ Hamming 與高斯／平方誤差率失真函數。
- 解釋覆蓋可達性與逆定理的資訊計算。
- 以交替更新計算有限字母率失真點。
- 把實際量化與學習目標連到誠實的速率、品質量測。

先看全貌

無損編碼要求精確原始資料；有損編碼要求有用替代品。後者必須先給「有用」數學意義：哪些差異重要、成本多大，以及限制指平均誤差，還是罕見的大失敗。

> **用最簡單的話說**
> 你只能用幾個詞向朋友描述一幅畫，無法提到每個小點。若目標是認出動物，形狀比背景細點重要；若目標是精確重畫，省略同樣細節就是錯誤。

本章依 Cover 與 Thomas 第 10 章 [1]，以及 Stanford 的率失真講義 [2,3]。核心結果對假想重建法則最小化互資訊。最佳化定義極限，本身不建立有限檔案格式，也不保證神經解碼器行為。

這直接連到視覺：像素重建、機器可讀特徵與合理生成影像，保留不同性質。比較速率、失真以前，應明確說明目標與資源慣例。

先選失真，再選編碼器

誤差量指定接收器容忍哪些來源差異。沒有任何通用失真分數能涵蓋所有任務。

令 X 為來源符號，$\hat{X}$ 為可能屬於不同字母集合的重建。非負失真函數為 $d(x,\hat{x})$。長度 n 區塊的失真為

$$d_n(x^n,\hat{x}^n)=\frac{1}{n}\sum_{t=1}^{n} d(x_t,\hat{x}_t).$$

t 索引符號，n 是區塊長度，d_n 是每符號平均失真。常見限制是 $\mathbb{E}[d_n(X^n, \widehat{X}^n)] \le D$；$D$ 是允許平均值，期望依指定來源與編碼。

Hamming 失真在符號正確時零，其他時一。實數振幅的平方誤差為 $(x - \hat{x})^2$。前者計不符次數，後者更重視大數值偏差，回答不同問題。

平方誤差依賴單位。來源與重建乘以 a，失真乘以 a^2。率失真曲線應寫明來源正規化、維度平均與失真單位。

一般加性模型替每符號計成本再平均。結構化任務可能需要區塊準則，例如保留關係、罕見特徵或決策。仍可研究，但不會自動等同單字母問題。

平均失真不會限制每個重建的最壞誤差。小平均可能伴隨罕見大誤差。需要尾端機率、最大誤差或各群組保證時，應另列條件，不要從平均值推讀。

重建字母集合也是定義的一部分。限制為小碼本、整數像素或特定模型族，會縮小可行集合，無限制理論最佳值可能無法達到。反過來，允許輸出任務標籤而非影像，會改變字母集合與失真問題。公平比較應先說明選擇，再展示速率，讓讀者判斷系統是否保留同一物件。

操作與資訊論率失真

實際編碼發送有限描述之一；測試通道描述來源與重建可能的統計關係。定理連結它們的漸近極限。

區塊編碼器把 X^n 映到 M 個索引之一，解碼器把索引映到 $\widehat{X}^n$。固定速率為

$$R_n = \frac{1}{n} \log_2 M.$$

M 是可發送描述數，R_n 是每來源符號位元。操作函數 $R(D)$ 是指定失真下，漸近可達速率的下確界。

獨立無記憶來源與適當加性失真設定下，率失真定理給

$$R(D) = \min_{p(\hat{x}|x):\, \mathbb{E}[d(X,\widehat{X})] \le D} I(X; \widehat{X}).$$

來源法則 $p(x)$ 固定，$p(\hat{x} \mid x)$ 遍歷正規化重建法則。最小值符號適用最佳解存在的設定；更一般連續情況可能需要下確界與正則條件 [1,2]。

此重建法則常稱**測試通道（test channel）**。與通訊容量不同，它不最佳化來源輸入。容量在給定物理通道下最大化傳輸資訊，率失真在重建限制下最小化所需資訊。

隨機測試通道是描述所需聯合分布的數學物件。操作定理以區塊碼逼近資訊速率。逐符號獨立抽取含雜訊重建，本身不會說明接收器需要多少位元才能重現抽樣。

基本形狀與零速率重建

容許更多誤差，不會需要更多位元。某個失真以上，接收器可以固定輸出同一重建，完全不需訊息。

定義

$$D_0 = \min_{\hat{x}} \mathbb{E}[d(X, \hat{x})].$$

D_0 是最佳常數重建的平均失真，$\hat{x}$ 不依觀察來源選擇。$D \geq D_0$ 時，零速率可行。

標準設定下 $R(D)$ 非負、非遞增且凸。非遞增是因更大的失真容許值包含先前所有可行編碼。

凸性有分時解釋。長區塊的 α 比例使用第一種碼，其餘用第二種：

$$R(\alpha D_1 + (1-\alpha)D_2) \leq \alpha R(D_1) + (1-\alpha)R(D_2).$$

$\alpha \in [0,1]$ 是比例，D_1, D_2 是各失真，右側是平均速率。預定區塊邊界可以避免逐符號發送選擇。

零失真時，有限來源若只有完全相同才零成本，就回到一般熵極限。若數種不同來源值都同樣正確，零失真可能需要少於來源熵。真正連續高斯來源的平方誤差零失真速率則發散。

這些條件很重要。沒有指定失真零集合或字母集合，就說「零失真等於熵」，可能抹掉任務表示能壓縮的原因。

Hamming 誤差下的二元來源

公平二元來源要每符號一位元才能精確保留。容許受控的不符比例，會降低必要速率。

對 $X \sim \text{Bernoulli}(p)$、$0 < p \leq 1/2$ 與 Hamming 失真，

$$R(D) = \begin{cases} h_2(p) - h_2(D), & 0 \leq D \leq p, \\ 0, & D \geq p. \end{cases}$$

p 是一的機率，D 是不符機率，h_2 是二元熵。零速率重建永遠為零，因此錯誤占來源符號比例 p。

為求下界，令 $E = X \oplus \widehat{X}$ 是不符指示，$\oplus$ 表二元互斥或。則

$$I(X; \widehat{X}) = H(X) - H(E \mid \widehat{X}) \geq h_2(p) - h_2(D).$$

E 為一的機率至多 $D \leq p \leq 1/2$，二元熵在此範圍遞增。條件化不能增加離散熵。

等號以**反向**建構達到：選 $\widehat{X}$ 與獨立 $E \sim \text{Bernoulli}(D)$，再令 $X = \widehat{X} \oplus E$。重建機率滿足

$$\Pr(\widehat{X} = 1) = \frac{p - D}{1 - 2D}.$$

$D < p \leq 1/2$ 避開公平來源的奇異端點；端點由連續性或直接常數重建處理。建構符合指定來源法則，並讓不符與重建獨立。

公平來源在 $D = 1/2$ 前為 $1 - h_2(D)$。有偏來源若直接以前向機率 D 翻轉 X，一般不會達到相同最佳值。獨立條件的方向是推導一部分。

建立最佳二元聯合表

把機率逐項寫出，反向建構就更清楚。取來源機率 $p = 0.2$、允許不符 $D = 0.1$。若 $q = \Pr(\widehat{X} = 1)$，重建與不符獨立，則

$$p = q(1-D) + (1-q)D = D + q(1-2D).$$

p 是要求的來源一機率，q 是重建一機率，D 是獨立反向翻轉機率。解得 $q = (0.2-0.1)/(1-0.2) = 0.125$。重建機率乘獨立誤差機率，得到：

聯合機率	$\widehat{X}=0$	$\widehat{X}=1$	來源邊際
$X=0$	0.7875	0.0125	0.8
$X=1$	0.0875	0.1125	0.2
重建邊際	0.875	0.125	1.0

非對角項總和為 0.1，恰好符合失真限制。每個重建欄除以總和後，來源與重建不同的機率都是 0.1。因此 $H(X \mid \widehat{X}) = h_2(0.1)$，必要資訊為每來源符號 $h_2(0.2) - h_2(0.1) = 0.252933$ 位元。

等價的**前向**條件通道不對稱。給定 $X=0$，輸出一的機率是 $0.0125/0.8 = 0.015625$；給定 $X=1$，輸出零的機率是 $0.0875/0.2 = 0.4375$。罕見一的錯誤大、常見零的錯誤小，合計達到指定平均誤差，並符合最佳後驗不確定性。

相較之下，以 0.1 機率獨立前向翻轉來源，平均失真相同，但重建一的機率為 0.26。資訊變成 $h_2(0.26) - h_2(0.1) = 0.357751$ 位元，大於最佳值。「百分之十雜訊」不足以定義最佳測試通道；方向與獨立關係也重要。

```
import numpy as np

def entropy(p):
    p = np.asarray(p, dtype=float)
    positive = p > 0
    return -np.sum(p[positive] * np.log2(p[positive]))

source_one, D = 0.2, 0.1
q = (source_one-D)/(1-2*D)
joint = np.array([[(1-q)*(1-D), q*D],
                  [(1-q)*D, q*(1-D)]])
px, pr = joint.sum(axis=1), joint.sum(axis=0)
information = entropy(px)+entropy(pr)-entropy(joint)
assert np.allclose(px, [0.8, 0.2])
assert np.isclose(joint[0, 1]+joint[1, 0], D)
assert np.allclose([joint[1, 0]/pr[0], joint[0, 1]/pr[1]], D)
exact = entropy(px)-entropy([D, 1-D])
assert np.isclose(information, exact)
print("Backward joint table:", joint)
print("Optimal information:", information)
```

計算驗證的是統計最佳值，仍需要長區塊碼，才能把該資訊速率轉成傳送的重建索引。

平方誤差下的高斯來源

較分散的高斯來源，需要更多位元才能有相同絕對精度。要求更小誤差，速率以對數增加。

對 $X \sim \mathcal{N}(0, \sigma^2)$ 與 $d(x, \hat{x}) = (x - \hat{x})^2$，

$$R(D) = \max\left\{0, \frac{1}{2}\log_2 \frac{\sigma^2}{D}\right\}, \qquad D > 0.$$

σ^2 是來源變異數，D 是相同平方單位的均方誤差。$D \geq \sigma^2$ 時，零重建不用位元。$D \to 0$ 時，速率無界增加 [1,3]。

令 $E = X - \widehat{X}$ 是重建誤差。相容有限微分熵下，

$$\begin{aligned} I(X; \widehat{X}) &= h(X) - h(E \mid \widehat{X}) \\ &\geq h(X) - h(E) \geq \frac{1}{2}\log_2 \frac{\sigma^2}{D}. \end{aligned}$$

互資訊良好定義時，條件化降低微分熵；高斯最大熵以二階動差限制誤差熵。此式是非零速率範圍的下界。

資訊論最小值的可達建構使用獨立 $\widehat{X} \sim \mathcal{N}(0, \sigma^2 - D)$ 與 $E \sim \mathcal{N}(0, D)$，並令 $X = \widehat{X} + E$。再次是反向測試通道：誤差與重建獨立，不是與來源獨立。

等價失真速率關係為 $D(R) = \sigma^2 2^{-2R}$，描述漸近最佳區塊編碼。簡單純量量化器在其速率下，一般位於極限上方。

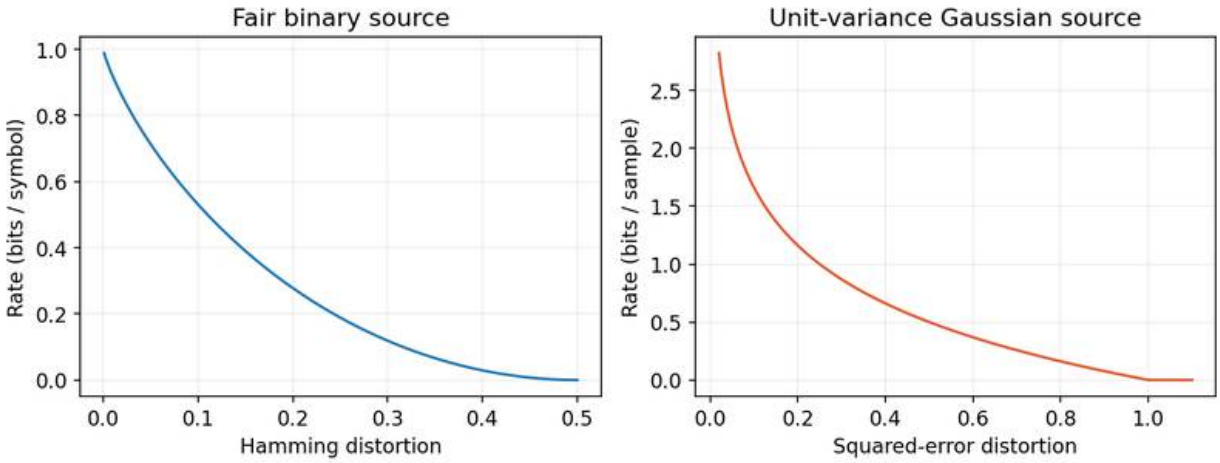


圖 *1* — 具有不同失真意義的解析例子。橫軸數值不能當成共同品質分數比較。

程式：測試高斯反向建構

完整合成實驗驗證來源變異數、誤差變異數與資訊公式。它沒有序列化區塊碼，只測最佳化聯合分布。

```
import numpy as np

rng = np.random.default_rng(38)
variance, distortion = 1.0, 0.2
```

```
reconstruction = rng.normal(0, np.sqrt(variance-distortion), size=100000)
error = rng.normal(0, np.sqrt(distortion), size=100000)
source = reconstruction + error
rho = np.corrcoef(source, reconstruction)[0, 1]
information = -0.5 * np.log2(1-rho**2)
exact = 0.5 * np.log2(variance/distortion)
print("Source variance / error MSE:", source.var(), np.mean(error**2))
print("Error-reconstruction correlation:", np.corrcoef(error, reconstruction)[0, 1])
print("Gaussian-model information / exact rate:", information, exact)
assert abs(source.var()-variance) < 0.02
assert abs(np.mean(error**2)-distortion) < 0.01
assert abs(information-exact) < 0.02
```

獨立性在取樣時就建立；有限相關只是檢查。任意非高斯變數的經驗相關很小，不能單獨證明獨立。

為何覆蓋不同於通道打包

有損來源碼需要足夠重建代表，在允許誤差內覆蓋可能來源區塊。通道碼則分隔傳輸代表，抵抗雜訊。

固定可行來源重建聯合分布，從重建邊際抽許多碼字。對典型來源區塊，隨機重建符合聯合關係的機率，指數尺度為 $2^{-nI(X;\widehat{X})}$。

代表數超過約 $2^{nI(X;\widehat{X})}$ 並帶小速率餘裕，就很可能找到匹配。編碼器選索引，解碼器回傳共享重建；聯合典型性把所選點對連到目標失真。

這是覆蓋論證。精確證明需要典型性定義與來源、失真假設；指數尺度解釋速率，不是有限碼表現公式。

逆定理使用傳輸索引 J：

$$nR_n \geq H(J) \geq I(X^n;\widehat{X}^n) \geq \sum_{t=1}^{n} R(D_t) \geq nR\left(\frac{1}{n}\sum_t D_t\right).$$

J 遍歷 M 個描述，$R_n = \log_2 M/n$，$D_t = \mathbb{E}[d(X_t,\widehat{X}_t)]$，來源無記憶。第一資訊界使用重建由索引決定；逐符號步驟用來源獨立與資訊論定義；最後用凸性。

編碼傳達的來源資訊，不能超過索引所含資訊。這說明解碼器再精細的先驗，也不能以零來源速率恢復任意遺失細節；它可以依指定失真選有用或可能的重建。

計算有限測試通道

離散字母下，交替更新重建邊際與來源條件重建法則。懲罰參數會沿率失真前緣移動。

最小化自然單位目標 $I_{\text{nats}}(X;\widehat{X}) + \beta\mathbb{E}[d]$，$\beta \geq 0$。給目前重建邊際 $q_{\hat{x}}$，

$$V(\hat{x}\mid x) = \frac{q_{\hat{x}}\exp[-\beta d(x,\hat{x})]}{\sum_z q_z \exp[-\beta d(x,z)]}, \qquad q_{\hat{x}}^{\text{new}} = \sum_x p_x V(\hat{x}\mid x).$$

p_x 是固定來源法則，V 是測試通道，z 索引重建，β 以自然單位衡量失真懲罰。位元速率將自然單位資訊除以 $\ln 2$。

教科書 [1] 討論 Blahut–Arimoto 交替計算。正初始邊際涵蓋全部候選重建符號。解出後應計算實際失真與互資訊，不要把 β 本身稱為失真。

指數測試通道更新從哪裡來？

交替規則來自受限制最小化，不是因為某公式看起來像機率。暫時固定正的重建邊際 q_j，對來源狀態 i 最小化列貢獻

$$\sum_j V_{ij}[\,\ln(V_{ij}/q_j) + \beta d_{ij}], \qquad \sum_j V_{ij} = 1.$$

V_{ij} 是給定來源 i 的重建 j 條件機率，q_j 是固定參照邊際，d_{ij} 是失真，β 是自然單位懲罰。正來源權重 p_i 乘此列，但不改變最小化的條件分布。

加入列正規化乘子 η_i，對 V_{ij} 微分，駐點方程為 $\ln(V_{ij}/q_j) + 1 + \beta d_{ij} + \eta_i = 0$。取指數使 V_{ij} 正比於 $q_j \exp(-\beta d_{ij})$，列正規化再提供更新式分母。

接著固定 V，其誘導邊際 $q_j^{\text{new}} = \sum_i p_i V_{ij}$ 在正規化 q 中最小化交叉熵部分。指定有限字母設定下，交替兩步降低懲罰目標。幾何也很清楚：β 大時各列集中於低失真重建，邊際因子則偏向母體常使用的重建。

公平二元來源與 Hamming 失真下，對稱性保持 q 均勻，非對角機率因此是 $D = 1/(1 + e^{\beta})$，速率為 $1 - h_2(D)$。程式檢查的正是此解析關係。它直接解釋懲罰，而不把懲罰值當成實際失真。

程式：對照精確二元曲線

實作檢查重建邊際收斂，以獨立解析公式驗證點。容許誤差是數值更新容許值，不是前章容量上下界間隙。

```
import numpy as np
from scipy.special import logsumexp

def entropy(p):
    p = np.asarray(p)
    return -np.sum(p[p > 0] * np.log2(p[p > 0]))
def rd_point(p, distortion_matrix, beta):
    p = np.asarray(p, dtype=float)
    d = np.asarray(distortion_matrix, dtype=float)
    q = np.full(d.shape[1], 1/d.shape[1])
    for _ in range(50000):
        scores = np.log(q)[None, :] - beta*d
        V = np.exp(scores-logsumexp(scores, axis=1, keepdims=True))
        updated = p @ V
        if np.max(abs(updated-q)) < 1e-13:
            q = updated
            break
```

```
        q = updated
    else:
        raise RuntimeError("Rate-distortion updates did not converge")
    joint = p[:, None]*V
    positive = joint > 0
    ratio = np.divide(V, q[None, :], out=np.ones_like(V), where=positive)
    rate = np.sum(joint[positive]*np.log2(ratio[positive]))
    distortion = np.sum(joint*d)
    return rate, distortion, V

d = np.array([[0, 1], [1, 0]])
for beta in [0.0, 1.0, 2.0, 4.0]:
    rate, D, V = rd_point([0.5, 0.5], d, beta)
    exact = 1-entropy([D, 1-D])
    assert abs(rate-exact) < 1e-10
    print("Penalty / D / rate / exact:", beta, D, rate, exact)
rate, D, V = rd_point([0.8, 0.2], d, 4.0)
exact = entropy([0.8, 0.2])-entropy([D, 1-D])
assert D < 0.2 and abs(rate-exact) < 1e-9
print("Biased-source D / rate / exact:", D, rate, exact)
```

任意失真矩陣的小邊際更新差，是有用收斂檢查，但不是嚴格全域數值憑證。應看目標改變、支撐與獨立特殊例子。很大懲罰可能造成機率下溢，需要更仔細實作。

純量量化是具體可達設計

把來源取整至有限解析度格點，再無損描述格點索引。取整造成失真，機率編碼降低索引速率。

步距 Δ 下使用

$$K = \operatorname{round}(X/\Delta), \qquad \widehat{X} = \Delta K, \qquad D_\Delta = \mathbb{E}[(X - \widehat{X})^2].$$

K 是離散索引，$\widehat{X}$ 是格點重建，D_Δ 是平方誤差失真。熵編碼索引的速率由索引分布決定，再加實際編碼成本。

密度夠平滑、步距夠小時，常用近似 $D_\Delta \approx \Delta^2/12$，把量化誤差視為格內近似均勻。粗步距、裁切附近或結構化集中數值，不一定符合。

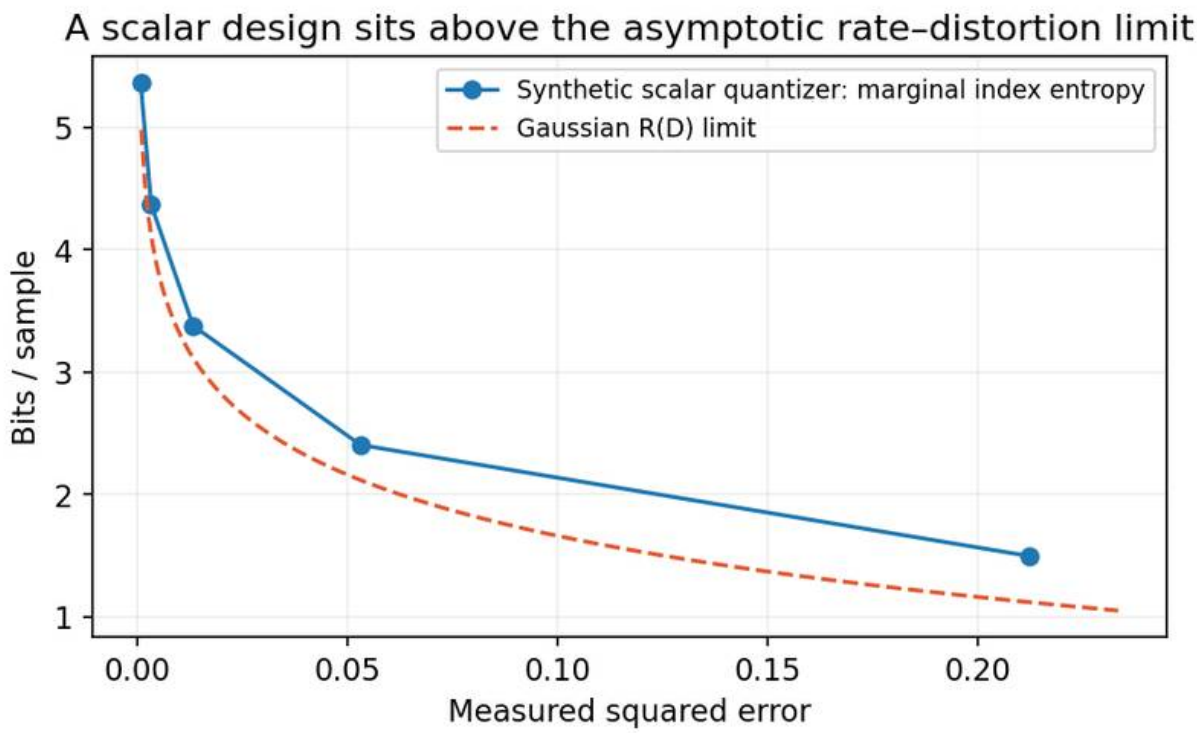


圖 *2* — *20,000* 個合成單位高斯樣本。索引熵是理想邊際編碼統計，沒有宣稱實際位元流長度。

```
import numpy as np

rng = np.random.default_rng(26)
source = rng.normal(size=20000)
for step in [0.1, 0.2, 0.4, 0.8, 1.6]:
    index = np.rint(source/step).astype(int)
    reconstruction = step*index
    _, counts = np.unique(index, return_counts=True)
    p = counts/counts.sum()
    rate_estimate = -np.sum(p*np.log2(p))
    mse = np.mean((source-reconstruction)**2)
    gaussian_limit = max(0, 0.5*np.log2(1/mse))
    print("Step / index entropy / MSE / reference rate:",
          step, rate_estimate, mse, gaussian_limit)
```

經驗分布來自同一批量化樣本，所以是描述統計。比較學得熵模型時，應另用訓練觀測擬合，在固定保留資料評分，並以明確慣例傳輸或共享參數。

索引熵不是實際壓縮檔大小。編碼器還須指定索引支撐、超界處理、重建步距、訊息長度與熵編碼終止。有限經驗估計可能在理論關係附近波動，不會推翻母體定理。

平行高斯來源使用反向注水

有限描述預算應更仔細保留高變異來源方向。低變異方向的整體變動若已在誤差容許值內，就可以捨棄。

對變異數 σ_i^2 的獨立高斯座標，總平方誤差預算為 D_{total}：

$$R(D_{\text{total}}) = \min_{\sum_i D_i \le D_{\text{total}}} \sum_i \max\left\{0, \frac{1}{2}\log_2 \frac{\sigma_i^2}{D_i}\right\}.$$

i 索引座標，D_i 是各自均方誤差，R 是完整來源向量的位元。正速率範圍中，微分目標使最佳作用中失真相等：

$$D_i = \min(\theta, \sigma_i^2), \qquad \sum_i \min(\theta, \sigma_i^2) = D_{\text{total}}.$$

θ 是反向水位，在預算小於總來源變異數時選擇。變異數低於水位的座標重建為均值，付出全部變異數但不付速率。

與通道注水的相似很有啟發，但角色不同。通道配置在雜訊地板上花訊號功率，最大化運送資訊；來源配置允許誤差到來源變異數，最小化所需描述資訊。

相關高斯向量的正交共變異數特徵基底，會產生獨立座標且保留總平方誤差，配置因此作用於特徵值。非線性任務失真或非高斯分布，不一定能簡化成相同計算。

這也說明低維重建的好處與限制。捨棄低變異座標，對總平方誤差便宜，但該座標仍可能決定特定標籤。任務相關與變異數是不同準則。

反向注水配置例

取三個獨立高斯座標，變異數為 4、1、0.25，每向量允許總平方誤差一。若每座標都給三分之一誤差，第三座標的預算會浪費：直接重建為零，已經只有 0.25 誤差。

其餘兩座標的最佳共同誤差水位滿足 $\theta + \theta + 0.25 = 1$，得到 $\theta = 0.375$。失真為 0.375、0.375、0.25，總和一。必要速率為

$$R = \frac{1}{2}\log_2 \frac{4}{0.375} + \frac{1}{2}\log_2 \frac{1}{0.375} + 0 \approx 2.415037.$$

R 是每個三座標來源向量的位元數。前兩座標需要正描述速率，第三重建為均值，速率零。此失真預算是每向量總和，不是每座標平均；將預算除以三，才是同一操作點的另一種慣例。

共同誤差水位來自對作用中座標速率微分：每增加一單位失真，速率降低的幅度為 $1/(2\ln 2\, D_i)$。讓邊際節省相等，就讓作用中 D_i 相等。當 D_i 到達來源變異數，已經沒有速率可省，該座標離開作用集合。

小變異座標仍可能編碼重要標籤。反向注水對此高斯平方誤差問題最佳，但標籤保留需要另一失真。這個例子把最佳化條件與接收者目標直接連起來。

比較操作點時，不要改變問題

公平率失真比較固定來源資料、品質準則與資源假設。否則看似更好的曲線，可能只是在解更容易問題。

經驗操作點是完整程序下的量測速率與失真。應說明速率為固定長度、平均變長、理想負對數機率，還是實際序列化位元。

本章理論曲線使用無記憶母體來源與漸近長碼。有限實驗有抽樣不確定性、有限區塊、受限模型族與實作成本。把全部叫同一間隙，會掩蓋原因。

不同編碼設定應使用同一保留觀測。量化器、機率模型與學得轉換使用訓練資料擬合；驗證資料選懲罰或操作點；最後測試集估計已選程序表現。

母體異質時，應回報有意義群組的失真。常見例子平均可能掩蓋某子群很差的重建。這不推翻平均失真定理，而是指出該平均未充分表達應用。

也應說明解碼隨機性。相同索引下，確定性與隨機生成解碼器可能有不同誤差分布。重建同一測試輸入多次可以揭露變動；看到原圖後才挑最佳隨機重建，加入真實接收器可能沒有的選擇步驟。

最後，共享旁資訊須可比。帶任務資料庫、預訓練先驗或傳送調適資訊的解碼器，與沒有這些資訊的程序不同。共享資源有用，但可用性與成本慣例應伴隨前緣宣稱。

表示、任務與學得速率懲罰

表示可以捨棄無關細節，保留任務。只有指定任務並在目標母體評估，這才有幫助。

學得編碼器可能產生量化潛在變數 Z，具有熵模型 q，解碼器回傳 $\widehat{X}$。常見訓練目標為

$$\mathcal{L} = \mathbb{E}[-\log_2 q(Z)] + \lambda \mathbb{E}[d(X, \widehat{X})].$$

第一項是位元單位理想編碼成本代理，$\lambda \geq 0$ 平衡指定失真，期望使用訓練分布。此目標不會自動等於母體率失真函數。

潛在字母集合、熵模型失配、有限編碼器、旁資料流與模型成本，影響實際速率。架構限制可達重建；最佳化可能找到局部或模型受限取捨，而非資訊論最小值。

若接收器需要標籤 Y，適當準則可評估從 Z 推得決策，而非重建全部 X。這是不同來源失真問題。保留某標籤的特徵，可能捨棄後續不相關任務需要的資訊。

感知準則可能獎勵合理紋理，即使紋理與來源不同；忠實重建準則可能懲罰同一改變。沒有通用正確目標，文字、圖形與評估都要保留區別。

多種失真可以一起回報，不需合成沒有解釋的單一分數。例如分開數值忠實度、明確任務誤差與分布真實性準則。

現代觀點

Yang、Mandt 與 Theis 的 2023 年專論 [4]，區分無損熵編碼、有損轉換設計、量化、感知目標與任務導向編碼。分類能定位系統哪些部分改變速率、哪些改變失真、哪些改變評估目標。

Huang 與 Wu 的 2024 年綜述 [5]，明確對照學得影像壓縮的人類與機器使用者。共同結論是：表示的用途依賴接收者；一個重建分數不能證明保留所有下游分析。

Hofer 與 Böhme 的 2024 年預印本 [6]，建立神經影像錯誤壓縮分類，鼓勵檢查哪些內容改變，而不假設平均數值誤差已完整描述重建行為。此處依已核對狀態引用為預印本。

由率失真理論，我們推論應評估**前緣**，而非單一有利操作點：固定測試母體上的量測位元流大小，對明確失真。來源正規化、模型共享假設與解碼資源預算應一致。

開放問題包括不可逆壓縮後的任務轉移、罕見細節失真量、分布偏移，以及生成先驗的公平資源計算。更長區塊或更精細解碼器可以改善特定取捨，卻可能增加延遲或依賴更強共享假設。

下一章把資訊延伸到編碼以外：散度描述統計證據，最大熵在限制下選分布，對數成長則連結預測與決策。

重點整理

- 失真定義有損重建可以改變什麼。
- 率失真固定來源，最佳化重建法則。
- 標準前緣非遞增且凸。
- 二元與高斯最佳解使用仔細定義的反向測試通道。
- 覆蓋與資訊計算連結測試通道和區塊碼。
- 交替更新需要明確對數與懲罰單位。
- 量化索引熵不同於實際序列化速率。
- 學習或感知目標，須依指定接收者與失真評估。

練習

1. 對機率 0.2 的 Bernoulli 來源，求零速率失真，以及失真 0、0.05、0.1 的速率。

提示

非零速率範圍用 $h_2(p) - h_2(D)$。

2. 建立 $p = 0.2$、$D = 0.1$ 的反向二元聯合分布，比較直接以前向機率 0.1 翻轉來源。

提示

讓不符與重建獨立。

3. 變異數 4 的高斯來源，求平方誤差 1、0.25 所需速率，並解釋改變座標單位後的結果。

提示

速率依賴變異數失真比。

4. 把有限測試通道程式延伸至三符號與非對稱失真矩陣，分開回報懲罰、實際速率與實際失真。

提示

檢查每列正規化，重建聯合分布。

5. 在純量量化加入裁切，分別比較中央與尾端誤差，不只回報平均。

提示

分開追蹤飽和例子。

6. 替小二元影像設計兩種失真：精確像素保留，以及保留是否存在任何前景像素。說明零失真速率為何不同。

提示

任務辨認影像等價類。

參考文獻

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapter 10. 出版社書頁與目錄。
2. T. Weissman, “Lossy Compression & Rate Distortion Theory,” Stanford EE276 Lecture 13 notes, 2024. 第一手講義。
3. T. Weissman, “Lossy Compression & Rate Distortion Theory Continued,” Stanford EE276 Lecture 14 notes, 2024. 第一手講義。
4. Y. Yang, S. Mandt and L. Theis, “An Introduction to Neural Data Compression,” *Foundations and Trends in Computer Graphics and Vision*, 15(2), 113–200, 2023. 出版專論與作者版本。
5. C.-H. Huang and J.-L. Wu, “Unveiling the Future of Human and Machine Coding: A Survey of End-to-End Learned Image Compression,” *Entropy*, 26(5), 357, 2024. 出版綜述。
6. N. Hofer and R. Böhme, “A Taxonomy of Miscompressions: Preparing Image Forensics for Neural Compression,” arXiv preprint, 2024. 第一手預印本。

IT 07 ・統計證據、最大熵與成長

透過明確統計模型，連結概似比、罕見經驗分布、Fisher 資訊、最大熵與對數成長。

網站版本: https://blog.shyandram.dev/zh-tw/learn/information-theory/it-07-statistics-decision-growth/

你將學到

- 把概似比解讀為證據，而非後驗機率。
- 以經驗型態解釋散度控制的罕見事件速率。
- 區分假說檢定誤差取捨與參數估計。
- 推導 Fisher 資訊與正則不偏估計下界。
- 從明確限制建立最大熵分布。
- 把對數最佳配置、通用混合連到編碼悔損。

先看全貌

資訊量不只用於儲存與通訊。它們也量化觀測如何區分競爭解釋、分布對參數變化多敏感，以及不良預測決策在重複試驗中多昂貴。

> **用最簡單的話說**
>
> 兩個袋子有不同顏色比例。看過許多珠子後，你可以問哪個袋子更能解釋它們。同樣機率也能幫你描述珠子字串、估計比例，或選重複遊戲策略。每個問題用觀測的方式不同。

書籍對照為 Cover 與 Thomas 第 11 章統計、第 12 章最大熵、第 6 章賭局與編碼，以及第 16 章投資組合理論 [1]。Stanford 型態方法講義 [2] 提供易讀的有限字母推導。

本章使用完全指定機率與報酬的數學決策遊戲，目的在揭露資訊恆等式，不從歷史資料推論實際回報。每節都區分來源法則、選定模型、觀測樣本與決策準則。

概似比比較解釋

某模型給樣本較大機率時，樣本便較支持該模型。重複獨立觀測會相加對數證據。

對支撐相容、完全指定的分布 P, Q，定義

$$\Lambda(x^n) = \log \frac{P(x^n)}{Q(x^n)} = \sum_{t=1}^{n} \log \frac{P(x_t)}{Q(x_t)}.$$

x^n 是長度 n 觀測樣本，最後等式假設各模型內的觀測獨立。log 可自然或底數二，但須一致。正值在概似比較中偏向 P。

P 下，期望二進位對數比為 $nD_{\mathrm{KL}}(P\|Q)$；Q 下為 $-nD_{\mathrm{KL}}(Q\|P)$。方向不同，是因期望使用不同法則。

概似比不是後驗機率。計算後驗勝算，需要乘先驗勝算；選擇行動，需要納入錯誤決策損失。零閾值適合某些對稱先驗與損失，不適合所有檢定。

簡單假說下，Neyman–Pearson 概似比原則描述指定偽警報限制的最強力檢定 [1]。「簡單」是分布固定，不是帶未知擬合參數的模型族。

若用同一樣本估計參數，就是另一套程序。模型選擇、干擾參數與閾值校準，都可能改變有限誤差。把擬合分類器分數當證據前，先用完全指定小模型理解。

檢定誤差形成取捨

更難宣布對立假說，可以減少偽警報，卻可能漏掉更多真對立情況。報告不應省略任一誤差。

設虛無分布 P_0、對立分布 P_1。拒絕區域 A 給

$$\alpha = P_0(X^n \in A), \qquad \beta = P_1(X^n \notin A).$$

α 是第一型或偽警報機率，β 是第二型或漏判對立機率，A 是宣布對立的集合。

Bernoulli 機率 $p_0 < p_1$ 時，對立相對虛無的對數概似比，是一的計數遞增函數。因此，計數閾值實作概似比檢定。

離散計數可能讓確定性閾值無法精確達到任意偽警報目標。邊界隨機化可以數學上達到中間操作點；部署規則應說明是否隨機化。

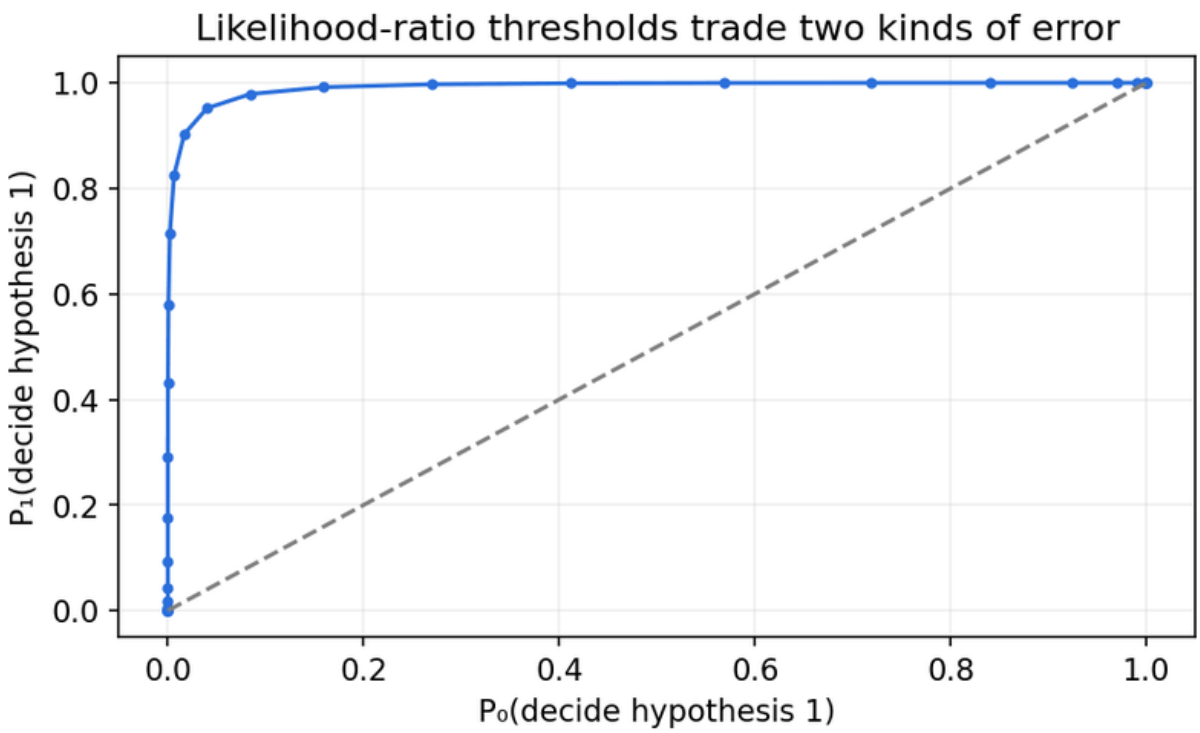


圖 *1* — 精確二項機率。偵測機率等於一減第二型誤差；每點對應明確計數閾值。

程式：精確誤差與 Monte Carlo 檢查

完整例子選擇首個符合 5% 偽警報限制的計數閾值，計算漏判機率，再分別驗證兩個法則。

```
import numpy as np
from scipy.stats import binom

n, p0, p1 = 30, 0.3, 0.6
thresholds = np.arange(n+2)
false_alarm = binom.sf(thresholds-1, n, p0)
detection = binom.sf(thresholds-1, n, p1)
threshold = thresholds[np.flatnonzero(false_alarm <= 0.05)[0]]
alpha = binom.sf(threshold-1, n, p0)
beta = binom.cdf(threshold-1, n, p1)
rng = np.random.default_rng(40)
observed_alpha = np.mean(rng.binomial(n, p0, 100000) >= threshold)
observed_beta = np.mean(rng.binomial(n, p1, 100000) < threshold)
print("Threshold / alpha / beta:", threshold, alpha, beta)
print("Simulation alpha / beta:", observed_alpha, observed_beta)
assert abs(observed_alpha-alpha) < 0.005
assert abs(observed_beta-beta) < 0.005
```

閾值使用已知分布，不在測試集搜尋。學習應用應在驗證資料調閾值，再以獨立測試觀測估計誤差。資料群組與時間相依可能改變獨立性假設。

評估集中對立比例，影響正確宣布頻率，卻不改變固定檢定與法則的兩種條件誤差。即使偽警報機率小，對立極罕見時，仍可能有很多錯誤宣布。解讀時需結合條件誤差與目標先驗盛行率。這是條件機率直接結果，不是概似比原則失敗。應說明回報量以虛無、對立，還是檢定輸出為條件。

計算例：證據不等於信心

例子的虛無機率 0.3、對立機率 0.6，長度 30 區塊含 k 個一時，對立相對虛無的對數概似比為

$$\Lambda_k = k \ln \frac{0.6}{0.3} + (30 - k) \ln \frac{0.4}{0.7}.$$

k 是觀測計數，Λ_k 使用自然對數，兩個分數比較競爭法則下的一與零機率。18 個一時，$\Lambda_{18} \approx 5.761260$，概似比約 317.748375，證據明顯偏向對立。

若對立的先驗機率為 0.01，先驗勝算為 $0.01/0.99$。後驗勝算約 $317.748375 \times 0.01/0.99 = 3.209580$，對應後驗機率約 0.762447。概似比不是 99.7% 信心值，先驗會改變後驗解讀。

更實際的區別，是閾值檢定的全部陽性宣布。程式選閾值 14，偽警報約 0.040053，漏判約 0.048112。如果只有 1% 案例來自對立，陽性真正屬對立的機率為

$$\Pr(H_1 \mid \text{positive}) = \frac{\pi(1-\beta)}{\pi(1-\beta) + (1-\pi)\alpha} \approx 0.193588.$$

$\pi = 0.01$ 是對立先驗盛行率，α 是偽警報機率，β 是漏判機率。符號 H_1 表示案例由對立產生的事件。即使兩種條件誤差都低於 5%，此母體多數陽性仍是偽警報。

結論來自計算兩族群預期產生的陽性數，沒有改變概似比最佳性或量測條件誤差。決策報告應說明條件事件，讓讀者區分個別樣本後驗與所有陽性的平均組成。

```
import numpy as np
from scipy.stats import binom

n, p0, p1, prior = 30, 0.3, 0.6, 0.01
count = 18
log_ratio = count*np.log(p1/p0)+(n-count)*np.log((1-p1)/(1-p0))
posterior = 1/(1+((1-prior)/prior)*np.exp(-log_ratio))
threshold = 14
alpha = binom.sf(threshold-1, n, p0)
beta = binom.cdf(threshold-1, n, p1)
positive_precision = prior*(1-beta)/(prior*(1-beta)+(1-prior)*alpha)
assert alpha <= 0.05 and binom.sf(threshold-2, n, p0) > 0.05
print("Likelihood ratio / sample posterior:", np.exp(log_ratio), posterior)
print("Probability alternative given positive:", positive_precision)
```

經驗型態揭露散度為何控制罕見事件

許多字串具有同一直方圖。數量以熵指數成長，單一來源機率以交叉熵指數縮小，差就是散度。

大小 k 的有限字母中，長度 n 序列的型態 $\hat{p}$ 是經驗符號分布。型態類 $T(\hat{p})$ 包含所有相同計數序列。獨立來源法則 q 下，

$$q(x^n) = 2^{-n[H(\hat{p})+D_{\mathrm{KL}}(\hat{p}\|q)]}.$$

x^n 具有型態 $\hat{p}$，$H(\hat{p})$ 是經驗熵，散度以位元計。將相同對數機率按計數分組，即得公式。

類別大小的保守多項式界為

$$(n+1)^{-k}2^{nH(\hat{p})} \le |T(\hat{p})| \le 2^{nH(\hat{p})}.$$

$|T|$ 是序列數，n,k 是區塊長度與字母大小。乘單一序列機率得到

$$(n+1)^{-k}2^{-nD_{\mathrm{KL}}(\hat{p}\|q)} \le q(T(\hat{p})) \le 2^{-nD_{\mathrm{KL}}(\hat{p}\|q)}.$$

$q(T)$ 是型態類總機率。有限長度多項式因子重要，但會從正規化對數指數中消失 [1,2]。

方向值得強調：第一引數是異常經驗型態，第二是實際生成來源。這不同於在固定真來源下平均的編碼失配計算。

經驗分布集合的主要指數行為，由允許型態中最小散度控制。Sanov 定理在有限字母與集合正則條件下，形式化此大偏差陳述 [1]。

程式：精確二元型態機率與指數

獨立例子比較二項機率與指數界，使用對數避免下溢，並檢查有限多項式間隙。

```
import numpy as np
from scipy.stats import binom

q, target = 0.3, 0.6
for n in [30, 100, 300, 1000]:
    count = int(round(n*target))
    empirical = count/n
    divergence = (empirical*np.log2(empirical/q) +
                  (1-empirical)*np.log2((1-empirical)/(1-q)))
    negative_log_probability = -binom.logpmf(count, n, q)/np.log(2)
    exponent = n*divergence
    assert exponent <= negative_log_probability + 1e-10
    assert negative_log_probability <= exponent + 2*np.log2(n+1) + 1e-10
    print("Length / observed normalized exponent / divergence:",
          n, negative_log_probability/n, divergence)
```

罕見精確型態，不同於單邊尾端或典型鄰域。尾端指數來自對事件全部計數最佳化，不是直接代入最極端計數。

從單一型態到型態集合

由經驗分布集合 $\mathcal{A}$ 定義的事件，至多有 $(n+1)^k$ 種型態。加總上界得到

$$q(\hat{p} \in \mathcal{A}) \le (n+1)^k 2^{-n \inf_{v \in \mathcal{A}_n} D_{\mathrm{KL}}(v \| q)}.$$

$\mathcal{A}_n$ 是 $\mathcal{A}$ 中長度 n 可實現型態，v 是候選型態，下確界選 q 下最不意外的型態。空事件機率零，另行處理。

罕見事件最可能部分，常在最接近來源的邊界，而非最戲劇化極端。二元來源機率 0.3，要求頻率至少 0.6 的尾端，主要由接近 0.6 而非一控制。

有限型態格點重要：所需邊界可能在可達分數間。精確二項總和處理離散性；漸近散度圖像解釋趨勢，同時保留取整與多項式因子。

這可幫助解讀表面分布偏移。異常直方圖可作為反對來源法則的證據，卻不能辨認原因。相依、選擇偏差、量測改變與不同來源，都可能改變頻率。

非對稱與對稱假說指數

不同方式分配兩種檢定誤差，會導向不同資訊極限。散度只是其中一種，不是所有分類問題的答案。

假設虛無 P、對立 Q 下觀測獨立，第一型誤差保留在固定非平凡容許值內。最佳第二型指數趨近

$$-\frac{1}{n} \ln \beta_n \to D_{\mathrm{nats}}(P \| Q).$$

β_n 是對立下誤接受虛無的機率，D_{nats} 使用自然對數。這是適當來源與支撐條件下的經典 Stein 漸近設定 [1]。

方向依接受區域：典型虛無觀測必須仍被接受，對立看起來像虛無的機率，具有 P 到 Q 散度指數。交換假說會反轉散度，也可能改變指數。

固定非零先驗與對稱 Bayesian 誤差目標下，指數改為 Chernoff 資訊：

$$C_{\mathrm{Ch}}(P, Q) = -\min_{0 \le s \le 1} \ln \sum_{x} P(x)^s Q(x)^{1-s}.$$

s 在兩法則間內插，x 遍歷有限字母。雖然加總項看似非對稱，整個量是對稱的。

指數描述大獨立樣本，不是短樣本精確誤差。前面的二項實驗直接計算有限情況。前置因子與決策邊界重要時，把漸近指數當精確機率可能不準。

複合假說、相依、改變中的母體與不等損失，都需要各自分析。資訊極限可解釋哪些來源差異有助區分，仍留下實際校準與抽樣設計問題。

推論中的充分性與資料處理

某摘要對參數充分，表示它在指定模型族中保留樣本的全部參數證據。

對模型族 $p_\theta(x^n)$，若給定統計量 $T(X^n)$ 的樣本條件法則不依賴 θ，就稱充分。分解準則為

$$p_\theta(x^n) = g_\theta(T(x^n))h(x^n).$$

θ 是參數，T 是統計量，g_θ 是依參數的因子，h 是與參數無關因子，須有相應模型族與支配假設 [1]。

獨立 Bernoulli 試驗的一計數，對未知機率充分；順序在此族不含額外參數證據。Markov 族則可能需要順序，單一一計數一般不再充分。

隨機參數 Θ 有指定先驗時，充分性給出 $I(\Theta; X^n) = I(\Theta; T(X^n))$ 所需條件關係。任意摘要只有資料處理上界。

「在模型族中」很重要。某假設模型的充分統計量，可能丟棄模型錯誤證據。平均值能估位置，卻隱藏多峰、相依或離群模式。

這連到學得表示：保留某目標資訊，不同於保留原資料全部診斷性質。充分性宣稱應伴隨任務與來源族。

Fisher 資訊衡量局部參數敏感度

Fisher 資訊衡量參數微移時，對數機率改變多強。它是模型局部性質，不同於隨機變數互資訊。

對純量參數 θ 與正則密度或質量函數 p_θ，定義分數與資訊：

$$s_\theta(X) = \frac{\partial}{\partial\theta} \ln p_\theta(X), \qquad \mathcal{J}(\theta) = \mathbb{E}_\theta[s_\theta(X)^2].$$

s_θ 是分數（score），$\mathcal{J}$ 是每觀測 Fisher 資訊，期望在參數化模型下。正則條件下，分數期望零。

n 個獨立觀測的資訊相加成 $n\mathcal{J}(\theta)$。正則不偏估計器 $\hat\theta$ 的 Cramér–Rao 不等式為

$$\operatorname{Var}_\theta(\hat\theta) \geq \frac{1}{n\mathcal{J}(\theta)}.$$

變異數描述固定參數的重複樣本。此界需要可微性、微分與積分交換，以及適當支撐、不偏假設 [1]。

Bernoulli 機率 $\theta \in (0,1)$ 的 $\mathcal{J}(\theta) = 1/[\theta(1-\theta)]$。樣本均值不偏，變異數 $\theta(1-\theta)/n$，達到該界。

資訊會隨參數座標改變。把機率重參數化為對數勝算，會以參數導數平方縮放資訊。統計族不變，只是參數誤差單位改變。

支撐依參數的非正則模型或有偏估計器，需要不同界。經驗變異數小，若均值偏離參數，也不能單獨證明準確。

推導 Bernoulli 資訊與變異數界

單一 Bernoulli 觀測 $x \in \{0,1\}$ 的對數概似為 $x\ln\theta + (1-x)\ln(1-\theta)$。微分得到

$$s_\theta(x) = \frac{x}{\theta} - \frac{1-x}{1-\theta} = \frac{x-\theta}{\theta(1-\theta)}.$$

$\theta \in (0,1)$ 是一的機率，s_θ 是分數。其兩種值為 $1/\theta$、$-1/(1-\theta)$，按機率 θ、$1-\theta$ 平均平方，得到 $\mathcal{J}(\theta) = 1/[\theta(1-\theta)]$。

為何倒數限制估計變異數？令 T 是依 n 個獨立觀測建立的正則不偏估計器，S_n 是分數總和。在正則假設下，對不偏性 $\mathbb{E}_\theta T = \theta$ 微分：

$$1 = \frac{d}{d\theta}\mathbb{E}_\theta T = \mathbb{E}_\theta[(T-\theta)S_n].$$

T 是觀測樣本的固定函數，$S_n = \sum_{i=1}^{n} s_\theta(X_i)$ 是總分數，且 $\mathbb{E}_\theta S_n = 0$。右側是估計器與分數的共變異數。共變異數平方不超過兩個變異數乘積；再利用 $\mathrm{Var}_\theta(S_n) = n\mathcal{J}(\theta)$，就得到變異數界。

$\theta = 0.3$、$n = 100$ 時，每觀測資訊約 4.761905，變異數下界 0.0021。樣本均值達到下界，因為中心化估計誤差與總分數成正比。獨立樣本數增加四倍，變異數除以四，標準差除以二。分數變異數能相加是因獨立；反覆測量同一相關試驗，不足以支持此縮放。

Fisher 資訊是局部散度曲率

適當二次可微正則條件下，鄰近參數分布滿足

$$D_{\mathrm{nats}}(p_\theta \| p_{\theta+\delta}) = \frac{1}{2}\mathcal{J}(\theta)\delta^2 + o(\delta^2).$$

δ 是小參數位移，$\mathcal{J}(\theta)$ 是自然對數分數資訊，$o(\delta^2)$ 的餘項除以 δ^2 趨零。底數二散度的首項要除以 $\ln 2$。

一階導數消失，因分數期望零；正則模型的二階導數給 Fisher 資訊。這連結局部估計敏感度與檢定、編碼使用的全域散度。

它是局部近似。大參數移動可能涉及不同曲率、支撐改變或多峰。把一個 Fisher 值當整個族的精確距離，會丟棄差異。

最大熵讓限制明確

若只知道少數期望，就從符合期望的分布選最大熵。答案依指定限制與允許支撐。

有限字母的特徵 $\phi_j(x)$ 要求 $\mathbb{E}[\phi_j(X)] = m_j$。帶正規化的最大熵給指數族形式

$$p_\lambda(x) = \frac{\exp(\sum_j \lambda_j \phi_j(x))}{Z(\lambda)}, \qquad Z(\lambda) = \sum_x \exp(\sum_j \lambda_j \phi_j(x)).$$

λ_j 是符合動差限制的乘子，Z 正規化質量函數。正號只是乘子符號慣例；存在性與內部支撐條件重要。

微分 $\ln Z$ 給受限制特徵期望。只知道均值、沒有支撐，不會決定熵最佳值。有界區間固定支撐無其他限制，給均勻密度；實數固定均值與變異數給高斯；非負支撐固定均值給指數密度 [1]。

最大熵是透明建模原則，不是來源真的屬於此族的證據。未量測相依與高階動差仍可能存在。應記錄限制、擬合乘子，再用獨立資料測預測。

指數形式也解釋與散度的關係。候選分布若符合最大熵模型的特徵期望，平均模型對數機率只依賴共享動差。同一對數底數下，候選到模型的相對熵，等於模型熵減候選熵。

這再次是限制把交叉熵項變成固定量。前章高斯最大熵證明就是連續動差例子。共同代數說明最大熵、指數族與資訊投影為何反覆一起出現。

推導與解讀最大熵分布

最佳化先使用自然對數，只會縮放熵與乘子。把正規化與特徵限制加到熵目標：

$$\mathcal{F}(p) = -\sum_x p_x \ln p_x + \alpha\left(\sum_x p_x - 1\right) + \sum_j \lambda_j \left(\sum_x p_x \phi_j(x) - m_j\right).$$

p_x 是候選機率，α 是正規化乘子，λ_j 是特徵乘子，$\phi_j(x)$ 是特徵 j，m_j 是要求期望。內部最佳點對各機率微分，得到 $-\ln p_x - 1 + \alpha + \sum_j \lambda_j \phi_j(x) = 0$；取指數再正規化，就得到前述指數形式。

狀態 0、1、2、平均 0.7 只有一個特徵 $\phi(x) = x$，相對權重為 $1, e^{\lambda}, e^{2\lambda}$。均勻分布平均一，較低目標平均因此需要負乘子。程式找到 $\lambda \approx -0.466121$，機率約 0.494780、0.310440、0.194780。

模型在符合平均的同時，將機率散布於全部允許狀態。點質量或兩狀態混合也可能符合某些動差，但此處熵較低。可行邊界上，部分機率可能為零，有限內部乘子不一定足夠。支撐、動差與存在條件是數學問題的一部分，不是選好喜歡密度後才補上的選擇。

程式：解有限最大熵模型並驗證 Fisher 變異數

完整例子求指定均值的三狀態分布，再比較 Bernoulli 估計器變異數與精確不偏界。

```
import numpy as np
from scipy.optimize import brentq
from scipy.special import softmax

states = np.array([0.0, 1.0, 2.0])
target_mean = 0.7
probability = lambda lam: softmax(lam*states)
lam = brentq(lambda value: probability(value) @ states-target_mean, -20, 20)
p = probability(lam)
assert abs(p @ states-target_mean) < 1e-12
print("Maximum-entropy multiplier / probabilities:", lam, p)

rng = np.random.default_rng(41)
theta, n = 0.3, 100
estimates = rng.binomial(n, theta, size=100000)/n
fisher = 1/(theta*(1-theta))
bound = 1/(n*fisher)
print("Empirical estimator variance / exact bound:", estimates.var(), bound)
assert abs(estimates.var()-bound) < 0.0001
```

最大熵計算精確使用列出的支撐；允許其他狀態會改變最佳化。變異數計算檢查合成正則例子，不是所有估計器的通用下界。

對數成長把失配轉成決策成本

重複乘法遊戲中，策略錯誤會累積。指定公平報酬模型下，最大期望對數成長使最佳配置符合結果機率。

結果 i 以機率 p_i 發生。配置比例 $q_i > 0$ 到其條件報酬，$\sum_i q_i = 1$。公平參考法則 $r_i > 0$ 在結果 i 發生時付乘數 $1/r_i$。期望二進位對數成長為

$$g(p, q; r) = \sum_i p_i \log_2 \frac{q_i}{r_i} = D_{\mathrm{KL}}(p\|r) - D_{\mathrm{KL}}(p\|q).$$

p 是結果法則，q 是配置，r 是報酬倒數機率向量，g 是每次試驗對數乘數。恆等式使用正規化法則與指定條件報酬遊戲 [1]。

最佳解在來源支撐為 $q = p$。失配策略精確損失 $D(p\|q)$ 的期望對數成長。兩結果皆乘數二時，r 公平，最佳值為 $1 - h_2(p)$。

這是預測與成長的 Kelly 連結，不表示每個投資組合都有公平條件報酬，或機率已知。加入現金、交易成本、槓桿限制或不同報酬，會改變最佳化。

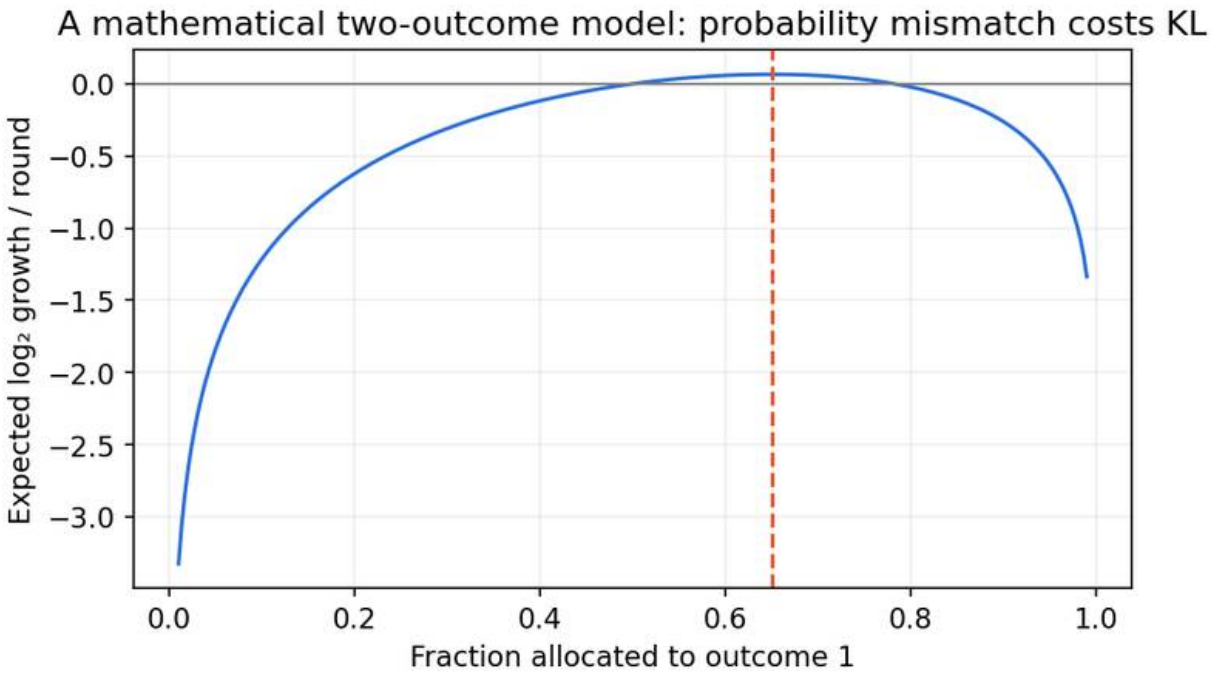


圖 2 — 解析雙結果報酬模型。相對最佳值的下降，就是機率模型散度。

每次結果前可觀察旁資訊 Y 時，條件最佳化配置。相對固定最佳配置的改善為

$$\mathbb{E}_Y[g(p_{X|Y}, p_{X|Y}; r)] - g(p_X, p_X; r) = I(X; Y).$$

X 是結果，Y 是旁資訊，參考報酬法則 r 不變。在此特定對數成長遊戲，互資訊有操作價值。

通用混合連結投資組合與編碼

不只選一個固定配置，而是混合許多固定策略的財富。過去結果決定各策略之後貢獻多少。

正報酬向量 R_t 與固定單純形配置 b 的累積財富為

$$W_n(b) = \prod_{t=1}^{n} b^\top R_t, \qquad W_n^{\mathrm{mix}} = \int W_n(b)\,\pi(db).$$

t 索引試驗，b 非負且總和一，R_t 含各分量相對報酬，π 是固定配置的先驗測度。混合有等價逐步配置，依過去財富加權策略。

Cover 與 Thomas 第 16 章在指定條件下，發展通用投資組合與長期比較最佳固定配置的想法 [1]。基準是事後固定配置，不是無限制、知道未來的交易者。

雙結果公平條件遊戲中，機率參數均勻先驗，給與 beta 混合相同的預測機制。t 次後，一的逐步機率為 $(N_1 + 1)/(t + 2)$。這是整數計數先驗，不同於無損編碼的半計數預測器。

程式：混合成長與最佳固定配置

完整例子以對數計財富，用 beta 積分求均勻混合。比較已知來源策略、回顧最佳固定策略與因果混合。

```
import numpy as np
from scipy.special import betaln

rng = np.random.default_rng(42)
p, n = 0.65, 2000
outcomes = rng.binomial(1, p, size=n)
ones, zeros = int(outcomes.sum()), n-int(outcomes.sum())
fitted = ones/n
oracle_log_wealth = n*np.log(2)+ones*np.log(p)+zeros*np.log(1-p)
best_log_wealth = n*np.log(2)+ones*np.log(fitted)+zeros*np.log(1-fitted)
mixture_log_wealth = n*np.log(2)+betaln(ones+1, zeros+1)
sequential_log_wealth, previous_ones = 0.0, 0
for t, outcome in enumerate(outcomes):
    q = (previous_ones+1)/(t+2)
    sequential_log_wealth += np.log(2*(q if outcome else 1-q))
    previous_ones += int(outcome)
assert abs(sequential_log_wealth-mixture_log_wealth) < 1e-9
assert mixture_log_wealth <= best_log_wealth
print("Source / best fixed / mixture growth bits per trial:",
      oracle_log_wealth/n/np.log(2), best_log_wealth/n/np.log(2),
      mixture_log_wealth/n/np.log(2))
print("Mixture regret to best fixed, bits:",
      (best_log_wealth-mixture_log_wealth)/np.log(2))
```

這是受控合成報酬實驗。最佳固定參數用整段序列，作為基準；混合則因果更新。對數財富減最佳基準，與負對數機率悔損有相同代數，但方向相反。

現代觀點

Hellström、Durisi、Guedj 與 Raginsky 的 2025 年出版專論 [3]，統一資訊論與 PAC-Bayesian 泛化界。分類區分期望、機率與條件互資訊表示，清楚指出統計保證需要指定隨機物件與評估準則。

El Gamal 的 2026 年擬議學習章 [4]，以教科書式說明連結散度恆等式與統計學習。它是草稿補充；本章把現代觀點搭配已出版第二版，不把擬議章當成完成新版。

我們推論，同一量只有指定模型後，才能具有不同操作意義。散度可以是編碼懲罰、異常型態指數、證據速率或對數成長損失。方向、對數底數與極限設定可能各異。

學習推論的開放問題包括大參數空間可靠資訊估計、相依或偏移資料保證，以及能對實際模型選擇程序提供資訊的界。資訊項無窮或難估時，形式不等式可能變得無用。

實際工作應搭配獨立有限實驗：精確小機率驗證實作、保留資料概似評估擬合、決策誤差評估用途。單一熵數值不能代替三者。

最後一章結合多來源、多接收者、較強不等式，並以小型分散式資訊專題連到現代學習。

重點整理

- 概似證據、後驗勝算與決策閾值不同。
- 檢定同時回報偽警報與漏判機率。
- 型態機率有散度指數，須明確引數方向。
- 假說指數依所選誤差設定。
- 充分性相對於模型族與目標參數。
- Fisher 資訊給局部正則估計極限。
- 最大熵依賴支撐與指定限制。
- 對數成長失配、通用混合悔損共享編碼恆等式。

練習

1. 反轉二項例子的虛無與對立，找 5% 偽警報限制閾值，比較漏判誤差。

提示

拒絕區域變低計數。

2. 按經驗計數分組符號，推導型態機率恆等式，說明散度引數順序。

提示

計數乘來源對數機率。

3. 比較 Bernoulli 機率 0.3、0.6 的雙向 Stein 散度，說明非對稱結果不違反對稱 Chernoff 指數。

提示

先固定接受區域，再選散度。

4. 將 Bernoulli 機率重參數化為對數勝算，推導 Fisher 資訊並解釋誤差單位。

提示

乘機率導數平方。

5. 求狀態 0、1、2、3、均值 1.2 的最大熵分布，再加入二階動差限制，描述額外乘子。

提示

第二特徵是狀態平方。

6. 公平條件遊戲中，來源機率 0.65，求配置 0.5、0.65、0.8 的期望成長，驗證散度損失，並說明哪種策略知道未來。

提示

沒有策略知道未來；事後基準只在觀察後擬合常數。

參考文獻

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapters 6, 11–12 and 16. 出版社書頁與目錄。
2. T. Weissman, “Method of Types,” Stanford EE276 Lecture 15 notes, 2024. 第一手講義。
3. F. Hellström, G. Durisi, B. Guedj and M. Raginsky, “Generalization Bounds: Perspectives from Information Theory and PAC-Bayes,” *Foundations and Trends in Machine Learning*, 18(1), 1–223, 2025. 出版社資訊，作者版本。
4. A. El Gamal, “Information Theory and Statistical Learning,” proposed textbook chapter, arXiv preprint, 2026. 草稿與出版狀態。

IT 08 ・網路、資訊不等式與學習

分析分散式來源與多使用者，連結進階不等式和學習，並執行有限分散式解碼專題。

網站版本: https://blog.shyandram.dev/zh-tw/learn/information-theory/it-08-networks-information-learning/

你將學到

- 描述分散式來源與多重存取通道的速率區域。
- 解釋廣播、中繼與一般網路為何需要不同論證。
- 使用 log-sum、Pinsker、熵功率與條件資訊不等式。
- 區分表示資訊與訓練樣本資訊。
- 執行小型分散式同位檢查解碼器，明確回報有限誤差。
- 把完整書籍章節地圖連到現代學習，不過度宣稱極限。

先看全貌

一個發送者與接收者會給單一容量，多個來源或接收者則給相容速率區域。提高某速率可能降低另一個；資訊也可能只有合在一起才有用，各分量單看卻沒有用。

> **用最簡單的話說**
>
> 兩位朋友各知道一條線索。單看任一線索可能毫無幫助，合起來卻解開謎題。若兩人同時說話，還得共享同一吵雜房間。我們必須追蹤哪些線索共享、誰聽到什麼，以及每人能說多少。

最後一章依 Cover 與 Thomas 第 15 章網路與第 17 章不等式 [1]。其他十五章已由前面內容與IT 01 閱讀地圖涵蓋。最後以有限合成實驗連結來源相依、編碼、解碼與任務資訊。

課程使用已出版第二版，並以當代綜述與明確標示的 2026 年擬議章補充。

分散式來源編碼：相關性節省速率

兩編碼器可以各自描述相關觀測，讓聯合解碼器利用關係，不必每個都獨立發送全部位元。

獨立點對 (X_t, Y_t) 依固定有限聯合法則。第一編碼器只看 X^n，第二只看 Y^n，共同解碼器重建兩區塊。Slepian–Wolf 漸近近乎無損區域為

$$R_X \geq H(X \mid Y), \qquad R_Y \geq H(Y \mid X), \qquad R_X + R_Y \geq H(X, Y).$$

R_X, R_Y 是每對來源樣本由各編碼器發送的位元，熵使用指定聯合來源。邊界不等式描述閉包；編碼以適當速率餘裕操作，使區塊誤差趨零 [1]。

個別界假設另一來源可以在解碼時幫忙；總和界計全部聯合不確定性。沒有假設編碼器看見另一方目前區塊；允許此觀測會定義不同問題。

公平 X、$Y = X \oplus E$、獨立 Bernoulli 雜訊機率 e 下，兩邊際熵都一，聯合熵 $1 + h_2(e)$。忽略相依的分別編碼，漸近需每對兩位元；聯合解碼可利用較小聯合不確定性。

一個角點把 Y 以完整邊際速率傳送，並把 X 壓到接近 $H(X \mid Y)$。解碼器已有重建 Y 作旁資訊，啟發後面的同位檢查症候實驗。

隨機分箱提供存在論證：把來源區塊放入箱，再用另一已解碼來源，找發送箱中可能匹配區塊。箱索引比完整來源索引短，相關性解開歧義。

這不保證每個有限點對精確重建。罕見或模糊區塊可能失敗。真正有限無損程序，若要求每個允許實現都恢復，需額外逃脫機制或其他處理。

隨機分箱為什麼有三個速率限制？

各編碼器把每個來源區塊隨機指派到箱，傳送箱索引。解碼器在兩個指定箱中尋找聯合典型點對。競爭點對可能有三種錯誤：只有 X 錯、只有 Y 錯，或兩者都錯。

競爭點對	典型候選數尺度	每候選碰撞機率
X 錯、Y 正確	$2^{nH(X\mid Y)}$	2^{-nR_X}
X 正確、Y 錯	$2^{nH(Y\mid X)}$	2^{-nR_Y}
兩者都錯	$2^{nH(X,Y)}$	$2^{-n(R_X+R_Y)}$

n 是配對區塊長度，R_X, R_Y 是傳送箱索引速率，熵指數依固定聯合來源。尺度省略小典型性容許誤差與有限前置因子。獨立箱指派給碰撞因子，聯合典型性則限制需要考慮的候選數。

候選數乘碰撞機率，再用聯集界限。適當餘裕下，三項全部趨零的條件，正是兩個別速率超過條件熵、總和超過聯合熵。真正點對未通過典型性測試的機率也趨零。平均所有指派後，便證明某固定共享分箱規則具有小區塊錯誤 [1]。

證明解釋相關性的好處與限制。解碼旁資訊把 X 候選數從完整來源目錄降成條件目錄，卻不消除區分剩餘候選的需求。總和限制則處理兩個別界限無法單獨排除的同時歧義。

公平位元、翻轉機率 0.1 時，條件熵約 0.468996，聯合熵每對 1.468996 位元。角點約 $(0.468996, 1)$、$(1, 0.468996)$，對稱點的各編碼器速率是 0.734498。各給 0.5 雖通過個別界限，卻違反總和界限。這些是極限邊界值，實際漸近構造會保留餘裕，不保證短區塊恰在邊界可行。

多重存取：獨立訊息共享輸出

兩發送者都可對同一接收器通訊。接收器觀測不只限制每人速率，也限制資訊總和。

有限無記憶通道 $W(y \mid x_1, x_2)$、獨立訊息與固定乘積輸入法則的區域包括

$$R_1 \le I(X_1; Y \mid X_2), \qquad R_2 \le I(X_2; Y \mid X_1), \qquad R_1 + R_2 \le I(X_1, X_2; Y).$$

X_1, X_2 是發送者輸入，Y 是共享輸出，R_1, R_2 是每次共同通道使用位元。對乘積輸入法則取聯集與分時，得到一般雙使用者多重存取容量區域 [1]。

個別界的條件化表示知道另一輸入時，接收器如何區分該使用者訊息；總和界處理聯合解碼。各自最大化個別式，不會取消共享總和限制。

分時引入共同預定變數 T，給定 T 時輸入為乘積。各界也條件於 T，把操作點凸化。共享排程不代表發送者取得任意依訊息的合作能力。

獨立實數高斯訊號、功率 P_1, P_2、加性雜訊變異數 N 下，

$$R_1 \le \tfrac{1}{2}\log_2(1 + P_1/N),$$
$$R_2 \le \tfrac{1}{2}\log_2(1 + P_2/N),$$
$$R_1 + R_2 \le \tfrac{1}{2}\log_2(1 + (P_1 + P_2)/N).$$

一次使用含兩個同時實數輸入，訊號有獨立訊息與標準平均功率限制。二分之一保留實數慣例。

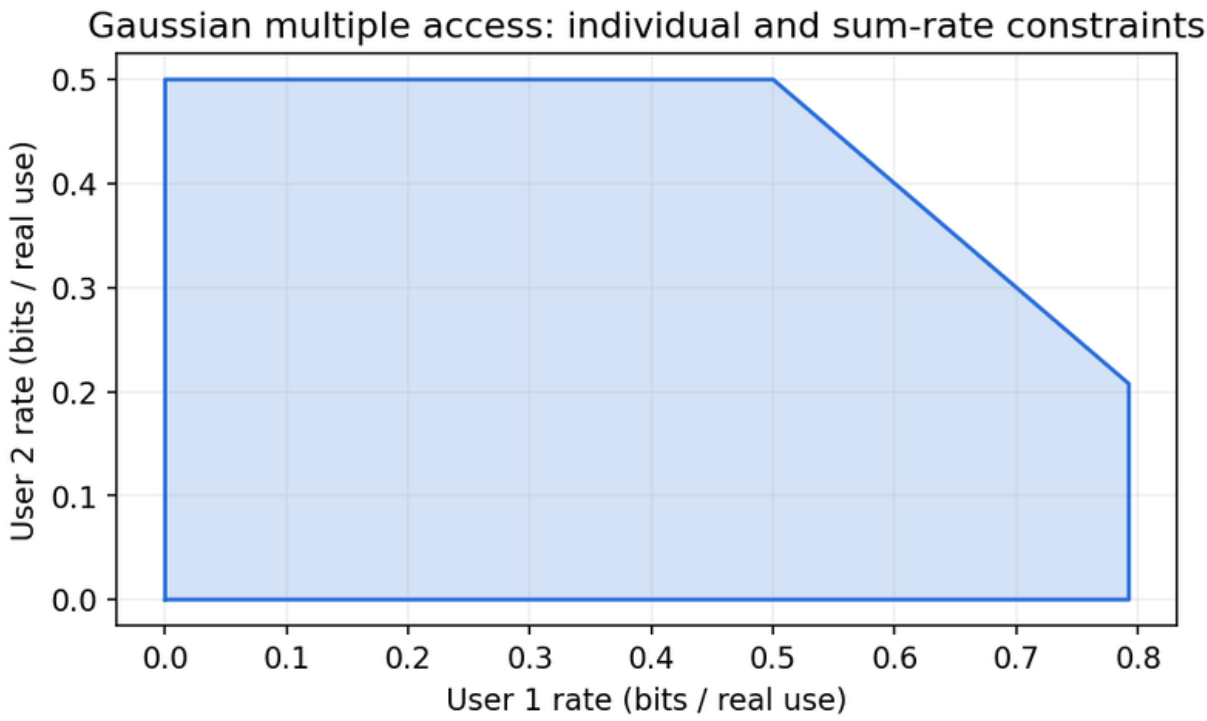


圖 *1* — P_1=2、P_2=1、N=1。總速率限制使兩人的個別最大值無法同時達到。

逐次解碼可達角點：先把另一訊息當雜訊，解一個訊息，再移除已解碼貢獻。順序改變誰取得較有利的條件速率。聯合解碼與分時描述更廣區域。

手算兩種高斯解碼順序

圖中功率 $P_1 = 2$、$P_2 = 1$，雜訊變異數 $N = 1$。使用者一的個別界限約每次 0.792481 位元，使用者二為 0.5，共享總和界限一。點 $(0.75, 0.4)$ 通過兩個別界，卻違反總和；$(0.6, 0.35)$ 則通過三者。

先解使用者一時，暫時把使用者二的獨立高斯訊號當成變異數一的額外雜訊，因此第一人的速率

限制為 $\frac{1}{2}\log_2(1+2/(1+1))=0.5$。成功解碼並相減後，第二人只剩原雜訊，可達 0.5，得到角點 $(0.5,0.5)$。

先解第二人時，其初始速率為 $\frac{1}{2}\log_2(1+1/(1+2))\approx 0.207519$。移除後，第一人可達 $\frac{1}{2}\log_2(1+2/1)\approx 0.792481$，得到角點 $(0.792481,0.207519)$。兩角點總和都是一，但解碼順序改變分配。

相減論證假設第一訊息解碼正確。第一個錯誤決策可能傳播至第二個，因此有限接收器必須評估聯合訊息錯誤，也評估個別錯誤。漸近角點可指引設計，不應掩蓋此實作問題。

廣播與中繼改變資訊模式

一發送者、兩接收者，不是兩發送者、一接收者的反向問題。先聽再傳的中繼，又是另一種模式。

物理退化廣播通道的 $X\to Y_1\to Y_2$，讓接收器一在指定統計意義較強。疊加建構使用 $U\to X\to Y_1\to Y_2$ 的輔助變數 U：

$$R_2\le I(U;Y_2),\qquad R_1\le I(X;Y_1\mid U).$$

R_2 是較弱接收器私人訊息速率，R_1 是較強接收器私人訊息速率，U 是兩者可解的層。對適當 $p(u,x)$ 取聯集，給退化通道容量區域 [1]。

Markov 順序是假設接收觀測的關係，不只是依樣本準確度排序。一般廣播通道不一定退化，不能繼承此精確區域。

無記憶中繼的來源輸入 X_s、中繼輸入 X_r、中繼觀測 Y_r、目的端觀測 Y_d，有 cut-set 上界

$$C_{\text{relay}}\le \max_{p(x_s,x_r)} \min\{I(X_s;Y_r,Y_d\mid X_r),I(X_s,X_r;Y_d)\}.$$

第一切割把來源與中繼加目的端分開；第二把來源加中繼與目的端分開。上界輸入聯合法則允許統計合作，作為外部鬆弛。

Cut-set 界不會自動可達。中繼可以解訊息、描述觀測或採其他因果策略，各有條件。許多一般網路容量仍未解。

重點是先畫實際資訊模式，再用公式：獨立訊息、相關來源、旁觀測、因果中繼與接收需求，都是不同假設。單一點對點容量不能概括全部。

聯合用途可能不見於邊際資訊

兩個各自沒有資訊的特徵，可能一起決定目標。一次只排名一特徵，可能把兩者都丟掉。

令 X_1,X_2 是獨立公平位元，$Y=X_1\oplus X_2$。則

$$I(X_1;Y)=I(X_2;Y)=0,\qquad I(X_1,X_2;Y)=1,\qquad I(X_1;Y\mid X_2)=1.$$

Y 是二元互斥或目標。任一邊際輸入仍讓目標公平，知道兩者就精確決定；給定另一輸入，第一輸入顯示完整相關性。

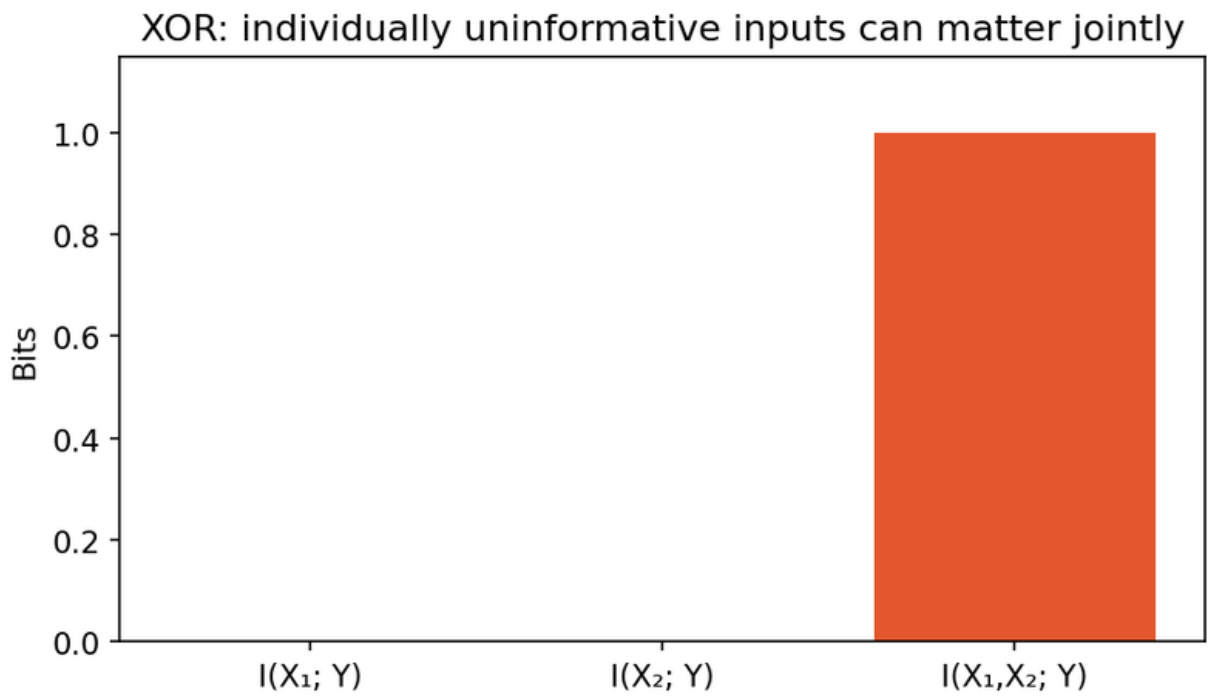


圖 2 — 精確合成聯合法則。示範聯合用途，沒有援引特定冗餘與協同分解。

「協同」可非正式描述只在合併後出現的資訊，但一般量化冗餘與協同分解，需要額外定義。三個互資訊值本身，不會定義任意變數的通用分解。

這對多模態學習重要。單看弱的模態可能在上下文有用；兩個強邊際特徵也可能大量重疊。應分析聯合法則與接收任務，不把邊際相關等同唯一貢獻。

計算函數是另一個分散式問題

假設各編碼器觀察獨立公平區塊 X_1^n, X_2^n，解碼器只需逐座標互斥或。輸出每對有一位元熵，很容易以為每對總共一位元就夠。

這忽略誰看輸入。額外把第二原始區塊給解碼器：若能從兩傳送索引求互斥或，就能恢復第一區塊。因輸入獨立，第二編碼器索引沒有提供額外第一區塊資訊。

因此，本例第一編碼器要使誤差趨零，就必須傳約每座標一位元；對稱地，第二也一樣。所需函數熵雖一，分散式總需求仍可能兩位元。

若中央編碼器看到兩輸入，就可直接用每對一位元傳互斥或。目標不變，觀測模式不同。這具體說明任務導向速率宣稱應附網路圖。

本例也區分兩種「聯合資訊」。接收者需要聯合計算，發送者卻沒有聯合觀測。不能只因某中央特徵算好後足夠，就假設學習系統已取得它。

實際應用可能有相關輸入、互動、共同隨機性或其他旁資訊，都會改變問題。未推導相應限制前，不能套用獨立非互動例子的速率結論。

較強不等式延伸工具

資訊不等式常比較合併結果、改變分布或加入獨立雜訊後還剩什麼。單位與假設決定意義。

非負 a_i、正 b_i 的 log-sum 不等式為

$$\sum_i a_i \ln \frac{a_i}{b_i} \geq A \ln \frac{A}{B}, \qquad A = \sum_i a_i, \quad B = \sum_i b_i.$$

i 索引元素，A, B 是總和，零項用標準連續慣例。全零 A 時兩側零。此式支援散度凸性與聚合資料處理形式 [1]。

在分割每格內使用，可證合併結果不增加相對熵。精細量測能區分的分布差異，粗摘要可能已看不到。

有限機率向量 p, q 的 Pinsker 不等式為

$$\mathrm{TV}(p, q) = \frac{1}{2} \sum_i |p_i - q_i| \leq \sqrt{\frac{1}{2} D_{\mathrm{nats}}(p \| q)}.$$

總變差限制事件機率差，散度使用自然對數。若以位元回報，根號內散度要乘 $\ln 2$。小散度可控制事件機率差，大散度則可能讓界無用。

離散熵也具次模性：

$$H(X_A) + H(X_B) \geq H(X_{A \cup B}) + H(X_{A \cap B}).$$

A, B 是變數索引集合，X_A 是相應變數群，空群熵零。差是非重疊部分給定交集的條件互資訊，因此非負。

這是子集熵關係，不表示每個任務相關函數都次模。XOR 提醒我們，另一特徵加入後，特徵資訊可以增加。

熵功率與高斯雜訊

微分熵能轉成類似變異數的量。獨立連續訊號相加後，熵功率不小於原本總和。

d 維向量有有限自然單位微分熵時，定義熵功率

$$\mathcal{N}(X) = \frac{1}{2\pi e} \exp\left(\frac{2h_{\mathrm{nats}}(X)}{d} \right).$$

d 是維度，h_{nats} 使用自然對數。純量高斯熵功率等於變異數；等向高斯向量則等於共同座標變異數。

相同維度獨立向量的熵功率不等式為

$$\mathcal{N}(X + Y) \geq \mathcal{N}(X) + \mathcal{N}(Y).$$

X, Y 須滿足相應密度與有限性假設。標準非退化設定下，獨立高斯向量且共變異數成比例時等號成立 [1]。

de Bruijn 恆等式連結雜訊平滑與 Fisher 資訊：

$$\frac{d}{dt} h_{\text{nats}}(X + \sqrt{t}\,G) = \frac{1}{2} J_{\text{loc}}(X + \sqrt{t}\,G).$$

G 是獨立標準高斯向量，$t > 0$ 是每座標雜訊變異數，$J_{\text{loc}}(V) = \mathbb{E}\|\nabla \ln f_V(V)\|^2$ 是位置 Fisher 資訊。需要平滑性與可積分性。

此 Fisher 是與位置估計相關的密度梯度量，不是前章任意參數 Fisher。明確定義可避免混淆同名物件。

加雜訊可以增加觀測熵，卻減少原始資訊，沒有矛盾：熵功率描述輸出散布，資料處理描述保留關係。現代含雜訊表示模型應區分兩者。

表示資訊與樣本資訊

特徵可保留目標，限制原始細節；訓練模型也可揭露對特定樣本依賴多少。這是不同資訊問題。

$Y \to X \to Z$ 的資訊瓶頸目標為

$$\min_{p(z|x)} I(X;Z) - \beta I(Z;Y).$$

X 是觀測，Y 是選定相關目標，Z 是可能隨機的表示，$\beta \geq 0$ 平衡壓縮與目標資訊。Goldfeld 與 Polyanskiy 的出版教學文 [2] 發展理論與學習連結。

目標選擇相對任務的統計表示，不證明每個神經網路自動符合相同最佳化，也不證明表示保留所有未來目標。

瓶頸如何移除干擾、保留標籤？

有限例子讓表示目標具體。令標籤 Y 與干擾位元 N 是獨立公平位元，觀測 $X = (Y, N)$ 包含兩者。比較完整表示 $Z = X$、只含標籤的 $Z = Y$，以及常數表示。

表示	$I(X;Z)$	$I(Z;Y)$	目標 $I(X;Z) - \beta I(Z;Y)$
兩個位元	2	1	$2 - \beta$
只含標籤	1	1	$1 - \beta$
常數	0	0	0

資訊單位都是位元，相關權重 β 無單位。只含標籤的表示保留此標籤全部需求，比完整表示少花一位元。$\beta < 1$ 時常數勝出，$\beta > 1$ 時保留標籤勝出；$\beta = 1$ 時標籤與常數平手。

結論不限於這三個例子。資訊鏈式法則給出

$$I(X;Z) - \beta I(Z;Y) = (1 - \beta) I(Y;Z) + I(N;Z \mid Y).$$

$X = (Y, N)$，N 是獨立干擾位元，Z 是任意由 X 產生的表示。條件項非負，因此 $\beta \leq 1$ 時全域下界零，由常數達到。$\beta > 1$ 時，利用 $I(Y;Z) \leq H(Y) = 1$ 得下界 $1 - \beta$，由 $Z = Y$ 達到。保留任務的最佳解來自鏈式法則與明確來源，不是「壓縮一定改善學習」的非正式主張。

若未來目標改成 $Y' = Y \oplus N$，舊的純標籤表示對此新公平目標互資訊零，完整觀測卻能決定它。不可逆任務壓縮即使對原相關準則最佳，也會限制未來任務轉移。

符號也區分本節兩種資訊問題：此處討論單一觀測與表示，後面的學習界限則討論模型 W 與全部訓練樣本 S。減少 Z 中干擾資訊，不直接證明 $I(W;S)$ 較小，也不直接建立數值泛化保證。

學習時，令 $S = (Z_1, \dots, Z_n)$ 是獨立訓練樣本，W 是學得模型。母體與經驗損失為

$$L(W) = \mathbb{E}_{Z\sim P}[\ell(W,Z)], \qquad \hat{L}_S(W) = \frac{1}{n}\sum_{i=1}^{n} \ell(W,Z_i).$$

P 是母體法則，新 Z 與訓練獨立，ℓ 是所選損失。相關資訊可為 $I(W;S)$，而非特徵資訊 $I(X;Z)$。

每個固定模型損失在 P 下都是 σ 次高斯時，標準期望界形式為

$$\left|\mathbb{E}[L(W) - \hat{L}_S(W)]\right| \le \sqrt{\frac{2\sigma^2 I_{\text{nats}}(W;S)}{n}}.$$

期望平均樣本與學習器隨機性，σ 是共同集中參數，資訊使用自然對數。$[0,1]$ 有界損失可用 $\sigma = 1/2$ [3]。

這是期望界，不自動成為某訓練網路的高機率憑證。確定性連續學習器可能有無窮樣本資訊，使界無用。條件資訊與其他精細框架處理不同隨機化、抽樣慣例 [3]。

程式：精確有限學習界

完整例子只依多數標籤學常數二元分類器。輸出是確定性位元，所以 $I(W;S) = H(W)$，全部期望都可對二項計數加總。

```
import numpy as np
from scipy.stats import binom

p = 0.65
for n in [5, 20, 100]:
    counts = np.arange(n+1)
    mass = binom.pmf(counts, n, p)
    prediction = counts >= n/2
    training_loss = np.where(prediction, (n-counts)/n, counts/n)
    population_loss = np.where(prediction, 1-p, p)
    gap = mass @ (population_loss-training_loss)
    probability_one = mass[prediction].sum()
    probabilities = np.array([probability_one, 1-probability_one])
    positive = probabilities > 0
    information_nats = -np.sum(probabilities[positive]*np.log(probabilities[positive]))
    bound = np.sqrt(information_nats/(2*n))
    assert abs(gap) <= bound + 1e-12
    print("Length / expected gap / information bound:", n, gap, bound)
```

計算納入模型與樣本的選擇相依。把所選分類器當成與訓練標籤獨立，會抹掉正在分析的泛化間隙。

專題：同位檢查分箱與旁資訊解碼

編碼器替二元區塊發送短同位檢查標籤。解碼器結合標籤與相關區塊，選最可能原始資料。

二元區塊 $x \in \{0,1\}^n$、具有 r 個獨立列的公開二元矩陣 H，傳送症候

$$s = Hx \pmod 2.$$

H 為 r 乘 n，s 含 r 個同位檢查位元，乘法用二元體加法與乘法。每個症候辨認一箱候選區塊。

解碼器取得 $y = x \oplus e$，獨立翻轉機率低於二分之一，再從症候相容候選選與 y Hamming 距離最小者。指定法則下，這是最大條件概似。

編碼器從不看 y，矩陣與區塊長度共享，解碼器取得 y 作旁資訊。負載速率為 r/n，不含傳送或重建旁資訊程序。

因此，實驗對應來源編碼角點，不是免費壓縮兩觀測。若第二區塊也須傳送，其長度應計入總速率。共享旁資訊是有來源的假設，不是不明解碼優勢。

程式：窮舉有限解碼與整區塊誤差

完整 CPU 例子採十位元區塊，讓窮舉可行。它在二元體檢查矩陣秩，不使用實數數值秩。

```
import numpy as np

rng = np.random.default_rng(43)
n, trials, flip = 10, 3000, 0.1
patterns = ((np.arange(2**n)[:, None] >> np.arange(n)) & 1).astype(np.uint8)
source_indices = rng.integers(2**n, size=trials)
source = patterns[source_indices]
side = source ^ rng.binomial(1, flip, size=source.shape).astype(np.uint8)

def binary_rank(matrix):
    work = matrix.copy()
    rank = 0
    for column in range(work.shape[1]):
        pivots = np.flatnonzero(work[rank:, column])
        if len(pivots) == 0:
            continue
        pivot = rank + pivots[0]
        work[[rank, pivot]] = work[[pivot, rank]]
        for row in range(work.shape[0]):
            if row != rank and work[row, column]:
                work[row] ^= work[rank]
        rank += 1
        if rank == work.shape[0]:
            break
    return rank

for parity_bits in [4, 6, 8]:
    while True:
        H = rng.integers(2, size=(parity_bits, n), dtype=np.uint8)
```

```
        if binary_rank(H) == parity_bits:
            break
    weights = 2**np.arange(parity_bits)
    syndromes = ((patterns @ H.T) % 2) @ weights
    bins = [np.flatnonzero(syndromes == s) for s in range(2**parity_bits)]
    assert all(len(group) == 2**(n-parity_bits) for group in bins)
    decoded = np.empty(trials, dtype=int)
    for t, (index, observation) in enumerate(zip(source_indices, side)):
        candidates = bins[syndromes[index]]
        distances = np.count_nonzero(patterns[candidates] != observation, axis=1)
        decoded[t] = candidates[np.argmin(distances)]
    block_error = np.mean(decoded != source_indices)
    print("Parity bits / payload rate / full-block error:",
          parity_bits, parity_bits/n, block_error)
conditional_entropy = -flip*np.log2(flip)-(1-flip)*np.log2(1-flip)
print("Asymptotic side-information reference:", conditional_entropy)
```

這是有限示範，不是達容量建構。不同隨機矩陣有不同誤差；增加同位位元，不保證每個獨立抽樣矩陣都單調改善。參考熵是長區塊極限，不是十位元精確預測。

平手依候選順序確定處理。雜訊可能把旁區塊移得更近錯誤但症候相容的來源。要求每次含雜訊試驗都往返完全一致，是錯誤測試：程序刻意回報有限訊息誤差。

真正無損檔案程序應提供逃脫或互動修正，明確計傳送成本。此處學習目標是看旁資訊如何縮小箱，以及有限歧義如何保留。

現代觀點

Hellström 等人的 2025 年出版專論 [3]，透過期望、機率與條件資訊表示，整理資訊論與 PAC-Bayesian 泛化。分類幫助區分平均選擇相依界，與單一部署預測器保證。

Nguyen 等人的 2025 年語意通訊綜述 [4]，區分接收者意圖建模、生成解碼與學得聯合編碼。網路觀點把接收者知識與任務目標放中心，應納入數學資訊模式。

El Gamal 的 2026 年擬議章 [5]，把教科書討論延伸至統計學習，草稿狀態保持明確。現代推論與生成模型可以用相同散度工具，卻另有計算與評估問題。

開放問題包括一般多使用者區域、短區塊分散式碼、真實相依與通道偏移、罕見內容任務準則，以及高維學習系統有用資訊界。新架構不會只因產生合理樣本就解決它們。

全課程推論是：解讀資訊宣稱前，先問五件事：涉及哪些隨機變數；誰觀察各變數；假設哪些來源與通道法則；需要什麼重建或決策；以及如何量測速率、誤差與資源成本。

資訊檢核可以把問題變成簡短實驗紀錄：

選擇	應記錄內容
來源	母體、分組與相依假設
觀測	各編碼器、接收器分別可用的變數
描述	字母集合、實際位元流、共享參數與旁資料流
通道	條件法則、記憶、功率與使用慣例

選擇	應記錄內容
目標	精確訊息、有損重建或指定任務
評估	固定測試集、誤差定義、速率與執行時間

即使沒有緊的定理，紀錄仍有用。它避免數字在訓練、分析與部署間悄悄變義，也讓負面結果提供資訊：不佳速率可能來自機率模型、重建限制或有限解碼器，而非同一底層極限。

課程後的閱讀與專題路徑

可以三次重讀完整書籍。先用熵、編碼、通道與失真例子鞏固第 1–5、7–10 章；再把第 6、11–14 章連到預測、證據、最大熵、通用碼與描述複雜度；最後讀第 15–17 章，把網路假設與不等式放在每個定理旁。

小專題可選明確聯合法則，或獨立收集資料集，配一個接收者目標。實作基準編碼器與解碼器，回報實際速率與決策誤差，再一次只改一元件。機率模型、量化器與通道碼回答不同實驗問題。

自迴歸視覺章 連結逐步機率分解與現代模型；表示章 發展任務特徵；語言模型章 連結條件預測與 token 序列。

資訊定義具體時，連結最強。Tokenization 改變字母與單位，學得潛在變數改變觀測，生成重建改變失真問題。比較數值資訊前，應先說明選擇。

重點整理

- 網路產生相容速率區域，不是通用純量速率。
- 分散式解碼利用相關性，不賦予未觀察輸入。
- 多重存取個別速率仍受共享總和限制。
- 廣播與中繼公式依特定資訊模式。
- 聯合任務資訊可能不見於個別特徵資訊。
- 進階不等式需要明確單位、支撐與獨立假設。
- 表示資訊與樣本相依泛化是不同物件。
- 有限解碼、實際速率與誠實誤差，完成理論圖像。

練習

1. 公平 X、$Y = X \oplus E$、翻轉機率 0.2 下，求 Slepian–Wolf 角點與聯合熵，說明各編碼器觀察什麼。

   提示

   條件熵用 $h_2(0.2)$。

2. 求圖參數的兩個高斯多重存取角點，驗證總和限制並說明逐次解碼順序。

提示

一人達個別最大值，另一填總和界。

3. 說明一般廣播通道為何不能繼承物理退化區域，指出缺少的條件關係。

提示

檢查 $X \to Y_1 \to Y_2$。

4. 對 $(0.6, 0.4)$、$(0.5, 0.5)$ 驗證 Pinsker 界，一致使用自然與二進位散度。

提示

以 $\ln 2$ 把位元轉 nats。

5. 把同位碼區塊長度改 12，比較數種同位位元數，保留窮舉成本，回報區塊誤差而不只位元誤差。

提示

候選表有 2^n 列。

6. XOR 目標下，比較只保留 X_1 與保留點對，再說明確定性連續表示的原輸入資訊為何可無窮，而離散任務資訊仍有界。

提示

區分目標熵與精確連續關係。

參考文獻

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapters 15 and 17. 出版社書頁與目錄。
2. Z. Goldfeld and Y. Polyanskiy, “The Information Bottleneck Problem and Its Applications in Machine Learning,” *IEEE Journal on Selected Areas in Information Theory*, 1(1), 2020, DOI: 10.1109/JSAIT.2020.2991561. 機構最終手稿紀錄。
3. F. Hellström, G. Durisi, B. Guedj and M. Raginsky, “Generalization Bounds: Perspectives from Information Theory and PAC-Bayes,” *Foundations and Trends in Machine Learning*, 18(1), 1–223, 2025. 出版社資訊，作者版本。
4. L. X. Nguyen, A. D. Raha, P. S. Aung, D. Niyato, Z. Han and C. S. Hong, “A Contemporary Survey on Semantic Communications: Theory of Mind, Generative AI, and Deep Joint Source-Channel Coding,” *IEEE Communications Surveys & Tutorials*, 2025. 出版資訊與作者版本。
5. A. El Gamal, “Information Theory and Statistical Learning,” proposed textbook chapter, arXiv preprint, 2026. 草稿與出版狀態。

篇 IX

部落格文章

介紹 Pixels → Perception

透過淺白說明、程式碼與互動實驗，學習影像處理、機器學習及其背後觀念的中英雙語教學網站。

網站版本: https://blog.shyandram.dev/zh-tw/blog/introducing-pixels-perception/

一張影像，起點是一個由數字組成的陣列。要理解這些數字代表什麼，就會碰到訊號、模式、學習與不確定性等問題。為什麼一個小小的運算，就能改變影像的樣貌？模型如何學會辨認物件？在相信它的輸出之前，又該檢查哪些事情？

想和大家介紹 Pixels → Perception，一個提供英文與繁體中文內容的教學網站，試著把這些問題串成有脈絡的學習路徑。網站涵蓋影像處理、圖形識別、機器學習、深度學習、電腦視覺、自然語言處理、最佳化與資訊理論，希望讓讀者能在直覺說明、背後的數學，以及可以親自觀察的實驗之間來回對照。

這個網站想處理的一個落差，是「看懂公式」與「理解公式用起來會發生什麼事」之間的距離。定義在紙上可能很清楚，但當你改變參數、觀察圖表，或遇到一個失敗的例子時，才更容易看見它的假設。教學的安排，就是希望把這些關係說清楚。

學習區分成八條主題路線：

- **基礎**：介紹網站的使用方式、建議學習路徑，以及表示與操作影像的基本觀念。
- **影像處理**：把影像視為訊號，從強度與空間運算，逐步延伸到復原、壓縮與分割。
- **自然語言處理**：探討如何表示語言、建立處理文字的系統，並連結語言學觀念與學習方法。
- **圖形識別**：從特徵、統計決策與分類出發，理解辨認模式時涉及的不確定性。
- **機器學習**：涵蓋數學基礎、神經網路、表徵學習、生成模型與評估。
- **視覺深度學習**：把相關觀念放進影像分類、物件偵測、復原、增強與追蹤等具體任務。
- **最佳化**：練習如何建立問題、表達限制條件，並選擇求解方式。
- **資訊理論**：研究不確定性、壓縮與通訊，從另一個角度理解其他路線中反覆出現的觀念。

這些主題其實有許多共通之處。分析視覺模型時使用的數學工具，也能幫助我們理解語言模型；關於影像品質的問題，可能需要回到成像過程的假設；訓練上的疑問，也可能在釐清最佳化目標之後變得明確。把這些路線放在同一個網站裡，是希望讀者既能掌握各主題的架構，也能沿著彼此的連結繼續探索。

教學採用一致的閱讀節奏：先說明學習目標，用淺白的方式建立全貌，再逐步介紹定義、公式與例子。程式碼與圖表提供可以實際檢查的對象；後面的段落則連結研究文獻、討論限制，並以重點整理與練習收尾。當某個說明預設了你還不熟悉的知識時，可以透過先備知識連結回頭補足。

例如，互動實驗裡有一個影像濾波示範。你可以修改小型權重矩陣中的數值，直接觀察輸出如何改變。這讓學習形成一個循環：先預測、調整權重、觀察結果，再回到文字說明。其他實驗也涵蓋不同主題路線中的觀念。操作與計算都在瀏覽器中進行，不必先設定程式執行環境，就能開始嘗試。

程式碼範例則讓讀者進一步查看運算本身。你可以修改範例、比較輸出，或透過練習確認自己的理解。部分教學也附有可執行的筆記本連結。這些小型範例著重於說明運作方式，並不代表已重

現完整研究系統的規模或效能。

英文與繁體中文版本希望提供相同的學習體驗。繁體中文內容採用臺灣常用的術語，頁首的語言切換則連結到對應頁面。當你習慣用其中一種語言理解概念，卻又需要熟悉另一種語言的專業用語時，就能交互對照。

網站也在有系統的教學之外，保留了部落格。課程提供依序學習一個主題的路徑，部落格則容納介紹、筆記與課程順序以外的討論。如果只是想找回某個觀念，也可以使用搜尋，不必從整條主題路線的開頭重新讀起。

關於網站的製作方式，也想清楚說明：許多內容初稿與程式碼是在 AI 協助下建立的。主題與課程架構由人決定，人工審閱仍是必要的工作。**目前的內容仍在審閱中**。說明讀起來流暢，仍可能有錯誤；範例能夠執行，也可能省略重要假設。當細節會影響你的理解或實作時，請搭配文中引用的原始資料查證，並把小型示範當作繼續探究的起點。

如果剛開始接觸這些主題，可以從入門指南與影像就是陣列開始。如果已經有想釐清的問題，可以直接瀏覽主題路線，或先打開互動實驗，再沿著連結閱讀相關教學。你可以循序讀下去、回頭補一段數學，也可以只花一點時間弄懂一個觀念。

邀請大家挑一個問題，先猜猜結果，再動手試一個例子。希望 Pixels → Perception 能幫助你把紙上看似抽象的概念，變成自己能夠說明、驗證與運用的知識。

歡迎造訪 Pixels → Perception 繁體中文版或英文版。

歡迎來到 Pixels → Perception

這個網站涵蓋哪些內容、教學如何安排，以及如何執行程式碼。

網站版本: https://blog.shyandram.dev/zh-tw/blog/welcome/

這是一個可以依序閱讀、也可以隨時跳著看的電腦視覺課程。我們從「影像就是一堆數字」開始，一路介紹到現代的深度網路。

內容安排

學習區分成八個單元：基礎、影像處理、自然語言處理、圖形識別、機器學習、電腦視覺深度學習、最佳化與資訊理論。每篇教學都會列出先備知識，讓你知道該先讀哪一篇。

執行程式碼

部分教學提供對應的 Jupyter 筆記本，其他教學則包含可直接執行的程式碼區塊。若有筆記本，點選頁面上方的**在 Colab 開啟**，就能直接在瀏覽器中執行。

這篇文章所在的部落格，則用來放課程以外的內容：論文筆記、工具評測與較長的文章。